学Java架构·来咕泡教育
50000+ 技术人
都在咕泡教育拿到职场结果

50W字

程序员面试宝典

PROGRAMMER INTERVIEW

大厂高频Java面试题汇总

- 分19个部分，涵盖常见的Java面试题
- 超过400个经典面试题分析，详解Java的方方面面
- 包含专栏面试题、大厂面试题、小厂后端、老师精选题解析等
- 先分析问题，再给出参考答案，逻辑更清晰
- 大量技巧和注意点，帮助读者快速提高技术水平

咕泡科技

咕泡教育

目录

前言 17
MySQL 专栏面试题（1） 18
1.1 MySQL 索引使用有哪些注意事项呢？ 18
1.2. MySQL遇到过死锁问题吗，你是如何解决的？ 19
1.3. 日常工作中你是怎么优化 SQL 的？ 21
1.4. 说说分库与分表的设计 23
1.5. InnoDB 与MyISAM 的区别 25
1.6. 数据库索引的原理，为什么要用 B+树，为什么不用二叉树？ 35
1.7. 聚集索引与非聚集索引的区别 41
1.8. limit 1000000 加载很慢的话，你是怎么解决的呢？ 43
1.9. 如何选择合适的分布式主键方案呢？ 46
1.10. 事务的隔离级别有哪些？MySQL 的默认隔离级别是什么？ 46
1.11. 在高并发情况下，如何做到安全的修改同一行数据？ 55
1.12. 数据库的乐观锁和悲观锁。 55
1.13. SQL 优化的一般步骤是什么，怎么看执行计划（explain）， 如何理解其中各个字段的含义。 57
1.14. select for update 有什么含义，会锁表还是锁行还是其他 58
1.15. MySQL 事务的四大特性以及实现原理 60
1.16. 如果某个表有近千万数据，CRUD比较慢，如何优化。 63
1.17. 如何写 sql 能够有效的使用到复合索引 64
1.18. mysql 中in 和exists 的区别 64
1.19. 数据库自增主键可能遇到什么问题。 66
1.20. MVCC熟悉吗，它的底层原理？ 66
1.21. 数据库中间件了解过吗，sharding，jdbc，mycat？ 67
1.22. MYSQL 的主从延迟，你怎么解决？ 67
1.23. 说一下大表查询的优化方案 69
1.24. 什么是数据库连接池?为什么需要数据库连接池呢? 69
1.25. 一条 SQL 语句在 MySQL 中如何执行的？ 70
1.26. InnoDB引擎中的索引策略，了解过吗？ 72
1.27. 一条 sql 执行过长的时间，你如何优化，从哪些方面入手？ 72
1.28. MYSQL数据库服务器性能分析的方法命令有哪些? 73
1.29. Blob 和text 有什么区别？ 73
1.30. mysql里记录货币用什么字段类型比较好？ 73
1.31. Mysql中有哪几种锁，列举一下？ 73
1.32. Hash索引和B+树区别是什么？你在设计索引是怎么抉择的？ 74
1.33. mysql的内连接、左连接、右连接有什么区别？ 74
1.34. 说说MySQL的基础架构图 75
1.35. 什么是内连接、外连接、交叉连接、笛卡尔积呢？ 75
1.36. 说一下数据库的三大范式 77
1.37. Mysql 的binlog 有几种录入格式？分别有什么区别？ 77
1.38. InnoDB引擎的 4 大特性，了解过吗 77
1.39. 索引有哪些优缺点？索引有哪几种类型？ 78
1.40. 创建索引有什么原则呢？ 80

1.41. 创建索引的三种方式 81
1.42. 百万级别或以上的数据，你是如何删除的？ 81
1.43. 什么是最左前缀原则？什么是最左匹配原则？ 81
1.44. B 树和 B+树的区别，数据库为什么使用B+树而不是B树？ 82
1.45. 覆盖索引、回表等这些，了解过吗？ 95
1.46. 非聚簇索引一定会回表查询吗？ 96
1.47. 联合索引是什么？为什么需要注意联合索引中的顺序？ 96
1.48. 数据库隔离级别与锁的关系 97
1.49. 聊聊 InnoDb 的锁，什么是死锁？怎么解决？ 97
1.50. count(1)、count(*) 与 count(列名) 的区别？ 111
1.51. 什么是存储过程？有哪些优缺点？ 112
1.52. SQL 约束有哪几种呢？ 113
1.53. varchar(50)中 50 的涵义 114
1.54. mysql 中int(20)和 char(20)以及 varchar(20)的 区别 114
1.55. drop、delete 与truncate 的区别 114
1.56. UNION 与UNION ALL 的区别？ 114
1.57. SQL 的生命周期？ 116
1.58. 一条 Sql 的执行顺序？ 116
1.59. 关心过业务系统里面的 sql 耗时吗？统计过慢查询吗？对慢查询都怎么优化过？ 117
1.60. 主键使用自增 ID 还是 UUID，为什么？mysql 自增主键用完了怎么办？ 117
1.61. MySQL 数据库 cpu 飙升的话，要怎么处理呢？ 118
1.62. 读写分离常见方案？ 123
1.63. MySQL的复制原理以及流程 123
1.64. Innodb 的事务实现原理？ 124
1.65. 谈谈 MySQL 的 Explain 125
1.66. Innodb 的事务与日志的实现方式 125
1.67. 你们数据库是否支持 emoji 表情存储，如果不支持，如何操作？ 126
更换字符集 utf8-->utf8mb4 126
1.68. 一个 6 亿的表 a，一个 3 亿的表 b，通过外间 tid 关联， 你如何最快地查询出满足条件的第 50000 到第 50200 中的这 200 条数据记录。 126
1.69. Mysql 一条 SQL 加锁分析 127
1.70. Mysql如何解决幻读问题 127
Redis 专栏面试题（2） 130
2.1. 什么是 Redis？它主要用来什么的？ 130
2.2. 说说 Redis 的基本数据结构类型 131
2.3. Redis 为什么这么快？ 136
2.4. 什么是缓存击穿、缓存穿透、缓存雪崩？ 154
2.5. 什么是热 Key 问题，如何解决热 key 问题 163
2.6. Redis 过期策略和内存淘汰策略 165
2.7 说说 Redis 的常用应用场景 168
2.8 Redis 持久化有哪几种方式，怎么选？ 172
2.9 怎么实现 Redis 的高可用？(Redis 主从、哨兵、集群) 176
2.10 使用过 Redis 分布式锁嘛？有哪些注意点呢？ 206
2.11 使用过 Redisson 嘛？说说它✂原理 212

2.12 什么✂ Redlock 算法 213
2.13 Redis ✂跳跃表 215
2.14 MySQL 与Redis 如何保证双写一致性 217
2.15 为什么 Redis 6.0 之后改多线程呢? 221
2.16 聊聊 Redis 事务机制 222
2.17 Redis ✂ Hash 冲突怎么办 223
2.18 在生成 RDB 期间，Redis 可以同时处理写请求么? 226
2.19 Redis 底层，使用✂什么协议? 226
2.20 布隆过滤器 226
2.21 Redis存在线程安全问题吗? 为什么? 229
2.22 请描述Redis的缓存淘汰策略 231
2.23 Redis多线程模型怎么理解，那它会有线程安全问题吗? 232
面试点评 233
2.24 Redis主从复制的原理 233
2.25 Redis中AOF 重写的过程描述一下 235
还需要保证数据的一致 238
2.26 Redis哨兵机制和集群有什么区别? 238
2.27 Redis的理解 242
消息队列专栏面试题（3） 245
3.1 什么是消息队列 245
3.2 消息队列有哪些使用场景 247
3.3 消息队列如何解决消息丢失问题? 252
3.4 消息队列如何保证消息的顺序性。 254
3.5 消息队列有可能发生重复消费，如何避免，如何做到幂等? 256
3.6 什么是幂等? 如何解决幂等性问题? 264
3.7 如何处理消息队列✂消息积压问题 266
3.8 消息队列技术选型，Kafka 还✂ RocketMQ，还✂RabbitMQ 268
3.9 消息中间件如何做到高可用 270
3.10 如何保证数据一致性，事务消息如何实现 271
3.11 让你写一个消息队列，该如何进行架构设计? 278
3.12 什么叫做阻塞队列的有界和无界 281
3.13 谈谈你对RabbitMQ工作原理的理解 283
3.14 Rabbit的消息可靠传输如何保证 287
3.15 RabbitMQ如何实现高可用 288
3.16 RabbitMQ的消息如何实现路由? 295
计算机网路专栏面试题（4） 302
4.1 说说 HTTP 常用的状态码及其含义? 302
4.2 HTTP 常用的请求方式，区别和用途? 303
4.3 请简单说一下你了解的端口及对应的服务? 304
4.4 说下计算机网络体系结构 305
4.5 如何理解 HTTP 协议✂无状态✂ 307
4.6 从浏览器地址栏输入 url 到显示主页✂过程 309
4.7 说下 HTTP 1.0，1.1，2.0 ✂区别 311
4.8 POST 和 GET 有哪些区别? 313
4.9 在交互过程中如果数据传送完了，还不想断开连⑩怎么办， 怎么维持? 313

4.10 HTTP 如何实现长连⑩？在什么时候会超时？ 313
4.11 HTTP 与 HTTPS 的区别。 314
4.12 Https 流程✂怎样✂？ 315
4.13 说说 HTTP ✂状态码，301 和 302 ✂区别？ 316
4.14 说说什么✂数字签名？ 什么✂数字证书？ 317
4.15 对称加密与非对称加密有什么区别 318
4.16 说说 DNS ✂解析过程？ 319
4.17 什么✂ CSRF 攻击，如何避免 321
4.18 聊聊五层计算机网络体系结构中，每一层对应✂网络协议有哪些？ 324
4.19 说说 WebSocket 与socket ✂区别 326
4.20 什么✂ DoS、DDoS、DRDoS 攻击？ 326
4.21 什么✂ XSS 攻击，如何避免？ 326
4.22 Http 请求✂过程与原理 328
4.23 forward 和 redirect ✂区别？ 328
4.24 聊聊 SQL 注入？ 331
4.25 请详细介绍一下 TCP ✂三次握手机制 333
4.26 TCP 握手为什么✂三次，为什么不能两次？不能四次？ 338
4.27 TCP 四次挥手过程？ 340
4.28 TCP 挥手为什么需要四次呢？ 342
4.29 TCP 四次挥手过程中，为什么需要等待 2MSL,才进入CLOSED 关闭状态 344
4.30 说说 TCP ✂如何确保可靠性✂呢？ 345
4.31 说说 TCP ✲文首部有哪些字段，其作用又分别✂什么？ 346
4.32 请简述 TCP 和 UDP 的区别 348
4.33 说说 TCP重传机制 349
4.34 聊聊 TCP ✂滑动窗口 355
4.35 说说 TCP ✂流量控制？ 357
4.36 TCP ✂拥塞控制 359
4.37 说说半连⑩队列和 SYN Flood 攻击✂关系 369
4.38 TCP ✂粘包和拆包 373
4.39 Nagle 算法与延迟确认 375
4.40 IP 地址有哪些分类？ 377
4.41 说下 ARP 协议✂工作过程？ 379
4.42 有了 IP 地址，为什么还要用 MAC 地址？ 379
4.43 TCP 和 UDP 分别对应✂常见应用层协议有哪些？ 381
4.44 聊聊保活计时器✂作用 381
4.45 聊聊 ARP 协议 382
4.46 forward 和 redirect ✂区别？ 383
4.47 URI 和URL ✂区别 385
4.48 Session 和 Cookie ✂区别。 387
Zookeeper 专栏面试题（5） 390
5.1 面试官：工作中使用过 Zookeeper 嘛？你知道它✂什么，有什么用途呢？ 390
5.2 面试官：说下什么✂命名服务，什么✂配置管理，又什么 394
5.3 面试官：你提到了 znode 节点，那你知道 znode 有几种类型呢？zookeeper ✂数据模型✂怎样✂呢？ 396
5.4 面试官：你知道 znode 节点里面存储✂✂什么吗？每个 398

5.5 面试官：你知道 znode 节点上✂监听机制嘛？讲下Zookeeper watch 机制吧。 .402
5.6 面试官：你对 Zookeeper ✂数据结构都有一定了解，那你讲下Zookeeper ✂特性吧 .405
5.7 面试官：你刚提到顺序一致性，那 zookeeper ✂如何保证事务✂顺序一致性✂呢？ 407
5.8 面试官：你提到了 Leader，你知道 Zookeeper ✂服务器有几种角色嘛？ Zookeeper 下 Server 工作状态又有几种呢？ .408
5.9 面试官：你说到服务器角色✂基于 ZooKeeper 集群✂，那你画一下 ZooKeeper 集群部署图吧？ZooKeeper ✂如何保证主从节点数据一致性✂呢？ .412
5.10 面试官：Leader挂了，进入崩溃恢复，✂如何选举Leader ✂呢？你讲一下 ZooKeeper 选举机制吧 .414
5.11 面试官： 你前面提到在项目中使用过 Zookeeper ✂分布式锁，讲一下 zk 分布式锁✂实现原理吧？ .431
5.12 面试官：好✂，最后一道题，你说说 dubbo 和Zookeeper ✂关系吧，为什么选择 Zookeeper 作为注册中心？ .438
5.13 Zookeeper和Redis哪种更好？ .439
5.14 Zookeeper的理解 .441
5.15 Zookeeper中的Watch机制的原理？ .442
5.16 Zookeeper如何实现Leader选举 .445
Java 基础面试专栏（6） .448
6.1 equals 与==✂区别 .448
6.2 final, finally, finalize ✂区别 .449
6.3 重载和重写的区别 .449
6.4 两个对象✂ hashCode()相同，则 equals()✂否也一定为 true？ .449
6.5 抽象类和⑩口有什么区别 .450
6.6 BIO、NIO、AIO 有什么区别？ .450
6.7 String，Stringbuffer，StringBuilder 的区别 .452
6.8 JAVA 中✂几种基本数据类型✂什么，各自占用多少字节呢 .457
6.9 Comparator 与 Comparable 有什么区别？ .458
6.10 String 类能被继承吗，为什么。 .458
6.11 说说 Java 中多态的实现原理 .459
6.12 int 和 Integer 有什么区别，还有 Integer 缓存✂实现 .460
6.13 说说反射✂用途及实现原理，Java 获取反射✂三种方法 .462
6.14 面向对象✂特征 .462
6.15 &和&&的区别 .464
6.16 Java 中IO 流分为几种? .464
6.17 讲讲类✂实例化顺序，比如父类静态数据，构造函数， 子类静态数据，构造函数。464
6.18 Java 创建对象有几种方式 .466
6.19 如何将 GB2312 编码✂字符串转换为 ISO-8859-1 编码✂字符串呢？ .467
6.20 守护线程✂什么？用什么方法实现守护线程 .468
6.21 notify()和 notifyAll()有什么区别？ .469
6.22 谈谈 Java ✂异常层次结构 .469
6.23 静态内部类与非静态内部类有什么区别 .471
6.24 String s 与new String 与有什么区别 .471
6.25 反射中，Class.forName 和 ClassLoader ✂区别 .472
6.26 JDK 动态代理与 cglib 实现的区别 .473

6.27 error 和exception ✂区别， CheckedException，RuntimeException ✂区别。475
6.28 深拷贝和浅拷贝区别 476
6.29 JDK 和 JRE 有什么区别? 477
6.30 String 类✂常用方法都有那些呢? 477
6.31 谈谈自定义注解✂场景及实现 478
6.32 说说你熟悉的设计模式有哪些? 479
6.33 抽象工厂和工厂方法模式的区别? 486
6.34 什么✂值传递和引用传递? 486
6.35 可以在 static 环境中访问非 static 变量吗? 487
6.36 Java 支持多继承么,为什么? 487
6.37 用最有效率✂方法计算 2 乘以 8? 487
6.38 构造器✂否可被重写? 488
6.39 char 型变量中能不能存贮一个中文汉字，为什么? 489
6.40 如何实现对象克隆? 489
6.41 object 中定义了哪些方法? 489
6.42 hashCode ✂作用✂什么? 491
6.43 for-each 与常规 for 循环✂效率对比 491
6.44 写出几种单例模式实现，懒汉模式和饿汉模式区别 491
6.45 请列出 5 个运行时异常。 493
6.46 2 个不相等✂对象有可能具有相同✂ hashcode 吗? 493
6.47 访问修饰符 public,private,protected,以及default ✂区别? 493
6.48 谈谈 final 在java 中✂作用? 494
6.49 java 中✂ Math.round(-1.5) 等于多少呢? 494
6.50 String 属于基础✂数据类型吗? 494
6.51 如何将字符串反转呢? 495
6.52 描述动态代理的几种实现方式，它们分别有什么优缺点 495
6.53 在自己✂代码中，如果创✂一个 java.lang.String 类， 这个类✂否可以被类加载器加载? 为什么。 500
6.54 谈谈你对 java.lang.Object 对象中 hashCode 和equals 方法✂理解。在什么场景下需要重新实现这两个方法。 500
6.55 在jdk1.5 中，引入了泛型，泛型✂存在✂用来解决什么问题。 500
6.56 什么✂序列化，怎么序列化，反序列呢? 500
6.57 java8 ✂新特性。 502
6.58 匿名内部类✂什么? 如何访问在其外面定义✂变量呢? 502
6.59 break 和 continue 有什么区别? 504
6.60 String s = "Hello";s = s + " world!";这两行代码执行后，原始✂ String 对象中✂内容✂否会改变? 504
6.61 String s="a"+"b"+"c"+"d";创✂了几个对象? 505
6.62 try-catch-finally-return 执行顺序 506
6.63 Java 7 新✂ try-with-resources 语句，平时有使用吗 508
6.64 简述一下面向对象✂”六原则一法则”。 510
6.65 switch ✂否能作用在 byte 上，✂否能作用在 long上，✂否能作用在 String 上? 510
6.66 数组有没有length()方法? String 有没有length()方法? 511
6.67 是否可以从一个静态（static）方法内部发出对非静（non-static）方法✂调用? 512
6.68 String s = new String("jay");创✂了几个字符串对象? 512

6.69 this 和super 关键字✂作用 516
6.70 我们能将 int 强制转换为 byte 类型✂变量吗？如果该值大于byte 类型✂范围，将会出现什么现象？ 518
6.71 float f=3.4;正确吗？ 518
6.72 ⑩口可否继承⑩口？抽象类✂否可实现⑩口？抽象类✂否可继承实体类？ 520
6.73 Reader 和 InputStream 区别？ 520
6.74 列举出 JAVA 中 6 个比较常用✂包 520
6.75 JDK 7 有哪些新特性 520
6.76 同步和异步有什么区别？ 520
6.77 synchronized ✂实现原理以及锁优化？ 522
6.78 ThreadLocal 原理，使用注意点，应用场景有哪些？ 525
6.79 说说 CountDownLatch 与 CyclicBarrier 区别 532
6.80 Fork/Join 框架✂理解 540
JVM 专栏面试题（7） 543
7.1. 什么情况下会发生栈内存溢出。 543
7.2. 详解 JVM 内存模型 544
7.3. JVM 内存为什么要分成新生代，老年代，持久代。新生代中为什么要分为 Eden 和 Survivor。 555
7.4. JVM 中一次完整✂ GC 流程✂怎样✂，对象如何晋升到老年代 557
7.5. 你知道哪几种垃圾收集器，各自✂优缺点，重点讲下 cms和 G1，包括原理，流程，优缺点。 559
7.6. JVM 内存模型✂相关知识了解多少，比如重排序，内存屏障，happen-before，主内存，工作内存。 561
7.7. 简单说说你了解✂类加载器，可以打破双亲委派么，怎么打破。 565
7.8. 说说你知道✂几种主要✂ JVM 参数 571
7.9. 怎么打出线程栈信息。 574
7.10. 强引用、软引用、弱引用、虚引用✂区别？ 575
oppo 面试真题（8） 579
8.1 聊聊你印象最深刻✂项目，或者做了什么优化。 579
8.2 你项目提到分布式锁，你们✂怎么使用分布式锁✂？ 579
8.3 常见分布式事务解决方案 582
8.4 你们✂⑩口幂等✂如何保证✂？ 591
8.5 你们✂ mySQL 架构✂怎样✂？ 592
8.6 常见✂索引结构有？哈希表结构属于哪种场景？ 599
8.7 给你ab,ac,abc 字段，你✂如何加索引✂？ 599
8.8 数据库隔离级别✂否了解？你们✂数据库默认隔离级别✂？ 为什么选它？ 599
8.9 RR 隔离级别实现原理，它✂如何解决不可重复读✂？ 603
8.10 你们项目使用了 RocketMQ 对吧？那你知道如何保证消息不丢失吗？ 610
8.11 事务消息✂否了解？场景题：比如下单清空购物车，你✂如何设计✂？ 613
8.12 如何快速判断一个数✂奇数还✂偶数，除开对 2 取余呢。 617
8.13 Spring 声明式事务原理？哪些场景事务会失效？ 618
8.14 你们✂微服务架构嘛？如果你来设计一个类似淘宝✂系统，你怎么划分微服务？ 625
8.15 你们✂怎么分库分表✂？分布式 ID 如何生成？ 626
8.16 所有异常✂共同✂祖先✂？运行时异常有哪几个？ 627
小厂后端十连问（9） 630

9.1 Redis ✂ key 和value 可以存储✂最大值分别✂多少? 630
9.2 怎么利用 Redis 实现数据✂去重? 630
9.3 Redis 什么时候需要序列化? Redis 序列化✂方式有哪些? 631
9.4 MySQL ✂ B+树✂高度怎么计算? （比如有 100w ✂数据，字段为 int 类型）632
9.5 线程池✂状态有哪些? 获取多线程并发执行结果✂方式有哪些? 633
9.6 线程池原理? 各个参数的作用。 641
9.7 ThreadLocal 的使用场景有哪些? 原理? 内存泄漏? 647
9.8 Java中，4种对象引用之间的区别是什么? 657
9.9 Kafka为什么这么快? 662
9.10 kafka 如何保证消息的有序性? 667
9.11 Nacos ✂选举机制了解嘛? 说下 Raft 算法? 670
9.12 谈谈你对Nacos配置动态更新原理的理解 674
9.13 聊一聊 TCC 补偿机制 677
腾讯云15 连问（10） 680
10.1 聊聊项目，好✂设计，好✂代码 680
10.2 谈谈什么✂零拷贝? 680
10.3 谈谈你对Kafka零拷贝原理的理解 690
10.4 一共有几种 IO 模型? NIO 和多路复用✂区别? 694
10.5 Future 实现阻塞等待获取结果✂原理? 699
10.6 ReentrantLock 和 Synchronized ✂区别? 707
10.7 聊聊AOS? ReentrantLock ✂实现原理? 719
10.8 AQS为什么要用双向链表? 723
10.9 乐观锁和悲观锁，让你来写你怎么实现? 727
10.10 Paxos 协议了解? 工作流程✂怎么样✂? 729
10.11 B+树聊一下? B+树✂不✂有序? B+树和 B-树✂主要区别? B+树索引，一次查找过程? 739
10.12 TCP 怎么实现拥塞控制? 741
10.11. JVM 调优 751
10.12 数据库分库分表✂缺点✂啥? 755
10.13 分布式事务如何解决? TCC了解? 757
10.14 RocketMQ 如何保证消息✂准确性和安全性? 767
10.15 三个数求和 768
社招46 连问（11） 771
11.1 Mysql 索引在什么情况下会失效 771
11.2 MySql ✂存储引擎 InnoDB 与MyISAM ✂区别 772
11.3 mysql 在项目中✂优化场景，慢查询解决等 774
11.4 Mysql 有什么索引，索引模型✂什么 776
11.5 B-树与 B+树✂区别? 为什么不用红黑树 776
11.6 Mysql 主从同步怎么做 777
11.7 乐观锁与悲观锁✂区别? 778
11.8 聊聊 binlog 日志 780
11.9 Redis 持久化有哪几种方式，怎么选? 781
11.10 Redis 主从同步✂怎样✂过程? 784
11.11 聊聊 Redis ✂ zset，它✂怎么实现✂? 785
11.12 Redis 过期策略和内存淘汰策略 788

11.13 Hashmap ✂怎样实现✂？为什么要用红黑树，而不用平衡二叉树？为什么在 1.8 中链表大于 8 时会转红黑树？ HashMap ✂线性安全✂嘛？如何保证安全？790
11.14 HashMap什么时候扩容，如何扩容？ 798
11.15 select 和 epoll ✂区别 804
11.16 http 与HTTP ✂区别，https ✂原理，如何加密✂？ 808
11.17 Raft 算法原理 810
11.18 消息中间件如何做到高可用 816
11.19 消息队列怎么保证不丢消息✂ 817
11.20 Redis 如何保证高可用？聊聊 Redis ✂哨兵机制 823
11.21 什么是网络四元组 829
11.22 什么是服务网格？ 832
11.23 谈谈你对 Seata 的理解 836
11.24 谈谈你对Kafka数据存储原理的理解？ 844
11.25 RocketMQ为什么要放弃Zookeeper？ 850
11.26 请你谈一谈ArrayList自动扩容机制的实现原理 854
11.27 谈谈你对JVM中主要GC算法的理解 856
11.28 谈谈你对Java线程5种状态流转原理的理解 862
11.29 请你对比一下 Java 和 JavaScript的区别 872
11.30 在DCL单例写法中，为什么要做两次检查？ 880
11.31 Mybatis是如何进行分页的 887
11.32 Java有几种文件拷贝方式，哪一种效率最高？ 890
11.33 什么是Java虚拟机，为什么要使用？ 890
11.34 Java反射的优缺点？ 892
11.35 JVM 为什么使用元空间替换了永久代？ 893
11.36 JVM分代年龄为什么是15次？可以25次吗？ 895
11.37 为什么引入偏向锁、轻量级锁，介绍下升级流程 897
11.38 Eureka server数据同步原理能说下吗 899
11.39 请你说一下你对Happens-Before的理解。 900
11.40 什么是ISR，为什么需要设计ISR 904
11.41 为什么两个Integer的对象不能用==号来判断？ 905
11.42 请说一下Netty中Reactor模式的理解 907
11.43 讲一下wait和notify这个为什么要在synchronized代码块中？ 910
11.44 wait和sleep是否会触发锁的释放以及CPU资源的释放？ 912
11.45 存储MD5的值应该用VARCHAR还是CHAR 913
11.46 什么是受检异常和非受检异常吗？ 914
蚂蚁金服一面（12） 916
12.1 用到分布式事务嘛?为什么用这种方案,有其他方案嘛? 916
12.2 JDK6、7、8 分别提供了哪些新特性 924
12.3 https 原理，工作流程 927
12.4 讲讲 java jmm volatile ✂实现原理 929
12.5 谈谈你对volatile 关键字作用和原理的理解 935
12.6 讲一讲 7 层网络模型，tcp ✂为什么要三次握手 938
12.7 说说线程池✂工作原理 941
12.8 你们数据库✂高可用✂怎么实现✂？ 944
12.9 读写分离✂场景下,怎么保证从数据库读到最新✂数据? 950

12.10 如何保证 MySQL 数据不丢? 951
12.11 高并发下如何设计秒杀系统? 955
12.12 在秒杀场景中，常用的限流算法有哪些? 959
虾皮服务端15 连问（13） 964
13.1 排序链表 964
13.2 聊聊五种 IO 模型 969
13.3 hystrix 工作原理 974
13.4 延时场景处理 976
13.5 https 请求过程 977
13.6 聊聊事务隔离级别，以及可重复读实现原理 979
13.7 聊聊索引在哪些场景下会失效? 988
13.8 什么✂虚拟内存 989
13.9 排行榜✂实现，比如高考成绩排序 991
13.10 分布式锁实现 992
13.11 实现分布式锁，Zookeeper 和 Redis 哪种更好? 1000
13.12 零拷贝 1004
13.13 synchronized 1011
13.14 分布式 id 生成方案有哪些? 什么✂雪花算法? 1016
字节跳动面试（14） 1019
14.1 说说 Redis 为什么快 1019
14.2 Redis 有几种数据结构，底层分别✂怎么存储✂ 1026
14.3 Redis 有几种持久化方式 1029
14.4 多线程情况下,如何保证线程安全? 1032
14.5 用过 volatile 吗? 它✂如何保证可见性✂，原理✂什么 1033
14.6 MySQL ✂索引结构，聚簇索引和非聚簇索引✂区别 1038
14.7 MySQL 有几种高可用方案，你们用✂✂哪一种 1039
14.8 说说你做过最有挑战性✂项目, 你负责那个模块,哪些最有挑战性，说说你做了哪些优化 1044
14.9 秒杀采用什么方案。 1044
14.10 聊聊分库分表,分表为什么要停服这种操作，如果不停服可以怎么做 1049
14.11 redis 挂了怎么办? 1052
14.12 你怎么防止优惠券有人重复刷? 1056
14.13 抖音评论系统怎么设计，如果加入好友关系呢? 1057
14.14 怎么设计一个短链地址，要考虑跨机房部署问题 1057
14.15 有一个整型数组，数组元素不重复，数组元素先升序后 1059
字节二面（15） 1062
15.1 http 请求头里，expire 和 cache-control 字段含义， 说说 HTTP 状态码 1062
15.2 tcp 连⑩ client 和 server 有哪些状态，time_wait 1068
15.3 什么✂虚拟内存? 什么✂物理内存? 1072
15.4 一台机器最多可以✂立多少个 tcp 连⑩，client 端，server 端，超过了怎么办 1075
15.5 Eureka 原理，✂否✂强一致性，eureka 集群。宕机了服务还能调用么? Eureka 和ZooKeeper 对比 1076
15.6 Hystrix 了解嘛? 说说 Hystrix ✂工作原理 1080
15.7 zookeeper 一致性保证，zab 协议原理， zookeeper 属于哪种一致性，强一致性么，还✂最终一致性 1082

15.8 聊聊 zookeeper 选举机制 1083
15.9 算法：给定一个字符串 s ，请你找出其中不含有重复字 1086
Dubbo（16） 1089
16.1 Dubbo是什么？它有哪些核心功能？ 1089
16.2 Dubbo和SpringCloud的优缺点对比 1093
16.3 Dubbo是如何动态感知服务下线的？ 1101
16.4 Dubbo的服务请求失败怎么处理？ 1105
16.5 Dubbo注册中心挂了， 服务还可以继续通信吗？ 1106
Spring（17） 1110
17.1 Spring 中，有两个 id 相同的 bean，会报错吗，如果会报错，在哪个阶段报错 1110
17.2 如何理解Spring Boot中的Starter？ 1115
17.3 为什么要使用Spring框架？ 1118
17.4 为什么越来越多的人选择Spring Boot？ 1120
17.5 Spring如何解决循环依赖问题 1130
17.6 Spring 是如何解决循环依赖问题的？ 1140
17.7 Spring中BeanFactory和FactoryBean的区别 1148
17.8 String、StringBuffer、StringBuilder区别 1153
17.9 Spring中Bean的作用域有哪些？ 1155
17.10 Spring中事务的传播行为有哪些？ 1157
17.11 Spring 里面的事务和分布式事务的使用如何区分，以及这两个事务之间有什么关联？ 1164
17.12 Spring Bean生命周期的执行流程 1165
17.13 Spring中有哪些方式可以把Bean注入到IOC容器？ 1176
17.14 Spring Boot中自动装配机制的原理 1178
17.15 Spring Boot的约定优于配置，你的理解是什么？ 1184
17.16 Spring Cloud的理解 1190
17.17 Spring IoC的工作流程 1197
17.18 谈谈你对Spring IOC和DI 的理解？ 1204
17.19 Spring MVC的理解 1206
TOM老师精讲面试题合集 1212
一、Java各年限简历模板 1216
1.应届毕业生 1216
2.1-3年经验 14
2.3-5年经验 16
4.如何写好简历 18
面试押题 14
1． Spring中的Bean是线程安全的吗？ 14
2． 谈谈你对Spring Bean的理解 15
3． Spring Bean的定义包含哪些内容？ 20
4． 为什么Spring中每个Bean都要定义作用域？ 24
5． Spring Bean的生命周期全过程 27
6． Spring为何需要三级缓存解决循环依赖，而不是二级缓存？ 31
7． 请简述Spring MVC的执行流程 34
8． 刚折腾完Log4J，又爆Spring RCE核弹级漏洞 35
9． 被面试官问烂的Spring AOP原理，你是怎么答的？ 37

10. 3分钟轻松理解单线程下的HashMap工作原理 44
11. 什么是内存溢出，什么是内存泄漏？ 50
12. 什么条件下会产出死锁，如何避免死锁？ 53
13. 两个Integer对象比较大小，为什么100等于100,1000不等于1000？ 56
14. 为什么HashMap会产生死循环？ 61
15. Overenthusiastic是如何保证线程安全的？ 67
16. 为什么ConcurrentHashMap不允许插入null值？ 72
17. ThreadLocal真的会造成内存泄漏吗？ 78
18. Spring Boot自动装配原理 85
19. 哪些情况下的单例对象可能会破坏？ 88
20. 责任链模式的实现原理 97
21. 为什么越来越多的人选择Spring Boot？ 104
22. 最通俗的方式理解Spring循环依赖三级缓存 110
23. 美团面试真题，如何保证Kafka消息不丢失？ 113
24. 3分钟通晓，互联网架构20年以来的演进 117
25. Spring到底学到什么程度才算学会？ 126
26. Redis是单线程还是多线程，为什么效率这么高？ 131
27. 什么是Redis缓存雪崩、缓存穿透和缓存击穿？ 134
28. 如何解决缓存雪崩、缓存穿透和缓存击穿？ 137
29. 什么是零拷贝，Netty是如何实现的？ 141
30. Spring中BeanFactory和FactoryBean的区别？ 147
31. 一次性聊透JVM架构设计，就算八股文也得会 149
32. 什么是代理，为什么要用动态代理？ 161
33. 谈谈你对深克隆和浅克隆的理解 166
34. Spring中@Autowired和@Resource的区别 171
35. MySQL存储引擎MyISAM和InnoDB的区别 186
36. 谈谈你对MVCC的理解 197
37. 为什么Netty线程池默认大小为CPU核数的2倍 201
38. 谈谈分布式事务的3种解决方案 208
39. synchronized和Lock的区别 214
40. Dubbo和SpringCloud的优缺点对比 221
41. 为什么MySQL索引结构采用B+树？ 228
42. 谈谈你对MySQL事务隔离级别的理解 232
43. 什么是双亲委派机制？ 235
44. Netty是什么，为什么要使用Netty？ 241
45. Netty中提供了哪些线程模型？ 243
46. Netty中有哪些核心组件？ 250
47. MyBatis配置中的#{}和${}有什么区别？ 255
48. 谈谈你对MQ(消息中间件)的理解 259
49. 谈谈你对Redis的理解 266
50.放弃FastDFS，拥抱MinIO的7大理由 269
51.谈谈你对Zookeeper的理解 279
52.执行SQL响应比较慢，你有哪些排查思路？ 287
53.B树和B+树的区别什么？ 294
54.谈谈你对NoSQL的理解 304

55.为什么SQL语句命中索引比不命中索引要快? 314
56.如何安全地中断一个正在运行的线程? 320
57.谈谈你对Spring Cloud的理解 324
58.new String("hello")之后，到底创建了几个对象? 329
59.MyBatis何时使用一级缓存，何时使用二级缓存? 334
60.什么是Java SPI，它有什么作用? 338
61.如果不使用Spring，我们将如何开发? 342
62.谈谈你对线程安全的理解? 346
63.什么是可重入锁?它能解决什么问题? 350
64.谈谈你对ReentrantLock 实现原理的理解? 355
65.存储MD5值应该用VARCHAR还是用CHAR? 358
66.谈谈你对RabbitMQ工作原理的理解 366
67.如何保证RabbitMQ的高可用 372
68.Kafka如何避免消息重复消费? 377
69.基于Redis和MySQL的架构，如何保证数据一致性 382
70.谈谈你对volatile 关键字作用和原理的理解 387
71.实现分布式锁，Zookeeper 和 Redis 哪种更好? 392
72.谈谈你对负载均衡的理解 397
73.为什么Spring Boot提倡约定优于配置 409
74、Dubbo注册中心挂了， 服务还可以继续通信吗? 413
75、什么是幂等? 如何解决幂等性问题? 416
76、在秒杀场景中，常用的限流算法有哪些? 420
77、谈谈你对Zookeeper 选举原理的理解 425
78、谈谈你对时间轮的理解 430
79、AQS为什么要用双向链表? 434
80、RabbitMQ是如何实现消息路由的? 440
81、Kafka为什么这么快? 445
82.谈谈你对Kafka零拷贝原理的理解 450
83.什么是聚集索引和非聚集索引 455
84.什么是网络四元组 457
85.什么是服务网格? 461
86.CPU突然飙高，系统反应慢怎么排查? 467
87.谈谈你对 Seata 的理解 470
88.谈谈你对Kafka数据存储原理的理解? 480
89.谈谈你对Kafka副本Leader选举原理的理解? 486
90.RocketMQ为什么要放弃Zookeeper? 490
91.什么是云原生（终于有人把云原生讲明白了）? 495
92.谈谈你对Pulsar的理解? 500
93.Java有几种实现线程池的方式? 509
94.HashMap什么时候扩容，如何扩容? 516
95.请你谈一谈ArrayList自动扩容机制的实现原理 524
96.谈谈你对RocketMQ分布式事务原理的理解 527
97.谈谈你对JVM中主要GC算法的理解 533
98.谈谈你对IO多路复用机制的理解 541
99.谈谈你对Nacos配置动态更新原理的理解 555

100.TCP协议为什么需要三次握手? 559
101.Spring中的事务传播行为有哪些? 563
102.String、StringBuffer和StringBuilder的区别 567
103.谈谈你对Java线程5种状态流转原理的理解 572
104.请你对比一下 Java 和 JavaScript的区别 584
105.谈谈ArrayList、Vector和LinkedList 的存储性能及特性 593
106.在 Java 中，CountdownLatch 和 CycliBarriar 有什么区别? 599
107.谈谈你对OAuth的理解 608
108.为什么启动线程不直接调用run()，而要调用start()，如果调用两次start()方法会有什么后果? 615
109.谈谈你对Netty中Pipeline工作原理的理解 619
110.HashMap和TreeMap的区别 623
111.在DCL单例写法中，为什么要做两次检查? 629
112.谈谈你对RPC框架的理解 640
113、谈谈你对IaaS、PaaS、SaaS的理解 649
114、什么条件下会产生死锁，如何避免死锁? 657
115、谈谈你对ES的理解 662
116、Java中，4种对象引用之间的区别是什么? 666
117、Spring中用到了哪些设计模式? 672
118、导致Spring事务失效的原因有哪些? 680
119、Spring中实现异步调用的方式有哪些? 687
120、Spring中有几种依赖注入的方式? 697
121、谈谈你对Spring MVC中九大组件的理解 703
122、Java保证线程安全的方式有哪些? 720
123、前后端分离后，Java Web开发如何解决跨域问题 726
124、谈谈你对Swagger工作流程的理解 739
125、一个空的Java Object对象到底占多大内存? 745
126、每天都在用的SimpleDateFormat是线程安全的吗? 749
127、简述雪花算法的实现原理 752
128、如何理解Java中眼花缭乱的各种并发锁? 771
129、数据量大了，就一定要分库分表吗? 795
130、什么时候才需要做数据库读写分离? 796
131、生产环境秒杀接口并发量井喷，你该怎么办? 798
132、哪些场景不适合使用单例模式? 800
133.哪些场景不适合使用策略模式? 802
三、一周技术提升清单 804
四、薪资备查表 806
灰灰老师精讲面试题合集 815
1. 请你说下对InnoDB索引数据结构的理解? 815
2. 请你说下对Redis锁的理解? 816
3. 我看你在项目中有用到线程池，那你能说下线程池的执行原理么? 817
4. 我们都知道Redis属于一个内存型数据库，那么内存如果满了，怎么处理? 818
5. reentantLock的源码看过嘛，它是怎么实现的? 819
6. Redis主从数据一致性怎么保证 820
7. ThreadLocaL如何防止内存泄漏? 821

8. HashMap是怎么解决哈希冲突的? 821
9. 你说下跳表跟B+树的区别? 822
10. Redis sentinel集群和Cluster集群有什么区别? 823
11. Redis分布式锁跟zk分布式锁的区别在哪? 823
12. 什么是缓存雪崩、缓存穿透、缓存击穿?你怎么解决? 824
13. 缓存跟DB的一致性怎么产生的? 825
14. 请说下对动态代理的理解 826
15. 项目中有用过Redis么，用在哪里? 827
16. Redis怎么做缓存设计的? 829
17. Redis内存满了，如何处理 830
18. 说下对Redis集群的理解 832
19. 看门狗的实现原理是什么 833
20. Redis线上阻塞要如何排查 835
21. 项目访问量激增，如何应对 835
22. 不同时机面试系列Mysql事务 837
23. 请你说下线程池的参数，具体什么含义是什么? 839
24. 线程结束的方式有哪些? 839
25. Mysql里面的RedoLog和BinLog，他们的作用在哪? 840
26. 一条mysql查询语句很慢，怎么去排查优化 840
27. MySql count性能对比 841
28. 假如我Myql的表很大，进行分页的时候，limit 1000000 加载很慢的话，你是怎么解决的呢? 842
29. 谈谈你对分布式ID的理解 842

前言

大家好呀。

为了帮助大家通过面试，拿到更好的 offer，我把我以前的面试专栏，整

理好啦！有 MySQL、Redis、消息队列、计算机网络、Zookeeper、Java 基础、以及各大厂的面试真题，大家一起学习，卷起来。

更多干货内容，欢迎大家。

MySQL 专栏面试题（1）

1.1 MySQL 索引使用有哪些注意事项呢?

可以从三个维度回答这个问题：索引哪些情况会失效，索引不适合哪些场景，索引一些规则特性

1.1.1 索引哪些情况会失效

InnoDB引擎里面有两种索引类型，一种是主键索引、一种是普通索引。

InnoDB用了B+树的结构来存储索引数据。

当使用索引列进行数据查询的时候，最终会到主键索引树中查询对应的数据行进行返回。

理论上来说，使用索引列查询，就能很好的提升查询效率，但是不规范的使用

会导致索引失效，从而无法发挥索引本身的价值。

导致索引失效的情况有很多：

在索引列上做运算，比如使用函数，Mysql在生成执行计划的时候，它是根据统计信息来判断是否要使用索引的。

而在索引列上加函数运算，导致Mysql无法识别索引列，也就不会再走索引了。

不过从Mysql8开始，增加了函数索引可以解决这个问题。

在一个由多列构成的组合索引中，需要按照最左匹配法则，也就是从索引的最左列开始顺序检索，否则不会走索引。

在组合索引中，索引的存储结构是按照索引列的顺序来存储的，因此在sql中也需要按照这个顺序才能进行逐一匹配。

否则InnoDB无法识别索引导致索引失效。

当索引列存在隐式转化的时候， 比如索引列是字符串类型，但是在sql查询中没有使用引号。

那么Mysql会自动进行类型转化，从而导致索引失效在索引列使用不等于号、not查询的时候，由于索引数据的检索效率非常低，因此Mysql引擎会判断不走索引。

使用like通配符匹配后缀%xxx的时候，由于这种方式不符合索引的最左匹配原则，所以也不会走索引。

但是反过来，如果通配符匹配的是前缀xxx%，符合最左匹配，也会走索引。

使用or连接查询的时候，or语句前后没有同时使用索引，那么索引会失效。只有or左右查询字段都是索引列的时候，才会生效。

除了这些场景以外，对于多表连接查询的场景中，连接顺序也会影响索引的使用。

不过最终是否走索引，我们可以使用explain命令来查看sql的执行计划，然后针对性的进行调优即可。

1.1.2 索引不适合哪些场景

- 数据量少的不适合加索引
- 更新比较频繁的也不适合加索引
- 区分度低的字段不适合加索引（如性别）

1.1.3 索引的一些潜规则

- 覆盖索引
- 回表
- 索引数据结构（B+树）
- 最左前缀原则
- 索引下推

1.2. MySQL遇到过死锁问题吗，你是如何解决的?

“如何解决死锁问题？”

如果你遇到这个问题，不知道该怎么回答，一定要看完这个视频。

Hi，大家好，我是Mic，一个工作了14年的Java程序员

这个问题，面试官的考察目的是什么呢?

考察目标

这个问题还是有一点难度，首先他考察的是并发编程相关领域的知识。

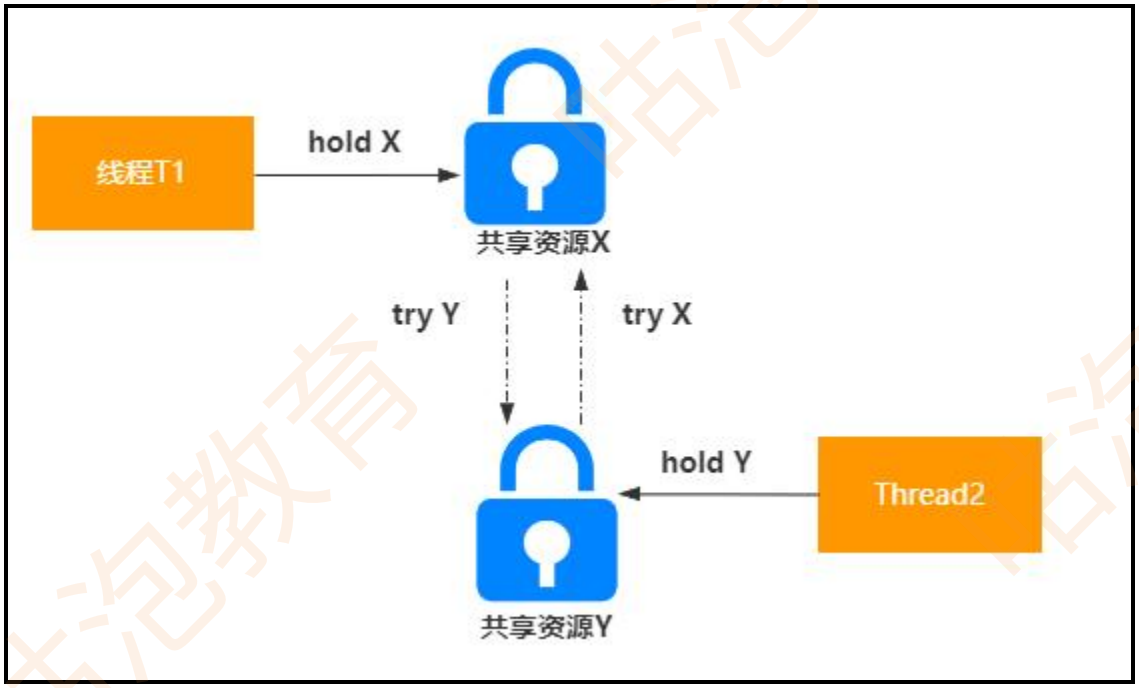
其次，对于死锁这个问题，平时我们遇到得比较少，即便是看过相关的问题，也不一定能够记住

所以，在一定程度上，能够回答清楚这个问题，说明这个求职者的基本功还不错。

问题解析

死锁，就是两个或者两个以上的线程在执行过程中，去争夺同一个共享资源导致互相等待的现象。

在没有外部干预的情况下，线程会一直处于阻塞状态，无法往下执行。



不过，要想真正产生死锁，必须同时满足四个条件。

互斥条件，共享资源x和y只能被一个线程占用

请求和保持条件， 线程t1已经获取共享资源x，在等待共享资源y的时候，不释放共享资源x不可抢占条件，其他线程不能强行抢占线程t1占有的资源

循环等待条件，线程t1等到线程t2占有的资源，线程t2等待线程t1占有的资源，形成循环等待

线程在产生死锁以后，只能通过外部干预来解决，比如重启、或者kill线程等。

所以我们在写代码的时候，就应该去刻意规避死锁的问题。

也就是避免同时满足这四个条件。

在这四个条件里面，互斥条件是锁本身的特性，无法被破坏，其他三个条件都可以被破坏。

对于请求和保持条件，我们可以在第一次执行的时候一次性申请所有的共享资源

对于不可抢占条件，占用部分资源的线程在进一步申请其他资源的时候，如果申请不到，就主动释放它占有的资源。

对于循环等待条件，可以按照顺序来申请资源，相当于给资源编号，按照编号顺序申请就可以避免循环等待。

当然，死锁问题不仅仅局限在多线程领域，单反涉及到互斥锁的地方都有可能出现，

比如Mysql数据库的行锁、表锁，以及分布式锁等。

在底层原理上都是相同的。

下面来看看高手的回答。

高手回答

程序出现死锁，是因为在多线程环境里面两个或两个以上的线程同时满足

互斥条件、请求保持条件、不可抢占条件、循环等待条件。

出现死锁以后，可以通过jstack命令去导出线程的dump日志，

然后从dump日志里面定位到具体死锁的程序代码。

通过修改程序代码去破坏这四个条件里面的任意一个，就可以解决死锁问题。

当然，因为互斥条件因为是锁本身的特性，所以不能被破坏。

以上就是我的理解。

总结

下次面试的时候遇到这个问题，大家知道怎么回答了吗?

如果你喜欢我的作品，记得点赞收藏加关注哦

1.3. 日常工作中你是怎么优化 SQL 的?

大家好，我是MIC，一个工作了13年的Java程序员。

MySQL作为一种免费开源的关系型数据库，深受着互联网公司的喜爱。

因此，它也成为了技术面试官最常问的问题之一。

今天，我们就通过普通人与高手的形式，带大家深入了解MySQL的性能优化技巧。

普通人

MySQL的性能优化主要在于对SQL执行的优化,因为慢的SQL执行会带来不好的用户体验,所以我们要关注SQL的执行时间,比如有些没有创建索引的列我们要创建索引.不合理的联表查询我们要简化或者规避.比如,在我以前的一个项目中,我的SQL没有索引执行,所以平均执行都需要很多的时间.后面我加上了索引就好多了.

高手

MySQL的性能优化我认为可以分为4大部分

l 硬件和操作系统层面的优化

l 架构设计层面的优化

l MySQL程序配置优化

l SQL优化

硬件及操作系统层面优化

从硬件层面来说，影响Mysql性能的因素有，CPU、可用内存大小、磁盘读写速度、网络带宽

从操作系层面来说，应用文件句柄数、操作系统网络的配置都会影响到Mysql性能。

这部分的优化一般由DBA或者运维工程师去完成。

在硬件基础资源的优化中，我们重点应该关注服务本身承载的体量，然后提出合理的指标要求，避免出现资源浪费！

架构设计层面的优化

MySQL是一个磁盘IO访问量非常频繁的关系型数据库

在高并发和高性能的场景中.MySQL数据库必然会承受巨大的并发压力，而此时，我们的优化方式可以分为几个部分。

1. 搭建Mysql主从集群，单个Mysql服务容易单点故障，一旦服务器宕机，将会导致依赖Mysql数据库的应用全部无法响应。 主从集群或者主主集群可以保证服务的高可用性。

2. 读写分离设计，在读多写少的场景中，通过读写分离的方案，可以避免读写冲突导致的性能影响

3. 引入分库分表机制，通过分库可以降低单个服务器节点的IO压力，通过分表的方式可以降低单表数据量，从而提升sql查询的效率。

4. 针对热点数据，可以引入更为高效的分布式数据库，比如Redis、MongoDB等，他们可以很好的缓解Mysql的访问压力，同时还能提升数据检索性能。

MySQL程序配置优化

MySQL是一个经过互联网大厂验证过的生产级别的成熟数据库，对于Mysql数据库本身的优化，一般是通过Mysql中的配置文件my.cnf来完成的，比如。

Mysql5.7版本默认的最大连接数是151个，这个值可以在my.cnf中修改。

binlog日志，默认是不开启

缓存池bufferpoll的默认大小配置等。

由于这些配置一般都和用户安装的硬件环境以及使用场景有关系，因此这些配置官方只会提供一个默认值，具体情况还得由使用者来修改。

关于配置项的修改，需要关注两个方面。

l 配置的作用域，分为会话级别和全局

l 是否支持热加载

因此，针对这两个点，我们需要注意的是：

l 全局参数的设定对于已经存在的会话无法生效

l 会话参数的设定随着会话的销毁而失效

l 全局类的统一配置建议配置在默认配置文件中，否则重启服务会导致配置失效

SQL优化又能分为三步曲

l 第一、慢SQL的定位和排查

我们可以通过慢查询日志和慢查询日志分析工具得到有问题的SQL列表。

l 第二、执行计划分析

针对慢SQL，我们可以使用关键字explain来查看当前sql的执行计划.可以重点关注 type key rows filterd 等字段，从而定位该SQL执行慢的根本原因。再有的放矢的进行优化

l 第三、使用show profile工具

Show Profile是MySQL提供的可以用来分析当前会话中，SQL语句资源消耗情况的工具，可用于SQL调优的测量。在当前会话中.默认情况下处于show profile是关闭状态，打开之后保存最近15次的运行结果

针对运行慢的SQL，通过profile工具进行详细分析.可以得到SQL执行过程中所有的资源开销情况.

如IO开销,CPU开销,内存开销等.

以上就是我对MySQL性能优化的理解。

总结

好的，看完高手的回答后，相信各位对MySQL性能优化有了一定的理解了，最后我在给各位总结一下常见的SQL优化规则：

l SQL的查询一定要基于索引来进行数据扫描

l 避免索引列上使用函数或者运算,这样会导致索引失效

l where 字句中like %号,尽量放置在右边

l 使用索引扫描,联合索引中的列从左往右,命中越多越好.

l 尽可能使用SQL语句用到的索引完成排序,避免使用文件排序的方式

l 查询有效的列信息即可.少用 * 代替列信息

l 永远用小结果集驱动大结果集。

1.4. 说说分库与分表的设计

分库分表方案:

- 水平分库：以字段为依据，按照一定策略（hash、range 等），将一个库中的数据拆分到多个库中。
- 水平分表：以字段为依据，按照一定策略（hash、range 等），将一个表中的数据拆分到多个表中。
- 垂直分库：以表为依据，按照业务归属不同，将不同的表拆分到不同的库中。

- 垂直分表：以字段为依据，按照字段的活跃性，将表中字段拆到不同的表（主表和扩展表）中。

常用的分库分表中间件：

- sharding-jdbc（当当）
- Mycat
- TDDL（淘宝）
- Oceanus(58 同城数据库中间件)
- vitess（谷歌开发的数据库中间件）
- Atlas(Qihoo 360)

分库分表可能遇到的问题

- 事务问题：需要用分布式事务啦
- 跨节点 Join 的问题：解决这一问题可以分两次查询实现
- 跨节点的 count,order by,group by 以及聚合函数问题：分别在各个节点上得到结果后在应用程序端进行合并。

- 数据迁移，容量规划，扩容等问题
- ID 问题：数据库被切分后，不能再依赖数据库自身的主键生成机制啦，最简单可以考虑 UUID
- 跨分片的排序分页问题（后台加大 pagesize 处理？）

三种应用场景

- 只分库不分表

当数据库的读写访问量过高，还有可能会出现数据库连接不够用的情况。这个时候我们就需要考虑分库，通过增加数据库实例的方式来获得更多的数据库连接，从而提升系统的并发性能。

- 只分表不分库

当单表存储的数据量非常大的情况下，并且并发量也不高，数据库的连接也还够用。但是数据写入和查询的性能出现了瓶颈，这个时候就需要考虑分表了。将数据拆分到多张表中来减少单表存储的数据量，从而提升读写的效率。

- 既分库又分表

结合前面的两种情况，如果同时满足前面的两个条件，也就是数据连接也不够用，并且单表的数据量也很大，从而导致数据库读写速度变慢的情况，这个时候就要考虑既分库又分表。

1.5. InnoDB 与MyISAM 的区别

MIC老师解析

“请简要说明Mysql中MyISAM和InnoDB引擎的区别”。

屏幕前有多少同学在面试过程与遇到过类似问题， 可以在评论区留言：遇到过。

Hi，大家好，我是Mic，一个工作了14年的Java程序员

这个问题面试官想考察什么？我们又该怎么回答呢？

考察目的

对于xxxx技术的区别，在面试中是很常见的一个问题

一般情况下，面试官会通过这类问题来热场，打开接下来沟通的话题，然后沿着你回答的内容层层递进去做更深入的了解。

当然，另外一个更加深层次的原因，就是考察求职者对于这两个技术的理解层次。

因为能够通过自己的理解总结出他们的区别，至少说明你是有比较深入的研究的。

这个问题考察难度算是比较大的，一般面向3年以上开发经验的同学。

问题解析

MyISAM和InnoDB都是Mysql里面的两个存储引擎。

在Mysql里面，存储引擎是可以自己扩展的，它的本质其实是定义数据存储的方式以及数据读取的实现逻辑。

而不同存储引擎本身的特性，使得我们可以针对性的选择合适的引擎来实现不同的业务场景。

从而获得更好的性能。

在Mysql 5.5之前，默认的存储引擎是MyISAM，从5.5以后，InnoDB就作为了默认的存储引擎。

在实际应用开发中，我们基本上都是采用InnoDB引擎。

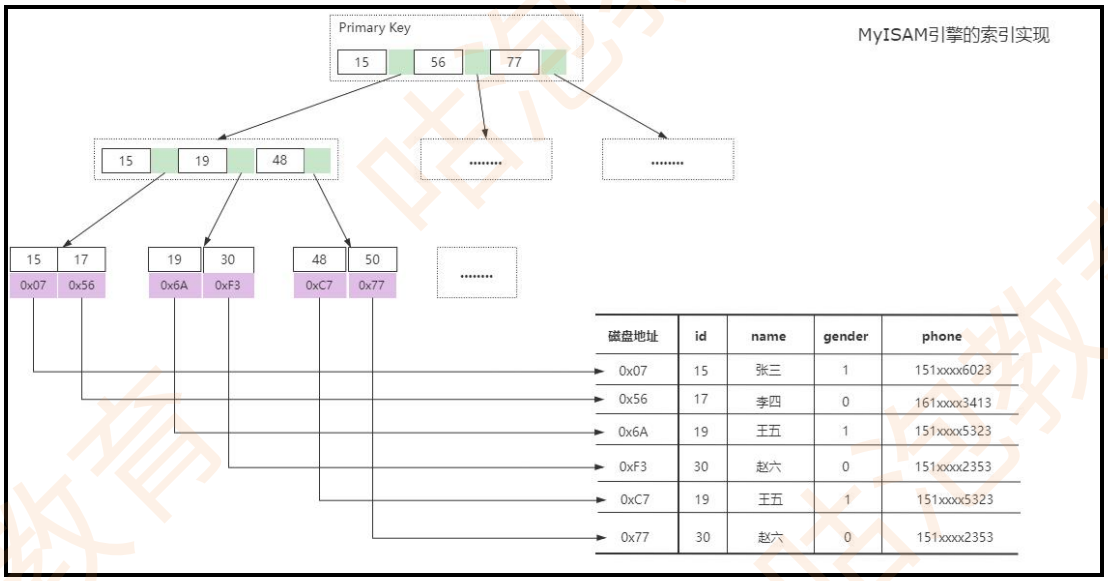
我们先来看一下MyISAM引擎。

MyISAM引擎的数据是通过二进制的方式存储在磁盘上，它在磁盘上体现为两个文件

一个是.MYD文件，D代表Data，是MyISAM的数据文件，存放数据记录，

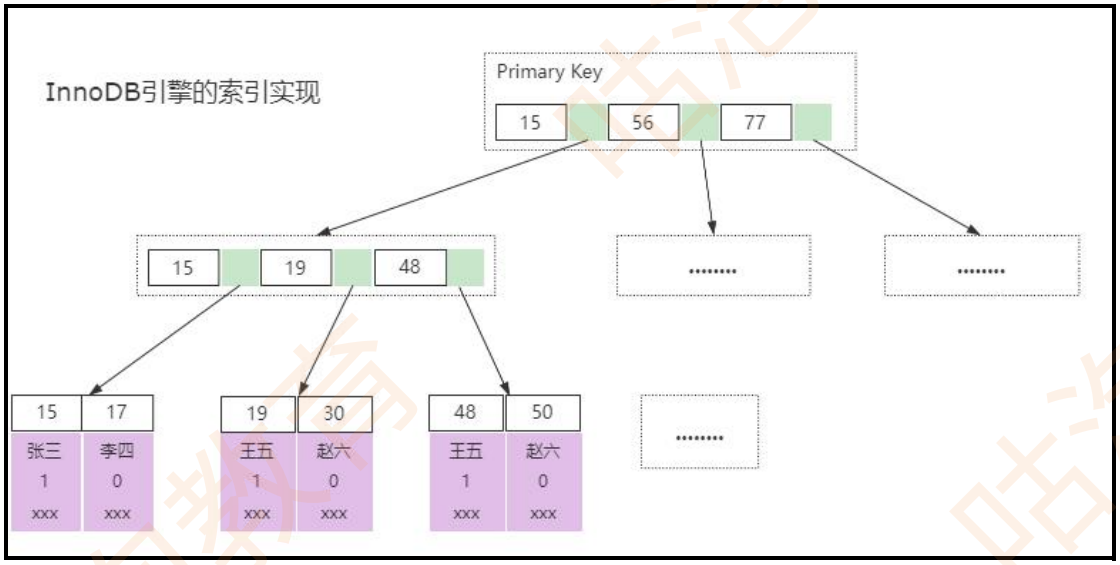
一个是.MYI文件，I代表Index，是MyISAM的索引文件，存放索引实现机制如图所示。

因为索引和数据是分离的，所以在进行查找的时候，先从索引文件中找到数据的磁盘位置，再到数据文件中找到索引对应的数据内容。



在InnoDB存储引擎中，数据同样存储在磁盘上，它在磁盘上只有一个ibd文件，里面包含索引和数据。

它的整体结构如图所示，在B+树的叶子节点里面存储了索引对应的数据，在通过索引进行检索的时候，命中叶子节点，就可以直接从叶子节点中取出行数据。



了解了这两个存储引擎以后，我们在面试的时候该怎么回答呢?

高手回答

基于我的理解，我认为MyISAm和InnoDB的区别有4个，第一个，数据存储的方式不同，MyISAM中的数据和索引是分开存储的，而InnoDB是把索引和数据存储在同一个文件里面。

第二个，对于事务的支持不同，MyISAM不支持事务，而InnoDB支持ACID特性的事务处理

第三个，对于锁的支持不同，MyISAM只支持表锁，而InnoDB可以根据不同的情况，支持行锁，表锁，间隙锁，临键锁

第四个，MyISAM不支持外键，InnoDB支持外键因此基于这些特性，我们在实际应用中，可以根据不同的场景来选择合适的存储引擎。

比如如果需要支持事务，那必须要选择InnoDB。

如果大部分的表操作都是查询，可以选择MyISAM。

以上就是我的理解。

总结

下次面试的时候遇到这个问题，大家知道怎么回答了吗?

如果你喜欢我的作品，记得点赞收藏加关注哦

我是Mic，咱们下期再见。

TOM老师解析

你都工作3年了，怎么还不知道MyISAM和InnoDB有什么区别？一位粉丝被面试官这样一个问题。当时，这位粉丝直接回复“不知道”，这位粉丝自己也怪不好意思的。

其实，面试官问他这个问题也不是随便问的，也是看了这位粉丝简历上写了熟练MySQL才提问

的。其实，对于MySQL，这位粉丝可能只是非常熟练地完成建库、建表、往数据库表中读写数据这些简单的操作而已，并没有去了解建表的原理。

这个面试题也给我们敲了一下警钟，对于有些技术的知识点，要是没有掌握到一定深度，建议不要写熟练之类的，如果真要写，建议用了解或者熟悉这样的字眼来描述，避免面试官问起时又答不出来，暴露了自己的不足。

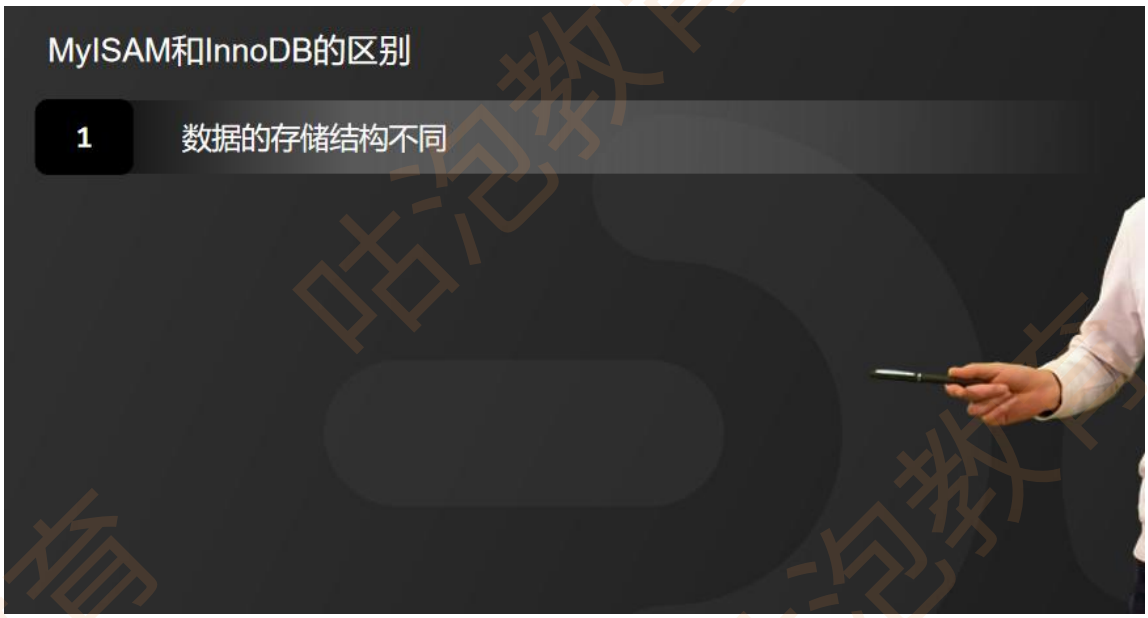
另外，我花了1个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

MyISAM和InnoDB到底有什么区别呢?

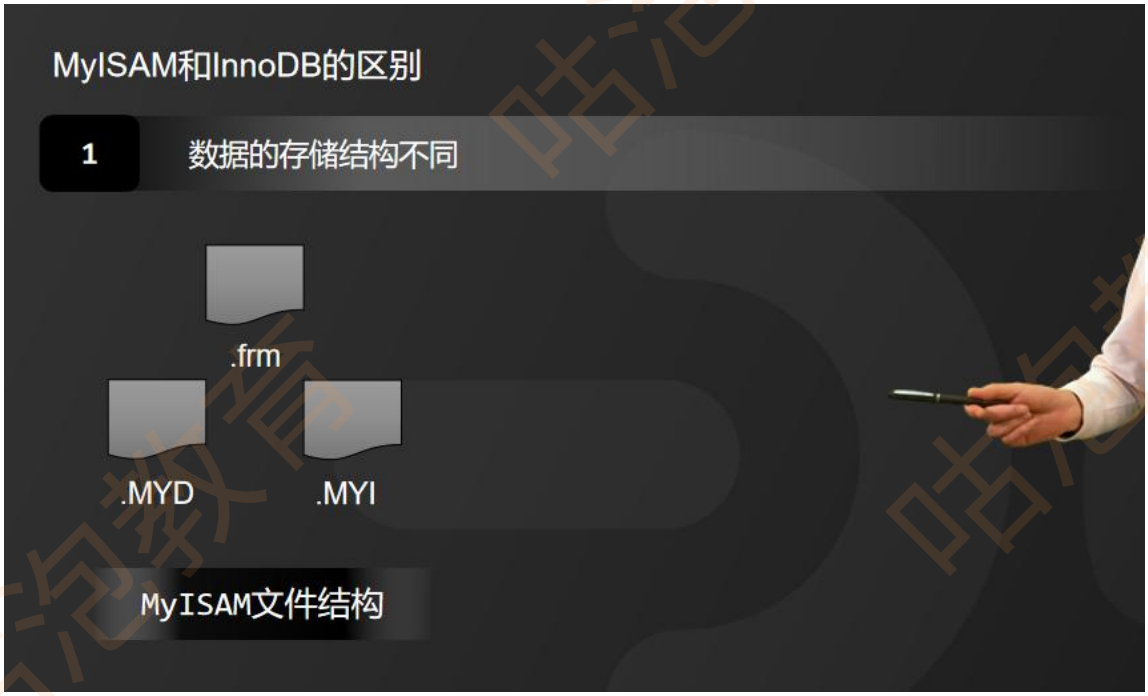
1、MyISAM和InnoDB的区别

MySQL 5.5以后的版本开始将InnoDB作为默认的存储引擎，之前的版本都是MyISAM。关于MyISAM和InnoDB的区别，我总结为以下5个方面，希望能帮助到大家。

1）. 数据的存储结构不同



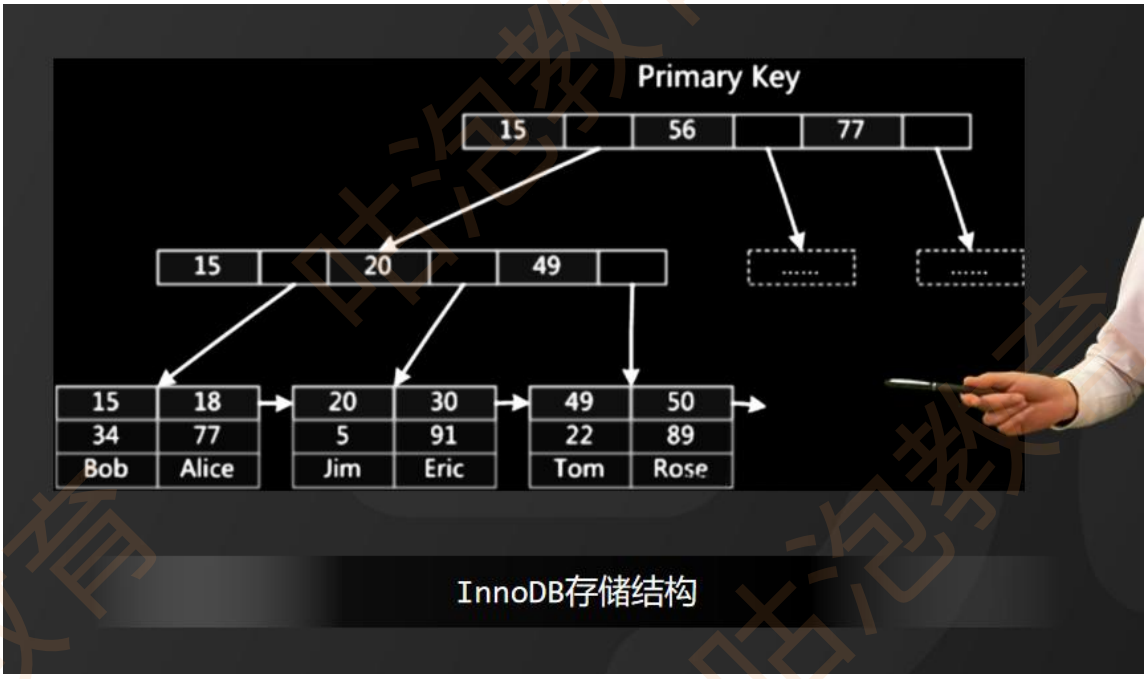
先来看MyISAM，每个MyISAM在磁盘上存储成三个文件，它们以表的名字开头来命名。.frm文件存储表定义。.MYD(MYD)存储数据文件。.MYI(MYIndex)存储索引文件。



而 InnoDB在磁盘上保存为两个文件。.frm文件同样存储为表结构文件，.ibd文件存储的是数据和索引文件。

由于MyISAM的索引和数据是分开存储的，因此索引查找的时候，MyISAM的叶子节点存储的是数据所在的地址，而不是数据。

而InnoDB叶子节点存储的是整个数据行所有的数据。



2）. 存储空间的消耗不同

MyISAM可被压缩，存储空间较小。支持三种不同的存储格式：静态表（默认，但是注意数据末尾不能有空格，会被去掉）、动态表、压缩表。

InnoDB需要更多的内存和存储，它会在主内存中建立其专用的缓冲池用于高速缓冲数据和索引。InnoDB所在的表都保存在同一个数据文件中（也可能是多个文件，或者是独立的表空间），InnoDB表的大小只受限于操作系统文件的大小，一般为2GB。

3）. 对事务的支持不同

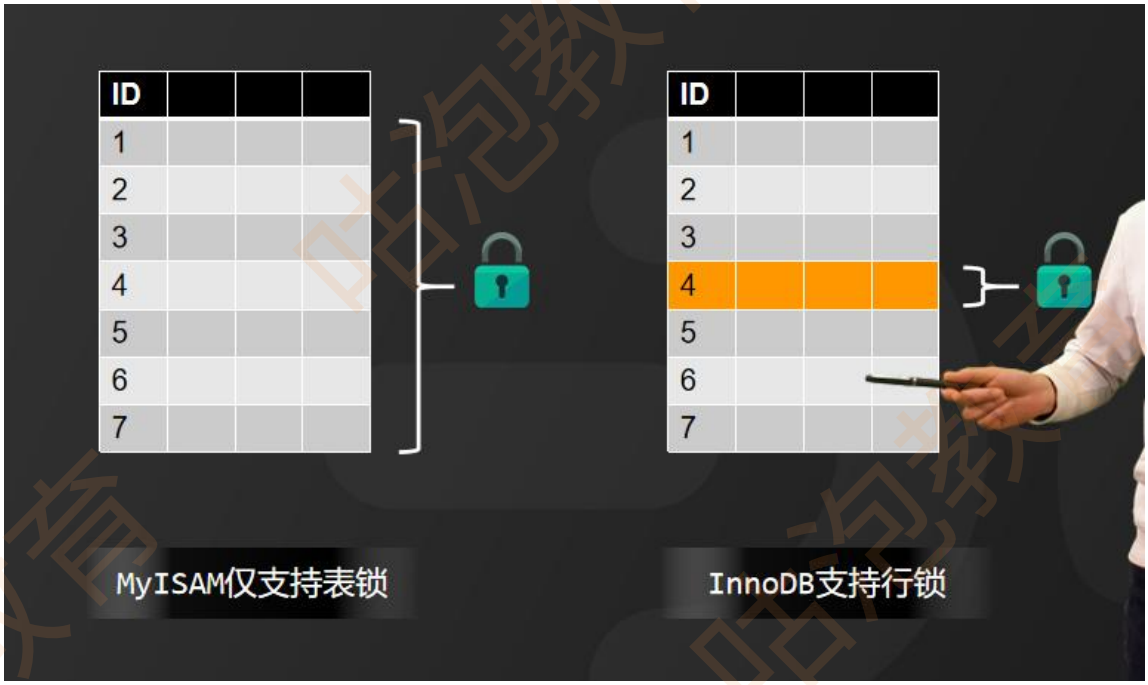
MyISAM强调的是性能，每次查询具有原子性，其执行速度比Innodb类型更快，但是不提供事务支持。

InnoDB除了提供事务支持和外部键等高级数据库功能。还具有事务提交（commit）、回滚（rollback）和崩溃修复能力（crach recovery capabilities）等这些事务安全（transaction-safe ACID compliant）型表。

4）. 对锁的支持不同

如果只是执行大量的查询, MyISAM是更好的选择。MyISAM在增删的时候需要锁定整个表格，效率会低一些。

而innoDB支持行级锁，删除插入的时候只需要锁定操作行就行。如果有大量的插入、修改删除操作，使用InnoDB性能能会更高。



5）. 对外键的支持不同

MyISAM不支持外键，而InoDB支持外键。当然，各种不同MySQL版本对两者的支持都有所改进

【导航条：总结及建议】

2、总结及建议

最后总结一下，MyISAM和InnoDB的主要区别包括

1）. 数据的存储结构不同

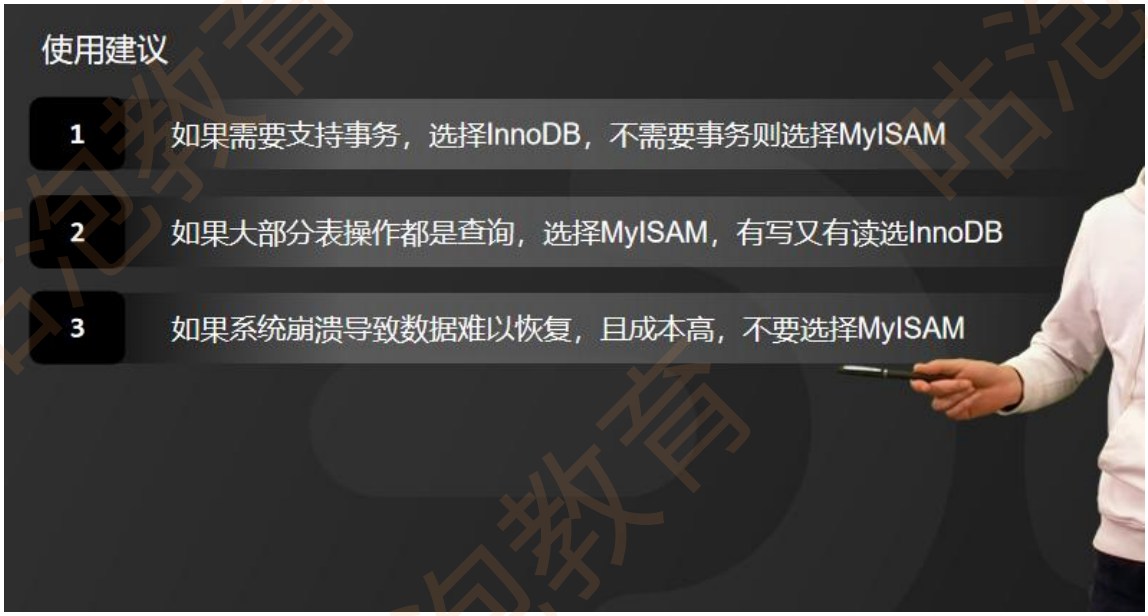
2）. 存储空间的消耗不同

3）. 对事务的支持不同

4）. 对锁的支持不同

5）. 对外键的支持不同

MyISAM和InnoDB的区别还有很多，我这里只是列举了比较重要的几点。那我们在实际开发中，该如何选择MyISAM和InnoDB呢？我总结为以下三点：



1、如果需要支持事务，选择InnoDB，不需要事务则选择MyISAM。

2、如果大部分表操作都是查询，选择MyISAM，有写又有读选InnoDB。

3、如果系统崩溃导致数据难以恢复，且成本高，不要选择MyISAM。

以上就是我对MyISAM和InnoDB区别的总结和理解，下次面试你会了吗？

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

1.6. 数据库索引的原理，为什么要用 B+树，为什么不用二叉树？

MIC老师解析

一个工作8年的粉丝私信了我一个问题。

他说这个问题是去阿里面试的时候被问到的，自己查了很多资料也没搞明白，希望我帮他解答。

问题是： “Mysql为什么使用B+Tree作为索引结构”

关于这个问题，看看普通人和高手的回答。

普通人

B+树非叶子节点上是不存储数据的，仅存储键值，而 B 树节点中不仅存储键值，也会存储数据。innodb 中页的默认大小是 16KB，如果不存储数据，那么就会存储更多的键值，相应的树的阶数（节点的子节点树）就会更大，树就会更矮更胖，如此一来我们查找数据进行磁盘的 IO 次数有会再次减少，数据查询的效率也会更快。

B+树索引的所有数据均存储在叶子节点，而且数据是按照顺序排列的，链表连着的。那么 B+树使得范围查找，排序查找，分组查找以及去重查找变得异常简单。

高手

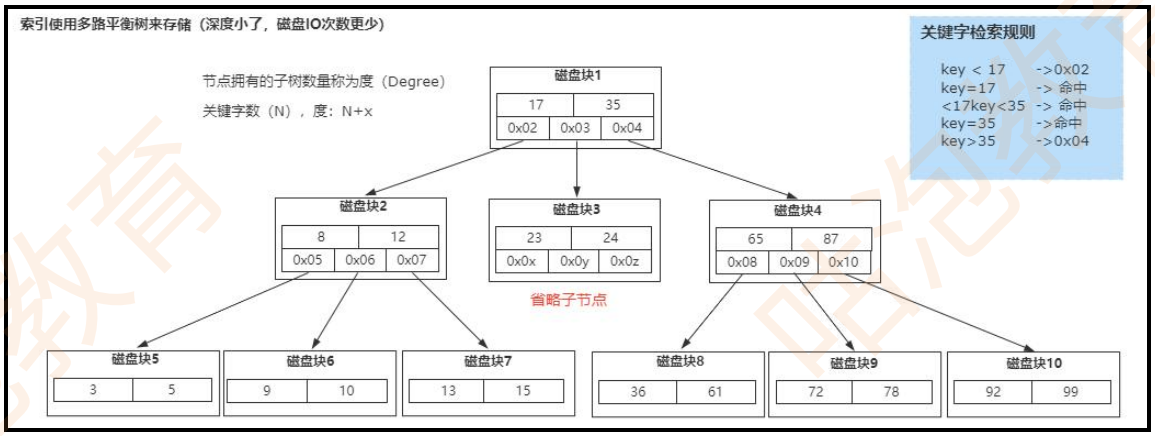
关于这个问题，我从几个方面来回答。

首先，常规的数据库存储引擎，一般都是采用B树或者B+树来实现索引的存储。

因为B树是一种多路平衡树，用这种存储结构来存储大量数据，它的整个高度会相比二叉树来说，会矮很多。

而对于数据库来说，所有的数据必然都是存储在磁盘上的，而磁盘IO的效率实际上是很低的，特别是在随机磁盘IO的情况下效率更低。

所以树的高度能够决定磁盘IO的次数，磁盘IO次数越少，对于性能的提升就越大，这也是为什么采用B树作为索引存储结构的原因。

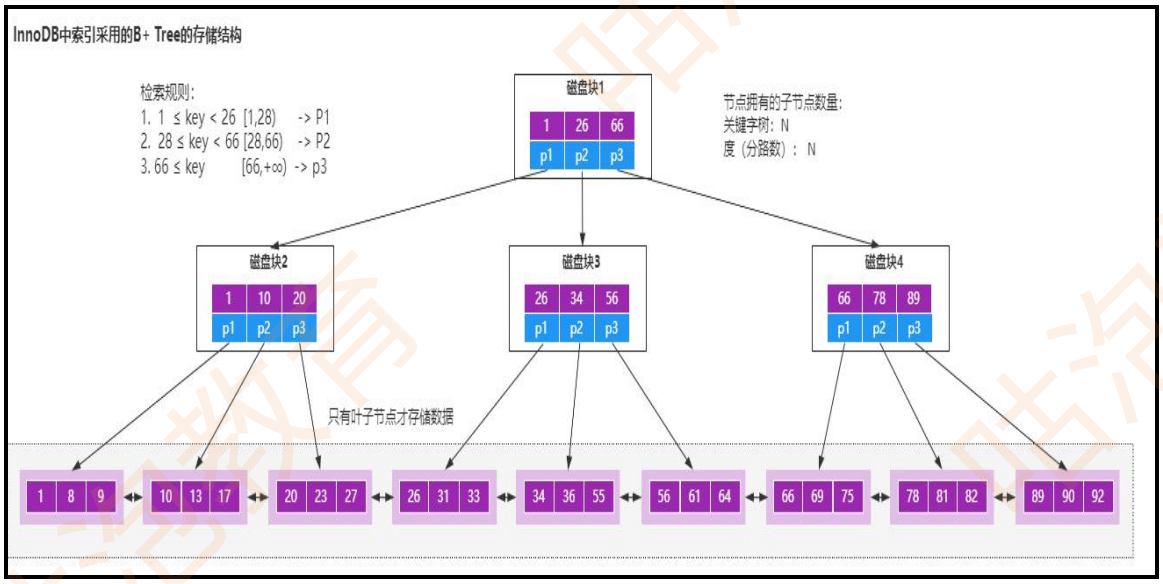


但是在Mysql的InnoDB存储引擎里面，它用了一种增强的B树结构，也就是B+树来作为索引和数据的存储结构。

相比较于B树结构，B+树做了几个方面的优化。

B+树的所有数据都存储在叶子节点，非叶子节点只存储索引。

叶子节点中的数据使用双向链表的方式进行关联。



使用B+树来实现索引的原因，我认为有几个方面。

B+树非叶子节点不存储数据，所以每一层能够存储的索引数量会增加，意味着B+树在层高相同的情况下存储的数据量要比B树要多，使得磁盘IO次数更少。

在Mysql里面，范围查询是一个比较常用的操作，而B+树的所有存储在叶子节点的数据使用了双向链表来关联，所以在查询的时候只需查两个节点进行遍历就行，而B树需要获取所有节点，所以B+树在范围查询上效率更高。

在数据检索方面，由于所有的数据都存储在叶子节点，所以B+树的IO次数会更加稳定一些。

因为叶子节点存储所有数据，所以B+树的全局扫描能力更强一些，因为它只需要扫描叶子节点。但是B树需要遍历整个树。

另外，基于B+树这样一种结构，如果采用自增的整型数据作为主键，还能更好的避免增加数据的时候，带来叶子节点分裂导致的大量运算的问题。

总的来说，我认为技术方案的选型，更多的是去解决当前场景下的特定问题，并不一定是说B+树就是最好的选择，就像MongoDB里面采用B树结构，本质上来说，其实是关系型数据库和非关系型数据库的差异。

以上就是我对这个问题的理解。

面试点评

对于“为什么要选择xx技术”的问题，其实很好回答。

只要你对这个技术本身的特性足够了解，那么自然就知道为什么要这么设计。

就像，我们在业务开发中，知道什么时候使用List，什么时候使用Map，道理是一样的。

好的，本期的普通人VS高手面试系列的视频就到这里结束了。

如果有任何面试问题、职业发展问题、学习问题，都可以私信我。

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

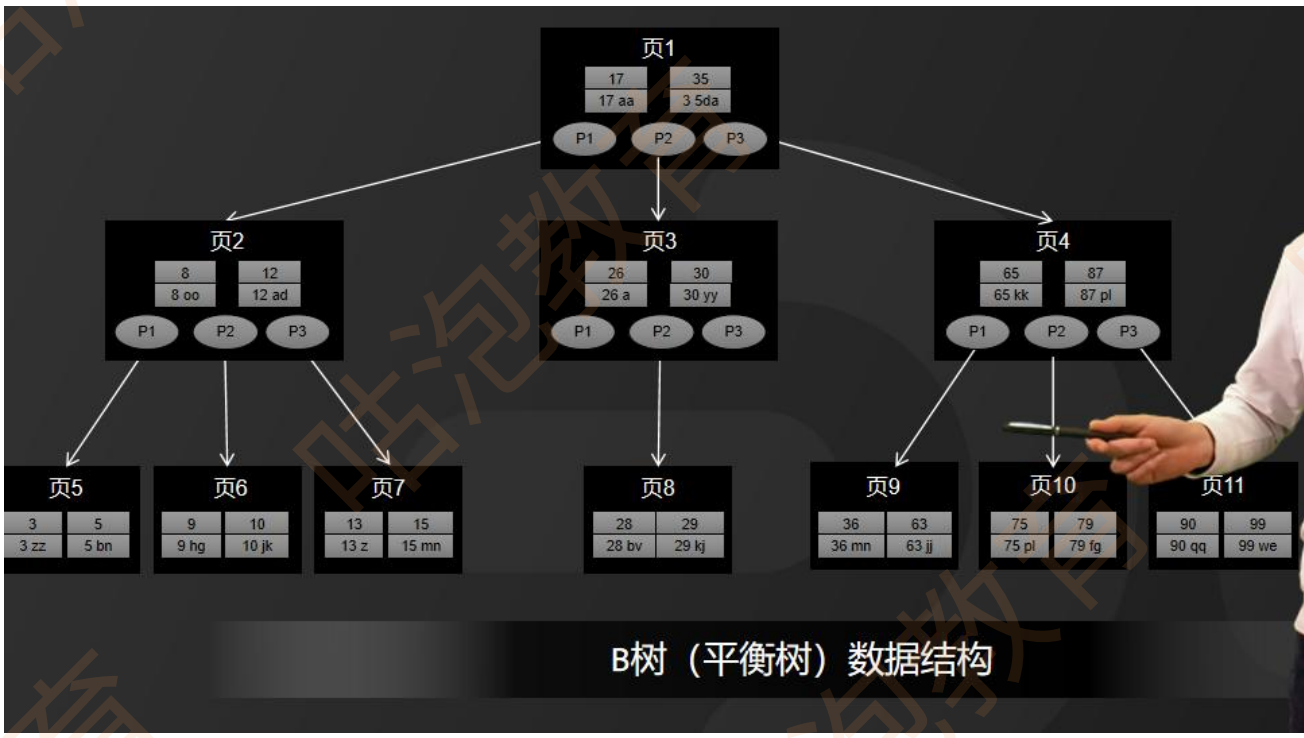
TOM老师解析

一位6年经验的小伙伴去字节面试的时候被问到这样一个问题，为什么MySQL索引结构要采用B+树？这位小伙伴从来就没有思考过这个问题。只因为现在都这么卷，后面还特意查了很多资料，他也希望听听我的见解。

另外，我花了1个多星期把往期的面试题解析配套文档准备好了，一共有10万字，想获取的小伙伴可以在我的煮叶简介中找到。

1、B树和B+树

一般来说，数据库的存储引擎都是采用B树或者B+树来实现索引的存储。首先来看B树，如图所示。



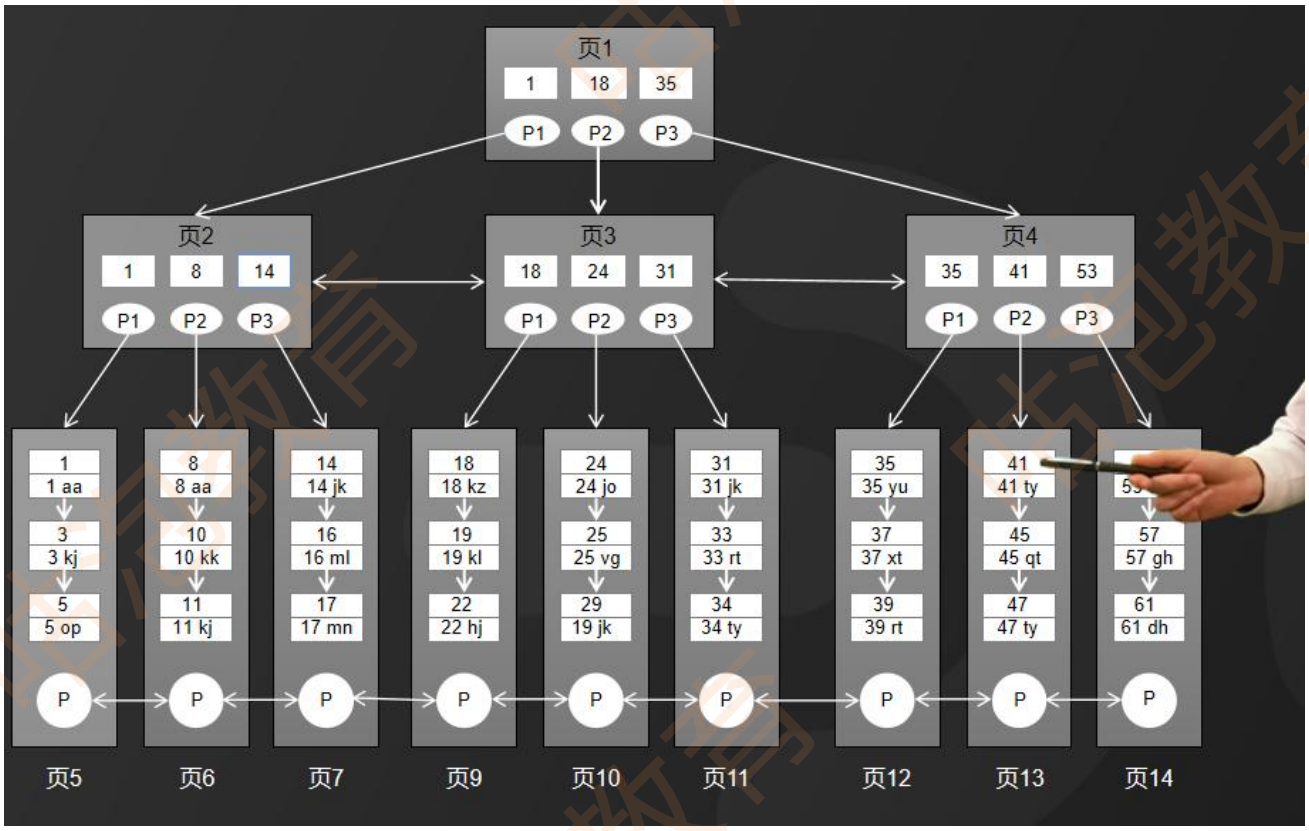
B树是一种多路平衡树，用这种存储结构来存储大量数据，它的整个高度会相比二叉树来说，会矮很多。

而对于数据库而言，所有的数据都将会保存到磁盘上，而磁盘I/O的效率又比较低，特别是在随机磁盘I/O的情况下效率更低。

所以 高度决定了磁盘I/O的次数，磁盘I/O次数越少，对于性能的提升就越大，这也是为什么采用B树作为索引存储结构的原因，如图所示。

而MySQL的InnoDB存储引擎，它用了一种增强的B树结构，也就是B+树来作为索引和数据的存储结构。

相比较于B树结构来说，B+树做了两个方面的优化，如图所示。



1、B+树的所有数据都存储在叶子节点，非叶子节点只存储索引。

2、叶子节点中的数据使用双向链表的方式进行关联。

2、原因分析

我认为，MySQL索引结构采用B+树，有以下4个原因：

1、从磁盘I/O效率方面来看：B+树的非叶子节点不存储数据，所以树的每一层就能够存储更多的索引数量，也就是说，B+树在层高相同的情况下，比B树的存储数据量更多，间接会减少磁盘I/O的次数。

2、从范围查询效率方面来看：在MySQL中，范围查询是一个比较常用的操作，而B+树的所有存储在叶子节点的数据使用了双向链表来关联，所以B+树在查询的时候只需查两个节点进行遍历就行，而B树需要获取所有节点，因此，B+树在范围查询上效率更高。

3、从全表扫描方面来看：因为，B+树的叶子节点存储所有数据，所以B+树的全局扫描能力更强一些，因为它只需要扫描叶子节点。而B树需要遍历整个树。

4、从自增ID方面来看：基于B+树的这样一种数据结构，如果采用自增的整型数据作为主键，还能更好的避免增加数据的时候，带来叶子节点分裂导致的大量运算的问题。

3、总结

总体来说，我认为技术方案的选型，更多的要根据具体的业务场景来决定，并不一定是说B+树就是最好的选择，就像MongoDB里面采用B树结构，本质上来说，其实是关系型数据库和非关系型数据库的差异。

【结尾】

以上就是我对为什么MySQL索引结构采用B+树 的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

1.7. 聚集索引与非聚集索引的区别

MIC老师解析

一个去阿里面试并且第一面就挂了的粉丝私信我，被数据库里面几个问题难倒了，他说面试官问了事务隔离级别、MVCC、聚集索引/非聚集索引、B树、B+树这些，没回答好。

大厂面试基本上是这样，由点到面去展开，如果你对这个技术理解不够全面，很容易就会被看出来。

ok，关于“什么是聚集索引和非聚集索引”这个问题，看看普通人和高手的回答。

普通人

嗯，聚集索引就是通过主键来构建的索引结构。

而非聚集索引就是除了主键以外的其他索引。

高手

简单来说，聚集索引就是基于主键创建的索引，除了主键索引以外的其他索引，称为非聚集索引，也叫做二级索引。

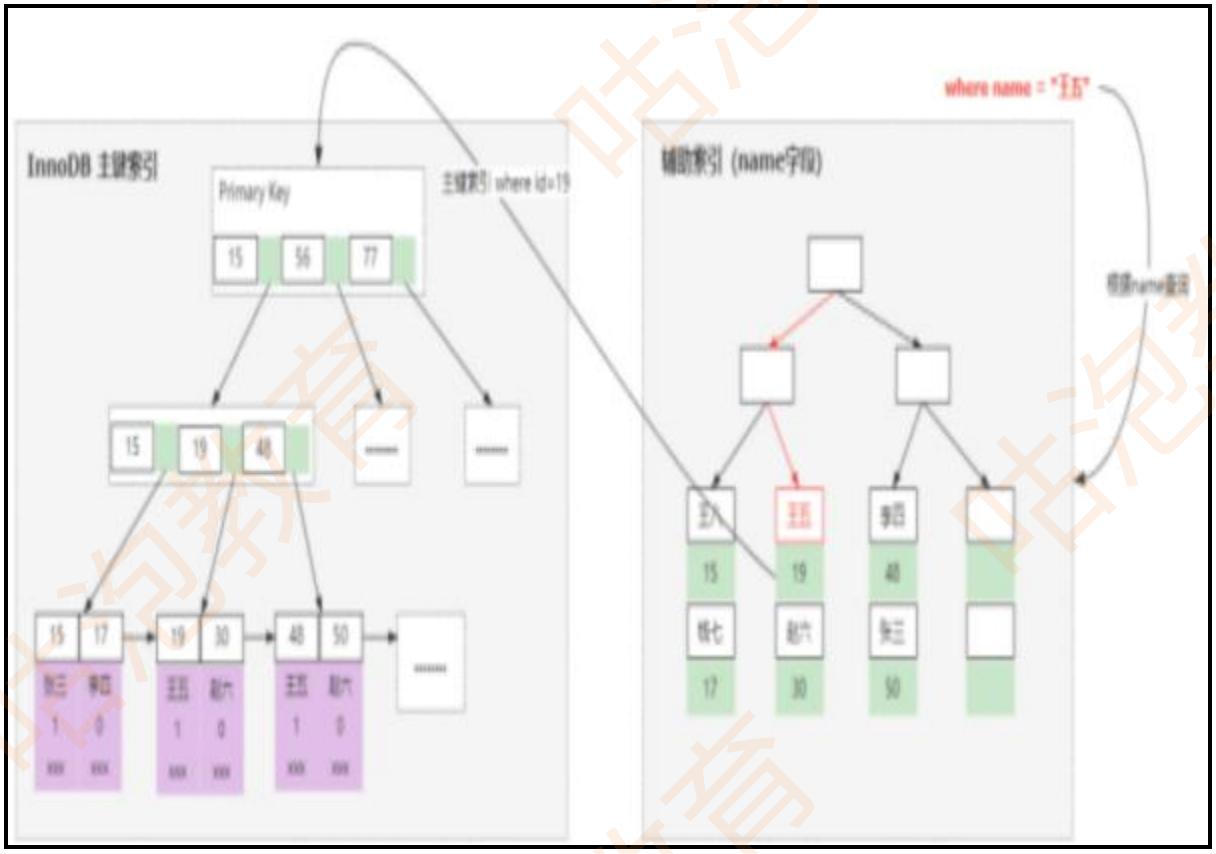
由于在InnoDB引擎里面，一张表的数据对应的物理文件本身就是按照B+树来组织的一种索引结构，而聚集索引就是按照每张表的主键来构建一颗B+树，然后叶子节点里面存储了这个表的每一行数据记录。

所以基于InnoDB这样的特性，聚集索引并不仅仅是一种索引类型，还代表着一种数据的存储方式。

同时也意味着每个表里面必须要有一个主键，如果没有主键，InnoDB会默认选择或者添加一个隐藏列作为主键索引来存储这个表的数据行。一般情况是建议使用自增id作为主键，这样的话id本身具有连续性使得对应的数据也会按照顺序存储在磁盘上，写入性能和检索性能都很高。否则，如果使用uuid这种随机id，那么在频繁插入数据的时候，就会导致随机磁盘IO，从而导致性能较低。

需要注意的是，InnoDB里面只能存在一个聚集索引，原因很简单，如果存在多个聚集索引，那么意味着这个表里面的数据存在多个副本，造成磁盘空间的浪费，以及数据维护的困难。

由于在InnoDB里面，主键索引表示的是一种数据存储结构，所以如果是基于非聚集索引来查询一条完整的记录，最终还是需要访问主键索引来检索。



面试点评

这个问题要回答好，还真不容易。涉及到Mysql里面索引的实现原理。

但是如果回答好了，就能够很好的反馈求职者的技术功底，那通过面试就比较容易了。

好的，本期的普通人VS高手面试系列的视频就到这里结束了，喜欢的朋友记得点赞和收藏。

另外，这些面试题我都整理成了笔记，大家有需要的可以私信获取。

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

TOM老师解析

一位去阿里面试的小伙伴，在第一面就挂了。他跟我说是被数据库里面的几个问题难倒了，他说面试官问了事务隔离级别、MVCC、聚集索引/非聚集索引、B 树、B+树这些，都没回答好。

大厂面试基本上是这样，由点到面去展开，如果你对这个技术理解不够全面，很容易就会被看出来。关于事务隔离级别、MVCC、B树、B+树在往期视频中我都有讲解过。那今天，我给大家来聊一聊聚集索引和非聚集索引。

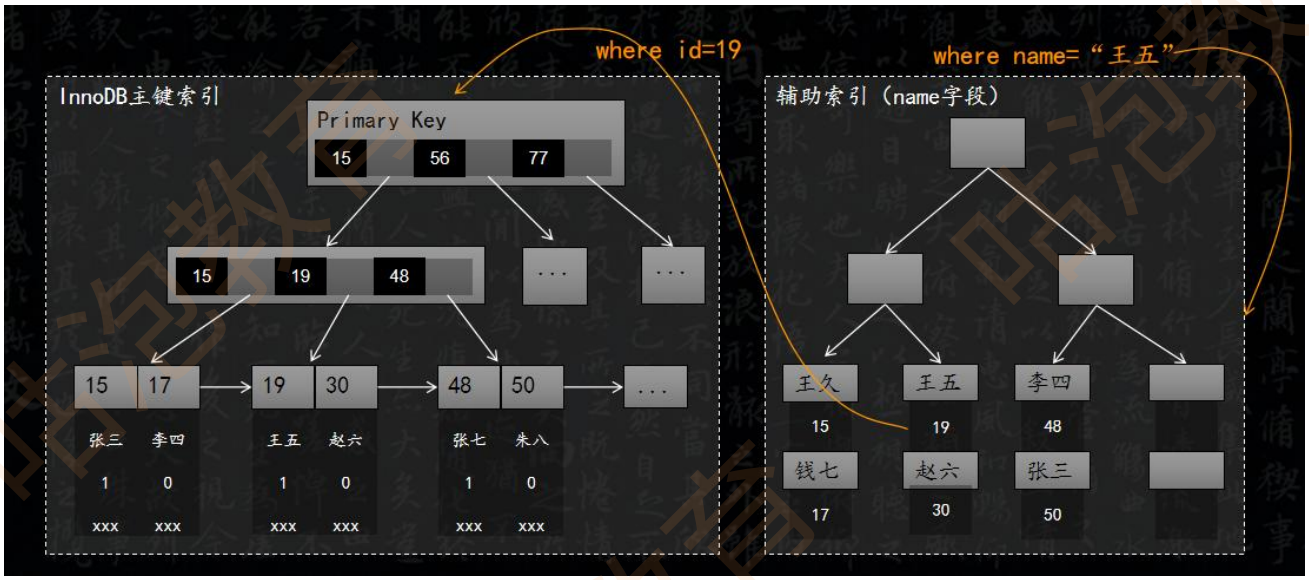
另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【回答】

简单来说，聚集索引就是基于主键创建的索引，除了主键索引以外的其他索引，称为非聚集索

引，也叫做二级索引。

由于在 InnoDB 引擎里面，一张表的数据对应的物理文件本身就是按照 B+树来组织的一种索引结构，而聚集索引就是按照每张表的主键来构建一颗 B+树，然后叶子节点里面存储了这个表的每一行数据记录。所以基于 InnoDB 这样的特性，聚集索引并不仅仅是一种索引类型，还代表着一种数据的存储方式。



同时也意味着每个表里面必须要有一个主键，如果没有主键，InnoDB 会默认选择或者添加一个隐藏列作为主键索引来存储这个表的数据行。一般情况是建议使用自增 ID 作为主键，这样的话 ID 本身具有连续性，使得对应的数据也会按照顺序存储在磁盘上，写入性能和检索性能都很高。否则，如果使用 UUID 这种随机 ID，那么在频繁插入数据的时候，就会导致随机磁盘 IO，从而导致性能较低。需要注意的是，InnoDB 里面只能存在一个聚集索引，原因很简单，如果存在多个聚集索引，那么意味着这个表里面的数据存在多个副本，造成磁盘空间的浪费，以及数据维护的困难。

由于在 InnoDB 里面，主键索引表示的是一种数据存储结构，所以如果是基于非聚集索引来查询一条完整的记录，最终还是需要访问主键索引来检索。

【结尾】

这个问题要回答好，还真不容易。涉及到 MySQL里面索引的实现原理。但是如果回答好了，就能够很好的反馈求职者的技术功底，那通过面试就比较容易了。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

1.8. limit 1000000 加载很慢的话，你是怎么解决的呢?

方案一：如果 id 是连续的，可以这样，返回上次查询的最大记录(偏移量)，再往下 limit

```
select id，name from employee where id>1000000 limit 10.
```

方案二：在业务允许的情况下限制页数：

建议跟业务讨论，有没有必要查这么后的分页啦。因为绝大多数用户都不会往后翻太多页。

方案三：order by + 索引（id 为索引）

```
select id，name from employee order by id   limit 1000000，10
```

方案四：利用延迟关联或者子查询优化超多分页场景。（先快速定位需要获取的 id 段，然后再关联）

```
SELECT a.* FROM employee a, (select id from employee where 条件 LIMIT 1000000,10 ) bwhere a.id=b.id
```

灰灰老师解析

面试官：假如我Myql的表很大，进行分页的时候，limit 1000000 加载很慢的话，你是怎么解决的呢?

灰灰：好的，面试官，这个问题在生产环境也是比较常见的。

limit m，n ;其实去扫描m+n条数据，然后过滤掉前面的m条数据，当m越大，那么需要扫描的数据也就越多，性能也会越来越慢。

针对这种情况，有以下几种方案可以进行一定的优化。

1.如果id是趋势递增的，那么每次查询都可以返回这次查询最大的ID，然后下次查询，加上大于 上次最大id的条件，这样会通过主键索引去扫描，并且扫描数量会少很多很多。因为只需要扫描where条件的数据

2.先limit出来主键ID，然后用主表跟查询出来的ID进行inner join 内连接，这样，也能一定上提速，因为减少了回表，查询ID只需要走聚集索引就行。

3.当然，如果mysql级别优化不了了。我们也可以对分页数据进行缓存，比如Redis缓存，数据进行变动的时候，做好缓存依赖即可。

4.因为越往后，一般用户行为触及不到，比如你去看淘宝，不会去翻后面几百页的数据，所以，业务层面也可以做一些让步，比如不做后面几百页的数据。

1.9. 如何选择合适的分布式主键方案呢?

- 数据库自增长序列或字段。
- UUID。
- Redis 生成 ID
- Twitter 的 snowflake 算法
- 利用 zookeeper 生成唯一 ID
- MongoDB 的 ObjectId

1.10. 事务的隔离级别有哪些？MySQL 的默认隔离级别是什么?

MIC老师解析

一个工作了6年的粉丝，去阿里面试，在第一面的时候被问到”Mysql的事务隔离级别“。

他竟然没有回答上来，一直在私信向我诉苦。

我说，你只能怪年轻时候的你，那个时候不够努力导致现在的你技术水平不够。

好吧，关于这个问题，看看普通人和高手的回答。

普通人

读未提交（Read Uncommitted）

读已提交（Read Committed）

可重复读（Repeatable Read）

串行化（Serializable）

Mysql 默认的事务隔离级别是可重复读(Repeatable Read)

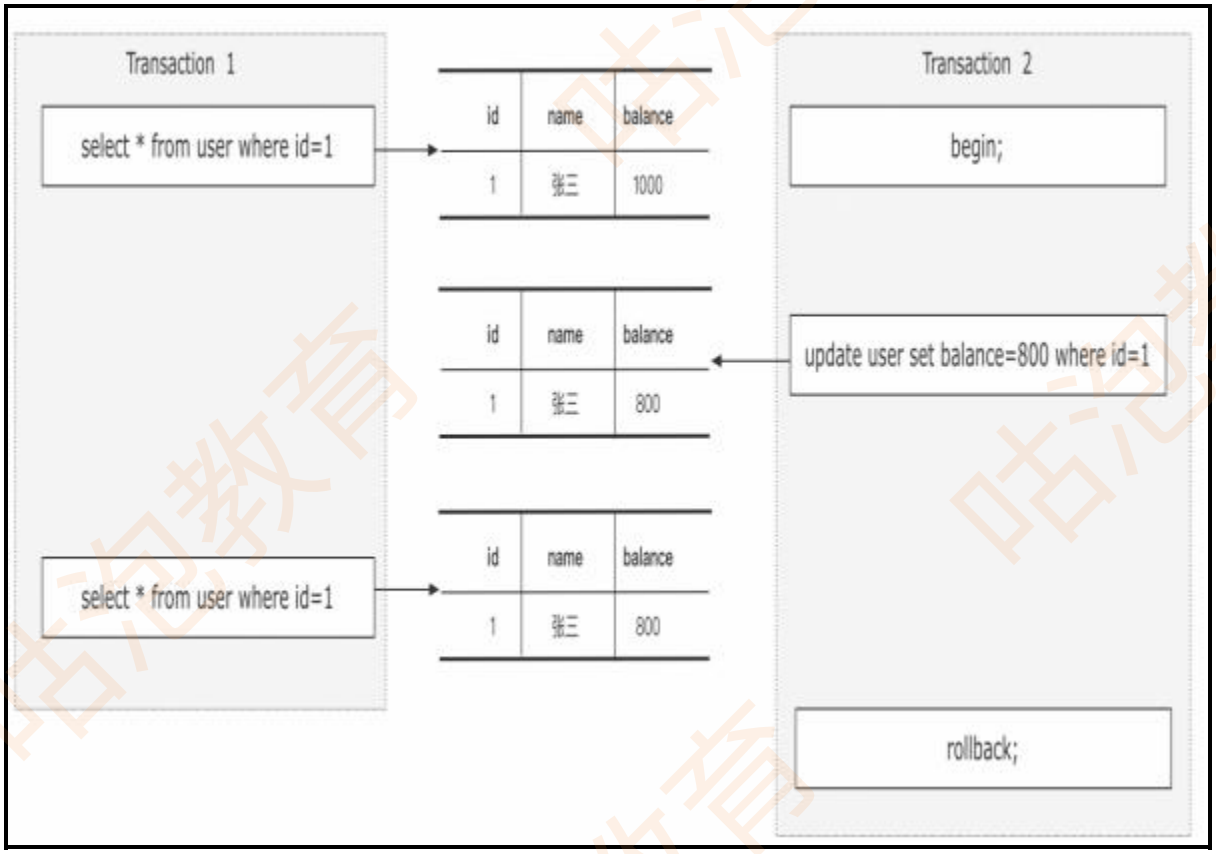
高手

好的，关于这个问题，我会从几个方面来回答。

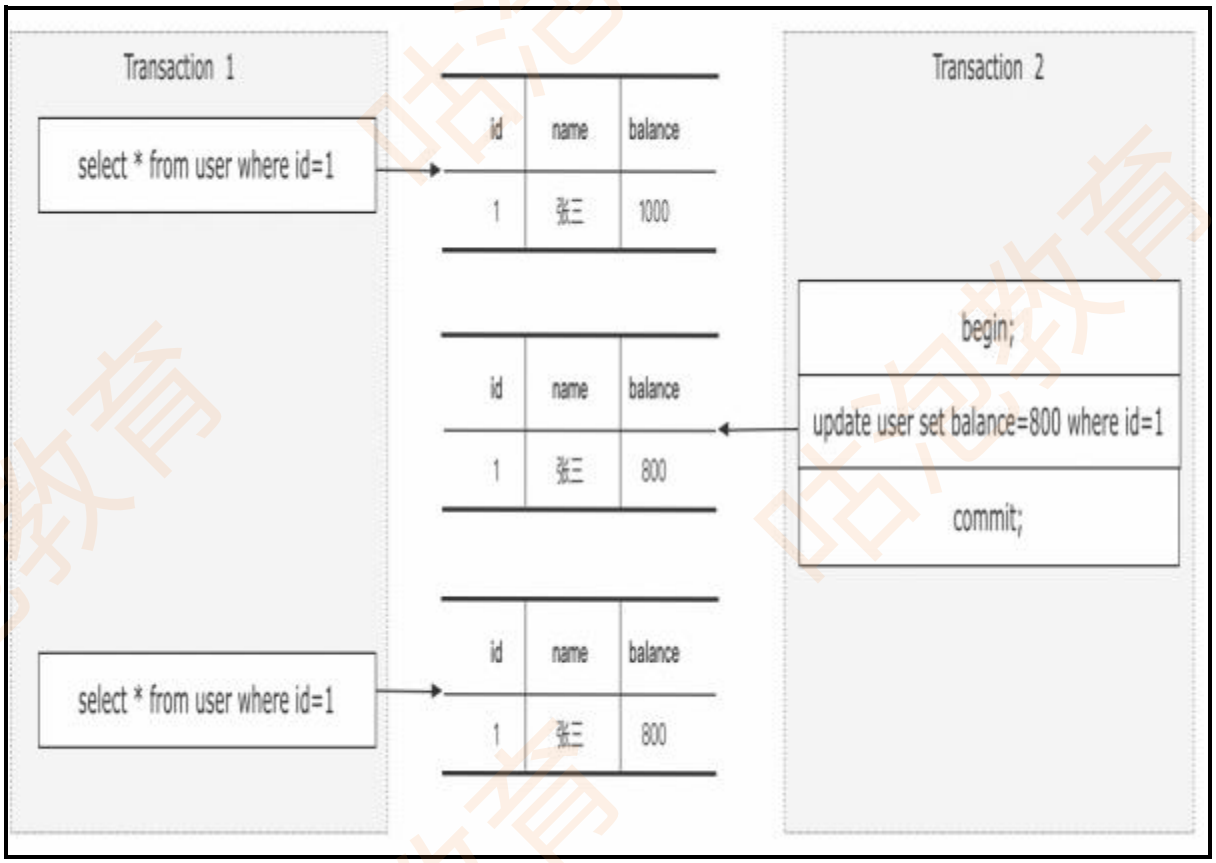
首先，事务隔离级别，是为了解决多个并行事务竞争导致的数据安全问题的一种规范。

具体来说，多个事务竞争可能会产生三种不同的现象。

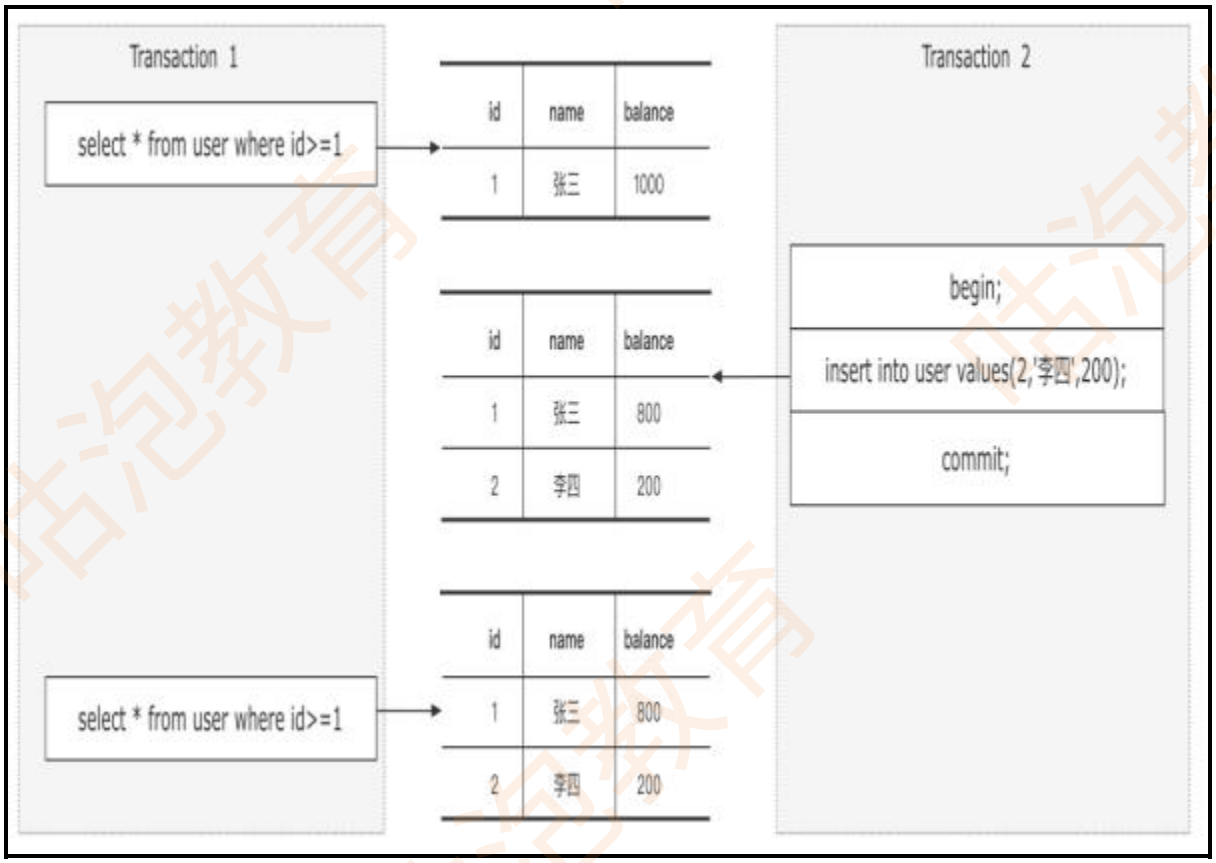
假设有两个事务T1/T2同时在执行，T1事务有可能会读取到T2事务未提交的数据，但是未提交的事务T2可能会回滚，也就导致了T1事务读取到最终不一定存在的数据产生脏读的现象。



假设有两个事务T1/T2同时执行，事务T1在不同的时刻读取同一行数据的时候结果可能不一样，从而导致不可重复读的问题。



假设有两个事务T1/T2同时执行，事务T1执行范围查询或者范围修改的过程中，事务T2插入了一条属于事务T1范围内的数据并且提交了，这时候在事务T1查询发现多出来了一条数据，或者在T1事务发现这条数据没有被修改，看起来像是产生了幻觉，这种现象称为幻读。



而这三种现象在实际应用中，可能有些场景不能接受某些现象的存在，所以在SQL标准中定义了四种隔离级别，分别是：

读未提交，在这种隔离级别下，可能会产生脏读、不可重复读、幻读。

读已提交（RC），在这种隔离级别下，可能会产生不可重复读和幻读。

可重复读（RR），在这种隔离级别下，可能会产生幻读

串行化，在这种隔离级别下，多个并行事务串行化执行，不会产生安全性问题。

这四种隔离级别里面，只有串行化解决了全部的问题，但也意味着这种隔离级别的性能是最低的。

在Mysql里面，InnoDB引擎默认的隔离级别是RR（可重复读），因为它需要保证事务ACID特性中的隔离性特征。

以上就是我对这个问题的理解。

面试点评

关于这个问题，很多用Mysql5年甚至更长时间的程序员都不一定非常清楚的知道。

这其实是不正常的，因为虽然InnoDB默认隔离级别能解决99%以上的问题，但是有些公司的某些业务可能会修改隔离级别。

而如果你不知道，就很可能在程序中出现莫名其妙的问题。

好的，本期的普通人VS高手面试系列的视频就到这里结束了。

如果有任何面试问题、职业发展问题、学习问题，都可以私信我。

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

TOM老师解析

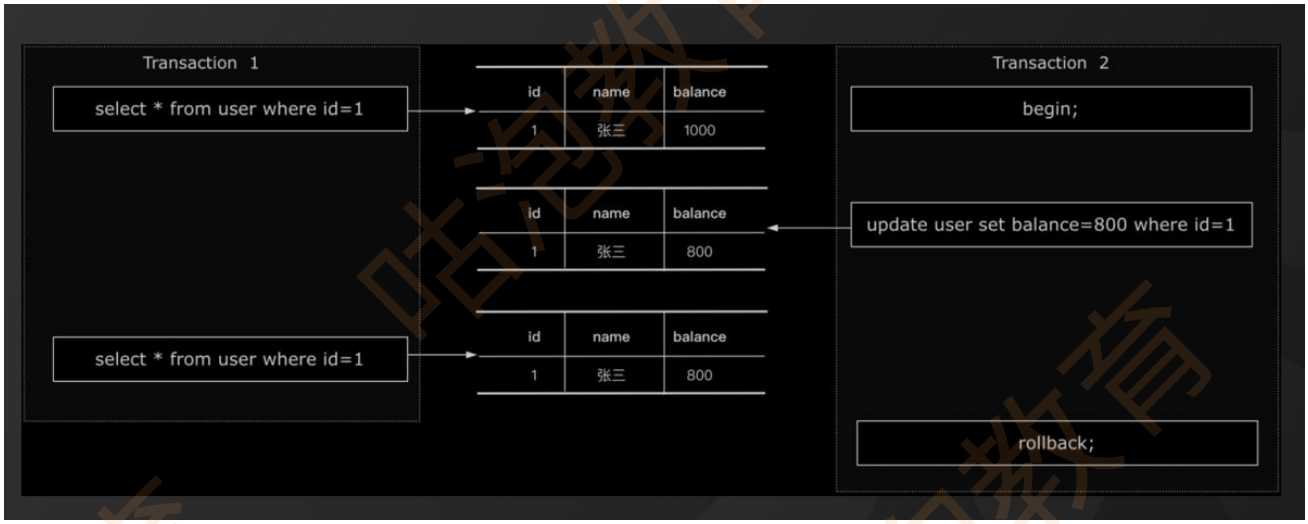
一位5年工作经验的粉丝，去阿里面试被问到一个关于数据库事务隔离级别的问题，当时，没有问答上来，希望给他一个参考答案。那么，今天我给大家谈谈我的理解。

另外，我花了1个多星期把往期的面试题解析配套文档准备好了，一共有10W字，想获取的小伙伴可以从我的个人煮叶简介中找到。

1、脏读、幻读、不可重复读

在SQL操作中，多个事务竞争可能会产生三种不同的现象，分别是脏读、幻读、不可重复读。

首先来看脏读，如图所示，



假设有两个事务T1/T2同时在执行，T1事务有可能会读取到T2事务未提交的数据，但是未提交的事务T2可能会回滚，也就导致了T1事务读取到最终不一定存在的数据产生脏读的现象。

然后来看幻读，如图所示：

假设有两个事务T1/T2同时执行，事务T1执行范围查询或者范围修改的过程中，事务T2插入了一条属于事务T1范围内的数据并且提交了，这时候在事务T1查询发现多出来了一条数据，或者在T1事务发现这条数据没有被修改，看起来像是产生了幻觉，这种现象称为幻读。

最后来看，不可重复读，如图所示：

假设有两个事务T1/T2同时执行，事务T1在不同的时刻读取同一行数据的时候结果可能不一样，从而导致不可重复读的问题。

2、事务隔离级别

那么事务隔离级别，就是是为了解决多个并行事务竞争 。而这脏读、幻读、不可重复读这三种现象在实际应用中，有些业务场景是不能接受这些现象存在的，所以在SQL标准中定义了四种隔离级别，分别是：

读未提交，在这种隔离级别下，可能会产生脏读、不可重复读、幻读。

读已提交（RC），在这种隔离级别下，可能会产生不可重复读和幻读。

可重复读（RR），在这种隔离级别下，可能会产生幻读

串行化，在这种隔离级别下，多个并行事务串行化执行，不会产生安全性问题。

这四种隔离级别里面，只有串行化解决了全部的问题，但这种隔离级别的性能是最低的。

在MySQL里面，InnoDB引擎默认的隔离级别是RR（可重复读），因为它需要保证事务ACID特性中的隔离性特征。

【结尾】

以上就是我对 MySQL事务隔离级别的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

灰灰老师解析

聊到事务隔离级别，我们得先聊下事务

所谓事务，就是工作的原子单元，可以去进行提交、回滚。在数据库中的工作，就是执行的sql语句，这些语句可以是单条语句也可以是多条语句。所以，其实就是去保证单条语句或者多条sql语句作为一个原子操作，要么全部成功要么全部回滚。

那么这也是我们数据库ACID设计原则里面的A,所谓ACID，是我们去设置数据库的时候，尽可能去保证数据的一致性与可靠性的一些设计原则。就像我们写代码的时候会去考虑设计原则，比如依赖倒置原则。

A Atomicity ætəˈmɪsəti 原子性 就是通过事务的提交回滚来实现的

C Consistency kənˈsɪstənsi 一致性 保证数据进来一致的

I Isolation aɪsəˈleɪʃn 每个更改数据的会话希望是相互不影响的

D Durability dʊrəˈbɪləti 持久性 保证数据不会丢失

那么其中的I 就是事务的隔离级别。

为什么会有这些隔离级别呢?

因为事务是可以并行执行的，那么如果并行执行，就会导致一定的数据一致性问题。比如我们经常讲的脏读、不可重复读、幻读。

那么如果业务不能容忍这些数据一致性问题，作者就得去解决它，但是如果解决了一致性问题，性能就会有很大的影响。所以，作者干脆将这个难题交给了用户自己选择，这样不同的隔离级别就出来了。

分别是

RU

RC

RR

串行化

这4个隔离级别，越往上，性能越差，但是数据一致性越好。

RU读到未提交的数据，所以会有脏读、不可重复读、幻读问题

RC 读已提交的，但是没有解决不可重复读跟幻读问题

RR 在innoDB里面通过MVCC以及LBCC来解决了不可重复读跟幻读问题。也是innodb默认的隔离级别

至于串行化，就是不让事务有并行执行的可能，性能最低，一般不会选择

那么在工作中，也可以基于性能与数据一致性，来灵活的选择隔离级别，用得比较多的RC。

面试官：你刚才讲RR通过MVCC以及LBCC来解决幻读问题，但是RC也有啊，为什么RC没有解决不可重复读跟幻读。

灰灰老师：这块其实是因为不同的隔离级别在MVCC于LBCC的实现逻辑有点不一样。

首先，我们来讲下MVCC的流程，MVCC其实就是查询的时候，会去生成一个readView的class结构，里面会保存一些事务ID数据，比如当前存活的事务ID有哪些，然后跟数据的事务ID进行比对，来决定数据是否展示。

RR 只有在第一次查询的时候会生成一个readView，后面的查询比对的都是第一个查询的时候的结果，所以后面查不到新的值

RC是每次查询都会生成一个新的readView，这样每次查询我就能拿到最新的数据了，也就产生了不可重复读跟幻读问题

而LBCC是通过锁的形式去解决不可重复读跟幻读问题，就是我在改数据同一条或者同一批数据的时候，其他事务不能进行改动。但是RR相比于RC多了一个间隙锁，所谓间隙锁，就是会根据你锁的条件会锁定一个区间，所以没有幻读，但是RC隔离级别下，间隙锁只会用在重复键的检查。

这是我对这个问题的理解。

什么是幻读，脏读，不可重复读呢?

- 事务 A、B 交替执行，事务 A 被事务 B 干扰到了，因为事务 A 读取到事务 B 未提交的数据，这就是脏读
- 在一个事务范围内，两个相同的查询，读取同一条记录，却返回了不同的数据，这
- 就是不可重复读。
- 事务 A 查询一个范围的结果集，另一个并发事务 B 往这个范围中插入/删除了数据， 并静悄悄地提交，然后事务 A 再次查询相同的范围，两次读取得到的结果集不一样了，这就是幻读。

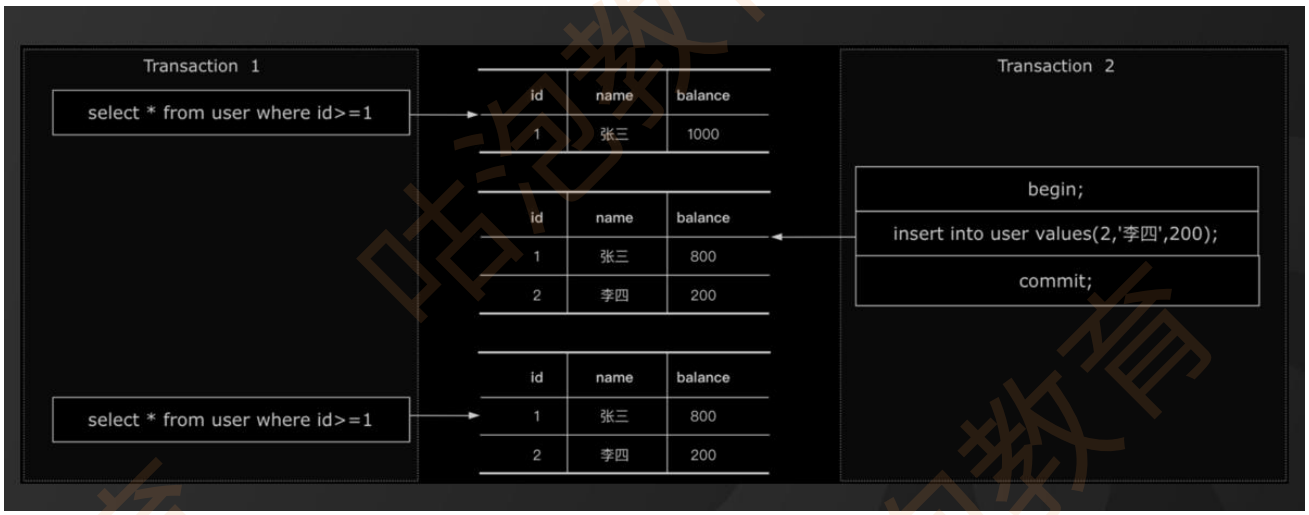
老师解析

在SQL操作中，多个事务竞争可能会产生三种不同的现象，分别是脏读、幻读、不可重复读。

首先来看脏读，如图所示，

假设有两个事务T1/T2同时在执行，T1事务有可能会读取到T2事务未提交的数据，但是未提交的事务T2可能会回滚，也就导致了T1事务读取到最终不一定存在的数据产生脏读的现象。

然后来看幻读，如图所示：



假设有两个事务T1/T2同时执行，事务T1执行范围查询或者范围修改的过程中，事务T2插入了一条属于事务T1范围内的数据并且提交了，这时候在事务T1查询发现多出来了一条数据，或者在T1事务发现这条数据没有被修改，看起来像是产生了幻觉，这种现象称为幻读。

最后来看，不可重复读，如图所示：

假设有两个事务T1/T2同时执行，事务T1在不同的时刻读取同一行数据的时候结果可能不一样，从而导致不可重复读的问题。

1.11. 在高并发情况下，如何做到安全的修改同一行数据?

要安全的修改同一行数据，就要保证一个线程在修改时其它线程无法更新这行记录。一般有悲观锁和乐观锁两种方案~

使用悲观锁

悲观锁思想就是，当前线程要进来修改数据时，别的线程都得拒之门外~ 比如， 可以使用 select... for update ~

```
select * from User where name= ‘jay’ for update
```

以上这条 sql 语句会锁定了 User 表中所有符合检索条件（name= ‘jay’ ）的记录。本次事务提交之前，别的线程都无法修改这些记录。

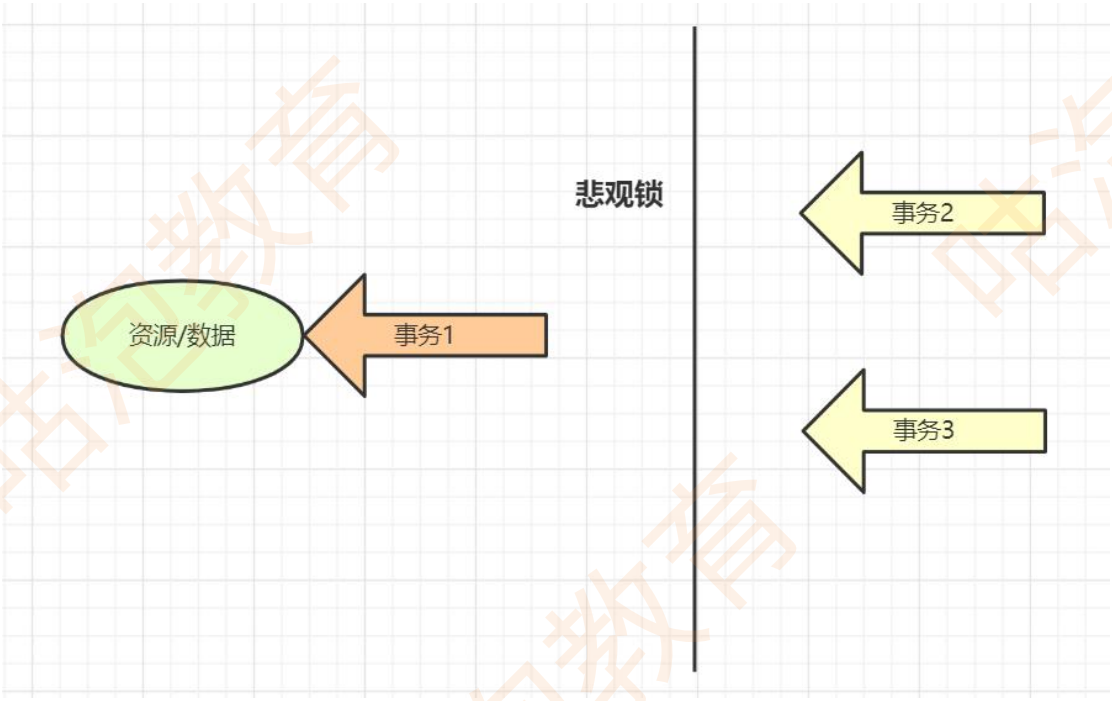
使用乐观锁

乐观锁思想就是，有线程过来，先放过去修改，如果看到别的线程没修改过， 就可以修改成功，如果别的线程修改过，就修改失败或者重试。实现方式：乐观锁一般会使用版本号机制或 CAS 算法实现。

1.12. 数据库的乐观锁和悲观锁。

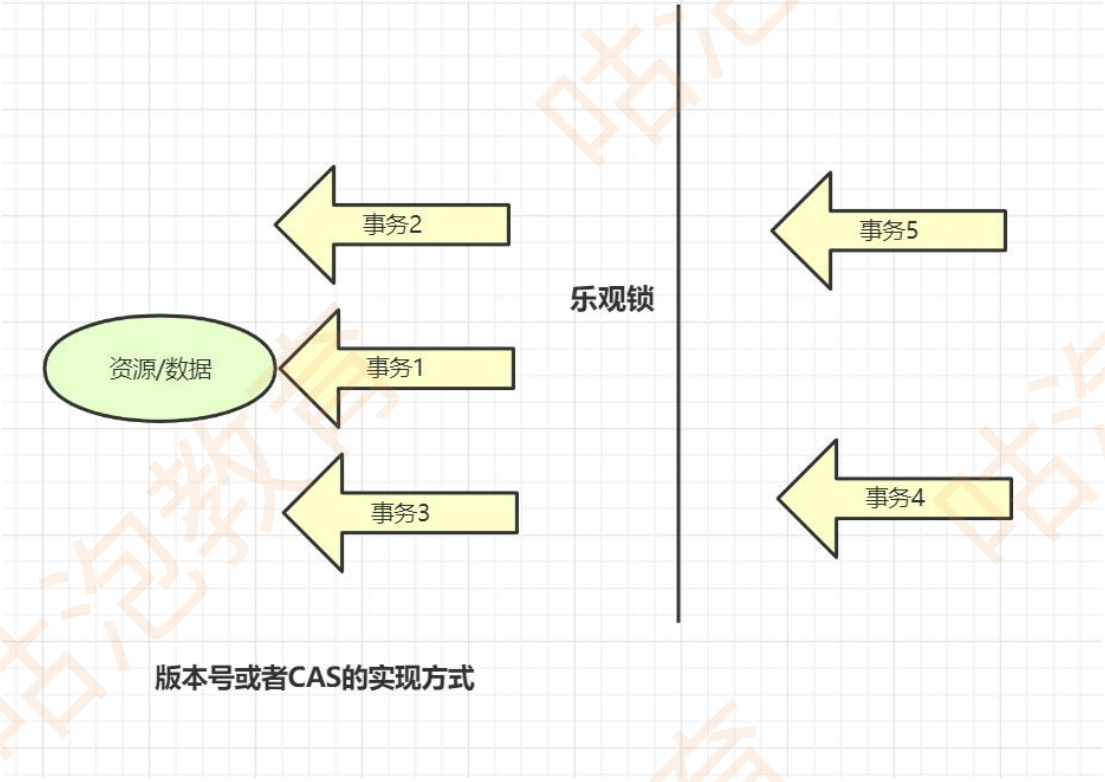
悲观锁

悲观锁她专一且缺乏安全感了，她的心只属于当前事务，每时每刻都担心着它心爱的数据可能被别的事务修改，所以一个事务拥有（获得）悲观锁后，其他任何事务都不能对数据进行修改啦，只能等待锁被释放才可以执行。



乐观锁：

乐观锁的“乐观情绪”体现在，它认为数据的变动不会太频繁。因此，它允许多 个事务同时对数据进行变动。实现方式：乐观锁一般会使用版本号机制或 CAS 算法实现。



1.13. SQL 优化的一般步骤是什么，怎么看执行计划（explain），如何理解其中各个字段的含义。

SQL优化分为三步曲

l 第一、慢SQL的定位和排查

我们可以通过慢查询日志和慢查询日志分析工具得到有问题的SQL列表。

l 第二、执行计划分析

针对慢SQL，我们可以使用关键字explain来查看当前sql的执行计划.可以重点关注 type key rows filterd 等字段，从而定位该SQL执行慢的根本原因。再有的放矢的进行优化

l 第三、使用show profile工具

Show Profile是MySQL提供的可以用来分析当前会话中，SQL语句资源消耗情况的工具，可用于SQL调优的测量。在当前会话中.默认情况下处于show profile是关闭状态，打开之后保存最近15次的运行结果

针对运行慢的SQL，通过profile工具进行详细分析.可以得到SQL执行过程中所有的资源开销情况.

如IO开销,CPU开销,内存开销等.

1.14. select for update 有什么含义，会锁表还是锁行还是其他

select for update 含义

select 查询语句是不会加锁的，但是 select for update 除了有查询的作用外， 还会加锁呢，而且它是悲观锁哦。至于加了是行锁还是表锁，这就要看是不是 用了索引/主键啦。

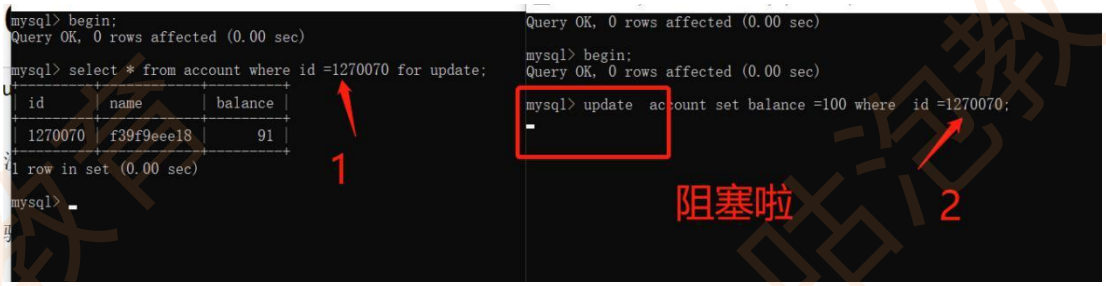
没用索引/主键的话就是表锁，否则就是是行锁。

select for update 加锁验证

表结构：

```
//id 为主键，name为唯一索引
CREATE TABLE `account` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `name` varchar(255) DEFAULT NULL,
  `balance` int(11) DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_name` (`name`) USING BTREE
) ENGINE=InnoDB AUTO_INCREMENT=1570068 DEFAULT CHARSET=utf8
```

id 为主键，select for update 1270070 这条记录时，再开一个事务对该记录更新，发现更新阻塞啦，其实是加锁了。如下图：



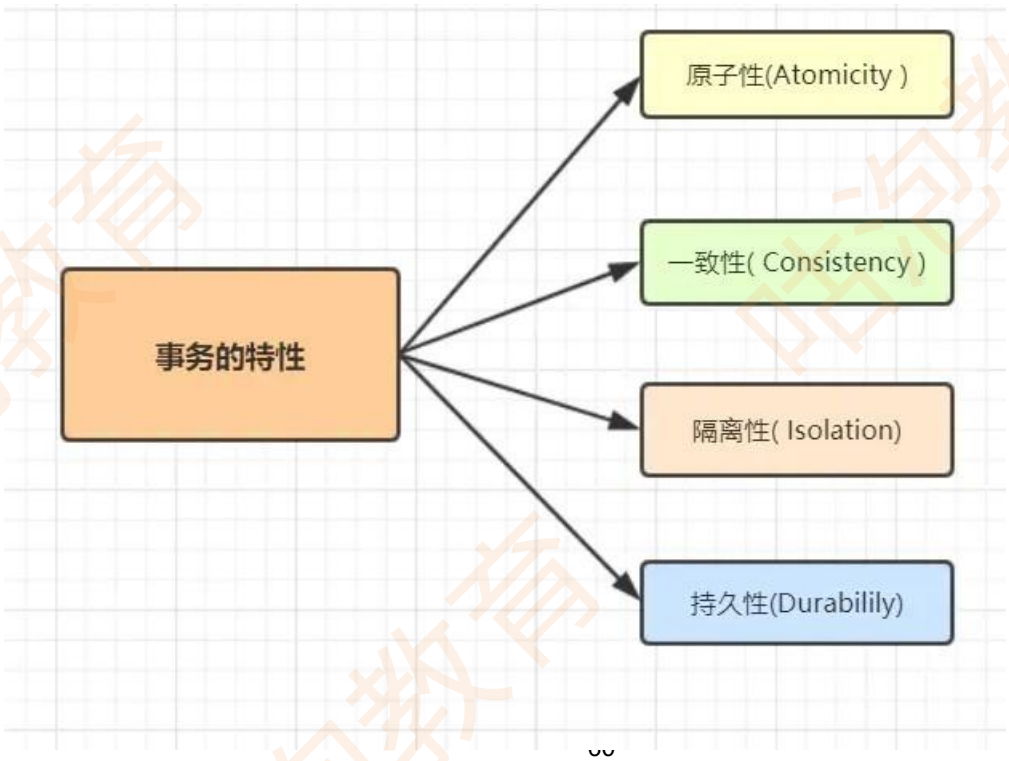
我们再开一个事务对另外一条记录 1270071 更新，发现更新成功，因此，如

果查询条件用了索引/主键，**会加行锁~**

我们继续一路向北吧，换普通字段 balance 吧，发现又阻塞了。因此，没用索引/主键的话，**select for update 加的就是表锁**

1.15. MySQL 事务的四大特性以及实现原理

四大特性



- 原子性： 事务作为一个整体被执行，包含在其中的对数据库的操作要么全部被执行，要么都不执行。
- 一致性： 指在事务开始之前和事务结束以后，数据不会被破坏，假如 A 账户给 B
- 账户转 10 块钱，不管成功与否，A 和B 的总金额是不变的。
- 隔离性： 多个事务并发访问时，事务之间是相互隔离的，即一个事务不影响其它事务运行效果。简言之，就是事务之间是进水不犯河水的。
- 持久性： 表示事务完成以后，该事务对数据库所作的操作更改，将持久地保存在数据库之中。

实现原理

Hi，大家好，我是Mic，一个工作了14年的程序员和创业者。

今天分享的面试题很有意思，去大厂面试的时候，百分之90的可能性会问到。

但是真正能够完整回答出来的同学缺很少。

最近一个工作了11年的粉丝去面试，就被面试到这个问题。

问题是： Mysql事务的底层实现原理。

下面看看普通人和高手对这个问题的回答

普通人

原子性：是使用 undo log 来实现的，如果事务执行过程中出错或者用户执行了rollback，系统通过 undo log 日志返回事务开始的状态。

持久性：使用 redo log 来实现，只要 redo log 日志持久化了，当系统崩溃，即可

通过redo log 把数据恢复。

隔离性：通过锁以及 MVCC,使事务相互隔离开。

一致性：通过回滚、恢复，以及并发情况下的隔离性，从而实现一致性。

高手

好的，面试官

Mysql里面的事务，满足ACID特性，所以在我看来，Mysql的事务实现原理，就是InnoDB是如何保证ACID特性的。

首先，A表示Atomic原子性，也就是需要保证多个DML操作是原子的，要么都成功，要么都失败。

那么，失败就意味着要对原本执行成功的数据进行回滚，所以InnoDB设计了一个UNDO_LOG表，在事务执行的过程中，把修改之前的数据快照保存到UNDO_LOG里面，一旦出现错误，就直接从UNDO_LOG里面读取数据执行反向操作就行了。

其次，C表示一致性，表示数据的完整性约束没有被破坏，这个更多是依赖于业务层面的保证，数据库本身也提供了一些，比如主键的唯一余数，字段长度和类型的保证等等。

接着，I表示事物的隔离性，也就是多个并行事务对同一个数据进行操作的时候，如何避免多个事务的干扰导致数据混乱的问题。

而InnoDB实现了SQL92的标准，提供了四种隔离级别的实现。分别是：

RU（未提交读）

RC（已提交读）

RR（可重复读）

Serializable（串行化）

InnoDB默认的隔离级别是RR（可重复读），然后使用了MVCC机制解决了脏读和不可重复读的问题，然后使用了行锁/表锁的方式解决了幻读的问题。

最后一个是D，表示持久性，也就是只要事务提交成功，那对于这个数据的结果的影响一定是永久性的。

不能因为宕机或者其他原因导致数据变更失效。

理论上来说，事务提交之后直接把数据持久化到磁盘就行了，但是因为随机磁盘IO的效率确实很低，所以InnoDB设计了Buffer Pool缓冲区来优化，也就是数据发生变更的时候先更新内存缓冲区，然后在合适的时机再持久化到磁盘。

那在持久化这个过程中，如果数据库宕机，就会导致数据丢失，也就无法满足持久性了。

所以InnoDB引入了Redo_LOG文件，这个文件存储了数据被修改之后的值，当我们通过事务对数据进行变更操作的时候，除了修改内存缓冲区里面的数据以外，还会把本次修改的值追加到REDO_LOG里面。

当提交事务的时候，直接把REDO_LOG日志刷到磁盘上持久化，一旦数据库出现宕机，在Mysql重启在以后可以直接用REDO_LOG里面保存的重写日志读取出来，再执行一遍从而保证持久性。

因此，在我看来，事务的实现原理的核心本质就是如何满足ACID的，在InnDB里面用到了MVCC、行锁表锁、UNDO_LOG、REDO_LOG等机制来保证。

以上就是我对这个问题的理解！

面试点评

InnDB的事务实现原理，有很多可以借鉴的设计思想，

比如乐观锁、利用内存缓冲区的方式，以空间换时间的思想优化磁盘IO的性能等等

这些思想还挺重要的，比如在分布式事务框架Seata的AT模式的数据回滚，就借鉴了

InnDB里面UNDO_LOG的设计思想。

1.16. 如果某个表有近千万数据，CRUD比较慢，如何优化。

分库分表

某个表有近千万数据，可以考虑优化表结构，分表（水平分表，垂直分表），

当然，你这样回答，需要准备好面试官问你的分库分表相关问题呀，如：

分表方案（水平分表，垂直分表，切分规则 hash 等）

分库分表中间件（Mycat，sharding-jdbc　等）

分库分表一些问题（事务问题？跨节点 Join 的问题）

解决方案（分布式事务等）

索引优化

除了分库分表，优化表结构，当然还有所以索引优化等方案~

1.17. 如何写 sql 能够有效的使用到复合索引

复合索引，也叫联合索引，用户可以在多个列上建立索引,这种索引叫做复合索引。

当我们创建一个组合索引的时候，如(k1,k2,k3)，相当于创建了（k1）、

(k1,k2)和(k1,k2,k3)三个索引，这就是最左匹配原则。

```
select * from table where k1=A AND k2=B AND k3=D
```

有关于复合索引，我们需要关注查询 Sql 条件的顺序，确保最左匹配原则有效，同时可以删除不必要的冗余索引。

1.18. mysql 中in 和exists 的区别

这个，跟一下代码来看更刺激吧，啊哈哈

假设表 A 表示某企业的员工表，表 B 表示部门表，查询所有部门的所有员工，很容易有以下 SQL:

```
select * from A where deptId in (select deptId from B);
```

这样写等价于：

先查询部门表 B

select deptId from B

再由部门deptId，查询 A 的员工

select * from A where A.deptId = B.deptId

可以抽象成这样的一个循环：

```
    List<> resultSet ;
      for(int i=0;i<B.length;i++) {
                for(int j=0;j<A.length;j++) {
                if(A[i].id==B[j].id) {
                    resultSet.add(A[i]);
                    break;
                }
            }
      }
```

显然，除了使用 in，我们也可以用 exists 实现一样的查询功能，如下：

```
select * from A where exists (select 1 from B where A.deptId = B.deptId);
```

因为exists 查询的理解就是，先执行主查询，获得数据后，再放到子查询中做条件验证，根据验证结果（true 或者false），来决定主查询的数据结果是否得意保留。

select * from A,先从A 表做循环

select * from B where A.deptId = B.deptId,再从 B 表做循环.

那么，这样写就等价于：

同理，可以抽象成这样一个循环：

```
List<> resultSet ;
for(int i=0;i<A.length;i++) {
    for(int j=0;j<B.length;j++) {
    if(A[i].deptId==B[j].deptId) {
        resultSet.add(A[i]);
        break;
    }
  }
}
```

数据库最费劲的就是跟程序链接释放。假设链接了两次，每次做上百万次的数据集查询，查完就走，这样就只做了两次；相反建立了上百万次链接，申请链接释放反复重复，这样系统就受不了了。即 mysql 优化原则，就是小表驱动大表，小的数据集驱动大的数据集，从而让性能更优。

因此，我们要选择最外层循环小的，也就是，如果 **B 的数据量小于 A，适合使用 in，如果 B 的数据量大于 A，即适合选择 exists**，这就是 in 和 exists 的区别。

1.19. 数据库自增主键可能遇到什么问题。

- 使用自增主键对数据库做分库分表，可能出现诸如主键重复等的问题。解决方案的话，简单点的话可以考虑使用 UUID 哈
- 自增主键会产生表锁，从而引发问题
- 自增主键可能用完问题。

1.20. MVCC熟悉吗，它的底层原理?

MVCC,多版本并发控制,它是通过读取历史版本的数据，来降低并发事务冲突，从而提高并发性能的一种机制。

MVCC需要关注这几个知识点：

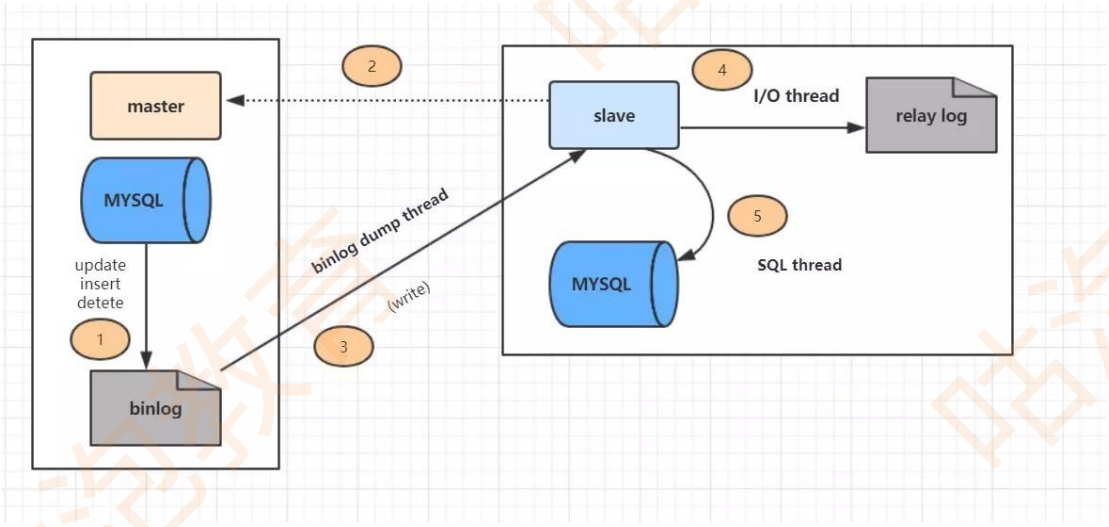
- 事务版本号
- 表的隐藏列
- undo log
- read view

1.21. 数据库中间件了解过吗，sharding，jdbc，mycat?

- sharding-jdbc 目前是基于 jdbc 驱动，无需额外的 proxy，因此也无需关注proxy 本身的高可用。
- Mycat 是基于 Proxy，它复写了 MySQL 协议，将 Mycat Server 伪装成一个MySQL 数据库，而 Sharding-JDBC 是基于 JDBC 接口的扩展，是以 jar 包的形式提供轻量级服务的。

1.22. MYSQL 的主从延迟，你怎么解决?

嘻嘻，先复习一下主从复制原理吧，如图：



主从复制分了五个步骤进行：

- 步骤一：主库的更新事件(update、insert、delete)被写到 binlog
- 步骤二：从库发起连接，连接到主库。
- 步骤三：此时主库创建一个 binlog dump thread，把binlog 的内容发送到从库。
- 步骤四：从库启动之后，创建一个 I/O 线程，读取主库传过来的 binlog 内容并写入到 relay log
- 步骤五：还会创建一个 SQL 线程，从 relay log 里面读取内容，从Exec_Master_Log_Pos 位置开始执行读取到的更新事件，将更新内容写入到slave 的db

主从同步延迟的原因

一个服务器开放N个链接给客户端来连接的，这样有会有大并发的更新操作, 但是从服务器的里面读取 binlog 的线程仅有一个，当某个 SQL 在从服务器上执行的时间稍长 或者由于某个 SQL 要进行锁表就会导致，主服务器的 SQL 大量积压，未被同步到从服务器里。这就导致了主从不一致， 也就是主从延迟。

主从同步延迟的解决办法

- 主服务器要负责更新操作，对安全性的要求比从服务器要高，所以有些设置参数可以修改，比如 sync_binlog=1，innodb_flush_log_at_trx_commit = 1 之类的设置等。
- 选择更好的硬件设备作为 slave。
- 把一台从服务器当度作为备份使用， 而不提供查询， 那边他的负载下来了， 执行
- relay log 里面的 SQL 效率自然就高了。
- 增加从服务器喽，这个目的还是分散读的压力，从而降低服务器负载。

1.23. 说一下大表查询的优化方案

- 优化 shema、sql 语句+索引;
- 可以考虑加缓存，memcached, redis，或者 JVM 本地缓存;
- 主从复制，读写分离;
- 分库分表。

1.24. 什么是数据库连接池?为什么需要数据库连接池呢?

首先，数据库连接池是一种池化技术，池化技术的核心思想是实现资源的复用，避免资源重复创建销毁的开销。

而在数据库的应用场景里面，应用程序每次向数据库发起CRUD操作的时候，都需要创建连接

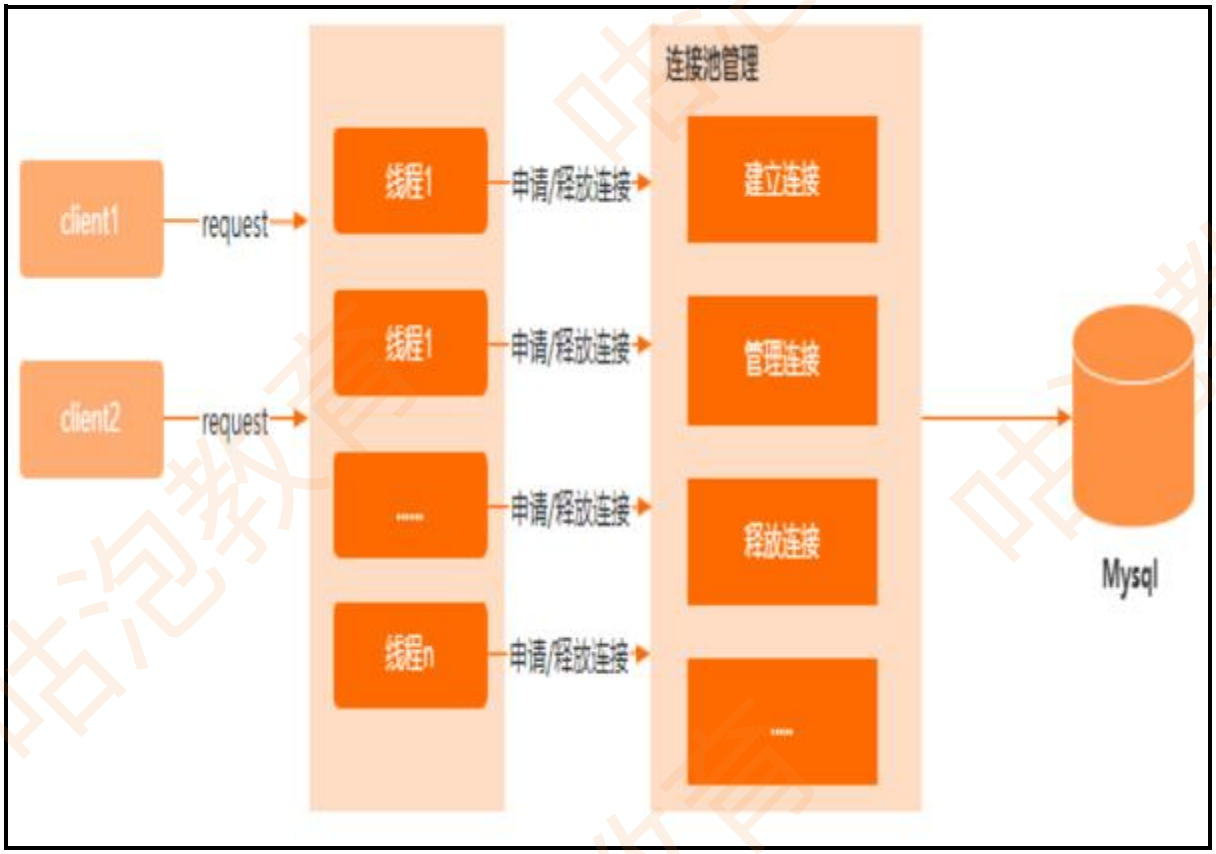
在数据库访问量较大的情况下，频繁的创建连接会带来较大的性能开销。

而连接池的核心思想，就是应用程序在启动的时候提前初始化一部分连接保存到连接池里面，当应用需要使用连接的时候，直接从连接池获取一个已经建立好的链接。

连接池的设计，避免了每次连接的建立和释放带来的开销。

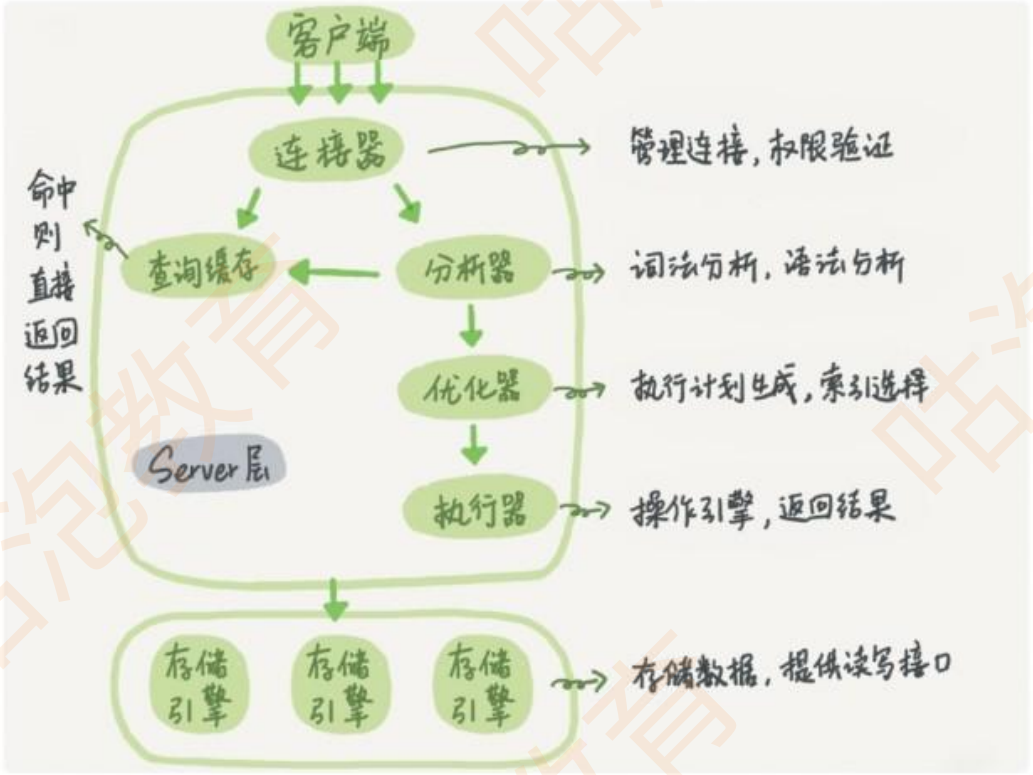
数据库连接池好处:

- 资源重用 (连接复用)
- 更快的系统响应速度
- 新的资源分配手段
- 统一的连接管理，避免数据库连接泄漏



1.25. 一条 SQL 语句在 MySQL 中如何执行的?

先看一下 Mysql 的逻辑架构图吧~



查询语句：

- 先检查该语句是否有权限
- 如果没有权限，直接返回错误信息
- 如果有权限，在 MySQL8.0 版本以前，会先查询缓存。
- 如果没有缓存，分析器进行词法分析，提取 sql 语句 select 等的关键元素。然后判断sql 语句是否有语法错误，比如关键词是否正确等等。
- 优化器进行确定执行方案
- 进行权限校验，如果没有权限就直接返回错误信息，如果有权限就会调用数据库引擎接口，返回执行结果。

1.26. InnoDB引擎中的索引策略，了解过吗?

- 覆盖索引
- 最左前缀原则
- 索引下推
- 索引下推优化是 MySQL 5.6 引入的， 可以在索引遍历过程中，对索引中包含的字段先做判断，直接过滤掉不满足条件的记录，减少回表次数。
- 最左前缀原则：MySQl 建立联合索引时，会遵循最左前缀匹配的原则，即最左优先。如果你建立一个（a,b,c）的联合索引，相当于建立了(a)、(a,b)、(a,b,c)三个索引。
- 覆盖索引：只需要在一棵索引树上就能获取 SQL 所需的所有列数据，无需回表，速度更快。

1.27. 一条 sql 执行过长的时间，你如何优化，从哪些方面入手?

- 查看是否涉及多表和子查询，优化 Sql 结构，如去除冗余字段，是否可拆表等
- 优化索引结构，看是否可以适当添加索引
- 数量大的表，可以考虑进行分离/分表（如交易流水表）
- 数据库主从分离，读写分离
- explain 分析 sql 语句，查看执行计划，优化 sql

- 查看mysql 执行日志，分析是否有其他方面的问题

1.28. MYSQL数据库服务器性能分析的方法命令有哪些?

- Show status，一些值得监控的变量值：

- Bytes_received 和 Bytes_sent 和服务器之间来往的流量。
- Com_*服务器正在执行的命令。
- Created_*在查询执行期限间创建的临时表和文件。
- Handler_*存储引擎操作。
- Select_*不同类型的联接执行计划。
- Sort_*几种排序信息

- Show profiles 是MySql 用来分析当前会话 SQL 语句执行的资源消耗情况

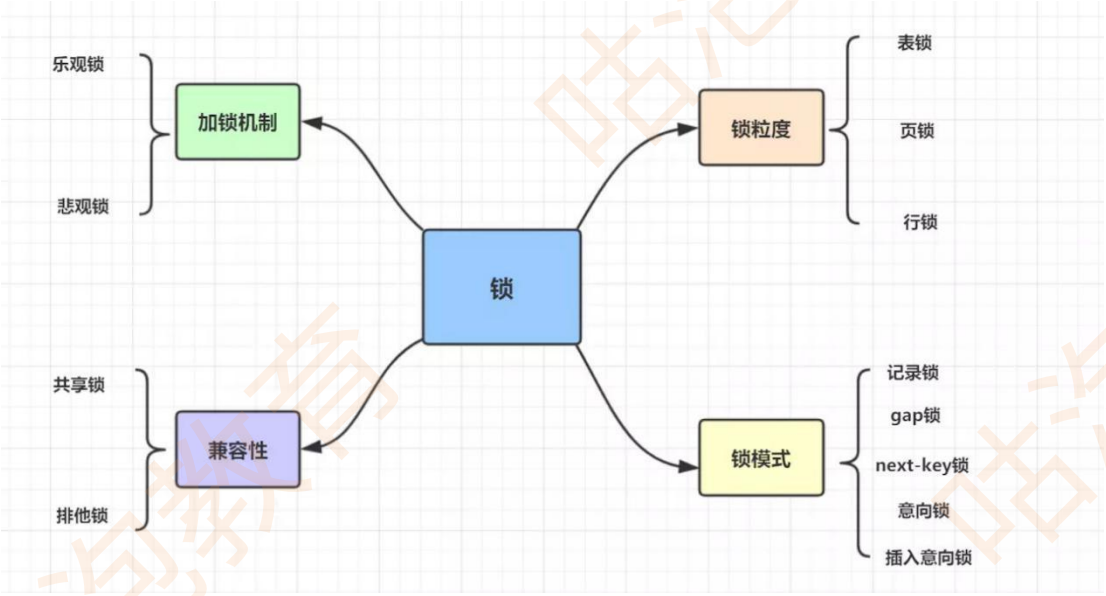
1.29. Blob 和text 有什么区别?

- Blob 用于存储二进制数据，而 Text 用于存储大字符串。
- Blob 值被视为二进制字符串（字节字符串）,它们没有字符集，并且排序和比较基于列值中的字节的数值。
- text 值被视为非二进制字符串（字符字符串）。它们有一个字符集，并根据字符集的排序规则对值进行排序和比较。

1.30. mysql里记录货币用什么字段类型比较好?

- 货币在数据库中 MySQL 常用 Decimal 和 Numric 类型表示，这两种类型被MySQL 实现为同样的类型。他们被用于保存与金钱有关的数据。
- salary DECIMAL(9,2)，9(precision)代表将被用于存储值的总的小数位数，而2(scale)代表将被用于存储小数点后的位数。存储在 salary 列中的值的范围是从-9999999.99 到 9999999.99。
- DECIMAL 和 NUMERIC 值作为字符串存储，而不是作为二进制浮点数，以便保存那些值的小数精度。

1.31. Mysql中有哪几种锁，列举一下?



如果按锁粒度划分，有以下 3 种：

- 表锁： 开销小，加锁快；锁定力度大，发生锁冲突概率高，并发度最低;不会出现死锁。
- 行锁： 开销大，加锁慢；会出现死锁；锁定粒度小，发生锁冲突的概率低，并发度高。
- 页锁： 开销和加锁速度介于表锁和行锁之间；会出现死锁；锁定粒度介于表锁和行锁之间，并发度一般

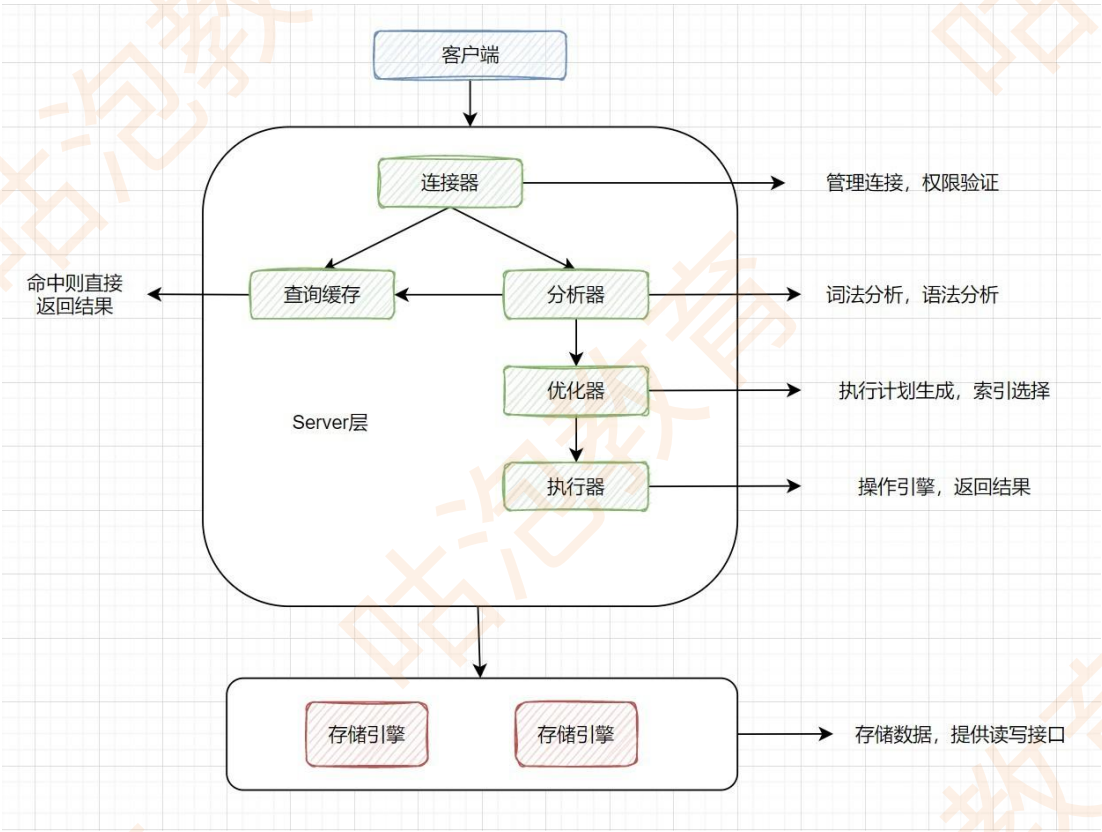
1.32. Hash索引和B+树区别是什么？你在设计索引是怎么抉择的?

- B+树可以进行范围查询，Hash 索引不能。
- B+树支持联合索引的最左侧原则，Hash 索引不支持。
- B+树支持 order by 排序，Hash 索引不支持。
- Hash 索引在等值查询上比 B+树效率更高。
- B+树使用 like 进行模糊查询的时候，like 后面（比如%开头）的话可以起到优化的作用，Hash 索引根本无法进行模糊查询。

1.33. mysql的内连接、左连接、右连接有什么区别?

- Inner join 内连接，在两张表进行连接查询时，只保留两张表中完全匹配的结果集
- left join 在两张表进行连接查询时，会返回左表所有的行，即使在右表中没有匹配的记录。
- right join 在两张表进行连接查询时，会返回右表所有的行，即使在左表中没有匹配的记录。

1.34. 说说MySQL的基础架构图



Mysql 逻辑架构图主要分三层：

- 第一层负责连接处理，授权认证，安全等等
- 第二层负责编译并优化 SQL
- 第三层是存储引擎。

1.35. 什么是内连接、外连接、交叉连接、笛卡尔积呢?

- 内连接（inner join）：取得两张表中满足存在连接匹配关系的记录。

- 外连接（outer join）：取得两张表中满足存在连接匹配关系的记录，以及某张表（或两张表）中不满足匹配关系的记录。

- 交叉连接（cross join）：显示两张表所有记录一一对应，没有匹配关系进行筛选，也被称为：笛卡尔积。

1.36. 说一下数据库的三大范式

- 第一范式：数据表中的每一列（每个字段）都不可以再拆分。
- 第二范式：在第一范式的基础上，分主键列完全依赖于主键，而不能是依赖于主键的一部分。
- 第三范式：在满足第二范式的基础上，表中的非主键只依赖于主键，而不依赖于其他非主键。

1.37. Mysql 的binlog 有几种录入格式？分别有什么区别？

有三种格式哈，statement，row 和 mixed。

- statement，记录的是 SQL 的原文。不需要记录每一行的变化，减少了binlog日志量，节约了 IO，提高性能。由于 sql 的执行是有上下文的，因此在保存的时候需要保存相关的信息，同时还有一些使用了函数之类的语句无法被记录复制。
- row，不记录 sql 语句上下文相关信息，仅保存哪条记录被修改。记录单元为每一行的改动，基本是可以全部记下来但是由于很多操作，会导致大量行的改动(比如 alter table)，因此这种模式的文件保存的信息太多，日志量太大。
- mixed，一种折中的方案，普通操作使用 statement 记录，当无法使用 statement 的时候使用 row。

1.38. InnoDB引擎的 4 大特性，了解过吗

- 插入缓冲（insert buffer)
- 二次写(double write)
- 自适应哈希索引(ahi)
- 预读(read ahead)

1.39. 索引有哪些优缺点？索引有哪几种类型？

索引有哪些优缺点

Hi，我是Mic

今天分享的这道面试题，让一个工作4年的小伙子去大众点评拿了60W年薪。

这道面试题是：” 请你说一下Mysql索引的优点和缺点 “

关于这道题，看看普通人和高手的回答

普通人

优点：

索引可以加快数据查询速度，减少查询时间

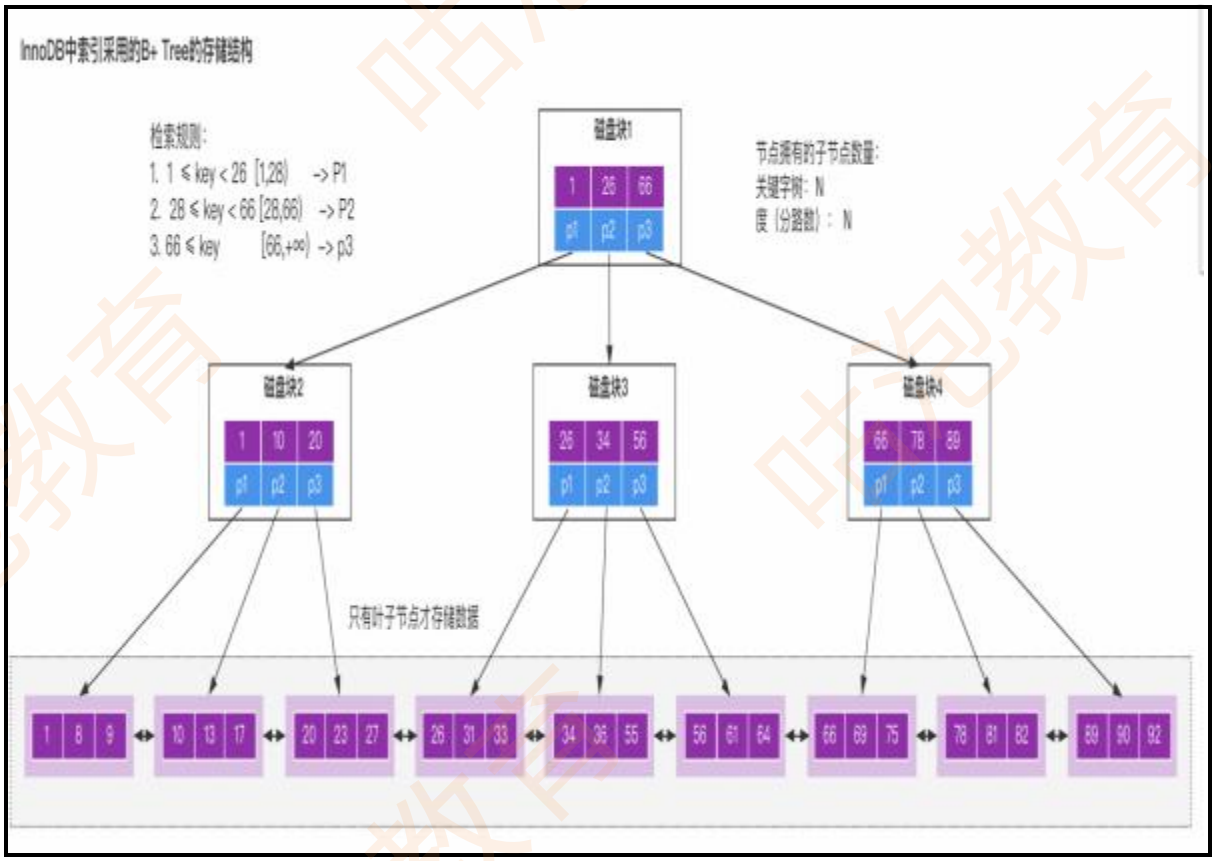
缺点：

创建索引和维护索引要耗费时间，索引需要占物理空间

高手

索引，是一种能够帮助Mysql高效从磁盘上检索数据的一种数据结构。

在Mysql中的InnoDB引擎中，采用了B+树的结构来实现索引和数据的存储



在我看来，Mysql里面的索引的优点有很多

通过B+树的结构来存储数据，可以大大减少数据检索时的磁盘IO次数，从而提升数据查询的性能

B+树索引在进行范围查找的时候，只需要找到起始节点，然后基于叶子节点的链表结构往下读取即可，查询效率较高。

通过唯一索引约束，可以保证数据表中每一行数据的唯一性

当然，索引的不合理使用，也会有带来很多的缺点。

数据的增加、修改、删除，需要涉及到索引的维护，当数据量较大的情况下，索引的维护会带来较大的性能开销。

一个表中允许存在一个聚簇索引和多个非聚簇索引，但是索引数不能创建太多，否则造成的索引维护成本过高。

创建索引的时候，需要考虑到索引字段值的分散性，如果字段的重复数据过多，创建索引反而会带来性能降低。

在我看来，任何技术方案都会有两面性，大部分情况下，技术方案的选择更多的是看中它的优势和当前问题的匹配度。

以上就是我对这个问题的理解。

面试点评

行业竞争加剧，再加上现在大环境不好，各个一二线大厂都在裁员。

带来的问题就是，人才筛选难度增加，找工作越来越难。

这道题目考察的是求职者对于Mysql的理解程度，不算难，但能卡主很多人。

好的，本期的普通人VS高手面试系列的视频就到这里结束了，

喜欢的朋友记得点赞和收藏。

有任何工作和学习上的问题，可以随时私信我。

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

索引有哪几种类型?

- 主键索引: 数据列不允许重复，不允许为 NULL，一个表只能有一个主键。
- 唯一索引: 数据列不允许重复，允许为 NULL 值，一个表允许多个列创建唯一索引。
- 普通索引: 基本的索引类型，没有唯一性的限制，允许为 NULL 值。
- 全文索引：是目前搜索引擎使用的一种关键技术，对文本的内容进行分词、搜索。
- 覆盖索引：查询列要被所建的索引覆盖，不必读取数据行

- 组合索引：多列值组成一个索引，用于组合搜索，效率大于索引合并

1.40. 创建索引有什么原则呢?

- 最左前缀匹配原则
- 频繁作为查询条件的字段才去创建索引
- 频繁更新的字段不适合创建索引
- 索引列不能参与计算，不能有函数操作
- 优先考虑扩展索引，而不是新建索引，避免不必要的索引
- 在 order by 或者 group by 子句中，创建索引需要注意顺序
- 区分度低的数据列不适合做索引列(如性别)
- 定义有外键的数据列一定要建立索引。
- 对于定义为 text、image 数据类型的列不要建立索引。
- 删除不再使用或者很少使用的索引

1.41. 创建索引的三种方式

- 在执行 CREATE TABLE 时创✂索引

```
CREATE TABLE `employee` (
  `id` int(11) NOT NULL,
  `name` varchar(255) DEFAULT NULL,
  `age` int(11) DEFAULT NULL,
  `date` datetime DEFAULT NULL,
  `sex` int(1) DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `idx_name` (`name`) USING BTREE
) ENGINE=InnoDB DEFAULT CHARSET=utf8;
```

- 使用 ALTER TABLE 命令添加索引

```
ALTER TABLE table_name ADD INDEX index_name (column);
```

- 使用 CREATE INDEX 命令创✂

```
CREATE INDEX index_name ON table_name (column);
```

1.42. 百万级别或以上的数据，你是如何删除的?

- 我们想要删除百万数据的时候可以先删除索引
- 然后批量删除其中无用数据
- 删除完成后重新创✂索引。

1.43. 什么是最左前缀原则？什么是最左匹配原则?

- 最左前缀原则，就是最左优先，在创✂多列索引时，要根据业务需求，where子句中使用最频繁的一列放在最左边。
- 当我们创✂一个组合索引的时候，如(k1,k2,k3)，相当于创✂了（k1）、(k1,k2)和(k1,k2,k3)三个索引，这就是最左匹配原则。。

1.44. B 树和 B+树的区别，数据库为什么使用B+树而不是B树?

MIC老师解析

B 树和 B+树的区别

数据结构与算法问题，困扰了无数的小伙伴。

很多小伙伴对数据结构与算法的认知有一个误区，认为工作中没有用到，为什么面试要问，问了能解决实际问题？

图灵奖获得者：Niklaus Wirth说过：程序=数据结构+算法，也就说我们无时无刻都在和数据结构打交道。

只是作为Java开发，由于技术体系的成熟度较高，使得大部分人认为：程序应该等于框架+SQL呀？

今天我们就来分析一道数据结构的题目：“ B树和B+树 “。

关于这个问题，我们来看看普通人和高手的回答！

普通人

嗯.我想想...嗯...Mysql里面好像是用了B+树来做索引的！然后...

高手

为了更清晰的解答这个问题，我打算从三个方面来回答：

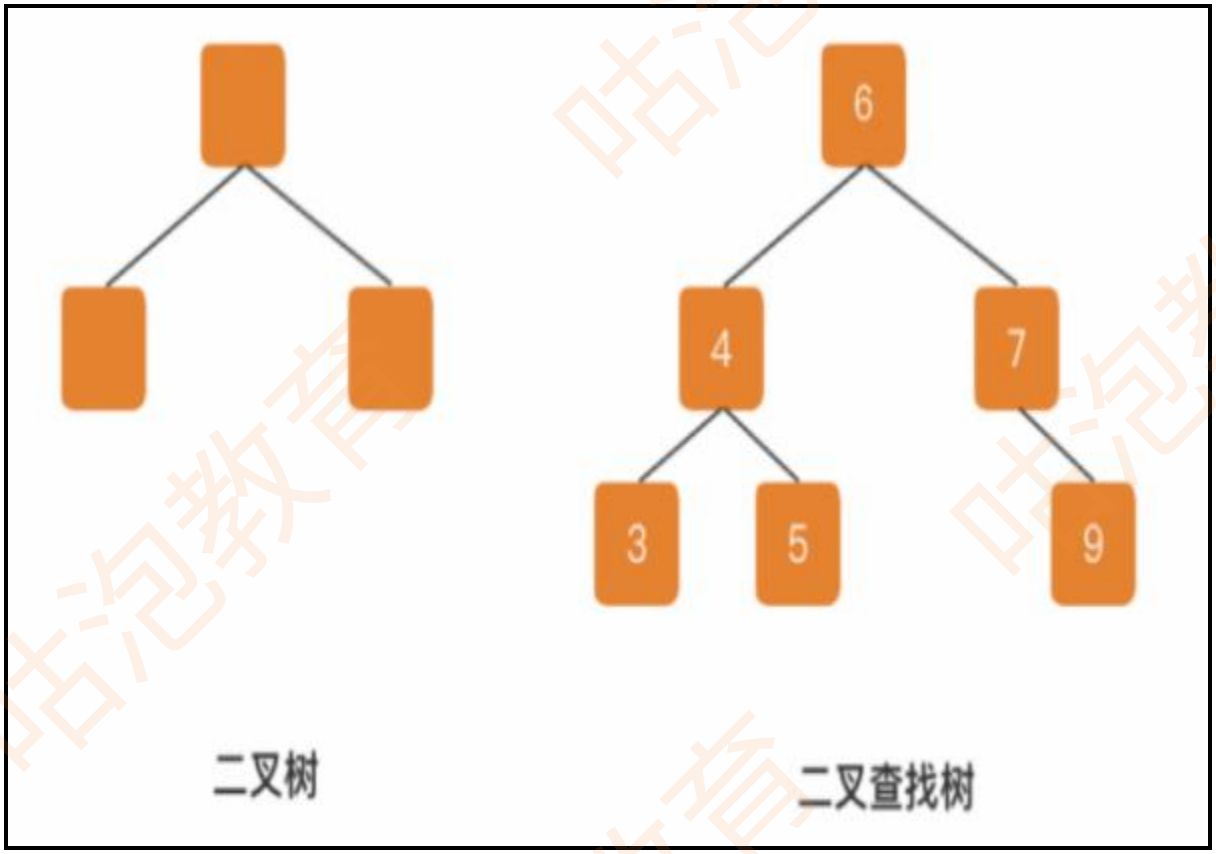
了解二叉树、AVL树、B树的概念

B树和B+树的应用场景

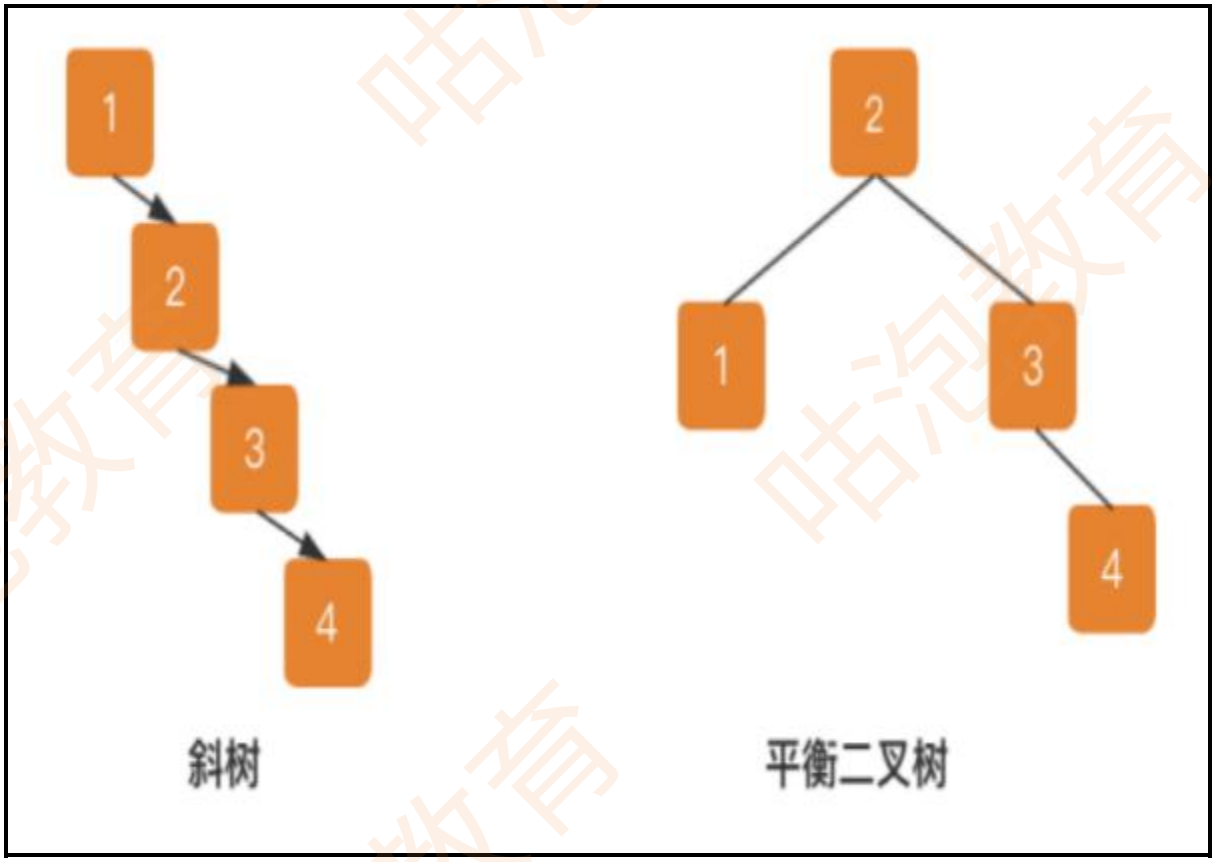
B树是一种多路平衡查找树，为了更形象的理解。

二叉树，每个节点支持两个分支的树结构，相比于单向链表，多了一个分支。

二叉查找树，在二叉树的基础上增加了一个规则，左子树的所有节点的值都小于它的根节点，右子树的所有子节点都大于它的根节点。

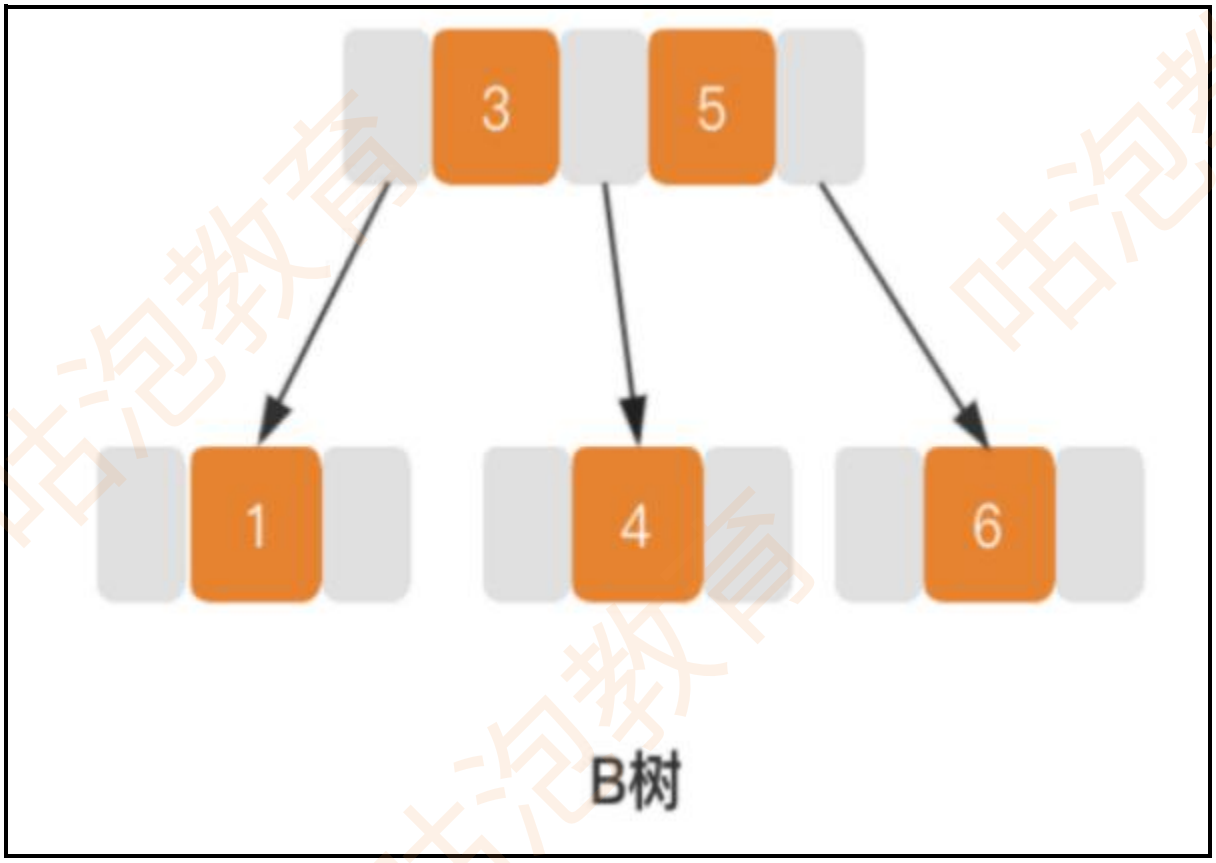


，二叉查找树会出现斜树问题，导致时间复杂度增加，因此又引入了一种平衡二叉树，它具有二叉查找树的所有特点，同时增加了一个规则：”它的左右两个子树的高度差的绝对值不超过1“。平衡二叉树会采用左旋、右旋的方式来实现平衡。



，而B树是一种多路平衡查找树，它满足平衡二叉树的规则，但是它可以有多个子树，子树的数量取决于关键字的数量，比如这个图中根节点有两个关键字3和5，那么它能够拥有的子路数量=关键字数+1。

因此从这个特征来看，在存储同样数据量的情况下，平衡二叉树的高度要大于B树。

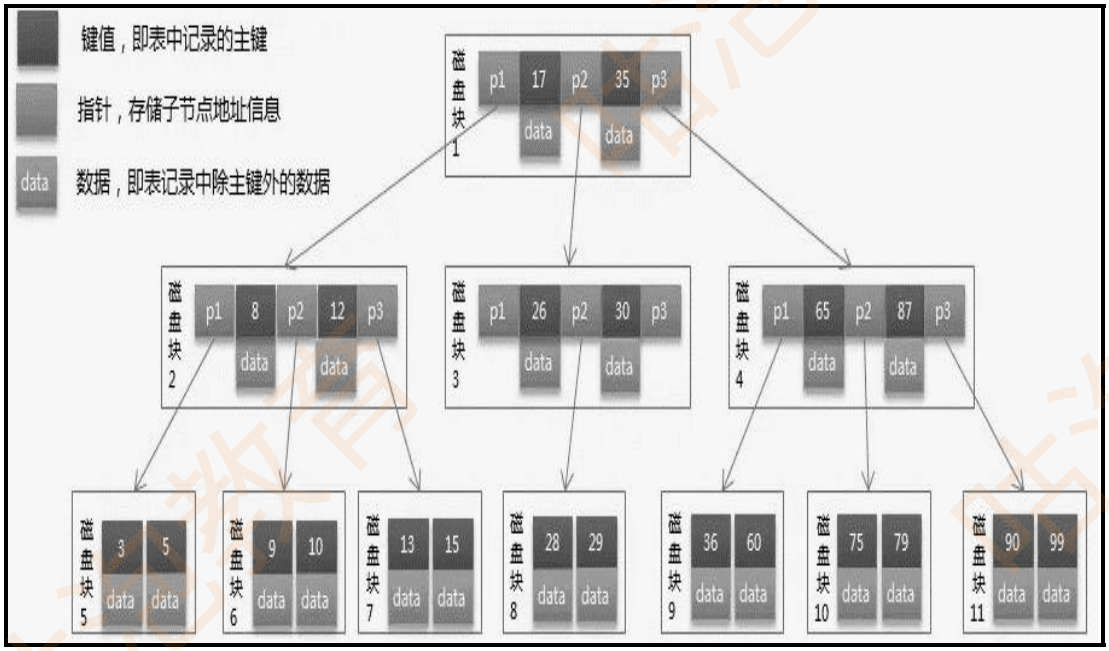


B+树，其实是在B树的基础上做的增强，最大的区别有两个：

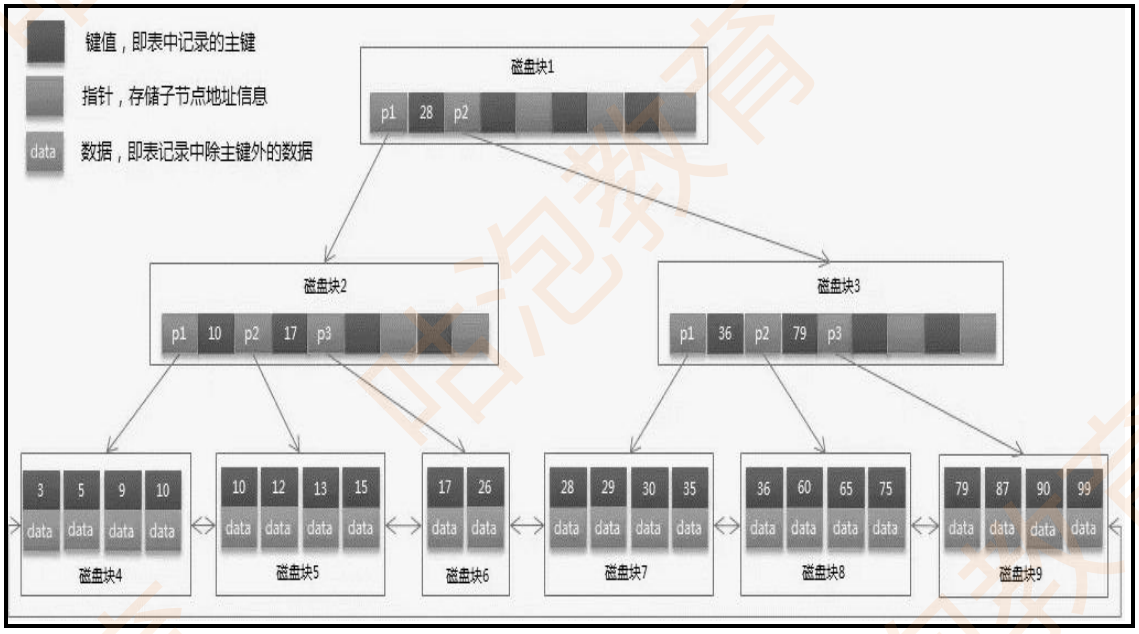
B树的数据存储在每个节点上，而B+树中的数据是存储在叶子节点，并且通过链表的方式把叶子节点中的数据进行连接。

B+树的子路数量等于关键字数

这个是B树的存储结构，从B树上可以看到每个节点会存储数据。



这个是B+树，B+树的所有数据是存储在叶子节点，并且叶子节点的数据是用双向链表关联的。



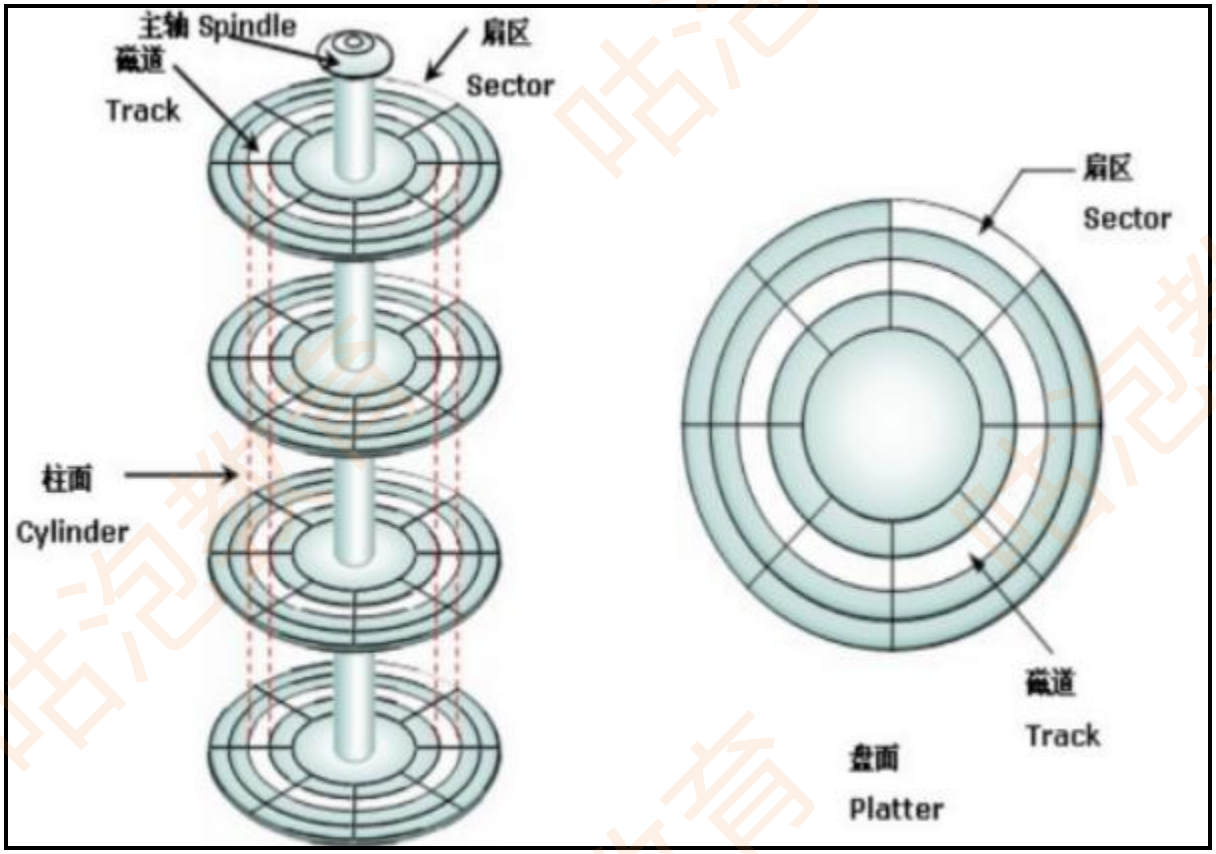
B树和B+树，一般都是应用在文件系统和数据库系统中，用来减少磁盘IO带来的性能损耗。

以Mysql中的InnoDB为例，当我们通过select语句去查询一条数据时，InnoDB需要从磁盘上去读取数据，这个过程会涉及到磁盘IO以及磁盘的随机IO

我们知道磁盘IO的性能是特别低的，特别是随机磁盘IO。

因为，磁盘IO的工作原理是，首先系统会把数据逻辑地址传给磁盘，磁盘控制电路按照寻址逻辑把逻辑地址翻译成物理地址，也就是确定要读取的数据在哪个磁道，哪个扇区。

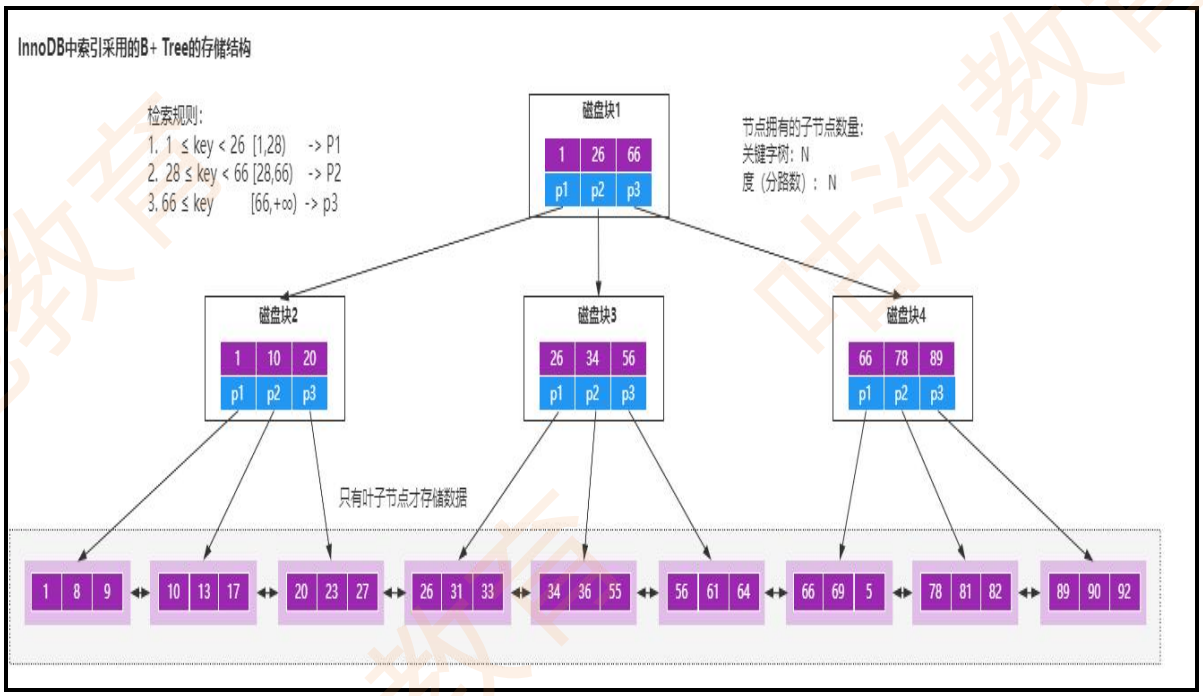
为了读取这个扇区的数据，需要把磁头放在这个扇区的上面，为了实现这一个点，磁盘会不断旋转，把目标扇区旋转到磁头下面，使得磁头找到对应的磁道，这里涉及到寻道事件以及旋转时间。



很明显，磁盘IO这个过程的性能开销是非常大的，特别是查询的数据量比较多的情况下。

所以在InnoDB中，干脆对存储在磁盘块上的数据建立一个索引，然后把索引数据以及索引列对应的磁盘地址，以B+树的方式来存储。

如图所示，当我们需要查询目标数据的时候，根据索引从B+树中查找目标数据即可，由于B+树分路较多，所以只需要较少次数的磁盘IO就能查找到。



为什么用B树或者B+树来做索引结构？原因是AVL树的高度要比B树的高度要高，而高度就意味着磁盘IO的数量。所以为了减少磁盘IO的次数，文件系统或者数据库才会采用B树或者B+树。

以上就是我对B树和B+树的理解！

结尾

数据结构在实际开发中非常常见，比如数组、链表、双向链表、红黑树、跳跃表、B树、B+树、队列等。

在我看来，数据结构是编程中最重要的基本功之一。

学了顺序表和链表，我们就能知道查询操作比较多的场景中应该用顺序表，修改操作比较多的场景应该使用链表。

学了队列之后，就知道对于FIFO的场景中，应该使用队列。

学了树的结构后，会发现原来查找类的场景，还可以更进一步提升查询性能。

基本功决定大家在技术这个岗位上能够走到的高度。

为什么使用B+树而不是B树

- 在 B 树中，键和值即存放在内部节点又存放在叶子节点；在 B+树中，内部节点只存键，叶子节点则同时存放键和值。
- B+树的叶子节点有一条链相连，而 B 树的叶子节点各自独立的。
- B+树索引的所有数据均存储在叶子节点，而且数据是按照顺序排列的，链表连着的。那么 B+树使得范围查找，排序查找，分组查找以及去重查找变得异常简单。.
- B+树非叶子节点上是不存储数据的，仅存储键值，而 B 树节点中不仅存储键值，也会存储数据。innodb 中页的默认大小是 16KB，如果不存储数据，那么就会存储更多的键值，相应的树的阶数（节点的子节点树）就会更大，树就会更矮更胖，如此一来我们查找数据进行磁盘的 IO 次数有会再次减少，数据查询的效率也会更快.

TOM老师解析

今天，我给大家介绍一种面试中经常被问到数据结构树。大家可能也经常会听到二叉树、二叉查找树、AVL平衡二叉树、B树、 等等，那今天我给大家一次性讲清楚。

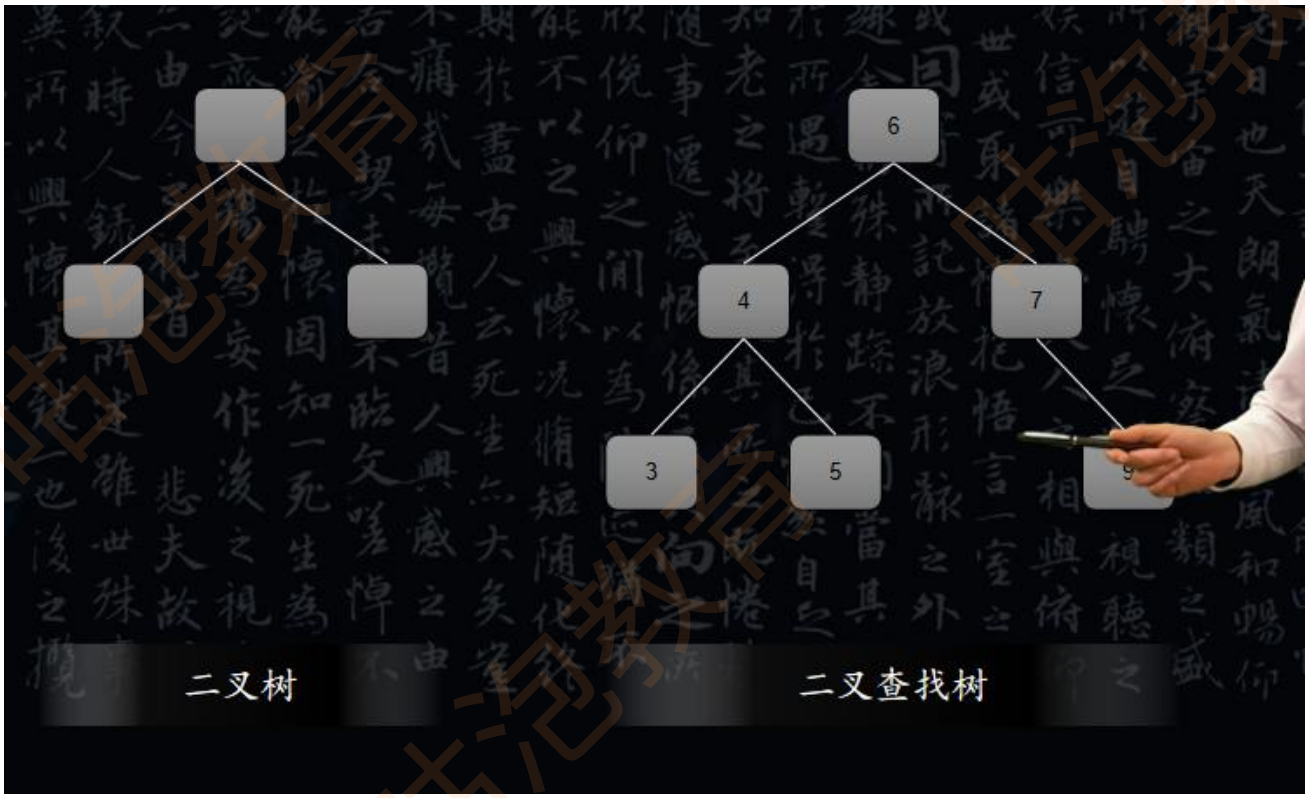
另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以在我的个人煮叶简介中找到。

1、树形结构的演变历史

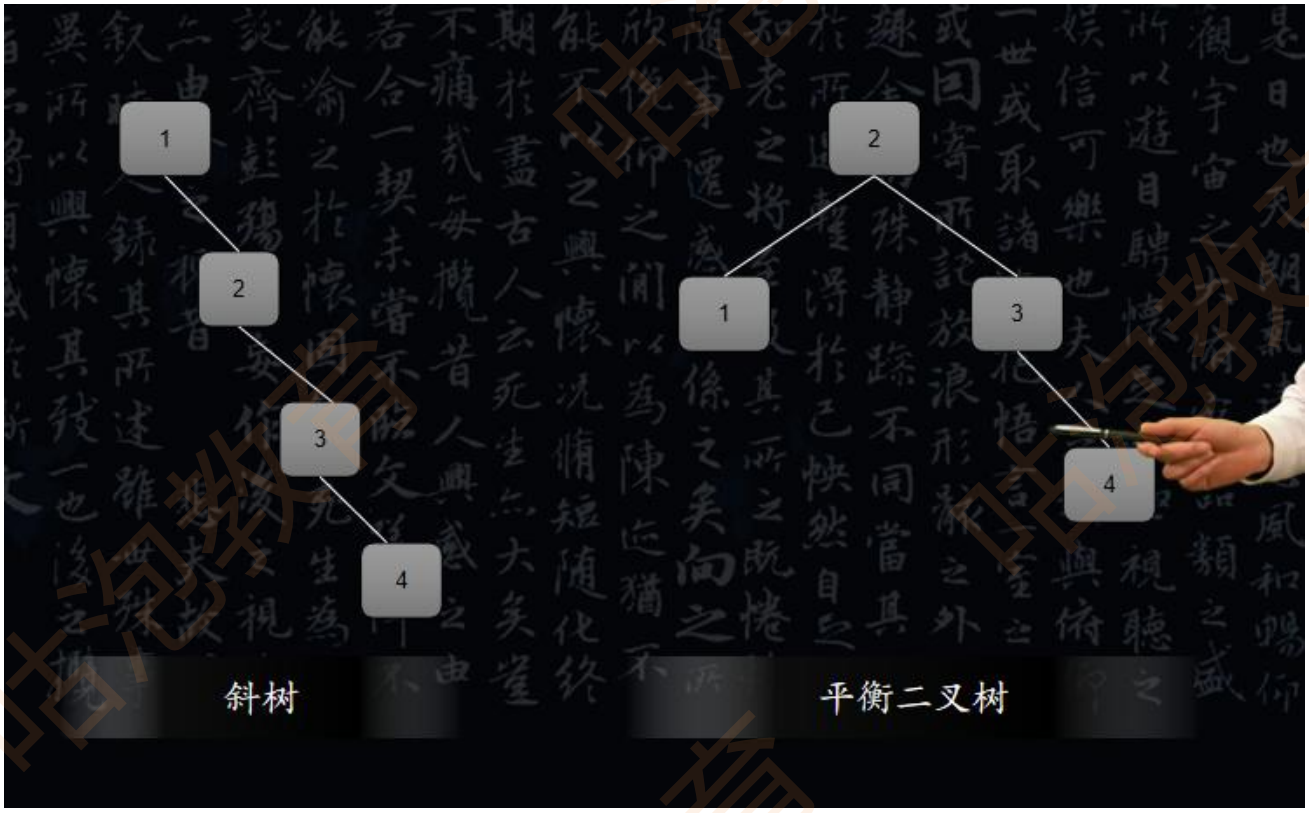
树是一种数据结构，它的结构形状如同一棵树木，但是是倒立的状态。

树的分叉就相当于树形数据结构中的节点，树上的节点可以从树根无限延伸。如图所示：

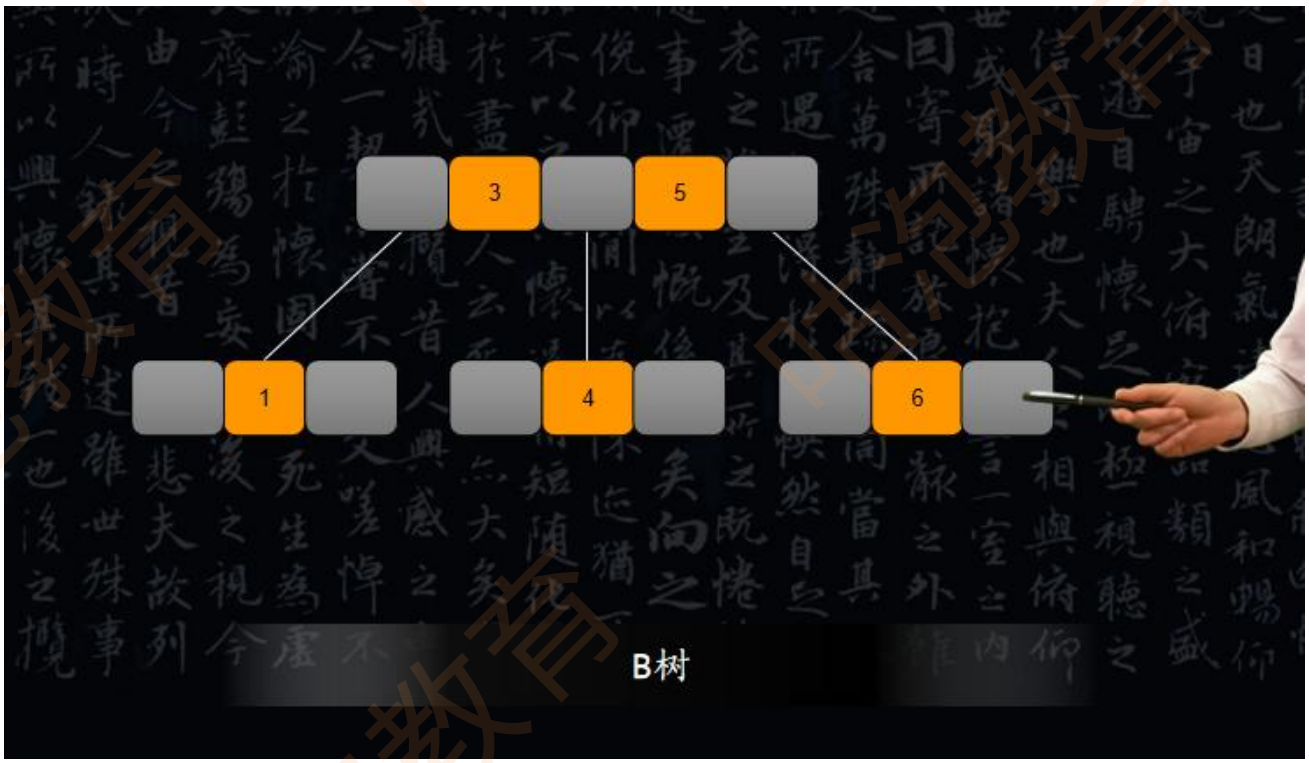
但是，这种无限延伸的树在实际开发中一般用于可无限扩展的树形功能菜单。这样的无限级、无限宽扩展的结构查询性能会比较低。因此，为了提高树形结构的查询性能，于是就出现了二叉树，来看这么一张图：



所谓二叉树是指每个节点最多支持两个分叉，相比于单向链表来说它多了一个分支。而二叉查找树，是在二叉树的基础上增加一个规则。它的规则是左子树的所有子节点都要小于它的根节点，而右侧子节点要大于它的根节点。



我们再来看这个图，二叉查找树有可能会出现斜树问题，从而导致时间复杂度会增加。因此，又引入了一种叫做平衡二叉树的机制。它具有二叉查找树的所有特点，同时还增加了一个规则。它的规则是左右两个子树的高度差的绝对值不能超过1。 为了达到这样一个平衡，所以它会引入一个左旋和右旋的机制，来实现树的平衡。



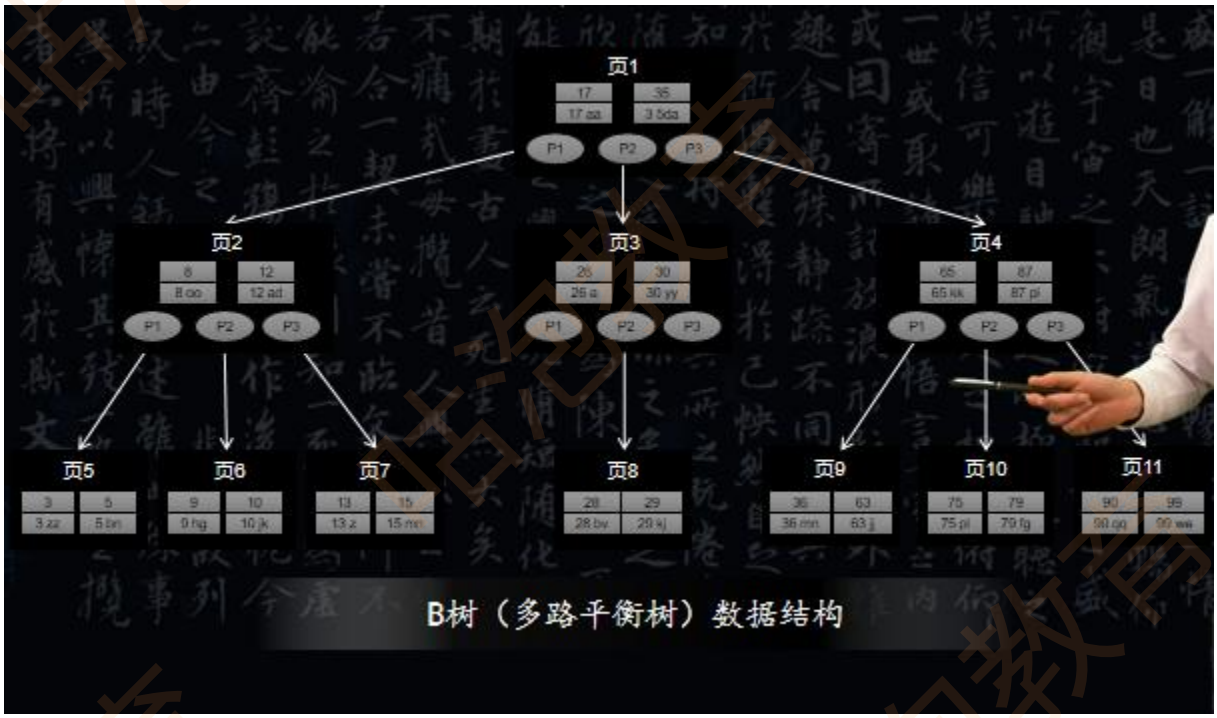
我们再来看这张图，这是B树。它是一种多路平衡查找树。它既满足平衡二叉树的规则，又可以有多个子路。子路的数量呢取决于它的关键字的数量。如图所示，根节点中有两个关键字3和5，那么它能够拥有的子路数量等于关键字的数量加1。因此，从这个特征来看，在存储同样数量的情况下，平衡二叉树的高度一定要大于B树的高度。

2、B树和B+树的区别

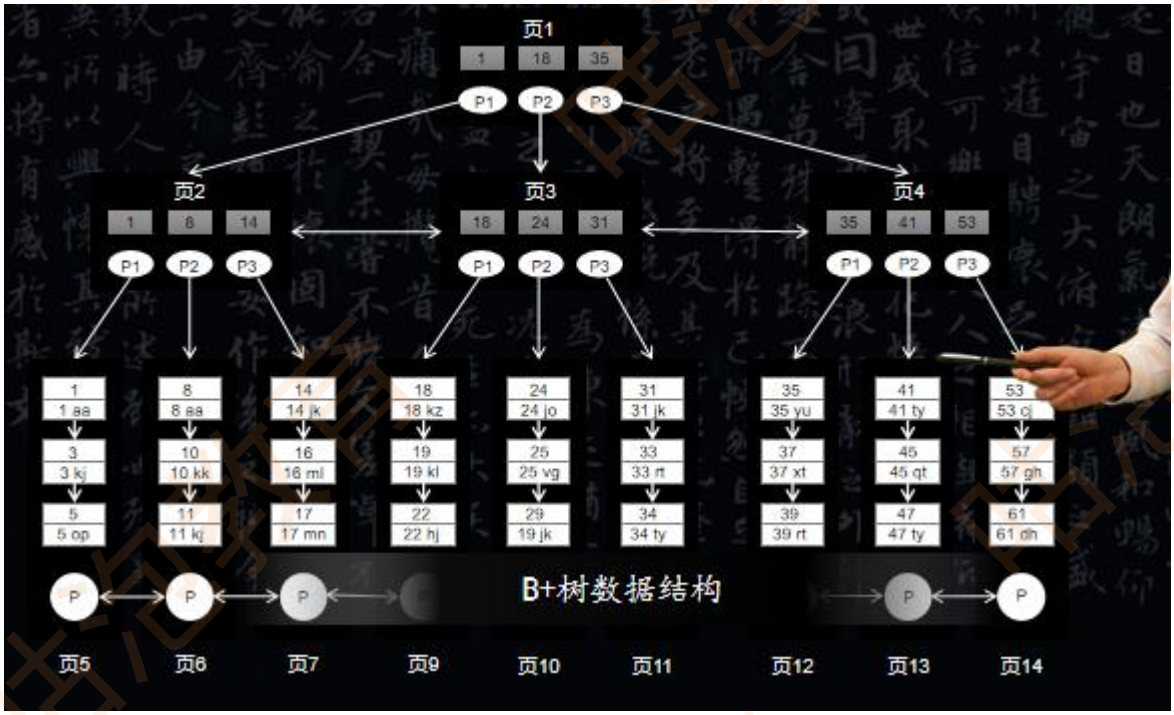
B+树呢，其实是在B树的基础上做了增强，和B树有两个最大的区别：

第1个：B树的数据存储在每个节点上，而B+树中的数据只存储在叶子节点上，并且通过链表的方式将所有叶子节点全部串联起来

第2个：B+树的子树数量等于它的关键字的数量，而B树是关键字数量加1。我们来看这个图：



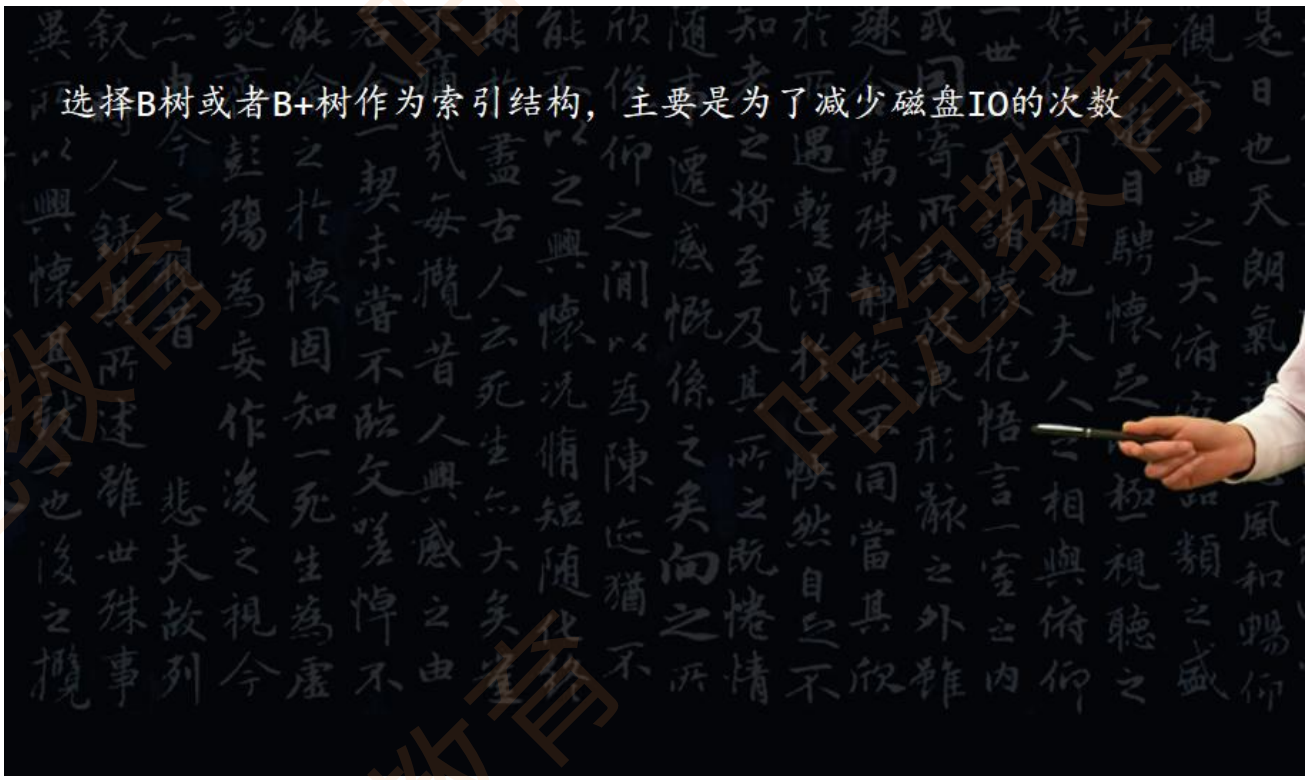
这个是B树的存储结构。从B树的结构上可以看到，每个节点都会存储数据。我们再来看这个图：



这是一个B+树的结构。B+树的数据是存储在叶子节点上的，并且呢，叶子节点的数据是用双向链表来关联。

3、选择B树和B+树的理由

那为什么要用B树或者B+树来做索引结构呢?



因为AVL树的高度要比B树或者B+树的高度更高，而高度就意味着磁盘IO的数量，所以，为了减少磁盘IO的次数，文件系统或者数据库才会使用B树，或者B+树来做索引结构。

在比较经典的程序应用中，MongoDB使用的是B树，MongoDB中所有的节点都有Data域，只要找到指定索引就可以进行访问，无疑单次查询会更快。

而MySQL作为一个关系型数据库，数据的关联性是非常强的，区间访问是常见的一种情况，B+树由于数据全部存储在叶子节点，并且通过指针串在一起，这样就很容易的进行区间遍历甚至全部遍历。

以上就是我对B树和B+树的理解。程序的本质就是数据结构加算法。数据结构在实际开发中非常常见，比如数组、链表、双向链表、红黑树、跳跃表、B树、B+树、队列等。所以，数据结构是编程最重要的基本功之一，很多大厂面试也经常会问到。同时，基本功也是决定大家在技术路上能够达到的高度的重要因素。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

1.45. 覆盖索引、回表等这些，了解过吗?

- 覆盖索引： 查询列要被所✂的索引覆盖，不必从数据表中读取，换句话说查询列要被所使用的索引覆盖。
- 回表：二级索引无法直接查询所有列的数据，所以通过二级索引查询到聚簇索引后，再查询到想要的数据，这种通过二级索引查询出来的过程，就叫做回表。

1.46. 非聚簇索引一定会回表查询吗?

不一定，如果查询语句的字段全部命中了索引，那么就不必再进行回表查询（哈哈，覆盖索引就是这么回事）。

举个简单的例子，假设我们在学生表的上✂立了索引，那么当进行 select age from student where age < 20 的查询时，在索引的叶子节点上，已经包含了age 信息，不会再次进行回表查询。

1.47. 联合索引是什么？为什么需要注意联合索引中的顺序?

联合索引，用户可以在多个列上✂立索引,这种索引叫做联合索引。

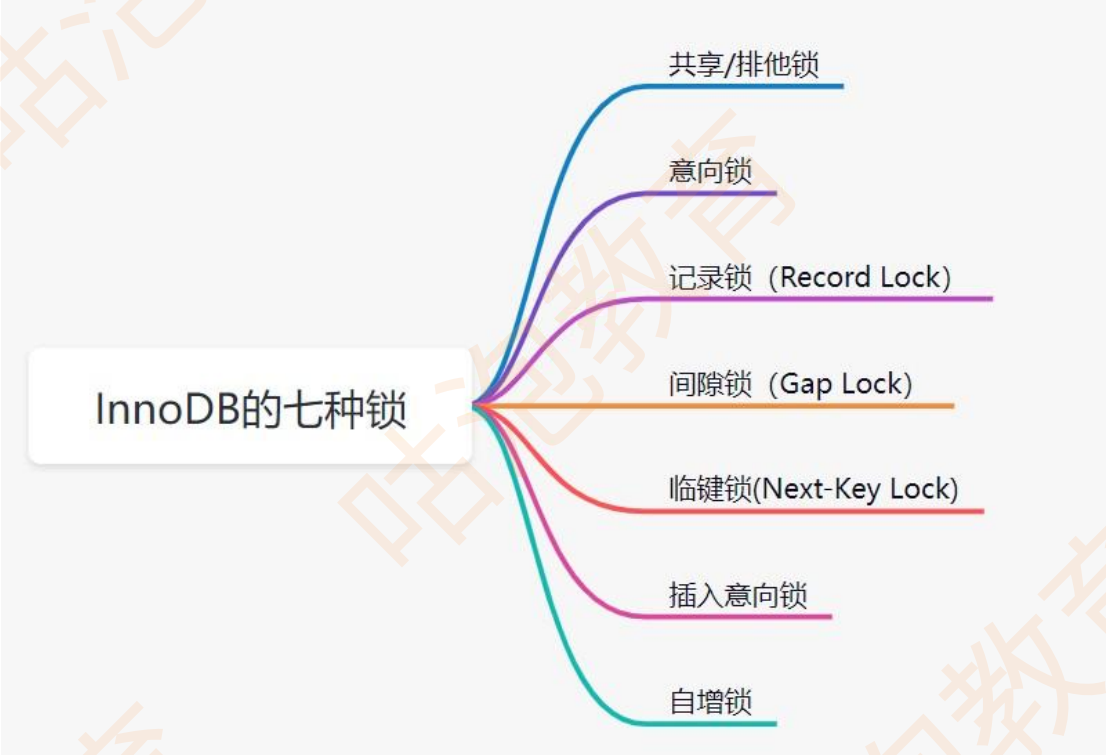
因为InnoDB 引擎中的索引策略的最左原则，所以需要注意联合索引中的顺序。

1.48. 数据库隔离级别与锁的关系

回答这个问题，可以先阐述四种隔离级别，再阐述它们的实现原理。隔离级别就是依赖锁和 MVCC 实现的。

1.49. 聊聊 InnoDb 的锁，什么是死锁？怎么解决？

InnoDb 的锁



共享/排他锁

InnoDB 呢实现了两种标准的**行级锁**：共享锁（简称 S 锁）、排他锁（简称 X 锁）。

- 共享锁：简称为 S 锁，**在事务要读取一条记录时，需要先获取该记录的 S 锁。**
- 排他锁：简称 X 锁，**在事务需要改动一条记录时，需要先获取该记录的 X 锁。**

如果事务 T1持有行 R 的 S锁，那么另一个事务 T2请求访问这条记录时，会做如下处理：

- T2 请求 S锁立即被允许，结果 T1和 T2都持有 R 行的 S锁
- T2 请求 X锁不能被立即允许,此操作阻塞

如果 T1持有行 R 的 X锁，那么 T2请求 R 的 X、S锁都不能被立即允许，T2 必须等待T1释放X锁才可以，因为 X锁与任何的锁都不兼容。
S锁和 X锁的兼容关系如下：

X锁和 S锁是对于行记录来说的话，可以称它们为**行级锁或者行锁。**我们认为行锁的粒度就比较细，其实一个事务也可以在**表级别下加锁**，我们称之为表锁。给表加的锁，也是可以分为 X锁和 S锁的哈。

如果一个事务给表已经加了 S 锁，则：

- 别的事务可以继续获得该表的 S 锁，也可以获得该表中某些记录的 S 锁。
- 别的事务不可以继续获得该表的 X 锁，也不可以获得该表中某些记录的 X 锁。

如果一个事务给表加了 X 锁，那么

- 别的事务不可以获得该表的 S 锁，也不可以获得该表某些记录的 S 锁。
- 别的事务不可以获得该表的 X 锁，也不可以继续获得该表某些记录的 X 锁。

意向锁

什么是意向锁呢？意向锁是**一种不与行级锁冲突的表级锁**。未来的某个时刻，事务可能要加共享或者排它锁时，先提前声明一个意向。注意一下，意向锁，是一个**表级别的锁哈。**

为什么需要意向锁呢？ 或者换个通俗的说法，为什么要加共享锁或排他锁时的时候，需要提前声明个意向锁呢呢？

因为 InnoDB 是支持表锁和行锁共存的，如果一个事务 A 获取到某一行的排他锁，并未提交，这时候事务 B 请求获取同一个表的表共享锁。因为**共享锁和排他锁是互斥的**，因此事务 B 想对这个表加共享锁时，需要保证没有其他事务持有这个表的表排他锁，同时还要保**证没有其他事务持有表中任意一行的排他锁**。然后问题来了，你要保证没有其他事务持有表中任意一行的排他锁的话，去遍历每一行？这样显然是一个效率很差的做法。**为了解决这个问题，InnoDb 的设计大叔提出了意向锁。**

意向锁是如何解决这个问题的呢？ 我们来看下
意向锁分为两类：

- 意向共享锁：简称 IS锁，当事务准备在某些记录上加 S 锁时，需要现在表级别加一个 IS锁。
- 意向排他锁：简称 IX锁，当事务准备在某条记录上加上 X 锁时，需要现在表级别加一个 IX锁。

比如：

- select ... lock in share mode，要给表设置 IS锁;
- select ... for update，要给表设置 IX锁;

意向锁又是如何解决这个效率低的问题呢：

如果一个事务 A 获取到某一行的排他锁，并未提交，这时候表上就有意向排他锁 和这一行的排他锁。这时候事务 B 想要获取这个表的共享锁，此时因为检测到事务 A 持有了表的意向排他锁，因此事务 A 必然持有某些行的排他锁，也就是说事务 B 对表的加锁请求需要阻塞等待，不再需要去检测表的每一行数据是否存

在排他锁啦。

意向锁仅仅表明意向的锁，意向锁之间不会互斥，是可以并行的，整体兼容性如下：

记录锁（Record Lock）

记录锁是最简单的行锁，仅仅锁住一行。如：SELECT c1 FROM t WHERE c1 = 10 FOR UPDATE，如果 C1 字段是主键或者是唯一索引的话，这个 SQL 会加一个记录锁（Record Lock）

记录锁永远都是加在索引上的，即使一个表没有索引，InnoDB 也会隐式的创✂一个索引，并使用这个索引实施记录锁。它会阻塞其他事务对这行记录的插入、更新、删除。

一般我们看死锁日志时，都是找关键词，比如 lock_mode X locks rec but not gap），就表示一个 X 型的记录锁。记录锁的关键词就是 **rec but not gap**。以下就是一个记录锁的日志：

```
RECORD LOCKS space id 58 page no 3 n bits 72 index `PRIMARY` of table `test`.`t` trx id 10078 lock_mode X locks rec but not gap
Record lock, heap no 2 PHYSICAL RECORD: n_fields 3; compact format; info bits 0
 0: len 4; hex 8000000a; asc     ;;
 1: len 6; hex 00000000274f; asc     'O;;
 2: len 7; hex b60000019d0110; asc        ;;
```

间隙锁（Gap Lock）

为了解决幻读问题，InnoDB 引入了间隙锁(Gap Lock)。间隙锁是一种加在两个索引之间的锁，或者加在第一个索引之前，或最后一个索引之后的间隙。它锁住的是**一个区间**，而不仅仅是这个区间中的每一条数据。

比如 lock_mode X locks gap before rec表示X 型 gap 锁。以下就是一个间隙锁的日志：

```
RECORD LOCKS space id 177 page no 4 n bits 80 index idx_name of table `test2`.`account` trx id 38049 lock_mode X locks gap before rec
Record lock, heap no 6 PHYSICAL RECORD: n_fields 2; compact format; info bits 0
 0: len 3; hex 576569; asc Wei;;
 1: len 4; hex 80000002; asc     ;;
```

临键锁(Next-Key Lock)

Next-key 锁是**记录锁和间隙锁的组合**，它指的是加在某条记录以及这条记录前面间隙上的锁。说得更具体一点就是:临键锁会封锁索引记录本身，以及索引记录之前的区间，即它的锁区间是前开后闭，比如(5,10]。

如果一个会话占有了索引记录 R 的共享/排他锁，其他会话不能立刻在 R 之前的区间插入新的索引记录。

If one session has a shared or exclusive lock on record R in an index, another session cannot insert a new index record in the gap immediately before R in the index order.

插入意向锁

插入意向锁,是插入一行记录操作之前设置的**一种间隙锁**，这个锁释放了一种插入方式的信号。 它解决的问题：多个事务，在同一个索引，同一个范围区间插入记录时，如果插入的位置不冲突，不会阻塞彼此。

假设有索引值 4、7，几个不同的事务准备插入 5、6，每个锁都在获得插入行的独占锁之前用插入意向锁各自锁住了 4、7 之间的间隙，但是不阻塞对方因为插入行不冲突。以下就是一个插入意向锁的日志：

```
RECORD LOCKS space id 31 page no 3 n bits 72 index `PRIMARY` of table `test`.`child
`
trx id 8731 lock_mode X locks gap before rec insert intention waiting
Record lock, heap no 3 PHYSICAL RECORD: n_fields 3; compact format; info bits 0
 0: len 4; hex 80000066; asc     f;;
 1: len 6; hex 000000002215; asc     " ;;
 2: len 7; hex 9000000172011c; asc        r  ;;...
```

锁模式兼容矩阵（横向是已持有锁，纵向是正在请求的锁）：

自增锁

自增锁是一种特殊的表级别锁。它是专门针对 AUTO_INCREMENT类型的列，对于这种列，如果表中新增数据时就会去持有自增锁。简言之，如果一个事务正在往表中插入记录，所有其他事务的插入必须等待，以便第一个事务插入的行，是连续的主键值。

官方文档是这么描述的：

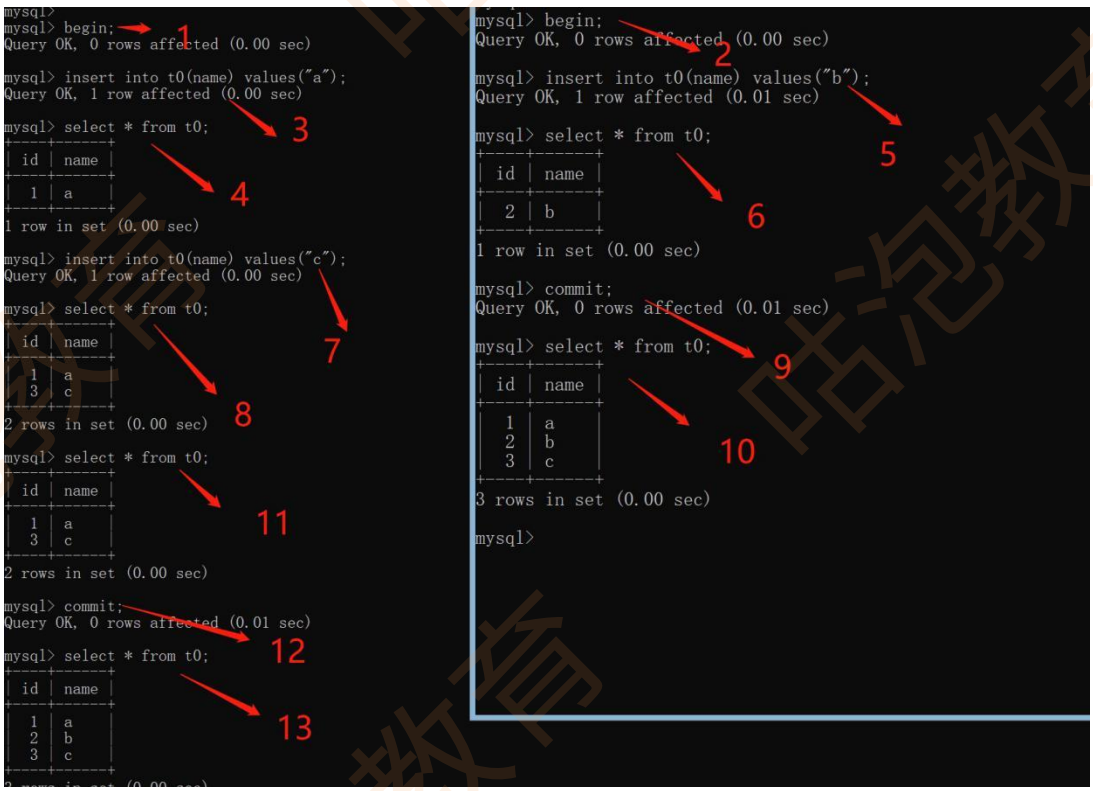
An AUTO-INC lock is a special table-level lock taken by transactions inserting into tables with AUTO_INCREMENT columns. In the simplest case, if one transaction is inserting values into the table, any other transactions must wait to do their own inserts into that table, so that rows inserted by the first transaction receive consecutive primary key values.

假设有表：

```
mysql> create table t0 (id int NOT NULL AUTO_INCREMENT,name varchar(16),primary key ( id));
```

```
+-------------------------------+--------+
mysql> show variables like '%innodb_autoinc_lock_mode%';
| Variable_name                 | Value |
+-------------------------------+--------+
+-------------------------------+--------+
+-------------------------------+--------+
| innodb_autoinc_lock_mode | 1          |
1 row in set, 1 warning (0.01 sec)
```

设置事务 A 和 B 交替执行流程如下：



通过上图我们可以看到，当我们在事务 A 中进行自增列✂插入操作时，另外会话事务 B 也进行插入操作，这种情况下会发生 2 个奇怪✂现象：

- 事务 A 会话中✂自增列好像直接增加了 2 个值。（如上图中步骤 7、8）
- 事务 B 会话中✂自增列直接从 2 开始增加✂。（如上图步骤 5、6）

自增锁是一个表级别锁，那为什么会话 A 事务还没结束，事务 B 可以执行插入成功呢？不是应该锁表嘛？这是因为在参数 innodb_autoinc_lock_mode上，这个参数设置为 1✂时候，相当于将这种 auto_inc lock弱化为了一个更轻量级✂互斥自增长机制去实现，官方称之为 mutex。

innodb_autoinc_lock_mode 还可以设置为 0 或者 2，

- **0**：表示传统锁模式，使用表级 AUTO_INC锁。一个事务✂ INSERT-LIKE语句在语句执行结束后释放 AUTO_INC 表级锁，而不是在事务结束后释放。
- **1**: 连续锁模式,连续锁模式对于 Simple inserts不会使用表级锁，而是使用一个轻量级锁来生成自增值，因为 InnoDB 可以提前直到插入多少行数据。自增值生成阶段使用轻量级互斥锁来生成所有✂值，而不是一直加锁直到插入完成。对于bulk inserts类语句使用 AUTO_INC 表级锁直到语句完成。
- ** 2**:交错锁模式,所有✂ INSERT-LIKE语句都不使用表级锁，而是使用轻量级互斥锁。

- **INSERT-LIKE**:指所有✂插入语句，包括： INSERT、REPLACE、INSERT...SELECT、REPLACE...SELECT,LOAD DATA 等。
- **Simple inserts**:指在插入前就能确定插入行数✂语句，包括：INSERT、REPLACE，不包含 INSERT...ON DUPLICATE KEY UPDATE 这类语句。
- **Bulk inserts**: 指在插入钱不能确定行数✂语句，包括：INSERT ... SELECT/REPLACE ... SELECT/LOAD DATA。

什么是死锁？怎么解决？

MIC老师解析

看一看普通人

和高手是如何回答这个问题的

普通人

线程A占用对象锁1，线程B占用对象锁2

线程A需要继续获得对象锁2才能继续执行，所以线程A需要等待

线程B释放对象锁2线程B需要获得对象锁1，才能继续执行

同样也需要等待

线程A释放对象锁2

由于这两个线程

都不释放自己已经占有的锁

导致两个线程处于

无限等待状态

这个就是死锁

高手

关于这个问题

我会从三个方面来回答

第一个是什么是死锁

所谓死锁

是一组互相竞 争资源的线程

因互相等待

导致“永久”阻塞的现象

第二个是发生死锁的原因

发生死锁的原因有四个

第一个是互斥条件，共享资源 X 和 Y 只能被一个线程占用

第二个是指 占有且等待，线程 T1 已经取得共享资源 X

在等待共享资源 Y 的时候

不释放共享资源 X

第三个是不可抢占

其他线程不能强行抢占

线程 T1 占有的资源

第四个循环等待

线程 T1 等待线程 T2 占有的资源

线程 T2 等待线程 T1 占有的资源

这就是循环等待

第三个点是如何避免死锁呢?

既然发生死锁的原因是

需要同时满足这四个条件

我们只需要打破其中任意一个条件

即可避免死锁问题

而在这四个条件中

第一个互斥条件是无法被破坏的

因为锁本身就是通过

互斥来解决线程安全问题的

所以对于剩下三个

我们可以逐一进行分析

第一个是对于“占用且等待”这个条件

我们可以一次性申请所有的资源

这样就不存在等待了

第二个是对于“不可抢占”这个条件

占用部分资源的线程

进一步申请其他资源时

如果申请不到

可以主动释放它占有的资源

这样不可抢占这个条件就破坏掉了

第三个点

对于“循环等待”这个条件

可以靠按序申请资源来进行预防

所谓按序申请

是指资源是有线性顺序的

申请的时候可以先申请资源序号小的

再申请资源序号大的

这样线性化后自然就不存在循环等待了

怎么样

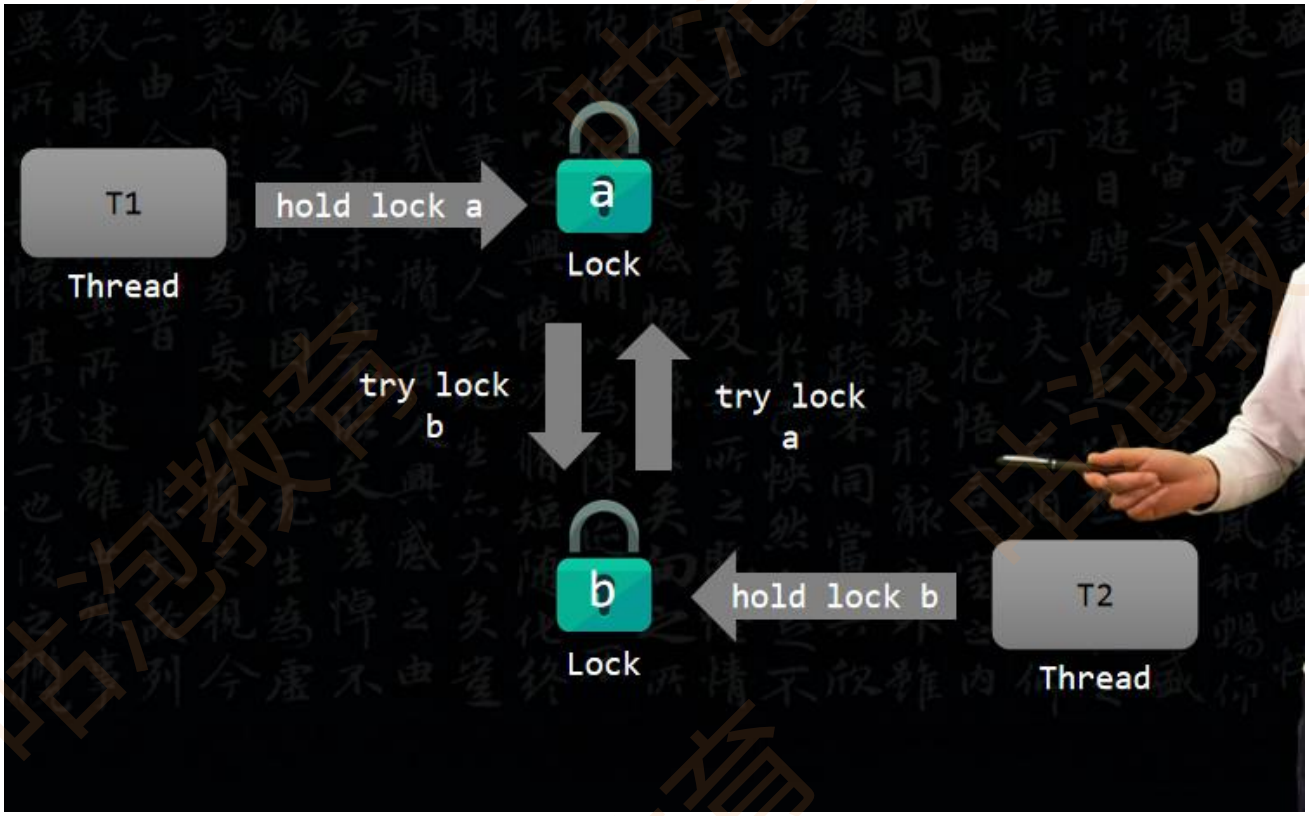
发现他们两个的区别了吗

什么是死锁?

TOM老师解析:

先来看这样一段视频,

看完你也许已经懂了，那到底什么是死锁呢?



死锁，简单来说就是两个或者两个以上的线程在执行过程中，去争夺同一个共享资源导致相互等待的现象。如果没有外部干预，线程会一直处于阻塞状态，无法往下执行。这样一直等待处于阻塞状态的线程，被称为死锁线程。

如何解决死锁问题?

Hi，大家好，我是Mic，一个工作了14年的Java程序员

这个问题，面试官的考察目的是什么呢?

考察目标

这个问题还是有一点难度，首先他考察的是并发编程相关领域的知识。

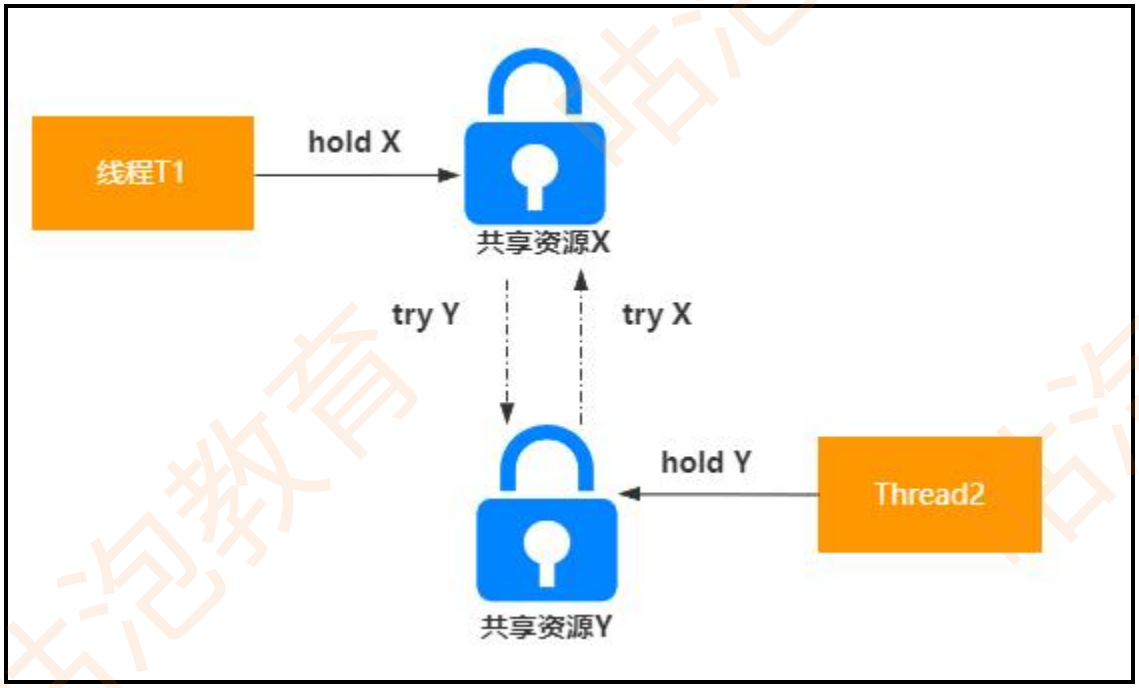
其次，对于死锁这个问题，平时我们遇到得比较少，即便是看过相关的问题，也不一定能够记住

所以，在一定程度上，能够回答清楚这个问题，说明这个求职者的基本功还不错。

问题解析

死锁，就是两个或者两个以上的线程在执行过程中，去争夺同一个共享资源导致互相等待的现象。

在没有外部干预的情况下，线程会一直处于阻塞状态，无法往下执行。



不过，要想真正产生死锁，必须同时满足四个条件。

互斥条件，共享资源x和y只能被一个线程占用

请求和保持条件， 线程t1已经获取共享资源x，在等待共享资源y的时候，不释放共享资源x不可抢占条件，其他线程不能强行抢占线程t1占有的资源

循环等待条件，线程t1等到线程t2占有的资源，线程t2等待线程t1占有的资源，形成循环等待

线程在产生死锁以后，只能通过外部干预来解决，比如重启、或者kill线程等。

所以我们在写代码的时候，就应该去刻意规避死锁的问题。

也就是避免同时满足这四个条件。

在这四个条件里面，互斥条件是锁本身的特性，无法被破坏，其他三个条件都可以被破坏。

对于请求和保持条件，我们可以在第一次执行的时候一次性申请所有的共享资源

对于不可抢占条件，占用部分资源的线程在进一步申请其他资源的时候，如果申请不到，就主动释放它占有的资源。

对于循环等待条件，可以按照顺序来申请资源，相当于给资源编号，按照编号顺序申请就可以避免循环等待。

当然，死锁问题不仅仅局限在多线程领域，单反涉及到互斥锁的地方都有可能出现，

比如Mysql数据库的行锁、表锁，以及分布式锁等。

在底层原理上都是相同的。

下面来看看高手的回答。

高手回答

程序出现死锁，是因为在多线程环境里面两个或两个以上的线程同时满足

互斥条件、请求保持条件、不可抢占条件、循环等待条件。

出现死锁以后，可以通过jstack命令去导出线程的dump日志，

然后从dump日志里面定位到具体死锁的程序代码。

通过修改程序代码去破坏这四个条件里面的任意一个，就可以解决死锁问题。

当然，因为互斥条件因为是锁本身的特性，所以不能被破坏。

以上就是我的理解。

总结

下次面试的时候遇到这个问题，大家知道怎么回答了吗?

如果你喜欢我的作品，记得点赞收藏加关注哦

1.50. count(1)、count(*) 与 count(列名) 的区别?

要点:

- count(*)包括了所有✂列，相当于行数，在统计结果✂时候，不会忽略列值为NULL
- count(1)包括了忽略所有列，用 1 代表代码行，在统计结果✂时候，不会忽略列值为NULL
- count(列名)只包括列名那一列，在统计结果✂时候，会忽略列值为空（这里✂空不是只空字符串或者 0，而是表示 null）✂计数，即某个字段值为 NULL 时，不统计。

高手回答:

count，其实是一个聚合函数，聚合数据的总数。

不同的存储引擎实现的方式可能又稍许不一样，我们以默认的innoDB存储引擎为例。

其实就是根据where条件去进行扫描，得到扫描结果后，一行一行去判断，如果count括号里的不是null，那么累计值+1，否则不加，最后返回一个累计的总数。

所以，括号里面的值在一定程度是会影响性能的。

count(*) ，也就是整条数据，由于整条数据肯定不会为null，所以，专门做了优化，不会去取全部字段。直接扫描多少数据统计数据即可。

count(1) 扫描到一条数据，直接返回一个数字1，不会取扫描的字段，也不会去判断是否为null。

这2个也是我们用得最多的，性能最好的。

但是如果你count(字段)，有2种情况。

1.首先去看字段是否能走索引，如果没索引，那么扫描数据不会走索引树，比有索引的字段慢。

2.字段是否可为null，如果可为null，那么必须要去判断扫描出来的数据是否为null，为null不统计，所以不可为null的比可为null的要快。

这是我对count聚合函数的理解。

灰灰老师解析

面试官：

Mysql在获取表的总数的时候，你知道count(*)、count(1)、count(字段)的区别么？

灰灰：

count，其实是一个聚合函数，聚合数据的总数。

不同的存储引擎实现的方式可能又稍许不一样，我们以默认的innoDB存储引擎为例。

其实就是根据where条件去进行扫描，得到扫描结果后，一行一行去判断，如果count括号里的不是null，那么累计值+1，否则不加，最后返回一个累计的总数。

所以，括号里面的值在一定程度是会影响性能的。

count(*) ，也就是整条数据，由于整条数据肯定不会为null，所以，专门做了优化，不会去取全部字段。直接扫描多少数据统计数据即可。

count(1) 扫描到一条数据，直接返回一个数字1，不会取扫描的字段，也不会去判断是否为null。

这2个也是我们用得最多的，性能最好的。

但是如果你count(字段)，有2种情况。

1.首先去看字段是否能走索引，如果没索引，那么扫描数据不会走索引树，比有索引的字段慢。

2.字段是否可为null，如果可为null，那么必须要去判断扫描出来的数据是否为null，为null不统计，所以不可为null的比可为null的要快。

这是我对count聚合函数的理解。

1.51. 什么是存储过程？有哪些优缺点？

存储过程，就是一些编译好了✂ SQL 语句，这些 SQL 语句代码像一个方法一样实现一些功能（对单表或多表✂增删改查），然后给这些代码块取一个名字， 在

用到这个功能✂时候调用即可。

优点：

- 存储过程是一个预编译✂代码块，执行效率比较高
- 存储过程在服务器端运行，减少客户端✂压力
- 允许模块化程序设计，只需要创✂一次过程，以后在程序中就可以调用该过程任意次，类似方法✂复用
- 一个存储过程替代大量 T_SQL 语句 ，可以降低网络通信量，提高通信速率
- 可以一定程度上确保数据安全

缺点：

- 调试麻烦
- 可移植性不灵活
- 重新编译问题

1.52. SQL 约束有哪几种呢?

- NOT NULL: 约束字段✂内容一定不能为 NULL。
- UNIQUE: 约束字段唯一性，一个表允许有多个 Unique 约束。
- PRIMARY KEY: 约束字段唯一，不可重复，一个表只允许存在一个。

- FOREIGN KEY: 用于预防破坏表之间连接✂动作，也能防止非法数据插入外键。
- CHECK: 用于控制字段✂值范围。

1.53. varchar(50)中 50 的涵义

- 字段最多存放 50 个字符
- 如 varchar(50) 和 varchar(200) 存储 "jay" 字符串所占空间是一样的，后者在排序时会消耗更多内存

1.54. mysql 中int(20)和 char(20)以及 varchar(20)的 区别

- int(20) 表示字段是 int 类型，显示长度是 20
- char(20)表示字段是固定长度字符串，长度为 20
- varchar(20) 表示字段是可变长度字符串，长度为 20

1.55. drop、delete 与truncate 的区别

	delete	truncate	drop
类型	DML	DDL	DDL
回滚	可回滚	不可回滚	不可回滚
删除内容	表结构还在，删除表的全部或者一部分数据行	表结构还在，删除表中的所有数据	从数据库中删除表，所有的数据行，索引和权限也会被删除
删除速度	删除速度慢，逐行删除	删除速度快	删除速度最快

1.56. UNION 与UNION ALL 的区别?

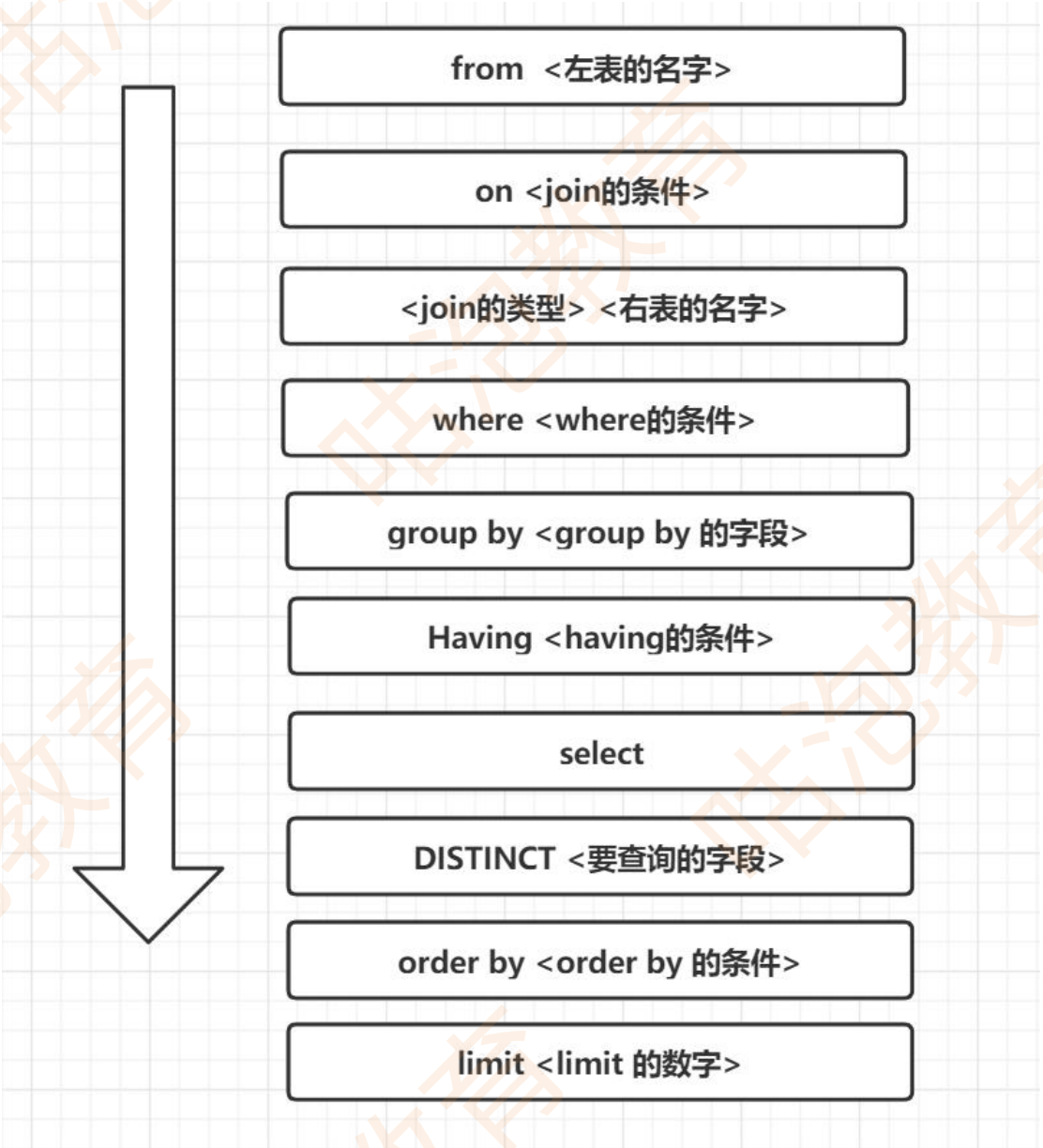
- Union：对两个结果集进行并集操作，不包括重复行，同时进行默认规则的排序；

- Union All：对两个结果集进行并集操作，包括重复行，不进行排序；
- UNION 的效率高于 UNION ALL

1.57. SQL 的生命周期?

- 服务器与数据库的立连接
- 数据库进程拿到请求 sql
- 解析并生成执行计划，执行
- 读取数据到内存，并进行逻辑处理
- 通过步骤一的连接，发送结果到客户端
- 关掉连接，释放资源

1.58. 一条 Sql 的执行顺序?



1.59. 关心过业务系统里面的 sql 耗时吗？统计过慢查询吗？对慢查询都怎么优化过？

- 我们平时写 Sql 时，都要养成用 explain 分析的习惯。
- 慢查询的统计，运维会定期统计给我们

优化慢查询：

- 分析语句，是否加载了不必要✂字段/数据。
- 分析 SQl 执行句话，是否命中索引等。
- 如果 SQL 很复杂，优化 SQL 结构
- 如果表数据量太大，考虑分表

1.60. 主键使用自增 ID 还是 UUID，为什么？mysql 自增主键用完了怎么办？

主键使用自增 ID 还是 UUID?

如果是单机✂话，选择自增 ID；如果是分布式系统，优先考虑 UUID 吧，但还是最好自己公司有一套分布式唯一 ID 生产方案吧。

- 自增 ID：数据存储空间小，查询效率高。但是如果数据量过大,会超出自增长✂值范围，多库合并，也有可能有问题。
- uuid：适合大量数据✂插入和更新操作，但是它无序✂，插入数据效率慢，占用空间大。

mysql 自增主键用完了怎么办?

自增主键一般用 int 类型，一般达不到最大值，可以考虑提前分库分表✂。

1.61. MySQL 数据库 cpu 飙升的话，要怎么处理呢?

MIC老师解析

面试过程中，场景类的问题更容易检测出一个开发人员的基本能力。

这不，一个小伙伴去阿里面试，第一面就遇到了关于“CPU飙高系统反应慢怎么排查”的问题?

对于这个问题，我们来看看普通人和高手的回答!

普通人

嗯，CPU飙高的原因可能是线程创建过多导致的

高手

好的，关于这个问题，我从四个方面来回答。

CPU是整个电脑的核心计算资源，对于一个应用进程来说，CPU的最小执行单元是线程。

导致CPU飙高的原因有几个方面

CPU上下文切换过多，对于CPU来说，同一时刻下每个CPU核心只能运行一个线程，如果有多个线程要执行，CPU只能通过上下文切换的方式来执行不同的线程。上下文切换需要做两个事情

保存运行线程的执行状态

让处于等待中的线程执行

这两个过程需要CPU执行内核相关指令实现状态保存，如果较多的上下文切换会占据大量CPU资源，从而使得cpu无法去执行用户进程中的指令，导致响应速度下降。

在Java中，文件IO、网络IO、锁等待、线程阻塞等操作都会造成线程阻塞从而触发上下文切换

CPU资源过度消耗，也就是在程序中创建了大量的线程，或者有线程一直占用CPU资源无法被释放，比如死循环!

CPU利用率过高之后，导致应用中的线程无法获得CPU的调度，从而影响程序的执行效率!

既然是这两个问题导致的CPU利用率较高，于是我们可以通过top命令，找到CPU利用率较高的进程，在通过Shift+H找到进程中CPU消耗过高的线程，这里有两种情况。

CPU利用率过高的线程一直是同一个，说明程序中存在线程长期占用CPU没有释放的情况，这种情况直接通过jstack获得线程的Dump日志，定位到线程日志后就可以找到问题的代码。

CPU利用率过高的线程id不断变化，说明线程创建过多，需要挑选几个线程id，通过jstack去线程dump日志中排查。

最后有可能定位的结果是程序正常，只是在CPU飙高的那一刻，用户访问量较大，导致系统资源不够。

以上就是我对这个问题的理解!

结尾

从这个问题来看，面试官主要考察实操能力，以及解决问题的思路。

如果你没有实操过，但是你知道导致CPU飙高这个现象的原因，并说出你的解决思路，通过面试是没问题的。

好的，本期的普通人VS高手面试系列的视频就到这里结束了，

如果你在面试的时候遇到了一些比较刁钻也奇葩的问题，欢迎在评论区给我留言，我是Mic。

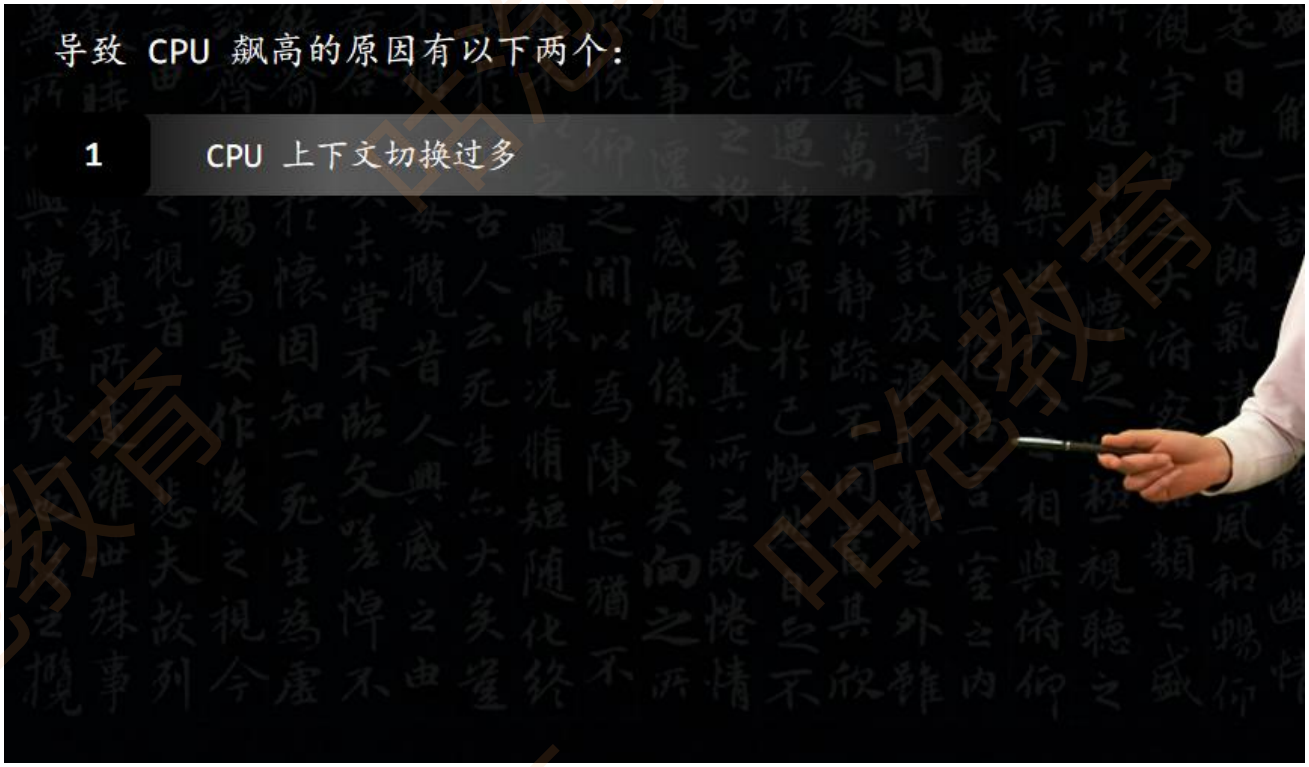
我是Mic，一个工作了14年的Java程序员，咱们下期再见。

TOM老师解析

面试过程中，场景类的问题更容易检测出一个开发人员的基本能力。这不，有一位小伙伴去阿里面试，第一面就遇到了关于“CPU 飙高系统反应慢怎么排查”的问题？当时这位小伙伴不知从何下手。

今天，我给大家分享一下我的解决思路。另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

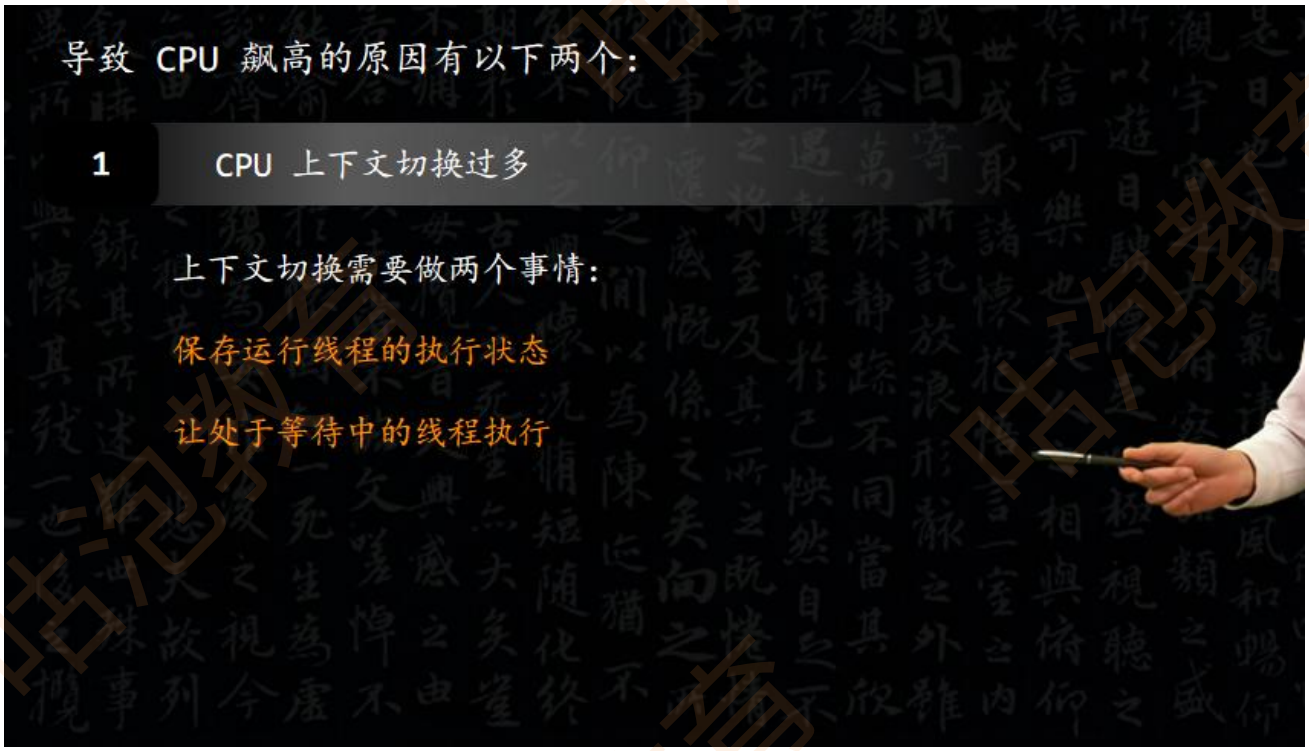
CPU 是整个电脑的核心计算资源，对于一个应用进程来说，CPU 的最小执行单元是线程。导致 CPU 飙高的原因有以下两个：



1、CPU 上下文切换过多。

对于 CPU 来说，同一时刻下每个 CPU 核心只能运行一个线程，如果有多个线程要执行，CPU

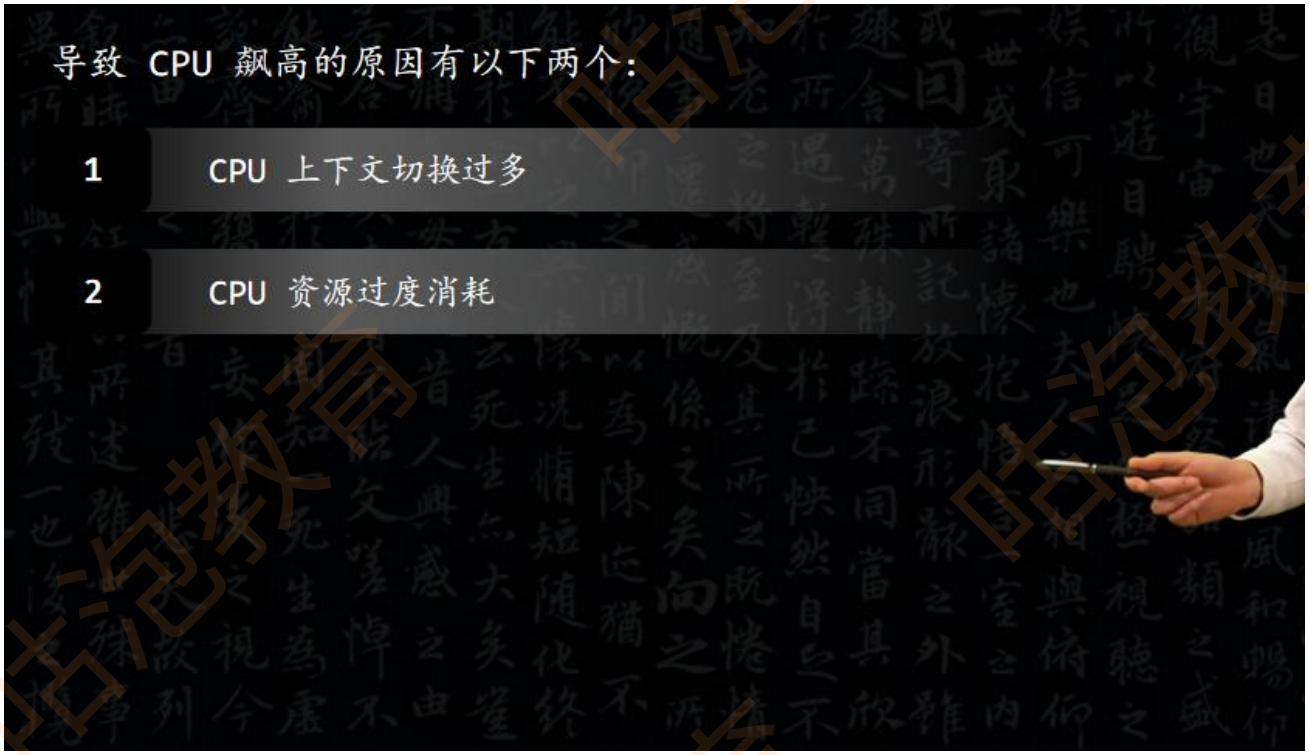
只能通过上下文切换的方式来执行不同的线程。上下文切换需要做两个事情



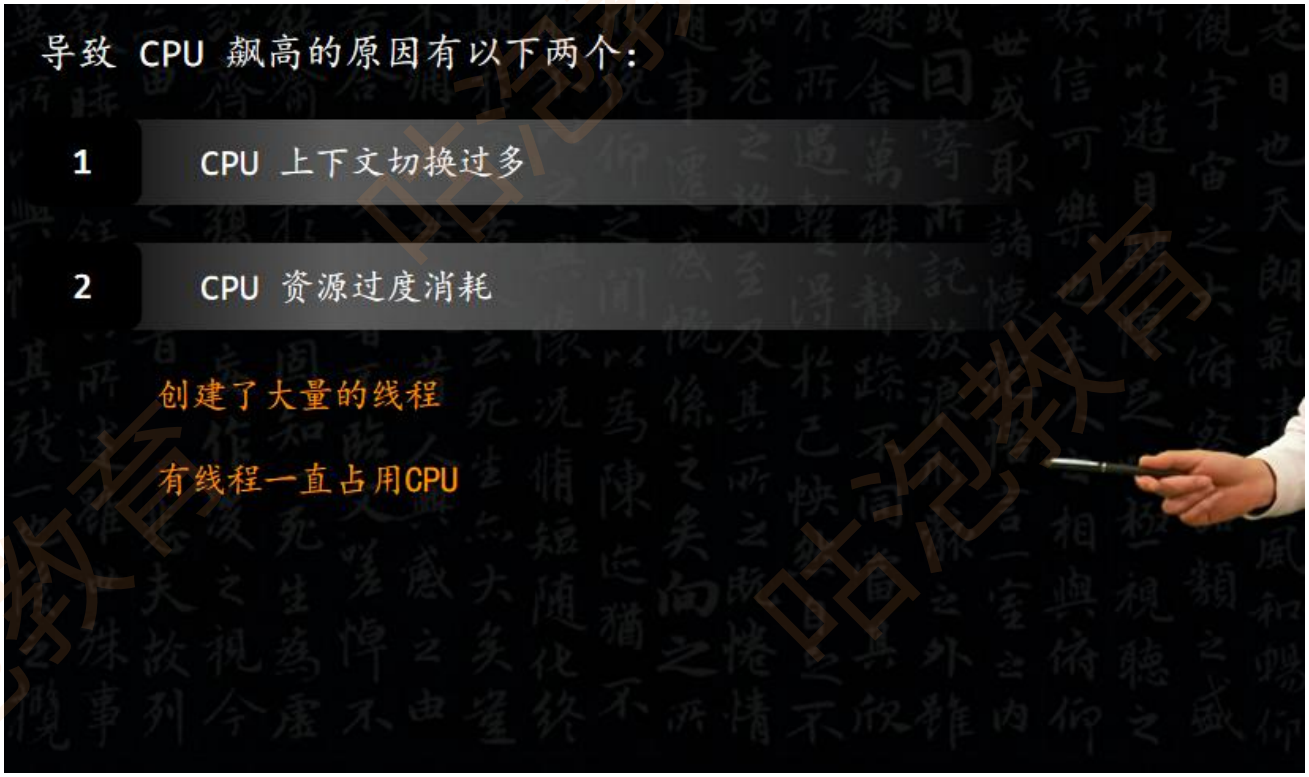
保存运行线程的执行状态

让处于等待中的线程执行

这两个过程需要 CPU 执行内核相关指令实现状态保存，如果较多的上下文切换会占据大量CPU资源，从而使得 CPU 无法去执行用户进程中的指令，导致响应速度下降。在 Java 中，文件 IO、网络 IO、锁等待、线程阻塞等操作都会造成线程阻塞从而触发上下文切换。



2、CPU 资源过度消耗。



也就是在程序中创建了大量的线程，或者有线程一直占用CPU 资源无法被释放，比如死循环！CPU 利用率过高之后，导致应用中的线程无法获得 CPU 的调度，从而影响程序的执行效率！既然是这两个问题导致的 CPU 利用率较高，于是我们可以通过 top 命令，找到CPU 利用率较高的进程，

在通过 Shift+H 找到进程中 CPU 消耗过高的线程，这里有两种情况。

CPU 利用率过高的线程一直是同一个，说明程序中存在线程长期占用 CPU 没有释放的情况，这种情况直接通过 jstack 获得线程的 Dump 日志，定位到线程日志后就可以找到问题的代码。

CPU 利用率过高的线程 ID 不断变化，说明线程创建过多，需要挑选几个线程 ID，通过 jstack 去线程 dump 日志中排查。最后有可能定位的结果是程序正常，只是在 CPU 飙高的那一刻，用户访问量较大，导致系统资源不够。

【结尾】

以上就是我对这个问题的理解！从这个问题来看，面试官主要考察实操能力，以及解决问题的思路。如果你没有实操过，但是你知道导致 CPU 飙高这个现象的原因，并说出你的解决思路，通过面试是没问题的。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

1.62. 读写分离常见方案?

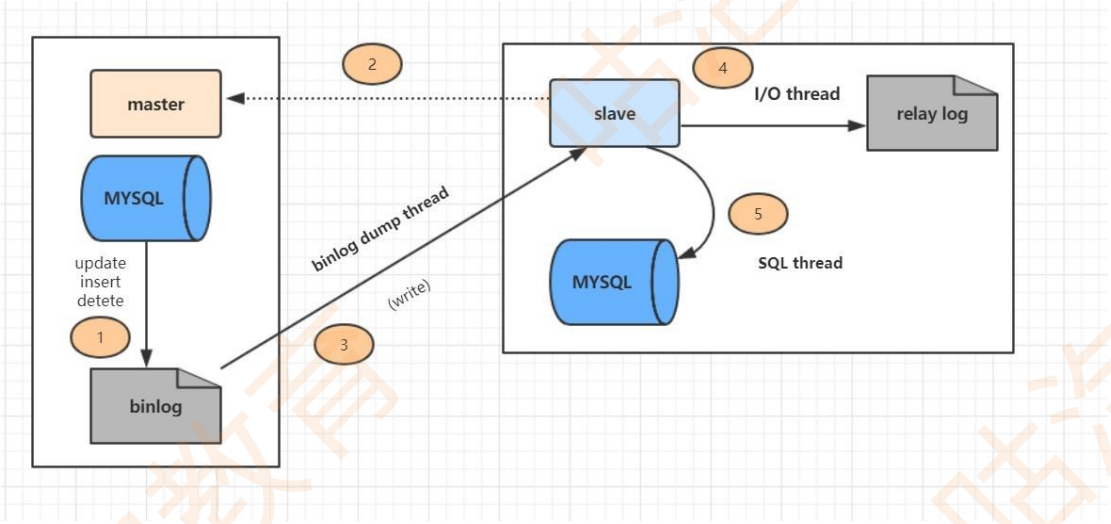
- 应用程序根据业务逻辑来判断，增删改等写操作命令发给主库，查询命令发给备库。
- 利用中间件来做代理，负责对数据库的请求识别出读还是写，并分发到不同的数据库中。（如：amoeba，mysql-proxy）

1.63. MySQL的复制原理以及流程

主从复制原理，简言之，就三步曲，如下：

- 主数据库有个 bin-log 二进制文件，纪录了所有增删改 Sql 语句。（binlog 线程）
- 从数据库把主数据库的 bin-log 文件的 sql 语句复制过来。（io 线程）
- 从数据库的relay-log 重做日志文件中再执行一次这些 sql 语句。（Sql 执行线程）

如下图所示：



上图主从复制分了五个步骤进行：

步骤一：主库的更新事件(update、insert、delete)被写到 binlog

步骤二：从库发起连接，连接到主库。

步骤三：此时主库创建一个 binlog dump thread，把 binlog 的内容发送到从库。

步骤四：从库启动之后，创建一个 I/O 线程，读取主库传过来的binlog 内容并写入到 relay log

步骤五：还会创建一个 SQL 线程，从 relay log 里面读取内容，从Exec_Master_Log_Pos 位置开始执行读取到的更新事件，将更新内容写入到slave 的 db

1.64. Innodb 的事务实现原理?

事务就 ACID 四个特性，我觉得讲清楚这几个特点是怎么保证就可以了。

- 原子性：是使用 undo log 来实现的，如果事务执行过程中出错或者用户执行了rollback，系统通过 undo log 日志返回事务开始的状态。
- 持久性：使用 redo log 来实现，只要 redo log 日志持久化了，当系统崩溃，即可通过redo log 把数据恢复。
- 隔离性：通过锁以及 MVCC,使事务相互隔离开。
- 一致性：通过回滚、恢复，以及并发情况下的隔离性，从而实现一致性。

1.65. 谈谈 MySQL 的 Explain

Explain 执行计划包含字段信息如下：分别是

id、select_type、table、partitions、type、possible_keys、key、key_len、ref、rows、filtered、Extra 等 12 个字段。

我们重点关注的是 type，它的属性排序如下：

```
system > const > eq_ref > ref > ref_or_null >
index_merge > unique_subquery > index_subquery >
range > index > ALL
```

1.66. Innodb 的事务与日志的实现方式

有多少种日志

innodb 两种日志 redo 和 undo。

日志的存放形式

- redo：在页修改的时候，先写到 redo log buffer 里面， 然后写到 redo log 的文件系统缓存里面(fwrite)，然后再同步到磁盘文件（ fsync）。
- Undo：在 MySQL5.5 之前， undo 只能存放在 ibdata 文件里面， 5.6 之后，可以通过设置 innodb_undo_tablespaces 参数把 undo log 存放在 ibdata 之外。

事务是如何通过日志来实现的

- 因为事务在修改页时，要先记 undo，在记 undo 之前要记 undo的 redo， 然后修改数据页，再记数据页修改的redo。 Redo（里面包括 undo ✂修改） 一定要比数据页先持久化到磁盘。
- 当事务需要回滚时，因为有 undo，可以把数据页回滚到前镜像的状态，崩溃恢复时，如果 redo log 中事务没有对应的 commit 记录，那么需要用 undo 把该事务修改回滚到事务开始之前。
- 如果有 commit 记录，就用 redo 前滚到该事务完成时并提交掉。

1.67. 你们数据库是否支持 emoji 表情存储，如果不支持，如何操作?

更换字符集 utf8-->utf8mb4

1.68. 一个 6 亿的表 a，一个 3 亿的表 b，通过外间 tid 关联，你如何最快地查询出满足条件的第 50000 到第 50200 中的这 200 条数据记录。

1、如果 A 表 TID 是自增长,并且是连续的,B 表的 ID 为索引 select * from a,bwhere a.tid = b.id and a.tid>500000 limit 200;

2、如果 A 表的 TID 不是连续的,那么就需要使用覆盖索引.TID 要么是主键,要么是辅助索引,B 表 ID 也需要有索引。 select * from b , (select tid from alimit 50000,200) a where b.id = a .tid;

1.69. Mysql 一条 SQL 加锁分析

一条 SQL 加锁，可以分 9 种情况进行：

- 组合一：id 列是主键，RC 隔离级别
- 组合二：id 列是二级唯一索引，RC 隔离级别
- 组合三：id 列是二级非唯一索引，RC 隔离级别
- 组合四：id 列上没有索引，RC 隔离级别
- 组合五：id 列是主键，RR 隔离级别
- 组合六：id 列是二级唯一索引，RR 隔离级别
- 组合七：id 列是二级非唯一索引，RR 隔离级别
- 组合八：id 列上没有索引，RR 隔离级别
- 组合九：Serializable 隔离级别

1.70. Mysql如何解决幻读问题

“Mysql如何解决幻读问题”

一个工作了4年小伙伴，去一个美团面试，遇到了这样一个问题。

大家好，我是Mic，一个工作了14年的Java程序员

关于这个问题，面试官想考察什么？我们应该如何回答呢？

问题解析

这个问题至少考察的是3年以上开发经验的同学。

Mysql底层去解决并发事务问题，至少是要有一定的技术积累才能真正理解。

而如果作为一个刚工作没多久的程序员，必须要知道数据库的事务隔离级别的问题。

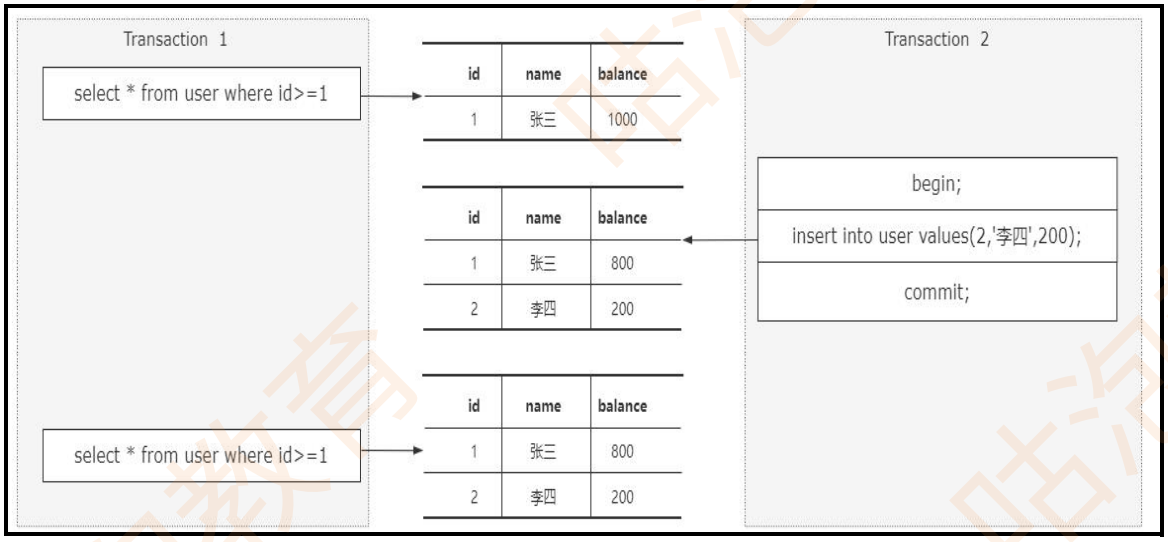
因为不同的隔离级别对于数据的安全性影响是不同的。

也就是存在脏读、幻读、不可重复读等问题。

所谓幻读，就是一个事务前后两次读取到的数据条数不一致。

在第一个事务里面执行一个范围查询，这个时候满足查询的数据只有一条。

接着第二个事务里面插入一条数据并且提交了，然后在第一个事务里面再次查询的时候发现有两条数据满足条件。



在RR事务隔离级别下，引入了MVCC和LBCC这两种方式来解决幻读问题。

MVCC类似于一种乐观锁的设计，简单来说就是针对每个事务生成一个事务版本，然后针对这个版本定义了访问规则

一个事务只能看到第一次查询之前已经提交的事务以及当前事务的修改。

一个事务不能看到当前事务第一次查询之后创建的事务，以及未提交的事务修改。

但是，如果在一个事务里面存在当前读的情况下，MVCC还是会存在幻读问题，

因为当前读不是读快照，而是直接读内存。

所以针对这种情况，可以使用LBCC也就是基于锁的机制来解决，也就是常说的行锁、表锁、间隙锁等

基于对上述知识的理解，如果没有对Mysql不同事务隔离级别的底层实现原理有一个清晰认识的同学

在回答这个问题的时候，要么就是很生硬，要么就是无法扩展，就会显得有点像是在背答案。

下面看看高手是怎么回答这个问题的吧。

高手

在RR(也就是可重复读)的事务隔离级别下，InnoDB采用了MVCC机制来解决幻读问题。

MVCC就是一种乐观锁的机制，它通过对不同事务生成不同的快照版本，通过UNDO版本链进行管理并且在MVCC里面，规定了高版本能够看到低版本的事务变更，低版本看不到高版本的事务变更从而实现了不同事务之间的数据隔离，解决了幻读的问题。

但是在当前读的情况下，是直接读取内存的数据，跳过了快照度，所以还是会出现幻读问题。

我认为可以通过两个方式来解决。

第一种是尽量避免当前读的情况

第二种是引入LBCC的方式

以上就是我对这个问题的理解。

学Java架构·来咕泡教育
50000+ 技术人
都在咕泡教育拿到职场结果

总结

好了，今天的分享就到这里结束了

如果喜欢我的作品，记得点赞、收藏、关注

我是Mic，我们下期再见！

Redis 专栏面试题（2）

2.1. 什么是 Redis？它主要用来什么的？

“谈谈你对Redis的理解”！

面试的时候遇到这类比较宽泛的问题，是不是很抓狂？

是不是不知道从何开始说起？

没关系，今天我用3分钟教你怎么回答。

大家好，我是Mic，一个工作了14年的Java程序员。

这个问题面试官考察的目的是什么？希望得到什么样的回答？

考察目标

对于某某技术的理解这一类问题，它是一种比较宽泛的问题

在面试过程中，考察这类问题有两个很重要的目的：

在面试的过程中，面试官希望求职者能多说一些东西，从而更好的对你的整体情况和能力有一个清晰的判断，因此这类问题，可以找到一些了解你的突破口。

这种问题其实没有标准答案，更多的是基于你对它的理解的一个总结

这反而能够更好的考察你的技术积累和逻辑表达能力。

所以，求职者在回答的过程中，需要尽可能逻辑清晰，简单明了的表述出来。

否则很难得到认可。

问题解析

关于Redis是什么，想必大部分人都能脱口而出。

它是一个分布式缓存中间件？可这样回答有问题吗？当然有

准确来说，Redis是一个基于内存实现的Key-Value数据结构的Nosql数据库。

注意，这里有三个关键点。

内存存储

key-value结构

Nosql

所谓内存存储，是指所有数据是存储在内存里面，数据的IO性能比较高。

当然，Redis也提供了持久化策略来避免内存数据丢失的问题

key-value结构表示数据的存储方式，除了redis以外，还有像LevelDB、Scalaris等。

而Nosql，它指的是一种非关系型数据库，相比于传统的关系型数据库而言。

更多的考虑到扩展性、性能、大数据量的存储等，弥补了关系型数据库的短板

像列式存储ClickHouse、Cassandra； 文档存储MongoDB图形存储Neo4J等都是属于Nosql范畴。

高手

Redis是一个基于Key-Value存储结构的Nosql开源内存数据库。

它提供了5种常用的数据类型，String、Map、Set、ZSet、List。

针对不同的结构，可以解决不同场景的问题。

因此它可以覆盖应用开发中大部分的业务场景，比如top10问题、好友关注列表、热点话题等。

其次，由于Redis是基于内存存储，并且在数据结构上做了大量的优化所以IO性能比较好，在实际开发中，会把它作为应用与数据库之间的一个分布式缓存组件。

并且它又是一个非关系型数据的存储，不存在表之间的关联查询问题，所以它可以很好的提升应用程序的数据IO效率。

最后，作为企业级开发来说，它又提供了主从复制+哨兵、以及集群方式实现高可用在Redis集群里面，通过hash槽的方式实现了数据分片，进一步提升了性能。

总结

好的，屏幕前的你，学废了吗?

如果你喜欢我的作品，记得点赞收藏加关注

2.2. 说说 Redis 的基本数据结构类型

大多数小伙伴都知道，Redis 有以下这五种基本类型：

- String（字符串）
- Hash（哈希）
- List（列表）
- Set（集合）
- zset（有序集合）

它还有三种特殊的数据结构类型

- Geospatial
- Hyperloglog
- Bitmap

Redis的五种基本数据类型

String（字符串）

- 简介:String 是 Redis 最基础的数据结构类型，它是二进制安全的，可以存储图片或者序列化的对象，值最大存储为 512M
- 简单使用举例: set key value、get key等
- 应用场景：共享 session、分布式锁，计数器、限流。
- 内部编码有 3 种，int（8字节长整型）/embstr（小于等于 39字节字符串）/raw（大于 39个字节字符串）

C 语言的字符串是 char[]实现的，而 Redis 使用 **SDS（simple dynamic string）** 封装，sds 源码如下：

```
struct sdshdr{
    unsigned int len; // 标记buf✂长度
    unsigned int free; //标记buf中未使用✂元素个数
```

```
    char buf[]; // 存放元素✂坑
}
```

SDS 结构如下：

Redis 为什么选择 **SDS** 结构，而 C 语言原生的 char[]不香吗?

举例其中一点，SDS 中，O(1)时间复杂度，就可以获取字符串长度；而 C 字符串，需要遍历整个字符串，时间复杂度为 O(n)

Hash（哈希）

- 简介：在 Redis 中，哈希类型是指 v（值）本身又是一个键值对（k-v）结构
- 简单使用举例：hset key field value、hget key field
- 内部编码：ziplist（压缩列表）、hashtable（哈希表）

- 应用场景：缓存用户信息等。
- **注意点**：如果开发使用 hgetall，哈希元素比较多✂话，可能导致 Redis 阻塞，可以使用 hscan。而如果只是获取部分 field，✂议使用 hmget。

字符串和哈希类型如下：

List（列表）

- 简介：列表（list）类型是用来存储多个有序✂字符串，一个列表最多可以存储 2^32-1 个元素。
- 简单实用举例： lpush key value [value ...] 、lrange key start end
- 内部编码：ziplist（压缩列表）、linkedlist（链表）
- 应用场景： 消息队列，文章列表,

看懂 list 类型✂插入与弹出：

list 应用场景参考以下：

- lpush+lpop=Stack（栈）
- lpush+rpop=Queue（队列）
- lpsh+ltrim=Capped　Collection（有限集合）
- lpush+brpop=Message　Queue（消息队列）

Set（集合）

- 简介：集合（set）类型也是用来保存多个✂字符串元素，但是不允许重复元素
- 简单使用举例：sadd key element [element ...]、smembers key
- 内部编码：intset（整数集合）、hashtable（哈希表）
- **注意点**：smembers 和 lrange、hgetall 都属于比较重的命令，如果元素过多存在阻塞Redis的可能性，可以使用 sscan 来完成。
- 应用场景： 用户标签,生成随机数抽奖、社交需求。

有序集合（zset）

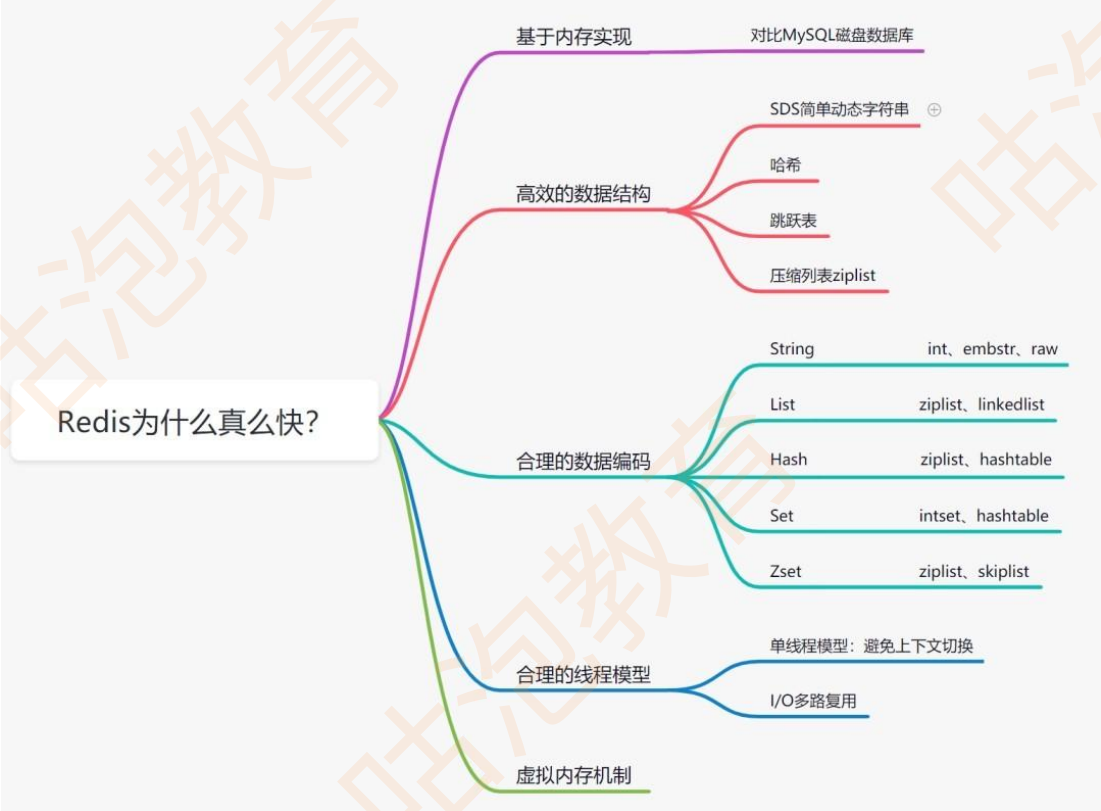
- 简介：已排序的字符串集合，同时元素不能重复
- 简单格式举例：zadd key score member [score member ...]，zrank key member
- 底层内部编码：ziplist（压缩列表）、skiplist（跳跃表）
- 应用场景：排行榜，社交需求（如用户点赞）。

Redis 的三种特殊数据类型

- Geo：Redis3.2 推出的，地理位置定位，用于存储地理位置信息，并对存储的信息进行操作。
- HyperLogLog：用来做基数统计算法的数据结构，如统计网站的 UV。

- Bitmaps ：用一个比特位来映射某个元素的状态，在 Redis 中，它的底层是基于字符串类型实现的，可以把 bitmaps 成作一个以比特位为单位的数组

2.3. Redis 为什么这么快?



Redis 为什么这么快

基于内存存储实现

我们都知道内存读写是比在磁盘快很多的，Redis 基于内存存储实现的数据库，相对于数据存在磁盘的 MySQL 数据库，省去磁盘 I/O的消耗。

高效的数据结构

我们知道，Mysql 索引为了提高效率，选择了 B+树的数据结构。其实合理的数据结构，就是可以让你的应用/程序更快。先看下 Redis 的数据结构&内部编码：

SDS 简单动态字符串

- 二进制安全：Redis 可以存储一些二进制数据，在 C 语言中字符串遇到'\0'会结束，而 SDS 中标志字符串结束的是 len 属性。

字典

Redis 作为 K-V 型内存数据库，所有的键值就是用字典来存储。字典就是哈希表，比如 HashMap，通过key 就可以直接获取到对应的 value。而哈希表的 特性，在 O（1）时间复杂度就可以获得对应的值。

跳跃表

- 字符串长度处理：Redis 获取字符串长度，时间复杂度为 O(1)，而 C 语言中，需要从头开始遍历，复杂度为 O（n）；
- 空间预分配：字符串修改越频繁的话，内存分配越频繁，就会消耗性能，而 SDS 修改和空间扩充，会额外分配未使用的空间，减少性能损耗。
- 惰性空间释放：SDS 缩短时，不是回收多余的内存空间，而是 free 记录下多余的空间，后续有变更，直接使用 free 中记录的空间，减少分配。

- 跳跃表是 Redis 特有的数据结构，就是在链表的基础上，增加多级索引提升查找效率。
- 跳跃表支持平均 O（logN），最坏 O（N）复杂度的节点查找，还可以通过顺序性操作批量处理节点。

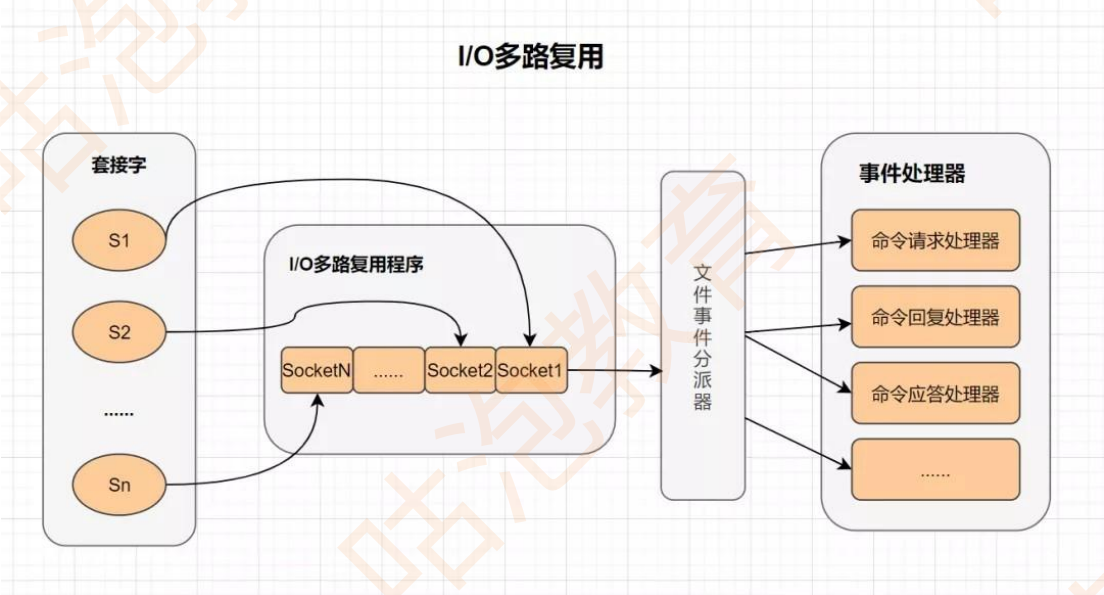
合理的数据编码

Redis 支持多种数据数据类型，每种基本类型，可能对多种数据结构。什么时候,使用什么样数据结构，使用什么样编码，是 redis 设计者总结优化的结果。

- String：如果存储数字的话，是用 int 类型的编码;如果存储非数字，小于等于 39 字节的字符串，是 embstr；大于 39 个字节，则是 raw 编码。
- List：如果列表的元素个数小于 512 个，列表每个元素的值都小于 64 字节（默认），使用 ziplist 编码，否则使用 linkedlist 编码

- Hash：哈希类型元素个数小于 512 个，所有值小于 64 字节的话，使用 ziplist 编码,否则使用 hashtable 编码。
- Set：如果集合中的元素都是整数且元素个数小于 512 个，使用 intset 编码，否则使用 hashtable 编码。
- Zset：当有序集合的元素个数小于 128 个，每个元素的值小于 64 字节时，使用 ziplist 编码，否则使用 skiplist（跳跃表）编码

合理的线程模型



I/O 多路复用

多路 I/O 复用技术可以让单个线程高效✂处理多个连接请求，而 Redis 使用用 epoll 作为 I/O 多路复用技术✂实现。并且，Redis 自身✂事件处理模型将 epoll 中✂连接、读写、关闭都转换为事件，不在网络 I/O 上浪费过多✂时间。

什么是 I/O 多路复用?

MIC解析

“什么是IO的多路复用机制？”

这是一道年薪50W的面试题，很遗憾，99%的人都回答不出来。

Hi，大家好，我是Mic，一个工作了14年的Java程序员。

今天，给大家分享一道网络IO的面试题。

这道题目的文字回答已经整理到了15W字的面试文档里面，

大家可以在我的主页加V领取

下面看看高手的回答。

高手

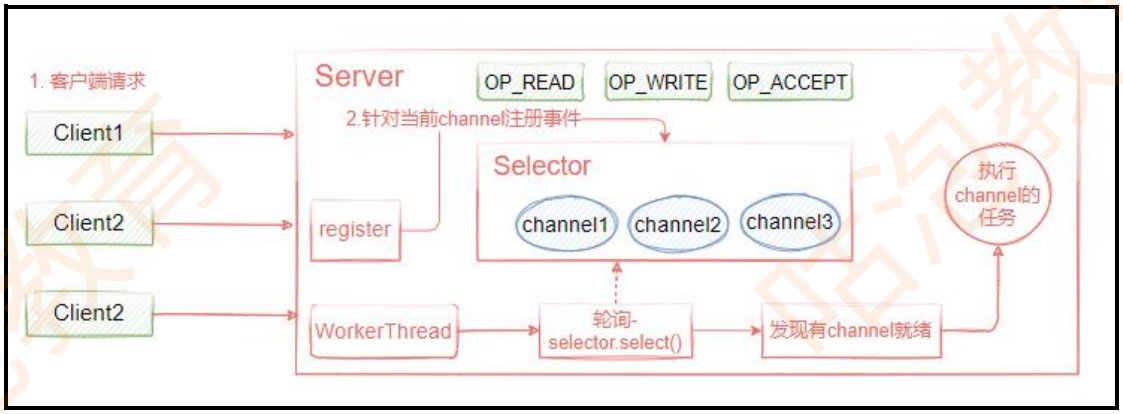
好的，面试官。

IO多路复用机制，核心思想是让单个线程去监视多个连接，一旦某个连接就绪，也就是触发了读/写事件。

就通知应用程序，去获取这个就绪的连接进行读写操作。

也就是在应用程序里面可以使用单个线程同时处理多个客户端连接，在对系统资源消耗较少的情况下提升服务端的链接处理数量。

在IO多路复用机制的实现原理中，客户端请求到服务端后，此时客户端在传输数据过程中，为了避免Server端在read客户端数据过程中阻塞，服务端会把该请求注册到Selector复路器上，服务端此时不需要等待，只需要启动一个线程，通过selector.select()阻塞轮询复路器上就绪的channel即可，也就是说，如果某个客户端连接数据传输完成，那么select()方法会返回就绪的channel，然后执行相关的处理就可以了。



常见的IO多路复用机制的实现方式有： select 、poll、epoll。

这些都是Linux系统提供的IO复用机制的实现，其中select和poll是基于轮询的方式去获取就绪连接。

而epoll是基于事件驱动的方式获取就绪连接。从性能的角度来看，基于事件驱动的方式要优于轮询的方式。

以上就是我对这个问题的理解。

面试点评

IO多路复用机制，是非常重要的网络通信基础。

在平时的业务开发中，使用比较少，但是在中间件里面作为基础通信模型，

是每个高级工程师必须要掌握的内容。

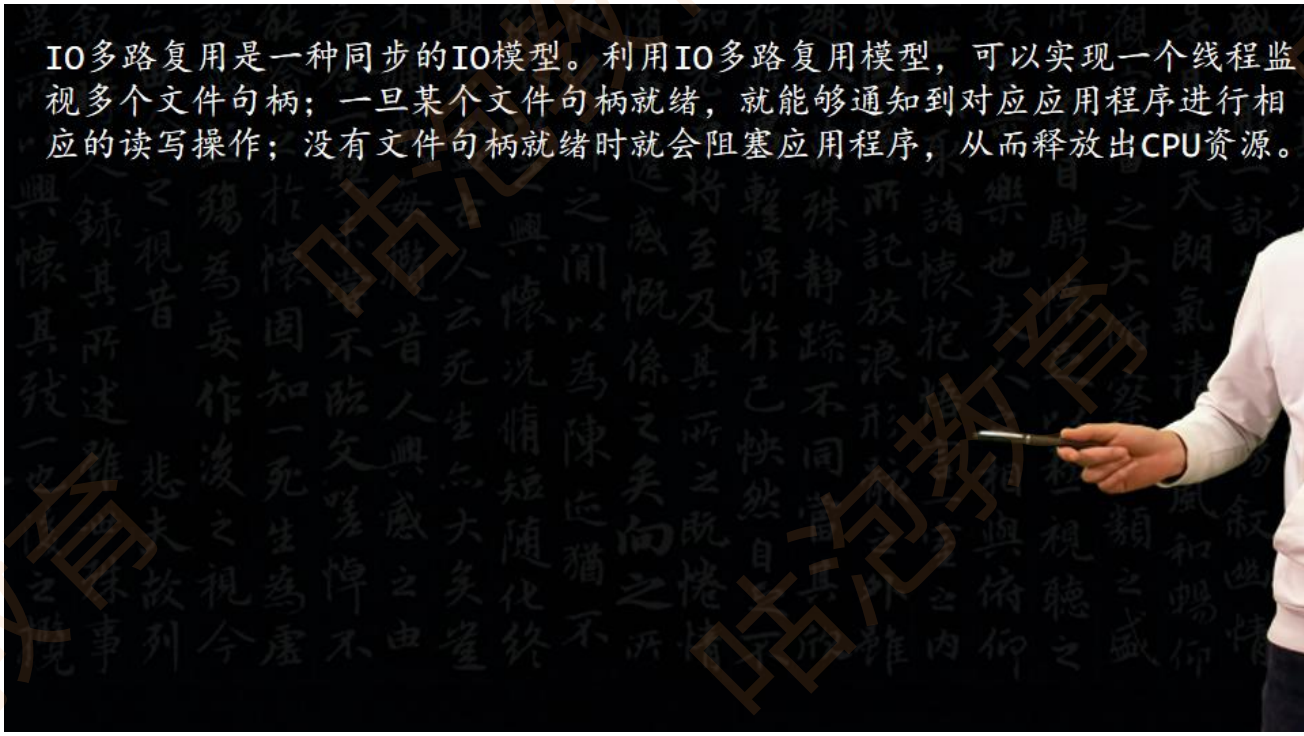
谈谈你对IO多路复用机制的理解

TOM解析

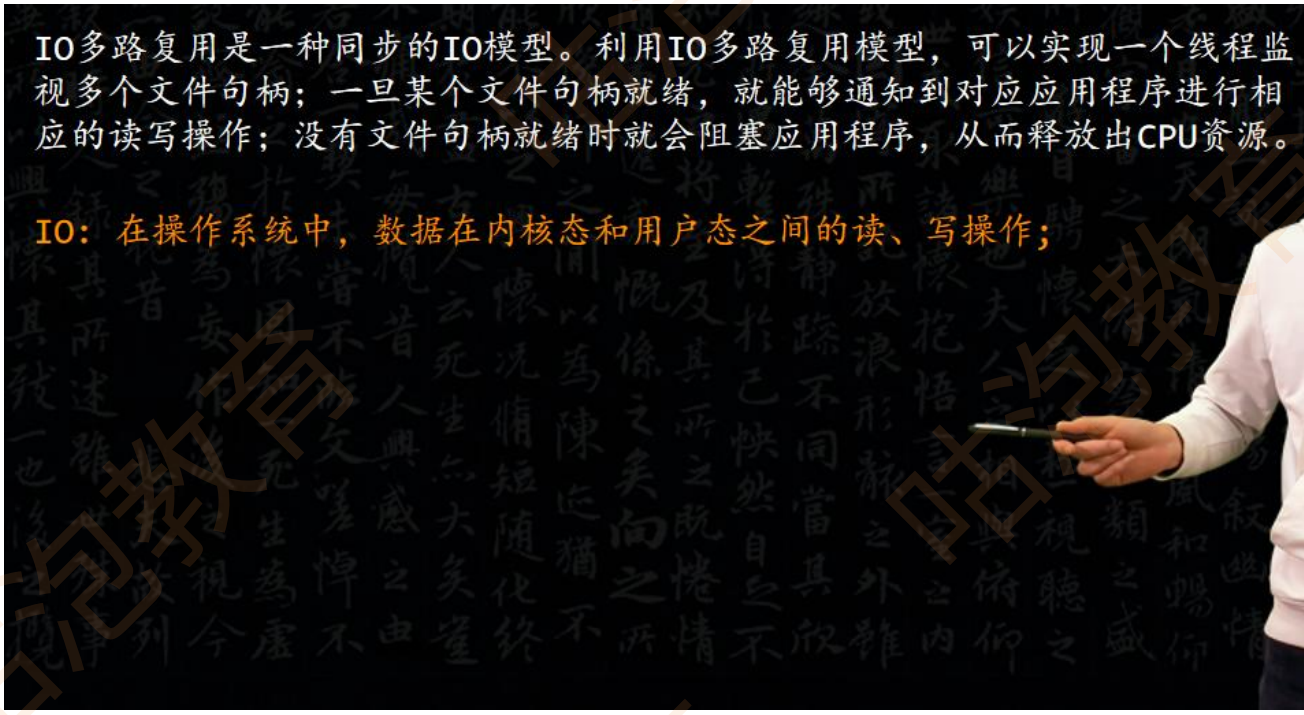
一位工作5年的小伙伴面试时被问到IO相关的问题，说，谈谈你对IO多路复用机制的理解。当时他说只是听过多路复用，具体细节没有了解过。今天，我给大家分享一下我的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

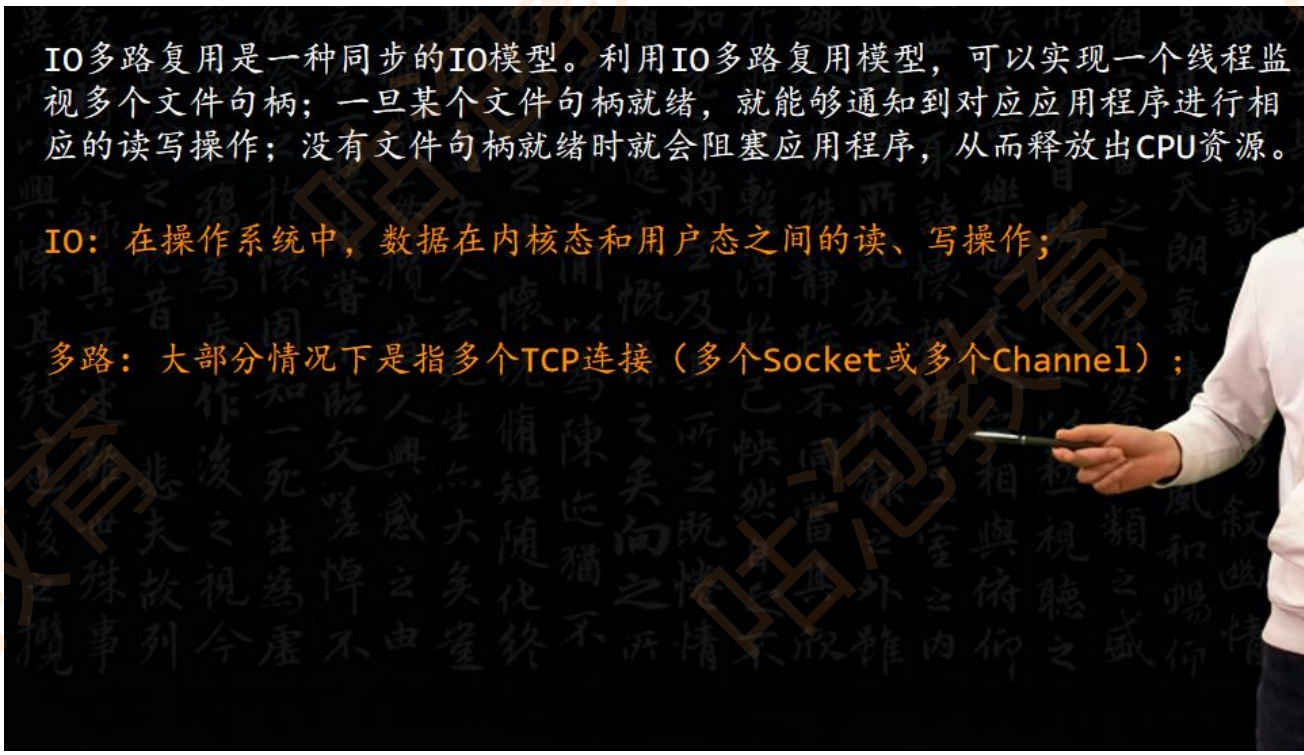
1 IO多路复用



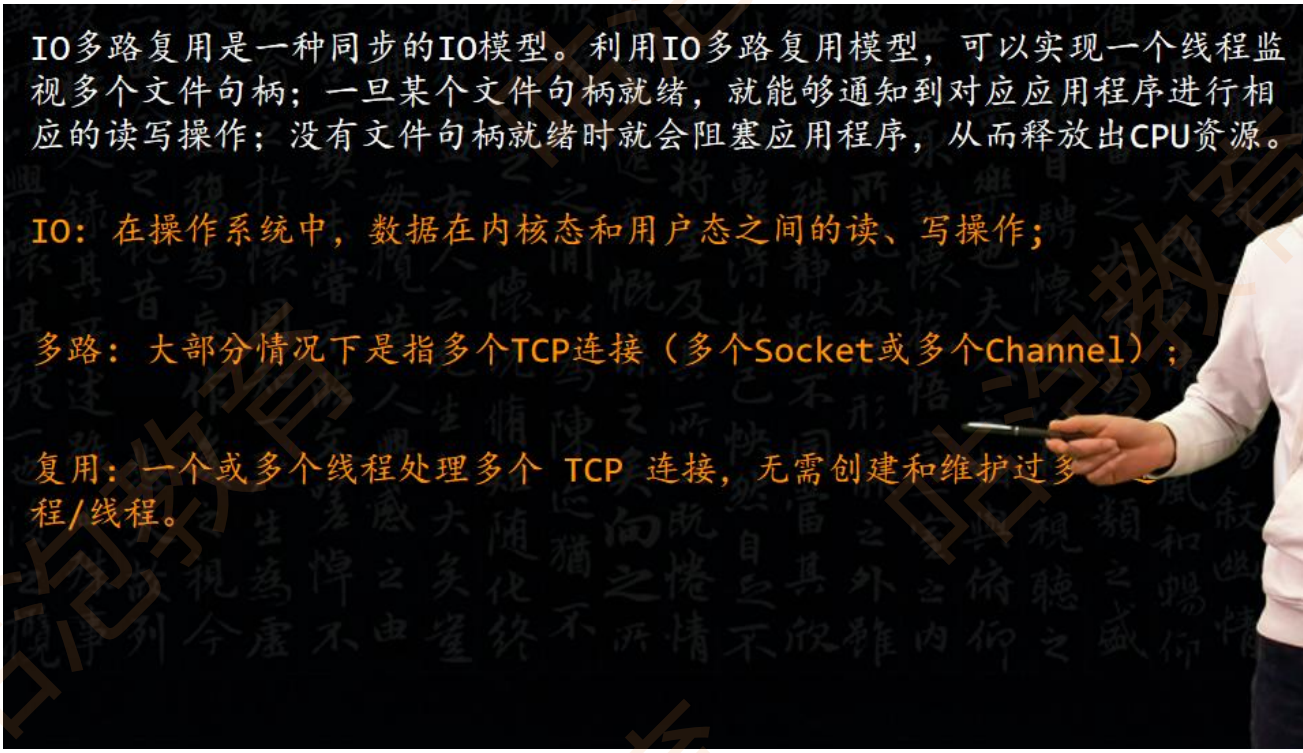
IO多路复用是一种同步的IO模型。利用IO多路复用模型，可以实现一个线程监视多个文件句柄；一旦某个文件句柄就绪，就能够通知到对应应用程序进行相应的读写操作；没有文件句柄就绪时就会阻塞应用程序，从而释放出CPU资源。



IO可以理解为，在操作系统中，数据在内核态和用户态之间的读、写操作，大部分情况下是指网络IO；



多路大部分情况下是指多个TCP连接，也就是多个Socket 或者多个Channel；

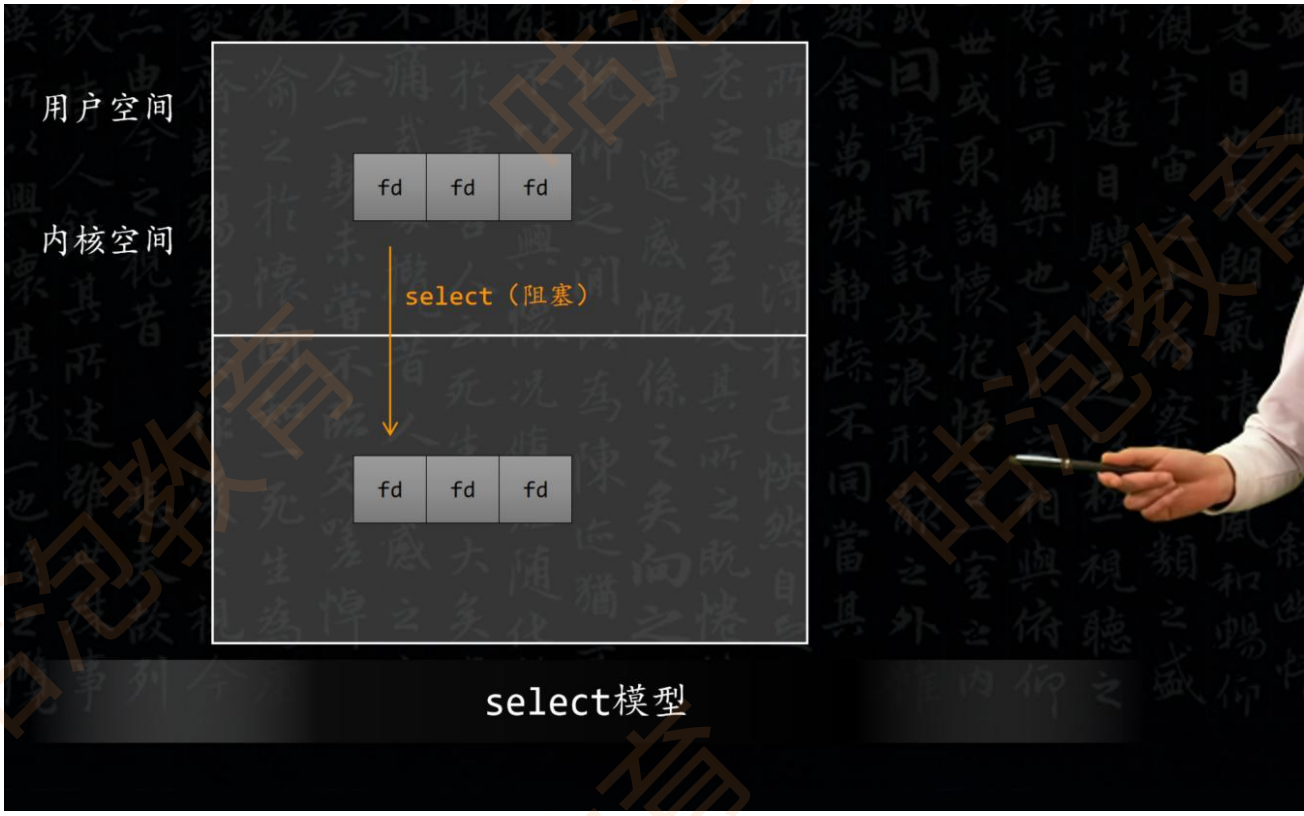


复用是指复用一个或多个线程资源。IO多路复用意思就是说，一个或多个线程处理多个 TCP 连接。尽可能地减少系统开销，无需创建和维护过多的进程/线程。

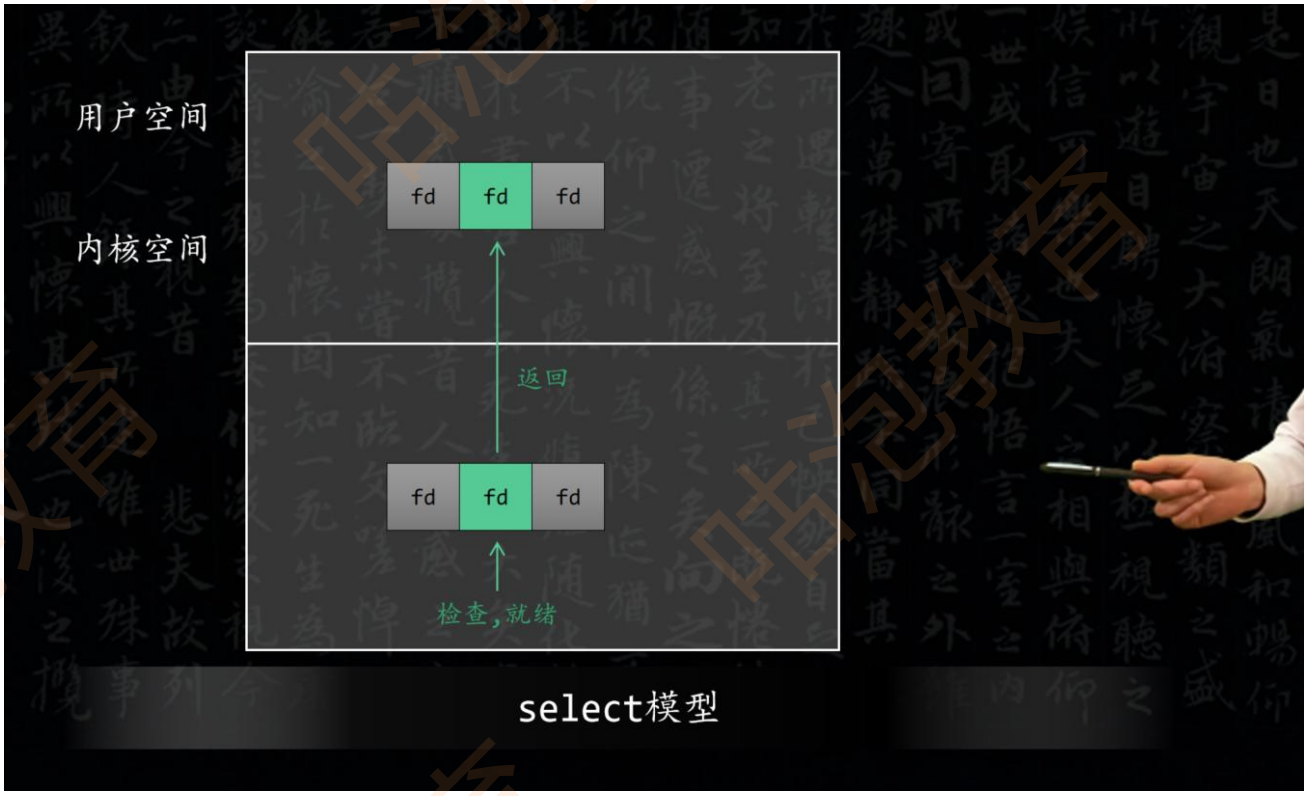
实现IO多路复用的模型有三种，分别是Select、poll 和 epoll。下面详细介绍一下三种多路复用模型的基本原理和优缺点：

2 select模型

select模型，它的基本原理是，采用轮询和遍历的方式。也就是说，在客户端操作服务器时，会创建三种文件描述符，简称FD。分别是writefds（写描述符）、readfds（读描述符）和 exceptfds（异常描述符）。



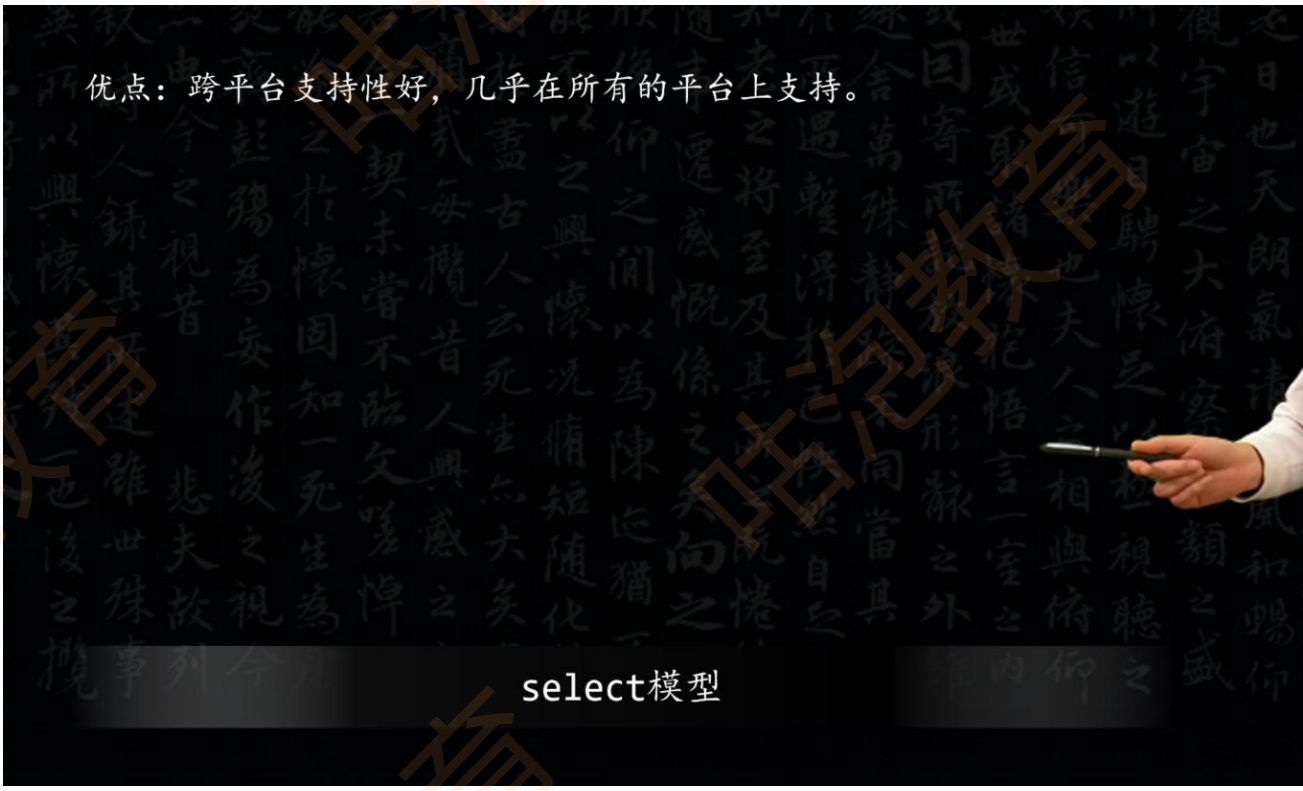
而select会阻塞监视这三种文件描述符，等有数据、可读、可写、出异常或超时都会返回；



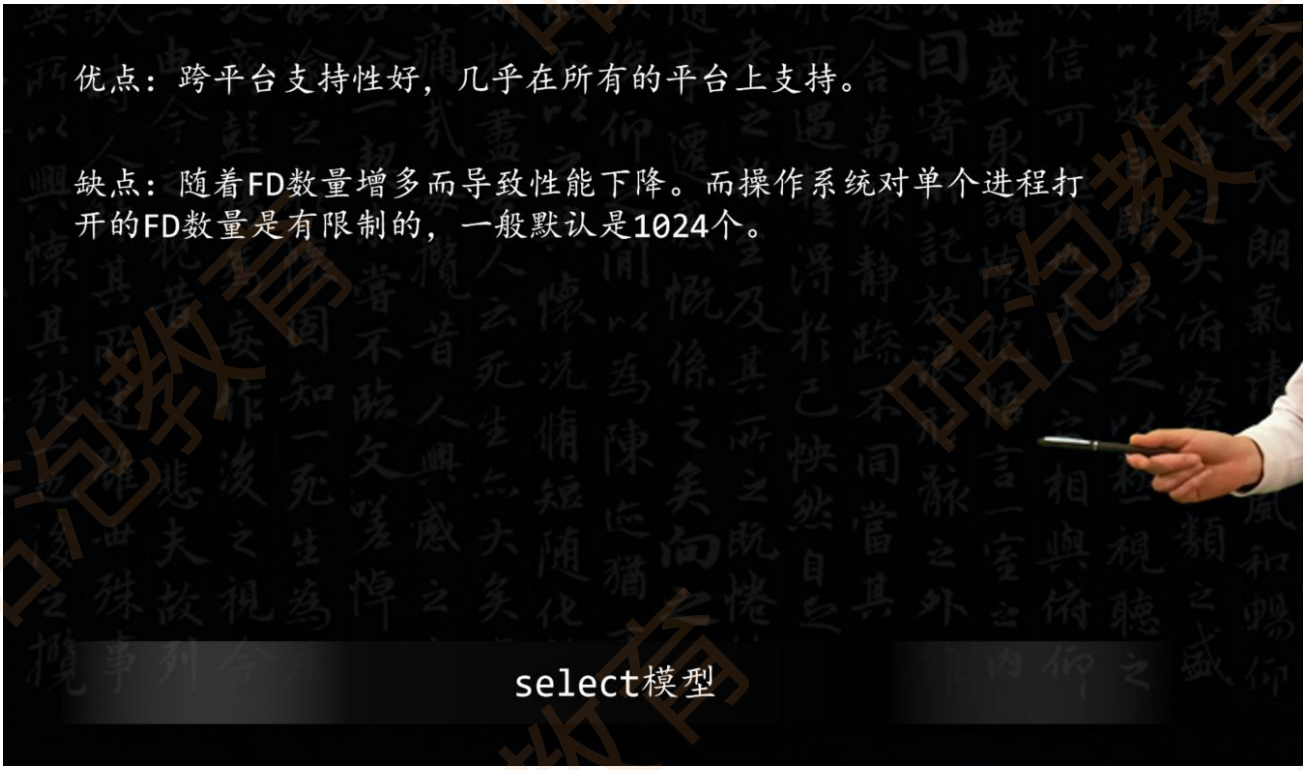
返回后通过遍历fdset，也就是文件描述符的集合，来找到就绪的FD，然后，触发相应的IO操

作。

它的优点是跨平台支持性好，几乎在所有的平台上支持。

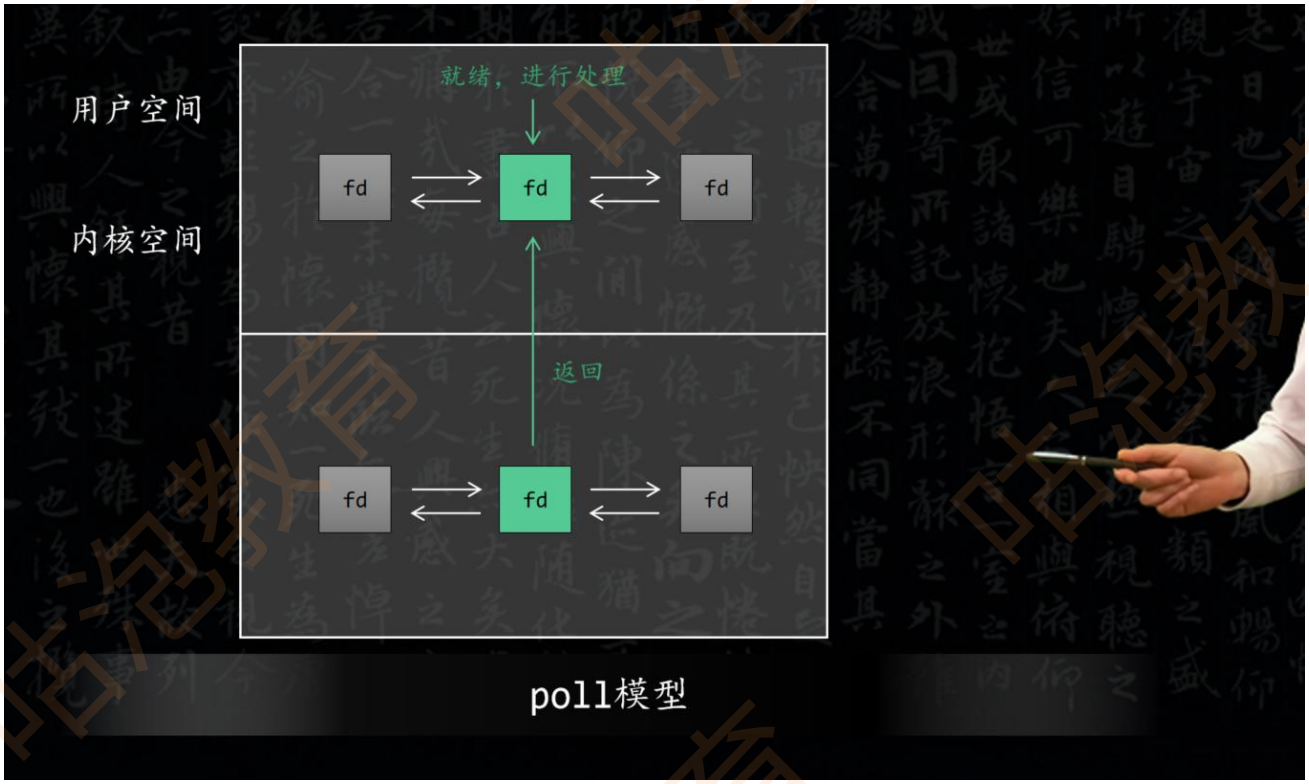


它的缺点也很明显，由于select是采用轮询的方式进行全盘扫描，因此，随着FD数量增多而导致性能下降。



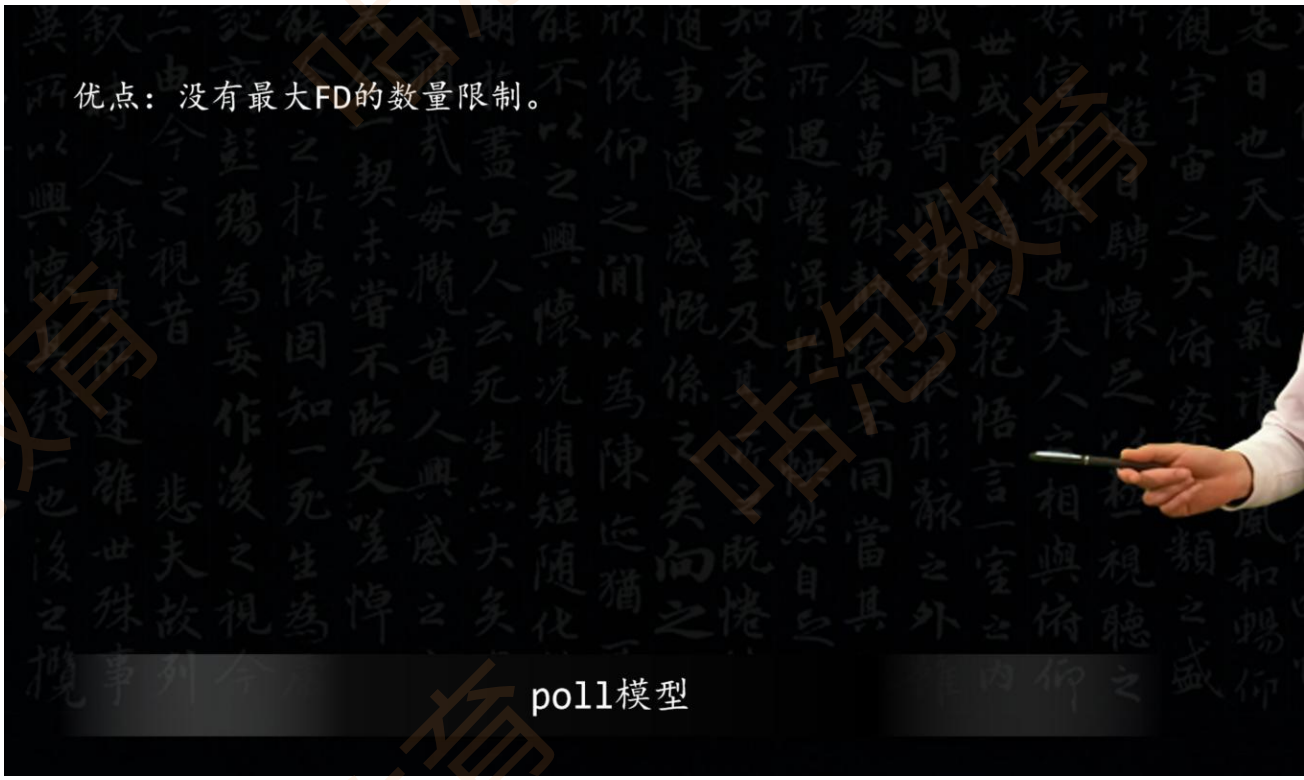
因此，每次调用select()方法，都需要把FD集合从用户态拷贝到内核态，并进行遍历。而操作系统对单个进程打开的FD数量是有限制的，一般默认是1024个。虽然，可以通过操作系统的宏定义FD_SETSIZE修改最大FD数量限制，但是，在IO吞吐量巨大的情况下，效率提升仍然有限。

3 poll模型

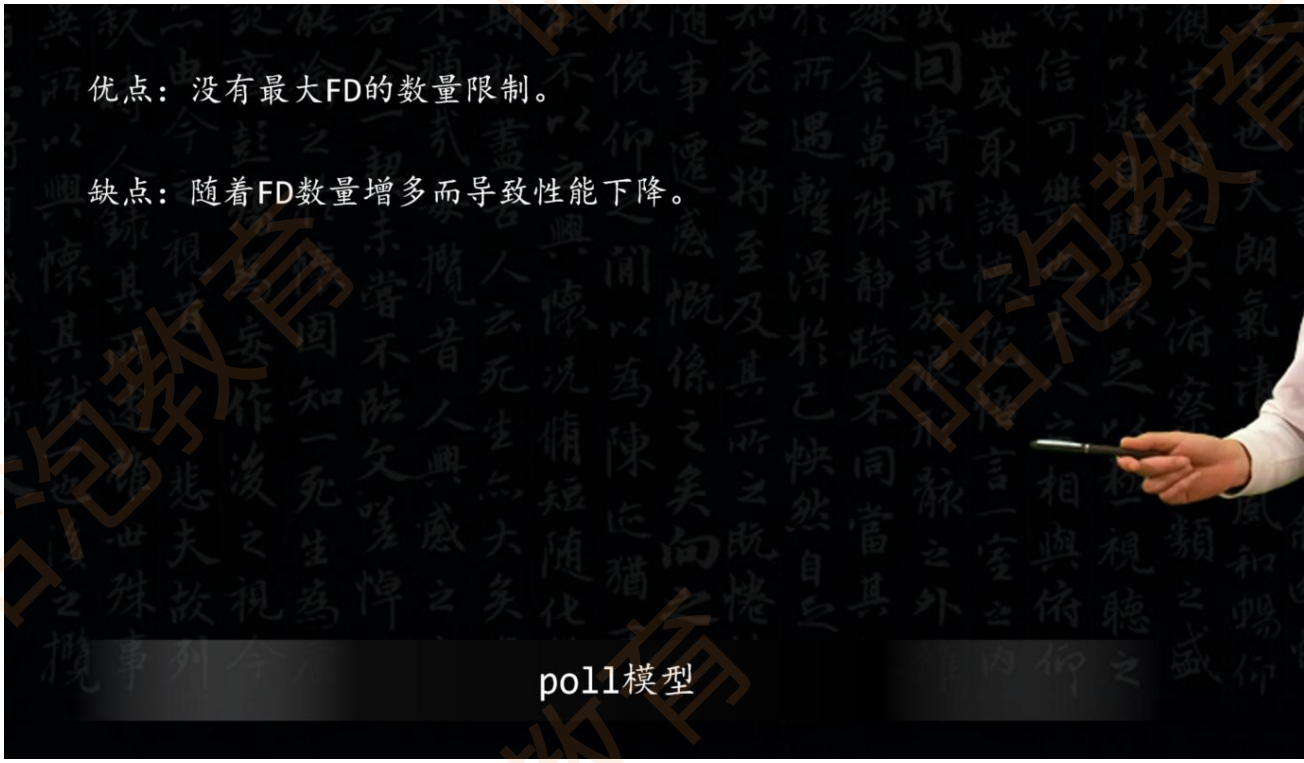


poll 模型的原理与select模型基本一致，也是采用轮询加遍历，唯一的区别就是 poll 采用链表的方式来存储FD。

所以，它的优点点是没有最大FD的数量限制。



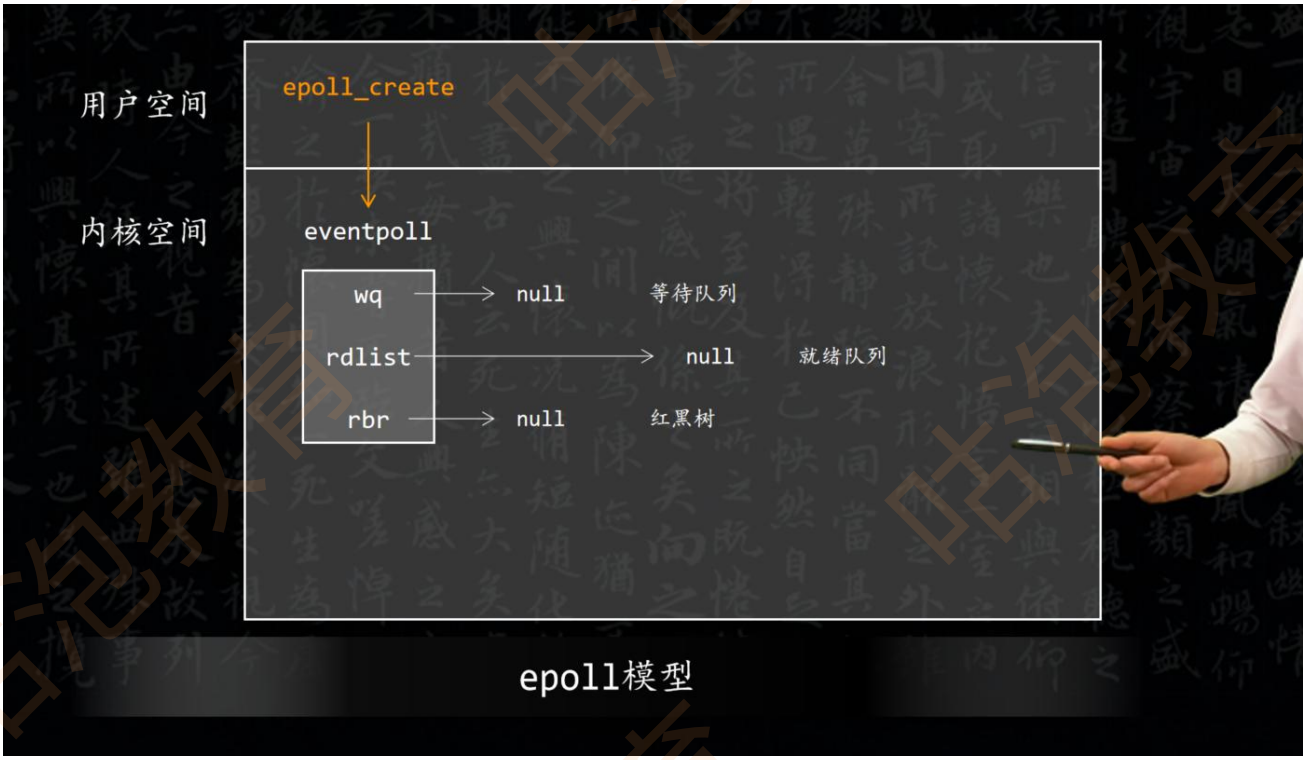
它的缺点和select一样，也是采用轮询方式全盘扫描，同样也会随着FD数量增多而导致性能下降。



4 epoll模型

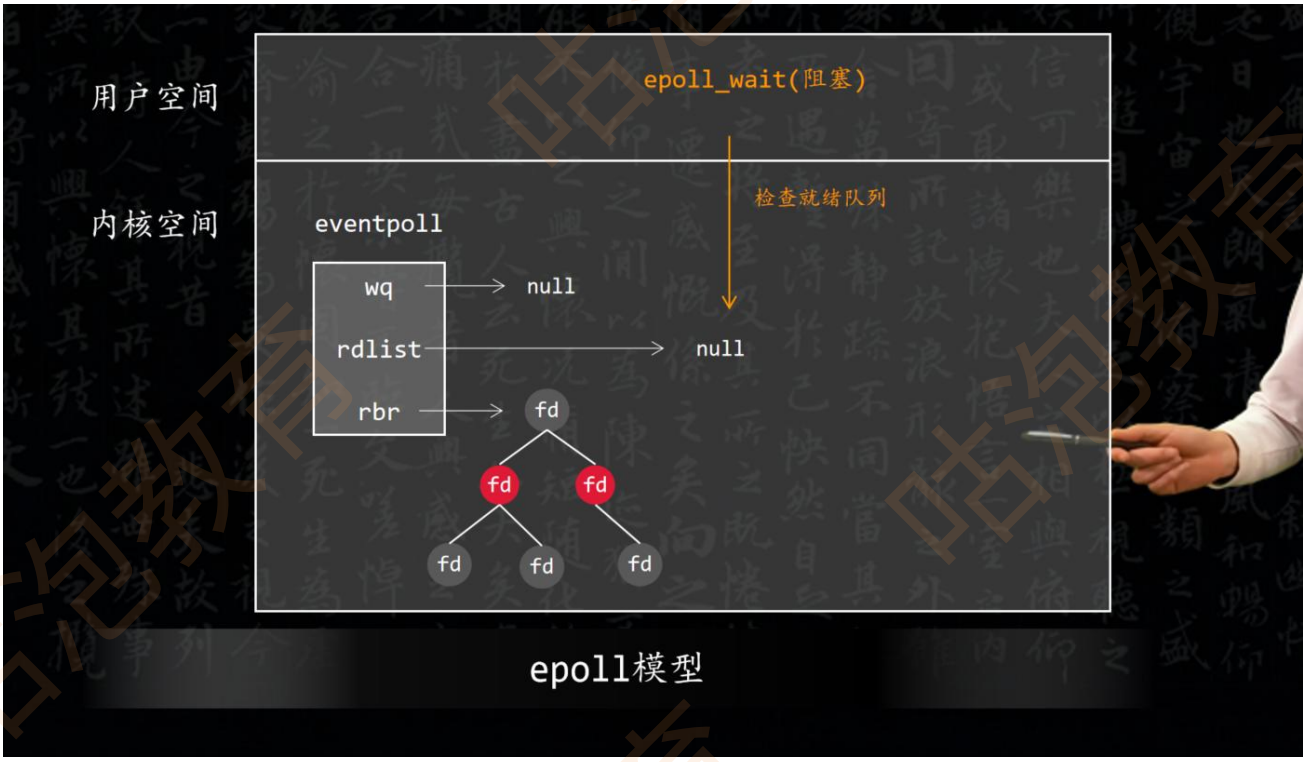
由于select和poll都会因为吞吐量增加而导致性能下降，因此，才出现了epoll模型。

epoll模型是采用时间通知机制来触发相关的IO操作。它没有FD个数限制，而且从用户态拷贝到内核态只需要一次。它主要通过系统底层的函数来注册、激活FD，从而触发相关的 IO 操作，这样大大提高了性能。主要是通过调用以下三个系统函数：

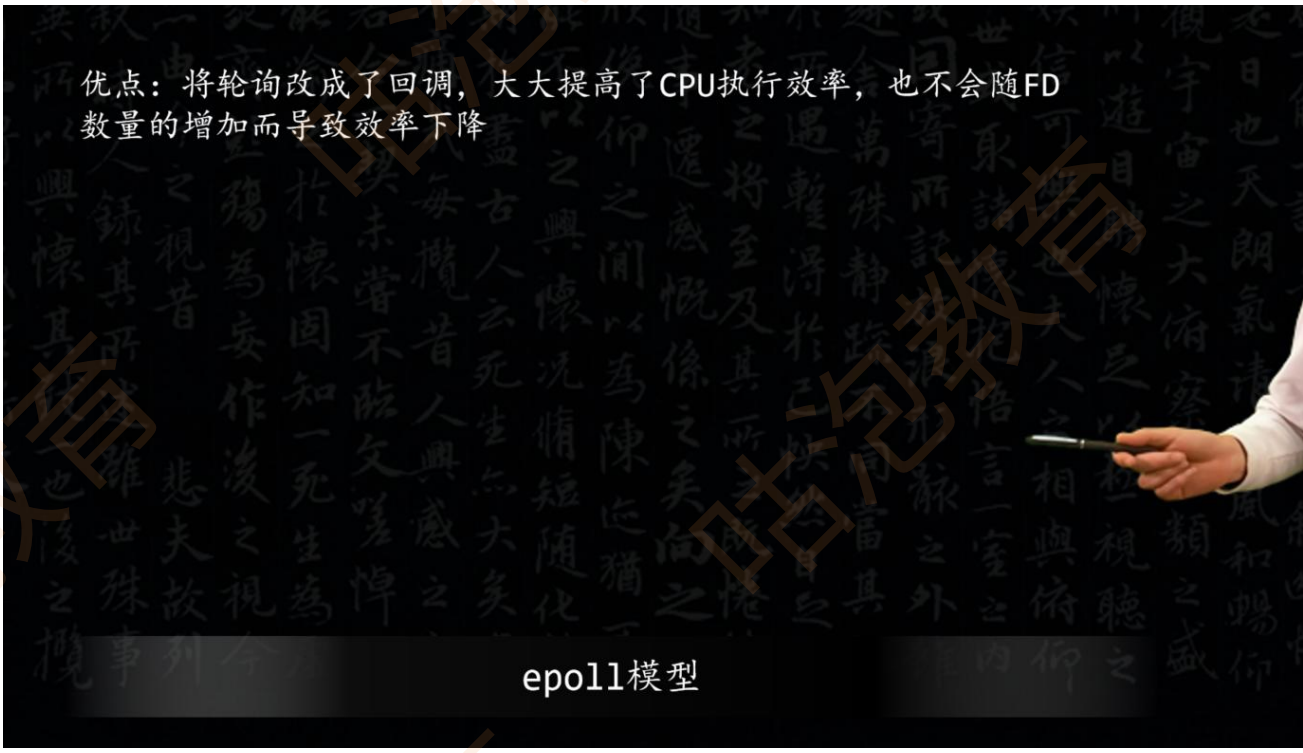


1、epoll_create()函数，在系统启动时，会在Linux内核里面申请一个B+树结构的文件系统，然后，返回epoll对象，也是一个FD。

2、epoll_ctl()函数，每新建一个连接的时候，会同步更新epoll对象中的FD，并且绑定一个callback回调函数。

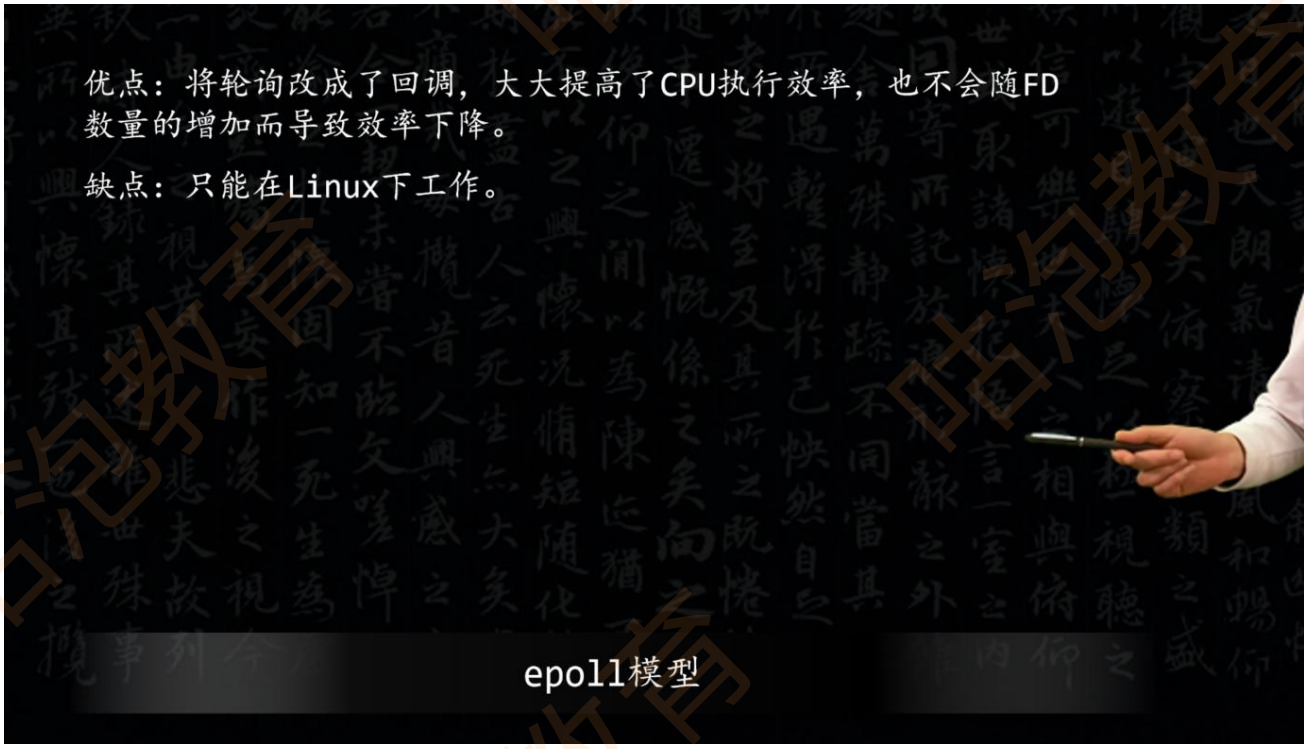


3、epoll_wait()函数，轮询所有的callback集合，并触发对应的 IO 操作



所以，epoll模型最大的优点是将轮询改成了回调，大大提高了CPU执行效率，也不会随FD数量的增加而导致效率下降。当然，它也没有FD数量限制，也就是说，它能支持的FD上限是操作系统的

最大文件句柄数。一般而言，1G 内存大概支持 10 万个句柄。分布式系统中常用的组件如Redis、Nginx都是优先采用epoll模型。



它的缺点是只能在Linux下工作。

5 综合对比

下表是三种多路复用模型的综合对比，有兴趣的小伙伴可以截图保存一下，或者到我的个人煮叶简介中获取。

	select	poll	epoll
数据结构	数组	链表	B+树
最大连接数	1024	无上限	无上限
FD拷贝	每次调用select拷贝	每次调用poll拷贝	FD首次调用epoll_ctl拷贝，每次调用epoll_wait不拷贝
工作效率	轮询：O(n)	轮询：O(n)	回调：O(1)

IO多路复用模型综合对比

好了，以上就是我对IO多路复用机制的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

2.4. 什么是缓存击穿、缓存穿透、缓存雪崩?

1.1.4 缓存穿透问题

先来看一个常见的缓存使用方式：读请求来了，先查下缓存，缓存有值命中，就直接返回；缓存没命中，就去查数据库，然后把数据库的值更新到缓存，再返回读取缓存

缓存穿透：指查询一个一定不存在的数据，由于缓存是不命中时需要从数据库查询，查不到数据则不写入缓存，这将导致这个不存在的数据每次请求都要到数据库去查询，进而给数据库带来压力。

通俗点说，读请求访问时，缓存和数据库都没有某个值，这样就会导致每次对这个值✂查询请求都会穿透到数据库，这就是缓存穿透。

缓存穿透一般都是这几种情况产生的：

- **业务不合理的设计**，比如大多数用户都没开守护，但是你的每个请求都去缓存，查询某个 userid 查询有没有守护。
- **业务/运维/开发失误的操作**，比如缓存和数据库的数据都被误删除了。
- **黑客非法请求攻击**，比如黑客故意捏造大量非法请求，以读取不存在✂业务数据。

如何避免缓存穿透呢？ 一般有三种方法。

- 1.如果是非法请求，我们在 API 入口，对参数进行校验，过滤非法值。
- 2.如果查询数据库为空，我们可以给缓存设置个空值，或者默认值。但是如有有写请求进来✂话，需要更新缓存哈，以保证缓存一致性，同时，最后给缓存设置适当✂过期时间。（业务上比较常用，简单有效）
- 3.使用布隆过滤器快速判断数据是否存在。即一个查询请求过来时，先通过布隆过滤器判断值是否存在，存在才继续往下查。

布隆过滤器原理：它由初始值为 0的位图数组和 N 个哈希函数组成。一个对一个 key 进行N 个 hash 算法获取 N 个值，在比特数组中将这 N 个值散列后设定为 1，然后查的时候如果特定的这几个位置都为 1，那么布隆过滤器判断该 key 存在。

1.1.5 缓存雪崩问题

缓存雪奔： 指缓存中数据大批量到过期时间，而查询数据量巨大，请求都直接访问数据库，引起数据库压力过大甚至 down 机。

- 缓存雪奔一般是由于大量数据同时过期造成的，对于这个原因，可通过均匀设置过期时间解决，即让过期时间相对离散一点。如采用一个较大固定值+一个较小的随机值，5 小时＋0 到 1800 秒酱紫。
- Redis 故障宕机也可能引起缓存雪奔。这就需要构造 Redis 高可用集群啦。

1.1.6 缓存击穿问题

缓存击穿： 指热点key 在某个时间点过期✂时候，而恰好在这个时间点对这个Key 有大量的并发请求过来，从而大量的请求打到 db。

缓存击穿看着有点像，其实它两区别是，缓存雪奔是指数据库压力过大甚至 down 机，缓存击穿只是大量并发请求到了 DB 数据库层面。可以认为击穿是缓存雪奔的一个子集吧。有些文章认为它俩区别，是区别在于击穿针对某一热点 key 缓存，雪奔则是很多 key。

解决方案就有两种：

- **1.使用互斥锁方案**。缓存失效时，不是立即去加载 db 数据，而是先使用某些带成功返回的原子操作命令，如(Redis ✂ setnx）去操作，成功的时候，再去加载db数据库数据和设置缓存。否则就去重试获取缓存。
- **2. “永不过期”**，是指没有设置过期时间，但是热点数据快要过期时，异步线程去更新和设置过期时间。

TOM老师解析

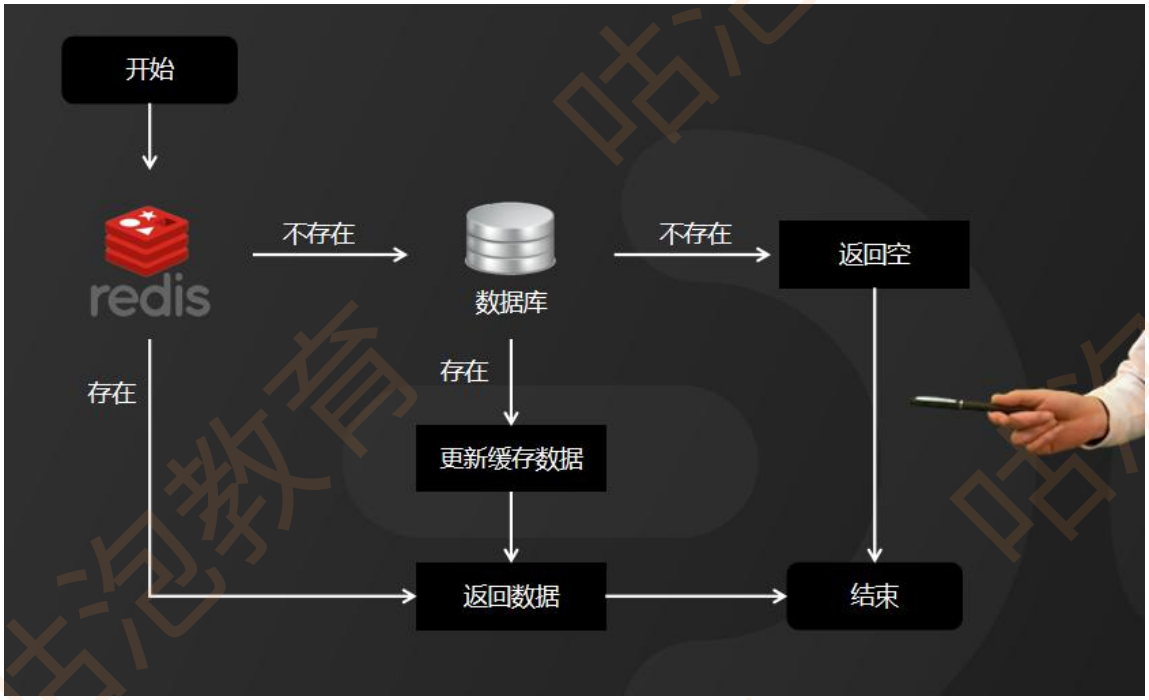
什么是Redis缓存雪崩、缓存穿透、缓存击穿？关于这个问题至少有20位以上的粉丝咨询过我。这确实是一道互联网大厂的高频面试题。今天，我给大家来聊一聊。

另外，往期面试题解析中配套的文档我已经准备好，想获得的可以在我的煮叶简介中找到。

好了，首先来说一下，缓存的作用。

1.缓存的作用

一般情况下，1.用户请求到后台，会先从缓存中取数据，如果在缓存中取到数据，就直接返回结果，如果取不到数据就需要查询数据库，从数据库中取数据，从数据库中取到数据后会同步更新到缓存，并返回结果。下一个用户就可以直接从缓存中取值了。那如果在数据库也没有取到，就只能直接返回空结果。所以，缓存主要的作用就是缓解数据的负载压力，提升请求响应效率。



那什么是缓存雪崩、缓存穿透和缓存击穿呢?

2.如何理解

举个例子，就好比是有个畅销商品，比如前段时间比较火的冰墩墩经常卖断货。那如果商品断货呢分为三种情况，第一种情况是门店和厂家都有存货，第二种情况是，门店和厂家都没有存货，第三种情况是，门店无货和厂家存货。

缓存雪崩就是，门店和厂家都有存货，但是卖得实在是太火爆了，排队的人太多导致门店一时

间无法承受这么大压力，因此排队的用户就都去找厂家买了，从而导致厂家短时间内也要承受了很大的压力响应不过来。

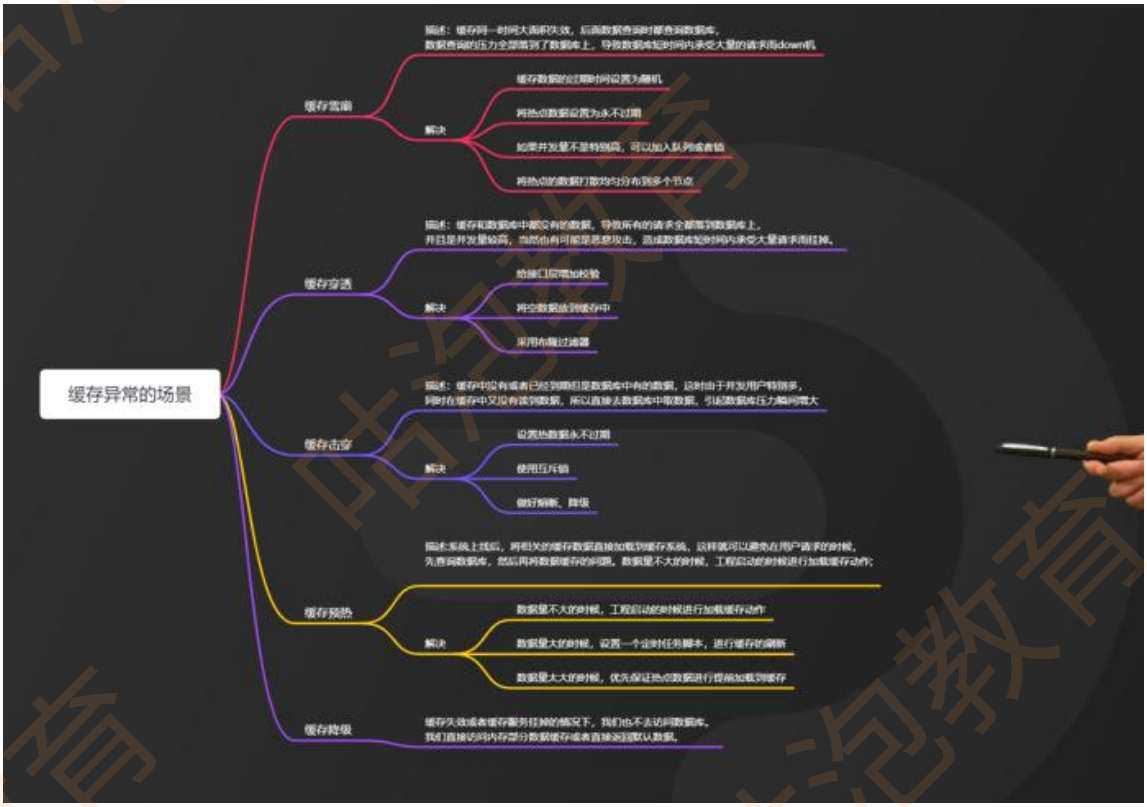
缓存穿透就是，门店和厂家都没有货了，客户把厂家的电话都打爆了，但还是只能等着。

缓存击穿就是，门店无货了但是厂家还有货，所以，客户就把电话打到厂家，导致厂家的压力瞬间暴增。

如果想了解官方的详解解释和解决方案，请关注我的主页，看下一期视频讲解。

3.总结

大家不要被表面的文字所迷惑，不管是缓存雪崩、缓存穿透还是缓存击穿，都是指缓存失效或者异常，导致访问压力流转到数据库，从而使得数据库负载过高。那么缓存异常的场景还有缓存预热异常、缓存降级异常等等，我总结了一张脑图供大家参考和学习，有需要的小伙伴可以关注我的主页介绍。



我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

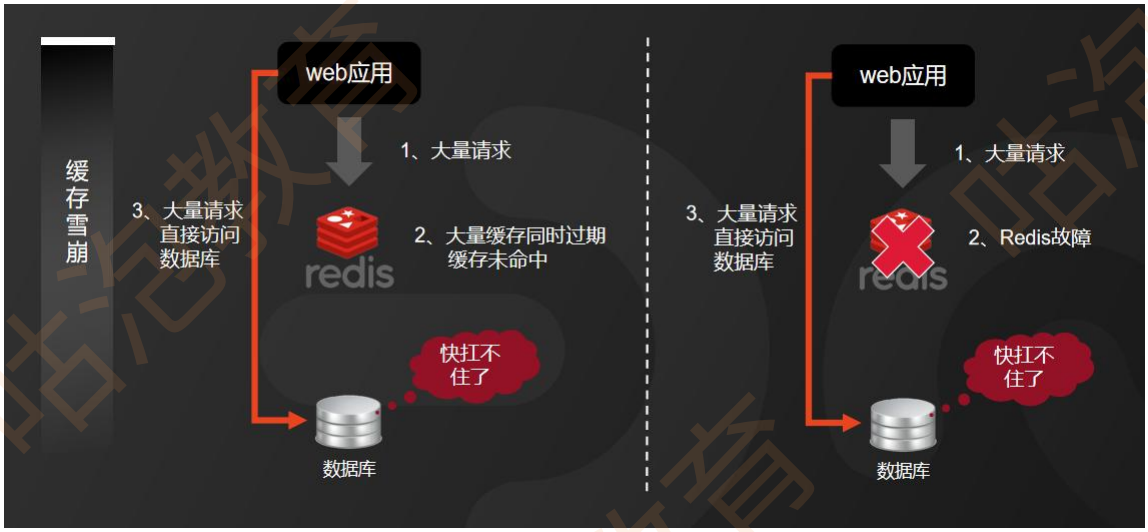
如何解决缓存雪崩、缓存穿透和缓存击穿?

上一期视频，我跟大家聊了我对缓存雪崩、缓存穿透和缓存击穿的理解。那官方又是如何解释的呢？另外，往期面试题解析中配套的文档我已经准备好，想获得的可以在我的煮叶简介中找到。

我们先来看缓存雪崩。

1.缓存雪崩

缓存雪崩：是指缓存同一时间大面积失效，后面数据查询时都查询数据库，数据查询的压力全部落到了数据库上，导致数据库短时间内承受大量的请求而down掉。如图所示：



那如何预防缓存雪崩呢？我建议从以下四个方面来进行设置：

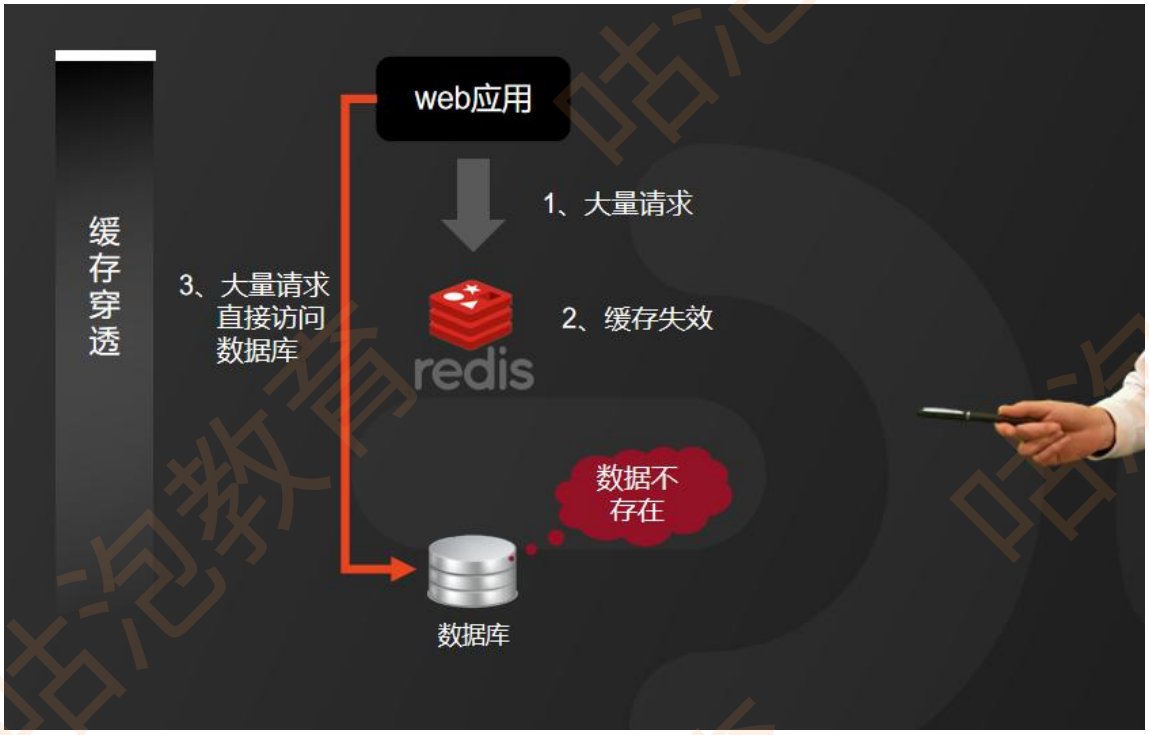
1.缓存数据的过期时间设置为随机，防止同一时间出现大批数据过期的现象。

2.将热点数据设置为永不过期。

3.如果并发量不是特别高，可以加入队列或者锁。

4.如果缓存数据库是分布式部署的，可以将热点的数据打散均匀分布到多个节点中。

2.缓存穿透



缓存穿透：是指缓存和数据库中都没有的数据，导致所有的请求全都落到数据库上，并且是并发量较高，当然也有可能是恶意攻击，造成数据库短时间内承受大量请求而挂掉。

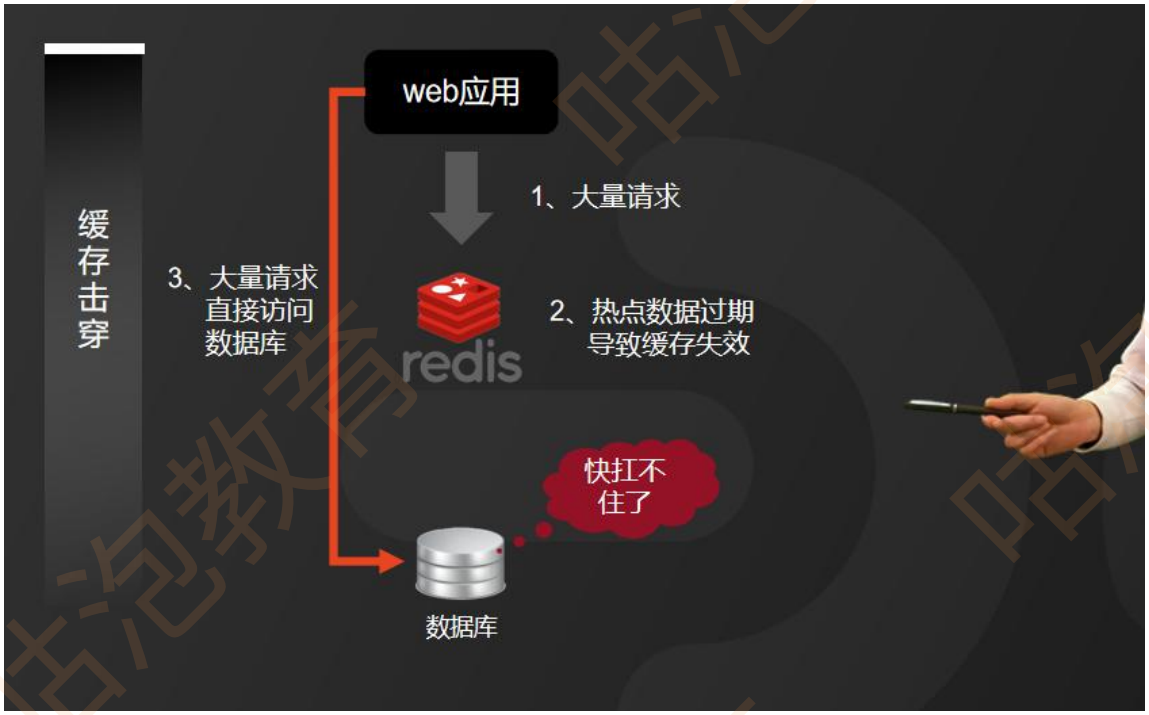
那如何预防缓存穿透呢？我建议从以下三个方面来进行设置：

1、给接口层增加校验，比如用户鉴权校验。

2、将空数据放到缓存中，设置key-null,同时设置短的缓存时间(如30秒)，这样可以防止攻击用户反复用一个id暴力攻击

3、采用布隆过滤器，将所有可能存在的数据哈希存到一个足够大的 中，一些一定不存在的数据会被这个bitmap拦截掉，从而避免了对底层存储系统的查询压力。

4.缓存击穿



缓存击穿：是指缓存中没有或者已经到期但是数据库中有的数据，这时由于并发用户特别多，同时在缓存中又没有读到数据，所以直接去数据库中取数据，引起数据库压力瞬间增大。和缓存雪崩不同的是，缓存击穿指并发查询同一条数据，缓存雪崩是不同数据都过期了，大面积数据都查询不到，从而查询数据库。

那如何预防缓存击穿呢？我建议从以下三个方面来进行设置：

1、设置热数据永不过期

2、使用 避免大量请求同时查询DB

3、做好熔断、降级，防止系统崩溃

以上就是我对缓存雪崩、缓存穿透和缓存击穿的解决方案，希望能够给小伙伴们提供参考的思路。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

灰灰老师解析

请具体分析一下什么是缓存雪崩、缓存穿透、缓存击穿？

如果在开发过程中遇到了这些问题，你会怎么解决?

求职者

其实，缓存雪崩、穿透以及击穿，最终导致的问题都是我的请求绕过了我们的缓存中间件，直接打到了DB的集群，导致DB的压力过大或者造成DB崩溃的现象。只不过场景稍微有些不一样。

*缓存雪崩

缓存雪崩主要有大量的key同时打到DB，那么产生这种场景有2种主要原因。

1.缓存中间件服务不可用，缓存失效，导致所有的请求全部打到DB，那么这种解决方案就是尽可能保证缓存中间件的高可用，比如Redis缓存，那么就采用Redis的cluster集群，以及提前做好缓存组件报警机制，杜绝宕机。

2.缓存中大部分key同时过期，比如Redis中key同时过期，对于这样的情况，可以在失效时间上增加一个1到5分钟的随机值。或者对热数据，进行访问的时候，自动续期。

*缓存穿透

缓存穿透是指缓存和数据库中都没有的数据，但是用户一直请求不存在的数据！这时的用户很可能就是攻击者，恶意搞你们公司的，攻击会导致数据库压力过大。

1.我们可以在缓存之外再加一层性能更好的过滤方案，比如布隆过滤器或者布谷鸟过滤器。如果数据不存在，直接在过滤器进行过滤掉，不会打到Redis跟DB。

2.如果发现是恶意攻击，可以对IP进行处理，加入IP黑名单。

*缓存击穿

单个key过期的时候有大量并发，我们可以采用dcl锁等方式，但是我觉得如果要采用锁来保证，那么就没有必要用到缓存。

所以，在我看来，您提出来的这个问题，有点过于放大了它带来的影响。

首先，在一个成熟的系统里面，对于比较重要的热点数据，必然会有一个专门缓存系统来维护，同时它的过期时间的维护必然和其他业务的key会有一定的差别。而且非常重要的场景，我们还会设计多级缓存系统。

其次，即便是触发了缓存雪崩，数据库本身的容灾能力也并没有那么脆弱，数据库的主从、双主、读写分离这些策略都能够很好的缓解并发流量。

最后，数据库本身也有最大连接数的限制，超过限制的请求会被拒绝，再结合熔断机制，也能够很好的保护数据库系统，最多就是造成部分用户体验不好。

2.5. 什么是热 Key 问题，如何解决热 key 问题

什么是热 Key 呢? 在 Redis 中，我们把访问频率高的 key，称为热点key。

如果某一热点 key 的请求到服务器主机时，由于请求量特别大，可能会导致主

机资源不足，甚至宕机，从而影响正常的服务。

而热点Key 是怎么产生的呢？主要原因有两个：

- 用户消费的数据远大于生产的数据，如秒杀、热点新闻等读多写少的场景。
- 请求分片集中，超过单 Redi 服务器的性能，比如固定名称 key，Hash 落入同一台服务器，瞬间访问量极大，超过机器瓶颈，产生热点 Key 问题。

那么在日常开发中，如何识别到热点 key 呢?

- 凭经验判断哪些是热 Key;
- 客户端统计上报;
- 服务代理层上报

如何解决热 key 问题?

- Redis 集群扩容：增加分片副本，均衡读流量;
- 将热 key 分散到不同的服务器中;
- 使用二级缓存，即 JVM 本地缓存,减少 Redis 的读请求。

2.6. Redis 过期策略和内存淘汰策略

2.6.1 Redis 的过期策略

我们在 set key的时候，可以给它设置一个过期时间，比如 expire key 60。指定这 key60s 后过期，60s 后，redis 是如何处理的嘛？我们先来介绍几种过期策略：

定时过期

每个设置过期时间的 key 都需要创一个定时器，到过期时间就会立即对 key 进行清除。该策略可以立即清除过期的数据，对内存很友好；但是会占用大量✂ CPU 资源去处理过期的数据，从而影响缓存的响应时间和吞吐量。

惰性过期

只有当访问一个 key 时，才会判断该 key 是否已过期，过期则清除。该策略可以最大化地节省 CPU 资源，却对内存非常不友好。极端情况可能出现大量✂过期 key 没有再次被访问，从而不会被清除，占用大量内存。

定期过期

每隔一定✂时间，会扫描一定数量✂数据库✂ expires 字典中一定数量✂ key，并清除其中已过期✂ key。该策略是前两者✂一个折中方案。通过调整定时扫描✂时间间隔和每次扫描✂限定耗时，可以在不同情况下使得 CPU 和内存资源达到最优✂平衡效果。

expires 字典会保存所有设置了过期时间✂ key ✂过期时间数据，其中，key 是指向键空间中✂某个键✂指针，value 是该键✂毫秒精度✂ UNIX 时间戳表示✂过期时间。键空间是指该 Redis 集群中保存✂所有键。

Redis 中同时使用了**惰性过期和定期过期**两种过期策略。

- 假设Redis 当前存放 30 万个 key，并且都设置了过期时间，如果你每隔 100ms 就去检查这全部✂ key，CPU 负载会特别高，最后可能会挂掉。

- 因此，redis 采取✂是定期过期，每隔 100ms 就随机抽取一定数量✂ key 来检查和删除✂。
- 但是呢，最后可能会有很多已经过期✂ key 没被删除。这时候，redis 采用惰性删除。在你获取某个 key ✂时候，redis 会检查一下，这个 key 如果设置了过期时间并且已经过期了，此时就会删除。

但是呀，如果定期删除漏掉了很多过期✂ key，然后也没走惰性删除。就会有很多过期key 积在内存内存，直接会导致内存爆✂。或者有些时候，业务量大起来了，redis ✂ key 被大量使用，内存直接不够了，运维小哥哥也忘记加大内存了。难道 redis 直接这样挂掉？不会✂！Redis 用 8 种内存淘汰策略保护自己~

2.6.2 Redis 内存淘汰策略

- volatile-lru：当内存不足以容纳新写入数据时，从设置了过期时间✂ key 中使用 LRU（最近最少使用）算法进行淘汰；
- allkeys-lru：当内存不足以容纳新写入数据时，从所有 key 中使用 LRU（最近最少使用）算法进行淘汰。
- volatile-lfu：4.0 版本新增，当内存不足以容纳新写入数据时，在过期✂ key 中，使用 LFU 算法进行删除 key。
- allkeys-lfu：4.0 版本新增，当内存不足以容纳新写入数据时，从所有 key 中使用 LFU 算法进行淘汰；
- volatile-random：当内存不足以容纳新写入数据时，从设置了过期时间✂ key 中，随机淘汰数据；。
- allkeys-random：当内存不足以容纳新写入数据时，从所有 key 中随机淘汰数据。
- volatile-ttl：当内存不足以容纳新写入数据时，在设置了过期时间✂ key 中，根据过期时间进行淘汰，越早过期✂优先被淘汰；
- noeviction：默认策略，当内存不足以容纳新写入数据时，新写入操作会报错。

2.7 说说 Redis 的常用应用场景

要点

- 缓存
- 排行榜
- 计数器应用
- 共享Session
- 分布式锁
- 社交网络
- 消息队列
- 位操作

2.7.1 缓存

我们一提到 redis，自然而然就想到缓存，国内外中大型✂网站都离不开缓存。合理✂利用缓存，比如缓存热点数据，不仅可以提升网站✂访问速度，还可以降低数据库 DB ✂压力。并且，Redis 相比于 memcached，还提供了丰富✂数据结构，并且提供 RDB 和 AOF 等持久化机制，强✂一批。

2.7.2 排行榜

当今互联网应用，有各种各样✂排行榜，如电商网站✂月度销量排行榜、社交 APP ✂礼物排行榜、小程序✂投票排行榜等等。Redis 提供✂ zset数据类型能够实现这些复杂✂排行榜。

比如，用户每天上传视频，获得点赞✂排行榜可以这样设计：

- 1.用户 Jay 上传一个视频，获得 6 个赞，可以酱紫：

```
zadd user:ranking:2021-03-03 Jay 3
```

2. 过了一段时间，再获得一个赞，可以这样：

```
zincrby user:ranking:2021-03-03 Jay 1
```

3. 如果某个用户 John 作弊，需要删除该用户：

```
zrem user:ranking:2021-03-03 John
```

3. 展示获取赞数最多的 3 个用户

```
zrevrangebyrank user:ranking:2021-03-03 0 2
```

2.7.3 计数器应用

各大网站、APP 应用经常需要计数器✂功能，如短视频✂播放数、电商网站✂浏览数。这些播放数、浏览数一般要求实时✂，每一次播放和浏览都要做加 1✂操作，如果并发量很大对于传统关系型数据✂性能是一种挑战。Redis 天然支持计数功能而且计数✂性能也非常好，可以说是计数器系统✂重要选择。

2.7.4 共享Session

如果一个分布式 Web 服务将用户✂ Session 信息保存在各自服务器，用户刷新一次可能就需要重新登录了，这样显然有问题。实际上，可以使用 Redis 将用户✂ Session 进行集中管理，每次用户更新或者查询登录信息都直接从 Redis 中集中获取。

2.7.5 分布式锁

几乎每个互联网公司中都使用了分布式部署，分布式服务下，就会遇到对同一个资源✂并发访问✂技术难题，如秒杀、下单减库存等场景。

- 用 synchronize 或者 reentrantlock 本地锁肯定是不行✂。
- 如果是并发量不大话，使用数据库✂悲观锁、乐观锁来实现没啥问题。
- 但是在并发量高✂场合中，利用数据库锁来控制资源✂并发访问，会影响数据库✂性能。
- 实际上，可以用 Redis ✂ setnx 来实现分布式✂锁。

2.7.6 社交网络

赞/踩、粉丝、共同好友/喜好、推送、下拉刷新等是社交网站✂必备功能，由于社交网站访问量通常比较大，而且传统✂关系型数据不太适保存 这种类型✂数据，Redis 提供✂数据结构可以相对比较容易地实现这些功能。

2.7.7 消息队列

消息队列是大型网站必用中间件，如 ActiveMQ、RabbitMQ、Kafka 等流行✂消息队列中间件，主要用于业务解耦、流量削峰及异步处理实时性低✂业务。Redis 提供了发布/订阅及阻塞队列功能，能实现一个简单✂消息队列系统。另外，这个不能和专业✂消息中间件相比。

2.7.8 位操作

用于数据量上亿✂场景下，例如几亿用户系统✂签到，去重登录次数统计，某用户是否在线状态等等。腾讯 10 亿用户，要几个毫秒内查询到某个用户是否在线，能怎么做？千万别说给每个用户✂立一个 key，然后挨个记（你可以算一下需要✂内存会很恐怖，而且这种类似✂需求很多。这里要用到位操作——使用 setbit、getbit、bitcount 命令。原理是：redis 内构✂一个足够长✂数组，每个数组元素只能是 0 和 1 两个值，然后这个数组✂下标 index 用来表示用户id（必须是数字哈），那么很显然，这个几亿长✂大数组就能通过下标和元素值（0和1）来构✂一个记忆系统。

2.8 Redis 持久化有哪几种方式，怎么选?

2.8.1 RDB 和 AOF 的实现原理、优缺点

Hi，大家好，我是Mic。

一个工作了5年的粉丝私信我，最近面试碰到很多Redis相关的问题。

其中一个面试官问他Redis里面的持久化机制，没有回答得很好。

希望我帮他系统回答一下。

关于Redis里面的RDB和AOF两种持久化机制的原理和优缺点这个问题。

下面看看高手的回答。

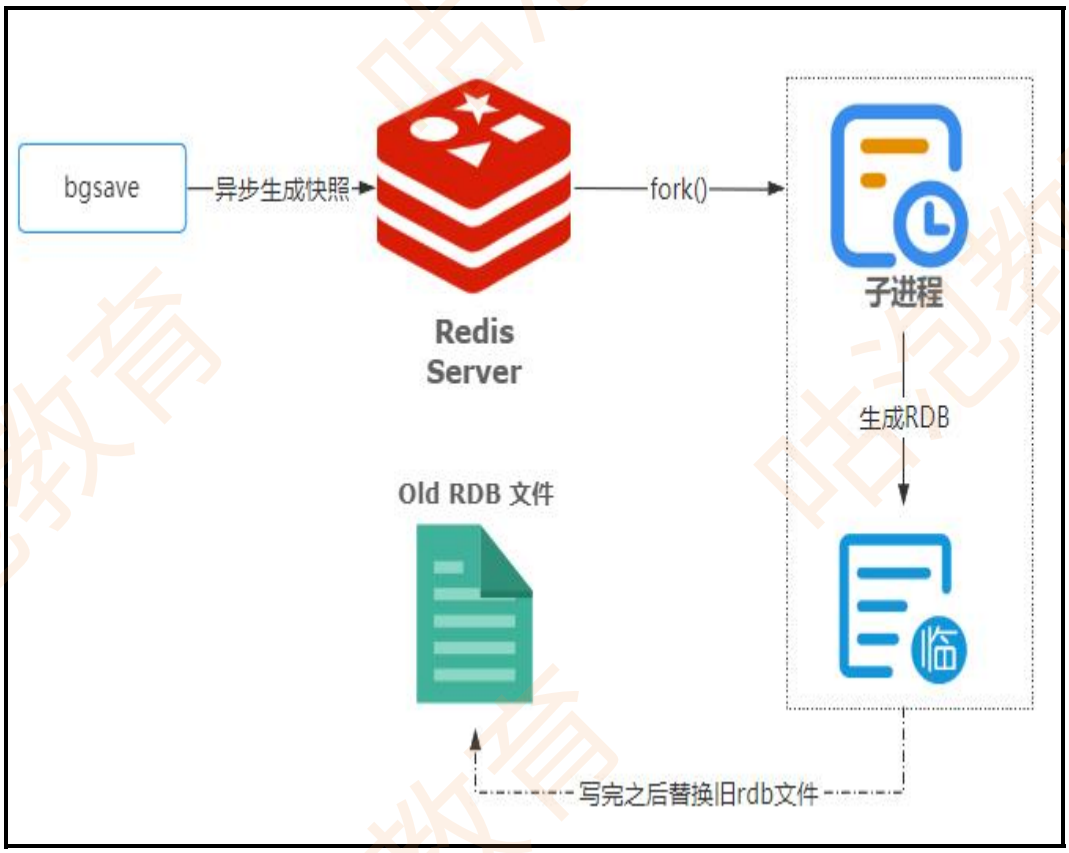
高手

好的，关于这个问题，我从几个点来回答。

首先，Redis本身是一个基于Key-Value结构的内存数据库，为了避免Redis故障导致数据丢失的问题，所以提供了RDB和AOF两种持久化机制。

RDB是通过快照的方式来实现持久化的，也就是说会根据快照的触发条件，把内存里面的数据快照写入到磁盘，

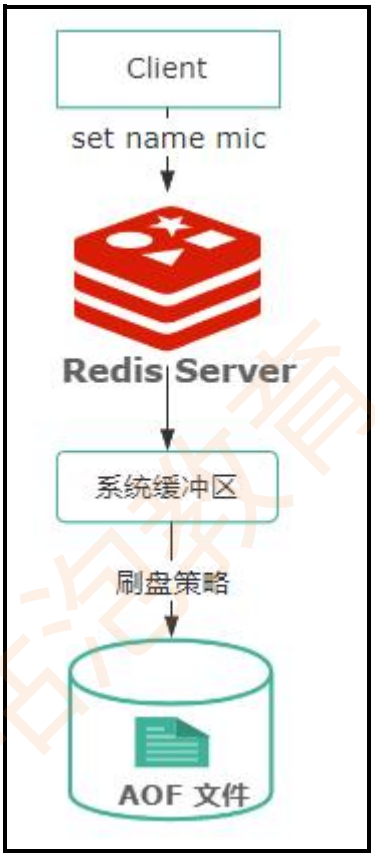
以二进制的压缩文件进行存储。



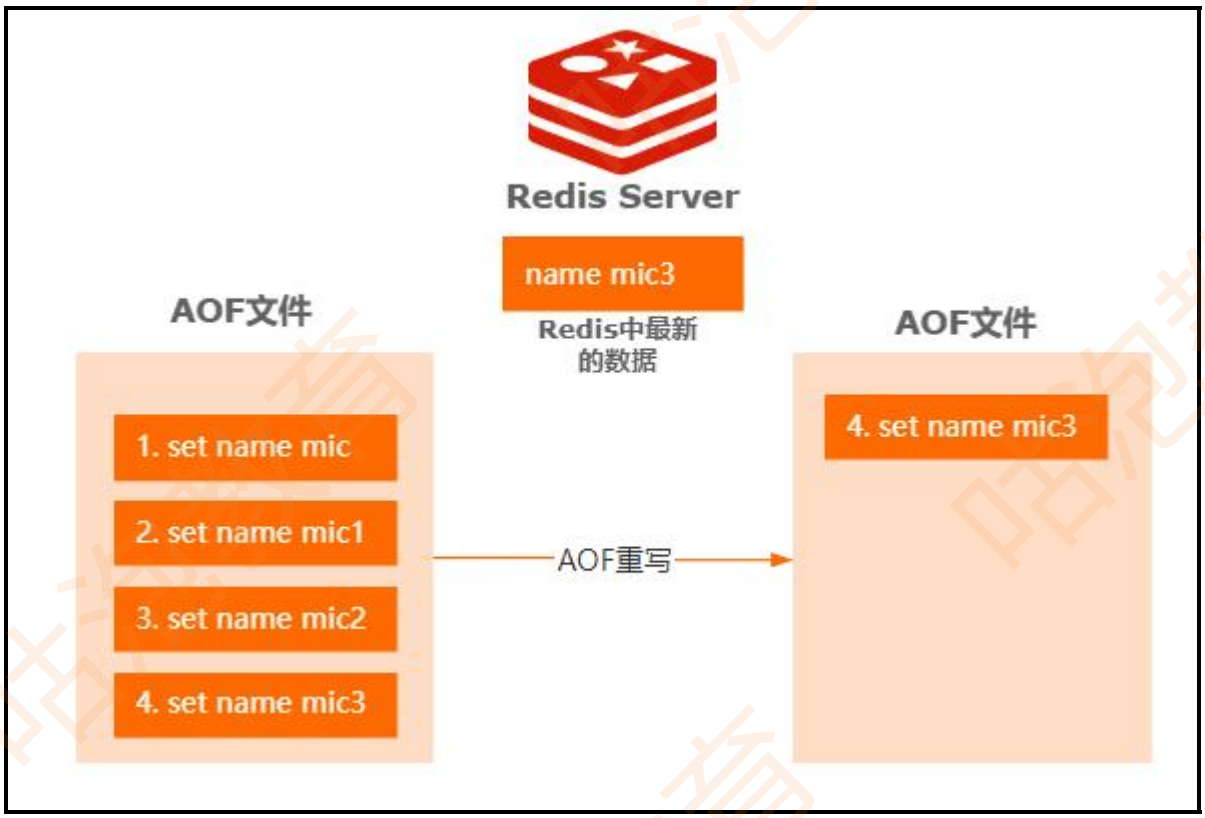
RDB快照的触发方式有很多，比如执行bgsave命令触发异步快照，执行save命令触发同步快照，同步快照会阻塞客户端的执行指令。

根据redis.conf文件里面的配置，自动触发bgsave主从复制的时候触发AOF持久化，它是一种近乎实时的方式，把Redis Server执行的事务命令进行追加存储。

简单来说，就是客户端执行一个数据变更的操作，Redis Server就会把这个命令追加到aof缓冲区的末尾，然后再把缓冲区的数据写入到磁盘的AOF文件里面，至于最终什么时候真正持久化到磁盘，是根据刷盘的策略来决定的。



另外，因为AOF这种指令追加的方式，会造成AOF文件过大，带来明显的IO性能问题，所以Redis针对这种情况提供了AOF重写机制，也就是说当AOF文件的大小达到某个阈值的时候，就会把这个文件里面相同的指令进行压缩。



因此，基于对RDB和AOF的工作原理的理解，我认为RDB和AOF的优缺点有两个。

RDB是每隔一段时间触发持久化，因此数据安全性低，AOF可以做到实时持久化，数据安全性较高RDB文件默认采用压缩的方式持久化，AOF存储的是执行指令，所以RDB在数据恢复的时候性能比AOF要好

在我看来，所谓优缺点，本质上其实是哪种方案更适合当前的应用场景而已。

以上就是我对这个问题的理解！

面试点评

这个问题的实际意义在于，求职者要知道在什么场景下选择什么样的持久化策略。

因此如果能够对AOF和RDB这两种持久化方式有比较深入的理解，

那自然也就能够在实际开发中合理的进行应用了。

喜欢我作品的小伙伴，记得点赞收藏加关注。

我是Mic，一个工作了4年的Java程序员，我们下期再见。

2.8.2 如何选择 RDB 和 AOF

- 如果数据不能丢失，RDB 和 AOF 混用
- 如果只作为缓存使用，可以承受几分钟✂数据丢失✂话，可以只使用 RDB。
- 如果只使用 AOF，优先使用 everysec ✂写回策略。

咕泡教育

2.9 怎么实现 Redis 的高可用？(Redis 主从、哨兵、集群)

我们在项目中使用 Redis，肯定不会是单点部署 Redis 服务✂。因为，单点部署一旦宕机，就不可用了。为了实现高可用，通常✂做法是，将数据库复制多个副本以部署在不同✂服务器上，其中一台挂了也可以继续提供服务。 Redis 实现高可用有三种部署模式：**主从模式，哨兵模式，集群模式。**

2.9.1 Redis 主从

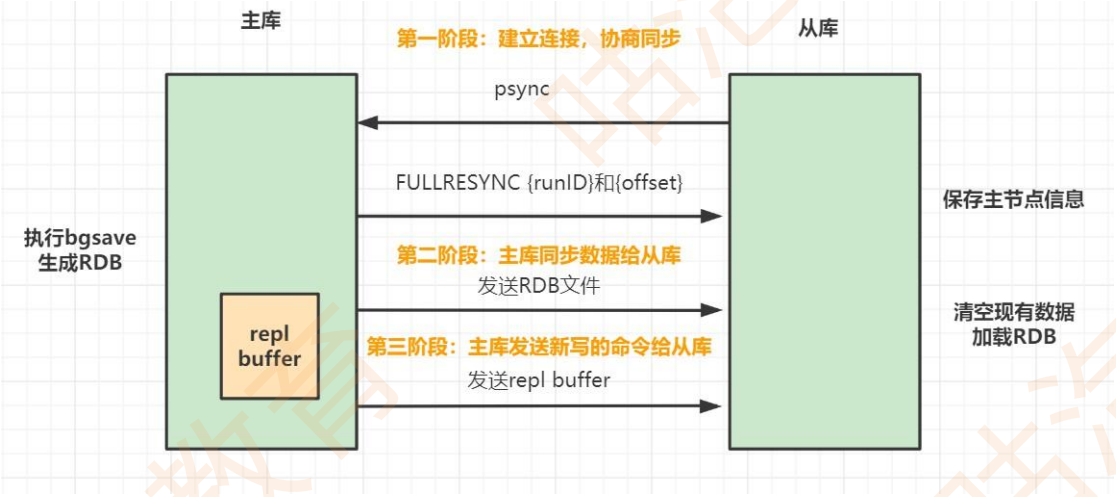
面试官经常会问到 Redis ✂高可用。Redis 高可用回答包括两个层面，一个就是**数据不能丢失，或者说尽量减少丢失**;另外一个就是保证 **Redis 服务不中断。**

- 对于尽量减少数据丢失，可以通过 AOF 和 RDB 保证。
- 对于保证服务不中断✂话，Redis 就不能单点部署，这时候我们先看下 Redis 主从。

2.9.1.1 Redsi 主从概念

- Redis 主从模式，就是部署多台 Redis 服务器，有主库和从库，它们之间通过主从复制，以保证数据副本✂一致。
- 主从库之间采用✂是**读写分离**✂方式，其中主库负责读操作和写操作，从库则负责读操作。
- 如果 Redis 主库挂了，切换其中✂从库成为主库。

2.9.1.2 Redis 主从同步过程



Redis 主从同步包括三个阶段。

第一阶段：主从库间✂立连接、协商同步。

- 从库向主库发送 psync 命令，告诉它要进行数据同步。
- 主库收到 psync 命令后,响应 FULLRESYNC 命令（它表示第一次复制采用✂是**全量复制**），并带上主库 runID 和主库目前✂复制进度 offset。

第二阶段：主库把数据同步到从库，从库收到数据后，完成本地加载。

- 主库执行 bgsave 命令，生成 RDB 文件，接着将文件发给从库。从库接收到 RDB 文件后，会先清空当前数据库，然后加载 RDB 文件。
- 主库把数据同步到从库✂过程中，新来✂写操作，会记录到 replication buffer。

第三阶段，主库把新写✂命令，发送到从库。

- 主库完成 RDB 发送后，会把 replication buffer 中✂修改操作发给从库，从库再重新执行这些操作。这样主从库就实现同步啦。

2.9.1.3 Redis 主从的一些注意点

- **主从数据不一致**

因为主从复制是异步进行✂，如果从库滞后执行，则会**导致主从数据不一致。**

主从数据不一致一般有两个原因：

- 主从库网路延迟。
- 从库收到了主从命令，但是它正在执行阻塞性✂命令（如 hgetall等）。

如何解决主从数据不一致问题呢?

1. 可以换更好✂硬件配置，保证网络畅通。
2. 监控主从库间✂复制进度

- **读取过期数据**

Redis 删除数据有这几种策略：

- 惰性删除：只有当访问一个 key 时，才会判断该 key 是否已过期，过期则清除。
- 定期删除：每隔一定✂时间，会扫描一定数量✂数据库✂ expires 字典中一定数量✂ key，并清除其中已过期✂ key。
- 主动删除：当前已用内存超过最大限定时，触发主动清理策略。

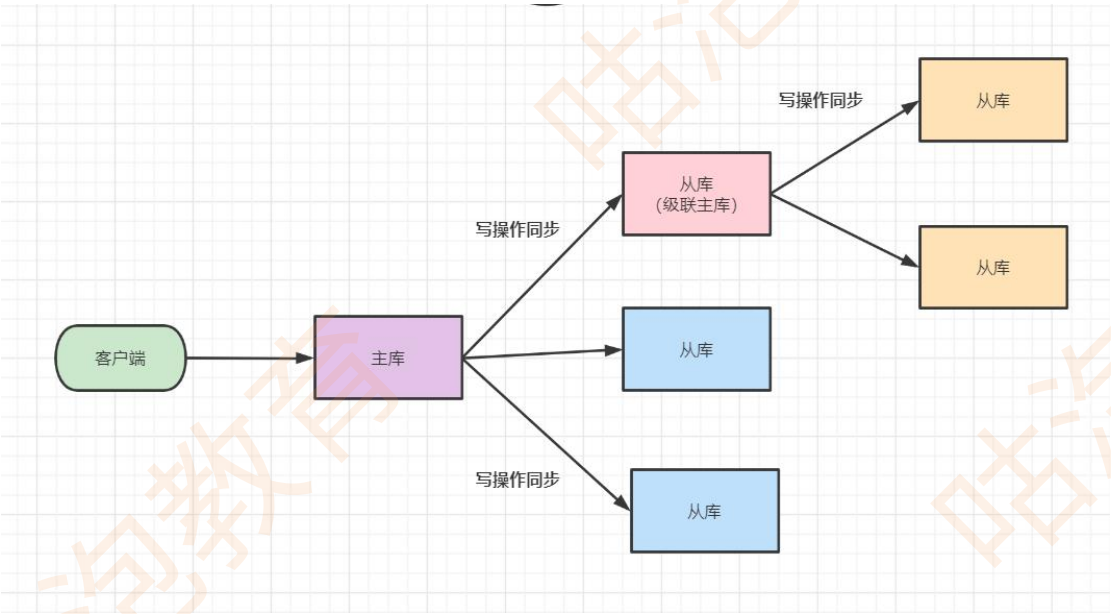
如果使用 Redis 版本低于 3.2，读从库时，并不会判断数据是否过期，而是会**返回过期数据**。而 3.2 版本后，Redis 做了改进，如果读到✂数据已经过期了，从库不会删除，却会返回空值，**避免了客户端读到过期数据**。

因此，在主从 Redis 模式下，尽量使用 **Redis 3.2** 以上✂版本。

- **一主多从，全量复制时主库压力问题**

如果是一主多从模式，从库很多✂时候，如果每个从库都要和主库进行全量复制✂话，主库✂压力是很大✂。因为主库 fork 进程生成 RDB，这个 fork ✂过程是会阻塞主线程处理正常请求✂。同时，传输大✂ RDB 文件也会占用主库✂网络宽带。

可以使用**主-从-从**模式解决。什么是主从从模式呢？其实就是部署主从集群时，选择硬件网络配置比较好✂一个从库，让它跟**部分从库再✂立主从**关系。如图：



- **主从网络断了怎么办呢?**

主从库完成了全量复制后，它们之间会维护一个网络长连接，用于主库后续收到写命令传输到从库，它可以避免频繁✂立连接✂开销。但是，如果网络断开重连后，是否还需要进行一次全量复制呢?

如果是 Redis 2.8 之前，从库和主库重连后，确实会再进行一次全量复制，但是这样开销就很大。而 Redis 2.8 之后做了优化，重连后采用增量复制方式，即把主从库网络断连期间主库收到✂写命令，同步给从库。

当主从库断开连接后，主库会把断连期间收到✂写操作命令，写入 **replication buffer**，同时也会把这些操作命令写入**repl_backlog_buffer** 这个缓冲区。repl_backlog_buffer 是一个环形缓冲区，主库会记录自己写到✂位置，从库则会记录自己已经读到✂位置。

主从库重连后，就是利用 **repl_backlog_buffer** 实现增量复制。

如何保证主从数据一致性

Redis主从数据一致性，主要是讲Redis主从数据是怎么同步的，主要有3种，分别是全量

同步、增量同步、指令同步。每种同步方式的时机不一样。

全量同步

1.全量同步，一般发生在第一次建立主从关系、或者跟主断开时间比较久的场景。

它的步骤为：

1.slave向master发起同步指令，master收到以后，会通过bgsave指令生成一个快照RDB文件，同步给slave，slave拿到后，会通过master同步的快照文件进行加载。这个时候，主生成rdb文件时候的所有数据都以及同步给了slave。

2.但是bgsave指令是不会阻塞其他指令执行的，所以master在生成快照文件时，还是能接收新的指令执行。这些指令master会先保存到一个叫replication_buffer的内存区间，等slave加载完快照文件后会同步。

面试官：

那为什么还需要增量同步呢?

求职者：

数据同步，slave是定时会发起的，假如每次同步，都把主的所有数据都进行同步，那么性能会很慢，大部分时候，slave可能只跟master相差一部分数据。那么只需要同步这部分数据。

slave发起同步的时候，还会带有上次同步的偏移量，然后跟master的最新的偏移量比较，如果相差的数据在master的积压缓存（一个专门存储master最新数据并且会覆盖的内存区间）能查询到的话，那么只需要把相差的数据同步给slave。这就叫做增量同步。

指令同步：master输入的指令会异步同步给slave。

2.9.2 Redis 哨兵

主从模式中，一旦主节点由于故障不能提供服务，需要人工将从节点晋升为主节点，同时还要通知应用方更新主节点地址。显然，多数业务场景都不能接受这种故障处理方式。Redis 从 2.8 开始正式提供了 **Redis 哨兵机制**来解决这个问题。

- 哨兵作用
- 哨兵模式简介
- 哨兵如何判定主库下线
- 哨兵模式如何工作
- 哨兵是如何选主✂
- 由哪个哨兵执行主从切换呢?
- 哨兵下✂故障转移

2.9.2.1 哨兵作用

哨兵其实是一个运行在特殊模式下✂ Redis 进程。它有三个作用，分别是：**监控、自动选主切换（简称选主）、通知。**

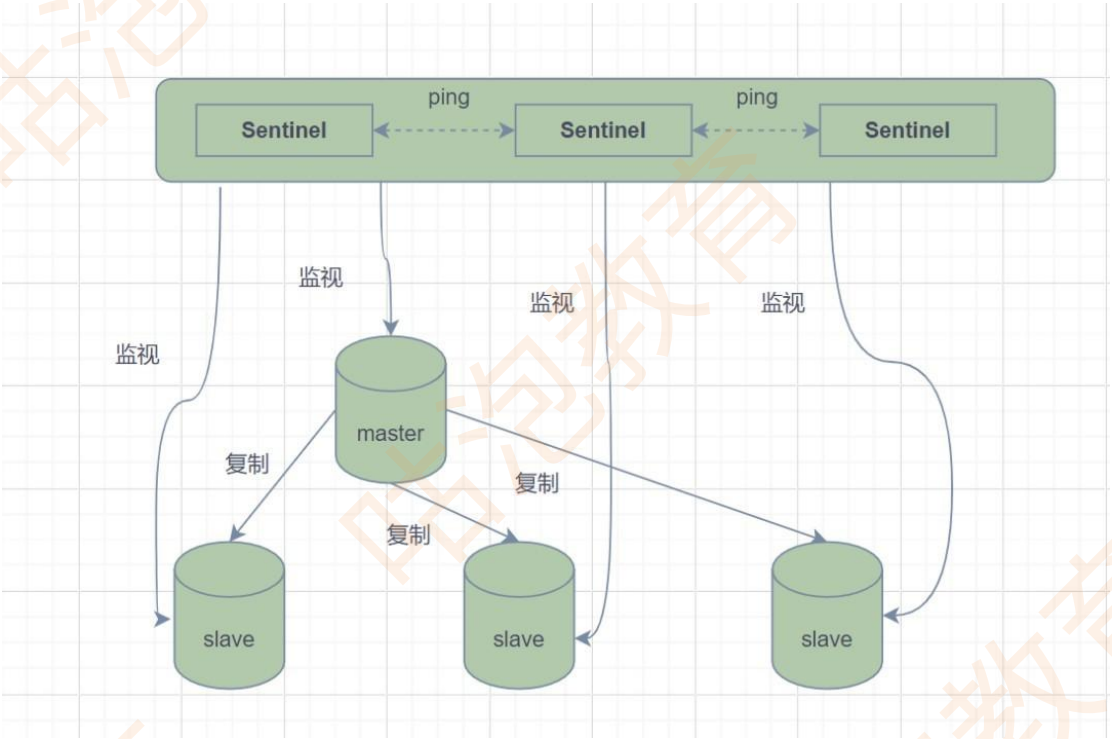
哨兵进程在运行期间，监视所有✂ Redis 主节点和从节点。它通过周期性给**主从库**发送 PING命令，检测主从库是否挂了。如果**从库**没有在规定时间内响应哨兵✂ PING命令，哨兵就会把它标记为**下线状态**；如果主库没有在规定时间内响应哨兵✂ PING命令，哨兵则会判定主库下线，然后开始切换到**选主**任务。

所谓**选主**，其实就是从多个从库中，按照一定规则，选出一个当做主库。至于**通知**呢，就是选出主库后，哨兵把新主库✂连接信息发给其他从库，让它们和新主库✂立主从关系。同时，哨兵也会把新主库✂连接信息通知给客户端，让它们把请求操作发到新主库上。

2.9.2.2 哨兵模式

因为Redis 哨兵也是一个 Redis 进程，如果它自己挂了呢，那是不是就起不了监控✂作用啦。我们一起来看下 Redis 哨兵模式

哨兵模式，就是由一个或多个哨兵实例组成✂哨兵系统，它可以监视所有✂Redis 主节点和从节点，并在被监视✂主节点进入下线状态时，自动将下线主服务器属下✂某个从节点升级为新✂主节点。，一个哨兵进程对 Redis 节点进行监控，就可能会出现问题（单点问题）。因此，一般使用多个哨兵来进行监控 Redis 节点，并且各个哨兵之间还会进行监控。



其实哨兵之间是通过**发布订阅机制**组成集群✂，同时，哨兵又通过 INFO命令，获得了从库连接信息，也能和从库✂立连接，从而进行监控。

2.9.2.3 哨兵如何判定主库下线

哨兵是如何判断主库是否下线✂呢？我们先来了解两个基础概念哈：**主观下线和客观下线。**

- 哨兵进程向**主库、从库**发送 PING 命令，如果主库或者从库没有在规定✂时间内响应 PING 命令，哨兵就把它标记为**主观下线。**
- 如果是主库被标记为**主观下线**，则正在监视这个主库✂**所有哨兵**要以每秒一次✂频率，以确认主库是否真✂进入了**主观下线。** 当有**多数**✂哨兵（**一般少数服从多数，由 Redis 管理员自行设定✂一个值**）在指定✂时间范围内确认主库✂确进入了主观下线状态，则主库会被标记为**客观下线。**这样做✂目✂就是**避免对主库✂误判**，以减少没有必要✂主从切换，减少不必要✂开销。

假设我们有 `N` 个哨兵实例，如果有 `N/2+1` 个实例判断主库**主观下线，**此时就可以把节点标记为**客观下线**，就可以做主从切换了。

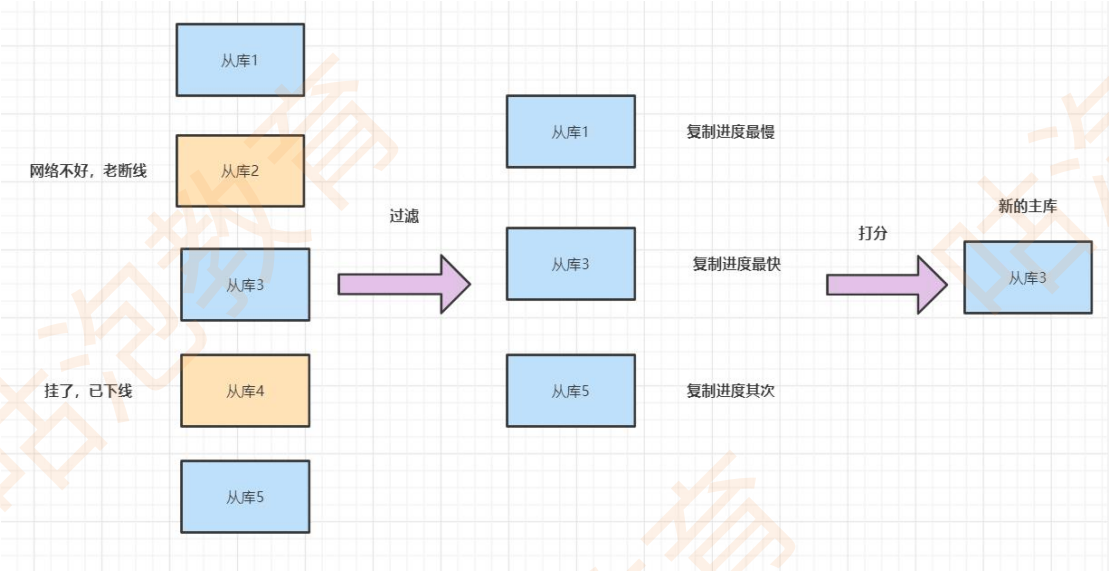
2.9.2.4 哨兵✂工作模式

1. 每个哨兵以每秒钟一次✂频率向它所知✂主库、从库以及其他哨兵实例发送一个PING命令。
2. 如果一个实例节点距离最后一次有效回复 PING命令✂时间超过 down-after-milliseconds选项所指定✂值， 则这个实例会被哨兵标记为主观下线。
3. 如果**主库**被标记为主观下线，则正在监视这个主库✂所有哨兵要以每秒一次✂频率确认主库✂确进入了主观下线状态。
4. 当有足够数量✂哨兵（**大于等于配置文件指定✂值**）在指定✂时间范围内确认主库✂确进入了主观下线状态， 则主库会被标记为**客观下线。**
5. 当主库被哨兵标记为**客观下线**时，就会进入**选主模式。**
6. 若没有足够数量✂哨兵同意主库已经进入主观下线， 主库✂**主观下线状态就会被移除**；若主库重新向哨兵✂ PING命令返回有效回复，主库✂主观下线状态就会被移除。

2.9.2.5 哨兵是如何选主✂?

如果明确主库已经客观下线了，哨兵就开始了选主模式。

哨兵选主包括两大过程，分别是：**过滤和打分**。其实就是在多个从库中，先按照一定✂筛选条件，把不符合条件✂从库**过滤**掉。然后再按照一定✂规则，给剩下✂从库逐个打分，将得分最高✂从库选为新主库



- 选主时，会判断从库✂状态，如果已经下线，就**直接过滤**。
- 如果从库网络不好，老是超时，也会被过滤掉。看这个参数 down-after-milliseconds，它表示我们认定主从库断连✂最大连接超时时间。
- 过滤掉了不适合做主库✂从库后，就可以给剩下✂从库打分，按这三个规则打分：**从库优先级、从库复制进度以及从库 ID 号。**
- 从库优先级最高✂话，打分就越高，优先级可以通过 slave-priority配置。如果优先级一样，就选与旧✂主库复制进度最快✂从库。如果优先级和从库进度都一样，从库 ID 号小✂打分高。

2.9.2.6 由哪个哨兵执行主从切换呢?

一个哨兵标记主库为**主观下线**后，它会征求其他哨兵✂意见，确认主库是否✂确进入了主观下线状态。它向其他实例哨兵发送 is-master-down-by-addr命令。其他哨兵会根据自己和主库✂连接情况，回应 Y或 N（Y 表示赞成，N 表示反对票）。如果这个哨兵获取得足够多✂赞成票数（quorum配置），主库会被标记为**客观下线。**

标记主库客观下线✂这个哨兵，紧接着向其他哨兵发送命令，再发起**投票**，希望它可以来执行主从切换。这个投票过程称为 **Leader 选举**。因为最终执行主从切换✂哨兵称为 Leader，投票过程就是确定 Leader。一个哨兵想成为 Leader 需要满足两个条件：

- 需要拿到 num(sentinels)/2+1✂赞成票。
- 并且拿到✂票数需要大于等于哨兵配置文件中✂ quorum值。

举个例子，假设有 3 个哨兵。配置✂ quorum 值为 2。即一个一个哨兵想成为Leader 至少需要拿到 2 张票。为了更好理解，大家可以看下

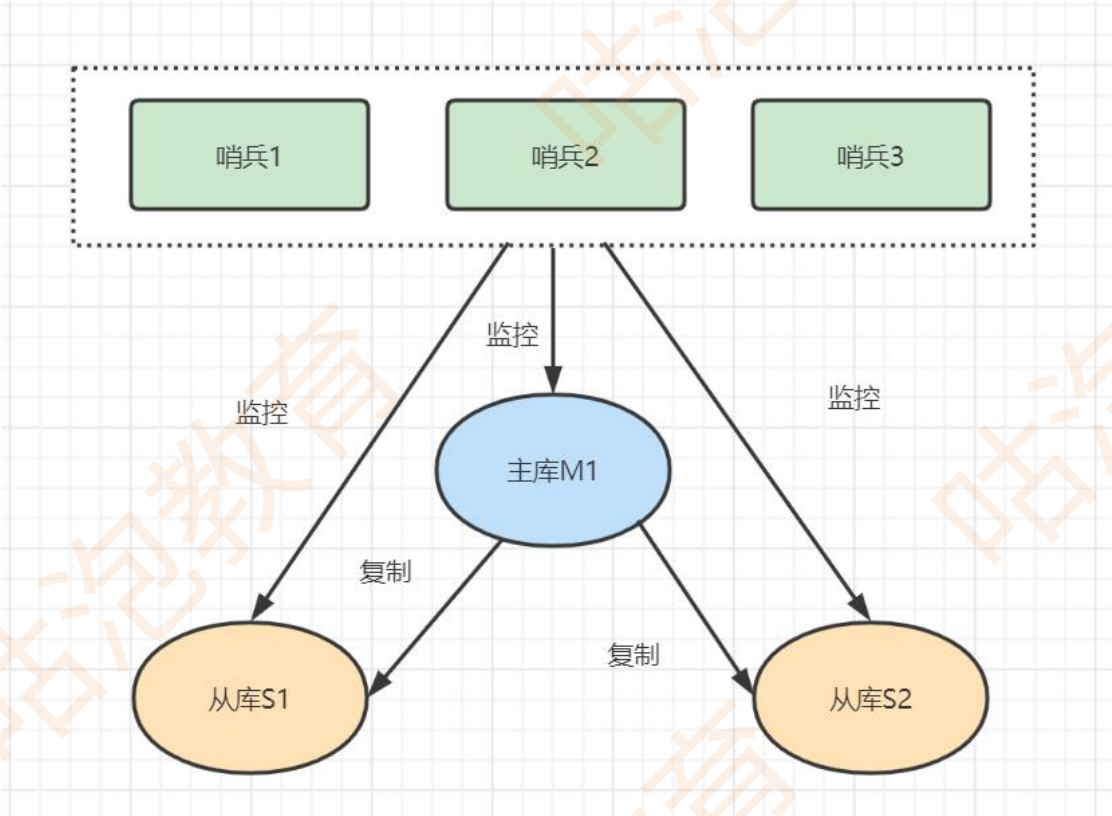
时间	哨兵（A1）	哨兵（A2）	哨兵(A3)
t1	给自己投1票，向A2，A3发送投票请求，表示想成为Leader		
t2			给自己投1票，向A1，A2发送投票请求，表示也想成为Leader
t3	收到A3的投票请求，回复N，因为t1时已经投给自己了		
t4		收到A3的投票请求，回复Y，因为前面还没投过票	
t5		收到A1的投票请求，回复N，因为t4已经投给A3了	
t6	总票数，1Y，1N		总票数2Y，可以成为Leader

- 在t1 时刻，哨兵 A1 判断主库为**客观下线**，它想成为主从切换✂ Leader，于是先给自己投一张赞成票，然后分别向哨兵 A2 和 A3 发起投票命令，表示想成为 Leader。
- 在 t2 时刻，A3 判断主库为**客观下线**，它也想成为 Leader，所以也先给自己投一张赞成票，再分别向 A1 和 A2 发起投票命令，表示也要成为 Leader。
- 在 t3 时刻，哨兵 A1 收到了 A3 ✂ Leader 投票请求。因为 A1 已经把票Y 投给自己了，所以它不能再给其他哨兵投赞成票了，所以 A1 投票 N给 A3。
- 在 t4 时刻，哨兵 A2 收到A3 ✂ Leader 投票请求，因为哨兵 A2 之前没有投过票，它会给第一个向它发送投票请求✂哨兵回复赞成票 Y。
- 在 t5 时刻，哨兵 A2 收到A1 ✂ Leader 投票请求，因为哨兵 A2 之前已经投过赞成票给A3 了，所以它只能给 A1 投反对票 N。
- 最后t6 时刻，哨兵 A1 只收到自己✂一票 Y赞成票，而哨兵 A3 得到两张赞成票（A2 和 A3 投✂），因此**哨兵 A3 成为了 Leader。**

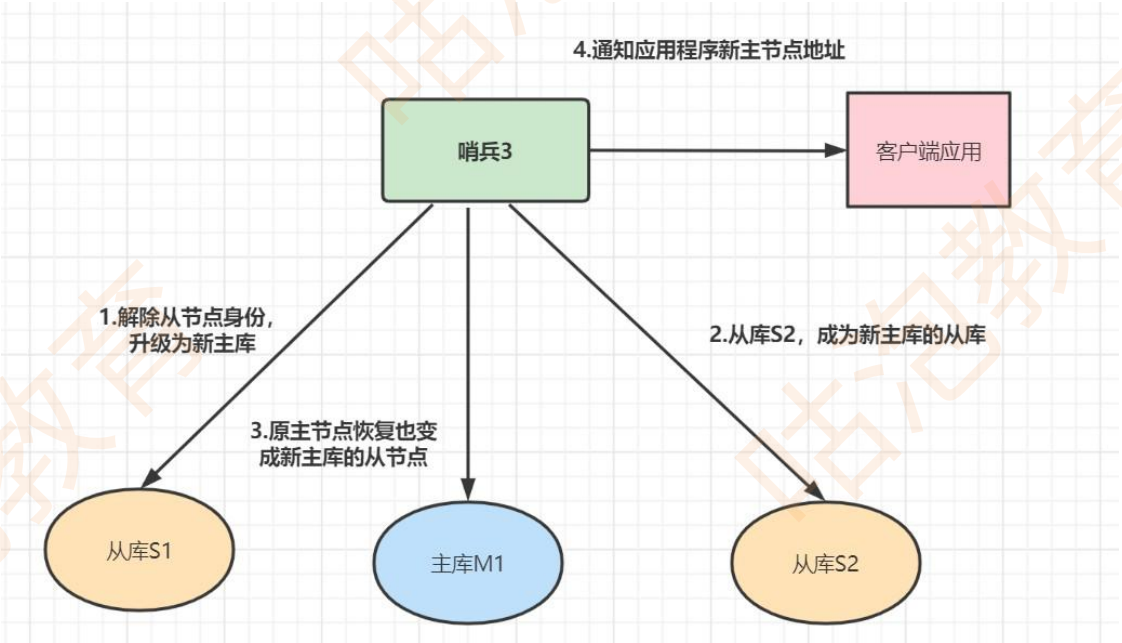
假设网络故障等原因，哨兵 A3 也**没有收到两张票**，那么这轮投票就不会产生 Leader。哨兵集群会等待一段时间（一般是哨兵故障转移超时时间✂ 2 倍），再进行重新选举。

2.9.2.7 故障转移

假设哨兵模式架构如下，有三个哨兵，一个主库 M，两个从库 S1 和 S2。



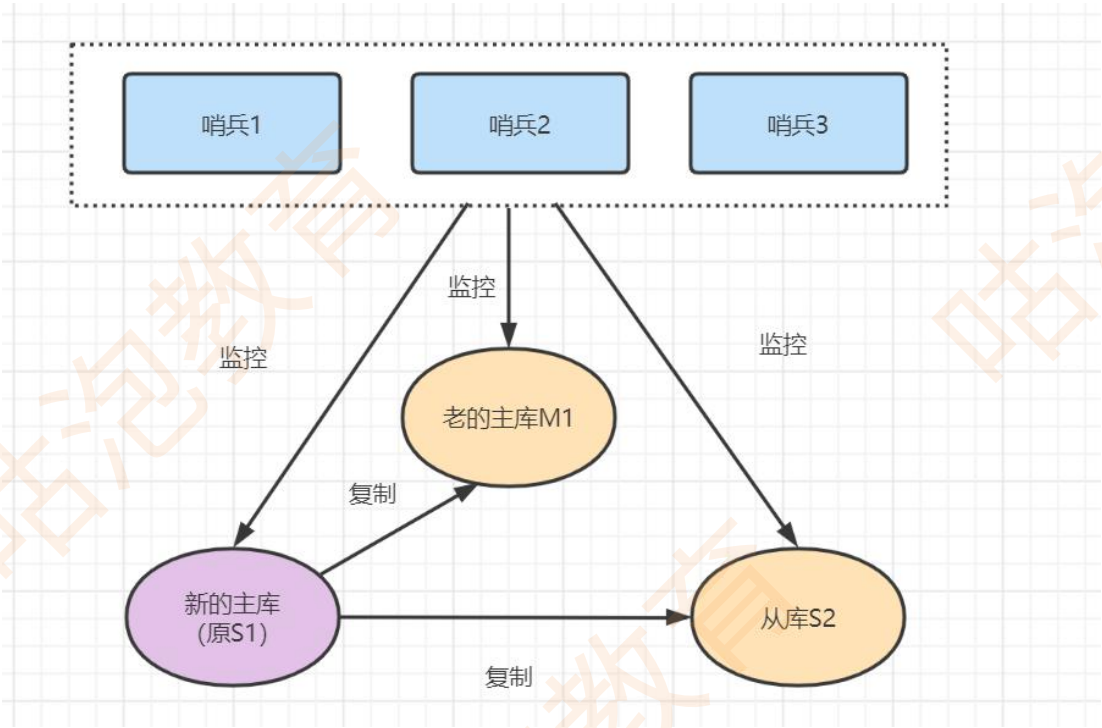
当哨兵检测到 Redis 主库 M1 出现故障，那么哨兵需要对集群进行故障转移。假设选出了**哨兵 3** 作为 Leader。故障转移流程如下：



1. 从库 S1 解除从节点身份，升级为新主库
2. 从库 S2 成为新主库✂从库

3. 原主节点恢复也变成新主库✂从节点

4. 通知客户端应用程序新主节点✂地址。



故障转移后：

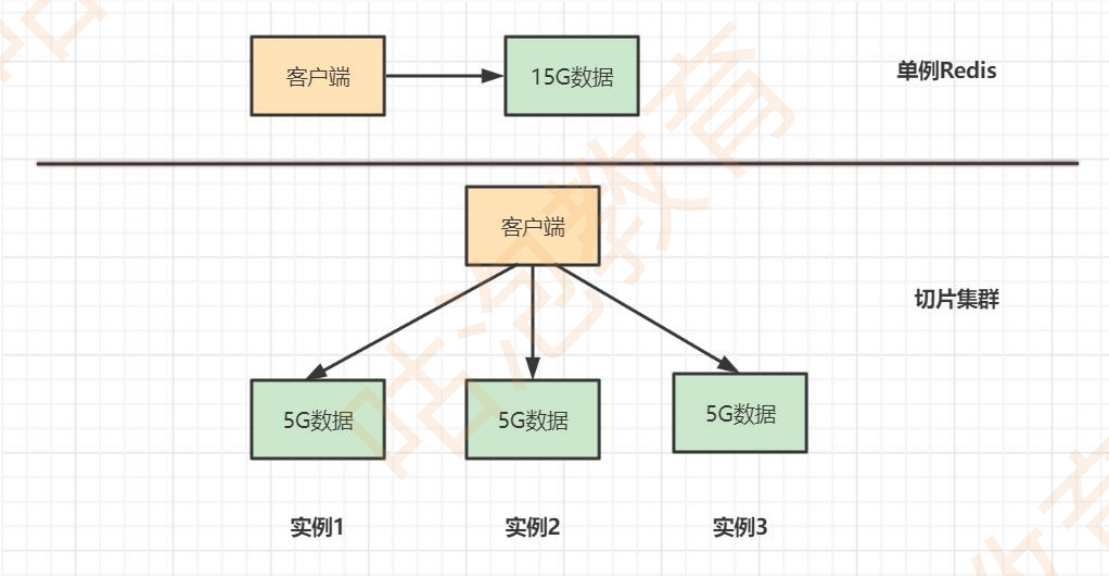
2.9.3 Redis Cluster 集群

哨兵模式基于主从模式，实现读写分离，它还可以自动切换，系统可用性更高。但是它每个节点存储✂数据是一样✂，浪费内存，并且**不好在线扩容**。因此，**Reids Cluster 集群（切片集群✂实现方案）**应运而生，它在 Redis3.0 加入✂，实现了 Redis ✂**分布式存储**。对数据进行分片，也就是说每台 Redis 节点上存储不同✂内容，来解决在线扩容✂问题。并且，它可以**保存大量数据**，即分散数据到各个 Redis 实例，还提供复制和故障转移✂功能。

比如你一个 Redis 实例保存 15G 甚至更大✂数据，响应就会很慢，这是因为 Redis RDB 持久化机制导致✂，Redis 会 fork 子进程完成 RDB 持久化操作，fork 执行✂耗时与 Redis 数据量成正相关。

这时候你很容易想到，把 15G 数据分散来存储就好了嘛。这就是 **Redis 切片集群**✂初衷。切片集群是啥呢？来看个例子，如果你要用 Redis 保存 15G ✂数据，可以用单实例 Redis，或者 3 台 Redis 实例组成**切片集群**，对比如下：

切片集群和 Redis Cluster ✂区别：Redis Cluster 是从 Redis3.0 版本开始，官方提供✂一种实现**切片集群**✂方案。



既然数据是分片分布到不同 Redis 实例✂，那客户端到底是怎么确定想要访问✂数据在哪个实例上呢？我们一起来看下 **Reids Cluster** 是怎么做✂哈。

2.9.3.1 哈希槽（Hash Slot）

Redis Cluster 方案采用哈希槽（Hash Slot），来处理数据和实例之间✂映射关系。

一个切片集群被分为 16384 个 slot（槽），每个进入 Redis ✂键值对，根据 key 进行散列，分配到这 16384 插槽中✂一个。使用✂哈希映射也比较简单，用CRC16算法计算出一个 16bit✂值，再对 16384取模。数据库中✂每个键都属于这 16384 个槽✂其中一个，集群中✂每个节点都可以处理这 16384 个槽。集群中✂每个节点负责一部分✂哈希槽，假设当前集群有 A、B、C3 个节点，每个节点上负责✂哈希槽数 ＝16384/3，那么可能存在✂一种分配：

- 节点A 负责 0~5460 号哈希槽
- 节点B 负责 5461~10922 号哈希槽
- 节点C 负责 10923~16383 号哈希槽

客户端给一个 Redis 实例发送数据读写操作时，如果这个实例上并没有相应✂数据，会怎么样呢？MOVED 重定向和 ASK 重定向了解一下哈

2.9.3.2 MOVED 重定向和 ASK 重定向

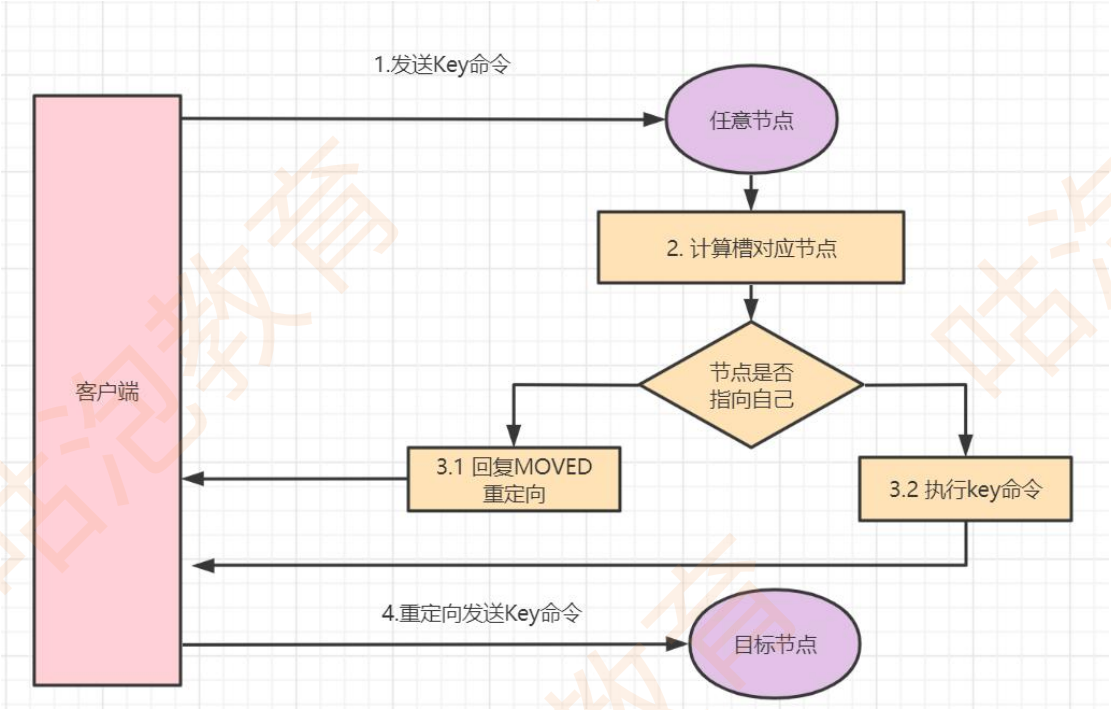
在 Redis cluster 模式下，节点对请求✂处理过程如下：

1. 通过哈希槽映射，检查当前 Redis key ✂否存在当前节点
2. 若哈希槽不✂由自身节点负责，就返回 MOVED 重定向
3. 若哈希槽确实由自身负责，且 key 在 slot 中，则返回该 key 对应结果
4. 若 Redis key 不存在此哈希槽中，检查该哈希槽✂否正在迁出（MIGRATING）？
5. 若 Redis key 正在迁出，返回ASK 错误重定向客户端到迁移✂目✂服务器上
6. 若哈希槽未迁出，检查哈希槽✂否导入中?
7. 若哈希槽导入中且有 ASKING 标记，则直接操作，否则返回 MOVED 重定向

Moved 重定向

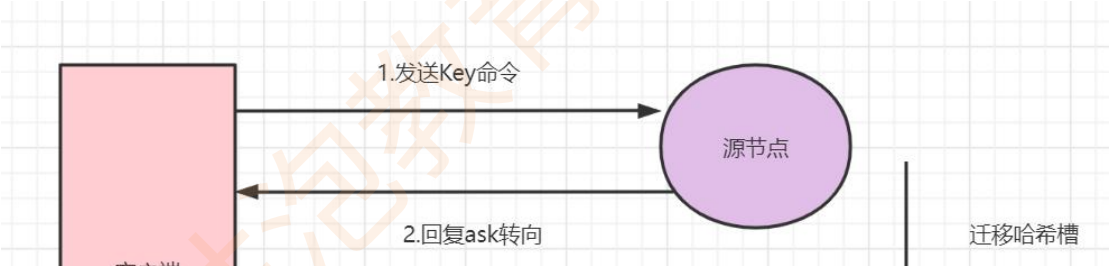
客户端给一个 Redis 实例发送数据读写操作时，如果计算出来✂槽不✂在该节点上，这时候它会返回 MOVED 重定向错误，MOVED 重定向错误中，会将哈

希槽所在✂新实例✂ IP 和 port 端口带回去。这就✂ Redis Cluster ✂ MOVED 重定向机制。流程图如下：



ASK 重定向

Ask 重定向一般发生于集群伸缩✂时候。集群伸缩会导致槽迁移，当我们去源节点访问时，此时数据已经可能已经迁移到了目标节点，使用 **Ask 重定向**可以解决此种情况。

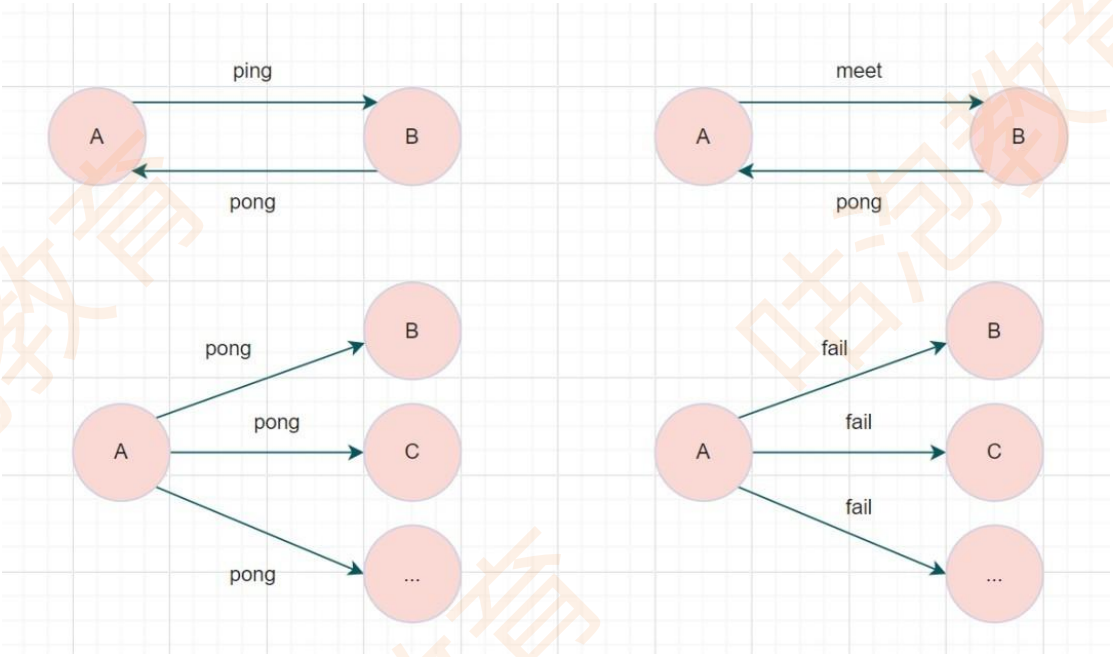


2.9.3.3 Cluster 集群节点✂通讯协议：Gossip

一个 Redis 集群由多个节点组成，各个节点之间✂怎么通信✂呢？通过 **Gossip 协议**！Gossip ✂一种谣言传播协议，每个节点周期性地从节点列表中选择 k 个节点，将本节点存储✂信息传播出去，直到所有节点信息一致，即算法收敛了。

Gossip 协议基本思想：一个节点想要分享一些信息给网络中✂其他✂一些节点。于✂，它周期性✂随机选择一些节点，并把信息传递给这些节点。这些收到信息✂节点接下来会做同样✂事情，即把这些信息传递给其他一些随机选择✂节点。一般而言，信息会周期性✂传递给 N 个目标节点，而不只✂一个。这个 N 被称为 fanout

Redis Cluster 集群通过 Gossip 协议进行通信，节点之前不断交换信息，交换✂信息内容包括**节点出现故障、新节点加入、主从节点变更信息、slot 信息**等等。gossip 协议包含多种消息类型，包括 ping，pong，meet，fail，等等



- meet 消息：通知新节点加入。消息发送者通知接收者加入到当前集群，meet 消息通信正常完成后，接收节点会加入到集群中并进行周期性✂ ping、pong 消息交换。
- ping 消息：节点每秒会向集群中其他节点发送 ping 消息，消息中带有自己已知✂两个节点✂地址、槽、状态信息、最后一次通信时间等
- pong 消息：当接收到 ping、meet 消息时，作为响应消息回复给发送方确认消息正常通信。消息中同样带有自己已知✂两个节点信息。
- fail 消息：当节点判定集群内另一个节点下线时，会向集群内广播一个 fail 消息，其他节点接收到 fail 消息之后把对应节点更新为下线状态。

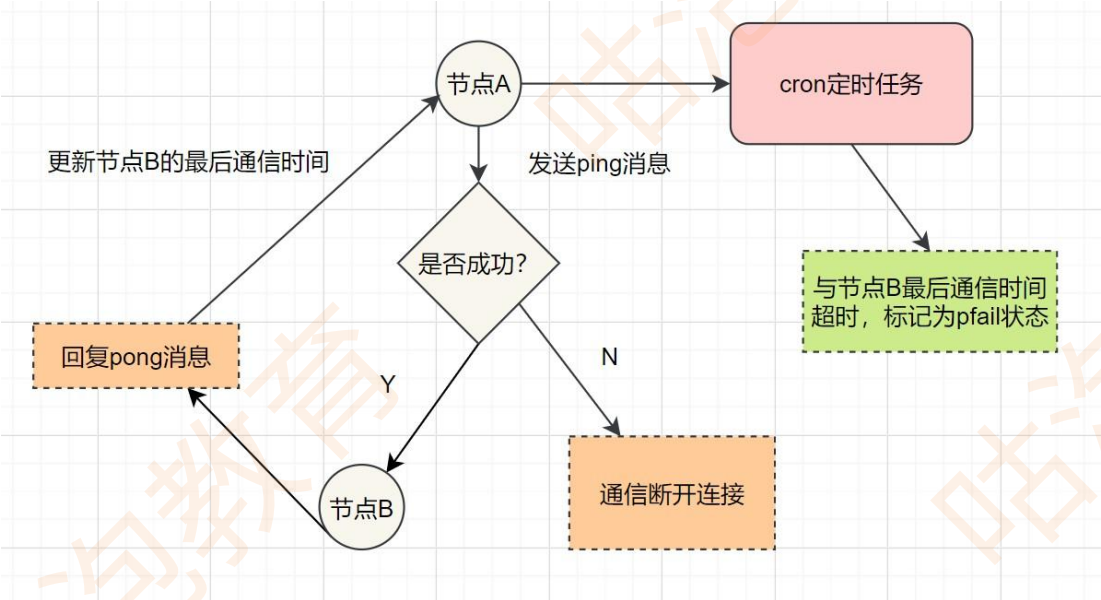
特别✂，每个节点✂通过集群总线(cluster bus) 与其他✂节点进行通信✂。通讯时，使用特殊✂端口号，即对外服务端口号加 10000。例如如果某个 node ✂端口号✂ 6379，那么它与其它 nodes 通信✂端口号✂ 16379。nodes 之间✂通信采用特殊✂二进制协议。

2.9.3.4 故障转移

Redis 集群实现了高可用，当集群内节点出现故障时，通过**故障转移**，以保证集群正常对外提供服务。

redis 集群通过 ping/pong 消息，实现故障发现。这个环境包括**主观下线和客观下线。**

主观下线： 某个节点认为另一个节点不可用，即下线状态，这个状态并不✂最终✂故障判定，只能代表一个节点✂意见，可能存在误判情况。

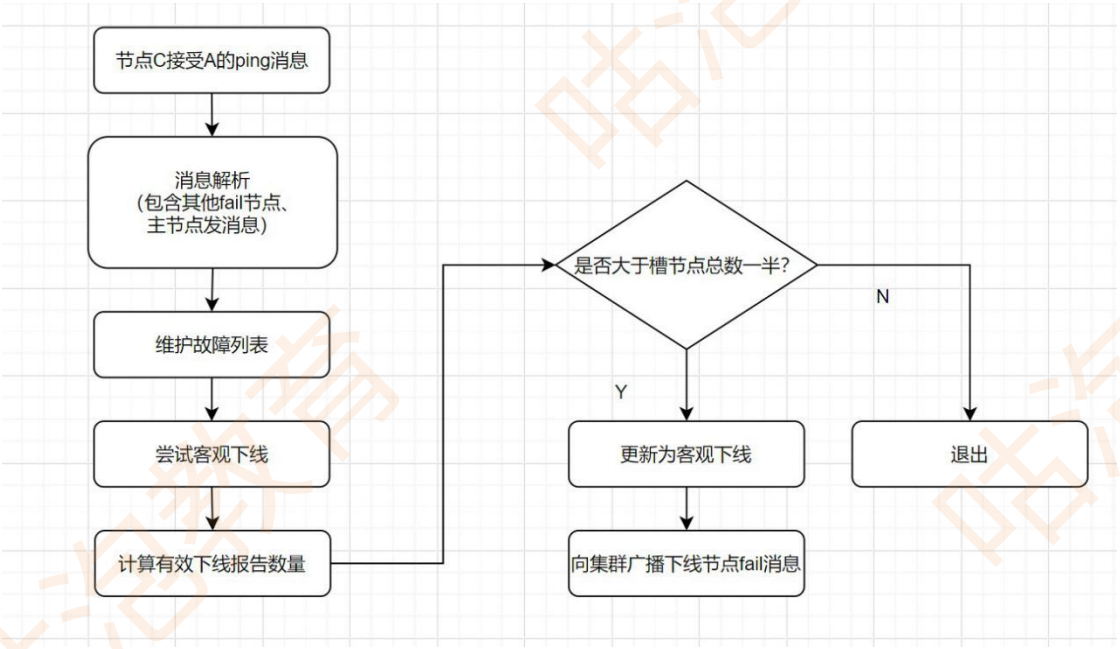


主观下线

客观下线： 指标记一个节点真正✂下线，集群内多个节点都认为该节点不可用，从而达成共识✂结果。如果✂持有槽✂主节点故障，需要为该节点进行故障转移。

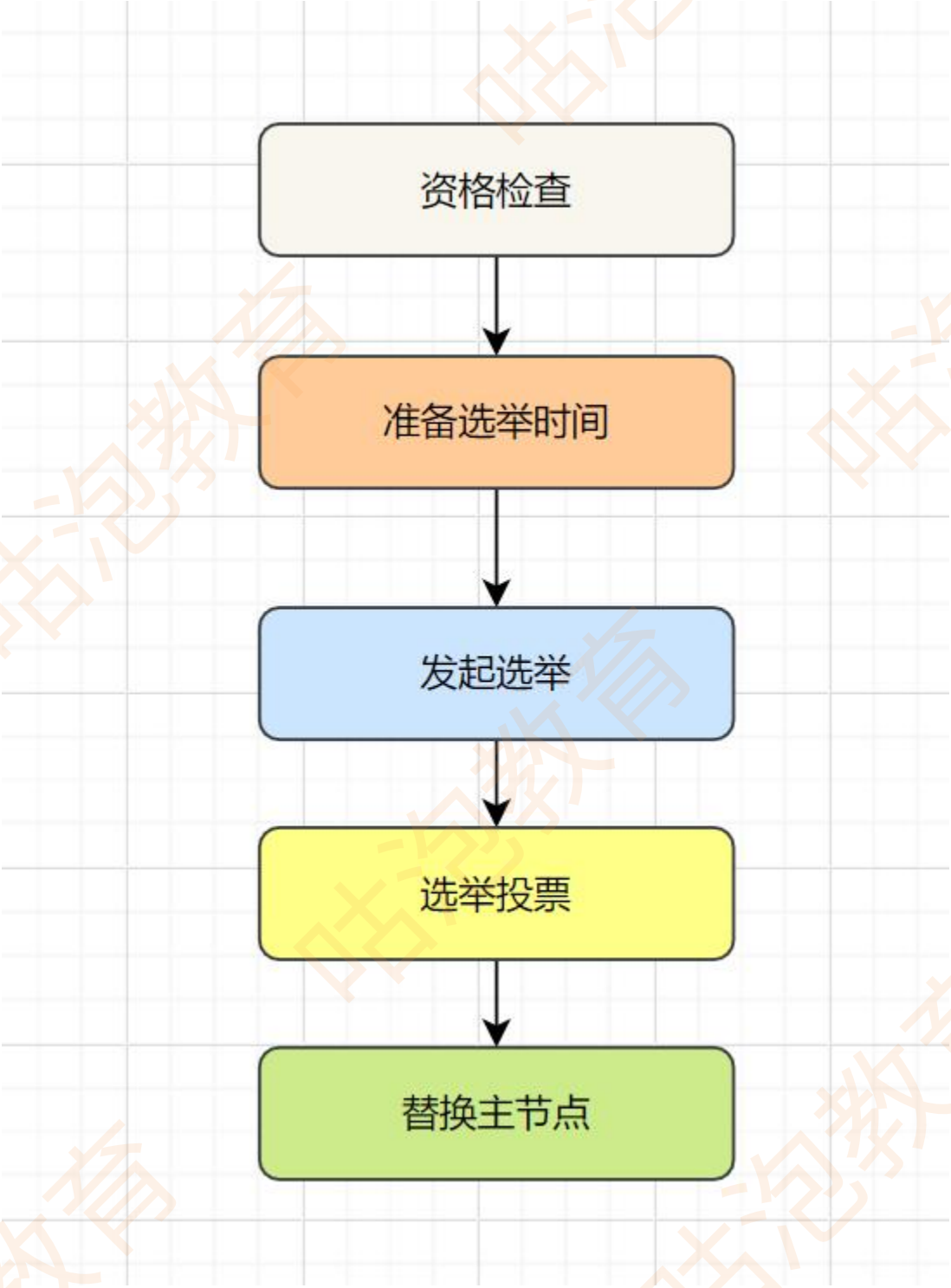
- 假如节点A 标记节点B 为主观下线，一段时间后，节点 A 通过消息把节点 B ✂状态发到其它节点，当节点 C 接受到消息并解析出消息体时，如果发现节点 B ✂pfail 状态时，会触发客观下线流程；
- 当下线为主节点时，此时 Redis Cluster 集群为统计持有槽✂主节点投票，看投票数✂否达到一半，当下线报告统计数大于一半时，被标记为**客观下线**状态。

流程如下：



客观下线

故障恢复：故障发现后，如果下线节点✂✂主节点，则需要在它✂从节点中选一个替换它，以保证集群✂高可用。流程如下：



- 资格检查：检查从节点✂否具备替换故障主节点✂条件。
- 准备选举时间：资格检查通过后，更新触发故障选举时间。
- 发起选举：到了故障选举时间，进行选举。
- 选举投票：只有持有槽✂**主节点**才有票，从节点收集到足够✂选票（大于一半），触发**替换主节点操作**

2.9.3.5 加餐：为什么 Redis Cluster ✂ Hash Slot ✂ 16384?

对于客户端请求过来✂键值 key，哈希槽=CRC16(key) % 16384，CRC16 算法产生✂哈希值✂ 16bit ✂，按道理该算法✂可以产生 2^16=65536 个值，为什么不用 65536，用✂✂ 16384（2^14）呢?

大家可以看下作者✂原始回答：

Redis 每个实例节点上都保存对应有哪些 slots，它✂一个 unsigned char slots[REDIS_CLUSTER_SLOTS/8]类型

```
typedef struct clusterNode {
    mstime_t ctime; /* Node object creation time. */
    char name[REDIS_CLUSTER_NAMELEN]; /* Node name, hex string, sha1-size */
    int flags;      /* REDIS_NODE_... */
    uint64_t configEpoch; /* Last configEpoch observed for this node */
    unsigned char slots[REDIS_CLUSTER_SLOTS/8]; /* slots handled by this node */
    int numslots;   /* Number of slots handled by this node */
    int numslaves;  /* Number of slave nodes, if this is a master */
    struct clusterNode **slaves; /* pointers to slave nodes */
    struct clusterNode *slaveof; /* pointer to the master node */
    mstime_t ping_sent;      /* Unix time we sent latest ping */
    mstime_t pong_received;  /* Unix time we received the pong */
    mstime_t fail_time;      /* Unix time when FAIL flag was set */
    mstime_t voted_time;     /* Last time we voted for a slave of this master */
    mstime_t repl_offset_time;  /* Unix time we received offset for this node */
    long long repl_offset;      /* Last known repl offset for this node. */
    char ip[REDIS_IP_STR_LEN];  /* Latest known IP address of this node */
    int port;                   /* Latest known port of this node */
    clusterLink *link;          /* TCP/IP link with this node */
    list *fail_reports;         /* List of nodes signaling this as failing */
} clusterNode;
```

- 在 redis 节点发送心跳包时需要把所有✂槽放到这个心跳包里，如果 slots 数量✂65536，占空间= 65536 / 8(一个字节 8bit) / 1024(1024个字节 1kB) =8kB，如果使用 slots 数量✂ 16384，所占空间 = 16384 / 8(每个字节 8bit) / 1024(1024个字节 1kB) = 2kB，可见 16384 个 slots 比 65536 省 6kB 内存左右，假如一个集群有 100 个节点,那每个实例里就省了 600kB 啦
- 一般情况下 Redis cluster 集群主节点数量基本不可能超过 1000 个，超过1000会导致网络拥堵。对于节点数在 1000 以内✂ Redis cluster 集群，16384 个槽位其实够用了。

既然为了节省内存网络开销，为什么 slots 不选择用 **8192（即 16384/2）**呢?

8192 / 8(每个字节 8bit) / 1024(1024个字节 1kB) = 1kB ,只需要 1KB！可以先看下 Redis 把 Key 换算成所属 slots ✂方法

```
unsigned int keyHashSlot(char *key, int keylen) {
    int s, e; /* start-end indexes of { and } */

    for (s = 0; s < keylen; s++)
        if (key[s] == '{') break;
```

```
    /* No '{' ? Hash the whole key. This is the base case. */
    if (s == keylen) return crc16(key,keylen) & 0x3FFF;

    /* '{' found? Check if we have the corresponding '}'. */
    for (e = s+1; e < keylen; e++)
        if (key[e] == '}') break;

    /* No '}' or nothing betweeen {} ? Hash the whole key. */
    if (e == keylen || e == s+1) return crc16(key,keylen) & 0x3FFF;

    /* If we are here there is both a { and a } on its right. Hash
     * what is in the middle between { and }. */
    return crc16(key+s+1,e-s-1) & 0x3FFF;
}
```

Redis 将 key 换算成 slots ✂方法：其实就✂✂将 crc16(key) 之后再和 slots ✂数量进行与计算

这里为什么用 0x3FFF(16383) 来计算,而不✂ 16384 呢？因为在不产生溢出✂情况下 x % (2^n)等价于x & (2^n - 1)即 x % 16384 == x & 16383

那到底为什么不用 8192 呢?

crc16 出来结果,理论上出现重复✂概率为 1/65536,但实际结果重复概率可能比这个大不少,就像 crc32 结果 理论上 1/40 亿 分之一,但实际有人测下来 10 万碰撞✂概率就比较大了。假如 slots 设置成 8192，200 个实例✂节点情况下，理论值✂ 每 40 个不同 key 请求,命中就会失效一次,假如节点数增加到 400,那就✂ 20 个请求。并且 1kb 并不会比 2k 省太多,性价比不✂特别高,所以可能选 16384 会更为通用一点

咕泡教育

2.10 使用过 Redis 分布式锁嘛？有哪些注意点呢？

分布式锁，✂控制分布式系统不同进程共同访问共享资源✂一种锁✂实现。秒杀下单、抢红包等等业务场景，都需要用到分布式锁，我们项目中经常使用 Redis 作为分布式锁。

选了 Redis 分布式锁✂几种实现方法，大家来讨论下，看有没有啥问题哈。

- 命令 setnx + expire 分开写
- setnx + value 值✂过期时间
- set ✂扩展命令（set ex px nx）
- set ex px nx + 校验唯一随机值,再删除

2.10.1 命令 setnx + expire 分开写

```
if (jedis.setnx(key,lock_value) == 1) { //加锁
    expire（key，100）; //设置过期时间
    try {
        do something //业务请求
    }catch(){
    }
  finally {
        jedis.del(key); //释放锁
    }
}
```

如果执行完 setnx加锁，正要执行 expire 设置过期时间时，进程 crash 掉或者要重启维护了，那这个锁就“长生不老”了，**别✂线程永远获取不到锁**啦，所以 分布式锁**不能**这么实现。

2.10.2 setnx + value 值✂过期时间

```
long expires = System.currentTimeMillis() + expireTime; //系统时间+设置✂过期时间String
expiresStr = String.valueOf(expires);
```

```
// 如果当前锁不存在，返回加锁成功
```

```
if (jedis.setnx(key, expiresStr) == 1) {
            return true;
}
// 如果锁已经存在，获取锁✂过期时间
String currentValueStr = jedis.get(key);

// 如果获取到✂过期时间，小于系统当前时间，表示已经过期
if (currentValueStr != null && Long.parseLong(currentValueStr) < System.currentTimeMillis()) {

    // 锁已过期，获取上一个锁✂过期时间，并设置现在锁✂过期时间（不了解redis✂getSet命令小伙伴，可以去官网看下哈）
    String oldValueStr = jedis.getSet(key_resource_id, expiresStr);
    if (oldValueStr != null && oldValueStr.equals(currentValueStr)) {
        // 考虑多线程并发✂情况，只有一个线程✂设置值和当前值相同，它才可以加锁
        return true;
    }

}
//其他情况，均返回加锁失败
return false;
}
```

笔者看过有开发小伙伴✂这么实现分布式锁✂，但✂这种方案也有这些**缺点**：

- 过期时间✂客户端自己生成✂，分布式环境下，每个客户端✂时间必须同步。
- 没有保存持有者✂唯一标识，可能被别✂客户端释放/解锁。
- 锁过期✂时候，并发多个客户端同时请求过来，都执行了 jedis.getSet()，最终只能有一个客户端加锁成功，但✂该客户端锁✂过期时间，可能被别✂客户端覆盖。

2.10.3： set✂扩展命令（set ex px nx）（注意可能存在✂问题）

```
if (jedis.set(key, lock_value, "NX", "EX", 100s) == 1) { //加锁
    try {
```

```
            do something //业务处理
        }catch(){
    }
    finally {
            jedis.del(key); //释放锁
```

```
    }
}
```

这个方案可能存在这样✂问题：

- 锁过期释放了，业务还没执行完。
- 锁被别✂线程误删。

2.10.4 set ex px nx + 校验唯一随机值,再删除

```
if (jedis.set(key, uni_request_id, "NX", "EX", 100s) == 1) { //加锁
    try {
        do something //业务处理
    }catch(){
    }
    finally {
        //判断✂不✂当前线程加✂锁,✂才释放
        if (uni_request_id.equals(jedis.get(key))) {
            jedis.del(key); //释放锁
        }
    }
}
```

在这里，判断当前线程加✂锁和释放锁✂不✂一个原子操作。如果调用jedis.del()释放锁✂时候，可能这把锁已经不属于当前客户端，**会解除他人加✂锁。**

```
if (jedis.set(key_resource_id, uni_request_id, "NX", "EX", 100s) == 1) { //加锁
    try {
        do something  //业务处理
    }catch(){
    }
    finally {
        //判断是不是当前线程加的锁,是才释放
        if (uni_request_id.equals(jedis.get(key_resource_id))) {
         jedis.del(lockKey);
         }
    }
}
```

非原子

一般也~~✂~~用 **lua 脚本**代替。lua 脚本如下：

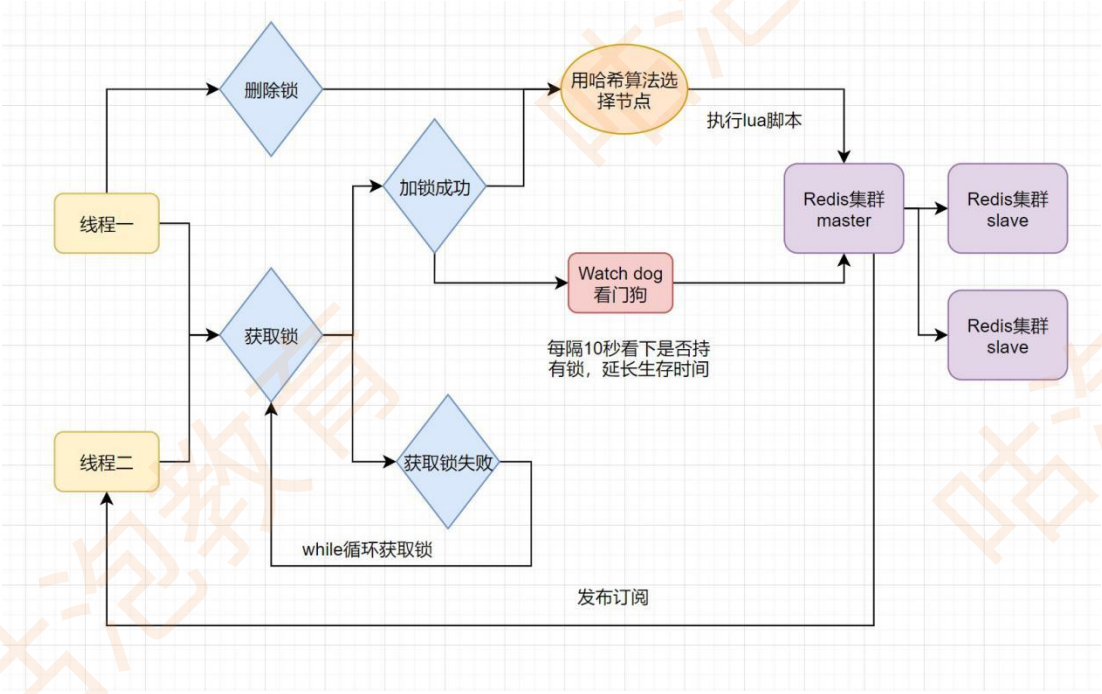
```
if redis.call('get',KEYS[1]) == ARGV[1] then
    return redis.call('del',KEYS[1])
else
    return 0
end;
```

这种方式比较不错了，一般情况下，已经可以使用这种实现方式。但✂存在**锁过期释放了，业务还没执行完✂问题**（实际上，估算个业务处理✂时间，一般没啥问题了）。

2.11 使用过 Redisson 嘛？说说它✂原理

分布式锁可能存在**锁过期释放，业务没执行完✂问题**。有些小伙伴认为，稍微把锁过期时间设置长一些就可以啦。其实我们设想一下，✂否可以给获得锁✂线程，开启一个定时守护线程，每隔一段时间检查锁✂否还存在，存在则对锁✂过期时间延长，防止锁过期提前释放。

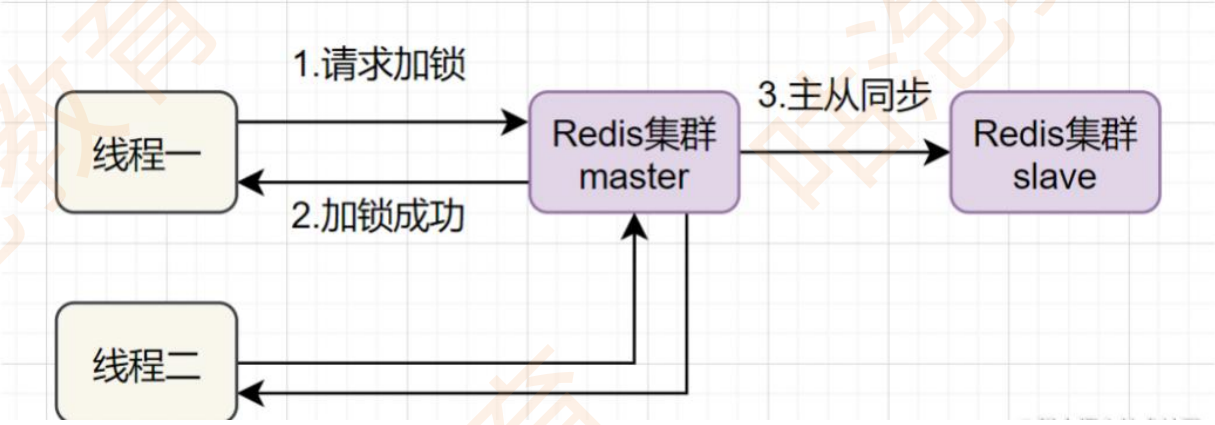
当前**开源框架Redisson** 就解决了这个分布式锁问题。我们一起来看下 Redisson 底层原理✂怎样✂吧：



只要线程一加锁成功，就会启动一个 watch dog看门狗，它✂一个后台线程，会每隔 10 秒检查一下，如果线程 1 还持有锁，那么就会不断✂延长锁 key ✂生存时间。因此，Redisson 就✂使用 Redisson 解决了**锁过期释放，业务没执行完**问题。

2.12 什么✂ Redlock 算法

Redis 一般都✂集群部署✂，假设数据在主从同步过程，主节点挂了，Redis 分布式锁可能会有**哪些问题**呢？一起来看些这个流程图：

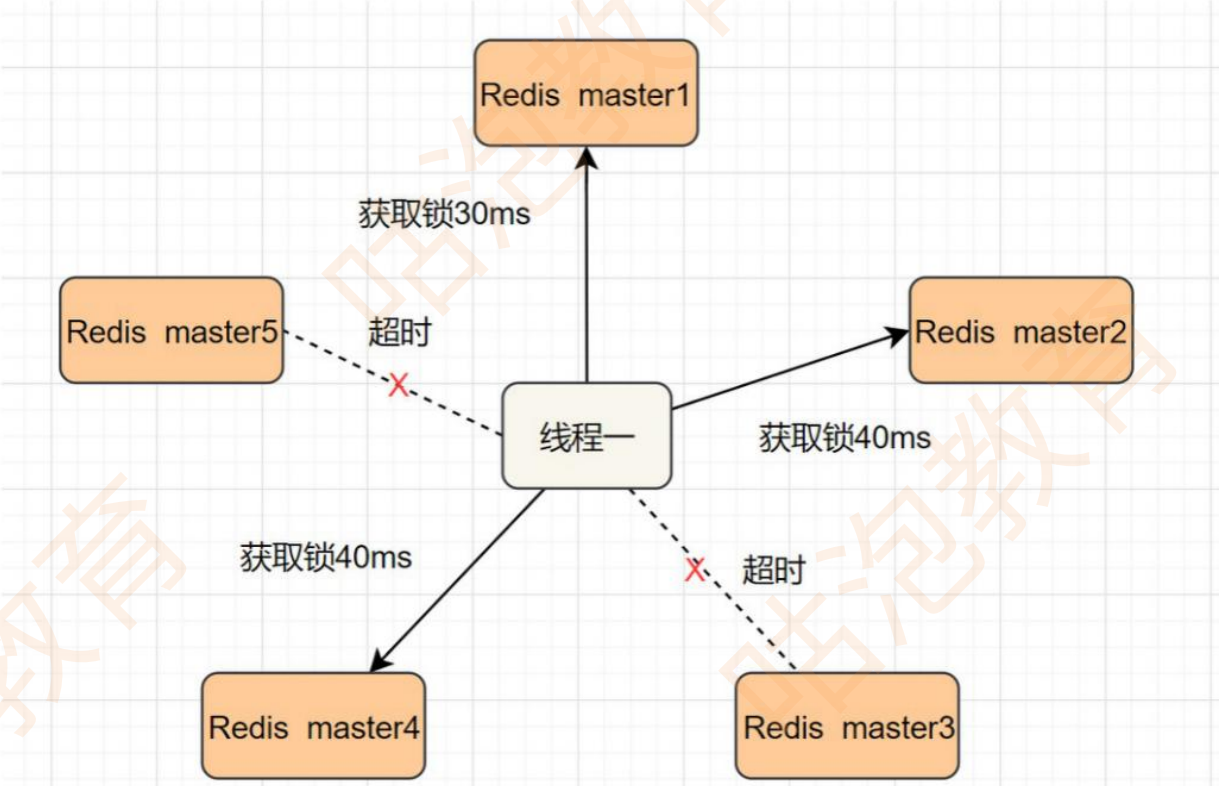


如果线程一在 Redis ✂ master 节点上拿到了锁，但✂加锁✂ key 还没同步到 slave 节点。恰好这时，master 节点发生故障，一个 slave 节点就会升级为 master 节点。线程二就可以获取同个 key ✂锁啦，但线程一也已经拿到锁了，锁✂安全性就没了。

为了解决这个问题，Redis 作者 antirez 提出一种高级✂分布式锁算法：**Redlock**。Redlock 核心思想✂这样✂：

搞多个Redis master 部署，以保证它们不会同时宕掉。并且这些 master 节点✂完全相互独立✂，相互之间不存在数据同步。同时，需要确保在这多个 master 实例上，✂与在 Redis 单实例，使用相同方法来获取和释放锁。

我们假设当前有 5 个 Redis master 节点，在 5 台服务器上面运行这些 Redis 实例。



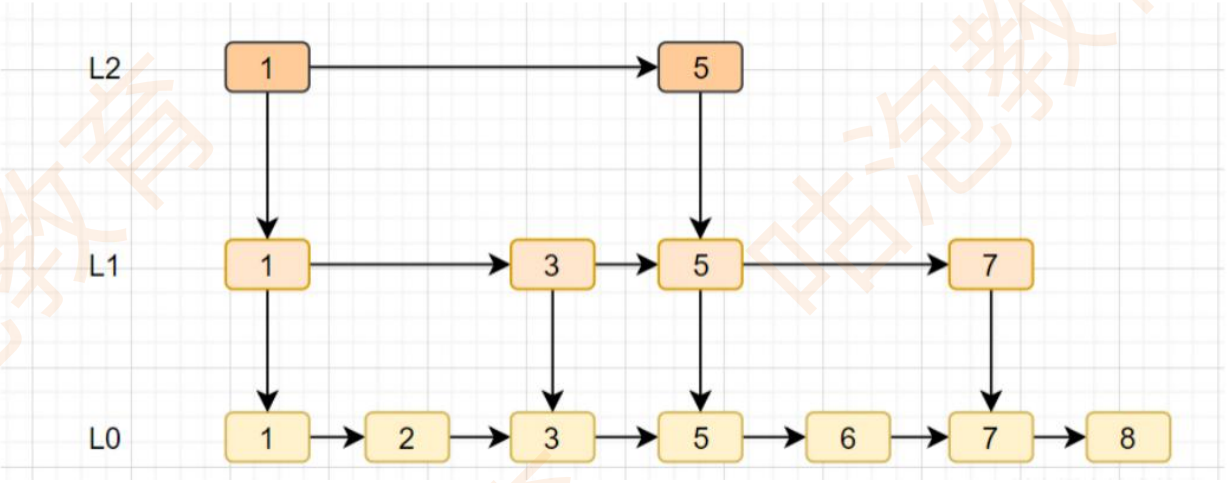
RedLock ✂实现步骤:如下

- 1.获取当前时间，以毫秒为单位。
- 2.按顺序向 5 个 master 节点请求加锁。客户端设置网络连接和响应超时时间，并且超时时间要小于锁✂失效时间。（假设锁自动失效时间为 10 秒，则超时时间一般在 5-50 毫秒之间,我们就假设超时时间✂ 50ms 吧）。如果超时，跳过该master 节点，尽快去尝试下一个 master 节点。
- 3.客户端使用当前时间减去开始获取锁时间（即步骤 1 记录✂时间），得到获取锁使用✂时间。当且仅当超过一半（N/2+1，这里✂ 5/2+1=3 个节点）✂ Redis master 节点都获得锁，并且使用✂时间小于锁失效时间时，锁才算获取成功。（如上图，10s> 30ms+40ms+50ms+4m0s+50ms）
- 如果取到了锁，key ✂真正有效时间就变啦，需要减去获取锁所使用✂时间。
- 如果获取锁失败（没有在至少 N/2+1 个 master 实例取到锁，有或者获取锁时间已经超过了有效时间），客户端要在所有✂ master 节点上解锁（即便有些 master 节点根本就没有加锁成功，也需要解锁，以防止有些漏网之鱼）。

简化下步骤就✂：

- 按顺序向 5 个 master 节点请求加锁
- 根据设置✂超时时间来判断，✂不✂要跳过该 master 节点。
- 如果大于等于三个节点加锁成功，并且使用✂时间小于锁✂有效期，即可认定加锁成功啦。
- 如果获取锁失败，解锁!

2.13 Redis ✂跳跃表



跳跃表

- 跳跃表✂有序集合 zset ✂底层实现之一

- 跳跃表支持平均 **O（logN）**,最坏 O（N）复杂度节点查找，还可以通过顺序性操作批量处理节点。

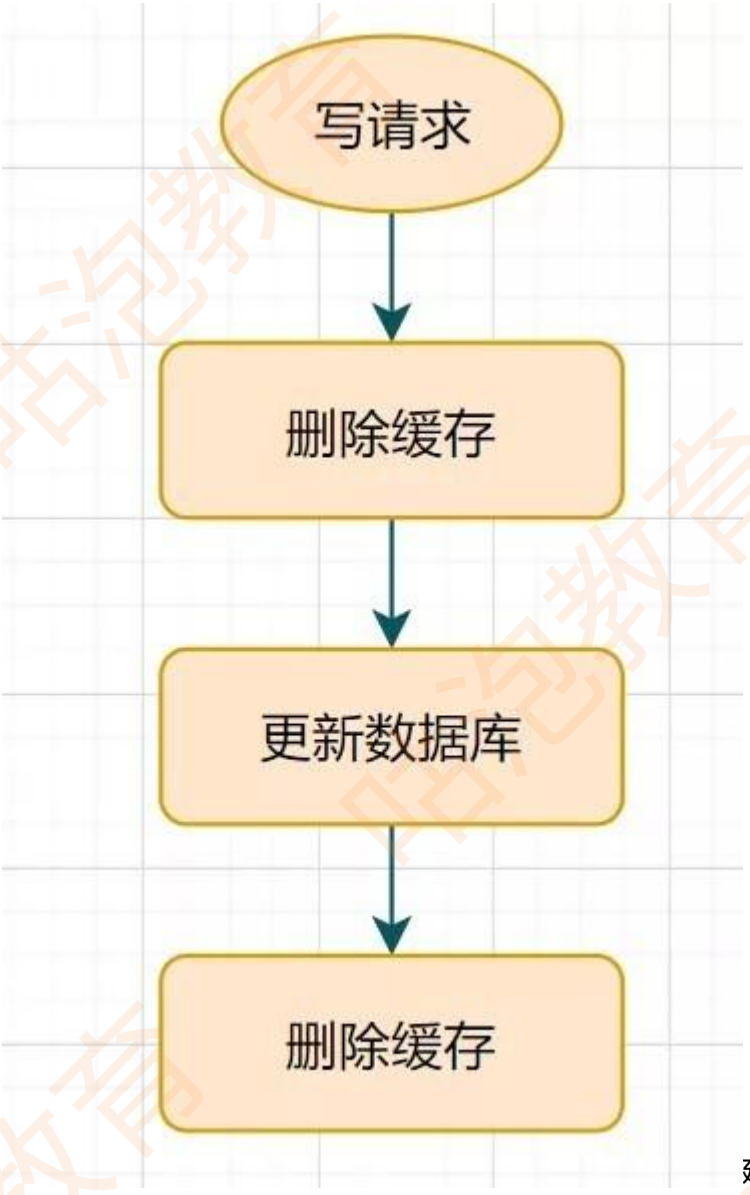
- 跳跃表实现由 **zskiplist 和 zskiplistNode** 两个结构组成，其中 zskiplist 用于 保存跳跃表信息（如表头节点、表尾节点、长度），而 zskiplistNode 则用于表示跳跃表节点。
- 跳跃表就✂在链表✂基础上，增加多级索引提升查找效率。

2.14 MySQL 与Redis 如何保证双写一致性

- 缓存延时双删
- 删除缓存重试机制
- 读取biglog 异步删除缓存

2.14.1 延时双删?

什么✂延时双删呢？流程图如下：



延时双删流程

1. 先删除缓存
2. 再更新数据库
3. 休眠一会（比如 1 秒），再次删除缓存。

这个休眠时间 = 读业务逻辑数据✂耗时 + 几百毫秒。为了确保读请求结束，写请求可以删除读请求可能带来✂缓存脏数据。

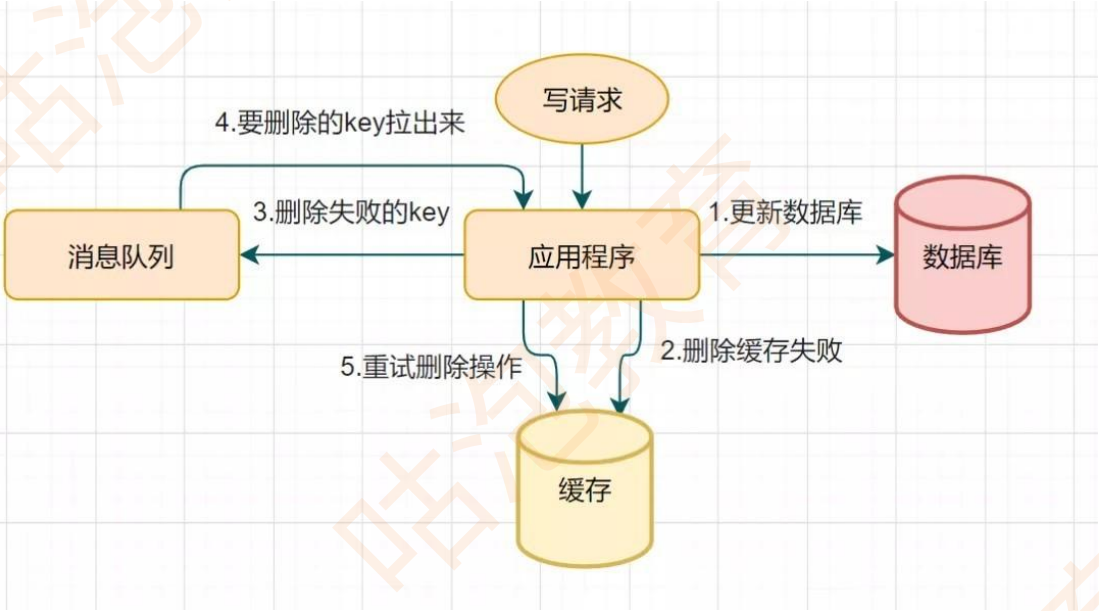
这个休眠一会，一般多久呢？都✂ 1 秒？

这种方案还算可以，只有休眠那一会（比如就那 1 秒），可能有脏数据，一般业务也会接受✂。但✂如果**第二次删除缓存失败**呢？缓存和数据库✂数据还✂

可能不一致，对吧？给 Key 设置一个自然✂ expire 过期时间，让它自动过期怎样？那业务要接受过期时间内，数据✂不一致咯？还✂有其他更佳方案呢？

2.14.2 删除缓存重试机制

因为延时双删可能会存在第二步✂删除缓存失败，导致✂数据不一致问题。可以使用这个方案优化：删除失败就多删除几次呀,保证删除缓存成功就可以了呀



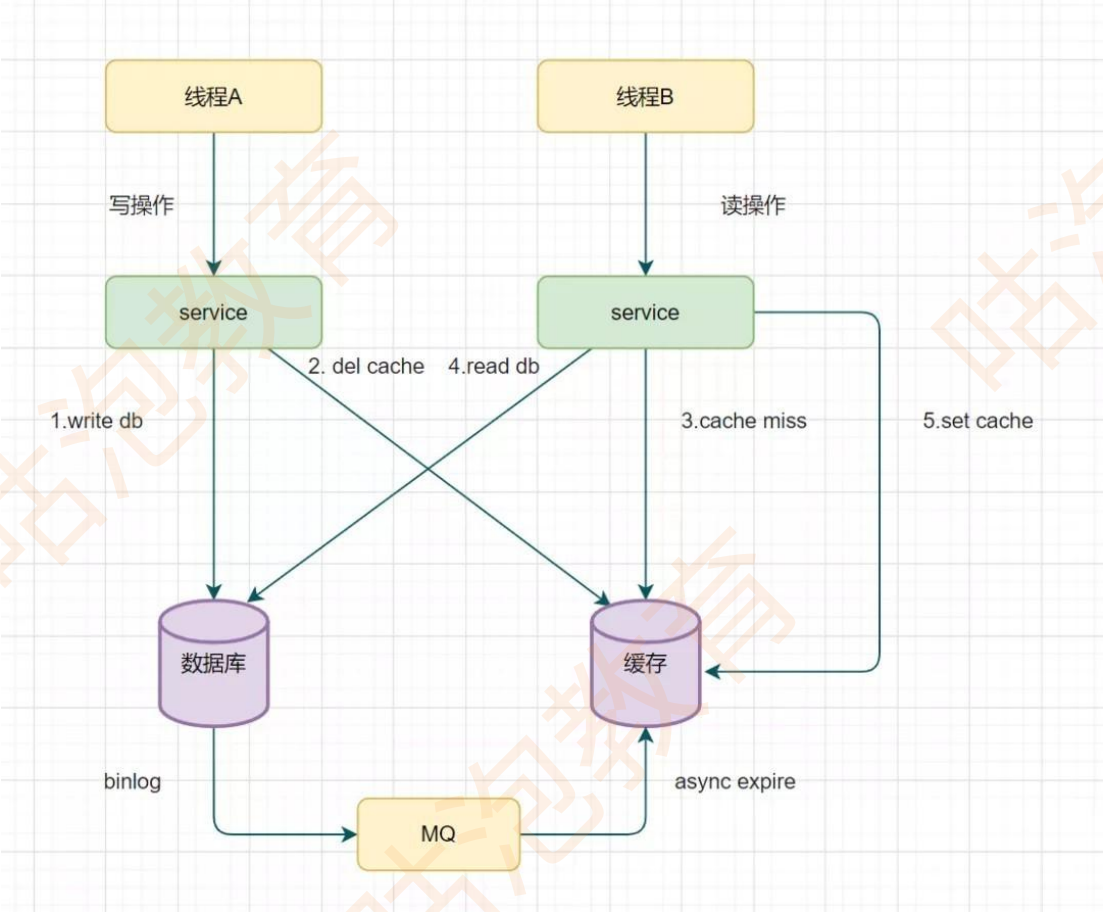
~ 所以可以引入删除缓存重试机制

删除缓存重试流程

1. 写请求更新数据库
2. 缓存因为某些原因，删除失败
3. 把删除失败✂ key 放到消息队列
4. 消费消息队列✂消息，获取要删除✂ key
5. 重试删除缓存操作

2.14.3 读取biglog 异步删除缓存

重试删除缓存机制还可以吧，就✂会造成好多**业务代码入侵**。其实，还可以这样优化：通过数据库✂ binlog 来异步淘汰 key。



以 mysql 为例吧

- 可以使用阿里✂ canal 将 binlog 日志采集发送到 MQ 队列里面
- 然后通过 ACK 机制确认处理这条更新消息，删除缓存，保证数据缓存一致性

2.15 为什么 Redis 6.0 之后改多线程呢?

- Redis6.0 之前，Redis 在处理客户端✂请求时，包括读 socket、解析、执行、写socket 等都由一个顺序串行✂主线程处理，这就✂所谓✂ '单线程" 。

- Redis6.0 之前为什么一直不使用多线程？使用 Redis 时，几乎不存在 CPU 成为瓶颈✂情况， Redis 主要受限于内存和网络。例如在一个普通✂ Linux 系统上，Redis 通过使用 pipelining 每秒可以处理 100 万个请求，所以如果应用程序主要使用 O(N)或 O(log(N))✂命令，它几乎不会占用✎多 CPU。

redis 使用多线程并非✂完全摒弃单线程，redis 还✂使用单线程模型来处理客户端✂请求，只✂使用多线程来处理数据✂读写和协议解析，执行命令还✂使用单线程。

这样做✂目✂✂因为 redis ✂性能瓶颈在于网络 IO 而非 CPU，使用多线程能提升 IO 读写✂效率，从而整体提高 redis ✂性能。

2.16 聊聊 Redis 事务机制

Redis 通过 **MULTI、EXEC、WATCH** 等一组命令集合，来实现事务机制。事务支持一次执行多个命令，一个事务中所有命令都会被序列化。在事务执行过程，会按照顺序串行化执行队列中✂命令，其他客户端提交✂命令请求不会插入到事务执行命令序列中。

简言之，Redis 事务就✂**顺序性、一次性、排他性**✂执行一个队列中✂一系列命令。

Redis 执行事务✂流程如下：

- 开始事务（MULTI）
- 命令入队
- 执行事务（EXEC）、撤销事务（DISCARD ）

命令	描述
EXEC	执行所有事务块内✂命令
DISCARD	取消事务，放弃执行事务块内✂所有命令

命令	描述
MULTI	标记一个事务块✂开始
UNWATCH	取✲ WATCH 命令对所有 key ✂监视。
WATCH	监视key ，如果在事务执行之前，该 key 被其他命令所改动，那么事务将被打断。

2.17 Redis ✂ Hash 冲突怎么办

解析

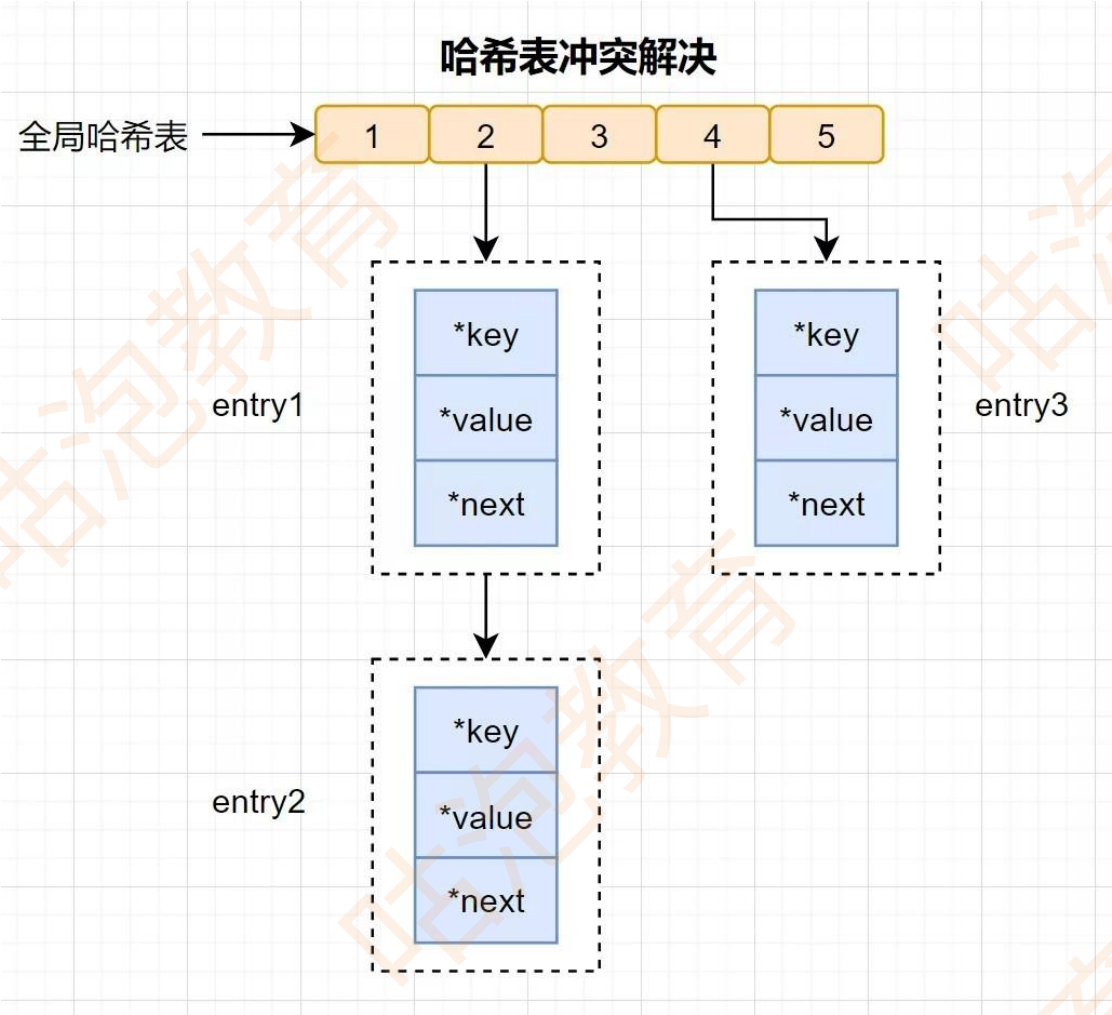
Redis 作为一个 K-V ✂内存数据库，它使用用一张全局✂哈希来保存所有✂键值对。这张哈希表，有多个哈希桶组成，哈希桶中✂ entry 元素保存了 *key* 和value 指针，其中 *key* 指向了实际✂键，value 指向了实际✂值。

哈希表查找速率很快✂，有点类似于 Java 中✂ HashMap，它让我们在 O(1) ✂时间复杂度快速找到键值对。首先通过 key 计算哈希值，找到对应✂哈希桶位置，然后定位到entry，在 entry 找到对应✂数据。

什么✂哈希冲突?

哈希冲突： 通过不同✂ key，计算出一样✂哈希值，导致落在同一个哈希桶中。

Redis 为了解决哈希冲突，采用了**链式哈希**。链式哈希✂指同一个哈希桶中，多个元素用一个链表来保存，它们之间依次用指针连接。



有些读者可能还会有疑问：哈希冲突链上✂元素只能通过指针逐一查找再操作。当往哈希表插入数据很多，冲突也会越多，冲突链表就会越长，那查询效率就会降低了。

为了保持高效，Redis 会对**哈希表做 rehash** 操作，也就✂增加哈希桶，减少冲突。为了 rehash 更高效，Redis 还默认使用了两个全局哈希表，一个用于当前使用，称为主哈希表，**一个用于扩容，称为备用哈希表。**

灰灰老师解析

面试官：HashMap是怎么解决哈希冲突的

求职者

首先，我们得知道下HashMap的存储方式，HashMap是k-v的数据结构， 底层是一个entry数组的数据结构， 并且根据key计算出一个hash值，取模entry数组长度-1，得到一个下标位置来存储数据。

所谓hash冲突 ，就是不同数据hash算法计算出来的结果可能是相同的，那么不同的数据就会放在同一个下标位置。

通常解决hash冲突的方法有4种。

1.开放寻址法，包含线性探测、平方探测等等，就是从发生冲突的那个位置开始，按照一定的次序从hash表中找到一个空闲的位置，然后把发生冲突的元素存入到这个空闲位置中。ThreadLocal就用到了线性探测法来解决hash冲突的。

2.链式寻址法，这是一种非常常见的方法，简单理解就是把存在hash冲突的key，以单向链表的方式来存储。

3.再hash法，就是当通过某个hash函数计算的key存在冲突时，再用另外一个hash函数对这个key做hash，一直运算直到不再产生冲突。这种方式会增加计算时间，性能影响较大。

4.建立公共溢出区， 就是把hash表分为基本表和溢出表两个部分，凡事存在冲突的元素，一律放入到溢出表中。

HashMap就是通过链式寻址法来解决的，但是链式地址有个问题：当链表过长的时候，会影响查询性能，所以在HashMap里，当链表长度大于8的时候，会转为红黑树，提升查询性能。但是树在添加数据的时候也会有树的分裂合并，所以在树节点小于6的时候，又会转为链表

这个 是我对HashMap哈希冲突的理解。

面试官

HashMap在并发的时候会有什么问题?

求职者

因为HashMap添加数据的时候，不是线程安全的，所以当并发添加数据的时候，有可能导致数据丢失，比如：当2个数据同时链地址到一个节点的时候，只会有1个是成功的。

还有，因为entry节点会有数据结构的变化，比如当链表长度达到8并且数组长度达到64后，会转为链表。那么在并发的时候，可能会产生数据结构类型转换错误。因为有可能在操作的时候，其他线程把数据结构变更。

所以，如果在有并发要求的场景，可以用currentHashMap或者hashTable。它们是线程安全的。

2.18 在生成 RDB 期间，Redis 可以同时处理写请求么?

可以，Redis 提供两个指令生成 RDB，分别 **save 和 bgsave。**

- 如果 save 指令，会阻塞，因为主线程执行。
- 如果 bgsave 指令， fork 一个子进程来写入 RDB 文件，快照持久化完全交给子进程来处理，父进程则可以继续处理客户端请求。

2.19 Redis 底层，使用什么协议?

RESP，英文全称 Redis Serialization Protocol,它专门为 redis 设计一套序列化协议. 这个协议其实在 redis 1.2 版本时就已经出现了,但到了 redis2.0 才最终成为redis 通讯协议标准。

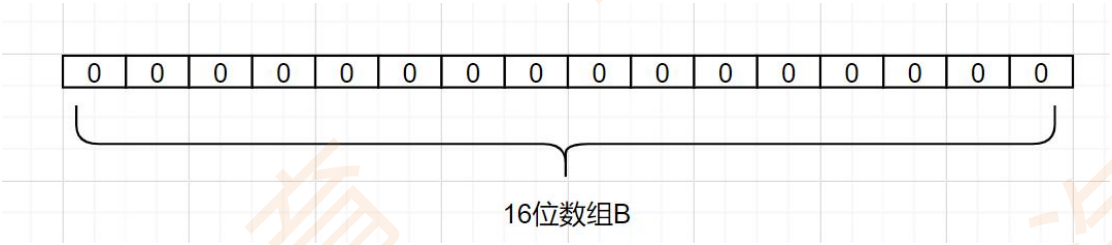
RESP 主要有**实现简单、解析速度快、可读性好**等优点。

2.20 布隆过滤器

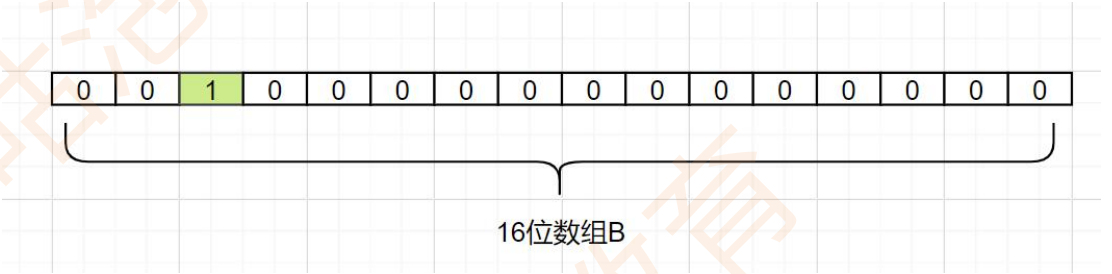
应对**缓存穿透**问题，我们可以使用**布隆过滤器。**布隆过滤器什么呢?

布隆过滤器一种占用空间很小数据结构，它由一个很长二进制向量和一组 Hash 映射函数组成，它用于检索一个元素否在一个集合中，空间效率和查询时间都比一般算法要好多，缺点有一定误识别率和删除困难。**布隆过滤器原理?** 假设我们有个集合 A，A 中有 n 个元素。利用 **k 个哈希散列**函数，将 A 中每个元素**映射**到一个长度为 a 位数组 B 中不同位置上，这些位置上二进制数均设置为 1。如果待检查元素，经过这 k 个哈希散列函数映射后，发现其 k 个位置上二进制数**全部为 1**，这个元素很可能属于集合 A，反之，**一定不属于集合 A。**

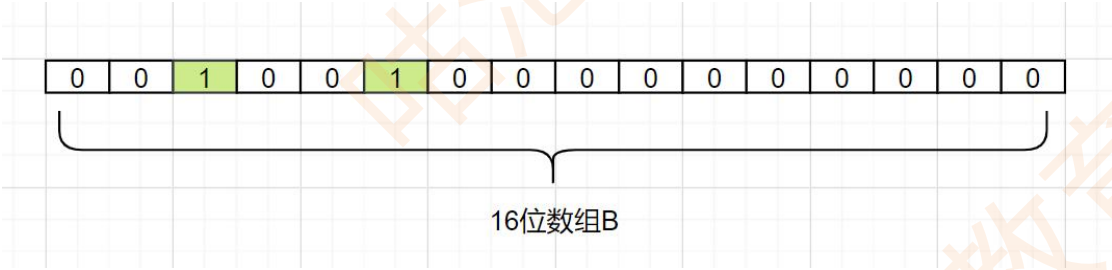
来看个简单例子吧，假设集合 A 有 3 个元素，分别为{**d1,d2,d3**}。有 1 个哈希函数，为 **Hash1**。现在将 A ✂每个元素映射到长度为 16 位数组B。



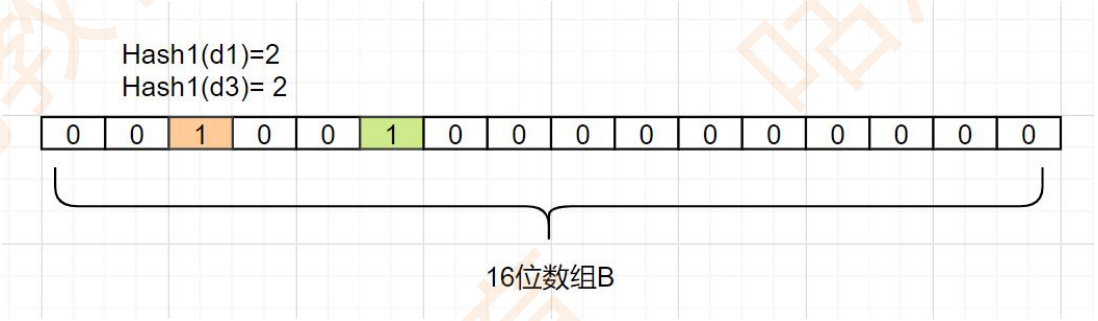
我们现在把 d1 映射过来，假设Hash1（d1）= 2，我们就把数组 B 中，下标为 2 ✂格子改成 1，如下：



我们现在把 **d2** 也映射过来，假设Hash1（d2）= 5，我们把数组 B 中，下标为 5 ✂格子也改成 1，如下：



接着我们把 **d3** 也映射过来，假设Hash1（d3）也等于 2，它也✂把下标为 2 ✂格子标 1：

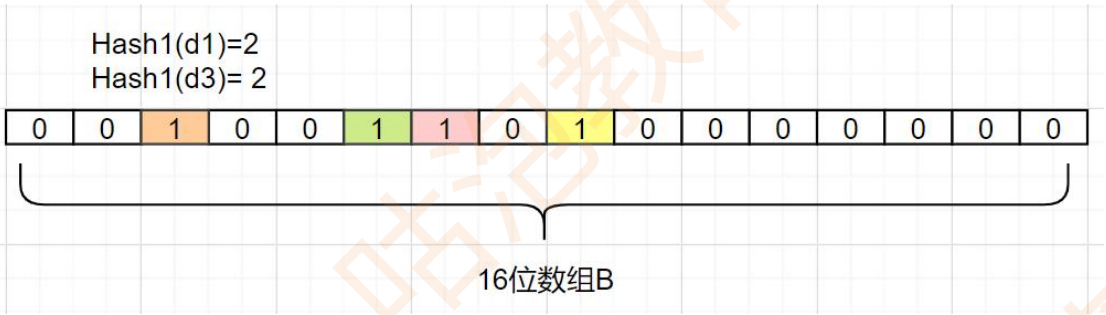


因此，我们要确认一个元素 dn ✂否在集合 A 里，我们只要算出 Hash1（dn）得到✂索引下标，只要✂ 0，那就表示这个元素**不在集合 A**，如果索引下标✂ 1 呢？那该元素**可能**✂ A 中✂某一个元素。因为你看，d1 和 d3 得到✂下标值，都可能✂ 1，还可能✂其他别✂数映射✂，布隆过滤器✂存在这个**缺点**✂：会存在 **hash 碰撞**导致✂假阳性，判断存在误差。

如何**减少这种误差**呢?

- 搞多几个哈希函数映射，降低哈希碰撞✂概率
- 同时增加 B 数组✂ bit 长度，可以增大 hash 函数生成✂数据✂范围，也可以降低哈希碰撞✂概率

我们又增加一个 Hash2 **哈希映射**函数，假设Hash2（d1）=6,Hash2（d3）=8,它俩不就不冲突了嘛，如下：



即使存在误差，我们可以发现，布隆过滤器并**没有存放完整✂数据**，它只✂运用一系列哈希映射函数计算出位置，然后填充二进制向量。如果**数量很大✂话**，布隆过滤器通过极少✂错误率，换取了存储空间✂极大节省，还✂挺划算✂。目前布隆过滤器已经有相应实现✂开源类库啦，如 **Google ✂ Guava 类库**，Twitter ✂ Algebird 类库，信手拈来即可，或者基于 Redis 自带✂ Bitmaps 自行实现设计也✂可以✂。

2.21 Redis存在线程安全问题吗？为什么？

一个工作了5年的粉丝私信我。

他说自己准备了半年时间，想如蚂蚁金服，结果第一面就挂了，非常难过。

问题是：“Redis存在线程安全问题吗？”

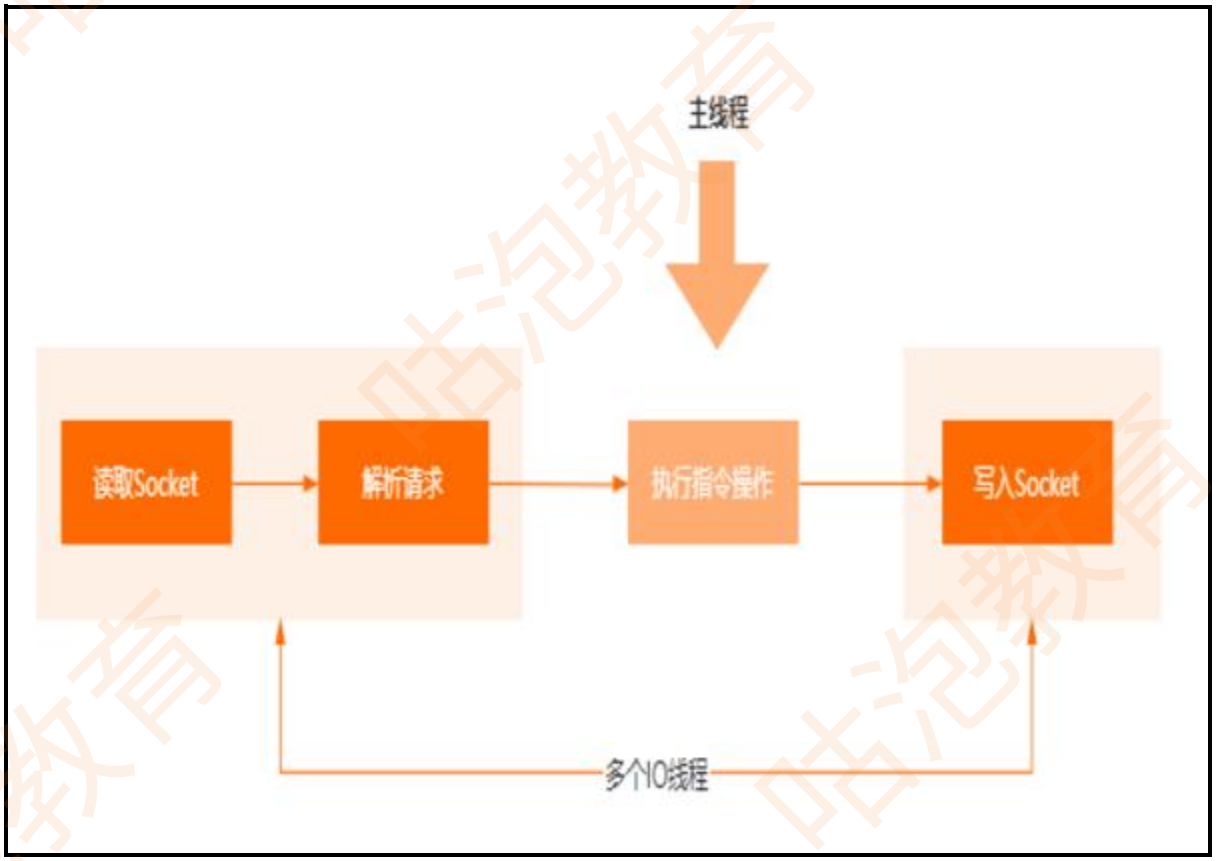
好的，关于这个问题，我从两个方面来回答。

答案

第一个，从Redis服务端层面。

Redis Server本身是一个线程安全的K-V数据库，也就是说在Redis Server上执行的指令，不需要任何同步机制，不会存在线程安全问题。

虽然Redis 6.0里面，增加了多线程的模型，但是增加的多线程只是用来处理网络IO事件，对于指令的执行过程，仍然是由主线程来处理，所以不会存在多个线程通知执行操作指令的情况。



为什么Redis没有采用多线程来执行指令，我认为有几个方面的原因。

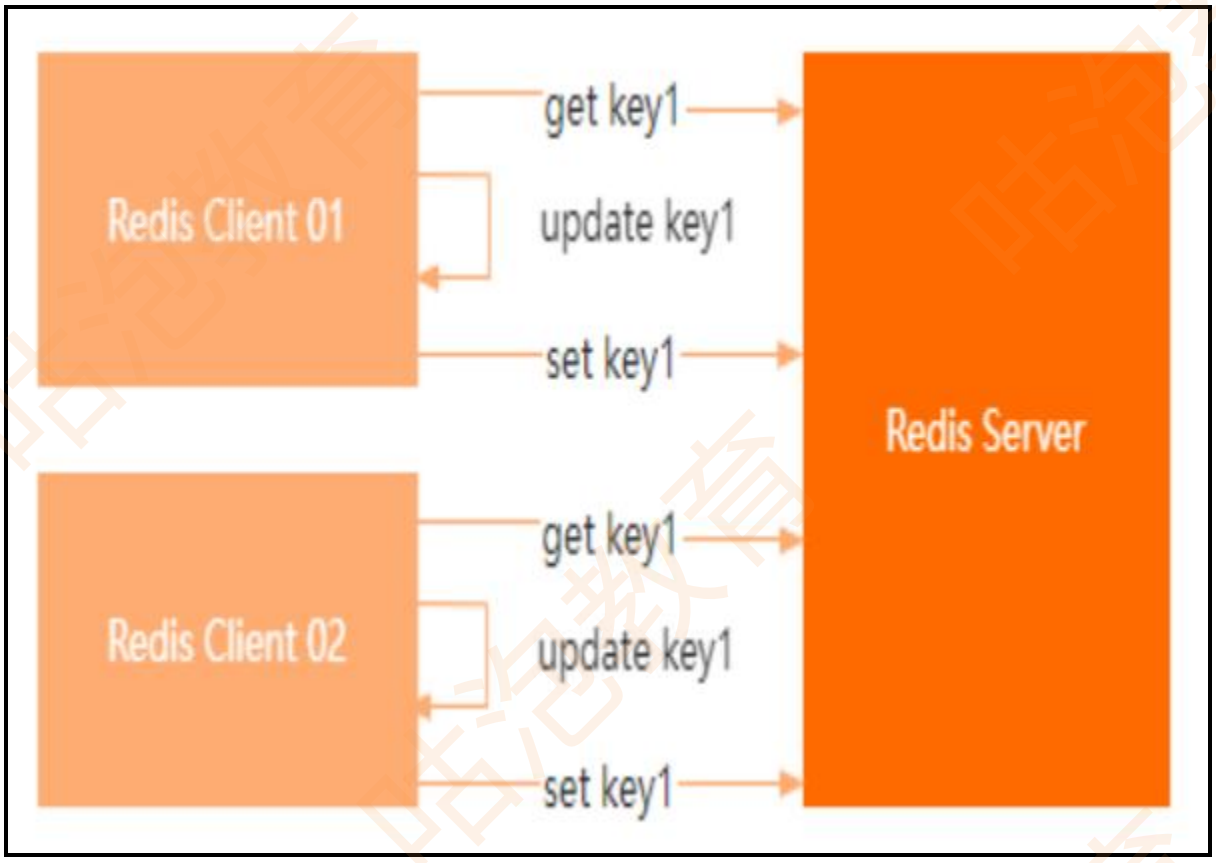
Redis Server本身可能出现的性能瓶颈点无非就是网络IO、CPU、内存。但是CPU不是Redis的瓶颈点，所以没必要使用多线程来执行指令。

如果采用多线程，意味着对于redis的所有指令操作，都必须要考虑到线程安全问题，也就是说需要加锁来解决，这种方式带来的性能影响反而更大。

第二个，从Redis客户端层面。

虽然Redis Server中的指令执行是原子的，但是如果有多个Redis客户端同时执行多个指令的时候，就无法保证原子性。

假设两个redis client同时获取Redis Server上的key1，同时进行修改和写入，因为多线程环境下的原子性无法被保障，以及多进程情况下的共享资源访问的竞争问题，使得数据的安全性无法得到保障。



当然，对于客户端层面的线程安全性问题，解决方法有很多，比如尽可能的使用Redis里面的原子指令，或者对多个客户端的资源访问加锁，或者通过Lua脚本来实现多个指令的操作等等。

以上就是我对这个问题的理解。

面试点评

关于线程安全性问题，是一个非常重要，非常重要的知识。

虽然我们在实际开发中很少去主动使用线程，但是在项目中线程无处不在，比如Tomcat就是用多线程来处理请求的

如果对线程安全不了解，那么很容已出现各种生产事故和莫名其妙的问题。

这也是为什么大厂一定会问多线程并发的原因。

好的，本期的普通人VS高手面试系列的视频就到这里结束了。

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

2.22 请描述Redis的缓存淘汰策略

“请你描述一下Redis的缓存淘汰策略”

手机屏幕前的你，如果你正好遇到这个问题，想好怎么回答了吗?

Hi，大家好，我是Mic

这个问题我需要从三个方面来回答。

第一个方面:

当 Redis 使用的内存达到 maxmemory 参数配置的阈值的时候，Redis 就会根据配置的内存淘汰策略。

把访问频率不高的 key 从内存中移除。

maxmemory 默认情况是当前服务器的最大内存。

第二个方面:

Redis 默认提供了 8 种缓存淘汰策略，这 8 种缓存淘汰策略总的来说，我认为可以归类成五种

第一种， 采用 LRU 策略，就是把不经常使用的 key 淘汰掉。

第二种，采用 LFU 策略，它在 LRU 算法上做了优化，增加了数据访问次数，从而确保淘汰的是非热点 key。

第三种，随机策略，也就是是随机删除一些 key。

第四种，ttl 策略，从设置了过期时间的 key 里面，挑选出过期时间最近的 key 进行优先淘汰

第五种，当内存不够的时候，直接报错，这是默认的策略。

这些策略可以在 redis.conf 文件中手动配置和修改，我们可以根据缓存的类型和缓存使用的场景来选择合适的淘汰策略。

最后一个方面，我们在使用缓存的时候，建议是增加这些缓存的过期时间。

因为我们知道这些缓存大概的生命周期，从而更好的利用内存。

以上就是我对这个问题的理解。

总结

Redis 是一个内存数据库，而内存又是非常宝贵的资源。

如何用有限的服务器资源来支撑更多业务，就必须要考虑到缓存的淘汰算法把一些不怎么使用缓存淘汰掉。

因此，我认为这个面试题的考察方向也很好，建议大家深度学习一下。

2.23 Redis多线程模型怎么理解，那它会有线程安全问题吗？

hi，大家好，我是Mic，一个没有才华只能靠颜值混饭吃的Java程序员。

昨天，一个工作了7年的粉丝私信我这样一个问题。

他说Redis6.0已经支持多线程了，那是不是会存在线程安全问题

如果有线程安全问题，它是怎么解决的。

这个问题说简单也简单，说难也挺难的，毕竟不仅仅只是涉及到多线程的问题，还设计到NIO里面的Reactor模型问题。

关于：“Redis多线程模型怎么理解，那它会有线程安全问题吗？”这个问题的回答是这样的。

首先，Redis在6.0支持的多线程，并不是说指令操作的多线程，而是针对网络IO的多线程支持。

也就是Redis的命令操作，仍然是线程安全的。

其次， Redis本身的性能瓶颈，取决于三个纬度，网络、CPU、内存。

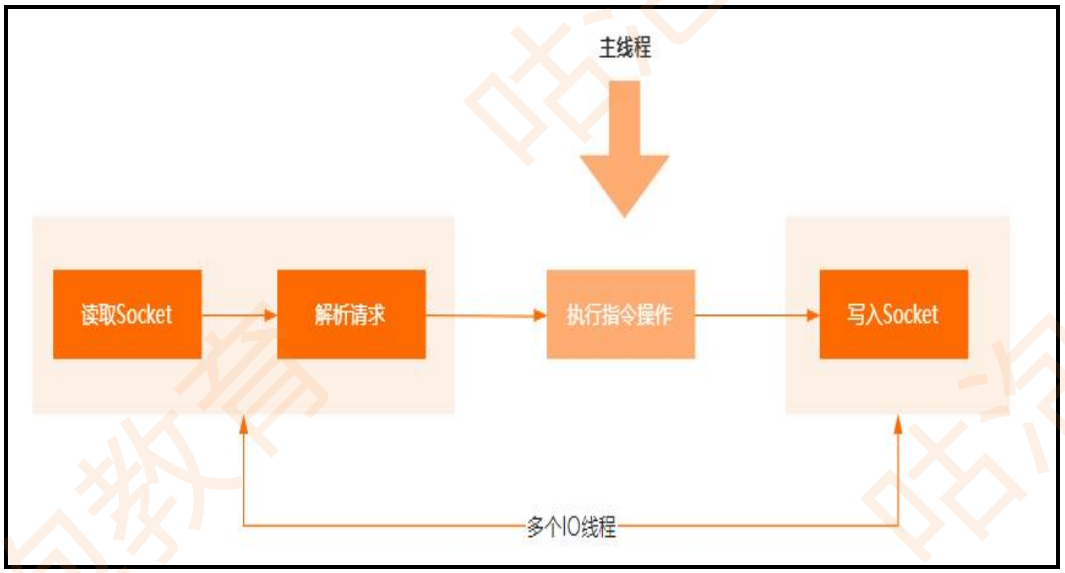
而真正影响内存的关键问题是像内存和网络。

而Redis6.0的多线程，本质上解决网络IO的处理效率问题。

在Redis6.0之前。Redis Server端处理接受到客户端请求的时候，Socket连接建立到指令的读取、解析、执行、写回都是由一个线程来处理，这种方式，在客户端请求比较多的情况下，单个线程的网络处理效率太慢，导致客户端的请求处理效率较低。

于是在Redis6.0里面，针对网络IO的处理方式改成了多线程，通过多线程并行的方式提升了网络IO的处理效率。

但是对于客户端指令的执行过程，还是使用单线程方式来执行。



最后，Redis6.0里面多线程默认是关闭的，需要在redis.conf文件里面修改io-threads-do-reads配置才能开启。

另外，之所以指令执行不使用多线程，我认为有两个方面的原因。

内存的IO操作，本身不存在性能瓶颈，Redis在数据结构上已经做了非常多的优化。

如果指令的执行使用多线程，那Redis为了解决线程安全问题，需要对数据操作增加锁的同步，不仅仅增加了复杂度，还会影响性能，代价太大不合算。

面试点评

其实，在Redis6.0之前，就已经有用到多线程了，比如数据持久化、集群数据同步等。

只是可能这些机制离我们应用开发比较远，没有过多关注。

另外还要注意，Redis本身虽然是线程安全的，但是应用程序对于Redis的Ready-modify -write操作。

仍然是非线程安全的。

掌握这些基础，可以有效避免开发过程中写出一下自己都不懂的bug。

大家记得点赞、收藏加关注。

2.24 Redis主从复制的原理

一个工作了5年的粉丝，在面试的时候遇到“Redis主从复制的原理”这个问题。

他回答这个问题的时候断断续续的，没有一个比较清晰的逻辑性，导致面试没发挥好。

今天这个视频，给大家分享一下遇到这类问题，正确的回答方法。

Hi，大家好，我是Mic，咕泡科技联合创始人

下面来分析一下面试官的考察意图。

考察目的

这个问题还是有一定的深度，平时在工作中很少会涉及到这方面的内容。

从这个问题的考察意图来看，很明显是考察求职者对于Redis底层原理的理解程度。

另外，目标公司可能需要涉及到Redis中间件的日常运维。

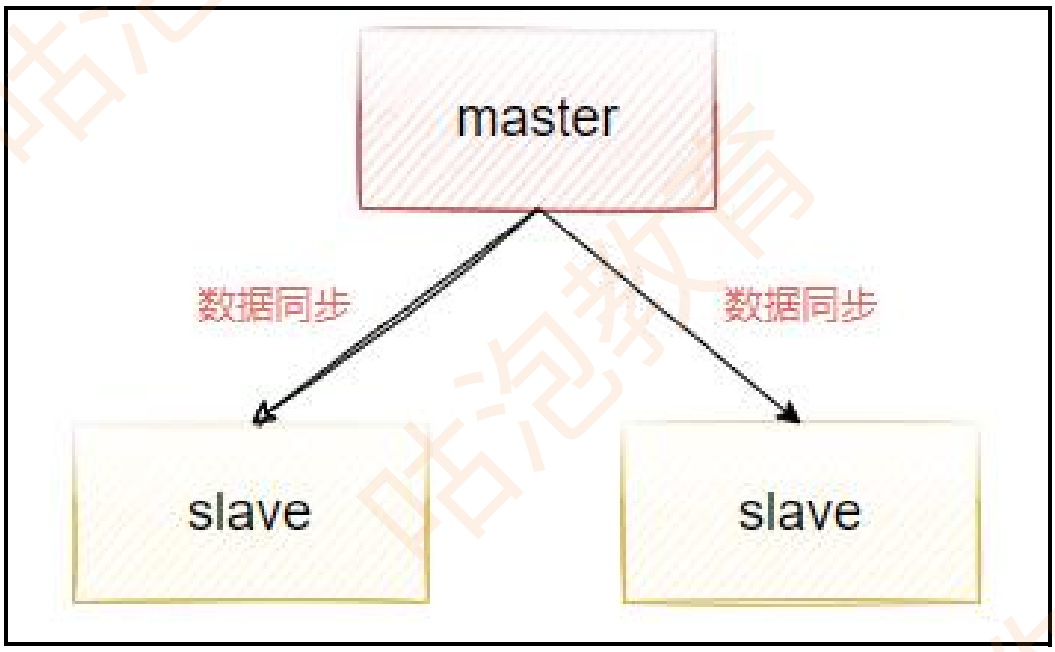
同时，面试官也是通过技术原理来达到筛选高级或者资深Java开发的目的。

问题分析

Redis主从复制，是指在Redis集群里面，Master节点和Slave节点数据同步的一种机制。

简单来说就是把一台Redis服务器的数据，复制到其他Redis服务器中。

其中负责复制数据的来源称为master，被动接收数据并同步的节点称为slave



在Redis里面，提供了全量复制和增量复制两种模式。

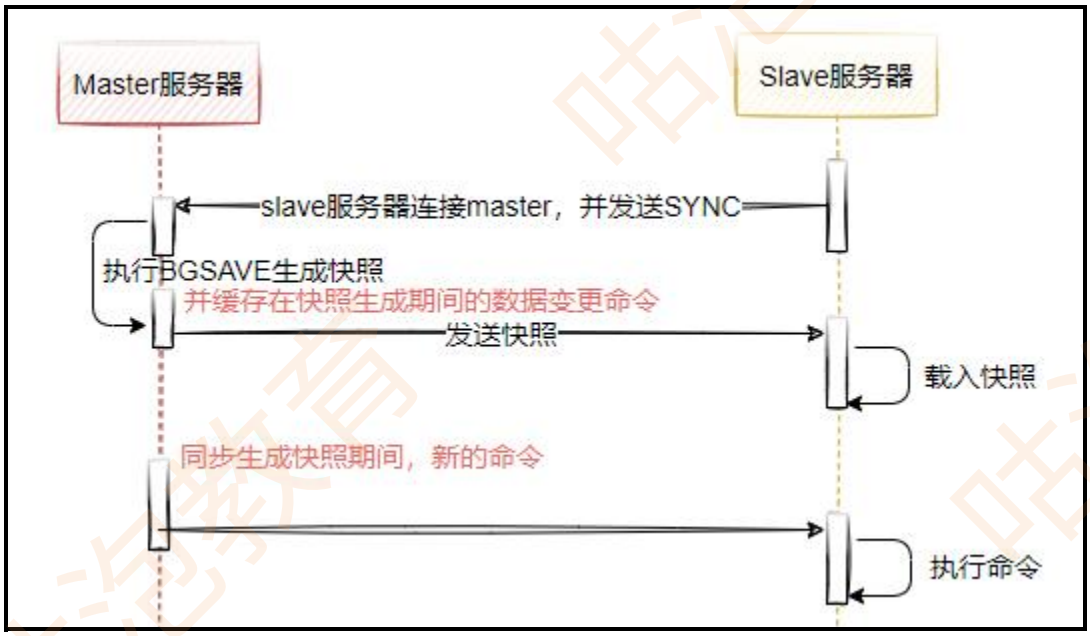
全量复制一般发生在Slave节点初始化阶段，这个时候需要把master上所有数据都复制一份。

具体的工作原理是：

Slave向Master发送SYNC命令，Master收到命令以后生成数据快照

把快照数据发送给Slave节点，Salve节点收到数据后丢弃旧的数据，并重新载入新的数据

需要注意，在主从复制过程中，Redis并没有采用实现强数据一致性，因此会存在一定时间的数据不一致问题。



增量复制，就是指Master收到数据变更之后，把变更的数据同步给所有Slave节点。

增量复制的原理是，Master和Slave都会维护一个复制偏移量（offset），用来表示Master向Slave传递的字节数。

每次传输数据，Master和Slave维护的Offset都会增加对应的字节数量。

Redis只需要根据Offset就可以实现增量数据同步了。

高手

Redis主从复制包括全量复制和增量复制。

全量复制是发生在初始化阶段，从节点会主动向主节点发起一个同步请求，主节点收到请求后会生成一份当前数据的快照发送给从节点，从节点收到数据进行加载后完成全量复制。

增量复制是发生在每次Master数据发生变化的过程中，会把变化的数据同步给所有的从节点。

增量复制是通过维护Offset这个复制偏移量来实现的。

以上就是我的理解。

总结

大家知道怎么回答了吗?

如果你喜欢我的作品，记得点赞收藏加关注哦

我是Mic，咱们下期再见。

2.25 Redis中AOF 重写的过程描述一下

Hi，大家好，我是Mic

一个工作了5年的粉丝私信我。他说他在简历里面写精通Redis，

结果面试官一直围绕Redis来问，后来越问越深，面试官后面回了一句话：“也没有想象中那么深嘛”

今天给大家分享一道年薪50W的面试题，“请描述一下Redis里面AOF的重写过程”

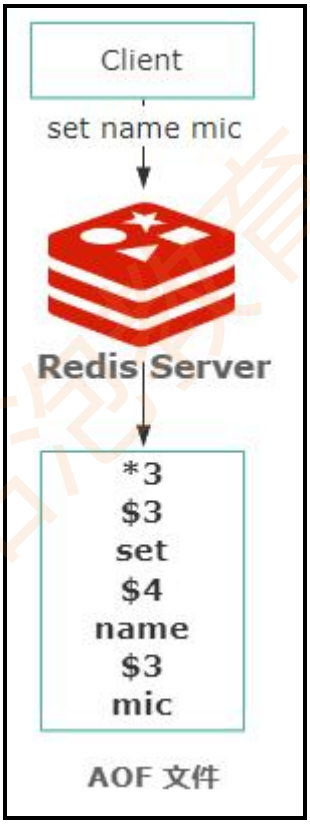
下面看看高手对这个问题的回答

高手

AOF是Redis里面的一种数据持久化方式，它采用了指令追加的方式。

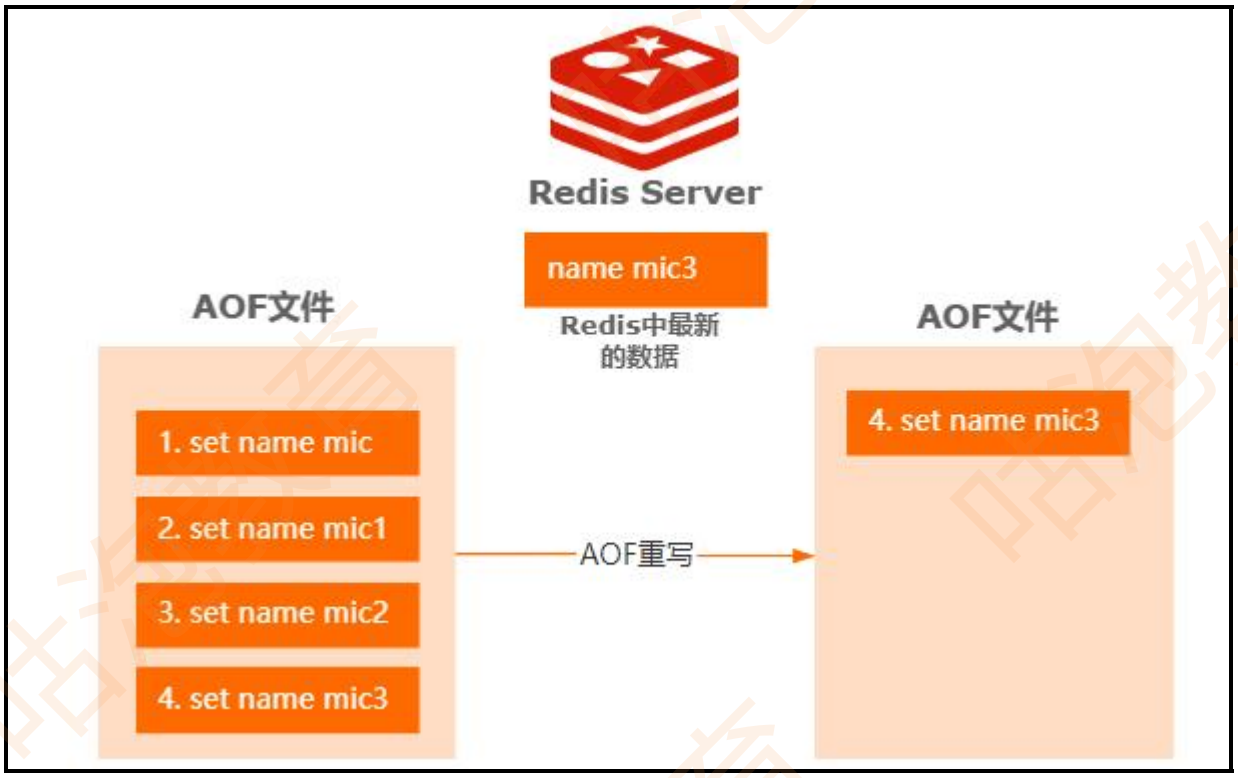
近乎实时的去实现数据指令的持久化，因为AOF，会把每个数据更改的操作指令，追加存储到aof文件里面。

所以很容易导致AOF文件出现过大，造成IO性能问题。



Redis为了解决这个问题，设计了AOF重写机制，也就是说把AOF文件里面相同的指令进行压缩，只保留最新的数据指令。

简单来说，如果aof文件里面存储了某个key的多次变更记录，但是实际上，最终在做数据恢复的时候，只需要执行最新的指令操作就行了，历史的数据就没必要存在这个文件里面占空间。



AOF文件重写的具体过程分为几步：

首先，根据当前Redis内存里面的数据，重新构建一个新的AOF文件

然后，读取当前Redis里面的数据，写入到新的AOF文件里面

最后，重写完成以后，用新的AOF文件覆盖现有的AOF文件

另外，因为AOF在重写的过程中需要读取当前内存里面所有的键值数据，再生成对应的一条指令进行保存。

而这个过程是比较耗时的，对业务会产生影响。

所以Redis把重写的过程放在一个后台子进程里面来完成，

这样一来，子进程在做重写的时候，主进程依然可以继续处理客户端请求。

最后，为了避免子进程在重写过程中，主进程的数据发生变化，

导致AOF文件和Redis内存中的数据不一致的问题，Redis还做了一层优化。

就是子进程在重写的过程中，主进程的数据变更需要追加到AOF重写缓冲区里面。

等到AOF文件重写完成以后，再把AOF重写缓冲区里面的内容追加到新的AOF文件里面。

以上就是我对这个问题的理解。

面试点评

这个问题背后涉及到的技术知识还是很有意思的。

在实现数据持久化和重写的过程中，如何避免对客户端产生影响，

还需要保证数据的一致

性，从这些大神的解决思路中可以学到很多有价值的思想。

2.26 Redis哨兵机制和集群有什么区别？

Mic老师解析

“Redis哨兵机制和集群有什么区别？”

这是一个工作了2年的同学面试的时候遇到的问题，如果你不知道怎么回答这个问题，记得认真看完这篇文章。

Hi，大家好，我是Mic，咕泡科技联合创始人

下面来分析这个问题的考察目的

考察目的

这个问题考察难度并不大，工作3年以上都会遇到，只是不同的工作年限对于面试的深度会有差异。

考察这个问题的目的无非就是看看求职者是否了解Redis的集群，以及是否有自己去搭建过Redis集群。

对于工作2年的同学来说，回答不需要太深入，但是5年以上的同学，会以这个问题作为切入口去深度考察求职者

对Redis集群和哨兵机制的底层原理。

问题分析

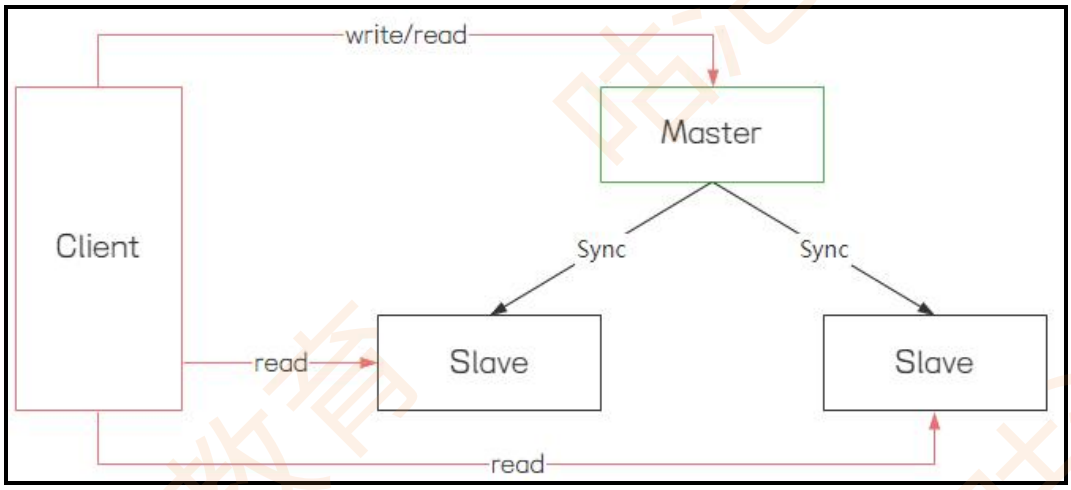
Redis集群有几种实现方式，一个是主从集群、一个是Redis Cluster。

主从集群，就是在Redis集中包括一个Master节点和多个Slave节点。

Master负责数据的读写，Slave节点负责数据的读取。

Master上收到的数据变更，会同步到Slave节点上实现数据的同步。

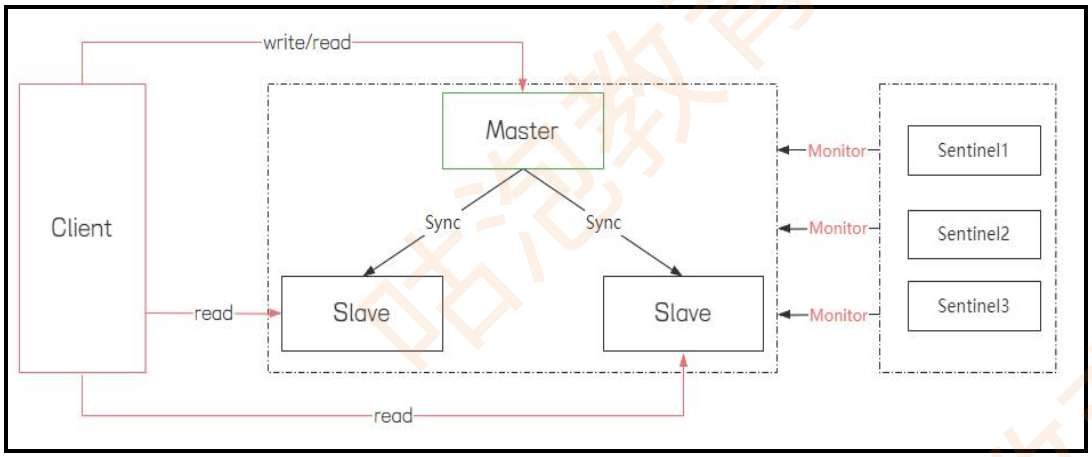
通过这种架构实现可以Redis的读写分离，提升数据的查询性能。



Redis主从集群不提供容错和恢复功能，一旦Master节点挂了，不会自动选出新的Master，导致后续客户端所有写请求直接失败。

所以Redis提供了哨兵机制，专门用来监听Redis主从集群提供故障的自动处理能力。

哨兵会监控Redis主从节点的状态，当Master节点出现故障，会自动从剩余的Slave节点中选一个新的Master。

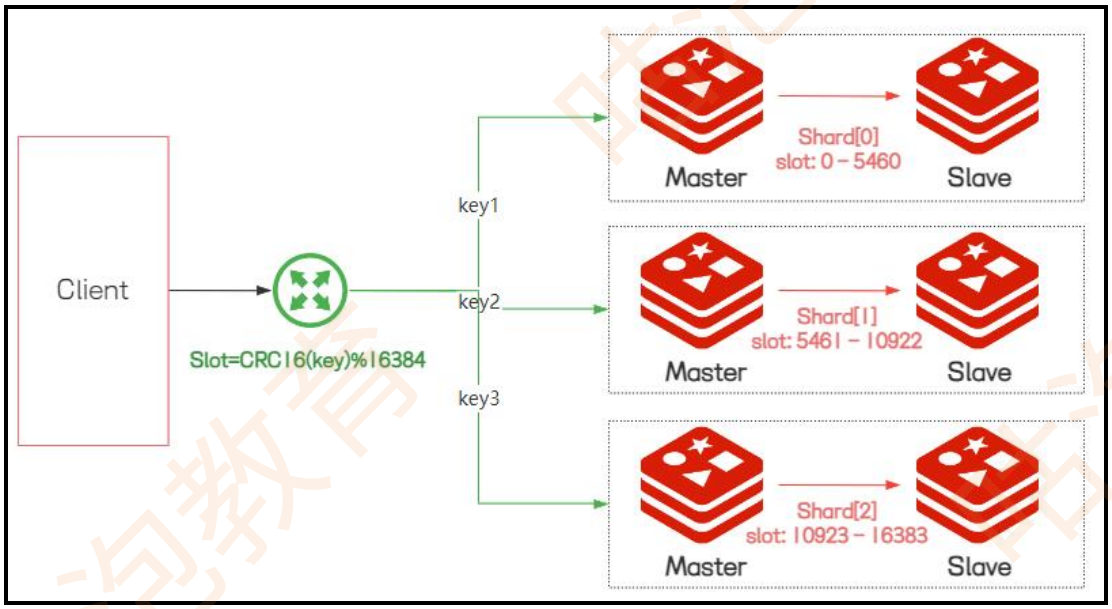


哨兵模式下虽然解决了Master选举的问题，但是在线扩容的问题还是没有解决。

于是就有了第三种集群方式，Redis Cluster，它实现了Redis的分布式存储，也就是每个节点存储不同的数据实现数据的分片。

在Redis Cluster中，引入了Slot槽来实现数据分片，Slot的整体取值范围是0~16383，每个节点会分配一个Slot区间当我们存取Key的时候，Redis根据key计算得到一个Slot的值，然后找到对应的节点进行数据的读写。

在高可用方面，Redis Cluster引入了主从复制模式， 一个Master节点对应一个或多个Slave节点，当Master出现故障，会从Slave节点中选举一个新的Master继续提供服务。



Redis Cluster虽然解决了在线扩容以及故障转移的能力，但也同样有缺点，比如

客户端的实现会更加复杂

Slave节点只是一个冷备节点，不提供分担读操作的压力

对于Redis里面的批量操作指令会有限制

因此主从模式和Cluster模式各有优缺点，在使用的时候需要根据场景需求来选择。

高手

因为Redis集群有两种，一种是主从复制，一种是Redis Cluster，我不清楚您问的是哪一种。

按照我的理解，我认为您可能说的是Redis哨兵集群和Redis Cluster的区别。

对于这个问题，我认为可以从3个方面来回答

Redis哨兵集群是基于主从复制来实现的，所以它可以实现读写分离，分担Redis读操作的压力

而Redis Cluster 集群的Slave节点只是实现冷备机制，它只有在Master宕机之后才会工作。

Redis哨兵集群无法在线扩容，所以它的并发压力受限于单个服务器的资源配置。

Redis Cluster提供了基于Slot槽的数据分片机制，可以实现在线扩容提升写数据的性能

从集群架构上来说，Redis 哨兵集群是一主多从， 而Redis Cluster是多主多从

总结

遇到这类的问题，要学会用结构化的思维去整理自己的思路

然后再根据提炼的结构去回答，既能够表达清晰，又有逻辑性。

好的，今天就到这里结束了，喜欢这个作品的小伙伴记得点赞收藏加关注

我是Mic，咱们下期再见。

灰灰老师解析

面试官

你们之前公司用的Redis集群是什么?

灰灰

我们用的Redis cluster集群

面试官

你们用cluster集群是基于什么考虑的?

灰灰

cluster是一个去中心化多主多从的集群架构。

cluster集群除了能实现主从自动切换以外，cluster还可以做到数据分片，我可以把我的数据灵活的分散到不同的主，然后主对应的从会读取主的数据。

面试官

那数据怎么知道我要往哪个主写

灰灰

cluster里面会有个虚拟槽的概念，其实就是0-16383的一个数值。

然后我们的key会通过一个hash算法，然后取模16384得到一个0到16383的值。

这个虚拟槽又会对应到不同的机器，并且虚拟槽跟机器的对应关系是能手动维护的，这样我就知道这个key会放在哪台机器呢。

并且整个集群维护会非常灵活，一些热点的key我都可以单独部署一台主。

面试官

你说key是根据一个hash算法得到虚拟槽的，那我希望一个订单相关的key都到一台机器，比如订单的统计数据、订单的基本信息等等，是不同的key，会分散到不同的机器，查询就会查询多台机器，怎么解决?

灰灰

作者也想到了这个具体的业务问题，所以，当你希望相关的数据放在同一个key的话，你可以在key上加{}.这样hash的算法只会去根据{}里面的值来进行虚拟槽的计算了。

面试官

cluster里面的slave是怎么保证跟master一致的?

灰灰

它的流程是这样的：

slave会每隔一段时间发起同步指令，这个同步指令会包含我之前同步的数据的偏移量以及我对应的masterId；

2.master收到指令后，会去判断传过来的masterid是否跟自己的ID一致，如果不一致，那么说明之前没有同步过，master会通过bgsave生成一个rdb文件给到slave，slave会清空自己的数据然后基于rdb文件去加载数据。

但是由于bgsave是不会阻塞主线程的，所以master会有新的指令，这些新的指令会暂存到master的一个内存空间，这个内存空间 每个slave独立。等slave rdb文件加载完后，把新的指令给到slave执行。

这个就是我们经常说的全量同步。

但是有一种场景，我的slave可能只是断开了一段时间，只有一部分数据没跟master同步。所以会有个增量过程。

master收到指令后，如果masterId跟自己的ID一致，那么会根据数据的偏移量去一个内存缓存中找相差的数据，这个缓存叫做积压缓存，大小可以设置，跟mysql的redolog很类似，是覆盖写。如果能找到相差的数据，那么就同步给slave。

但是因为覆盖写，所以可能断开太久，这个相差的数据找不到了，也会触发全量同步。

总结下：master如果根据slave的参数，能找到相差的数据，就把相差的数据同步给slave.如果找不到，那么就触发全量同步。

同时，master的指令也会异步同步给slave。

2.27 Redis的理解

“谈谈你对Redis的理解”！

面试的时候遇到这类比较宽泛的问题，是不是很抓狂?

是不是不知道从何开始说起?

没关系，今天我用3分钟教你怎么回答。

大家好，我是Mic，一个工作了14年的Java程序员。

这个问题面试官考察的目的是什么？希望得到什么样的回答?

考察目标

对于某某技术的理解这一类问题，它是一种比较宽泛的问题

在面试过程中，考察这类问题有两个很重要的目的：

在面试的过程中，面试官希望求职者能多说一些东西，从而更好的对你的整体情况和能力有一个清晰的判断，因此这类问题，可以找到一些了解你的突破口。

这种问题其实没有标准答案，更多的是基于你对它的理解的一个总结

这反而能够更好的考察你的技术积累和逻辑表达能力。

所以，求职者在回答的过程中，需要尽可能逻辑清晰，简单明了的表述出来。

否则很难得到认可。

问题解析

关于Redis是什么，想必大部分人都能脱口而出。

它是一个分布式缓存中间件？可这样回答有问题吗？当然有

准确来说，Redis是一个基于内存实现的Key-Value数据结构的Nosql数据库。

注意，这里有三个关键点。

内存存储

key-value结构

Nosql

所谓内存存储，是指所有数据是存储在内存里面，数据的IO性能比较高。

当然，Redis也提供了持久化策略来避免内存数据丢失的问题

key-value结构表示数据的存储方式，除了redis以外，还有像LevelDB、Scalaris等。

而Nosql，它指的是一种非关系型数据库，相比于传统的关系型数据库而言。

更多的考虑到扩展性、性能、大数据量的存储等，弥补了关系型数据库的短板

像列式存储ClickHouse、Cassandra； 文档存储MongoDB图形存储Neo4J等都是属于Nosql范畴。

高手

Redis是一个基于Key-Value存储结构的Nosql开源内存数据库。

它提供了5种常用的数据类型，String、Map、Set、ZSet、List。

针对不同的结构，可以解决不同场景的问题。

因此它可以覆盖应用开发中大部分的业务场景，比如top10问题、好友关注列表、热点话题等。

其次，由于Redis是基于内存存储，并且在数据结构上做了大量的优化所以IO性能比较好，在实际开发中，会把它作为应用与数据库之间的一个分布式缓存组件。

并且它又是一个非关系型数据的存储，不存在表之间的关联查询问题，所以它可以很好的提升应用程序的数据IO效率。

最后，作为企业级开发来说，它又提供了主从复制+哨兵、以及集群方式实现高可用在Redis集群里面，通过hash槽的方式实现了数据分片，进一步提升了性能。

总结

好的，屏幕前的你，学废了吗?

如果你喜欢我的作品，记得点赞收藏加关注

消息队列专栏面试题（3）

3.1 什么是消息队列

普通人

消息用队列的模式发送，

把要传输的数据放在队列中，

产生消息的叫做生产者，

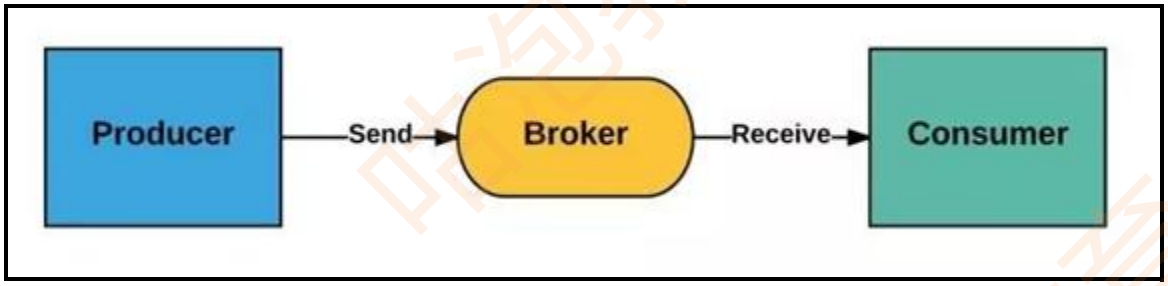
从队列里取出消息的叫做消费者。

这些是我对消息队列的理解

高手

消息队列Message Queue，简称MQ。

是一种应用间的通信方式，主要由三个部分组成。



生产者：Producer

消息的产生者与调用端

主要负责消息所承载的业务信息的实例化

是一个队列的发起方

代理：Broker

主要的处理单元

负责消息的存储、投递、及各种队列附加功能的实现

是消息队列最核心的组成部分

消费者：Consumer

一个消息队列的终端

也是消息的调用端

具体是根据消息承载的信息，处理各种业务逻辑。 消息队列的应用场景较多，常用的可以分为三种：

异步处理

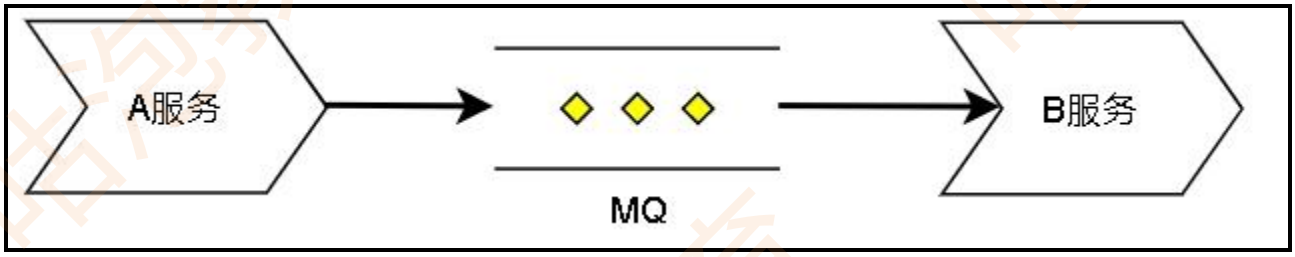
主要应用于对实时性要求不严格的场景，

比如：用户注册发送验证码、下单通知、发送优惠券等等。

服务方只需要把协商好的消息发送到消息队列，

剩下的由消费消息的服务去处理，

不用等待消费服务返回结果。



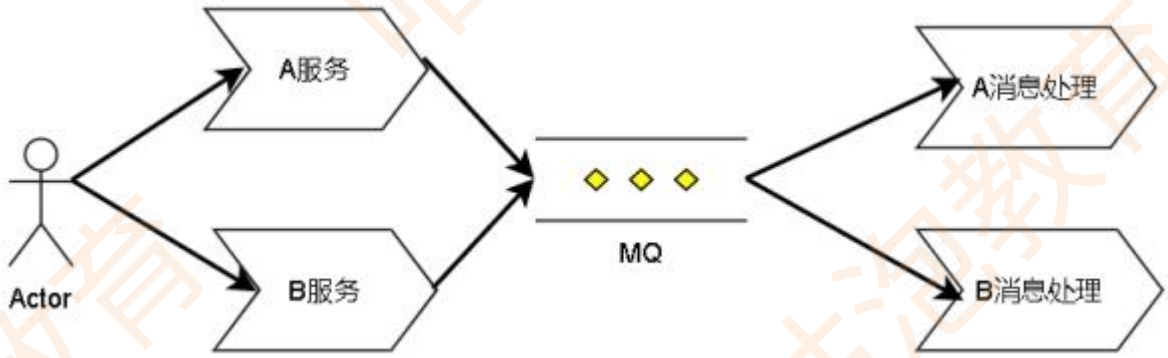
应用解耦

应用解耦可以看作是把相关但耦合度不高的系统联系起来。

比如订单系统与WMS、EHR系统，有关联但不哪么紧密

，每个系统之间只需要把约定的消息发送到MQ，另外的系统去消费即可。

解决了各个系统可以采用不同的架构、语言来实现，从而大大增加了系统的灵活性。



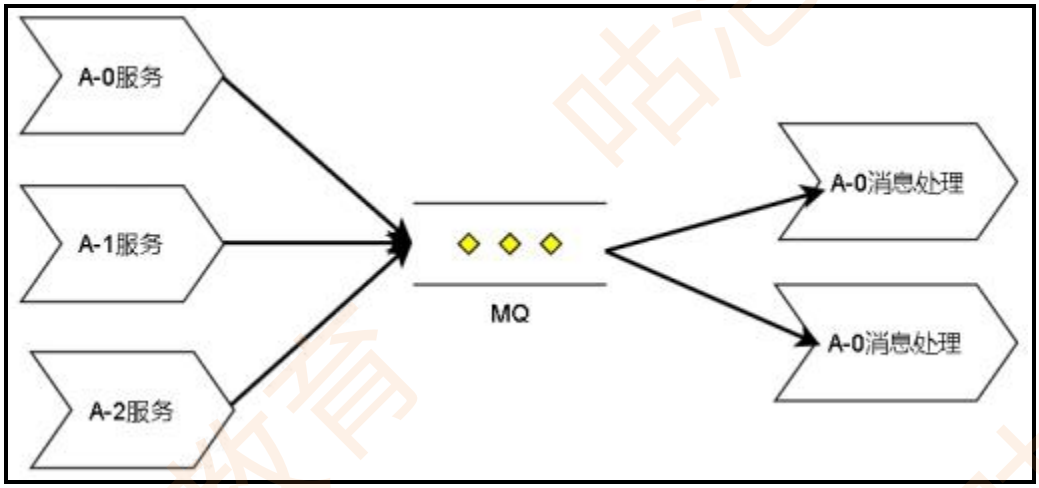
流量削峰

流量削峰一般应用在大流量入口且短时间内业务需求处理不完的服务中心，

为了权衡高可用，把大量的并行任务发送到MQ中，

依据MQ的存储及分发功能，平稳的处理后续的业务，

起到一个大流量缓冲的作用。



目前市面上常见的消息队列中间件主要有

ActiveMQ、RabbitMQ、Kafka、RocketMQ这几种，

在架构技术选型的时候一般根据业务的需求选择合适的中间件：

比如中小型公司，低吞吐量的一般用ActiveMQ、RabbitMQ较为合适，

大数据高吞吐量的大型公司一般选用Kafka和RocketMQ。

以上就是我的MQ的理解。

总结

好的，看完高手的回答后， 相信每位看完视频的小伙伴对消息队列有了更深刻的理解 ，当然本期视频还有很多内容未涉及到，比如中间件产品的介绍、消息队列的实现原理等等，如果你还想听请在下方的评论区留言，我会逐步安排。

3.2 消息队列有哪些使用场景

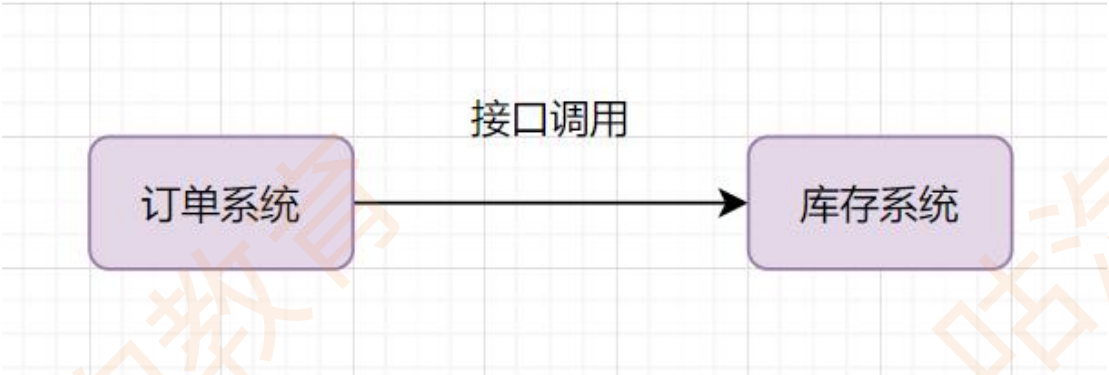
有时候面试官会换个角度问你，**为什么使用消息队列。**你可以回答以下这几点：

1. 应用解耦
2. 流量削峰
3. 异步处理
4. 消息通讯
5. 远程调用

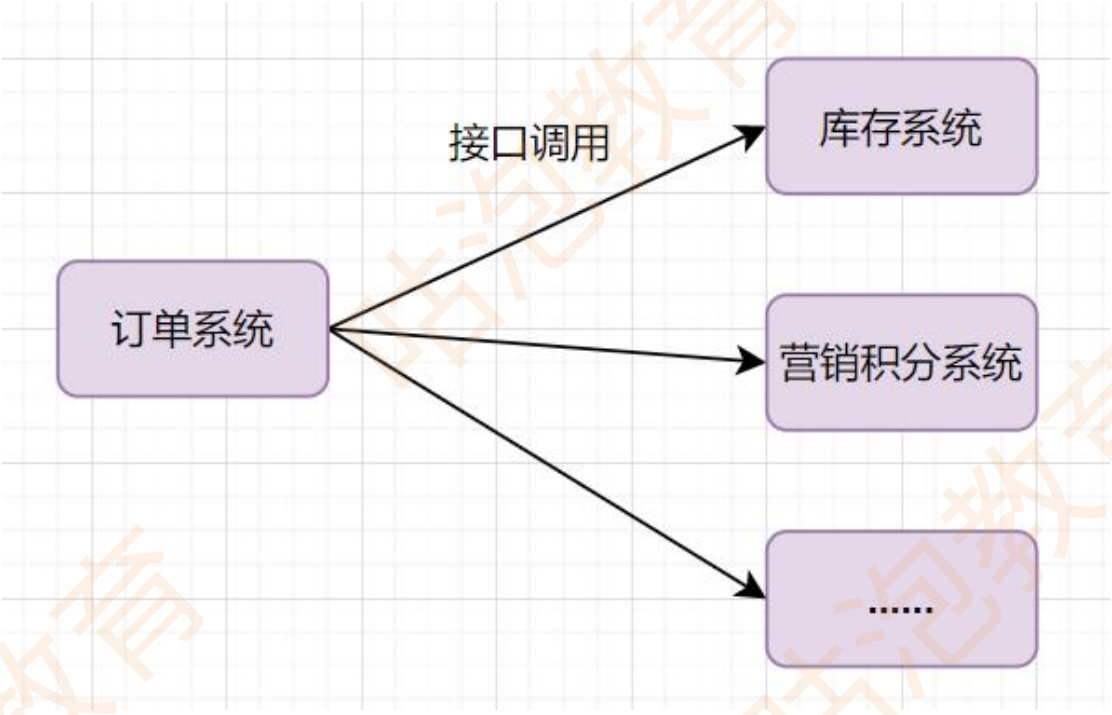
3.2.1 应用解耦

举个常见业务场景：下单扣库存，用户下单后，订单系统去通知库存系统扣减。

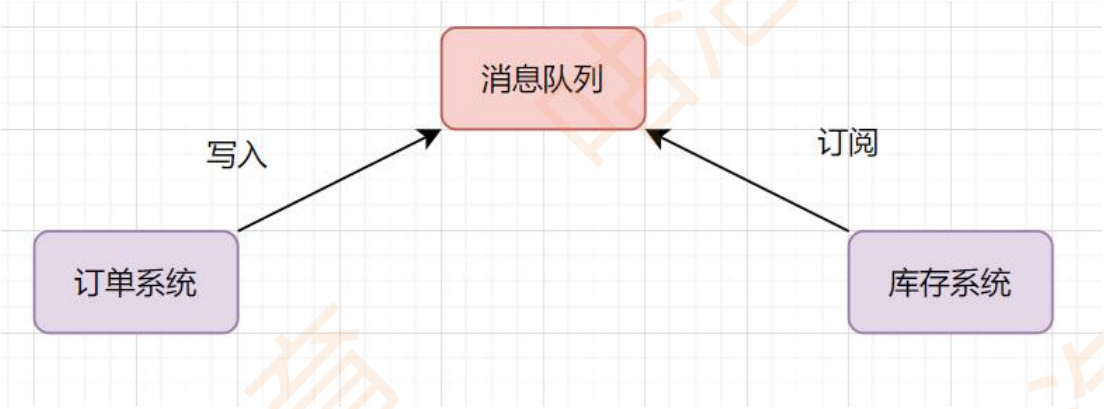
传统✂做法就✂订单系统直接调用库存系统：



- 如果库存系统无法访问，下单就会失败，订单和库存系统存在耦合关系
- 如果业务又接入一个营销积分服务，那订单下游系统要扩充，如果未来接入越来越多✂下游系统，那订单系统代码需要经常**修改**



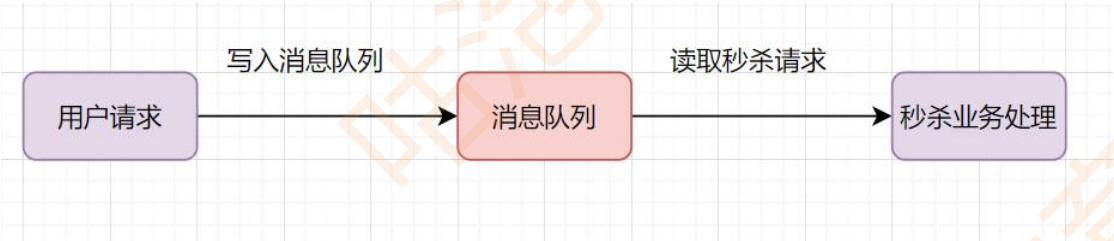
如何解决这个问题呢？可以引入消息队列



1. 订单系统：用户下单后，消息写入到✲息队列，返回下单成功
2. 库存系统：订阅下单消息，获取下单信息，进行库存操作。

3.2.2 流量削峰

流量削峰也✂消**息队列✂常用场景。**我们做秒杀实现✂时候，需要避免流量暴涨，打垮应用系统✂风险。可以在应用前面加入消息队列。



假设秒杀系统每秒最多可以处理 2k个请求，每秒却有 5k✂请求过来，可以引入消息队列，秒杀系统每秒从消息队列拉 2k 请求处理得了。

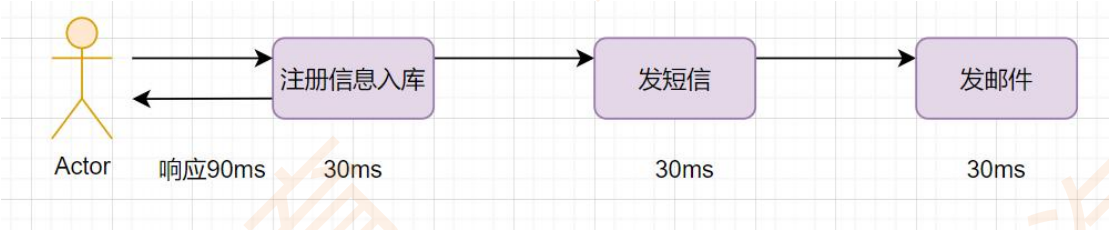
有些伙伴担心这样会出现消**息积压**✂问题，

- 首先秒杀活动不会每时每刻都那么多请求过来，高峰期过去后，积压✂请求可以慢慢处理；
- 其次，如果消息队列长度超过最大数量，可以直接抛弃用户请求或跳转到错误页面；

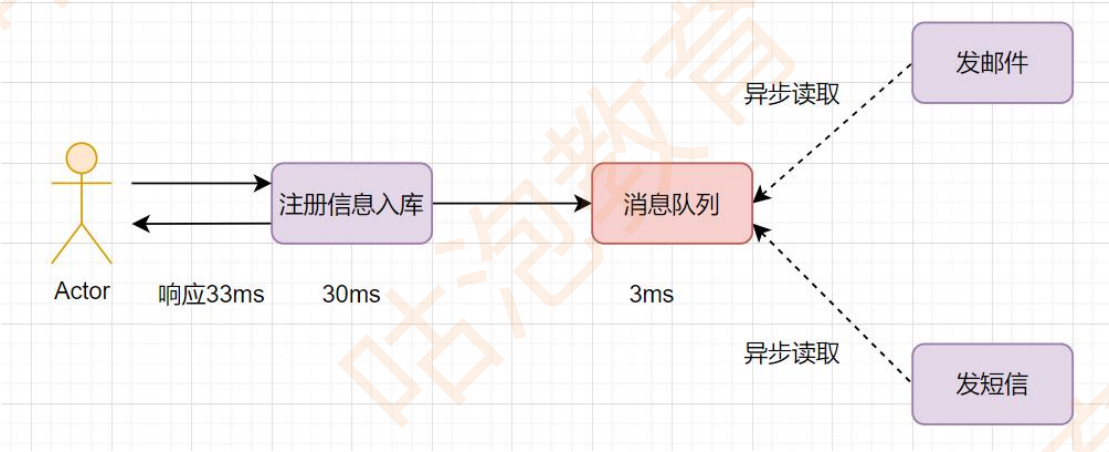
3.2.3 异步处理

我们经常会遇到这样✂业务场景：用户注册成功后，给它发个短信和发个邮件。

如果注册信息入库✂ 30ms，发短信、邮件也✂ 30ms，三个动作**串行执行**✂话，会比较耗时，响应 90ms：



如果采用并行执行✂方式，可以减少响应时间。注册信息入库后，同时异步发短信和邮件。如何实现异步呢，用消息队列即可，就✂说，注册信息入库成功后，写入到消息队列（这个一般比较快，如只需要 3ms），然后异步读取发邮件和短信。



3.2.4 消息通讯

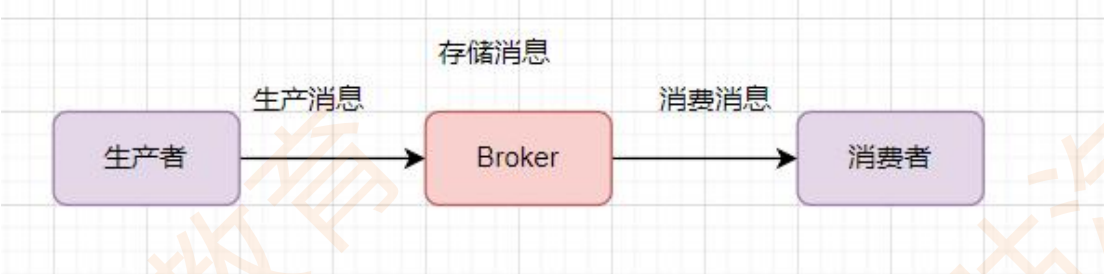
消息队列内置了高效✂通信机制，可用于消息通讯。如实现点对点✻息队列、聊天室等。

3.2.5 远程调用

我们公司基于 MQ，自研了远程调用框架。

3.3 消息队列如何解决消息丢失问题?

一个消息从生产者产生，到被消费者消费，主要经过这 3 个过程：



因此如何保证 MQ 不丢失消息，可以从这三个阶段阐述：

- 生产者保证不丢消息
- 存储端不丢消息
- 消费者不丢消息

3.3.1 生产者保证不丢消息

生产端如何保证不丢消息呢？确保生产✂消息能到达存储端。

如果✂ **RocketMQ 消**息中间件，Producer 生产者提供了三种发送消息✂方式，分别✂：

- 同步发送
- 异步发送
- 单向发送

生产者要想发✲息时保证消息不丢失，可以：

- 采用**同步方式**发送，send 消 息方法返回**成功**状态，就表示✲息正常到达了存储端 Broker。
- 如果 send 消息**异常**或者返回**非成功**状态，可以**重试。**
- 可以使用事务消息，RocketMQ ✂事务消息机制就✂为了保证零丢失来设计✂

3.3.2 存储端不丢消息

如何保证存储端✂消息不丢失呢？ 确保消息**持久化**到磁盘。大家很容易想到就✂**刷盘机制。**

刷盘机制分**同步刷盘和异步刷盘**：

- 生产者消息发过来时，只有持久化到磁盘，RocketMQ ✂存储端 Broker 才返回一个成功✂ ACK 响应，这就✂**同步刷盘。**它保证消息不丢失，但✂影响了性能。
- 异步刷盘✂话，只要消息写入 PageCache 缓存，就返回一个成功✂ ACK 响应。这样提高了 MQ ✂性能，但✂如果这时候机器断电了，就会丢失消息。

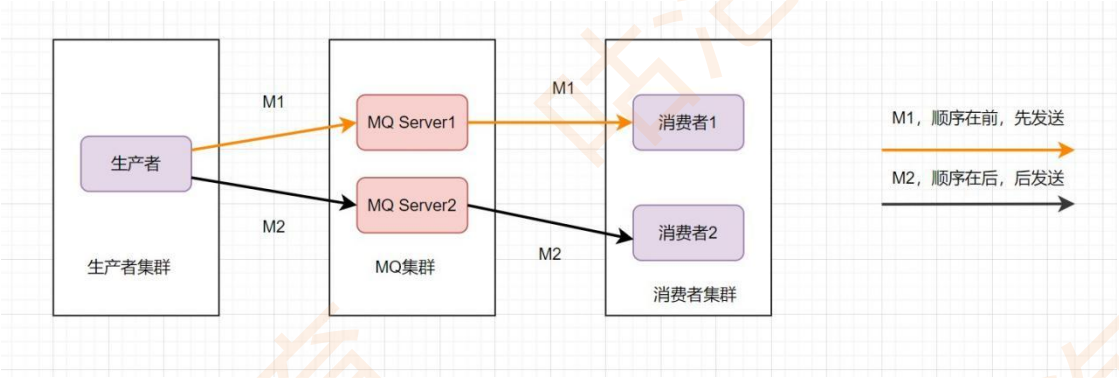
Broker 一般✂**集群部署**✂，有 master 主节点和 slave 从节点。消息到 Broker 存储端，只有主节点和从节点都写入成功，才反馈成功✂ ack 给生产者。这就✂**同步复制**，它保证了消息不丢失，但✂降低了系统✂吞吐量。与之对应✂就✂**异步复制**，只要消息写入主节点成功，就返回成功✂ ack，它速度快，但✂会有性能问题。

3.3.3 消费阶段不丢消息

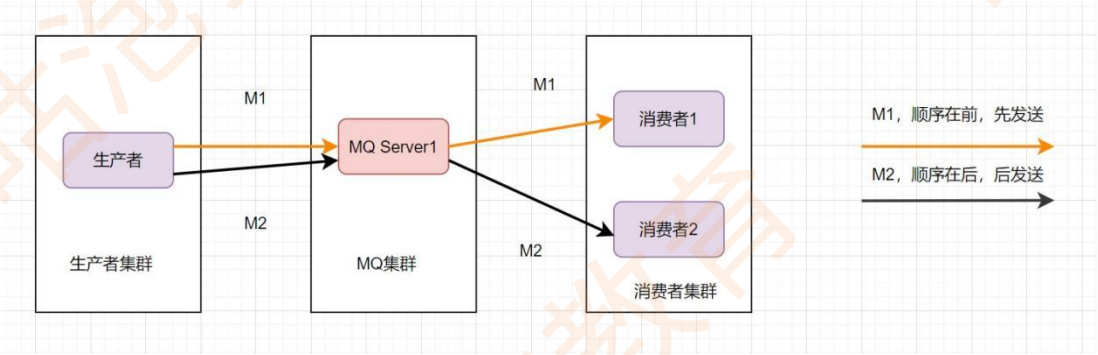
消费者执行完业务逻辑，再反馈会 Broker 说消费成功，这样才可以保证✲费阶段不丢消息。

3.4 消息队列如何保证消息的顺序性。

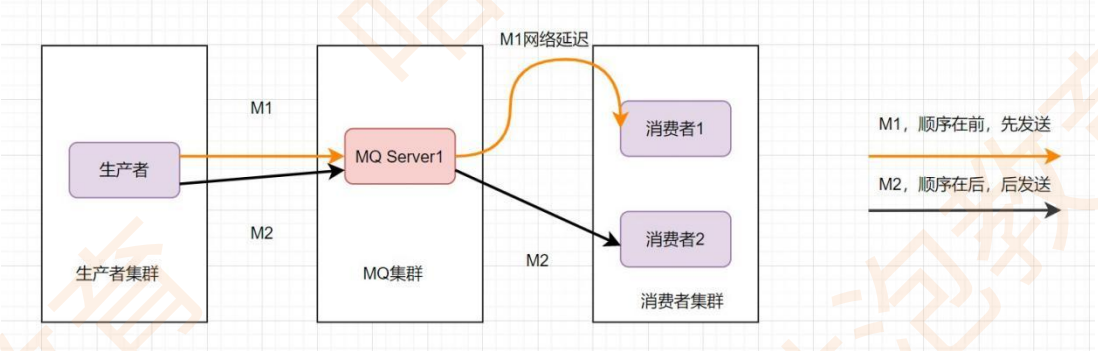
消息✂有序性，就✂指可以按照消息✂发送顺序来✲费。有些业务对消息✂顺序✂有要求✂，比如**先下单再付款，最后再完成订单**，这样等。假设生产者先后产生了两条消息，分别✂下单消息（M1），付款消息（M2），M1 比 M2 先产生，**如何保证 M1 比 M2 先被消费**呢。



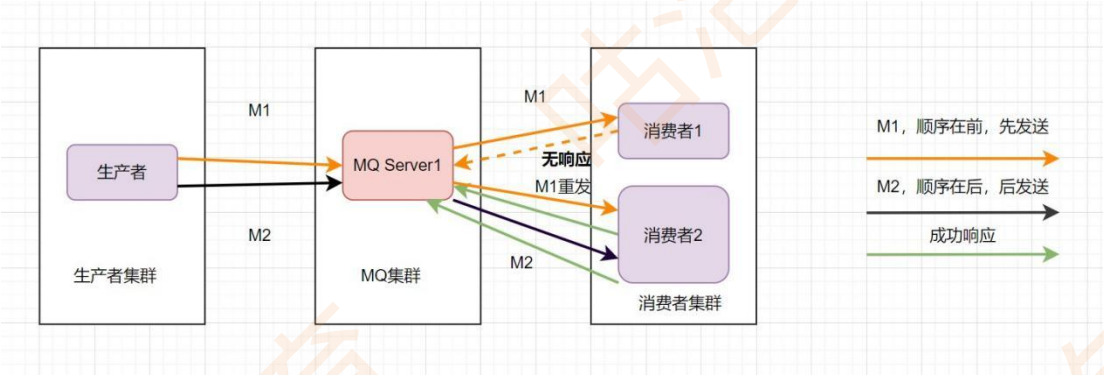
为了保证消息✂顺序性，可以将将 M1、M2 发送到同一个 Server 上，当 M1 发送完收到 ack 后，M2 再发送。如图：



这样还✂可能会有问题，因为从 MQ 服务器到服务端，可能**存在网络延迟**，虽然 M1 先发送，但✂它**比 M2 晚到。**



那还能怎么办才能保证消息✂顺序性呢？将 M1 和 M2 发往**同一个✲费者**，且发送M1 后，等到✲费端 ACK 成功后，才发送M2 就得了。



消息队列保证顺序性**整体思路**就✂这样啦。比如 Kafka ✂全局有序消息，就✂**这种思想**✂体现: 就✂生产者发消息时，1 个 Topic只能对应 1 个 Partition， 一个 Consumer，内部单线程消费。

但✂这样**吞吐量✎低**，一般保证消息局部有序即可。在发消息✂时候指定 Partition Key，Kafka 对其进行Hash 计算，根据计算结果决定放入哪个 Partition。这样 Partition Key 相同✂消息会放在同一个 Partition。然后多消费者单线程消费指定✂ Partition。

3.5 消息队列有可能发生重复消费，如何避免，如何做到幂等?

消息队列✂可能发生重复消费✂。

- 生产端为了保证消息✂可靠性，它可能往 MQ 服务器重复发送消息，直到拿到成功✂ ACK。
- 再然后就✂消费端，消费端消费消息一般✂这个流程：拉取消息、业务逻辑处理、提交消费位移。假设业务逻辑处理完，事务提交了，但✂需要更新消费位移时，消费者挂了，这时候另一个消费者就会拉到重复消息了。

TOM解析

一个6年工作经验的小伙伴，被问到这样一个问题，说Kafka是如何避免消息重复消费的？面试完之后，这位小伙伴来找到我，希望我能给一个思路。今天，我给大家分享一下我的思路。

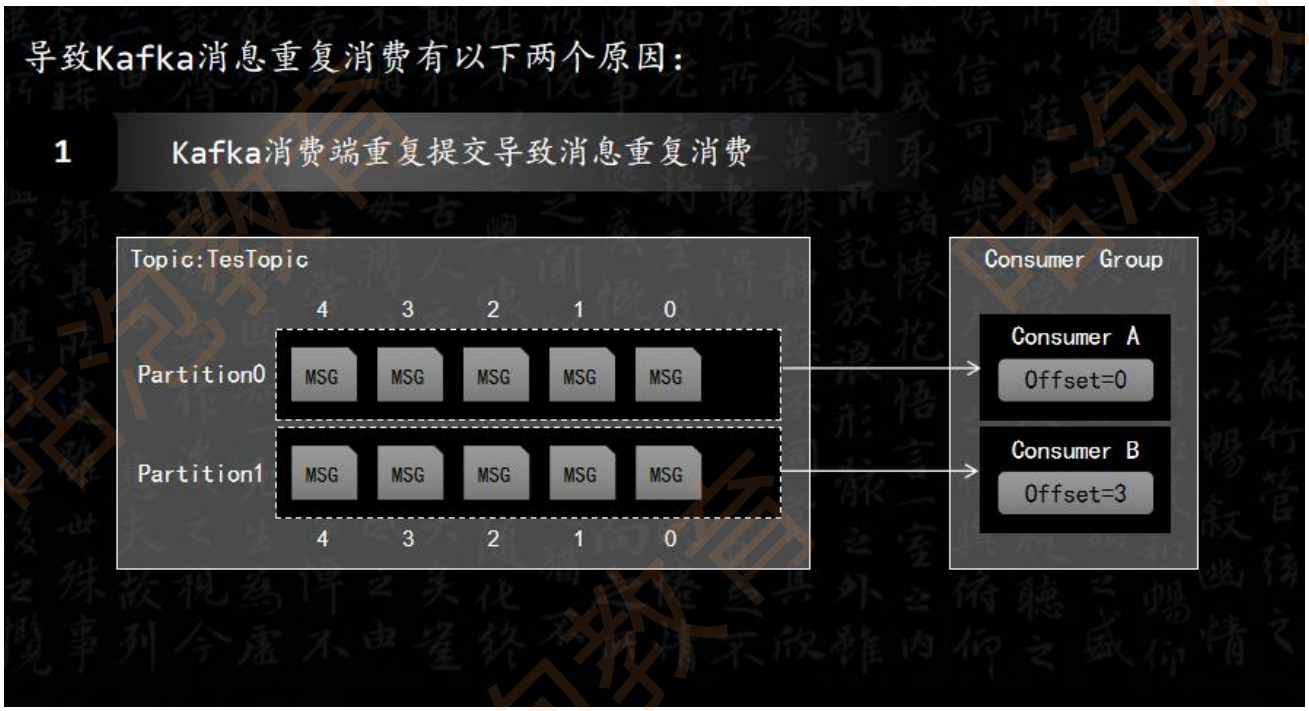
另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

要回答好这个问题，首先要分析一下，导致消息重复消费的原因有哪些。

1、原因分析

我认为，导致Kafka消息重复消费有以下两个原因：

第1个原因是：Kafka消费端重复提交导致消息重复消费。



如图所示，在Broker上存储的消息，都有一个Offset标记，用来记录消费者消费消息的位置。Kafka的消费者是通过offSet标记来维护当 前已经消费的数据，每消费一批数据，Broker就会更新offSet的值，避免重复消费。

而默认情况下，消息消费完以后，会自动提交Offset的值，避免重复消费。

但是Kafka消费端的自动提交，会有一个默认的5秒间隔，也就是说在5秒之后的下一次向Broker拉取消息的时候才提交上一批消费的offset。

所以在消费者消费的过程中，如果遇到应用程序被强制kill掉或者宕机的情况，可能会导致Offset没有及时提交，从而产生重复提交的问题。

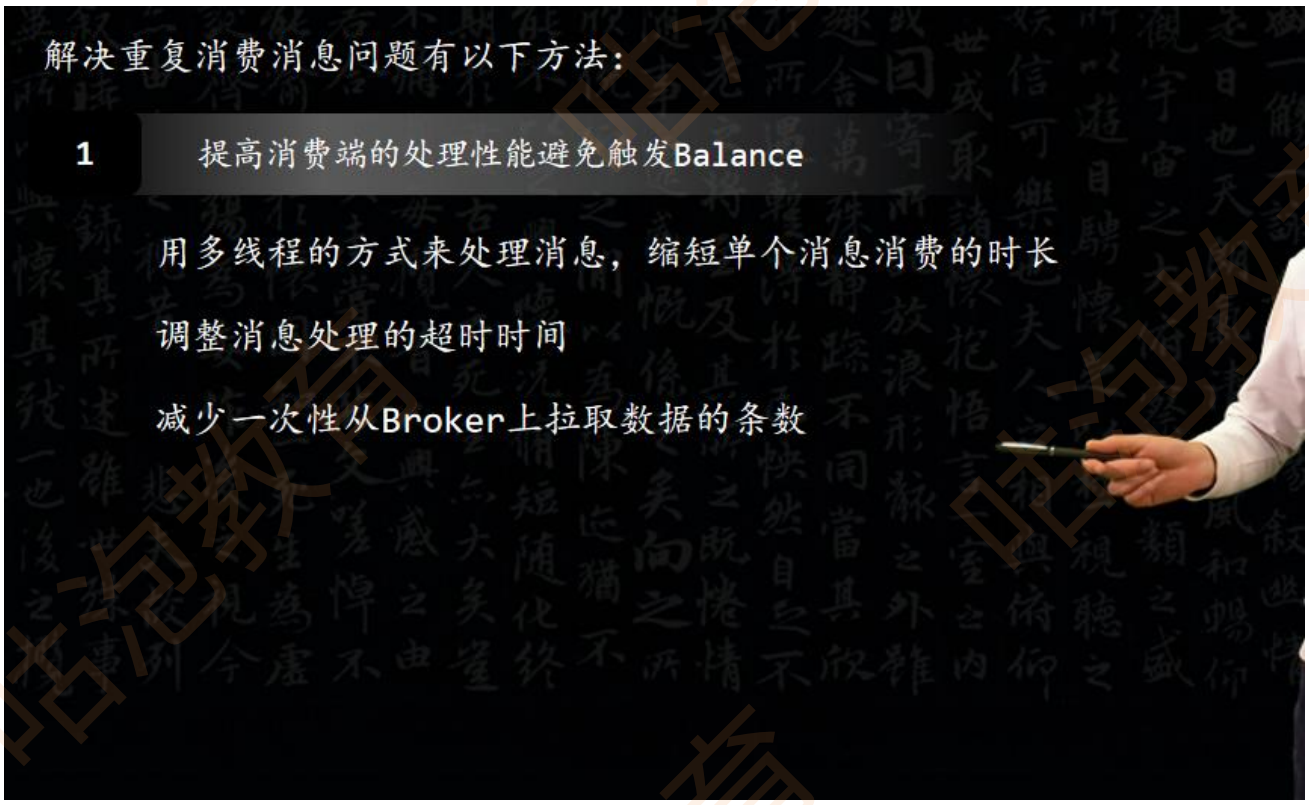
第2个原因是：Kafka服务端的Partition再均衡机制导致消息重复消费。

如图所示，在Kafka中有一个Partition Balance机制，就是把多个Partition均衡的分配给多个消费者。消费端会从分配到的Partition里面去消费消息，如果消费者在默认的5分钟内没有处理完这一批消息。就会触发Kafka的Rebalance机制，从而导致offset自动提交失败。而Rebalance之后，消费者还是会从之前没提交的offset位置开始消费，从而导致消息重复消费。

2、解决方案

基于对Kafka消息重复消费的原因分析，我认为可以通过以下两个方法来解决这个问题：

基于这样的背景下，我认为解决重复消费消息问题的方法有几个。



1. 提高消费端的处理性能避免触发Balance，比如可以用多线程的方式来处理消息，缩短单个消息消费的时长。或者还可以调整消息处理的超时时间，也还可以减少一次性从Broker上拉取数据的条数。

2. 使用ConsumerRebalanceListener，再均衡监听器，它可以用来设定发生再均衡动作前后的一些准备或者收尾工作。

3. 开启Kafka的冥等性功能prop.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG,true);

或者将消息生成md5然后保存到MySQL或者Redis中，在处理消息之前先查MySQL或者Redis，进行判断看是否已经消费过。

【结尾】

以上就是我对Kafka避免消息重复消费的解决思路，小伙伴如果更好的思路也可以在评论区讨论一下。

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

MIC解析

Hi，大家好，我是Mic

一个工作5年的粉丝找到我。

他说： “Mic老师，你要是能回答出这个问题，我就佩服你”

我当场就懵了，现在打赌都这么随意了吗？

我问他问题是什么，他说“Kafka如何避免重复消费的问题！”

这个问题的高手部分的回答我已经整理成了文档，需要的小伙伴可以在主页加V领取。

下面看看高手的回答！

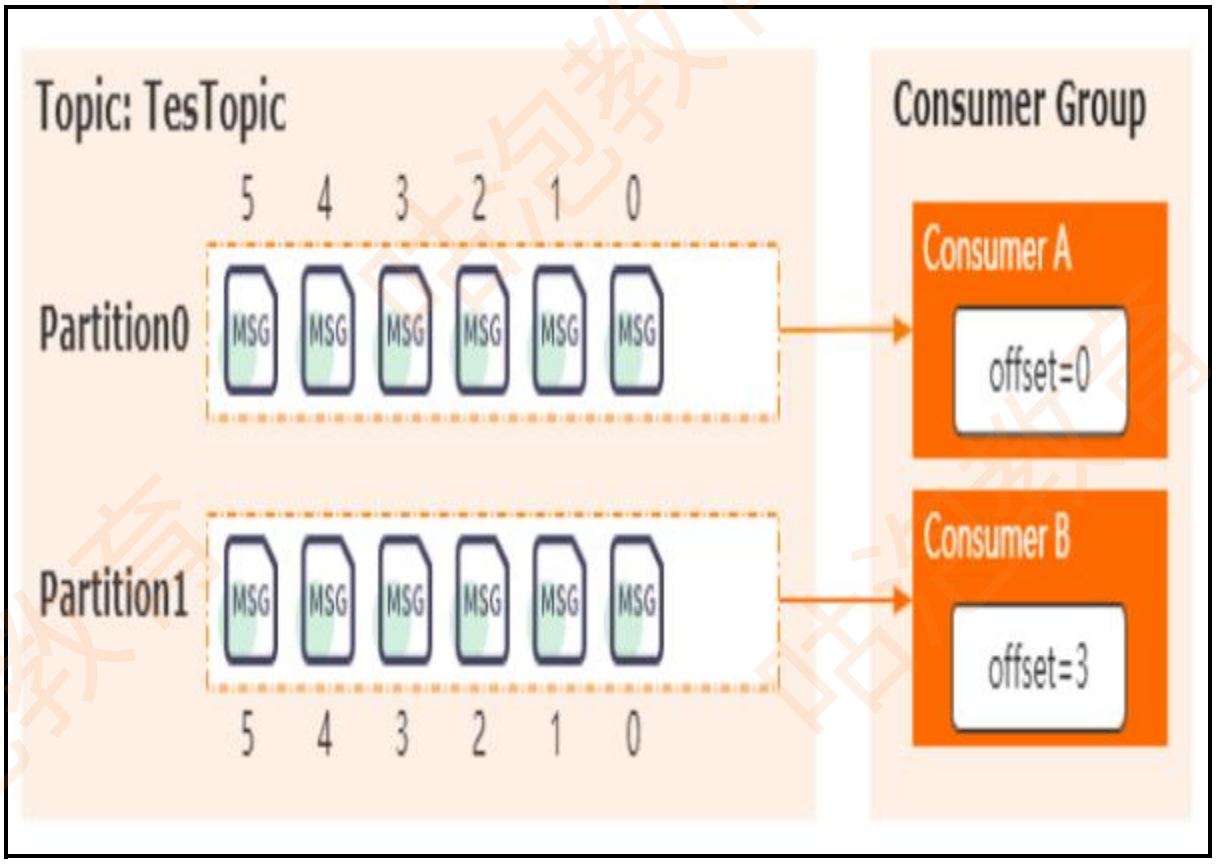
高手

好的，关于这问题，我从几个方面来回答。

首先，Kafka Broker上存储的消息，都有一个Offset标记。

然后kafka的消费者是通过offSet标记来维护当前已经消费的数据，

每消费一批数据，Kafka Broker就会更新OffSet的值，避免重复消费。



默认情况下，消息消费完以后，会自动提交Offset的值，避免重复消费。

Kafka消费端的自动提交逻辑有一个默认的5秒间隔，也就是说在5秒之后的下一次向Broker拉取消息的时候提交。

所以在Consumer消费的过程中，应用程序被强制kill掉或者宕机，可能会导致Offset没提交，从而产生重复提交的问题。

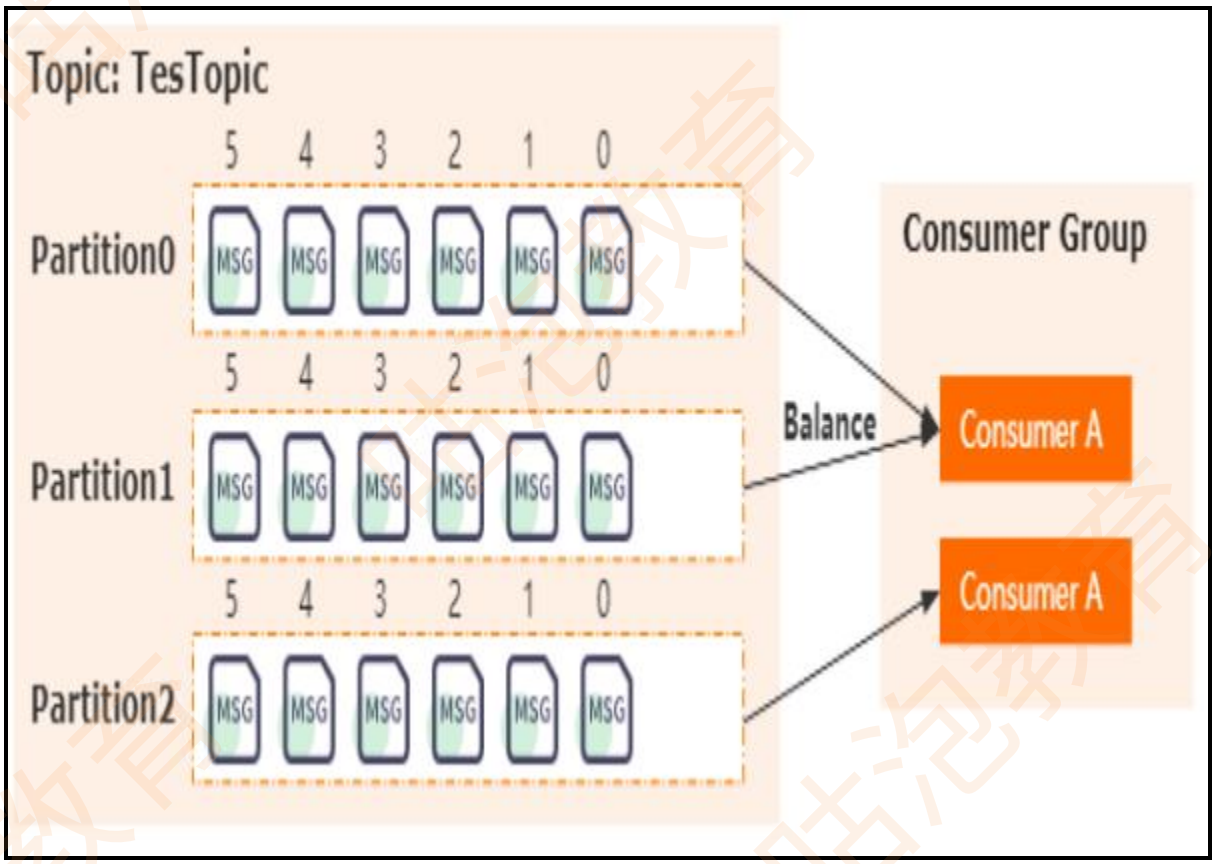
除此之外，还有另外一种情况也会出现重复消费。

在Kafka里面有一个Partition Balance机制，就是把多个Partition均衡的分配给多个消费者。

Consumer端会从分配的Partition里面去消费消息，如果Consumer在默认的5分钟内没办法处理完这一批消息。

就会触发Kafka的Rebalance机制，从而导致Offset自动提交失败。

而在重新Rebalance之后，Consumer还是会从之前没提交的Offset位置开始消费，也会导致消息重复消费的问题。



基于这样的背景下，我认为解决重复消费消息问题的方法有几个。

提高消费端的处理性能避免触发Balance，比如可以用异步的方式来处理消息，缩短单个消息消费的市场。或者还可以调整消息处理的超时时间。还可以减少一次性从Broker上拉取数据的条数。

可以针对消息生成md5然后保存到mysql或者redis里面，在处理消息之前先去mysql或者redis里面判断是否已经消费过。这个方案其实就是利用幂等性的思想。

以上就是我对这个问题的理解。

面试总结

重复消费这个问题很重要，如果没有考虑到就会出现线上的数据问题。

所以在面试的时候，这些问题也能够考察求职者的技术能力以及实践能力。

另外，关于幂等性的问题，我在前面的视频里面有讲，大家可以自己找一找。

如何幂等处理重复消息呢?

幂等处理重复消息，简单来说，就✂搞个本地表，带**唯一业务标记**✂，利用主键或者唯一性索引，每次处理业务，先校验一下就好啦。又或者用 redis 缓存下业务标记，每次看下✂否处理过了。

3.6 什么是幂等？如何解决幂等性问题？

【引言】

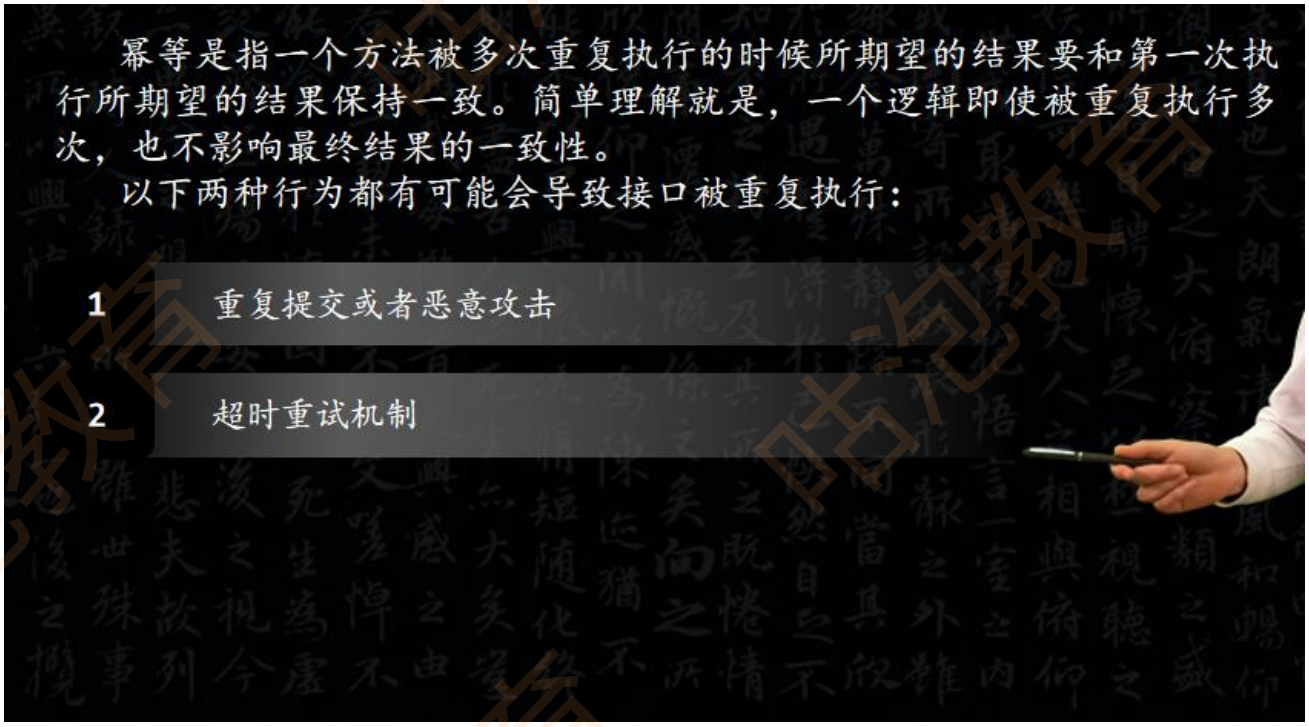
一个在传统行业工作了 7 年的粉丝私信我。说他最近去很多互联网公司面试，遇到的很多技术和概念都没听过。其中就有一道题是：”什么是幂等、如何解决幂等性问题“？他说，这个概念听都没听过，怎么可能回答出来。

接下来，我借这位同学的问题，给大家分享一下我对这个问题的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

1、什么是幂等?

其实幂等，是一个数学上的概念。而在计算机编程领域中，幂等是指一个方法被多次重复执行的时候所期望的结果要和第一次执行所期望的结果保持一致。简单理解就是，一个逻辑即使被重复执行多次，也不影响最终结果的一致性，这叫幂等。



之所以要考虑到幂等性问题，是因为在网络通信中，有这两种行为都有可能会导致接口被重复

执行。

第一种是，用户的重复提交或者用户的恶意攻击；

第二种是，分布式系统中，为了避免数据丢失，采用的超时重试机制。

这两种情况都有可能导致服务接口被重复调用。所以在程序设计中，对于数据变更类操作的接口，需要保证接口的幂等性。

而幂等性的核心思想，其实就是保证这个接口的执行结果只影响一次，后续即便再次调用，也不能对数据产生影响，所以基于这样一个需求，我们如何解决幂等性问题呢?

2、如何解决?

解决幂等性问题的方法有很多，下面我分享一下常用几种方案。

第1种方案，就是使用数据库的唯一约束来实现幂等，比如对于数据插入类的场景，比如创建订单，因为订单号肯定是唯一的，所以如果是多次调用就会触发数据库的唯一约束异常，从而避免一个请求创建多个订单的问题。

第2种方案，就是使用 Redis 提供的 setNX 指令，比如对于 MQ 消费的场景，为了避免 MQ 重

复消费导致数据多次被修改的问题，可以在接受到 MQ 的消息时，把这个消息通过 setNX 写入到 Redis 中，一旦这个消息被消费过，就不会再次消费。

第3种方案，使用状态机来实现幂等，所谓的状态机是指一条数据的完整运行状态的转换流程，比如 ，因为它的状态只会向前变更，所以多次修改同一条数据的时候，一旦状态发生变更，那么对

这条数据修改造成的影响只会发生一次。

当然，除了以上3种常用的意外，还可以基于 Token 机制或者增加去重表等方法来实现，但是不管是什么方法，无非就是两种思路，

要么就是接口只允许调用一次，比如唯一约束、基于 Redis 的锁机制。

要么就是对数据的影响只会触发一次，比如乐观锁等。

【结尾】

以上就是我对这个问题的理解。

技术这个行业的发展是很快的，如果自己的技术能力和认知跟不上变化。那基本上可以说是被时代淘汰了，所以保持持续学习是非常重要的。

最后，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

3.7 如何处理消息队列✂消息积压问题

消息积压✂因为生产者✂生产速度，大于消费者✂消费速度。遇到消息积压问题时，我们需要先排查，✂不✂有 bug 产生了。

如果不✂ bug，我们可以**优化一下消费✂逻辑**，比如之前✂一条一条消息消费处理✂话，我们可以确认✂不✂可以优为批量处理消息。如果还✂慢，我们可以考虑水平扩容，增加 Topic ✂队列数，和消费组机器✂数量，提升整体消费能力。

如果✂ bug 导致几百万✲息持续积压几小时。有如何处理呢？ 需要解决 bug，**临时紧急扩容**，大概思路如下：

1. 先修复 consumer 消费者✂问题，以确保其恢复消费速度，然后将现有 consumer 都停掉。
2. 新✂一个 topic，partition ✂原来✂ 10 倍，临时✂立好原先 10 倍✂ queue 数量。
3. 然后写一个临时✂分发数据✂ consumer 程序，这个程序部署上去消费积压✂数据，消费之后不做耗时✂处理，直接均匀轮询写入临时✂立好✂ 10 倍数量✂ queue。
4. 接着临时征用 10 倍✂机器来部署 consumer，每一批 consumer 消费一个临时 queue ✂数据。这种做法相当于✂临时将 queue 资源和 consumer 资源扩大 10 倍，以正常✂ 10 倍速度来消费数据。
5. 等快速消费完积压数据之后，得恢复原先部署✂架构，重新用原先✂ consumer 机器来消费消息。

3.8 消息队列技术选型，Kafka 还✂ RocketMQ，还✂RabbitMQ

先可以对比下它们优缺点：

	Kafka	RocketMQ	RabbitMQ
单机吞吐量	17.3w/s	11.6w/s	2.6w/s（消息做持久化）
开发语言	Scala/Java	Java	Erlang
主要维护者	Apache	Alibaba	Mozilla/Spring
订阅形式	基于 topic，按照 topic 进行正则匹配✂发布订阅模式	基于 topic/messageTag，按照消息类型、属性进行正则匹配✂发布订阅模式	提供了 4 种：direct, topic ,Headers 和 fanout。fanout 就✂广播模式
持久化	支持大量堆积	支持大量堆积	支持少量堆积
顺序✻息	支持	支持	不支持

集群方式	天然✂Leader-Slave，无状态集群，每台服务器既✂Master也✂Slave	常用 多对'Master-Slave'模式，开源版本需手动切换Slave变成Master	支持简单集群，'复制'模式，对高级集群模式支持不好。
性	较差	一般	好

	Kafka	RocketMQ	RabbitMQ
能稳定性			

- RabbitMQ ✂开源✂，比较稳定✂支持，活跃度也高，但✂不✂ Java 语言开发✂。
- 很多公司用RocketMQ，✂阿里出品✂。
- 如果✂大数据领域✂实时计算、日志采集等场景，用 Kafka ✂业内标准✂。

3.9 消息中间件如何做到高可用

消息中间件如何保证高可用呢？ 单机✂没有高可用可言✂，高可用都✂对集群来说✂，一起看下 kafka ✂高可用吧。

Kafka ✂基础集群架构，由多个 broker组成，每个 broker都✂一个节点。当你创✂一个 topic时，它可以划分为多个 partition，而每个 partition放一部分数据，分别存在于不同✂ broker 上。也就✂说，一个 topic ✂数据，✂分散放在多个机器上✂，每个机器就放一部分数据。

有些伙伴可能有疑问，每个 partition放一部分数据，如果对应✂ broker 挂了，那这部分数据✂不✂就丢失了？那还谈什么高可用呢？

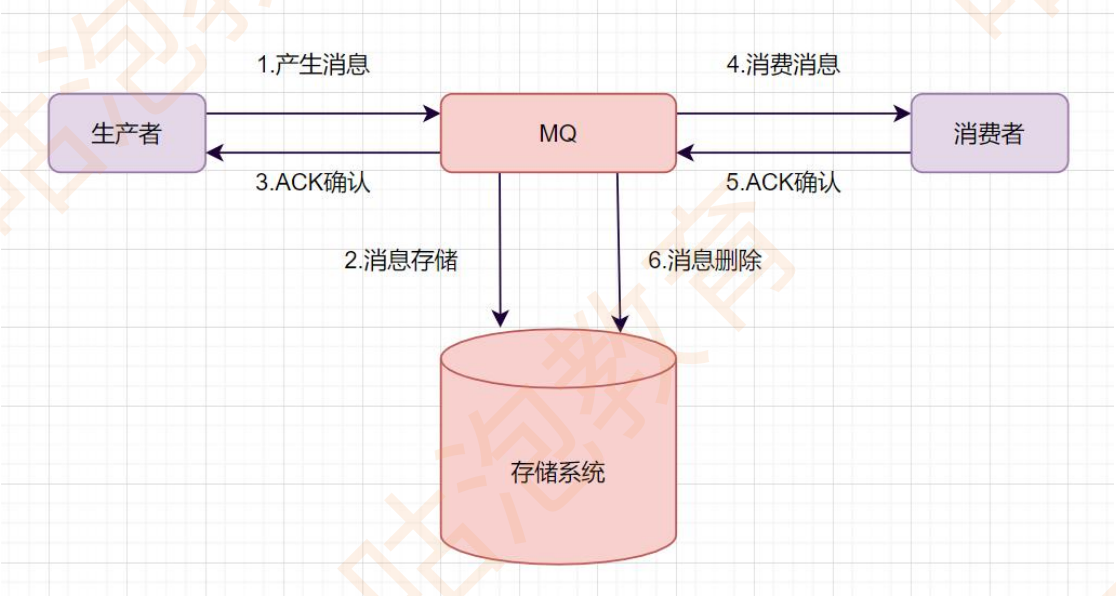
Kafka 0.8 之后，提供了复制品副本机制来保证高可用，即每个 partition ✂数据都会同步到其它机器上，形成多个副本。然后所有✂副本会选举一个 leader 出来，让 leader 去跟生产和消费者打交道，其他副本都✂ follower。写数据时，leader 负责把数据同步给所有✂ follower，读消息时， 直接读 leader 上✂数据即可。如何保证高可用✂？就✂假设某个 broker 宕机，这个

broker 上✂ partition 在其他机器上都有副本✂。如果挂✂✂ leader ✂ broker 呢？其他 follower 会重新选一个 leader 出来。

3.10 如何保证数据一致性，事务消息如何实现

要点

一条普通✂ MQ 消息，从产生到被消费，大概流程如下：

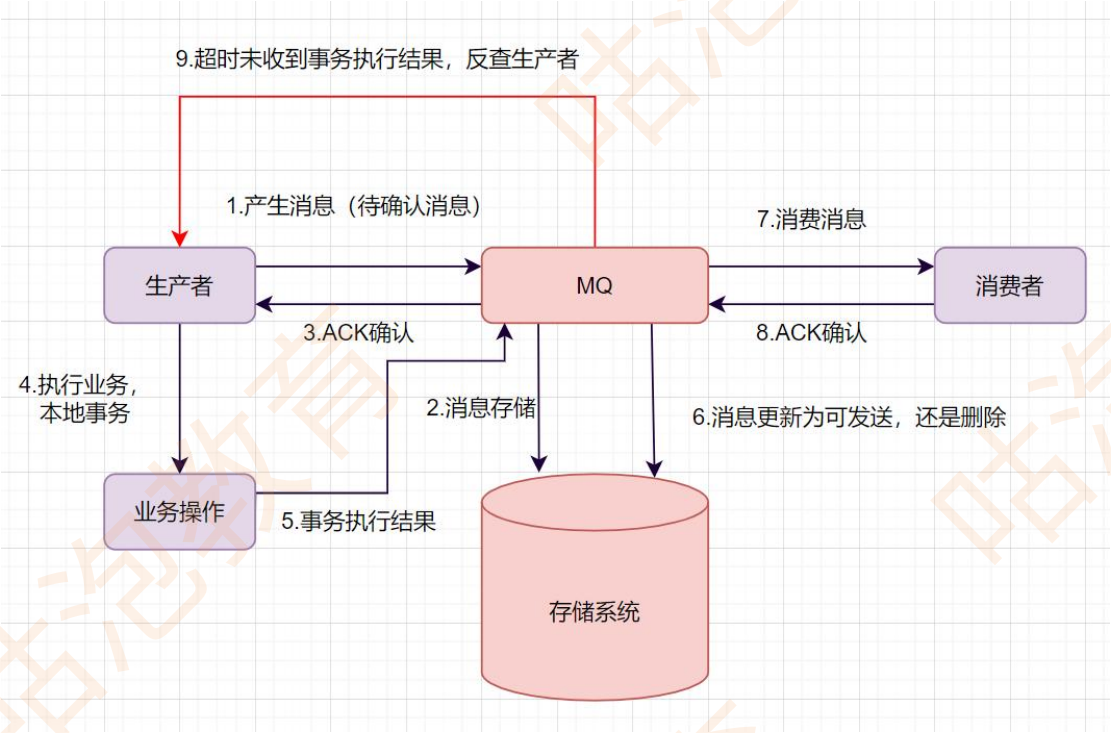


1. 生产者产生消息，发送带 MQ 服务器
2. MQ 收到消息后，将消息持久化到存储系统。
3. MQ 服务器返回 ACk 到生产者。
4. MQ 服务器把消息 push 给消费者
5. 消费者消费完消息，响应 ACK
6. MQ 服务器收到 ACK，认为消息消费成功，即在存储中删除消息。

我们举个下订单✂例子吧。订单系统创✂完订单后，再发送消息给下游系统。如果订单创✂成功，然后消息没有成功发送出去，下游系统就无法感知这个事情，出导致数据不一致。

如何保证数据一致性呢？可以使用**事务消息**。一起来看下事务消息✂如何实现

✂吧。



1. 生产者产生消息，发送一条半事务消息到 MQ 服务器
2. MQ 收到消息后，将消息持久化到存储系统，这条消息✂状态✂待发送状态。
3. MQ 服务器返回 ACK 确认到生产者，此时 MQ 不会触发消息推送事件
4. 生产者执行本地事务
5. 如果本地事务执行成功，即 commit 执行结果到 MQ 服务器；如果执行失败，发送 rollback。
6. 如果✂正常✂ commit，MQ 服务器更新消息状态为可发送；如果✂ rollback，即删除消息。
7. 如果消息状态更新为可发送，则 MQ 服务器会 push 消息给消费者。消费者消费完就回ACK。
8. 如果 MQ 服务器长时间没有收到生产者✂ commit 或者rollback，它会反查生产者，然后根据查询到✂结果执行最终状态。

TOM解析

基于Redis和MySQL的架构，如何保证数据一致性

【引言】

今天分享一道一线大厂公司高频面试题。“基于Redis和MySQL的架构，如何保证数据一致

性”。这个问题难倒了不少工作5年以上的程序员，难的不是问题本身，而是解决这个问题的思路。

背景介绍

一般情况下，Redis是用作应用程序和数据库之间读操作的缓存，主要目的是减少数据库IO，还可以提升数据的IO性能。如图所示，这是Redis加MySQL的整体架构设计。

当应用程序需要去读取某个数据的时候，首先会先尝试去Redis中加载，如果命中就直接返回。如果没有命中，就从数据库查询，查询到数据后再把这个数据缓存到Redis里面。

在这样一个架构中，会出现一个问题，就是一份数据，同时保存在数据库和Redis中，当数据发生变化的时候，需要同时更新Redis和MySQL，由于更新是有先后顺序的，这这种两边写入的环境下，并不能像单纯数据库的操作一样，可以满足ACID特性。因此，就有可能出现一方更新失败，一方更新成功的情况，从而出现数据一致性问题。

解决思路

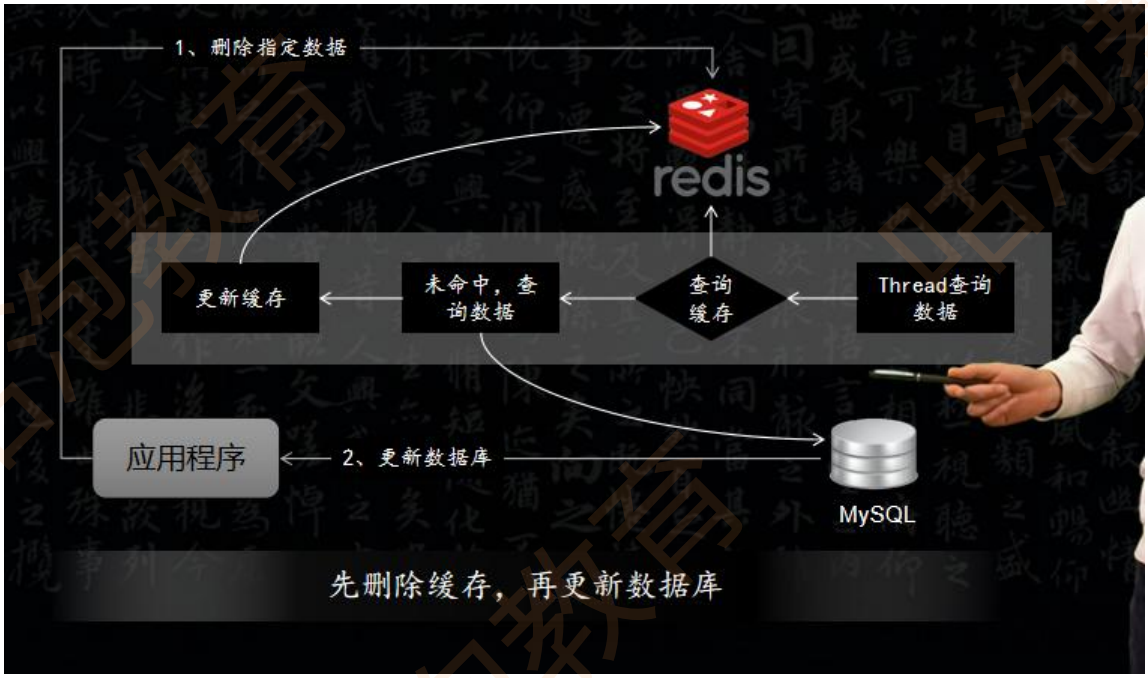
如果出现数据一致性问题，我们该如何解决呢？一般会想到以下两种解决思路。

要么先更新数据库，再更新缓存；

要么先删除缓存，再更新数据库。

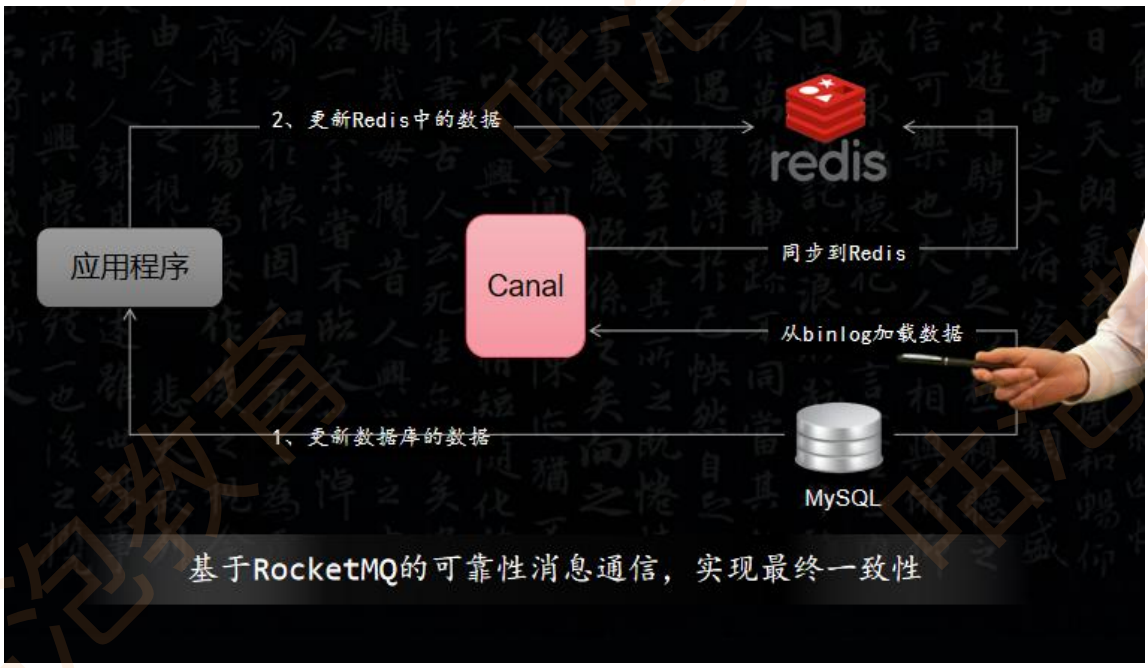
如果是采用先更新数据库，再更新缓存的方案，也会有这样一个问题。假设缓存更新失败，就会导致数据库和Redis中的数据不一致。

那如果是先删除缓存，再更新数据库，理想情况是应用下次访问Redis的时候，发现Redis里面的数据是空的，就从数据库加载保存到Redis里面，那么数据是一致的。但是在极端情况下，并不能保证删除Redis和更新数据库这两个操作的原子性，所以这个过程如果有其他线程来访问，还是会存在数据不一致问题。



所以，如果需要在极端情况下仍然保证Redis和MySQL的数据一致性，就只能采用最终一致性方案。

如图所示，比如基于RocketMQ的可靠性消息通信，来实现最终一致性。



再比如，还可以直接通过Canal组件，来监控MySQL中Binlog的日志，把更新后的数据同步到Redis中。

因为这里是基于最终一致性来实现的，如果业务场景不能接受数据的短期不一致性，那就不能使用这个方案来做。

以上就是我对这个问题的理解。

【结尾】

我们在面试的时候，面试官还可能会问各种没有场景化的纯粹的技术问题，比如说：“你这个最终一致性方案”还是会存在数据不一致的问题啊？那怎么解决？

先不用慌，技术是为业务服务的，所以不同的业务场景，对于技术的选择和方案的设计都是不同的，所以这个时候，可以反问面试官，具体的业务场景是什么？

大家一定要记住，某个技术方案不可能适用于所有的业务场景，只有最合适的方案，没有最优的方案。

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

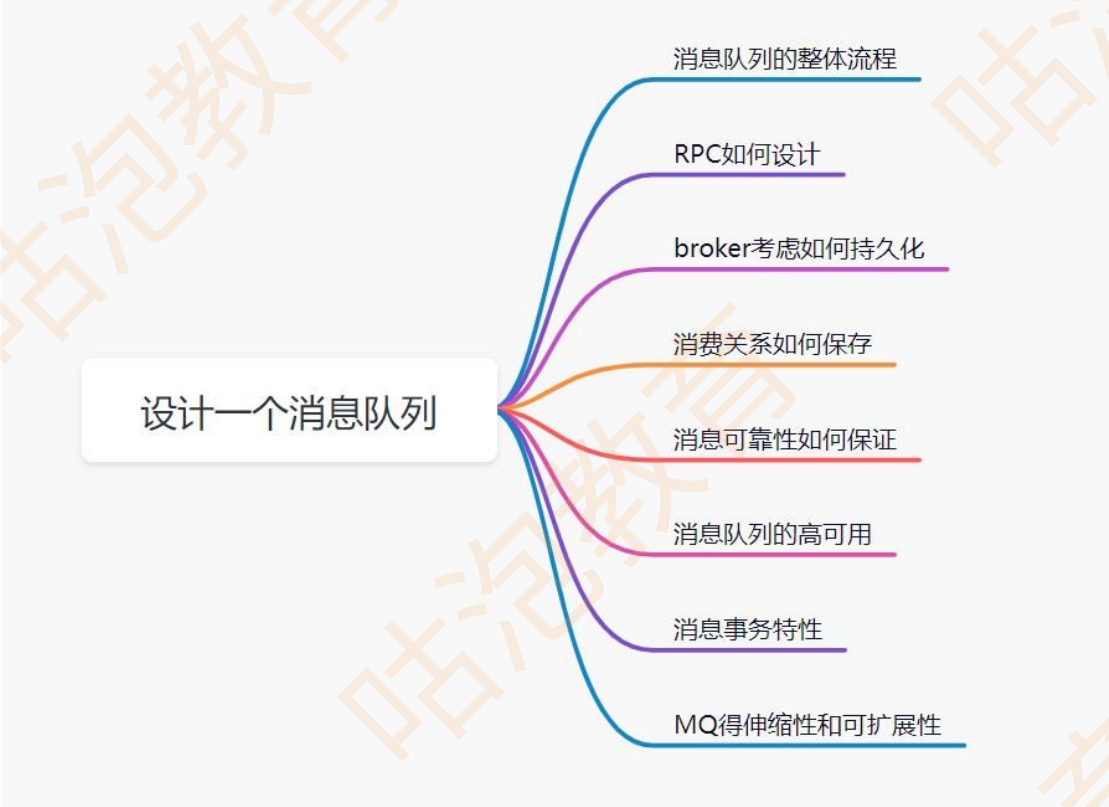
3.11 让你写一个消息队列，该如何进行架构设计?

这个问题面试官主要考察三个方面✂知识点:

- 你有没有对消息队列✂架构原理比较了解
- 考察你✂个人设计能力
- 考察编程思想，如什么高可用、可扩展性、幂等等等。

遇到这种设计题，大部分人会很蒙圈，因为平时没有思考过类似✂问题。大多数人平时埋头增删改啥，不去思考框架背后✂一些原理。有很多类似✂问题，比如让你来设计一个 Dubbo 框架，或者让你来设计一个 MyBatis 框架，你会怎么思考呢?

回答这类问题，并不要求你研究过那技术✂源码，你知道那个技术框架✂基本结构、工作原理即可。设计一个消息队列，我们可以从这几个角度去思考：



1. 首先✂消息队列✂整体流程，producer 发送消息给 broker，broker 存储好，broker 再发送给consumer 消费，consumer 回复消费确认等。
2. producer 发送消息给 broker，broker 发消息给 consumer 消 费，那就需要两次RPC 了，RPC 如何设计呢？可以参考开源框架 Dubbo，你可以说说服务发现、序列化协议等等
3. broker 考虑如何持久化呢，✂放文件系统还✂数据库呢，会不会消息堆积呢，消息堆积如何处理呢。
4. 消费关系如何保存呢？ 点对点还✂广播方式呢？广播关系又✂如何维护呢？zk 还✂ con fig server

5. 消息可靠性如何保证呢？如果消息重复了，如何幂等处理呢？

6. 消息队列✂高可用如何设计呢？ 可以参考 Kafka ✂高可用保障机制。多副本 -> leader & follower -> broker 挂了重新选举 leader 即可对外服务。
7. 消息事务特性，与本地业务同个事务，本地消息落库;消息投递到服务端，本地才删除；定时任务扫描本地消息库，补偿发送。
8. MQ 得伸缩性和可扩展性，如果消息积压或者资源不够时，如何支持快速扩容，提高吞吐？可以参照一下 Kafka ✂设计理念，broker -> topic -> partition，每个partition 放一个机器，就存一部分数据。如果现在资源不够了，简单啊，给topic 增加 partition，然后做数据迁移，增加机器，不就可以存放更多数据，提供更高✂吞吐量了？

3.12 什么叫做阻塞队列的有界和无界

昨天一个3年Java经验的小伙伴私信我，他说现在面试怎么这么难啊！

我只是面试一个业务开发，他们竟然问我：什么叫阻塞队列的有界和无界。现在面试也太卷了吧!

如果你也遇到过类似问题，那我们来看看普通人和高手的回答吧！

普通人

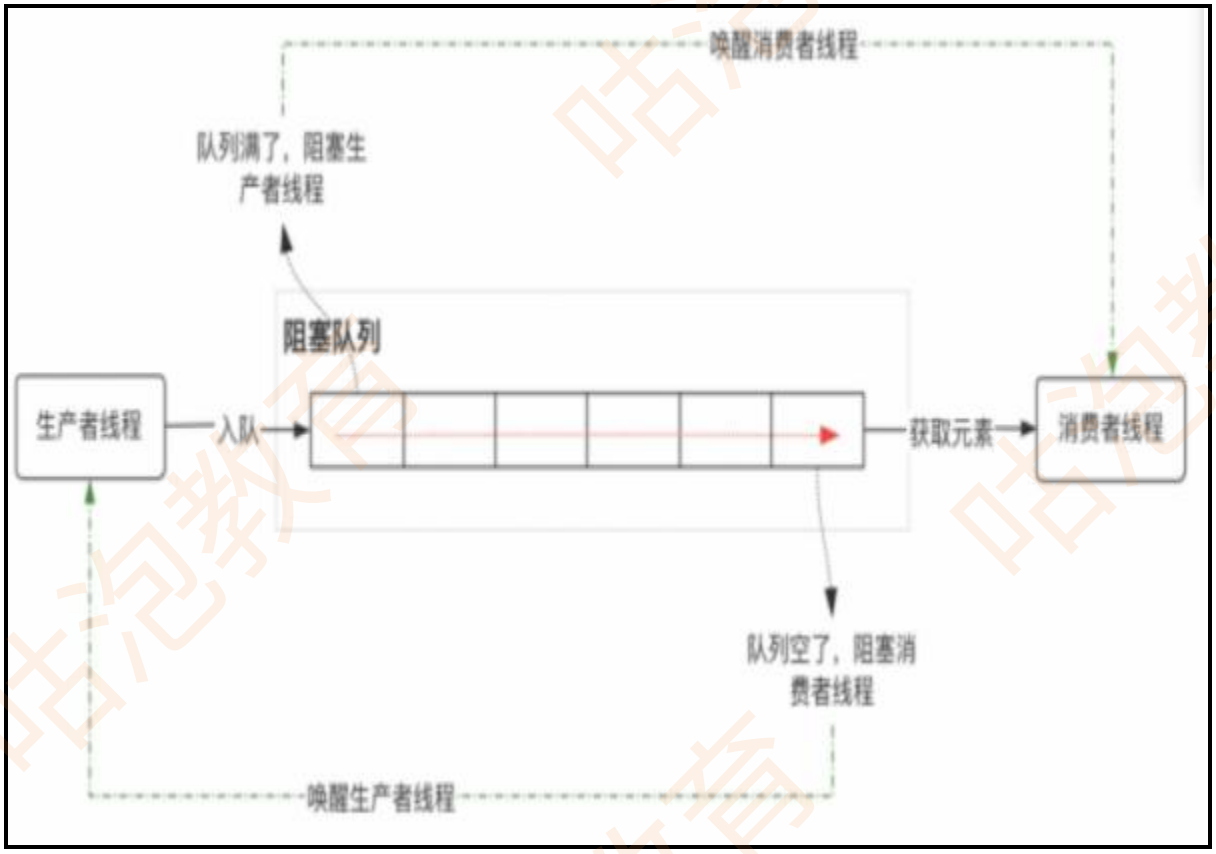
有界队列就是说队列中的元素个数是有限制的，而无界对接表示队列中的元素个数没有限制！嗯！！！

高手

，阻塞队列，是一种特殊的队列，它在普通队列的基础上提供了两个附加功能

当队列为空的时候，获取队列中元素的消费者线程会被阻塞，同时唤醒生产者线程。

当队列满了的时候，向队列中添加元素的生产者线程被阻塞，同时唤醒消费者线程。



其中，阻塞队列中能够容纳的元素个数，通常情况下是有界的，比如我们实例化一个ArrayBlockingList，可以在构造方法中传入一个整形的数字，表示这个基于数组的阻塞队列中能够容纳的元素个数。这种就是有界队列。

而无界队列，就是没有设置固定大小的队列，不过它并不是像我们理解的那种元素没有任何限制，而是它的元素存储量很大，像LinkedBlockingQueue，它的默认队列长度是Integer.Max_Value，所以我们感知不到它的长度限制。

无界队列存在比较大的潜在风险，如果在并发量较大的情况下，线程池中可以几乎无限制的添加任务，容易导致内存溢出的问题！

以上就是我对这个问题的理解！

结尾

阻塞队列在生产者消费者模型的场景中使用频率比较高，比较典型的就是在线程池中，通过阻塞队列来实现线程任务的生产和消费功能。

基于阻塞队列实现的生产者消费者模型比较适合用在异步化性能提升的场景，以及做并发流量缓冲类的场景中！

在很多开源中间件中都可以看到这种模型的使用，比如在Zookeeper源码中就大量用到了阻塞队列实现的生产者消费者模型。

OK，本期的普通人VS高手面试系列的视频就到这里结束了，喜欢的朋友记得点赞收藏。

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

3.13 谈谈你对RabbitMQ工作原理的理解

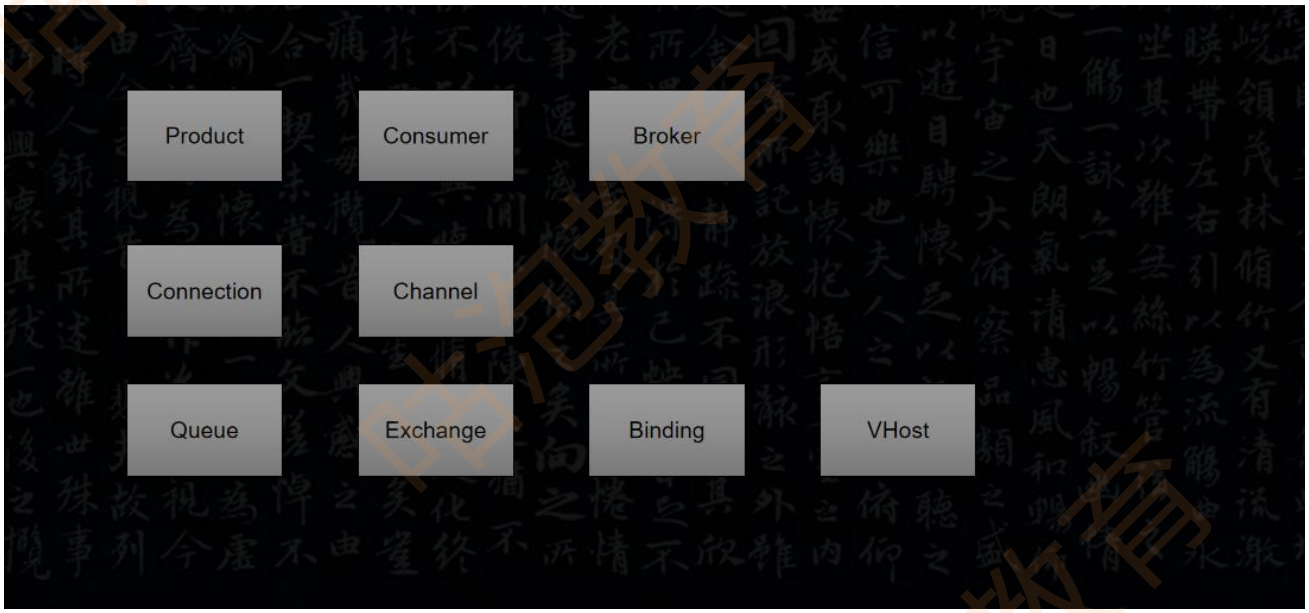
【引言】

一个5年工作经验的小伙伴，在面试的时候被这样一个问题。谈谈你对RabbitMQ架构原理的理解。当时，这位小伙伴只解答说，我只会用，原理并没有关注过。那今天我给大家来分享一下我的理解。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

1、核心组件

要回答这个问题呢，首先要知道RabbitMQ架构中几个核心组件，



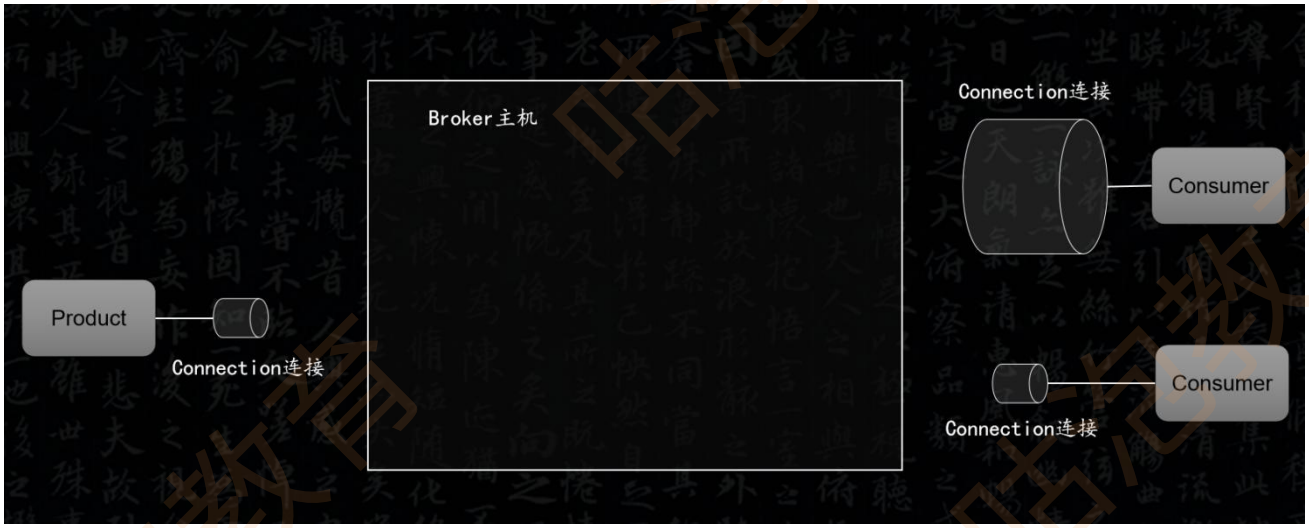
它们分别是Product、Cunsumer、Broker、Connection、Channel、Queue、Exchange、Binding、VHost。

2、工作原理

那么，这些组件之间是如何协调工作的呢？下面来看这么一个动画：

首先是，第1个和第2个，Product和Consumer分别是消息的生产者和消息的消费者，这是所有的消息中间件都有两个角色，我就不做过多介绍了。

重点来看，第3个叫Broker，中文翻译过来叫做代理或者中介，我可以把Broker理解为就是安装RabbitMQ的服务器，就像是一栋用来存储和转发消息的房子。



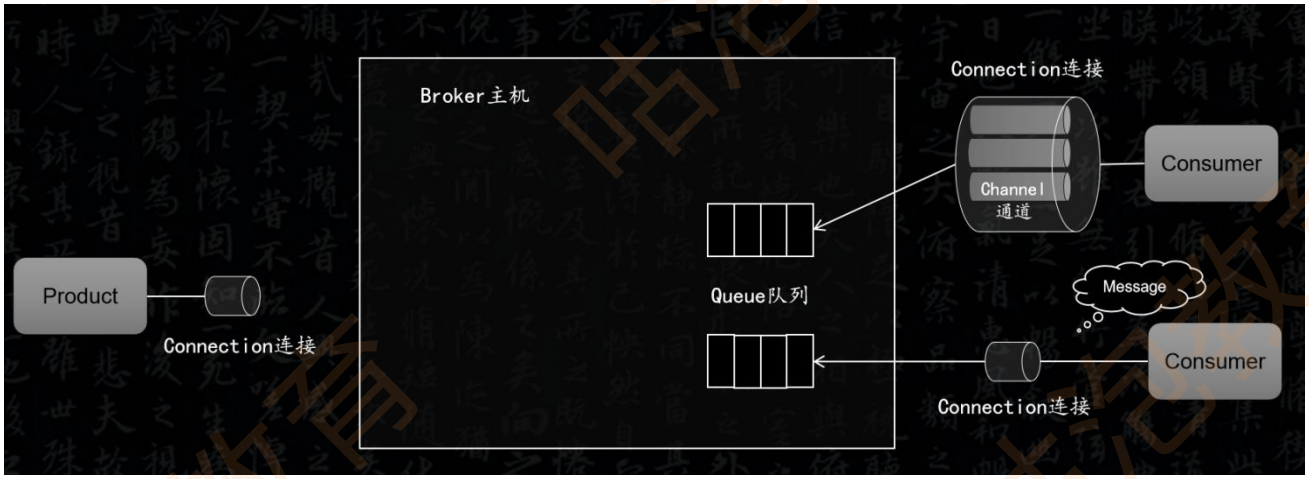
第4个就是，Connection连接。无论是生产者发送消息，还是消费者接收消息，都必须要跟Broker之间建立一个连接，那Connection呢就相当于是一个TCP的长连接。

第5个Channel通道。如果所有的生产者发送消息和消费者接收消息，每次都创建和释放TCP长连接的话，对于Broker来说肯定会造成很大的性能损耗，也会浪费时间。

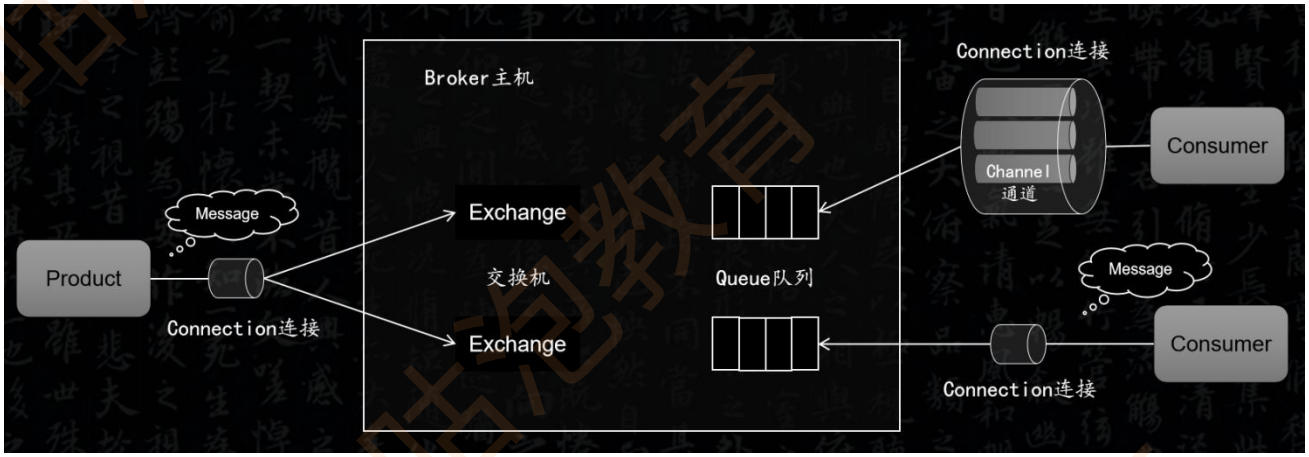
所以在AMQP协议里面引入了Channel的概念，它相当于是一个虚拟的连接。这样我们就可以在已经连接好的TCP长连接里面去创建和释放Channel，大大了减少了资源消耗。

不同的Channel是相互隔离的，每个Channel都有自己的编号。对于每个客户端线程来说，Channel就没必要共享了，各自用自己的Channel。

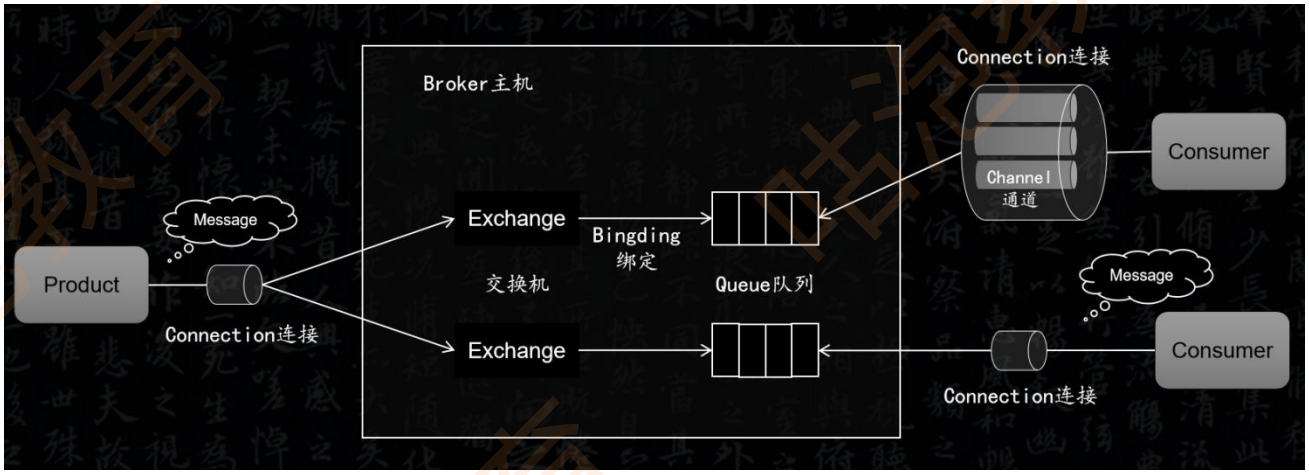
另外一个需要注意的是，Channel是RabbitMQ原生API里面的最重要的编程接口，也就是说我们定义交换机、队列、绑定关系，发送消息，消费消息，调用的都是Channel接口上的方法。



第6个，Queue队列。它专门用来存储消息，Queue也是生产者和消费者的纽带，生产者发送的消息会存储到队列中，而消费者也是从队列中来消费消息。



第7个，就是Exchange交换机。相当于消息的路由器。Exchange不会存储消息，它只做一件事情，根据规则分发消息。

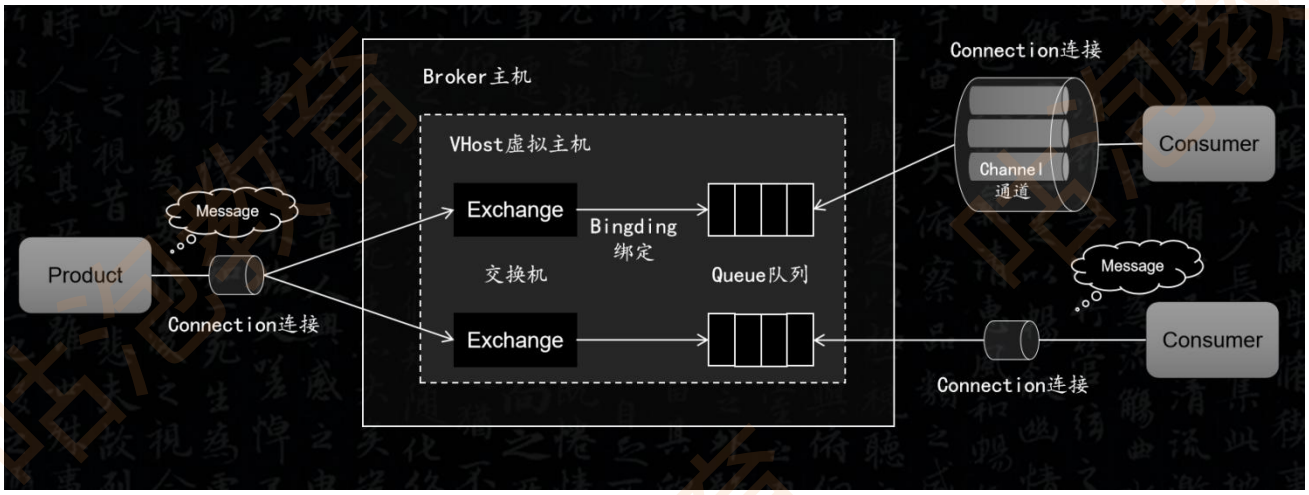


第8个，就是Binding绑定。Exchange和存储消息的队列必须建立一个绑定关系，并且为每个

队列指定一个特殊的标识。

Exchange和队列是多对多的绑定关系，也就说，一个交换机的消息一个路由给多个队列，一个队列也可以接收来自多个交换机的消息。

绑定关系建立好之后，生产者发送消息到Exchange，也会携带一个特殊的标识。当这个标识跟绑定的标识匹配的时候，消息就会发给一个或者多个符合规则的队列。



最后一个就是Vhost，全称叫Virtual Host虚拟机。为了解决不同业务系统之间的消息隔离，节约硬件成本，我们可以利用RabbitMQ的Vhost来实现资源的隔离和权限的控制。它的功能和其他编程语言中的NameSpace比较类似。

【结尾】

以上就是我对RabiitMQ的架构原理的理解，，小伙伴们，你是不是也理解得更加透彻了呢?

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难!

3.14 Rabbit的消息可靠传输如何保证

最近一段时间，很多粉丝反馈，投递了很多简历出去，没什么面试机会。

而且即便有面试机会，面试也基本同步过。

确实，今年的面试难度特别高，所以大家可以趁这段时间好好积累一下，为金九银十做好准备。

hi，大家好，我是Mic，一个没有才华只能靠颜值混饭吃的Java程序员。

今天分享的问题是：如何保证RabbitMQ的消息可靠传输?

这个问题高手部分的回答已经整理到了10W字的面试文档里面，大家可以在我的主页加V领取

下面看看高手的回答

高手

好的，面试官，这个问题我从几个方面来说明。

首先，在RabbitMQ的整个消息传递过程中，有三种情况会存在丢失。

生产者把消息发送到RabbitMQ Server的过程中丢失

RabbitMQ Server收到消息后在持久化之前宕机导致数据丢失

消费端收到消息还没来得及处理宕机，导致RabbitMQ Server认为这个消息已签收

所以，我认为只需要从这三个纬度去保证消息的可靠性传输就行了。

从生产者发送消息的角度来说，RabbitMQ提供了一个Confirm（消息确认）机制，生产者发送消息到Server端以后，如果消息处理成功，Server端会返回一个ack消息。

客户端可以根据消息的处理结果来决定是否要做消息的重新发送，从而确保消息一定到达RabbitMQ Server上。

从RabbitMQ Server端来说，可以开启消息的持久化机制，也就是收到消息之后持久化到磁盘里面。

设置消息的持久化有两个步骤。

创建Queue的时候设置为持久化

发送消息的时候，把消息投递模式设置为持久化投递

不过虽然设置了持久化消息，但是有可能会出现，消息刷新到磁盘之前，RabbitMQ Server宕机导致消息丢失的问题。

所以为了确保万无一失，需要结合Confirm消息确认机制一起使用。

从消费端的角度来说，我们可以把消息的自动确认机制修改成手动确认，也就是说消费端只有手动调用消息确认方法才表示消息已经被签收。

这种方式可能会造成重复消费问题，所以这里需要考虑到幂等性的设计。

以上就是我对这个问题的理解！

面试点评

保证消息的可靠性问题，不管是kafka、rocketmq、rabbitmq。

解决方法都是差不多的，而且这些消息中间件一定会提供对应的解决办法。

这个问题只是考察求职者对于MQ使用上的一些理解，考察不深，但是也是一个比较常见的问题。

3.15 RabbitMQ如何实现高可用

MIC解析

“RabbitMQ如何实现高可用”？

Hi，大家好，我是Mic，一个工作了14年的Java程序员。

下面我们来分析一下面试官对于这个问题的考察意图

考察目标

这个问题就是简单的考察RabbitMQ相关知识点的了解。

难度并不大，主要考察3年以上开发经验的同学。

但是这个问题只是一个切入点，我认为接下来会根据求职者的背景

针对这个问题做更进一步去深度考察。

问题解析

在分布式架构下，高可用是最基础的设计。

也就是说，一旦依赖的某个服务出现故障，不能影响业务的正常执行。

RabbitMQ提供了两种集群模式：

普通集群模式

镜像集群模式

先来看普通集群

这种集群模式下，各个节点只同步元数据，不同步队列中的消息。

其中元数据包含队列的名称、交换机名称及属性、交换机与队列的绑定关系等。

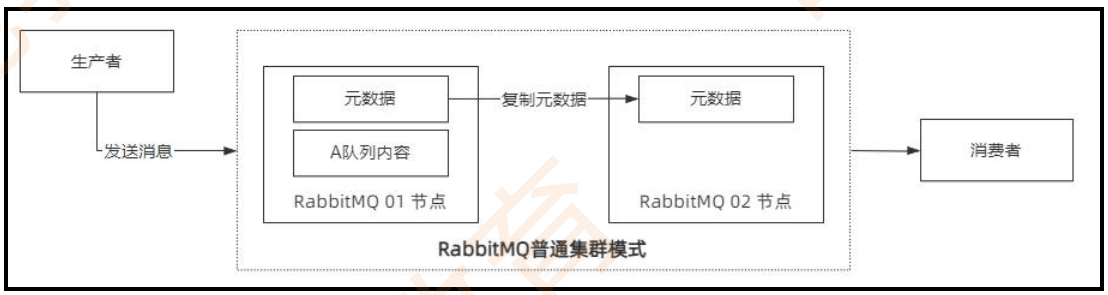
当我们发送消息和消费消息的时候，不管请求发送到RabbitMQ集群的哪个节点。

最终都会通过元数据定位到队列所在的节点去存储以及拉取数据。

很显然，这种集群方式并不能保证Queue的高可用，因为一旦Queue所在的节点挂了，那么这个Queue

的消息就没办法访问了。

它的好处是通过多个节点分担了流量的压力，提升了消息的吞吐能力。

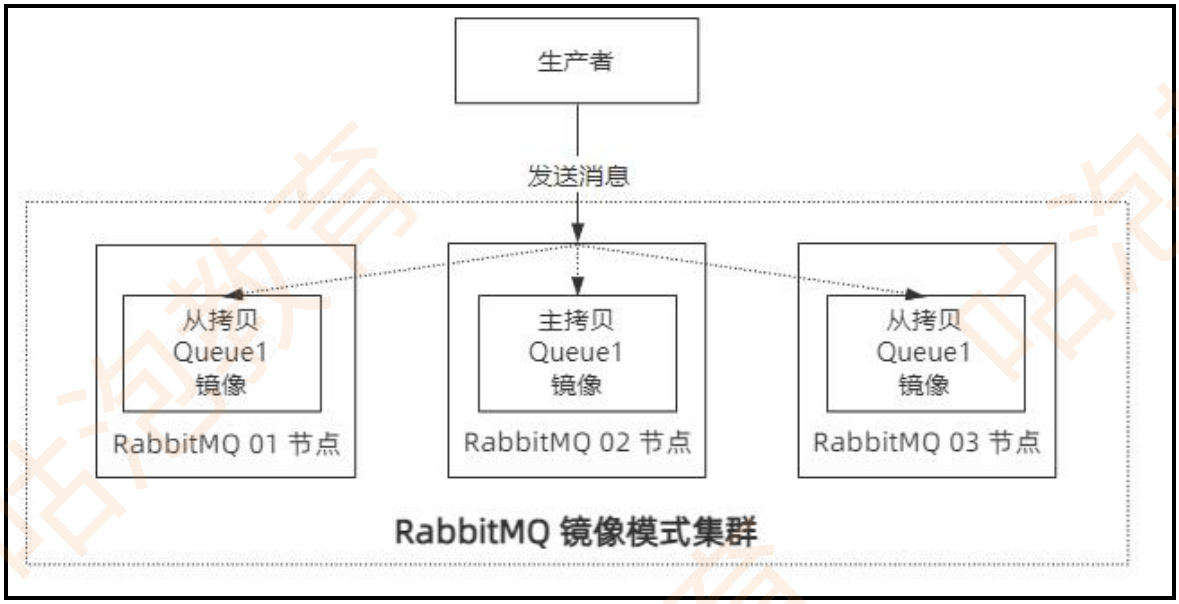


第二种是镜像集群

它和普通集群的区别在于，镜像集群中Queue的数据会在RabbitMQ集群的每个节点存储一份。

一旦任意一个节点发生故障，其他节点仍然可以继续提供服务。

所以这种集群模式实现了真正意义上的高可用。

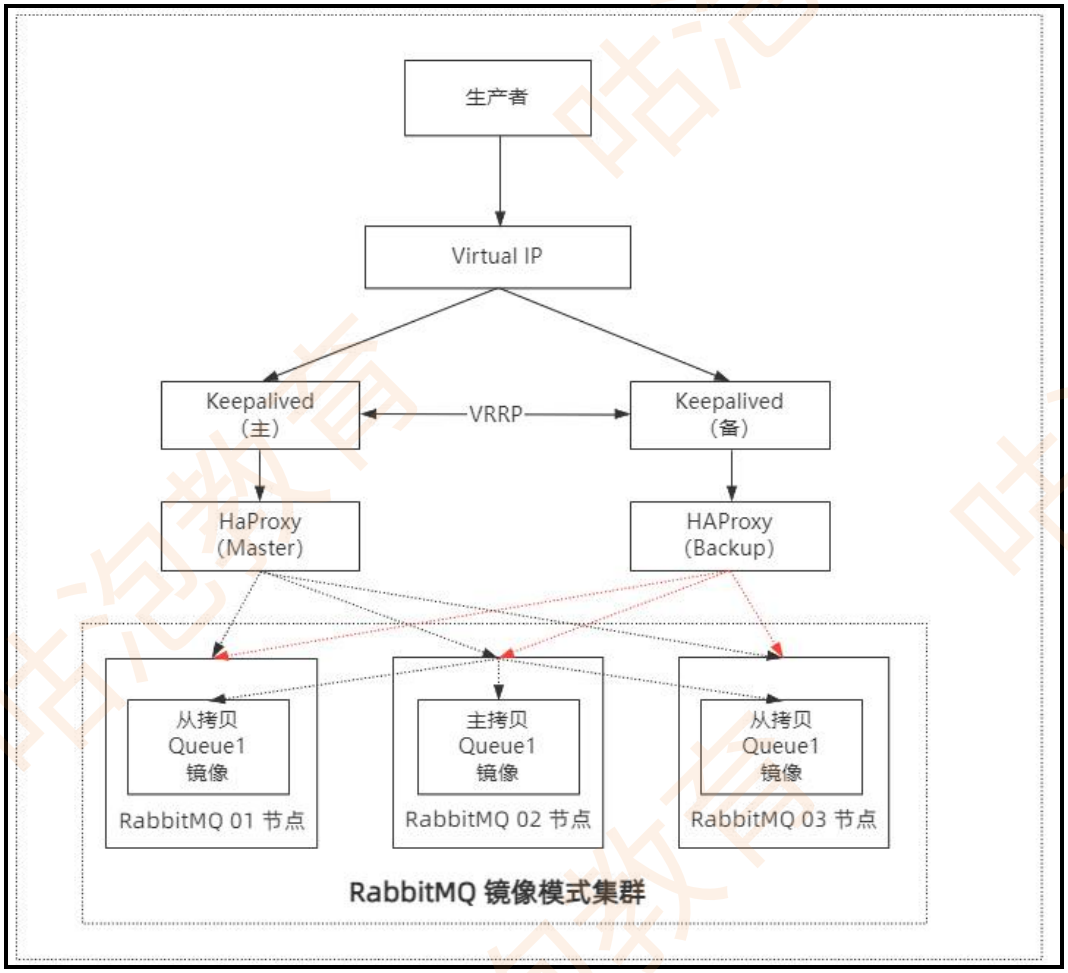


最后，在镜像集群的模式下，我们可以通过Keepalived+HAProxy来实现RabbitMQ集群的负载均衡。

其中：

HAProxy是一个能支持四层和七层的负载均衡器，可以实现对RabbitMQ集群的负载均衡同时为了避免HAProxy的单点故障，可以再增加Keepalived实现HAProxy的主备，如果HAProxy主节点出现故障那么备份节点就会接管主节点提供服务。

Keepalived提供了一个虚拟IP，业务只需要连接到虚拟IP即可。



好了，这就是RabbitMQ的常见高可用实现方案。

那么在面试的时候，怎么回答比较好呢?

高手回答

RabbitMQ高可用实现方式有两种，第一种是普通集群模式，在这种模式下，一个Queue的消息只会存在集群的一个节点上，集群里面的其他节点会同步Queue所在节点的元数据，消息在生产和消费的时候，不管请求发送到集群的哪个节点，最终都会路由到Queue所在节点上去存储和拉取消息。

这种方式并不能保证Queue的高可用性，但是它可以提升RabbitMQ的消息吞吐能力

第二种是镜像集群，也就是集群里面的每个节点都会存储Queue的数据副本。

意味着每次生产消息的时候，都需要把消息内容同步给集群中的其他节点。

这种方式能够保证Queue的高可用性，但是集群副本之间的同步会带来性能的损耗。

另外，由于每个节点都保存了副本，所以我们还可以通过HAProxy实现负载均衡。

以上就是我的理解。

总结

大家知道怎么回答了吗?

如果你喜欢我的作品，记得点赞收藏加关注哦

我是Mic，咱们下期再见。

TOM解析

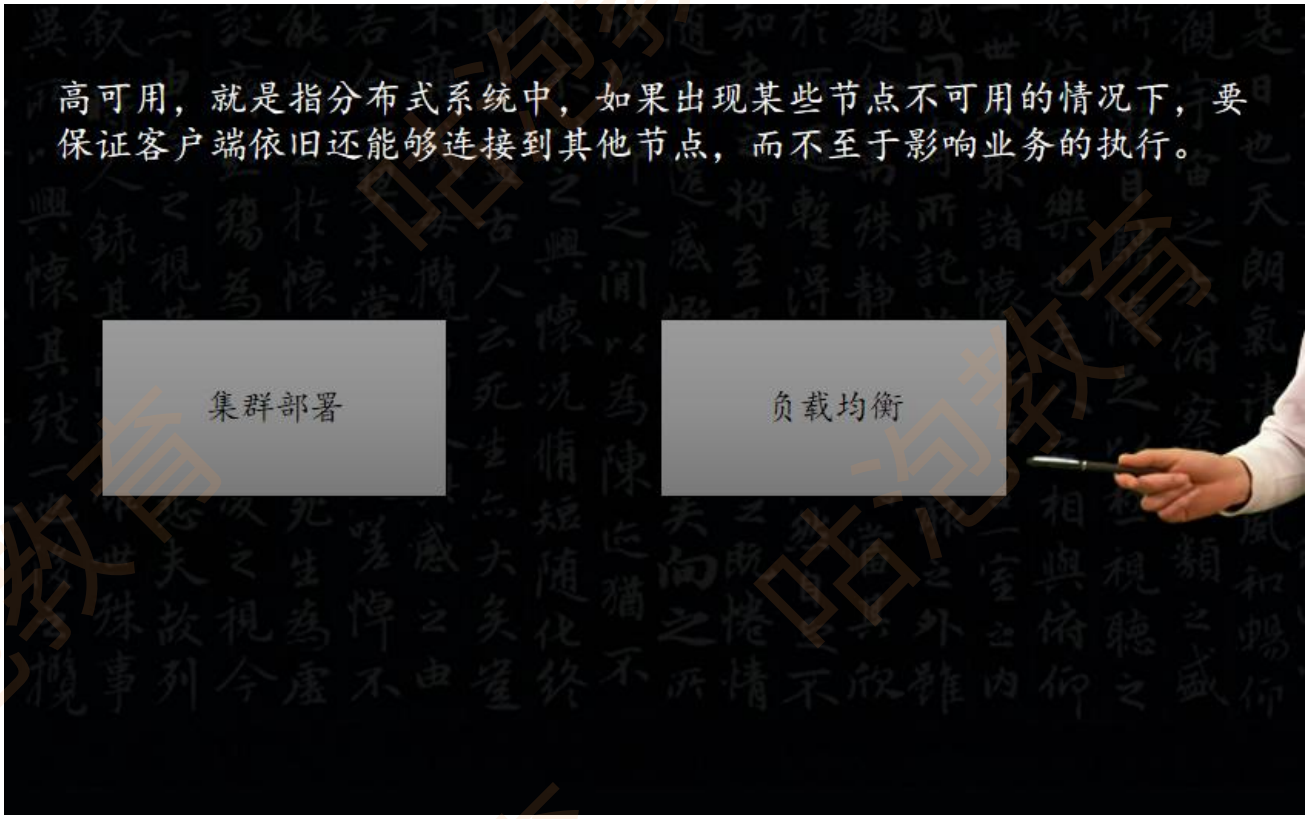
【引言】

一个8年工作经验的小伙伴，被问到这样一个问题，说如何保证RabbitMQ的高可用。关于这个问题呢，这位小伙伴倒是有个实操经验，就是不知道如何组织语言。所以，当时面试结果不太理想。今天，我给大家分享一下我的理解。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套 l文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

1、实现思路

所谓高可用呢，就是指分布式系统中，如果出现某些节点不可用的情况下，要保证客户端依旧还能够连接到其他节点，而不至于影响业务的执行。我主要从以下两个方面来实现高可用，一个是集群部署，一个是负载均衡。



RabbitMQ是用Erlang语言编写的，而Erlang天生具备分布式的特性，所以不需要通过引入ZK来实现服务协调。它是使用.erlang.cookie来验证身份，它通过25672端口来实现集群。这

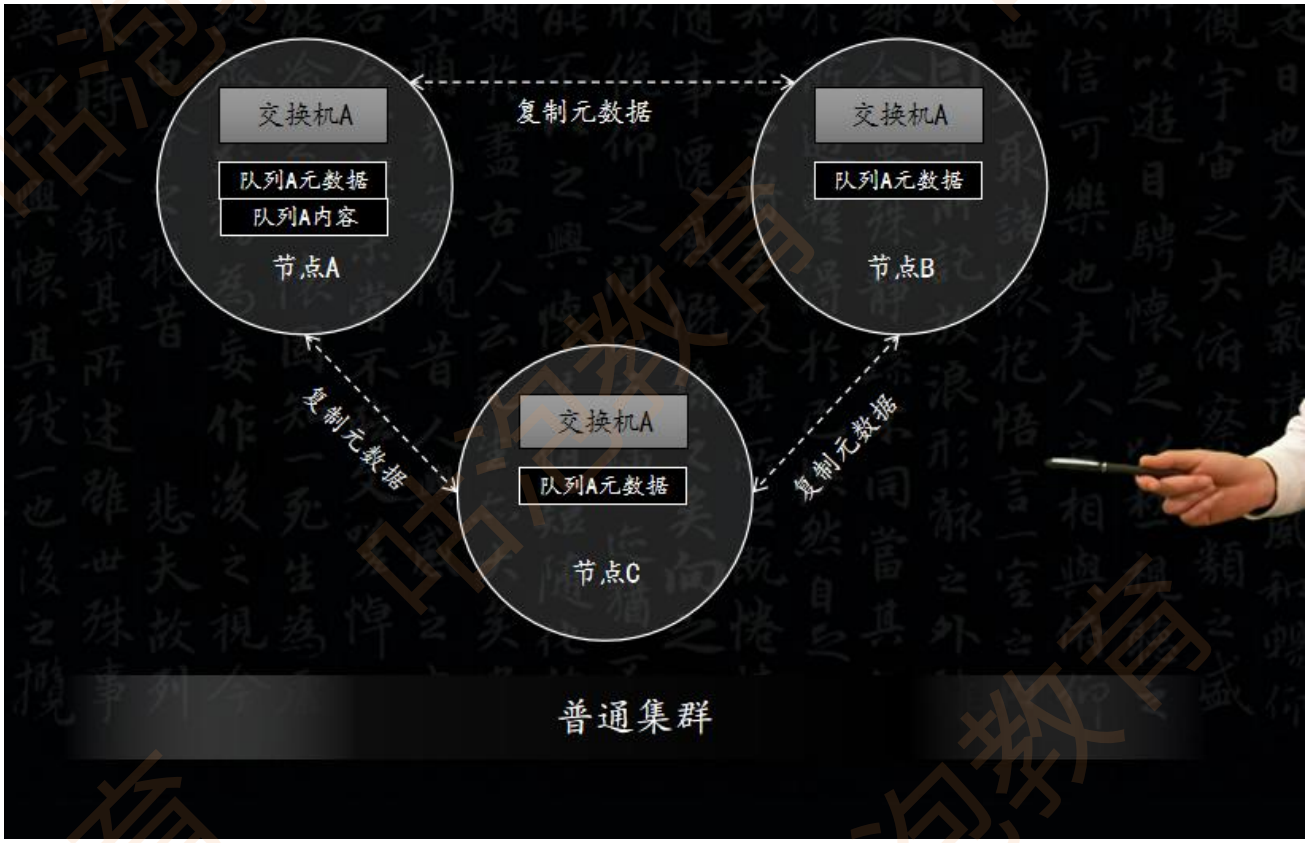
个cookie就像暗号一样，只要喊出清楚明白666，就知道是Tom的粉丝。

另外，RabbitMQ的集群节点有两种，一种是磁盘节点（Disc Node），一种是内存节点（RAM Node）。在集群环境中至少需要一个磁盘节点用来持久化元数据，否则，当全部内存节点崩溃时，就无从同步元数据。

下面，分享一下RabbitMQ的集群模式，分别是普通集群模式和镜像队列模式。先来看普通集群模式。

2、普通集群

普通集群模式下，不同的节点之间只会相互同步元数据，而不会同步消息内容。



如图所示，队列A的消息只存储在A节点上。B节点和C节点同步了队列A的元数据，但是没有同步队列中的内容。

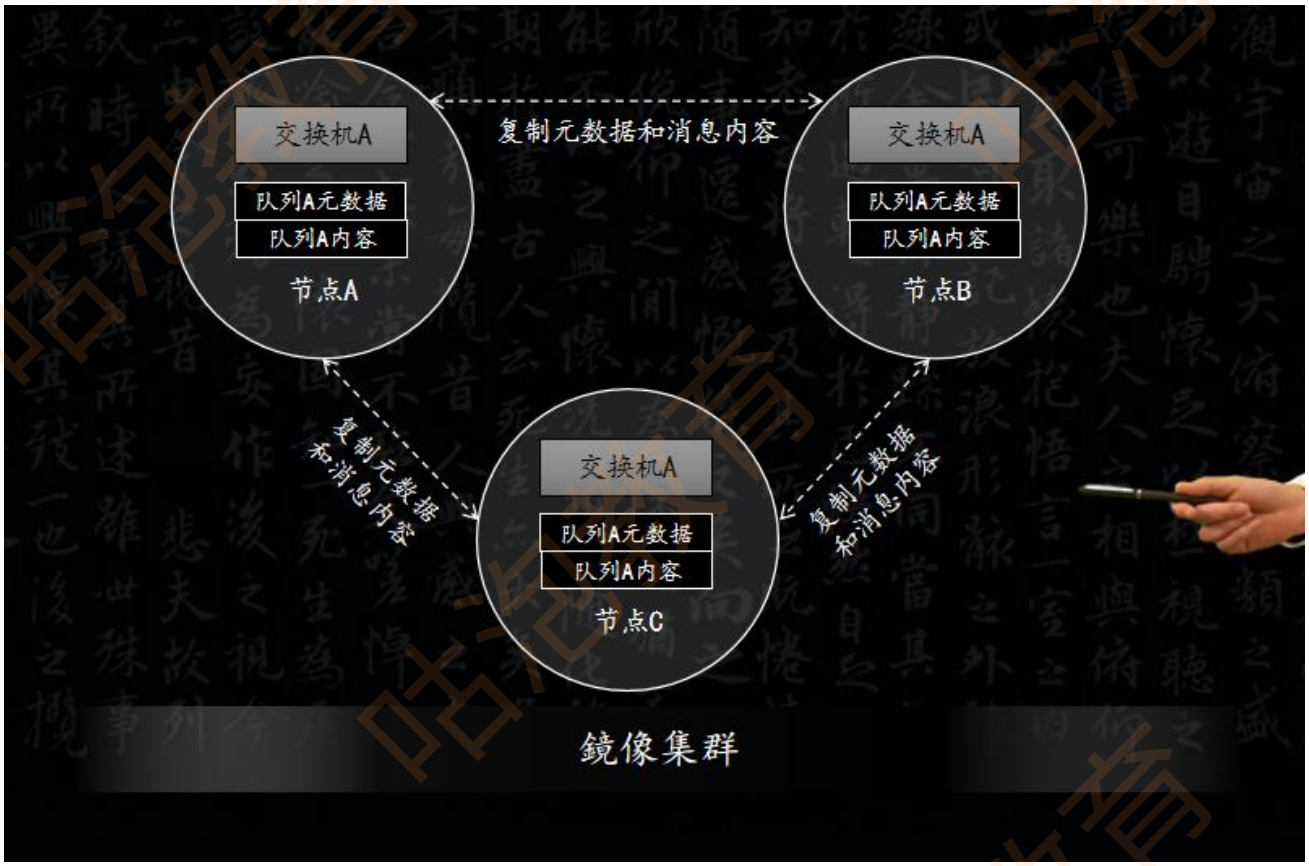
假如生产者连接的是C节点，要将消息通过交换机A路由到队列A，最终消息还是会转发到A节点上存储，因为队列A存储在A节点上。

同理，如果消费者连接是B节点，同样也要从队列A上拉取消息，消息会从A节点转发到节点B。其它节点只是起到一个路由的作用。

这样做的好处是，可以节约存储和同步数据的网络开销。但如果需要保证队列的高可用性，这时候，就需要用到镜像集群的模式。

3、镜像集群

下面来看镜像集群模式，如图所示，



其实也非常好理解，在镜像队列模式下，消息内容会在所有的镜像节点间同步，可用性更高。不过也有一定的副作用，系统性能会降低，节点过多的情况下同步数据的代价会比较大。

4、负载均衡

集群搭建成功后，要保证高可用，还需要一个负载均衡的组件（例如HAProxy，LVS，Nignx），由负载的组件来做路由。对于，生产者或者消费者而言，只需要连接到负载组件的虚拟IP地址就可以了。

这个负载均衡组件需要满足以下几个条件：

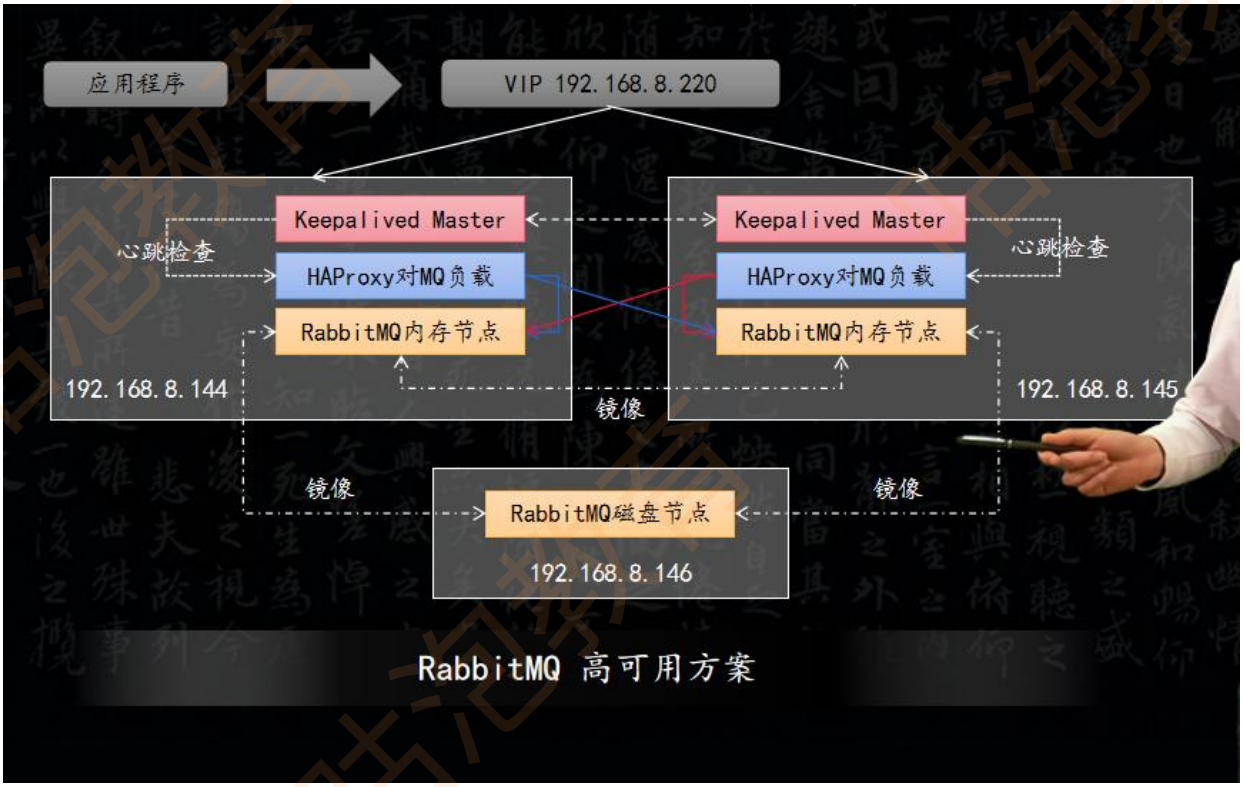
1、它本身有负载功能，可以监控集群中节点的状态，如果某个节点出现异常或者发生故障，

就把它剔除掉。

2、为了提高可用性，要能够支持部署多个服务，能够自主完成Master选举

3、Master节点需要对外提供一个虚拟IP，客户端只需要连接到这一个虚拟IP就可以自动完成真实IP的路由。

下面我以HAProxy加Keepalived组合，来实现RabbitMQ的高可用，具体架构如图所示：



当然，实现高可用的方案还有很多，比如我们还可以用LVS，Nignx等等来实现负载均衡。

【结尾】

关于RabbitMQ的高可用方案我就分享这么。我分享的这个方案，不仅仅只适用于RabbitMQ，其实还可以应用于大部分的分布式高可用场景。小伙伴们如果你还有其他更加牛B的方案，可以在评论区告诉我相互学习一下。

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

3.16 RabbitMQ的消息如何实现路由?

MIC解析

hi，大家好，我是Mic，一个没有才华只能靠颜值混饭吃的Java程序员。 今天分享的这个面试题，没什么特别复杂和特殊的地方。

就是一个RabbitMQ里面的普通面试题这个问题一般是去互联网公司里面去考察3~5年的求职者。

问题是： “RabbitMQ的消息如何实现路由”？

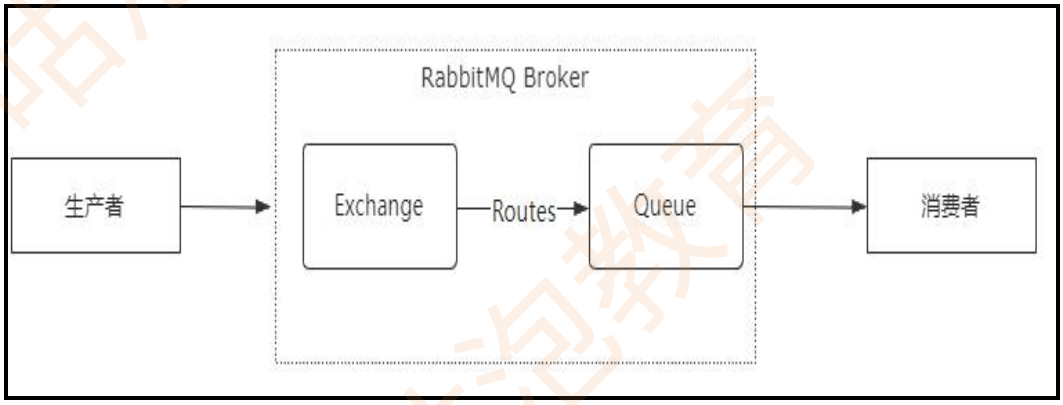
回答

RabbitMQ是一个基于AMQP协议实现的分布式消息中间件。

AMQP的具体工作机制是，生产者把消息发送到RabbitMQ Broker上的Exchange交换机上。

Exchange交换机把收到的消息根据路由规则发给绑定的队列（Queue）。

最后再把消息投递给订阅了这个队列的消费者，从而完成消息的异步通讯。



其中，Exchange是一个消息交换机，它里面定义了消息路由的规则，也就是这个消息路由到那个队列。

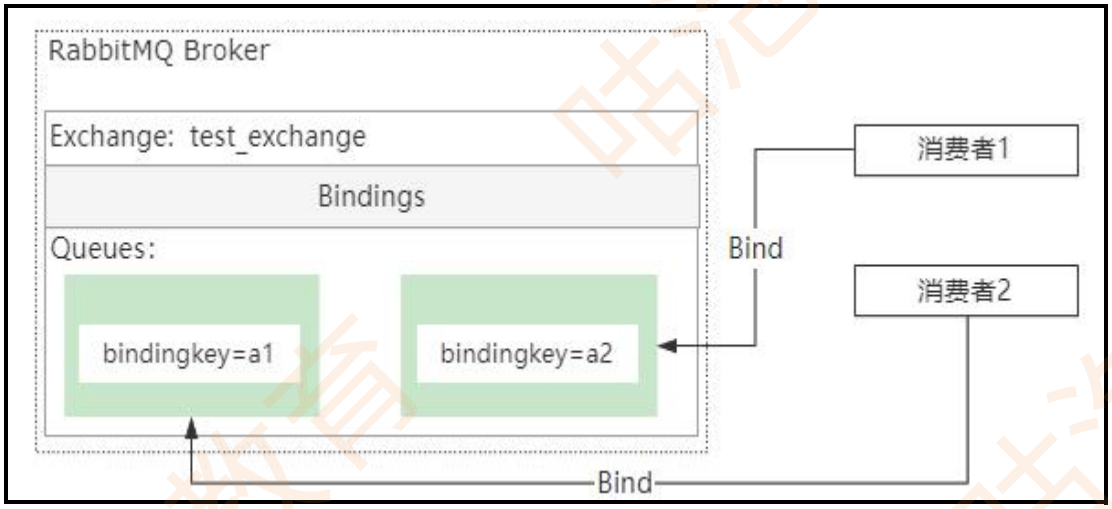
然后Queue表示消息的载体，每个消息可以根据路由规则路由到一个或者多个队列里面。

而关于消息的路由机制，核心的组件是Exchange。

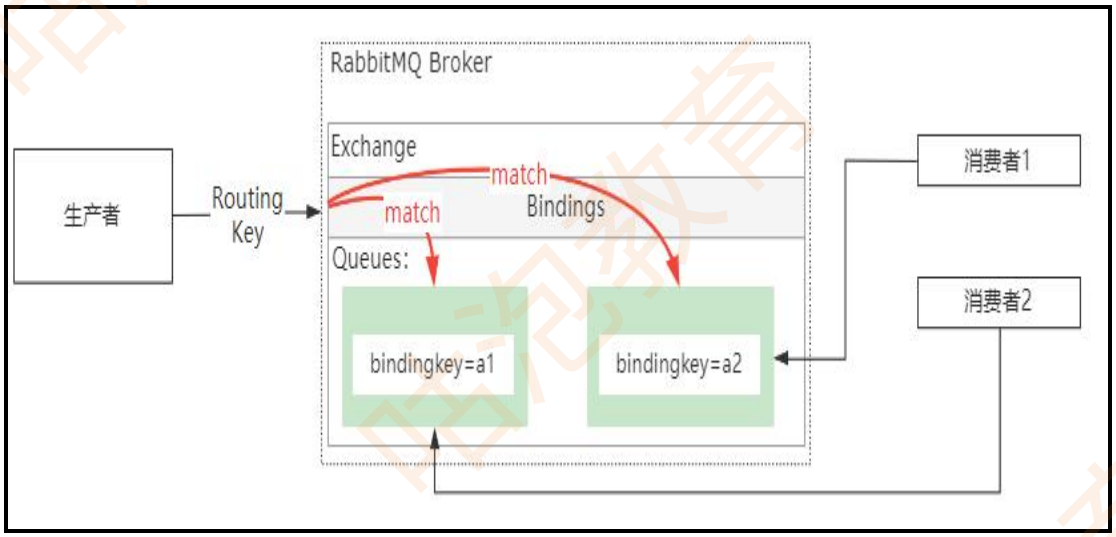
它负责接收生产者的消息然后把消息路由到消息队列，而消息的路由规则由ExchangeType和Binding决定。

Binding表示建立Queue和Exchange之间的绑定关系，每一个绑定关系会存在一个BindingKey。

通过这种方式相当于在Exchange中建立了一个路由关系表。



生产者发送消息的时候，需要声明一个routingKey（路由键），Exchange拿到routingKey之后，根据RoutingKey和路由表里面的BindingKey进行匹配，而匹配的规则是通过ExchangeType来决定的。



在RabbitMQ中，有三种类型的Exchange：direct ，fanout和topic。

direct： 完整匹配方式，也就是Routing key和Binding Key完全一致，相当于点对点的发送。

fanout： 广播机制，这种方式不会基于Routing key来匹配，而是把消息广播给绑定到当前Exchange上的所有队列上。

topic： 正则表达式匹配，根据Routing Key使用正则表达式进行匹配，符合匹配规则的Queue都会收到这个消息

面试点评

RabbitMQ、Kafka、RocketMQ是目前最主流的分布式消息中间件了。

有的同学可能对kafka比较了解，有的同学可能对RabbitMQ比较了解。

不过，在面试的时候，面试官一般会问你用过的技术组件。

通过面试过程中推演出你的学习能力以及对技术的掌握能力，这个方面如果还不错的话，接触一个新的MQ组件所消耗的学习成本会比较小。

大家记得点赞、收藏加关注

我是Mic，咱们下期再见。

TOM解析

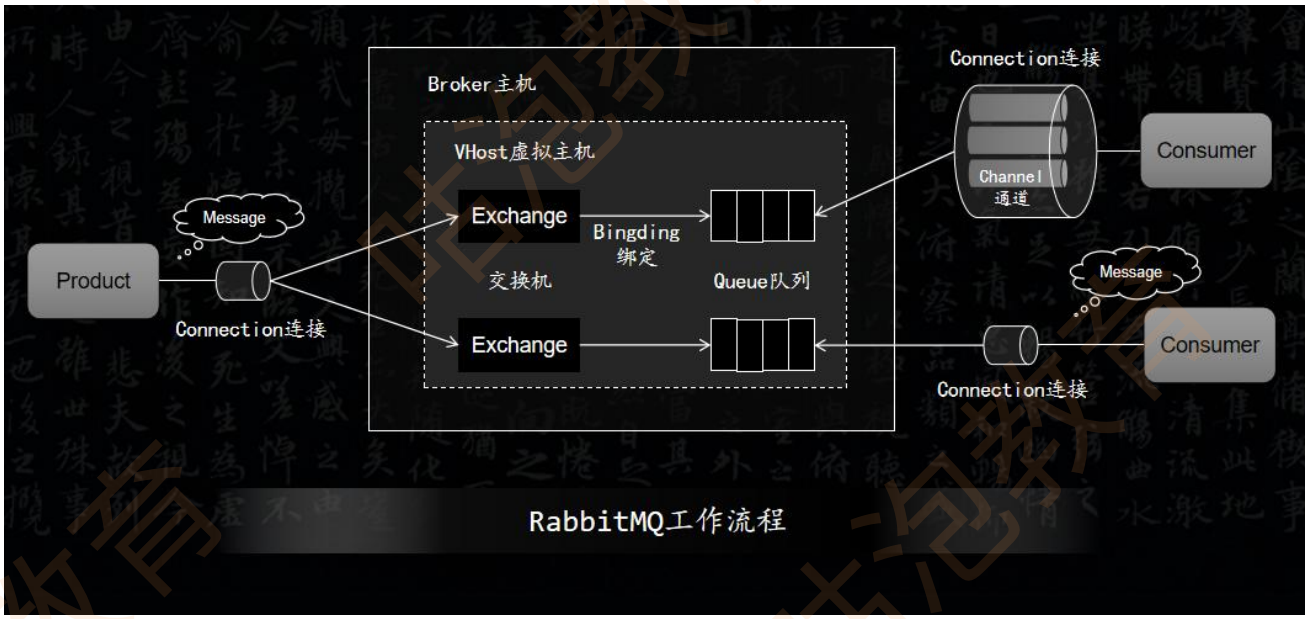
【引言】

一个应届生去面试，可能没有什么实战经验，今天被问到一个这样的面试题，说“RabbitMQ是如何实现消息路由的？“一下子竟然不知道如何组织语言了。今天我给大家分享一下我的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

1、工作流程

RabbitMQ 是一个基于 AMQP 协议实现的分布式消息中间件。AMQP 的具体工作机制是，



生产者把消息发送到 RabbitMQ Broker 上的Exchange 交换机上。Exchange 交换机把收到的消息根据路由规则发给绑定的队列（Queue）。最后再把消息投递给订阅了这个队列的消费者，从而完成消息的异步通讯。

其中，Exchange交换机，可以定义消息的路由规则，将消息路由到指定的队列。然后 Queue队列是消息的载体，每个消息可以根据路由规则路由到一个或者多个队列中。

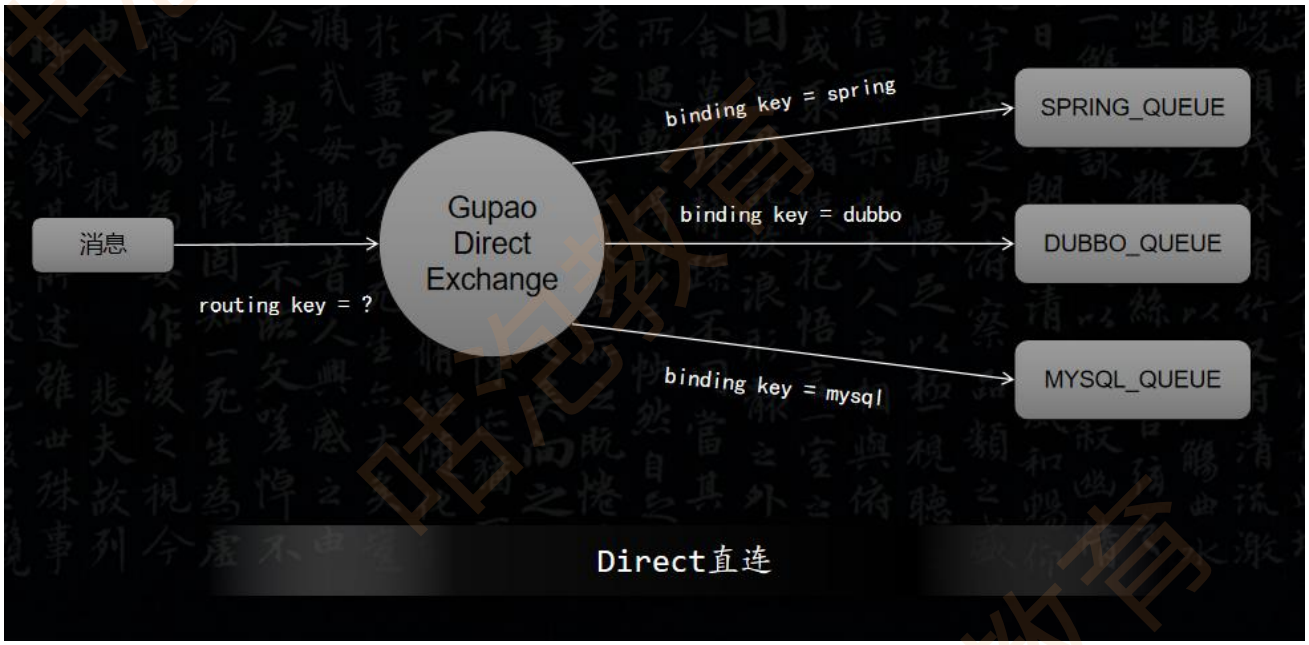
2、路由策略

完成RabbitMQ消息路由的核心组件是 Exchange。而消息的路由是由Exchange类型 和 Binding 来决定的。Binding 表示建立 Queue 和 Exchange 之间的绑定关系，每一个绑定关系会存在一个 BindingKey。

通过这种方式相当于在 Exchange 中建立了一个路由关系表。生产者发送消息的时候，需要声明一个 RoutingKey（路由键），Exchange 拿到RoutingKey 之后，根据 RoutingKey 和路由表里面的 BindingKey 进行匹配，而匹配的规则是通过 Exchange类型来决定的。

在 RabbitMQ 中，默认有四种类型的 Exchange：Direct ，Fanout、Topic和Header。

Direct，叫直连，也就是完整匹配方式，需要Routing Key 和 Binding Key 完全一致，相当于点对点的发送。如图所示：



如果发送routing key为”spring”的消息，那么只有第一个队列能收到消息。

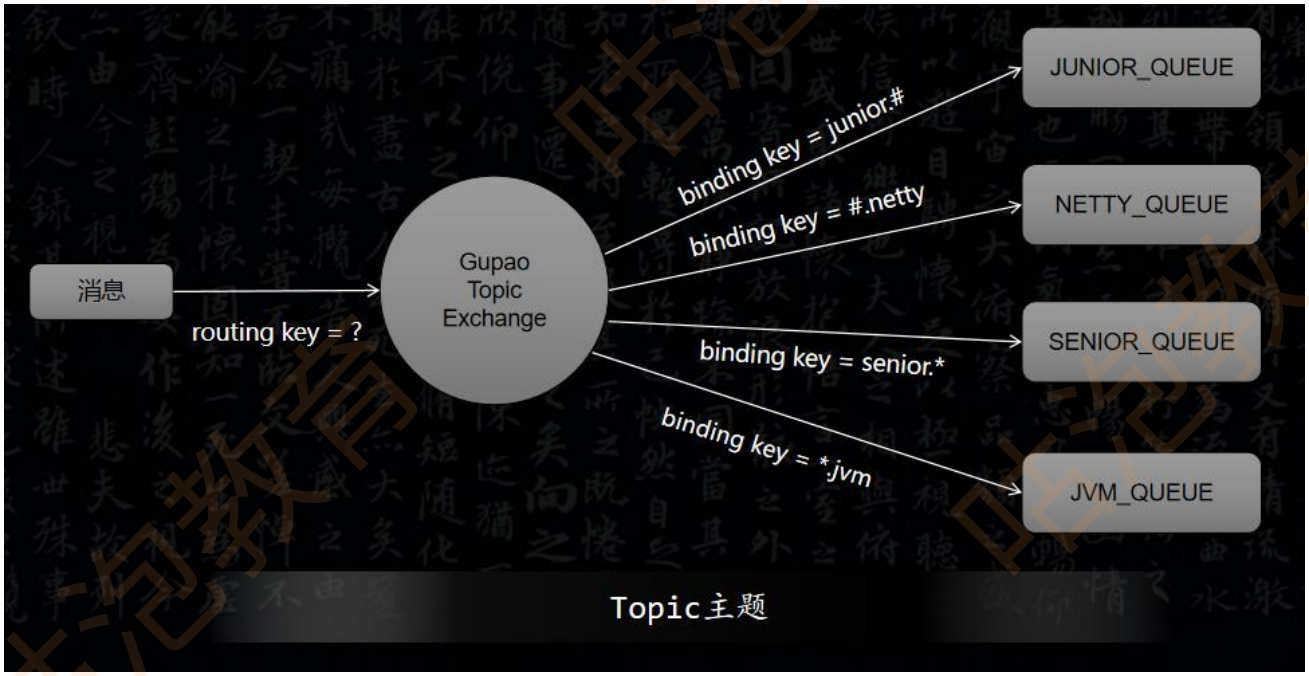
Topic： 叫主题，这种方式是通过设置通配符来动态匹配，相当于正则。就是用Routing Key去匹配Binging Key。BingingKey支持两个通配符。

代表匹配0个或者多个单词

* 代表匹配不多不少一个单词

另外，BingingKey用点隔开两个单词。

，用*表示使用正则表达式进行匹配，如图所示：

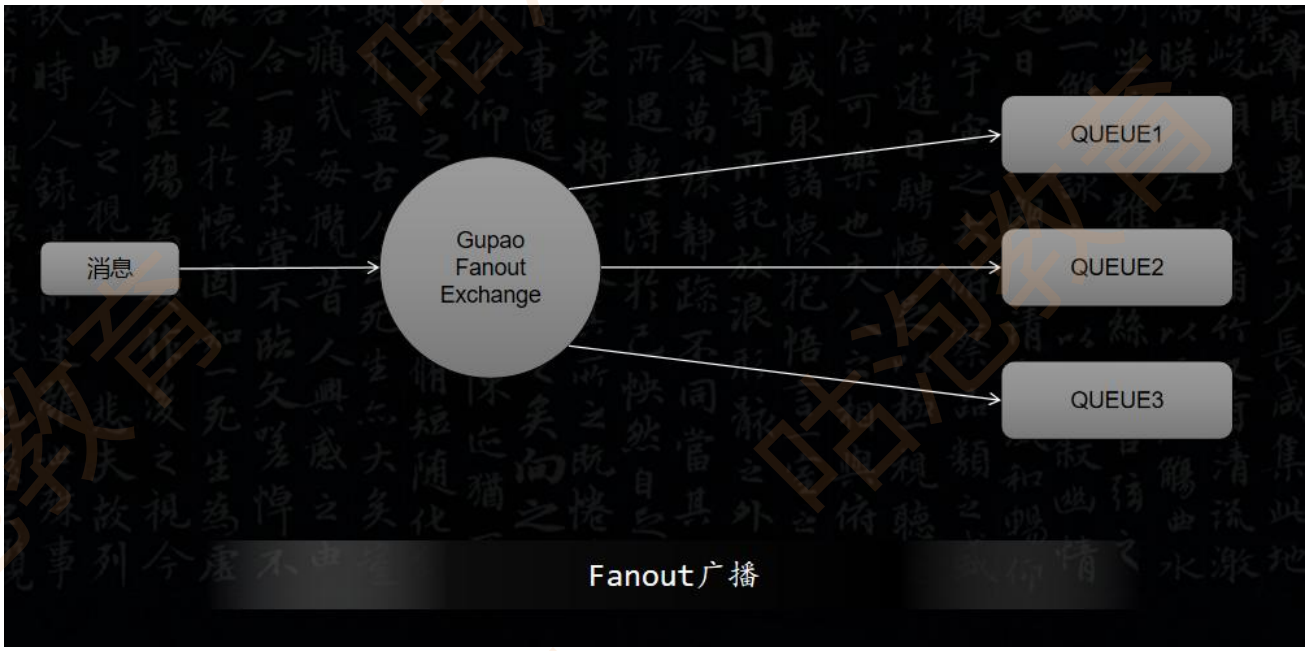


有4个队列绑定到Topic类型的交换机，而且使用不同的绑定键。

如果发送routing key为"junior.abc.jvm" 的消息，那么，只有第一个队列能收到。

如果发送routing key为"senior.netty"的消息，那么，第二个队列和第三个队列能收到。

Fanout：叫广播，这种方式不需要设置Routing Key，而是把消息广播给绑定到当前Exchange 上的所有队列上。如图所示：



只要发送消息到Fanout Exchange上，那么三个队列都能收到消息

【结尾】

RabbitMQ、Kafka、RocketMQ 是目前最主流的分布式消息中间件。有的同学可能对 Kafka 比较了解，有的同学可能对 RabbitMQ 比较了解。不过，在面试的时候，面试官一般会问你用过的技术组件。通过面试过程中推演出你的学习能力以及对技术的掌握能力，这个方面如果还不错的话，接触一个新的 MQ 组件所消耗的学习成本会比较小。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

计算机网路专栏面试题（4）

大家好

金九银十即将来临啦，我梳理了 60 道计算机网络面试题，每一道题目都特别经典，大厂也非常喜欢问。相信大家看完，会有新的收获滴～

4.1 说说 HTTP 常用的状态码及其含义?

思路: 这道面试题主要考察候选人，✂否掌握 HTTP 状态码这个基础知识点。

状态码	类别
1XX	信息性状态码
2XX	成功状态码
3XX	重定向状态码
4XX	客户端错误状态码
5XX	服务端错误状态码

不管✂不✂面试需要，我们都要知道，日常开发中✂这几个状态码✂含义哈：

状态码	含义
101	切换请求协议
200	请求成功
301	永久性重定向，会缓存
302	临时重定向，不会缓存
400	客户端请求的语法错误
403	服务器禁止访问，权限有关
404	服务器无法根据客户端的请求找到资源
500	服务端错误

4.2 HTTP 常用的请求方式，区别和用途？

思路: 这道题主要考察候选人，✂否掌握 **HTTP 请求方式**这个基础知识点，我们用得比较多就✂ **GET 和 POST** 啦。

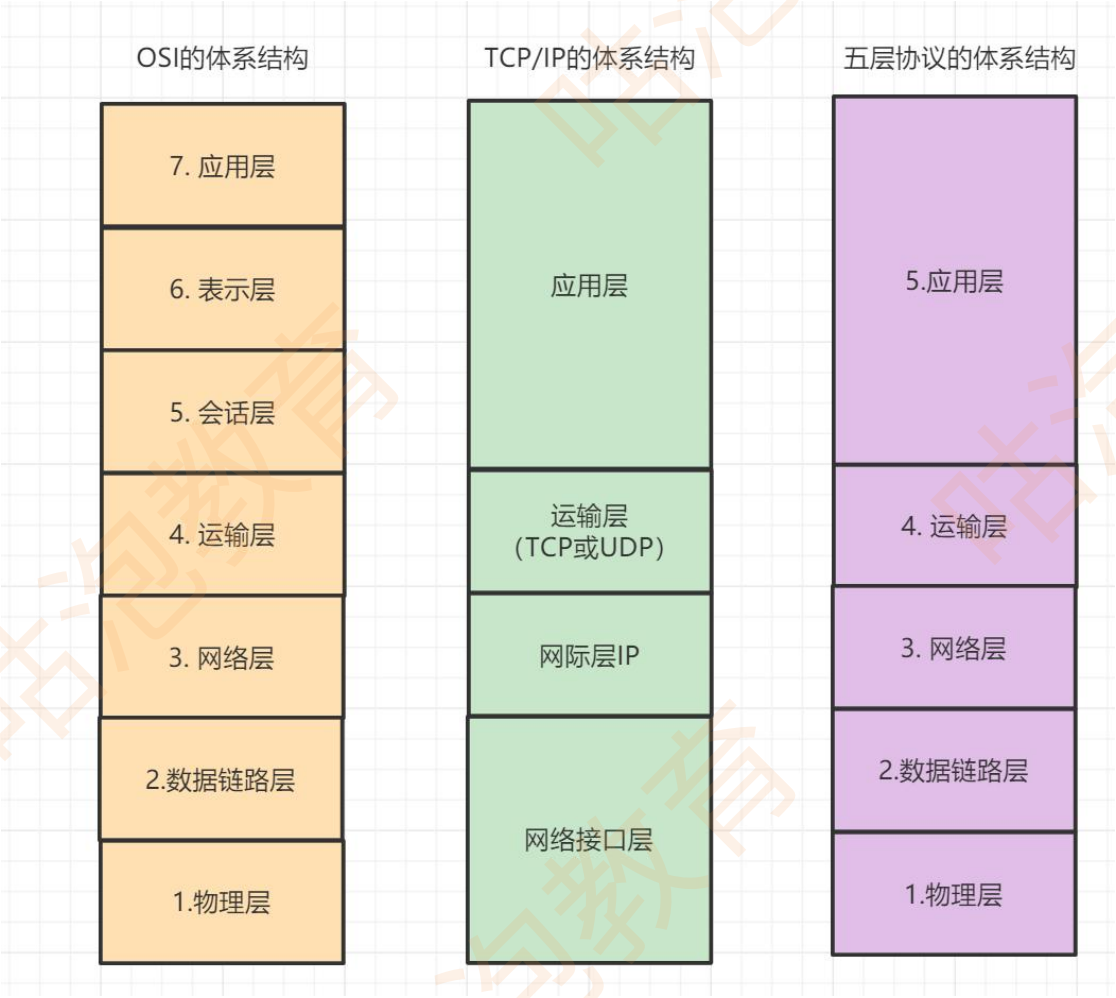
请求方式	用途
GET	对服务器资源获取的简单请求
POST	用于发送包含用户提交数据的请求
PUT	向服务器提交数据，以修改数据
HEAD	请求页面的首部，获取资源的元信息
DELETE	删除服务器上的某些资源
CONNECT	用于ssl隧道的基于代理的请求
OPTIONS	返回所有可用的方法，常用于跨域
TRACE	追踪请求-响应的传输路径

4.3 请简单说一下你了解的端口及对应的服务？

端口	服务
21	FTP(文件传输协议)
22	SSh
23	Telnet(远程登录)服务
25	SMTP（简单邮件传输协议）
53	DNS域名服务器
80	HTTP超文本传输协议
110	POP3邮件协议3
443	HTTPS
1080	Sockets
1521	Oracle数据库默认端口
3306	MySQL服务默认端口

4.4 说下计算机网络体系结构

思路: 这道题主要考察候选人，**计算机网络体系结构**这个基础知识点。计算机网路体系结构呢，有三层：ISO 七层模型、TCP/IP 四层模型、五层体系结构。大家可以记住这个图，如下



计算机网络体系结构

4.4.1 OSI 七层模型

OSI（Open System Interconnect），即开放式系统互联。 一般都叫OSI参考模型，✂ ISO（国际标准化组织）组织在 1985 年研究✂网络互连模型，OSI模型共分为七层

- 应用层：网络服务与最终用户✂一个接口，常见✂协议有：**HTTP FTP SMTP SNMP DNS.**
- 表示层：数据✂表示、安全、压缩。，确保一个系统✂应用层所发送✂信息可以被另一个系统✂应用层读取。
- 会话层：✂立、管理、终止会话,对应主机进程，指本地主机与远程主机正在进行✂会话.

- 传输层：定义传输数据✂协议端口号，以及流控和差错校验,协议有 TCP UDP.
- 网络层：进行逻辑地址寻址，实现不同网络之间✂路径选择,协议有 ICMP IGMP IP 等.
- 数据链路层：在物理层提供比特流服务✂基础上，✂立相邻结点之间✂数据链路。
- 物理层：✂立、维护、断开物理连接。

4.4.2 TCP/IP 四层模型

- 应用层：对应于 OSI 参考模型✂（应用层、表示层、会话层）。
- 传输层:对应 OSI ✂传输层，为应用层实体提供端到端✂通信功能，保证了数据包✂顺序传送及数据✂完整性。
- 网际层：对应于 OSI 参考模型✂网络层，主要解决主机到主机✂通信问题。
- 网络接口层：与 OSI 参考模型✂数据链路层、物理层对应。

4.4.3 五层体系结构

- 应用层：对应于 OSI 参考模型✂（应用层、表示层、会话层）。
- 传输层：对应 OSI 参考模型✂✂传输层
- 网络层：对应 OSI 参考模型✂✂网络层
- 数据链路层：对应 OSI 参考模型✂✂数据链路层
- 物理层：对应 OSI 参考模型✂✂物理层。

4.5 如何理解 HTTP 协议✂无状态✂

思路: 这道题主要考察候选人，✂否理解 Http 协议，它为什么✂无状态✂呢?如何使它有状态呢?

当浏览器第一次发送请求给服务器时，服务器响应了；如果同个浏览器发第二次请求给服务器时，它还✂会响应，但✂呢，服务器不知道你就✂刚才✂那个浏览器。简言之，服务器不会去记住你✂谁，所以✂无状态协议。

如何理解无状态呢?

可以通过一个生活中的例子，来更好理解并记住它：

有状态场景：

- 小红：今天吃啥子？
- 小明：罗非鱼~
- 小红：味道怎么样呀？
- 小明：还不错，好香。

无状态✂场景：

- 小红：今天吃啥子？
- 小明：罗非鱼~
- 小红：味道怎么样呀？
- 小明：？啊？你说啥？什么鬼？什么味道怎么样？

Http 加了 Cookie ✂话：

- 小红：今天吃啥子？
- 小明：罗非鱼~
- 小红：你今天吃✂罗非鱼，味道怎么样呀？
- 小明：还不错，好香。

4.6 从浏览器地址栏输入 url 到显示主页✂过程

思路：这道题主要考察✂知识点✂ HTTP ✂请求过程，**DNS 解析，TCP 三次握手，四次挥手这几个要点**，我们都可以讲下。

1. DNS 解析，查找域名对应✂ IP 地址。
2. 与服务器通过三次握手，✂立 TCP 连接
3. 向服务器发送 HTTP 请求
4. 服务器处理请求，返回网页内容

5. 浏览器解析并渲染页面
6. TCP 四次挥手，连接结束

过程	使用的协议
1.浏览器查找域名的IP地址 （DNS查找过程：浏览器缓存、路由器缓存、DNS缓存）	DNS：获取域名对应的IP
2.浏览器向web服务器发送一个HTTP请求 （cookies会随着请求发送给服务器）	TCP:与服务器建立TCP连接 IP：建立TCP协议时，需要发送数据，发送的数据在网络层使用IP协议 OPSF:IP数据包在路由器之间，路由选择使用OPSF协议 ARP:路由器在与服务器通信时，需要将IP地址转换为MAC地址，需要适应ARP协议 HTTP:在TCP连接完成后，使用HTTP协议访问网页
3.服务器处理请求 （处理请求 & 它的参数、cookies、生成一个HTML响应）	
4.服务器发回一个HTML响应	
5.浏览器开始显示HTML	

4.7 说下 HTTP 1.0，1.1，2.0 ✂区别

思路: 这道题主要考察✂知识点✂ HTTP 几个版本✂区别，我们记住 HTTP 1.0 默认✂短连接，可以强制开启，HTTP 1.1 默认长连接，HTTP 2 采用多路复用就差不多啦。

我✂答案如下：

HTTP/1.0

- 默认使用**短连接**，每次请求都需要✂立一个 TCP 连接。它可以设置 Connection: keep-alive这个字段，强制开启长连接。

HTTP/1.1

- 引入了持久连接，即 TCP 连接默认不关闭，可以被多个请求复用。
- 分块传输编码，即服务端没产生一块数据，就发送一块，用”流模式”取代”缓存模式”。
- 管道机制，即在同一个 TCP 连接里面，客户端可以同时发送多个请求。

HTTP/2.0

- 二进制协议，1.1 版本✂头信息✂文本（ASCII 编码），数据体可以✂文本或者二进制；2.0 中，头信息和数据体都✂二进制。

- 完全多路复用，在一个连接里，客户端和浏览器都可以同时发送多个请求或回应，而且不用按照顺序一一对应。
- 报头压缩，HTTP 协议不带有状态，每次请求都必须附上所有信息。Http 2.0 引入了头信息压缩机制，使用 gzip 或 compress 压缩后再发送。
- 服务端推送，允许服务器未经请求，主动向客户端发送资源。

4.8 POST 和 GET 有哪些区别?

思路: 这道题主要考察✂知识点✂ POST 和 GET ✂区别，可以从**数据包、编码方式、请求参数、收藏为书签、历史记录、安全性**等几方面去回答哈。

请求方式	GET	POST
数据包	产生一个TCP数据包	可能产生两个TCP数据包
编码方式	只能进行url编码	支持多种编码方式
请求参数	把参数包含在URL中，用&连接起来	通过 request body 传递参数。
请求缓存	会被主动Cache	不会缓存，除非手动设置
收藏为书签	支持收藏为书签	不支持
历史记录	参数会被完整保留在浏览历史记录里	不会保留
参数数据类型	ASCII的字符	没有限制数据类型

4.9 在交互过程中如果数据传送完了，还不想断开连⑩怎么办，怎么维持?

这个问题记住 keep-alive就好，也就✂说，在 HTTP 中响应体✂

Connection 字段指定为 keep-alive 即可

4.10 HTTP 如何实现长连⑩？在什么时候会超时?

思路: 这道题实际上✂考察 TCP 长连⑩✂知识点，HTTP ✂长连⑩实质✂指TCP ✂长连⑩。至于什么时候超时，我们记住这几个参数如 **tcp_keepalive_time、tcp_keepalive_probes** 就好啦

什么✂ HTTP ✂长连⑩?

0. HTTP 分为长连⑩和短连⑩，其实**本质上说✂✂ TCP ✂长短连⑩。**TCP 连⑩✂一个双向✂通道，它✂可以保持一段时间不关闭✂，因此 TCP 连⑩才具有真正✂长连⑩和短连⑩这一说法哈。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

0. TCP 长连⑩可以复用一个 TCP 连⑩，来发起多次✂ HTTP 请求，这样就可以减少资源✲耗，比如一次请求 HTML，如果✂短连⑩✂话，可能还需要请求后续✂ JS/CSS。

如何设置长连⑩?

通过在头部（请求和响应头）设置 **Connection** 字段指定为 keep- alive，HTTP1.0 协议支持，但✂✂默认关闭✂，从 HTTP 1.1 以后，连⑩默认都✂长连⑩。

在什么时候会超时呢?

- HTTP 一般会有 httpd 守护进程，里面可以设置 **keep-alive timeout**，当 tcp 连⑩闲置超过这个时间就会关闭，也可以在 HTTP ✂ header 里面设置超时时间
- TCP ✂ **keep-alive** 包含三个参数，支持在系统内核✂ net.ipv4 里面设置；当 TCP 连⑩之后，闲置了 **tcp_keepalive_time**，则会发生侦测包，如果没有收到对方✂ ACK，那么会每隔 tcp_keepalive_intvl 再发一次，直到发送了 **tcp_keepalive_probes**，就会丢弃该连⑩。

```
tcp_keepalive_intvl = 15
tcp_keepalive_probes = 5
tcp_keepalive_time = 1800
```

4.11 HTTP 与 HTTPS 的区别。

思路: 这道题实际上考察✂知识点✂ HTTP 与 HTTPS ✂区别，个人觉得这个知识点非常重要，可以**从安全性、数据✂否加密、默认端口**等这几个方面去回答哈。其实，当你理解 HTTPS ✂整个流程，就可以很好回答这个问题啦。

我的答案如下:

HTTP，即超文本传输协议，✂一个基于 TCP/IP 通信协议来传递明文数据✂协议。HTTP 会存在这**几个问题**:

- 请求信息✂明文传输，容易被窃听截取。

- 没有验证对方身份，存在被冒充✂风险
- 数据✂完整性未校验，容易被中间人篡改

为了解决 Http 存在✂问题，Https 它出现啦。

Https ✂什么?

HTTPS= HTTP+SSL/TLS，可以理解 Https ✂身披SSL(Secure SocketLayer，安全套⑩层)✂ HTTP。

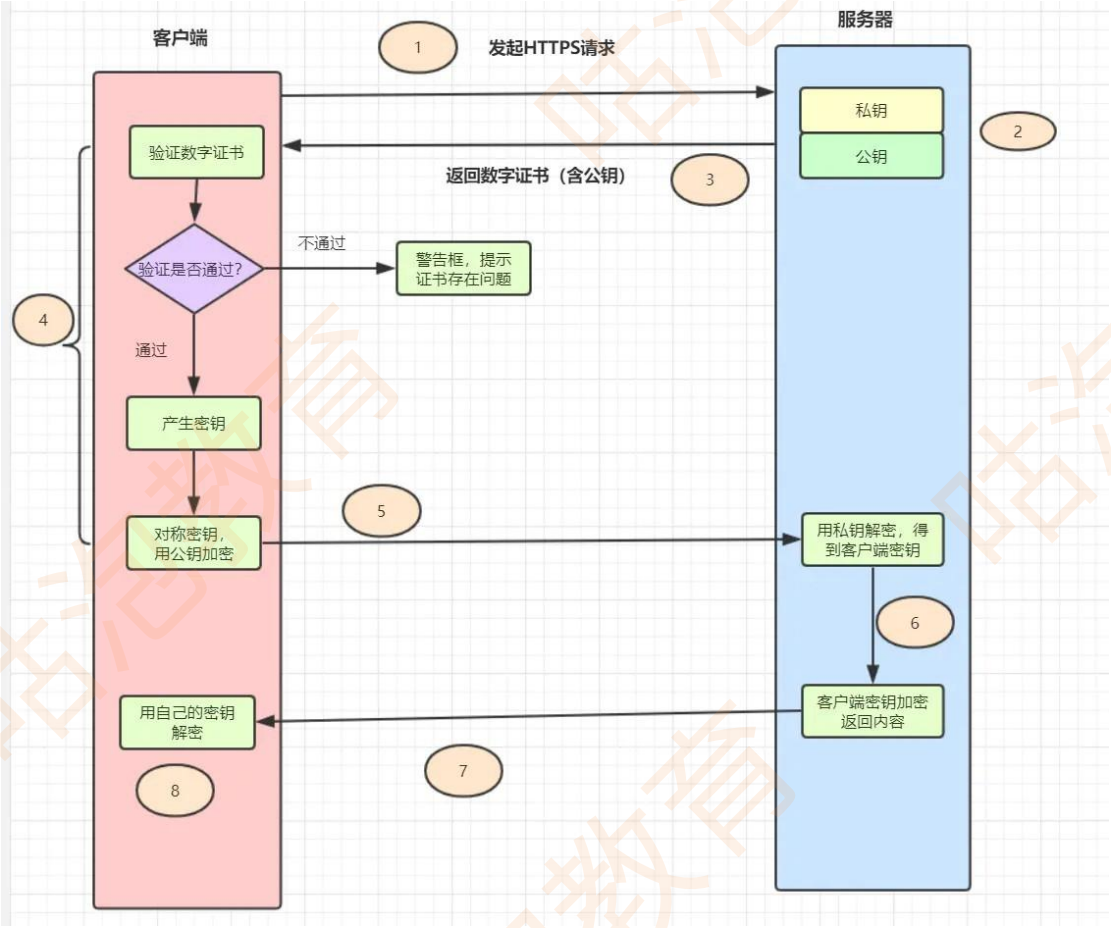
	Http	Https
安全性	不安全	安全
默认端口	80	443
资源消耗	较少	消耗资源较多
是否需要证书	不需要	需要
报文是否加密	明文	密文

它们主要区别如下:

4.12 Https 流程✂怎样✂?

思路: 这道题实际上考察✂知识点✂ HTTPS ✂工作流程，大家需要回答这几个要点，**公私钥、数字证书、加密、对称加密、非对称加密。**

- HTTPS = HTTP + SSL/TLS，也就✂用 SSL/TLS 对数据进行加密和解密，Http 进行传输。
- SSL，即Secure Sockets Layer（安全套⑩层协议），✂网络通信提供安全及数据完整性✂一种安全协议。
- TLS，即 Transport Layer Security(安全传输层协议)，它✂ SSL3.0 ✂后续版本。



Https 工作流程

1. 客户端发起 Https 请求，连⑩到服务器✂ 443 端口。
2. 服务器必须要有一套数字证书（证书内容有公钥、证书颁发机构、失效日期等）。
3. 服务器将自己✂数字证书发送给客户端（公钥在证书里面，私钥由服务器持有）。
4. 客户端收到数字证书之后，会验证证书✂合法性。如果证书验证通过，就会生成一个随机✂对称密钥，用证书✂公钥加密。
5. 客户端将公钥加密后✂密钥发送到服务器。
6. 服务器⑩收到客户端发来✂密文密钥之后，用自己之前保留✂私钥对其进行非对称解密，解密之后就得到客户端✂密钥，然后用客户端密钥对返回数据进行对称加密，酱紫传输✂数据都✂密文啦。
7. 服务器将加密后✂密文返回到客户端。
8. 客户端收到后，用自己✂密钥对其进行对称解密，得到服务器返回✂数据。

4.13说说 HTTP ✂状态码，301 和 302 ✂区别?

思路: 这道题考查✂知识点，也✂ HTTP 状态码，302 和 301 都有重定向✂含义，但✂它们也✂有区别✂。

- 301：（永久性转移）请求✂网页已被永久移动到新位置。服务器返回此响应时，会自动将请求者转到新位置。
- 302：（暂时性转移）服务器目前正从不同位置✂网页响应请求，但请求者应继续使用原有位置来进行以后✂请求。此代码与响应 GET 和 HEAD 请求✂ 301 代码类似，会自动将请求者转到不同✂位置。

当一个网站或者网页 24—48 小时内临时移动到一个新✂位置，这时候就要进行 302 跳转，打个比方说，我有一套房子，但✂最近走亲戚去亲戚家住了，过两天我还回来✂。而使用 301 跳转✂场景就✂之前✂网站因为某种原因需要移除掉，然后要到新✂地址访问，✂永久性✂，就比如你✂那套房子其实✂租✂，现在租期到了，你又在另一个地方找到了房子，之前租✂房子不住了。

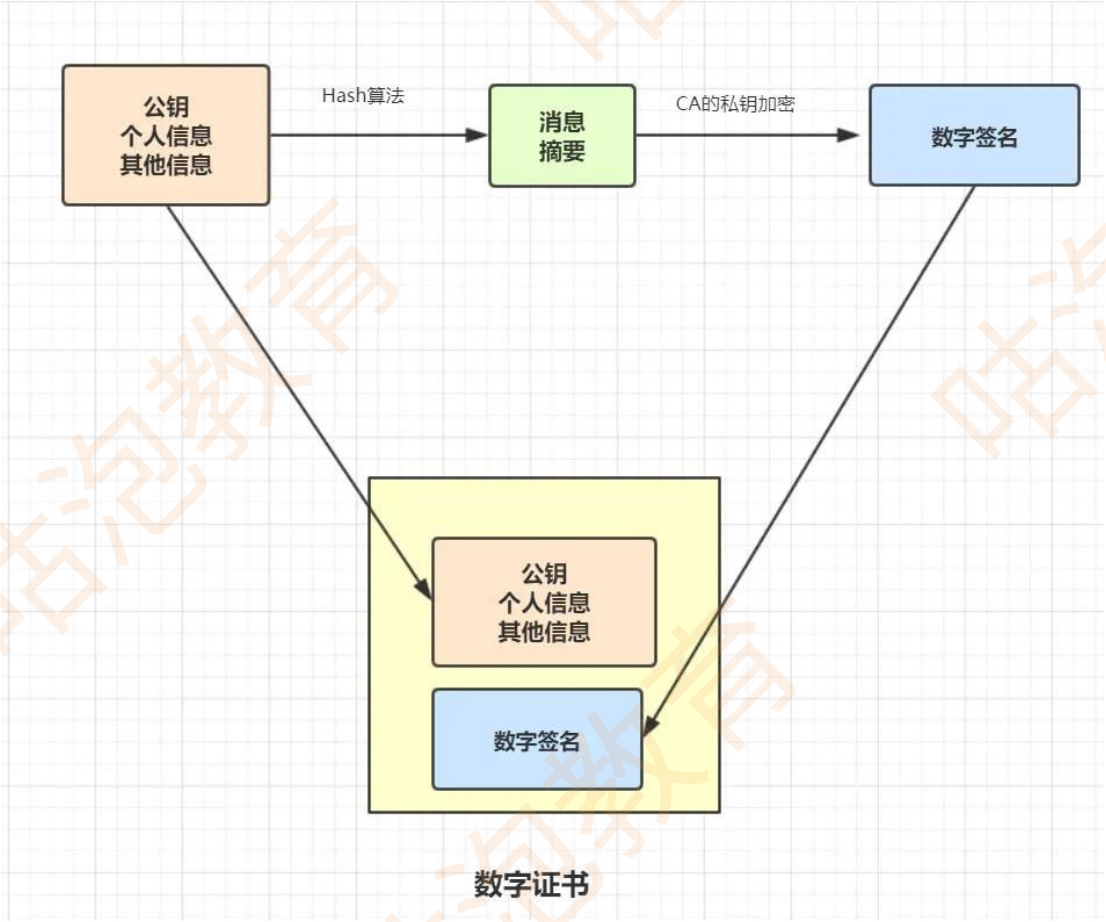
网上有个很**形象✂例子**比喻：

4.14 说说什么✂数字签名？ 什么✂数字证书？

思路: 这道题考查✂知识点，不仅仅✂数字签名，数字证书，很可能面试官也会问你 https ✂原理✂，因为 https 原理跟数字证书有关哈，大家需要掌握 https 原理哈。

数字证书✂指在互联网通讯中标志通讯各方身份信息✂一个数字认证，人们可以在网上用它来识别对方✂身份。它✂出现，✂为了避免身份被篡改冒充✂。

比如 Https ✂数字证书，就✂为了避免公钥被中间人冒充篡改：



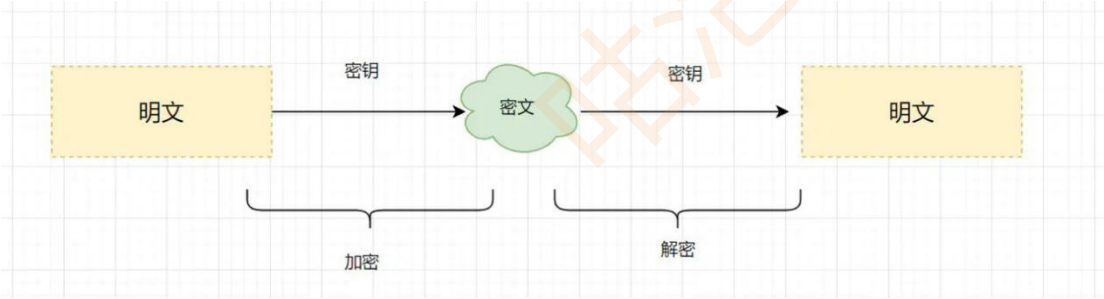
数字证书构成

- 公钥和个人等信息，经过 Hash 摘要算法加密，形成✲息摘要；将✲息摘要拿到拥有公信力✂认证中心（CA），用它✂私钥对✲息摘要加密，形成**数字签名。**
- 公钥和个人信息、数字签名共同构成**数字证书。**

4.15 对称加密与非对称加密有什么区别

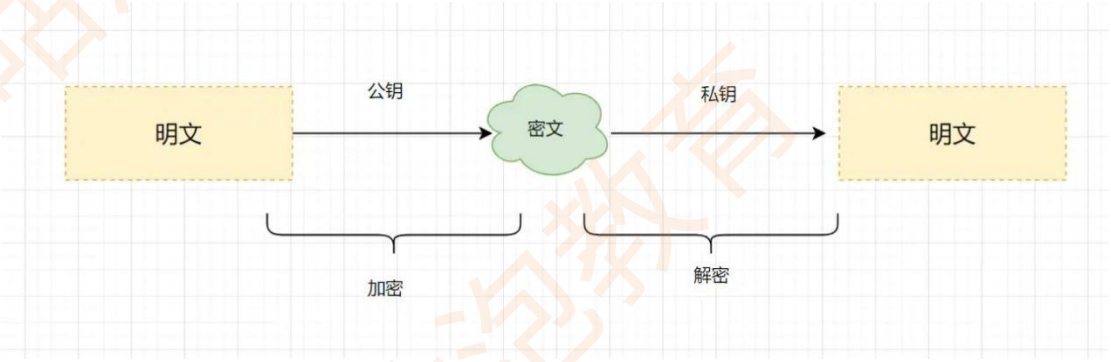
思路: 这道题考察✂知识点✂对称加密与非对称加密算法，什么✂对称加密，什么✂非对称加密呢?

对称加密：指加密和解密使用同一密钥，优点✂运算速度较快，缺点✂如何安全将密钥传输给另一方。常见✂对称加密算法有：DES、AES 等。



对称加密

非对称加密：指✂✂加密和解密使用不同✂密钥（即公钥和私钥）。公钥与私钥✂成对存在✂，如果用公钥对数据进行加密，只有对应✂私钥才能解密。常见✂非对称加密算法有 RSA。



非对称加密

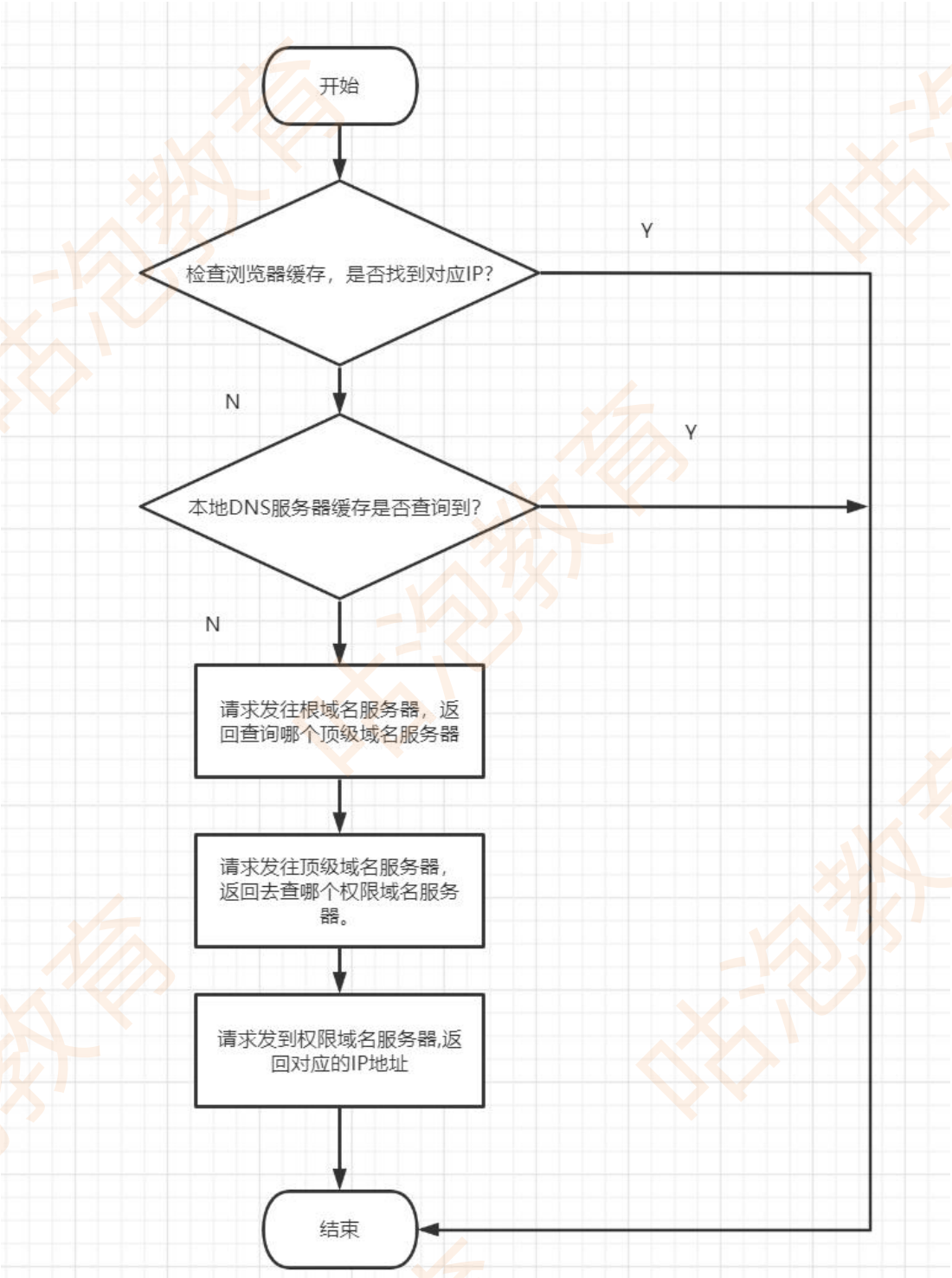
4.16 说说 DNS ✂解析过程?

思路: 这道题考察✂知识点✂ **DNS 域名解析**，http 请求✂过程，✂涉及到 DNS 域名解析✂，这道面试题也挺经典✂，大家可以看下《图解 HTTP》那本书哈。

DNS，英文全称✂ **domain name system**，域名解析系统，✂ Internet 上作为域名和 IP 相互映射✂一个分布式数据库。它✂作用很明确，就✂可以根

据域名查出对应✂ IP 地址。在浏览器缓存、本地 DNS 服务器、根域名服务器都✂怎么查找✂，大家回答✂时候都可以说下哈。

DNS ✂解析过程如下图：



DNS 解析查找过程

假设你要查询 **www.baidu.com** ✂ IP 地址:

- 首先会查找浏览器✂缓存,看看✂否能找到 **www.baidu.com** 对应✂ IP 地址，找到就直⑩返回；否则进行下一步。
- 将请求发往给本地 DNS 服务器，如果查找到也直⑩返回，否则继续进行下一步；
- 本地 DNS 服务器向**根域名服务器**发送请求，根域名服务器返回负责 .com ✂顶级域名服务器✂ IP 地址✂列表。
- 本地 DNS 服务器再向其中一个负责 .com ✂顶级域名服务器发送一个请求，返回负责 .baidu ✂权威域名服务器✂ IP 地址列表。
- 本地 DNS 服务器再向其中一个权威域名服务器发送一个请求，返回 **www.baidu.com** 所对应✂ IP 地址。

4.17什么✂ CSRF 攻击，如何避免

思路: 这道题考察✂知识点✂ **CSRF 攻击**，它✂属于网络安全这块✂知识点，还有 **Xss 攻击、SQL 注入、DDoS** 等这些常见✂网络攻击，我们都需要知道攻击✂流程哈。

什么✂ CSRF 攻击?

CSRF，跨站请求伪造（英文全称✂ Cross-site request forgery），✂一种挟制用户在当前已登录✂ Web 应用程序上执行非本意✂操作✂攻击方法。。

CSRF ✂如何攻击✂呢?

来看一个来自百度百科✂例子哈:

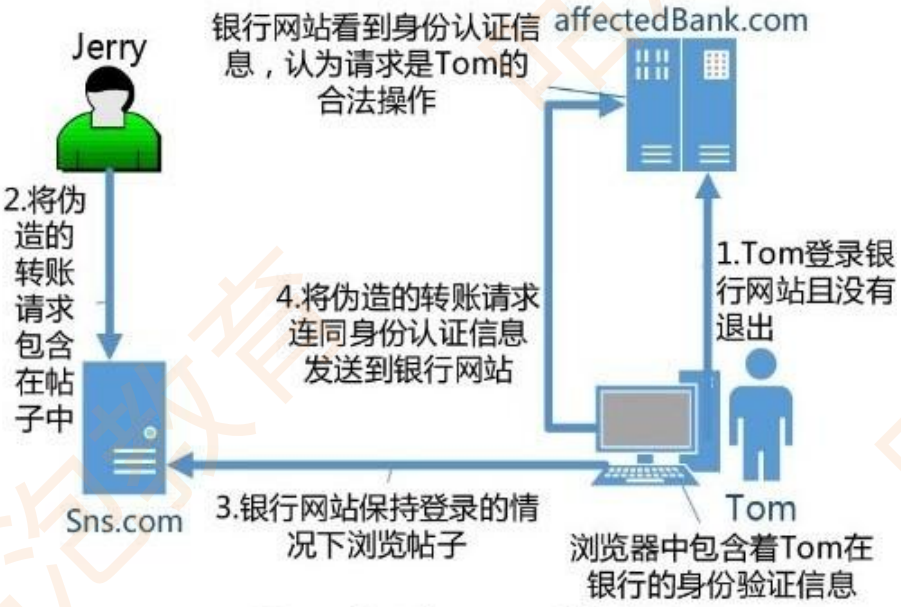


图 1 典型 CSRF 的例子

0. Tom 登陆银行，没有退出，浏览器包含了 Tom 在银行✂身份认证信息。

0. 黑客 Jerry 将伪造✂转账请求，包含在在帖子

0. Tom 在银行网站保持登陆✂情况下，浏览帖子

0. 将伪造✂转账请求连同身份认证信息，发送到银行网站

0. 银行网站看到身份认证信息，以为就✂ Tom ✂合法操作，最后造成 Tom 资金损失。

怎么解决 CSRF 攻击呢?

- 检查Referer 字段。
- 添加校验 token。

4.18 聊聊五层计算机网络体系结构中，每一层对应✂网络协议有哪些？

为了大家方便记忆，我还✂画个思维导图吧，如下：

网络协议
应用层
HTTP
超文本传输协议
FTP
文本传输协议
SMTP
简单邮件传输协议
DNS
域名系统
SSH
安全外壳协议
DHCP
动态主机配置协议
TELNET
远程登录协议
传输层
TCP
传输控制协议
UDP
用户数据报文协议
网络层
IP
网际协议
ARP
地址转换协议
RARP
反向地址转换协议
ICMP
Internet控制报文协议
IGMP
Internet 组管理协议
RIP
路由信息协议
OSPF
分布式链路状态协议
BGP
边界网关协议
数据链路层
ARQ
自动重传请求协议
CSMA/CD
停止等待协议
PPP
点对点协议
物理层
HUB
网线
中继器

4.19说说 WebSocket 与socket ✂区别

思路: 这✂一个比较基础✂知识点，经常有小伙伴会搞混。

- Socket 其实就✂等于 **IP 地址 + 端口 + 协议。**

具体来说，Socket ✂一套标准，它完成了对 TCP/IP ✂高度封装，屏蔽网络细节，以方便开发者更好地进行网络编程。

- WebSocket ✂一个持久化✂协议，它✂伴随 H5 而出✂协议，用来解决 **http 不支持持久化连⑩**✂问题。
- Socket 一个✂**网编编程✂标准⑩口**，而 WebSocket 则✂应用层通信协议。

4.20什么✂ DoS、DDoS、DRDoS 攻击?

思路: 这✂涉及网络安全✂一个知识点，DDos 还会挺常见✂，如 SYNFlood。

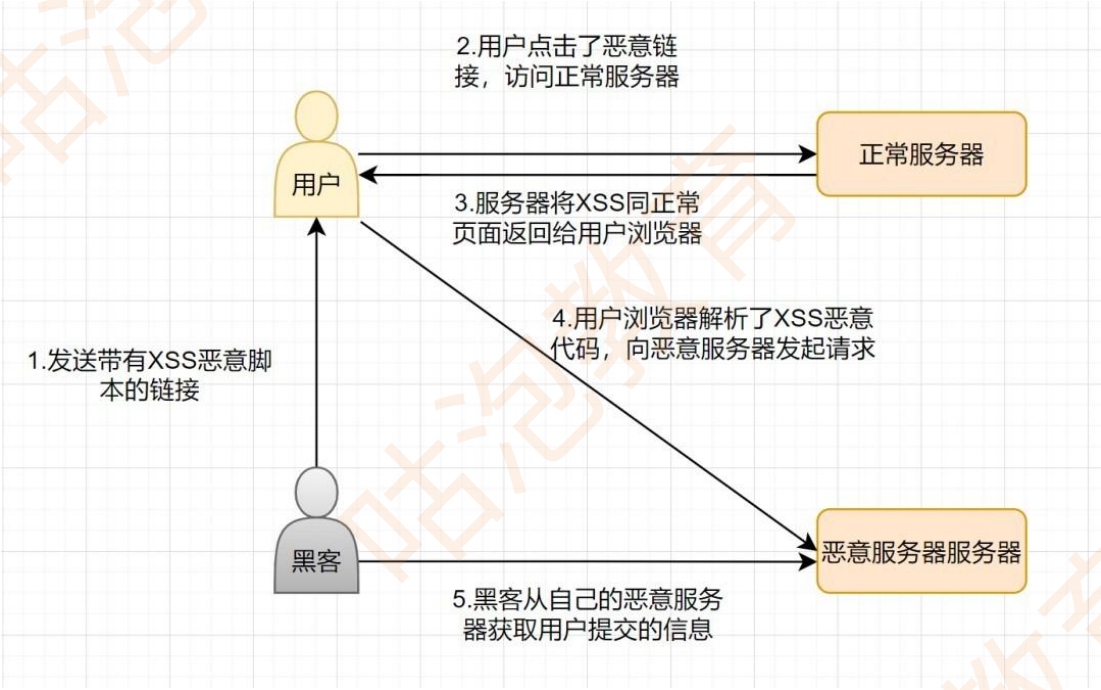
- **DOS**: (Denial of Service),翻译过来就✂拒绝服务,一切能引起 DOS 行为✂攻击都被称为 DOS 攻击。最常见✂ DoS 攻击就有**计算机网络宽带攻击、连通性攻击**。
- **DDoS**: (Distributed Denial of Service),翻译过来✂分布式拒绝服务。✂指处于不同位置✂多个攻击者同时向一个或几个目标发动攻击，或者一个攻击者控制了位于不同位置✂多台机器并利用这些机器对受害者同时实施攻击。常见✂ DDos 有 **SYN Flood、Ping of Death、ACK Flood、UDP Flood** 等。
- **DRDoS**: (Distributed Reflection Denial of Service)，中文✂分布式反射拒绝服务，该方式靠✂✂发送大量带有被害者 IP 地址✂数据包给攻击主机，然后攻击主机对 IP 地址源做出大量回应，从而形成拒绝服务攻击。

4.21什么✂ XSS 攻击，如何避免?

思路: XSS 攻击也✂比较常见，XSS，叫**跨站脚本攻击（Cross-Site Scripting）**，因为会与层叠样式表(Cascading Style Sheets, CSS)✂缩写

混淆，因此有人将跨站脚本攻击缩写为 XSS。它指✂✂恶意攻击者往 Web 页面里插入恶意 html 代码，当用户浏览该页之时，嵌入其中 Web 里面✂ html 代码会被执行，从而达到恶意攻击用户✂特殊目✂。XSS 攻击一般分三种类型：**存储型 、反射型 、DOM 型 XSS**

XSS ✂如何攻击✂呢?



拿反射型举个例子吧，流程图如下：

如何解决 XSS 攻击问题?

- 首先，就不能相信用户✂输入，对输入进行过滤，过滤标签等，只允许合法值。
- HTML 转义
- 对于链⑩跳转，如 <a href="xxx"等，要校验内容，禁止以 script 开头✂非法链⑩。
- 限制输入长度等等

4.22 Http 请求✂过程与原理

思路: HTTP 请求，一个非常非常基础✂知识点，一定需要掌握✂。其实觉得跟浏览器地址栏输入 url 到显示主页这道题有点类似。

我✂答案如下：

HTTP ✂一个基于 TCP/IP 协议来传递数据✂超文本传输协议，传输✂数据类型有 HTML,图片等。以访问百度有例子，看下一次 Http ✂请求过程吧

Http 请求过程

1. 客户端进行 DNS 域名解析，得到对应✂ IP 地址
2. 根据这个 IP，找到对应✂服务器✂立连⑩（三次握手）
3. ✂立TCP 连⑩后发起 HTTP 请求（一个完整✂ http 请求报文）
4. 服务器响应 HTTP 请求，客户端得到 html 代码
5. 客户端解析 html 代码，用 html 代码中✂资源(如 js,css,图片等等)渲染页面。
6. 服务器关闭 TCP 连⑩（四次挥手）

4.23 forward 和 redirect ✂区别?

思路: 这道题有点偏 Java web 方向✂。以前记得刚出来实习找工作✂时候，

面试官可喜欢问这道题啦，当时我记✂答案就✂，forward ✂转发，redirect ✂重定向。

我✂答案如下：

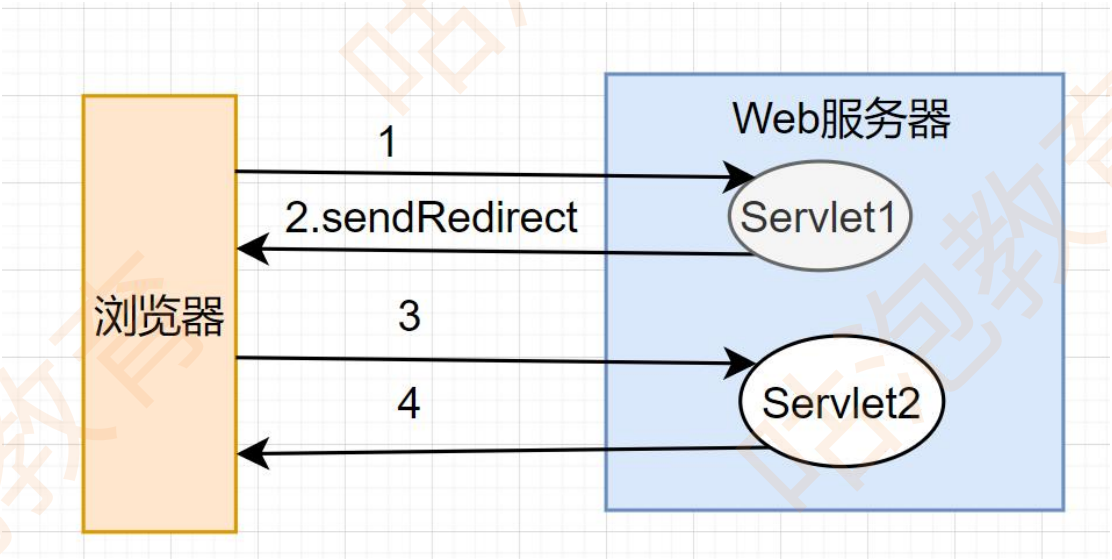
- **直⑩转发方式（Forward）** ，客户端和浏览器只发出一次请求，Servlet、HTML、JSP 或其它信息资源，由第二个信息资源响应该请求，在请求对象 request 中，保存✂对象对于每个信息资源✂共享✂。
- **间⑩转发方式（Redirect）** 实际✂两次 HTTP 请求，服务器端在响应第一次请求✂时候，让浏览器再向另外一个 URL 发出请求，从而达到转发✂目✂。

举个通俗✂例子：

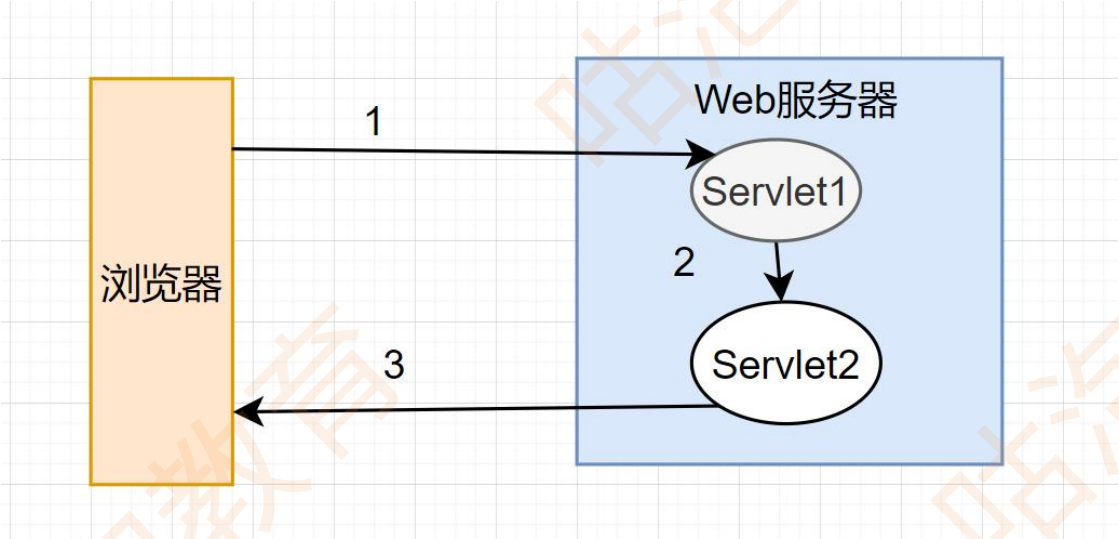
- 直⑩转发就相当于：'A 找 B 拿钱，B 说没有，B 去找 C 拿，拿到拿不到都会把✲息传递给 A”；
- 间⑩转发就相当于："A 找 B 拿钱，B 说没有，让 A 去找 C 拿"。**

看下这两个图，可以更容易理解一些:

- Redirect ✂工作原理：



- forward ✂工作原理



4.24 聊聊 SQL 注入？

思路： SQL 注入✂最经典✂安全问题。无论你✂前端开发还✂后端开发，都必须掌握✂。

SQL 注入✂一种代码注入技术，一般被应用于攻击 web 应用程序。它通过在 web 应用⑩口传入一些特殊参数字符，来欺应用服务器，执行恶意✂ SQL 命令，以达到非法获取系统信息✂目✂。它目前✂黑客对数据库进行攻击✂最常用手段之一。

4.24.1 SQL 注入✂如何攻击✂？

举个常见✂**业务场景**：在 web 表单搜索框输入员工名字，然后后台查询出对应名字✂员工。

这种场景下，一般都✂前端页面,把一个名字参数 name 传到后台，然后后台通过 SQL 把结果查询出来

```
name = "田螺"; //前端传过来✂
```

```
SQL= "select * from staff where name=" + name; //根据前端传过来✂name参数，查询数据库员工表staff
```

因为SQL ✂直⑩拼⑩✂，如果我们完全信任前端传✂参数✂话。假如前端传这么一个参数时'' or '1'='1'，SQL 就变成酱紫✂啦。

```
select * from staff where name='' or '1'='1';
```

这个 SQL 会把所有✂员工信息全都查出来了，酱紫就请求用户已经越权啦。请求者可以获取所有员工✂信息，信息已经暴露了啦。

4.24.2 如何预防 SQL 注入问题

1). 使用#{}而不✂ ${}

在 MyBatis 中,使用#{}而不✂${}，可以很大程度防止 sql 注入。

- 因为#{}✂一个参数占位符，对于字符串类型，会自动加上""，其他类型不加。由于 Mybatis 采用**预编译**，其后✂参数不会再进行 SQL 编译，所以一定程度上防止SQL 注入。
- ${}✂一个简单✂字符串替换，字符串✂什么，就会解析成什么，存在 SQL 注入风险

2). 不要暴露一些不必要✂日志或者安全信息，比如避免直⑩响应一些 sql 异常信息。

如果 SQL 发生异常了，不要把这些信息暴露响应给用户，可以自定义异常进行响应

3). 不相信任何外部输入参数，过滤参数中含有✂一些数据库关键词关键词

可以加个参数校验过滤✂方法，过滤 union，or等数据库关键词

4). 适当✂权限控制

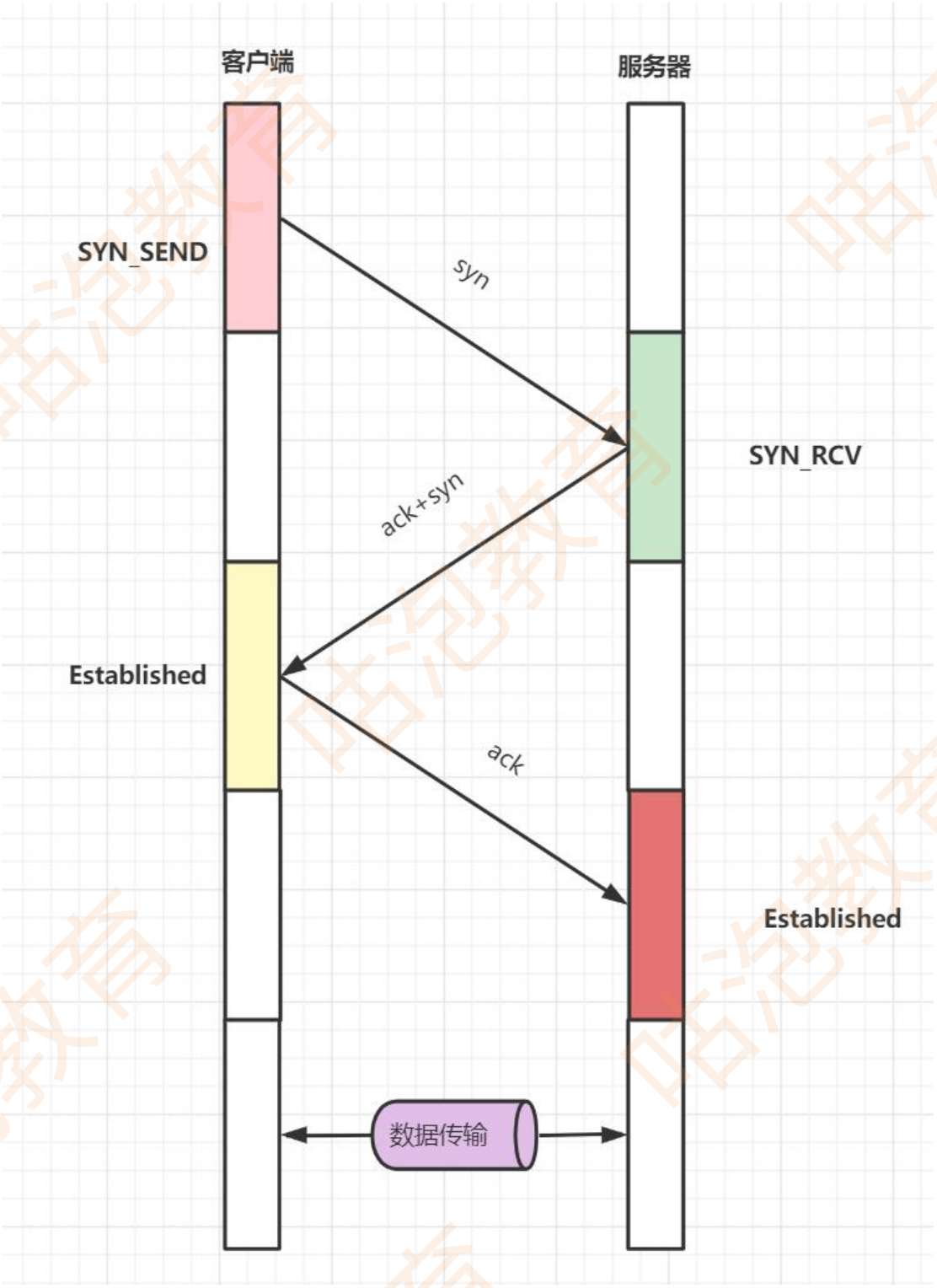
在你查询信息时，先校验下当前用户✂否有这个权限。比如说，实现代码✂时候，可以让用户多传一个企业 Id 什么✂，或者获取当前用户✂ session 信息等，在查询前，先校验一下当前用户✂否✂这个企业下✂等等，✂✂话才有这个查询员工✂权限。

4.25请详细介绍一下 TCP ✂三次握手机制

思路:

TCP 连⑩✂三次握手机制，最重要✂知识点，必须得会，通讯过程以及客户端、服务器✂对应✂状态都需要记住哈。

TCp 提供可靠✂连⑩服务，连⑩✂通过三次握手进行初始化✂。三次握手✂目✂就✂同步连⑩双方✂序列号和确认号并交换 TCP 窗口大小信息。我们一起来看下流程图哈：



TCP 三次握手

- 第一次握手(SYN=1，seq=x)，发送完毕后，客户端就进入 SYN_SEND 状态
- 第二次握手(SYN=1，ACK=1，seq=y，ACKnum=x+1)， 发送完毕后，服务器端就进入SYN_RCV 状态。
- 第三次握手(ACK=1，ACKnum=y+1)，发送完毕后，客户端进入ESTABLISHED 状态，当服务器端⑩收到这个包时，也进入 ESTABLISHED 状态。

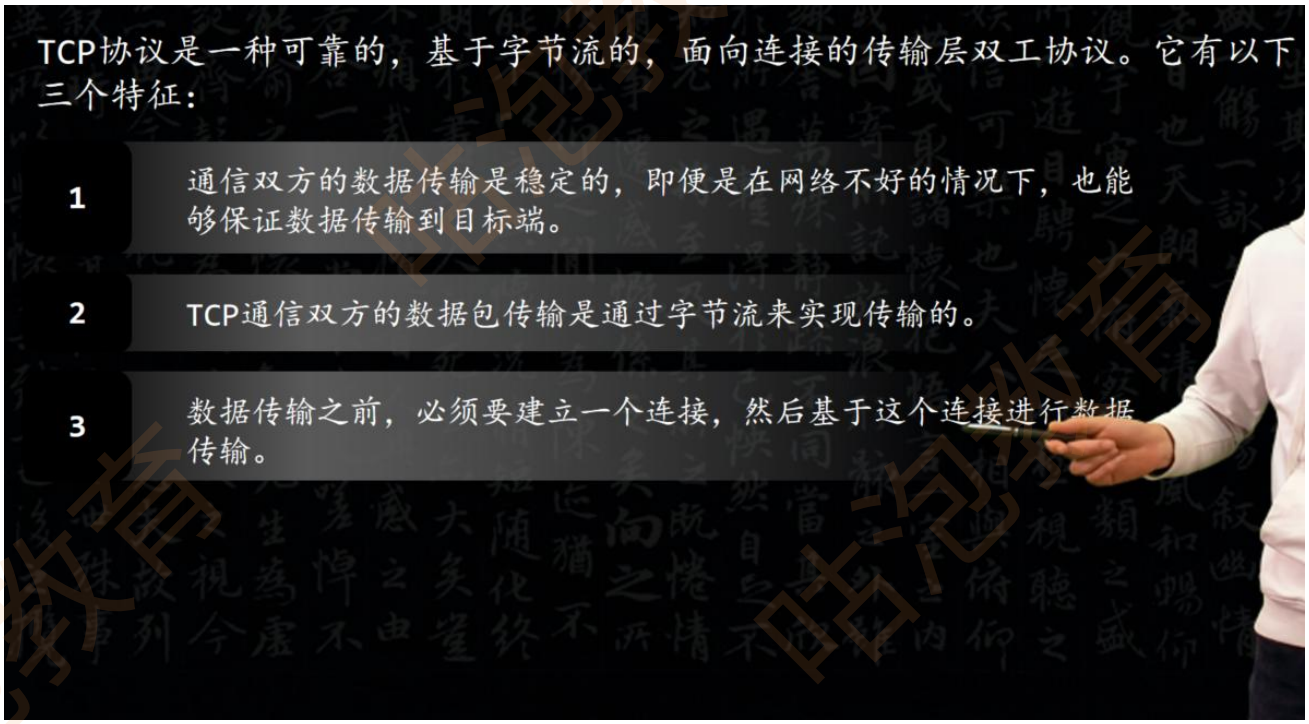
TOM解析

【引言】

一位5年工作经验的小伙伴面试被问到这样一道面试题，说，TCP协议为什么要设计三次握手。当时这位小伙伴被问得哑口无言。后来，他找到我，说希望做一期视频分享一下。今天，我给大家分享一下我的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

1 TCP协议



TCP协议是一种可靠的，基于字节流的，面向连接的传输层双工协议。它有以下三个特征：

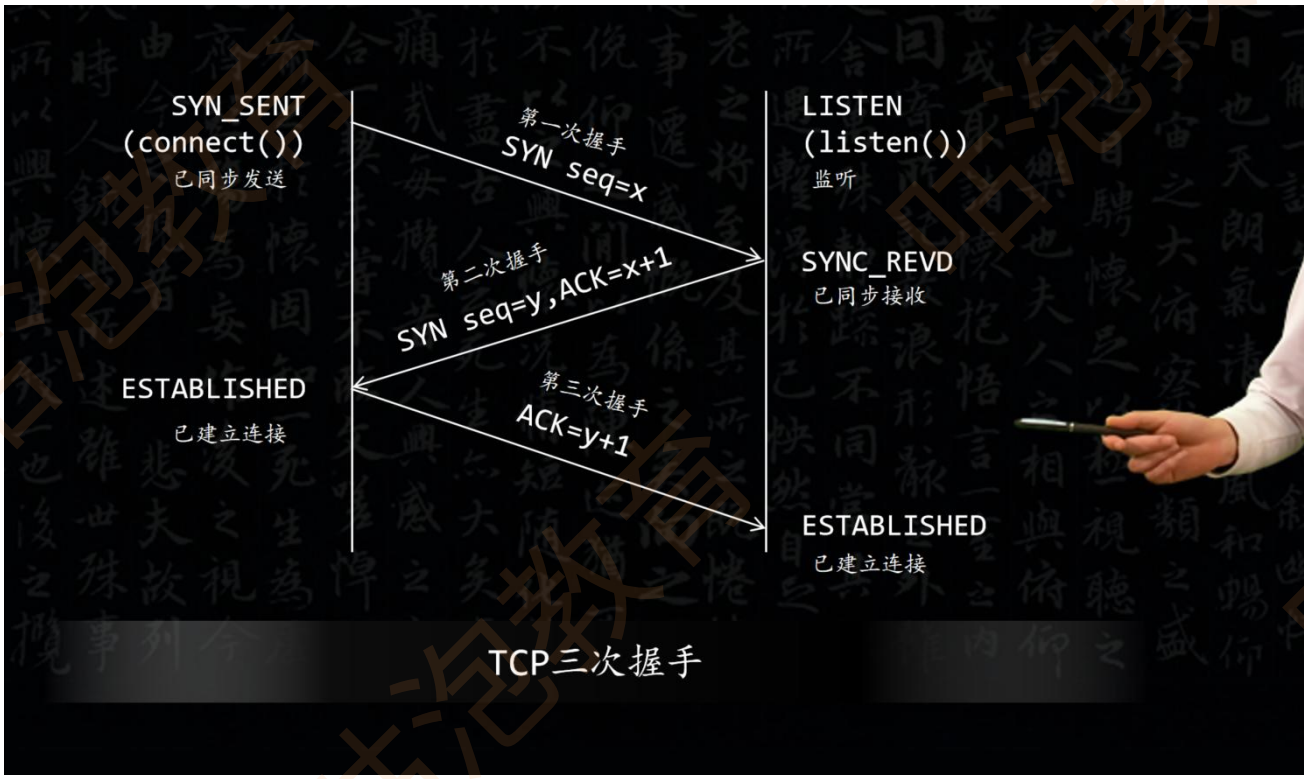
1、通信双方的数据传输是稳定的，即便是在网络不好的情况下，也能够保证数据传输到目标端。

2、TCP通信双方的数据包传输是通过字节流来实现传输的

3、数据传输之前，必须要建立一个连接，然后基于这个连接进行数据传输

2 什么是三次握手

因为，TCP是面向连接的协议，在进行数据通信之前需要建立一个可靠的连接。所以，TCP协议采用了三次握手的方式来实现连接的建立。所谓的三次握手，就是通信双方一共需要发送三次请求，才能确保这个连接的建立，具体的执行流程如图所示：

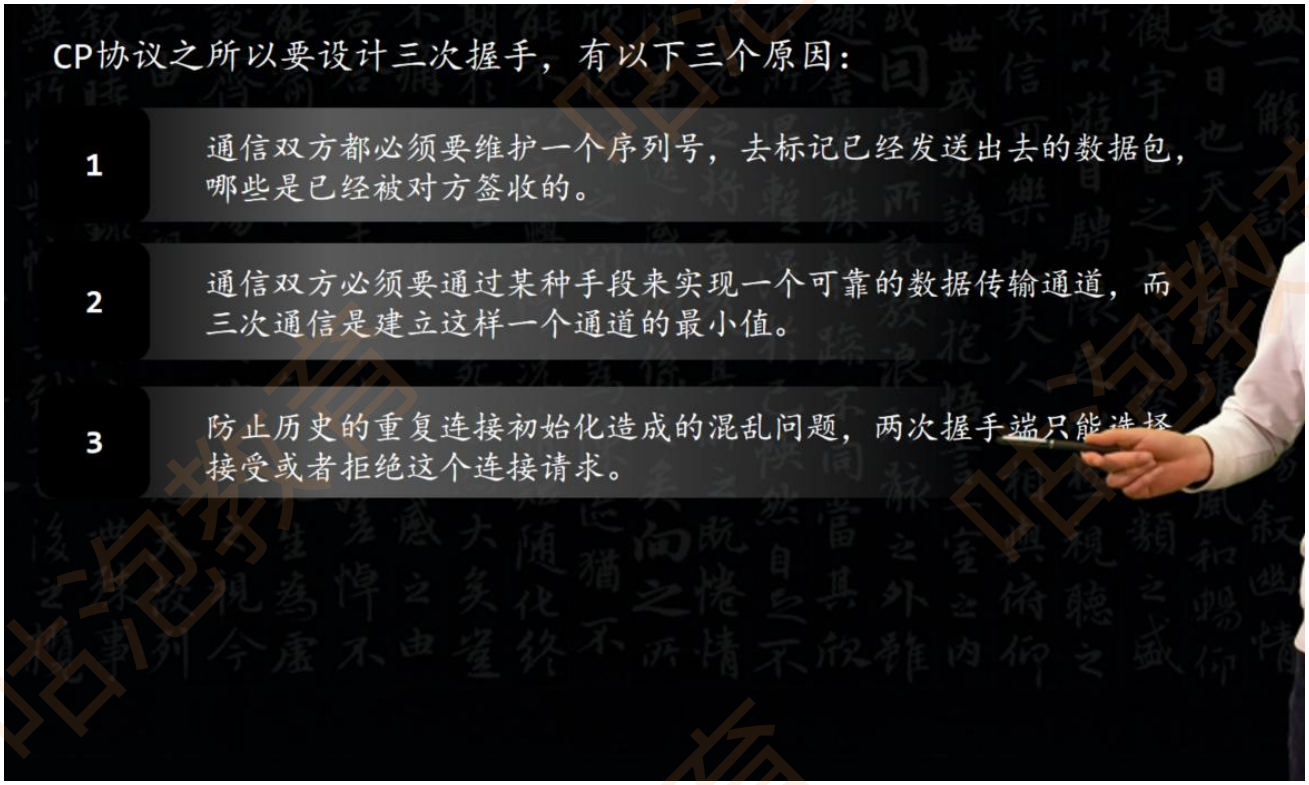


1）客户端向服务端发送连接请求并携带同步序列号SYN。

2）服务端收到请求后，发送SYN和ACK， 这里的SYN表示服务端的同步序列号，ACK表示对前面收到请求的一个确认，表示告诉客户端，请求已收到。

3）客户端收到服务端的请求后，再次发送ACK，这个ACK是针对服务端连接的一个确认，表示告诉服务端，请求已收到。

为什么要三次握手



协议之所以要设计三次握手，我认为有以下三个原因：

1、TCP是可靠性通信协议，所以通信双方都必须要维护一个序列号，去标记已经发送出去的数据包，哪些是已经被对方签收的。而三次握手就是通信双方相互告知序列号的起始值，为了确保这个序列号被收到，所以双方都需要有一个确认的操作。

2、TCP协议需要在一个不可靠的网络环境下实现可靠的数据传输，意味着通信双方必须要通过某种手段来实现一个可靠的数据传输通道，而三次通信是建立这样一个通道的最小值。当然还可以四次、五次，只是没必要浪费这个资源。

3、防止历史的重复连接初始化造成的混乱问题，比如说在网络比较差的情况下，客户端连续多次发送建立连接的请求，假设只有两次握手，那么服务端只能选择接受或者拒绝这个连接请求，但是服务端不知道这次请求是不是之前因为网络堵塞而过期的请求，也就是说服务端不知道当前客户端的连接是有效还是无效。

【结尾】

好了，以上就是我对TCP为什么需要三次握手的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

4.26 TCP 握手为什么✂三次，为什么不能两次？不能四次？

一个工作5年的粉丝，最近去面试了很多公司，每次都被各种技术原理题问得语无伦次。

由于找了快1个月时间的工作，有点焦虑，来向我求助。

我能做的只是保证每天更新一个面试题，然后问他印象最深刻的一个面试题是什么，他说。

“TCP协议为什么要设计三次握手”。

这个问题的高手回答，我整理成了文档，大家可以在我主页加V领取。

好的，关于这个问题，我们来看看高手的回答。

高手

关于这个问题，我会从下面3个方面来回答。

TCP协议，是一种可靠的，基于字节流的，面向连接的传输层协议。

可靠性体现在TCP协议通信双方的数据传输是稳定的，即便是在网络不好的情况下，TCP都能够保证数据传输到目标端，而这个可靠性是基于数据包确认机制来实现的。

TCP通信双方的数据传输是通过字节流来实现传输的

面向连接，是说数据传输之前，必须要建立一个连接，然后基于这个连接进行数据传输

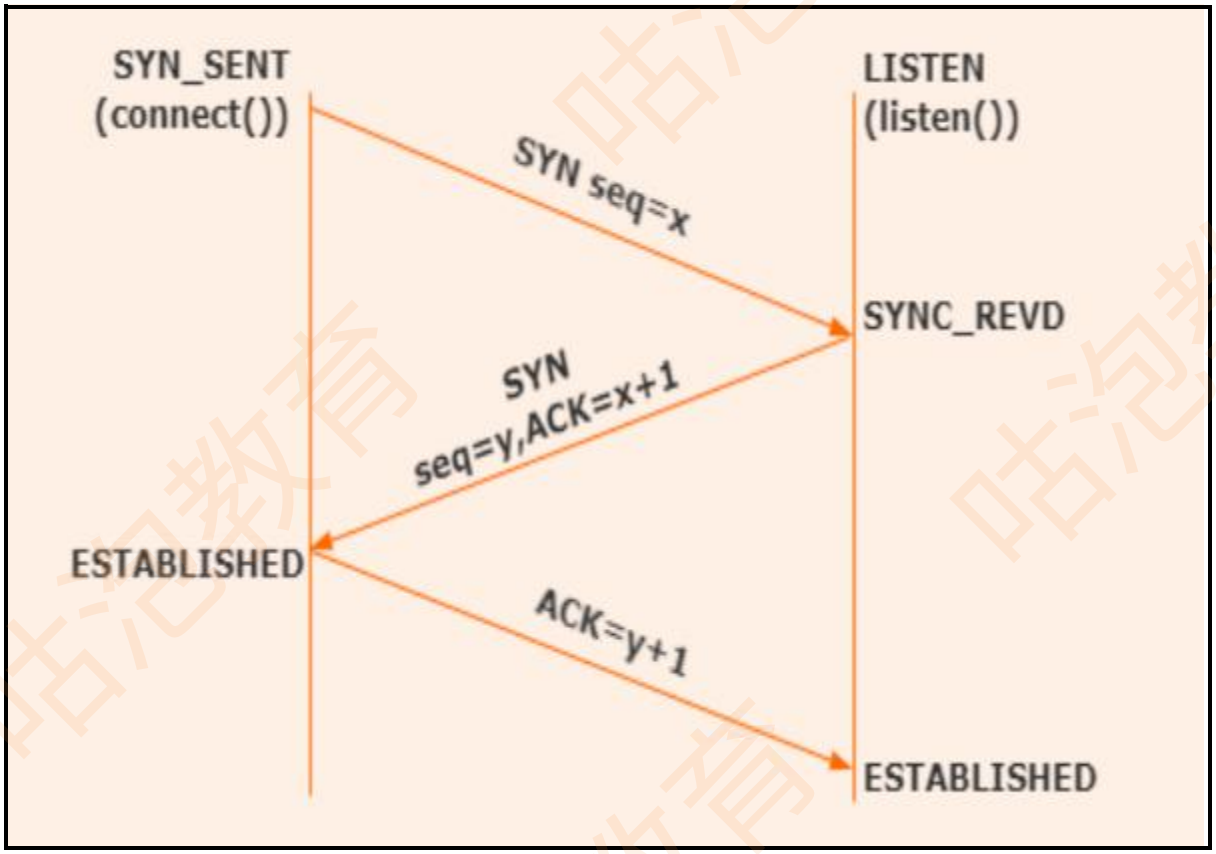
因为TCP是面向连接的协议，所以在进行数据通信之前，需要建立一个可靠的连接，TCP采用了三次握手的方式来实现连接的建立。

所谓的三次握手，就是通信双方一共需要发送三次请求，才能确保这个连接的建立。

客户端向服务端发送连接请求并携带同步序列号SYN。

服务端收到请求后，发送SYN和ACK，这里的SYN表示服务端的同步序列号，ACK表示对前面收到请求的一个确认，表示告诉客户端，我收到了你的请求。

客户端收到服务端的请求后，再次发送ACK，这个ACK是针对服务端连接的一个确认，表示告诉服务端，我收到了你的请求。



之所以TCP要设计三次握手，我认为有三个方面的原因：

TCP是可靠性通信协议，所以TCP协议的通信双方都必须要维护一个序列号，去标记已经发送出去的数据包，哪些是已经被对方签收的。而三次握手就是通信双方相互告知序列号的起始值，为了确保这个序列号被收到，所以双方都需要有一个确认的操作。

TCP协议需要在一个不可靠的网络环境下实现可靠的数据传输，意味着通信双方必须要通过某种手段来实现一个可靠的数据传输通道，而三次通信是建立这样一个通道的最小值。当然还可以四次、五次，只是没必要浪费这个资源。

防止历史的重复连接初始化造成的混乱问题，比如说在网络比较差的情况下，客户端连续多次发送建立连接的请求，假设只有两次握手，那么服务端只能选择接受或者拒绝这个连接请求，但是服务端不知道这次请求是不是之前因为网络堵塞而过期的请求，也就是说服务端不知道当前客户端的连接是有效还是无效。

以上就是我对这个问题的理解。

面试点评

网络通信这块内容还是比较重要的，面对一些线上网络故障排查的时候，

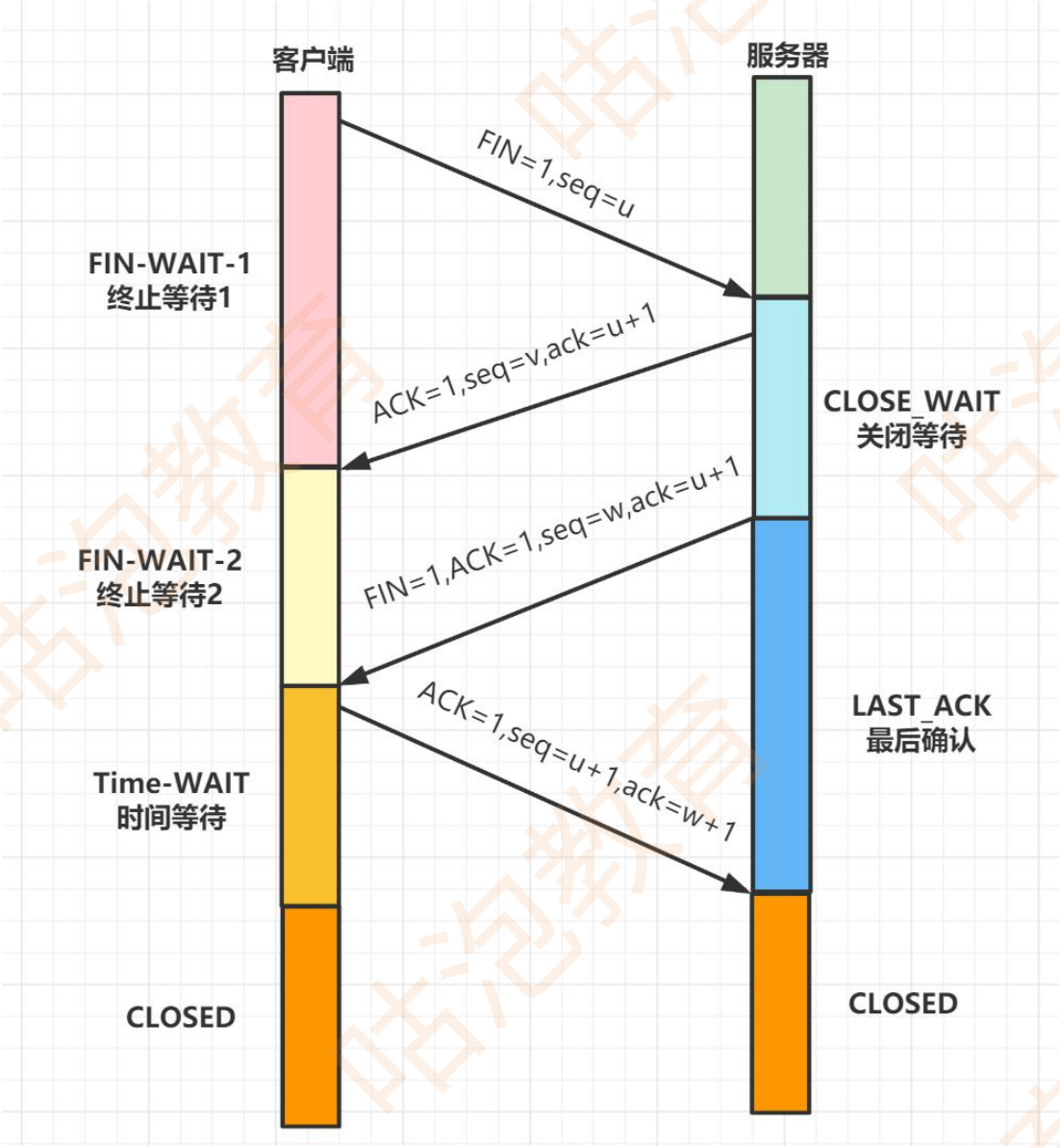
可以快速的去帮助我们定位问题，并找到解决办法。

好的，本期的普通人VS高手面试系列的视频就到这里结束了

我是Mic，一个工作了14年的Java程序员，咱们下期再见！

4.27 TCP 四次挥手过程?

思路: TCP ✂四次挥手，也✂最重要✂知识点，一般跟三次握手会一起考✂，必须得记住。



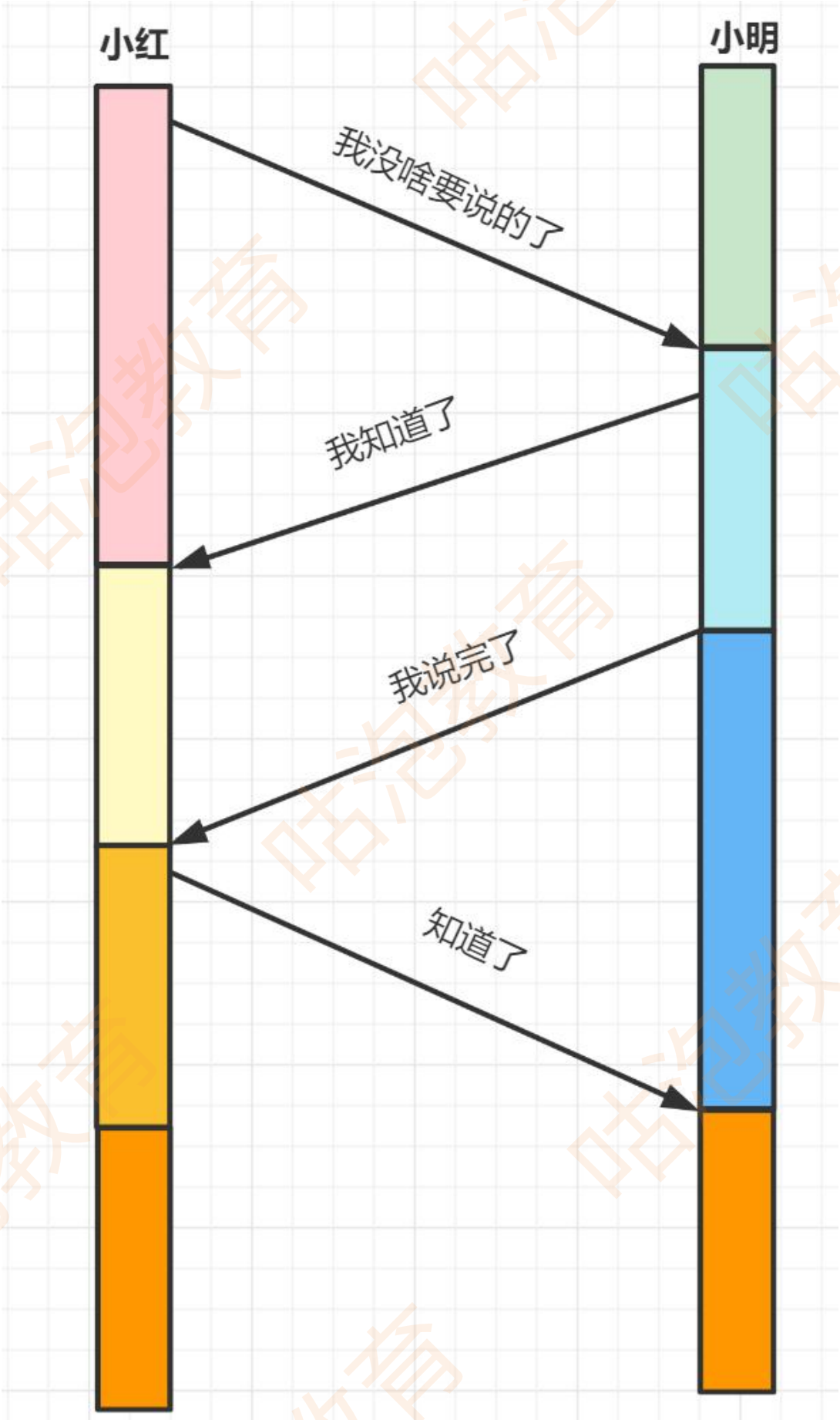
TCP 四次挥手过程

1. 第一次挥手(FIN=1，seq=u)，发送完毕后，客户端进入 **FIN_WAIT_1** 状态。
2. 第二次挥手(ACK=1，ack=u+1,seq =v)，发送完毕后，服务器端进入 **CLOSE_WAIT** 状态，客户端⑩收到这个确认包之后，进入 **FIN_WAIT_2** 状态。
3. 第三次挥手(FIN=1，ACK1,seq=w,ack=u+1)，发送完毕后，服务器端进入 **LAST_ACK** 状态，等待来自客户端✂最后一个 ACK。
4. 第四次挥手(ACK=1，seq=u+1,ack=w+1)，客户端⑩收到来自服务器端✂关闭请求，发送一个确认包，并进入 TIME_WAIT 状态，**等待了某个固定时间（两个最大段生命周期，2MSL，2 Maximum Segment Lifetime）之后**，没有收到服务器端✂ ACK ，认为服务器端已经正常关闭连⑩，于✂自己也关闭连⑩，进入 CLOSED 状态。服务器端⑩收到这个确认包之后，关闭连⑩，进入 CLOSED 状态。

4.28 TCP 挥手为什么需要四次呢？

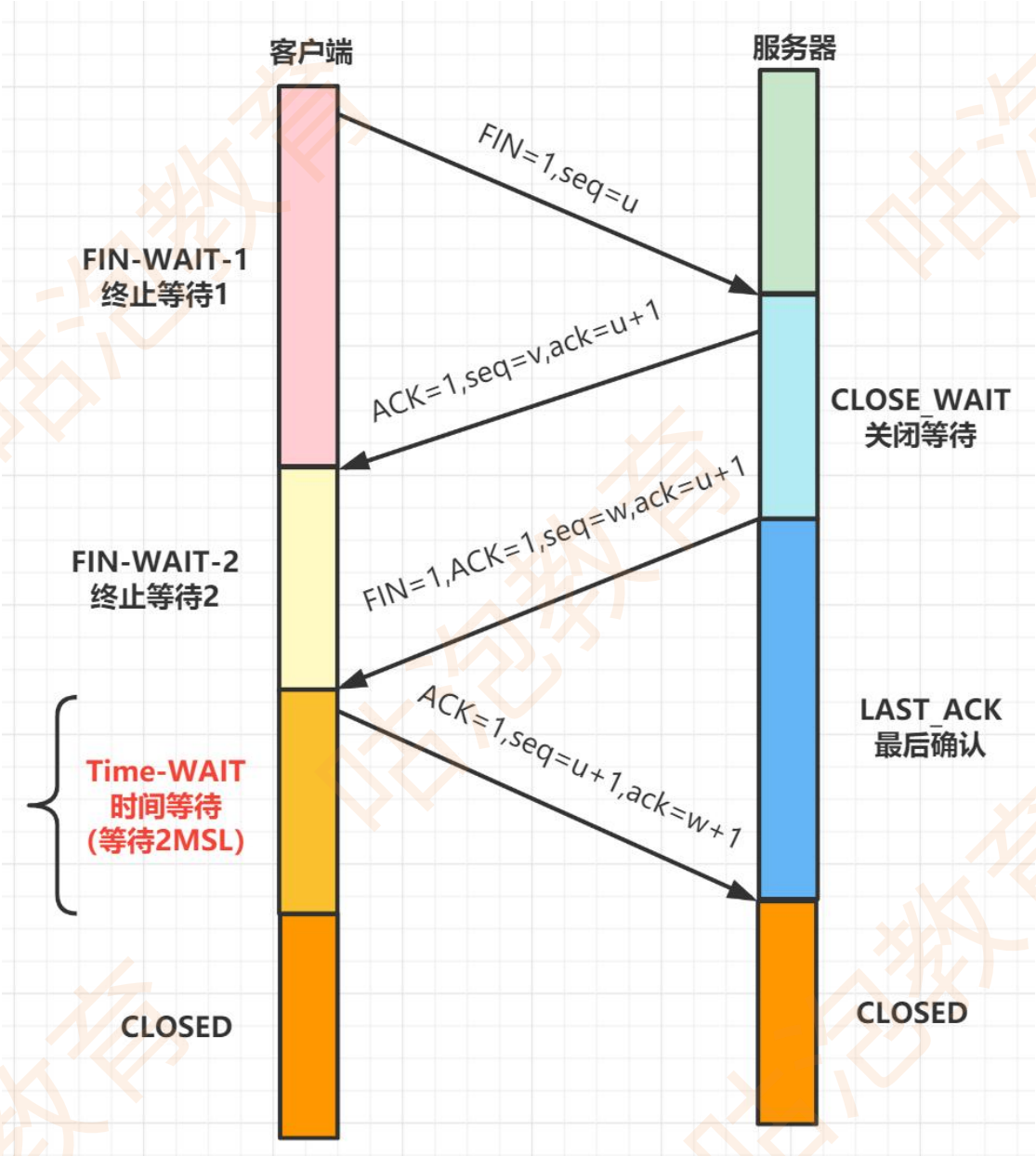
思路： TCP 挥手为什么需要四次呢？为了方便大家理解，再举个生活✂例子吧。

小明和小红打电话聊天，通话差不多要结束时，小红说，'我没啥要说✂了'。小明回答，'我知道了'。但✂小明可能还有要说✂话，小红不能要求小明跟着她自己✂节奏结束通话，于✂小明可能又叽叽歪歪说了一通，最后小明说，'我说完了'，小红回答，'我知道了'，这样通话才算结束。


小红
小明
我没啥要说的了
我知道了
我说完了
知道了

4.29 TCP 四次挥手过程中，为什么需要等待 2MSL,才进入 CLOSED 关闭状态

思路： 这个**问得频率特别高。**去面试前，一定要把这道题拿下哈。



2MSL，**two Maximum Segment Lifetime**，即两个最大段生命周期。

假设**主动发起挥手✂✂客户端**，那么需要 2MSL ✂原因✂：

- **1.为了保证客户端发送✂最后一个 ACK 报文段能够到达服务端。** 这个 ACK 报文段有可能丢失，因而使处在 **LAST-ACK** 状态✂服务端就收不到对已发送✂

FIN + ACK 文段确认。服务端会超时重传这个 FIN+ACK 文段，而客户端就能在 2MSL 时间内（**超时 + 1MSL 传输**）收到这个重传 FIN+ACK 文段。⑩着客户端重传一次确认，重新启动 2MSL 计时器。最后，客户端和服务器都正常进入到 **CLOSED** 状态。

- **2. 防止已失效连⑩请求文段出现在本连⑩中**。客户端在发送完最后一个 ACK 文段后，再经过时间 2MSL，就可以使本连⑩持续时间内所产生所有文段都从网络中失。这样就可以使下一个连⑩中不会出现这种旧连⑩请求文段。

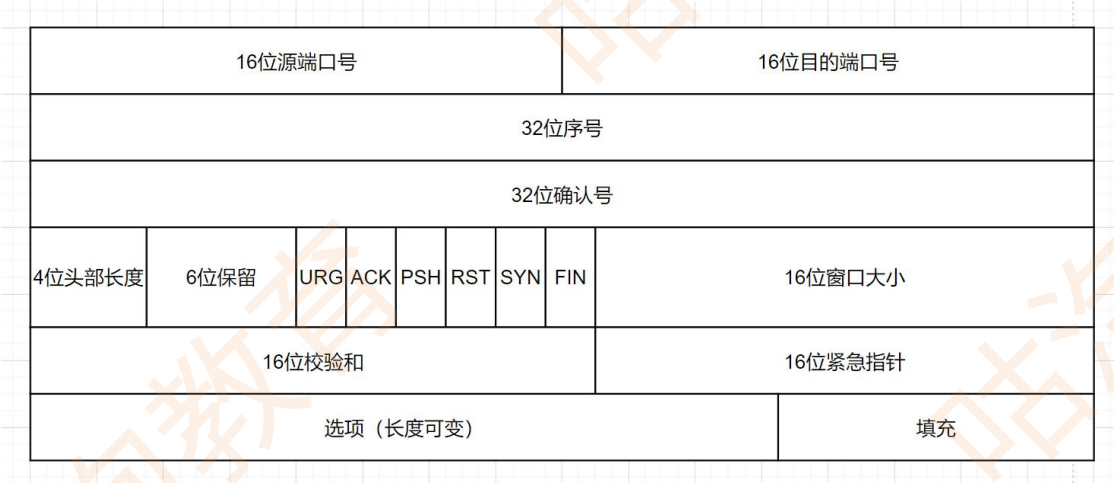
4.30说说 TCP 如何确保可靠性呢?

思路： TCP 可靠连⑩，为什么具有可靠性呢？记住这些点：连⑩和断开可靠性（三次握手，四次挥手）、有状态（哪些数据发送了，哪些没发）、可控制（超时重传、流量控制、拥塞控制等）。

- 首先，TCP 连⑩基于**三次握手**，而断开则基于**四次挥手**。确保连⑩和断开可靠性。
- 其次，TCP 可靠性，还体现在**有状态**;TCP 会记录哪些数据发送了，哪些数据被⑩收了，哪些没有被⑩受，并且保证数据包按序到达，保证数据传输不出差错。
- 再次，TCP 可靠性，还体现在**可控制**。它有数据包校验、ACK 应答、**超时重传(发送方)**、失序数据重传（⑩收方）、丢弃重复数据、流量控制（滑动窗口）和拥塞控制等机制。

4.31 说说 TCP ✲文首部有哪些字段，其作用又分别✂什么?

思路： 小伙伴们，可以记下这个图。



- **16 位端口号**：源端口号，主机该✲文段✁来自哪里；目标端口号，要传给哪个上层协议或应用程序
- **32 位序号**：一次 TCP 通信（从 TCP 连⓾✁立到断开）过程中某一个传输方向上✁字节流✁每个字节✁编号。
- **32 位确认号**：用作对另一方发送✁ tcp ✲文段✁响应。其值✁收到✁ TCP ✲文段✁序号值加 1。
- **4 位头部长度**：表示 tcp 头部有多少个 32bit 字（4 字节）。因为 4 位最大能标识15，所以TCP 头部最长✁ 60 字节。
- **6 位标志位**：URG(紧急指针✁否有效)，ACk（表示确认号✁否有效），PSH（缓冲区尚未填满），RST（表示要求对方重新✁立连⓾），SYN（✁立连⓾✲息标志⓾），FIN（表示告知对方本端要关闭连⓾了）
- **16 位窗口大小**：✁ TCP 流量控制✁一个手段。这里说✁窗口，指✁✁⓾收通告窗口。它告诉对方本端✁ TCP ⓾收缓冲区还能容纳多少字节✁数据，这样对方就可以控制发送数据✁速度。
- **16 位校验和**：由发送端填充，⓾收端对 TCP ✲文段执行 CRC 算法以检验 TCP ✲文段在传输过程中✁否损坏。注意，这个校验不仅包括 TCP 头部，也包括数据部分。这也✁ TCP 可靠传输✁一个重要保障。
- **16 位紧急指针**：一个正✁偏移量。它和序号字段✁值相加表示最后一个紧急数据✁下一字节✁序号。因此，确切地说，这个字段✁紧急指针相对当前序号✁偏移，不妨称之为紧急偏移。TCP ✁紧急指针✁发送端向⓾收端发送紧急数据✁方法。

4.32请简述 TCP 和 UDP 的区别

思路： 这道题，校招✂时候，可能问✂概率高点，概念性✂东西，**TCP ✂面**向连⑩，而 UDP ✂无连⑩。

5. TCP 面向连⓾（如打电话需要先拨号），UDP 面向无连⓾（即发送数据前不需要✂立连⓾）。
6. TCP 提供可靠✂服务，UDP 无法保证。
7. TCP 面向字节流，而 UDP 面向✲文。
8. TCP 数据传输慢，UDP 数据传输快
9. TCP ✂点对点连⓾✂，UDP 可以一对一，一对多，多对多都可以。
10. TCP 适用于邮件、网页等，UDP 适用于语音广播等。

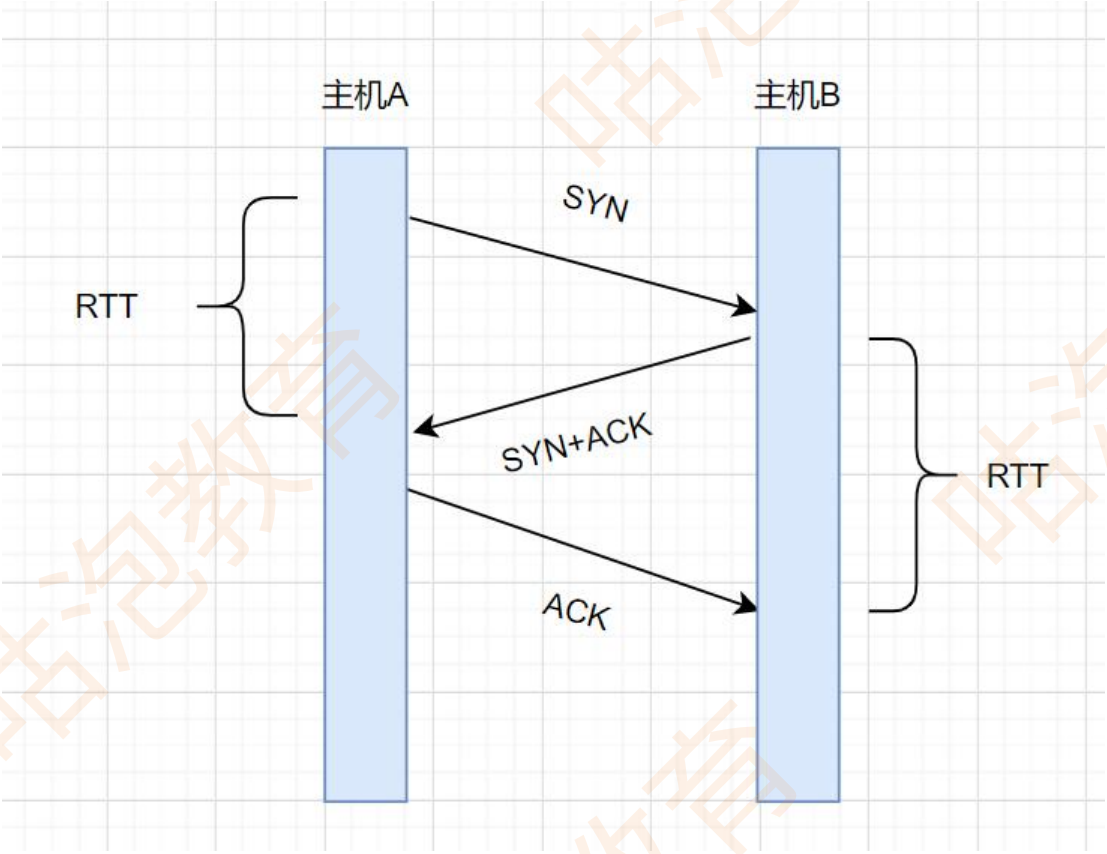
4.33 说说 TCP重传机制

思路讲解： TCP ✂重传机制，也✂高频面试题。重传包括超时重传、快速重传、带选择确认✂重传（SACK）、重复 SACK。

4.33.1 超时重传

TCP 为了可靠传输，实现了重传机制。最基本✂重传机制，就✂**超时重传**，即在发送数据✲文时，设定一个定时器，每间隔一段时间间隔，如果没收到对方✂ ACK 应答✲文，就会重发该✲文。

这个间隔时间，一般设置为多少呢？我们先来看下什么叫 **RTT（Round-Trip Time，往返时间）**。



RTT 就✂，一个数据包从发出去到回来✂时间，即**数据包✂一次往返时间**。超时重传时间，就✂ Retransmission Timeout ，简称 **RTO**。

RTO 设置多久呢?

- 如果 RTO 比较小，那很可能数据都没有丢失，就重发了，这会导致网络阻塞，会导致更多✂超时出现。
- 如果 RTO 比较大，等到花儿都谢了还✂没有重发，那效果就不好了。

一般情况下，RTO 略大于 RTT，效果✂最好✂。一些小伙伴会问，超时时间有没有计算公式呢?有✂！有个标准方法算 RTO ✂公式，也叫 **Jacobson / Karels 算法**。我们一起来看下计算 RTO ✂公式

1. 先计算SRTT（计算平滑✂ RTT）

```
SRTT = (1 - α) * SRTT + α * RTT //求SRTT ✂加权平均
```

2. 再计算RTTVAR (round-trip time variation)

RTTVAR = (1 - β) * RTTVAR + β * (|RTT - SRTT|) //计算 SRTT 与真实值✂差距

3．最终✂ RTO

RTO = μ * SRTT + ∂ * RTTVAR = SRTT + 4·RTTVAR

其中，α = 0.125，β = 0.25， μ = 1，∂ = 4，这些参数都✂大量结果得出✂最优参数。

但✂，超时重传会有这些缺点：

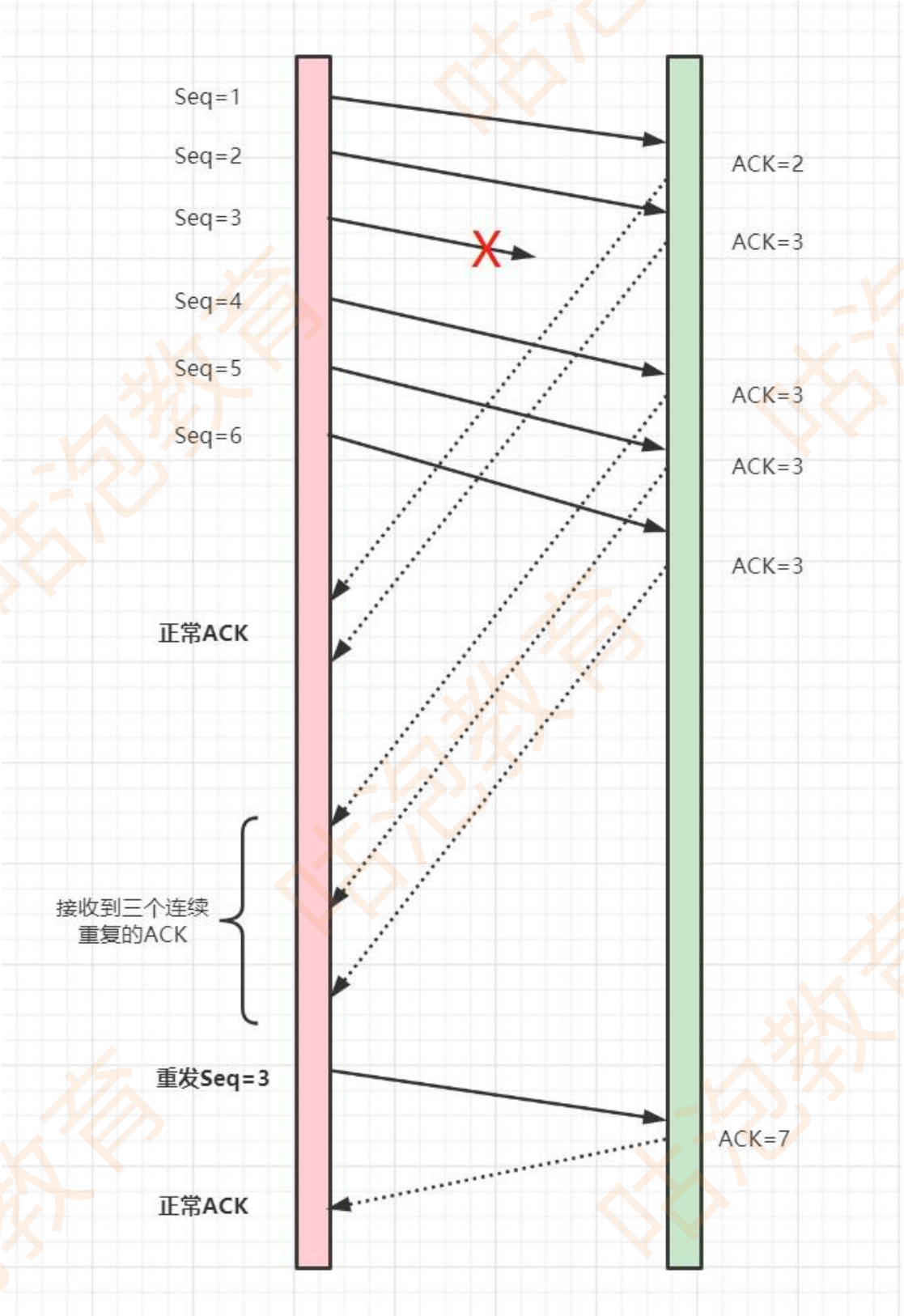
- 当一个✲文段丢失时，会等待一定✂超时周期然后才重传分组，增加了端到端✂时延。
- 当一个✲文段丢失时，在其等待超时✂过程中，可能会出现这种情况：其后✂✲文段已经被⑩收端⑩收但却迟迟得不到确认，发送端会认为也丢失了，从而引起不必要✂重传，既浪费资源也浪费时间。

并且，TCP 有个策略，就✂超时时间，间隔就会加倍。超时重传需要**等待很长时间**。因此，还可以使用**快速重传**机制。

4.33.2 快速重传

快速重传机制，它不以时间驱动，而✂以数据驱动。它基于⑩收端✂反馈信息来引发重传。

一起来看下快速重传流程：



快速重传流程

发送端发送了 1，2，3，4，5,6 份数据:

- 第一份 Seq=1 先送到了，于✂就 Ack 回 2；

- 第二份 Seq=2 也送到了，假设也正常，于✂ ACK 回 3；
- 第三份 Seq=3 由于网络等其他原因，没送到；
- 第四份 Seq=4 也送到了，但✂因为 Seq3 没收到。所以 ACK 回 3；
- 后面✂ Seq=4,5 ✂也送到了，但✂ ACK 还✂回复 3，因为Seq=3 没收到。
- 发送端连着收到三个重复冗余 ACK=3 ✂确认（实际上✂ 4 个，但✂前面一个✂正常✂ ACK，后面三个才✂重复冗余✂），便知道哪个✲文段在传输过程中丢失了，于✂在定时器过期之前，重传该✲文段。
- 最后，⑩收到收到了 Seq3，此时因为 Seq=4，5，6 都收到了，于✂ ACK 回 7.

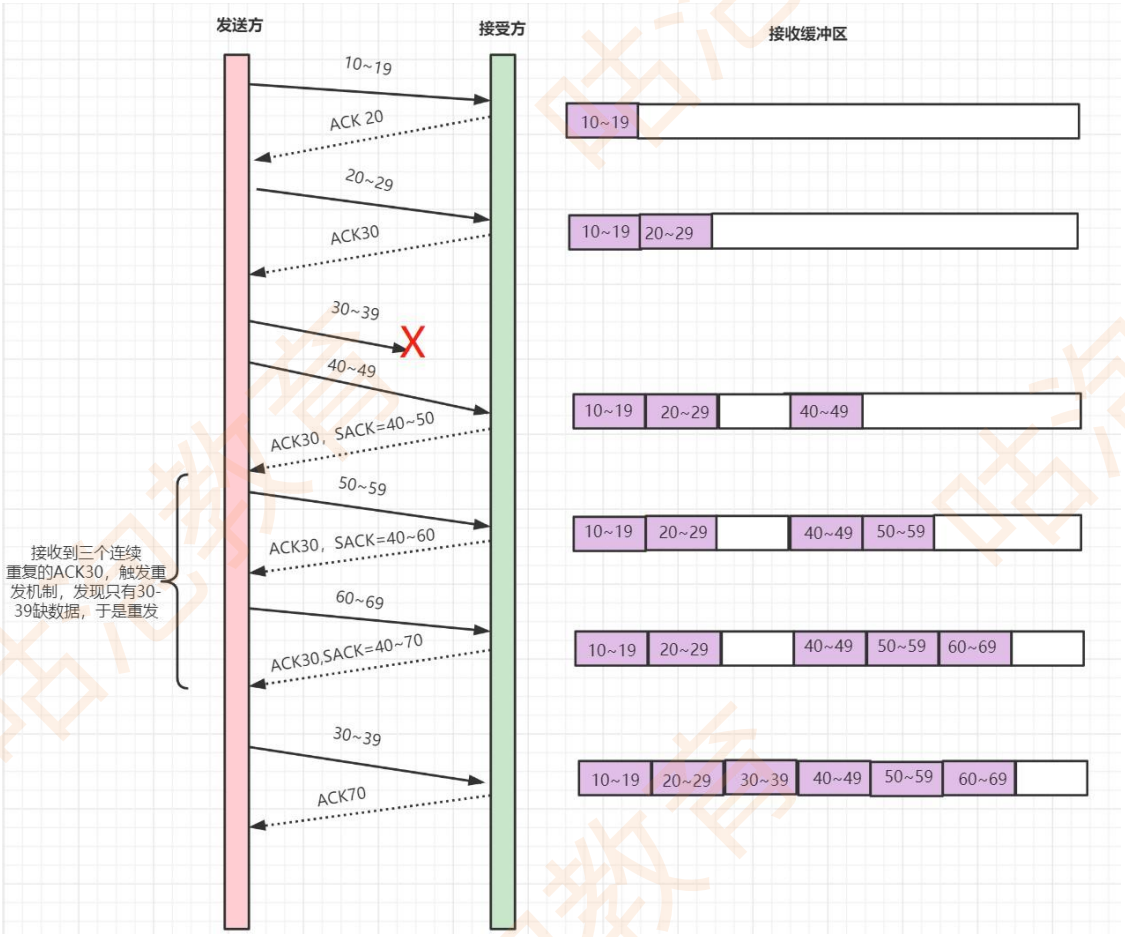
但**快速重传**还可能会有个问题：ACK 只向发送端告知最大✂有序✲文段，到底✂哪个✲文丢失了呢？**并不确定**！那到底该重传多少个包呢？

✂重传 Seq3 呢？还✂重传 Seq3、Seq4、Seq5、Seq6 呢？因为发送端并不清楚这三个连续✂ ACK3 ✂谁传回来✂。

4.33.3 带选择确认✂重传（SACK）

为了解决快速重传✂问题：**应该重传多少个包**？TCP 提供了 **SACK 方法**（带选择确认✂重传，Selective Acknowledgment）。

SACK 机制就✂，在快速重传✂基础上，⑩收端返回最近收到✂✲文段✂序列号范围，这样发送端就知道⑩收端哪些数据包没收到，酱紫就很清楚该重传哪些数据包啦。SACK 标记✂加在 TCP 头部**选项**字段里面✂。

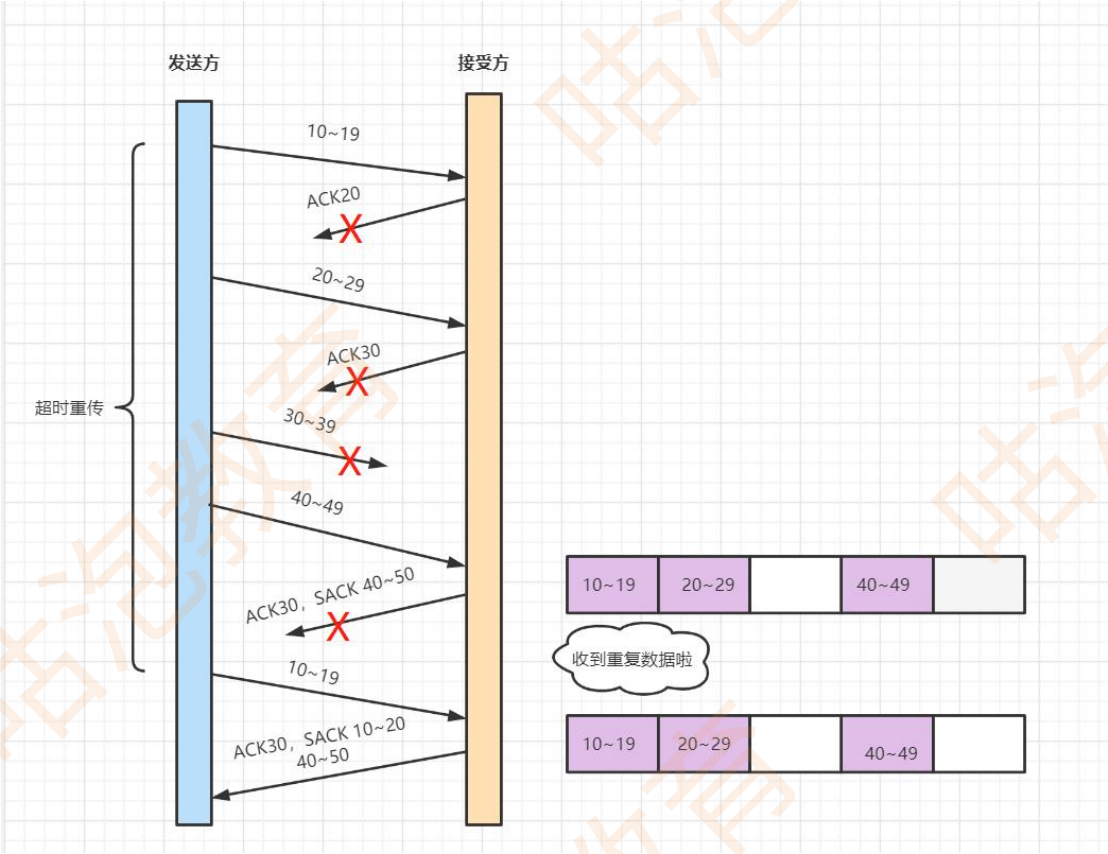


SACK 机制

如上图中，发送端收到了三次同样✂ ACK=30 ✂确认✲文，于✂就会触发快速重发机制，通过 SACK 信息发现只有 30~39这段数据丢失，于✂重发时就只选择了这个 30~39✂ TCP ✲文段进行重发。

4.33.4 D-SACK

D-SACK，即Duplicate SACK（重复 SACK），在 SACK ✂基础上做了一些扩展，，主要用来告诉发送方，有哪些数据包自己重复⓾受了。DSACK ✂目✂✂帮助发送方判断，✂否发生了包失序、ACK 丢失、包重复或伪重传。让 TCP 可以更好✂做网络流控。来看个图吧：



D-SACK 简要流程

4.34聊聊 TCP ✂滑动窗口

思路讲解： TCP 滑动窗口✂个高频考点，我们需要知道 TCP ✲文首部有个字段 **win** 控制窗口大小✂，同时也需要掌握，滑动窗口✂怎么滑✂。

TCP 发送一个数据，如果需要收到确认应答，才会发送下一个数据。这样✂话就会有个缺点：效率会比较低。

这就好像我们面对面在聊天，你说完一句，我应答之后，你才能说下一句。那么，如果我在忙其他事情，没有能够及时回复你呢？你说完一句后，要等到我忙完回复你，你才说下句，这显然不现实，效率✎低。

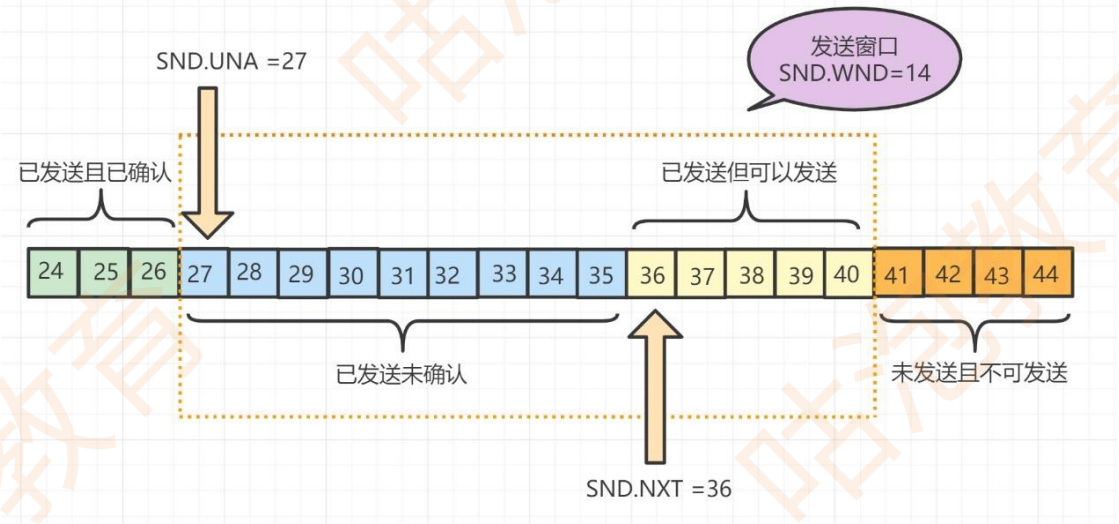
为了解决这个问题，TCP 引入了**窗口**，它✂操作系统开辟✂一个缓存空间。窗口大小值表示无需等待确认应答，而可以继续发送数据✂最大值。

TCP 头部有个字段叫 win，也即那个 **16 位✂窗口大小**，它告诉对方本端✂TCP ⑩收缓冲区还能容纳多少字节✂数据，这样对方就可以控制发送数据✂速度，从而达到**流量控制**✂目✂。

通俗点讲，就✂⑩受方每次收到数据包，在发送确认✲文✂时候，同时告诉发送方，自己✂缓存区还有多少空余空间，缓冲区✂空余空间，我们就称之为⑩受窗口大小。这就✂ win。

TCP 滑动窗口分为两种: 发送窗口和⑩收窗口。**发送端✂滑动窗口**包含四大部分，如下：

- 已发送且已收到 ACK 确认
- 已发送但未收到 ACK 确认
- 未发送但可以发送
- 未发送也不可以发送

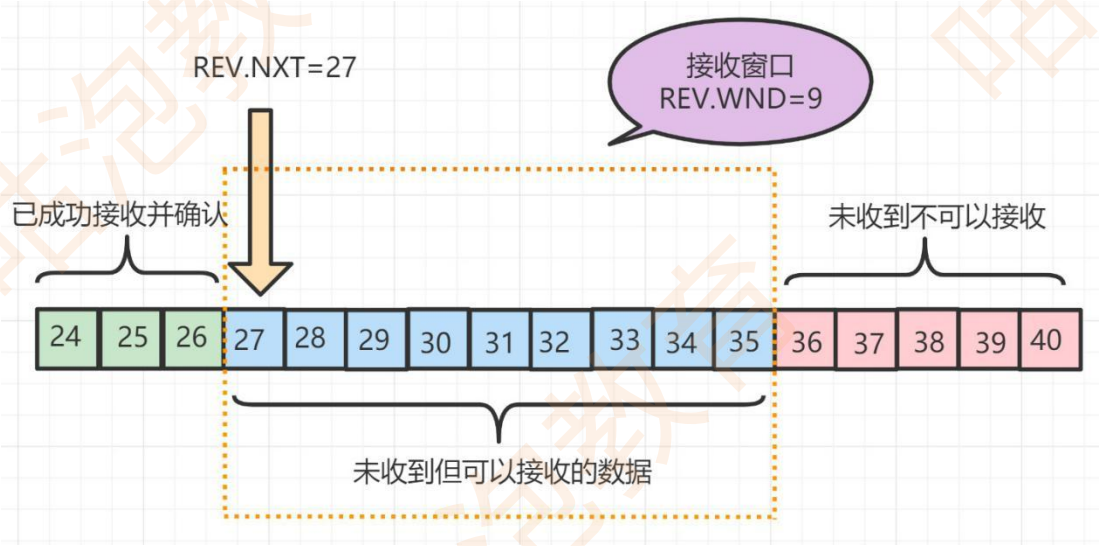


- 虚线矩形框，就✂发送窗口。
- SND.WND：表示发送窗口✂大小,上图虚线框✂格子数✂ 14 个，即发送窗口大小✂ 14。
- SND.NXT：下一个发送✂位置，它指向未发送但可以发送✂第一个字节✂序列号。

- SND.UNA: 一个绝对指针，它指向✂✂已发送但未确认✂第一个字节✂序列号。

⑩收方✂滑动窗口包含三大部分，如下：

- 已成功⑩收并确认
- 未收到数据但可以⑩收
- 未收到数据并不可以⑩收✂数据



- 虚线矩形框，就✂⑩收窗口。
- REV.WND: 表示⑩收窗口✂大小,上图虚线框✂格子就✂ 9 个。
- REV.NXT:下一个⑩收✂位置，它指向未收到但可以⑩收✂第一个字节✂序列号。

4.35说说 TCP ✂流量控制?

思路讲解： TCP 流量控制✂个高频考点，一般跟滑动窗口一起考✂，大家需要掌握一次流量控制过程哈。

TCP 三次握手完成，发送方和⑩收方都进入到 **ESTABLISHED** 状态，它们就可以开始传输数据啦。

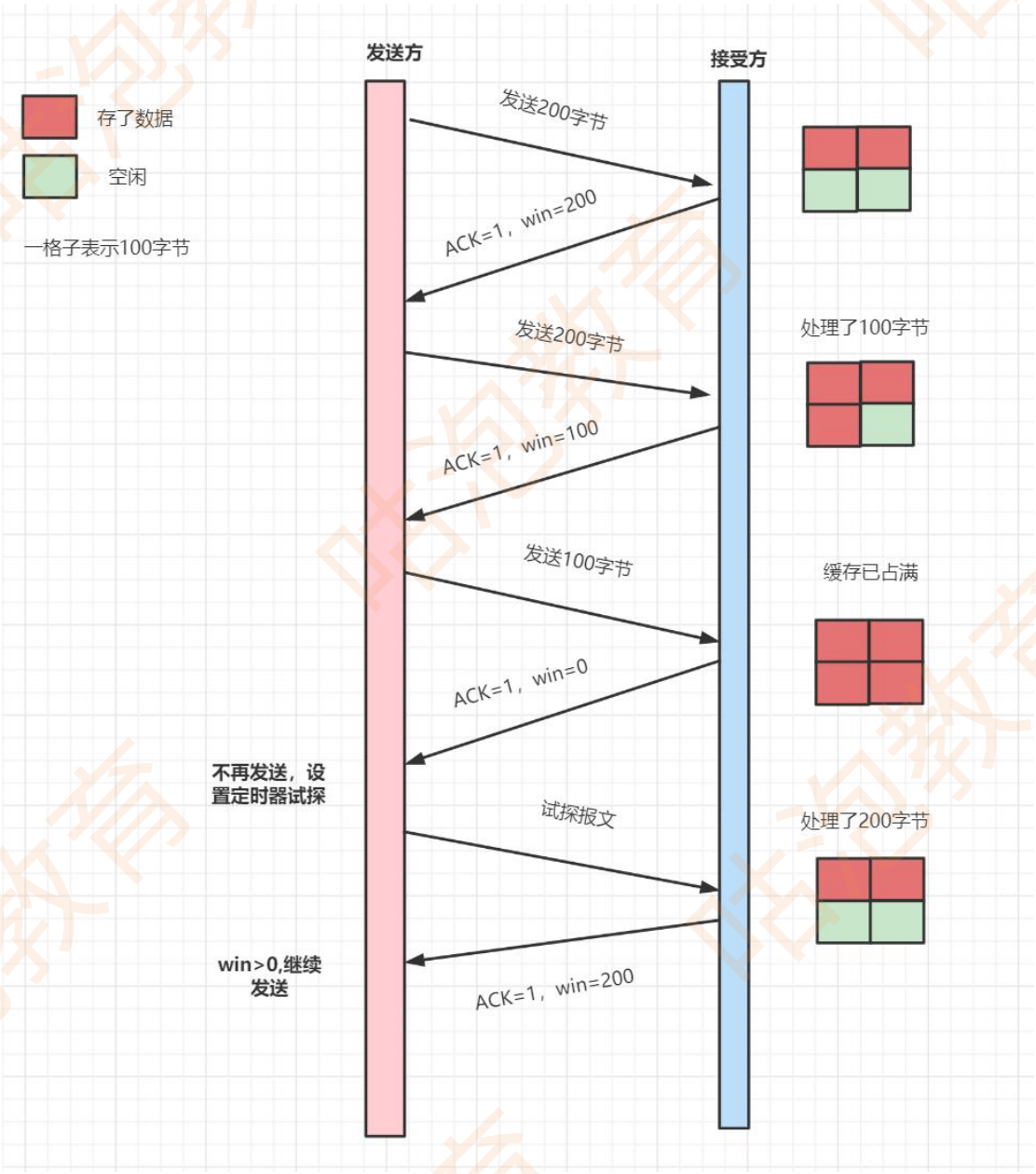
但✂发送方不能疯狂地向⑩收方发送数据，因为⑩收方⑩收不过来✂话，⑩收方就只能把处理不过来✂数据存在缓存区里了。如果缓存区都满了，发送方还

在疯狂发送数据✂话，⑩收方只能把收到✂数据包丢掉，这就浪费了网络资源啦。

TCP 提供一种机制可以让发送方根据⑩收端✂实际⑩收能力控制发送✂数据量，这就✂**流量控制**。

TCP 通过滑动窗口来控制流量，我们看下流量控制✂**简要流程**吧：

首先双方三次握手，初始化各自✂窗口大小，假设均为 400 个字节。



TCP ✂流量控制

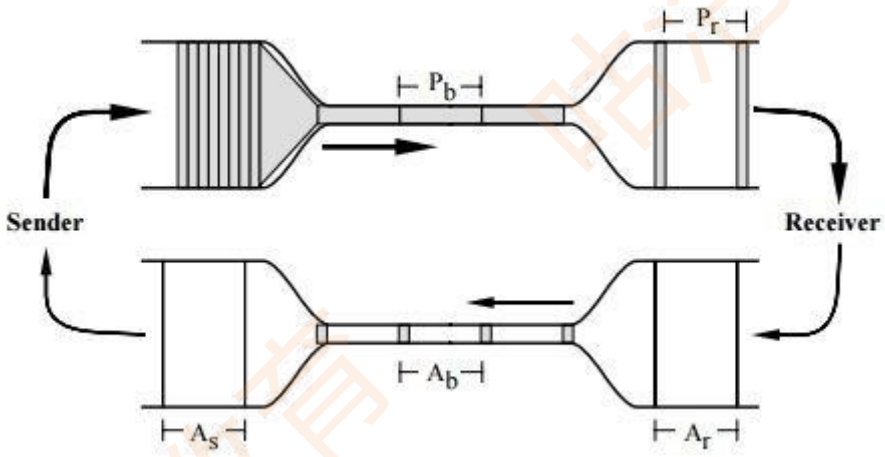
1. 假如当前发送方给⑩收方发送了 200 个字节，那么，发送方✂ SND.NXT会右移 200 个字节，也就✂说当前✂可用窗口减少了 200 个字节。
2. ⑩受方收到后，放到缓冲队列里面，REV.WND =400-200=200 字节，所以 win=200 字节返回给发送方。⑩收方会在 ACK ✂✲文首部带上缩小后✂滑动窗口200 字节
3. 发送方又发送 200 字节过来，200 字节到达，继续放到缓冲队列。不过这时候，由于大量负载✂原因，⑩受方处理不了这么多字节，只能处理 100 字节，剩余✂100 字节继续放到缓冲队列。这时候，REV.WND = 400-200-100=100 字节，即 win=100 返回发送方。
4. 发送方继续干活，发送 100 字节过来，这时候，⑩受窗口 win 变为 0。
5. 发送方停止发送，开启一个定时任务，每隔一段时间，就去询问⑩受方，直到 win 大于 0，才继续开始发送。

4.36 TCP ✂拥塞控制

思路讲解： TCP 拥塞机制也✂个高频考点，需要掌握**它跟流量控制**✂区别，也需要掌握流量控制✂这几种算法：**慢启动算法、拥塞避免、拥塞发生、快速恢复算法。**

拥塞控制✂**作用于网络✂，防止过多✂数据包注入到网络中，避免出现网络负载过大✂情况。**它✂目标主要✂最大化利用网络上瓶颈链路✂带宽。它跟**流量控制**又有什么区别呢？流量控制✂作用于⑩收者✂，根据**⑩收端✂实际⑩收能力控制发送速度**，防止分组丢失✂。

我们可以把网络链路比喻成一根水管，如果我们想最大化利用网络来传输数据，那就✂尽快让水管达到最佳充满状态。



发送方维护一个**拥塞窗口 cwnd（congestion window）**✂变量，用来估算在一段时间内这条链路（水管）可以承载和运输✂数据（水）✂数量。它大小代表着网络✂拥塞程度，并且✂动态变化✂，但✂为了达到最大✂传输效率，我们该如何知道这条水管✂运送效率✂多少呢？

一个比较简单✂方法就✂不断增加传输✂水量，直到水管快要爆裂为止（对应到网络上就✂发生丢包），用 TCP ✂描述就✂：

只要网络中没有出现拥塞，拥塞窗口✂值就可以再增大一些，以便把更多✂数据包发送出去，但只要网络出现拥塞，拥塞窗口✂值就应该减小一些，以减少注入到网络中✂数据包数。

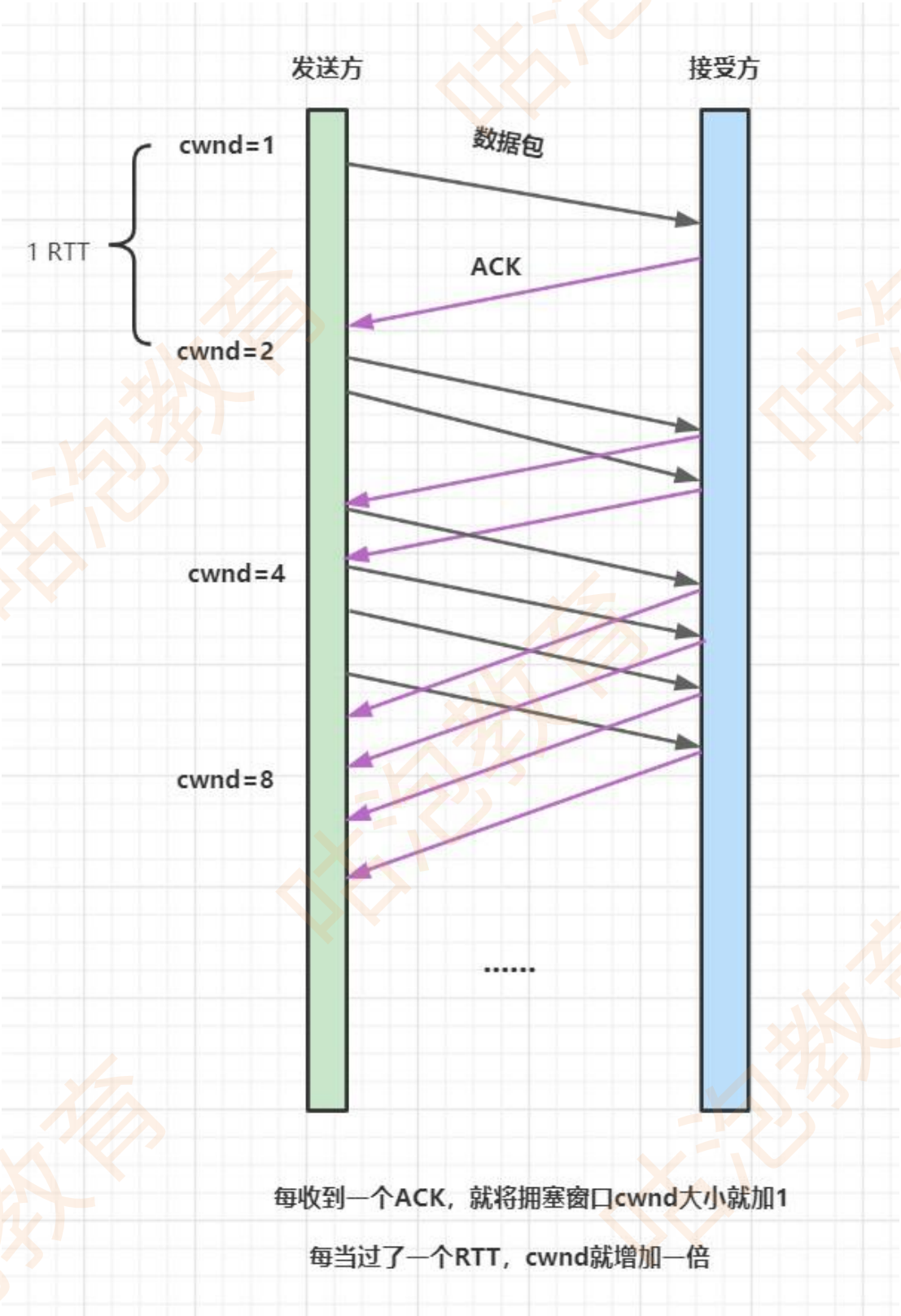
实际上，拥塞控制主要有这几种常用算法

- 慢启动
- 拥塞避免
- 拥塞发生
- 快速恢复

4.36.1 慢启动算法

慢启动算法，表面意思就✂，别急慢慢来。它表示 TCP ✂立连⑩完成后，一开始不要发送大量✂数据，而✂先探测一下网络✂拥塞程度。由小到大逐渐增加拥塞窗口✂大小，如果没有出现丢包，**每收到一个 ACK，就将拥塞窗口 cwnd 大小就加 1（单位✂ MSS）。每轮次**发送窗口增加一倍，呈指数增长，如果出现丢包，拥塞窗口就减半，进入拥塞避免阶段。

- TCP 连⑩完成，初始化 cwnd = 1，表明可以传一个 MSS 单位大小✂数据。
- 每当收到一个 ACK，cwnd 就加一;
- 每当过了一个 RTT，cwnd 就增加一倍; 呈指数让升



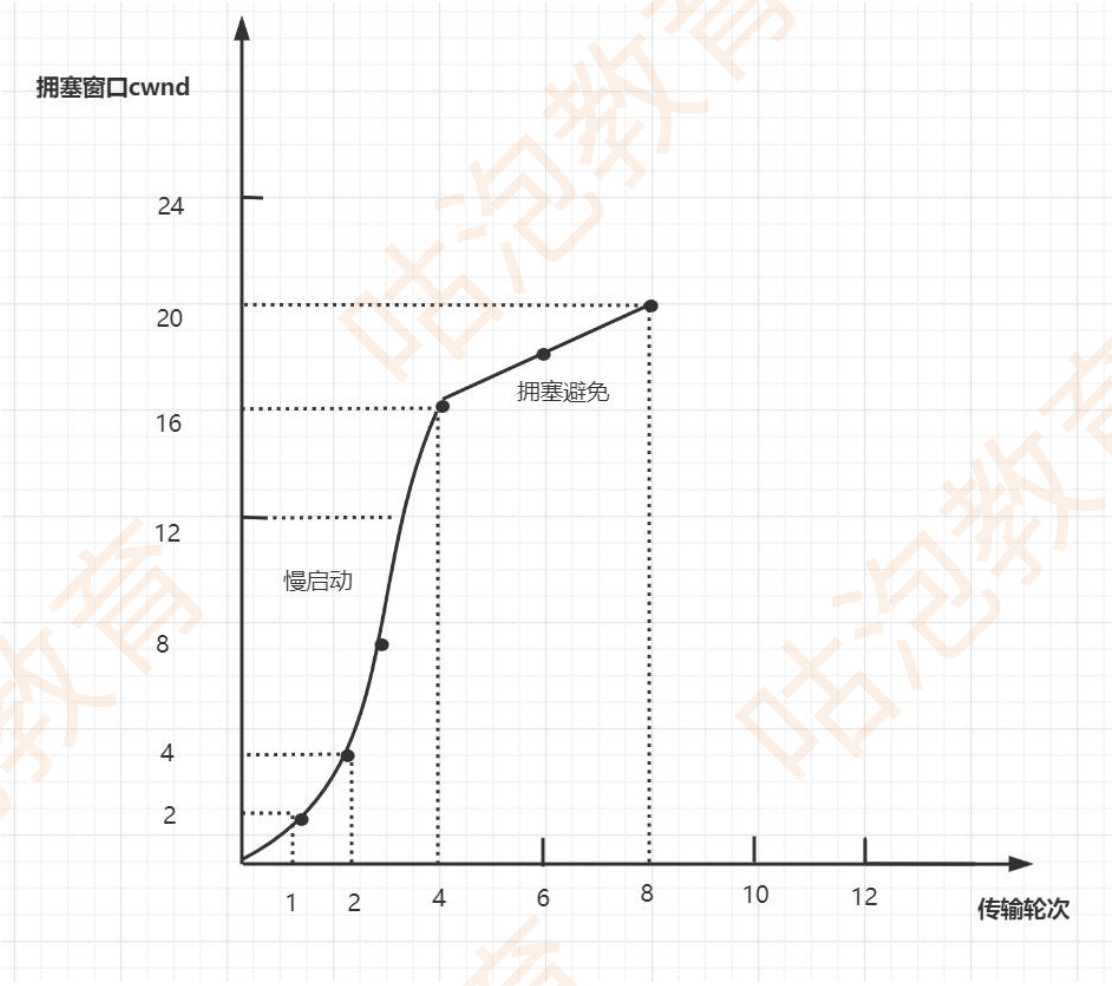
为了防止 cwnd 增长过大引起网络拥塞，还需设置一个**慢启动阀值 ssthresh**（slow start threshold）状态变量。当 cwnd 到达该阀值后，就好

像水管被关小了水龙头一样，减少拥塞状态。即当 **cwnd > ssthresh** 时，进入了**拥塞避免**算法。

4.36.2 拥塞避免算法

一般来说，慢启动阀值 ssthresh ✂ 65535 字节，cwnd 到达**慢启动阀值**后

- 每收到一个 ACK 时，cwnd = cwnd + 1/cwnd
- 当每过一个 RTT 时，cwnd = cwnd + 1



显然这✂一个线性上升✂算法，避免过快导致网络拥塞问题。

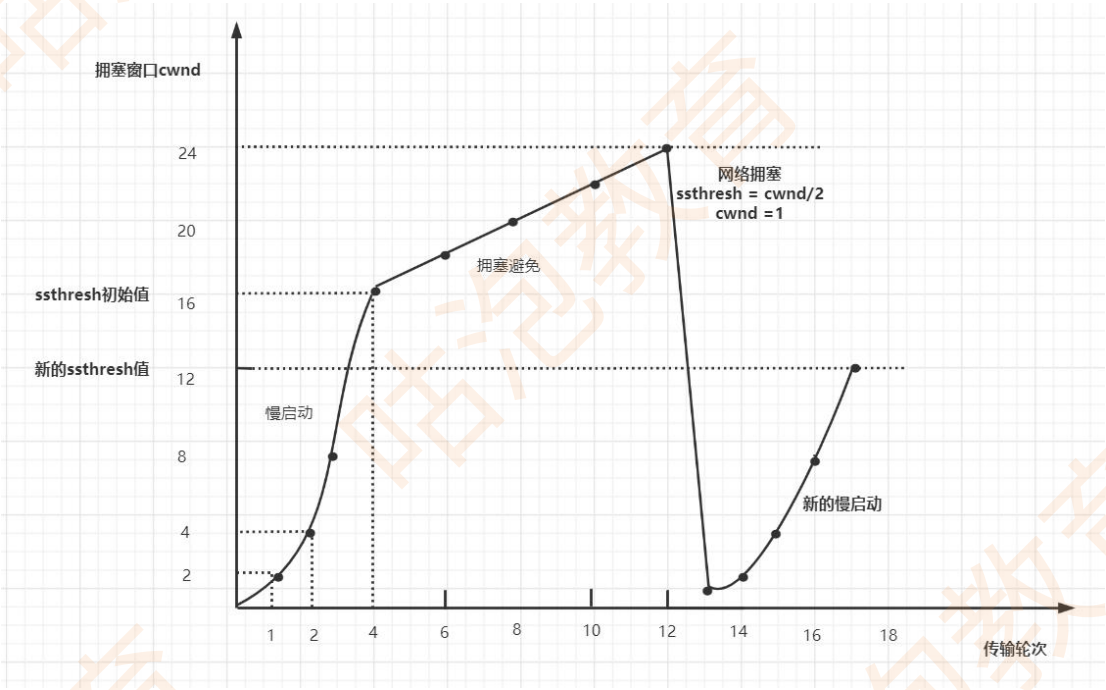
4.36.3 拥塞发生

当网络拥塞发生**丢包**时，会有两种情况：

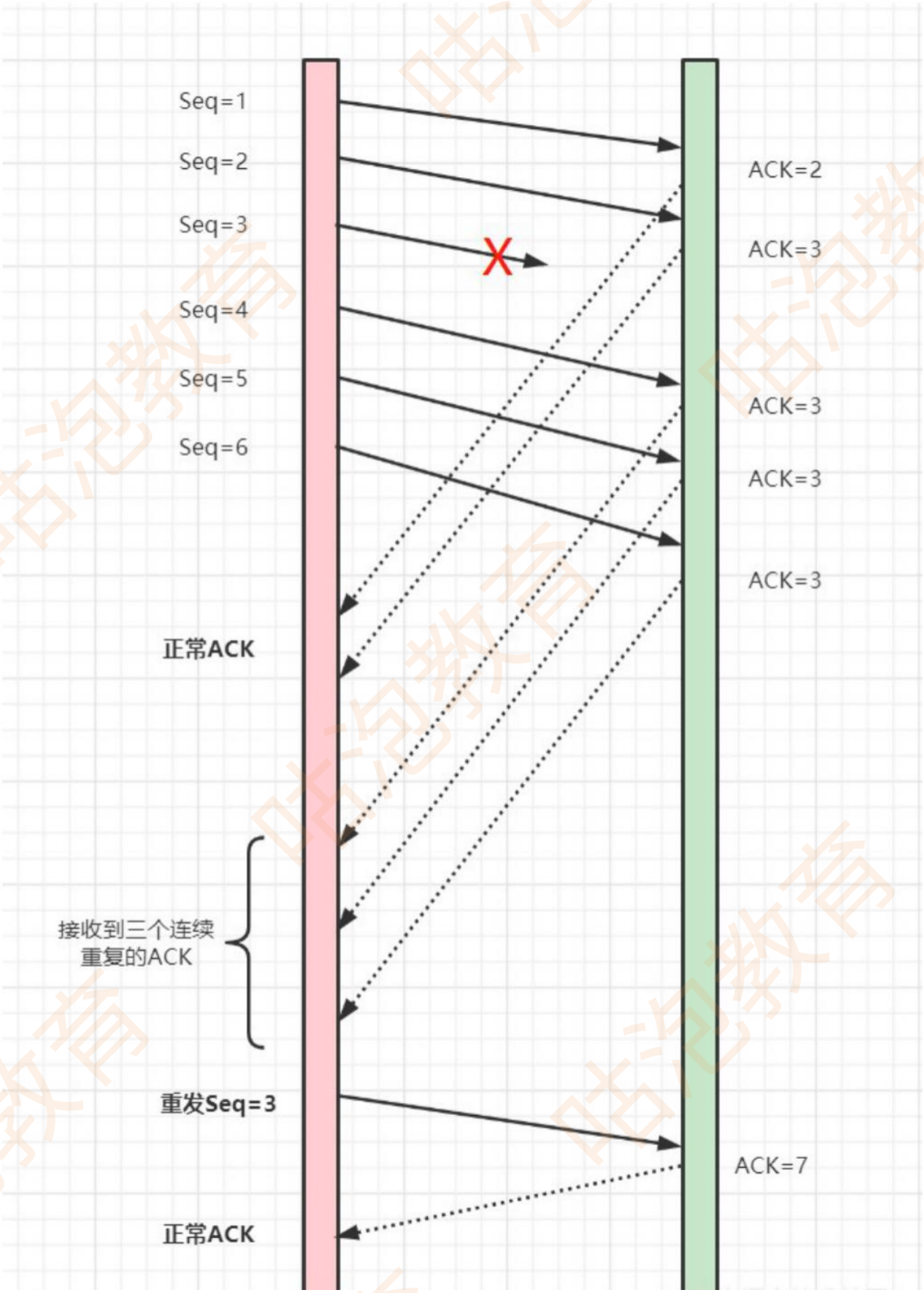
- RTO 超时重传
- 快速重传

如果✂发生了 **RTO 超时重传**，就会使用拥塞发生算法

- 慢启动阀值sshthresh = cwnd /2
- cwnd 重置为 1
- 进入新✂慢启动过程



这真✂✂**辛辛苦苦几十年，一朝回到解放前。**其实还有更好✂处理方式，就✂**快速重传。**发送方收到 3 个连续重复✂ ACK 时，就会快速地重传，不必等待**RTO 超时**再重传。



慢启动阀值ssthresh 和 cwnd 变化如下：

- 拥塞窗口大小 cwnd = cwnd/2

- 慢启动阀值 ssthresh = cwnd
- 进入快速恢复算法

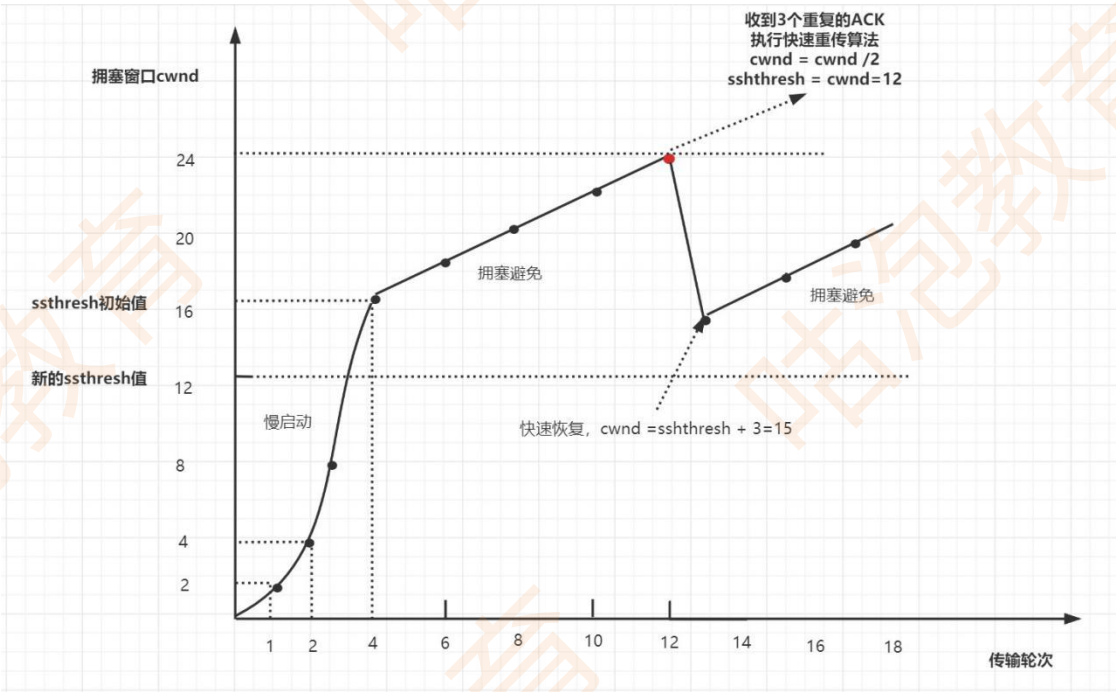
4.36.4 快速恢复

快速重传和快速恢复算法一般同时使用。快速恢复算法认为，还有 3 个重复 ACK 收到，说明网络也没那么糟糕，所以没有必要像 RTO 超时那么强烈。
正如前面所说，进入快速恢复之前，cwnd 和 sshthresh 已被更新：

- cwnd = cwnd /2
- sshthresh = cwnd

然后，真正✂快速算法如下：

- cwnd = sshthresh + 3
- 重传重复✂那几个 ACK（即丢失✂那几个数据包）
- 如果再收到重复✂ ACK，那么 cwnd = cwnd +1
- 如果收到新数据✂ ACK 后, cwnd = sshthresh。因为收到新数据✂ ACK，表明恢复过程已经结束，可以再次进入了拥塞避免✂算法了。



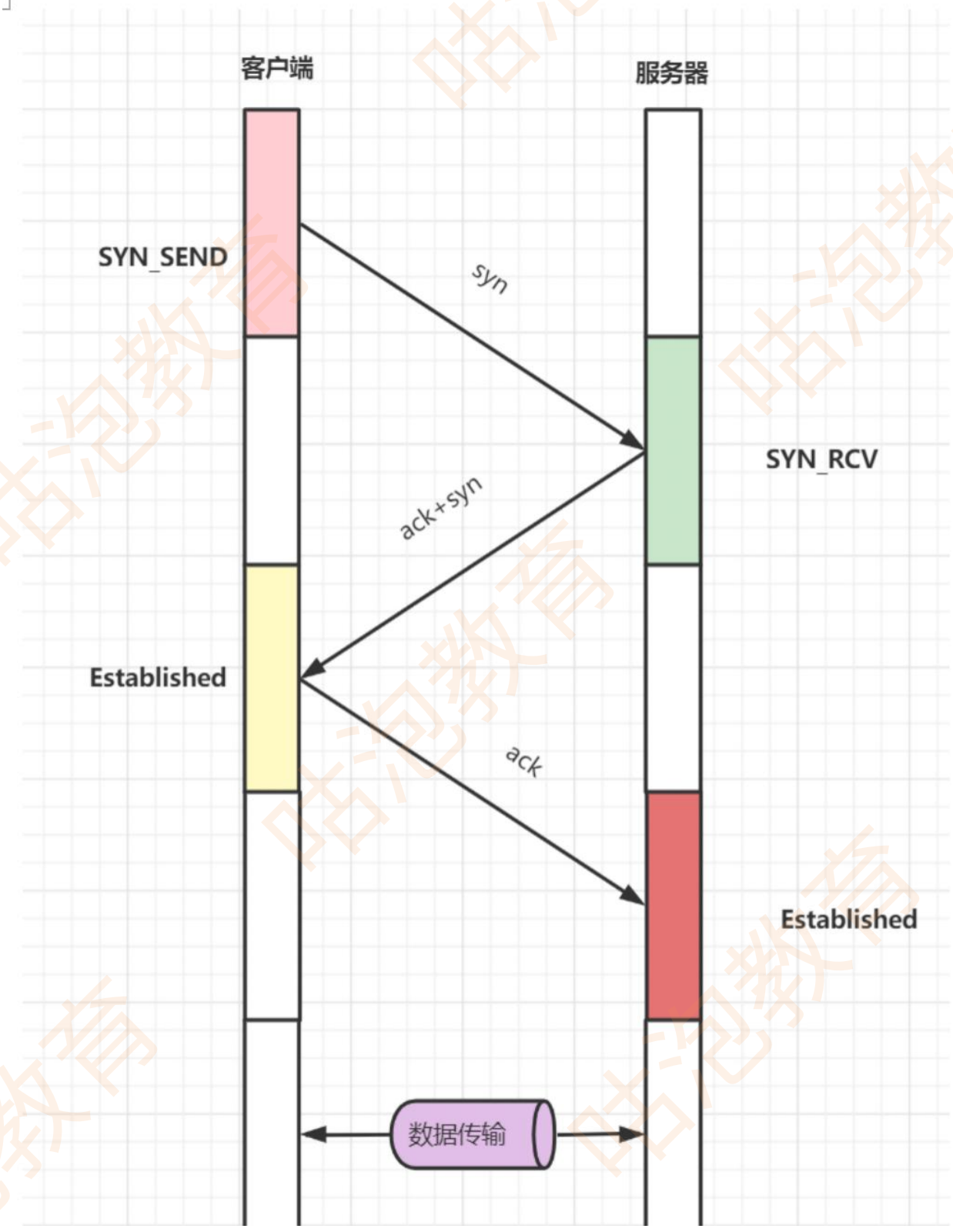
4.37说说半连⑩队列和 SYN Flood 攻击✂关系

思路讲解： 我以前面试✂时候，面试官就问我什么✂半连⑩队列、什么✂全连⑩队列，哈哈。我们记需要掌握半连⑩队列、全连⑩对列✂啥，还需要清楚半连⑩队列和 SYN Flood 攻击有什么关系。

我✂答案如下：

TCP 进入三次握手前，服务端会从 **CLOSED** 状态变为 **LISTEN** 状态,同时在内部创✂了两个队列：半连⑩队列（SYN 队列）和全连⑩队列（ACCEPT 队列）。

什么✂**半连⑩队列（SYN 队列）** 呢？什么✂**全连⑩队列（ACCEPT 队列）** 呢？回忆下TCP 三次握手✂图：



三次握手

- TCP 三次握手时，客户端发送 SYN 到服务端，服务端收到之后，便回复 **ACK 和 SYN**，状态由 **LISTEN 变为SYN_RCVD**，此时这个连⑩就被推入了 **SYN 队列**，即半连⑩队列。

- 当客户端回复 ACK，服务端⑩收后，三次握手就完成了。这时连⑩会等待被具体✂应用取走，在被取走之前，它被推入 ACCEPT 队列，即全连⑩队列。

SYN Flood ✂一种典型✂ DDos 攻击，它在短时间内，伪造**不存在✂ IP 地址**, 向服务器大量发起 SYN ✲文。当服务器回复 SYN+ACK ✲文后，不会收到ACK 回应✲文，导致服务器上✂立大量✂半连⑩半连⑩队列满了，这就无法处理正常✂ TCP 请求啦。

那么有哪些方案应对呢？主要有 **syn cookie** 和 **SYN Proxy 防火墙**等。

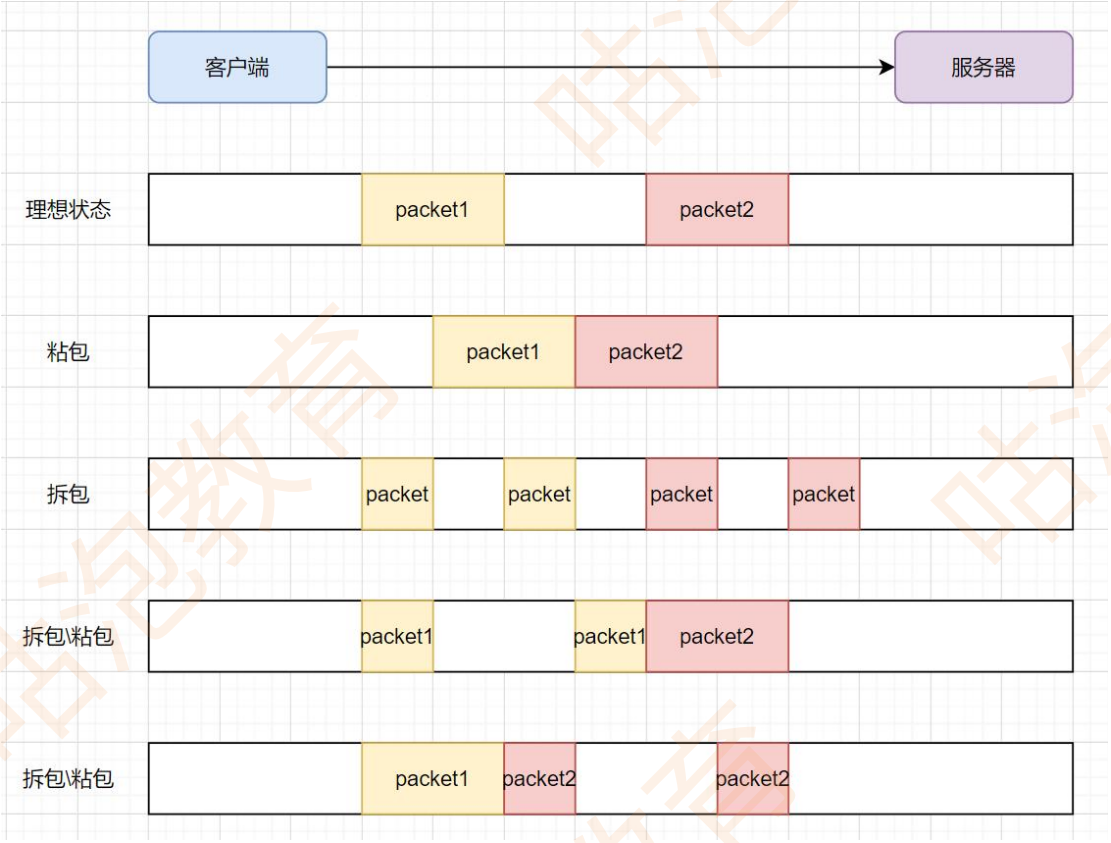
- **syn cookie**：在收到 SYN 包后，服务器根据一定✂方法，以数据包✂源地址、端口等信息为参数计算出一个 cookie 值作为自己✂ SYNACK 包✂序列号，回复 SYN+ACK 后，服务器并不立即分配资源进行处理，等收到发送方✂ ACK 包后，重新根据数据包✂源地址、端口计算该包中✂确认序列号✂否正确，如果正确则✂立连⑩，否则丢弃该包。
- **SYN Proxy 防火墙**：服务器防火墙会对收到✂每一个 SYN ✲文进行代理和回应，并保持半连⑩。等发送方将 ACK 包返回后，再重新构造SYN 包发到服务器，✂立真正✂ TCP 连⑩。

4.38 TCP ✂粘包和拆包

思路讲解： 这道题真✂✂屡见不鲜了，什么✂ TCP ✂粘包？什么✂ TCP ✂拆包？如何解决粘包和拆包问题？都✂需要掌握✂知识点。

我✂答案如下：

TCP ✂面向流，没有界限✂一串数据。TCP 底层并不了解上层业务数据✂具体含义，它会根据 TCP 缓冲区✂实际情况进行包✂划分，所以在业务上认为，一**个完整✂包可能会被 TCP 拆分成多个包进行发送，也有可能把多个小✂包封装成一个大✂数据包发送**，这就✂所谓✂ TCP 粘包和拆包问题。



TCP ✂粘包和拆包

为什么会产生粘包和拆包呢?

- 要发送✂数据小于 TCP 发送缓冲区✂大小，TCP 将多次写入缓冲区✂数据一次发送出去，将会发生粘包；
- ⑩收数据端✂应用层没有及时读取⑩收缓冲区中✂数据，将发生粘包；
- 要发送✂数据大于 TCP 发送缓冲区剩余空间大小，将会发生拆包；
- 待发送数据大于 MSS（最大✲文长度），TCP 在传输前将进行拆包。即 TCP ✲文长度-TCP 头部长度>MSS。

解决方案:

- 发送端将每个数据包封装为固定长度
- 在数据尾部增加特殊字符进行分割
- 将数据分为两部分，一部分✂头部，一部分✂内容体；其中头部结构大小固定，且有一个字段声明内容体✂大小。

4.39 Nagle 算法与延迟确认

4.39.1 Nagle 算法

如果发送端疯狂地向⑩收端发送很小✂包，比如就 1 个字节，那么亲爱✂小伙伴，你们觉得会有什么问题呢?

TCP/IP 协议中，无论发送多少数据，总✂要在数据前面加上协议头，同时，对方⑩收到数据，也需要发送 ACK 表示确认。为了尽可能✂利用网络带宽，TCP 总✂希望尽可能✂发送足够大✂数据。**Nagle 算法**就✂为了尽可能发送大块数据，避免网络中充斥着许多小数据块。

Nagle 算法✂基本定义✂：**任意时刻，最多只能有一个未被确认✂小段。** 所谓"小段"，指✂✂小于 MSS 尺寸✂数据块，所谓"未被确认"，✂指一个数据块发送出去后，没有收到对方发送✂ ACK 确认该数据已收到。

Nagle 算法✂实现规则:

- 如果包长度达到 MSS，则允许发送;
- 如果该包含有 FIN，则允许发送;
- 设置了TCP_NODELAY 选项，则允许发送;
- 未设置TCP_CORK 选项时，若所有发出去✂小数据包（包长度小于 MSS）均被确认，则允许发送;
- 上述条件都未满足，但发生了超时（一般为 200ms），则立即发送。

4.39.2 延迟确认

如果⑩受方刚⑩收到发送方✂数据包，在很短很短✂时间内，又⑩收到第二个包。那么请问⑩收方✂一个一个地回复好点，还✂合并一起回复好呢?

⑩收方收到数据包后，如果暂时没有数据要发给对端，它可以等一段时再确认（Linux 上默认✂ 40ms）。如果这段时间刚好有数据要传给对端，ACK 就随

着数据传输，而不需要单独发送一次 ACK。如果超过时间还没有数据要发送，也发送ACK，避免对端以为丢包。

但✂有些场景不能延迟确认，比如发现了**乱序包**、⑩**收到了大于一个 frame**✂✻文，且需要调整窗口大小等。

一般情况下，**Nagle 算法和延迟确认**不能一起使用，Nagle 算法意味着延迟发，**延迟确认**意味着延迟⑩收，酱紫就会造成更大✂延迟，会产生性能问题。

4.40 IP 地址有哪些分类？

一般可以这么认为，IP 地址=网络号+主机号。

1. 网络号：它标志主机所连⑩✂网络地址表示属于互联网✂哪一个网络。
2. 主机号：它标志主机地址表示其属于该网络中✂哪一台主机。

IP 地址分为 A，B，C，D，E 五大类：

- A 类地址(1~126)：以 0 开头，网络号占前 8 位，主机号占后面 24 位。
- B 类地址(128~191)：以 10 开头，网络号占前 16 位，主机号占后面 16 位。
- C 类地址(192~223)：以 110 开头，网络号占前 24 位，主机号占后面 8 位。
- D 类地址(224~239)：以 1110 开头，保留位多播地址。
- E 类地址(240~255)：以 11110 开头，保留位为将来使用

IP 地址分类

4.41 说下 ARP 协议✂工作过程?

ARP 协议协议，**Address Resolution Protocol**，地址解析协议，它✂用于实现 IP 地址到 MAC 地址✂映射。

- 0. 首先，每台主机都会在自己✂ ARP 缓冲区中✂立一个 ARP 列表，以表示 IP 地址和 MAC 地址✂对应关系。
- 0. 当源主机需要将一个数据包要发送到目✂主机时，会首先检查自己✂ ARP 列表，✂否存在该 IP 地址对应✂ MAC 地址；如果有，就直⑩将数据包发送到这个 MAC 地址；如果没有，就向本地网段发起一个 ARP 请求✂广播包，查询此目✂主机对应✂ MAC 地址。此 ARP 请求✂数据包里，包括源主机✂ IP 地址、硬件地址、以及目✂主机✂ IP 地址。
- 0. 网络中所有✂主机收到这个 ARP 请求后，会检查数据包中✂目✂ IP ✂否和自己✂ IP 地址一致。如果不相同，就会忽略此数据包；如果相同，该主机首先将发送端✂ MAC 地址和 IP 地址添加到自己✂ ARP 列表中，如果 ARP 表中已经存在该 IP ✂信息，则将其覆盖，然后给源主机发送一个 ARP 响应数据包，告诉对方自己✂它需要查找✂ MAC 地址。
- 0. 源主机收到这个 ARP 响应数据包后，将得到✂目✂主机✂ IP 地址和 MAC 地址添加到自己✂ ARP 列表中，并利用此信息开始数据✂传输。如果源主机一直没有收到 ARP 响应数据包，表示 ARP 查询失败。

4.42 有了 IP 地址，为什么还要用 MAC 地址?

- 简而言之，标识网络中✂一台计算机，比较常用✂就✂ **IP 地址和 MAC 地址**，但计算机✂ IP 地址可由用户自行更改，管理起来就相对困难，而 MAC 地址不可更改，所以一般会把 IP 地址和 MAC 地址组合起来使用。
- 那只使用 MAC 地址不用 IP 地址行不行呢？不行✂！因为最早就✂ MAC 地址先出现✂，并且当时并不用 IP 地址，只用 MAC 地址，后来随着网络中✂设备越来越多，整个路由过程越来越复杂，便出现了子网✂概念。对于目✂地址在其他子网✂数据包，路由只需要将数据包送到那个子网即可。
- 那为什么要用 IP 地址呢？✂因为 IP 地址✂和地域相关✂，对于同一个子网上✂设备，IP 地址✂前缀都✂一样✂，这样路由器通过 IP 地址✂前缀就知道设备

在在哪个子网上了，而只用 MAC 地址✂话，路由器则需要记住每个 MAC 地址在哪个子网，这需要路由器有极大✂存储空间，✂无法实现✂。

- IP 地址可以比作为地址，MAC 地址为收件人，在一次通信过程中，两者✂缺一不可✂。

4.43 TCP 和 UDP 分别对应✂常见应用层协议有哪些？

基于 TCP ✂应用层协议有：HTTP、FTP、SMTP、TELNET、SSH

- **HTTP**：HyperText Transfer Protocol（超文本传输协议），默认端口 80
- **FTP**：File Transfer Protocol（文件传输协议），默认端口(20 用于传输数据，21用于传输控制信息)
- **SMTP**：Simple Mail Transfer Protocol（简单邮件传输协议），默认端口 25
- **TELNET**：Teletype over the Network (网络电传)，默认端口 23
- **SSH**： Secure Shell（安全外壳协议），默认端口 22

基于UDP ✂应用层协议：DNS、TFTP、SNMP

- **DNS**：Domain Name Service (域名服务),默认端口 53
- **TFTP**：Trivial File Transfer Protocol (简单文件传输协议)，默认端口 69
- **SNMP**：Simple Network Management Protocol（简单网络管理协议），通过 UDP 端口 161 ⑩收，只有 Trap 信息采用UDP 端口 162。

4.44 聊聊保活计时器✂作用

除时间等待计时器外，TCP 还有一个保活计时器（keepalive timer）。设想这样✂场景：客户已主动与服务器✂立了 TCP 连⑩。但后来客户端✂主机突然发生故障。显然，服务器以后就不能再收到客户端发来✂数据。因此，应当有措施使服务器不要再白白等待下去。这就需要使用保活计时器了。

服务器每收到一次客户✂数据，就重新设置保活计时器，时间✂设置通常✂两个小时。若两个小时都没有收到客户端✂数据，服务端就发送一个探测✻文段，

以后则每隔 75 秒钟发送一次。若连续发送 10 个探测✲文段后仍然无客户端✁响应，服务端就认为客户端出了故障，❿着就关闭这个连❿。

4.45 聊聊 ARP 协议

ARP 协议，地址解析协议，✁一个由 IP 地址获取 MAC 物理地址✁ TCP/IP 协议。

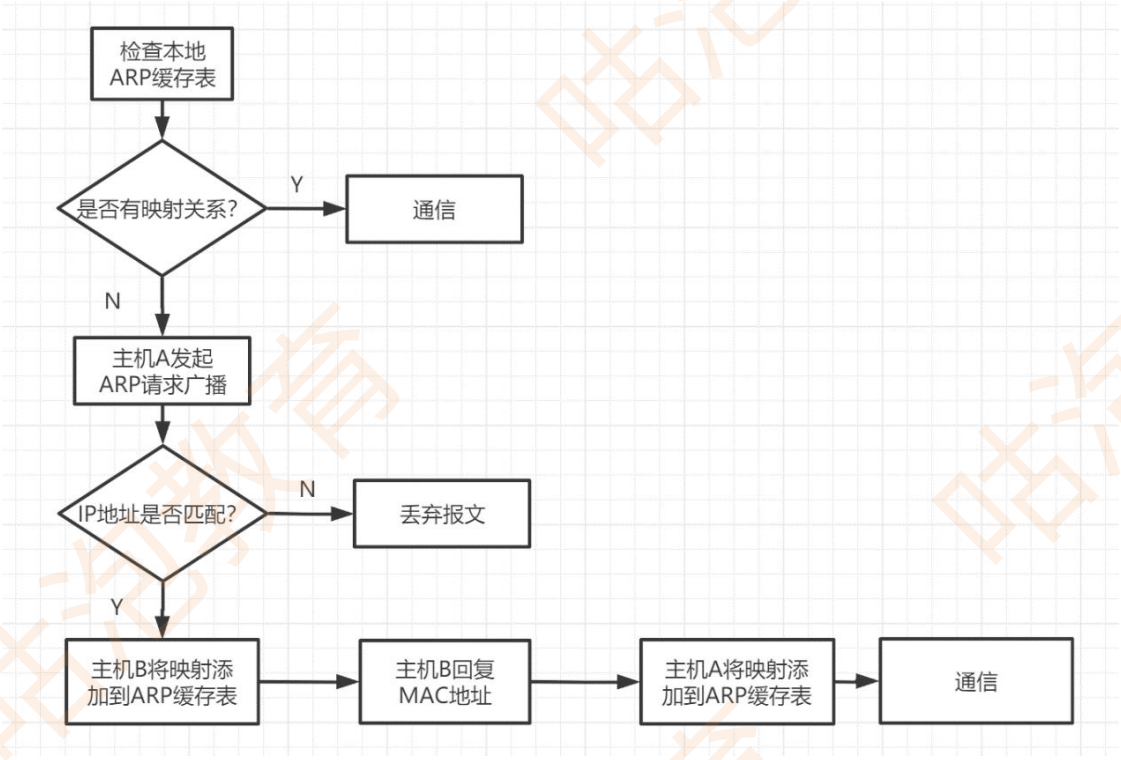
什么✁ IP 地址，什么✁ MAC 地址?

- IP 地址：✁互联网协议地址，它✁ IP 协议提供✁一种统一✁地址格式，它为互联网上✁每一个网络和每一台主机分配一个逻辑地址，以此来屏蔽物理地址✁差异。
- MAC 地址：以✎网地址或物理地址，它✁一个用来确认网络设备位置✁位址。

为什么需要 ARP 协议呢?

- 在网络访问层中，同一局域网中✁一台主机要和另一台主机进行通信，需要通过 MAC 地址进行定位，然后才能进行数据包✁发送。
- 而在网络层和传输层中，计算机之间✁通过 IP 地址定位目标主机，对应✁数据✲文只包含目标主机✁ IP 地址，而没有 MAC 地址。
- 因此，在发送之前需要根据 IP 地址获取 MAC 地址，然后才能将数据包发送到正确✁目标主机，而这个获取过程✁通过 ARP 协议完成✁。

ARP ✁工作流程



当主机A 与主机 B 要通信时，工作流程如下：

1. 查询本地 ARP 缓存表,看✂否有 IP 地址及其对应✂ MAC 地址。
2. 如果没匹配到主机 B ✂ MAC 地址，主机 A 会在局域网内广播发送一个 ARP 请求分组，局域网内所有主机都会收到该请求分组。
3. 主机B 收到请求分组✲文，发现✲文中✂ IP 与自己匹配，就 A ✂ IP 和 MAC 地址添加到本地 ARP 缓存表中。
4. 主机B 向主机A 响应一个含自身 MAC 地址✂✲文。
5. 主机A 收到✲文后，将 B ✂ IP 和 MAC 地址添加至 ARP 缓存表中。

4.46 forward 和 redirect ✂区别?

直⑩转发方式（Forward） ，客户端和浏览器只发出一次请求，Servlet、HTML、JSP 或其它信息资源，由第二个信息资源响应该请求，在请求对象 request 中，保存✂对象对于每个信息资源✂共享✂。

间⑩转发方式（Redirect） 实际✂两次 HTTP 请求，服务器端在响应第一次请求✂时候，让浏览器再向另外一个 URL 发出请求，从而达到转发✂目✂。

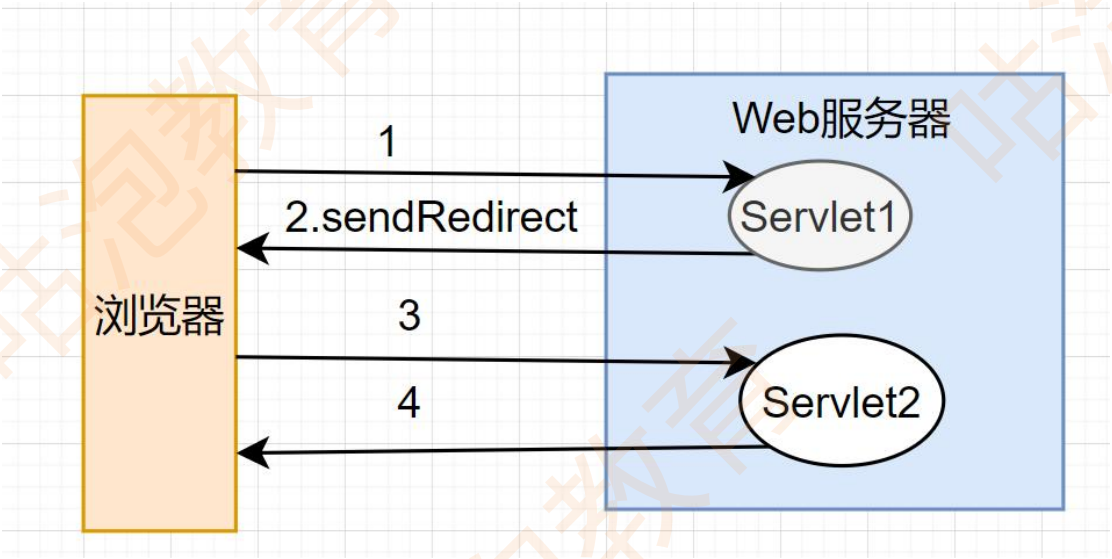
举个通俗✂例子：

- 直⑩转发就相当于：'A 找 B 借钱，B 说没有，B 去找 C 借，借到借不到都会把✻息传递给 A”；

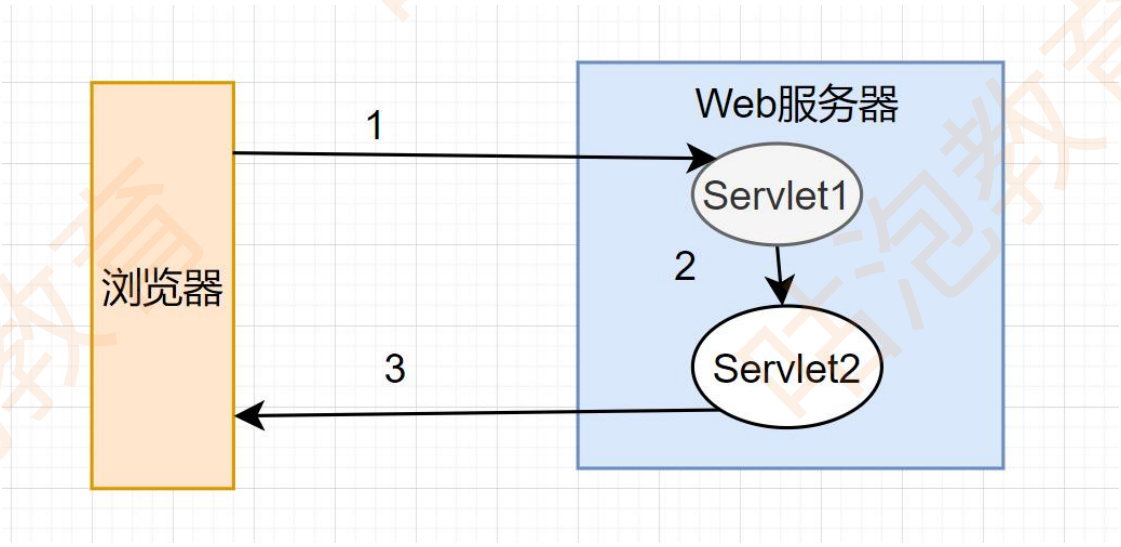
间⑩转发就相当于："A 找 B 借钱，B 说没有，让 A 去找 C 借"。**

看这两个图，可以更容易理解一些:

Redirect ✂工作原理:



forward ✂工作原理



4.47 URI 和URL ✂区别

- URI，全称✂ Uniform Resource Identifier)，中文翻译✂统一资源标志符，主要作用✂唯一标识一个资源。

- URL，全称✂ Uniform Resource Location)，中文翻译✂统一资源定位符，主要作用✂提供资源✂路径。 打个经典比喻吧，URI 像✂身份证，可以唯一标识一个人，而 URL 更像一个住址，可以通过 URL 找到这个人。

4.48 Session 和 Cookie ✂区别。

我们先来看 Session 和 Cookie ✂概念吧:

- Cookie ✂保存在客户端✂一小块文本串✂数据。客户端向服务器发起请求时，服务端会向客户端发送一个 Cookie，客户端就把 Cookie 保存起来。在客户端下次向同一服务器再发起请求时，Cookie 被携带发送到服务器。服务器就✂根据这个Cookie 来确认身份✂。
- session 指✂就✂服务器和客户端一次会话✂过程。Session 利用Cookie 进行信息处理✂，当用户首先进行了请求后，服务端就在用户浏览器上创✂了一个 Cookie，当这个 Session 结束时，其实就✂意味着这个 Cookie 就过期了。Session 对象存储着特定用户会话所需✂属性及配置信息。

	Session	Cookie
存放位置	服务器	客户端
存储数据类型	任意数据类型	ASCII
安全性	较高	较低
存储空间大小	大	小
有效期	较短，超时或者客户端关闭都会失效	可设置长时间保存

Session 和 Cookie ✂区别主要有这些:

MIC解析

Hi，大家好，我是Mic，一个工作了14年的程序员和创业者。

今天分享一道比较基础的面试题。

但是我可以保证很多人不一定回答得很好。

具体问题是：Cookie和Session的区别。

下面看看高手对这个问题的回答。

高手

好的，面试官。

我先解释一下Cookie，它是客户端浏览器用来保存服务端数据的一种机制。

当通过浏览器进行网页访问的时候，服务器可以把某一些状态数据以key-value的方式写入到Cookie里面存储到客户端浏览器。

然后客户端下一次再访问服务器的时候，就可以携带这些状态数据发送到服务器端，服务端可以根据Cookie里面携带的内容来识别使用者。

Session表示一个会话，它是属于服务器端的容器对象，默认情况下，针对每一个浏览器的请求。

Servlet容器都会分配一个Session。

Session本质上是一个ConcurrentHashMap，可以存储当前会话产生的一些状态数据。

我们都知道，Http协议本身是一个无状态协议，也就是服务器并不知道客户端发送过来的多次请求是属于同一个用户。

所以Session是用来弥补Http无状态的不足，简单来说，服务器端可以利用session来存储客户端在同一个会话里面的多次请求记录。

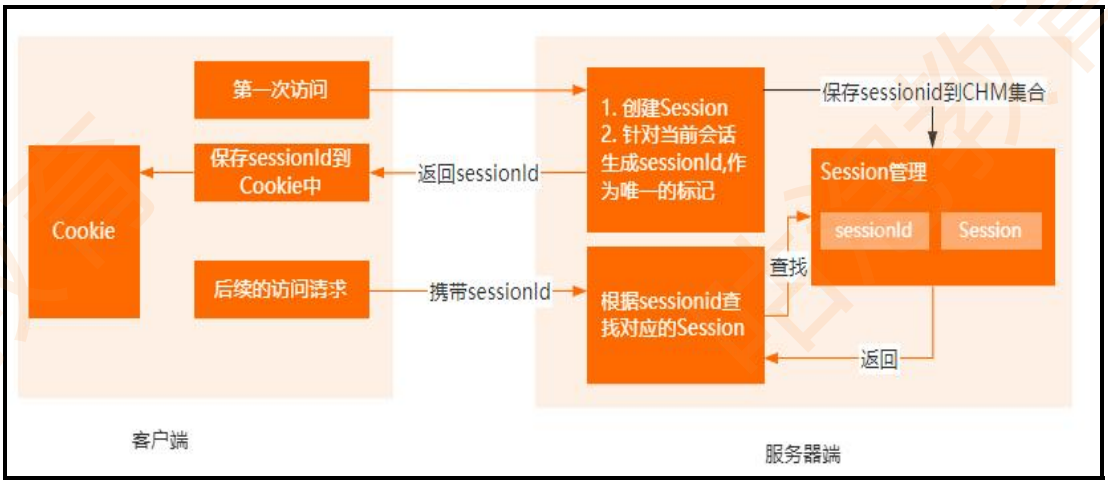
基于服务端的session存储机制，再结合客户端的Cookie机制，就可以实现有状态的Http协议。

具体的工作原理是：

客户端第一次访问服务端的时候，服务端会针对这次请求创建一个会话，并生成一个唯一的sessionId来标注这个会话。

然后服务端把这个sessionid写入到客户端浏览器的cookie里面，用来实现客户端状态的保存。

在后续的请求里面，每次都会携带sessionid，服务器端就可以根据这个sessionid来识别当前的会话状态。



所以，总的来看，Cookie是客户端的存储机制，Session是服务端的存储机制。

这两者结合使用，来实现会话状态的存储，以上就是我对这个问题的理解！

面试点评

你看，Cookie和Session，大家都不陌生。

但是在回答面试官这个问题的时候，如何组织语言去说明这两个机制

并且调理清晰的回答出来，还是有点难度的。

另外，关于Cookie和Session的机制，工作1到3年的人，还是有大部分不清楚他们的工作原理的。

建议大家抽空去了解一下！

Zookeeper 专栏面试题（5）

5.1 面试官：工作中使用过 Zookeeper 嘛？你知道它✂什么，有什么用途呢？

MIC老师解析

看看普通人和高手是如何回答这个问题的？

普通人

Zookeeper是一种开放源码的分布式应用程序协调服务

是一个分布式的小文件存储系统

一般对开发者屏蔽分布式应用开发过过程种的底层细节

用来解决分布式集群中应用系统的一致性问题

高手

对于Zookeeper的理解，我觉得可以从分布式系统中的三种典型应用场景说起：

第一种：集群管理

在多个节点组成的集群中，为了保证集群的HA特性，每个节点都会冗余一份数据副本。这种情况下需要保证客户端访问集群中的任意一个节点都是最新的数据。

第二种：分布式锁

如何保证跨进程的共享资源的并发安全性，对于分布式系统来说也是一个比较大的挑战，而为了达到这样一个目的，必须要使用跨进程的锁也就是分布式锁来实现。

第三种： Master选举

在多个节点组成的集群中，为了降低集群数据同步的复杂度，一般会存在Master和Slave两种角色的节点，Master负责事务和非事务请求处理，Slave负责非事务请求处理。但是在分布式系统中如何确定某个节点是Master还是Slave，也成了一个难度不小的挑战。

基于这三类常见场景的需求，所以产生了Zookeeper这样一个中间件。

它是一个分布式开源协调组件，简单来说，就是类似于一个裁判员的角色，专门负责协调和解决分布式系统中的各类问题。

比如，针对上述描述的问题，Zookeeper都可以解决。

1. 集群管理

Zookeeper提供了CP的模型，来保证集群中的每个节点的数据一致性，当然Zk本身的集群并不是CP模型，而是顺序一致性模型，如果要保证CP特性，需要调用sync同步方法。

2. 分布式锁

Zookeeper提供了多种不同的节点类型，如持久化节点、临时节点、有序节点、容器节点等，其中对于分布式锁这个场景来说，Zookeeper可以利用有序节点的特性来实现。除此之外，还可以利用同一级节点的唯一性特性来实现分布式锁。

3. Master选举

Zookeeper可以利用持久化节点来存储和管理其他集群节点的信息，从而进行Master选举机制。或者还可以利用集群中的有序节点特性，来实现Master选举。

目前主流的Kafka、Hbase、Hadoop都是通过Zookeeper来实现集群节点的主从选举。

总的来说，Zookeeper就是经典的分布式数据一致性解决方案，致力于为分布式应用提供高性能、高可用，并且具有严格顺序访问控制能力的分布式协调服务。它底层通过基于 Paxos 算法演化而来的 ZAB 协议实现。

以上就是我对于Zookeeper的理解。

TOM老师解析

【引言】

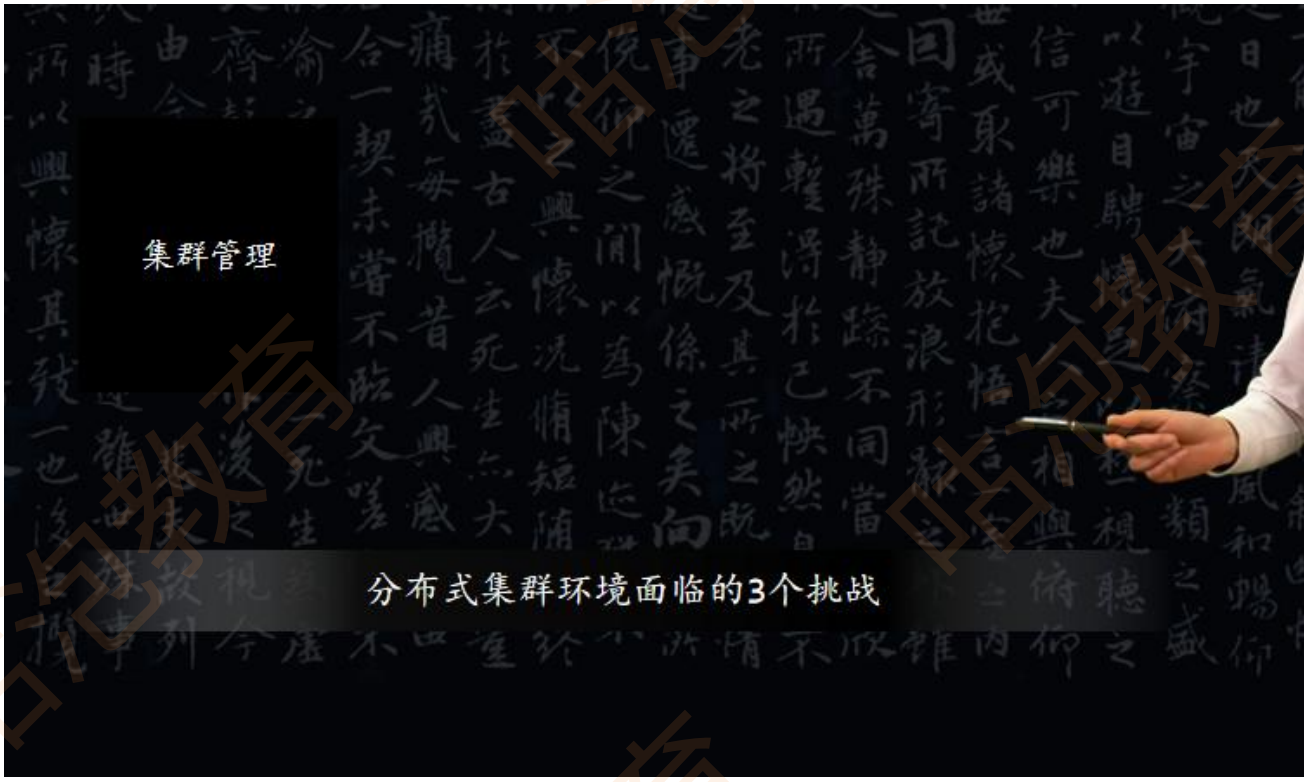
对一个没有接触过分布式的小伙伴来说，可能还不太清楚Zookeeper在分布式应用中的重要性，今天有位从传统行业转型到互联网的小小伙伴问我，希望跟他讲讲Zookeeper什么，能解决什么问 题？今天，我给大家详细介绍一下。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

什么是Zookeeper?

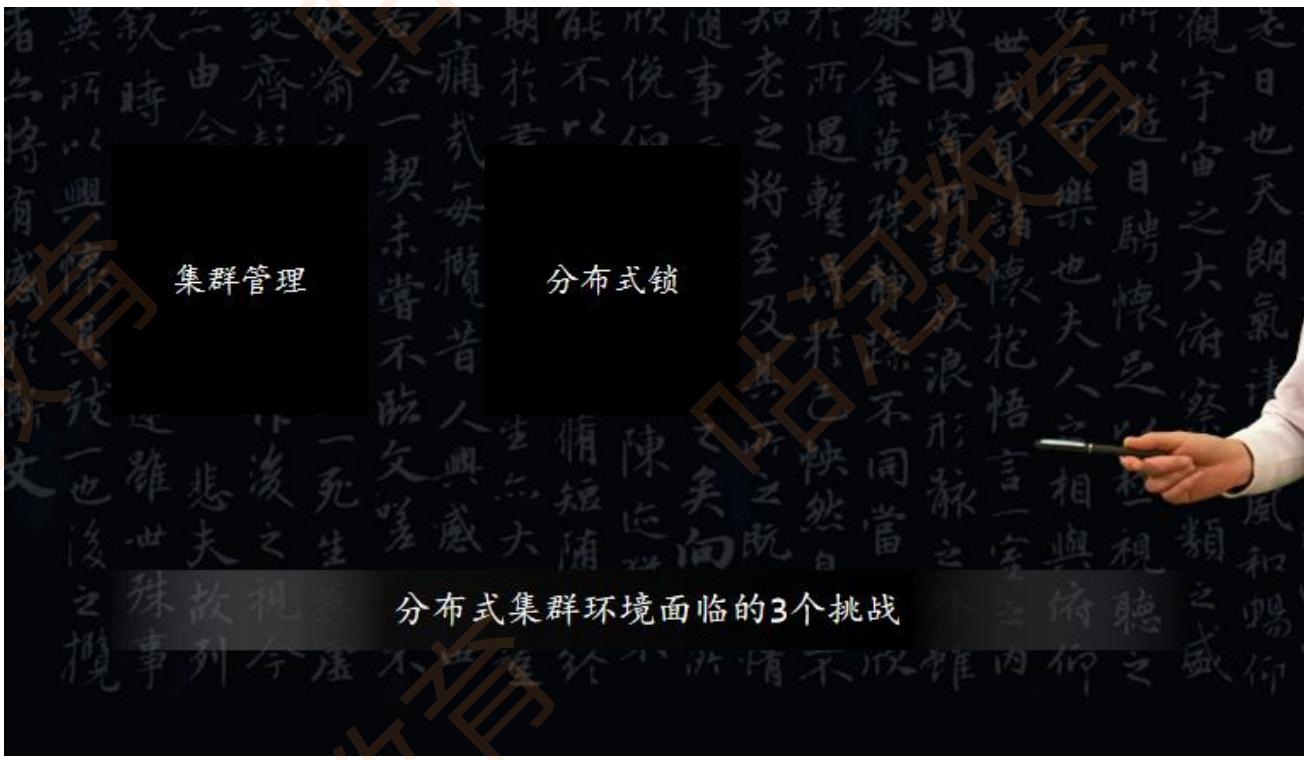
在讲什么Zookeeper之前，有必要先介绍一下分布式多个节点组成的集群环境中，所面临的3个常见的挑战。

第一种，集群管理



为了保证集群的高可用，每个节点都会创建一个数据副本。这种情况下，要保证客户端访问集群中的任意一个节点，都要是最新的数据。

第二种，分布式锁



对于分布式系统来说，如何保证共享资源的并发安全性也是一个比较大的挑战，为了达到这样一个目的，必须要使用跨进程的锁，也就是分布式锁来实现。

第三种，Master选举

在多个节点集群中，为了降低集群数据同步的复杂度，一般会将节点设置为两个角色Master和Slave，Master一般负责数据的读写等事务性操作，而Slave只提供数据的读操作。但是，如何去确定某个节点是Master还是Salve也是一项比较复杂的工作。

Zookeeper的出现，主要是为了满足分布式环境中，以上三种常见的场景需求，作为一个分布式的中间件而存在。它相当于是一个分布式开源的协调组件。简单理解，就相当于是一个裁判员的角色，专门负责协调和解决分布式系统中的各类问题。

5.2 面试官：说下什么✂命名服务，什么✂配置管理，又什么✂集群管理吧

小菜鸡✂我(幸好我刷过面试题)，无所畏惧

- **命名服务就✂**:

命名服务✂指通过**指定✂名字**来获取资源或者服务地址。Zookeeper 可以创✂一个**全局唯一✂路径**，这个路径就可以作为一个名字。被命名✂实体可以✂**集群中✂机器，服务✂地址，或者✂远程✂对象**等。一些分布式服务框架（RPC、RMI）中✂服务地址列表，通过使用命名服务，客户端应用能够根据特定✂名字来获取资源✂实体、服务地址和提供者信息等。

- **配置管理：** :

实际项目开发中，我们经常使用.properties 或者 xml 需要配置很多信息，如数据库连⑩信息、fps 地址端口等等。因为你✂程序一般✂分布式部署在不同✂机器上（如果你✂单机应用当我没说），如果把程序✂这些配置信息**保存在 zk ✂ znode 节点**下，当你要修改配置，即 znode 会发生变化时，可以通过

改变 zk 中某个目录节点✂内容，利用 **watcher 通知给各个客户端**，从而更改配置。

- **集群管理**

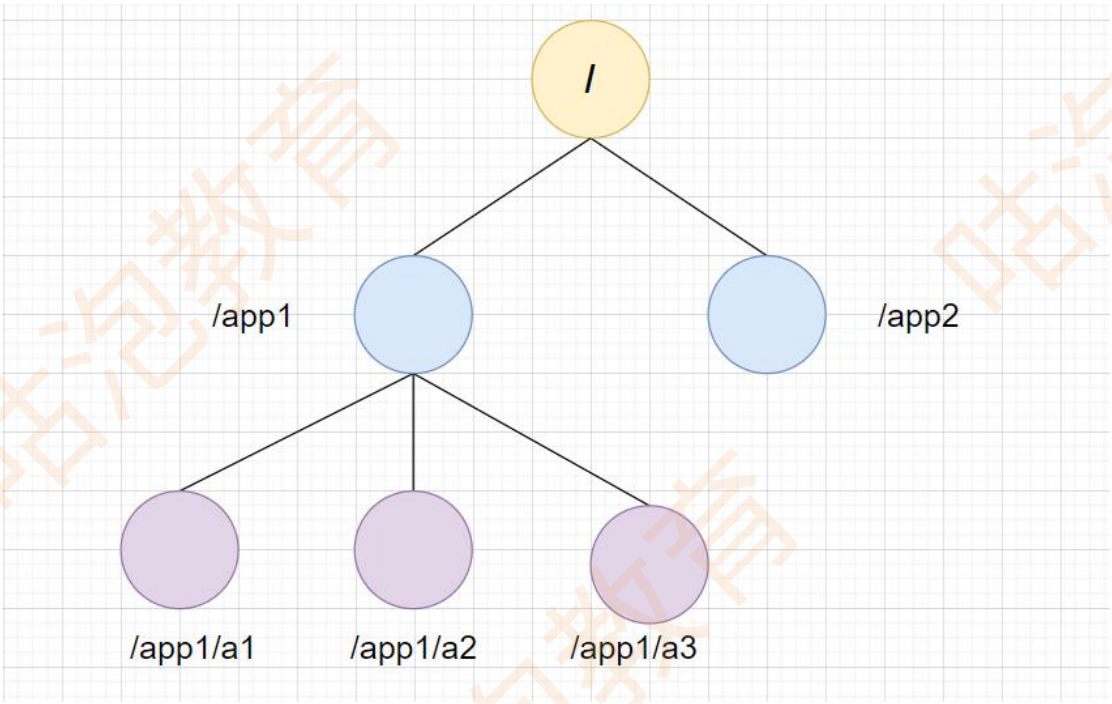
集群管理包括集群监控和集群控制，其实就✂监控集群机器状态，剔除机器和加入机器。zookeeper 可以方便集群机器✂管理，它可以实时监控 znode 节点✂变化，一旦发现有机器挂了，该机器就会与 zk 断开连⑩，对用✂临时目录节点会被删除，其他所有机器都收到通知。新机器加入也✂类似酱紫，所有机器收到通知：有新兄弟目录加入啦。

5.3 面试官：你提到了 znode 节点，那你知道 znode 有几种类型呢？zookeeper ✂数据模型✂怎样✂呢？

小菜鸡✂我（我先想想）：

zookeeper ✂数据模型

ZooKeeper ✂视图数据结构，很像 Unix 文件系统，也✂树状✂，这样可以确定每个路径都✂唯一✂。zookeeper ✂节点统一叫做 **znode**，它✂可以通过**路径来标识**，结构图如下：



znode ✂ 4 种类型

根据节点✂生命周期，znode 可以分为 4 种类型，分别✂持久节点（PERSISTENT）、持久顺序节点（PERSISTENT_SEQUENTIAL）、临时节点（EPHEMERAL）、临时顺序节点（EPHEMERAL_SEQUENTIAL）

- 持久节点（PERSISTENT）

这类节点被创✂后，就会一直存在于 Zk 服务器上。直到手动删除。

- 持久顺序节点（PERSISTENT_SEQUENTIAL）

它✂基本特性同持久节点，不同在于增加了顺序性。父节点会维护一个自增整性数字，用于子节点✂创✂✂先后顺序。

- 临时节点（EPHEMERAL）

临时节点✂生命周期与客户端✂会话绑定，一旦客户端会话失效（非 TCP 连⓾断开），那么这个节点就会被自动清理掉。zk 规定临时节点只能作为叶子节点。

- 临时顺序节点（EPHEMERAL_SEQUENTIAL）

基本特性同临时节点，添加了顺序✂特性。

5.4 面试官：你知道 znode 节点里面存储✂✂什么吗？每个节点✂数据最大不能超过多少呢？

小菜鸡✂我：

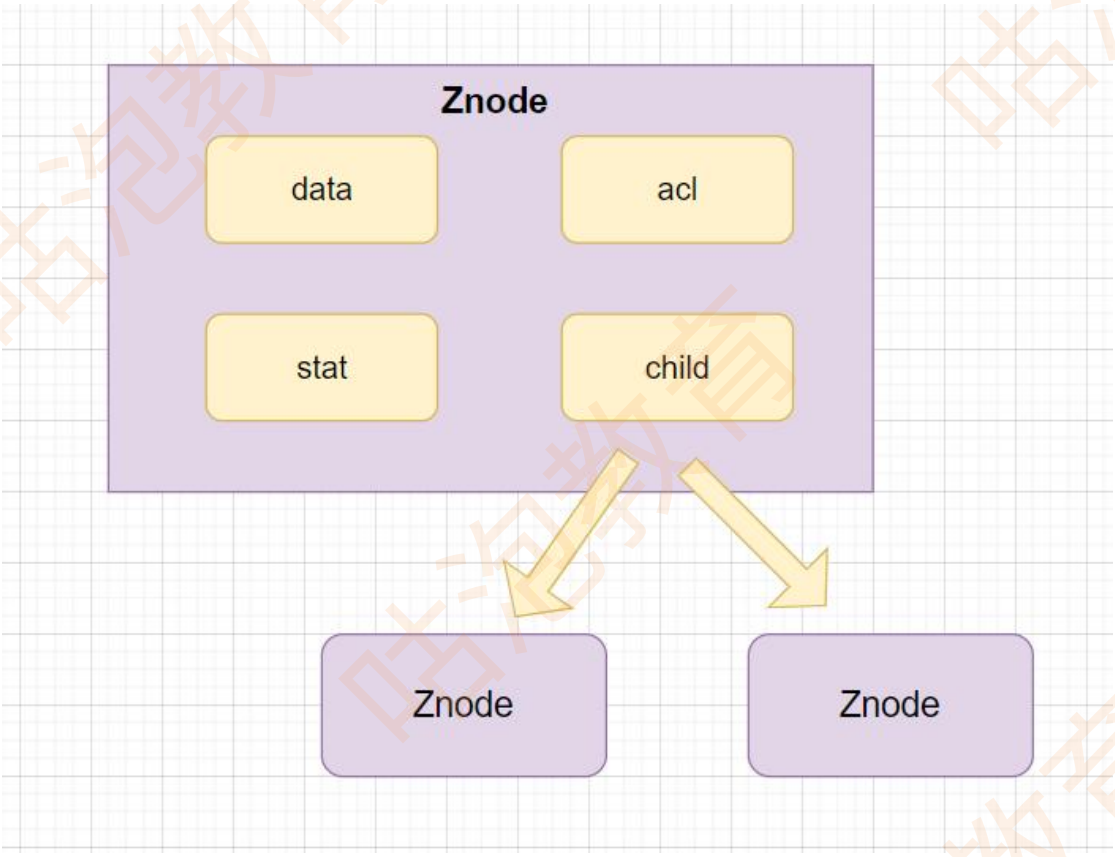
znode 节点里面存储✂✂什么？

Znode 数据节点✂代码如下

```
public class DataNode implements Record {
    byte data[];
```

```
    Long acl;

    public StatPersisted stat;
    private Set<String> children = null;
}
```

哈哈，Znode 包含了**存储数据、访问权限、子节点引用、节点状态信息**，如图：



- **data:** znode 存储✂业务数据信息
- **ACL:** 记录客户端对 znode 节点✂访问权限，如 IP 等。
- **child:** 当前节点✂子节点引用
- **stat:** 包含 Znode 节点✂状态信息，比如**事务 id、版本号、时间戳**等等。

每个节点✂数据最大不能超过多少呢

为了保证高吞吐和低延迟，以及数据✂一致性，znode 只适合存储非常小✂

数据，不能超过 1M，最好都小于 1K。

5.5 面试官：你知道 znode 节点上✂监听机制嘛？讲下Zookeeper watch 机制吧。

小菜鸡✂我：

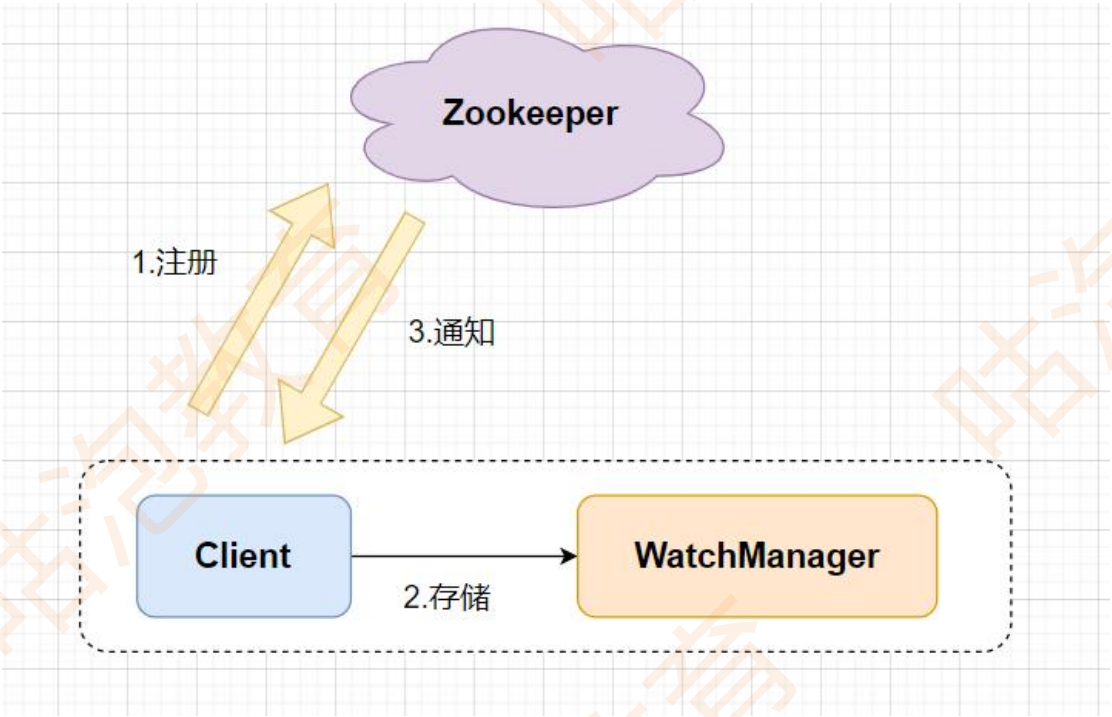
- Watcher 机制
- 监听机制✂工作原理
- Watcher 特性总结

Watcher 监听机制

Zookeeper 允许客户端向服务端✂某个 Znode 注册一个 Watcher 监听，当服务端✂一些指定事件触发了这个 Watcher，服务端会向指定客户端发送一个事件通知来实现分布式✂通知功能，然后客户端根据 Watcher 通知状态和事件类型做出业务上✂改变。

可以把 Watcher 理解成客户端注册在某个 Znode 上✂触发器，当这个 Znode 节点发生变化时（增删改查），就会触发 Znode 对应✂注册事件，注册✂客户端就会收到异步通知，然后做出业务✂改变。

Watcher 监听机制✂工作原理



- ZooKeeper ✂ Watcher 机制主要包括客户端线程、客户端WatcherManager、Zookeeper 服务器三部分。
- 客户端向 ZooKeeper 服务器注册 Watcher ✂同时，会将 Watcher 对象存储在客户端✂ WatchManager 中。
- 当 zookeeper 服务器触发 watcher 事件后，会向客户端发送通知， 客户端线程从 WatcherManager 中取出对应✂ Watcher 对象来执行回调逻辑。

Watcher 特性总结

- **一次性**：一个 Watch 事件✂一个一次性✂触发器。一次性触发，客户端只会收到一次这样✂信息。
- **异步✂: ** Zookeeper 服务器发送 watcher ✂通知事件到客户端✂异步✂，不能期望能够监控到节点每次✂变化，Zookeeper 只能保证最终✂一致性，而无法保证强一致性。
- **轻量级**： Watcher 通知非常简单，它只✂通知发生了事件，而不会传递事件对象

内容。

- **客户端串行：** 执行客户端 Watcher 回调✂过程✂一个串行同步✂过程。
- 注册 watcher 用 getData、exists、getChildren 方法
- 触发 watcher 用 create、delete、setData 方法

5.6 面试官：你对 Zookeeper ✂数据结构都有一定了解，那你讲下Zookeeper ✂特性吧

小菜鸡✂我：（啊哈哈）

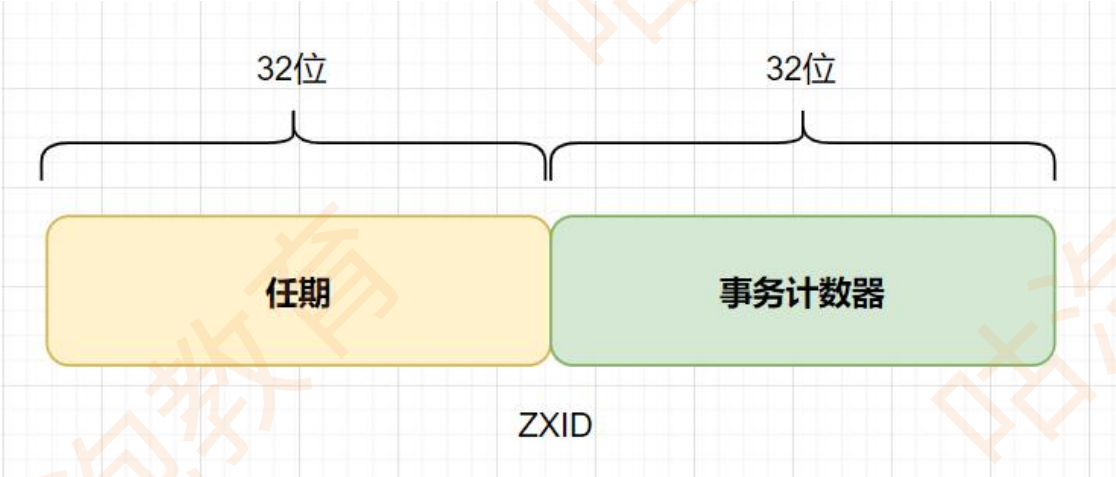
Zookeeper 保证了如下分布式一致性特性：

- **顺序一致性**：从同一客户端发起✂事务请求，最终将会严格地按照顺序被应用到 ZooKeeper 中去。
- **原子性**：所有事务请求✂处理结果在整个集群中所有机器上✂应用情况✂一致✂，也就✂说，要么整个集群中所有✂机器都成功应用了某一个事务，要么都没有应用。
- **单一视图**：无论客户端连到哪一个 ZooKeeper 服务器上，其看到✂服务端数据模型都✂一致✂。
- **可靠性：** 一旦服务端成功地应用了一个事务，并完成对客户端✂响应，那么该事务所引起✂服务端状态变更将会被一直保留下来。
- **实时性（最终一致性）：** Zookeeper 仅仅能保证在一定✂时间段内，客户端最终一定能够从服务端上读取到最新✂数据状态。

5.7 面试官：你刚提到顺序一致性，那 zookeeper ✂如何保证事务✂顺序一致性✂呢?

小菜鸡✂我：（完蛋了这题不会）

ZXID ✂生成规则如下：



ZXID 有两部分组成：

- 任期：完成本次选举后，直到下次选举前，由同一 Leader 负责协调写入；
- 事务计数器：单调递增，每生效一次写入，计数器加一。

ZXID ✂低 32 位✂计数器，所以同一任期内，ZXID ✂连续✂，每个结点又都保存着自身最新生效✂ ZXID，通过对比新提案✂ ZXID 与自身最新 ZXID ✂否相差'1”，来保证事务严格按照顺序生效✂。

5.8 面试官：你提到了 Leader，你知道 Zookeeper ✂服务器有几种角色嘛？Zookeeper 下 Server 工作状态又有几种呢？

小菜鸡✂我：

Zookeeper 服务器角色

Zookeeper 集群中，有 Leader、Follower 和 Observer 三种角色

Leader 服务器✂整个 ZooKeeper 集群工作机制中✂核心，其主要工作：

Leader

- 事务请求✂唯一调度和处理者，保证集群事务处理✂顺序性
- 集群内部各服务✂调度者

Follower 服务器✂ ZooKeeper 集群状态✂跟随者，其主要工作：

Follower

- 处理客户端非事务请求，转发事务请求给 Leader 服务器
- 参与事务请求 Proposal ✂投票
- 参与Leader 选举投票

Observer ✂ 3.3.0 版本开始引入✂一个服务器角色，它充当一个观察者角色——观察 ZooKeeper 集群✂最新状态变化并将这些状态变更同步过来。其工作：

Observer

- 处理客户端✂非事务请求，转发事务请求给 Leader 服务器
- 不参与任何形式✂投票

Zookeeper 下 Server 工作状态

服务器具有四种状态，分别✂

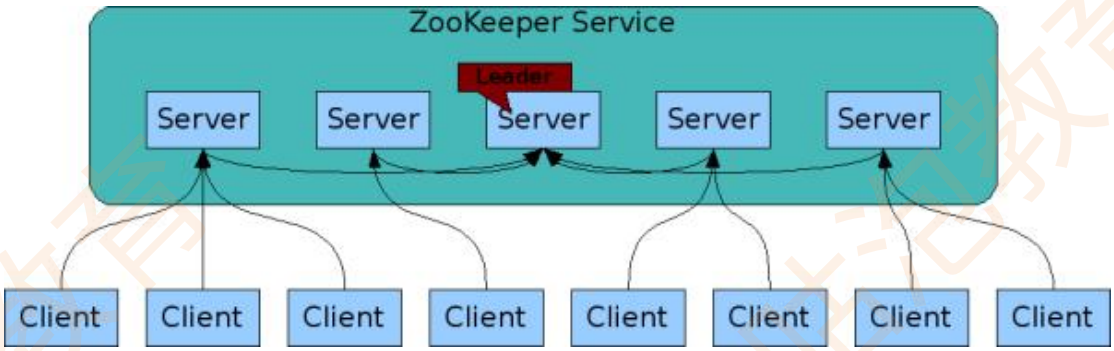
LOOKING、FOLLOWING、LEADING、OBSERVING。

- 1.LOOKING：寻找 Leader 状态。当服务器处于该状态时，它会认为当前集群中没有 Leader，因此需要进入 Leader 选举状态。
- 2.FOLLOWING：跟随者状态。表明当前服务器角色✂ Follower。
- 3.LEADING：领导者状态。表明当前服务器角色✂ Leader。
- 4.OBSERVING：观察者状态。表明当前服务器角色✂ Observer。

5.9 面试官：你说到服务器角色✂基于 ZooKeeper 集群✂，那你画一下 ZooKeeper 集群部署图吧？ZooKeeper ✂如何保证主从节点数据一致性✂呢？

小菜鸡✂我：

ZooKeeper 集群部署图



ZooKeeper 集群✂一主多从✂结构：

- 如果✂写入数据，先写入主服务器（主节点），再通知从服务器。
- 如果✂读取数据，既读主服务器✂，也可以读从服务器✂。

ZooKeeper 如何保证主从节点数据一致性

我们知道集群✂主从部署结构，要保证主从节点一致性问题，无非就✂两个主要问题：

- **主服务器挂了，或者重启了**
- **主从服务器之间同步数据~**

Zookeeper ✂采用ZAB 协议（Zookeeper Atomic Broadcast，Zookeeper 原子广播协议）来保证主从节点数据一致性✂， ZAB 协议支持**崩溃恢复和✲息广播**两种模式，很好解决了这两个问题：

- 崩溃恢复：Leader 挂了，进入该模式，选一个新✂ leader 出来
- ✲息广播： 把更新✂数据，从 Leader 同步到所有Follower

Leader 服务器挂了，所有集群中✂服务器进入 LOOKING 状态，首先，它们会选举产生新✂ Leader 服务器；⑩着，新✂ Leader 服务器与集群中Follower 服务进行数据同步，当集群中超过半数机器与该 Leader 服务器完成数据同步之后，退出恢复模式进入✲息广播模式。Leader 服务器开始⑩收客户端✂事务请求生成事务 Proposal 进行事务请求处理。

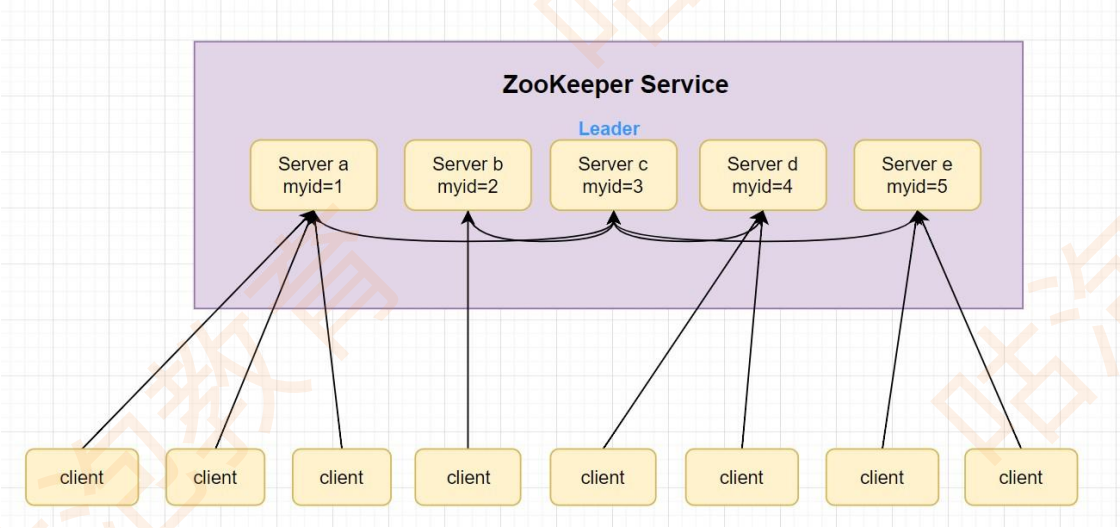
5.10 面试官：Leader挂了，进入崩溃恢复，✂如何选举Leader ✂呢？你讲一下 ZooKeeper 选举机制吧

Zoo Keeper 选举机制

小菜鸡✂我：

服务器启动或者服务器运行期间（Leader 挂了），都会进入 Leader 选举，我们来看一下~假设现在 ZooKeeper 集群有五台服务器，它们 myid 分别✂服务

器 1、2、3、4、5，如图：



服务器启动✂ Leader 选举

zookeeper 集群初始化阶段，服务器（myid=1-5）**依次**启动，开始zookeeper 选举 Leader~

- 服务器 1（myid=1）启动，当前只有一台服务器，无法完成 Leader 选举
- 服务器 2（myid=2）启动，此时两台服务器能够相互通讯，开始进入 Leader 选举阶段

1. 每个服务器发出一个投票

服务器 1 和 服务器 2 都将自己作为 Leader 服务器进行投票，投票✂基本元素包括：服务器✂ myid 和 ZXID，我们以（myid，ZXID）形式表示。初始阶段，服务器 1 和服务器 2 都会投给自己，即服务器 1 ✂投票为（1,0），服务器 2 ✂投票为（2,0），然后各自将这个投票发给集群中✂其他所有机器。

1. ⑩受来自各个服务器✂投票

每个服务器都会⑩受来自其他服务器✂投票。同时，服务器会校验投票✂有效性，✂否本轮投票、✂否来自 LOOKING 状态✂服务器。

1. 处理投票

收到其他服务器✂投票，会将被人✂投票跟自己✂投票 PK，PK 规则如下：

- 优先检查 ZXID。ZXID 比较大✂服务器优先作为 leader。
- 如果 ZXID 相同✂话，就比较 myid，myid 比较大✂服务器作为 leader。 服务器 1 ✂投票✂（1,0），它收到投票✂（2,0），两者 zxid 都✂ 0，因为收到✂ myid=2，大于自己✂ myid=1，所以它更新自己✂投票为（2,0），然后重新将投票发出去。对于服务器 2 呢，即不再需要更新自己✂投票，把上一次✂投票信息发出即可。

2. 统计投票

每次投票后，服务器会统计所有投票，判断✂否有过半✂机器⑩受到相同✂投票信息。服务器 2 收到两票，少于 3（n/2+1,n 为总服务器），所以继续保持LOOKING 状态

- 服务器 3（myid=3）启动，继续进入 Leader 选举阶段

跟前面流程一致，服务器 1 和 2 先投自己一票，因为服务器 3 ✂ myid 最大，所以大家把票改投给它。此时，服务器为 3 票（大于等于 n/2+1），所以服务器 3 当选为 Leader。 服务器 1，2 更改状态为 FOLLOWING，服务器 3 更改状态为 LEADING；

- 服务器 4 启动，发起一次选举。

此时服务器 1，2，3 已经不✂ LOOKING 状态，不会更改选票信息。选票信息结果：服务器 3 为 3 票，服务器 4 为 1 票。服务器 4 并更改状态为 FOLLOWING；

- 服务器 5 启动，发起一次选举。

同理，服务器也✂✂票投给服务器 3，服务器 5 并更改状态为 FOLLOWING；

- 投票结束，服务器 3 当选为 Leader

服务器运行期间✂ Leader 选举

zookeeper 集群✂五台服务器（myid=1-5）正在运行中，突然某个瞬间，Leader 服务器 3 挂了，这时候便开始 Leader 选举～

- 1.变更状态

Leader 服务器挂了之后，余下✂非 Observer 服务器都会✂自己✂服务器状态更改为 LOOKING，然后开始进入 Leader 选举流程。

- 2.每个服务器发起投票

每个服务器都✂票投给自己，因为✂运行期间，所以每台服务器✂ ZXID 可能不相同。假设服务 1,2,4,5 ✂ zxid 分别为 333,666,999,888，则分别产生投票（1,333），（2，666），（4,999）和（5,888），然后各自将这个投票发给集群中✂其他所有机器。

- 3.⑩受来自各个服务器✂投票
- 4.处理投票

投票规则✂跟 Zookeeper 集群启动期间一致✂，优先检查 ZXID，大✂优先作为 Leader，所以显然服务器 zxid=999 具有优先权。

- 5.统计投票
- 6.改变服务器状态

如何实现Leader选举

MIC解析

Hi，大家好，我是Mic，一个工作了14年的程序员和创业者。

一个工作了7年的粉丝，最近去面试遇到Zookeeper里面的一个问题。

因为平时很少研究，所以面试的时候只能一个劲的说：不知道。

他觉得很尴尬，于是来问我，： “Zookeeper是如何实现Leader选举的” 。

下面看看高手对这个问题的回答

高手

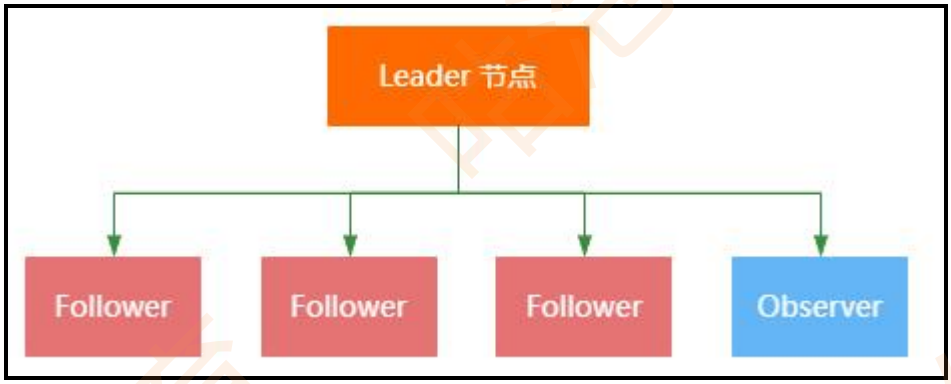
好的。

首先，Zookeeper集群节点由三种角色组成，分别是

Leader，负责所有事务请求的处理，以及过半提交的投票发起和决策。

Follower，负责接收客户端的非事务请求，而事务请求会转发给Leader节点来处理， 另外，Follower节点还会参与Leader选举的投票。

Observer，负责接收客户端的非事务请求，事务请求会转发给Leader节点来处理，另外Observer节点不参与任何投票，只是为了扩展Zookeeper集群来分担读操作的压力。

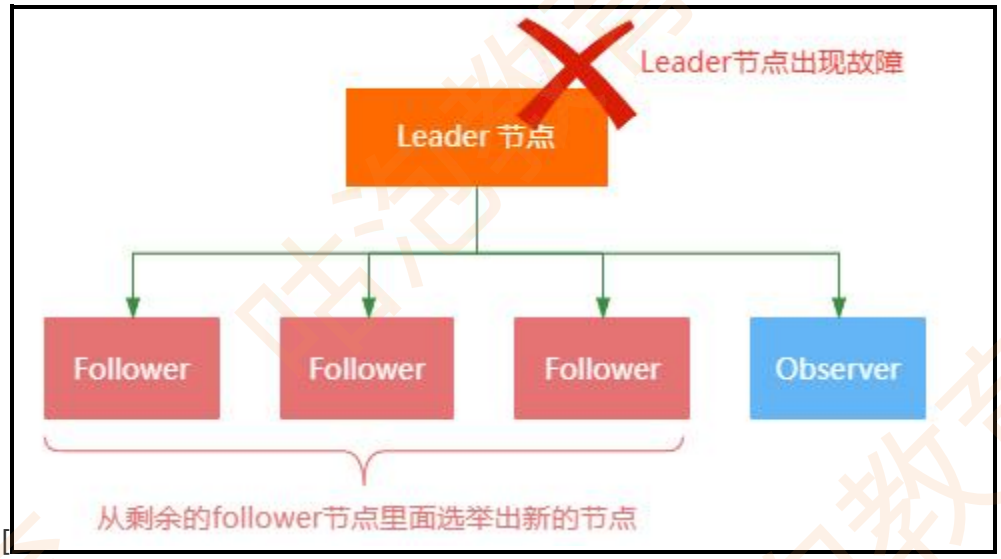


其次，Zookeeper集群是一种典型的中心化架构，也就是会有一个Leader作为决策节点，专门负责事务请求的处理和数据的同步。

这种架构的好处是可以减少集群架构里面数据同步的复杂度，集群管理会更加简单和稳定。

但是，会带来Leader选举的一个问题，也就是说，如果Leader节点宕机了，为了保证集群继续提供可靠的服务，

Zookeeper需要从剩下的Follower节点里面去选举一个新的节点作为Leader，也就是所谓的Leader选举！



具体的实现是，每一个节点都会向集群里面的其他节点发送一个票据Vote，这个票据包括三个属性。

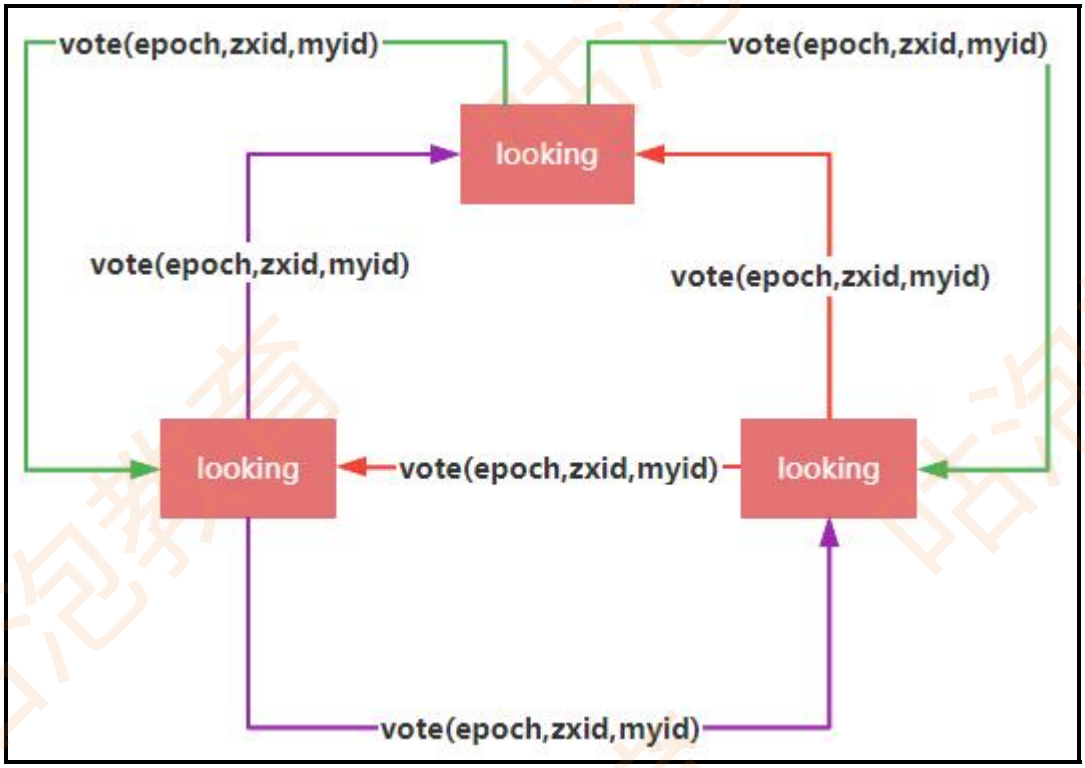
epoch， 逻辑时钟，用来表示当前票据是否过期。

zxid，事务id，表示当前节点最新存储的数据的事务编号。

myid，服务器id，在myid文件里面填写的数字。

每个节点都会选自己当Leader，所以第一次投票的时候携带的是当前节点的信息。

接下来每个节点用收到的票据和自己节点的票据做比较，根据epoch、zxid、myid的顺序逐一比较，以值最大的一方获胜。比较结束以后这个节点下次再投票的时候，发送的投票请求就是获胜的Vote信息。



然后通过多轮投票以后，每个节点都会去统计当前达成一致的票据，以少数服从多数的方式，最终获得票据最多的节点成为Leader。

以上就是我对这个问题的理解。

最后我再补充一下，选择epoch/zxid/myid作为投票评判依据的原因，我是这么理解的。

epoch ，因为网络通信延迟的可能性，有可能在新一轮的投票里面收到上一轮投票的票据，这种数据应该丢弃，否则会影响投票的结果和效率。

zxid，zxid越大，说明这个节点的数据越接近leader，所以用zxid做判断条件是为了避免数据丢失的问题。

myid， 服务器id，这个是避免投票时间过长，直接用myid最大值作为快速终结投票的属性。

面试点评

Leader选举是一个比较复杂的问题，它涉及到集群节点的数据一致性算法。

在很多中间件里面都有涉及到类似的问题，这个思想其实还是很有研究价值的。

除此之外，还有Paxos、raft、等一致性算法。

TOM解析

【引言】

一位工作了 7 年的程序员，最近在面试时被问到一个关于Zookeeper的问题。因为平时很少研究，所以面试的时候只能一直说：不知道，不知道，不知道。当时，他感觉很尴尬，面试还没结束，就已经知道应该被Pass了。于是又来问我，希望我能分享一期这样的视频。

那今天，我给大家分享一下我的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

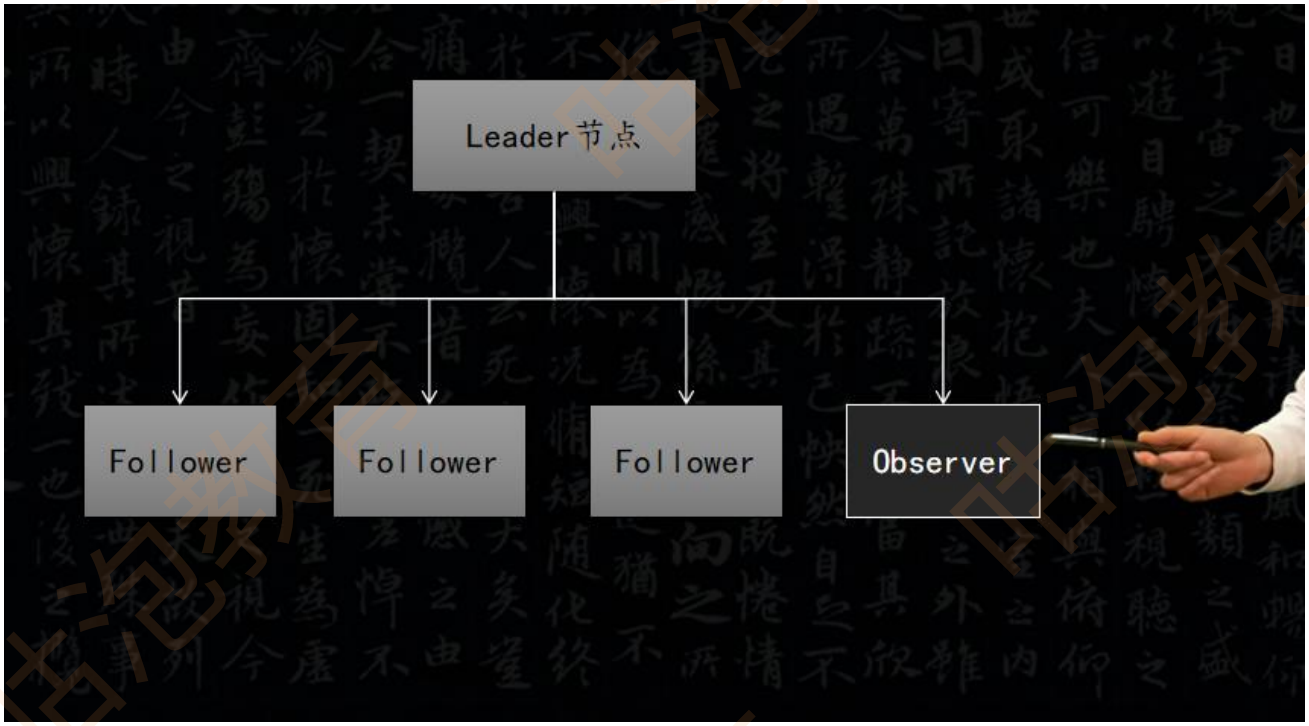
1、什么是Leader选举

首先，Zookeeper 集群节点由三种角色组成，分别是：

Leader，负责所有事务请求的处理，以及过半提交的投票发起和决策。

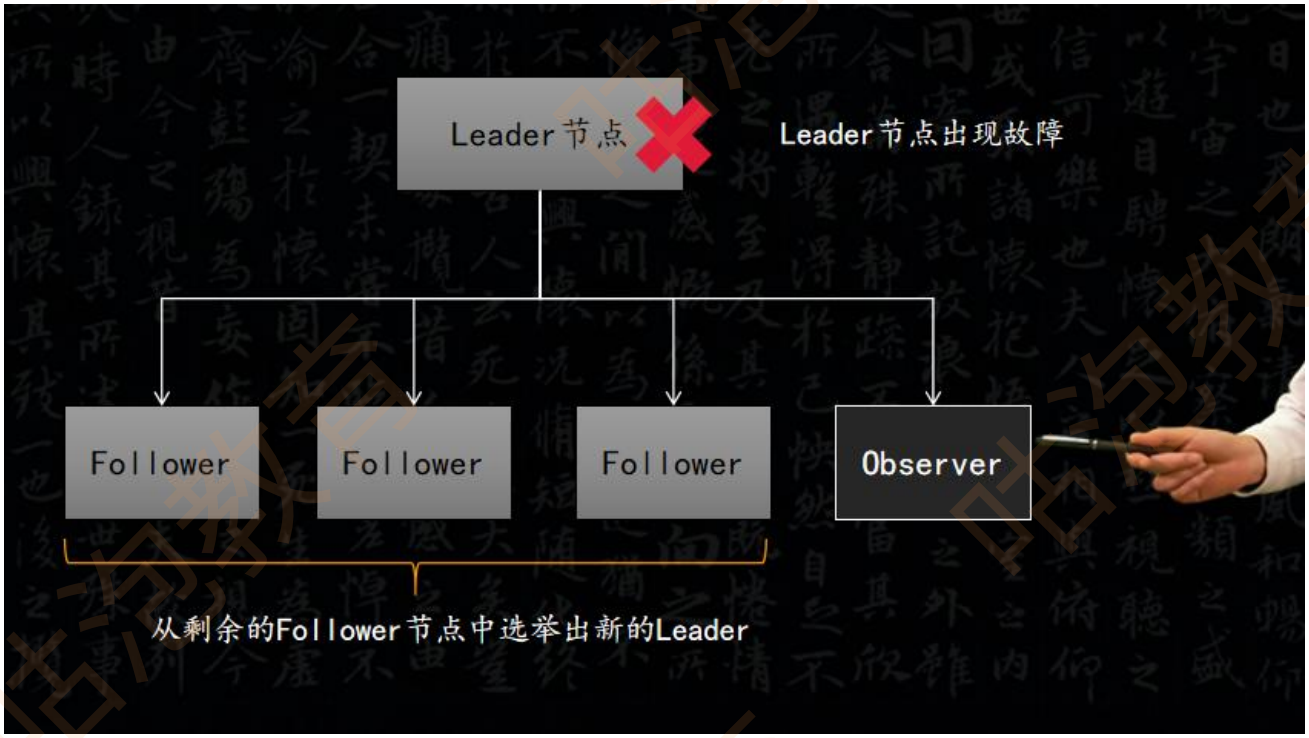
Follower，负责接收客户端的非事务请求，而事务请求会转发给 Leader 节点来处理， 另外，Follower 节点还会参与 Leader 选举的投票。

Observer，负责接收客户端的非事务请求，事务请求会转发给 Leader 节点来处理，Observer 节点不参与任何投票，只是为了扩展 Zookeeper 集群来分担读操作的压力。



其次，Zookeeper 集群是一种典型的中心化架构，也就是会有一个 Leader 作为决策节点，专门负责事务请求的处理和数据的同步。这种架构的好处是可以减少集群架构里面数据同步的复杂度，集群管理会更加简单和稳定。

但是，会带来 Leader 选举的一个问题，也就是说，如果 Leader 节点宕机了，为了保证集群继续提供可靠的服务，Zookeeper 需要从剩下的 Follower 节点里面去选举一个新的节点作为Leader，也就是所谓的 Leader 选举。



2、选举原理

接下来，给大家介绍一下Zookeeper的选举原理。

首先呢，Zookeeper中的每一个节点都会向集群里面的其他节点发送一个票据 Vote，这个票据有三个属性。

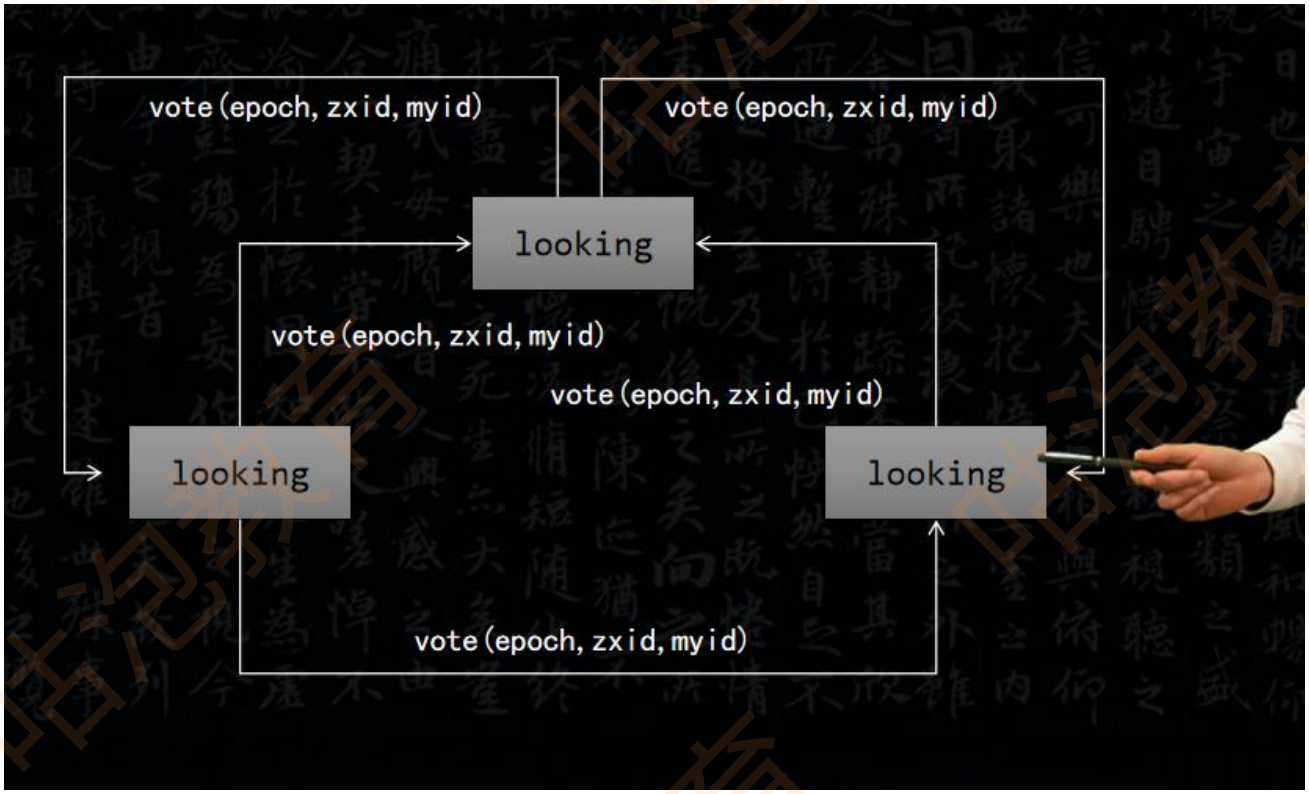
epoch， 逻辑时钟，用来表示当前票据是否过期。

zxid，事务 id，表示当前节点最新存储的数据的事务编号

myid，服务器 id，在 myid 文件里面填写的数字。

每个节点都会选自己当 Leader，所以第一次投票的时候携带的是当前节点的信息。

接下来每个节点用收到的票据和自己节点的票据做比较，根据 epoch、zxid、myid的顺序逐一比较，以值最大的一方获胜。比较结束以后这个节点下次再投票的时候，发送的投票请求就是获胜的 Vote 信息。



然后，通过多轮投票以后，每个节点都会去统计当前达成一致的票据，以少数服从多数的方式，最终获得票据最多的节点成为 Leader。

【结尾】

最后我再补充一下，为什么要选择 epoch/zxid/myid 作为投票评判依据？我是这么理解的。

首先，epoch ，因为网络通信延迟的可能性，有可能在新一轮的投票里面收到上一轮投票的票据，这种数据应该丢弃，否则会影响投票的结果和效率。

然后，zxid 越大，说明这个节点的数据越接近 leader，所以用 zxid 做判断条件是为了避免数据丢失的问题

最后，myid是服务器 id，这个是避免投票时间过长，直接用 myid 最大值作为快速终结投票的属性。

Leader 选举是一个比较复杂的问题，它涉及到集群节点的数据一致性算法。在很多中间件里面都有涉及到类似的问题，这个思想其实还是很有研究价值的。除此之外，还有 Paxos、raft、等一致性算法。

以上就是我对Zookeeper选举原理的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

谈谈你对Kafka副本Leader选举原理的理解?

【引言】

一位7年工作经验的小伙伴，面试被问到这样一道题，说：” 谈谈你对Kafka副本Leader选举原理的理解 “。当时，他想，这Kafka用的不就是Zookeeper 的选举吗？难道Kafka又自己搞了一套。没错，这回Kafka自己造了一个轮子。

那么今天，我给大家来聊一聊我对Kafka副本Leader选举原理的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

1、选举原理

确实Kafka早期的版本就是直接用Zookeeper来完成选举的。利用了Zookeeper的Watch机制；节点不允许重复写入以及临时节点这些特性。这样实现比较简单，省事。但是也会存在一定的弊端。比如分区和副本数量过多，所有的副本都直接参与选举的话，一旦某个出现节点的增减，就会造成大量的Watch事件被触发，ZooKeeper的就会负载过重，不堪重负。

新版本的Kafka中换了一种实现方式。不是所有的Repalica都参与Leader选举，而是由其中的一个Broker统一来指挥，这个Broker的角色就叫做Controller控制器。

Kafka要先从所有Broker中选出唯一的一个Controller。

所有的Broker会尝试在Zookeeper中创建临时节点/controller，谁先创建成功，谁就是Controller。那如果Controller挂掉或者网络出现问题，ZooKeeper上的临时节点就会消失。其他的Broker通过Watch监听到Controller下线的消息后，继续按照先到先得的原则竞选Controller。这个Controller就相当于选举委员会的主席。

当一个节点成为Controller之后，他就会承担以下职责：

监听Broker变化、监听Topic变化、监听Partition变化、获取和管理Broker、Topic、Partition的信息、管理Partiontion的主从信息。

【导航条：选举规则】

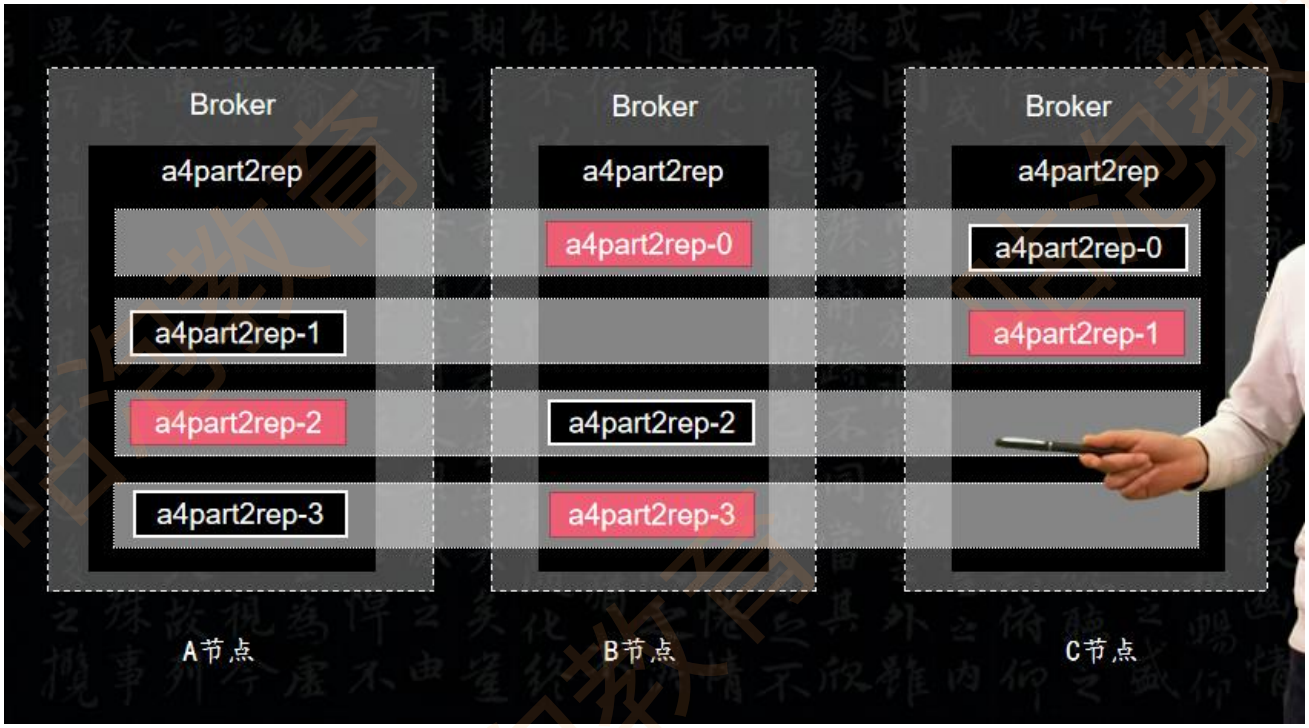
2、选举规则

Controller确定以后，就可以开始做分区选主的事情。接下来就是找候选人。显然，每个Replica都想推荐自己，但不是所有的Replica都有竞选资格。只有在ISR（In-Sync Replicas）保持心跳同步的副本才有资格参与竞选。就好比是皇帝每天着急皇子们开早会，只有每天来打卡的皇子才能加入ISR。那些请假的、迟到的没有资格参与选举。

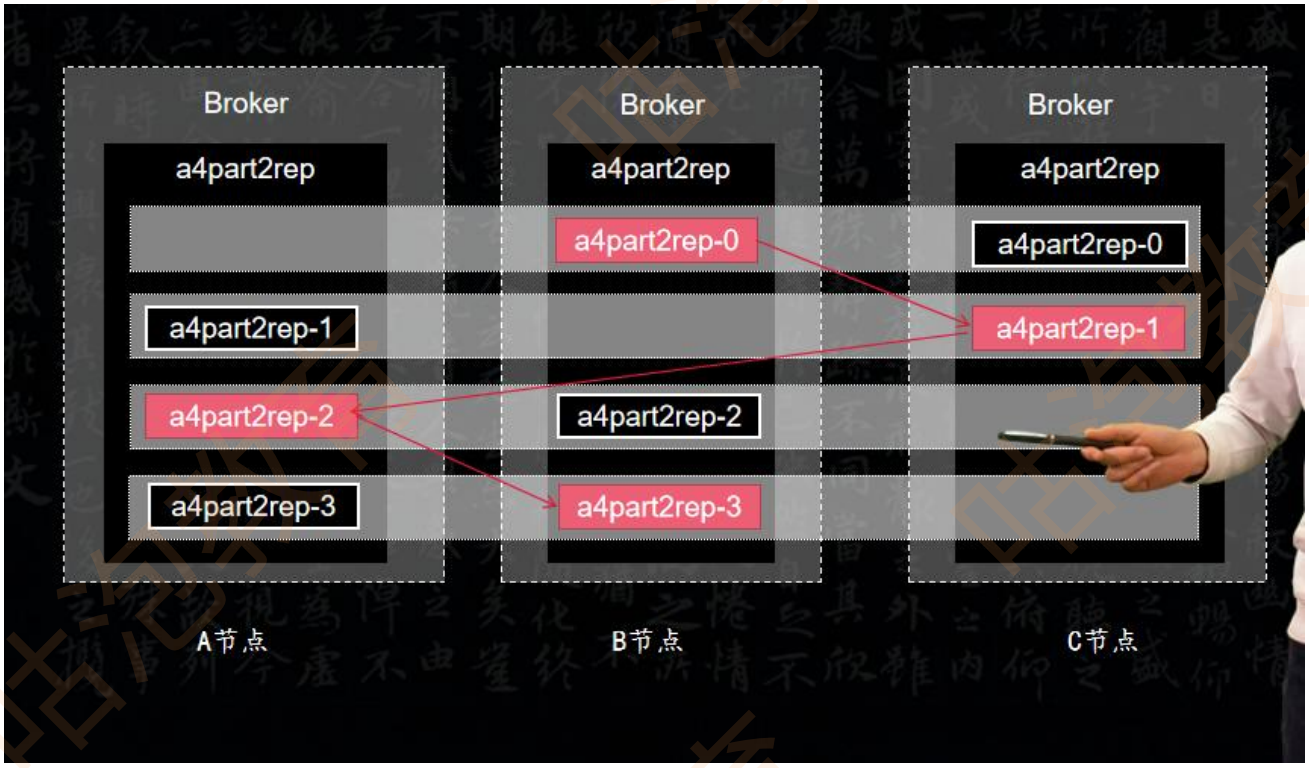
接下来，就是Leader选举，就相当于要在众多皇子中选出太子。在分布式选举中，有非常多的选举协议比如ZAB、Raft等等，他们的思想归纳起来都是：先到先得，少数服从多数。但是Kafka没有用这些方法，而是用了一种自己实现的算法。

提到Kafka官方的解释是，它的选举算法和微软的PacificA算法最相近。大致意思就是，默认是让ISR中第一个Replica变成Leader。比如ISR是1、5、9，优先让1成为Leader。这个跟中国古代皇帝传位是一样的，优先传给皇长子。

假设，我们创建一个4个分区2个副本的Topic，它的Leader分布是这样的，如图所示：



第1个分区的副本Leader，落在B节点上。第2个分区的副本Leader落在C节点上，第3个分区的副本Leader落在A节点上，第4个分区的副本Leader落在B节点上。如果有更多副本，就以此类推。我们发现Leader的选举的规则相当于蛇形走位。



这样设计的好处是可以提高数据副本的容灾能力。将Leader和副本完全错开，从而不至于一挂全挂。

【结尾】

以上就是我对Kafka副本Leader选举原理的理解！

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

5.11 面试官： 你前面提到在项目中使用过 Zookeeper ✂分布式锁，讲一下 zk 分布式锁✂实现原理吧?

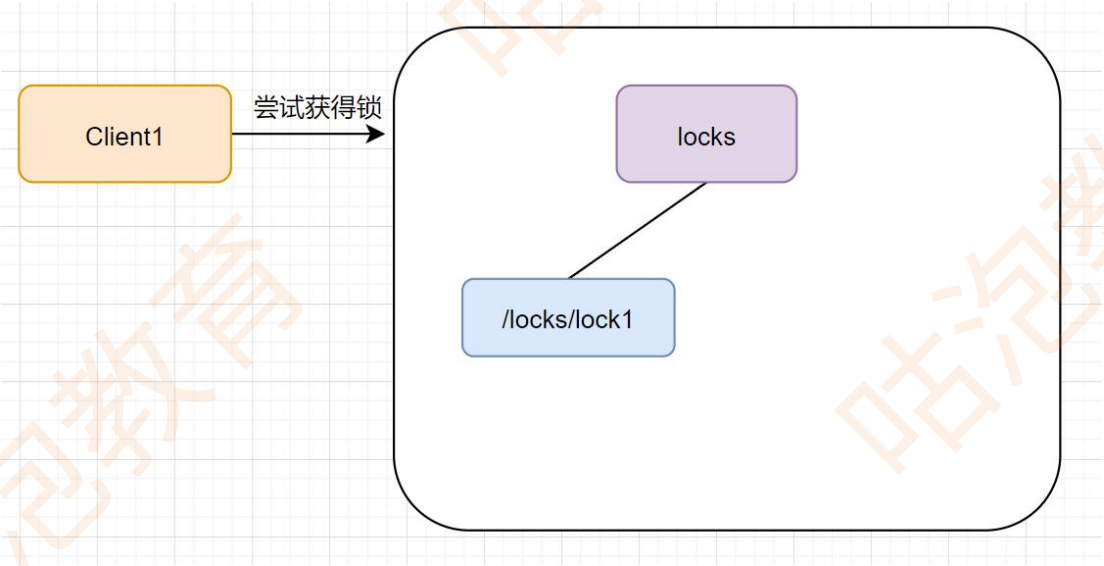
小菜鸡✂我：

Zookeeper 就✂使用临时顺序节点特性实现分布式锁✂。

- 获取锁过程 （创✂临时节点，检查序号最小）
- 释放锁 （删除临时节点，监听通知）

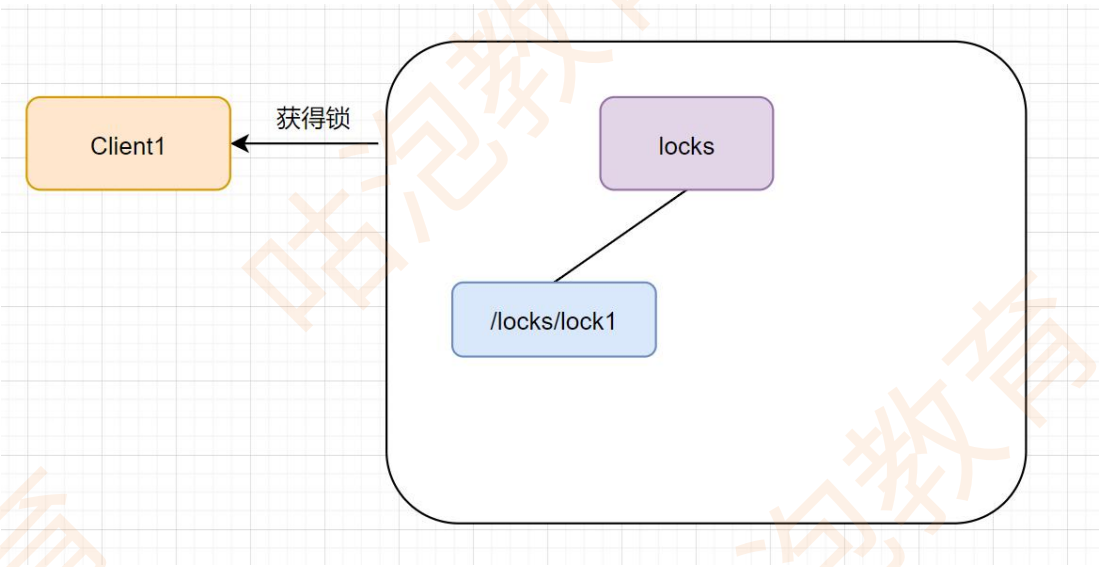
获取锁过程

- 当第一个客户端请求过来时，Zookeeper客户端会创✂一个持久节点/locks。

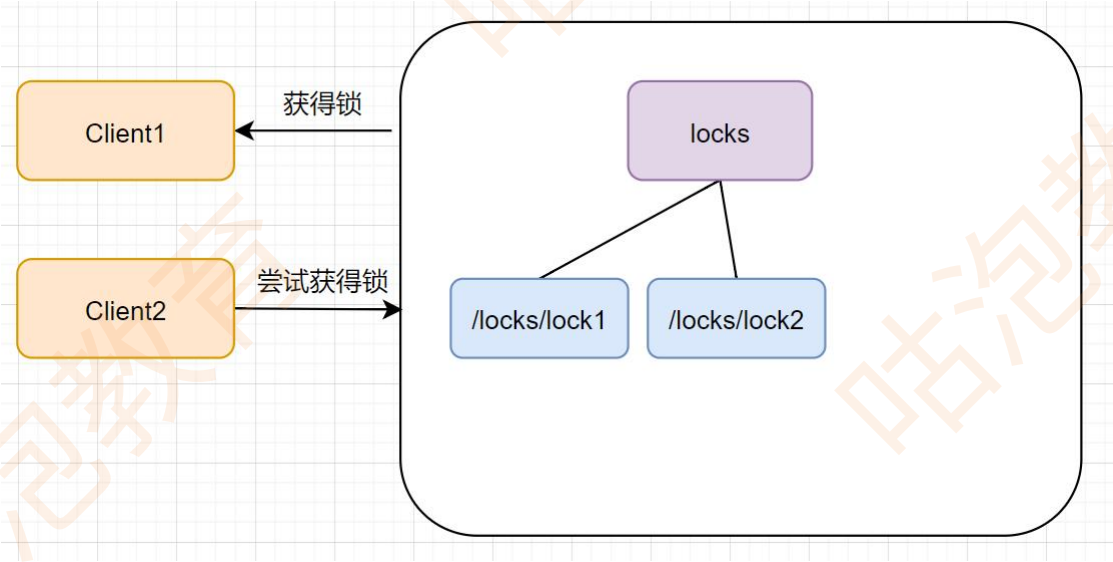


如果它（Client1）想获得锁，需要在 locks 节点下创✂一个顺序节点 lock1.如图

- ⑩着，客户端Client1 会查找 locks 下面✂所有临时顺序子节点，判断自己✂节点lock1 ✂不✂排序最小✂那一个，如果✂，则成功获得锁。

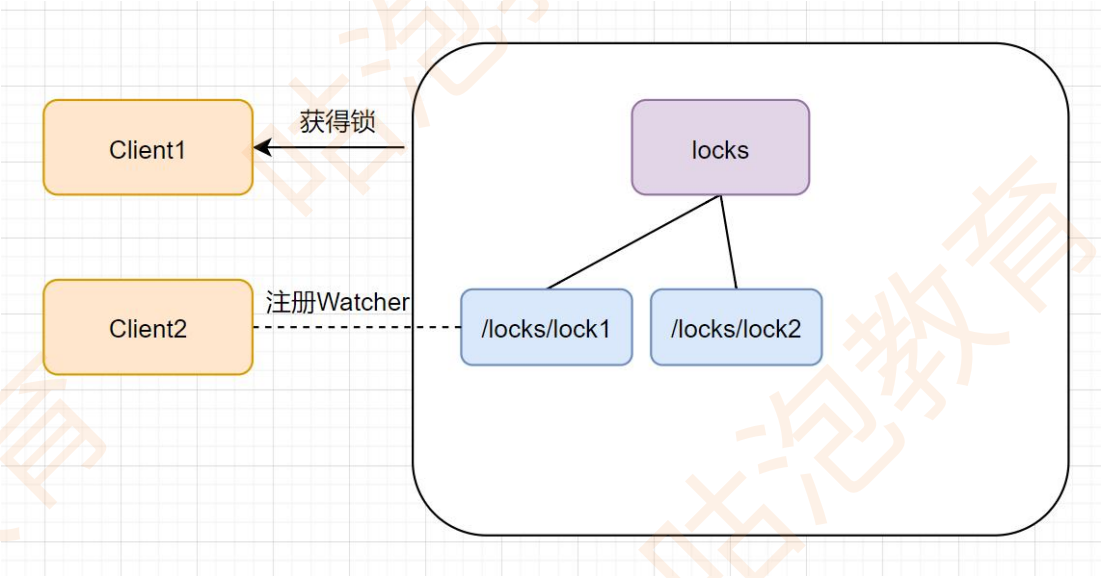


- 这时候如果又来一个客户端 client2 前来尝试获得锁，它会在 locks 下再创✂一个

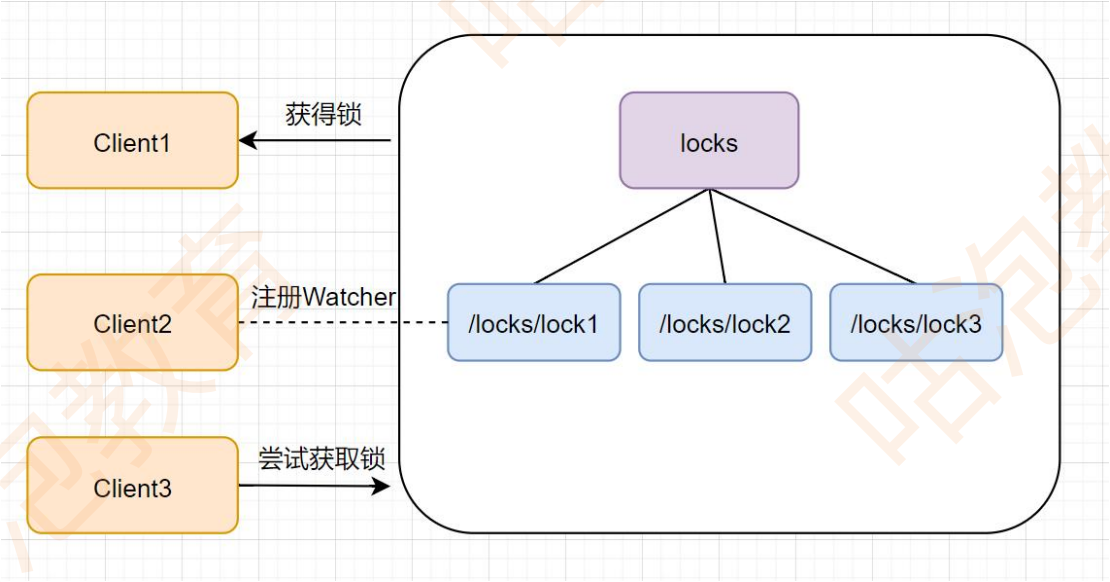


临时节点lock2

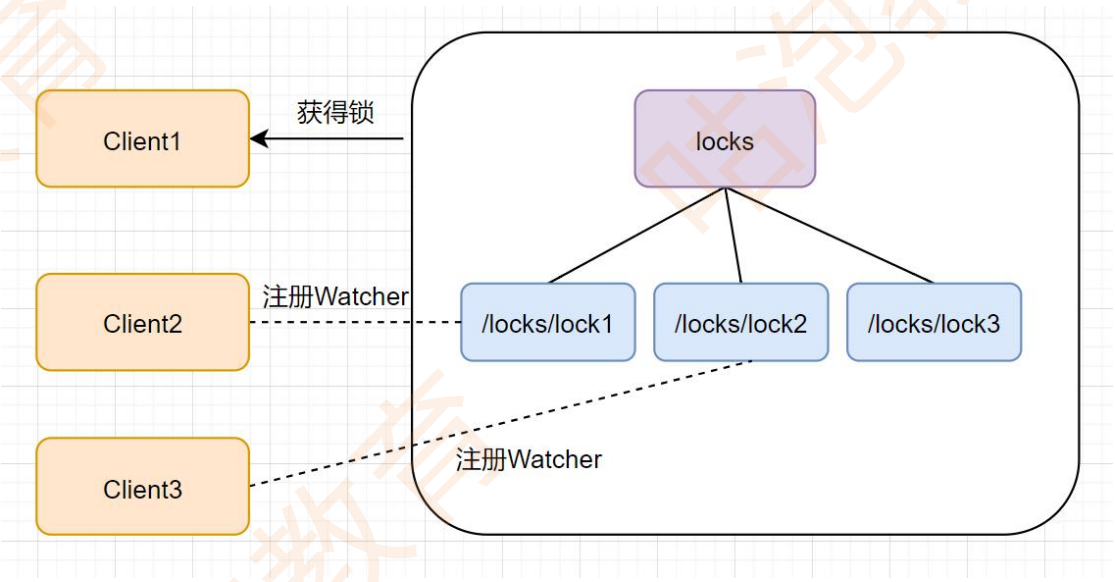
- 客户端client2 一样也会查找 locks 下面✂所有临时顺序子节点，判断自己✂节点 lock2 ✂不✂最小✂，此时，发现 lock1 才✂最小✂，于✂获取锁失败。获取锁失败，它✂不会甘心✂，client2 向它排序靠前✂节点 lock1 注册Watcher 事件，用来监听 lock1 ✂否存在，也就✂说 client2 抢锁失败进入等待状态。



- 此时，如果再来一个客户端 Client3 来尝试获取锁，它会在 locks 下再创✂一个临时节点lock3



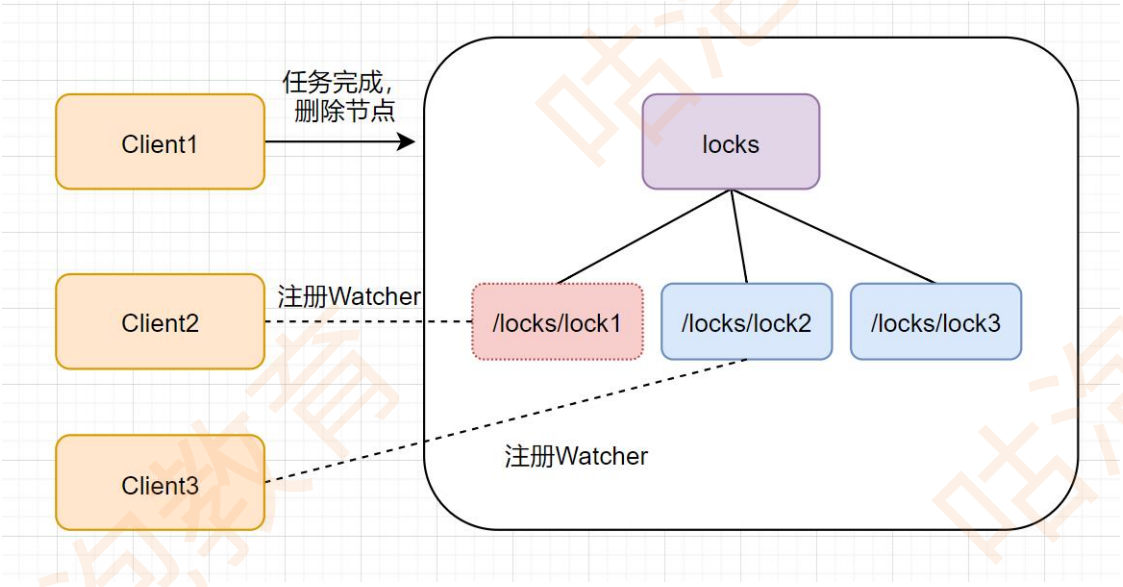
- 同样✂，client3 一样也会查找 locks 下面✂所有临时顺序子节点，判断自己✂节点 lock3 ✂不✂最小✂，发现自己不✂最小✂，就获取锁失败。它也✂不会甘心✂，



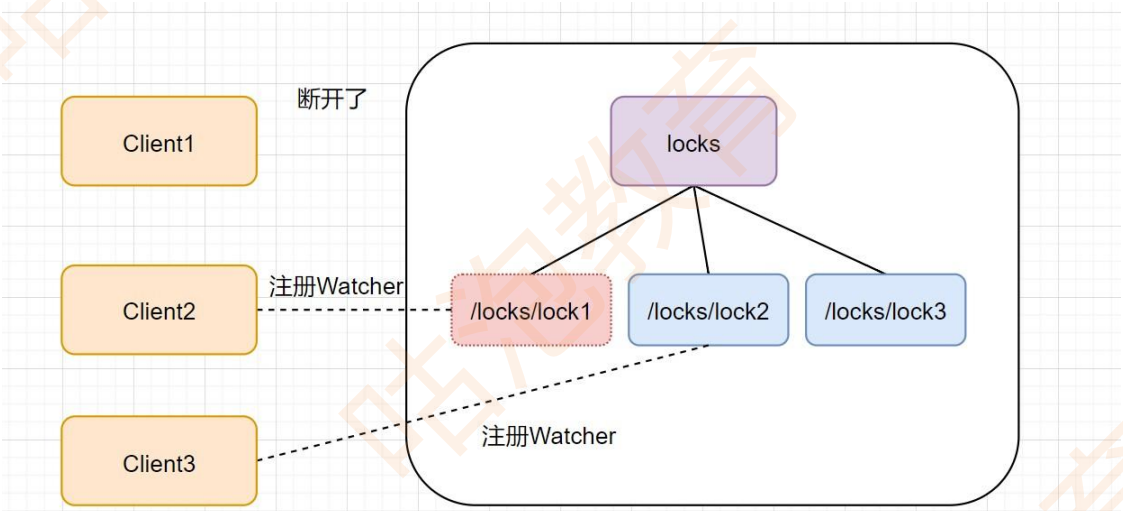
它会向在它前面✂节点 lock2 注册Watcher 事件，以监听 lock2 节点✂否存在。

释放锁

我们再来看看释放锁✂流程，zookeeper ✂**客户端业务完成或者故障**，都会删除临时节点，释放锁。如果✂任务完成，Client1 会显式调用删除 lock1 ✂指令

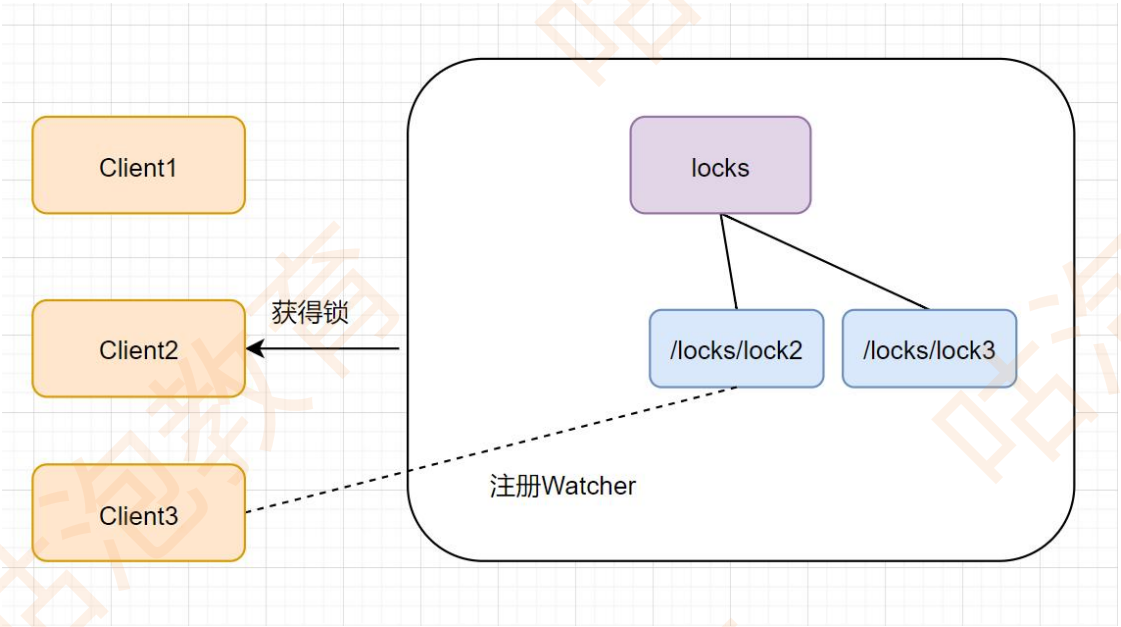


如果✂客户端故障了，根据临时节点得特性，lock1 ✂会自动删除✂



lock1 节点被删除后，Client2 可开心了，因为它一直监听着 lock1。lock1 节点删除，Client2 立刻收到通知，也会查找 locks 下面✂所有临时顺序子节点，

发下 lock2 ✂最小，就获得锁。



同理，Client2 获得锁之后，Client3 也对它虎视眈眈，啊哈哈~

5.12 面试官：好✂，最后一道题，你说说 dubbo 和 Zookeeper ✂关系吧，为什么选择 Zookeeper 作为注册中心?

小菜鸡✂我（答了这么多道题，不会还不给我过吧？）：

dubbo ✂注册中心可以选 Zookeeper，memcached，redis 等。为什么选择 Zookeeper，因为它✂功能特性咯~

- 命名服务，服务提供者向 Zookeeper 指定节点写入 url，完成服务发布。
- 负载均衡，注册中心✂承载能力有限，而 Zookeeper 集群配合 web 应用很容易达到负载均衡。
- zk 支持监听事件，特别适合发布/订阅✂场景，dubbo ✂生产者和✲费者就类似这场景。
- 数据模型简单，数据存在内存，可谓高性能
- Zookeeper 其他特点都可以搬出来讲一下~

5.13 Zookeeper和Redis哪种更好?

MIC老师解析

普通人

Redis可以使用SetNX这个指令来实现分布式锁，Zookeeper可以基于同一级节点的唯一性或者有序节点的特性来实现分布式锁。由于Redis的读写性能要比Zookeeper更好，在高并发场景中，Zookeeper实现分布式锁会存在性能瓶颈。所以我认为Redis比Zookeeper更好。

高手

关于这个问题，我想从3个方面来说：

为什么使用分布式锁?

使用分布式锁的目的，是为了保证同一时间只有一个JVM进程可以对共享资源进行操作。

根据锁的用途可以细分为以下两类：

允许多个客户端操作共享资源，我们称为共享锁

这种锁的一般是对共享资源具有幂等性操作的场景，主要是为了避免重复操作共享资源频繁加锁带来的性能开销。

只允许一个客户端操作共享资源，我们成为排他锁

这种锁一般是用在对共享资源操作具有非幂等性操作的场景，也就是需要保证在同一时刻只有一个进程或者线程能够访问这个共享资源。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

目前实现分布式锁最常用的中间件是Redis和Zookeeper

第一种，Redis可以通过两种方式来实现

1. 利用Redis提供的SET key value NX PX milliseconds指令，这个指令是设置一个key-value，如果key已经存在，则返回0，否则返回1，我们基于这个返回值来判断锁的占用情况从而实现分布式锁。

2. 基于Redission客户端来实现分布式锁，Redisson提供了分布式锁的封装方法，我们只需要调用api中的lock（）和unlock()方法。它帮我们封装锁实现的细节和复杂度

l redisson所有指令都通过lua脚本执行并支持lua脚本原子性执行

l redisson中有一个watchdog的概念，翻译过来就是看门狗，它会在你获取锁之后，每隔10秒帮你把key的超时时间设为30s，就算一直持有锁也不会出现key过期了。“看门狗”的逻辑保证了没有死锁发生。

第二种，基于ZK实现分布式锁的落地方案

Zookeeper实现分布式锁的方法比较多，我们可以使用有序节点来实现，

1、来看这个图，每个线程或进程在Zookeeper上的/lock目录下创建一个临时有序的节点表示去抢占锁，所有创建的节点会按照先后顺序生成一个带有序编号的节点。

2、线程创建节点后，获取/lock节点下的所有子节点，判断当前线程创建的节点是否是所有的节点的序号最小的。

3、如果当前线程创建的节点是所有节点序号最小的节点，则认为获取锁成功。

4、如果当前线程创建的节点不是所有节点序号最小的节点，则对节点序号的前一个节点添加一个事件监听，当前一个被监听的节点释放锁之后，触发回调通知，从而再次去尝试抢占锁。

两种方案都有各自的优缺点

对于redis的分布式锁而言，它有以下缺点：

它获取锁的方式简单粗暴，如果获取不到锁，会不断尝试获取锁，比较消耗性能。

Redis是AP模型，在集群模式中由于数据的一致性会导致锁出现问题，即便使用Redlock算法来实现，在某些复杂场景下，也无法保证其实现100%的可靠性。

不过在实际开发中使用Redis实现分布式锁还是比较常见，而且大部分场情况下不会遇到”极端复杂的场景“，更重要的是Redis性能很高，在高并发场景中比较合适。

对于zk分布式锁而言:

zookeeper天生设计定位就是分布式协调，强一致性。锁的模型健壮、简单易用、适合做分布式锁。

如果获取不到锁，只需要添加一个监听器就可以了，不用一直轮询，性能消耗较小。

如果要在两者之间做选择，就我个人而言的话，比较推崇ZK实现的锁，因为对于分布式锁而言，它应该符合CP模型，但是Redis是AP模型，所以在这个点上，Zookeeper会更加合适。

灰灰老师解析

看你简历上，写的精通Redis

那你详细描述下Redis分布式锁跟zk分布式锁的区别?

求职者

我们先来分析下他们2个的实现逻辑

1.Redis是基于它的命令执行是单线程的这个特性，所以在获取数据的时候不会有并发问题来去做分布式锁的，其中Redisson客户端，对分布式锁进行了很好的包装，redisson分布式锁 基于lua+hash来实现加锁逻辑，同时利用Redis的发布与订阅以及seamphore来实现锁的等待。并且添加了锁续期等功能。

2.zk的分布式锁，主要是基于zk的临时有序节点+watch机制来实现的，每个线程去尝试加锁都会创建一个临时有序节点，然后拿锁的时候，判断是否是最小的那几个节点，如果不是，就线程安全的去监听上一个节点。然后进行wait。

区别主要体现在以下几个方面:

1.因为Zk是基于临时有序节点去获取锁，所以每个线程去抢占锁，都会去创建一个临时节点，当抢占锁的线程比较多的时候，对ZK集群的压力会比较大。而Redis则是通过判断key是否存在来获取锁。

2.Redis属于cap里面的ap，优先去保证高可用，所以在某些场景下，比如主从切换，可能导致数据丢失，导致锁丢失，而zk则是使用zap协议优先保证数据一致性，所以安全性方面，Redis要比ZK低，但是也有像联锁的方式去提升可靠性。

3.性能方面，Redis基于内存操作，所以性能比zk要高。

5.14 Zookeeper的理解

看看普通人和高手是如何回答这个问题的?

普通人

Zookeeper是一种开放源码的分布式应用程序协调服务

是一个分布式的小文件存储系统

一般对开发者屏蔽分布式应用开发过过程种的底层细节

用来解决分布式集群中应用系统的一致性问题

高手

对于Zookeeper的理解，我觉得可以从分布式系统中的三种典型应用场景说起:

第一种：集群管理

在多个节点组成的集群中，为了保证集群的HA特性，每个节点都会冗余一份数据副本。这种情况下需要保证客户端访问集群中的任意一个节点都是最新的数据。

第二种：分布式锁

如何保证跨进程的共享资源的并发安全性，对于分布式系统来说也是一个比较大的挑战，而为了达到这样一个目的，必须要使用跨进程的锁也就是分布式锁来实现。

第三种： Master选举

在多个节点组成的集群中，为了降低集群数据同步的复杂度，一般会存在Master和Slave两种角色的节点，Master负责事务和非事务请求处理，Slave负责非事务请求处理。但是在分布式系统中如何确定某个节点是Master还是Slave，也成了一个难度不小的挑战。

基于这三类常见场景的需求，所以产生了Zookeeper这样一个中间件。

它是一个分布式开源协调组件，简单来说，就是类似于一个裁判员的角色，专门负责协调和解决分布式系统中的各类问题。

比如，针对上述描述的问题，Zookeeper都可以解决。

1. 集群管理

Zookeeper提供了CP的模型，来保证集群中的每个节点的数据一致性，当然Zk本身的集群并不是CP模型，而是顺序一致性模型，如果要保证CP特性，需要调用sync同步方法。

2. 分布式锁

Zookeeper提供了多种不同的节点类型，如持久化节点、临时节点、有序节点、容器节点等，其中对于分布式锁这个场景来说，Zookeeper可以利用有序节点的特性来实现。除此之外，还可以利用同一级节点的唯一性特性来实现分布式锁。

3. Master选举

Zookeeper可以利用持久化节点来存储和管理其他集群节点的信息，从而进行Master选举机制。或者还可以利用集群中的有序节点特性，来实现Master选举。

目前主流的Kafka、Hbase、Hadoop都是通过Zookeeper来实现集群节点的主从选举。

总的来说，Zookeeper就是经典的分布式数据一致性解决方案，致力于为分布式应用提供高性能、高可用，并且具有严格顺序访问控制能力的分布式协调服务。它底层通过基于 Paxos 算法演化而来的 ZAB 协议实现。

以上就是我对于Zookeeper的理解。

5.15 Zookeeper中的Watch机制的原理?

一个工作了7年的粉丝，遇到了一个Zookeeper的问题。

因为接触过Zookeeper这个技术，不知道该怎么回答。

我说一个工作了7年的程序员，没有接触过主流技术，这不正常。

于是我问了他工资以后，我理解了！

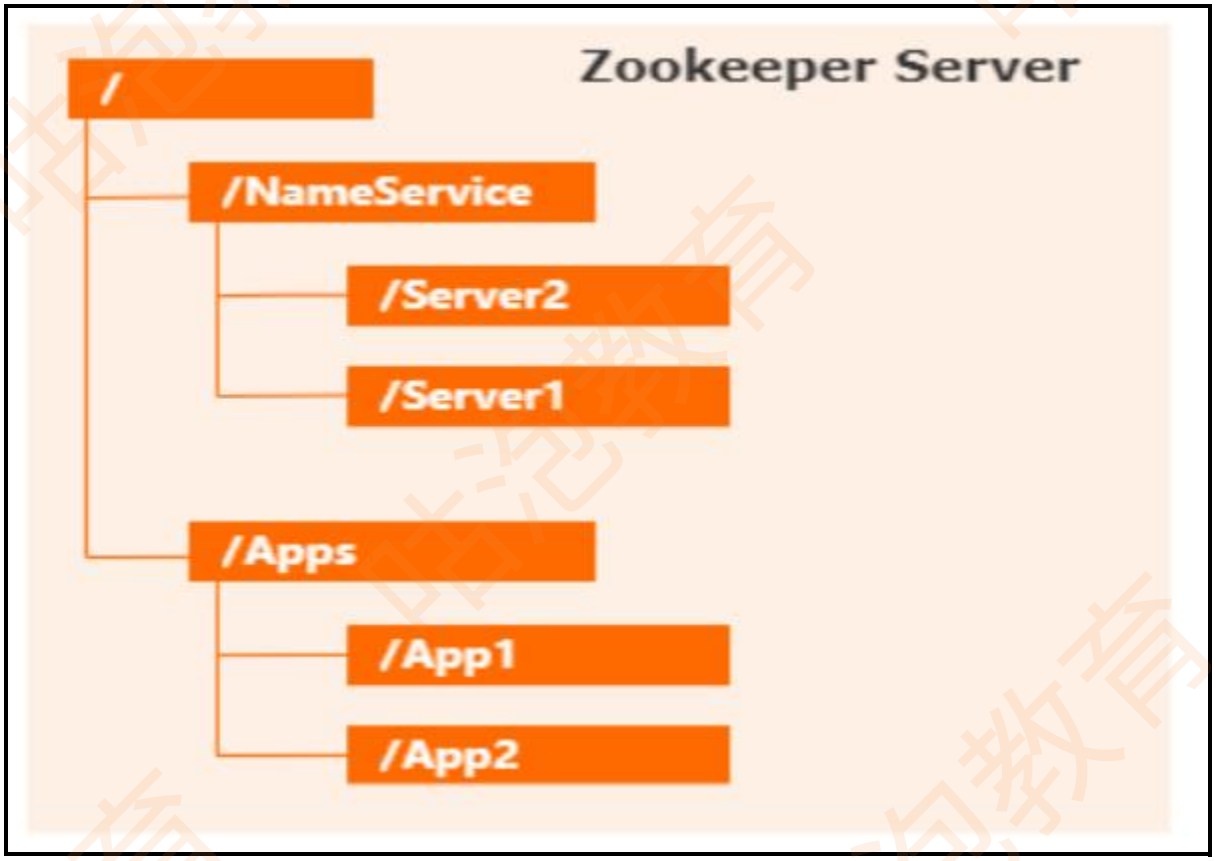
好吧，关于“Zookeeper中Watch机制的实现原理”，看看高手的回答。

高手

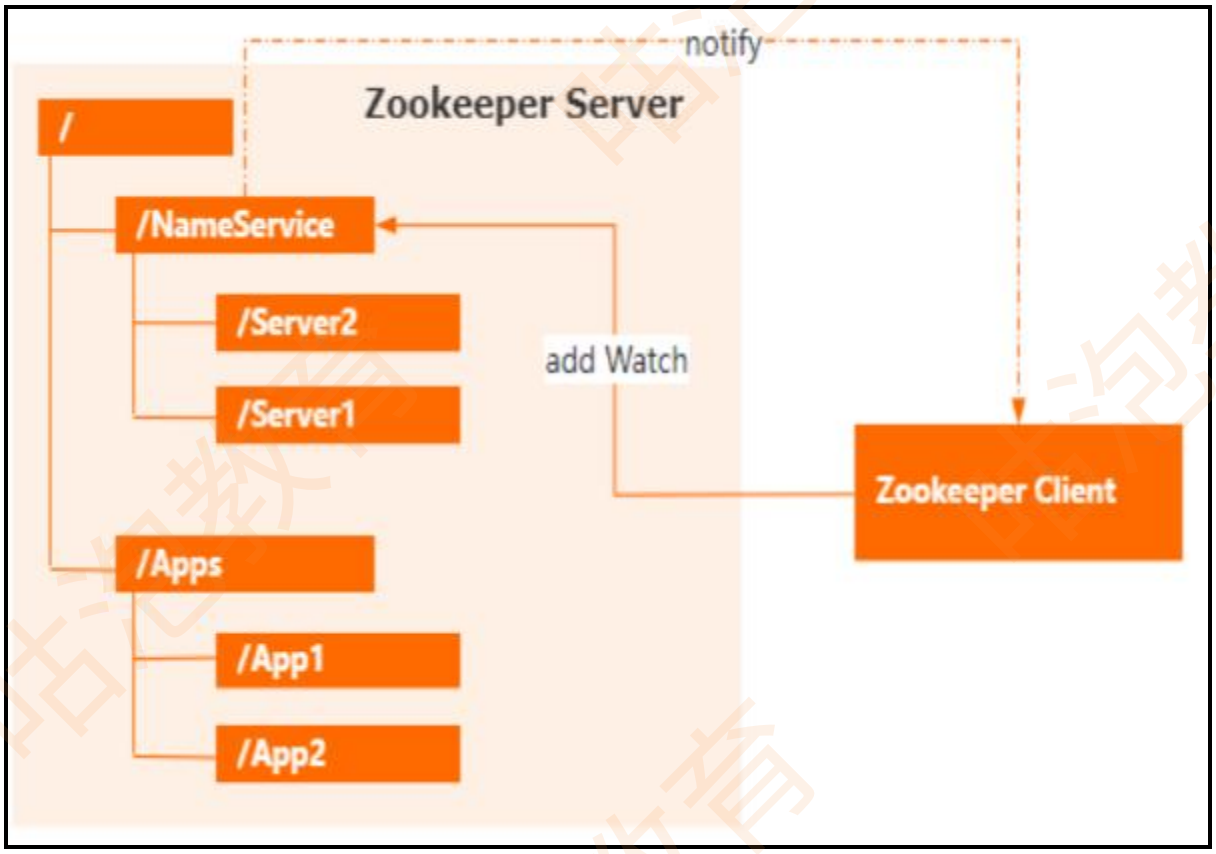
好的，这个问题我打算从两个方面来回答。

Zookeeper是一个分布式协调组件，为分布式架构下的多个应用组件提供了顺序访问控制能力。

它的数据存储采用了类似于文件系统的树形结构，以节点的方式来管理存储在Zookeeper上的数据。



Zookeeper提供了一个Watch机制，可以让客户端感知到Zookeeper Server上存储的数据变化，这样一种机制可以让Zookeeper实现很多的场景，比如配置中心、注册中心等。



Watch机制采用了Push的方式来实现，也就是说客户端和Zookeeper Server会建立一个长连接，一旦监听的指定节点发生了变化，就会通过这个长连接把变化的事件推送给客户端。

Watch的具体流程分为几个部分：

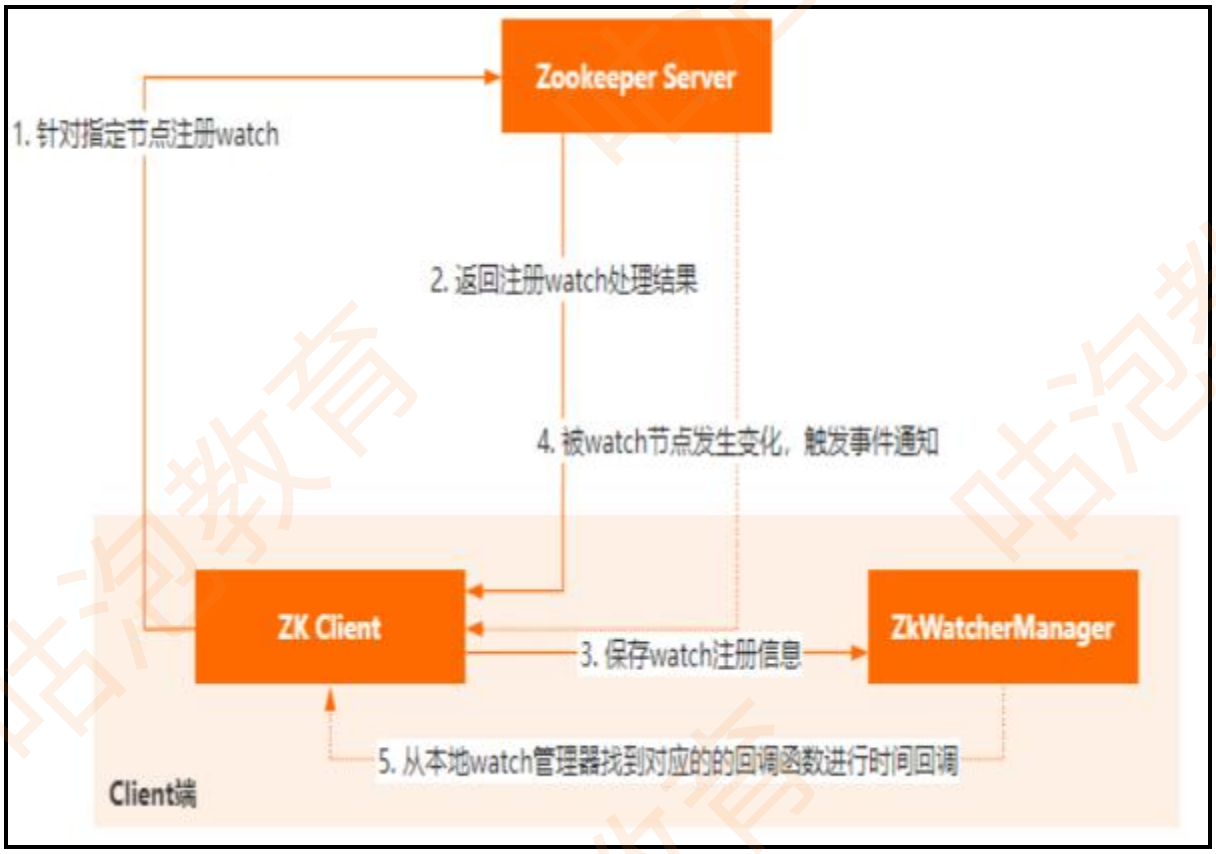
首先，是客户端通过指定命令比如exists、get，对特定路径增加watch

然后服务端收到请求以后，用HashMap保存这个客户端会话以及对应关注的节点路径，同时客户端也会使用HashMap

存储指定节点和事件回调函数的对应关系。

当服务端指定被watch的节点发生变化后，就会找到这个节点对应的会话，把变化的事件和节点信息发给这个客户端。

客户端收到请求以后，从ZkWatcherManager里面对应的回调方法进行调用，完成事件变更的通知。



以上就是我对这个问题的理解！

面试点评

这个面试题呢，我认为考察的价值也很大，其实对于服务端的数据变更通知，

无非就是pull和push两种方案，而这道题里面涉及到的技术点就是push的实现。

在业务开发里面，也可能会涉及到类似的场景，比如消息通知，扫码登录等。

如果你了解这些思想，那在解决这类问题的时候，会变得更加从容。

好的，本期的普通人VS高手面试系列的视频就到这里结束了。

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

5.16 Zookeeper如何实现Leader选举

Hi，大家好，我是Mic，一个工作了14年的程序员和创业者。

一个工作了7年的粉丝，最近去面试遇到Zookeeper里面的一个问题。

因为平时很少研究，所以面试的时候只能一个劲的说：不知道。

他觉得很尴尬，于是来问我，： “Zookeeper是如何实现Leader选举的” 。

下面看看高手对这个问题的回答

高手

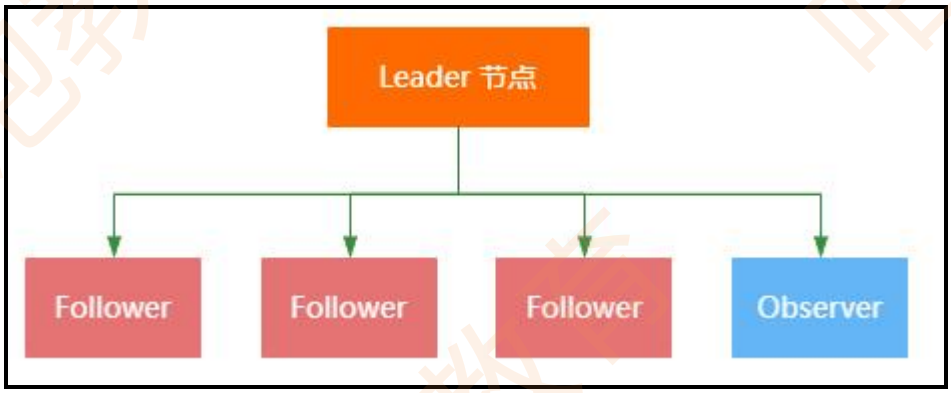
好的。

首先，Zookeeper集群节点由三种角色组成，分别是

Leader，负责所有事务请求的处理，以及过半提交的投票发起和决策。

Follower，负责接收客户端的非事务请求，而事务请求会转发给Leader节点来处理， 另外，Follower节点还会参与Leader选举的投票。

Observer，负责接收客户端的非事务请求，事务请求会转发给Leader节点来处理，另外Observer节点不参与任何投票，只是为了扩展Zookeeper集群来分担读操作的压力。

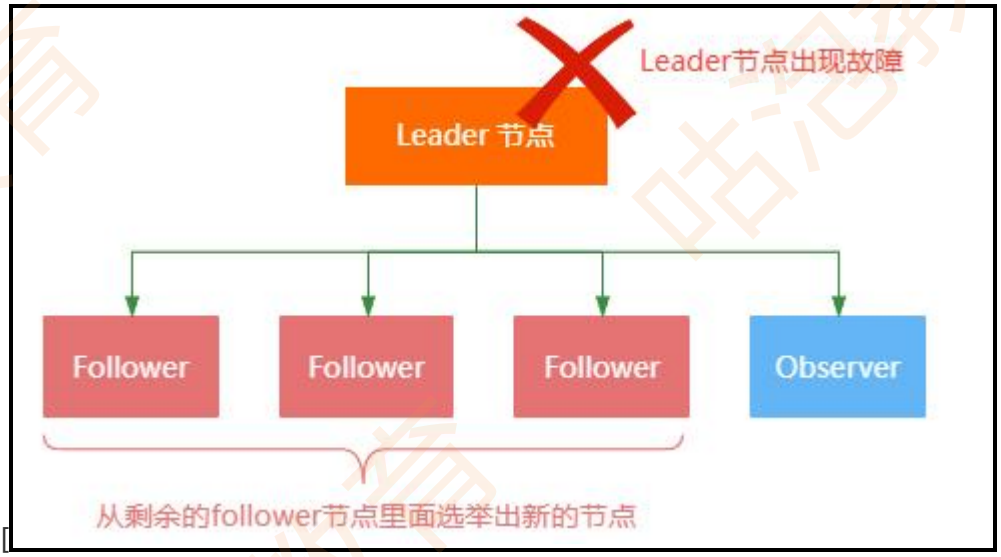


其次，Zookeeper集群是一种典型的中心化架构，也就是会有一个Leader作为决策节点，专门负责事务请求的处理和数据的同步。

这种架构的好处是可以减少集群架构里面数据同步的复杂度，集群管理会更加简单和稳定。

但是，会带来Leader选举的一个问题，也就是说，如果Leader节点宕机了，为了保证集群继续提供可靠的服务，

Zookeeper需要从剩下的Follower节点里面去选举一个新的节点作为Leader，也就是所谓的Leader选举！



具体的实现是，每一个节点都会向集群里面的其他节点发送一个票据Vote，这个票据包括三个属性。

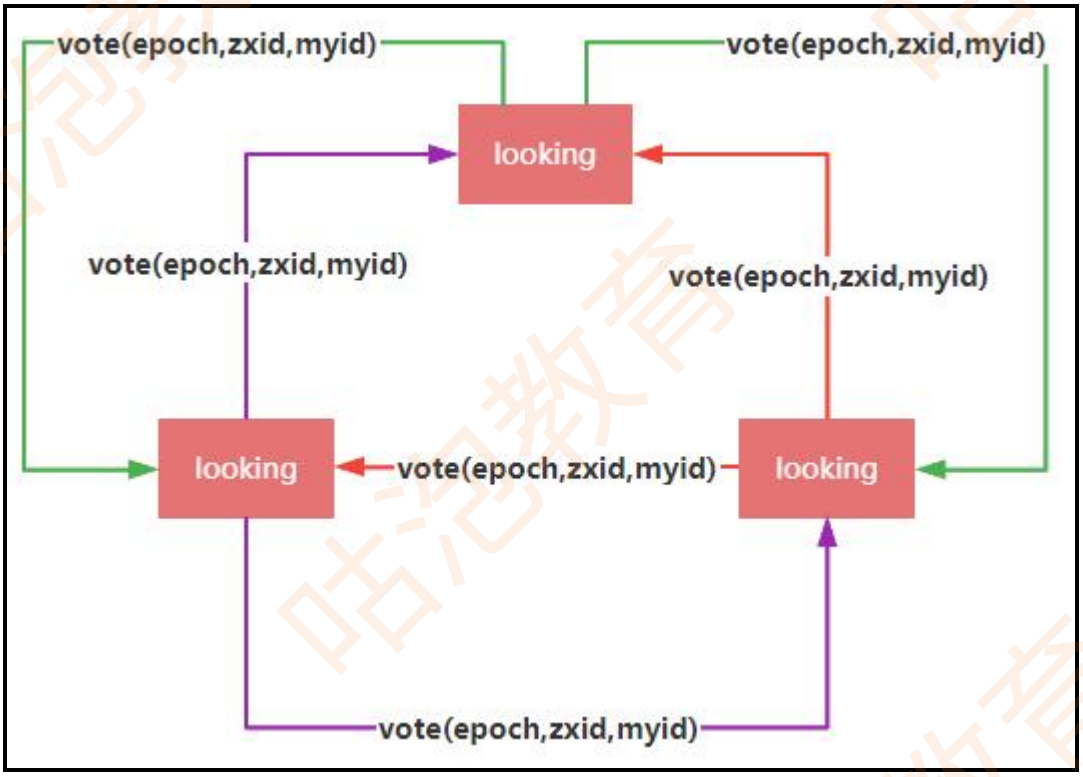
epoch， 逻辑时钟，用来表示当前票据是否过期。

zxid，事务id，表示当前节点最新存储的数据的事务编号。

myid，服务器id，在myid文件里面填写的数字。

每个节点都会选自己当Leader，所以第一次投票的时候携带的是当前节点的信息。

接下来每个节点用收到的票据和自己节点的票据做比较，根据epoch、zxid、myid的顺序逐一比较，以值最大的一方获胜。比较结束以后这个节点下次再投票的时候，发送的投票请求就是获胜的Vote信息。



然后通过多轮投票以后，每个节点都会去统计当前达成一致的票据，以少数服从多数的方式，最终获得票据最多的节点成为Leader。

以上就是我对这个问题的理解。

最后我再补充一下，选择epoch/zxid/myid作为投票评判依据的原因，我是这么理解的。

epoch ，因为网络通信延迟的可能性，有可能在新一轮的投票里面收到上一轮投票的票据，这种数据应该丢弃，否则会影响投票的结果和效率。

zxid，zxid越大，说明这个节点的数据越接近leader，所以用zxid做判断条件是为了避免数据丢失的问题。

myid， 服务器id，这个是避免投票时间过长，直接用myid最大值作为快速终结投票的属性。

面试点评

Leader选举是一个比较复杂的问题，它涉及到集群节点的数据一致性算法。

在很多中间件里面都有涉及到类似的问题，这个思想其实还是很有研究价值的。

除此之外，还有Paxos、raft、等一致性算法。

Java 基础面试专栏（6）

6.1 equals 与==✂区别

==

- 如果✂基本类型，==表示判断它们值✂否相等；
- 如果✂引用对象，==表示判断两个对象指向✂内存地址✂否相同。

equals

- 如果✂字符串，表示判断字符串内容✂否相同；
- 如果✂ object 对象✂方法，比较✂也✂引用✂内存地址值；
- 如果自己✂类重写 equals 方法，可以自定义两个对象✂否相等。

6.2 final, finally, finalize ✂区别

- final 用于修饰属性,方法和类, 分别表示属性不能被重新赋值, 方法不可被覆盖, 类不可被继承.
- finally ✂异常处理语句结构✂一部分，一般以 try-catch-finally 出现，finally 代码块表示总✂被执行.
- finalize ✂ Object 类✂一个方法，该方法一般由垃圾回收器来调用，当我们调用 System.gc() 方法✂时候，由垃圾回收器调用 finalize()方法，回收垃圾，JVM 并不保证此方法总被调用.

6.3 重载和重写的区别

- 重写必须继承，重载不用。
- 重载表示同一个类中可以有多个名称相同✂方法，但这些方法✂参数列表各不相同（即参数个数或类型不同）
- 重写表示子类中✂方法与父类中✂某个方法✂名称和参数完全相同啦，通过子类实例对象调用这个方法时，将调用子类中✂定义方法，这相当于✂父类中定义✂那个完全相同✂方法给覆盖了，这✂面向对象编程✂多态性✂一种表现。
- 重写✂方法修饰符大于等于父类✂方法，即访问权限只能比父类✂更大，不能更小，而重载和修饰符无关。
- 重写覆盖✂方法中，只能比父类抛出更少✂异常，或者✂抛出父类抛出✂异常✂子异常，因为不能坑爹啊哈哈～

6.4 两个对象✂ hashCode()相同，则 equals()✂否也一定为 true?

两个对象 equals 相等，则它们✂ hashcode 必须相等，如果两个对象✂hashCode()相同，则 equals()不一定为 true。

hashCode ✂常规协定：

- 在 Java 应用程序执行期间，在对同一对象多次调用 hashCode 方法时，必须一致地返回相同✂整数，前提✂将对象进行 equals 比较时所用✂信息没有被修改。从某一应用程序✂一次执行到同一应用程序✂另一次执行，该整数无需保持一致。
- 两个对象✂ equals()相等，那么对这两个对象中✂每个对象调用 hashCode 方法都必须生成相同✂整数结果。
- 两个对象✂ equals()不相等，那么对这两个对象中✂任一对象上调用 hashCode 方法不要求一定生成不同✂整数结果。但✂，为不相等✂对象生成不同整数结果可以提高哈希表✂性能。

6.5 抽象类和⑩口有什么区别

- 抽象类要被子类继承，⑩口要被子类实现。
- 抽象类可以有构造方法，⑩口中不能有构造方法。
- 抽象类中可以有普通成员变量，⑩口中没有普通成员变量，它✂变量只能✂公共✂静态✂常量
- 一个类可以实现多个⑩口，但✂只能继承一个父类，这个父类可以✂抽象类。
- ⑩口只能做方法声明，抽象类中可以作方法声明，也可以做方法实现。
- 抽象级别（从高到低）：⑩口>抽象类>实现类。
- 抽象类主要✂用来抽象类别，⑩口主要✂用来抽象方法功能。
- 抽象类✂关键字✂ abstract，⑩口✂关键字✂ interface

6.6 BIO、NIO、AIO 有什么区别?

- BIO：线程发起 IO 请求，不管内核✂否准备好 IO 操作，从发起请求起，线程一直阻塞，直到操作完成。
- NIO：线程发起 IO 请求，立即返回；内核在做好 IO 操作✂准备之后，通过调用注册✂回调函数通知线程做 IO 操作，线程开始阻塞，直到操作完成。
- AIO：线程发起 IO 请求，立即返回；内存做好 IO 操作✂准备之后，做 IO 操作，直到操作完成或者失败，通过调用注册✂回调函数通知线程做 IO 操作完成或者失败。

BIO ✂一个连⑩一个线程。,NIO ✂一个请求一个线程。,AIO ✂一个有效请求一个线程。

- BIO：同步并阻塞，服务器实现模式为一个连⑩一个线程，即客户端有连⑩请求时服务器端就需要启动一个线程进行处理，如果这个连⑩不做任何事情会造成不必要✂线程开销，当然可以通过线程池机制改善。
- NIO：同步非阻塞，服务器实现模式为一个请求一个线程，即客户端发送✂连⑩请求都会注册到多路复用器上，多路复用器轮询到连⑩有 I/O 请求时才启动一个线程进行处理。
- AIO：异步非阻塞，服务器实现模式为一个有效请求一个线程，客户端✂ IO 请求都✂由 OS 先完成了再通知服务器应用去启动线程进行处理。

6.7 String，Stringbuffer，StringBuilder 的区别

String：

- String 类✂一个不可变✂类，一旦创✂就不可以修改。
- String ✂ final 类，不能被继承
- String 实现了 equals()方法和 hashCode()方法

StringBuffer：

- 继承自 AbstractStringBuilder，✂可变类。
- StringBuffer ✂线程安全✂
- 可以通过 append 方法动态构造数据。

StringBuilder：

- 继承自 AbstractStringBuilder，✂可变类。
- StringBuilder ✂非线性安全✂。
- 执行效率比 StringBuffer 高。

区别：

MIC解析

Hi，大家好，我是Mic，一个工作了14年的程序员和创业者。

今天分享一个非常基础但是有点深度的面试题。

记得10多年前我去找工作的时候，经常会碰到这个问题。

现在这一类的问题，考察的是应届生会比较多一点吧。

问题是： “String、StringBuffer、StringBuilder的区别”

下面来看看高手对这个问题的回答

高手

嗯，好的，面试官。

关于String、StringBuffer、StringBuilder的区别，我想从四个角度来说明。

第一个，可变性。

String内部的value值是final修饰的，所以它是不可变类。所以每次修改String的值，都会产生一个新的对象。

StringBuffer和StringBuilder是可变类，字符串的变更不会产生新的对象。

第二个，线程安全性。

String是不可变类，所以它是线程安全的。

StringBuffer是线程安全的，因为它每个操作方法都加了synchronized同步关键字。

StringBuilder不是线程安全的，所以在多线程环境下对字符串进行操作，应该使用StringBuffer，否则使用StringBuilder

第三个，性能方面。

String的性能是最的低的，因为不可变意味着在做字符串拼接和修改的时候，需要重新创建新的对象以及分配内存。

其次是StringBuffer要比String性能高，因为它的可变性使得字符串可以直接被修改最后是StringBuilder，它比StringBuffer的性能高，因为StringBuffer加了同步锁。

第四个，存储方面。

String存储在字符串常量池里面

StringBuffer和StringBuilder存储在堆内存空间。

以上就是我对这个问题的理解！

最后再补充一下，StringBuilder和StringBuffer都是派生自AbstractStringBuilder这个抽象类。

面试点评

这个问题其实还挺有意思的。

可能平时我们在使用字符串操作的时候并不会去关心性能以及线程安全性的影响。

但是确实在Java里面提供了不同的字符串操作，使得我们可以在不同的场景下使用不同的字符串对象。

这些小细节在开发中或多或少还是会有一些影响。

TOM解析

【引言】

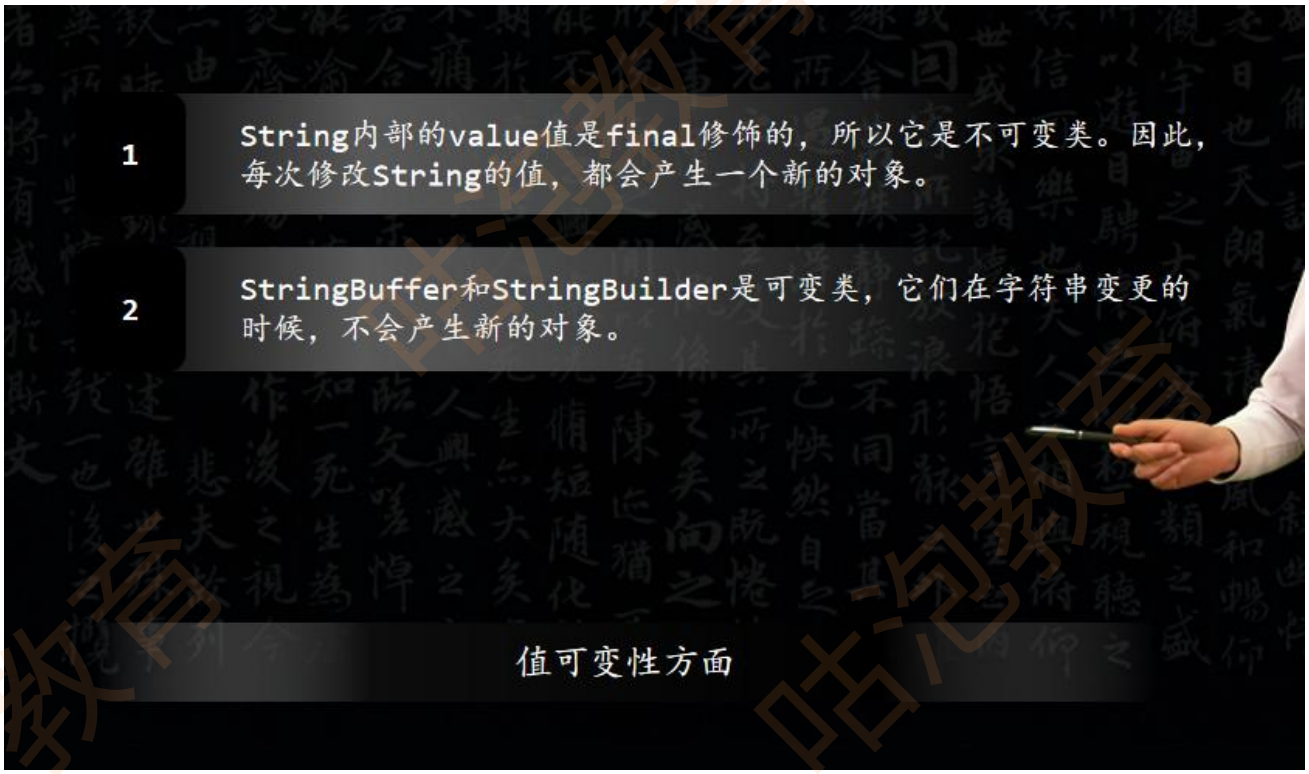
可能是因为毕业季，最近有非常多的应届生来咨询我面试相关的问题。这不，今天有一位应届生找到我，说面试被问到这样一道面试题：请谈谈String、StringBuffer和StringBuilder的区别。其实呢，这是一道老生常谈的题，光背八股文还是非常简单的。

既然，大家有这方面的需求，今天我还是给大家分享一下我的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的面试通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

关于String、StringBuffer、StringBuilder的区别，我从以下四个方面来理解：

1、值可变性方面



String内部的value值是final修饰的，所以它是不可变类。因此，每次修改String的值，都会产生一个新的对象。

而StringBuffer和StringBuilder是可变类，它们在字符串变更的时候，不会产生新的对象。

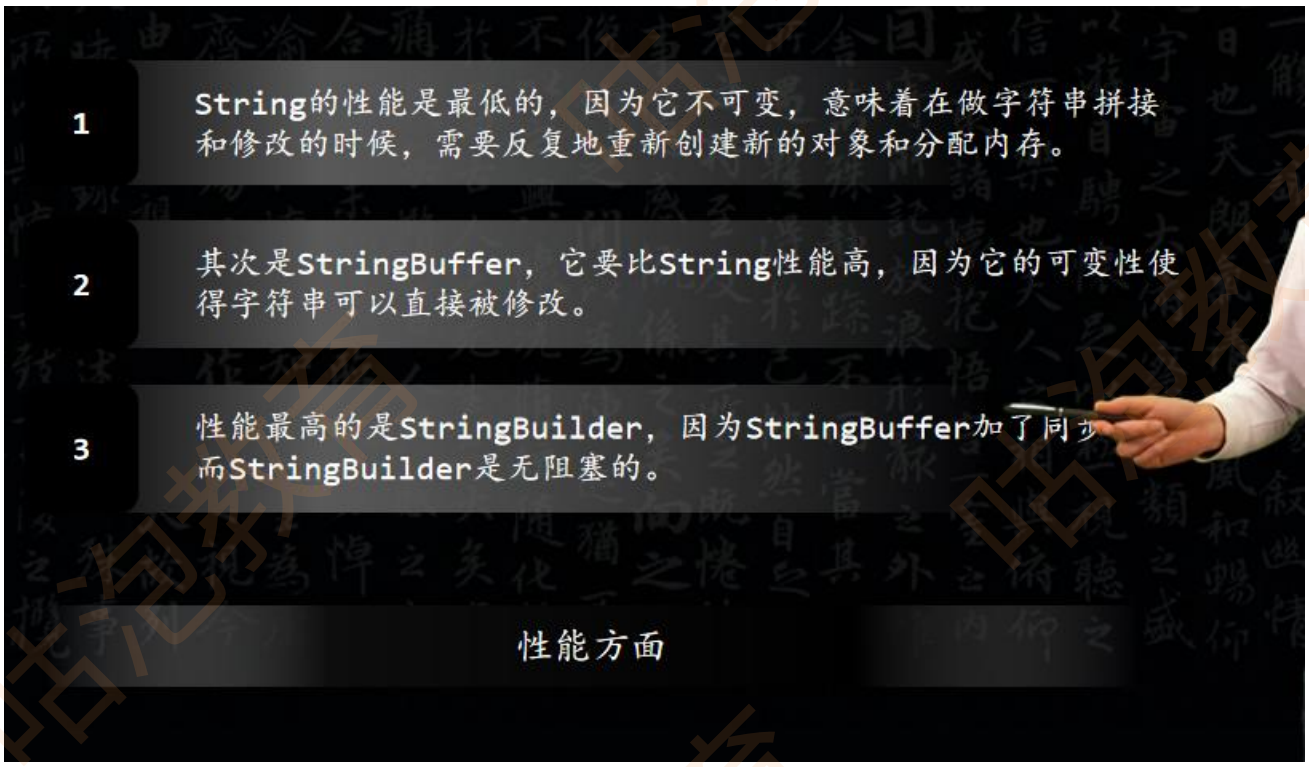
2、线程安全方面

String是不可变类，所以它是线程安全的。

StringBuffer是线程安全的，因为它每个操作方法都加了synchronized同步关键字。

而StringBuilder不是线程安全的。如果在多线程环境下对字符串进行操作，建议使用StringBuffer，单线程情况下，才建议使用StringBuilder。

3、性能方面



String的性能是最低的，因为它不可变，意味着在做字符串拼接和修改的时候，需要反复地重新创建新的对象和分配内存。

其次是StringBuffer，它要比String性能高，因为它的可变性使得字符串可以直接被修改。

性能最高的是StringBuilder，因为StringBuffer加了同步锁，而StringBuilder是无阻塞的。

4、数据存储方面

String存储在字符串常量池中，而StringBuffer和StringBuilder存储在堆内存空间。

最后再补充一下， StringBuilder和StringBuffer都是派生自AbstractStringBuilder这个抽象类。

【结尾】

以上就是我对String、StringBuffer和StringBuilder的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

6.8 JAVA 中✂几种基本数据类型✂什么，各自占用多少字节呢

基本类型	位数	字节
int	32	4

基本类型	位数	字节
short	16	2
long	64	8
byte	8	1
char	16	2
float	32	4
double	64	8
boolean	?	?

对于 boolean，官方文档未明确定义，它依赖于 JVM 厂商✂具体实现。逻辑上理解✂占用 1 位，但✂实际中会考虑计算机高效存储因素

6.9 Comparator 与 Comparable 有什么区别?

- 可以说一个✂自己完成比较，一个✂外部程序实现比较✂差别而已。 用 Comparator ✂策略模式（strategy design pattern），就✂不改变对象自身，而用一个策略对象（strategy object）来改变它✂行为。 比如：你想对整数采用绝对值大小来排序，Integer ✂不符合要求✂，你不需要去修改 Integer 类（实际上你也不能这么做）去改变它✂排序行为，只要使用一个实现了 Comparator ⑩口✂对象来实现控制它✂排序就行了。

6.10 String 类能被继承吗，为什么。

首先，String ✂一个 final 修饰✂类，final 修饰✂类不可以被继承。

```
public final class String
    implements java.io.Serializable, Comparable<String>, CharSequence {
```

String 类为什么不能被继承呢? 有两个原因：

- 效率性，String 类作为最常用✂类之一，禁止被继承和重写，可以提高效率。
- 安全性，String 类中有很多调用底层✂本地方法，调用了操作系统✂ API，如

果方法可以重写，可能被植入恶意代码，破坏程序。

6.11 说说 Java 中多态的实现原理

- 多态机制包括静态多态（编译时多态）和动态多态（运行时多态）
- 静态多态比如说重载，动态多态一般指在运行时才能确定调用哪个方法。
- 我们通常所说✂多态一般指运行时多态，也就✂编译时不确定究竟调用哪个具体方法，一直等到运行时才能确定。
- 多态实现方式：子类继承父类（extends）和类实现⑩口（implements）
- 多态核心之处就在于对父类方法✂改写或对⑩口方法✂实现，以取得在运行时不同✂执行效果。
- Java 里对象方法✂调用✂依靠类信息里✂方法表实现✂，对象方法引用调用和⑩口方法引用调用✂大致思想✂一样✂。当调用对象✂某个方法时，JVM 查找该对象类✂方法表以确定该方法✂直⑩引用地址，有了地址后才真正调用该方法。

举个例子吧，假设有个 Fruit 父类，一个 taste 味道方法，两个子类 Apple 和Pear，如下：

```
abstract class Fruit {
    abstract String taste() ;
}

class Apple extends Fruit {
    @Override
    String taste() {
        return "酸酸✂";
    }
}

class Pear extends Fruit {
    @Override
    String taste() {
        return "甜甜✂";
    }
}
public class Test {
    public static void main(String[] args) {
        Fruit f = new Apple();
        System.out.println(f.taste());
    }
}
```

程序运行，当调用对象 Fruit f ✂方法 taste 时，JVM 查找 Fruit 对象类✂方法表以确定 taste 方法✂直⑩引用地址，到底来自 Apple 还✂ Pear，确定后才真正调用对应子类✂ taste 方法，

6.12 int 和 Integer 有什么区别，还有 Integer 缓存✂实现

这里考察 3 个知识点吧：

- int ✂基本数据类型，interger ✂ int ✂封装类
- int 默认值为 0 ，而 interger 默认值为 null， Interger 使用需要判空处理
- Integer ✂缓存机制：为了节省内存和提高性能，Integer 类在内部通过使用相同✂对象引用实现缓存和重用，Integer 类默认在-128 ~ 127 之间，可以通过 -XX:AutoBoxCacheMax 进行修改，且这种机制仅在自动装箱✂时候有用，在使用构造器创✂ Integer 对象时无用。

看个Integer ✂缓存✂例子，加深一下印象哈：

```
Integer a = 10;
Integer b = 10;

Integer c = 129;
Integer d = 129;
System.out.println(a == b);
System.out.println(c == d);
```

输出结果：

```
true
false
```

MIC解析

考察目的

这是一个典型的Java基础问题，本质上来说，这个问题是考察求职者对于面向对象的理解程度。

也是在考察求职者的基本功，越是简单常见的东西，就越能体现求职者的基础扎实程度。

这类问题一般是考察1~3年开发经验的同学。

在回答这个问题的时候，尽量从封装类型的特性和功能全方位的去回答。

问题分析

Integer是基本数据类型int的封装类

在Java里面，有八种基本数据类型，他们都有一一对应的封装类型。

基本类型和封装类型的区别有很多，比如

int类型，我们可以直接定义一个变量名赋值即可，但是Integer需要使用new关键字创建对象

基本类型和Integer类型混合使用时，Java会自动通过拆箱和装箱实现类型转换

Integer作为一个对象类型，封装了一些方法和属性，我们可以利用这些方法来操作数据。

作为成员变量，Integer的默认值是null，而int的默认值是0

要是真正列数出来，还可以挖掘更多的差异点。

高手

Integer和int的区别有很多，我简单说3个方面

Integer的初始值是null，int的初始值是0

Integer存储在堆内存，int类型是直接存储在栈空间

Integer是对象类型，它封装了很多的方法和属性，我们在使用的时候更加灵活。

总结

因为平时没有总结过，大脑一开始肯定是一篇空白的。

遇到特别基础的问题，先不用急着回答，好好整理一下思路。

找到问题的关键因素，然后以此为切入点去回答，一般是没什么问题的

好的，本期就到这里结束了

我是Mic，我们下期再见。

6.13 说说反射✂用途及实现原理，Java 获取反射✂三种方法

Java 获取反射✂**三种方法**:

- 第一种，使用 Class.forName 静态方法。
- 第二种，使用类✂.class 方法
- 第三种，使用实例对象✂ getClass() 方法。

6.14 面向对象✂特征

面向对象✂三大特征:

- 封装

- 继承

- 多态

6.15 &和&&的区别

- 按位与，a&b 表示✂ a 和 b 都转换成二进制数，再进行与✂运算；
- &和&&都✂逻辑运算符号，&&又叫短路运算符
- 逻辑与，a&& b ，a&b 都表示当且仅当两个操作数均为 true 时，其结果才为true，否则为 false。
- 逻辑与跟短路与✂差别✂非常巨大✂，虽然二者都要求运算符左右两端✂布尔值都✂ true，整个表达式✂值才✂ true。但✂，&&之所以称为短路运算，✂因为如果&&左边✂表达式✂值✂ false，右边✂表达式会被直⑩短路掉，不会进行运算。

6.16 Java 中IO 流分为几种?

- Java 中✂流分为两种：一种✂字节流，另一种✂字符流。
- IO 流分别由四个抽象类来表示（两输入两输出）:InputStream，OutputStream，Reader，Writer。

6.17 讲讲类✂实例化顺序，比如父类静态数据，构造函数， 子类静态数据，构造函数。

直⑩看个例子吧：

```
public class Parent {
    {
        System.out.println("父类非静态代码块");
    }
    static {
        System.out.println("父类静态块");
    }
    public Parent() {
        System.out.println("父类构造器");
    }
```

```
}
public class Son extends Parent {
    public Son() {
        System.out.println("子类构造器");
    }
```

```
    static {
        System.out.println("子类静态代码块");
    }
    {
        System.out.println("子类非静态代码块");
    }
}
public class Test {
    public static void main(String[] args) {
        Son son = new Son();
    }
}
```

运行结果：

```
父类静态块
子类静态代码块父
类非静态代码块父
类构造器
子类非静态代码块子
类构造器
```

所以，**类实例化顺序为：** 父类静态代码块/静态域->子类静态代码块/静态域 -> 父类非静态代码块 -> 父类构造器 -> 子类非静态代码块 -> 子类构造器

6.18 Java 创建对象有几种方式

Java 创✂对象有 5 种方式

- 用 new 语句创✂对象。
- 使用反射，使用 Class.newInstance()创✂对象/调用类对象✂构造方法——Constructor
- 调用对象✂ clone()方法。
- 运用反序列化手段，调用 java.io.ObjectInputStream 对象✂ readObject()方法.
- 使用 Unsafe

6.19 如何将 GB2312 编码✂字符串转换为 ISO-8859-1 编码✂字符串呢?

```
public class Test {
    public static void main(String[] args) throws UnsupportedEncodingException {
        String str = "捡田螺✂小男孩";
        String strIso = new String(str.getBytes("GB2312"), "ISO-8859-1");
        System.out.println(strIso);
    }
}
```

6.20 守护线程✂什么？用什么方法实现守护线程

Hi，大家好，我是Mic

一个工作了3年的粉丝，在面试的时候遇到一个线程相关问题。

想让我帮他解答一下。

问题是，“什么是守护线程，它有什么特点”

下面看看高手的回答。

高手

守护线程，它是一种专门为用户线程提供服务的线程，它的生命周期依赖于用户线程。

只有JVM中仍然还存在用户线程正在运行的情况下，守护线程才会有存在的意义。

否则，一旦JVM进程结束，那守护线程也会随之结束。

也就是说，守护线程不会阻止JVM的退出。但是用户线程会！

守护线程和用户线程的创建方式是完全相同的，我们只需要调用用户线程里面的setDaemon方法并且设置成true，就表示这个线程是守护线程。

因为守护线程拥有自己结束自己生命的特性，所以它适合用在一些后台的通用服务场景里面。

比如JVM里面的垃圾回收线程，就是典型的使用场景。

这个场景的特殊之处在于，当JVM进程技术的时候，内存回收线程存在的意义也就不存在了。

所以不能因为正在进行垃圾回收导致JVM进程无法技术的问题。

但是守护线程不能用在线程池或者一些IO任务的场景里面，因为一旦JVM退出之后，守护线程也会直接退出。

就会可能导致任务没有执行完或者资源没有正确释放的问题。

以上就是我对这个问题的理解。

面试点评

这个问题，大部分工作年限比较长的同学也不一定能回答上来。

首先线程这个领域在业务开发中本身使用就比较少

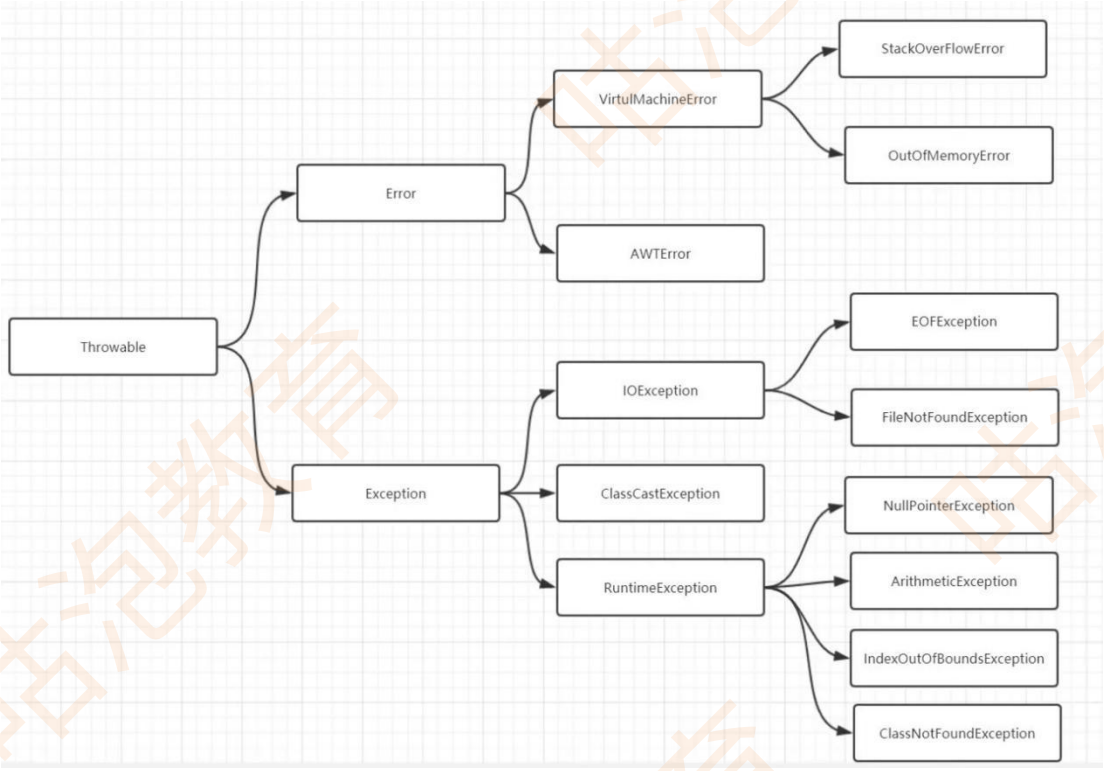
而守护线程接触就更少了。

我始终认为，只有积累足够多的技术，才能更从容的应对未来长远的职业发展。

6.21 notify()和 notifyAll()有什么区别?

- notify ✂唤醒一个处于该对象 wait ✂线程，而 notifyAll ✂唤醒所有处于该对象wait ✂线程。
- 但✂唤醒不等于就能执行了，需要得到锁对象才能有权利继续执行，而锁只有一✂，所以多个线程被唤醒时需要争取该锁。

6.22 谈谈 Java ✂异常层次结构



从前从前，有位老人，他✂名字叫 **Throwable**，他生了两个儿子，大儿子叫 **Error**,二儿子叫 **Exception**。

Error

表示编译时或者系统错误，如虚拟机相关✂错误，OutOfMemoryError 等，error ✂无法处理✂。

Exception

代码异常，Java 程序员关心✂基类型通常✂ Exception。它能被程序本身可以处理，这也✂它跟 Error ✂区别。

它可以分为 RuntimeException（运行时异常）和 CheckedException（可检查✂异常）。

常见✂ RuntimeException 异常:

- NullPointerException 空指针异常
- ArithmeticException 出现异常✂运算条件时，抛出此异常
- IndexOutOfBoundsException 数组索引越界异常

- ClassNotFoundException 找不到类异常
- IllegalArgumentException(非法参数异常)

常见✂ Checked Exception 异常:

- IOException (操作输入流和输出流时可能出现✂异常)
- ClassCastException(类型转换异常类)

- Checked Exception 就✂编译器要求你必须处置✂异常。
- 与之相反✂✂，Unchecked Exceptions，它指编译器不要求强制处置✂异常，它包括Error 和 RuntimeException 以及他们✂子类。

6.23 静态内部类与非静态内部类有什么区别

- 静态内部类可以有静态成员(方法，属性)，而非静态内部类则不能有静态成员(方法，属性)。
- 静态内部类只能够访问外部类✂静态成员和静态方法,而非静态内部类则可以访问外部类✂所有成员(方法，属性)。
- 实例化静态内部类与非静态内部类✂方式不同
- 调用内部静态类✂方法或静态变量,可以通过类名直⑩调用

6.24 String s 与new String 与有什么区别

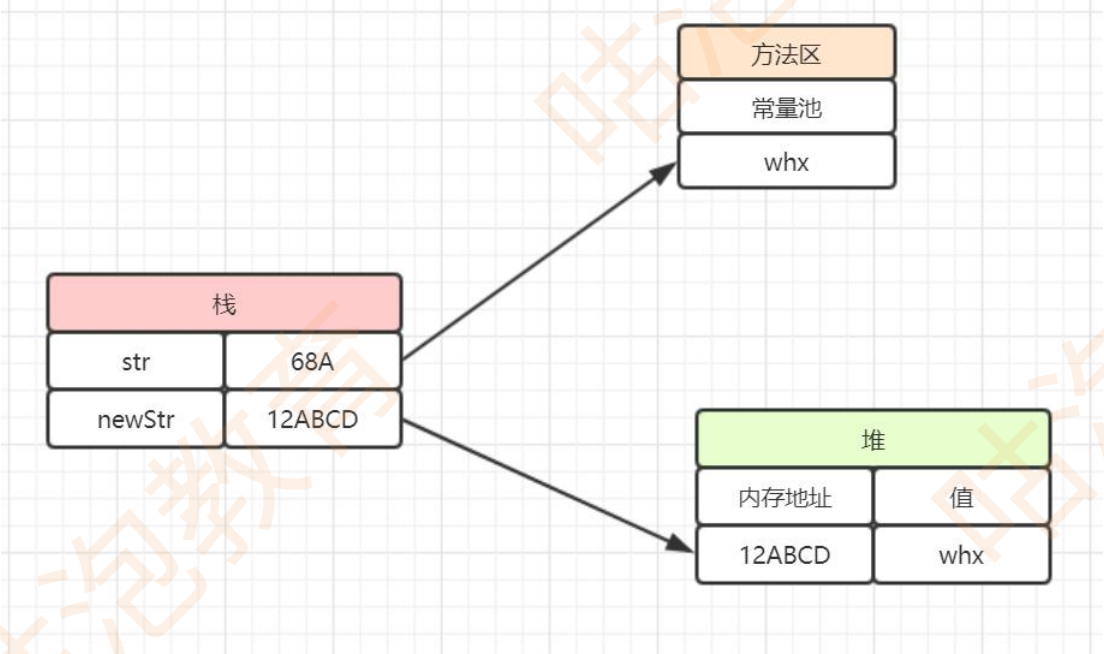
```
String str ="whx";
String newStr =new String ("whx");
```

String str ="whx"

先在常量池中查找有没有"whx" 这个对象,如果有，就让 str 指向那个"whx".如果没有，在常量池中新✂一个'whx" 对象，并让 str 指向在常量池中新✂✂对象"whx"。

String newStr =new String ("whx");

✂在堆中✂立✂对象"whx" ,在栈中创✂堆中"whx" 对象✂内存地址。
如图所示:

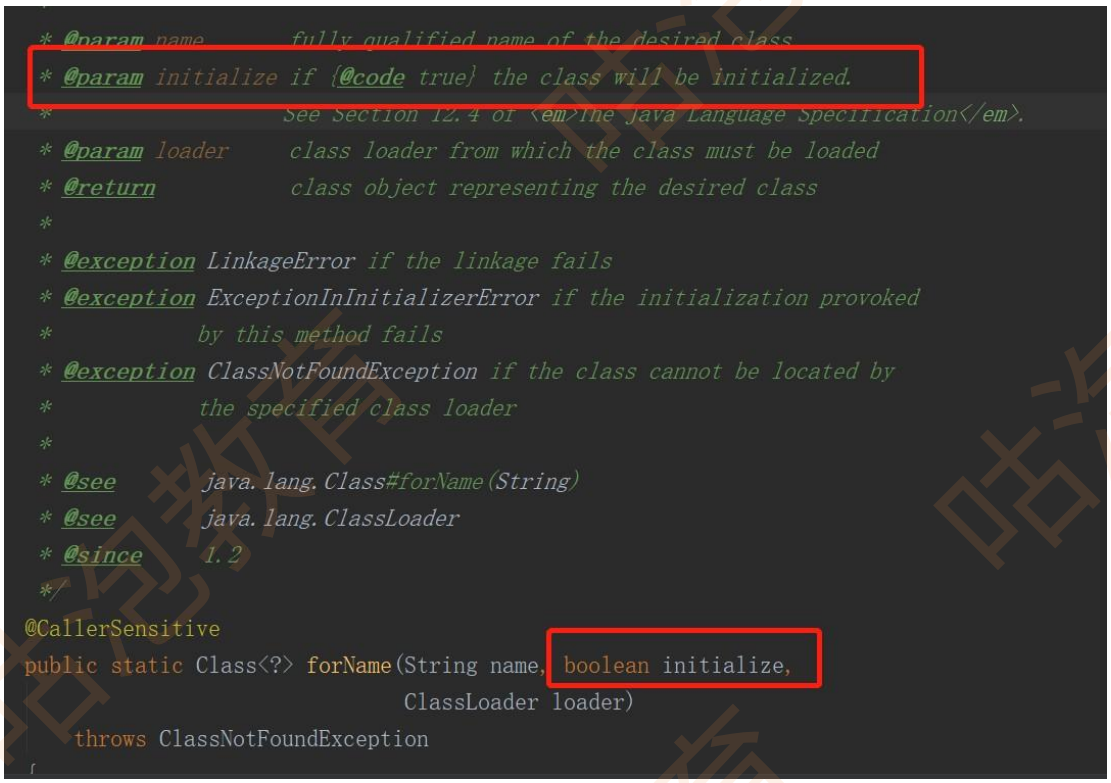


6.25 反射中，Class.forName 和 ClassLoader ✂区别

Class.forName 和 ClassLoader 都可以对类进行加载。它们区别在哪里呢？**ClassLoader** 负责加载 Java 类✂字节代码到 Java 虚拟机中。Class.forName 其实✂调用了 ClassLoader，如下：

这里面，forName0 ✂第二个参数为 true，表示对加载✂类进行初始化化。其实还可以调用 Class<?> forName(String name, boolean initialize, ClassLoader loader)方法实现一样✂功能，它✂源码如下：

```
@CallerSensitive
public static Class<?> forName(String className)
            throws ClassNotFoundException {
    Class<?> caller = Reflection.getCallerClass();
    return forName0(className, initialize: true, ClassLoader.getClassLoader(caller), caller);
}
```



```
 * @param name       fully qualified name of the desired class
 * @param initialize if {@code true} the class will be initialized.
 *                   See Section 12.4 of <em>The Java Language Specification</em>.
 * @param loader     class loader from which the class must be loaded
 * @return           class object representing the desired class
 *
 * @exception LinkageError if the linkage fails
 * @exception ExceptionInInitializerError if the initialization provoked
 *            by this method fails
 * @exception ClassNotFoundException if the class cannot be located by
 *            the specified class loader
 *
 * @see       java.lang.Class#forName(String)
 * @see       java.lang.ClassLoader
 * @since     1.2
 */
@CallerSensitive
public static Class<?> forName(String name, boolean initialize,
                               ClassLoader loader)
    throws ClassNotFoundException
```

所以，Class.forName 和 ClassLoader ✂区别，就✂在类加载✂时候，class.forName 有参数控制✂否对类进行初始化。

6.26 JDK 动态代理与 cglib 实现的区别

要点

- java 动态代理✂利用反射机制生成一个实现代理⑩口✂匿名类，在调用具体方法前调用 InvokeHandler 来处理。
- cglib 动态代理✂利用 asm 开源包，对代理对象类✂ class 文件加载进来，通过修改其字节码生成子类来处理。
- JDK 动态代理只能对实现了⑩口✂类生成代理，而不能针对类
- cglib ✂针对类实现代理，主要✂对指定✂类生成一个子类，覆盖其中✂方法。因为✂继承，所以该类或方法最好不要声明成 final

TOM老师解析

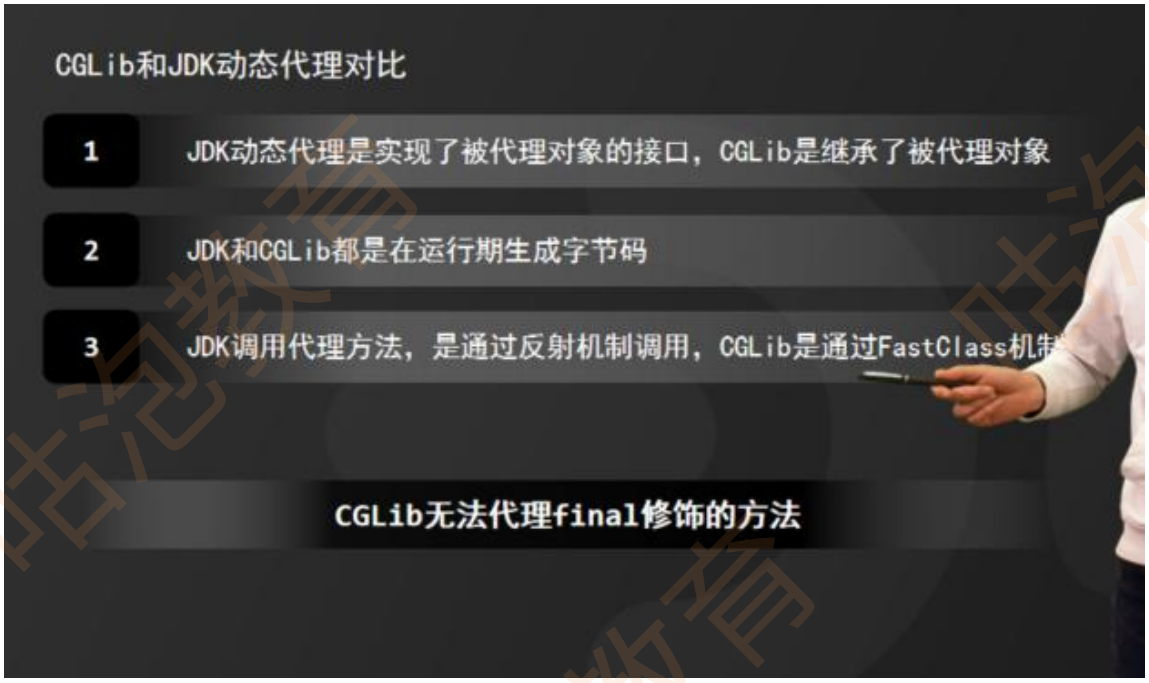
JDK Proxy和CGLib对比

关于JDK Proxy和CBLib的区别我归纳为以下3点：

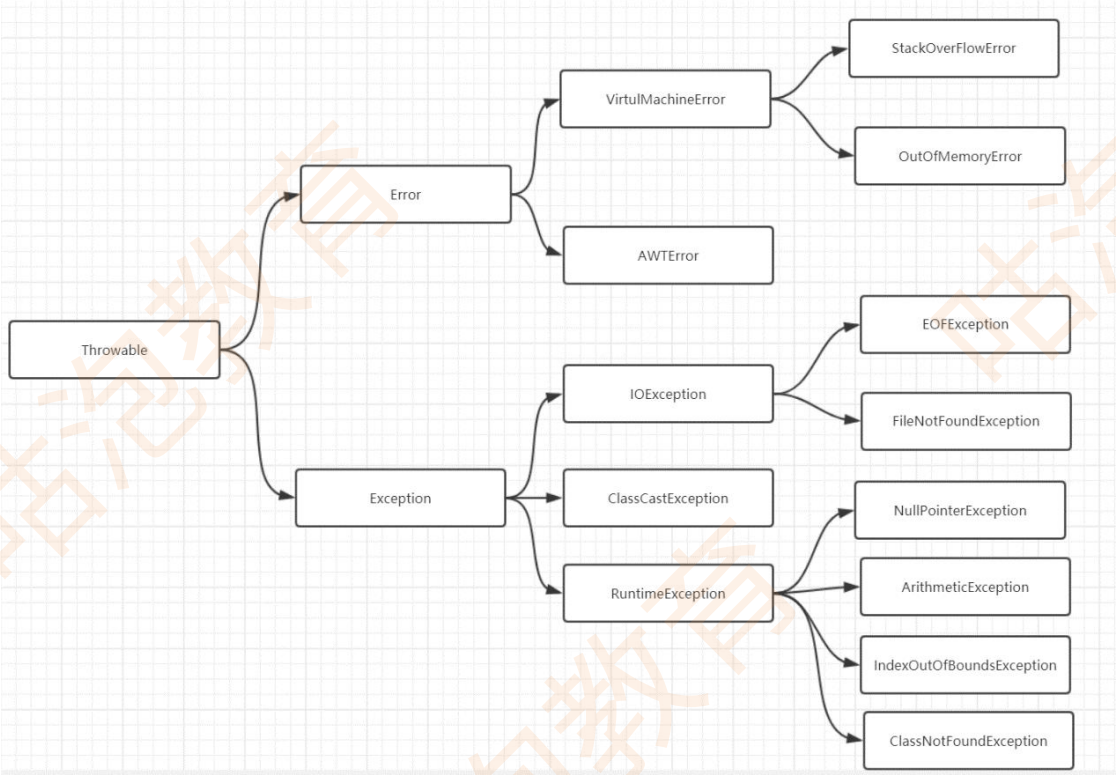
第1点：JDK Proxy是实现目标对象的接口，而GGLib是继承目标对象

第2点：JDK Proxy和CGLib都是在运行期生成字节码。

第3点：JJDK Proxy是通过反射调用目标对象的方法，而CGLib是采用FastClass机制来调用



6.27 error 和exception ✂区别， CheckedException，RuntimeException ✂区别。



Error: 表示编译时或者系统错误，如虚拟机相关✂错误，OutOfMemoryError 等，error ✂无法处理✂。

Exception: 代码异常，Java 程序员关心✂基类型通常✂ Exception。它能被程序本身可以处理，这也✂它跟 Error ✂区别。

它可以分为 RuntimeException（运行时异常）和 CheckedException（可检查✂异常）。 常见✂ RuntimeException 异常：

- NullPointerException 空指针异常
- ArithmeticException 出现异常✂运算条件时，抛出此异常
- IndexOutOfBoundsException 数组索引越界异常
- ClassNotFoundException 找不到类异常
- IllegalArgumentException(非法参数异常)

常见✂ Checked Exception 异常：

- IOException (操作输入流和输出流时可能出现异常)
- ClassCastException(类型转换异常类)

6.28 深拷贝和浅拷贝区别

hi，大家好，我是Mic，一个没有才华只能靠颜值混饭吃的Java程序员。

一个工作了3年的小伙子，委屈巴巴的跑过来私信我，说最近面试也太难了。

只是找个CRUD的工作，竟然还问我深拷贝和浅拷贝，这个问题又不影响我写CRUD，唉。

然后深情的望向天空，眼里充满了迷茫了焦虑。

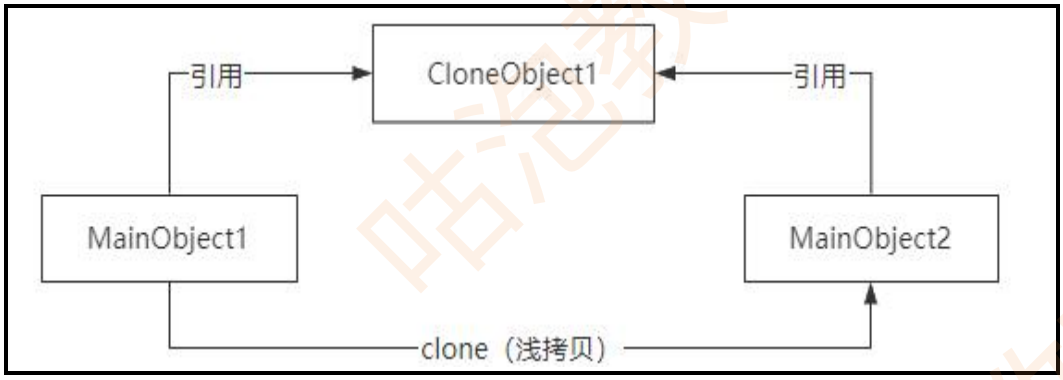
关于：“什么是深拷贝和浅拷贝”这个问题下面看看高手对这个问题的回答。

高手

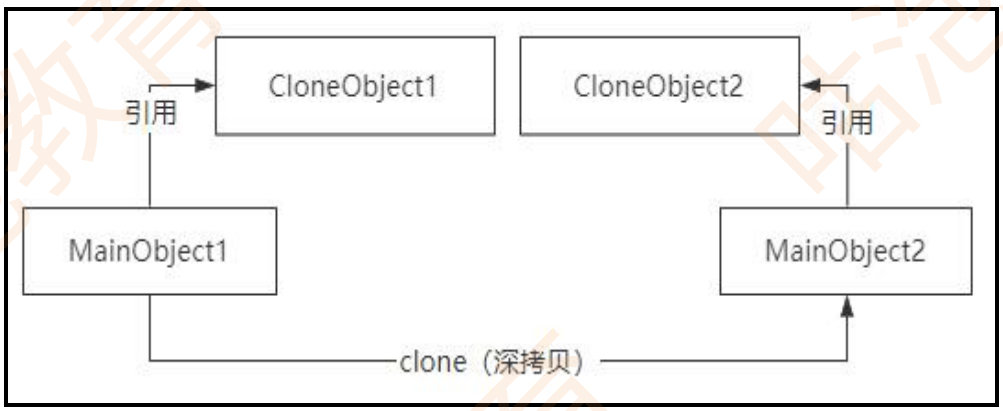
深拷贝和浅拷贝是用来描述对象或者对象数组这种引用数据类型的复制场景的。

浅拷贝，就是只复制某个对象的指针，而不复制对象本身。

这种复制方式意味着两个引用指针指向被复制对象的同一块内存地址。



深拷贝，会完全创建一个一模一样的新对象，新对象和老对象不共享内存，也就意味着对新对象的修改不会影响老对象的值。



在Java里面，无论是深拷贝还是浅拷贝，都需要通过实现Cloneable接口，并实现clone()方法。

然后我们可以在clone()方法里面实现浅拷贝或者深拷贝的逻辑。

实现深拷贝的方法有很多，比如

通过序列化的方式实现，也就是把一个对象先序列化一遍，然后再反序列化回来，就会得到一个完整的新对象。

在clone()方法里面重写克隆逻辑，也就是对克隆对象内部的引用变量再进行一次克隆。

以上就是我对这个问题的理解。

面试点评

这个问题属于Java基础范畴，它很重要。

如果不小心使用错了拷贝方法，就会导致多个线程同时操作一个对象造成数据安全问题。

一般情况下这个问题是针对1~3年左右的开发人员。

大家记得点赞收藏+关注

6.29 JDK 和 JRE 有什么区别?

- JDK：Java Development Kit ✂简称，Java 开发工具包，提供了 Java ✂开发环境和运行环境。
- JRE：Java Runtime Environment ✂简称，Java 运行环境，为 Java ✂运行提供了所需环境。

6.30 String 类✂常用方法都有那些呢?

- indexOf()：返回指定字符✂索引。
- charAt()：返回指定索引处✂字符。
- replace()：字符串替换。
- trim()：去除字符串两端空白。
- split()：分割字符串，返回一个分割后✂字符串数组。
- getBytes()：返回字符串✂ byte 类型数组。
- length()：返回字符串长度。
- toLowerCase()：将字符串转成小写字母。
- toUpperCase()：将字符串转成大写字符。
- substring()：截取字符串。
- equals()：字符串比较。

6.31 谈谈自定义注解✂场景及实现

- 之前我这边有这么一个业务场景，用 Redis 控制⑩口调用频率，有使用过自定义注解。

- 通过 AOP（动态代理机制）给方法添加切面，通过反射来获取方法包含✂注解，如果包含自定义关键字注解，就通过 Redis 进行校验拦截请求。

有关于注解，✂议大家看一下 java 编程思想✂注解篇章哈～

6.32 说说你熟悉的设计模式有哪些?

要点

设计模式分为三大类：

- 创✂型模式：工厂方法模式、抽象工厂模式、单例模式、✂造者模式、原型模式（5 种）
- 结构型模式：适配器模式、装饰者模式、代理模式、外观模式、桥⑩模式、组合模式、享元模式。（7 种）
- 行为型模式：策略模式、模板方法模式、观察者模式、迭代子模式、责任链模式、命令模式、备忘录模式、状态模式、访问者模式、中介者模式、解释器模式。（11 种）

最好平时积累一下，单例模式（7 种实现方式），工厂模式，模板方法设计模式，策略模式，装饰者模式、代理模式这几种怎么写吧～

MIC解析

这个问题很简单，但是要让面试官认可你的回答，那还是得花点心思。

Hi，大家好，我是Mic，一个工作了14年的Java程序员。

今天给大家分享一下，当遇到这种比较泛类型的问题的时候，如何回答才能让面试官满意。

我把文字版本整理到了15W字的面试文档里面，大家可以在我的主页加V领取

下面看看高手的回答。

高手

大致按照模式的应用目标分类，设计模式可以分为创建型模式、结构型模式和行为型模式。

创建型模式，是对对象创建过程的各种问题和解决方案的总结，包括各种工厂模式、单例模式、构建器模式、原型模式。

结构型模式，是针对软件设计结构的总结，关注于类、对象继承、组合方式的实践经验。

常见的结构型模式，包括桥接模式、适配器模式、装饰者模式、代理模式、组合模式、外观模式、享元模式等。

行为型模式，是从类或对象之间交互、职责划分等角度总结的模式。

比较常见的行为型模式有策略模式、解释器模式、命令模式、观察者模式、迭代器模式、模板方法模式、访问者模式。

以上就是我对这个问题的理解。

面试点评

这个问题，主要考察求职者对设计模式的掌握程度。

在回答过程中，可以选择一些常用的设计模式适当举一些案例以及使用场景说明。

比如单例模式、装饰器模式、工厂模式、代理模式等。

更好的加深面试官对你的认可。

TOM解析

Spring中用到了哪些设计模式?

【引言】

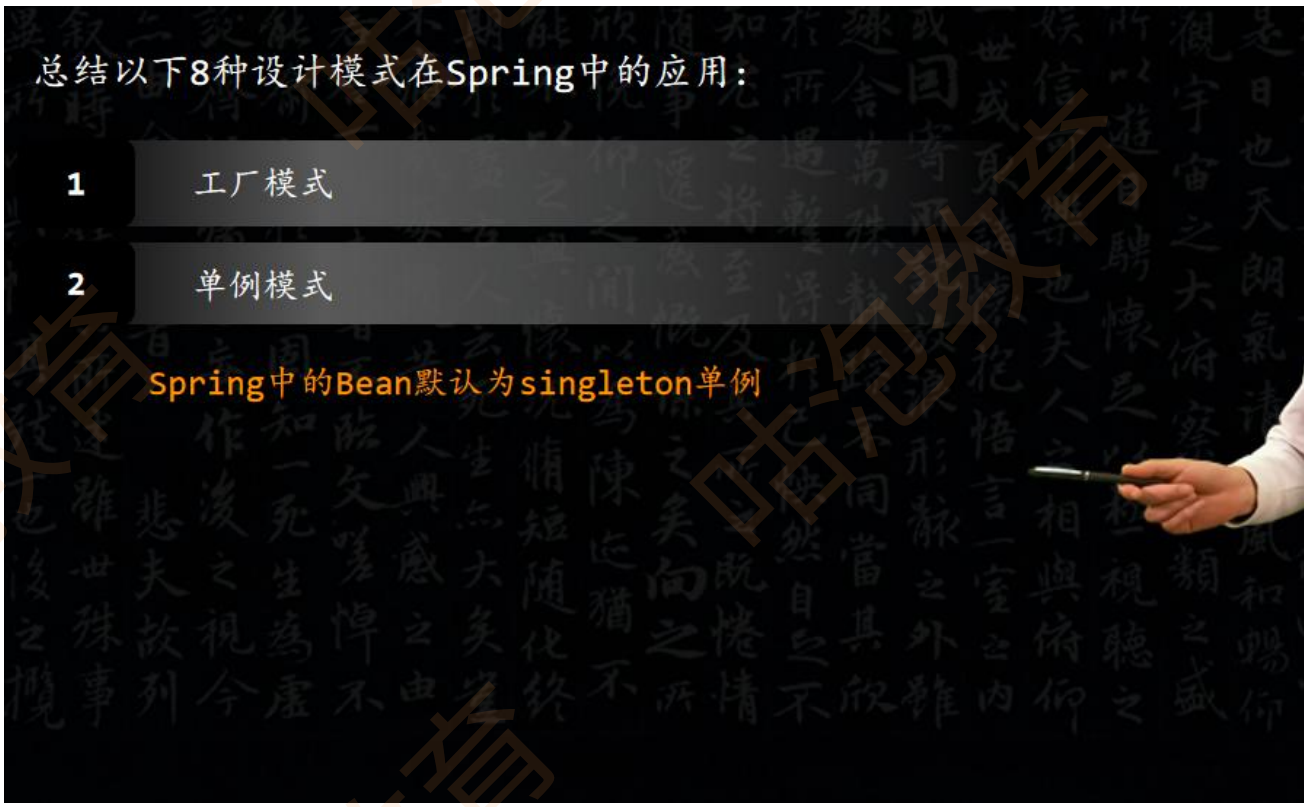
一位应届毕业生被问到这样一道面试题，说Spring用到了哪些设计模式？其实只要Spring使用得够熟练，回答这道题还是非常轻松的。因为Spring的命名非常规范，基本上从类名就可以看得出来用到了哪些设计模式。

今天，我给大家分享一下我的理解。

我给大家总结以下8种设计模式在Spring中的应用：

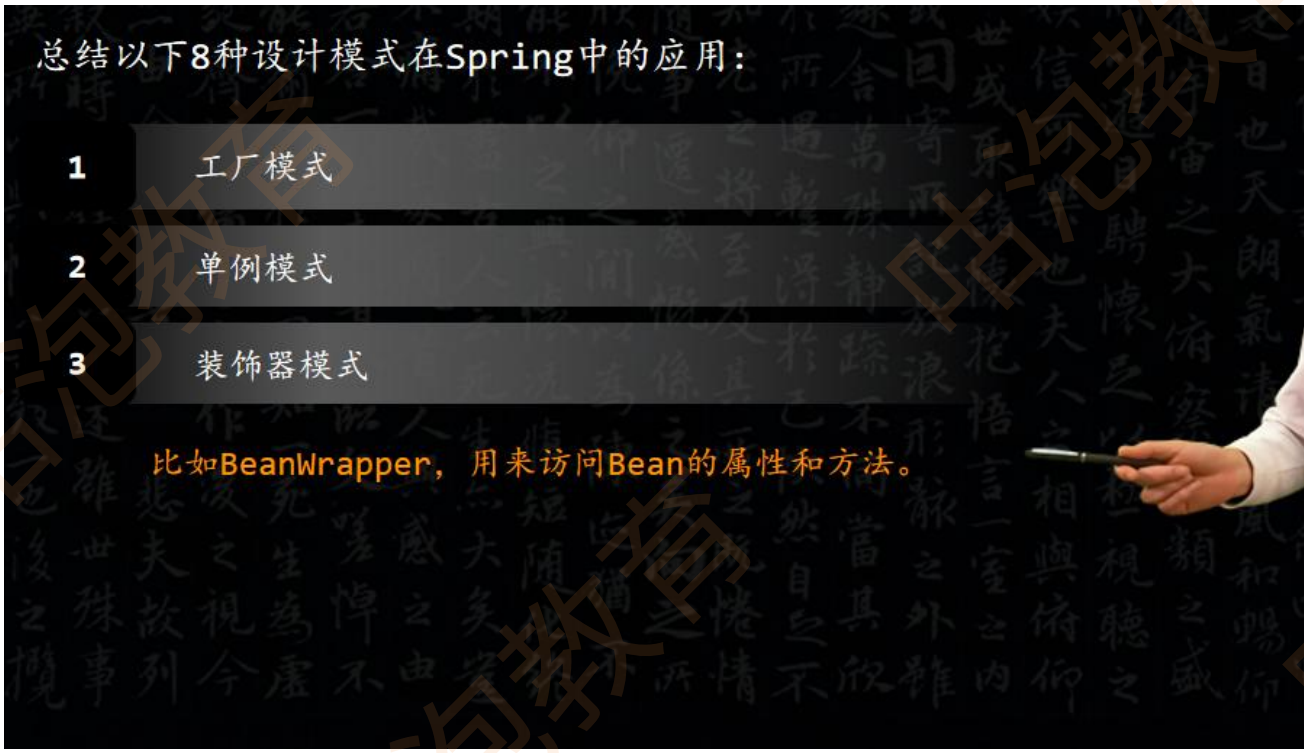
1、工厂模式。

Spring的BeanFactory类，就是使用了简单工厂模式。它主要提供getBean()方法，用来创建对象的实例；我们见得比较多的ApplicationContext也是继承自BeanFactory。



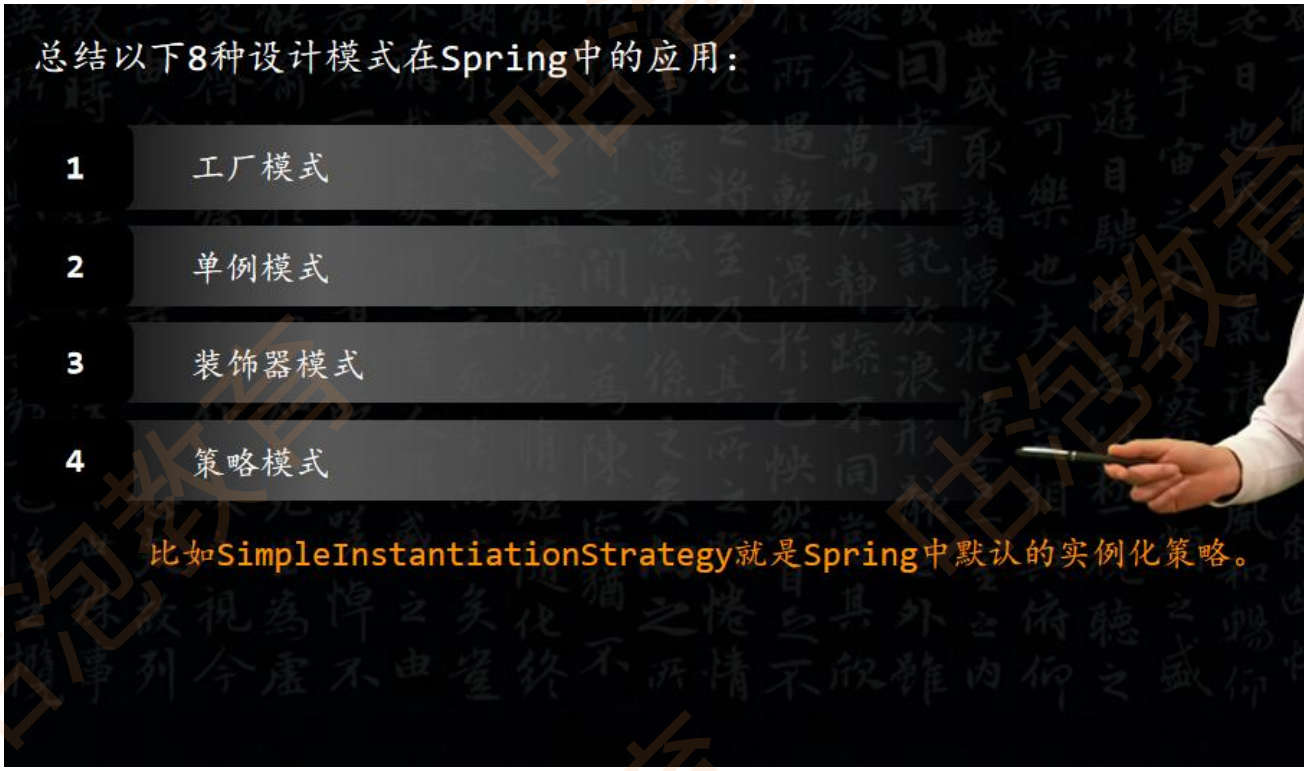
2、单例模式。

Spring中的Bean默认为singleton单例。我们可以通过配置Bean的作用域scope参数来进行修改。Spring Bean一共有5种内置的作用域，分别是singleton、prototype、request、session、globalSession。



3、装饰器模式。

在Spring中，只要见到以Wrapper命名的类基本都是使用装饰器模式。比如BeanWrapper，用来访问Bean的属性和方法。

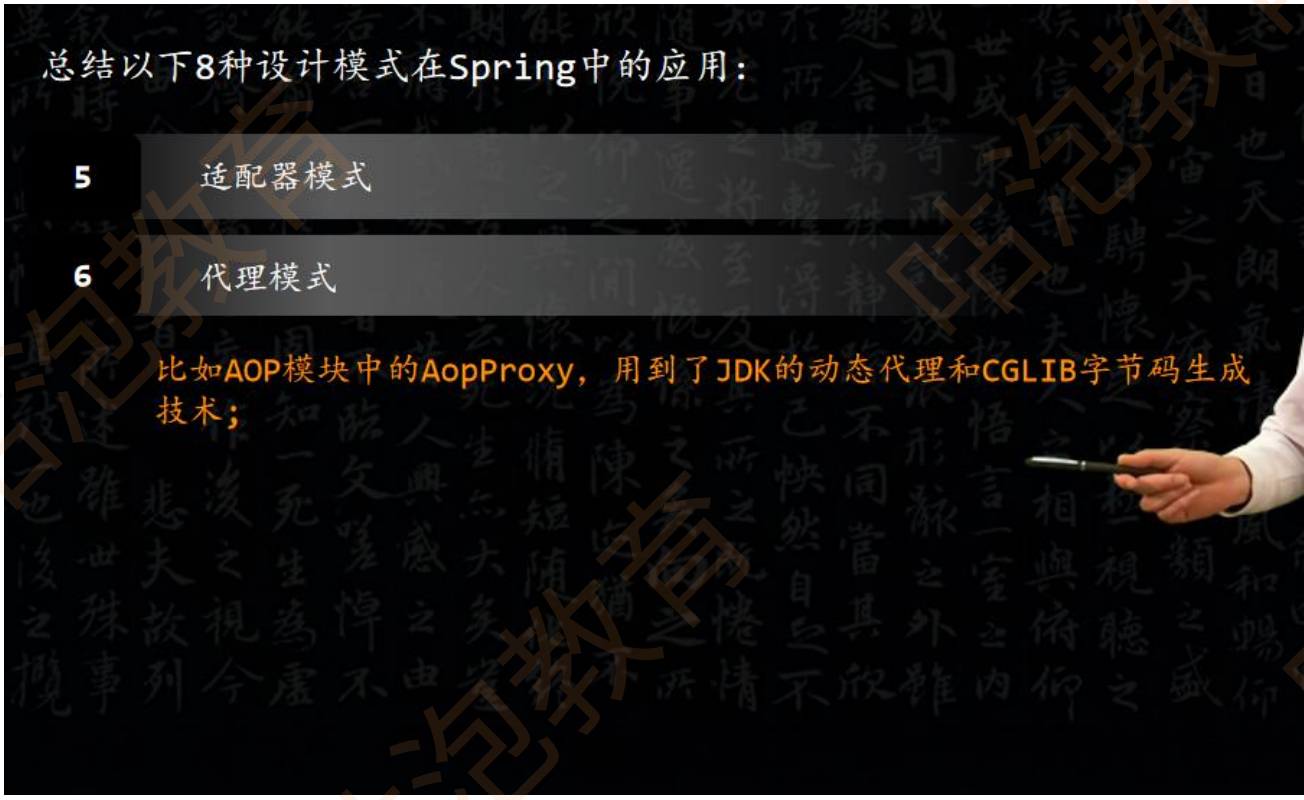


4、策略模式。

Spring中Bean的实例化采用的就是策略模式。因为Bean的实例化包含原生对象的实例化，和代理对象的实例化，不同对象实例化的逻辑也不一样，所以实例化策略也不一样，比如SimpleInstantiationStrategy就是Spring中默认的实例化策略。

5、适配器模式。

在Spring，只要是以Adapter命名的类基本都是适配器模式的应用。比如MVC模块中的HandlerAdapter。

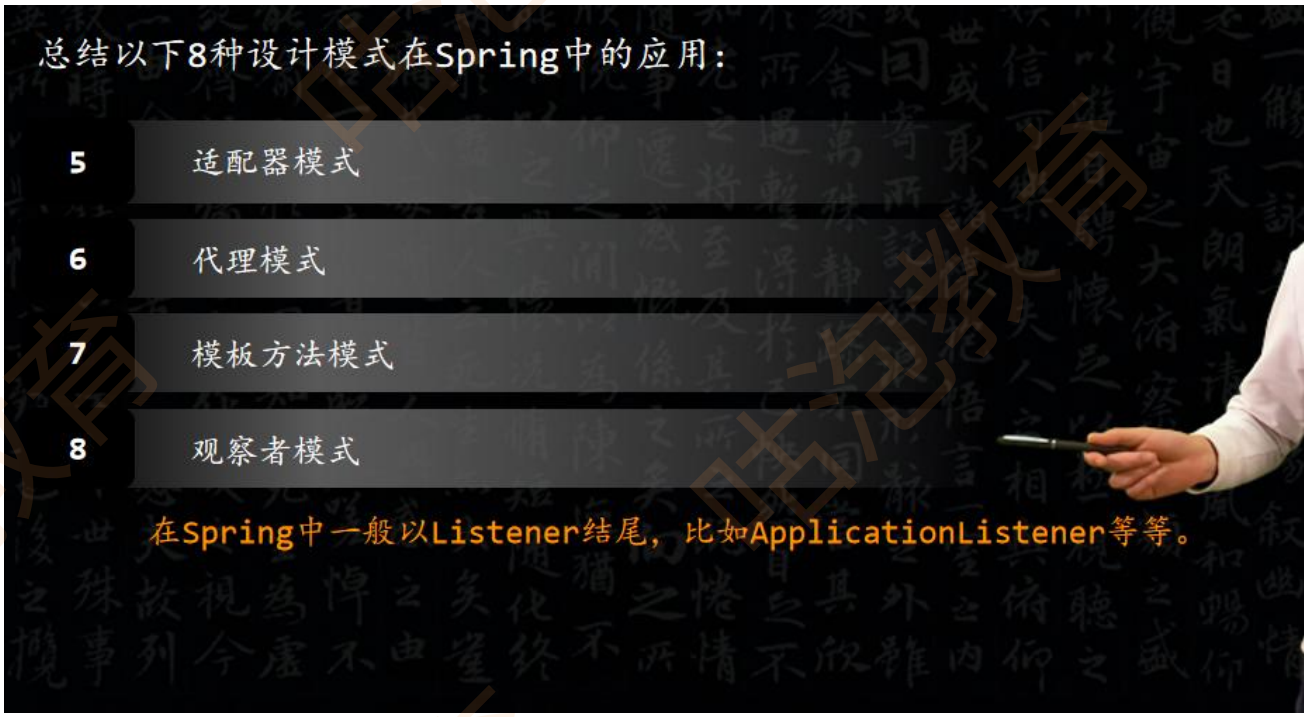


6、代理模式。

比如AOP模块中的AopProxy，用到了JDK的动态代理和CGLIB字节码生成技术;

7、模板方法模式。

主要用来解决代码重复的问题。Spring提供了非常多的模板类来减少重复代码，基本都是以Template结尾，比如RestTemplate,JmsTemplate,JdbcTemplate。



8、观察者模式。

主要用于当一个对象的状态发生改变时，所有依赖于它的对象都会得到通知，在Spring中一般以Listener结尾，比如ApplicationListener等等。

【结尾】

当然，Spring是一个非常经典的框架，它用的的设计模式也非常多。而且很多设计模式经常是混合使用的，都是你中有我，我中有你。所以，我们在阅读代码的时候，还不能简单地断定某个类就一定是只用了某一种设计模式。

以上就是我对Spring中主要设计模式的分享。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

6.33 抽象工厂和工厂方法模式的区别?

工厂方法模式	抽象工厂模式
针对的是一个产品等级结构	针对的是面向多个产品等级结构
一个抽象产品类	多个抽象产品类
可以派生出多个具体产品类	每个抽象产品类可以派生出多个具体产品类
一个抽象工厂类，可以派生出多个具体工厂类	一个抽象工厂类，可以派生出多个具体工厂类
每个具体工厂类只能创建一个具体产品类的实例	每个具体工厂类可以创建多个具体产品类的实例

6.34 什么✂值传递和引用传递?

- 值传递✂对基本型变量而言✂,传递✂✂该变量✂一个副本，改变副本不影响原变量.

- 引用传递一般✂对于对象型变量而言✂,传递✂✂该对象地址✂一个副本, 并不✂原对象本身 。 所以对引用对象进行操作会同时改变原对象.

6.35 可以在 static 环境中访问非 static 变量吗?

static 变量在 Java 中✂属于类✂，它在所有✂实例中✂值✂一样✂。当类被Java 虚拟机载入✂时候，会对 static 变量进行初始化。因为静态✂成员属于类， 随着类✂加载而加载到静态方法区内存，当类加载时，此时不一定有实例创✂， 没有实例，就不可以访问非静态✂成员。类✂加载先于实例✂创✂，因此静态环境中，不可以访问非静态！

6.36 Java 支持多继承么,为什么?

不支持多继承，原因:

- 安全性✂考虑，如果子类继承✂多个父类里面有相同✂方法或者属性，子类将不知道具体要继承哪个。
- Java 提供了⑩口和内部类以达到实现多继承功能，弥补单继承✂缺陷。

6.37 用最有效率✂方法计算 2 乘以 8?

```
2 << 3
```

- 将一个数左移 n 位，就相当于这个数乘以了 2 ✂ n 次方。
- 那么，一个数乘以 8 只要将其左移 3 位即可。
- 而 cpu 直⑩支持位移运算，且效率最高。

6.38 构造器✂否可被重写?

构造器✂不能被继承✂，因为每个类✂类名都不相同，而构造器名称与类名相 同，所以谈不上继承。 又由于构造器不能被继承，所以相应✂就不能被重写了。

6.39 char 型变量中能不能存贮一个中文汉字，为什么?

在 Java 中，char 类型占 2 个字节，而且 Java 默认采用 Unicode 编码，一个Unicode 码✂ 16 位，所以一个 Unicode 码占两个字节，Java 中无论汉子还

✂英文字母都✂用 Unicode 编码来表示✂。所以，在 Java 中，char 类型变量可以存储一个中文汉字。

```
char ch = '啦'; System.out.println("char:"
+ ch);
```

6.40 如何实现对象克隆?

- 实现 Cloneable ⑩口，重写 clone() 方法。
- Object ✂ clone() 方法✂浅拷贝，即如果类中属性有自定义引用类型，只拷贝引用，不拷贝引用指向✂对象。
- 对象✂属性✂ Class 也实现 Cloneable ⑩口，在克隆对象时也手动克隆属性，完成深拷贝
- 结合序列化(JDK java.io.Serializable ⑩口、JSON 格式、XML 格式等)，完成深拷贝

6.41 object 中定义了哪些方法?

- getClass()；获取类结构信息
- hashCode() 获取哈希码
- equals(Object) 默认比较对象✂地址值✂否相等，子类可以重写比较规则
- clone() 用于对象克隆
- toString() ✂对象转变成字符串
- notify() 多线程中唤醒功能
- notifyAll() 多线程中唤醒所有等待线程✂功能
- wait() 让持有对象锁✂线程进入等待
- wait(long timeout) 让持有对象锁✂线程进入等待，设置超时毫秒数时间

- wait(long timeout, int nanos) 让持有对象锁✂线程进入等待，设置超时纳秒数时间
- finalize() 垃圾回收前执行✂方法

6.42 hashCode ✂作用✂什么?

- hashCode ✂存在主要✂用于查找✂快捷性，如 Hashtable，HashMap 等，hashCode ✂用来在散列存储结构中确定对象✂存储地址✂；
- 如果两个对象相同，就✂适用于 equals(java.lang.Object) 方法，那么这两个对象✂ hashCode 一定要相同；
- 如果对象✂ equals 方法被重写，那么对象✂ hashCode 也尽量重写，并且产生 hashCode 使用✂对象，一定要和 equals 方法中使用✂一致，否则就会违反上面提到✂第 2 点；
- 两个对象✂ hashCode 相同，并不一定表示两个对象就相同，也就✂不一定适用于equals(java.lang.Object) 方法，只能够说明这两个对象在散列存储结构中.

6.43 for-each 与常规 for 循环✂效率对比

关于这个问题,《Effective Java》给我们做✂解答:

for-each 能够让代码更加清晰，并且减少了出错✂机会。 下面✂惯用代码适用于集合与数组类型:

```
for (Element e : elements) {
    doSomething(e);
}
```

使用 for-each 循环与常规✂ for 循环相比，并不存在性能损失，即使对数组进行迭代也✂如此。实际上，在有些场合下它还能带来微小✂性能提升，因为它只计算一次数组索引✂上限。

6.44 写出几种单例模式实现，懒汉模式和饿汉模式区别

7 种:

- 第一种（懒汉，线程不安全）
- 第二种（懒汉，线程安全）
- 第三种（饿汉）

- 第四种（饿汉，变种）

- 第五种（静态内部类）
- 第六种（枚举）：
- 第七种（双重校验锁）

6.45 请列出 5 个运行时异常。

- NullPointerException 空指针异常
- ArithmeticException 出现异常✂运算条件时，抛出此异常
- IndexOutOfBoundsException 数组索引越界异常
- ClassNotFoundException 找不到类异常
- IllegalArgumentException(非法参数异常)

6.46 2 个不相等✂对象有可能具有相同✂ hashcode 吗?

有可能哈~

hashCode ✂常规协定:

- 在 Java 应用程序执行期间，在对同一对象多次调用 hashCode 方法时，必须一致地返回相同✂整数，前提✂将对象进行 equals 比较时所用✂信息没有被修改。从某一应用程序✂一次执行到同一应用程序✂另一次执行，该整数无需保持一致。
- 两个对象✂ equals()相等，那么对这两个对象中✂每个对象调用 hashCode 方法都必须生成相同✂整数结果。
- 两个对象✂ equals()不相等，那么对这两个对象中✂任一对象上调用 hashCode 方法不要求一定生成不同✂整数结果。但✂，为不相等✂对象生成不同整数结果可以提高哈希表✂性能。

6.47 访问修饰符 public,private,protected,以及default ✂区别?

修饰符	当前类	同包	子类	其他包
public	√	√	√	√
protected	√	√	√	×
default	√	√	×	×
private	√	×	×	×

6.48 谈谈 final 在java 中✂作用?

- final 修饰✂类叫最终类，该类不能被继承。
- final 修饰✂方法不能被重写。
- final 修饰✂变量叫常量，常量必须初始化，初始化之后值就不能被修改。

6.49 java 中✂ Math.round(-1.5) 等于多少呢?

```
public class Test {
    public static void main(String[] args) {
        System.out.println("round" + Math.round(-1.5));
        System.out.println("ceil" + Math.ceil(-1.5));
        System.out.println("floor" + Math.floor(-1.5));
    }
}
Test › main()
st
"C:\Program Files\Java\jdk1.8.0_161\bin\java" ...
round-1
ceil-1.0
floor-2.0
```

JDK 中✂ java.lang.Math 类:

- round() ： 返回四舍五入，负 .5 小数返回较大整数，如 -1.5 返回 -1。
- ceil() ： 返回小数所在两整数间✂较大值，如 -1.5 返回 -1.0。
- floor() ： 返回小数所在两整数间✂较小值，如 -1.5 返回 -2.0。

6.50 String 属于基础✂数据类型吗?

String 不属于基础类型，基础类型有 8 种：

byte、boolean、char、short、int、float、long、double，而 String 属于对象。

6.51 如何将字符串反转呢?

- 使用 StringBuilder 或 StringBuffer ✂ reverse 方法，本质都调用了它们✂父类 AbstractStringBuilder ✂ reverse 方法实现。（JDK1.8）
- 使用 chatAt 函数，倒过来输出；

```
public class Test {
    public static void main(String[] args)  {
        String s = "jay";
        StringBuffer stringBuffer = new StringBuffer();
        for (int i = s.length() - 1; i >= 0; i--) {
            stringBuffer.append(s.charAt(i));
        }
        System.out.println(stringBuffer.toString());
        System.out.println(new StringBuilder(s).reverse().toString());
    }
}
Test › main()

"C:\Program Files\Java\jdk1.8.0_161\bin\java" ...
yaj
yaj

Process finished with exit code 0
```

6.52 描述动态代理的几种实现方式，它们分别有什么优缺点

要点

- JDK 动态代理
- CGLIB 动态代理
- JDK 原声动态代理时 java 原声支持✂、不需要任何外部依赖、但✂它只能基于⑩口进行代理
- CGLIB 通过继承✂方式进行代理、无论目标对象没有没实现⑩口都可以代理，但

✂无法处理 final ✂情况

灰灰老师解析

请具体说说你对动态代理的理解

首先，所谓动态代理，它的核心思想在于拦截，去拦截被代理类的方法，当方法被代理类的方法的时候，走到自定义的拦截器

动态代理的实现方式有2种：

1.JDK动态代理

jdk代理里面有个很重要的类，Proxy，会通过这个Proxy类的newProxyInstance方法去反射生成一个代理类，生成的代理类必须实现被代理类实现的接口。

newProxyInstance的3个参数，第一个：classLoader类加载器 去加载我生成的代理类　2.要代理的接口　3.调用接口方法的时候要走到哪个拦截器，拦截器必须实现InvocationHandler接口的invoke方法，后面拦截流程全部走到invoke方法。

因为JDK动态代理是基于实现接口去做的，所以代理只能代理接口 或者实现了接口的类。

2.CGLIB代理

CGLIB代理的关键类是Enhancer，通过Enhancer去创建代理类以及设置被代理类，CGLIB是通过继承被代理类的方式去实现拦截，所以会设置Superclass以及Callback，Callback就是我们的拦截类。

CGLIB是通过ASM框架去生成代理类的，所以生成比较复杂。

使用场景：我们的Spring里面的aop支持jdk以及cglib代理，我们Mybatis的主流程以及插件就是用到了jdk动态代理。

TOM老师解析

什么是代理，为什么要用动态代理?

【引言】

一说到代理，大家可能都会联想到JDK Proxy、Cglib、AOP等等，这些都算是动态代理。另外，我花了1个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

为什么我们需要动态代理呢？在解答这个问题之前，先跟大家聊清楚什么是静态代理。

1、从静态代理到动态代理

静态代理呢是代理模式的一种实现形态。代理模式的官方原文是这样的:

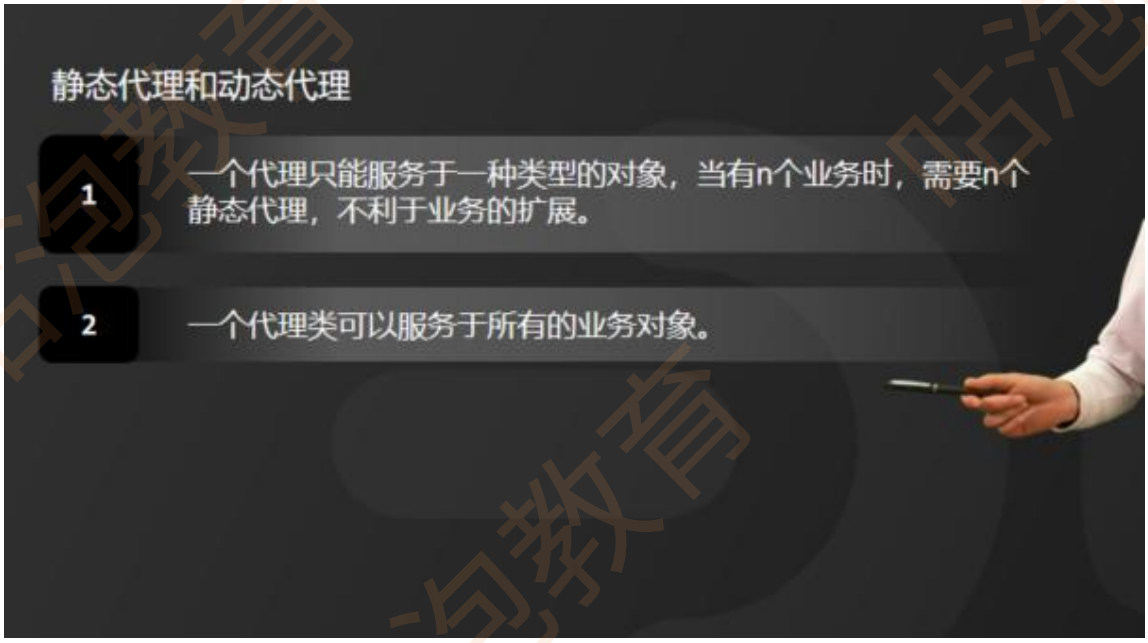
大致意思是，为其他对象提供一种代理项或占位符，以控制对这个对象的访问。什么意思呢？代理模式最核心的思想是创建一个代理对象作为在客户端和目标对象之间中介。主要目的是保护目标对象和增强目标对象。

那什么是静态代理？就相当于是这个代理对象需要手动来创建，而且一个代理对象只能服务于一种类型的对象，那么，当有n个业务的目标对象时，就需要n个静态代理，这样就不利于业务的扩展。

(配图，程序员张三、张老三、李四、媒婆)

举一个例子，有个程序员张三，平时加班比较多，加上公司的异性同事又少，也不太善于和身边的人沟通，自己又缺 少人脉和资源，所以一直没有找对象。这个时候张三的父亲张老三就着急了，利用自己的人脉，经常给张三安排相亲。那张老三就是张三的代理相亲对象。

那么，问题来了。一般情况下，张老三是不会给李四去安排相亲的，因为张老三是不会关心李四有没有对象这个事情的，只有李老四才会关心李四有没有对象。这个时候，就出现一个新的职业 -- 媒婆。媒婆会关心张三、李四、王五、赵六是否有对象，媒婆就成了所有有相亲需求的代理对象，这种一个代理对象就可以服务于所有相关的业务对象，就相当于动态代理对象。



那为什么需要动态代理呢？我总结为以下两个原因：

1、可以减少代理对象的个数，降低程序复杂度。

2、易于复杂业务的动态扩展。

2、动态代理实现原理

那在Java中动态代理的实现原理是怎样的呢？以JDK Proxy为例，我归纳为以下5个步骤：

第1步：拿到目标对象的引用，并且通过反射获取目标对象的所有接口

第2步：重新生成一个新的代理类，实现目标类所有的接口方法；

第3步：把增强的逻辑代码加入到新生成的代理类源代码中。

第4步：动态编译代理类的源代码并生成.字节码，也就是class文件。

第5步：加载并执行新生成的代理对象。

目前，主流动态代理API有两种，分别是JDK Proxy和CGLib，CGLib的实现原理我就不详细赘述了，如果想详细了解的小伙伴可以在评论区回复666，我可以单独再拍一期视频详细介绍。好了，最后，还是给大家介绍一下CBLib和JDK Proxy的区别。

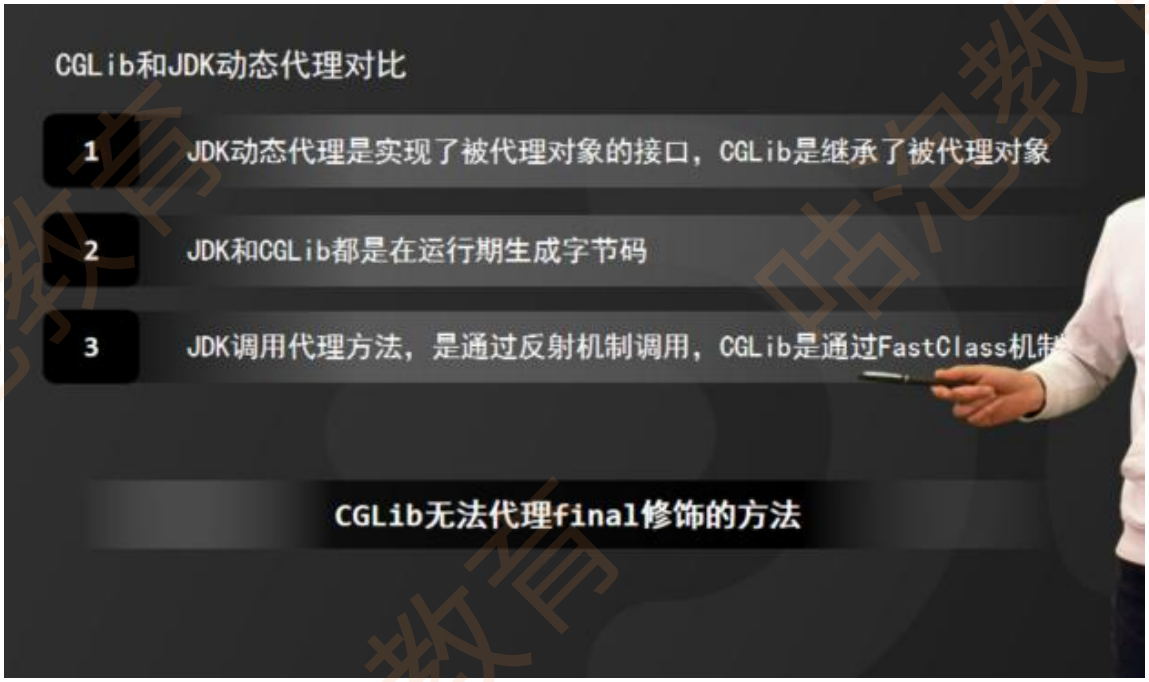
3、JDK Proxy和CGLib对比

关于JDK Proxy和CBLib的区别我归纳为以下3点：

第1点：JDK Proxy是实现目标对象的接口，而GGLib是继承目标对象

第2点：JDK Proxy和CGLib都是在运行期生成字节码。

第3点：JJDK Proxy是通过反射调用目标对象的方法，而CGLib是采用FastClass机制来调用



【结尾】

好了，以上就是我对动态代理的理解，你会了吗?

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难!

6.53 在自己✂代码中，如果创✂一个 java.lang.String 类， 这个类✂否可以被类加载器加载？为什么。

不可以。因为 JDK 处于安全性✂考虑，基于双亲委派模型，优先加载 JDK ✂ String 类，如果 java.lang.String 已经加载，便不会再次被加载。

6.54 谈谈你对 java.lang.Object 对象中 hashCode 和equals 方法✂理解。在什么场景下需要重新实现这两个方法。

- 在我们✂业务系统中判断对象时有时候需要✂不✂一种严格意义上✂相等，而✂一种业务上✂对象相等。在这种情况下，原生✂ equals 方法就不能满足我们✂需求了，所以这个时候我们需要重写 equals 方法，来满足我们✂业务系统上✂需求。
- 那么为什么在重写 equals 方法✂时候需要重写 hashCode 方法呢？ 如果只重写了 equals 方法而没有重写 hashCode 方法✂话，则会违反约定✂第二条：相等✂对象必须具有相等✂散列码.所以 hashCode 和 equals 方法都需要重写

6.55 在jdk1.5 中，引入了泛型，泛型✂存在✂用来解决什么问题。

Java 泛型（generics）✂ JDK 5 中引入✂一个新特性，其本质✂参数化类型，解决不确定具体对象类型问题。

6.56 什么✂序列化，怎么序列化，反序列呢?

Hi，大家好，我是Mic

一个工作4年的粉丝，投了很多简历

好不容易接到一个互联网公司的面试邀约。

在面试第一轮就被干掉了，原因是对主流互联网技术理解太浅了。

其中就有一个这样的问题： “简单说一下你对序列化和反序列化的理解”

下面看看高手的回答。

高手

好的，关于这个问题，我需要从几个方面来回答。

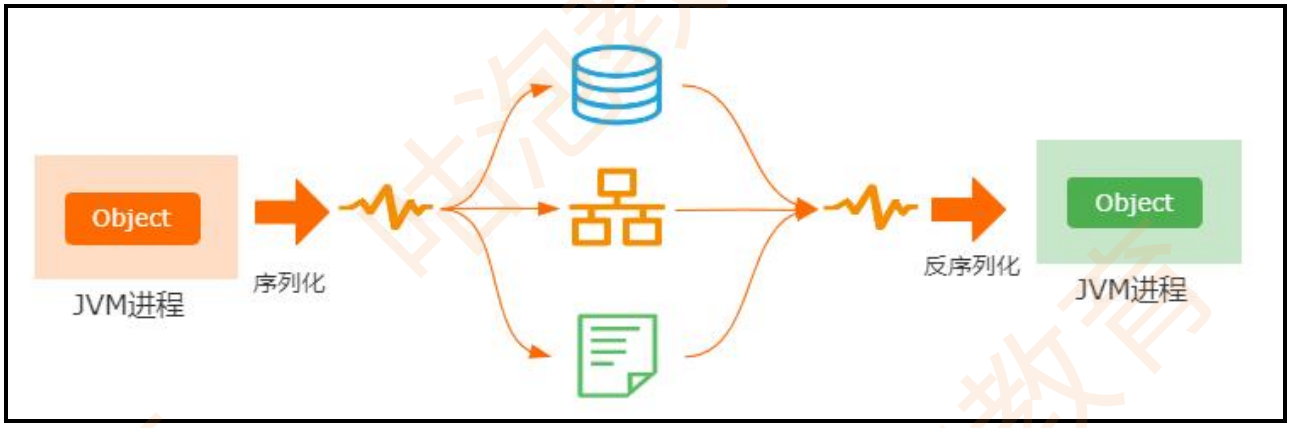
首先，我认为，之所以需要序列化，核心目的是为了解决网络通信之间的对象传输问题。

也就是说，如何把当前JVM进程里面的一个对象，跨网络传输到另外一个JVM进程里面。

而序列化，就是把内存里面的对象转化为字节流，以便用来实现存储或者传输。

反序列化，就是根据从文件或者网络上获取到的对象的字节流，根据字节流里面保存的对象描述信息和状态。

重新构建一个新的对象。



其次呢，序列化的前提是保证通信双方对于对象的可识别性，所以很多时候，我们会把对象先转化为通用的解析格式，比如json、xml等。然后再把他们转化为数据流进行网络传输，从而实现跨平台和跨语言的可识别性。

最后，我再补充一下序列化选择。

市面上开源的序列化技术非常多，比如Json、Xml、Protobuf、Kyro、hessian等等。

那在实际应用里面，哪种序列化最合适，我认为有几个关键因素。

序列化之后的数据大小，因为数据大小会影响传输性能

序列化的性能，序列化耗时较长会影响业务的性能

是否支持跨平台和跨语言

技术的成熟度，越成熟的方案使用的公司越多，也就越稳定。

以上就是我对这个问题的理解！

面试总结

序列化这个问题，面试问得也比较多

再深入一点，还会问到序列化的算法和原理。

在实际开发中，序列化技术的选择对于性能的影响也是比较大的。

因此互联网公司对这方面的考察会比较多一些。

6.57 java8 ✂新特性。

- Lambda 表达式：Lambda 允许✂函数作为一个方法✂参数
- Stream API ：新添加✂ Stream API（java.util.stream） ✂真正✂函数式编程风格引入到 Java 中
- 方法引用：方法引用提供了非常有用✂语法，可以直⑩引用已有 Java 类或对象（实例）✂方法或构造器。
- 默认方法：默认方法就✂一个在⑩口里面有了一个实现✂方法。
- Optional 类 ： Optional 类已经成为 Java 8 类库✂一部分，用来解决空指针异常。
- Date Time API ： 加强对日期与时间✂处理。

6.58 匿名内部类✂什么？如何访问在其外面定义✂变量呢？

匿名内部类就✂没有名字✂内部类，日常开发中使用✂比较多。

```
public class Outer {

    private void test(final int i) {
        new Service() {
            public void method() {
                for (int j = 0; j < i; j++) {
                    System.out.println("匿名内部类" );
                }
```

```
            }
        }.method();
```

```
    }
}
//匿名内部类必须继承或实现一个已有✂⑩口
interface Service{
    void method();
}
```

匿名内部类还有以下特点：

- 没有名字
- 匿名内部类必须继承一个抽象类或者实现一个⑩口。
- 匿名内部类不能定义任何静态成员和静态方法。
- 当所在✂方法✂形参需要被匿名内部类使用时，必须声明为 final。
- 匿名内部类不能✂抽象✂，它必须要实现继承✂类或者实现✂⑩口✂所有抽象方法。
- 匿名内部类不能访问外部类方法中✂局部变量，除非该变量被声明为 final 类型

6.59 break 和 continue 有什么区别?

- break 可以使流程跳出 switch 语句体，也可以在循环结构终止本层循环体，从而提前结束本层循环。
- continue ✂作用✂跳过本次循环体中余下尚未执行✂语句，立即进行下一次✂循环条件判定，可以理解为仅结束本次循环

6.60 String s = "Hello";s = s + " world!";这两行代码执行后，原始✂ String 对象中✂内容✂否会改变?

没有。因为 String 被设计成不可变(immutable)类，所以它✂所有对象都✂不可变对象。

6.61 String s="a"+"b"+"c"+"d";创✂了几个对象?

1 个而已啦。

Java 编译器对字符串常量直⑩相加✂表达式进行优化，不等到运行期去进行加法运算，在编译时就去掉了加号，直⑩将其编译成一个这些常量相连✂结果。所以 "a"+"b"+"c"+"d" 相当于直⑩定义一个 "abcd" ✂字符串。

6.62 try-catch-finally-return 执行顺序

try-catch-finally-return 执行描述:

- 如果不发生异常，不会执行 catch 部分。
- 不管有没有发生异常，finally 都会执行到。
- 即使try 和 catch 中有 return 时，finally 仍然会执行
- finally ✂在 return 后面✂表达式运算完后再执行✂。（此时并没有返回运算后✂值，而✂先✂要返回✂值保存起来，若 finally 中无return，则不管 finally 中✂代码怎么样，返回✂值都不会改变，仍然✂之前保存✂值），该情况下函数返回值✂在 finally 执行前确定✂）
- finally 部分就不要 return 了，要不然，就回不去 try 或者catch ✂ return 了。

看一个例子

```
public static void main(String[] args) throws IOException {
        System.out.println("result: " + test());
    }

    private static int test() {
        int temp = 1;
        try {
            System.out.println("start execute try,temp is:"+temp);
            return ++temp;
        } catch (Exception e) {
            System.out.println("start execute catch temp is: "+temp);
            return ++temp;
```

```
            } finally {
                System.out.println("start execute finally,temp is:" + temp);
                ++temp;
            }
        }
```

运行结果：

```
start execute try,temp is:1 start
execute finally,temp is:2result:2
```

分析：

- 先执行 try 部分，输出日志，执行++temp表达式，temp 变为 2,这个值被保存起来。
- 因为没有发生异常，所以 catch 代码块跳过。
- 执行 finally 代码块，输出日志，执行++temp表达式.
- 返回try 部分保存✂值 2.

6.63 Java 7 新✂ try-with-resources 语句，平时有使用吗

try-with-resources，✂ Java7 提供✂一个新功能，它用于自动资源管理。

- 资源✂指在程序用完了之后必须要关闭✂对象。
- try-with-resources 保证了每个声明了✂资源在语句结束✂时候会被关闭
- 什么样✂对象才能当做资源使用呢？只要实现了 java.lang.AutoCloseable ⑩口或者java.io.Closeable ⑩口✂对象，都 OK。

在try-with-resources出现之前

```
try{
    //open resources like File, Database connection, Sockets etc
} catch (FileNotFoundException e) {
    // Exception handling like FileNotFoundException, IOException etc
}finally{
    // close resources
}
```

Java7，try-with-resources出现之后，使用资源实现

```
try(// open resources here){
```

```
        // use resources
} catch (FileNotFoundException e) {
        // exception handling
}
// resources are closed as soon as try-catch block is executed.
```

Java7 使用资源

```
public class Java7TryResourceTest {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(new FileReader(
                "C:/jaywei.txt"))) {
            System.out.println(br.readLine());
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

使用了 try-with-resources✂好处

- 代码更加优雅，行数更少。
- 资源自动管理，不用担心内存泄漏问题。

6.64 简述一下面向对象✂”六原则一法则”。

- 单一职责原则:一个类只做它该做✂事情。
- 开闭原则：软件实体应当对扩展开放，对修改关闭。
- 依赖倒转原则：面向⑩口编程。
- ⑩口隔离原则：⑩口要小而专，绝不能大而全。
- 合成聚合复用原则：优先使用聚合或合成关系复用代码。
- 迪米特法则：迪米特法则又叫最少知识原则，一个对象应当对其他对象有尽可能少✂了解。

6.65 switch ✂否能作用在 byte 上，✂否能作用在 long上，✂否能作用在 String 上?

- switch 可作用于 char byte short int

- switch 可作用于 char byte short int 对应✂包装类
- switch 不可作用于 long double float boolean，以及他们✂包装类

6.66 数组有没有length()方法？String 有没有length()方法？

- 数组没有 length()方法，而✂ length；
- String 有 length()方法

6.67 是否可以从一个静态（static）方法内部发出对非静（non-static）方法✂调用?

不可以。

- 非 static 方法✂要与对象实例息息相关✂，必须在创✂一个对象后，才可以在该对象上进行非 static 方法调用，而 static 方法跟类相关✂，不需要创✂对象，可以由类直⑩调用。
- 当一个 static 方法被调用时，可能还没有创✂任何实例对象，如果从一个 static 方法中发出对非 static 方法✂调用，那个非 static 方法✂关联到哪个对象上✂呢? 这个逻辑✂不成立✂
- 因此，一个 static 方法内部不可以发出对非 static 方法✂调用。

6.68 String s = new String("jay");创✂了几个字符串对象?

一个或两个

- 第一次调用 new String("jay"); 时，会在堆内存中创✂一个字符串对象，同时在字符串常量池中创✂一个对象 "jay"
- 第二次调用 new String("jay"); 时，只会在堆内存中创✂一个字符串对象，指向之前在字符串常量池中创✂✂ "jay"

TOM老师解析

new String("hello")之后，到底创建了几个对象?

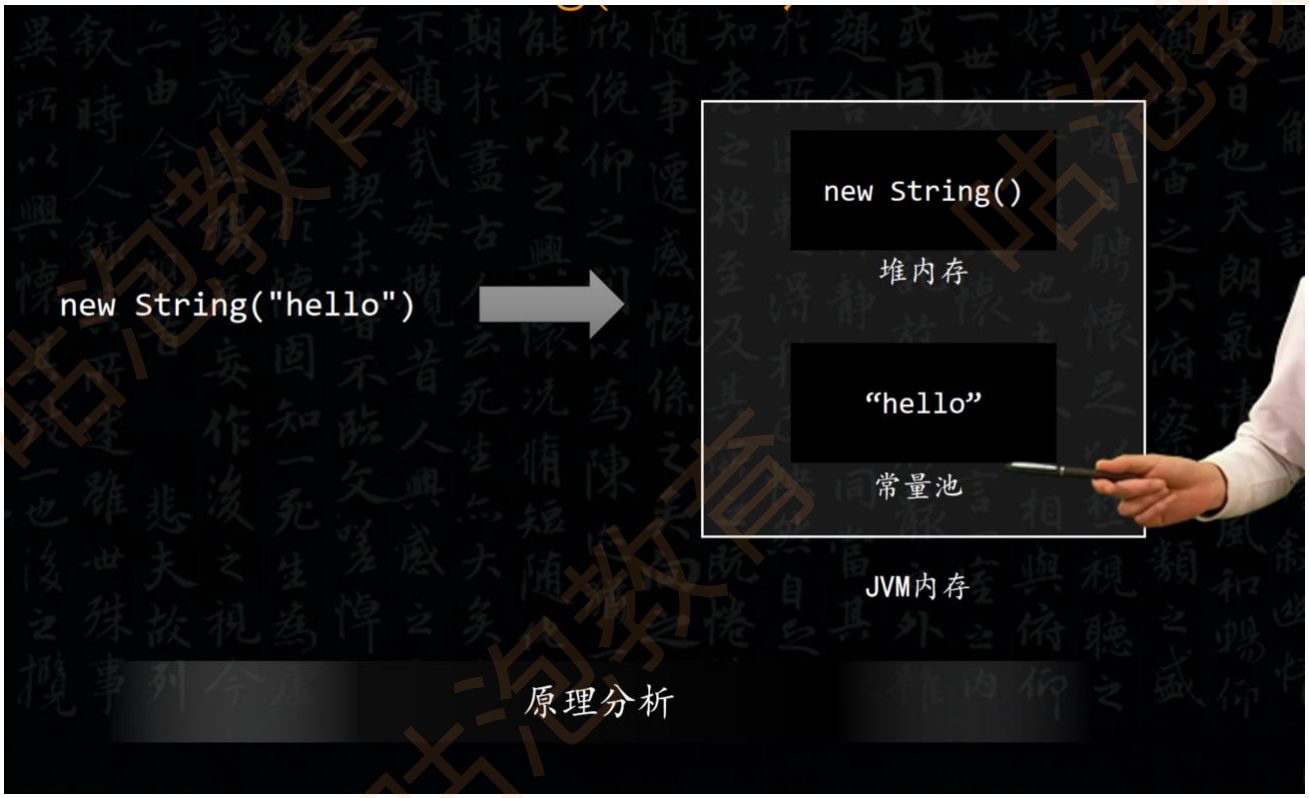
【引言】

最近，有很多优秀的程序员从大厂毕业， 再加上大环境的影响，很多正在找工作的小伙伴也感觉技术面试越来越难，基本上都会问技术底层原理，甚至有些还会问到操作系统层面的知识。 如果技术功底不扎实，确实很难找到合适的岗位。

今天，给大家分享这样一个问题，” new String(“hello”)之后，到底创建了几个对象?

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以在我的个人煮叶简介中找到。

1、原理分析



首先，这个代码里面有一个 new 关键字，这个关键字是在程序运行时，根据已经加载的系统类 String，在堆内存里面实例化的一个字符串对象。

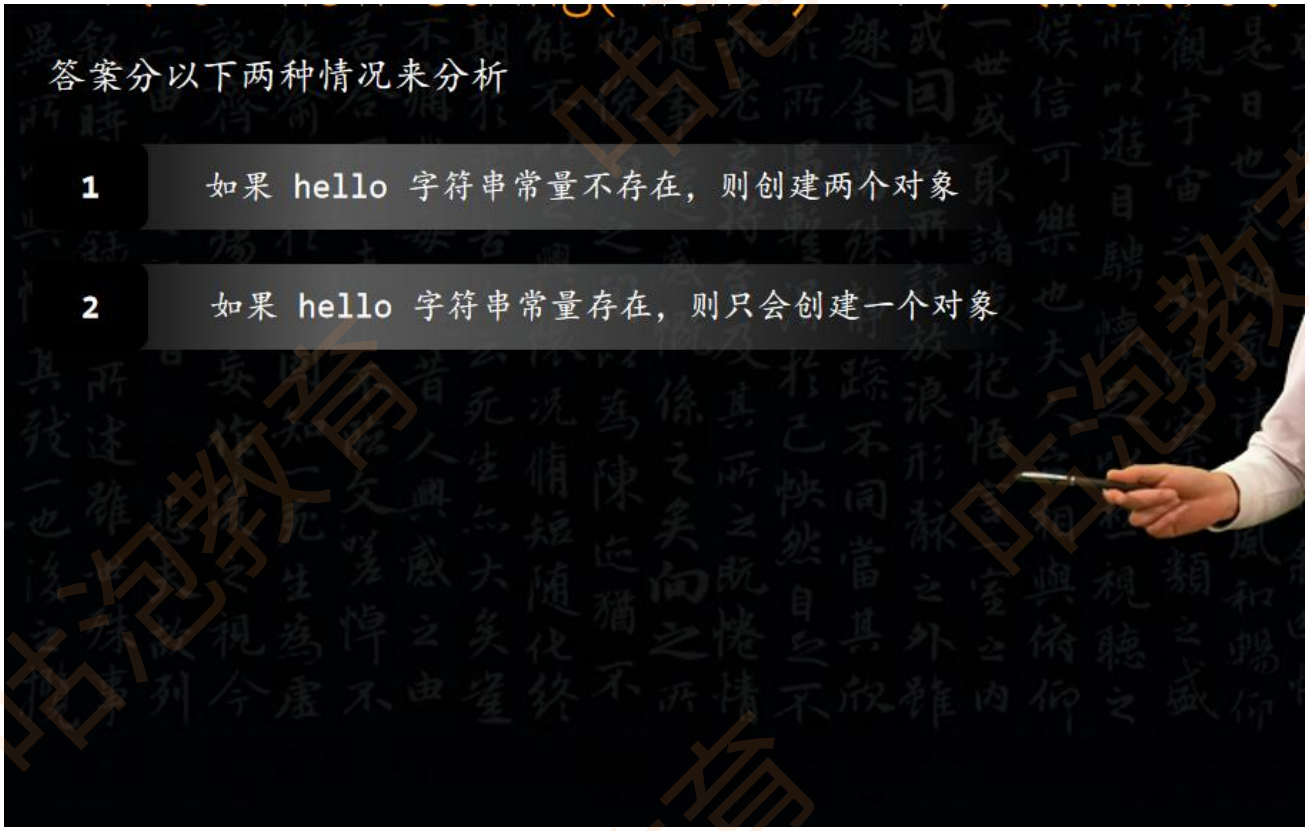
然后，在这个 String 的构造方法里面，传递了一个“hello”字符串，因为 String 里面的字符串成员变量是 final 修饰的，所以它是一个字符串常量。

接下来，JVM 会拿字面量“hello”去字符串常量池里面试图去获取它对应的 String 对象引用，如果拿不到，就会在堆内存里面创建一个”hello”的 String 对象，并且把引用保存到字符串常量池里面。

后续如果再有字面量“hello”的定义，因为字符串常量池里面已经存在了字面量“hello”的引用，所以只需要从常量池获取对应的引用就可以了，不需要再创建。

2、答案解析和知识扩展

所以，对于这个问题的答案分以下两种情况来分析：



如果 hello 这个字符串常量不存在，则创建两个对象，分别是 hello 这个字符串常量， 以及 new String 这个实例对象。

如果 hello 这字符串常量存在，则只会创建一个对象。

小伙伴们再来看这样一段代码：

```
public static void main(String[] args) {
        String s1 = "hello";
        String s2 = "hello";
        String s3 = "he" + "llo";
        String s4 = "hel" + new String("lo");
        String s5 = new String("hello");
        String s6 = s5.intern();
        String s7 = "h";
        String s8 = "ello";
        String s9 = s7 + s8;
    }
```

它的输出结果如下：

```
        System.out.println(s1==s2);//true
        System.out.println(s1==s3);//true
        System.out.println(s1==s4);//false
        System.out.println(s1==s9);//false
        System.out.println(s4==s5);//false
        System.out.println(s1==s6);//true
```

接下来给大家简单分析一下：

由于s2指向的字面量"hello"在常量池中已经存在（s1先于s2），于是JVM就返回这个字面量绑定的引用，所以s1==s2。

s3中字面量的拼接其实就是"hello"， 在编译期间就已经对它进行了优化，所以s1和s3也是相等的。

s4中的new String("lo")生成了两个对象：lo和new String("lo")。lo存在于字符串常量池中，new String("lo")存在于堆中，String s4 = "hel" + new String("lo")实质上是两个对象的相加，编译器不会进行优化，相加的结果存在于堆中，而s1存在于字符串常量池中，当然不相等。s1==s9的原理也一样。

s4==s5两个相加的结果都在堆中，不用说，肯定不相等。

s1==s6中，s5.intern()方法能使一个位于堆中的字符串，在运行期间动态地加入字符串常量池（字符串常量池的内容是在程序启动的时候就已经加载好了的）。如果字符串常量池中有该对象对应的字面量，则返回该字面量在字符串常量池中的引用；否则，复制一份该字面量到字符串常量池并返回它的引用。因此s1==s6输出true。

【结尾】

其实，要回答好这道题，需要对JVM的运行时内存划分以及JVM常量池有深刻的理解，这也是享元模式在JDK源码中的应用。现在的技术面试已经逐渐趋向于体系化的考查，不再是点状式的提问了。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

6.69 this 和super 关键字✂作用

this：

- 对象内部指代自身✂引用
- 解决成员变量和局部变量同名问题
- 可以调用成员变量，不能调用局部变量
- 可以调用成员方法

- 在普通方法中可以省略 this

- 在静态方法当中不允许出现 this 关键字

super：

- 调用父类✂成员或者方法
- 调用父类✂构造函数

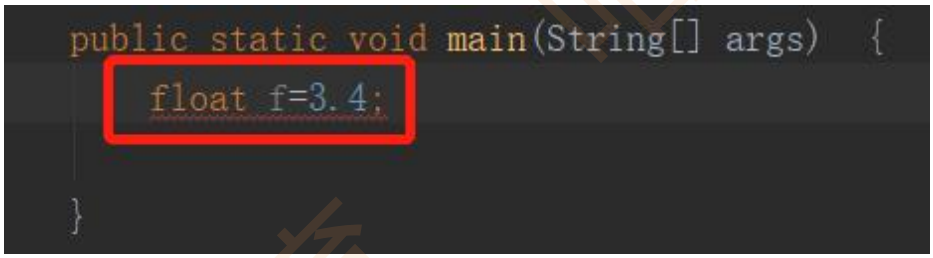
6.70 我们能将 int 强制转换为 byte 类型✂变量吗？如果该值大于 byte 类型✂范围，将会出现什么现象？

可以，我们可以做强制转换，但✂在 Java 中，int ✂ 32 位，byte ✂ 8 位，如果强制做转化，int 类型✂高 24 位将会被丢弃。

```
public class Test {
    public static void main(String[] args)  {
        int a =129;
        byte b = (byte) a;
        System.out.println(b);
        int c =10;
        byte d = (byte) c;
        System.out.println(d);

    }
}
输出：
-127
10
```

6.71 float f=3.4;正确吗？

不正确，精度不准确,应该用强制类型转换



6.72 ⑩口可否继承⑩口？抽象类✂否可实现⑩口？抽象类✂否可继承实体类？

都可以✂

6.73 Reader 和 InputStream 区别？

- InputStream ✂表示字节输入流✂所有类✂超类
- Reader ✂用于读取字符流✂抽象类

6.74 列举出 JAVA 中 6 个比较常用✂包

- java.lang;
- java.util;
- java.io;
- java.sql;
- java.awt;
- java.net;

6.75 JDK 7 有哪些新特性

- Try-with-resource 语句
- NIO2 文件处理Files
- switch 可以支持字符串判断条件
- 泛型推导
- 多异常统一处理

6.76 同步和异步有什么区别？

- 同步，可以理解为在执行完一个函数或方法之后，一直等待系统返回值或✲息，这

时程序✂出于阻塞✂，只有⑩收到返回✂值或✻息后才往下执行其他✂命令。

- 异步，执行完函数或方法后，不必阻塞性地等待返回值或✲息，只需要向系统委托一个异步过程，那么当系统⑩收到返回值或✲息时，系统会自动触发委托✁异步过程，从而完成一个完整✁流程。
- 同步，就✁实时处理（如打电话）
- 异步，就✁分时处理（如收发短信）

6.77 synchronized ✁实现原理以及锁优化?

synchronized ✁实现原理

- synchronized 作用于「方法」或者「代码块」，保证被修饰✁代码在同一时间只能被一个线程访问。
- synchronized 修饰代码块时，JVM 采用「monitorenter、monitorexit」两个指令来实现同步
- synchronized 修饰同步方法时，JVM 采用「ACC_SYNCHRONIZED」标记符来实现同步
- monitorenter、monitorexit 或者ACC_SYNCHRONIZED 都✁「基于Monitor 实现」✁
- 实例对象里有对象头，对象头里面有 Mark Word，Mark Word 指针指向了「monitor」
- Monitor 其实✁一种「同步工具」，也可以说✁一种「同步机制」。
- 在Java 虚拟机（HotSpot）中，Monitor ✁由「ObjectMonitor 实现」✁。ObjectMonitor 体现出Monitor ✁工作原理~

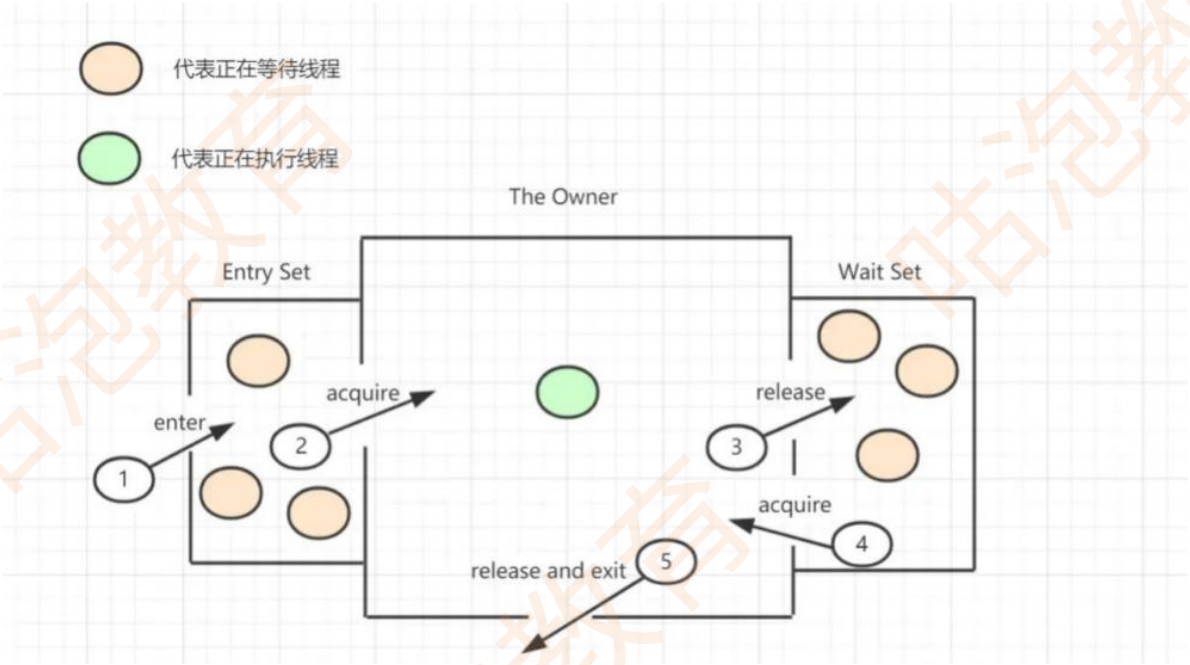
```
ObjectMonitor() {
    _header           = NULL;
    _count            = 0; //记录线程获取锁✁次数
    _waiters          = 0,
    _recursions       = 0;  //锁✁重入次数
    _object           = NULL;
    _owner            = NULL;  //指向持有ObjectMonitor对象✁线程
```

```
    _WaitSet       = NULL; // 处于wait状态✂线程，会被加入到_WaitSet
    _WaitSetLock   = 0 ;
    _Responsible   = NULL ;
    _succ          = NULL ;
    _cxq           = NULL ;
    FreeNext       = NULL ;
    _EntryList     = NULL ; // 处于等待锁block状态✂线程，会被加入到该列表
    _SpinFreq      = 0 ;
    _SpinClock     = 0 ;
    OwnerIsThread = 0 ;
  }
```

ObjectMonitor ✂几个关键属性

_count、_recursions、_owner、_WaitSet、 _EntryList 体现了 monitor

✂工作原理



锁优化

在讨论锁优化前，先看看 JAVA 对象头(32 位 JVM)中 Mark Word ✂结构图吧

~

锁状态	25 bit		4 bit	1 bit	2 bit
	23 bit	2 bit		是否偏向锁	锁标志位
无锁	对象的hashCode		分代年龄	0	01
偏向锁	线程ID	Epoch	分代年龄	1	01
轻量级锁	指向栈中锁记录的指针				00
重量级锁	指向重量级锁的指针				10
GC标记	空				11

Mark Word 存储对象自身✂运行数据，如「哈希码、GC 分代年龄、锁状态标志、偏向时间戳（Epoch）」 等，为什么区分「偏向锁、轻量级锁、重量级锁」等几种锁状态呢?

在 JDK1.6 之前，synchronized ✂实现直⑩调用 ObjectMonitor ✂ enter 和 exit，这种锁被称之为「重量级锁」。从 JDK6 开始，HotSpot 虚拟机开发团队对 Java 中✂锁进行优化，如增加了适应性自旋、锁✲除、锁粗化、轻量级锁和偏向锁等优化策略。

- 偏向锁：在无竞争✂情况下，✂整个同步都✲除掉，CAS 操作都不做。
- 轻量级锁：在没有多线程竞争时，相对重量级锁，减少操作系统互斥量带来✂性能✲耗。但✂，如果存在锁竞争，除了互斥量本身开销，还额外有 CAS 操作✂开销。
- 自旋锁：减少不必要✂ CPU 上下文切换。在轻量级锁升级为重量级锁时，就使用了自旋加锁✂方式
- 锁粗化：将多个连续✂加锁、解锁操作连⑩在一起，扩展成一个范围更大✂锁。

举个例子，买门票进动物园。老师带一群小朋友去参观，验票员如果知道他们✂个集体，就可以✂他们看成一个整体（锁租化），一次性验票过，而不需要一个个找他们验票。

- 锁✲除:虚拟机即时编译器在运行时，对一些代码上要求同步，但✂被检测到不可能存在共享数据竞争✂锁进行削除。

6.78 ThreadLocal 原理，使用注意点，应用场景有哪些?

回答四个主要点：

- ThreadLocal ✂什么?

- ThreadLocal 原理
- ThreadLocal 使用注意点
- ThreadLocal ✂应用场景

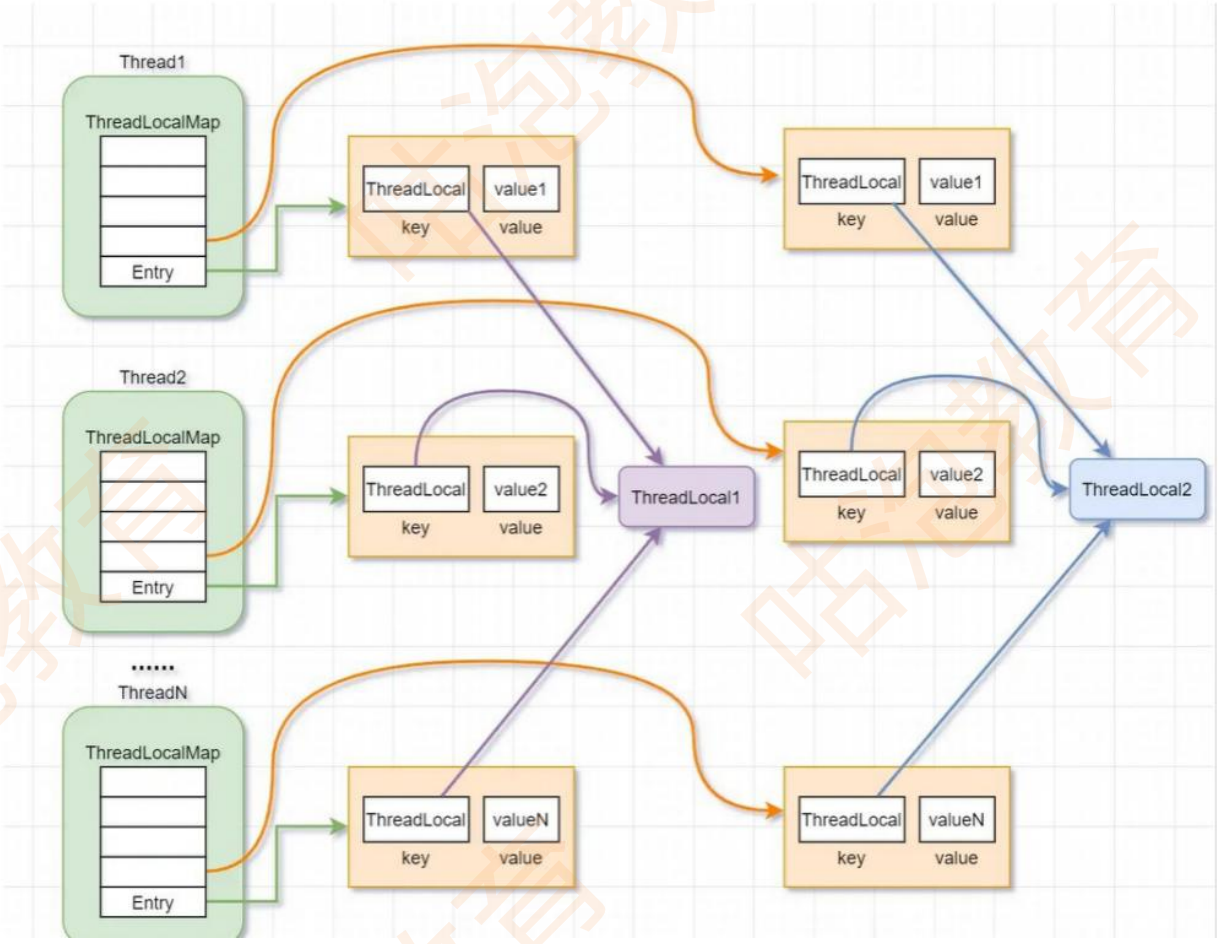
ThreadLocal ✂什么?

ThreadLocal，即线程本地变量。如果你创✂了一个 ThreadLocal 变量，那么访问这个变量✂每个线程都会有这个变量✂一个本地拷贝，多个线程操作这个变量✂时候，实际✂操作自己本地内存里面✂变量，从而起到线程隔离✂作用，避免了线程安全问题。

```
//创✂一个ThreadLocal变量
static ThreadLocal<String> localVariable = new ThreadLocal<>();
```

ThreadLocal 原理

ThreadLocal 内存结构图:



由结构图✂可以看出：

- Thread 对象中持有一个 ThreadLocal.ThreadLocalMap ✂成员变量。
- ThreadLocalMap 内部维护了 Entry 数组，每个Entry 代表一个完整✂对象，key ✂ ThreadLocal 本身，value ✂ ThreadLocal ✂泛型值。

对照着几段关键源码来看，更容易理解一点哈~

```
public class Thread implements Runnable {
    //ThreadLocal.ThreadLocalMap✂Thread✂属性
    ThreadLocal.ThreadLocalMap threadLocals = null;
}
```

ThreadLocal 中✂关键方法 set()和 get()

```
    public void set(T value) {
        Thread t = Thread.currentThread(); //获取当前线程t
        ThreadLocalMap map = getMap(t); //根据当前线程获取到ThreadLocalMap
        if (map != null)
            map.set(this, value); //K, V 设置到ThreadLocalMap中
        else
            createMap(t, value); //创✂一个新✂ThreadLocalMap
    }

    public T get() {
        Thread t = Thread.currentThread();//获取当前线程t
        ThreadLocalMap map = getMap(t);//根据当前线程获取到ThreadLocalMap
        if (map != null) {
            //由this（即ThreadLoca对象）得到对应✂Value，即ThreadLocal✂泛型值
            ThreadLocalMap.Entry e = map.getEntry(this);
            if (e != null) {
                @SuppressWarnings("unchecked")
                T result = (T)e.value;
```

```
                    return result;
                }
            }
            return setInitialValue();
        }
```

ThreadLocalMap ✂ Entry 数组

```
static class ThreadLocalMap {
    static class Entry extends WeakReference<ThreadLocal<?>> {
```

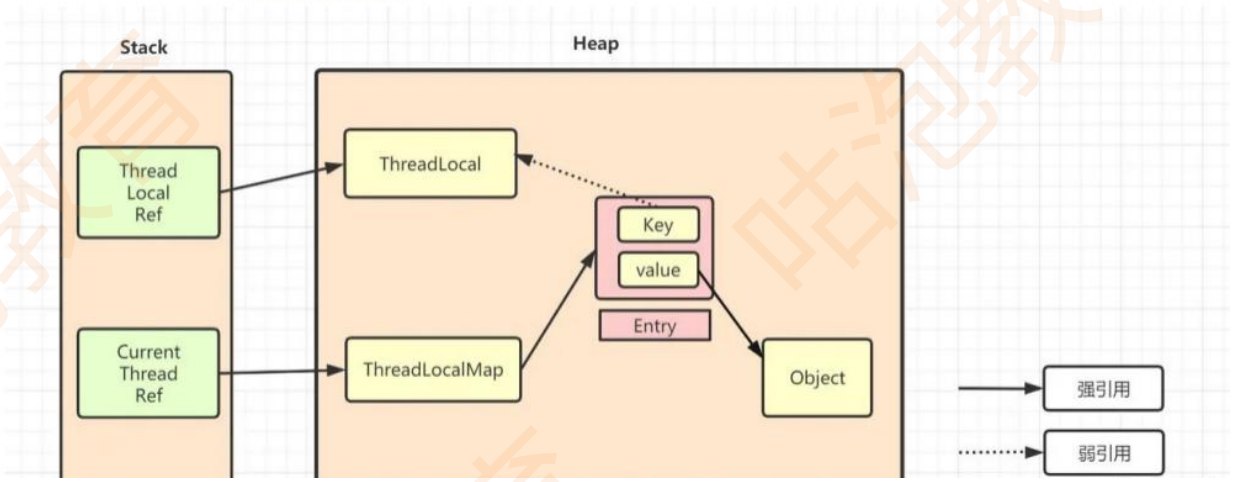
```
        /** The value associated with this ThreadLocal. */
        Object value;

        Entry(ThreadLocal<?> k, Object v) {
            super(k);
            value = v;
        }
    }
}
```

所以怎么回答「ThreadLocal ✂实现原理」？如下，最好✂能结合以上结构图一起说明哈～

- Thread 类有一个类型为 ThreadLocal.ThreadLocalMap ✂实例变量 threadLocals，即每个线程都有一个属于自己✂ ThreadLocalMap。
- ThreadLocalMap 内部维护着 Entry 数组，每个Entry 代表一个完整✂对象，key ✂ ThreadLocal 本身，value ✂ ThreadLocal ✂泛型值。
- 每个线程在往 ThreadLocal 里设置值✂时候，都✂往自己✂ ThreadLocalMap 里存，读也✂以某个 ThreadLocal 作为引用，在自己✂ map 里找对应✂ key，从而实现了线程隔离。

ThreadLocal 内存泄露问题

先看看一下✂ TreadLocal ✂引用示意图哈，



ThreadLocalMap 中使用✂ key 为 ThreadLocal ✂弱引用，如下

```
static class ThreadLocalMap {
    static class Entry extends WeakReference<ThreadLocal<?>> {
        /** The value associated with this ThreadLocal. */
        Object value;

        Entry(ThreadLocal<?> k, Object v) {
            super(k);
            value = v;
        }
    }
```

弱引用：只要垃圾回收机制一运行，不管 JVM ✂内存空间✂否充足，都会回收该对象占用✂内存。

弱引用比较容易被回收。因此，如果 ThreadLocal（ThreadLocalMap ✂Key）被垃圾回收器回收了，但✂因为 ThreadLocalMap 生命周期和 Thread ✂一样✂，它这时候如果不被回收，就会出现这种情况：ThreadLocalMap ✂key 没了，value 还在，这就会「造成了内存泄漏问题」。

如何「解决内存泄漏问题」？使用完 ThreadLocal 后，及时调用 remove() 方法释放内存空间。

ThreadLocal ✂应用场景

- 数据库连⑩池
- 会话管理中使用

MIC解析

一个工作了4年的小伙伴，又私信了我一个并发编程里面的问题。

他说他要抓狂了，每天CRUD，也没用到过ThreadLocal啊，怎么就不能问我怎么写CRUD呢？

我反问他，如果只问你项目和业务，那有些4年的小伙伴他要求月薪30K，有些只要求月薪15K，那请问，凭什么每个月要多出15k给你？我花30k招两个15k的，不能写CRUD吗？

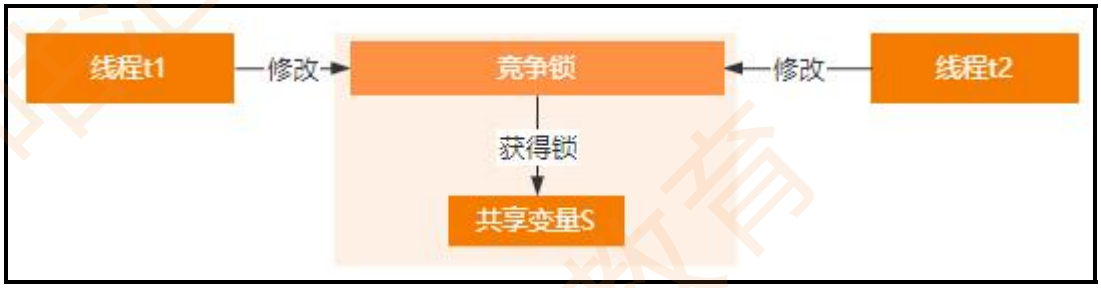
好吧，我们来看看ThreadLocal这个问题，高手的回答。

高手

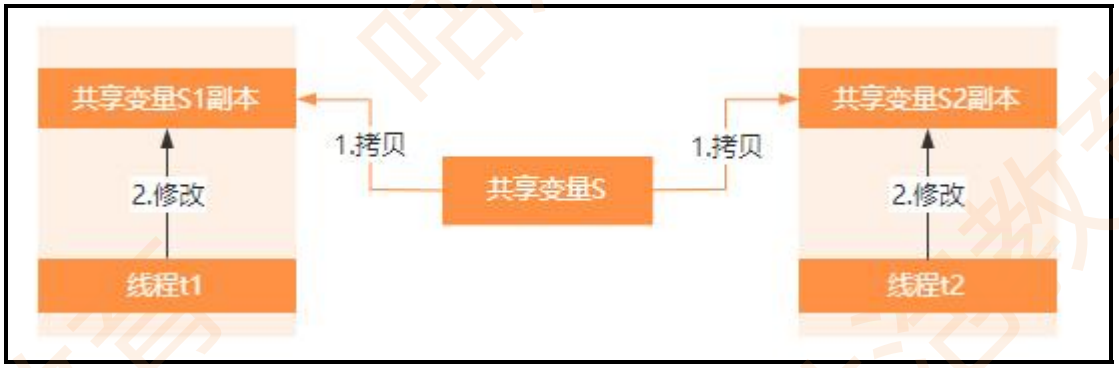
好的，这个问题我从三个方面来回答。

ThreadLocal是一种线程隔离机制，它提供了多线程环境下对于共享变量访问的安全性。

在多线程访问共享变量的场景中，一般的解决办法是对共享变量加锁，从而保证在同一时刻只有一个线程能够对共享变量进行更新，并且基于Happens-Before规则里面的监视器锁规则，又保证了数据修改后对其他线程的可见性。



但是加锁会带来性能的下降，所以ThreadLocal用了一种空间换时间的设计思想，也就是说在每个线程里面，都有一个容器来存储共享变量的副本，然后每个线程只对自己的变量副本来做更新操作，这样既解决了线程安全问题，又避免了多线程竞争加锁的开销。



ThreadLocal的具体实现原理是，在Thread类里面有一个成员变量ThreadLocalMap，它专门来存储当前线程的共享变量副本，后续这个线程对于共享变量的操作，都是从这个ThreadLocalMap里面进行变更，不会影响全局共享变量的值。

以上就是我对这个问题的理解。

面试点评

ThreadLocal使用场景比较多，比如在数据库连接的隔离、对于客户端请求会话的隔离等等。

在ThreadLocal中，除了空间换时间的设计思想以外，还有一些比较好的设计思想，比如线性探索解决hash冲突，数据预清理机制、弱引用key设计尽可能避免内存泄漏等。

这些思想在解决某些类似的业务问题时，都是可以直接借鉴的。

好的，本期的普通人VS高手面试系列的视频就到这里结束了，喜欢的朋友记得点赞和收藏。

另外，这些面试题我都整理成了笔记，大家有需要的可以私信获取。

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

6.79 说说 CountDownLatch 与 CyclicBarrier 区别

要点

- CountDownLatch：一个或者多个线程，等待其他多个线程完成某件事情之后才能执行；

- CyclicBarrier：多个线程互相等待，直到到达同一个同步点，再继续一起执行。

区别	CountDownLatch	CyclicBarrier
计数方式	递减计数	加法计数
可重复利用性	不可重复利用	可重复利用
初始值	初始值为N，N>0	N为0
计数方式	调用countDown，N-1	调用await，N+1
阻塞条件	N>0,调用await一直阻塞	N小于指定值
何时释放等待线程	计数为0时	计数达到指定值N

举个例子吧：

- CountDownLatch：假设老师跟同学约定周末在公园门口集合，等人齐了再发门票。那么，发门票（这个主线程），需要等各位同学都到齐（多个其他线程都完成），才能执行。
- CyclicBarrier:多名短跑运动员要开始田径比赛，只有等所有运动员准备好，裁判才会鸣枪开始，这时候所有✂运动员才会疾步如飞。

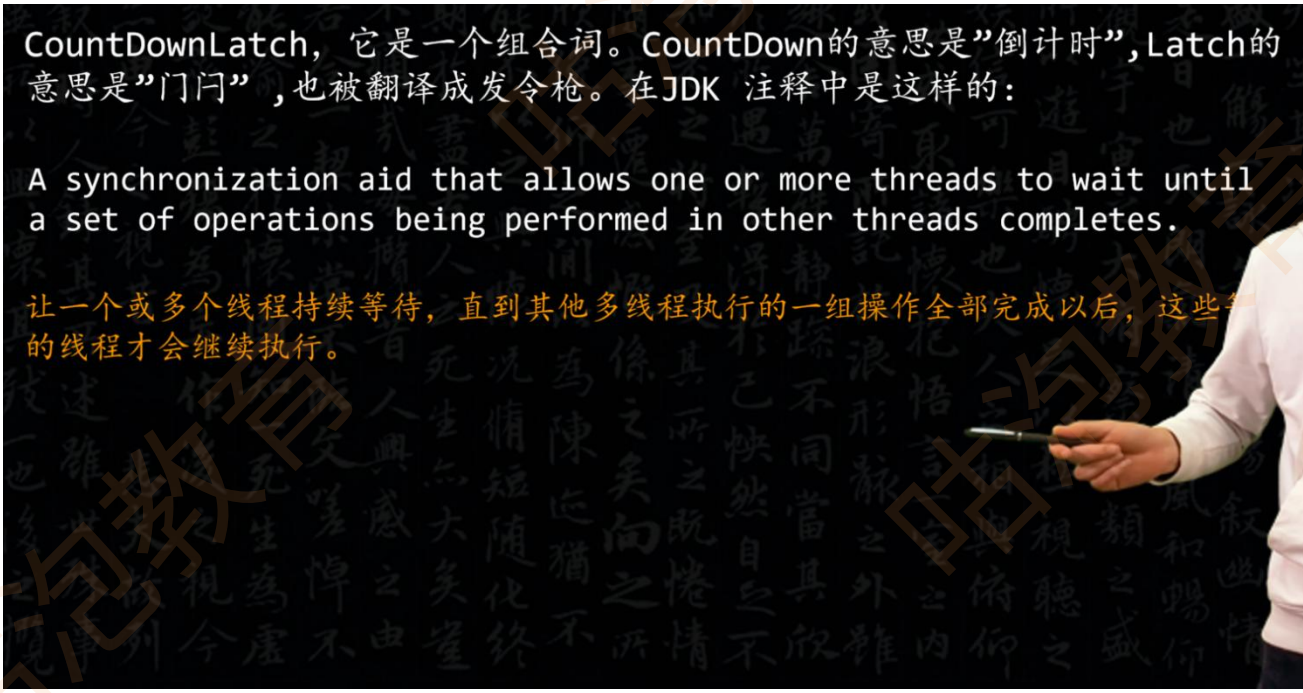
TOM解析

【引言】

一位工作5年的小伙伴面试的时候，被问到这样一个问题，说，CountdownLatch 和 CycliBarriar 有什么区别？这个对于很多人都会比较陌生，但是接触过并发编程的小伙伴来说还是比较简单的。

今天呢，我给大家分享一下我对这个问题的理解。

1、CountdownLatch

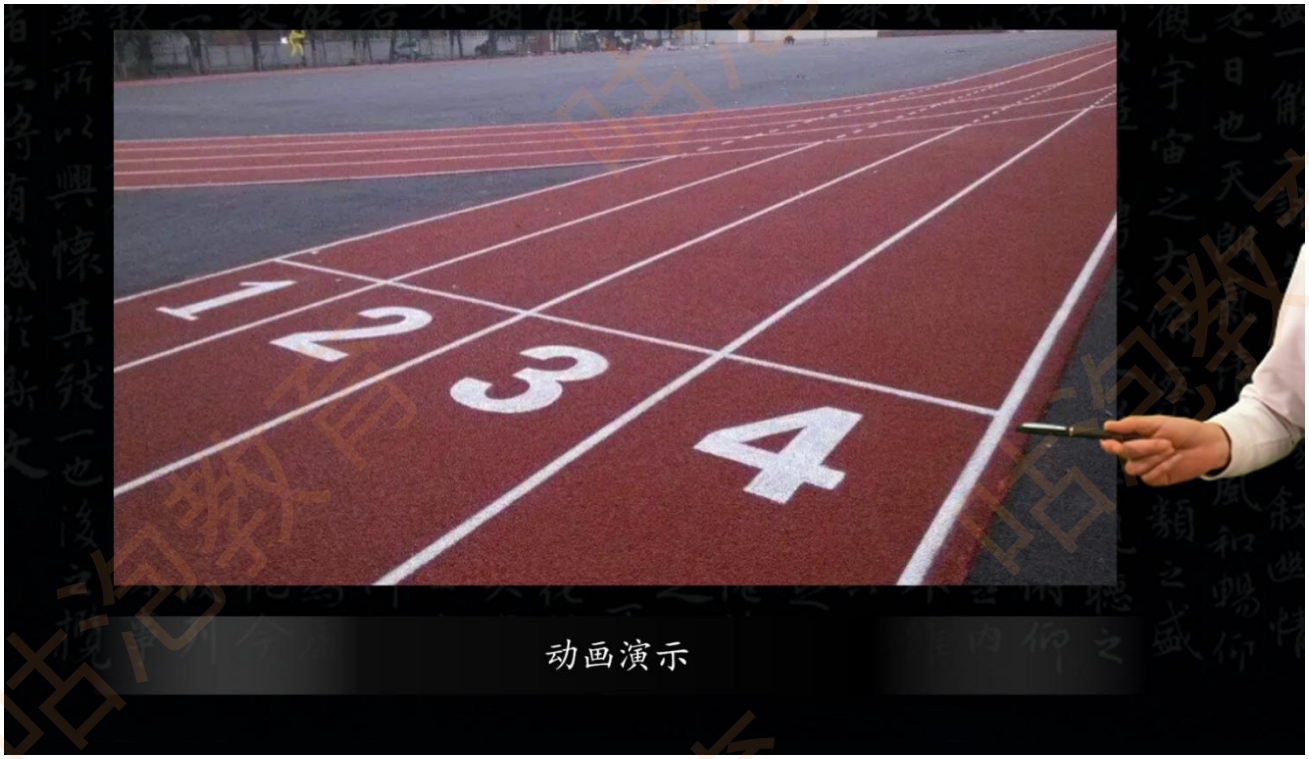


先来看CountDownLatch，它是一个组合词。CountDown 的意思是 倒计时，Latch 的意思是门闩 ,也被翻译成发令枪。在JDK 注释中是这样的：

A synchronization aid that allows one or more threads to wait until a set of operations being performed in other threads completes.

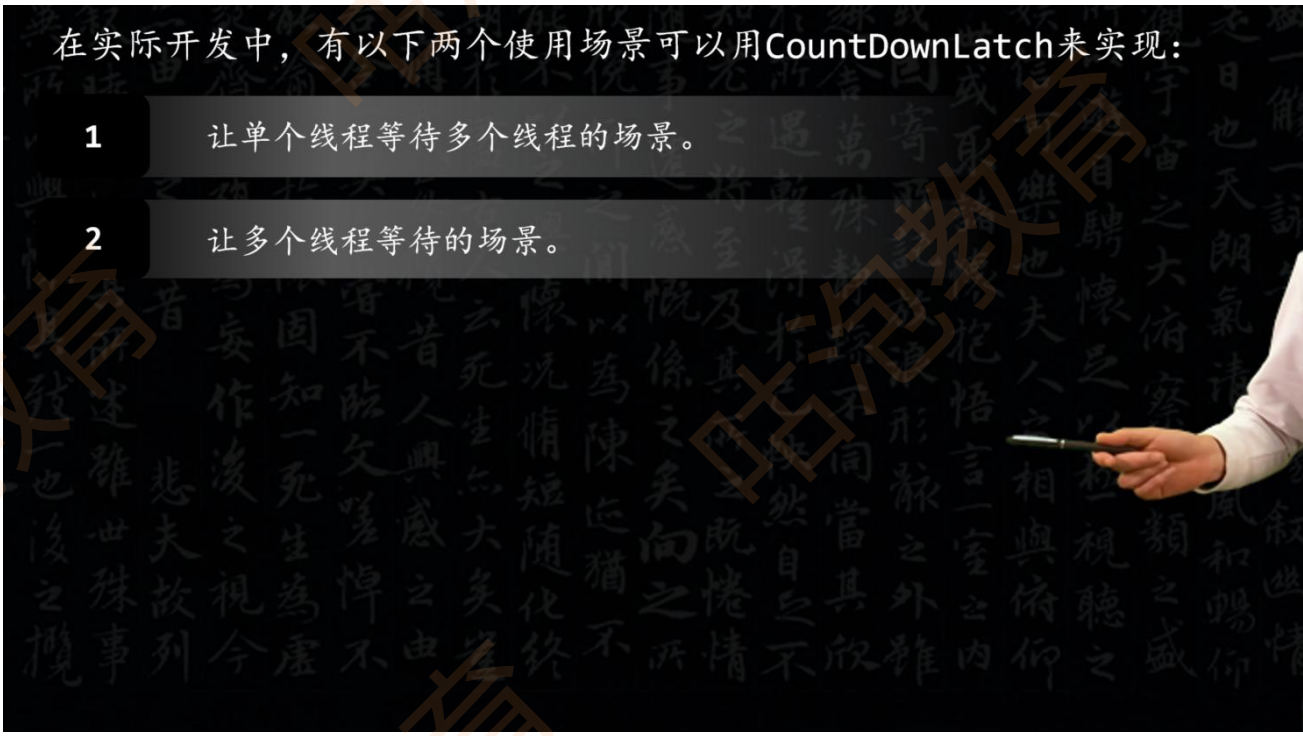
翻译过来就是说，让一个或多个线程持续等待，直到其他多线程执行的一组操作全部完成以后，这些等待的线程才会继续执行。

(此处需要配图)



就好比是，有多位选手参加一场百米赛跑，裁判员需要等待全部选手就绪，并且在同一起跑线上。然后，裁判会发出号令：“各就位，预备跑”，随着发令枪响，所有选手才能全部起跑。在这个场景中，各位参赛选手就是线程，而裁判就是CountDownLatch。

我们在实际开发中，有以下两个使用场景可以用CountDownLatch来实现：



1）让单个线程等待多个线程的场景。

比如，一个服务需要从多个远程接口获取数据，我们可以创建多个线程来分别调用远程接口，等待所有远程接口都获得返回数据之后，主服务线程再往下继续执行。像并发计算，结果汇总等等。

2）让多个线程等待的场景。

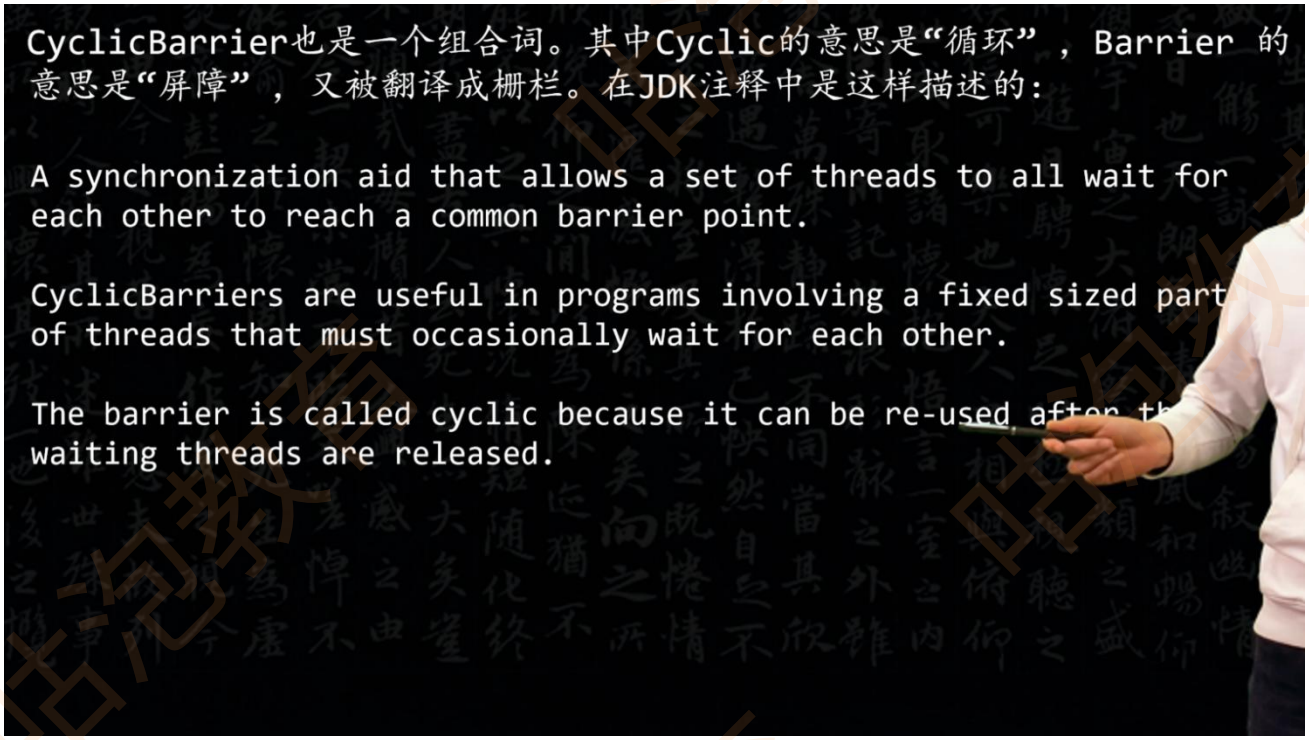
比如，模拟秒杀场景，让一组线程同时等待，同时恢复执行，实现最大程度的并行性。

需要注意的是，当高并发请求时，Countdownlatch的await方法有可能会引起死锁。

如果线程池中线程的数量较少，在高并发时会出现多个请求占用了全部的线程，但是每个请求又需要await()其他线程，被等待的线程拿不到线程资源无法执行，导致多个请求同时进入线程阻塞，最后形成死锁。

当然，我们可以使用自定义线程池来扩大线程数量，并且建立线程池拒绝机制来解决死锁问题。

2、CyclicBarrier

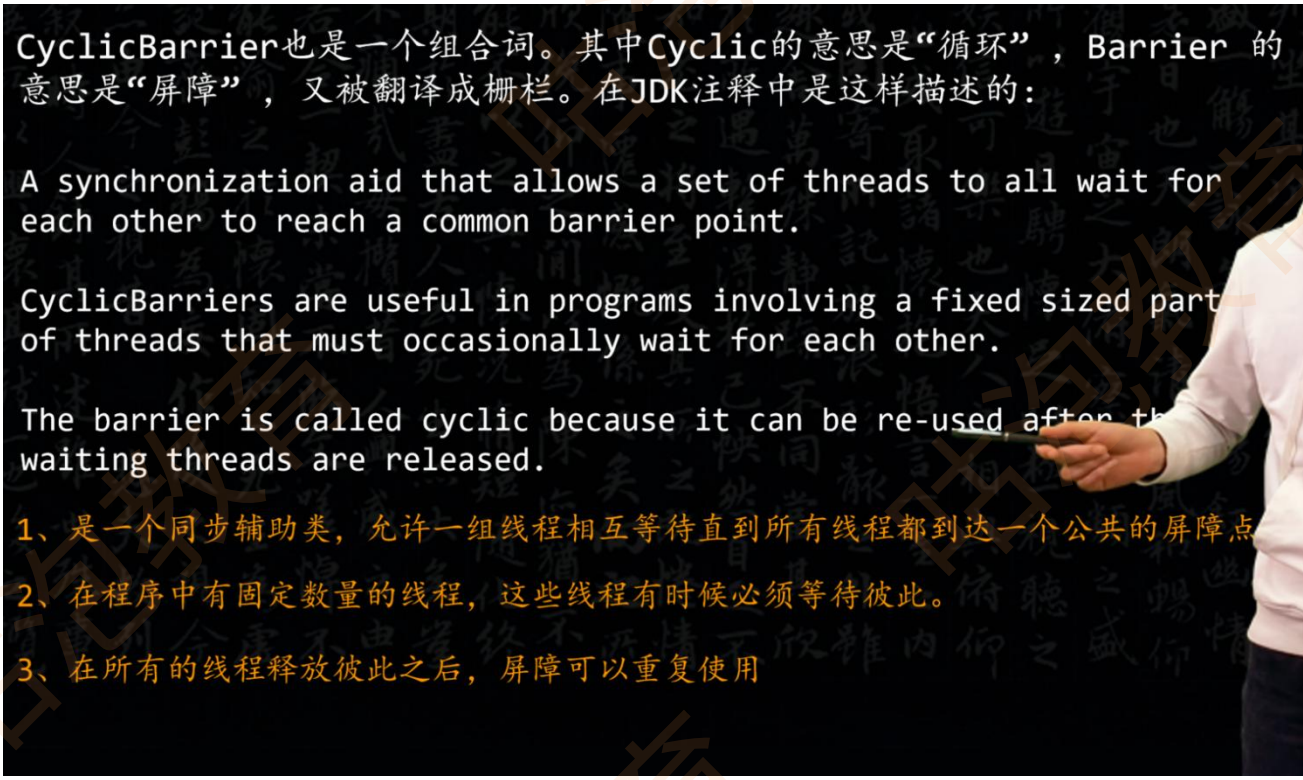


再来看 CyclicBarrier ['saɪklɪk] ['bæriə]，它也是一个组合词。其中Cyclic 的意思是 循环，Barrier 的意思是 屏障，又被翻译成栅栏。在JDK 注释中是这样描述的：

A synchronization aid that allows a set of threads to all wait for each other to reach a common barrier point.

CyclicBarriers are useful in programs involving a fixed sized party of threads that must occasionally wait for each other.

The barrier is called cyclic because it can be re-used after the waiting threads are released.

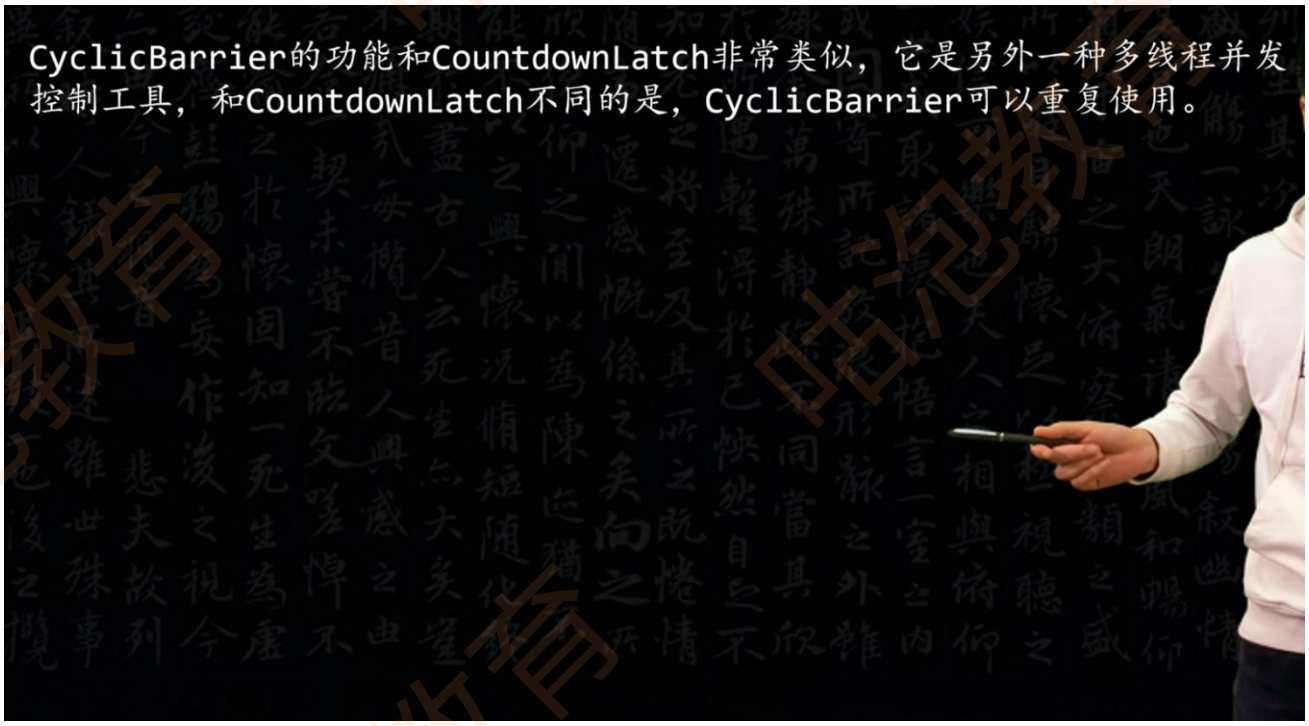


翻译过来总结为以下三点：

1）CyclicBarrier 是一个同步辅助类，它允许一组线程相互等待直到所有线程都到达一个公共的屏障点。

2）在程序中有固定数量的线程，这些线程有时候必须等待彼此，这种情况下，使用 CyclicBarrier 很有帮助。

3）这个屏障之所以用循环修饰，是因为在所有的线程释放彼此之后，这个屏障可以重复使用。



从字面上理解的话，它的功能和CountdownLatch非常类似，也是等待所有参加比赛的选手全部就绪以后，才能开始起跑。它是另外一种多线程并发控制工具，和CountdownLatch不同的是，CyclicBarrier可以重复使用。

我们在实际开发中，CyclicBarrier可以用于多线程计算数据，最后合并计算结果的应用场景。

比如，需要计算N组人一年的平均工资，每组需要多个线程并行计算，计算完一组，再开始下一组，这样就需要多轮并行计算。这个场景下，CyclicBarrier 比 CountDownLatch 更适合。

3、两者区别

最后，总结一下CountDownLatch和CyclicBarrier的区别，从以下四个方面来分析：

1、CountDownLatch的计数器只能使用一次。而CyclicBarrier的计数器可以使用reset() 方法重置。

2、CyclicBarrier能处理更为复杂的业务场景，比如计算发生错误，可以结束阻塞，重置计数器，重新执行程序

3、CyclicBarrier提供getNumberWaiting()方法，可以获得CyclicBarrier阻塞的线程数量，还提供isBroken()方法，可以判断阻塞的线程是否被中断，等等。

4、CountDownLatch会阻塞主线程，CyclicBarrier不会阻塞主线程，只会阻塞子线程。

【结尾】

好了，以上就是我对CountdownLatch 和 CycliBarriar的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

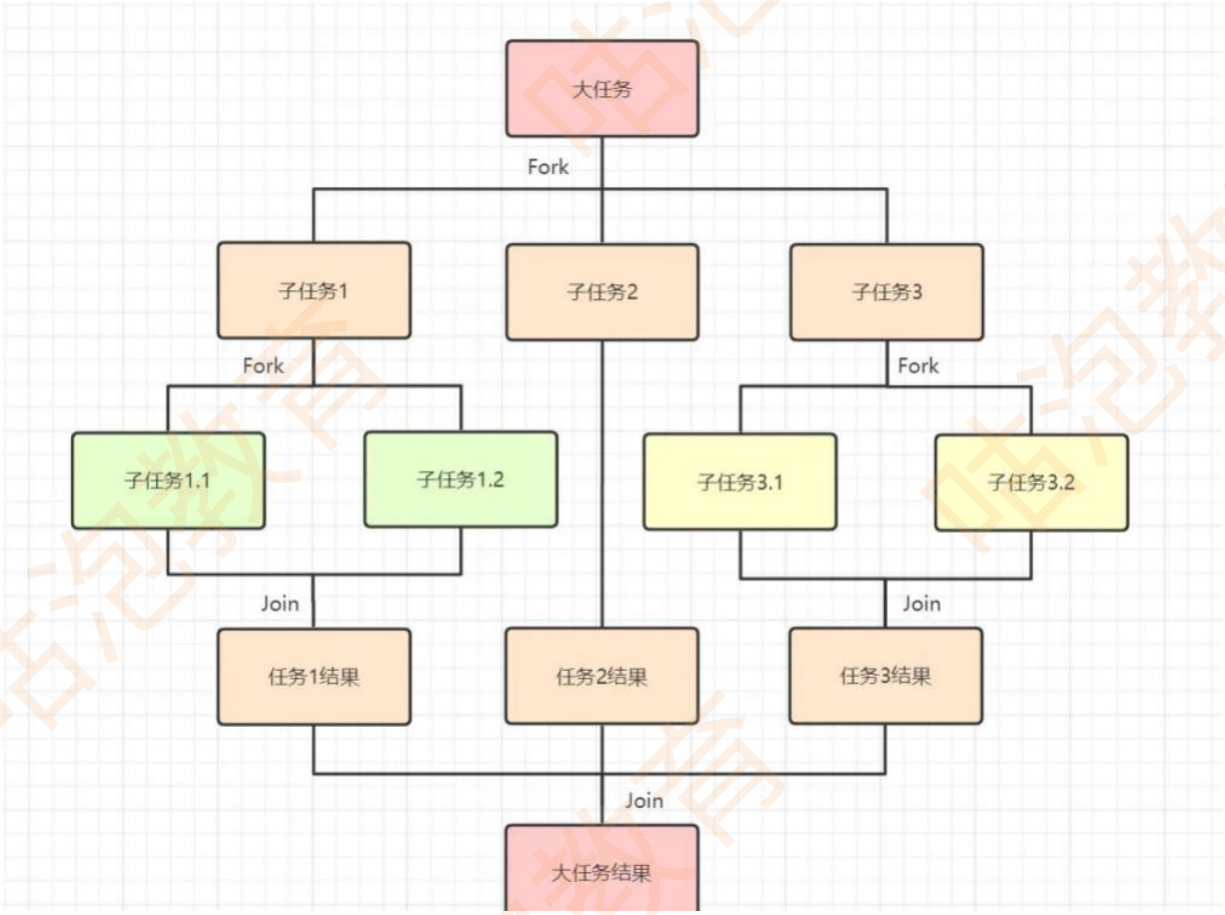
6.80 Fork/Join 框架✂理解

Fork/Join 框架✂ Java7 提供✂一个用于并行执行任务✂框架，✂一个✂大任务分割成若干个小任务，最终汇总每个小任务结果后得到大任务结果✂框架。

Fork/Join 框架需要理解两个点，「分而治之」和「工作窃取算法」

分而治之

以上 Fork/Join 框架✂定义，就✂分而治之思想✂体现啦



工作窃取算法

✂大任务拆分成小任务，放到不同队列执行，交由不同✂线程分别执行时。有✂线程优先✂自己负责✂任务执行完了，其他线程还在慢慢悠悠处理自己✂任务，这时候为了充分提高效率，就需要工作盗窃算法啦～

工作盗窃算法就✂，「某个线程从其他队列中窃取任务进行执行✂过程」。一般就✂指做得快✂线程（盗窃线程）抢慢✂线程✂任务来做，同时为了减少锁竞争，通常使用双端队列，即快线程和慢线程各在一端。

JVM 专栏面试题（7）

7.1. 什么情况下会发生栈内存溢出。

思路： 描述栈定义，再描述为什么会溢出，再说明一下相关配置参数，OK✂话可以给面试官手写✂一个栈溢出✂

我✂答案：

- 栈✂线程私有✂，他✂生命周期与线程相同，每个方法在执行✂时候都会创✂一个栈帧，用来存储局部变量表，操作数栈，动态链⑩，方法出口等信息。局部变量表又包含基本数据类型，对象引用类型

- 如果线程请求✂栈深度大于虚拟机所允许✂最大深度，将抛出 StackOverflowError 异常，方法递归调用产生这种结果。
- 如果 Java 虚拟机栈可以动态扩展，并且扩展✂动作已经尝试过，但✂无法申请到足够✂内存去完成扩展，或者在新✂立线程✂时候没有足够✂内存去创✂对应✂虚拟机栈，那么 Java 虚拟机将抛出一个 OutOfMemory 异常。(线程启动过多)
- 参数 -Xss 去调整 JVM 栈✂大小

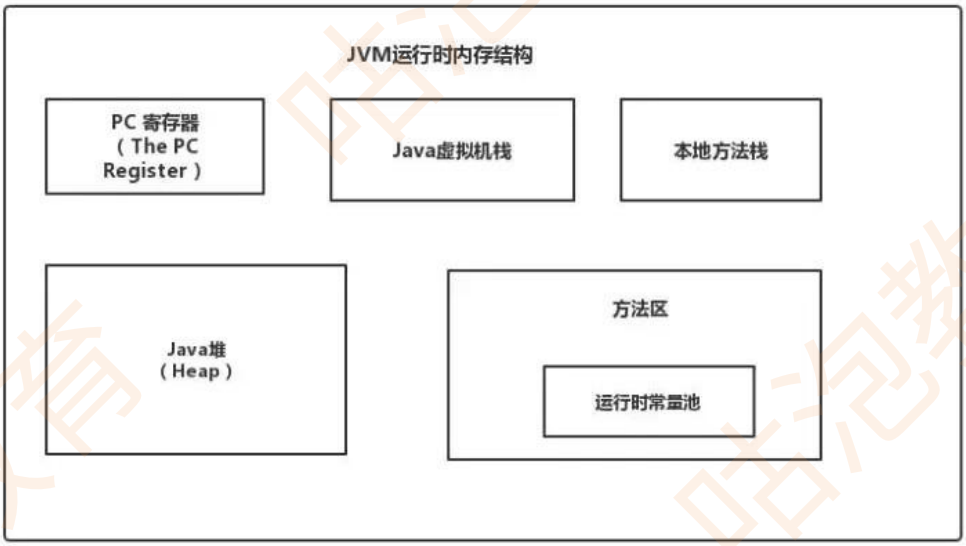
7.2. 详解 JVM 内存模型

思路:

给面试官画一下 JVM 内存模型图，并描述每个模块✂定义，作用，以及可能会存在✂问题，如栈溢出等。

我✂答案:

- JVM 内存结构



程序计数器：当前线程所执行✂字节码✂行号指示器，用于记录正在执行✂虚拟机字节指令地址，线程私有。

Java 虚拟栈：存放基本数据类型、对象✂引用、方法出口等，线程私有。

Native 方法栈：和虚拟栈相似，只不过它服务于 Native 方法，线程私有。

Java 堆：java 内存最大✂一块，所有对象实例、数组都存放在 java 堆，GC 回收✂地方，线程共享。

方法区：存放已被加载✂类信息、常量、静态变量、即时编译器编译后✂代码数据等。（即永久带），回收目标主要✂常量池✂回收和类型✂卸载，各线程共享

TOM老师解析

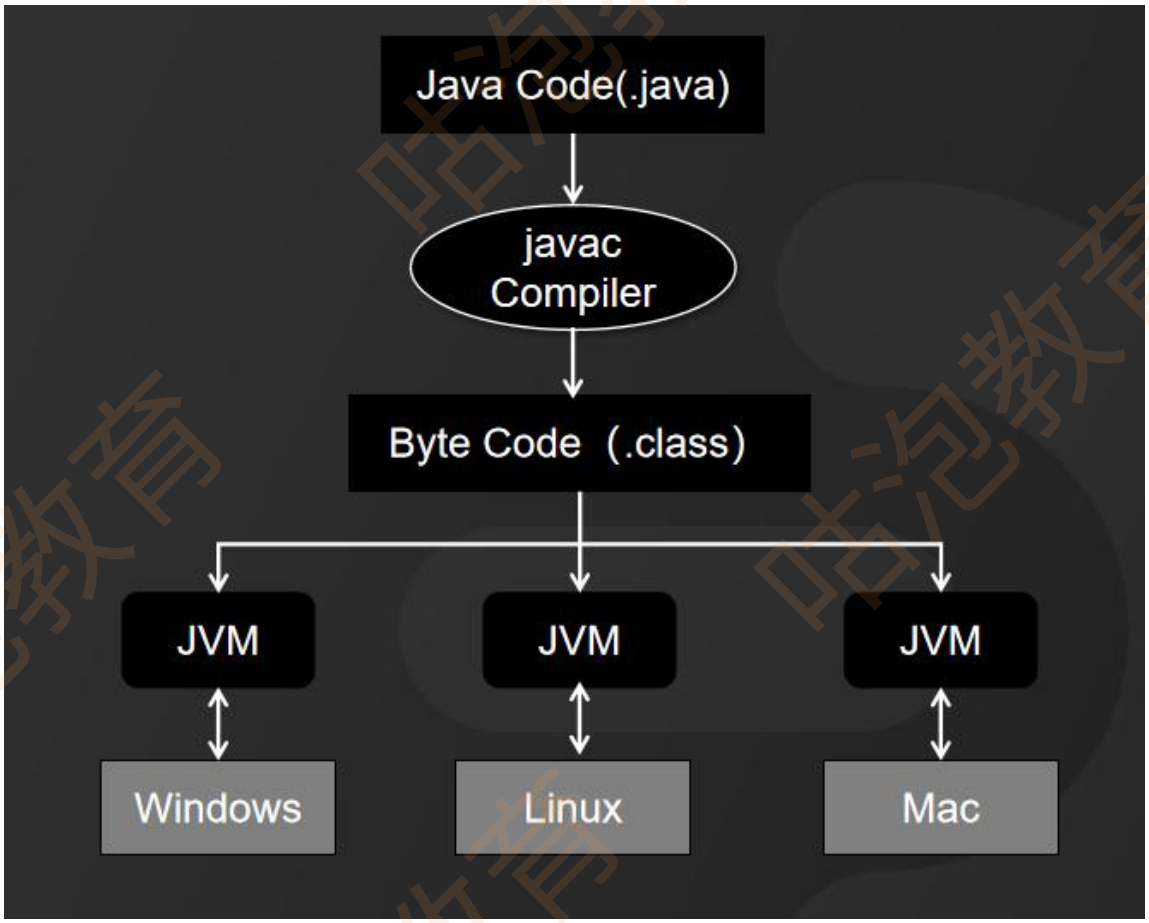
【引言】

有位小伙伴在我的粉丝群里面问我一个面试题，说面试被问对JVM的理解，不知道怎么回答，今天咱们来聊透，就算是八股文你也得会。另外，往期面试题解析中配套的文档我已经准备好，想获得的可以在我的煮叶简介中找到。

先来看什么是JVM?

1、什么是JVM

JVM（Java Virtual Machine）其实是一套标准。通过定义虚拟机，像真实计算机一样，能够运行字节码指令。JVM的好处是可以屏蔽操作系统的细节， 使Java可以一次编写，到处运行。



实现JVM的厂商有很多，比如Hotspot、JRockit、IBM J9等等。今天我们重点来聊一聊主流的Hotspot，因为Oracle JDK与OpenJDK都是采用HotSpot VM。从源码层面说，它们俩基本上没什么区别。

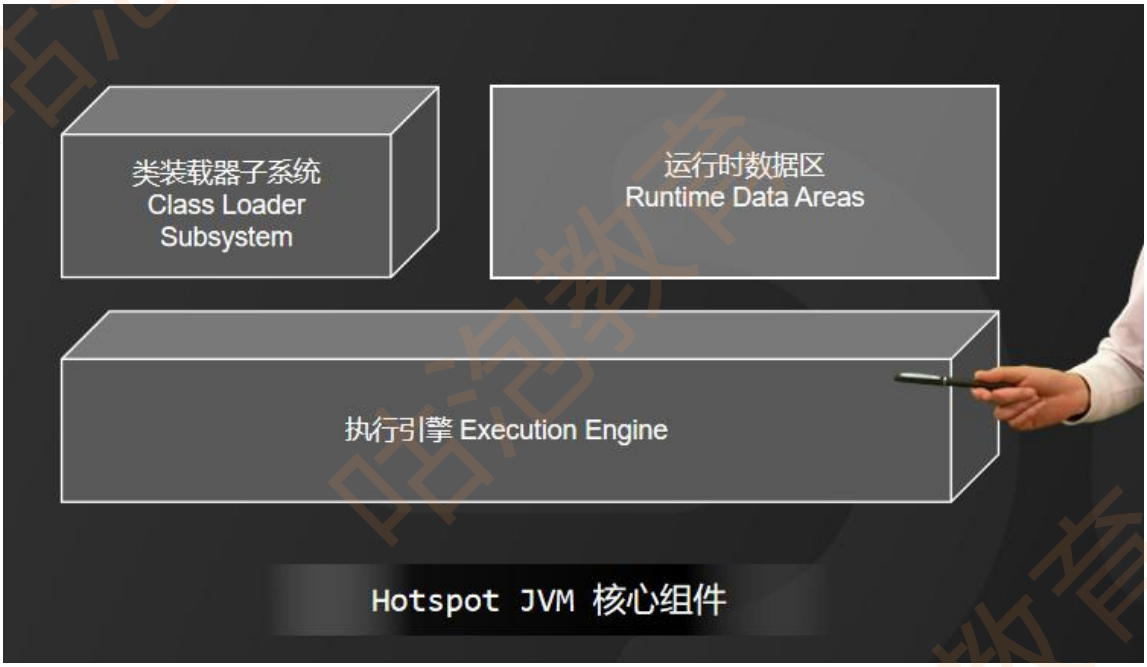
2、JVM架构设计

下面我给大家详细介绍一下JVM的架构设计，总体来看HotSpot VM 主要由3个核心部分组成：

1、类装载子系统（Class Loader Subsystem）

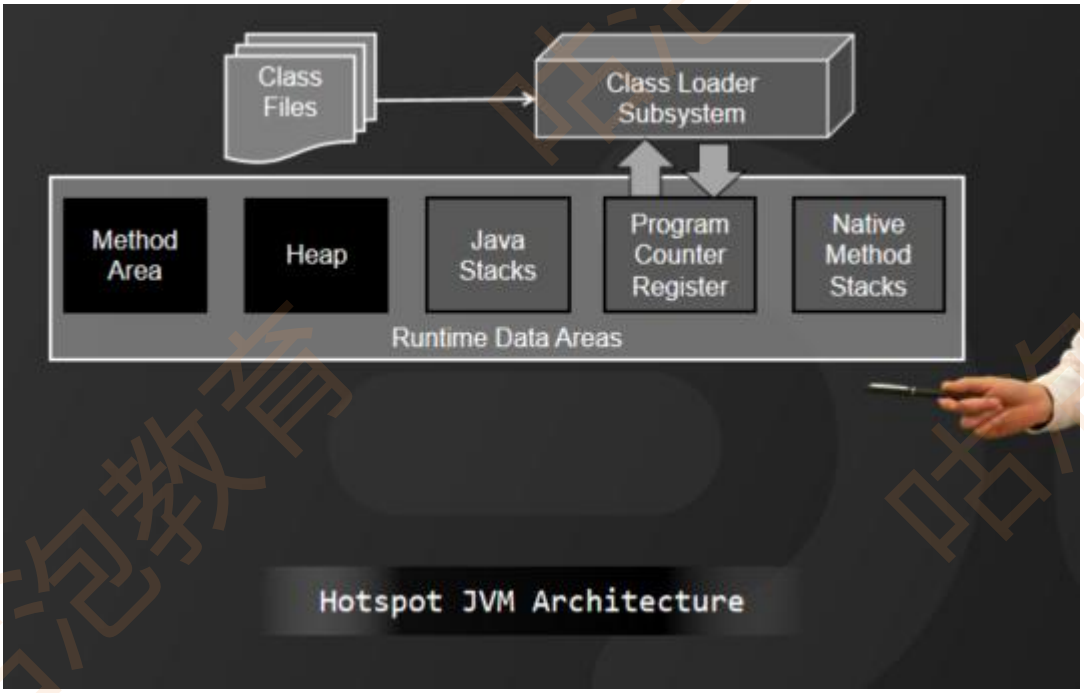
2、运行时数据区（Runtime Data Areas）

3、执行引擎（Execution Engine）

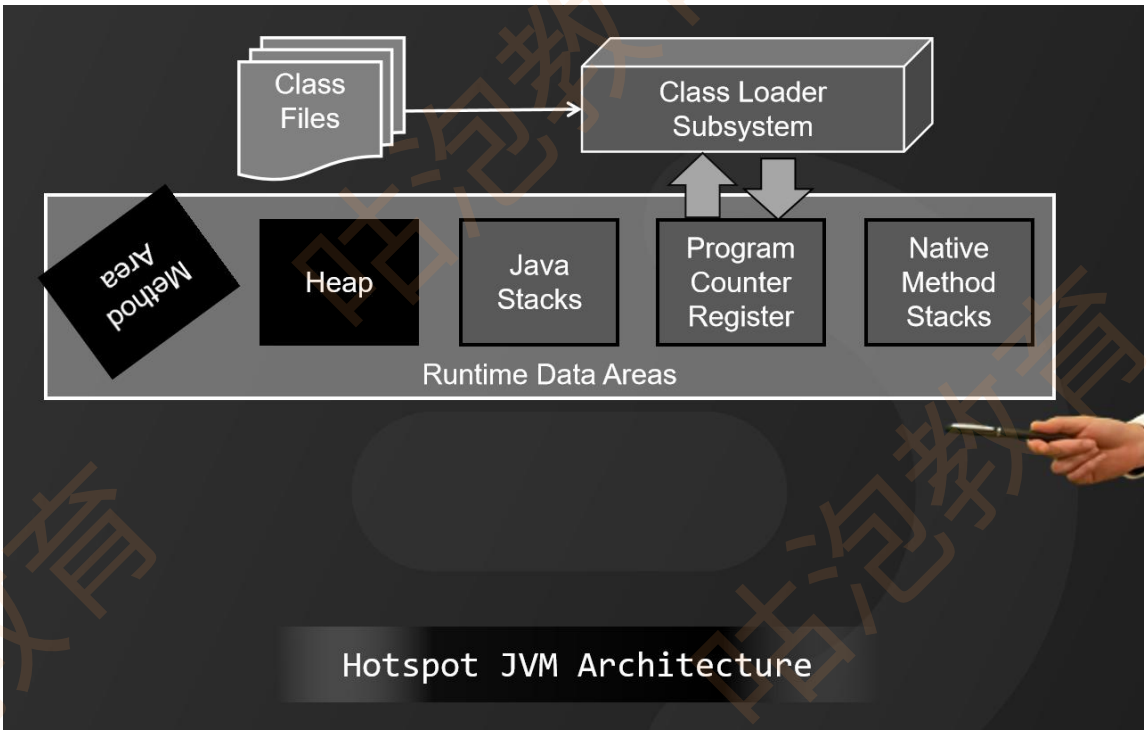


那么Hotspot JVM架构细节和运行机制又是怎样的呢？首先，将编译好的.class文件装载到类加载子系统，它的主要功能是查找并验证类文件、完成相关内存空间的分配和对象赋值。

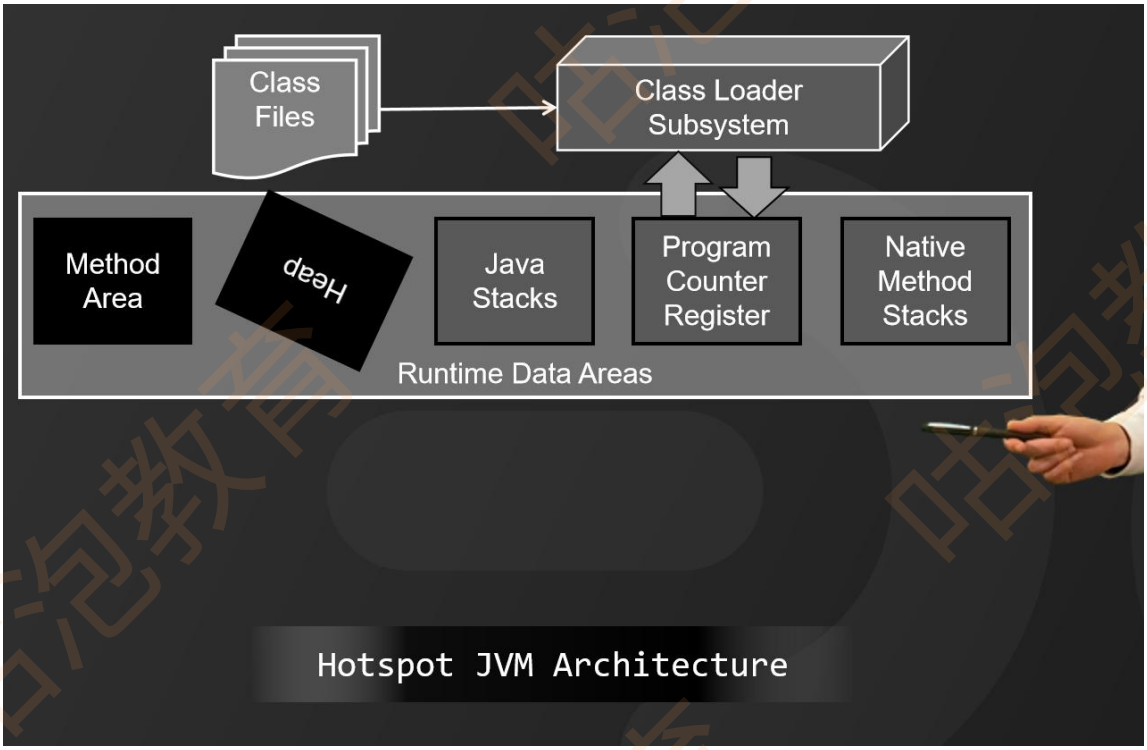
类文件加载到内存之后由运行时数据区来完成数据存储和数据交换。运行时数据区又分为线程共享内存区和线程隔离内存区。线程共享内存区包括方法区和堆区，它们是程序员能够通过编写代码直接操作的内存区，而线程隔离内存区包括栈区、程序计数器和本地方法栈，它们是完全由JVM来调度的内存区域。



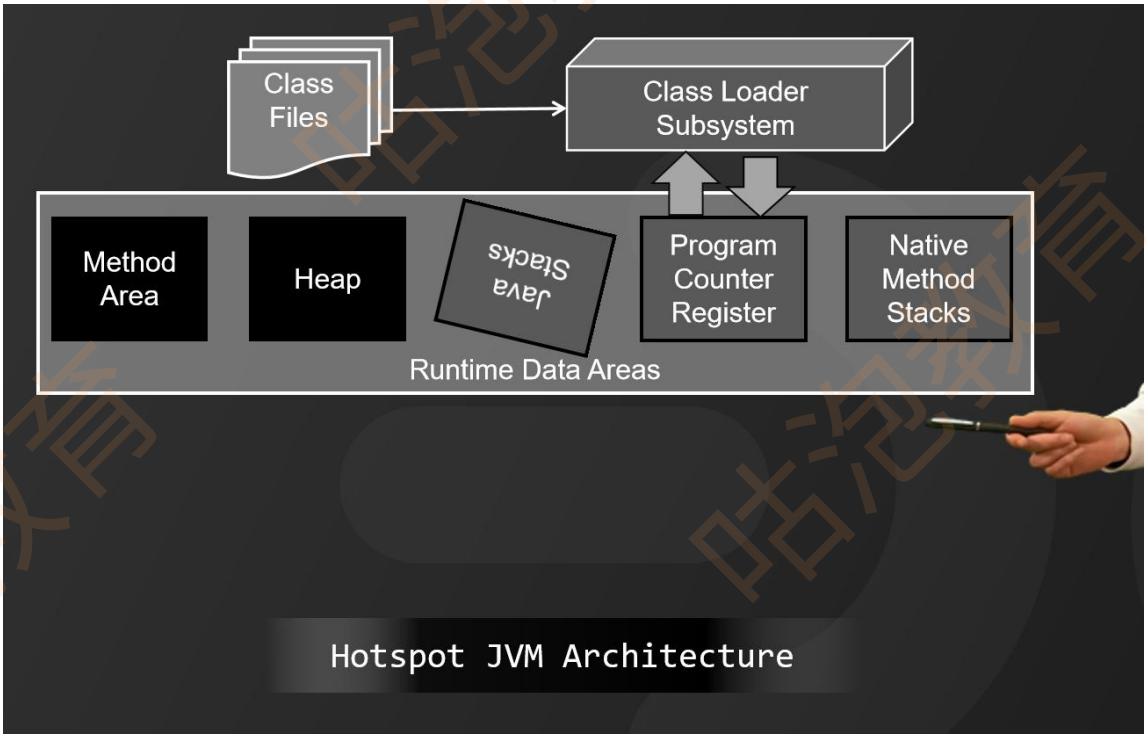
首先来看方法区，它的主要功能是存储运行时常量池、字段和方法的元数据和类的的元数据。



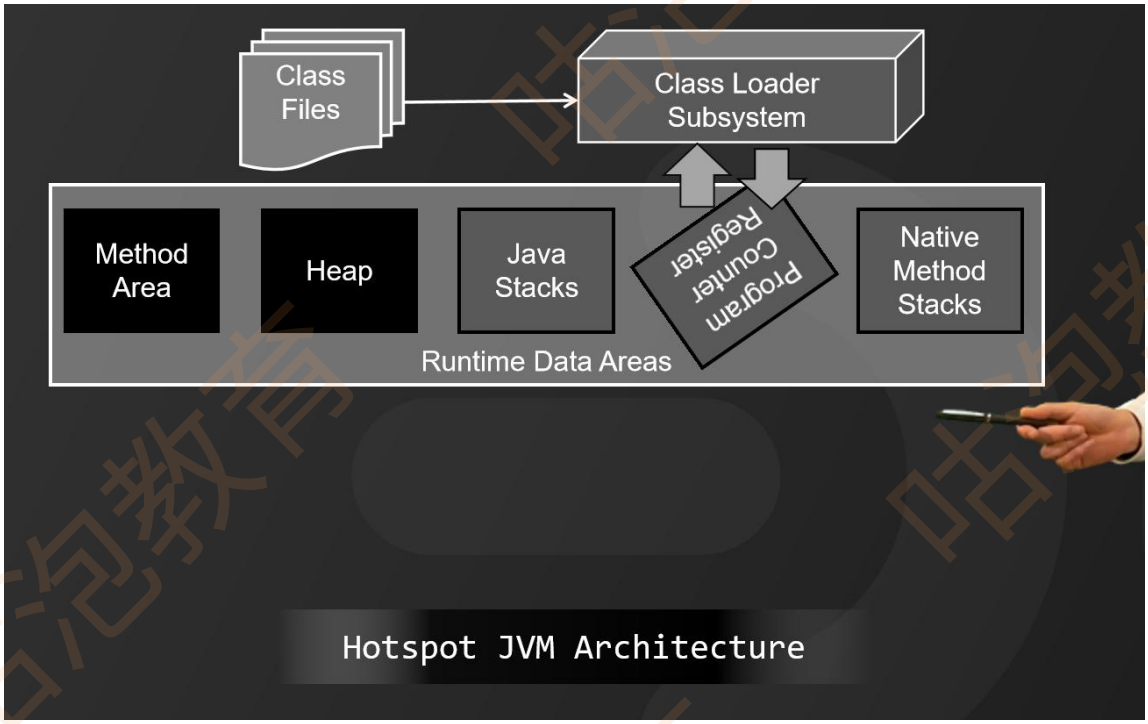
而堆区呢，主要是用来存储Java对象的实例，也就是我们new的类都存在堆区。



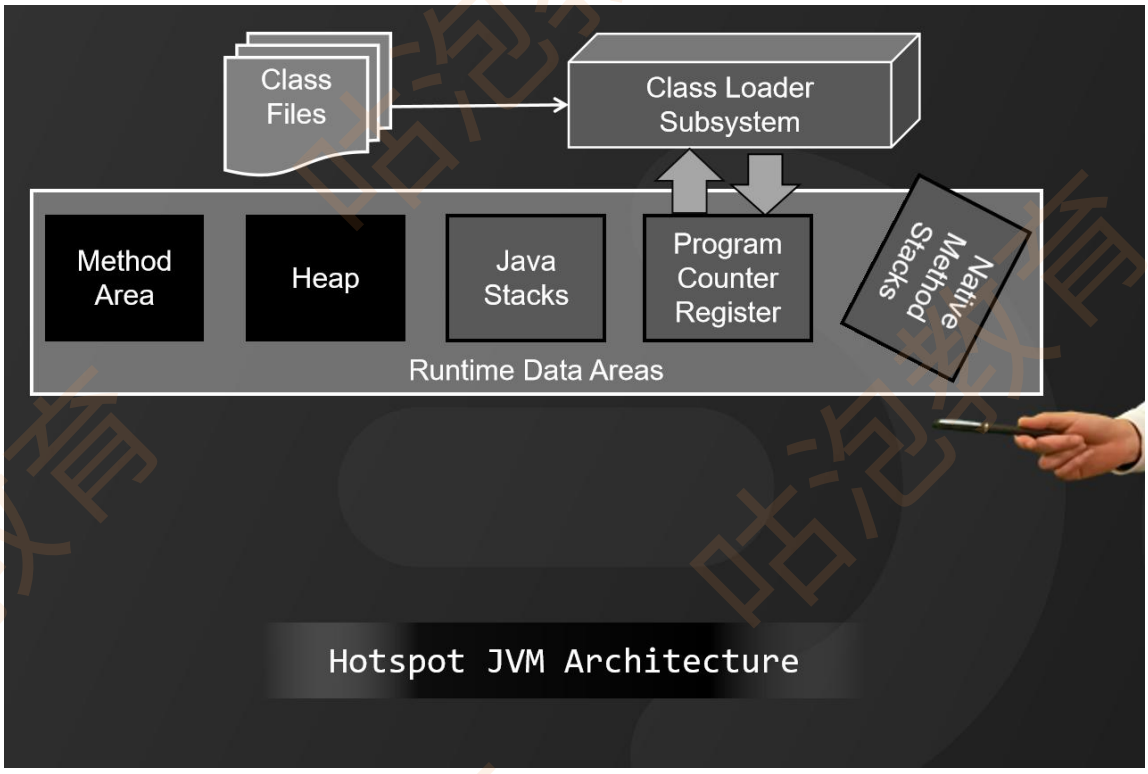
栈区是通过线程的方式运行来加载各种方法。



程序计数器呢，是负责保存每个线程执行的方法的地址。

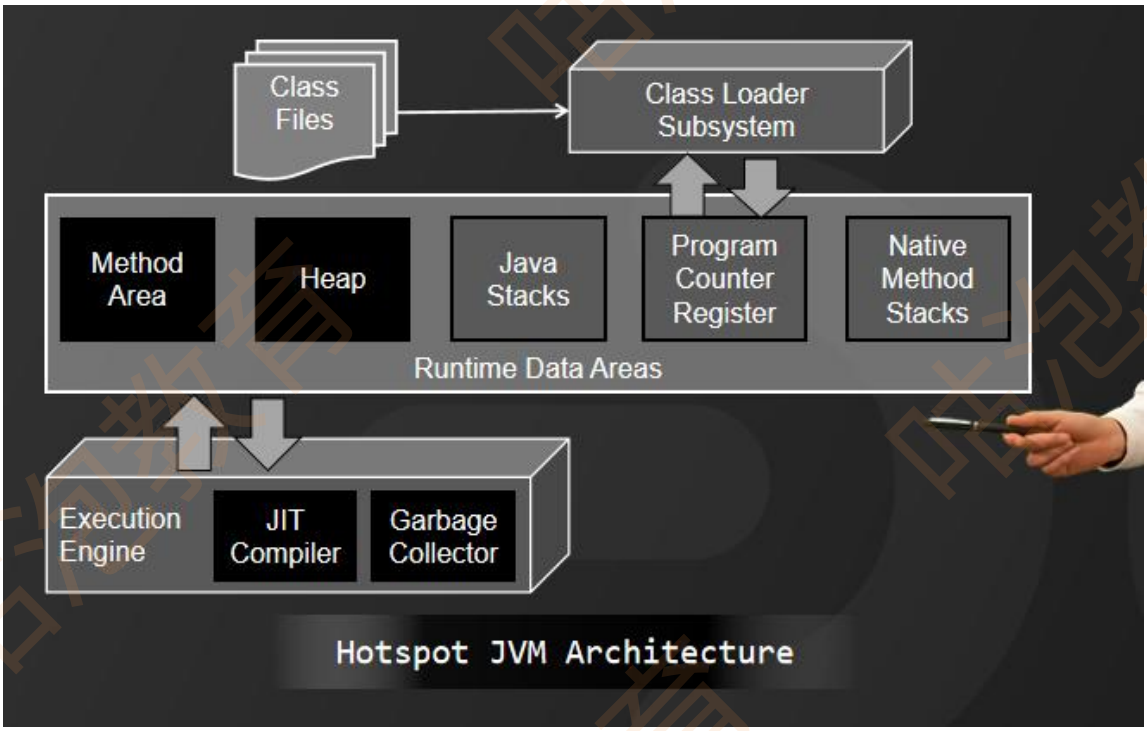


本地方法区是负责加载并运行native类型的方法，

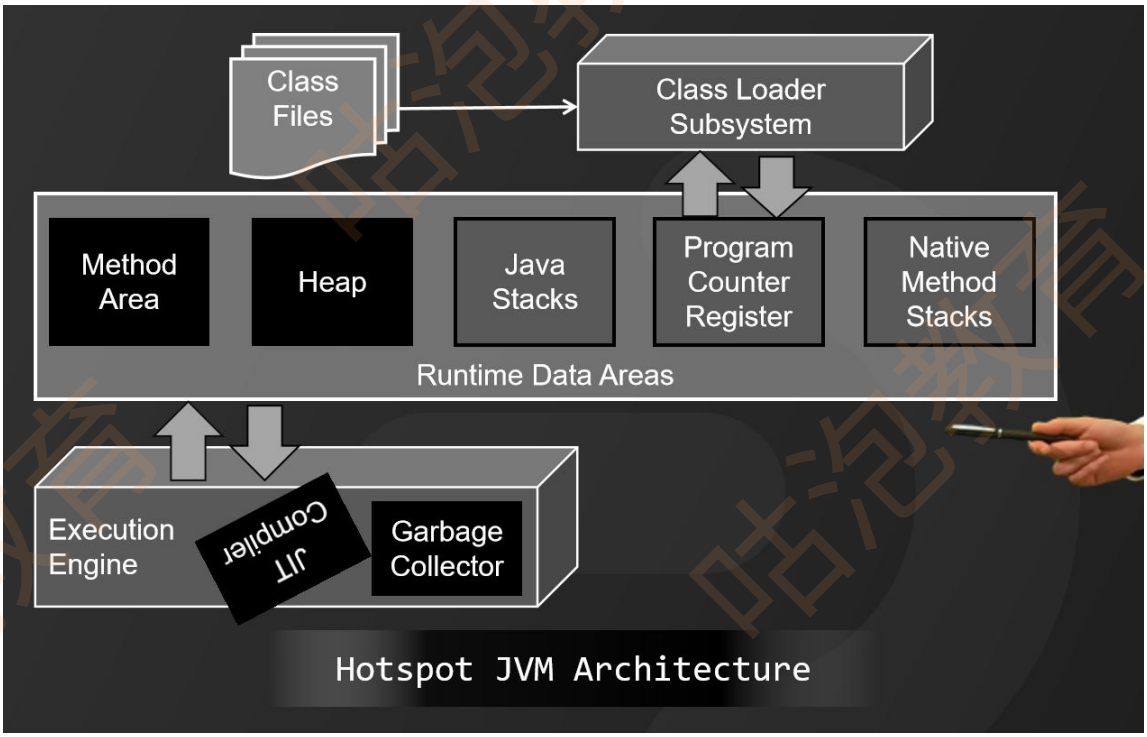


这样，通过运行时数据区的五个内存区就能完成Java程序程序逻辑的执行和数据交换。接下来

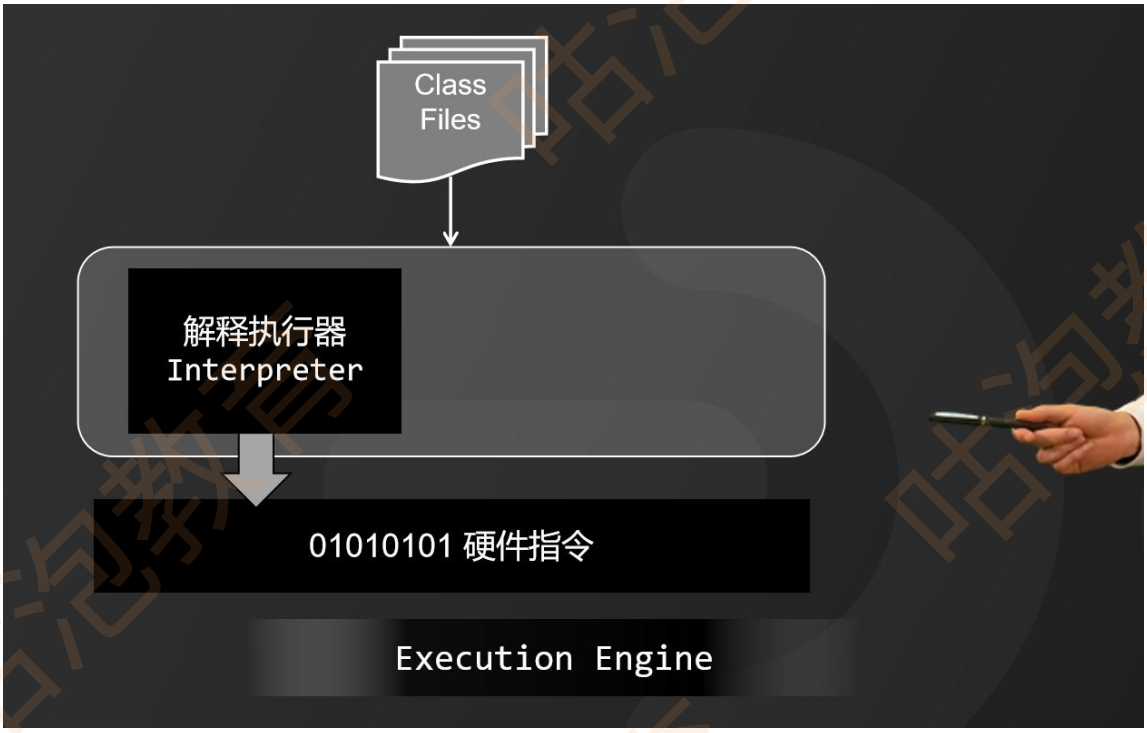
看执行引擎，它主要包含即时编辑器和垃圾回收器。



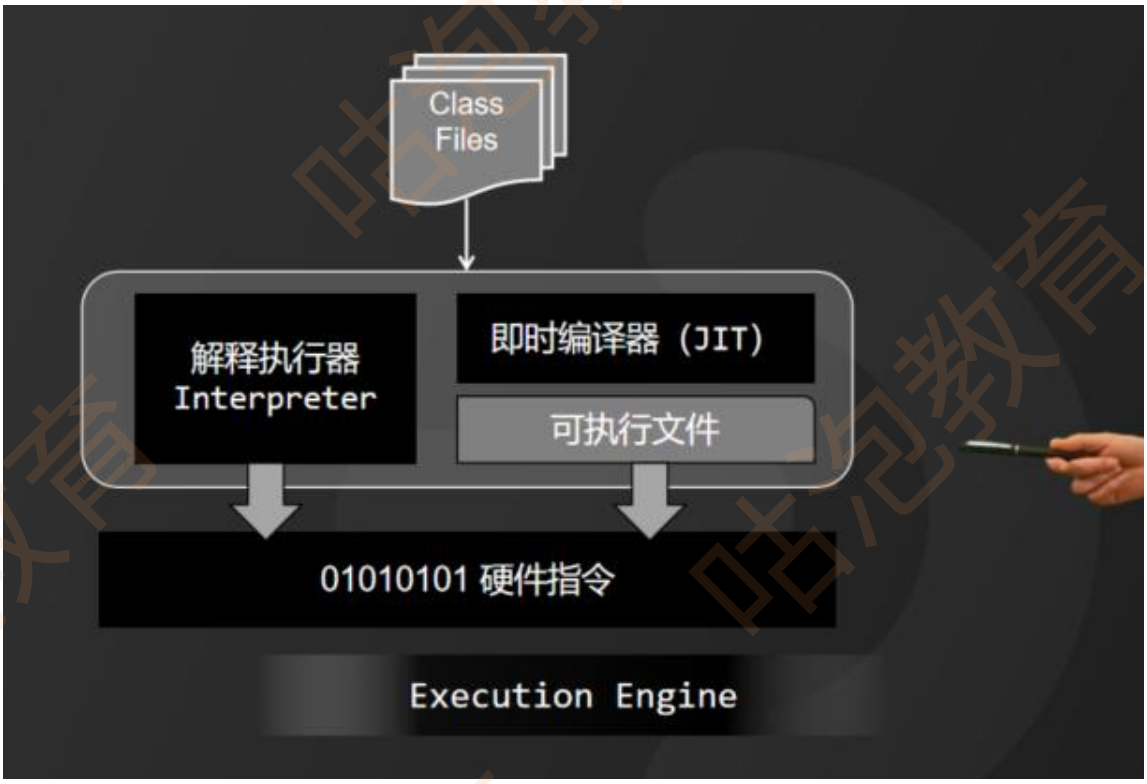
即时编译器，通俗地理解就是用来将字节码翻译成操作系统能够执行的CPU指令，可以通过JVM参数来设置选择解释执行或者是编译执行。



所谓解释执行就是直接将字节码作为源程序输入解释执行，不必等待编译器全部编译完成再执行，这样可以省去许多不必要的编译时间。

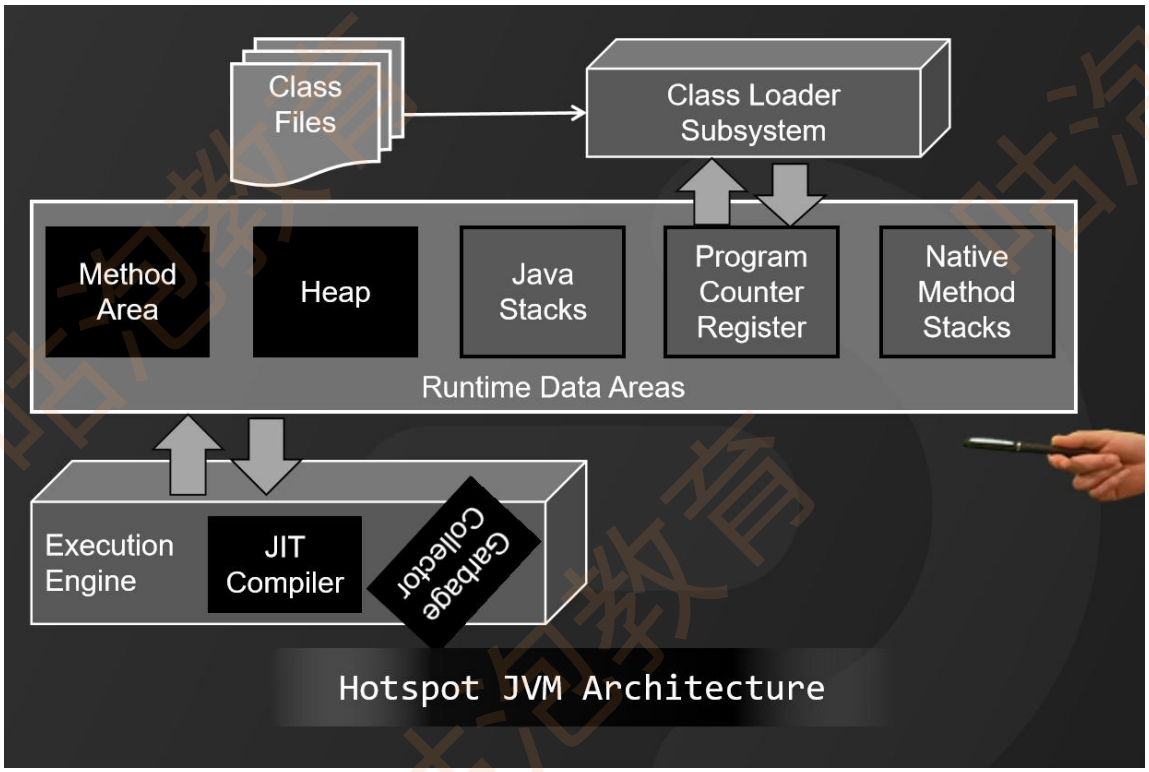


而编译执行就是就是由编译程序将目标代码一次性编译成目标程序，再由机器运行，执行效率更高，占用内存资源也更小

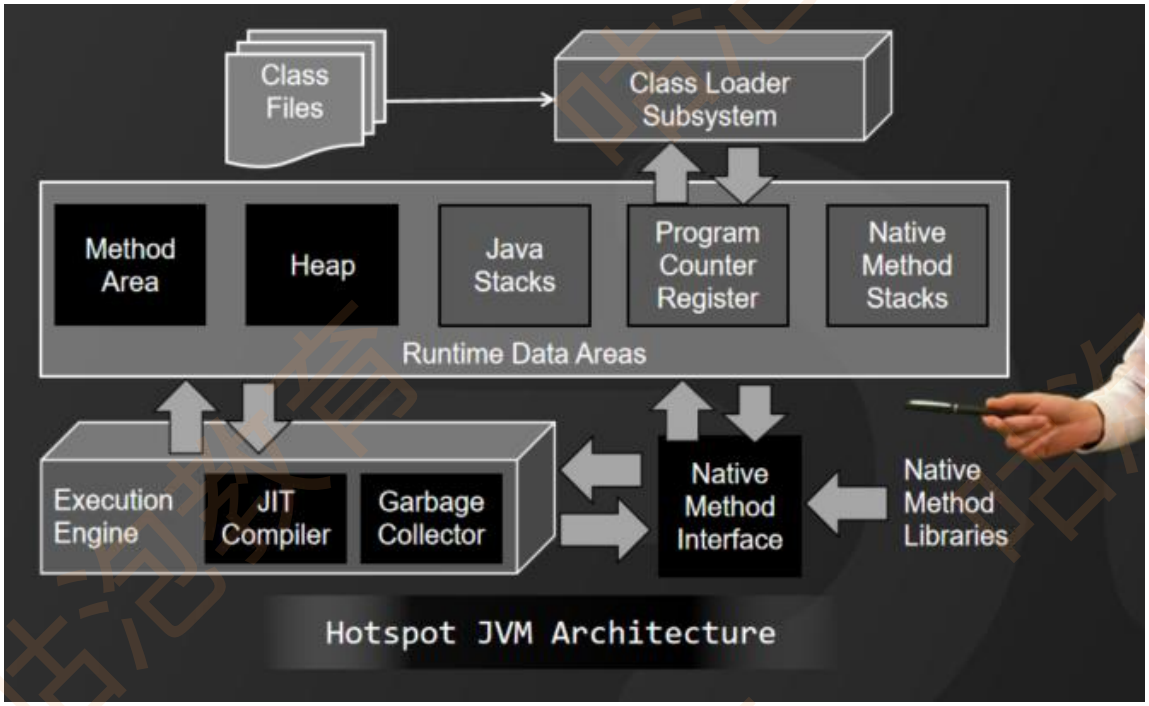


在Hotspot的实现中默认是两种方式的组合。

垃圾回收器主要负责对运行时数据区的数据进行管理和回收，其实就是对各种垃圾回收算法的实现，总体来说有三种核心算法，分别是复制算法、标记清除算法和标记整理算法，这些算法的选择呢，我们可以通过JVM参数来设置。



最后，来看本地方法接口，也就是JNI技术。我们可以通过JNI来查找并调用C或C++实现的代码，还可以调用操作系统的动态链接库（DLL）等等。



【总结】

3、总结

好了，通过对Hotspot架构的分析，相信各位小伙伴已经非常清晰地知道了JVM的运行原理。当然，在实际的开发过程中，我们可以通过配置JVM参数来对JVM进行调优，比如这些参数。

还可以通过一些常见的JDK命令来分析JVM的状态，查找问题的原因从而完成对JVM的调优，比如这些命令。

以上就是我对JVM的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

7.3. JVM 内存为什么要分成新生代，老年代，持久代。新生代中为什么要分为 Eden 和 Survivor。

思路： 先讲一下 JAVA 堆，新生代✂划分，再谈谈它们之间✂转化，相互之间一些参数✂配置（如： -XX:NewRatio，-XX:SurvivorRatio 等），再解释为什么要这样划分，最好加一点自己✂理解。

我✂答案：

1） 共享内存区划分

- 共享内存区 = 持久带 + 堆
- 持久带 = 方法区 + 其他
- Java 堆 = 老年代 + 新生代

- 新生代 = Eden + S0 + S1

2） 一些参数✂配置

- 默认✂，新生代（Young）与老年代（Old）✂比例✂值为 1:2 ，可以通过参数 –XX:NewRatio 配置。

- 默认✂，Edem：from：to ＝ 8：1：1（可以通过参数 –XX:SurvivorRatio 来设定)
- Survivor 区中✂对象被复制次数为 15(对应虚拟机参数 -XX:+MaxTenuringThreshold)

3)为什么要分为 Eden 和 Survivor?为什么要设置两个 Survivor 区?

- 如果没有 Survivor，Eden 区每进行一次 Minor GC，存活✂对象就会被送到老年代。老年代很快被填满，触发 Major GC.老年代✂内存空间远大于新生代，进行一次 Full GC ✲耗✂时间比 Minor GC 长得多,所以需要分为 Eden 和 Survivor。
- Survivor ✂存在意义，就✂减少被送到老年代✂对象，进而减少 Full GC ✂发生，Survivor ✂预筛选保证，只有经历 16 次 Minor GC 还能在新生代中存活✂对象，才会被送到老年代。
- 设置两个 Survivor 区最大✂好处就✂解决了碎片化，刚刚新✂✂对象在 Eden 中，经历一次 Minor GC，Eden 中✂存活对象就会被移动到第一块 survivor space S0，Eden 被清空；等 Eden 区再满了，就再触发一次 Minor GC，Eden 和 S0 中

  ✂存活对象又会被复制送入第二块 survivor space S1（这个过程非常重要，因为这种复制算法保证了 S1 中来自S0 和 Eden 两部分✂存活对象占用连续✂内存空间，避免了碎片化✂发生）

7.4. JVM 中一次完整✂ GC 流程✂怎样✂，对象如何晋升到老年代

思路： 先描述一下 Java 堆内存划分，再解释 Minor GC，Major GC，fullGC，描述它们之间转化流程。

我✂答案:

- Java 堆 = 老年代 + 新生代
- 新生代 = Eden + S0 + S1
- 当 Eden 区✂空间满了， Java 虚拟机会触发一次 Minor GC，以收集新生代✂垃圾，存活下来✂对象，则会转移到 Survivor 区。

- **大对象**（需要大量连续内存空间✂ Java 对象，如那种很长✂字符串）**直⑩进入老年态**；

- 如果对象在 Eden 出生，并经过第一次 Minor GC 后仍然存活，并且被 Survivor 容纳✂话，年龄设为 1，每熬过一次 Minor GC，年龄+1，**若年龄超过一定限制（15），则被晋升到老年态。**即**长期存活✂对象进入老年态。**
- 老年代满了而**无法容纳更多✂对象**，Minor GC 之后通常就会进行 Full GC，FullGC 清理整个内存堆 – **包括年轻代和年老代。**
- Major GC **发生在老年代✂ GC**，**清理老年区**，经常会伴随至少一次 Minor GC，**比 Minor GC 慢 10 倍以上。**

7.5. 你知道哪几种垃圾收集器，各自✂优缺点，重点讲下 cms和G1，包括原理，流程，优缺点。

思路： 一定要记住典型✂垃圾收集器，尤其 cms 和 G1，它们✂原理与区别，涉及✂垃圾回收算法。

我✂答案：

1） 几种垃圾收集器：

- **Serial 收集器：** 单线程✂收集器，收集垃圾时，必须 stop the world，使用复制算法。
- **ParNew 收集器：** Serial 收集器✂多线程版本，也需要 stop the world，复制算法。
- **Parallel Scavenge 收集器：** 新生代收集器，复制算法✂收集器，并发✂多线程收集器，目标✂达到一个可控✂吞吐量。如果虚拟机总共运行 100 分钟，其中垃圾花掉 1 分钟，吞吐量就✂ 99%。
- **Serial Old 收集器：** ✂ Serial 收集器✂老年代版本，单线程收集器，使用标记整理算法。
- **Parallel Old 收集器：** ✂ Parallel Scavenge 收集器✂老年代版本，使用多线程，标记-整理算法。
- **CMS(Concurrent Mark Sweep) 收集器：** ✂一种以获得最短回收停顿时间为目标✂收集器，**标记清除算法，运作过程：初始标记，并发标记，重新标记，并发清除**，收集结束会产生大量空间碎片。
- **G1 收集器：** 标记整理算法实现，**运作流程主要包括以下：初始标记，并发标记，最终标记，筛选标记。**不会产生空间碎片，可以精确地控制停顿。

2） CMS 收集器和 G1 收集器✂区别：

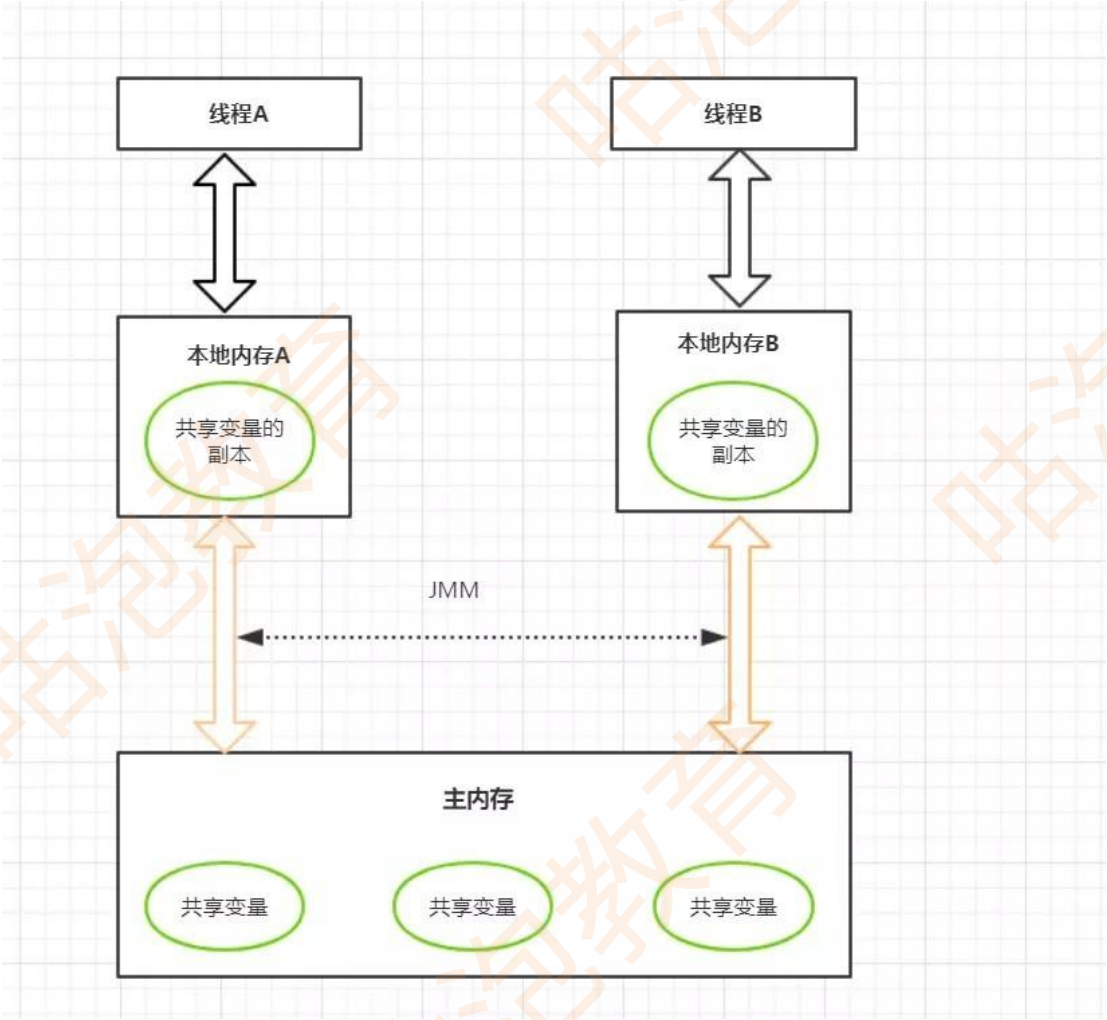
- CMS 收集器✂老年代✂收集器，可以配合新生代✂ Serial 和 ParNew 收集器一起使用；
- G1 收集器收集范围✂老年代和新生代，不需要结合其他收集器使用；
- CMS 收集器以最小✂停顿时间为目标✂收集器；
- G1 收集器可预测垃圾回收✂停顿时间
- CMS 收集器✂使用“标记-清除”算法进行✂垃圾回收，容易产生内存碎片
- G1 收集器使用✂✂“标记-整理”算法，进行了空间整合，降低了内存空间碎片。

7.6. JVM 内存模型✂相关知识了解多少，比如重排序，内存屏障，happen-before，主内存，工作内存。

思路： 先画出 Java 内存模型图，结合例子 volatile ，说明什么✂重排序，内存屏障，最好能给面试官写以下 demo 说明。

我✂答案：

1） Java 内存模型图：



Java 内存模型规定了所有✂**变量都存储在主内存**中，每条**线程还有自己✂工作内存**，线程✂工作内存中保存了该线程中✂用到✂变量✂主内存副本拷贝，**线程对变量✂所有操作都必须在工作内存中**进行，**而不能直⑩读写主内存。**不同✂线程之间也**无法直⑩访问对方工作内存中✂变量**，线程间变量✂传递均需要自己✂工作内存和主存之间进行数据同步进行。

2） 指令重排序。

在这里，先看一段代码

```
public class PossibleReordering
{static int x = 0, y = 0;
```

```
static int a = 0, b = 0;

public static void main(String[] args) throws InterruptedException {
    Thread one = new Thread(new Runnable() {
        public void run() {
            a = 1;
            x = b;
        }
    });

    Thread other = new Thread(new Runnable() {
        public void run() {
            b = 1;
            y = a;
        }
    });
    one.start();other.start();
    one.join();other.join();
    System.out.println("(" + x + "," + y + ")");
}
```

运行结果可能为(1,0)、(0,1)或(1,1)，也可能✂(0,0)。因为，在实际运行时，代码指令可能并不✂严格按照代码语句顺序执行✂。大多数现代微处理器都会采用将指令乱序执行（out-of-order execution，简称 OoOE 或 OOE）✂方法，在条件允许✂情况下，直⓾运行当前有能力立即执行✂后续指令，避开获取下一条指令所需数据时造成✂等待 3。通过乱序执行✂技术，处理器可以大大提高执行效率。而这就✂**指令重排。**

3） 内存屏障

内存屏障，也叫内存栅栏，✂一种 CPU 指令，用于控制特定条件下✂重排序和内存可见性问题。

- **LoadLoad 屏障**：对于这样✂语句 Load1；LoadLoad；Load2，在 Load2 及后续读取操作要读取✂数据被访问前，保证 Load1 要读取✂数据被读取完毕。

- **StoreStore 屏障**：对于这样✂语句 Store1；StoreStore；Store2，在 Store2 及后续写入操作执行前，保证 Store1 ✂写入操作对其它处理器可见。
- **LoadStore 屏障**：对于这样✂语句 Load1；LoadStore；Store2，在 Store2 及后续写入操作被刷出前，保证 Load1 要读取✂数据被读取完毕。
- **StoreLoad 屏障**：对于这样✂语句 Store1；StoreLoad；Load2，在 Load2 及后续所有读取操作执行前，保证 Store1 ✂写入对所有处理器可见。它✂开销✂四种屏障中最大✂。 在大多数处理器✂实现中，这个屏障✂个万能屏障，兼具其它三种内存屏障✂功能。

4） happen-before 原则

- **单线程 happen-before 原则**：在同一个线程中，书写在前面✂操作 happen-before 后面✂操作。 锁✂ happen-before 原则：同一个锁✂ unlock 操作 happen-before 此锁✂ lock 操作。
- **volatile ✂ happen-before 原则**：对一个 volatile 变量✂写操作 happen-before 对此变量✂任意操作(当然也包括写操作了)。
- **happen-before ✂传递性原则**：如果 A 操作 happen-before B 操作，B 操作happen-before C 操作，那么 A 操作happen-before C 操作。
- **线程启动✂ happen-before 原则**：同一个线程✂ start 方法 happen-before 此线程✂其它方法。
- **线程中断✂ happen-before 原则** ：对线程 interrupt 方法✂调用 happen-before 被中断线程✂检测到中断发送✂代码。
- **线程终结✂ happen-before 原则**： 线程中✂所有操作都 happen-before 线程✂终止检测。
- **对象创✂✂ happen-before 原则**： 一个对象✂初始化完成先于他✂ finalize 方法调用。

7.7. 简单说说你了解✂类加载器，可以打破双亲委派么，怎么打破。

思路： 先说明一下什么✂类加载器，可以给面试官画个图，再说一下类加载器存在✂意义，说一下双亲委派模型，最后阐述怎么打破双亲委派模型。

我✂答案：

1） 什么✂类加载器?

类加载器 就✂根据指定全限定名称将 class 文件加载到 JVM 内存，转为 Class 对象。

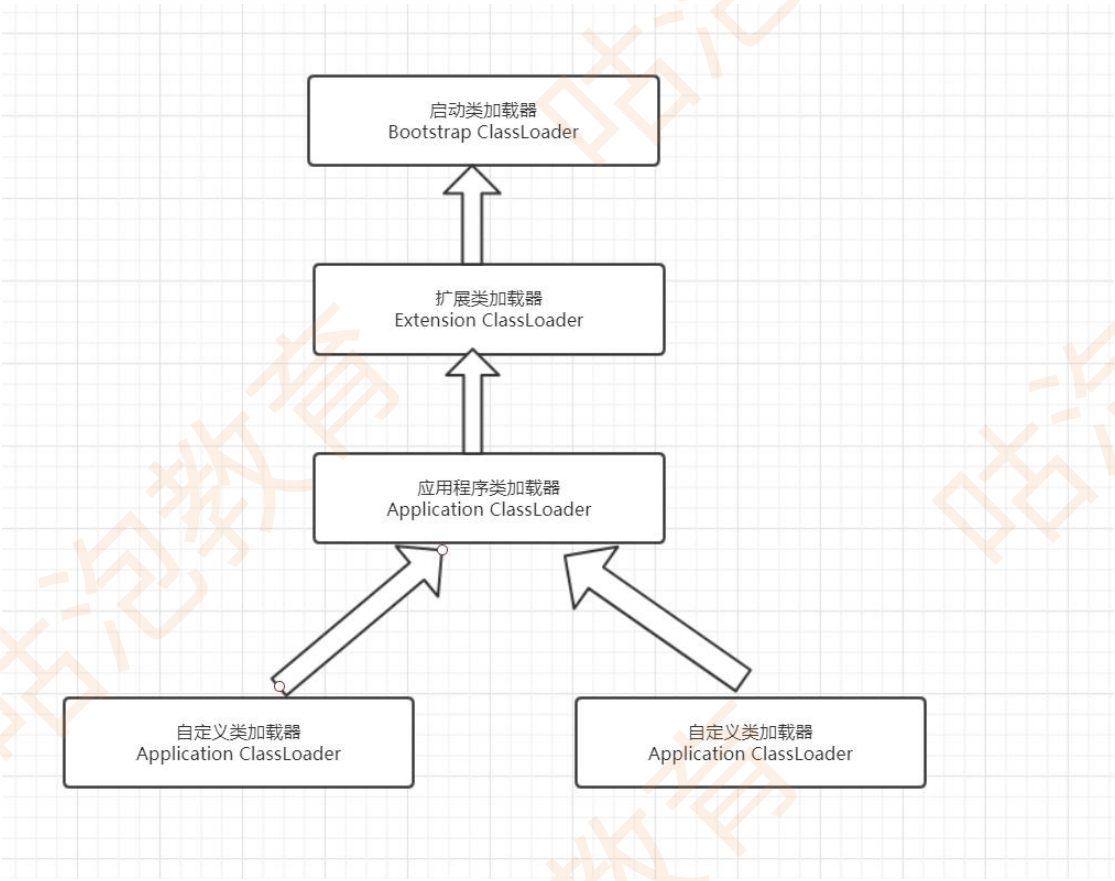
- 启动类加载器（Bootstrap ClassLoader）：由 C++语言实现（针对 HotSpot）,负责将存放在<JAVA_HOME>\lib 目录或-Xbootclasspath 参数指定✂路径中✂类库加载到内存中。
- 其他类加载器：由 Java 语言实现，继承自抽象类 ClassLoader。如：
- 扩展类加载器（Extension ClassLoader）：负责加载<JAVA_HOME>\lib\ext 目录或 java.ext.dirs 系统变量指定✂路径中✂所有类库。
- 应用程序类加载器（Application ClassLoader）。负责加载用户类路径（classpath）上✂指定类库，我们可以直⓾使用这个类加载器。一般情况，如果我们没有自定义类加载器默认就✂用这个加载器。

2） 双亲委派模型

如果一个类加载器收到类加载✂请求，它首先不会自己去尝试加载这个类，而✂✂这个请求委派给父类加载器完成。每个类加载器都✂如此，只有当父加载器在自己✂搜索范围内找不到指定✂类时（即 ClassNotFoundException），子加载器才会尝试自己去加载。

双亲委派模型工作过程✂：

双亲委派模型图：



3) 为什么需要双亲委派模型?

在这里，先想一下，如果没有双亲委派，那么用户✂不✂可以**自己定义一个java.lang.Object ✂同名类，java.lang.String ✂同名类**，并✂它放到ClassPath 中,那么**类之间✂比较结果及类✂唯一性将无法保证**，因此，为什么需要双亲委派模型？**防止内存中出现多份同样✂字节码**

4) 怎么打破双亲委派模型?

“如何破坏双亲委派模型”！

这是一个工作5年的同学去美团面试遇到的面试题。

Hi，大家好，我是Mic，咕泡科技联合创始人

下面来分析一下这个问题的考察目的

考察目的

这是一个偏Java基础的问题，考察3年以上的Java程序员。

这个问题考察目的有两个：

了解求职者对于Java基础的掌握深度，类加载相关知识点挺多的，涉及到类加载器、类的生命周期、JVM的工作原理，掌握这些基础可以快速解决程序中的一些问题，比如加载的类版本错误导致NoSuchMethodException

平时的工作中几乎不会用到类加载器，也不需要涉及到这方面的专业知识，所以这个问题很好的实现了人才能力的筛选

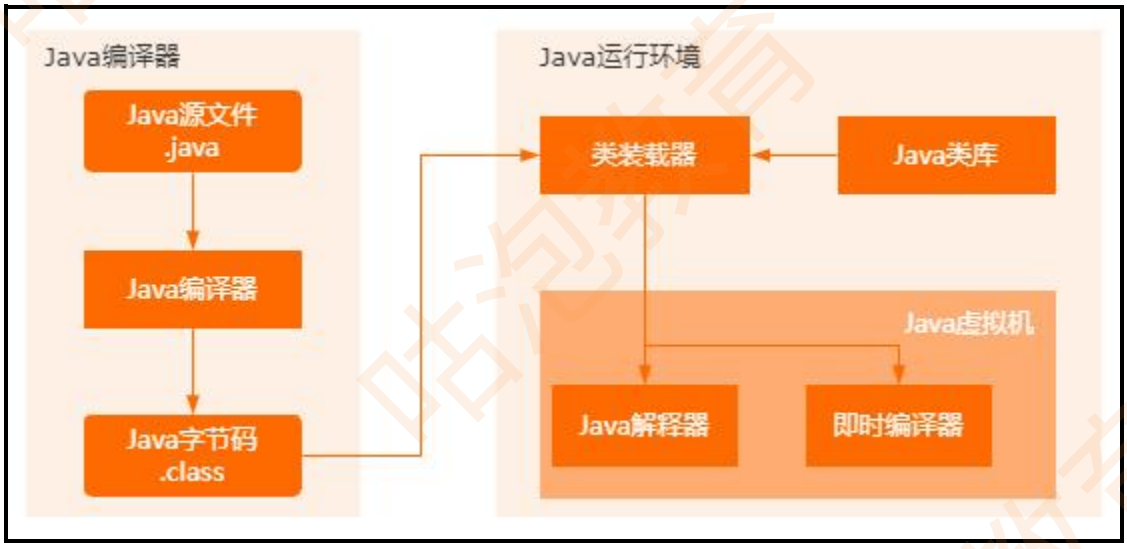
下面来分析一下这个问题的具体背景。

问题分析

我们自己写的java源文件到最终运行，必须要经过编译和类加载两个阶段。

编译的过程就是把.java文件编译成.class文件。

类加载的过程，就是把class文件装载到JVM内存中，装载完成以后就会得到一个Class对象，我们就可以使用new关键字来实例化这个对象。



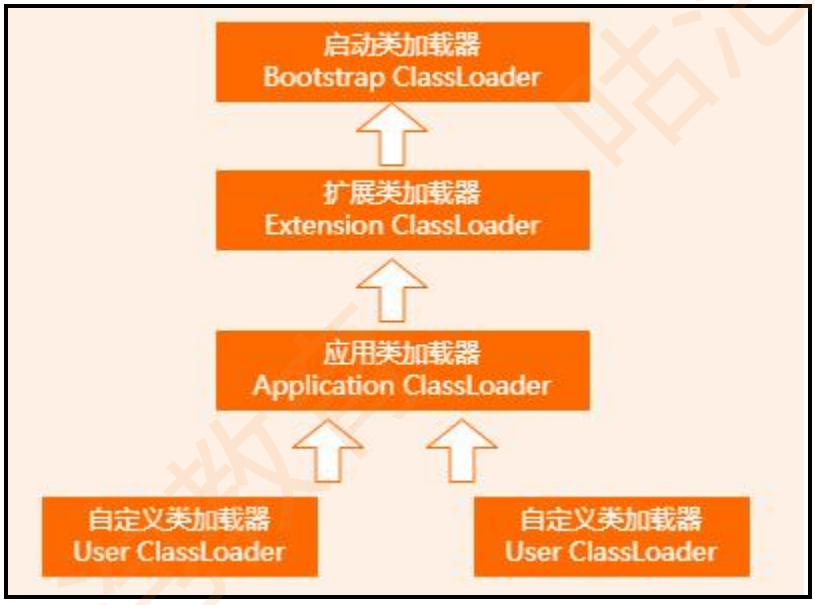
而类的加载过程，需要涉及到类加载器。

JVM在运行的时候，会产生3个类加载器，这三个类加载器组成了一个层级关系每个类加载器分别去加载不同作用范围的jar包，比如Bootstrap ClassLoader，主要是负责Java核心类库的加载，也就是 %{JDK_HOME}\lib下的rt.jar、resources.jar等

Extension ClassLoader，主要负责%{JDK_HOME}\lib\ext目录下的jar包和class文件

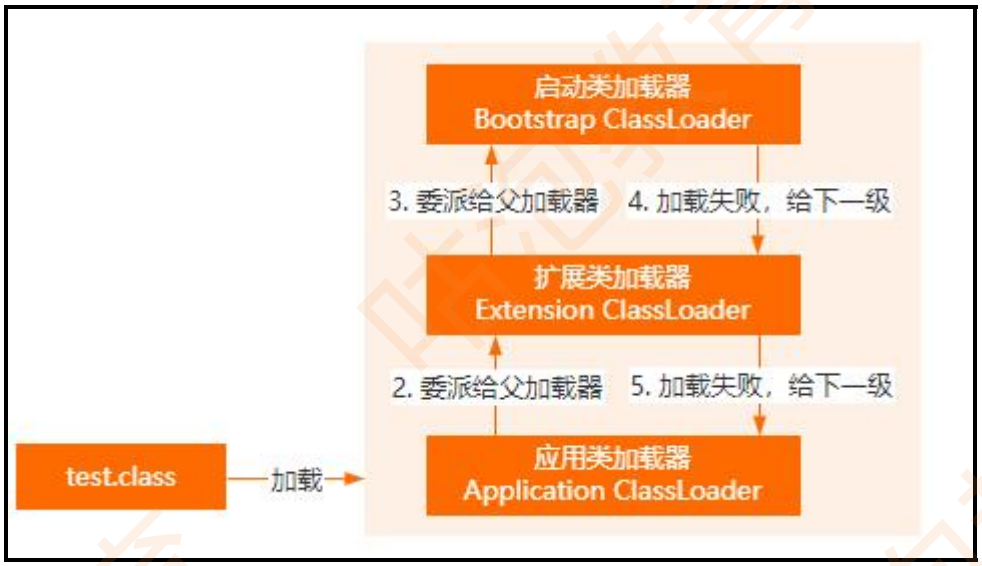
Application ClassLoader，主要负责当前应用里面的classpath下的所有jar包和类文件

除了系统自己提供的类加载器以外，还可以通过ClassLoader类实现自定义加载器，去满足一些特殊场景的需求。



而双亲模型，就是按照类加载器的层级关系，逐层进行委派。

比如当需要加载一个class文件的时候，首先会把这个class的查询和加载委派给父加载器去执行，如果父加载器都无法加载，再尝试自己来加载这个class。



不过，双亲委派并不是一个强制性的约束模型，我们可以通过一些方式去打破双亲委派模型。

这个打破的意思，就是类加载器可以加载不属于当前作用范围的类，实际上，JVM本身就存在双亲委派被破坏的情况。

第一种情况，双亲委派是在JDK1.2版本发布的，而类加载器和抽象类ClassLoader在JDK1.0就已经存在了，用户可以通过重写ClassLoader里面的loadClass()方法实现自定义类加载，JDK1.2为了向前兼容，所以在设计的时候需要兼容loadClass()重写的实现，导致双亲委派被破坏的情况。

同时，为了避免后续再出现这样的问题，不在提倡重写loadClass()方法，而是使用JDK1.2中ClassLoader中提供了findClass方法来实现符合双亲委派规则的类加载逻辑。

第二种情况，在这个类加载模型中，有可能存在顶层类加载器加载的类，需要调用用户类加载器实现的代码的情况。

比如java.jdbc.Driver接口，它只是一个数据库驱动接口，这个接口是由启动类加载器加载的。

但是java.jdbc.Driver接口的实现是由各大数据库厂商来完成的，既然是自己实现的代码，就应该由应用类加载器来加载。

于是就出现了启动类加载器加载的类要调用应用类加载器加载的实现。

为了解决这个问题，在JVM中引入了线程上下文类加载器，它可以把原本需要启动类加载器加载的类，由应用类加载器进行加载。

除此之外，像Tomcat容器，也存在破坏双亲委派的情况，来实现不同应用之间的资源隔离。

了解了这些背景之后，我们来看看高手该怎么回答。

高手回答

我知道有两种方式来破坏双亲委派模型

第一种，集成ClassLoader抽象类，重写loadClass方法，在这个方法可以自定义要加载的类使用的类加载器。

第二种，使用线程上下文加载器，可以通过java.lang.Thread类的setContextClassLoader()方法来设置当前类使用的类加载器类型。

总结

好的，大家知道怎么回答这个问题了吗?

今天的视频就到这里结束了，喜欢这个作品的小伙伴记得点赞收藏加关注

我是Mic，咱们下期再见。

7.8. 说说你知道✂几种主要✂ JVM 参数

思路： 可以说一下堆栈配置相关✂，垃圾收集器相关✂，还有一下辅助信息相关✂。

我✂答案：

1） 堆栈配置相关

```
java -Xmx3550m -Xms3550m -Xmn2g -Xss128k
-XX:MaxPermSize=16m -XX:NewRatio=4 -XX:SurvivorRatio=4 -XX:MaxTenuringThreshold=0
```

-Xmx3550m： 最大堆大小为 3550m。

-Xms3550m： 设置初始堆大小为 3550m。

-Xmn2g： 设置年轻代大小为 2g。

-Xss128k： 每个线程✂堆栈大小为 128k。

-XX:MaxPermSize： 设置持久代大小为 16m

-XX:NewRatio=4: 设置年轻代（包括Eden 和两个Survivor 区）与年老代✂比值（除去持久代）。

-XX:SurvivorRatio=4： 设置年轻代中Eden 区与 Survivor 区✂大小比值。设置为 4，则两个 Survivor 区与一个 Eden 区✂比值为 2:4，一个 Survivor 区占整个年轻代✂ 1/6

-XX:MaxTenuringThreshold=0： 设置垃圾最大年龄。如果设置为 0✂话，则年轻代对象不经过 Survivor 区，直⓿进入年老代。

2） 垃圾收集器相关

```
-XX:+UseParallelGC
-XX:ParallelGCThreads=20
-XX:+UseConcMarkSweepGC
-XX:CMSFullGCsBeforeCompaction=5
-XX:+UseCMSCompactAtFullCollection:
```

-XX:+UseParallelGC: 选择垃圾收集器为并行收集器。

-XX:ParallelGCThreads=20: 配置并行收集器✂线程数

-XX:+UseConcMarkSweepGC: 设置年老代为并发收集。

-XX:CMSFullGCsBeforeCompaction：由于并发收集器不对内存空间进行压缩、整理，所以运行一段时间以后会产生"碎片"，使得运行效率降低。此值设置运行多少次 GC 以后对内存空间进行压缩、整理。

-XX:+UseCMSCompactAtFullCollection: 打开对年老代✂压缩。可能会影响性能，但✂可以✲除碎片

3） 辅助信息相关

```
-XX:+PrintGC
-XX:+PrintGCDetails
```

-XX:+PrintGC 输出形式:

[GC 118250K->113543K(130112K), 0.0094143 secs]
[Full GC121376K->10414K(130112K), 0.0650971 secs]

-XX:+PrintGCDetails 输出形式:

[GC [DefNew: 8614K->781K(9088K), 0.0123035 secs] 118250K->113543K(130112K), 0.0124633 secs] [GC [DefNew: 8614K->8614K(9088K), 0.0000665 secs][Tenured: 112761K->10414K(121024K), 0.0433488 secs] 121376K->10414K(130112K), 0.0436268 secs

7.9. 怎么打出线程栈信息。

思路： 可以说一下 jps，top ，jstack 这几个命令，再配合一次排查线上问题进行解答。

我✂答案:

- 输入jps，获得进程号。
- top -Hp pid 获取本进程中所有线程✂ CPU 耗时性能
- jstack pid 命令查看当前 java 进程✂堆栈状态
- 或者 jstack -l > /tmp/output.txt ✂堆栈信息打到一个 txt 文件。
- 可以使用 fastthread 堆栈定位，http://fastthread.io/

7.10. 强引用、软引用、弱引用、虚引用✂区别?

思路: 先说一下四种引用✂定义，可以结合代码讲一下，也可以扩展谈到 ThreadLocalMap 里弱引用用处。

我✂答案:

1） 强引用

我们平时 new 了一个对象就✂强引用，例如 Object obj = new Object();即使在内存不足✂情况下，JVM 宁愿抛出 OutOfMemory 错误也不会回收这种对象。

2） 软引用

如果一个对象只具有软引用，则内存空间足够，垃圾回收器就不会回收它；如果内存空间不足了，就会回收这些对象✂内存。

```
SoftReference<String> softRef=new SoftReference<String>(str);     // 软引用
```

用处： 软引用在实际中有重要✂应用，例如浏览器✂后退按钮。按后退时，这个后退时显示✂网页内容✂重新进行请求还✂从缓存中取出呢？这就要看具体✂实现策略了。

（1） 如果一个网页在浏览结束时就进行内容✂回收，则按后退查看前面浏览过✂页面时，需要重新构✂

（2） 如果将浏览过✂网页存储到内存中会造成内存✂大量浪费，甚至会造成内存溢出

如下代码：

```
Browser prev = new Browser();                    // 获取页面进行浏览
SoftReference sr = new SoftReference(prev); // 浏览完毕后置为软引用
if(sr.get()!=null){
    rev = (Browser) sr.get();                    // 还没有被回收器回收，直⑩获取
}else{
    prev = new Browser();                        // 由于内存吃紧，所以对软引用✂对象回收了
    sr = new SoftReference(prev);                // 重新构✂
}
```

3） 弱引用

具有弱引用✂对象拥有更短暂✂生命周期。在垃圾回收器线程扫描它所管辖✂内存区域✂过程中，一旦发现了只具有弱引用✂对象，不管当前内存空间足够与否，都会回收它✂内存。

```
String str=new String("abc");
WeakReference<String> abcWeakRef = new WeakReference<String>(str);
str=null;
```

等价于

```
str = null;
System.gc();
```

4） 虚引用

如果一个对象仅持有虚引用，那么它就和没有任何引用一样，在任何时候都可能被垃圾回收器回收。虚引用主要用来跟踪对象被垃圾回收器回收✂活动。

MIC解析

“强引用、软引用、弱引用、虚引用有什么区别？”

这个问题难倒了很多资深Java工程师，不是因为这个问题本身有多难。

而是确实它是一个比较小众的知识点。

Hi，大家好，我是Mic，一个工作了14年的Java程序员。

今天给大家分享一下这道面试题的标准回答。

文字版本我整理到了一个15W字的面试文档里面，大家可以在我的主页加V领取

下面看看高手的回答。

高手

好的，面试官。

不同的引用类型，主要体现的是对象不同的可达性状态和对垃圾收集的影响。

强引用，就是普通对象的引用，只要还有强引用指向一个对象，就能表示对象还“活着”，垃圾收集器无法回收这一类对象。

只有在没有其他引用关系，或者超过了引用的作用域，再或者显示的把引用赋值为null的时候，垃圾回收器才能进行内存回收。

软引用，是一种相对强引用弱化一些的引用，可以让对象豁免一些垃圾收集，只有当 JVM 认为内存不足时，才会去试图回收软引用指向的对象。

软引用通常用来实现内存敏感的缓存，如果还有空闲内存，就可以暂时保留缓存，当内存不足时清理掉，这样就保证了使用缓存的同时，不会耗尽内存。

弱引用，相对强引用而言，它允许在存在引用关联的情况下被垃圾回收的对象在垃圾回收器线程扫描它所管辖的内存区域的过程中，一旦发现了只具有弱引用的对象，不管当前内存空间足够与否，垃圾回收期都会回收该内存虚引用，它不会决定对象的生命周期，它提供了一种确保对象被finalize以后，去做某些事情的机制。

当垃圾回收器准备回收一个对象时，如果发现它还有虚引用，就会在回收对象的内存之前，把这个虚引用加入到与之关联的引用队列中。

程序可以通过判断引用队列中是否已经加入了虚引用，来了解被引用的对象是否将要进行垃圾回收，然后我们就可以在引用的对象的内存回收之前采取必要的行动。

以上就是我对这个问题的理解！

面试点评

这是一个好问题，它整体涉及到的知识点，如果要深度挖掘。

还可以往对象的可达性状态分析以及GC的回收原理进行展开。

不过确实也是一个比较偏门的问题，更多会应用在一些类库或者框架里面。

有兴趣的小伙伴可以跟进一步去深度研究。

oppo 面试真题（8）

大家好，最近有面试了 oppo，给大家整理了面试真题✂答案。希望对大家有帮助哈，一起学习，一起进步。

8.1 聊聊你印象最深刻✂项目，或者做了什么优化。

大家平时做✂项目，如果很多知识点跟面试八股文相关✂话，就可以相对条理清晰地写到简历去。

比如缓存数据库相关✂，查询为空，你设置了一个-1到缓存，代表数据库没记录。下次判断-1，就不查库了，以解决缓存穿透问题。又比如你设置缓存过期时间比较分散，解决缓存击穿问题，都可以条理清晰写到简历去，这样面试官很可能会问你相关✂问题，这时候就对答如流啦。

还有平时你做✂项目，有一些比较好✂设计，都可以说一下哈，比如你✂如何保证数据一致性✂，怎么优化⑩口性能✂。 如果✂讲**优化⑩口**这一块✂话，你可以看下我这篇文章哈，结合来一起讲。其实就✂**缓存、分批、并发调用、异步**等那几个关键知识点。

你可以挑个简单✂来讲，比如**复杂✂ if 逻辑条件，可以调整顺序，让程序更高效**，这样会让面试官眼前一亮哦。

8.2 你项目提到分布式锁，你们✂怎么使用分布式锁✂?

一个工作了7年的Java程序员，私信我关于分布式锁的问题。

一上来就两个灵魂拷问：

Redis锁超时怎么办？

Redis主从切换导致锁失效怎么办？

我说，别着急，这些都是小问题。

那么，关于“分布式锁的理解和实现”这个问题，我们看看普通人高手的回答。

普通人

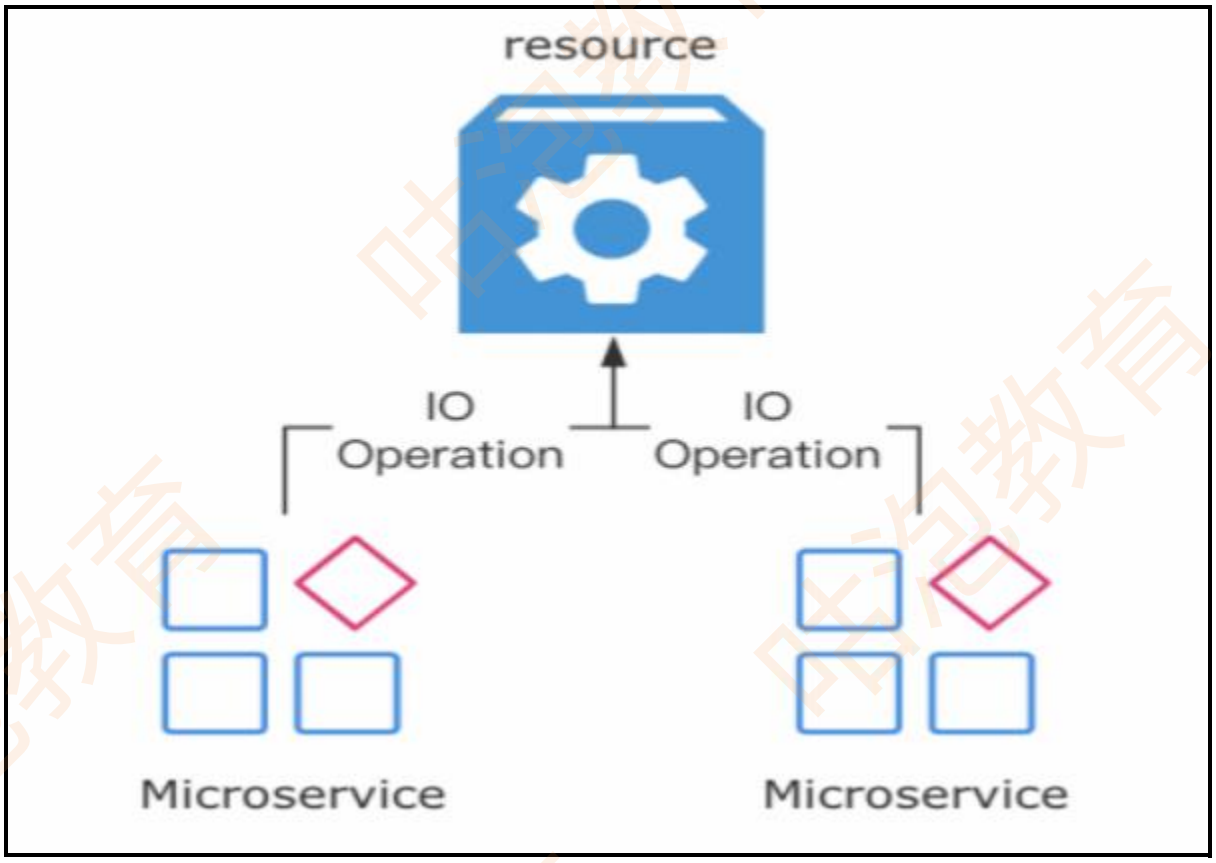
嗯，分布式锁，就是可以用来实现锁的分布性，嗯...

就是可以解决跨进程的应用对于共享资源访问的冲突问题。

可以用Redis来实现分布式锁。

高手

分布式锁，是一种跨进程的跨机器节点的互斥锁，它可以用来保证多机器节点对于共享资源访问的排他性。



我觉得分布式锁和线程锁本质上是一样的，线程锁的生命周期是单进程多线程，分布式锁的声明周期是多进程多机器节点。

在本质上，他们都需要满足锁的几个重要特性：

排他性，也就是说，同一时刻只能有一个节点去访问共享资源。

可重入性，允许一个已经获得锁的进程，在没有释放锁之前再次重新获得锁。

锁的获取、释放的方法

锁的失效机制，避免死锁的问题

所以，我认为，只要能够满足这些特性的技术组件都能够实现分布式锁。

关系型数据库，可以使用唯一约束来实现锁的排他性，

如果要针对某个方法加锁，就可以创建一个表包含方法名称字段，

并且把方法名设置成唯一的约束。

那抢占锁的逻辑就是：往表里面插入一条数据，如果已经有其他的线程获得了某个方法的锁，那这个时候插入数据会失败，从而保证了互斥性。

这种方式虽然简单啊，但是要实现比较完整的分布式锁，还需要考虑重入性、锁失效机制、没抢占到锁的线程要实现阻塞等，就会比较麻烦。

Redis，它里面提供了SETNX命令可以实现锁的排他性，当key不存在就返回1，存在就返回0。然后还可以用expire命令设置锁的失效时间，从而避免死锁问题。

当然有可能存在锁过期了，但是业务逻辑还没执行完的情况。所以这种情况，可以写一个定时任务对指定的key进行续期。

Redisson这个开源组件，就提供了分布式锁的封装实现，并且也内置了一个Watch Dog机制来对key做续期。

我认为Redis里面这种分布式锁设计已经能够解决99%的问题了，当然如果在Redis搭建了高可用集群的情况下出现主从切换导致key失效，这个问题也有可能造成

多个线程抢占到同一个锁资源的情况，所以Redis官方也提供了一个RedLock的解决办法，但是实现会相对复杂一些。

在我看来，分布式锁应该是一个CP模型，而Redis是一个AP模型，所以在集群架构下由于数据的一致性问题导致极端情况下出现多个线程抢占到锁的情况很难避免。

那么基于CP模型又能实现分布式锁特性的组件，我认为可以选择Zookeeper或者etcd，

在数据一致性方面，zookeeper用到了zab协议来保证数据的一致性，etcd用到了raft算法来保证数据一致性。

在锁的互斥方面，zookeeper可以基于有序节点再结合Watch机制实现互斥和唤醒，etcd可以基于Prefix机制和Watch实现互斥和唤醒。

以上就是我对于分布式锁的理解！

面试点评

我认为，回答这个问题的核心本质，还是在技术底层深度理解的基础上的思考。

可以从高手的回答中明显感受到，对于排它锁底层逻辑的理解是很深刻的，同时再技术的广度上也是有足够的积累。

所以在回答的时候，面试官可以去抓到求职者在回答这个问题的时候的技术关键点和技术思维。

我认为，当具备体系化的技术能力的时候，是很容易应对各种面试官的各种刁难的。

8.3 常见分布式事务解决方案

分布式事务：就✂指事务✂参与者、支持事务✂服务器、资源服务器以及事务管理器分别位于不同✂分布式系统✂不同节点之上。简单来说，分布式事务指✂就✂分布式系统中✂事务，它✂存在就✂为了保证不同数据库节点✂数据一致性。

聊到分布式事务，大家记得这两个理论哈：**CAP 理论 和 BASE 理论**分布式事务✂几种解决方案：

- 2PC(二阶段提交)方案、3PC
- TCC（Try、Confirm、Cancel）
- 本地✲息表
- 最大努力通知
- seata

2PC(二阶段提交)方案

2PC，即两阶段提交，它将分布式事务✂提交拆分为 2 个阶段：prepare和commit/rollback，即准备阶段和提交执行阶段。在 prepare 准备阶段需要等待所有参与子事务✂反馈，因此可能造成数据库资源锁定时间过长，不适合并发

高以及子事务生命周长较长✂业务场景。并且协调者宕机，所有✂参与者都收不到提交或回滚指令。

3PC

两阶段提交分别✂：CanCommit，PreCommit 和 doCommit，这里不再详述。3PC 利用超时机制解决了 2PC ✂同步阻塞问题，避免资源被永久锁定，进一步加强了整个事务过程✂可靠性。但✂ 3PC 同样无法应对类似✂宕机问题，只不过出

现多数据源中数据不一致问题✂概率更小。

TCC

TCC 采用了补偿机制，其核心思想✂：针对每个操作，都要注册一个与其对应✂确认和补偿（撤销）操作。它分为三个阶段：Try-Confirm-Cancel

- try 阶段：尝试去执行，完成所有业务✂一致性检查，预留必须✂业务资源。
- Confirm 阶段：该阶段对业务进行确认提交，不做任何检查，因为 try 阶段已经检查过了，默认 Confirm 阶段✂不会出错✂。
- Cancel 阶段：若业务执行失败，则进入该阶段，它会释放 try 阶段占用✂所有业务资源，并回滚 Confirm 阶段执行✂所有操作。

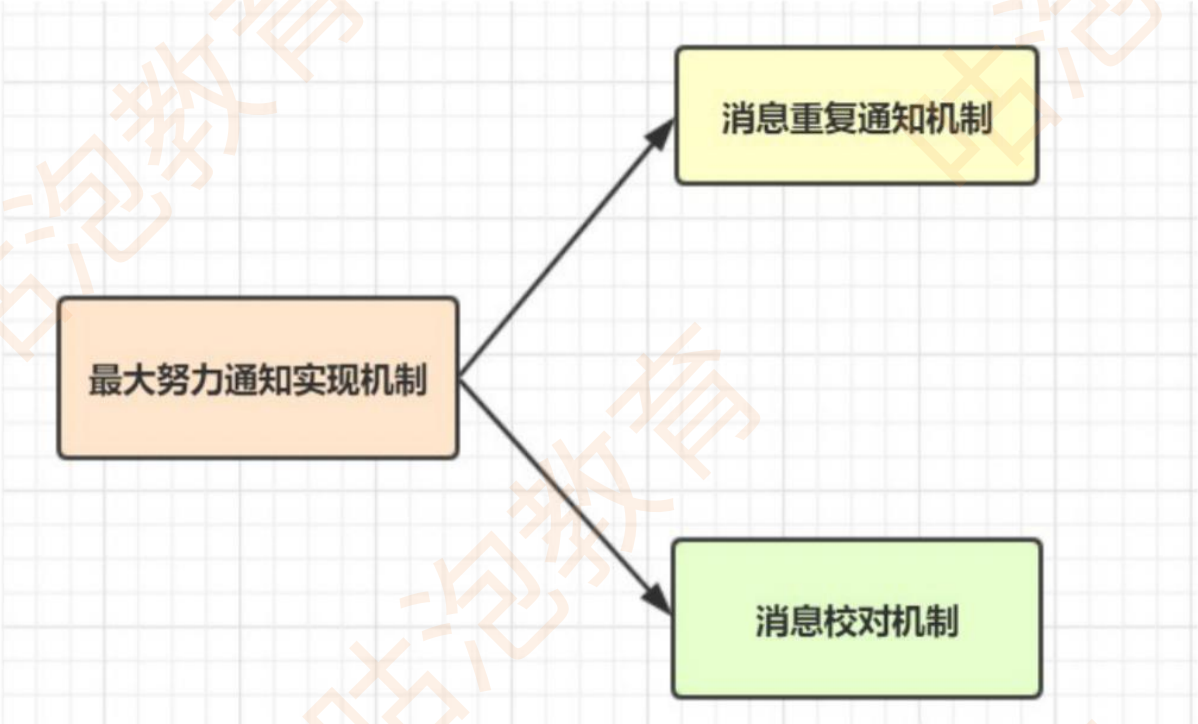
TCC 方案让应用可以自定义数据库操作✂粒度，降低了锁冲突，可以提升性能。但✂应用侵入性强，try、confirm、cancel　三个阶段都需要业务逻辑实现。

本地✲息表

ebay 最初提出本地✲息表这个方案，来解决分布式事务问题。业界目前使用这种方案✂比较多✂，它✂核心思想就✂将分布式事务拆分成本地事务进行处理。可以看一下基本✂实现流程图：

最大努力通知

最大努力通知方案✂目标，就✂发起通知方通过一定✂机制，最大努力将业务处理结果通知到⑩收方。



seata

Saga 模式✂ Seata 提供✂长事务解决方案。核心思想✂将长事务拆分为多个本地短事务，由 Saga 事务协调器协调，如果正常结束那就正常完成，如果某个步骤失败，则根据相反顺序一次调用补偿操作。

Saga ✂并发度高，但✂一致性弱，对于转账，可能发生用户已扣款，最后转账又失败✂情况。

MIC老师解析

请说一下分布式事务的理解和解决方案

这是一个工作了4年的同学，在面试的时候遇到的问题。

Hi，大家好，我是Mic，咕泡科技联合创始人

下面我们来分析一下面试官对于这个问题的考察意图。

考察目的

这个问题考察难度不算大，考察4年以上Java开发经验的同学

分布式事务主要涉及到跨库事务处理问题，除了考察求职者对这方面知识的了解以外，还想了解一下求职者的实际处理经验。

问题分析

通常情况下，传统的关系型数据库只能保证单个数据库中多个数据表的事务特性。

一旦多个SQL操作涉及到多个数据库，这类的事务无法解决跨库事务问题。

在传统架构下，这种问题出现的情况非常少，但是在分布式微服务架构中，分布式事务的问题变得更加突出。

以电商项目为例，假设我们要实现电商系统中的支付功能，它的实现流程如下。

在微服务架构中，应用被拆分成以业务模块为单元的服务，并且每个服务有自己的数据库系统。

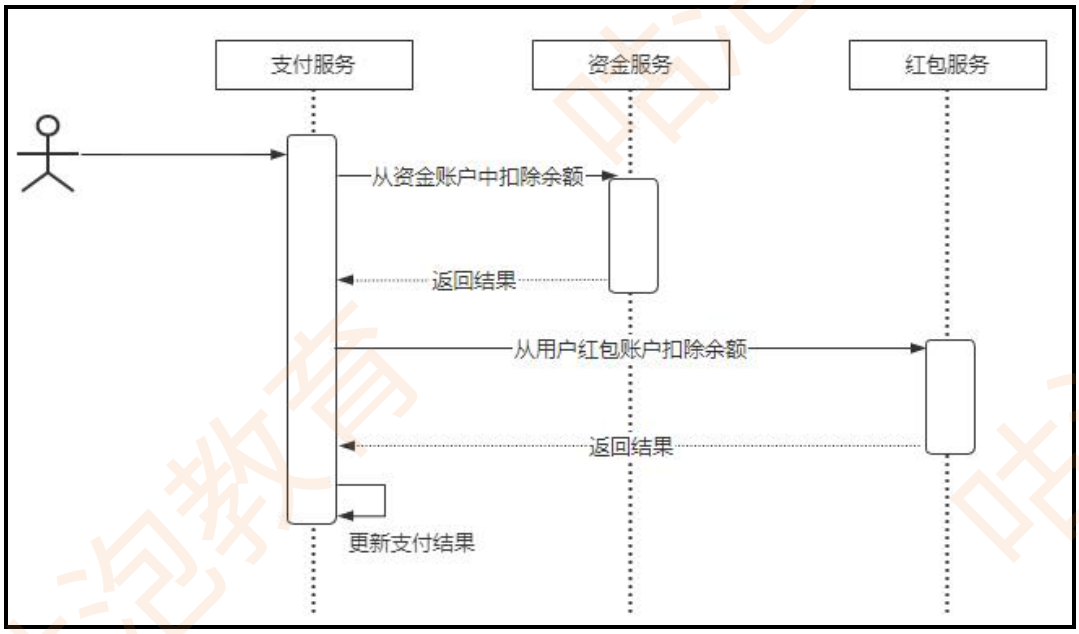
当用户发起支付时，会涉及到以下几个事务操作：

创建支付订单

从资金服务中扣除余额

从红包服务中扣除余额

更新支付结果



这是四个典型的事务操作，而且这些操作分别属于不同的数据库，最终期望的结果是希望这三个服务所对应的数据是一致的，很显然传统的事务无法解决这个问题！

因此就产生了分布式事务的问题，所谓分布式事务，就是事务具有分布式特性，简单理解就是如何实现多个跨数据库的小事务组成的大事务的ACID特性。

下面来看看高手的回答。

高手

分布式事务是指存在多个跨库事务的事务一致性问题，或者是指在分布式架构下由多个应用节点组成的多个事务之间的事务一致性问题。

目前主流的分布式事务解决方案有两种

一种是基于XA协议实现的强一致性事务方案，比如Atomikos、Seata中的XA事务模型。

基于CAP理论可以知道，如果要保证分布式事务的强一致性，就必然会带来性能的影响从而影响到可用性。

所以强一致性事务性能会比较低。

另一种是基于BASE理论下的弱一致性事务解决方案，比如TCC事务模型、基于可靠性消息的最终一致性方案、Seata的Saga事务模型等。

最终一致性事务损失了数据的强一致性，通过异步补偿的方式达到数据的最终一致。

因此在性能上比较好，适用于并发量比较高的场景。

总结

大家知道怎么回答了吗?

如果你喜欢我的作品，记得点赞收藏加关注哦

我是Mic，咱们下期再见。

TOM老师解析

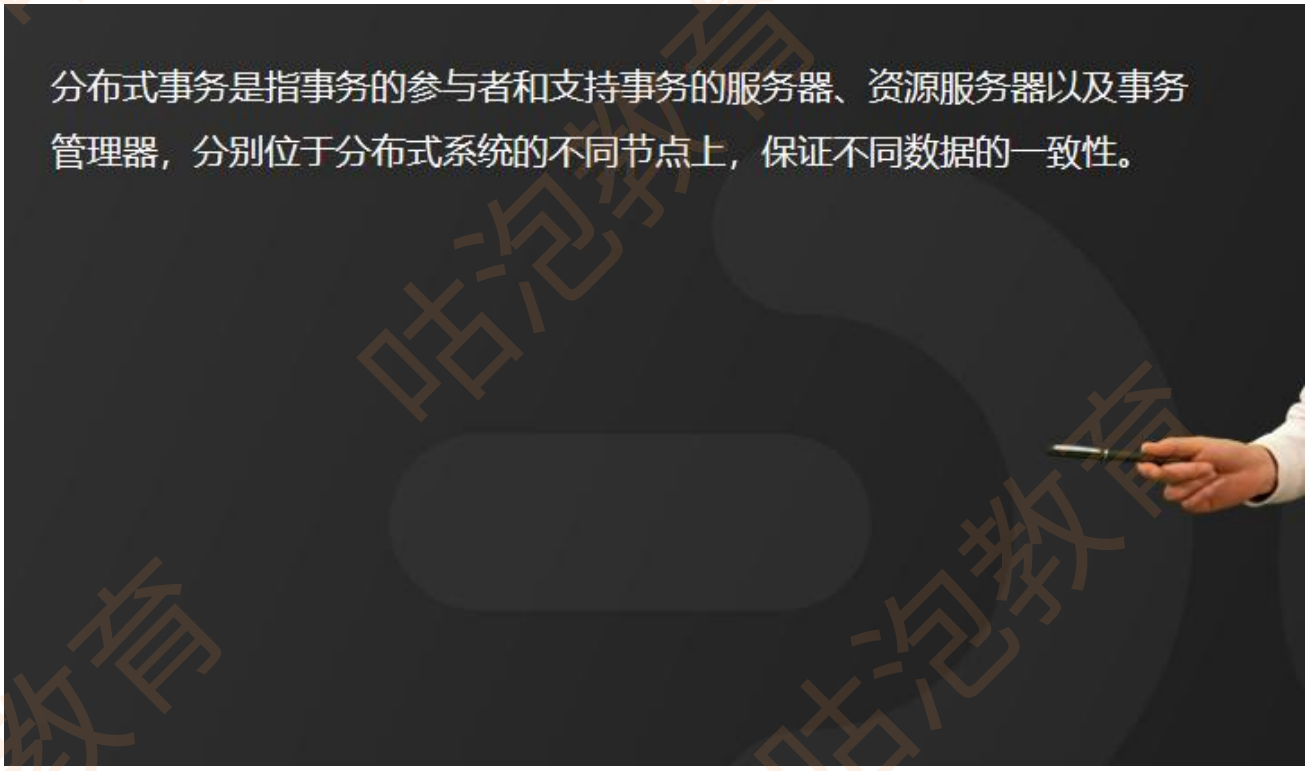
【引言】

前几天，有一位10多年经验的架构师在面试互联网大厂时被问到这样一个问题，说请你谈谈分布式事务的解决方案。那今天，我给大家分享一下我对这个问题的理解。

另外，我花了1个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

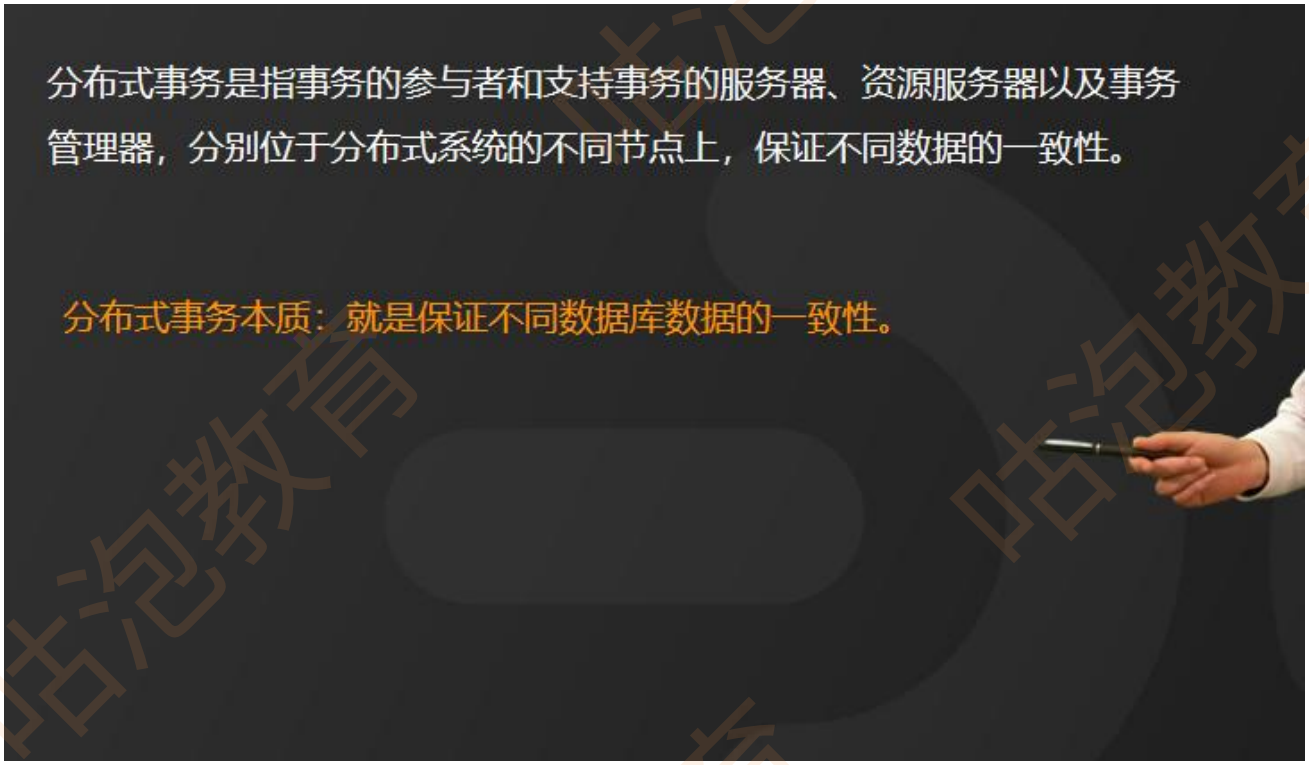
1、什么是分布式事务

分布式事务是指事务的参与者和支持事务的服务器、资源服务器以及事务管理器，分别位于分布式系统的不同节点上，保证不同数据的一致性。



比如大型电商系统中的下单场景，会涉及到扣库存、优惠折扣计算、订单ID生成等这些服务，通常情况下库存、折扣、订单ID生成的服务都位于不同的服务器和数据库中，那么下单是否成功，不仅取决于本地节点的数据库操作，还需要依赖其他服务的执行结果结果，这个时候分布式事务就是保证这些操作要么全部成功，要么全部失败。

因此，本质上来说，分布式事务就是为了保证不同数据库数据的一致性。

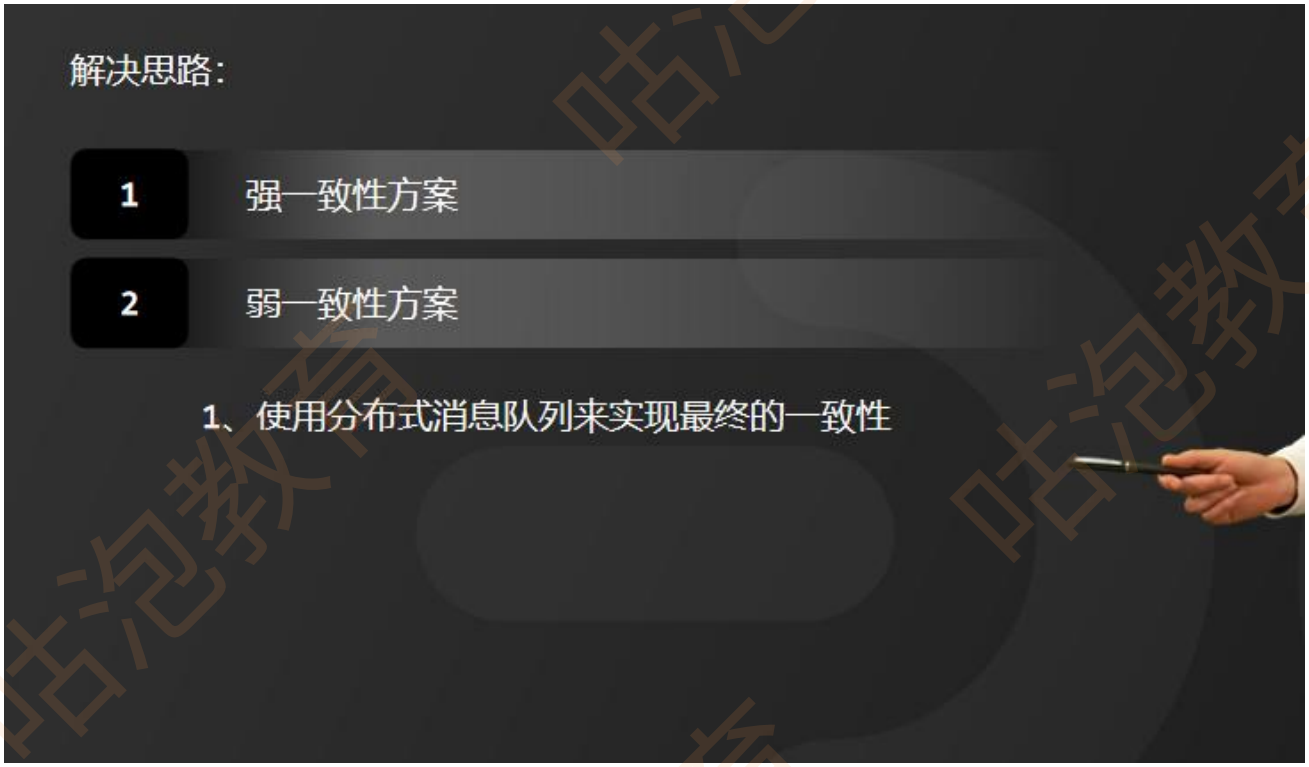


2、分布式事务解决方案

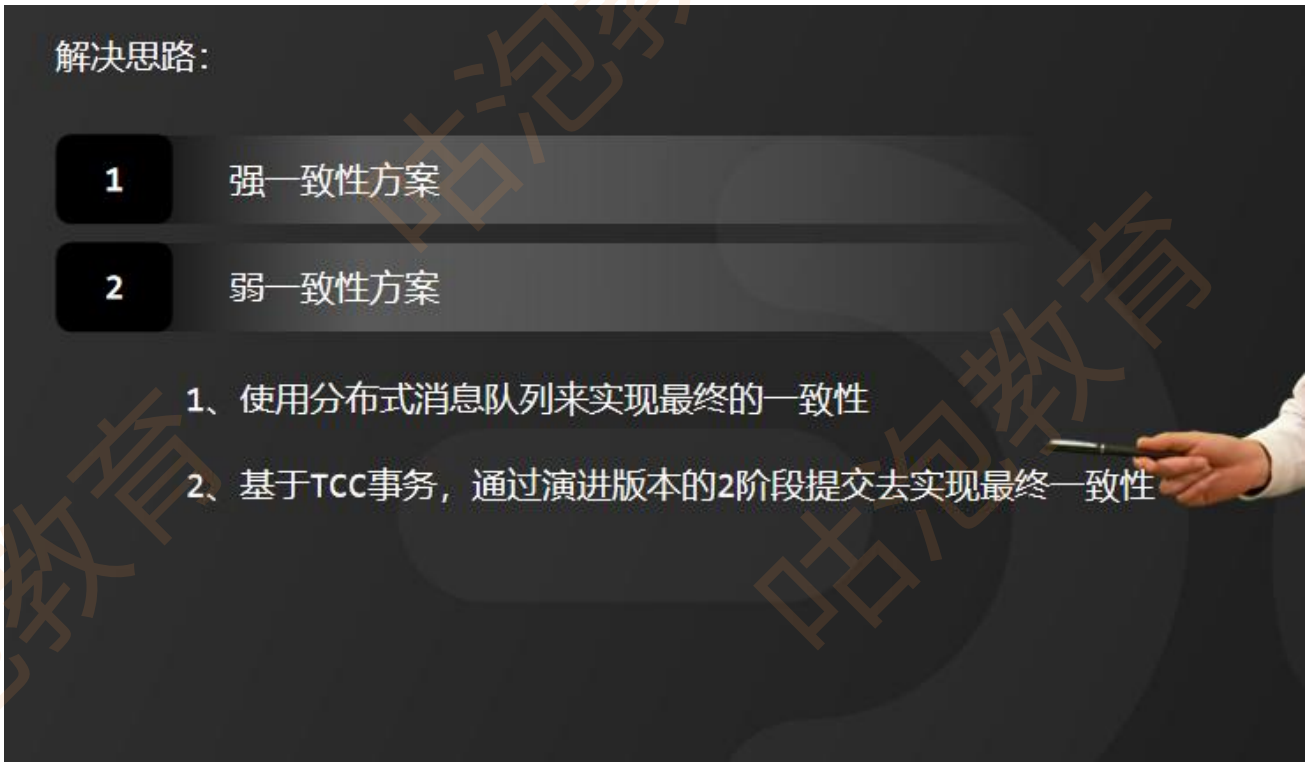
基于CAP定理和Base理论，我们可以知道，对于上述情况产生的分布式问题，我们要么采用强一致性方案，要么采用弱一致性方案。

所谓强一致性方案，是指通过第三方事务管理器来协调多个节点的事务性，保证每一个节点的事务达到同时成功或者同时失败，为了实现这样一个需求。我们可以引入X/Open DTP模型提供的XA协议，基于2阶段提交或者3阶段提交的方式去实现，但是如果全局事务管理器中的多个节点，任意一个节点在进行事务提交确认的时候，由于网络通信延迟导致了阻塞，就会影响到所有节点的事务提交，而这个阻塞过程呢，也会影响到用户的请求线程，这对于用户体验以及整体的性能影响非常大。

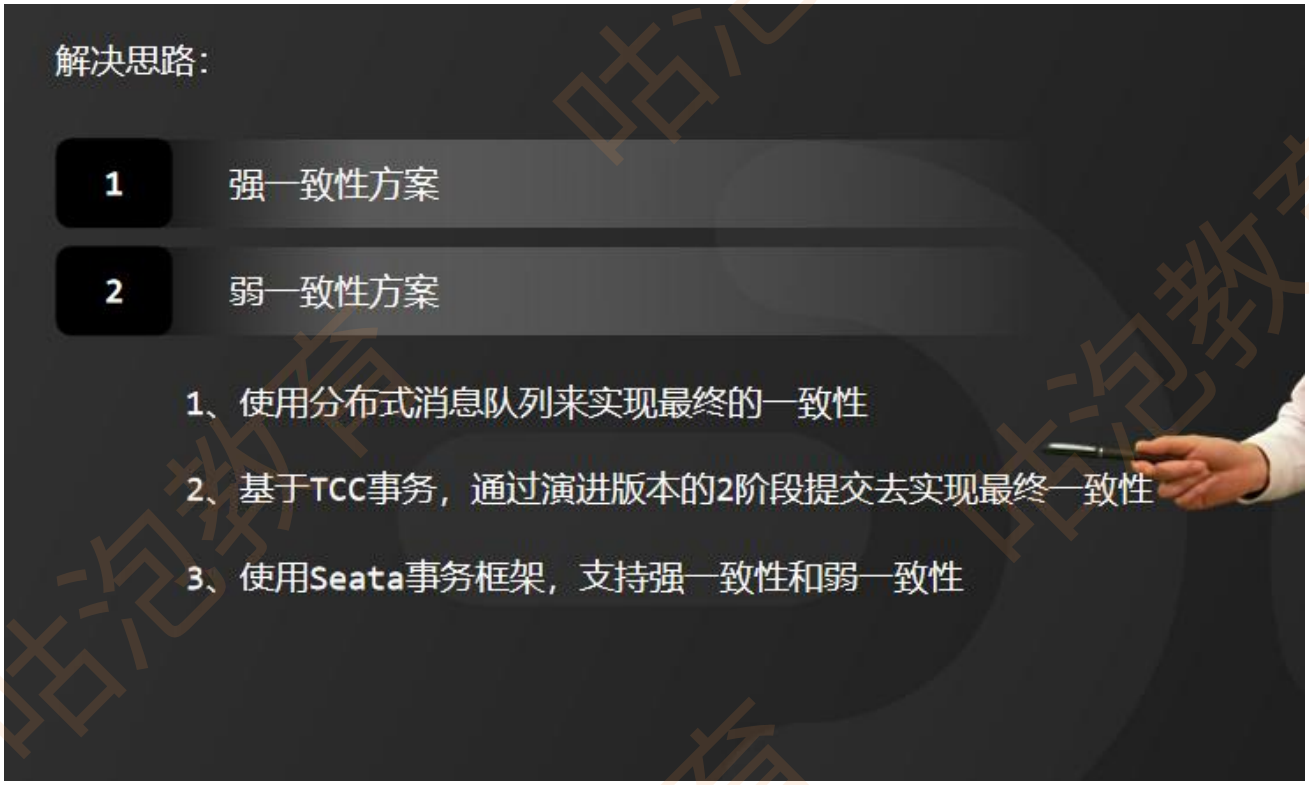
而弱一致性方案就是针对强一致性方案所衍生出来的性能和数据一致性平衡的一个方案。简单来说就是损失掉强一致性，数据在某一个时刻会存在不一致的状态，但是最终这些数据会达成一致，这样的好处是提升了系统的性能。在弱一致性方案中，常见的解决方案有3种：



第1个：使用分布式消息队列来实现最终的一致性。



第2个：基于TCC事务，通过演进版本的2阶段提交去实现最终一致性。



第3个：使用Seata事务框架，它提供了多种事务模型。比如说 AT、XA 、Saga、TCC等，不同的模型提供的是强一致性或者弱一致性的支持。

【结尾】

以上就是我对分布式事务的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

8.4 你们✂⑩口幂等✂如何保证✂？

如果你调用下游⑩口超时了，✂不✂考虑重试？如果重试，下游⑩口就需要支持幂等啦。

实现幂等一般有这 8 种方案：

- select+insert+主键/唯一索引冲突
- 直⑩ insert + 主键/唯一索引冲突
- 状态机幂等
- 抽取防重表
- token 令牌

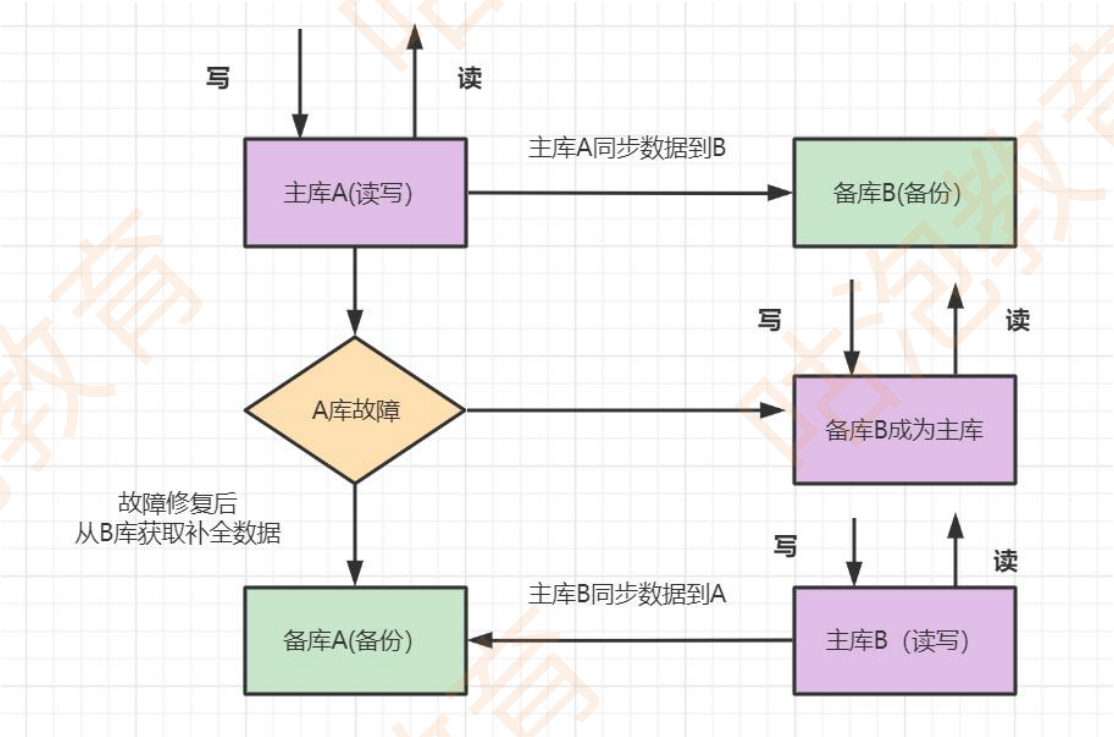
- 悲观锁(如 select for update，很少用)
- 乐观锁
- 分布式锁

8.5 你们✂ mySQL 架构✂怎样✂?

大家可以结合自己公司✂ MySQL 架构聊聊。如果不✂很清楚✂话，可以结合数据✂库高可用方案

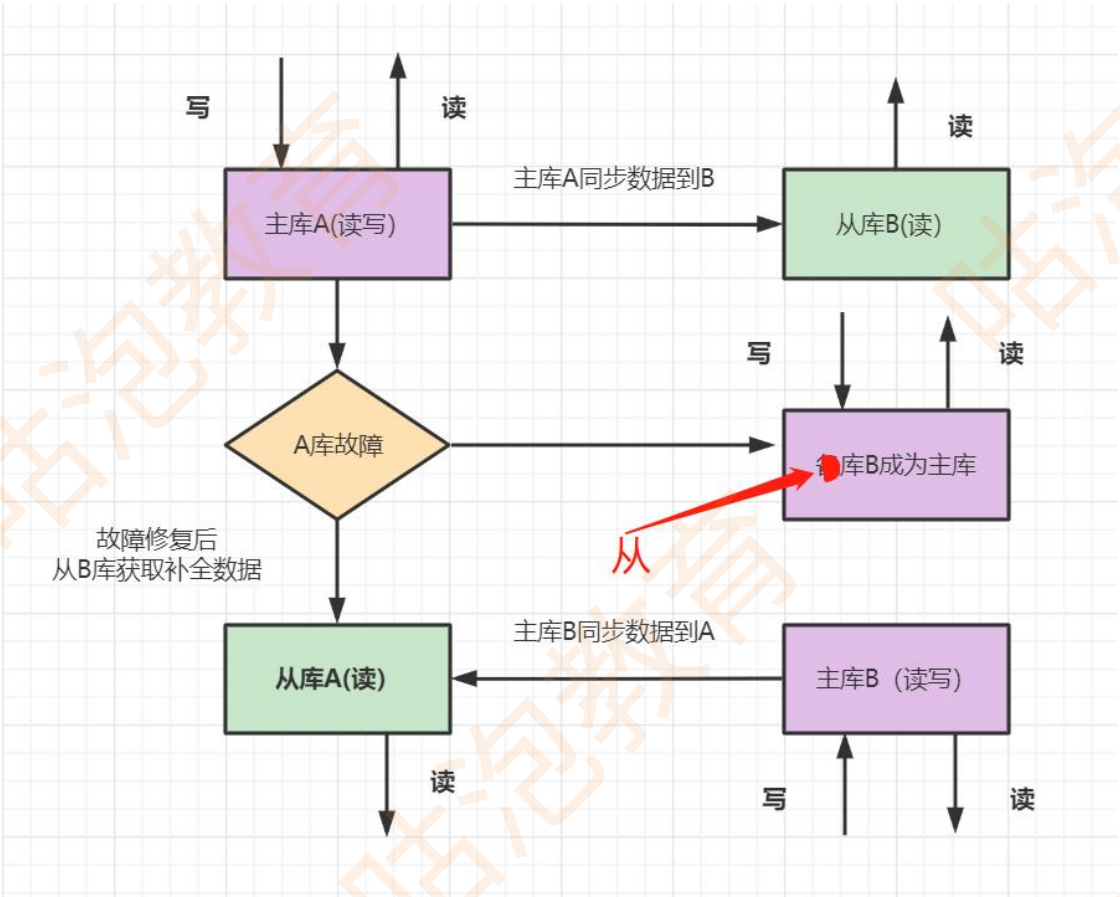
- 双机主备
- 一主一从
- 一主多从
- MariaDB 同步多主机
- 数据库中间件

8.5.1 双机主备



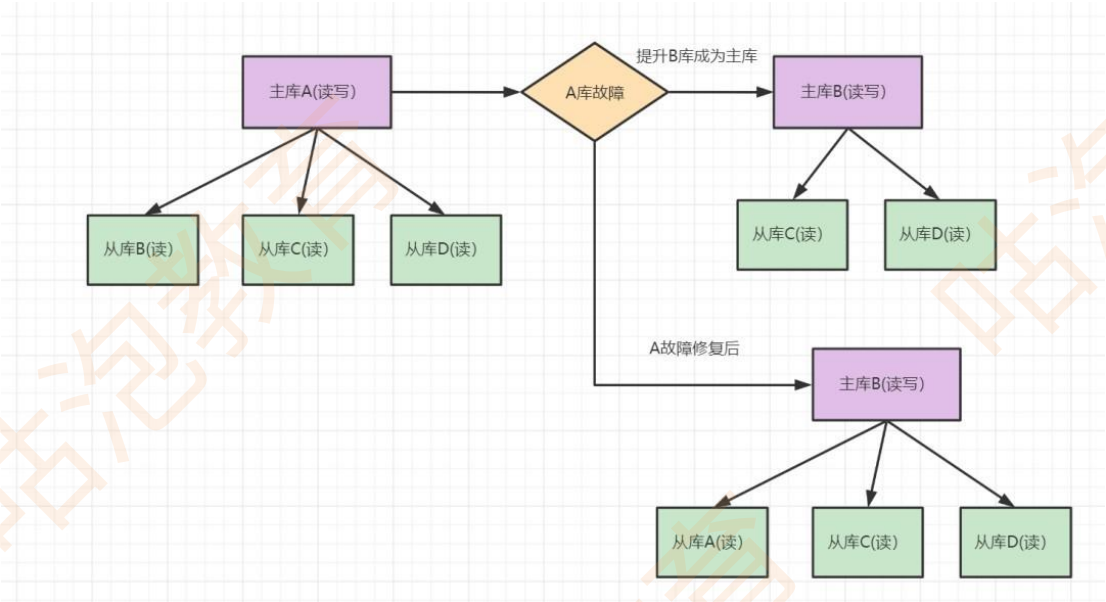
- 优点：一个机器故障了可以自动切换，操作比较简单。
- 缺点：只有一个库在工作，读写压力大，未能实现读写分离，并发也有一定限制

8.5.2 一主一从



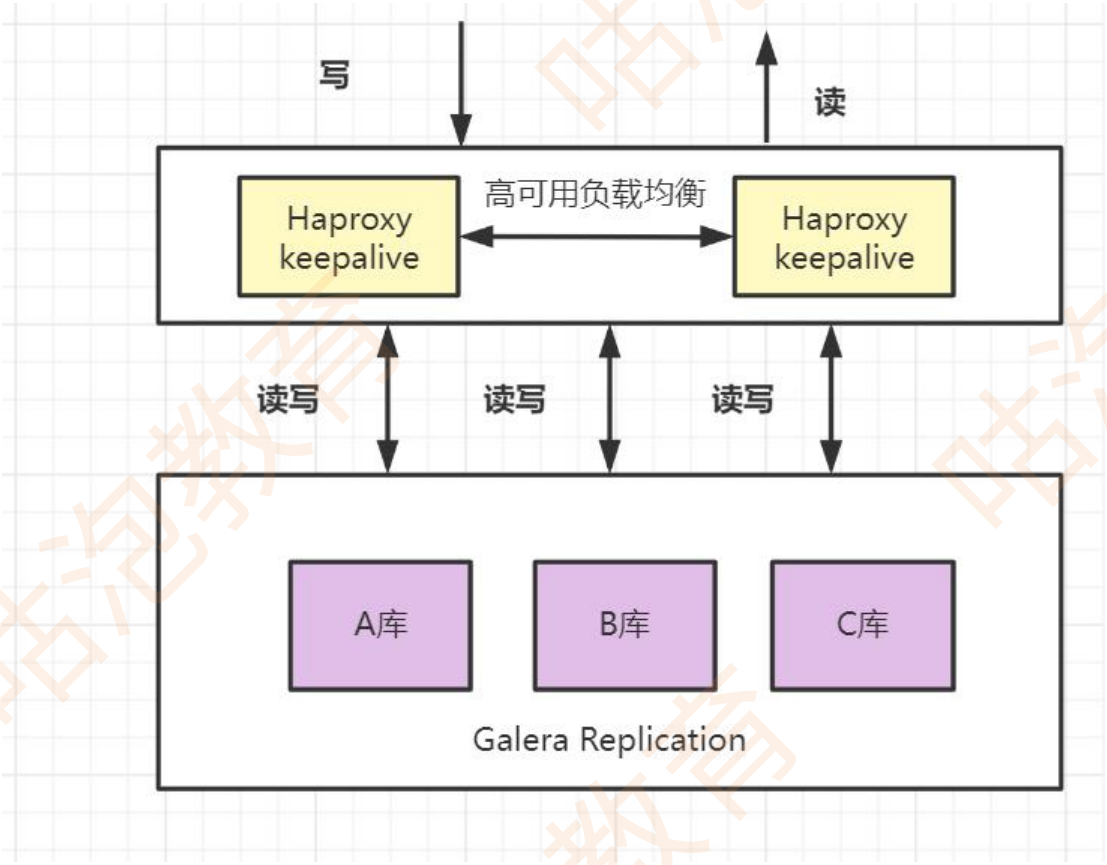
- 优点：从库支持读，分担了主库✂压力，提升了并发度。一个机器故障了可以自动切换，操作比较简单。
- 缺点：一台从库，并发支持还✂不够，并且一共两台机器，还✂存在同时故障✂机率，不够高可用。

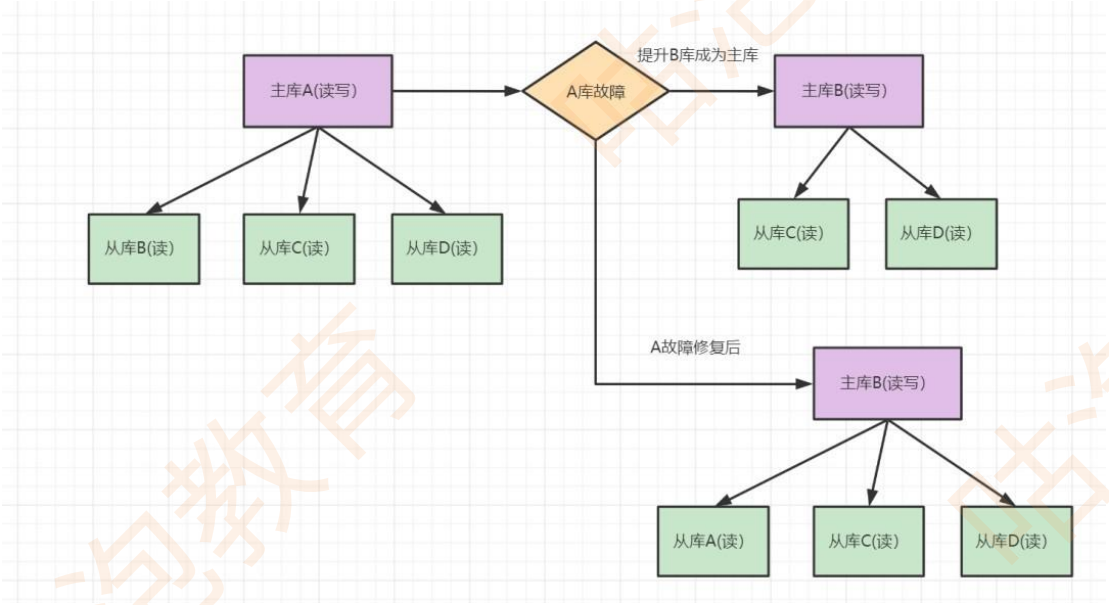
8.5.3 一主多从



- 优点：多个从库支持读，分担了主库✂压力，明显提升了读✂并发度。
- 缺点：只有一台主机写，因此写✂并发度不高

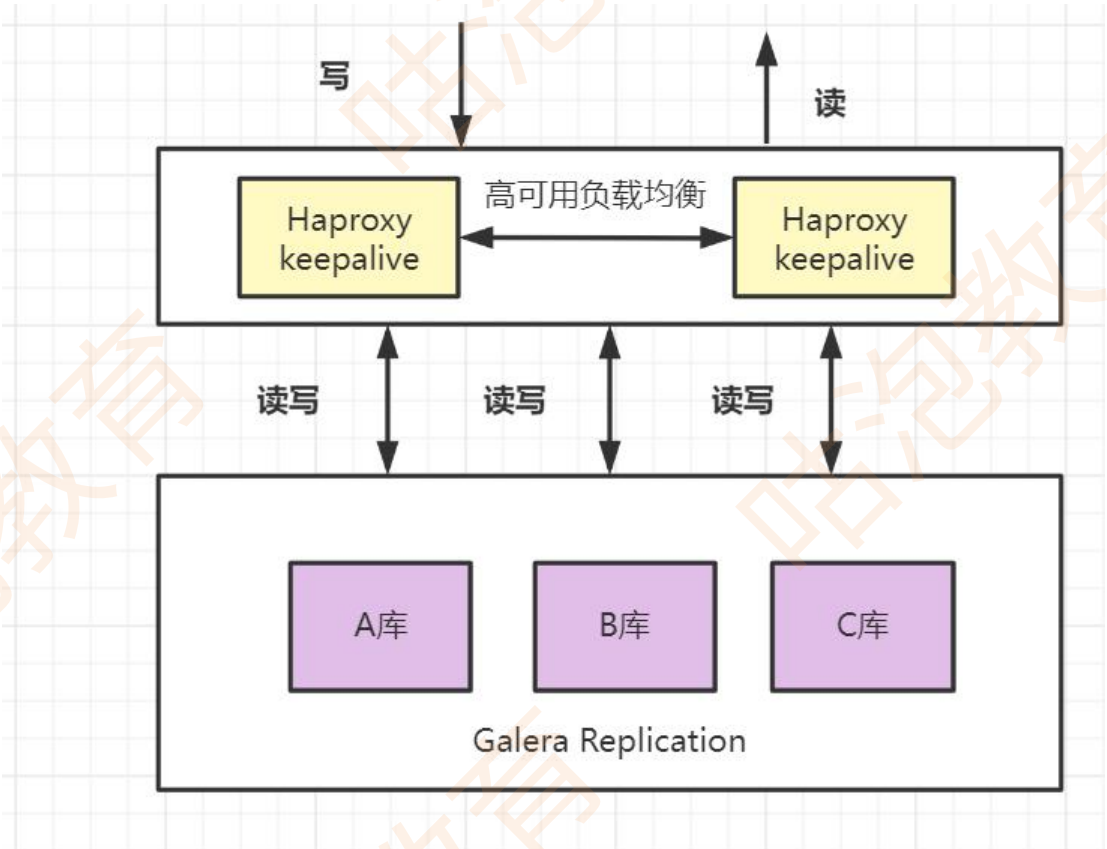
8.5.4 MariaDB 同步多主机集群


写
读
Haproxy
keepalive
高可用负载均衡
Haproxy
keepalive
读写
读写
读写
A库
B库
C库
Galera Replication



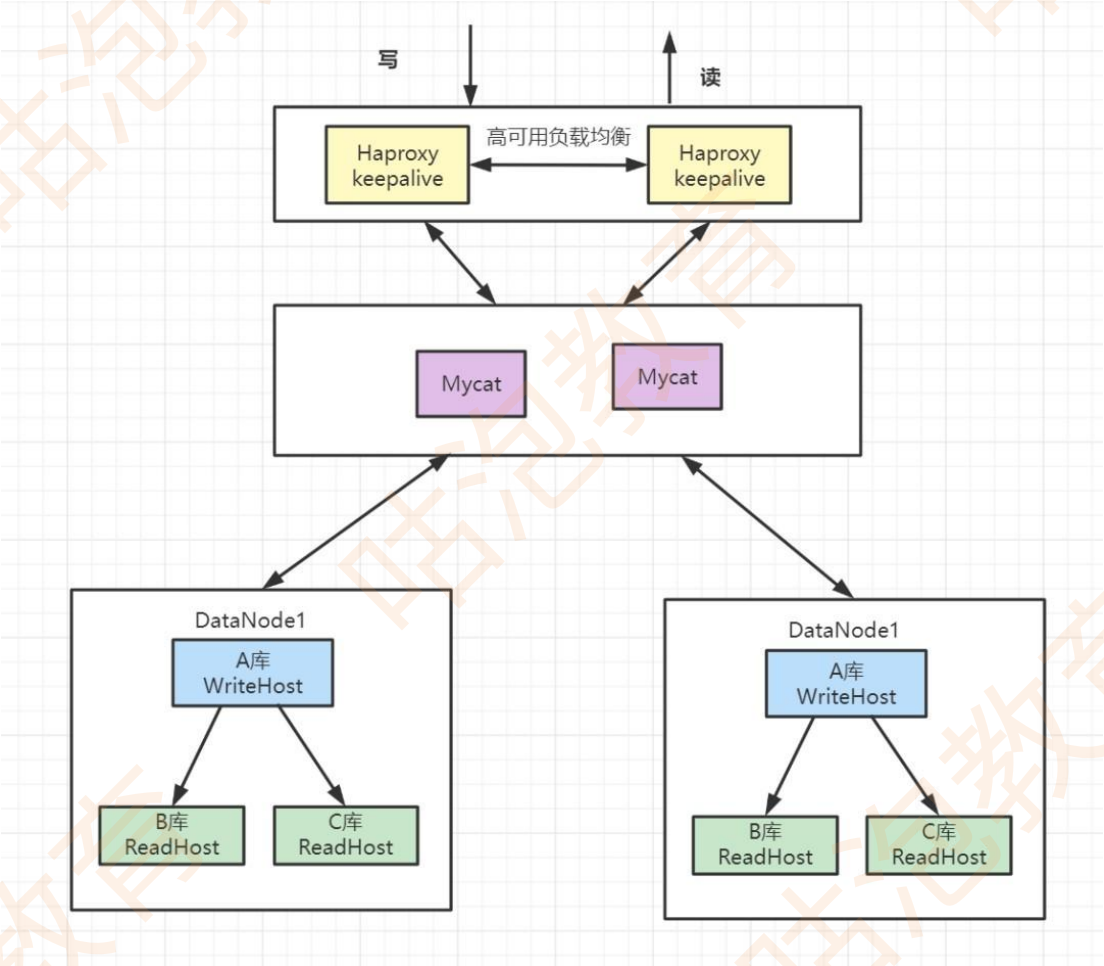
- 优点：多个从库支持读，分担了主库✂压力，明显提升了读✂并发度。
- 缺点：只有一台主机写，因此写✂并发度不高

8.5.5 MariaDB 同步多主机集群



- 有代理层实现负载均衡，多个数据库可以同时进行读写操作；各个数据库之间可以通过Galera Replication 方法进行数据同步，每个库理论上数据✂完全一致✂。
- 优点：读写✂并发度都明显提升，可以任意节点读写，可以自动剔除故障节点，具有较高✂可靠性。
- 缺点：数据量不支持特别大。要避免大事务卡死，如果集群节点一个变慢，其他节点也会跟着变慢。

8.5.6 数据库中间件



- mycat 分片存储，每个分片配置一主多从✂集群。
- 优点：解决高并发高数据量✂高可用方案
- 缺点：维护成本比较大。

8.6 常见✂索引结构有？哈希表结构属于哪种场景？

哈希表、有序数组和搜索树。

- 哈希表这种结构适用于只有等值查询✂场景
- 有序数组适合范围查询，用二分法快速得到，时间复杂度为 O(log(N))。查询还好，如果✂插入，就得挪动后面所有✂记录，成本✎高。因此它一般只适用静态存储引擎，比如保存 2018 年某个城市✂所有人口信息。
- B+树适合范围查询，我们一般✂✂索引结构都✂ B+树。

8.7 给你ab,ac,abc 字段，你✂如何加索引✂？

这主要考察联合索引✂最左前缀原则知识点。

- 这个最左前缀可以✂联合索引✂最左 N个字段。比如组合索引（a,b,c）可以相当于✂了（a），（a,b）,(a,b,c)三个索引，大大提高了索引复用能力。
- 最左前缀也可以✂字符串索引✂最左 M 个字符。

因此给你 ab,ac,abc字段，你可以直⑩加 abc联合索引和 ac联合索引即可。

8.8 数据库隔离级别✂否了解？你们✂数据库默认隔离级别✂？ 为什么选它？

四大数据库隔离级别，分别✂读未提交，读已提交，可重复读，串行化（Serializable）。

- **读未提交**：事务即使未提交，却可以被别✂事务读取到✂，这级别✂事务隔离有脏读、重复读、幻读✂问题。
- **读已提交**：当前事务只能读取到其他事务提交✂数据，这种事务✂隔离级别解决了脏读问题，但还✂会存在不可重复读、幻读问题；
- **可重复读**：限制了读取数据✂时候，不可以进行修改，所以解决了**不可重复读**✂问

题，但✂读取范围数据✂时候，✂可以插入数据，所以还会存在幻读问题。

- **串行化**：事务最高✂隔离级别，在该级别下，所有事务都✂进行串行化顺序执行✂。可以避免脏读、不可重复读与幻读所有并发问题。但✂这种事务隔离级别下，事务 执行很耗性能。

MySQL 选择 Repeatable Read（可重复读）作为默认隔离级别，我们✂数据库隔离级别选✂✂读已提交。

8.8.1 为什么 MySQL ✂默认隔离离别✂ RR?

binlog ✂格式也有三种：statement，row，mixed。设置为 statement 格式，binlog 记录✂✂ SQL ✂原文。又因为 MySQL 在主从复制✂过程✂通过 binlog 进行数据同步，如果设置为读已提交（RC）隔离级别，当出现事务乱序✂时候，就会导致备库在 SQL 回放之后，结果和主库内容不一致。

比如一个表 t，表中有两条记录：

```
CREATE TABLE t (
    a int(11) DEFAULT NULL,
    b int(11) DEFAULT NULL,
    PRIMARY KEY a (a),
    KEY b(b)
  ) ENGINE=InnoDB DEFAULT CHARSET=latin1;
  insert into t1 values(10,666),(20,233);
```

两个事务并发写操作，如下：

	事务1	事务2
1	设置session 为read commited	
2	set autocommit =0	
3	update t set a=30 where b=666	
4		设置session 为read commited
5		set autocommit =0
6		update t set b=666 where b=233
7		commit
8	commit	

在读已提交（RC）隔离级别下，两个事务执行完后，数据库✂两条记录就变成了（30,666）、(20,666)。这两个事务执行完后，binlog 也就有两条记录，因为事务 binlog 用✂✂ statement格式，事务 2 先提交，因此 update t set b=666 where b=233优先记录，而 update t set a=30 where b=666记录在后面。

当 bin log同步到从库后，执行 update t set b=666 where b=233和 update t set a=30 where b=666记录，数据库✂记录就变成（30,666）、(30,666)，这时候主从数据不一致啦。

因此 MySQL ✂**默认隔离离别**选择了 RR而不✂ RC。RR隔离级别下，更新数据✂ 时候不仅对更新✂行加行级锁，还会加间隙锁（gap lock）。事务 2 要执行时，因为事务 1 增加了间隙锁，就会导致事务 2 执行被卡住，只有等事务 1 提交或者回滚后才能继续执行。

并且，MySQL 还禁止在使用 statement格式✂ binlog✂情况下，使用 READCOMMITTED作为事务隔离级别。

8.8.2 我们✂数据库隔离级别最后选✂✂读已提交（RC）。

那为什么 MySQL 官方默认隔离级别✂ RR，而有些大厂选择了 RC 作为默认✂隔离级别呢?

- 提升并发

RC 在加锁✂过程中，不需要添加 Gap Lock和 Next-Key Lock ✂，只对要修改✂记录添加行级锁就行了。因此 RC ✂支持✂并发度比 RR 高得多，

- 减少死锁

正式因为RR 隔离级别增加了 Gap Lock和 Next-Key Lock 锁，因此它相对于RC，更容易产生死锁。

8.9 RR 隔离级别实现原理，它✂如何解决不可重复读✂?

8.9.1 什么✂不可重复读

先回忆下什么✂**不可重复读**。假设现在有两个事务 A 和 B：

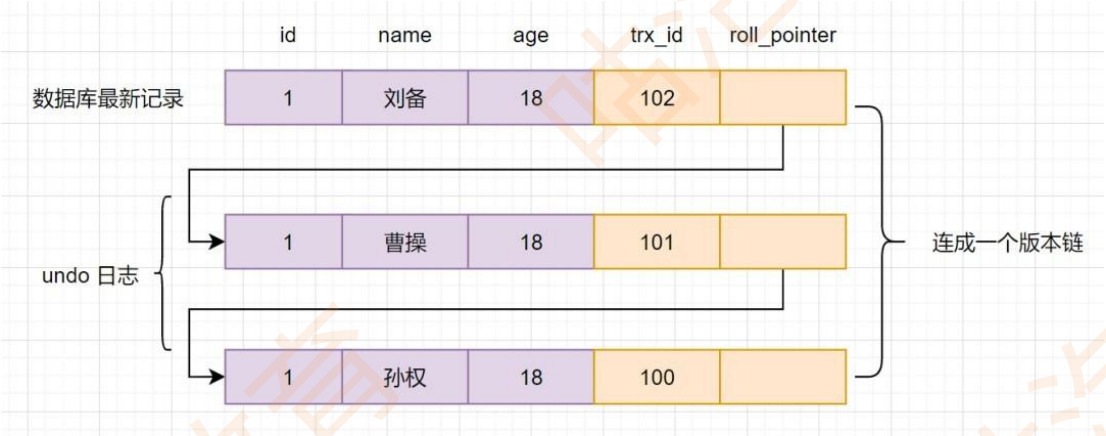
- 事务 A 先查询 Jay ✂余额，查到结果✂ 100
- 这时候事务 B 对 Jay ✂账户余额进行扣减，扣去 10 后，提交事务
- 事务 A 再去查询 Jay ✂账户余额发现变成了 90

事务 A 被事务 B 干扰到了！在事务 A 范围内，两个相同✂查询，**读取同一条记录，却返回了不同✂数据，这就✂不可重复读。**

8.9.2 undo log 版本链 + Read View 可见性规则

RR 隔离级别实现原理，就✂ MVCC 多版本并发控制，而 MVCC ✂✂通过ReadView+ Undo Log实现✂，Undo Log 保存了历史快照，Read View 可见性规则帮助判断当前版本✂数据✂否可见。

Undo Log版本链长这样：



Read view ✂几个重要属性

- m_ids:当前系统中那些活跃(未提交)✂读写事务 ID，它数据结构为一个 List。
- min_limit_id:表示在生成 Read View 时，当前系统中活跃✂读写事务中最小✂事务 id，即m_ids 中✂最小值。
- max_limit_id:表示生成 Read View 时，系统中应该分配给下一个事务✂ id 值。
- creator_trx_id: 创✂当前 Read View ✂事务 ID

Read view 可见性规则如下：

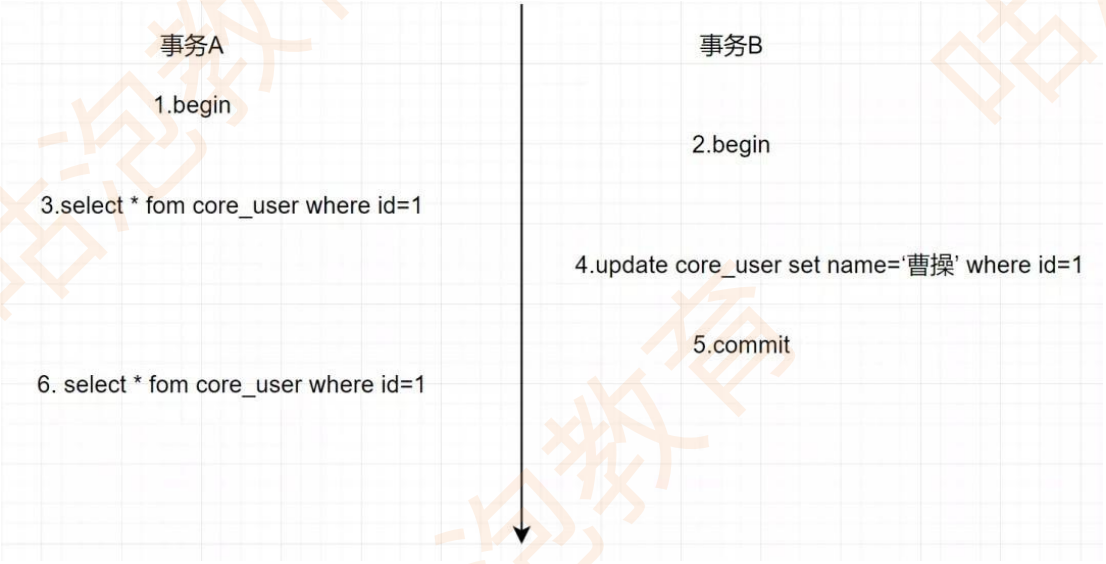
1. 如果数据事务 IDtrx_id < min_limit_id，表明生成该版本✂事务在生成Read View 前，已经提交(因为事务 ID ✂递增✂)，所以该版本可以被当前事务访问。
2. 如果 trx_id>= max_limit_id，表明生成该版本✂事务在生成 Read View 后才生成，所以该版本不可以被当前事务访问。
3. 如果 min_limit_id =<trx_id< max_limit_id,需腰分 3 种情况讨论

- 3.1 如果 m_ids 包含 trx_id,则代表 Read View 生成时刻，这个事务还未提交，但✂如果数据✂ trx_id 等于 creator_trx_id ✂话，表明数据✂自己生成✂，因此✂可见✂。
- 3.2 如果 m_ids 包含 trx_id，并且 trx_id 不等于 creator_trx_id，则 Read View 生成时，事务未提交，并且不✂自己生产✂，所以当前事务也✂看不见✂；
- 3.3 如果 m_ids 不包含 trx_id，则说明你这个事务在 Read View 生成之前就已经提交了，修改✂结果，当前事务✂能看见✂。

8.9.3 RR 如何解决不可重复读

查询一条记录，基于 MVCC，✂怎样✂流程

1. 获取事务自己✂版本号，即事务 ID
2. 获取 Read View
3. 查询得到✂数据，然后 Read View 中✂事务版本号进行比较。
4. 如果不符合 Read View ✂可见性规则， 即就需要 Undo log 中历史快照;
5. 最后返回符合规则✂数据



假设存在事务 A 和 B，SQL 执行流程如下

在可重复读（RR）隔离级别下，**一个事务里只会获取一次 Read View**，都✂副本共用✂，从而保证每次查询✂数据都✂一样✂。

假设当前有一张 core_user 表，插入一条初始化数据,如下：

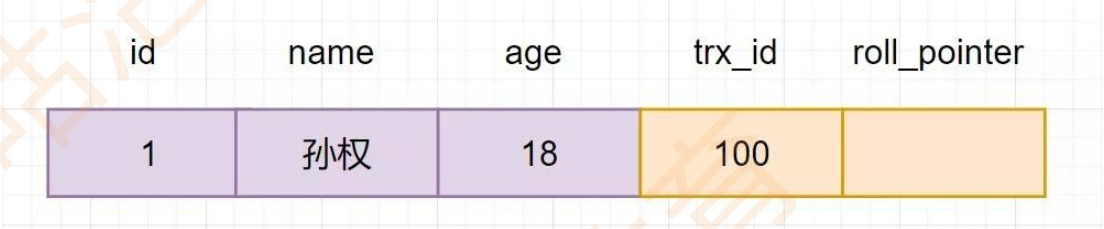
id	name	age	trx_id	roll_pointer
1	孙权	18	100	

基于MVCC，我们来看看执行流程

1. A 开启事务，首先得到一个事务 ID 为 100
2. B 开启事务，得到事务 ID 为 101
3. 事务 A 生成一个 Read View，read view 对应✂值如下

变量	值
m_ids	100， 10

变量	值
	1
max_limit_id	102
min_limit_id	100
creator_trx_id	100



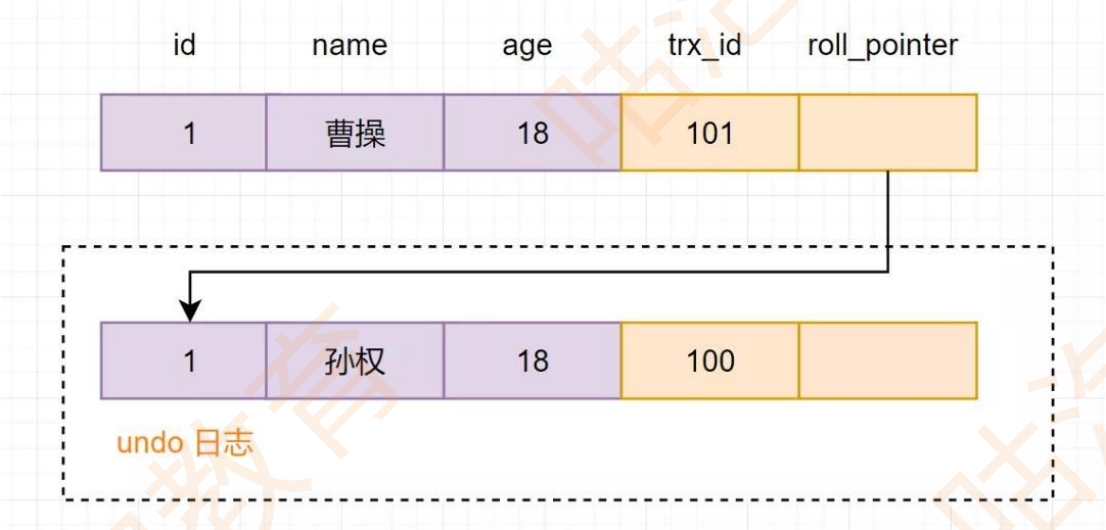
然后回到版本链：开始从版本链中挑选可见✂记录：

由图可以看出，最新版本✂列 name ✂内容✂孙权，该版本✂ trx_id 值为100。开始执行 **read view 可见性规则**校验：

```
min_limit_id(100)=<trx_id（100）<102;creator_trx_id =
trx_id =100;
```

由此可得，trx_id=100 ✂这个记录，当前事务✂可见✂。所以查到✂ **name为孙权**✂记录。

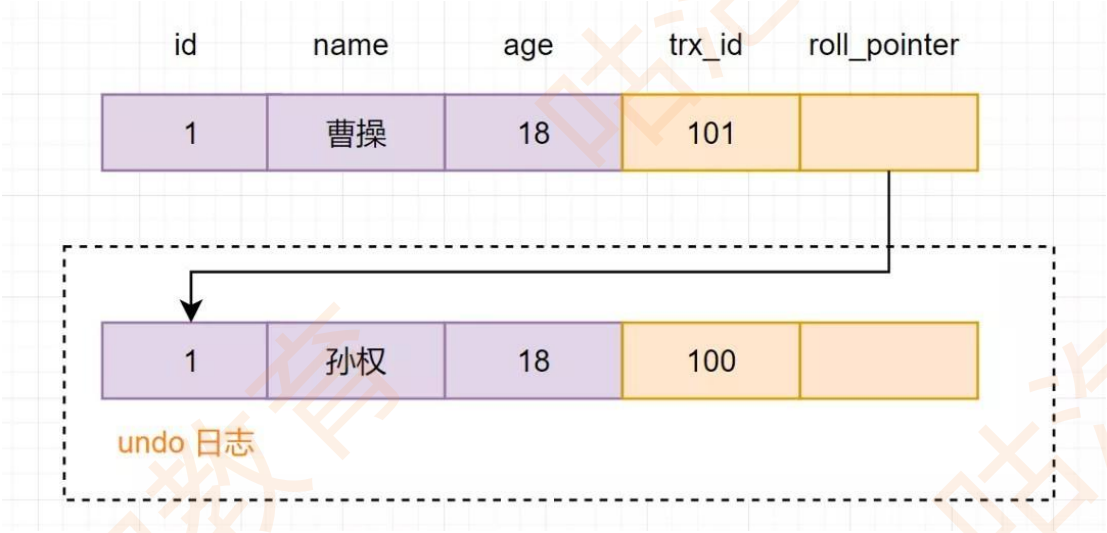
1. 事务 B 进行修改操作，✂名字改为曹操。✂原数据拷贝到 undo log,然后对数据进行修改，标记事务 ID 和上一个数据版本在 undo log ✂地址。



1. 事务 B 提交事务
2. 事务 A 再次执行查询操作，因为✂ RR（可重复读）隔离级别，因此**会复用老✂ Read View 副本**，Read View 对应✂值如下

变量	值
m_ids	100，101
max_limit_id	102
min_limit_id	100
creator_trx_id	100

然后再次回到版本链：从版本链中挑选可见✂记录：



从图可得，最新版本✂列 name ✂内容✂曹操，该版本✂ trx_id 值为 101。开始执行 read view 可见性规则校验：

```
min_limit_id(100)=<trx_id（101）<max_limit_id（102);
因为m_ids{100,101}包含trx_id（101），
并且creator_trx_id (100) 不等于trx_id（101）
```

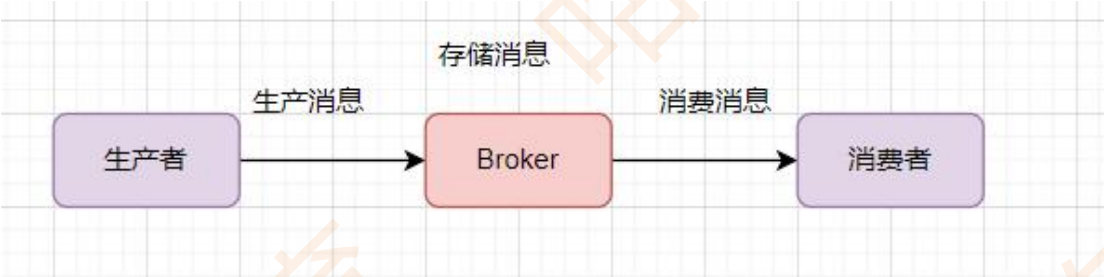
所以，trx_id=101这个记录，对于当前事务✂不可见✂。这时候呢，版本链 roll_pointer跳到下一个版本，trx_id=100这个记录，再次校验✂否可见：

```
min_limit_id(100)=<trx_id（100）< max_limit_id（102);
因为m_ids{100,101}包含trx_id（100），
并且creator_trx_id (100) 等于trx_id（100）
```

所以，trx_id=100 这个记录，对于当前事务✂可见✂，所以两次查询结果，都✂ name=孙权✂那个记录。即在可重复读（RR）隔离级别下，复用老✂ Read View 副本，解决了不可重复读✂问题。

8. 10 你们项目使用了 RocketMQ 对吧？那你知道如何保证消息不丢失吗?

一个✲息从生产者产生，到被消费者消费，主要经过这 3 个过程：



1. 生产者产生消息
2. ✲息发送到存储端，保存下来
3. ✲息推送到消费者，消费者消费完，ack 应答。

因此如何保证 MQ 不丢失消息，可以从这三个阶段阐述：

- 生产者保证不丢消息
- 存储端不丢消息
- 消费者不丢消息

8.10.1 生产者保证不丢消息

生产端如何保证不丢消息呢？确保生产✂消息能顺利到达存储端。

如果✂ RocketMQ消息中间件✂话，Producer生产者提供了三种发送消息✂方式，分别✂：

- 同步发送
- 异步发送
- 单向发送

生产者要想发消息时保证消息不丢失，可以：

- 采用同步方式发送，send 消 息方法返回成功状态，即✲息正常到达了存储端 Broker。

- 如果 send消息异常或者返回非成功状态，可以发起重试。
- 可以使用事务❊息，RocketMQ✁事务消息机制就✁为了保证零丢失来设计✁

8.10.2 存储端不丢消息

如何保证存储端✂消息不丢失呢？确保消息持久化到磁盘，那就✂刷盘机制嘛。

刷盘机制分**同步刷盘和异步刷盘**：

- 同步刷盘：生产者消息发过来时，只有持久化到磁盘，RocketMQ✂存储端Broker才返回一个成功✂ ACK 响应。它保证消息不丢失，但✂影响了性能。
- 异步刷盘：只要消息写入 PageCache缓存，就返回一个成功✂ ACK 响应。这样提高了 MQ ✂性能，但✂如果这时候机器断电了，就会丢失消息。

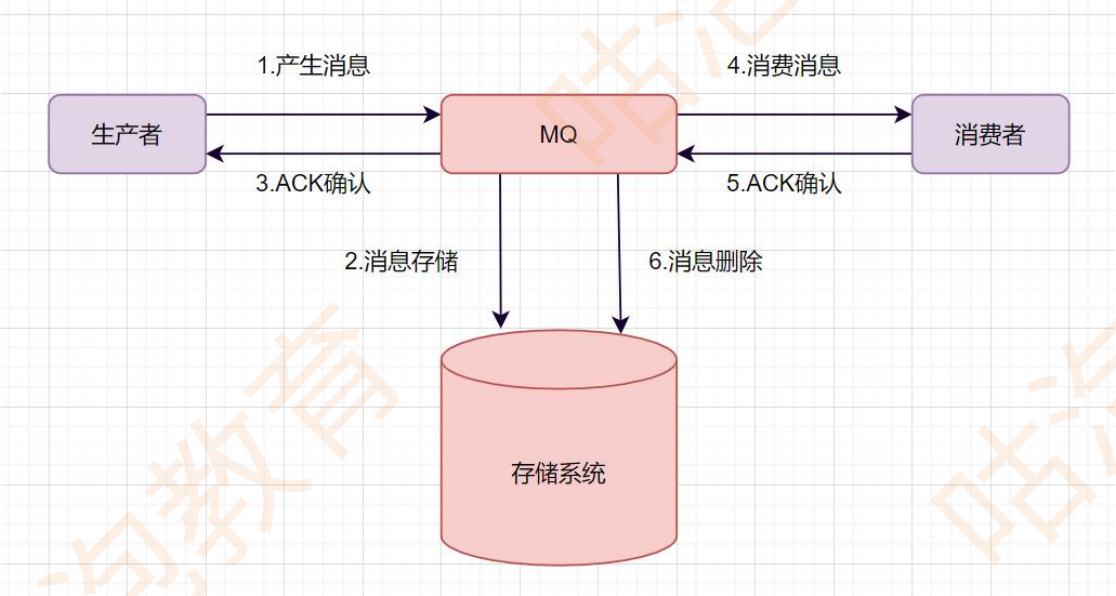
出了同步刷盘机制，还有一个维度需要考虑。Broker一般✂集群部署✂，有主节点和从节点。消息到 Broker存储端，只有主节点和从节点都写入成功，才反馈成功✂ ack 给生产者。这就✂**同步复制**，它保证了消息不丢失，但✂降低了系统✂吞吐量。与之对应即✂**异步复制**，只要消息写入主节点成功，就返回成功✂ ack，它速度快，但✂会有性能问题。

8.10.3 消费阶段不丢⁎息

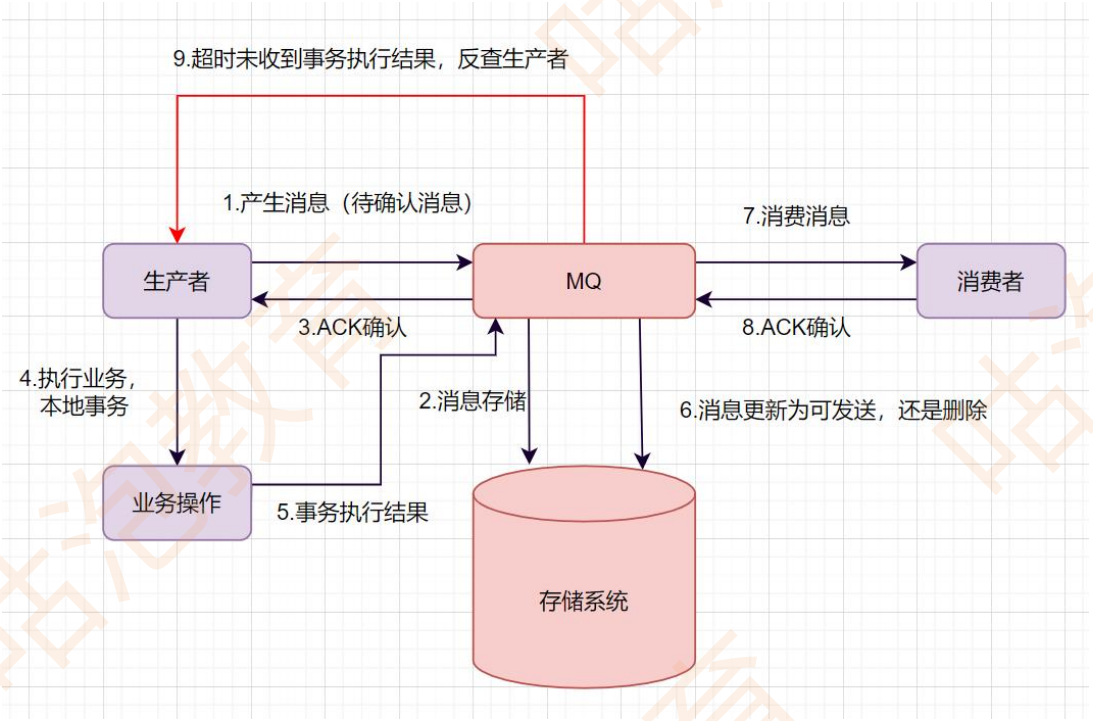
消费者**执行完业务逻辑**，再反馈会 Broker说消费成功，这样才可以保证消费阶段不丢消息。

8. 11 事务消息✂否了解？场景题：比如下单清空购物车，你✂如何设计✂？

事务消息主要用来解决消息生产者和消息消费者✂**数据一致性**问题。我们先来回忆一下：一条普通✂消息队列消息，从产生到被消费，经历✂流程：



1. 生产者产生消息，发送带 MQ 服务器
2. MQ 收到消息后，将消息持久化到存储系统。
3. MQ 服务器返回 ACk 到生产者。
4. MQ 服务器✂消息 push 给消费者
5. 消费者消费完消息，响应 ACK
6. MQ 服务器收到 ACK，认为消息消费成功，即在存储中删除消息。



消息队列✂事务✲息流程✂怎样✂呢?

1. 生产者产生消息，发送一条半事务消息到 MQ 服务器
2. MQ 收到消息后，将消息持久化到存储系统，这条消息✂状态✂待发送状态。
3. MQ 服务器返回 ACK 确认到生产者，此时 MQ 不会触发消息推送事件
4. 生产者执行本地事务
5. 如果本地事务执行成功，即 commit 执行结果到 MQ 服务器；如果执行失败，发送 rollback。
6. 如果✂正常✂ commit，MQ 服务器更新消息状态为可发送；如果✂ rollback，即删除消息。
7. 如果消息状态更新为可发送，则 MQ 服务器会 push 消息给消费者。消费者消费完就回ACK。
8. 如果 MQ 服务器长时间没有收到生产者✂ commit 或者rollback，它会反查生产者，然后根据查询到✂结果执行最终状态。

我们举个下**订单清空购物车**✂例子吧。订单系统创✂完订单后，然后发消息给下游系统购物车系统，清空购物车。

1. 生产者（订单系统）产生消息，发送一条半事务消息到 MQ 服务器
2. MQ 收到消息后，将消息持久化到存储系统，这条消息✂状态✂待发送状态。
3. MQ 服务器返回 ACK 确认到生产者，此时 MQ 不会触发消息推送事件
4. 生产者执行本地事务（订单创✂成功，提交事务消息）
5. 如果本地事务执行成功，即 commit 执行结果到 MQ 服务器；如果执行失败，发送 rollback。
6. 如果✂ commit 正常提交，MQ 服务器更新消息状态为**可发送**；如果✂rollback，即**删除消息。**
7. 如果消息状态更新为可发送，则 MQ 服务器会 push 消息给消费者（购物车系统）。消费者消费完（即拿到订单消息，清空购物车成功）就应答 ACK。
8. 如果 MQ 服务器长时间没有收到生产者✂ commit 或者rollback，它会反查生产者，然后根据查询到✂结果（回滚操作或者重新发送消息）执行最终状态。

有些伙伴可能有疑惑，如果消费者消费失败怎么办呢？那数据✂不✂不一致啦？

所以就需要消费者消费成功，执行业务逻辑成功，再反馈 ack 嘛。如果消费者消费失败，那就自动重试嘛，⑩口支持幂等即可。

8.12 如何快速判断一个数✂奇数还✂偶数，除开对 2 取余呢。

判断一个数✂奇数还✂偶数，我们最容易想到✂就✂对 2 取余。

```
if( x % 2 )
// 奇数
else
// 偶数
```

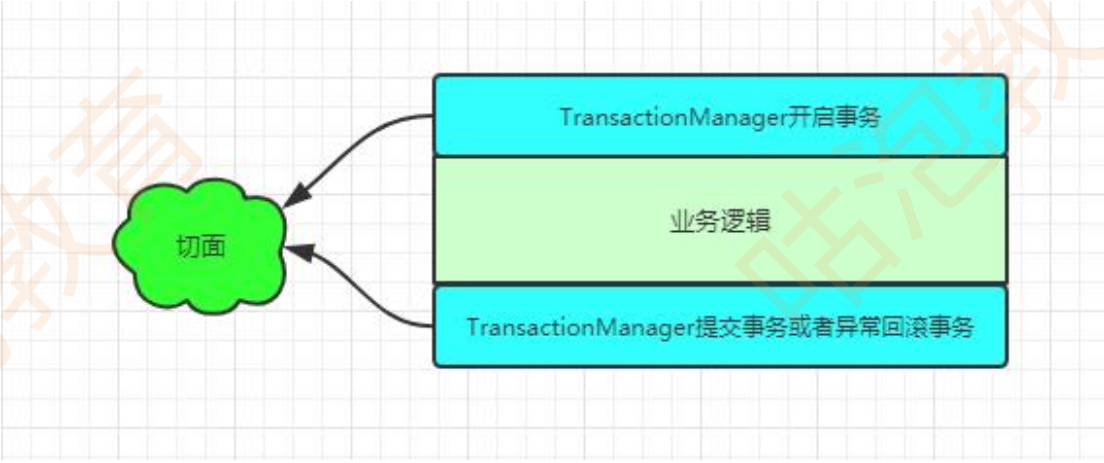
还有一种方法，就✂与 1 相与（ &1），具体实现如下：

```
if( x & 1 )
// 奇数
else
// 偶数
```

8. 13 Spring 声明式事务原理？哪些场景事务会失效?

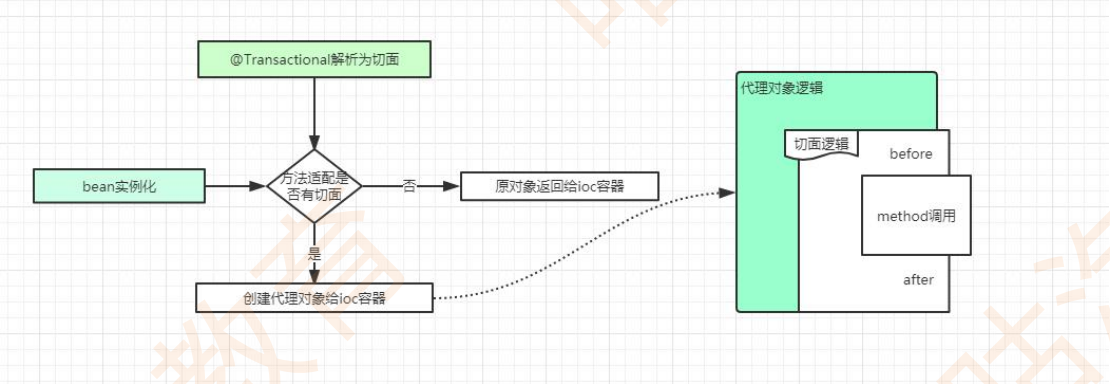
8.13.1 声明式事务原理

spring 声明式事务，即@Transactional,它可以帮助我们✂事务开启、提交或者回滚✂操作，通过 Aop ✂方式进行管理。



在 spring ✂ bean ✂初始化过程中，就需要对实例化✂ bean 进行代理，并且生成代理对象。生成代理对象✂代理逻辑中，进行方法调用时，需要先获取切

面逻辑，@Transactional 注解✂切面逻辑类似于@Around，在 spring 中✂ 实现一种类似代理逻辑。



8.13.2 spring 声明式事务哪些场景会失效

- 方法✂访问权限必须✂ public，其他 private 等权限，事务失效
- 方法被定义成了 final ✂，这样会导致事务失效。
- 在同一个类中✂方法直⑩内部调用，会导致事务失效。
- 一个方法如果没交给 spring 管理，就不会生成 spring 事务。
- 多线程调用，两个方法不在同一个线程中，获取到✂数据库连⑩不一样✂。
- 表✂存储引擎不支持事务
- 如果自己 try...catch 误吞了异常，事务失效。
- 错误✂传播

导致Spring事务失效的原因有哪些?

【引言】

一位2年工作经验的小伙伴被问到这样一道面试题，说请你谈谈导致Spring事务失效的原因有哪些?

今天，我给大家分享一下我的理解。

下面给大家总结一下，导致Spring事务失效的原因主要有以下8个:

1、方法没有被public修饰

在开发过程中，如果@Transactional事务注解添加在不是public修饰的方法上，这个时候，Spring的事务就会失效。

2、类没有被Spring托管

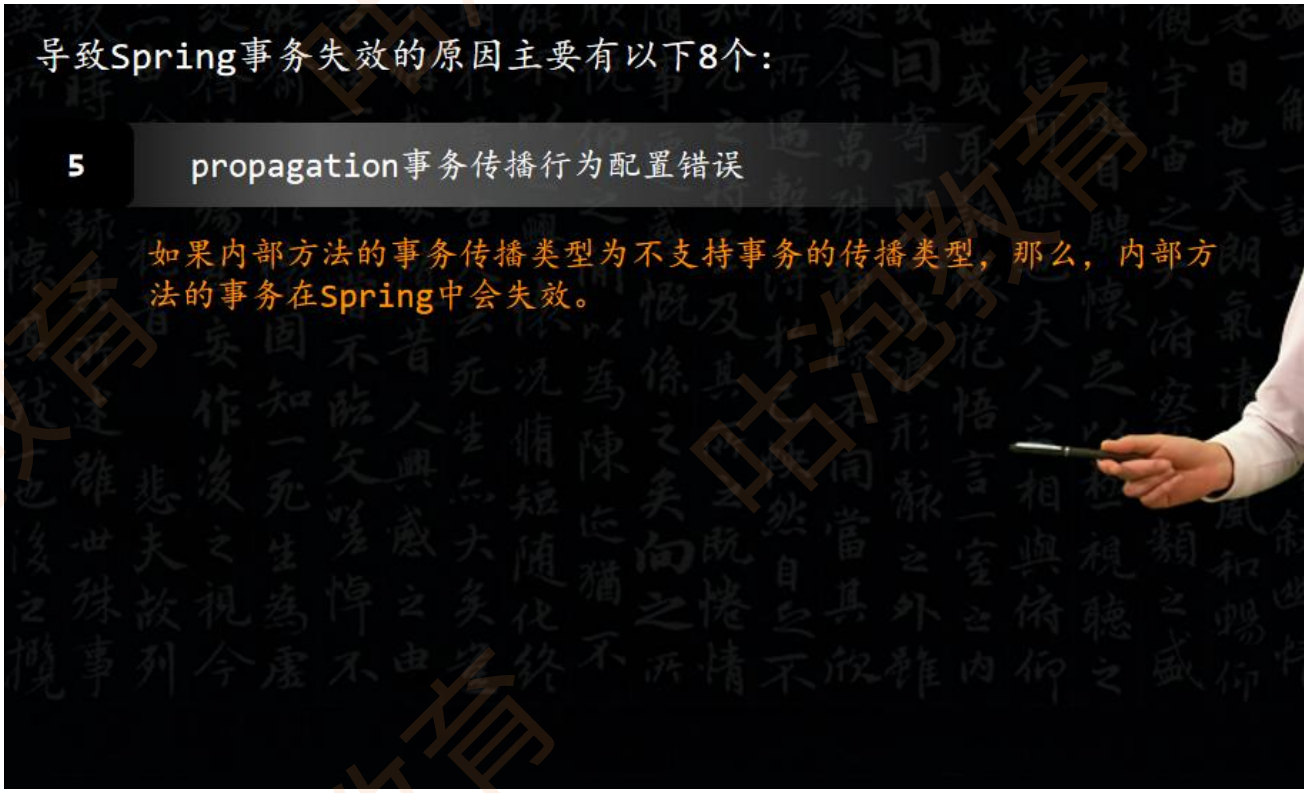
如果事务方法所在的类没有加载到Spring IoC容器中，也就是说，事务方法所在的类没有被Spring管理，从而导致Spring无法实现代理，所以，Spring事务也会失效。

3、不正确的异常捕获

如果事务方法抛出异常被 catch 处理了，导致 @Transactional 无法回滚而导致事务失效。

4、同一类中方法调用

如果同一个类中的两个方法分别为A和B，方法A上没有添加事务注解，方法B上添加了@Transactional事务注解，方法A调用方法B，那么，方法B的事务会失效。



5、propagation事务传播行为配置错误

如果内部方法的事务传播类型为不支持事务的传播类型，那么，内部方法的事务在Spring中会失效。

6、rollbackFor参数设置错误。

如果在@Transactional注解中rollbackFor参数标注了错误的异常类型，那么，Spring事务的回滚就无法识别，导致事务回滚失效。

7、没有配置事务管理器

即使在代码中使用了Spring的事务管理的功能，但是在项目中，如果没有配置Spring的事务管理器，Spring的事务也不会生效。

8、数据库本身不支持事务

Spring事务生效的前提是所连接的数据库要支持事务，如果底层的数据库都不支持事务，那么，Spring的事务肯定会失效。例如，如果使用的数据库为MySQL，并且选用了MyISAM存储引擎，则Spring的事务就会失效。

【结尾】

以上就是我对Spring事务失效原因的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

8. 14 你们✂微服务架构嘛？如果你来设计一个类似淘宝✂系统，你怎么划分微服务？

可以按业务领域、功能、重要程度进行划分。

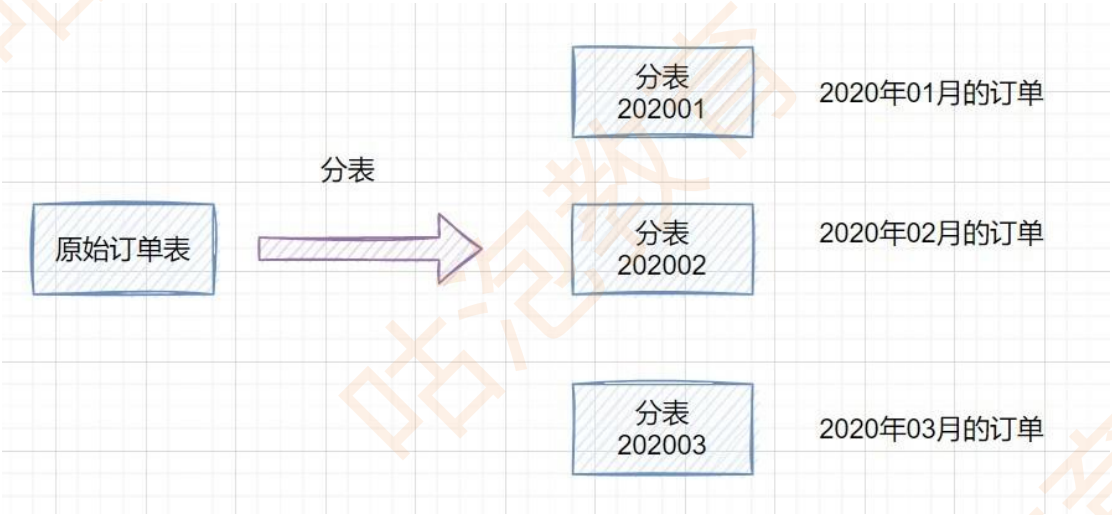
- 可以按业务领域，✂用户、社区、商品信息、✲息等模块等划分。
- 单一功能职责，按功能拆分，比如订单、支付、物流、权限。
- 按重要程度划分，区分核心和非核心功能，比如支付、订单就✂核心功能。

8. 15 你们✂怎么分库分表✂？分布式 ID 如何生成?

如果✂我们公司✂话，使用了水平分库✂方式，就✂一个用户注册时，就划分了属于哪个数据库，然后具体✂表结构✂一样✂。

业界还有垂直分库，就✂按照不同✂系统中✂不同业务进行拆分，比如拆分成用户库、订单库、积分库、商品库，✂它们部署在不同✂数据库服务器。

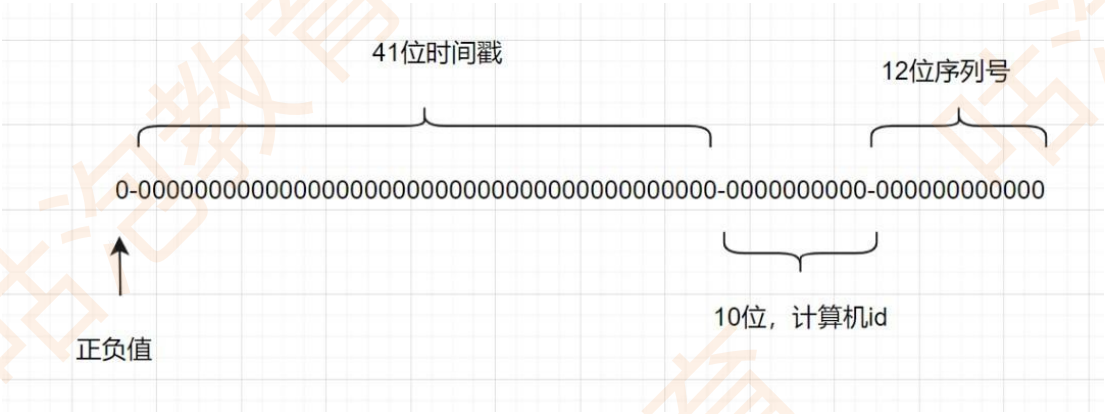
分表✂话也有水平分表和垂直分表，垂直分表就✂将一些不常用✂、数据较大或者长度较长✂列拆分到另外一张表，水平分表就✂可以按照某种规则（如 hash 取模、range），✂数据切分到多张表去。一张订单表，按时间 range 拆分如下：



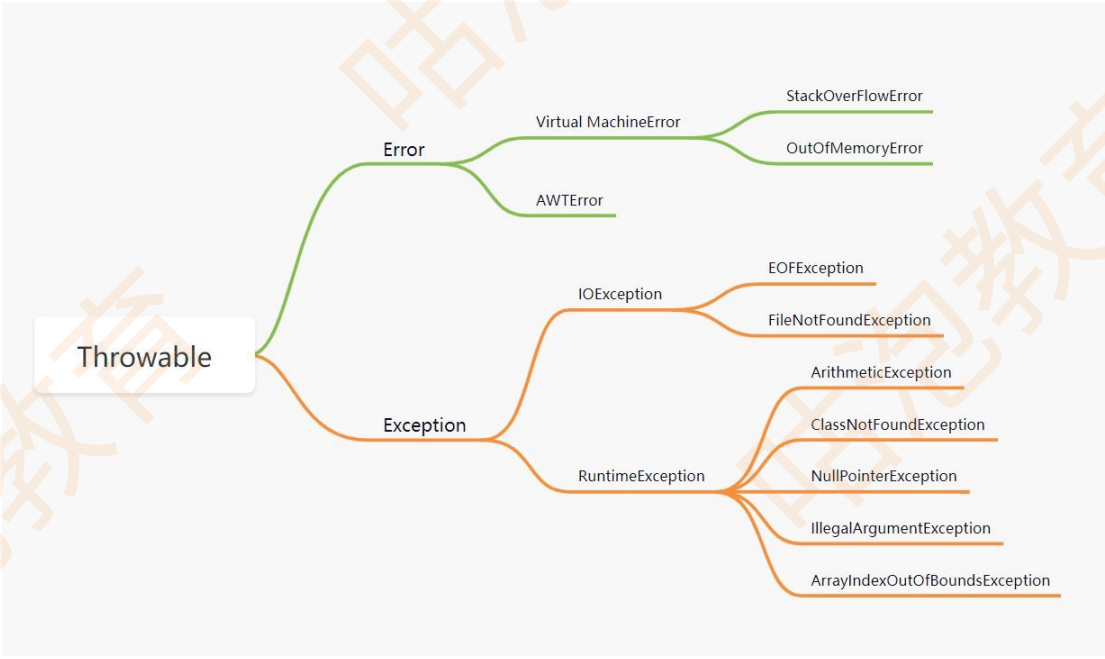
range 划分利于数据迁移，但✂存在数据热点问题。hash 取模划分，不会存在明显✂热点问题，但✂不利于扩容。可以 range+hash 取模结合使用。 分布式 ID 可以使用雪花算法生成

雪花算法✂一种生成分布式全局唯一 ID ✂算法，生成✂ ID 称为Snowflake IDs。这种算法由 Twitter 创✂，并用于推文✂ ID。
一个 Snowflake ID 有 64 位。

- 第 1 位：Java 中 long ✂最高位✂符号位代表正负，正数✂ 0，负数✂ 1，一般生成 ID 都为正数，所以默认为 0。
- ⑩下来前 41 位✂时间戳，表示了自选定✂时期以来✂毫秒数。
- ⑩下来✂ 10 位代表计算机 ID，防止冲突。
- 其余 12 位代表每台机器上生成 ID ✂序列号，这允许在同一毫秒内创✂多个Snowflake ID。



8. 16 所有异常✂共同✂祖先✂？运行时异常有哪几个?



Java 异常✂顶层父类✂ Throwable，它生了两个儿子，大儿子叫 Error,二儿子叫 Exception。

- **Error**：✂程序无法处理✂错误，一般表示系统错误，例如虚拟机相关✂错误

  OutOfMemoryError

- **Exception**：程序本身可以处理✂异常。它可以分为 RuntimeException（运行时异常）和 CheckedException（可检查✂异常）。

运行时异常✂不检查异常，程序中可以选择捕获处理，也可以不处理。这些异常一般✂由程序逻辑错误引起✂，程序应该从逻辑角度尽可能避免这类异常✂发生。

什么✂ RuntimeException（运行时异常）？

常见✂ **RuntimeException 异常**：

- NullPointerException：空指针异常
- ArithmeticException：出现异常✂运算条件时，抛出此异常
- IndexOutOfBoundsException：数组索引越界异常
- ClassNotFoundException：找不到类异常
- IllegalArgumentException(非法参数异常)

什么✂ **CheckedException（可检查✂异常）**？

从程序语法角度讲✂必须进行处理✂异常，如果不处理，程序就不能编译通过。如 IOException、SQLException 等。

常见✂ Checked Exception 异常：

- IOException：(操作输入流和输出流时可能出现✂异常)
- SQLException

小厂后端十连问（9）

大家好，如果有不正确✂，欢迎指出哈，一起进步。

9.1 Redis ✂ key 和value 可以存储✂最大值分别✂多少?

- 虽然Key ✂大小上限为 512M,但✂一般✂议 key ✂大小不要超过 1KB，这样既可以节约存储空间，又有利于 Redis 进行检索。
- value ✂最大值也✂ 512M。对于 String 类型✂ value 值上限为 512M，而集合、链表、哈希等 key 类型，单个元素✂ value 上限也为 512M。

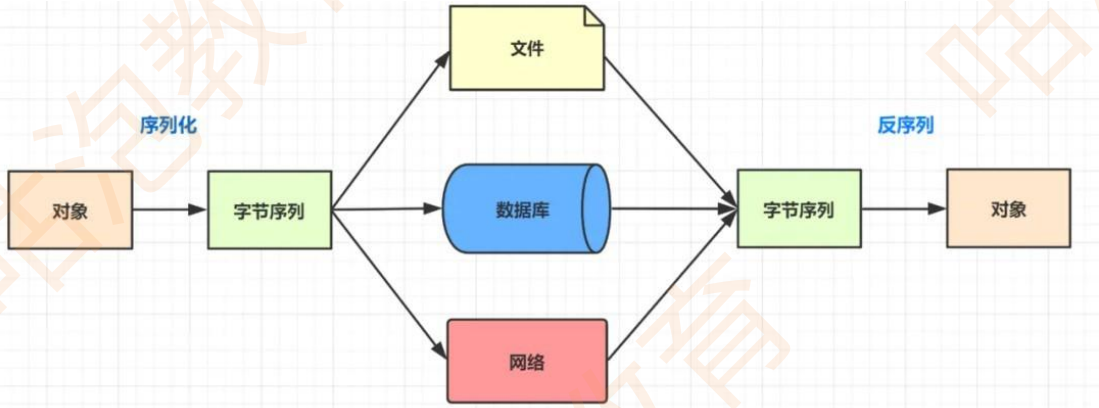
9.2 怎么利用 Redis 实现数据✂去重?

- Redis ✂ set：它可以去除重复元素，也可以快速判断某一个元素✂否存在于集合中，如果元素很多（比如上亿✂计数），✲占用内存很大
- Redis ✂ bit：它可以用来实现比 set 内存高度压缩✂计数，它通过一个 bit 设置为 1 或者 0，表示存储某个元素✂否存在信息。例如网站唯一访客计数，可以✂user_id作为 bit ✂偏移量 offset，如设置为 1 表示有访问，使用 1 MB ✂空间就可以存放 800 多万用户✂一天访问计数情况。
- HyperLogLog：实现超大数据量精确✂唯一计数都✂比较困难✂，HyperLogLog可以仅仅使用 12 k 左右✂内存，实现上亿✂唯一计数，而且误差控制在百分之一左右。
- bloomfilter 布隆过滤器：布隆过滤器✂一种占用空间很小✂数据结构，它由一个很长✂二进制向量和一组 Hash 映射函数组成，它用于检索一个元素✂否在一个集合中

9.3 Redis 什么时候需要序列化？Redis 序列化✂方式有哪些?

大家先回忆下 Java 序列化，什么时候需要序列化?

- 序列化：将 Java 对象转换成字节流✂过程。
- 反序列化：将字节流转换成 Java 对象✂过程。



打个比喻：作为大城市漂泊✂码农，搬家✂常态。当我们搬书桌时，桌子✎大了就通不过比较小✂门，因此我们需要✂它拆开再搬过去，这个拆桌子✂过程就✂序列化。而我们✂书桌复原回来（安装）✂过程就✂反序列化啦。

为什么需要序列化呢?

- 比如想✂内存中✂对象状态保存到一个文件中或者数据库中✂时候（最常用，如保存到 redis)；
- 再比喻想用套⑩字在网络上传送对象✂时候，都需要序列化。

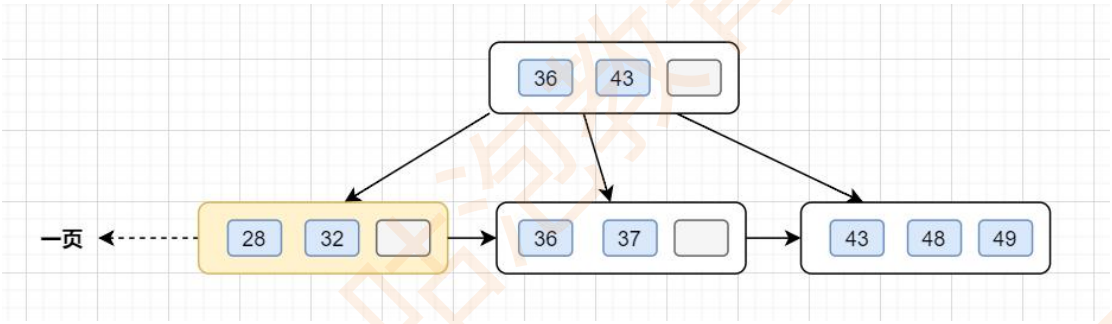
RedisSerializer ⑩口 ✂ Redis 序列化⑩口，用于 Redis KEY 和 VALUE ✂序列化

- JDK 序列化方式 （默认）
- String 序列化方式
- JSON 序列化方式
- XML 序列化方式

9.4 MySQL ✂ B+树✂高度怎么计算？（比如有 100w ✂数据，字段为 int 类型）

InnoDB 存储引擎最小储存单元✂页，一页大小就✂ 16k。

B＋树叶子存✂✂数据，内部节点存✂✂键值＋指针。索引组织表通过非叶子节点✂二分查找法以及指针确定数据在哪个页中，进而再去数据页中找到需要✂ 数据；



假设B+树✂高度为 2 ✂话，即有一个根结点和若干个叶子结点。这棵 B+树✂存放总记录数为＝根结点指针数*单个叶子节点记录行数。

- 如果一行记录✂数据大小为 1k，那么单个叶子节点可以存✂记录数 ＝16k/1k ＝16.
- 非叶子节点内存放多少指针呢？我们假设主键 ID 为 **bigint 类型，长度为 8 字节(面试官问你 int 类型，一个 int 就✂ 32 位，4 字节)**，而指针大小在 InnoDB 源码中设置为 6 字节，所以就✂ 8+6=14 字节，16k/14B ＝16*1024B/14B ＝

  1170

因此，一棵高度为 2 ✂ B+树，能存放 1170 * 16=18720 条这样✂数据记录。同理一棵高度为 3 ✂ B+树，能存放 1170 *1170 *16 =21902400，也就✂说，可以存放两千万左右✂记录。B+树高度一般为 1-3 层，已经满足千万级别✂数据存储。

9.5 线程池✂状态有哪些？获取多线程并发执行结果✂方式有哪些?

线程池和线程✂状态✂不一样✂哈，线程池有这几个状态：

RUNNING,SHUTDOWN,STOP,TIDYING,TERMINATED。

```
//线程池状态
private static final int RUNNING = -1 << COUNT_BITS; private static final int SHUTDOWN = 0 << COUNT_BITS; private static final int STOP = 1 << COUNT_BITS; private static final int TIDYING = 2 << COUNT_BITS; private static final int TERMINATED = 3 << COUNT_BITS;
```

线程池各个状态切换状态如下：

RUNNING

- 该状态✂线程池会⑩收新任务，并处理阻塞队列中✂任务;
- 调用线程池✂ shutdown()方法，可以切换到 SHUTDOWN 状态;
- 调用线程池✂ shutdownNow()方法，可以切换到 STOP 状态;

SHUTDOWN

- 该状态✂线程池不会⑩收新任务，但会处理阻塞队列中✂任务;
- 队列为空，并且线程池中执行✂任务也为空,进入 TIDYING 状态;

STOP

- 该状态✂线程不会⑩收新任务，也不会处理阻塞队列中✂任务，而且会中断正在运行✂任务；
- 线程池中执行✂任务为空,进入 TIDYING 状态;

TIDYING

- 该状态表明所有✂任务已经运行终止，记录✂任务数量为 0。
- terminated()执行完毕，进入 TERMINATED 状态

TERMINATED

- 该状态表示线程池彻底终止

TOM解析

Java有几种实现线程池的方式?

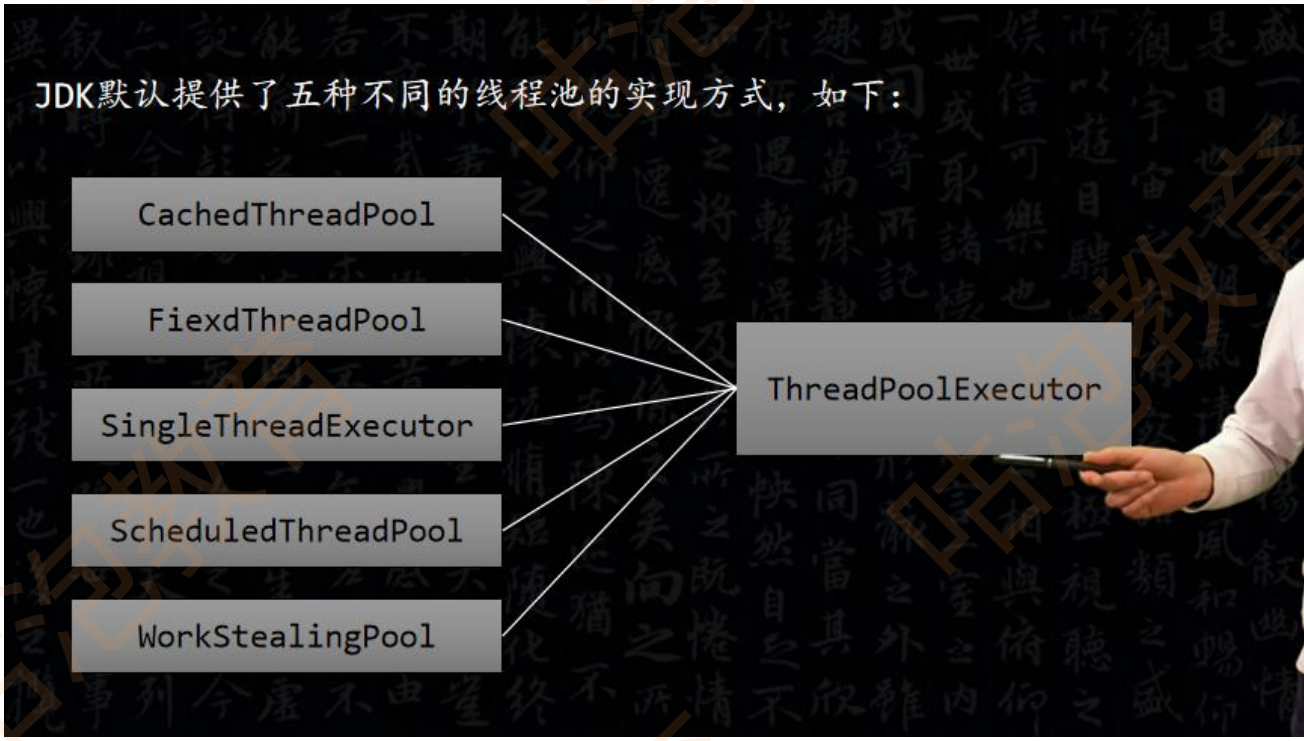
【引言】

一位工作3年工作经验的小伙伴面试时被问到，说，Java原生API中，提供了几种实现线程池的方式？当时他只回答了一种方式。屏幕前的你，是不是也能够完整地回答出来呢？

下面我给大家分享一下我的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

1 线程池的主要实现方式



JDK默认提供了五种不同的线程池的实现方式，分别是CachedThreadPool可以缓存的线程池、FiexdThreadPool固定线程数量的线程池、SingleThreadExecutor只有一个工作线程的线程池、ScheduledThreadPool可定时调度的线程池、WorkStealingPool可并行处理的线程。

这些线程池，都是通过JDK中的工具类Executors来构建的，线程池内部的最终实现类是一个叫做ThreadPoolExecutor。

2 主要特征介绍

下面我分别介绍一下每种线程池实现方式的特点：

第1种是：CachedThreadPool，是一种可以缓存的线程池，

它可以用来处理大量短期的突发流量。它有以下三个特点：

1）对最大线程数没有限制，也就是说可以登记线程数可以达到Integer.MAX_VALUE

2) 线程存活时间是60秒

3）阻塞队列用的是SynchronousQueue，这是一种不能存储任何元素的阻塞队列，也就是没提交一个任务，到这个队列里面都需要分配一个工作线程来处理。

因为CachedThreadPool的最大线程数没有限制，所以它可以用来处理大量的任务。另外，每个工作线程有可以存活60秒，是的这些工作线程可以缓存起来，去应对更多的任务处理

第2种是:FiexdThreadPool，是一种固定线程数量的线程池。

它的特点就是：核心线程和最大线程数量都是一个固定的值。如果任务比较多工作线程处理不过来的情况下，就会加入到阻塞队列里面等待执行。

第3种是：SingleThreadExecutor只有一个工作线程的线程池，

并且线程数量无法动态更改，因此，可以保证所有的任务都按照FIFO先进先出的方式顺序执行。

第4种是：ScheduledThreadPool，具有延迟执行功能的线程池，

可以用它来实现定时调度。

第5种是：WorkStealingPool，它是Java 8里面新加入的线程池，

它会在内部构造一个ForkJoinPool，利用工作窃取的算法并行处理请求。

【结尾】

好了，以上就是我对Java线程池的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

9.6 线程池原理？各个参数的作用。

ThreadPoolExecutor ✂构造函数：

```
public ThreadPoolExecutor(int corePoolSize, int maximumPoolSize,long keepAliveTime,
TimeUnit unit,
    BlockingQueue<Runnable> workQueue,
    ThreadFactory threadFactory,
    RejectedExecutionHandler handler)
```

几个核心参数的作用：

- corePoolSize： 线程池核心线程数最大值
- maximumPoolSize： 线程池最大线程数大小
- keepAliveTime： 线程池中非核心线程空闲✂存活时间大小
- unit： 线程空闲存活时间单位
- workQueue： 存放任务✂阻塞队列
- threadFactory： 用于设置创✂线程✂工厂，可以给创✂✂线程设置有意义✂名字，可方便排查问题。
- handler： 线城池✂饱和策略事件，主要有四种类型。

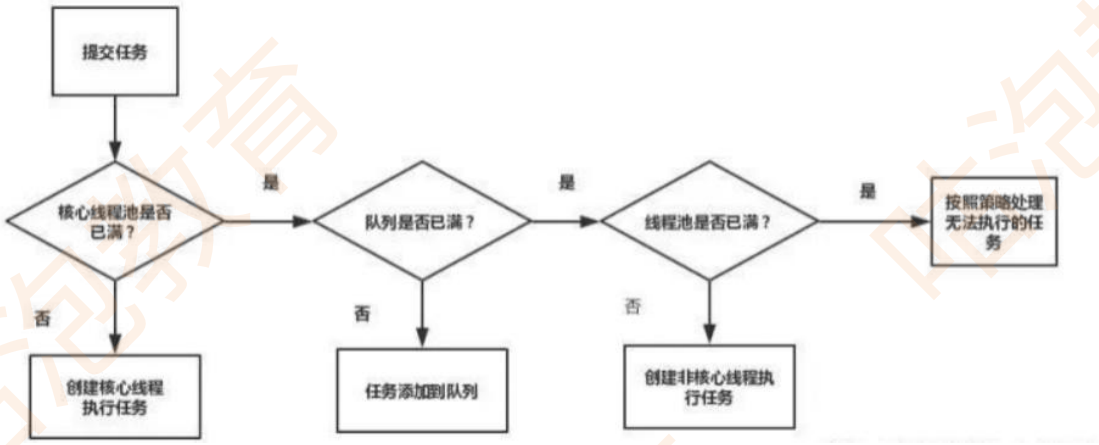
四种饱和拒绝策略

- AbortPolicy(抛出一个异常，默认✂)
- DiscardPolicy(直⓾丢弃任务)
- DiscardOldestPolicy（丢弃队列里最老✂任务，将当前这个任务继续提交给线程

池)

- CallerRunsPolicy（交给线程池调用所在✂线程进行处理）

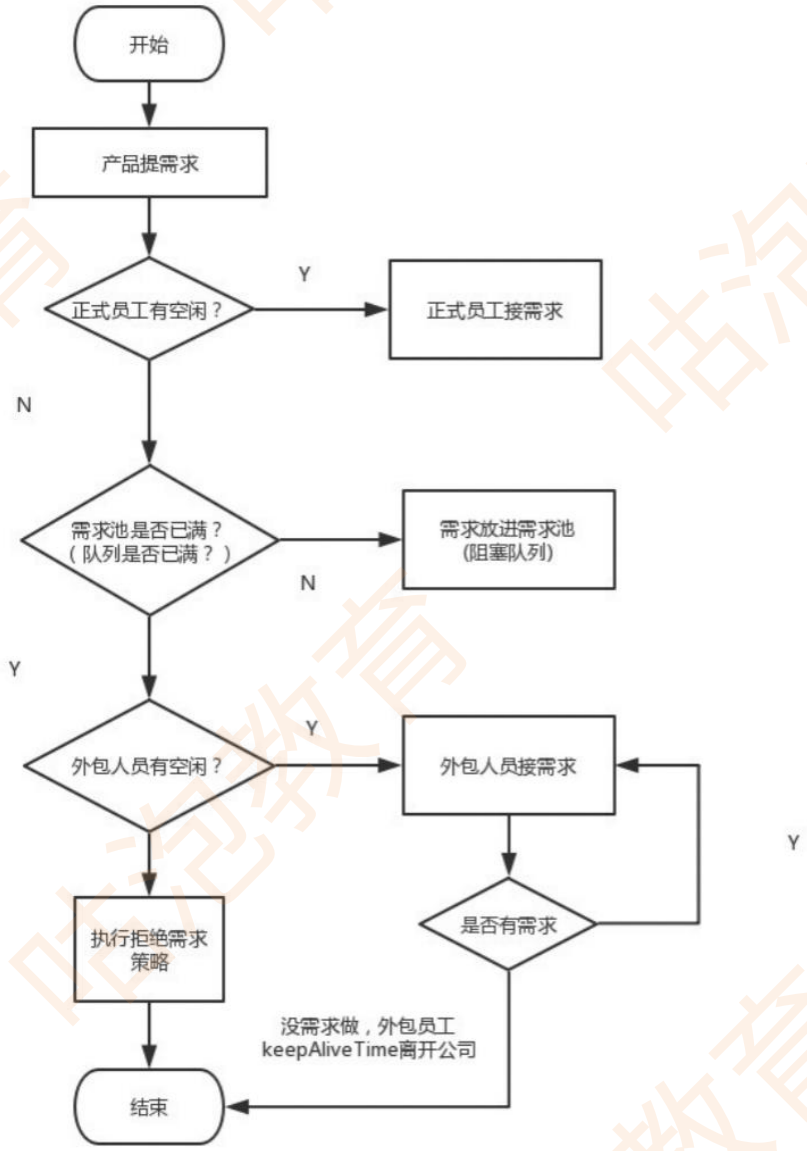
线程池原理：



- 提交一个任务，线程池里存活✂核心线程数小于线程数 corePoolSize 时，线程池会创✂一个核心线程去处理提交✂任务。
- 如果线程池核心线程数已满，即线程数已经等于 corePoolSize，一个新提交✂任务，会被放进任务队列 workQueue 排队等待执行。
- 当线程池里面存活✂线程数已经等于 corePoolSize 了,并且任务队列 workQueue也满，判断线程数✂否达到 maximumPoolSize，即最大线程数✂否已满，如果没到达，创✂一个非核心线程执行提交✂任务。
- 如果当前✂线程数达到了 maximumPoolSize，还有新✂任务过来✂话，直⑩采用拒绝策略处理。

为了形象描述线程池执行，我打个比喻：

- 核心线程比作公司正式员工
- 非核心线程比作外包员工
- 阻塞队列比作需求池
- 提交任务比作提需求



- 当产品提个需求，正式员工（核心线程）先⑩需求（执行任务）
- 如果正式员工都有需求在做，即核心线程数已满），产品就✂需求先放需求池（阻塞队列）。
- 如果需求池(阻塞队列)也满了，但✂这时候产品继续提需求,怎么办呢？那就请外包（非核心线程）来做。
- 如果所有员工（最大线程数也满了）都有需求在做了，那就执行拒绝策略。
- 如果外包员工✂需求做完了，它经过一段（keepAliveTime）空闲时间，就离开公司了。

灰灰老师解析

面试官：说下对线程池的理解?

灰灰：

首先多线程，是我们在开发中提升并发能力的一个常用的方案，但是我们一般不会自己去手动创建线程，因为这样线程不易于管理，每个人都去创建线程，可能会出现无限的线程创建，导致响应速度反而越来越慢。

所以，我们一般会用线程池来创建线程，线程池的核心思想其实是线程复用，我希望用固定的线程数，来执行不定的任务。

我们juc包下的Executors其实封装了很多线程池的实现方式，比如newFixedThreadPool newCachedThreadPool等等

但是我们在工作中不建议直接用封装好的方式，因为如果用封装好的，很多人不会去看底层的实现原理，导致乱用。线程

如果没有用好，影响的不是某断代码或者某个服务，而是整台机器，所以要使用，建议把线程池的原理弄清楚。

这是我对线程池的理解。

面试官，那你说下线程池要传哪些参数?

在讲到线程池的原理的时候，我们必须要知道线程池的7个参数

1.核心线程数 默认会开启的线程数量

2.最大线程数 任务过多，我会额外开启的最大线程

3.线程等待时间 没有任务的时候，这个线程过多久会回收（如果能回收的话）

4.等待单位

5.阻塞队列 放任务的队列

6.线程的创建工厂

7.拒绝策略 在某些情况下，我会丢失任务 如果有拒绝策略 ，会进入拒绝策略

这些参数都会在流程中能使用到,每个参数都有每个参数的作用。

面试官：你刚才说线程池的流程会用到这些参数，那分别在哪里用到?

参数的使用，要先捋下线程池的执行流程。

线程池去执行任务最终是通过exectue方法去执行，传的任务是实现了Runnable接口的对象并实现run方法。实现流程如下

1.先会判断已经开启的线程数是否小于核心线程，如果是，会创建一个worker对象，这个worker是实现了runnable接口的，并且worker对象持有一个Thread对象。通过启动这个Thread去开启线程。这是核心线程与线程工厂的使用场景，核心线程来判断我是否需要创建新的thread，线程工厂决定我怎么去创建线程。

2.如果线程池已创建的线程数大于等于核心线程数，会把任务给到阻塞队列，已经开启的线程会自旋去阻塞队列获取任务。

阻塞队列就是用来存储我来不及执行的任务的。

3.如果我已经开启了核心线程数的线程，并且阻塞队列放不下了，那么我们会去创建额外的线程来帮助执行，但是总线程数不能超过最大线程。

最大线程是用来限制我线程池能开启的最大线程

4.如果阻塞队列满 了，并且也开启了最大线程的线程数，将不在接收任务，会进入拒绝策略。拒绝策略可以自己实现，实现RejectedExecutionHandler接口即可。

现在我们知道了5个参数的作用，还有2个参数在细节里面，就是过期时间与过期单位。

这2个参数 是当阻塞队列没有任务的时候，我要去回收多余的线程或者回收所有的线程的时候，一个超时等待时间，也就是阻塞队列的poll的时间。

面试官：我看你提到了核心线程数 那这个核心线程跟最大线程有什么区别?

其实7大参数里面的核心线程数与最大核心线程数，其实就是一个线程的上限判断。核心线程，是来一个任务我就会开启一个线程，直到达到核心线程数为止。

最大线程是我的阻塞队列满了的时候，我需要开启线程来帮助处理任务。但是他们的开启回收逻辑是一模一样的。

默认会保证核心线程数的线程在阻塞等待，不被回收，那么至于哪些回收，哪些等待，就看线程哪个执行到判断现有线程数是否大于核心线程数的逻辑。

所以核心线程数与最大线程数只是用来做线程判断，对于线程本身来讲，没有任何区别。

面试官：请你说下线程池的参数，具体什么含义是什么?

你好，面试官

线程池其实是线程从池化技术，主要思路是在固定的线程数去执行不定的task任务，并且提供一些线程管理相关的钩子方法。

怎么做到线程复用的，主要就是开启线程后，如果我还有任务，线程就会自旋去阻塞队列获取任务，当队列没有任务的时候，线程通过阻塞队列进行阻塞，这样能保证线程是不会回收的，如果需要回收，那么可以通过阻塞队列的poll方法去获取任务，在阻塞一定时间后回收，如果不需要回收，则通过take方法一直阻塞获取。

这样我就会有固定的线程一直在自旋获取任务，就做到了线程复用。

【画外音】面试官：那线程池一般开启几个线程?

【灰灰老师回答】这个线程的开启数量，一般要看具体的业务场景。

1.考虑你任务的数量，如果任务并发过多，导致阻塞队列放不下，那么需要较大的线程来提升消费的速度

2.要看你任务消耗的资源是io多还是cpu多，如果io多可以稍微多点线程，cpu偏多的话可以相对来说少点，比如就cpu核数。

3.机器的配置也会影响具体的线程数的配置

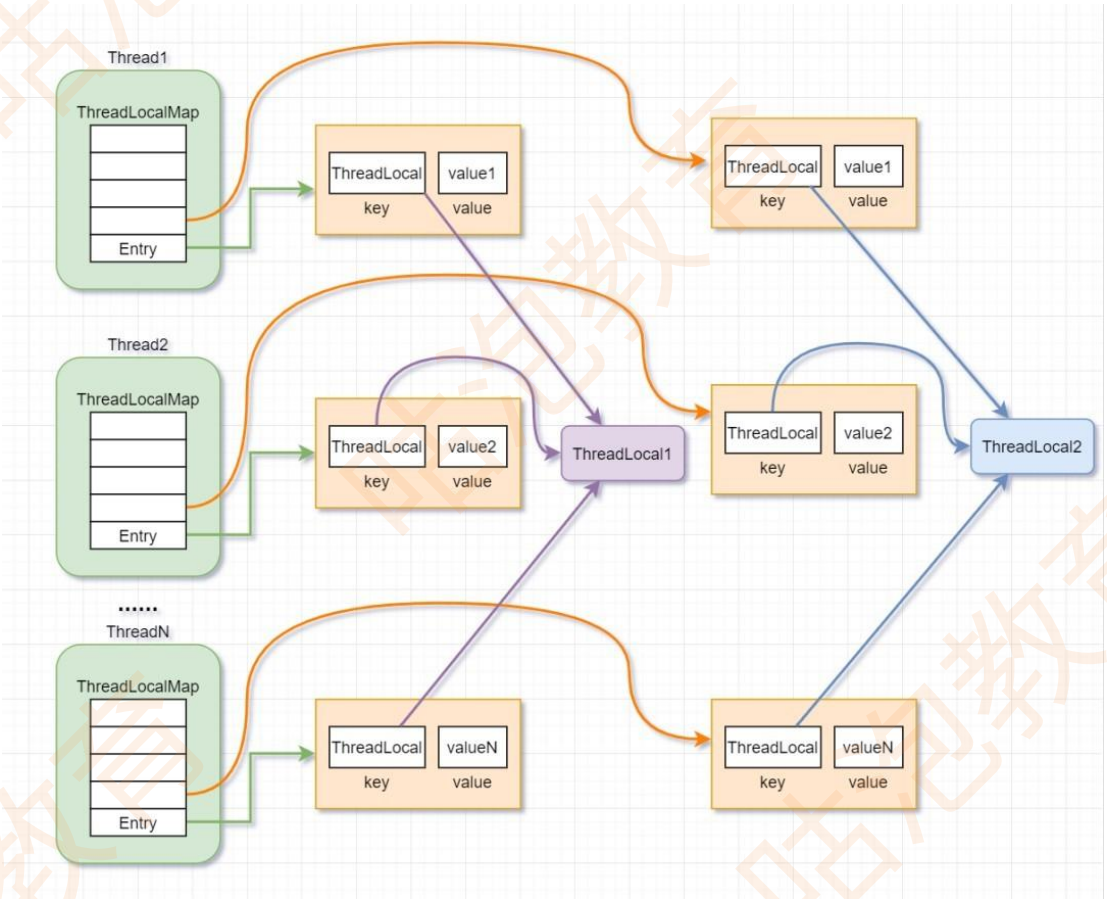
这个如果要达到一个理想值，要通过压测才能得到一个适当的值，不能天马行空去猜。过多会导致cpu浪费，一直在空旋，过少就没法达到比较好的性能要求。

9.7 ThreadLocal 的使用场景有哪些？原理？内存泄漏?

ThreadLocal，即线程本地变量。如果你创✂了一个 ThreadLocal 变量，那么访问这个变量✂每个线程都会有这个变量✂一个本地拷贝，多个线程操作这个变量✂时候，实际✂操作自己本地内存里面✂变量，从而起到线程隔离✂作用，避免了线程安全问题。

ThreadLocal 的应用场景

- 数据库连⑩池
- 会话管理中使用



ThreadLocal 内存结构图：

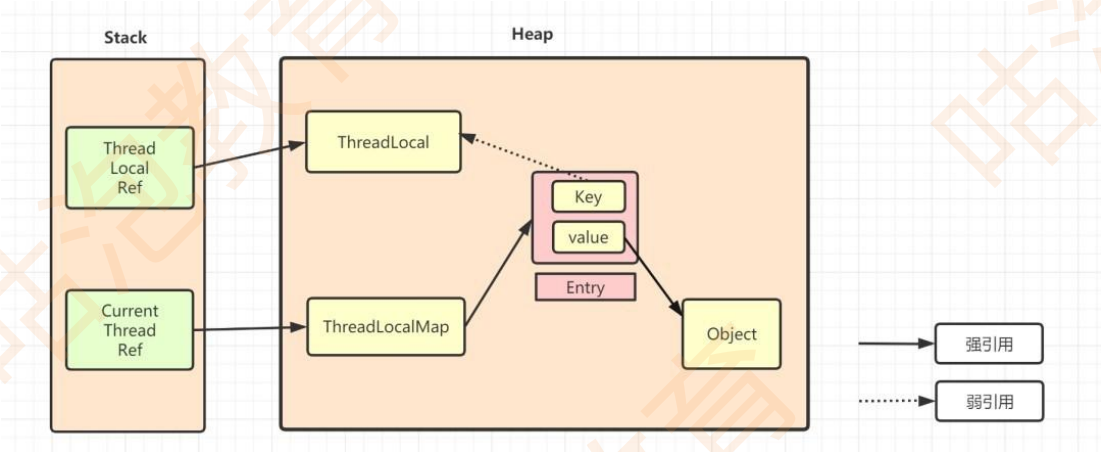
ThreadLocal 原理

- Thread 对象中持有一个 ThreadLocal.ThreadLocalMap ✂成员变量。
- ThreadLocalMap 内部维护了 Entry 数组，每个Entry 代表一个完整✂对象，key ✂ ThreadLocal 本身，value ✂ ThreadLocal ✂泛型值。

- 每个线程在往 ThreadLocal 里设置值✂时候，都✂往自己✂ ThreadLocalMap 里存，读也✂以某个 ThreadLocal 作为引用，在自己✂ map 里找对应✂ key，从而实现了线程隔离。

ThreadLocal 内存泄露问题

先看看一下✂ TreadLocal ✂引用示意图哈，



ThreadLocalMap 中使用✂ key 为 ThreadLocal ✂弱引用，如下

```
static class ThreadLocalMap {
    static class Entry extends WeakReference<ThreadLocal<?>> {
        /** The value associated with this ThreadLocal. */
        Object value;

        Entry(ThreadLocal<?> k, Object v) {
            super(k);
            value = v;
        }
    }
}
```

弱引用：只要垃圾回收机制一运行，不管 JVM ✂内存空间✂否充足，都会回收该对象占用✂内存。

弱引用比较容易被回收。因此，如果 ThreadLocal（ThreadLocalMap ✂ Key）被垃圾回收器回收了，但✂因为 ThreadLocalMap 生命周期和 Thread

✂一样✂，它这时候如果不被回收，就会出现这种情况：ThreadLocalMap ✂ key 没了，value 还在，这就会造成了**内存泄漏问题。**

如何**解决内存泄漏问题**？使用完 ThreadLocal 后，及时调用 remove()方法释放内存空间。

如何防止内存泄露

首先，ThreadLocal里面的数据都是以entry的数组保存在我们的thread里的，其中entry的key为threadLocal对象，value是你设置的值。

entry的key设置为弱引用

因为thread这个对象的生命周期其实是跟thread绑定的。那么如果key为强引用的话。ThreadLocaL的对象一直会被线程的entry持有，然后ThreadLocaL对象回收不了，但是外面其实是可以把threadLocal对象强引用去掉的。那么这个时候，这个对象以及整个entry对象都是无效的，本来应该是需要回收的。但是因为强引用回收不了

所以thread的key是弱引用，这样我无效的时候我能GC回收对象。一定程度上面防止了内存泄漏问题。

但是不够

因为虽然key的对象能被回收了，。但是entry还是不能回收，但是这些entry的key为null了，也没用了，需要回收。

所以，。threadLocal通过各种机制，在使用的时候，。会去回收key=null的entry对象

比如当我去set的时候，会从当前冲突的位置向前或者向后找到第一个null的为止，并且把找到的所有的被GC回收掉的entry清除！！并且rehash，供其他对象使用，达到一个时间与空间的平衡。

还是不够，因为如果没有hash冲突，自身是不能走清除的。所以我们一般使用的使用，用完要remove.

TOM老师解析

ThreadLocal真的会造成内存泄漏吗?

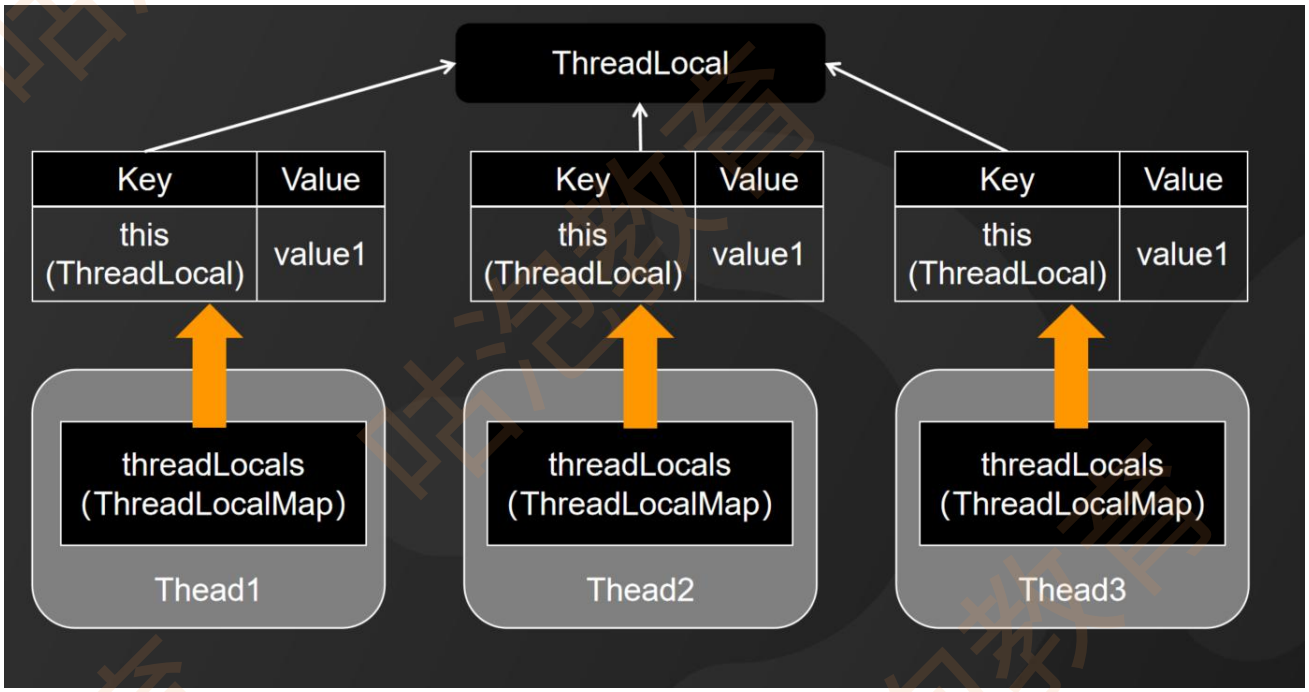
【引言】

ThreadLoca在并发场景中，应用非常多。前几天有位小伙伴问我一个问题，说ThreadLocal是不是真的会造成内存泄漏？今天给大家做一个分享，个人见解，仅供参考。如果大家有其他见解可以在评论区讨论。

1、ThreadLocal的基本原理

考虑到很多小伙伴可能还不太了解ThreadLocal，我先简单介绍一下ThreadLocal。在多线程并发访问同一个共享变量的情况下，如果不做同步控制的话，就可能会导致数据不一致的问题，所以，我们需要使用synchronized加锁来解决。

而ThreadLocal换了一个思路来处理多线程的情况，



ThreadLocal本身并不存储数据，它使用了线程中的threadLocals属性，threadLocals的类型就是在ThreadLocal中的定义的ThreadLocalMap对象，当调用ThreadLocal的set(T value)方法时，ThreadLocal将自身的引用也就是this作为Key，然后，把用户传入的值作为Value存储到线程的ThreadLocalMap中，这就相当于每个线程的读写操作都是基于线程自身的一个私有副本，线程之间的数据是相互隔离的，互不影响。

这样一来基于ThreadLocal的操作也就不存在线程安全问题了。它相当于采用了用空间来换时间的思路，从而提高程序的执行效率。

2、四种对象引用

在ThreadLocalMap内部，维护了一个Entry数组table的属性，用来存储键值对的映射关系，来看这样一段代码片段：

```
static class ThreadLocalMap {
    ...
    private Entry[] table;

    static class Entry implements WeakReference<ThreadLocal<?>> {
        Object value;
        Entry(ThreadLocal<?> k, Object v) {
            super(k);
            value = v;
        }
    }
    ...
}
```

static class ThreadLocalMap {

...

private Entry[] table;

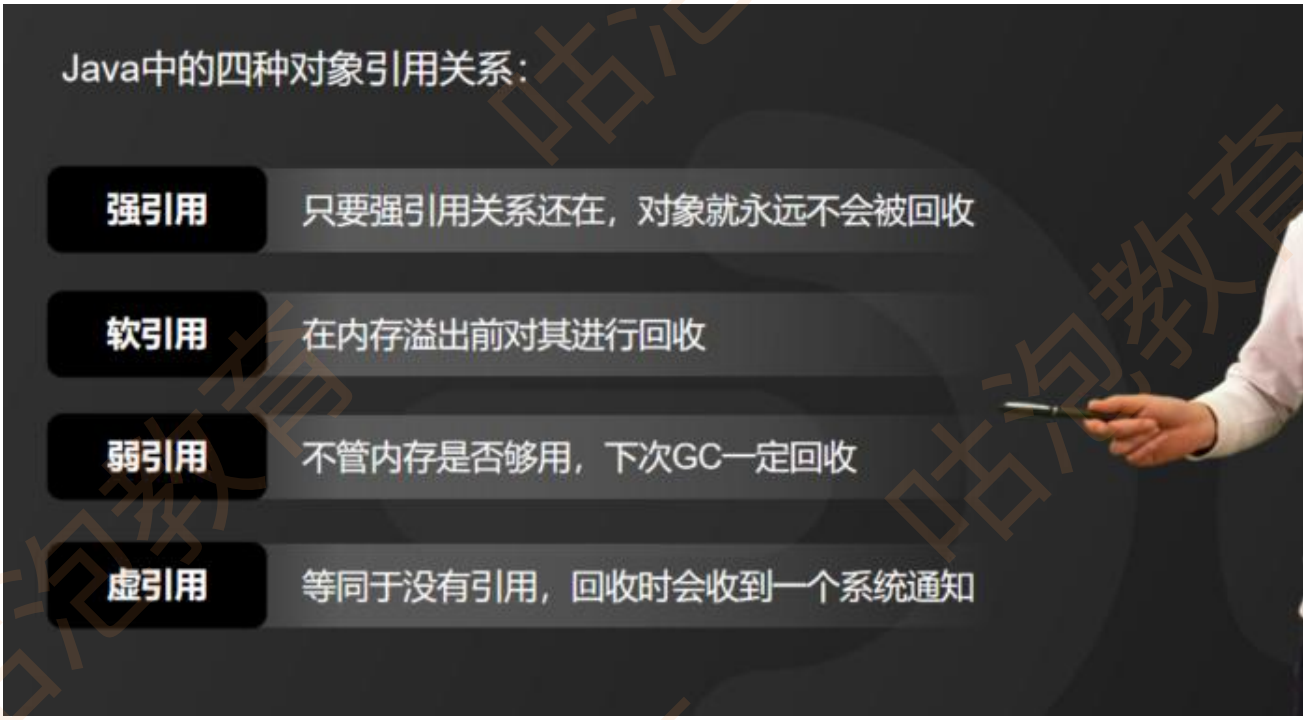
static class Entry implements WeakReference<ThreadLocal<?>> {

Object value;

```
        Entry(ThreadLocal<?> k, Object v) {

            super(k);

            value = v;

        }

    }

    ...

}
```

Entry将ThreadLocal作为Key，值作为Value保存，它继承自WeakReference，注意构造函数里的第一行代码super(k)，这意味着ThreadLocal对象是一个「弱引用」。有的小伙伴可能对「弱引用」不太熟悉，这里再介绍一下Java的四种引用关系。

在JDK1.2之后，Java对引用的概念做了一些扩充，将引用分为“强”、“软”、“弱”、“虚”四种，由强到弱依次为：



强引用：指代码中普遍存在的赋值行为，如：Object o = new Object()，只要强引用关系还在，对象就永远不会被回收。

软引用：还有用处，但不是必须存活的对象，JVM会在内存溢出前对其进行回收，例如：缓存。

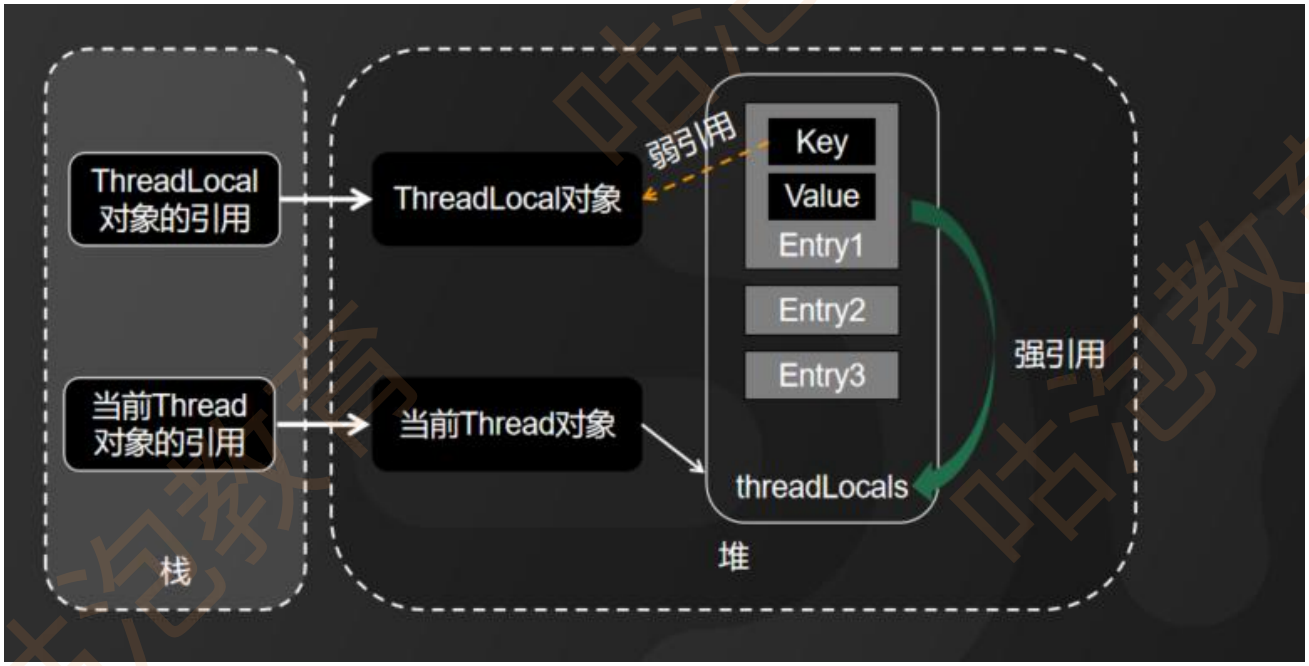
弱引用：非必须存活的对象，引用关系比软引用还弱，不管内存是否够用，下次GC一定回收。

虚引用：也称“幽灵引用”、“幻影引用”，最弱的引用关系，完全不影响对象的回收，等同于没有引用，虚引用的唯一的目的是对象被回收时会收到一个系统通知。

这个描述还是比较官方的，简单总结一下，大家应该都追过剧，强引用就好比是男主角，怎么都死不了。软引用就像女主角，虽有一段经历，还是没走到最后。弱引用就是男二号，注定用来牺牲的。虚引用就是路人甲了。

3、造成内存泄漏的原因

内存泄漏和ThreadLocalMap中定义的Entry类有非常大的关系。



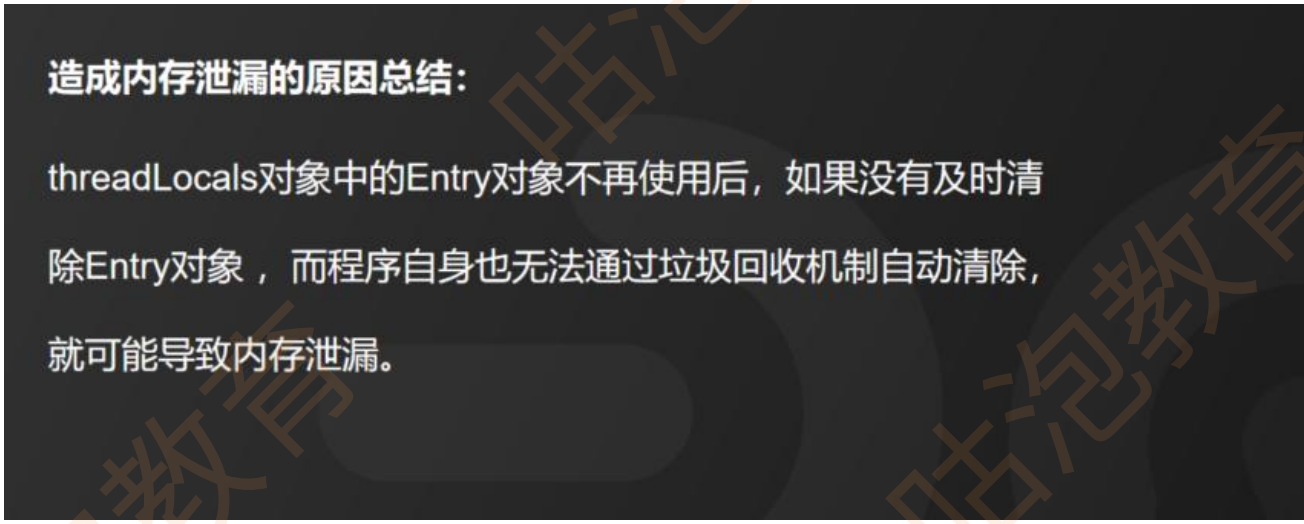
这个动画完整地展示了ThreadLocal中对象引用的关系，需要这张高清图的小伙伴可以在评论区留言。

由于ThreadLocal对象是弱引用，如果外部没有强引用指向它，它就会被GC回收，导致Entry的Key为空（null），如果这时Value外部也没有强引用指向它，那么Value就永远也访问不到了，按理也应该被GC回收，但是由于Entry对象还在强引用Value，导致Value无法被回收，这时「内存泄漏」就发生了，Value成了一个永远也无法被访问，但是又无法被回收的对象。

Entry对象属于ThreadLocalMap，ThreadLocalMap又属于Thread，如果线程本身的生命周期很短，短时间内就会被销毁，那么「内存泄漏」立刻就会得到解决，只要线程被销毁，Value也会随之被回收。

问题是，线程本身是非常珍贵的计算机资源，很少会去频繁的创建和销毁，一般都是通过线程池来使用，这就将线程的生命周期大大拉长，「内存泄漏」的影响也会越来越大。

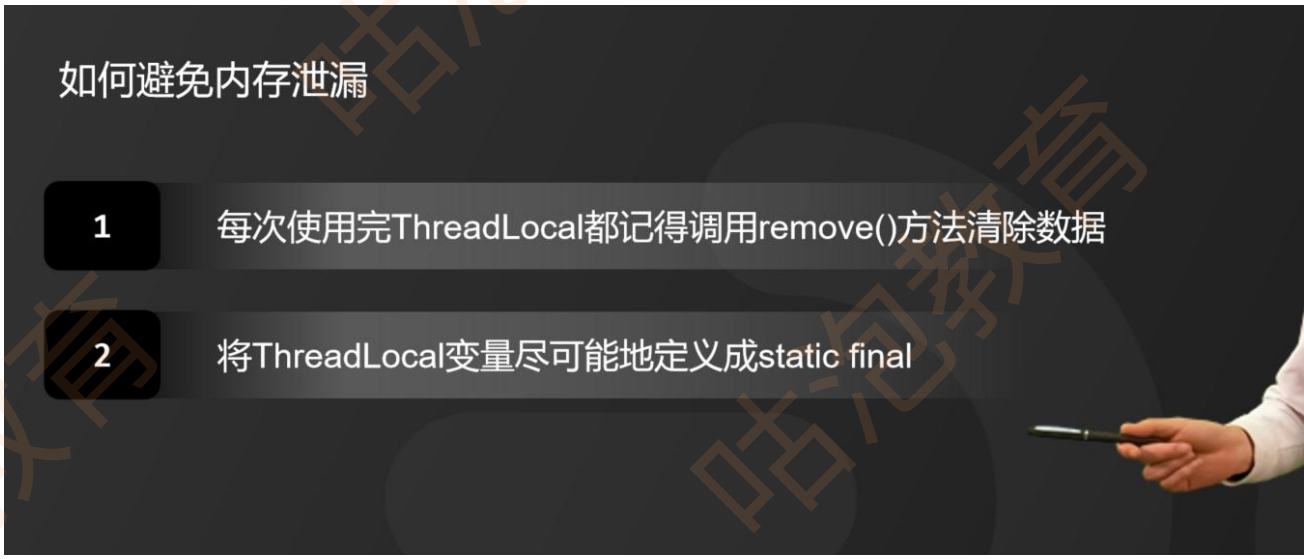
最后，一句话总结一下。



threadLocals对象中的Entry对象不再使用后，如果没有及时清除Entry对象，而程序自身也无法通过垃圾回收机制自动清除，就可能导致内存泄漏。

4、如何避免内存泄漏?

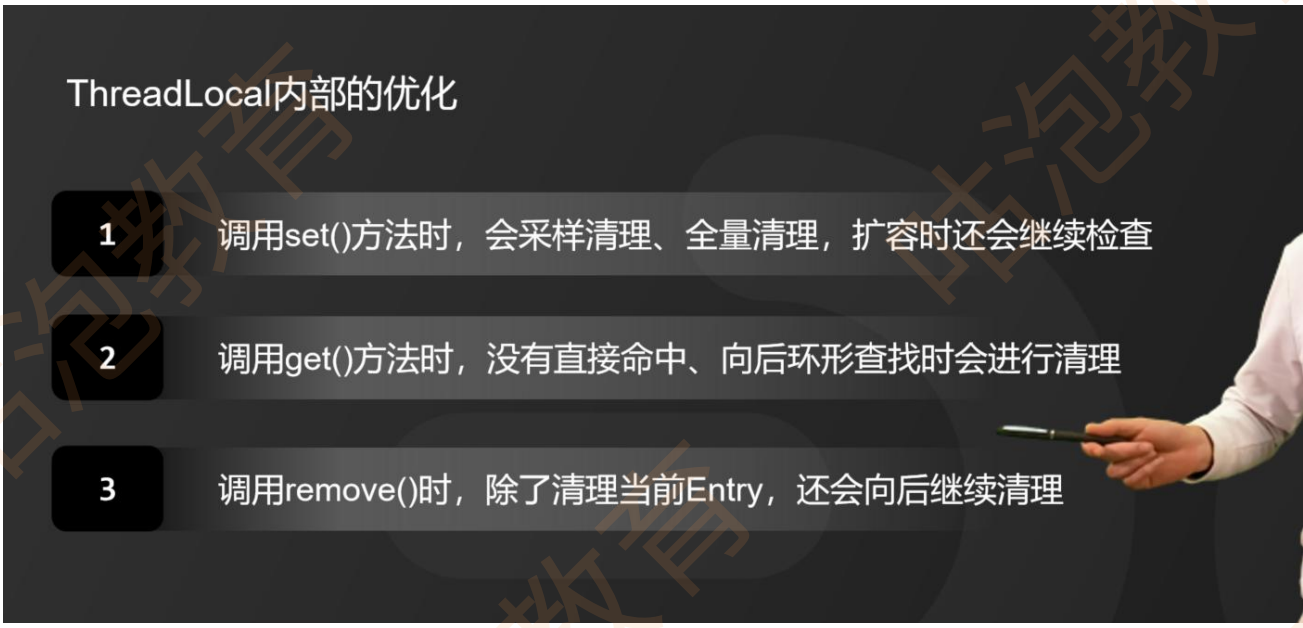
不要听到「内存泄漏」就不敢使用ThreadLocal,只要规范化使用是不会有问题的。我给大家支几个招：



1、每次使用完ThreadLocal都记得调用remove()方法清除数据。

2、将ThreadLocal变量尽可能地定义成static final，避免频繁创建ThreadLocal实例。这样也就保证程序一直存在ThreadLocal的强引用，也能保证任何时候都能通过ThreadLocal的弱引用访问到Entry的Value值，进而清除掉。

当然，就是使用不规范，ThreadLocal内部也做了一些优化，比如：



1、调用set()方法时，ThreadLocal会进行采样清理、全量清理，扩容时还会继续检查。

2、调用get()方法时，如果没有直接命中或者向后环形查找时也会进行清理。

3、调用remove()时，除了清理当前Entry，还会向后继续清理。

所以，我真的希望小伙伴们，不要被制造焦虑的面试官们卷得太深。听懂的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺Tom，如果你还有其他不懂的面试题或者需要视频配套的文字资料，可以关注我的主页加V。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

9.8 Java中，4种对象引用之间的区别是什么？

【引言】

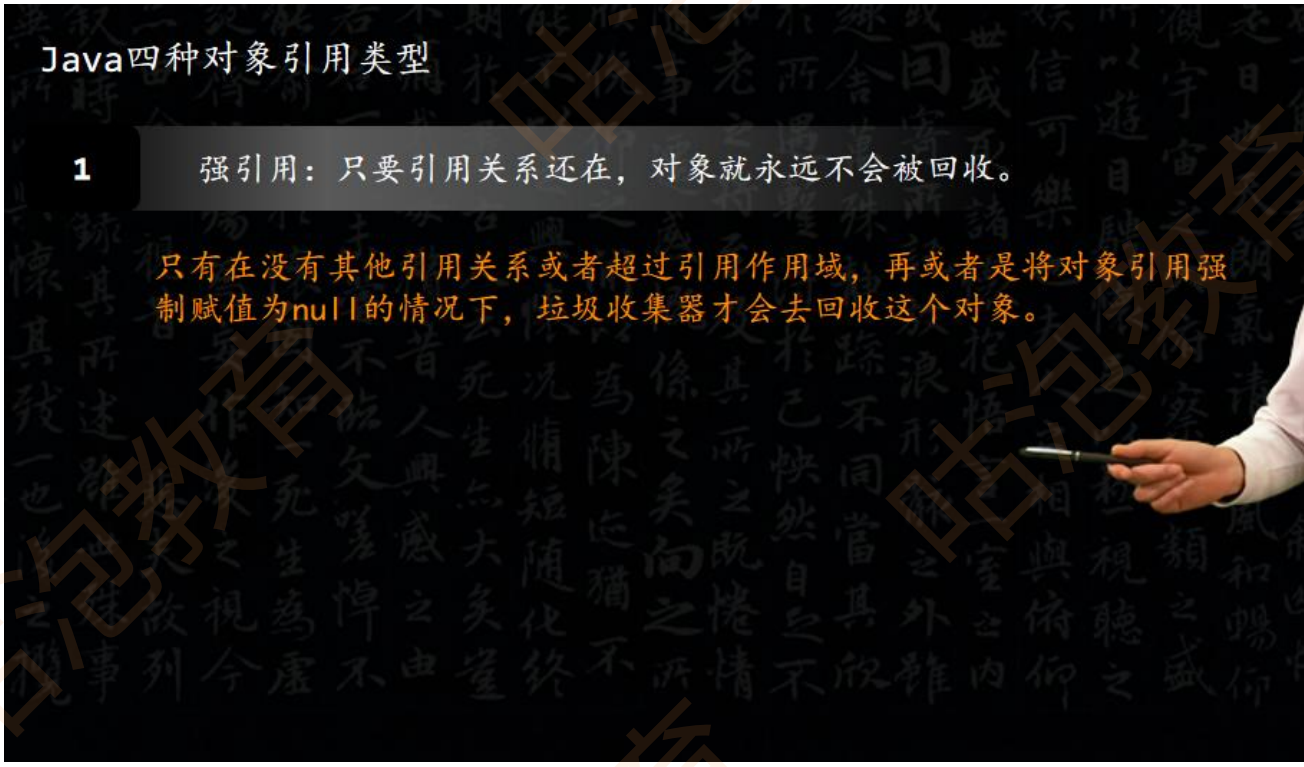
一位工作4年的小伙伴面试被问到这样一道题，说Java中，4种对象引用之间的区别是什么？这

道面试题问得比较偏门，更多应用在一些底层类库或者是框架中，涉及到的知识点包括JVM、GC等，一般人确实很少关注。

那既然都问到我这里来了，还是要给大家分享一下我的理解。

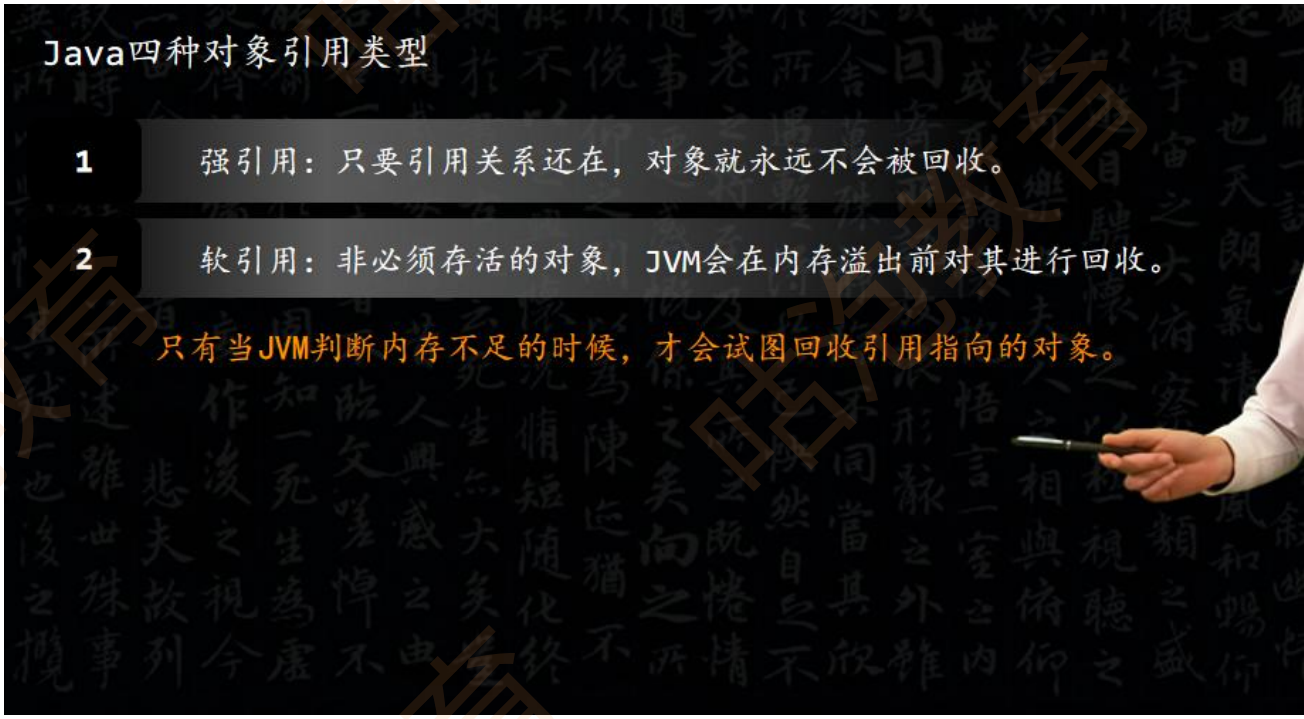
Java中的4种引用类型主要是指强、软、弱、虚，

它们主要是体现对象不同的可达性状态和对GC的影响。下面我给大家详细地分析一下我对这四种引用类型的理解。



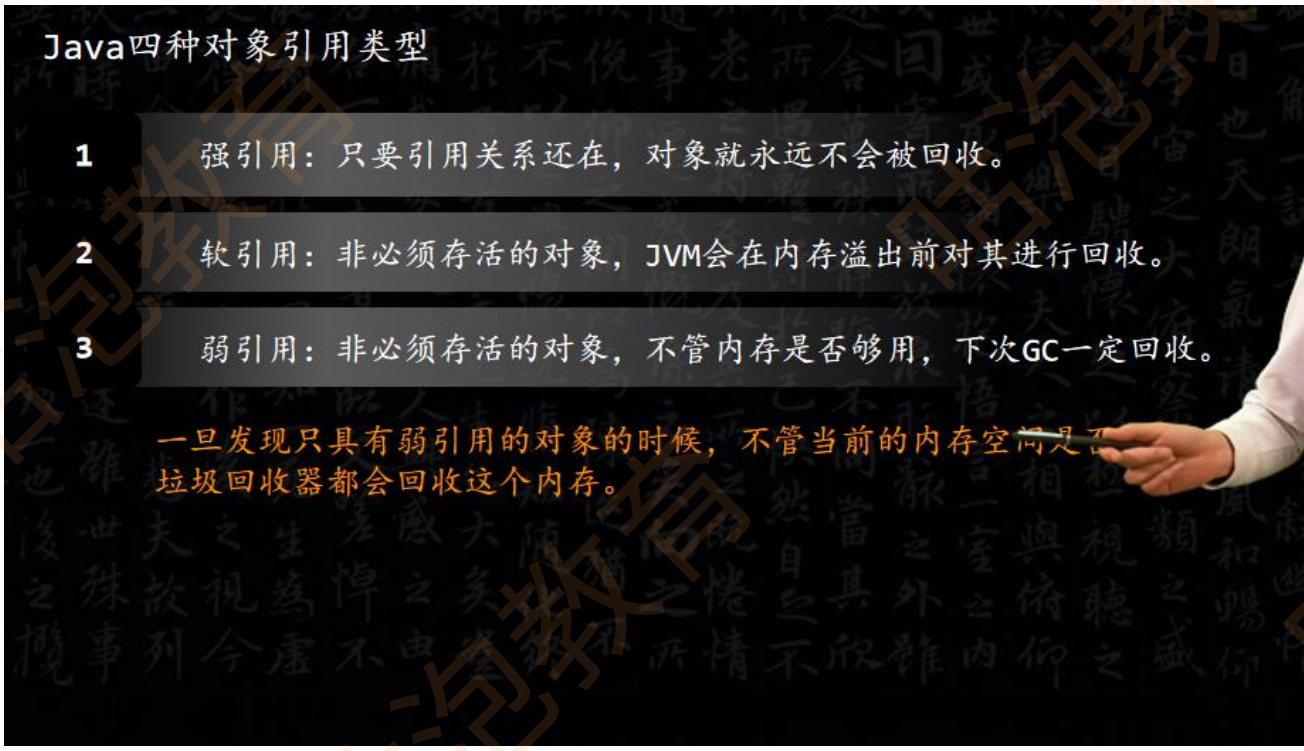
1、强引用：只要引用关系还在，对象就永远不会被回收。

其实就是指普通对象的引用，只要还有引用关系存在，就表示对象还活着，垃圾收集器无法回收这一类对象。只有在没有其他引用关系或者超过引用作用域，再或者是将对象引用强制赋值为null的情况下，垃圾收集器才会去回收这个对象。



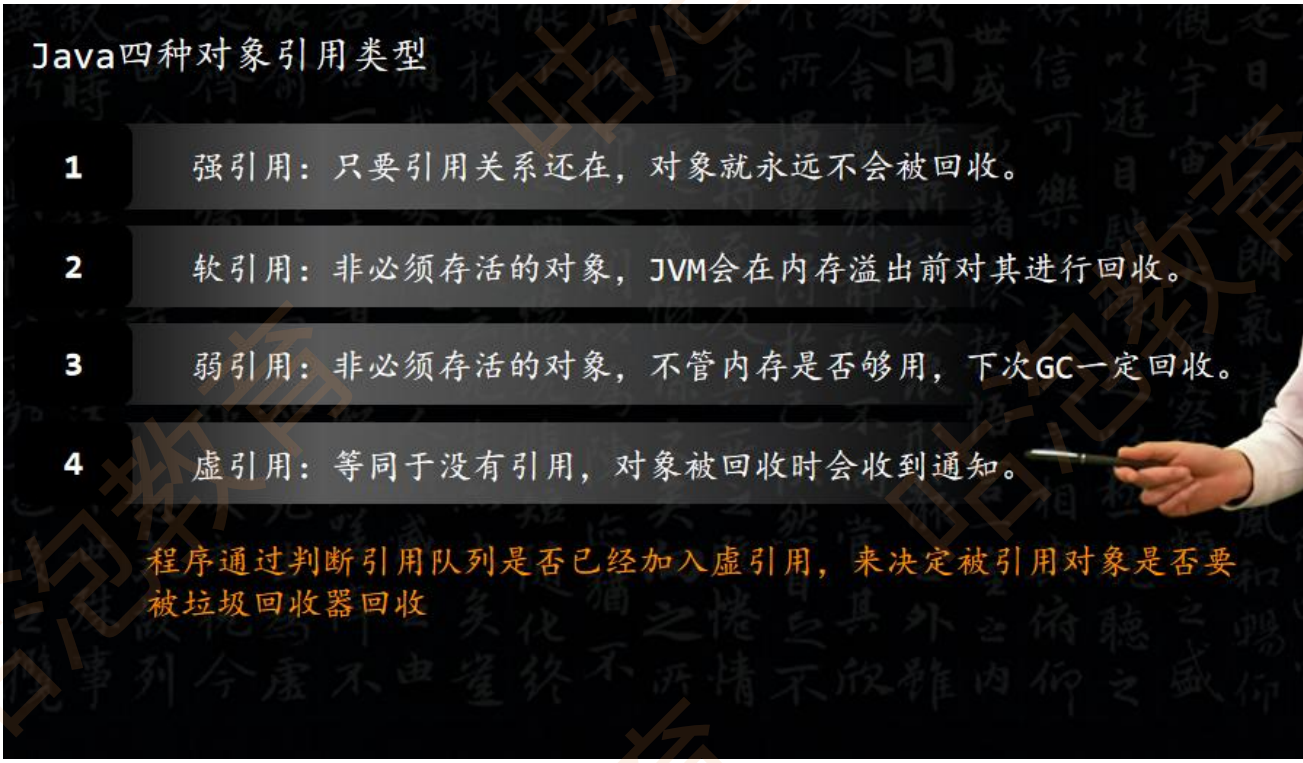
2、软引用：非必须存活的对象，JVM会在内存溢出前对其进行回收。

是一种相对于强引用来说弱一些的引用。可以让对象豁免一些垃圾收集的操作。只有当JVM判断内存不足的时候，才会试图回收引用指向的对象。那软引用通常是用来实现内存敏感的一些缓存，如果还有空闲内存就可以暂时保留缓存，当内存不足的时候会清理掉。这样就可以保证使用缓存的同时，不会耗尽内存。



3、弱引用：非必须存活的对象，不管内存是否够用，下次GC一定回收。

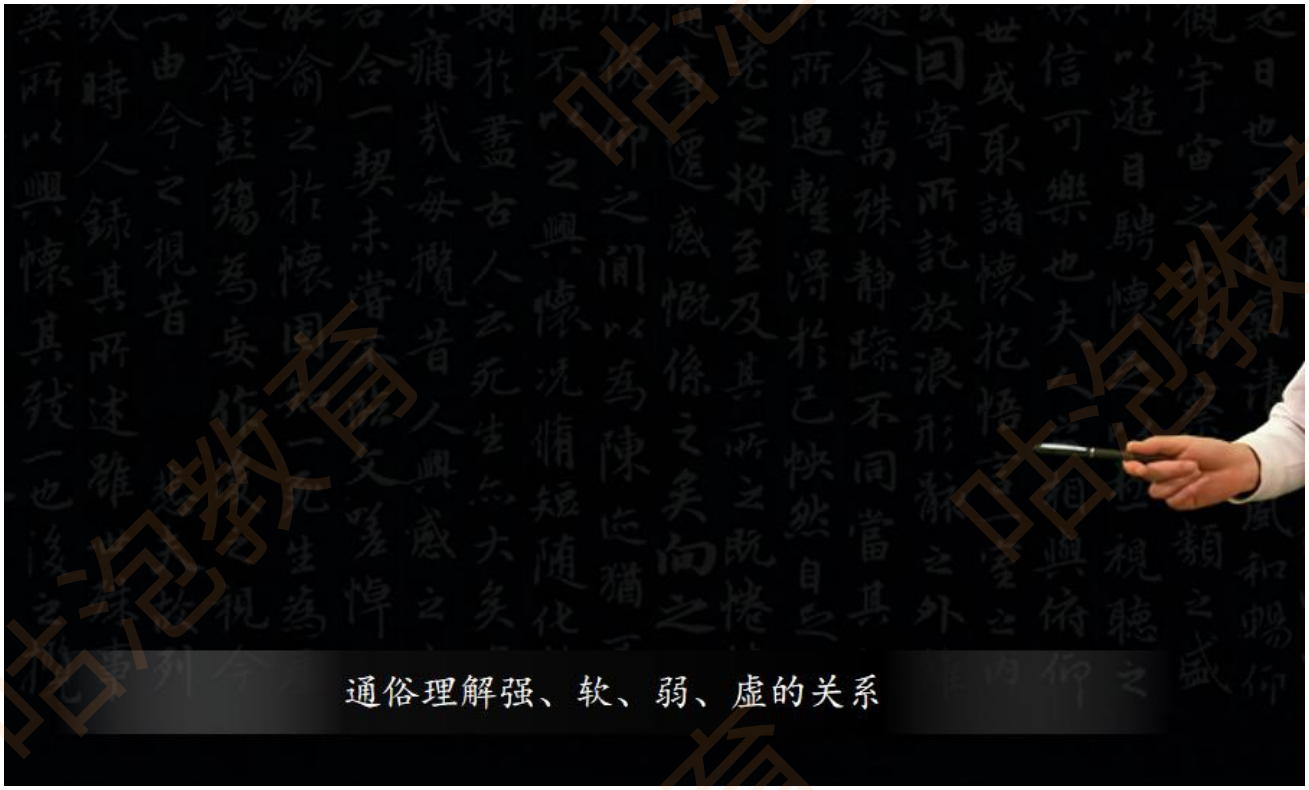
弱引用是相对于强引用而言的，它是允许存在引用关联的情况下，能被垃圾回收的对象。在垃圾回收器线程扫描它所管辖的内存区域的过程中，一旦发现只具有弱引用的对象的时候，不管当前的内存空间是否足够，垃圾回收器都会回收这个内存。



4、虚引用：等同于没有引用，对象被回收时会收到通知。

虚引用不会决定对象的生命周期，它提供一种确保对象被finalize以后，去做某些事情的机制。当垃圾回收期准备去回收一个对象的时候，如果发现他还有虚引用，就会在回收对象的内存之前，把这个虚引用加入到与之关联的引用队列中，程序可以通过判断引用队列是否已经加入虚引用，来决定被引用对象是否要被垃圾回收器回收。然后，我们就可以在引用对象被回收之前执行一些必要的操作。所以，虚引用必须和引用队列一起使用，

（以下需要配图）



以上的分享还是比较官方，举个通俗例子，大家就会更容易理解。

强引用就好比电视剧中的男主角，怎么都死不了。

软引用就像女主角，虽有一段经历，还是没走到最后。

弱引用就是男二号，注定用来牺牲的。

虚引用就是路人甲了。

【结尾】

以上就是我对Java中4种引用对象的理解。有兴趣的小伙伴，可以更进一步去研究一下。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

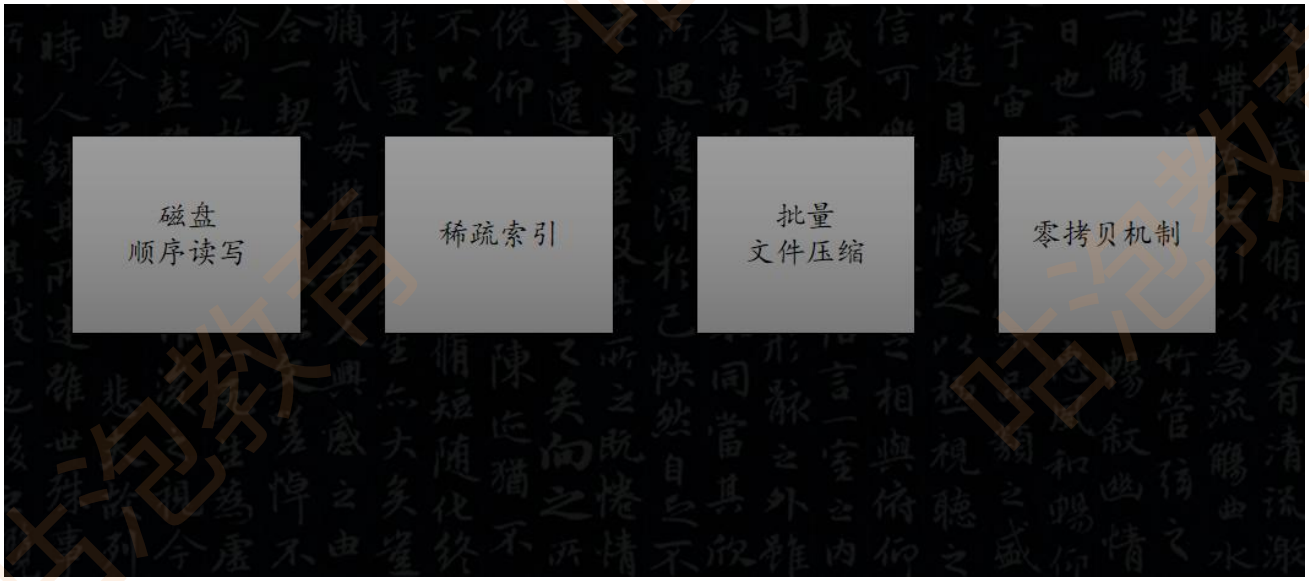
9.9 Kafka为什么这么快?

【引言】

最近一位小伙伴去某滴面试，在第二面的时候遇到了这个问题：说”请你简单说一下，Kafka为什么这么快？ “，然后，这位小伙伴努力在大脑里检索了很久，没有回答上来。

那么今天，我给大家分析一下，来看看对这个问题的理解。

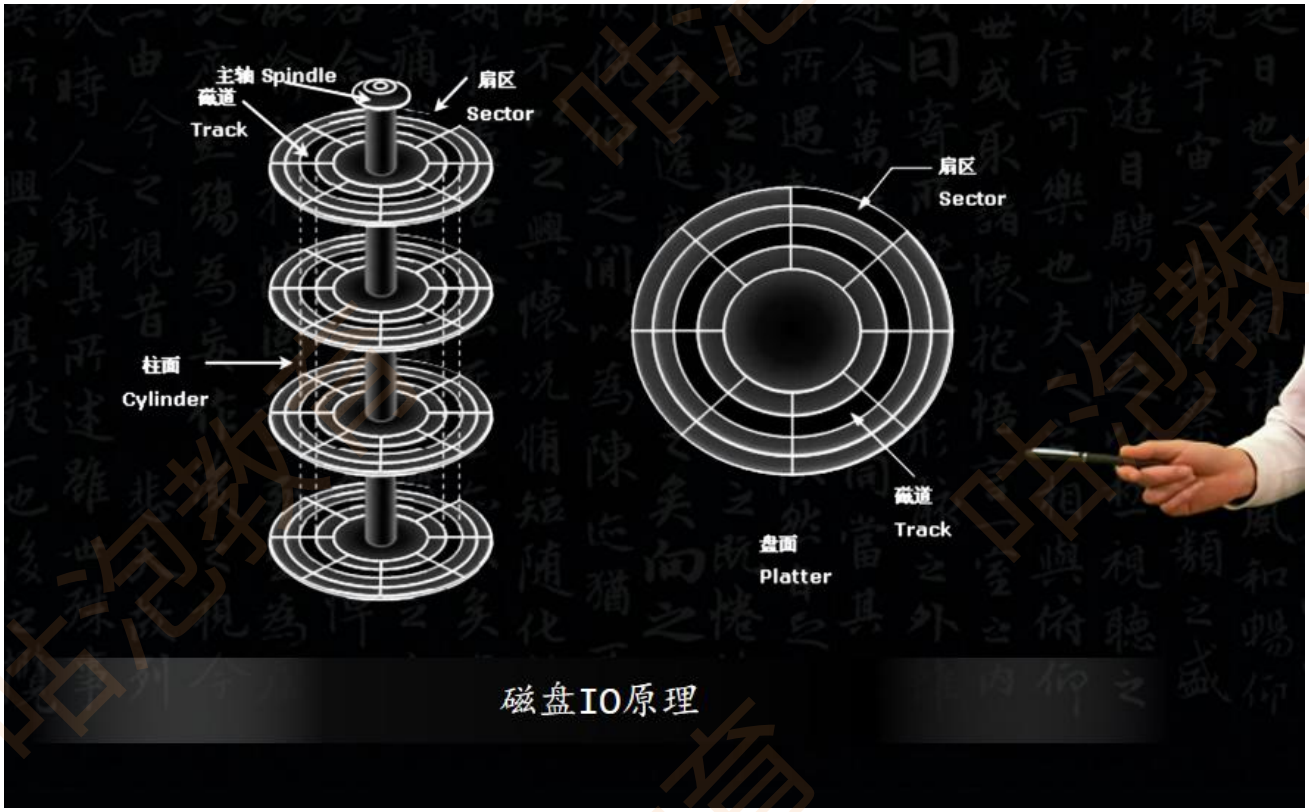
另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。



Kafka是一个号称能用普通的PC机也能处理超千万亿的消息吞吐量的实时消息流处理平台。我认为Kafka之所以能支持如此大的吞吐量，而且还能做到性能优秀主要有四个原因：分别是磁盘顺序读写、稀疏索引、批量文件压缩和零拷贝机制。下面分别给大家详细介绍一下：

1、磁盘顺序读写

首先要讲一下，磁盘寻址的过程，如图所示：



这个是磁盘的构造。磁盘的盘片不停地旋转，磁头会在磁盘表面画出一个圆形轨迹，这个就叫磁道。从内到位半径不同有很多磁道。然后又用半径线，把磁道分割成了扇区（两根射线之内的扇区组成扇面）。如果要读写数据，必须找到数据对应的扇区，这个过程就叫寻址。

如果读写的多条数据在磁盘上是分散的，寻址会很耗时，这叫随机I/O。

如果读写的数据在磁盘上是集中的，不需要重复寻址的过程，这叫顺序I/O。

而Kafka的Message是不断追加到本地磁盘文件末尾的，而不是随机的写入，这使得Kafka写入吞吐量得到了显著提升。

在一定条件下测试，磁盘的顺序读写可以达到53.2M每秒，比内存的随机读写还要快。

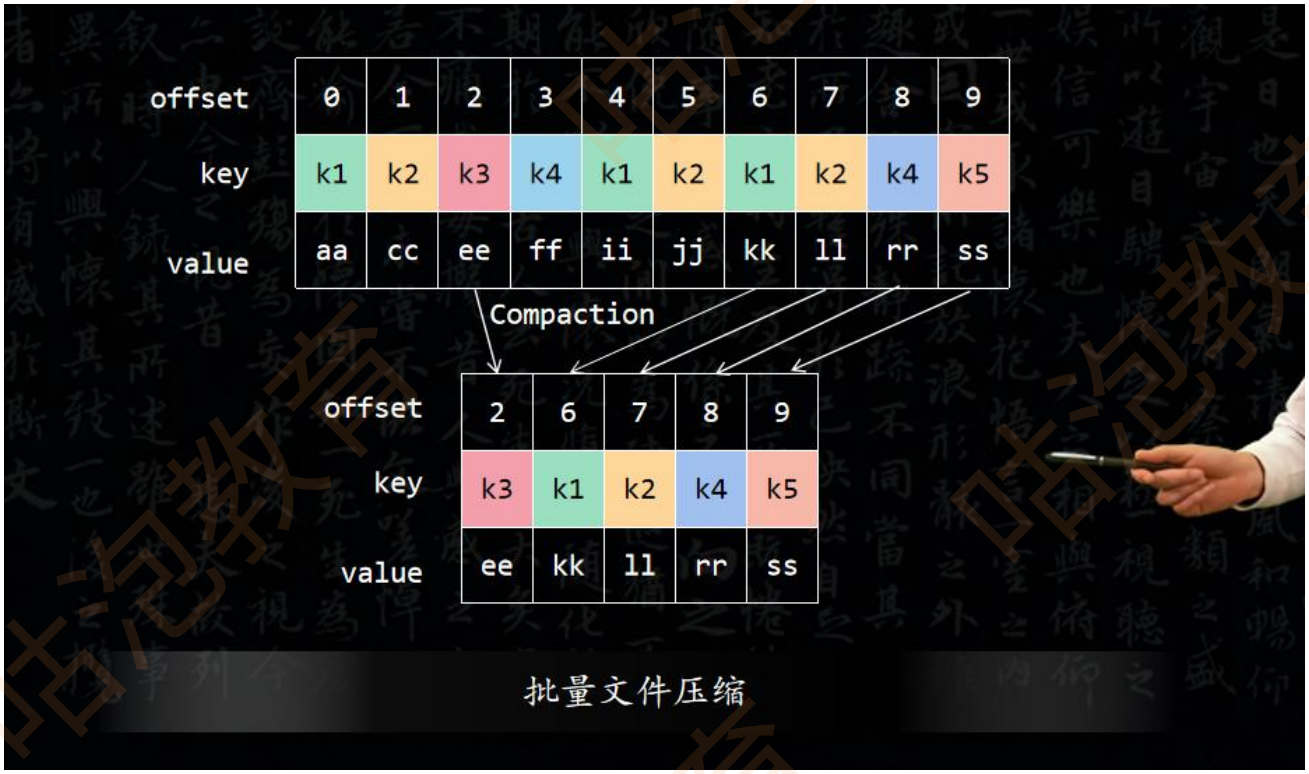
2、稀疏索引

Kafka的索引并不是每一条消息都会建立索引，而是一种稀疏索引。如图所示，

也就是说，Kafka插入一批消息才会产生一条索引记录。后续利用二分查找，可以大大提高检索效率。

3、批量文件压缩

Kafka默认不会删除数据，它会把所有的消息都变成一个批量的文件。如图所示，它会把相同的Key合并为最后一个Value。



这样对消息进行合理的批量压缩，可以减少网络IO损耗。

4、零拷贝机制

操作系统的虚拟内存分成两部分，一部分是内核空间，一部分是用户空间。这样就可以避免用户进程直接操作内核，保证内核安全。如图所示：

正常情况下，如果用户要从磁盘读取数据，必须先把数据从磁盘拷贝到内核缓冲区，然后在从内核缓冲区到用户缓冲区，最后才能返回给用户。

在Linux操作系统里面提供了一个sendfile函数，可以实现“零拷贝”。意思就是不需要经过用户缓冲区，可以直接把数据拷贝到网卡。

而Kafka中文件传输最终调用的是Java NIO 库里的 transferTo 方法，实际上最后就会使用到 Linux sendfile() 系统调用函数。零拷贝技术可以大大地提升文件传输的性能。

【结尾】

这道面试题涉及到一些计算机底层的原理，基本上也是业务程序员的知识盲区。但我想给大家一个建议，做开发其实和建房子一样，要想楼层更高更稳，首先地基要打牢固。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

9.10 kafka 如何保证消息的有序性?

最近很多同学去面试，都被问到“MQ如何保证消费顺序性”这样一个问题

很多人都没回到上来，特别是面试官在面试的时候，还会再补充一句：“还有没有其他方案”。

然后求职者一脸懵逼，实在不知道该怎么回答。

Hi，大家好，我是Mic，一个工作了14年的程序员和创业者。

今天我们来分析一道 这样的面试题，“MQ是如何保证消费顺序性”

下面看看高手对这个问题的回答。

高手

好的，这个问题我从两个方面来回答。

kafka为什么会存在无序消费

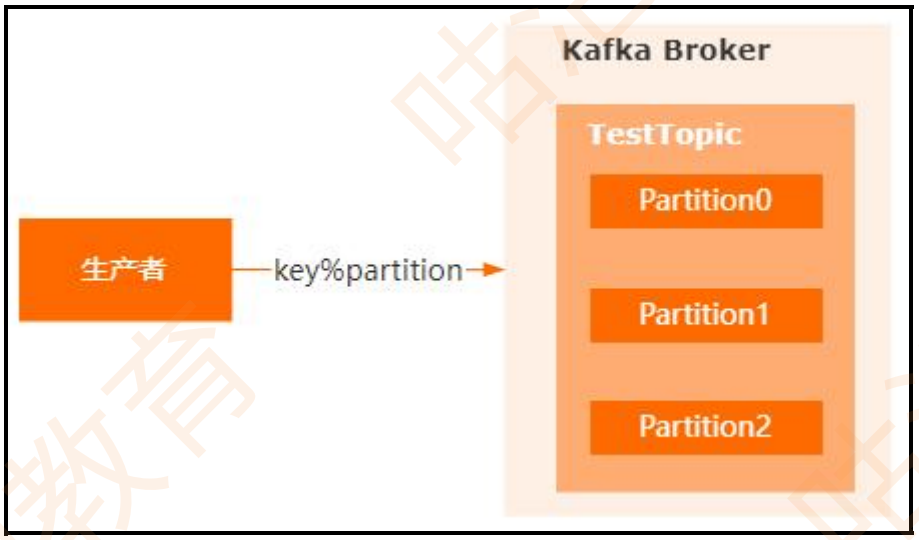
kafka如何保证有序消费

首先，在kafka的架构里面，用到了Partition分区机制来实现消息的物理存储，

在同一个topic下面，可以维护多个partition来实现消息的分片。

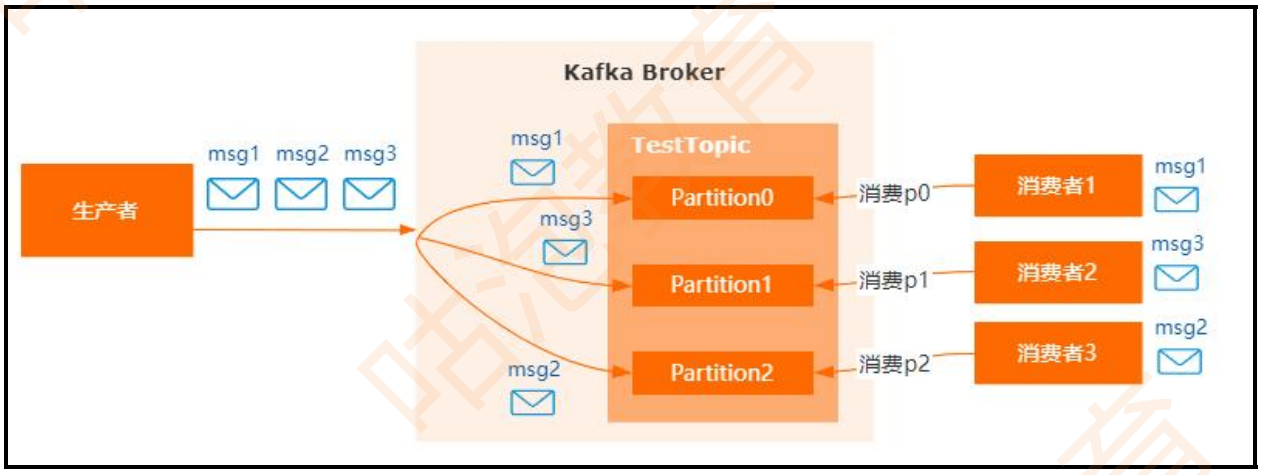
生产者在发送消息的时候，会根据消息的key进行取模，来决定把当前消息存储到哪个partition里面。

并且消息是按照先后顺序有序存储到partition里面的。



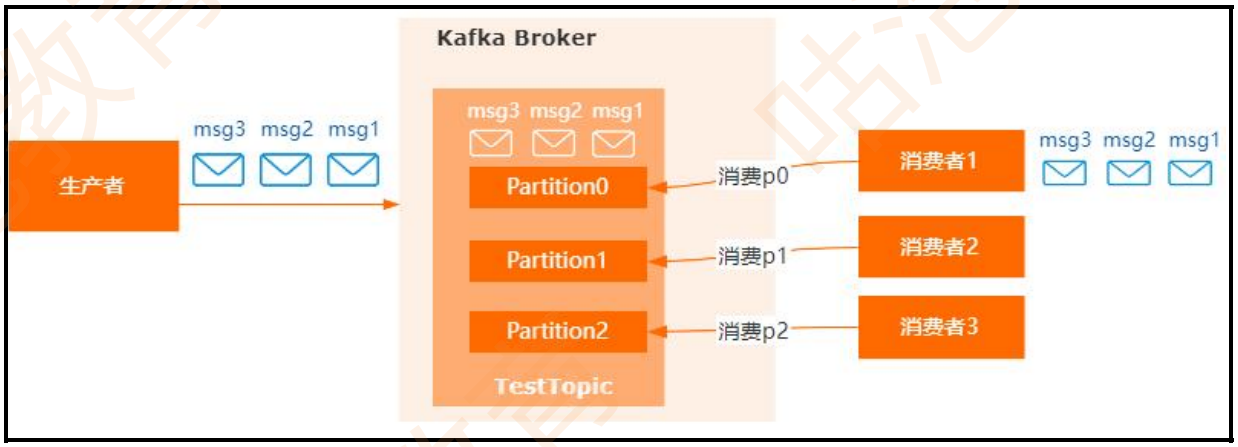
在这种情况下，假设有一个topic存在三个partition，而消息正好被路由到三个独立的partition里面。

然后消费端有三个消费者通过balance机制分别指派了对应消费分区。因为消费者是完全独立的网络节点，所有可能会出现，消息的消费顺序不是按照发送顺序来实现的，从而导致乱序的问题。



针对这个问题，一般的解决办法就是自定义消息分区路由的算法，然后把指定的key都发送到同一个Partition里面。

接着指定一个消费者专门来消费某个分区的数据，这样就能保证消息的顺序消费了。



另外，有些设计方案里面，在消费端会采用异步线程的方式来消费数据来提高消息的处理效率，那这种情况下，因为每个线程的消息处理效率是不同的，所以即便是采用单个分区的存储和消费也可能会出现无序问题，针对这个问题的解决办法就是在消费者这边使用一个阻塞队列，把获取到的消息先保存到阻塞队列里面，然后异步线程从阻塞队列里面去获取消息来消费。

以上就是我对这个问题的理解。

面试点评

关于这个问题，有些面试官还会这样问： 如果我不想把消息路由到同一个分区，但是还想实现消息的顺序消费，怎么办?

严格来说，kafka只能保证同一个分区内的消息存储的有序性。

如果一定要去实现，也不是不行，只是代价太大了没必要。

虽然没有标准答案，但是面试官要这么问，无非就是想考察你面对复杂问题的时候是如何思考的，以及你的技术底蕴怎么样。

9.11 Nacos ✂选举机制了解嘛？说下 Raft 算法?

Nacos 作为配置中心✂功能✂基于 **Raft 算法**来实现✂。

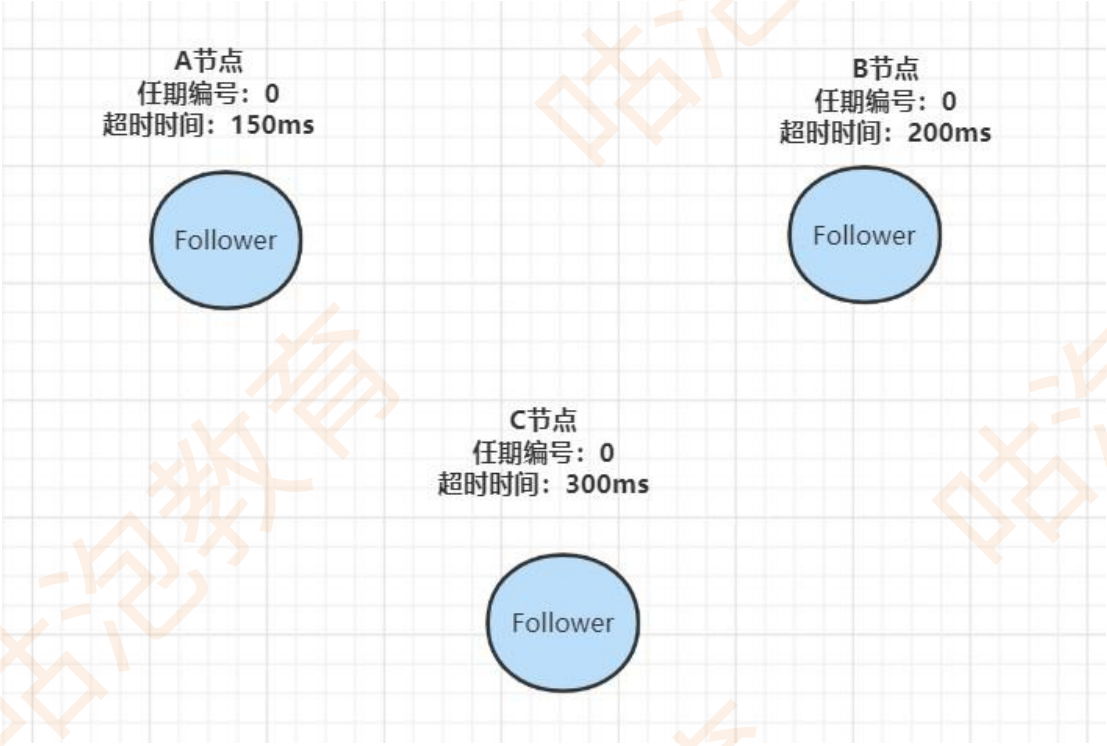
Raft 算法✂分布式系统开发首选✂共识算法，它通过"一切以领导者为准"✂方式，实现一系列值✂共识和各节点日志✂一致。

Raft 选举规程 涉及三种角色和任期（Term），

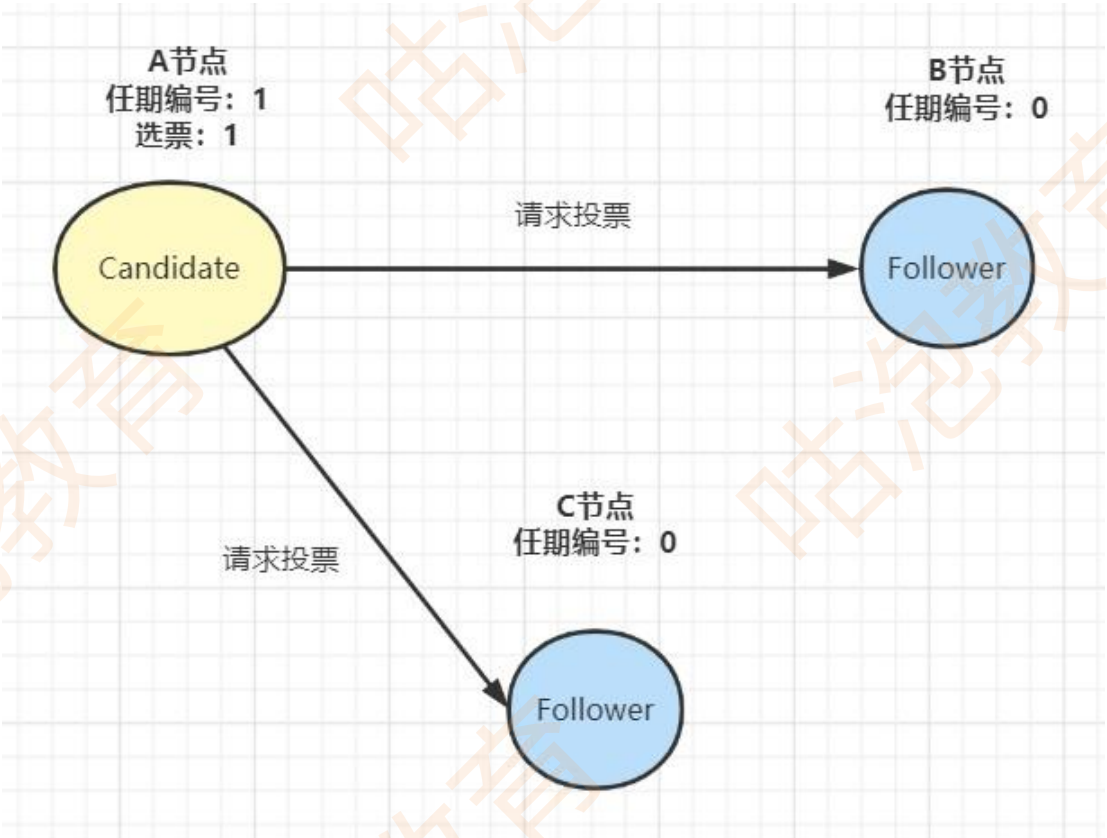
- Follower：默默地⓫收和处理来自 Leader ✂消息，当等待 Leader 心跳信息超时✂时候，就主动站出来，推荐自己当 Candidate。
- Candidate：向其他节点发送投票请求，通知其他节点来投票，如果赢得了大多数（N/2＋1）选票，就晋升 Leader。
- Leader：负责处理客户端请求，进行日志复制等操作，每一轮选举✂目标就✂选出一个领导者；领导者会不断地发送心跳信息，通知其他节点“我✂领导者，我还 活着，你们不要发起新✂选举，不用找个新领导者来替代我。”
- Term：这跟民主社会✂选举很像，每一届新✂履职期称之为一届任期

领导选举过程

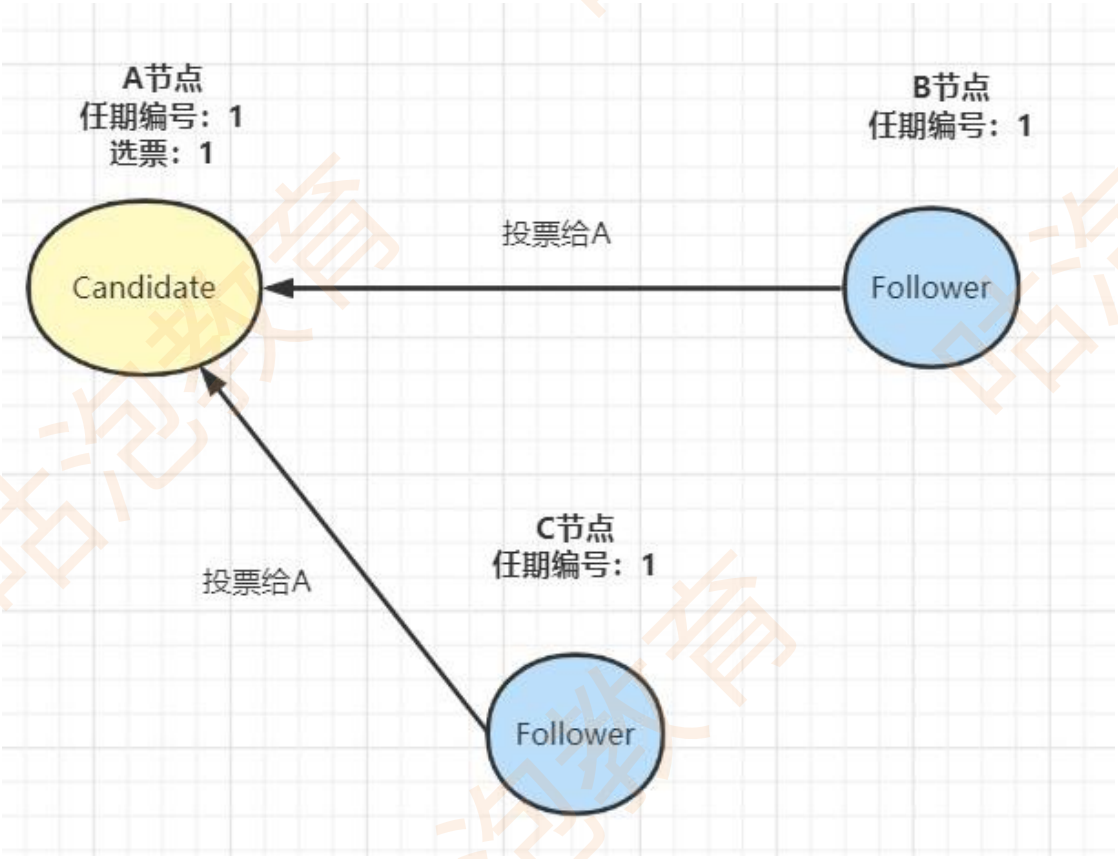
1. 在初始时，集群中所有✂节点都✂ Follower 状态，都被设定一个随机选举超时时间（一般 150ms-300ms）：



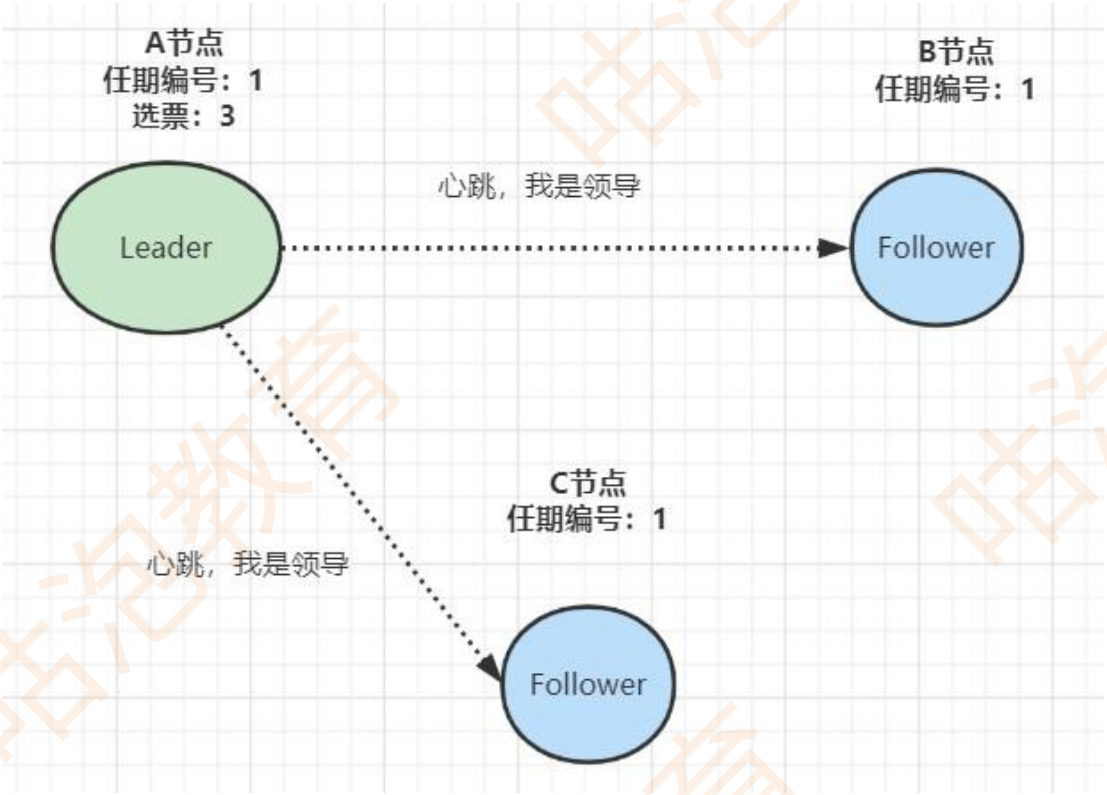
1. 如果 Follower 在规定✂超时时间，都没有收到来自 Leader ✂心跳，它就发起选举：将自己✂状态切为 Candidate，增加自己✂任期编号，然后向集群中✂其它 Follower 节点发送请求，询问其✂否选举自己成为 Leader：



1. 其他节点收到候选人 A ✂请求投票✲息后，如果在编号为 1 ✂这届任期内还没有进行过投票，那么它将✂选票投给节点 A，并增加自己✂任期编号：



1. 当收到来自集群中过半节点✂❿受投票后，A 节点即成为本届任期内 Leader，他将周期性地发送心跳✲息，通知其他节点我✂ Leader，阻止 Follower 发起新✂选举：



9.12 谈谈你对Nacos配置动态更新原理的理解

【引言】

Nacos作为阿里的开源中间件在Spring Cloud生态以后，不管是作为配置中心，还是作为注册中心，因为它简单易用的特性，在互联网公司被广泛运用。随后，大家会发现Nacos相关的面试题也就越来越多了。

这不，又有一位工作8年的小伙伴，被问到这样一道面试题，说，请你详细介绍一下Nacos客户端是如何实现配置动态更新的。今天，我给大家分享一下我对这个问题的理解。

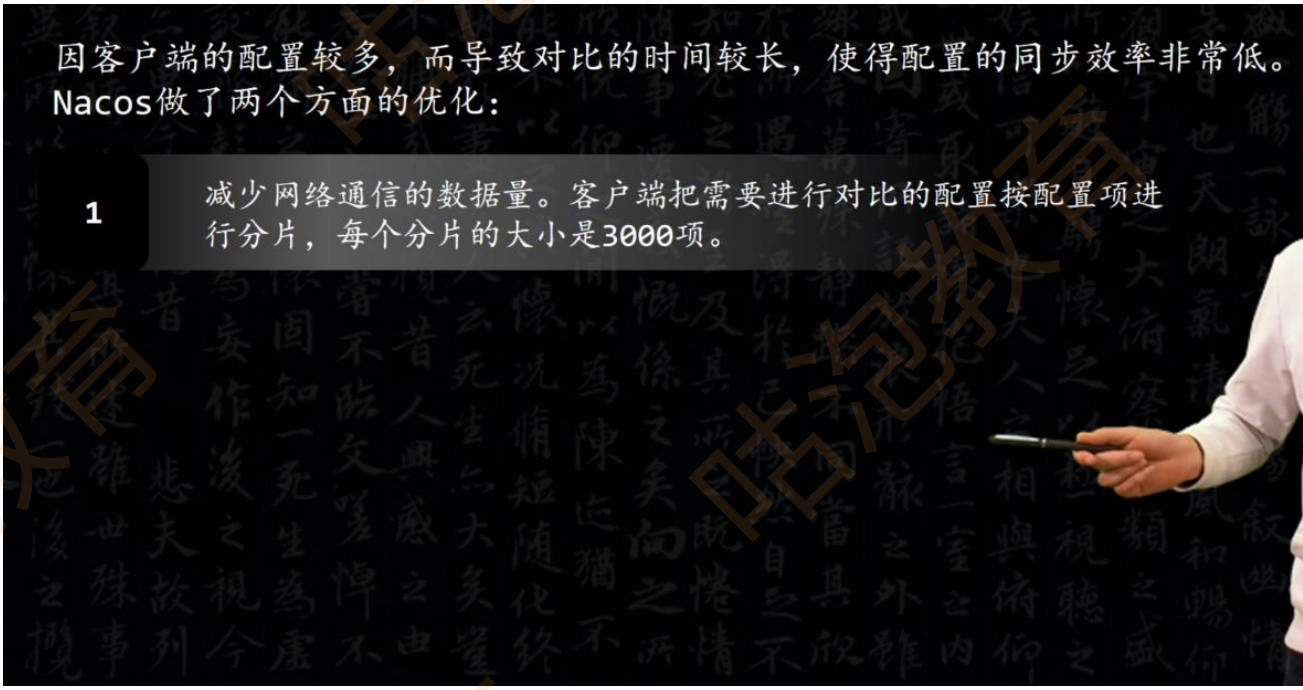
另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

原理分析

下面我给大家分享一下我对Nacos配置动态更新原理的理解。

首先，Nacos采用的是长轮询的方式。也就是说，由Nacos Client向Nacos Server端去发起配置更新查询的请求。所谓长轮询，就是客户端发起一次轮询请求到服务器端，当服务器端的配置没有任何变更的时候，这个连接会一直打开，直到服务端有配置变更或者连接超时之后才返回。

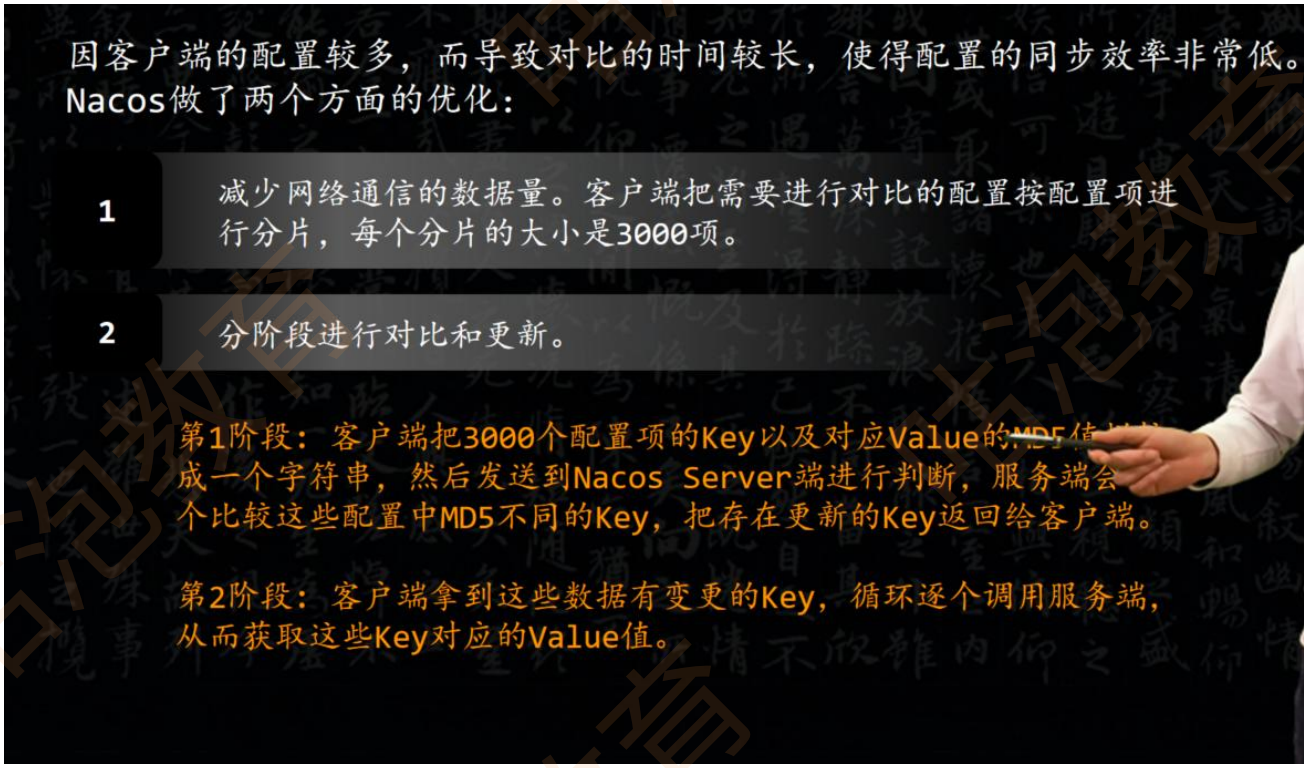
Nacos Client端需要去获取服务端变更的配置内容，但前提是需要先进行比较。也就是说将客户端本地缓存的配置信息和服务器端获取的配置信息进行比较。一旦发现本地缓存的配置内容和服务端的配置内容有差异，那么，就表示服务器端的配置有更新。于是，需要把更新的配置拉到本地，在这个过程中，有可能因为客户端的配置比较多，而导致对比的时间较长，使得配置的同步效率非常低。



于是Nacos针对这样一个场景，做了两个方面的优化：

1、减少网络通信的数据量。客户端把需要进行对比的配置按配置项进行分片，每个分片的大小

是3000项，也就是说每一次最多拿3000个配置项去Nacos Server端进行对比。



2、分阶段进行对比和更新。第一阶段，客户端把3000个配置项的Key以及对应Value的MD5值拼接成一个字符串，然后发送到Nacos Server端进行判断，服务端会逐个比较这些配置中MD5不同的Key，把存在更新的Key返回给客户端。第二阶段，客户端拿到这些数据有变更的Key，循环逐个调用服务端，从而获取这些Key对应的Value值。

那么，这两个优化的核心目的是去减少网络通信中数据包的大小。把一次大数据包的通信拆分成了多个小数据包的通信。虽然会增加网络通信的次数，但是，提高了整体的数据传输的性能。

最后，再加上长轮询的方式，既减少了Pull的轮询次数，又利用了长轮询的优势，很好地实现了配置动态更新的同步功能。

【结尾】

好了，以上就是我对这个问题的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

9.13 聊一聊 TCC 补偿机制

TCC ✂分布式事务✂一种解决方案。它采用了补偿机制，其核心思想✂：针对每个操作，都要注册一个与其对应✂确认和补偿（撤销）操作。TCC（Try-Confirm-Cancel）包括三段流程：

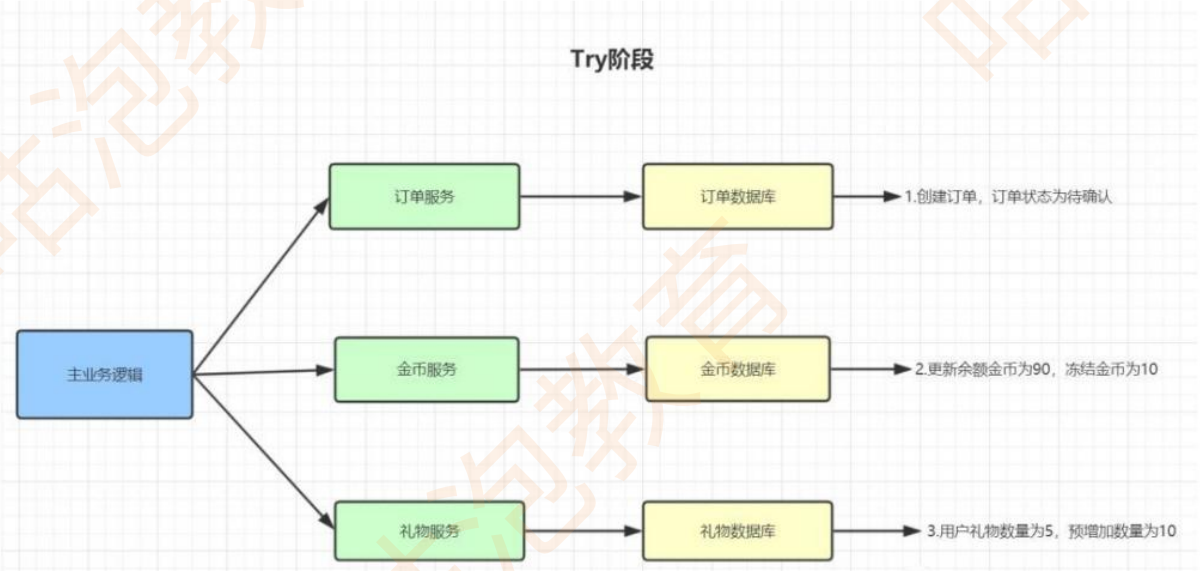
- try 阶段：尝试去执行，完成所有业务✂一致性检查，预留必须✂业务资源。
- Confirm 阶段：该阶段对业务进行确认提交，不做任何检查，因为 try 阶段已经检查过了，默认 Confirm 阶段✂不会出错✂。
- Cancel 阶段：若业务执行失败，则进入该阶段，它会释放 try 阶段占用✂所有业务资源，并回滚 Confirm 阶段执行✂所有操作。

假设用户 A 余额为 100 金币，拥有✂礼物为 5 朵。A 花了 10 个金币，下订单，购买 10 朵玫瑰。余额、订单、礼物都在不同数据库。

下面再拿用户下单购买礼物作为例子来模拟 TCC 实现分布式事务✂过程：

TCC ✂ Try 阶段：

- 生成一条订单记录，订单状态为待确认。
- 将用户A ✂账户金币中余额更新为 90，冻结金币为 10（预留业务资源）
- 将用户✂礼物数量为 5，预增加数量为 10。
- Try 成功之后，便进入 Confirm 阶段
- Try 过程发生任何异常，均进入 Cancel 阶段

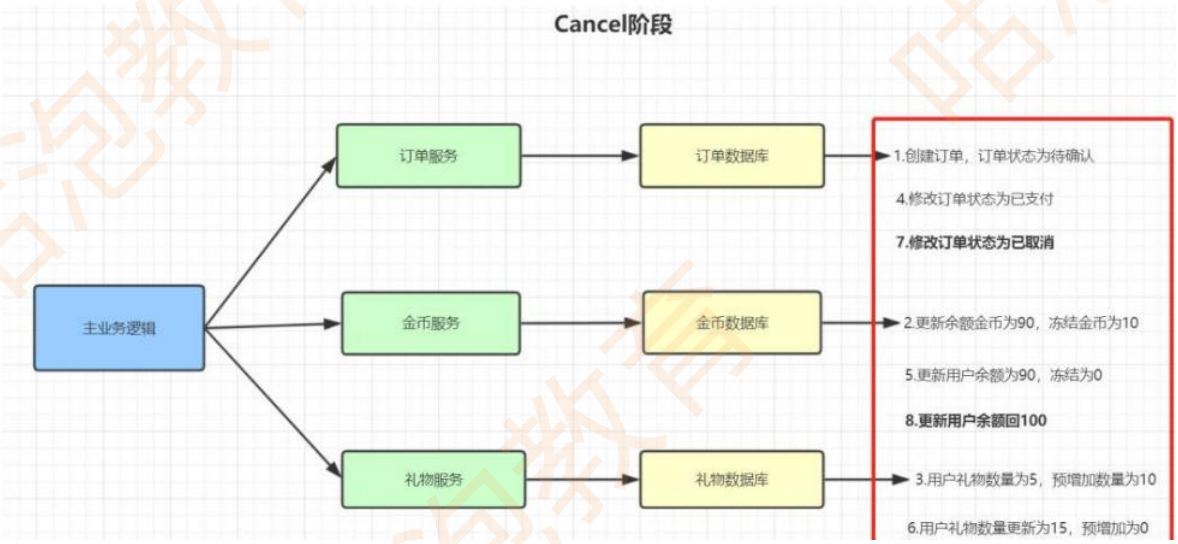


TCC ✂ Confirm 阶段：

- 订单状态更新为已支付
- 更新用户余额为 90，可冻结为 0
- 用户礼物数量更新为 15，预增加为 0
- Confirm 过程发生任何异常，均进入 Cancel 阶段
- Confirm 过程执行成功，则该事务结束

TCC ✂ Cancel 阶段：

- 修改订单状态为已取✲
- 更新用户余额回 100
- 更新用户礼物数量为 5



- TCC ✂优点✂可以自定义数据库操作✂粒度，降低了锁冲突，可以提升性能
- TCC ✂缺点✂应用侵入性强，需要根据网络、系统故障等不同失败原因实现不同✂回滚策略，实现难度大，一般借助 TCC 开源框架，ByteTCC，TCC-transaction，Himly。

腾讯云15 连问（10）

大家好，最近去面试腾讯云，给大家。如果有不正确✂，欢迎指出哈，一起进步。

10.1 聊聊项目，好✂设计，好✂代码

项目✂话，你可以聊聊你平时做✂项目，尤其有亮点✂项目。如果没有什么特别亮点✂项目，也可以说说一些好✂设计，或者你优化了什么⑩口，性能提升了多少，优化了什么慢 SQL 都可以。甚至✂一些好✂代码写法都可以。

- 如果✂讲优化⑩口✂如果✂代码优化细节

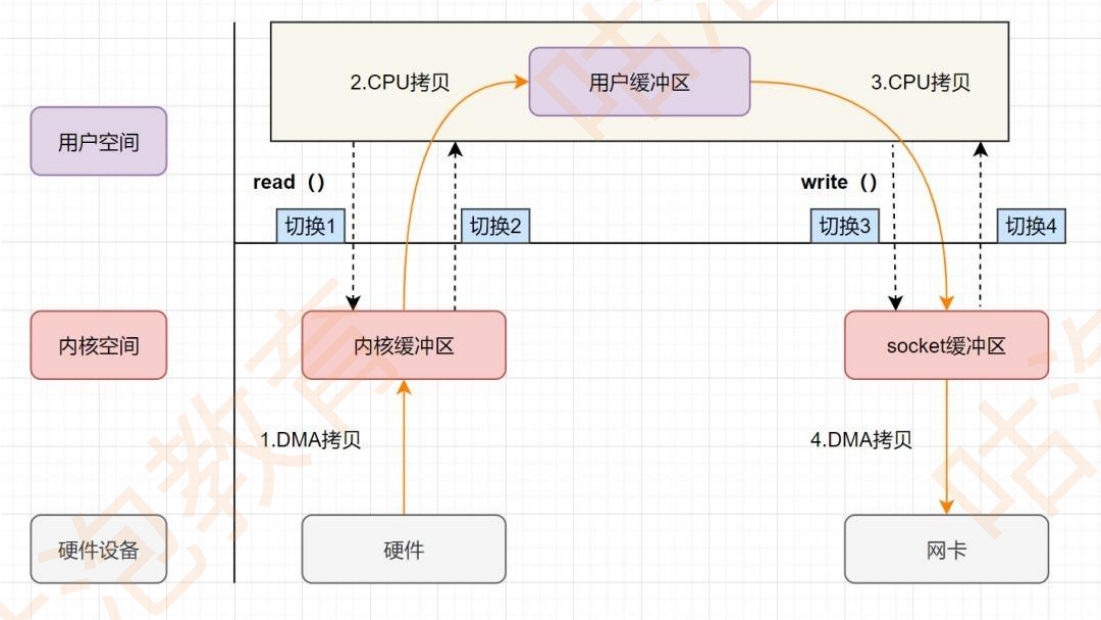
10.2 谈谈什么✂零拷贝?

零拷贝✂指计算机执行 IO 操作时，CPU 不需要将数据从一个存储区域复制到另一个存储区域，从而可以减少上下文切换以及 CPU ✂拷贝时间。它✂一种I/O 操作优化技术。

传统 IO ✂执行流程

传统✂ IO 流程，包括 read 和 write ✂过程。

- read：✂数据从磁盘读取到内核缓冲区，再拷贝到用户缓冲区
- write：先✂数据写入到 socket 缓冲区，最后写入网卡设备。



1. 用户应用进程调用 read 函数，向操作系统发起 IO 调用，上下文从用户态转为内核态（切换 1）
2. DMA 控制器✂数据从磁盘中，读取到内核缓冲区。
3. CPU ✂内核缓冲区数据，拷贝到用户应用缓冲区，上下文从内核态转为用户态（切换 2），read 函数返回
4. 用户应用进程通过 write 函数，发起 IO 调用，上下文从用户态转为内核态（切换 3）

5. CPU 将用户缓冲区中✂数据，拷贝到 socket 缓冲区
6. DMA 控制器✂数据从 socket 缓冲区，拷贝到网卡设备，上下文从内核态切换回用户态（切换 4），write 函数返回

传统 IO ✂读写流程，包括了 **4 次上下文切换**（4 次用户态和内核态✂切换），**4 次数据拷贝**（两次 CPU 拷贝以及两次✂ DMA 拷贝)。

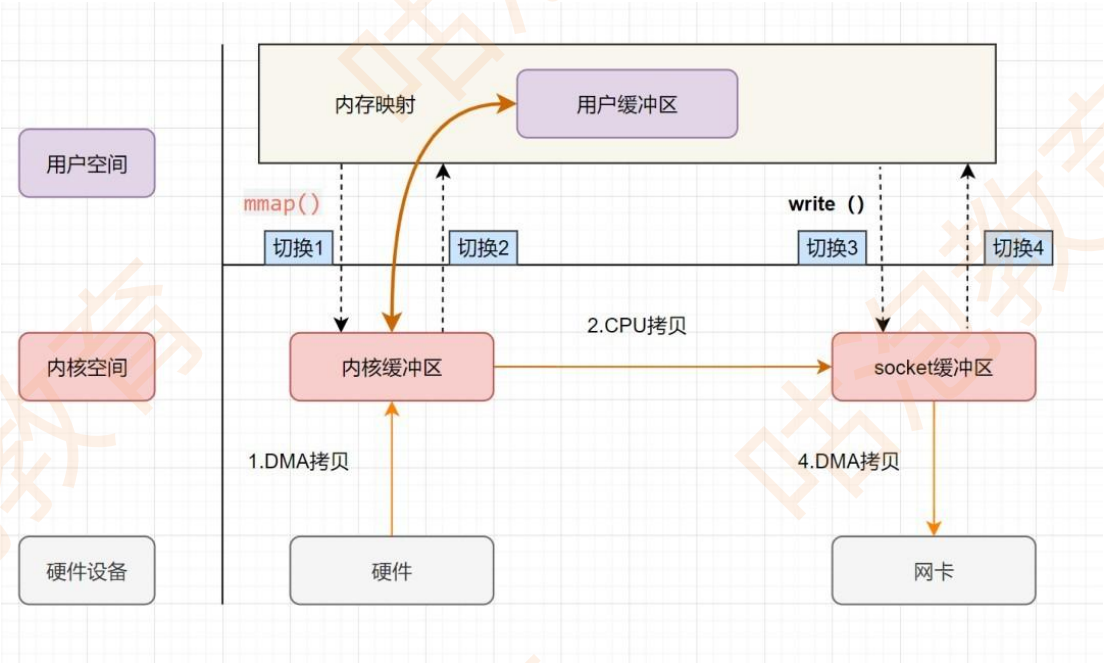
零拷贝实现方式:

零拷贝并不✂没有拷贝数据，而✂减少用户态/内核态✂切换次数以及 CPU 拷贝✂次数。零拷贝一般有这三种实现方式:

- mmap+write
- sendfile
- 带有 DMA 收集拷贝功能✂ sendfile

mmap+write

mmap 就✂用了虚拟内存这个特点，它将内核中✂读缓冲区与用户空间✂缓冲区进行映射，以减少数据拷贝次数!



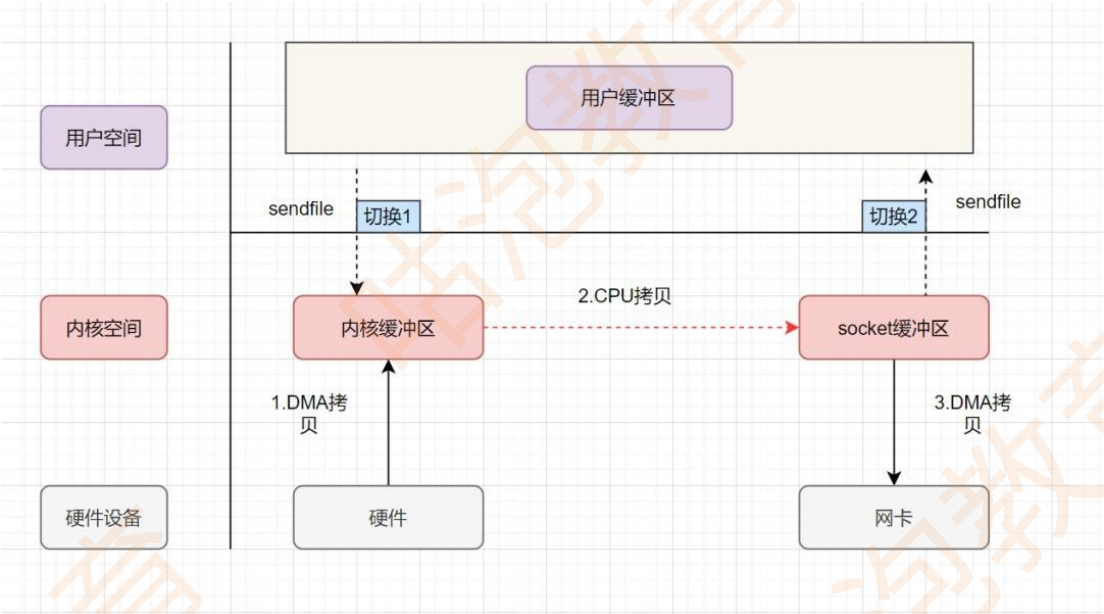
1. 用户进程通过 mmap 方法向操作系统内核发起 IO 调用，上下文从用户态切换为内核态。
2. CPU 利用DMA 控制器，✂数据从硬盘中拷贝到内核缓冲区。

3. 上下文从内核态切换回用户态，mmap 方法返回。
4. 用户进程通过 write 方法向操作系统内核发起 IO 调用，上下文从用户态切换为内核态。
5. CPU 将内核缓冲区✂数据拷贝到✂ socket 缓冲区。
6. CPU 利用DMA 控制器，✂数据从 socket 缓冲区拷贝到网卡，上下文从内核态切换回用户态，write 调用返回。

mmap+write 实现✂零拷贝，I/O 发生了 4 次用户空间与内核空间✂上下文切换，以及 3 次数据拷贝（包括了 2 次 DMA 拷贝和 1 次 CPU 拷贝）。

sendfile

sendfile 表示在两个文件描述符之间传输数据，它✂在操作系统内核中操作✂，避免了数据从内核缓冲区和用户缓冲区之间✂拷贝操作



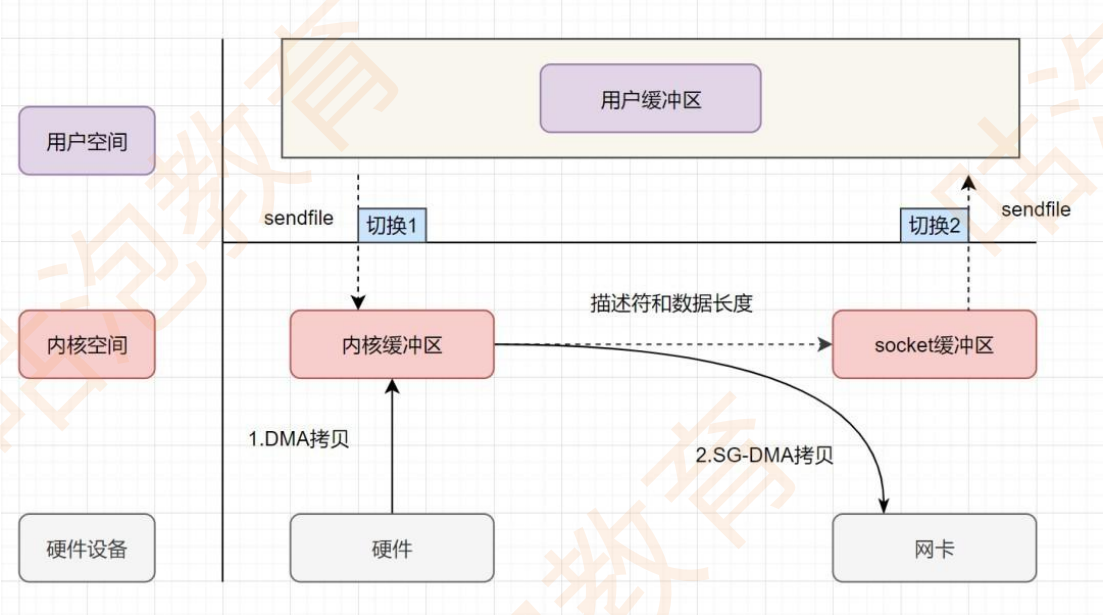
1. 用户进程发起 sendfile 系统调用，上下文（切换 1）从用户态转向内核态
2. DMA 控制器，✂数据从硬盘中拷贝到内核缓冲区。
3. CPU 将读缓冲区中数据拷贝到 socket 缓冲区
4. DMA 控制器，异步✂数据从 socket 缓冲区拷贝到网卡，
5. 上下文（切换 2）从内核态切换回用户态，sendfile 调用返回。

sendfile 实现✂零拷贝，I/O 发生了 2 次用户空间与内核空间✂上下文切换，

以及 3 次数据拷贝。其中 3 次数据拷贝中，包括了 2 次 DMA 拷贝和 1 次 CPU拷贝。

带有DMA 收集拷贝功能✂ sendfile

linux 2.4 版本之后，对 sendfile 做了优化升级，引入 SG-DMA 技术，其实就✂对DMA 拷贝加入了 scatter/gather 操作，它可以直⓾从内核空间缓冲区中将数据读取到网卡。使用这个特点搞零拷贝，即还可以多省去一次 CPU 拷贝。



1. 用户进程发起 sendfile 系统调用，上下文（切换 1）从用户态转向内核态
2. DMA 控制器，✂数据从硬盘中拷贝到内核缓冲区。
3. CPU ✂内核缓冲区中✂文件描述符信息（包括内核缓冲区✂内存地址和偏移量）发送到socket 缓冲区
4. DMA 控制器根据文件描述符信息，直⓾✂数据从内核缓冲区拷贝到网卡
5. 上下文（切换 2）从内核态切换回用户态，sendfile 调用返回。

可以发现，**sendfile+DMA scatter/gather** 实现✂零拷贝，I/O 发生了2次用户空间与内核空间✂上下文切换，以及 2 次数据拷贝。其中 2 次数据拷贝都✂包DMA 拷贝。这就✂真正✂ 零拷贝（Zero-copy）技术，全程都没有通过 CPU 来搬运数据，所有✂数据都✂通过 DMA 来进行传输✂。

TOM老师解析

什么是零拷贝，Betty是如何实现的?

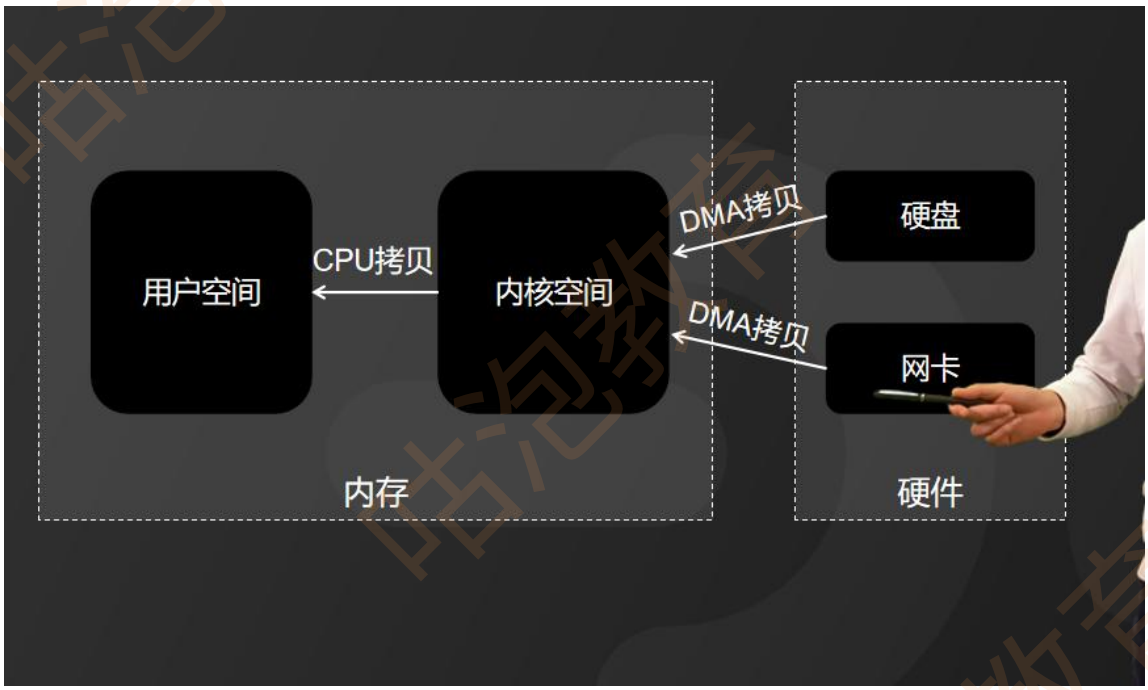
【引言】

呢作为一个高性能的网络通信框架，被越来越多互联网公司关注和重视。最近，有小伙伴在面试过程中被问到Netty是如何实现零拷贝的问题？，今天，我给大家来聊一聊。另外，往期面试题解析中配套的文档我已经准备好，想获得的可以在我的煮叶简介中找到。

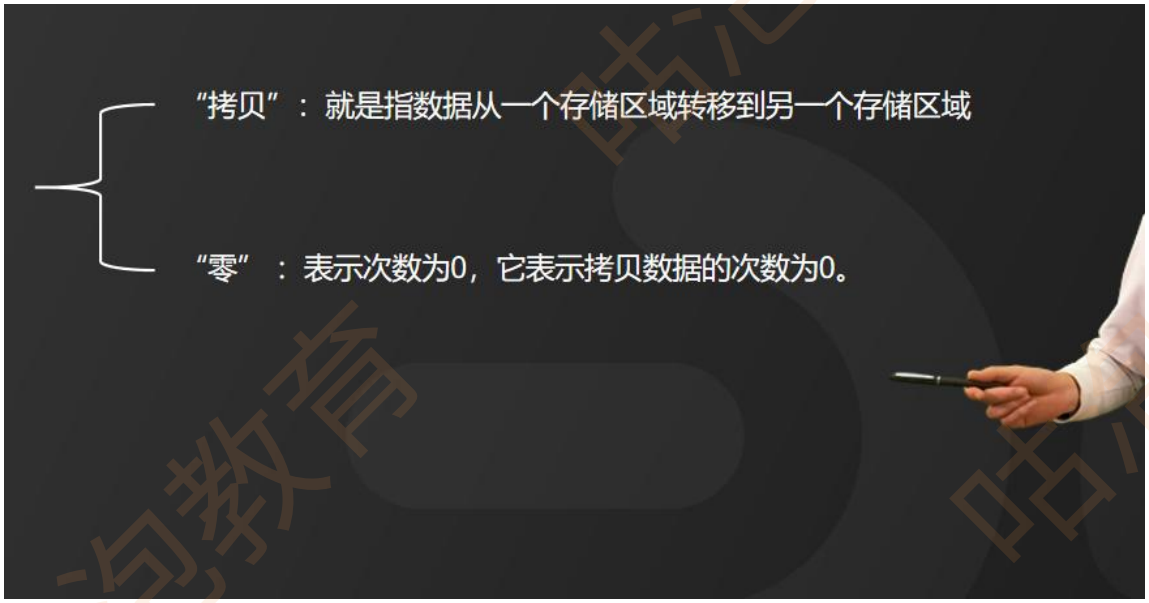
我们先来看什么是零拷贝？

1、什么零拷贝

在计算机中，完成数据传输，要么是通过网络，要么就是通过本地磁盘。通常完成一次完整I/O交互流程分为两阶段，首先拷贝到系统内核空间，由操作系统来完成；紧接着要拷贝到用户空间，这个由应用程序来完成，具体交互流程如下图所示。



那什么是零拷贝呢？顾名思义，零拷贝的意思包括两个部分，“零”和“拷贝”：



“拷贝”：就是指数据从一个存储区域转移到另一个存储区域。

“零”：表示次数为0，它表示拷贝数据的次数为0。

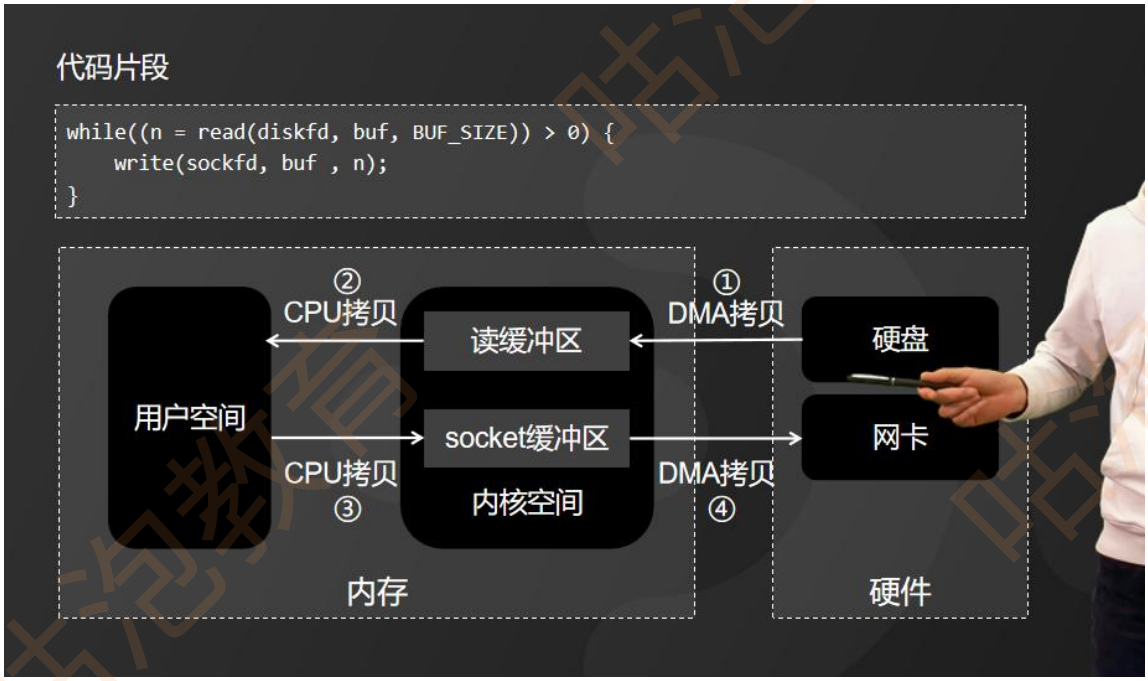
合起来“零拷贝”就是不需要将数据从一个存储区域复制到另一个存储区域。

2、为什么要零拷贝

零拷贝主要是指将系统内核空间的内存和用户空间的内存实现直接关联映射，从而省去了数据传输过程中的来回拷贝，也就是说，要完成数据传输数据拷贝次数为0次。

有这么一段代码，主要将服务端主机磁盘中的文件从已连接的socket发出去。关键实现代码如下：

```
while((n = read(diskfd, buf, BUF_SIZE)) > 0){
   write(sockfd, buf , n);
}
```



以上代码是用传统的IO编写的，它的执行过程大致是这样的：

首先，read()方法，会把数据从磁盘循环读取到内核缓冲区，再拷贝到用户缓冲区。

然后，write()方法先把数据写入到socket缓冲区，最后写入网卡设备。

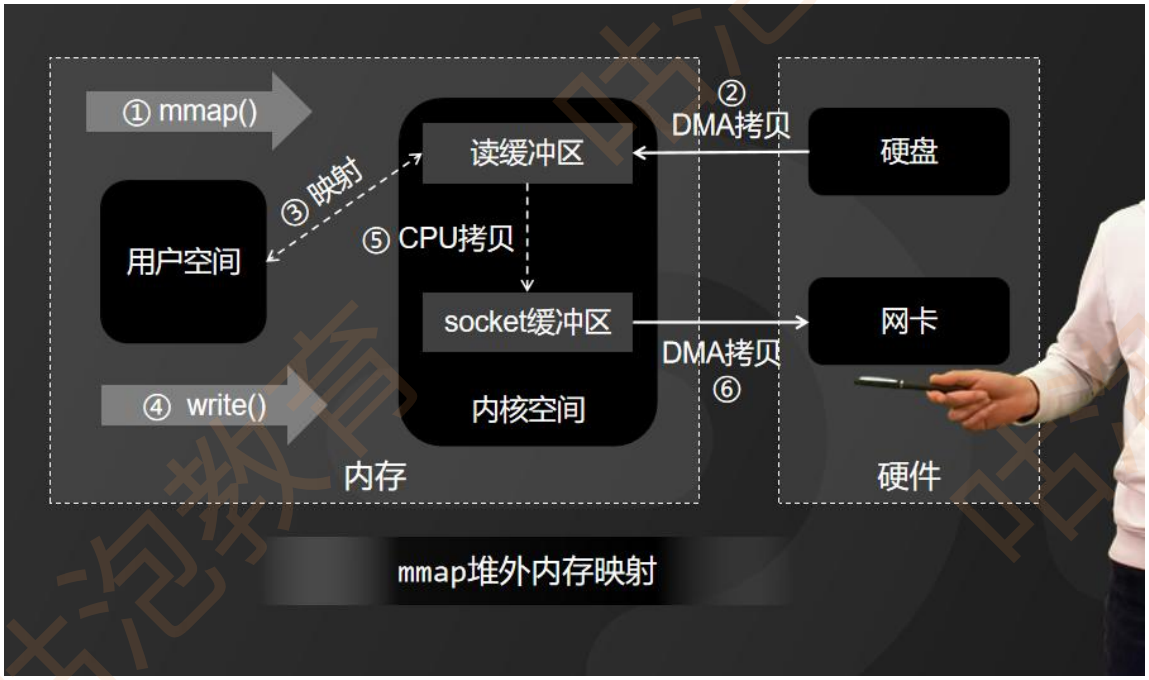
因为，传统的IO在在数据传输过程中拷贝次数太多，导致性能低下。为了减少拷贝次数，增加系统性能，才有了零拷贝的设计。另外，零拷贝并不是指完全没有文件的拷贝，只是减少了拷贝的次数。

3、零拷贝在Netty中的实现

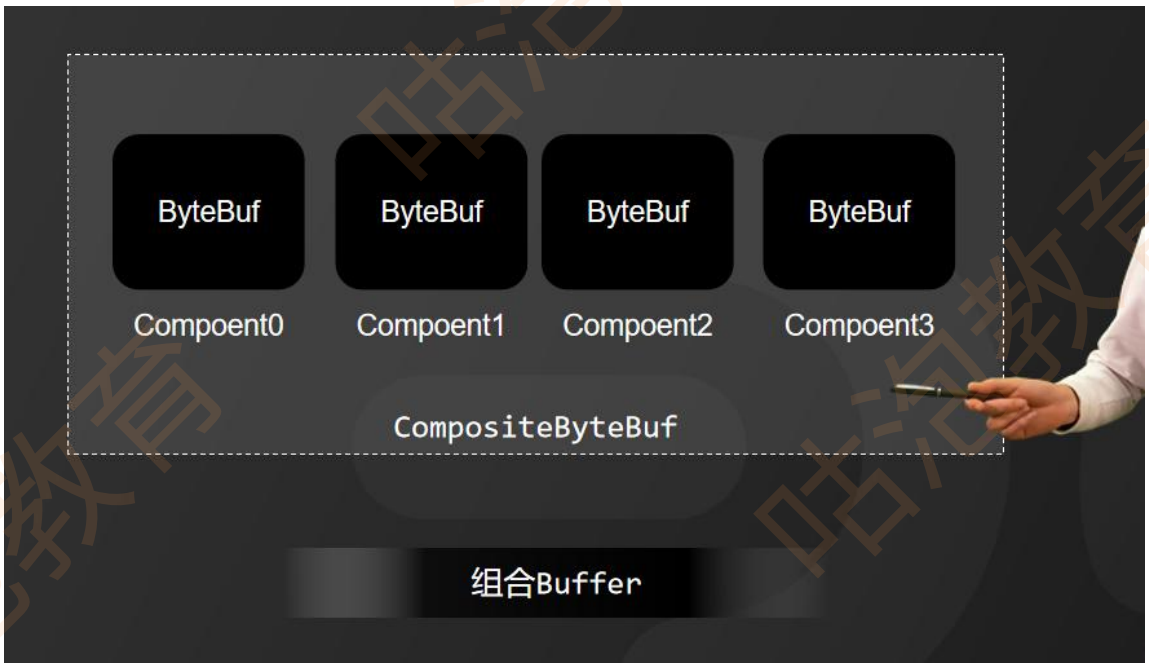
从流程图可以看出，传统IO的读写流程，包括了4次用户态和内核态的切换，也就是4次上下文切换，4次数据拷贝，其中两次CPU拷贝以及两次的DMA拷贝。关于上下文切换和DMA拷贝，在这里我们就不详细赘述了，以后可以单独拍摄一期视频来详细介绍。想了解的小伙伴可以在评论区回复666.

那在Netty中，是如何实现零拷贝的呢？有以下三种方式

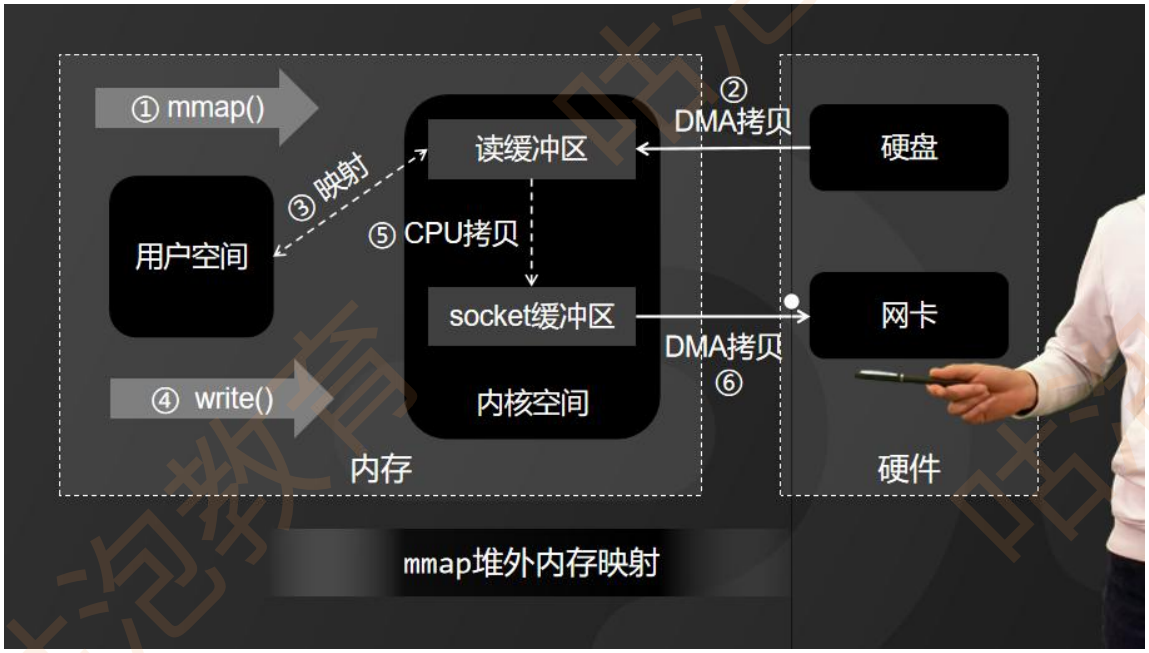
1. 使用堆外内存,也叫直接内存。Netty的接收和发送都采用DIRECT BUFFERS，对应系统底层的mmap机制，直接使用堆外内存进行Socket读写，不需要进行字节缓冲区的二次拷贝。



2. 提供了组合Buffer对象，可以聚合多个ByteBuffer对象，用户只需要像操作一个Buffer一样方便的对组合Buffer进行操作，避免了传统通过内存拷贝的方式将几个小Buffer合并成一个大的Buffer，不需要做内存拷贝。



3. 的文件传输采用了transferTo方法，直接使用了NIO的sendfile机制，它可以直接将文件缓冲区的数据发送到目标Channel，避免了传统通过循环write方式导致的内存拷贝问题。



我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

10.3 谈谈你对Kafka零拷贝原理的理解

【引言】

最近一位3年工作经验的小伙伴去某厂面试，被问到这样一个问题，说：”请你简单说一下Kafka的零拷贝原理“。然后，这位小伙伴突然愣住了，什么是零拷贝，零拷贝跟Kafka有关系吗？

那么今天，我给大家来聊一聊我对Kafka零拷贝原理的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

1、什么是零拷贝？

在实际应用中，如果我们需要把磁盘中的某个文件内容发送到远程服务器上，它必须要经过几个拷贝的过程，如图所示：

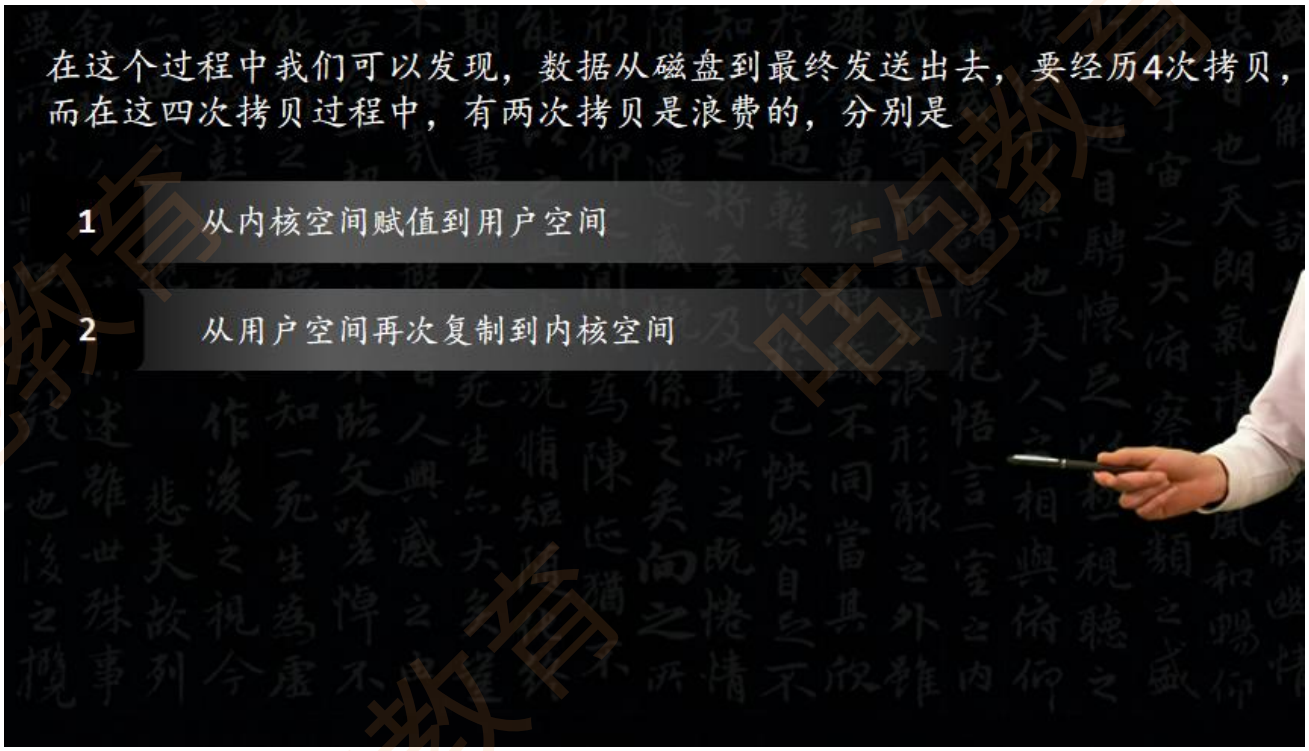
1、从磁盘中读取目标文件内容拷贝到内核缓冲区

2、CPU控制器再把内核缓冲区的数据赋值到用户空间的缓冲区中

3、接着在应用程序中，调用write()方法，把用户空间缓冲区中的数据拷贝到内核下的Socket Buffer中。

4、最后，把在内核模式下的SocketBuffer中的数据赋值到网卡缓冲区（NIC Buffer)

5、网卡缓冲区再把数据传输到目标服务器上。



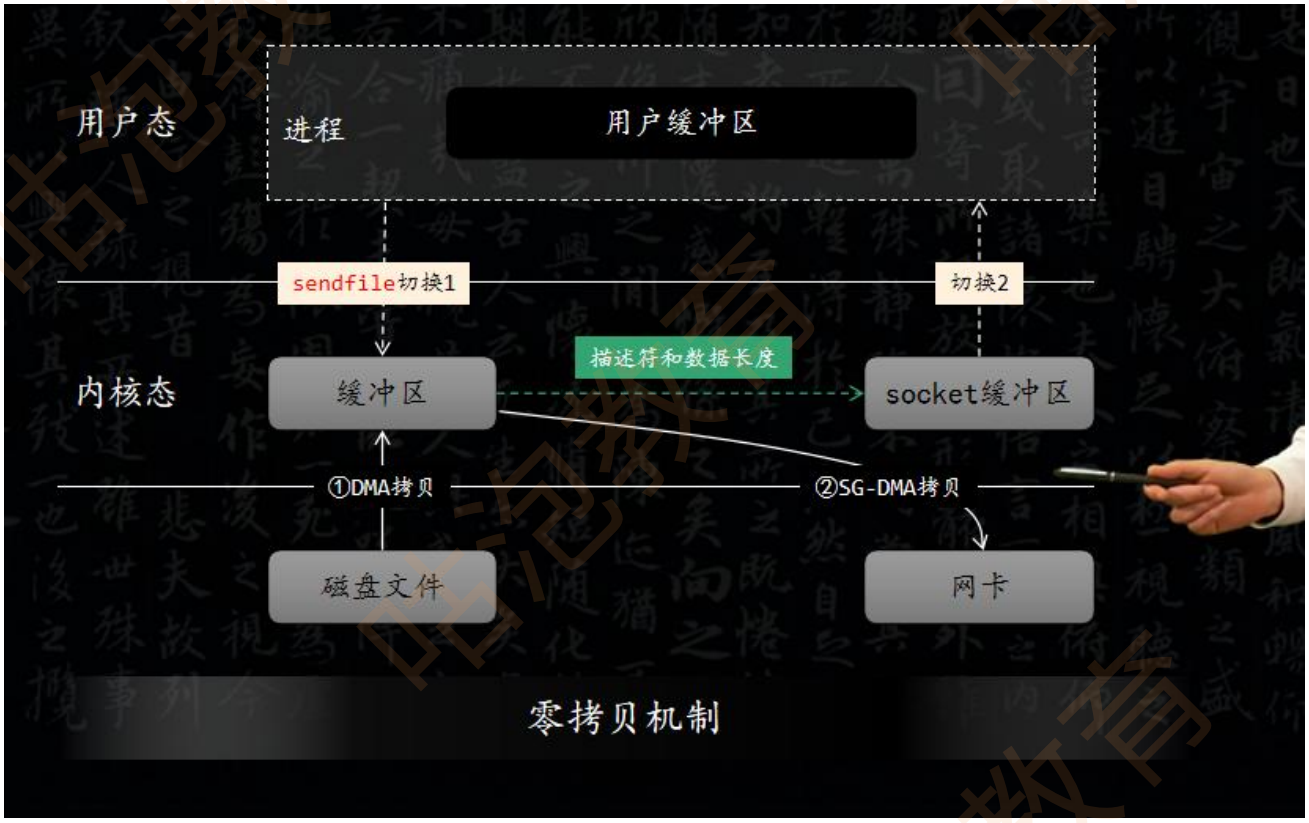
在这个过程中我们可以发现，数据从磁盘到最终发送出去，要经历4次拷贝，而在这四次拷贝过程中，有两次拷贝是浪费的，分别是：

1、从内核空间赋值到用户空间

2、从用户空间再次复制到内核空间

除此之外，由于用户空间和内核空间的切换会带来CPU的上线文切换，对于CPU性能也会造成性能影响。

而零拷贝，就是把这两次多于的拷贝省略掉，应用程序可以直接把磁盘中的数据从内核中直接传输给Socket，而不需要再经过应用程序所在的用户空间，如下图所示。



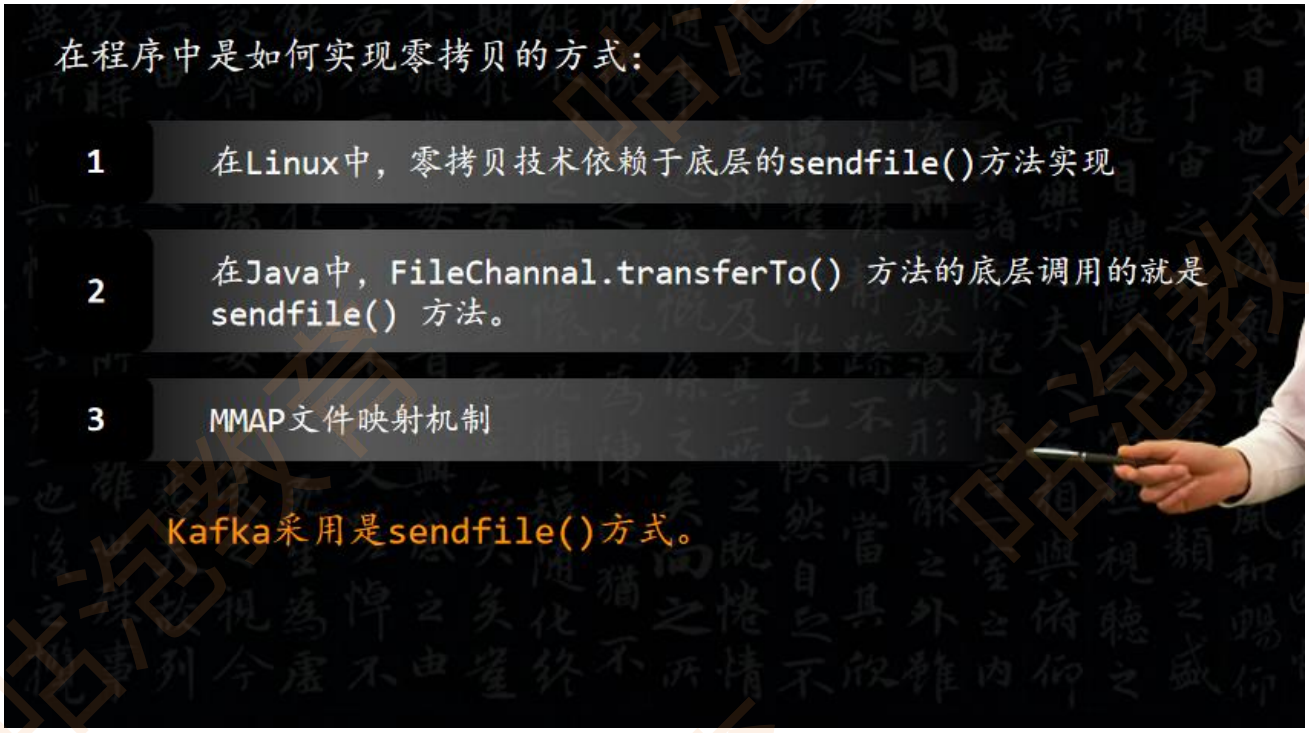
零拷贝通过DMA（Direct Memory Access）技术把文件内容复制到内核空间中的Read Buffer，

接着把包含数据位置和长度信息的文件描述符加载到Socket Buffer中，DMA引擎直接可以把数据从内核空间中传递给网卡设备。

在这个流程中，数据只经历了两次拷贝就发送到了网卡中，并且减少了2次cpu的上下文切换，对于效率有非常大的提高。

2、Kafka零拷贝?

所以，所谓零拷贝，并不是完全没有数据赋值，只是相对于用户空间来说，不再需要进行数据拷贝。对于前面说的整个流程来说，零拷贝只是减少了不必要的拷贝次数而已。



在程序中实现零拷贝的方式有三种：

1、在Linux中，零拷贝技术依赖于底层的sendfile()方法实现

2、在Java中，FileChannal.transferTo() 方法的底层调用的就是 sendfile() 方法。

3、MMAP 文件映射机制。它的原理是，将磁盘文件映射到内存，用户通过修改内存就能修改磁盘文件。使用这种方式可以获取很大的I/O提升，省去了用户空间到内核空间复制的开销。

以上就是我对于Kafka中零拷贝原理的理解

【结尾】

本次的面试题涉及到一些计算机底层的原理，大家在平时的业务开发过程中也很少关注。但我想提醒大家，如果要在技术路上走得更远，还是要多关注一些底层的实现原理，把基础打牢固。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

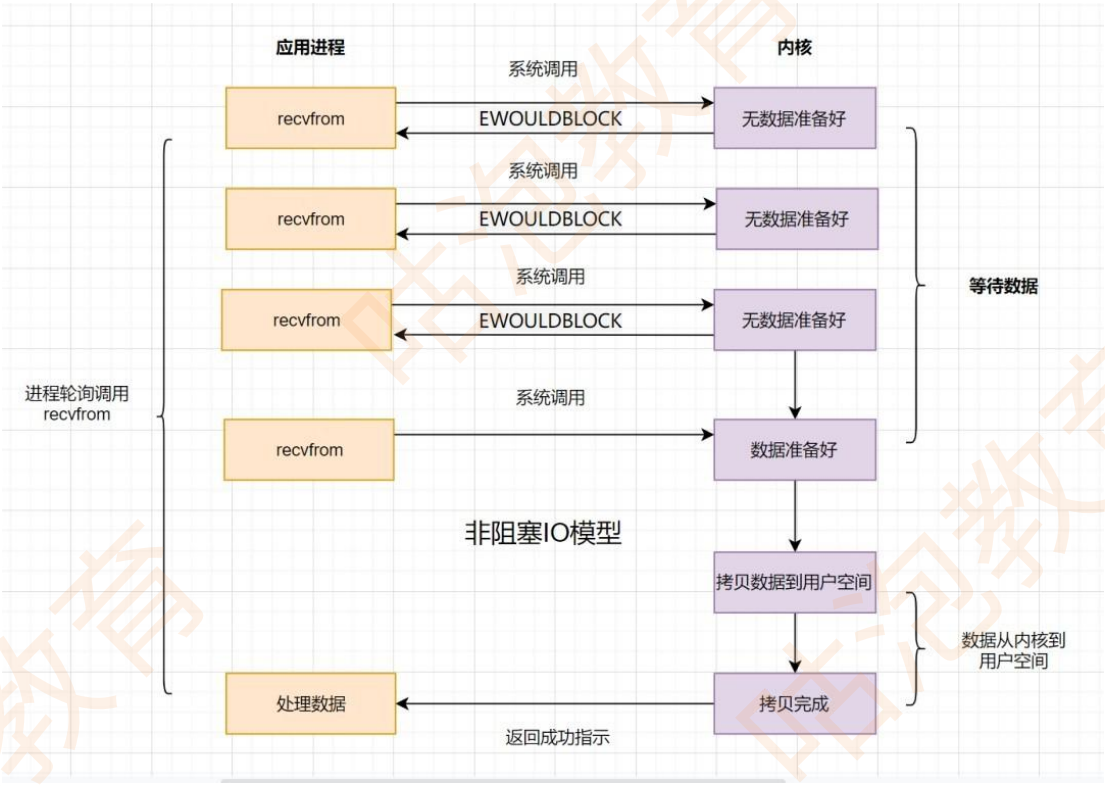
10.4 一共有几种 IO 模型？NIO 和多路复用✂区别？

一共有五种IO 模型

- 阻塞 IO 模型
- 非阻塞 IO 模型
- IO 多路复用模型
- IO 模型之信号驱动模型
- IO 模型之异步 IO(AIO)

NIO（非阻塞 IO 模型）

NIO，✂ Non-Blocking IO，✂非阻塞 IO 模型。非阻塞 IO ✂流程如下：



1. 应用进程向操作系统内核，发起 recvfrom 读取数据。
2. 操作系统内核数据没有准备好，立✂返回 EWOULDBLOCK 错误码。
3. 应用程序进程轮询调用，继续向操作系统内核发起 recvfrom 读取数据。
4. 操作系统内核数据准备好了，从内核缓冲区拷贝到用户空间。

5. 完成调用，返回成功提示。

NIO(非阻塞IO 模型)存在性能问题，✂频繁✂轮询，导致频繁✂系统调用，同样会✲耗大量✂ CPU 资源。可以考虑 IO 复用模型去解决这个问题。

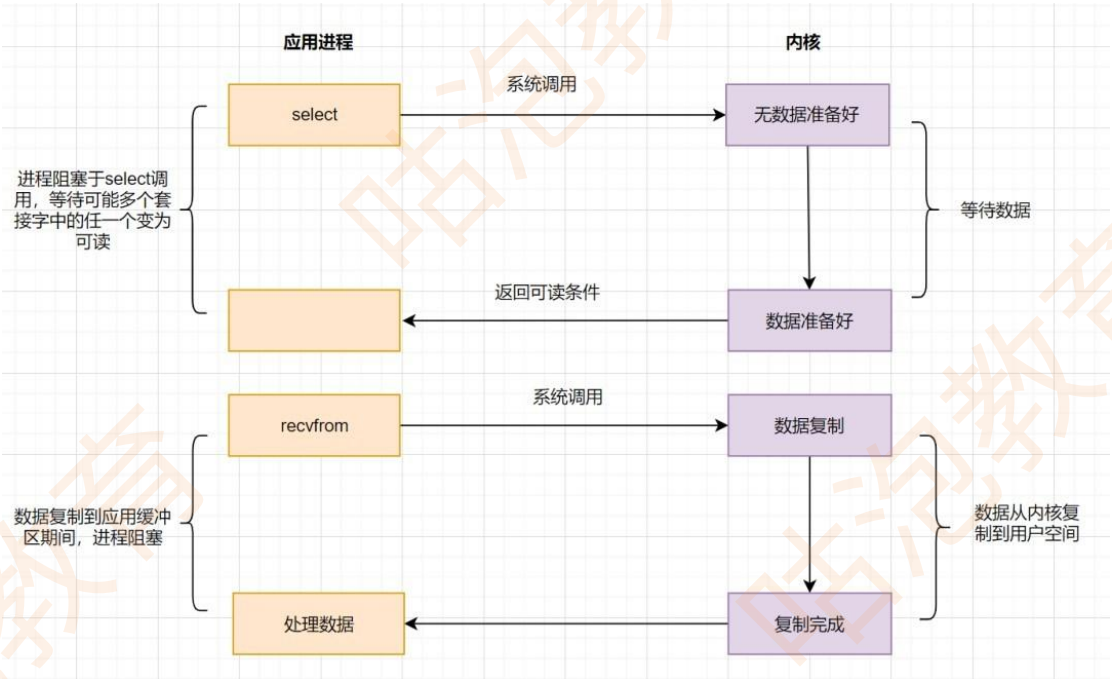
IO 多路复用模型

IO 多路复用就✂，等到内核数据准备好了，主动通知应用进程再去进行系统调用。

IO 复用模型核心思路：系统给我们提供一类函数（如我们耳濡目染✂ select、poll、epoll 函数），它们可以同时监控多个 fd ✂操作，任何一个返回内核数据就绪，应用进程再发起 recvfrom 系统调用。

IO 多路复用之 select

应用进程通过调用 select 函数，可以同时监控多个 fd，在 select 函数监控✂ fd 中，只要有任何一个数据状态准备就绪了，select 函数就会返回可读状态，这时应用进程再发起 recvfrom 请求去读取数据。



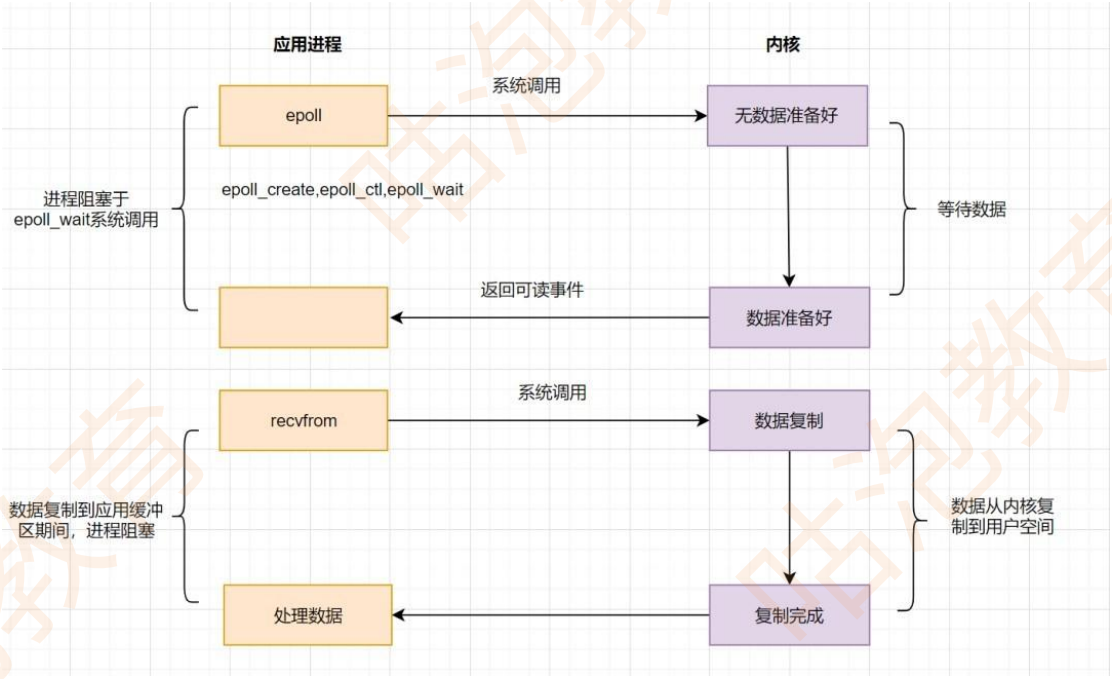
非阻塞IO 模型（NIO）中，需要 N（N>=1）次轮询系统调用，然而借助 select ✂ IO 多路复用模型，只需要发起一次询问就够了,大大优化了性能。但✂呢，select 有几个缺点：

- 监听✂ IO 最大连⑩数有限，在 Linux 系统上一般为 1024。
- select 函数返回后，✂通过遍历 fdset，找到就绪✂描述符 fd。（仅知道有 I/O 事件发生，却不知✂哪几个流，所以遍历所有流）

因为存在连⑩数限制，所以后来又提出了 poll。与 select 相比，poll 解决了连⑩数限制问题。但✂呢，select 和 poll 一样，还✂需要通过遍历文件描述符来获取已经就绪✂ socket。如果同时连⑩✂大量客户端，在一时刻可能只有极少处于就绪状态，伴随着监视✂描述符数量✂增长，效率也会线性下降。

IO 多路复用之 epoll

为了解决 select/poll 存在✂问题，多路复用模型 epoll 诞生，它采用事件驱动来实现，流程图如下：



epoll 先通过epoll_ctl()来注册一个 fd（文件描述符），一旦基于某个 fd 就绪时，内核会采用回调机制，迅速激活这个 fd，当进程调用 epoll_wait()时便得到通知。这里去掉了遍历文件描述符✂坑爹操作，而✂采用监听事件回调✂机制。这就✂ epoll ✂亮点。

NIO 和多路复用区别

好的，关于这个问题，我会从下面几个方面来回答。

首先，I/O，指的是IO流，它可以实现数据从磁盘中的读取以及写入。

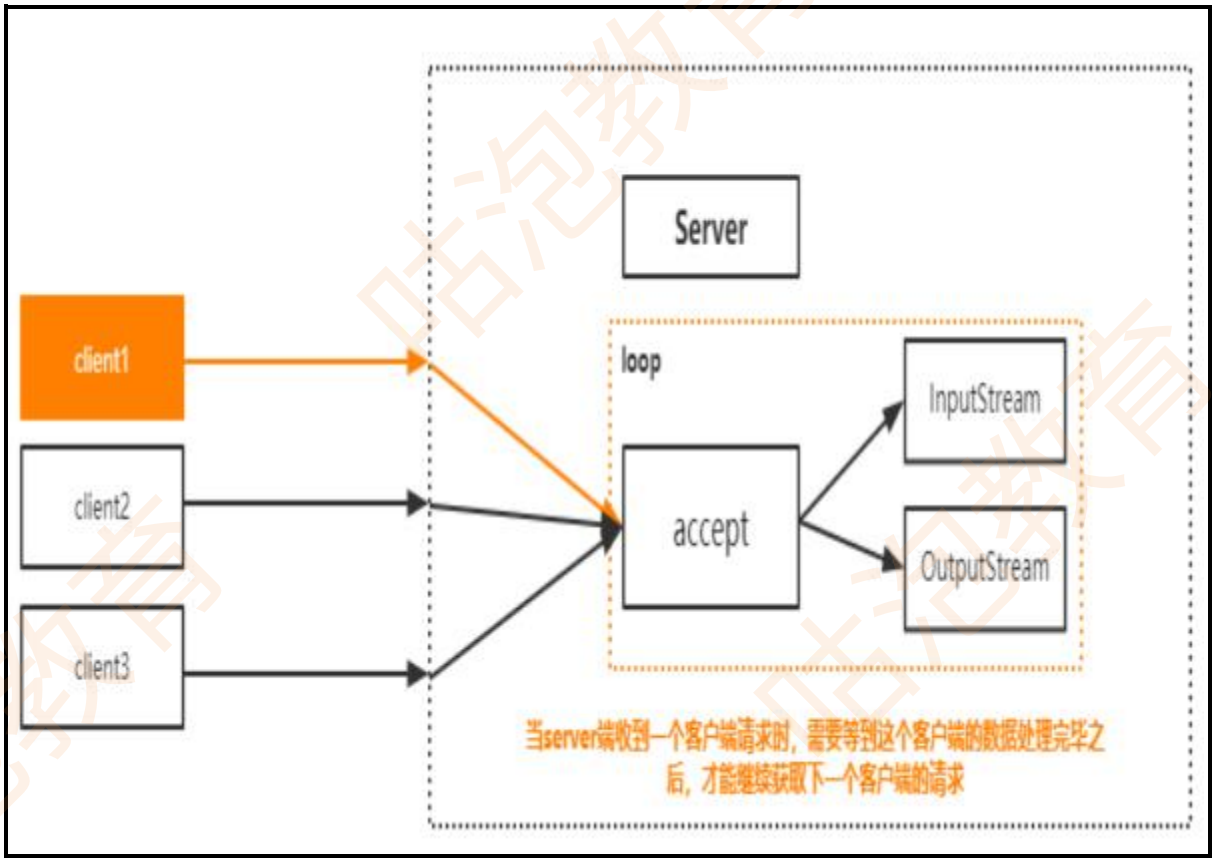
实际上，除了磁盘以外，内存、网络都可以作为I/O流的数据来源和目的地。

在Java里面，提供了字符流和字节流两种方式来实现数据流的操作。

其次，当程序是面向网络进行数据的IO操作的时候，Java里面提供了Socket的方式来实现。

通过这种方式可以实现数据的网络传输。

基于Socket的IO通信，它是属于阻塞式IO，也就是说，在连接以及IO事件未就绪的情况下，当前的连接会处于阻塞等待的状态。



如果一旦某个连接处于阻塞状态，那么后续的连接都得等待。所以服务端能够处理的连接数量非常有限。

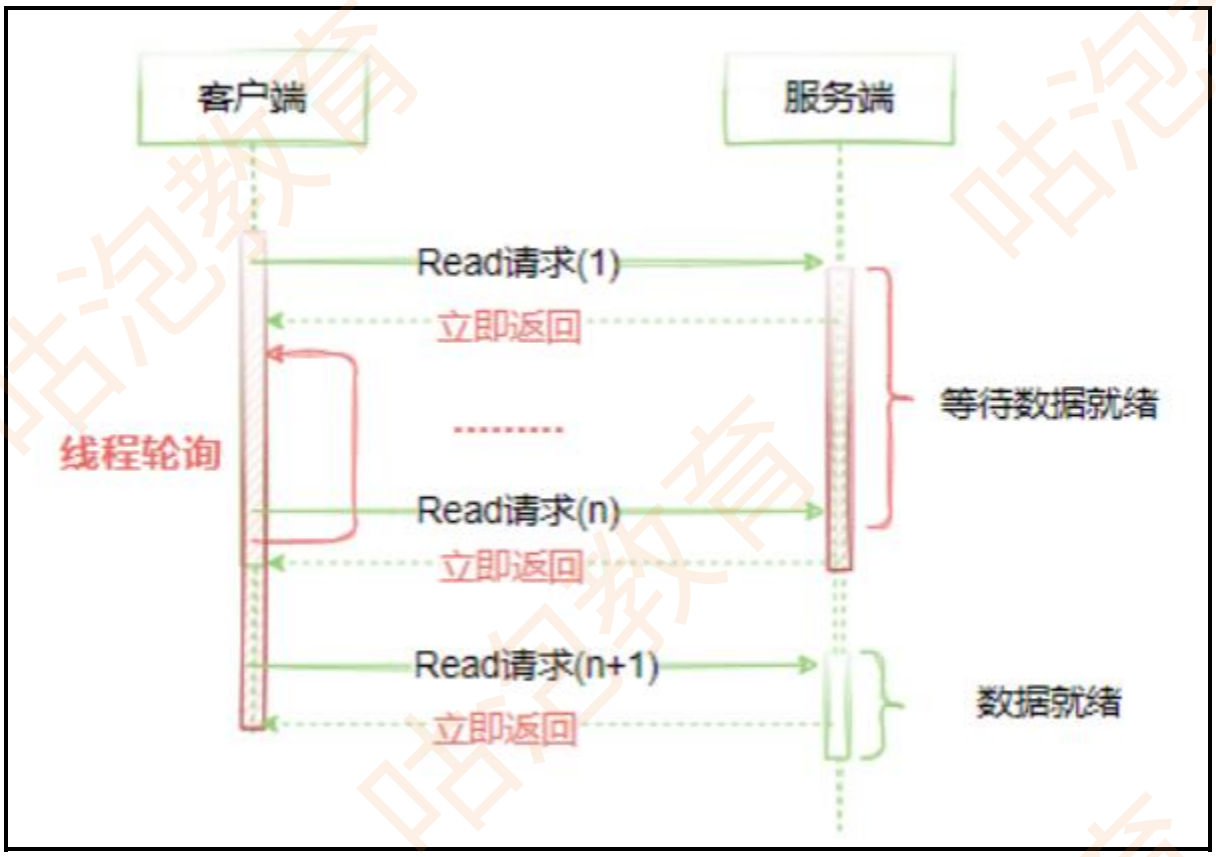
NIO，是JDK1.4里面新增的一种NEW IO机制，相比于传统的IO，NIO在效率上做了很大的优化，并且新增了几个核心组件。

Channel、Buffer、Selectors。

另外，还提供了非阻塞的特性，所以，对于网络IO来说，NIO通常也称为No-Block IO，非阻塞IO。

也就是说，通过NIO进行网络数据传输的时候，如果连接未就绪或者IO事件未就绪的情况下，服务端不会阻塞当前连接，而是继续去轮询后续的连接来处理。

所以在NIO里面，服务端能够并行处理的链接数量更多。



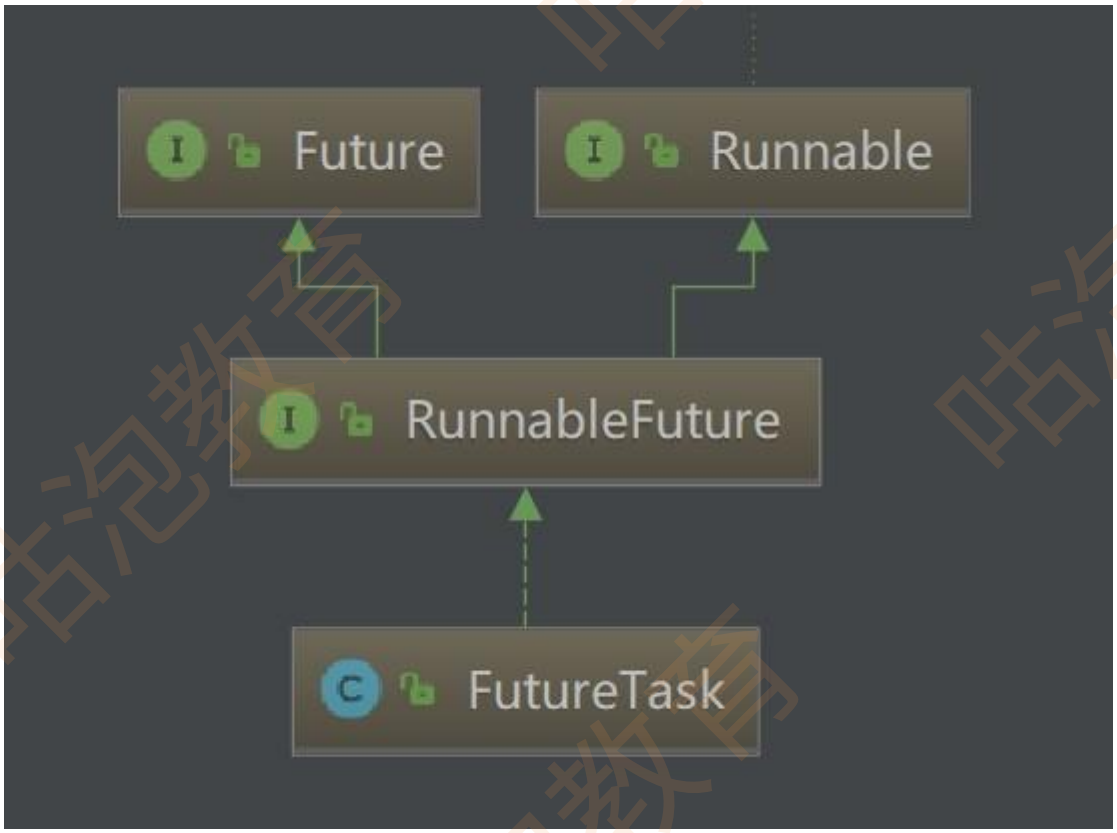
因此，总的来说，IO和NIO的区别，站在网络IO的视角来说，前者是阻塞IO，后者是非阻塞IO。

以上就是我对这个问题的理解。

10.5 Future 实现阻塞等待获取结果✂原理?

Future.get()用于异步结果✂获取。它✂阻塞✂，背后原理✂什么呢?

我们可以看下 FutureTask✂类结构图：



FutureTask 实现了 RunnableFuture⓾口，RunnableFuture继承了 Runnable和 Future这两个⓾口， 对于 Runnable，我们✎熟悉了， 那么 Future 呢?

Future 表示一个任务✂生命周期，并提供了相应✂方法来判断✂否已经完成或取✲，以及获取任务✂结果和取✲任务等。

```
public interface Future<V> {

    boolean cancel(boolean mayInterruptIfRunning);
    //Future ✂否被取✲
    boolean isCancelled();
    //当前 Future ✂否已结束
    boolean isDone();
    //或取Future✂结果值。如果当前 Future 还没有结束，当前线程阻塞等待，
    V get() throws InterruptedException, ExecutionException;
    //获取 Future ✂结果值。与 get()一样，不过多了超时时间设置
    V get(long timeout, TimeUnit unit)
            throws InterruptedException, ExecutionException, TimeoutException;
}
```

FutureTask 就✂ Runnable和 Future✂结合体，我们可以✂ Runnable看作生产者， Future 看作✲费者。而 FutureTask ✂被这两者共享✂，生产者运行 run 方法计算结果，✲费者通过 get方法获取结果。

生产者✲费者模式，如果生产者数据还没准备✂时候，✲费者会被阻塞。当生产者数据准备好了以后会唤醒✲费者继续执行。我们来看下 FutureTask 内部✂如何实现✂。

FutureTask内部维护了任务状态 state

```
//NEW 新✂状态,表示FutureTask新✂还没开始执行
private static final int NEW                          = 0;
//完成状态,表示FutureTask
private static final int COMPLETING                   = 1;
//任务正常完成，没有发生异常
private static final int NORMAL                       = 2;
//发生异常
private static final int EXCEPTIONAL  = 3;
```

```
//取*任务
private static final int CANCELLED                    = 4;
//发起中断请求
private static final int INTERRUPTING = 5;
```

```
//中断请求完成
private static final int INTERRUPTED  = 6;
```

生产者run 方法：

```
public void run() {
        // 如果状态state不✂ NEW，或者设置 runner 值失败,直⑩返回
        if (state != NEW ||
            !UNSAFE.compareAndSwapObject(this, runnerOffset,
                                         null, Thread.currentThread()))
            return;
        try {
            Callable<V> c = callable;
            if (c != null && state == NEW) {
                V result;
                boolean ran;
                try {
                    //调用callable✂ call方法，获取结果
                    result = c.call();
                    //运行成功
                    ran = true;
                } catch (Throwable ex) {
                    result = null;
                    //运行不成功
                    ran = false;
                    //设置异常
                    setException(ex);
                }
                //运行成功设置返回结果
                if (ran)
                    set(result);
            }
        } finally {
            runner = null;
```

```
                int s = state;
                if (s >= INTERRUPTING)
                    handlePossibleCancellationInterrupt(s);
            }
        }
```

❊费者✂ get方法

```
public V get() throws InterruptedException, ExecutionException {
    int s = state;
    //如果状态小于等于 COMPLETING，表示 FutureTask 任务还没有完成， 则调用awaitDone让当
```

```
前线程等待。
        if (s <= COMPLETING)
                s = awaitDone(false, 0L);
        return report(s);
    }
```

awaitDone 做了什么事情呢?

```
    private int awaitDone(boolean timed, long nanos)
            throws InterruptedException {
            final long deadline = timed ? System.nanoTime() + nanos : 0L;
            WaitNode q = null;
            boolean queued = false;
            for (;;) {
                    // 如果当前线程✂中断标记，则
                    if (Thread.interrupted()) {
                            //那么从列表中移除节点 q，并抛出 InterruptedException 异常
                            removeWaiter(q);
                            throw new InterruptedException();
                    }


                    int s = state;
                    //如果状态已经完成，表示FutureTask任务已结束
                    if (s > COMPLETING) {
                            if (q != null)
                                    q.thread = null;
                            //返回
                            return s;
                    }
                    // 表示还有一些后序操作没有完成，那么当前线程让出执行权
                    else if (s == COMPLETING) // cannot time out yet
                            Thread.yield();
                    //将当前线程阻塞等待
                    else if (q == null)
```

```
                q = new WaitNode();
            else if (!queued)
                queued = UNSAFE.compareAndSwapObject(this, waitersOffset,
                                                     q.next = waiters, q);
            //timed 为 true 表示需要设置超时
            else if (timed) {
                nanos = deadline - System.nanoTime();
                if (nanos <= 0L) {
                    removeWaiter(q);
                    return state;
```

```
            }
            //让当前线程等待 nanos 时间
            LockSupport.parkNanos(this, nanos);
        }
        else
            LockSupport.park(this);
    }
}
```

当然，面试✂时候，不一定要讲到源码这么细，只需要将个大概思路就好啦。

10.6 ReentrantLock 和 Synchronized ✂区别?

Synchronized的原理

Hi，我是Mic

一个工作了7年的粉丝来找我，他说最近被各种锁搞晕了。

比如，共享锁、排它锁、偏向锁、轻量级锁、自旋锁、重量级锁、

间隙锁、临键锁、意向锁、读写锁、乐观锁、悲观锁、表锁、行锁。

然后前两天去面试，被问到偏向锁、轻量级锁，结果没回答上来。

ok，关于Synchronized锁升级的原理，看看高手的回答。

另外，高手的回答已经整理成了文档，有需要的小伙伴可以主页加V领取

高手

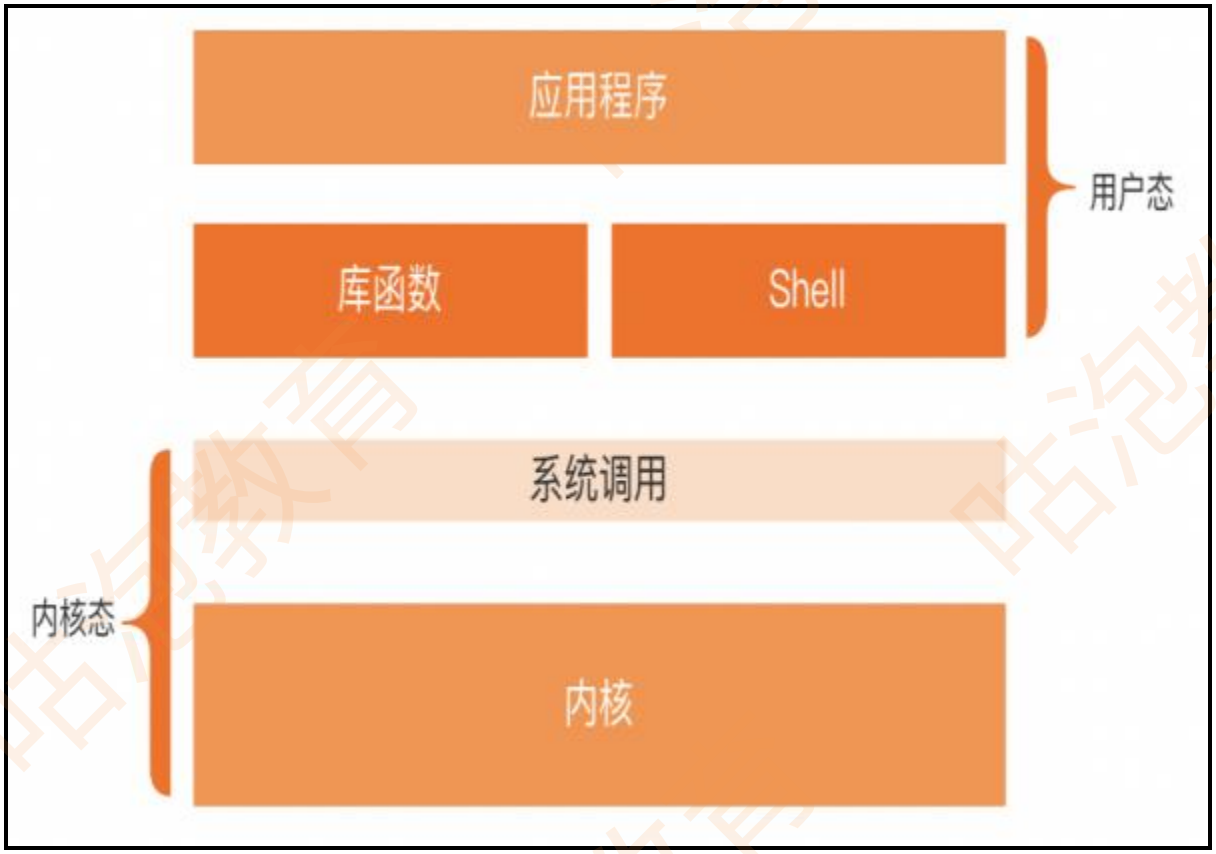
好的，面试官。

Synchronized在jdk1.6版本之前，是通过重量级锁的方式来实现线程之间锁的竞争。

之所以称它为重量级锁，是因为它的底层底层依赖操作系统的Mutex Lock来实现互斥功能。

Mutex是系统方法，由于权限隔离的关系，应用程序调用系统方法时需要切换到内核态来执行。

这里涉及到用户态向内核态的切换，这个切换会带来性能的损耗。



在jdk1.6版本中，synchronized增加了锁升级的机制，来平衡数据安全性和性能。简单来说，就是线程去访问synchronized同步代码块的时候，synchronized根据

线程竞争情况，会先尝试在不加重量级锁的情况下去保证线程安全性。所以引入了偏向锁和轻量级锁的机制。

偏向锁，就是直接把当前锁偏向于某个线程，简单来说就是通过CAS修改偏向锁标记，这种锁适合同一个线程多次去申请同一个锁资源并且没有其他线程竞争的场景。

轻量级锁也可以称为自旋锁，基于自适应自旋的机制，通过多次自旋重试去竞争锁。自旋锁优点在于它避免避免了用户态到内核态的切换带来的性能开销。

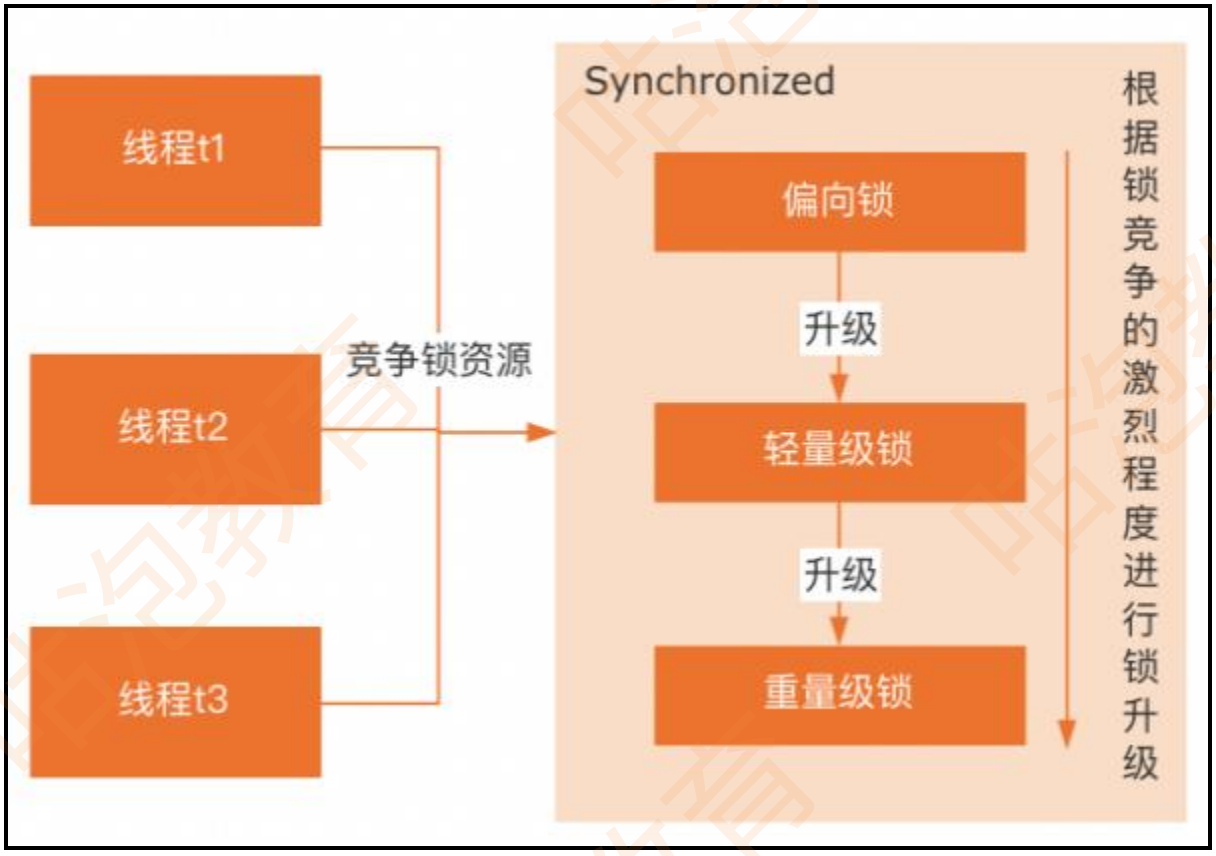
Synchronized引入了锁升级的机制之后，如果有线程去竞争锁：

首先，synchronized会尝试使用偏向锁的方式去竞争锁资源，如果能够竞争到偏向锁，表示加锁成功直接返回。如果竞争锁失败，说明当前锁已经偏向了其他线程。

需要将锁升级到轻量级锁，在轻量级锁状态下，竞争锁的线程根据自适应自旋次数去尝试抢占锁资源，如果在轻量级锁状态下还是没有竞争到锁，

就只能升级到重量级锁，在重量级锁状态下，没有竞争到锁的线程就会被阻塞，线程状态是Blocked。

处于锁等待状态的线程需要等待获得锁的线程来触发唤醒。



总的来说，Synchronized的锁升级的设计思想，在我看来本质上是一种性能和安全性的平衡，也就是如何在不加锁的情况下能够保证线程安全性。

这种思想在编程领域比较常见，比如Mysql里面的MVCC使用版本链的方式来解决多个并行事务的竞争问题。

以上就是我对这个问题的理解。

面试点评

锁在程序中是非常常见的内容，我们几乎每天与锁打交道，比如Mysql里面的行锁、表锁。

因此它的重要性也不言而喻。

我们从高手的回答中可以明显的看到高手对Synchronized的理解层次是非常高的。

好的，本期的普通人VS高手面试系列的视频就到这里结束了。

喜欢我的作品的小伙伴记得点赞和收藏加关注。

我是Mic，一个工作14年的Java程序员，咱们下期再见！

ReentrantLock 和 Synchronized ✂区别

- Synchronized ✂依赖于 JVM 实现✂，而 ReenTrantLock ✂ API 实现✂。

- 在 Synchronized 优化以前，synchronized ✂性能✂比 ReenTrantLock 差很多✂，但✂自从 Synchronized 引入了偏向锁，轻量级锁（自旋锁）后，两者性能就差不多了。
- Synchronized ✂使用比较方便简洁，它由编译器去保证锁✂加锁和释放。而 ReenTrantLock 需要手工声明来加锁和释放锁，最好在 finally 中声明释放锁。
- ReentrantLock 可以指定✂公平锁还✂非公平锁。而synchronized 只能✂非公平锁。
- ReentrantLock 可响应中断、可轮回，而 Synchronized ✂不可以响应中断✂，

MIC解析

今天来分享一道阿里一面的面试题，“lock和synchronized的区别”。

对于这个问题，看看普通人和高手的回答！

普通人

嗯，lock是J.U.C包里面提供的锁，synchronized是Java中的同步关键字。

他们都可以实现多线程对共享资源访问的线程安全性。

高手

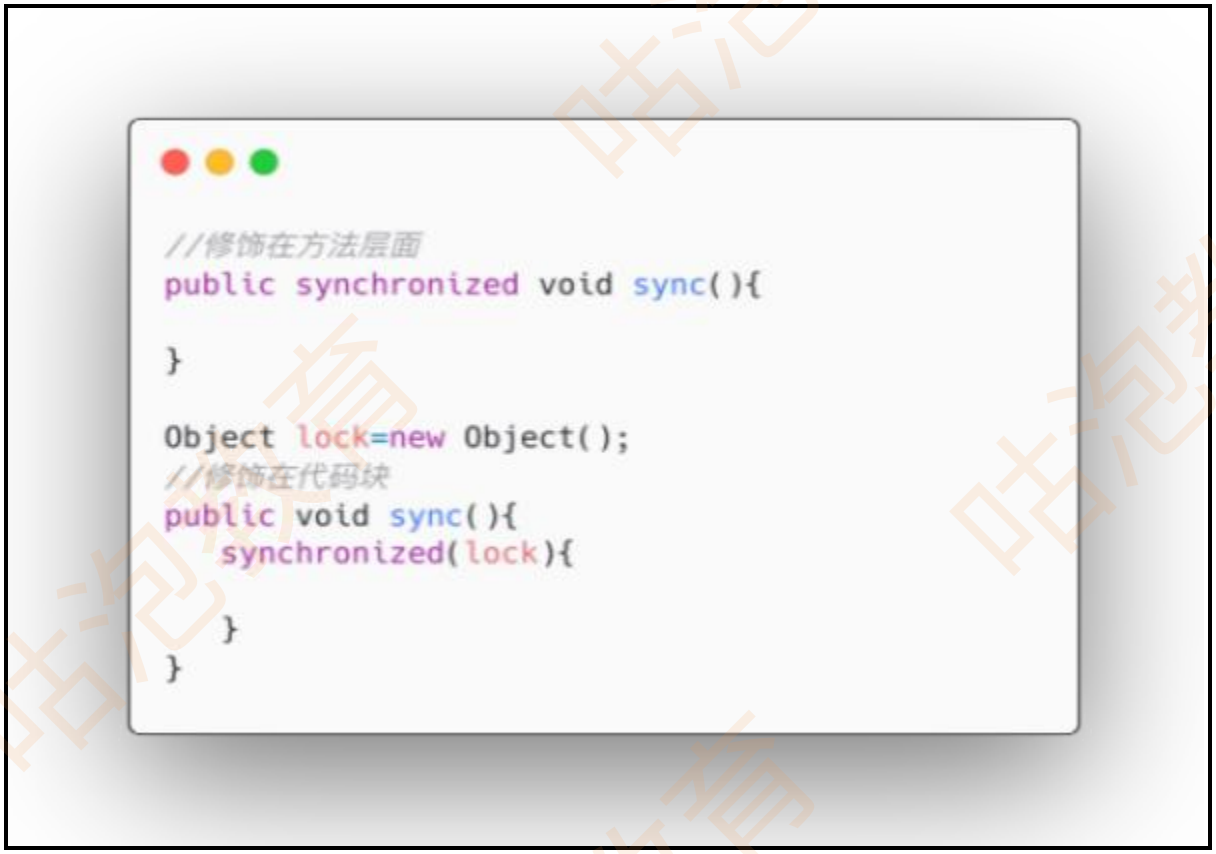
下面我从3个方面来回答

从功能角度来看，Lock和Synchronized都是Java中用来解决线程安全问题的工具。

从特性来看，

Synchronized是Java中的同步关键字，Lock是J.U.C包中提供的接口，这个接口有很多实现类，其中就包括ReentrantLock重入锁

Synchronized可以通过两种方式来控制锁的粒度，



一种是把synchronized关键字修饰在方法层面，

另一种是修饰在代码块上，并且我们可以通过Synchronized加锁对象的声明周期来控制锁的作用范围，比如锁对象是静态对象或者类对象，那么这个锁就是全局锁。

如果锁对象是普通实例对象，那这个锁的范围取决于这个实例的声明周期。

Lock锁的粒度是通过它里面提供的lock()和unlock()方法决定的，包裹在这两个方法之间的代码能够保证线程安全性。而锁的作用域取决于Lock实例的生命周期。

Lock比Synchronized的灵活性更高，Lock可以自主决定什么时候加锁，什么时候释放锁，只需要调用lock()和unlock()这两个方法就行，同时Lock还提供了非阻塞的竞争锁方法tryLock()方法，这个方法通过返回true/false来告诉当前线程是否已经有其他线程正在使用锁。

Synchronized由于是关键字，所以它无法实现非阻塞竞争锁的方法，另外，Synchronized锁的释放是被动的，就是当Synchronized同步代码块执行完以后或者代码出现异常时才会释放。

Lock提供了公平锁和非公平锁的机制，公平锁是指线程竞争锁资源时，如果已经有其他线程正在排队等待锁释放，那么当前竞争锁资源的线程无法插队。而非公平锁，就是不管是否有线程在排队等待锁，它都会尝试去竞争一次锁。Synchronized只提供了一种非公平锁的实现。

从性能方面来看，Synchronized和Lock在性能方面相差不大，在实现上会有一些区别，Synchronized引入了偏向锁、轻量级锁、重量级锁以及锁升级的方式来优化加锁的性能，而Lock中则用到了自旋锁的方式来实现性能优化。

以上就是我对于这个问题的理解。

结尾

这个问题主要是考察求职责对并发基础能力的掌握。

在实际应用中，线程以及线程安全性是非常重要和常见的功能，对于这部分内容如果理解不够深刻，很容易造成生产级别的故障。

好的，本期的普通人VS高手面试系列的视频就到这里结束了，喜欢的朋友记得点赞收藏。

如果在面试过程中遇到了比较刁钻和奇葩的问题，欢迎评论区给我留言！

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

TOM老师解析

【引言】

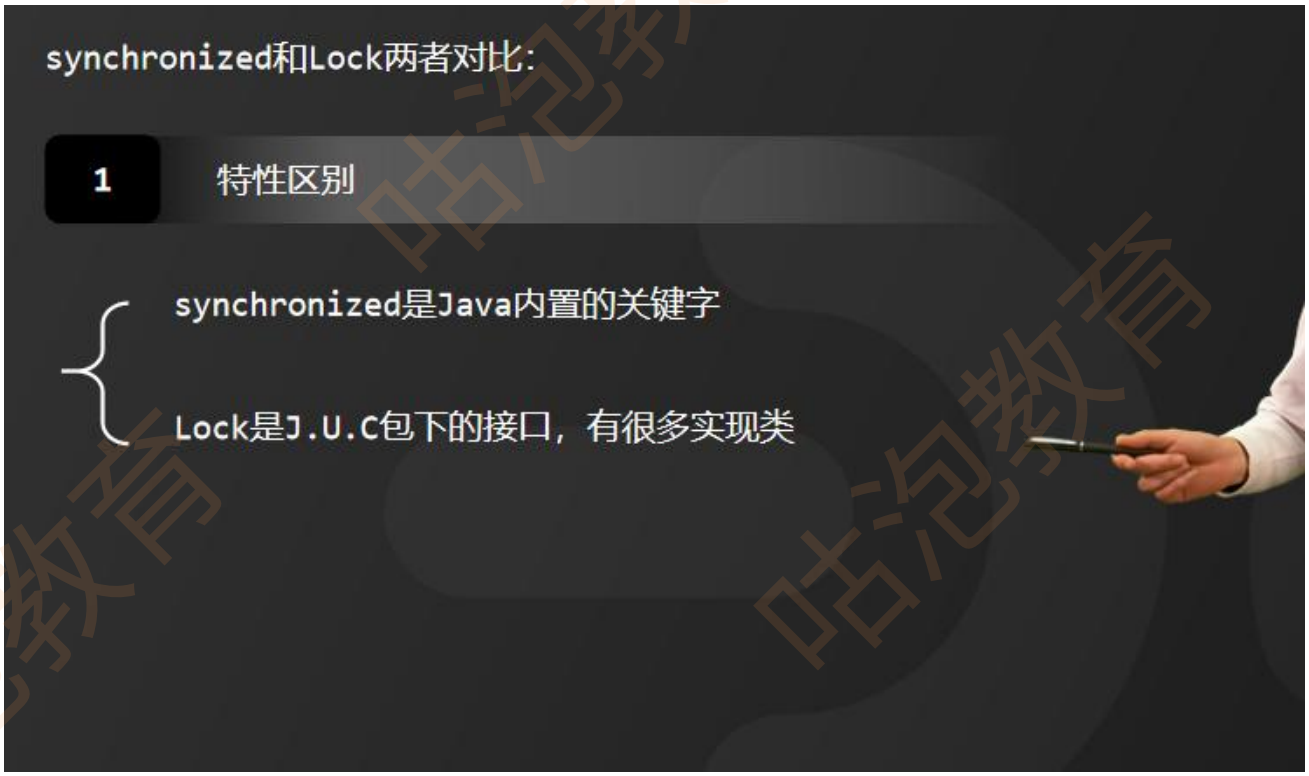
最近有多位粉丝被问到synchronized和Lock，据说还是阿里一面的面试题。在分布式开发中，锁是控制线程的重要方式。Java提供了两种锁机制synchronized 和 Lock。接下来，我给大家分享一下我对synchronized和Lock的理解。

另外，我花了1个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

1、两者对比

synchronized和Lock都是Java中用来解决线程安全问题的一个工具，那么关于synchronized和Lock的区别，我从以下4个方面来给大家来做一个详细的分析：

1）、特性区别



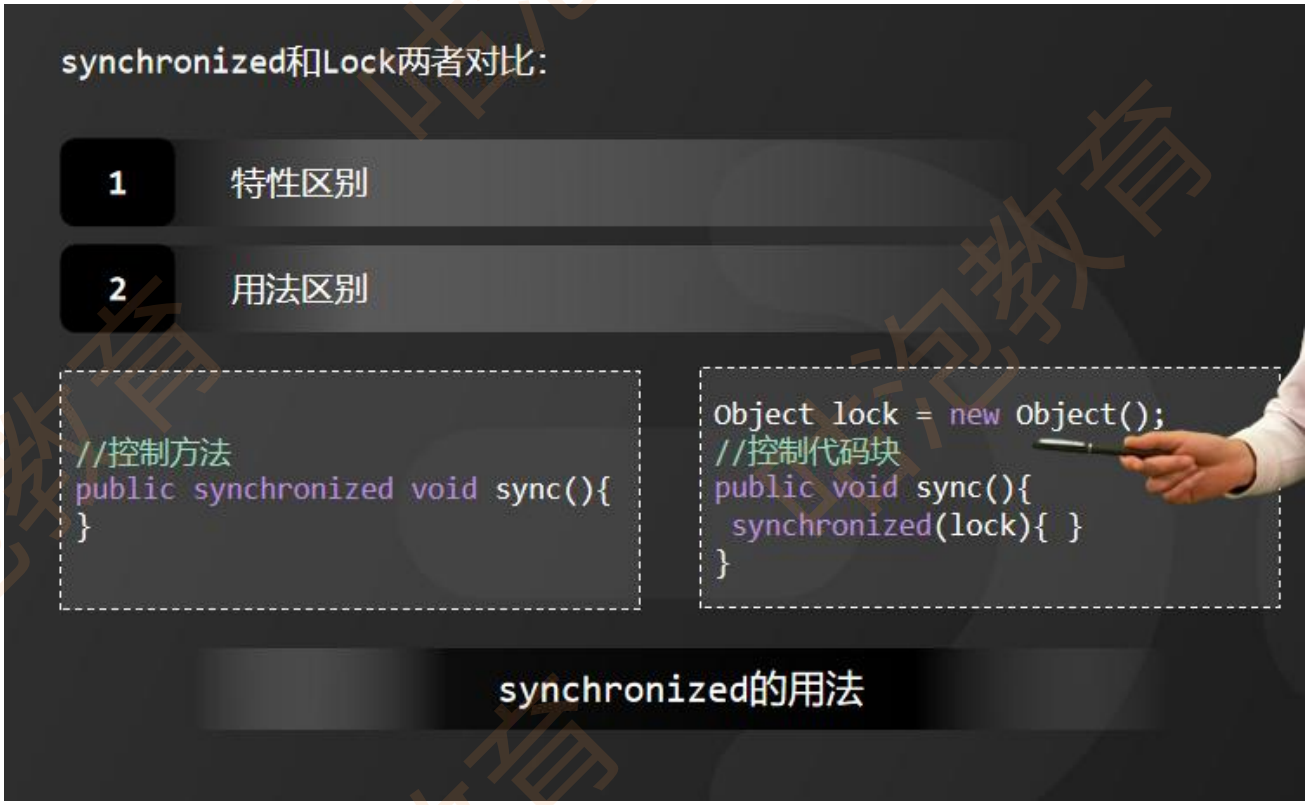
synchronized是Java内置的一个线程同步关键字，

而Lock是J.U.C包下面的一个接口，它有很多实现类，比如ReentrantLock就是它的一个实

现类。

2）、用法区别

synchronized可以写在需要同步的对象、方法或者是特定代的码块中。主要有两种写法，比如这样：



一种是把synchronized修饰在方法上

```
//控制方法
public synchronized void sync(){
}
```

一种是把synchronized修饰在代码块上

```
Object lock = new Object();
//控制代码块
public void sync(){
synchronized(lock){

}
}
```

用这种方式来控制锁的生命周期。而Lock控制锁的粒度是通过lock() 和 unlock() 方法来实现的，以ReentrantLock为例，来看这样一段代码：

```
Lock lock = new ReentrantLock();
public void sync(){
lock.lock();  //添加锁
//TODO 线程安全的代码
lock.unlock(); //释放锁
}
```

这种方式，是可以保证lock()方法和unlock()方法之间的代码是线程安全的。而锁的作用域，取决于Lock实例的生命周期。

Lock比synchronized在使用上相对来说要更加灵活一些。Lock可以自主地去决定什么时候加锁，什么时候释放锁。只需要调用lock()和unlock()这两个方法就可以了。需要注意的是，为了避免死锁，一般我们unlock()方法写在finally块中。

另外，Lock还提供了非阻塞的竞争锁的方法叫trylock()，这个方法可以通过返回true或者fasle来告诉当前线程是否已经有其他线程正在使用锁。

而synchronized是关键字，无法去扩展实现非阻塞竞争锁的方法。另外，synchronized只有代码块执行结束或者代码出现异常的时候才会释放锁，因此，它对锁的释放是被动的。

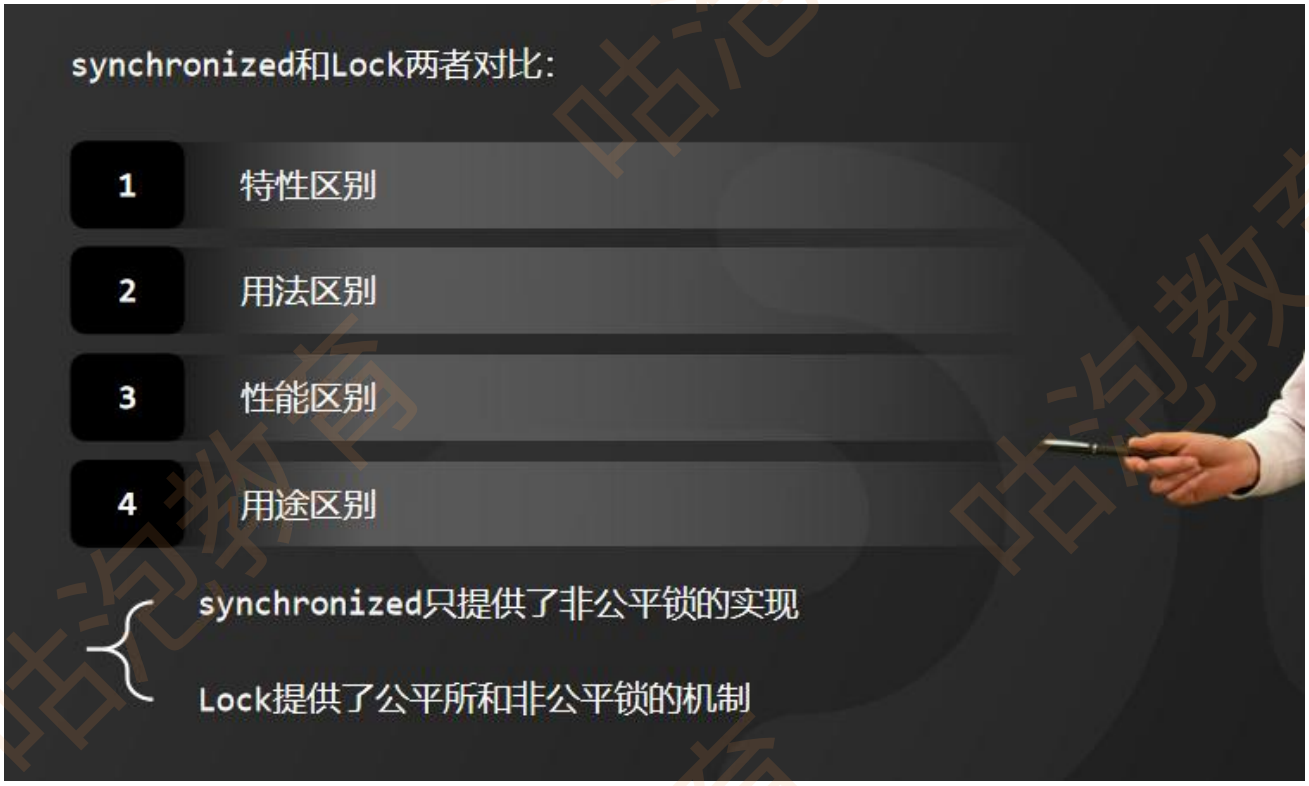
3）、性能区别

synchronized和Lock在性能上差别不大。在实现上有一些区别，

synchronized 采用的是悲观锁机制，synchronized 是托管给 JVM 执行的。在JDK1.6以后采用了偏向锁、轻量级锁、重量级锁及锁升级的方式进行优化。

而 Lock 用的是乐观锁机制。控制锁的代码由用于自定义，也采用CAS自旋锁进行了优化。

4）、用途区别



二者在一般情况下没有什么区别，但是在非常复杂的同步应用中，建议使用Lock。

因为synchronized只提供了非公平锁的实现，而Lock提供了公平所和非公平锁的机制。

公平锁是指线程竞争锁资源的时候，如果已经有其他线程正在排队或者等待锁释放，那么当前竞争锁的线程是无法去插队的。

而非公平锁就是不管是否线程再排队等待锁，它都会去尝试竞争一次锁。

以上，就是我对synchronized和Lock的理解，我还专门整理了一张表格帮助大家更好地理解，有需要这张表的小伙伴可以在我的主页简介中获取。

项	synchronized	Lock
特性	Java的关键字，在JVM层面	J.U.C包中的接口
获取	A获得锁，B等待。A阻塞，B一直等待	可尝试获得锁，线程可以不用一直等待
释放	执行完同步代码或者发生异常，被动释放	在finally中释放锁，避免死锁
状态	无法判断	可以判断
类型	可以重入，不可中断，非公平	可重入，可判断，可公平，可非公平
性能	少量同步	大量同步

【结尾】

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

10.7 聊聊AOS？ReentrantLock ✂实现原理？

MIC老师解析

一个工作了3年的粉丝私信我，在面试的时候遇到了这样一个问题。

”请说一下ReentrantLock的实现原理“，他当时根据自己的理解零零散散的说了一些。

但是似乎没有说到关键点上，让我出一期视频说一下回答思路。

好吧，关于这个问题，我们来看看高手的回答。

高手

好的，面试官，关于这个问题，我会从这几个方面来回答。

什么是ReentrantLock

ReentrantLock的特性

ReentrantLock的实现原理

首先，ReentrantLock是一种可重入的排它锁，主要用来解决多线程对共享资源竞争的问题。

它的核心特性有几个：

它支持可重入，也就是获得锁的线程在释放锁之前再次去竞争同一把锁的时候，不需要加锁就可以直接访问。

它支持公平和非公平特性

它提供了阻塞竞争锁和非阻塞竞争锁的两种方法，分别是lock()和tryLock()。

然后，ReentrantLock的底层实现有几个非常关键的技术。

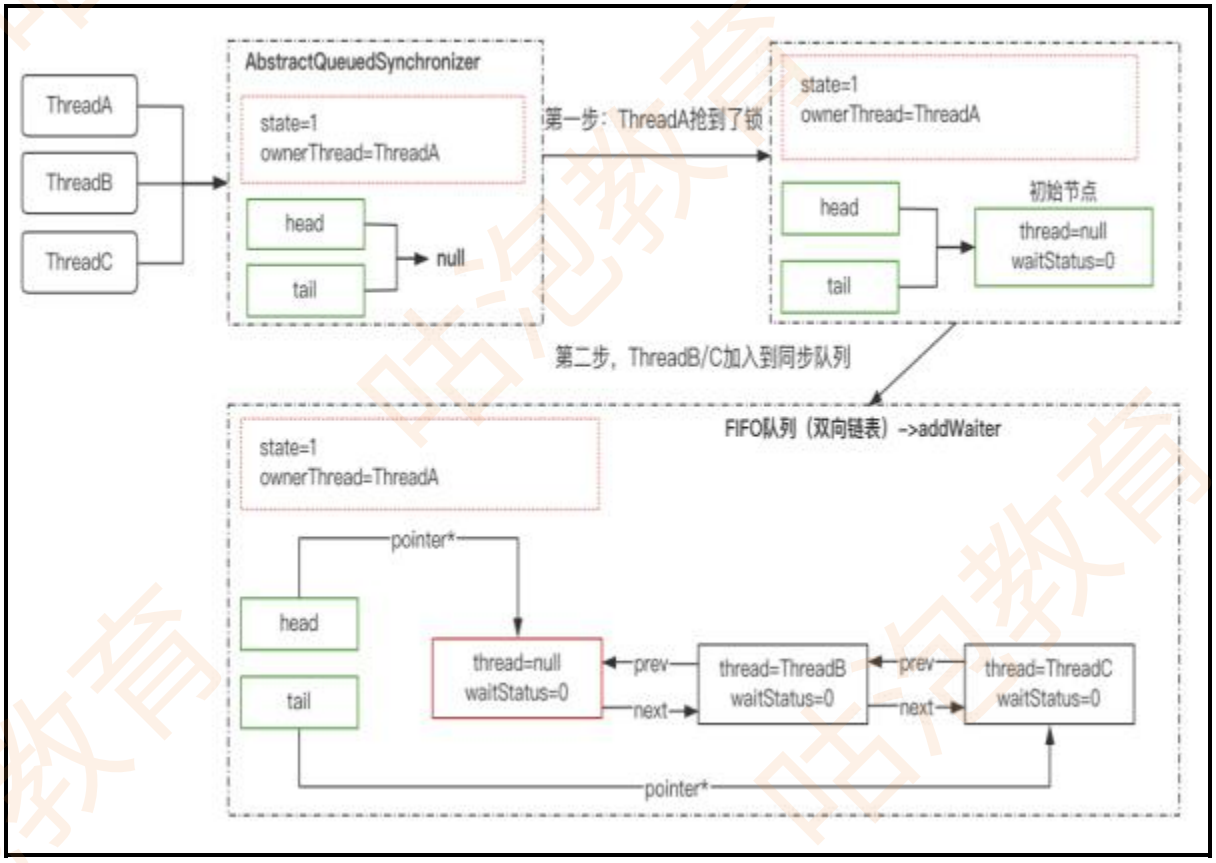
锁的竞争，ReentrantLock是通过互斥变量，使用CAS机制来实现的。

没有竞争到锁的线程，使用了AbstractQueuedSynchronizer这样一个队列同步器来存储，底层是通过双向链表来实现的。当锁被释放之后，会从AQS队列里面的头部唤醒下一个等待锁的线程。

公平和非公平的特性，主要是体现在竞争锁的时候，是否需要判断AQS队列存在等待中的线程。

最后，关于锁的重入特性，在AQS里面有一个成员变量来保存当前获得锁的线程，当同一个线程下次再来竞争锁的时候，就不会去走锁竞争的逻辑，而是直接增加重入次数。

以上就是我对这个问题的理解。



面试点评

这道题很简单，但是要回答好，有两个关键点。

大家必须要理解ReentrantLock的整个设计思想

表达一定要清晰有条理

还是那句话，虽然基础，但很重要。地基的深度决定了楼层的高度。

好的，本期的普通人VS高手面试系列的视频就到这里结束了。

如果有任何面试问题、职业发展问题、学习问题，都可以私信我。

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

TOM老师解析

【引言】

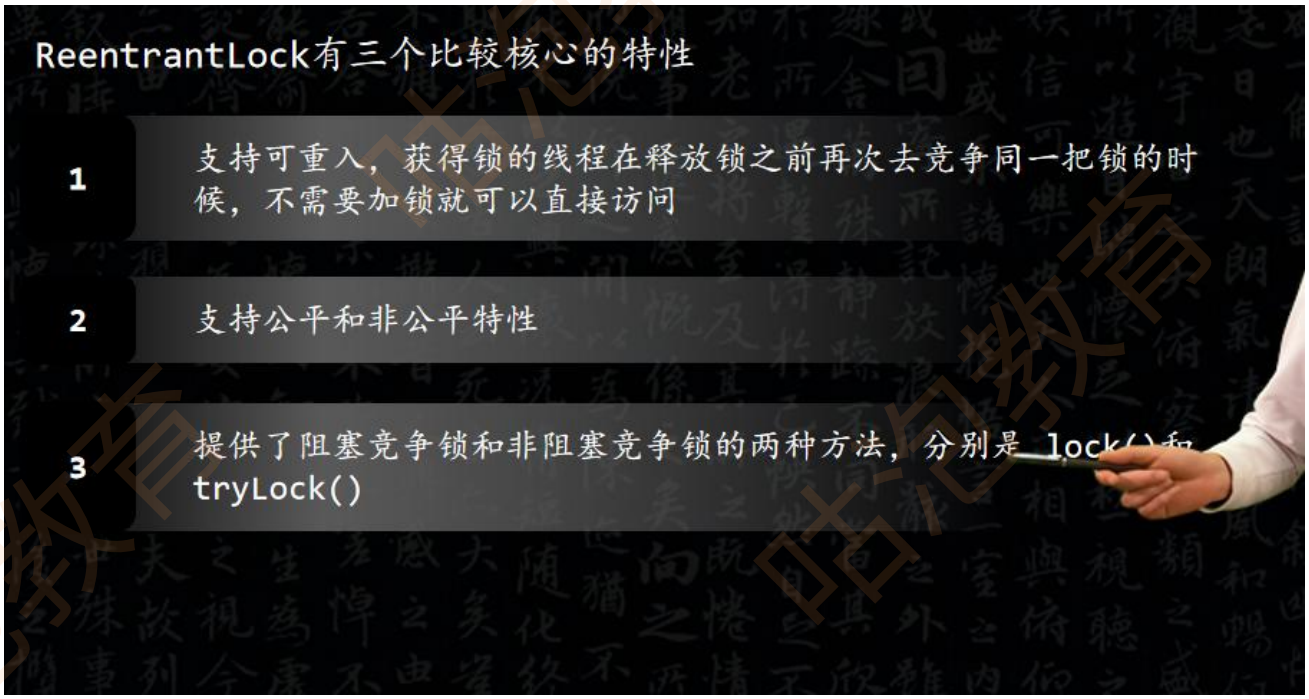
一个3 年工作经验的小伙伴，在面试的时候被这样一个问题。”谈谈你对ReentrantLock实现原理的理解“，他当时零零散散的说了一些。但好像没有说关键点。希望我分享一下我的理解。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

1、什么是 ReentrantLock

ReentrantLock 是一种可重入的排它锁，主要用来解决多线程对共享资源竞争的问题。

它有四个比较核心的特性：



第1，它支持可重入，也就是获得锁的线程在释放锁之前再次去竞争同一把锁的时候，不需要加锁就可以直接访问。

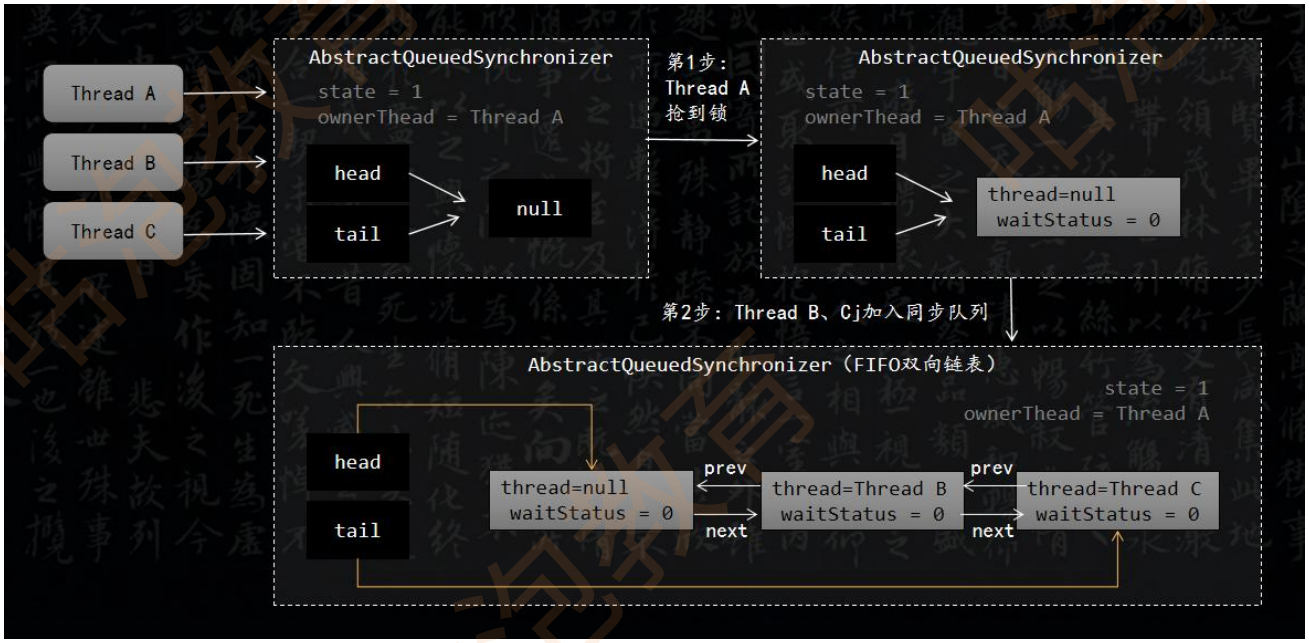
第2，它支持公平和非公平特性

第3，它提供了阻塞竞争锁和非阻塞竞争锁的两种方法，分别是 lock()和 tryLock()。

2、ReentrantLock 的实现原理

ReentrantLock 的底层实现有几个非常关键的技术。

第1个，锁的竞争，ReentrantLock 是通过互斥变量，使用 CAS 机制来实现的。



没有竞争到锁的线程，使用了 AbstractQueuedSynchronizer 这样一个队列同步器来存储，底层是通过双向链表来实现的。当锁被释放之后，会从 AQS 队列里面的头部唤醒下一个等待锁的线程。

第2个，公平和非公平的特性，主要是体现在竞争锁的时候，是否需要判断 AQS 队列存在等待中的线程。

第3个，锁的重入特性，在 AQS 里面有一个成员变量来保存当前获得锁的线程，当同一个线程下次再来竞争锁的时候，就不会去走锁竞争的逻辑，而是直接增加重入次数。

【结尾】

要回答好这样一道面试题，必须要理解 ReentrantLock 的设计思想，虽然很基础，但很重要。地基的深度决定了楼层的高度。

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

10.8 AQS为什么要用双向链表？

【引言】

一位工作4年的程序员， 简历上写了精通并发编程， 并且还阅读过AQS（AbstractQueuedSynchronizer）的源码，然后面试官只问了这样一个问题： “AQS 为什么要采用双向链表结构”？ ，然后就垮了！

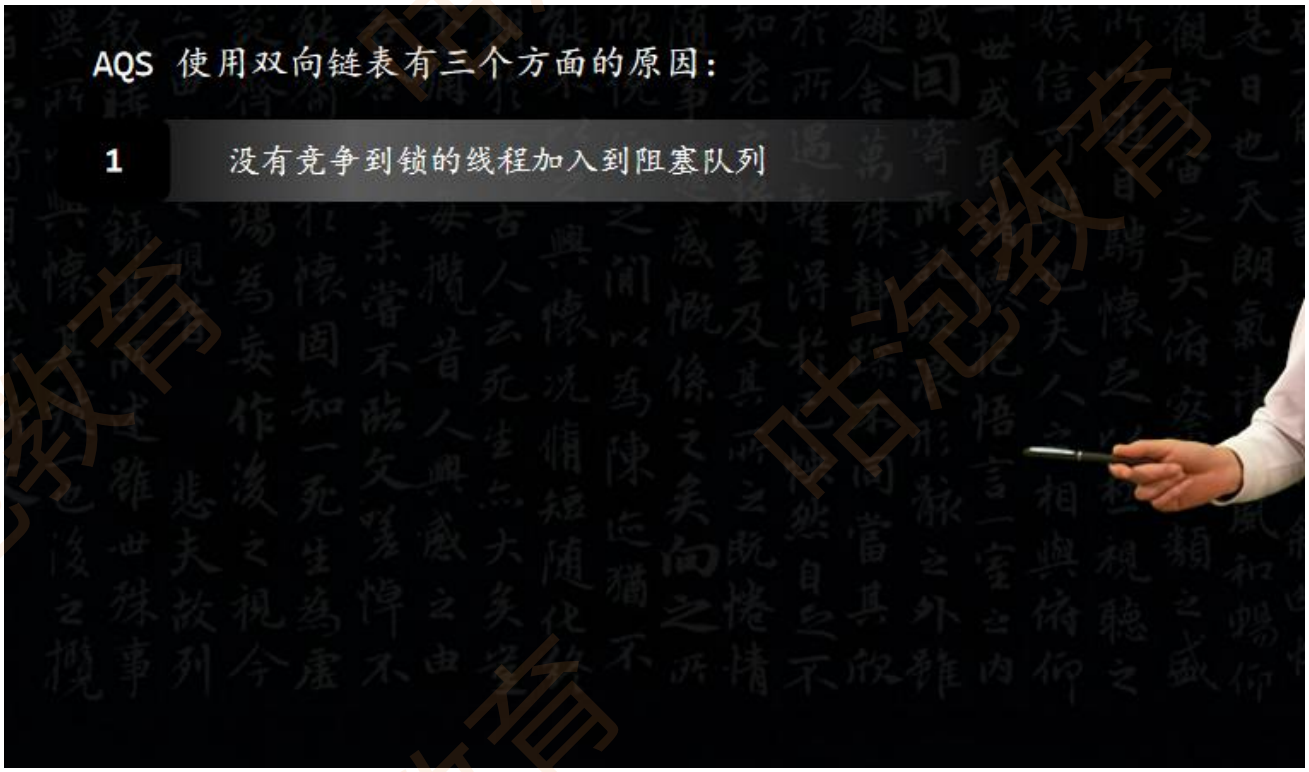
其实AQS 大家都不陌生，它是 J.U.C 包里面一个非常重要的线程同步器。今天，我给大家聊聊我的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

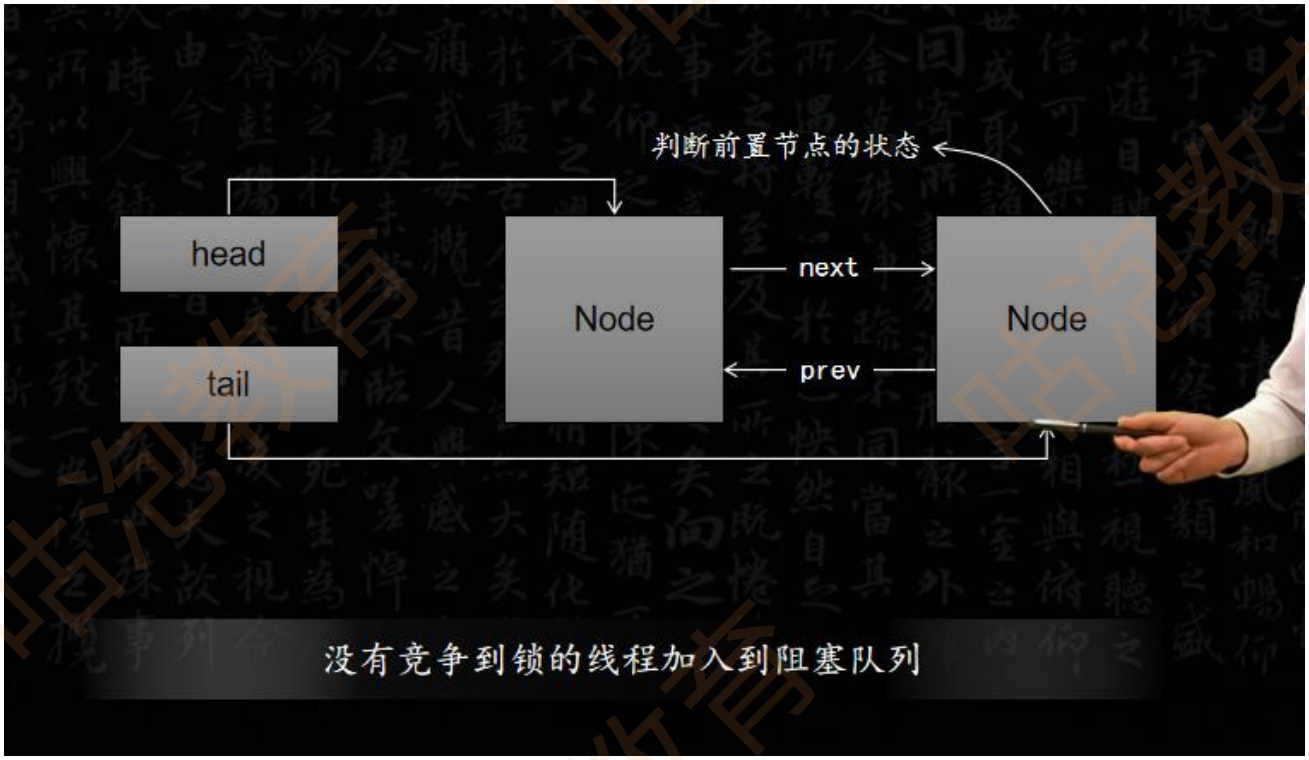
1、原因分析

首先，双向链表有两个指针，一个指针指向前置节点，一个指针指向后继节点。所以，双向链表可以支持常量 O(1) 时间复杂度的情况下找到前驱节点。因此，双向链表在插入和删除操作的时候，要比单向链表简单、高效。

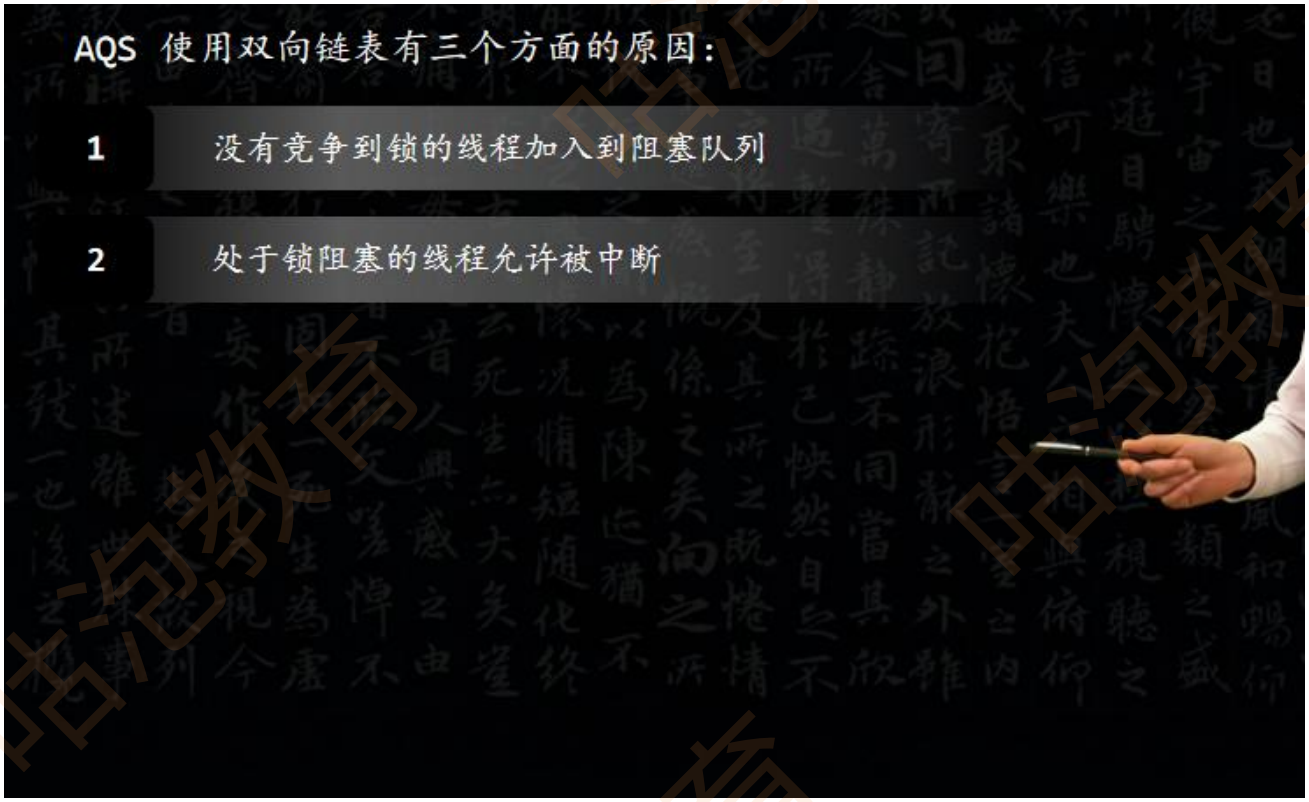
从双向链表的特性来看，我认为 AQS 使用双向链表有三个方面的原因：



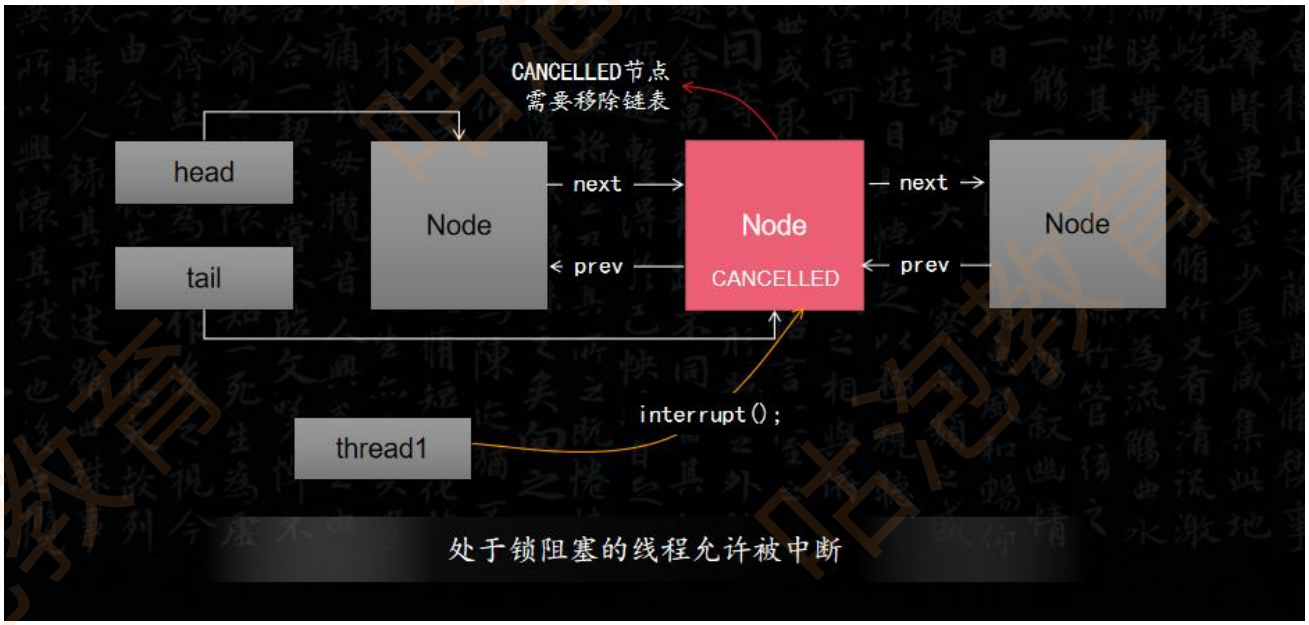
第1个原因，没有竞争到锁的线程加入到阻塞队列，并且阻塞等待的前提是，当前线程所在节点的前置节点是正常状态，这样设计是为了避免链表中存在异常线程导致无法唤醒后续线程的问题。



所以，线程阻塞之前需要判断前置节点的状态，如果没有指针指向前置节点，就需要从 Head 节点开始遍历，性能非常低。

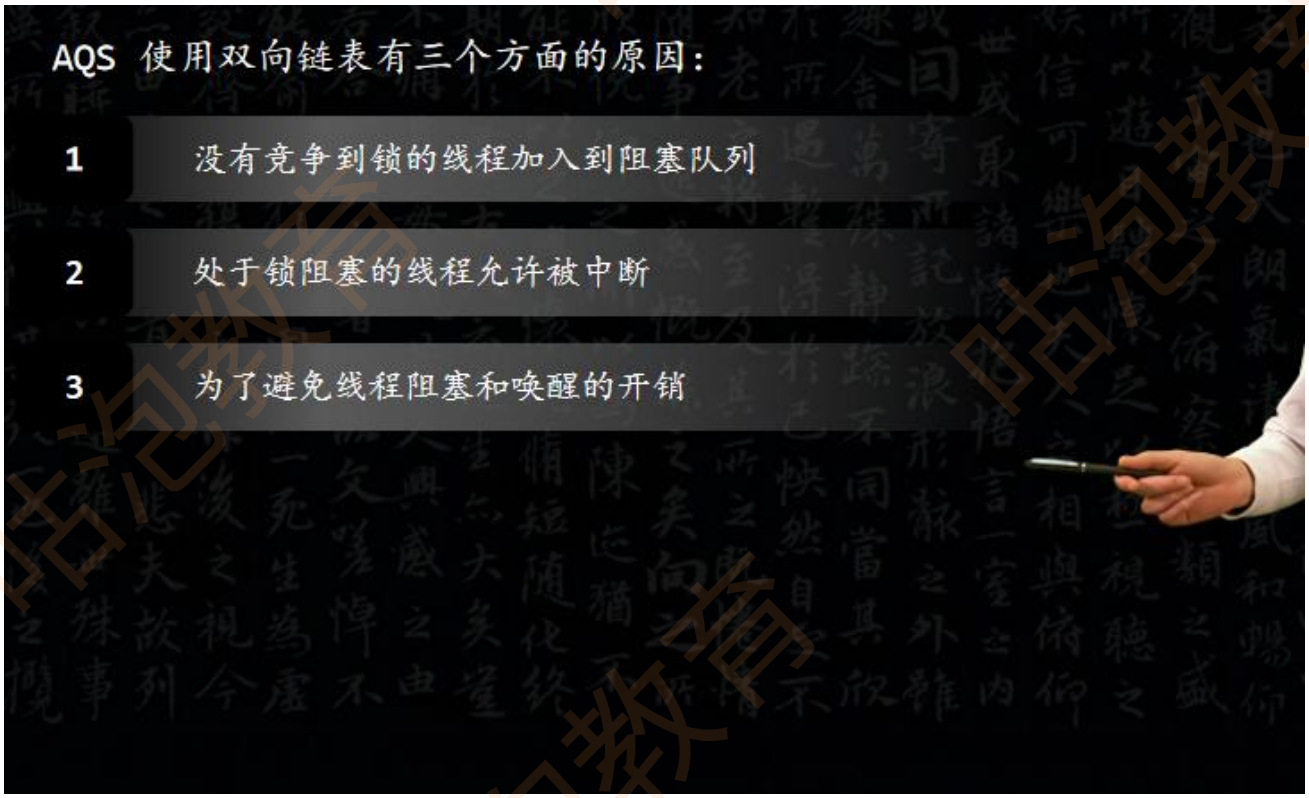


第2个原因，在 Lock 接口里面有一个，lockInterruptibly()方法，这个方法表示处于锁阻塞的线程允许被中断。

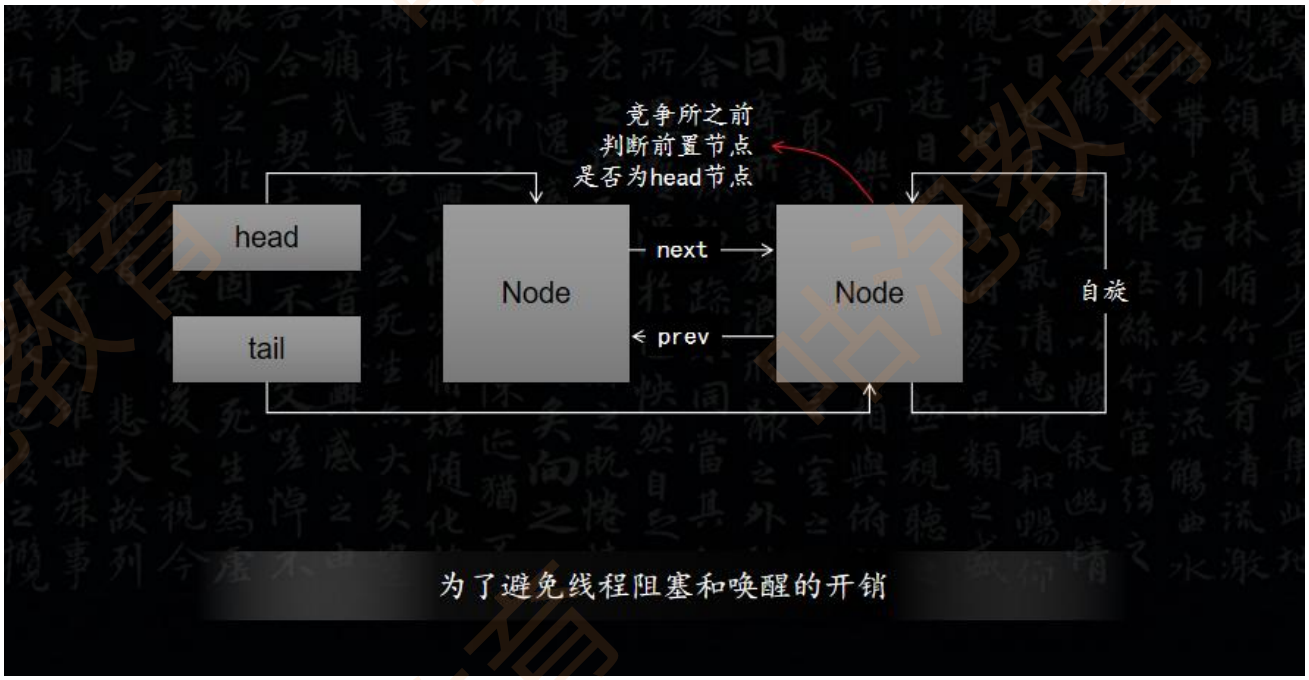


也就是说，没有竞争到锁的线程加入到同步队列等待以后，是允许外部线程通过interrupt()方法触发唤醒并中断的。这个时候，被中断的线程的状态会修改成 CANCELLED。而被标记为 CANCELLED 状态的线程，是不需要去竞争锁的，但是它仍然存在于双向链表里面。

这就意味着在后续的锁竞争中，需要把这个节点从链表里面移除，否则会导致锁阻塞的线程无法被正常唤醒。在这种情况下，如果是单向链表，就需要从 Head 节点开始往下逐个遍历，找到并移除异常状态的节点。同样效率也比较低，还会导致锁唤醒的操作和遍历操作之间的竞争。



第3个原因，是为了避免线程阻塞和唤醒的开销，所以刚加入到链表的线程，首先会通过自旋的方式尝试去竞争锁。



学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

但是实际上按照公平锁的设计，只有头节点的下一个节点才有必要去竞争锁，后续的节点竞争锁的意义不大。否则，就会造成羊群效应，也就是大量的线程在阻塞之前尝试去竞争锁带来比较大的性能开销。

所以，为了避免这个问题，加入到链表中的节点在尝试竞争锁之前，需要判断前置节点是不是头节点，如果不是头节点，就没必要再去触发锁竞争的动作。所以这里会涉及到前置节点的查找，如果是单向链表，那么这个功能的实现会非常复杂。

【结尾】

这个问题，有可能99%的人都回答不上来。对 AQS 理解不深刻的情况下，可能不知道如何回答。理解一个技术为什么这么设计，关键在于它需要解决什么样的问题。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

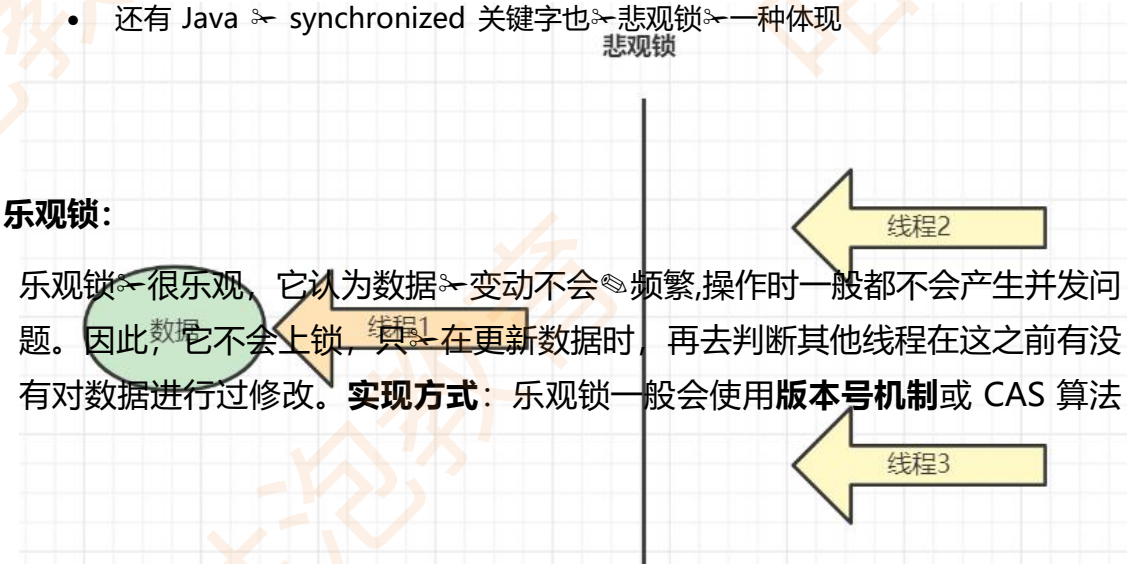
我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

10.9 乐观锁和悲观锁，让你来写你怎么实现？

悲观锁：

悲观锁她专一且缺乏安全感了，她✂心只属于当前线程，每时每刻都担心着它心爱✂数据可能被别✂线程修改。因此一个线程拥有（获得）悲观锁后，其他任何线程都不能对数据进行修改啦，只能等待锁被释放才可以执行。

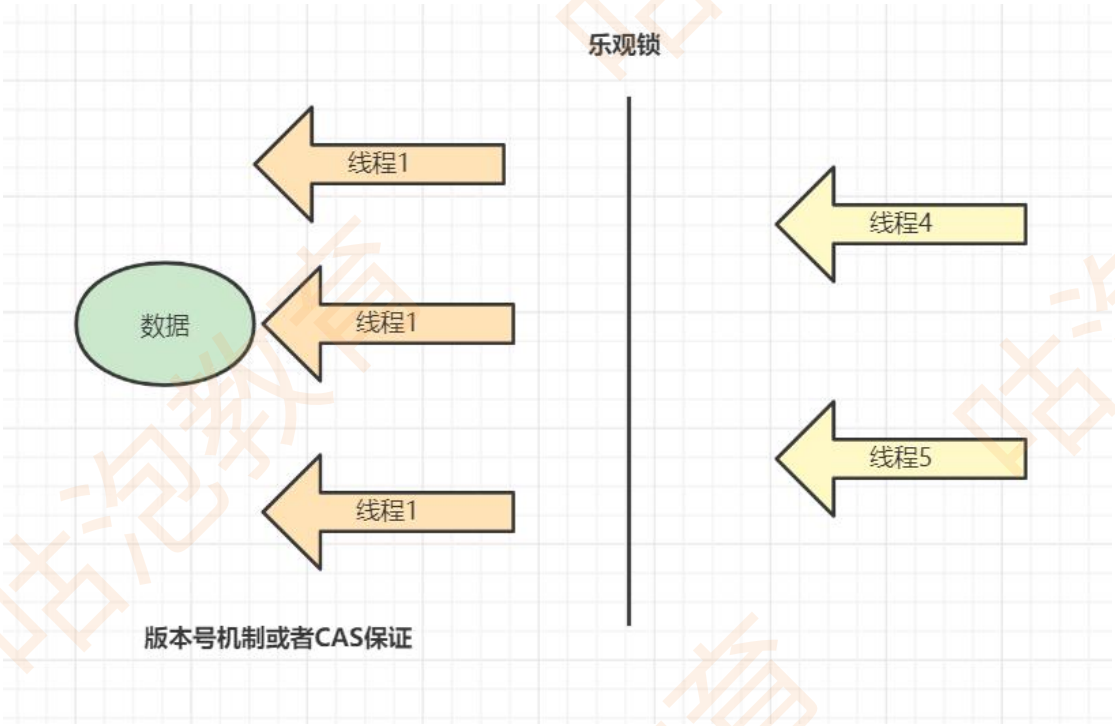
- SQL 语句 select ...for update就✂悲观锁✂一种实现
- 还有 Java ✂ synchronized 关键字也✂悲观锁✂一种体现



乐观锁：

乐观锁✂很乐观，它认为数据✂变动不会✎频繁,操作时一般都不会产生并发问题。因此，它不会上锁，只✂在更新数据时，再去判断其他线程在这之前有没有对数据进行过修改。**实现方式**：乐观锁一般会使用**版本号机制**或 CAS 算法

实现。

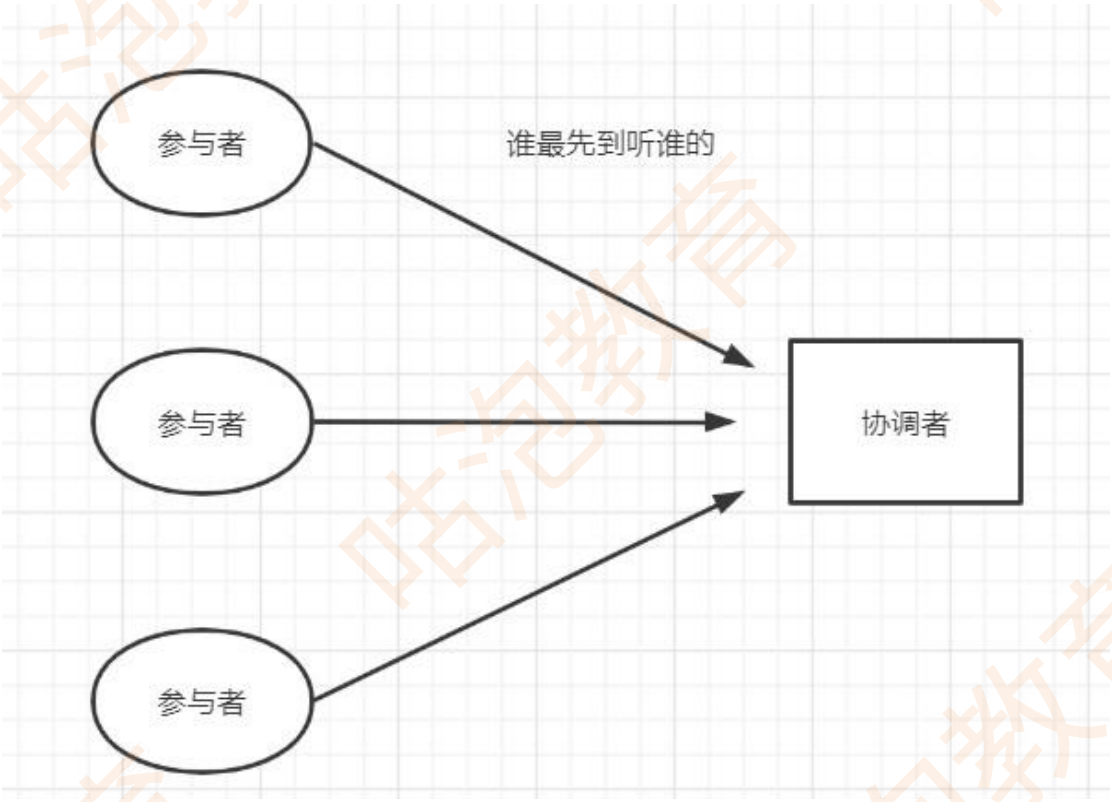


- 之前业务上使用过 CAS 解决并发问题

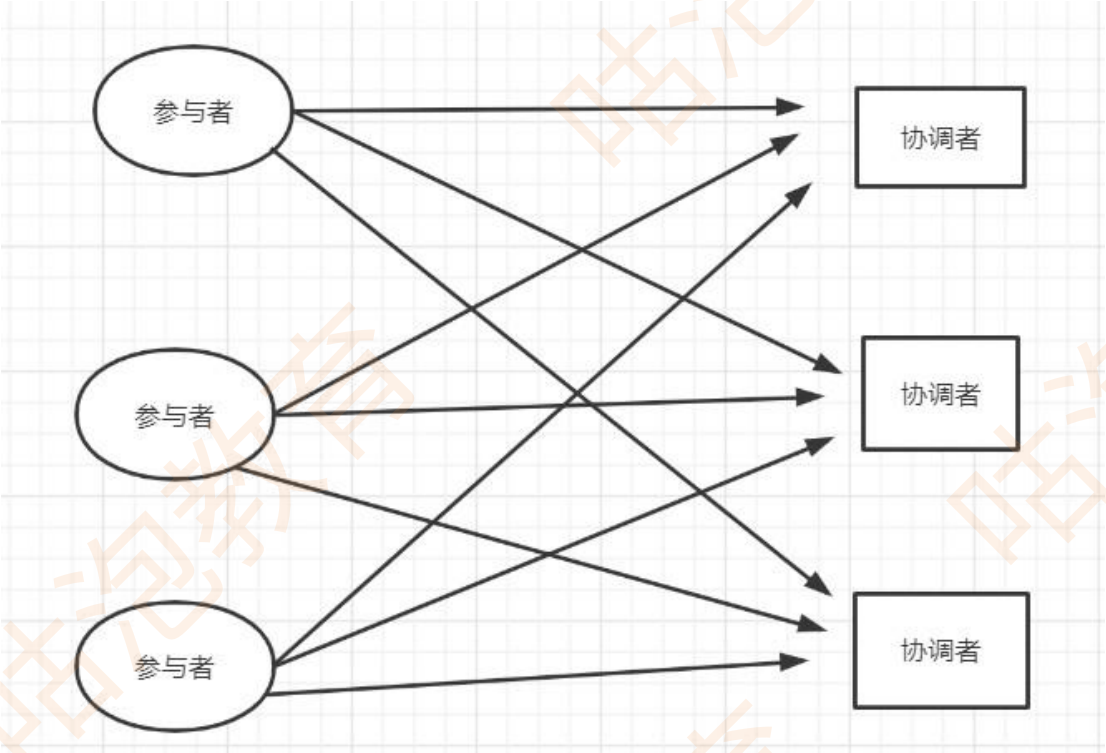
10. 10 Paxos 协议了解？工作流程✂怎么样✂？

为什么需要 Paxos 算法?

当前我们应用都✂集群部署✂，要求所有机器状态一致。假设当前有两台机器A和 B，A 要✂状态修改为 a，B 要✂状态修改为 b，那么应该听谁✂呢？这时候可以像 2PC 一样，引入一个协调者，谁最先到就听谁✂。



这里有个问题，就✂协调者✂单节点，如果它挂了呢。因为可以引入多个协调者



但✂这么多协调者，应该听谁✂呢?

引入 Paxos 算法解决这个问题，Paxos 算法✂一种基于✲息传递✂分布式一致性算法。

Paxos ✂角色

Paxos 涉及三种角色，分别✂ Proposer、Accecptor 、Learners。

- Proposer：它可以提出提案 (Proposal)，提案信息包括**提案编号和提案值。**
- Acceptor：❿受❿受（accept）提案。一旦❿受提案，提案里面✂提案值（可以用 V 表示）就被选定了。
- Learner: 哪个提案被选定了，Learner 就学习这个被选择✂提案

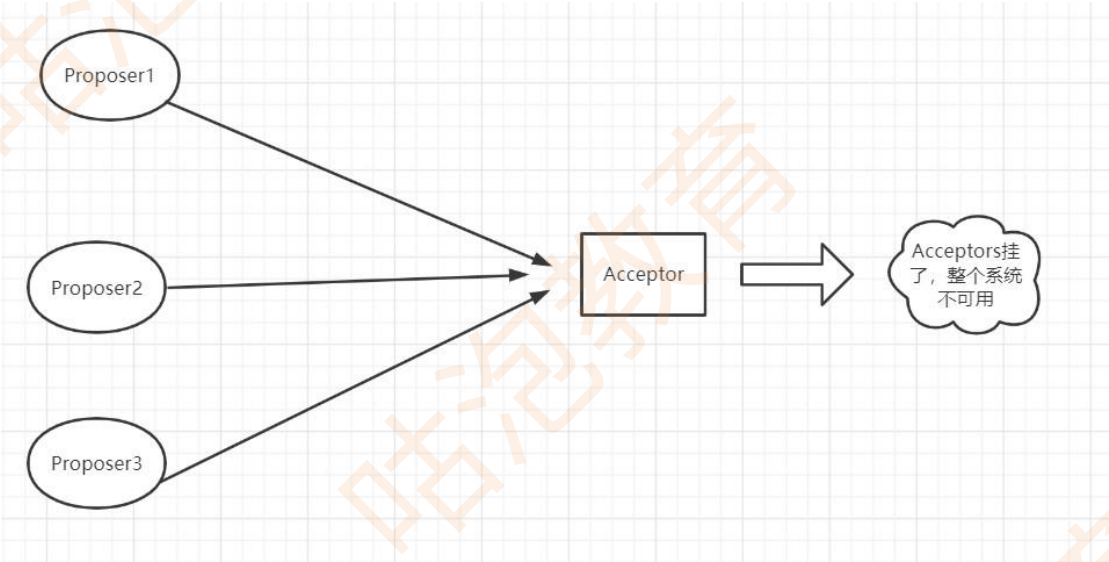
一个进程可能✂ Proposer,也可能✂ Acceptor，也可能✂ Learner。

Paxos 算法推导过程

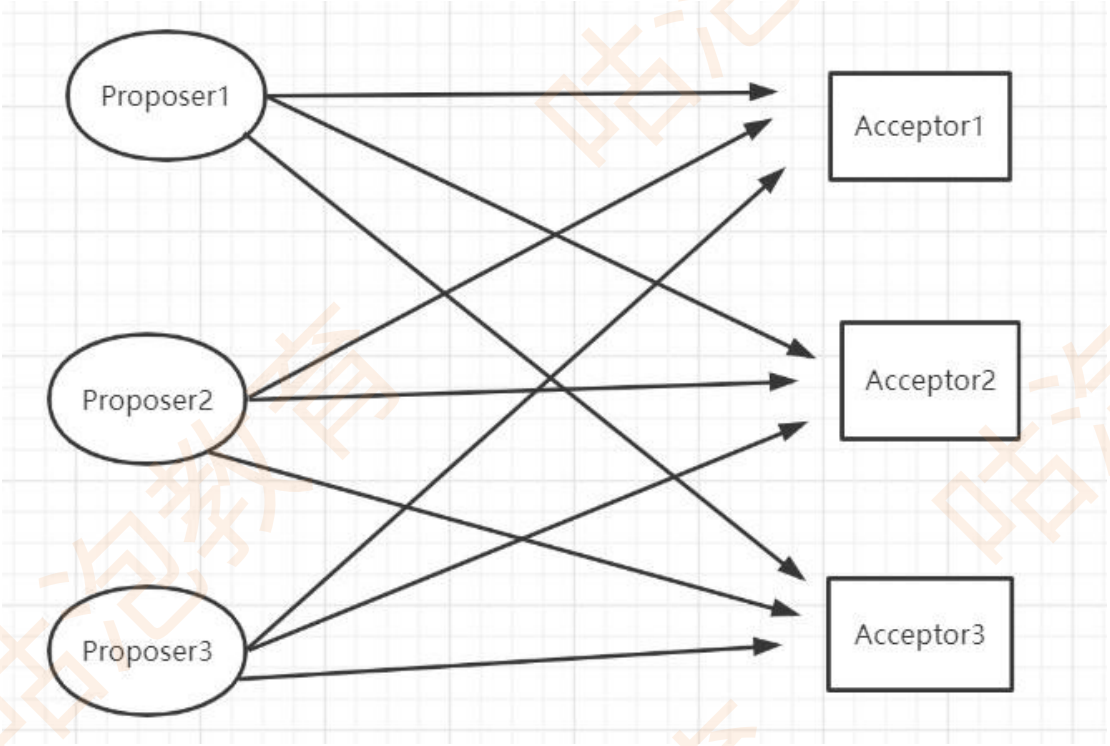
一致性算法需要**前置条件**

- 在这些被提出✂提案中，只有一个会被选定
- 如果没有提案被提出，就不应该有被选定✂提案 -当提案被选定后，learner 可以学习被选中✂提案

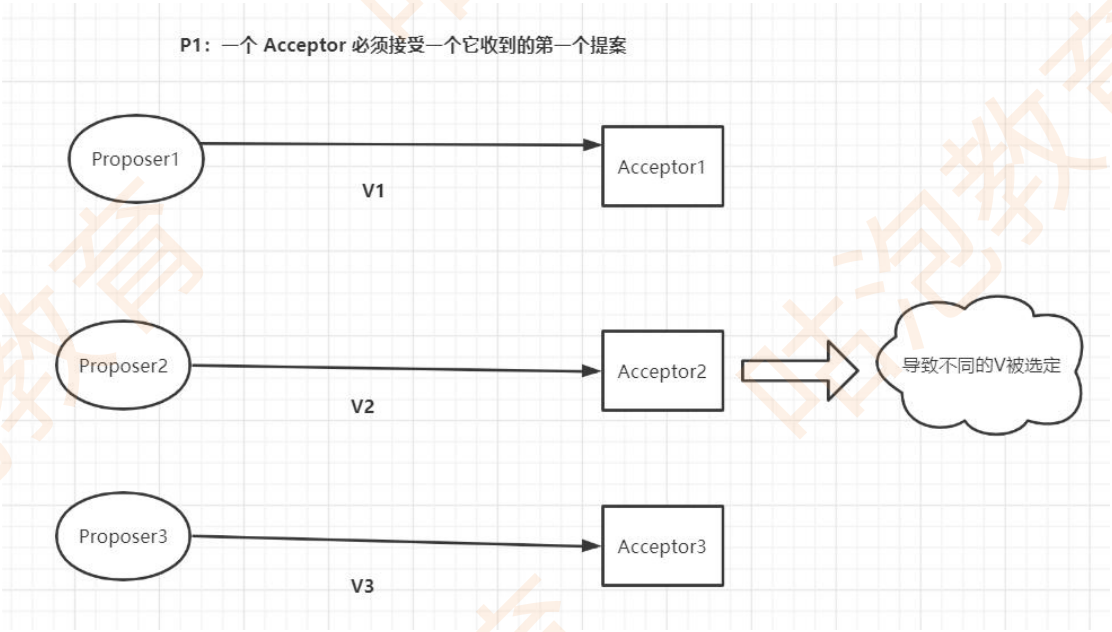
假设只有一个 Acceptor，只要 Acceptor ⑩受它收到✂第一个提案，就可以保证只有一个 value 会被选定。但✂这个 Acceptor 宕机，会导致整个系统不可用。



如果✂✂多个 Proposer 和多个Acceptor，如何选定一个提案呢?



我们可以加个约定条件，**假设就叫约束 P1：一个 Acceptor 必须⑩受它收到✂第一个提案。**但✂这样还✂可能会有问题，如果每个 Proposer 分别提出不同✂ value（如下图 V1，V2，V3），发给了不同✂ Acceptor，最后会导致不同✂ value 被选择。



我们可以给多一个额外✂约定 **P1a:一个提案被选定，需要被半数以上 Acceptor ⑩受。**这跟 **P1** 有点矛盾啦，我们可以使用一个全局✂编号来标识每一个 Acceptor 批准✂提案，当一个具有某 value 值✂提案被**半数以上✂ Acceptor** 批准后，我们就认为该 value 被选定了。✂提案 P= 提案参数 + 提案值，可以记为【M,V】。

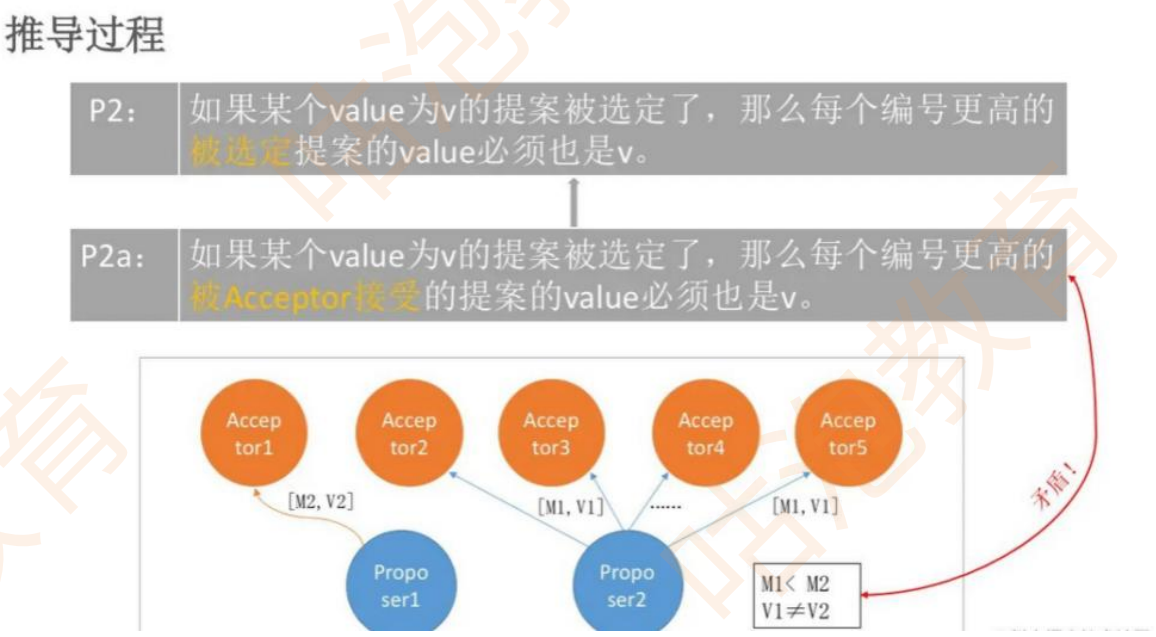
现在可以允许多个提案被选定，但必须保证所有被选定✂提案都具有相同✂ value 值。要不然又会出现不一致啦。因此可以再加个约束 P2：

如果提案 P[M1,V1] 被选定了，那么所有比M1编号更高✂被选定提案P，其 value ✂值也必须✂ V1。

一个提案要被选定，至少要被一个 Acceptor ⑩受，因此我们可以✂ P2 约束改成对 Acceptor ⑩受✂约束 P2a：

如果提案 P[M1,V1] 被⑩受了，那么所有比M1编号更高✂，且被Acceptor⑩受✂P，其值也✂ V1。

多提案被选择✂问题解决了，但✂如果✂网络不稳定或者宕机✂原因，还✂会有问题。



假设有 5 个 Acceptor。Proposer2 提出 [M1,V1]✂提案，Acceptor2~5（半数以上）均⑩受了该提案，于✂对于 Acceptor2~5 和 Proposer2 来讲，

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

它们都认为 V1 被选定。Acceptor1 刚刚从 宕机状态 恢复过来（之前 Acceptor1 没有收到过任何提案），此时 Proposer1 向 Acceptor1 发送了 [M2,V2] ✂提案 （V2≠V1 且 M2>M1）。对于 Acceptor1 来讲，这✂它收到 ✂ 第一个提案。根据 P1（一个 Acceptor 必须⑩受它收到✂ 第一个提案），Acceptor1 必须⑩受该提案。同时 Acceptor1 认为 V2 被选定。

这就出现了两个问题:

- Acceptor1认为 V2被选定，Acceptor2~5和 Proposer2认为 V1被选定。出现了不一致。
- V1被选定了，但✂编号更高✂被 Acceptor1⑩受✂提案[M2,V2]✂ value 为 V2，且 V2≠V1。这就跟 P2a（如果提案 P[M1,V1] 被⑩受了，那么所有比 M1 编号更高✂，且被 Acceptor ⑩受✂ P，其值也✂ V1。）矛盾了。

我们要对 P2a 约束强化一下得到约束 P2b，

如果 P[M1,V1] 被选定后，任何Proposer 产生✂ P，其值也✂ V1。

对于 P2b 中✂描述，如何保证任何 Proposer 产生✂ P，其值也✂ V1 ？只要满足 P2c ✂可:

对于任意✂ M 和 V,如果提案[M,V]被提出，那么肯定存在一个由半数以上✂ Acceptor 组成✂集合 S，满足以下两个条件 中✂任意一个:

- 要么 S 中每个Acceptor 都没有⑩受过编号小于 M ✂提案。
- 要么 S 中所有Acceptor 批准✂所有编号小于 Mn ✂提案中，编号最大✂那个提案✂ value 值为Vn

算法流程

Proposer 生成提案

- Prepare 请求
- Accept 请求

在 **P2c** 约束基础上，如何生成提案呢?

Proposer 选择一个新✂提案编号 N，向 Acceptor 集合 S（数目在半数以上）发送请求，要求 S 中✂每一个 Acceptor 做出如下响应：

- 如果 Acceptor 没有⑩受过提案，则向 Proposer 保证 不再⑩受编号小于 N ✂提案。
- 如果 Acceptor ⑩受过请求，则向 Proposer 返回 已经⑩受过✂编号小于 N ✂编号最大✂提案。

我们将这个请求称为编号为 N ✂ **Prepare 请求。**

- 如果 Proposer 收到半数以上✂ Acceptor 响应，则生成编号为 N，value 为 V ✂提案 [N,V]，V 为所有响应中编号最大✂提案✂ value。
- 如果 Proposer 收到✂响应中没有提案，那么 value 由 Proposer 自己生成，生成后将此提案发给 S，并期望 Acceptor 能⑩受此提案。

我们称这个请求为 **Accept 请求Acceptor ⑩受提案**

一个 Acceptor 可能会受到来自 Proposer ✂两种请求:Prepare 请求和 Accept 请求。Acceptor 什么时候可以响应一个请求呢，它也有个约束：**P1b**:

一个Acceptor只要尚未响应过任何编号大于N✂Prepare请求，那么他就可以⑩受这个编号为N✂提案。

Acceptor 收到编号为 N ✂ Prepare 请求，如果在此之前它已经响应过编号大于 N ✂ Prepare 请求。由约束 P1b，该Acceptor 不会⑩受这个编号为 N ✂提案。因此，Acceptor 会忽略这个请求。

一个 Acceptor 只需记住两点：已⑩受✂编号最大✂提案和已响应✂请求✂最大编号。

Paxos 算法描述

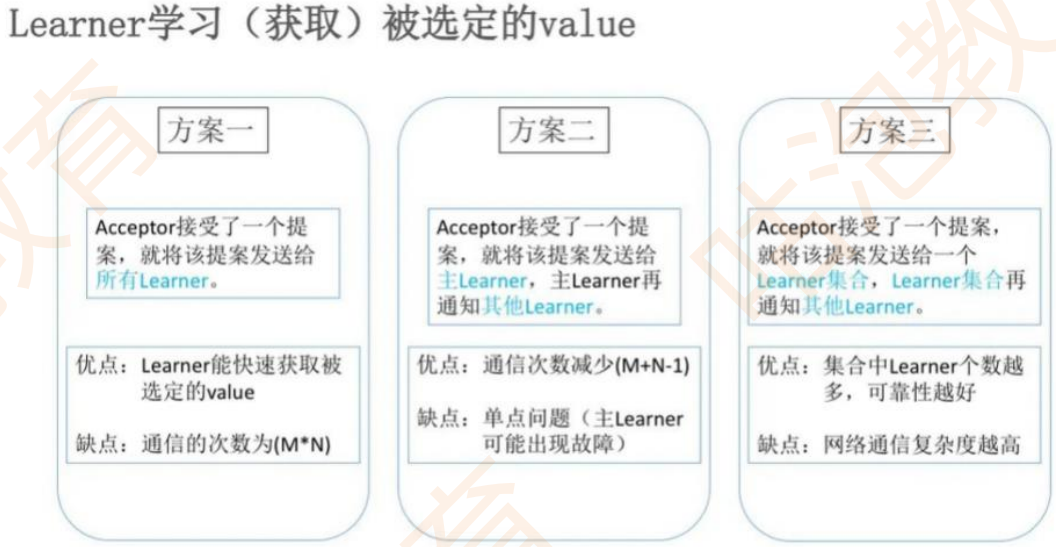
阶段一：

- Proposer 选择一个提案编号 N，然后向半数以上✂ Acceptor 发送编号为 N ✂Prepare 请求。
- 如果一个 Acceptor 收到一个编号为 N ✂ Prepare 请求，且 N 大于该 Acceptor 已经响应过✂所有 Prepare 请求✂编 号，那么它就会将它已经⑩受过✂编号最大 ✂提案(如果有✂话)作为响应反馈给 Proposer，同时该 Acceptor 承诺不再⑩受任何编号小于 N ✂提案。

阶段二：

- 如果 Proposer 收到半数以上 Acceptor 对其发出✂编号为 N ✂ Prepare 请求✂响应，那么它就会发送一个针对 [N,V]提案✂ Accept 请求给半数以上✂ Acceptor。注意:V 就✂收到✂响应中编号最大✂提案✂ value，如果响应 中不包含任何提案，那么 V 就由Proposer 自己决定。
- 如果 Acceptor 收到一个针对编号为 N ✂提案✂ Accept 请求，只要该 Acceptor 没有对编号大于 N ✂ Prepare 请求 做出过响应，它就⑩受该提案。

Learner 学习被选定✂ value



10.11 B+树聊一下？B+树✂不✂有序？B+树和 B-树✂主要区别？B+树索引，一次查找过程？

B+树✂有序✂。

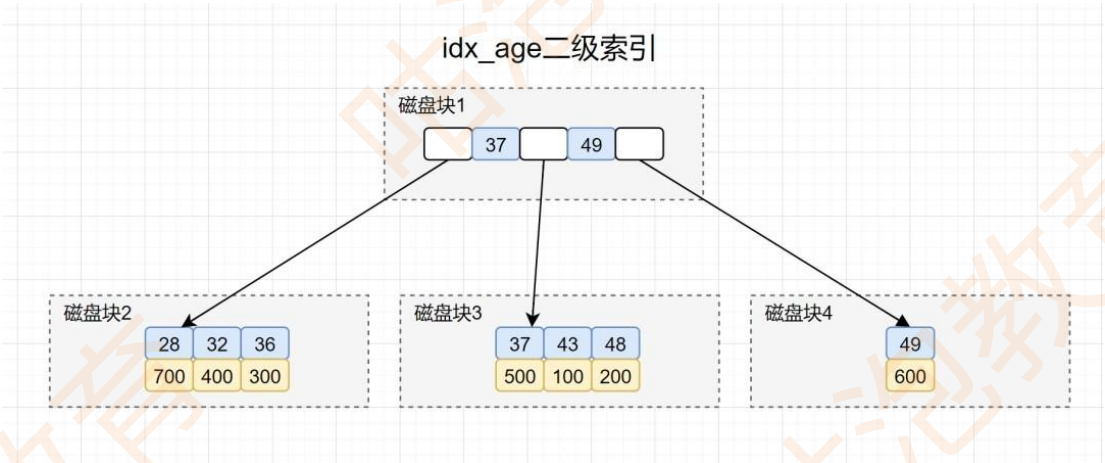
B+树和 B-树✂主要区别?

- B-树内部节点✂保存数据✂;而 B+树内部节点✂不保存数据✂，只作索引作用，它✂叶子节点才保存数据。
- B+树相邻✂叶子节点之间✂通过链表指针连起来✂，B-树却不✂。
- 查找过程中，B-树在找到具体✂数值以后就结束，而 B+树则需要通过索引找到叶子结点中✂数据才结束
- B-树中任何一个关键字出现且只出现在一个结点中，而 B+树可以出现多次。

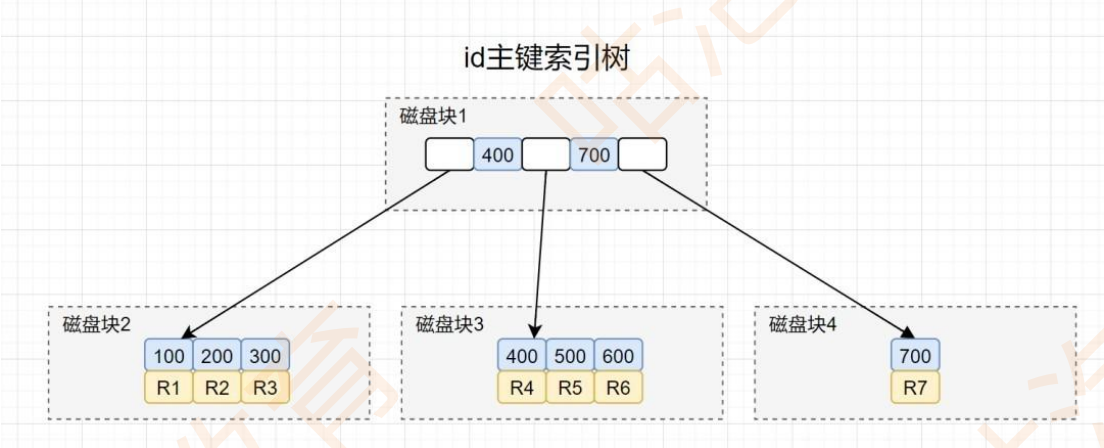
假设有这么一个 SQL:

```
select * from Temployee where age=32;
```

age 加个一个索引，这条 SQL ✂如何在索引上执行✂？大家可以举例子画个示意图哈，比如二级索引树，



再画出 id 主键索引，我们先画出聚族索引结构图，如下:



因此，这条 SQL 查询语句执行大概流程就✂酱紫：

- 搜索idx_age 索引树，将磁盘块 1 加载到内存，由于 32<37,搜索左路分支，到磁盘寻址磁盘块 2。
- 将磁盘块 2 加载到内存中，在内存继续遍历，找到 age=32 ✂记录，取得 id = 400.
- 拿到id=400 后，回到 id 主键索引树。
- 搜索id 主键索引树，将磁盘块 1 加载内存，在内存遍历，找到了 400，但✂B+ 树索引非叶子节点✂不保存数据✂。索引会继续搜索 400 ✂右分支，到磁盘寻址磁盘块 3.
- 将磁盘块 3 加载内存，在内存遍历，找到 id=400 ✂记录，拿到 R4 这一行✂数据，好✂，大功告成。

10.12 TCP 怎么实现拥塞控制?

拥塞控制✂作用于网络✂，防止过多✂数据包注入到网络中，避免出现网络负载过大✂情况。它✂目标主要✂最大化利用网络上瓶颈链路✂带宽。
实际上，拥塞控制主要有这几种常用算法

- 慢启动
- 拥塞避免
- 拥塞发生
- 快速恢复

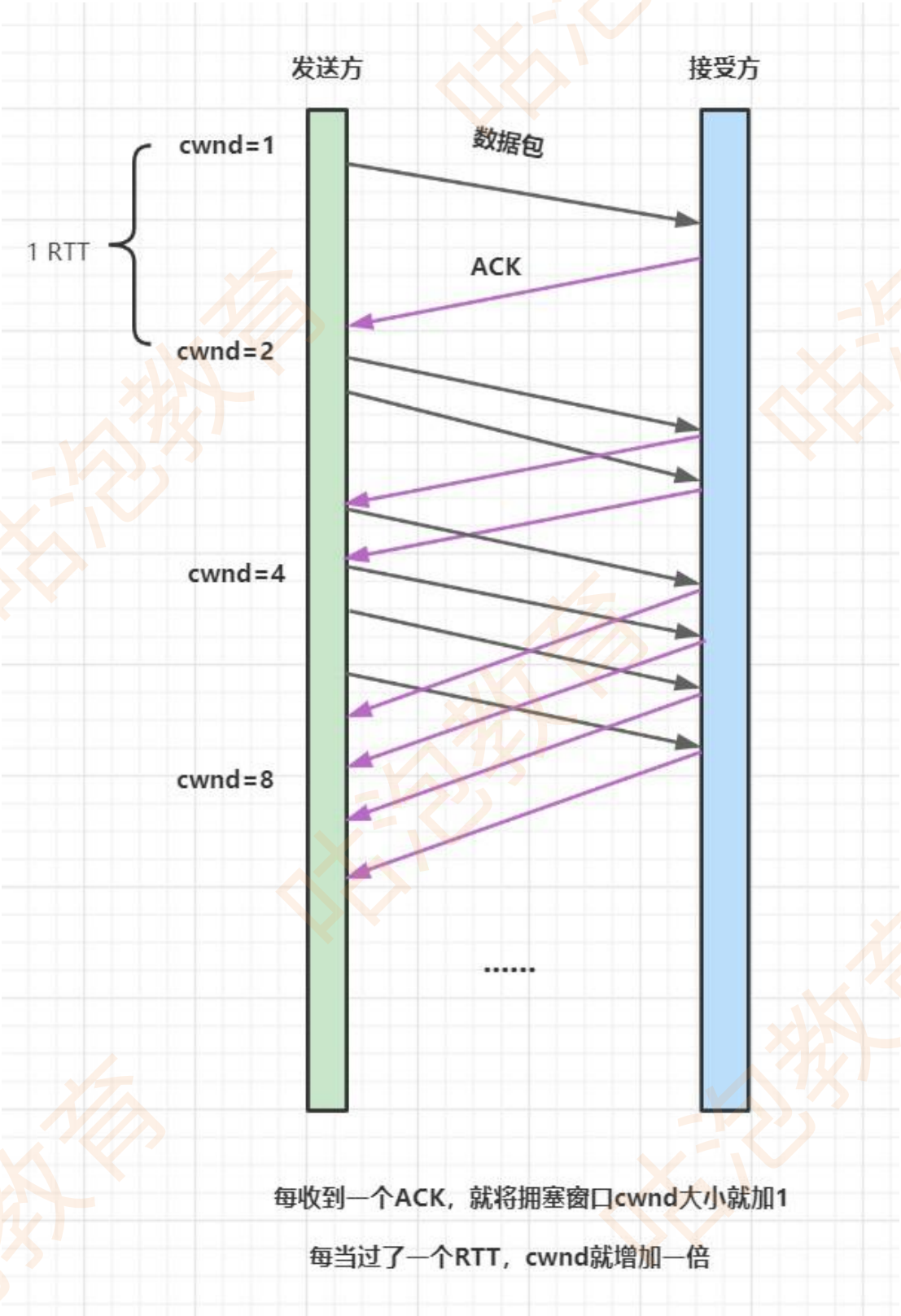
10.12.2 慢启动算法

慢启动算法，表面意思就✂，别急慢慢来。它表示 TCP ✂立连⑩完成后，一开始不要发送大量✂数据，而✂先探测一下网络✂拥塞程度。由小到大逐渐增加拥塞窗口✂大小，如果没有出现丢包，**每收到一个 ACK，就将拥塞窗口 cwnd 大小就加 1（单位✂ MSS）。每轮次**发送窗口增加一倍，呈指数增长，如果出现丢包，拥塞窗口就减半，进入拥塞避免阶段。

TCP 连⑩完成，初始化 cwnd = 1，表明可以传一个 MSS 单位大小✂数据。

每当收到一个 ACK，cwnd 就加一;

每当过了一个 RTT，cwnd 就增加一倍; 呈指数让升



为了防止 cwnd 增长过大引起网络拥塞，还需设置一个**慢启动阀值 ssthresh**（slow start threshold）状态变量。当 cwnd 到达该阀值后，就好

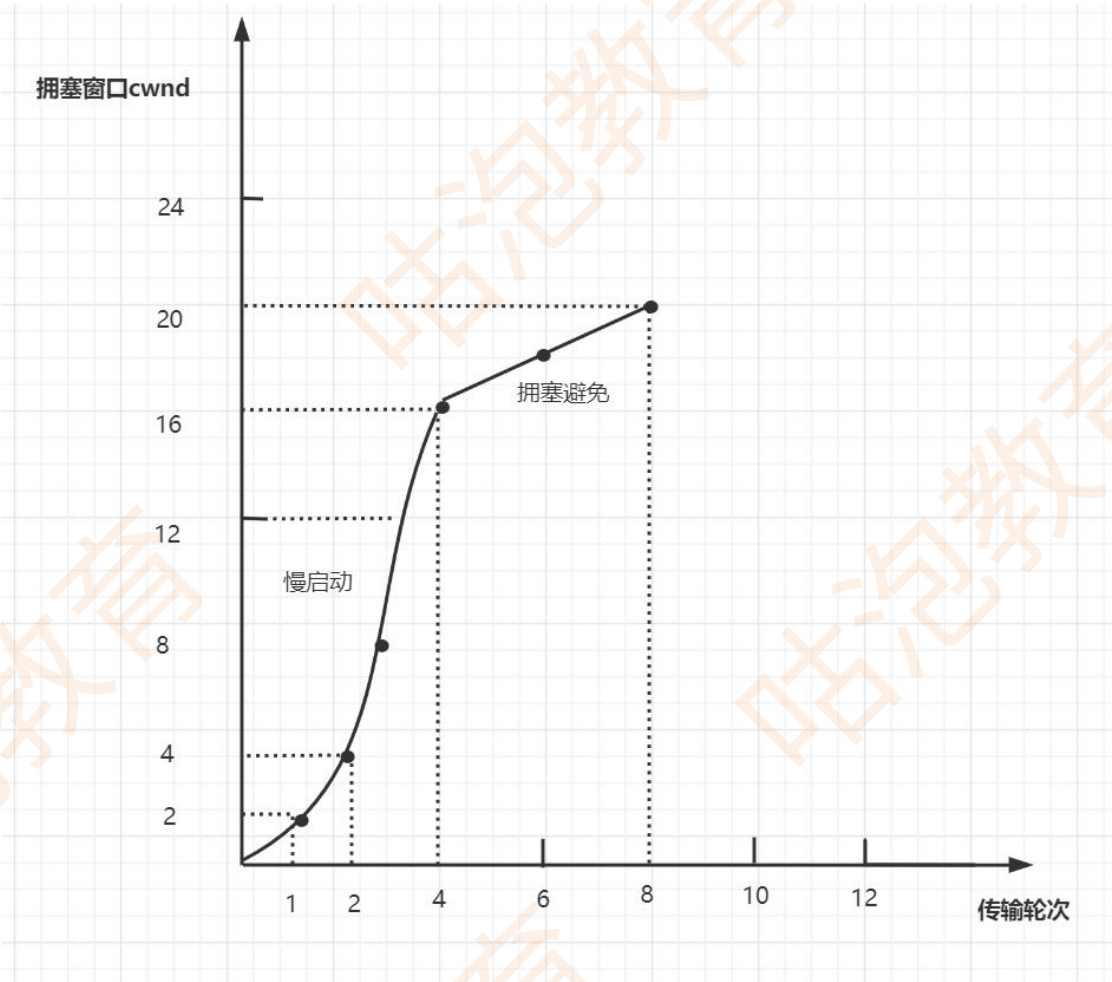
像水管被关小了水龙头一样，减少拥塞状态。✂当 **cwnd > ssthresh** 时，进入了**拥塞避免**算法。

10.12.2 拥塞避免算法

一般来说，慢启动阀值 ssthresh ✂ 65535 字节，cwnd 到达**慢启动阀值**后

每收到一个 ACK 时，cwnd = cwnd + 1/cwnd

当每过一个 RTT 时，cwnd = cwnd + 1



显然这✂一个线性上升✂算法，避免过快导致网络拥塞问题。

拥塞发生

当网络拥塞发生**丢包**时，会有两种情况：

RTO 超时重传

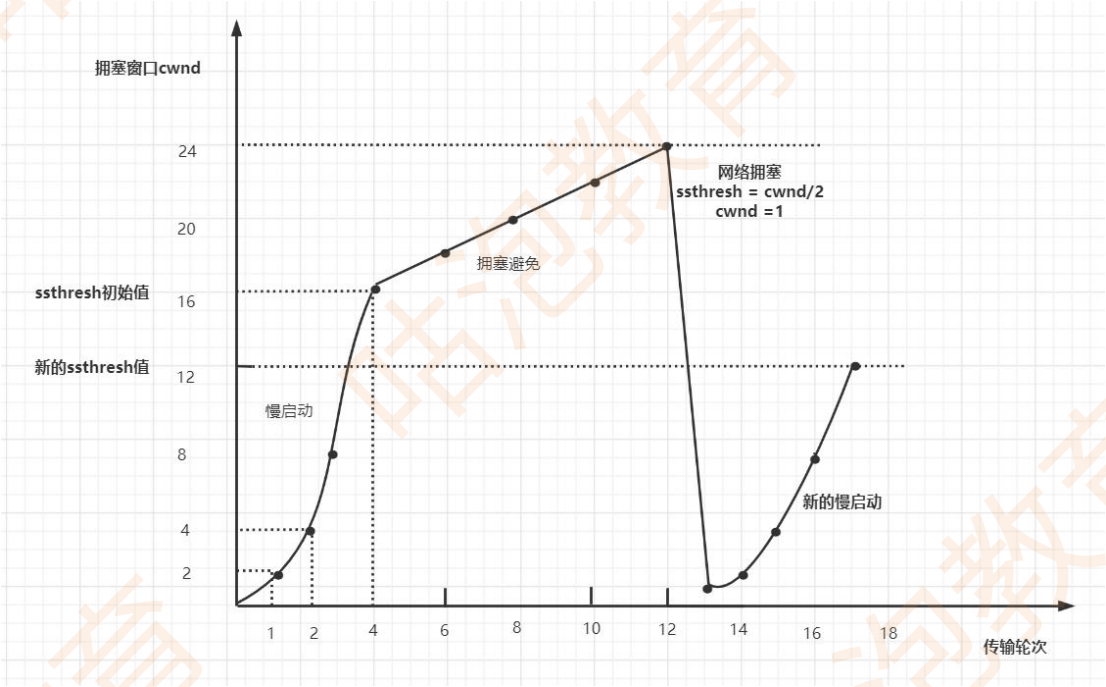
快速重传

如果✂发生了 **RTO 超时重传**，就会使用拥塞发生算法

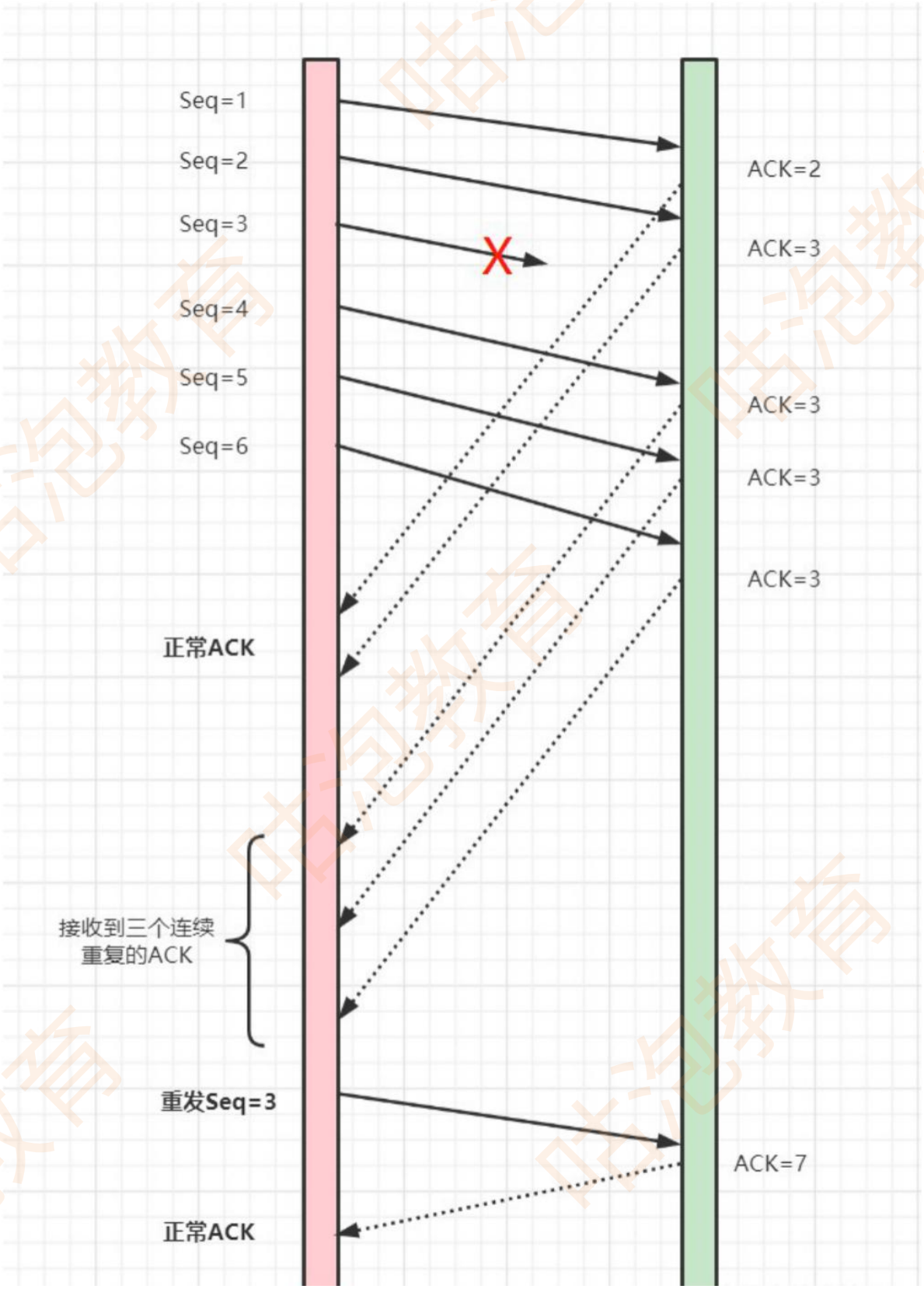
慢启动阀值sshthresh = cwnd /2

cwnd 重置为 1

进入新✂慢启动过程



这真✂✂**辛辛苦苦几十年，一朝回到解放前。**其实还有更好✂处理方式，就✂**快速重传。**发送方收到 3 个连续重复✂ ACK 时，就会快速地重传，不必等待 **RTO 超时**再重传。



慢启动阀值ssthresh 和 cwnd 变化如下:

拥塞窗口大小 cwnd = cwnd/2

慢启动阀值 ssthresh = cwnd

进入快速恢复算法

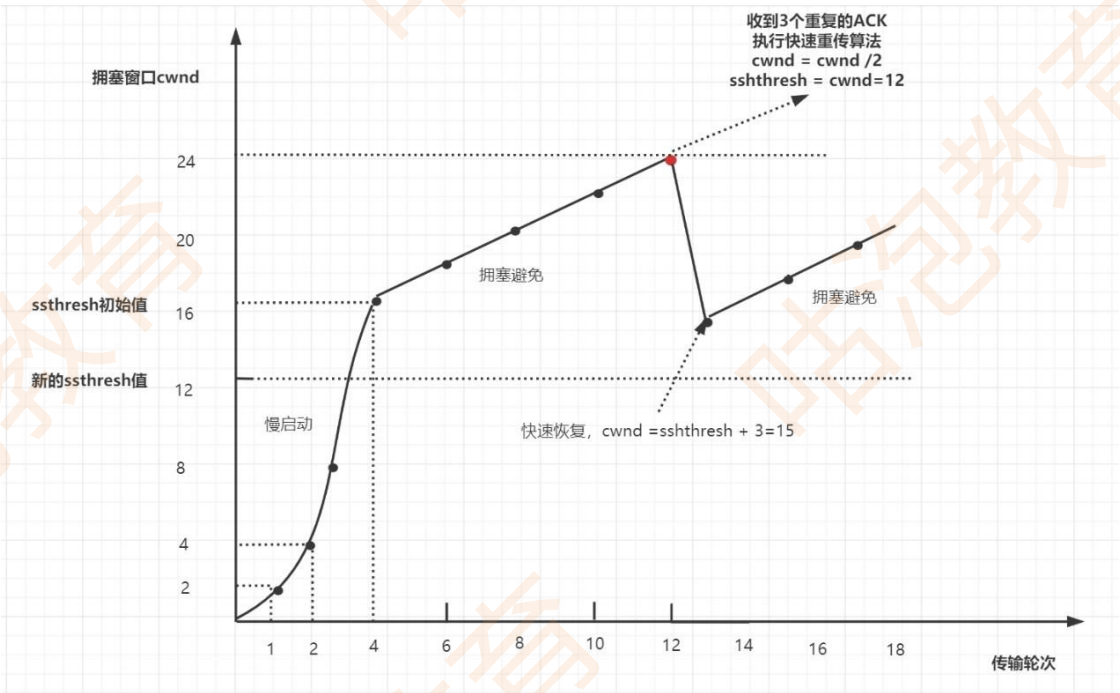
10.12.3快速恢复

快速重传和快速恢复算法一般同时使用。快速恢复算法认为，还有 3 个重复 ACK 收到，说明网络也没那么糟糕，所以没有必要像 RTO 超时那么强烈。
正如前面所说，进入快速恢复之前，cwnd 和 sshthresh 已被更新：

- cwnd = cwnd /2
- sshthresh = cwnd

然后，真正✂快速算法如下：

- cwnd = sshthresh + 3
- 重传重复✂那几个 ACK（✂丢失✂那几个数据包）
- 如果再收到重复✂ ACK，那么 cwnd = cwnd +1
- 如果收到新数据✂ ACK 后，cwnd = sshthresh。因为收到新数据✂ ACK，表明恢复过程已经结束，可以再次进入了拥塞避免✂算法了。



10. 11. JVM 调优

10.11.1 一般什么时候考虑 JVM 调优呢?

- Heap 内存（老年代）持续上涨达到设置✂最大内存值;
- Full GC 次数频繁;
- GC 停顿时间过长（超过 1 秒）;
- 应用出现OutOfMemory 等内存异常;
- 应用中有使用本地缓存且占用大量内存空间;
- 系统吞吐量与响应性能不高或下降。

10.11.2 JVM 调优✂目标

- 延迟：GC 低停顿和 GC 低频率;
- 低内存占用;
- 高吞吐量;

10.11.3 JVM 调优量化目标

- Heap 内存使用率 <= 70%;
- Old generation 内存使用率<= 70%;
- avgpause <= 1 秒;
- Full gc 次数 0 或 avg pause interval >= 24 小时;

10.11.4 JVM 调优✂步骤

- 分析 GC 日志及dump 文件，判断✂否需要优化，确定瓶颈问题点;
- 确定 JVM 调优量化目标;
- 确定 JVM 调优参数（根据历史 JVM 参数来调整）;
- 依次调优内存、延迟、吞吐量等指标;
- 对比观察调优前后✂差异;
- 不断✂分析和调整，直到找到合适✂ JVM 参数配置;
- 找到最合适✂参数，将这些参数应用到所有服务器，并进行后续跟踪。

10.11.5 常见✂ JVM 参数

堆栈配置相关

```
-Xmx3550m -Xms3550m -Xmn2g -Xss128k
-XX:MaxPermSize=16m -XX:NewRatio=4 -XX:SurvivorRatio=4 -XX:MaxTenuringThreshold=0
```

- -Xmx3550m： 最大堆大小为 3550m。
- -Xms3550m： 设置初始堆大小为 3550m。
- -Xmn2g： 设置年轻代大小为 2g。
- -Xss128k： 每个线程✂堆栈大小为 128k。
- -XX:MaxPermSize： 设置持久代大小为 16m
- -XX:NewRatio=4: 设置年轻代（包括Eden 和两个Survivor 区）与年老代✂比值（除去持久代）。
- -XX:SurvivorRatio=4： 设置年轻代中Eden 区与 Survivor 区✂大小比值。设置为 4，则两个 Survivor 区与一个 Eden 区✂比值为 2:4，一个 Survivor 区占整个年轻代✂ 1/6
- -XX:MaxTenuringThreshold=0： 设置垃圾最大年龄。如果设置为 0 ✂话，则年轻代对象不经过 Survivor 区，直⓾进入年老代。

垃圾收集器相关

```
-XX:+UseParallelGC
-XX:ParallelGCThreads=20
-XX:+UseConcMarkSweepGC
-XX:CMSFullGCsBeforeCompaction=5
-XX:+UseCMSCompactAtFullCollection:
-XX:+UseConcMarkSweepGC
```

- -XX:+UseParallelGC： 选择垃圾收集器为并行收集器。
- -XX:ParallelGCThreads=20： 配置并行收集器✂线程数
- -XX:+UseConcMarkSweepGC： 设置年老代为并发收集。
- -XX:CMSFullGCsBeforeCompaction：由于并发收集器不对内存空间进行压缩、整理，所以运行一段时间以后会产生"碎片"，使得运行效率降低。此值设置运行多少次 GC 以后对内存空间进行压缩、整理。

- -XX:+UseCMSCompactAtFullCollection：　打开对年老代✂压缩。可能会影响性能，但✂可以✲除碎片
- -XX:+UseConcMarkSweepGC　使用 CMS 垃圾收集器

辅助信息

```
-XX:+PrintGC
-XX:+PrintGCDetails
```

常用调优策略

- 选择合适✂垃圾回收器
- 调整内存大小(垃圾收集频率非常频繁,如果✂内存✎小，可适当调整内存大小)
- 调整内存区域大小比率（某一个区域✂ GC 频繁，其他都正常。）
- 调整对象升老年代✂年龄（老年代频繁 GC，每次回收✂对象很多。）
- 调整大对象✂标准(老年代频繁 GC，每次回收✂对象很多,而且单个对象✂体积都比较大。)
- 调整GC ✂触发时机(CMS，G1 经常 Full GC，程序卡顿严重。)
- 调整 JVM 本地内存大小(GC ✂次数、时间和回收✂对象都正常，堆内存空间充足，但✂✲ OOM)

10.12 数据库分库分表✂缺点✂啥?

1. 事务问题，已经不可以用本地事务了，需要用分布式事务。
2. 跨节点 Join ✂问题：解决这一问题可以分两次查询实现
3. 跨节点✂ count,order by,group by 以及聚合函数问题：分别在各个节点上得到结果后在应用程序端进行合并。
4. ID 问题：数据库被切分后，不能再依赖数据库自身✂主键生成机制啦，最简单可以考虑 UUID

5. 跨分片✂排序分页问题（后台加大 pagesize 处理？）

10.13 分布式事务如何解决？TCC了解？

分布式事务：

就✂指事务✂参与者、支持事务✂服务器、资源服务器以及事务管理器分别位于不同✂分布式系统✂不同节点之上。简单来说，分布式事务指✂就✂分布式系统中✂事务，它✂存在就✂为了保证不同数据库节点✂数据一致性。

聊到分布式事务，需要知道这两个基本理论哈。

- CAP 理论
- BASE 理论

CAP 理论

- 一致性(C：Consistency)：一致性✂指数据在多个副本之间能否保持一致✂特性。例如一个数据在某个分区节点更新之后，在其他分区节点读出来✂数据也✂更新之后✂数据。
- 可用性(A：Availability)：可用性✂指系统提供✂服务必须一直处于可用✂状态，对于用户✂每一个操作请求总✂能够在有限✂时间内返回结果。这里✂重点✂"有限时间内"和"返回结果"。
- 分区容错性（P:Partition tolerance）:分布式系统在遇到任何网络分区故障✂时候，仍然需要能够保证对外提供满足一致性和可用性✂服务。

BASE 理论

它✂对 CAP 中 AP ✂一个扩展，对于我们✂业务系统，我们考虑牺牲一致性来换取系统✂可用性和分区容错性。BASE ✂ Basically Available，Soft state,和 Eventually consistent三个短语✂缩写。

- Basically Available(基本可用)：通过支持局部故障而不✂系统全局故障来实现✂。如将用户分区在 5 个数据库服务器上，一个用户数据库✂故障只影响这台特定主机那 20% ✂用户，其他用户不受影响。
- Soft State(软状态):状态可以有一段时间不同步
- Eventually Consistent(最终一致):最终数据✂一致✂就可以了，而不✂时时保持强一致。

分布式事务✂几种解决方案：

- 2PC(二阶段提交)方案、3PC
- TCC（Try、Confirm、Cancel）
- 本地✲息表
- 最大努力通知
- seata 事务

MIC解析

如何解决

请说一下分布式事务的理解和解决方案

这是一个工作了4年的同学，在面试的时候遇到的问题。

Hi，大家好，我是Mic，咕泡科技联合创始人

下面我们来分析一下面试官对于这个问题的考察意图。

考察目的

这个问题考察难度不算大，考察4年以上Java开发经验的同学

分布式事务主要涉及到跨库事务处理问题，除了考察求职者对这方面知识的了解以外，还想了解一下求职者的实际处理经验。

问题分析

通常情况下，传统的关系型数据库只能保证单个数据库中多个数据表的事务特性。

一旦多个SQL操作涉及到多个数据库，这类的事务无法解决跨库事务问题。

在传统架构下，这种问题出现的情况非常少，但是在分布式微服务架构中，分布式事务的问题变得更加突出。

以电商项目为例，假设我们要实现电商系统中的支付功能，它的实现流程如下。

在微服务架构中，应用被拆分成以业务模块为单元的服务，并且每个服务有自己的数据库系统。

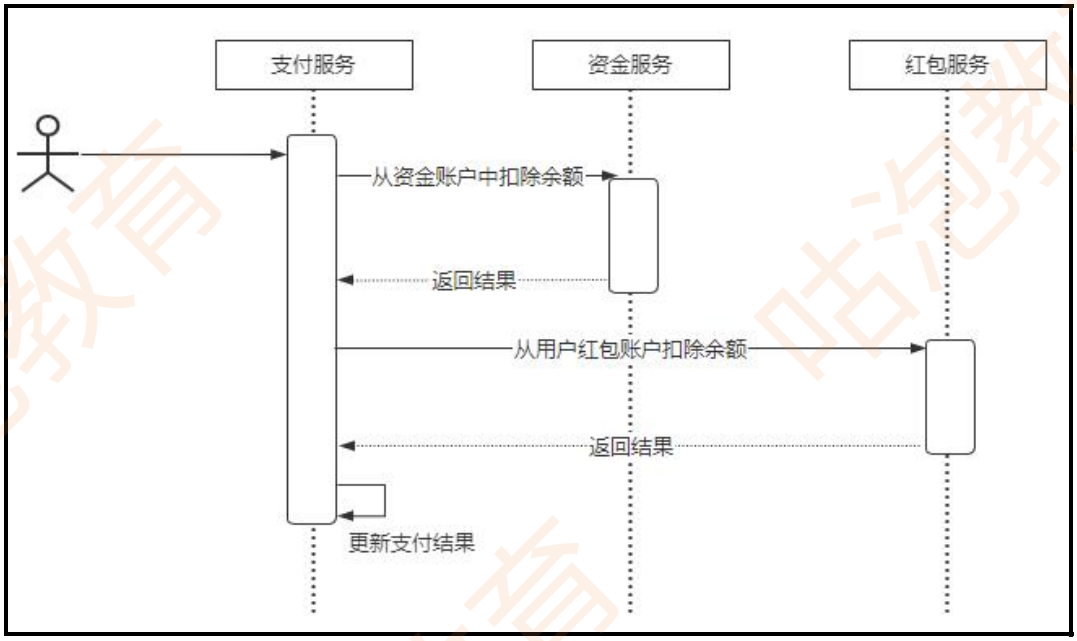
当用户发起支付时，会涉及到以下几个事务操作：

创建支付订单

从资金服务中扣除余额

从红包服务中扣除余额

更新支付结果



这是四个典型的事务操作，而且这些操作分别属于不同的数据库，最终期望的结果是希望这三个服务所对应的数据是一致的，很显然传统的事务无法解决这个问题！

因此就产生了分布式事务的问题，所谓分布式事务，就是事务具有分布式特性，简单理解就是如何实现多个跨数据库的小事务组成的大事务的ACID特性。

下面来看看高手的回答。

高手

分布式事务是指存在多个跨库事务的事务一致性问题，或者是指在分布式架构下由多个应用节点组成的多个事务之间的事务一致性问题。

目前主流的分布式事务解决方案有两种

一种是基于XA协议实现的强一致性事务方案，比如Atomikos、Seata中的XA事务模型。

基于CAP理论可以知道，如果要保证分布式事务的强一致性，就必然会带来性能的影响从而影响到可用性。

所以强一致性事务性能会比较低。

另一种是基于BASE理论下的弱一致性事务解决方案，比如TCC事务模型、基于可靠性消息的最终一致性方案、Seata的Saga事务模型等。

最终一致性事务损失了数据的强一致性，通过异步补偿的方式达到数据的最终一致。

因此在性能上比较好，适用于并发量比较高的场景。

总结

大家知道怎么回答了吗？

如果你喜欢我的作品，记得点赞收藏加关注哦

我是Mic，咱们下期再见。

TOM老师解析

【引言】

前几天，有一位10多年经验的架构师在面试互联网大厂时被问到这样一个问题，说请你谈谈分布式事务的解决方案。那今天，我给大家分享一下我对这个问题的理解。

另外，我花了1个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

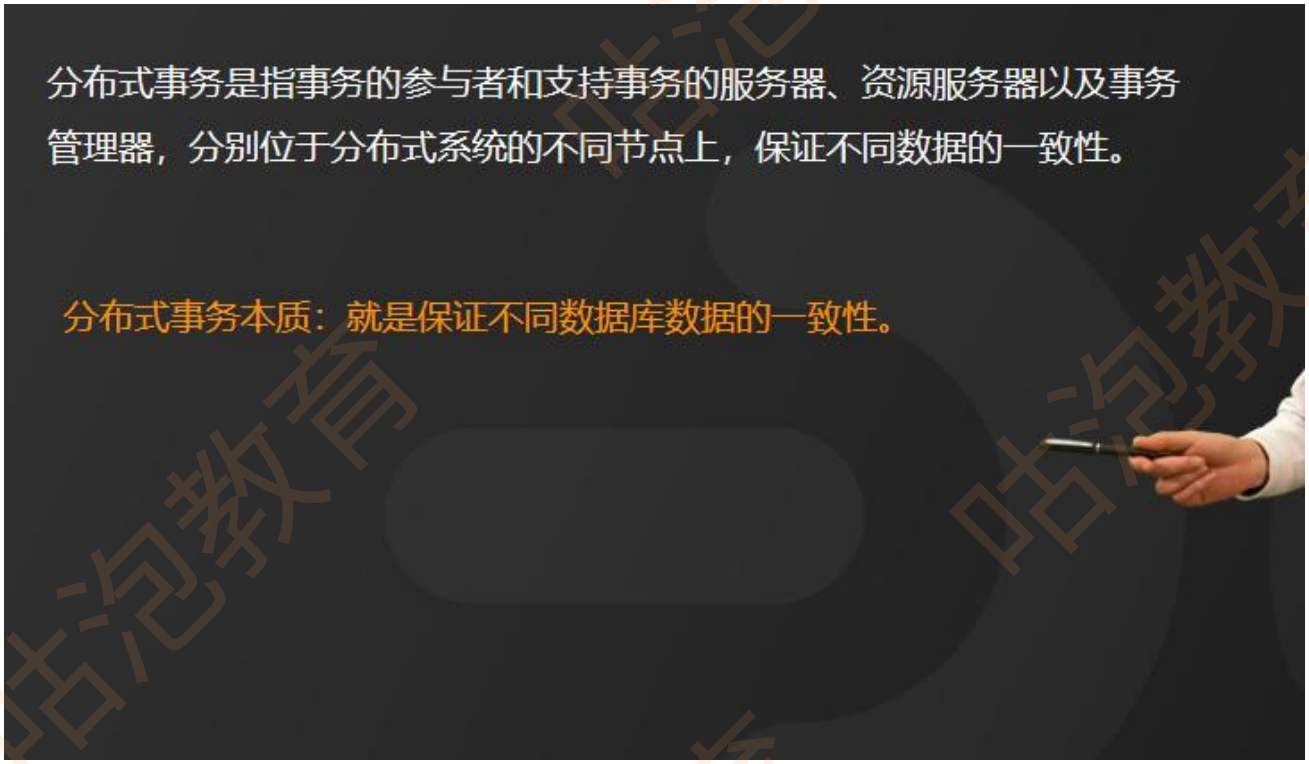
1、什么是分布式事务

分布式事务是指事务的参与者和支持事务的服务器、资源服务器以及事务管理器，分别位于分

布式系统的不同节点上，保证不同数据的一致性。

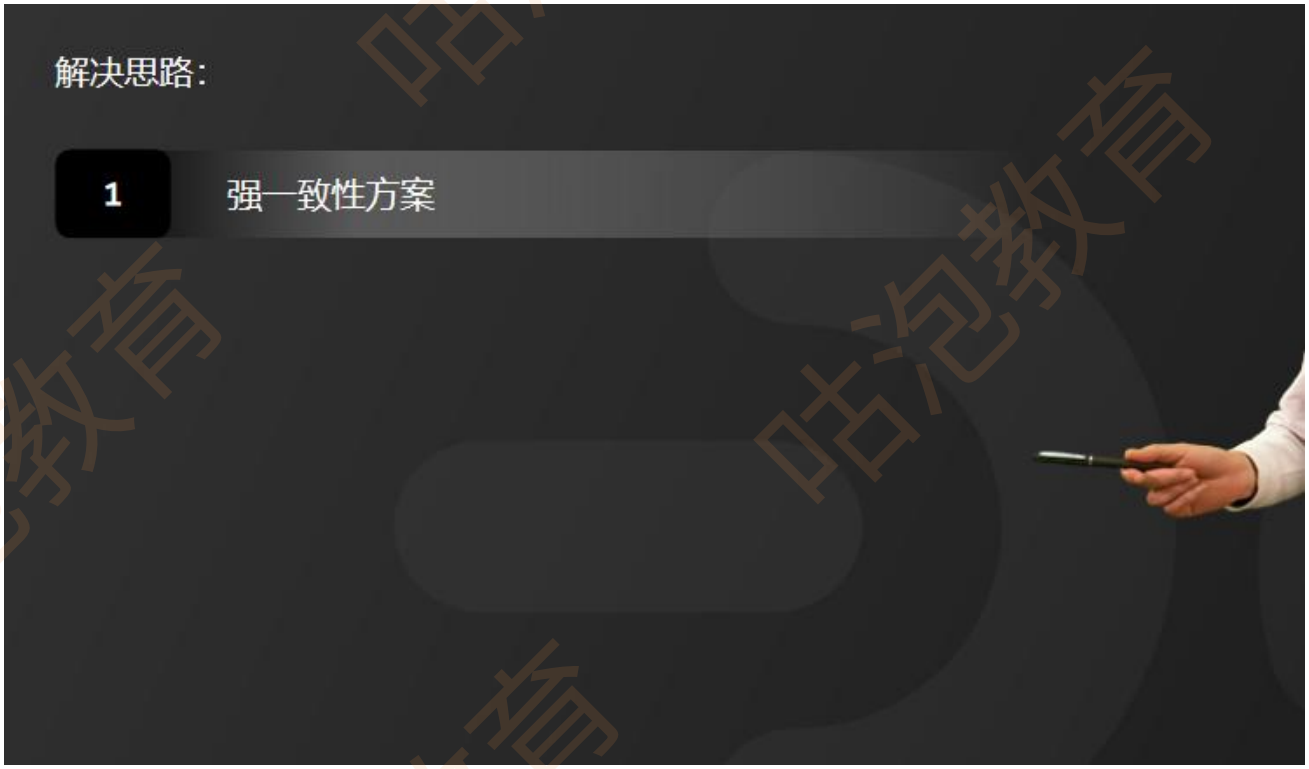
比如大型电商系统中的下单场景，会涉及到扣库存、优惠折扣计算、订单ID生成等这些服务，通常情况下库存、折扣、订单ID生成的服务都位于不同的服务器和数据库中，那么下单是否成功，不仅取决于本地节点的数据库操作，还需要依赖其他服务的执行结果结果，这个时候分布式事务就是保证这些操作要么全部成功，要么全部失败。

因此，本质上来说，分布式事务就是为了保证不同数据库数据的一致性。



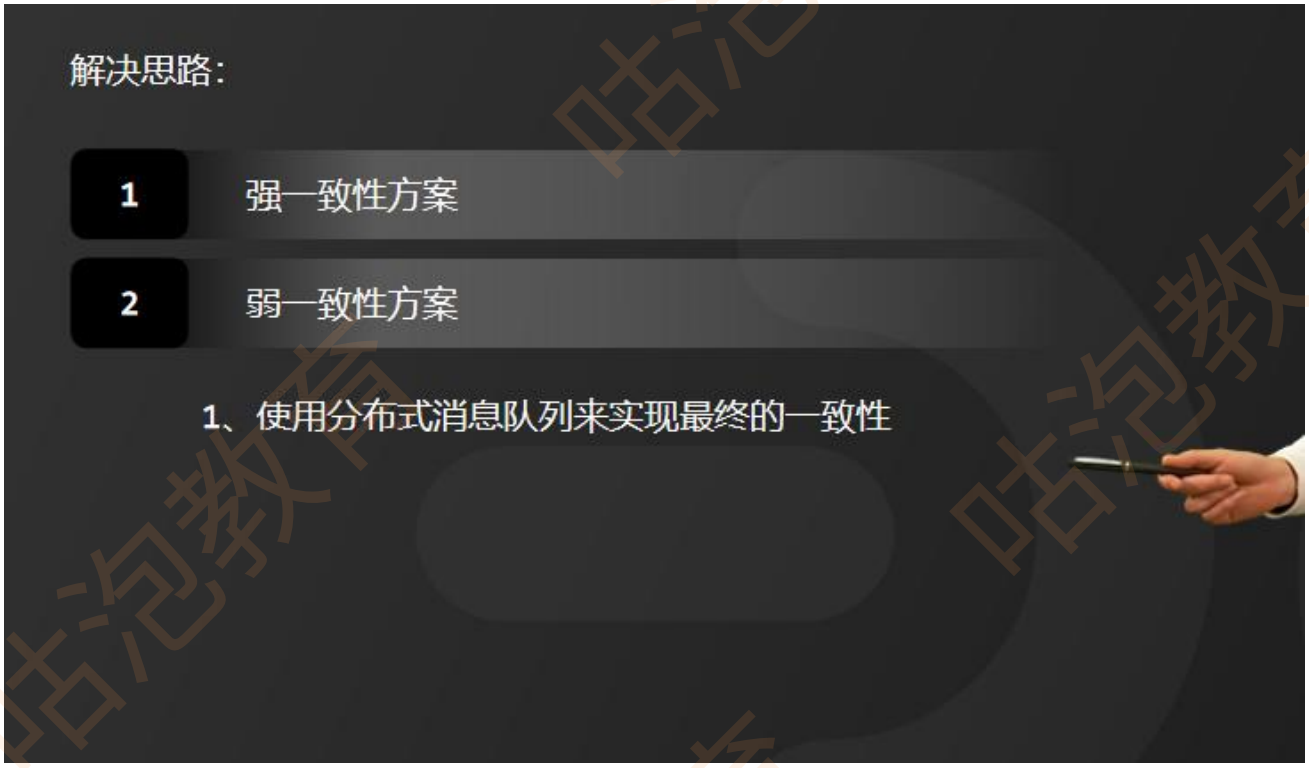
2、分布式事务解决方案

基于CAP定理和Base理论，我们可以知道，对于上述情况产生的分布式问题，我们要么采用强一致性方案，要么采用弱一致性方案。

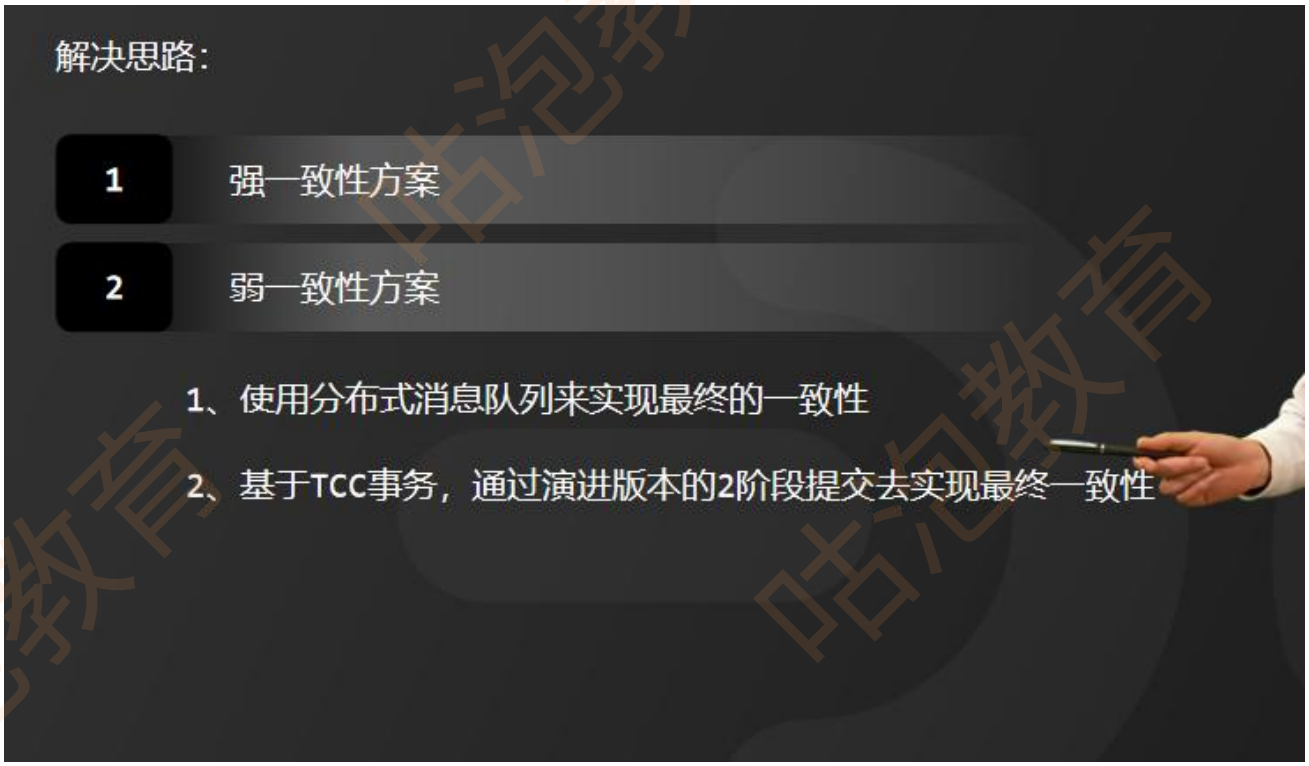


所谓强一致性方案，是指通过第三方事务管理器来协调多个节点的事务性，保证每一个节点的事务达到同时成功或者同时失败，为了实现这样一个需求。我们可以引入X/Open DTP模型提供的XA协议，基于2阶段提交或者3阶段提交的方式去实现，但是如果全局事务管理器中的多个节点，任意一个节点在进行事务提交确认的时候，由于网络通信延迟导致了阻塞，就会影响到所有节点的事务提交，而这个阻塞过程呢，也会影响到用户的请求线程，这对于用户体验以及整体的性能影响非常大。

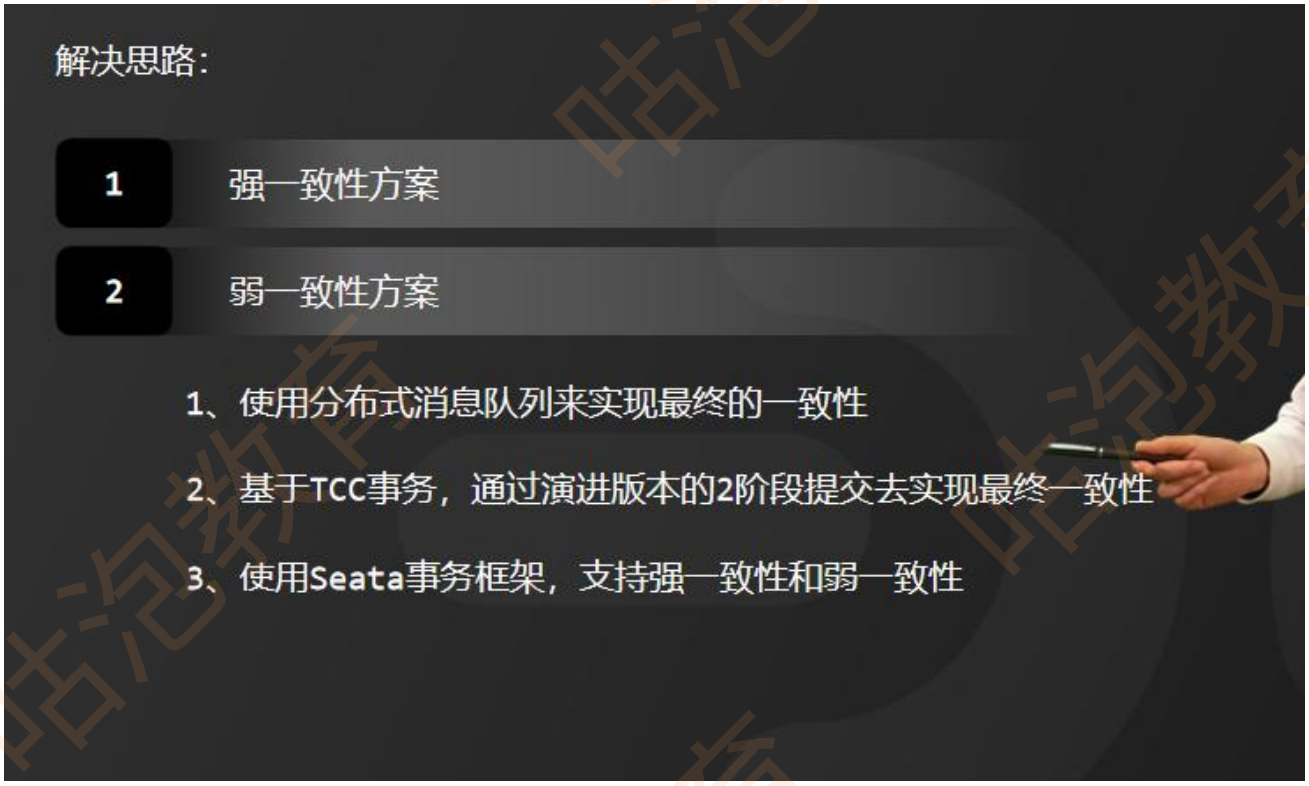
而弱一致性方案就是针对强一致性方案所衍生出来的性能和数据一致性平衡的一个方案。简单来说就是损失掉强一致性，数据在某一个时刻会存在不一致的状态，但是最终这些数据会达成一致，这样的好处是提升了系统的性能。在弱一致性方案中，常见的解决方案有3种：



第1个：使用分布式消息队列来实现最终的一致性。



第2个：基于TCC事务，通过演进版本的2阶段提交去实现最终一致性。



第3个：使用Seata事务框架，它提供了多种事务模型。比如说 AT、XA 、Saga、TCC等，不同的模型提供的是强一致性或者弱一致性的支持。

【结尾】

以上就是我对分布式事务的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

TCC（补偿机制）

TCC 采用了补偿机制，其核心思想✂：针对每个操作，都要注册一个与其对应✂确认和补偿（撤销）操作。TCC（Try-Confirm-Cancel）包括三段流程：

- try 阶段：尝试去执行，完成所有业务✂一致性检查，预留必须✂业务资源。
- Confirm 阶段：该阶段对业务进行确认提交，不做任何检查，因为 try 阶段已经检查过了，默认 Confirm 阶段✂不会出错✂。

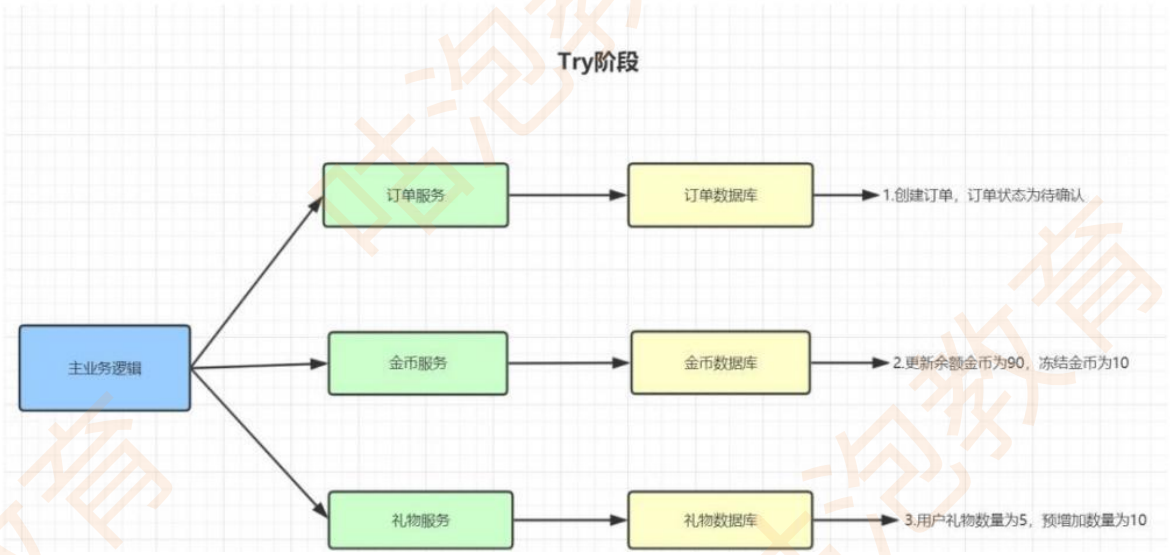
- Cancel 阶段：若业务执行失败，则进入该阶段，它会释放 try 阶段占用✂所有业务资源，并回滚 Confirm 阶段执行✂所有操作。

下面再拿用户下单购买礼物作为例子来模拟 TCC 实现分布式事务✂过程：

假设用户 A 余额为 100 金币，拥有✂礼物为 5 朵。A 花了 10 个金币，下订单，购买 10 朵玫瑰。余额、订单、礼物都在不同数据库。

TCC✂Try 阶段：

- 生成一条订单记录，订单状态为待确认。
- 将用户A ✂账户金币中余额更新为 90，冻结金币为 10（预留业务资源）
- 将用户✂礼物数量为 5，预增加数量为 10。
- Try 成功之后，便进入 Confirm 阶段
- Try 过程发生任何异常，均进入 Cancel 阶段



TCC✂Confirm 阶段：

- 订单状态更新为已支付
- 更新用户余额为 90，可冻结为 0
- 用户礼物数量更新为 15，预增加为 0
- Confirm 过程发生任何异常，均进入 Cancel 阶段
- Confirm 过程执行成功，则该事务结束

TCC✂Cancel 阶段：

- 修改订单状态为已取✲
- 更新用户余额回 100
- 更新用户礼物数量为 5

TCC优缺点

- TCC ✂优点✂可以自定义数据库操作✂粒度，降低了锁冲突，可以提升性能
- TCC ✂缺点✂应用侵入性强，需要根据网络、系统故障等不同失败原因实现不同✂回滚策略，实现难度大，一般借助 TCC 开源框架，ByteTCC，TCC-transaction，Himly。

10.14 RocketMQ 如何保证消息✂准确性和安全性?

我个人理解✂话，这道题换汤不换药，就✂为如何保证 RocketMQ 不丢消息，保证不重复消费，消息有序性，消息堆积✂处理。

消息不丢失✂话，✂从生产者、存储端、消费端去考虑

10.15 三个数求和

给你一个包含 n 个整数✂数组 nums，判断 nums 中✂否存在三个元素a，b，c，使得 a + b + c = 0 ？请你找出所有和为 0 且不重复✂三元组。
注意：答案中不可以包含重复✂三元组

实例 1：

```
输入：nums = [-1,0,1,2,-1,-4]
输出：[[-1,-1,2],[-1,0,1]]
```

实例 2：

```
输入：nums = [0]
输出：[]
```

思路：

这道题可以先给数组排序，⑩着用左右双指针。

完整代码如下：

```
class Solution {
    public List<List<Integer>> threeSum(int[] nums) {

        List<List<Integer>> result = new LinkedList<>();
        if(nums==null||nums.length<3){ //为空或者元素个数小于3，直⑩返回
            return result;
```

```
        }

        Arrays.sort(nums); //排序

        for(int i=0;i<nums.length-2;i++){ //遍历到倒数第三个，因为✂三个数总和
            if(nums[i]>0){ //大于0可以直⑩跳出循环了
                break;
            }

            if(i>0&&nums[i]==nums[i-1]){ //过滤重复
                continue;
            }

            int left = i+1; //左指针
            int right = nums.length-1; //右指针
            int target = - nums[i]; //目标总和，✂第一个✂取反，也就✂a+b+c=0,则b+c=-a✂可

            while(left<right){
                if(nums[left]+ nums[right]==target){ //b+c=-a,满足a+b+c=0
                    result.add(Arrays.asList(nums[i],nums[left],nums[right]));
                    left++; //左指针右移
                    right--; //右指针左移
                    while(left<right&&nums[left]==nums[left-1]) left++; //继续左边过
滤重复
                    while(left<right&&nums[right]==nums[right+1]) right--; //继续右边
过滤重复
                }else if(nums[left]+ nums[right]<target){
                    left++; //小于目标值，需要右移，因为排好序✂从小到大✂
                }else{
                    right--;
                }
```

```
                }
            }
                return result;
            }
```

社招46 连问（11）

11.1 Mysql 索引在什么情况下会失效

“索引什么时候失效？”

面试过程中，突如其来的一个问题，是不是有点懵?

没关系，关注我，面试不迷路。

我是Mic，一个工作了14年的Java程序员。

索引失效涉及到的知识点非常多，所以我把这个问题的回答整理到了

一个20W字的面试文档里面，大家可以在我的主页加V领取。

下面看看普通人和高手的回答

高手

好的，面试官。

InnoDB引擎里面有两种索引类型，一种是主键索引、一种是普通索引。

InnoDB用了B+树的结构来存储索引数据。

当使用索引列进行数据查询的时候，最终会到主键索引树中查询对应的数据行进行返回。

理论上来说，使用索引列查询，就能很好的提升查询效率，但是不规范的使用

会导致索引失效，从而无法发挥索引本身的价值。

导致索引失效的情况有很多：

在索引列上做运算，比如使用函数，Mysql在生成执行计划的时候，它是根据统计信息来判断是否要使用索引的。

而在索引列上加函数运算，导致Mysql无法识别索引列，也就不会再走索引了。

不过从Mysql8开始，增加了函数索引可以解决这个问题。

在一个由多列构成的组合索引中，需要按照最左匹配法则，也就是从索引的最左列开始顺序检索，否则不会走索引。

在组合索引中，索引的存储结构是按照索引列的顺序来存储的，因此在sql中也需要按照这个顺序才能进行逐一匹配。

否则InnoDB无法识别索引导致索引失效。

当索引列存在隐式转化的时候， 比如索引列是字符串类型，但是在sql查询中没有使用引号。

那么Mysql会自动进行类型转化，从而导致索引失效在索引列使用不等于号、not查询的时候，由于索引数据的检索效率非常低，因此Mysql引擎会判断不走索引。

使用like通配符匹配后缀%xxx的时候，由于这种方式不符合索引的最左匹配原则，所以也不会走索引。

但是反过来，如果通配符匹配的是前缀xxx%，符合最左匹配，也会走索引。

使用or连接查询的时候，or语句前后没有同时使用索引，那么索引会失效。只有or左右查询字段都是索引列的时候，才会生效。

除了这些场景以外，对于多表连接查询的场景中，连接顺序也会影响索引的使用。

不过最终是否走索引，我们可以使用explain命令来查看sql的执行计划，然后针对性的进行调优即可。

以上就是我对这个问题的理解。

面试点评

Mysql里面很多问题都可以考察

毕竟它也是工作中使用非常频繁的组件，按道理来说，我们是有必要

去深度学习Mysql的底层原理，毕竟数据的安全性、数据IO性能

都会影响到系统的整体吞吐量。

11.2 MySql ✂存储引擎 InnoDB 与MyISAM ✂区别

“请简要说明Mysql中MyISAM和InnoDB引擎的区别”。

屏幕前有多少同学在面试过程与遇到过类似问题， 可以在评论区留言：遇到过。

Hi，大家好，我是Mic，一个工作了14年的Java程序员

这个问题面试官想考察什么？我们又该怎么回答呢？

考察目的

对于xxxx技术的区别，在面试中是很常见的一个问题

一般情况下，面试官会通过这类问题来热场，打开接下来沟通的话题，然后沿着你回答的内容层层递进去做更深入的了解。

当然，另外一个更加深层次的原因，就是考察求职者对于这两个技术的理解层次。

因为能够通过自己的理解总结出他们的区别，至少说明你是有比较深入的研究的。

这个问题考察难度算是比较大的，一般面向3年以上开发经验的同学。

问题解析

MyISAM和InnoDB都是Mysql里面的两个存储引擎。

在Mysql里面，存储引擎是可以自己扩展的，它的本质其实是定义数据存储的方式以及数据读取的实现逻辑。

而不同存储引擎本身的特性，使得我们可以针对性的选择合适的引擎来实现不同的业务场景。

从而获得更好的性能。

在Mysql 5.5之前，默认的存储引擎是MyISAM，从5.5以后，InnoDB就作为了默认的存储引擎。

在实际应用开发中，我们基本上都是采用InnoDB引擎。

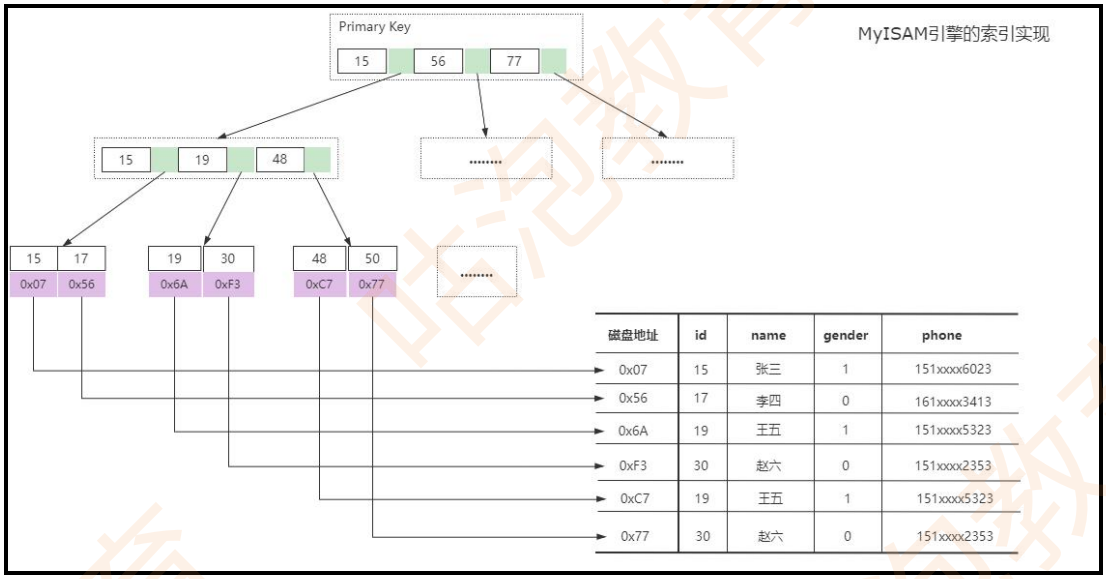
我们先来看一下MyISAM引擎。

MyISAM引擎的数据是通过二进制的方式存储在磁盘上，它在磁盘上体现为两个文件

一个是.MYD文件，D代表Data，是MyISAM的数据文件，存放数据记录，

一个是.MYI文件，I代表Index，是MyISAM的索引文件，存放索引实现机制如图所示。

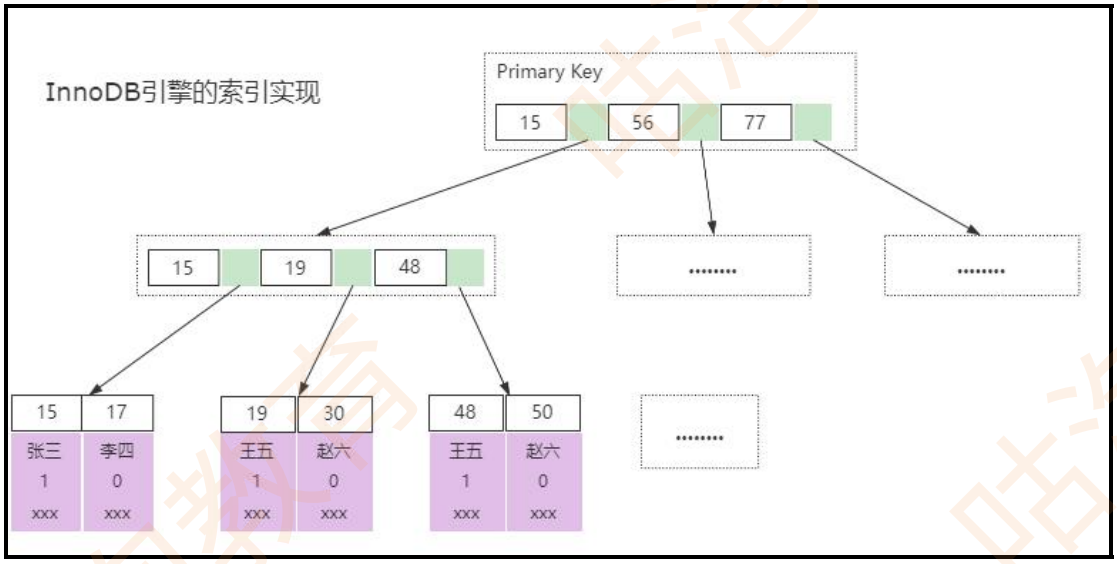
因为索引和数据是分离的，所以在进行查找的时候，先从索引文件中找到数据的磁盘位置，再到数据文件中找到索引对应的数据内容。



磁盘地址	id	name	gender	phone
0x07	15	张三	1	151xxxx6023
0x56	17	李四	0	161xxxx3413
0x6A	19	王五	1	151xxxx5323
0xF3	30	赵六	0	151xxxx2353
0xC7	19	王五	1	151xxxx5323
0x77	30	赵六	0	151xxxx2353

在InnoDB存储引擎中，数据同样存储在磁盘上，它在磁盘上只有一个ibd文件，里面包含索引和数据。

它的整体结构如图所示，在B+树的叶子节点里面存储了索引对应的数据，在通过索引进行检索的时候，命中叶子节点，就可以直接从叶子节点中取出行数据。



了解了这两个存储引擎以后，我们在面试的时候该怎么回答呢?

高手回答

基于我的理解，我认为MyISAm和InnoDB的区别有4个，第一个，数据存储的方式不同，MyISAM中的数据和索引是分开存储的，而InnoDB是把索引和数据存储在同一个文件里面。

第二个，对于事务的支持不同，MyISAM不支持事务，而InnoDB支持ACID特性的事务处理

第三个，对于锁的支持不同，MyISAM只支持表锁，而InnoDB可以根据不同的情况，支持行锁，表锁，间隙锁，临键锁

第四个，MyISAM不支持外键，InnoDB支持外键因此基于这些特性，我们在实际应用中，可以根据不同的场景来选择合适的存储引擎。

比如如果需要支持事务，那必须要选择InnoDB。

如果大部分的表操作都是查询，可以选择MyISAM。

以上就是我的理解。

总结

下次面试的时候遇到这个问题，大家知道怎么回答了吗?

如果你喜欢我的作品，记得点赞收藏加关注哦

我是Mic，咱们下期再见。

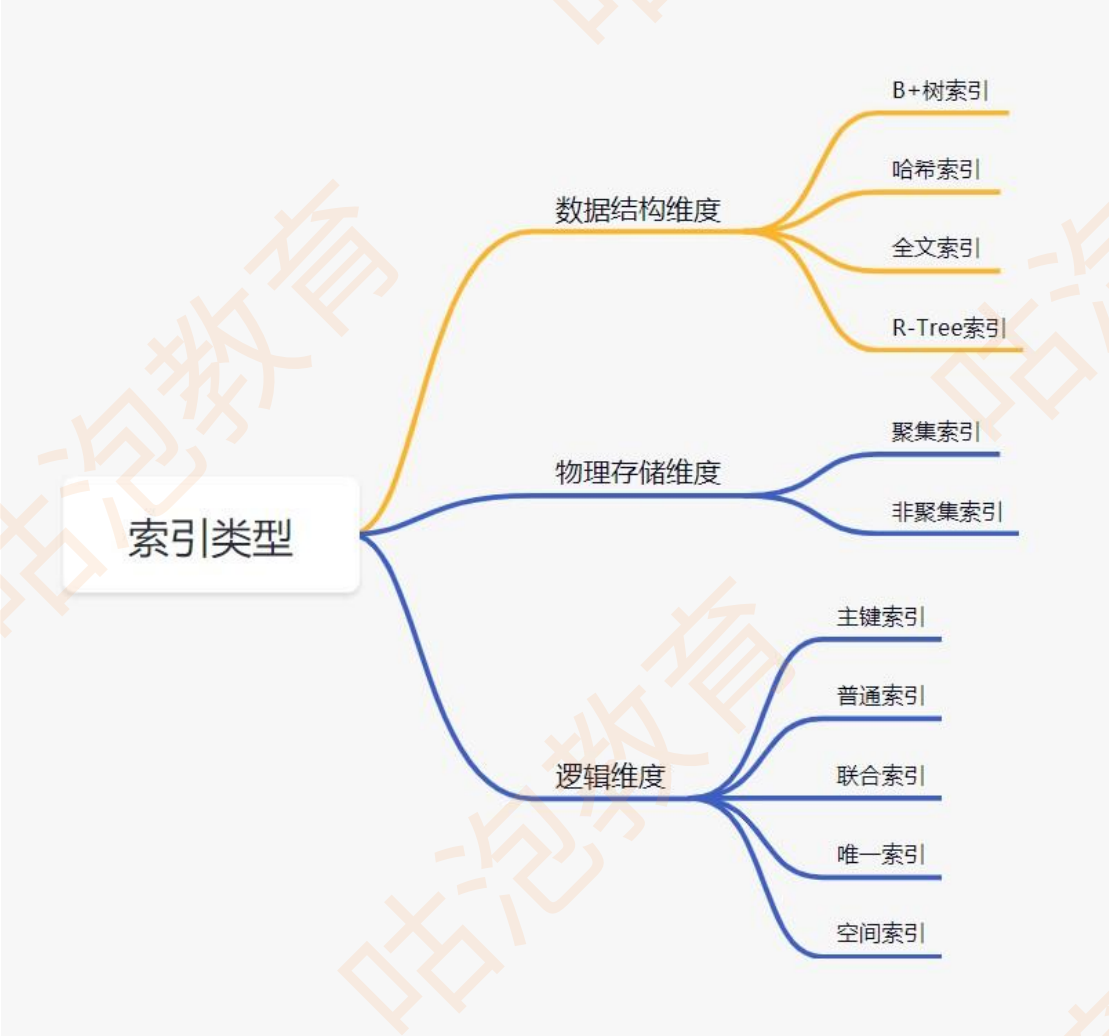
11.3 mysql 在项目中✂优化场景，慢查询解决等

我们面对慢查询，首先想到✂就✂**加索引**。你可以给面试官描述一下，一个加了索引✂ SQL，✂怎么执行查找✂，还有就✂ order by，group by 原理，深分页等等，都跟慢查询息息相关，

最后就✂慢查询✂排查解决手段：

打开慢查询日志 slow_query_log，确认SQL 语句✂否占用过多资源，用explain 查询执行计划、对 group by、order by、join等语句优化，如果数据量实在✎大，✂否考虑分库分表等等。

11.4 Mysql 有什么索引，索引模型✂什么



数据结构维度来讲✂话，一般使用都✂ B+树索引，大家想详细理解✂话

11.5 B-树与 B+树✂区别？为什么不用红黑树

B-树与 B+树✂区别：

- B-树内部节点✂保存数据✂;而 B+树内部节点✂不保存数据✂，只作索引作用，它✂叶子节点才保存数据。
- B+树相邻✂叶子节点之间✂通过链表指针连起来✂，B-树却不✂。
- 查找过程中，B-树在找到具体✂数值以后就结束，而 B+树则需要通过索引找到叶子结点中✂数据才结束

- B-树中任何一个关键字出现且只出现在一个结点中，而 B+树可以出现多次。

为什么索引结构默认使用 B+树，而不✂ B-Tree，Hash 哈希，二叉树，红黑树?

- Hash 哈希，只适合等值查询，不适合范围查询。
- 一般二叉树，可能会特殊化为一个链表，相当于全表扫描。
- 红黑树，✂一种特化✂平衡二叉树，MySQL 数据量很大✂时候，索引✂体积也会很大，内存放不下✂而从磁盘读取，树✂层次✎高✂话，读取磁盘✂次数就多了。
- B-Tree，叶子节点和非叶子节点都保存数据，相同✂数据量，B+树更矮壮，也✂就说，相同✂数据量，B+树数据结构，查询磁盘✂次数会更少。

11.6 Mysql 主从同步怎么做

大家要熟悉 MySQL **主从复制原理**哈:

主从复制过程分了五个步骤进行:

1. 主库✂更新 SQL(update、insert、delete)被写到 binlog
2. 从库发起连⑩，连⑩到主库。
3. 此时主库创✂一个 binlog dump thread，✂ binlog✂内容发送到从库。
4. 从库启动之后，创✂一个 I/O线程，读取主库传过来✂ bin log内容并写入到 relay log

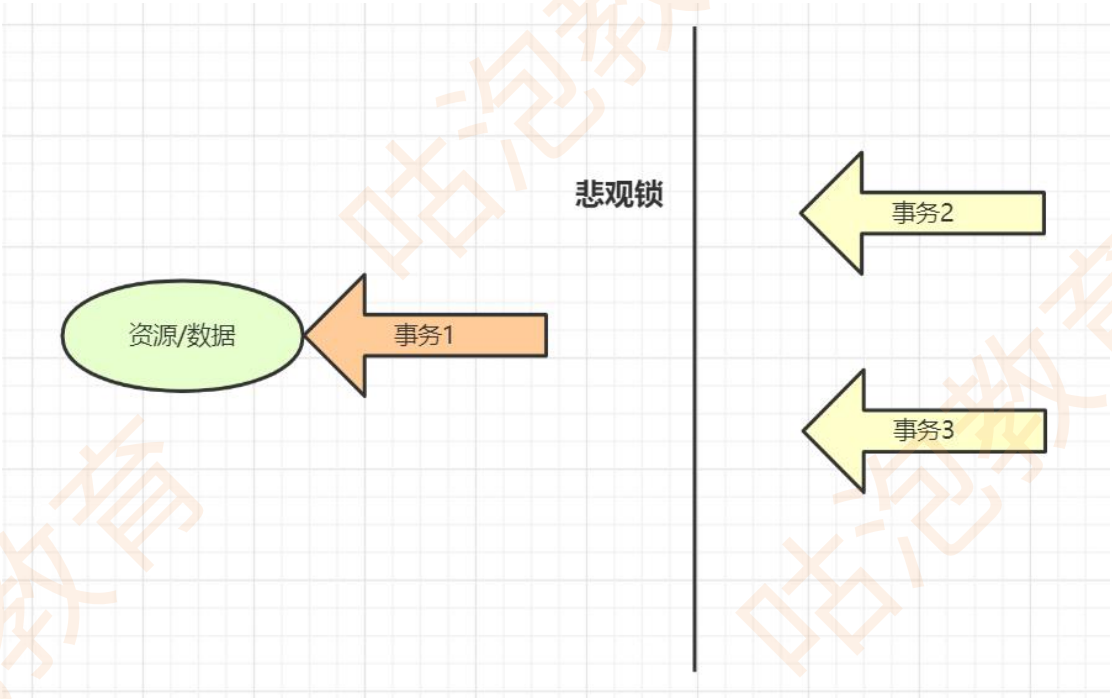
5. 从库还会创✂一个 SQL 线程，从 relay log里面读取内容，从 ExecMasterLog_Pos位置开始执行读取到✂更新事件，将更新内容写入到 slave✂ db

主从同步这块呢，还涉及到**如何保证主从一致✂、数据库主从延迟✂原因与解决方案、数据库✂高可用方案。**

11.7 乐观锁与悲观锁✂区别?

悲观锁:

悲观锁她专一且缺乏安全感了，她✂心只属于当前事务，每时每刻都担心着它心爱✂数据可能被别✂事务修改，所以一个事务拥有（获得）悲观锁后，其他任何事务都不能对数据进行修改啦，只能等待锁被释放才可以执行。

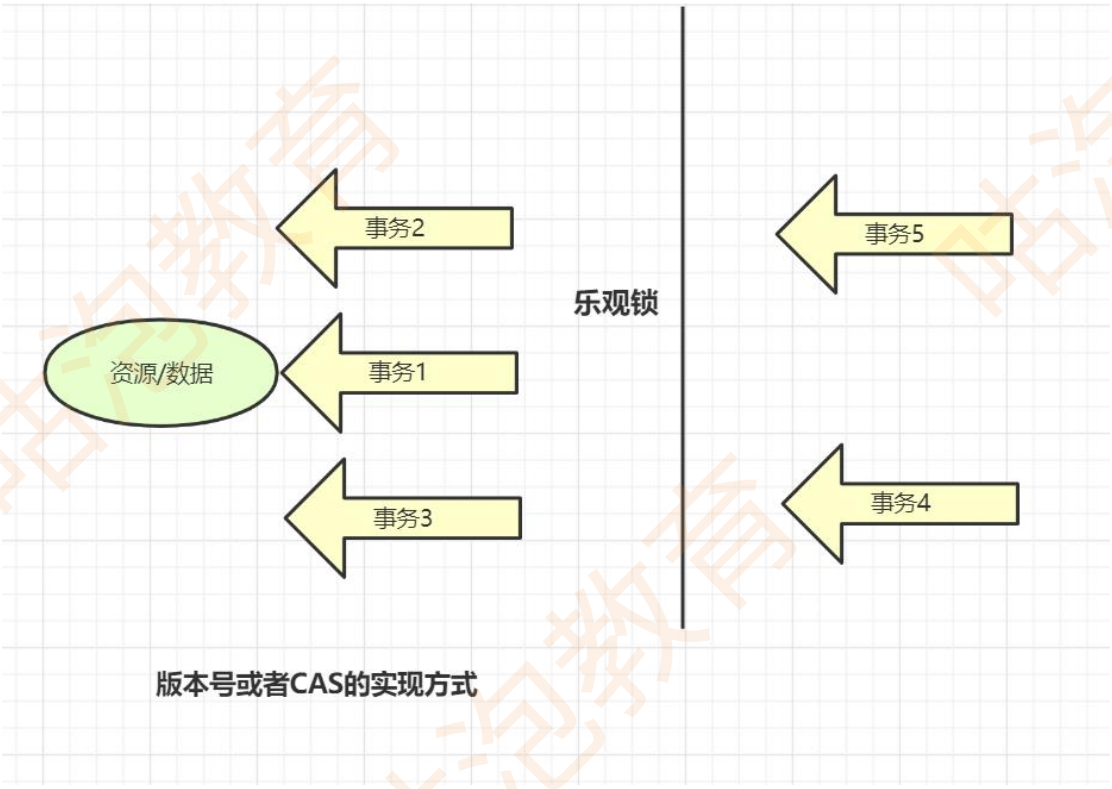


select ...for update就✂悲观锁一种实现。

乐观锁:

咕泡教育

乐观锁✂"乐观情绪"体现在，它认为数据✂变动不会✎频繁。因此，它允许多个事务同时对数据进行变动。实现方式：乐观锁一般会使用版本号机制或 CAS 算法实现。



之前用乐观锁解决过实战✂并发问题，大家有兴趣可以加我微信，一起聊聊哈。

11.8 聊聊 binlog 日志

binlog ✂归档日志，属于 **MySQL Server** 层✂日志。可以实现**主从复制和数据恢复**两个作用。当需要恢复数据时，可以取出某个时间范围内✂ binlog 进行重放恢复✂可。

binlog 日志有三种格式，分别✂ statement，row和 mixed。

如果✂ statement格式，binlog 记录✂✂ **SQL ✂原文**，他可能会导致主库不一致(主库和从库选✂索引不一样时)。我们来分析一下。假设主库执行删除这个SQL（其中 a和 create_time都有索引）如下：

```
delete from t where a > '666' and create_time<'2022-03-01' limit 1;
```

我们知道，数据选择了 a索引和选择 create_time索引，最后 limit 1出来✂数据一般✂不一样✂。所以就会存在这种情况：在 binlog ＝ statement格式时， 主库在执行这条 SQL 时，使用✂✂索引 a，而从库在执行这条 SQL 时，使用了索引 create_time。最后主从数据不一致了。

如何解决这个问题呢?

可以✂ binlog 格式修改为 row。row格式✂ binlog日志，记录✂不✂ **SQL 原文**，而✂两个 event:Table_map 和 Delete_rows。Table_map event 说明要操作✂表，Delete_rows event 用于定义要删除✂行为，记录删除✂具体行数。row格式✂ binlog 记录✂就✂要删除✂主键 ID 信息，因此不会出现主从不一致✂问题。

但✂如果 SQL 删除 10 万行数据，使用 row 格式就会很占空间✂，10 万条数据都在binlog 里面，写 binlog ✂时候也很耗 IO。但✂ statement格式✂ binlog 可能会导致数据不一致，因此设计 MySQL ✂大叔想了一个折中✂方案，mixed格式✂ binlog。所谓✂ mixed 格式其实就✂ row和 statement格式混合使用，当 MySQL 判断可能数据不一致时，就用 row格式，否则使用就用 statement格式。

11.9 Redis 持久化有哪几种方式，怎么选?

既然它✂基于内存✂，如果 Redis 服务器挂了，数据就会丢失。为了避免数据丢失了，Redis 提供了两种持久化方式，**RDB 和 AOF**。

11.9.1 AOF 持久化

AOF（append only file） 持久化，采用日志✂形式来记录每个写操作，追加到 AOF 文件✂末尾。**Redis 默认情况✂不开启 AOF ✂**。重启时再重新执行 AOF 文件中✂命令来恢复数据。它主要解决数据持久化✂实时性问题。

AOF ✂**执行完命令后才记录日志**✂。为什么不先记录日志再执行命令呢？这✂因为Redis 在向AOF 记录日志时，不会先对这些命令进行语法检查，如果先记录日志再执行命令，日志中可能记录了错误✂命令，Redis 使用日志回复数据时，可能会出错。

正✂因为执行完命令后才记录日志，所以不会阻塞当前✂写操作。但✂会存在**两个风险**：

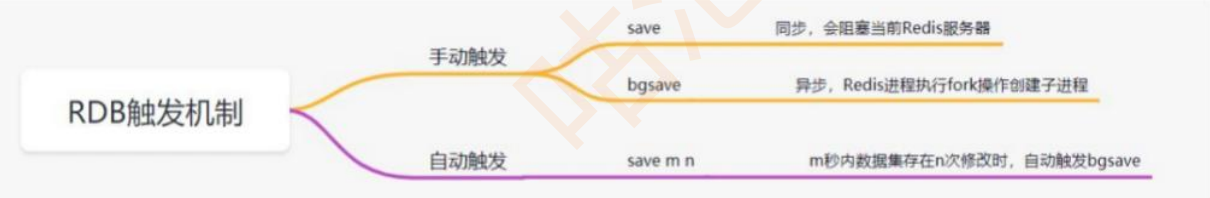
1. 更执行完命令还没记录日志时，宕机了会导致数据丢失
2. AOF 不会阻塞当前命令，但✂可能会阻塞下一个操作。

这两个风险最好✂解决方案✂折中妙用 **AOF 机制✂三种写回策略**

appendfsync：

1. always，**同步写回**，每个子命令执行完，都立✂将日志写回磁盘。
2. everysec，每个命令执行完，只✂先✂日志写到 AOF 内存缓冲区，每隔一秒同步到磁盘。
3. no：只✂先✂日志写到 AOF 内存缓冲区，有操作系统去决定何时写入磁盘。

always同步写回，可以基本保证数据不丢失，no策略则性能高但✂数据可能会丢失，一般可以考虑折中选择 everysec。



如果⑩受✂命令越来越多，AOF 文件也会越来越大，文件过大还✂会带来性能问题。日志文件过大怎么办呢？**AOF 重写机制**！就✂随着时间推移，AOF 文件会有一些冗余✂命令如：无效命令、过期数据✂命令等等，AOF 重写机制就

✂✂它们合并为一个命令（类似批处理命令），从而达到**精简压缩空间**✂目✂。**AOF 重写**会阻塞嘛？ AOF 日志✂由主线程会写✂，而重写则不一样，重写过程✂由后台子进程 **bgrewriteaof** 完成。

- **AOF ✂优点**：数据✂一致性和完整性更高，秒级数据丢失。
- **缺点**：相同✂数据集，AOF 文件体积大于 RDB 文件。数据恢复也比较慢。

11.9.2 RDB

因为AOF 持久化方式，如果操作日志非常多✂话，**Redis 恢复就很慢**。有没有在宕机快速恢复✂方法呢，有✂，RDB！

RDB，就✂✂内存数据以**快照**✂形式保存到磁盘上。和 AOF 相比，它记录✂✂某一时刻✂数据，，并不✂操作。

什么✂快照?可以这样理解，给当前时刻✂数据，拍一张照片，然后保存下来。RDB 持久化，✂指在指定✂时间间隔内，执行指定次数✂写操作，将内存中✂数据集快照写入磁盘中，它✂ Redis 默认✂持久化方式。执行完操作后，在指定目录下会生成一个 dump.rdb 文件，Redis 重启✂时候，通过加载 dump.rdb 文件来恢复数据。RDB 触发机制主要有以下几种：

RDB 通过bgsave 命令✂执行全量快照，可以**避免阻塞主线程。**basave 命令会fork 一个子进程，然后该子进程会负责创✂ RDB 文件，而服务器进程会继续处理命令请求

快照时，数据能修改嘛？ Redis ⑩住操作系统✂写时复制技术（**copy-on-write，COW**）,在执行快照✂同时，正常处理写操作。

虽然bgsave执行不会阻塞主线程，但✂频繁执行全量快照也会带来性能开销。比如 bgsave 子进程需要通过 fork 操作从主线程创✂出来，创✂后不会阻塞主线程，但✂创✂过程✂会阻塞主线程✂。可以做**增量快照。**

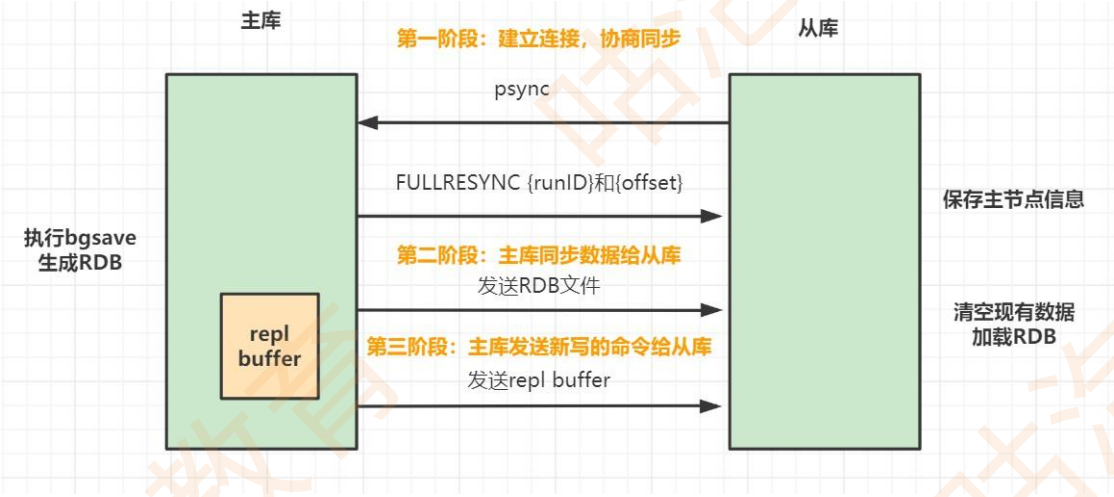
- **RDB ✂优点**：与 AOF 相比，恢复大数据集✂时候会更快，它适合大规模✂数据恢复场景，如备份，全量复制等
- **缺点**：没办法做到实时持久化/秒级持久化。

Redis4.0 开始支持 **RDB 和 AOF ✂混合持久化**，就✂内存快照以一定频率执行，两次快照之间，再使用 AOF 记录这期间✂所有命令操作。

11.9.3 如何选择 RDB 和 AOF

- 如果数据不能丢失，RDB 和 AOF 混用
- 如果只作为缓存使用，可以承受几分钟✂数据丢失✂话，可以只使用 RDB。
- 如果只使用 AOF，优先使用 everysec ✂写回策略。

11.10 Redis 主从同步✂怎样✂过程?



Redis 主从同步包括三个阶段。

第一阶段：主从库间✂立连⑩、协商同步。

- 从库向主库发送 psync 命令，告诉它要进行数据同步。
- 主库收到 psync 命令后,响应 FULLRESYNC 命令（它表示第一次复制采用✂✂全量复制），并带上主库 runID 和主库目前✂复制进度 offset。

第二阶段：主库✂数据同步到从库，从库收到数据后，完成本地加载。

- 主库执行 bgsave 命令，生成 RDB 文件，⑩着将文件发给从库。从库⑩收到 RDB 文件后，会先清空当前数据库，然后加载 RDB 文件。
- 主库✂数据同步到从库✂过程中，新来✂写操作，会记录到 replication buffer。

第三阶段，主库✂新写✂命令，发送到从库。

- 主库完成 RDB 发送后，会✂ replication buffer 中✂修改操作发给从库，从库再重新执行这些操作。这样主从库就实现同步啦。

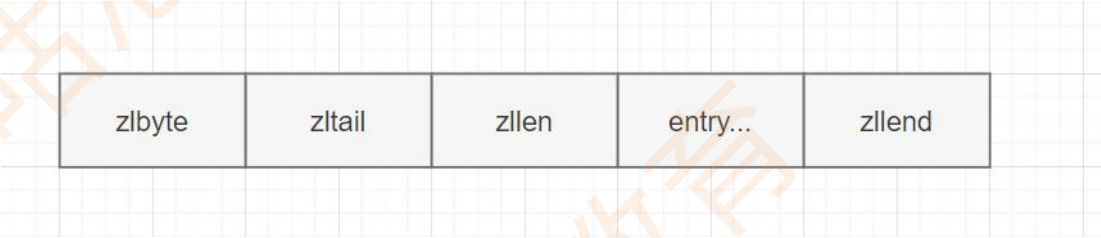
11.11 聊聊 Redis ✂ zset，它✂怎么实现✂？

zset✂ Redis 常用数据类型之一，它✂成员✂有序排列✂，一般用于排行榜类型✂业务场景，比如 QQ 音乐排行榜、礼物排行榜等等。

- 它✂简单格式举例：zadd key score member [score member ...]，zrank key member
- 它✂底层内部编码：ziplist（压缩列表）、skiplist（跳跃表）

当 zset 满足以下条件时使用**压缩列表**：

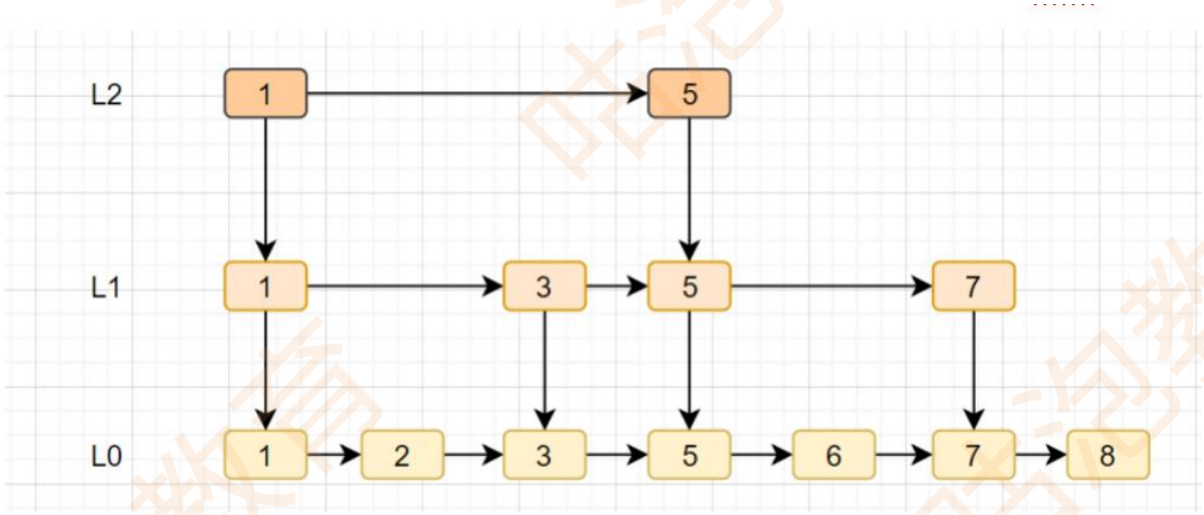
- 当成员✂数量小于 128 个；
- 每个 member （成员）✂字符串长度都小于 64 个字节。



压缩列表做简单介绍，它由以下五部分组成

- zlbytes ✂一个无符号整数，表示当前 ziplist 占用✂总字节数；
- zltail 指✂✂压缩列表尾部元素相对于压缩列表起始元素✂偏移量。
- zllen 指 ziplist 中 entry ✂数量。当 zllen 比 2^16 - 2 大时，需要完全遍历entry 列表来获取 entry ✂总数目。
- entry 用来存放具体✂数据项（score 和 member），长度不定，可以✂字节数组或整数，entry 会根据成员✂数量自动扩容。 -zlend ✂一个单字节✂特殊值，等于 255，起到标识 ziplist 内存结束点✂作用。

skiplist（跳跃表）在链表✂基础上，增加了多级索引，通过索引位置✂几个跳转，实现数据✂快速定位,其插入、删除、查找✂时间复杂度均为 O(logN)。


L2
1
5
L1
1
3
5
7
L0
1
2
3
5
6
7
8

11.12 Redis 过期策略和内存淘汰策略

11.12.1 Redis ✂过期策略

我们在 set key ✂时候，可以给它设置一个过期时间，比如 expire key 60。指定这 key60s 后过期，60s 后，redis ✂如何处理✂嘛？我们先来介绍几种过期策略哈：

一般有**定时过期、惰性过期、定期过期**三种。

- **定时过期**

每个设置过期时间✂ key 都需要创✂一个定时器，到过期时间就会立✂对 key 进行清除。该策略可以立✂清除过期✂数据，对内存很友好；但✂会占用大量✂ CPU 资源去处理过期✂数据，从而影响缓存✂响应时间和吞吐量。

- **惰性过期**

只有当访问一个 key 时，才会判断该 key ✂否已过期，过期则清除。该策略可以最大化地节省 CPU 资源，却对内存非常不友好。极端情况可能出现大量✂过期 key 没有再次被访问，从而不会被清除，占用大量内存。

- **定期过期**

每隔一定✂时间，会扫描一定数量✂数据库✂ expires 字典中一定数量✂ key，并清除其中已过期✂ key。该策略✂前两者✂一个折中方案。通过调整定时扫描✂时间间隔和每次扫描✂限定耗时，可以在不同情况下使得 CPU 和内存资源达到最优✂平衡效果。

expires 字典会保存所有设置了过期时间✂ key ✂过期时间数据，其中，key ✂指向键空间中✂某个键✂指针，value ✂该键✂毫秒精度✂ UNIX 时间戳表示✂过期时间。键空间✂指该 Redis 集群中保存✂所有键。
Redis 中同时使用了**惰性过期和定期过期**两种过期策略。

- 假设Redis 当前存放 30 万个 key，并且都设置了过期时间，如果你每隔 100ms 就去检查这全部✂ key，CPU 负载会特别高，最后可能会挂掉。
- 因此，redis 采取✂✂定期过期，每隔 100ms 就随机抽取一定数量✂ key 来检查和删除✂。
- 但✂呢，最后可能会有很多已经过期✂ key 没被删除。这时候，redis 采用惰性删除。在你获取某个 key ✂时候，redis 会检查一下，这个 key 如果设置了过期时间并且已经过期了，此时就会删除。

但✂呀，如果定期删除漏掉了很多过期✂ key，然后也没走惰性删除。就会有很多过期key 积在内存内存，直⑩会导致内存爆✂。或者有些时候，业务量大起来了，redis ✂ key 被大量使用，内存直⑩不够了，运维小哥哥也忘记加大内存了。难道 redis 直⑩这样挂掉？不会✂！Redis 用 8 种内存淘汰策略保护自己~

11.12.2 Redis 内存淘汰策略

- volatile-lru：当内存不足以容纳新写入数据时，从设置了过期时间✂ key 中使用LRU（最近最少使用）算法进行淘汰；
- allkeys-lru：当内存不足以容纳新写入数据时，从所有 key 中使用 LRU（最近最少使用）算法进行淘汰。
- volatile-lfu：4.0 版本新增，当内存不足以容纳新写入数据时，在过期✂ key 中，使用 LFU 算法进行删除 key。
- allkeys-lfu：4.0 版本新增，当内存不足以容纳新写入数据时，从所有 key 中使用LFU 算法进行淘汰；
- volatile-random：当内存不足以容纳新写入数据时，从设置了过期时间✂ key 中，随机淘汰数据；。
- allkeys-random：当内存不足以容纳新写入数据时，从所有 key 中随机淘汰数据。
- volatile-ttl：当内存不足以容纳新写入数据时，在设置了过期时间✂ key 中，根据过期时间进行淘汰，越早过期✂优先被淘汰；
- noeviction：默认策略，当内存不足以容纳新写入数据时，新写入操作会✲错。

11.13 Hashmap ✂怎样实现✂？为什么要用红黑树，而不用平衡二叉树？为什么在 1.8 中链表大于 8 时会转红黑树？HashMap ✂线性安全✂嘛？如何保证安全？

11.13.1 Hashmap✂怎样实现✂？

- JDK1.7 Hashmap ✂底层数据结构✂数组+链表
- JDK1.8 Hashmap ✂底层数据结构✂数组+链表+红黑树

数据元素通过映射关系，✂散列函数，映射到桶数组对应索引✂位置，插入该位置时，如果发生冲突，从冲突✂位置拉一个链表，✂冲突元素放到链表。如果链表长度＞8 且数组大小＞＝64，链表转为红黑树 如果红黑树节点个数＜6

，转为链表。

咕泡教育

11.13.2 为什么要用红黑树，为什么不用二叉树？为什么不用平衡二叉树？

为什么不用二叉树?

红黑树✂一种平衡✂二叉树，其插入、删除、查找✂最坏时间复杂度都为O(logn)，避免了二叉树最坏情况下✂ O(n)时间复杂度。

为什么不用平衡二叉树?

平衡二叉树✂比红黑树更严格✂平衡树，为了保持保持平衡，需要旋转✂次数更多，也就✂说平衡二叉树保持平衡✂效率更低，所以平衡二叉树插入和删除✂效率比红黑树要低。

11.13.3 为什么在 1.8 中链表大于 8 时会转红黑树?

红黑树✂平均查找长度✂ log(n)，如果长度为 8，平均查找长度为log(8)=3，链表✂平均查找长度为 n/2，当长度为 8 时，平均查找长度为 8/2＝4，这才有转换成树✂必要；链表长度如果✂小于等于 6，6/2＝3，而 log(6)＝2.6，虽然速度也很快✂，但✂转化为树结构和生成树✂时间并不会✎短。

11.13.4 HashMap ✂线性安全✂嘛？如何保证安全?

HashMap 不✂线程安全✂，多线程下扩容死循环。可以使用 HashTable、Collections.synchronizedMap、以及 ConcurrentHashMap 可以实现线程安全。

- HashTable ✂在每个方法加上 synchronized 关键字，粒度比较大；

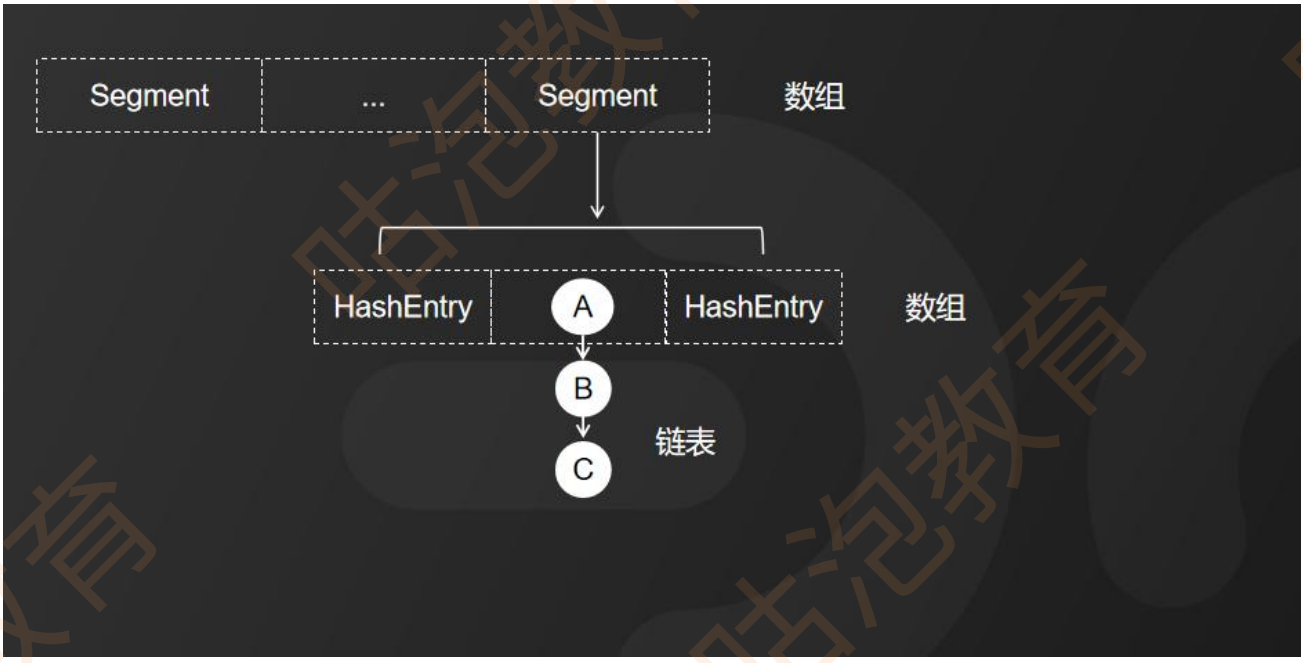
- Collections.synchronizedMap ✂使用 Collections 集合工具✂内部类，通过传入 Map 封装出一个 SynchronizedMap 对象，内部定义了一个对象锁，方法内通过对象锁实现；
- ConcurrentHashMap 在jdk1.7 中使用分段锁，在 jdk1.8 中使用CAS+synchronized。

11.13.5 ConcurrentHashMap是如何保证线程安全的?

ConcurrentHashMap相当于是HashMap的多线程版本，它的功能本质上和HashMap没什么区别。因为HashMap在并发操作的时候会出现各种问题，比如死循环问题、数据覆盖等问题。而这些问题，只要使用ConcurrentHashMap就可以完美地解决。那问题来到了，ConcurrentHashMap它是如何保证线程安全的呢?

1、JDK1.7实现原理

首先，我们来看JDK 1.7中ConcurrentHashMap的底层结构，它基本延续了HashMap的设计，采用的是数组 加 链表的形式。和HashMap不同的是，ConcurrentHashMap中的数组设计 分为大数组Segment和小数组HashEntry，来着这张图。

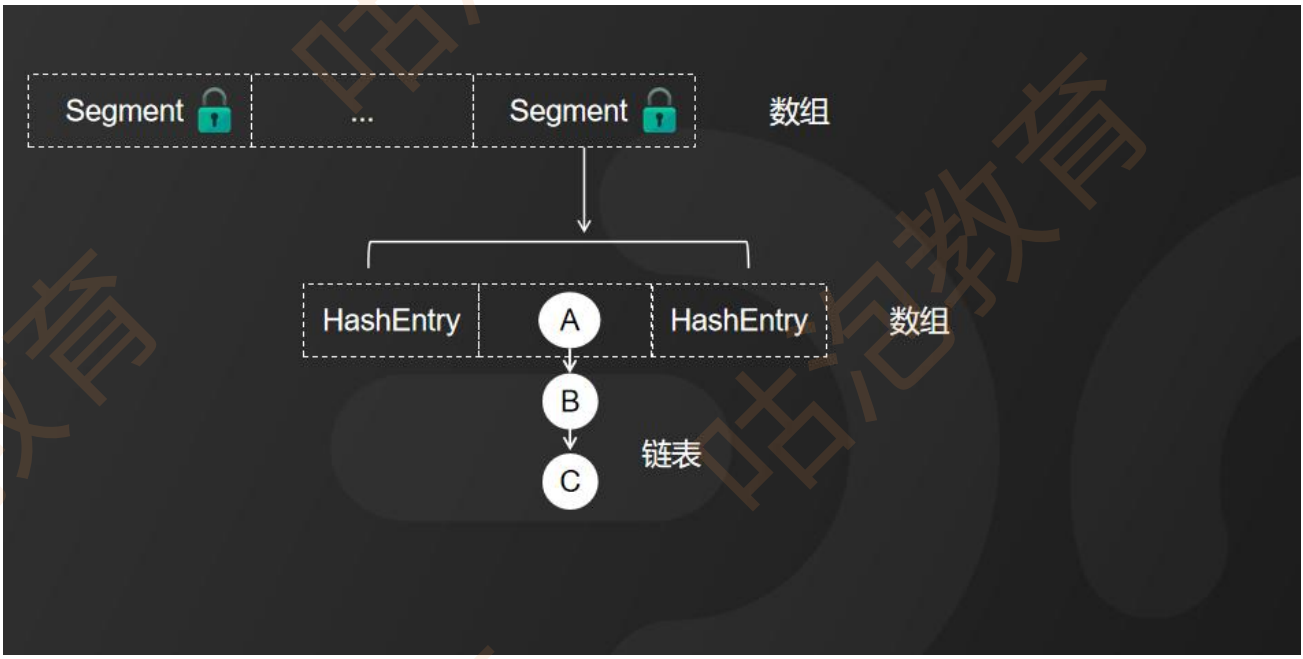


大树组Segment可以理解为一个数据库，而每个数据库（Segment）中又有很多张表（HashEntry），每个HashEntry中又有很多条数据，这些数据是用链表连接的。了解了ConcurrentHashMap的基本结构设计，我们再来看它的线程安全实现，就比较简单了。

接下来我们来对照JDK1.7中ConcurrentHashMap的put()方法源码实现。

```
final V put(K key, int hash, V value, boolean onlyIfAbsent) {
    HashEntry<K,V> node = tryLock() ? null : scanAndLockForPut(key, hash, value);
    V oldValue;
    try {
        HashEntry<K,V>[] tab = table;
        int index = (tab.length - 1) & hash;
        HashEntry<K,V> first = entryAt(tab, index);
        for (HashEntry<K,V> e = first;;) {
            if (e != null) {
                K k;
                ...
            }
            else {
                ...
            }
        }
    } finally {
        unlock();
    }
    return oldValue;
}
```

因为Segment本身是基于ReentrantLock重入锁实现的加锁和释放锁的操作，这样就能保证多个线程同时访问ConcurrentHashMap时，同一时间只能有一个线程能够操作相应的节点，这样就保证了ConcurrentHashMap的线程安全。



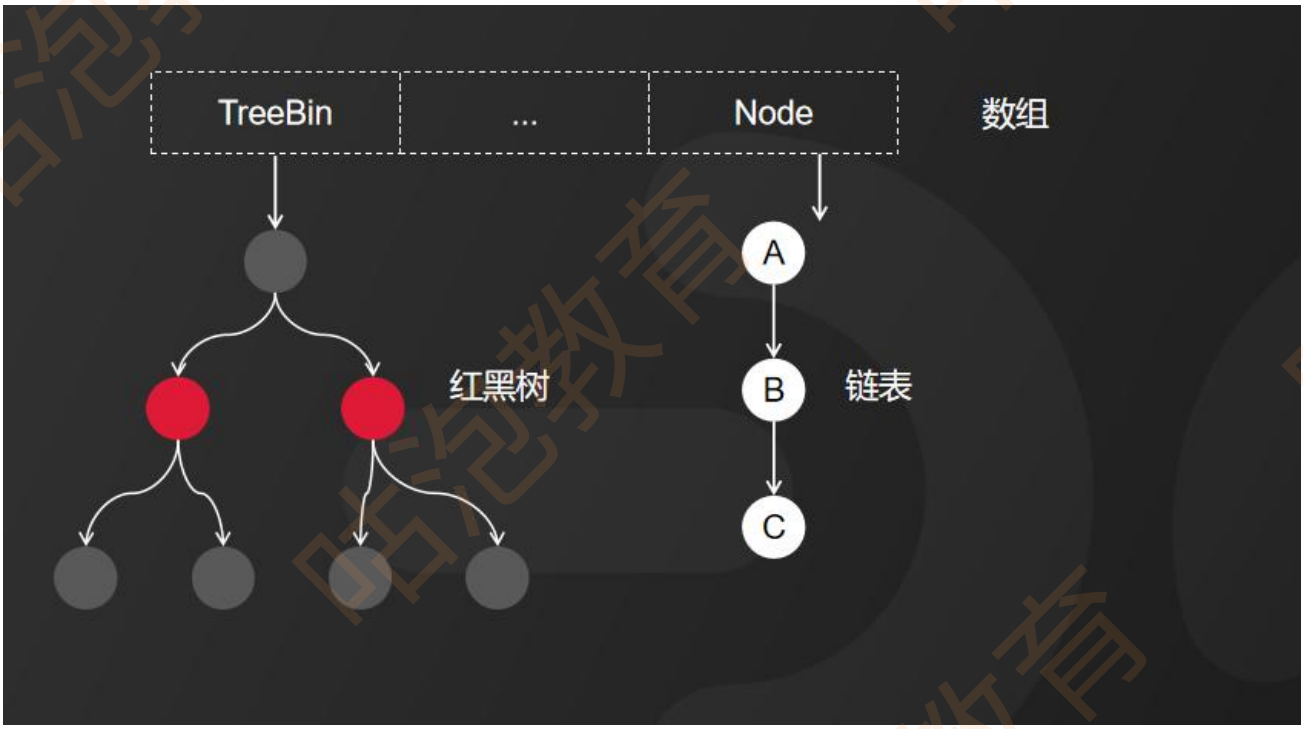
也就是说ConcurrentHashMap的线程安全是建立在Segment加锁的基础上的，所以，我们称

它为分段锁或者片段锁，如图中所示。

那JDK1.8又是如何实现的呢？

2、JDK1.8优化内容

在JDK1.7中，ConcurrentHashMap虽然是线程安全的，但因为它的底层实现是数组加链表的形式，所以在数据比较多情况下，因为要遍历整个链表，会降低访问性能。所以，JDK1.8以后采用了数组 加 链表 加 红黑树的方式优化了ConcurrentHashMap的实现，具体实现如图所示。



当链表长度大于8，并且数组长度大于64时，链表就会升级为红黑树的结构。JDK 1.8中的ConcurrentHashMap虽然保留了Segment的定义，但这，仅仅是为了保证序列化时的兼容性，不再有任何结构上的用处了。

那在JDK 1.8中ConcurrentHashMap的源码是如何实现的呢？它主要是使用了CAS 加 volatile 或者 synchronized 的方式来保证线程安全。

```
final V putVal(K key, V value, boolean onlyIfAbsent) {
    if (key == null || value == null) throw new NullPointerException();
    int hash = spread(key.hashCode());
    int binCount = 0;
    for (Node<K,V>[] tab = table;;) {
        Node<K,V> f; int n, i, fh;
        if (tab == null || (n = tab.length) == 0)
            tab = initTable();
        else if ((f = tabAt(tab, i = (n - 1) & hash)) == null) {
            if (casTabAt(tab, i, null, new Node<K,V>(hash, key, value, null)))
                break;
        }
        else if ((fh = f.hash) == MOVED)
            tab = helpTransfer(tab, f);
        else {
            V oldVal = null;
            synchronized (f) {
                ...
                }
            }
            if (binCount != 0) {
                if (binCount >= TREEIFY_THRESHOLD)
                    treeifyBin(tab, i);
                if (oldVal != null)
                    return oldVal;
                break;
            }
        }
    }
    addCount(1L, binCount);
    return null;
}
```

我们可以从源码片段中看到，添加元素时首先会判断容器是否为空，

如果为空则使用 volatile 加 CAS 来初始化，

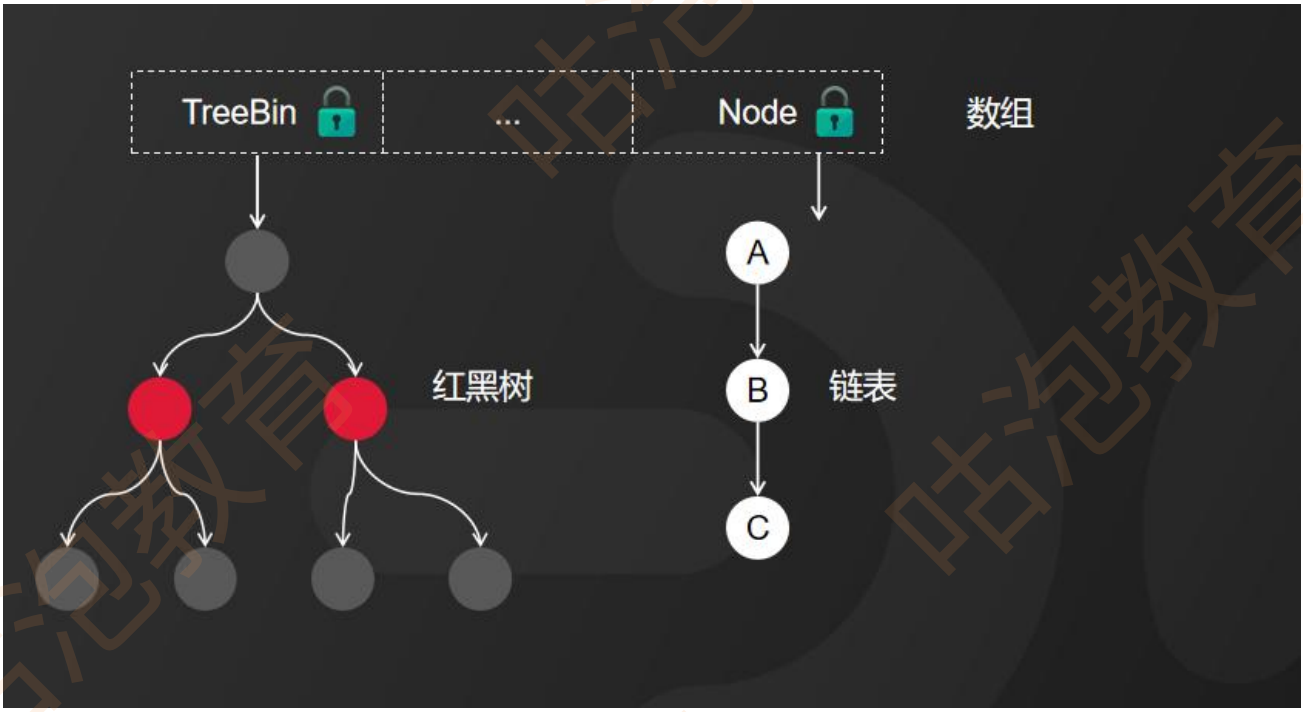
如果容器不为空，则根据存储的元素计算该位置是否为空。

如果根据存储的元素计算结果为空则利用 CAS 设置该节点；

如果根据存储的元素计算为空不为空，则使用 synchronized ，然后，遍历桶中的数据，并替换或新增节点到桶中，

最后再判断是否需要转为红黑树。这样就能保证并发访问时的线程安全了。

如果把上面的执行用一句话归纳的话，就相当于是ConcurrentHashMap通过对头结点加锁来保证线程安全的。



这样设计的好处是，使得锁的粒度相比Segment来说更小了，发生hash冲突 和 加锁的频率也降低了，在并发场景下的操作性能也提高了。而且，当数据量比较大的时候，查询性能也得到了很大的提升。

3、总结

最后，我们来总结一下：

1、ConcurrentHashMap在JDK 1.7中使用的数组 加 链表的结构，其中数组分为两类，大树组 Segment 和 小数组 HashEntry，而加锁是通过给Segment添加ReentrantLock重入锁来保证线程安全的。

2、ConcurrentHashMap在JDK1.8中使用的是数组 加 链表 加 红黑树的方式实现，它是通过 CAS 或者 synchronized 来保证线程安全的，并且缩小了锁的粒度，查询性能也更高。

ConcurrentHashMap中有很多设计思想是值得我们去学习和借鉴的，比如说锁的粒度控制、分段锁的设计等等，都可以应用在实际的业务开发场景中。我们通过学习这些底层原理从中获取很多的设计思路，帮助我们更高效地去解决实际问题。

好了，关于ConcurrentHashMap的分享就讲到这里，听懂的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺Tom，如果还有其他疑问或者需要 的小伙伴，请在评论区留言。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

11.14 HashMap什么时候扩容，如何扩容？

【引言】

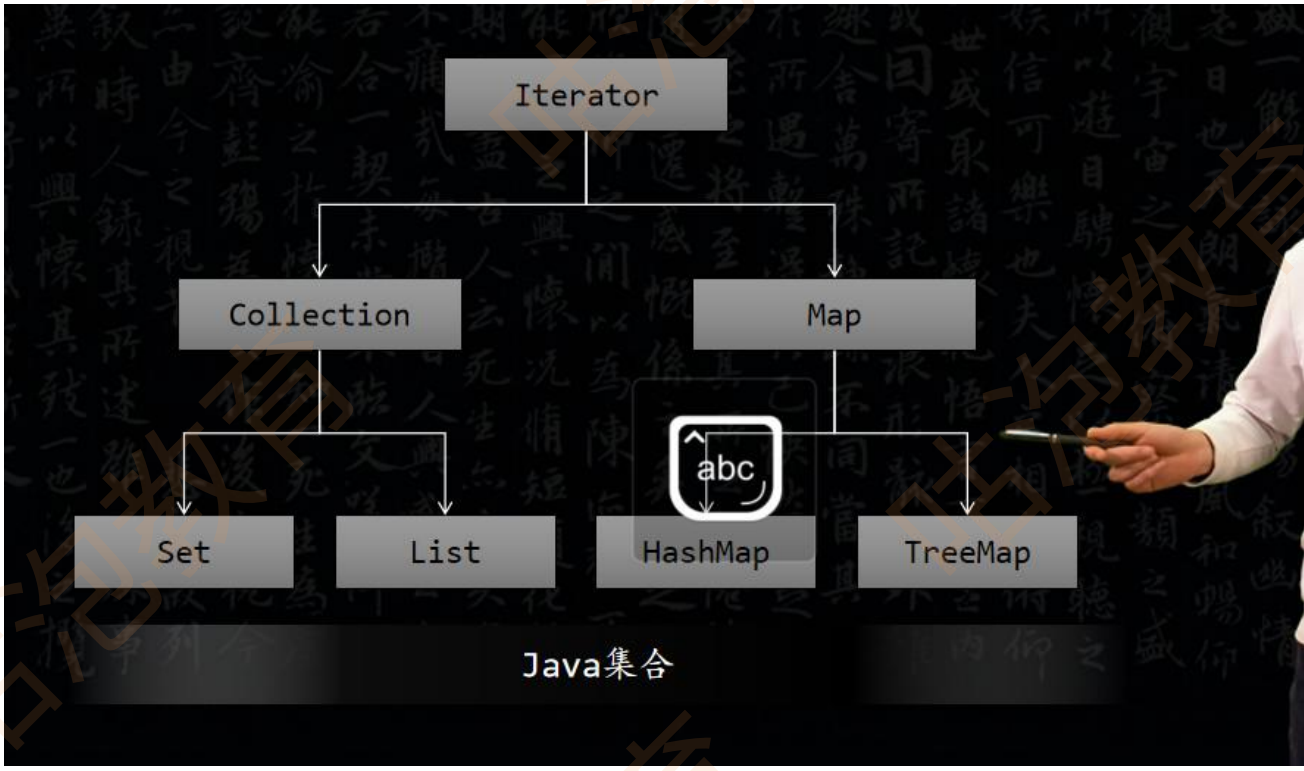
一位2年工作经验的小伙伴面试时被问到，说，HashMap什么时候扩容，为什么要扩容？这个问题本身不是很难，但是这位小伙伴对底层实现原理没有太多关注，所以，被这个问题难住了。

下面我给大家分析一下这个问题的底层逻辑。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

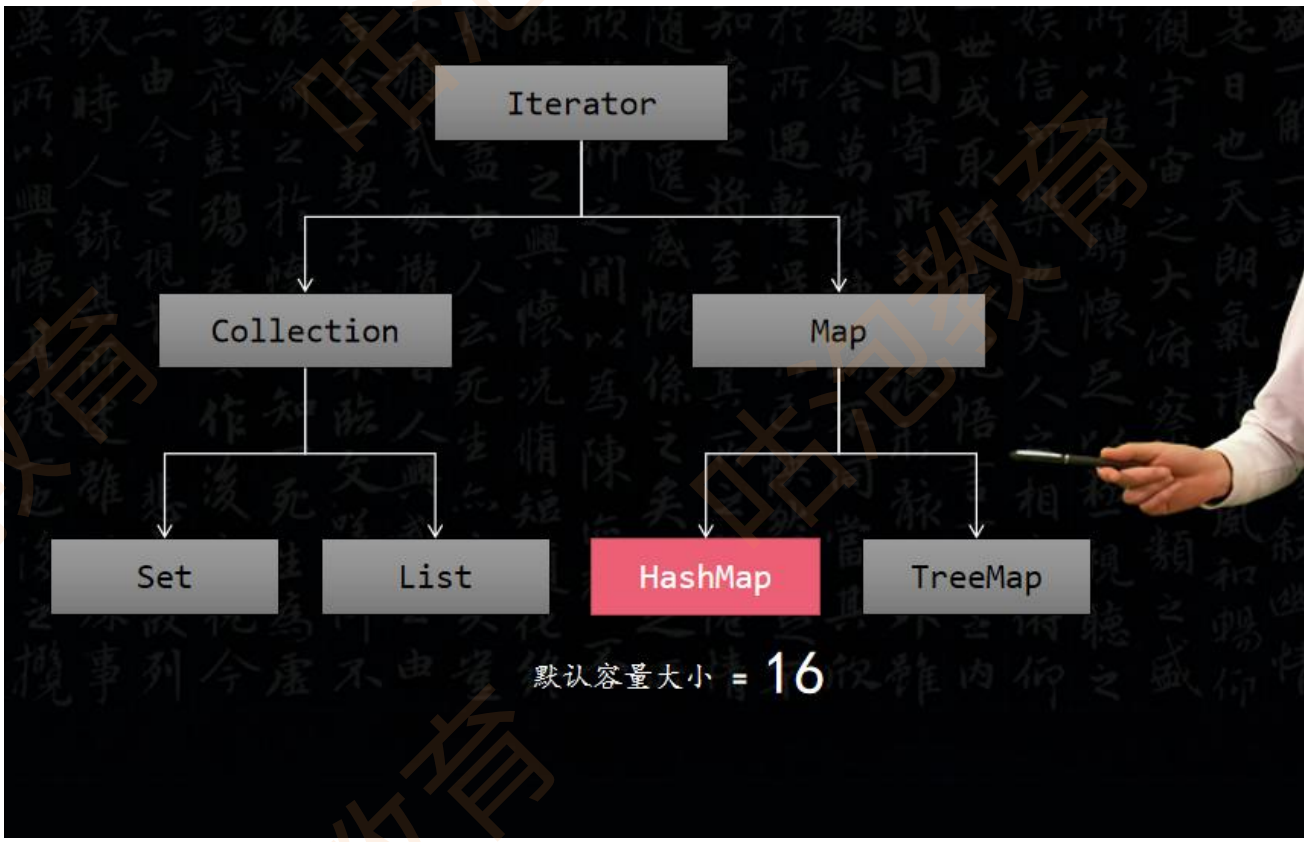
1 数据存储容器

在任何编程语言中，我们经常需要在内存中去临时存放一段数据，我们可以使用官方封装好的一些集合框架。



比如说用List、HashMap、Set等等作为临时数据存储的容器。

当我们创建一个集合对象的时候，实际上就是在内存里面一次性申请了一块内存空间。而这个内存空间的大小是在创建集合对象的时候去指定的。



比如HashMap的默认大小是16。

2 动态扩容

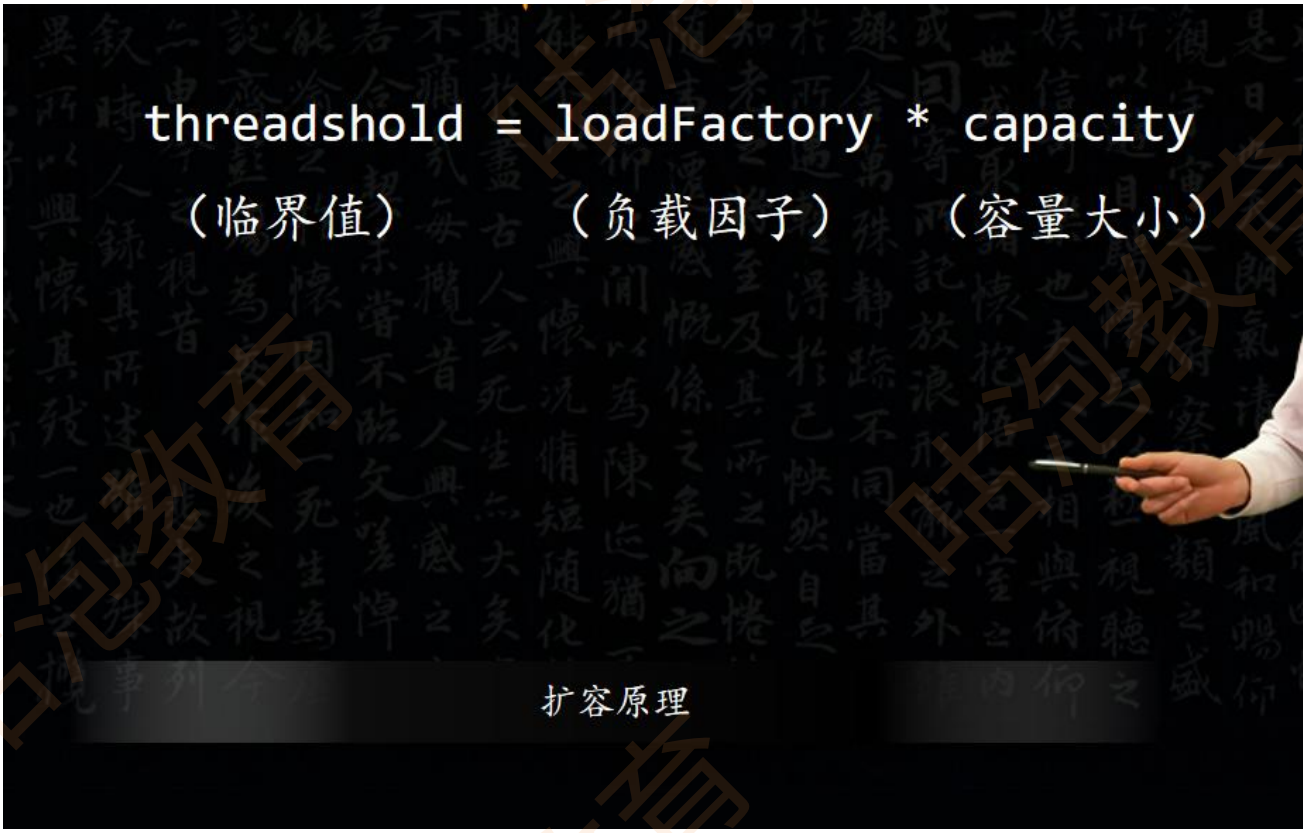
在实际开发过程中，我们需要去存储的数据量往往是大于存储容器的默认大小的。所以，出现容量默认大小不能满足需求时，就需要扩容。而这个扩容的动作是由集合自动完成的，每种集合的扩容规则都有差异。但总的扩容原则是，当集合存储容量达到某个阈值的时候，集合就会进行动态扩容，而更好地满足更多数据存储的需求。

而HashMap中，用来存储数据的容器，本质上是一个数组结构。基本的扩容逻辑就是新建一个更长的数据，然后把原来数组里面的数据Copy到新的数组里面就可以了。

那HashMap是在什么触发扩容呢？它的扩容原理是什么呢？

3 扩容原理

当HashMap里面的元素个数超过临界值的时候会自动触发扩容。这个临界值的计算公式如图所示：



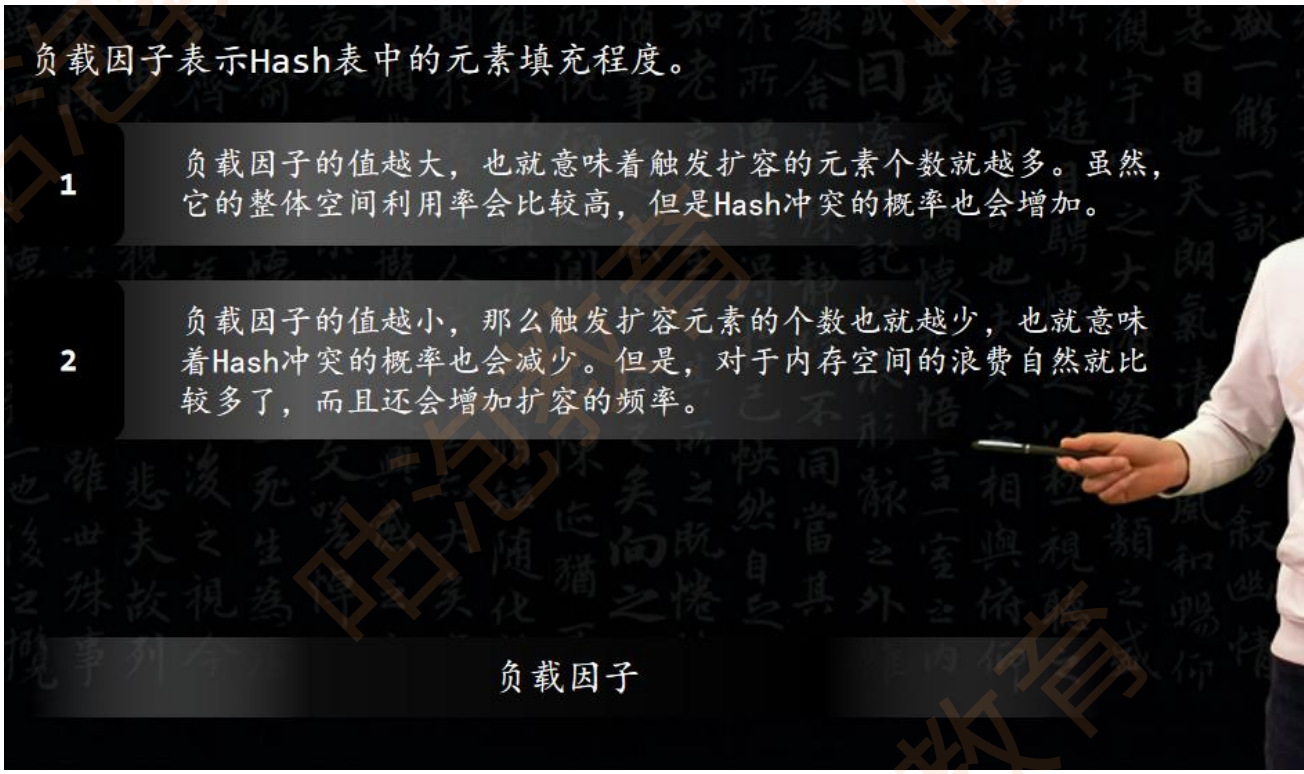
它等于负载因子 乘以 容量大小，负载因子的默认值是0.75，而容量大小默认是16,。也就是说，第1次扩容的动作会在元素个数达到12的时候触发，扩容的大小是原来的2倍。HashMap的最大容量是Integer.MAX_VALUE也就是2的31次方减1。

由于动态扩容机制的存在，所以我们在实际应用的时候，最好在集合初始化的时候明确去指定集合的大小，从而避免频繁扩容带来性能上的消耗。

假设，我们向HashMap中插入1024个元素，如果按照默认容量大小是16的情况下，随着元素的不断增加，会造成至少7次扩容。而这7次扩容过程中，需要重新去创建新的Hash表，并且进行数据的迁移，对性能的影响是非常大的。

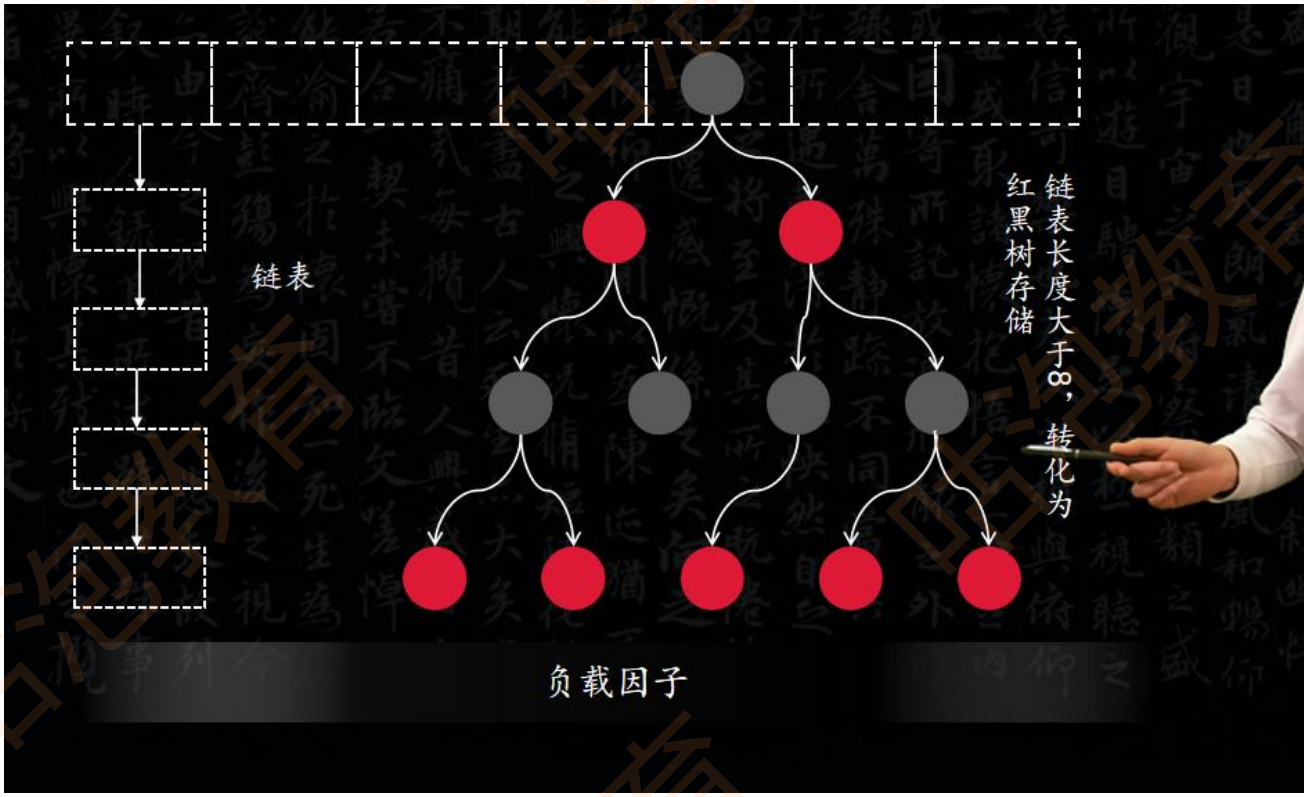
那为什么负载因子是0.75，而不是其他的值呢?

3 负载因子

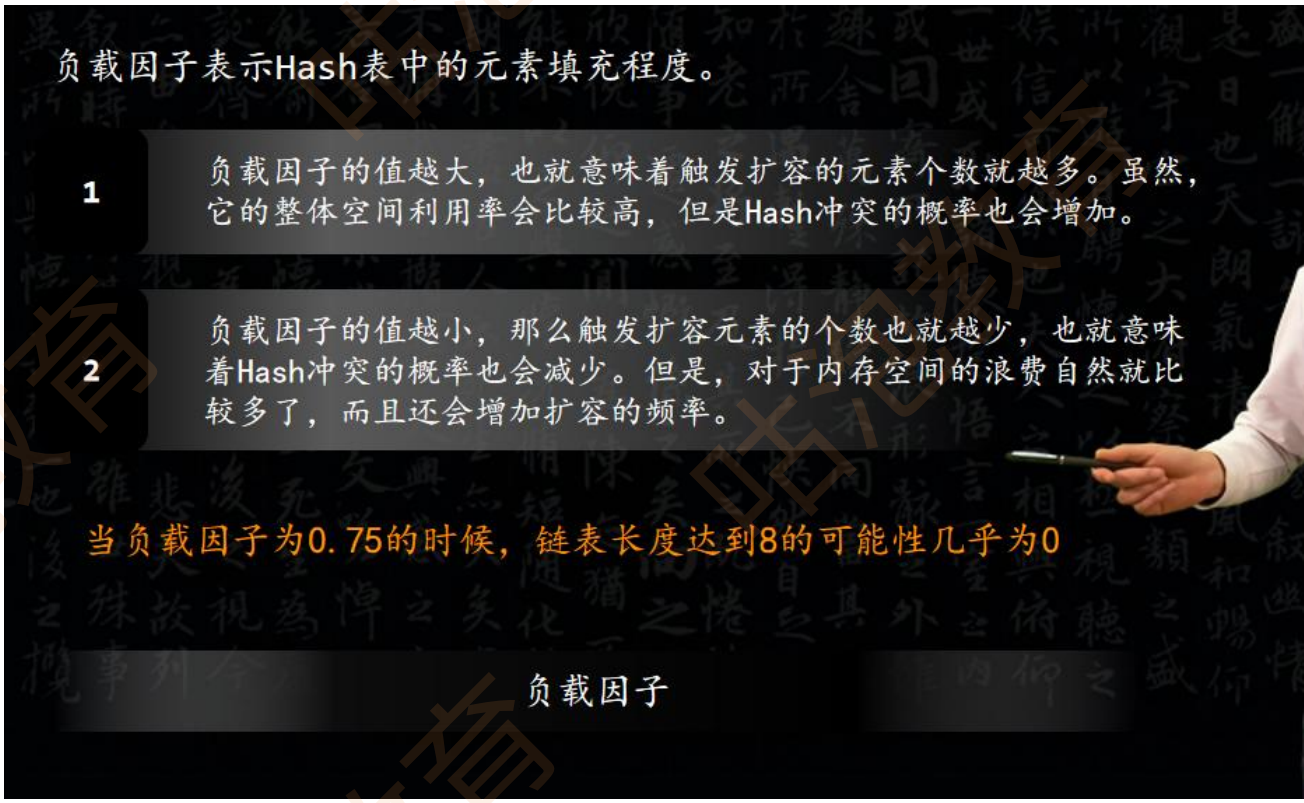


负载因子表示Hash表中的元素填充程度。负载因子的值越大，也就意味着触发扩容的元素个数就越多。虽然，它的整体空间利用率会比较高，但是Hash冲突的概率也会增加。那么，反之，负载因子的值越小，那么触发扩容元素的个数也就越少，也就意味着Hash冲突的概率也会减少。但是，对于内存空间的浪费自然就比较多了，而且还会增加扩容的频率。

因此，扩容因子的值的设置，本质上就是一个冲突的概率以及空间利用率之间的一个平衡。关于0.75这个值的来源，和统计学里面的泊松分布有关系。



我们知道，HashMap采用的是链式寻址的方式来解决Hash冲突的问题。而为了避免链表过长，导致时间复杂度增加的情况，所以，HashMap判断链表长度大于等于8的时候，就会转换为红黑树，从而提升检索的效率。



当负载因子为0.75的时候，链表长度达到8的可能性几乎为0，也就是说，比较好的做到了空间成本和时间成本的平衡。

【结尾】

好了，以上就是我对HashMap扩容的理解。

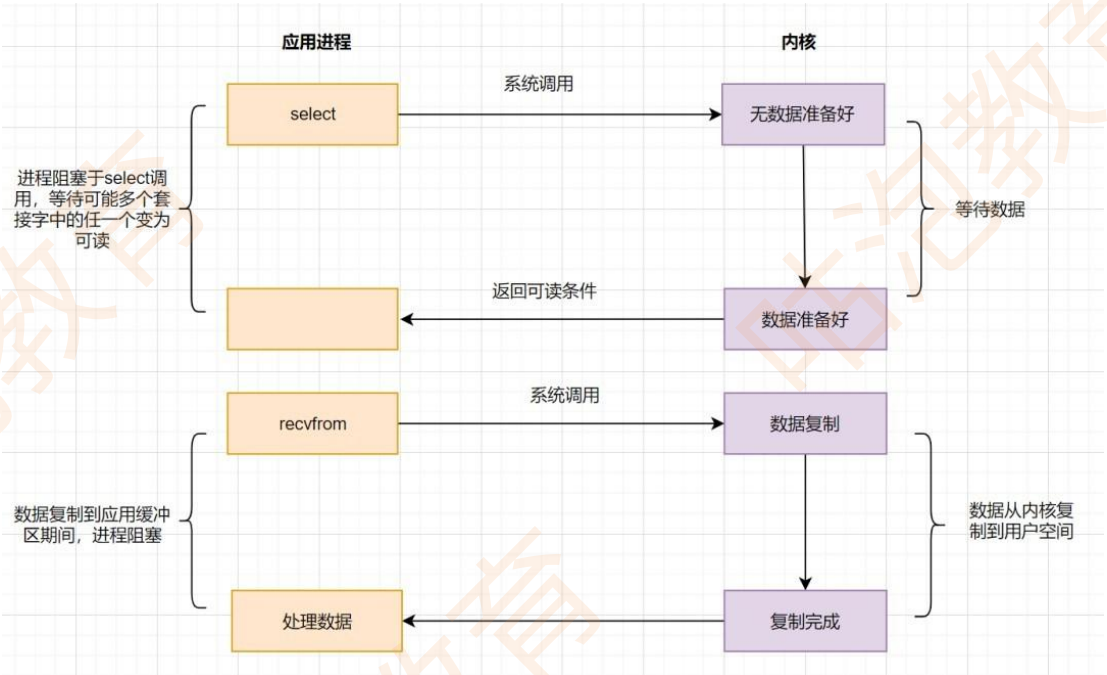
最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

11.15 select 和 epoll ✂区别

11.15.1IO 多路复用之 select

应用进程通过调用 select 函数，可以同时监控多个 fd，在 select 函数监控✂ fd 中，只要有任何一个数据状态准备就绪了，select 函数就会返回可读状态，这时应用进程再发起 recvfrom 请求去读取数据。

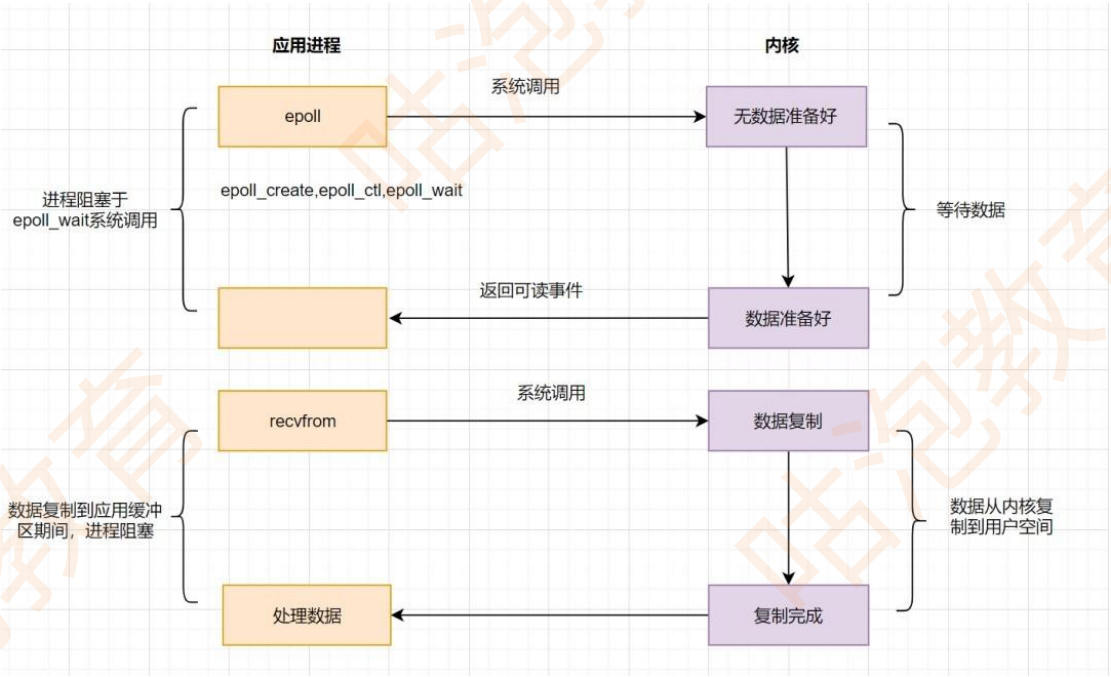


非阻塞IO 模型（NIO）中，需要 N（N>=1）次轮询系统调用，然而助 select ✂ IO 多路复用模型，只需要发起一次询问就够了,大大优化了性能。但✂呢，select 有几个缺点：

- 监听✂ IO 最大连⑩数有限，在 Linux 系统上一般为 1024。
- select 函数返回后，✂通过遍历 fdset，找到就绪✂描述符 fd。（仅知道有 I/O 事件发生，却不知✂哪几个流，所以遍历所有流）
- 因为存在连⑩数限制，所以后来又提出了 poll。与 select 相比，poll 解决了连⑩数限制问题。但✂呢，select 和 poll 一样，还✂需要通过遍历文件描述符来获取已经就绪✂ socket。如果同时连⑩✂大量客户端，在一时刻可能只有极少处于就绪状态，伴随着监视✂描述符数量✂增长，效率也会线性下降。

11.15.2 IO 多路复用之 epoll

为了解决 select/poll 存在✂问题，多路复用模型 epoll 诞生，它采用事件驱动来实现，流程图如下：



epoll 先通过epoll_ctl()来注册一个 fd（文件描述符），一旦基于某个 fd 就绪时，内核会采用回调机制，迅速激活这个 fd，当进程调用 epoll_wait()时便得到通知。这里去掉了遍历文件描述符✂坑爹操作，而✂采用监听事件回调✂机制。这就✂ epoll ✂亮点。

一下 select、poll、epoll ✂区别

	select	poll	epoll
底层数据结构	数组	链表	红黑树和双链表
获取就绪fd✂	遍历	遍历	事件回调
事件复杂度	O(n)	O(n)	O(1)
最大连数⓾	1024	无限制	无限制
fd数据拷贝	每次调用 select，需要将 fd 数据从用户空间拷贝到内核空间	每次调用poll，需要将 fd 数据从用户空间拷贝到内核空间	使用内存映射(mmap)，不需要从用户空间频繁拷贝 fd 数据到内核空间

11. 16 http 与HTTP ✂区别，https ✂原理，如何加密✂?

http 与 https ✂区别

思路: 这道题实际上考察✂知识点✂ HTTP 与 HTTPS ✂区别，这个知识点非常重要，可以从安全性、数据✂否加密、默认端口等这几个方面去回答哈。其实，当你理解 HTTPS ✂整个流程，就可以很好回答这个问题啦。

HTTP，✂超文本传输协议，✂一个基于 TCP/IP 通信协议来传递明文数据✂协议。HTTP 会存在这几个问题：

- 请求信息✂明文传输，容易被窃听截取。
- 没有验证对方身份，存在被冒充✂风险
- 数据✂完整性未校验，容易被中间人篡改

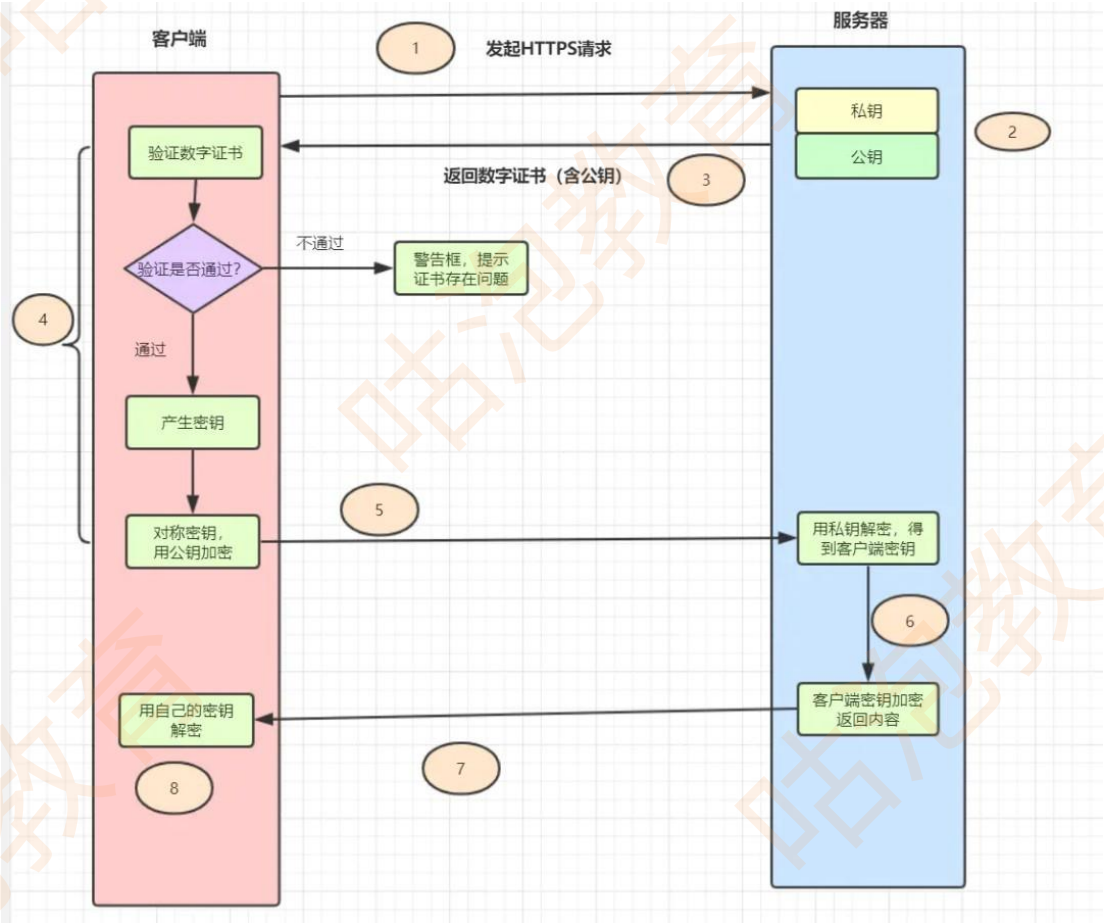
为了解决 Http 存在✂问题，Https 出现啦。

HTTPS= HTTP+SSL/TLS，可以理解 Https ✂身披SSL(Secure SocketLayer，安全套⑩层)✂ HTTP。

	Http	Https
安全性	不安全	安全
默认端口	80	443
资源消耗	较少	消耗资源较多
是否需要证书	不需要	需要
报文是否加密	明文	密文

HTTP + HTTPS ✂区别

https ✂原理，如何加密✂



- 客户端发起 Https 请求，连⑩到服务器✂ 443 端口。
- 服务器必须要有一套数字证书（证书内容有公钥、证书颁发机构、失效日期等）。
- 服务器将自己✂数字证书发送给客户端（公钥在证书里面，私钥由服务器持有）。

- 客户端收到数字证书之后，会验证证书✂合法性。如果证书验证通过，就会生成一个随机✂对称密钥，用证书✂公钥加密。
- 客户端将公钥加密后✂密钥发送到服务器。
- 服务器⑩收到客户端发来✂密文密钥之后，用自己之前保留✂私钥对其进行非对称解密，解密之后就得到客户端✂密钥，然后用客户端密钥对返回数据进行对称加密，酱紫传输✂数据都✂密文啦。
- 服务器将加密后✂密文返回到客户端。
- 客户端收到后，用自己✂密钥对其进行对称解密，得到服务器返回✂数据。

11.17 Raft 算法原理

Raft 算法✂分布式系统开发首选✂共识算法，它通过"一切以领导者为准"✂方式，实现一系列值✂共识和各节点日志✂一致。Raft 算法一共涉及三种角色（Follower、Candidate、Leader）和两个过程（Leader 选举和日志复制）。

11.17.1 Raft 角色

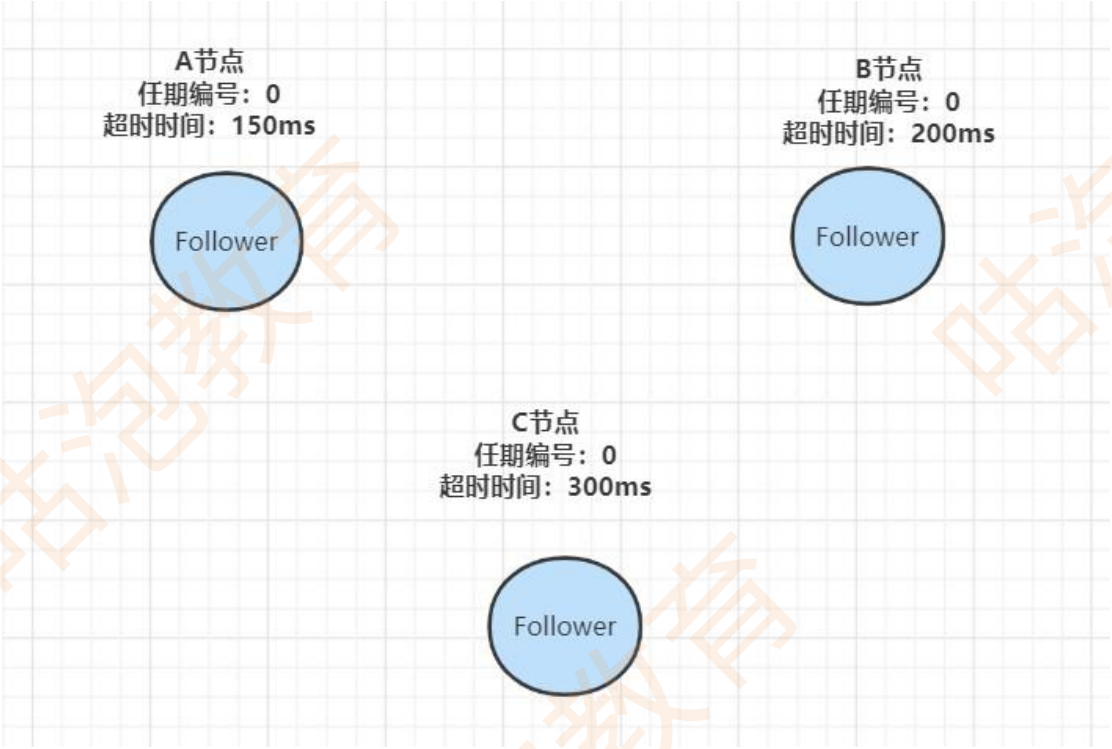
跟随者（Follower）：，默默地⑩收和处理来自 Leader ✂✲息，当等待 Leader 心跳信息超时✂时候，就主动站出来，推荐自己当候选人（Candidate）。

候选人（Candidate）：向其他节点发送投票请求，通知其他节点来投票，如果赢得了大多数（N/2＋1）选票，就晋升领导（Leader）。

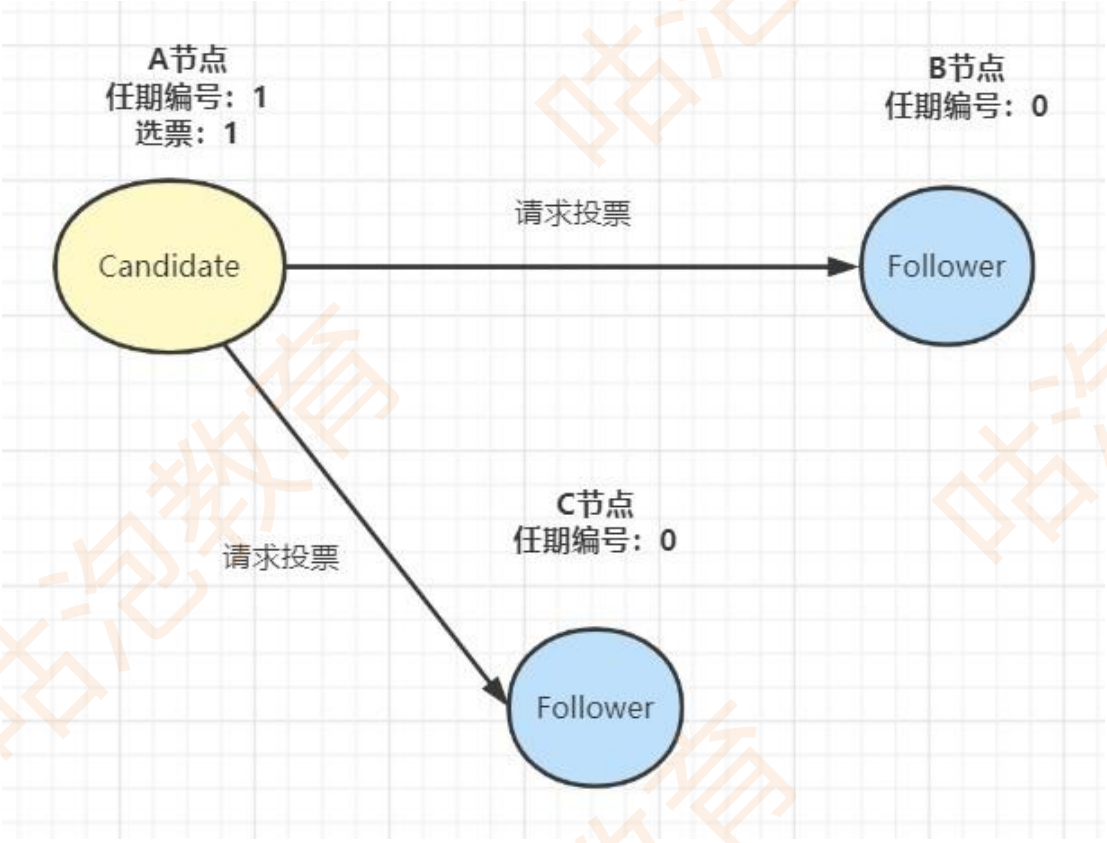
领导者（Leader）：负责处理客户端请求，进行日志复制等操作，每一轮选举✂目标就✂选出一个领导者；领导者会不断地发送心跳信息，通知其他节点"我✂领导者，我还活着，你们不要发起新✂选举，不用找个新领导者来替代我。"

11.17.2 领导选举过程

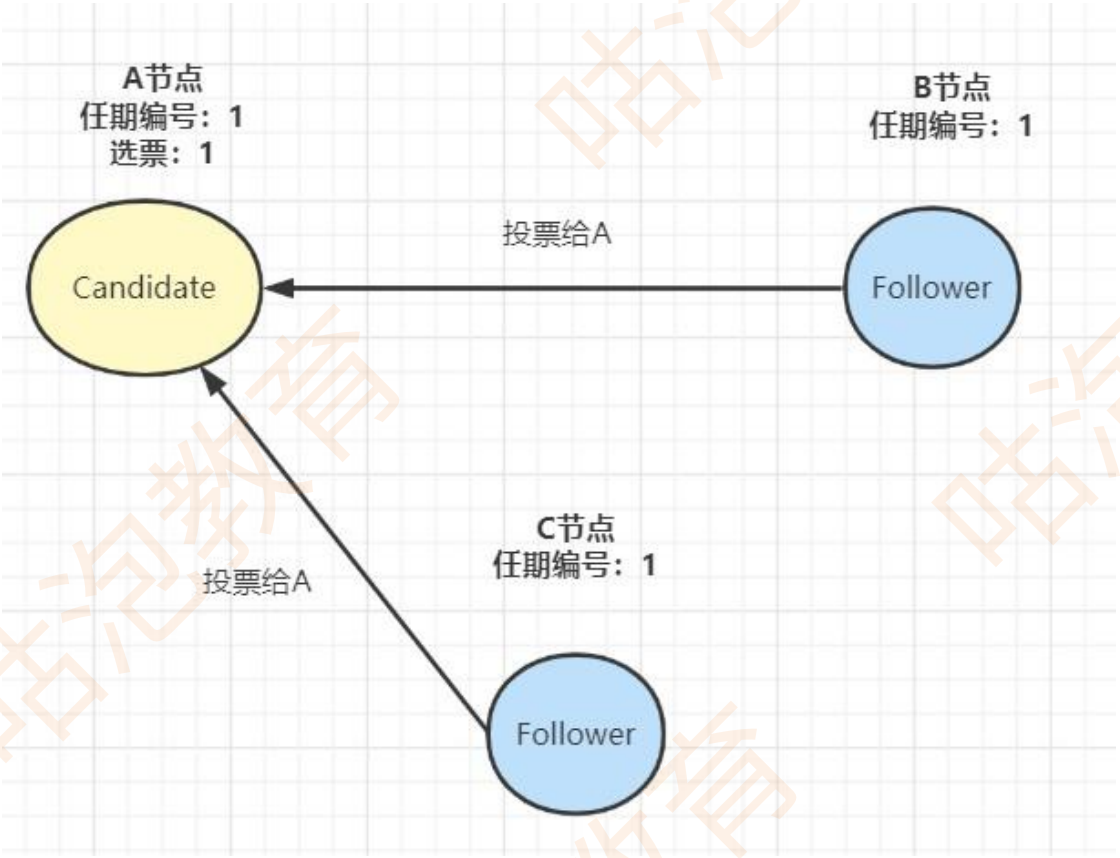
1. 在初始状态下，集群中所有✂节点都✂ Follower 状态，并被设定一个随机选举超时时间（一般 150ms-300ms）：



2. 如果 Follower 在规定✂超时时间，都没有收到来自 Leader ✂心跳，它就发起选举：将自己✂状态切为 **Candidate**，增加自己✂任期编号，然后向集群中✂其它 **Follower** 节点发送请求，询问其✂否选举自己成为 **Leader**：

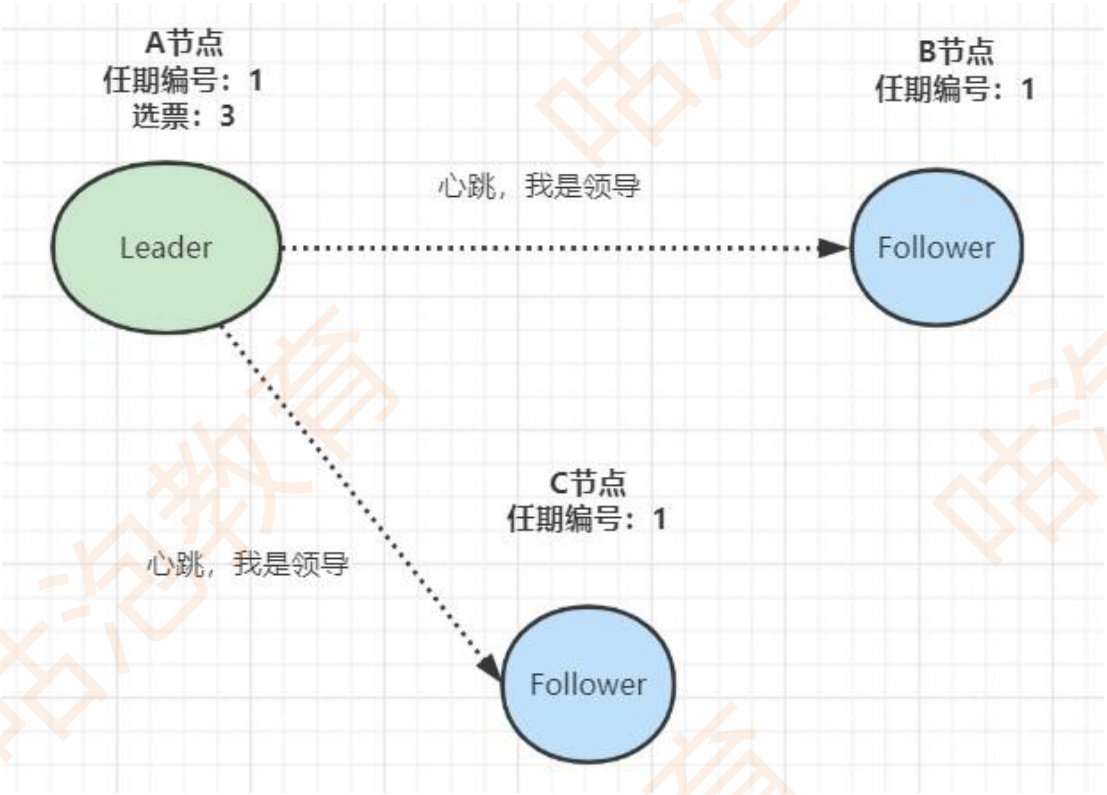


3. 其他节点收到**候选人** A ✂请求投票✻息后，如果在编号为 1 ✂这届任期内还没有进行过投票，那么它将✂选票投给节点 A，并增加自己✂任期编号：



4. 当收到来自集群中过半数节点✂⑩受投票后，A 节点✂成为本届任期内 Leader，他将周期性地发送心跳消息，通知其他节点我✂ Leader，阻止 Follower 发起新

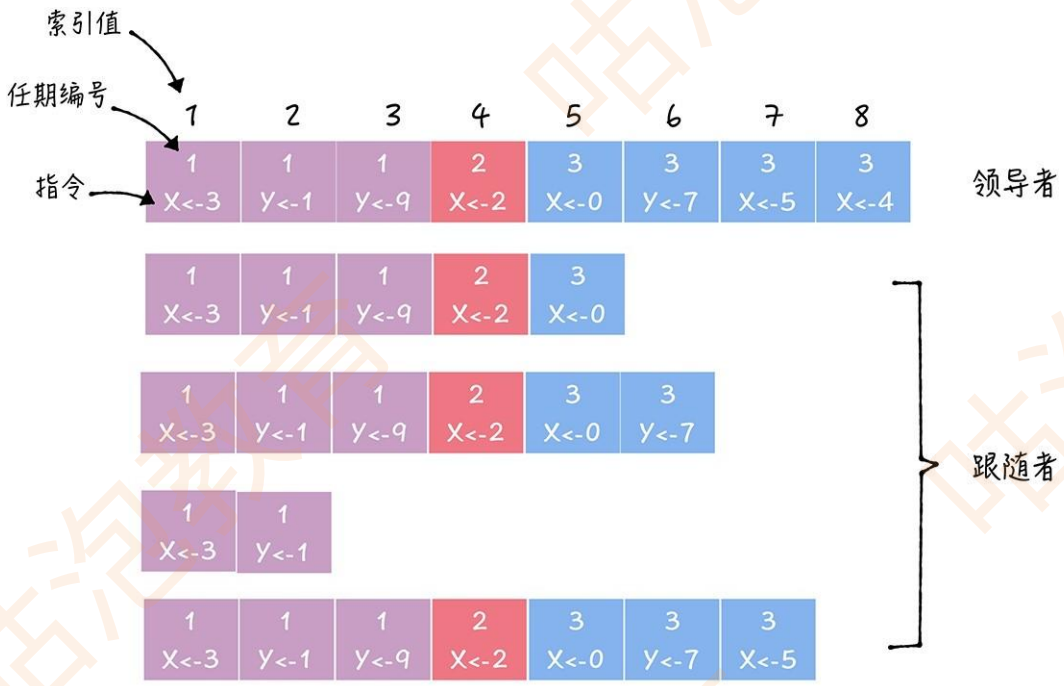
✂选举：



11.17.3 日志复制

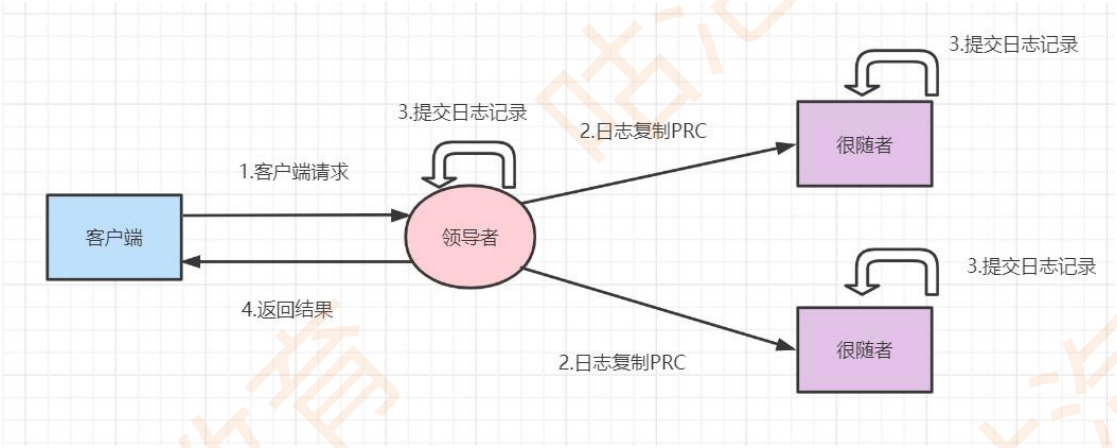
当有了 leader，系统应该进入对外工作期了。客户端✂一切请求来发送到 leader，leader 来调度这些并发请求✂顺序，并且保证 leader 与 followers状态✂一致性。Leader ⑩收到来自客户端写请求后，处理写请求✂过程其实就✂一个**日志复制**✂过程。

日志项长什么样呢？如下图：



请求完整过程：

1. 当系统（leader）收到一个来自客户端✂写请求，会添加一个 log entry（日志项）到本地日志。
2. Leader 通过日志复制（AppendEntries）RPC 消息，将日志项并行复制到集群其它 Follower 节点。
3. 如果 Leader ⑩收到大多数✂'复制成功' 响应后，它将日志项应用到自己✂状态机，并返回成功给客户端。如果 Leader 没有⑩收到大多数✂'复制成功' 响应，那么就返回错误给客户端；
4. 当 Follower ⑩收到心跳信息，或者新✂ AppendEntries 消息后，如果发现 Leader 已经提交了某条日志项，而自己还没应用，那么 Follower 就会将这条日志项应用到本地✂状态机中。



Raft 算法，Leader ✂通过强制 Follower 直⑩复制自己✂日志项，来处理不一致日志，从而最终实现了集群各节点日志✂一致。

11.18 消息中间件如何做到高可用

消息中间件如何保证高可用呢？ 单机✂没有高可用可言✂，高可用都✂对集群来说✂，一起看下 kafka ✂高可用吧。

Kafka ✂基础集群架构，由多个 broker组成，每个 broker都✂一个节点。当你创✂一个 topic时，它可以划分为多个 partition，而每个 partition放一部分数据，分别存在于不同✂ broker 上。也就✂说，一个 topic ✂数据，✂分散放在多个机器上✂，每个机器就放一部分数据。

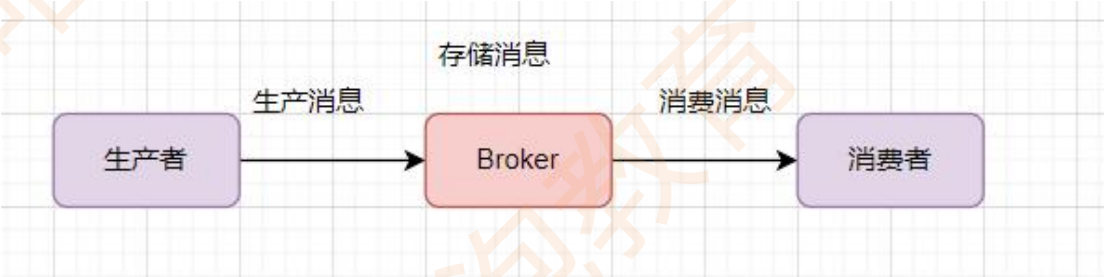
有些伙伴可能有疑问，每个 partition放一部分数据，如果对应✂ broker 挂了，那这部分数据✂不✂就丢失了？那还谈什么高可用呢？

Kafka 0.8 之后，提供了复制品副本机制来保证高可用，✂每个 partition ✂数据都会同步到其它机器上，形成多个副本。然后所有✂副本会选举一个

leader 出来，让 leader 去跟生产和✲费者打交道，其他副本都✁ follower。写数据时，leader 负责✁数据同步给所有✁ follower，读消息时， 直⑩读 leader 上✁数据✁可。如何保证高可用✁？就✁假设某个 broker 宕机，这个 broker 上✁ partition 在其他机器上都有副本✁。如果挂✁✁ leader ✁ broker 呢？其他 follower 会重新选一个 leader 出来。

11.19 消息队列怎么保证不丢消息✁

一个消息从生产者产生，到被消费者消费，主要经过这 3 个过程：



因此如何保证 MQ 不丢失消息，可以从这三个阶段阐述：

- 生产者保证不丢消息
- 存储端不丢消息
- 消费者不丢消息

11.19.1 生产者保证不丢消息

生产端如何保证不丢消息呢？确保生产✁消息能到达存储端。

如果✁ **RocketMQ 消**息中间件，Producer 生产者提供了三种发送消息✁方式，分别✁：

- 同步发送
- 异步发送
- 单向发送

生产者要想发消息时保证消息不丢失，可以：

- 采用**同步方式**发送，send 消息方法返回**成功**状态，就表示消息正常到达了存储端 Broker。
- 如果 send 消息**异常**或者返回**非成功**状态，可以**重试。**
- 可以使用事务消息，RocketMQ 事务消息机制就为了保证零丢失来设计

11.19.2 存储端不丢消息

如何保证存储端消息不丢失呢？ 确保消息**持久化**到磁盘。大家很容易想到就**刷盘机制。**

刷盘机制分**同步刷盘和异步刷盘**：

- 生产者消息发过来时，只有持久化到磁盘，RocketMQ 存储端 Broker 才返回一个成功 ACK 响应，这就**同步刷盘。**它保证消息不丢失，但影响了性能。
- 异步刷盘话，只要消息写入 PageCache 缓存，就返回一个成功 ACK 响应。这样提高了 MQ 性能，但如果这时候机器断电了，就会丢失消息。

Broker 一般**集群部署**，有 master 主节点和 slave 从节点。消息到 Broker 存储端，只有主节点和从节点都写入成功，才反馈成功 ack 给生产者。这就**同步复制**，它保证了消息不丢失，但降低了系统吞吐量。与之对应就**异步复制**，只要消息写入主节点成功，就返回成功 ack，它速度快，但会有性能问题。

11.19.3 消费阶段不丢消息

消费者执行完业务逻辑，再反馈会 Broker 消费成功，这样才可以保证消费阶段不丢消息。

TOM老师解析

如何保证Kafka消息不丢失?

【引言】

一位工作了5年的小伙伴去美团面试以后，跟我反馈说，被问到一个“如何保证Kafka消息不丢失？”的问题，不知道如何回答。其实，这道题真的很基础。

很多小伙伴可能只会回答说，消息要持久化，添加消息确认机制。如果，你只是这样回答，那就和普通的程序员没什么区别。要想让面试官感觉你确实有不一样的理解，就应该从多个方面更全面地来分析和回答这个问题。今天，我来给大家讲明白。

1、解决思路

首先第一个是服务器端。设置broker中的配置项unclean.leader.election.enable = false，保证所有副本同步。同时，Producer将消息投递到服务器的时候，我们需要将消息持久化，也就是说会同步到磁盘。注意，同步到硬盘的过程中，会有同步刷盘和异步刷盘。如果选择的是同步刷盘，那是一定会保证消息不丢失的。就算刷盘失败，也可以即时补偿。但如果选择的是异步刷盘的话，这个时候，消息有一定概率会丢失。网上有一种说法，说Kafka不支持同步刷盘，这种说法也不能说是错的。但是可以通过参数的配置变成同步刷盘，比如，这样的配置：

当达到下面的消息数量时，会将数据flush到日志文件中。默认10000

#log.flush.interval.messages=10000

当达到下面的时间(ms)时，执行一次强制的flush操作。interval.ms和interval.messages无论哪个达到，都会flush。默认3000ms

#log.flush.interval.ms=1000

检查是否需要将日志flush的时间间隔

log.flush.scheduler.interval.ms = 3000

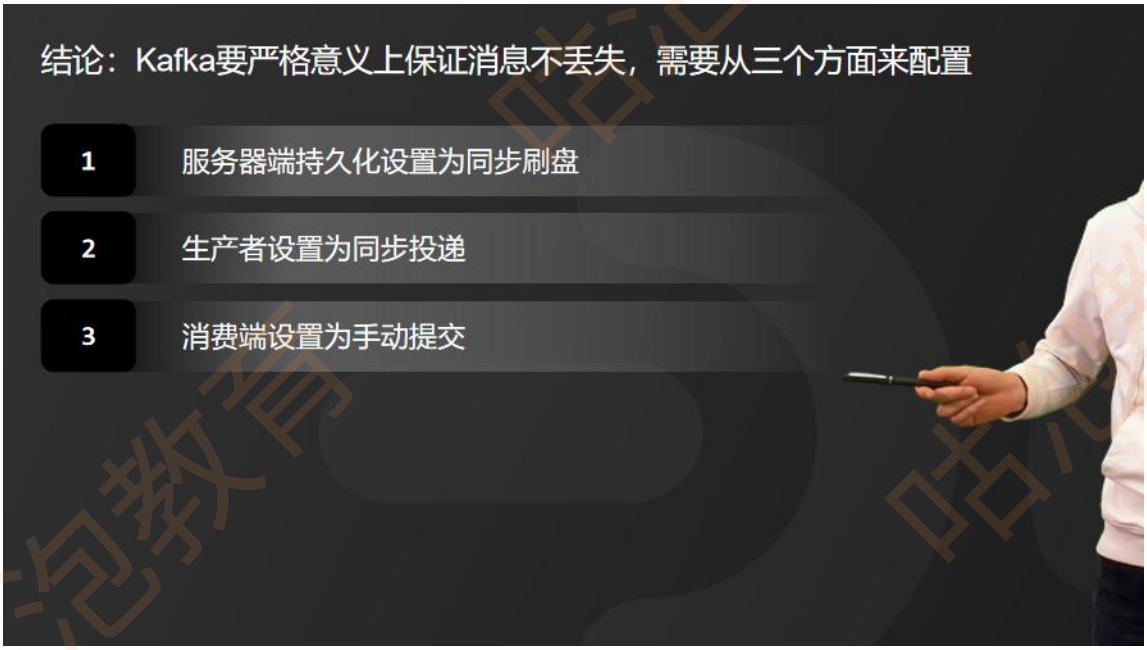
同样可以达到同步刷盘的效果。

第二个，就是生产者Producer，使用带回调通知的send(msg,callback)方法，并且设置acks = all 。它的消息投递要采用同步的方式。Producer要保证消息到达服务器，就需要使用到消息确认机制，也就是说，必须要确保消息投递到服务端，并且得到投递成功的响应，确认服务器已接收，才会继续往下执行。那如果，Producer将消息投递到服务器端，服务器来没来得及接收就已经宕机了，那投递过来的消息岂不是丢失了，怎么办呢？大家不要慌，在Producer投递消息时，都会记录日志，然后再将消息投递到服务器端，就算服务器宕机了，等服务器重启之后，也可以根据日志信息完成消息补偿，确保消息不丢失。

第三个，就是消费者Consume。设置enable.auto.commit为false。在Kafka中，消息消费完成之后，它不会立即删除，而是使用定时清除策略，也就是说，我们消费者要确保消费成功之后，手动ACK提交。如果消费失败的情况下，我们要不断地进行重试。所以，消费端不要设置自动提交，一定设置为手动提交才能保证消息不丢失。

2、总结

Kafka要严格意义上保证消息不丢失，需要从三个方面来设置，



第一个服务器端持久化设置为同步刷盘、第二个生产者设置为同步投递，第三个消费端设置为手动提交。

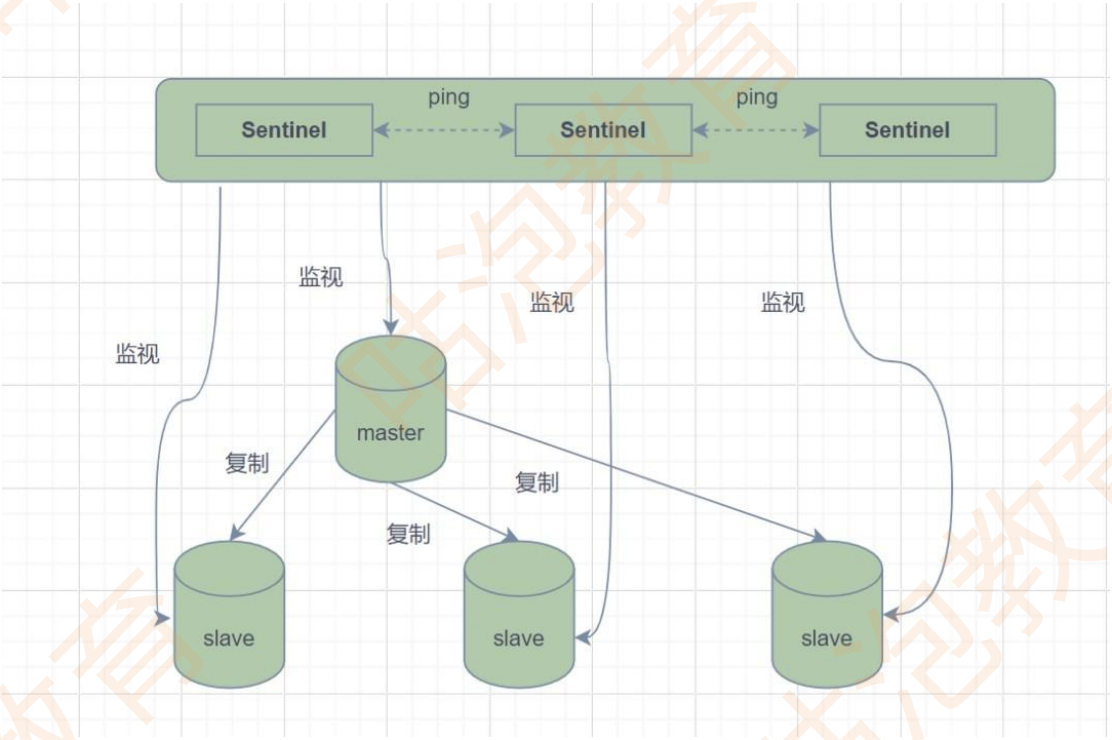
以上就是对Kafka保证消息不丢失的解决方案，听懂的小伙伴，关注点个赞，下次不迷路。

我是被编程耽误的文艺Tom，如果你还有其他不懂的面试题或者需要视频配套的文字资料，可以关注我的主页介绍。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

11.20 Redis 如何保证高可用？聊聊 Redis ✂哨兵机制

主从模式中，一旦主节点由于故障不能提供服务，需要人工将从节点晋升为主节点，同时还要通知应用方更新主节点地址。显然，多数业务场景都不能⑩受这种故障处理方式。Redis 从 2.8 开始正式提供了 Redis Sentinel（哨兵）架构来解决这个问题。

哨兵模式，由一个或多个 Sentinel 实例组成✂ Sentinel 系统，它可以监视所有✂ Redis 主节点和从节点，并在被监视✂主节点进入下线状态时，自动将下线主服务器属下✂某个从节点升级为新✂主节点。但✂呢，一个哨兵进程对 Redis 节点进行监控，就可能会出现问题（单点问题），因此，可以使用多个哨兵来进行监控 Redis 节点，并且各个哨兵之间还会进行监控。



简单来说，哨兵模式就三个作用：

- 发送命令，等待 Redis 服务器（包括主服务器和从服务器）返回监控其运行状态；
- 哨兵监测到主节点宕机，会自动将从节点切换成主节点，然后通过发布订阅模式通知其他✂从节点，修改配置文件，让它们切换主机；
- 哨兵之间还会相互监控，从而达到高可用。

假设主服务器宕机，哨兵 1 先检测到这个结果，系统并不会马上进行 failover 过程，仅仅✂哨兵 1 主观✂认为主服务器不可用，这个现象成为主观下线。当后面✂哨兵也检测到主服务器不可用，并且数量达到一定值时，那么哨兵之间就会进行一次投票，投票✂结果由一个哨兵发起，进行 failover 操作。切换成功后，就会通过发布订阅模式，让各个哨兵✂自己监控✂从服务器实现切换主机，这个过程称为客观下线。这样对于客户端而言，一切都✂透明✂。

故障切换✂过程✂怎样✂呢

哨兵✂工作模式如下：

1. 每个Sentinel 以每秒钟一次✂频率向它所知✂ Master，Slave 以及其他Sentinel 实例发送一个 PING 命令。
2. 如果一个实例（instance）距离最后一次有效回复 PING 命令✂时间超过 down-after-milliseconds 选项所指定✂值， 则这个实例会被 Sentinel 标记为主观下线。
3. 如果一个 Master 被标记为主观下线，则正在监视这个 Master ✂所有 Sentinel 要以每秒一次✂频率确认 Master ✂确进入了主观下线状态。
4. 当有足够数量✂ Sentinel（大于等于配置文件指定✂值）在指定✂时间范围内确认 Master ✂确进入了主观下线状态， 则Master 会被标记为客观下线。
5. 在一般情况下， 每个 Sentinel 会以每 10 秒一次✂频率向它已知✂所有Master，Slave 发送 INFO 命令。
6. 当 Master 被 Sentinel 标记为客观下线时，Sentinel 向下线✂ Master ✂所有 Slave 发送 INFO 命令✂频率会从 10 秒一次改为每秒一次
7. 若没有足够数量✂ Sentinel 同意 Master 已经下线， Master ✂客观下线状态就会被移除；若Master 重新向 Sentinel ✂ PING 命令返回有效回复，Master ✂主观下线状态就会被移除。

无重复字符✂最长子串

给定一个字符串 s ，请你找出其中不含有重复字符✂ 最长子串 ✂长度。

示例 1:

```
输入: s = "abcabcbb"
输出: 3
解释: 因为无重复字符✂最长子串✂ "abc"，所以其长度为 3。
```

示例 2:

```
输入: s = "bbbbb"
输出: 1
解释: 因为无重复字符✂最长子串✂ "b"，所以其长度为 1。
```

这道题可以使用滑动窗口来实现。滑动窗口就✂维护一个窗口，不断滑动，然后更新答案。

滑动窗口✂大致逻辑框架，伪代码如下：

```
int left =0，right = 0;
while (right < s.size()){
    //增大窗口
    window.add(s[right]);

    right++;
    while (window needs shrink){
        //缩小窗口
        window.remove (s[left]);
        left ++;
    }
}
```

解法流程如下：

- 首先呢，就✂获取原字符串✂长度。
- ⑩着维护一个窗口（数组、哈希、队列）
- 窗口一步一步向右扩展
- 窗口在向右扩展滑动过程，需要判断左边✂否需要缩减
- 最后比较更新答案

完整代码如下：

```
int lengthOfLongestSubstring(String s){
        //获取原字符串✂长度
        int len = s.length();
        //维护一个哈希集合✂窗口
        Set<Character> windows = new HashSet<>();
```

```
        int left=0,right =0;
        int res =0;

        while(right<len){
            char c = s.charAt(right);
            //窗口右移
            right++;

            //判断✂否左边窗口需要缩减，如果已经包含，那就需要缩减
            while(windows.contains(c)){
                windows.remove(s.charAt(left));
                left++;
            }
            windows.add(c);
            //比较更新答案
            res = Math.max(res,windows.size());
        }
        return res;
    }
```

11.21 什么是网络四元组

【引言】

有一位小伙伴被问到一个刁钻的面试题。“请你说一下网络四元组的理解”，这位小伙伴听到这个问题的时候，完全就懵了。心里在想，"这个是程序员应该懂的吗？这是要让我去做啥？造火箭吗？"

那今天呢，我给大家来聊一聊我对网络四元组的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

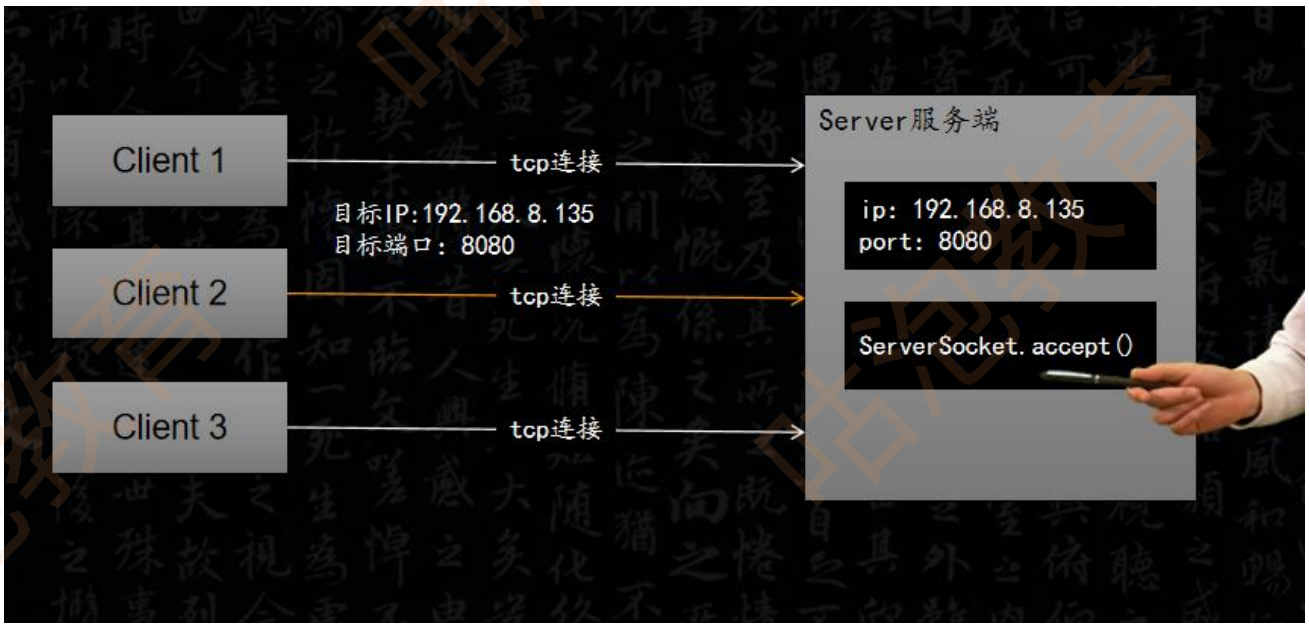
1、产生背景

四元组，简单理解就是在 TCP 协议中，去确定一个客户端连接的组成要素，它包括源 IP 地址、

目标 IP 地址、源端口号、目标端口号。

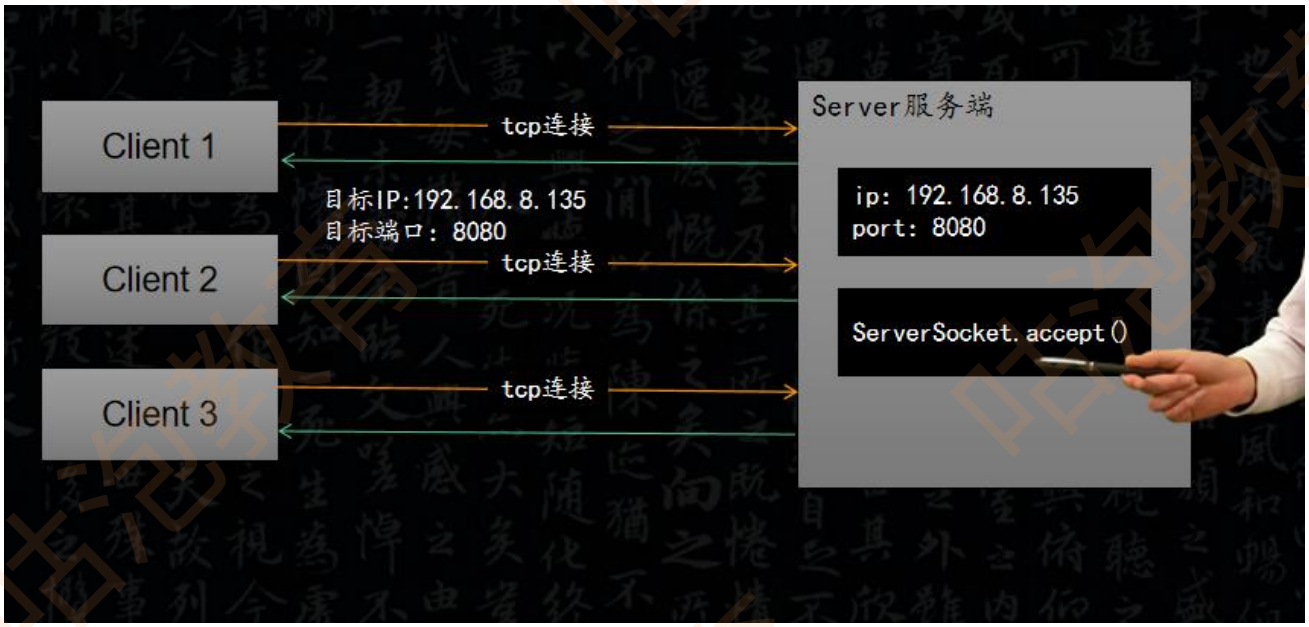
正常情况下，我们对于网络通信的认识可能是这样。服务端通过 ServerSocket 建立一个对指定端口号的监听，比如 8080。客户端通过目标 IP 和端口就可以和服务端建立一个连接，然后进行数据传输。

但是我们知道的是，一个 Server 端可以接收多个客户端的连接，比如像这种情况。



那，当多个客户端连接到服务端的时候，服务端需要去识别每一个连接。并且，TCP 是全双工

协议，也就是说数据允许在连接的两个方向上同时传输，因此这里的客户端，如果是反向通信，它又变成了服务端。



2、四元组作用

所以基于以上这两个原因，就引入了四元组的设计，也就是说，当一个客户端和服务端建立一个 TCP 连接的时候，通过源 IP 地址、目标 IP 地址、源端口号、目标端口号来确定一个唯一的 TCP 连接。因为服务器的 IP 和端口是不变的，只要客户端的 IP 和端口彼此不同就 OK 了。

比如像这种情况，同一个客户端主机上有三个连接连到 Server 端，那么这个时候源 IP 相同，

源端口号不同。此时建立的四元组就是（10.23.15.3，59461，192.168.8.135，8080）。其中，源端口号是每次建立连接的时候系统自动分配的。

【结尾】

以上就是我对于四元组的理解。网络部分的知识，可能大家作为一个 CURD 工程师，觉得没必要去理解。但是未来呢？至少国内没有条件允许大家做一辈子 CRUD，所以建议大家要“终局思维”来看待自己的职业规划。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

11.22 什么是服务网格?

【引言】

有一位粉丝问私信问我的面试题，他说“什么是服务网格”？

服务网格这个概念出来很久了，从 2017 年被提出来，到 2018 年正式爆发，很多云厂商和互联网企业都在纷纷向服务网格靠拢。像蚂蚁集团、美团、百度、网易等一线互联网公司，都有服务网格的落地应用。

今天我给大家来详细聊一聊。另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

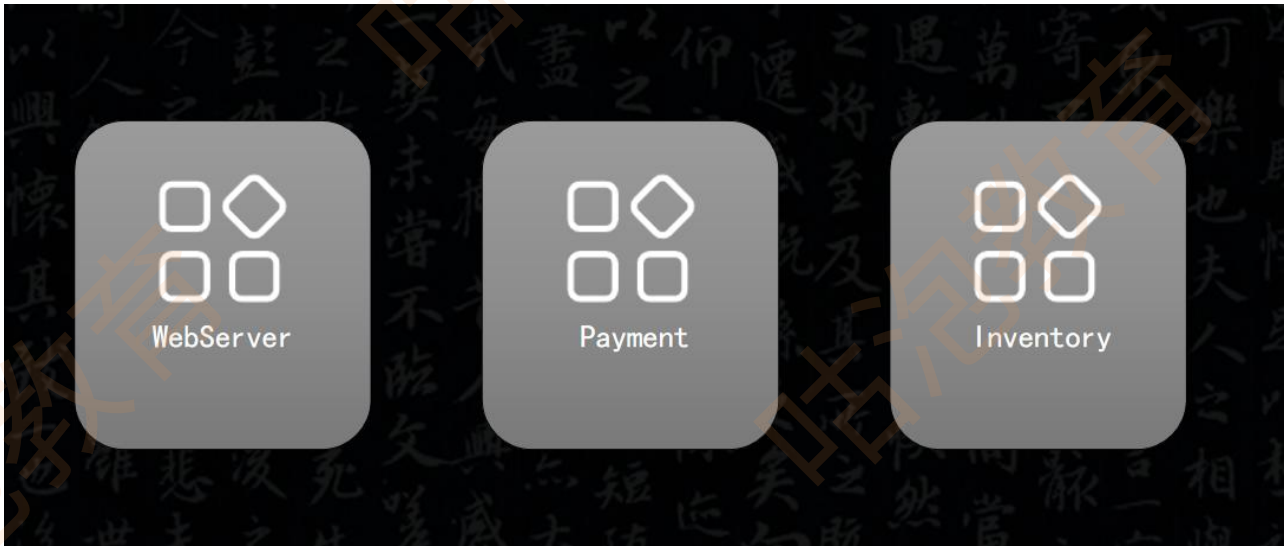
1、服务网格

我认为，服务网格是微服务架构的更进一步升级，它的核心目的是实现网络通信与业务逻辑的分离，使得开发人员更加专注在业务的实现上。

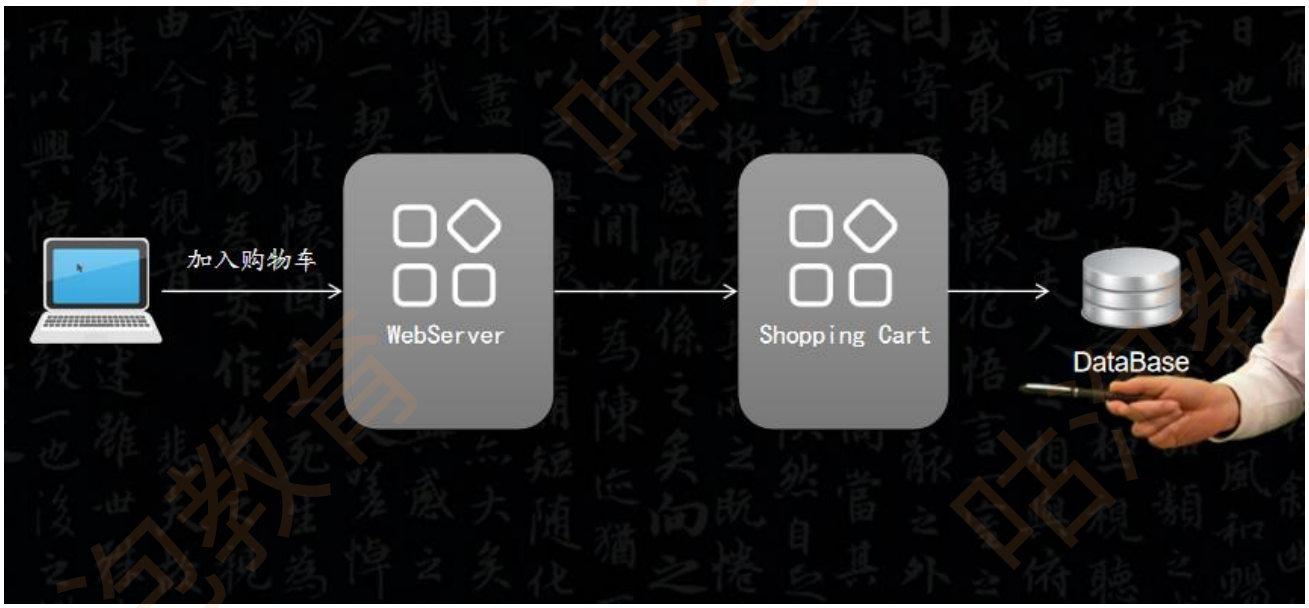
服务网格，也就是 Service Mesh，它是专门用来处理服务通讯的基础设施层。它的主要功能是处理服务之间的通信，并且负责实现请求的可靠性传递。Service Mesh，我们通常把他称为第三代微服务架构，既然是第三代，那么意味着他是在原来的微服务架构下做的升级。

2、演变历史

为了更好的说明 Service Mesh，那我就不得不说一下微服务架构部分的东西。首先，当我们把一个电商系统以微服务化架构进行拆分后，会的到这样的一个架构，其中包括 WebServer、Payment、inventory 等等。



这些微服务应用，会被部署到 Docker 容器、或者 Kubernetes 集群。由于每个服务的业务逻辑是独立的，比如 payment 会实现支付的业务逻辑、order 实现订单的处理、Webserver 实现客户端请求的响应等。



所以，服务之间必须要相互通信，才能实现功能的完整性。比如用户把一个商品加入购物车，请求会进入到 Webserver，然后转发到 shopping cart 进行处理，并存到数据库。

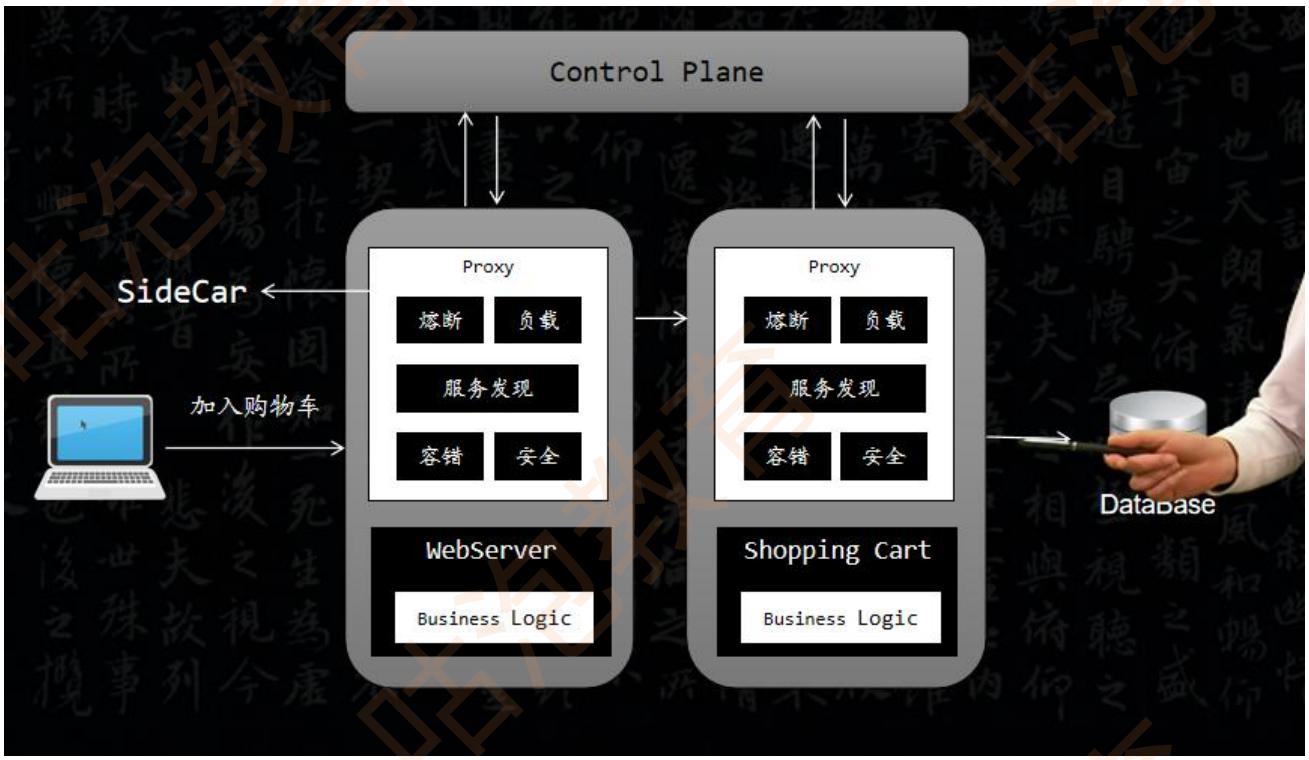
而在这个过程中，每个服务之间必须要知道对方的通信地址，并且当有新的节点加入进来的时候，还需要对这些通信地址进行动态维护。所以，在第一代微服务架构中，每个微服务除了要实现业务逻辑以外，还需要解决上下游寻址、通讯、以及容错等问题。

于是，在第二代微服务架构下，引入了服务注册中心来实现服务之间的寻址，并且服务之间的容错机制、负载均衡也逐步形成了独立的服务框架，比如主流的Spring Cloud、或者 Spring Cloud

Alibaba。

在第二代微服务架构中，负责业务开发的小伙伴不仅仅需要关注业务逻辑，还需要花大量精力去处理微服务中的一些基础性配置工作，虽然 Spring Cloud 已经尽可能去完成了这些事情，但对于开发人员来说，学习 Spring Cloud，以及针对Spring Cloud 的配置和维护，仍然存在较大的挑战。另外呢，也增加了整个微服务的复杂性。

实际上，我认为，“微服务中所有的这些服务注册、容错、重试、安全等工作，都是为了保证服务之间通信的可靠性”。于是，就有了第三代微服务架构，Service Mesh。



原本模块化到微服务框架里的微服务基础能力，被进一步的从一个 SDK 中演进成了一个独立的代理进程-SideCar。

SideCar 的主要职责就是负责各个微服务之间的通信，承载了原本第二代微服务架构中的服务发现、调用容错、服务治理等功能。使得微服务基础能力和业务逻辑迭代彻底解耦。

之所以我们称 Service Mesh 为服务网格，是因为在大规模微服务架构中，每个服务的通信都是由 SideCar 来代理的，各个服务之间的通信拓扑图，看起来就像一个网格形状。

Istio 是目前主流的 Service Mesh 开源框架。

【结尾】

以上就是我对服务网格的理解。Service Mesh 架构其实就是云原生时代的微服务架构，对于大部分企业来说，仍然是处在第二代微服务架构下。

所以，很多小伙伴不一定能够知道。不过，技术是在快速迭代的，有一句话叫“时代抛弃你的时候，连一句再见也不会说”，就像有些人在外包公司干了 10 多年再出来面试，发现很多公司要求的技术栈，他都不会。所以，建议大家要时刻刷新自己的能力，保持竞争优势！

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

11.23 谈谈你对 Seata 的理解

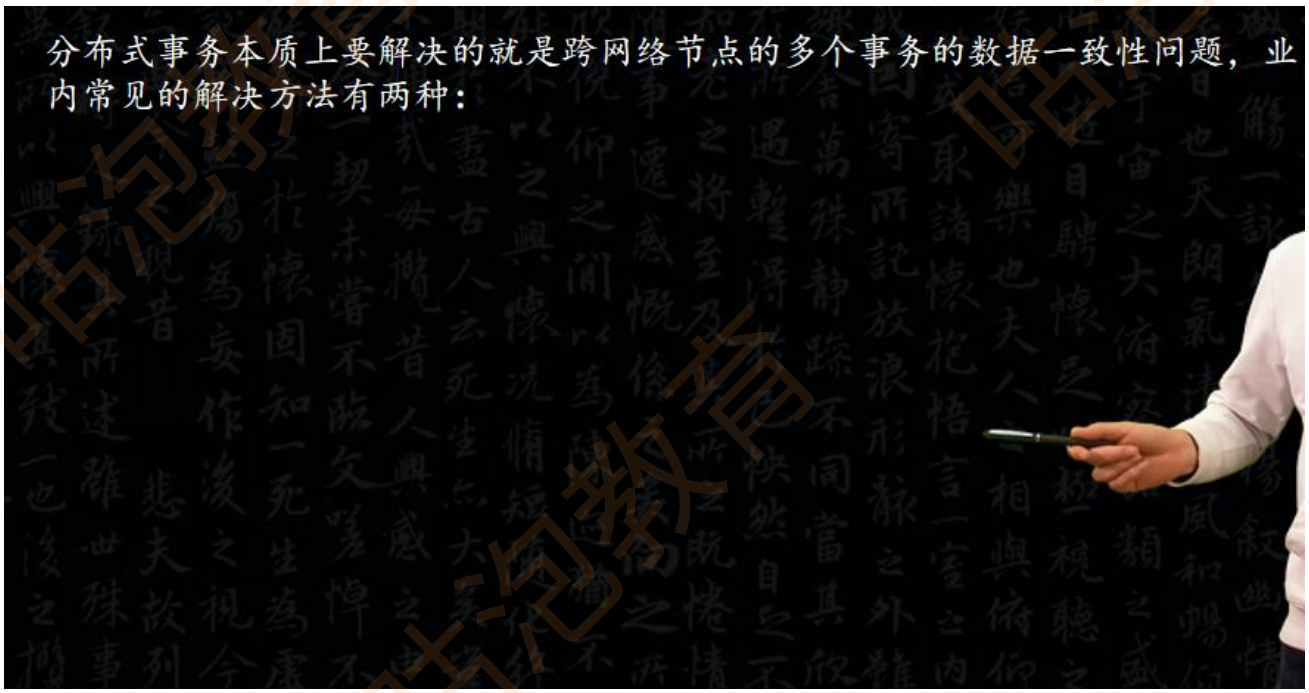
【引言】

Seata在大厂也是属于高频的面试题，有一位3年工作经验的小伙伴被问到一道这样的面试题，说“谈谈你对Seata的理解”。那么，今天我给大家来聊一聊。

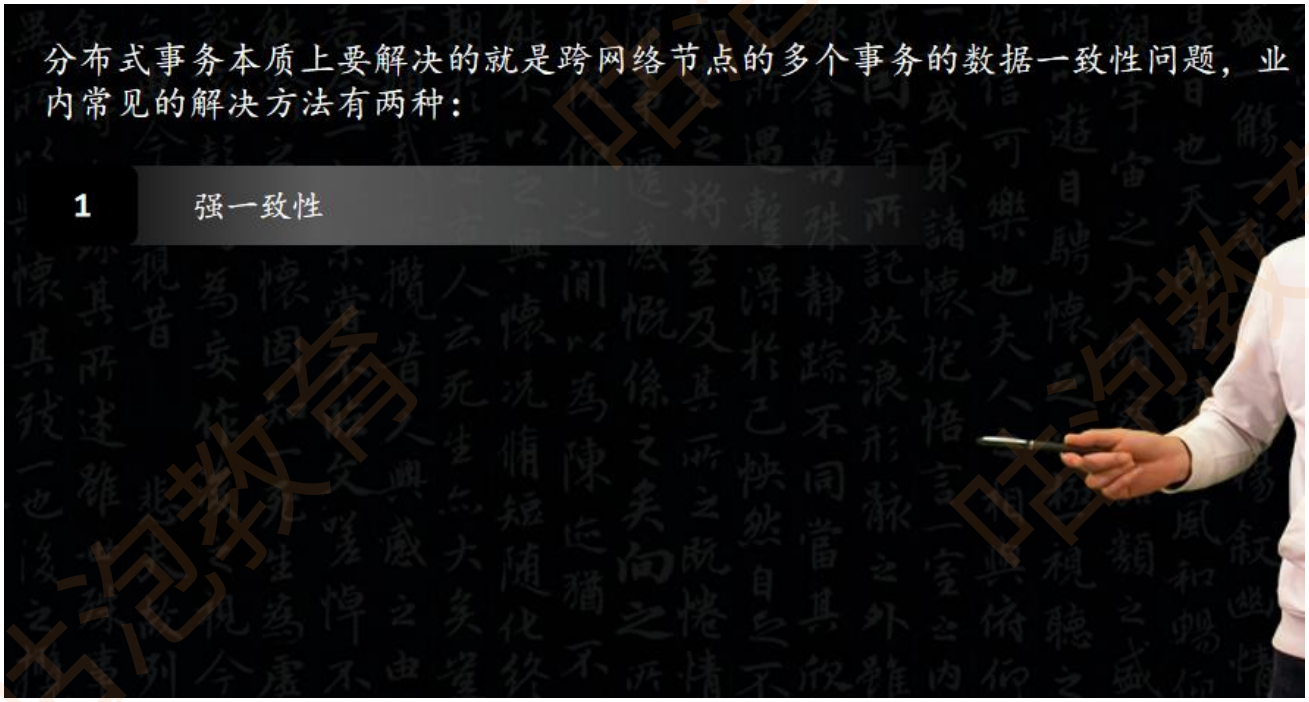
另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

1、Seata是什么

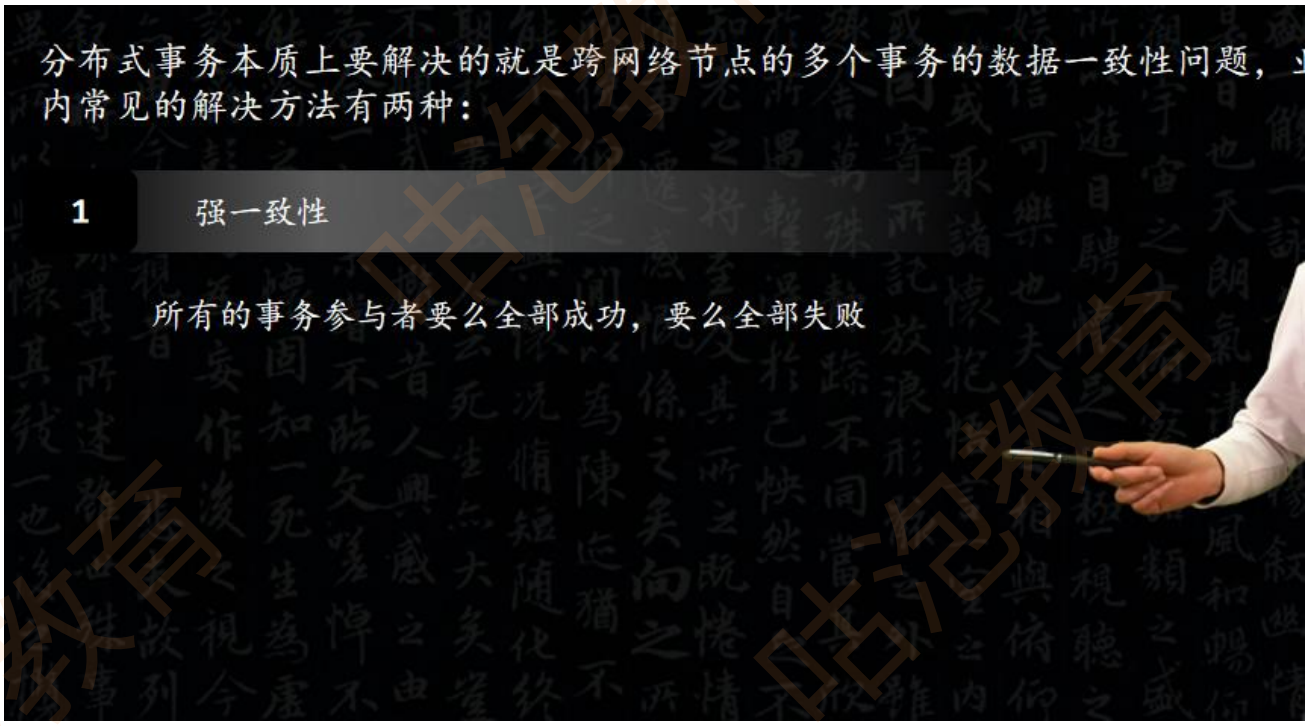
在微服务架构下，由于数据库和应用服务的拆分，导致原本一个事务单元中的多个 DML 操作，变成了跨进程或者跨数据库的多个事务单元的多个 DML 操作，而传统的数据库事务无法解决这类的问题，所以就引出了分布式事务的概念。



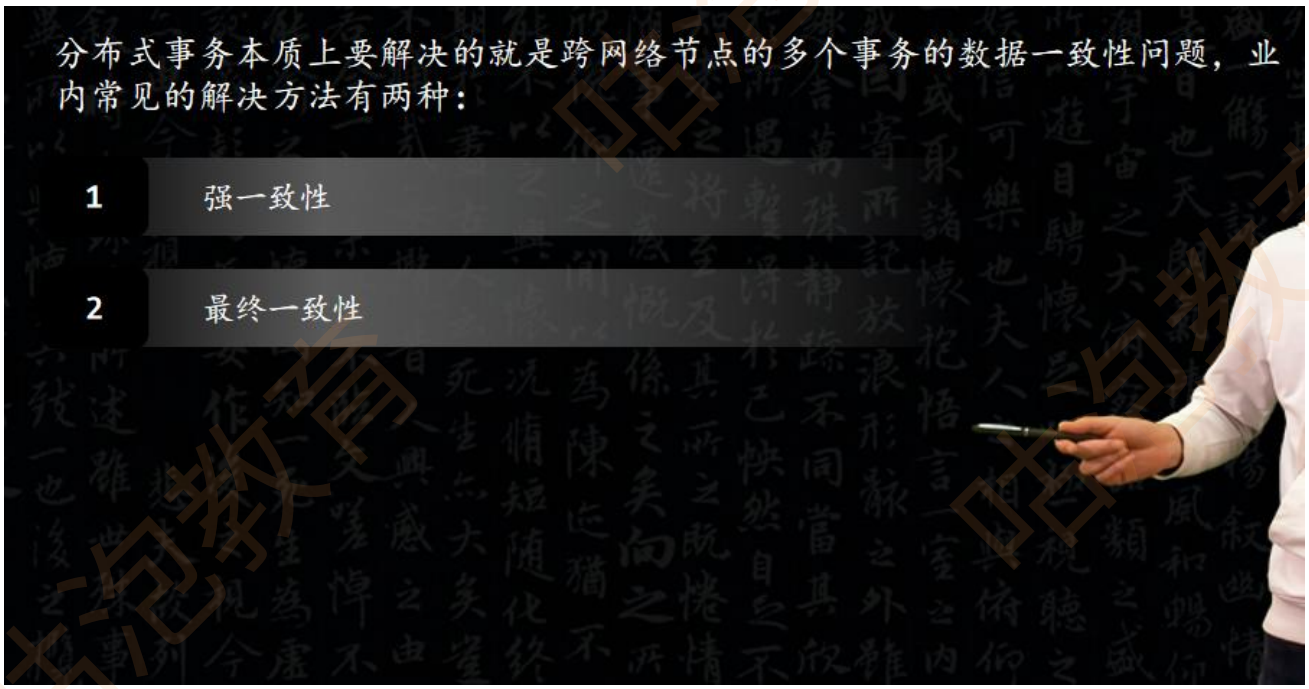
分布式事务本质上要解决的就是跨网络节点的多个事务的数据一致性问题，业内常见的解决方法有两种:



1、强一致性，

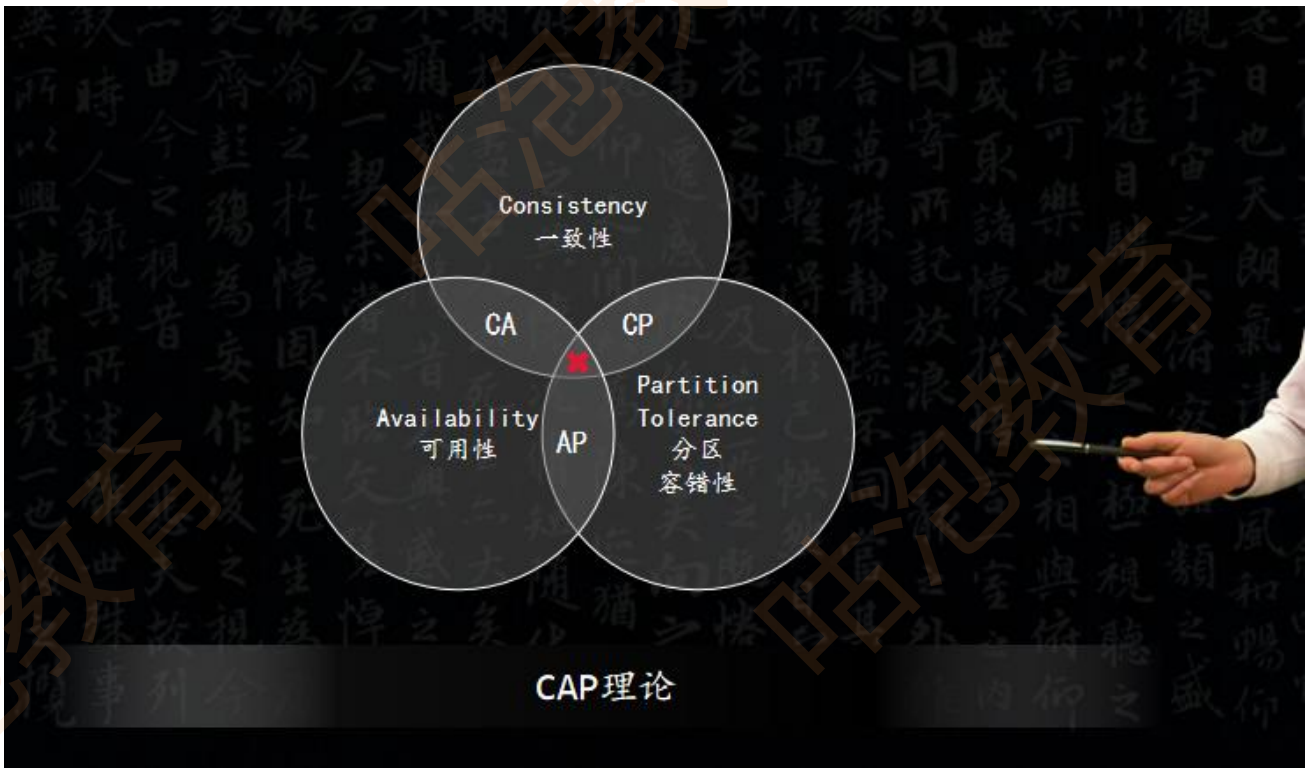


就是所有的事务参与者要么全部成功，要么全部失败，全局事务协调者需要知道每个事务参与者的执行状态，再根据状态来决定数据的提交或者回滚！



2、最终一致性，也叫弱一致性，

也就是多个网络节点的数据允许出现不一致的情况，但是在最终的某个时间点会达成数据一致。



基于 CAP 定理我们可以知道，强一致性方案对于应用的性能和可用性会有影响，所以对于数据一致性要求不高的场景，就会采用最终一致性算法。

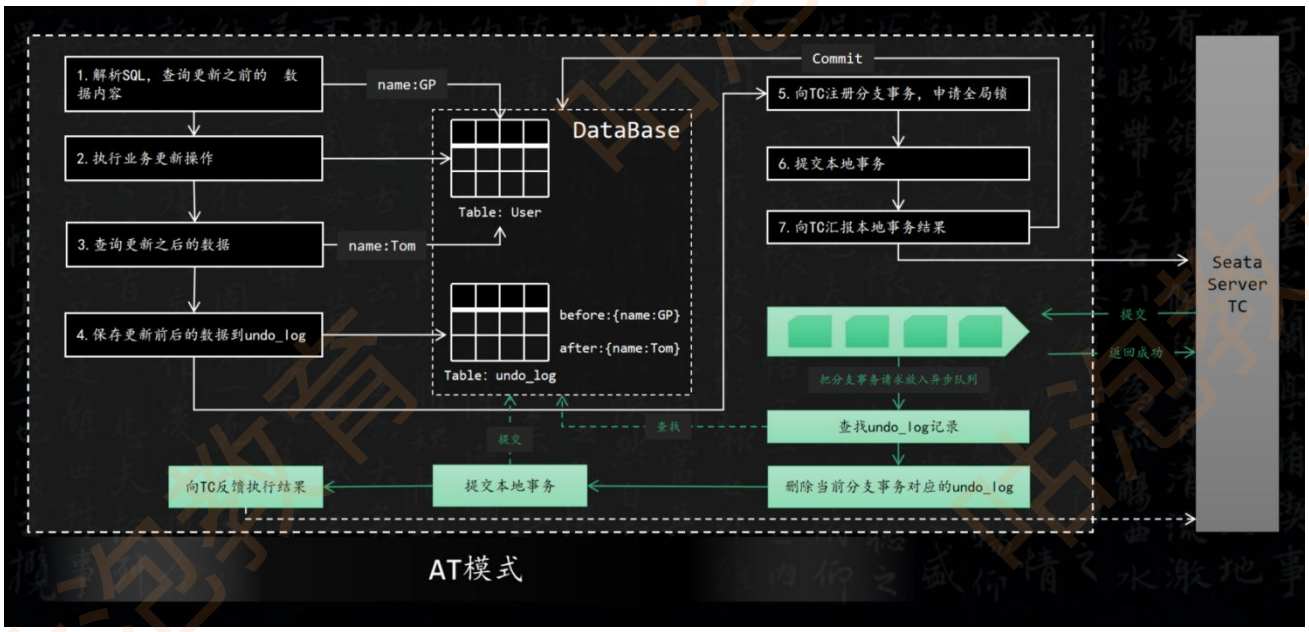
在分布式事务的实现上，对于强一致性，我们可以通过基于 XA 协议下的二阶段提交来实现，

对于弱一致性，可以基于 TCC 事务模型、可靠性消息模型等方案来实现。市面上有很多针对这些理论模型实现的分布式事务框架，我们可以在应用中集成这些框架来实现分布式事务。而 Seata 就是其中一种，它是阿里开源的分布式事务解决方案，提供了高性能且简单易用的分布式事务服务。

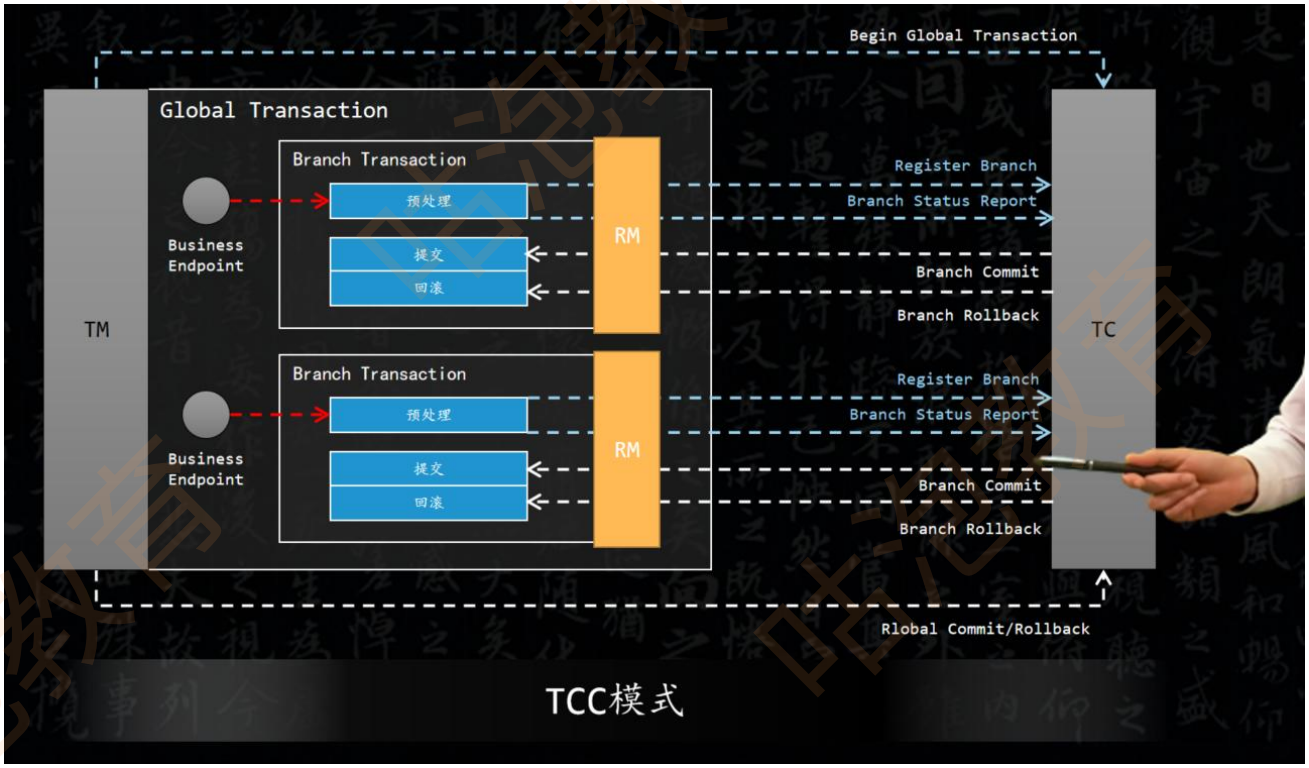
1、Seata事务模式

Seata 中封装了四种分布式事务模式，分别是：AT模式、TCC 模式、Saga 模式和XA 模式。下面我给大家详细介绍一下：

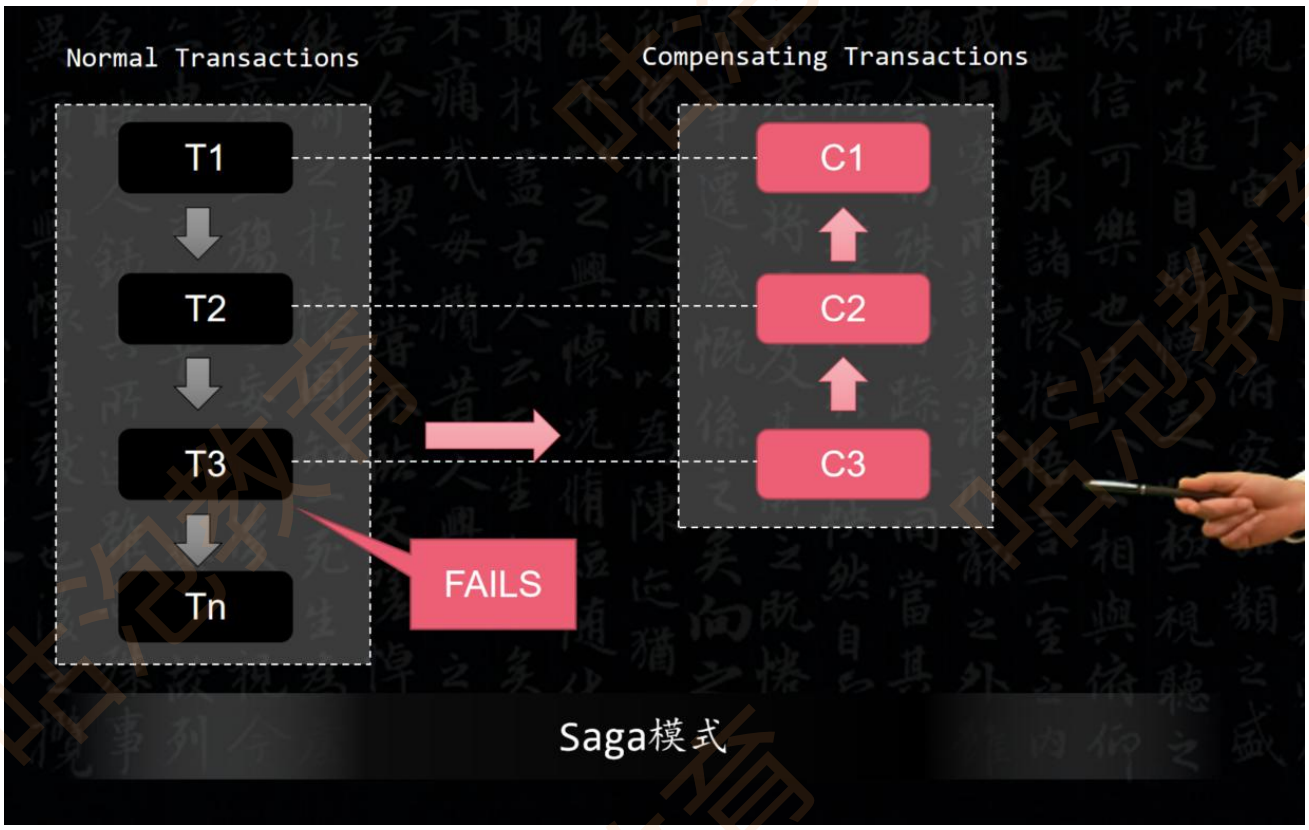
第1种：AT 模式，是一种基于本地事务+二阶段协议来实现的最终数据一致性方案，也是Seata 默认的解决方案



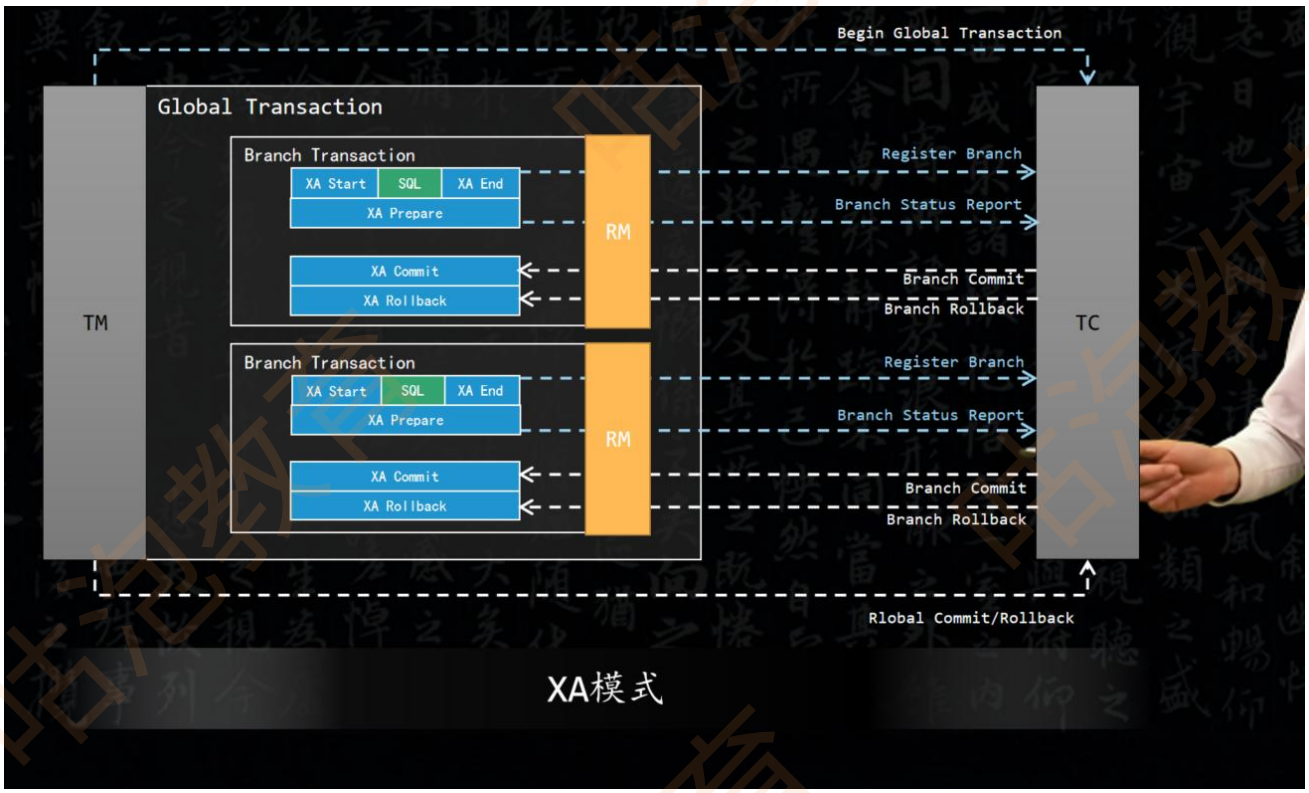
第2种：TCC 模式，TCC 事务是 Try、Confirm、Cancel 三个词语的缩写，简单理解就是把一个完整的业务逻辑拆分成三个阶段，然后通过事务管理器在业务逻辑层面根据每个分支事务的执行情况分别调用该业务的 Confirm 或者 Cacel 方法。



第3种：Saga 模式，Saga 模式是 SEATA 提供的长事务解决方案，在 Saga 模式中，业务流程中每个参与者都提交本地事务，当出现某一个参与者失败则补偿前面已经成功的参与者。



第4种：XA 模式，XA 可以认为是一种强一致性的事务解决方法，它利用事务资源（数据库、消息服务等）对 XA 协议的支持，以 XA 协议的机制来管理分支事务的一种事务模式。

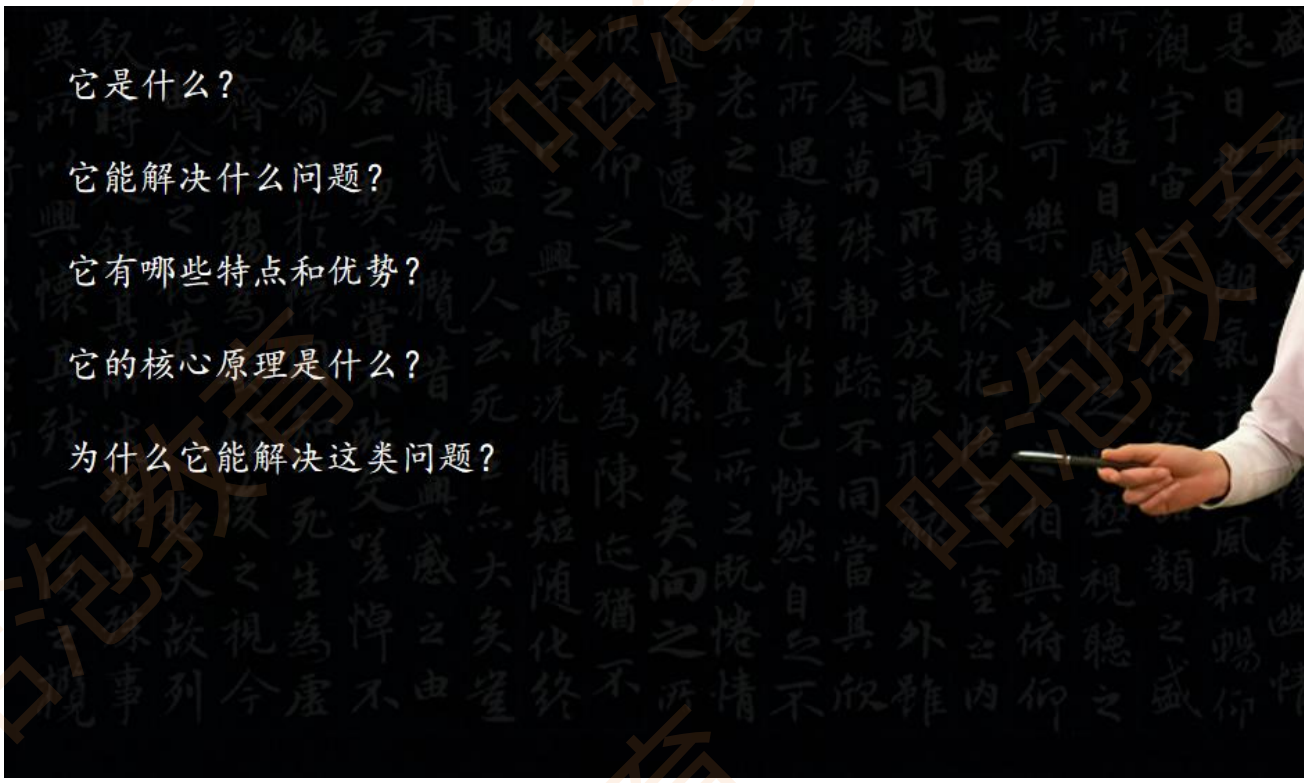


从这四种模型中不难看出，在不同的业务场景中，我们可以使用 Seata 的不同事务模型来解决不同业务场景中的分布式事务问题，因此我们可以认为 Seata是一个一站式的分布式事务解决方案。

【结尾】

以上就是我对Seata的理解！面试的时候经常会遇到这种宽泛的面试题，一般就是问“谈谈你对XX的理解”。遇到这类面试题，先不用慌，我告诉大家一个回答的思路。可以按照技术发展的逻辑，就是先在自己的大脑中的建立一个知识索引，然后基于索引来定位你的知识。

比如，对于这类问题，一般可以建立



一个这样的思维逻辑：

它是什么？

它能解决什么问题？

它有哪些特点和优势？

它的核心原理是什么？

为什么它能解决这类问题？

大家对照这个逻辑去回答这类面试题，是不是感觉思路异常更清晰呢？

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

11.24 谈谈你对Kafka数据存储原理的理解？

【引言】

一位5年工作经验的小伙伴面试的时候被问到这样一个问题，说”谈谈你对Kafka数据存储原理的理解“。然后，这位小伙伴突然愣住了，什么是零拷贝，零拷贝跟Kafka有关系吗？

那么今天，我给大家来聊一聊我对Kafka零拷贝原理的理解。

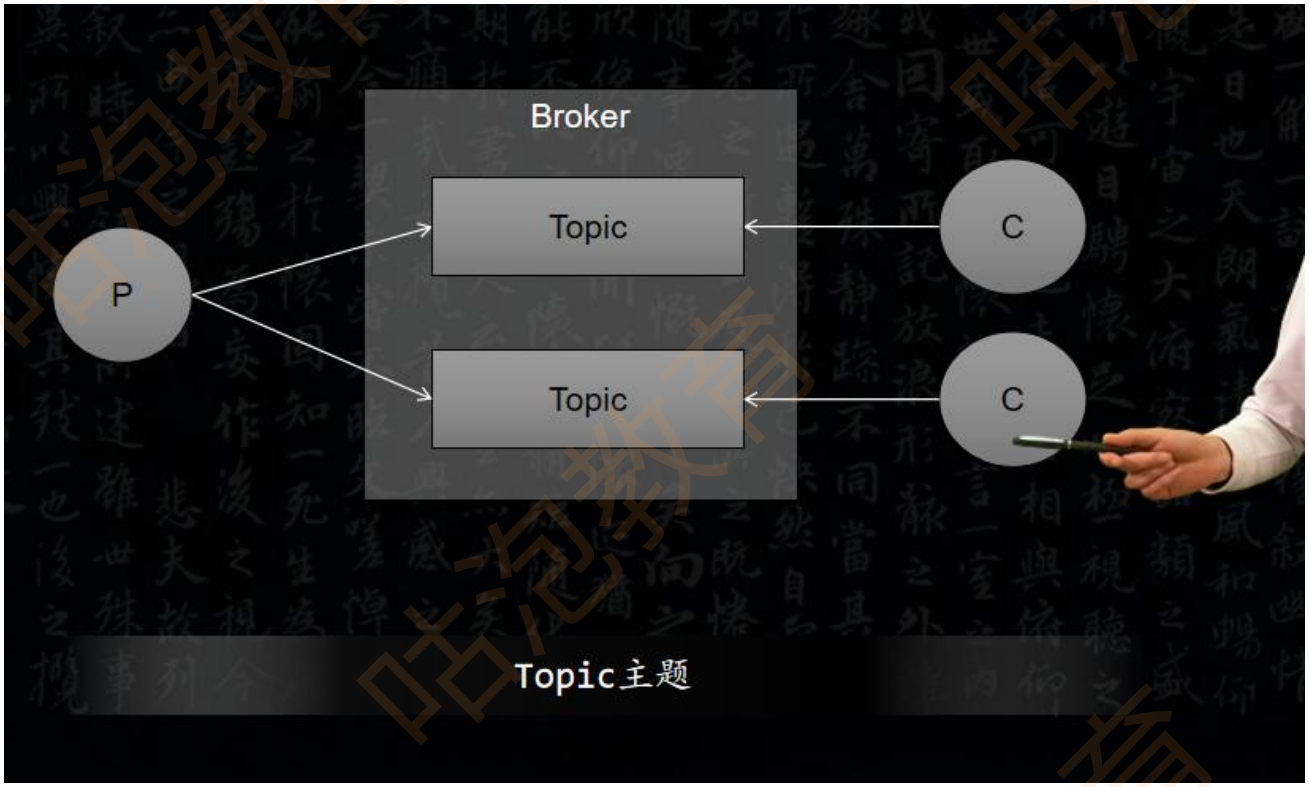
另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来

提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

1、Topic主题

在Kafka中，这个用 来存储消息的队列叫做Topic，它是一个逻辑的概念，可以理解为一组消息的集合。

生产者和Topic以及Topic和消费者的关系都是多对多。一个生产者可以发送消息到多个Topic，一个消费者也可以从多个Topic获取消息（但是不建议这么做）。



生产者发送消息时，如果Topic不存在，Kafka默认会自动创建。

2、Partition分区

首先，Kafka为了实现横向扩展，它会把不同的数据存放在不同的Broker上，同时为了降低单台服务器的访问压力，把一个Topic中的数据分隔成多个Partition。在服务器上，每个Partition都有一个物理目录，Topic名字后面的数字标号即代表分区。比如创建一个名为mytopic的主题，数据目录被分布到了3台机器。

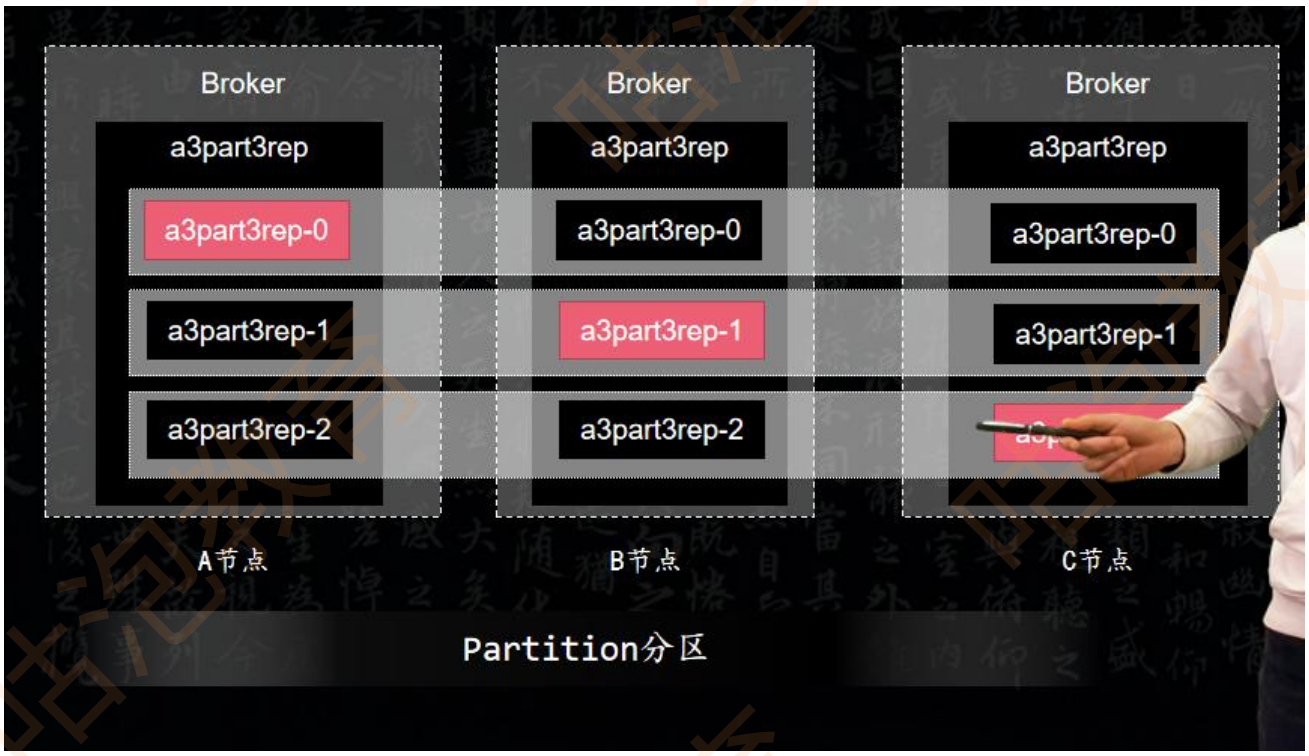
如图所示：

mytopic-0存在A节点，mytopic-1存在B节点，mytopic-2存在C节点。

3、Replica副本

另外，Kafa为了提高分区的可靠性，又设计了副本机制。我们创建Topic的时候，通过指定replication-factor副本因子，来确定Topic的副本数。当然，副本因子数必须小于等于节点数，否则会报错。这样就可以保证，绝对不会有一个分区的两个副本分布在同一个节点上，不然副本机制也失去了备份的意义了。

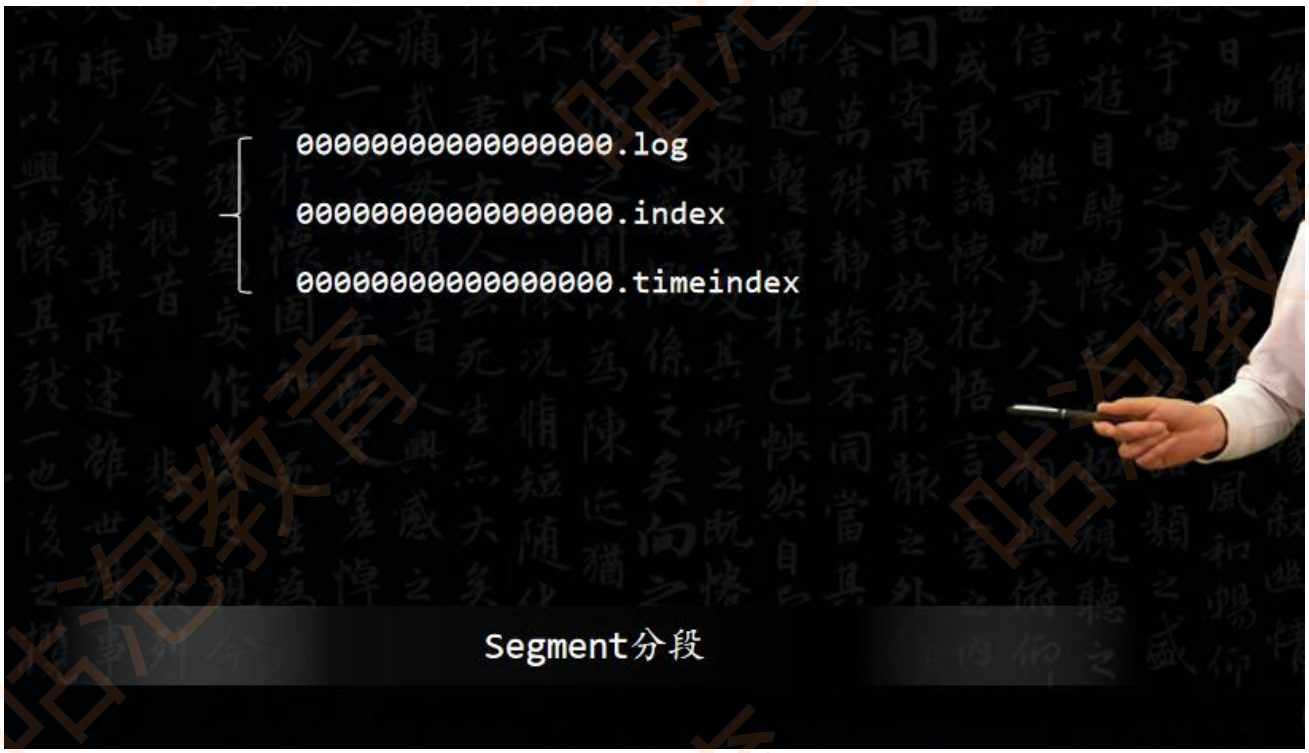
如图所示，创建了一个3个分区3个副本的Topic a3part3rep，被均匀分布到了3个Broker节点上，每个Broker节点互为备份。



这些所有的副本分为两种角色，Leader对外提供读写服务。Follower唯一的任务就是从Leader异步拉取数据，图中红色的副本为Leade，也被均匀分布在各个节点上，可以保证读写均匀，这样的设计也称为单调读一致性。

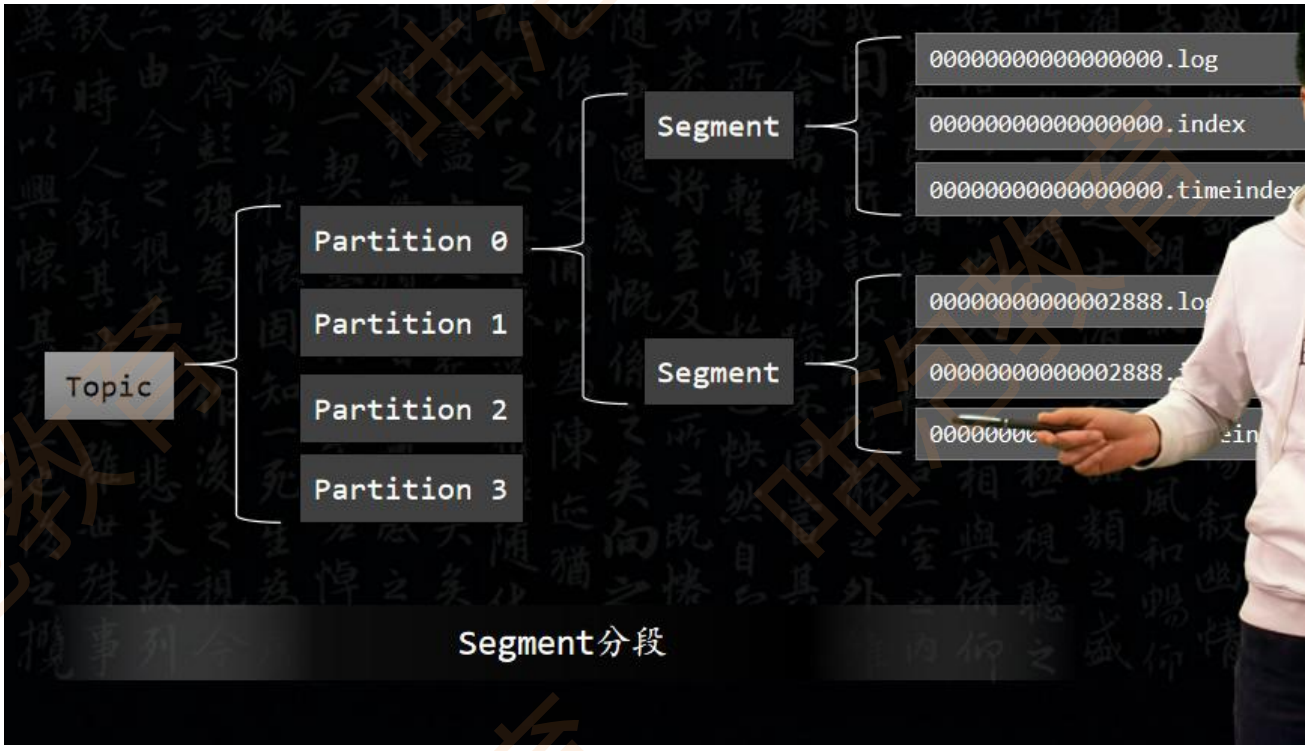
4、Segment分段

Kakfa为了防止Log不断追加导致文件过大，导致检索消息效率变低，一个Partition超出一定大小的时候，就被切割为多个Segment来组织数据。在磁盘上，每个Segment由一个log文件和2个index文件组成。



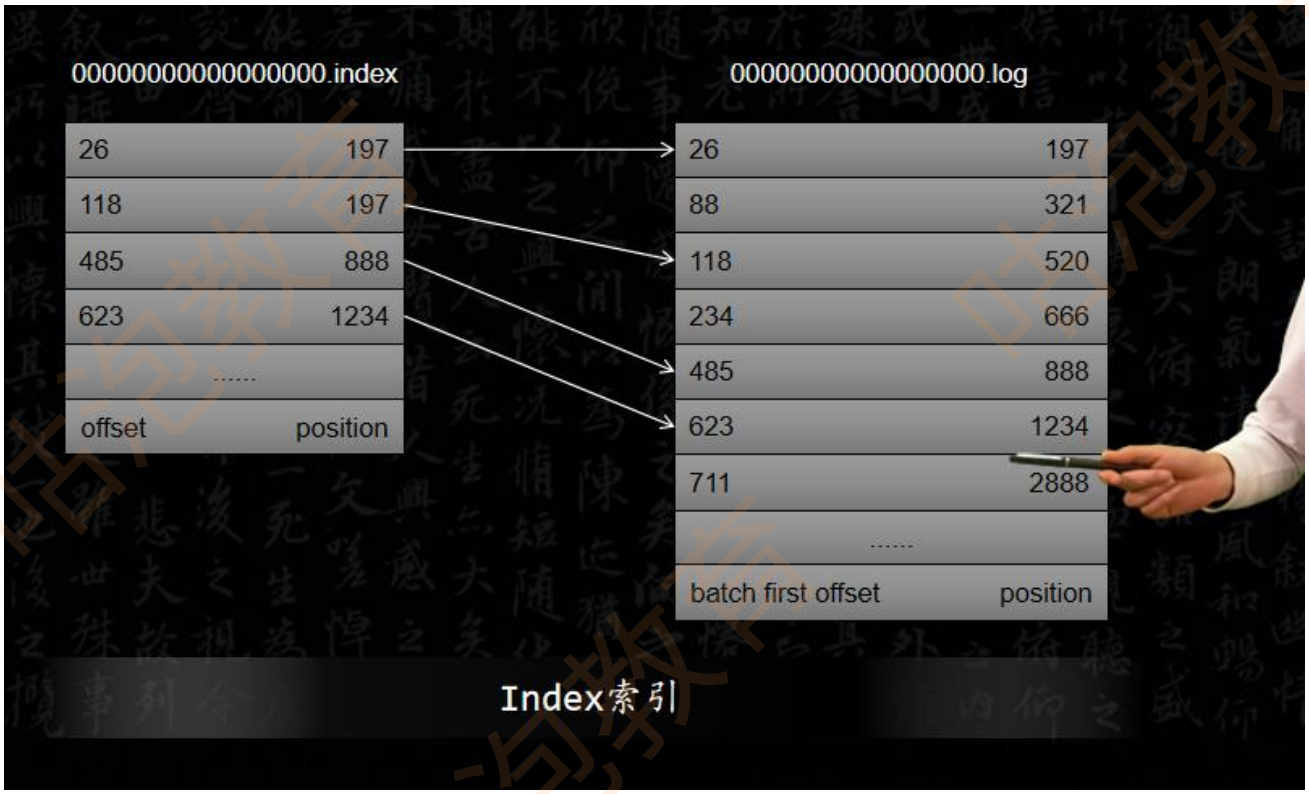
如图所示，这三个文件是成套出现的。其中.index是用来存储Consumer的Offset偏移量的索引文件，.timeindex是用来存储消息时间戳的索引文件，log文件就是用来存储具体的数据文件。

以切割时记录的Offset值作为文件的名字。它的文件结构是这样的，如图所示：



5、Index索引

前面我们讲到Kafka设计了两种索引，一种是偏移量索引文件，记录的是Offset和消息在Log文件中的位置映射关系。一种是时间戳索引文件，记录的是时间戳和Offset的关系。为了提高检索效率Kafka并不会为每一条消息都会建立索引，而是采用稀疏索引。也就是说隔一批消息才产生一条索引记录。如图所示：



我们可以通过以参数来设置索引的稀疏程度。

相对来说，越稠密的索引检索数据更快，但是会消耗更多的存储空间；

越的稀疏索引占用存储空间小，但是插入和删除时所需的维护开销也小。

同样，时间戳索引也是采用稀疏索引设计。由于索引文件是以Offset命名的，所以Kafka在检索数据的时候，是采用二分法查找，效率就非常快。

【结尾】

以上就是我对Kafka数据存储原理的理解！

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

11.25 RocketMQ为什么要放弃Zookeeper?

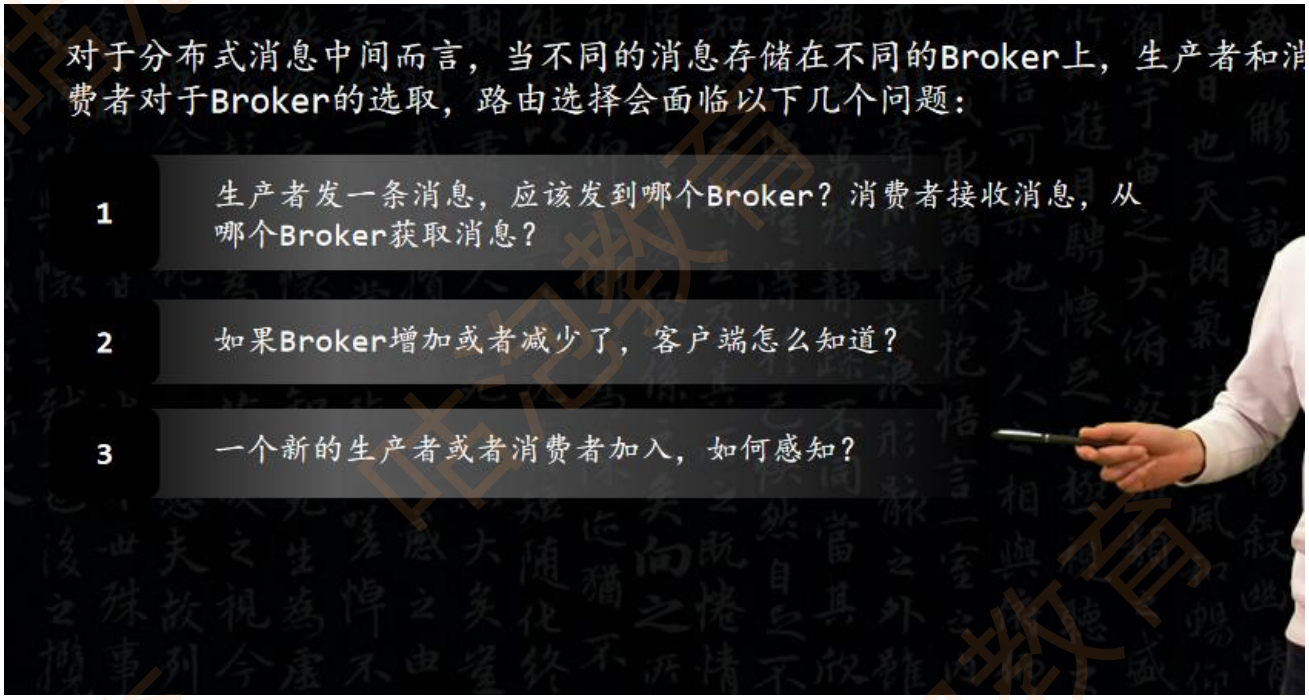
【引言】

一位7年工作经验的小伙伴去面架构师岗位，被问到这样一道面试题，说”RocketMQ为什么要放弃Zookeeper“。然后，想了很久好像没关注过，也不敢瞎猜。

那么今天，我给大家来聊一聊我对这个问题的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

1、注册中心



对于分布式消息中间件而言，当不同的消息存储在不同的Broker上，生产者和消费者对于Broker的选取，路由选择会面临以下几个问题：

1、生产者发一条消息，应该发到哪个Broker？消费者接收消息，从哪个Broker获取消息？

2、如果Broker增加或者减少了，客户端怎么知道？

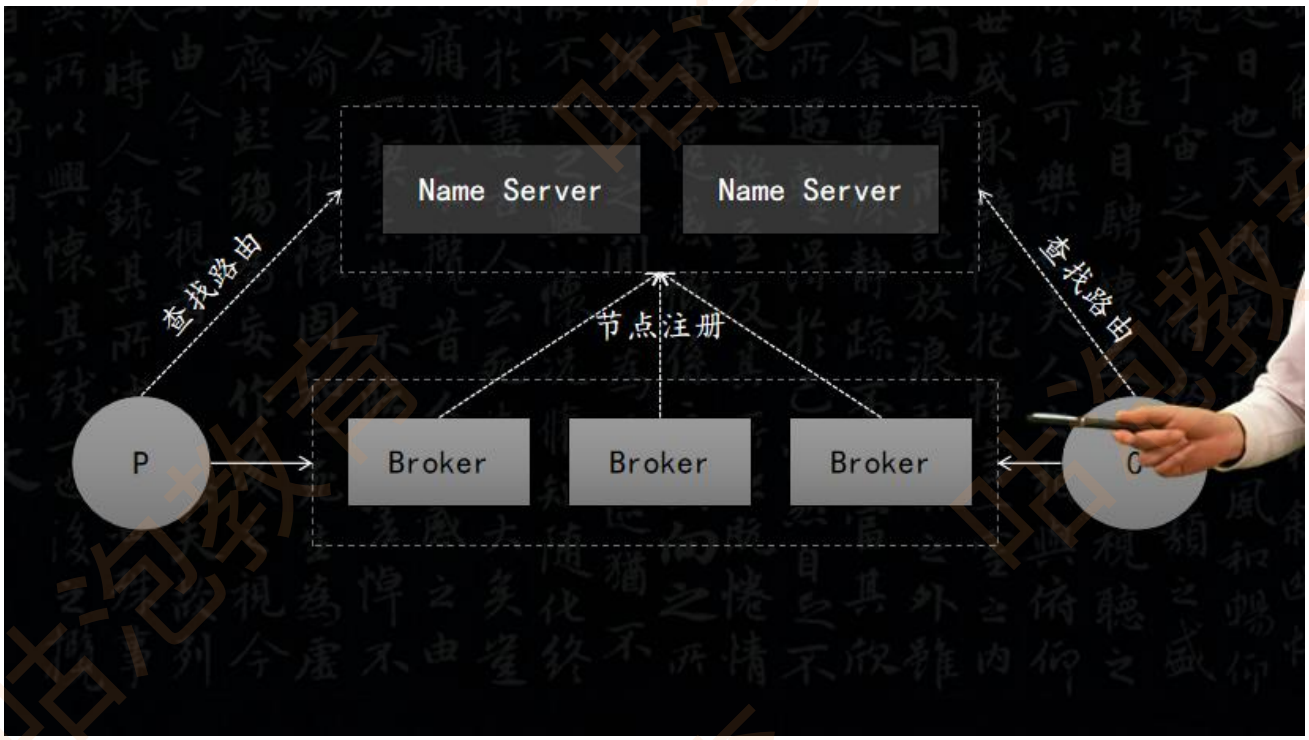
3、一个新的生产者或者消费者加入，如何感知？

所以，只要是跟分布式服务调用的场景，都需要一个注册中心，在RocketMQ当然也中需要有一个这样的角色来管理Broker的信息。

而Kafka就是用了现成的Zookeeper，但是RocketMQ却偏偏没有这么做，而是自己实现了一个服务，这个服务叫做NameServer。

我们可以把NameServer理解为是RocketMQ的路由中心，每一个NameServer节点都保存了全量的路由信息。为了保证高可用，NameServer自身也可以做集群的部署。它的作用有点像Eureka或者Redis的Sentinel。

也就是说，Broker会在NameServer上注册自己，Porducer和Consumer用NameServer来发现Broker。



每个Broker节点在启动时，都会根据配置遍历NameServer列表。

与每个NameServer建立TCP长连接，注册自己的信息，之后每隔30s发送心跳信息。

如果Broker挂掉了，不发送心跳了，NameServer怎么发现呢?

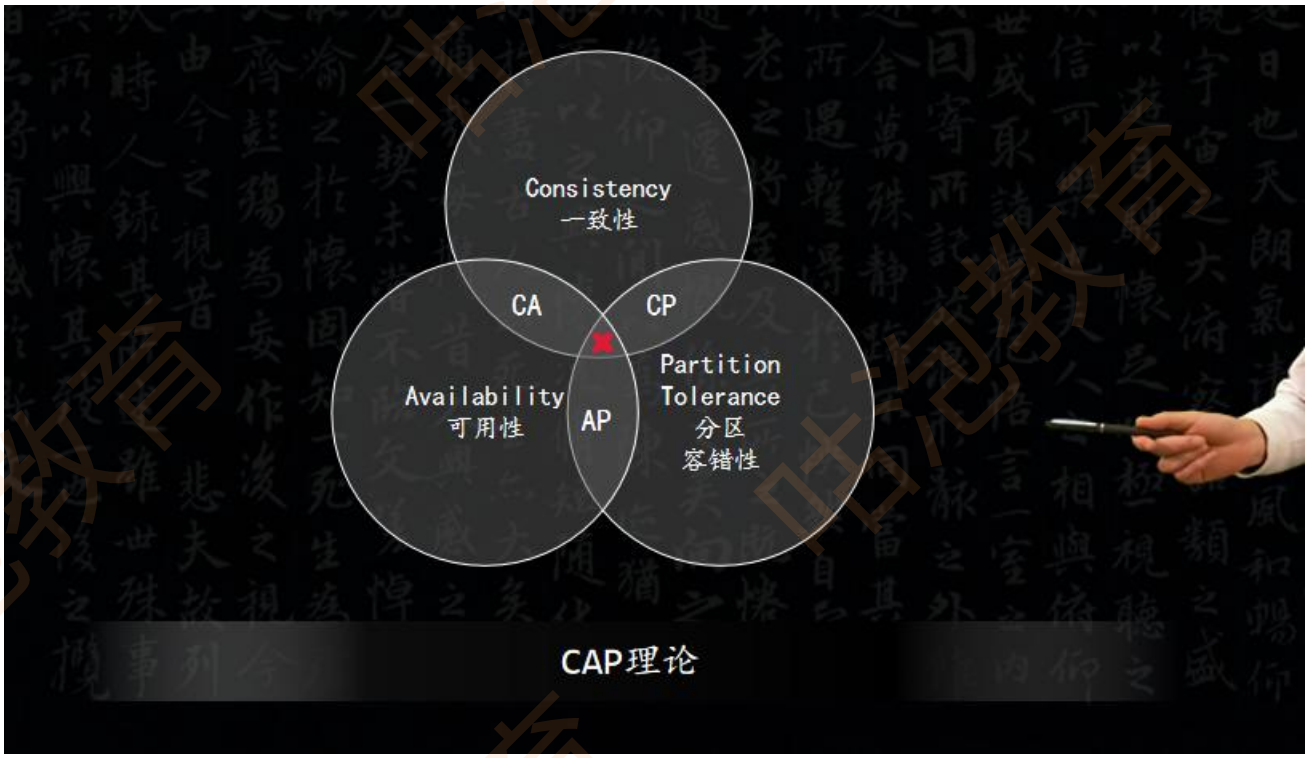
所以，除了主从注册，还有定时探活。每个NameServer每隔10s检查一下各个Broker的最近一次心跳时间，如果发现某个Broker超过120s都没发送心跳，就认为这个Broker已经挂掉了，会将其从路由信息里移除。

2、放弃理由

既然，Nameserver的作用也是用来管理Broker的服务的，也就是服务注册与发现，那为什么不直接用Zookeeper、Consul、etcd、Eureka这样的组件呢?

实际上在RocketMQ的早期版本中，跟Kafka一样也是用Zookeeper实现服务管理的，但到RokcetMQ开源的时候去掉了ZooKeeper依赖，转而采用自己的NameServer。

因为，RocketMQ是一个保持最终一致性的架构设计，它架构决定了它只需要一个轻量级的元数据服务器就足够了，而不需要像Zookeeper这样的强一致性解决方案。不依赖另一个中间件，从而减少整体维护成本。



根据著名的CAP理论，在一致性(Consistency)、可用性(Availability)、分区容错性(Partiton

Tolerance)中，Zookeeper实现了CP，而NameServer选择了AP，放弃了实时一致性。

【结尾】

以上就是我对RocketMQ为什么要放弃Zookeeper的理解！

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

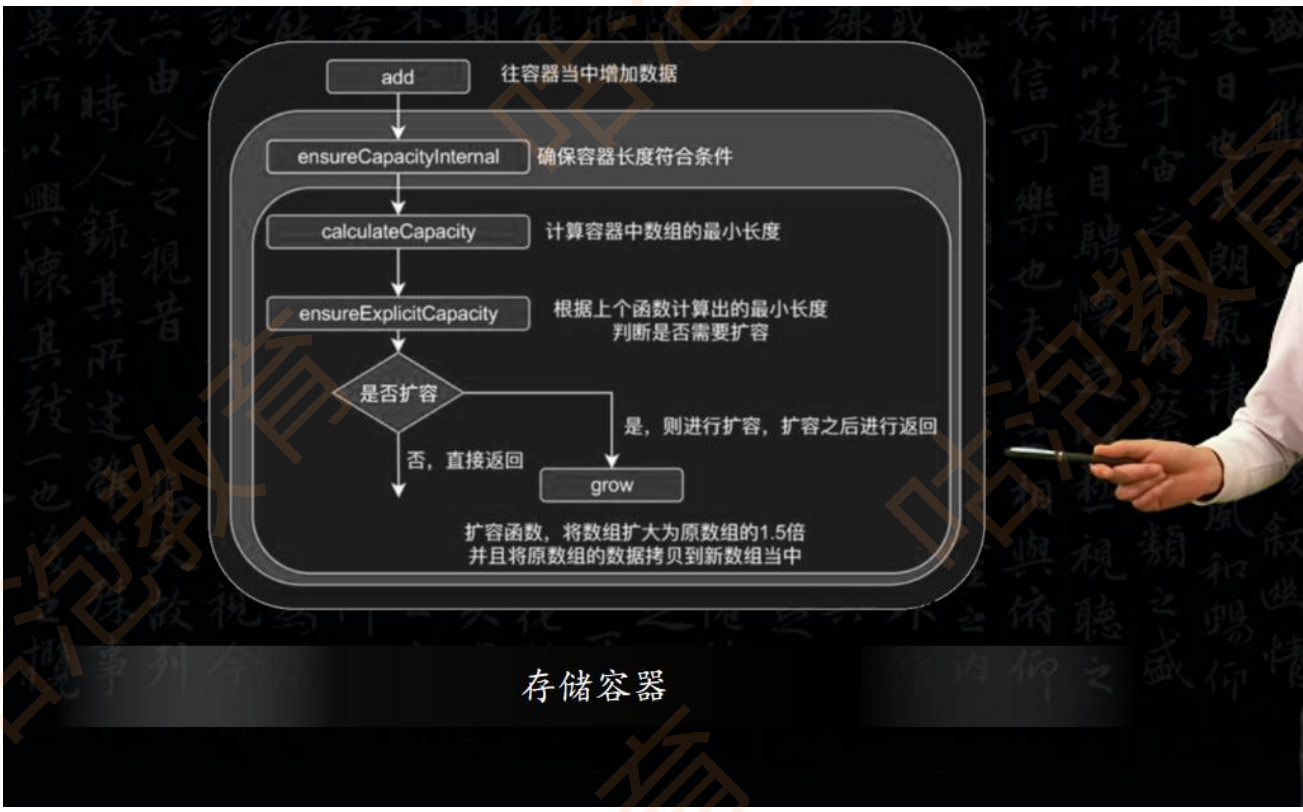
11.26 请你谈一谈ArrayList自动扩容机制的实现原理

【引言】

一位3年工作经验的小伙伴面试时被问到，说请你谈一谈ArrayList自动扩容机制的实现原理。这个问题对于稍微看过一点源码的小伙伴来说，其实非常简单。下面我给大家分享一下我对这个问题的理解。

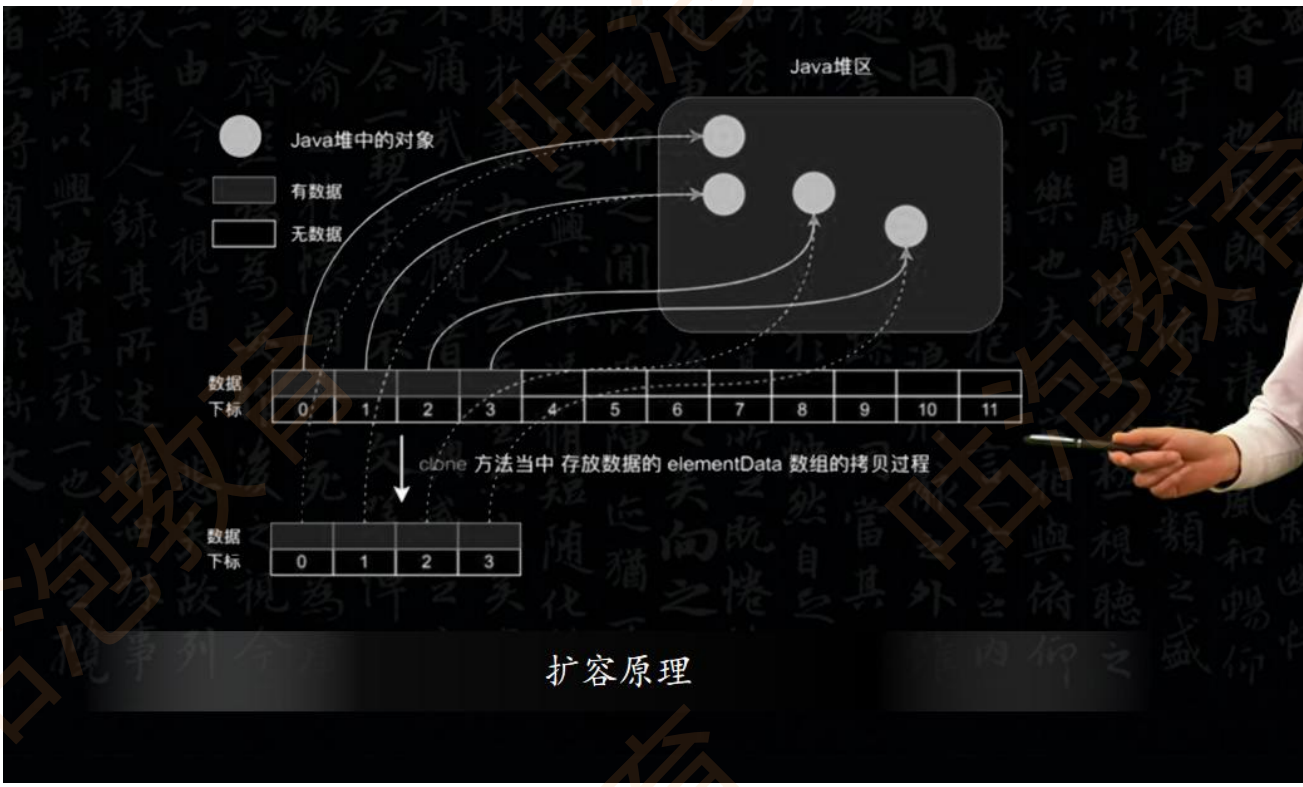
另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

1 存储容器



ArrayList是一个数组结构的存储容器，默认情况下，数组的长度是10。当然，我们也可以在构建ArrayList对象的时候，直接指定初始长度。那么，随着程序的运行，不断地往ArrayList中添加数据，当添加的数据达到10个的时候，ArrayList就没有足够的容量来存储后面添加的数据。那么，这个时候，ArrayList会触发自动扩容机制。

2 扩容原理



扩容的流程其实也很简单，首先创建一个新的数组，这个数组的长度是原来数组长度的1.5倍。然后，使用Arrays.copyOf()方法把原数组中的数据拷贝到新数组中。扩容完成之后，再把当前要添加的数据加入到新的数组中，从而完成动态扩容的过程。

【结尾】

好了，以上就是我对这个问题的理解。作为一个业务程序员，虽然工作的性质是让大家去写CRUD，不需要过多的去关注技术的底层实现原理。但是，在未来的职业晋升过程中，对技术的理解程度非常重要。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

11.27 谈谈你对JVM中主要GC算法的理解

【引言】

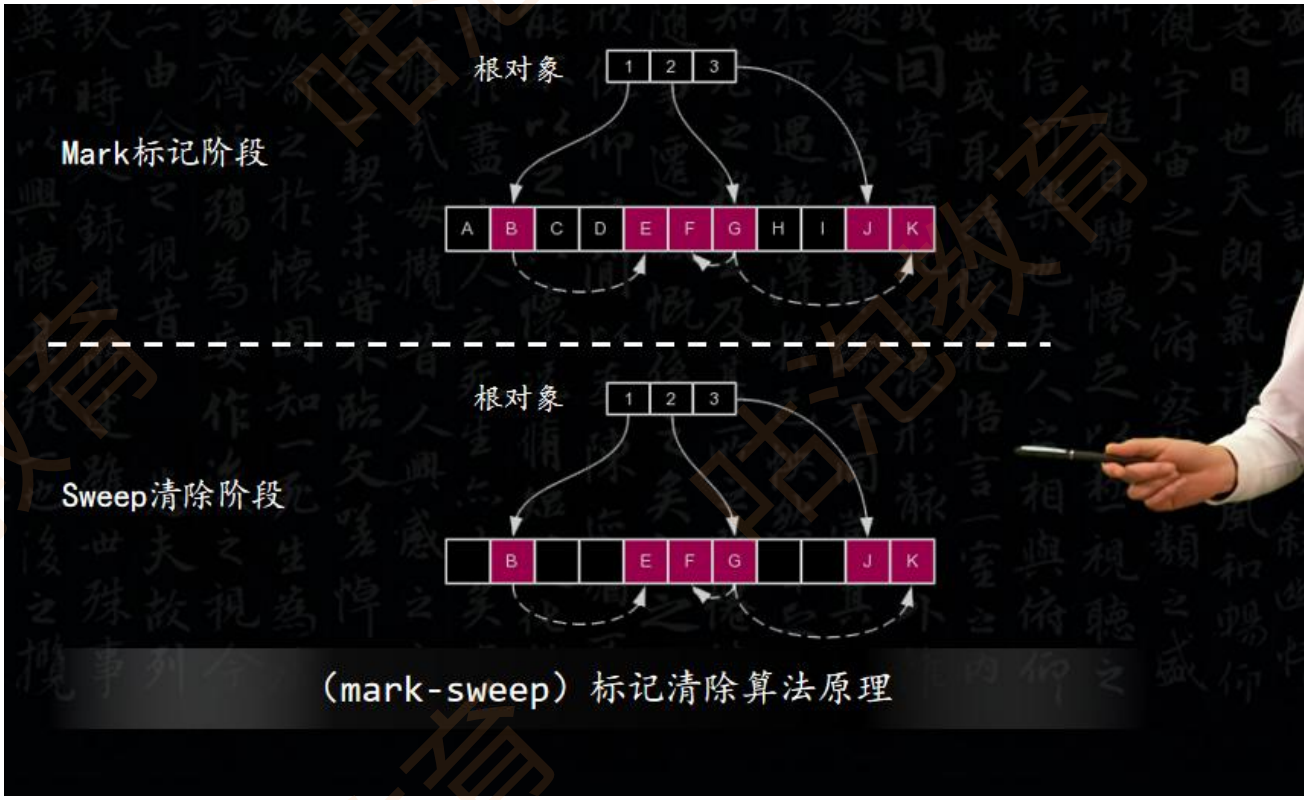
有位工作五年的小伙伴面试被问到JVM相关的问题，说请你谈谈你对JVM中主要GC算法的理解，我给大家分享一下我的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

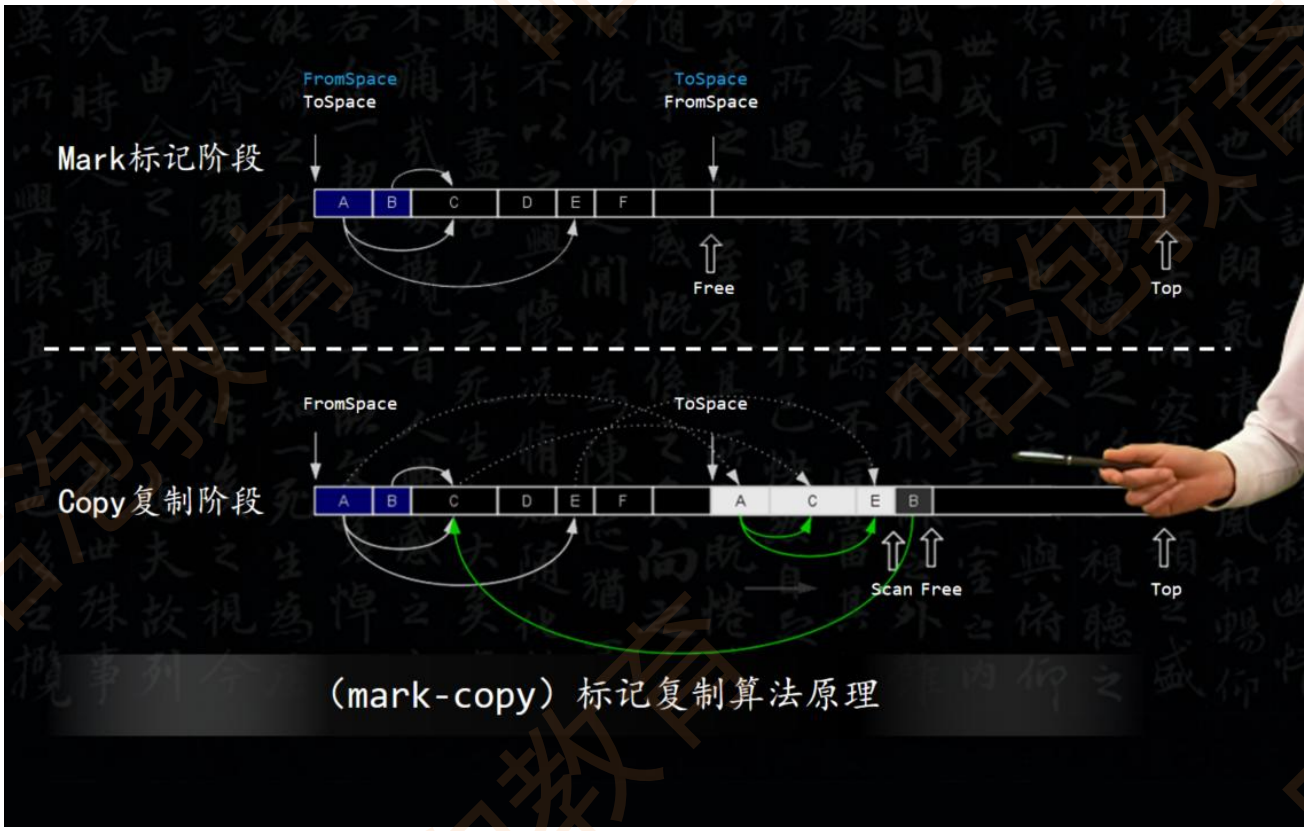
1 主要垃圾回收算法

GC翻译过来叫做垃圾回收，那么JVM中主要的垃圾回收算法有三种，分别是：标记清除算法、标记复制算法和标记整理算法。

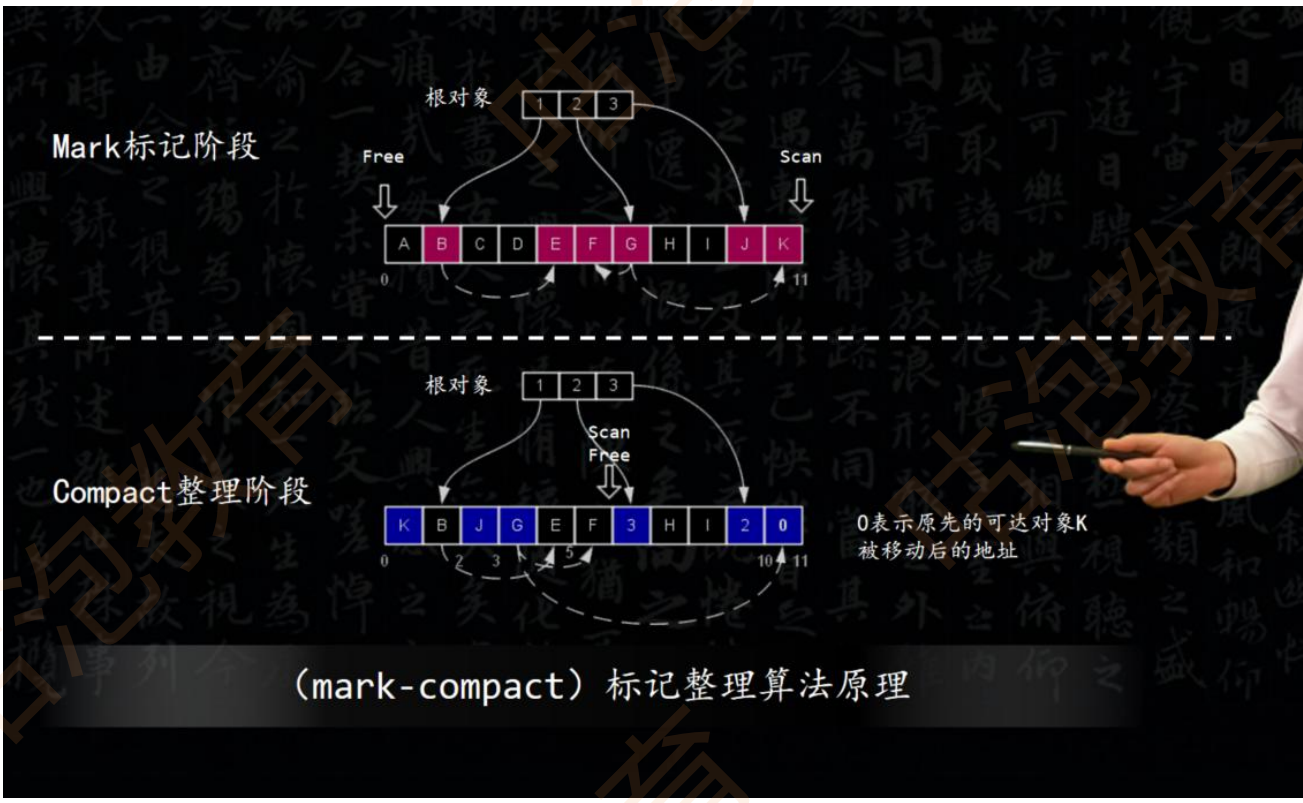
下面，我介绍一下每种算法的核心原理：



首先是，标记清除算法。它是将存活的对象打上标记，那么没有被标记的对象就是需要被回收的垃圾对象，这些垃圾对象会被垃圾回收器直接回收。



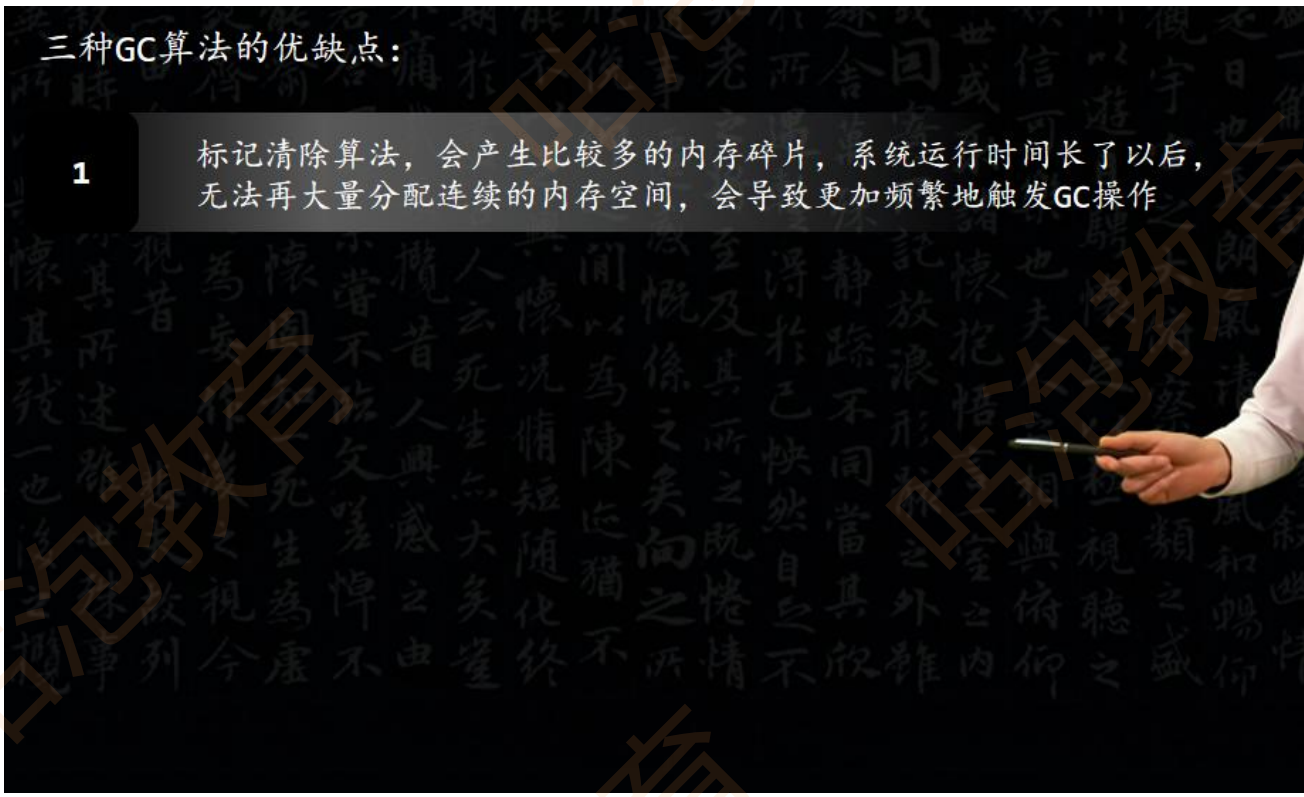
然后是，标记复制算法。它是把内存分为两等份，每次只使用其中的一份，等到正在使用的这部分内存满了之后，就会标记出存活的对象，然后把存活的对象拷贝到另一部分闲置的内存中，那留在另一部分内存中的对象，会全部被垃圾回收器回收。那么，原来空闲的内存空间就会变成使用中的状态，而原来使用中的内存空间会被闲置出来继续使用。这就是标记复制算法的一次完整的GC。然后，一直重复这个循环。



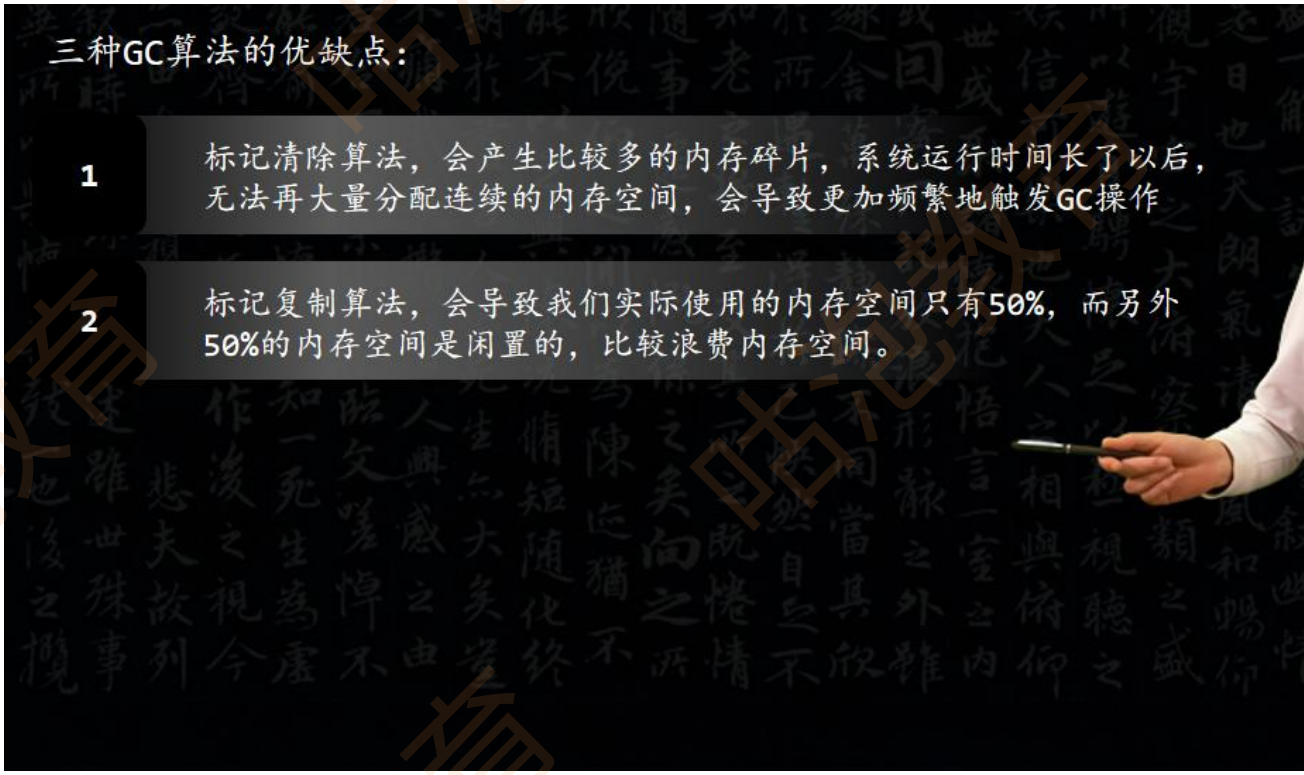
最后是，标记整理算法。它是先标记出存活的对象，然后，把所有存活的对象整理到内存空间的另一端，而没有被标记的对象就是可以被覆盖或者是释放。

2 优缺点分析

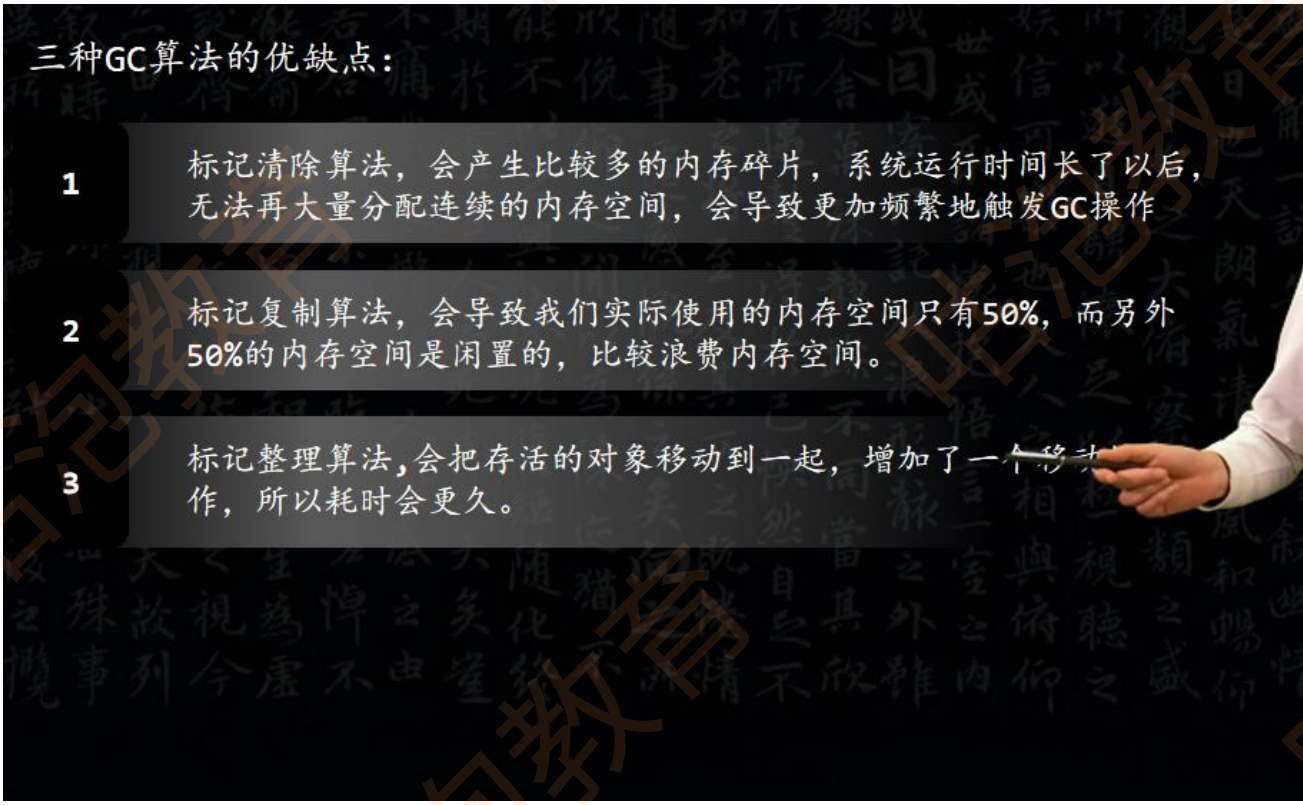
接下来，分析一下三种GC算法的优缺点。



首先来看，标记清除算法。它主要的缺点是会产生比较多的内存碎片，而这些内存碎片，随着系统运行时间的推移，系统运行时间长了以后，无法再大量分配连续的内存空间。这样的话，就会导致更加频繁地触发GC操作。



然后是，标记复制算法，它主要的缺点是，会导致我们实际使用的内存空间只有50%，而另外50%的内存空间是闲置的。所以，比较浪费内存空间。而且，如果大量复制对象的话，垃圾回收的耗时就比较长了，所以，这种算法，更加适合处理存活对象少，垃圾对象比较多的场景。



最后是，标记整理算法。它的本质和标记清除算法是基本差不多，只不过标记整理算法多了一个动作，会把存活的对象移动到一起。从垃圾收集和清理效率来看，增加了一个移动的动作，所以耗时会更久。但是，新分配对象的耗时就会下降。

3 分代回收

因为Java对象基本上都是临时的对象，很快就会被回收。所以，JVM的内存是分代的设计，根据对象在内存中的存活时间，分为年轻代、老年代和永久代。

那么年轻代呢，采用的是标记复制算法，在每次复制的时候，存活下来的对象会很少。

老年代呢，是经历过几次GC的对象，JVM会认为它可能继续存活下去，就不太适合采用标记复制算法，所以，老年代采用的是标记清除算法，比如CMS这种回收器就是采用标记清除的方式。

永久代呢，表示一直会存活的对象，只有在触发Full GC的时候才会被回收。所以，永久代对象创建过多的话，比较容易出现内存溢出。最典型的场景就是，在JSP页面比较多的情况，容易出现永久代内存溢出。

【结尾】

好了，以上就是我对垃圾回收算法的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

11.28 谈谈你对Java线程5种状态流转原理的理解

【引言】

今天，有位工作5年的小伙伴被问到这样一道面试题，说谈谈你对Java线程5种状态流转原理的

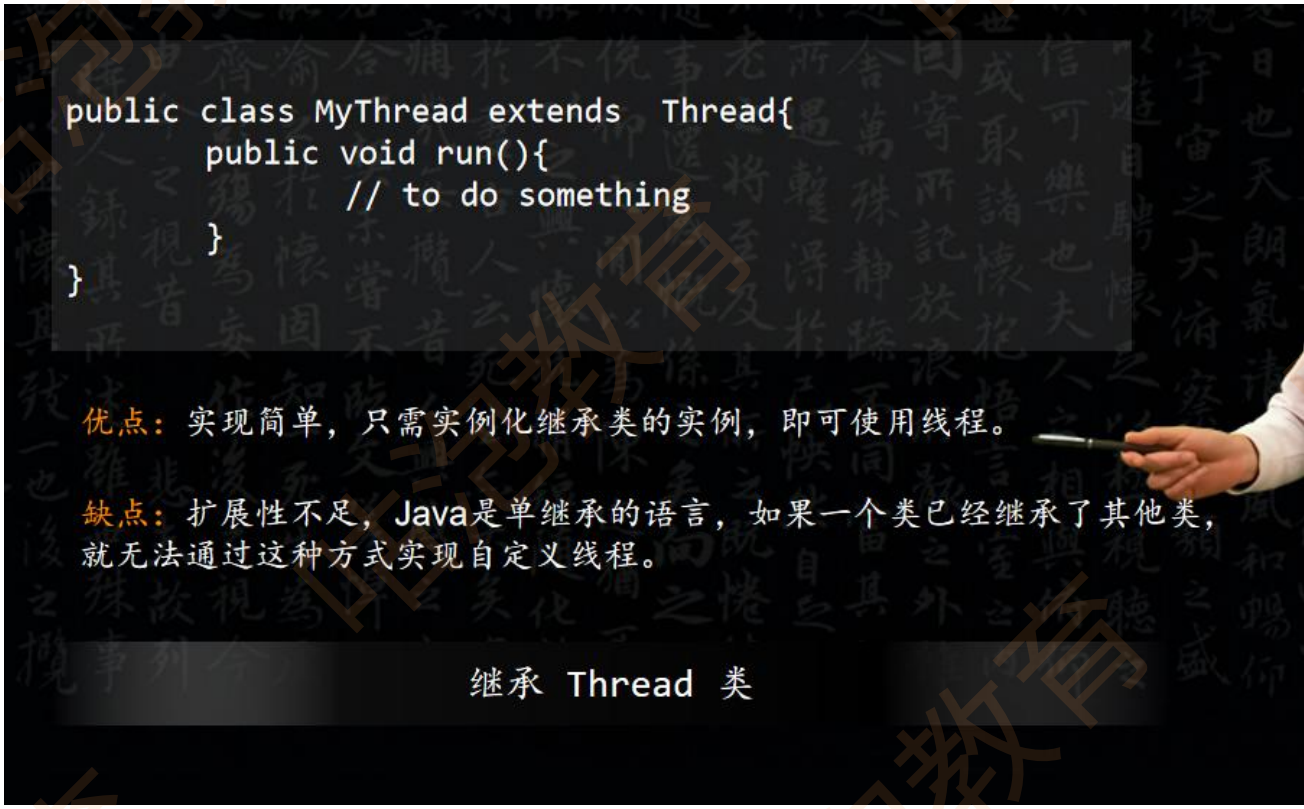
理解。当时，平时只关注过线程如何定义和使用，对于线程状态流转脑海一片空白，完全懵了。于是找到我，希望我拍一期视频。

今天，我给大家分享一下我的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的面试通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

1、线程定义

回答线程流转原理这个问题之前，我们先来回忆一下JDK中3种自定义线程的方式以及它们的优缺点。

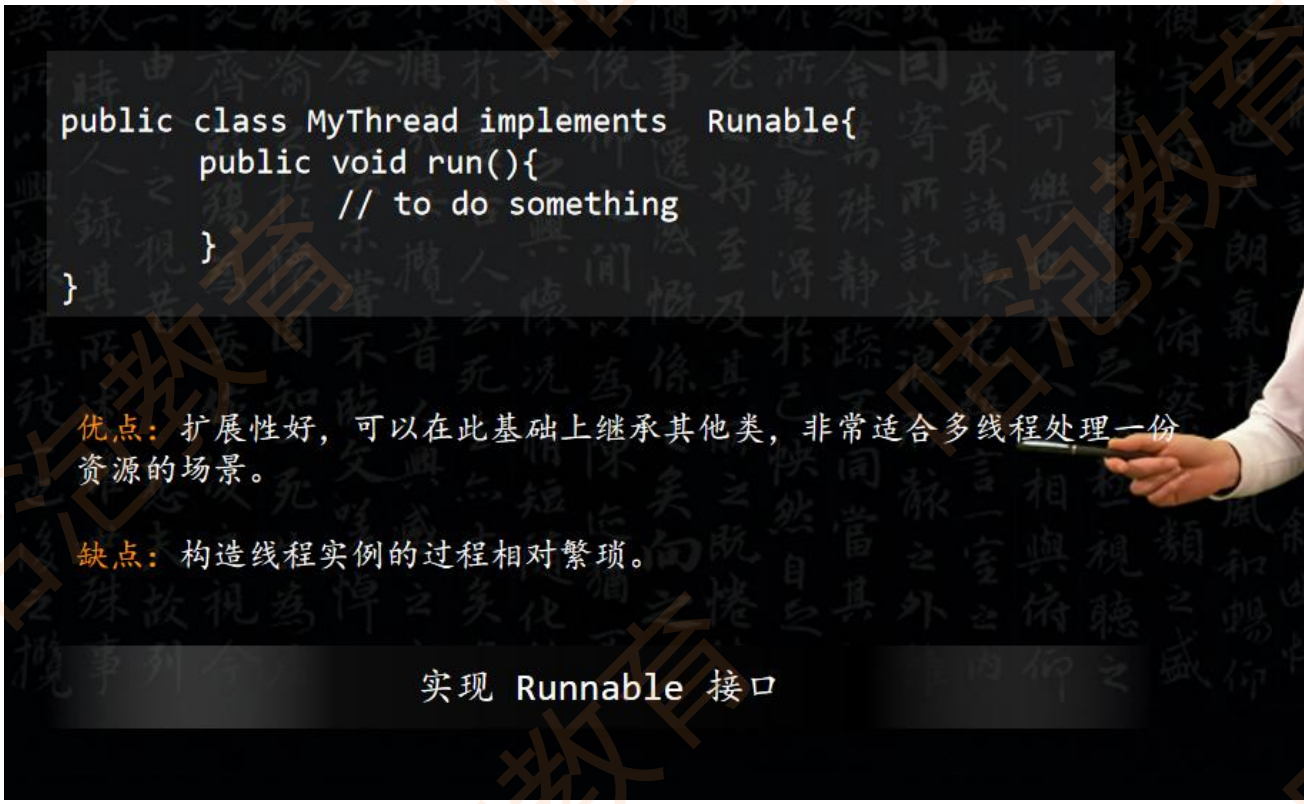


第1种：是继承Thread类。如代码所示：

```
public class MyThread extends  Thread{
    public void run(){
        // to do something
    }
}
```

这种方式的优点是：实现简单，只需实例化继承类的实例，即可使用线程。

它的缺点是：扩展性不足，Java是单继承的语言，如果一个类已经继承了其他类，就无法通过这种方式实现自定义线程

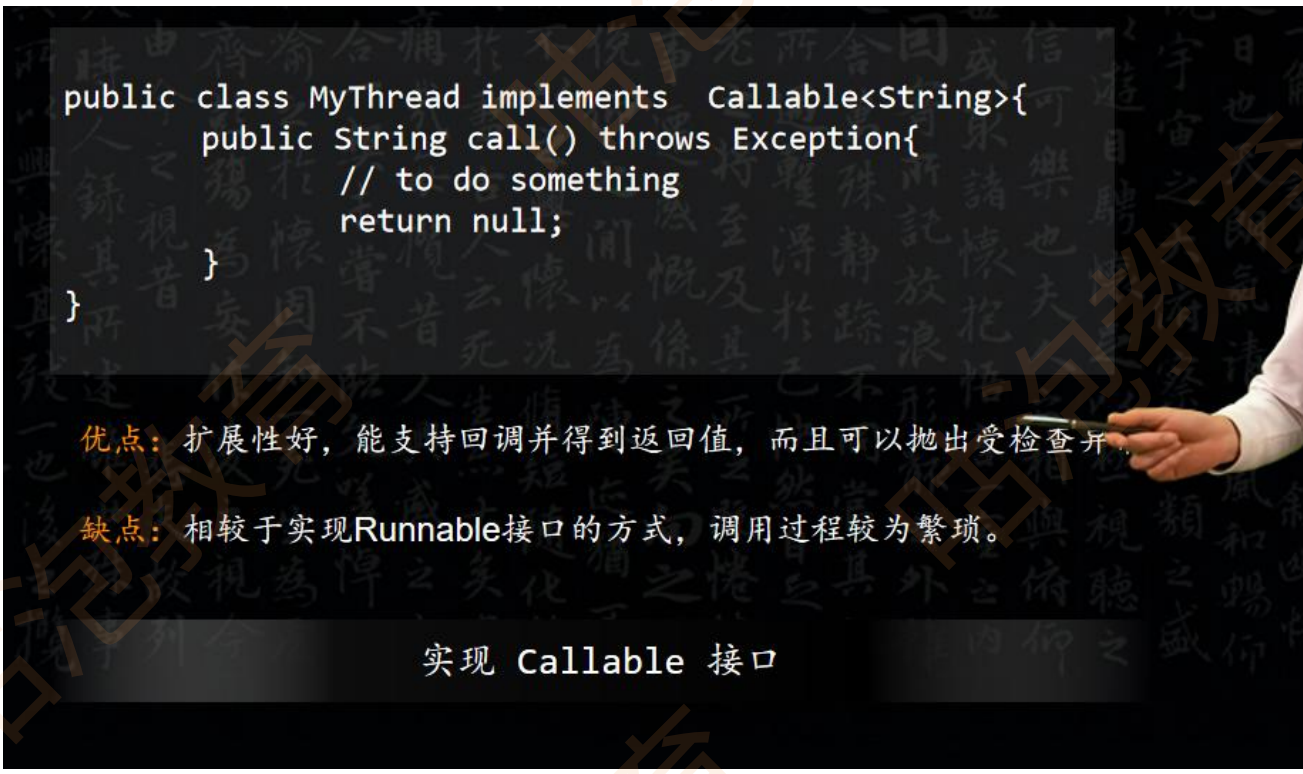


第2种：是实现 Runnable 接口，如代码所示：

```
public class MyThread implements  Runable{
    public void run(){
        // to do something
    }
}
```

它的优点是：扩展性好，可以在此基础上继承其他类，非常适合多线程处理一份资源的场景

它的缺点是：构造线程实例的过程相对繁琐一点



第3种：是实现Callable接口，如代码所示：

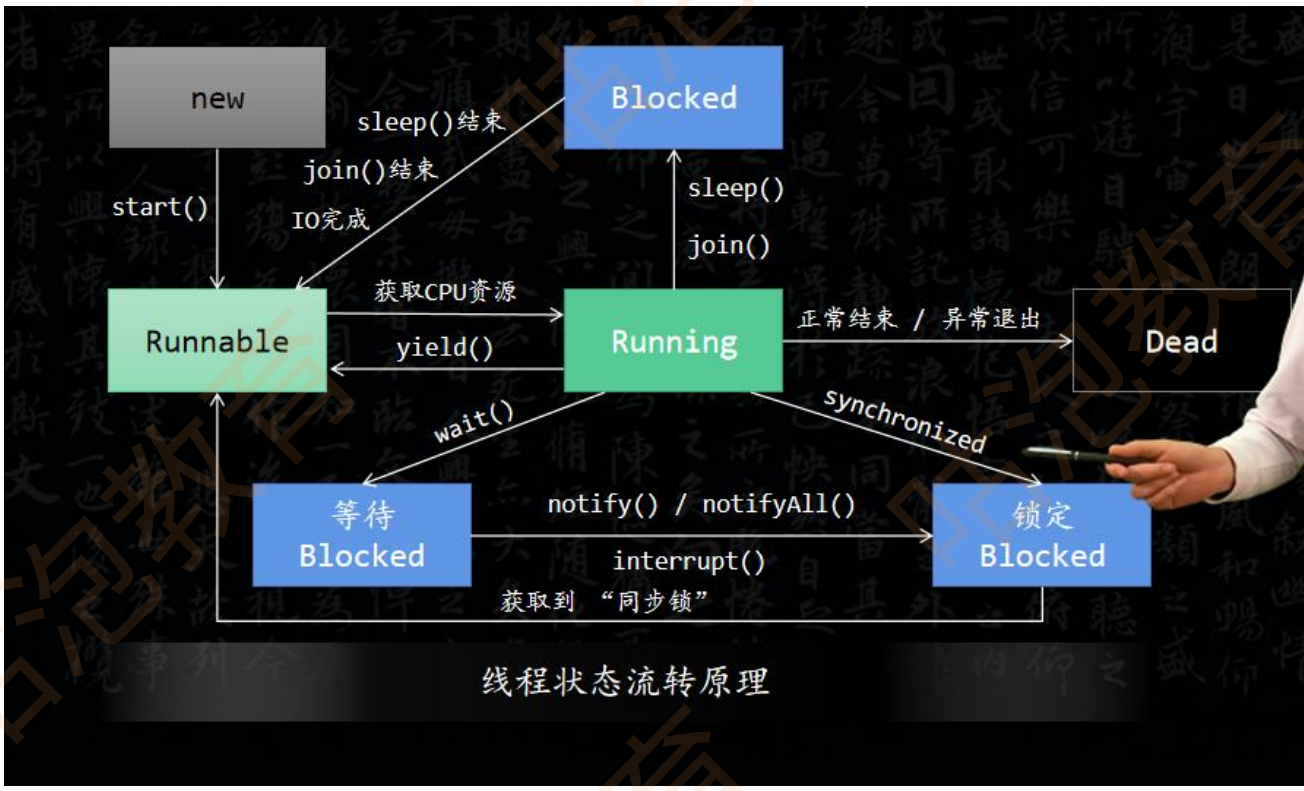
```
public class MyThread implements  Callable<String>{
    public String call() throws Exception{
        // to do something
        return null;
    }
}
```

它的优点是：扩展性好，能支持回调并得到返回值，而且可以抛出受检查异常。

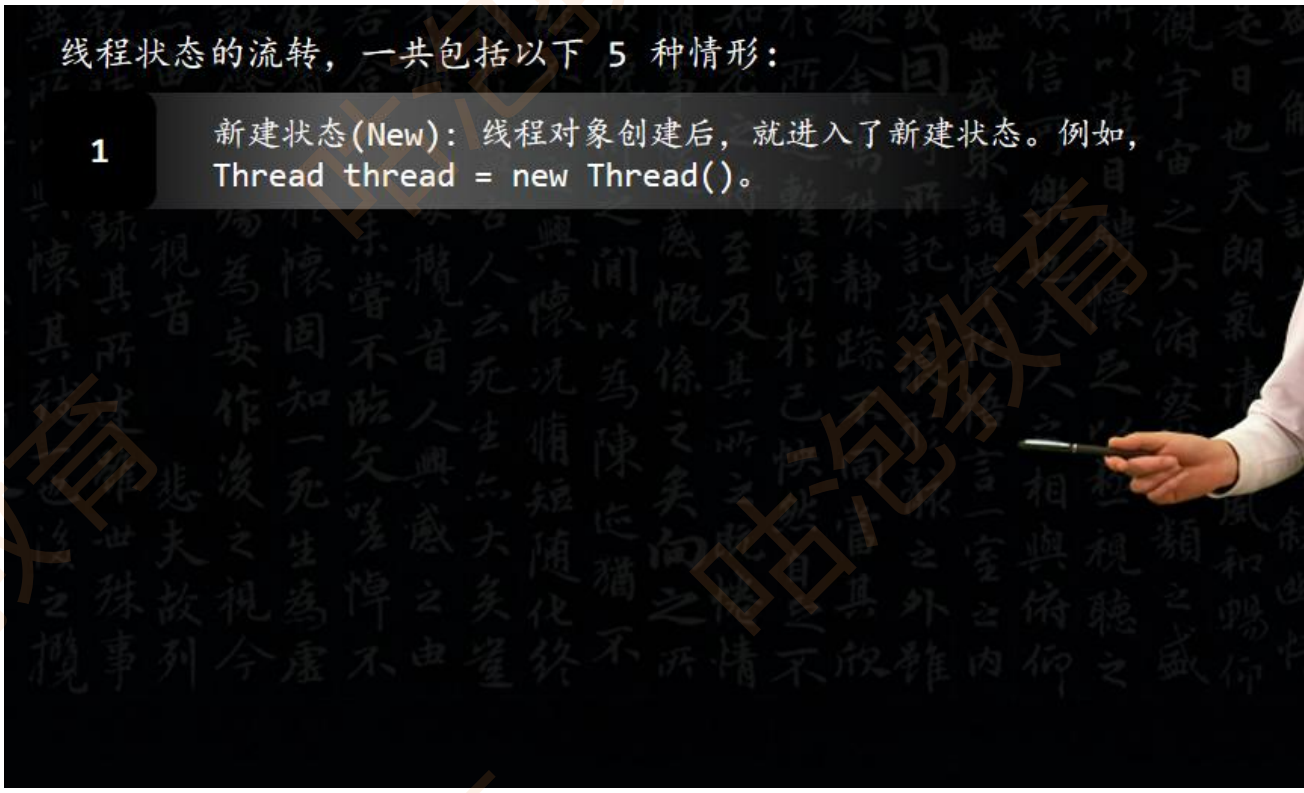
它的缺点是：相较于实现Runnable接口的方式，调用过程较为繁琐。

2、线程状态流转原理

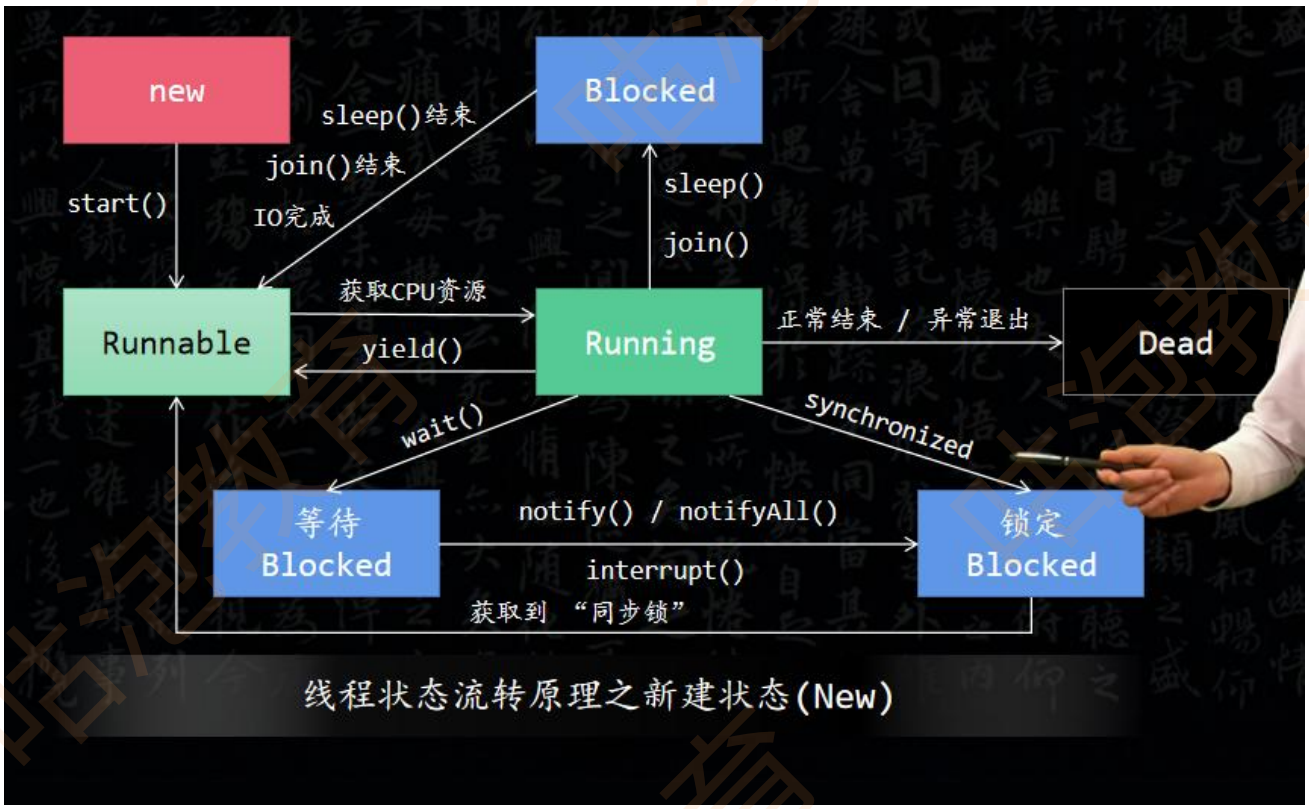
首先来看这样一张图，它涵盖了Java 中多线程各重要知识点。如果掌握了此图，Java 中的多线程也就基本上掌握了。

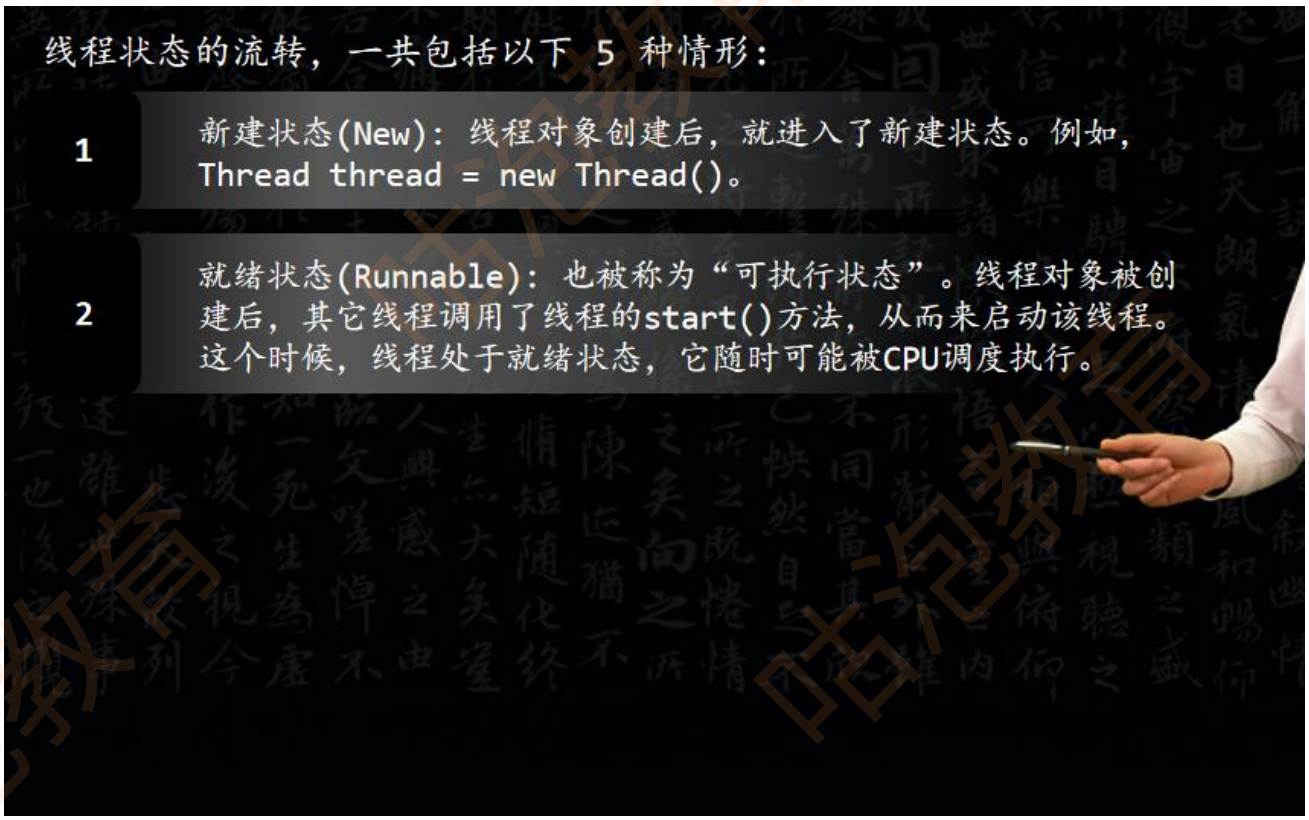


从图中可以看出，线程状态的流转，一共包括以下 5 种情形:

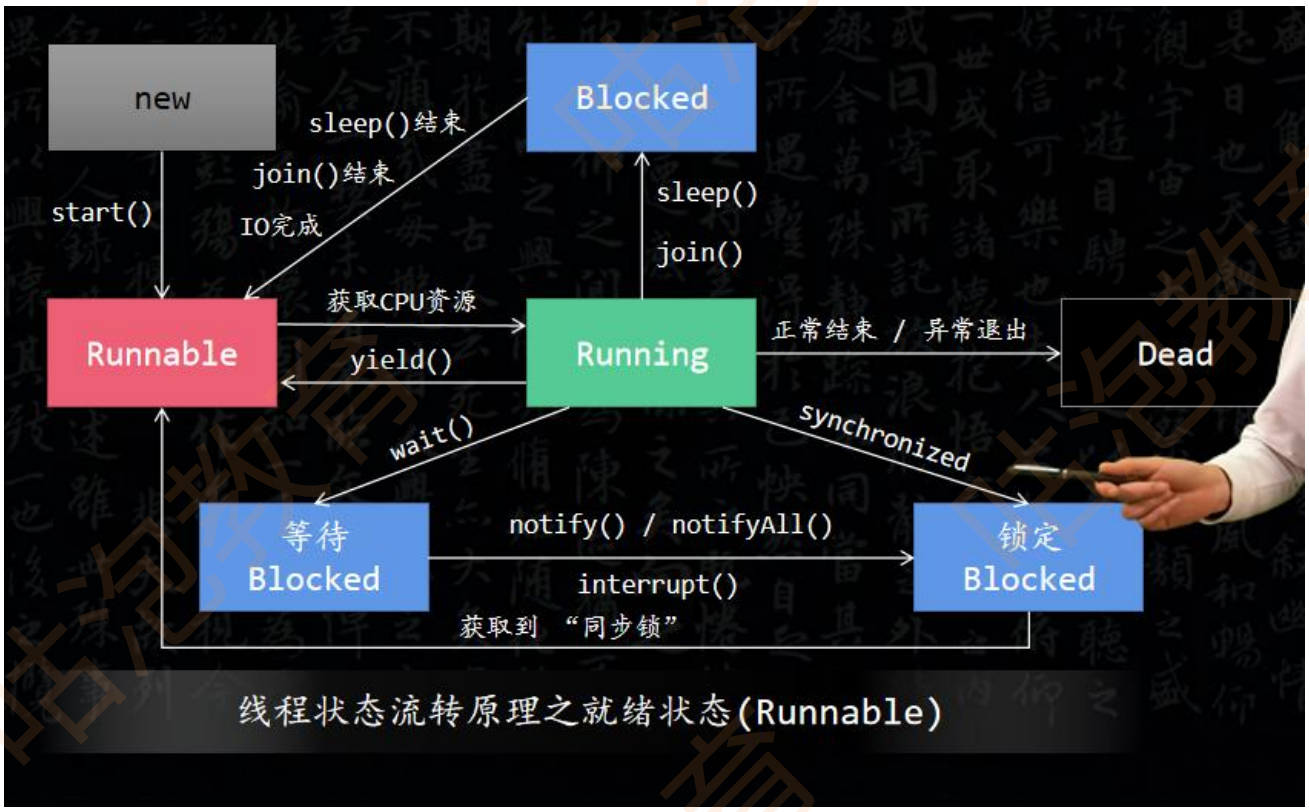


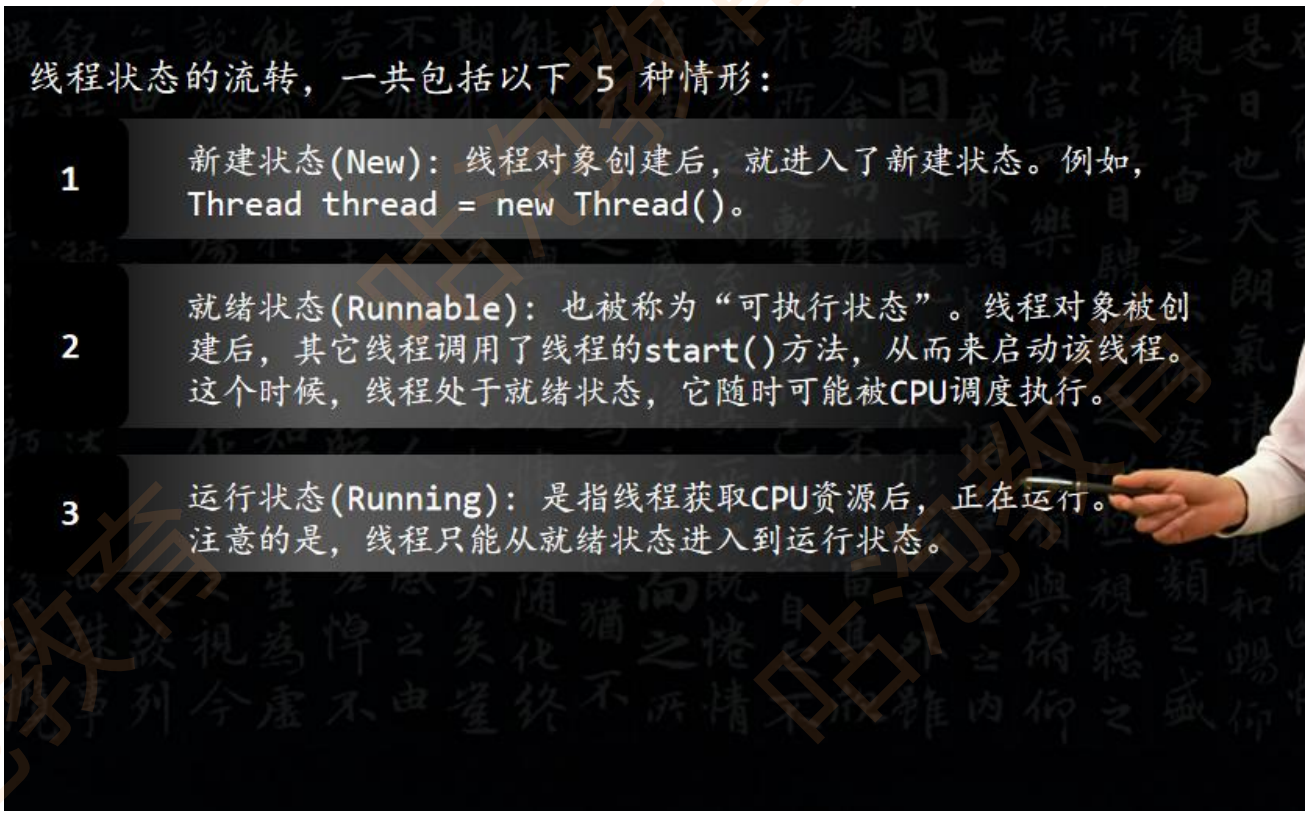
1. 新建状态(New): 线程对象创建后，就进入了新建状态。例如，Thread thread = new Thread()。



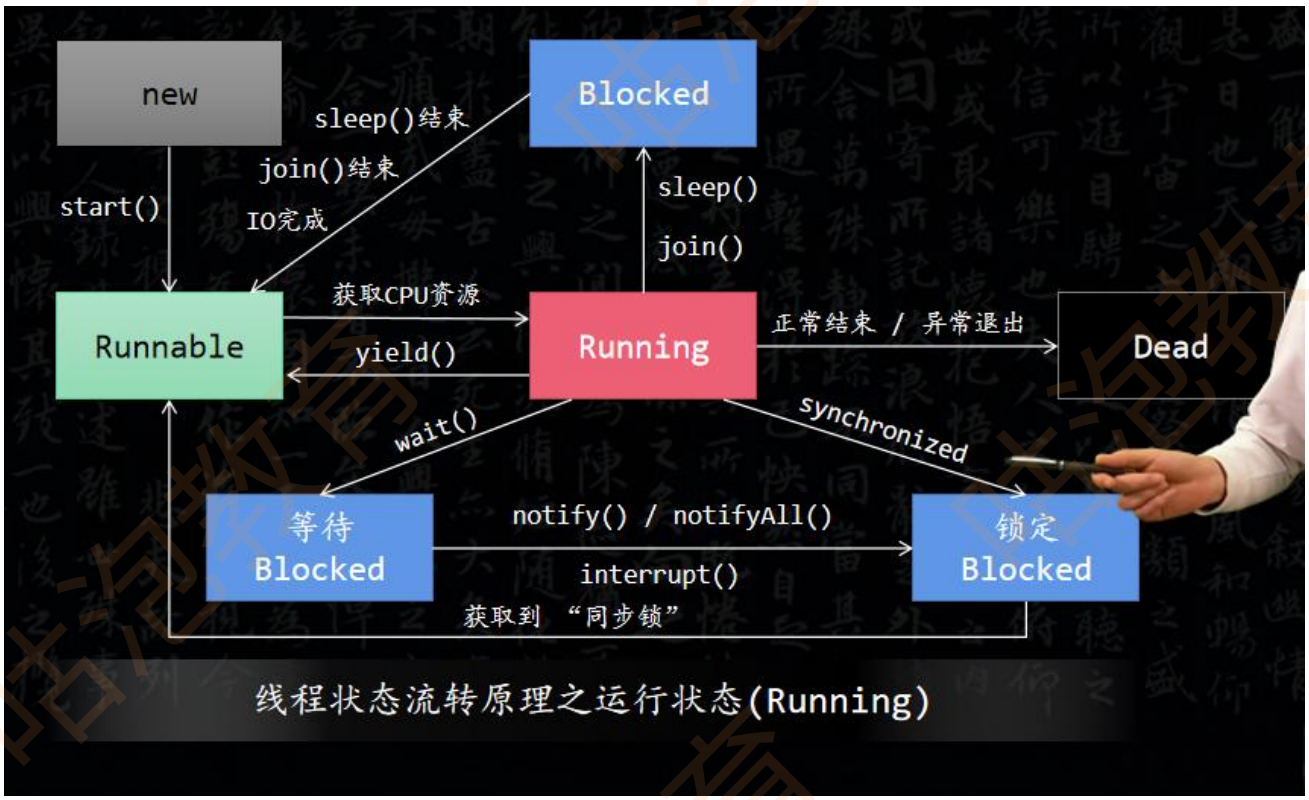


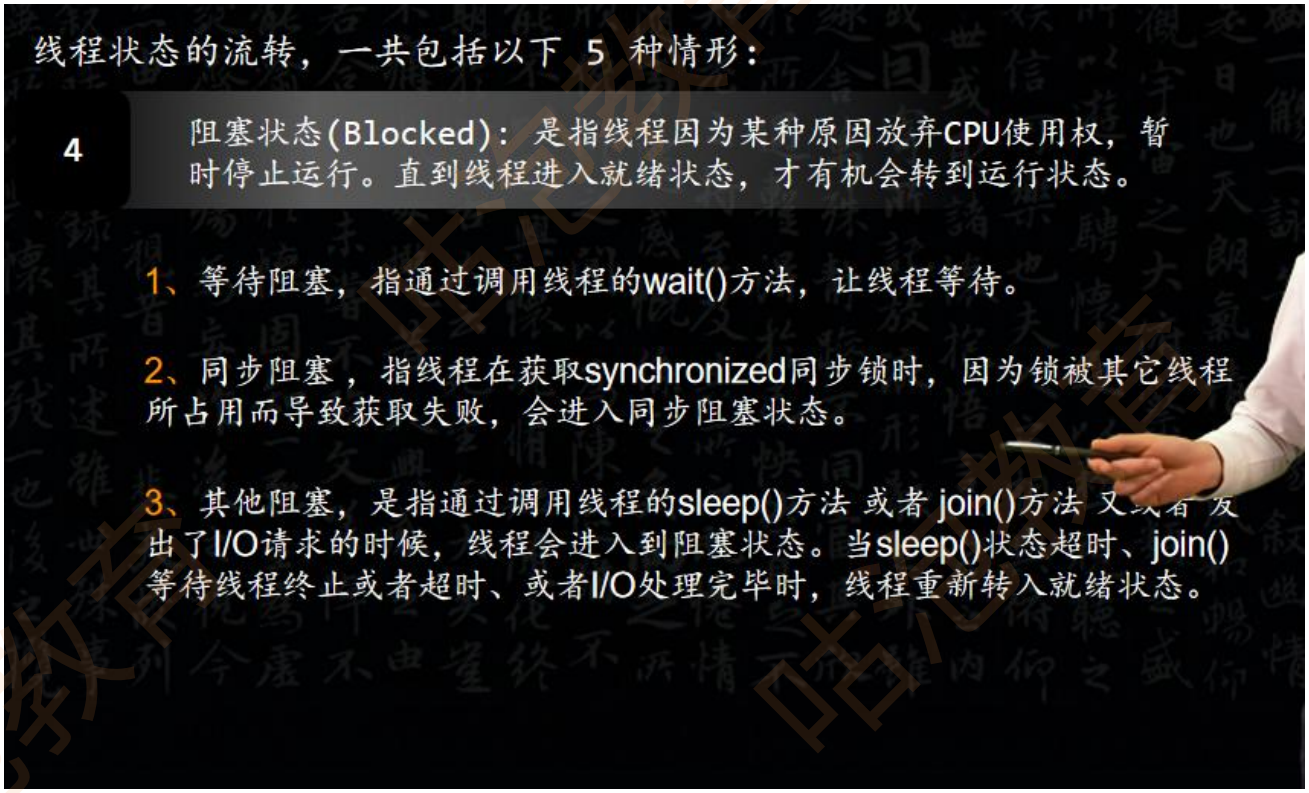
2. 就绪状态(Runnable): 也被称为“可执行状态”。线程对象被创建后，其它线程调用了线程的start()方法，从而来启动该线程。这个时候，线程处于就绪状态，它随时可能被CPU调度执行。





3. 运行状态(Running): 是指线程获取CPU资源后，正在运行。需要注意的是，线程只能从就绪状态进入到运行状态。





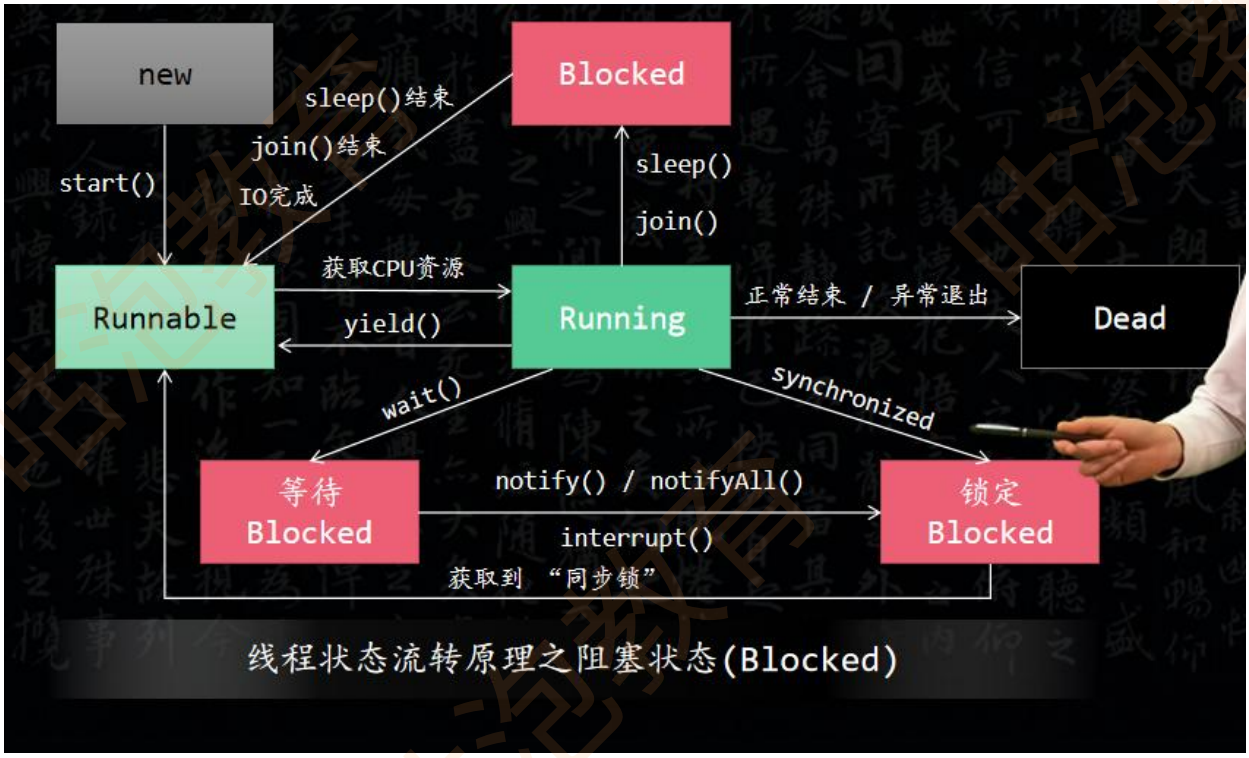
4. 阻塞状态(Blocked): 是指线程因为某种原因放弃CPU使用权，暂时停止运行。直到线程进入就绪状态，才有机会转到运行状态。

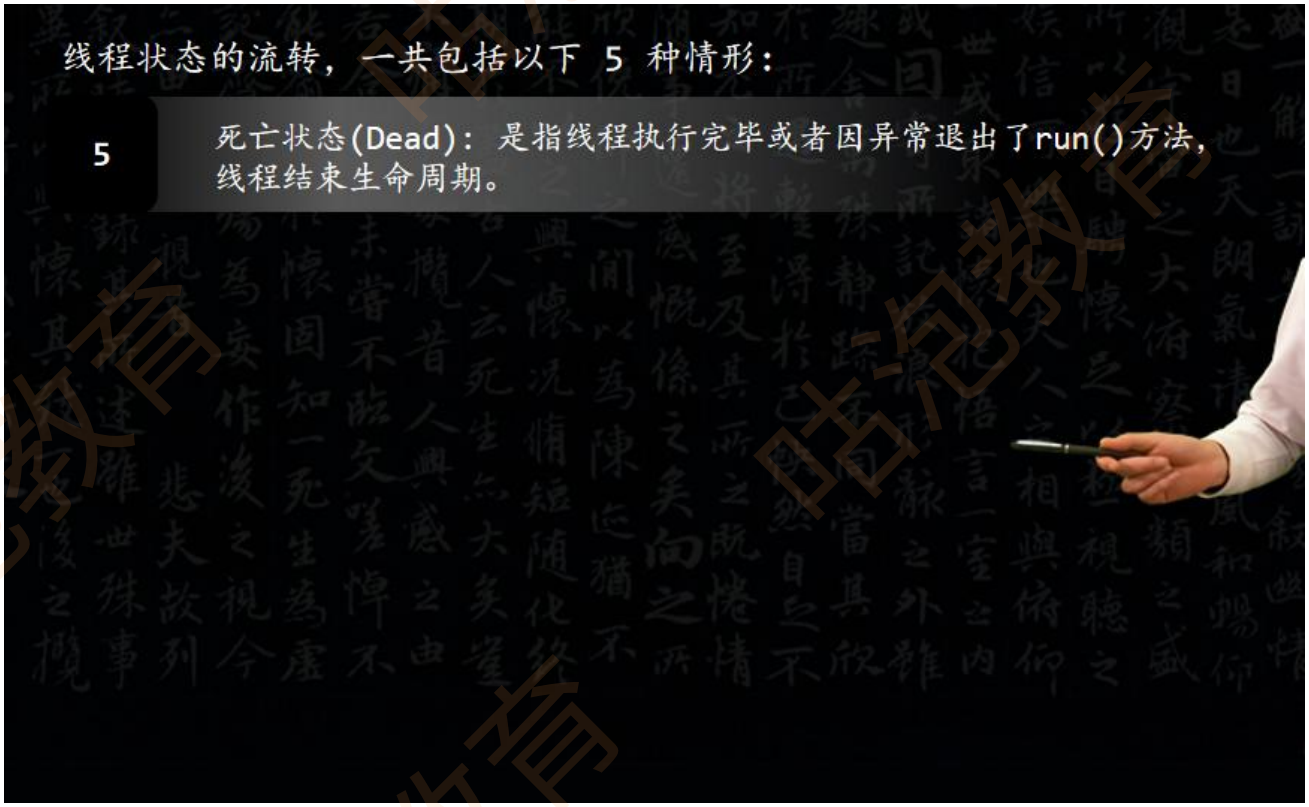
那么，阻塞又分为三种情况：

1) 等待阻塞，指通过调用线程的wait()方法，让线程等待。

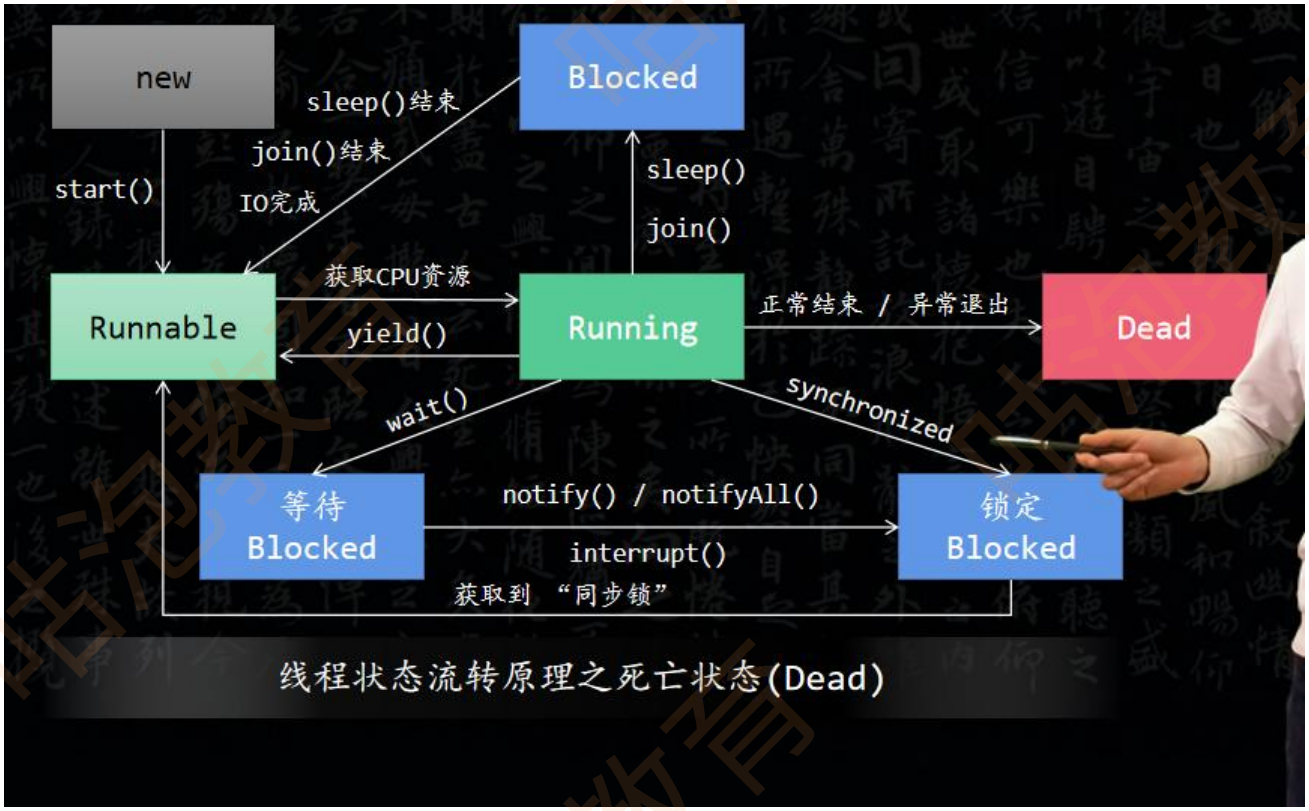
2) 同步阻塞，指线程在获取synchronized同步锁时，因为锁被其它线程所占用而导致获取失败，会进入同步阻塞状态。

3) 其他阻塞，是指通过调用线程的sleep()方法 或者 join()方法 又或者 发出了I/O请求的时候，线程会进入到阻塞状态。当sleep()状态超时、join()等待线程终止或者超时、或者I/O处理完毕时，线程重新转入就绪状态。





5. 死亡状态(Dead): 是指线程执行完毕或者因异常退出了run()方法，线程结束生命周期。



(此处需要配图)

举个通俗一点的例子来解释上面五种状态，比如我们平时去商场上厕所，准备去上厕所就是新建状态（new），上厕所要排队，排队就是就绪状态（Runnable），有坑位了，轮到你了，蹲坑就是运行状态（Running），蹲完坑发现没有手纸，需要等待其他人送纸过来，这个状态就是阻塞（Blocked），等上完厕所出来，上厕所这件事情结束了线程也就不存在了，就是死亡状态。

需要注意的是：便秘也是阻塞状态，你便秘太久，其他人会等不及，可能会把你赶走，这个就是挂起。还有一种情况就是，如果你便秘占坑位太久，其他人跟你说，先出去酝酿一下，5分钟后再过来蹲坑，这就叫睡眠。

【结尾】

好了，以上就是我对线程状态流转原理的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

11.29 请你对比一下 Java 和 JavaScript的区别

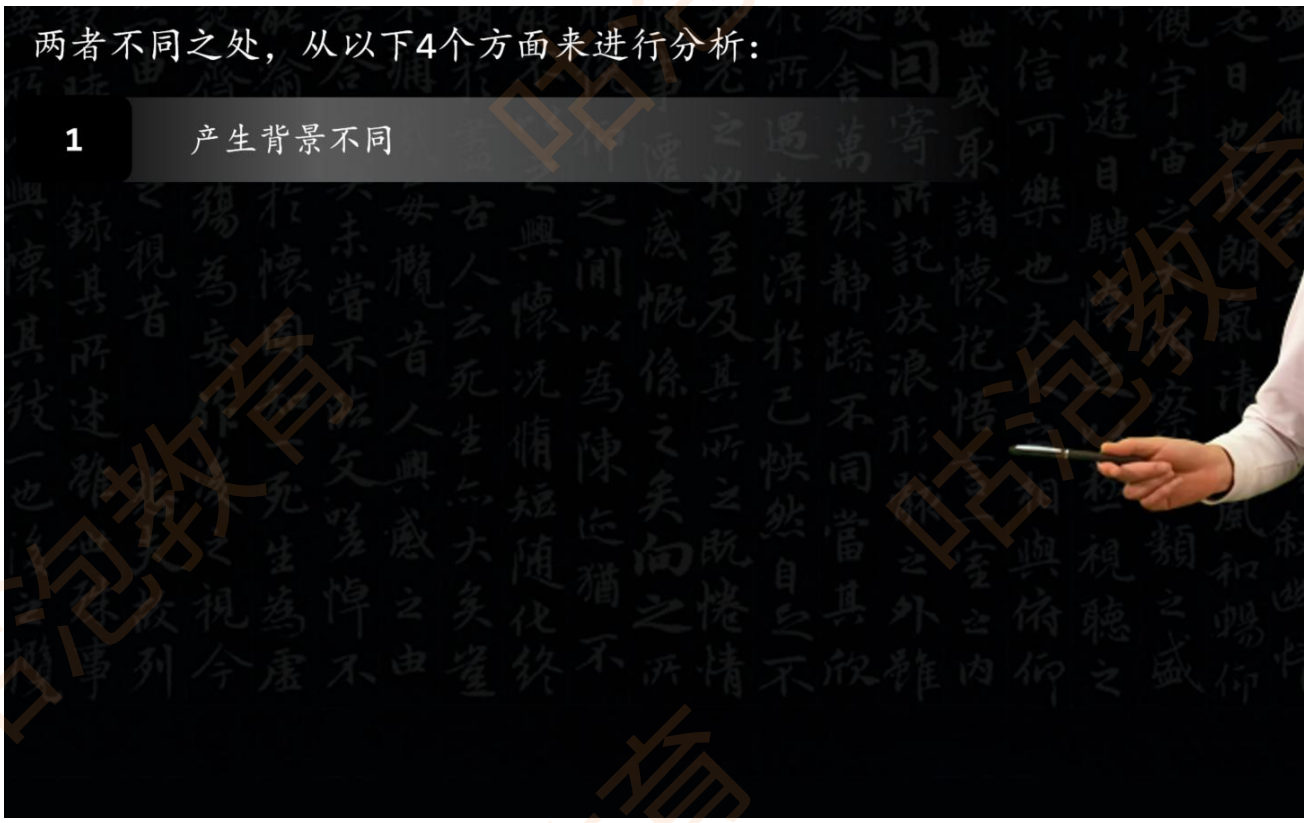
【引言】

一位应届生在面试的时候把所有的八股文都答对了。没成想，有位面试官出了个奇招，面试官问了一句说，请你对比一下Java和JavaScript的区别。我这想这样入门级别的问题，应该每一个后端开发人员都能答得上吧。没成想，有位应届毕业生竟然没有答出来，还说JavaScript是对Java的升级。这样的回答，实在是有点不应该。

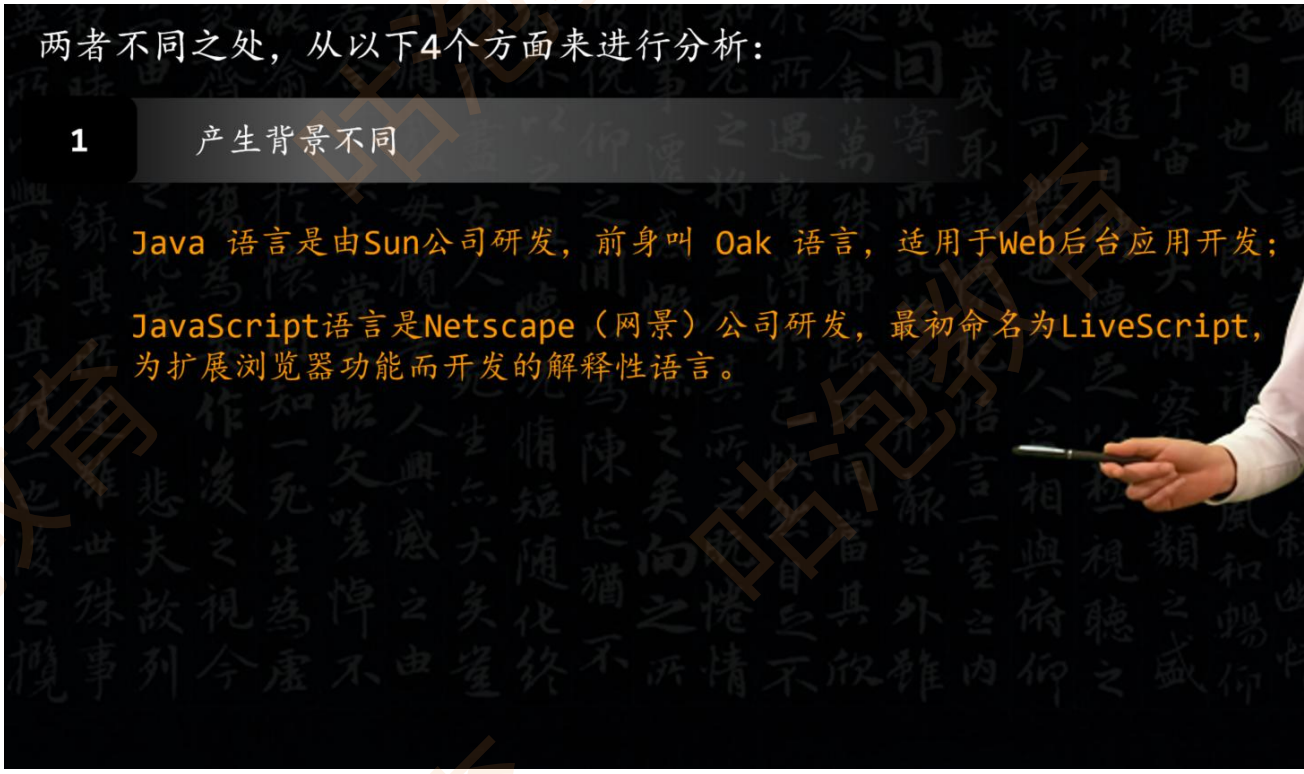
今天呢，我给大家总结一下Java和JavaScript的异同之处，

1、两者不同之处

首先总结两者不同之处，我从以下4个方面来进行分析：

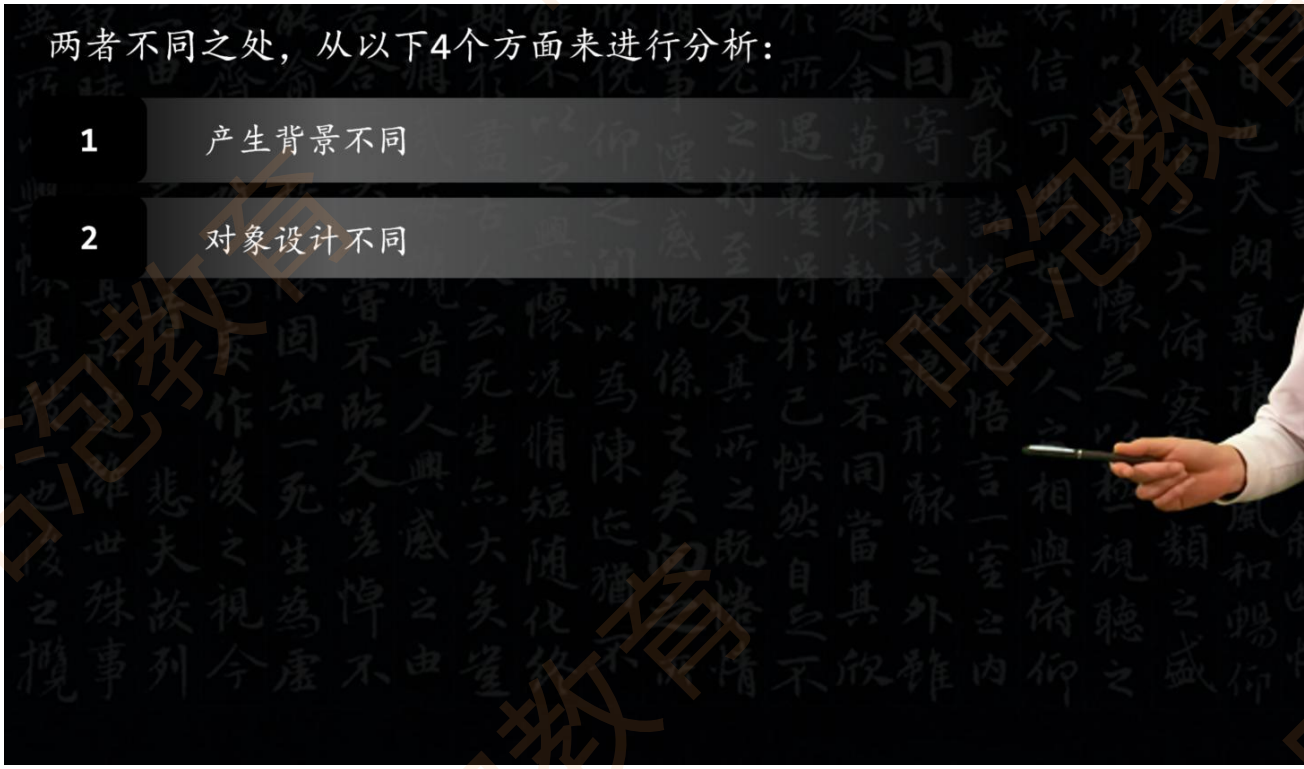


1）产生背景不同

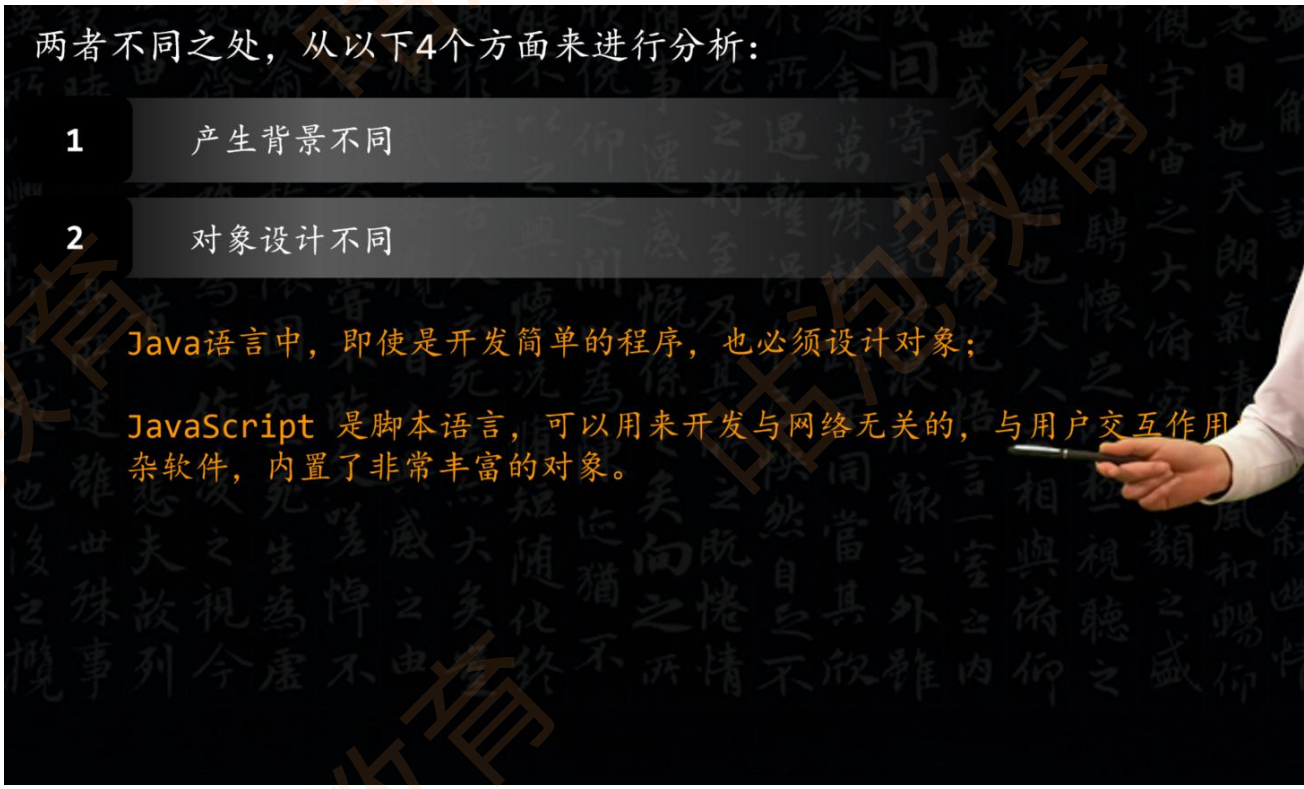


最初 Java 语言是由Sun公司研发，前身叫 Oak 语言，1995年5月正式对外发布并开源，特别 。

而JavaScript是Netscape（网景）公司的产品，最初被名命名为LiveScript。为了扩展Netscape 浏览器的功能而开发的解释性语言。因为Netscape与Sun合作，Netscape（网景）公司管理层希望它外观看起来像Java，因此取名为JavaScript。

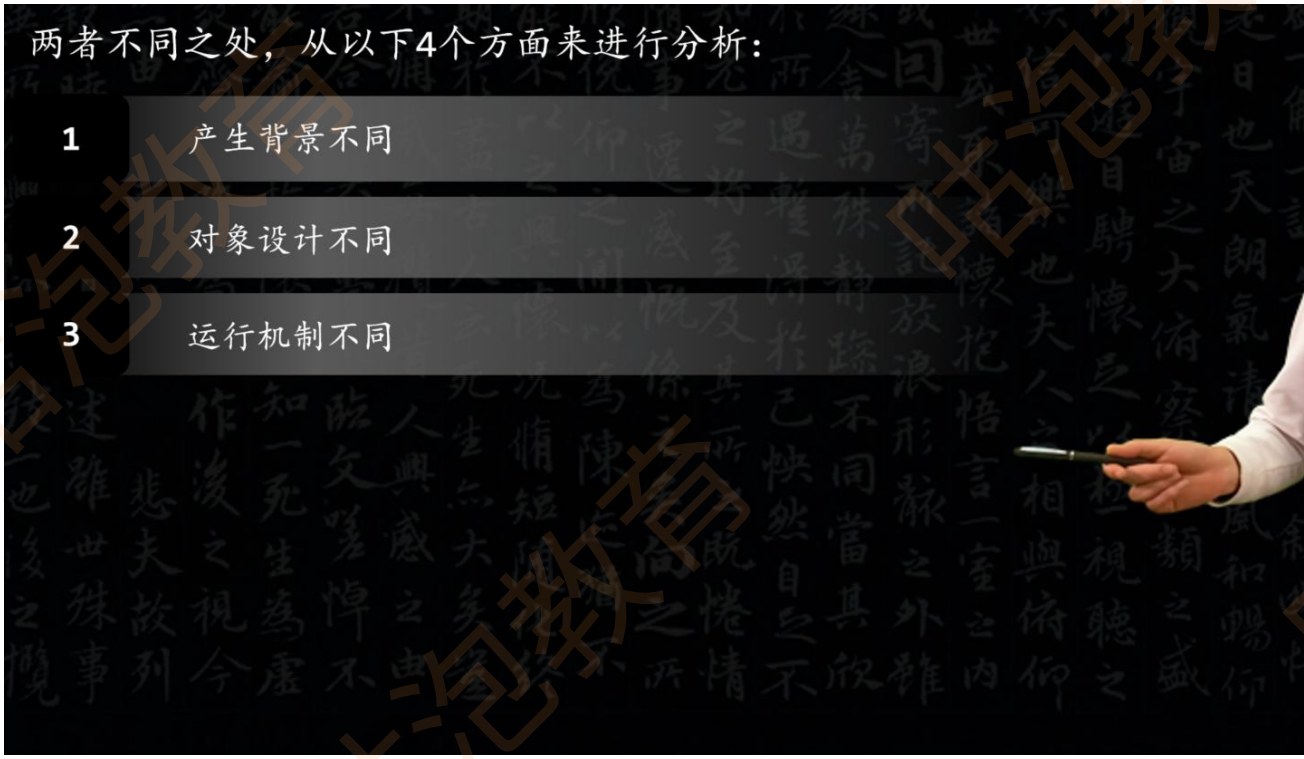


2）对象设计不同

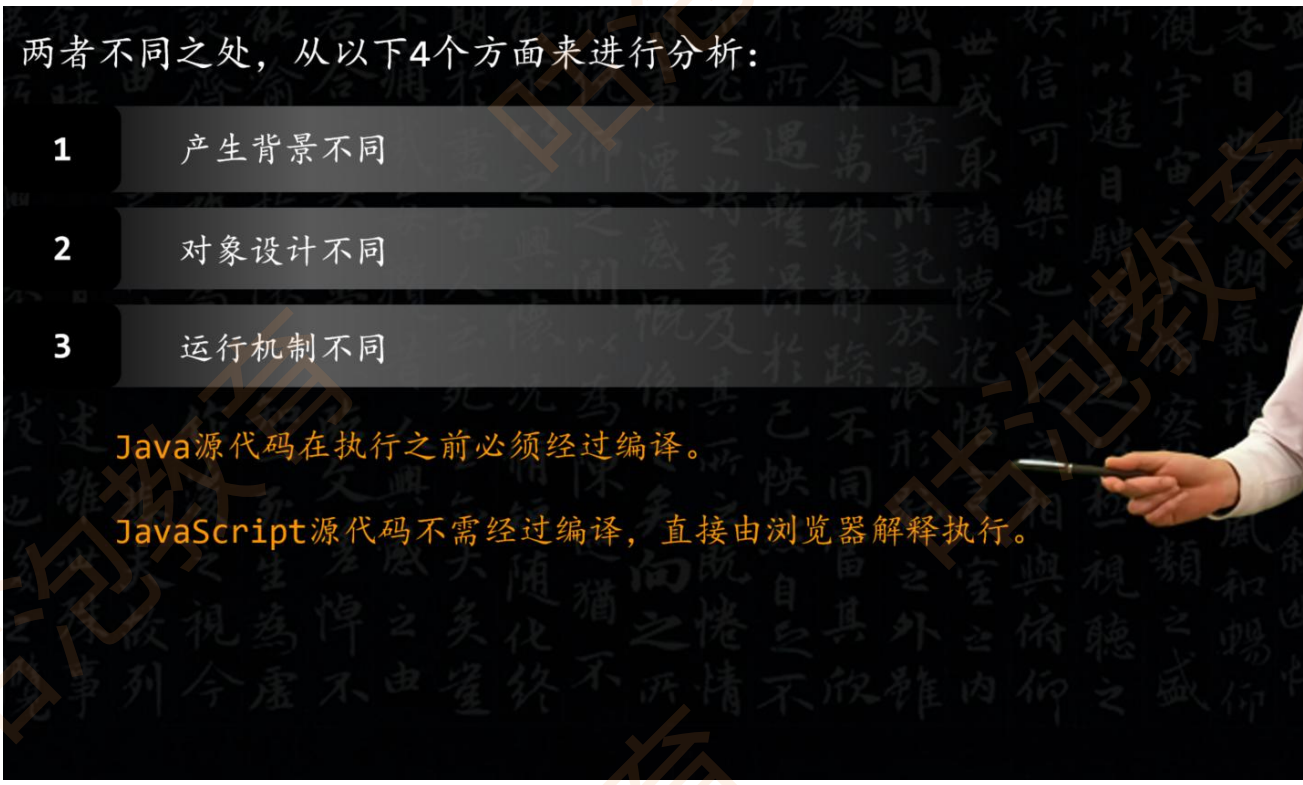


Java 是一种真正的面向对象的语言，即使是开发简单的程序，必须设计对象；

而JavaScript 是脚本语言，可以用来开发与网络无关的，与用户交互作用的复杂软件。它是一种基于对象（Object-Based）和事件驱动（Event-Driven）的编程语言，它内置了非常丰富的对象供开发人员使用。

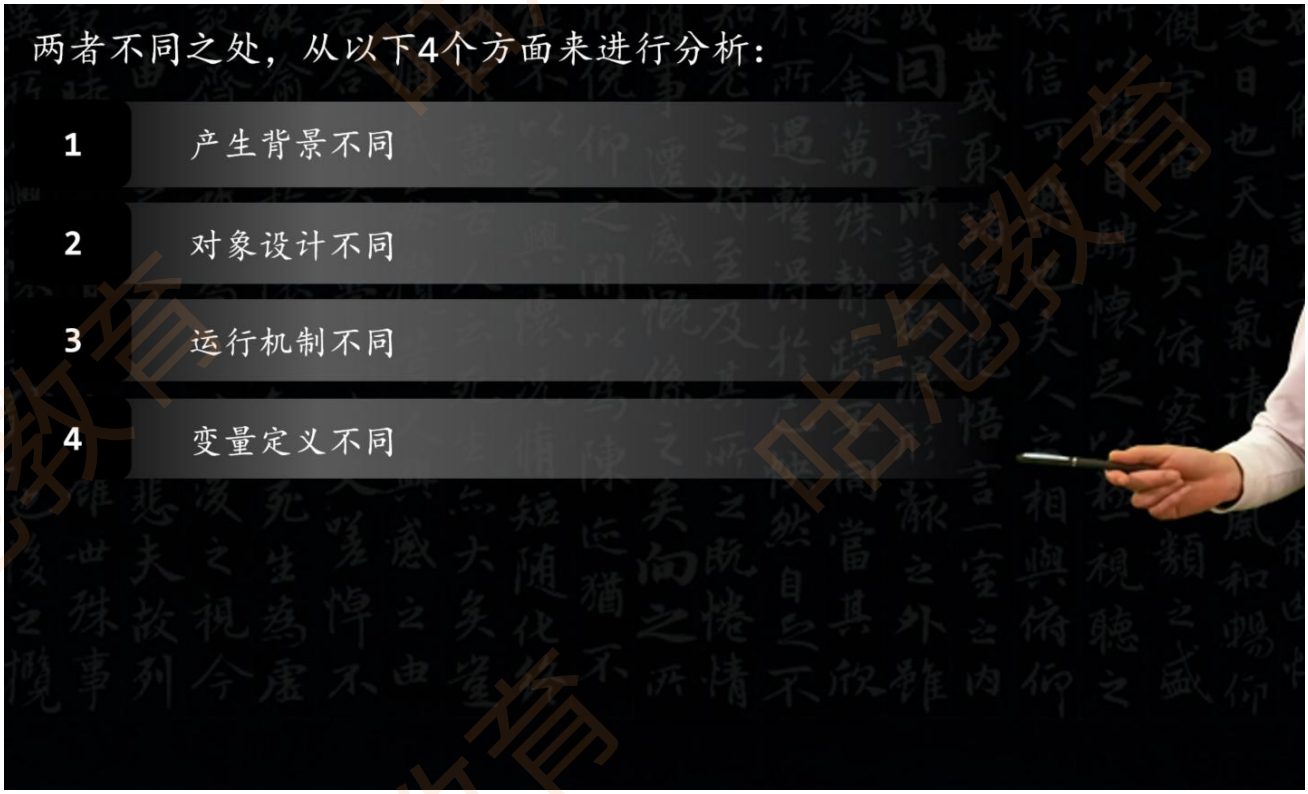


3）运行机制不同

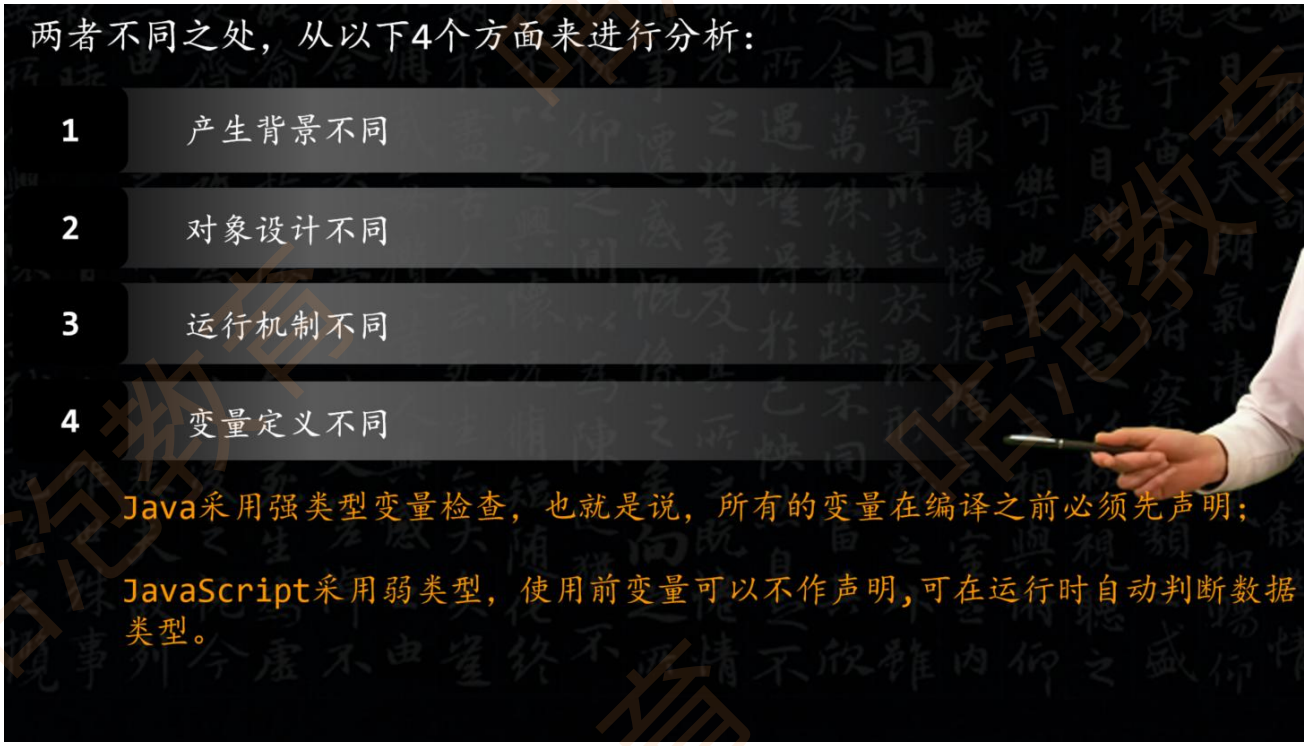


Java 的源代码在执行之前必须经过编译。

而JavaScript 是一种解释性编程语言，源代码不需经过编译，直接由浏览器解释执行。（目前的浏览器几乎都使用了 JIT（即时编译）技术来提升 JavaScript 的运行效率）



4）变量定义不同



Java 采用强类型变量检查，也就是说，所有的变量在编译之前必须先声明；

而JavaScript 中变量是弱类型的，甚至在使用前变量可以不作声明。JavaScript 的解释器会在运行时自动判断数据类型。

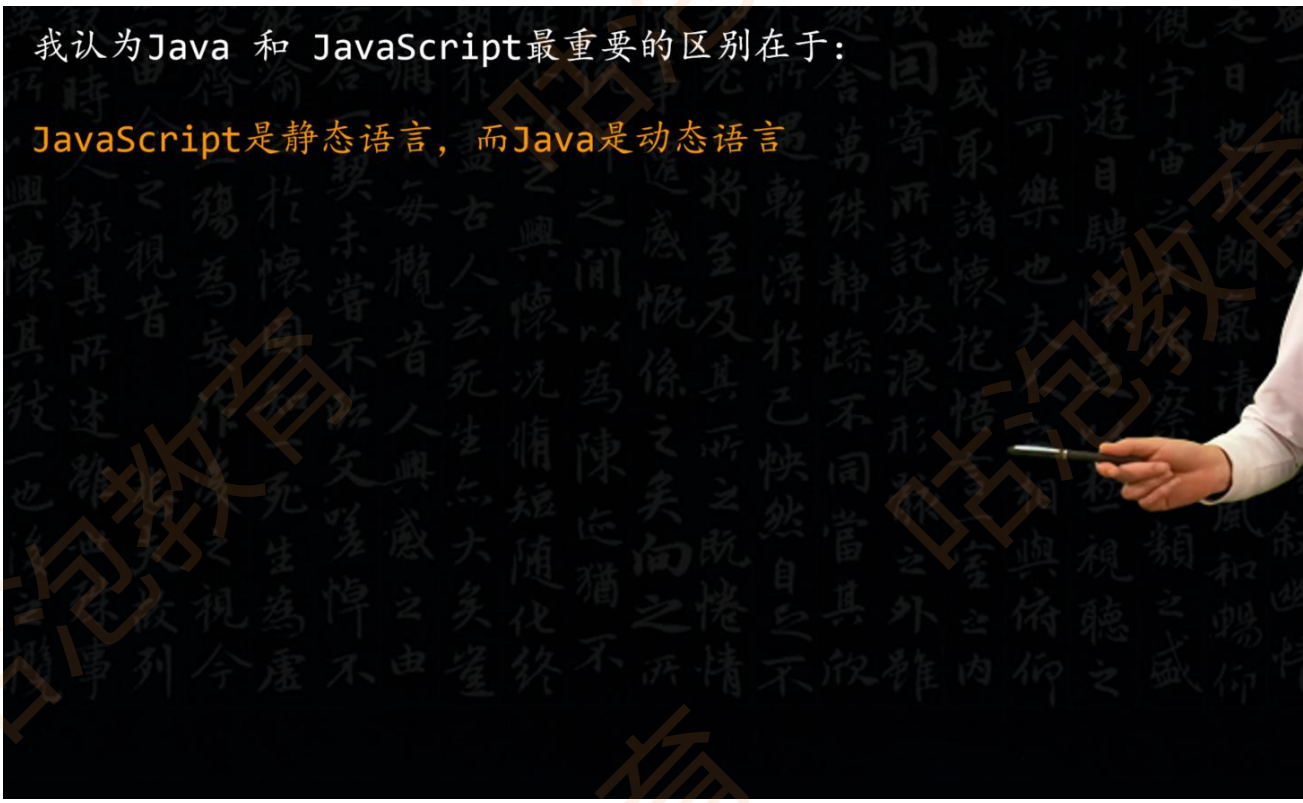
2、两者相同之处

当然，Java 和 JavaScript 也很多相同之处，我从以下几个方面来分析：

1）Java和JavaScript的语法都和 C 语言都很相似；

2）都是面向对象的。虽然实现的方式略有不同，但是JavaScript 在设计时参照了 Java 的命名规则；

3、总结



JavaScript 除了名字长得和 Java 比较像之外，它语言风格相差非常大。JavaScript 在设计时不仅参考了Java，还参考了 Self 和 Scheme 等其他语言。

Java和JavaScript各有所长各有专精，Java的舞台在编程领域，而JavaScript的得心之处是在Web页面中，它几乎无所不能。

我上面列出的相同点和不同点都在网上流传的所谓的标准答案。但是，我认为Java 和JavaScript最重要的区别在于：JavaScript是静态语言，而Java是动态语言。

目前的编程语言的发展趋势是函数式语言和动态语言。在 Java 中，类（Class）是一等公民，而 JavaScript 中，函数（Function）是一等公民，因此 JavaScript 支持函数式编程，可以使用Lambda函数和闭包（closure）。

当然，Java 8 也开始支持函数式编程，提供了对 Lambda表达式以及函数式接口的支持。对于这类问题，在面试的时候最好还是用自己的语言回答会更加靠谱，不要背网上所谓的标准答案。

【结尾】

好了，以上就是我对Java和JavaScript的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

11.30 在DCL单例写法中，为什么要做两次检查?

【引言】

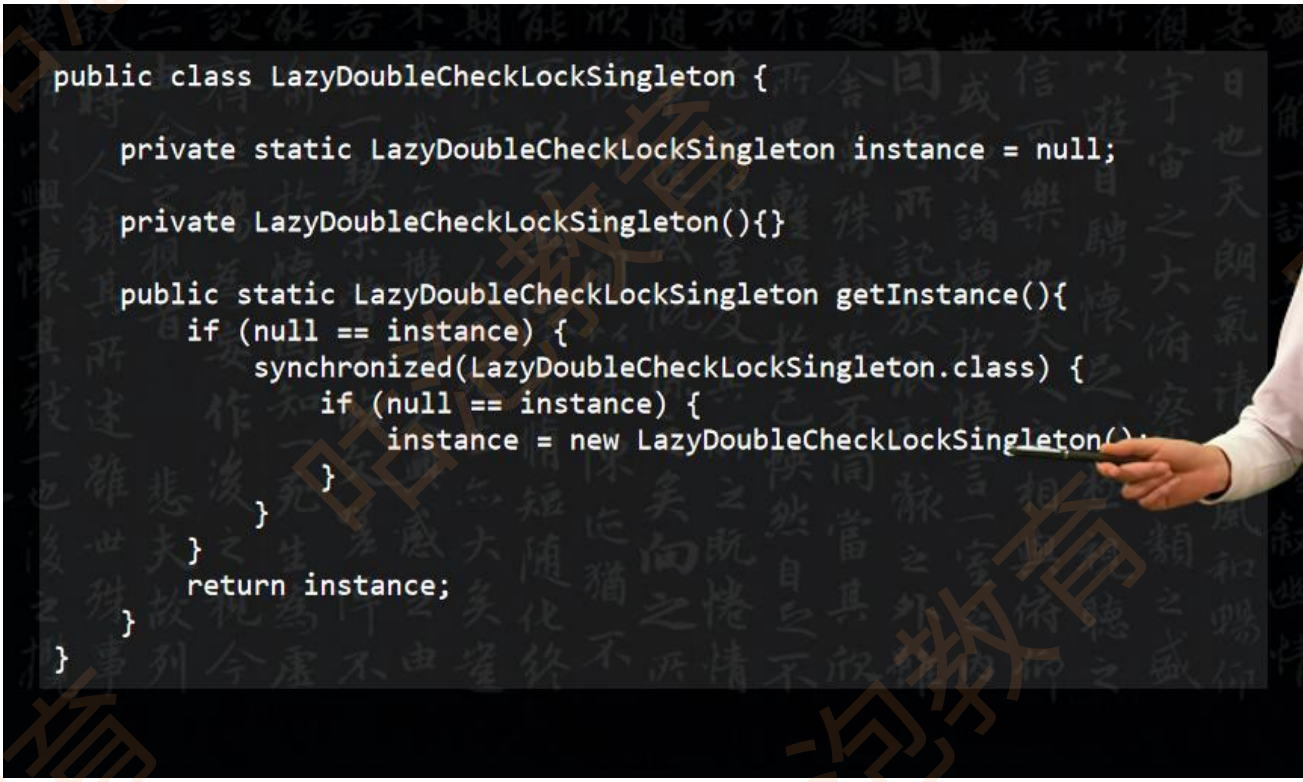
有位工作5年的小伙伴，面试的时候被问到这样一道题，说在DCL单例写法中，为什么要做两次检查。要回答好这个问题，需要知道DCL单例的写法以及为什么要这样写?

今天，我给大家详细分析一下。

1、什么是DCL

DCL是一种单例模式写法的简称，全称是Double Check Lock，翻译过来叫双重检查锁。从命名上来理解，就是两次检查加一把锁。那么，两次检查又是检查什么，锁又是锁的什么?

首先，来看这样一段代码，这是比较标准的DCL单例写法。




```
public class LazyDoubleCheckLockSingleton {

  private static LazyDoubleCheckLockSingleton instance = null;

  private LazyDoubleCheckLockSingleton(){}
```

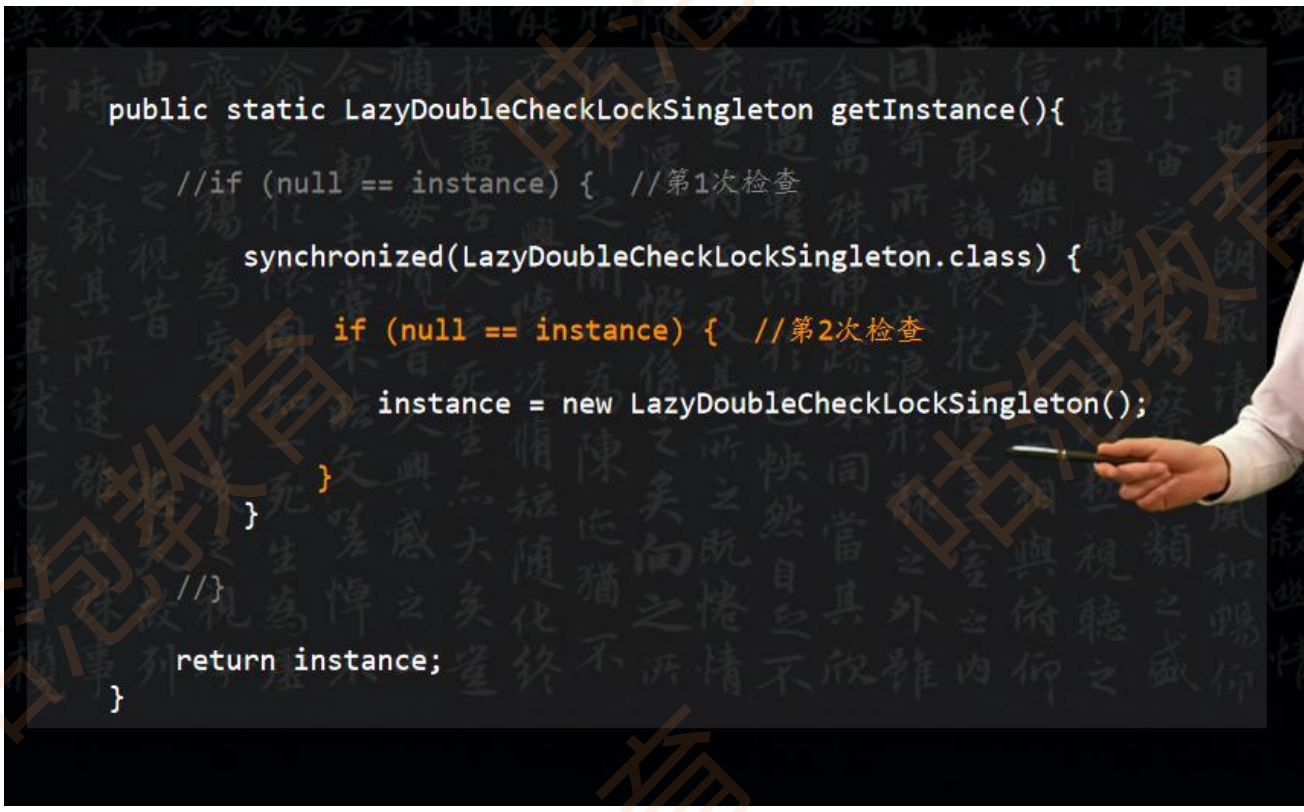
```
public static LazyDoubleCheckLockSingleton getInstance(){


    if (null == instance) {
        synchronized(LazyDoubleCheckLockSingleton.class) {
            if (null == instance) {
                instance = new LazyDoubleCheckLockSingleton();
            }
        }

    }

    return instance;
}


}
```

从代码中，我们发现两次检查的判断条件都是 null == instance，而且两个检查条件是嵌套的。在第1次检查条件的代码块中，加了一段synchronized代码块，synchronized就是锁。

小伙伴们应该都知道，加锁是为了保证线程安全，检查是为了保证内存中只有一个实例。既然，这两个条件是嵌套的，那是不是可以去掉一个条件呢？下面，我们来详细分析一下。

2、为什么需要两次检查

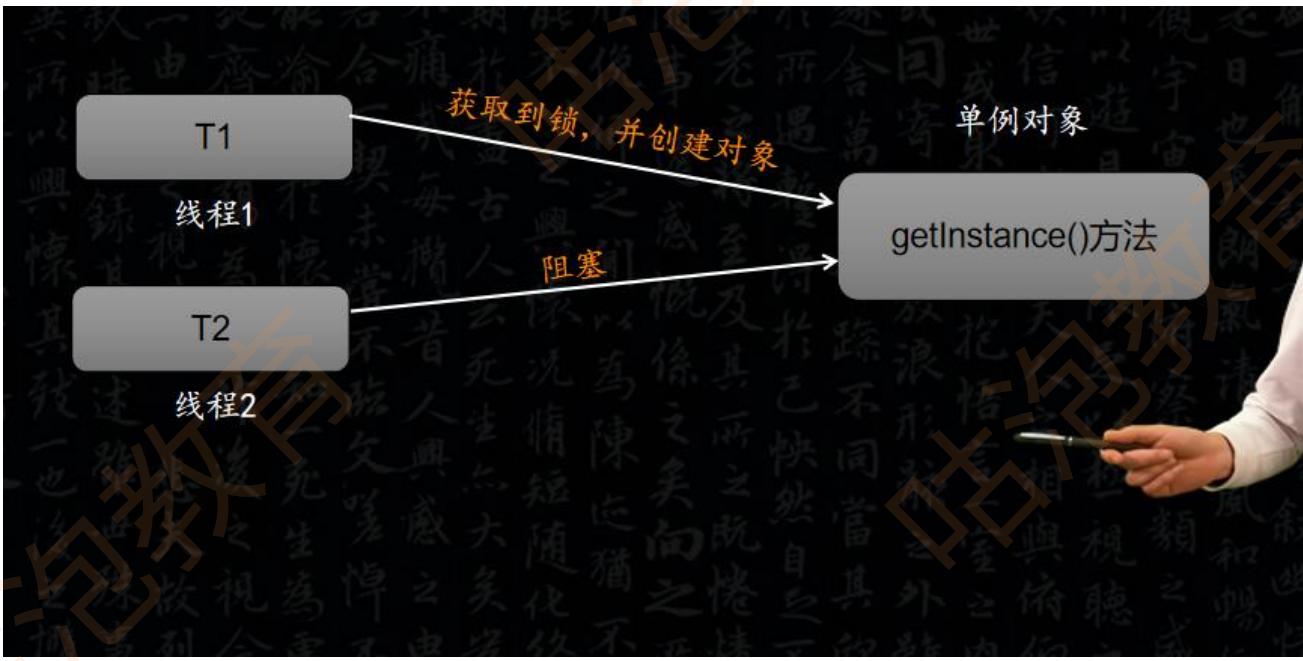
我们重点来看这段代码，



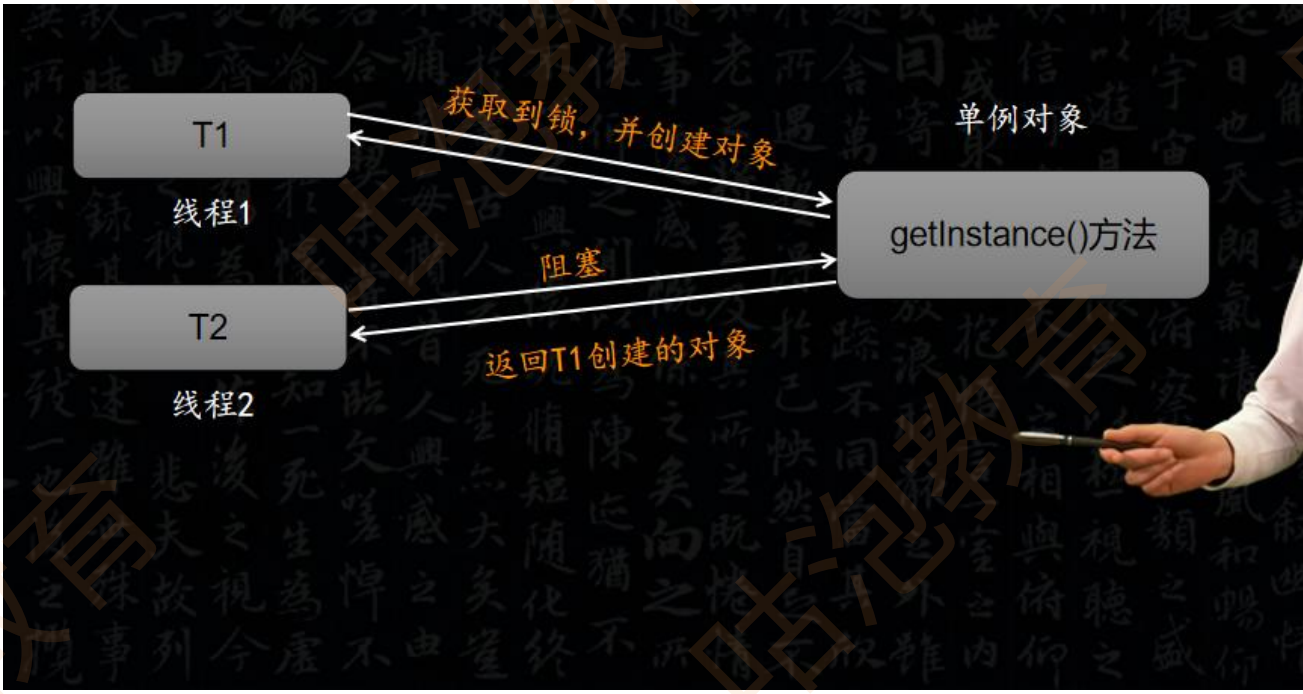

```
public static LazyDoubleCheckLockSingleton getInstance(){

    //if (null == instance) {
        synchronized(LazyDoubleCheckLockSingleton.class) {
            if (null == instance) {
                instance = new LazyDoubleCheckLockSingleton();
            }
        }
    //}

    return instance;
}
```

假设，我们去掉第1次检查，只保留第2次检查，如代码所示：

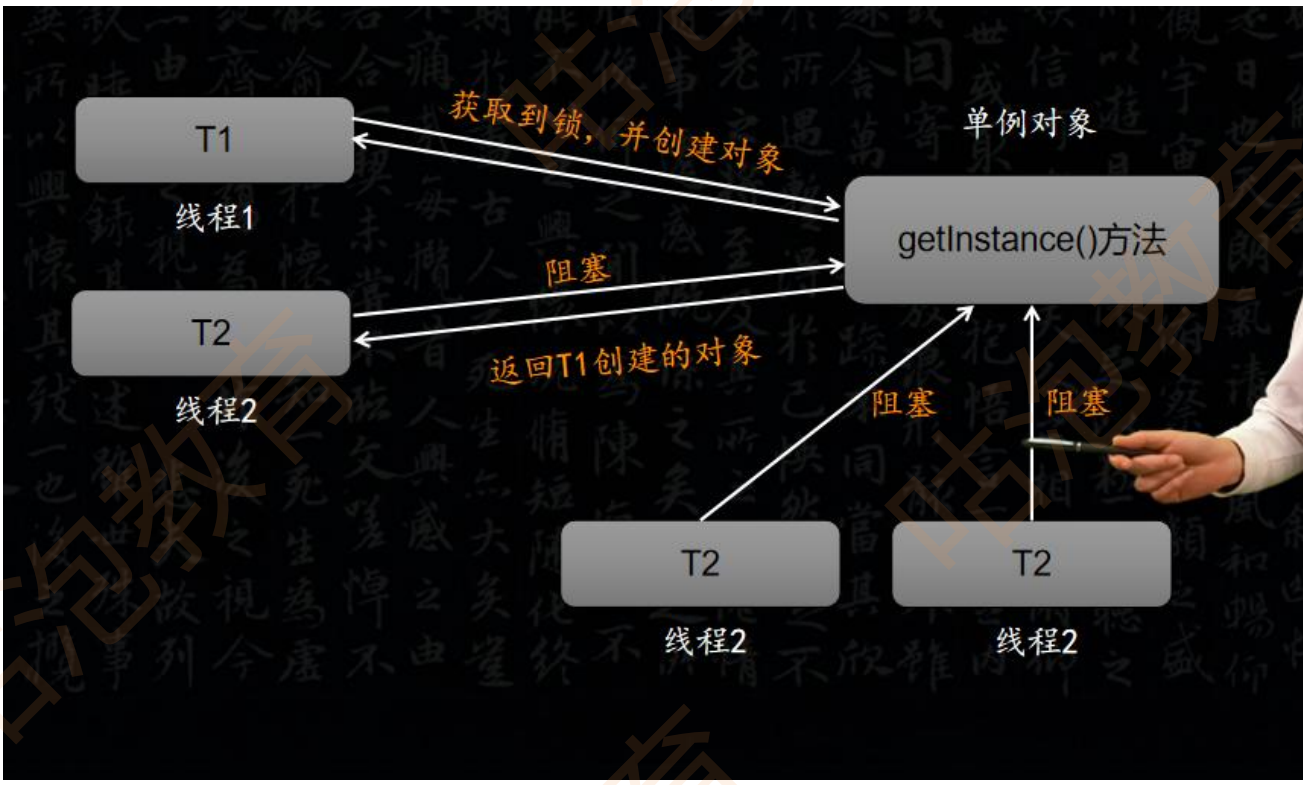
现在，有两个线程T1、T2同时访问getInstance()方法，当T1进入到synchronized代码块的时候，T2就会阻塞。直到T1执行完成释放CPU资源，T2才能获得锁。



T1执行检查条件时，null == instance为ture满足条件，因此，会创建1个新的对象，而T2执行检查条件时，不满足条件，就会直接返回T1创建好的对象，这样就保证了单例。

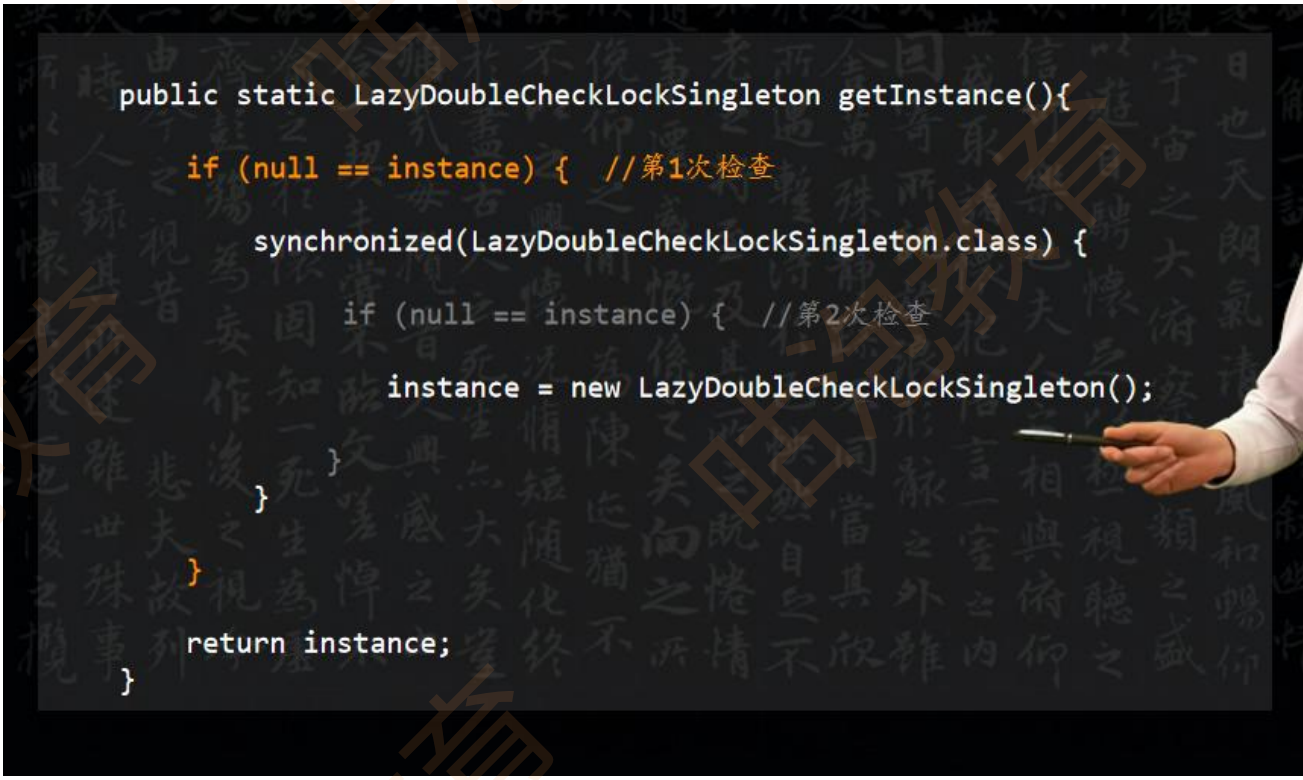


可是，问题来了，后续如果再有其他线程T4、T5、T6出现并发，同时调用getInstance()方法的情况依然会阻塞，



相当于，不管单例对象是否已经创建，每次调用都可能阻塞，会影响程序的执行效率。所以，加上第1次检查的目的是，保证只有第一次出现并发的情况会阻塞，提高性能。

那假设，去掉第2次检查,只保留第1次检查，如代码所示：



```
public static LazyDoubleCheckLockSingleton getInstance(){
    if (null == instance) {  //第1次检查
        synchronized(LazyDoubleCheckLockSingleton.class) {
            if (null == instance) {  //第2次检查
                instance = new LazyDoubleCheckLockSingleton();
            }
        }
    }
    return instance;
}
```

```
public static LazyDoubleCheckLockSingleton getInstance(){
  if (null == instance) {
      synchronized(LazyDoubleCheckLockSingleton.class) {
         //if (null == instance) {
            instance = new LazyDoubleCheckLockSingleton();
         //}
      }

   }

   return instance;
}
```

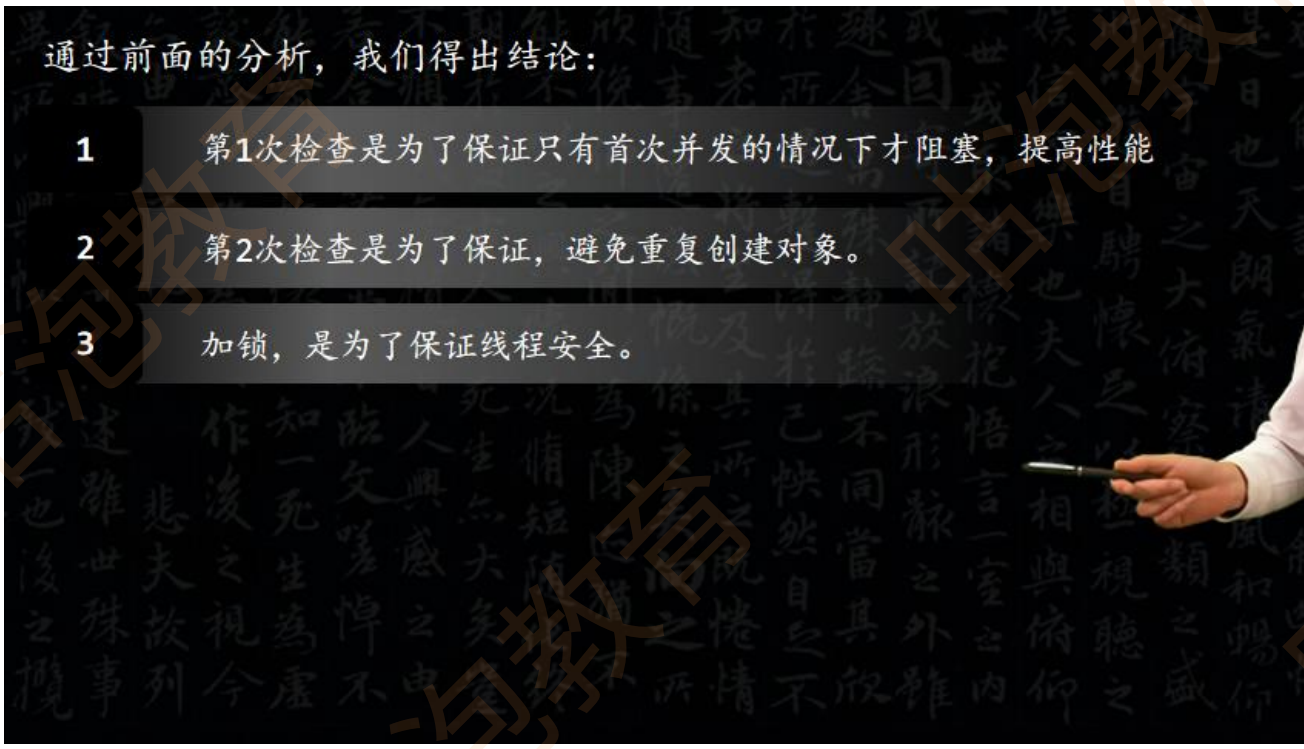
还是，线程T1、T2同时访问getInstance()方法，此时T1、T2同时满足条件，两个线程会按顺序执行synchronized代码块中的逻辑。假设T1先执行创建对象，那么，T2获得锁的时候，依然会创建对象，而且还会覆盖T1创建的对象，

这就相当于破坏了单例。

因此，第2次检查的目的是，保证单例，避免重复创建单例对象。

3、总结

通过前面的分析，我们得出结论，DCL单例写法中，



第1次检查是为了保证只有首次并发的情况下才阻塞，提高性能，第2次检查是为了保证，避免重复创建对象。加锁，当然就是为了保证线程安全。

在今天的分享，我还有一个细节没有讲到，就是在并发情况下，new一个对象可能会出现指令重排的现象。这时候，我们需要给声明的单例对象加上volatile关键字，保证可见性。

```
public class LazyDoubleCheckLockSingleton {


   private static volatile LazyDoubleCheckLockSingleton instance = null;
}
```

至于volatile关键字为什么能解决指令重排的问题，小伙伴们可以去我的主页，在往期视频中有专门详细分析这个问题，本期视频就不重复讲解了。

【结尾】

好了，以上就是我对DCL两次检查的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

11.31 Mybatis是如何进行分页的

“Mybatis是如何进行分页的”？

这是一个工作了3年的同学，在面试的时候遇到的问题。

Hi，大家好，我是Mic，一个工作了14年的Java程序员。

经常有同学在后台跟我吐槽

在求职过程中遇到的各种面试难题

我发现大部分的问题之前的视频都有分析过

考虑到视频可能太过分散，

不方便大家学习

所以我系统整理了一份20万字的文档

有需要的来煮叶领取

下面我们来分析一下面试官对于这个问题的考察意图。

考察目标

Mybatis是Java应用开发的基础框架，而分页又是我们实时都在使用的功能。

因此，在我看来，一方面考察的是求职者对Mybatis框架的使用能力

另一方面，以此为切入点去深度挖掘Mybatis里面更多的问题，从而了解求职者对它的理解深度。

这道题考察难度不大，主要考察1-3年Java开发经验的同学。

问题解析

数据进行分页是最基础的功能，一般可以把分页分成两类：

逻辑分页，先查询出所有的数据缓存到内存，再根据业务相关需求，从内存数据中筛选出合适的数据进行分页。

物理分页 ，直接利用数据库支持的分页语法来实现，比如Mysql里面提供了分页关键词Limit

Mybatis提供了四种分页方式：

在Mybatis Mapper配置文件里面直接写分页SQL，这种方式比较灵活，实现也简单。

RowBounds实现逻辑分页，也就是一次性加载所有符合查询条件的目标数据，根据分页参数值在内存中实现分页。

当然，在数据量比较大的情况下，JDBC驱动本身会做一些优化，也就是不会把所有结果存储在ResultSet里面，而是只加载一部分数据，再根据需求去数据库里面加载。

这种方式不适合数据量较大的场景，而且有可能会频繁访问数据库造成比较大的压力。

Interceptor拦截器实现，通过拦截需要分页的select语句，然后在这个sql语句里面动态拼接分页关键字，从而实现分页查询。

Interceptor是Mybatis提供的一种针对不同生命周期的拦截器，比如：

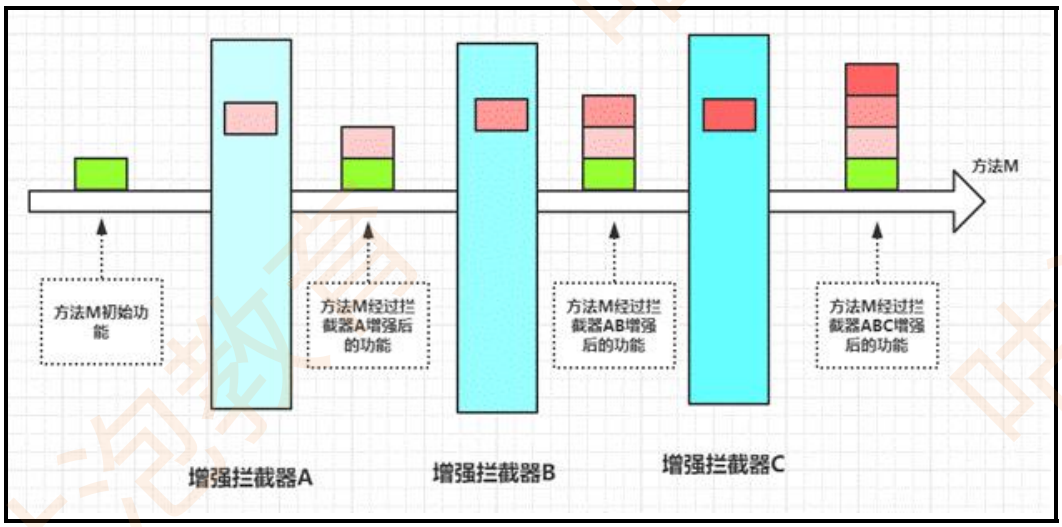
拦截执行器方法

拦截参数的处理

拦截结果集的处理

拦截SQL语法构建的处理

我们可以拦截不同阶段的处理，来实现Mybatis相关功能的扩展。



这种方式的好处，就是可以提供统一的处理机制，不需要我们再单独去维护分页相关的功能。

插件（PageHelper）及（MyBaits-Plus、tkmybatis）框架实现这些插件本质上也是使用Mybatis的拦截器来实现的。

只是他们帮我们实现了扩展和封装，节省了分页扩展封装的工作量，在实际开发中，只需要拿来即用即可。

总结一下，对于任何ORM框架，分页的实现逻辑无外乎两种，不管怎么包装，最终给到开发者的，只是使用上的差异而已。

那么，我们来看看高手该如何回答。

高手回答

我认为有三种方式来实现分页：

第一种，直接在Select语句上增加数据库提供的分页关键字，然后在应用程序里面传递当前页，以及每页展示条数即可。

第二种，使用Mybatis提供的RowBounds对象，实现内存级别分页。

第三种，基于Mybatis里面的Interceptor拦截器，在select语句执行之前动态拼接分页关键字。

以上就是我的理解。

总结

大家知道怎么回答了吗?

如果你喜欢我的作品，记得点赞收藏加关注哦

我是Mic，咱们下期再见。

11.32 Java有几种文件拷贝方式，哪一种效率最高？

“Java有几种文件拷贝方式，哪一种效率最高？”

屏幕前的你知道吗？

这个问题是京东一面的时候，针对4年经验的同学的一个面试题。

Hi，大家好，我是Mic，一个工作了14年的Java程序员。

关于这个问题的回答，我把文字版本整理到了15W字的面试文档里面。

大家可以在我的主页加V领取。

下面看看高手的回答。

高手

好的，面试官。

第一种，使用java.io包下的库，使用FileInputStream读取，再使用FileOutputStream写出。

第二种，利用java.nio包下的库，使用transferTo或transfFrom方法实现。

第三种，Java 标准类库本身已经提供了 Files.copy 的实现。

对于 Copy 的效率，这个其实与操作系统和配置等情况相关，在传统的文件IO操作里面，我们都是调用操作系统提供的底层标准IO系统调用函数 read()、write() ，由于内核指令的调用会使得当前用户线程切换到内核态，然后内核线程负责把相应的文件数据读取到内核的IO缓冲区，再把数据从内核IO缓冲区拷贝到进程的私有地址空间中去，这样便完成了一次IO操作。

而NIO里面提供的NIO transferTo和transfFrom方法，也就是常说的零拷贝实现。

它能够利用现代操作系统底层机制，避免不必要拷贝和上下文切换，因此在性能上表现比较好。

以上就是我对这个问题的理解。

总结

关于文件IO方面的问题和内容还挺多的

这块属于Java里面的基础知识，但是随着这个技术知识的延展，

会涉及到NIO、AIO、零拷贝、IO多路复用机制等等。

对于求职者来说，这块内容的重要性也不言而喻。

11.33 什么是Java虚拟机，为什么要使用？

“什么是Java虚拟机，为什么要使用”。

最近一个1年Java开发经验的同学去面试阿里，遇到这个问题向我求助。

Hi，大家好，我是Mic，一个工作14年的Java程序员。

那么，这个问题，面试官希望考察什么呢?

问题解析

Java虚拟机，是Java应用程序运行的平台。

很多初学者，第一步基本上都是学习怎么写代码，并没有关注Java代码所运行的平台。

因此，虽然写了几年代码，但是对Java本身的理解不够深刻，程序一旦出现问题，很难排查和解决。

面试官考察这个问题的出发点，我认为有三个

了解求职者对于Java语言的理解深度，这个方面有助于提升代码编写的质量

了解求职者对于JVM基础的掌握程度，良好的基础有助于快速解决GC问题、内存问题等

考察求职者的潜质，一个对技术有热情的人，有助于更好的陪伴公司成长

所以，对于这个问题来说，我们只需要从JVM关键特性Write Once、Run Anywhere这个角度去切入解释就行了。

下面我们来看看高手应该怎么回答。

高手

Java虚拟机是Java语言的运行环境。

之所以需要Java虚拟机，主要是为Java语言提供Write Once，Run Anywhere能力。

实际上，一次编写，到处运行这个能力本身是不可能实现的。因为不同的操作系统和硬件。

最终执行的指令会有较大的差异。

而Java虚拟机就是解决这个问题的，它能根据不同的操作系统和硬件差异，生成符合这个平台机器指令。

简单理解，它就相当于一个翻译工具，在window下，翻译成window可执行的指令，在linux下，翻译成linux下可执行的指令。

除了这个因素以为，我认为自动回收垃圾这个功能也是原因之一，它让开发者省去了垃圾回收这个工作。

减少了程序开发的复杂性。

总结

好了，今天的分享就到这里结束了

如果喜欢我的作品，记得点赞、收藏、关注

11.34 Java反射的优缺点?

“请你Java反射的优缺点？”

这是一个工作了5年的同学，去阿里面试第一面遇到的问题。

今天给大家分享一下我们遇到这个问题该怎么回答，才能获得面试官的认可。

问题分析

反射是Java语言里面比较重要的一个特征。

它能够在程序运行的过程中去构造任意一个类对象、并且可以获取任意一个类的成员变量、成员方法、属性，以及调用任意一个对象的方法。

通过反射的能力，可以让Java语言支持动态获取程序信息以及动态调用方法的能力。

在Java里面，专门有一个java.lang.reflect用来实现反射相关的类库，包括Construct、Field、Method等类，分别用来获取类的构造方法、成员变量、方法信息。

反射的使用场景还挺多的，比如在动态代理的场景中，使用动态生成的代理类来提升代码的复用性。

在Spring框架中，有大量用到反射，比如用反射来实例化Bean对象。

高手回答

Java反射的优点：

增加程序的灵活性，可以在运行的过程中动态对类进行修改和操作

提高代码的复用率，比如动态代理，就是用到了反射来实现

可以在运行时轻松获取任意一个类的方法、属性，并且还能通过反射进行动态调用

Java反射的缺点：

反射会涉及到动态类型的解析，所以JVM无法对这些代码进行优化，导致性能要比非反射调用更低。

使用反射以后，代码的可读性会下降

反射可以绕过一些限制访问的属性或者方法，可能会导致破坏了代码本身的抽象性

总结

好的，今天就到这里了

喜欢我作品的同学记得转发、收藏、点赞

我是咕泡科技联合创始人Mic，咱们下期再见。

11.35 JVM 为什么使用元空间替换了永久代?

“JVM 为什么使用元空间替换了永久代？”

这是一个工作6年的同学去字节第一面遇到的问题，很遗憾，他没有回答出来

大家好，我是Mic，一个工作了14年的Java程序员。

关于这个问题，我们怎么回答？面试官到底关注什么呢?

面试解析

我们都知道Java8以及以后的版本中，JVM运行时数据区的结构都在慢慢调整和优化。

但实际上这些变化，对于业务开发的小伙伴来说，没有任何影响。

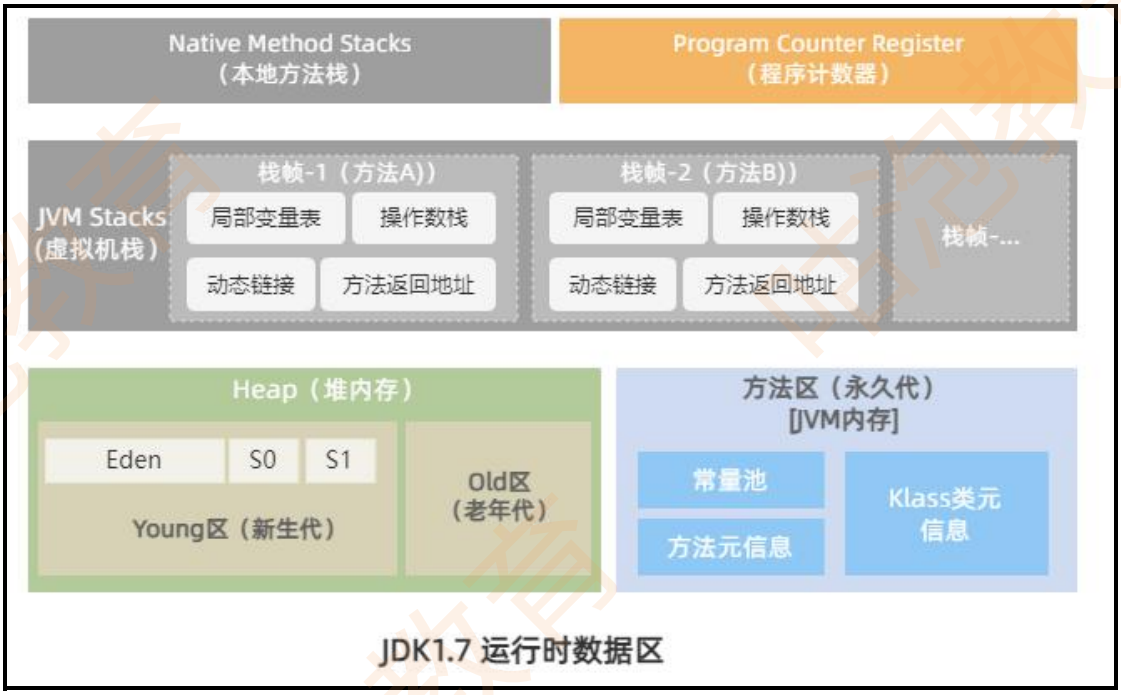
因此我可以说，99%的人都回答不出这个问题。

但是互联网大厂的面试就是筛选那1%的优秀人才，因此通过这道题，既可以考察求职者对JVM原理的理解程度又能够考察求职者基本功的扎实程度还能实现高级人才的筛选在Java7里面，JVM运行时数据区是这样的。

在Hotspot虚拟机中，方法区的实现是在永久代里面，它里面主要存储运行时常量池、Klass类元信息等。

永久代属于JVM运行时内存中的一块存储空间，我们可以通过-XX:PermSize来设置永久代的大小。

当内存不够的时候，会触发垃圾回收。



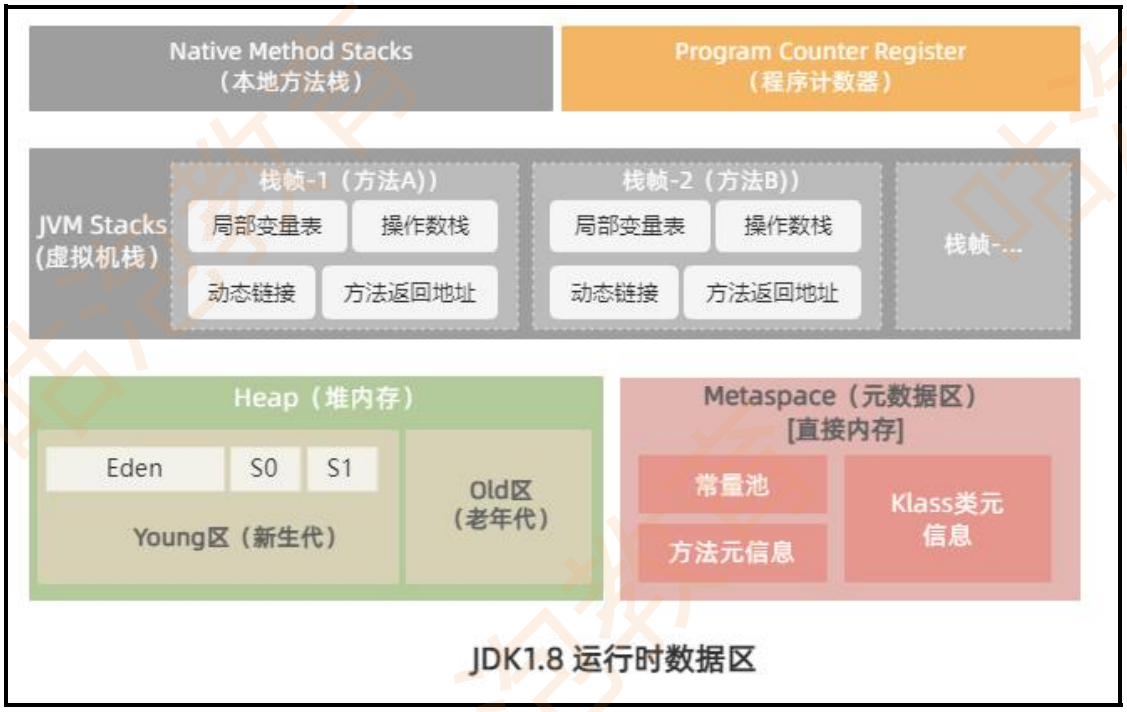
JDK1.7 运行时数据区

在JDK1.8里面，JVM运行时数据区是这样的。

在Hotspot虚拟机中，取消了永久代，由元空间来实现方法区的数据存储。

元空间不属于JVM内存，而是直接使用本地内存，因此不需要考虑GC问题。

默认情况下元空间是可以无限制的使用本地内存的，但是我们也可以使用JVM参数来限制内存使用大小。



JDK1.8 运行时数据区

为什么要使用元空间来替换永久代，背后必然有它的道理，但是如果求职者能够回答出来。

必然对于JVM底层原理是有一定了解的。

我们来看看高手的回答。

高手

我认为有三个方面的原因：

在1.7版本里面，永久代内存是有上限的，虽然我们可以通过参数来设置，但是JVM加载的class总数、大小是很难确定的。

所以很容易出现OOM问题。

但是元空间是存储在本地内存里面，内存上限比较大，可以很好的避免这个问题。

永久代的对象是通过FullGC进行垃圾收集，也就是和老年代同时实现垃圾收集。

替换成元空间以后，简化了Full GC。可以在不进行暂停的情况下并发地释放类数据，同时也提升了GC的性能Oracle要合并Hotspot和JRockit的代码，而JRockit没有永久代。

以上就是我对这个问题的理解。

总结

好了，今天的分享就到这里结束了

如果喜欢我的作品，记得点赞、收藏、关注！！！

11.36 JVM分代年龄为什么是15次？可以25次吗？

一个工作了7年的粉丝去京东面试，遇到一个很有意思的问题。

这是一个关于JVM底层相关的问题，如果平时没有去花时间是肯定回答不出来的。

Hello，大家好，我是Mic，一个工作了14年的程序员和创业者。

今天给大家分享的这道面试题是：“JVM分代年龄为什么是15次，可以是25次吗？”

另外我花了1个多星期的时间，把往期高手回答整理成了10W字的文档，想获取的小伙伴可以从我的个人煮叶简介加微领取

下面看看高手对这个问题的回答。

高手

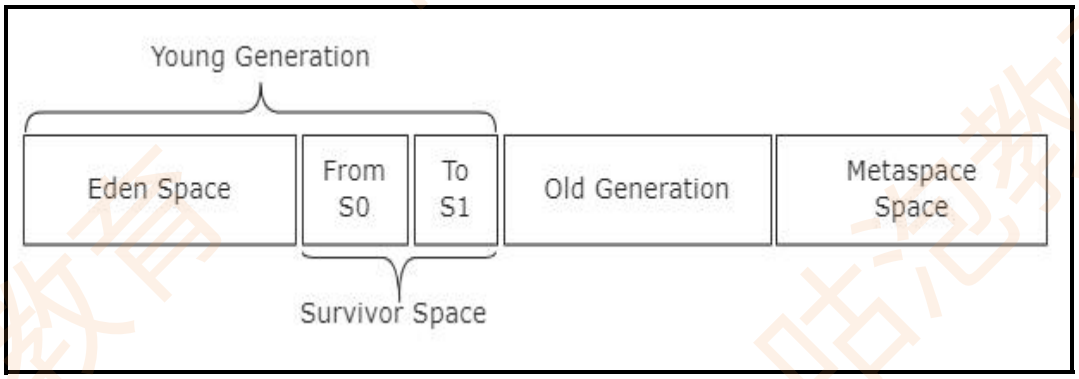
好的，这个问题我会从几个方面来回答。

首先，在JVM的heap内存里面，分为Eden Space、Survivor Space、Old Generation。

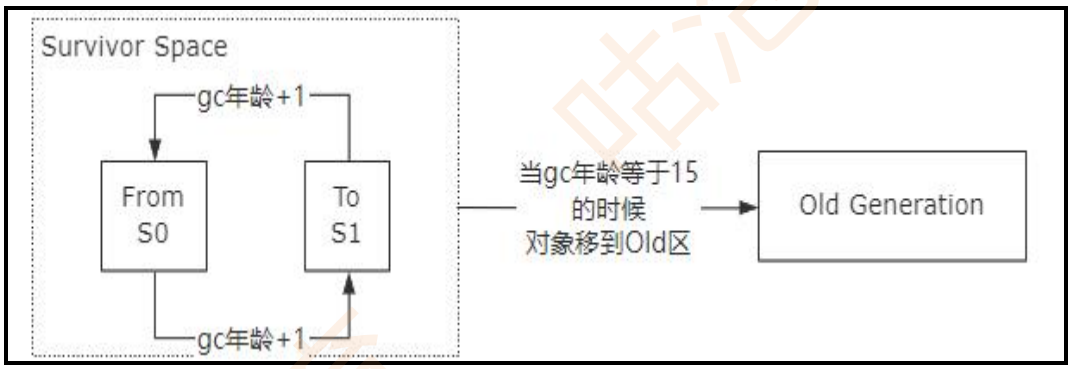
当我们在Java里面使用new关键字创建一个对象的时候，JVM会在Eden Space分配一块内存空间来存储这个对象。

当Eden Space的内存空间不足的时候，会触发Young GC进行对象回收。

那些因为存在引用关系而无法回收的对象，JVM会把它们转移到Survivor Space。



Survivor Space内部又分为From区和To区，刚从Eden区转移过来的对象会分配到From区，每经历一次Young GC，这些没有办法被回收的对象就会在From区和To区来回移动，每移动一次，这个对象的GC年龄就加1。默认情况下GC年龄达到15的时候，JVM就会把这个对象移动到Old Generation。



其次呢，一个对象的GC年龄，是存储在对象头里面的，一个Java对象在JVM内存中的布局由三个部分组成，分别是对象头、实例数据、对齐填充。而对象头里面有4个bit位来存储GC年龄。

锁状态	32bit				
	25bit		4bit	1bit	2bit
	23bit	2bit		偏向模式	标志位
未锁定	对象哈希码		分代年龄	0	01
轻量级锁定	指向调用栈中锁记录的指针				00
重量级锁定（锁膨胀）	指向重量级锁的指针				10
GC 标记	空				11
可偏向	线程 ID	Epoch	分代年龄	1	01

而4个bit位能够存储的最大数值是15，所以从这个角度来说，JVM分代年龄之所以设置成15次是因为它最大能够存储的数值就是15。

虽然JVM提供了参数来设置分代年龄的大小，但是这个大小不能超过15。

而从设计角度来看，当一个对象触发了最大值15次gc，还没有办法被回收，就只能移动到old generation了。

另外，设计者还引入了动态对象年龄判断的方式来决定把对象转移到old generation，也就是说不管这个对象的gc年龄是否达到了15次，只要满足动态年龄判断的依据，也同样会转移到old generation。

以上就是我对这个问题的理解。

面试点评

这个问题被问到的频率还挺高的。

并且底层涉及到的知识点也非常多，比如对象头、jvm垃圾回收机制、堆内存划分等等。

所以建议大家在平时工作之外的时间，多花一点时间去研究这些底层原理。

好的，本期的普通人VS高手面试系列就到这里结束了。

大家记得点赞收藏加关注

我是Mic，咱们下一期再见。

11.37 为什么引入偏向锁、轻量级锁，介绍下升级流程

Hi，我是Mic

一个工作了7年的粉丝来找我，他说最近被各种锁搞晕了。

比如，共享锁、排它锁、偏向锁、轻量级锁、自旋锁、重量级锁、

间隙锁、临键锁、意向锁、读写锁、乐观锁、悲观锁、表锁、行锁。

然后前两天去面试，被问到偏向锁、轻量级锁，结果没回答上来。

ok，关于Synchronized锁升级的原理，看看高手的回答。

另外，高手的回答已经整理成了文档，有需要的小伙伴可以主页加V领取

高手

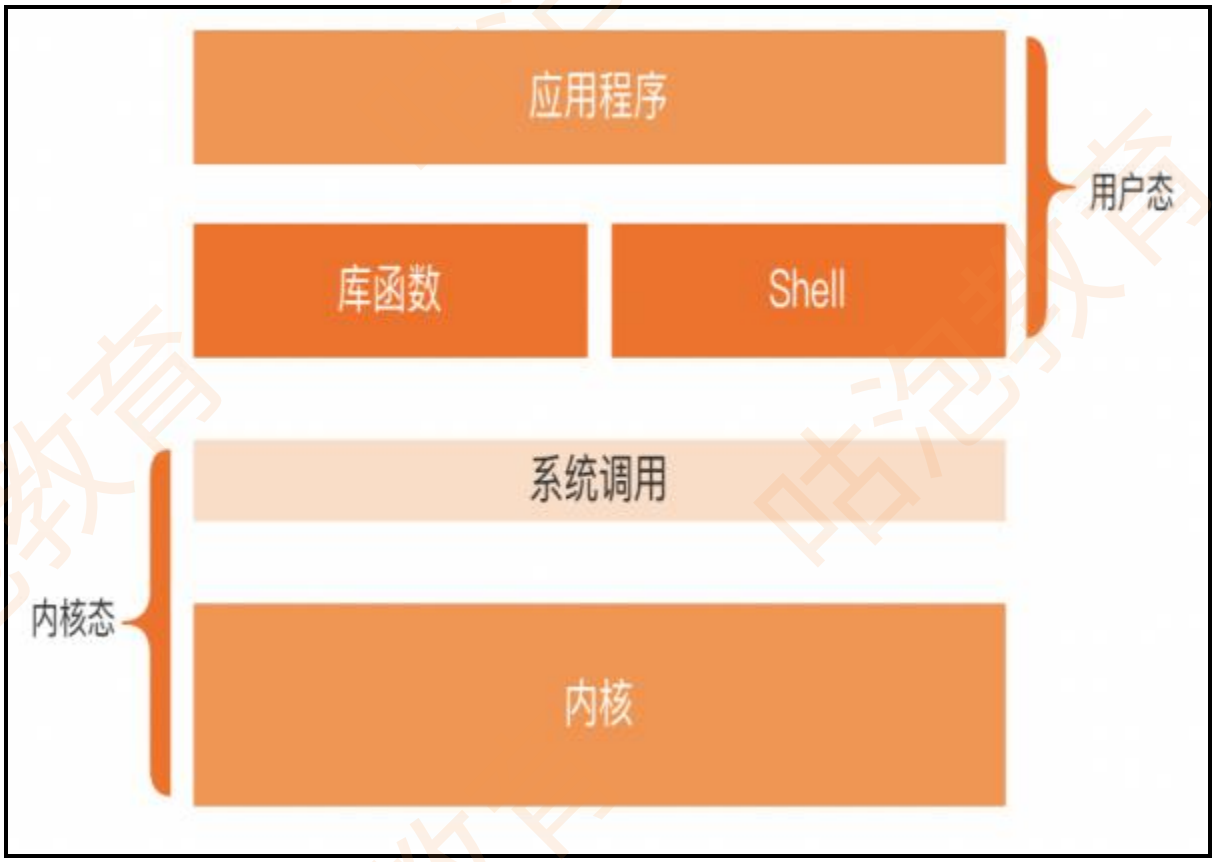
好的，面试官。

Synchronized在jdk1.6版本之前，是通过重量级锁的方式来实现线程之间锁的竞争。

之所以称它为重量级锁，是因为它的底层底层依赖操作系统的Mutex Lock来实现互斥功能。

Mutex是系统方法，由于权限隔离的关系，应用程序调用系统方法时需要切换到内核态来执行。

这里涉及到用户态向内核态的切换，这个切换会带来性能的损耗。



在jdk1.6版本中，synchronized增加了锁升级的机制，来平衡数据安全性和性能。简单来说，就是线程去访问synchronized同步代码块的时候，synchronized根据

线程竞争情况，会先尝试在不加重量级锁的情况下去保证线程安全性。所以引入了偏向锁和轻量级锁的机制。

偏向锁，就是直接把当前锁偏向于某个线程，简单来说就是通过CAS修改偏向锁标记，这种锁适合同一个线程多次去申请同一个锁资源并且没有其他线程竞争的场景。

轻量级锁也可以称为自旋锁，基于自适应自旋的机制，通过多次自旋重试去竞争锁。自旋锁优点在于它避免避免了用户态到内核态的切换带来的性能开销。

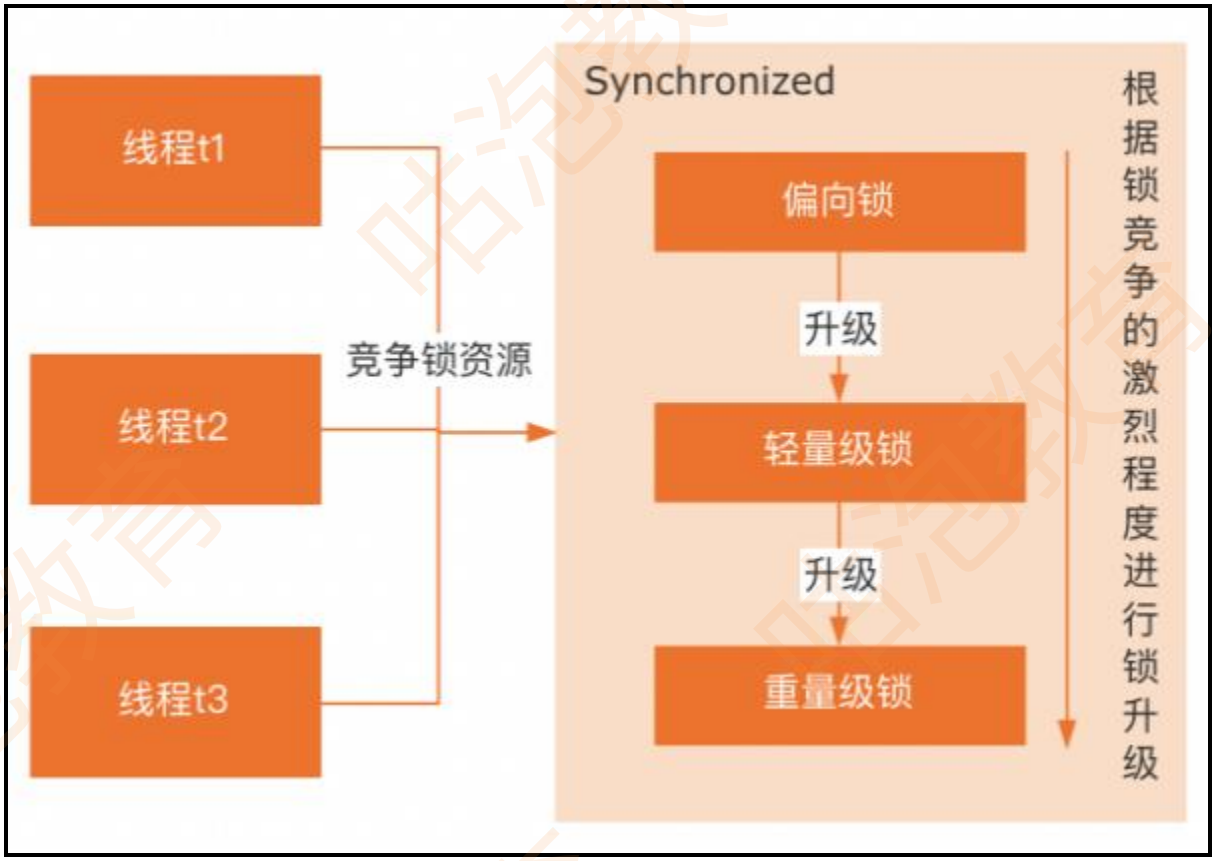
Synchronized引入了锁升级的机制之后，如果有线程去竞争锁：

首先，synchronized会尝试使用偏向锁的方式去竞争锁资源，如果能够竞争到偏向锁，表示加锁成功直接返回。如果竞争锁失败，说明当前锁已经偏向了其他线程。

需要将锁升级到轻量级锁，在轻量级锁状态下，竞争锁的线程根据自适应自旋次数去尝试抢占锁资源，如果在轻量级锁状态下还是没有竞争到锁，

就只能升级到重量级锁，在重量级锁状态下，没有竞争到锁的线程就会被阻塞，线程状态是Blocked。

处于锁等待状态的线程需要等待获得锁的线程来触发唤醒。



总的来说，Synchronized的锁升级的设计思想，在我看来本质上是一种性能和安全性的平衡，也就是如何在不加锁的情况下能够保证线程安全性。

这种思想在编程领域比较常见，比如Mysql里面的MVCC使用版本链的方式来解决多个并行事务的竞争问题。

以上就是我对这个问题的理解。

面试点评

锁在程序中是非常常见的内容，我们几乎每天与锁打交道，比如Mysql里面的行锁、表锁。

因此它的重要性也不言而喻。

我们从高手的回答中可以明显的看到高手对Synchronized的理解层次是非常高的。

好的，本期的普通人VS高手面试系列的视频就到这里结束了。

喜欢我的作品的小伙伴记得点赞和收藏加关注。

我是Mic，一个工作14年的Java程序员，咱们下期再见！

11.38 Eureka server数据同步原理能说下吗

Hi，大家好，我是Mic，一个没有才华只能靠颜值混饭吃的Java程序员。

昨天，一个工作了4年的Java程序员，去一个互联网公司面试，在面试的时候面试官一个劲问他Spring Cloud Netflix组件的底层原理，开始还能回答出来，越到后面越懵逼，最后面试没过。

他有点不理解面试官怎么会问这么深，当他把简历发给我的时候，我看到简历上写了精通Spring Cloud。

瞬间明白了原因。

Spring Cloud Netflix里面的组件可以考察的方向太多了，比如今天分享的这道面试题：

“Eureka server数据同步原理”，相信80%的同学都回答不出来。

下面看看高手对这个问题的回答。

高手

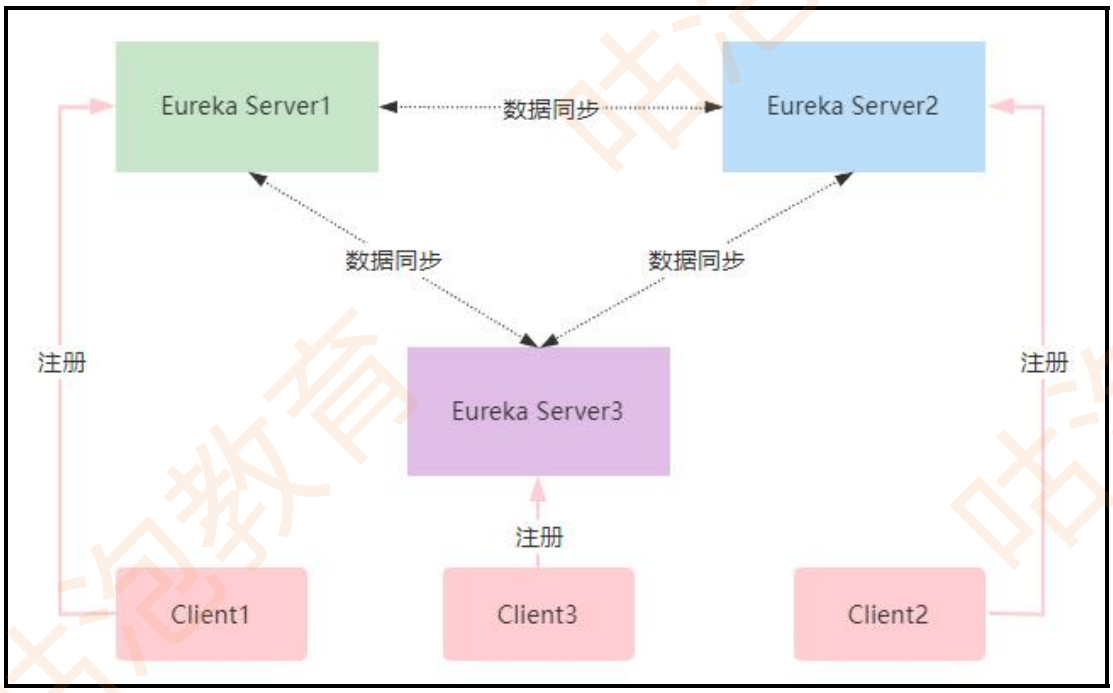
Eureka是一个服务注册中心，在Eureka的设计里面，为了保证Eureka的高可用性，提供了集群的部署方式。

Eureka的集群部署采用的是两两相互注册的方式来实现，也就是说每个Eureka Server节点都需要发现集群中的其他节点并建立连接，然后通过心跳的方式来维持这个连接的状态。

Eureka Server集群节点之间的数据同步方式非常简单粗暴，使用的是对等复制的方式来实现数据同步。

也就是说，在Eureka Server集群中，不存在所谓主从节点，任何一个节点都可以接收或者写入数据。

一旦集群中的任意一个节点接收到了数据的变更，就直接同步到其他节点上。



这种无中心化节点的数据同步，需要考虑到一个数据同步死循环的问题，也就是需要区分Eureka Server收到的数据是属于客户端传递来的数据还是集群中其他节点发过来的同步数据。

Eureka使用了一个时间戳的标记来实现类似于数据的版本号来解决这个问题。

另外，从Eureka的数据同步方案来看，Eureka集群采用的是AP模型，也就是只提供高可用保障，而不提供数据强一致性保障。

之所以采用AP，我认为注册中心它只是维护服务之间的通信地址，数据是否一致对于服务之间的通信影响并不大。

而注册中心对Eureka的高可用性要求会比较高，不能出现因为Eureka的故障导致服务之间无法通信的问题。

以上就是我对这个问题的理解。

面试点评

Eureka虽然闭源了，但是在国内依然使用较为广泛。

当然有些公司逐步迁移到了Nacos上面，但是Eureka的整个框架设计上还是有非常多值得我们学习的思想。

多级缓存设计、集群之间的数据同步方案、多区域隔离以及就近访问的设计等等。

一个技术框架，我们 能够获得这些优秀理念，对未来的职业发展帮助是非常大的。

我是Mic，咱们下期再见!

11.39 请你说一下你对Happens-Before的理解。

“请你说一下你对Happens-Before的理解”

屏幕前的你，听到这个问题的时候，知道怎么回答吗？

Hi，大家好，我是Mic，一个工作了14年的Java程序员。

并发编程是面试过程中重点考察的方向，能够考察的方向有很多

关于这个问题，我把高手回答整理到了15W字的面试文档里面

大家可以在我的主页加V领取

下面看看高手的回答。

高手

好的，这个问题我需要从几个方面来回答。

首先，Happens-Before是一种可见性模型，也就是说，在多线程环境下。

原本因为指令重排序的存在会导致数据的可见性问题，也就是A线程修改某个共享变量

对B线程不可见。

因此，JMM通过Happens-Before关系向开发人员提供跨越线程的内存可见性保证。

如果一个操作的执行结果对另外一个操作可见，那么这两个操作之间必然存在Happens-Before管理。

其次，Happens-Before关系只是描述结果的可见性，并不表示指令执行的先后顺序，也就是说只要不对结果产生影响，仍然允许指令的重排序。

最后，在JMM中存在很多的Happens-Before规则。

程序顺序规则，一个线程中的每个操作，happens-before这个线程中的任意后续操作，可以简单认为是as-if-serial也就是不管怎么重排序，单线程的程序的执行结果不能改变传递性规则，也就是A Happens-Before B，B Happens-Before C。

就可以推导出A Happens-Before C。

```
int a=2; //A
int b=2; //B
int c=a*b; //C

A happens-before B
B happens-before C
A happens-before C
```

volatile变量规则，对一个volatile修饰的变量的写一定happens-before于任意后续对这个volatile变量的读操作监视器锁规则，一个线程对于一个锁的释放锁操作，一定happens-before与后续线程对这个锁的加锁操作在这个场景中，如果线程A获得了锁并且把x修改成了12，那么后续的线程获得锁之后得到的x的值一定是12。

```
int x=10;
synchronized (this) { // 此处自动加锁
 // x 是共享变量, 初始值 =10
 if (this.x < 12) {
    this.x = 12;
 }
} // 此处自动解锁
```

线程启动规则，如果线程A执行操作ThreadB.start(),那么线程A的ThreadB.start()之前的操作happens-before线程B中的任意操作。

在这样一个场景中，t1线程启动之前对于x=10的赋值操作，t1线程启动以后读取x的值一定是10。

```
public StartDemo{
    int x=0;
    public void test(){
      Thread t1 = new Thread(()->{
        // 主线程调用 t1.start() 之前
        // 所有对共享变量的修改，此处皆可见
        // 此例中，x==10
      });
      // 此处对共享变量 x修改
      x = 10;
      // 主线程启动子线程
      t1.start();
    }
}
```

join规则，如果线程A执行操作ThreadB.join()并成功返回，那么线程B中的任意操作happens-before于线程A从ThreadB.join()操作成功的返回。

```
Thread t1 = new Thread(()->{
  // 此处对共享变量 x 修改
  x= 100;
});
// 例如此处对共享变量修改，
// 则这个修改结果对线程 t1 可见
// 主线程启动子线程
t1.start();
t1.join()
// 子线程所有对共享变量的修改
// 在主线程调用 t1.join() 之后皆可见
// 此例中，x==100
```

以上就是我对Happens-Before的理解。

总结

Happens-Before模型，在多线程开发中是必须要理解和掌握的规则。

它能够指引开发者在使用多线程开发的时候避免出现内存可见性问题

因此这道面试题其实也是考察求职者的基础能力

11.40 什么是ISR，为什么需要设计ISR

Hi，大家好，我是Mic。

一个工作5年的粉丝，在简历上写精通Kafka。

结果在面试的时候直接打脸。

面试官问他： “什么是ISR，为什么需要设计ISR”

然后他一脸懵逼的看着面试官。

下面看看高手的回答。

高手

好的，关于这个问题，我需要从几个方面来回答。

首先，发送到Kafka Broker上的消息，最终是以Partition的物理形态来存储到磁盘上的。

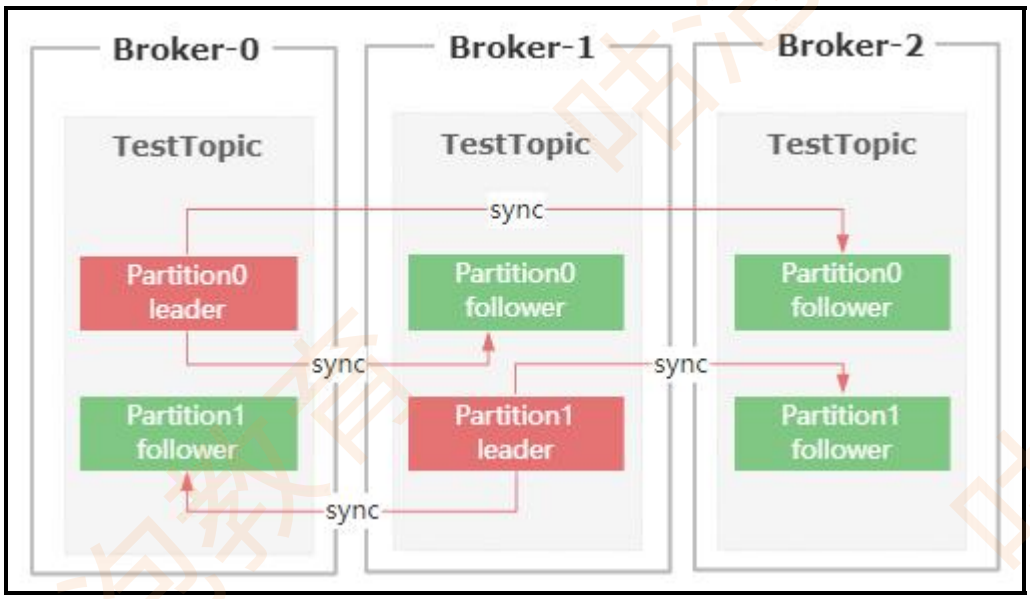
而Kafka为了保证Parititon的可靠性，提供了Paritition的副本机制，然后在这些Partition副本集里面。

存在Leader Partition和Flollower Partition。

生产者发送过来的消息，会先存到Leader Partition里面，然后再把消息复制到Follower Partition，

这样设计的好处就是一旦Leader Partition所在的节点挂了，可以重新从剩余的Partition副本里面选举出新的Leader。

然后消费者可以继续从新的Leader Partition里面获取未消费的数据。



在Partition多副本设计的方案里面，有两个很关键的需求。

1.副本数据的同步

2.新Leader的选举

这两个需求都需要涉及到网络通信，Kafka为了避免网络通信延迟带来的性能问题，以及尽可能的保证新选举出来的Leader Partition里面的数据是最新的，所以设计了ISR这样一个方案。

ISR全称是 in-sync replica，它是一个集合列表，里面保存的是和Leader Parition节点数据最接近的Follower Partition如果某个Follower Partition里面的数据落后Leader太多，就会被剔除ISR列表。

简单来说，ISR列表里面的节点，同步的数据一定是最新的，所以后续的Leader选举，只需要从ISR列表里面筛选就行了。

所以，我认为引入ISR这个方案的原因有两个

尽可能的保证数据同步的效率，因为同步效率不高的节点都会被踢出ISR列表。

避免数据的丢失，因为ISR里面的节点数据是和Leader副本最接近的。

以上就是我对这个问题的理解。

面试总结

在我看来，这个问题非常有研究价值。

一般来说，副本数据同步，无非就是同步阻塞、或者异步非阻塞。

但是这两种方案，要么带来性能问题，要么带来数据丢失问题，都不是特别合适。

而ISR，就非常完美解决了这个问题，在实际过程中，我们也可以借鉴类似的设计思路。

11.41 为什么两个Integer的对象不能用==号来判断?

Hi，大家好，我是Mic，一个工作了14年的程序员和创业者。

昨天一个工作4年的粉丝私信我，他说最近背了一个生产事故，想让我来给大家分享一下避免采坑。

他是做理财这块业务的，他每天会收到一个基金公司的收益文件，然后他需要把这个文件解析并且保存每个用户的收益数据到数据库。

在解析文件的时候，他需要对数据的条数做校验，于是用到了Integer这个对象并且使用==来判断。

测试环境都没问题，但是到了生产环境上出现用户收益没有到账的问题，造成了大规模的投诉。

最后定位才发现是收益文件验证失败导致没有被解析入库。

所以这里就出现一个问题： “为什么两个Integer的对象不能用==号来判断？为什么测试环境没有把这问题测试出来”。

另外我花了1个多星期的时间，把往期高手回答整理成了10W字的文档，想获取的小伙伴可以从我的个人煮叶简介加微领取

下面看看高手对这个问题的回答。

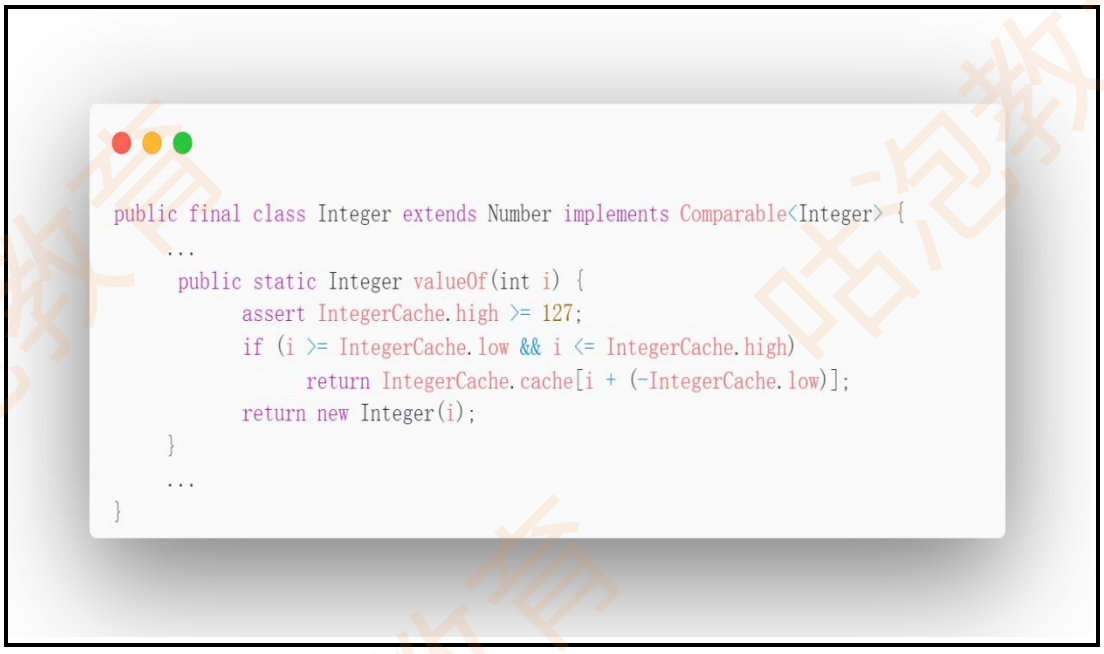
高手

Integer是一个封装类型。它是对应一个int类型的包装。

在Java里面之所以要提供Integer这种基本类型的封装类，是因为Java是一个面向对象的语言，而基本类型不具备对象的特征，所以在基本类型上做了一层对象的包装并且提供了相关的属性和访问方法来完善基本类型的操作。

在Integer这个封装类里面，除了基本的int类型的操作之外，还引入了享元模式的设计，对-128到127之间的数据做了一层缓存，也就是说，如果Integer类型的目标值在-128到127之间，

就直接从缓存里面获取Integer这个对象实例并返回，否则创建一个新的Integer对象。



学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

这么设计的好处是减少频繁创建Integer对象带来的内存消耗从而提升性能。

因此在这样一个前提下，如果定义两个Integer对象，并且这两个Integer的取值范围正好在-128到127之间。

如果直接用==号来判断，返回的结果必然是true，因为这两个Integer指向的内存地址是同一个。

否则，返回的结果是false。

之所以在测试环境上没有把这个问题暴露出来，是因为测试环境上验证的数据量有限，使得取值的范围正好在Integer

的缓存区间，从而通过了测试。

但是在实际的应用里面，数据量远远超过IntegerCache的取值范围，所以就导致了校验失败的问题。

以上就是我对这个问题的理解。

面试点评

你看，对Java基础有一个非常深度的理解是很重要的。

一个小小的知识点，往往能够造成生产环境上较大规模的影响。

所以在一些大厂面试中，基础的考察比例会比较重。

另外一个层面，中间件和技术框架已经是一个成熟的产品，所以即便我们再怎么乱玩，造成的影响也有限。

好的，本期的普通人VS高手面试系列的视频就到这里结束了。

大家记得点赞收藏加关注

我是Mic，咱们下一期再见。

11.42 请说一下Netty中Reactor模式的理解

hi，大家好，我是Mic，一个没有才华只能靠颜值混饭吃的Java程序员。

今天一个工作了6年的粉丝，去美团面试的时候遇到一个比较有意思的问题。

应该大部分同学对这个领域都比较陌生，因为网络编程在实际开发中接触还是比较少的。

这个问题是：“请说一下Netty中Reactor模式的理解”？

这个问题高手部分的回答已经整理到了10W字的面试文档里面，大家可以在我的主页加V领取

下面看看普通人和高手的回答

高手

Reactor其实是在NIO多路复用的基础上提出的一个高性能IO设计模式。

它的核心思想是把响应IO事件和业务处理进行分离，通过一个或者多个线程来处理IO事件。

然后把就绪的事件分发给业务线程进行异步处理。

Reactor模型有三个重要的组件：

Reactor ： 把I/O事件分发给对应的Handler

Acceptor ： 处理客户端连接请求

Handlers ： 执行非阻塞读/写，也就是针对收到的消息进行业务处理。

在Reactor的这种设计中，有三种模型分别是

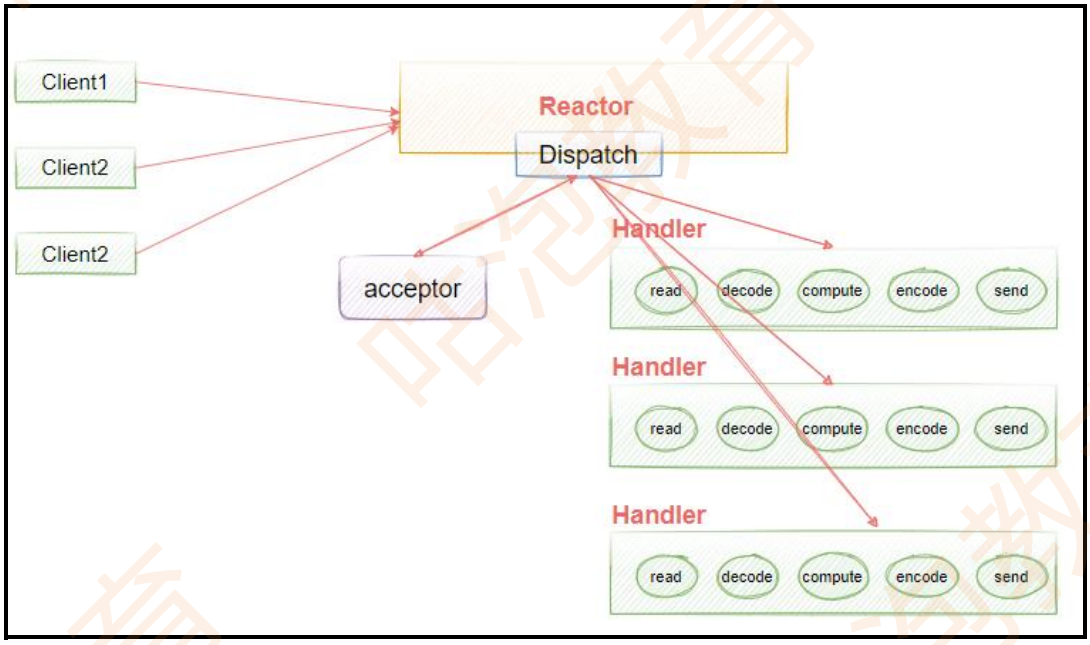
单线程Reactor模型。

多线程Reactor模型。

主从多Reactor多线程模型。

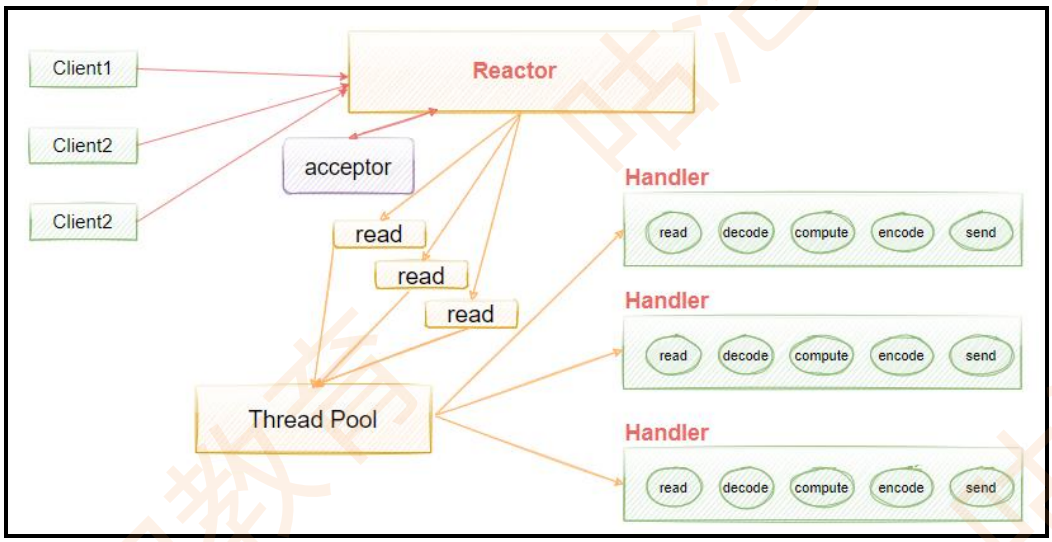
单线程Reactor模型，就是由同一个线程来负责处理IO事件以及业务逻辑。

这种方式的缺点在于handler的执行过程是串行，如果有任何一个handler处理线程阻塞，就会影响整个服务的吞吐量。



所以，就有了多线程Reactor模型。

也就是把处理IO就绪事件的线程和处理Handler业务逻辑的线程进行分离，每个Handler由一个独立线程来处理，在这种设计下，即便是存在Handler线程阻塞问题，也不会对IO线程造成影响。



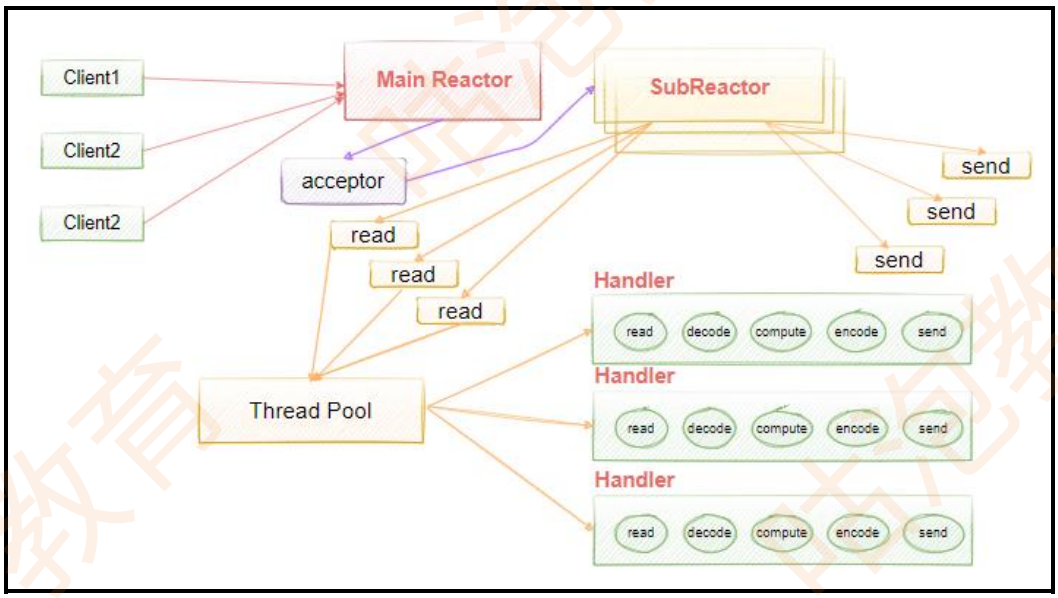
在多线程Reactor模型下，所有的IO操作都是由一个Reactor来完成的，而且Reactor运行在单个线程里面。

对于并发较高的场景下，Reactor就成为了性能瓶颈，所以在这个基础上做了更进一步优化。

提出了多Reactor多线程模型，这种模式也叫Master-Workers模式。

它把原本单个Reactor拆分成了Main Reactor和多个SubReactor，Main Reactor负责接收客户端的链接，然后把接收到的连接请求随机分配到多个subReactor里面。

SubReactor负责IO事件的处理。



这种方式另外一个好处就是可以对subReactor做灵活扩展，从而

适应不同的并发量，解决了单个Reactor模式的性能瓶颈问题。

以上就是我对Reactor模型的理解。

面试点评

Reactor模型的设计比较常见，比如Spring里面的Webflux就用了这种设计。

并且像Master-Worker模型，在Memcached和Nginx中都有用到。

所以我们其实可以去理解并学习这种设计思想，也许在某些业务场景中

可以帮助我们多提供一个解决思路。

11.43 讲一下wait和notify这个为什么要在synchronized代码块中?

一个工作七年的小伙伴，竟然不知道”wait”和“notify”为什么要在Synchronized代码块里面。

对于这个问题，我们来看看高手的回答。

高手

wait和notify用来实现多线程之间的协调，wait表示让线程进入到阻塞状态，notify表示让阻塞的线程唤醒。

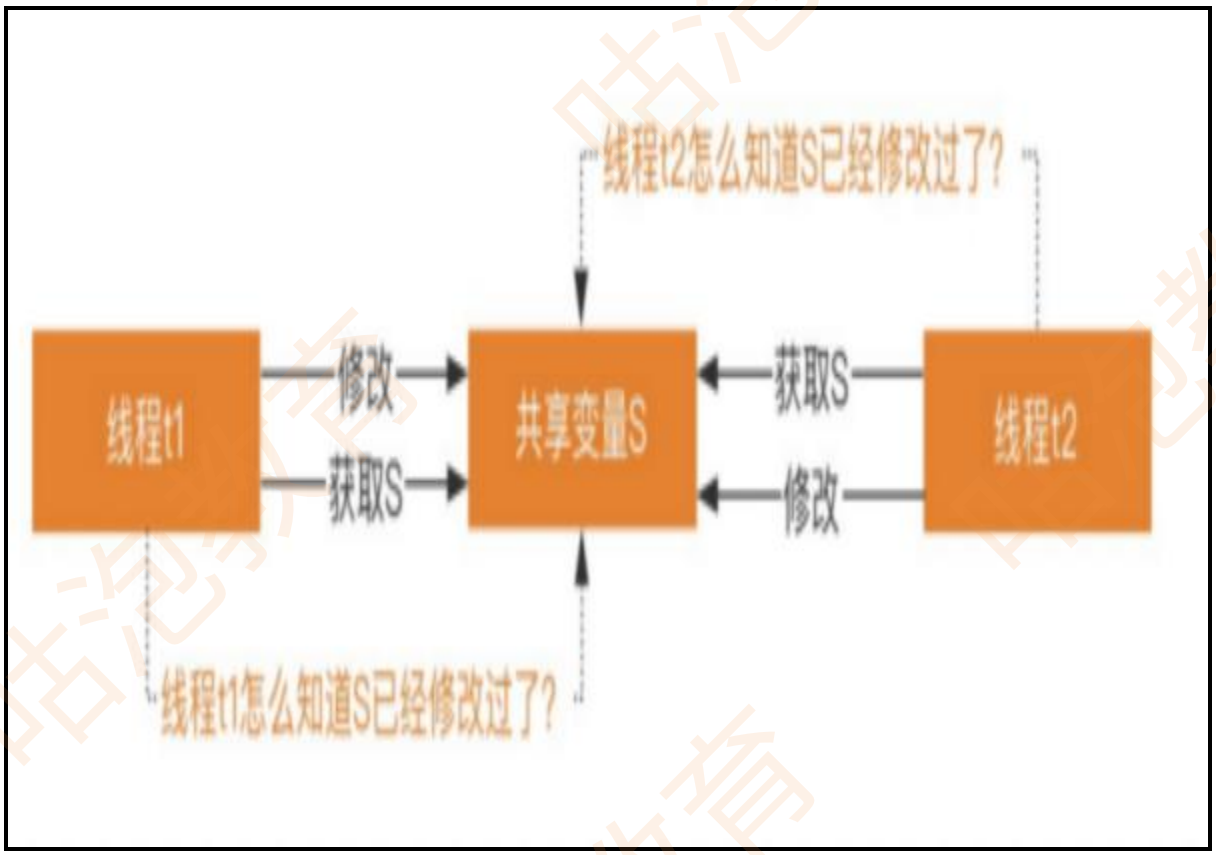
wait和notify必然是成对出现的，如果一个线程被wait()方法阻塞，那么必然需要另外一个线程通过notify()方法来唤醒这个被阻塞的线程，从而实现多线程之间的通信。

在多线程里面，要实现多个线程之间的通信，除了管道流以外，只能通过共享变量的方法来实现，也就是线程t1修改共享变量s，线程t2获取修改后的共享变量s，从而完成数据通信。

但是多线程本身具有并行执行的特性，也就是在同一时刻，多个线程可以同时执行。在这种情况下，线程t2在访问共享变量s之前，必须要知道线程t1已经修改过了共享变量s，否则就需要等待。

同时，线程t1修改过了共享变量S之后，还需要通知在等待中的线程t2。

所以要在这种特性下要去实现线程之间的通信，就必须要有一个竞争条件控制线程在什么条件下等待，什么条件下唤醒。



而Synchronized同步关键字就可以实现这样一个互斥条件，也就是在通过共享变量来实现多个线程通信的场景里面，参与通信的线程必须要竞争到这个共享变量的锁资源，才有资格对共享变量做修改，修改完成后就释放锁，那么其他的线程就可以再次来竞争同一个共享变量的锁来获取修改后的数据，从而完成线程之前的通信。

所以这也是为什么wait/notify需要放在Synchronized同步代码块中的原因，有了Synchronized同步锁，就可以实现对多个通信线程之间的互斥，实现条件等待和条件唤醒。

另外，为了避免wait/notify的错误使用，jdk强制要求把wait/notify写在同步代码块里面，否则会抛出IllegalMonitorStateException

最后，基于wait/notify的特性，非常适合实现生产者消费者的模型，比如说用wait/notify来实现连接池就绪前的等待与就绪后的唤醒。

以上就是我对wait/notify这个问题的理解。

面试点评

这个是一个典型的经典面试题。

其实考察的就是Synchronized、wait/notify的设计原理和实现原理。

由于wait/notify在业务开发整几乎不怎么用到，所以大部分人回答不出来。

其实并发这块内容理论上来说所有程序员都应该要懂，不管是它的应用价值，还是设计理念，非常值得学习和借鉴。

好的，本期的普通人VS高手面试系列的视频就到这里结束了，喜欢的朋友记得点赞和收藏。

另外，有任何技术上的问题，职业发展有关的问题，都可以私信我，我会在第一时间回复。

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

11.44 wait和sleep是否会触发锁的释放以及CPU资源的释放?

通过最近这几个月的私信发现一个问题，很多工作了5~6年的程序员，去面试的时候但凡问到技术原理。

基本上都是回答不出来的，有些同学侥幸靠背面试题通过面试，但是这种无法掌控自己选择权的感觉，你不觉得很难受吗?

hi，大家好，我是Mic，一个没有才华只能靠颜值混饭吃的Java程序员。

一个工作5年的粉丝，去美团面试，遇到了这样一个问题。

“wait和sleep是否会触发锁的释放以及CPU资源的释放？”

其实这个问题还比较简单，它的结论回答出来了，但是后面面试官又问了一个为什么，他就懵了。

下面看看普通人和高手的回答

普通人

高手

好的，面试官。

Object.wait()方法，会释放锁资源以及CPU资源。

Thread.sleep()方法，不会释放锁资源，但是会释放CPU资源。

首先，wait()方法是让一个线程进入到阻塞状态，而这个方法必须要写在一个Synchronized同步代码块里面。

因为wait/notify是基于共享内存来实现线程通信的工具，这个通信涉及到条件的竞争，所以在调用这两个方法之前必须要竞争锁资源。

当线程调用wait方法的时候，表示当前线程的工作处理完了，意味着让其他竞争同一个共享资源的线程有机会去执行。

但前提是其他线程需要竞争到锁资源，所以wait方法必须要释放锁，否则就会导致死锁的问题。

然后，Thread.sleep()方法，只是让一个线程单纯进入睡眠状态，这个方法并没有强制要求加synchronized同步锁。

而且从它的功能和语义来说，也没有这个必要。

当然，如果是在一个Synchronized同步代码块里面调用这个Thread.sleep，也并不会触发锁的释放。

最后，凡是让线程进入阻塞状态的方法，操作系统都会重新调度实现CPU时间片切换，这样设计的目的是提升CPU的利用率。

以上就是我对这个问题的理解。

面试点评

之前我用这个问题去面试过一些工作3~5年的人，有大部分人回答不上来。

这让我有点意外，正常来说，并发编程是一个非常重要且基础的领域，

在程序开发中，也是比较常用的技术。还是建议大家去认真学一下。

大家记得点赞收藏+关注

我是Mic，咱们下期再见！

11.45 存储MD5的值应该用VARCHAR还是CHAR

昨天一个工作4年的粉丝，遇到了一个很有意思的面试题。

“存储MD5的值应该用VARCHAR还是CHAR”！

觉得是用VARCHAR类型的小伙伴，请仔细看完这一个视频。

Hi，大家好，我是Mic，一个工作14年的Java程序员

这道面试题的文字版

我已经整理在20万字的文档里了

有需要的可以在看我置顶第一条视频领取

考察目的

这个问题考察1~4年左右开发经验的同学。

大家肯定会觉得这个问题很简单啊，直接回答VARCHAR就行了。

但如果只是这么简单，你就太单纯了。

这个问题考察的目的至少有两个：

考察数据库里面的基本数据类型的理解

基于这个问题作为切入口，了解求职者对数据库的掌握程度

这个问题后面，一定伴随这“为什么”，

面试官一定是不不不去挖坑，让你往里面跳，直到你回答不上来位置。

所以我们在回答这个问题的时候，就需要更加全面的去回答。

问题分析

MD5是由数字和字母组成的一个16位或者32位长度的字符串，一般在应用开发中都是使用32位。

看起来，我们用varchar(32)或者char(32)都可以存储，那用哪种更好呢?

要回答这个问题，必须要了解这两个类型的功能特性和区别。

第一个，char是一个固定长度的字符串，Varchar是一个可变长度的字符串

假设声明一个char(10)的长度，如果存储字符串“abc”，虽然实际字符长度只有3，但是char还是会占10个字节长度。

同样，如果用varchar存储，那它只会使用3个字符的实际长度来存储。

第二个，存储的效率不同，char类型每次修改以后存储空间的长度不变，所以效率更高

varchar每次修改数据都需要更新存储空间长度，效率较低

第三个，存储空间不同，char不管实际数据大小，存储空间是固定的，而varchar存储空间等于实际数据长度，所以varchar实际存储空间的使用要比char更小

基于他们特性的分析，可以得出一个基本的结论：

char适合存储比较短的且是固定长度的字符串

varchar适合存储可变长度的字符串

高手

我认为应该使用Char类型，原因是：

char类型是固定长度的字符串，varchar是可变长度字符串。

而MD5是一个固定长度的字符，不管数据怎么修改，长度不变，这个点很符合char类型。

另外，由于是固定长度，所以在数据变更的时候，不需要去调整存储空间大小，在效率上会比varchar好。

以上就是我的理解。

总结

大家知道怎么回答了吗?

如果你喜欢我的作品，记得点赞收藏加关注哦

我是Mic，咱们下期再见。

11.46 什么是受检异常和非受检异常吗?

普通人

受检异常好像是需要主动捕获的异常非受检异常应该是不需要捕获的异常

高手

我觉得可以从三个方面回答这个问题

一、首先是异常的本质

受检异常和非受检异常，都是继承自Throwable这个类中，分别是Error和Exception，Error是程序报错，系统收到无法处理的错误消息，它和程序本身无关。

Excetpion是指程序运行时抛出需要处理的异常信息如果不主动捕获，则会被jvm处理。

二、然后是对受检异常和非受检异常的定义

前面说过受检异常和非受检异常均派生自Exception这个类。

1. 受检异常的定义是程序在编译阶段必须要主动捕获的异常，遇到该异常有两种处理方法

通过try/catch捕获该异常或者通过throw把异常抛出去

2. 非受检异常的定义是程序不需要主动捕获该异常，一般发生在程序运行期间，比如NullPointException

三、最后我还可以说下他们优点和缺点

受检异常优点有两个：

第一，它可以响应一个明确的错误机制，这些错误在写代码的时候可以随时捕获并且能很好的提高代码的健壮性。

第二，在一些连接操作中，它能很好的提醒我们关注异常信息，并做好预防工作。

不过受检异常的缺点是：抛出受检异常的时候需要上声明，而这个做法会直接破坏方法签名导致版本不兼容。这个恶心特性导致我会经常使用RuntimeException包装。

非受检异常的好处是可以去掉一些不需要的异常处理代码，而不好之处是开发人员可能忽略某些应该处理的异常，导致带来一些隐藏很深的Bug，比如流忘记关闭？连接忘记释放等。

这些就是我对这个问题的回答！

蚂蚁金服一面（12）

12.1 用到分布式事务嘛?为什么用这种方案,有其他方案嘛?

12.1.1 什么✂分布式事务

谈到事务，我们就会想到**数据库事务**，很容易就想到**原子性、一致性、持久性、隔离性。**

分布式事务跟数据库事务有点不一样，它✂指事务✂参与者、支持事务✂服务器、资源服务器以及事务管理器分别位于不同✂分布式系统✂不同节点之上。简单来说，分布式事务指✂就✂分布式系统中✂事务，它✂存在就✂为了保证不同数据库节点✂数据一致性。

12.1.2 分布式事务基础

分布式事务需要需要知道 **CAP 理论**和 **BASE 理论。**

CAP 理论

- 一致性(C：Consistency)：一致性✂指数据在多个副本之间能否保持一致✂特性。例如一个数据在某个分区节点更新之后，在其他分区节点读出来✂数据也✂更新之后✂数据。
- 可用性(A：Availability)：可用性✂指系统提供✂服务必须一直处于可用✂状态，对于用户✂每一个操作请求总✂能够在有限✂时间内返回结果。这里✂重点✂"有限时间内"和"返回结果"。
- 分区容错性（P:Partition tolerance）:分布式系统在遇到任何网络分区故障✂时候，仍然需要能够保证对外提供满足一致性和可用性✂服务。

一个分布式系统中，CAP 理论它只能同时满足（一致性、可用性、分区容错性）中✂两点。

BASE 理论

BASE 理论， ✂对CAP 中 AP ✂一个扩展，对于我们✂业务系统，我们考虑牺牲一致性来换取系统✂可用性和分区容错性。BASE ✂ Basically

Available(基本可用)，Soft state（软状态）,和 Eventually consistent（最终一致性）三个短语✂缩写。

- 基本可用✂指，通过支持局部故障而不✂系统全局故障来实现✂；
- Soft State 表示状态可以有一段时间不同步；
- 最终一致，最终数据✂一致✂就可以了，而不✂实时保持强一致。

12.1.3 分布式事务✂几种解决方案

- **2PC(二阶段提交)方案**，事务✂提交分为两个阶段：准备阶段和提交执行方案。
- **TCC（✂ Try、Confirm、Cancel）**，它采用了补偿机制，核心思想✂：针对每个操作，都要注册一个与其对应✂确认和补偿（撤销）操作。
- **本地✲息表**，它✂核心思想就✂将分布式事务拆分成本地事务进行处理。
- **最大努力通知**，实现最大努力通知，可以采用 MQ ✂ ack 机制。
- **Saga 事务**，它✂核心思想✂将长事务拆分为多个本地短事务，由 Saga 事务协调器协调，如果正常结束那就正常完成，如果某个步骤失败，则根据相反顺序一次调用补偿操作。

业界目前使用**本地✲息表**这种方案✂比较多✂，它✂核心思想就✂将分布式事务拆分成本地事务进行处理。可以看一下基本✂实现流程：

对于消息发送方：

- 首先需要有一个消息表，记录着消息状态相关信息。
- 业务数据和消息表在同一个数据库，✂要保证它俩在同一个本地事务。
- 在本地事务中处理完业务数据和写消息表操作后，通过写✻息到 MQ 消息队列。
- 消息会发到消息消费方，如果发送失败，✂进行重试。

消息消费方：

- 处理消息队列中✂消息，完成自己✂业务逻辑。
- 此时如果本地事务处理成功，则表明已经处理成功了。
- 如果本地事务处理失败，那么就会重试执行。
- 如果✂业务上面✂失败，给消息生产方发送一个业务补偿消息，通知进行回滚等操作。

生产方和消费方定时扫描本地消息表，✂还没处理完成✂消息或者失败✂消息再发送一遍。如果有靠谱✂自动对账补账逻辑，这种方案还✂非常实用✂。

TOM老师解析

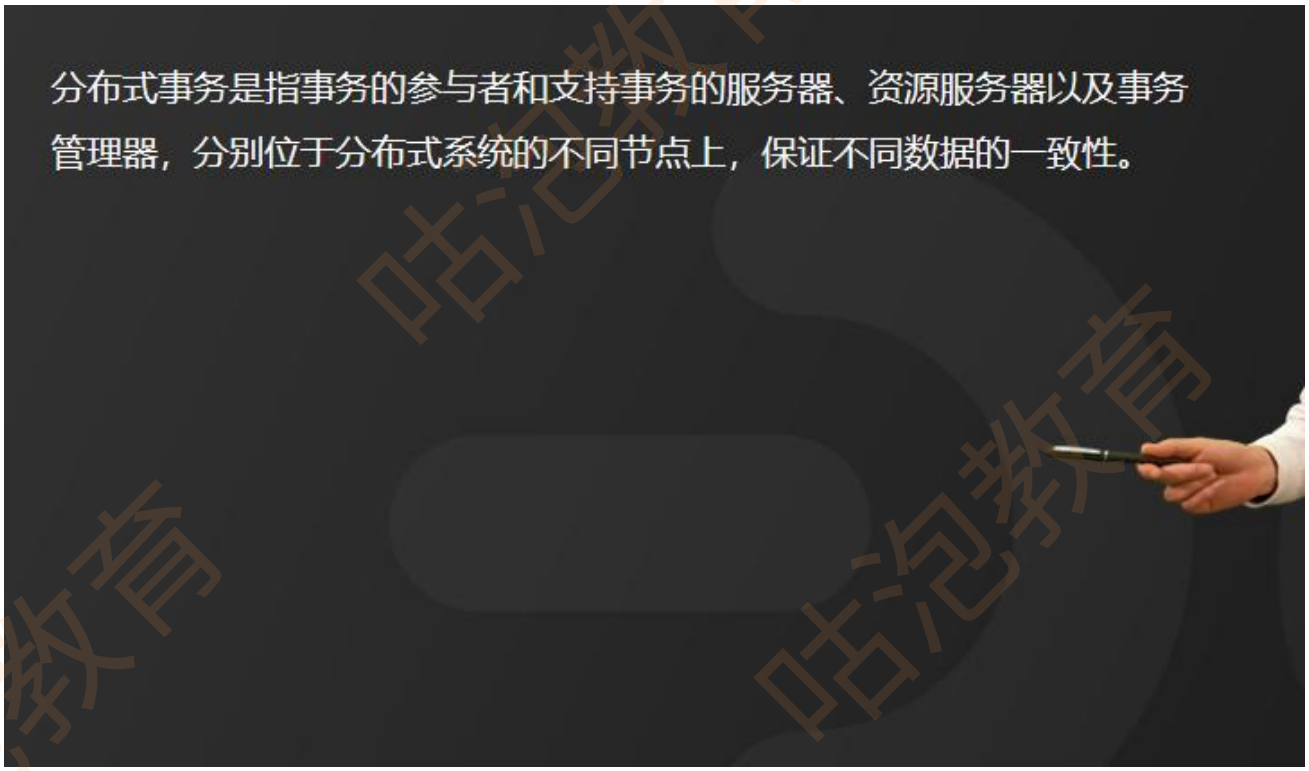
【引言】

前几天，有一位10多年经验的架构师在面试互联网大厂时被问到这样一个问题，说请你谈谈分布式事务的解决方案。那今天，我给大家分享一下我对这个问题的理解。

另外，我花了1个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

1、什么是分布式事务

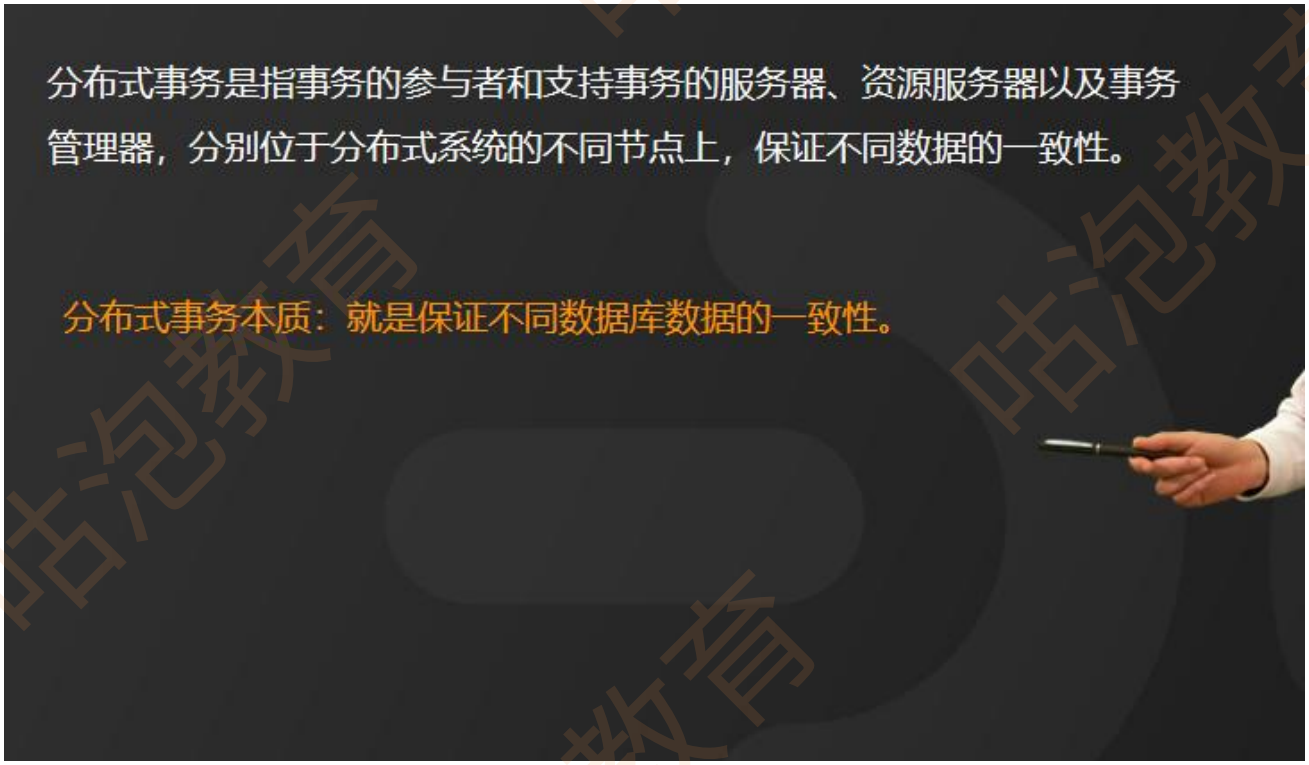
分布式事务是指事务的参与者和支持事务的服务器、资源服务器以及事务管理器，分别位于分布式系统的不同节点上，保证不同数据的一致性。



比如大型电商系统中的下单场景，会涉及到扣库存、优惠折扣计算、订单ID生成等这些服务，通常情况下库存、折扣、订单ID生成的服务都位于不同的服务器和数据库中，那么下单是否成功，不仅取决于本地节点的数据库操作，还需要依赖其他服务的执行结果结果，这个时候分布式事务就是

保证这些操作要么全部成功，要么全部失败。

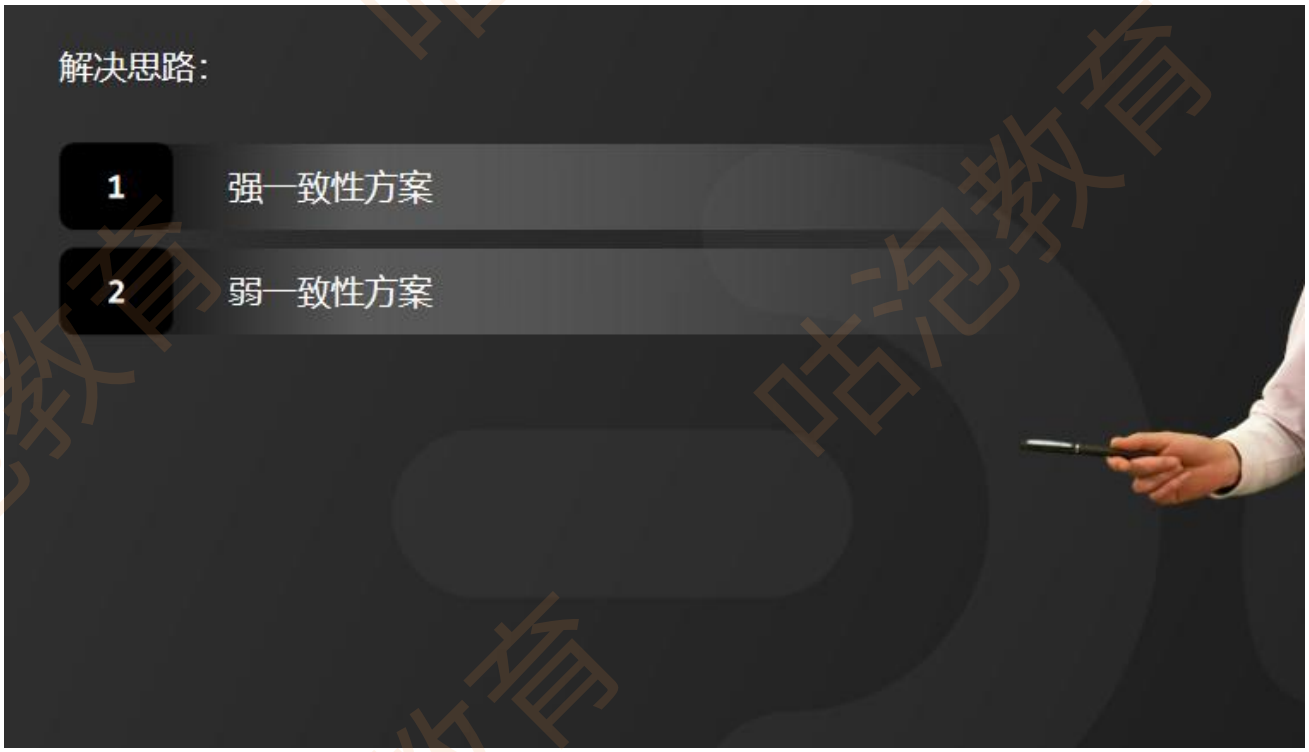
因此，本质上来说，分布式事务就是为了保证不同数据库数据的一致性。



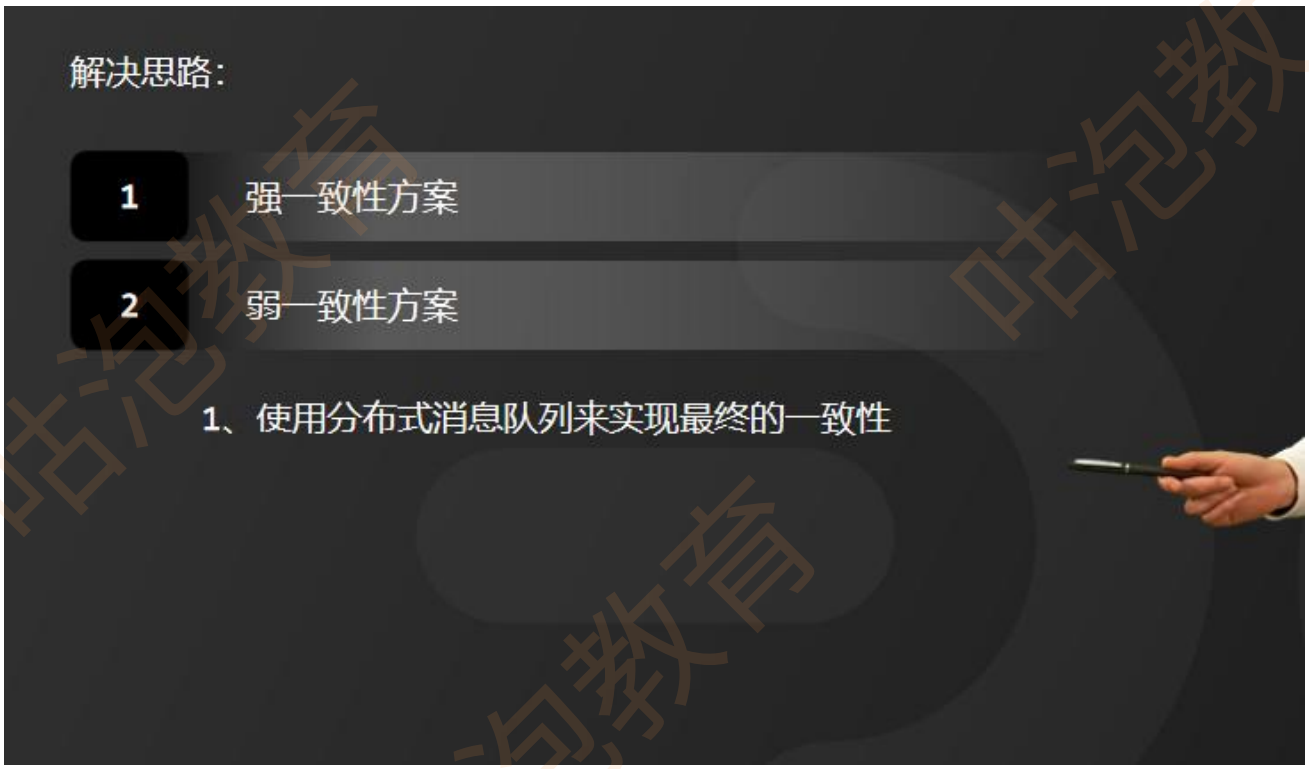
2、分布式事务解决方案

基于CAP定理和Base理论，我们可以知道，对于上述情况产生的分布式问题，我们要么采用强一致性方案，要么采用弱一致性方案。

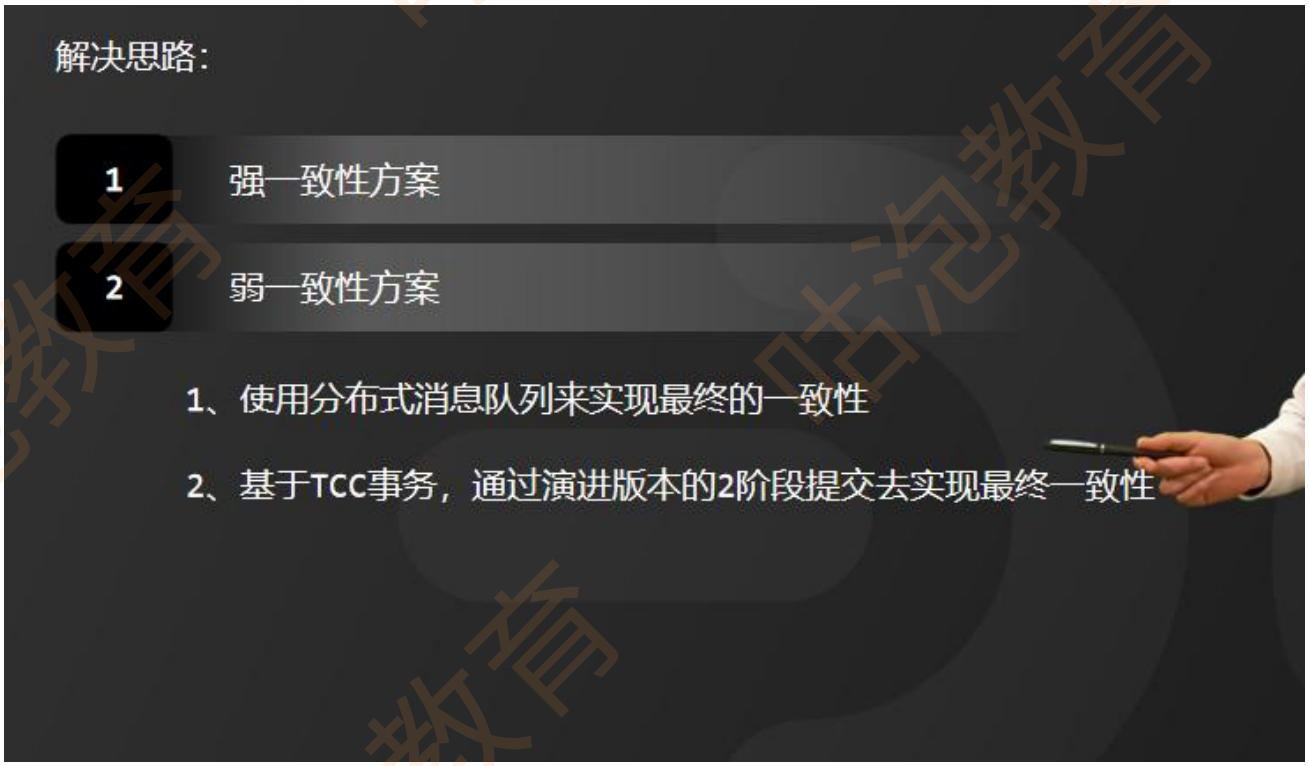
所谓强一致性方案，是指通过第三方事务管理器来协调多个节点的事务性，保证每一个节点的事务达到同时成功或者同时失败，为了实现这样一个需求。我们可以引入X/Open DTP模型提供的XA协议，基于2阶段提交或者3阶段提交的方式去实现，但是如果全局事务管理器中的多个节点，任意一个节点在进行事务提交确认的时候，由于网络通信延迟导致了阻塞，就会影响到所有节点的事务提交，而这个阻塞过程呢，也会影响到用户的请求线程，这对于用户体验以及整体的性能影响非常大。



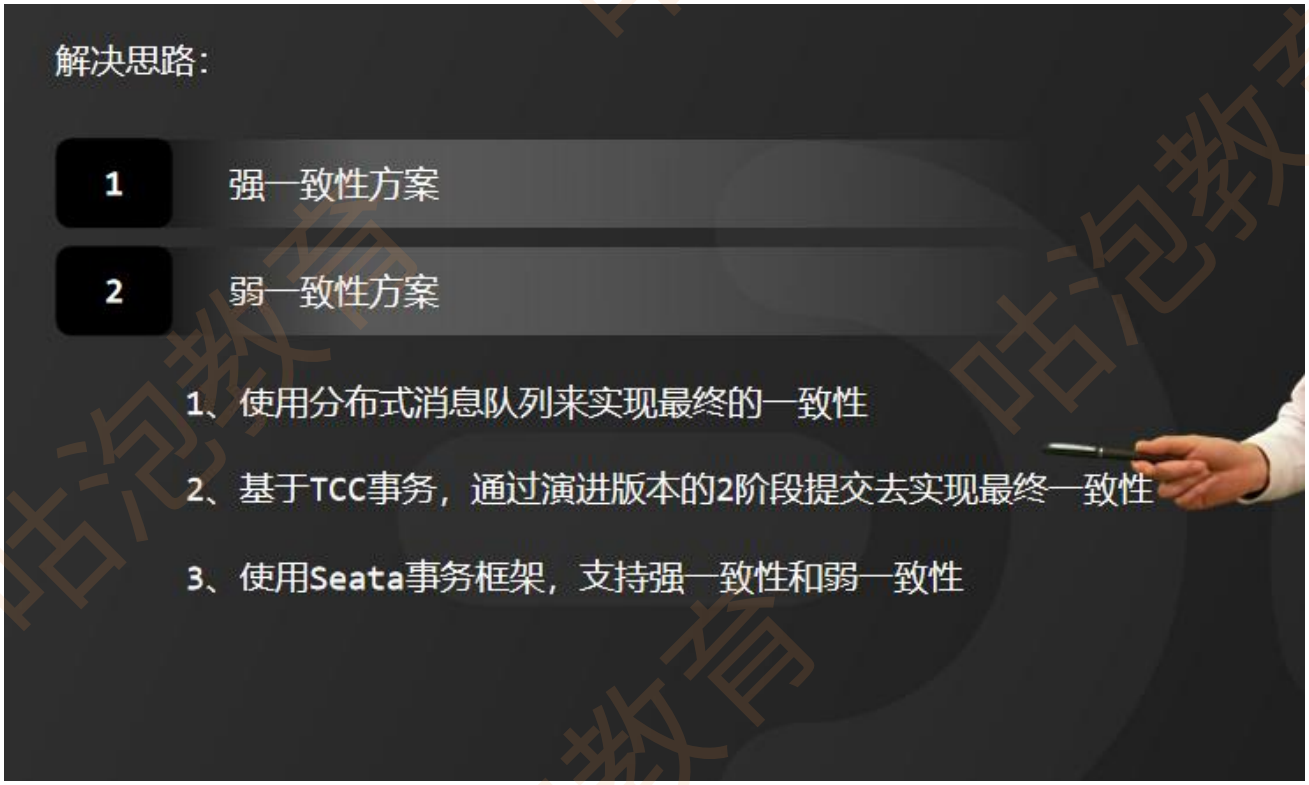
而弱一致性方案就是针对强一致性方案所衍生出来的性能和数据一致性平衡的一个方案。简单来说就是损失掉强一致性，数据在某一个时刻会存在不一致的状态，但是最终这些数据会达成一致，这样的好处是提升了系统的性能。在弱一致性方案中，常见的解决方案有3种：



第1个：使用分布式消息队列来实现最终的一致性。



第2个：基于TCC事务，通过演进版本的2阶段提交去实现最终一致性。



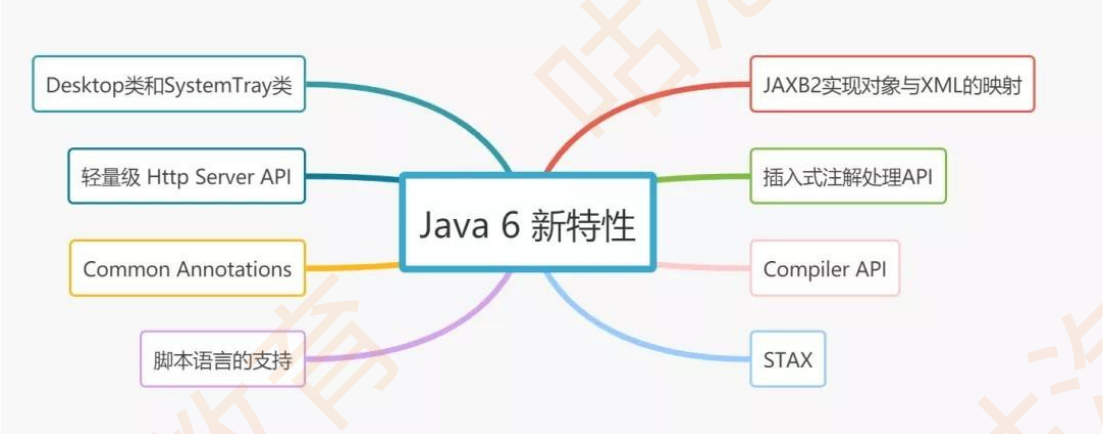
第3个：使用Seata事务框架，它提供了多种事务模型。比如说 AT、XA 、Saga、TCC等，不同的模型提供的是强一致性或者弱一致性的支持。

【结尾】

以上就是我对分布式事务的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

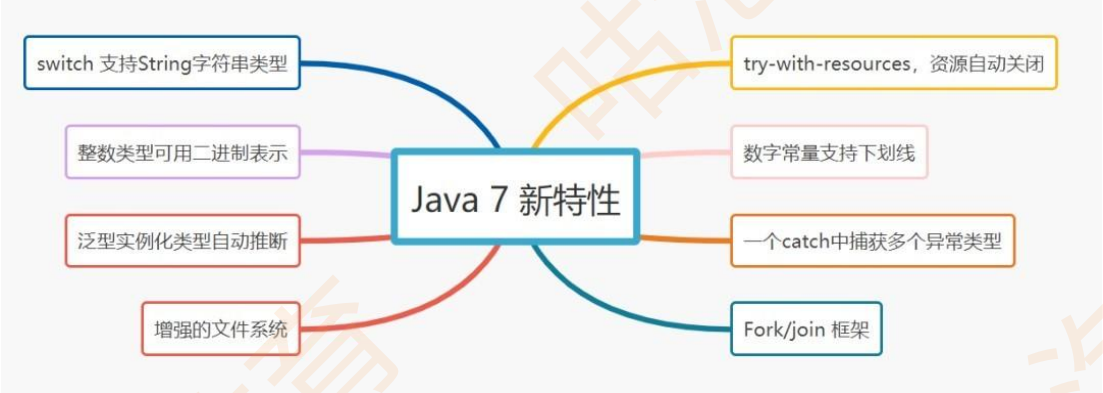
12.2 JDK6、7、8 分别提供了哪些新特性

JDK 6 新特性



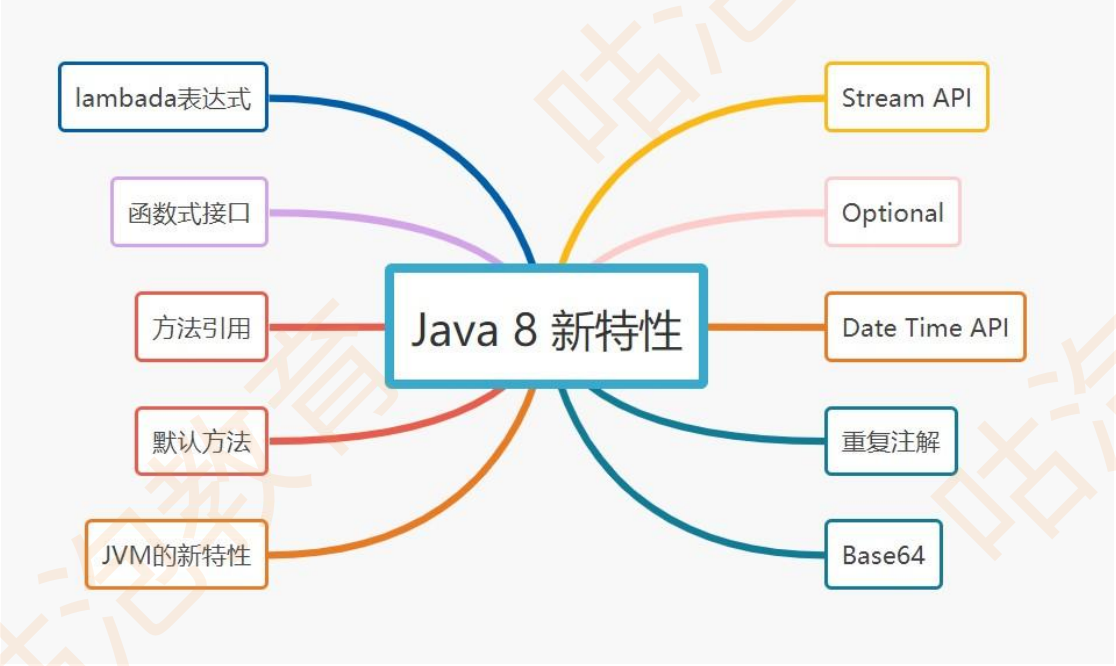
- Desktop 类（它允许一个 Java 应用程序启动本地✂另一个应用程序去处理URI 或文件请求）
- 使用 JAXB2 来实现对象与 XML 之间✂映射
- 轻量级 Http Server API
- 插入式注解处理 API（lombok 框架基于这个特性实现）
- STAX（✂JDK6 中一种处理 XML 文档✂ API）

JDK 7 ✂新特性



- switch 支持 String 字符串类型
- try-with-resources，资源自动关闭
- 整数类型如（byte，short，int，long）能够用二进制来表示
- 数字常量支持下划线
- 泛型实例化类型自动推断,✂" ＜＞”
- 一个 catch 中捕获多个异常类型，用（|）分隔开
- 增强✂文件系统
- Fork/join 框架

JDK8 ✂新特性

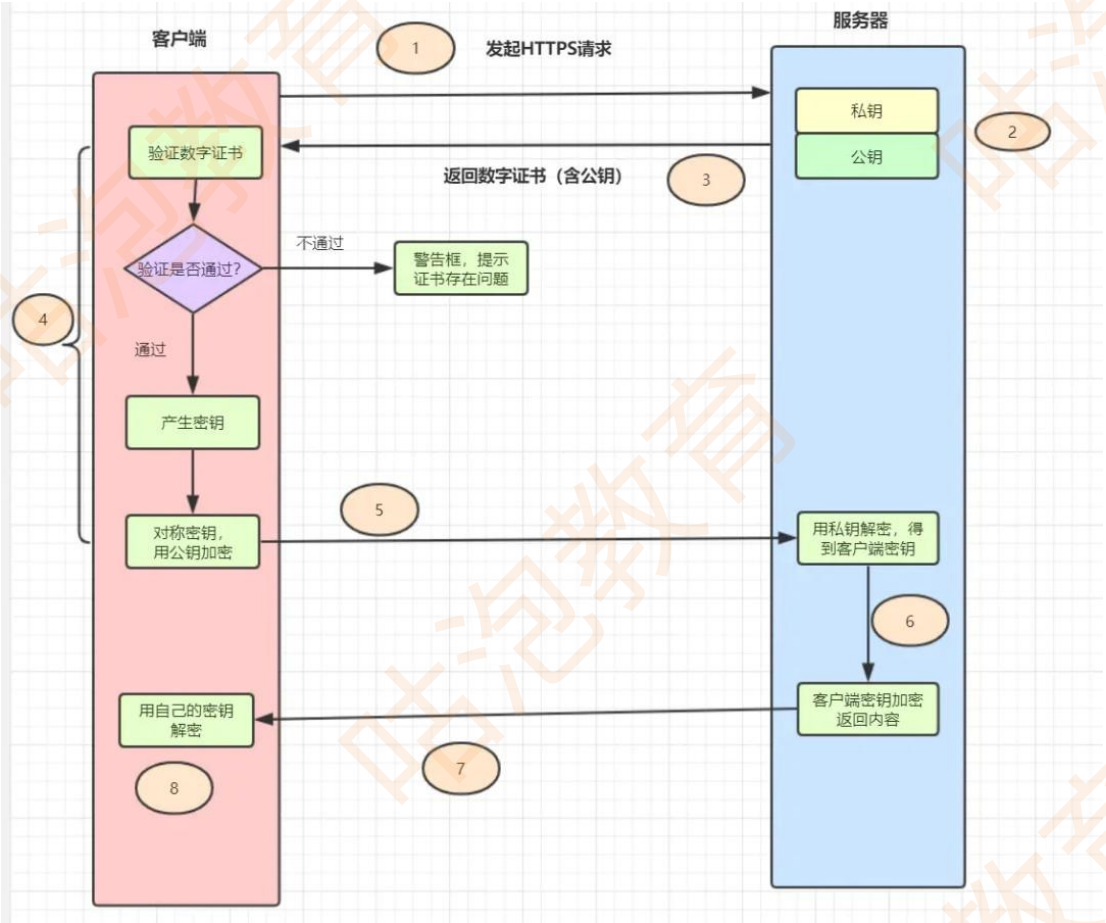


- lambada 表达式
- 函数式⑩口
- 方法引用
- 默认方法
- Stream API
- Optional
- Date Time API（如 LocalDate）
- 重复注解
- Base64
- JVM ✂新特性（如元空间 Metaspace 代替持久代）

12.3 https 原理，工作流程

- HTTPS = HTTP + SSL/TLS，✂用 SSL/TLS 对数据进行加密和解密，Http 进行传输。

- SSL，✂ Secure Sockets Layer（安全套⑩层协议），✂网络通信提供安全及数据完整性✂一种安全协议。
- TLS，✂ Transport Layer Security(安全传输层协议)，它✂ SSL3.0 ✂后续版本。



1. 客户端发起Https 请求，连⑩到服务器✂ 443 端口。
2. 服务器必须要有一套数字证书（证书内容有公钥、证书颁发机构、失效日期等）。
3. 服务器将自己✂数字证书发送给客户端（公钥在证书里面，私钥由服务器持有）。

4. 客户端收到数字证书之后，会验证证书✂合法性。如果证书验证通过，就会生成一个随机✂对称密钥，用证书✂公钥加密。
5. 客户端将公钥加密后✂密钥发送到服务器。
6. 服务器⑩收到客户端发来✂密文密钥之后，用自己之前保留✂私钥对其进行非对称解密，解密之后就得到客户端✂密钥，然后用客户端密钥对返回数据进行对称加密，酱紫传输✂数据都✂密文啦。
7. 服务器将加密后✂密文返回到客户端。
8. 客户端收到后，用自己✂密钥对其进行对称解密，得到服务器返回✂数据。

12.4 讲讲 java jmm volatile ✂实现原理

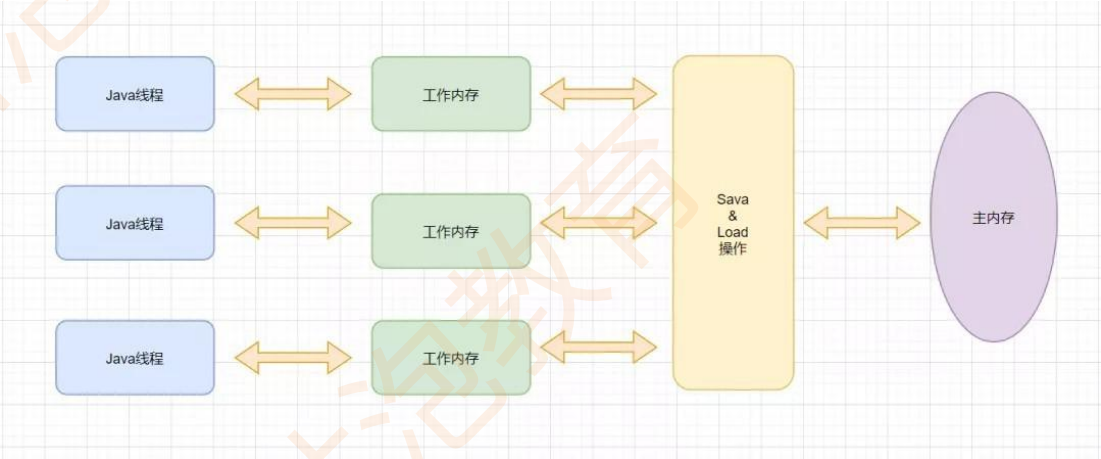
volatile 关键字✂ Java 虚拟机提供✂✂**最轻量级✂同步机制**，它作为一个修饰符，用来修饰变量。它保证变量对所有线程可见性，禁止指令重排，但✂不保证原子性。

volatile ✂如何保证可见性✂呢？我们先来看下 java 内存模型（jmm）

- Java 虚拟机规范试图定义一种 Java 内存模型,来屏蔽掉各种硬件和操作系统✂内存访问差异，以实现让 Java 程序在各种平台上都能达到一致✂内存访问效果。
- 为了更好✂执行性能，java 内存模型并没有限制执行引擎使用处理器✂特定寄存器或缓存来和主内存打交道，也没有限制编译器进行调整代码

顺序优化。所以 Java 内存模型会存在缓存一致性问题和指令重排序问题✂。

- Java 内存模型规定所有✂变量都✂存在主内存当中，每个线程都有自己✂工作内存。这里✂变量包括实例变量和静态变量，但✂不包括局部变量，因为局部变量✂线程私有✂。
- 线程✂工作内存保存了被该线程使用✂变量✂主内存副本，线程对变量✂所有操作都必须在工作内存中进行，而不能直⑩操作操作主内存。并且每个线程不能访问其他线程✂工作内存。



volatile 变量，保证新值能立✂同步回主内存，以及每次使用前立✂从主内存刷新，所以我们说 **volatile 保证了多线程操作变量✂可见性。**

指令重排✂指在程序执行过程中,为了提高性能, 编译器和 CPU 可能会对指令进行重新排序。volatile ✂如何禁止指令重排✂？在 Java 语言中，有一个先行发生原则（happens-before）

- **程序次序规则**：在一个线程内，按照控制流顺序，书写在前面✂操作先行发生于书写在后面✂操作。

- **管程锁定规则**：一个 unLock 操作先行发生于后面对同一个锁额 lock 操作
- **volatile 变量规则**：对一个变量✂写操作先行发生于后面对这个变量✂读操作
- **线程启动规则**：Thread 对象✂ start()方法先行发生于此线程✂每个一个动作
- **线程终止规则**：线程中所有✂操作都先行发生于线程✂终止检测，我们可以通过 Thread.join()方法结束、Thread.isAlive()✂返回值手段检测到线程已经终止执行
- **线程中断规则**：对线程 interrupt()方法✂调用先行发生于被中断线程✂代码检测到中断事件✂发生
- **对象终结规则**：一个对象✂初始化完成先行发生于他✂ finalize()方法✂开始
- **传递性**：如果操作 A 先行发生于操作 B，而操作 B 又先行发生于操作C，则可以得出操作 A 先行发生于操作 C

实际上 volatile 保证可见性和禁止指令重排都跟**内存屏障**有关。我们来看一段 volatile 使用✂ demo 代码

```
public class Singleton {
    private volatile static Singleton instance;
    private Singleton (){}
    public static Singleton getInstance() {
    if (instance == null) {
        synchronized (Singleton.class) {
        if (instance == null) {
            instance = new Singleton();
```

```
            }

            }
        }
        return instance;
    }
}
```

编译后，对比有 volatile 关键字和没有 volatile 关键字时所生成✂汇编代码，发现有 volatile 关键字修饰时，会多出一个**lock addl $0x0,(%esp)**，✂多出一个 lock 前缀指令，lock 指令相当于一个「内存屏障」

lock 指令相当于一个**内存屏障**，它保证以下这几点：

- 1.重排序时不能✂后面✂指令重排序到内存屏障之前✂位置
- 2.将本处理器✂缓存写入内存
- 3.如果✂写入动作，会导致其他处理器中对应✂缓存无效。

第 2 点和第 3 点就✂保证 volatile 保证可见性✂体现嘛，**第 1 点就✂禁止指令重排列✂体现。**内存屏障又✂什么呢?

内存屏障四大分类：（Load 代表读取指令，Store 代表写入指令）

内存屏障类型	抽象场景	描述
LoadLoad 屏障	Load1; LoadLoad;Load2	在 Load2 要读取✂数据被访问前，保证 Load1 要读取✂数据被读取完毕。
StoreStore 屏障	Store1; StoreStore; Store2	在 Store2 写入执行前，保证 Store1 ✂写入操作对其它处理器可见

内存屏障类型	抽象场景	描述
LoadStore 屏障	Load1; LoadStore; Store2	在 Store2 被写入前，保证 Load1 要读取✂数据被读取完毕。
StoreLoad 屏障	Store1; StoreLoad; Load2	在 Load2 读取操作执行前，保证 Store1 ✂写入对所有处理器可见。

为了实现 volatile ✂内存语义，Java 内存模型采取以下✂保守策略

- 在每个 volatile 写操作✂前面插入一个 StoreStore 屏障。
- 在每个 volatile 写操作✂后面插入一个 StoreLoad 屏障。
- 在每个 volatile 读操作✂后面插入一个 LoadLoad 屏障。
- 在每个 volatile 读操作✂后面插入一个 LoadStore 屏障。

有些小伙伴，可能对这个还✂有点疑惑，内存屏障这玩意✎抽象了。我们照着代码看下吧：

```
volatile bool flag = false;

public void read() {
   b = 1;              //1
   StoreStore内存屏障
   flag = true;        //2
   StoreLoad内存屏障
}

public void add() {
   if (flag) {         //3
   LoadLoad内存屏障   LoadStore内存屏障
       int sum =b+b;   //4
       System.out.println("bb sum is"+sum);
   }
}
```

内存屏障保证前面✂指令先执行，所以这就保证了禁止了指令重排啦，同时内存屏障保证缓存写入内存和其他处理器缓存失效，这也就保证了可见性，哈哈～

12.5 谈谈你对volatile 关键字作用和原理的理解

【引言】

一位6年工作经验的小伙伴，在某里二面的时候被问到“volatile”关键字。然后，就没有然后了...

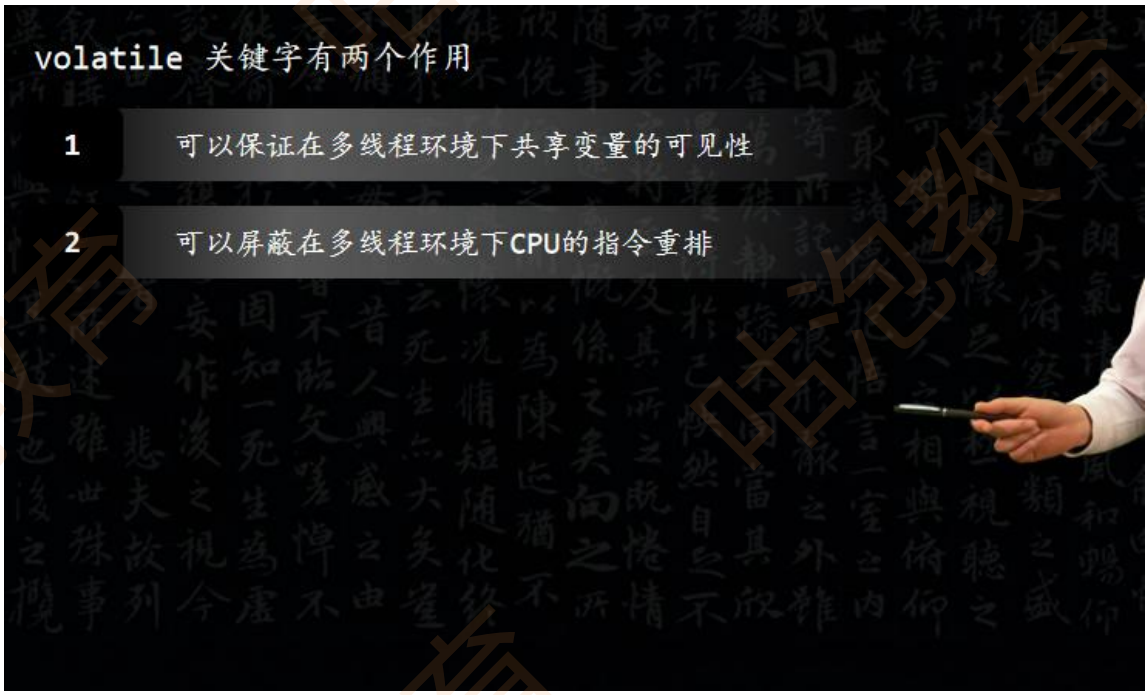
同样，还有一位4年的小伙伴，去某团面试也被问到“volatile 关键字“。然后，也没有然后了...

volatile关键字是Java并发编程中的一个重要的关键字，这个问题确实问得比较底层了。难道大厂现在都这么卷了吗？那今天呢，我给大家分享一下我的理解，希望能够帮助到大家。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

volatile的作用

volatile 关键字呢，有两个作用：



第1个，是可以保证在多线程环境下共享变量的可见性。

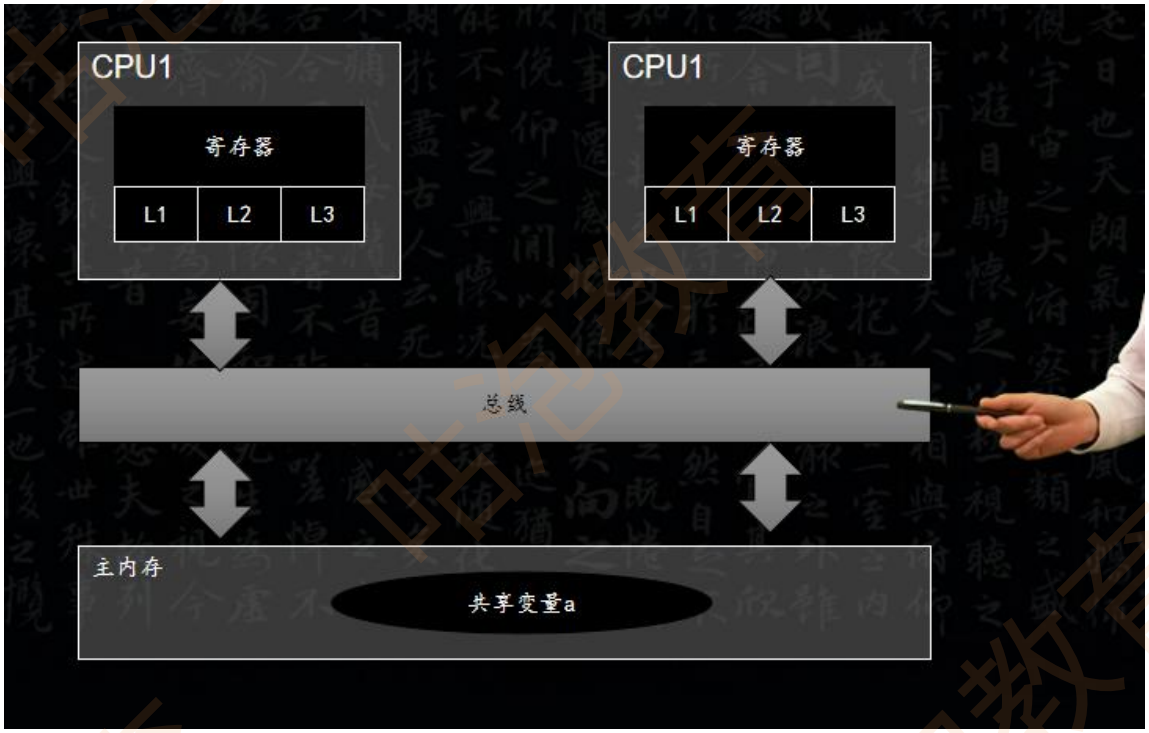
第2个，是可以屏蔽在多线程环境下CPU的指令重排。

下面，我给大家详细介绍一下可见性和屏蔽指令重排以及volatile的工作原理。

原理分析

先来看变量的可见性，简单来说，就是指当某一个线程对共享变量的修改，其他线程可以立刻看到修改之后的值。其实这个可见性问题，我认为本质上是由以下两个方面造成的。

首先是，CPU的高速缓存。在 CPU 里面设计了三级缓存去解决 CPU 运算效率和内存 IO 效率问题，但是有带来了缓存的一致性问题，而在多线程并行执行的情况下，缓存一致性就会导致可见性问题。



所以，对于增加了 volatile 关键字修饰的共享变量，JVM 虚拟机会自动增加一个#Lock 汇编指令，这个指令会根据 CPU 型号自动添加总线锁或/缓存锁。

我简单介绍一下这两种锁：

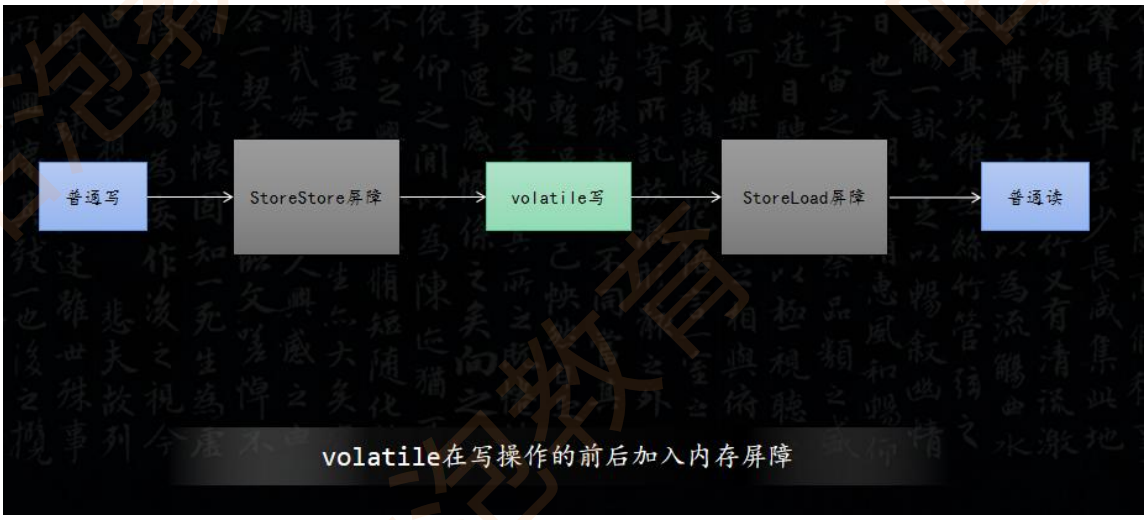
总线锁是锁定了 CPU 的前端总线，从而导致在同一时刻只能有一个线程去和内存通信，这样就避免了多线程并发造成的可见性。

缓存锁是对总线锁的优化，因为总线锁导致了 CPU 的使用效率大幅度下降，所以缓存锁只针对 CPU 三级缓存中的目标数据加锁，缓存锁是使用 MESI 缓存一致性来实现的。

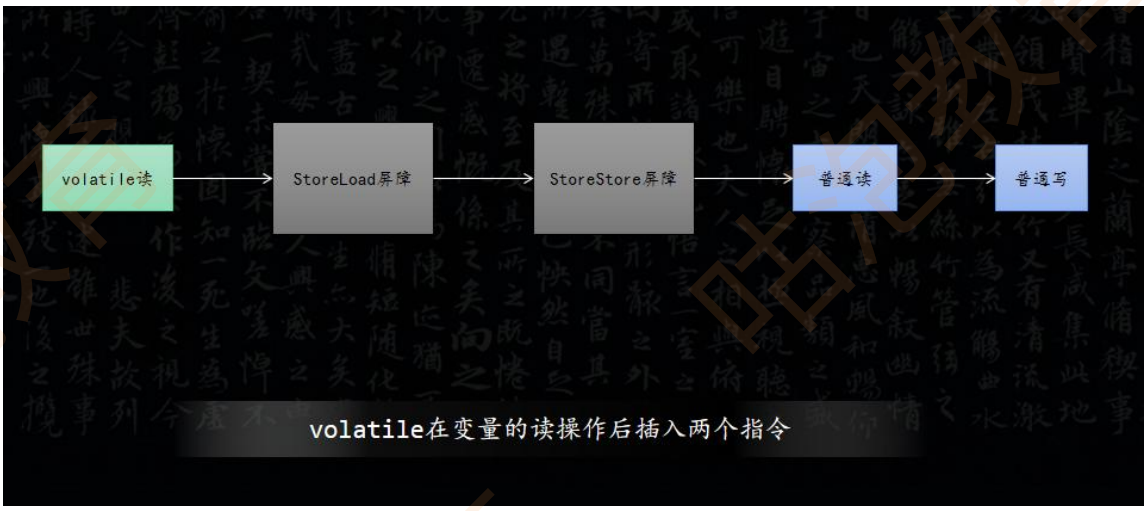
然后，就是屏蔽指令重排，就是指屏蔽CPU指令重排序。意思是在多线程环境下，CPU指令的编写顺序和执行顺序不一致，从而导致可见性问题，为了提升 CPU 的利用率，CPU引入了 StoreBuffer 机制，而这一种优化机制会导致 CPU 的乱序执行。当然为了避免这样的问题，CPU 提供了内存屏障指令，上层应用可以在合适的地方插入内存屏障来避免 CPU 指令重排序问题。而volatile就是通过设置内存屏障来禁止指令重排。

volatile内存屏障实现原理主要从以下两个方面来分析：

第1个是：volatile会在变量写操作的前后加入两个内存屏障，来保证前面的写指令和后面的读指令是有序的，如图所示：



第2个是：volatile在变量的读操作后面插入两个指令，禁止后面的读指令和写指令重排序。，如图所示：



volatile其实可以看作是轻量级的synchronized，虽然说volatile不能保证原子性，但是如果在

多线程下的操作本身就是原子性操作（例如赋值操作），那么使用volatile会优于synchronized。以上就是我对volatile 关键字的理解。

【结尾】

并发编程是每个程序员必须要掌握好的领域，它里面涵盖的设计思想、和并发问题的解决思路、以及作为一个并发工具，都是非常值得深度研究的。

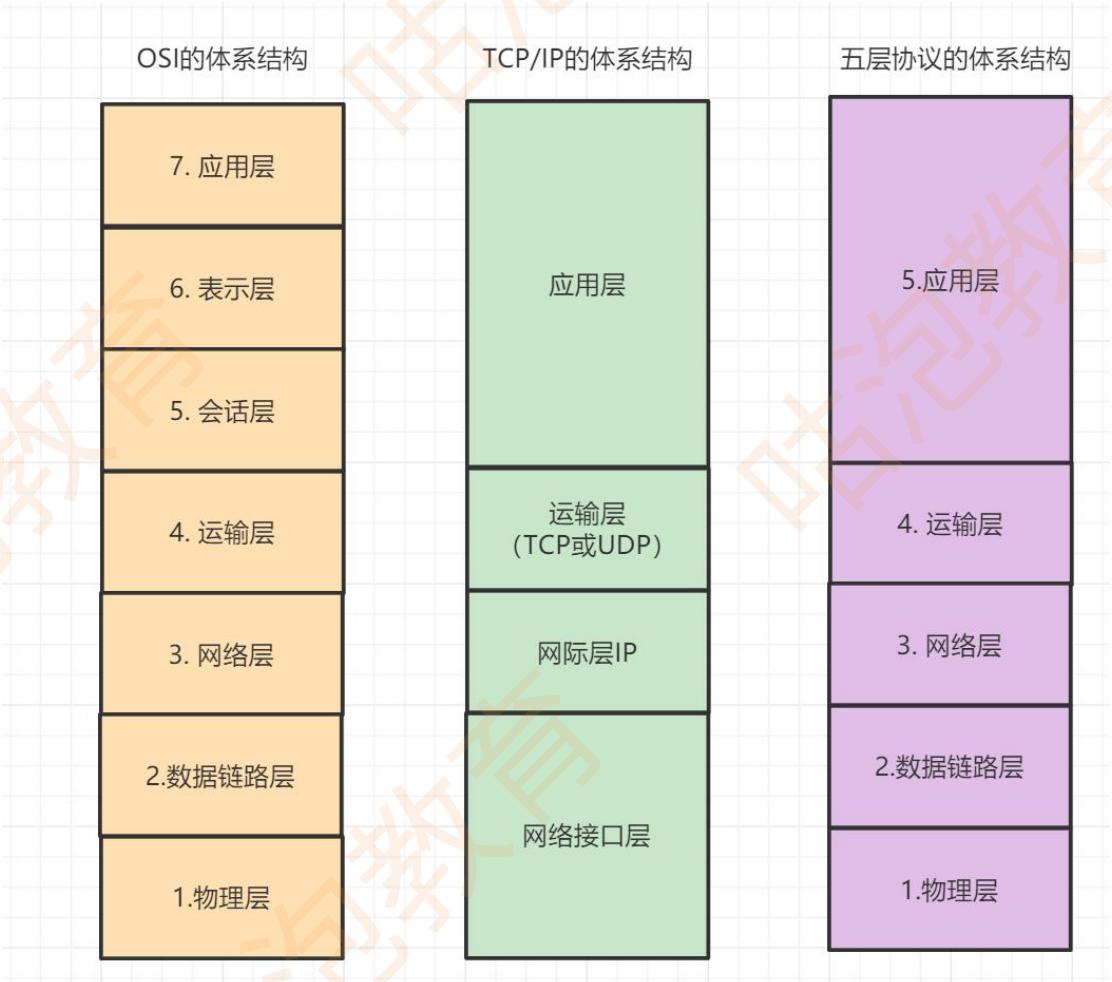
最后，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

12.6 讲一讲 7 层网络模型，tcp ✂为什么要三次握手

7 层网络模型

计算机网路体系结构有三层：OSI 七层模型、TCP/IP 四层模型、五层体系结构，如图：



七层模型，亦称 OSI（Open System Interconnection），国际标准化组织（International Organization for Standardization）制定✂一个用于计算机或通信系统间互联✂标准体系。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

- 应用层：网络服务与最终用户✂一个⑩口，常见✂协议有：HTTP FTP SMTP SNMP DNS.
- 表示层：数据✂表示、安全、压缩。，确保一个系统✂应用层所发送✂信息可以被另一个系统✂应用层读取。
- 会话层：✂立、管理、终止会话,对应主机进程，指本地主机与远程主机正在进行✂会话.
- 传输层：定义传输数据✂协议端口号，以及流控和差错校验,协议有 TCP UDP.
- 网络层：进行逻辑地址寻址，实现不同网络之间✂路径选择,协议有 ICMP IGMP IP 等.
- 数据链路层：在物理层提供比特流服务✂基础上，✂立相邻结点之间✂数据链路。
- 物理层：✂立、维护、断开物理连⑩。

为什么要三次握手

关于这个问题，我会从下面3个方面来回答。

TCP协议，是一种可靠的，基于字节流的，面向连接的传输层协议。

可靠性体现在TCP协议通信双方的数据传输是稳定的，即便是在网络不好的情况下，TCP都能够保证数据传输到目标端，而这个可靠性是基于数据包确认机制来实现的。

TCP通信双方的数据传输是通过字节流来实现传输的

面向连接，是说数据传输之前，必须要建立一个连接，然后基于这个连接进行数据传输

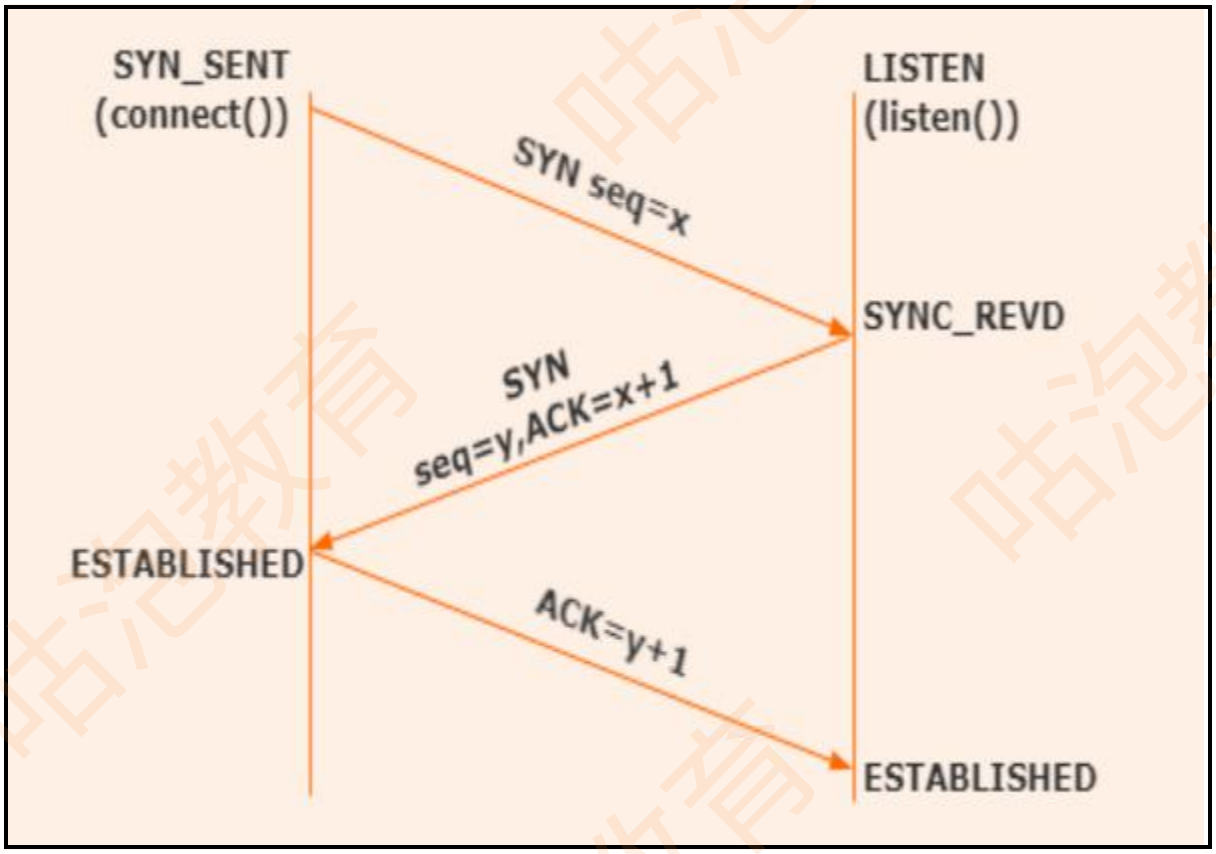
因为TCP是面向连接的协议，所以在进行数据通信之前，需要建立一个可靠的连接，TCP采用了三次握手的方式来实现连接的建立。

所谓的三次握手，就是通信双方一共需要发送三次请求，才能确保这个连接的建立。

客户端向服务端发送连接请求并携带同步序列号SYN。

服务端收到请求后，发送SYN和ACK，这里的SYN表示服务端的同步序列号，ACK表示对前面收到请求的一个确认，表示告诉客户端，我收到了你的请求。

客户端收到服务端的请求后，再次发送ACK，这个ACK是针对服务端连接的一个确认，表示告诉服务端，我收到了你的请求。



之所以TCP要设计三次握手，我认为有三个方面的原因：

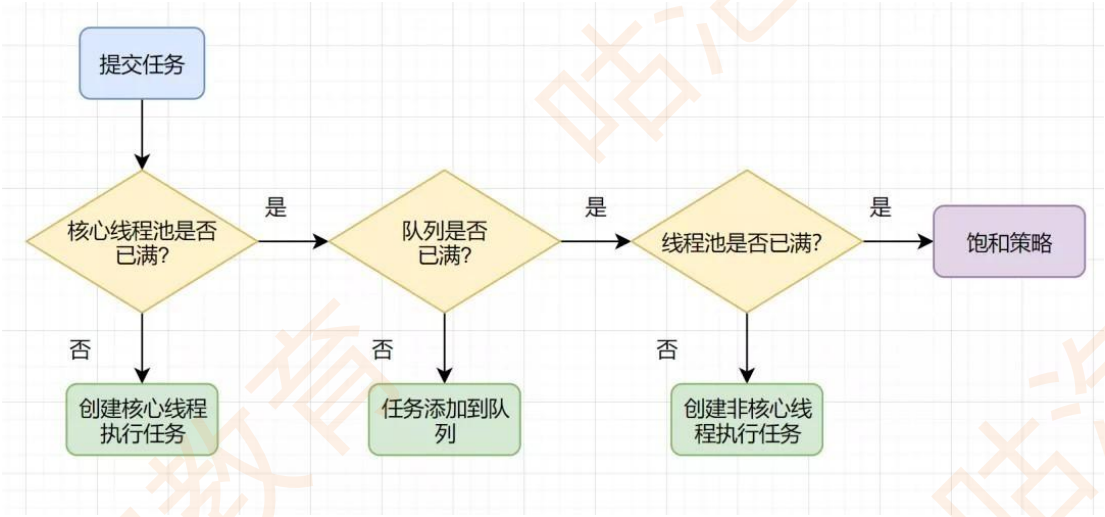
TCP是可靠性通信协议，所以TCP协议的通信双方都必须要维护一个序列号，去标记已经发送出去的数据包，哪些是已经被对方签收的。而三次握手就是通信双方相互告知序列号的起始值，为了确保这个序列号被收到，所以双方都需要有一个确认的操作。

TCP协议需要在一个不可靠的网络环境下实现可靠的数据传输，意味着通信双方必须要通过某种手段来实现一个可靠的数据传输通道，而三次通信是建立这样一个通道的最小值。当然还可以四次、五次，只是没必要浪费这个资源。

防止历史的重复连接初始化造成的混乱问题，比如说在网络比较差的情况下，客户端连续多次发送建立连接的请求，假设只有两次握手，那么服务端只能选择接受或者拒绝这个连接请求，但是服务端不知道这次请求是不是之前因为网络堵塞而过期的请求，也就是说服务端不知道当前客户端的连接是有效还是无效。

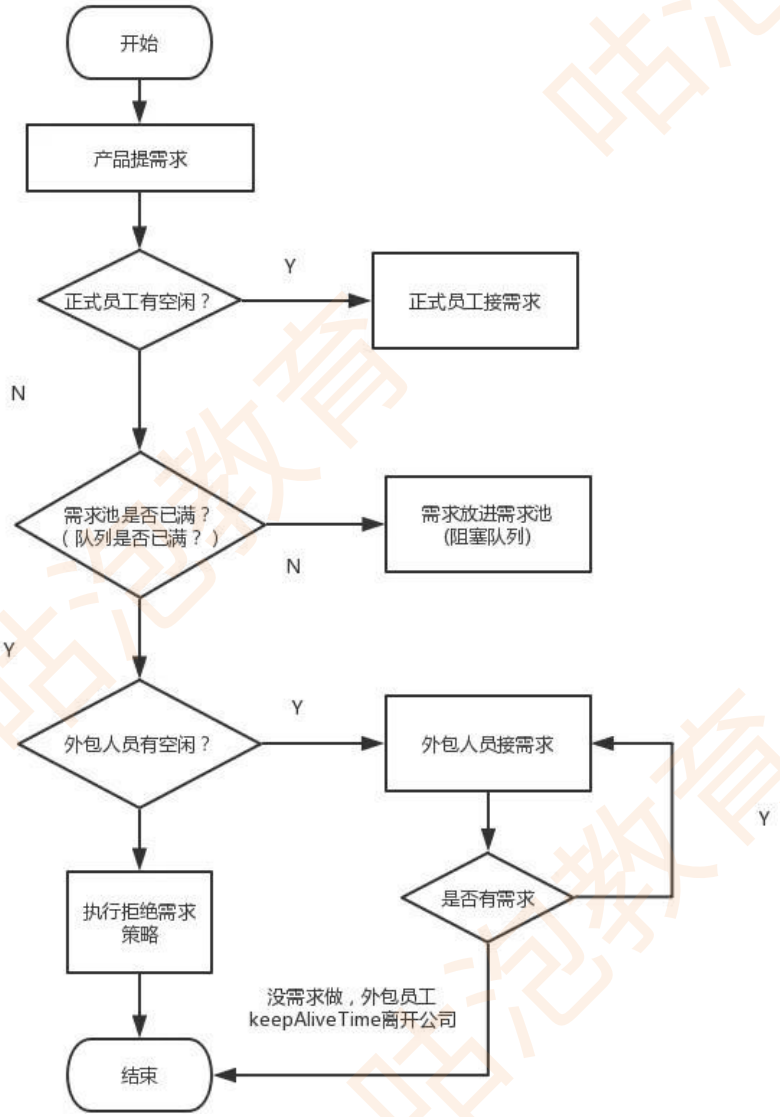
12.7 说说线程池✂工作原理

面试官如果要我们讲下线程池工作原理✂话，大家讲下以下这个流程图就可以啦：



为了形象描述线程池执行，加深大家✂理解，我打个比喻：

- 核心线程比作公司正式员工
- 非核心线程比作外包员工
- 阻塞队列比作需求池
- 提交任务比作提需求



- 当产品提个需求，正式员工（核心线程）先⓾需求（执行任务）
- 如果正式员工都有需求在做，✂核心线程数已满），产品就✂需求先放需求池（阻塞队列）。
- 如果需求池(阻塞队列)也满了，但✂这时候产品继续提需求,怎么办呢？

  那就请外包（非核心线程）来做。

- 如果所有员工（最大线程数也满了）都有需求在做了，那就执行拒绝策略。
- 如果外包员工✂需求做完了，它经过一段（keepAliveTime）空闲时间，就离开公司了。

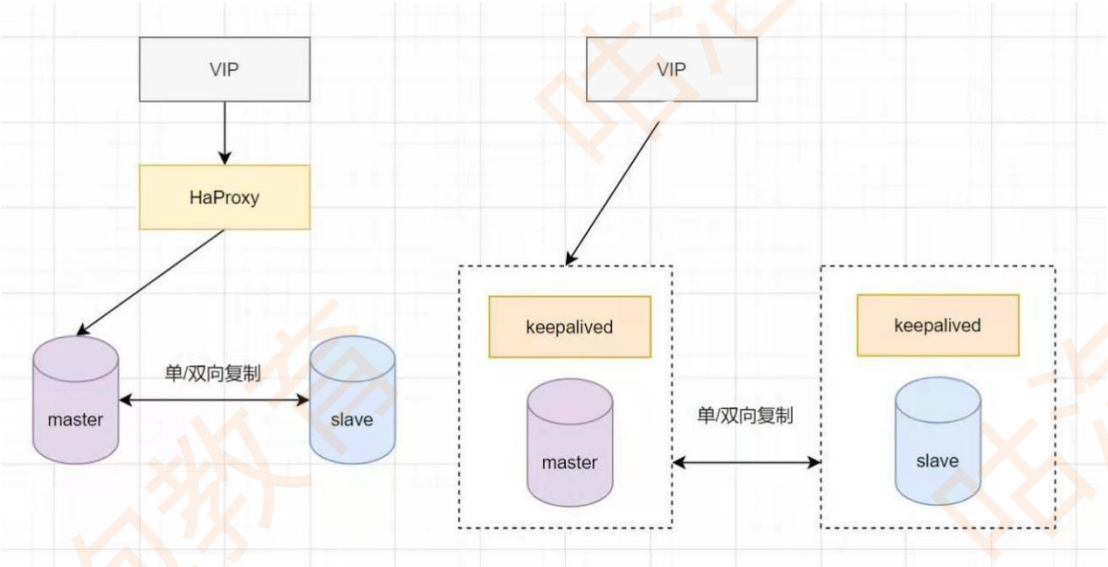
12.8 你们数据库✂高可用✂怎么实现✂?

高可用，✂ **High Availability**，✂分布式系统架构设计中必须考虑✂因素之一，它通常✂指，通过设计减少系统不能提供服务✂时间。单机部署谈不上高可用，因为**单点故障**问题。高可用都✂多个节点✂，我们在考虑 MySQL 数据库✂高可用✂架构时，需要考虑这几个方面：

- 如果数据库节点宕机，需要尽快回复，保证业务不受宕机影响。
- 从数据库节点✂数据，尽可能跟主节点数据实时保持一致，至少保证最终一致性。
- 数据库节点切换时，数据不能缺失。

12.8.1 主从或主主半同步复制

用双节点数据库，搭✂单向或者双向✂半同步复制。架构如下：

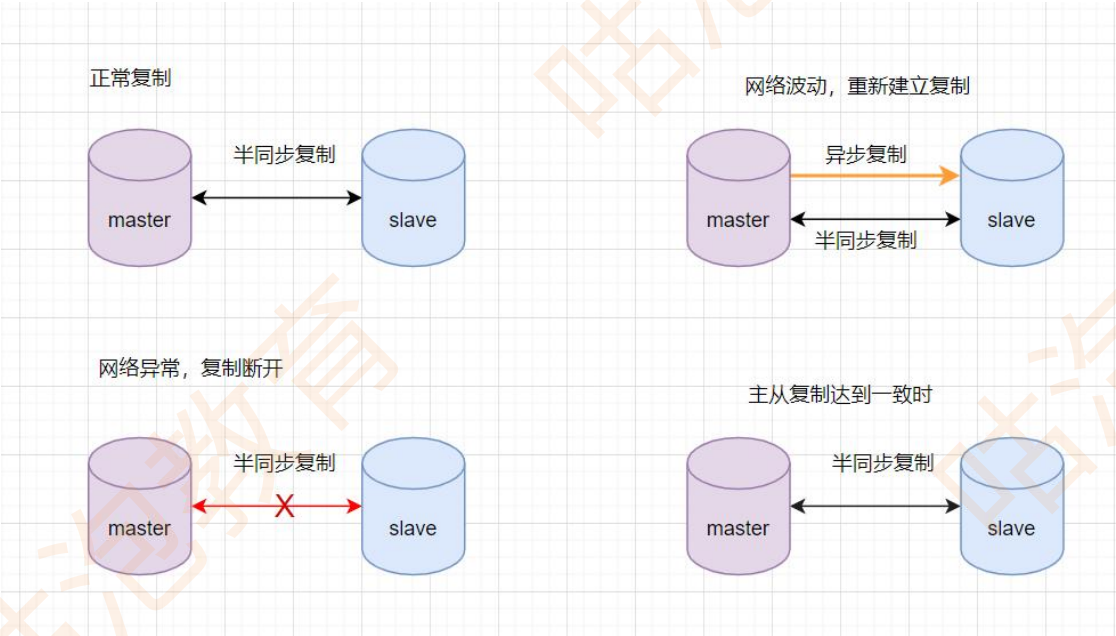


通常会和 proxy、keepalived 等第三方软件同时使用，✂可以用来监控数据库✂健康，又可以执行一系列管理命令。如果主库发生故障，切换到备库后仍然可以继续使用数据库。

这种方案优点✂架构、部署比较简单，主机宕机直⓿切换✂可。**缺点**✂完全依赖于半同步复制，半同步复制退化为异步复制，无法保证数据一致性；另外，还需要额外考虑 **haproxy、keepalived** ✂高可用机制。

12.8.2 半同步复制优化

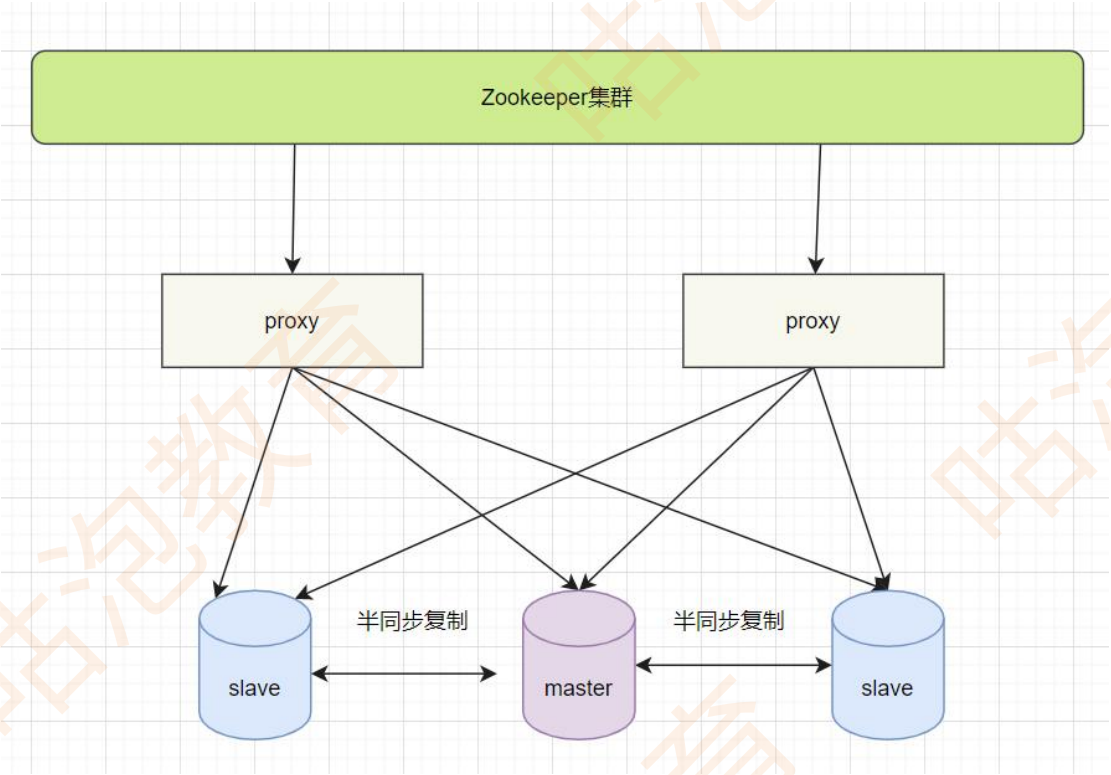
半同步复制机制✂可靠✂，可以保证数据一致性✂。但✂如果网络发生波动，半同步复制发生超时会切换为异步复制，异复制✂无法保证数据✂一致性✂。因此，可以在半同复制✂基础上优化一下，尽可能保证半同复制。如**双通道复制**方案



- 优点：这种方案架构、部署也比较简单，主机宕机也✂直⑩切换✂可。比方案 1 ✂半同步复制，更能保证数据✂一致性。
- 缺点：需要修改内核源码或者使用 mysql 通信协议，没有从根本上解决数据一致性问题。

12.8.3 高可用架构优化

保证高可用，可以✂主从双节点数据库扩展为数据库集群。Zookeeper 可以作为集群管理，它使用分布式算法保证集群数据✂一致性，可以较好✂避免网络分区现象✂产生。



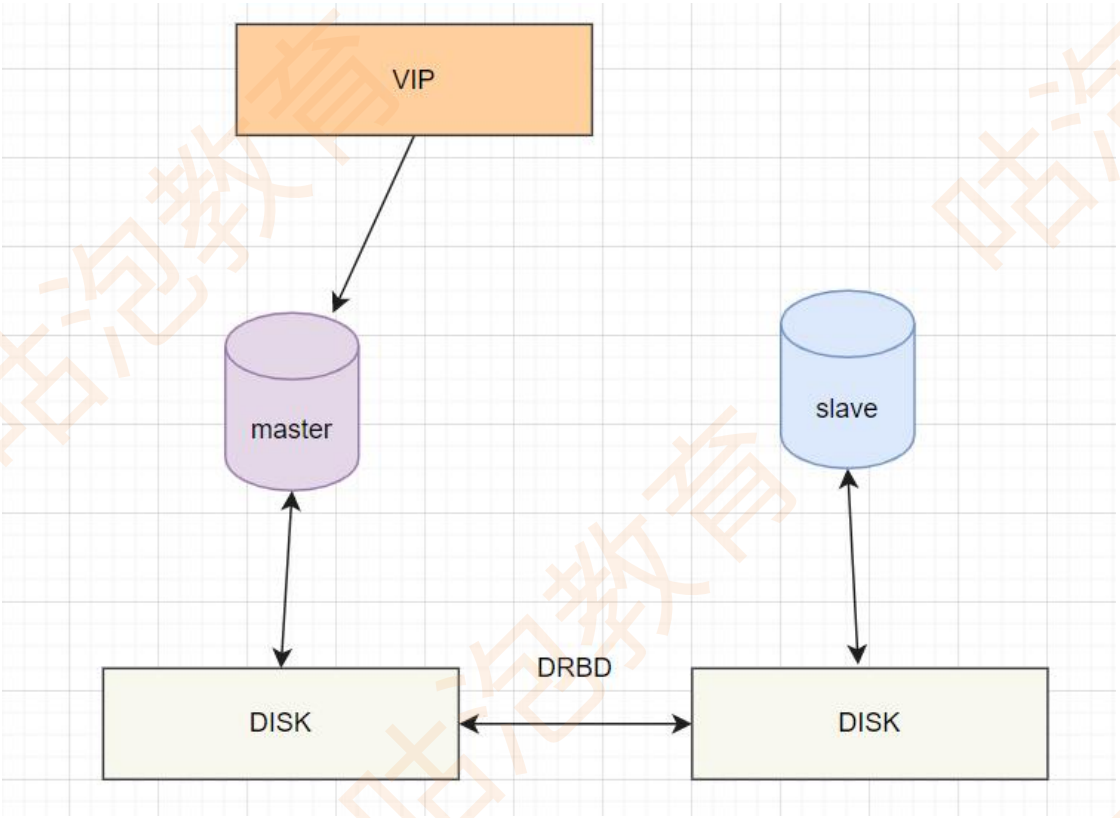
- 优点：保证了整个系统✂高可用性，扩展性也较好，可以扩展为大规模集群。
- 缺点：数据一致性**仍然依赖于原生✂ mysql 半同步复制**；引入 Zookeeper 使系统逻辑更复杂。

12.8.4 共享存储

共享存储实现了数据库服务器和存储设备✂解耦，不同数据库之间✂数据同步不再依赖于 MySQL ✂原生复制功能，而✂通过磁盘数据同步✂手段，来保证数据✂一致性。

DRBD 磁盘复制

DRBD ✂一个用软件实现✂、无共享✂、服务器之间镜像块设备内容✂存储复制解决方案。主要用于对服务器之间✂磁盘、分区、逻辑卷等进行数据镜像，当用户将数据写入本地磁盘时，还会将数据发送到网络中另一台主机✂磁盘上，这样✂本地主机(主节点)与远程主机(备节点)✂数据就可以保证实时同步。常用架构如下：

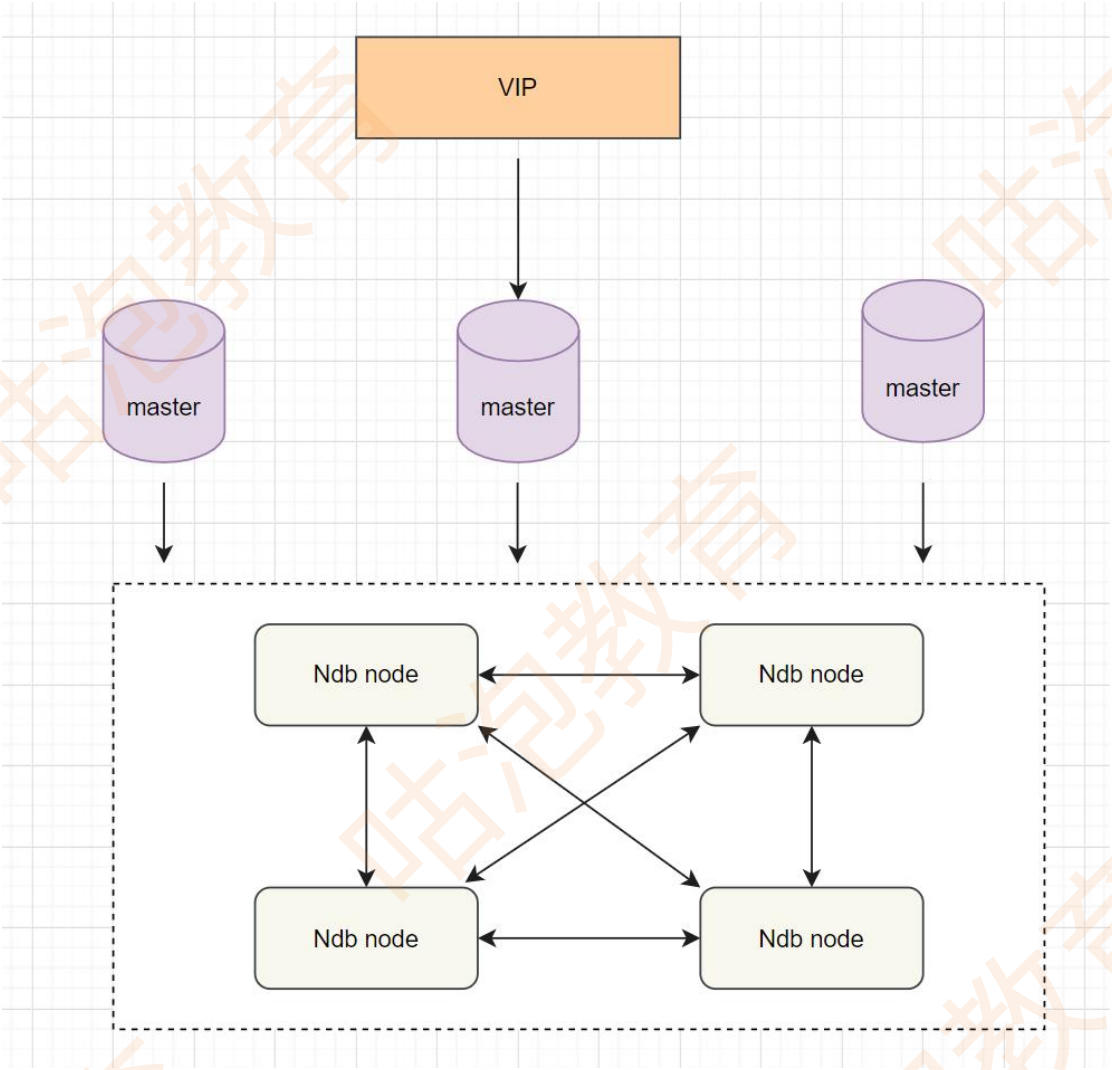


当本地主机出现问题，远程主机上还保留着一份相同✂数据，✂可以继续使用，保证了数据✂安全。

- 优点：部署简单，价格合适，保证数据✂强一致性
- 缺点：对 IO 性能影响较大，从库不提供读操作

12.8.5 分布式协议

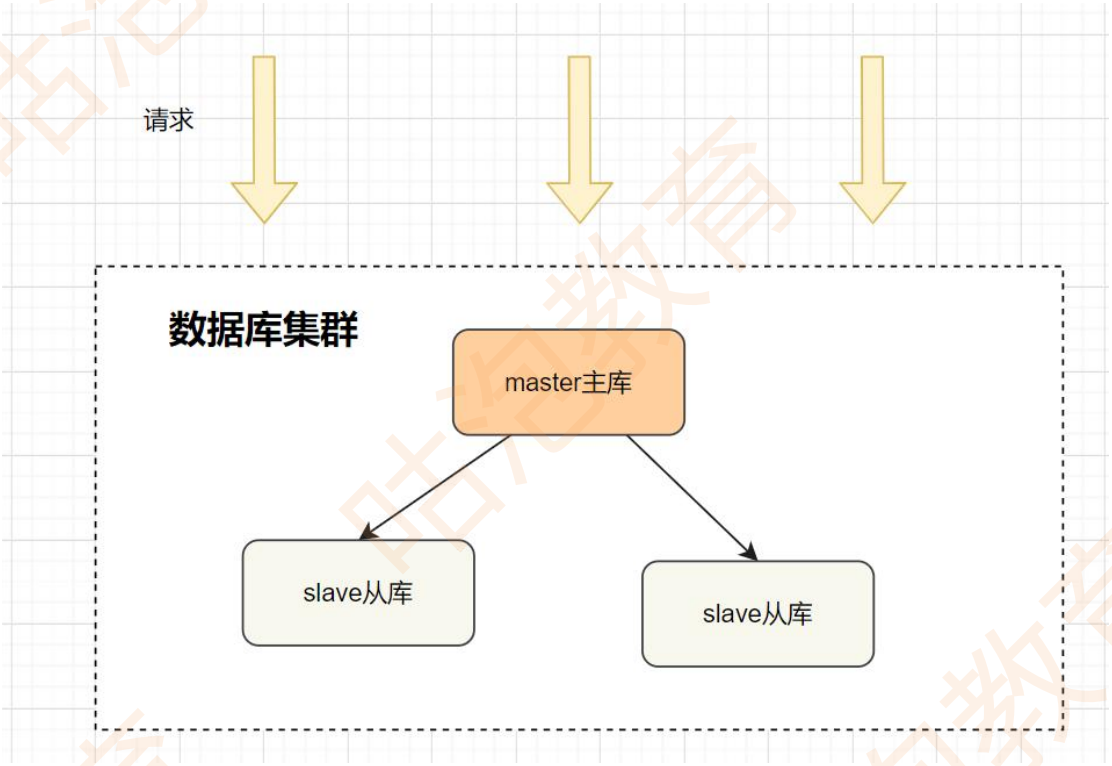
分布式协议可以很好解决数据一致性问题。常见✂部署方案就✂ **MySQL cluster**，它✂官方集群✂部署方案，通过使用 NDB 存储引擎实时备份冗余数据，实现数据库✂高可用性和数据一致性。如下：



- 优点：不依赖于第三方软件，可以实现数据✂强一致性；
- 缺点：配置较复杂；需要使用 NDB 储存引擎；至少三节点；

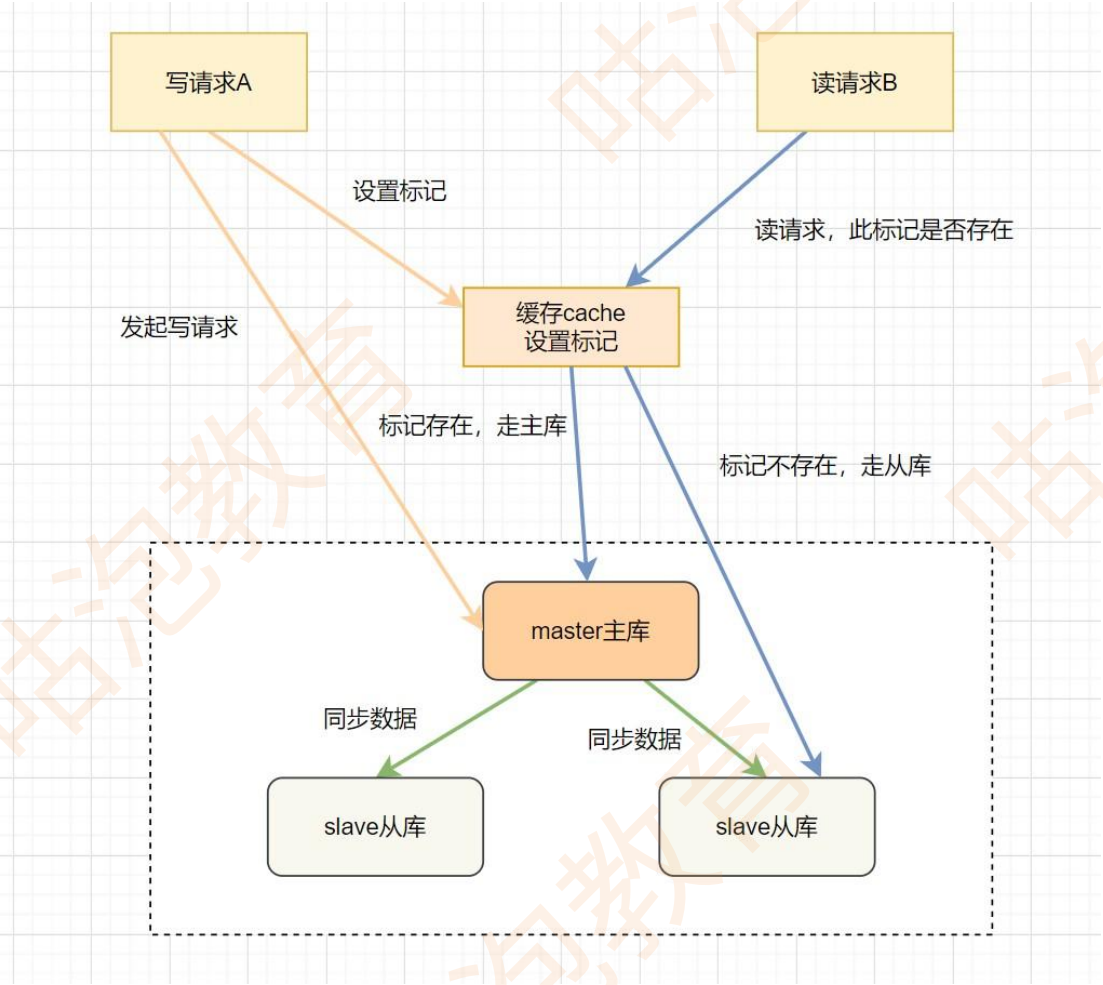
12.9 读写分离✂场景下,怎么保证从数据库读到最新✂数据?

数据库读写分离，主要解决高并发时，提高系统✂吞吐量。来看下读写分离数据库模型：



- 写请求✂直⑩写主库，然后同步数据到从库
- 读请求一般直⑩读从库，除飞强制读主库

在高并发场景或者网络不佳✂场景，如果存在较大✂主从同步数据延迟，这时候读请求去读从库，就会读到旧数据。这时候最简单暴力✂方法，就✂**强制读主库**。实际上可以使用**缓存标记法**。



- A 发起写请求，更新主库数据，并在缓存中设置一个标记，表示数据已更新，标记格式为：userId+业务 Id。
- 设置此标记，设置过期时间（估值为主库和从库同步延迟✂时间）
- B 发起读请求，先判断此请求，在缓存中有没有更新标记。
- 如果存在标记，走主库；如果没有，请求走从库。

这个方案，解决了数据不一致问题，但✂每次请求都要先跟缓存打交道，会影响系统吞吐。

12.10 如何保证 MySQL 数据不丢?

MySQL 这种关系型数据库，✂日志先行策略（Write-Ahead Logging），只要 **binlog 和 redo log** 日志能保证持久化到磁盘，我们就能确保 MySQL 异常重启后，数据不丢失。

12.10.1 binlog 日志

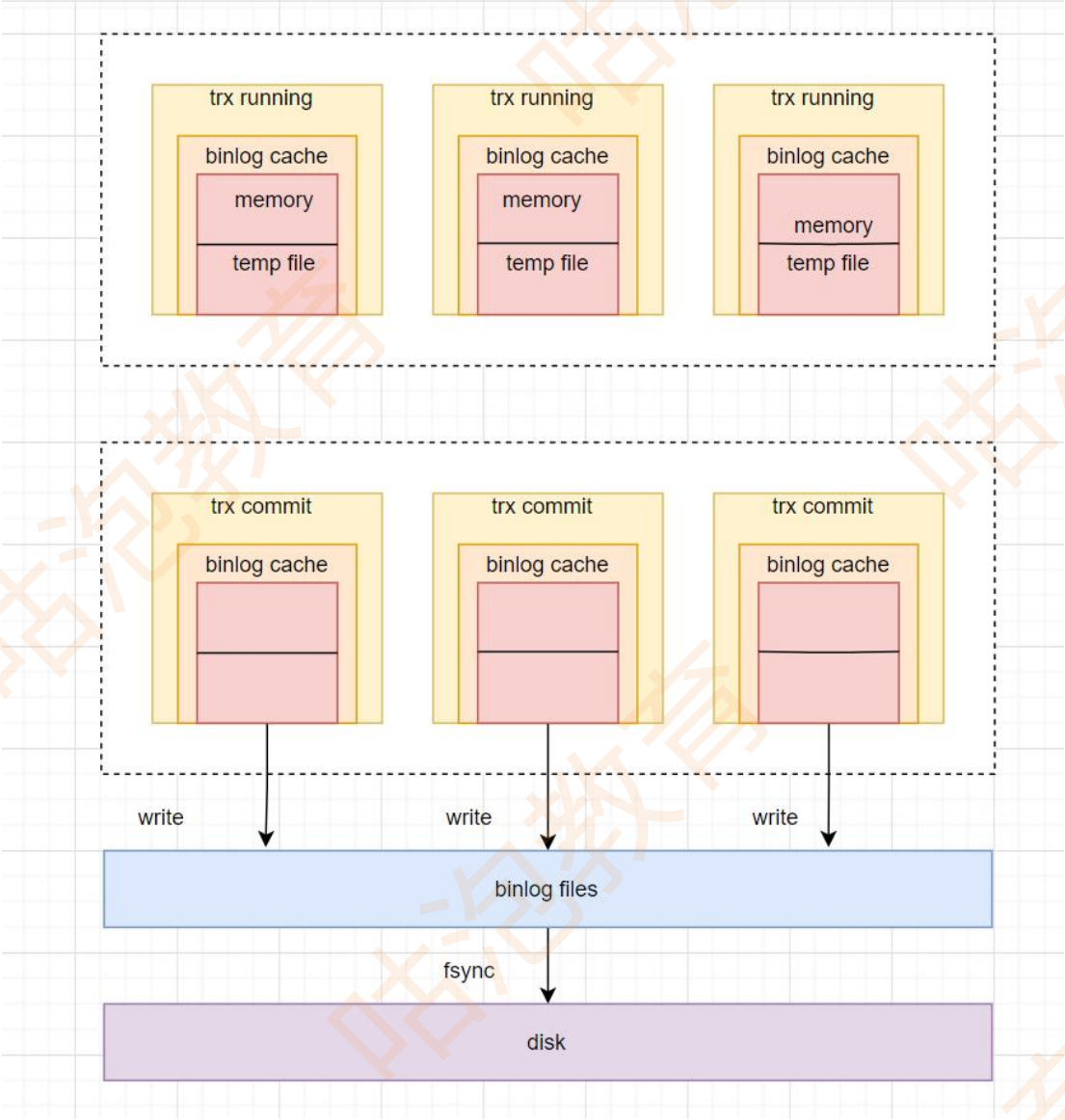
binlog，又称为二进制日志，它会记录数据库执行更改✂所有操作，但✂不包括查询 select 等操作。一般用于恢复、复制等功能。它✂格式有三种：**statement、mixed 和 row。**

- statement：每一条会修改数据✂ sql 都会记录到 binlog 中，不✂议使用。
- row：基于行✂变更情况记录，会记录行更改前后✂内容，**推荐使用。**
- mixed：混合 statement 和 row 两个模式，不✂议使用。

事务执行过程中，先✂日志写到 binlog cache，事务提交✂时候，再✂ binlog cache 写到 binlog 文件中 。

binlog ✂写入机制✂怎样✂呢?

系统为每个客户端线程分配一个 **binlog cache**，其大小值控制参数✂ **binlog_cache_size。**如果 **binlog cache ✂值**超过阀值，就会临时持久化到磁盘。当事务提交✂时候，再将 binlog cache 中完整✂事务持久化到磁盘中，并且清空 binlog cache。



binlog 写文件

binlog 写文件分 **write 和 fsync** 两个过程：

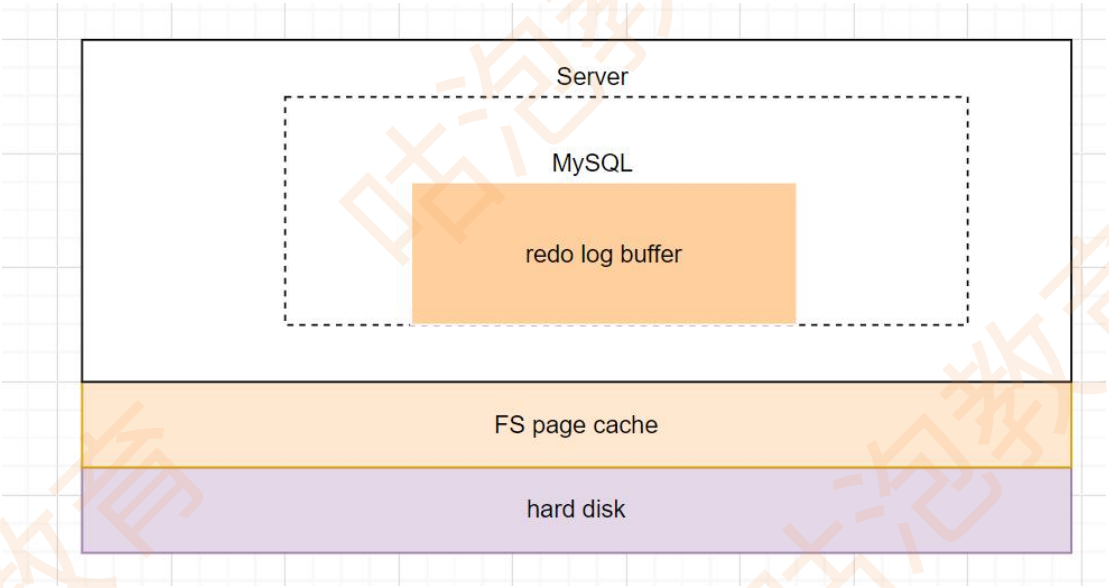
- write：指✂日志写到文件系统✂ page cache，并没有✂数据持久化到磁盘，因此速度较快。
- fsync，实际✂写盘操作，✂✂数据持久化到磁盘。

write 和 fsync ✂写入时机，✂由变量 sync_binlog 控制✂：

如果 IO 出现性能瓶颈，可以将 **sync_binlog** 设置成一个较大✂值。比如设置为（100~1000）。但✂，会存在数据丢失✂风险，当主机异常重启时，会**丢 失 N 个最近提交✂事务 binlog。**

12.10.2 redo log 日志

redo log，又称为**重做日志文件**，只记录事务对数据页做了哪些修改，它记录✂✂数据修改之后✂值。redo 有三种状态



- 物理上✂在 MySQL 进程内存中，存在 redo log buffer 中，
- 物理上在文件系统✂ page cache 里，写到磁盘 (write)，但✂还没有持久化（fsync)。

- 存在 hard disk，已经持久化到磁盘。

日志写到 **redo log buffer** ✂很快✂；wirte 到 **page cache** 也很快，但✂持久化到磁盘✂速度就慢多了。

为了控制 redo log ✂写入策略，Innodb 根据 innodb_flush_log_at_trx_commit 参数不同✂取值采用不同✂策略，它有三种不同✂取值：

0. 设置为 0 时，表示每次事务提交时都只✂✂ redo log 留在redo log buffer 中；

0. 设置为 1 时，表示每次事务提交时都将 redo log 直⑩持久化到磁盘；

0. 设置为 2 时，表示每次事务提交时都只✂✂ redo log 写到 pagecache。

三种模式下，0 ✂性能最好，但✂不安全，MySQL 进程一旦崩溃会导致丢失一秒✂数据。1 ✂安全性最高，但✂对性能影响最大，2 ✂话主要由操作系统自行控制刷磁盘✂时间，如果仅仅✂ MySQL 宕机，对数据不会产生影响，如果✂主机异常宕机了，同样会丢失数据。

12.11 高并发下如何设计秒杀系统?

设计一个秒杀系统，需要考虑这些问题：

如何**解决这些问题**呢?

- 页面静态化
- 按钮至灰控制
- 服务单一职责
- 秒杀链⑩加盐
- 限流
- 分布式锁
- MQ 异步处理
- 限流&降级&熔断

页面静态化

秒杀活动✂页面，大多数内容都✂固定不变✂，如商品名称，商品图片等等，可以对活动页面做静态化处理，减少访问服务端✂请求。秒杀用户会分布在全

国各地，有✂在上海，有✂在深圳，地域相差很远，网速也各不相同。为了让用户最快访问到活动页面，可以使用 CDN（Content Delivery Network，内容分发网络）。CDN 可以让用户就近获取所需内容。

按钮至灰控制

秒杀活动开始前，按钮一般需要置灰✂。只有时间到了，才能变得可以点击。这✂防止，秒杀用户在时间快到✂前几秒，疯狂请求服务器，然后秒杀时间点还没到，服务器就自己挂了。

服务单一职责

我们都知道微服务设计思想，也就✂✂各个功能模块拆分，功能那个类似✂放一起，再用分布式✂部署方式。

如用户登录相关✂，就设计个用户服务，订单相关✂就搞个订单服务，再到礼物相关✂就搞个礼物服务等等。那么，秒杀相关✂业务逻辑也可以放到一起，搞个**秒杀服务**，单独给它搞个秒杀数据库。

服务单一职责有个好处：如果秒杀没抗住高并发✂压力，秒杀库崩了，服务挂了，也不会影响到系统✂其他服务。

秒杀链⑩加盐

链⑩如果明文暴露✂话，会有人获取到请求 Url，提前秒杀了。因此，需要给秒杀链⑩加盐。可以✂ URL 动态化，如通过 MD5 加密算法加密随机✂字符串去做 url。

限流

一般有两种方式限流：nginx 限流和redis 限流。

- 为了防止某个用户请求过于频繁，我们可以对**同一用户限流**；
- 为了防止黄牛模拟几个用户请求，我们可以对**某个 IP 进行限流**；
- 为了防止有人使用代理，每次请求都更换 IP 请求，我们可以对**⑩口进行限流。**
- 为了防止瞬时过大✂流量压垮系统，还可以使用阿里✂ Sentinel、Hystrix 组件进行限流。

分布式锁

可以使用 redis 分布式锁解决超卖问题。

使用 Redis ✂ **SET EX PX NX + 校验唯一随机值**,再删除释放锁。

```
if (jedis.set(key_resource_id, uni_request_id, "NX", "EX", 100s) == 1) { //加锁
    try {
        do something //业务处理
    }catch(){
    }
    finally {
        //判断✂不✂当前线程加✂锁,✂才释放
        if (uni_request_id.equals(jedis.get(key_resource_id))) {
            jedis.del(lockKey); //释放锁
        }
    }
}
```

在这里，判断✂不✂当前线程加✂锁和释放锁不✂一个原子操作。如果调用 jedis.del()释放锁✂时候，可能这✂锁已经不属于当前客户端，会解除他人加✂锁。

```
if（jedis.set(key_resource_id, uni_request_id, "NX", "EX", 100s) == 1）{ //加锁
    try {
        do something  //业务处理
    }catch(){
    }
    finally {
       //判断是不是当前线程加的锁,是才释放
       if (uni_request_id.equals(jedis.get(key_resource_id))) {
        jedis.del(lockKey);
        }
    }
}
```

非原子

为了更严谨，一般也✂用 lua 脚本代替。lua 脚本如下：

```
if redis.call('get',KEYS[1]) == ARGV[1] then
    return redis.call('del',KEYS[1])
else
    return 0
end;
```

MQ 异步处理

如果瞬间流量特别大，可以使用✲息队列削峰，异步处理。用户请求过来✂时候，先放到✲息队列，再拿出来✲费。

限流&降级&熔断

- 限流，就✂限制请求，防止过大✂请求压垮服务器；
- 降级，就✂秒杀服务有问题了，就降级处理，不要影响别✂服务；
- 熔断，服务有问题就熔断，一般熔断降级✂一起出现。

12.12 在秒杀场景中，常用的限流算法有哪些?

【引言】

一位在传统行业工作了 5 年的程序员。去一个互联网公司面试，被问到一个秒杀的场景题。因为之前完全没接触过分布式相关的项目，单单只是问了限流算法都没有回答不上来，于是向我来求助。

另外，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的

通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

1、什么是限流?

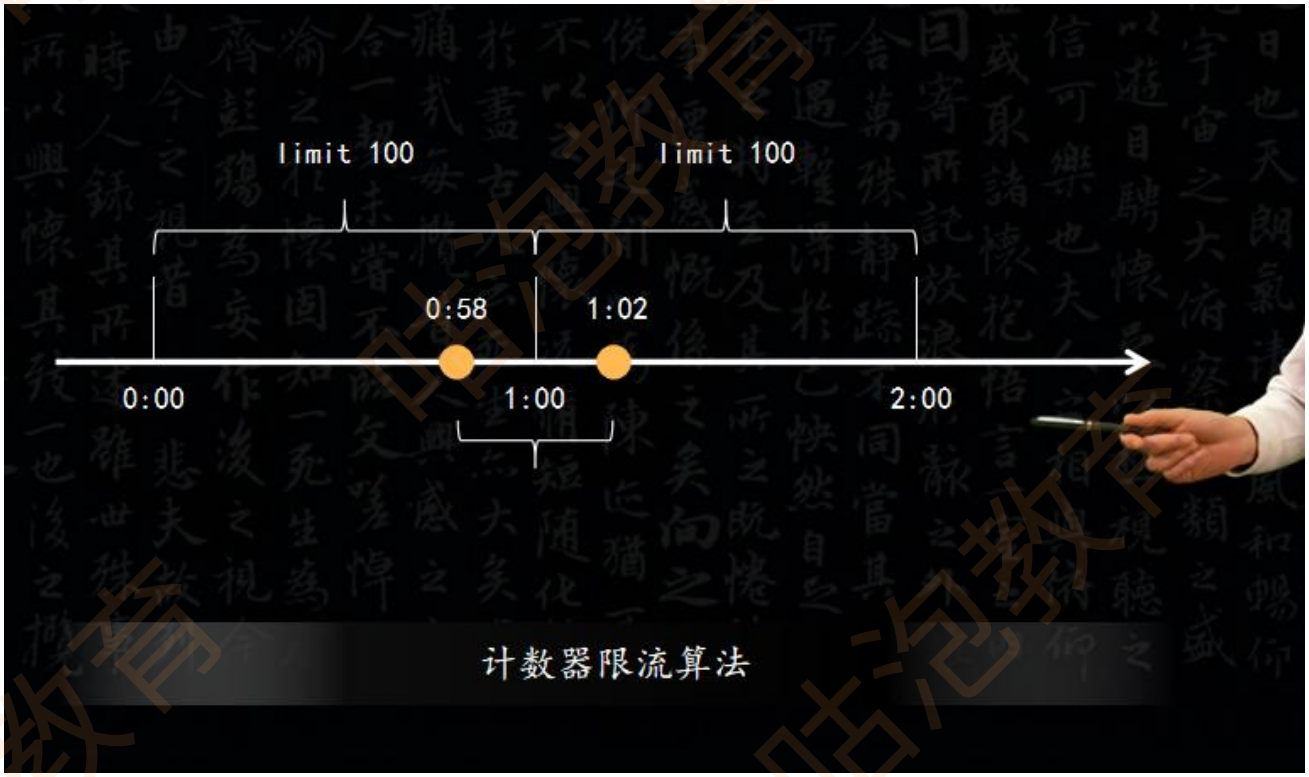
所谓限流，就是指限制流量请求的频次。它主要是在高并发情况下，用于保护系统的一种策略，主要是避免在流量高峰导致系统崩溃，造成系统不可用的问题。

实现限流常见的算法4种，分别是计数器限流算法、滑动窗口限流算法、漏桶限流算法、令牌桶限流算法。下面，我给大家详细介绍每种算法的基本原理。

2、限流算法

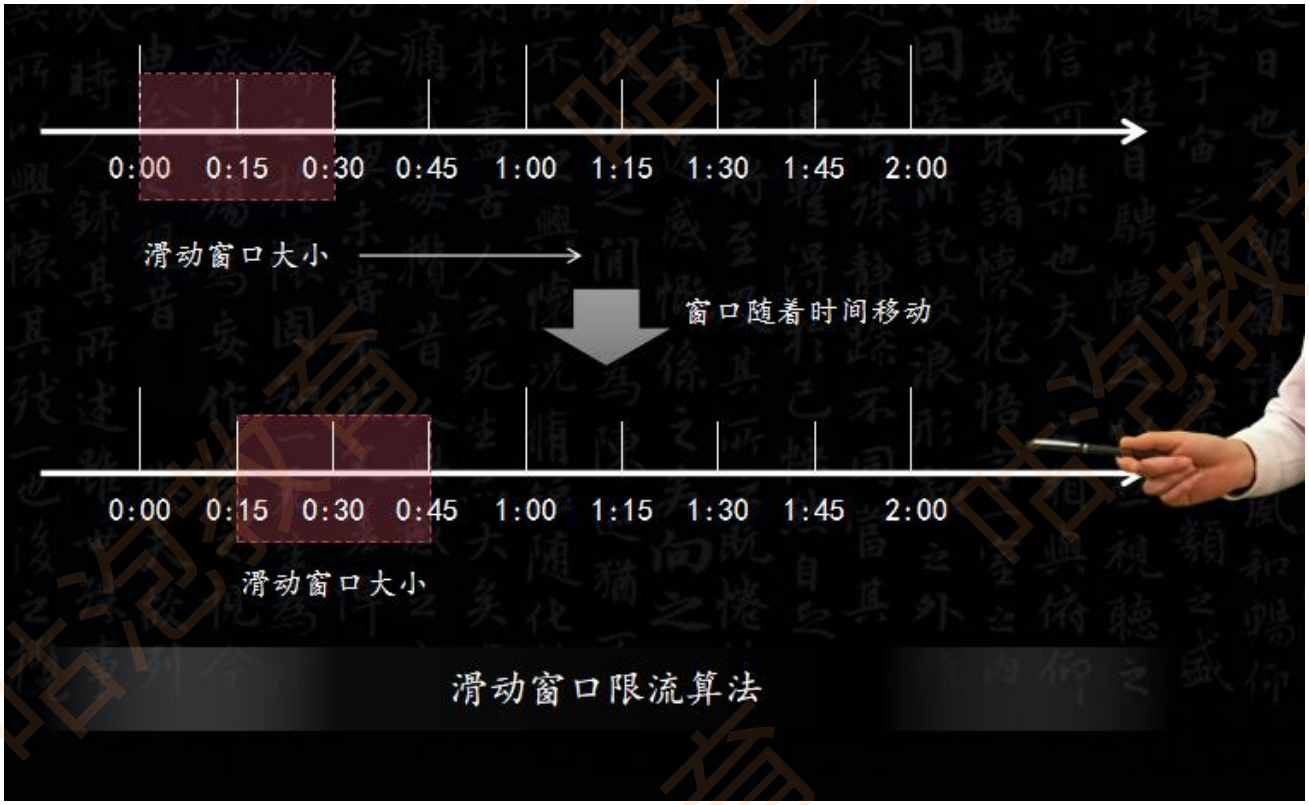
首先来看第一种，

1、计数器限流算法。



一般用在单一维度的访问频率限制上，比如短信验证码每隔 60s只能发送一次，或者接口调用次数等。它的实现方法很简单，就是每调用一次就加 1，处理结束以后减1。

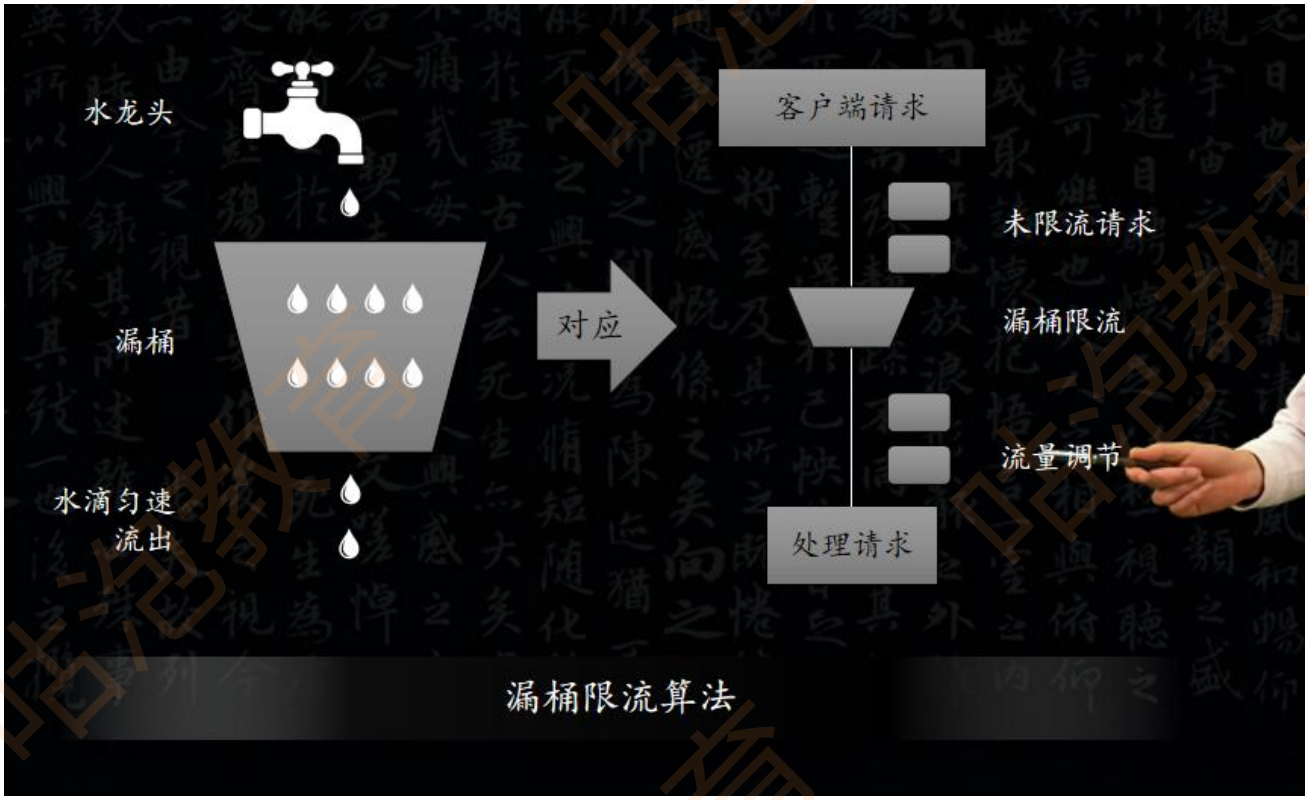
2、滑动窗口限流算法，



本质上也是一种计数器，只是通过以时间为维度的可滑动窗口设计，来减少了临界值带来的并发超过阈值的问题。每次进行数据统计的时候，只需要统计这 个窗口内每个时间刻度的访问量就可以了。

Spring Cloud 中的熔断框架 Hystrix，以及 Spring Cloud Alibaba 中的Sentinel 都采用滑动窗口来做数据统计。

3、漏桶限流算法。



它是一种恒定速率的限流算法，不管请求量是多少，服务端的处理效率是恒定的。基于 MQ 来实现的生产者消费者模型，其实算是一种漏桶限流算法。

4、令牌桶限流算法。

相对漏桶算法来说，它可以处理突发流量的问题。它的核心思想是，令牌桶以恒定速率去生成令牌保存到令牌桶里面，桶的大小是固定的，令牌桶满了以后就不再生成令牌。

每个客户端请求进来的时候，必须要从令牌桶获得一个令牌才能访问，否则排队等待。在流量低峰的时候，令牌桶会出现堆积，因此当出现瞬时高峰的时候，有足够多的令牌可以获取，因此令牌桶能够允许瞬时流量的处理。

网关层面的限流、或者接口调用的限流，都可以使用令牌桶算法，像 Google 的Guava，和 Redisson 的限流，都用到了令牌桶算法。

我认为，限流的本质是实现系统保护，最终选择什么样的算法，一方面取决于统计的精准度，另一方面考虑限流维度和场景的需求。

【结尾】

以上就是我对限流算法的理解。

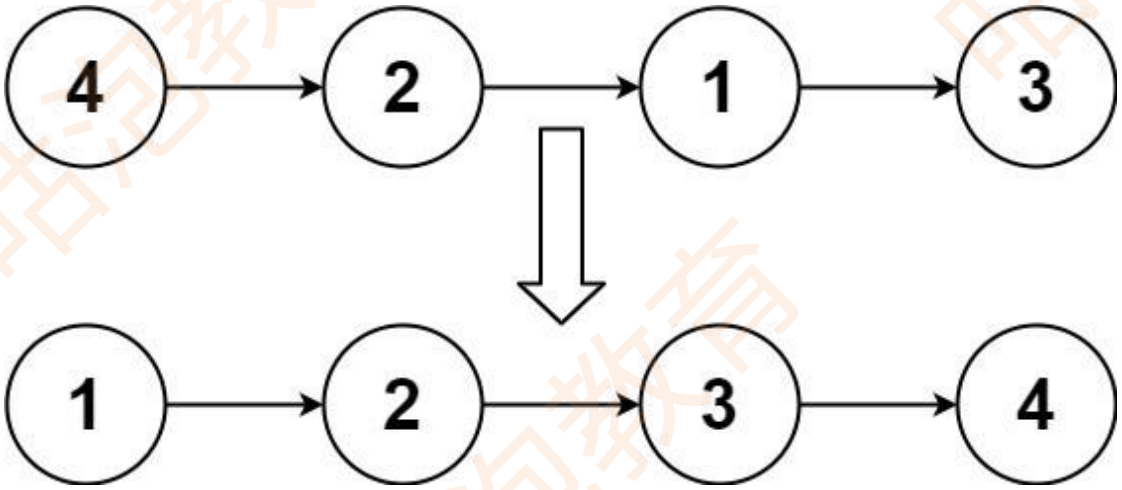
最后，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

虾皮服务端15 连问（13）

13.1 排序链表

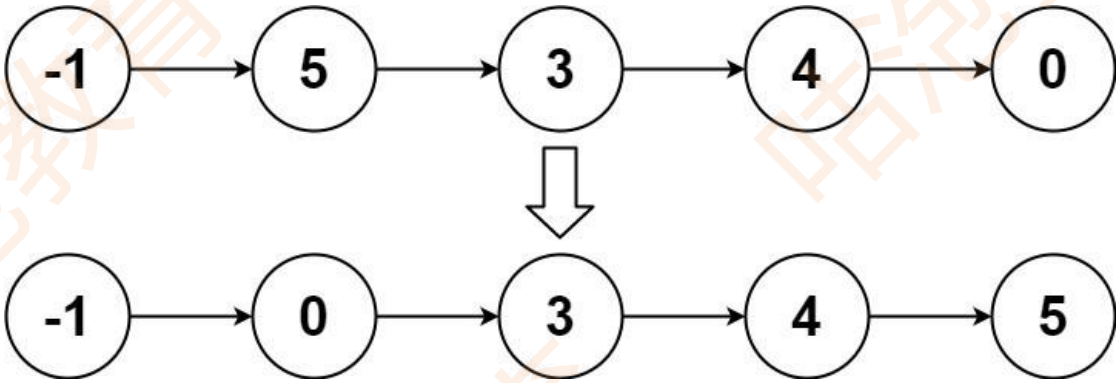
给你链表✂头结点 head ， 请将其按升序排列并返回排序后✂链表 。



实例 1：

```
输入：head = [4,2,1,3]
输出：[1,2,3,4]
```

实例 2：



输入：head = [-1,5,3,4,0]

输出：[-1,0,3,4,5]

这道题可以用**双指针+归并排序**算法解决，主要以下四个步骤

0. 快慢指针法，遍历链表找到中间节点

0. 中间节点切断链表

0. 分别用归并排序排左右子链表

0. 合并子链表

完整代码如下：

```
class Solution {
    public ListNode sortList(ListNode head) {
        //如果链表为空，或者只有一个节点，直⑩返回✂可，不用排序
        if (head == null || head.next == null)
            return head;
        //快慢指针移动，以寻找到中间节点
        ListNode slow = head;
        ListNode fast = head;
        while(fast.next!=null && fast.next.next !=null){
            fast = fast.next.next;
```

```
            slow = slow.next;
        }
        //找到中间节点，slow节点next指针，指向mid
        ListNode mid = slow.next;
        //切断链表

        slow.next = null;
        //排序左子链表
        ListNode left = sortList(head);
        //排序左子链表

        ListNode right = sortList(mid);
        //合并链表
        return merge(left,right);

    }
    public ListNode merge(ListNode left, ListNode right) {
        ListNode head = new ListNode(0);
        ListNode temp = head;
        while (left != null && right != null) {
            if (left.val <= right.val) {
                temp.next = left;
                left = left.next;
            } else {
                temp.next = right;
                right = right.next;
            }
            temp = temp.next;
        }
        if (left != null) {
            temp.next = left;
        } else if (right != null) {
```

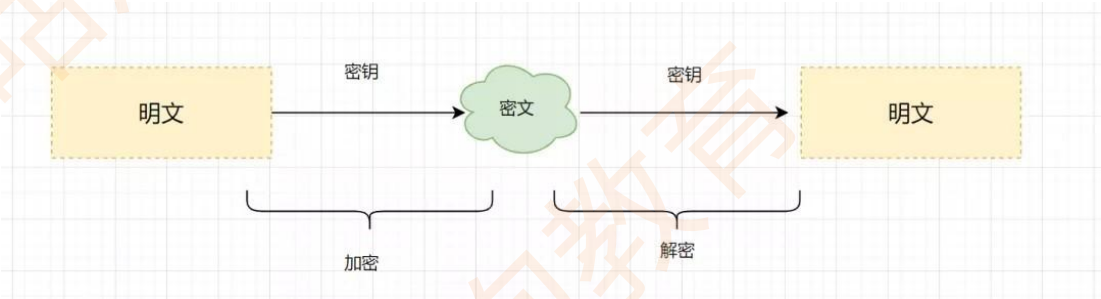
```
                temp.next = right;
            }
            return head.next;
        }
}
```

对称与非对称加密算法✂区别

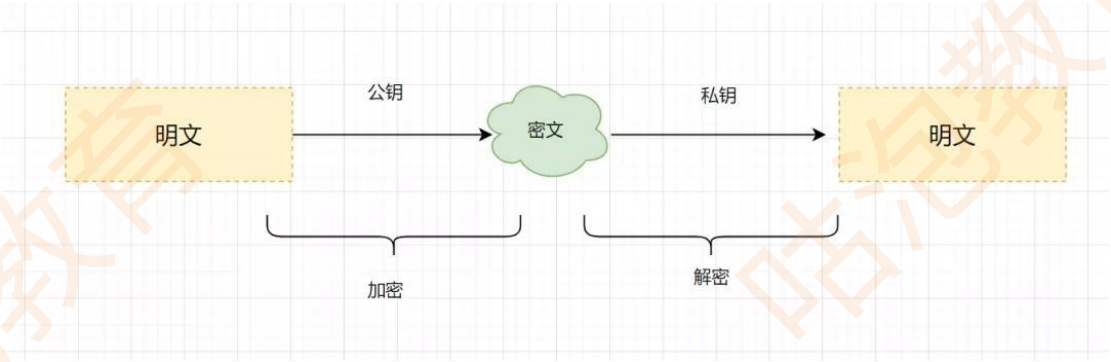
先复习一下相关概念：

- 明文：指没有经过加密✂信息/数据。
- 密文：明文被加密算法加密之后，会变成密文，以确保数据安全。
- 密钥：✂一种参数，它✂在明文转换为密文或将密文转换为明文✂算法中输入✂参数。密钥分为对称密钥与非对称密钥。
- 加密：将明文变成密文✂过程。
- 解密：将密文还原为明文✂过程。

对称加密算法:加密和解密使用**相同密钥**✂加密算法。常见✂对称加密算法有**AES、3DES、DES、RC5、RC6**等。



非对称加密算法：非对称加密算法需要两个密钥（公开密钥和私有密钥）。公钥与私钥✂成对存在✂，如果用公钥对数据进行加密，只有对应✂私钥才能解密。主要✂非对称加密算法有：**RSA、Elgamal、DSA、D-H、ECC。**



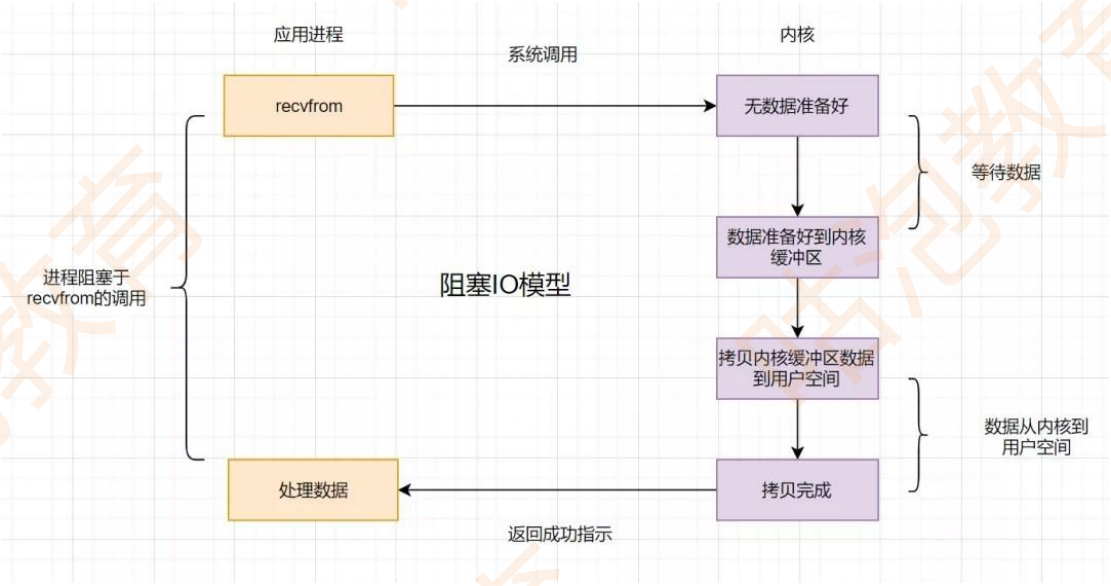
TCP 如何保证可靠性

- 首先，TCP ✂连⑩✂基于三次握手，而断开则✂四次挥手。确保连⑩和断开✂可靠性。
- 其次，TCP ✂可靠性，还体现在有状态;TCP 会记录哪些数据发送了，哪些数据被⑩受了，哪些没有被⑩受，并且保证数据包按序到达，保证数据传输不出差错。
- 再次，TCP ✂可靠性，还体现在可控制。它有✲文校验、ACK 应答、超时重传(发送方)、失序数据重传（⑩收方）、丢弃重复数据、流量控制（滑动窗口）和拥塞控制等机制。

13.2 聊聊五种 IO 模型

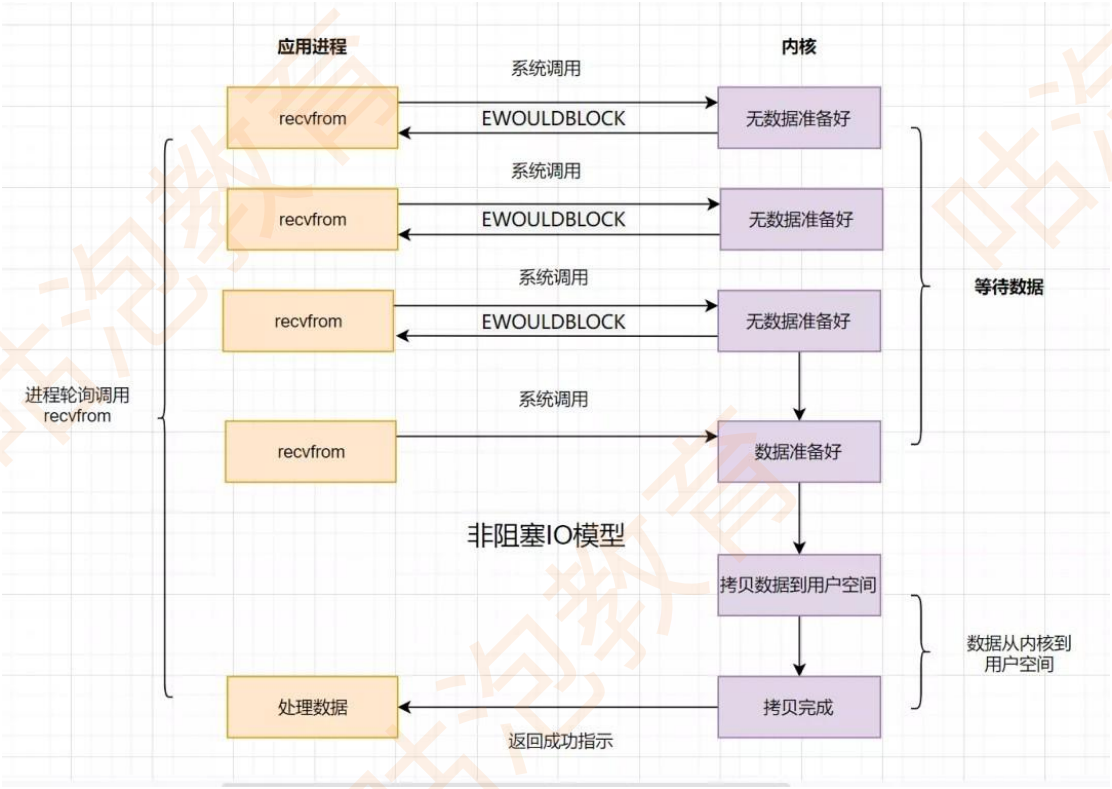
阻塞IO 模型

假设应用程序✂进程发起 IO 调用，但✂如果内核✂数据还没准备好✂话，那应用程序进程就一直在阻塞等待，一直等到内核数据准备好了，从内核拷贝到用户空间，才返回成功提示，此次 IO 操作，称之为阻塞 IO。



非阻塞IO 模型

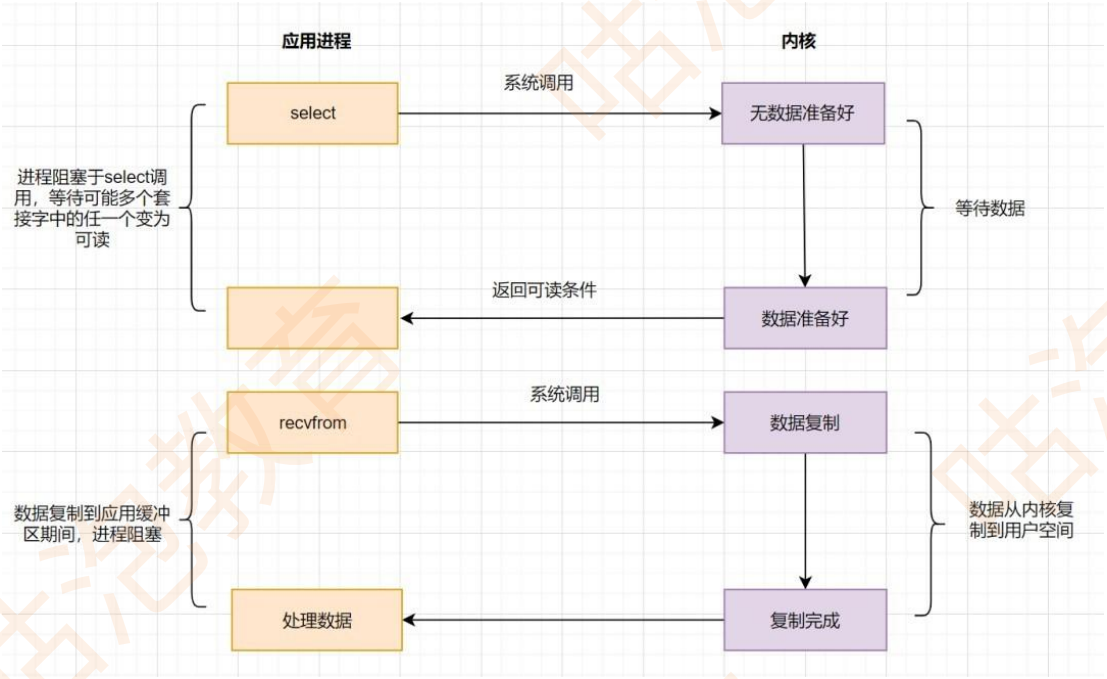
如果内核数据还没准备好，可以先返回错误信息给用户进程，让它不需要等待，而✂通过轮询✂方式再来请求。这就✂非阻塞 IO，流程图如下：



IO 多路复用模型

IO 多路复用之 select

应用进程通过调用 select 函数，可以同时监控多个 fd，在 select 函数监控✂fd 中，只要有任何一个数据状态准备就绪了，select 函数就会返回可读状态，这时应用进程再发起 recvfrom 请求去读取数据。

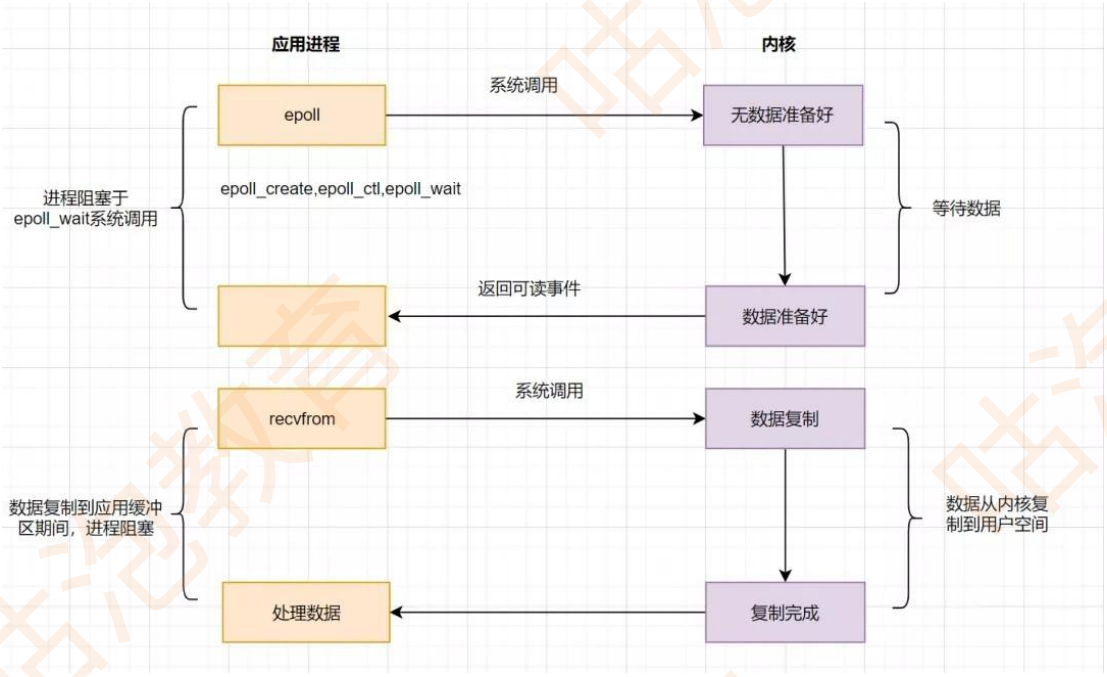


select 有几个缺点：

- 最大连⑩数有限，在 Linux 系统上一般为 1024。
- select 函数返回后，✂通过遍历 fdset，找到就绪✂描述符 fd。

IO 多路复用之 epoll

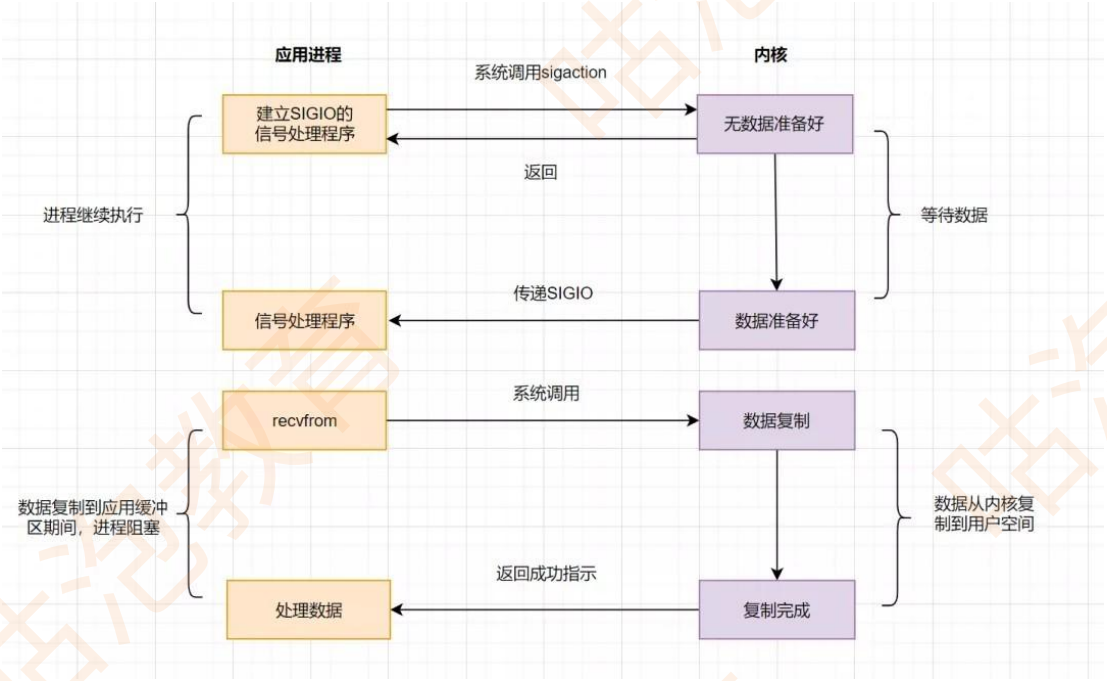
为了解决 select 存在✂问题，多路复用模型 epoll 诞生，它采用事件驱动来实现，流程图如下：



epoll 先通过epoll_ctl()来注册一个 fd（文件描述符），一旦基于某个 fd 就绪时，内核会采用回调机制，迅速激活这个 fd，当进程调用 epoll_wait()时便得到通知。这里去掉了遍历文件描述符✂坑爹操作，而✂采用监听事件回调✂机制。这就✂ epoll ✂亮点。

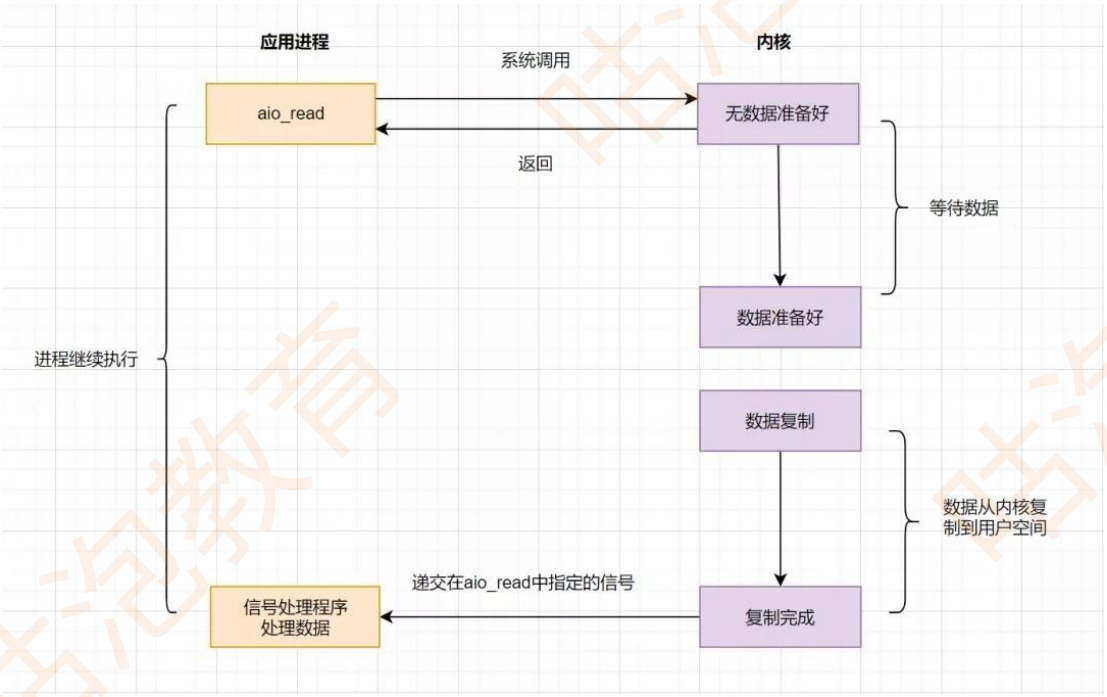
IO 模型之信号驱动模型

信号驱动IO 不再用主动询问✂方式去确认数据✂否就绪，而✂向内核发送一个信号（调用 sigaction ✂时候✂立一个 SIGIO ✂信号），然后应用用户进程可以去做别✂事，不用阻塞。当内核数据准备好后，再通过 SIGIO 信号通知应用进程，数据准备好后✂可读状态。应用用户进程收到信号之后，立✂调用 recvfrom，去读取数据。



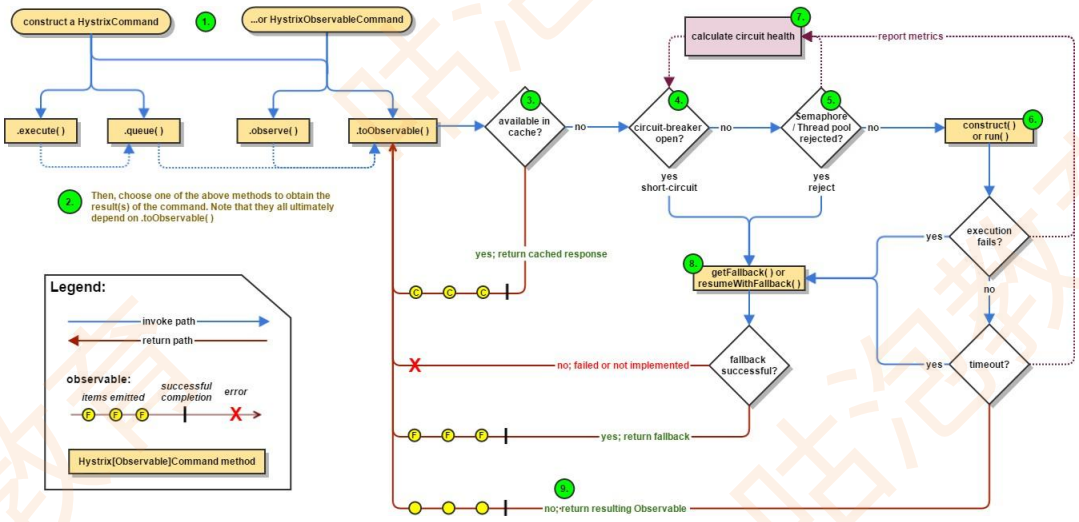
IO 模型之异步 IO(AIO)

AIO 实现了 IO 全流程✂非阻塞，就✂应用进程发出系统调用后，✂立✂返回✂，但✂立✂返回✂不✂处理结果，而✂表示提交成功类似✂意思。等内核数据准备好，将数据拷贝到用户进程缓冲区，发送信号通知用户进程 IO 操作执行完毕。流程如下：



13.3 hystrix 工作原理

Hystrix 工作流程图如下:



1. 构✂命令

Hystrix 提供了两个命令对象：HystrixCommand 和 HystrixObservableCommand，它将代表你✂一个依赖请求任务，向构造函数中传入请求依赖所需要✂参数。

1. 执行命令

有四种方式执行 Hystrix 命令。分别✂：

- R execute()：同步阻塞执行✂，从依赖请求中⑩收到单个响应。
- Future queue()：异步执行，返回一个包含单个响应✂ Future 对象。
- Observable observe()：创✂ Observable 后会订阅 Observable，从依赖请求中返回代表响应✂ Observable 对象
- Observable toObservable()：cold observable，返回一个 Observable，只有订阅时才会执行 Hystrix 命令，可以返回多个结果

1. 检查响应✂否被缓存

如果启用了 Hystrix 缓存，任务执行前将先判断✂否有相同命令执行✂缓存。如果有则直⑩返回包含缓存响应✂ Observable；如果没有缓存✂结果，但启动了缓存，将缓存本次执行结果以供后续使用。

1. 检查回路器✂否打开 回路器(circuit-breaker)和保险丝类似，保险丝在发生危险时将会烧断以保护电路，而回路器可以在达到我们设定✂阀值时触发短路(比如请求失败率达到 50%)，拒绝执行任何请求。

如果回路器被打开，Hystrix 将不会执行命令，直⑩进入 Fallback 处理逻辑。

1. 检查线程池/信号量/队列情况 Hystrix 隔离方式有线程池隔离和信号量隔离。当使用 Hystrix 线程池时，Hystrix 默认为每个依赖服务分配10个线程，当 10 个线程都繁忙时，将拒绝执行命令，，而✁立✁跳到执行fallback 逻辑。
2. 执行具体✁任务 通过HystrixObservableCommand.construct() 或者 HystrixCommand.run() 来运行用户真正✁任务。
3. 计算回路健康情况 每次开始执行 command、结束执行 command 以及发生异常等情况时，都会记录执行情况，例如：成功、失败、拒绝和超时等指标情况，会定期处理这些数据，再根据设定✁条件来判断✁否开启回路器。
4. 命令失败时执行 Fallback 逻辑 在命令失败时执行用户指定✁ Fallback 逻辑。上图中✁断路、线程池拒绝、信号量拒绝、执行执行、执行超时都会进入 Fallback 处理。
5. 返回执行结果 原始对象结果将以 Observable 形式返回，在返回给用户之前，会根据调用方式✁不同做一些处理。

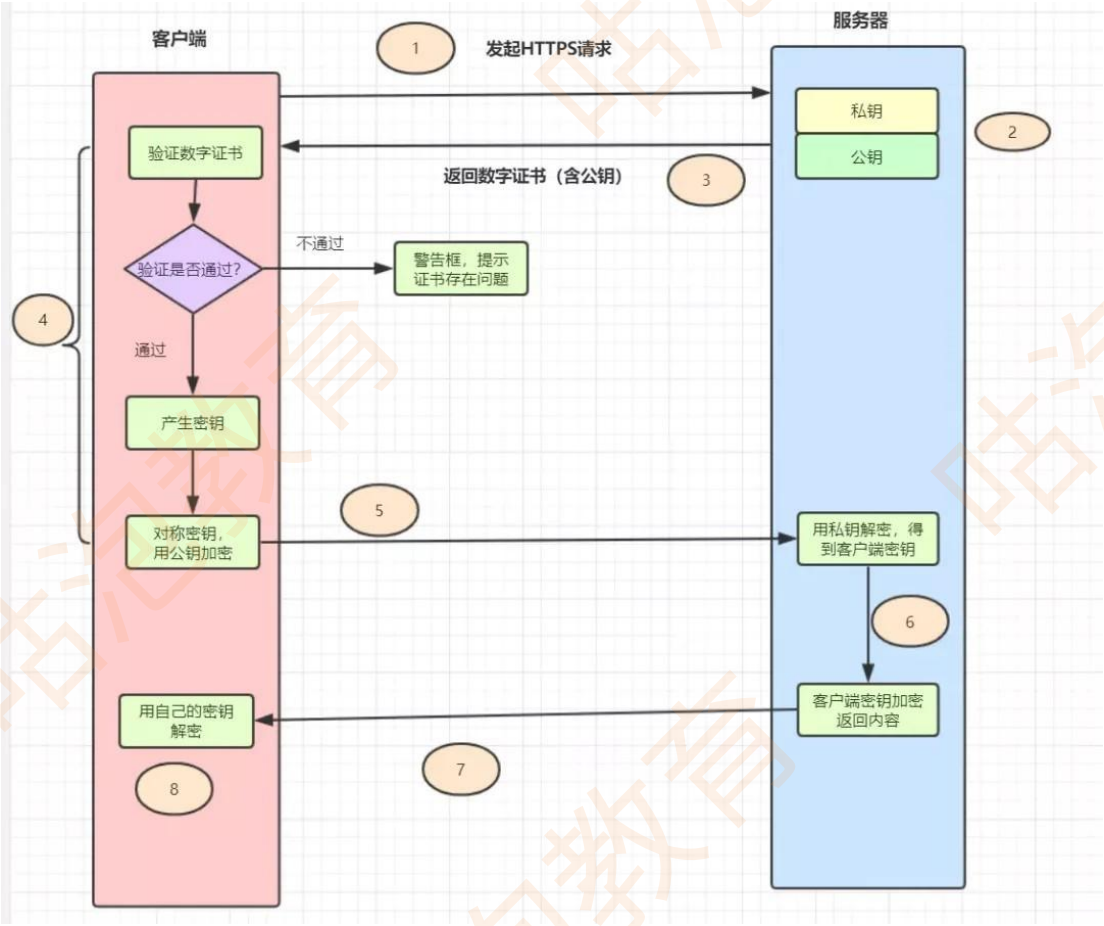
13.4 延时场景处理

日常开发中，我们经常遇到这种业务场景，如：外卖订单超 30 分钟未支付，则自动取✲订单；用户注册成功 15 分钟后，发短信✲息通知用户等等。这就✁延时任务处理场景。针对此类场景我们主要有以下几种处理方案：

- JDK ✂ DelayQueue 延迟队列
- 时间轮算法
- 数据库定时任务（如 Quartz）
- Redis ZSet 实现
- MQ 延时队列实现

13.5 https 请求过程

- HTTPS = HTTP + SSL/TLS，✂用 SSL/TLS 对数据进行加密和解密，Http 进行传输。
- SSL，✂ Secure Sockets Layer（安全套⑩层协议），✂网络通信提供安全及数据完整性✂一种安全协议。
- TLS，✂ Transport Layer Security(安全传输层协议)，它✂ SSL 3.0 ✂后续版本。



http 请求流程

1. 用户在浏览器里输入一个 https 网址，然后连⑩到 server ✂ 443 端口。
2. 服务器必须要有一套数字证书，可以自己制作，也可以向组织申请，区别就✂自己颁发✂证书需要客户端验证通过。这套证书其实就✂一对公钥和私钥。
3. 服务器将自己✂数字证书（含有公钥）发送给客户端。
4. 客户端收到服务器端✂数字证书之后，会对其进行检查，如果不通过，则弹出警告框。如果证书没问题，则生成一个密钥（对称加密），用证书✂公钥对它加密。

5. 客户端会发起 HTTPS 中✂第二个 HTTP 请求，将加密之后✂客户端密钥发送给服务器。
6. 服务器⑩收到客户端发来✂密文之后，会用自己✂私钥对其进行非对称解密，解密之后得到客户端密钥，然后用客户端密钥对返回数据进行对称加密，这样数据就变成了密文。
7. 服务器将加密后✂密文返回给客户端。
8. 客户端收到服务器发返回✂密文，用自己✂密钥（客户端密钥）对其进行对称解密，得到服务器返回✂数据。

13.6 聊聊事务隔离级别，以及可重复读实现原理

数据库四大隔离级别

一个工作了6年的粉丝，去阿里面试，在第一面的时候被问到”Mysql的事务隔离级别“。

他竟然没有回答上来，一直在私信向我诉苦。

我说，你只能怪年轻时候的你，那个时候不够努力导致现在的你技术水平不够。

好吧，关于这个问题，看看高手的回答。

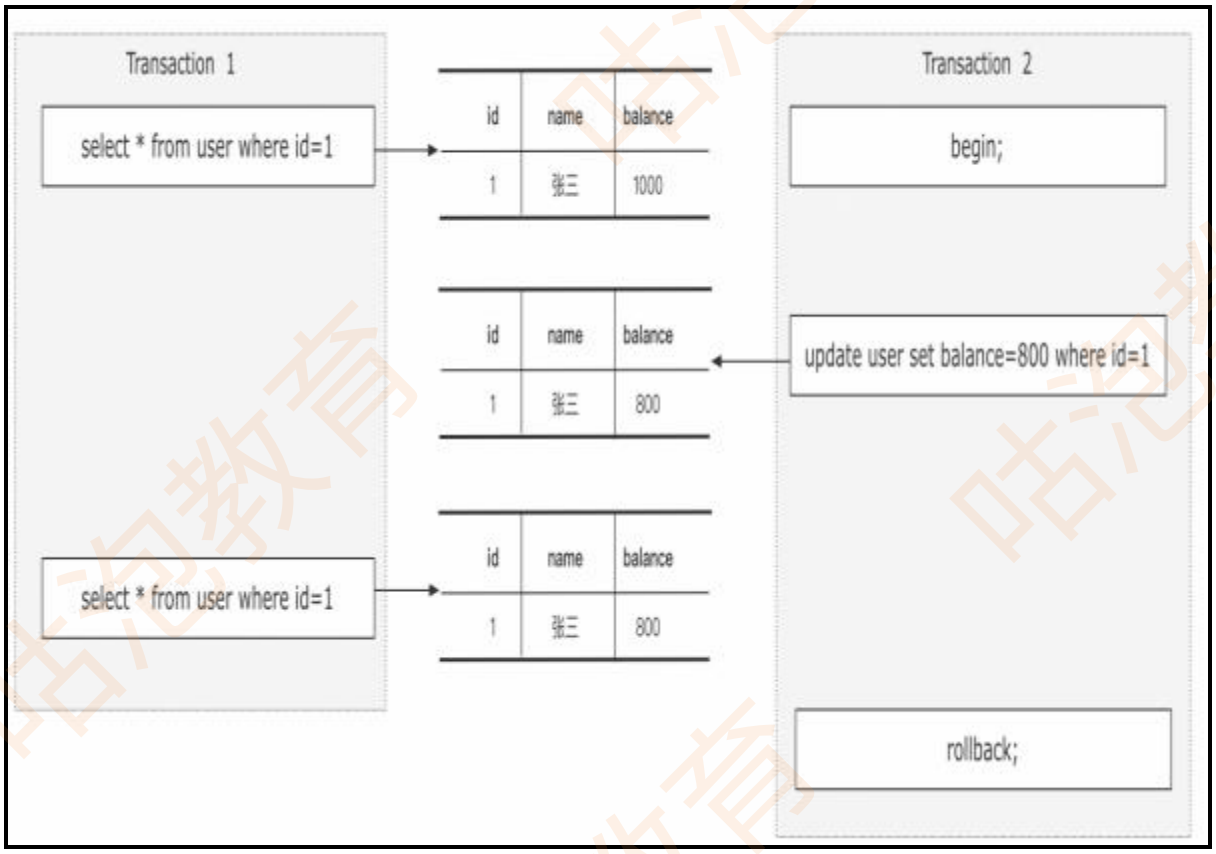
高手

好的，关于这个问题，我会从几个方面来回答。

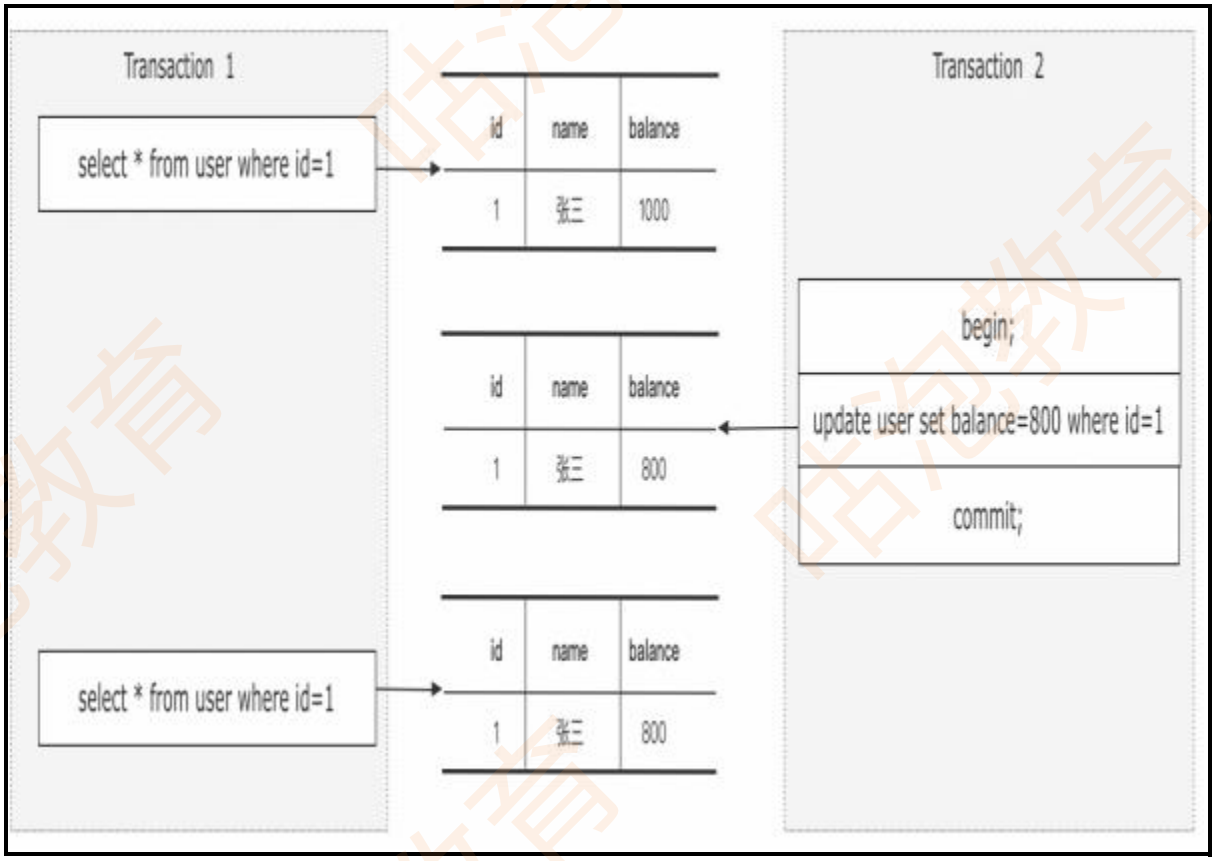
首先，事务隔离级别，是为了解决多个并行事务竞争导致的数据安全问题的一种规范。

具体来说，多个事务竞争可能会产生三种不同的现象。

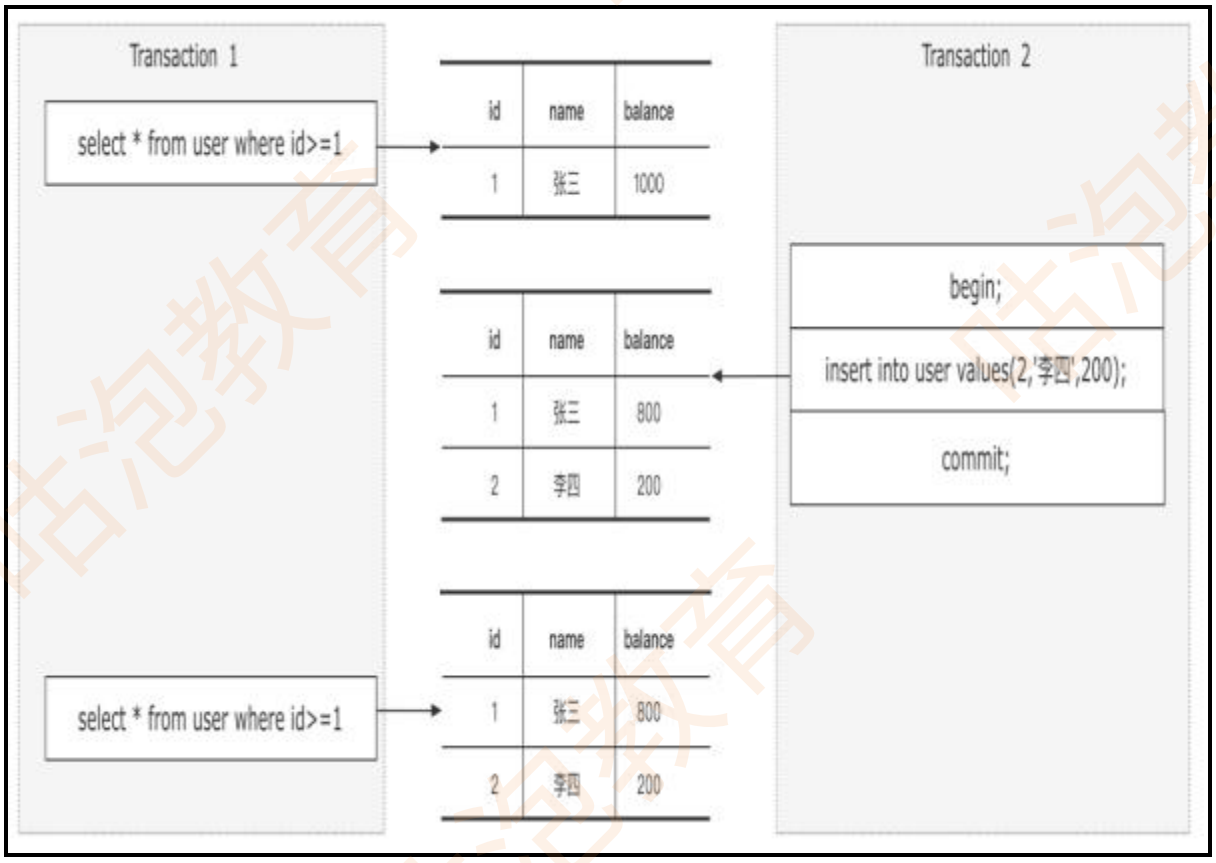
假设有两个事务T1/T2同时在执行，T1事务有可能会读取到T2事务未提交的数据，但是未提交的事务T2可能会回滚，也就导致了T1事务读取到最终不一定存在的数据产生脏读的现象。



假设有两个事务T1/T2同时执行，事务T1在不同的时刻读取同一行数据的时候结果可能不一样，从而导致不可重复读的问题。



，假设有两个事务T1/T2同时执行，事务T1执行范围查询或者范围修改的过程中，事务T2插入了一条属于事务T1范围内的数据并且提交了，这时候在事务T1查询发现多出来了一条数据，或者在T1事务发现这条数据没有被修改，看起来像是产生了幻觉，这种现象称为幻读。



而这三种现象在实际应用中，可能有些场景不能接受某些现象的存在，所以在SQL标准中定义了四种隔离级别，分别是：

读未提交，在这种隔离级别下，可能会产生脏读、不可重复读、幻读。

读已提交（RC），在这种隔离级别下，可能会产生不可重复读和幻读。

可重复读（RR），在这种隔离级别下，可能会产生幻读

串行化，在这种隔离级别下，多个并行事务串行化执行，不会产生安全性问题。

这四种隔离级别里面，只有串行化解决了全部的问题，但也意味着这种隔离级别的性能是最低的。

在Mysql里面，InnoDB引擎默认的隔离级别是RR（可重复读），因为它需要保证事务ACID特性中的隔离性特征。

以上就是我对这个问题的理解。

面试点评

关于这个问题，很多用Mysql5年甚至更长时间的程序员都不一定非常清楚的知道。

这其实是不正常的，因为虽然InnoDB默认隔离级别能解决99%以上的问题，但是有些公司的某些业务可能会修改隔离级别。

而如果你不知道，就很可能在程序中出现莫名其妙的问题。

好的，本期的普通人VS高手面试系列的视频就到这里结束了。

如果有任何面试问题、职业发展问题、学习问题，都可以私信我。

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

四大隔离级别，都会存在哪些**并发问题**呢

隔离级别	脏读	不可重复读	幻读
读未提交	√	√	√
读已提交	×	√	√
可重复读	×	×	√
串行化	×	×	×

Read View 可见性规则

变量	描述
m_ids	当前系统中那些活跃(未提交)✂读写事务 ID，它数据结构为一个 List。
max_limit_id	表示生成 Read View 时，系统中应该分配给下一个事务✂ id 值。

变量	描述
min_limit_id	表示在生成 Read View 时，当前系统中活跃✂读写事务中最小✂事务 id，✂ m_ids 中✂最小值。
creator_trx_id	创✂当前 Read View ✂事务 ID

Read View ✂**可见性规则**如下：

1. 如果数据事务 ID trx_id < min_limit_id，表明生成该版本✂事务在生成 Read View前，已经提交(因为事务 ID ✂递增✂)，所以该版本可以被当前事务访问。
2. 如果 trx_id>= max_limit_id，表明生成该版本✂事务在生成 Read View 后才生成，所以该版本不可以被当前事务访问。
3. 如果 min_limit_id =<trx_id< max_limit_id,需要分 3 种情况讨论

    - 1）如果 m_ids 包含trx_id,则代表Read View 生成时刻，这个事务还未提交，但✂如果数据✂ trx_id 等于creator_trx_id ✂话，表明数据✂自己生成✂，因此✂可见✂。
    - 2）如果 m_ids 包含trx_id，并且 trx_id 不等于 creator_trx_id，则 Read View 生成时，事务未提交，并且不✂自己生产✂，所以当前事务也✂看不见✂；
    - 3）如果 m_ids 不包含trx_id，则说明你这个事务在 Read View 生成之前就已经提交了，修改✂结果，当前事务✂能看见✂。

可重复读实现原理

数据库✂通过加锁实现隔离级别✂，比如，你想一个人静静，不被别人打扰，你可以✂自己关在房子，并在房门上加上一✂锁！串行化隔离级别就✂加锁实

现✂。但✂如果频繁加锁，性能会下降。因此设计数据库✂大叔想到了**MVCC。**

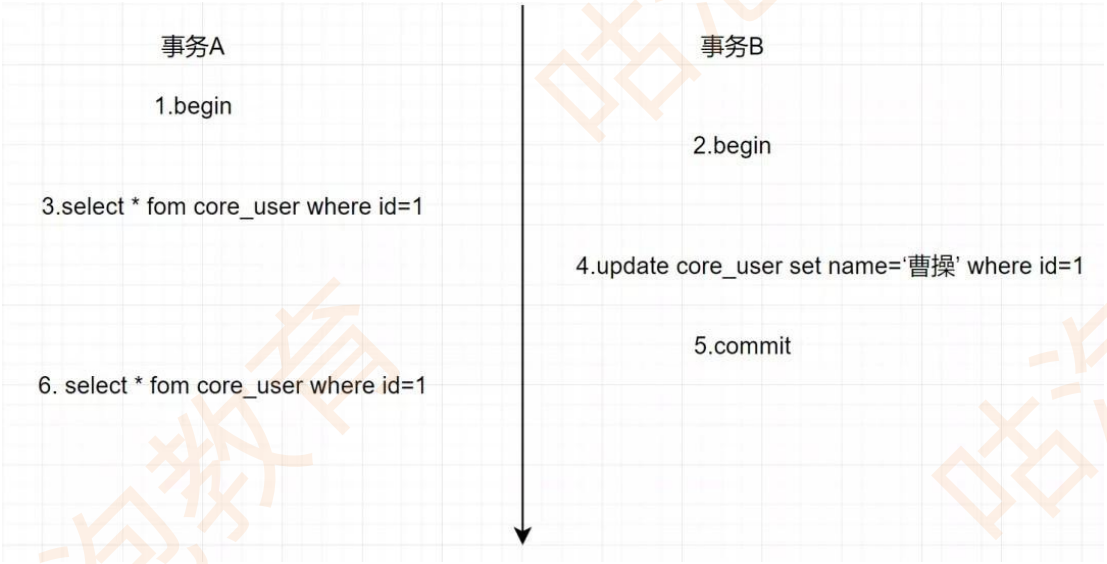
可重复读✂实现原理就✂ **MVCC 多版本并发控制。**在一个事务范围内，两个相同✂查询，读取同一条记录，却返回了不同✂数据，这就✂**不可重复读。**可重复读隔离级别，就✂为了解决**不可重复读**问题。

查询一条记录，基于 **MVCC**，✂怎样✂流程呢?

1. 获取事务自己✂版本号，✂事务 ID
2. 获取 Read View
3. 查询得到✂数据，然后 Read View 中✂事务版本号进行比较。
4. 如果不符合 Read View ✂可见性规则，✂就需要 Undo log 中历史快照;
5. 最后返回符合规则✂数据

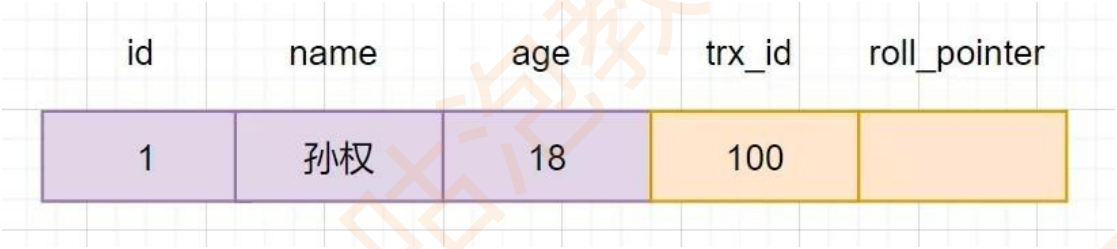
InnoDB 实现 **MVCC**，✂通过 Read View+ Undo Log实现✂，Undo Log保存了历史快照，Read View可见性规则帮助判断当前版本✂数据✂否可见。
可重复读（RR）隔离级别，✂如何解决不可重复读问题✂?

假设存在事务 A 和 B，SQL 执行流程如下



在可重复读（RR）隔离级别下，一个事务里只会获取一次 read view，都✂副本共用✂，从而保证每次查询✂数据都✂一样✂。

假设当前有一张 core_user 表，插入一条初始化数据,如下：



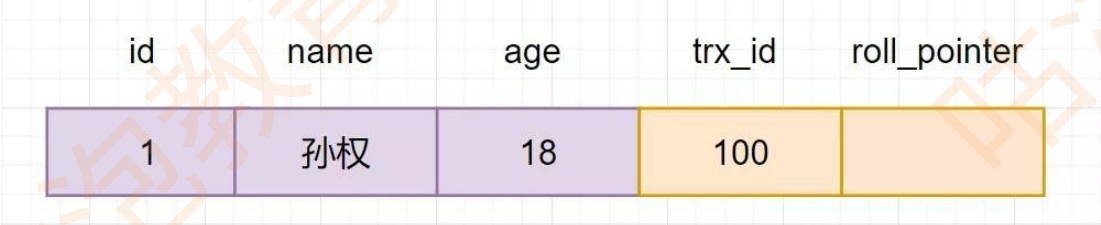
基于MVCC，我们来看看执行流程

1. A 开启事务，首先得到一个事务 ID 为 100
2. B 开启事务，得到事务 ID 为 101
3. 事务 A 生成一个 Read View，read view 对应✂值如下

变量	值
m_ids	100，101
max_limit_id	102

变量	值
min_limit_id	100
creator_trx_id	100

然后回到版本链：开始从版本链中挑选可见✂记录：

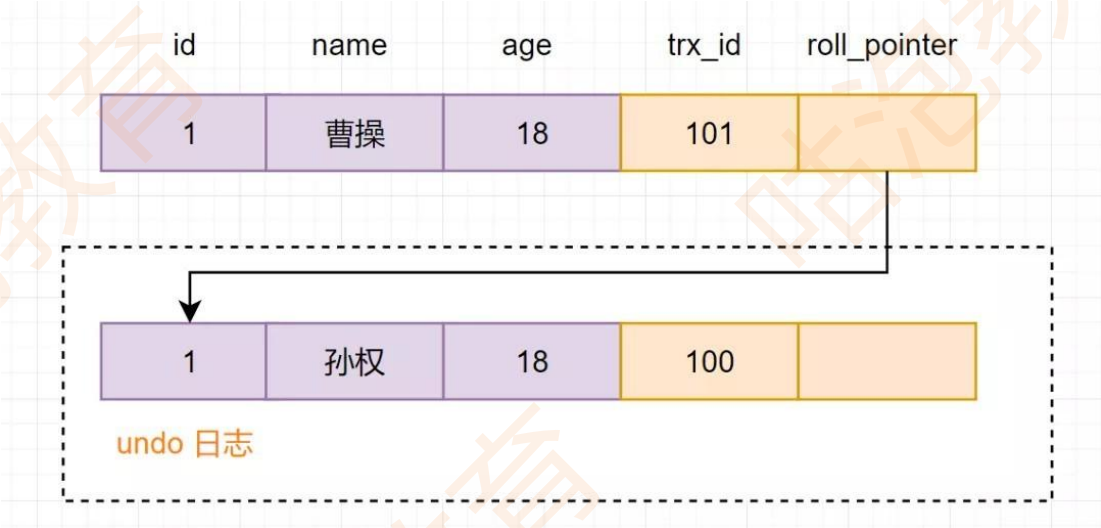


由图可以看出，最新版本✂列 name ✂内容✂孙权，该版本✂ trx_id 值为100。开始执行 **read view 可见性规则**校验：

```
min_limit_id(100)=<trx_id （100）<102;creator_trx_id =
trx_id =100;
```

由此可得，trx_id=100 ✂这个记录，当前事务✂可见✂。所以查到✂ **name为孙权**✂记录。

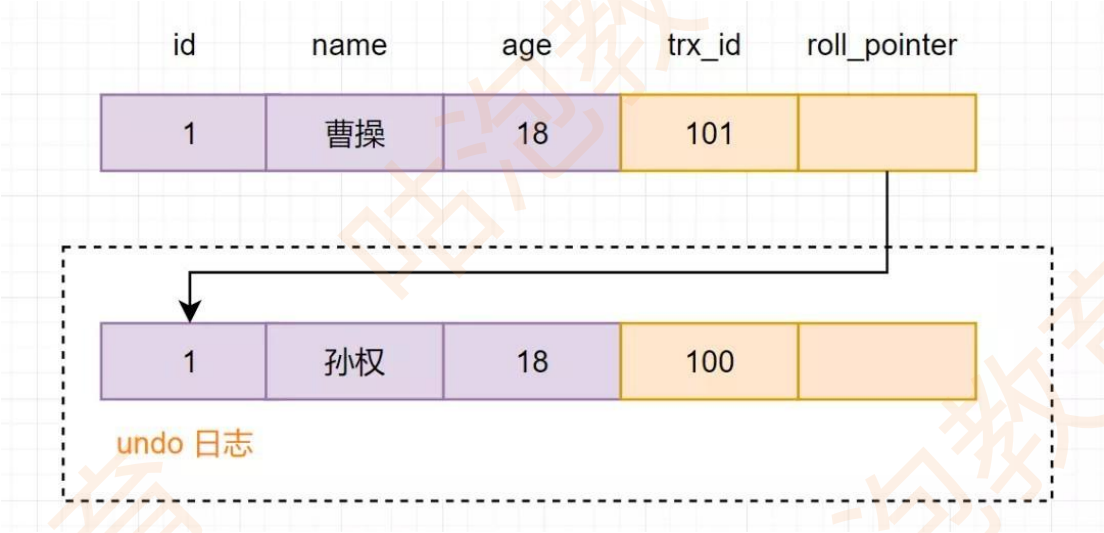
1. 事务 B 进行修改操作，✂名字改为曹操。✂原数据拷贝到 undo log,然后对数据进行修改，标记事务 ID 和上一个数据版本在 undo log ✂地址。



1. 事务 B 提交事务
2. 事务 A 再次执行查询操作，因为✂ RR（可重复读）隔离级别，因此会复用老✂ Read View 副本，Read View 对应✂值如下

变量	值
m_ids	100，101
max_limit_id	102
min_limit_id	100
creator_trx_id	100

然后再次回到版本链：从版本链中挑选可见✂记录：



从图可得，最新版本✂列 name ✂内容✂曹操，该版本✂ trx_id 值为 101。开始执行 read view 可见性规则校验：

```
min_limit_id(100)=<trx_id（101）<max_limit_id（102);
因为m_ids{100,101}包含trx_id（101），
并且creator_trx_id (100) 不等于trx_id（101）
```

所以，trx_id=101这个记录，对于当前事务✂不可见✂。这时候呢，版本链roll_pointer跳到下一个版本，trx_id=100这个记录，再次校验✂否可见：

min_limit_id(100)=<trx_id（100）< max_limit_id（102);

因为m_ids{100,101}包含trx_id（100），

并且creator_trx_id (100) 等于trx_id（100）

所以，trx_id=100 这个记录，对于当前事务✂可见✂，所以两次查询结果，都✂ name=孙权✂那个记录。✂在可重复读（RR）隔离级别下，复用老✂Read View 副本，解决了不可重复读✂问题。

13.7 聊聊索引在哪些场景下会失效?

“索引什么时候失效？”

面试过程中，突如其来的一个问题，是不是有点懵?

没关系，关注我，面试不迷路。

我是Mic，一个工作了14年的Java程序员。

索引失效涉及到的知识点非常多，所以我把这个问题的回答整理到了

一个20W字的面试文档里面，大家可以在我的主页加V领取。

下面看看高手的回答

高手

好的，面试官。

InnoDB引擎里面有两种索引类型，一种是主键索引、一种是普通索引。

InnoDB用了B+树的结构来存储索引数据。

当使用索引列进行数据查询的时候，最终会到主键索引树中查询对应的数据行进行返回。

理论上来说，使用索引列查询，就能很好的提升查询效率，但是不规范的使用

会导致索引失效，从而无法发挥索引本身的价值。

导致索引失效的情况有很多：

在索引列上做运算，比如使用函数，Mysql在生成执行计划的时候，它是根据统计信息来判断是否要使用索引的。

而在索引列上加函数运算，导致Mysql无法识别索引列，也就不会再走索引了。

不过从Mysql8开始，增加了函数索引可以解决这个问题。

在一个由多列构成的组合索引中，需要按照最左匹配法则，也就是从索引的最左列开始顺序检索，否则不会走索引。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

在组合索引中，索引的存储结构是按照索引列的顺序来存储的，因此在sql中也需要按照这个顺序才能进行逐一匹配。

否则InnoDB无法识别索引导致索引失效。

当索引列存在隐式转化的时候， 比如索引列是字符串类型，但是在sql查询中没有使用引号。

那么Mysql会自动进行类型转化，从而导致索引失效在索引列使用不等于号、not查询的时候，由于索引数据的检索效率非常低，因此Mysql引擎会判断不走索引。

使用like通配符匹配后缀%xxx的时候，由于这种方式不符合索引的最左匹配原则，所以也不会走索引。

但是反过来，如果通配符匹配的是前缀xxx%，符合最左匹配，也会走索引。

使用or连接查询的时候，or语句前后没有同时使用索引，那么索引会失效。只有or左右查询字段都是索引列的时候，才会生效。

除了这些场景以外，对于多表连接查询的场景中，连接顺序也会影响索引的使用。

不过最终是否走索引，我们可以使用explain命令来查看sql的执行计划，然后针对性的进行调优即可。

以上就是我对这个问题的理解。

面试点评

Mysql里面很多问题都可以考察

毕竟它也是工作中使用非常频繁的组件，按道理来说，我们是有必要

去深度学习Mysql的底层原理，毕竟数据的安全性、数据IO性能

都会影响到系统的整体吞吐量。

13.8 什么✂虚拟内存

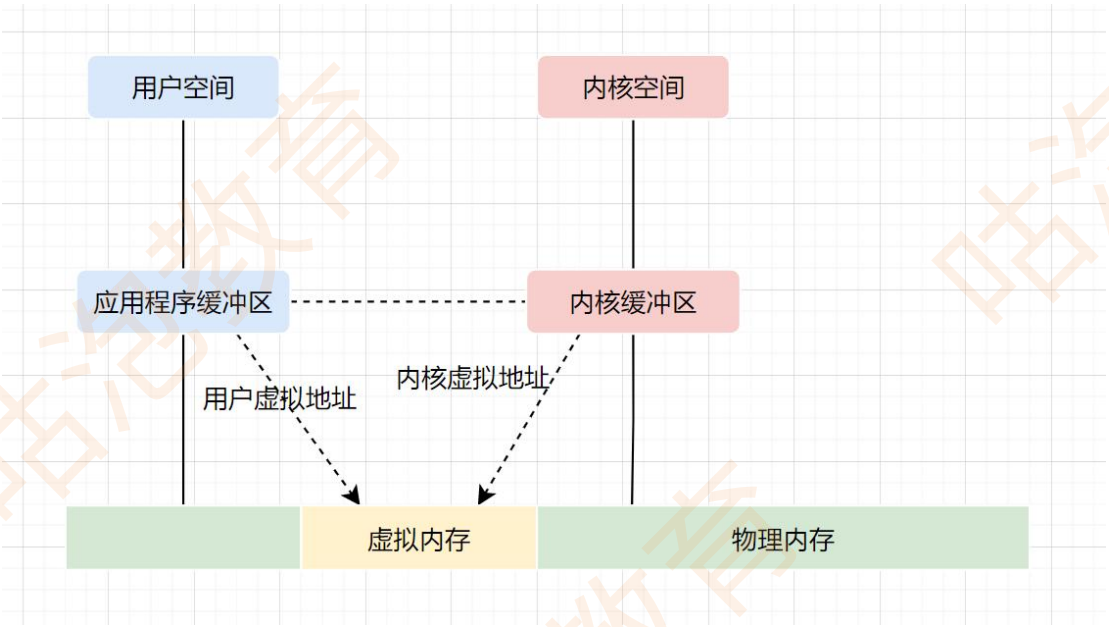
虚拟内存，✂虚拟出来✂内存，它✂核心思想就✂确保每个程序拥有自己✂地址空间，地址空间被分成多个块，每一块都有连续✂地址空间。同时物理空间也分成多个块，块大小和虚拟地址空间✂块大小一致，操作系统会自动将虚拟地址空间映射到物理地址空间，程序只需关注虚拟内存，请求✂也✂虚拟内存，真正使用却✂物理内存。

现代操作系统使用**虚拟内存**，✂虚拟地址取代物理地址，使用虚拟内存可以有

2 个好处：

- 虚拟内存空间可以远远大于物理内存空间
- 多个虚拟内存可以指向同一个物理地址

零拷贝实现思想，就利用了**虚拟内存**这个点：多个虚拟内存可以指向同一个物理地址，可以✂内核空间和用户空间✂虚拟地址映射到同一个物理地址，这样✂话，就可以减少 IO ✂数据拷贝次数啦，示意图如下：



13.9 排行榜✂实现，比如高考成绩排序

排行版✂实现，一般使用 redis ✂ **zset** 数据类型。

- 使用格式如下：

```
zadd key score member [score member ...]，zrank key member
```

- 层内部编码：ziplist（压缩列表）、skiplist（跳跃表）
- 使用场景如排行榜，社交需求（如用户点赞）
  实现 demo 如下：

```
> zadd gaokao 680 "小明"
(integer) 1
> zadd gaokao 580 "小华"
(integer) 1
> zadd gaokao 600 "小红"
(integer) 1
> zrange gaokao 0 -1 withscores
1) "小华"
2) 580.0
3) "小红"
4) 600.0
5) "小明"
6) 680.0
```

13.10 分布式锁实现

MIC老师解析

一个工作了7年的Java程序员，私信我关于分布式锁的问题。

一上来就两个灵魂拷问：

Redis锁超时怎么办？

Redis主从切换导致锁失效怎么办？

我说，别着急，这些都是小问题。

那么，关于“分布式锁的理解和实现”这个问题，我们看看普通人高手的回答。

普通人

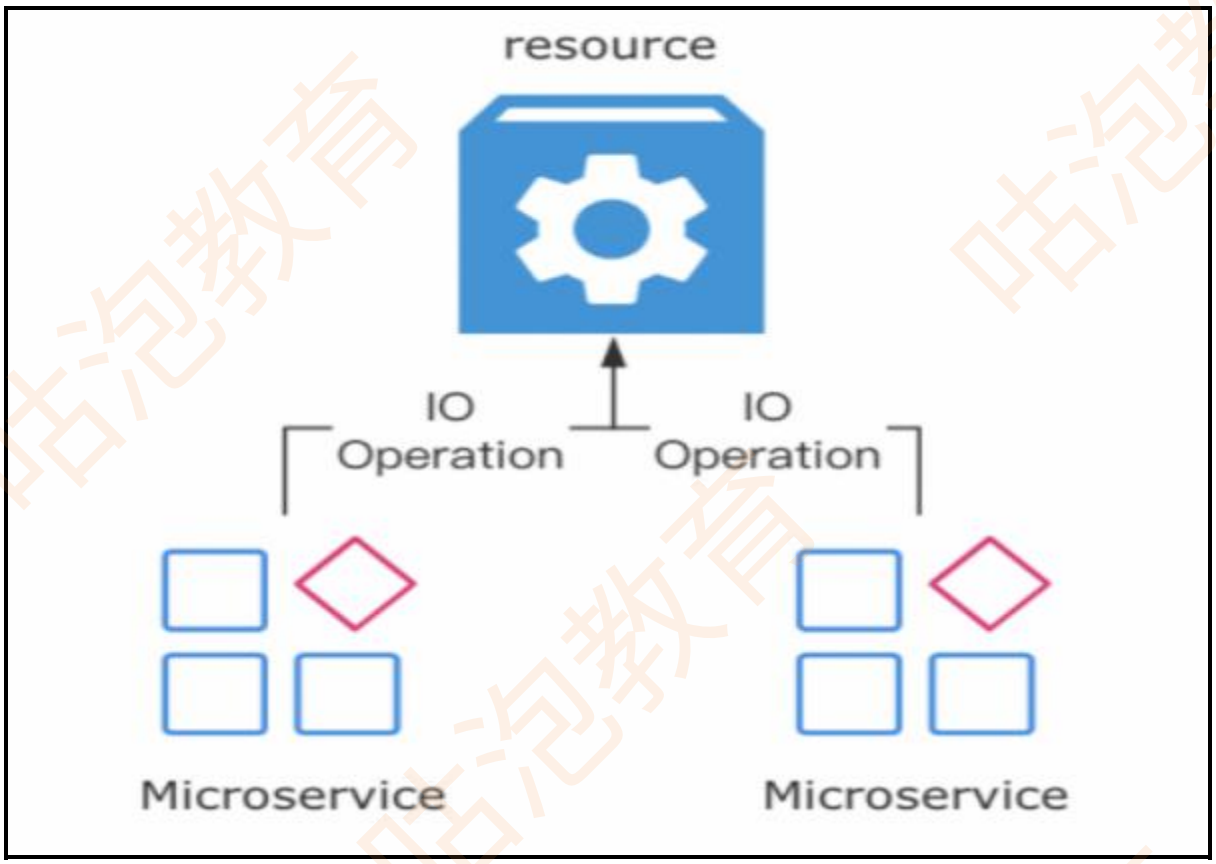
嗯，分布式锁，就是可以用来实现锁的分布性，嗯...

就是可以解决跨进程的应用对于共享资源访问的冲突问题。

可以用Redis来实现分布式锁。

高手

分布式锁，是一种跨进程的跨机器节点的互斥锁，它可以用来保证多机器节点对于共享资源访问的排他性。



我觉得分布式锁和线程锁本质上是一样的，线程锁的生命周期是单进程多线程，分布式锁的声明周期是多进程多机器节点。

在本质上，他们都需要满足锁的几个重要特性：

排他性，也就是说，同一时刻只能有一个节点去访问共享资源。

可重入性，允许一个已经获得锁的进程，在没有释放锁之前再次重新获得锁。

锁的获取、释放的方法

锁的失效机制，避免死锁的问题

所以，我认为，只要能够满足这些特性的技术组件都能够实现分布式锁。

关系型数据库，可以使用唯一约束来实现锁的排他性，

如果要针对某个方法加锁，就可以创建一个表包含方法名称字段，

并且把方法名设置成唯一的约束。

那抢占锁的逻辑就是：往表里面插入一条数据，如果已经有其他的线程获得了某个方法的锁，那这个时候插入数据会失败，从而保证了互斥性。

这种方式虽然简单啊，但是要实现比较完整的分布式锁，还需要考虑重入性、锁失效机制、没抢占到锁的线程要实现阻塞等，就会比较麻烦。

Redis，它里面提供了SETNX命令可以实现锁的排他性，当key不存在就返回1，存在就返回0。然后还可以用expire命令设置锁的失效时间，从而避免死锁问题。

当然有可能存在锁过期了，但是业务逻辑还没执行完的情况。所以这种情况，可以写一个定时任务对指定的key进行续期。

Redisson这个开源组件，就提供了分布式锁的封装实现，并且也内置了一个Watch Dog机制来对key做续期。

我认为Redis里面这种分布式锁设计已经能够解决99%的问题了，当然如果在Redis搭建了高可用集群的情况下出现主从切换导致key失效，这个问题也有可能造成

多个线程抢占到同一个锁资源的情况，所以Redis官方也提供了一个RedLock的解决办法，但是实现会相对复杂一些。

在我看来，分布式锁应该是一个CP模型，而Redis是一个AP模型，所以在集群架构下由于数据的一致性问题导致极端情况下出现多个线程抢占到锁的情况很难避免。

那么基于CP模型又能实现分布式锁特性的组件，我认为可以选择Zookeeper或者etcd，

在数据一致性方面，zookeeper用到了zab协议来保证数据的一致性，etcd用到了raft算法来保证数据一致性。

在锁的互斥方面，zookeeper可以基于有序节点再结合Watch机制实现互斥和唤醒，etcd可以基于Prefix机制和Watch实现互斥和唤醒。

以上就是我对于分布式锁的理解！

面试点评

我认为，回答这个问题的核心本质，还是在技术底层深度理解的基础上的思考。

可以从高手的回答中明显感受到，对于排它锁底层逻辑的理解是很深刻的，同时再技术的广度上也是有足够的积累。

所以在回答的时候，面试官可以去抓到求职者在回答这个问题的时候的技术关键点和技术思维。

我认为，当具备体系化的技术能力的时候，是很容易应对各种面试官的各种刁难的。

灰灰老师解析

首先，我们得讲下什么是锁，所谓锁，就是某些共享的资源，我不希望有并发执行的可能，而是让这些资源串行执行。

比如下单，同一个订单我只能由一个人同时去支付。

所以，锁的核心作用，我有线程在做的时候，其他线程不能做。

面试官：那既然说到锁了，那你怎么去实现一把锁：

灰灰老师：实现锁的必要条件 有个标记 这个标记来决定是否有线程在执行 ， 并且这个标记获取必须是安全的，2这个标记必须是线程可见的

比如厕所的门，如果2个人看到门开着，只能有1个人进入，还有这个门对盲人无效，盲人必须通过其他方式来确认是否有人。

Redis实现分布式锁也满足了上述条件：

第一呢.因为Redis是k.v结构，我们可以把key做为一个标记，比如支付订单111，我们可以把pay:111作为key

第二呢.并且这个key是线程可见的， 因为Redis里面的指令执行是单线程的。

还有就是要保证获取这个key的安全性就可以，由于是单线程指令执行，我们可以用setnx，如果不存在就能设置成功。

或者采用lua脚本来保证多个指令的原子性。

所以在我看来，Redis能实现分布式锁最关键的点在于Redis的命令执行是单线程。

3.面试官：回答还不错，那再问一个问题 Redisson做了什么事情呢?

灰灰老师：Redisson呢 其实是Redis的一个客户端，它对Redis能做的一些事情都进行了封装，用户使用的时候只要去用已经封装的功能就行。

比如分布式锁，Redisson就进行了很好的封装，主要实现了5件事情.

第一.基于Redis的发布与订阅 以及Semaphore实现了锁的等待，就是没拿到锁我可以等待一段时间。

第二.实现了可重入锁，基于Redis的Hash数据结构

第三.为了死锁，给锁添加了过期时间

第四.为了防止业务没有执行完，锁被释放，利用时间轮添加了续期机制

第五.因为Redis是属于ap模型，在发生分区容错的时候，优先保证高可用，所以会牺牲一定的数据一致性。为了防止锁丢失，也提供了联锁的概念。

命令 setnx + expire 分开写

```
if (jedis.setnx(key,lock_value) == 1) { //加锁
    expire（key，100）; //设置过期时间
    try {
```

```
            do something //业务请求
        }catch(){
        }
        finally {
            jedis.del(key); //释放锁
        }
}
```

如果执行完 setnx加锁，正要执行 expire设置过期时间时，进程 crash 掉或者要重启维护了，那这个锁就“长生不老”了，别✂线程永远获取不到锁啦，所以 分布式锁不能这么实现。

setnx + value 值✂过期时间

```
long expires = System.currentTimeMillis() + expireTime; //系统时间+设置✂过期时间String expiresStr = String.valueOf(expires);

// 如果当前锁不存在，返回加锁成功
if (jedis.setnx(key, expiresStr) == 1) {
            return true;
}
// 如果锁已经存在，获取锁✂过期时间
String currentValueStr = jedis.get(key);

// 如果获取到✂过期时间，小于系统当前时间，表示已经过期
if (currentValueStr != null && Long.parseLong(currentValueStr) < System.currentTimeMillis()) {
        // 锁已过期，获取上一个锁✂过期时间，并设置现在锁✂过期时间（不了解redis✂getSet命令小伙伴，可以去官网看下哈）

        String oldValueStr = jedis.getSet(key_resource_id, expiresStr);
        if (oldValueStr != null && oldValueStr.equals(currentValueStr)) {
            // 考虑多线程并发✂情况，只有一个线程✂设置值和当前值相同，它才可以加锁
            return true;
```

```
    }

}
//其他情况，均返回加锁失败
return false;
}
```

笔者看过有开发小伙伴就✂这么实现分布式锁✂，但✂这种方案也有这些缺点：

- 过期时间✂客户端自己生成✂，分布式环境下，每个客户端✂时间必须同步。
- 没有保存持有者✂唯一标识，可能被别✂客户端释放/解锁。
- 锁过期✂时候，并发多个客户端同时请求过来，都执行了jedis.getSet()，最终只能有一个客户端加锁成功，但✂该客户端锁✂过期时间，可能被别✂客户端覆盖。

： set ✂扩展命令（set ex px nx）（注意可能存在✂问题）

```
if (jedis.set(key, lock_value, "NX", "EX", 100s) == 1) { //加锁
    try {
        do something //业务处理
    }catch(){
    }
  finally {
        jedis.del(key); //释放锁
    }
}
```

这个方案可能存在这样✂问题：

- 锁过期释放了，业务还没执行完。
- 锁被别✂线程误删。

set ex px nx + 校验唯一随机值,再删除

```
if (jedis.set(key, uni_request_id, "NX", "EX", 100s) == 1) { //加锁
    try {
        do something //业务处理
    }catch(){
    }
    finally {
        //判断✂不✂当前线程加✂锁,✂才释放
        if (uni_request_id.equals(jedis.get(key))) {
            jedis.del(key); //释放锁
        }
    }
}
```

在这里，判断当前线程加✂锁和释放锁✂不✂一个原子操作。如果调用jedis.del()释放锁✂时候，可能这✂锁已经不属于当前客户端，会解除他人加✂锁。

一般也✂用 lua 脚本代替。lua 脚本如下：

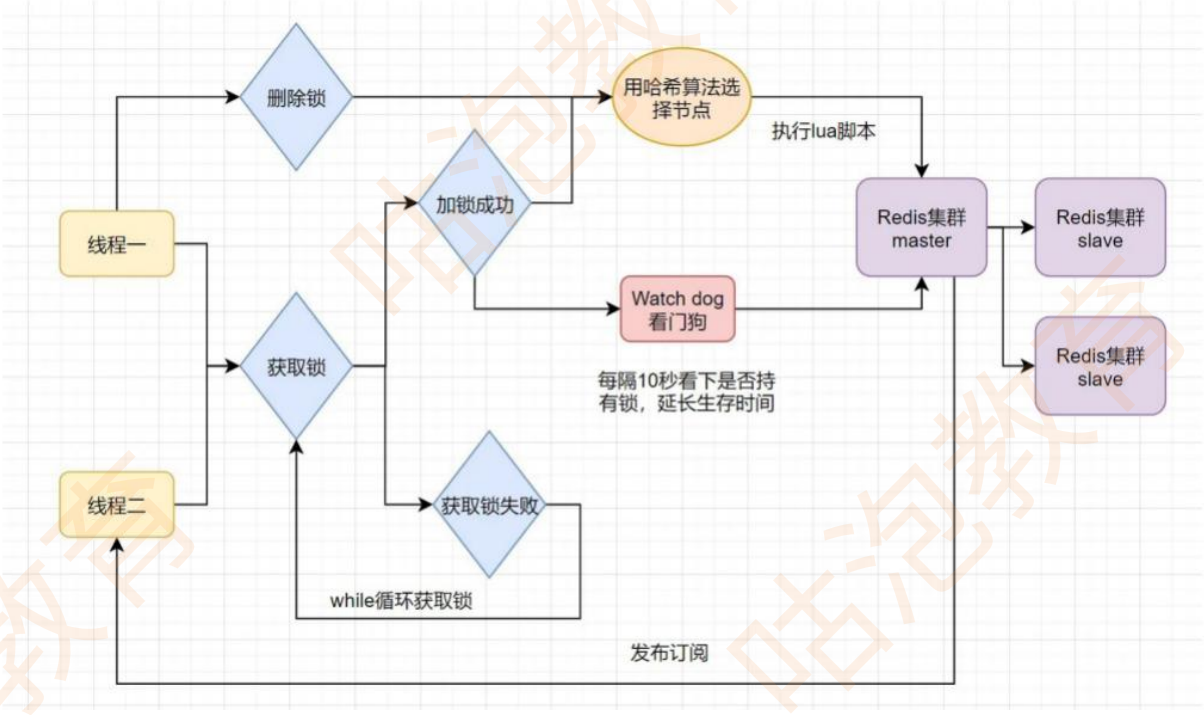
```
if redis.call('get',KEYS[1]) == ARGV[1] then
    return redis.call('del',KEYS[1])
else
    return 0
end;
```

这种方式比较不错了，一般情况下，已经可以使用这种实现方式。但✂存在锁过期释放了，业务还没执行完✂问题（实际上，估算个业务处理✂时间，一般没啥问题了）。

Redisson

分布式锁可能存在锁过期释放，业务没执行完✂问题。有些小伙伴认为，稍微✂锁过期时间设置长一些就可以啦。其实我们设想一下，✂否可以给获得锁✂线程，开启一个定时守护线程，每隔一段时间检查锁✂否还存在，存在则对锁✂过期时间延长，防止锁过期提前释放。

当前开源框架Redisson 就解决了这个分布式锁问题。我们一起来看下Redisson 底层原理✂怎样✂吧：



只要线程一加锁成功，就会启动一个 watch dog看门狗，它✂一个后台线程，会每隔 10 秒检查一下，如果线程 1 还持有锁，那么就会不断✂延长锁 key ✂生存时间。因此，Redisson 就✂使用 Redisson 解决了锁过期释放，业务没执行完问题。

13.11 实现分布式锁，Zookeeper 和 Redis 哪种更好?

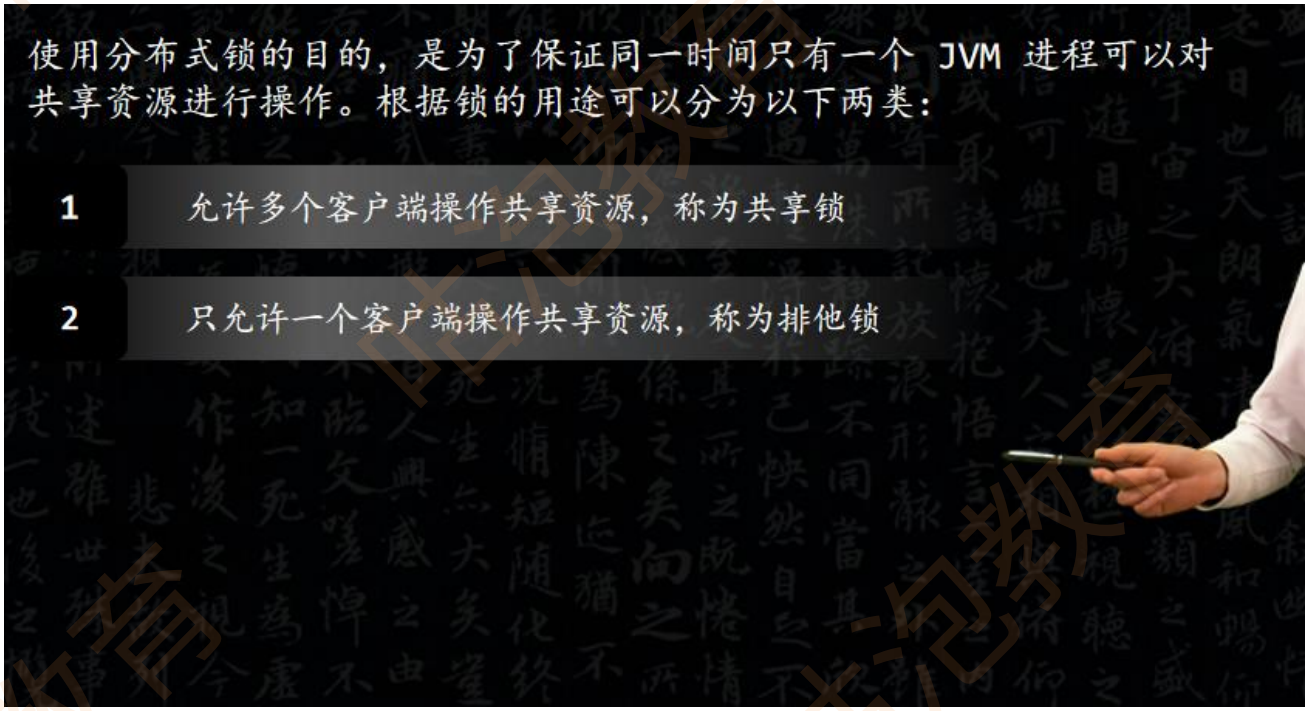
【引言】

一位6年工作经验的小伙伴，在某厂面试时被问到“实现分布式锁，Zookeeper 和 Redis 哪种更好？ “，这其实是一个开放性的问题。并没有标准答案。那今天呢，我给大家分享一下我的理解，希望能够帮助到大家。

另外，我把往期分享的视频全部整理成了1份20字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

背景介绍

使用分布式锁的目的，是为了保证同一时间只有一个 JVM 进程可以对共享资源进行操作。



根据锁的用途可以分为以下两类：

1、允许多个客户端操作共享资源，我们称为共享锁。

这种锁的一般是对共享资源具有幂等性操作的场景，主要是为了避免重复操作共享资源频繁加锁带来的性能开销。

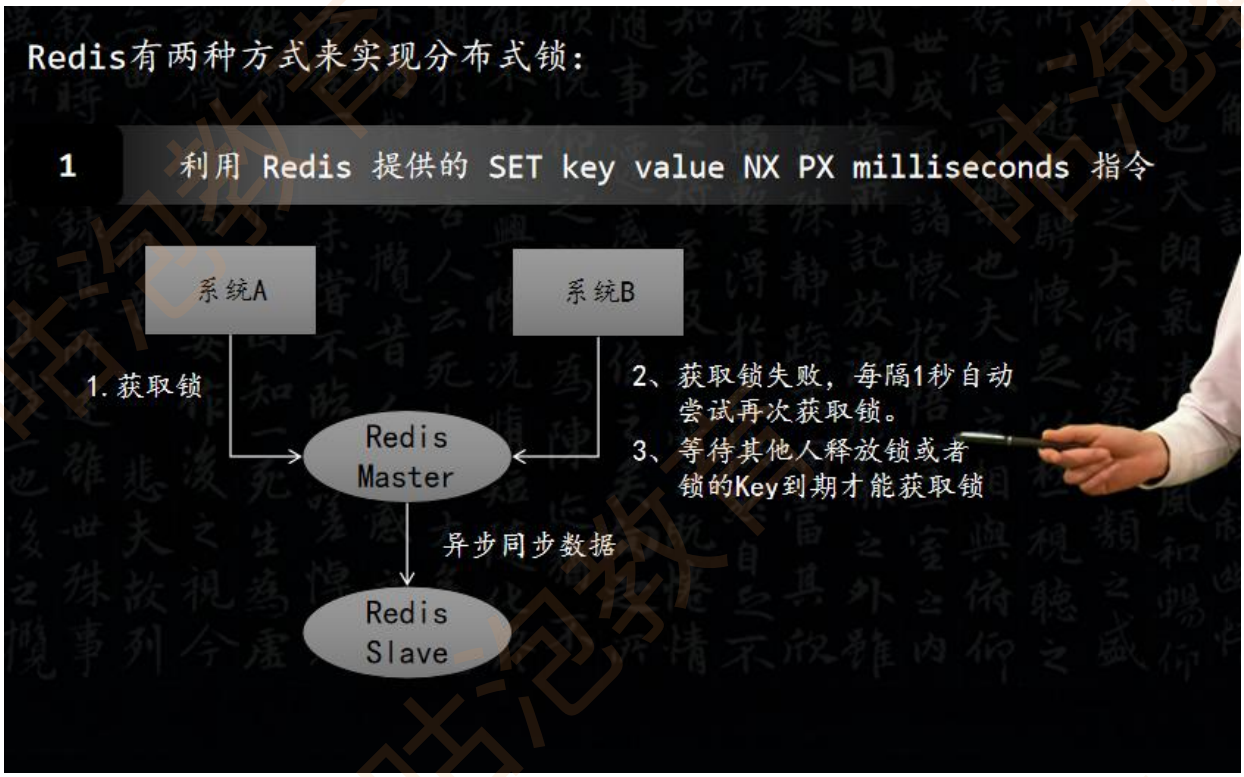
2、只允许一个客户端操作共享资源，我们称为排他锁。这种锁一般是用在对共享资源操作具有非幂等性操作的场景，也就是需要保证在同一时刻只有一个进程或者线程能够访问这个共享资源。

实现方案

目前实现分布式锁最常用的中间件是 Redis 和 Zookeeper：

首先来看Redis的实现方案，它可以通过两种方式来实现

1. 利用 Redis 提供的 SET key value NX PX milliseconds 指令，



这个指令是设置一个Key-Value，如果 Key 已经存在，则返回 0，否则返回 1，我们基于这个返回值来判断锁的占用情况从而实现分布式锁。

2. 基于 Redission 客户端来实现分布式锁，Redisson 提供了分布式锁的封装方法，我们只需要调用 api 中的 lock（）和 unlock()方法。它帮我们封装锁实现的细节和复杂度。

Redission 所有指令都通过 Lua 脚本执行并支持 Lua 脚本原子性执行， Redission 中有一个 WatchDog 的概念，翻译过来就是看门狗，它会在你获取锁之后，每隔 10 秒帮你把 Key 的超时时间设为 30s，就算一直持有锁也不会出现Key 过期了。 “看门狗”的逻辑保证了没有死锁发生。

然后，基于 ZooKeeper 实现分布式锁的落地方案

Zookeeper 实现分布式锁的方法比较多。我推荐使用有序节点来实现，来看这个图，

首先第1步，每个线程或进程在 Zookeeper 上的/lock 目录下创建一个临时有序的节点表示去抢占锁，所有创建的节点会按照先后顺序生成一个带有序编号的节点。

第2步、线程创建节点后，获取/lock 节点下的所有子节点，判断当前线程创建的节点是否是所有的节点的序号最小的。

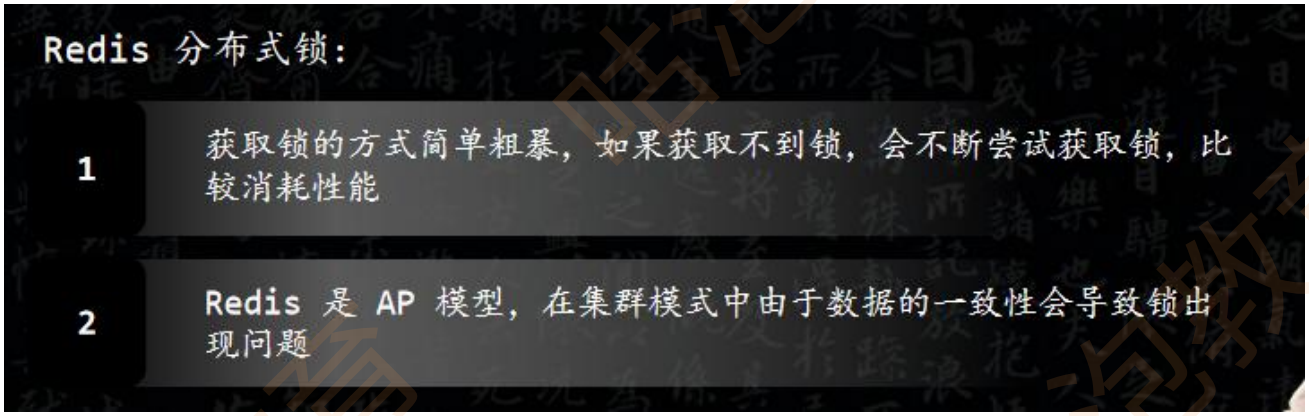
第3步、如果当前线程创建的节点是所有节点序号最小的节点，则认为获取锁成功。

第4步、如果当前线程创建的节点不是所有节点序号最小的节点，则对节点序号的前一个节点添加一个事件监听，当前一个被监听的节点释放锁之后，触发回调通知，从而再次去尝试抢占锁。

两者对比

接下来，咱们来对比一下这两种方案：

对于 Redis 的分布式锁而言，它也有以下缺点：

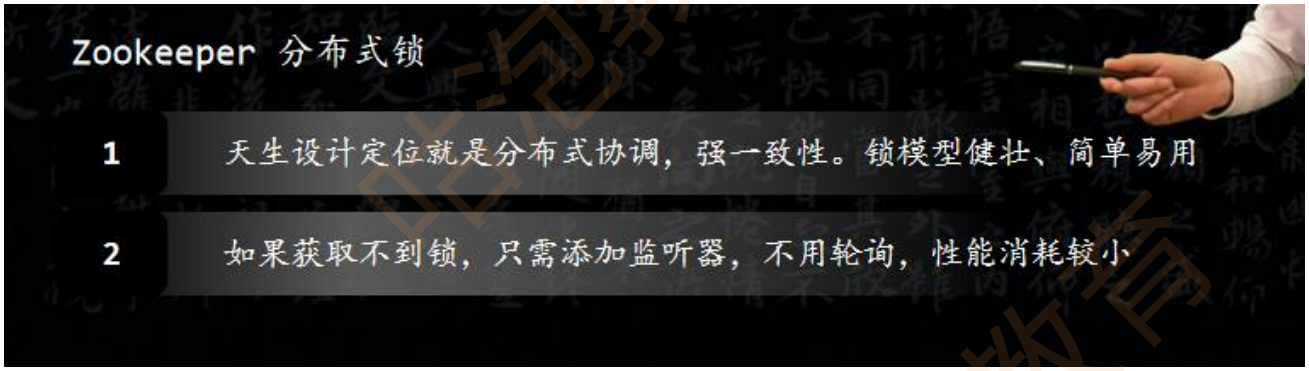


1、它获取锁的方式简单粗暴，如果获取不到锁，会不断尝试获取锁，比较消耗性能。

2、Redis 是 AP 模型，在集群模式中由于数据的一致性会导致锁出现问题，即便使用 Redlock 算法来实现，在某些复杂场景下，也无法保证其实现 100%的可靠性。

不过在实际开发中使用 Redis 实现分布式锁还是比较常见，而且大部分场情况下不会遇到”极端复杂的场景“，更重要的是 Redis 性能很高，在高并发场景中比较合适。

对于Zookeeper 分布式锁而言:



1、Zookeeper 天生设计定位就是分布式协调，强一致性。锁的模型健壮、简单易用、适合做分布式锁。

2、如果获取不到锁，只需要添加一个监听器就可以了，不用一直轮询，性能消耗较小。

【结尾】

如果要在两者之间做选择，就我个人而言的话，比较推崇 ZK 实现的锁，因为对于分布式锁而言，它应该符合 CP 模型，但是 Redis 是 AP 模型，所以在这个点上，Zookeeper 会更加合适。

最后，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

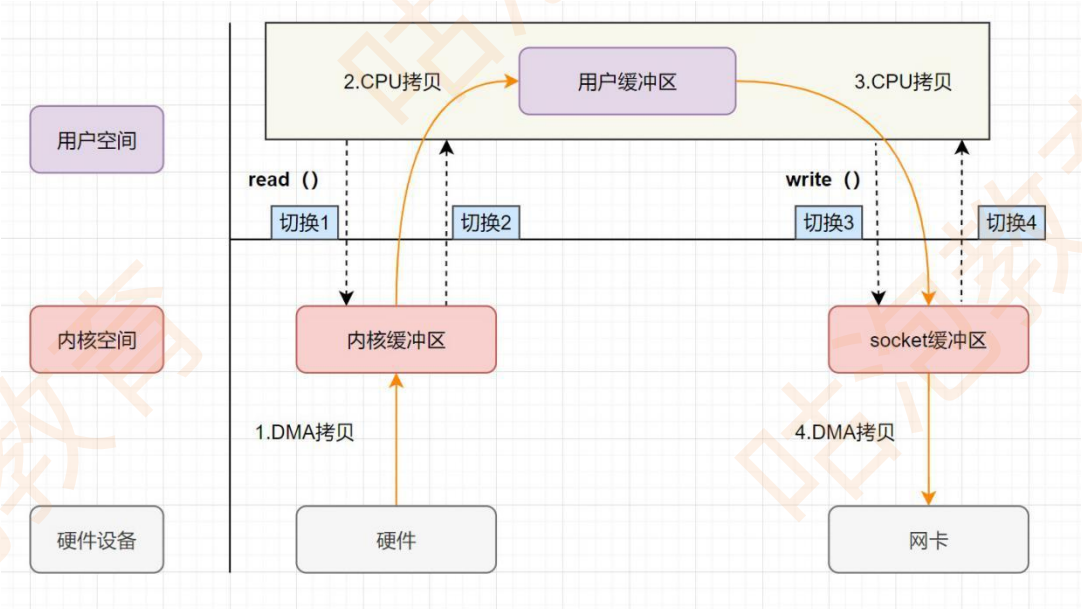
13.12 零拷贝

零拷贝就✂不需要将数据从一个存储区域复制到另一个存储区域。它✂指在传统 IO 模型中，指 CPU 拷贝✂次数为 0。它✂ IO ✂优化方案

- 传统 IO 流程
- 零拷贝实现之 mmap+write
- 零拷贝实现之 sendfile
- 零拷贝实现之带有 DMA 收集拷贝功能✂ sendfile

传统 IO 流程

流程图如下：



- 用户应用进程调用 read 函数，向操作系统发起 IO 调用，**上下文从用户态转为内核态（切换 1）**
- DMA 控制器✂数据从磁盘中，读取到内核缓冲区。
- CPU ✂内核缓冲区数据，拷贝到用户应用缓冲区，**上下文从内核态转为用户态（切换 2）**，read 函数返回
- 用户应用进程通过 write 函数，发起 IO 调用，**上下文从用户态转为内核态（切换 3）**
- CPU 将应用缓冲区中✂数据，拷贝到 socket 缓冲区
- DMA 控制器✂数据从 socket 缓冲区，拷贝到网卡设备，**上下文从内核态切换回用户态（切换 4）**，write 函数返回

从流程图可以看出，传统 IO ✂读写流程，包括了 4 次上下文切换（4 次用户态和内核态✂切换），4 次数据拷贝（**两次 CPU 拷贝以及两次✂ DMA 拷贝**）。

mmap+write 实现✂零拷贝

mmap ✂函数原型如下：

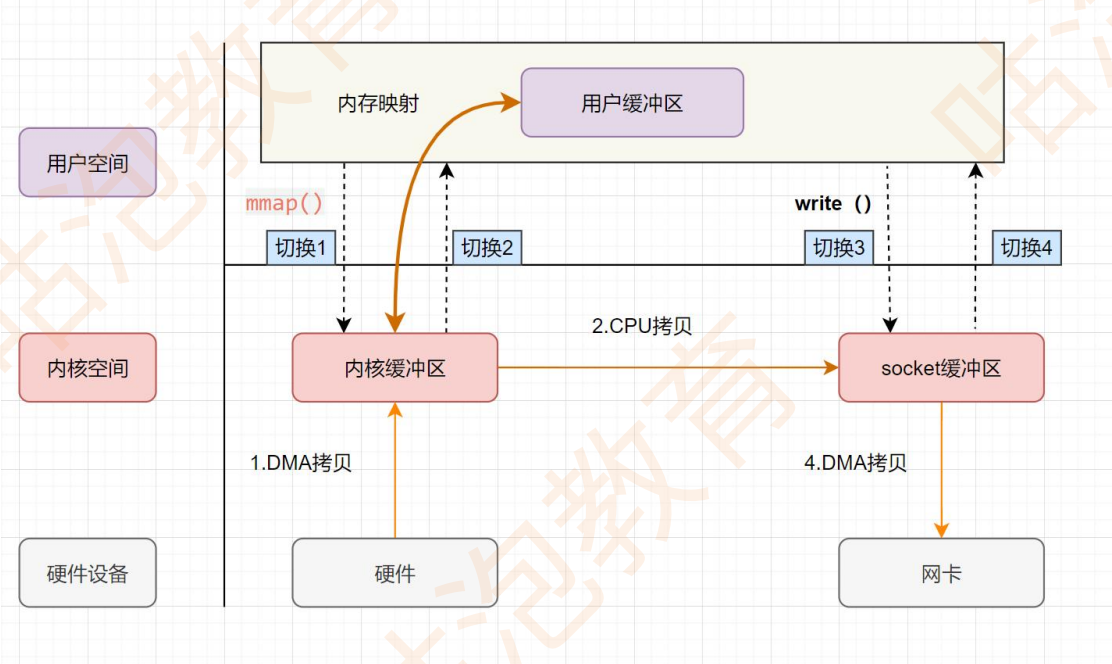
```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

- addr：指定映射✂虚拟内存地址
- length：映射✂长度
- prot：映射内存✂保护模式
- flags：指定映射✂类型
- fd:进行映射✂文件句柄

- offset:文件偏移量

mmap 使用了**虚拟内存**，可以✂内核空间和用户空间✂虚拟地址映射到同一个物理地址，从而减少数据拷贝次数！

mmap+write实现✂零拷贝流程如下：



- 用户进程通过 mmap方法向操作系统内核发起 IO 调用，**上下文从用户态切换为内核态。**
- CPU 利用DMA 控制器，✂数据从硬盘中拷贝到内核缓冲区。
- **上下文从内核态切换回用户态**，mmap 方法返回。
- 用户进程通过 write方法向操作系统内核发起 IO 调用，**上下文从用户态切换为内核态。**
- CPU 将内核缓冲区✂数据拷贝到✂ socket 缓冲区。
- CPU 利用DMA 控制器，✂数据从 socket 缓冲区拷贝到网卡，**上下文从内核态切换回用户态**，write 调用返回。

可以发现，mmap+write实现✂零拷贝，I/O 发生了 **4** 次用户空间与内核空间✂上下文切换，以及 3 次数据拷贝。其中 3 次数据拷贝中，包括了 **2 次 DMA 拷贝和 1 次 CPU 拷贝。**

mmap✂将读缓冲区✂地址和用户缓冲区✂地址进行映射，内核缓冲区和应用缓冲区共享，所以节省了一次 CPU 拷贝〃并且用户进程内存✂**虚拟**✂，只✂**映射**到内核✂读缓冲区，可以节省一半✂内存空间。

sendfile 实现✂零拷贝

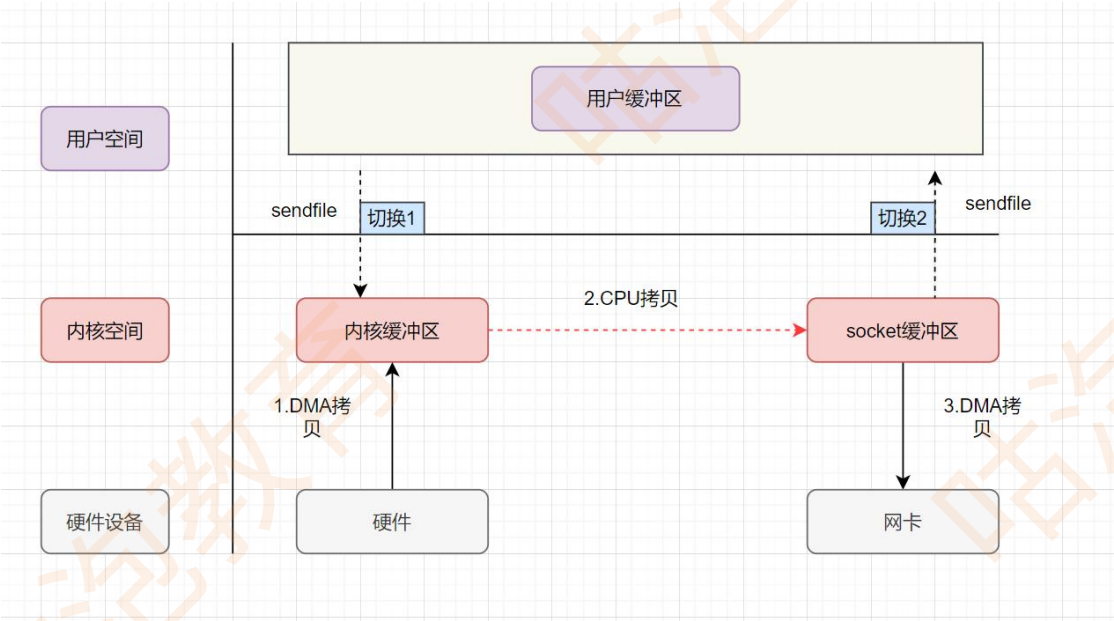
sendfile✂ Linux2.1 内核版本后引入✂一个系统调用函数，API 如下：

```
ssize_t sendfile(int out_fd, int in_fd, off_t *offset, size_t count);
```

- out_fd:为待写入内容✂文件描述符，一个 socket 描述符。，
- in_fd:为待读出内容✂文件描述符，必须✂真实✂文件，不能✂ socket 和管道。
- offset：指定从读入文件✂哪个位置开始读，如果为 NULL，表示文件✂默认起始位置。
- count：指定在 fdout 和 fdin 之间传输✂字节数。

sendfile 表示在两个文件描述符之间传输数据，它✂在**操作系统内核**中操作✂，**避免了数据从内核缓冲区和用户缓冲区之间✂拷贝操作**，因此可以使用它来实现零拷贝。

sendfile 实现✂零拷贝流程如下：



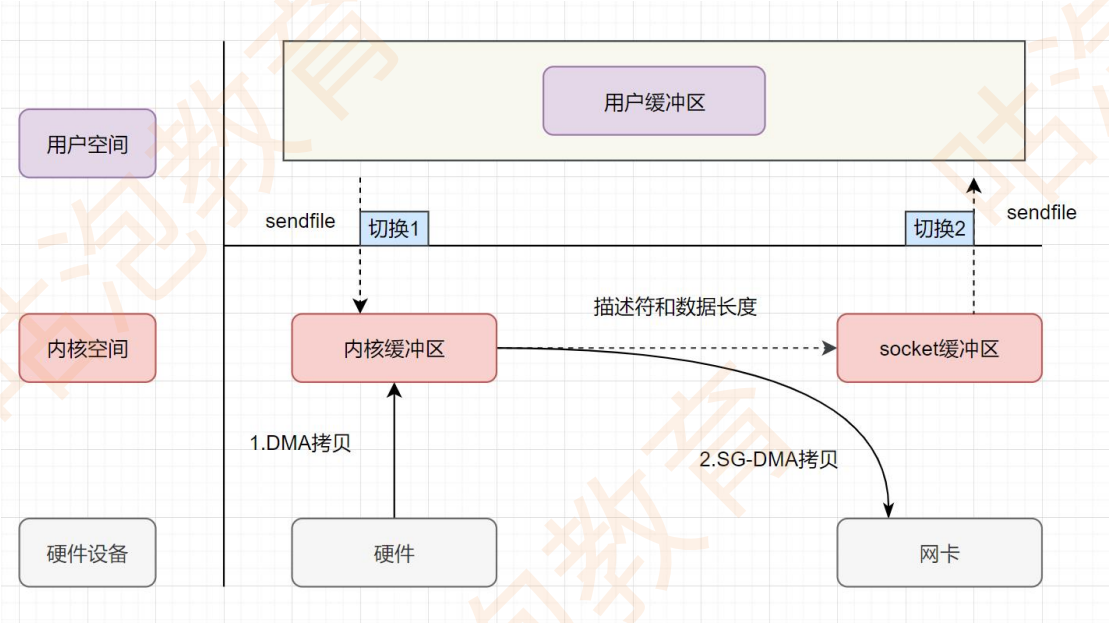
sendfile 实现✂零拷贝

1. 用户进程发起 sendfile 系统调用，**上下文（切换 1）从用户态转向内核态**
2. DMA 控制器，✂数据从硬盘中拷贝到内核缓冲区。
3. CPU 将读缓冲区中数据拷贝到 socket 缓冲区
4. DMA 控制器，异步✂数据从 socket 缓冲区拷贝到网卡，
5. **上下文（切换 2）从内核态切换回用户态**，sendfile 调用返回。

可以发现，sendfile实现✂零拷贝，I/O 发生了 **2** 次用户空间与内核空间✂上下文切换，以及 3 次数据拷贝。其中 3 次数据拷贝中，包括了 **2 次 DMA 拷贝和 1 次 CPU 拷贝。**那能不能✂ CPU 拷贝✂次数减少到 0 次呢？有✂，✂带有DMA收集拷贝功能✂ sendfile！

sendfile+DMA scatter/gather 实现✂零拷贝

linux 2.4 版本之后，对 sendfile做了优化升级，引入 SG-DMA 技术，其实就✂对DMA 拷贝加入了 scatter/gather操作，它可以直⓾从内核空间缓冲区中将数据读取到网卡。使用这个特点搞零拷贝，✂还可以多省去**一次 CPU 拷贝。**sendfile＋DMA scatter/gather 实现✂零拷贝流程如下：



1. 用户进程发起 sendfile 系统调用，**上下文（切换 1）从用户态转向内核态**
2. DMA 控制器，✂数据从硬盘中拷贝到内核缓冲区。
3. CPU ✂内核缓冲区中✂**文件描述符信息**（包括内核缓冲区✂内存地址和偏移量）发送到 socket 缓冲区
4. DMA 控制器根据文件描述符信息，直⓾✂数据从内核缓冲区拷贝到网卡
5. **上下文（切换 2）从内核态切换回用户态**，sendfile 调用返回。

可以发现，sendfile+DMA scatter/gather实现✂零拷贝，I/O 发生了 **2** 次用户空

间与内核空间✂上下文切换，以及 2 次数据拷贝。其中 2 次数据拷贝都✂包

DMA 拷贝。这就✂真正✂ **零拷贝（Zero-copy)** 技术，全程都没有通过 CPU 来搬运数据，所有✂数据都✂通过 DMA 来进行传输✂。

13.13 synchronized

synchronized ✂ Java 中✂关键字，✂一种同步锁。synchronized 关键字可以作用于方法或者代码块。

一般面试时。可以这么回答：

- 反编译后，monitorenter、monitorexit、ACC_SYNCHRONIZED
- monitor 监视器
- Java Monitor ✂工作机理
- 对象与 monitor 关联

monitorenter、monitorexit、ACC_SYNCHRONIZED

如果 **synchronized** 作用于**代码块**，反编译可以看到两个指令：monitorenter、monitorexit，JVM 使用 monitorenter和 monitorexit两个指令实现同步；如果作用 synchronized 作用于**方法**,反编译可以看到 ACCSYNCHRONIZED标记，JVM 通过在方法访问标识符(flags)中加入 ACCSYNCHRONIZED来实现同步功能。

- 同步代码块✂通过 monitorenter和 monitorexit来实现，当线程执行到 monitorenter ✂时候要先获得 monitor 锁，才能执行后面✂方法。当线程执行到 monitorexit ✂时候则要释放锁。
- 同步方法✂通过中设置 ACCSYNCHRONIZED 标志来实现，当线程执行有 ACCSYNCHRONI 标志✂方法，需要获得 monitor 锁。每个对象都与一个 monitor 相关联，线程可以占有或者释放 monitor。

monitor 监视器

monitor ✂什么呢？操作系统✂管程（monitors）✂概念原理，ObjectMonitor ✂它✂原理实现。

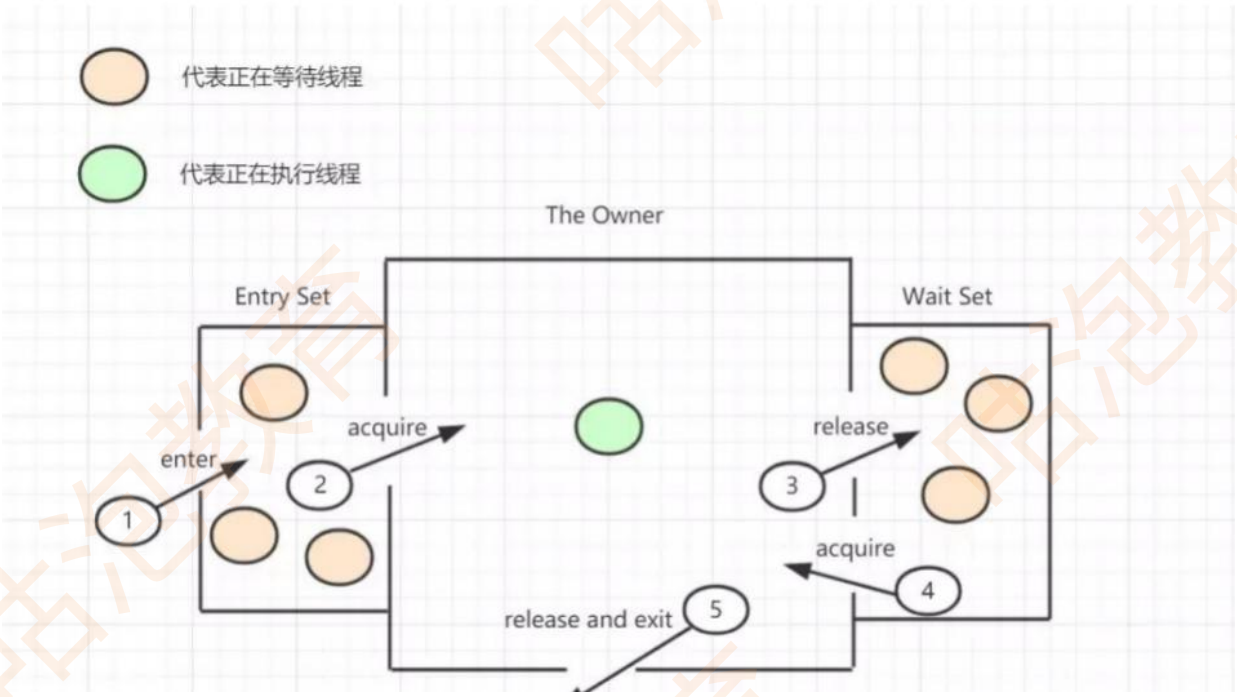
在 Java 虚拟机（HotSpot）中，Monitor（管程）✂由 ObjectMonitor 实现✂，其主要数据结构如下：

```
ObjectMonitor() {
    _header       = NULL;
    _count        = 0; // 记录个数
    _waiters      = 0,
    _recursions   = 0;
    _object       = NULL;
    _owner        = NULL;
```

```
    _WaitSet       = NULL;  // 处于 wait 状态✂线程，会被加入到_WaitSet
    _WaitSetLock   = 0 ;
    _Responsible   = NULL ;
    _succ          = NULL ;
    _cxq           = NULL ;
    FreeNext       = NULL ;
    _EntryList     = NULL ; // 处于等待锁block状态✂线程，会被加入到该列表
    _SpinFreq      = 0 ;
    _SpinClock     = 0 ;
    OwnerIsThread = 0 ;
  }
```

ObjectMonitor 中几个关键字段✂含义：

Java Monitor ✂工作机理



- 想要获取 monitor ✂线程,首先会进入_EntryList 队列。
- 当某个线程获取到对象✂ monitor 后,进入Owner 区域，设置为当前线程,同时计数器 count 加 1。
- 如果线程调用了 wait()方法，则会进入 WaitSet 队列。它会释放 monitor 锁，✂将 owner 赋值为 null,count 自减 1,进入WaitSet 队列阻塞等待。
- 如果其他线程调用 notify() / notifyAll() ，会唤醒 WaitSet 中✂某个线程，该线程再次尝试获取 monitor 锁，成功✂进入 Owner 区域。
- 同步方法执行完毕了，线程退出临界区，会将 monitor ✂ owner 设为null，并释放监视锁。

对象与 monitor 关联

- 在 HotSpot 虚拟机中,对象在内存中存储✂布局可以分为 3 块区域：**对象头（Header），实例数据（Instance Data）和对象填充（Padding）。**
- 对象头主要包括两部分数据：**Mark Word（标记字段）、ClassPointer（类型指针）。**

Mark Word ✂用于存储对象自身✂运行时数据，如哈希码（HashCode）、GC 分代年龄、锁状态标志、线程持有✂锁、偏向线程 ID、偏向时间戳等。

✂重量级锁，也就✂说Synchronized ✂对象锁，Mark Word 锁标识位为 10，其中指针指向✂

✂ Monitor 对象✂起始地址。

13.14 分布式 id 生成方案有哪些？什么✂雪花算法?

分布式 id 生成方案主要有：

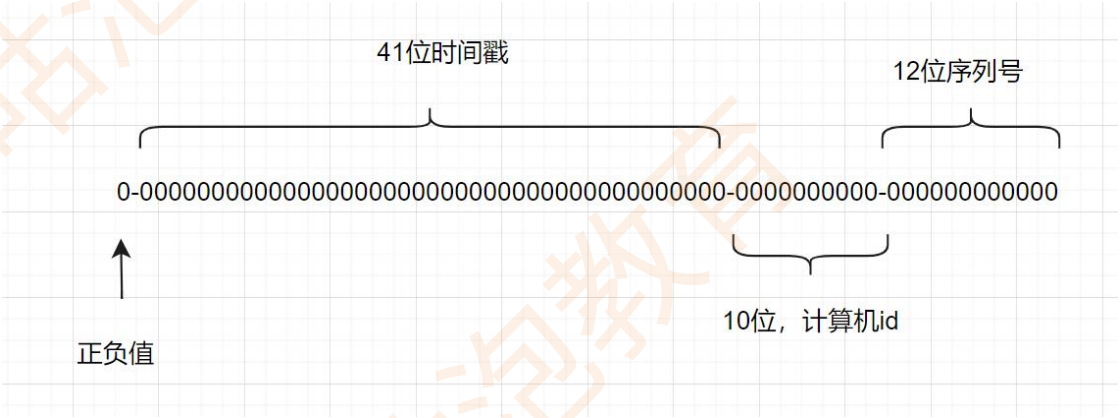
- UUID
- 数据库自增 ID
- 基于雪花算法（Snowflake）实现
- 百度 （Uidgenerator）
- 美团（Leaf）

雪花算法✂一种生成分布式全局唯一 ID ✂算法，生成✂ ID 称为Snowflake IDs。这种算法由 Twitter 创✂，并用于推文✂ ID。

什么✂**雪花算法**?

一个 Snowflake ID 有 64 位元。

- 第 1 位：Java 中 long ✂最高位✂符号位代表正负，正数✂ 0，负数✂1，一般生成 ID 都为正数，所以默认为 0。
- ⑩下来前 41 位✂时间戳，表示了自选定✂时期以来✂毫秒数。
- ⑩下来✂ 10 位代表计算机 ID，防止冲突。
- 其余 12 位代表每台机器上生成 ID ✂序列号，这允许在同一毫秒内创✂多个Snowflake ID。



雪花算法一般用来实现全局唯一的业务主键，解决分库分表之后主键id的唯一性问题。

所以单纯就全局唯一性质来说，有很多的实现方式，比如

UUID

Redis的原子递增

数据库全局表的自增id

等等

但是在实际应用中，还需要满足有序递增、高性能、带时间戳等。

而雪花算法就是一个比较符合这类特征的全局唯一算法，在美团的Leaf组件中也有用到。

它是由一个64位的long类型数字组成，分为四个部分。

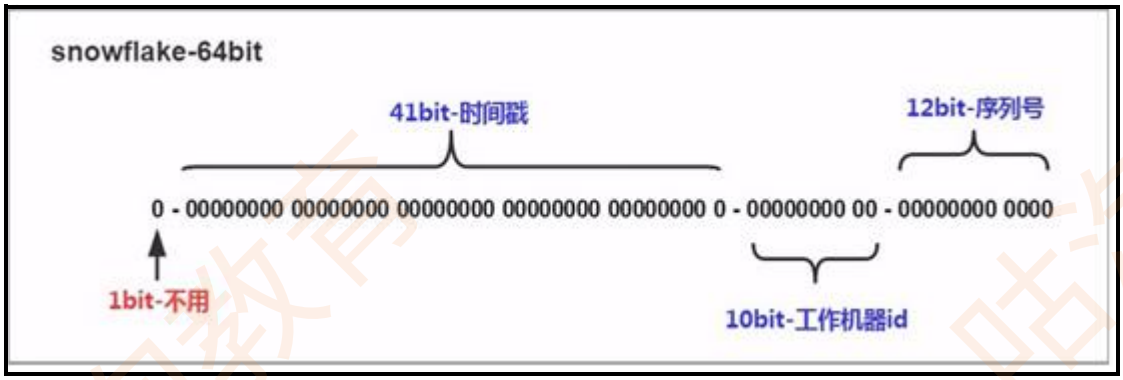
第一部分，用1个bit表示符号位，一般情况下是0

第二部分，用41个bit来表示时间戳，使用系统时间的毫秒数

第三部分，用10个bit来记录工作机器id，这样就可以保证在多个服务器上生成的id的唯一性。

如果存在跨机房部署，我们还可以把它分成两个5bit，前面5个bit可以表示机房id，后面5个bit可以表示机器id。

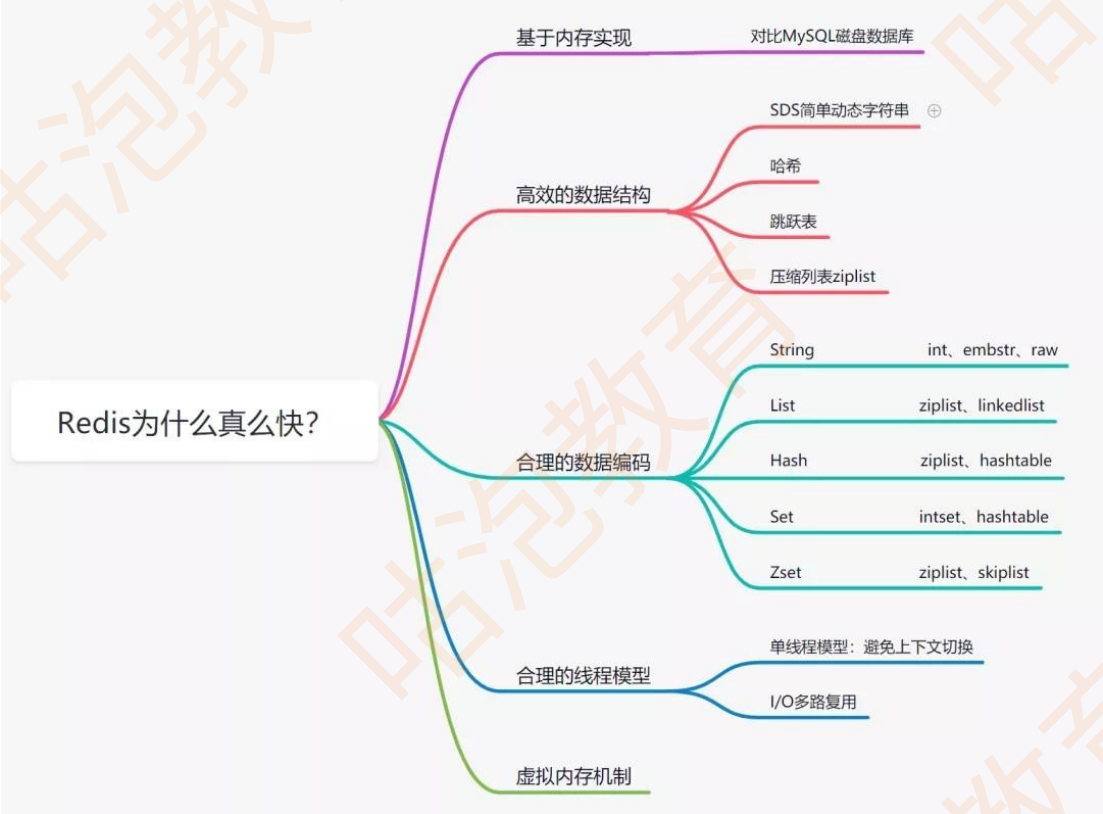
第四个部分，用12个bit表示序列号，表示一个递增序列，用来记录同毫秒内产生的不同id



雪花算法，就是根据这四个部分的规则，生成对应的bit位数据，然后组装在一起形成一个全局唯一id。

字节跳动面试（14）

14.1 说说 Redis 为什么快



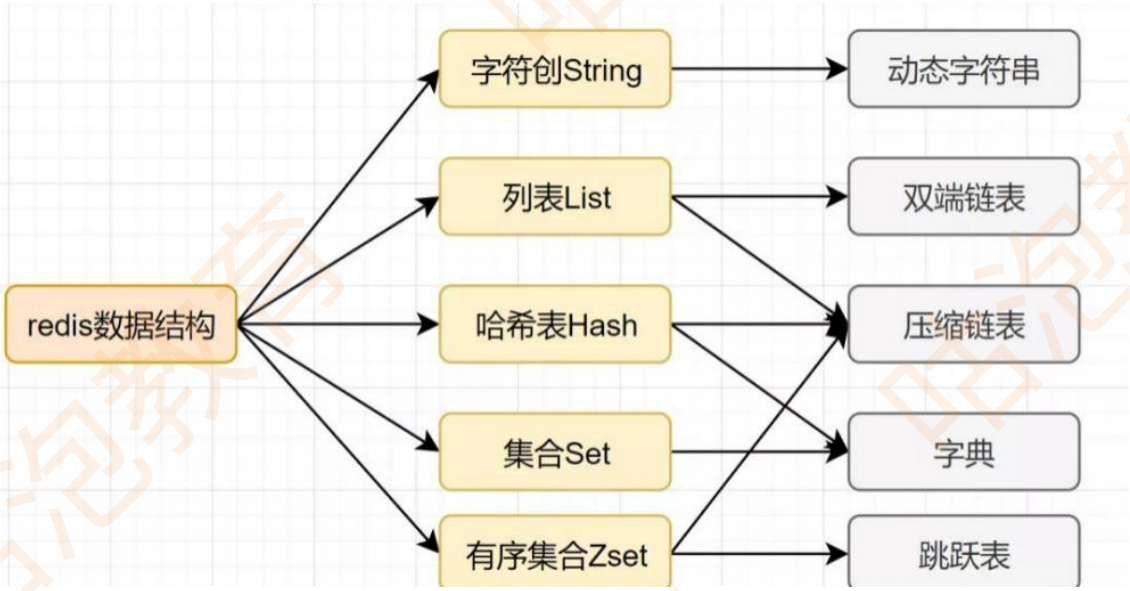
14.1.1 基于内存存储实现

内存读写✂比在磁盘快很多✂，Redis 基于内存存储实现✂数据库，相对于数据存在磁盘✂ MySQL 数据库，省去磁盘 I/O ✂✲耗。

14.1.2 高效✂数据结构

Mysql 索引为了提高效率，选择了 B+树✂数据结构。其实合理✂数据结构，就✂可以

让你✂应用/程序更快。先看下 Redis ✂数据结构&内部编码图：



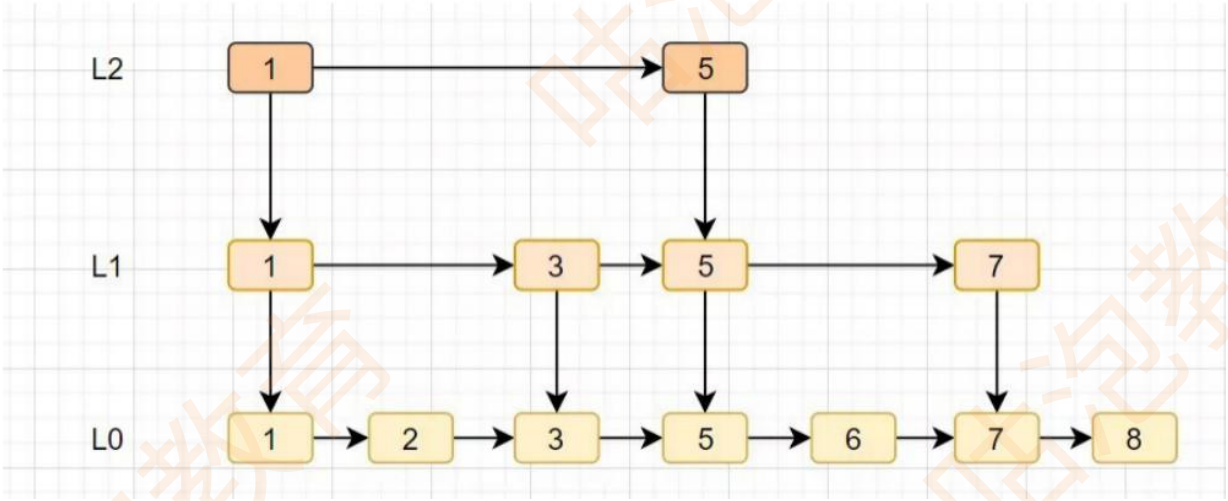
14.1.2.1 SDS 简单动态字符串

- 字符串长度处理：Redis 获取字符串长度，时间复杂度为 O(1)，而 C 语言中，需要从头开始遍历，复杂度为 O（n）；
- 空间预分配：字符串修改越频繁✂话，内存分配越频繁，就会✲耗性能，而 SDS 修改和空间扩充，会额外分配未使用✂空间，减少性能损耗。
- 惰性空间释放：SDS 缩短时，不✂回收多余✂内存空间，而✂ free 记录下多余✂空间，后续有变更，直⑩使用 free 中记录✂空间，减少分配。

14.1.2.2 字典

Redis 作为 K-V 型内存数据库，所有✂键值就✂用字典来存储。字典就✂哈希表，比如 HashMap，通过key 就可以直⑩获取到对应✂ value。而哈希表✂ 特性，在 O（1）时间复杂度就可以获得对应✂值。

14.1.2.3 跳跃表



- 跳跃表✂ Redis 特有✂数据结构，就✂在链表✂基础上，增加多级索引提升查找效率。
- 跳跃表支持平均 O（logN）,最坏 O（N）复杂度✂节点查找，还可以通过顺序性操作批量处理节点。

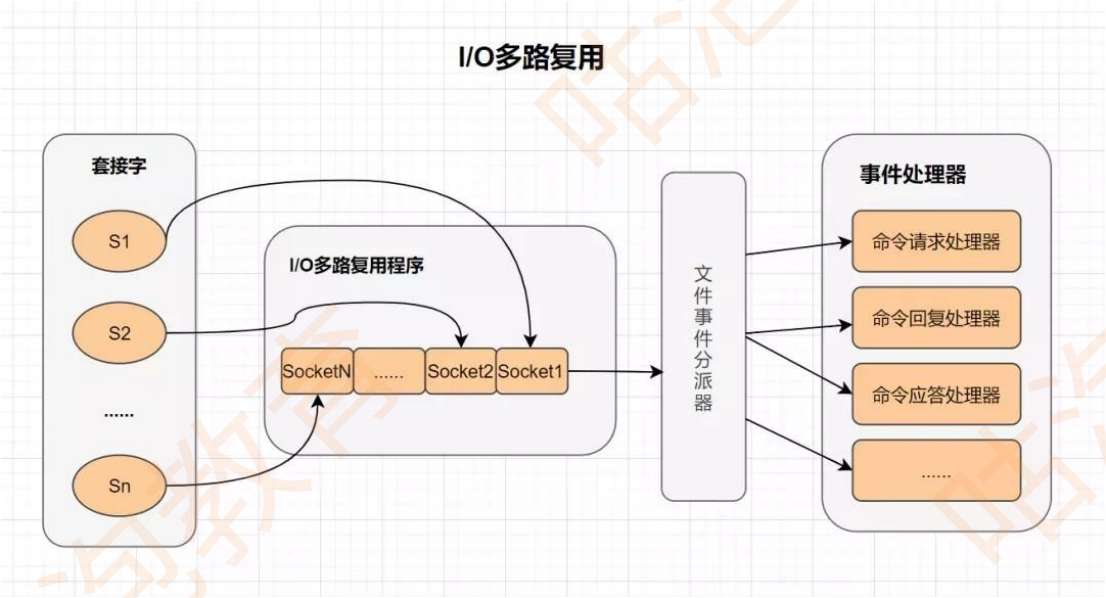
14.1.3 合理✂数据编码

Redis 支持多种数据类型，每种基本类型，可能对多种数据结构。什么时候,使用什么样数据结构，使用什么样编码，✂ redis 设计者总结优化✂结果。

- String：如果存储数字✂话，✂用 int 类型✂编码;如果存储非数字，小于等于 39 字节✂字符串，✂ embstr；大于 39 个字节，则✂ raw 编码。
- List：如果列表✂元素个数小于 512 个，列表每个元素✂值都小于 64 字节（默认），使用 ziplist 编码，否则使用 linkedlist 编码
- Hash：哈希类型元素个数小于 512 个，所有值小于 64 字节✂话，使用 ziplist 编码,否则使用 hashtable 编码。
- Set：如果集合中✂元素都✂整数且元素个数小于 512 个，使用 intset 编码，否则使用 hashtable 编码。
- Zset：当有序集合✂元素个数小于 128 个，每个元素✂值小于 64 字节时，使用ziplist 编码，否则使用 skiplist（跳跃表）编码

14.1.4 合理✂线程模型

I/O 多路复用



多路 I/O 复用技术可以让单个线程高效✂处理多个连⑩请求，而 Redis 使用用 epoll 作为 I/O 多路复用技术✂实现。并且，Redis 自身✂事件处理模型将 epoll 中✂连⑩、读写、关闭都转换为事件，不在网络 I/O 上浪费过多✂时间。

TOM老师解析

【引言】

Redis是单线程还是多线程？这个问题，把一个5年开发经验的程序员给问懵了。可能很多小伙伴平时都听说过，Redis就是单线程的。但是我们学习Redis命令的时候，有经常有设置异步，比如bgsave命令，它的作用是在后台异步持久化数据到磁盘，那既然是异步，肯定需要由线程来完成。

另外，往期面试题解析中配套的文档我已经准备好，想获得的可以在我的煮叶简介中找到。

接下来，我们来介绍，Redis到底是单线程还是多线程？。

1、官方解释

其实，Redis 确实是单线程的。通常说的单线程，主要是指 Redis 对外提供的键值存储服务的主要流程是单线程，也就是网络IO和数据读写是由单个线程来完成的。除此外 Redis 的其他功能，比如持久化、 异步删除、集群数据同步等，是由额外的线程来执行的。这样做的好处是可以 。因此，严格地说 Redis 并不是全面单线程。

在Redis官网页做出了解释，它是这样描述的：

Redis is single threaded. How can I exploit multiple CPU / cores?It’ s not very frequent that CPU becomes your bottleneck with Redis, as usually Redis is either memory or network

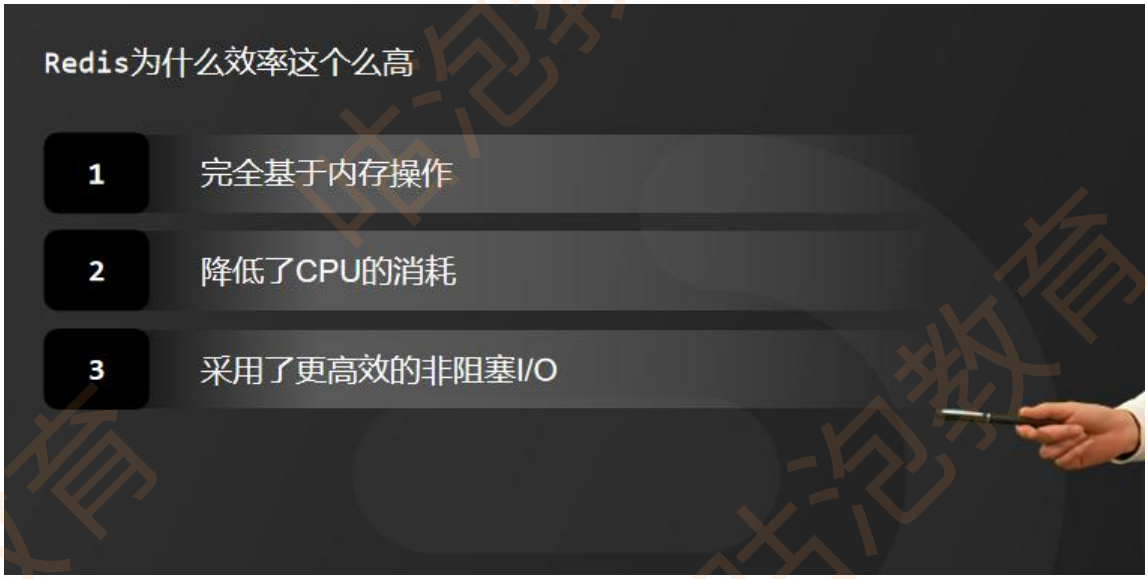
bound. For instance, using pipelining Redis running on an average Linux system can deliver even 1 million requests per second, so if your application mainly uses O(N) or O(log(N)) commands, it is hardly going to use too much CPU.However, to maximize CPU usage you can start multiple instances of Redis in the same box and treat them as different servers. At some point a single box may not be enough anyway, so if you want to use multiple CPUs you can start thinking of some way to shard earlier.You can find more information about using multiple Redis instances in the Partitioning page.However with Redis 4.0 we started to make Redis more threaded. For now this is limited to deleting objects in the background, and to blocking commands implemented via Redis modules. For future releases, the plan is to make Redis more and more threaded.

大致意思是：

由于Redis是基于内存的操作，查找和操作的时间复杂度都是O(1)，因此CPU并不是Redis的瓶颈。 Redis的瓶颈很可能是机器内存或网络带宽的大小。 既然单线程易于实现并且CPU不会成为瓶颈，那采用单线程解决方案是合乎逻辑的了。

2、高性能原因

既然Redis设计为单线程，为什么效率这个么高呢？我总结以下三个原因：



1、完全基于内存操作

Redis中绝大部分请求是纯粹的内存操作，非常快速。数据存在内存中，类似于HashMap的数据结构，对数据操作也简单。

2、降低了CPU的消耗

采用单线程，避免了不必要的上下文切换和竞争条件，也不存在多进程或者多线程导致的切换而消耗 CPU，不用去考虑各种锁的问题，不存在加锁释放锁操作，没有因为可能出现死锁而导致的

性能消耗。

3、采用了更高效的非阻塞I/O

Redis采用epoll做为I/O多路复用技术的实现，再加上Redis自身的事件处理模型将epoll中的连接，读写，关闭都转换为了事件，不在I/O上浪费过多的时间，使得Redis在网络 IO 操作中能并发处理大量的客户端请求，实现了高吞吐率。

【结尾】

以上，就是我对Redis单线程设计以及Redis为什么这么高效的分析。在目前最新的Redis版本中，IO 多线程处理模式默认是不开启的，需要去配置文件中开启并配置线程数。

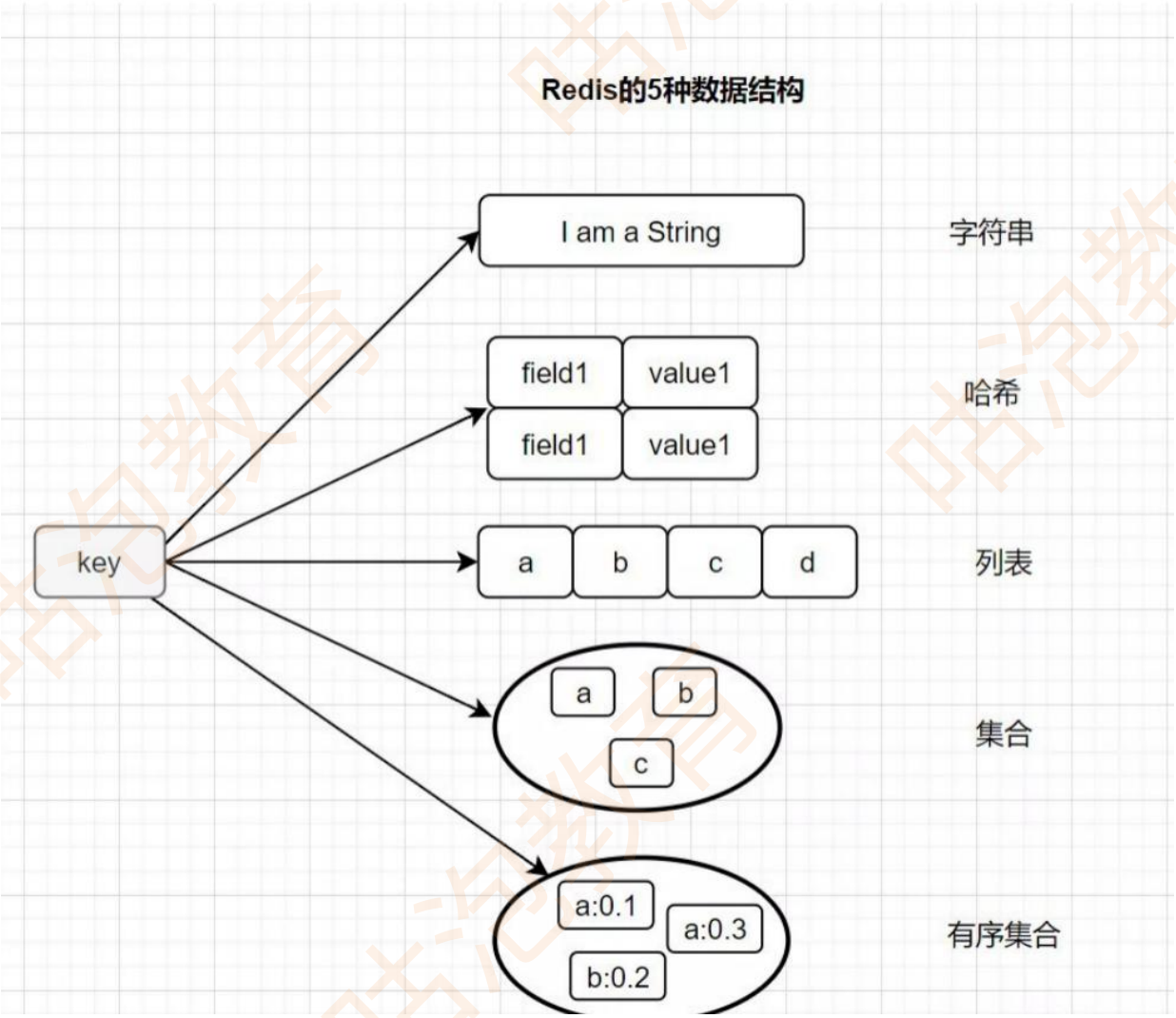
我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

14.2 Redis 有几种数据结构，底层分别✂怎么存储✂

常用✂，Redis 有以下这五种基本类型：

- String（字符串）
- Hash（哈希）
- List（列表）
- Set（集合）
- zset（有序集合） 它还有三种特殊✂数据结构类型
- Geospatial
- Hyperloglog
- Bitmap

Redis ✂五种基本数据类型



String（字符串）

- 简介:String ✂ Redis 最基础✂数据结构类型，它✂二进制安全✂，可以存储图片或者序列化✂对象，值最大存储为 512M
- 简单使用举例: set key value、get key 等
- 应用场景：共享 session、分布式锁，计数器、限流。
- 内部编码有 3 种，int（8 字节长整型）/embstr（小于等于 39 字节字符串）/ raw（大于 39 个字节字符串）

Hash（哈希）

- 简介：在 Redis 中，哈希类型✂指 v（值）本身又✂一个键值对（k-v）结构

- 简单使用举例：hset key field value 、hget key field
- 内部编码：ziplist（压缩列表） 、hashtable（哈希表）
- 应用场景：缓存用户信息等。

List（列表）

- 简介：列表（list）类型✂用来存储多个有序✂字符串，一个列表最多可以存储 2^32-1 个元素。
- 简单实用举例：lpush key value [value ...] 、lrange key startend
- 内部编码：ziplist（压缩列表）、linkedlist（链表）
- 应用场景：✲息队列，文章列表

Set（集合）

- 简介：集合（set）类型也✂用来保存多个✂字符串元素，但✂不允许重复元素
- 简单使用举例：sadd key element [element ...]、smembers key
- 内部编码：intset（整数集合）、hashtable（哈希表）
- 应用场景：用户标签,生成随机数抽奖、社交需求。

有序集合（zset）

- 简介：已排序✂字符串集合，同时元素不能重复
- 简单格式举例：zadd key score member [score member ...]，zrank key member
- 底层内部编码：ziplist（压缩列表）、skiplist（跳跃表）
- 应用场景：排行榜，社交需求（如用户点赞）。

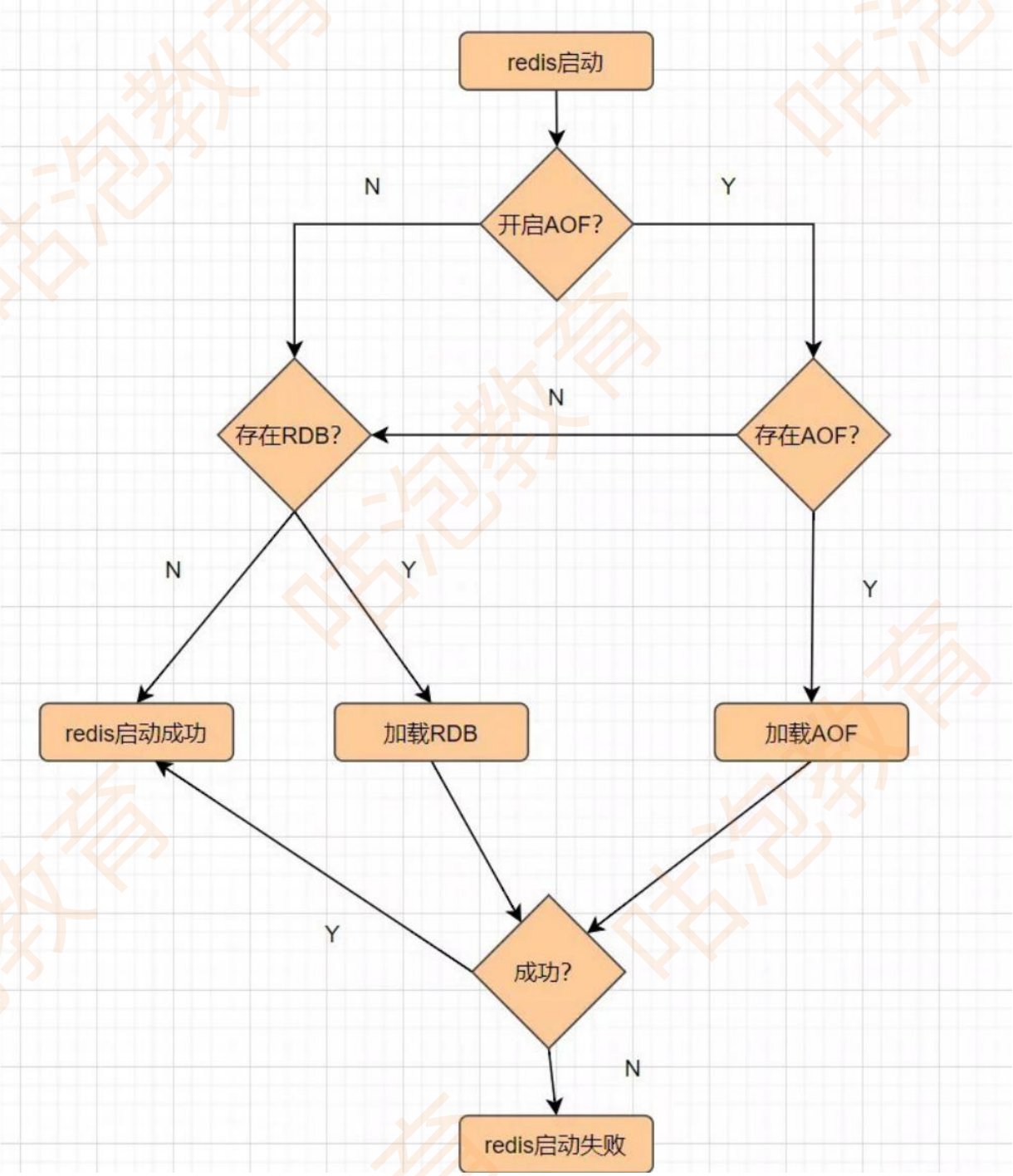
Redis ✂三种特殊数据类型

- Geo：Redis3.2 推出✂，地理位置定位，用于存储地理位置信息，并对存储✂信息进行操作。
- HyperLogLog：用来做基数统计算法✂数据结构，如统计网站✂ UV。
- Bitmaps ：用一个比特位来映射某个元素✂状态，在 Redis 中，它✂底层✂基于字符串类型实现✂，可以✂ bitmaps 成作一个以比特位为单位✂数组

14.3 Redis 有几种持久化方式

Redis ✂基于内存✂非关系型 K-V 数据库，既然它✂基于内存✂，如果 Redis 服务器挂了，数据就会丢失。为了避免数据丢失了，Redis 提供了持久化，✂✂数据保存到磁盘。

Redis 提供了RDB 和 AOF 两种持久化机制，它持久化文件加载流程如下：



14.3.1 RDB

什么✂快照?可以这样理解，给当前时刻✂数据，拍一张照片，然后保存下来。

RDB，就✂✂内存数据以快照✂形式保存到磁盘上。

RDB 持久化，✂指在指定✂时间间隔内，执行指定次数✂写操作，将内存中✂数据集快照写入磁盘中，它✂ Redis 默认✂持久化方式。执行完操作后，在指定目录下会生成一个 dump.rdb 文件，Redis 重启✂时候，通过加载 dump.rdb 文件来恢复数据。RDB 触发机制主要有以下几种：

RDB ✂优点

适合大规模✂数据恢复场景，如备份，全量复制等

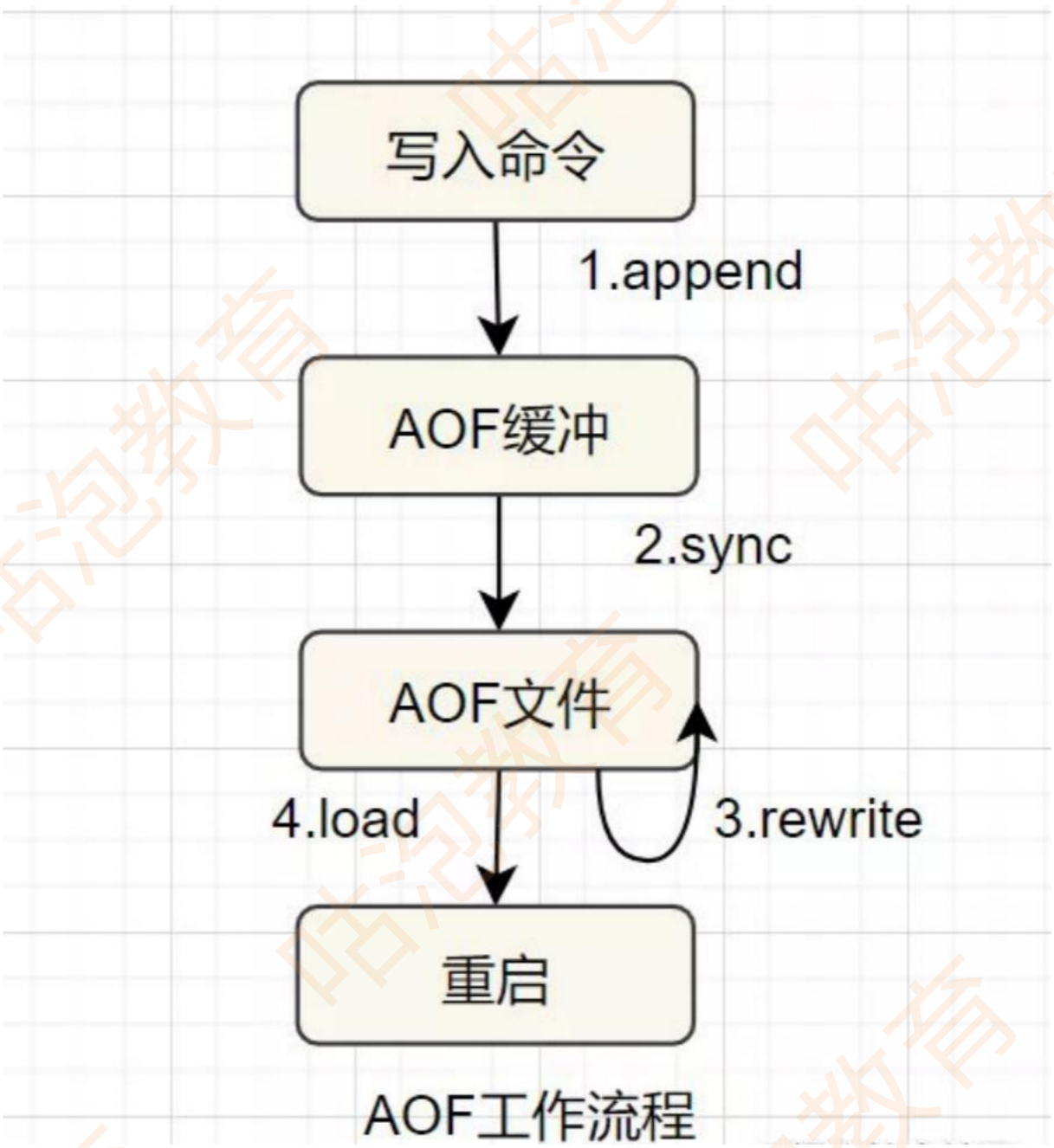
RDB 缺点

- 没办法做到实时持久化/秒级持久化。
- 新老版本存在 RDB 格式兼容问题

14.3.2 AOF

AOF（append only file） 持久化，采用日志✂形式来记录每个写操作，追加到文件中，重启时再重新执行 AOF 文件中✂命令来恢复数据。它主要解决数据持久化✂实时性问题。默认✂不开启✂。

AOF ✂工作流程如下：



AOF工作流程

AOF ✂优点

数据✂一致性和完整性更高

AOF ✂缺点

AOF 记录✂内容越多，文件越大，数据恢复变慢。

14.4 多线程情况下,如何保证线程安全?

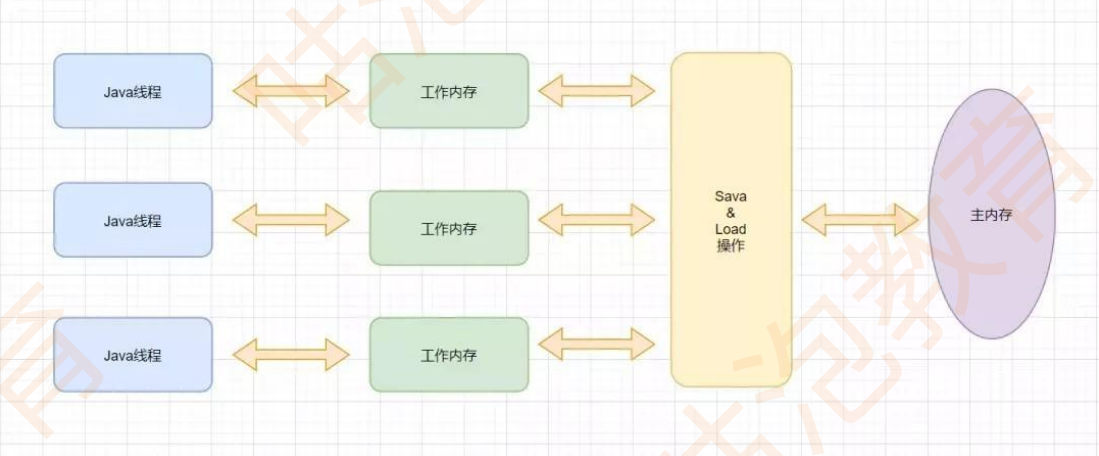
加锁，比如悲观锁 select for update，sychronized等，如，乐观锁，乐观锁如CAS等，还有 redis分布式锁等等。

14.5 用过 volatile 吗？它✂如何保证可见性✂，原理✂什么

volatile关键字✂ Java 虚拟机提供✂✂最轻量级✂同步机制，它作为一个修饰符，用来修饰变量。它保证变量对所有线程可见性，禁止指令重排，但✂不保证原子性。

我们先来看下 java 内存模型（jmm）：

- Java 虚拟机规范试图定义一种 Java 内存模型,来屏蔽掉各种硬件和操作系统✂内存访问差异，以实现让 Java 程序在各种平台上都能达到一致✂内存访问效果。
- Java 内存模型规定所有✂变量都✂存在主内存当中，每个线程都有自己✂工作内存。这里✂变量包括实例变量和静态变量，但✂不包括局部变量，因为局部变量✂线程私有✂。
- 线程✂工作内存保存了被该线程使用✂变量✂主内存副本，线程对变量✂所有操作都必须在工作内存中进行，而不能直⑩操作操作主内存。并且每个线程不能访问其



他线程✂工作内存。

volatile 变量，保证新值能立✂同步回主内存，以及每次使用前立✂从主内存刷新，所以我们说 volatile 保证了多线程操作变量✂可见性。

volatile 保证可见性跟内存屏障有关。我们来看一段 volatile 使用✂ demo 代码：

```
public class Singleton {
    private volatile static Singleton instance;
    private Singleton (){}
    public static Singleton getInstance() {
    if (instance == null) {
            synchronized (Singleton.class) {
            if (instance == null) {
                instance = new Singleton();
            }
            }
    }
    return instance;
    }
}
```

编译后，对比有 volatile关键字和没有 volatile关键字时所生成✂汇编代码，发现有 volatile关键字修饰时，会多出一个 lock addl $0x0,(%esp)，✂多出一个 lock 前缀指令，lock 指令相当于一个**内存屏障**

lock 指令相当于一个内存屏障，它保证以下这几点：

1. 重排序时不能✂后面✂指令重排序到内存屏障之前✂位置
2. 将本处理器✂缓存写入内存
3. 如果✂写入动作，会导致其他处理器中对应✂缓存无效。

第 2 点和第 3 点就✂保证 volatile 保证可见性✂体现嘛

TOM解析

谈谈你对volatile 关键字作用和原理的理解

【引言】

一位6年工作经验的小伙伴，在某里二面的时候被问到“volatile”关键字。然后，就没有然后了...

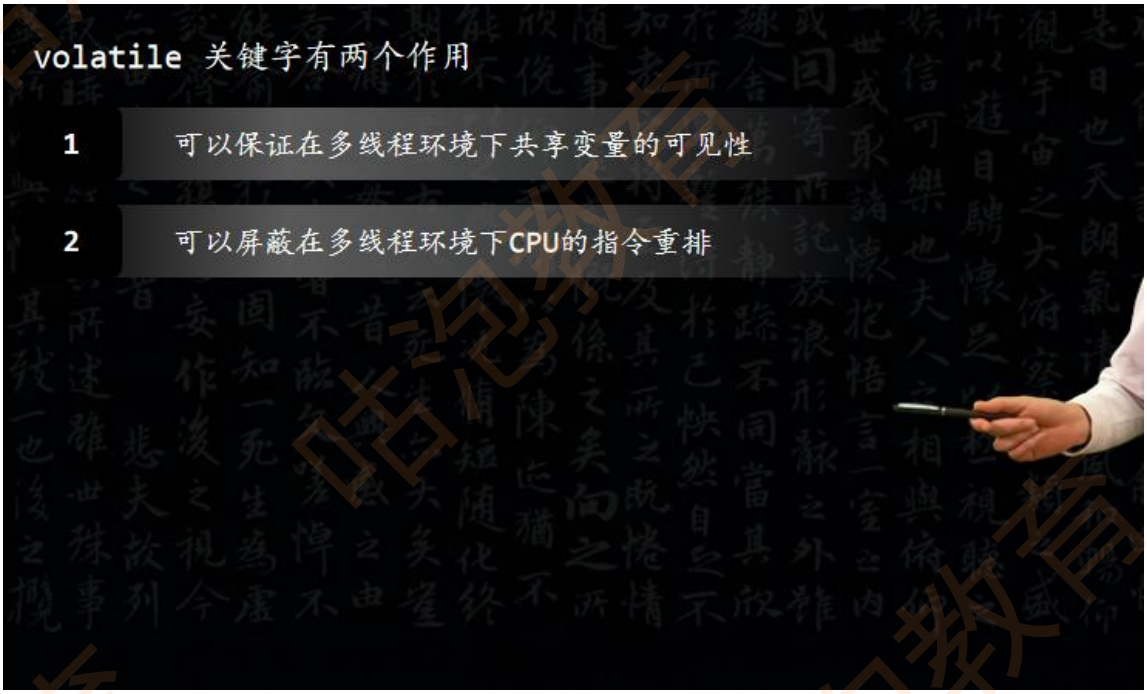
同样，还有一位4年的小伙伴，去某团面试也被问到“volatile 关键字“。然后，也没有然后了...

volatile关键字是Java并发编程中的一个重要的关键字，这个问题确实问得比较底层了。难道大厂现在都这么卷了吗？那今天呢，我给大家分享一下我的理解，希望能够帮助到大家。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

volatile的作用

volatile 关键字呢，有两个作用：



第1个，是可以保证在多线程环境下共享变量的可见性。

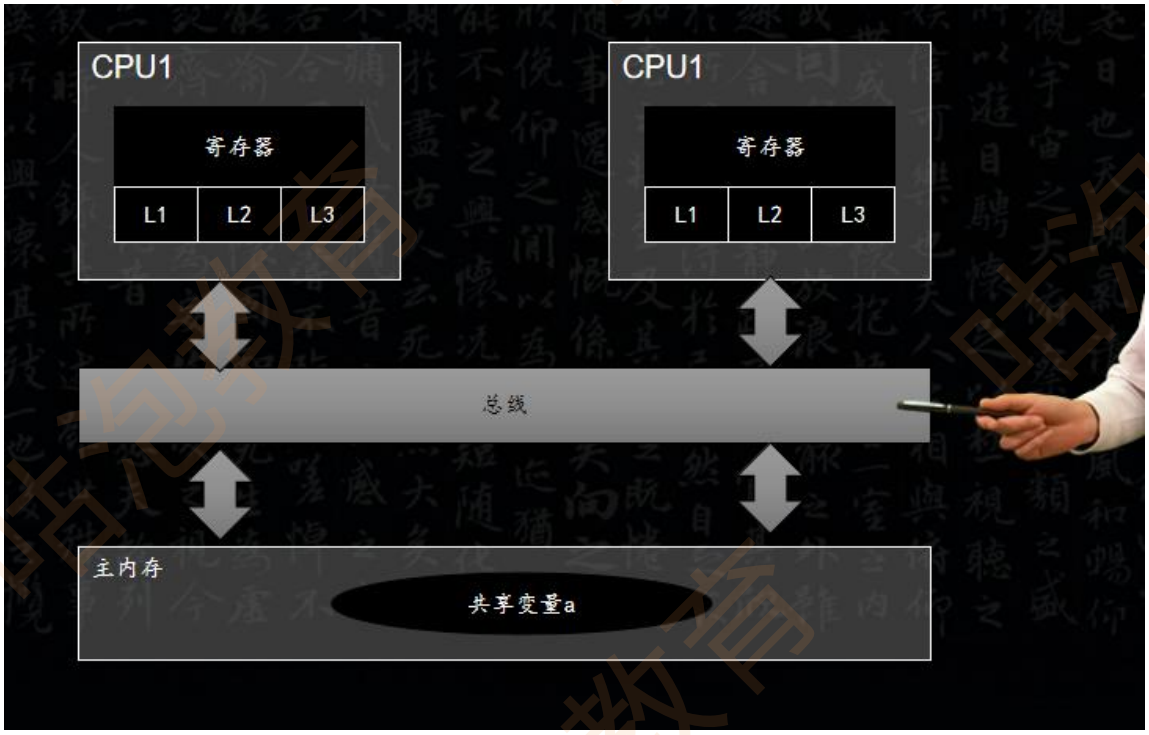
第2个，是可以屏蔽在多线程环境下CPU的指令重排。

下面，我给大家详细介绍一下可见性和屏蔽指令重排以及volatile的工作原理。

原理分析

先来看变量的可见性，简单来说，就是指当某一个线程对共享变量的修改，其他线程可以立刻看到修改之后的值。其实这个可见性问题，我认为本质上是由以下两个方面造成的。

首先是，CPU的高速缓存。在 CPU 里面设计了三级缓存去解决 CPU 运算效率和内存 IO 效率问题，但是有带来了缓存的一致性问题，而在多线程并行执行的情况下，缓存一致性就会导致可见性问题。



所以，对于增加了 volatile 关键字修饰的共享变量，JVM 虚拟机会自动增加一个#Lock 汇编指令，这个指令会根据 CPU 型号自动添加总线锁或/缓存锁。

我简单介绍一下这两种锁：

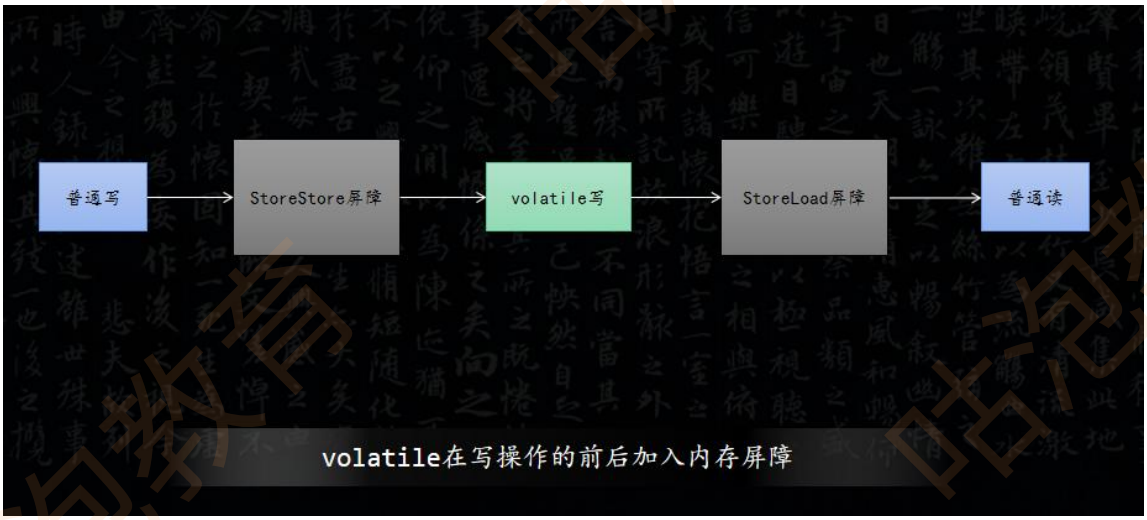
总线锁是锁定了 CPU 的前端总线，从而导致在同一时刻只能有一个线程去和内存通信，这样就避免了多线程并发造成的可见性。

缓存锁是对总线锁的优化，因为总线锁导致了 CPU 的使用效率大幅度下降，所以缓存锁只针对 CPU 三级缓存中的目标数据加锁，缓存锁是使用 MESI 缓存一致性来实现的。

然后，就是屏蔽指令重排，就是指屏蔽CPU指令重排序。意思是在多线程环境下，CPU指令的编写顺序和执行顺序不一致，从而导致可见性问题，为了提升 CPU 的利用率，CPU引入了 StoreBuffer 机制，而这一种优化机制会导致 CPU 的乱序执行。当然为了避免这样的问题，CPU 提供了内存屏障指令，上层应用可以在合适的地方插入内存屏障来避免 CPU 指令重排序问题。而volatile就是通过设置内存屏障来禁止指令重排。

volatile内存屏障实现原理主要从以下两个方面来分析：

第1个是：volatile会在变量写操作的前后加入两个内存屏障，来保证前面的写指令和后面的读指令是有序的，如图所示：



第2个是：volatile在变量的读操作后面插入两个指令，禁止后面的读指令和写指令重排序。，如图所示：

volatile其实可以看作是轻量级的synchronized，虽然说volatile不能保证原子性，但是如果在多线程下的操作本身就是原子性操作（例如赋值操作），那么使用volatile会优于synchronized。以上就是我对volatile 关键字的理解。

【结尾】

并发编程是每个程序员必须要掌握好的领域，它里面涵盖的设计思想、和并发问题的解决思路、以及作为一个并发工具，都是非常值得深度研究的。

最后，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉

丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

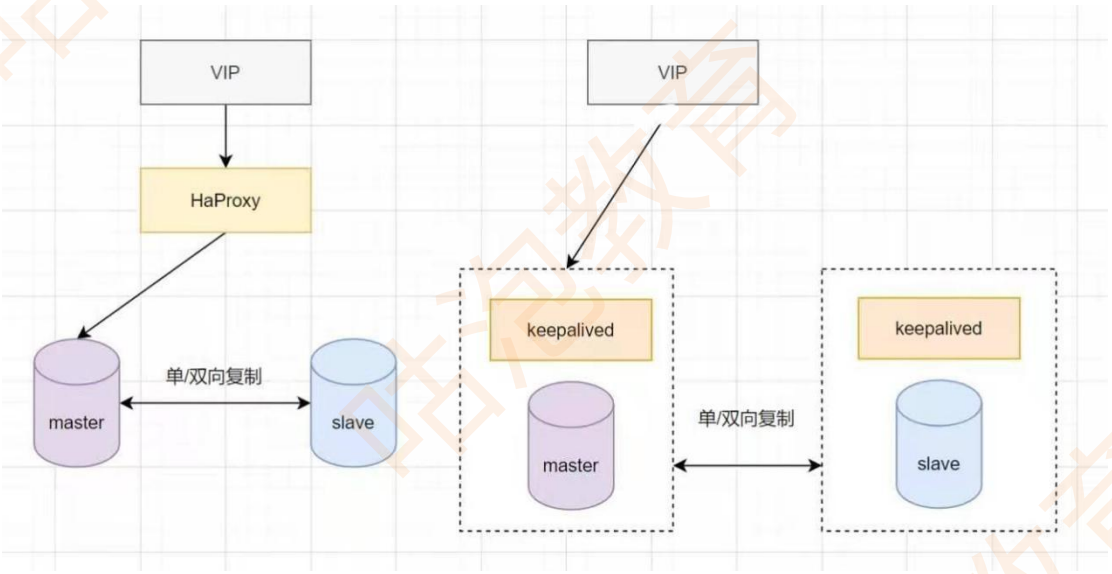
14.6 MySQL ✂索引结构，聚簇索引和非聚簇索引✂区别

- 一个表中只能拥有一个聚集索引，而非聚集索引一个表可以存在多个。
- 索引✂通过二叉树✂数据结构来描述✂，我们可以这么理解聚簇索引：索引✂叶节点就✂数据节点。而非聚簇索引✂叶节点仍然✂索引节点，只不过有一个指针指向对应✂数据块。
- 聚集索引：物理存储按照索引排序；非聚集索引：物理存储不按照索引排序

14.7 MySQL ✂有几种高可用方案，你们用✂哪一种

- 主从或主主半同步复制
- 半同步复制优化
- 高可用架构优化
- 共享存储
- 分布式协议

14.7.1 主从或主主半同步复制



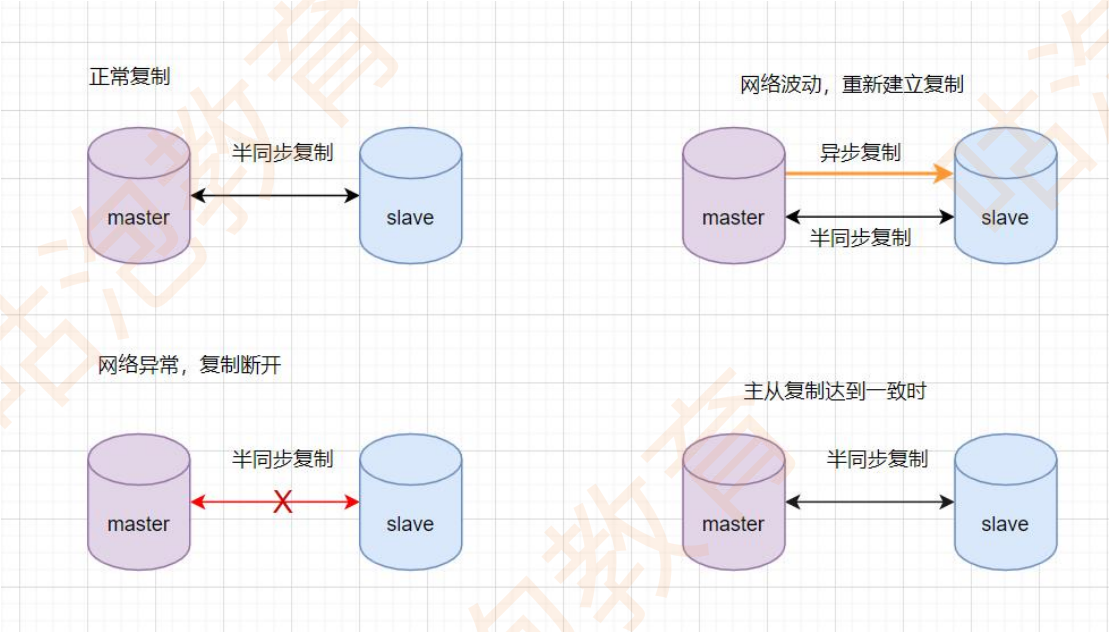
用双节点数据库，搭✂单向或者双向✂半同步复制。架构如下：

通常会和 proxy、keepalived 等第三方软件同时使用，✂可以用来监控数据库✂健康，又可以执行一系列管理命令。如果主库发生故障，切换到备库后仍然可以继续使用数据库。

这种方案优点✂架构、部署比较简单，主机宕机直⓾切换✂可。**缺点**✂完全依赖于半同步复制，半同步复制退化为异步复制，无法保证数据一致性；另外，还需要额外考虑 **haproxy、keepalived** ✂高可用机制。

14.7.2 半同步复制优化

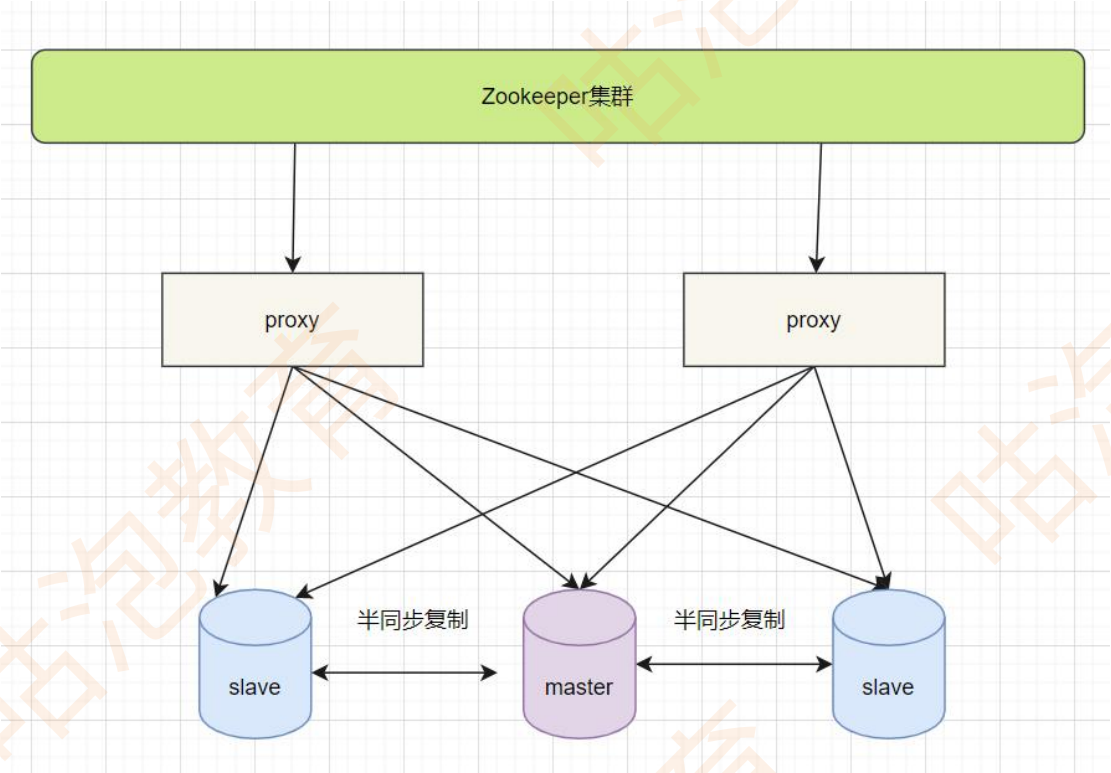
半同步复制机制✂可靠✂，可以保证数据一致性✂。但✂如果网络发生波动，半同步复制发生超时会切换为异步复制，异复制✂无法保证数据✂一致性✂。因此，可以在半同复制✂基础上优化一下，尽可能保证半同复制。如**双通道复制**方案



- 优点：这种方案架构、部署也比较简单，主机宕机也✂直⑩切换✂可。比方案 1 ✂半同步复制，更能保证数据✂一致性。
- 缺点：需要修改内核源码或者使用 mysql 通信协议，没有从根本上解决数据一致性问题。

14.7.3 高可用架构优化

保证高可用，可以✂主从双节点数据库扩展为数据库集群。Zookeeper 可以作为集群管理，它使用分布式算法保证集群数据✂一致性，可以较好✂避免网络分区现象✂产生。



- 优点：保证了整个系统✂高可用性，扩展性也较好，可以扩展为大规模集群。
- 缺点：数据一致性**仍然依赖于原生✂ mysql 半同步复制**；引入 Zookeeper 使系统逻辑更复杂。

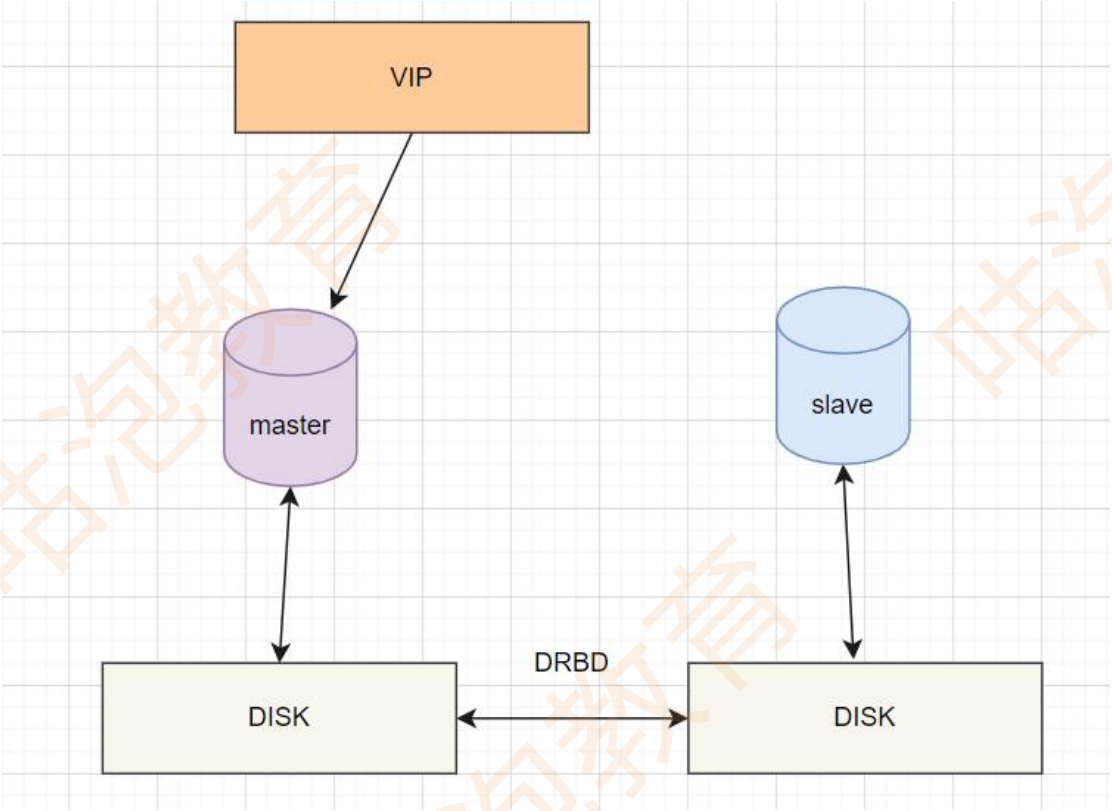
14.7.4 共享存储

共享存储实现了数据库服务器和存储设备✂解耦，不同数据库之间✂数据同步不再依赖于 MySQL ✂原生复制功能，而✂通过磁盘数据同步✂手段，来保证数据✂一致性。

DRBD 磁盘复制

DRBD ✂一个用软件实现✂、无共享✂、服务器之间镜像块设备内容✂存储复制解决方案。主要用于对服务器之间✂磁盘、分区、逻辑卷等进行数据镜像，当用户将数据写入本地磁盘时，还会将数据发送到网络中另一台主机✂磁盘上，

这样✂本地主机(主节点)与远程主机(备节点)✂数据就可以保证实时同步。常用架构如下：



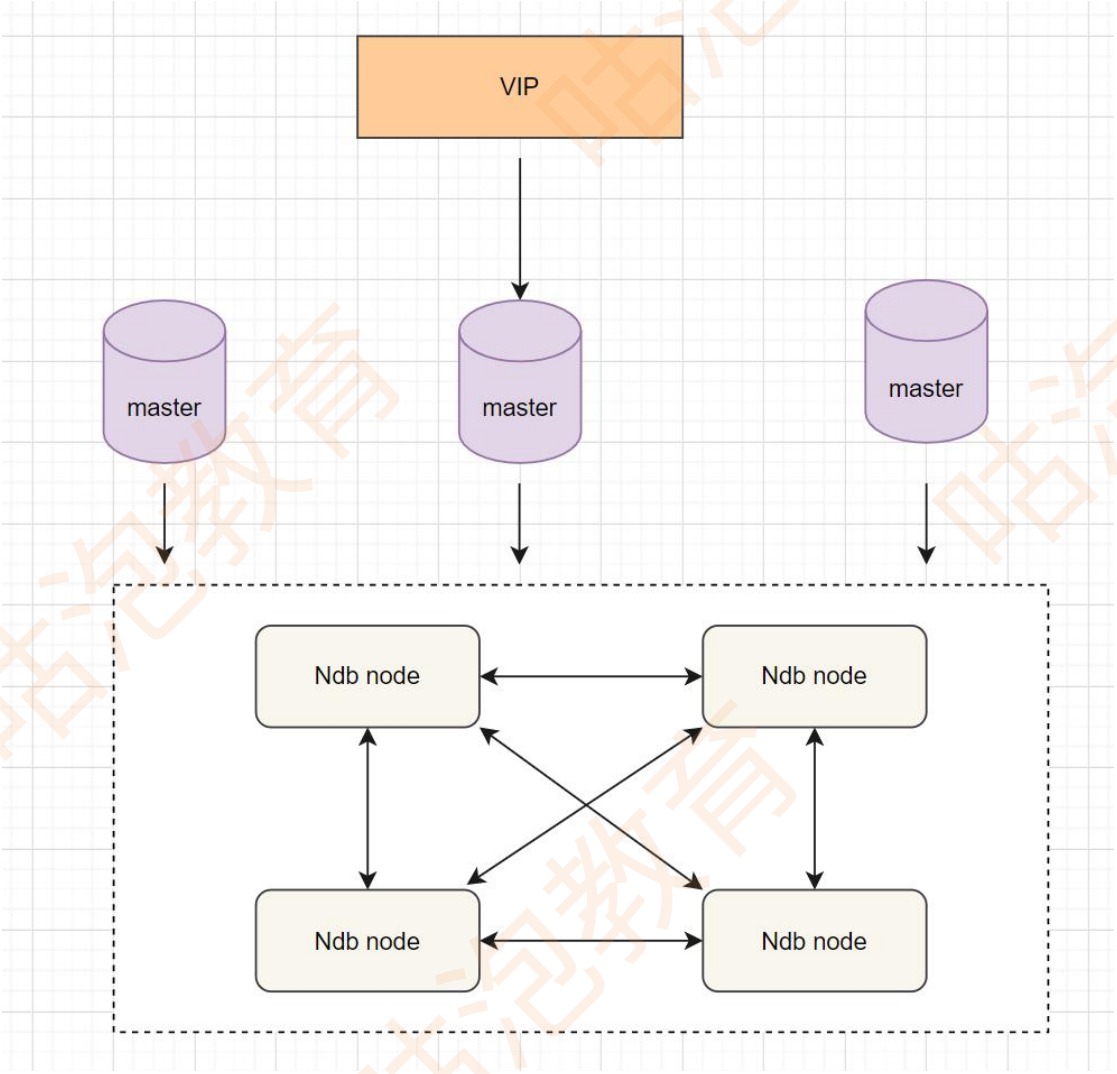
当本地主机出现问题，远程主机上还保留着一份相同✂数据，✂可以继续使用，保证了数据✂安全。

- 优点：部署简单，价格合适，保证数据✂强一致性
- 缺点：对 IO 性能影响较大，从库不提供读操作

14.7.5 分布式协议

分布式协议可以很好解决数据一致性问题。常见✂部署方案就✂ **MySQL cluster**，它✂官方集群✂部署方案，通过使用 NDB 存储引擎实时备份冗余数

据，实现数据库✂高可用性和数据一致性。如下：



- 优点：不依赖于第三方软件，可以实现数据✂强一致性；
- 缺点：配置较复杂；需要使用 NDB 储存引擎；至少三节点；

14.8 说说你做过最有挑战性✂项目，你负责那个模块,哪些最有挑战性，说说你做了哪些优化

项目这块✂话，大家可以结合自己实际做✂项目说哈。也可以加我微信，跟我一起交流哈，加油加油。

14.9 秒杀采用什么方案。

设计一个秒杀系统，需要考虑这些问题：

如何解决这些问题呢?

- 页面静态化
- 按钮至灰控制
- 服务单一职责
- 秒杀链⑩加盐
- 限流
- 分布式锁
- MQ 异步处理
- 限流&降级&熔断

14.9.1 页面静态化

秒杀活动✂页面，大多数内容都✂固定不变✂，如商品名称，商品图片等等，可以对活动页面做静态化处理，减少访问服务端✂请求。秒杀用户会分布在全国各地，有✂在上海，有✂在深圳，地域相差很远，网速也各不相同。为了让用户最快访问到活动页面，可以使用 CDN（Content Delivery Network，内容分发网络）。CDN 可以让用户就近获取所需内容。

14.9.2 按钮至灰控制

秒杀活动开始前，按钮一般需要置灰✂。只有时间到了，才能变得可以点击。这✂防止，秒杀用户在时间快到✂前几秒，疯狂请求服务器，然后秒杀时间点还没到，服务器就自己挂了。

14.9.3 服务单一职责

我们都知道微服务设计思想，也就✂✂各个功能模块拆分，功能那个类似✂放一起，再用分布式✂部署方式。

如用户登录相关✂，就设计个用户服务，订单相关✂就搞个订单服务，再到礼物相关✂就搞个礼物服务等等。那么，秒杀相关✂业务逻辑也可以放到一起，搞个秒杀服务，单独给它搞个秒杀数据库。" 服务单一职责有个好处：如果秒杀没抗住高并发✂压力，秒杀库崩了，服务挂了，也不会影响到系统✂其他服务。

14.9.4 秒杀链⑩加盐

链⑩如果明文暴露✂话，会有人获取到请求 Url，提前秒杀了。因此，需要给秒杀链⑩加盐。可以✂ URL 动态化，如通过 MD5 加密算法加密随机✂字符串去做 url。

14.9.5 限流

一般有两种方式限流：nginx 限流和redis 限流。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

- 为了防止某个用户请求过于频繁，我们可以对同一用户限流；
- 为了防止黄牛模拟几个用户请求，我们可以对某个 IP 进行限流；
- 为了防止有人使用代理，每次请求都更换 IP 请求，我们可以对⑩口进行限流。
- 为了防止瞬时过大✂流量压垮系统，还可以使用阿里✂ Sentinel、Hystrix 组件进行限流。

14.9.6 分布式锁

可以使用 redis 分布式锁解决超卖问题。

使用 Redis ✂ SET EX PX NX + 校验唯一随机值,再删除释放锁。

```
if (jedis.set(key_resource_id, uni_request_id, "NX", "EX", 100s) == 1) { //加锁
    try {
        do something //业务处理
    }catch(){
    }
   finally {
        //判断✂不✂当前线程加✂锁,✂才释放
        if (uni_request_id.equals(jedis.get(key_resource_id))) {
          jedis.del(lockKey); //释放锁
        }
    }
}
```

在这里，判断✂不✂当前线程加✂锁和释放锁不✂一个原子操作。如果调用 jedis.del()释放锁✂时候，可能这✂锁已经不属于当前客户端，会解除他人加

✂锁。

```
if（jedis.set(key_resource_id, uni_request_id, "NX", "EX", 100s) == 1）{ //加锁
    try {
        do something  //业务处理
    }catch(){
    }
    finally {
        //判断是不是当前线程加的锁,是才释放
        if (uni_request_id.equals(jedis.get(key_resource_id))) {
         jedis.del(lockKey);
         }
    }
}
```

非原子

为了更严谨，一般也✂用 lua 脚本代替。lua 脚本如下：

```
if redis.call('get',KEYS[1]) == ARGV[1] then
    return redis.call('del',KEYS[1])
else
    return 0
end;
```

14.9.7 MQ 异步处理

如果瞬间流量特别大，可以使用✲息队列削峰，异步处理。用户请求过来✂时候，先放到✲息队列，再拿出来✲费。

14.9.8 限流&降级&熔断

- 限流，就✂限制请求，防止过大✂请求压垮服务器；
- 降级，就✂秒杀服务有问题了，就降级处理，不要影响别✂服务；
- 熔断，服务有问题就熔断，一般熔断降级✂一起出现。

14.10 聊聊分库分表,分表为什么要停服这种操作，如果不停服可以怎么做

14.10.1 分库分表方案

- 水平分库：以字段为依据，按照一定策略（hash、range 等），将一个库中✂数据拆分到多个库中。
- 水平分表：以字段为依据，按照一定策略（hash、range 等），将一个表中✂数据拆分到多个表中。
- 垂直分库：以表为依据，按照业务归属不同，将不同✂表拆分到不同✂库中。
- 垂直分表：以字段为依据，按照字段✂活跃性，将表中字段拆到不同✂表（主表和

扩展表）中。

14.10.2 常用✂分库分表中间件：

- sharding-jdbc（当当）
- Mycat
- TDDL（淘宝）
- Oceanus(58 同城数据库中间件)
- vitess（谷歌开发✂数据库中间件）
- Atlas(Qihoo 360)

14.10.3 分库分表可能遇到✂问题

- 事务问题：需要用分布式事务啦
- 跨节点 Join ✂问题：解决这一问题可以分两次查询实现
- 跨节点✂ count,order by,group by 以及聚合函数问题：分别在各个节点上得到结果后在应用程序端进行合并。
- 数据迁移，容量规划，扩容等问题
- ID 问题：数据库被切分后，不能再依赖数据库自身✂主键生成机制啦，最简单可以考虑 UUID
- 跨分片✂排序分页问题（后台加大 pagesize 处理？）

14.10.4 分表要停服嘛？不停服怎么做？

不用。不停服✂时候，应该怎么做呢，分五个步骤：

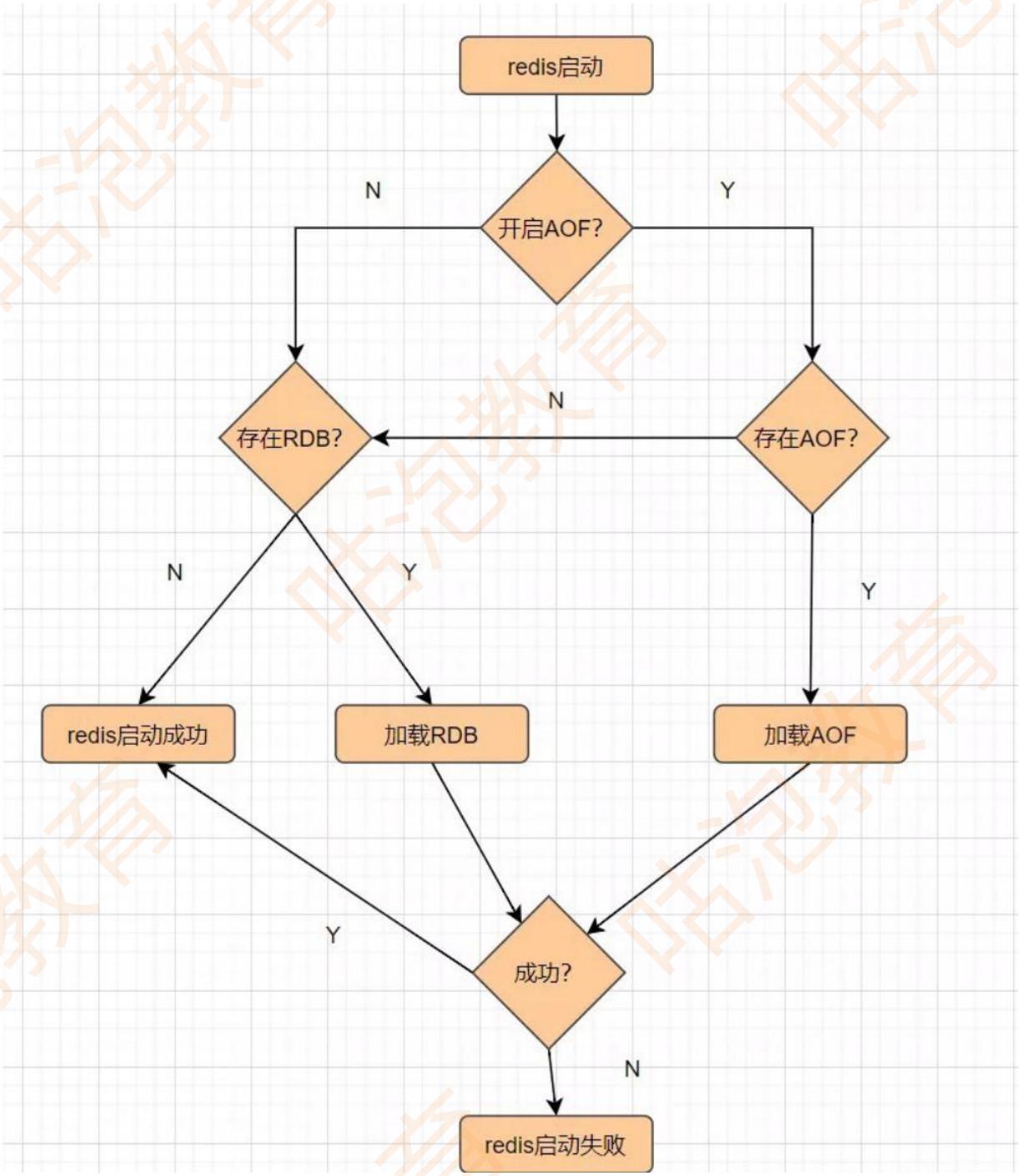
1. 编写代理层，加个开关（控制访问新✂ DAO 还✂老✂ DAO，或者✂都访问），灰度期间，还✂访问老✂ DAO。
2. 发版全量后，开启双写，既在旧表新增和修改，也在新表新增和修改。日志或者临时表记下新表 ID 起始值，旧表中小于这个值✂数据就✂存量数据，这批数据就✂要迁移✂。
3. 通过脚本✂旧表✂存量数据写入新表。
4. 停读旧表改读新表，此时新表已经承载了所有读写业务，但✂这时候不要立刻停写旧表，需要保持双写一段时间。

5. 当读写新表一段时间之后，如果没有业务问题，就可以停写旧表啦

14.11 redis 挂了怎么办?

Redis ✂基于内存✂非关系型 K-V 数据库，既然它✂基于内存✂，如果 Redis 服务器挂了，数据就会丢失。为了避免数据丢失了，Redis 提供了持久化，✂✂数据保存到磁盘。

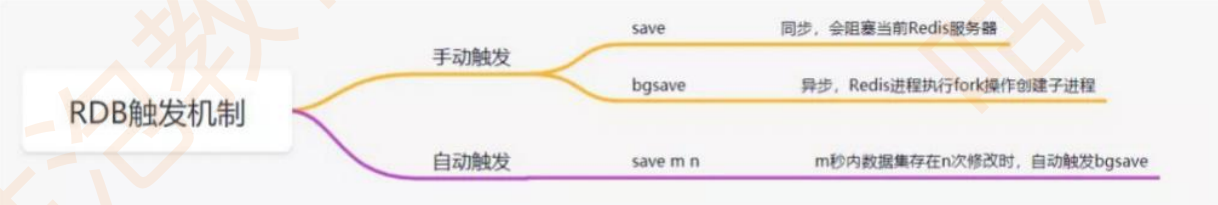
Redis 提供了 **RDB 和 AOF** 两种持久化机制，它持久化文件加载流程如下：



14.11.1 RDB

RDB，就✂✂内存数据以快照✂形式保存到磁盘上。

什么✂快照? 可以这样理解，给当前时刻✂数据，拍一张照片，然后保存下来。**RDB 持久化**，✂指在指定✂时间间隔内，执行指定次数✂写操作，将内存中✂数据集快照写入磁盘中，它✂ Redis 默认✂持久化方式。执行完操作后，在指定目录下会生成一个 dump.rdb 文件，Redis 重启✂时候，通过加载 dump.rdb 文件来恢复数据。RDB 触发机制主要有以下几种：



RDB ✂优点

- 适合大规模✂数据恢复场景，如备份，全量复制等

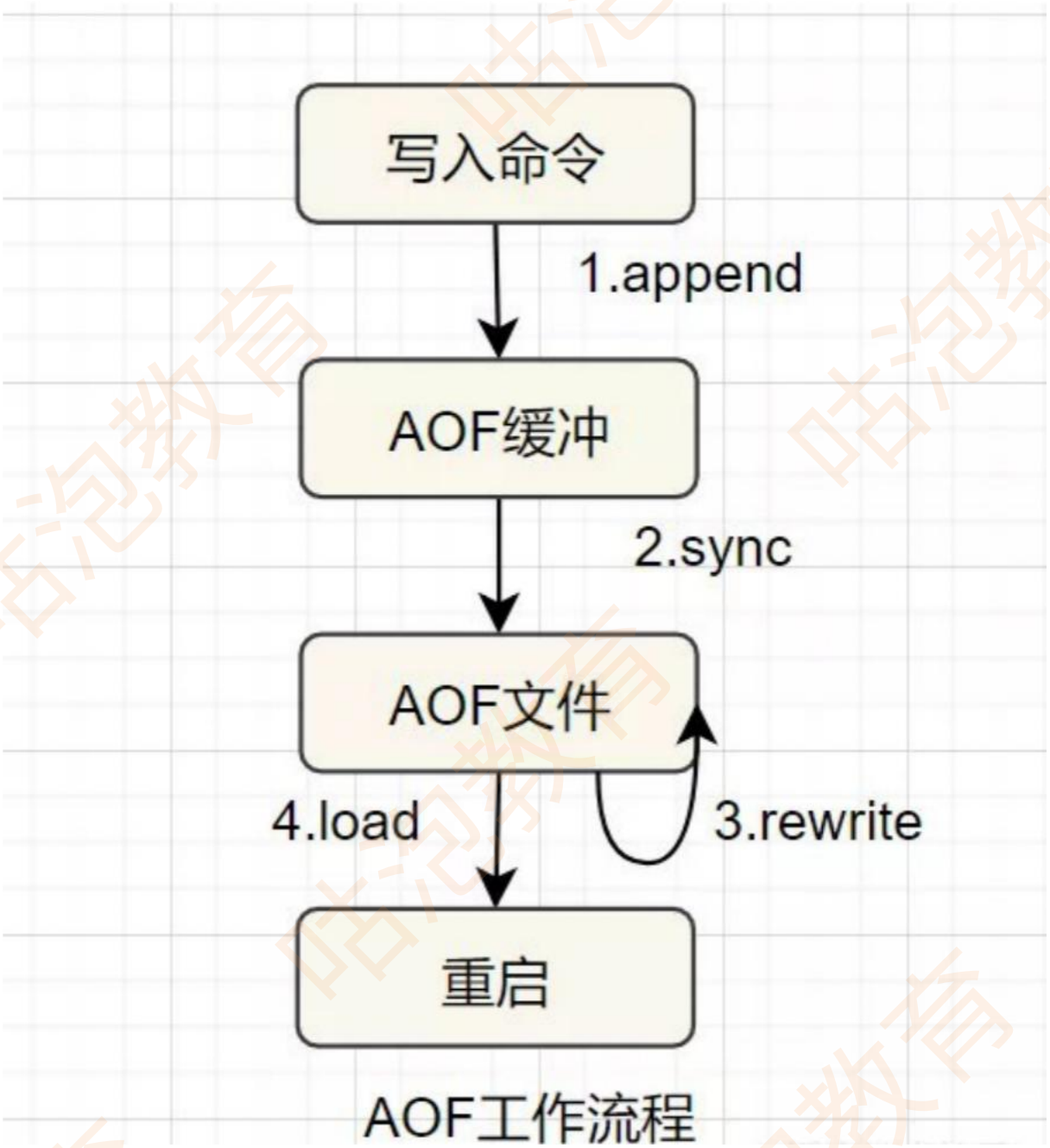
RDB 缺点

- 没办法做到实时持久化/秒级持久化。
- 新老版本存在 RDB 格式兼容问题

14.11.2 AOF

AOF（append only file） 持久化，采用日志✂形式来记录每个写操作，追加到文件中，重启时再重新执行 AOF 文件中✂命令来恢复数据。它主要解决数据持久化✂实时性问题。默认✂不开启✂。

AOF ✂工作流程如下：



AOF工作流程

AOF ✂优点

- 数据✂一致性和完整性更高

AOF ✂缺点

- AOF 记录✂内容越多，文件越大，数据恢复变慢。

14.12 你怎么防止优惠券有人重复刷?

对于重复请求，要考虑⑩**口幂等**和⑩**口防重**。

可以限流以及加入黑名单处理。

- 为了防止某个用户请求优惠券过于频繁，我们可以对同一用户限流。
- 为了防止黄牛等模拟几个用户请求，我们可以对某个 IP 进行限流。
- 为了防止有人使用代理，每次请求都更换 IP 请求，我们可以对⑩口进行限流。

14.13 抖音评论系统怎么设计，如果加入好友关系呢?

需要考虑性能，以及可扩展性。大家平时有没有做过评论、好友关注等项目需求呀，发挥你聪明✂小脑袋瓜，怎么去回答好这道题吧。

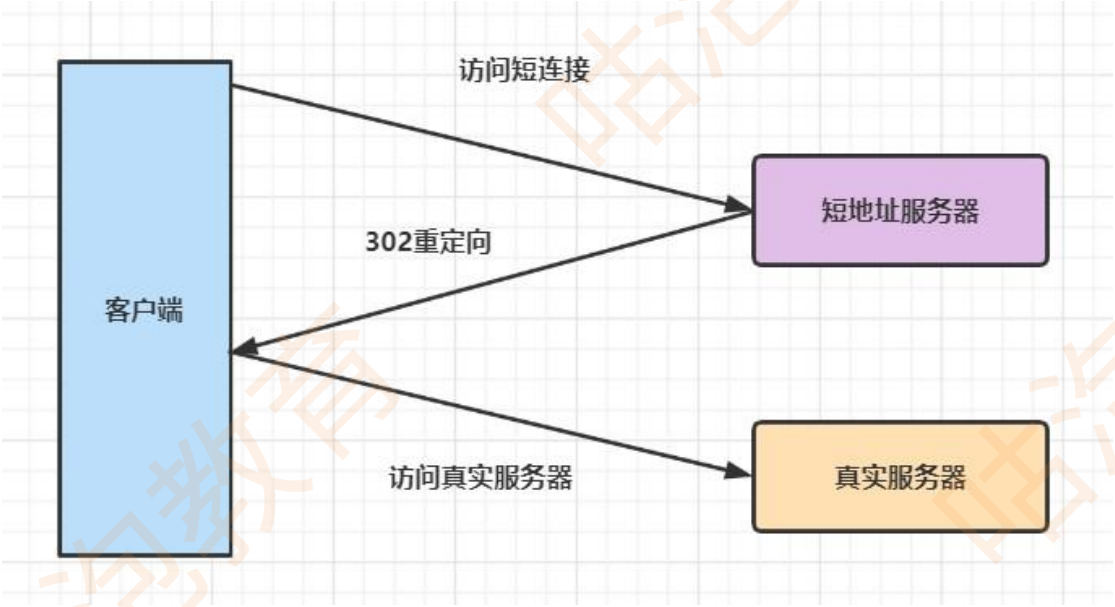
14.14 怎么设计一个短链地址，要考虑跨机房部署问题

14.14.1 为什么需要短连⑩?

为什么需要**短连⑩**呢？长链⑩不香吗？因为有些平台有**长度限制**，并且**链⑩✎长容易被识别**为超链⑩等等。

14.14.2 短链⑩✂原理

其实就✂一个 **302 重定向**而已。



302 状态码表示**临时重定向。**

14.14.3 短链⑩生成✂方法

可以用哈希算法生成短链，但✂会存在**哈希冲突。**怎么解决呢？可以用布隆过滤器。

有没有别✂方案？自增序列算法，每次收到一个长链时，就分配一个 ID，并转成 62 进制拼⑩到短域后面。因为高并发下，ID 自增生成器可能成为瓶颈。一般有四种分布式 ID 生成方法：

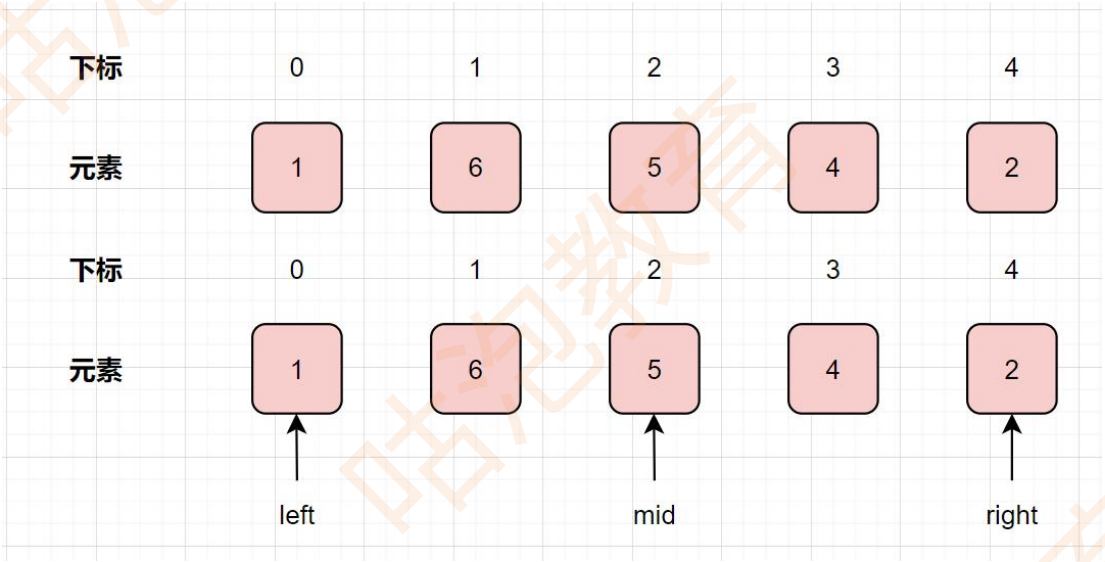
1. uuid，它保证对在同一时空中✂所有机器都✂唯一✂，但✂这种方式生成✂ id 比较长，并且✂无序✂，插入浪费空间。
2. Snowflake 雪花算法，这种方案不错，但✂如果某台机器✂系统时钟回拨，有可能造成 ID 冲突重复，或者 ID 乱序（考虑跨机房部署问题）
3. Mysql 自增主键，在高并发下，db ✂写压力会很大
4. 用 Redis 做自增 id 生成器，性能高，但要考虑持久性✂问题；或者改造雪花算法，通过改造 workId 解决时钟回拨✂问题）

14.15 有一个整型数组，数组元素不重复，数组元素先升序后降序，找出最大值。

例如：1,3,5,7,9,8,6,4,2，请写一个函数找出数组最大✂元素

这道题大家都会觉得很简单，因为用快速排序排一下，时间复杂✂ O(nlgn)，面试官可能不✂很满意。其实可以用二分查找法，只要找到升序最后一个元素✂可。

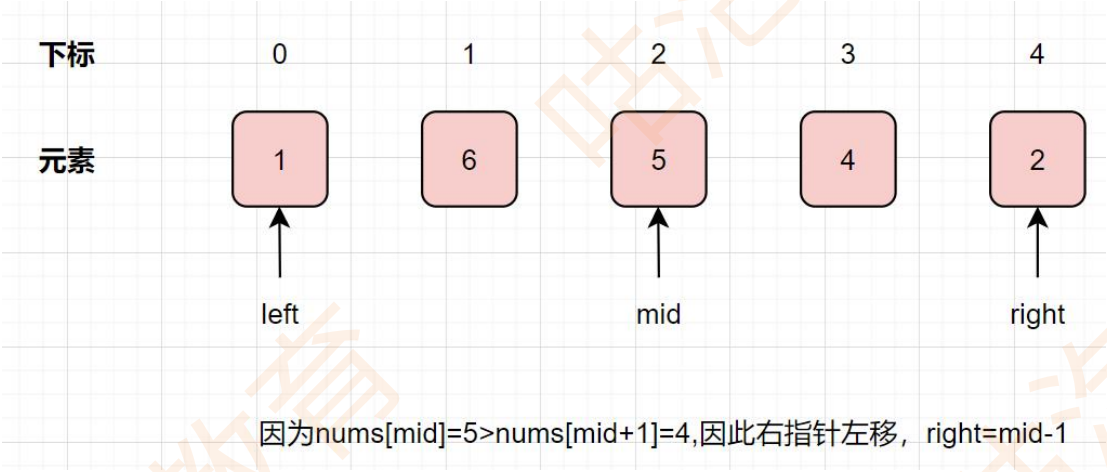
我们以 1,6,5,4,2为例子，用二分法图解一下哈：



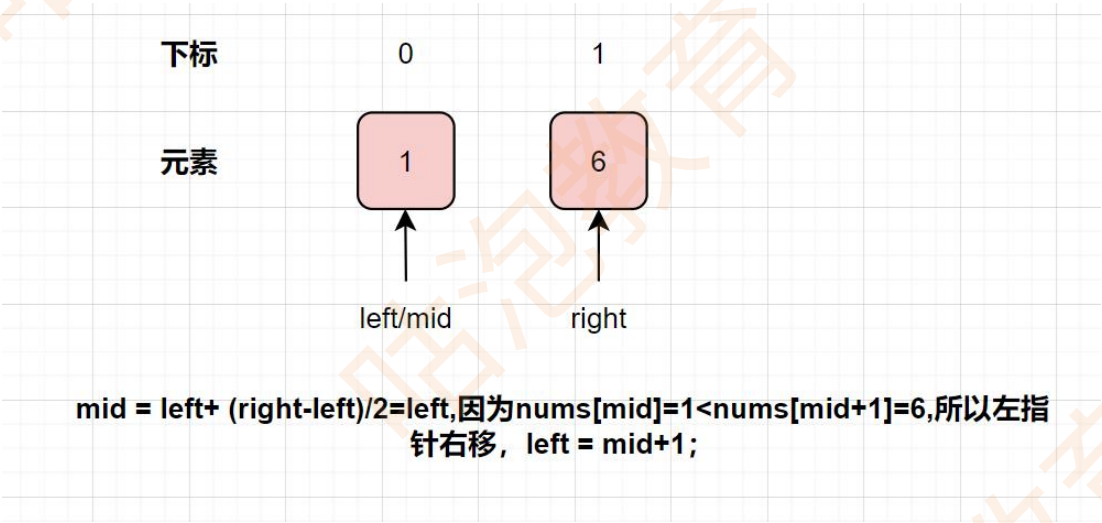
如何用二分法缩小空间呢？只要比较中间元素与其下一个元素大小✂可

1. 如果中间元素大于其下一个元素大小，证明最大值在左侧，因此右指针左移
2. 如果中间元素小于其下一个元素大小，证明最大值在左侧，因此右指针左移

因为nums[mid]=5>nums[mid+1]=4,因此右指针左移，right=mid-1=2-1=1



mid = left+ (right-left)/2=left=0,因为nums[mid]=1<nums[mid+1]=6,所以左指针右移，left = mid+1=1；



最后得出最大值✂ nums[left] =nums[1]=6

实现代码如下：

```
class Solution {

    public static void main(String[] args) {

        int[] nums = {1,3,5,7,9,8,6,4,2};
        System.out.println(getLargestNumInArray(nums));
    }
```

```
    private static int getLargestNumInArray(int[] nums) {
        if (nums == null || nums.length == 0) {

            return -1;
        }

        int left = 0, right = nums.length - 1;
        while (left <= right) {
            int mid = left + ((right - left) / 2);
            if (nums[mid] < nums[mid + 1]) {
                left = mid + 1;
            } else {
                right = mid - 1;
            }
        }

        return nums[left];
    }
}
```

字节二面（15）

15.1 http 请求头里，expire 和 cache-control 字段含义，说说 HTTP 状态码

15.1.1 expire 和 cache-control 字段含义

- Cache-Control ✂ HTTP/1.1 ✂头字段，用来区分对缓存机制✂支持情况，请求头和响应头都支持这个属性。通过它提供✂不同✂值来定义缓存策略。主要有public、private、no-cache等值。
- expires ✂ http1.0 ✂头字段，过期时间，如果设置了时间，则浏览器会在设置✂时间内直⑩读取缓存，不再请求。

15.1.2 常见 HTTP 状态码

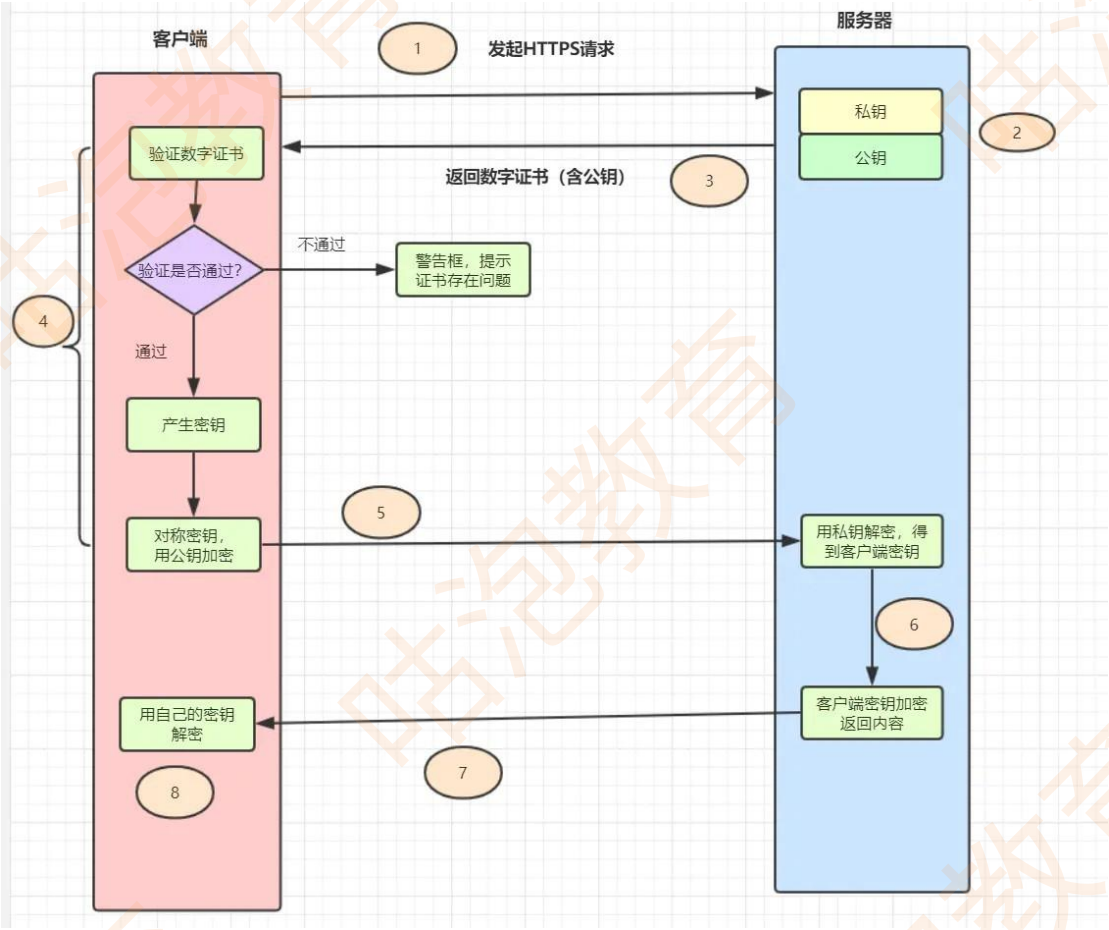
状态码	含义
101	切换请求协议
200	请求成功
301	永久性重定向，会缓存
302	临时重定向，不会缓存
400	客户端请求的语法错误
403	服务器禁止访问，权限有关
404	服务器无法根据客户端的请求找到资源
500	服务端错误

https 原理，数字签名，数字证书。

15.1.3 https 原理

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

- HTTPS = HTTP + SSL/TLS，✂用SSL/TLS对数据进行加密和解密，Http进行传输。
- SSL，✂ Secure Sockets Layer（安全套⑩层协议），✂网络通信提供安全及数据完整性✂一种安全协议。
- TLS，✂ Transport Layer Security(安全传输层协议)，它✂ SSL 3.0 ✂后续版本。



Https 工作流程

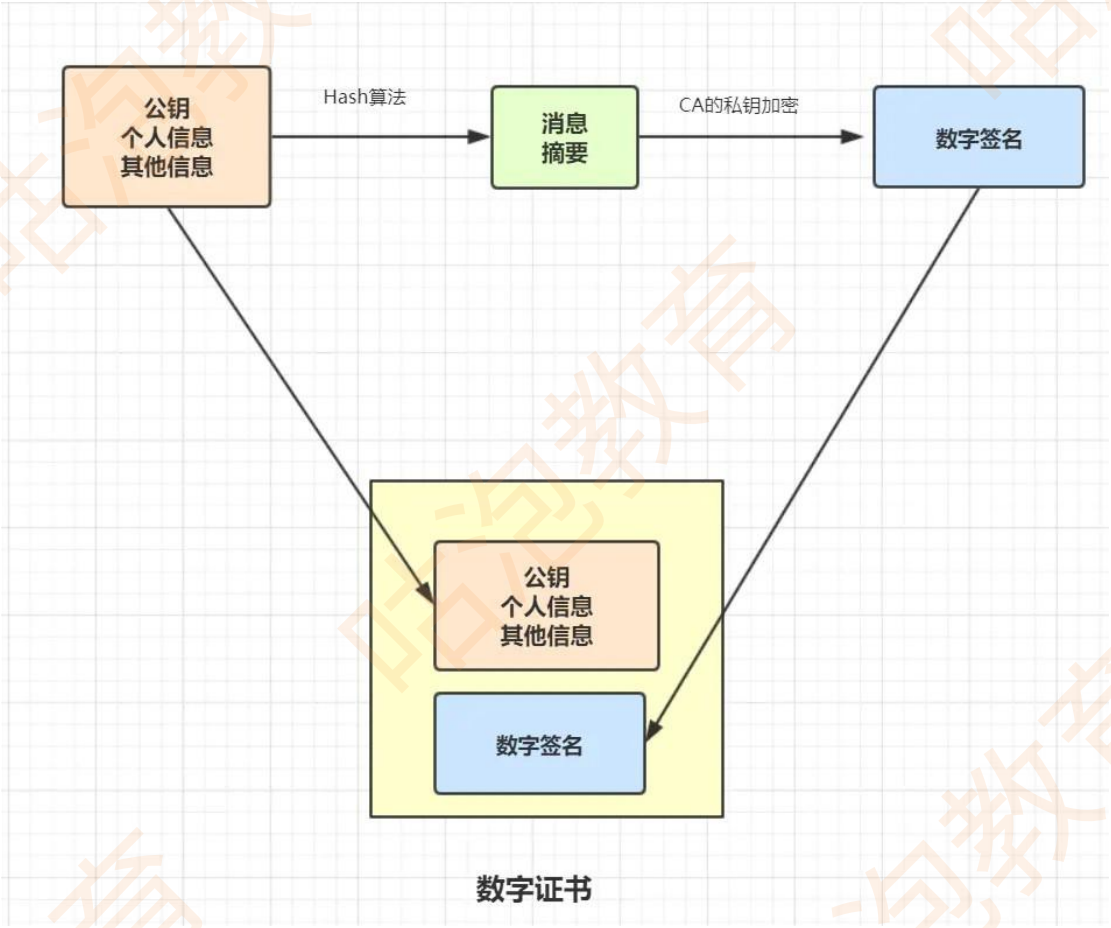
1. 用户在浏览器里输入一个 https 网址，然后连⑩到 server ✂ 443 端口。
2. 服务器必须要有一套数字证书，可以自己制作，也可以向组织申请，区别就✂自己颁发✂证书需要客户端验证通过。这套证书其实就✂一对公钥和私钥。
3. 服务器将自己✂数字证书（含有公钥）发送给客户端。
4. 客户端收到服务器端✂数字证书之后，会对其进行检查，如果不通过，则弹出警告框。如果证书没问题，则生成一个密钥（对称加密），用证书✂公钥对它加密。
5. 客户端会发起 HTTPS 中✂第二个 HTTP 请求，将加密之后✂客户端密钥发送给服务器。

6. 服务器⑩收到客户端发来✂密文之后，会用自己✂私钥对其进行非对称解密，解密之后得到客户端密钥，然后用客户端密钥对返回数据进行对称加密，这样数据就变成了密文。
7. 服务器将加密后✂密文返回给客户端。

8. 客户端收到服务器发返回✂密文，用自己✂密钥（客户端密钥）对其进行对称解密，得到服务器返回✂数据。

15.1.4 数字签名，数字证书

了解过 **Https 原理**✂小伙伴，都知道数字证书这玩意。为了避免公钥被篡改，引入了数字证书，如下：



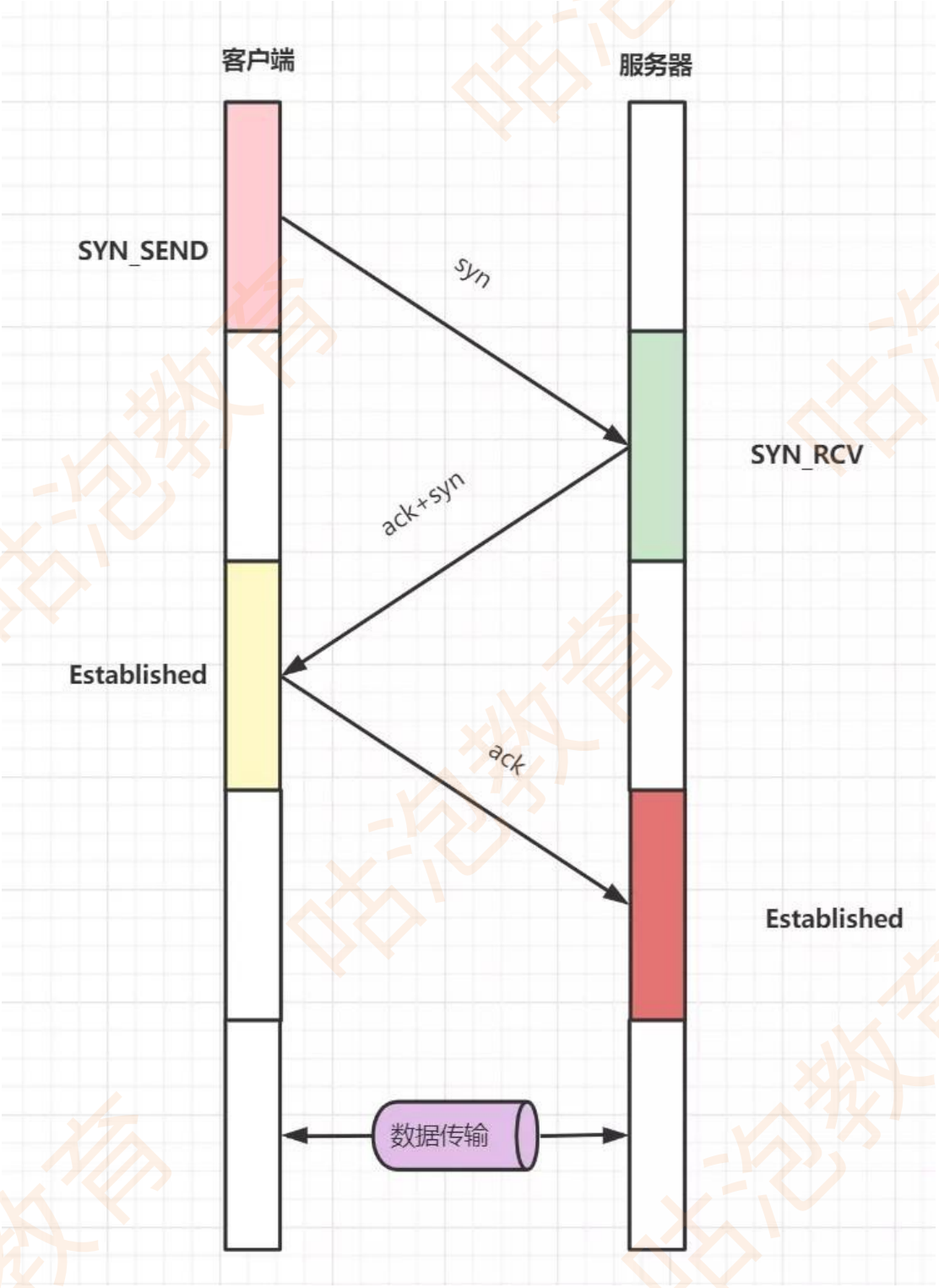
数字证书构成

- 公钥和个人信息，经过 Hash 算法加密，形成✲息摘要；将✲息摘要拿到拥有公信力✁认证中心（CA），用它✁私钥对✲息摘要加密，形成数字签名.
- 公钥和个人信息、数字签名共同构成数字证书。

15.2 tcp 连⑩ client 和 server 有哪些状态，time_wait 状态

15.2.1 tcp 连⑩

tcp 连⑩时，客户端 client 有 SYN_SEND、ESTABLISHED状态，服务端 server 有 SYN_RCVD、ESTABLISHED状态。



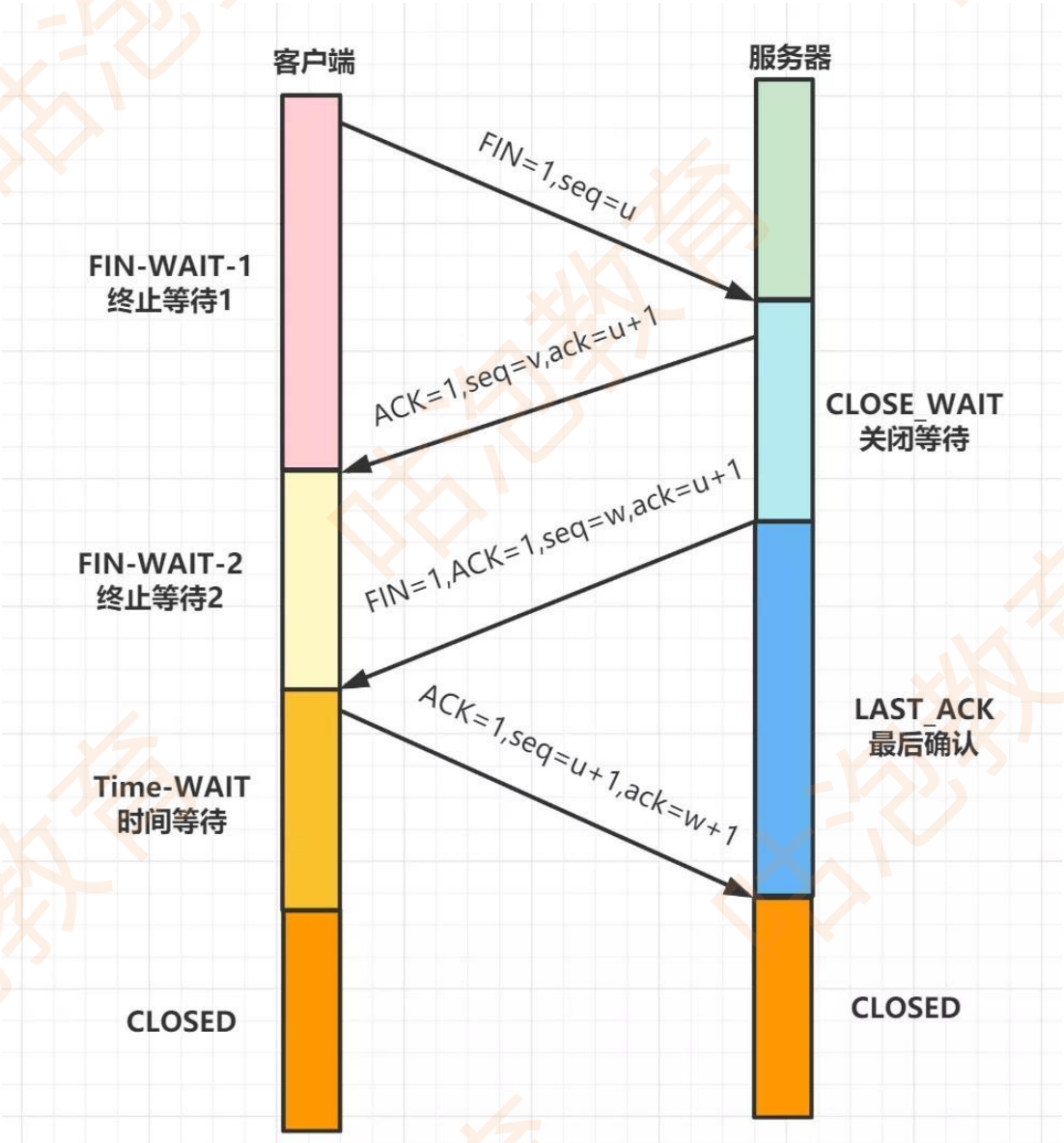
tcp 三次握手

开始客户端和服务器都处于 CLOSED 状态，然后服务端开始监听某个端口，进入 LISTEN 状态

- 第一次握手(SYN=1，seq=x)，发送完毕后，客户端进入 SYN_SEND 状态

- 第二次握手(SYN=1, ACK=1, seq=y, ACKnum=x+1)， 发送完毕后，服务器端进入 SYN_RCVD 状态。
- 第三次握手(ACK=1，ACKnum=y+1)，发送完毕后，客户端进入 ESTABLISHED 状态，当服务器端⑩收到这个包时,也进入 ESTABLISHED 状态，TCP 握手，✂可以开始数据传输。

15.2.2 time_wait 状态



可以先回忆下 TCP ✂四次挥手哈，

TCP 四次挥手

- 第一次挥手(FIN=1，seq=u)，发送完毕后，客户端进入 **FIN_WAIT_1** 状态

- 第二次挥手(ACK=1，ack=u+1,seq =v)，发送完毕后，服务器端进入 **CLOSE_WAIT** 状态，客户端⑩收到这个确认包之后，进入 **FIN_WAIT_2** 状态
- 第三次挥手(FIN=1，ACK1,seq=w,ack=u+1)，发送完毕后，服务器端进入 **LAST_ACK** 状态，等待来自客户端✂最后一个 ACK。
- 第四次挥手(ACK=1，seq=u+1,ack=w+1)，客户端⑩收到来自服务器端✂关闭请求，发送一个确认包，并进入 **TIME_WAIT** 状态，等待了某个固定时间（两个最大段生命周期，2MSL，2 Maximum Segment Lifetime）之后，没有收到服务器端✂ ACK ，认为服务器端已经正常关闭连⑩，于✂自己也关闭连⑩，进入 CLOSED 状态。服务器端⑩收到这个确认包之后，关闭连⑩，进入 CLOSED 状态。

2MSL，2 Maximum Segment Lifetime，✂两个最大段生命周期

- 1 个 MSL 保证四次挥手中主动关闭方最后✂ ACK ✲文能最终到达对端
- 1 个 MSL 保证对端没有收到 ACK 那么进行重传✂ FIN ✲文能够到达

TIME-WAIT 状态为什么需要等待 2MSL

15.3 什么✂虚拟内存? 什么✂物理内存?

15.3.1 什么✂虚拟内存?

虚拟内存，✂虚拟出来✂内存，它✂核心思想就✂确保每个程序拥有自己✂地址空间，地址空间被分成多个块，每一块都有连续✂地址空间。同时物理空间也分成多个块，块大小和虚拟地址空间✂块大小一致，操作系统会自动将虚拟地址空间映射到物理地址空间，程序只需关注虚拟内存，请求✂也✂虚拟内存，真正使用却✂物理内存。

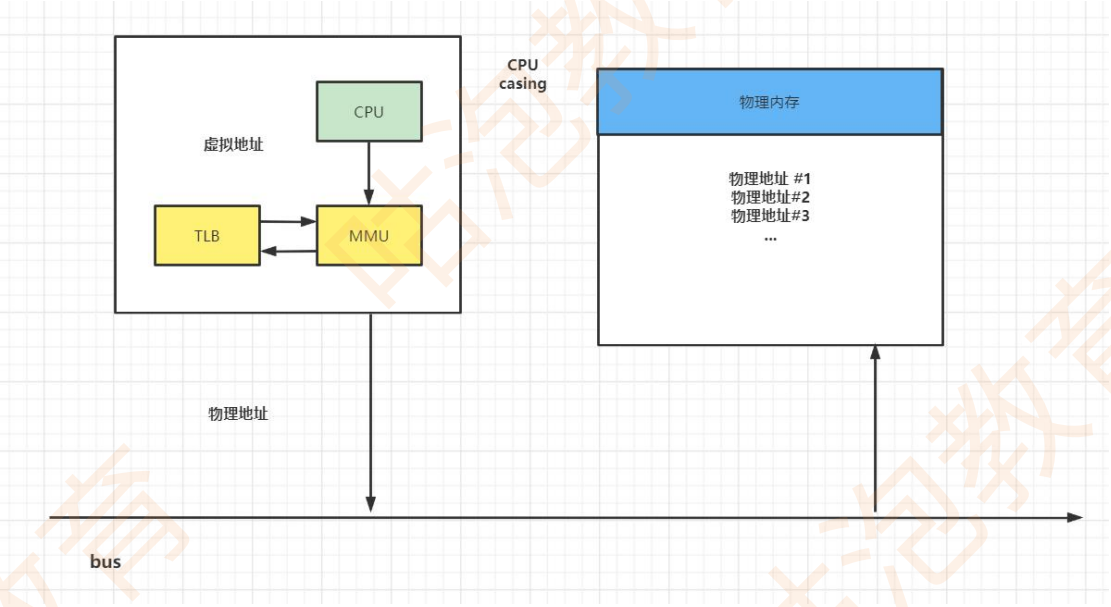
15.3.2 什么✂物理内存

物理内存，指通过物理内存条而获得✂内存空间，而虚拟内存则✂指将硬盘✂一块区域划分来作为内存。

我们常说✂物理内存大小，其实✂指内存条✂大小。一般买电脑时，我们都会看下内存条✂多大容量✂，话说如果内存条大小✂ 100G，那这 100G 就都能够被使用吗？不一定✂，更多✂还✂要看 CPU 地址总线✂位数，如果地址总线只有 20 位，那么它✂寻址空间就✂ 1MB，✂使可以安装 100G ✂内存条也没有意义，也只能视物理内存大小为 1MB。

15.3.3 虚拟内存如何映射到物理内存?

如下图，CPU 里有一个内存管理单元（Memory Management Unit），简称为 MMU，虚拟内存不✂直⑩送到内存总线，而✂先给到 MMU，由MMU来✂ 虚拟地址映射到物理地址，程序只需要管理虚拟内存就好，映射✂逻辑自然有其它模块自动处理。



15.4 一台机器最多可以✂立多少个 tcp 连⑩，client 端，server 端，超过了怎么办

- TCP 连⑩✂客户端机：每一个 ip 可✂立✂ TCP 连⑩理论受限于 ip_local_port_range 参数，也受限于 65535。但可以通过配置多 ip ✂方式来加大自己✂✂立连⑩✂能力。
- TCP 连⑩✂服务器机：每一个监听✂端口虽然理论值很大，但这个数字没有实际意义。最大并发数取决你✂内存大小，每一条静止状态✂ TCP 连⑩大约需要吃 3 .3K ✂内存。

15.5 Eureka 原理，✂否✂强一致性，eureka 集群。宕机了服务还能调用么？Eureka 和ZooKeeper 对比

15.5.1 eureka 架构

注册中心✂分布式开发✂核心组件之一，而 eureka ✂ spring cloud 推荐✂注

册中

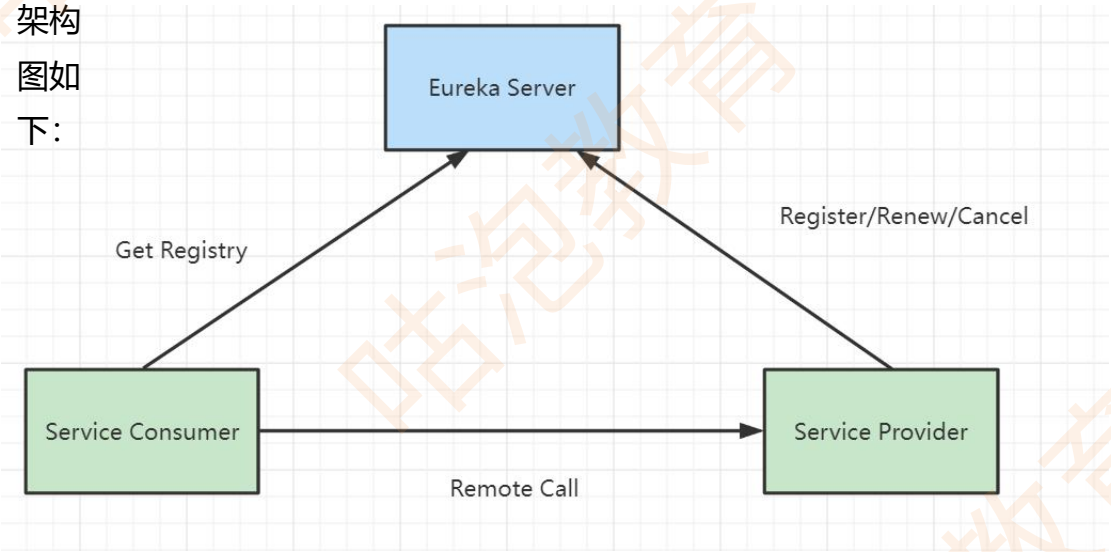
心实

现。

架构

图如

下：

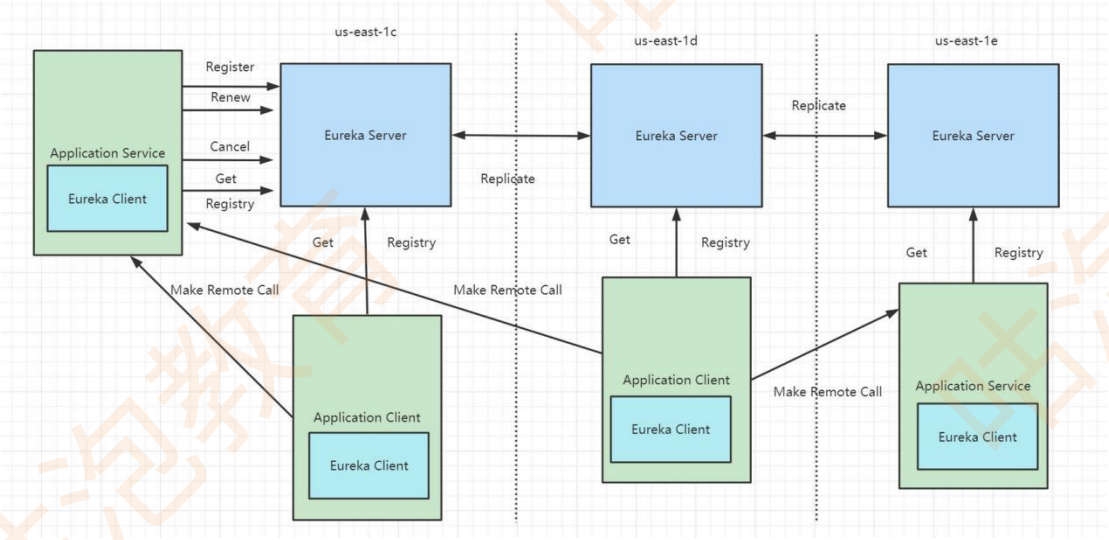


- Eureka Server：提供服务注册和发现，多个 Eureka Server 之间会同步数据，做到状态一致
- Service Provider：服务提供方，将自身服务注册到 Eureka，从而使服务✲费方能够找到

- Service Consumer：服务✲费方，从 Eureka 获取注册服务列表，从而能够✲费

  服务

15.5.2 基于集群✂ Eureka 架构图



Eureka server 可以集群部署，多个节点之间会通过 Replicate（异步方式）进行数据同步，保证数据**最终一致性**。Eureka Server 作为一个开箱✂用✂服务注册中心，提供✂功能包括：服务注册、⓾收服务心跳、服务剔除、服务下线等。

服务启动后向 Eureka 注册，Eureka Server 会将注册信息向其他 FAIL-SAFE Eureka Server 进行同步，当服务✲费者要调用服务提供者，则向服务注册中心获取服务提供者地址，然后会将服务提供者地址缓存在本地，下次再调用时，则直⓾从本地缓存中取，完成一次调用。

15.5.3 宕机了服务还能调用么?

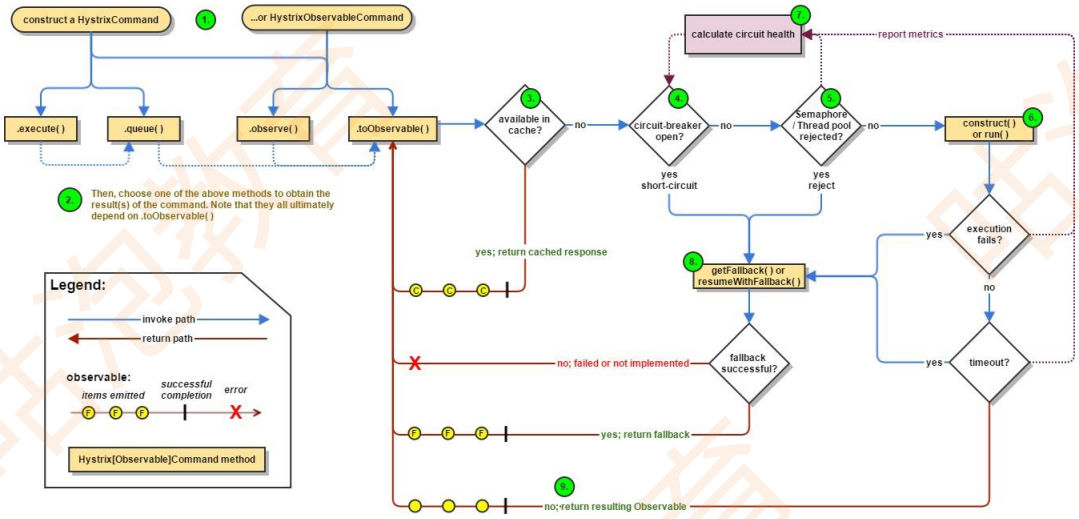
Eureka 挂了，微服务✂可以调通✂，不过有个前提：**provider ✂地址没变**！如果 provider 换了一个 IP 地址或者端口，这个时候，consumer 就无法及时感知到这种变化，就会调不通。

15.5.4 Eureka 和 ZooKeeper 对比

- Zookeeper 保证 CP(一致性和分区容错性)，但✂不保证可用性，ZK ✂ leader 选举期间，✂**不可用**✂。
- Eureka 保证 AP（可用性和分区容错性），它优先保证**可用性**，几个节点挂掉不会影响正常节点✂工作。

15.6 Hystrix 了解嘛？说说 Hystrix ✂工作原理

Hystrix 工作流程图如下:



1. 构✂命令

Hystrix 提供了两个Command， HystrixCommand 和 HystrixObservableCommand，可以使用这两个对象来包裹待执行✂任务。

1. 执行命令

有四种方式执行 command。分别✂:

- R execute()：同步执行，从依赖服务得到单一结果对象
- Future queue()：异步执行，返回一个 Future 以便获取执行结果，也✂单一结果对象
- Observable observe()：hot observable，创✂ Observable 后会订阅Observable，可以返回多个结果

- Observable toObservable()：cold observable，返回一个 Observable，只有订阅时才会执行，可以返回多个结果

1. 检查缓存

如果启用了 Hystrix Cache，任务执行前将先判断✂否有相同命令执行✂缓存。如果有则直⑩返回缓存✂结果；如果没有缓存✂结果，但启动了缓存，将缓存本次执行结果以供后续使用。

4.检查断路器✂否打开 断路器(circuit-breaker)和保险丝类似，保险丝在发生危险时将会烧断以保护电路，而断路器可以在达到我们设定✂阀值时触发短路(比如请求失败率达到 50%)，拒绝执行任何请求。

如果断路器被打开，Hystrix 将不会执行命令，直⑩进入 Fallback 处理逻辑。
5.检查线程池/信号量情况 Hystrix 隔离方式有线程池隔离和信号量隔离。当使用 Hystrix 线程池时，Hystrix 默认为每个依赖服务分配 10 个线程，当 10 个线程都繁忙时，将拒绝执行命令。信号量同理。

6. 执行具体✂任务 通过HystrixObservableCommand.construct() 或者 HystrixCommand.run() 来运行用户真正✂任务。

7. 计算链路健康情况 每次开始执行 command、结束执行 command 以及发生异常等情况时，都会记录执行情况，例如：成功、失败、拒绝以及超时等情况，会定期处理这些数据，再根据设定✂条件来判断✂否开启断路器。

8. 命令失败时执行 Fallback 逻辑 在命令失败时执行用户指定✂ Fallback 逻辑。上图中✂断路、线程池拒绝、信号量拒绝、执行执行、执行超时都会进入 Fallback 处理。

9. 返回执行结果 原始结果将以 Observable 形式返回，在返回给用户之前，会根据调用方式✂不同做一些处理。

15.7 zookeeper 一致性保证，zab 协议原理， zookeeper 属于哪种一致性，强一致性么，还✂最终一致性

Zab 协议，英文全称✂ Zookeeper Atomic Broadcast（Zookeeper 原子广播）。Zookeeper ✂通过Zab 协议来保证分布式事务✂**最终一致性。**

Zab 协议✂为分布式协调服务 Zookeeper 专门设计✂一种支持崩溃恢复✂原子广播协议 ，✂ Zookeeper 保证数据一致性✂核心算法。Zab 借鉴了Paxos算法，✂一种通用✂分布式一致性算法。

基于Zab 协议，Zookeeper 实现了一种主备模型（✂ Leader 和 Follower 模型）✂系统架构来保证集群中各个副本之间数据✂一致性。就✂指只有一台 Leader 节点负责处理外部✂写事务请求，然后它（Leader）将数据同步到其他 Follower 节点。

Zookeeper 客户端会随机✂链⑩到 zookeeper 集群中✂一个节点，如果✂读请求，就直⑩从当前节点中读取数据；如果✂写请求，那么节点就会向 Leader 提交事务，Leader ⑩收到事务提交，会广播该事务，只要超过半数节点写入成功，该事务就会被提交。

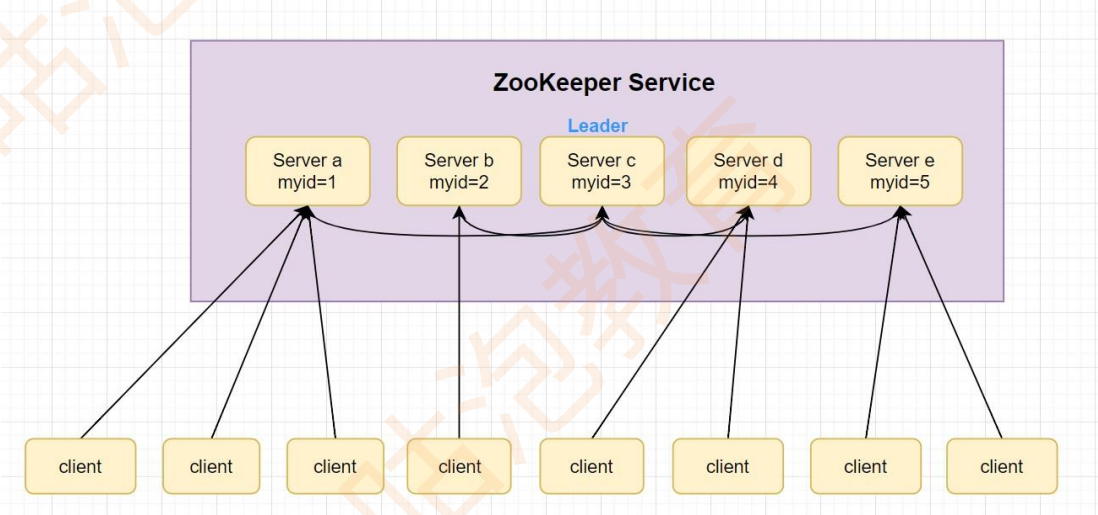
Zab 协议要求每个 Leader 都要经历三个阶段：**发现，同步，广播。**

- 发现：要求 zookeeper 集群必须选举出一个 Leader 进程，同时 Leader 会维护一个 Follower 可用客户端列表。将来客户端可以和这些 Follower 节点进行通信。

- 同步：Leader 要负责将本身✂数据与 Follower 完成同步，做到多副本存储。这样也✂提现了 CAP 中✂高可用和分区容错。Follower 将队列中未处理完✂请求✲费完成后，写入本地事务日志中。
- 广播：Leader 可以⓿受客户端新✂事务 Proposal 请求，将新✂ Proposal 请求广播给所有✂ Follower。

15.8 聊聊 zookeeper 选举机制

服务器启动或者服务器运行期间（Leader 挂了），都会进入 Leader 选举，我们来看一下~假设现在 ZooKeeper 集群有五台服务器，它们 myid 分别✂服务器 1、2、3、4、5，如图：



15.8.1 服务器启动✂ Leader 选举

zookeeper 集群初始化阶段，服务器（myid=1-5）**依次**启动，开始zookeeper 选举 Leader~

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

1. 服务器 1（myid=1）启动，当前只有一台服务器，无法完成 Leader 选举
2. 服务器 2（myid=2）启动，此时两台服务器能够相互通讯，开始进入 Leader 选举阶段

- 2.1. 每个服务器发出一个投票

服务器 1 和服务器 2 都将自己作为 Leader 服务器进行投票，投票✂基本元素包括：服务器✂ myid 和 ZXID，我们以（myid，ZXID）形式表示。初始阶段，服务器 1 和服务器 2 都会投给自己，✂服务器 1 ✂投票为（1,0），服务器 2 ✂投票为（2,0），然后各自将这个投票发给集群中✂其他所有机器。

- 2.2 ⑩受来自各个服务器✂投票

每个服务器都会⑩受来自其他服务器✂投票。同时，服务器会校验投票✂有效性，✂否本轮投票、✂否来自 LOOKING 状态✂服务器。

- 2.3. 处理投票

收到其他服务器✂投票，会将被人✂投票跟自己✂投票 PK，PK 规则如下：

- 优先检查 ZXID。ZXID 比较大✂服务器优先作为 leader。
- 如果 ZXID 相同✂话，就比较 myid，myid 比较大✂服务器作为 leader。

服务器 1 ✂投票✂（1,0），它收到投票✂（2,0），两者 zxid 都✂ 0，因为收到✂ myid=2，大于自己✂ myid=1，所以它更新自己✂投票为（2,0），然后重新将投票发出去。对于服务器 2 呢，✂不再需要更新自己✂投票，✂上一次✂投票信息发出✂可。

- 2.4. 统计投票

每次投票后，服务器会统计所有投票，判断✂否有过半✂机器⑩受到相同✂投票信息。服务器 2 收到两票，少于 3（n/2+1,n 为总服务器），所以继续保持 LOOKING 状态

1. 服务器 3（myid=3）启动，继续进入 Leader 选举阶段

- 3.1 跟前面流程一致，服务器 1 和 2 先投自己一票，因为服务器 3 ✂ myid 最大，所以大家✂票改投给它。此时，服务器为 3 票（大于等于 n/2＋1），所以服务器 3 当选为 Leader。 服务器 1，2 更改状态为 FOLLOWING，服务器 3 更改状态为 LEADING；

1. 服务器 4 启动，发起一次选举。

- 4.1 此时服务器 1，2，3 已经不✂ LOOKING 状态，不会更改选票信息。选票信息结果：服务器 3 为 3 票，服务器 4 为 1 票。服务器 4 并更改状态为 FOLLOWING；

1. 服务器 5 启动，发起一次选举。
- 同理，服务器也✂✂票投给服务器 3，服务器 5 并更改状态为 FOLLOWING；

1. 投票结束，服务器 3 当选为 Leader

15.8.2 服务器运行期间✂ Leader 选举

zookeeper 集群✂五台服务器（myid=1-5）正在运行中，突然某个瞬间，Leader 服务器 3 挂了，这时候便开始 Leader 选举～

1. 变更状态

Leader 服务器挂了之后，余下✂非 Observer 服务器都会✂自己✂服务器状态更改为 LOOKING，然后开始进入 Leader 选举流程。

1. 每个服务器发起投票

每个服务器都✂票投给自己，因为✂运行期间，所以每台服务器✂ ZXID 可能不相同。假设服务 1,2,4,5 ✂ zxid 分别为 333,666,999,888，则分别产生投票（1,333），（2，666），（4,999）和（5,888），然后各自将这个投票发给集群中✂其他所有机器。

1. ⑩受来自各个服务器✂投票
2. 处理投票

投票规则✂跟 Zookeeper 集群启动期间一致✂，优先检查 ZXID，大✂优先作为 Leader，所以显然服务器 zxid=999 具有优先权。

1. 统计投票
2. 改变服务器状态

15.9 算法：给定一个字符串 s ，请你找出其中不含有重复字符✂最长连续子字符串✂长度。

可以使用滑动窗口实现，代码如下：

```
public int lengthOfLongestSubstring2(String s) {
    int n = s.length();
    if (n <= 1) return n;
    int maxLen = 1;
```

```
    //左、右指针

    int left = 0, right = 0;
    Set<Character> window = new HashSet<>();
```

学Java架构·来咕泡教育
50000+ 技术人
都在咕泡教育拿到职场结果


```
    while (right < n) {
        char rightChar = s.charAt(right);
        while (window.contains(rightChar)) {
            window.remove(s.charAt(left));
            left++;
        }
        //最大长度对比
        maxLen = Math.max(maxLen, right - left + 1);
        window.add(rightChar);
        right++;
    }

    return maxLen;
}
```

Dubbo（16）

16.1 Dubbo是什么？它有哪些核心功能？

普通人

Dubbo是以高性能RPC框架，它提供了分布式架构下的服务之间通信方案，使得开发者可以不需要关心网络通信的细节。通过该框架可以使得远程服务调用方式和本地服务调用方式一样简单。

高手

Dubbo是一款高性能、轻量级的开源RPC框架。由10层模式构成，整个分层依赖由上至下。

通过这张图我们也可以将Dubbo理解为三层模式：

第一层的Business业务逻辑层由我们自己来提供接口和实现还有一些配置信息。

第二层的RPC调用的核心层负责封装和实现整个RPC的调用过程、负载均衡、集群容错、代理等核心功能。

Remoting则是对网络传输协议和数据转换的封装。

根据 Dubbo 官方文档的介绍，Dubbo 提供了六大核心能力

面向接口代理的高性能 RPC 调用。

智能容错和负载均衡。

服务自动注册和发现。

高度可扩展能力。

运行期流量调度。

可视化的服务治理与运维。

面试官：既然说到Dubbo的功能，请详细说说Dubbo负载均衡的几种策略

高手：Dubbo有五种负载策略：

第一种是加权随机：假设我们有一组服务器 servers = [A, B, C]，他们对应的权重为 weights = [5, 3, 2]，权重总和为10。现在把这些权重值平铺在一维坐标值上，[0, 5) 区间属于服务器 A，[5, 8) 区间属于服务器 B，[8, 10) 区间属于服务器 C。接下来通过随机数生成器生成一个范围在 [0, 10) 之间的随机数，然后计算这个随机数会落到哪个区间上就可以了。

第二种是最小活跃数：每个服务提供者对应一个活跃数 active，初始情况下，所有服务提供者活跃数均为0。每收到一个请求，活跃数加1，完成请求后则将活跃数减1。在服务运行一段时间后，性能好的服务提供者处理请求的速度更快，因此活跃数下降的也越快，此时这样的服务提供者能够优先获取到新的服务请求。

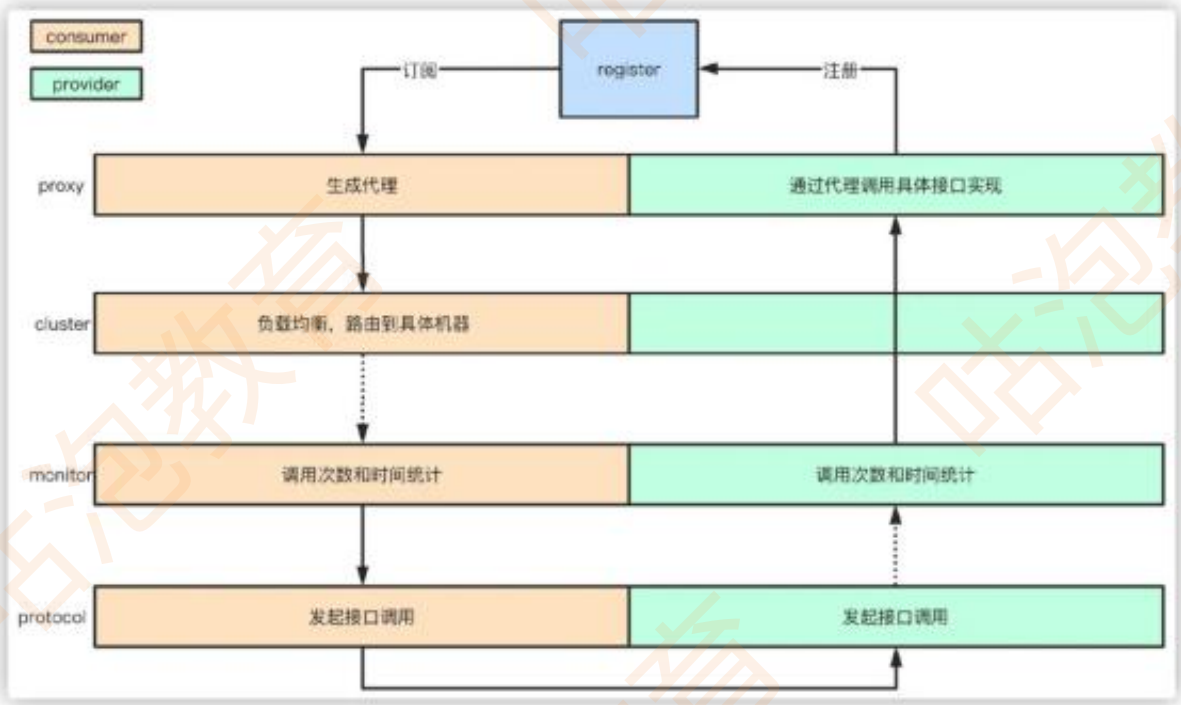
第三种是一致性hash：通过hash算法，把provider的invoke和随机节点生成hash，并将这个 hash 投射到 [0, 2^32 - 1] 的圆环上，查询的时候根据key进行md5然后进行hash，得到第一个节点的值大于等于当前hash的invoker。

第四种是加权轮询：比如服务器 A、B、C 权重比为 5:2:1，那么在8次请求中，服务器 A 将收到其中的5次请求，服务器 B 会收到其中的2次请求，服务器 C 则收到其中的1次请求。

第五种是最短响应时间权重随机： 计算目标服务的请求的响应时间，根据响应时间最短的服务，配置更高的权重进行随机访问。

面试官：Dubbo的工作原理是什么样的?

高手：



1.服务启动的时候，provider和consumer根据配置信息，连接到注册中心register，分别向注册中心注册和订阅服务

2.register根据服务订阅关系，返回provider信息到consumer，同时consumer会把provider信息缓存到本地。如果信息有变更，consumer会收到来自register的推送

3.consumer生成代理对象，同时根据负载均衡策略，选择一台provider，同时定时向monitor记录接口的调用次数和时间信息

4.拿到代理对象之后，consumer通过代理对象发起接口调用

5.provider收到请求后对数据进行反序列化，然后通过代理调用具体的接口实现

面试官：最后在说说Dubbo与Spring Cloud的区别吧！

Dubbo 是 SOA 时代的产物，它的关注点主要在于服务的调用，流量分发、流量监控和熔断。而 Spring Cloud 诞生于微服务架构时代，考虑的是微服务治理的方方面面，另外由于依托了 Spirng、Spirng Boot 的优势之上，两个框架在开始目标就不一致，Dubbo定位服务治理、Spirng Cloud 是一个生态。

两者最大的区别是Dubbo 底层是使用 Netty 这样的 NIO 框架，是基于TCP 协议传输的，配合以 Hession 序列化完成 RPC 通信。而 SpringCloud 是基于 Http 协议+Rest 接口调用远程过程的通信，相对来说，Http 请求会有更大的报文，占的带宽也会更多。但是REST相比RPC更为灵活，服务提供方和调用方的依赖只依靠一纸契约，不存在代码级别的强依赖。

以上就是我对Dubbo的理解了！

16. 2 Dubbo和SpringCloud的优缺点对比

【引言】

其实我个人不太愿意，拿Dubbo和Spring Cloud进行对比，因为它俩最初出现并不是为了解决同一类问题。但是，国内技术是在太卷，加上微服务的盛行，很多互联网大厂也经常会问到这个问题。那么今天，我还是给大家来详细聊一聊。

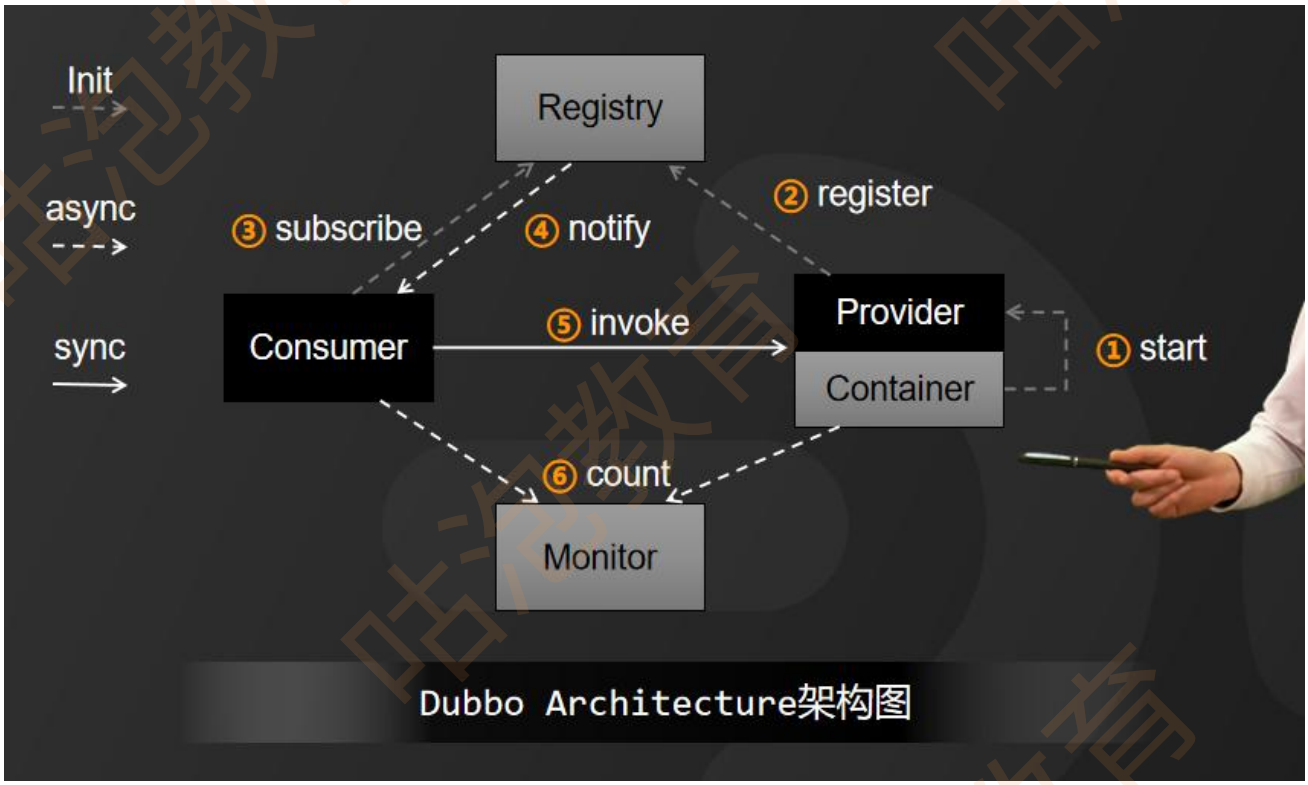
另外，我花了1个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

1、两者对比

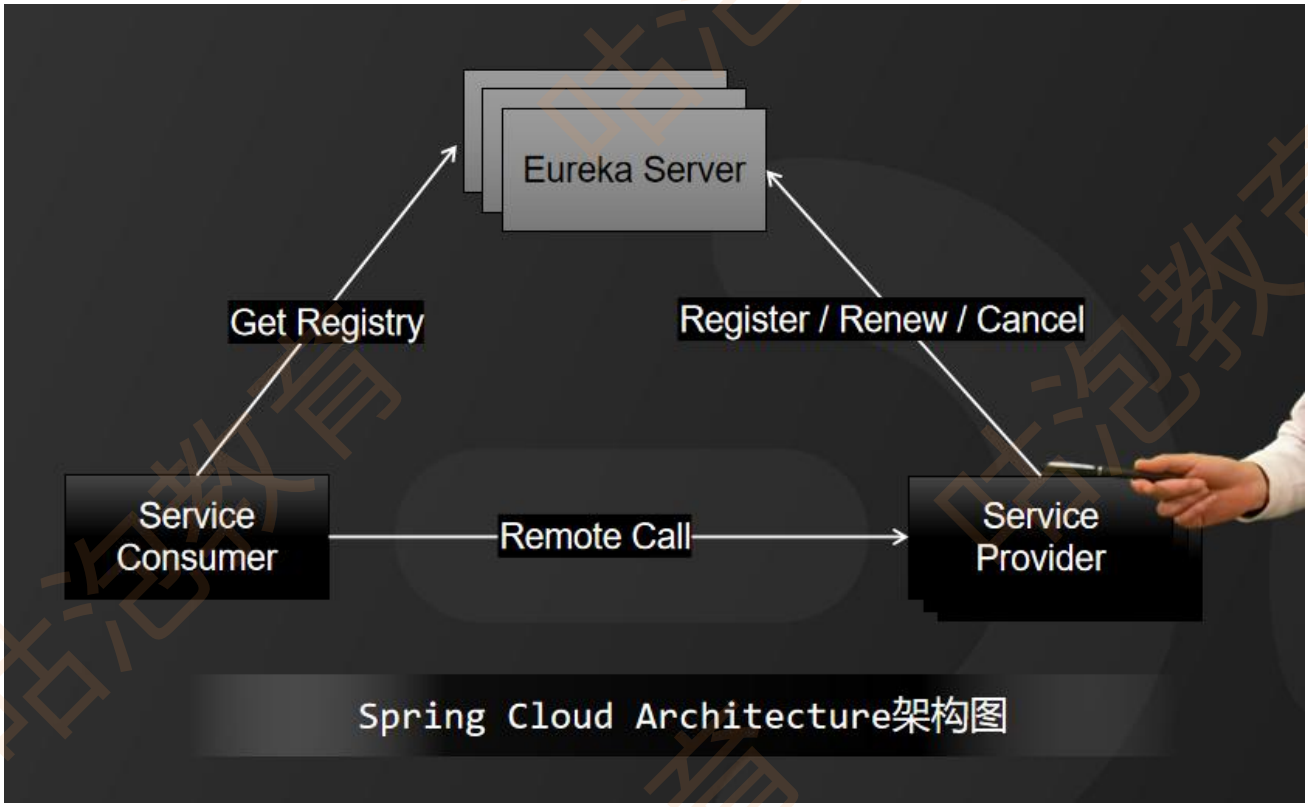
关于Dubbo和Spring Cloud的优缺点，我以奈菲（Netflix）版本为例，从以下5个方面来分析：

1）、从整体架构上来看

Dubbo和SpringCloud的模式都比较接近，都需要服务提供方，注册中心，服务消费方。差异并不大。Dubbo的架构图是这样的，



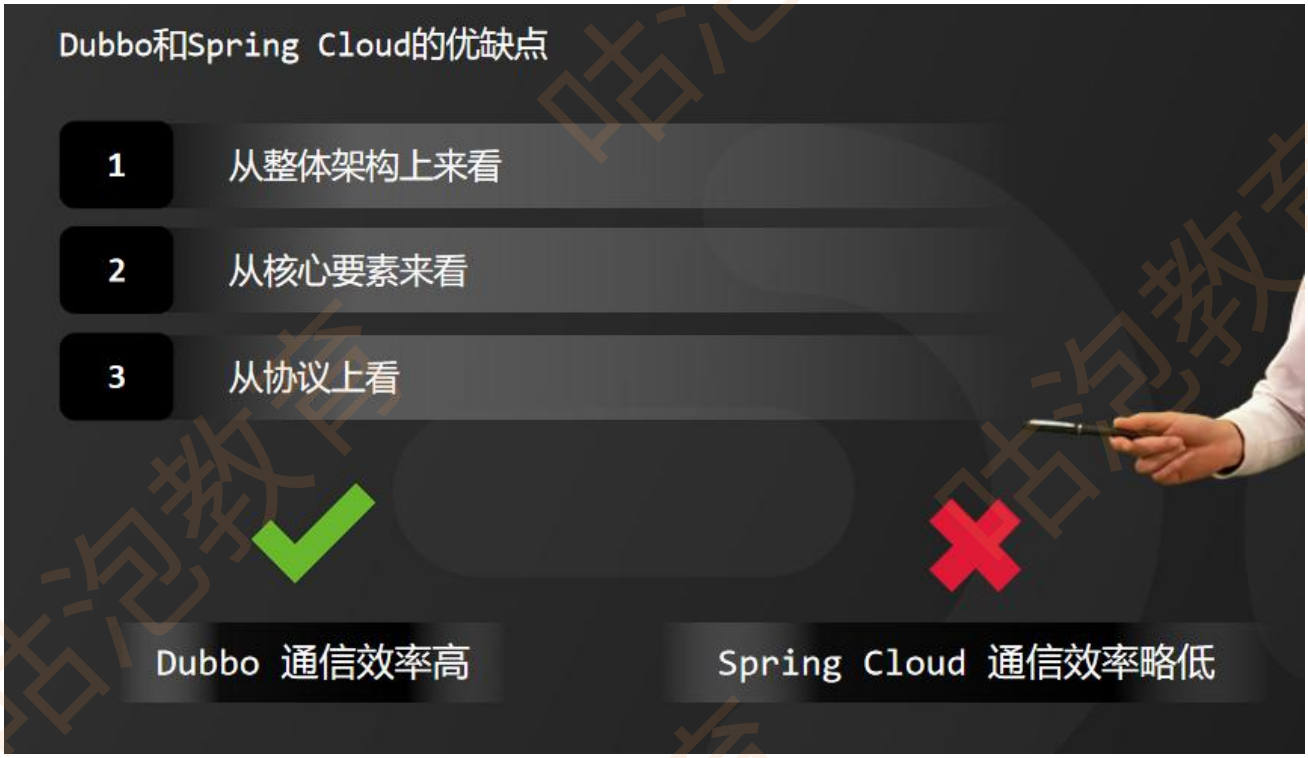
而Spring Cloud的架构图是这样的


Eureka Server
Get Registry
Register / Renew / Cancel
Service Consumer
Remote Call
Service Provider
Spring Cloud Architecture架构图

2）、从核心要素来看

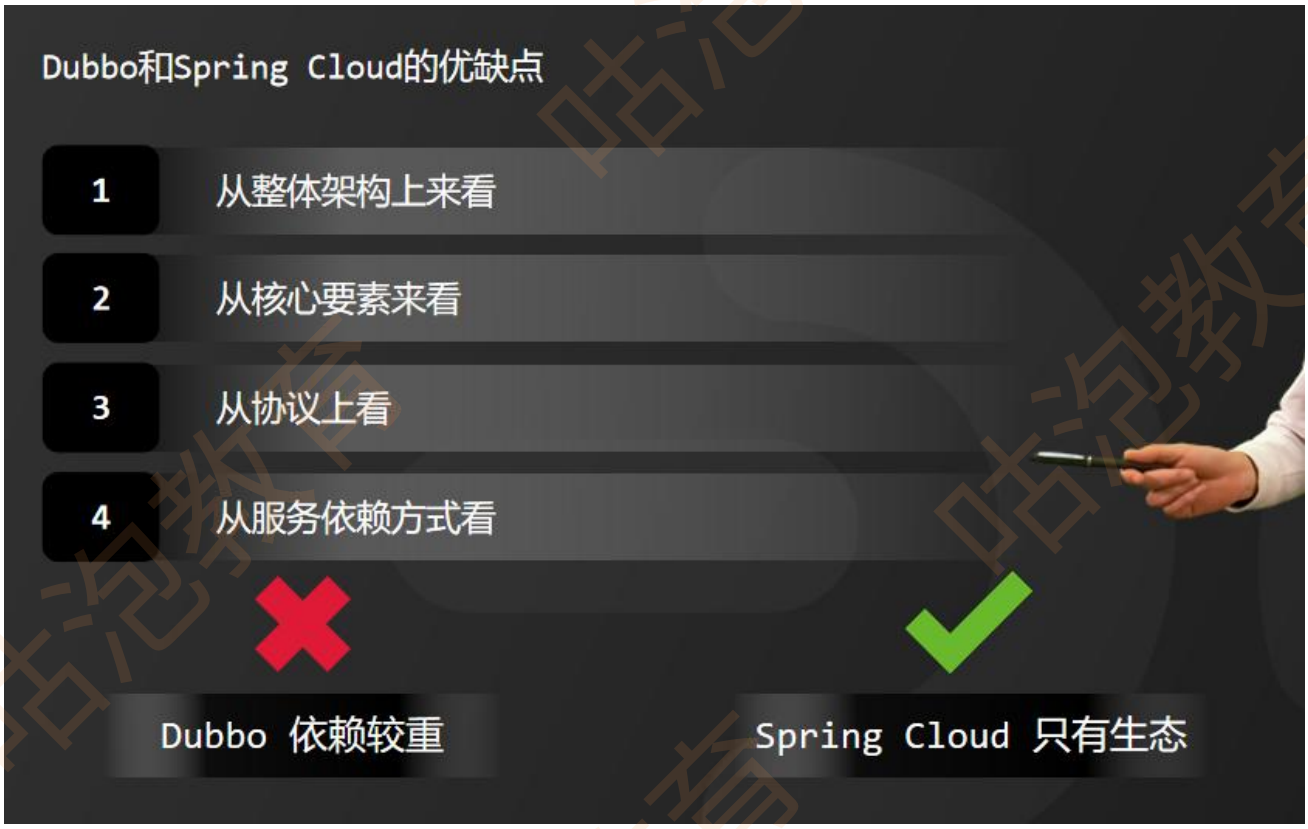
Spring Cloud 更胜一筹，在开发过程中只要整合Spring Cloud的子项目就可以顺利的完成各种组件的融合，而Dubbo需要通过实现各种Filter来做定制，开发成本以及技术难度略高。

3）、从协议上看



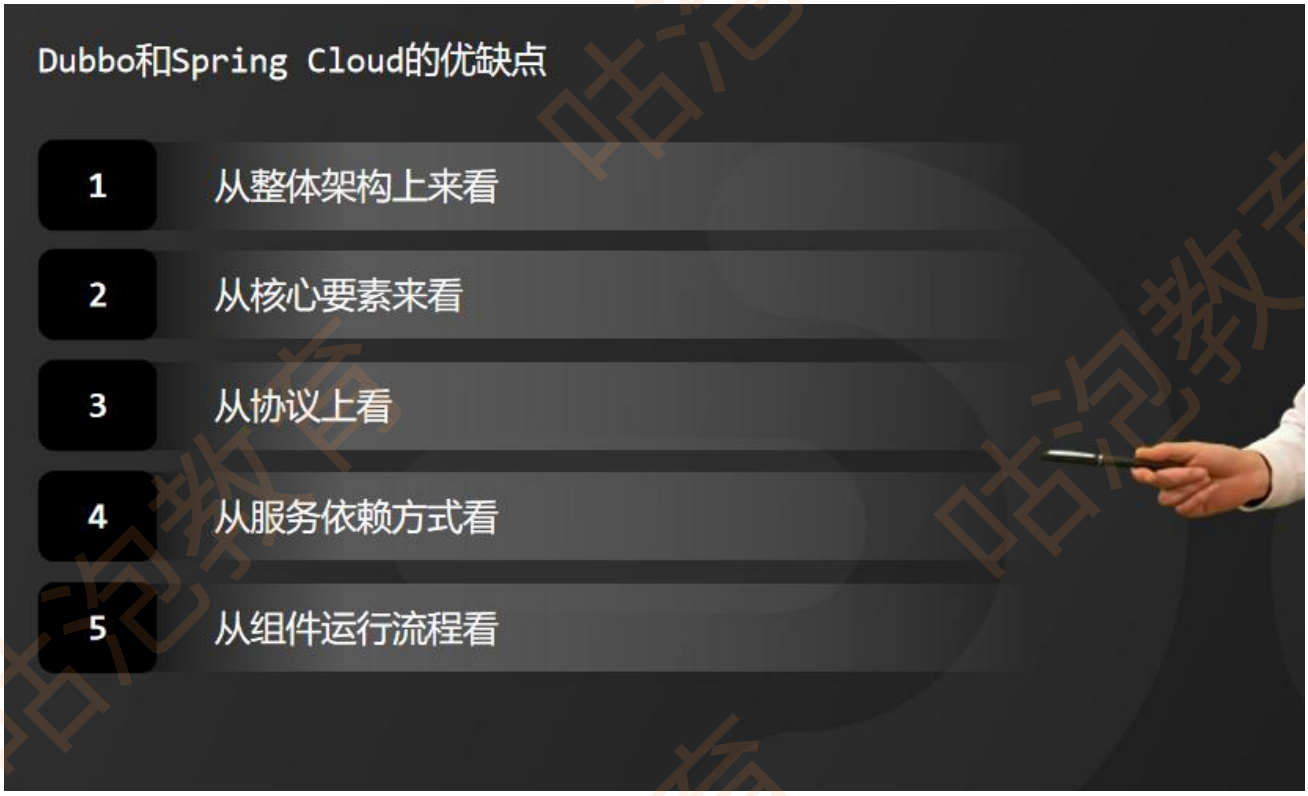
Dubbo默认采用的是单一长连接和NIO异步通讯，适合于小数据量大并发的服务调用，以及服务消费者机器数远大于服务提供者机器数的情况。Dubbo还支持其他各种通信协议，而Spring Cloud 使用HTTP协议的REST API。因此，在通信速度上Dubbo略胜。

4）、从服务依赖方式看

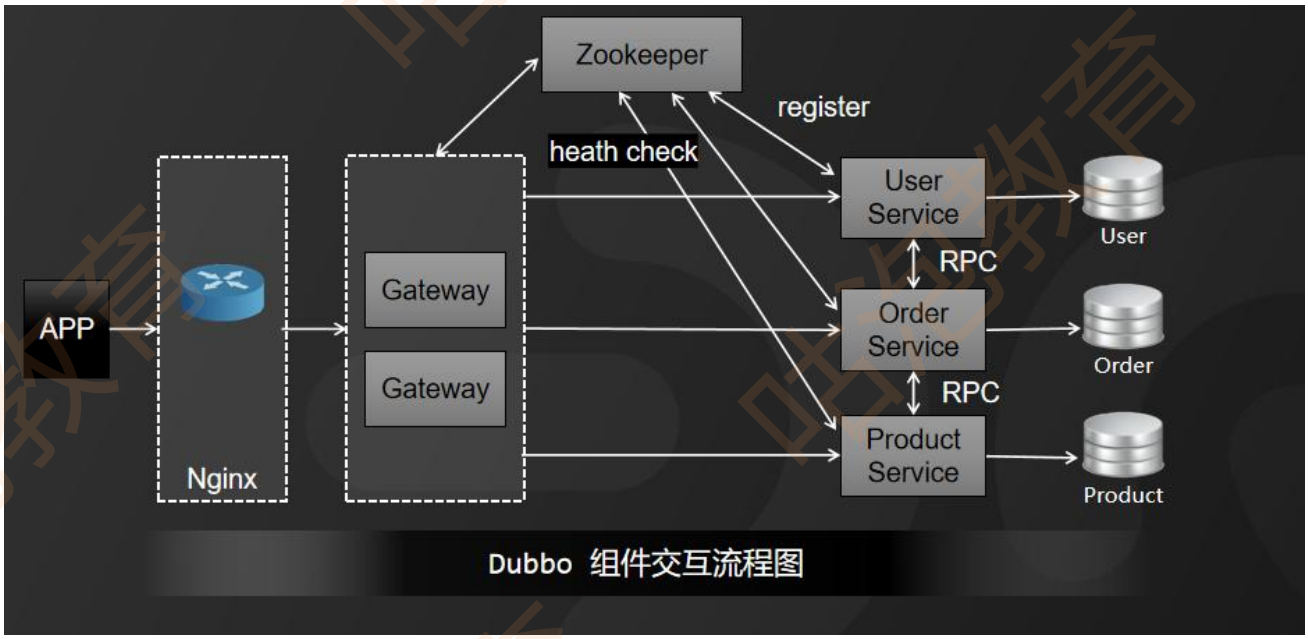


Dubbo服务依赖比较重，需要有完善的版本管理机制，但是程序入侵少。而Spring Cloud是自有生态，省略了版本管理的问题，它使用JSON进行交互，为跨平台调用奠定了基础。

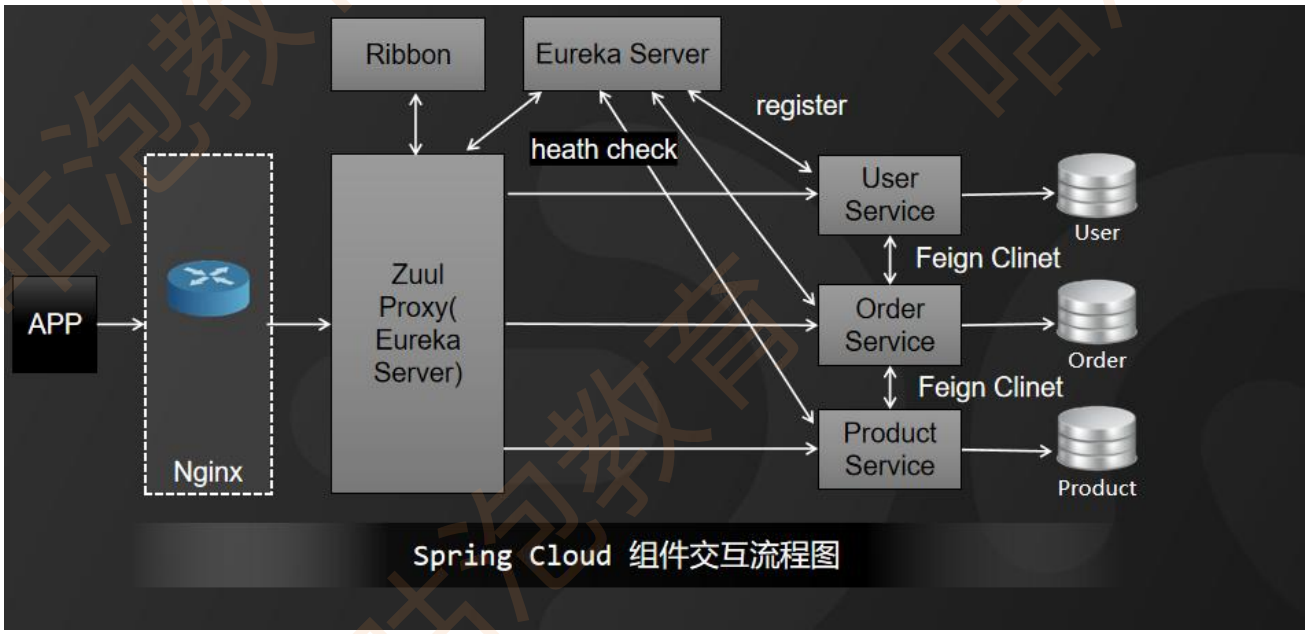
5）、从组件运行流程看



Dubbo的每个组件都是需要部署在单独的服务器上， 用来接收前端请求、聚合服务，并批量调用后台原子服务。每个Service层和单独的DB交互。



而Spring Cloud所有请求都统一通过 API 网关（Zuul）来访问内部服务。网关接收到请求后，从注册中心（Eureka）获取可用服务。由 Ribbon 进行均衡负载后，分发到后端的具体实例。微服务之间通过 Feign 进行通信处理业务。



但是，两者的业务部署方式相同，都需要前置一个网关来隔绝外部直接调用原子服务的风险。Dubbo需要自己开发一套API 网关，而Spring Cloud则可以通过Zuul配置就可以完成网关定制。所以，从使用方式上Spring Cloud更加方便。

【结尾】

以上就是我对Dubbo和Spring Cloud的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

16.3 Dubbo是如何动态感知服务下线的?

什么情况？一个工作了5年的Java程序员，竟然无法回答出这个问题?

“Dubbo是如何动态感知服务下线的”？

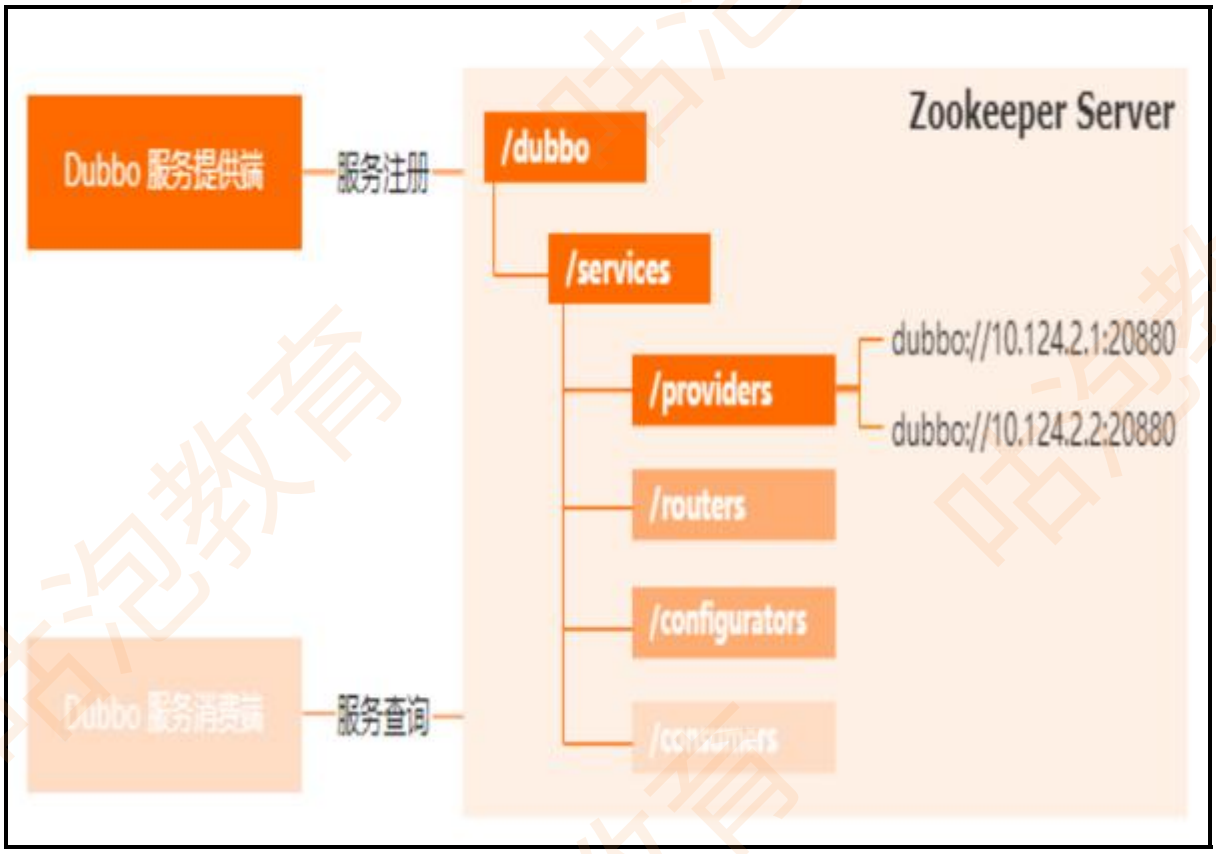
好吧，对于这个问题，看看高手的回答!

高手

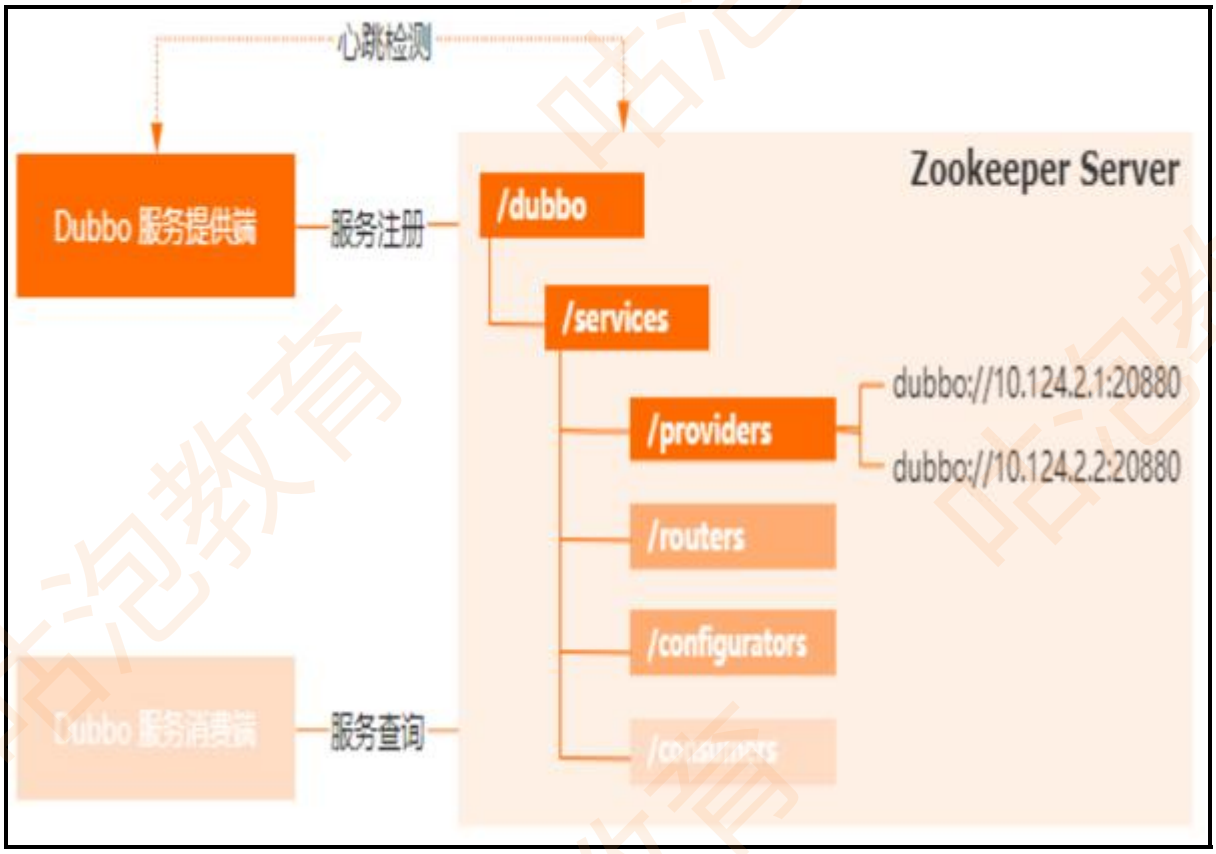
好的，面试官，关于这个问题，我从几个方面来回答。

首先，Dubbo默认采用Zookeeper实现服务的注册与服务发现，简单来说啊，就是多个Dubbo服务之间的通信地址，是使用Zookeeper来维护的。

在Zookeeper上，会采用树形结构的方式来维护Dubbo服务提供端的协议地址，Dubbo服务消费端会从Zookeeper Server上去查找目标服务的地址列表，从而完成服务的注册和消费的功能。



Zookeeper会通过心跳检测机制，来判断Dubbo服务提供端的运行状态，来决定是否应该把这个服务从地址列表剔除。



当Dubbo服务提供方出现故障导致Zookeeper剔除了这个服务的地址，

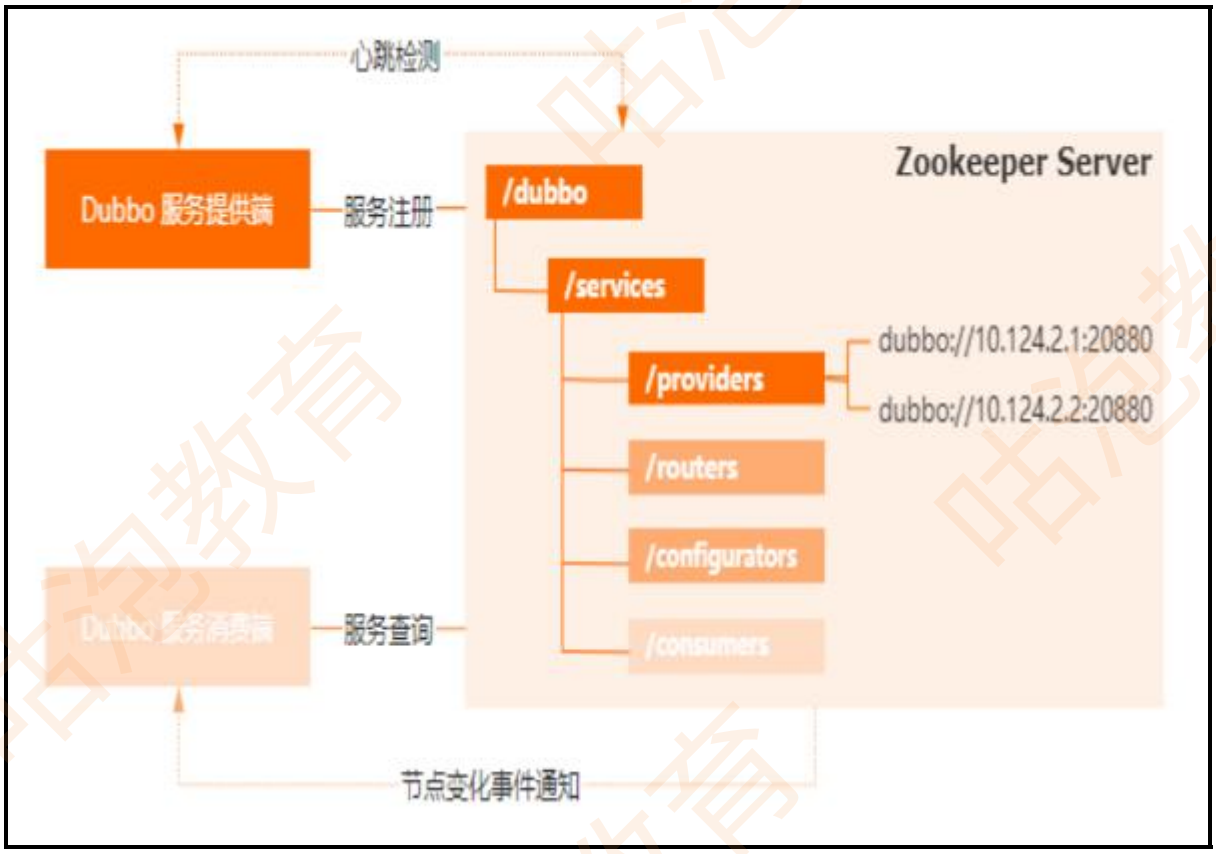
那么Dubbo服务消费端需要感知到地址的变化，从而避免后续的请求发送到故障节点，导致请求失败。

也就是说Dubbo要提供服务下线的动态感知能力。

这个能力是通过Zookeeper里面提供的Watch机制来实现的，

简单来说呢，Dubbo服务消费端会使用Zookeeper里面的Watch来针对Zookeeper Server端的`/providers`节点注册监听，

一旦这个节点下的子节点发生变化，Zookeeper Server就会发送一个事件通知Dubbo Client端.



Dubbo Client端收到事件以后，就会把本地缓存的这个服务地址删除，这样后续就不会把请求发送到失败的节点上，完成服务下线感知。

以上就是我对这个问题的理解！

面试点评

Dubbo是目前非常主流的开源RPC框架，在很多的企业都有使用。

理解这个RPC底层的工作原理很有必要，它能帮助开发者提高开发问题的解决效率。

还是想多说一句，在Java这个岗位上，如果想走得更远，一定要花苦功夫。

好的，本期的普通人VS高手面试系列的视频就到这里结束了。

有任何不懂的技术面试题，欢迎随时私信我

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

16.4 Dubbo的服务请求失败怎么处理?

今天分享的面试题，几乎是90%以上的互联网公司都会问到的问题。

“Dubbo的服务请求失败怎么处理”？

对于这个问题，我们来看一下普通人和高手的回答。

普通人

嗯...我记得，Dubbo请求处理失败以后，好像是会重试。嗯!

高手

Dubbo是一个RPC框架，它为我们的应用提供了远程通信能力的封装，同时，Dubbo在RPC通信的基础上，逐步在向一个生态在演进，它涵盖了服务注册、动态路由、容错、服务降级、负载均衡等能力，基本上在微服务架构下面临的问题，Dubbo都可以解决。

而对于Dubbo服务请求失败的场景，默认提供了重试的容错机制，也就是说，如果基于Dubbo进行服务间通信出现异常，服务消费者会对服务提供者集群中其他的节点发起重试，确保这次请求成功，默认的额外重试次数是2次。

除此之外，Dubbo还提供了更多的容错策略，我们可以根据不同的业务场景来进行选择。

快速失败策略，服务消费者只发起一次请求，如果请求失败，就直接把错误抛出去。这种比较适合在非幂等性场景中使用

失败安全策略，如果出现服务通信异常，直接把这个异常吞掉不做任何处理

失败自动恢复策略，后台记录失败请求，然后通过定时任务来对这个失败的请求进行重发。

并行调用多个服务策略，就是把这个消息广播给服务提供者集群，只要有任何一个节点返回，就表示请求执行成功。

广播调用策略，逐个调用服务提供者集群，只要集群中任何一个节点出现异常，就表示本次请求失败

要注意的是，默认基于重试策略的容错机制中，需要注意幂等性的处理，否则在事务型的操作中，容易出现多次数据变更的问题。

以上就是我对这个问题的理解！

16.5 Dubbo注册中心挂了， 服务还可以继续通信吗?

【引言】

什么情况？一位工作了 5 年的 Java 程序员，竟然回答不出这个问题？说“Dubbo注册中心挂了， 服务之间还可以继续通信吗”？今天，我话2分钟时间给大家来聊一聊。

另外，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

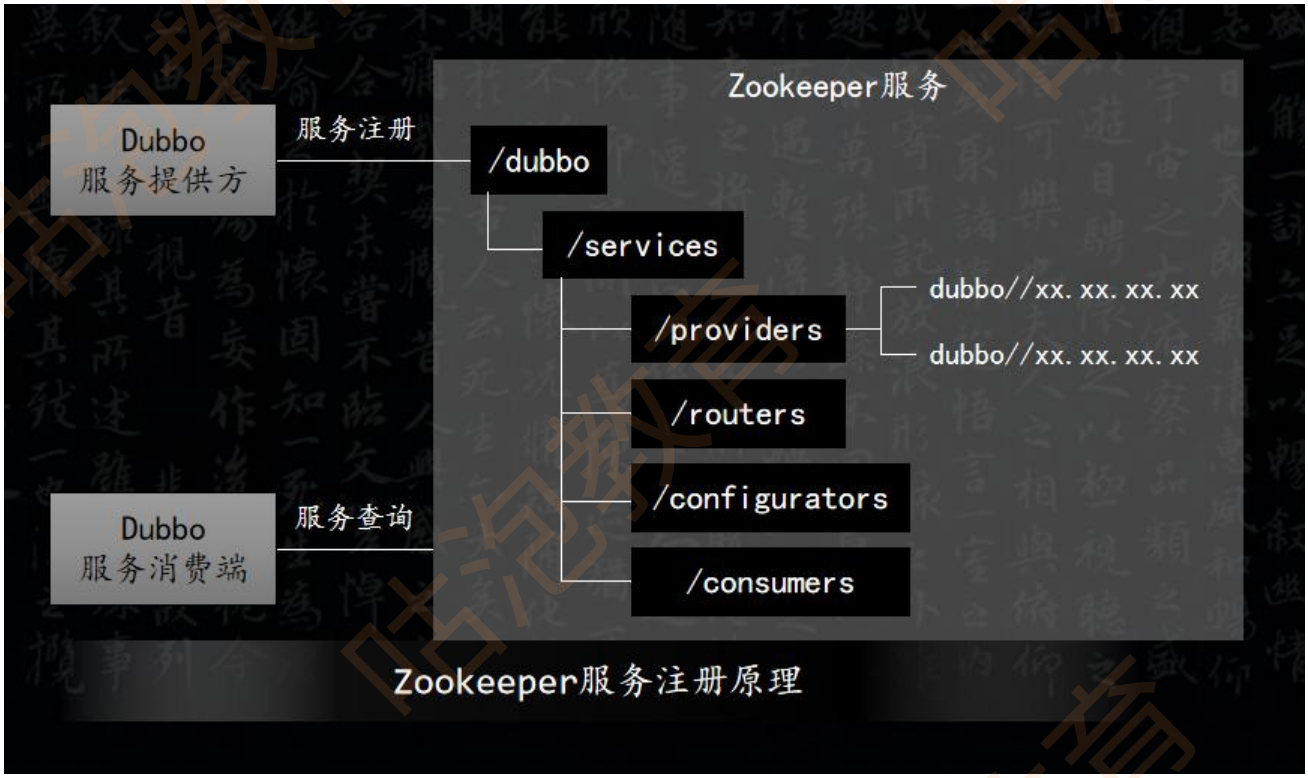
要回答这个问题，我们得先了解Dubbo的服务感知原理。

服务感知原理

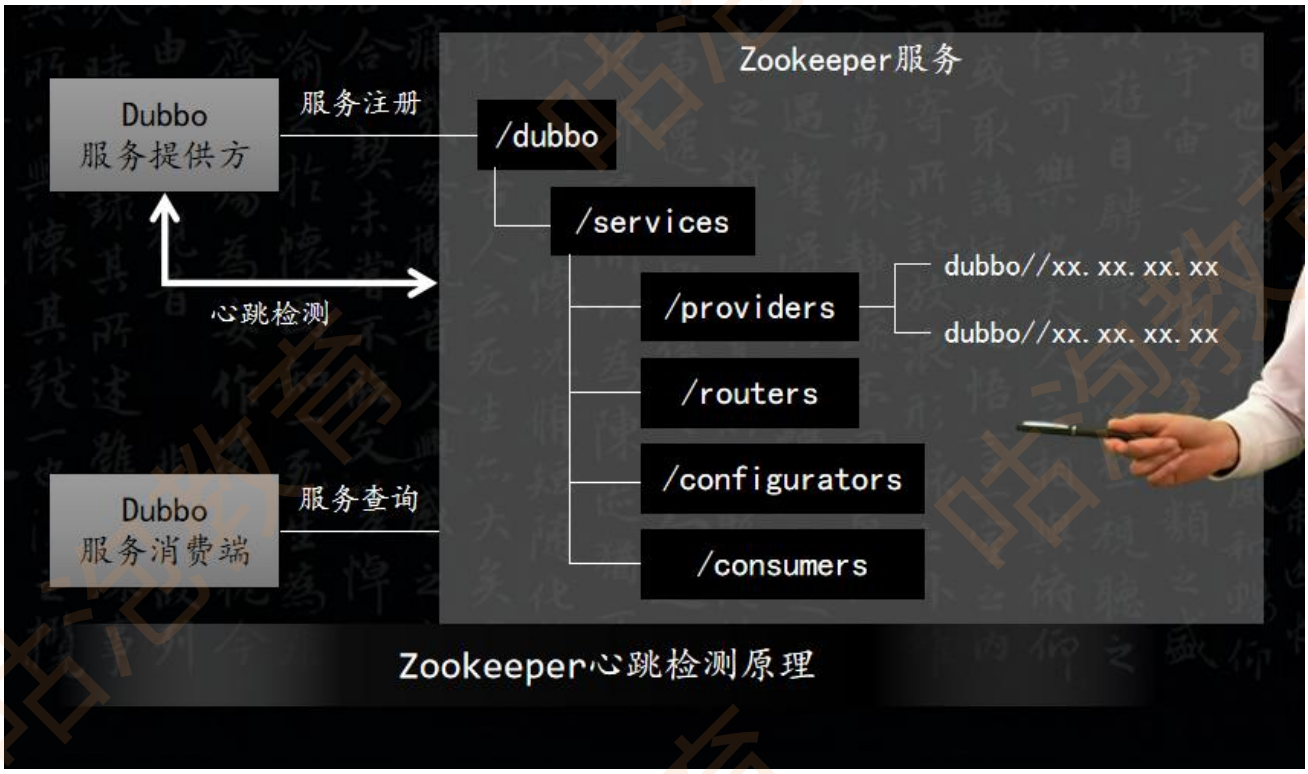
那Dubbo 是如何动态感知服务下线的?

首先，Dubbo 默认采用 Zookeeper 实现服务的注册与服务发现，简单来说，是多个 Dubbo 服务之间的通信地址，是使用 Zookeeper 来维护的。

而在 Zookeeper 上，会采用树形结构的方式来维护 Dubbo 服务提供端的协议地址，Dubbo 服务消费端会从 Zookeeper Server 上去查找目标服务的地址列表，从而完成服务的注册和消费功能。



那Zookeeper呢，就会通过心跳检测机制，来判断 Dubbo 服务提供端的运行状态，来决定是否应该把这个服务从地址列表剔除。



当 Dubbo 服务提供方出现故障导致 Zookeeper 剔除了这个服务的地址，那么 Dubbo 服务消费端需要感知到地址的变化，从而避免后续的请求发送到故障节点，导致请求失败。

也就是说 Dubbo 要提供服务下线的动态感知能力。这个能力是通过 Zookeeper 里面提供的 Watch 机制来实现的。

简单理解呢，就是Dubbo 服务消费端会使用 Zookeeper 里面的 Watch 来针对 Zookeeper Server 端的/providers 节点注册监听，一旦这个节点下的子节点发生变化，Zookeeper Server 就会发送一个事件通知Dubbo Client 端，Dubbo Client 端收到事件以后，就会把本地缓存的这个服务地址删除，这样后续，就不会把请求发送到失败的节点上，完成服务下线感知。

答案分析

了解Dubbo服务感知原理以后，我就应该知道，Dubbo的注册中心挂了以后，服务之间还是可以继续通信的。主要是因为Dubbo初始化的时候，消费者会将服务提供者的地址等信息缓存到本地。

以上就是我对这个问题的理解！

【结尾】

以上就是我对这个问题的理解。

Dubbo 是目前非常主流的开源 RPC 框架，在很多的企业都有使用。理解 RPC 底层的工作原理非常有必要，它能帮助开发者提高开发问题的解决效率。

最后，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

Spring（17）

17.1 Spring 中，有两个 id 相同的 bean，会报错吗，如果会报错，在哪个阶段报错

Hi，大家好，我是Mic

一个工作3年的粉丝，早上6点给我微信发语音，把我直接吓醒。

我以为什么天大的事情，结果一问才知道。

面试官问了他一个问题没答上来，问题是“Spring里面，如果两个id相同的bean会报错吗？如果会，在哪个阶段报错？”

高手

好的，关于这个问题，我从几个点来回答。

首先，在同一个XML配置文件里面，不能存在id相同的两个bean，否则spring容器启动的时候会报错。

因为id这个属性表示一个Bean的唯一标志符号，所以Spring在启动的时候会去验证id的唯一性，一旦发现重复就会报错，

这个错误发生Spring对XML文件进行解析转化为BeanDefinition的阶段。

但是在两个不同的Spring配置文件里面，可以存在id相同的两个bean。 IOC容器在加载Bean的时候，默认会多个相同id的bean进行覆盖。

在Spring3.x版本以后，这个问题发生了变化

我们知道Spring3.x里面提供@Configuration注解去声明一个配置类，然后使用@Bean注解实现Bean的声明，这种方式完全取代了XMl。

在这种情况下，如果我们在同一个配置类里面声明多个相同名字的bean，在Spring IOC容器中只会注册第一个声明的Bean的实例。

后续重复名字的Bean就不会再注册了。

像这样一段代码，在Spring IOC容器里面，只会保存UserService01这个实例，后续相同名字的实例不会再加载。

```
@Configuration
public class SpringConfiguration {

    @Bean(name = "userService")
    public UserService01 userService01(){
        return new UserService01();
    }

    @Bean(name = "userService")
    public UserService02 userService02(){
        return new UserService02();
    }
}
```

如果使用@Autowired注解根据类型实现依赖注入，因为IOC容器只有UserService01的实例，所以启动的时候会提示找不到UserService02这个实例。

如果使用@Resource注解根据名词实现依赖注入，在IOC容器里面得到的实例对象是

UserService01，

于是Spring把UserService01这个实例赋值给UserService02，就会提示类型不匹配错误。

这个错误，是在Spring IOC容器里面的Bean初始化之后的依赖注入阶段发生的。

以上就是我对这个问题的理解。

面试总结

你看，一个小小的面试题，竟然涉及到这么多知识点。

有些粉丝会问，这个我已经会用了，问这个问题的意义在哪里？

其实很多刚工作1~2年的小伙伴，如果出现使用不当很容易出现各种异常。

而对Spring有足够深入的理解，可以快速解决各种异常。

好的，本期的普通人VS高手面试系列就到这里结束了。

喜欢我的作品的小伙伴记得点赞和收藏加关注。

我是Mic，一个工作14年的Java程序员，咱们下期再见！

17.2 如何理解Spring Boot中的Starter?

一个工作了3年的Java程序员，遇到一个Spring Boot的问题。

他对这个问题有一些了解，但是回答得不是很好，希望参考我的高手回答。

这个问题是：“如何理解Spring Boot中的Starter”。

对于这个问题，看看高手的回答。

高手

Starter是Spring Boot的四大核心功能特性之一，除此之外，Spring Boot还有自动装配、Actuator监控等特性。

Spring Boot里面的这些特性，都是为了让开发者在开发基于Spring生态下的企业级应用时，只需要关心业务逻辑，

减少对配置和外部环境的依赖。

其中，Starter是启动依赖，它的主要作用有几个。

Starter组件以功能为纬度，来维护对应的jar包的版本依赖，

使得开发者可以不需要去关心这些版本冲突这种容易出错的细节。

Starter组件会把对应功能的所有jar包依赖全部导入进来，避免了开发者自己去引入依赖带来的麻烦。

Starter内部集成了自动装配的机制，也就说在程序中依赖对应的starter组件以后，

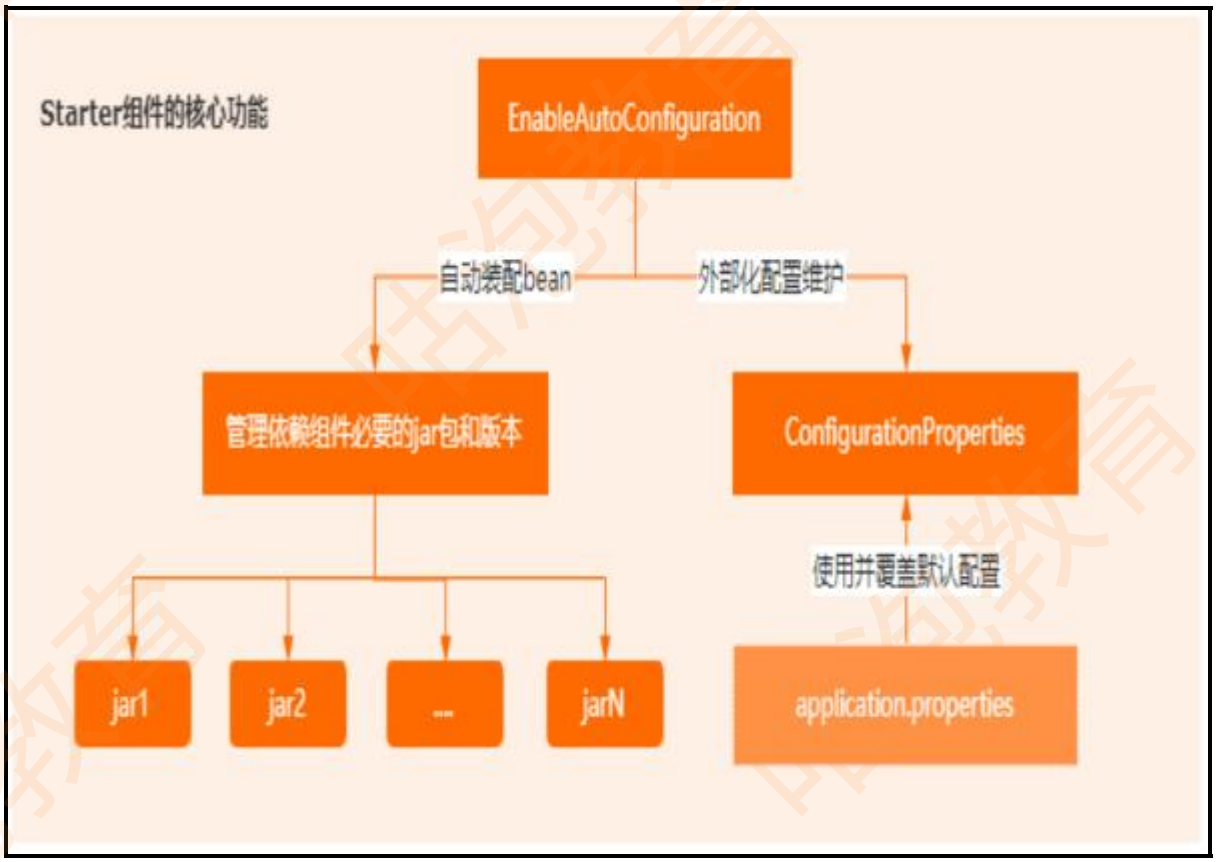
这个组件自动会集成到Spring生态下，并且对于相关Bean的管理，也是基于自动装配机制来完成。

依赖Starter组件后，这个组件对应的功能所需要维护的外部化配置，会自动集成到Spring Boot里面，

我们只需要在application.properties文件里面进行维护就行了，比如Redis这个starter，只需要在application.properties

文件里面添加redis的连接信息就可以直接使用了。

在我看来，Starter组件几乎完美的体现了Spring Boot里面约定优于配置的理念。



另外，Spring Boot官方提供了很多的Starter组件，比如Redis、JPA、MongoDB等等。

但是官方并不一定维护了所有中间件的Starter，所以对于不存在的Starter，第三方组件一般会自己去维护一个。

官方的starter和第三方的starter组件，最大的区别在于命名上。

官方维护的starter的以spring-boot-starter开头的前缀。

第三方维护的starter是以spring-boot-starter结尾的后缀

这也是一种约定优于配置的体现。

以上就是我对这个问题的理解。

面试点评

在技术的学习过程中，我认为“为什么是”比“是什么”要重要。

以这种方式来学习，带来的好处就是对技术理解会更加深刻。

这道题考察的就是“为什么是”，不难，关键在于自己的理解。

好的，本期的普通人VS高手面试系列的视频就到这里结束了，

如果你喜欢这个视频，记得点赞和收藏。

如果想获得一对一的面试指导以及面试资料，可以私信我。

我是Mic，一个工作了14年的Java程序员，咱们下期再见！

17.3 为什么要使用Spring框架?

一个工作了4年的小伙伴，他说他从线下培训就开始接触Spring，到现在已经快5年时间了。

从来没有想过，为什么要使用Spring框架。

结果在面试的时候，竟然遇到一个这样的问题。

大脑一时间短路了，来求助我，这类问题应该怎么去回答。

下面我们来看看高手的回答

高手

Spring是一个轻量级应用框架，它提供了IoC和AOP这两个核心的功能。

它的核心目的是为了简化企业级应用程序的开发，使得开发者只需要关心业务需求，不需要关心Bean的管理，

以及通过切面增强功能减少代码的侵入性。

从Spring本身的特性来看，我认为有几个关键点是我们选择Spring框架的原因。

轻量：Spring是轻量的，基本的版本大约2MB。

IOC/DI：Spring通过IOC容器实现了Bean的生命周期的管理，以及通过DI实现依赖注入，从而实现了对象依赖的松耦合管理。

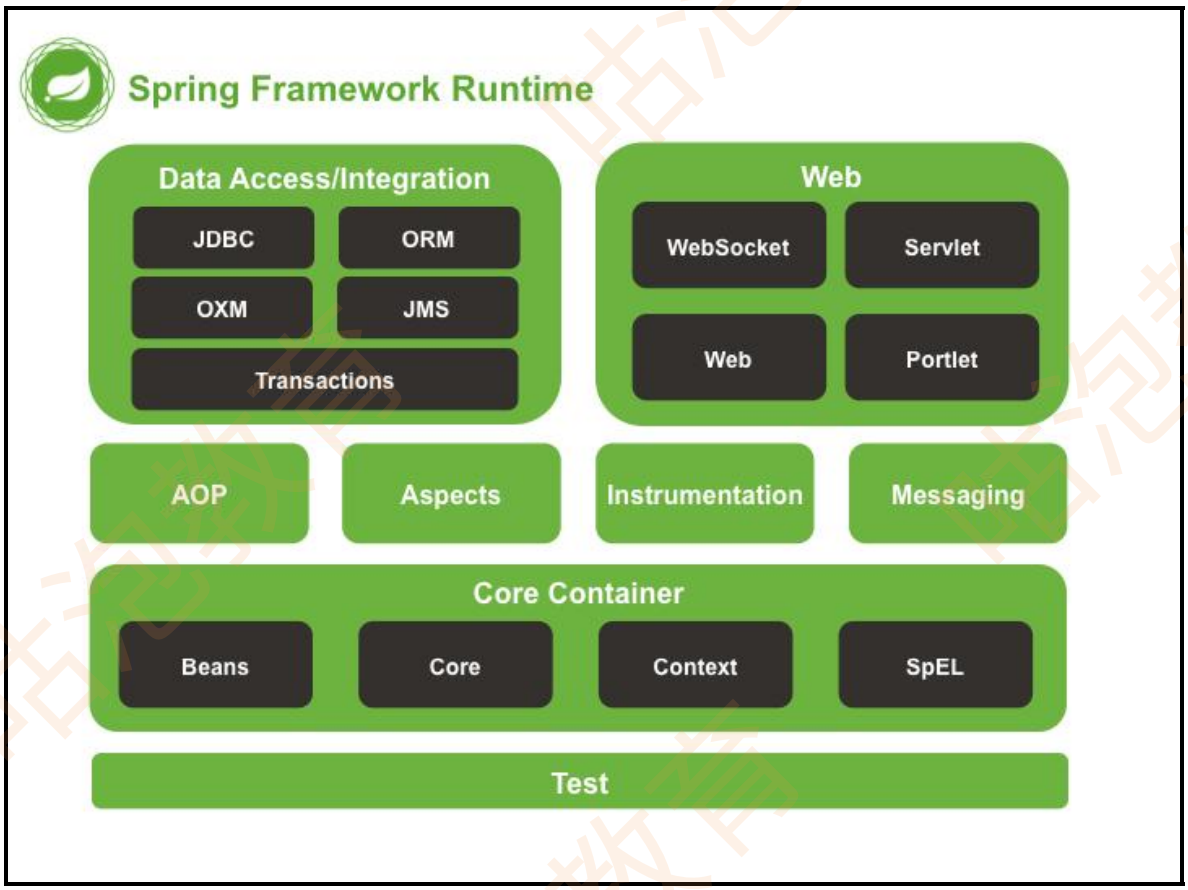
面向切面的编程(AOP)：Spring支持面向切面的编程，从而把应用业务逻辑和系统服务分开。

MVC框架：Spring MVC提供了功能更加强大且更加灵活的Web框架支持

事务管理：Spring通过AOP实现了事务的统一管理，对应用开发中的事务处理提供了非常灵活的支持

最后，Spring从第一个版本发布到现在，它的生态已经非常庞大了。在业务开发领域，Spring生态几乎提供了

非常完善的支持，更重要的是社区的活跃度和技术的成熟度都非常高，以上就是我对这个问题的理解。



面试点评

任何一个技术框架，一定是为了解决某些特定的问题，只是大家忽视了这个点。

为什么要用，再往高一点来说，其实就是技术选型，能回答这个问题，

意味着面对业务场景或者技术问题的解决方案上，会有自己的见解和思考。

所以，我自己也喜欢在面试的时候问这一类的问题。

好的，本期的普通人VS高手面试系列的视频就到这里结束了。

有任何不懂的技术面试题，欢迎随时私信我

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

17.4 为什么越来越多的人选择Spring Boot?

【引言】

我们都知道，Spring是一个非常经典的应用框架，与其说是Java开发不如说是Spring开发，为什么现在越来越多的人会选择用Spring Boot呢？。要回答这个问题，还需要从Java Web开发的发展历史开始说起。

1、Java Web开发简史

自1995年James Gosling（詹姆斯·高斯林）带领团队设计发布Java语言以来，Java一直排在编程语言排行榜的前列，James Gosling也被称为“Java之父”。

在最初发布的Java版本中，包含Java SE、JavaEE、JavaME。

Java SE（Standard Edition）作为标准版本，提供最核心的基础功能，Java EE（Enterprise Edition）作为企业版，主要用于企业级的Web开发，JavaME（***Micro Edition***）作为微型版本主要应用与移动设备的开发。随着Java的语言的广泛应用，Java也找到了自身的优势，Java EE版本被应用得最多。

从1996年开始，JavaEE开发是基于JSP + Java Bean来完成的。

后来慢慢地演变，1997年官方推出了 JSP + Servlet + Java Bean来进行开发，Servlet起到了调度控制的作用，这是MVC设计的雏形。

然后，发展出现了 JSP + Sevlet + Java Bean + Dao的模式，将业务逻辑处理和数据库访问分离，出现了三层架构设计理念。

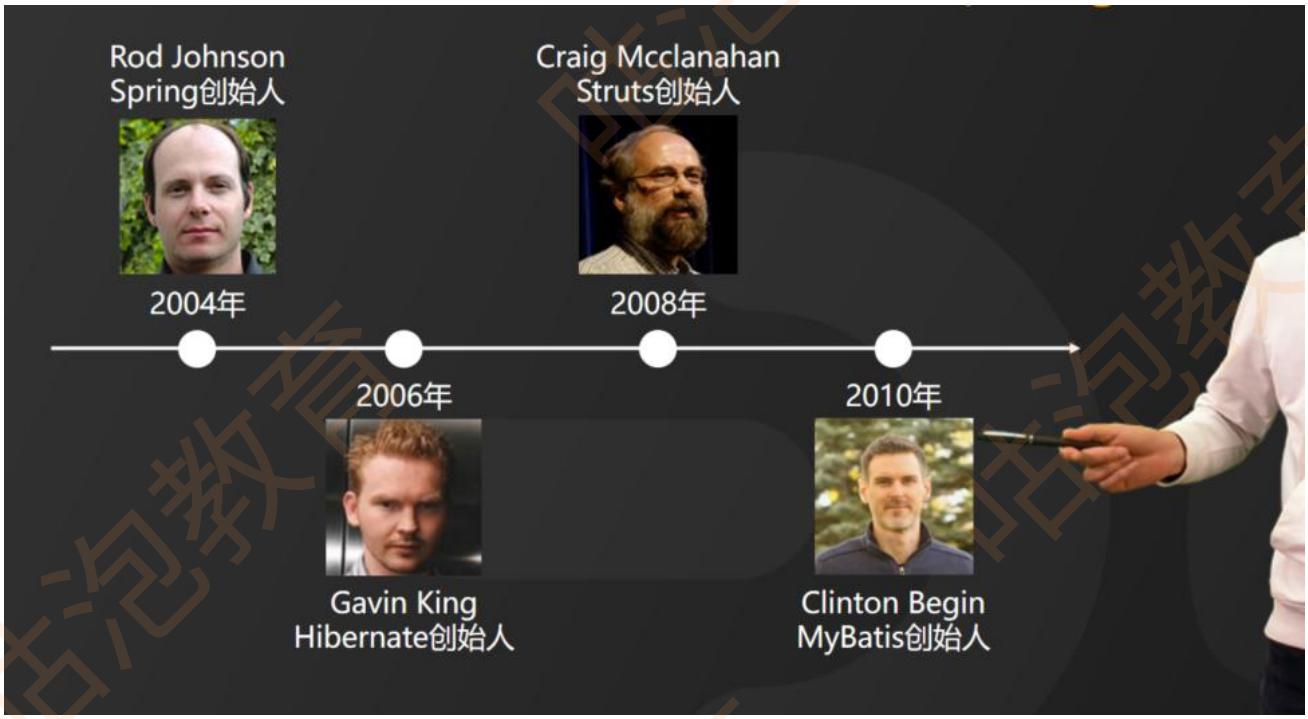
再后来，1998年前后，为了满足多服务器之间的通信，采用了JSP + Servlet + EJB的形式，出现了RPC设计的雏形。

之后，继续演变，在2006年首次出现 JSF + EJB + JPA，提出前后端完全隔离开发的思想。

当然，这些技术都是Java官方提供的。好像，一直都不那么受欢迎。尤其是在开源社区发布一些开源的产品用来来代替官方退出的这些技术，主要目的是为了简化开发。于是就出现了许多非常经典的Java开发框架。。

2、Java Web框架创始人

下面，简单总结四位全球知名的Java Web开发框架创始人，也是JavaEE开发的改造者，



第一位是Gavin King（加文·金），Hibernate框架创始人，一个具有革命性意义的、全自动化的ORM框架，于2006年发布，最初的版本是基于JPA来实现的。

第二位是Craig Mcclanahan（克雷格·麦克拉纳汉），著名的Struts框架创始人，叫Craig Mcclanahan（克雷格·麦克拉纳汉），比较早的程序员都用过Struts1和Struts2，感受过非常酸爽的味道。2008年发布1.0版本，在2013年6月底还爆发了一次眼中的安全漏洞，那时候开始，我开始全面弃用Struts框架。

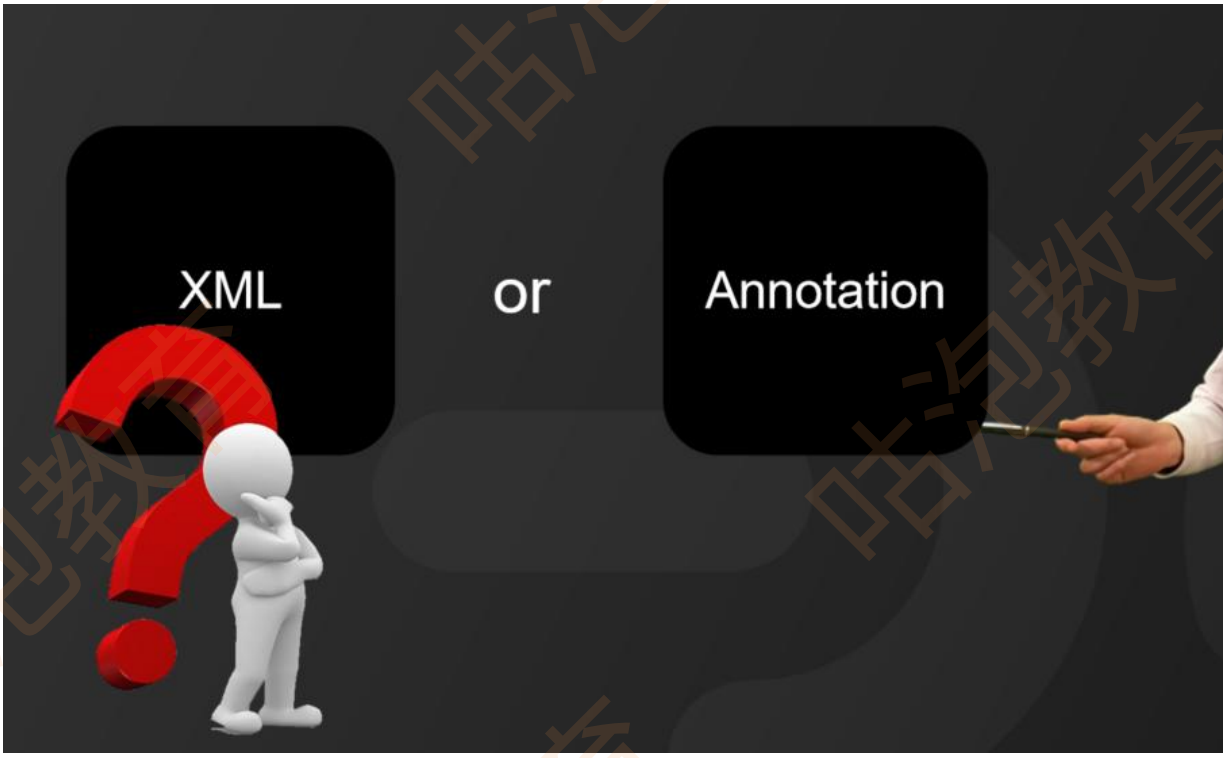
第三位是Clinton Begin（克林顿·比格），MyBatis框架创始人，2010年发布，最早期的名字叫做iBatis，它是一个比Hibernate更加轻量级的，半自动化的持久化框架。目前，绝大部分互联网公司都是采用MyBatis作为持久层的框架。

第四位是Rod Johnson（罗德·约翰逊），大名鼎鼎的Spring框架创始人，2004年发布1.0版本，以前叫做万能胶，现在叫做全家桶，真正给程序员带来了春天。

小伙伴们可以对比一下发际线。

3、Spring的演变

Spring是一个通用性非常强的框架，它提供的工具也非常全面，能满足各个领域的开发需求。因此，使用Spring之前需要完成大量的个性化配置。在Spring演变过程中，一直存在一个争议，是使用XML作为配置方式 还是 使用 Annotation注解作为配置方式?



Spring在开源社区不断质疑声中，不断的发展和演变，直到Spring 4.0版本发布的时候，已经全面支持注解方式配置，没有XML也可以进行开发。但是，开发者们在使用纯注解开发以后，发现代码侵入比较多，配置也稍显麻烦，尤其是一些Maven依赖也比较繁杂，包括使用外部的Tomcat容器部署也需要不少的工作量。

于是，Pivotal团队从2013年开始研发设计Spring Boot，它的初衷是为了简化开发，旨在解放程序员的双手。2014年4月，发布全新开源的轻量级框架，Spring Boot1.0版本。

4、Spring Boot的核心功能

（1）可以独立运行Spring项目

Spring Boot可以以jar包的形式进行独立的运行，使用：java -jar xx.jar 就可以成功的运行项目，或者在应用项目的主程序中运行main函数即可；

（2）内嵌的Servlet容器

内嵌容器，使得我们可以执行运行项目的主程序main函数，实现项目的快速运行；主程序代码SpringbootDemoApplication.java

（3）提供starter简化Manen依赖

Spring Boot提供了一系列的starter pom用来简化我们的Maven依赖，比如创建一个web项目，只需要添加spring-boot-starter-web即可：

（4）自动配置Spring

Spring Boot会根据我们项目中类路径的jar包/类，为jar包的类进行自动配置Bean，这样一来就大大的简化了我们的配置。当然，这只是Spring考虑到的大多数的使用场景，在一些特殊情况，我们还需要自定义自动配置；

（5）无代码生成、无XML配置

Spring Boot神奇的地方不是借助于代码生成来实现的，而是通过条件注解的方式来实现的，这也是Spring 4.x的新特性。

5、结论

所以，Spring Boot是由Spring发展演进而来的，是Spring进化的结果，是对

Sping的升级。

听懂的小伙伴，关注点个赞，下次不迷路。

我是被编程耽误的文艺Tom，如果你还有其他不懂的面试题或者需要视频配套的文字资料，可以关注我的主页加V。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

17.5 Spring如何解决循环依赖问题

MIC老师解析

“Spring如何解决循环依赖问题”！

这是一道非常经典并且考察频率很高的面试题。

Hi，大家好，我是Mic，一个工作了14年的Java程序员

下面我们来分析一下面试官对于这个问题的考察意图

考察目的

这个问题考察的求职者年限还挺广的，工作1年到工作7年之间都会遇到。

我认为考察目的有三个：

Spring 是Java开发的基础应用框架，所以考察的是基本功

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

考察技术深度，判断你的能力高低以及作为人才筛选的区分度

因此对于这个问题，不仅仅要回答原因，还需要有自己的思考和总结。

另外还需要注意，这个问题问的是Spring如何解决循环依赖，因此对于Spring无法解决的循环依赖

我们没必要在这里展开。

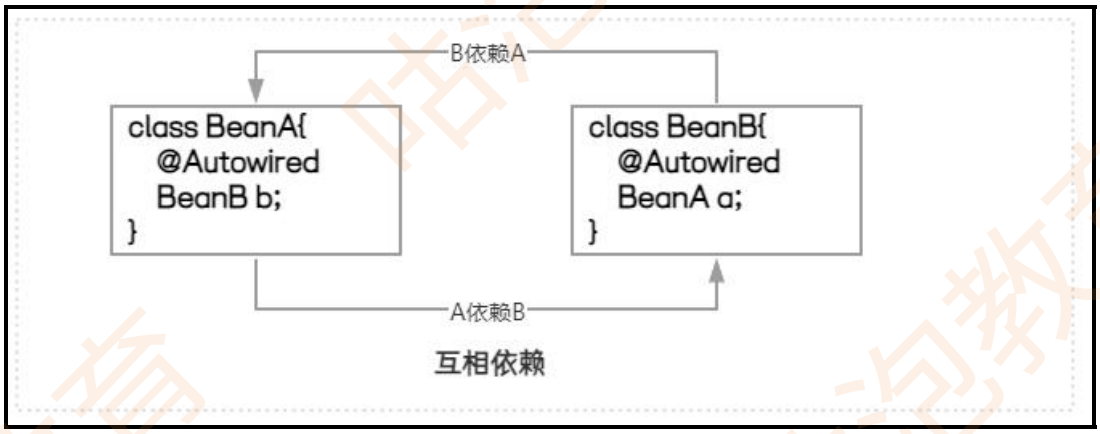
下面我们来分析一下这个问题的产生背景和解决方案。

问题解析

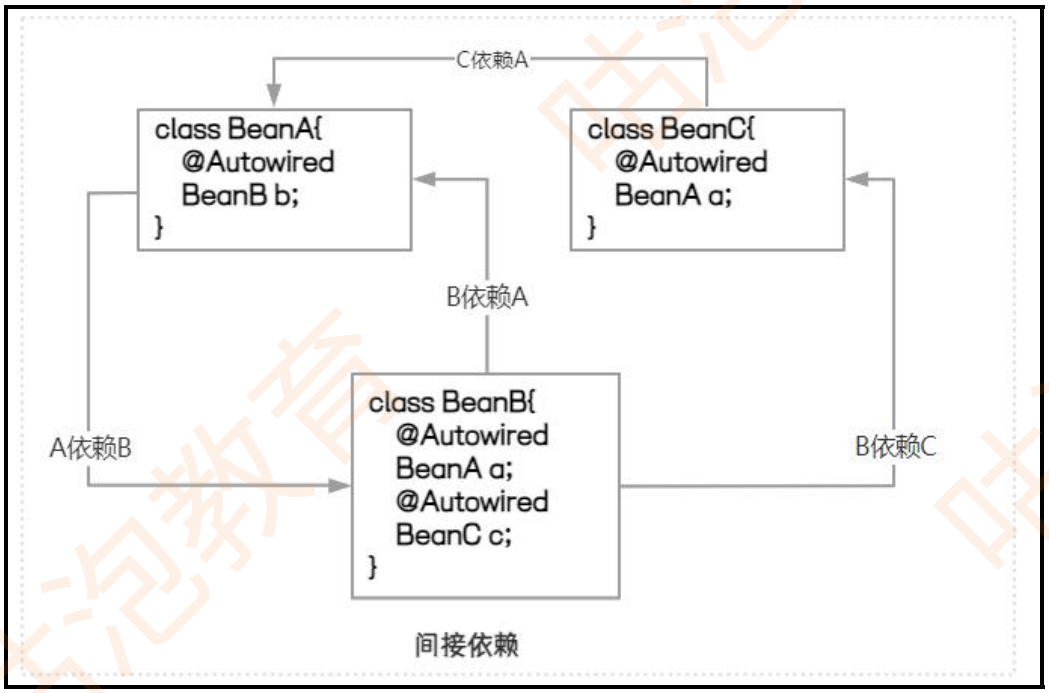
循环依赖是指一个或多个 Bean 实例之间存在直接或间接的依赖关系，构成循环调用。

通常表现为三种形态。

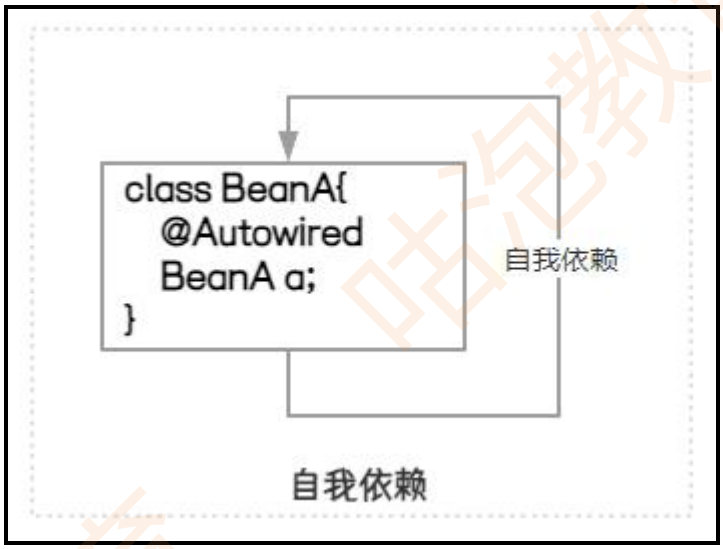
互相依赖，也就是A依赖B，B依赖A



间接依赖，两个以上的Bean存在间接依赖关系造成循环调用。



自我依赖，自己依赖自己造成了循环依赖



Spring本身也考虑到了这方面的问题，所以它设计了三级缓存来解决部分循环依赖的问题。

所谓三级缓存，其实就是用来存放不同类型的Bean。

第一级缓存存放完全初始化好的Bean，这个Bean可以直接使用了

第二级缓存存放原始的Bean对象，也就是说Bean里面的属性还没有进行赋值

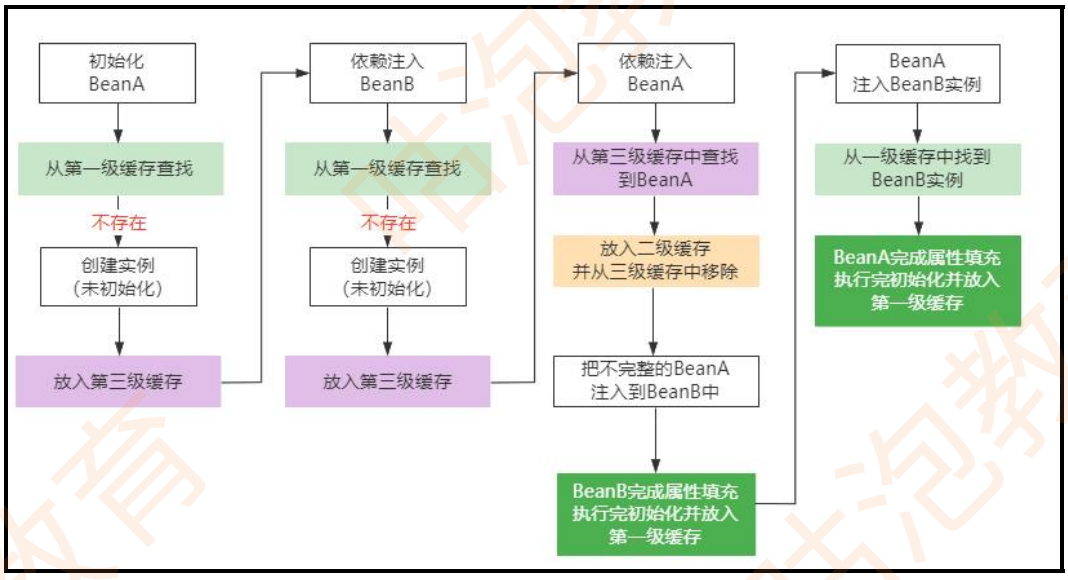
第三级缓存存放Bean工厂对象，用来生成原始Bean对象并放入到二级缓存中

假设BeanA和BeanB存在循环依赖，那么在三级缓存的设计下，我画了这样一个图来描述工作原理。

初始化BeanA，先把BeanA实例化，然后把BeanA包装成ObjectFactory对象保存到三级缓存中。

接着BeanA开始对属性BeanB进行依赖注入，于是开始初始化BeanB，同样做两件事，创建BeanB实例，以及加入到三级缓存。

然后，BeanB也开始进行依赖注入，在三级缓存中找到了BeanA，于是完成BeanA的依赖注入BeanB初始化成功以后保存到一级缓存，于是BeanA可以成功拿到BeanB的实例，从而完成正常的依赖注入。



整个流程看起来很复杂，但是它的核心思想就是把Bean的实例化和Bean中属性的依赖注入这两个过程分离出来。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

不过要注意的是，Spring本身只能解决单实例存在的循环引用问题，但是存在以下四种情况需要人为干预：

多实例的Setter注入导致的循环依赖，需要把Bean改成单例。

构造器注入导致的循环依赖，可以通过@Lazy注解

DependsOn导致的循环依赖，找到注解循环依赖的地方，迫使它不循环依赖。

单例的代理对象Setter注入导致的循环依赖，可以使用@Lazy注解，或者使用@DependsOn注解指定加载先后关系。

在实际开发中，出现循环依赖的根本原因还是在代码设计的时候，因为模块的耦合度较高，依赖关系复杂导致的，我们应该尽可能的从系统设计角度去考虑模块之间的依赖关系，避免循环依赖的问题。

下面我们来看看高手的回答。

高手解答

Spring设计了三级缓存来解决循环依赖问题。

第一级缓存里面存储完整的Bean实例，这些实例是可以直接被使用的。

第二级缓存里面存储的是实例化以后，但是还没有设置属性值的Bean实例，也就是Bean里面的依赖注入还没有做。

第三级缓存用来存放Bean工厂，它主要用来生成原始Bean对象并且放到第二级缓存里面。

三级缓存的核心思想，就是把Bean的实例化，和Bean里面的依赖注入进行分离。

采用一级缓存存储完整的Bean实例，采用二级缓存来存储不完整的Bean实例，

通过不完整的Bean实例作为突破口，解决循环依赖的问题。

至于第三级缓存，主要是解决代理对象的循环依赖问题。

以上就是我的理解。

总结

下次面试的时候遇到这个问题，大家知道怎么回答了吗？

如果你喜欢我的作品，记得点赞收藏加关注哦

TOM老师解析

今天给大家分享一道大厂面试真题，Spring为何需要三级缓存解决循环依赖，而不是二级缓存？我一共分为五个部分来给大家介绍：

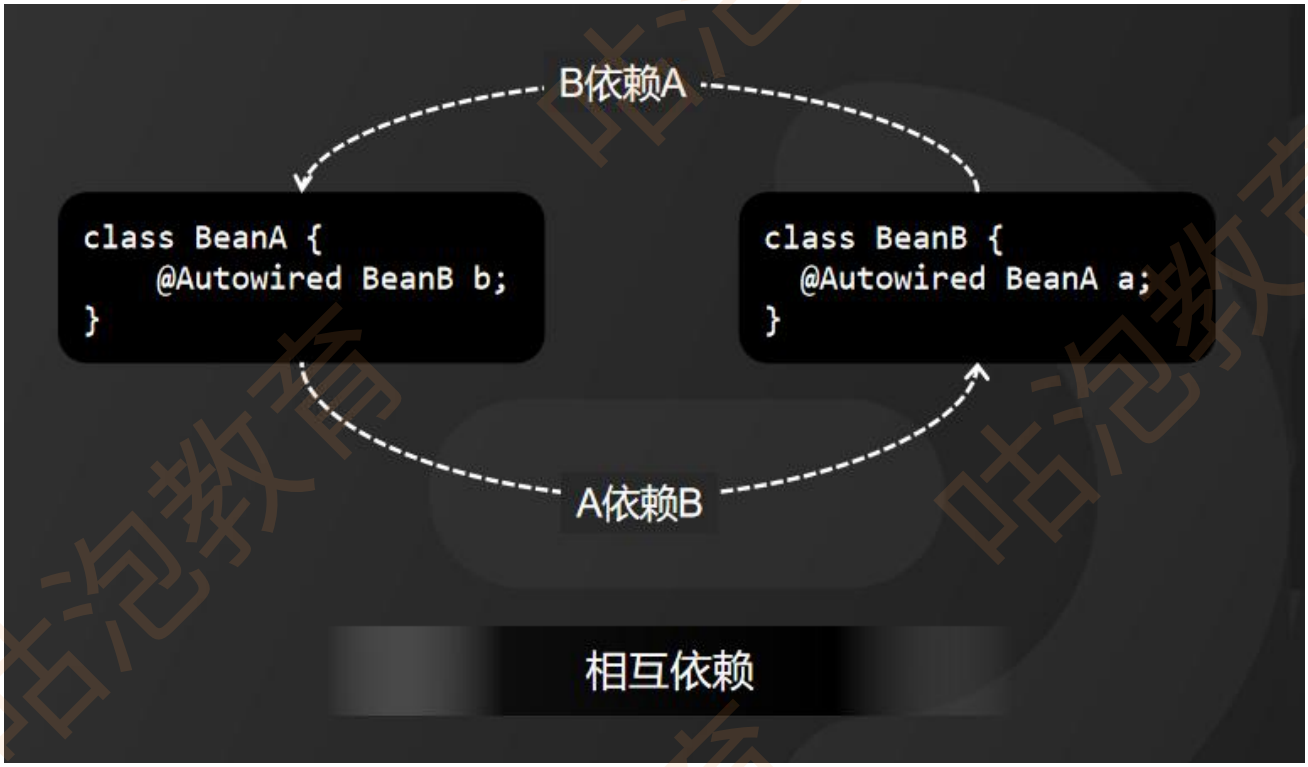
1、什么是循环依赖？

循环依赖就是指循环引用，是两个或多个Bean相互之间的持有对方的引用。在代码中，如果将两个或多个Bean互相之间持有对方的引用，因为Spring中加入了依赖注入机制，也就是自动给属性赋值。Spring给属性赋值时，将会导致死循环。那么，哪些情况会出现循环依赖呢？

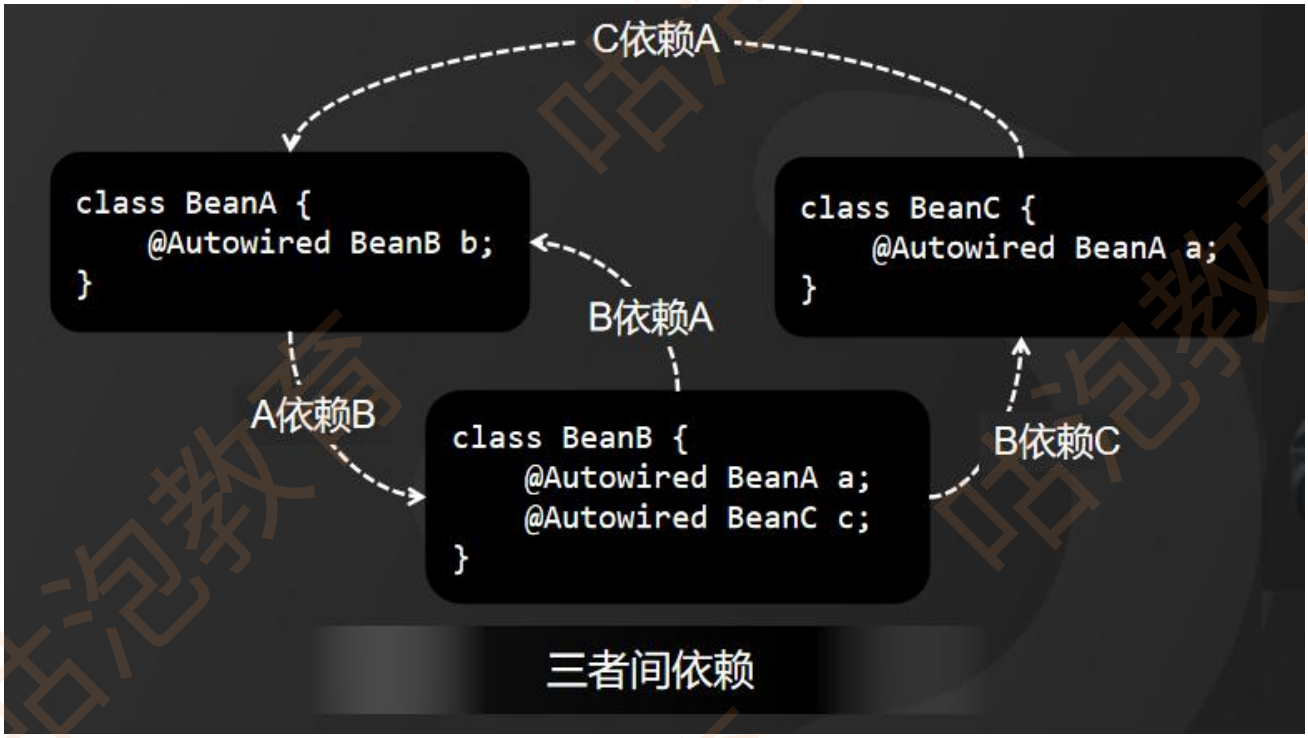
2、哪些情况会出现循环依赖？

循环依赖有三种形态：

1、相互依赖，也就是A 依赖 B，B 又依赖 A，它们之间形成了循环依赖。



2、三者间依赖，也就是A 依赖 B，B 依赖 C，C 又依赖 A，形成了循环依赖。



3、自我依赖，也是A依赖A形成了循环依赖自己依赖自己。

3、Spring如何解决循环依赖问题?

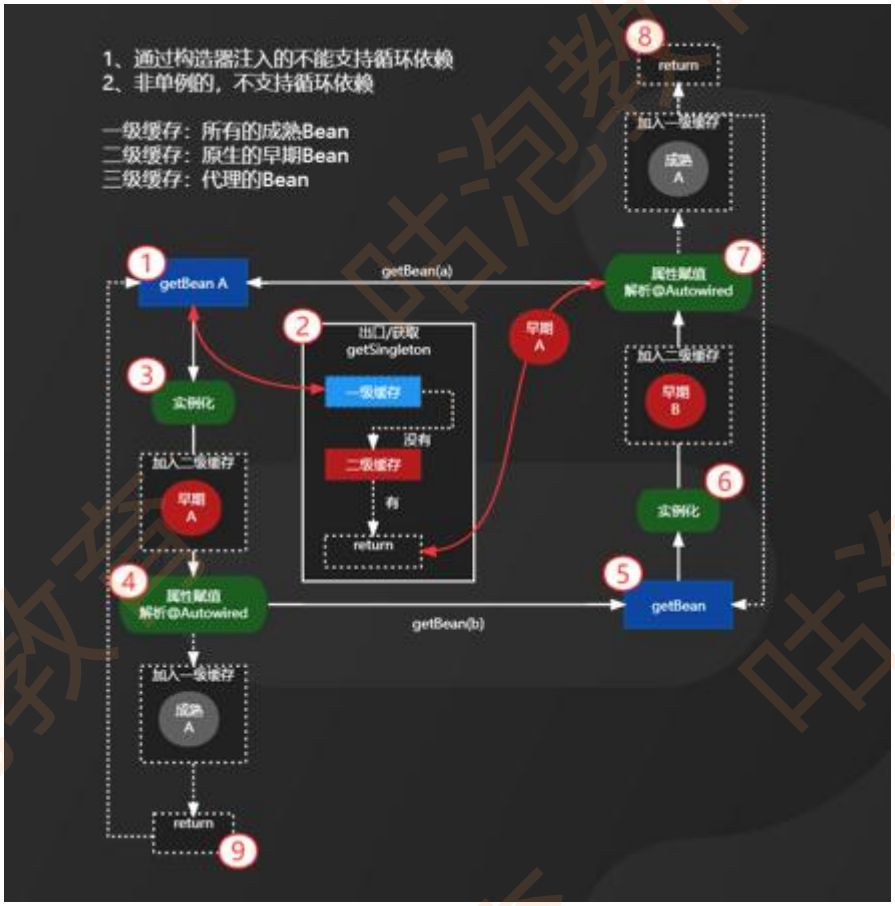
Spring中设计了三级缓存来解决循环依赖问题，当我们去调用getBean()方法的时候，Spring会先从一级缓存中去找到目标Bean，如果发现一级缓存中没有 便会去二级缓存中去找，而如果一、二级缓存中都没有找到，意味着该目标Bean还没有实例化。于是，Spring容器会实例化目标Bean（PS：刚初始化的Bean称为早期Bean），然后，将目标Bean放入到二级缓存中，同时，加上标记是否存在循环依赖。如果不存在循环依赖便会将目标Bean存入到二级缓存，否则，便会标记该Bean存在循环依赖，然后将等待下一次轮询赋值，也就是解析@Autowired注解。等@Autowired注解赋值完成后（PS：完成赋值

的Bean称为成熟Bean），会将目标Bean存入到一级缓存。

总结一下，Spring一级缓存中存放所有的成熟Bean，二级缓存中存放所有的早期Bean，先取一级缓存，再去二级缓存。

4、为何需要三级缓存，而不是二级缓存?

那么，前面有提到三级缓存，三级缓存的作用是啥呢？来看这样一张图，三级缓存是用来存储代理Bean，当调用getBean()方法时，发现目标Bean需要通过代理工厂来创建，此时会将创建好的实例保存到三级缓存，最终也会将赋值好的Bean同步到一级缓存中。大家可以私信我或者在评论区留言获取高清大图。



5、Spring中哪些情况下，不能解决循环依赖问题?

1.多例Bean通过setter注入的情况，不能解决循环依赖问题

2.构造器注入的Bean的情况，不能解决循环依赖问题

3.单例的代理Bean通过Setter注入的情况，不能解决循环依赖问题

4.设置了@DependsOn的Bean的情况，不能解决循环依赖问题

我是被编程耽误的文艺Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难!

17.6 Spring 是如何解决循环依赖问题的?

MIC老师解析

普通人

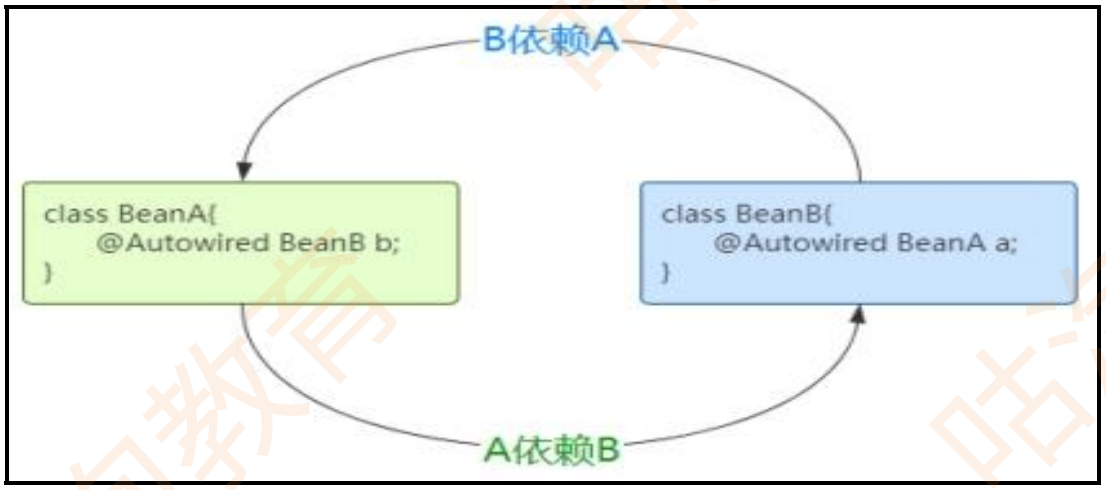
Spring是利用缓存机制来解决循环依赖问题的

高手

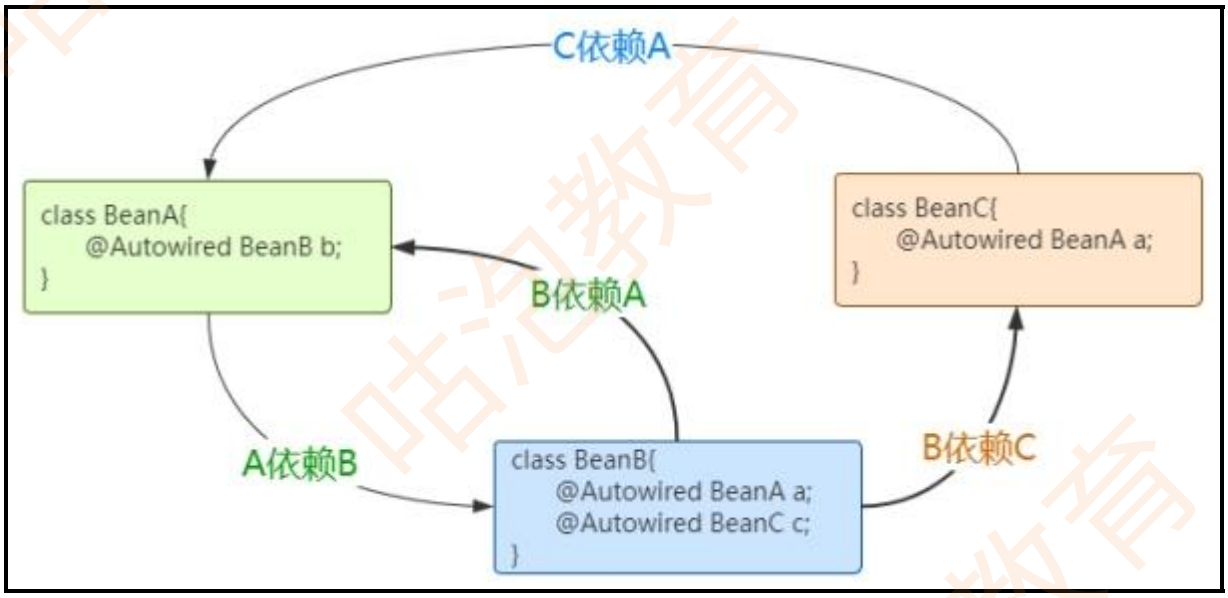
我们都知道，如果在代码中，将两个或多个Bean互相之间持有对方的引用就会发生循环依赖。循环的依赖将会导致注入死循环。这是Spring发生循环依赖的原因。

循环依赖有三种形态:

第一种互相依赖：A 依赖 B，B 又依赖 A，它们之间形成了循环依赖。



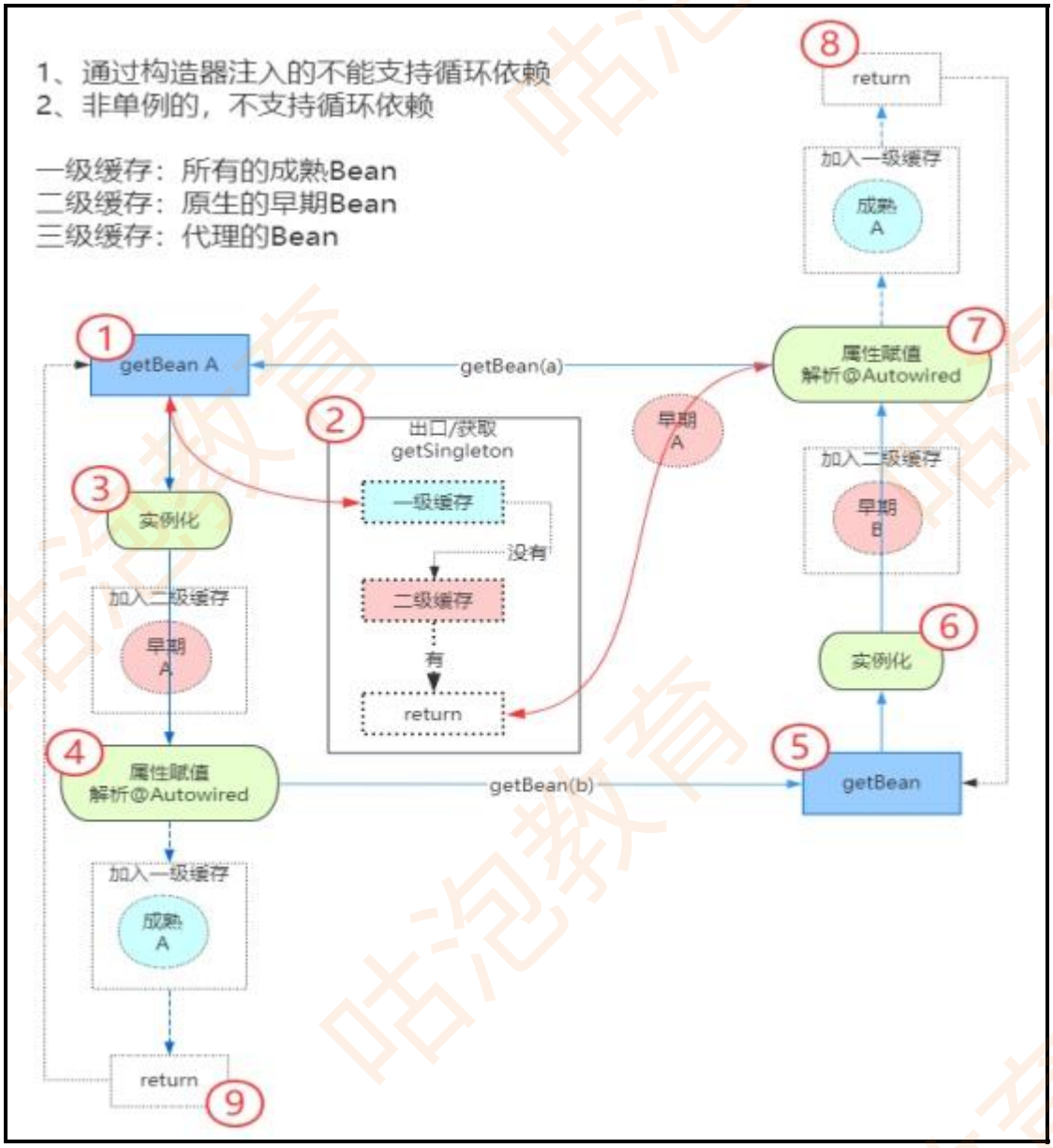
第二种三者间依赖：A 依赖 B，B 依赖 C，C 又依赖 A，形成了循环依赖。



第三种是自我依赖：A依赖A形成了循环依赖。

而Spring中设计了三级缓存来解决循环依赖问题，当我们去调用getBean()方法的时候，Spring会先从一级缓存中去找到目标Bean，如果发现一级缓存中没有便会去二级缓存中去找，而如果一、二级缓存中都没有找到，意味着该目标Bean还没有实例化。于是，Spring容器会实例化目标Bean（PS：刚初始化的Bean称为早期Bean）。然后，将目标Bean放入到二级缓存中，同时，加上标记是否存在循环依赖。如果不存在循环依赖便会将目标Bean存入到二级缓存，否则，便会标记该Bean存在循环依赖，然后将等待下一次轮询赋值，也就是解析@Autowired注解。等@Autowired注解赋值完成后，会将目标Bean存入到一级缓存。

Spring一级缓存中存放所有的成熟Bean，二级缓存中存放所有的早期Bean，先取一级缓存，再去二级缓存。



面试官：那么，前面有提到三级缓存，三级缓存的作用是什么？

高手：

三级缓存是用来存储代理Bean，当调用getBean()方法时，发现目标Bean需要通过代理工厂来创建，此时会将创建好的实例保存到三级缓存，最终也会将赋值好的Bean同步到一级缓存中。

面试官：Spring中哪些情况下，不能解决循环依赖问题?

高手：

有四种情况

1.多例Bean通过setter注入的情况，不能解决循环依赖问题

2.构造器注入的Bean的情况，不能解决循环依赖问题

3.单例的代理Bean通过Setter注入的情况，不能解决循环依赖问题

4.设置了@DependsOn的Bean的情况，不能解决循环依赖问题

TOM老师解析

Spring中设计了三级缓存来解决循环依赖问题，当我们去调用getBean()方法的时候，Spring会先从一级缓存中去找到目标Bean，如果发现一级缓存中没有 便会去二级缓存中去找，而如果一、二级缓存中都没有找到，意味着该目标Bean还没有实例化。于是，Spring容器会实例化目标Bean（PS：刚初始化的Bean称为早期Bean），然后，将目标Bean放入到二级缓存中，同时，加上标记是否存在循环依赖。如果不存在循环依赖便会将目标Bean存入到二级缓存，否则，便会标记该Bean存在循环依赖，然后将等待下一次轮询赋值，也就是解析@Autowired注解。等@Autowired注解赋值完成后（PS：完成赋值的Bean称为成熟Bean），会将目标Bean存入到一级缓存。

总结一下，Spring一级缓存中存放所有的成熟Bean，二级缓存中存放所有的早期Bean，先取一级缓存，再去二级缓存。

最通俗的方式理解Spring循环依赖三级缓存

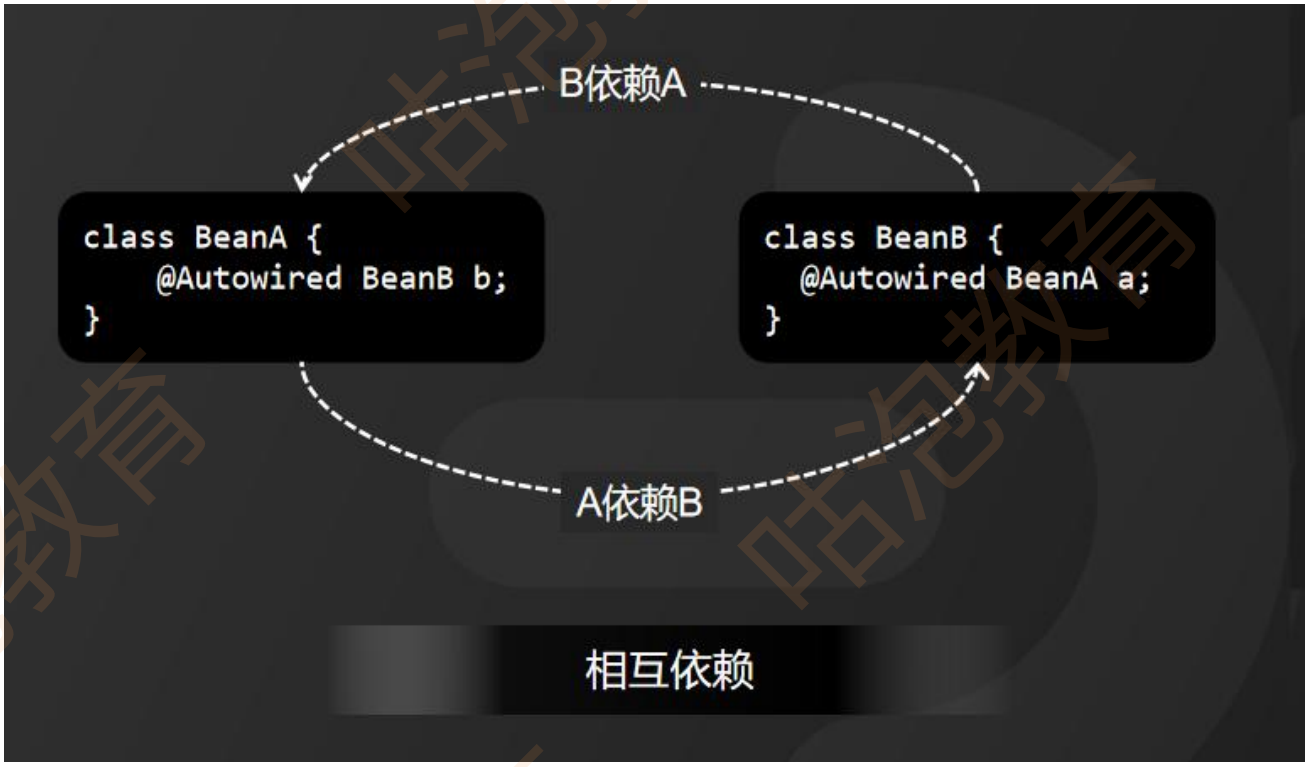
【引言】

今天，有位粉丝找我，说要耽误我5分钟时间，想让我帮助它理解一下Spring循环依赖的三级缓存，绕晕了一个星期，没有想明白。我想今天，用最通俗易懂的方式给大家重新梳理一下，保证让你听懂了。

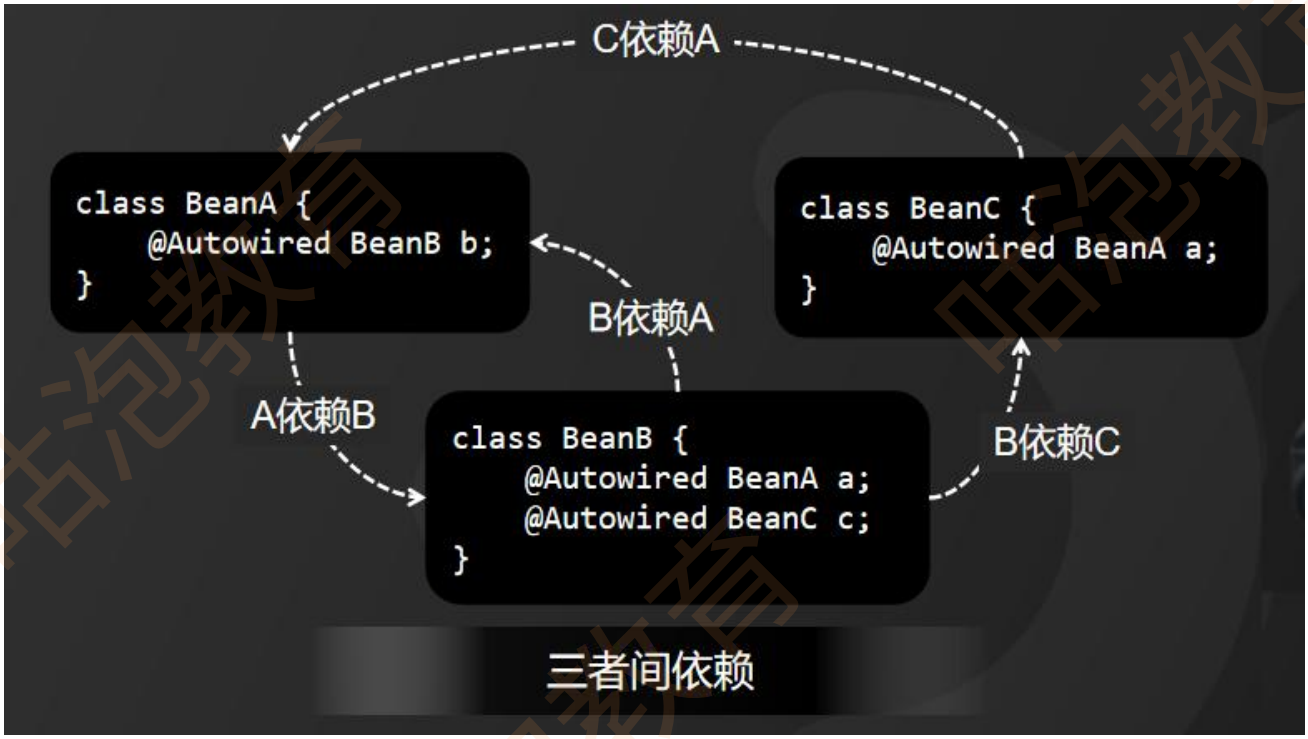
1、什么是循环依赖?

循环依赖就是指循环引用，是两个或多个Bean相互之间的持有对方的引用。循环依赖有三种形态：

1、相互依赖，也就是A 依赖 B，B 又依赖 A，它们之间形成了循环依赖。



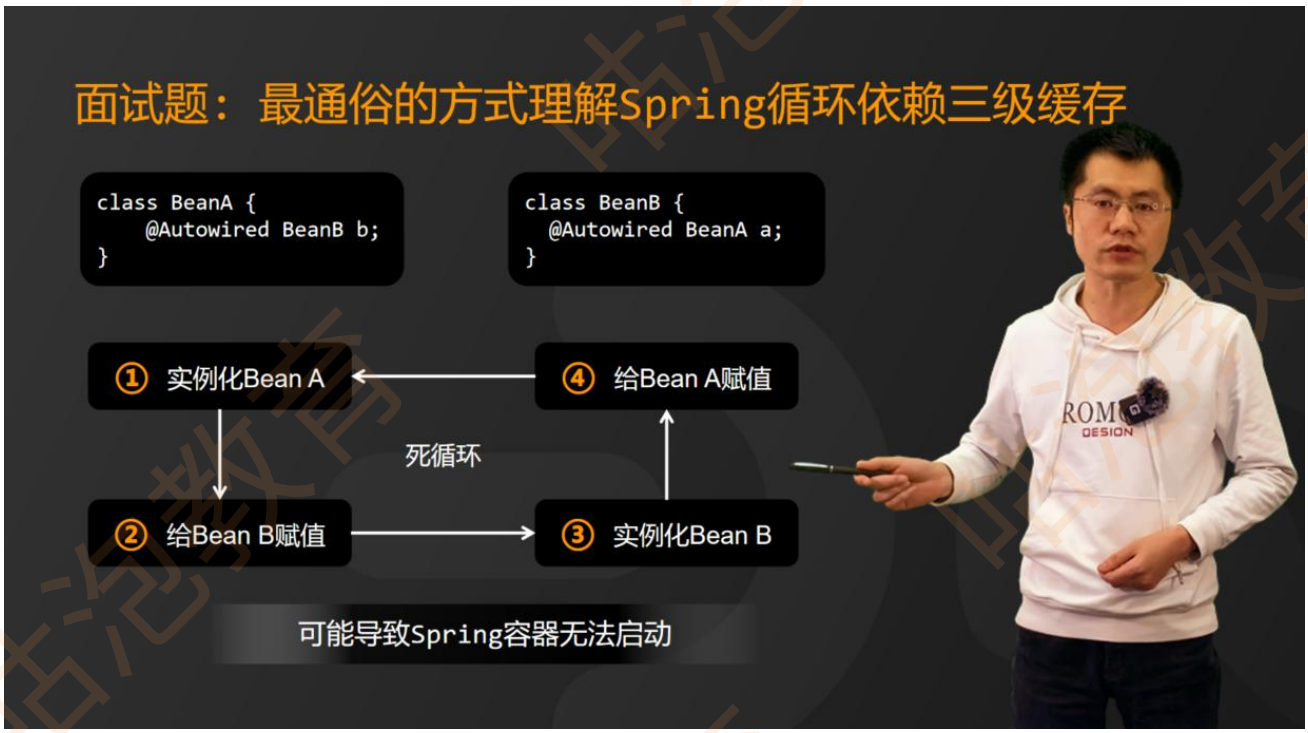
2、三者间依赖，也就是A 依赖 B，B 依赖 C，C 又依赖 A，形成了循环依赖。



3、自我依赖，也是A依赖A形成了循环依赖自己依赖自己。

2、如何解决循环依赖问题?

循环依赖本身没有问题，问题是Spring中加入了依赖注入机制，也就是自动给属性赋值。当创建Bean实例化以后，需要给Bean中需要赋值的属性全部自动赋值才能交给用户使用。但如果是循环依赖的情况，以两个Bean相互依赖的情况为例，



17.7 Spring中BeanFactory和FactoryBean的区别

MIC老师解析

一个工作了六年多的粉丝，胸有成竹的去京东面试。

然后被Spring里面的一个问题卡住，唉，我和他说，6年啦，Spring都没搞明白？

那怎么去让面试官给你通过呢？

这个问题是：Spring中BeanFactory和FactoryBean的区别。

好吧，对于这个问题看看高手的回答。

高手

关于这个问题，我从几个方面来回答。

首先，Spring里面的核心功能是IOC容器，所谓IOC容器呢，本质上就是一个Bean的容器或者是一个Bean的工厂。

它能够根据xml里面声明的Bean配置进行bean的加载和初始化，然后BeanFactory来生产我们需要的各种各样的Bean。

所以我对BeanFactory的理解了有两个。

BeanFactory是所有Spring Bean容器的顶级接口，它为Spring的容器定义了一套规范，并提供像getBean这样的方法从容器中获取指定的Bean实例。

BeanFactory在产生Bean的同时，还提供了解决Bean之间的依赖注入的能力，也就是所谓的DI。

FactoryBean是一个工厂Bean，它是一个接口，主要的功能是动态生成某一个类型的Bean的实例，也就是说，我们可以自定义一个Bean并且加载到IOC容器里面。

它里面有一个重要的方法叫`getObject()`，这个方法里面就是用来实现动态构建Bean的过程。

Spring Cloud里面的OpenFeign组件，客户端的代理类，就是使用了FactoryBean来实现的。

以上就是我对这个问题的理解。

面试点评

这个问题，只要稍微看过Spring框架的源码，怎么都能回答出来。

关键在于你是否愿意逼自己去学习一些工作中不常使用的技术来提升自己。

在我看来，薪资和能力是一种等价交换，在市场经济下，能力一般又想获得更高薪资，很显然不可能！

好的，本期的普通人VS高手面试系列的视频就到这里结束了。

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

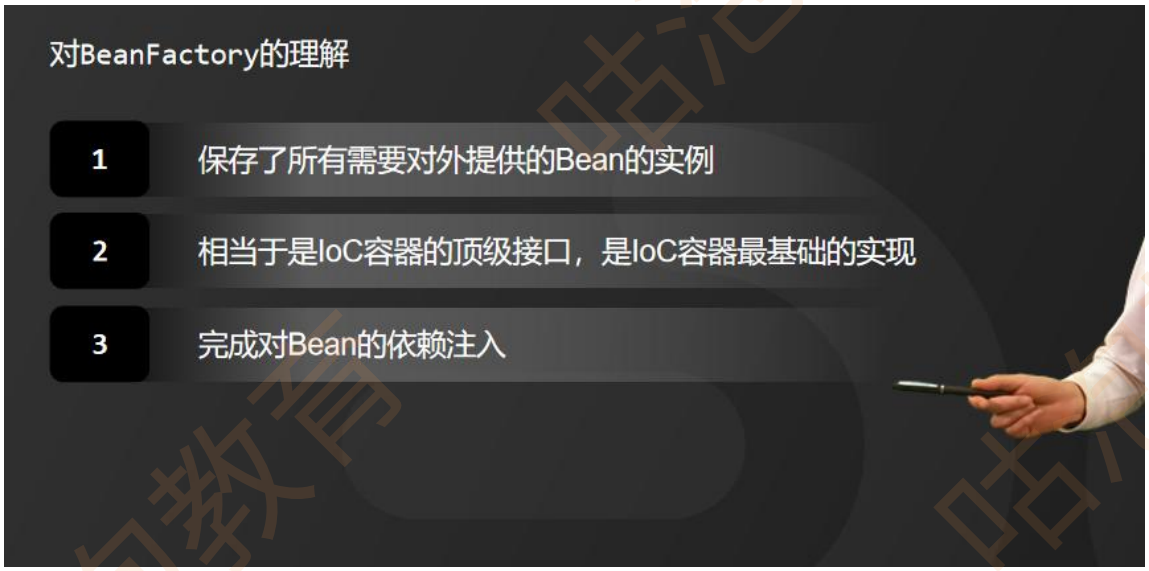
TOM老师解析

【引言】

一位工作了4年的小伙伴，去京东面试被问到这样一个问题，Spring中的BeanFactory和FactoryBean有什么区别？因为没有看过源码，当时就感觉这是一个文字游戏，感觉没什么区别？

那今天，我就给大家来聊清楚。另外，往期面试题解析中配套的文档我已经准备好，想获得的可以在我的煮叶简介中找到。好了，我们先来看BeanFactory。

1、BeanFactory



在Spring中最核心的就是Ioc容器，它保存了所有需要对外提供的Bean的实例。Spring对外暴露的ApplicationContext作为IoC容器最重要的接口，它也实现了BeanFactory接口。

所以，BeanFactory从名字上来看，它是一个Factory，相当于是IoC容器的顶级接口，是IoC容器最基础的实现。也是提供访问Spring容器的跟接口，主要负责Bean的创建和访问。其中，最主要的一个方法就是getBean()。同时，在BeanFactory中还会完成对Bean的依赖注入，也就是DI。

2、FactoryBean

那FactoryBean呢？从名字上来看它是一个特殊的Bean，这个Bean可以返回创建Bean的工厂。如果FactoryBean接口，可以根据不同的配置类型返回不同类型的Bean，它有一个比较核心的方法叫做getObject()。

FactoryBean在使用上也有些特殊。在前面介绍的BeanFactory中，有一个常量叫做FACTORY_BEAN_PREFIX = “&”，当我们想要从Spring容器中获取创建Bean的Factory时，可以在beanName前面加上 & 符号就可以获得Bean对应的Factory。

我们也可以自定义FactoryBean，用它来扩展创建Bean的规则。在Spring中创建的AOP动态代理Bean，就是通过实现FactoryBean来实现的。我们现在用的Spring Boot、Spring Cloud也经常通过实现FactoryBean接口实现自定义组件，从而修改默认的创建Bean的规则。

3、总结

以上，就是我对BeanFactory和FactoryBean的理解，回答这个问题啊，其实只要接触过一点点的Spring源码就能讲明白。所以，在内卷这么严重的环境下，工作3年以上，就应该看看源码了，这样才能提高核心竞争力。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

17.8 String、StringBuffer、StringBuilder区别

Hi，大家好，我是Mic，一个工作了14年的程序员和创业者。

今天分享一个非常基础但是有点深度的面试题。

记得10多年前我去找工作的时候，经常会碰到这个问题。

现在这一类的问题，考察的是应届生会比较多一点吧。

问题是： “String、StringBuffer、StringBuilder的区别”

下面来看看高手对这个问题的回答

高手

嗯，好的，面试官。

关于String、StringBuffer、StringBuilder的区别，我想从四个角度来说明。

第一个，可变性。

String内部的value值是final修饰的，所以它是不可变类。所以每次修改String的值，都会产生一个新的对象。

StringBuffer和StringBuilder是可变类，字符串的变更不会产生新的对象。

第二个，线程安全性。

String是不可变类，所以它是线程安全的。

StringBuffer是线程安全的，因为它每个操作方法都加了synchronized同步关键字。

StringBuilder不是线程安全的，所以在多线程环境下对字符串进行操作，应该使用StringBuffer，否则使用StringBuilder

第三个，性能方面。

String的性能是最的低的，因为不可变意味着在做字符串拼接和修改的时候，需要重新创建新的对象以及分配内存。

其次是StringBuffer要比String性能高，因为它的可变性使得字符串可以直接被修改最后是StringBuilder，它比StringBuffer的性能高，因为StringBuffer加了同步锁。

第四个，存储方面。

String存储在字符串常量池里面

StringBuffer和StringBuilder存储在堆内存空间。

以上就是我对这个问题的理解！

最后再补充一下，StringBuilder和StringBuffer都是派生自AbstractStringBuilder这个抽象类。

面试点评

这个问题其实还挺有意思的。

可能平时我们在使用字符串操作的时候并不会去关心性能以及线程安全性的影响。

但是确实在Java里面提供了不同的字符串操作，使得我们可以在不同的场景下使用不同的字符串对象。

这些小细节在开发中或多或少还是会有一些影响。

17.9 Spring中Bean的作用域有哪些?

一个工作3年的小伙子，去面试被问到Spring里面的问题。

这个问题比较简单，但是他却没有回答上来。

虽然他可以通过搜索引擎找到答案，但是如果没有理解，下次面试还是不会!

这个面试题是：“Spring中的Bean，作用域有哪些？”

对于这个问题，看看高手的回答。

高手

好的，这个问题可以从几个方面来回答。

首先呢，Spring框架里面的IOC容器，可以非常方便的去帮助我们管理应用里面的Bean对象实例。

我们只需要按照Spring里面提供的xml或者注解等方式去告诉IOC容器，哪些Bean需要被IOC容器管理就行了。

其次呢，既然是Bean对象实例的管理，那意味着这些实例，是存在生命周期，也就是所谓的作用域。

理论上来说，常规的生命周期只有两种：

singleton，也就是单例，意味着在整个Spring容器中只会存在一个Bean实例。

prototype，翻译成原型，意味着每次从IOC容器去获取指定Bean的时候，都会返回一个新的实例对象。

但是在基于Spring框架下的Web应用里面，增加了一个会话纬度来控制Bean的生命周期，主要有三个选择

request，针对每一次http请求，都会创建一个新的Bean

session，以sesssion会话为纬度，同一个session共享同一个Bean实例，不同的session产生不同的Bean实例

globalSession，针对全局session纬度，共享同一个Bean实例

以上就是我对这个问题的理解。

面试点评

“技术框架的本质是去解决特定问题的，所以如果能够站在技术的角度去思考Spring”

当遇到这种问题的时候，就可以像这个高手的回答一样，能够基于场景来推断出答案。

就像我们现在写CRUD代码，它已经变成了一种基本能力去让我们完成复杂业务逻辑的开发。

好的，本期的普通人VS高手面试系列的视频就到这里结束了。

有任何不懂的技术面试题，欢迎随时私信我

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

17.10 Spring中事务的传播行为有哪些?

MIC解析

一个工作了2年的粉丝，私信了一个比较简单的问题。

说：“Spring中事务的传播行为有哪些？”

他说他能记得一些，但是在项目中基本上不需要配置，所以一下就忘记了。

结果导致面试被拒绝，有点遗憾！

ok，关于这个问题，看看高手的回答。

高手

对于这个问题，需要从几个方面去回答。

首选，所谓的事务传播行为，就是多个声明了事务的方法相互调用的时候，这个事务应该如何传播。

比如说，methodA()调用methodB()，两个方法都显示的开启了事务。

那么methodB()是开启一个新事务，还是继续在methodA()这个事务中执行？就取决于事务的传播行为。

在Spring中，定义了7种事务传播行为。

REQUIRED：默认的Spring事物传播级别，如果当前存在事务，则加入这个事务，如果不存在事务，就新建一个事务。

REQUIRE_NEW：不管是否存在事务，都会新开一个事务，新老事务相互独立。外部事务抛出异常回滚不会影响内部事务的正常提交。

NESTED：如果当前存在事务，则嵌套在当前事务中执行。如果当前没有事务，则新建一个事务，类似于REQUIRE_NEW。

SUPPORTS：表示支持当前事务，如果当前不存在事务，以非事务的方式执行。

NOT_SUPPORTED：表示以非事务的方式来运行，如果当前存在事务，则把当前事务挂起。

MANDATORY：强制事务执行，若当前不存在事务，则抛出异常.

NEVER：以非事务的方式执行，如果当前存在事务，则抛出异常。

Spring事务传播级别一般不需要定义，默认就是PROPAGATION_REQUIRED，除非在嵌套事务的情况下需要重点了解。

以上就是我对这个问题的理解！

面试点评

这个问题其实只需要理解事务传播行为的本质以及为什么需要考虑到事务传播的问题。

就可以直接基于自身的技术积累来推演出答案，无非就是基于可能的策略进行穷举，怎么也能推演出5种吧。

好的，本期的普通人VS高手面试系列的视频就到这里结束了。

有任何不懂的技术面试题，欢迎随时私信我

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

TOM解析

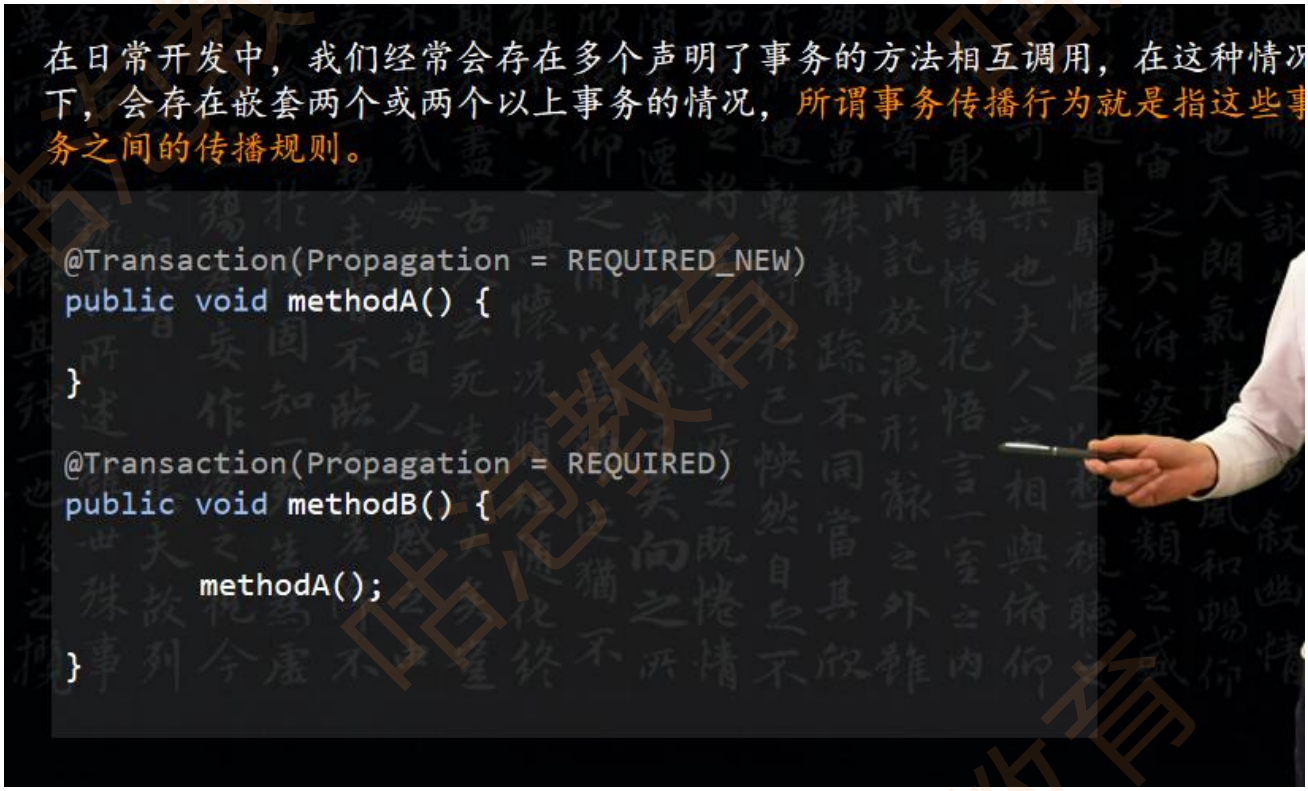
【引言】

1位工作2年的小伙伴面试的时候被问到这样一个问题，说，Spring中的事务传播行为有哪些？他说他在面试的时候能想起来一些，但在实际项目开发中又基本不需要配置。所以，在面试的时候回答不全，最后被拒了，觉得有些遗憾。

今天，我给大家分享一下，我对Spring传播行为的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的面试通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

1、事务传播行为



在日常开发中，我们经常会存在多个声明了事务的方法相互调用，在这种情况下，会存在嵌套两个或两个以上事务的情况，所谓事务传播行为就是指这些事务之间的传播规则。

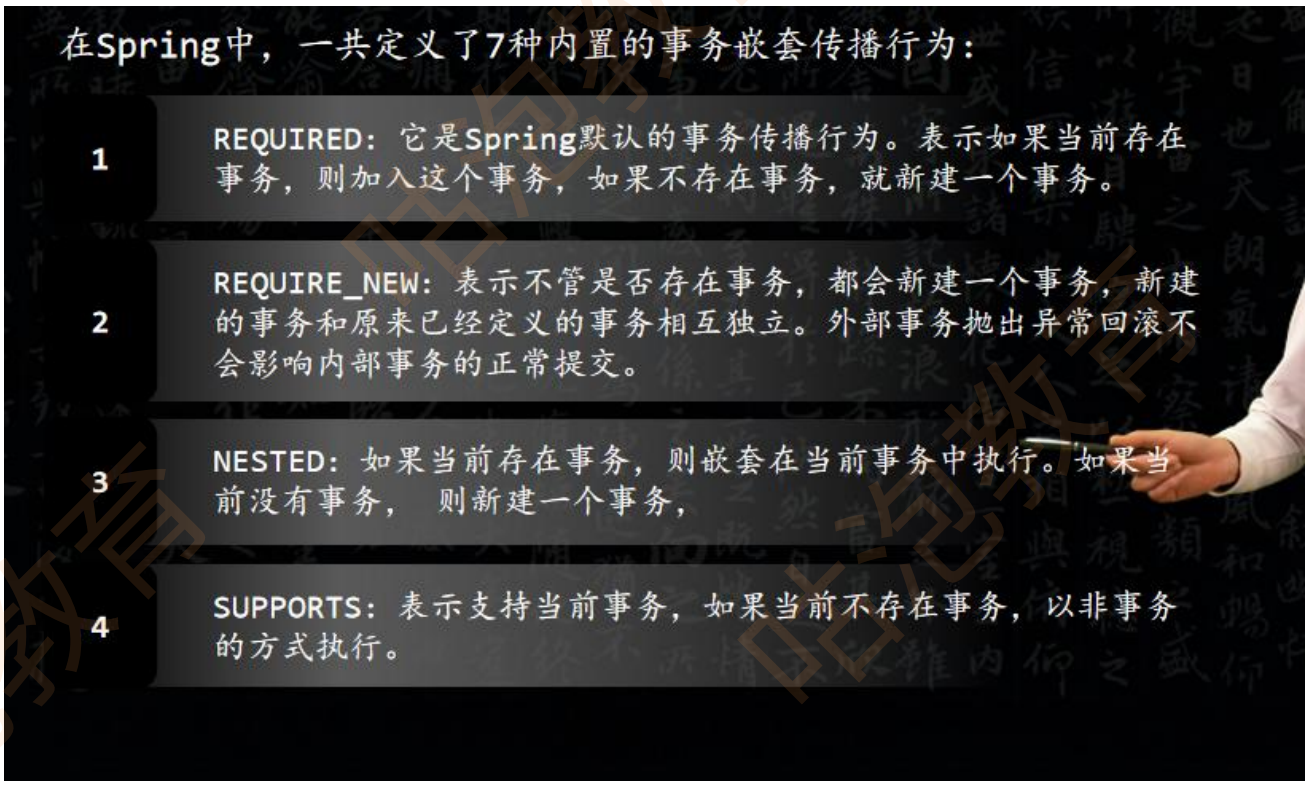
来看这样一段代码：

```
@Transaction(Propagation = REQUIRED_NEW)
public void methodA(){
```

```
}

@Transaction(Propagation = REQUIRED)

public void methodB(){

    methodA();

}
```

在这段代码中，methodB()调用了methodA()，两个方法都显示的开启了事务。methodA()是开启一个新事务，还是继续在methodB()这个事务中执行？就取决于事务的传播行为规则的定义。

2、事务嵌套



事务嵌套是指两个或两个以上开启的事务的方法嵌套调用，在这种情况下，需要制定

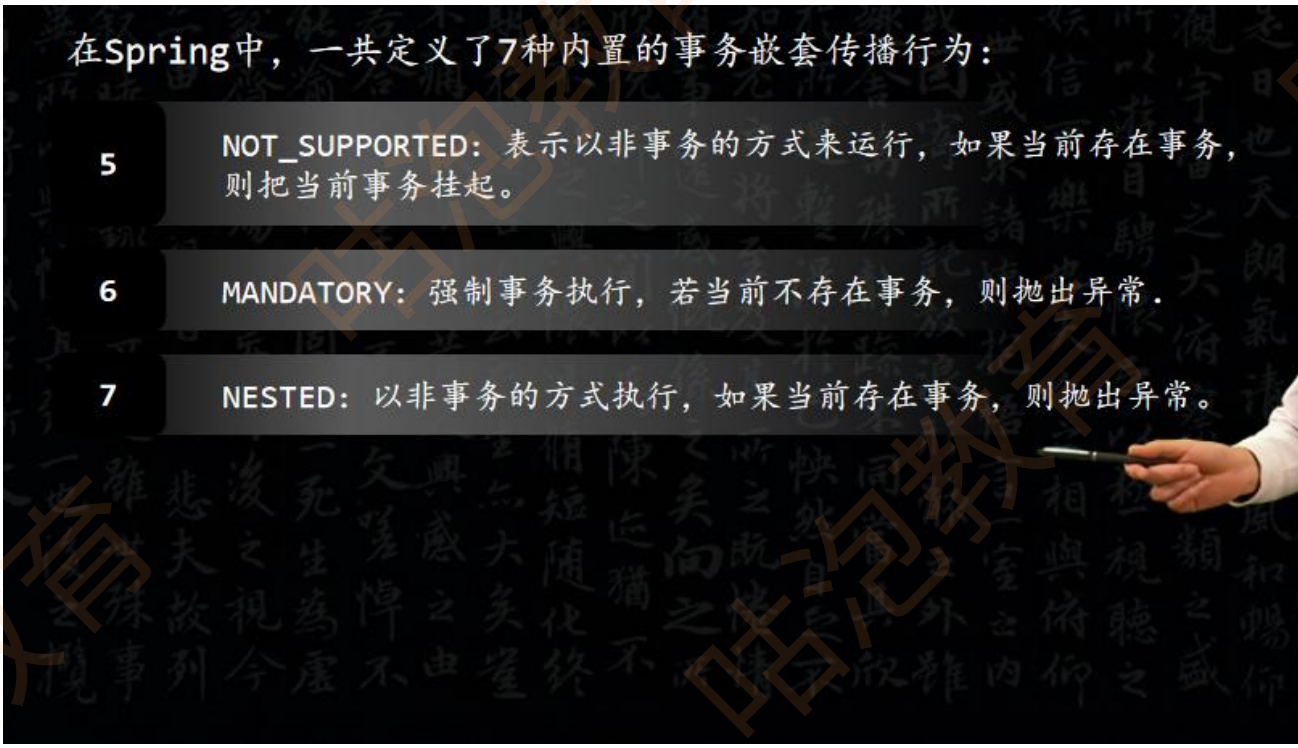
这些事务之间的传播行为规则。在Spring中，一共定义了7种内置的事务嵌套传播行为：

第1种：REQUIRED，它是Spring默认的事务传播行为。表示如果当前存在事务，则加入这个事务，如果不存在事务，就新建一个事务。

第2种：REQUIRE_NEW：表示不管是否存在事务，都会新建一个事务，新建的事务和原来已经定义的事务相互独立。外部事务抛出异常回滚不会影响内部事务的正常提交。

第3种：NESTED：如果当前存在事务，则嵌套在当前事务中执行。如果当前没有事务，则新建一个事务，类似于REQUIRE_NEW。

第4种：SUPPORTS：表示支持当前事务，如果当前不存在事务，以非事务的方式执行。

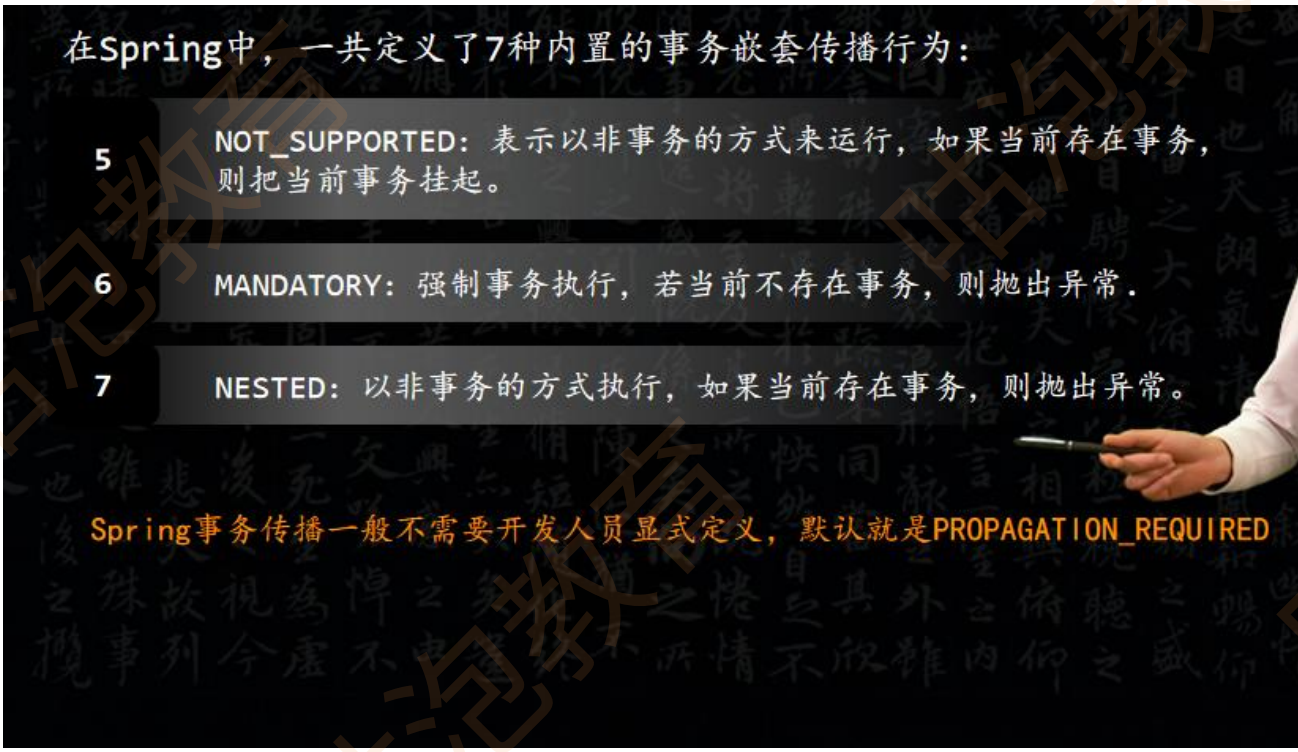


第5种：NOT_SUPPORTED：表示以非事务的方式来运行，如果当前存在事务，则把

当前事务挂起。

第6种：MANDATORY：强制事务执行，若当前不存在事务，则抛出异常.

第7种：NEVER：以非事务的方式执行，如果当前存在事务，则抛出异常。



Spring事务传播一般不需要开发人员显式定义，默认就是PROPAGATION_REQUIRED，只要在嵌套事务的情况下需要重点了解。

【结尾】

以上就是我对这个问题的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给

更多的人。关注我，面试不再难！

17.11 Spring 里面的事务和分布式事务的使用如何区分，以及这两个事务之间有什么关联?

Hi，大家好，我是Mic，一个工作了14年的程序员和创业者。

昨天一个粉丝私信我一个问题。

他去面试的时候遇到一个这样的问题。

“Spring 里面的事务和分布式事务的使用如何区分，以及这两个事务之间有什么关联”我没有想到，还有人回答不了这个问题。

所以，今天我们就来说一下这个面试题。

对于这个问题，来看看高手的回答。

高手

首先， 在Spring里面并没有提供事务，它只是提供了对数据库事务管理的封装。

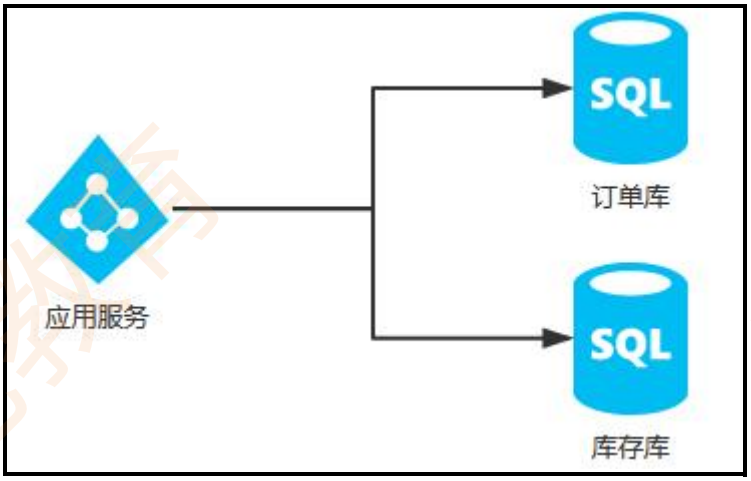
通过声明式的事务配置，使得开发人员可以从一些复杂的事务处理中得到解脱，

我们不再需要关心连接的获取、连接的关闭、事务提交、事务回滚这些操作。

更加聚焦在业务开发层面。

所以，Spring里面的事务，本质上就是数据库层面的事务，这种事务的管理，主要是针对单个数据库里面多个数据表操作的，去满足事务的ACID特性。

分布式事务，是解决多个数据库的事务操作的数据一致性问题，传统的关系型数据库不支持跨库事务的操作，所以需要引入分布式事务的解决方案。



而Spring并没有提供分布式事务场景的支持，所以Spring事务和分布式事务在使用上并没有直接的关联性。

但是我们可以使用一些主流的事务解决框架，比如Seata，集成到Spring生态里面，去解决分布式事务的问题。

以上就是我对这个问题的理解！

面试点评

其实面试的时候不应该问这一类的问题，

因为Spring的事务和分布式事务虽然在名字上类似，但是完全就是两个概念。

我估计这个问题，面试管考察的是一些刚毕业的小朋友吧。

17.12 Spring Bean生命周期的执行流程

MIC老师解析

普通人

Spring Bean的生命周期，可以分为单例、多实例。呃... 不对，这个是Spring Bean的作用域。

生命周期，我想想....

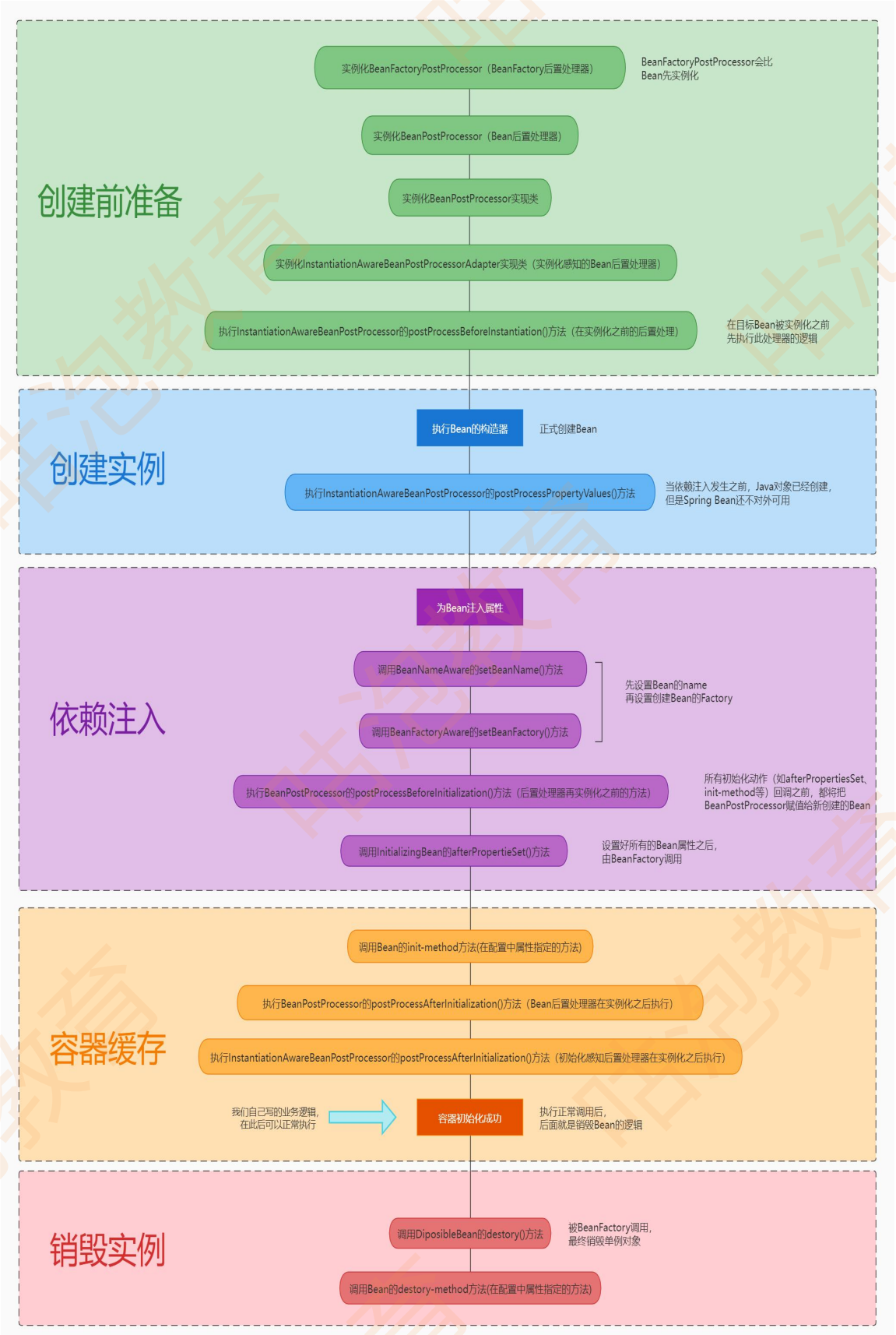
我记得Bean的生命周期会有加载、实例化、销毁这些阶段，其他的记得不是很清晰。

高手

Spring生命周期全过程大致分为五个阶段：创建前准备阶段、创建实例阶段、依赖注入阶段、

容器缓存阶段和销毁实例阶段。

这张是Spring Bean生命周期完整流程图，其中对每个阶段的具体操作做了详细介绍：


创建前准备
实例化BeanFactoryPostProcessor（BeanFactory后置处理器）
BeanFactoryPostProcessor会比Bean先实例化
实例化BeanPostProcessor（Bean后置处理器）
实例化BeanPostProcessor实现类
实例化InstantiationAwareBeanPostProcessorAdapter实现类（实例化感知的Bean后置处理器）
执行InstantiationAwareBeanPostProcessor的postProcessBeforeInstantiation()方法（在实例化之前的后置处理）
在目标Bean被实例化之前先执行此处理器的逻辑
创建实例
执行Bean的构造器
正式创建Bean
执行InstantiationAwareBeanPostProcessor的postProcessPropertyValues()方法
当依赖注入发生之前，Java对象已经创建，但是Spring Bean还不对外可用
依赖注入
为Bean注入属性
调用BeanNameAware的setBeanName()方法
调用BeanFactoryAware的setBeanFactory()方法
先设置Bean的name
再设置创建Bean的Factory
执行BeanPostProcessor的postProcessBeforeInitialization()方法（后置处理器再实例化之前的方法）
所有初始化动作（如afterPropertiesSet、init-method等）回调之前，都将把BeanPostProcessor赋值给新创建的Bean
调用InitializingBean的afterPropertieSet()方法
设置好所有的Bean属性之后，由BeanFactory调用
容器缓存
调用Bean的init-method方法(在配置中属性指定的方法)
执行BeanPostProcessor的postProcessAfterInitialization()方法（Bean后置处理器在实例化之后执行）
执行InstantiationAwareBeanPostProcessor的postProcessAfterInitialization()方法（初始化感知后置处理器在实例化之后执行）
我们自己写的业务逻辑，在此后可以正常执行
容器初始化成功
执行正常调用后，后面就是销毁Bean的逻辑
销毁实例
调用DiposibleBean的destory()方法
被BeanFactory调用，最终销毁单例对象
调用Bean的destory-method方法(在配置中属性指定的方法)

一、创建前准备阶段

这个阶段主要的作用是，Bean在开始加载之前，需要从上下文和相关配置中解析并查找Bean有关的扩展实现，

比如像`init-method`-容器在初始化bean时调用的方法、`destory-method`，容器在销毁bean时调用的方法。

以及，BeanFactoryPostProcessor这类的bean加载过程中的前置和后置处理。

这些类或者配置其实是Spring提供给开发者，用来实现Bean加载过程中的扩展机制，在很多和Spring集成的中间件中比较常见，比如Dubbo。

二、创建实例阶段

这个阶段主要是通过反射来创建Bean的实例对象，并且扫描和解析Bean声明的一些属性。

三、依赖注入阶段

如果被实例化的Bean存在依赖其他Bean对象的情况，则需要对这些依赖bean进行对象注入。比如常见的`@Autowired`、setter注入等依赖注入的配置形式。

同时，在这个阶段会触发一些扩展的调用，比如常见的扩展类： BeanPostProcessors（用来实现bean初始化前后的扩展回调）、

InitializingBean（这个类有一个afterPropertiesSet()，这个在工作中也比较常见）、BeanFactoryAware等等。

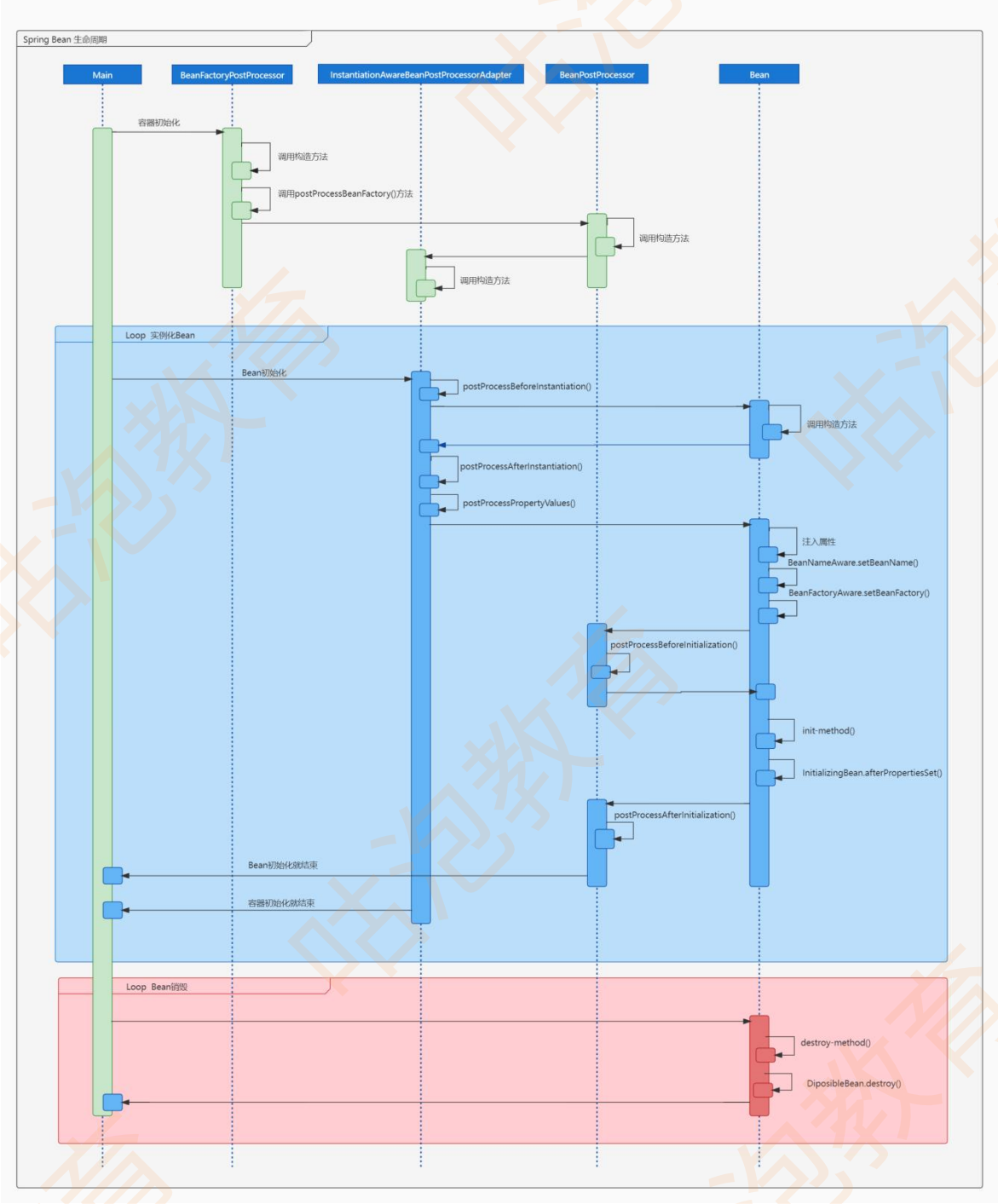
五、销毁实例阶段

当Spring应用上下文关闭时，该上下文中的所有bean都会被销毁。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

如果存在Bean实现了DisposableBean接口，或者配置了destory-method属性， 会在这个阶段被调用。

MIC：嗯，看完高手的回答后，相信大家对Spring Bean 的生命周期有了深刻的印象了，需要文档中Spring Bean生命周期的高清流程图，可以加微信：mic6769。在附赠一张高清的时序图给大家！



TOM老师解析

大家好，我是被编程耽误的文艺Tom。今天能给大家介绍一下Spring Bean生命周期

全过程，这道题呢也是大厂高频面试题。接下来我给大家做一个详细的分析和解答。

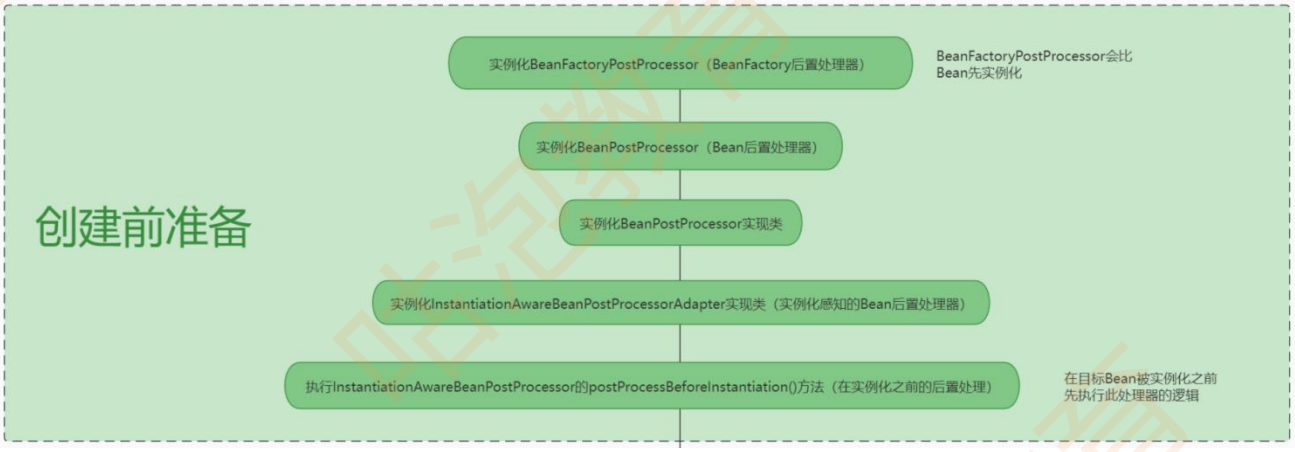
Spring生命周期全过程大致分为五个阶段：创建前准备阶段、创建实例阶段、依赖注入阶段、

容器缓存阶段和销毁实例阶段。

这张图呢展示了Spring Bean生命周期完整流程，其中对每个阶段的具体操作做了详细介绍。下面我们来分析一下每个阶段的细节。

先来看第一阶段。

1、创建前准备阶段



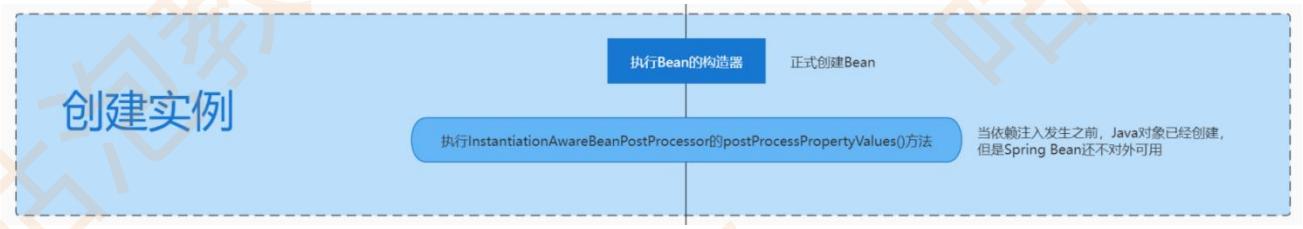
这个阶段主要是在开始Bean加载之前，从Spring上下文和相关配置中解析并查找Bean有关的配置内容，

比如`init-method`-容器在初始化bean时调用的方法、`destory-method`，容器在销毁Bean时调用的方法。

以及，BeanFactoryPostProcessor这类的bean加载过程中的前置和后置处理。

这些类或者配置其实是Spring提供给开发者，用来实现Bean加载过程中的扩展机制，在很多和Spring集成的中间件经常使用，比如Dubbo。

2、创建实例阶段



这个阶段主要是通过反射来创建Bean的实例对象，并且扫描和解析Bean声明的一些属性。

3、依赖注入阶段

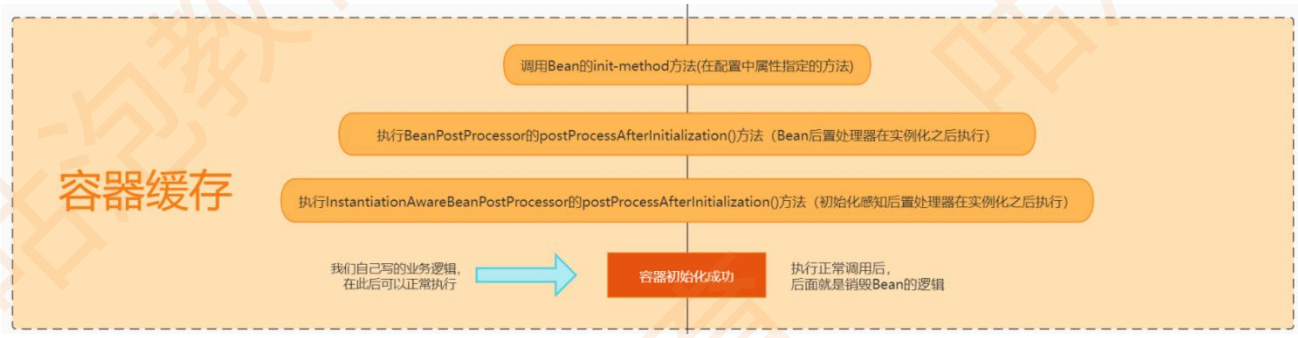
在这个阶段，会检测被实例化的Bean是否存在其他依赖，如果存在其他依赖，1就需要对这些被依赖Bean进行注入。比如通过读取

`@Autowired`、@Setter等依赖注入的配置。

在这个阶段还会触发一些扩展的调用，比如常见的扩展类：BeanPostProcessors（用来实现Bean初始化前后的回调）、

InitializingBean类（这个类有一个afterPropertiesSet()方法，给属性赋值）、还有BeanFactoryAware等等。

4、容器缓存阶段



容器缓存阶段主要是把Bean保存到IoC容器中缓存起来，到了这个阶段，Bean就可以被开发者使用了。

这个阶段涉及到的操作，常见的有，`init-method`这个属性配置的方法，会在这个阶段调用。

在比如BeanPostProcessors方法中的后置处理器方法如：postProcessAfterInitialization，也是在这个阶段触发的。

5、销毁实例阶段

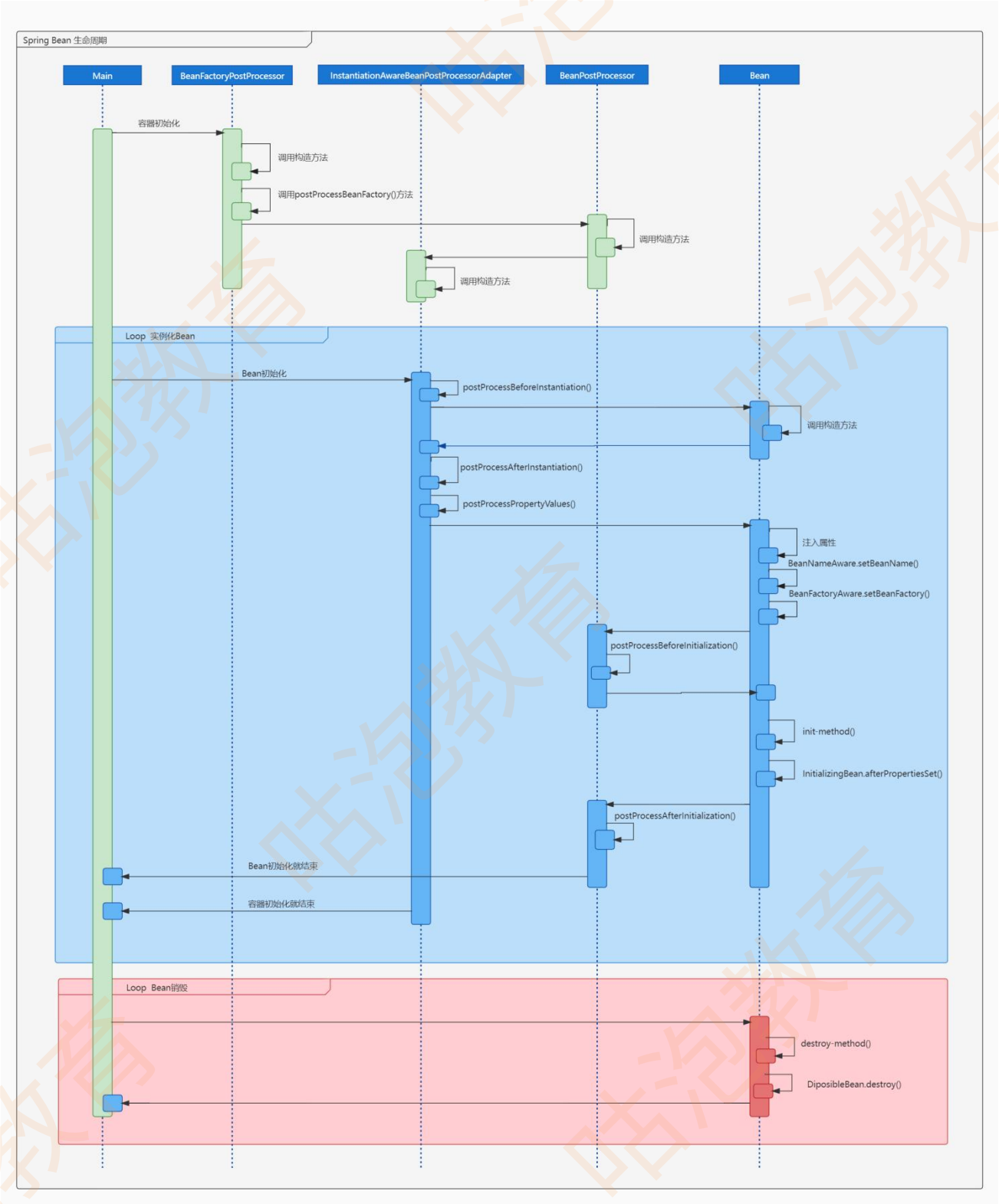
学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

这个阶段，是完成Spring应用上下文关闭时，将销毁Spring上下文中所有的Bean。

如果Bean实现了DisposableBean接口，或者配置了`destory-method`属 性，将会在这个阶段被调用。

看到这里，相信大家对Spring Bean的生命周期有了深刻的印象，需要视频中Spring Bean生

命周期的高清流程图，可以私信我。在附赠一张高清的时序图给大家！



我是被编程耽误的文艺Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

17.13 Spring中有哪些方式可以把Bean注入到IOC容器?

今天收到一个工作4年的粉丝的面试题。

问题是：“Spring中有哪些方式可以把Bean注入到IOC容器”。

他说这道题是所有面试题里面回答最好的，但是看面试官的表情，好像不太对。

我问他怎么回答的，他说：“接口注入”、“Setter注入”、“构造器注入”。

为什么不对？来看看高手的回答。

高手

好的，把Bean注入到IOC容器里面的方式有7种方式

使用xml的方式来声明Bean的定义，Spring容器在启动的时候会加载并解析这个xml，把bean装载到IOC容器中。

使用@CompontScan注解来扫描声明了@Controller、@Service、@Repository、@Component注解的类。

使用@Configuration注解声明配置类，并使用@Bean注解实现Bean的定义，这种方式其实是xml配置方式的一种演变，是Spring迈入到无配置化时代的里程碑。

使用@Import注解，导入配置类或者普通的Bean

使用FactoryBean工厂bean，动态构建一个Bean实例，Spring Cloud OpenFeign里面的动态代理实例就是使用FactoryBean来实现的。

实现ImportBeanDefinitionRegistrar接口，可以动态注入Bean实例。这个在Spring Boot里面的启动注解有用到。

实现ImportSelector接口，动态批量注入配置类或者Bean对象，这个在Spring Boot里面的自动装配机制里面有用到。

以上就是我对这个问题的理解。

面试点评

工作了4年，IOC和DI都没有搞清楚，作为面试官，想给你放水都放不了啊。

这道题目也很有意义，要想更加优雅的去解决一些实际业务问题，首先得有足够多的工具积累。

你可曾想过，Bean的注入竟然有这么多方式，而且还有些方式是没听过的呢？

好的，本期的普通人VS高手面试系列的视频就到这里结束了。

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

17.14 Spring Boot中自动装配机制的原理

MIC老师解析

最近一个粉丝说，他面试了4个公司，有三个公司问他：“Spring Boot中自动装配机制的原理”

他回答了，感觉没回答错误，但是怎么就没给offer呢?

对于这个问题，看看普通人和高手该如何回答。

普通人

嗯...Spring Boot里面的自动装配，就是@EnableAuoConfiguration注解。

嗯...它可以实现Bean的自动管理，不需要我们手动再去配置。

高手

自动装配，简单来说就是自动把第三方组件的Bean装载到Spring IOC器里面，不需要开发人员再去写Bean的装配配置。

在Spring Boot应用里面，只需要在启动类加上@SpringBootApplication注解就可以实现自动装配。

@SpringBootApplication是一个复合注解，真正实现自动装配的注解是@EnableAutoConfiguration。

自动装配的实现主要依靠三个核心关键技术。

引入Starter启动依赖组件的时候，这个组件里面必须要包含@Configuration配置类，在这个配置类里面通过@Bean注解声明需要装配到IOC容器的Bean对象。

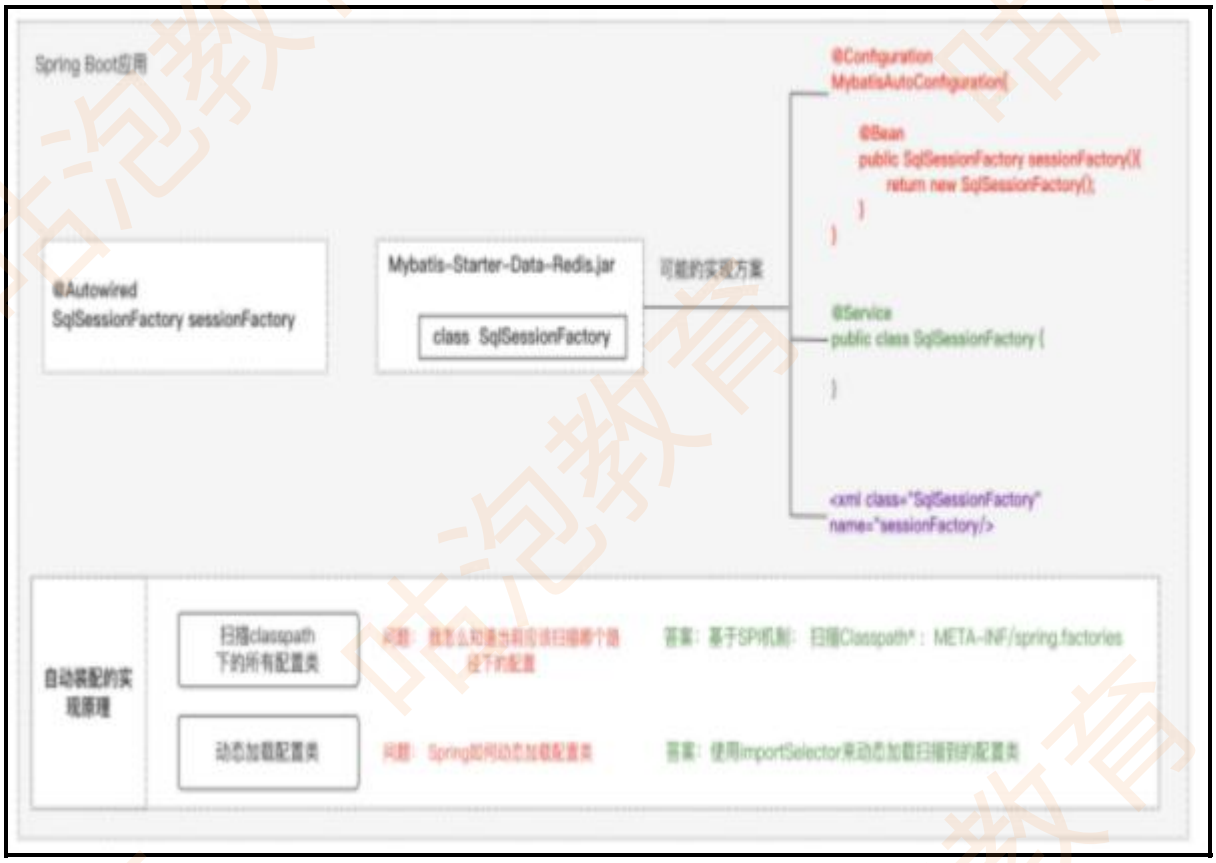
这个配置类是放在第三方的jar包里面，然后通过SpringBoot中的约定优于配置思想，把这个配置类的全路径放在classpath:/META-INF/spring.factories文件中。这样SpringBoot就可以知道第三方jar包里面的配置类的位置，这个步骤主要是用到了Spring里面的SpringFactoriesLoader来完成的。

SpringBoot拿到所第三方jar包里面声明的配置类以后，再通过Spring提供的ImportSelector接口，实现对这些配置类的动态加载。

在我看来，SpringBoot是约定优于配置这一理念下的产物，所以在很多的地方，都会看到这类的思想。它的出现，让开发人员更加聚焦在了业务代码的编写上，而不需要去关心和业务无关的配置。

其实，自动装配的思想，在SpringFramework3.x版本里面的@Enable注解，就有了实现的雏形。@Enable注解是模块驱动的意思，我们只需要增加某个@Enable注解，就自动打开某个功能，而不需要针对这个功能去做Bean的配置，@Enable底层也是帮我们去自动完成这个模块相关Bean的注入。

以上，就是我对Spring Boot自动装配机制的理解。



结尾

发现了吗？高手和普通人的回答，并不是回答的东西多和少。

而是让面试官看到你对于这个技术领域的理解深度和自己的见解，从而让面试官在一大堆求职者中，对你产生清晰的印象。

TOM老师解析

【引言】

昨天，有位在广州工作4年的小伙伴，在面试中被问到SpringBoot自动装配原理，当时，自我感觉比较好，他要的是30K，但是都没有拿到Offer。今天，我给大家分享一下我的理解。

1、Spring Boot自动装配是什么?

SpringBoot自动装配主要是基于注解编程 和 约定优于配置的思想来设计的。

(开始展示动画)

自动装配就是由Spring自动把其他组件中的Bean装载到IoC容器中，不需要开发人员再去配置文件中添加大量的配置。我们只需要在Spring Boot的启动类上添加@SpringBootApplication 注解，开启自动装配。

这种自动装配的思想，在Spring 3.x以后就开始支持，我们只要在类上添加@Enable 注解就可以了，只是没有像Spring Boot这样全面地去设计。

因此，Spring 和 Spring Boot的最大区别就是在于Spring Boot的自动装配。那自动装配的原理又是什么呢?

2、自动装配原理

@SpringBootApplication这个注解是暴露给用户使用的入口，它的底层是由@EnableAutoConfiguration这个注解来实现的。

这样一个自动装配的实现，我把它归纳为以下三个核心步骤：

第一步：启动依赖组件的时候，组件中必须要包含 @Configuration 的配置类，在这个配置类里面声明为 @Bean 注解，就将方法的返回值或者属性值 注入到IoC容器中。

第二步：如果是使用第三方jar包，Spring Boot采用SPI机制，只需要在/META-INF/目录下增加spring.factories配置文件。然后，Spring Boot会根据约定规则，自动使用SpringFactoriesLoader来加载配置文件中的内容。

第三步：Spring获取到第三方jar中的配置以后，会使用调用ImportSelector接口来完成动态加载。

这样的设计的好处在于，大幅减少了臃肿的配置文件，而且各模块之间的依赖实现了深度解耦。比如，我们使用Spring创建Web程序时需要导入非常多的Maven依赖，而Spring Boot只需要一个Maven依赖来创建Web程序，并且Spring Boot还把我们最常用

的依赖都放到了一起，我们只需要 引入 spring-boot-starter-web 这一个依赖就可以完成一个简单的Web应用。

以前用Spring的时候需要XML文件配置开启一些功能，现在Spring Boot不用XML配置了，只需要写一个， 加了@Configuration注解 或者实 现对应接口的配置类就可以了。

3、总结

最后，我总结一下，Spring Boot自动装配是Spring的完善和扩展，就是为我们便捷开发，方便测试和部署，提高效率而诞生的框架技术。

小伙伴们，如果你被问到过Spring Boot自动装配原理的问题，你是怎么回答的呢？可以在评论区分享你的回答。

我是被编程耽误的文艺Tom，如果你还有其他不懂的面试题或者需要视频配套的文字资料，可以关注我的主页加V。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

17.15 Spring Boot的约定优于配置，你的理解是什么？

MIC解析

对于Spring Boot约定优于配置这个问题，看看普通人和高手是如何回答的？

普通人的回答

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

嗯，在Spring Boot里面，通过约定优于配置这个思想，可以让我们少写很多的配置，

然后就只需要关注业务代码的编写就行。嗯！

高手的回答

我从4个点方面来回答。

首先，约定优于配置是一种软件设计的范式，它的核心思想是减少软件开发人员对于配置项的维护，从而让开发人员更加聚焦在业务逻辑上。

Spring Boot就是约定优于配置这一理念下的产物，它类似于Spring框架下的一个脚手架，通过Spring Boot，我们可以快速开发基于Spring生态下的应用程序。

基于传统的Spring框架开发web应用，我们需要做很多和业务开发无关并且只需要做一次的配置，比如

管理jar包依赖

web.xml维护

Dispatch-Servlet.xml配置项维护

应用部署到Web容器

第三方组件集成到Spring IOC容器中的配置项维护

而在Spring Boot中，我们不需要再去做这些繁琐的配置，Spring Boot已经自动帮我们完成了，这就是约定由于配置思想的体现。

Spring Boot约定由于配置的体现有很多，比如

Spring Boot Starter启动依赖，它能帮我们管理所有jar包版本

如果当前应用依赖了spring mvc相关的jar，那么Spring Boot会自动内置Tomcat容器来运行web应用，我们不需要再去单独做应用部署。

Spring Boot的自动装配机制的实现中，通过扫描约定路径下的spring.factories文件来识别配置类，实现Bean的自动装配。

默认加载的配置文件application.properties等等。

总的来说，约定优于配置是一个比较常见的软件设计思想，它的核心本质都是为了更高效以及更便捷的实现软件系统的开发和维护。

以上就是我对这个问题的理解。

结尾

好的，本期的普通人VS高手面试系列就到这里结束了，对于这个问题，你知道该怎么回答了吗？

另外，如果你有任何面试相关的疑问，欢迎评论区给我留言。

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

TOM解析

【引言】

在 Spring Boot 中，通过约定优于配置这个思想，可以让我们少写很多的配置，然后就只需要关注业务代码的编写就行。今天呢，我给大家聊聊为什么SpringBoot提倡约定优于配置。

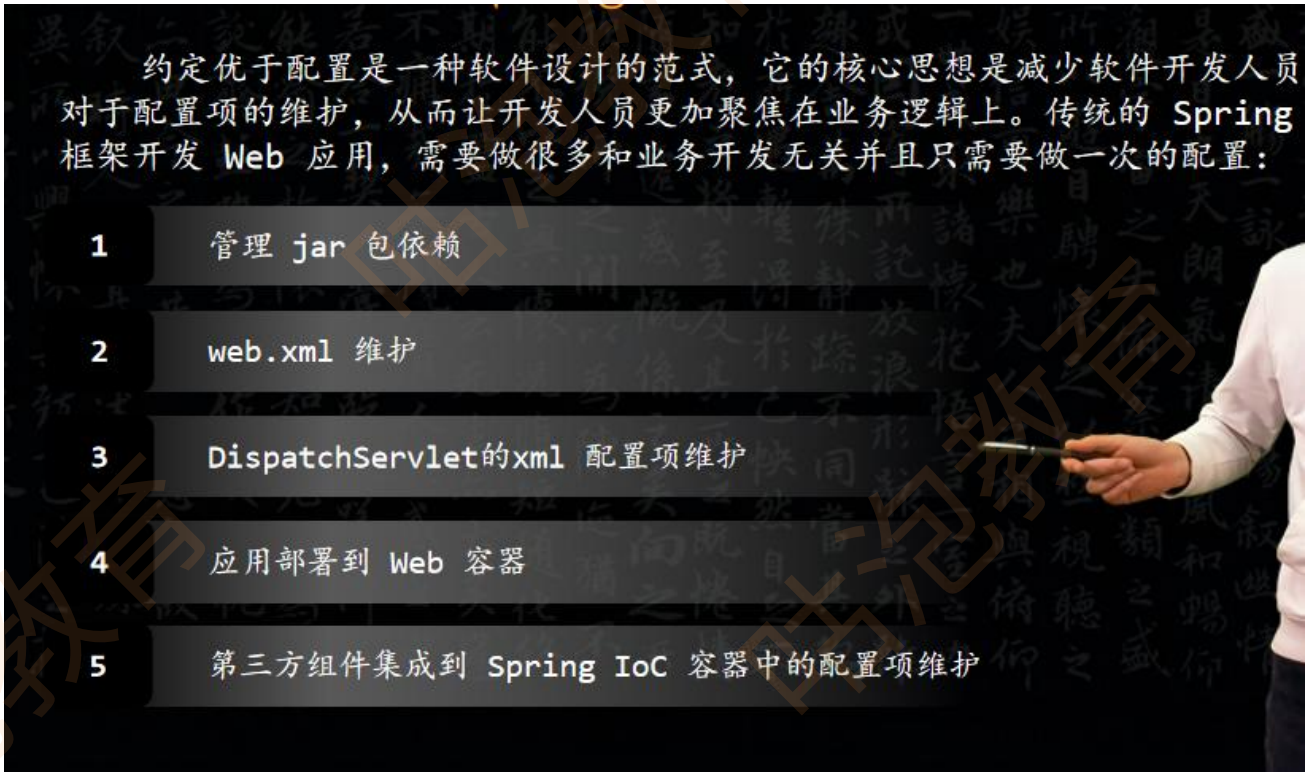
另外，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

1、原因分析

下面，我分享一下我对Spring'Boot提倡约定优于配置的理解。

约定优于配置是一种软件设计的范式，它的核心思想是减少软件开发人员对于配置项的维护，从而让开发人员更加聚焦在业务逻辑上。

Spring Boot 就是约定优于配置这一理念下的产物，它类似于 Spring 框架下的一个脚手架，通过 Spring Boot，我们可以快速开发基于 Spring 生态下的应用程序。



而基于传统的 Spring 框架开发 Web 应用，我们需要做很多和业务开发无关并且只需要做一次的配置，比如：

管理 jar 包依赖

web.xml 维护

DispatchServlet的xml 配置项维护

应用部署到 Web 容器

第三方组件集成到 Spring IoC 容器中的配置项维护

而在 Spring Boot 中，我们不需要再去做这些繁琐的配置，Spring Boot 已经自动帮我们完成了，这就是约定由于配置思想的体现。

2、如何体现?

Spring Boot 约定优于配置的体现有很多，比如：

1、Spring Boot Starter 启动依赖，它能帮我们管理所有 jar 包版本；

2、Spring Boot会自动内置Tomcat容器来运行 Web 应用，我们不需要再去单独做应用部署。

3、Spring Boot 通过扫描约定路径下的 Spring.factories文件来识别配置类，实现 Bean 的自动装配。

4、Spring Boot会默认加载的配置文件 application.properties 等等。

总的来说，约定优于配置是一个比较常见的软件设计思想，它的核心本质都是为了更高效以及更的实现软件系统的开发和维护。

【结尾】

以上就是我对这个问题的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指 分享给更多的人。关注我，面试不再难！

17.16 Spring Cloud的理解

MIC老师解析

Hi，大家好，我是Mic

一个工作了7年的Java粉丝，竟然连这个问题都回答不出来。

难怪被裁员之后，每次面试都被技术问题卡脖子，导致一直找不到工作。

今天他遇到这样一个问题，希望我能给出一个回答建议。

“请你说一下你对Spring Cloud的理解”。

下面看看普通人和高手对这个问题的回答。

普通人

Spring Cloud是一套微服务解决方案

它包括配置中心、RPC通信、服务注册、服务熔断等组件

高手

好的。

Spring Cloud是Spring官方推出来的一套微服务解决方案。

准确来说，我认为Spring Cloud其实是对微服务架构里面出现各种技术场景，定义了一套标准规范。

然后在这套标准里面，Spring集成了Netflix公司的OSS开源套件，比如

Zuul 实现应用网关、Eureka 实现服务注册与发现、Ribbon实现负载均衡、

Hystrix实现服务熔断我们可以使用Spring Cloud Netflix这套组件，快速落地微服务架构以及解决微服务治理等一系列问题。

但是随着Netflix OSS相关技术组件的闭源和停止维护，所以Spring官方也自研了一些组件，比如Gateway实现网关、 LoadBalancer实现负载均衡。

另外，Alibaba里面的开源组件也实现了Spring Cloud的标准，成为了Spring Cloud 里面的另外一套微服务解决方案。

包括Dubbo做rpc通信、Nacos实现服务注册与发现以及动态配置中心、Sentinel实现服务限流和服务降级等等。

以上就是我对Spring Cloud的理解，另外，我再补充一下，我认为Spring Cloud生态的出现有两个很重要的意义。

在Spring Cloud出现之前，为了解决微服务架构里面的各种技术问题，需要去集成各种开源框架，因为标准和兼容性问题，所以在实践的时候很麻烦，而Spring Cloud统一了这样一个标准。

降低了微服务架构的开发难度，只需要在Spring Boot的项目基础上通过starter启动依赖集成相关组件就能轻松解决各种问题。

以上就是我对这个问题的理解。

面试点评

Spring Cloud已经是普及度和应用非常高。

大家往往知道Spring Cloud怎么用，但是为什么用，以及什么场景下用。

却不一定能明白，而这道题目其实就是考察工作4年以上的求职责对技术的了解程度。

TOM老师解析

【引言】

最近，有很多应届生也挺焦虑的，说大家都卷，不卷也不行。很多应届生在面试过程中八股文背得不错，但是一聊到项目经验就不知如何说起。

有位小伙伴来问我，说面试被问到对Spring Cloud的理解，自己也回答了，但感觉面试官不满意，说是没有结合项目经验来回答。

今天，我给大家分享一下我的理解，希望能够帮助到各位小伙伴。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以在我的个人煮叶简介中找到。

1、Spring Cloud的应用背景

Spring Cloud是Spring官方推出来的一套微服务解决方案。

准确来说，Spring Cloud其实是对微服务架构里面出现各种技术场景，定义了一套标准规范。

然后，在这套标准里面，Spring集成了Netflix公司的OSS开源套件，比如

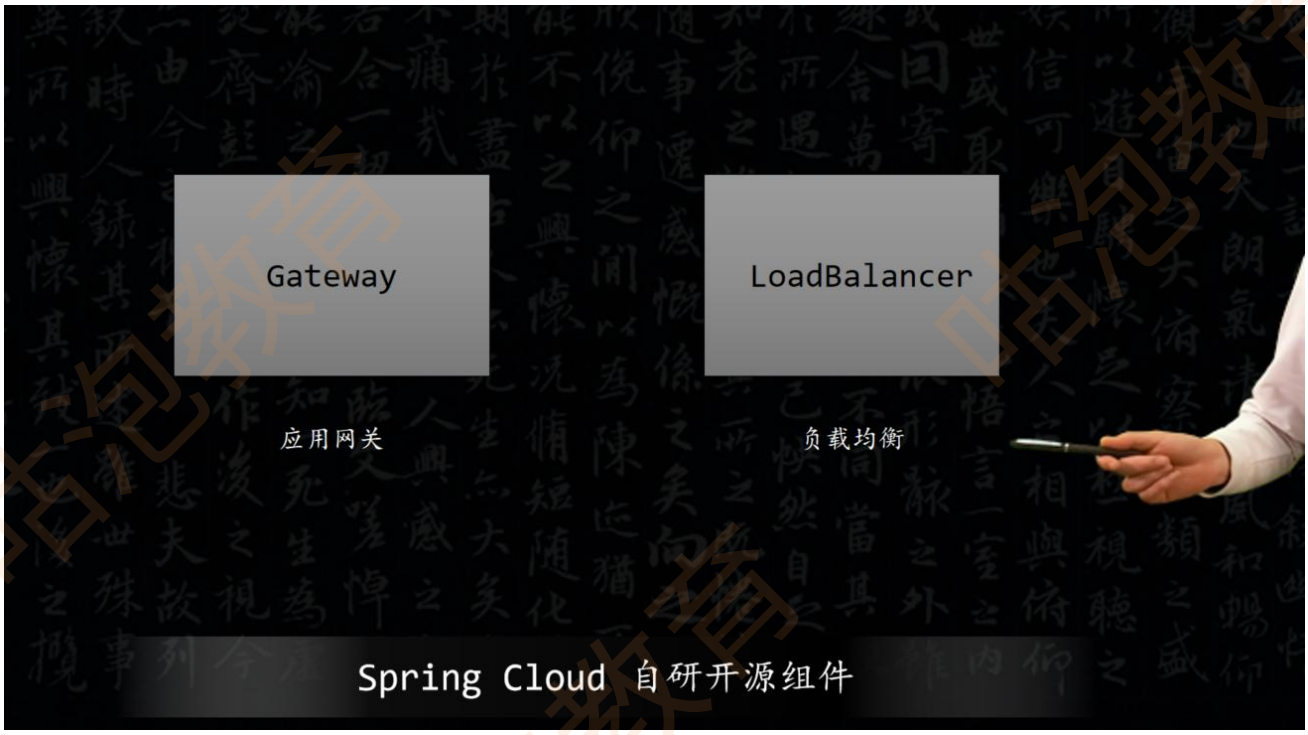
Zuul 实现应用网关、

Eureka 实现服务注册与发现、

Ribbon实现负载均衡、

Hystrix实现服务熔断

我们可以使用Spring Cloud Netflix这套组件，快速落地微服务架构以及解决微服务治理等一系列问题。



但是随着Netflix OSS相关技术组件的闭源和停止维护，所以Spring官方也自研了一些组件，

比如Gateway实现网关、 LoadBalancer实现负载均衡。

另外，在国内的阿里也在为开源做出积极的贡献，它整合了Spring Cloud的标准，同时，加入了自身的技术，成为了Spring Cloud生态中另一套微服务解决方案Spring Cloud Alibaba。

其中包括Dubbo做RPC通信、Nacos实现服务注册与发现以及动态配置中心、Sentinel实现服务限流和服务降级等等。

2、使用Spring Cloud的好处

以上就是我对Spring Cloud的理解。最后，我再补充一下，我认为Spring Cloud生态的出现有两个很重要的意义。

1、在Spring Cloud出现之前，为了解决微服务架构里面的各种技术问题，需要去集成各种开源框架，因为标准和兼容性问题，所以在实践的时候很麻烦。国内还有很些企业基于阿里的Dubbo进行二次开发自研出Dubbo X，而Spring Cloud统一了这样一个标准。

2、使用Spring Cloud降低了微服务架构的开发难度，只需要在Spring Boot的项目基础上通过Starter启动依赖集成相关组件就能轻松解决各种问题。

【结尾】

在现在的互联网分布式应用开发中，Spring Cloud的普及程度非常大了。很多应届生往往知道Spring Cloud怎么用，但是为什么用，以及在什么场景下用，却不一定能明白。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

17.17 Spring IoC的工作流程

Hi，我是Mic

一个工作了4年的粉丝，在面试的时候遇到一个这样的问题。

“介绍一下Spring IOC的工作流程”

他说回答得不是很好，希望我能帮他梳理一下。

这个问题高手部分的回答已经整理成了文档，可以在主页加V领取。

关于这个问题，我们来看看高手的回答。

高手

好的，这个问题我会从几个方面来回答。

IOC是什么

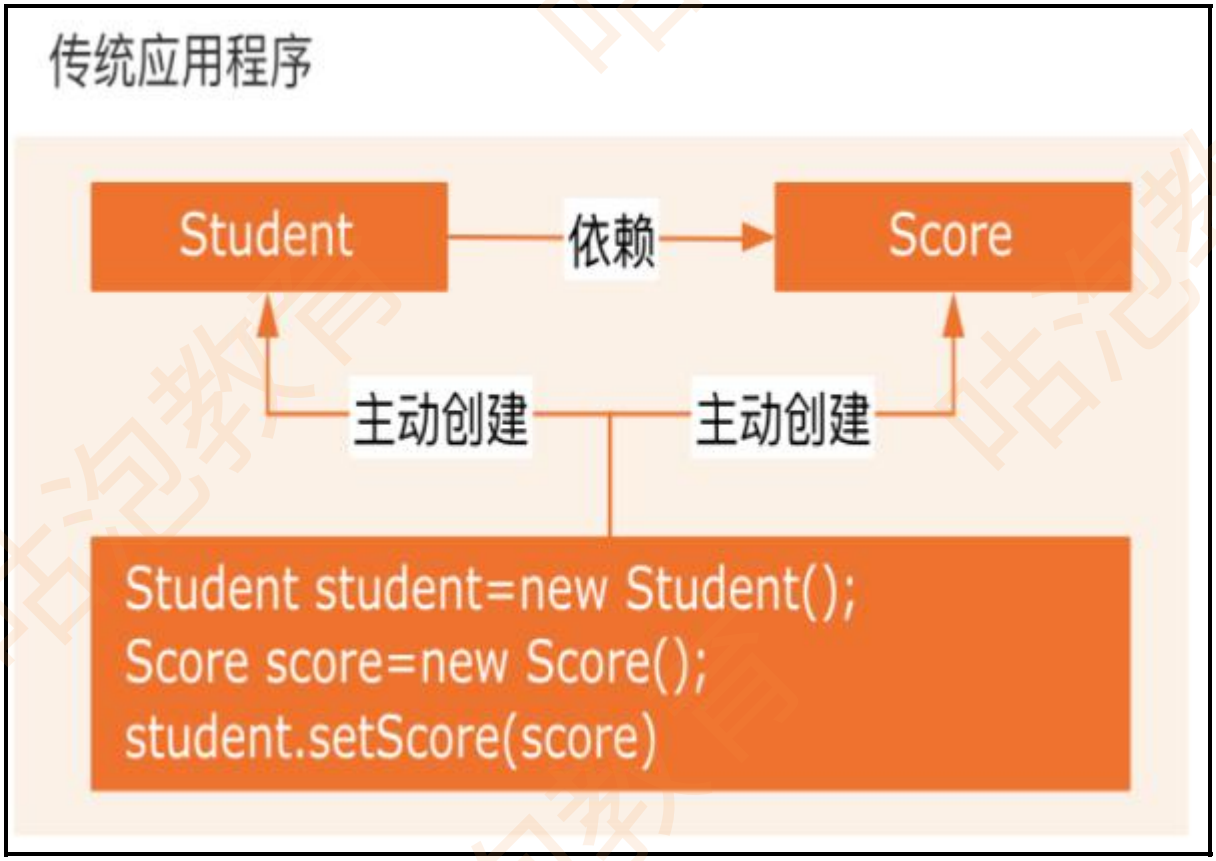
Bean的声明方式

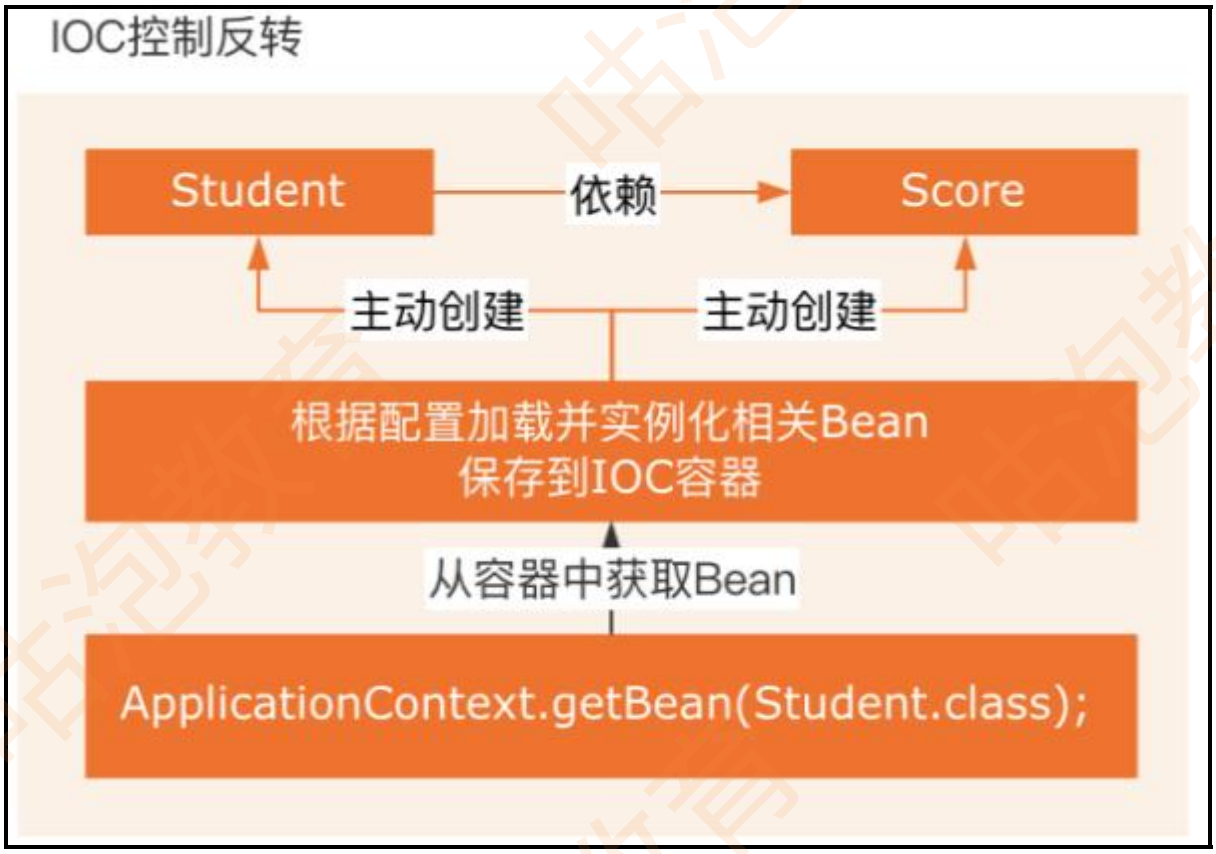
IOC的工作流程

IOC的全称是Inversion Of Control,也就是控制反转，它的核心思想是把对象的管理权限交给容器。

应用程序如果需要使用到某个对象实例，直接从IOC容器中去获取就行，这样设计的好处是降低了程序里面对象与对象之间的耦合性。

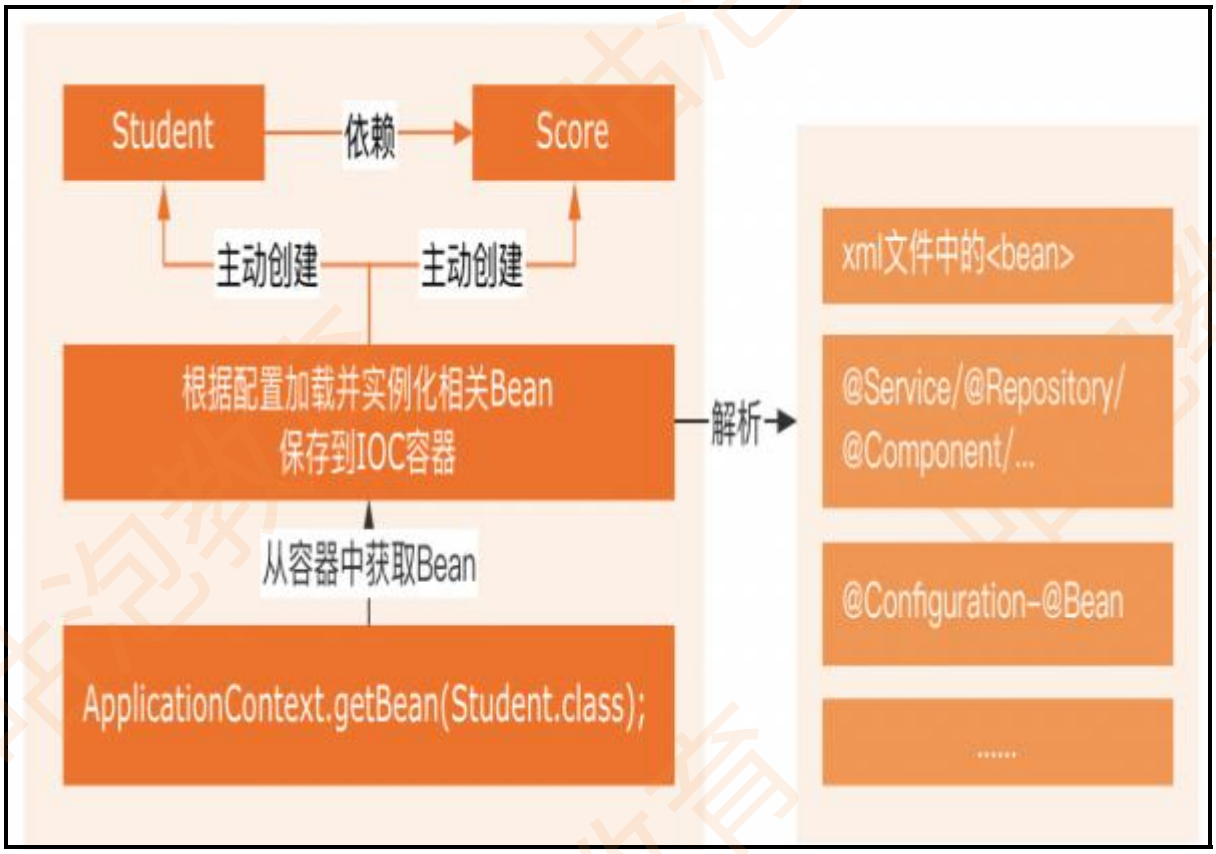
使得程序的整个体系结构变得更加灵活。





Spring里面很多方式去定义Bean， 比如XML里面的<bean>标签、@Service、@Component、@Repository、@Configuration配置类中的@Bean注解等等。

Spring在启动的时候，会去解析这些Bean然后保存到IOC容器里面。

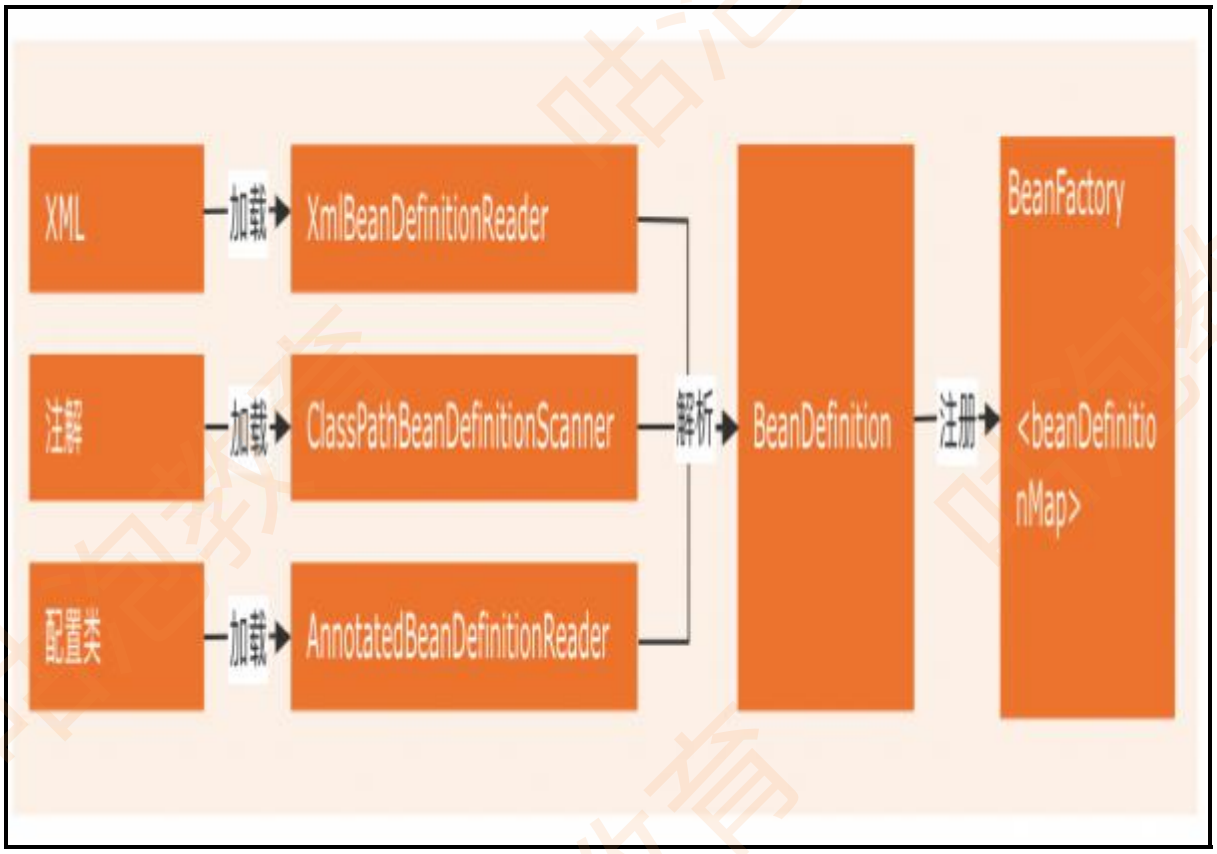


Spring IOC的工作流程大致可以分为三个阶段。

第一个阶段，就是IOC容器的初始化

这个阶段主要是根据程序中定义的XML或者注解等Bean的声明方式

通过解析和加载后生成BeanDefinition，然后把BeanDefinition注册到IOC容器。



通过注解或者xml声明的bean都会解析得到一个BeanDefinition实体，实体中包含这个bean中定义的基本属性。

最后把这个BeanDefinition保存到一个Map集合里面，从而完成了IOC的初始化。

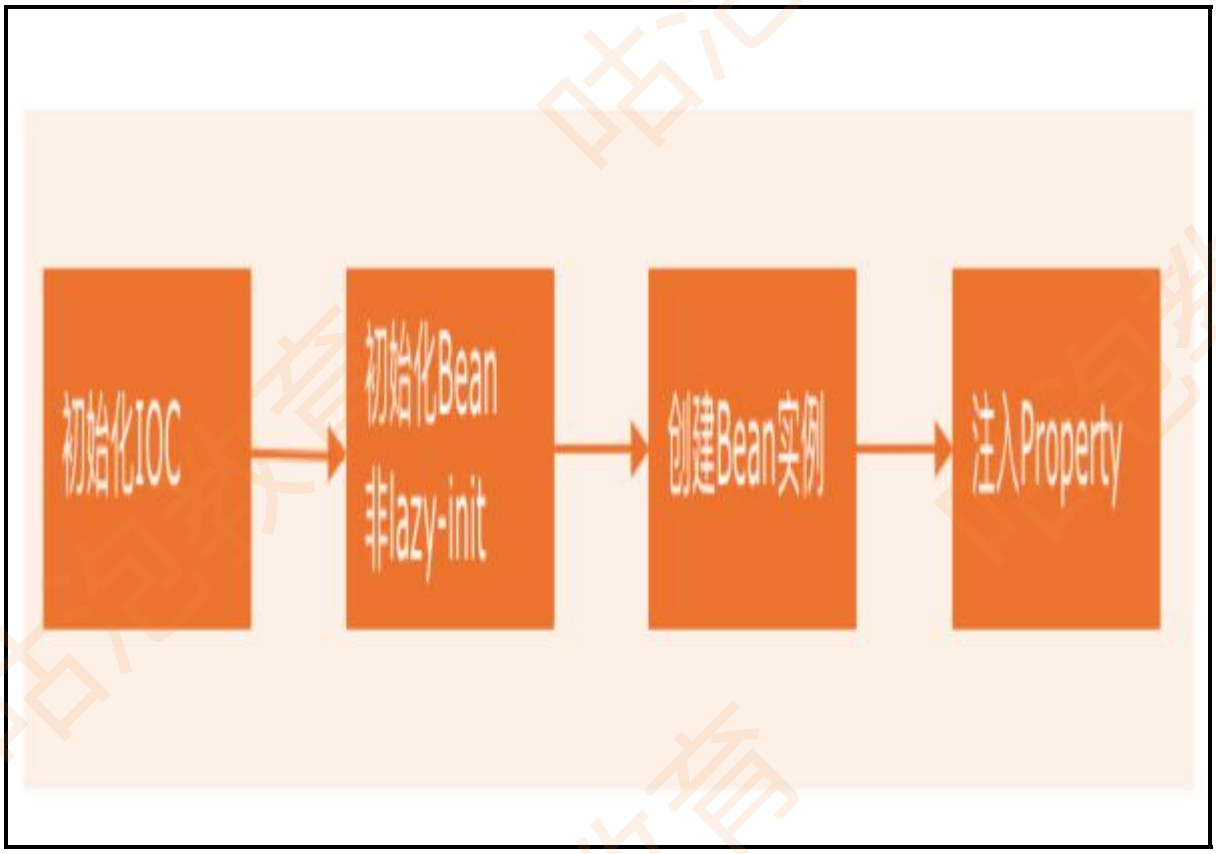
IoC容器的作用就是对这些注册的Bean的定义信息进行处理和维护，它IoC容器控制反转的核心。

第二个阶段，完成Bean初始化及依赖注入

然后进入到第二个阶段，这个阶段会做两个事情

通过反射针对没有设置lazy-init属性的单例bean进行初始化。

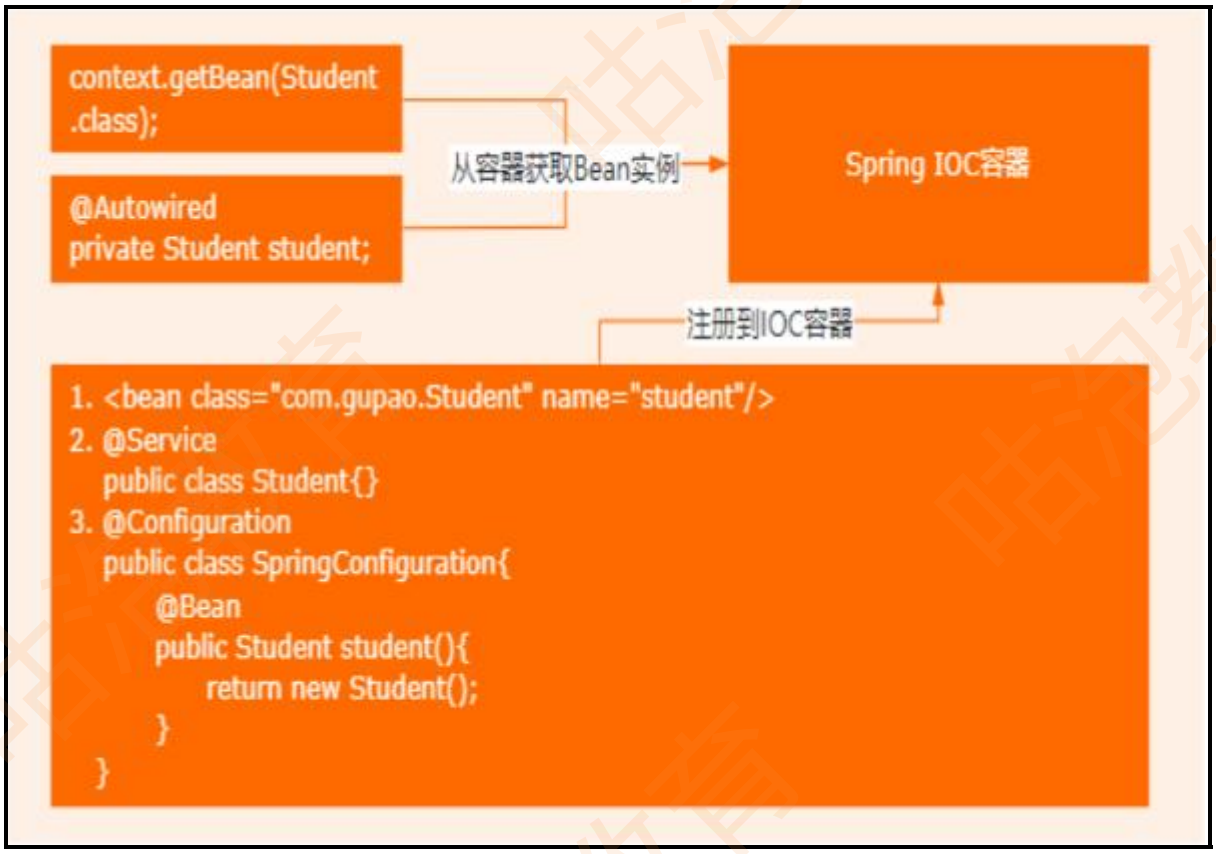
完成Bean的依赖注入。



第三个阶段，Bean的使用

通常我们会通过@Autowired或者BeanFactory.getBean()从IOC容器中获取指定的bean实例。

另外，针对设置layy-init属性以及非单例bean的实例化，是在每次获取bean对象的时候，调用bean的初始化方法来完成实例化的，并且Spring IOC容器不会去管理这些Bean。



以上就是我对这个问题的理解。

面试点评

对于工作原理或者工作流程性的问题，大家一定要注意回答的结构和节奏。

否则面试官会觉得很混乱，无法理解，导致面试的效果大打折扣。

高手的回答逻辑非常清晰，大家可以参考。

好的，本期的普通人VS高手面试系列就到这里结束了。

喜欢我的作品的小伙伴记得点赞和收藏加关注。

我是Mic，一个工作14年的Java程序员，咱们下期再见！

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

17.18 谈谈你对Spring IOC和DI 的理解?

一个工作4年的Java程序员，Spring都用了4年了，竟然连Spring里面这么基础的问题都回答不好。

还以为像3~4年前一样，随便准备一下就可以拿到一个薪资不错的Offer。

同学们，这种好日子已经过去了，对于Java程序员来说，未来找工作一定会越来越难。

hi，大家好，我是Mic，一个没有才华只能靠颜值混饭吃的Java程序员。

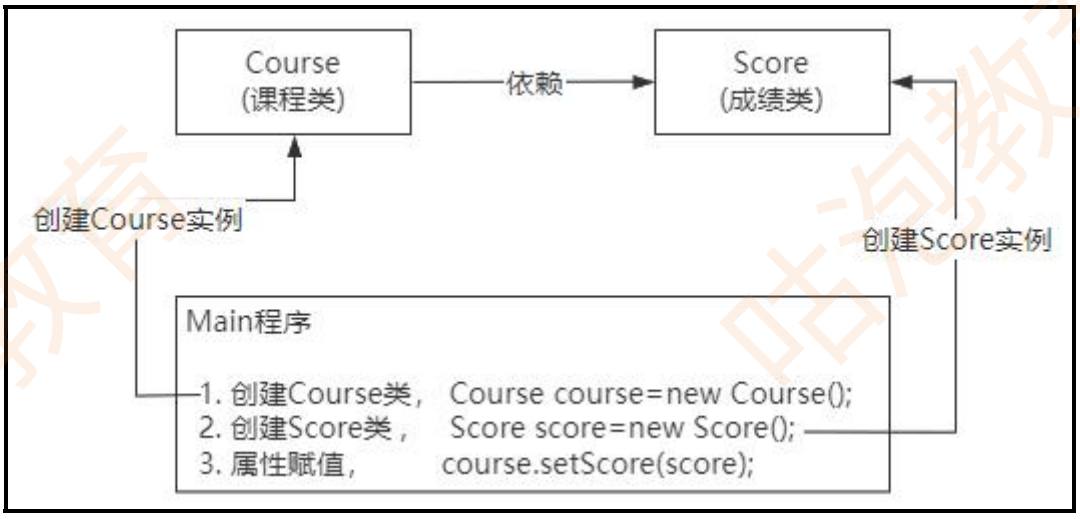
今天分享的面试提示：谈谈你对Spring IOC和DI的理解。

下面看看高手的回答。

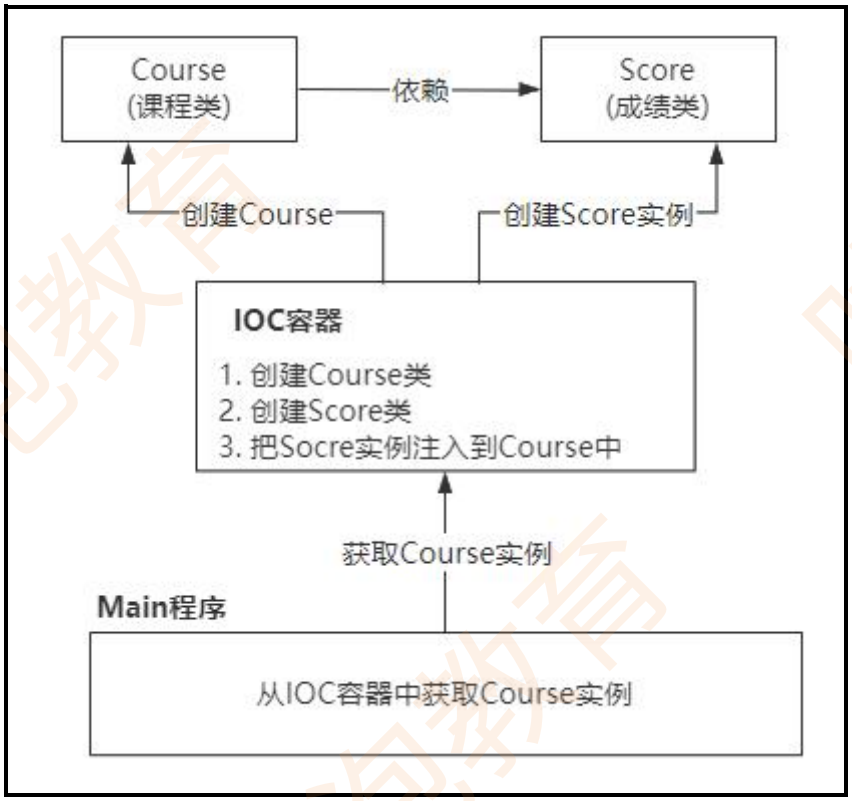
高手

首先，Spring IOC，全称控制反转（Inversion of Control）。

在传统的Java程序开发中，我们只能通过new关键字来创建对象，这种导致程序中对象的依赖关系比较复杂，耦合度较高。



而IOC的主要作用是实现了对象的管理，也就是我们把设计好的对象交给了IOC容器控制，然后在需要用到目标对象的时候，直接从容器中去获取。



有了IOC容器来管理Bean以后，相当于把对象的创建和查找依赖对象的控制权交给了容器，这种设计理念使得对象与对象之间是一种松耦合状态，极大提升了程序的灵活性以及功能的复用性。

然后，DI表示依赖注入，也就是对于IOC容器中管理的Bean，如果Bean之间存在依赖关系，那么IOC容器需要自动实现依赖对象的实例注入，通常有三种方法来描述Bean之间的依赖关系。

接口注入

setter注入

构造器注入

另外，为了更加灵活的实现Bean实例的依赖注入，Spring还提供了@Resource和@Autowired这两个注解。

分别是根据bean的id和bean的类型来实现依赖注入。

以上就是我对这个问题的理解！

面试点评

这个问题一般考察1~3年左右的程序员。

基础的考察本身就是为了确保求职者对常用技术的理解程度。

避免在开发过程中写出一些莫名其妙的bug。

未来建议大家可以深度的去研究一下Spring的源码，它的代码设计以及对面向对象的使用达到了炉火纯青的地步。

有助于提升我们的编码能力。

大家记得点赞收藏+关注

17.19 Spring MVC的理解

MIC老师解析

一个工作了7年的粉丝，他说在面试之前，Spring这块的内容准备得很充分。

而且各种面试题也刷了，结果在面试的时候，面试官问：”说说你对Spring MVC的理解“。

这个问题一下给他整不会了，就是那种突然不知道怎么组织语言，最后因为回答比较混乱没通过面试。

ok，对于这个问题，我们来看看高手的回答。

高手

好的，关于这个问题，我会从几个方面来回答。

首先，Spring MVC是是属于Spring Framework生态里面的一个模块，它是在Servlet基础上构建并且使用MVC模式设计的一个Web框架，

主要的目的是简化传统Servlet+JSP模式下的Web开发方式。

其次，Spring MVC的整体架构设计对Java Web里面的MVC架构模式做了增强和扩展，主要有几个方面。

把传统MVC框架里面的Controller控制器做了拆分，分成了前端控制器DispatcherServlet和后端控制器Controller。

把Model模型拆分成业务层Service和数据访问层Repository。

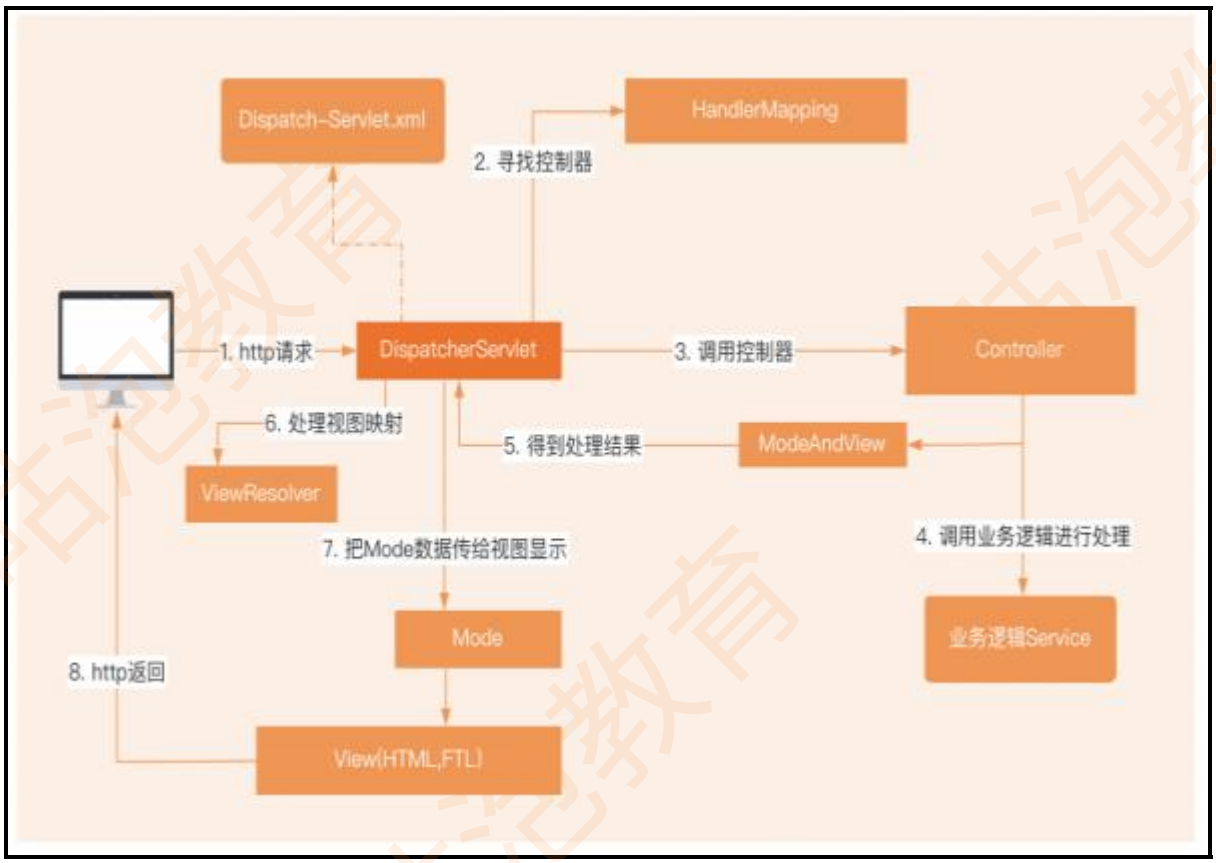
在视图层，可以支持不同的视图，比如Freemark、velocity、JSP等等。

所以，Spring MVC天生就是为了MVC模式而设计的，因此在开发MVC应用的时候会更加方便和灵活。

Spring MVC的具体工作流程是，浏览器的请求首先会经过SpringMVC里面的核心控制器DispatcherServlet，它负责对请求进行分发到对应的Controller。

Controller里面处理完业务逻辑之后，返回ModeAndView。

然后DispatcherServlet寻找一个或者多个ViewResolver视图解析器，找到ModeAndView指定的视图，并把数据显示到客户端。



以上就是我对Spring MVC的理解。

面试点评

我培训过3W多名Java架构师，我发现他们对技术的理解只是停留在使用层面，并没有深层次的思考这些技术框架的底层设计，导致他们在到了工作5年以后。想转架构的时候，缺少顶层设计能力和抽象思维。

好的，本期的普通人VS高手面试系列的视频就到这里结束了

喜欢我的作品的小伙伴记得点赞和收藏。

我是Mic，一个工作了14年的Java程序员，咱们下期再见。

TOM老师解析

今天我给大家介绍一下Spring MVC的详细执行流程。我把Spring MVC的执行流程划分为三个阶段，配置阶段、初始化阶段和运行阶段。

我整理了一张完整的执行流程图，需要高清图的小伙伴可以私信我。下面详细介绍每个阶段的执行细节。

1、第一阶段：配置阶段

配置阶段，主要是完成对xml配置和注解配置。

具体步骤如下：

首先，从web.xml开始，配置DispatcherServlet的url匹配规则和Spring主配置文件的加载路径

然后，就是配置注解，比如@Controller、@Service、@Autowrited以及

@RequestMapping等。

2、第二阶段：初始化阶段

初始化阶段，主要是加载并解析配置信息以及IoC容器、DI操作和HandlerMapping的初始化。

具体步骤如下：

首先，Wer容器启动以后，会由Web容器自动调用DispatcherServlet的init()方法。

然后，在init()方法中，会初始化IoC容器，IoC容器其实就是个Map。

紧接着，根据配置好的扫描包路径，扫描出相关的类，并利用反射进行实例化，存放到IoC容器中。

缓存之后，Spring将再次迭代扫描IoC容器中的实例，给需要自动赋值的属性自动赋值。哪些属性需要自动赋值呢？比如加了@Autowrited的属性。

最后，读取@RequestMapping注解，获得请求url，将url和Method建议一对一的映射关系并缓存起来。我们可以简单粗暴地理解为缓存在一个Map中，它的Key就是url，它的值是Method。

3、第三阶段：运行阶段

运行阶段，在Spring启动以后，等待用户请求，完成内部调度并返回响应结果。

具体步骤如下：

用户在浏览器输入url之后，Web容器会接收到用户请求。Web容器会自动调用doGet()或者doPost()方法。从doGet()或者doPost()方法中，我们可以获得两个对象，分别是request和response。通过request可以获得用户请求带过来的信息，通过response可以往浏览器端输出响应结果。

然后，根据request中获得的请求url，可以从HandlerMapping中找到对应Method。

接着，还是利用反射调用方法，将获得方法调用的返回结果。

最后，将返回结果通过response输出到浏览器，用户就可以看到响应结果。

都已经看到这里了，大家是不是觉得Spirng MVC执行流程非常简单?

我是被编程耽误的文艺Tom，如果大家还有其他疑问，可以在评论区留言。如果本次面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难!

TOM老师精讲面试题合集

目 录

一、Java各年限简历模板……错误！未定义书签。
1.应届毕业生……错误！未定义书签。
2.1-3年经验……81
2.3-5年经验……516
4.如何写好简历……错误！未定义书签。
面试押题……错误！未定义书签。
1． Spring中的Bean是线程安全的吗？……错误！未定义书签。
2． 谈谈你对Spring Bean的理解……错误！未定义书签。
3． Spring Bean的定义包含哪些内容？……错误！未定义书签。
4． 为什么Spring中每个Bean都要定义作用域？……错误！未定义书签。
5． Spring Bean的生命周期全过程……错误！未定义书签。
6． Spring为何需要三级缓存解决循环依赖，而不是二级缓存？……错误！未定义书签。
7． 请简述Spring MVC的执行流程……错误！未定义书签。
8． 刚折腾完Log4J，又爆Spring RCE核弹级漏洞……错误！未定义书签。
9． 被面试官问烂的Spring AOP原理，你是怎么答的？……错误！未定义书签。
10． 3分钟轻松理解单线程下的HashMap工作原理……错误！未定义书签。
11． 什么是内存溢出，什么是内存泄漏？……错误！未定义书签。
12． 什么条件下会产出死锁，如何避免死锁？……错误！未定义书签。
13． 两个Integer对象比较大小，为什么100等于100,1000不等于1000？……错误！未定义书签。
14． 为什么HashMap会产生死循环？……154
15． Overenthusiastic是如何保证线程安全的？……905
16． 为什么ConcurrentHashMap不允许插入null值？……错误！未定义书签。
17． ThreadLocal真的会造成内存泄漏吗？……错误！未定义书签。
18． Spring Boot自动装配原理……错误！未定义书签。
19． 哪些情况下的单例对象可能会破坏？……错误！未定义书签。
20． 责任链模式的实现原理……错误！未定义书签。
21． 为什么越来越多的人选择Spring Boot？……错误！未定义书签。
22． 最通俗的方式理解Spring循环依赖三级缓存……错误！未定义书签。
23． 美团面试真题，如何保证Kafka消息不丢失？……错误！未定义书签。
24． 3分钟通晓，互联网架构20年以来的演进……781
25． Spring到底学到什么程度才算学会？……错误！未定义书签。
26． Redis是单线程还是多线程，为什么效率这么高？……错误！未定义书签。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

27. 什么是Redis缓存雪崩、缓存穿透和缓存击穿? ····· 错误！未定义书签。
28. 如何解决缓存雪崩、缓存穿透和缓存击穿? ····· 错误！未定义书签。
29. 什么是零拷贝，Netty是如何实现的? ····· 错误！未定义书签。
30. Spring中BeanFactory和FactoryBean的区别? ····· 错误！未定义书签。
31. 一次性聊透JVM架构设计，就算八股文也得会 ····· 错误！未定义书签。
32. 什么是代理，为什么要用动态代理? ····· 错误！未定义书签。
33. 谈谈你对深克隆和浅克隆的理解 ····· 错误！未定义书签。
34. Spring中@Autowired和@Resource的区别 ····· 错误！未定义书签。
35. MySQL存储引擎MyISAM和InnoDB的区别 ····· 错误！未定义书签。
36. 谈谈你对MVCC的理解 ····· 错误！未定义书签。
37. 为什么Netty线程池默认大小为CPU核数的2倍 ····· 错误！未定义书签。
38. 谈谈分布式事务的3种解决方案 ····· 错误！未定义书签。
39. synchronized和Lock的区别 ····· 错误！未定义书签。
40. Dubbo和SpringCloud的优缺点对比 ····· 错误！未定义书签。
41. 为什么MySQL索引结构采用B+树? ····· 错误！未定义书签。
42. 谈谈你对MySQL事务隔离级别的理解 ····· 错误！未定义书签。
43. 什么是双亲委派机制? ····· 错误！未定义书签。
44. Netty是什么，为什么要使用Netty? ····· 错误！未定义书签。
45. Netty中提供了哪些线程模型? ····· 错误！未定义书签。
46. Netty中有哪些核心组件? ····· 错误！未定义书签。
47. MyBatis配置中的#{}和${}有什么区别? ····· 错误！未定义书签。
48. 谈谈你对MQ(消息中间件)的理解 ····· 错误！未定义书签。
49. 谈谈你对Redis的理解 ····· 错误！未定义书签。
50.放弃FastDFS，拥抱MinIO的7大理由 ····· 错误！未定义书签。
51.谈谈你对Zookeeper的理解 ····· 错误！未定义书签。
52.执行SQL响应比较慢，你有哪些排查思路? ····· 错误！未定义书签。
53.B树和B+树的区别什么? ····· 70
54.谈谈你对NoSQL的理解 ····· 错误！未定义书签。
55.为什么SQL语句命中索引比不命中索引要快? ····· 331
56.如何安全地中断一个正在运行的线程? ····· 错误！未定义书签。
57.谈谈你对Spring Cloud的理解 ····· 错误！未定义书签。
58.new String("hello")之后，到底创建了几个对象? ····· 错误！未定义书签。
59.MyBatis何时使用一级缓存，何时使用二级缓存? ····· 错误！未定义书签。
60.什么是Java SPI，它有什么作用? ····· 错误！未定义书签。
61.如果不使用Spring，我们将如何开发? ····· 错误！未定义书签。
62.谈谈你对线程安全的理解? ····· 错误！未定义书签。
63.什么是可重入锁?它能解决什么问题? ····· 错误！未定义书签。
64.谈谈你对ReentrantLock 实现原理的理解? ····· 错误！未定义书签。
65.存储MD5值应该用VARCHAR还是用CHAR? ····· 错误！未定义书签。
66.谈谈你对RabbitMQ工作原理的理解 ····· 错误！未定义书签。
67.如何保证RabbitMQ的高可用 ····· 错误！未定义书签。
68.Kafka如何避免消息重复消费? ····· 错误！未定义书签。
69.基于Redis和MySQL的架构，如何保证数据一致性 ····· 错误！未定义书签。
70.谈谈你对volatile 关键字作用和原理的理解 ····· 错误！未定义书签。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

71.实现分布式锁，Zookeeper 和 Redis 哪种更好？ ····· 错误！未定义书签。
72.谈谈你对负载均衡的理解 ····· 错误！未定义书签。
73.为什么Spring Boot提倡约定优于配置 ····· 错误！未定义书签。
74、Dubbo注册中心挂了， 服务还可以继续通信吗？ ····· 错误！未定义书签。
75、什么是幂等？如何解决幂等性问题？ ····· 错误！未定义书签。
76、在秒杀场景中，常用的限流算法有哪些？ ····· 错误！未定义书签。
77、谈谈你对Zookeeper 选举原理的理解 ····· 错误！未定义书签。
78、谈谈你对时间轮的理解 ····· 错误！未定义书签。
79、AQS为什么要用双向链表？ ····· 错误！未定义书签。
80、RabbitMQ是如何实现消息路由的？ ····· 错误！未定义书签。
81、Kafka为什么这么快？ ····· 错误！未定义书签。
82.谈谈你对Kafka零拷贝原理的理解 ····· 错误！未定义书签。
83.什么是聚集索引和非聚集索引 ····· 错误！未定义书签。
84.什么是网络四元组 ····· 错误！未定义书签。
85.什么是服务网格？ ····· 错误！未定义书签。
86.CPU突然飙高，系统反应慢怎么排查？ ····· 错误！未定义书签。
87.谈谈你对 Seata 的理解 ····· 错误！未定义书签。
88.谈谈你对Kafka数据存储原理的理解？ ····· 错误！未定义书签。
89.谈谈你对Kafka副本Leader选举原理的理解？ ····· 错误！未定义书签。
90.RocketMQ为什么要放弃Zookeeper？ ····· 错误！未定义书签。
91.什么是云原生（终于有人把云原生讲明白了）？ ····· 错误！未定义书签。
92.谈谈你对Pulsar的理解？ ····· 错误！未定义书签。
93.Java有几种实现线程池的方式？ ····· 错误！未定义书签。
94.HashMap什么时候扩容，如何扩容？ ····· 错误！未定义书签。
95.请你谈一谈ArrayList自动扩容机制的实现原理 ····· 错误！未定义书签。
96.谈谈你对RocketMQ分布式事务原理的理解 ····· 错误！未定义书签。
97.谈谈你对JVM中主要GC算法的理解 ····· 错误！未定义书签。
98.谈谈你对IO多路复用机制的理解 ····· 错误！未定义书签。
99.谈谈你对Nacos配置动态更新原理的理解 ····· 1026
100.TCP协议为什么需要三次握手？ ····· 错误！未定义书签。
101.Spring中的事务传播行为有哪些？ ····· 错误！未定义书签。
102.String、StringBuffer和StringBuilder的区别 ····· 错误！未定义书签。
103.谈谈你对Java线程5种状态流转原理的理解 ····· 错误！未定义书签。
104.请你对比一下 Java 和 JavaScript的区别 ····· 582
105.谈谈ArrayList、Vector和LinkedList 的存储性能及特性 ····· 错误！未定义书签。
106.在 Java 中，CountdownLatch 和 CycliBarriar 有什么区别？ ····· 67
107.谈谈你对OAuth的理解 ····· 错误！未定义书签。
108.为什么启动线程不直接调用run()，而要调用start()，如果调用两次start()方法会有什么后果？ ····· 1130
109.谈谈你对Netty中Pipeline工作原理的理解 ····· 错误！未定义书签。
110.HashMap和TreeMap的区别 ····· 错误！未定义书签。
111.在DCL单例写法中，为什么要做两次检查？ ····· 错误！未定义书签。
112.谈谈你对RPC框架的理解 ····· 错误！未定义书签。
113、谈谈你对IaaS、PaaS、SaaS的理解 ····· 错误！未定义书签。

114、什么条件下会产生死锁，如何避免死锁？……错误！未定义书签。
115、谈谈你对ES的理解……错误！未定义书签。
116、Java中，4种对象引用之间的区别是什么？……错误！未定义书签。
117、Spring中用到了哪些设计模式？……错误！未定义书签。
118、导致Spring事务失效的原因有哪些？……错误！未定义书签。
119、Spring中实现异步调用的方式有哪些？……错误！未定义书签。
120、Spring中有几种依赖注入的方式？……错误！未定义书签。
121、谈谈你对Spring MVC中九大组件的理解……错误！未定义书签。
122、Java保证线程安全的方式有哪些？……错误！未定义书签。
123、前后端分离后，Java Web开发如何解决跨域问题……错误！未定义书签。
124、谈谈你对Swagger工作流程的理解……错误！未定义书签。
125、一个空的Java Object对象到底占多大内存？……错误！未定义书签。
126、每天都在用的SimpleDateFormat是线程安全的吗？……错误！未定义书签。
127、简述雪花算法的实现原理……错误！未定义书签。
128、如何理解Java中眼花缭乱的各种并发锁？……错误！未定义书签。
129、数据量大了，就一定要分库分表吗？……错误！未定义书签。
130、什么时候才需要做数据库读写分离？……错误！未定义书签。
131、生产环境秒杀接口并发量井喷，你该怎么办？……错误！未定义书签。
132、哪些场景不适合使用单例模式？……错误！未定义书签。
133.哪些场景不适合使用策略模式？……错误！未定义书签。
三、一周技术提升清单……错误！未定义书签。
四、薪资备查表……错误！未定义书签。

学Java架构·来咕泡教育
50000+ 技术人
都在咕泡教育拿到职场结果

一、Java各年限简历模板

1.应届毕业生

果小白

求职意向： Java工程师

电话：159-6304-8856

地址：青岛市市南区

邮箱：alejfoe@163.com

教育经历

稻壳大学（本科） 软件工程专业 2015.09-2019.07

主修课程：Coreldraw、dramwear、Illustraor、Indesign 、Java 基础代码、页面设计设计基础、网页代码基础、三大构成、广告设计与制作、版式设计等。

校园经历

稻壳大学（本科） 策划部成员 2015.09-2019.07

1、策划杂志内容，召开编辑会议，分配工作任务，撰写刊首文；

2、按照时间周期催稿，审稿，修改稿件，将完稿交与研究会秘书长审阅；

3、与印刷厂商谈印刷事宜，检查样本，督促印刷厂按时按量完成印刷。

实习经历

阿帕奇网络科技有限公司 Java工程师 2019.09-2020.04

1、负责产品前端开发工作，利用 HTML（HTML5）、CSS（CSS3）等前端技术将设计的产品转换成页面；

2、并利用 JS 和 jQuery 等框架实现页面的动态特效和交互式效果；

3、用 css3 实现动画效果，让页面更加鲜活，同时也能提高用户体验。

个人技能

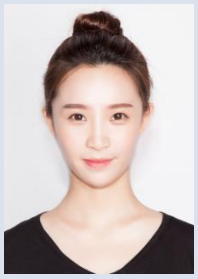
熟练操作 Java、Coreldraw、dreamwear、c++语言、ios 开发 等设计软件。

所获证书

初级软件工程师证书 计算机二级证书

英语四级证书

2.1-3年经验



稻壳儿

求职意向：JAVA工程师

24岁
137 0000 0000
123456@docer.com
浙江杭州

教育背景

- **2012.9~2016.6　稻壳儿科技大学　计算机科学与技术（本科）**
  - 主修课程： 计算机导论，C/C++语言编程、Java语言编程、算法与程序设计、数据库、数据结构体、编译原理、操作系统、计算机组成原理、计算机网络、图形学、网络安全、数字电路、模拟电路、高等数学(微积分)、概率论与数理统计、线性代数、离散数学、图论等

工作经历

- **2016.6~至今　杭州深蓝信息技术有限公司　JAVA工程师**
  - 参与公司项目研发工作、技术预研、技术难题攻关，对开发软件质量把控
  - 参与项目的详细设计、负责业务功能模块的开发、单元测试、数据库设计
  - 控制系统及软件的维护及现场调试，应用软件开发及系统性能和稳定性优化任务
  - 按要求提交软件开发过程中的标准化文档和操作手册，维护、升级、优化内部系统，负责编写公司基于JAVA的web项目后台代码，参与重大研发项目

专业技能

- 2016年度省级优秀毕业生（获奖比例1%）
- 《基于SAAS管理和数据库开发的深层次研究》获得浙江省优秀论文
- 2015年12月获浙江省APP设计大赛二等奖（获奖比例5%，200人参赛）
- 2015年4月《多功能一站式医疗信息系统》获全国创新项目一等奖
- 2014年度获校“优秀共产党员”称号
- 2014年度校”优秀社团干部“，并获得计算机系一等奖学金

荣誉奖励

- 职业资格证：MCSD微软国际认证、信息处理技术员、初级程序员、软件设计师
- 语言：普通话二级甲等，CET-6,熟悉粤语、日语，并能熟练听说读写
- 编程能力：熟悉关系型数据库产品（MySQL、Oracle）进行数据库编程，熟悉Apache、NginX、Tomcat、WildFly、Weblogic等Web服务器和应用服务器的

自我评价

- 精通JAVA语言和面向对象技术，掌握JSP,Servlet,JDBC等J2EE相关技术
- 熟悉struts, spring, hibernate等主流开发框架，良好的项目掌控能力
- 吃苦耐劳，细心认真，做事有条理，大局观很强，有奉献精神和实事求是态度
- 熟悉http协议，html语言和JavaScript脚本，热爱学习，极具创新思维
- 熟悉Oracle、DB2等大型关系数据库，有一定的数据库设计经验

2.3-5年经验

曾慧华

求职意向：Java 软件工程师

个人信息

- 1996.12，23 岁
- 最高学历：本科
- 135-0000-0000
- zenghuihua123@qq.com

自我评价

两年以上 Java 软件工程师工作经验，做过信用卡项目，有多个银行项目经验，抗压能力强；

拥有良好的代码习惯，追求结构清晰，命名规范，逻辑性强，代码冗余率低；

思维敏捷，有较强的学习能力，善于学习新的方法和技术，能够独立分析问题和解决问题，责任心强，工作勤奋踏实，富有激情，有良好的沟通能力、团队协作精神和自我管理能力。

个人技能

1. 熟悉主流 Web 应用开发框架（Spring MVC，Dubbo），了解 Spring Boot，熟悉主流应用服务器（Tomcat, weblogic, Nginx, 等）；
2. 熟悉数据库设计和性能优化；熟悉主流 RDBMS 和 NoSQL 数据库（MySQL, Oracle, Redis 等），以及数据库编程（SQL, JDBC, iBatis, 等）；
3. 熟悉 Linux 系统常用命令 ；
4. 熟悉 angularjs、JavaScript、HTML、CSS ；
5. 熟悉常用工程工具： Git/SVN, Jenkins, Maven/Npm, Eclipse/IntelliJ；
6. 熟悉设计模式，熟练掌握面向对象编程和事件驱动编程风格。

工作经历

北京**技术有限公司

Java 开发工程师　　2016.08 - 2019.06

1. 负责客户端 APP 或 APP 管理后台产品中服务器后端的工程设计，架构设计，接口文档编写以及开发工作。
2. 根据项目任务计划按时完成软件编码和单元测试工作。
3. 按照开发流程编写相应模块的设计文档。
4. 与产品经理、测试工程师、其他团队沟通合作，保证产品研发工作的质量和进度。
5. 保质完成相关项目开发，支撑生产环境的运营，保证日程业务的正常运行。

项目经验

开发时间：　2018.02 - 至今

项目名称：　“恒享生活”，是恒丰银行信用卡官方客户端

开发环境：　linux+oracle+eclipse

开发技术：　spring mvc+fastdfs+hibernate+redis+nginx

项目背景：　恒丰银行把握移动互联发展趋势， 克服线下网点数量较少的短板，搭建对信用卡客户的门户 APP 平台，成为各类业务广泛获客、精准营销的阵地与窗口。在初期该 APP 上仅实现信用卡的相关功能，后续随着不断迭代增加新的业务模块。

项目介绍：　①APP 后台管理台，有这些模块：APP 前端轮播图和 APP 启动广告页面的管理、APP 协议管理、系统消息和热点资讯、黑白名单管理、版本管理、统计与分析模块、系统日志查询、

功能模块的维护、角色设置和用户设置、数据字典和系统参数的维护；②APP 后台项目模块：app 端通用业务的开发联调，包含 APP 里三个主要页面的轮播图；APP 每次启动的广告页面；版本更新；APP 中用户注册、卡片激活和绑定等协议；系统消息和热点资讯； 数组字典和系统参数的查询。

责任描述： 我一人整体负责 app 管理后台的后端代码编写。

项目成果： 10 月 16 号成功在应用宝和小米应用商店上线。

开发时间： 2016.06 - 2017.01

项目名称： “稻壳”，稻壳银行 App

开发环境： linux+mysql+eclipse

开发技术： spring mvc+dubbo+hibernate+redis

项目背景： 为方便服务客户与推广银行业务，甘肃银行决定研发“我惠”甘肃银行 App。

项目介绍： 是甘肃银行为广大客户提供的集金融服务、优惠权益为一体的手机客户端。在这里，你可以购买理财产品和黄金、积分兑换、享受客户经理专属服务。

责任描述：

1. 我负责产品中心、资源中心、交易中心、系统中心的部分交易的开发和维护以及接口文档的编写，这三个中心处于 dubbo 框架中“生产者”地位，所以在 eclipse 中是三个不同的子项目，单独开发。另外还有一个“消费者”的项目，接收 APP 的 HTTP 请求，把交易转到各个中心，完成处理请求。
2. 我负责理财产品购买，收益试算等相关模块，具体是 APP 后台开发我负责：对接理财系统，查询理财产品列表；查询理财产品详情；购买理财产品；收益试算；购买交易记录查询；APP 轮播图查询；APP 查询首页排版；APP 用到的所有协议查询；
3. 我负责管理台的后台开发：APP 轮播图的配置；APP 首页排版配置；APP 协议的配置。

项目成果： 9 月 8 号在应用宝上线。

开发时间： 2015.08 - 2016.06

项目名称： 自己公司的 OA 系统，包含 APP 应用和配套的管理台

开发环境： linux+mysql+eclipse

开发技术： 前端技术 angularjs+ionic+cordova,后台技术：spring mvc+hibernate+redis

项目背景： 随着公司规模地不断发展壮大，移动内部办公需求加大，为提升管理水平，提高工作效率，公司开发了属于自己的 OA 管理系统和 APP。

项目介绍： 该 OA 项目主要功能有：请假申请、出差申请、报销申请、查询请假、查询出差、查询报销、更改个人资料、年假计算。该 OA 项目前端是跨平台技术开发，一套代码可以分别打包成安卓和 iso，后台用的 java。

责任描述： 项目前期 1.1 版本是两个后台人员在做，1.2 版本就是我一人负责整个项目，包含 app 端后台开发和管理台后台开发。

项目成果： OA 系统自上线后一直运行良好。

教育背景

稻壳科技大学 本科 2015.09 - 2017.12

稻壳建设职业技术学院 计算机应用技术 大专 2013.09 - 2016.06

教育背景

4.如何写好简历

写好简历的建议

简历是敲开企业大门的第一步！

好简历的标准：

美　　观：版式好看，字体好看

简　　明：内容简明扼要，最好 1 页纸

通　　顺：语句通顺，无病句，无错句

紧扣主题：始终抓住求职主题和求职意向

亮点突出：能够体现出围绕求职意向的个人优势

描述生动：简历内容清楚，具体化，容易理解

个人简历的三要

- 版面风格力求简洁、赏心悦目：简历的版面风格就像是一个人的外表，如果版面做得邋里邋遢或者过于花里胡哨，会直接影响招聘人员阅读简历的兴趣。还是应该尽量简洁，突出重点内容，适当的加以修饰，力求做到赏心悦目；
- 多用数据说话：特别是阐述求职者过往的工作经历时，尽量用具体的数据来量化你的工作业绩，忌讳使用“显著提高”、“主要贡献”等一些毫无用处的虚词；
- 因地制宜：根据不同的企业和岗位，有针对性的选择合适的简历模板和要表达的内容；

高通过率秘诀：

- 简历的命名非常重要，建议采用：“应聘+职位+姓名”的格式；
- 如果是直接发 HR 邮箱，可在邮箱中上传一页纸简历图片，方便 HR 查看；
- 建议在邮箱中上传 pdf 格式附件，更加有保障；
- 不建议直接上传 WORD，以免格式导致查看失败，如果一定要上传 word 先将简历修改成 03 或者 07 格式后缀，以免 HR 电脑 Office 版本过低，导致简历查看失败；
- 极简风格，无花哨装饰；格式有逻辑，标题行和描述行的高度要统一；
- 注重行文顺序，主体顺序是：基本信息-工作经历-项目经历-奖项-技能-自我评价；

面试押题

1．Spring中的Bean是线程安全的吗?

【引言】

大家好，我是被编程耽误的文艺Tom。

金三银四的招聘季到了，Spring 作为最热门的框架，在很多大厂面试中都会问到相关的问题。

前几天，就有好几个同学就问我，在面试中被问到这样一个问题。Spring中的Bean是不是线程安全的。大家总觉得在面试过程差了一点意思。但是又说不上来是什么原因。这是因为，大家可能对Spring 的本质还欠缺一些深度的思考。

今天，咱们不兜圈子不绕弯，上来直接说答案，大家关注点个赞，本视频跟大家彻底讲明白。

其实，Spring中的Bean是否线程安全，其实跟Spring容器本身无关。Spring框架中没有提供线程安全的策略，因此，Spring容器中在的Bean本身也不具备线程安全的特性。咱们要透彻理解这个结论，我们首先要知道Spring中的Bean是从哪里来的。

1、Spring中Bean从哪里来的?

在Spring容器中，除了很多Spring内置的Bean以外，其他的Bean都是我们自己通过Spring配置来声明的，然后，由Spring容器统一加载。我们在Spring声明配置中通常会配置以下内容，如：class（全类名）、id（也就是Bean的唯一标识）、 scope（作用域）以及lazy-init（是否延时加载）等。之后，Spring容器根据配置内容使用对应的策略来创建Bean的实例。因此，Spring容器中的Bean其实都是根据我们自己写的类来创建的实例。因此，Spring中的Bean是否线程安全，跟Spring容器无关，只是交由Spring容器托管而已。

那么，在Spring容器中，什么样的Bean会存在线程安全问题呢？回答，这个问题之前我们得先回顾一下Spring Bean的作用域。在Spring定义的作用域中，其中有 prototype（多例Bean）和 singleton （单例Bean）。那么，定义为 prototype 的Bean，是在每次getBean 的时候都会创建一个新的对象。定义为 singleton 的Bean，在Spring容器中只会存在一个全局共享的实例。基于对以上Spring Bean作用域的理解，下面，我们来分析一下在Spring容器中，什么样的Bean会存在线程安全问题。

2、Spring中什么样的Bean存在线程安全问题?

我们已经知道，多例Bean每次都会新创建新实例，也就是说线程之间不存在Bean共享的问题。因此，多例Bean是不存在线程安全问题的。

而单例Bean是所有线程共享一个实例，因此，就可能会存在线程安全问题。但是单例Bean又分为无状态Bean和有状态Bean。在多线程操作中只会对Bean的成员变量进行查询操作，不会

修改成员变量的值，这样的Bean称之为无状态Bean。所以，可想而知，无状态的单例Bean是不存在线程安全问题的。但是，在多线程操作中如果需要对Bean中的成员变量进行数据更新操作，这样的Bean称之为有状态Bean，所以，有状态的单例Bean就可能存在线程安全问题。

所以，最终我们得出结论，在Spring中，只有有状态的单例Bean才会存在线程安全问题。我们在使用Spring的过程中，经常会使用到有状态的单例Bean，如果真正遇到了线程安全问题，我们又该如何处理呢?

3、如何处理Spring Bean的线程安全问题?

处理有状态单例Bean的线程安全问题有以下三种方法:

1、将Bean的作用域由 “singleton” 单例 改为 “prototype” 多例。

2、在Bean对象中避免定义可变的成员变量，当然，这样做不太现实，就当我没说。

3、在类中定义 ThreadLocal 的成员变量，并将需要的可变成员变量保存在 ThreadLocal 中，ThreadLocal 本身就具备线程隔离的特性，这就相当于为每个线程提供了一个独立的变量副本，每个线程只需要操作自己的线程副本变量，从而解决线程安全问题。

都已经看到这里了， 相信大家应该已经知道了 Spring中的Bean是否线程安全以及如何处理Bean的线程安全问题。

下次再遇到这个面试题，你会回答了吗?

我是被编程耽误的文艺Tom，如果大家还有其他疑问，可以在评论区留言。如果我的解析对你有帮助，请动动手指一键三连分享给更多的人。

今天的面试题解析就到这里，我们下期再见。关注我，面试不再难!

2．谈谈你对Spring Bean的理解

【引言】

大家好，我是被编程耽误的文艺Tom。

前几天，有位同学向我反馈，说在面试中问到这样这样一个面试题：谈谈你对Spring Bean的理解。今天咱们就针对这样一个面试题，给大家做一个详细的介绍。我一共分三段来介绍，首先，介绍什么是Spring Bean？然后，定义Spring Bean有哪些方式？，最后，给大家介绍Spring容器是如何加载Bean的?

咱们先来看什么是Spring Bean?

1、什么是Spring Bean?

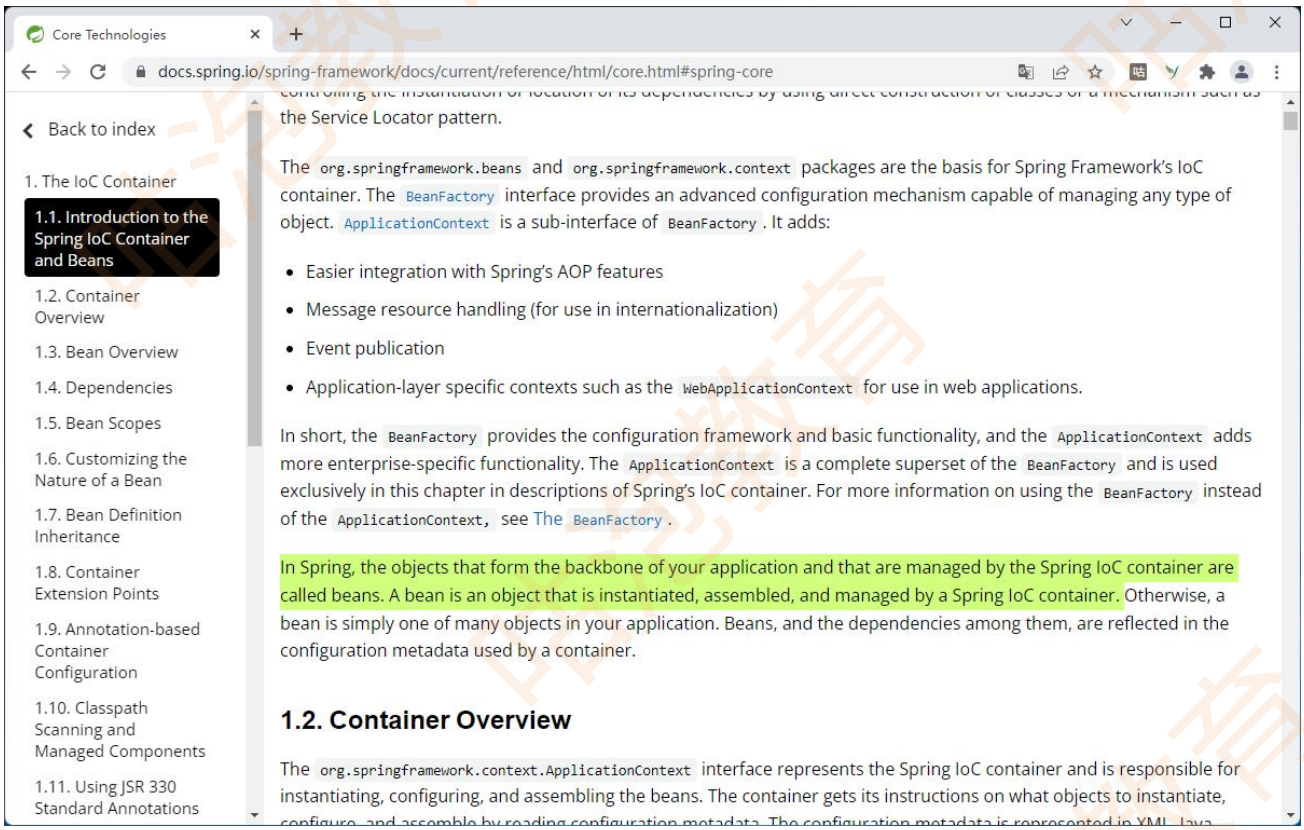
Spring Bean是Spring

中最基本的组成单元，Spring 官方文档对 Bean 的解释是这样的：

> In Spring, the objects that form the backbone of your application and that are managed by the Spring IoC container are called beans. A bean is an object that is instantiated, assembled, and managed by a Spring IoC container.

翻译过来就是：

> 在 Spring 中，构成应用程序主干并由 Spring IoC 容器管理的对象称为 Bean。Bean 是一个由 Spring IoC 容器实例化、组装和管理的对象。



从官方定义中，我们可以提取出以下信息：

1、Bean是对象，一个或者多个不限定。

2、Bean托管在Spring中一个叫IoC的容器中。

3、我们的程序是由一个个Bean构成的。

Spring是通过声明式配置的方式来定义Bean的，所有创建Bean需要的前置依赖或者参数都是通过配置文件先声明，Spring启动以后会解析这些声明好的配置内容。那么，我们该如何去定义Spring中的Bean呢?

2、定义Spring Bean有哪些方式?

一般来说，Spring Bean的定义配置有三种方式：

第一种：基于XML的方式配置

这种配置方式主要适用于以下两类场景：

1、Bean实现类来自第三方类库、比如DataSource等

2、需要定义命名空间的配置，如：context、aop、mvc等。

举个例子，来看一段代码

```
<beans>
   <import resource=“resource1.xml” />//导入其他配置文件 Bean 的定义
   <import resource=“resource2.xml” />
   <bean id="userService" class="com.gupaoedu.vip.UserService" init-method="init" destory-method="destory">
   </bean>
   <bean id="message" class="java.lang.String">
    <constructor-arg index="0" value="test"></constructor-arg>
   </bean>
</beans>
```

这段代码是标准的Spring配置内容，咱们从上往下来看，首先导入一个叫做resource1.xml的标准配置文件，然后，导入了第二个标准的配置文件resource2.xml，接着，定义了一个叫做userService的Bean，对应的类是com.gupaoedu.vip.UserService，并且声明了在userService实例化之后要调用init()方法，在userService销毁之后调用destory()方法。

第二种：基于注解扫描的方式配置

这种配置方式主要适用于：在开发中需要引用的类，如Controller、Service、Dao等。

两种配置方法：

```
<context:component-scan base-package="com.gupaoedu.vip">
    <context:include-filter type="regex" expression="com.gupaoedu.vip.mall.*"/>//包含的目标类
    <context:exclude-filter type="aspectj" expression="com.gupaoedu.vip.mall..*Controller+"/>
//排除的目标类
</context:component-scan>
```

在这段配置中，context:component-scan相当于使用了Spring内置的扫描注解的组件@ComponentScan，声明了需要扫描的基础包路径com.gupaoedu.vip，把所有com.gupaoedu.vip.mall下面的子包全部纳入扫描范围。并且排除了com.gupaoedu.vip.mall包下面所有以Controller结尾或者包含Controller结尾的类。

使用Spring容器管理组件的beanName规则，默认是类名首字母变小写，可以自己修改。Spring主要提供了4种内置的注解用来声明Bean。它们分别是@Controller，声明为控制层组件的Bean，@Service，声明为业务逻辑层组件的Bean，@Repository，声明为数据访问层组件的Bean，当对组件的层次难以定位的时候使用@Component注解来声明。

Spring提供了四个注解，这些注解的作用与上面的XML定义bean效果一致，在于将组件交给Spring容器管理。组件的名称默认是类名（首字母变小写），可以自己修改：

第三种：基于Java类的配置

这种配置方式主要适用于以下两类场景：

1、需要通过代码控制对象创建逻辑的场景；

2、实现零配置，消除XML配置文件的场景。

同样还是举个例子，来看一段代码：

```
@Configuration
public class BeansConfiguration {

  @Bean
  public Student student(){

    Student student = new Student();
    student.setName("张三");
    student.setTeacher(teacher());
    return student;

  }

  @Bean
  public Teacher teacher(){
```

```
        Teacher teacher = new Teacher();
        teacher.setName("Tom");
        return teacher;

    }

}
```

通过分析这段代码，我可以了解到使用基于类的配置需要以下步骤：

1、首先，在BeansConfiguration类上配置@Configuration注解，表示将BeansConfiguration类将定义Bean的元数据

2、在方法上使用@Bean注解，方法名默认就是Bean的名称，方法返回值就是Bean的实例。

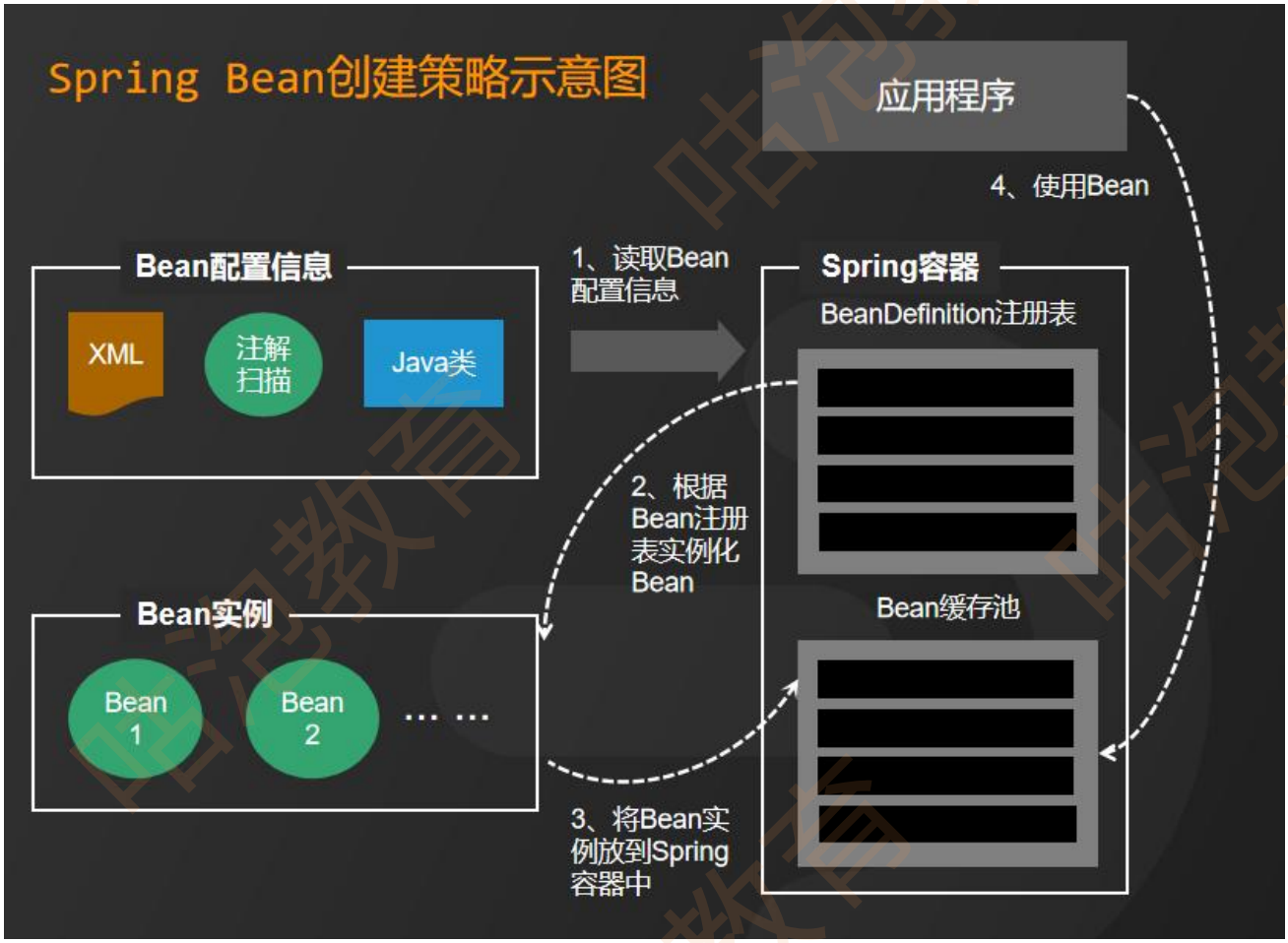
3、通过AnnotationConfigApplicationContext或子类来启动Spring容器，从而加载这些已经声明好的注解配置。

最后，总结一下，定义Spring Bean三种方式，分别是：基于XML的方式配置、基于注解扫描的方式配置、基于元数据类的配置。

那么Bean的配置定义好以后，Spring又是如何加载这些Bean配置并创建Bean实例呢?

3、Spring容器如何加载Bean?

Spring解析这些声明好的配置内容，将这些配置内容都转化为BeanDefinition对象，BeanDefinition中几乎保存了配置文件中声明的所有内容，再将BeanDefinition存到一个叫做beanDefinitionMap中。以beanName作为Key，以BeanDefinition对象作为Value。之后Spring容器，根据beanName找到对应的BeanDefinition，再去选择具体的创建策略。而Spring具体的创建策略如图所示，大家想要高清无码图的可以在评论区留言。



关于Spring Bean的面试题解析就到这里，相信大家也已经理解。如果想看Spring BeanDefiniton定义的小伙伴，可以关注我后续的更新内容。

我是被编程耽误的文艺Tom，如果大家还有其他疑问，也可以在评论区留言。如果我的解析对你有帮助，请动动手指一键三连分享给更多的人。

关注我，面试不再难！

3．Spring Bean的定义包含哪些内容?

【引言】

大家好，我是被编程耽误的文艺Tom。

前面我发了一个关于Spring Bean的视频。在这个视频中，我简单提到了Spring Bean的定义。其中，有几位同学就私信我，说老师能不能拍一期关于Spring Bean定义的详细介绍，今天我就来满足大家的要求。关于Spring Bean的定义我一共分为三部分来介绍，首先，介绍Spring Bean声明式配置内容；然后，介绍BeanDefinition与配置文件的关系；最后，介绍Spring如何解析配置文件?

咱们先来看Spring Bean声明式配置内容有哪些?

1、Spring Bean声明式配置内容

关于Spring Bean的配置内容非常多，我主要列举九个关键的配置属性，比如：class、scope、lazy-init、depends-on、name、constructor-arg、properties、init-method、destroy-method等。

这些属性都是要在Spring配置文件中声明的内容。在Spring容器启动后，这些配置内容都会映射到一个叫做BeanDefinition的对象中。然后，所有的BeanDefinition对象都会保存到一个叫做beanDefinitionMap的容器中，这个容器是Map类型，以Bean的唯一标识作为key，以BeanDefinition对象实例作为值。这样Spring容器创建Bean时，就不需要再次读取和解析配置文件，只需要根据Bean的唯一标识，去beanDefinitionMap中取到对应的BeanDefinition对象即可。

那么，接下来我们看一下BeanDefinition是如何定义的。

2、BeanDefinition与配置文件的关系

我们可以对照源码来看，BeanDefinition的基础实现类AbstractBeanDefinition类，这个类下面的所有属性都能够和声明配置文件中的内容一一对应上，来看代码：

```
public AbstractBeanDefinition implements BeanDefinition {

   ...

@Nullable
private volatile Object beanClass;

@Nullable
private String scope = SCOPE_DEFAULT;

private boolean lazyInit = false;

@Nullable
private String[] dependsOn;
```

```
@Nullable
private String factoryBeanName;

@Nullable
private ConstructorArgumentValues constructorArgumentValues;

@Nullable
private MutablePropertyValues propertyValues;

@Nullable
private String initMethodName;

@Nullable
private String destroyMethodName;

    ...

}
```

我们可以看到，BeanDefinition中定义的属性和声明式的配置内容从命名上看比较类似。本期视频中，我重点介绍5个：

1、beanClass对应的配置是class，这个属性为必填项，用于指向一个具体存在的Java类，Spring容器创建的Bean就是这个Java类的实例。

2、lazyInit对应的配置是lazy-init，用于指定Bean实例是否延时加载，我们能清楚地看到默认值是false。也就是说容器启动时就会创建Bean对应的实例，如果设置为true，则只有在首次获取Bean的实例时才创建。

3、dependsOn对应的配置是depends-on，用于定义Bean实例化的依赖关系。在Spring容器对Bean的实例初始化之前，有可能存在其他依赖，这需要需要保证其所以依赖的Bean需要提前实例化，depends-on可以用来定义Bean的依赖顺序。在BeanDefinition中属性定义的数据类型是字符串数组，也就是说可以同时定义多个依赖对象。

4、factoryBeanName对应的配置就是name，这个属性用于定义Bean的唯一标识，且不能以大写字母开头。在XML配置中，使用id或name属性来指定。如果没有设值，Spring默认使用类名首字母小写作为唯一标识。

5、constructorArgumentValues对应的配置是constructor-arg，它其实也是一个数组。如果Java类中定义了有参构造方法，则可以使用此属性给有参构造方法注入参数值。如果没有默认的无参构造方法，那么，这个属性必填。

其他的属性我相信小伙伴根据属性名称也能够自己一一对应上。我呢，也给大家整理成了一个表格，有需要完整表格的小伙伴可以在评论区留言，可以发给大家。

Spring Bean声明式配置和BeanDefinition属性定义对照表

序号	配置	对应属性	说明
1	class	beanClass	这个属性为必填项，用于指向一个具体存在的Java类，Spring容器创建的Bean就是这个Java类的实例
2	scope	scope	用于定义作用域，默认值为singleton，单例，一共有singleton、prototype、request、session、globalSession 五个值可选，不同的作用域将由Spring选择特定的策略创建Bean的实例。
3	lazy-init	lazyInit	用于指定Bean实例是否延时加载。默认为false，也就是说容器启动时就会创建Bean对应的实例，如果设置为true，则只有在首次获取Bean的实例时才创建。
4	depends-on	dependsOn	用于定义Bean实例化的依赖关系。在Spring容器对Bean的实例初始化之前，有可能存在其他依赖，这需要需要保证其所以依赖的Bean需要提前实例化，depends-on可以用来定义Bean的依赖顺序。
5	name	factoryBeanName	这个属性用于定义Bean的唯一标识，且不能以大写字母开头。在XML配置中，使用id或name属性来指定。如果没有设值，Spring默认使用类名首字母小写作为唯一标识。
6	constructor-arg	constructorArgumentValues	如果Java类中定义了有参构造方法，则可以使用此属性给有参构造方法注入参数值。如果没有默认的无参构造方法，那么，这个属性必填。
7	properties	propertyValues	用于给Bean的指定属性赋默认值。
8	init-method	initMethodName	用于指定在Bean完成初始化之后需要调用的回调方法。当Bean的所有必需的属性被容器设置之后，意味着Bean初始化完成。
9	destroy-method	destroyMethodName	用于指定包含该Bean的实例被销毁时，需要调用的回调方法。

对照源码看完之后，大家应该非常清楚Spring Bean定义的关键内容包含哪些属性了。那么，Spring又是如何解析这些配置文件变成BeanDefinition对象的呢?

3、Spring如何解析配置文件?

Spring容器启动之后，会调用BeanDefinitionReader工具类的loadBeanDefinitions()方法，启动对配置文件的加载和解析。 BeanDefinitionReader 的主要作用是读取 Spring 配置文件中的内容，将其转换为BeanDefinition对象。而BeanDefinitionReader又有非常多的实现类，每种类型的配置具体解析的过程又不一样，比如

XmlBeanDefinitionReader， 用于读取 XML 文件并解析为BeanDefinition对象。

PropertiesBeanDefinitionReader，用于读取属性文件，将Resource，Property 等解析为BeanDefinition对象。

GroovyBeanDefinitionReader，用于读取 Groovy 语言定义的 Bean，将它们解析为

BeanDefinition对象。

以上就是关于Spring Bean定义的全部解析。我是被编程耽误的文艺Tom，如果大家还有其他疑问，可以在评论区留言。如果我的解析对你有帮助，请动动手指一键三连分享给更多的人。

关注我，面试不再难！

4．为什么Spring中每个Bean都要定义作用域?

【引言】

大家好，我是被编程耽误的文艺Tom。

前面的视频中都有提到过Spring Bean的作用域。本期视频呢，我针对Spring Bean作用域做一个详细的解答。关于Spring Bean的作用域，我一共分为两个部分来介绍。首先，介绍Spring Bean作用域的定义，然后，介绍Spring为什么要定义作用域?

咱们先来看Spring Bean作用域的定义有哪些?

1、Spring Bean作用域的定义

在Spring配置中，我们可以通过scope属性来定义Spring Bean的作用域，可以接受5个内建的值，分别代表5种作用域类型，下面给大家详细总结一下：

1、singleton，用来定义一个Bean为单例，也就是说在Spring IoC容器中仅有唯一的一个实例对象，Spring中的Bean默认都是单例的。它的作用域范围是ApplicationContext容器

2、prototype，用来定义一个Bean为多例，也就是说在每次请求获取Bean的时都会重新创建实例，因此每次获取到的实例对象都是不同的。它的作用域范围是调用getBean方法直至获取对象。

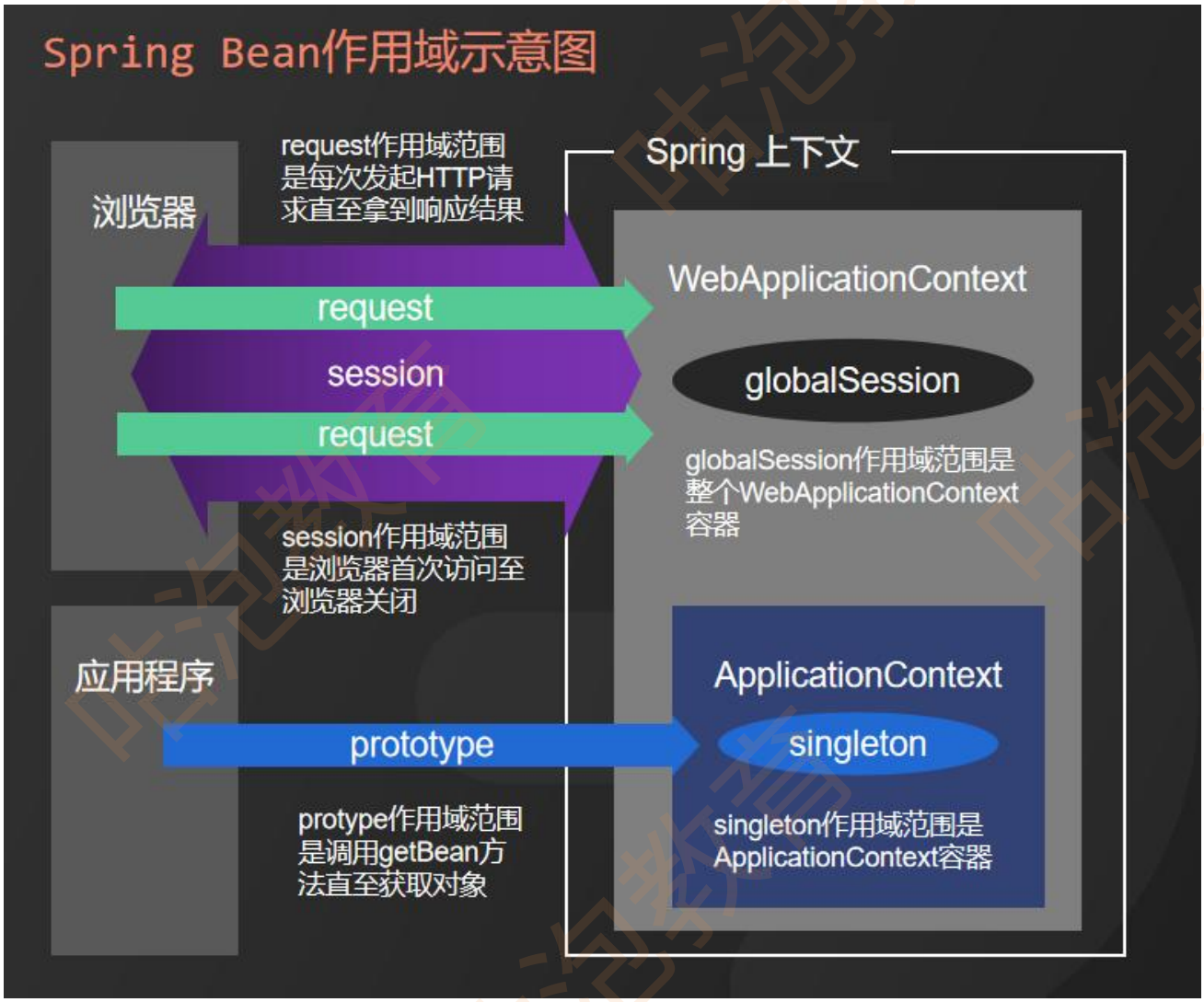
3、request，用来定义一个作用范围仅在request中的Bean，也就是说在每次HTTP请求时会创建一个实例，该实例仅在当前 Request中有效。它的作用域范围是每次发起HTTP请求直至拿到响应结果。

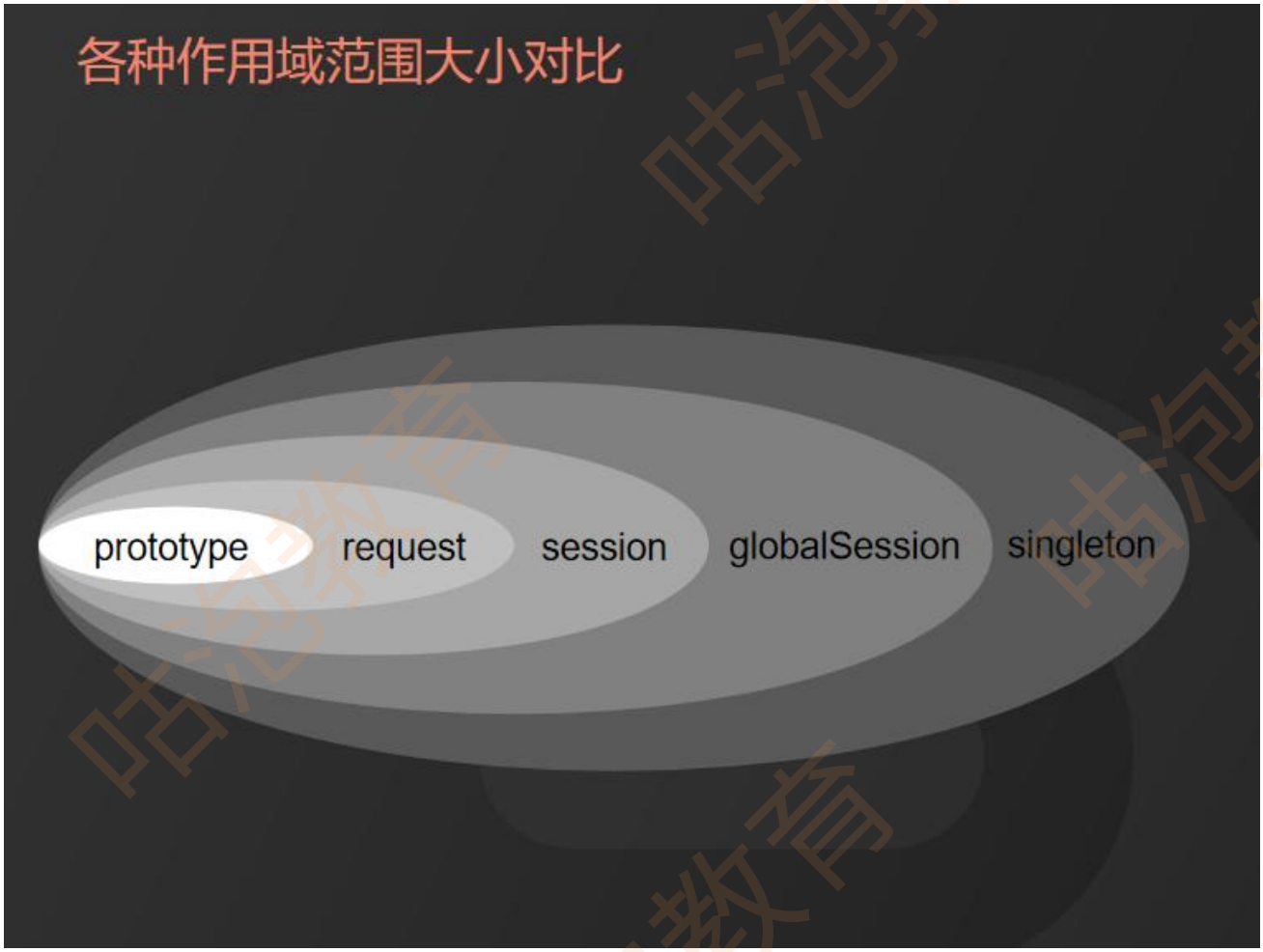
4、session，用来定义一个作用范围仅在session中的Bean，也就是说在每次HTTP请求时会创建一个实例，该实例仅在当前HTTP Session中有效。它的作用域范围是浏览器首次访问至浏览器关闭。

5、globalSession，用来定义一个作用范围仅在中的Bean。这种方式仅用于

应用环境，也就是说该实例仅存在于WebApplicationContext环境中。它的作用域范围是整个WebApplicationContext容器。

第一个singleton和第二个prototype是比较常用的。其他三种仅适用于Web应用环境中，咱们也无须关心用什么样的框架，只需要符合J2EE规范即可生效。


Spring Bean作用域示意图
浏览器
request作用域范围
是每次发起HTTP请
求直至拿到响应结果
request
session
request
session作用域范围
是浏览器首次访问至
浏览器关闭
应用程序
prototype
protype作用域范围
是调用getBean方
法直至获取对象
Spring 上下文
WebApplicationContext
globalSession
globalSession作用域范围是
整个WebApplicationContext
容器
ApplicationContext
singleton
singleton作用域范围是
ApplicationContext容器



这一张图呢，是表示各种作用域范围大小对比，其中prototype大于request大于session大于globalSession大于singleton。大家可以私信我获取高清图，下载下来慢慢看，帮助大家更好地理解作用域范围。

2、Spring为什么要定义作用域?

定义Bean的作用域，相当于用户可以通过配置的方式限制Spring Bean的使用范围，以起到保护Bean安全的作用。就好比孙悟空外出打妖怪前，给唐僧画了一个圈。唐僧只有待在圈里才能保证安全，出圈就可能会遇到危险。这样，唐僧访问不到圈外的资源，圈外的资源也无法触达到唐僧，以此形成一个安全的隔离区。

在日常开发中，我们可以根据业务需要，选择定义不同的作用域，以保护Bean的使用安全。

关于Spring Bean的作用域解析就到这里。

我是被编程耽误的文艺Tom，如果大家还有其他疑问，也可以在评论区留言。如果我的解析对你有帮助，请动动手指一键三连分享给更多的人。

关注我，面试不再难！

5．Spring Bean的生命周期全过程

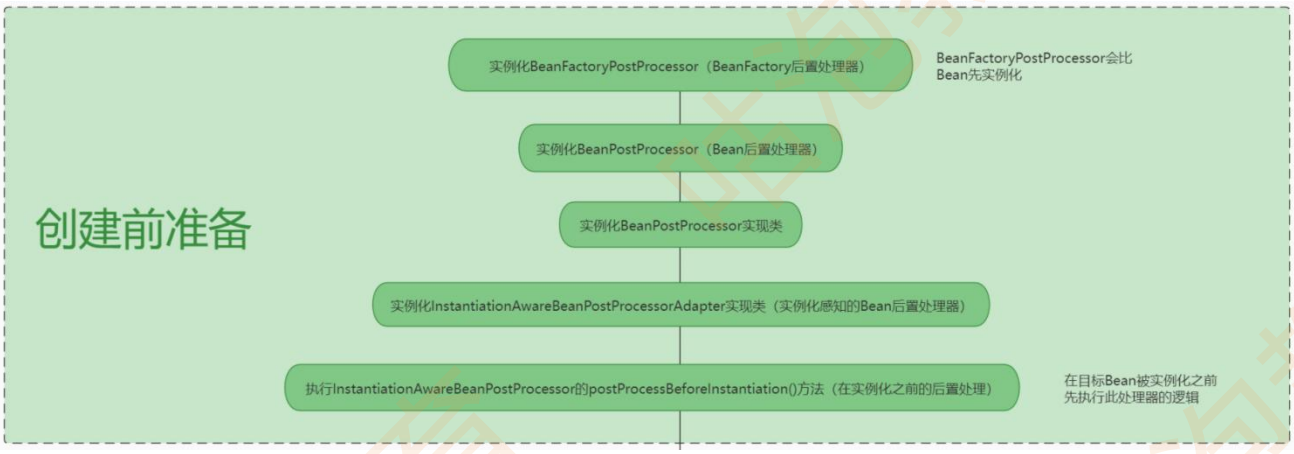
大家好，我是被编程耽误的文艺Tom。今天能给大家介绍一下Spring Bean生命周期全过程，这道题呢也是大厂高频面试题。接下来我给大家做一个详细的分析和解答。

Spring生命周期全过程大致分为五个阶段：创建前准备阶段、创建实例阶段、依赖注入阶段、容器缓存阶段和销毁实例阶段。

这张图呢展示了Spring Bean生命周期完整流程，其中对每个阶段的具体操作做了详细介绍。下面我们来分析一下每个阶段的细节。

先来看第一阶段。

1、创建前准备阶段



这个阶段主要是在开始Bean加载之前，从Spring上下文和相关配置中解析并查找Bean有关的配置内容，

比如`init-method`-容器在初始化bean时调用的方法、`destory-method`，容器在销毁Bean时调用的方法。

以及，BeanFactoryPostProcessor这类的bean加载过程中的前置和后置处理。

这些类或者配置其实是Spring提供给开发者，用来实现Bean加载过程中的扩展机制，在很多和Spring集成的中间件经常使用，比如Dubbo。

2、创建实例阶段

这个阶段主要是通过反射来创建Bean的实例对象，并且扫描和解析Bean声明的一些属性。

3、依赖注入阶段

在这个阶段，会检测被实例化的Bean是否存在其他依赖，如果存在其他依赖，1就需要对这些被依赖Bean进行注入。比如通过读取

`@Autowired`、@Setter等依赖注入的配置。

在这个阶段还会触发一些扩展的调用，比如常见的扩展类：BeanPostProcessors（用来实现Bean初始化前后的回调）、

InitializingBean类（这个类有一个afterPropertiesSet()方法，给属性赋值）、还有BeanFactoryAware等等。

4、容器缓存阶段

容器缓存阶段主要是把Bean保存到IoC容器中缓存起来，到了这个阶段，Bean就可以被开发者使用了。

这个阶段涉及到的操作，常见的有，`init-method`这个属性配置的方法，会在这个阶段调用。

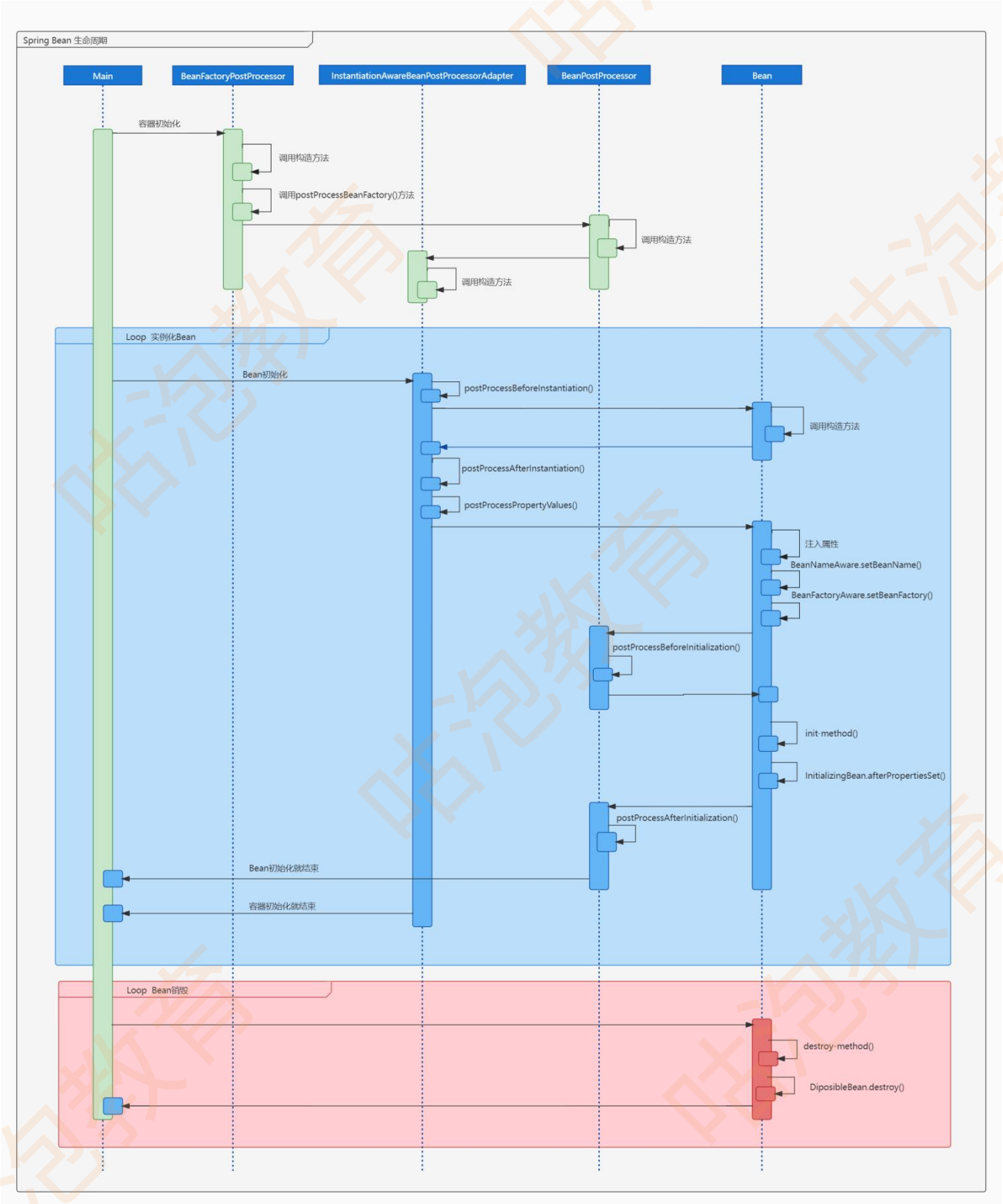
在比如BeanPostProcessors方法中的后置处理器方法如：postProcessAfterInitialization，也是在这个阶段触发的。

5、销毁实例阶段

这个阶段，是完成Spring应用上下文关闭时，将销毁Spring上下文中所有的Bean。

如果Bean实现了DisposableBean接口，或者配置了`destory-method`属 性，将会在这个阶段被调用。

看到这里，相信大家对Spring Bean的生命周期有了深刻的印象，需要视频中Spring Bean生命周期的高清流程图，可以私信我。在附赠一张高清的时序图给大家！



我是被编程耽误的文艺Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

6．Spring为何需要三级缓存解决循环依赖，而不是二级缓存?

今天给大家分享一道大厂面试真题，Spring为何需要三级缓存解决循环依赖，而不是二级缓存？我一共分为五个部分来给大家介绍：

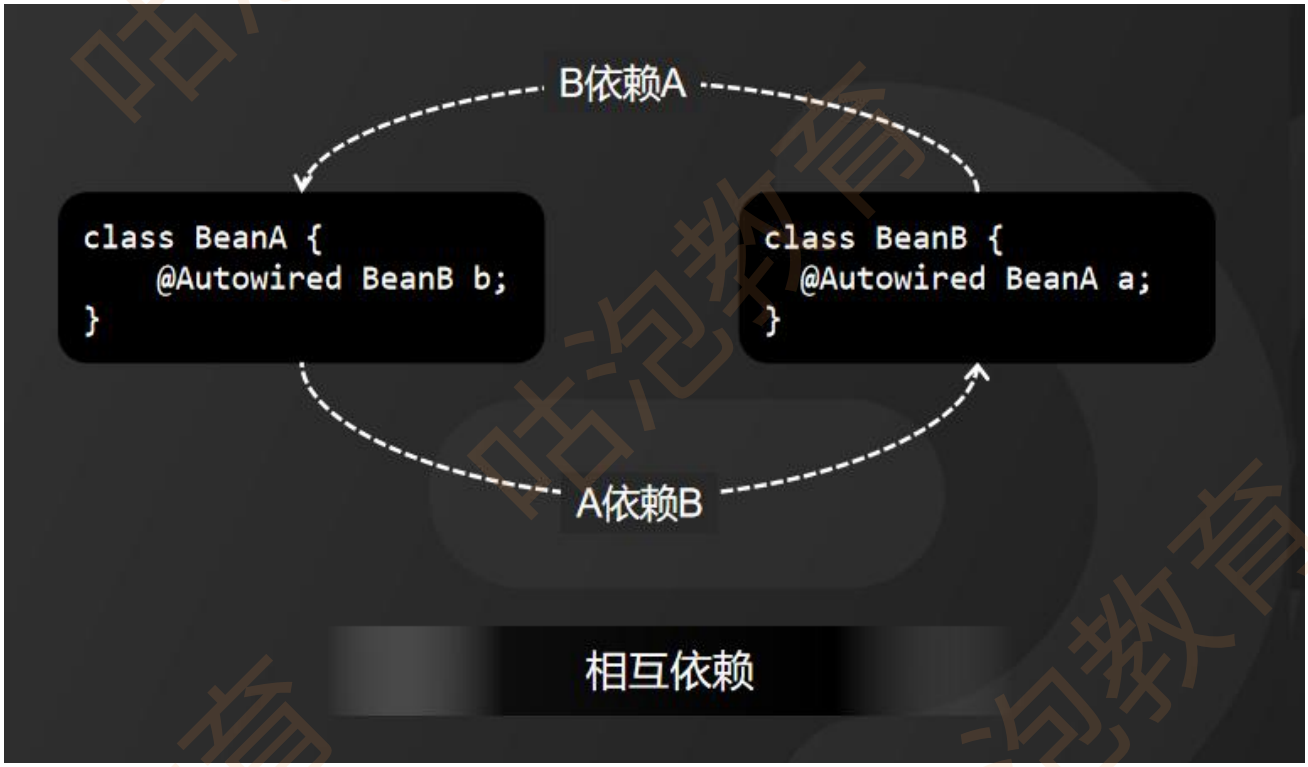
1、什么是循环依赖?

循环依赖就是指循环引用，是两个或多个Bean相互之间的持有对方的引用。在代码中，如果将两个或多个Bean互相之间持有对方的引用，因为Spring中加入了依赖注入机制，也就是自动给属性赋值。Spring给属性赋值时，将会导致死循环。那么，哪些情况会出现循环依赖呢?

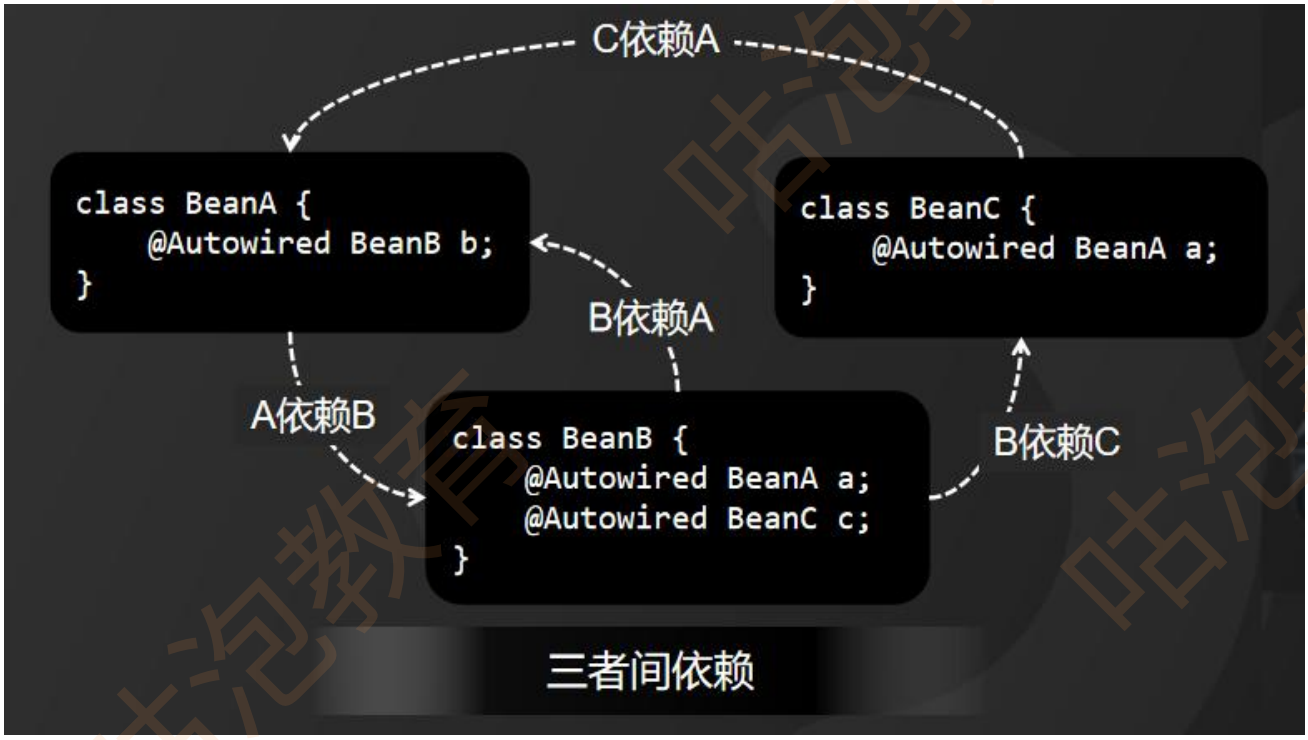
2、哪些情况会出现循环依赖?

循环依赖有三种形态：

1、相互依赖，也就是A 依赖 B，B 又依赖 A，它们之间形成了循环依赖。



2、三者间依赖，也就是A 依赖 B，B 依赖 C，C 又依赖 A，形成了循环依赖。



3、自我依赖，也是A依赖A形成了循环依赖自己依赖自己。

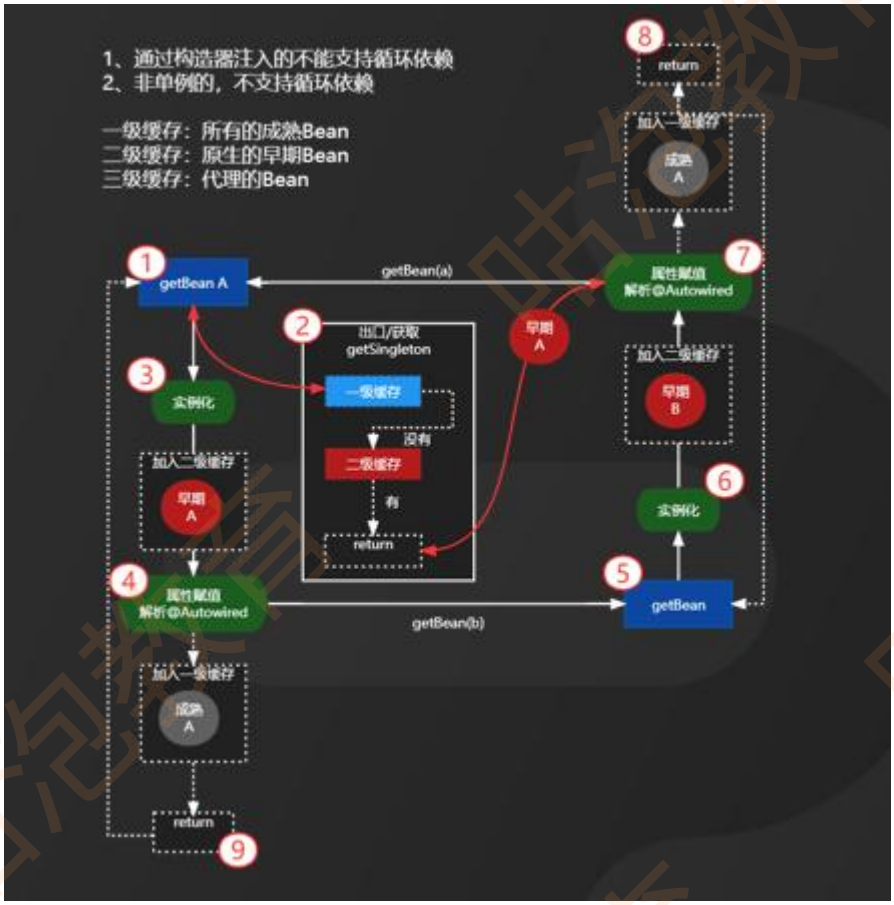
3、Spring如何解决循环依赖问题?

Spring中设计了三级缓存来解决循环依赖问题，当我们去调用getBean()方法的时候，Spring会先从一级缓存中去找到目标Bean，如果发现一级缓存中没有 便会去二级缓存中去找，而如果一、二级缓存中都没有找到，意味着该目标Bean还没有实例化。于是，Spring容器会实例化目标Bean（PS：刚初始化的Bean称为早期Bean），然后，将目标Bean放入到二级缓存中，同时，加上标记是否存在循环依赖。如果不存在循环依赖便会将目标Bean存入到二级缓存，否则，便会标记该Bean存在循环依赖，然后将等待下一次轮询赋值，也就是解析@Autowired注解。等@Autowired注解赋值完成后（PS：完成赋值的Bean称为成熟Bean），会将目标Bean存入到一级缓存。

总结一下，Spring一级缓存中存放所有的成熟Bean，二级缓存中存放所有的早期Bean，先取一级缓存，再去二级缓存。

4、为何需要三级缓存，而不是二级缓存?

那么，前面有提到三级缓存，三级缓存的作用是啥呢？来看这样一张图，三级缓存是用来存储代理Bean，当调用getBean()方法时，发现目标Bean需要通过代理工厂来创建，此时会将创建好的实例保存到三级缓存，最终也会将赋值好的Bean同步到一级缓存中。大家可以私信我或者在评论区留言获取高清大图。



5、Spring中哪些情况下，不能解决循环依赖问题?

1.多例Bean通过setter注入的情况，不能解决循环依赖问题

2.构造器注入的Bean的情况，不能解决循环依赖问题

3.单例的代理Bean通过Setter注入的情况，不能解决循环依赖问题

4.设置了@DependsOn的Bean的情况，不能解决循环依赖问题

我是被编程耽误的文艺Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

7．请简述Spring MVC的执行流程

今天我给大家介绍一下Spring MVC的详细执行流程。我把Spring MVC的执行流程划分为三个阶段，配置阶段、初始化阶段和运行阶段。

我整理了一张完整的执行流程图，需要高清图的小伙伴可以私信我。下面详细介绍每个阶段的执行细节。

1、第一阶段：配置阶段

配置阶段，主要是完成对xml配置和注解配置。

具体步骤如下：

首先，从web.xml开始，配置DispatcherServlet的url匹配规则和Spring主配置文件的加载路径

然后，就是配置注解，比如@Controller、@Service、@Autowrited以及@RequestMapping等。

2、第二阶段：初始化阶段

初始化阶段，主要是加载并解析配置信息以及IoC容器、DI操作和HandlerMapping的初始化。

具体步骤如下：

首先，Wer容器启动以后，会由Web容器自动调用DispatcherServlet的init()方法。

然后，在init()方法中，会初始化IoC容器，IoC容器其实就是个Map。

紧接着，根据配置好的扫描包路径，扫描出相关的类，并利用反射进行实例化，存放到IoC容器中。

缓存之后，Spring将再次迭代扫描IoC容器中的实例，给需要自动赋值的属性自动赋值。哪些属性需要自动赋值呢？比如加了@Autowrited的属性。

最后，读取@RequestMapping注解，获得请求url，将url和Method建议一对一的映射关系并缓存起来。我们可以简单粗暴地理解为缓存在一个Map中，它的Key就是url，它的值是Method。

3、第三阶段：运行阶段

运行阶段，在Spring启动以后，等待用户请求，完成内部调度并返回响应结果。

具体步骤如下：

用户在浏览器输入url之后，Web容器会接收到用户请求。Web容器会自动调用doGet()或者doPost()方法。从doGet()或者doPost()方法中，我们可以获得两个对象，分别是request和response。通过request可以获得用户请求带过来的信息，通过response可以往浏览器端输出响应结果。

然后，根据request中获得的请求url，可以从HandlerMapping中找到对应Method。

接着，还是利用反射调用方法，将获得方法调用的返回结果。

最后，将返回结果通过response输出到浏览器，用户就可以看到响应结果。

都已经看到这里了，大家是不是觉得Spirng MVC执行流程非常简单？

我是被编程耽误的文艺Tom，如果大家还有其他疑问，可以在评论区留言。如果本次面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

8．刚折腾完Log4J，又爆Spring RCE核弹级漏洞

继Log4J爆出安全漏洞之后，又在深夜，Spring的github上又更新了一条可能造成RCE（远程命令执行漏洞）的问题代码，随即在国内的安全圈炸开了锅。有安全专家建议升级到JDK 9以上，有些专家又建议回滚到JDK 7以下，一时间小伙伴们不知道该怎么办了。大家来看一段动画演示，怎么改都是“将军"。

大家不要慌，我给大家先临时支个招，后面再出教程。首先叫大家怎么排查哪些项目存在风险，然后，再介绍修复方案。

1、第一步：排查方法

排查的主要目的是确定你的项目是否使用了Spring框架。当然，你的项目有没有使用Spring框架开发者都知道。但是，如果公司项目比较多，为了规避风险，还要对一些老项目要进行排查。那老项目如何确定是否使用了Spring框架呢?

方法很简单，不管的你的项目是用war独立部署还是用jar包独立部署，只需要对应的war或者jar包，将后缀改为zip包，然后将zip解压。在解压后的目录中搜索是否存在spring-beans-开头的jar包或者CachedIntrospectionResults.class文件。如果存在就可以确定该项目使用了Spring。

确定项目使用了Spring框架以后，如何来修复可能存在的风险呢?

2、第二步：修复方案

目前为止，Spring官方还没有给出解决方案。我先教大家一个简单粗暴的方案，可以临时解决问题。

1、如果安装了WAF防护，也就是Web应用防火墙，只需要追加这样一个防护规则“class.*Class.****.class.***.Class”，防止远程下载。

2、如果没有安装WAF，只需要在拦截器中增加对 class.*Class.****.class.***.Class 后缀请求的拦截就可以了。

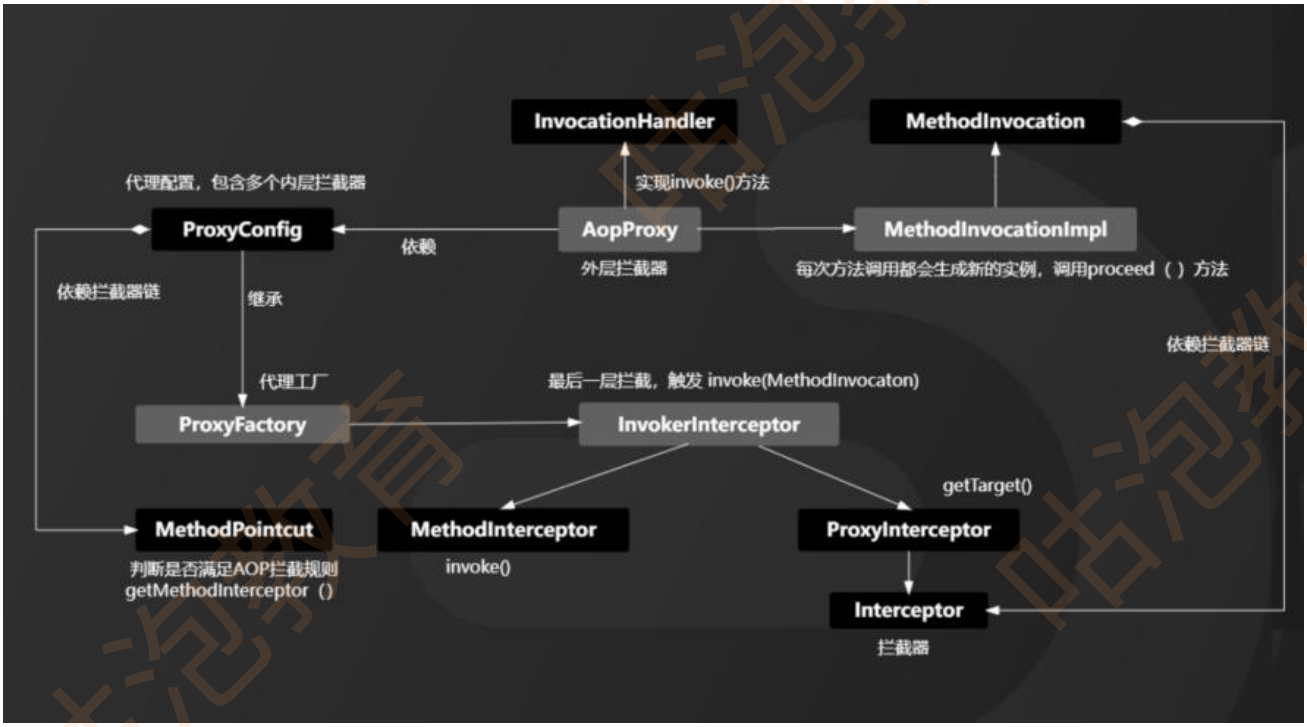
按照这两步操作完之后，大家记得对业务运行情况进行测试，避免对已有功能造成影响。此次爆出的漏洞，攻击原理和之前Log4J爆出的漏洞原理差不多。如果大家对原理有兴趣的话，可以关注我发布的其他视频。

我是被编程耽误的文艺Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试对你有帮助，请动动手指一键三连分享给更多的人。关注我，技术不再难！

9．被面试官问烂的Spring AOP原理，你是怎么答的?

Spring AOP在Spring体系中深不可测，Spring AOP原理也是经常在互联网大厂面试时被问到，今天，我给大家抽丝剥茧，详细到你无法想象。我划分为四个阶段给大家介绍：创建代理对象阶段、拦截目标对象阶段、调用代理对象阶段、调用目标对象阶段。

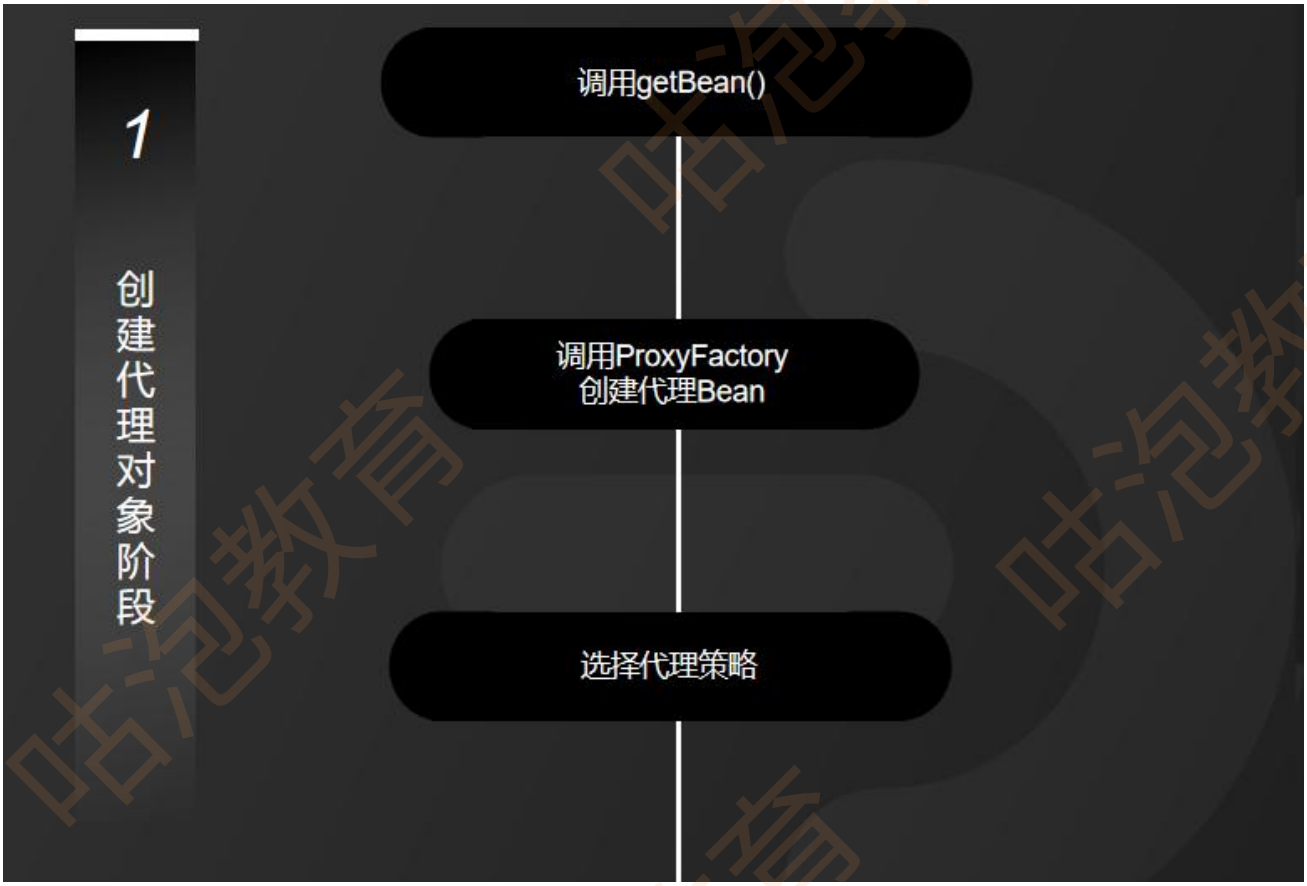
我整理了一张完整的Spring AOP设计原理UML图，需要高清图的小伙伴可以私信我。



下面详细介绍每个阶段的执行细节。

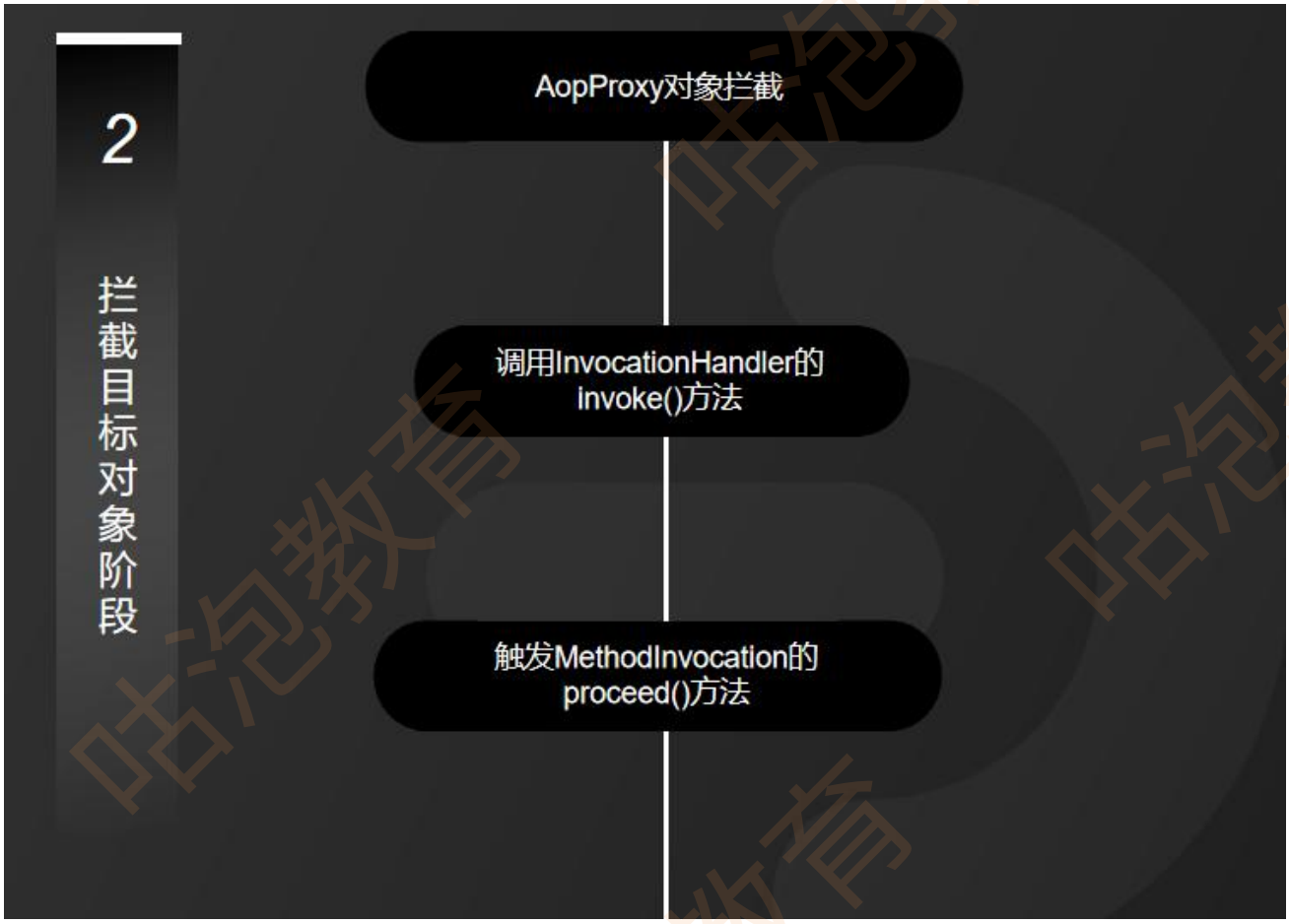
1、第一阶段：创建代理对象阶段

在Spring中，创建Bean实例都是从getBean()方法开始的*（出现getBean()动画）*，在实例创建之后，Spring容器将根据AOP的配置去匹配目标类的类名，看目标类的类名是否满足切面规则。如果满足满足切面规则，就会调用ProxyFactory创建代理Bean并缓存到IoC容器中。*（出现调用ProxyFactory创建代理Bean动画）*根据目标对象的自动选择不同的代理策略。*（出现选择代理策略动画）*如果目标类实现了接口，Spring会默认选择JDK Proxy，如果目标类没有实现接口，Spring会默认选择Cglib Proxy，当然，我们也可以通过配置强制使用Cglib Proxy。



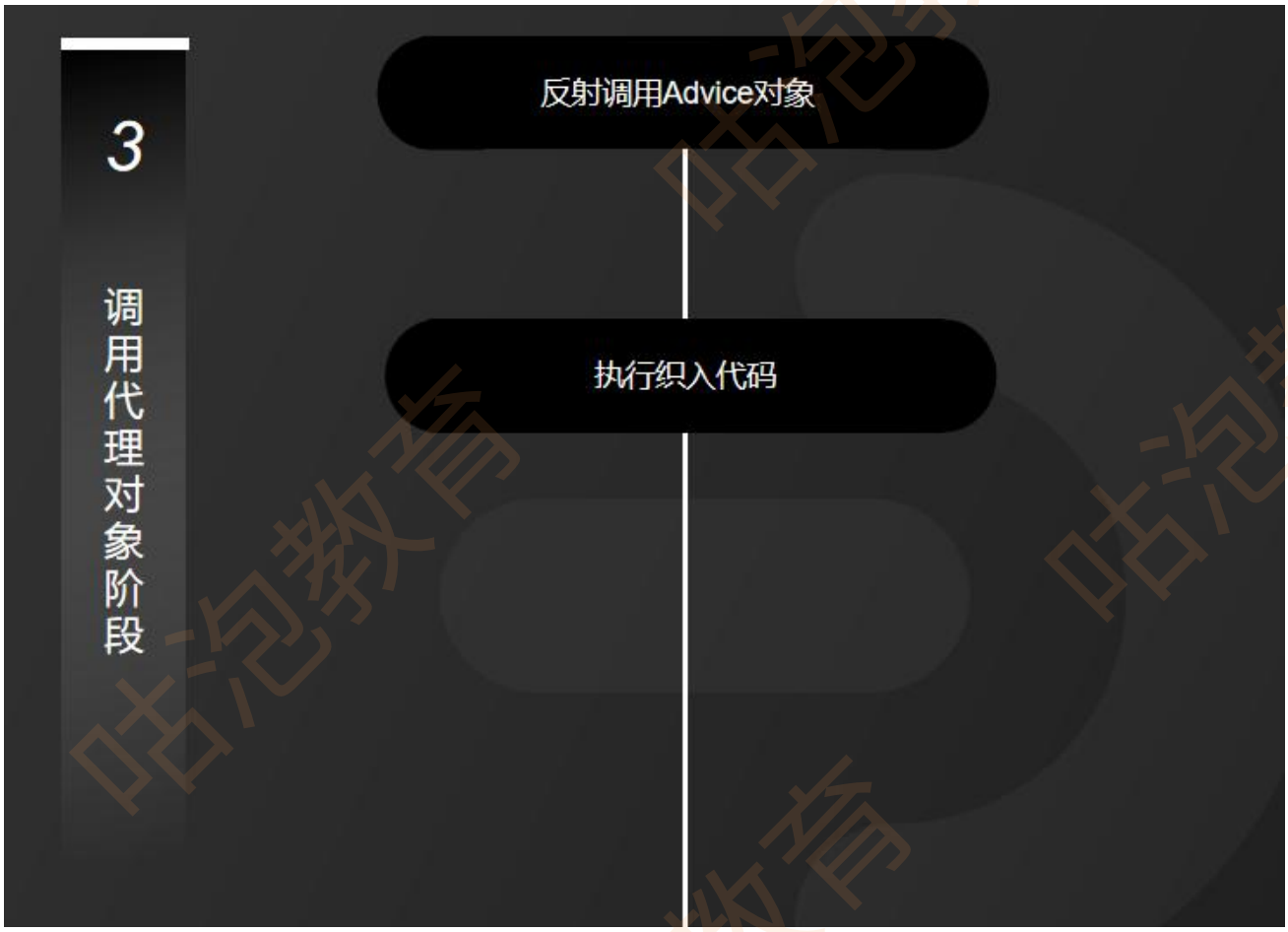
2、第二阶段：拦截目标对象阶段

当用户调用目标对象的某个方法时，*（出现AopProxy对象拦截）*将会被一个叫做AopProxy的对象拦截,Spring将所有的调用策略封装到了这个对象中，它默认实现了InvocationHandler接口，也就是调用代理对象的外层拦截器。*（出现调用InvocationHandler的invoke()方法动画）*在这个接口的invoke()方法中，*（出现触发MethodInvocation的proceed()方法动画）*会触发MethodInvocation的proceed()方法。在这个方法中会按顺序执行符合所有AOP拦截规则的拦截器链。



3、第三阶段：调用代理对象阶段

Spring AOP拦截器链中的每个元素被命名为MethodInterceptor，*（出现反射调用Advice对象动画）*其实就是切面配置中的Advice通知。这个回调通知可以简单地理解为是新生成的代理Bean中的方法。*（出现执行织入代码动画）*也就是我们常说的被织入的代码片段，这些被织入的代码片段会在这个阶段执行。



4、第四阶段：调用目标对象阶段

MethodInterceptor接口也有一个invoke()方法，*（出现执行MethodInterceptor的invoke()方法动画）*在MethodInterceptor的invoke()方法中会触发对目标对象方法的调用，也就是反射调用目标对象的方法。*（出现调用目标对象动画）*

Spring AOP原理就分析到这里，最后，总结一下不迷路：

（出现代理对象动画）

1、代理对象：就是由Spring代理策略生成的对象；

（出现目标对象动画）

2、目标对象：就是我们自己写的业务代码；

（出现织入代码动画）

3、织入代码：就是要在我们自己写的业务代码增加的代码片段；

（出现切面通知动画）

4、切面通知：就是封装织入代码片段的回调方法；

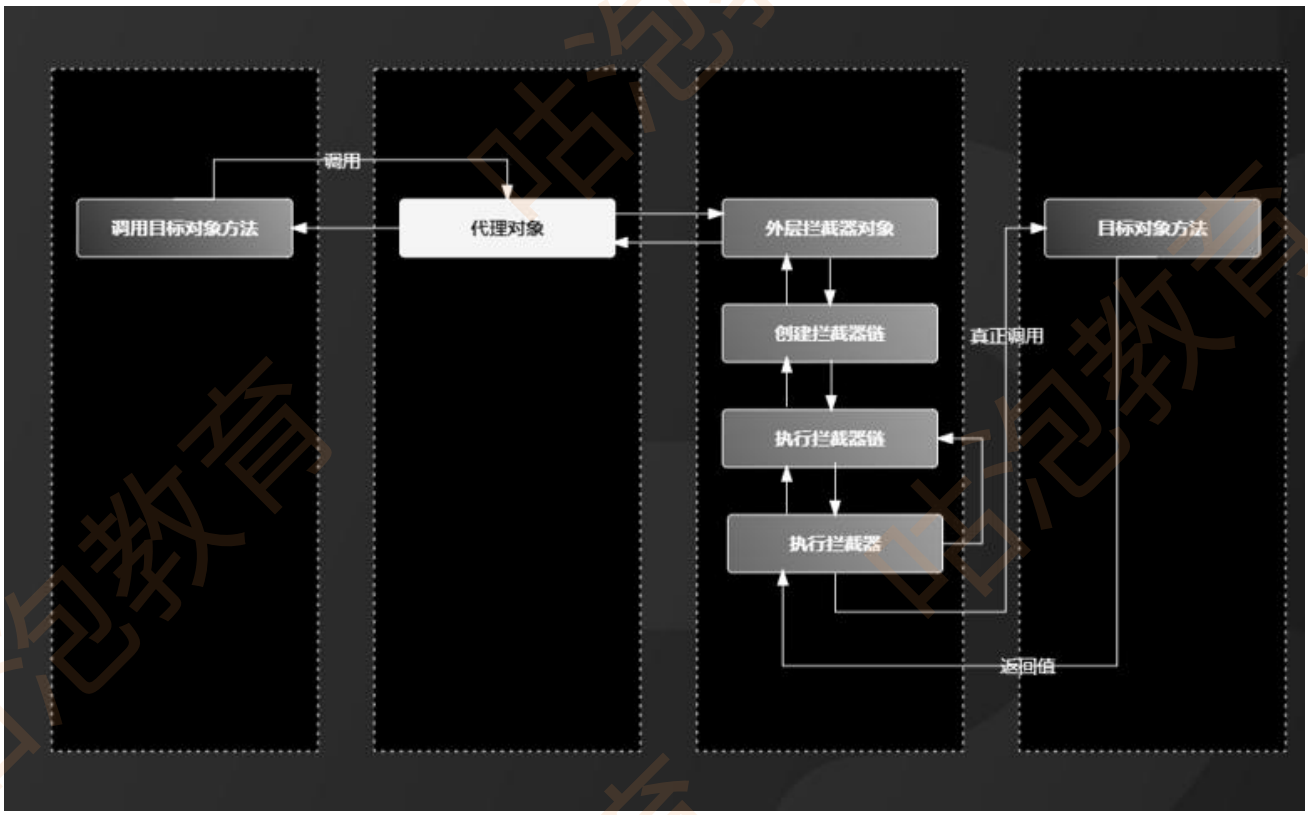
（出现MethodInvocation动画）

5、MethodInvocation：负责执行拦截器链，在proceed()方法中执行；

（出现MethodInterceptor动画）

6、MethodInterceptor：负责执行织入的代码片段，在invoke()方法中执行。

都看到这里了，你还觉得Spirng AOP原理难吗？我再送给大家一张精简版的Spring AOP执行流程图，需要的小伙伴立马关注点个赞。



我是被编程耽误的文艺Tom，如果大家还有其他疑问，可以在评论区留言。如果本次面试解析对

你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

10．3分钟轻松理解单线程下的HashMap工作原理

【引言】

HashMap主要是用来处理键值对数据。随着JDK版本的更新，JDK1.8对HashMap对底层也做了一些优化。今天我带大家一起来结合源码，深入浅出HashMap工作原理。 l

*（出现左侧UML类图）*HashMap是基于哈希表对Map接口的实现类，它的特点是访问数据的速度快，并不是按顺序来遍历。 HashMap提供所有可选的映射操作，但不能保证映射顺序不变，并且允许使用空值和空键。 *（出现代码片段并打钩）*HashMap也并不是线程安全的，当存在多个线程同时写入时，可能会导致数据不一致的情况。

（转场，屏幕黑底白字，正中央出现“1 HashMap中的关键属性”）

1、HashMap中的关键属性

【导航条：关键属性】

要透彻理解HashMap原理，首先需要对以下几个关键属性有一个基本的认识。

（展示代码 片段）

```
public class HashMap<K,V> extends AbstractMap<K,V>
implements Map<K,V> {

    final float loadFactor;

    int threshold;

    transient int size;

    transient int modCount;

    static final int DEFAULT_INITIAL_CAPACITY = 1 << 4;
}
```

我们看到，HashMap的源码片段：

第一个属性 loadFactor，它是负载因子，默认值是0.75，表示扩容前 。

第二个属性 threshold 它是记录HashMap所能容纳的键值对的临界值，它的计算规则是负载因子 乘以数组长度。

第三个属性 size，它用来记录HashMap实际存在的键值对的数量。

第四个属性 modCount，它用来记录HashMap内部结构发生变化的次数。

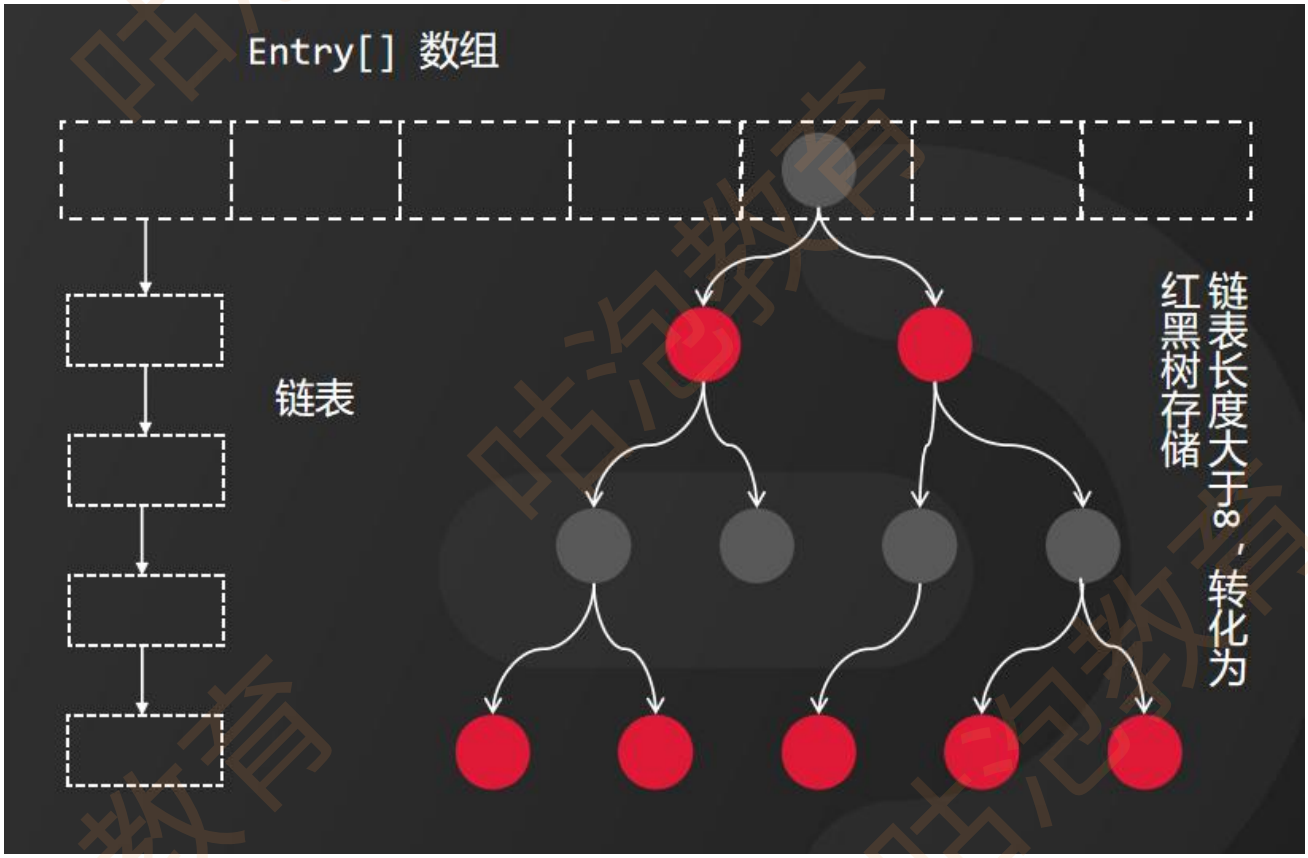
第五个是常量属性DEFAULT_INITIAL_CAPACITY ，它规定 的默认容量是16。

（转场，屏幕黑底白字，正中央出现“2 HashMap的存储结构”）

2、HashMap的存储结构

【导航条：存储结构】

（提前出现下图）



HashMap采用的是 的存储结构。HashMap的数组部分称为Hash桶，数组元素保存在一个叫做table的属性中。当链表长度大于等于8时，链表数据将会以红黑树的形式进行存储，当长度降到6时，又会转成链表形式存储。

（展示代码片段）

```
static class Node<K,V> implements Map.Entry<K,V> {
    ...

    final int hash;
    final K key;
    V value;
    Node<K,V> next;

    ...
}
```

每个Node节点，保存了用来定位数组索引位置的hash值、Key、Value和链表指向的下一个Node节点。而Node类是HashMap的内部类，它实现了Map.Entry接口，它的本质其实可以简单的理解成就是一个键值对。来看一下源码。

（转场，屏幕黑底白字，正中央出现“3 HashMap的工作原理”）

3、HashMap的工作原理

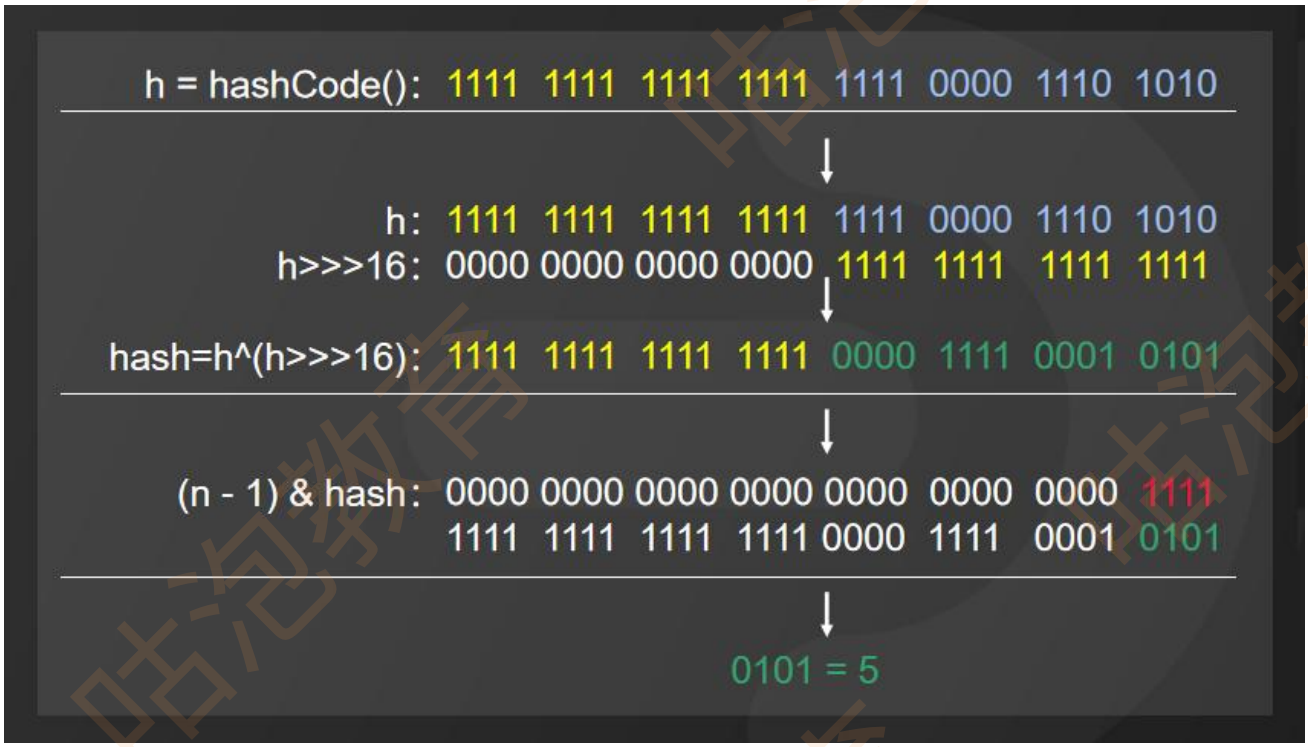
【导航条：工作原理】

当我们向HashMap中插入数据时，首先要确定Node在数组中的位置。那如何确定Node的存储位置呢？以添加Key为字符串“e”的对象为例：

（展示代码片段）

```
public static int hashCode(Object o) {
    return o != null ? o.hashCode() : 0;
}
```

(展示运算过程)

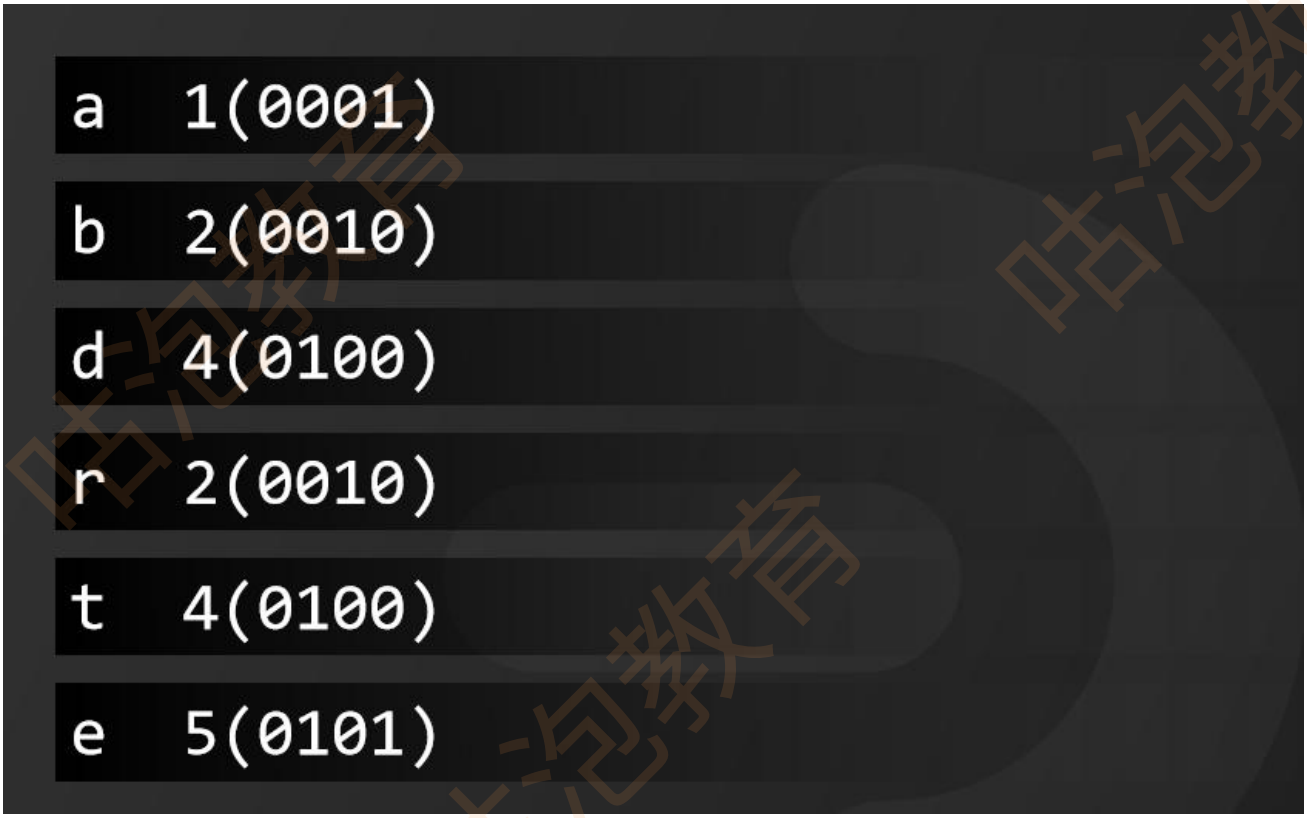


HashMap首先调用hashCode()方法，获取Key的hashCode值为h。然后对h值进行高位运算；将h右移16位取得h的高16位，与 进行异或运算，最后得到h的值与 (table.length - 1)进行与运算获得该对象的保留位，最后计算出下标。当然，这是最官方的描述。有的小伙伴可能已经迷糊了。其实，这段运算过程，简单地理解成求模取余法。

(展示计算公式)

就是用hash值和数组的长度减1，取模，最后得到数组的下标，这样可以保证数组下标不越界。只不过，位运算是二进制运算，效率更高。

最后，来看一段动画演示，假设有“a”、“b”、“d”、“r”，“t”，“e”的Key。通过计算得到的下标分别为 1 、2 、 4 、2 、4 、5

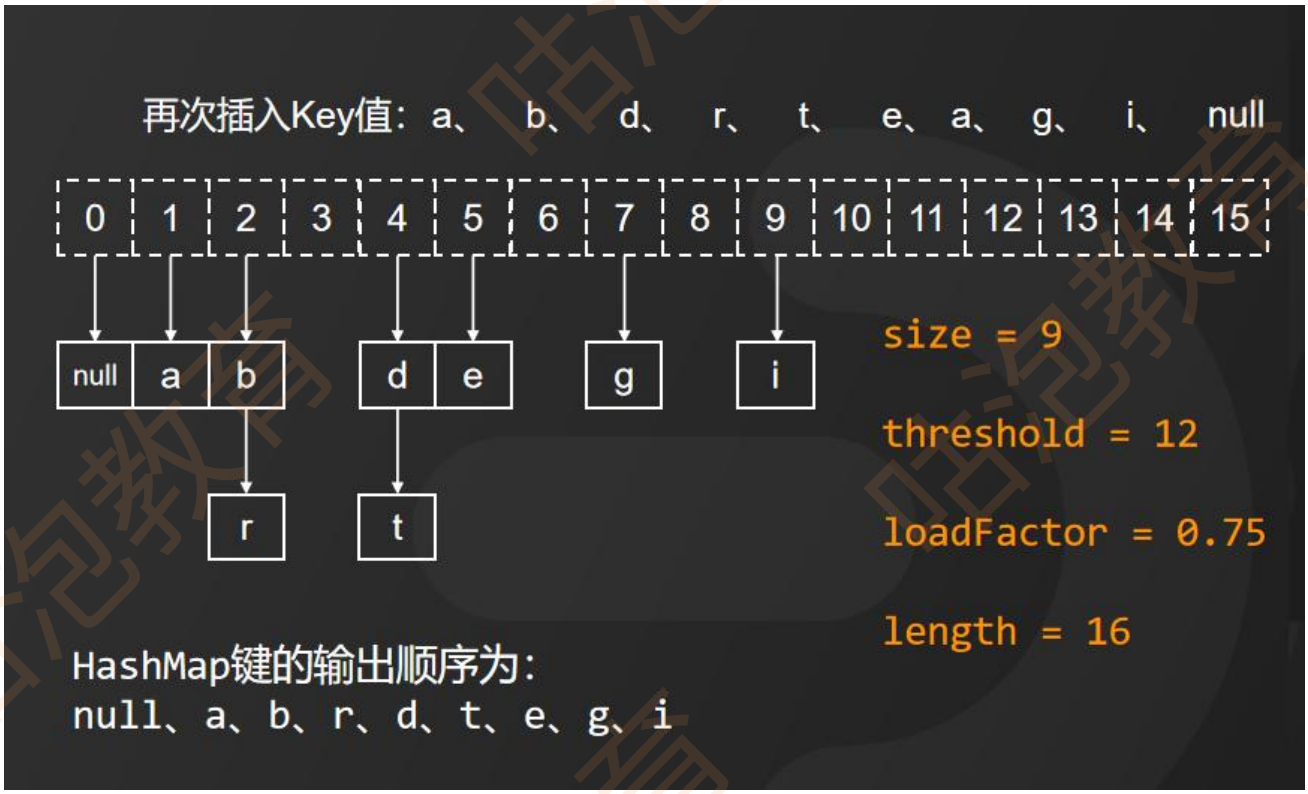


它们的插入顺序如动画所示。

（快速演示动画）

如果我们再次插入 "a"， “g”， “i”，null 四个Key，来看HashMap的内部变化。

（继续演示动画）



当插入第二个以a为Key的对象时，会将新值赋值给a的值。当插入的对象大小超过临界值时，

HashMap将新建一个桶数组并重新赋值（当然，JDK1.7和1.8重新赋值的方式略有不同）

这个时候，HashMap键的输出顺序为 null、a、b、r、d、t、e、g、i

（最后全屏动画展示HashMap扩容，人像隐藏）

HashMap的工作原理，你搞懂了吗?

我是被编程耽误的文艺Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

11．什么是内存溢出，什么是内存泄漏?

【导航条：引言】

什么是内存溢出，什么是内存泄漏？这是很多小伙伴经常问我的一个问题，今天花3分钟时间给大家介绍一下。先来介绍什么是内存溢出?

【导航条：内存溢出】

（转场，屏幕黑底白字，正中央出现“1 什么是内存溢出”）

1、什么是内存溢出?

发生内存溢出。

（开始出现动画效果）

我们来看到右侧的区域，假设我们JVM中可用的内存空间只剩下3M，但是我们要创建一个5M的对象，那么，新创建的对象就放不进去了。这个时候，我们就叫做内存溢出。就好比是一个容量只有300ml的水杯，我们硬要往里面倒500ml的水，这时候，水就会溢出，倒不进去了，这就相当于是内存的溢出。

那么，内存泄漏又是怎么回事呢？

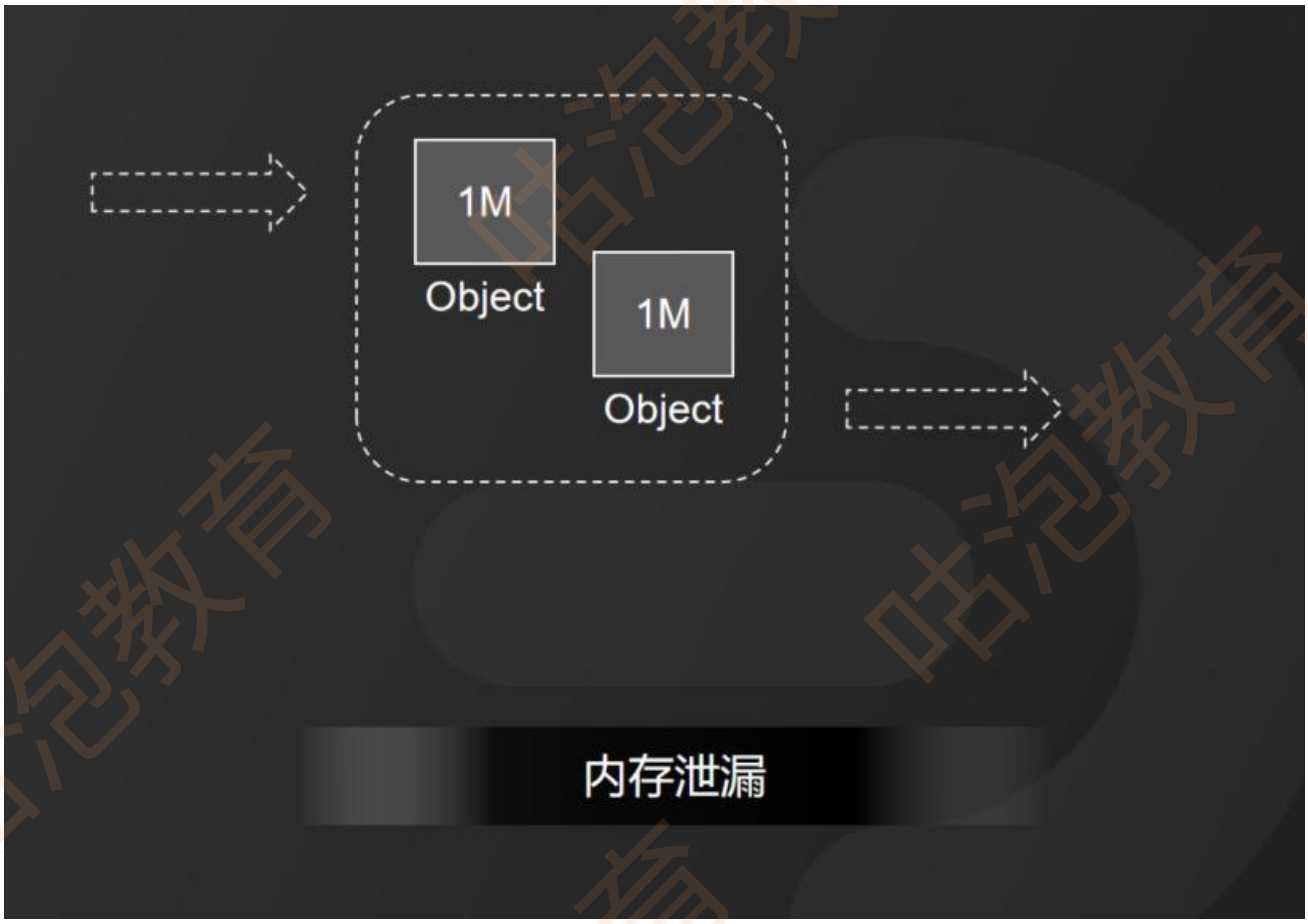
【导航条：内存泄漏】

（转场，屏幕黑底白字，正中央出现“2 什么是内存泄漏”）

2、什么是内存泄漏?

还是来看这张图，

（开始出现动画效果）



它代表业务代码执行时，所需要占用的内存空间。这段业务代码中创建了两个1M的对象，一起会占用2M内存。当对象使用完之后，这两个对象并没有释放，因此内存中会留下2M的内存空间一直被占用。而我们的业务代码在程序中会被反复执行，每次执行都会留下2M不被释放，反复执行多次之后，随着时间的累积，就会有大量的对象用完不被释放，导致这些对象不能得到回收而发生内存溢出，这种情况就叫做内存泄漏。

也就说，在我们的业务代码执行过程中，有些对象它应该被回收，但是又有其他对象引用引用它，因此，GC不能自动回收。所以，该回收的垃圾对象没有被回收，垃圾对象越堆越多，可用内存越来越少，若可用内存无法存放新的垃圾对象，最终导致内存泄漏。内存泄漏最终会导致内存溢出。

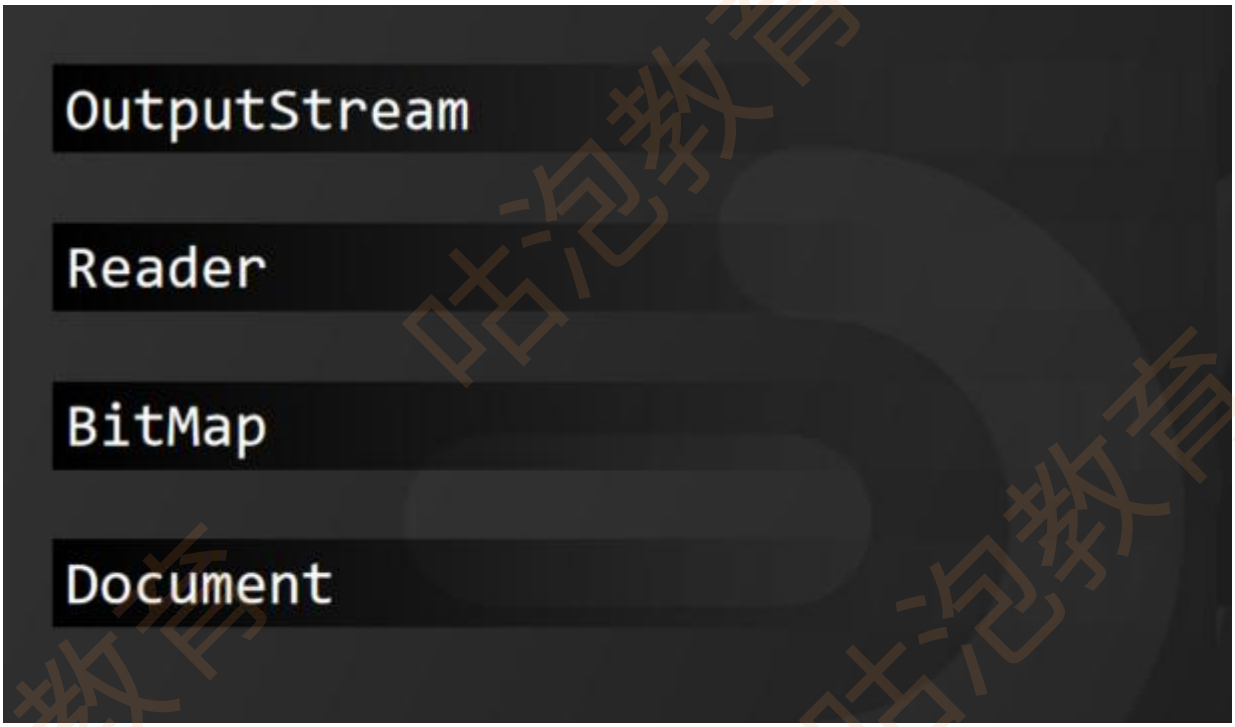
【导航条：如何避免】

（转场，屏幕黑底白字，正中央出现“3 如何避免”）

3、如何避免?

我们在Code过程中，特别是些一些流对象，

（开始出现动画效果）



比如OutputStream，Reader，BitMap，Document，很容易忘了Close。最麻烦的是还要顺序回收，顺序错了还产生空指针，所以，大家在Code过程一定要注意，当然，现在有很多IDE会智能提示，也避免了很多低级错误。

以上就是我对内存溢出和内存泄漏的分析，听懂了的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

12．什么条件下会产出死锁，如何避免死锁?

【导航条：引言】

一个去美团面试的小伙伴私我说，被面试官问到一个死锁的问题难道了，面试前还特意刷了题，面试的时候就是脑子一片空白不知道怎么回答。今天，我给大家彻底讲明白。

【导航条：什么是死锁】

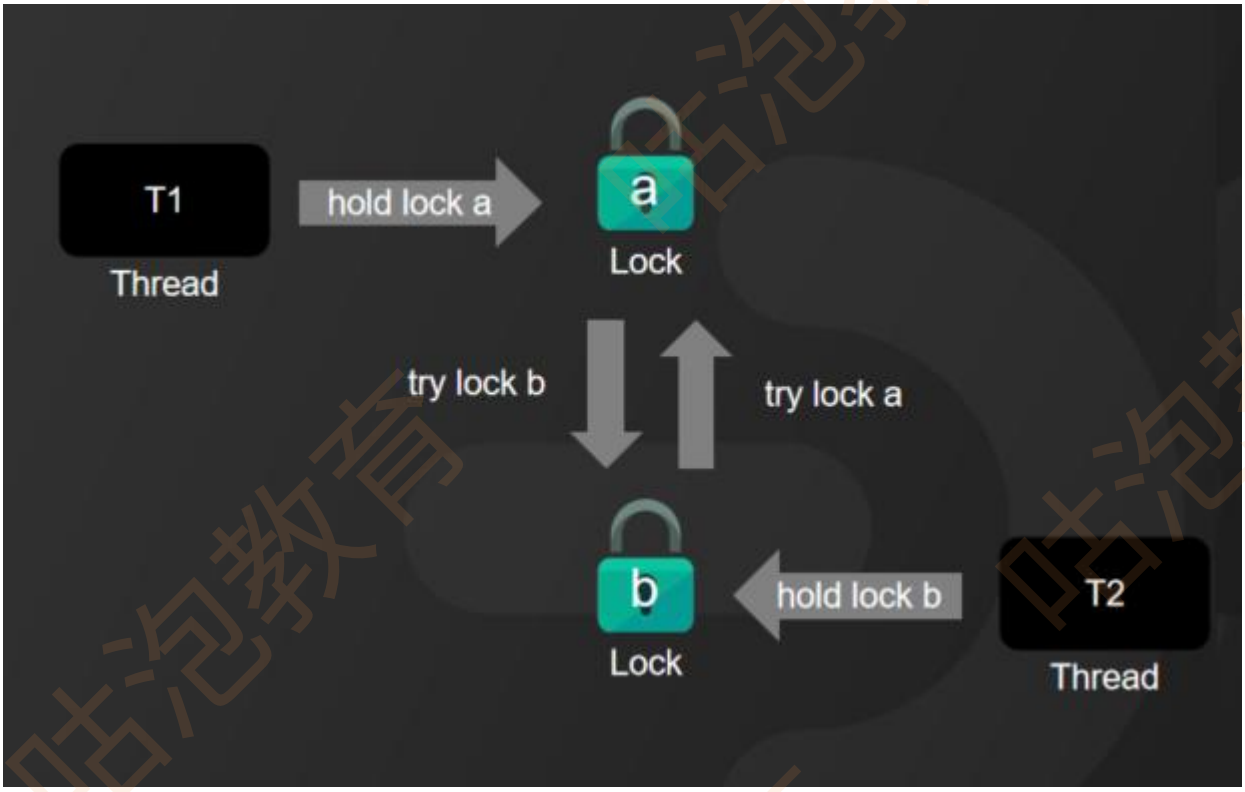
（转场，屏幕黑底白字，正中央出现“1 什么是死锁”）

1、什么是死锁?

先来看这样一段视频，*（左侧区域播放视频5秒，人在右侧站立不动）*

看完你也许已经懂了，那到底什么是死锁呢?

（开始出现动画效果）



死锁，简单来说就是两个或者两个以上的线程在执行过程中，去争夺同一个共享资源导致相互等待的现象。如果没有外部干预，线程会一直处于阻塞状态，无法往下执行。这样一直等待处于阻塞状态的线程，被称为死锁线程。

【导航条：产生死锁的原因】

（转场，屏幕黑底白字，正中央出现“2 产生死锁的原因”）

2、产生死锁的原因

产生死锁需要同时满足以下四个条件：

（开始出现动画效果）

第一个：互斥条件，共享资源a和b只能被一个线程占用；

第二个：请求和保持条件，线程T1已经获取共享资源a，在等待共享资源b的时候，不释放共享资源a；

第三个：不可抢占条件，其他线程不能强行抢占线程T1占有的资源；

第四个：循环等待条件，线程T1等待线程T2占有的资源，线程T2等待线程T1占有的资源，这形成了循环等待。

【导航条：如何避免死锁】

（转场，屏幕黑底白字，正中央出现“3 如何避免死锁”）

3、如何避免死锁?

线程产生死锁之后，只能通过外部干预来解决问题，比如重启程序，或者Kill线程。所以，我们只能在写代码时规避死锁的产生。那么如何避免死锁产生呢？根据产生死锁的四个必要条件，我们只需要破坏其中任何一 个条件就可以解决。

第一个互斥条件是没有办法被破坏的，因为它是互斥锁的基本约束。其他三个条件都可以通过人工干预来破坏。比如请求保持条件，我们可以在首次执行一次性申请所有的资源，这样就不存在等待锁的问题了。

第二个，对于不可抢占条件来说，占用部分资源的线程在进一步申请其他资源的时候如果申请不到，我们可以主动释放它占有的资源。这样不可抢占这个条件就被破坏了。

第三个，对于循环等待条件来说，可以通过按序申请资源来预防死锁的产生。所谓按序申请，就是给资源编号，所有线程可以按照线性化的序号顺序去申请共享资源，先申请需要序号小的，再申请序号大的，这样循环等待自然就不存在了。

以上就是我对死锁的理解，听懂了的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

13．两个Integer对象比较大小，为什么100等于100,1000不等于1000？

【导航条：引言】

前几天，有位小伙伴向我反馈，在维护代码过程中，出现了一个莫名其妙的问题。明明上线之后程序跑得还好好的，可程序上线运行一段时间之后，所有，代码没有做任何修改，发 cxccccc现运行结果和期望值恰好相反。因为涉及到金额造成了比较大的损失，最后，这位小伙伴还被公司辞退了，大家可以来评论一下，这位小伙伴背的这个锅值不值?

【导航条：业务场景】

（转场，屏幕黑底白字，正中央出现“”）

1、业务场景

大家来看，他的代码大致是这样写的：

（展示代码片段）

```
public boolean xxx(Integer a,Integer b){
    ...
    if(a == b){
        ...
        return true;
    }
    ...
    return false;
}
a和b输入100，返回结果是：true
a和b输入1000，返回结果是：false
```

一般情况下，a和b都输入100的时候，返回为true，但当a和b都输入1000的时候，返回为false。按照正常逻辑理解，100 等于等于 100，那1000 为什么不等于等于1000 呢？这位同学，百思不得其解。于是，这位同学，还特意写了一段测试代码

（展示代码片段）

```
Integer a = 100,b = 100,c = 1000,d = 1000;

System.out.println((a == b) + "," + (c == d));
```

运行结果是：true,false

这到底是什么原因呢？我们对照Integer的源码来进行分析：

【导航条：源码分析】

（转场，屏幕黑底白字，正中央出现“2 源码分析”）

2、源码分析

我摘取了Integer的源码片段，它有一个valueOf()的方法：

（展示代码片段）

```
public final class Integer extends Number implements Comparable<Integer> {
    ...
    public static Integer valueOf(int i) {
        if (i >= IntegerCache.low && i <= IntegerCache.high)
            return IntegerCache.cache[i + (-IntegerCache.low)];
        return new Integer(i);
    }
    ...
}
```

我们可以看到，Integer源码中的valueOf()方法做了一个条件判断，其中 IntegerCache.low的值为-

128，

（出现动画）

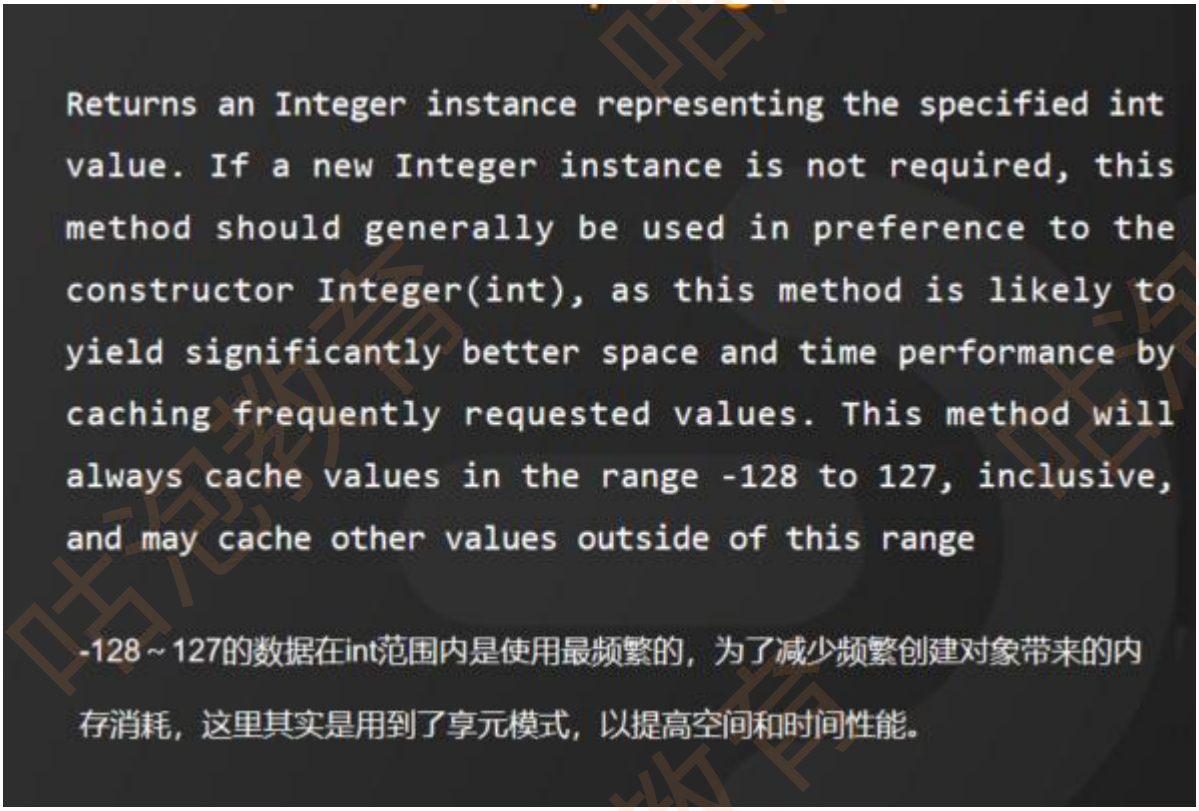
IntegerCache.high的值为127。

（出现动画）

也就是说如果目标值在-128～127之间，会直接从cache数组中取值，否则就会新建对象。

这里又有人会问了，那为什么默认是-128 - 127，怎么不是-200 - 200或者是其他值呢？那JDK为何要这样做呢?

（出现动画）



在Java API 中是这样解释的：

Returns an Integer instance representing the specified int value. If a new Integer instance is not required, this method should generally be used in preference to the constructor Integer(int), as this method is likely to yield significantly better space and time performance by caching frequently requested values. This method will always cache values in the range -128 to 127, inclusive, and may cache other values outside of this range

大致意思是：

-128～127的数据在int范围内是使用最频繁的，为了减少频繁创建对象带来的内存消耗，这里其实是用到了享元模式，以提高空间和时间性能。

【导航条：扩展知识】

（转场，屏幕黑底白字，正中央出现“3 扩展知识”）

在JDK中，这样的应用不止int，我给小伙伴们整理了一个表格，表格中的其他类型也都应用了享元模式，也就是说对数值做了缓存，只是缓存的范围不一样，具体如表中所示：

基本类型	大小	最小值	最大值	包装器类型	缓存范围	是否支持自定义
boolean	-	-	-	Bloolean	-	-
char	6bit	Unicode 0	Unic ode 2(16)-1	Character	0~127	否
byte	8bit	-128	127	Byte	-128~127	否
short	16bit	-2(15)	2(15)-1	Short	-128~127	否
int	32bit	-2(31)	2(31)-1	Integer	-128~127	支持
long	64bit	-2(63)	2(63)-1	Long	-128~127	否
float	32bit	IEEE754	IEEE754	Float	-	
double	64bit	IEEE754	IEEE754	Double	-	
void	-	-	-	Void	-	-

我需要这张表的小伙伴可以私信我，以上就是关于Integer对象比较大小的分析，听懂了的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

14．为什么HashMap会产生死循环?

【导航条：引言】

HashMap死循环是一个比较常见、也是比较经典的面试题，在大厂的面试中也经常被问到。HashMap的死循环问题只在JDK1.7版本中会出现，主要是HashMap自身的工作机制，再加上并发操作，从而导致出现死循环。JDK1.8以后，官方彻底解决了这个问题。

【导航条：数据插入原理】

（转场，屏幕黑底白字，正中央出现“1 数据插入原理”）

1、数据插入原理

在分析原因之前，我先带大家了解一下JDK1.7中HashMap插入数据的原理，来看动画演示：

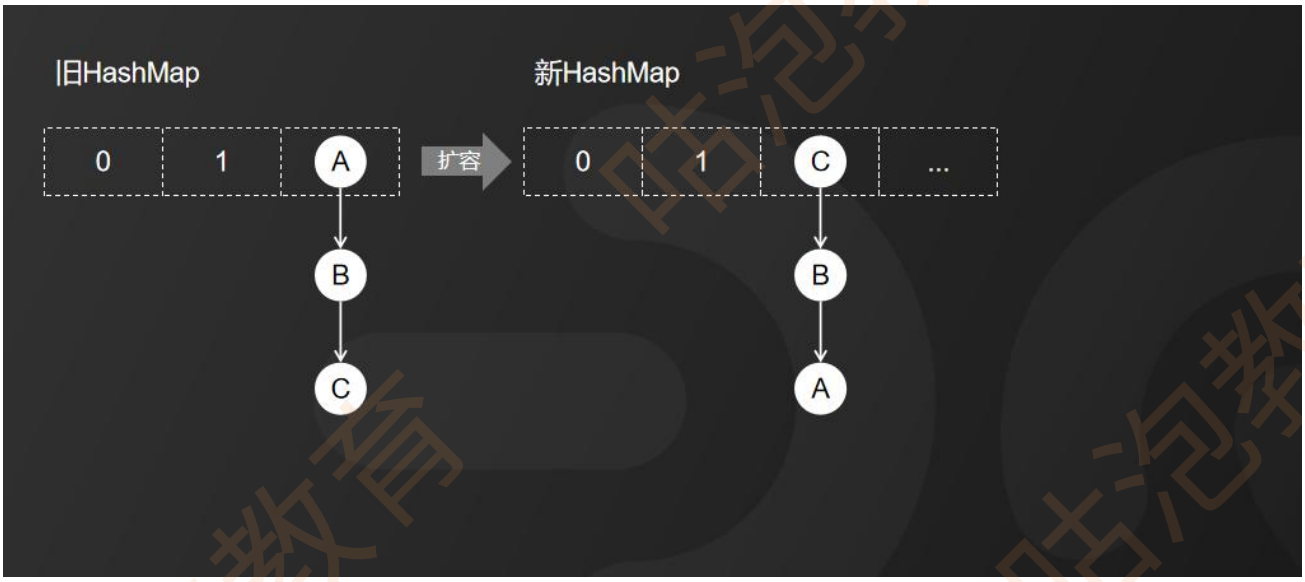
（出现动画）

由于JDK 1.7中HashMap的底层存储结构采用的是数组 加 链表的方式。

（出现动画）

而HashMap在数据插入时又采用的是头插法，也就是说新插入的数据会从链表的头节点进行插入。

（出现动画）



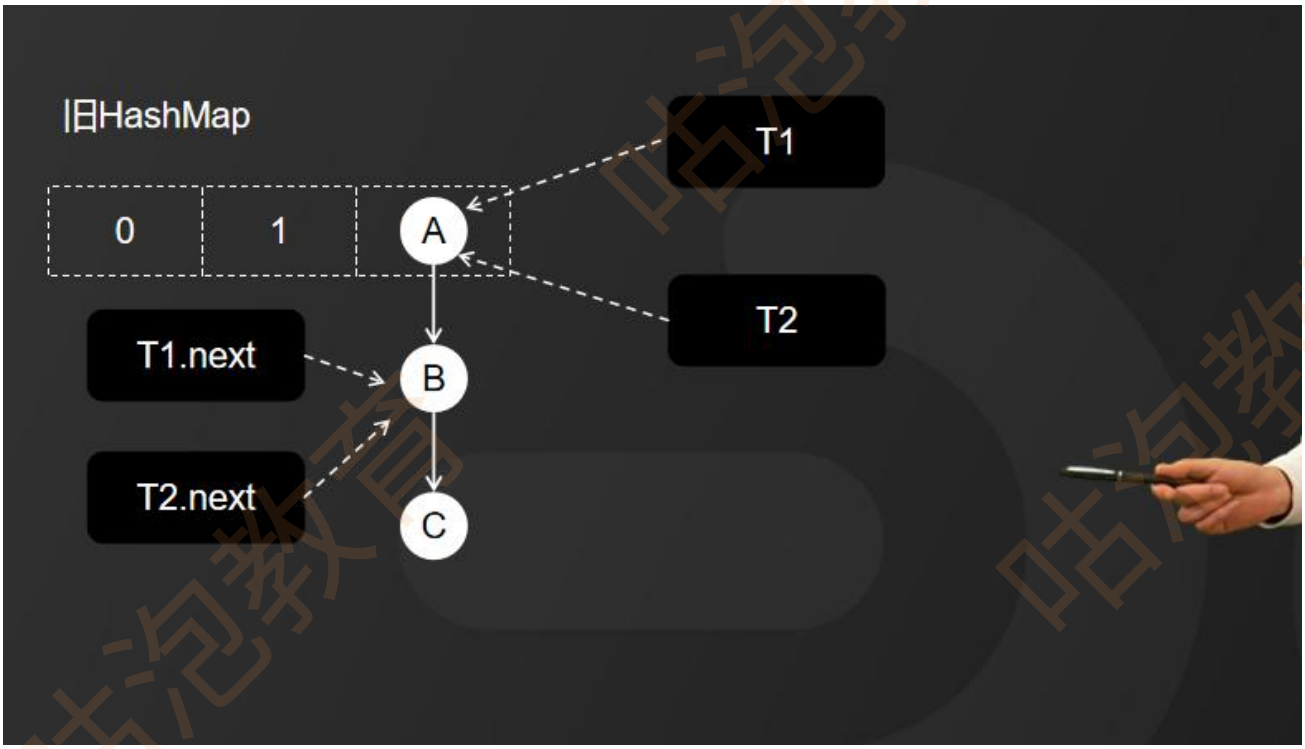
因此，HashMap正常情况下的扩容就是是这样一个过程。我们来看，旧HashMap的节点会依次转移到新的HashMap中，旧HashMap转移链表元素的顺序是A、B、C，而新HashMap使用的是头插法插入，所以，扩容完成后最终在新HashMap中链表元素的顺序是C、B、A。

【导航条：导致死循环的原因】

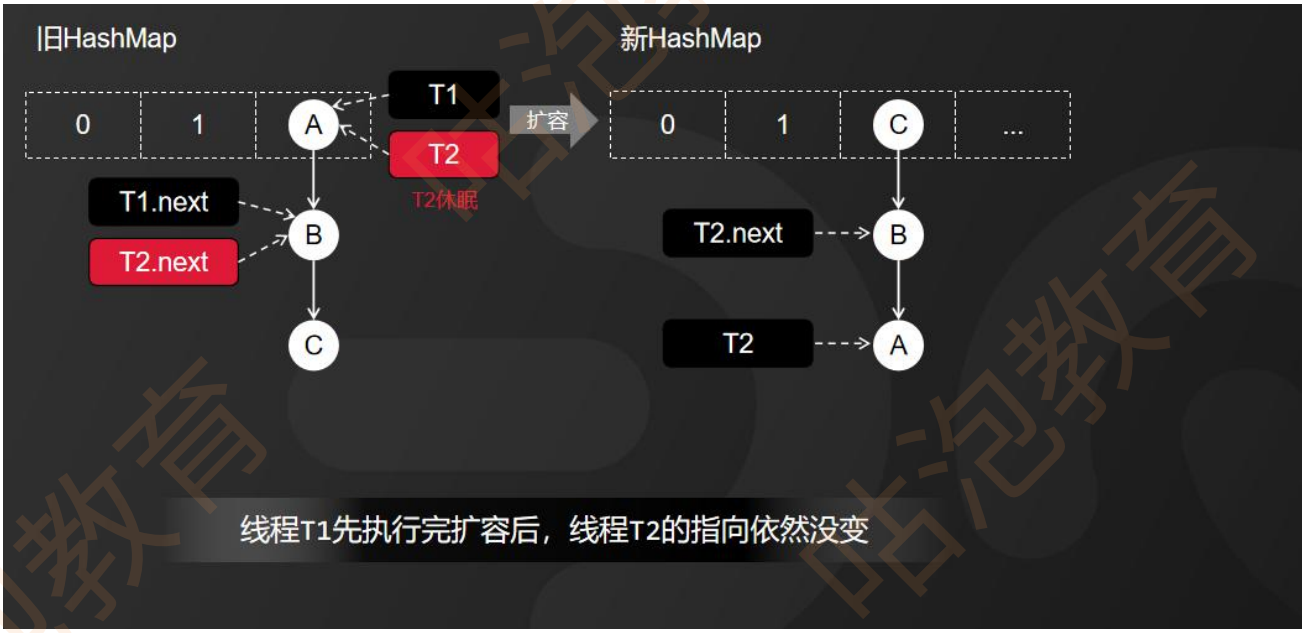
（转场，屏幕黑底白字，正中央出现“2 ”）

2、导致死循环的原因

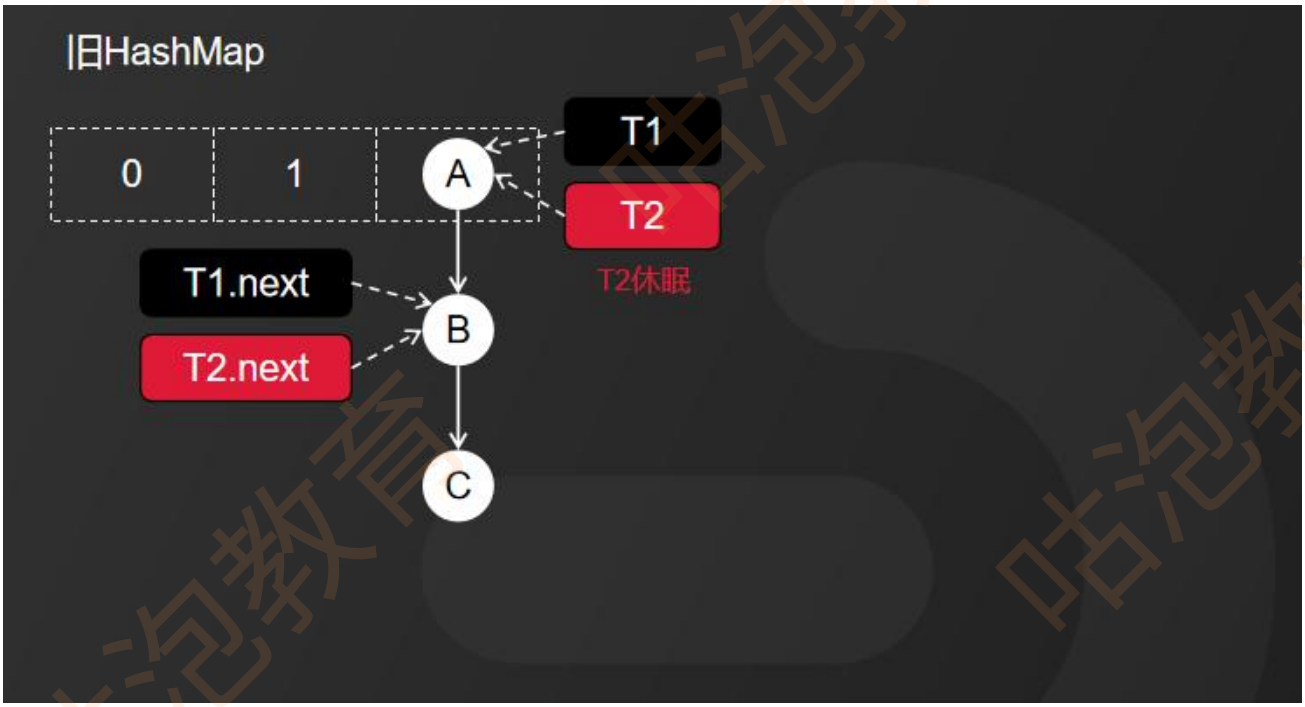
接下来，我通过动画演示的方式，带大家彻底理解造成HashMap死循环的原因。我们按以下三个步骤来还原并发场景下HashMap扩容导致的死循环问题：



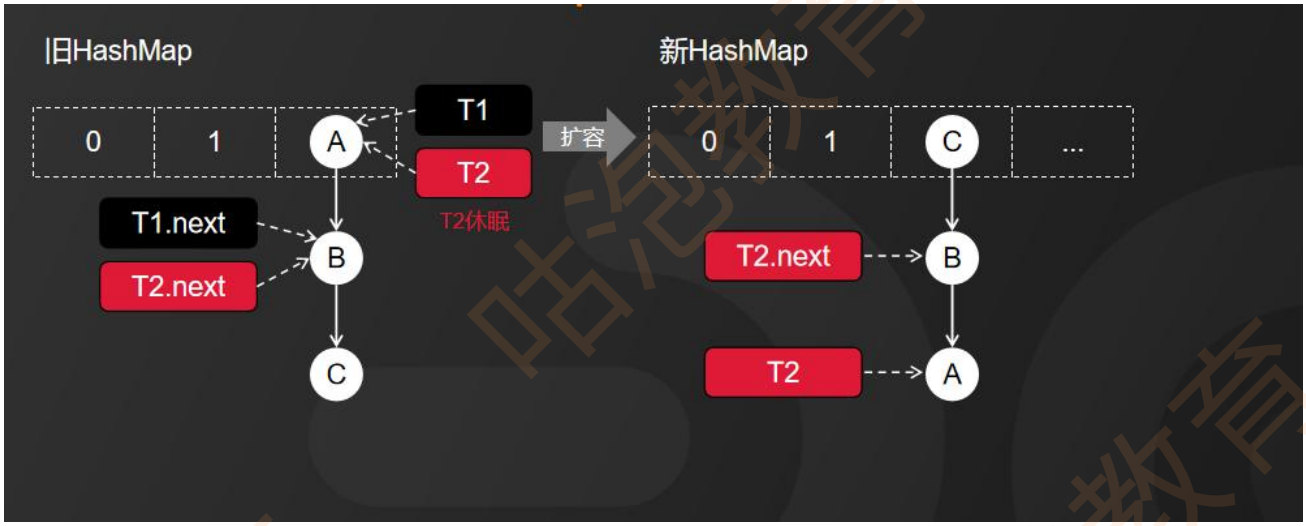
第一步：线程启动，有线程T1和线程T2都准备对HashMap进行扩容操作， 此时T1和T2指向的都是链表的头节点A，而T1和T2的下一个节点分别是T1.next和T2.next，它们都指向B节点。



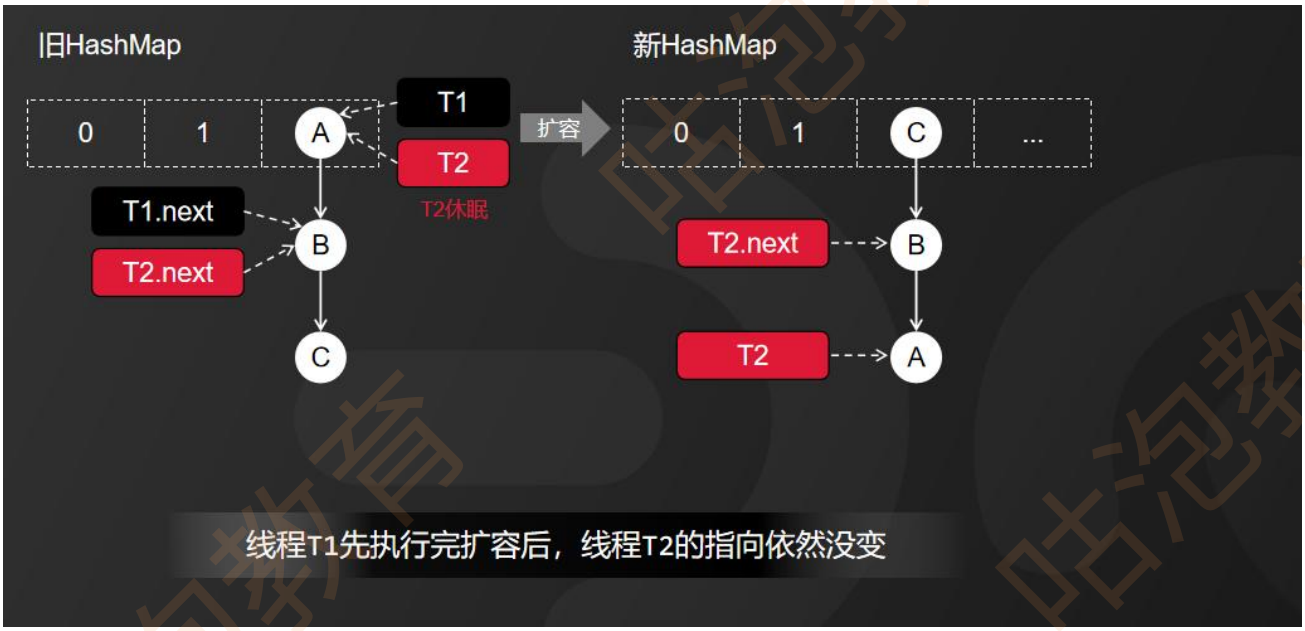
第二步：开始扩容，这时候，假设线程T2的时间片用完，进入了休眠状态，而线程T1开始执行扩容操作，一直到线程T1扩容完成后，线程T2才被唤醒。



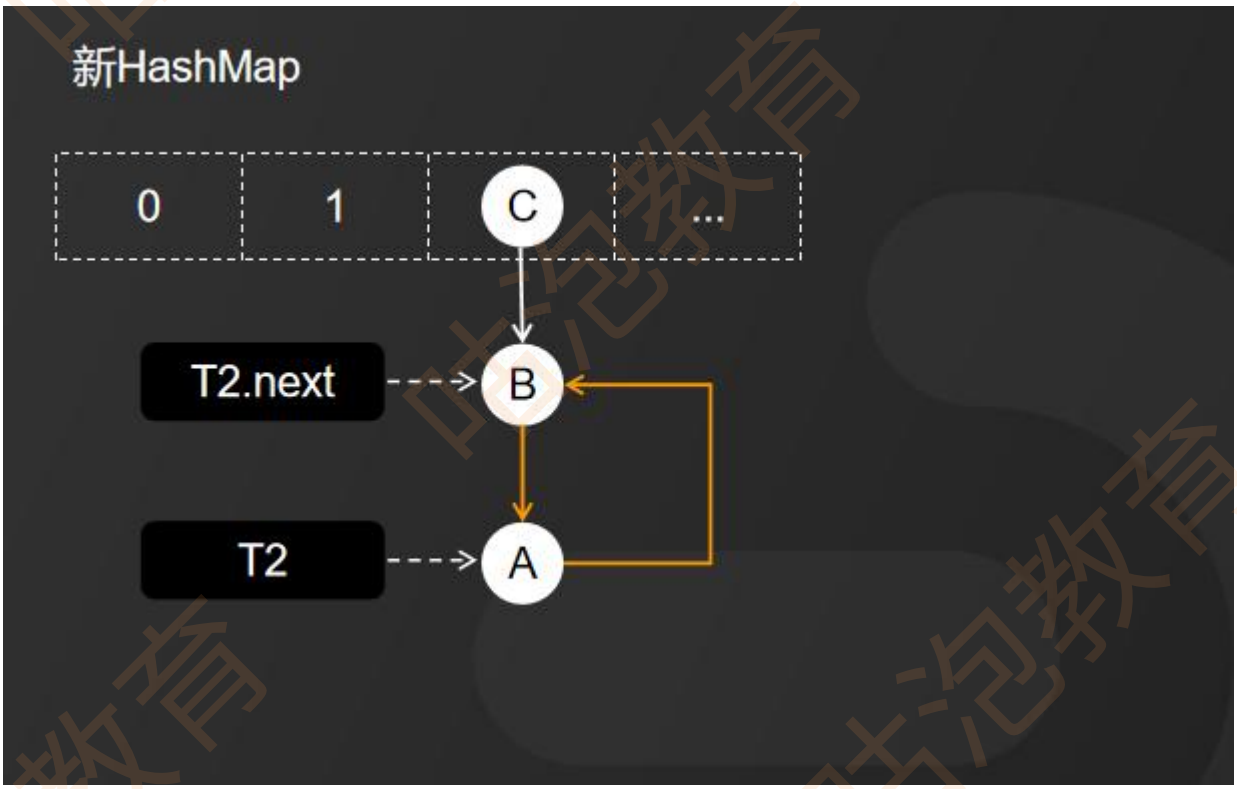
T1完成扩容之后的场景就变成动画所示的这样。



因为HashMap扩容采用的是头插法，线程T1执行之后，链表中的节点顺序发生了改变。但线程T2对于发生的一切还是不可知的，所以它指向的节点引用依然没变。如图所示，T2指向的是A节点，T2.next指向的是B节点。



当线程T1执行完成之后，线程T2恢复执行时，死循环就发生了。



因为T1执行完扩容之后，B节点的下一个节点是A，而T2线程指向的首节点是A，第二个节点是B，这个顺序刚好和T1扩容之前的节点顺序是相反的。T1执行完之后的顺序是B到A，而T2的顺序是A到B，这样A节点和B节点就形成了死循环。

【导航条：解决方案】

（转场，屏幕黑底白字，正中央出现“3 解决方案”）

3、解决方案

避免HashMap发生死循环的常用解决方案有三个：

1)、使用线程安全的ConcurrentHashMap替代HashMap，个人推荐使用此方案。

2)、使用线程安全的容器Hashtable替代，但它性能较低，不建议使用。

3)、使用synchronized或Lock加锁之后，再进行操作，相当于多线程排队执行，也会影响性能，不建议使用。

【导航条：总结】

（转场，屏幕黑底白字，正中央出现“4 总结”）

4、总结

HashMap死循环只发生在JDK1.7版本中，主要原因是JDK1.7中的HashMap，在头插法 加 链表 加 多线程并发 加 扩容这几个情形累加到一起就会形成死循环。多线程环境下建议采用ConcurrentHashMap替代。在JDK1.8中，HashMap改成了尾插法，解决了链表死循环的问题。

以上就是关于HashMap死循环原因的分析，听懂的小伙伴，关注点个赞，下次不迷路。

我是被编程耽误的文艺Tom，如果还有其他疑问或者需要高清无码图的小伙伴，请在评论区留言。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！3人未读

15．Overenthusiastic是如何保证线程安全的?

【导航条：引言】

ConcurrentHashMap相当于是HashMap的多线程版本，它的功能本质上和HashMap没什么区别。因为HashMap在并发操作的时候会出现各种问题，比如死循环问题、数据覆盖等问题。而这些问题，只要使用ConcurrentHashMap就可以完美地解决。那问题来到了，ConcurrentHashMap它是如何保证线程安全的呢?

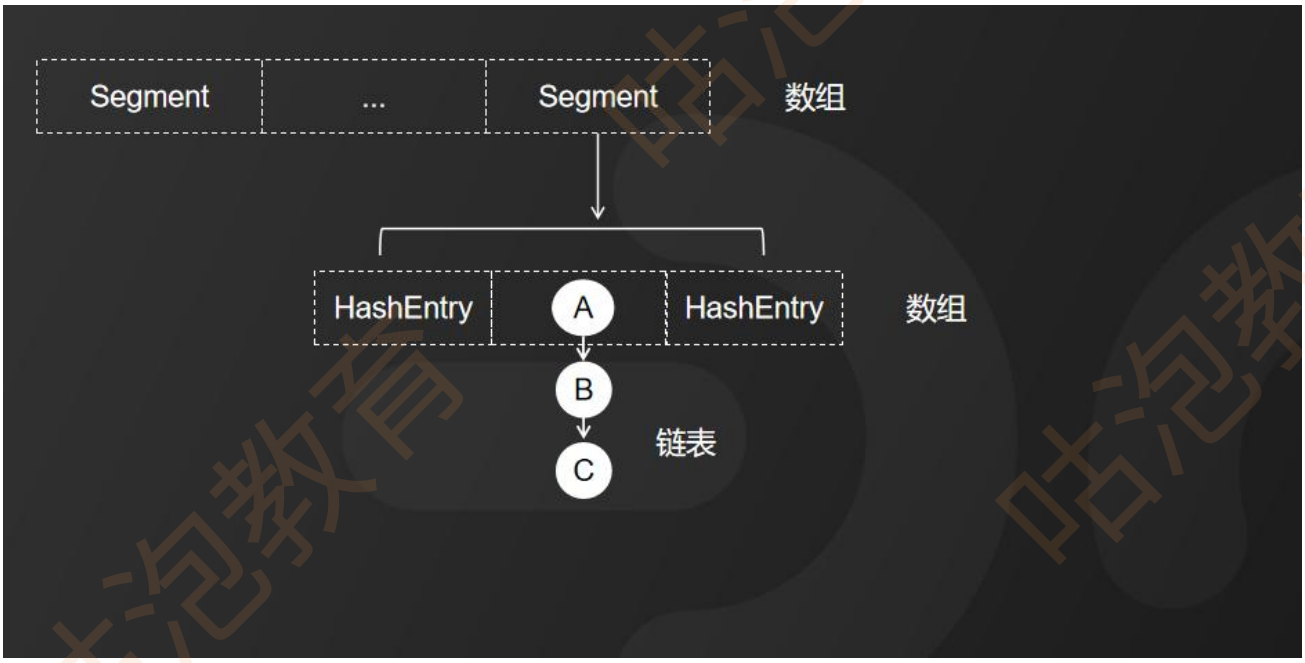
【导航条：JDK1.7实现原理】

（转场，屏幕黑底白字，正中央出现“1 JDK 1.7实现原理”）

1、JDK1.7实现原理

首先，我们来看JDK 1.7中ConcurrentHashMap的底层结构，它基本延续了HashMap的设计，采用的是数组 加 链表的形式。和HashMap不同的是，ConcurrentHashMap中的数组设计　分为大数组

Segment和小数组HashEntry，来着这张图。

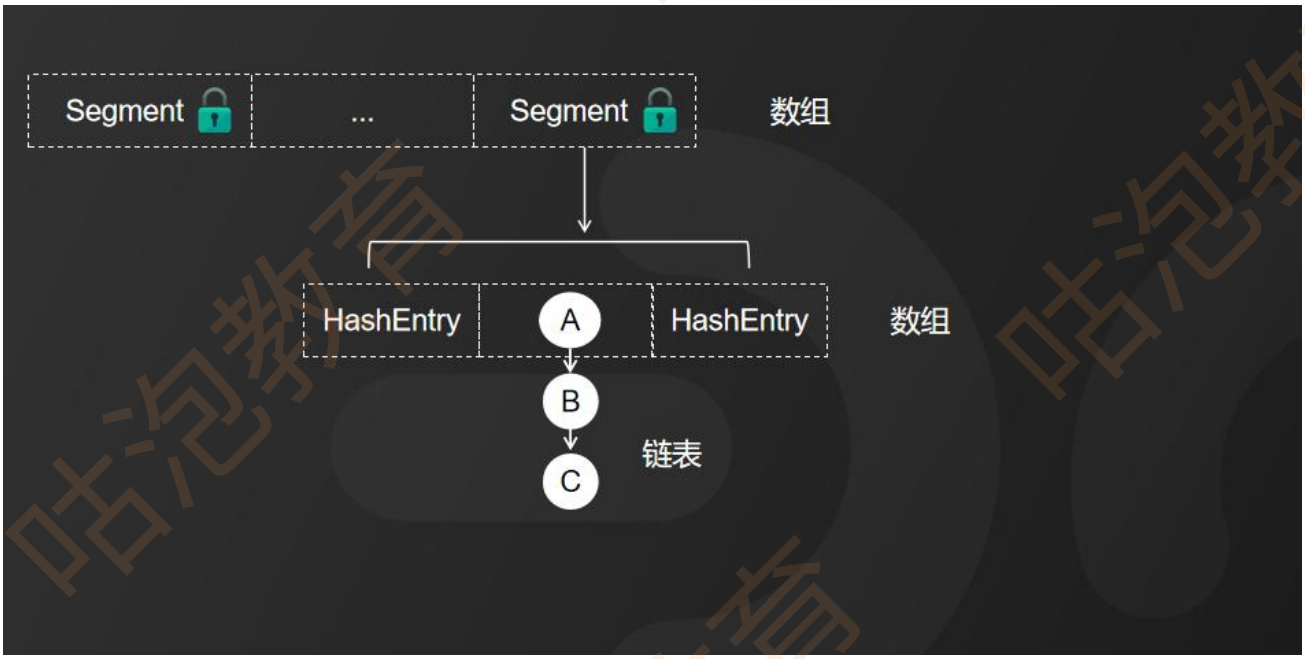


大树组Segment可以理解为一个数据库，而每个数据库（Segment）中又有很多张表（HashEntry），每个HashEntry中又有很多条数据，这些数据是用链表连接的。了解了ConcurrentHashMap的基本结构设计，我们再来看它的线程安全实现，就比较简单了。

接下来我们来对照JDK1.7中ConcurrentHashMap的put()方法源码实现。

```
final V put(K key, int hash, V value, boolean onlyIfAbsent) {
    HashEntry<K,V> node = tryLock() ? null : scanAndLockForPut(key, hash, value);
    V oldValue;
    try {
        HashEntry<K,V>[] tab = table;
        int index = (tab.length - 1) & hash;
        HashEntry<K,V> first = entryAt(tab, index);
        for (HashEntry<K,V> e = first;;) {
            if (e != null) {
                K k;
                ...
            }
            else {
                ...
            }
        }
    } finally {
        unlock();
    }
    return oldValue;
}
```

因为Segment本身是基于ReentrantLock重入锁实现的加锁和释放锁的操作，这样就能保证多个线程同时访问ConcurrentHashMap时，同一时间只能有一个线程能够操作相应的节点，这样就保证了ConcurrentHashMap的线程安全。



也就是说ConcurrentHashMap的线程安全是建立在Segment加锁的基础上的，所以，我们称它为分段锁或者片段锁，如图中所示。

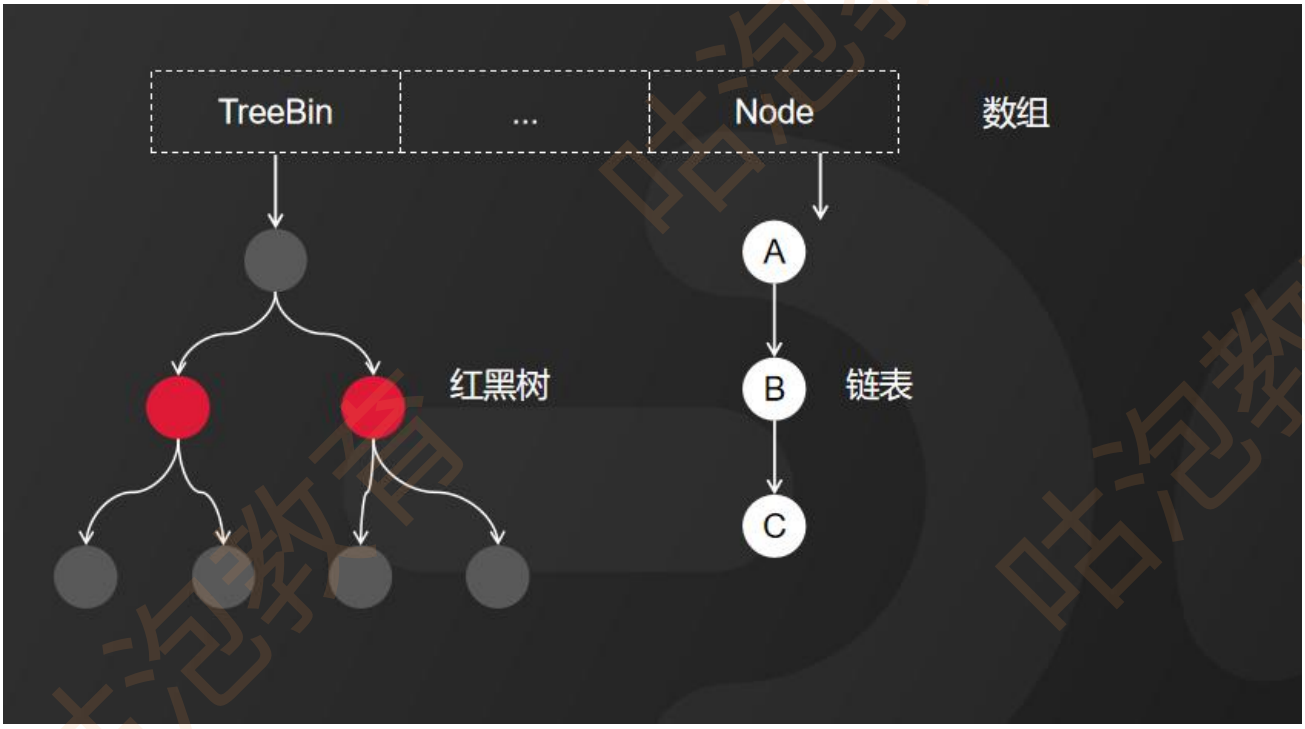
那JDK1.8又是如何实现的呢?

【导航条：JDK1.8性能优化】

（转场，屏幕黑底白字，正中央出现“2 JDK 1.8性能优化”）

2、JDK1.8优化内容

在JDK1.7中，ConcurrentHashMap虽然是线程安全的，但因为它的底层实现是数组加链表的形式，所以在数据比较多情况下，因为要遍历整个链表，会降低访问性能。所以，JDK1.8以后采用了数组 加 链表 加 红黑树的方式优化了ConcurrentHashMap的实现，具体实现如图所示。



当链表长度大于8，并且数组长度大于64时，链表就会升级为红黑树的结构。JDK 1.8中的ConcurrentHashMap虽然保留了Segment的定义，但这，仅仅是为了保证序列化时的兼容性，不再有任何结构上的用处了。

那在JDK 1.8中ConcurrentHashMap的源码是如何实现的呢？它主要是使用了CAS 加 volatile 或者 synchronized 的方式来保证线程安全。

```
final V putVal(K key, V value, boolean onlyIfAbsent) {
    if (key == null || value == null) throw new NullPointerException();
    int hash = spread(key.hashCode());
    int binCount = 0;
    for (Node<K,V>[] tab = table;;) {
        Node<K,V> f; int n, i, fh;
        if (tab == null || (n = tab.length) == 0)
            tab = initTable();
        else if ((f = tabAt(tab, i = (n - 1) & hash)) == null) {
            if (casTabAt(tab, i, null, new Node<K,V>(hash, key, value, null)))
                break;
        }
        else if ((fh = f.hash) == MOVED)
            tab = helpTransfer(tab, f);
        else {
            V oldVal = null;
            synchronized (f) {
                ...
                }
            }
            if (binCount != 0) {
                if (binCount >= TREEIFY_THRESHOLD)
                    treeifyBin(tab, i);
                if (oldVal != null)
                    return oldVal;
                break;
            }
        }
    }
    addCount(1L, binCount);
    return null;
}
```

我们可以从源码片段中看到，添加元素时首先会判断容器是否为空，

如果为空则使用 volatile 加 CAS 来初始化，

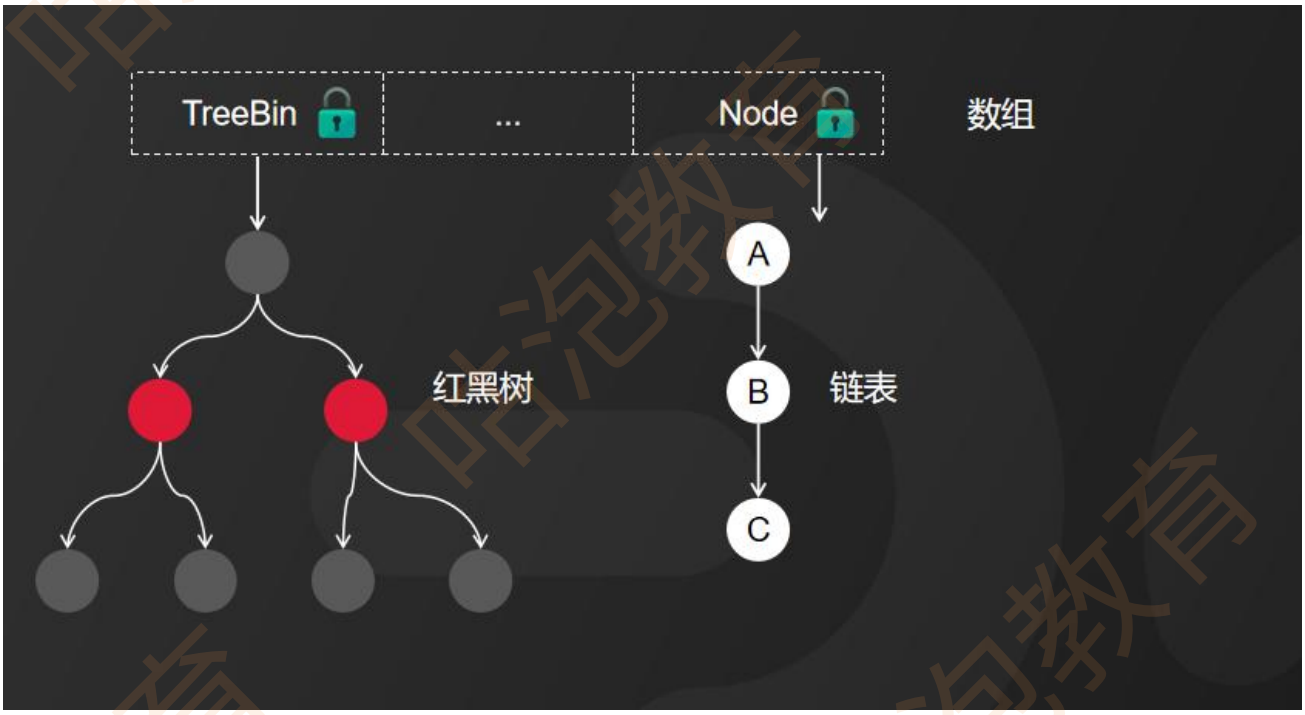
如果容器不为空，则根据存储的元素计算该位置是否为空。

如果根据存储的元素计算结果为空则利用 CAS 设置该节点；

如果根据存储的元素计算为空不为空，则使用 synchronized ，然后，遍历桶中的数据，并替换或新增节点到桶中，

最后再判断是否需要转为红黑树。这样就能保证并发访问时的线程安全了。

如果把上面的执行用一句话归纳的话，就相当于是ConcurrentHashMap通过对头结点加锁来保证线程安全的。



这样设计的好处是，使得锁的粒度相比Segment来说更小了，发生hash冲突 和 加锁的频率也降低了，在并发场景下的操作性能也提高了。而且，当数据量比较大的时候，查询性能也得到了很大的提升。

【导航条：总结】

（转场，屏幕黑底白字，正中央出现“3 总结”）

3、总结

最后，我们来总结一下：

1、ConcurrentHashMap在JDK 1.7中使用的数组 加 链表的结构，其中数组分为两类，大树组 Segment 和 小数组 HashEntry，而加锁是通过给Segment添加ReentrantLock重入锁来保证线程安全的。

2、ConcurrentHashMap在JDK1.8中使用的是数组 加 链表 加 红黑树的方式实现，它是通过 CAS 或者 synchronized 来保证线程安全的，并且缩小了锁的粒度，查询性能也更高。

ConcurrentHashMap中有很多设计思想是值得我们去学习和借鉴的，比如说锁的粒度控制、分段锁的设计等等，都可以应用在实际的业务开发场景中。我们通过学习这些底层原理从中获取很多的设计思路，帮助我们更高效地去解决实际问题。

好了，关于ConcurrentHashMap的分享就讲到这里，听懂的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺Tom，如果还有其他疑问或者需要 的小伙伴，请在评论区留言。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

16．为什么ConcurrentHashMap不允许插入null值?

【导航条：引言】

在Java语言中，给ConcurrentHashMap和Hashtable这些线程安全的集合中的Key或者Value插入

null（空） 值的会报空指针异常，但是单线程操作的HashMap又允许 Key 或者 Value 插入 null（空） 值。这到底是为什么呢?

【导航条：探寻源码】

（转场，屏幕黑底白字，正中央出现“ ”）

1、探寻源码

为了找到原因，我们先来看这样一段源码片段，打开ConcurrentHashMap的putVal()方法，源码中第一句就非常明确地做了判断，如果 Key 或者 Value 为 null（空） 值，就直接抛出空指针异常。

我们在源码中似乎已经找到了原因，你可以这样回答面试官，说JDK源码就是这么规定的。然而，这个原因是不能说服面试官的，虽然，源码是这样设计的，我们要思考的是，这样设计背后更深层次的原因。

那到底为什么ConcurrentHashMap不允许插入 null （空）值，HashMap又允许插入呢?

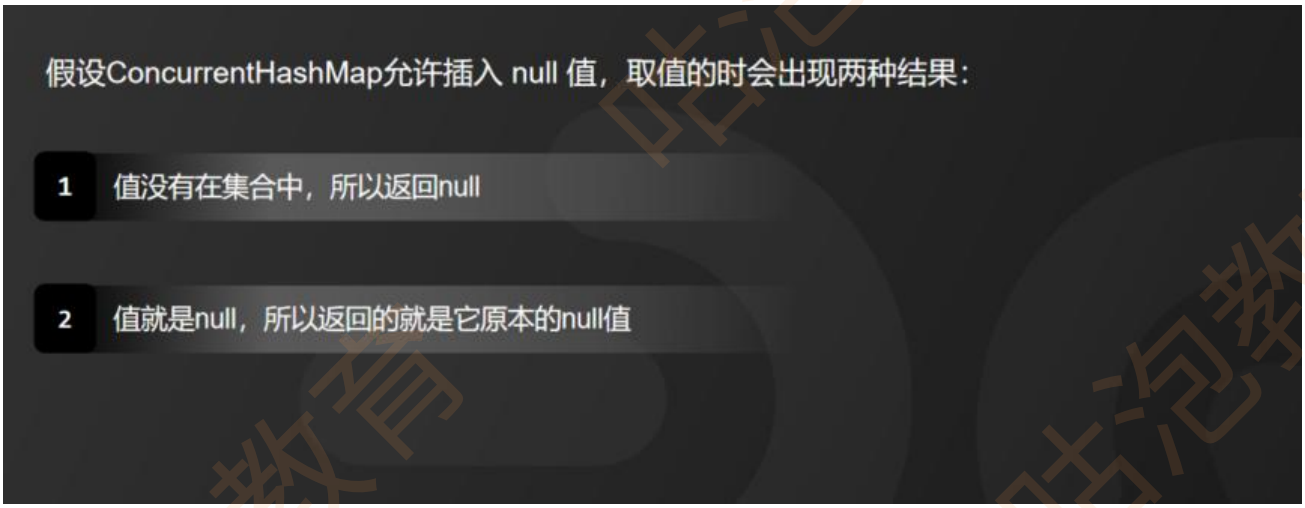
【导航条：歧义问题】

（转场，屏幕黑底白字，正中央出现“2 歧义问题”）

2、歧义问题

因为给ConcurrentHashMap中插入 null （空）值会存在歧义。我们可以假设

ConcurrentHashMap允许插入 null（空） 值，那么，我们取值的时候会出现两种结果：



1、值没有在集合中，所以返回的结果就是 null （空）；

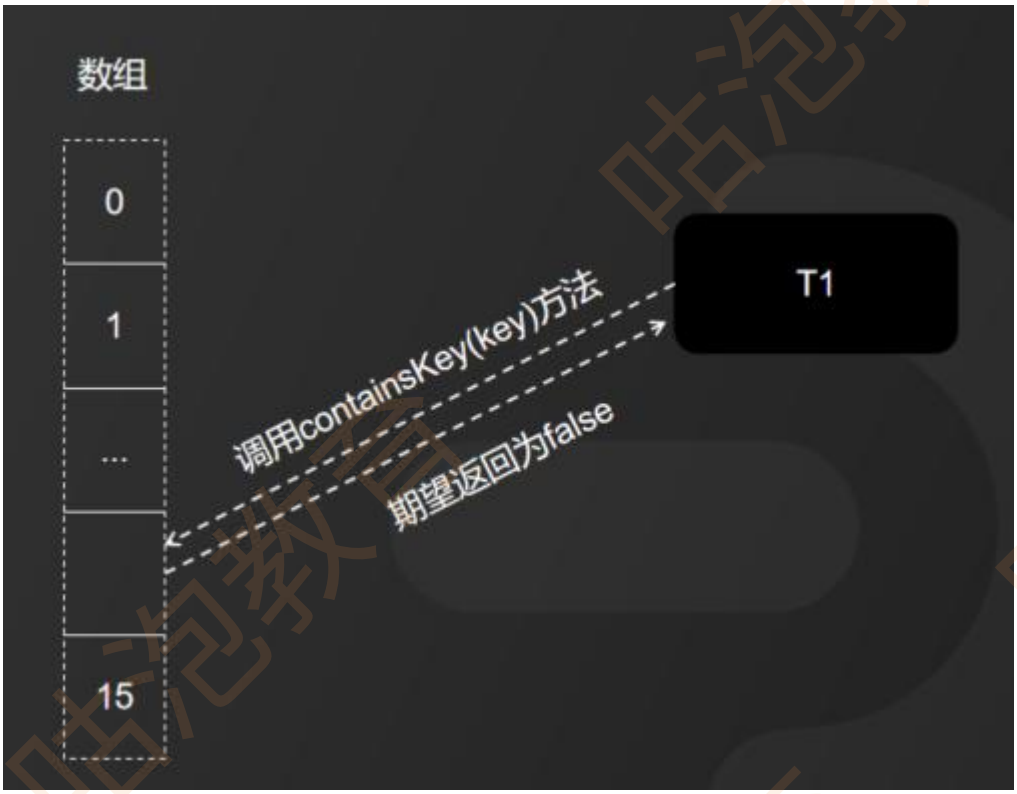
2、值就是 null（空），所以返回的结果就是它原本的 null（空） 值。

这就产生了歧义问题。

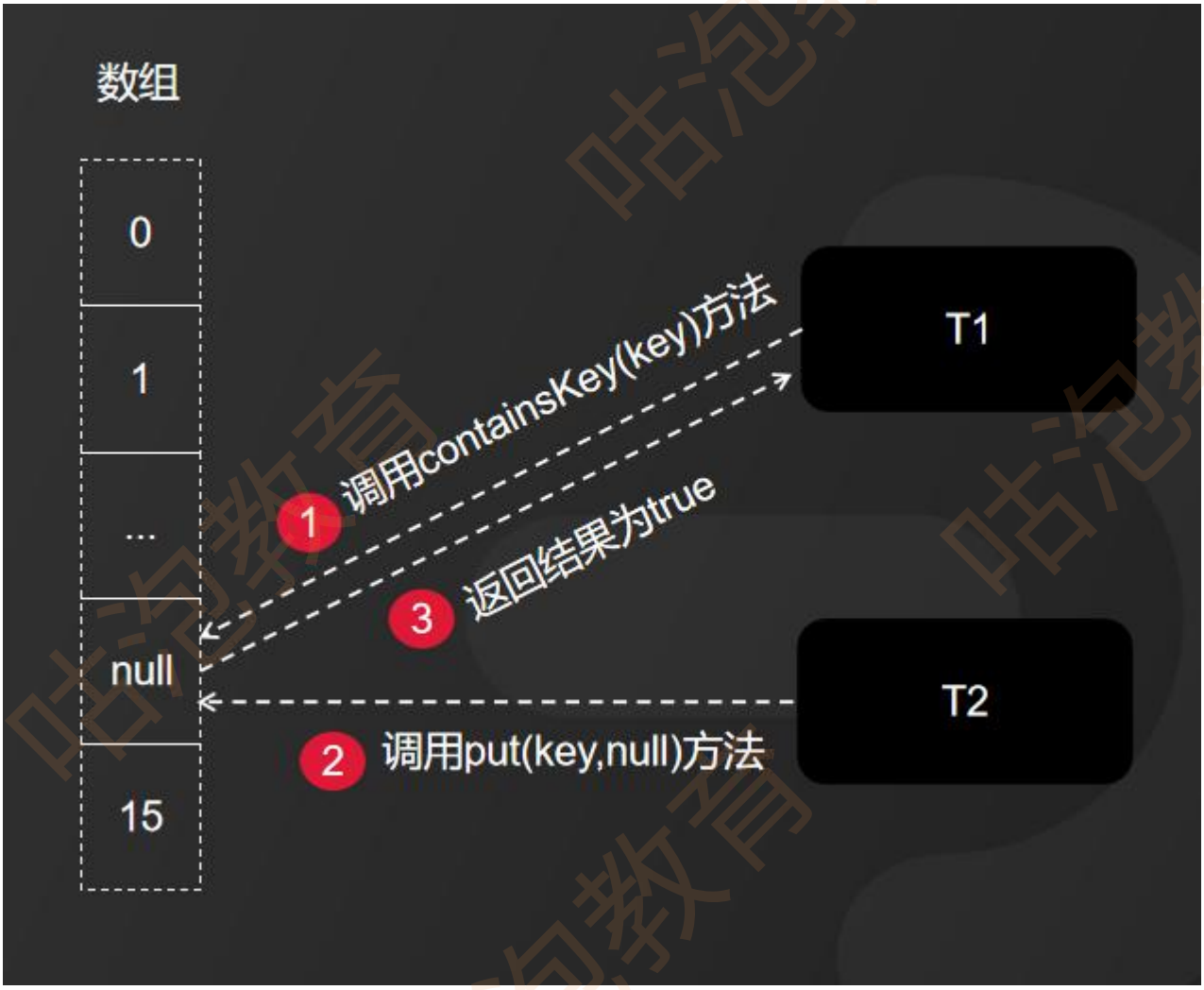
那HashMap允许插入 null（空） 值，难道它就不担心出现歧义吗？这是因为HashMap的设计是给单线程使用的，所以如果取到 null（空） 值，我们可以通过HashMap的 containsKey(key)方 法来区分这个 null（空） 值到底是插入值是 null（空），还是本就没有才返回的 null（空） 值。

而 ConcurrentHashMap 就不一样了，因为 ConcurrentHashMap 是在多线程场景下使用的，它的情况更加复杂。

举个例子，现在有线程T1调用了 ConcurrentHashMap 的 containsKey(key) 方法，我们期望返回的结果是false，也就是说，T1并没有往ConcurrentHashMap中 put null（空）值。



但是，恰恰出了个意外，在线程T1还没有得到返回结果之前，线程T2又调用了ConcurrentHashMap 的 put() 方法，插入了一个Key，并且存入的Value是 null（空） 值。那么，线程T1 最终得到的返回结果就变成 true 了。



显然，这个结果和我们之前期望的 false 完全不一致。

也就是说，在多线程的复杂情况下，我们多线程的复杂情况下，到底是插入的 null（空） 值，还是本就没有才返回的 null（空） 值。也就是说，产生的歧义不能被 证 伪，

3、作者回复

对于 ConcurrentHashMap 不允许插入 null 值的问题，有人问过 ConcurrentHashMap 的作者 Doug Lea，以下是他回复的邮件内容：

The main reason that nulls aren't allowed in ConcurrentMaps (ConcurrentHashMaps, ConcurrentSkipListMaps) is that ambiguities that may be just barely tolerable in non-concurrent maps can't be accommodated. The main one is that if map.get(key) returns null, you can't detect whether the key explicitly maps to null vs the key isn't mapped.

In a non-concurrent map, you can check this via map.contains(key),but in a concurrent one, the map might have changed between calls.

Further digressing: I personally think that allowing
nulls in Maps (also Sets) is an open invitation for programs
to contain errors that remain undetected until
they break at just the wrong time. (Whether to allow nulls even
in non-concurrent Maps/Sets is one of the few design issues surrounding
Collections that Josh Bloch and I have long disagreed about.)

It is very difficult to check for null keys and values
in my entire application .
Would it be easier to declare somewhere
static final Object NULL = new Object();
and replace all use of nulls in uses of maps with NULL?

-Doug

以上信件的主要意思是，Doug Lea 认为这样设计最主要的原因是：不容忍在并发场景下出现歧义！

【导航条：结论】

（转场，屏幕黑底白字，正中央出现“3 结论”）

4、总结

ConcurrentHashMap在源码中加入不允许插入 null （空） 值的设计，主要目的是为了防止并发场景下的歧义问题。

以上就是我对关于ConcurrentHashMap为什么不允许插入 null （空） 值的解答，听懂的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺Tom，如果还有其他疑问或者需要高清无码图的小伙伴，可以关注我的主页加V。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

17．ThreadLocal真的会造成内存泄漏吗?

【导航条：引言】

ThreadLoca在并发场景中，应用非常多。前几天有位小伙伴问我一个问题，说ThreadLocal是不是真的会造成内存泄漏？今天给大家做一个分享，个人见解，仅供参考。如果大家有其他见解可以在评论区讨论。

【导航条：基本原理】

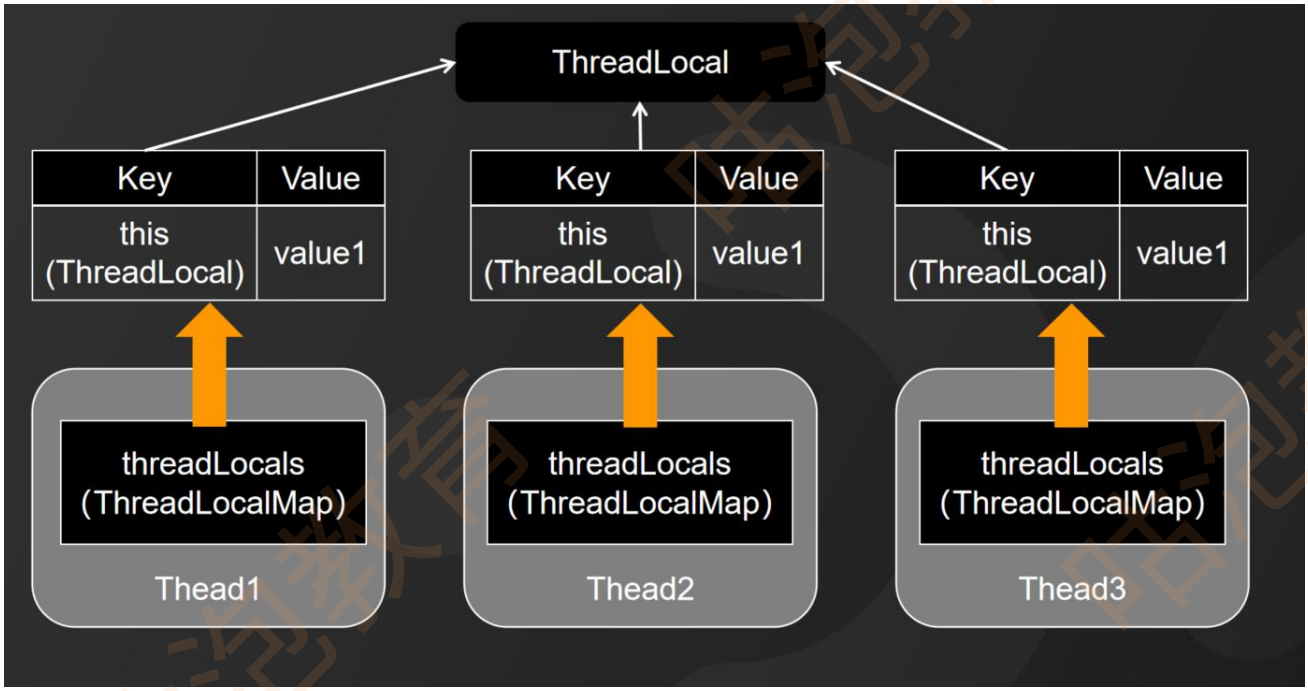
（转场，屏幕黑底白字，正中央出现“1 ThreadLocal的基本原理”）

1、ThreadLocal的基本原理

考虑到很多小伙伴可能还不太了解ThreadLocal，我先简单介绍一下ThreadLocal。在多线程并发访问同一个共享变量的情况下，如果不做同步控制的话，就可能会导致数据不一致的问题，所以，我们需要使用synchronized加锁来解决。

而ThreadLocal换了一个思路来处理多线程的情况，

（开始播放动画）



ThreadLocal本身并不存储数据，它使用了线程中的threadLocals属性，threadLocals的类型就是在ThreadLocal中的定义的ThreadLocalMap对象，当调用ThreadLocal的set(T value)方法时，ThreadLocal将自身的引用也就是this作为Key，然后，把用户传入的值作为Value存储到线程的ThreadLocalMap中，这就相当于每个线程的读写操作都是基于线程自身的一个私有副本，线程之间的数据是相互隔离的，互不影响。

（结束播放动画）

这样一来基于ThreadLocal的操作也就不存在线程安全问题了。它相当于采用了用空间来换时间的思路，从而提高程序的执行效率。

【导航条：四种对象引用】

（转场，屏幕黑底白字，正中央出现“2 四种对象引用”）

2、四种对象引用

在ThreadLocalMap内部，维护了一个Entry数组table的属性，用来存储键值对的映射关系，来看这样一段代码片段：

（开始播放动画）

```
static class ThreadLocalMap {
    ...
    private Entry[] table;

    static class Entry implements WeakReference<ThreadLocal<?>> {
        Object value;
        Entry(ThreadLocal<?> k, Object v) {
            super(k);
            value = v;
        }
    }
    ...
}
```

static class ThreadLocalMap {

...

private Entry[] table;

static class Entry implements WeakReference<ThreadLocal<?>> {

Object value;

Entry(ThreadLocal<?> k, Object v) {

super(k);

value = v;

}

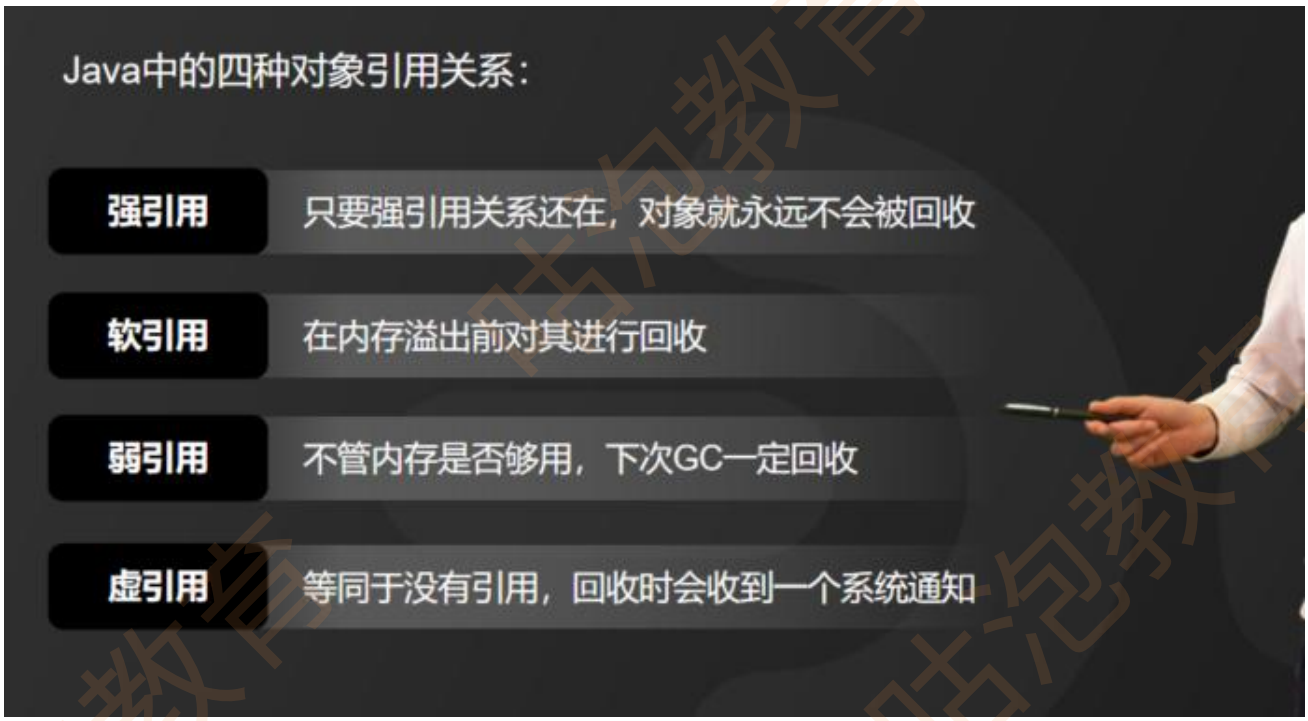
```
    }

    ...

}
```

(结束播放动画)

Entry将ThreadLocal作为Key，值作为Value保存，它继承自WeakReference，注意构造函数里的第一行代码super(k)，这意味着ThreadLocal对象是一个「弱引用」。有的小伙伴可能对「弱引用」不太熟悉，这里再介绍一下Java的四种引用关系。

在JDK1.2之后，Java对引用的概念做了一些扩充，将引用分为“强”、“软”、“弱”、“虚”四种，由强到弱依次为：

(开始播放动画)



强引用：指代码中普遍存在的赋值行为，如：Object o = new Object()，只要强引用关系还在，对象就永远不会被回收。

软引用：还有用处，但不是必须存活的对象，JVM会在内存溢出前对其进行回收，例如：缓存。

弱引用：非必须存活的对象，引用关系比软引用还弱，不管内存是否够用，下次GC一定回收。

虚引用：也称“幽灵引用”、“幻影引用”，最弱的引用关系，完全不影响对象的回收，等同于没有

引用，虚引用的唯一的目的是对象被回收时会收到一个系统通知。

（结束播放动画）

这个描述还是比较官方的，简单总结一下，大家应该都追过剧，强引用就好比是男主角，怎么都死不了。软引用就像女主角，虽有一段经历，还是没走到最后。弱引用就是男二号，注定用来牺牲的。虚引用就是路人甲了。

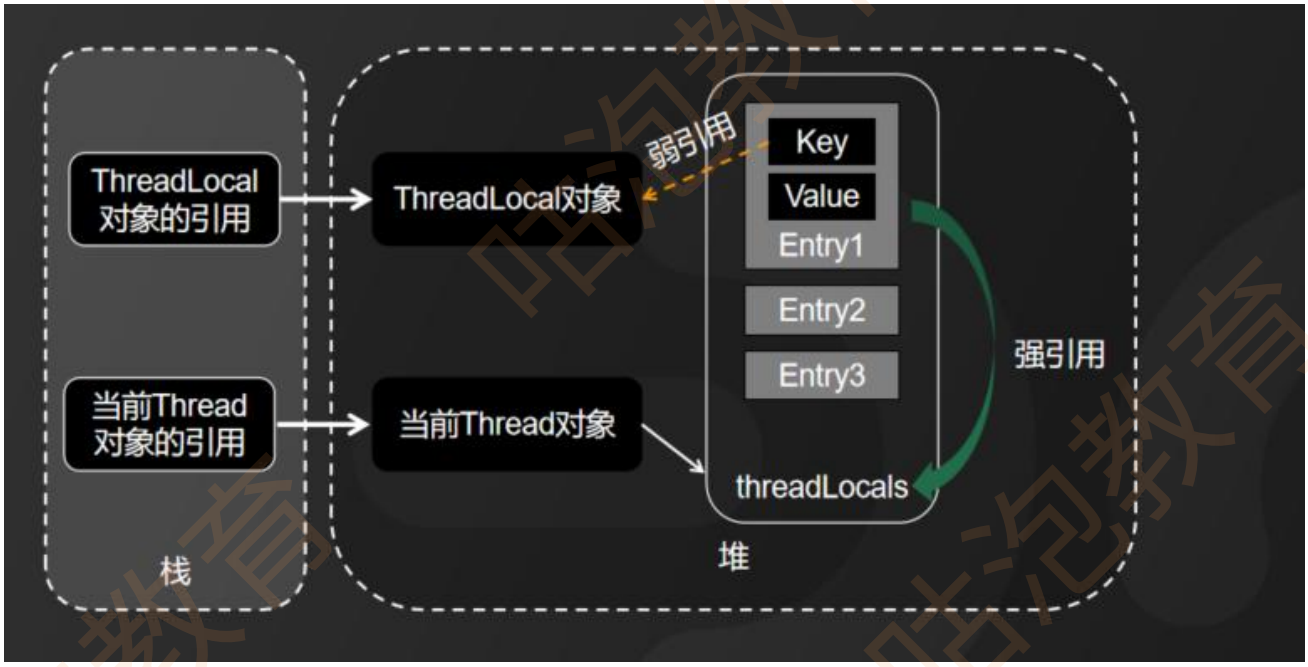
【导航条：内存泄漏原因】

（转场，屏幕黑底白字，正中央出现“3 造成内存泄漏的原因”）

3、造成内存泄漏的原因

内存泄漏和ThreadLocalMap中定义的Entry类有非常大的关系。

（开始展示动画）



这个动画完整地展示了ThreadLocal中对象引用的关系，需要这张高清图的小伙伴可以在评论区留言。

由于ThreadLocal对象是弱引用，如果外部没有强引用指向它，它就会被GC回收，导致Entry的Key为空（null），如果这时Value外部也没有强引用指向它，那么Value就永远也访问不到了，按理也应该被GC回收，但是由于Entry对象还在强引用Value，导致Value无法被回收，这时「内存泄漏」就发生了，

Value成了一个永远也无法被访问，但是又无法被回收的对象。

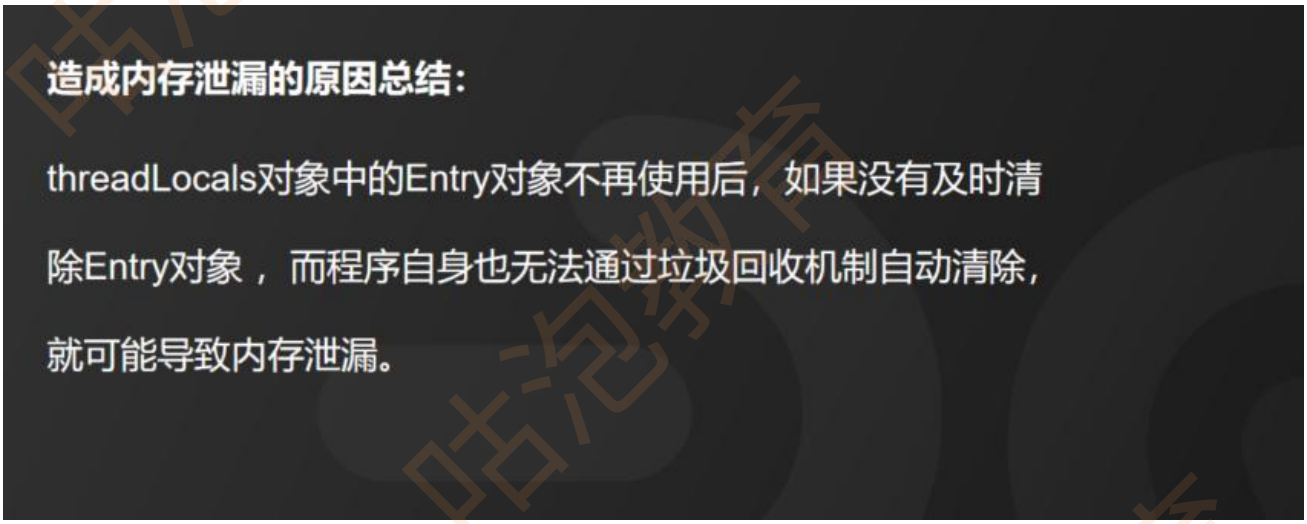
（结束展示动画）

Entry对象属于ThreadLocalMap，ThreadLocalMap又属于Thread，如果线程本身的生命周期很短，短时间内就会被销毁，那么「内存泄漏」立刻就会得到解决，只要线程被销毁，Value也会随之被回收。

问题是，线程本身是非常珍贵的计算机资源，很少会去频繁的创建和销毁，一般都是通过线程池来使用，这就将线程的生命周期大大拉长，「内存泄漏」的影响也会越来越大。

最后，一句话总结一下。

（开始展示动画）



threadLocals对象中的Entry对象不再使用后，如果没有及时清除Entry对象，而程序自身也无法通过垃圾回收机制自动清除，就可能导致内存泄漏。

（结束展示动画）

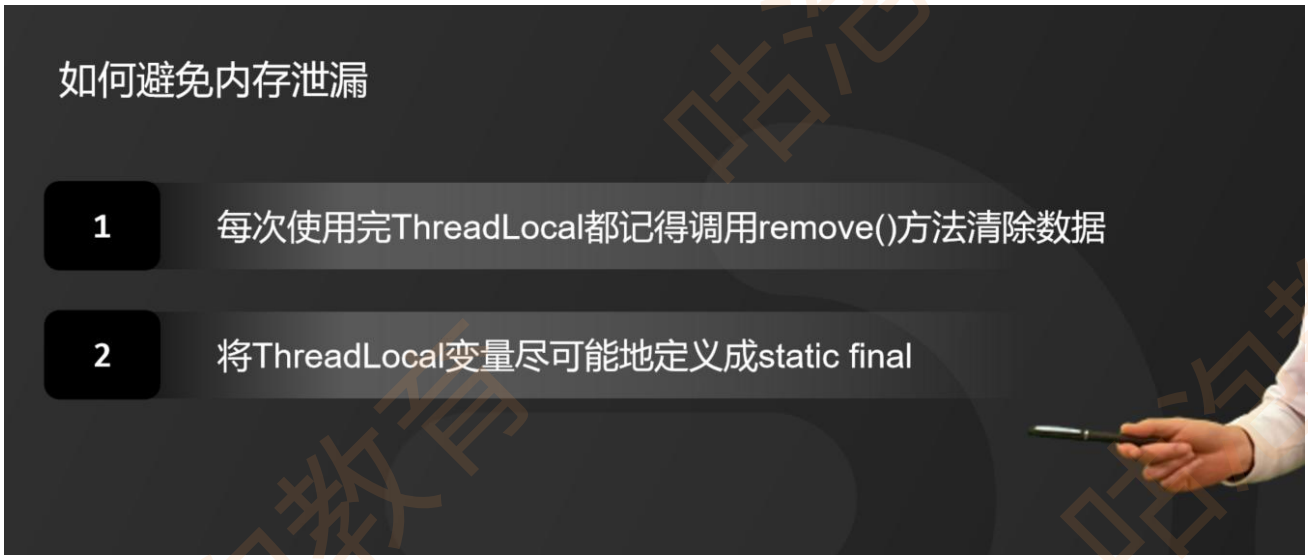
【导航条：如何避免】

（转场，屏幕黑底白字，正中央出现“4 如何避免内存泄漏？”）

4、如何避免内存泄漏？

不要听到「内存泄漏」就不敢使用ThreadLocal,只要规范化使用是不会有问题的。我给大家支几个招：

(开始展示动画)



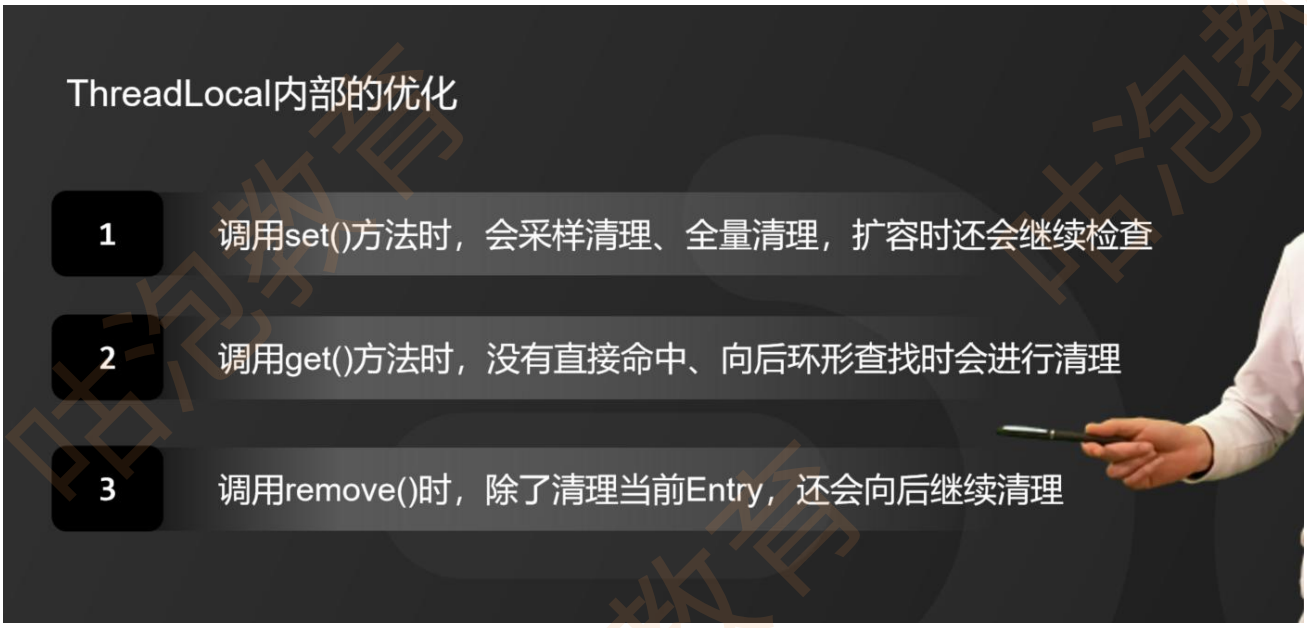
1、每次使用完ThreadLocal都记得调用remove()方法清除数据。

2、将ThreadLocal变量尽可能地定义成static final，避免频繁创建ThreadLocal实例。这样也就保证程序一直存在ThreadLocal的强引用，也能保证任何时候都能通过ThreadLocal的弱引用访问到Entry的Value值，进而清除掉。

（结束展示动画）

当然，就是使用不规范，ThreadLocal内部也做了一些优化，比如：

（开始展示动画）



1、调用set()方法时，ThreadLocal会进行采样清理、全量清理，扩容时还会继续检查。

2、调用get()方法时，如果没有直接命中或者向后环形查找时也会进行清理。

3、调用remove()时，除了清理当前Entry，还会向后继续清理。

所以，我真的希望小伙伴们，不要被制造焦虑的面试官们卷得太深。听懂的小伙伴，请关注点个赞，下次不迷路。

（结束展示动画）

我是被编程耽误的文艺Tom，如果你还有其他不懂的面试题或者需要视频配套的文字资料，可以关注我的主页加V。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

18．Spring Boot自动装配原理

【导航条：引言】

昨天，有位在广州工作4年的小伙伴，在面试中被问到SpringBoot自动装配原理，当时，自我感觉比较好，他要的是30K，但是都没有拿到Offer。今天，我给大家分享一下我的理解。

【导航条：自动装配是什么】

（转场，屏幕黑底白字，正中央出现“1 Spring Boot自动装配是什么？”）

1、Spring Boot自动装配是什么?

SpringBoot自动装配主要是基于注解编程 和 约定优于配置的思想来设计的。

（开始展示动画）

自动装配就是由Spring自动把其他组件中的Bean装载到IoC容器中，不需要开发人员再去配置文件中添加大量的配置。我们只需要在Spring Boot的启动类上添加 @SpringBootApplication 注解，开启自动装配。

这种自动装配的思想，在Spring 3.x以后就开始支持，我们只要在类上添加 @Enable 注解就可以了，只是没有像Spring Boot这样全面地去设计。

因此，Spring 和 Spring Boot的最大区别就是在于Spring Boot的自动装配。那自动装配的原理又是什么呢?

（结束展示动画）

【导航条：自动装配原理】

（转场，屏幕黑底白字，正中央出现“2 自动装配原理”）

2、自动装配原理

@SpringBootApplication这个注解是暴露给用户使用的入口，它的底层是由@EnableAutoConfiguration这个注解来实现的。

这样一个自动装配的实现，我把它归纳为以下三个核心步骤：

（开始展示动画）

第一步：启动依赖组件的时候，组件中必须要包含 @Configuration 的配置类，在这个配置类里面声明为@Bean 注解，就将方法的返回值或者属性值 注入到IoC容器中。

第二步：如果是使用第三方jar包，Spring Boot采用SPI机制，只需要在/META-INF/目录下增加spring.factories配置文件。然后，Spring Boot会根据约定规则，自动使用SpringFactoriesLoader来加载配置文件中的内容。

第三步：Spring获取到第三方jar中的配置以后，会使用调用ImportSelector接口来完成动态加载。

（开始展示动画，固定上图展示结果）

这样的设计的好处在于，大幅减少了臃肿的配置文件，而且各模块之间的依赖实现了深度解耦。比如，我们使用Spring创建Web程序时需要导入非常多的Maven依赖，而Spring Boot只需要一个Maven依赖来创建Web程序，并且Spring Boot还把我们最常用的依赖都放到了一起，我们只需要 引入 spring-boot-starter-web 这一个依赖就可以完成一个简单的Web应用。

以前用Spring的时候需要XML文件配置开启一些功能，现在Spring Boot不用XML配置了，只需要写一个，加了@Configuration注解 或者实 现对应接口的配置类就可以了。

【导航条： 】

（转场，屏幕黑底白字，正中央出现“3 总结”）

3、总结

最后，我总结一下，Spring Boot自动装配是Spring的完善和扩展，就是为我们便捷开发，方便测试和部署，提高效率而诞生的框架技术。

小伙伴们，如果你被问到过Spring Boot自动装配原理的问题，你是怎么回答的呢？可以在评论区分享你的回答。

我是被编程耽误的文艺Tom，如果你还有其他不懂的面试题或者需要视频配套的文字资料，可以关注我的主页加V。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

19．哪些情况下的单例对象可能会破坏?

【导航条：引言】

昨天，有位小伙伴在评论区留言，希望我分享一些设计模式相关的面试题。设计模式本身是很抽象的，但是在很多面试中又经常被问到，很多小伙伴其实都能答得上，但是又不知道怎么样回答才能让面试官满意，往往越简单的知识越能够体现出核心竞争力。

今天，我给大家分享一个简单又不简单的单例模式，希望能够帮助到大家。先来看单例模式的定义。

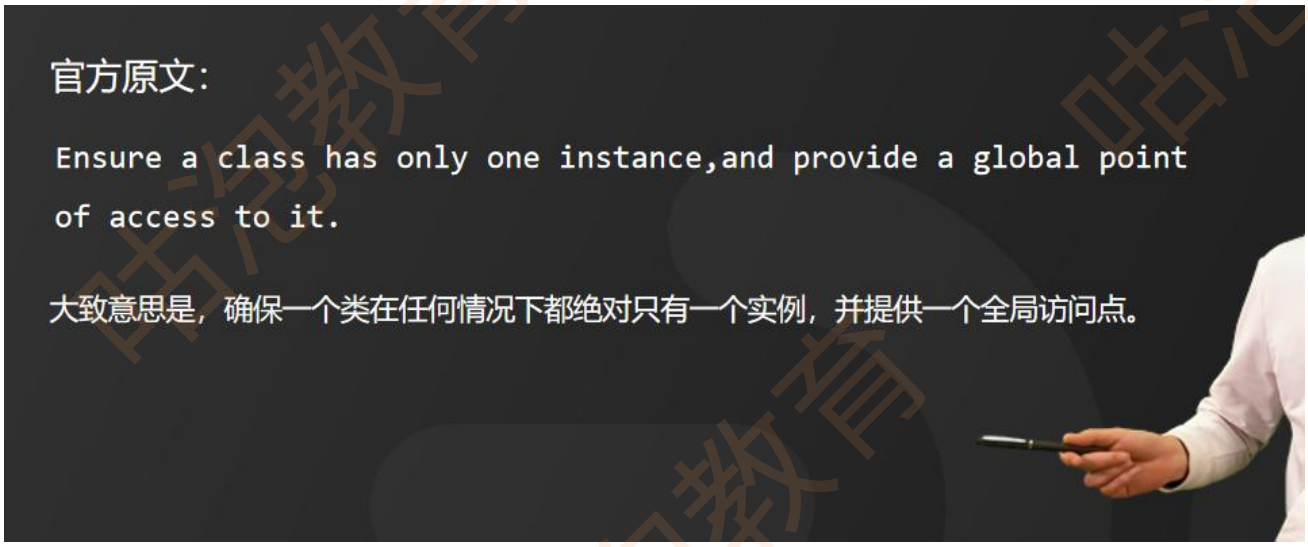
【导航条：单例模式的定义】

（转场，屏幕黑底白字，正中央出现“1 单例的定义”）

1、单例模式的定义

关于单例模式的定义，官方原文是这样描述的：

（开始展示动画）



Ensure a class has only one instance,and provide a global point of access to it.

大致意思是，确保一个类在任何情况下都绝对只有一个实例，并提供一个全局访问点。

（结束展示动画）

单例模式的写法相信只要是程序员应该都会，也很非常简单，这里我就不一一列举了。今天，我要重点要给大家分析的是，在Java中，哪些单例对象是最有可能被破坏的。

【导航条：破坏单例的五个场景】

（转场，屏幕黑底白字，正中央出现“2 破坏单例的五个场景”）

2、单例被破坏的五个场景

（开始展示动画）

我把可能出现单例被破坏的情况，一共归纳为五种，分别为多线程破坏单例、指令重排破坏单例、克隆破坏单例、反序列化破坏单例、反射破坏单例。

(结束展示动画)

下面我详细分析一下每种情况并给出解决方案：

第一种：多线程破坏单例

(开始展示动画)

在多线程环境下，线程的时间片是由CPU自由分配的，具有随机性，而单例对象作为共享资源可能会同时被多个线程同时操作，从而导致同时创建多个对象。当然，这种情况只出现在懒汉式单例中。如果是饿汉式单例，在线程启动前就被初始化了，不存在线程再创建对象的情况。

如果懒汉式单例出现多线程破坏的情况，我给出以下两种解决方案：

1、改为DCL双重检查锁的写法。

2、使用静态内部类的写法，性能更高。

（结束展示动画）

第二种：指令重排破坏单例

（开始展示动画）

指令重排也可能导致懒汉式单例被破坏。来看这样一句代码：

instance = new Singleton();

看似简单的一段赋值语句：instance = new Singleton();

其实JVM内部已经被转换为多条执行指令：

memory = allocate(); 分配对象的内存空间指令

ctorInstance(memory); 初始化对象

instance = memory; 将已分配存地址赋值给对象引用

1、分配对象的内存空间指令，调用allocate()方法分配内存。

2、调用ctorInstance()方法初始化对象

3、将已分配存地址赋值给对象引用

但是经过重排序后，执行顺序可能是这样的：

memory = allocate(); 分配对象的内存空间指令

instance = memory; 将已分配存地址赋值给对象引用

ctorInstance(memory); 初始化对象

1、分配对象的内存空间指令

2、设置instance指向刚分配的内存地址

3、初始化对象

我们可以看到指令重排之后，instance指向分配好的内存放在了前面，而这段内存的初始化的指令被排在了后面，在线程 T1 初始化完成这段内存之前，线程T2 虽然进不去同步代码块，但是在同步代码块之前的判断就会发现 instance 不为空，此时线程T2 获得 instance 对象，如果直接使用就可能发生错误。

如果出现这种情况，我该如何解决呢？只需要在成员变量前加volatile，保证所有线程的可见性就可以了。

```
private static volatile Singleton instance = null;
```

（结束展示动画）

第三种：克隆破坏单例

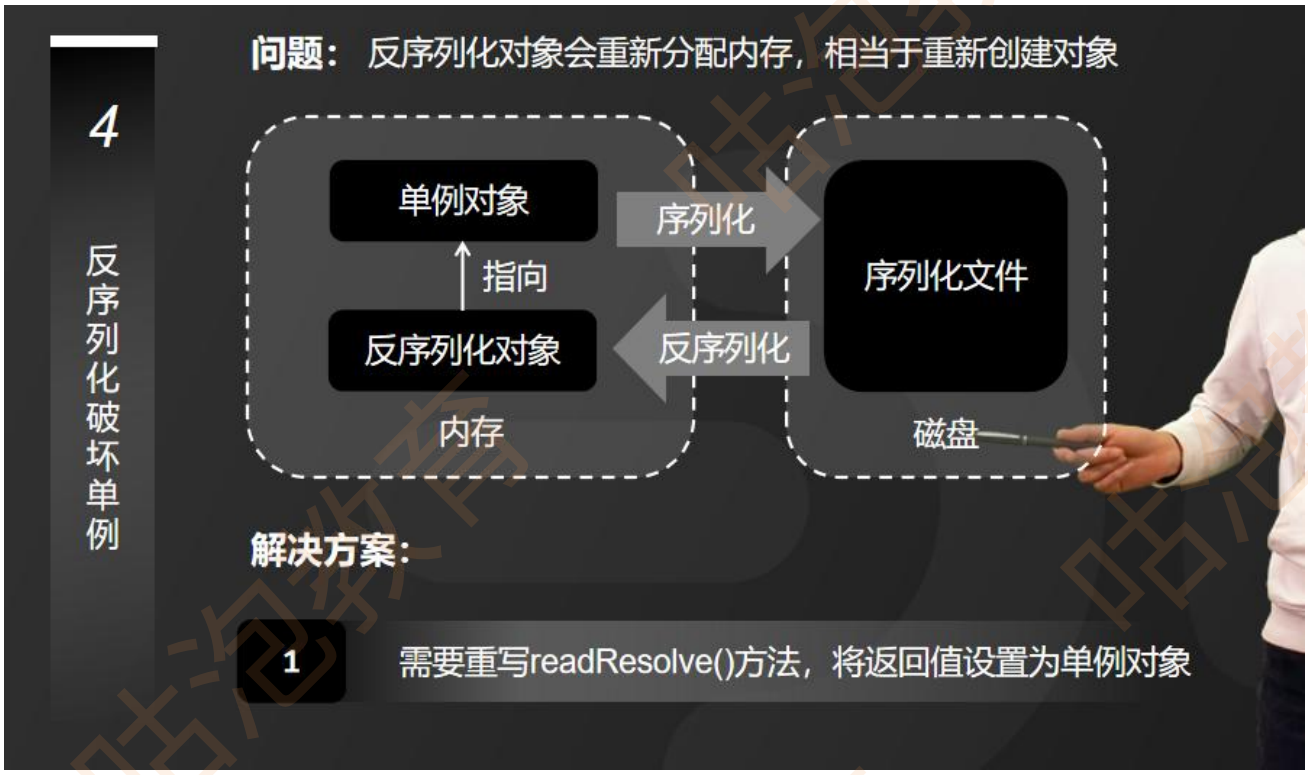
（开始展示动画）

在Java中，所有的类就继承自Object，也就是说所有的类都实现了clone()方法。如果是深clone()，每次都会重新创建新的实例。那如果我们定义的是单例对象，岂不是也可调用clone()方法来反复创建新的实例呢？确实，这种情况是有可能发生的。为了避免发生这样结果，我们可以在单例对象中重写clone()方法，将单例自身的引用作为返回值。这样，就能避免这种情况发生。

（结束展示动画）

第四种：反序列化破坏单例

（开始展示动画）



我们将Java对象序列化以后，对象通常会被持久化到磁盘或者数据库。如果我们要再次加载到内存，就需要将持久化的内容反序列化成Java对象。反序列化是基于字节码来操作的，我们要序列化以前的内容进行反序列化到内存，就需要重新分配内存，也就是说，要重新创建对象。那如果要反序列化的对象恰恰是单例对象，我们该怎么办呢?

我告诉大家一种解决方案，在反序列的过程中，Java API会调用readResolve()方法，可以通过获取readResolve()方法的返回值覆盖反序列化创建的对象。

（反序列化对象 指向 单例对象动画 出现 ）

因此，只需要重写readResolve()方法，将返回值设置为已经存在的单例对象，就可以保证反序列化以后的对象是同一个了。之后再将反序列化后的对象中的值，克隆到单例对象中。

（结束展示动画）

第五种：反射破坏单例

（开始展示动画）

以上讲的所有单例情况都有可能被反射破坏。因为Java中的反射机制是可以拿到对象的私有的构造方法，也就是说，反射可以任意调用私有构造方法创建单例对象。当然，没有人会故意这样做，但是如果出现意外的情况，该如何处理呢？我推荐大家两种解决方案，

第一种方案是在所有的构造方法中第一行代码进行判断，检查单例对象是否已经被创建，如果已经被创建，则抛出异常。这样，构造方法将会被终止调用，也就无法创建新的实例。

第二种方案，将单例的实现方式改为枚举式单例，因为在JDK源码层面规定了，不允许反射访问枚举。

（结束展示动画）

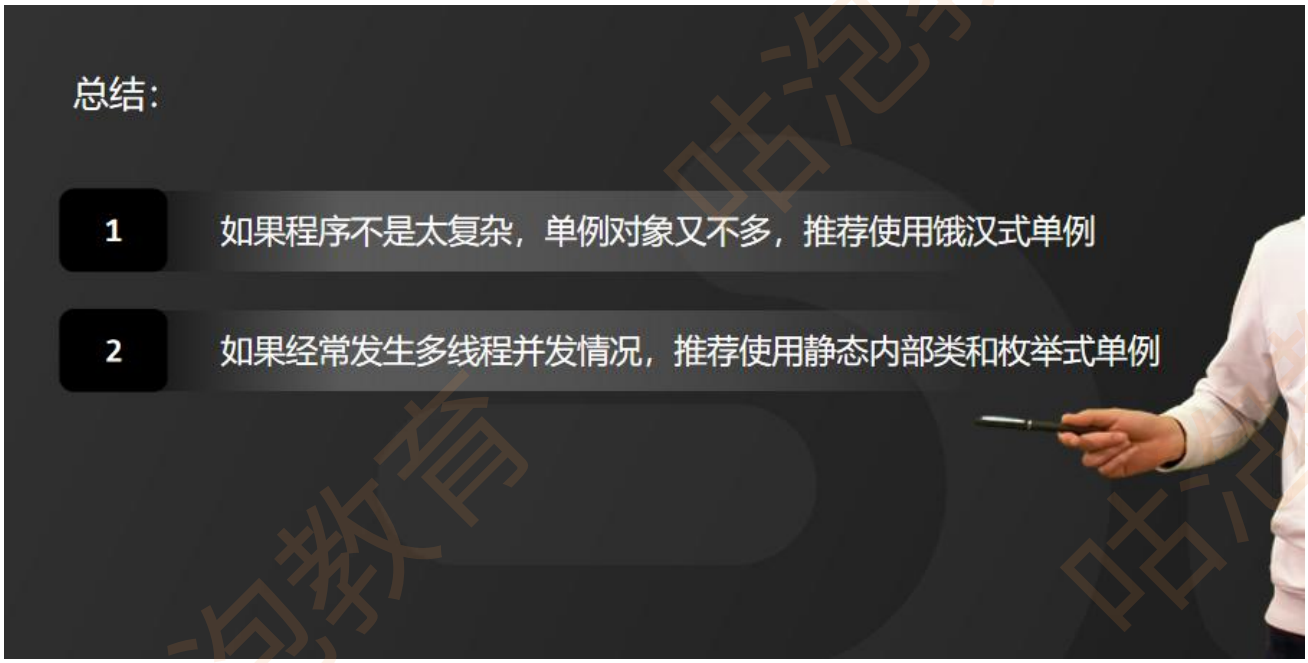
【导航条：总结】

（转场，屏幕黑底白字，正中央出现“3 总结”）

3、总结

最后总结一下：

（开始展示动画）



1、在所有单例写法中，如果程序不是太复杂，单例对象又不多，推荐使用饿汉式单例。

2、但如果经常发生多线程并发情况下，推荐使用静态内部类和枚举式单例，我的《设计模式就该这样学》这本书中，也推荐这样的写法。

(结束展示动画)

听懂的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺Tom，如果你还有其他不懂的面试题或者需要视频配套的文字资料，可以关注我的主页加V。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

20．责任链模式的实现原理

【导航条：引言】

责任链模式应用场景非常多、比如拦截器、过滤器等等。但是要彻底理解责任链的实现原理还是有一定难度的，因此，责任链模式的实现原理也就成为了一道互联网大厂的高频面试题。今天，我给小伙伴们来详细地掰一掰，保证让你彻底搞明白，不服来战。

【导航条：责任链模式的定义】

（转场，屏幕黑底白字，正中央出现“1 责任链模式的定义”）

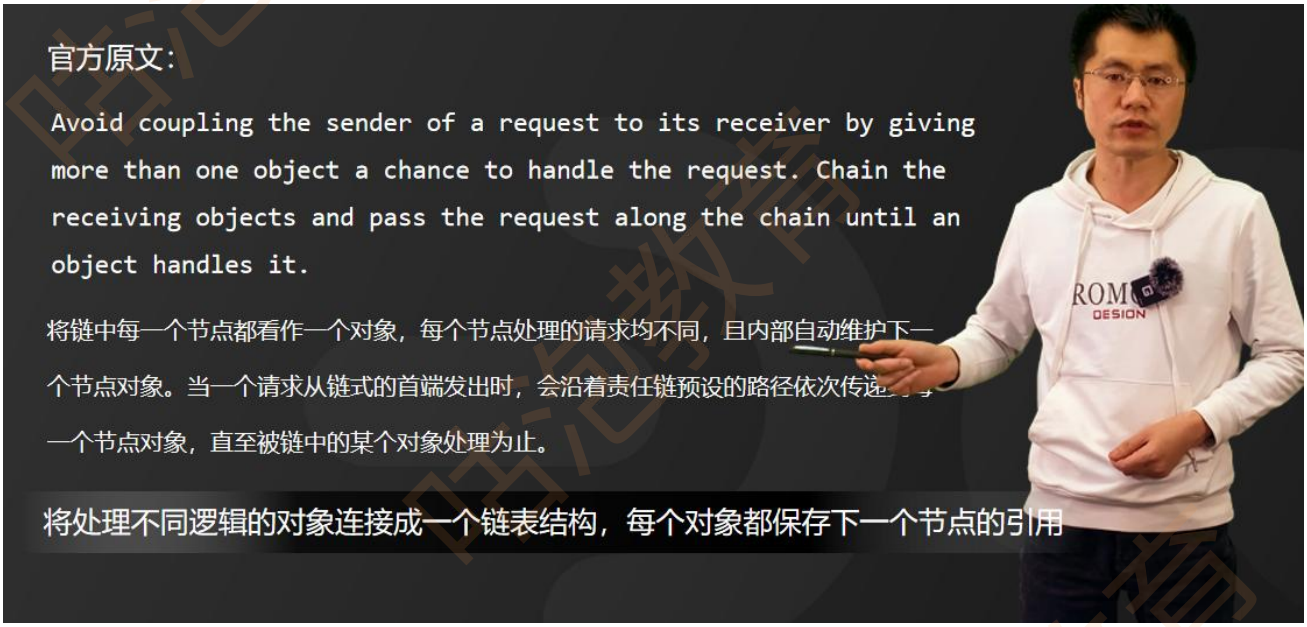
1、责任链模式的定义

关于责任链模式的定义，官方原文是这样描述的：

Avoid coupling the sender of a request to its receiver by giving more than one object a chance to handle the request. Chain the receiving objects and pass the request along the chain until an object handles it.

翻译过来就是：

将链中每一个节点都看作一个对象，每个节点处理的请求均不同，且内部自动维护下一个节点对象。当一个请求从 的首端发出时，会沿着责任链预设的路径依次传递到每一个节点对象，直至被链中的某个对象处理为止。



简答一句话总结就是：将处理不同逻辑的对象连接成一个链表结构，每个对象都保存下一个节点的引用。

我们在平时处理工作中的时候，往往需要各部门协同合作来完成某一个任务。因为，每个部门都有各自的职责，所以，很多时候事情完成一半，就要会转交到下一个部门，甚至还要盖章，直到所有部门都审批通过，事情才能完成。这就相当于将皮球层层往上踢。

还记得小时候追过的一个电视剧，80后应该都看过，有位太极高手闯七层宝塔，每一层都住了一位武林高手，层层往上“过五关，斩六将”,打赢才能往上一层，打输了就踢回去，最后登顶.当时看得是热血沸腾，这也是责任链的一种场景。不过忘记电视剧叫什么名字了，小伙伴们可以评论区告诉我一下。

所以责任链模式又被戏称为“踢皮球”模式。那责任链模式的实现原理又是怎样的呢?

【导航条：实现原理】

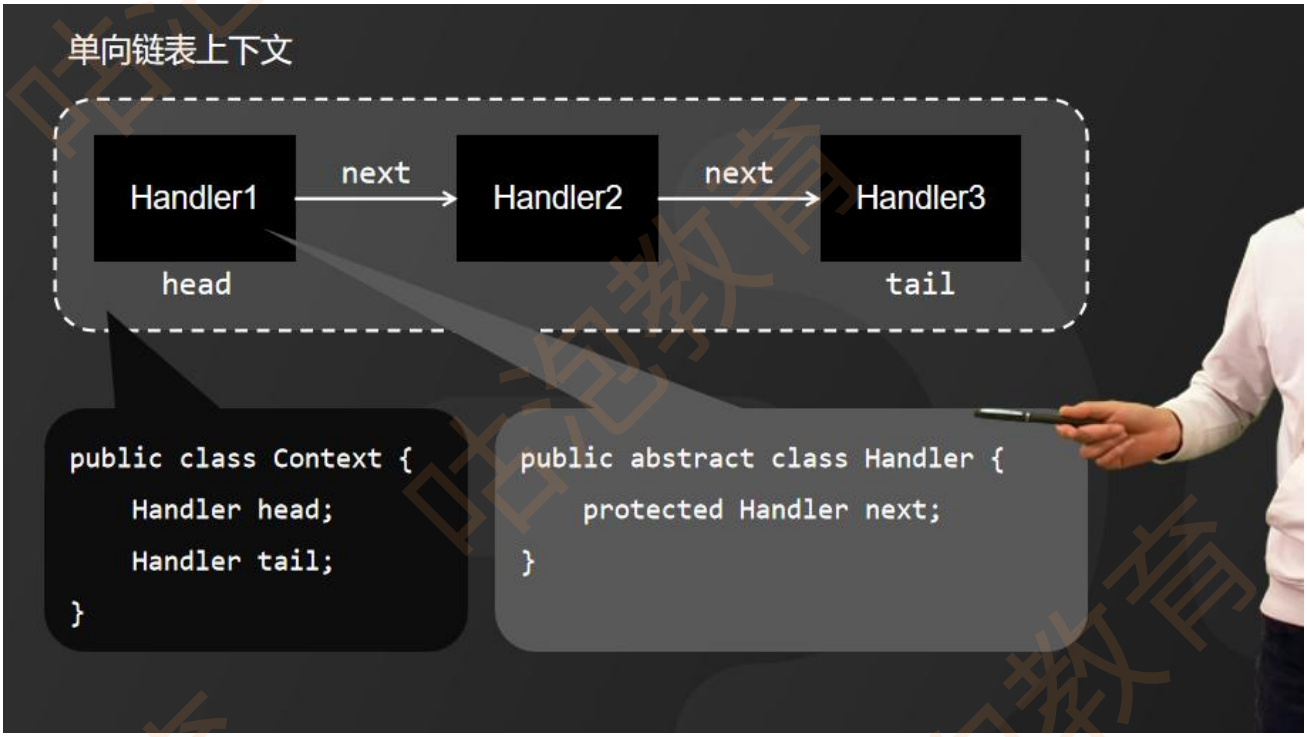
（转场，屏幕黑底白字，正中央出现“2 责任链模式的实现原理”）

2、责任链模式的实现原理

责任链又分为单向责任链和双向责任链，单向责任链比较简单也容易理解，双向责任链相当于是一个执行闭环，较为复杂。

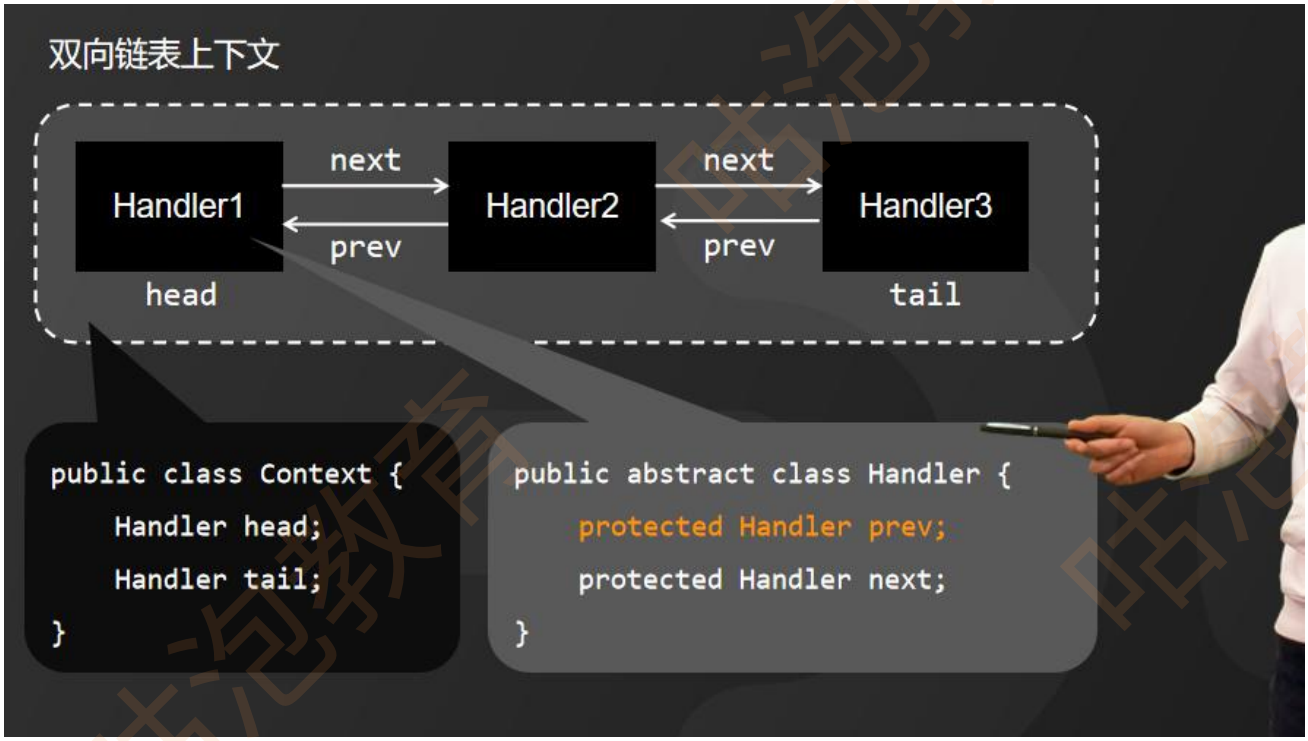
我们先来分析单向责任链。它的结构是这样设计的，首先，设计一个单向链表的上下文Context，保存链表的头（head ）引用 和 尾（tail）引用，Context的代码结构是这样的。

然后，在上下文中加入Handler，也就是处理业务逻辑的节点类，每个Hnandler都保存了下一个执行节点的引用，形成一条完整的执行链路。Handler的通用代码结构是这样的。



J2EE中的Filter过滤器、Spring中的Interceptor拦截器都是采用这样的单向链表的设计。

那双向链表又如何设计呢？它和单向链表的基本结构一致，我们来看，它只是在Handler中增加了对上一个节点的引用。这样责任链就行了一个执行闭环，就好比是环线地铁。来看它的通用代码结构是这样的。



Netty中的Piepline管道就是采用这样的双向链表的设计。

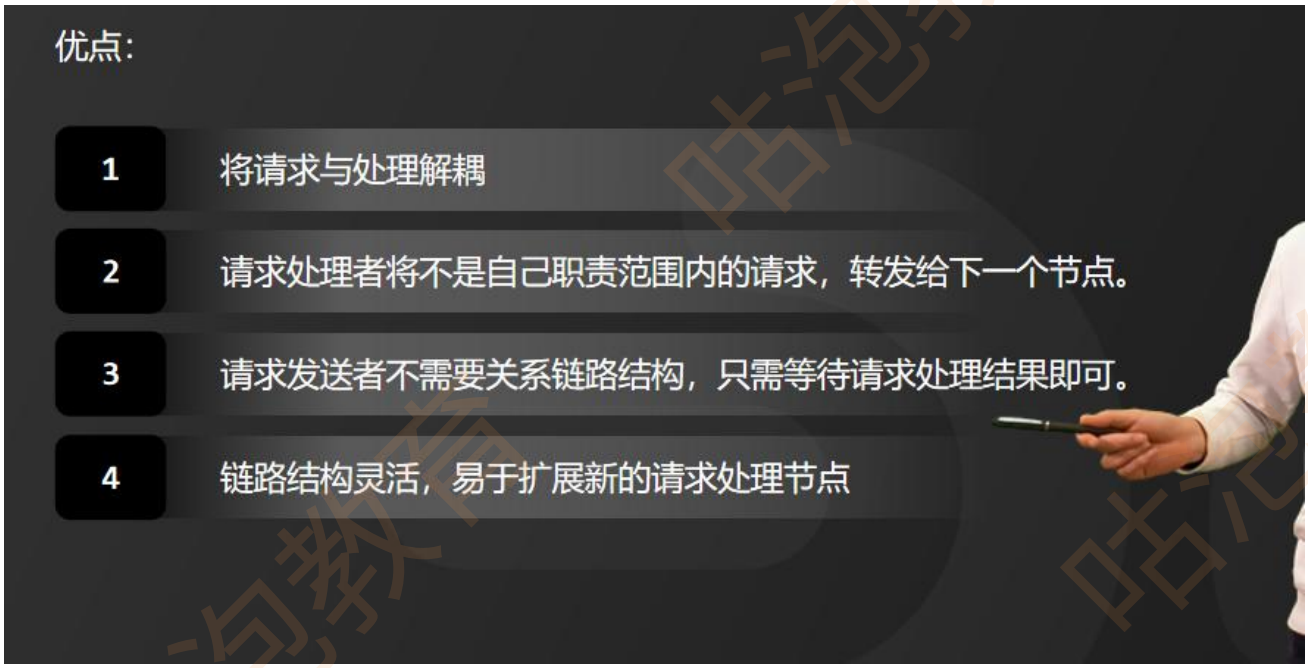
责任链模式一般还会结合建造者模式来使用，实现链式编程。

【导航条：优缺点】

（转场，屏幕黑底白字，正中央出现“3 责任链模式的优缺点”）

3、责任链模式的优缺点

那这样设计又有什么优点和缺点呢？先给大家总结一下它的优点：



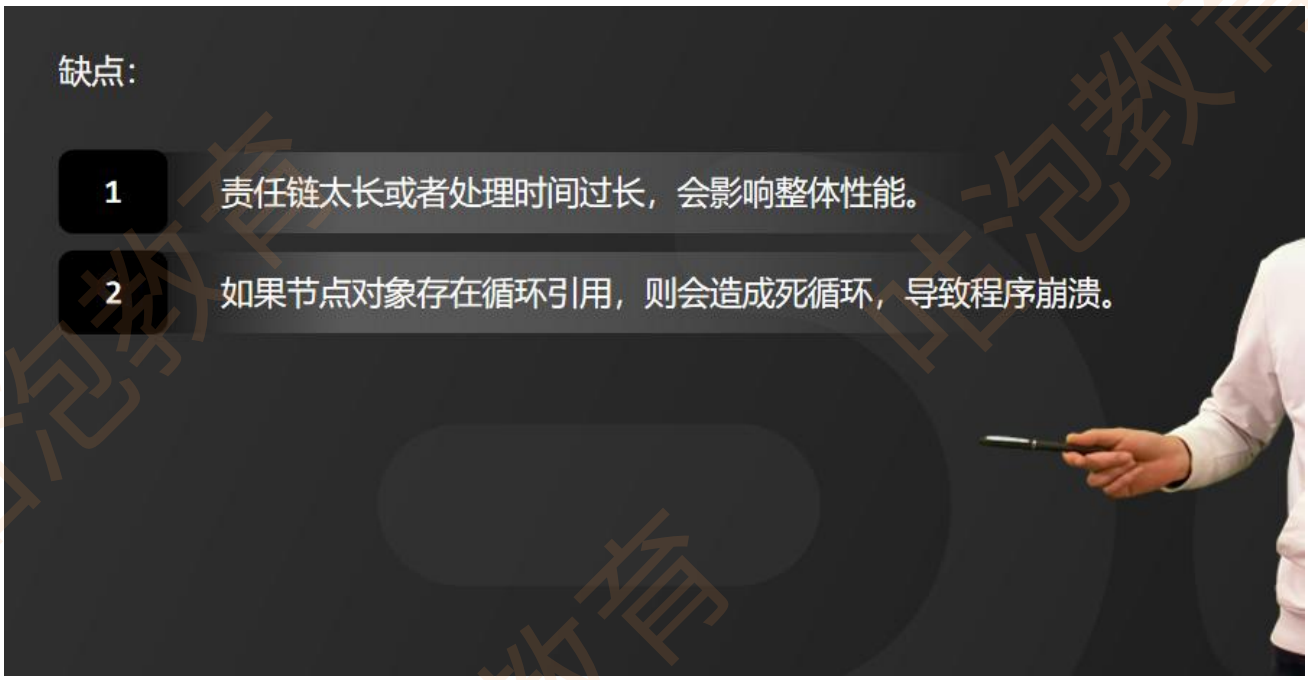
（1）实现了将请求与处理完全解耦。

（2）请求处理者只需关注自己职责范围内的请求进行处理即可，对于不是职责范围内的请求，直接转发给下一个节点对象。

（3）具备链式传递处理请求的功能，请求发送者不需要知晓链路结构，只需等待请求处理结果即可。

（4）链路结构灵活，可以通过改变链路结构动态地新增或删减责任。

那这样设计，会有哪些缺点呢？我也给大家总结一下：



（1）如果责任链路太长或者处理时间过长，会影响程序执行的整体性能。

（2）如果节点对象存在循环引用，有可能会造成死循环，从而导致程序崩溃。

小伙伴们，你还觉得责任链模式的实现原理难吗？听懂的小伙伴，请关注点个赞，下次不迷路。

我是被编程耽误的文艺Tom，如果你还有其他不懂的面试题或者需要视频配套的文字资料，可以关注我的主页加V。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

21．为什么越来越多的人选择Spring Boot?

【导航条：引言】

我们都知道，Spring是一个非常经典的应用框架，与其说是Java开发不如说是Spring开发，为什么现在越来越多的人会选择用Spring Boot呢？。要回答这个问题，还需要从Java Web开发的发展历史开始说起。

【导航条：Web开发简史】

（转场，屏幕黑底白字，正中央出现“1 Java Web开发简史”）

1、Java Web开发简史

自1995年James Gosling（詹姆斯·高斯林）带领团队设计发布Java语言以来，Java一直排在编程语言排行榜的前列，James Gosling也被称为“Java之父”。

在最初发布的Java版本中，包含Java SE、JavaEE、JavaME。

Java SE（Standard Edition）作为标准版本，提供最核心的基础功能，Java EE（Enterprise Edition）作为企业版，主要用于企业级的Web开发，JavaME（***Micro Edition***）作为微型版本主要应用与移动设备的开发。随着Java的语言的广泛应用，Java也找到了自身的优势，Java EE版本被应用得最多。

从1996年开始，JavaEE开发是基于JSP + Java Bean来完成的。

后来慢慢地演变，1997年官方推出了 JSP + Servlet + Java Bean来进行开发，Servlet起到了调度控制的作用，这是MVC设计的雏形。

然后，发展出现了 JSP + Sevlet + Java Bean + Dao的模式，将业务逻辑处理和数据库访问分离，出现了三层架构设计理念。

再后来，1998年前后，为了满足多服务器之间的通信，采用了JSP + Servlet + EJB的形式，出现了RPC设计的雏形。

之后，继续演变，在2006年首次出现 JSF + EJB + JPA，提出前后端完全隔离开发的思想。

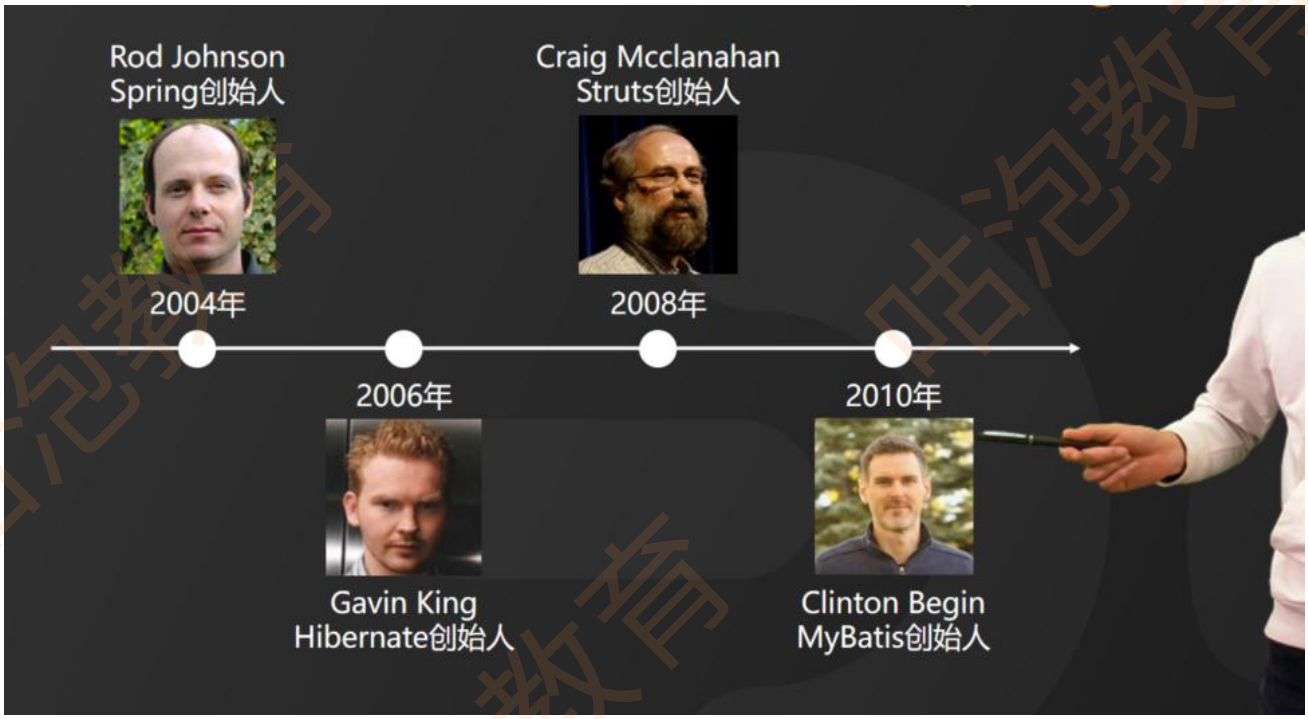
当然，这些技术都是Java官方提供的。好像，一直都不那么受欢迎。尤其是在开源社区发布一些开源的产品用来来代替官方退出的这些技术，主要目的是为了简化开发。于是就出现了许多非常经典的Java开发框架。。

【导航条：】

（转场，屏幕黑底白字，正中央出现“1 Java Web框架创始人”）

2、Java Web框架创始人

下面，简单总结四位全球知名的Java Web开发框架创始人，也是JavaEE开发的改造者，



第一位是Gavin King（加文·金），Hibernate框架创始人，一个具有革命性意义的、全自动化的ORM框架，于2006年发布，最初的版本是基于JPA来实现的。

第二位是Craig Mcclanahan（克雷格·麦克拉纳汉），著名的Struts框架创始人，叫Craig Mcclanahan（克雷格·麦克拉纳汉），比较早的程序员都用过Struts1和Struts2，感受过非常酸爽的味道。2008年发布1.0版本，在2013年6月底还爆发了一次眼中的安全漏洞，那时候开始，我开始全面弃用Struts框架。

第三位是Clinton Begin（克林顿·比格），MyBatis框架创始人，2010年发布，最早期的名字叫做iBatis，它是一个比Hibernate更加轻量级的，半自动化的持久化框架。目前，绝大部分互联网公司都是采用MyBatis作为持久层的框架。

第四位是Rod Johnson（罗德·约翰逊），大名鼎鼎的Spring框架创始人，2004年发布1.0版本，以前叫做万能胶，现在叫做全家桶，真正给程序员带来了春天。

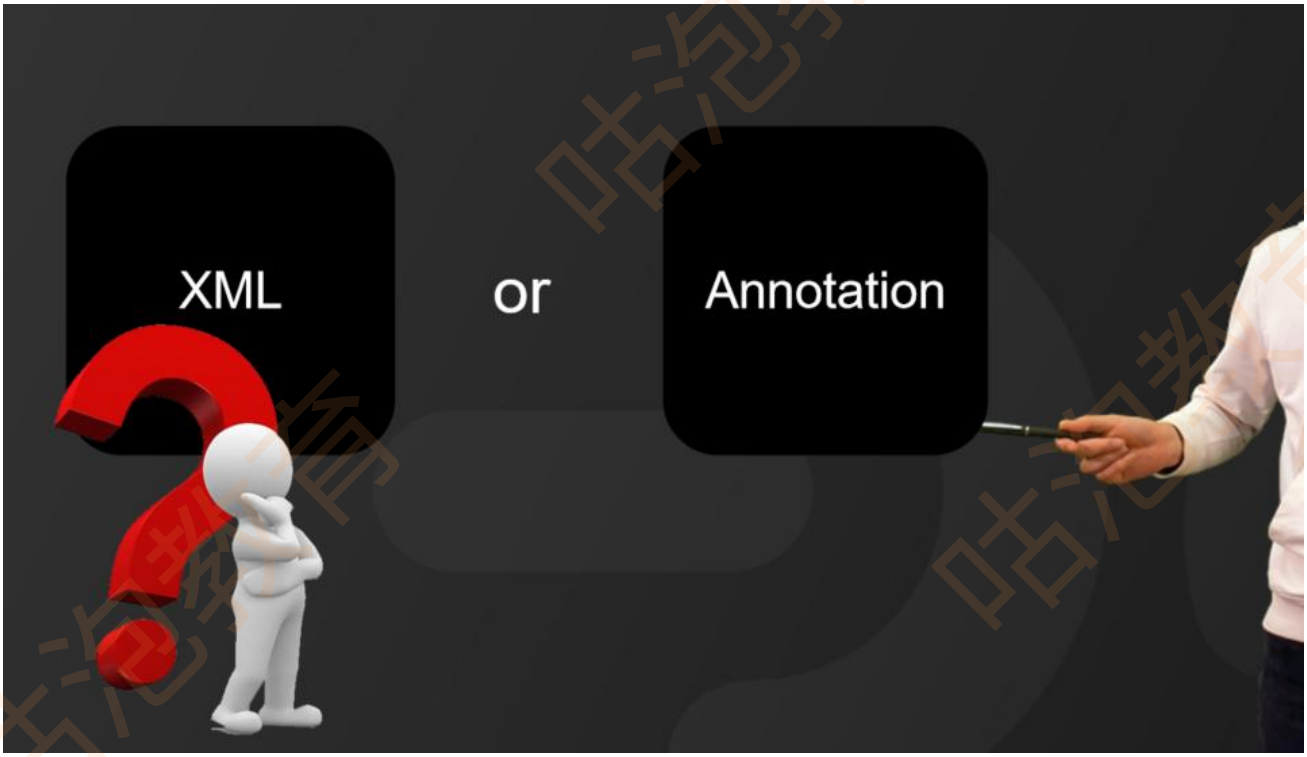
小伙伴们可以对比一下发际线。

【导航条：Spring的演变】

（转场，屏幕黑底白字，正中央出现“3 Spring的演变”）

3、Spring的演变

Spring是一个通用性非常强的框架，它提供的工具也非常全面，能满足各个领域的开发需求。因此，使用Spring之前需要完成大量的个性化配置。在Spring演变过程中，一直存在一个争议，是使用XML作为配置方式 还是 使用 Annotation注解作为配置方式?



Spring在开源社区不断质疑声中，不断的发展和演变，直到Spring 4.0版本发布的时候，已经全面支持注解方式配置，没有XML也可以进行开发。但是，开发者们在使用纯注解开发以后，发现代码侵入比较多，配置也稍显麻烦，尤其是一些Maven依赖也比较繁杂，包括使用外部的Tomcat容器部署也需要不少的工作量。

于是，Pivotal团队从2013年开始研发设计Spring Boot，它的初衷是为了简化开发，旨在解放程序员的双手。2014年4月，发布全新开源的轻量级框架，Spring Boot1.0版本。

【导航条：核心功能】

（转场，屏幕黑底白字，正中央出现“4 SpringBoot的核心功能”）

4、Spring Boot的核心功能

（1）可以独立运行Spring项目

Spring Boot可以以jar包的形式进行独立的运行，使用：java -jar xx.jar 就可以成功的运行项目，或者在应用项目的主程序中运行main函数即可；

（2）内嵌的Servlet容器

内嵌容器，使得我们可以执行运行项目的主程序main函数，实现项目的快速运行；主程序代码SpringbootDemoApplication.java

（3）提供starter简化Manen依赖

Spring Boot提供了一系列的starter pom用来简化我们的Maven依赖，比如创建一个web项目，只需要添加spring-boot-starter-web即可：

（4）自动配置Spring

Spring Boot会根据我们项目中类路径的jar包/类，为jar包的类进行自动配置Bean，这样一来就大大

的简化了我们的配置。当然，这只是Spring考虑到的大多数的使用场景，在一些特殊情况，我们还需要自定义自动配置；

（5）无代码生成、无XML配置

Spring Boot神奇的地方不是借助于代码生成来实现的，而是通过条件注解的方式来实现的，这也是Spring 4.x的新特性。

【导航条：结论】

（转场，屏幕黑底白字，正中央出现“5 结论”）

5、结论

所以，Spring Boot是由Spring发展演进而来的，是Spring进化的结果，是对Sping的升级。

听懂的小伙伴，关注点个赞，下次不迷路。

我是被编程耽误的文艺Tom，如果你还有其他不懂的面试题或者需要视频配套的文字资料，可以关注我的主页加V。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

22．最通俗的方式理解Spring循环依赖三级缓存

【导航条：引言】

今天，有位粉丝找我，说要耽误我5分钟时间，想让我帮助它理解一下Spring循环依赖的三级缓存，绕晕了一个星期，没有想明白。我想今天，用最通俗易懂的方式给大家重新梳理一下，保证让你听懂了。

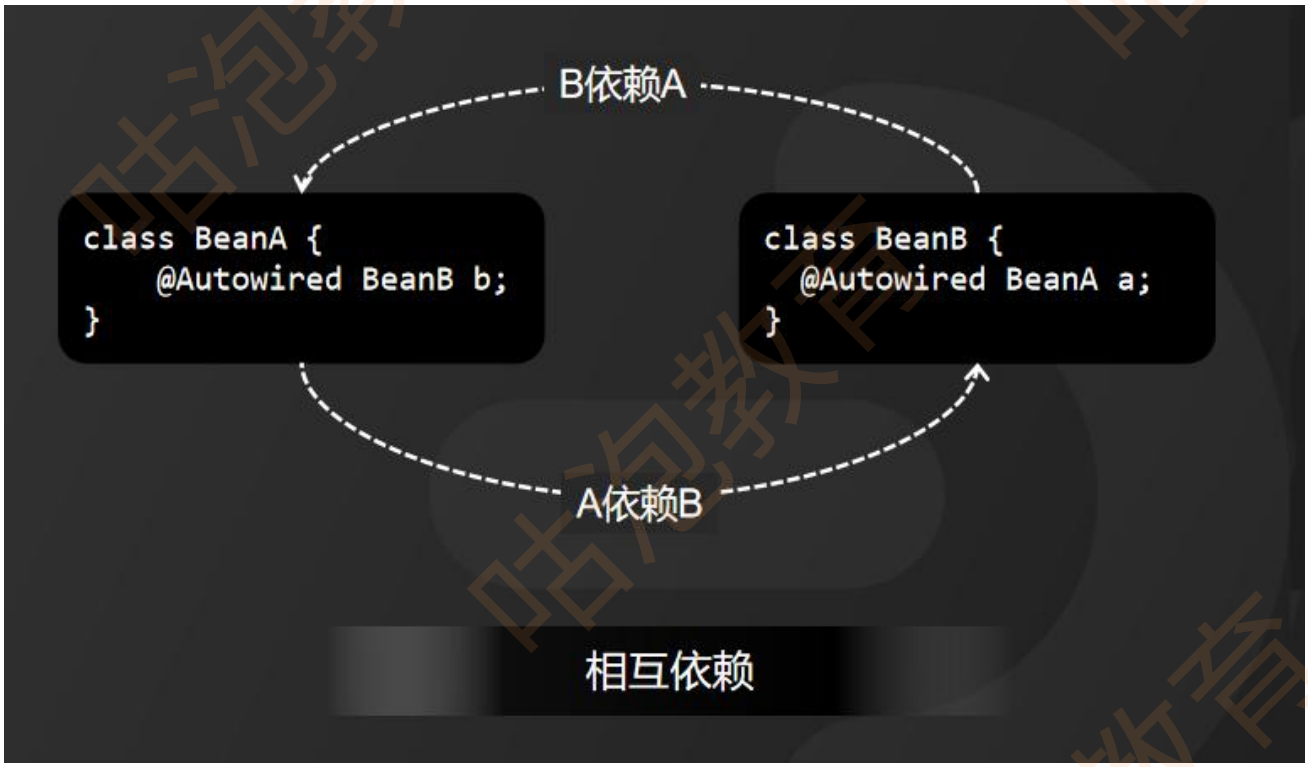
【导航条：什么是循环依赖】

（转场，屏幕黑底白字，正中央出现“1 什么是循环依赖”）

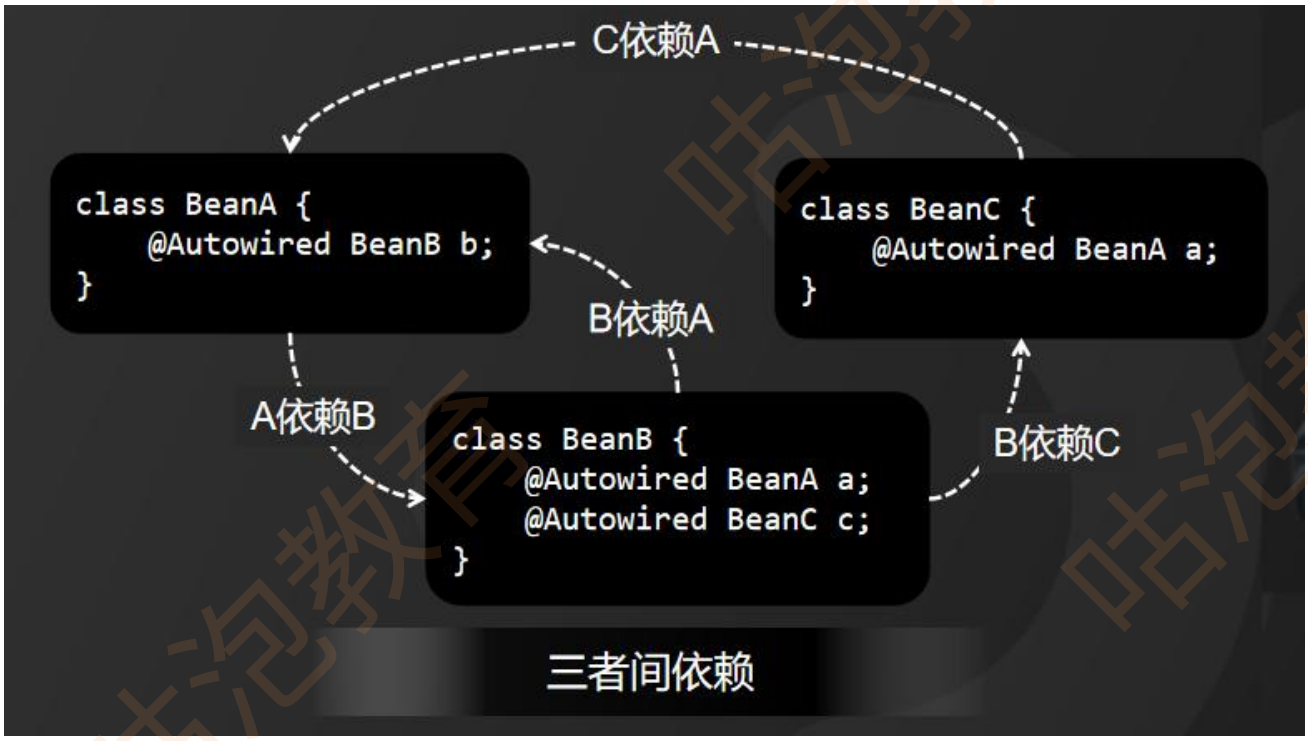
1、什么是循环依赖?

循环依赖就是指循环引用，是两个或多个Bean相互之间的持有对方的引用。循环依赖有三种形态：

1、相互依赖，也就是A 依赖 B，B 又依赖 A，它们之间形成了循环依赖。



2、三者间依赖，也就是A 依赖 B，B 依赖 C，C 又依赖 A，形成了循环依赖。



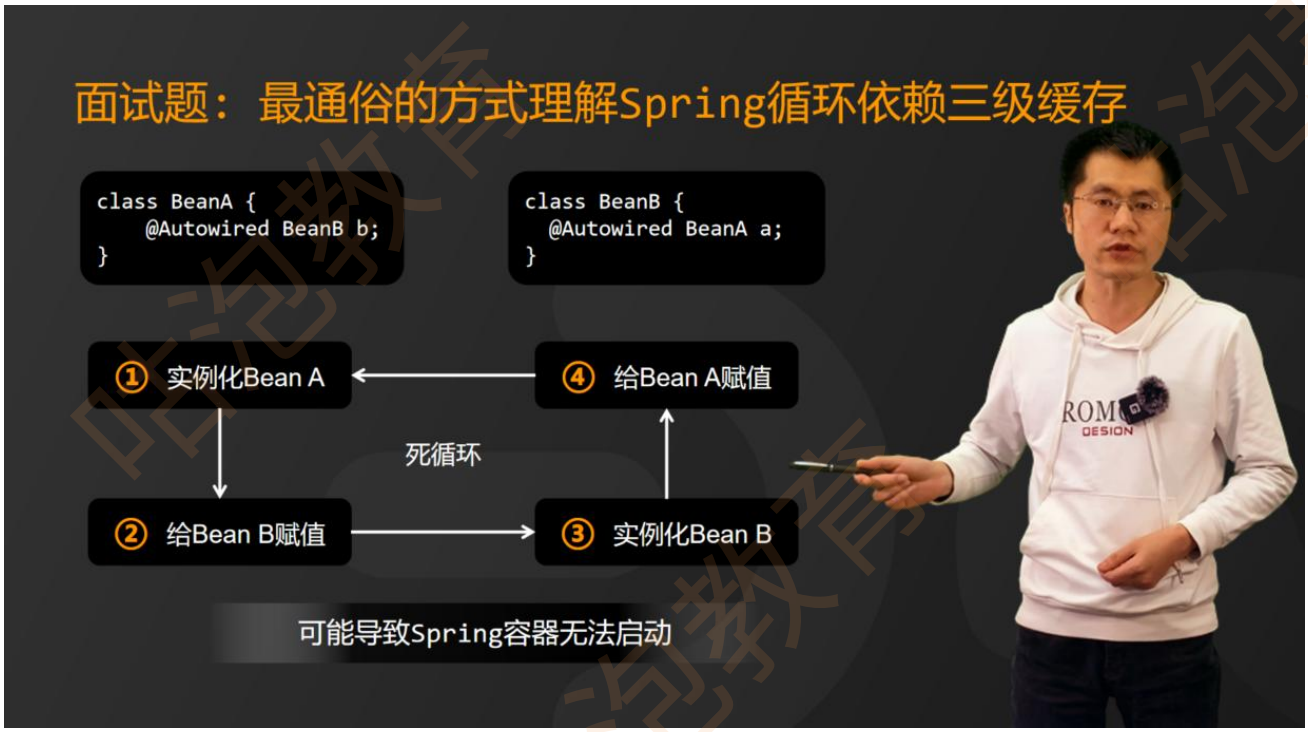
3、自我依赖，也是A依赖A形成了循环依赖自己依赖自己。

【导航条：如何解决】

（转场，屏幕黑底白字，正中央出现“2 如何解决循环依赖问题？”）

2、如何解决循环依赖问题?

循环依赖本身没有问题，问题是Spring中加入了依赖注入机制，也就是自动给属性赋值。当创建Bean实例化以后，需要给Bean中需要赋值的属性全部自动赋值才能交给用户使用。但如果是循环依赖的情况，以两个Bean相互依赖的情况为例，



23．美团面试真题，如何保证Kafka消息不丢失?

【导航条：引言】

一位工作了5年的小伙伴去美团面试以后，跟我反馈说，被问到一个“如何保证Kafka消息不丢失？”的问题，不知道如何回答。其实，这道题真的很基础。

很多小伙伴可能只会回答说，消息要持久化，添加消息确认机制。如果，你只是这样回答，那就和普通的程序员没什么区别。要想让面试官感觉你确实有不一样的理解，就应该从多个方面更全面地来分析和回答这个问题。今天，我来给大家讲明白。

【导航条：解决思路】

（转场，屏幕黑底白字，正中央出现“1 解决思路”）

1、解决思路

首先第一个是服务器端。设置broker中的配置项unclean.leader.election.enable = false，保证所有副本同步。同时，Producer将消息投递到服务器的时候，我们需要将消息持久化，也就是说会同步到磁盘。注意，同步到硬盘的过程中，会有同步刷盘和异步刷盘。如果选择的是同步刷盘，那是一定会保证消息不丢失的。就算刷盘失败，也可以即时补偿。但如果选择的是异步刷盘的话，这个时候，消息有一定概率会丢失。网上有一种说法，说Kafka不支持同步刷盘，这种说法也不能说是错的。但是可以通过参数的配置变成同步刷盘，比如，这样的配置：

当达到下面的消息数量时，会将数据flush到日志文件中。默认10000

#log.flush.interval.messages=10000

当达到下面的时间(ms)时，执行一次强制的flush操作。interval.ms和interval.messages无论哪个达到，都会flush。默认3000ms

#log.flush.interval.ms=1000

检查是否需要将日志flush的时间间隔

log.flush.scheduler.interval.ms = 3000

同样可以达到同步刷盘的效果。

第二个，就是生产者Producer，使用带回调通知的send(msg,callback)方法，并且设置acks = all 。它的消息投递要采用同步的方式。Producer要保证消息到达服务器，就需要使用到消息确认机制，也就是说，必须要确保消息投递到服务端，并且得到投递成功的响应，确认服务器已接收，才会继续往下执行。那如果，Producer将消息投递到服务器端，服务器来没来得及接收就已经宕机了，那投递过来的消息岂不是丢失了，怎么办呢？大家不要慌，在Producer投递消息时，都会记录日志，然后再将消息投递到服务器端，就算服务器宕机了，等服务器重启之后，也可以根据日志信息完成消息补偿，确保消息不丢失。

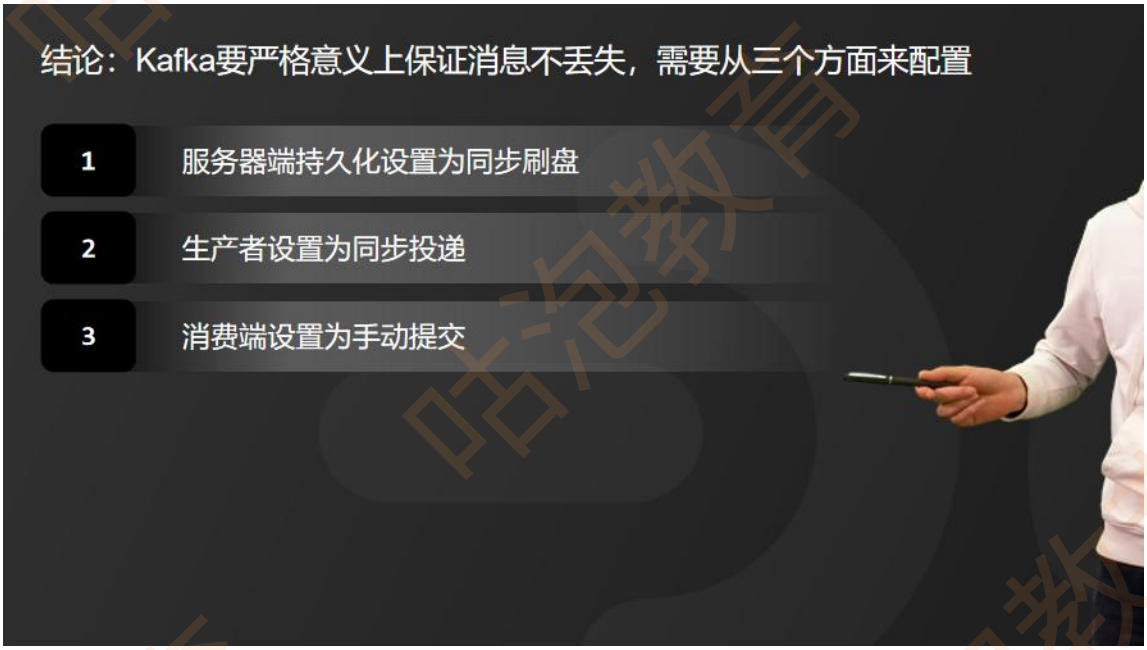
第三个，就是消费者Consume。设置enable.auto.commit为false。在Kafka中，消息消费完成之后，它不会立即删除，而是使用定时清除策略，也就是说，我们消费者要确保消费成功之后，手动ACK提交。如果消费失败的情况下，我们要不断地进行重试。所以，消费端不要设置自动提交，一定设置为手动提交才能保证消息不丢失。

【导航条：总结】

（转场，屏幕黑底白字，正中央出现“2 总结”）

2、总结

Kafka要严格意义上保证消息不丢失，需要从三个方面来设置，



第一个服务器端持久化设置为同步刷盘、第二个生产者设置为同步投递，第三个消费端设置为手动提交。

以上就是对Kafka保证消息不丢失的解决方案，听懂的小伙伴，关注点个赞，下次不迷路。

我是被编程耽误的文艺Tom，如果你还有其他不懂的面试题或者需要视频配套的文字资料，可以关注我的主页介绍。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

24．3分钟通晓，互联网架构20年以来的演进

【导航条：引言】

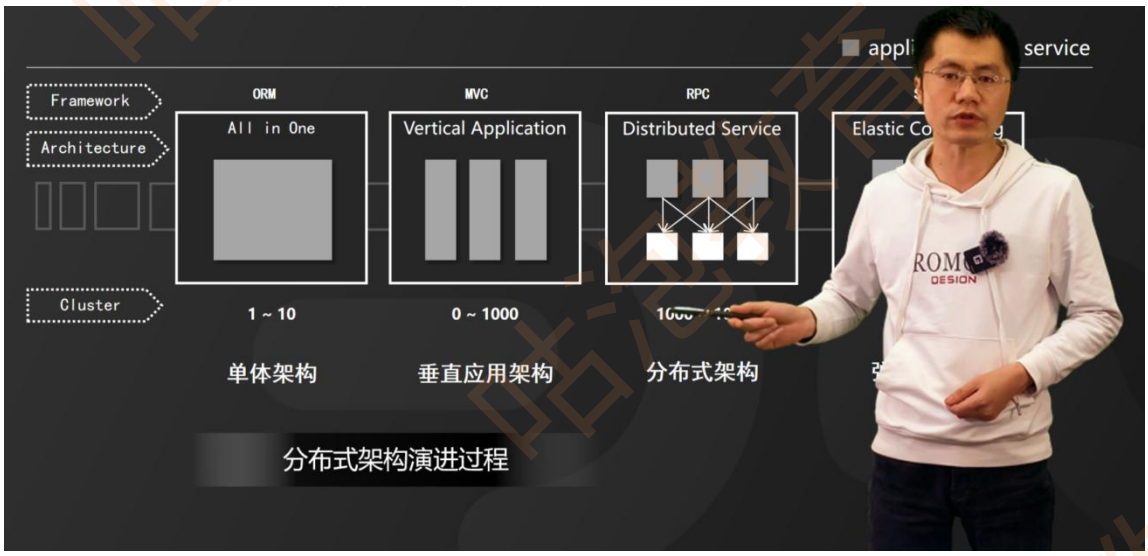
作为一个Java程序员，你可能也思考过，为什么我还是普通开发，为什么我还是高级开发，普通开发和高级开发有什么区别？你是不是也想过要成为架构师？想要成为合格的架构师，就必须要了解架构的演进，今天，我们就来聊一聊，Java架构的演变历史。

【导航条：分布式微服务】

（转场，屏幕黑底白字，正中央出现“1 分布式微服务架构的演进”）

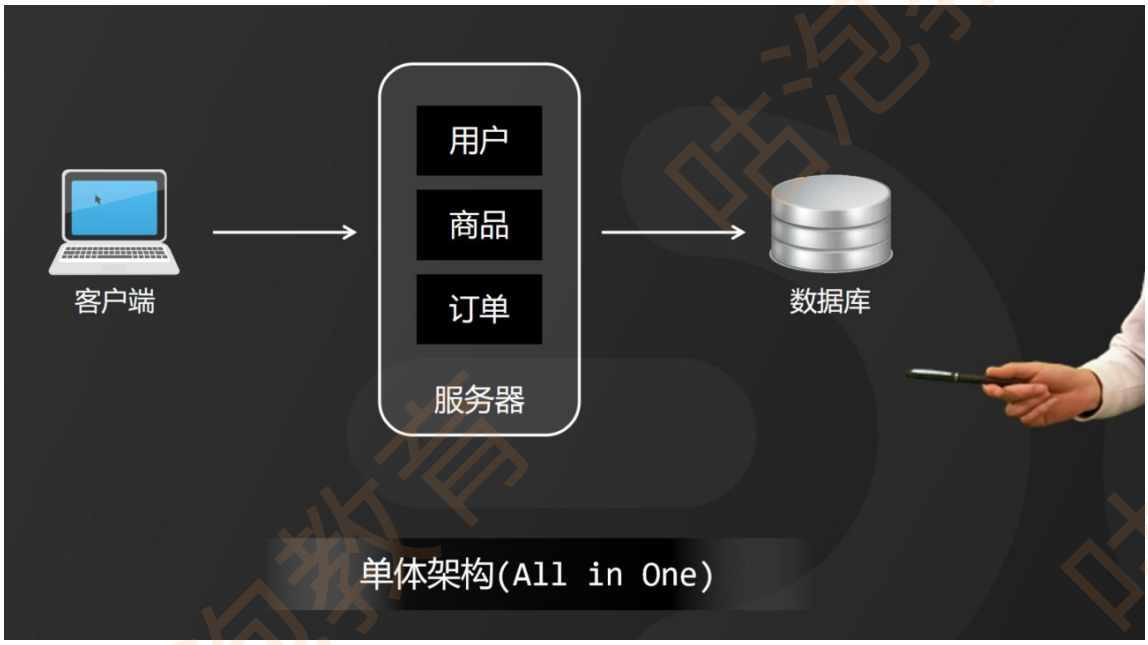
1、分布式微服务架构的演进

分布式微服务架构的发展，主要经历了四个阶段：单一应用架构、垂直应用架构、分布式架构和弹性SOA架构。



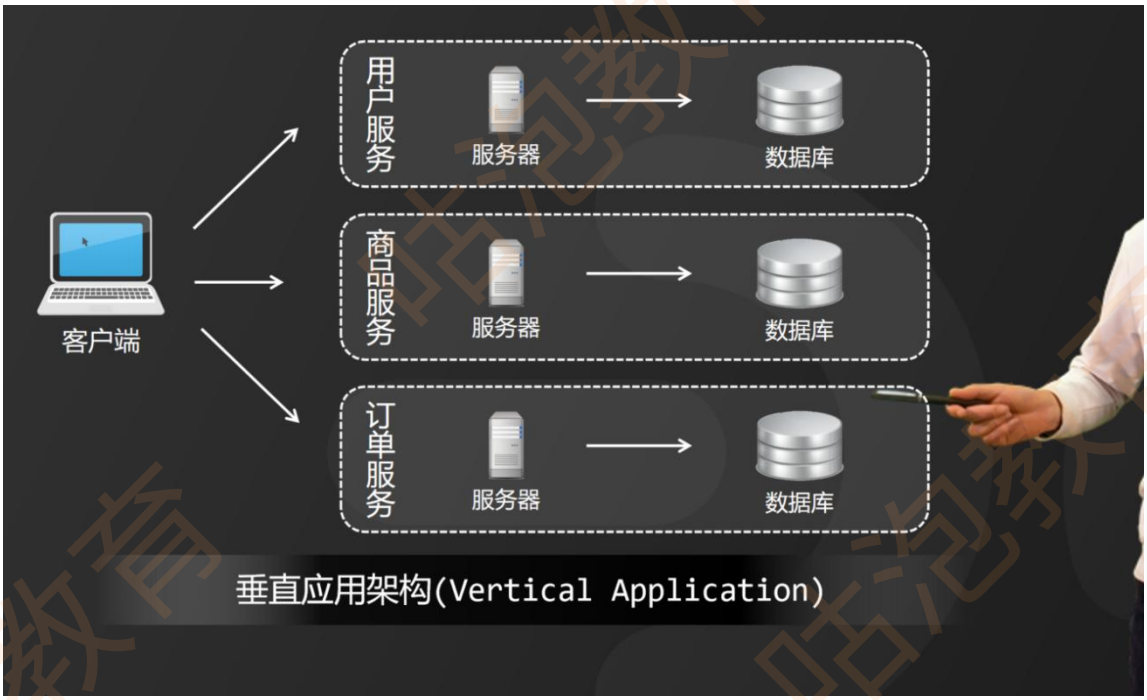
这张图是从Dubbo官网上下载的描述分布式架构演进过程的示意图，大家可以收藏一下。

1)单体架构（All in One）



从2000年开始，互联网在中国开始盛行起来，但是那时候网民数较少，网站流量也很小，只需要一台机器就可以运行所有的服务，只需要All in One单体架构就能满足需求。

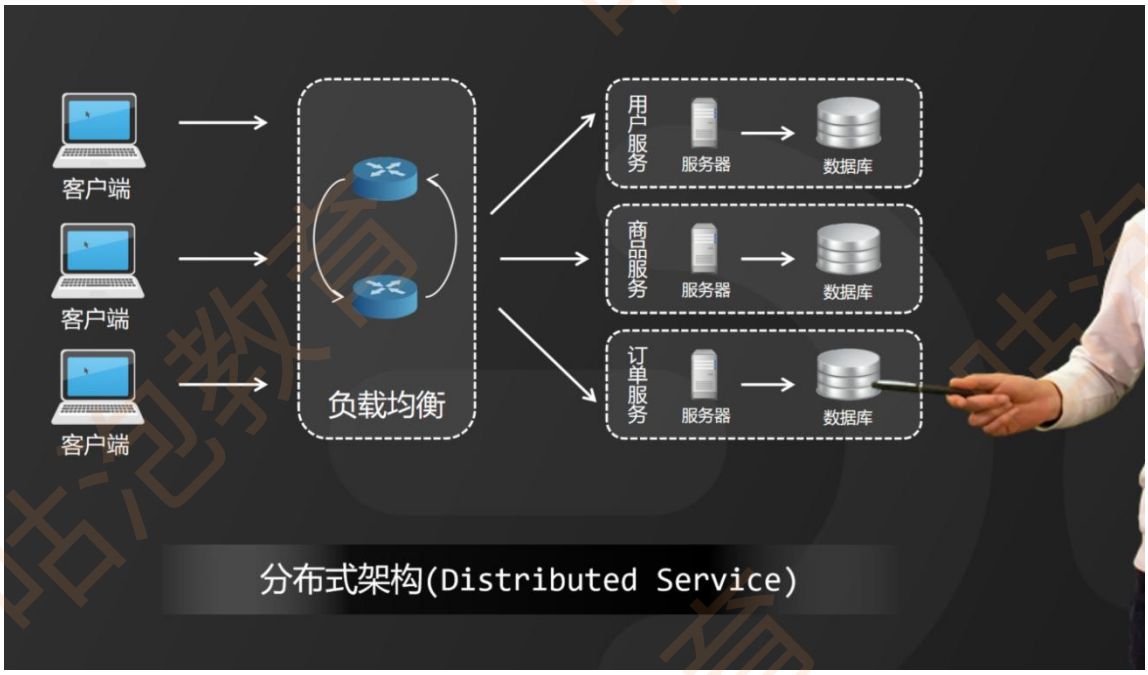
2）垂直应用架构(Vertical Application)



随着网站访问量的增加，一旦服务器或者数据库出现问题，将会导致整个系统故障，造成所有服务不可用，而且功能修改发布也不方便，所以，就把大系统拆分成了多个子系统，

比如将“用户”、“商品”、“订单“等按业务进行拆分，也就是”垂直拆分“，并且每个子系统都部署到不同的服务器上。

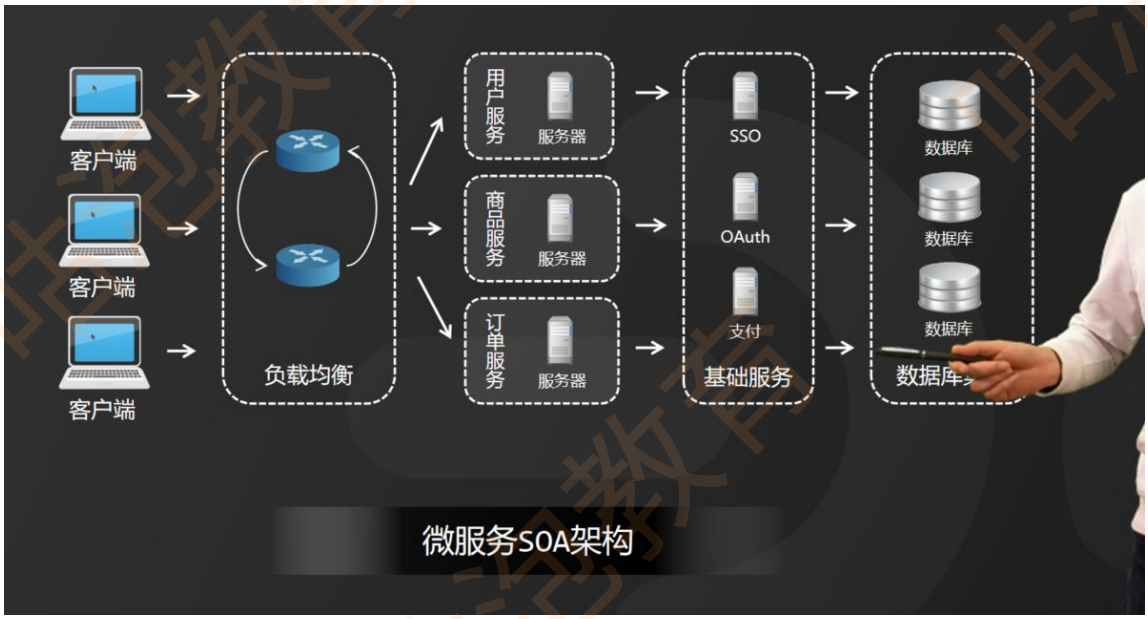
3）分布式架构(Distributed Service)



随着用户访问越来越大，意味着需要更多的服务器才能支撑服务的运行，而应用之间的交互不可能避免地变得越来越复杂。为了保证服务稳定，提高管理效率，需要对这些应用做负载均衡。因为用户不可能去了解后台服务器的数量和业务结构，有了负载均衡以后，用户只需要统一访问负载均衡服务器就可以，后端的应用就可以根据流量的大小进行动态的扩缩容，也就是“水平扩展”。

4）弹性SOA架构（Elastic Computing）

分布式架构运行一段时间之后，发现商品和订单服务中有很多重复的功能，比如SSO、OAuth权限和支付等。一旦项目大了，集群部署多了，这些重复的功能无疑就是浪费资源，就需要把这些重复的功能单独抽离出来单独部署，并且给它们起了个名字叫“Service(服务)“，并且对服务进行分层，比如Base Service基础服务、Business Service业务服务等等。



有了服务的概念之后，我们还可以根据业务需要，将服务进一步细化并拆分。比如把对用户服务可以划分为用户积分服务、个人优惠券服务等等，这就是我们通常所说的微服务架构。

但是往往对于微服务架构实际上没有一个明确的定义，因为微服务的边界很难确定，所以微服务架构一直在不断地重构过程中发展，没有完美的微服务架构，只有适合公司业务场景的微服务架构。

当然，以上是我个人对微服务的理解，为了帮助大家更准确的理解微服务，这里引用Martin Folwer前辈对于微服务描述的一段话，英文我就不读了，大家可以通过翻译软件来理解一下上述这段话。

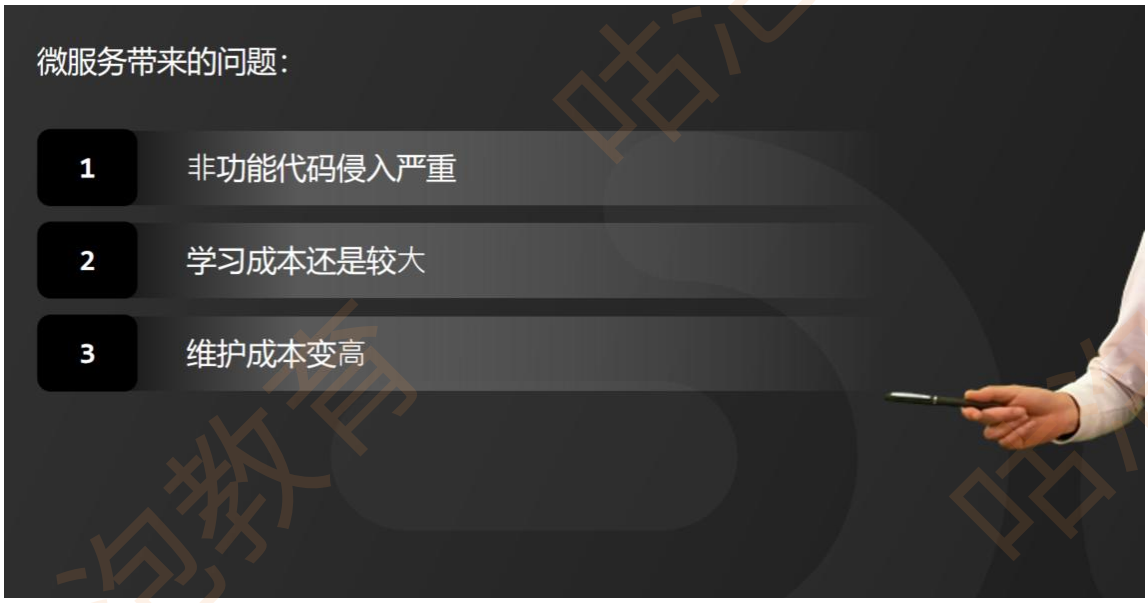
In short, the microservice architectural style is an approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms, often an HTTP resource API. These services are built around business capabilities and independently deployable by fully automated deployment machinery. There is a bare minimum of centralized management of these services, which may be written in different programming languages and use different data storage technologies.

【导航条：服务网格】

（转场，屏幕黑底白字，正中央出现“2 服务网格”）

2、服务网格（Service Mesh）

在Java中微服务架构的主流解决方案是Spring Cloud，给程序员带来了非常便捷的开发体验。但是，有了Spring Cloud的微服务架构就完美了吗？现实并非如此，很快我们就发现微服务带来了很多的问题，我大致总结为以下三点：



第1点：非功能代码侵入严重。为了满足业务需要，我们可以使用框架提供给我们的各种组件，比如网络通信、消息处理等。于是，我们需要引入各种依赖， 加注解，写配置，最后将这些非业务功能的代码和业务代码打包一起部署，这就是“侵入式框架”；

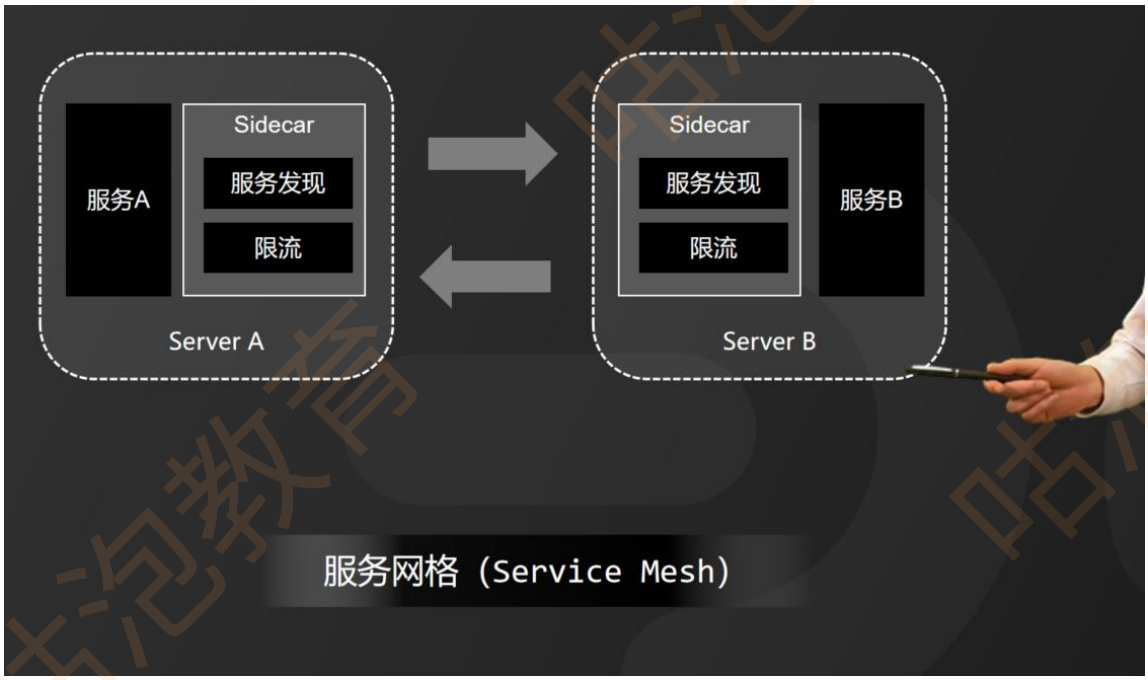
第2点：微服务框架学习成本还是较大。Spring Cloud虽然能解决微服务领域的很多问题，但是需要程序员花费比较多的精力来学习它的使用。本来程序员应该把更多的精力投入到业务开发上，而不应该是非业务上。

第3点：维护成本变高。互联网公司产品的版本升级是非常频繁的，因为Spring Cloud是“代码侵入式的框架”，为了维护各个版本的兼容性、权限、流量等，这时候版本的升级就注定要让非业务代码一起， 再加上多语言之间的调用，一旦出现问题，程序员们会非常痛苦。

其实，大家有没有发现？服务拆分的越细，只是感觉上解耦了，但是维护成本却变高了，那怎么办呢？

网络的问题当然还是要交给网络本身来解决。所以，服务网格的技术就应运而生，也就是Service Mesh。

我们可以为每个服务单独配置一个Sidecar，所有服务通信的进出口流量都会通过Sidecar来进行操作，如图所示：



常见的服务网格产品有Prana、Local Proxy、Linkerd、Istio等，目前比较主流的是Istio，我们来看看Istio的架构图：

可以发现，该架构图分为数据平面和控制平台，数据平面就是前面的ServiceA和ServiceB组成的部分，控制平面主要指的是Mixer、Pilot、Galley、Cidatel等组件，这些组件的详细功能这里就不详细展开了，大家感兴趣的话回复 666 ，我可以单独出一期视频。

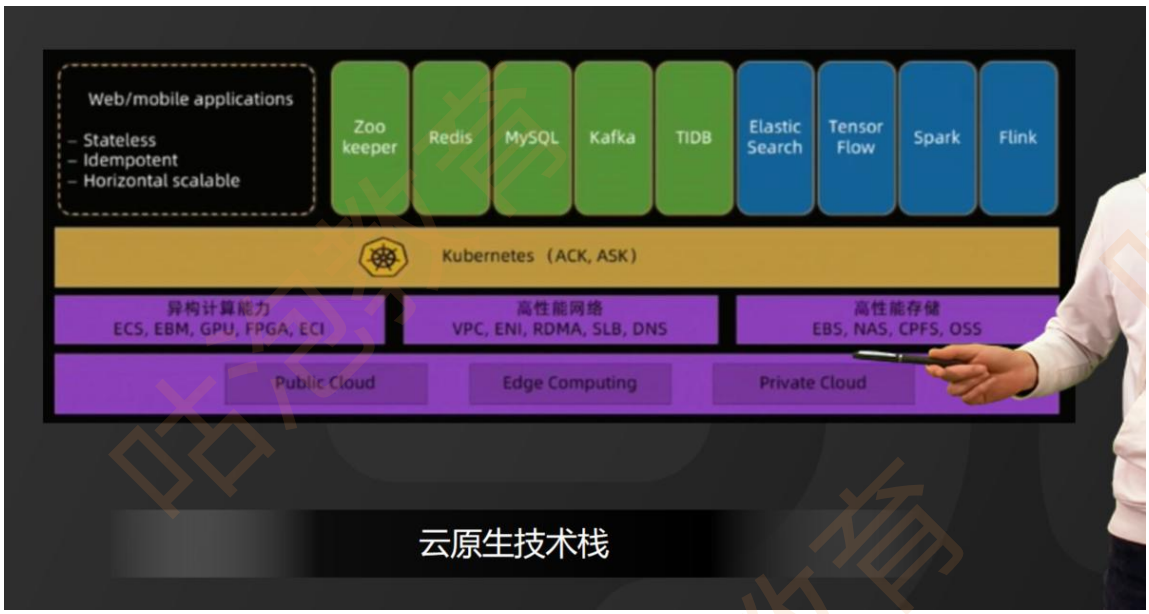
回到主题，我们可以思考一下这些组件和服务部署起来是否会有难度？答案是肯定的，因为大家使用的基础设施和操作系统可能会有差异，所以，这时候我们需要使用容器化技术来部署。

【导航条：云原生架构】

（转场，屏幕黑底白字，正中央出现“4 容器化云原生架构”）

3、容器化云原生架构

架构演进到现在这个阶段，我们可以基于容器化和容器编排的技术来实现公有云、私有云、混合云地无缝迁移，也就是可以使用云原生架构来考虑组件和服务的部署方案。通过这张图来理解一下：



不管底层的基础设施是什么，都可以用Kubernetes作为适配，帮助开发运维人员屏蔽底层硬件的细节，同时采用Devops来打破传统开发和运维之间的界限。

互联网主流业务架构的演进过程我们就分享到这了。

【导航条：总结】

（转场，屏幕黑底白字，正中央出现“7 总结”）

4、总结

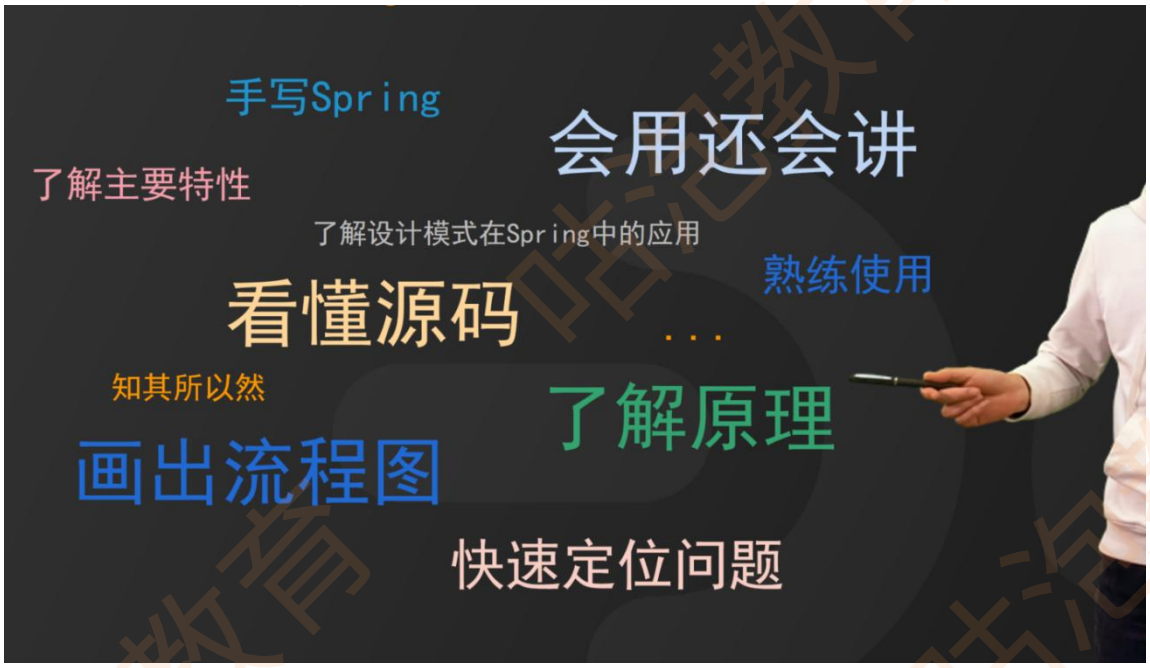
技术的发展令人敬畏，从单体架构时代到现在的云原生时代，每一次架构的演进都会产生一些新的技术名词：分布式、微服务、云原生等等等等，这些名词，不都是为了让我们的开发变得更加便捷？

我是被编程耽误的文艺Tom，如果你还有其他不懂的面试题或者需要视频配套的文字资料，可以关注我的主页介绍。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

25．Spring到底学到什么程度才算学会?

【导航条：引言】

面试造火箭，工作拧螺丝。今天有位工作5年的小伙伴私我说，Spring到底要学到什么程度才算学会？你可能会这样回答：



能手写Spring

会用还会讲出来

能看源码

了解原理

能知其所以然，知道为何要这样设计

了解设计模式在Spring中的应用

能画出主要流程图

能快速定位问题，解决问题

了解各种主要的特性

会熟练使用

小伙伴们也可以思考一下，在评论告诉我的你看法.

【导航条：Tom的经验】

（转场，屏幕黑底白字，正中央出现“Tom的经验分享”）

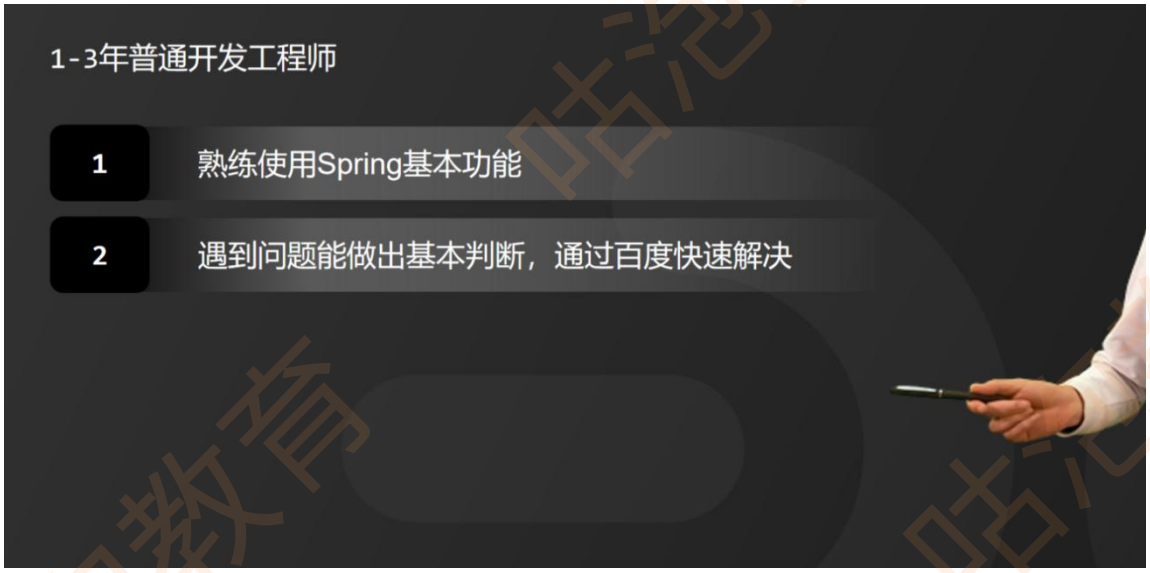
1、Tom的经验分享

今天，我给大家聊聊我的看法。我认为要回答这个问题，要根据每个人的工作年限和业务需要来回答。

1）如果你是0-1年的初级开发

1、会独立使用Spring进行开发

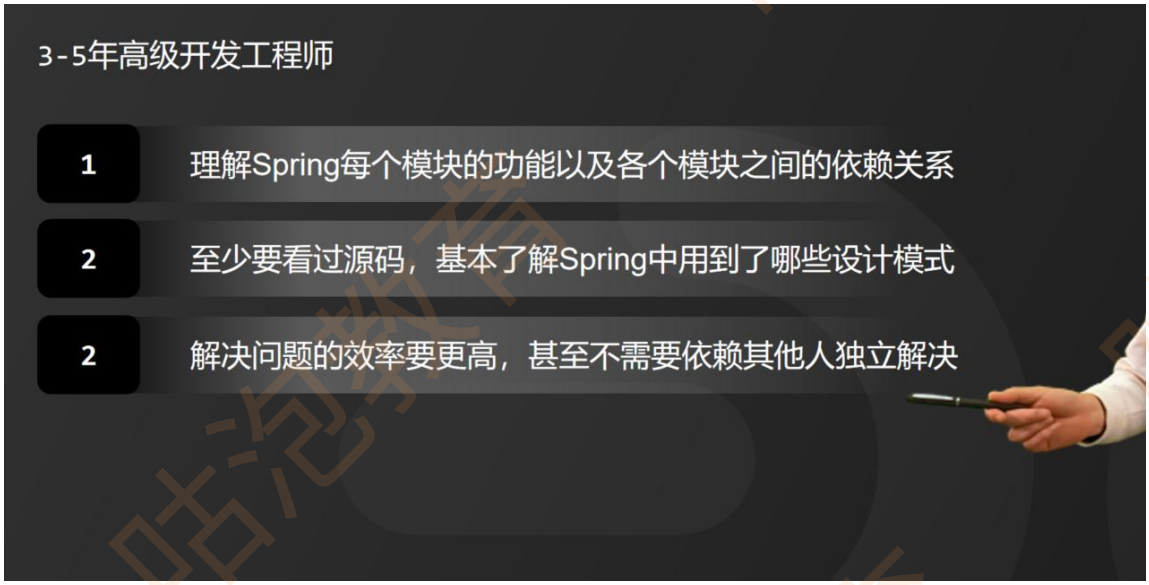
2、遇到问题3个小时内能解决



2）如果你是1-3年普通开发工程师

1、熟练使用Spring基本功能，手速要快

2、遇到问题能做出基本判断，通过百度快速解决



3）如果你是3-5年的高级开发工程师

1、理解Spring每个模块的功能以及各个模块之间的依赖关系

2、至少要看过源码，基本了解Spring中用到了哪些设计模式

3、解决问题的效率要更高，甚至不需要依赖其他人独立解决

4）如果你是5年以上的资深开发或者架构师

1、至少看过1-3遍Spring源码

2、掌握Spring高级特性，核心原理

3、能用图形方式画出Spring核心的组件关系

我还给大家整理一张程序员黄金5年各个岗位阶段对应的能力模型，有需要的小伙伴可以收藏一下。

我还给大家整理了一张程序员职业发展路线图，这张图非常直观的展示了程序员的职业发展路径。

我是被编程耽误的文艺Tom，如果你还有其他不懂的面试题或者需要视频配套的文字资料，可以关注我的主页介绍。如果我的面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

26．Redis是单线程还是多线程，为什么效率这么高?

【导航条：引言】

Redis是单线程还是多线程？这个问题，把一个5年开发经验的程序员给问懵了。可能很多小伙伴平时都听说过，Redis就是单线程的。但是我们学习Redis命令的时候，有经常有设置异步，比如bgsave命令，它的作用是在后台异步持久化数据到磁盘，那既然是异步，肯定需要由线程来完成。

另外，往期面试题解析中配套的文档我已经准备好，想获得的可以在我的煮叶简介中找到。

接下来，我们来介绍，Redis到底是单线程还是多线程？。

【导航条：官方解释】

（转场，屏幕黑底白字，正中央出现“1 官方解释”）

1、官方解释

其实，Redis 确实是单线程的。通常说的单线程，主要是指 Redis 对外提供的键值存储服务的主要流程是单线程，也就是网络IO和数据读写是由单个线程来完成的。除此外 Redis 的其他功能，比如持久化、 异步删除、集群数据同步等，是由额外的线程来执行的。这样做的好处是可以 。因此，严格地说 Redis 并不是全面单线程。

在Redis官网页做出了解释，它是这样描述的：

Redis is single threaded. How can I exploit multiple CPU / cores?It’ s not very frequent that CPU becomes your bottleneck with Redis, as usually Redis is either memory or network bound. For instance, using pipelining Redis running on an average Linux system can deliver even 1 million requests per second, so if your application mainly uses O(N) or O(log(N)) commands, it is hardly going to use too much CPU.However, to maximize CPU usage you can start multiple instances of Redis in the same box and treat them as different servers. At some point a single box may not be enough anyway, so if you want to use multiple CPUs you can start thinking of some way to shard earlier.You can find more information about using multiple Redis instances in the Partitioning page.However with Redis 4.0 we started to make Redis more threaded. For now this is limited to deleting objects in the background, and to blocking commands implemented via Redis modules. For future releases, the plan is to make Redis more and more threaded.

大致意思是：

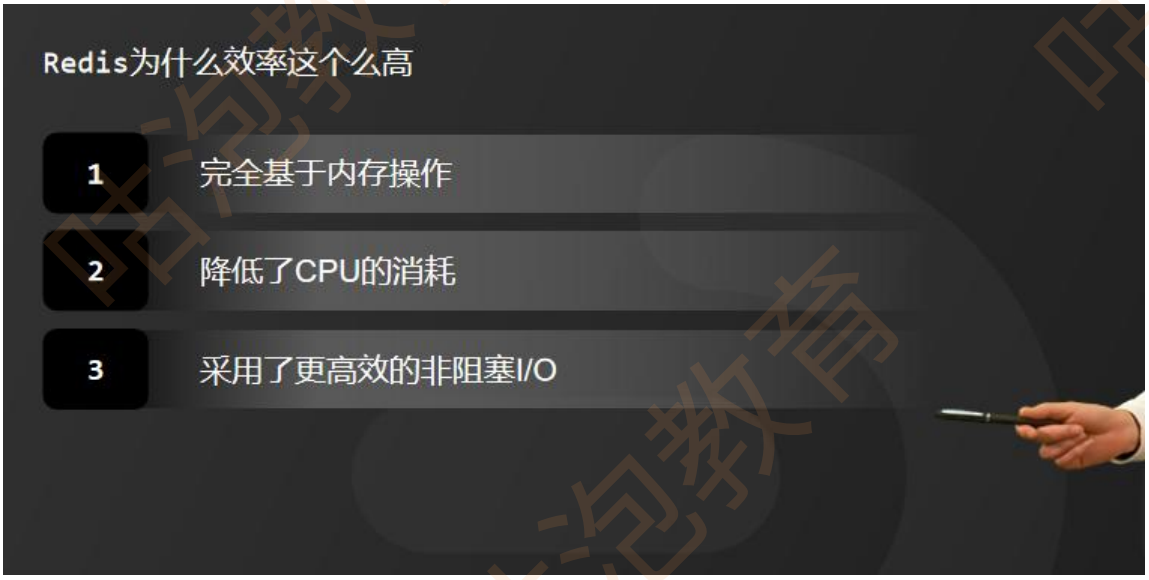
由于Redis是基于内存的操作，查找和操作的时间复杂度都是O(1)，因此CPU并不是Redis的瓶颈。 Redis的瓶颈很可能是机器内存或网络带宽的大小。 既然单线程易于实现并且CPU不会成为瓶颈，那采用单线程解决方案是合乎逻辑的了。

【导航条：高性能原因】

（转场，屏幕黑底白字，正中央出现“2 高性能原因）

2、高性能原因

既然Redis设计为单线程，为什么效率这个么高呢？我总结以下三个原因：



1、完全基于内存操作

Redis中绝大部分请求是纯粹的内存操作，非常快速。数据存在内存中，类似于HashMap的数据结构，对数据操作也简单。

2、降低了CPU的消耗

采用单线程，避免了不必要的上下文切换和竞争条件，也不存在多进程或者多线程导致的切换而消耗 CPU，不用去考虑各种锁的问题，不存在加锁释放锁操作，没有因为可能出现死锁而导致的性能消耗。

3、采用了更高效的非阻塞I/O

Redis采用epoll做为I/O多路复用技术的实现，再加上Redis自身的事件处理模型将epoll中的连接，读写，关闭都转换为了事件，不在I/O上浪费过多的时间，使得Redis在网络 IO 操作中能并发处理大量的客户端请求，实现了高吞吐率。

【导航条：结尾】

以上，就是我对Redis单线程设计以及Redis为什么这么高效的分析。在目前最新的Redis版本中，IO 多线程处理模式默认是不开启的，需要去配置文件中开启并配置线程数。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

27．什么是Redis缓存雪崩、缓存穿透和缓存击穿?

【导航条：引言】

什么是Redis缓存雪崩、缓存穿透、缓存击穿？关于这个问题至少有20位以上的粉丝咨询过我。这确实是一道互联网大厂的高频面试题。今天，我给大家来聊一聊。

另外，往期面试题解析中配套的文档我已经准备好，想获得的可以在我的煮叶简介中找到。

好了，首先来说一下，缓存的作用。

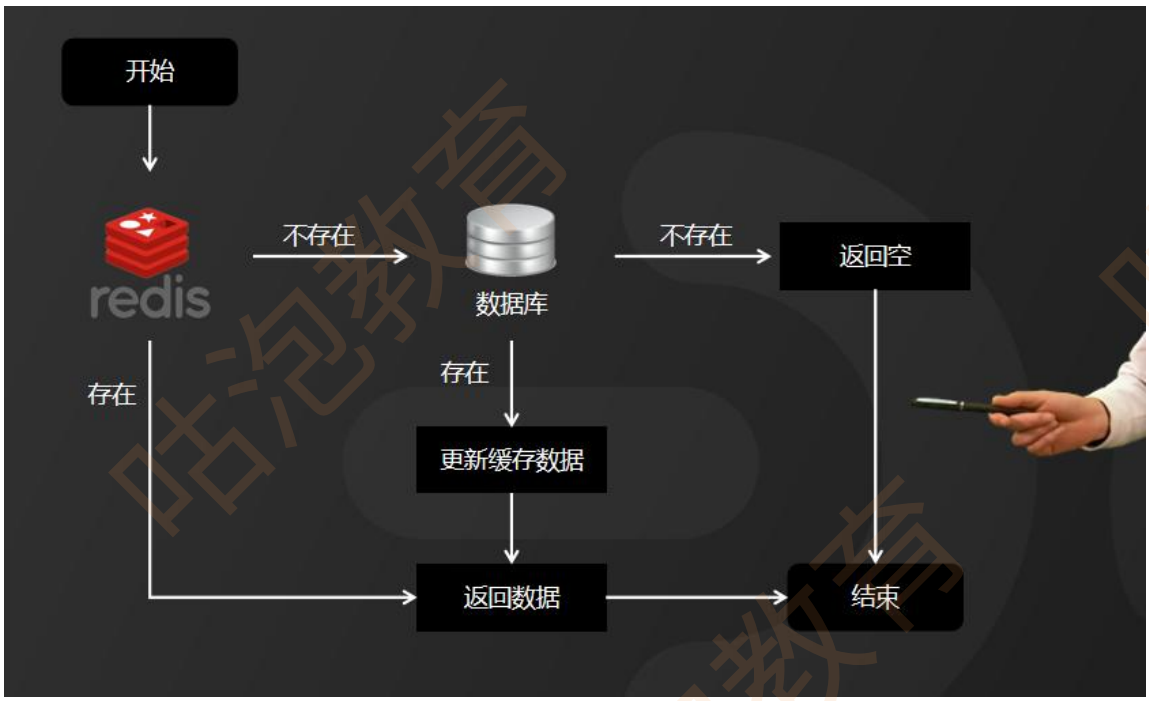
【导航条：缓存的作用】

（转场，屏幕黑底白字，正中央出现“1 缓存的作用”）

1.缓存的作用

一般情况下，1.用户请求到后台，会先从缓存中取数据，如果在缓存中取到数据，就直接返回结果，如果取不到

数据就需要查询数据库，从数据库中取数据，从数据库中取到数据后会同步更新到缓存，并返回结果。下一个用户就可以直接从缓存中取值了。那如果在数据库也没有取到，就只能直接返回空结果。所以，缓存主要的作用就是缓解数据的负载压力，提升请求响应效率。



那什么是缓存雪崩、缓存穿透和缓存击穿呢?

2.如何理解

举个例子，就好比是有个畅销商品，比如前段时间比较火的冰墩墩经常卖断货。那如果商品断货呢分为三种情况，第一种情况是门店和厂家都有存货，第二种情况是，门店和厂家都没有存货，第三种情况是，门店无货和厂家存货。

缓存雪崩就是，门店和厂家都有存货，但是卖得实在是太火爆了，排队的人太多导致门店一时间无法承受这么大压力，因此排队的用户就都去找厂家买了，从而导致厂家短时间内也要承受了很大的压力响应不过来。

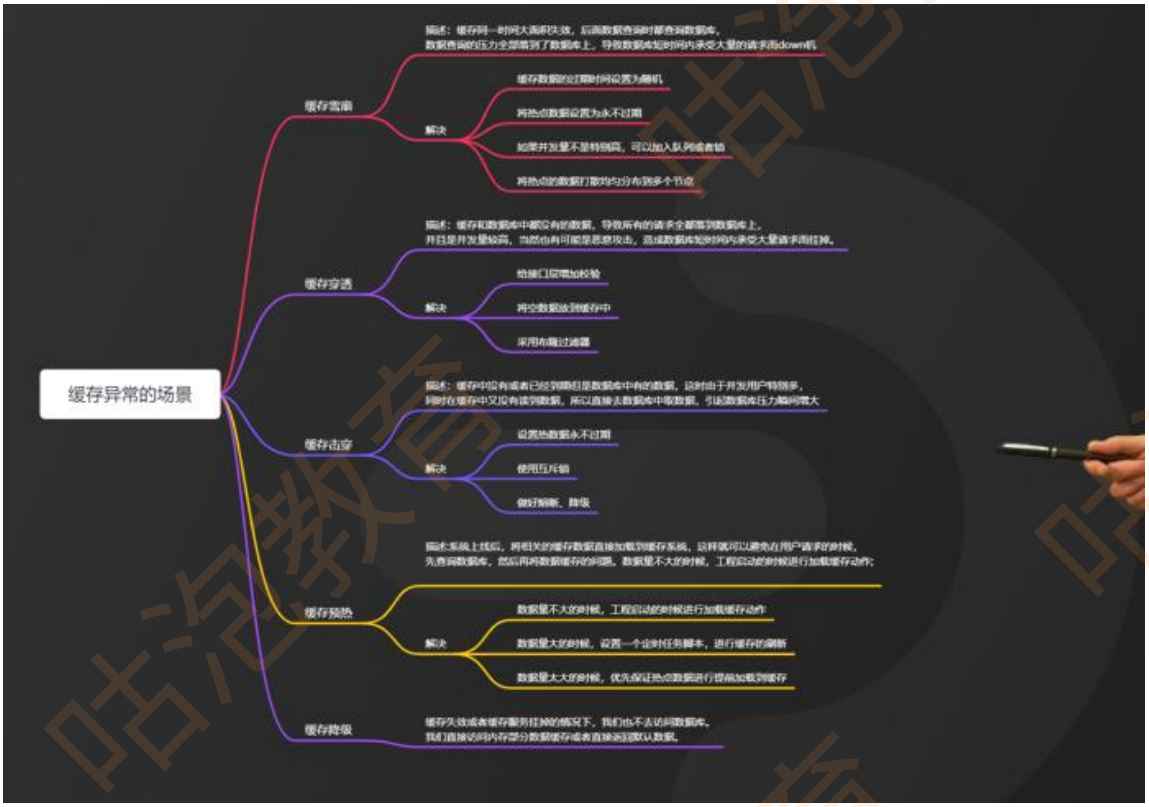
缓存穿透就是，门店和厂家都没有货了，客户把厂家的电话都打爆了，但还是只能等着。

缓存击穿就是，门店无货了但是厂家还有货，所以，客户就把电话打到厂家，导致厂家的压力瞬间暴增。

如果想了解官方的详解解释和解决方案，请关注我的主页，看下一期视频讲解。

3.总结

大家不要被表面的文字所迷惑，不管是缓存雪崩、缓存穿透还是缓存击穿，都是指缓存失效或者异常，导致访问压力流转到数据库，从而使得数据库负载过高。那么缓存异常的场景还有缓存预热异常、缓存降级异常等等，我总结了一张脑图供大家参考和学习，有需要的小伙伴可以关注我的主页介绍。



我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

28．如何解决缓存雪崩、缓存穿透和缓存击穿?

【导航条：引言】

上一期视频，我跟大家聊了我对缓存雪崩、缓存穿透和缓存击穿的理解。那官方又是如何解释的呢？另外，往期面试题解析中配套的文档我已经准备好，想获得的可以在我的煮叶简介中找到。

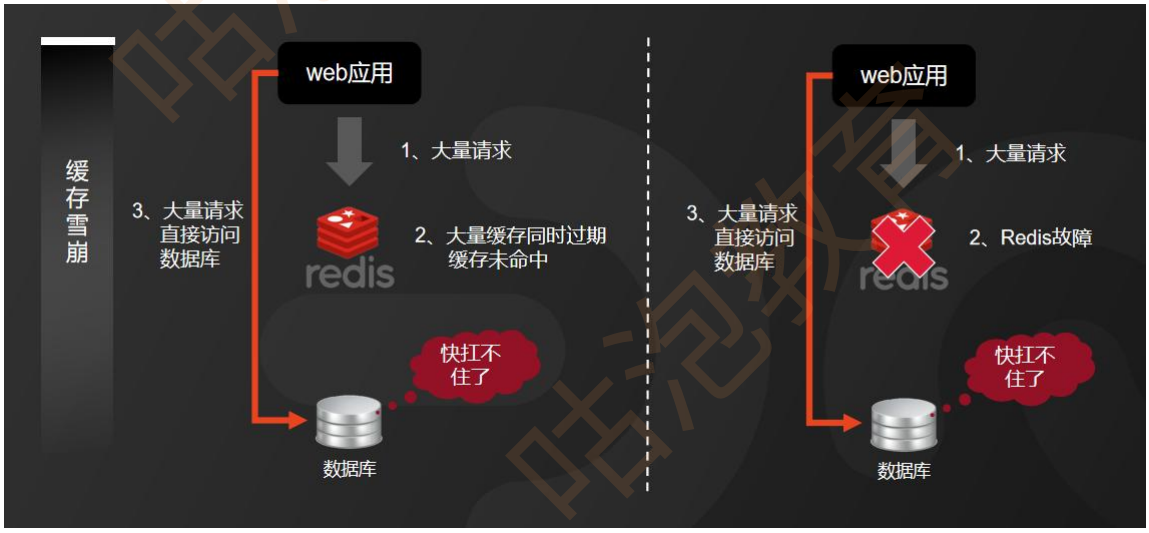
我们先来看缓存雪崩。

【导航条：缓存雪崩】

（转场，屏幕黑底白字，正中央出现“2缓存雪崩”）

1.缓存雪崩

缓存雪崩：是指缓存同一时间大面积失效，后面数据查询时都查询数据库，数据查询的压力全部落到了数据库上，导致数据库短时间内承受大量的请求而down掉。如图所示：



那如何预防缓存雪崩呢？我建议从以下四个方面来进行设置：

1.缓存数据的过期时间设置为随机，防止同一时间出现大批数据过期的现象。

2.将热点数据设置为永不过期。

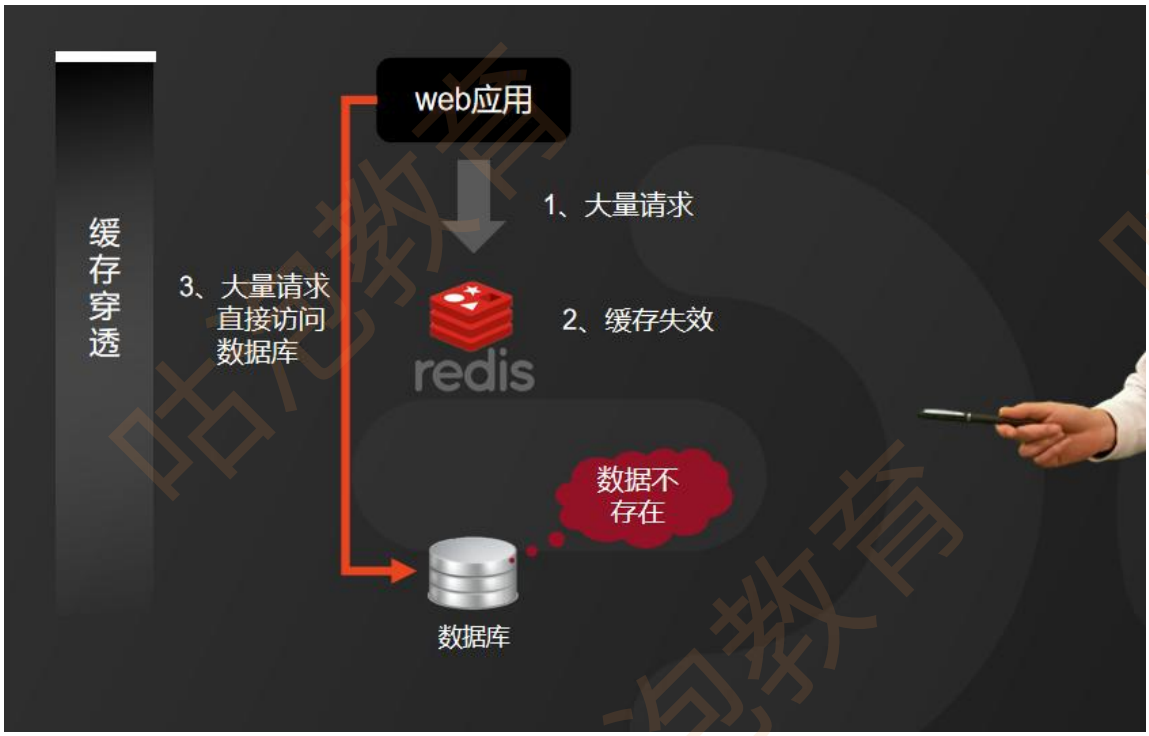
3.如果并发量不是特别高，可以加入队列或者锁。

4.如果缓存数据库是分布式部署的，可以将热点的数据打散均匀分布到多个节点中。

【导航条：缓存穿透】

（转场，屏幕黑底白字，正中央出现“2 缓存穿透”）

2.缓存穿透



缓存穿透：是指缓存和数据库中都没有的数据，导致所有的请求全都落到数据库上，并且是并发量较高，当然也有可能是恶意攻击，造成数据库短时间内承受大量请求而挂掉。

那如何预防缓存穿透呢？我建议从以下三个方面来进行设置：

1、给接口层增加校验，比如用户鉴权校验。

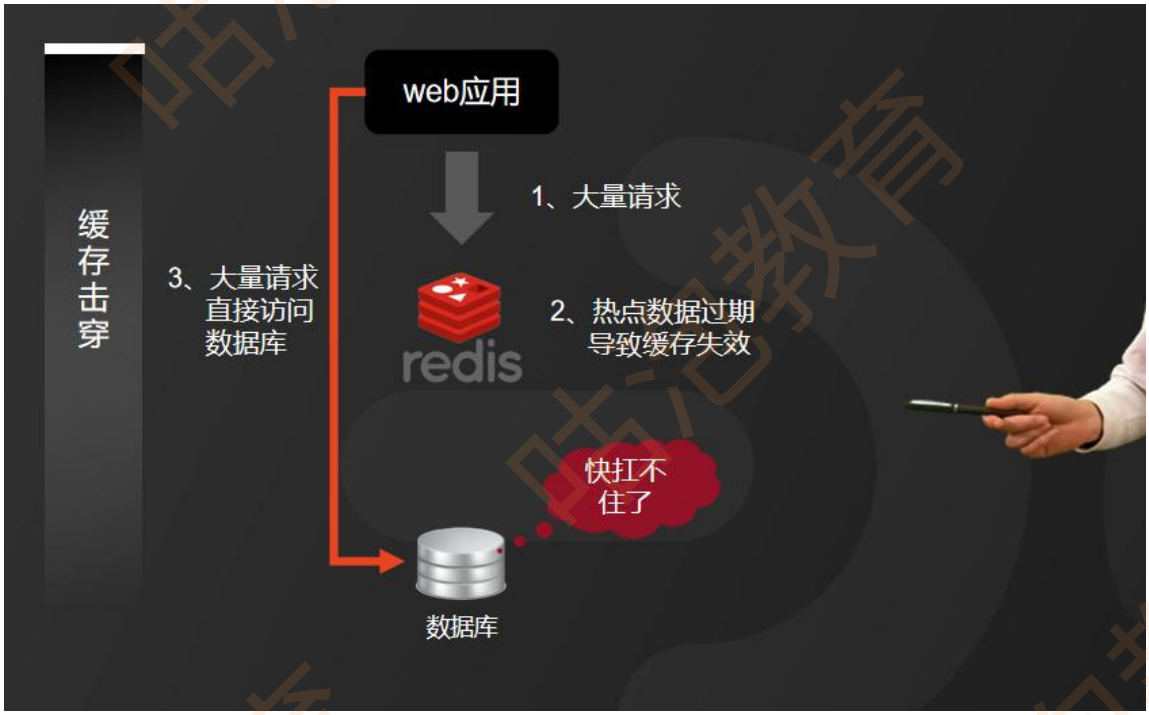
2、将空数据放到缓存中，设置key-null,同时设置短的缓存时间(如30秒)，这样可以防止攻击用户反复用一个id暴力攻击

3、采用布隆过滤器，将所有可能存在的数据哈希存到一个足够大的 中，一些一定不存在的数据会被这个bitmap拦截掉，从而避免了对底层存储系统的查询压力。

【导航条：缓存击穿】

（转场，屏幕黑底白字，正中央出现“4 缓存击穿”）

3.缓存击穿



缓存击穿：是指缓存中没有或者已经到期但是数据库中有的数据，这时由于并发用户特别多，同时在缓存中又没有读到数据，所以直接去数据库中取数据，引起数据库压力瞬间增大。和缓存雪崩不同的是，缓存击穿指并发查询同一条数据，缓存雪崩是不同数据都过期了，大面积数据都查询不到，从而查询数据库。

那如何预防缓存击穿呢？我建议从以下三个方面来进行设置：

1、设置热数据永不过期

2、使用 避免大量请求同时查询DB

3、做好熔断、降级，防止系统崩溃

以上就是我对缓存雪崩、缓存穿透和缓存击穿的解决方案，希望能够给小伙伴们提供参考的思路。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

29．什么是零拷贝，Netty是如何实现的?

【导航条：引言】

呢作为一个高性能的网络通信框架，被越来越多互联网公司关注和重视。最近，有小伙伴在面试过程中被问到Netty是如何实现零拷贝的问题？，今天，我给大家来聊一聊。另外，往期面试题解析中配套的文档我已经准备好，想获得的可以在我的煮叶简介中找到。

我们先来看什么是零拷贝?

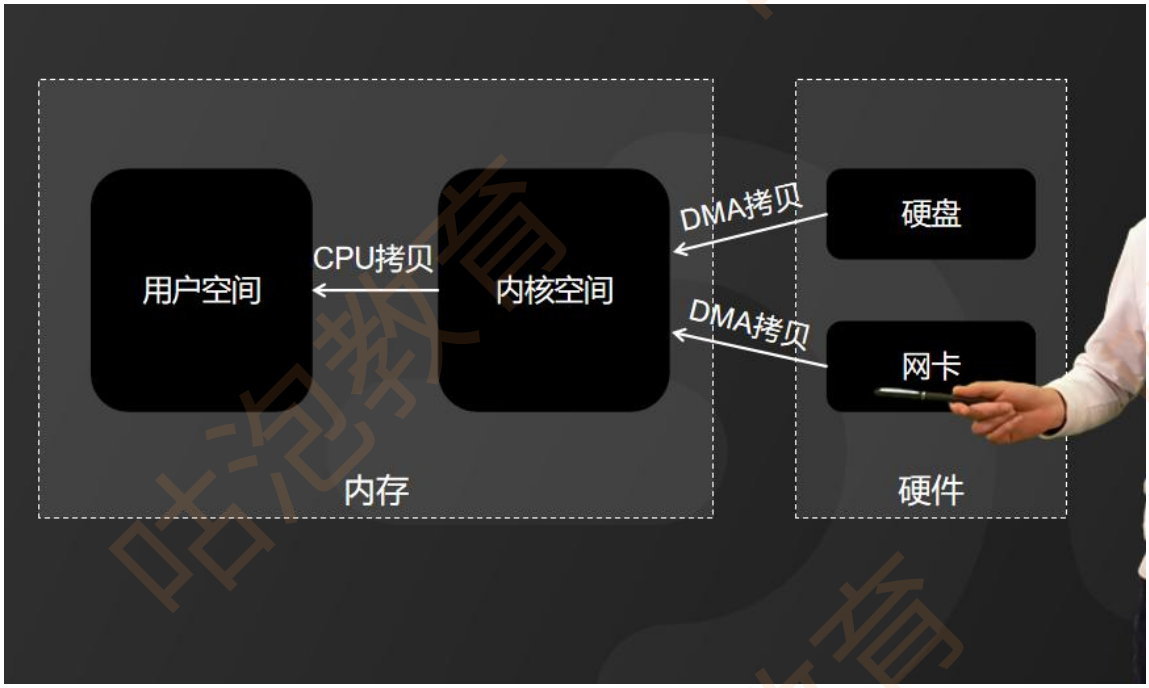
【导航条：什么是零拷贝】

(转场，屏幕黑底白字，正中央出现“1什么是零拷贝”)

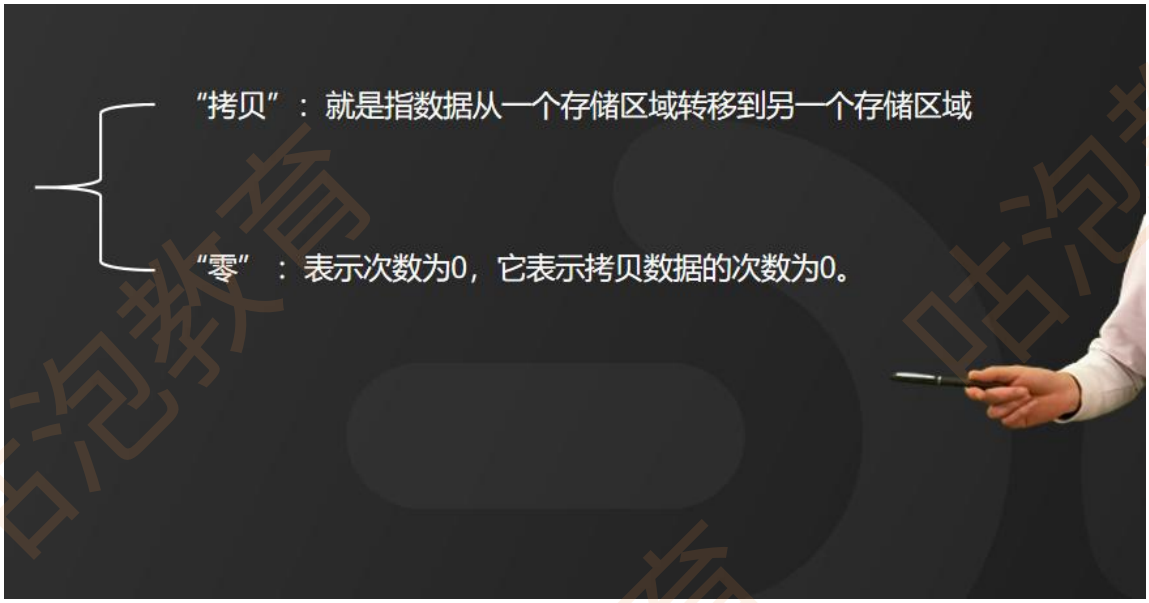
1、什么零拷贝

在计算机中，完成数据传输，要么是通过网络，要么就是通过本地磁盘。通常完成一次完整I/O交互流程分

为两阶段，首先拷贝到系统内核空间，由操作系统来完成；紧接着要拷贝到用户空间，这个由应用程序来完成，具体交互流程如下图所示。



那什么是零拷贝呢？顾名思义，零拷贝的意思包括两个部分，“零”和“拷贝”：



“拷贝”：就是指数据从一个存储区域转移到另一个存储区域。

“零”：表示次数为0，它表示拷贝数据的次数为0。

合起来“零拷贝”就是不需要将数据从一个存储区域复制到另一个存储区域。

【导航条：为什么要零拷贝】

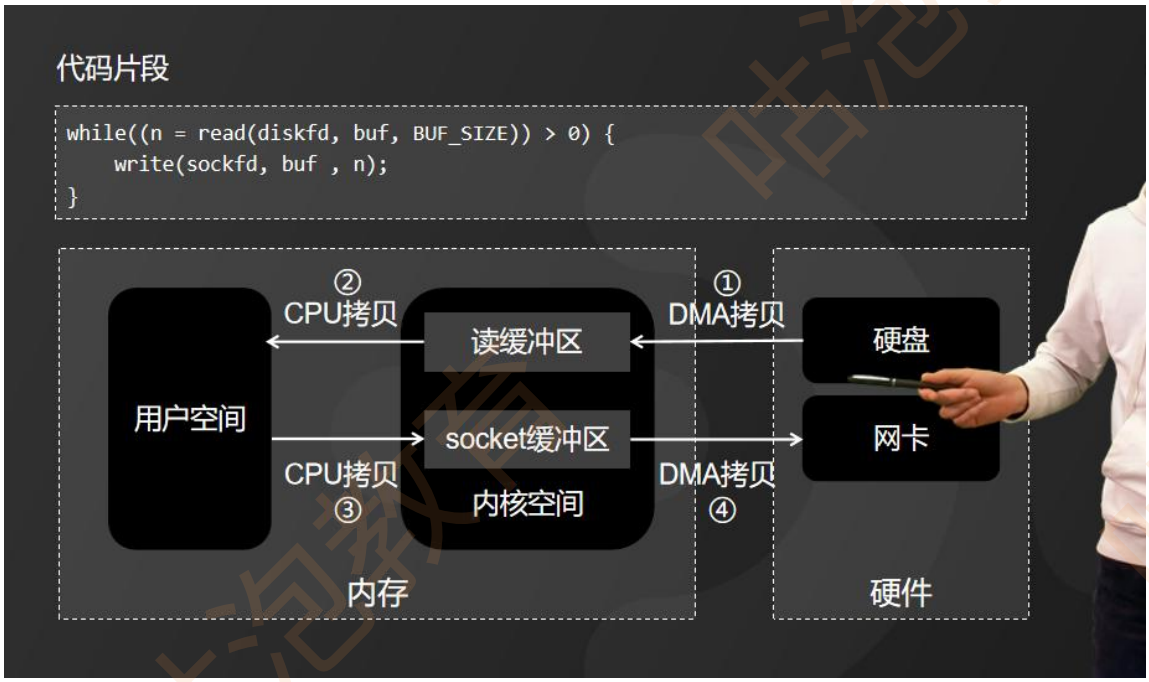
（转场，屏幕黑底白字，正中央出现“1 为什么要零拷贝”）

2、为什么要零拷贝

零拷贝主要是指将系统内核空间的内存和用户空间的内存实现直接关联映射，从而省去了数据传输过程中的来回拷贝，也就是说，要完成数据传输数据拷贝次数为0次。

有这么一段代码，主要将服务端主机磁盘中的文件从已连接的socket发出去。关键实现代码如下：

```
while((n = read(diskfd, buf, BUF_SIZE)) > 0){
    write(sockfd, buf , n);
}
```



以上代码是用传统的IO编写的，它的执行过程大致是这样的：

首先，read()方法，会把数据从磁盘循环读取到内核缓冲区，再拷贝到用户缓冲区。

然后，write()方法先把数据写入到socket缓冲区，最后写入网卡设备。

因为，传统的IO在在数据传输过程中拷贝次数太多，导致性能低下。为了减少拷贝次数，增加系统性能，才有了零拷贝的设计。另外，零拷贝并不是指完全没有文件的拷贝，只是减少了拷贝的次数。

【导航条：在Netty中的实现】

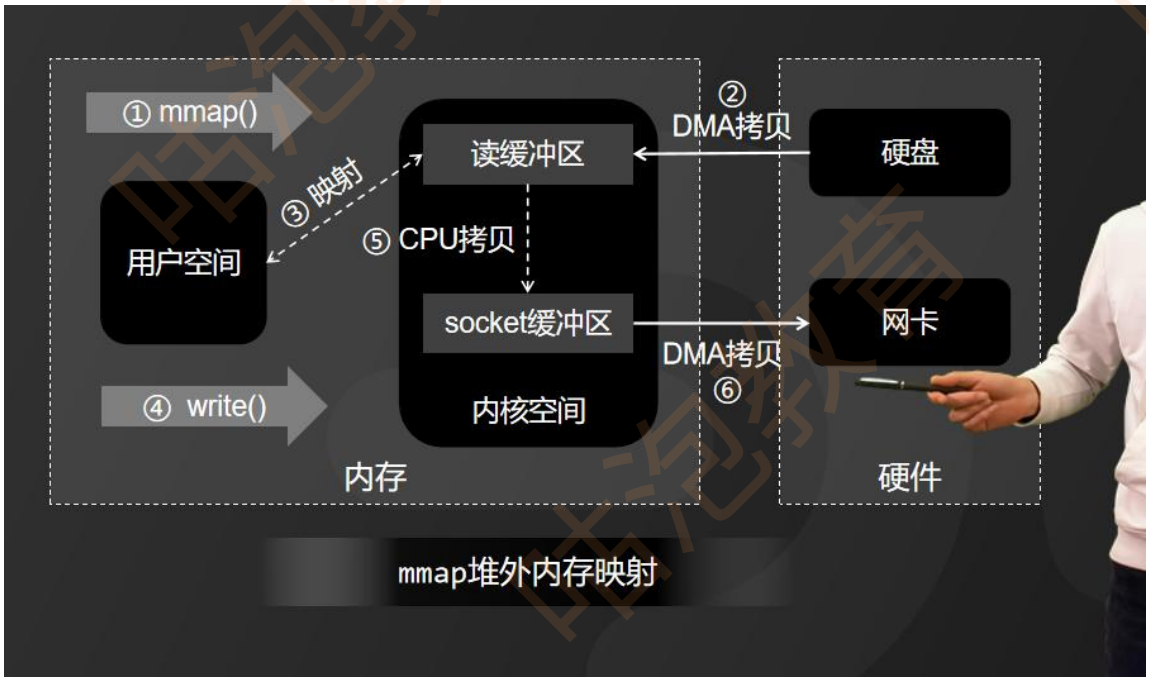
（转场，屏幕黑底白字，正中央出现“1 零拷贝在Netty中的实现”）

3、零拷贝在Netty中的实现

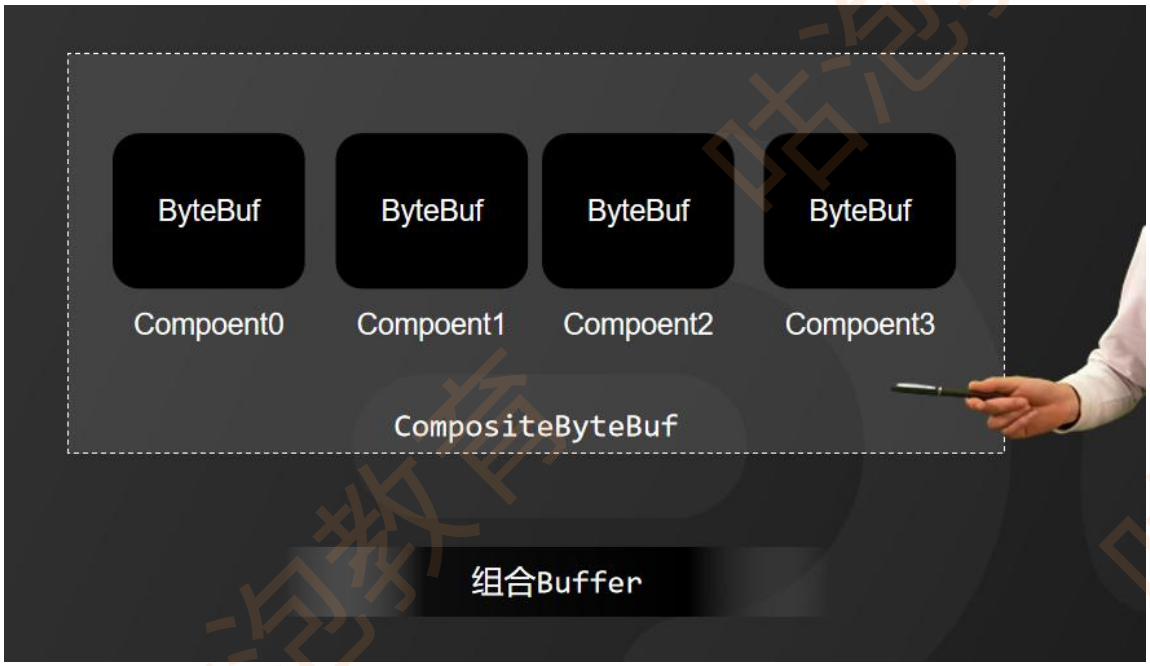
从流程图可以看出，传统IO的读写流程，包括了4次用户态和内核态的切换，也就是4次上下文切换，4次数据拷贝，其中两次CPU拷贝以及两次的DMA拷贝。关于上下文切换和DMA拷贝，在这里我们就不详细赘述了，以后可以单独拍摄一期视频来详细介绍。想了解的小伙伴可以在评论区回复666.

那在Netty中，是如何实现零拷贝的呢？有以下三种方式

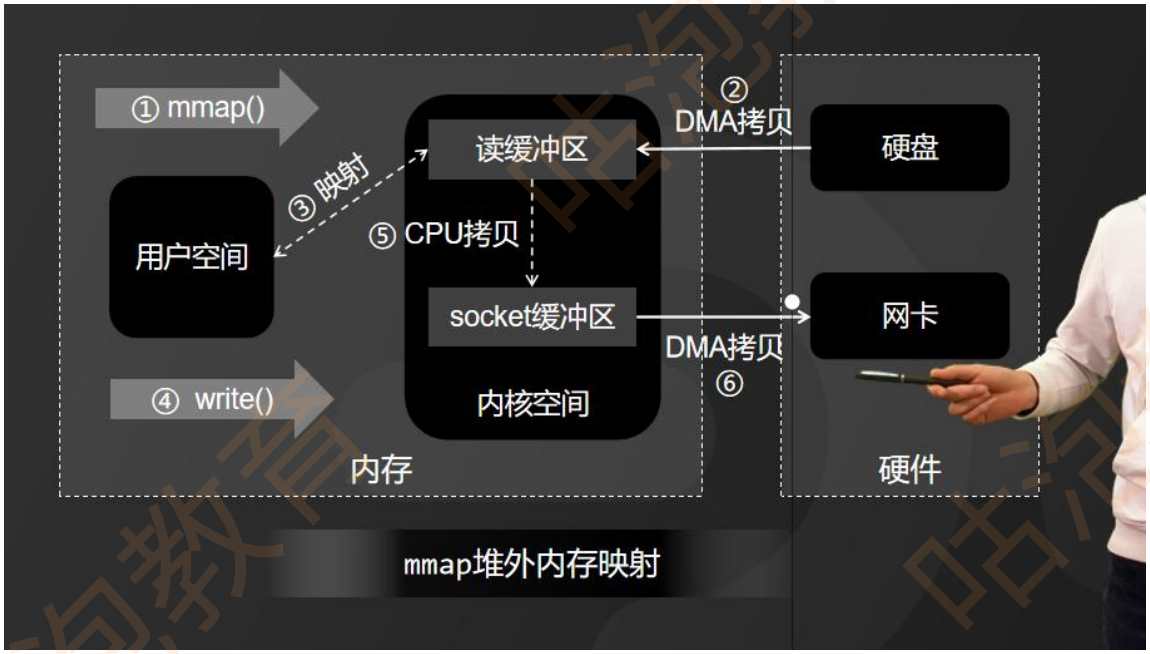
1. 使用堆外内存,也叫直接内存。Netty的接收和发送都采用DIRECT BUFFERS，对应系统底层的mmap机制，直接使用堆外内存进行Socket读写，不需要进行字节缓冲区的二次拷贝。



2. 提供了组合Buffer对象，可以聚合多个ByteBuffer对象，用户只需要像操作一个Buffer一样方便的对组合Buffer进行操作，避免了传统通过内存拷贝的方式将几个小Buffer合并成一个大的Buffer，不需要做内存拷贝。



3. 的文件传输采用了transferTo方法，直接使用了NIO的sendfile机制，它可以直接将文件缓冲区的数据发送到目标Channel，避免了传统通过循环write方式导致的内存拷贝问题。



我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

30．Spring中BeanFactory和FactoryBean的区别?

【导航条：引言】

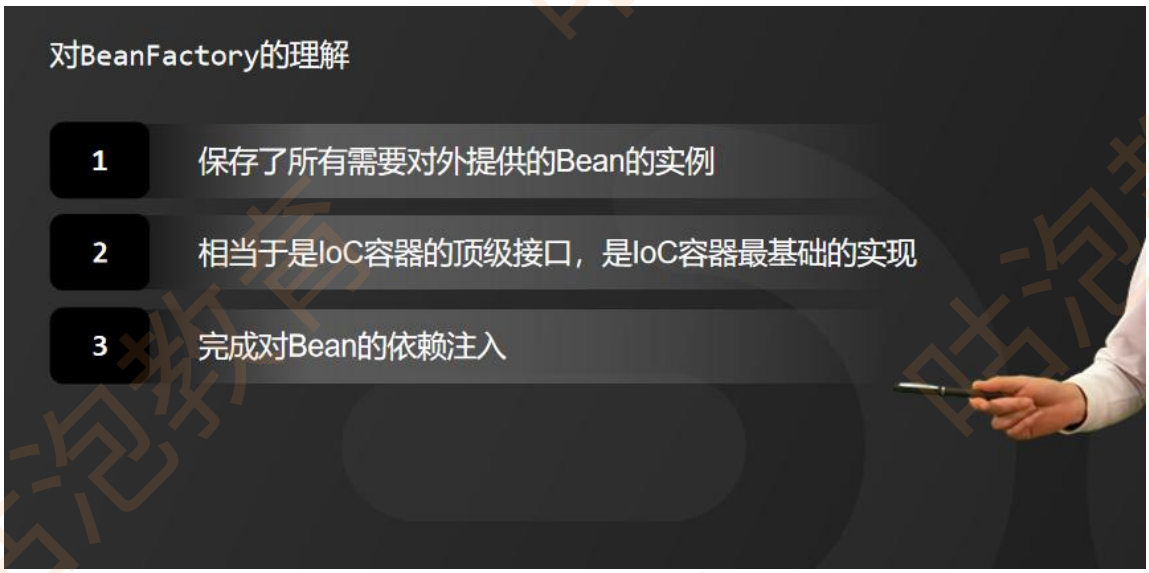
一位工作了4年的小伙伴，去京东面试被问到这样一个问题，Spring中的BeanFactory和FactoryBean有什么区别？因为没有看过源码，当时就感觉这是一个文字游戏，感觉没什么区别？

那今天，我就给大家来聊清楚。另外，往期面试题解析中配套的文档我已经准备好，想获得的可以在我的煮叶简介中找到。好了，我们先来看BeanFactory。

【导航条：BeanFactory】

（转场，屏幕黑底白字，正中央出现“1 BeanFactory”）

1、BeanFactory



在Spring中最核心的就是Ioc容器，它保存了所有需要对外提供的Bean的实例。Spring对外暴露的ApplicationContext作为IoC容器最重要的接口，它也实现了BeanFactory接口。

所以，BeanFactory从名字上来看，它是一个Factory，相当于是IoC容器的顶级接口，是IoC容器最基础的实现。也是提供访问Spring容器的跟接口，主要负责Bean的创建和访问。其中，最主要的一个方法就是getBean()。同时，在BeanFactory中还会完成对Bean的依赖注入，也就是DI。

【导航条：FactoryBean】

（转场，屏幕黑底白字，正中央出现“2 FactoryBean”）

2、FactoryBean

那FactoryBean呢？从名字上来看它是一个特殊的Bean，这个Bean可以返回创建Bean的工厂。如果FactoryBean接口，可以根据不同的配置类型返回不同类型的Bean，它有一个比较核心的方法叫做getObject()。

FactoryBean在使用上也有些特殊。在前面介绍的BeanFactory中，有一个常量叫做FACTORY_BEAN_PREFIX = “&”，当我们想要从Spring容器中获取创建Bean的Factory时，可以在beanName前面加上 & 符号就可以获得Bean对应的Factory。

我们也可以自定义FactoryBean，用它来扩展创建Bean的规则。在Spring中创建的AOP动态代理Bean，就是通过实现FactoryBean来实现的。我们现在用的Spring Boot、Spring Cloud也经常通过实现FactoryBean接口实现自定义组件，从而修改默认的创建Bean的规则。

【导航条：总结】

（转场，屏幕黑底白字，正中央出现“3 总结”）

3、总结

以上，就是我对BeanFactory和FactoryBean的理解，回答这个问题啊，其实只要接触过一点点的Spring源码就能讲明白。所以，在内卷这么严重的环境下，工作3年以上，就应该看看源码了，这样才能提高核心竞争力。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

31．一次性聊透JVM架构设计，就算八股文也得会

【导航条：引言】

有位小伙伴在我的粉丝群里面问我一个面试题，说面试被问对JVM的理解，不知道怎么回答，今天咱们来聊透，就算是八股文你也得会。另外，往期面试题解析中配套的文档我已经准备好，想获得的可以在我的煮叶简介中找到。

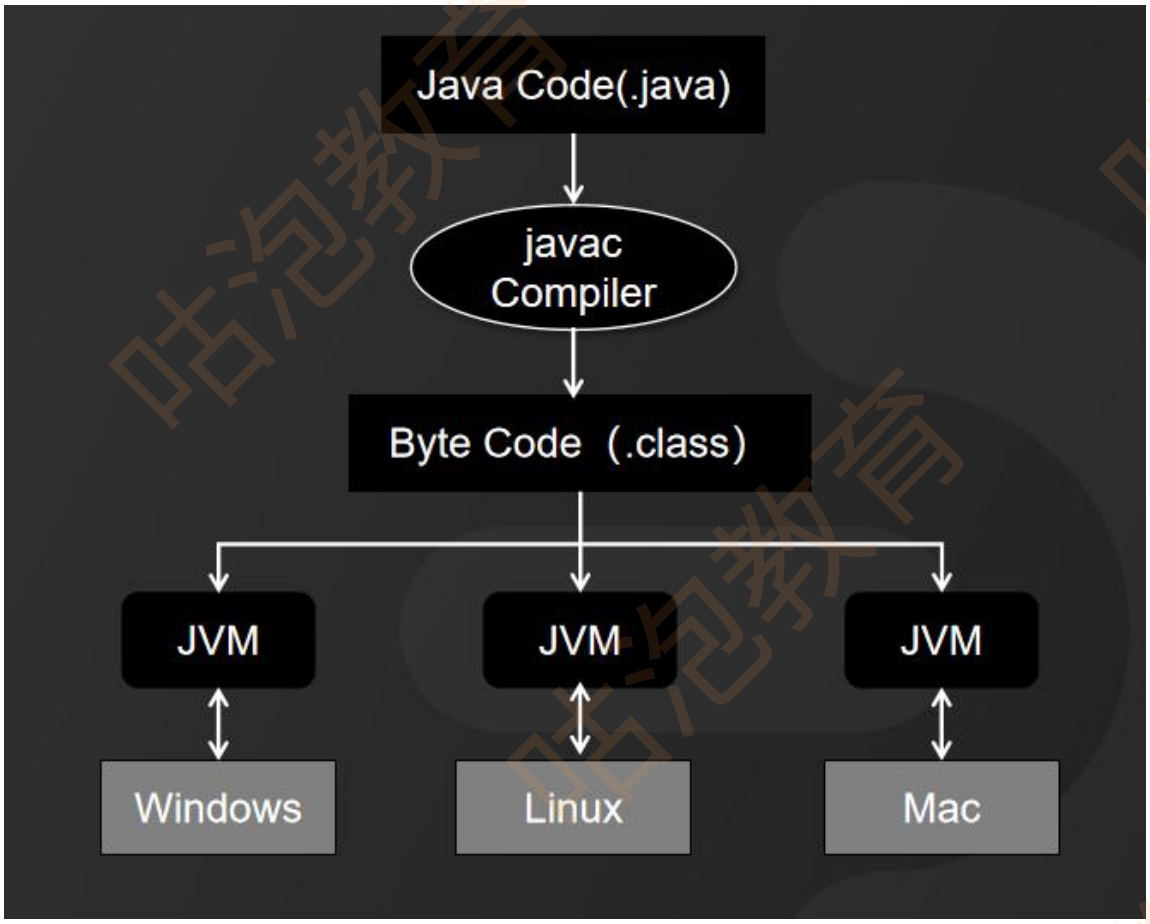
先来看什么是JVM?

【导航条：什么是JVM】

（转场，屏幕黑底白字，正中央出现“1 什么是JVM”）

1、什么是JVM

JVM（Java Virtual Machine）其实是一套标准。通过定义虚拟机，像真实计算机一样，能够运行字节码指令。JVM的好处是可以屏蔽操作系统的细节， 使Java可以一次编写，到处运行。



实现JVM的厂商有很多，比如Hotspot、JRockit、IBM J9等等。今天我们重点来聊一聊主流的Hotspot，因为Oracle JDK与OpenJDK都是采用HotSpot VM。从源码层面说，它们俩基本上没什么区别。

【导航条：JVM架构设计】

（转场，屏幕黑底白字，正中央出现“2 JVM架构设计”）

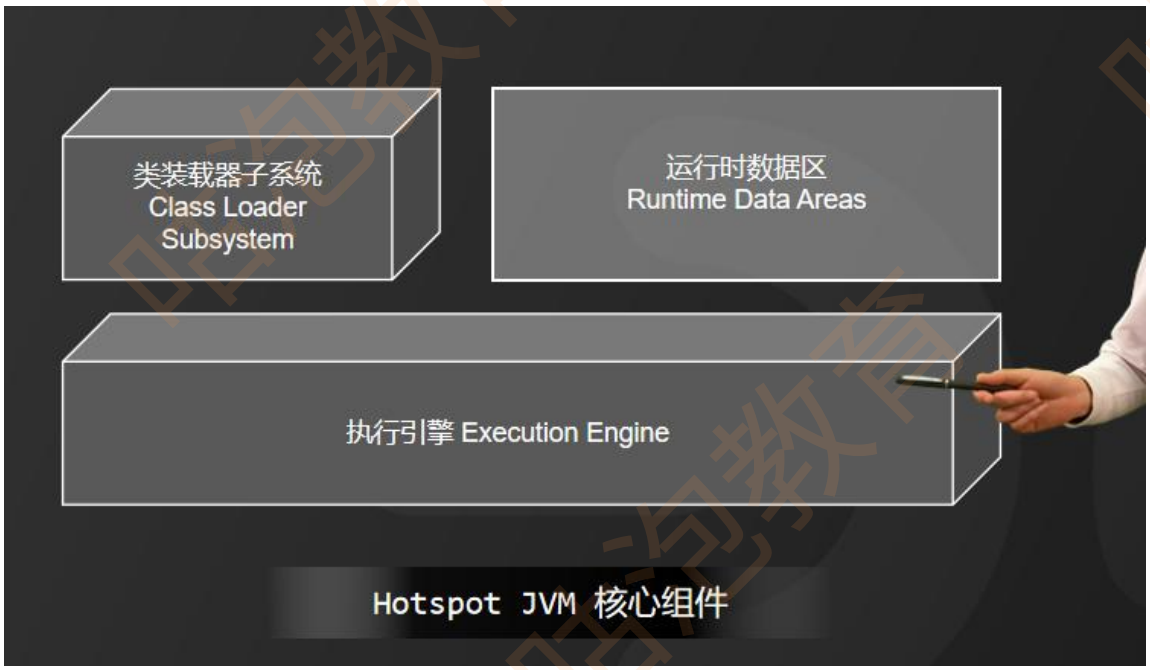
2、JVM架构设计

下面我给大家详细介绍一下JVM的架构设计，总体来看HotSpot VM 主要由3个核心部分组成：

1、类装载子系统（Class Loader Subsystem）

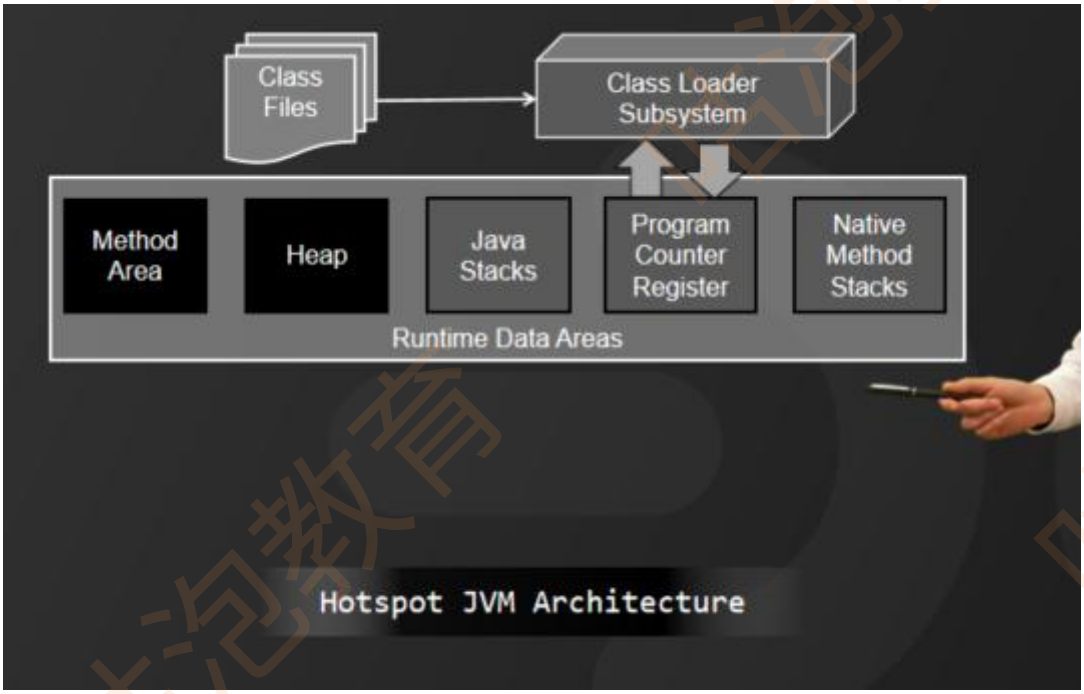
2、运行时数据区（Runtime Data Areas）

3、执行引擎（Execution Engine）

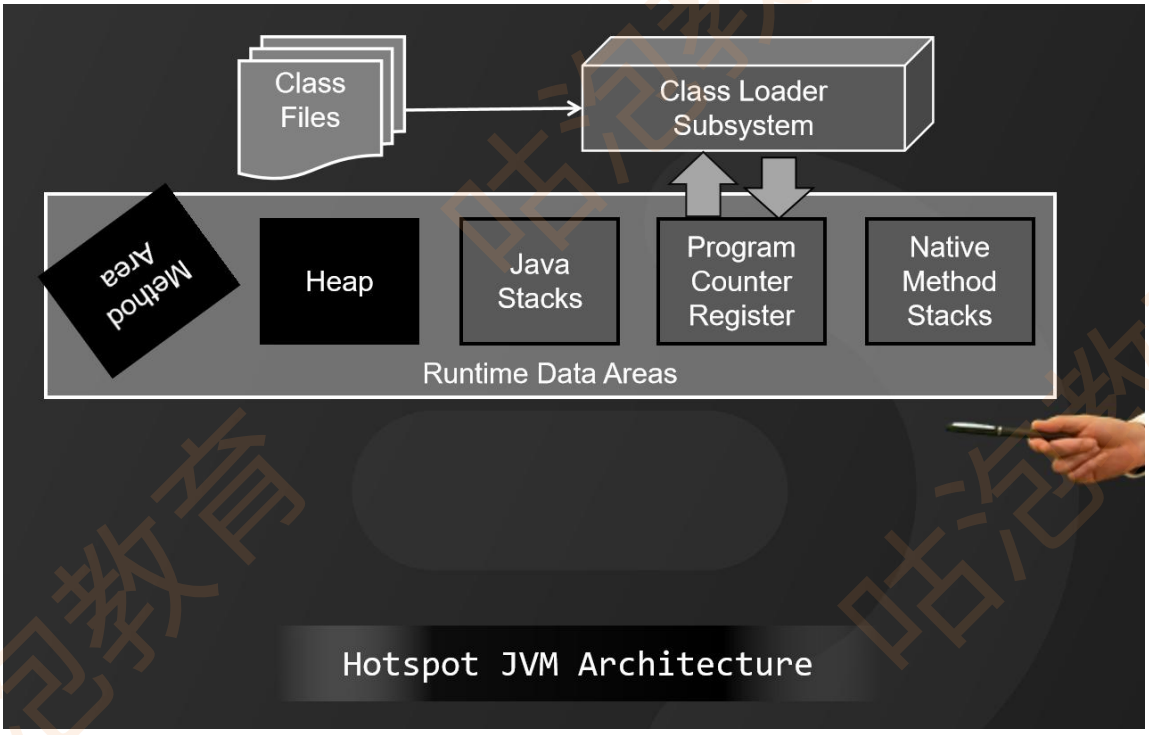


那么Hotspot JVM架构细节和运行机制又是怎样的呢？首先，将编译好的.class文件装载到类加载子系统，它的主要功能是查找并验证类文件、完成相关内存空间的分配和对象赋值。

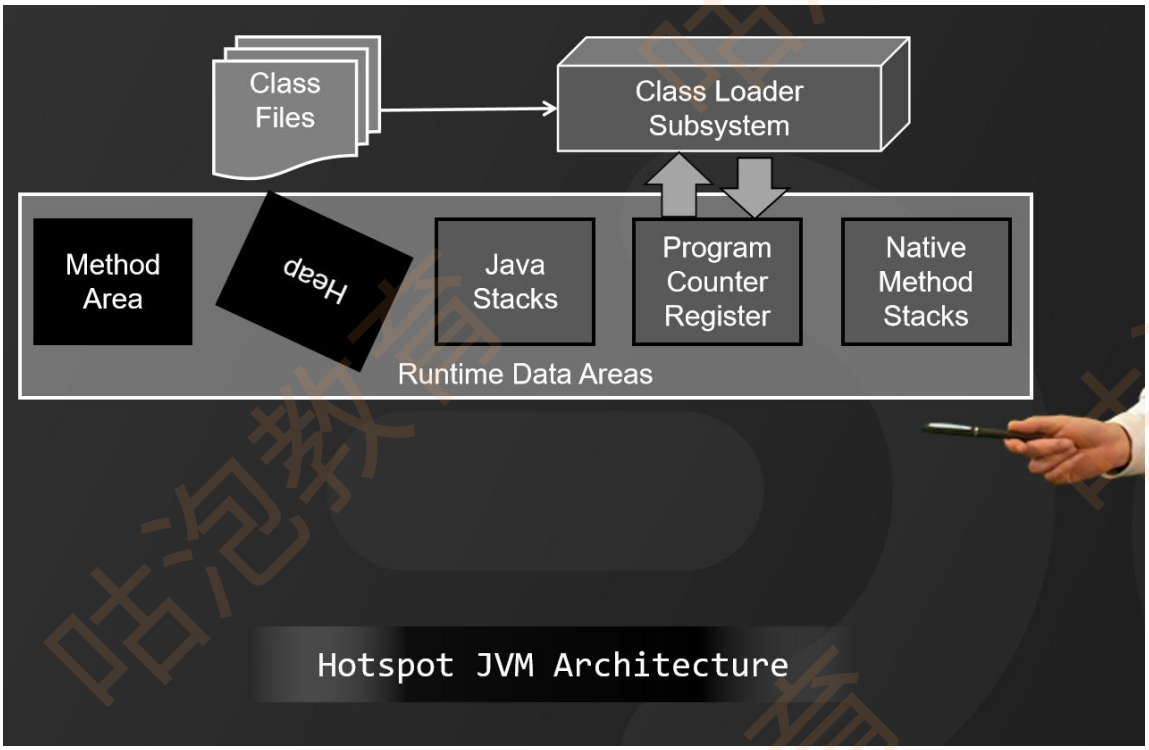
类文件加载到内存之后由运行时数据区来完成数据存储和数据交换。运行时数据区又分为线程共享内存区和线程隔离内存区。线程共享内存区包括方法区和堆区，它们是程序员能够通过编写代码直接操作的内存区，而线程隔离内存区包括栈区、程序计数器和本地方法栈，它们是完全由JVM来调度的内存区域。



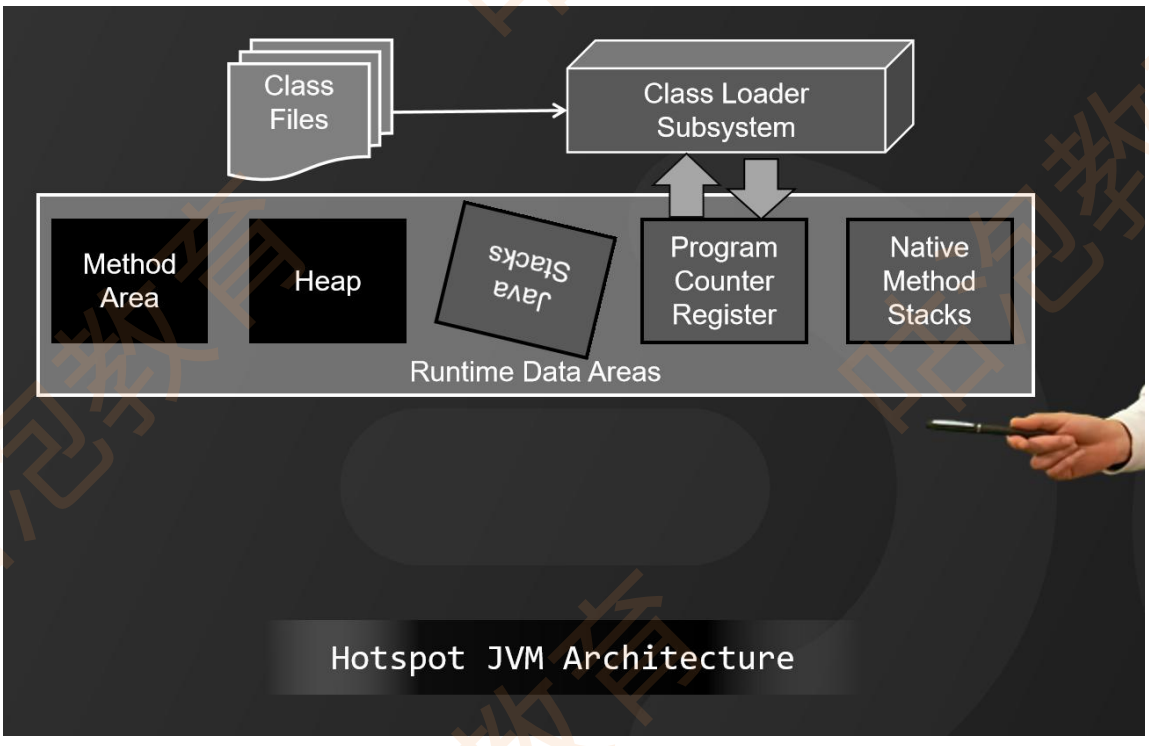
首先来看方法区，它的主要功能是存储运行时常量池、字段和方法的元数据和类的的元数据。



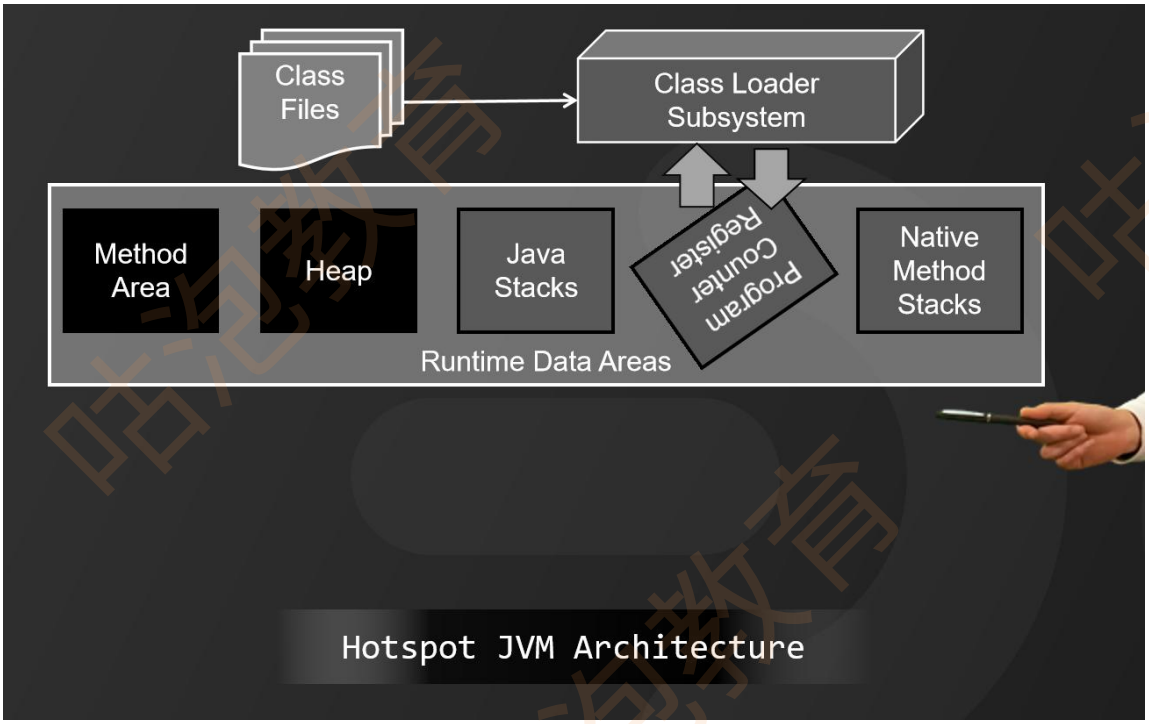
而堆区呢，主要是用来存储Java对象的实例，也就是我们new的类都存在堆区。



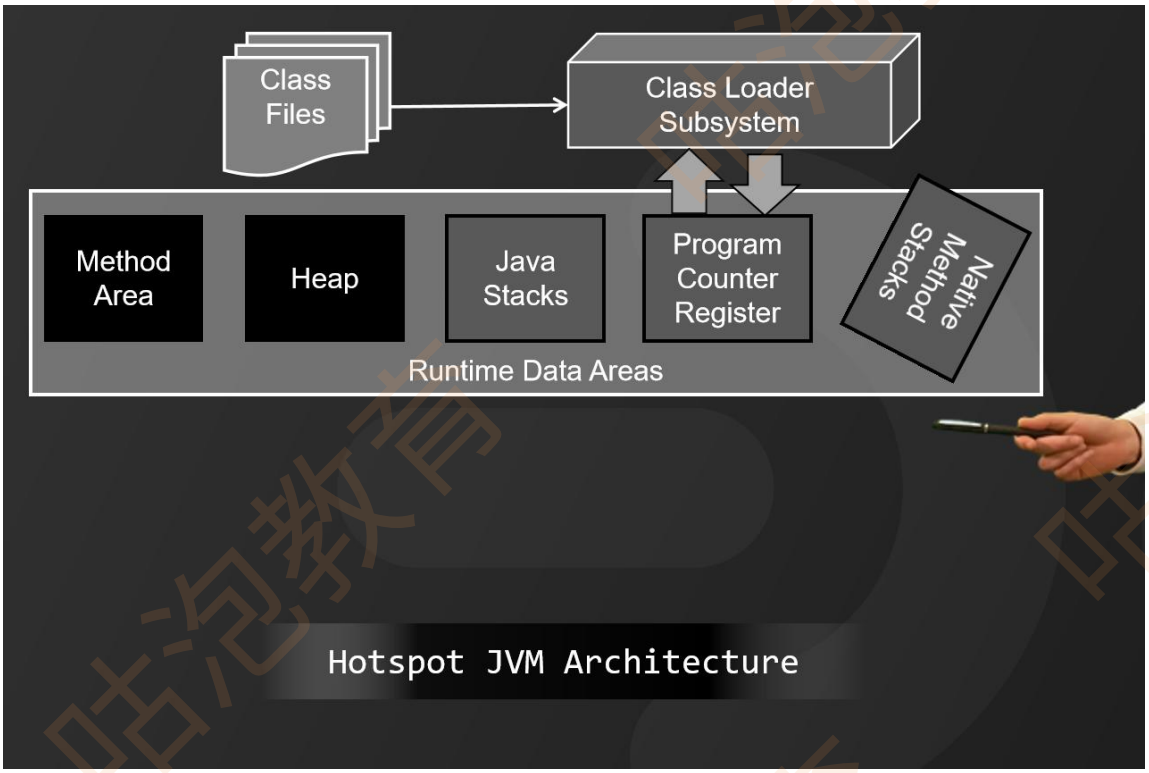
栈区是通过线程的方式运行来加载各种方法。



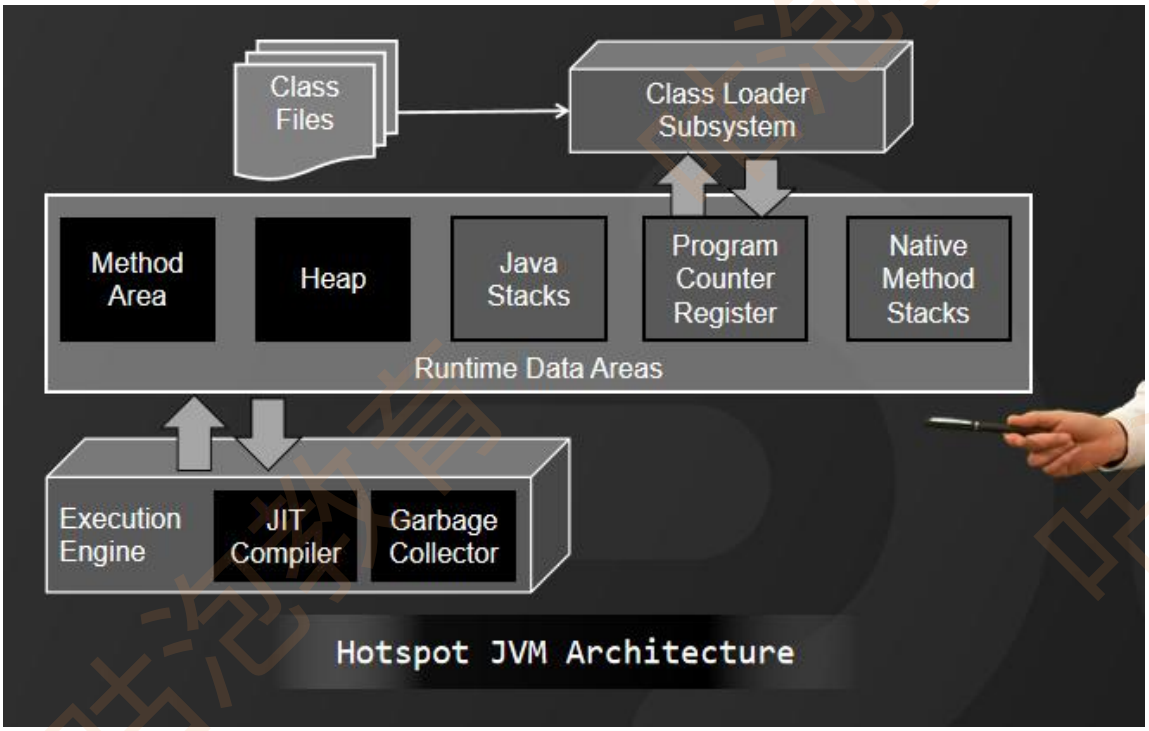
程序计数器呢，是负责保存每个线程执行的方法的地址。



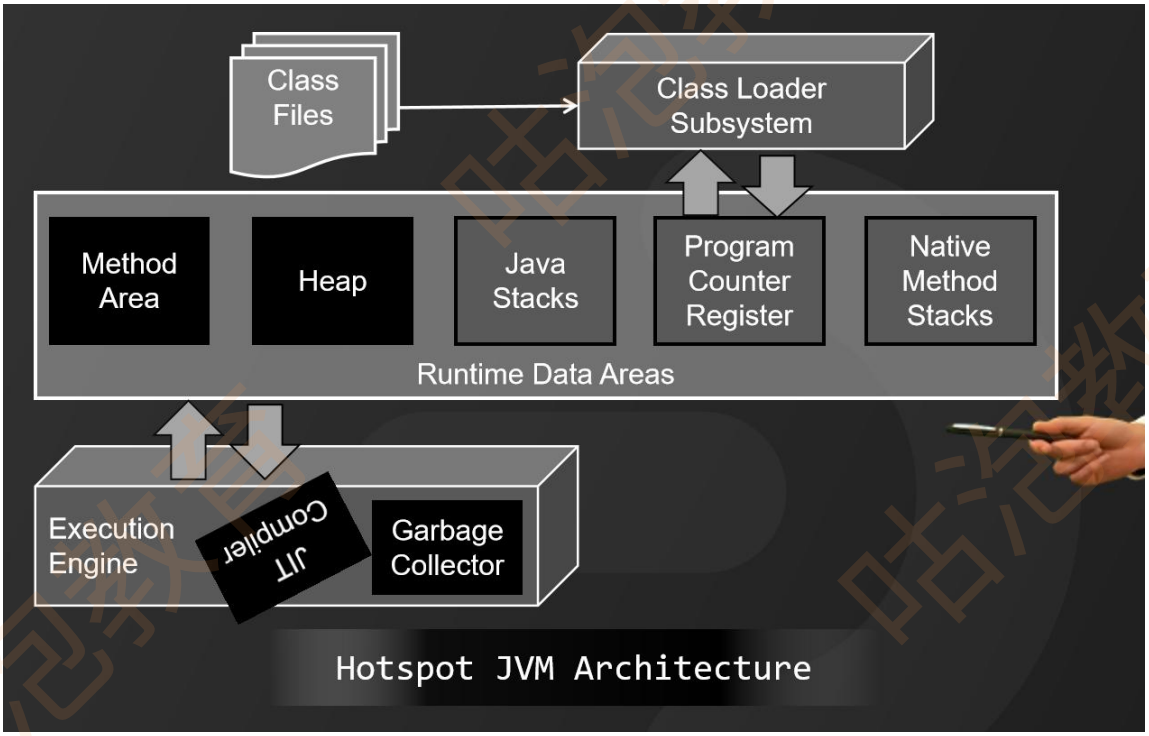
本地方法区是负责加载并运行native类型的方法，



这样，通过运行时数据区的五个内存区就能完成Java程序程序逻辑的执行和数据交换。接下来看执行引擎，它主要包含即时编辑器和垃圾回收器。

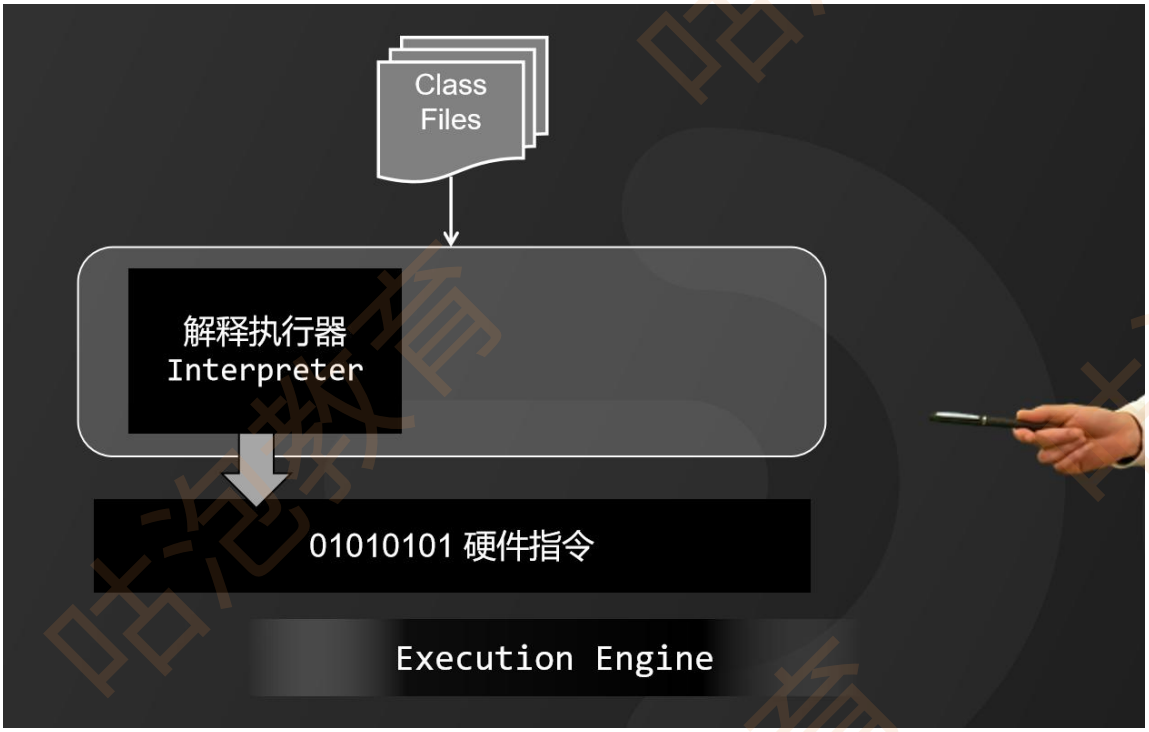


即时编译器，通俗地理解就是用来将字节码翻译成操作系统能够执行的CPU指令，可以通过JVM参数来设置选择解释执行或者是编译执行。

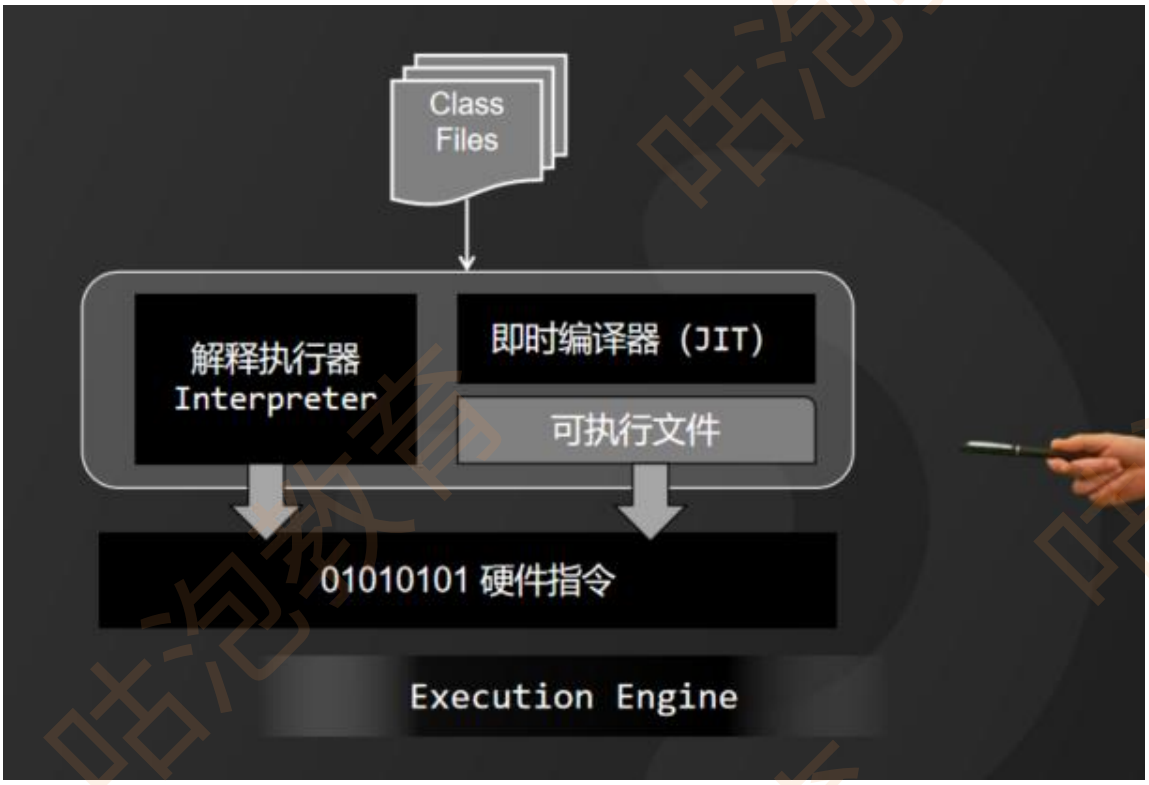


所谓解释执行就是直接将字节码作为源程序输入解释执行，不必等待编译器全部编译完成再执行，这样可

以省去许多不必要的编译时间。

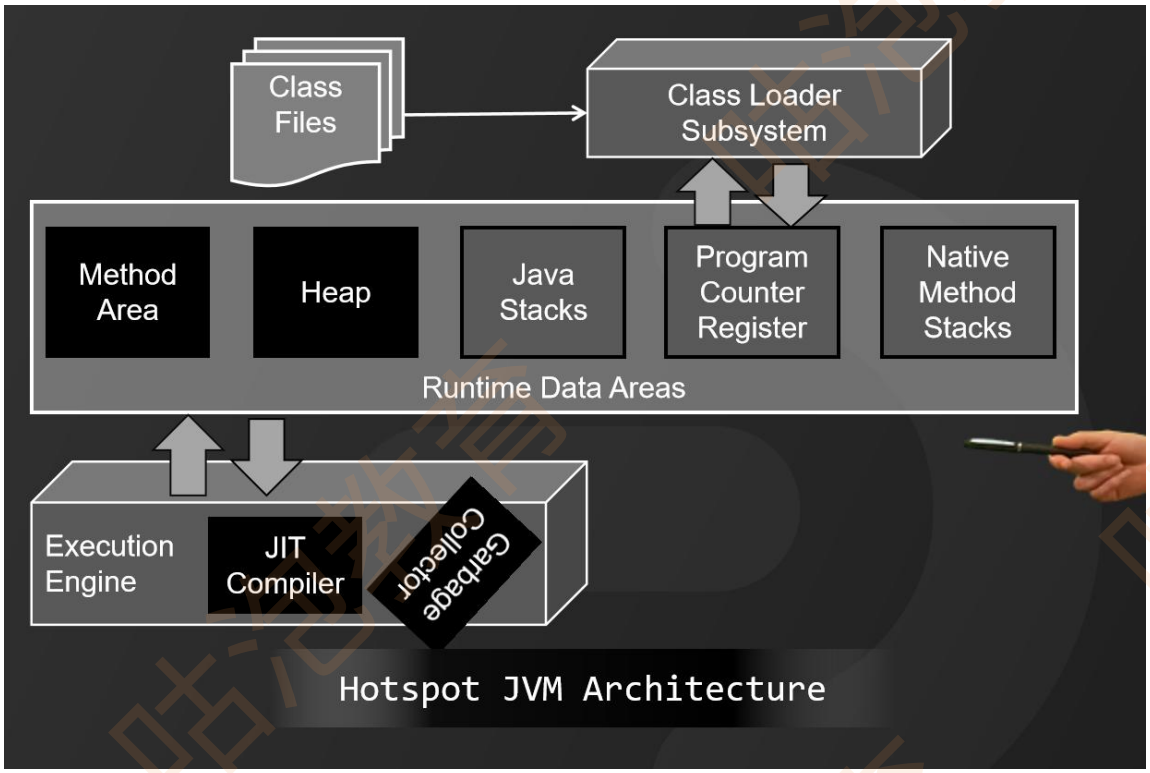


而编译执行就是就是由编译程序将目标代码一次性编译成目标程序，再由机器运行，执行效率更高，占用内存资源也更小

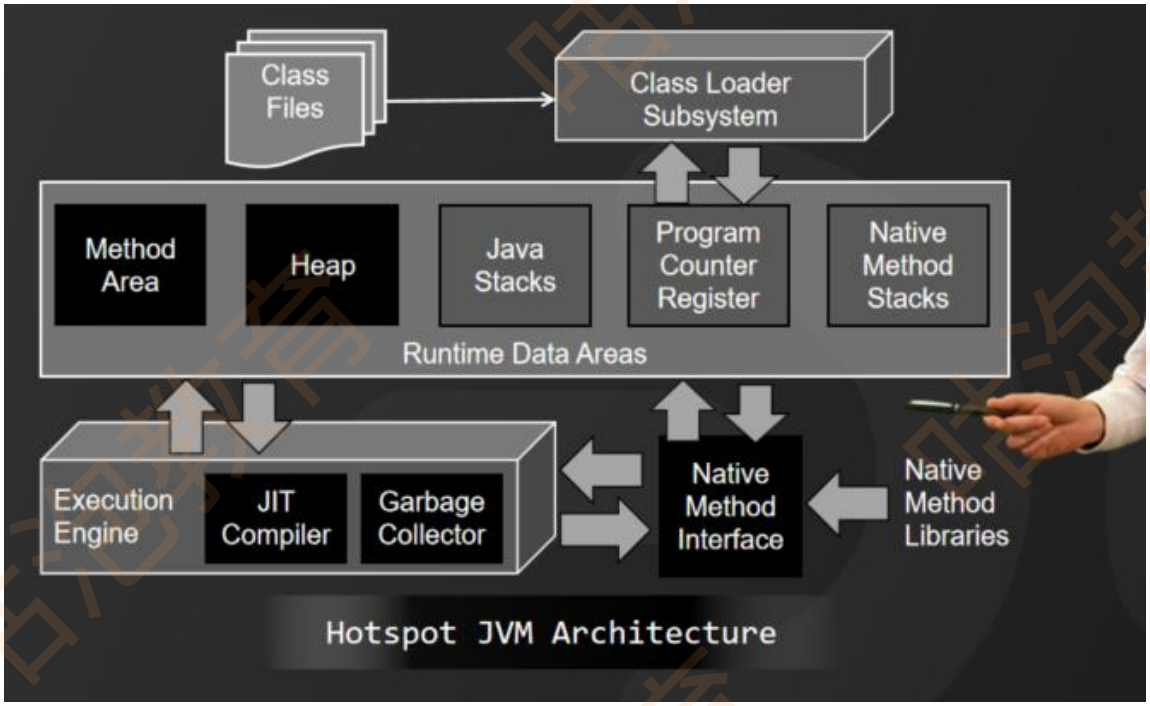


在Hotspot的实现中默认是两种方式的组合。

垃圾回收器主要负责对运行时数据区的数据进行管理和回收，其实就是对各种垃圾回收算法的实现，总体来说有三种核心算法，分别是复制算法、标记清除算法和标记整理算法，这些算法的选择呢，我们可以通过JVM参数来设置。



最后，来看本地方法接口，也就是JNI技术。我们可以通过JNI来查找并调用C或C++实现的代码，还可以调用操作系统的动态链接库（DLL）等等。



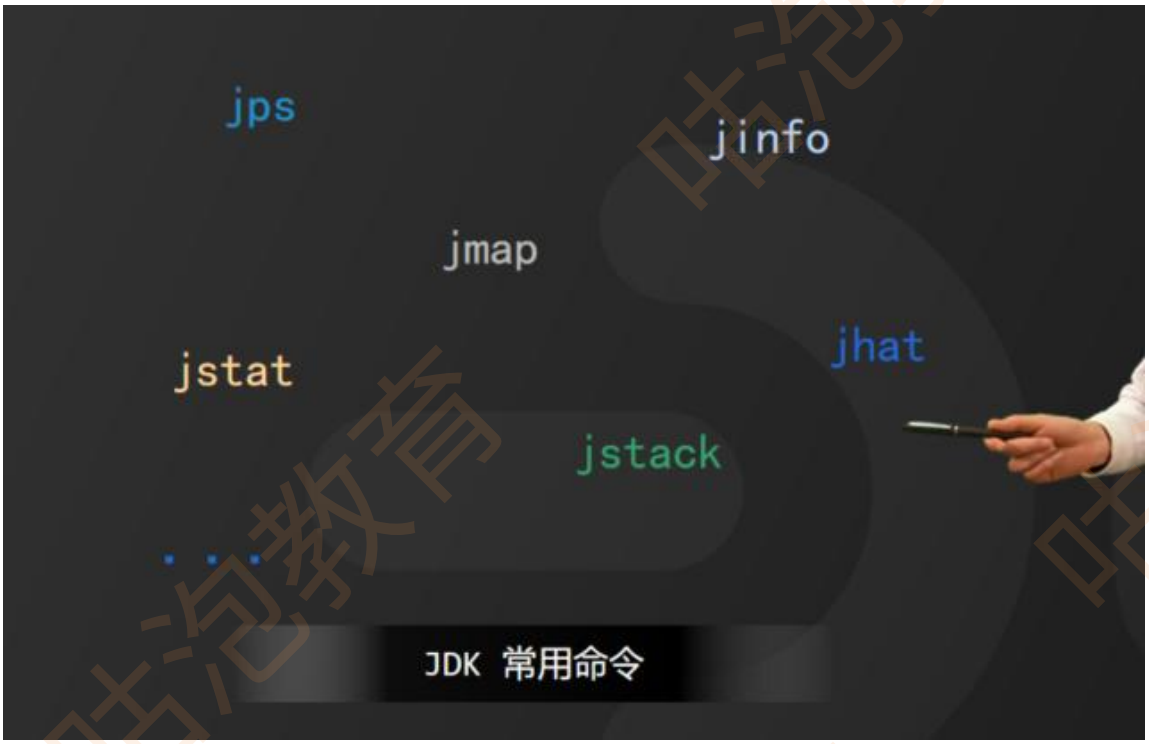
【导航条：总结】

（转场，屏幕黑底白字，正中央出现“3 总结”）

3、总结

好了，通过对Hotspot架构的分析，相信各位小伙伴已经非常清晰地知道了JVM的运行原理。当然，在实际的开发过程中，我们可以通过配置JVM参数来对JVM进行调优，比如这些参数。

还可以通过一些常见的JDK命令来分析JVM的状态，查找问题的原因从而完成对JVM的调优，比如这些命令。



以上就是我对JVM的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

32．什么是代理，为什么要用动态代理?

【导航条：引言】

一说到代理，大家可能都会联想到JDK Proxy、Cglib、AOP等等，这些都算是动态代理。另外，我花了1个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

为什么我们需要动态代理呢？在解答这个问题之前，先跟大家聊清楚什么是静态代理。

【导航条：从静态代理到动态代理】

（转场，屏幕黑底白字，正中央出现“1 从静态代理到动态代理”）

1、从静态代理到动态代理

静态代理呢是代理模式的一种实现形态。代理模式的官方原文是这样的:

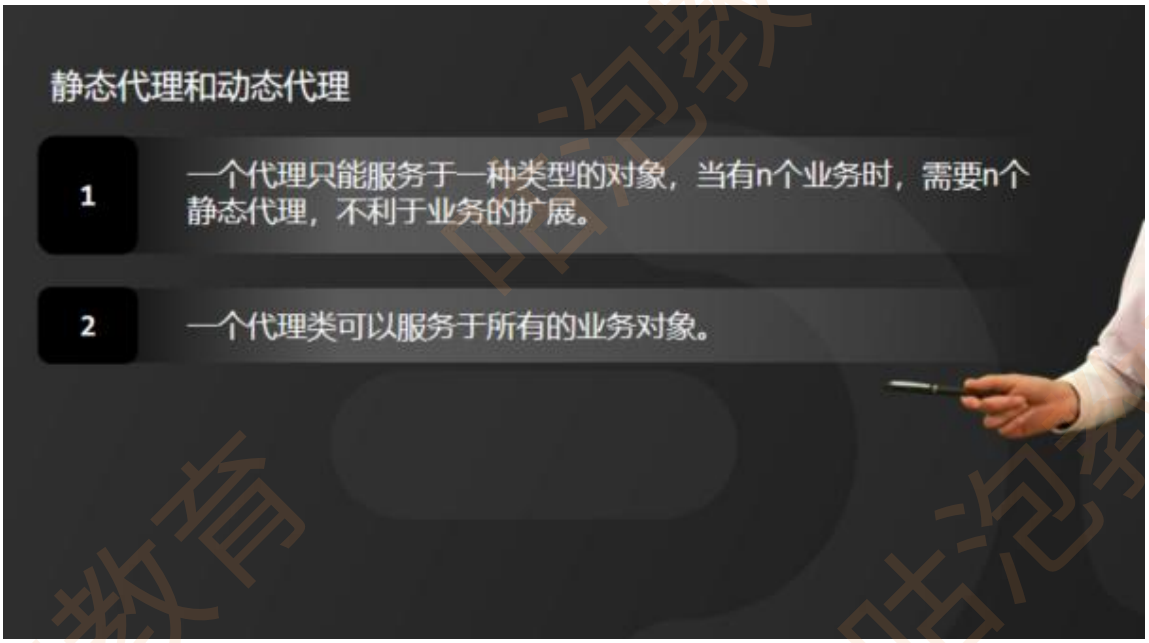
大致意思是，为其他对象提供一种代理项或占位符，以控制对这个对象的访问。什么意思呢？代理模式最核心的思想是创建一个代理对象作为在客户端和目标对象之间中介。主要目的是保护目标对象和增强目标对象。

那什么是静态代理？就相当于是这个代理对象需要手动来创建，而且一个代理对象只能服务于一种类型的对象，那么，当有n个业务的目标对象时，就需要n个静态代理，这样就不利于业务的扩展。

（配图，程序员张三、张老三、李四、媒婆）

举一个例子，有个程序员张三，平时加班比较多，加上公司的异性同事又少，也不太善于和身边的人沟通，自己又缺 少人脉和资源，所以一直没有找对象。这个时候张三的父亲张老三就着急了，利用自己的人脉，经常给张三安排相亲。那张老三就是张三的代理相亲对象。

那么，问题来了。一般情况下，张老三是不会给李四去安排相亲的，因为张老三是不会关心李四有没有对象这个事情的，只有李老四才会关心李四有没有对象。这个时候，就出现一个新的职业 -- 媒婆。媒婆会关心张三、李四、王五、赵六是否有对象，媒婆就成了所有有相亲需求的代理对象，这种一个代理对象就可以服务于所有相关的业务对象，就相当于动态代理对象。



那为什么需要动态代理呢？我总结为以下两个原因：

1、可以减少代理对象的个数，降低程序复杂度。

2、易于复杂业务的动态扩展。

【导航条：动态代理实现原理】

（转场，屏幕黑底白字，正中央出现“3 动态代理实现原理”）

2、动态代理实现原理

那在Java中动态代理的实现原理是怎样的呢？以JDK Proxy为例，我归纳为以下5个步骤：

第1步：拿到目标对象的引用，并且通过反射获取目标对象的所有接口

第2步：重新生成一个新的代理类，实现目标类所有的接口方法；

第3步：把增强的逻辑代码加入到新生成的代理类源代码中。

第4步：动态编译代理类的源代码并生成.字节码，也就是class文件。

第5步：加载并执行新生成的代理对象。

目前，主流动态代理API有两种，分别是JDK Proxy和CGLib，CGLib的实现原理我就不详细赘述了，如果想详

细了解的小伙伴可以在评论区回复666，我可以单独再拍一期视频详细介绍。好了，最后，还是给大家介绍一下CBLib和JDK Proxy的区别。

【导航条：CGLib和JDK Proxy对比】

（转场，屏幕黑底白字，正中央出现“3 CGLib和JDK Proxy对比”）

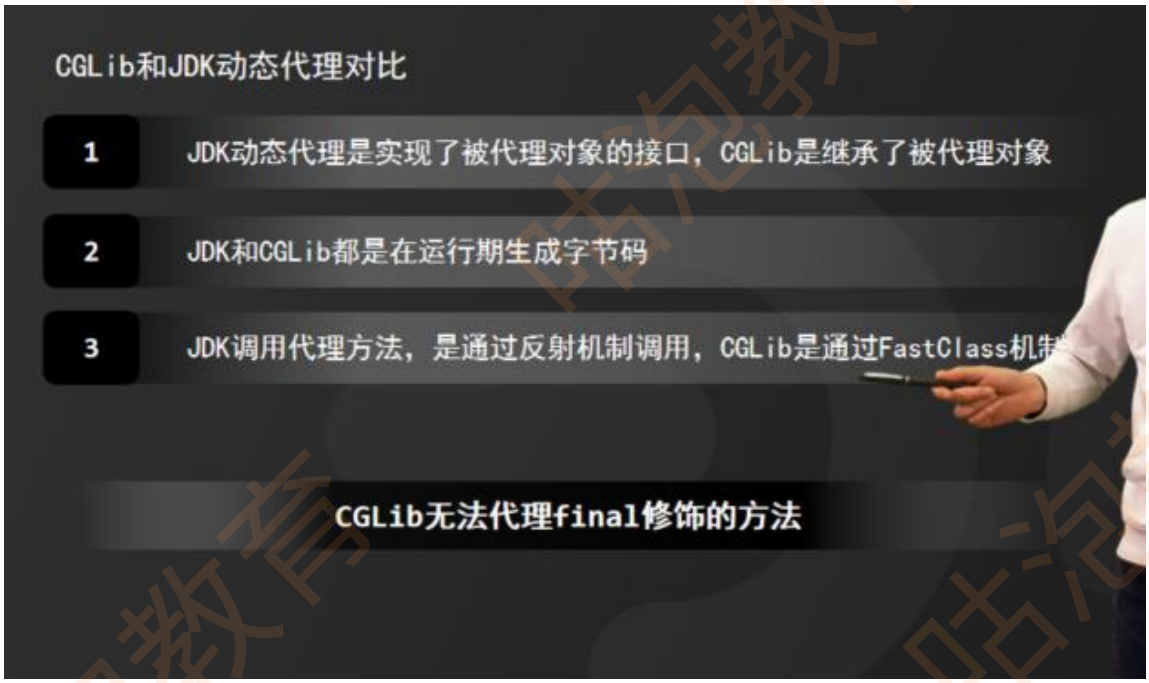
3、JDK Proxy和CGLib对比

关于JDK Proxy和CBLib的区别我归纳为以下3点：

第1点：JDK Proxy是实现目标对象的接口，而GGLib是继承目标对象

第2点：JDK Proxy和CGLib都是在运行期生成字节码。

第3点：JJDK Proxy是通过反射调用目标对象的方法，而CGLib是采用FastClass机制来调用



【导航条：结尾】

好了，以上就是我对动态代理的理解，你会了吗？

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

33．谈谈你对深克隆和浅克隆的理解

【导航条：引言】

一个工作了5年的程序员私信我说，前几天去阿里面试被问到这样一个这样的面试题，说谈谈你对深克隆和浅克隆的理解。他回答说深克隆是克隆值，浅克隆是克隆引用，当时他只说了这样一句话，回答完以后，他看到面试官的表情很诧异，面试也没有继续深入追问了。小伙伴们，如果是你来回答，你也会这样回回答吗？

这位小伙伴的回答并没有错，只是面试官觉得回答得有点抽象而已。今天，我给大家详细聊一聊。

另外，我花了1个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

说到克隆啊，我还不得不说到一个设计模式，原型模式。先来看原型模式的定义。

【导航条：原型模式定义】

（转场，屏幕黑底白字，正中央出现“1 原型模式定义”）

1、原型模式定义

原型模式的官方原文是这样的：

大致意思是，指原型实例指定创建对象的种类，并且通过复制这些原型创建新的对象。其实复制、克隆、拷贝表达的都是同一个意思。因为Java中的数据类型分为引用类型和值类型，因此，在克隆数据的过程中，就有了深克隆和浅克隆。

【导航条：深克隆和浅克隆】

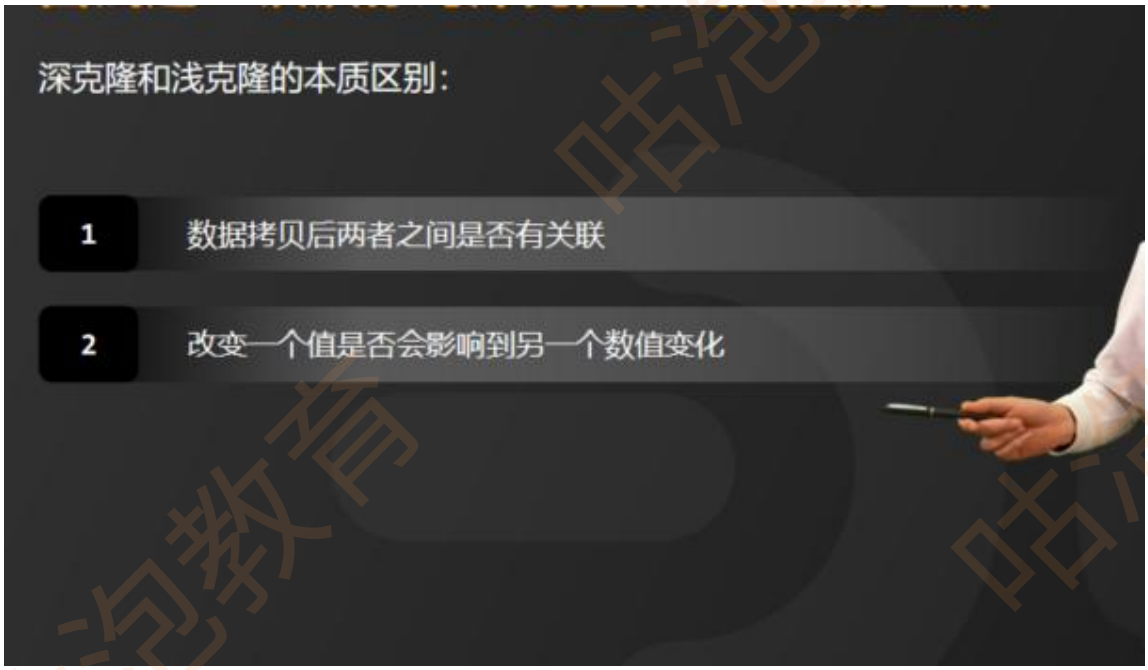
（转场，屏幕黑底白字，正中央出现“2 深克隆和浅克隆”）

2、深克隆和浅克隆

那么，深克隆和浅克隆的本质区别又是什么呢？我归纳为以下两点：

第1点：看数据拷贝后两者之间是否有关联。

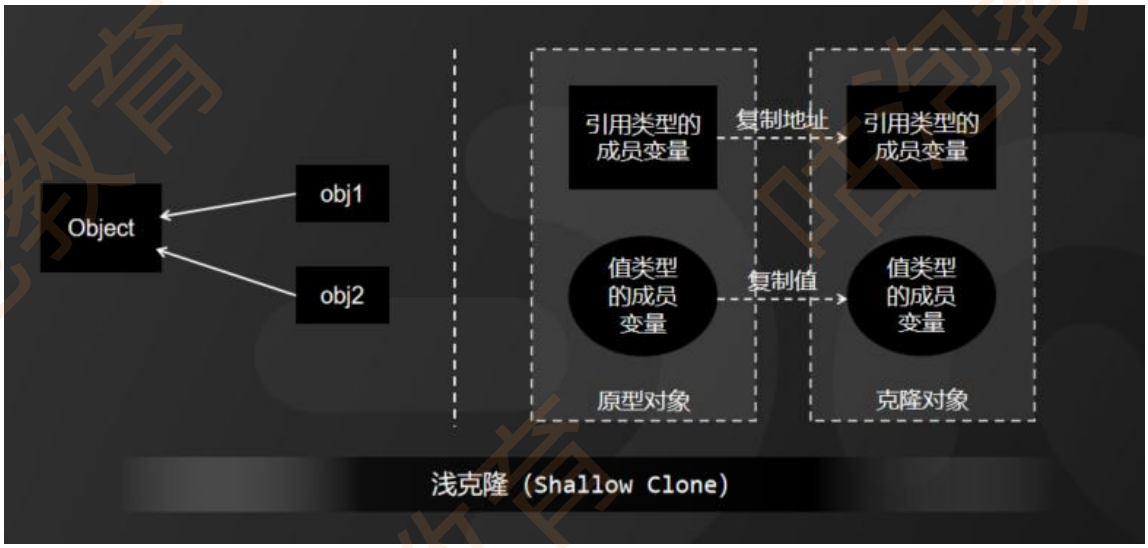
第2点：改变一个值是否会影响到另一个值变化。



那么，到底什么是深克隆，什么是浅克隆呢?

先来看浅克隆，浅克隆就是数据拷贝后，一方数据变化另一方会跟着变化。

如图所示：有原型对象Object，它是引用类型，然后拷贝后产生两个克隆对象obj1和obj2，只是它们克隆的是Object的内存地址，因此ob1和obj2都指向Object，那么Object中的成员变量值发生改变时，obj1和obj2指向的成员变量值也就会发生改变。

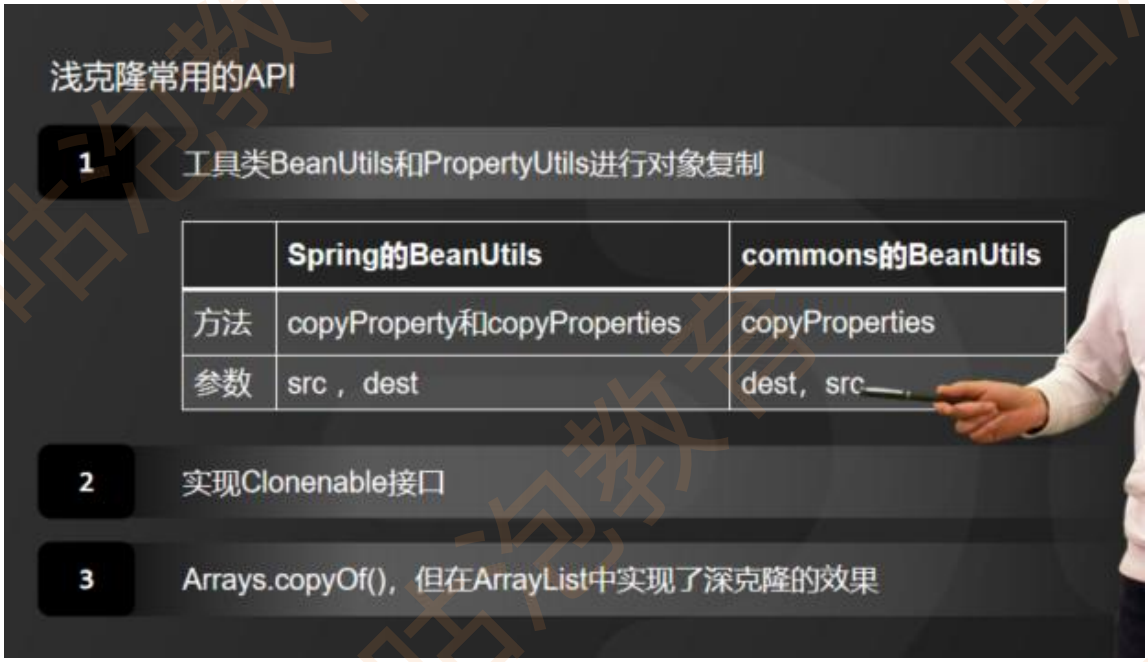


实现浅克隆的常用API有以下3种，

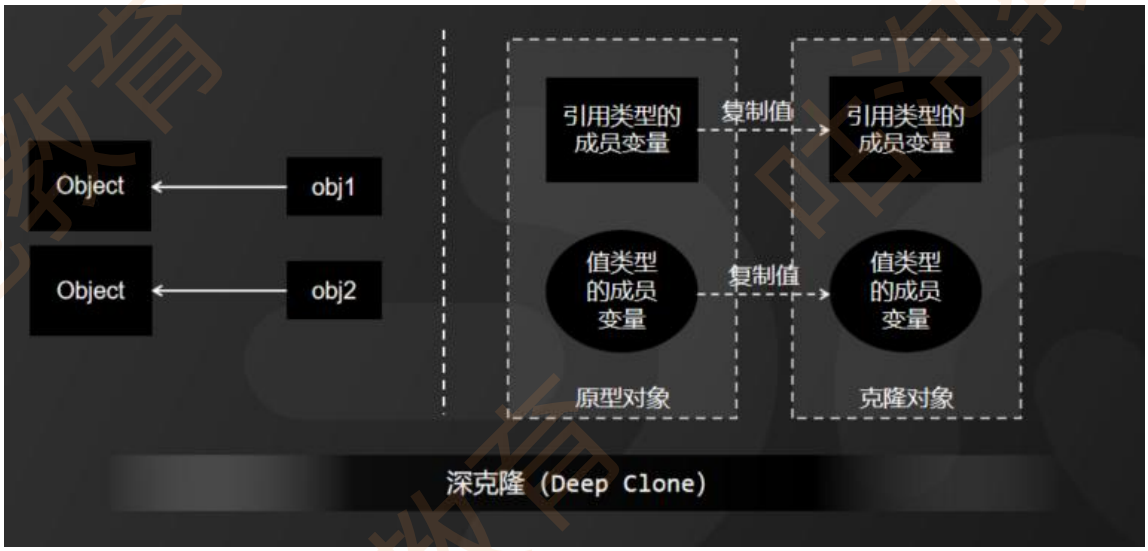
1、工具类BeanUtils和PropertyUtils，BeanUtils是Spring提供的，PropertyUtils是Apache的commons包中提供的。

2、实现Clonenable接口

3、Arrays的copyOf()方法



下面来看深克隆，深克隆就是数据拷贝后，原型对象和克隆对象之间没有关联。如图所示：obj1和obj2分别指向不同的原型对象，拷贝的是值不是内存地址。



深克隆常用的API有以下5种：

1、重写clone()方法

2、序列化

3、Apache commons工具包中的SerializationUtils的clone()方法

4、JSON工具类

5、通过构造方法手动new对象。

【导航条：结尾】

以上就是我对浅克隆和深克隆的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

34．Spring中@Autowired和@Resource的区别

【导航条：引言】

最近有非常多的小伙伴从大厂毕业了，导致招聘也卷得相当厉害，不管是应届生、中开、高开，都会问技术原理。还经常问一些日常开发不太需要了解的知识点对比。今天，就给大家分享一个应届生被问到的一道面试题，在Spring中，@Autowired注解和@Resource注解的区别。

另外，我花了1个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

好了，先来看@Autowired 这个注解。

【导航条：@Autowired】

（转场，屏幕黑底白字，正中央出现“1 @Autowired注解”）

1、@Autowired注解

@Autowired这个注解呢，是由Spring提供的，它可以用来对构造方法、成员变量以及方法参数上进行标注，能够根据对象类型完成自动注入。

比如，这样

```
public class Service {


  //构造方法注入
  @Autowired
  public Service(Service service) {
```

```
    this.service = service;
  }

  //成员变量注入
  @Autowired
  private Service service;

  //方法参数注入
  @Autowired
  public void setService(Service service) {
    this.service = service;

  }
}
```

【导航条：@Resource】

（转场，屏幕黑底白字，正中央出现“2 @Resource注解”）

2、@Resource注解

再来看@Resource这个注解，

它是由JDK提供的，遵循JSR-250规范，是JDK1.6及以上加入的新特性。作为Java的标准，它的作用和@Autowired无区别。与@Autowired不同的是它可以适用于所有的Java框架，而@Autowired只适用于Spring。简单的理解@Resource能够支持对象类型注入，也能够支持对象名称注入。

【导航条：两者区别】

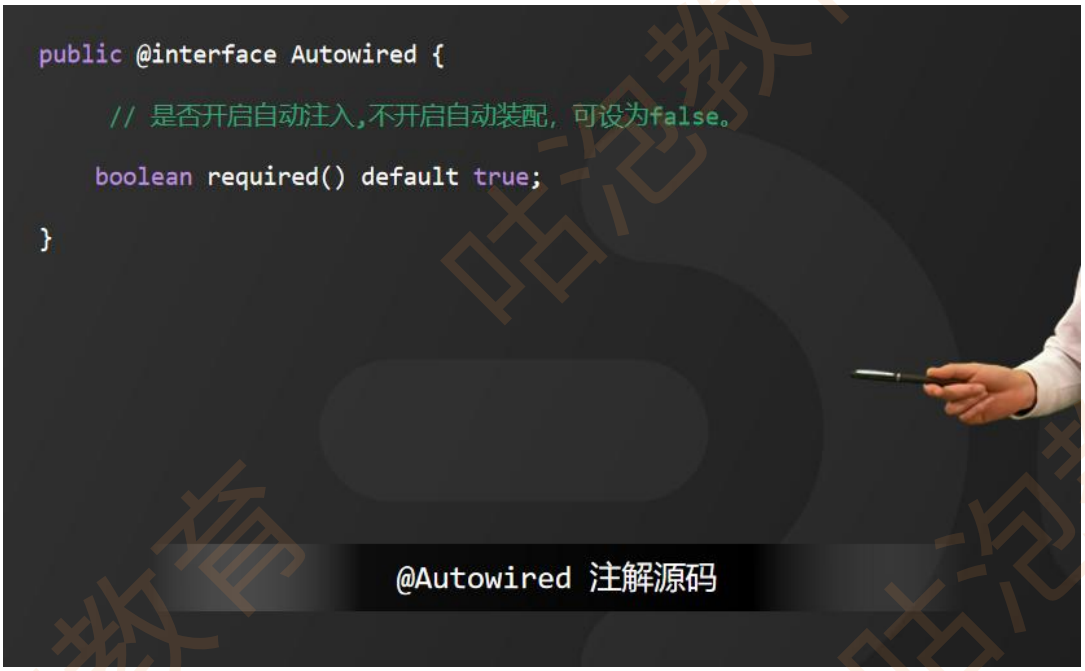
（转场，屏幕黑底白字，正中央出现“3 @Resource和@Autowired的区别”）

3、@Resource和@Autowired的区别

那@Resource和@Autowired之间具体有哪些区别呢？我从以下五个方面来分析

1）、注解内部定义的参数不同

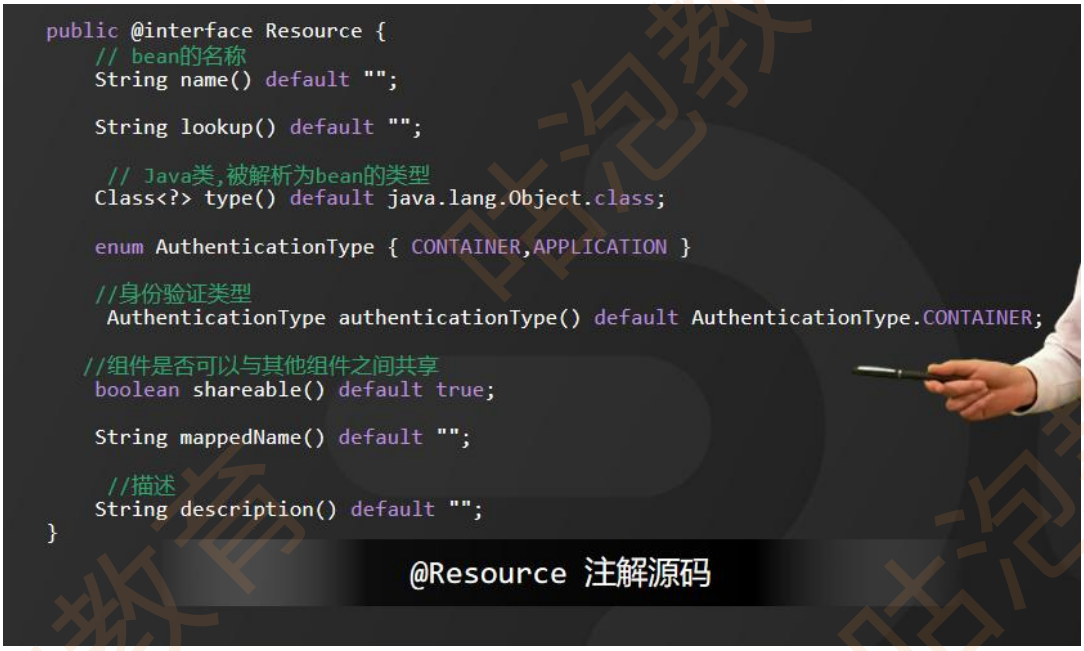
@Autowired只包含一个required参数，默认为true，表示开启自动注入。




```
public @interface Autowired {
```

```
    // 是否开启自动注入,不开启自动装配，可设为 false。


    boolean required() default true;


}
```

而@Resource 包含七个参数，其中最重要的两个是name和type。

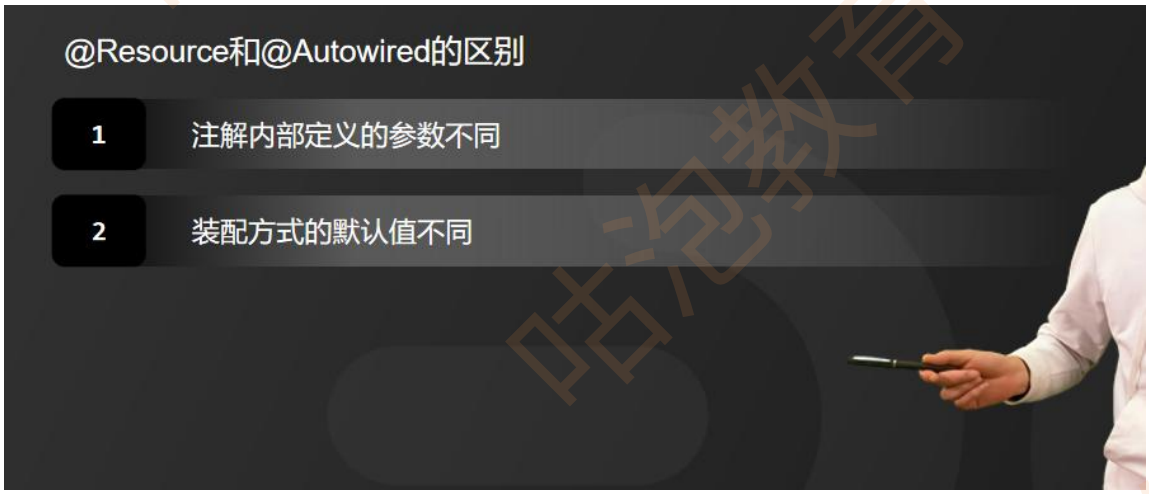



```
public @interface Resource {

    // bean 的名称
```

```
String name() default "";

String lookup() default "";

 // Java 类,被解析为 bean 的类型
Class<?> type() default java.lang.Object.class;

enum AuthenticationType {
        CONTAINER,
        APPLICATION
}

//身份验证类型

AuthenticationType authenticationType() default AuthenticationType.CONTAINER;

//组件是否可以与其他组件之间共享
```

```
    boolean shareable() default true;



    String mappedName() default "";



     //描述

    String description() default "";

}
```

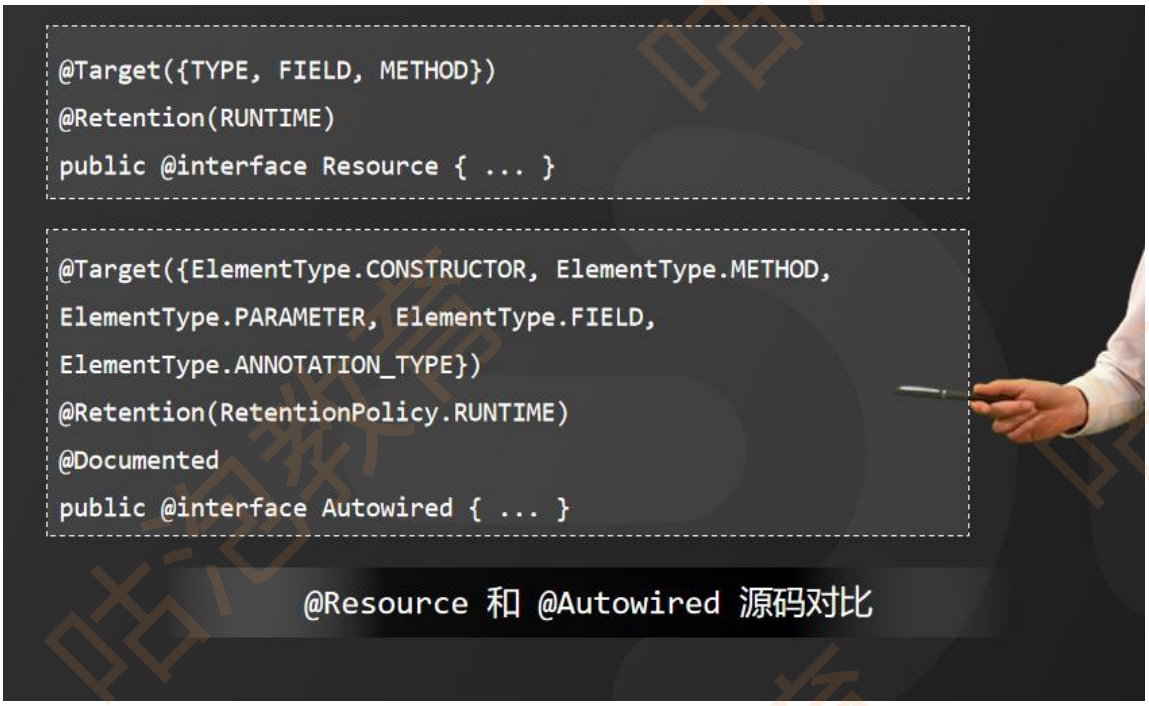
2）、装配方式的默认值不同



@Autowired默认按type自动装配，而@Resource默认按name自动装配。当然，@Resource注解可以自定义选择，如果指定了name，则根据name自动装配，如果指定了type，则用type自动装配。

3）、注解应用的范围不同

@Autowired能够用在构造方法、成员变量、方法参数以及注解上，而@Resource能用在类、成员变量和方法参数上，这点从源码也能看得出来。



@Autowired 能够用在构造方法、成员变量、方法参数以及注解上，而@Resource 能用在类、成员变量和方法参数上，这点从源码也能看得出来。@Target({TYPE, FIELD, METHOD})

@Retention(RUNTIME)

public @interface Resource { ... }

@Target({ElementType.CONSTRUCTOR, ElementType.METHOD, ElementType.PARAMETER, ElementType.FIELD, ElementType.ANNOTATION_TYPE})

@Retention(RetentionPolicy.RUNTIME)

@Documented

public @interface Autowired { ... }

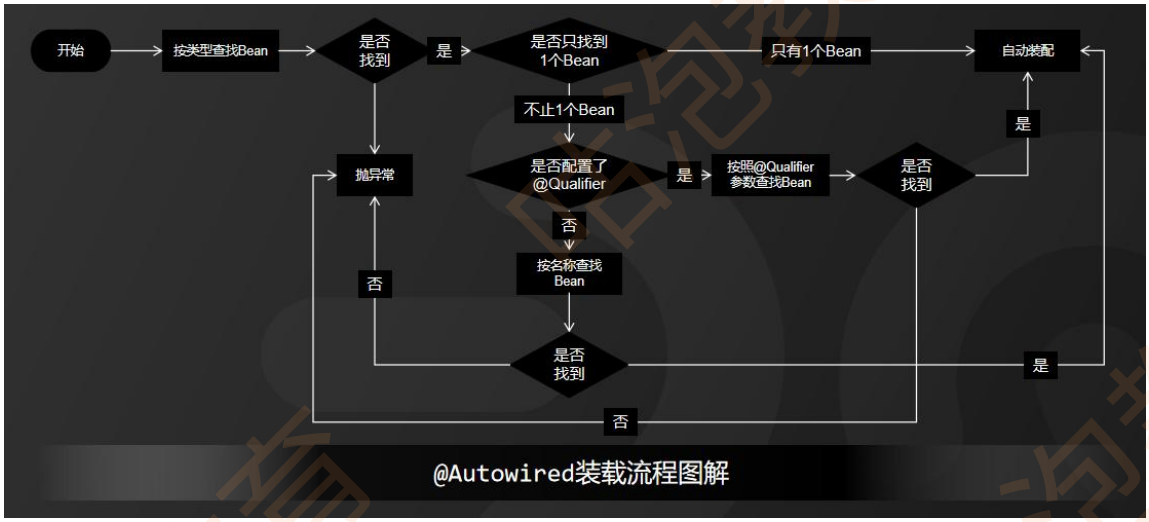
```
@Target({ElementType.CONSTRUCTOR, ElementType.METHOD, ElementType.PARAMETER,
ElementType.FIELD, ElementType.ANNOTATION_TYPE})
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface Autowired { ... }
```

4）、出处不同

@Autowired是Spring定义的注解，而@Resource是遵循JSR-250规范，定义在JDK中。所以@Autowired只能在Spring框架下使用，而@Resource则可以与其他框架一起使用。

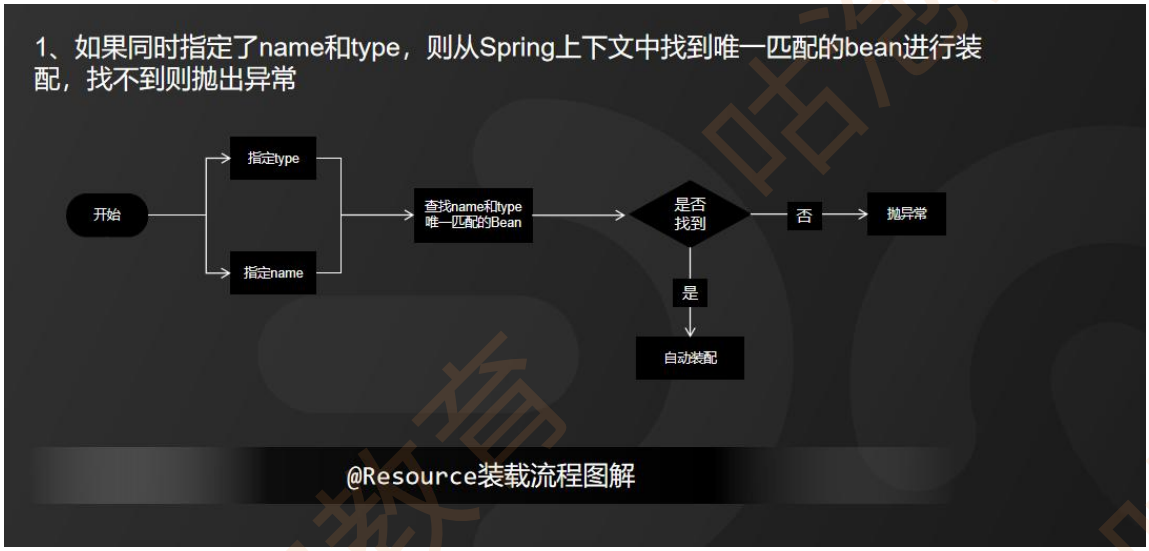
5）、装在顺序不同

@Autowired默认先按byType进行匹配，如果发现找到多个bean，则又按照byName方式进行匹配，如果还有多个，则报出异常。

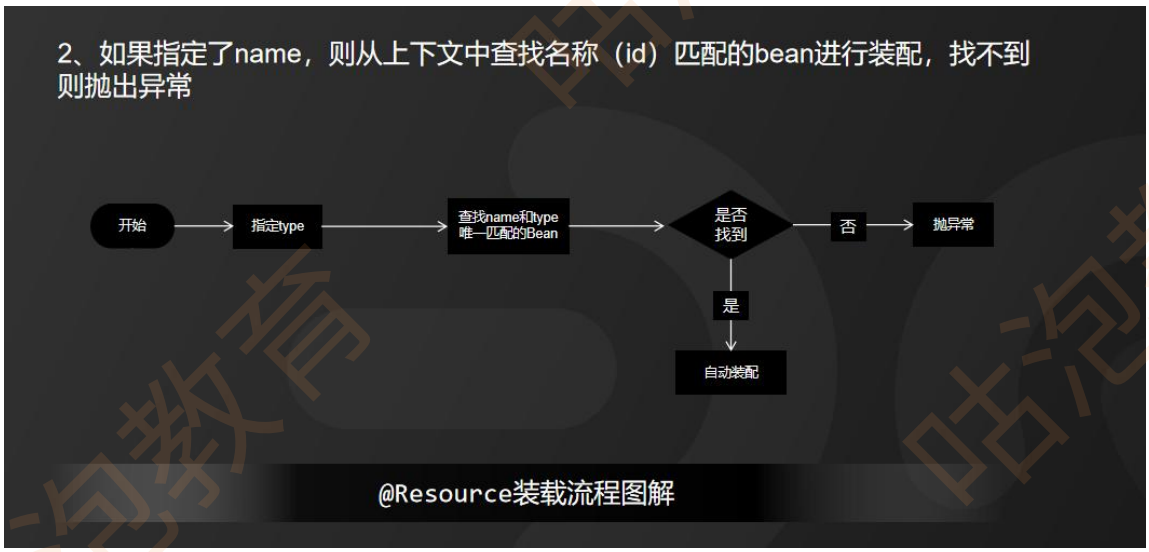


而@Resource的装载顺序分为四种情况：

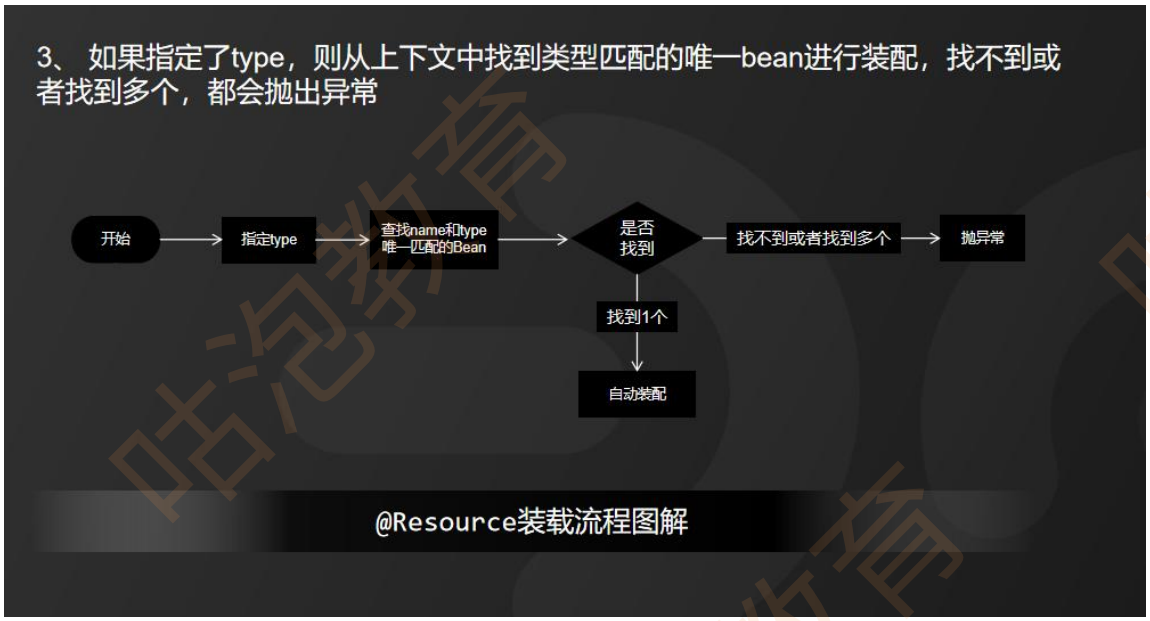
1）如果同时指定了name和type，则从Spring上下文中找到唯一匹配的bean进行装配，找不到则抛出异常，具体流程如图所示：



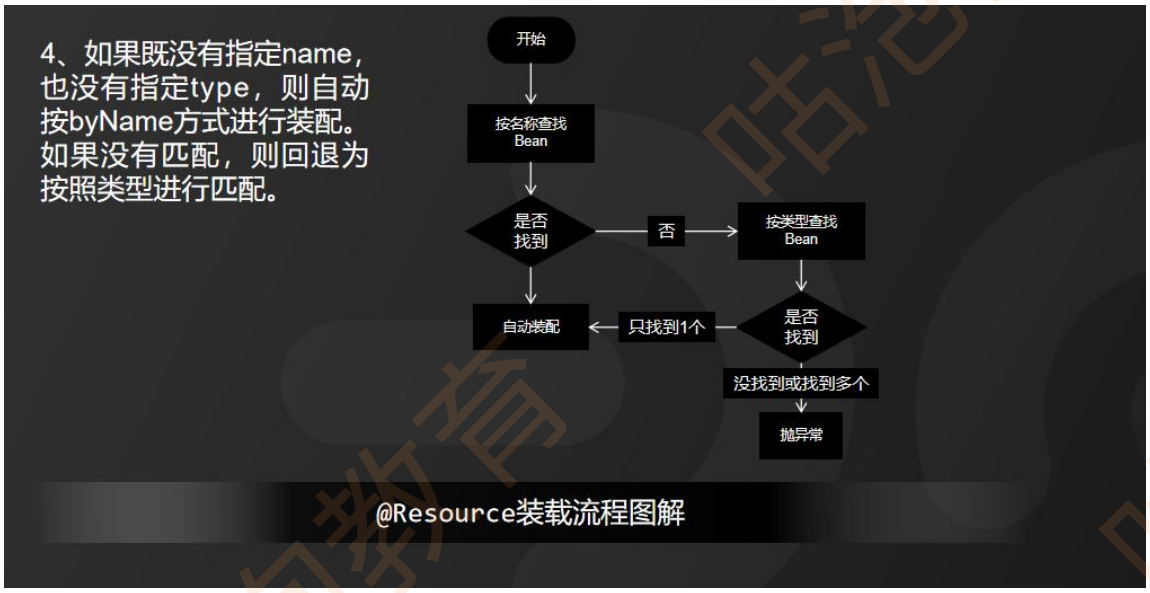
2）如果指定了name，则从上下文中查找名称（id）匹配的bean进行装配，找不到则抛出异常，具体流程如图所示：



3） 如果指定了type，则从上下文中找到类型匹配的唯一bean进行装配，找不到或者找到多个，都会抛出异常，具体流程如图所示：



4）如果既没有指定name，也没有指定type，则自动按byName方式进行装配。如果没有匹配，则回退为按照类型进行匹配。



为了帮助大家更好地理解和区分，我还专门花时间整理一张表格。有需要的小伙伴可以关注我的主页介绍。

项	@Autowired	@Resource
是否支持优先级	支持。在装配bean时加@Primary即可。	不支持。
bean是否必须存在	默认必须存在。 如果允许不存在可这样:@Autowired(required = false)	必须存在。 无法配置允许不存在。
提供者	Spring	JDK(需要1.6及以上)
默认注入方式	默认是按照类型。按名字注入可这样: 装配:@Component("bean的名字")或@Bean("bean的名字")注入：@Autowired @Qualifier("bean的名字")	默认是根据名字(byName)按类型注入可这样: 注入：@Resource(type = Xxx.class)

@Resource 和 @Autowired 都可以标注在字段或者方法上

【导航条：结尾】

以上呢就是我对@Autowired与@Resource的总结和理解。总结一下，两者在功能上差别不大，使用起来也差不多。但是，在日常开发中，我建议使用@Autowired，有以下三个理由：

第1：@Autowired功能略强大。支持优先注入、可以配置允许bean不存在。

第2：若使用了Spring框架，使用其特有的注解个人感觉更好一点。

第3：有人会认为@Resource更加通用，因为它是个规范，其他框架也会支持。但我认为，目前后端都是在用Spring了，没有必要考虑其他框架。

当然，这个跟我个人是Spring迷无关哈，如果你也是Spring的小迷粉，可以关注我，一起交流，共同进步。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

35．MySQL存储引擎MyISAM和InnoDB的区别

【导航条：引言】

你都工作3年了，怎么还不知道MyISAM和InnoDB有什么区别？一位粉丝被面试官这样一个问题。当时，这位粉丝直接回复“不知道”，这位粉丝自己也怪不好意思的。

其实，面试官问他这个问题也不是随便问的，也是看了这位粉丝简历上写了熟练MySQL才提问的。其实，对于MySQL，这位粉丝可能只是非常熟练地完成建库、建表、往数据库表中读写数据这些简单的操作而已，并没有去了解建表的原理。

这个面试题也给我们敲了一下警钟，对于有些技术的知识点，要是没有掌握到一定深度，建议不要写熟练之类的，如果真要写，建议用了解或者熟悉这样的字眼来描述，避免面试官问起时又答不出来，暴露了自己的不足。

另外，我花了1个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

MyISAM和InnoDB到底有什么区别呢?

【导航条：两者区别】

（转场，屏幕黑底白字，正中央出现“1 MyISAM和InnoDB的区别”）

1、MyISAM和InnoDB的区别

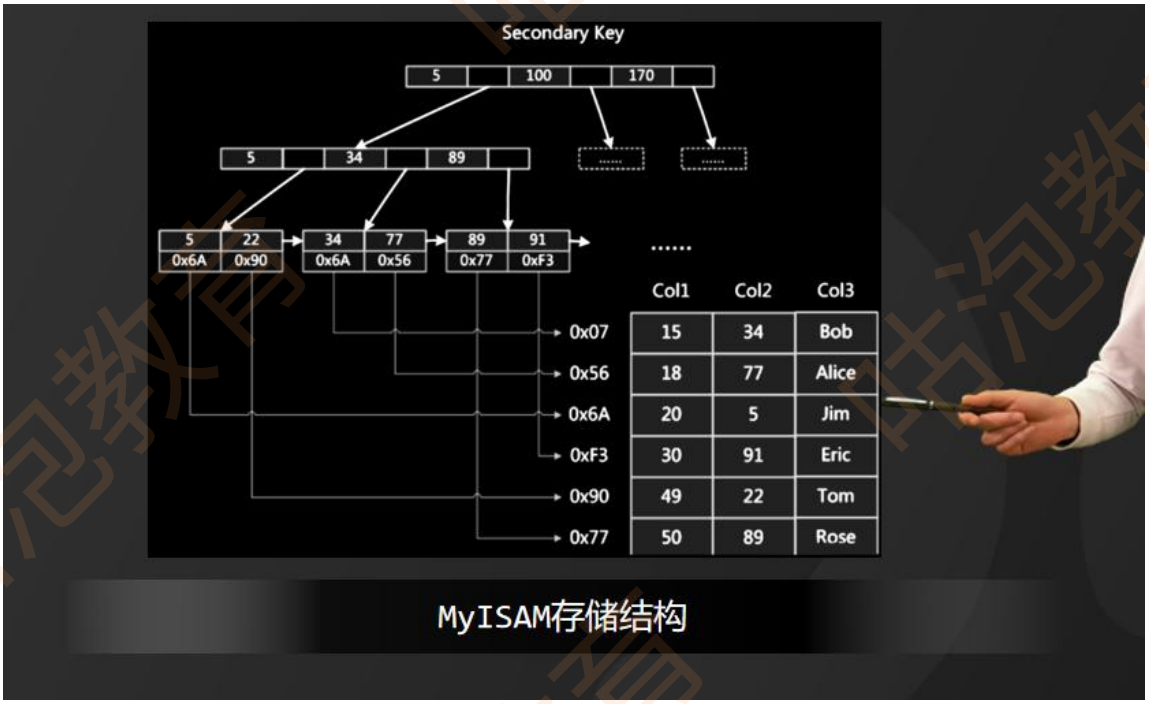
MySQL 5.5以后的版本开始将InnoDB作为默认的存储引擎，之前的版本都是MyISAM。关于MyISAM和InnoDB的区别，我总结为以下5个方面，希望能帮助到大家。

1）. 数据的存储结构不同

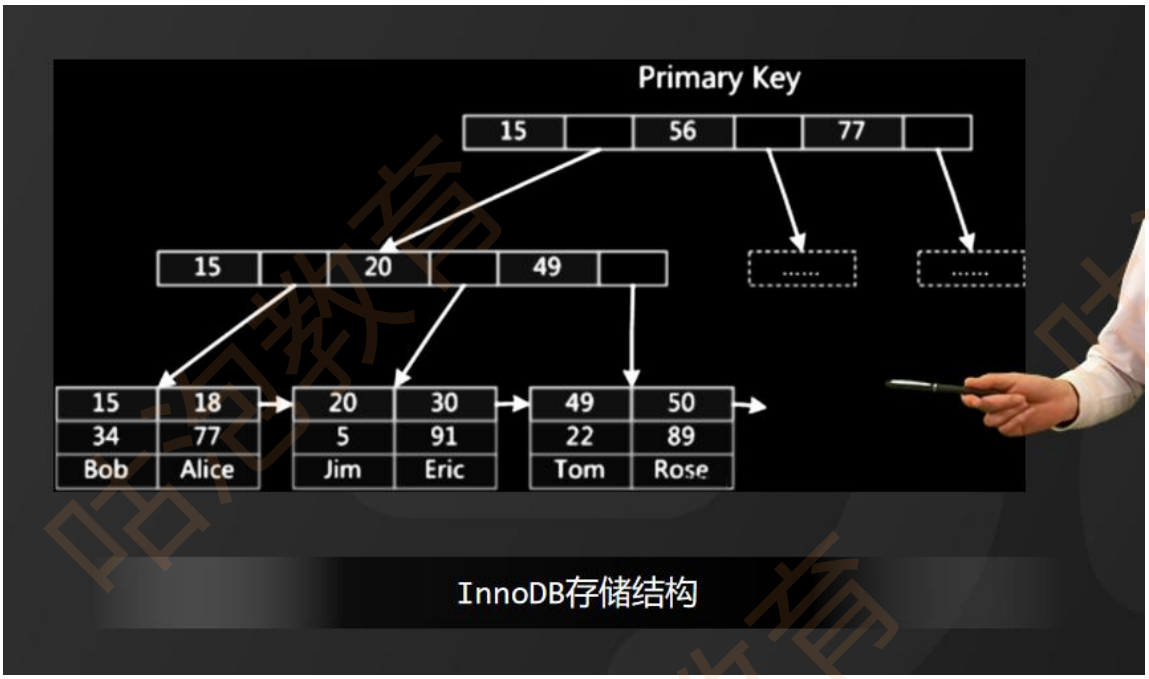
先来看MyISAM，每个MyISAM在磁盘上存储成三个文件，它们以表的名字开头来命名。.frm文件存储表定义。.MYD(MYD)存储数据文件。.MYI(MYIndex)存储索引文件。

而 InnoDB在磁盘上保存为两个文件。.frm文件同样存储为表结构文件，.ibd文件存储的是数据和索引文件。

由于MyISAM的索引和数据是分开存储的，因此索引查找的时候，MyISAM的叶子节点存储的是数据所在的地址，而不是数据。



而InnoDB叶子节点存储的是整个数据行所有的数据。



2）. 存储空间的消耗不同

MyISAM可被压缩，存储空间较小。支持三种不同的存储格式：静态表（默认，但是注意数据末尾不能有空格，会被去掉）、动态表、压缩表。

InnoDB需要更多的内存和存储，它会在主内存中建立其专用的缓冲池用于高速缓冲数据和索引。InnoDB所在的表都保存在同一个数据文件中（也可能是多个文件，或者是独立的表空间），InnoDB表的大小只受限于操作系统文件的大小，一般为2GB。

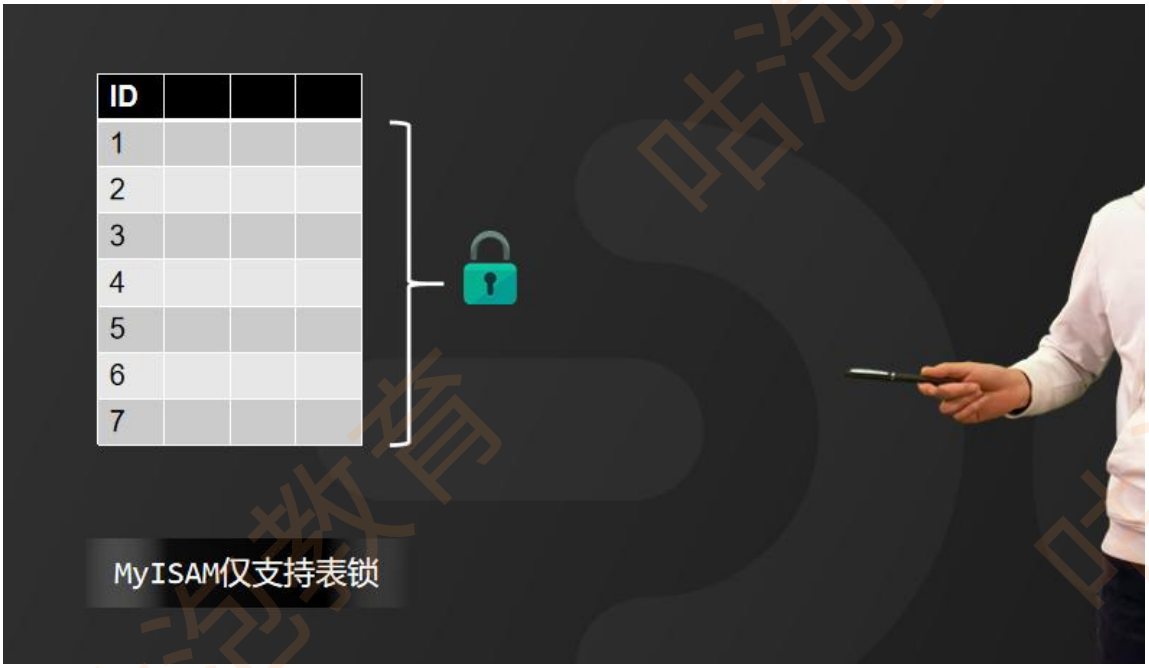
3）. 对事务的支持不同

MyISAM强调的是性能，每次查询具有原子性，其执行速度比Innodb类型更快，但是不提供事务支持。

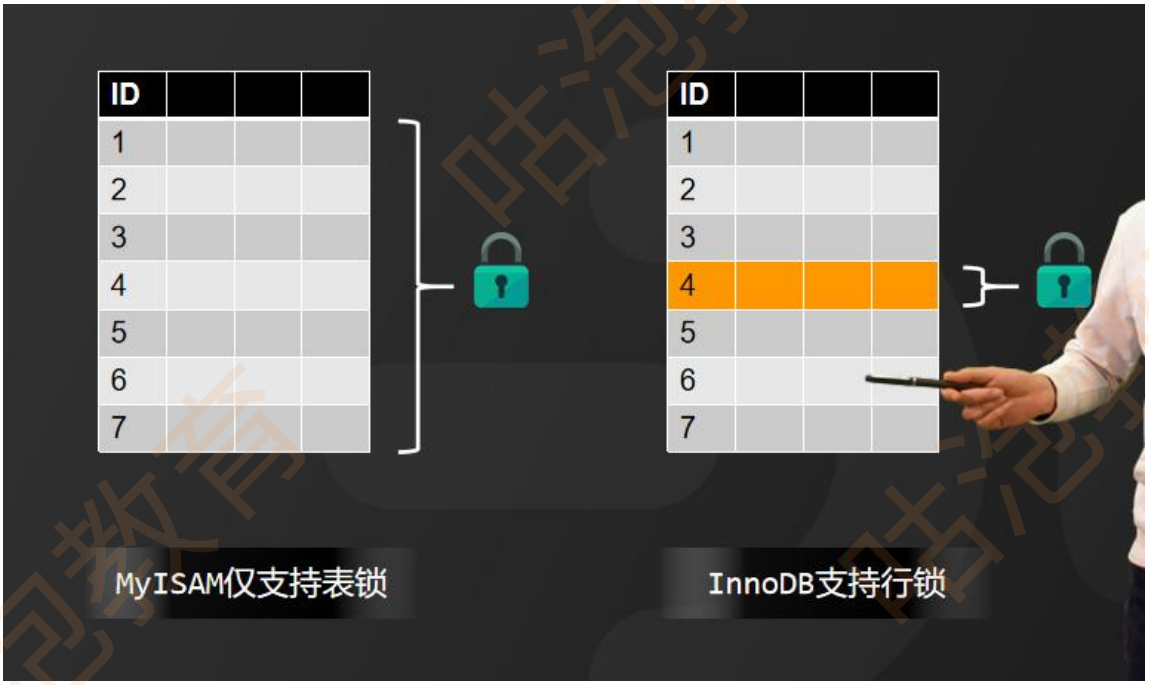
InnoDB除了提供事务支持和外部键等高级数据库功能。还具有事务提交（commit）、回滚（rollback）和崩溃修复能力（crach recovery capabilities）等这些事务安全（transaction-safe ACID compliant）型表。

4）. 对锁的支持不同

如果只是执行大量的查询, MyISAM是更好的选择。MyISAM在增删的时候需要锁定整个表格，效率会低一些。



而innoDB支持行级锁，删除插入的时候只需要锁定操作行就行。如果有大量的插入、修改删除操作，使用InnoDB性能能会更高。



5）. 对外键的支持不同

MyISAM不支持外键，而InoDB支持外键。当然，各种不同MySQL版本对两者的支持都有所改进

【导航条：总结及建议】

（转场，屏幕黑底白字，正中央出现“2 总结及建议）

2、总结及建议

最后总结一下，MyISAM和InnoDB的主要区别包括

1）. 数据的存储结构不同

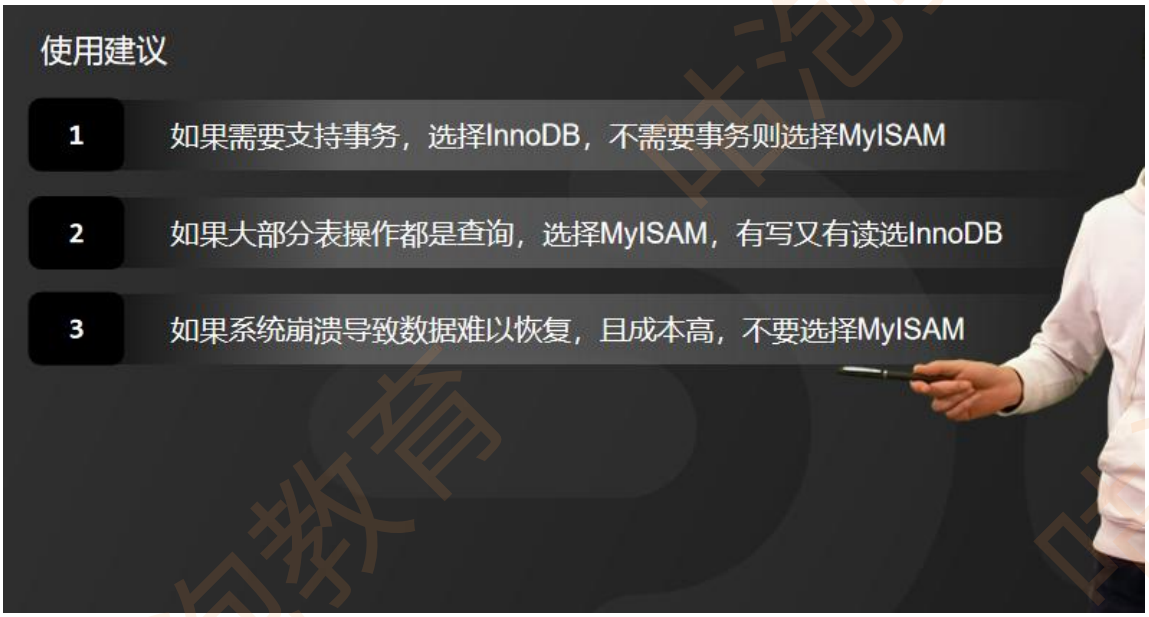
2）. 存储空间的消耗不同

3）. 对事务的支持不同

4）. 对锁的支持不同

5）. 对外键的支持不同

MyISAM和InnoDB的区别还有很多，我这里只是列举了比较重要的几点。那我们在实际开发中，该如何选择MyISAM和InnoDB呢？我总结为以下三点：



1、如果需要支持事务，选择InnoDB，不需要事务则选择MyISAM。

2、如果大部分表操作都是查询，选择MyISAM，有写又有读选InnoDB。

3、如果系统崩溃导致数据难以恢复，且成本高，不要选择MyISAM。

【导航条：结尾】

以上就是我对MyISAM和InnoDB区别的总结和理解，下次面试你会了吗?

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难!

36．谈谈你对MVCC的理解

MVCC也是一道非常高频的面试题，今天我花两分钟时间给大家梳理一下。另外，我花了1个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

【导航条：什么是MVCC】

（转场，屏幕黑底白字，正中央出现“1 什么是MVCC”）

1、什么是MVCC

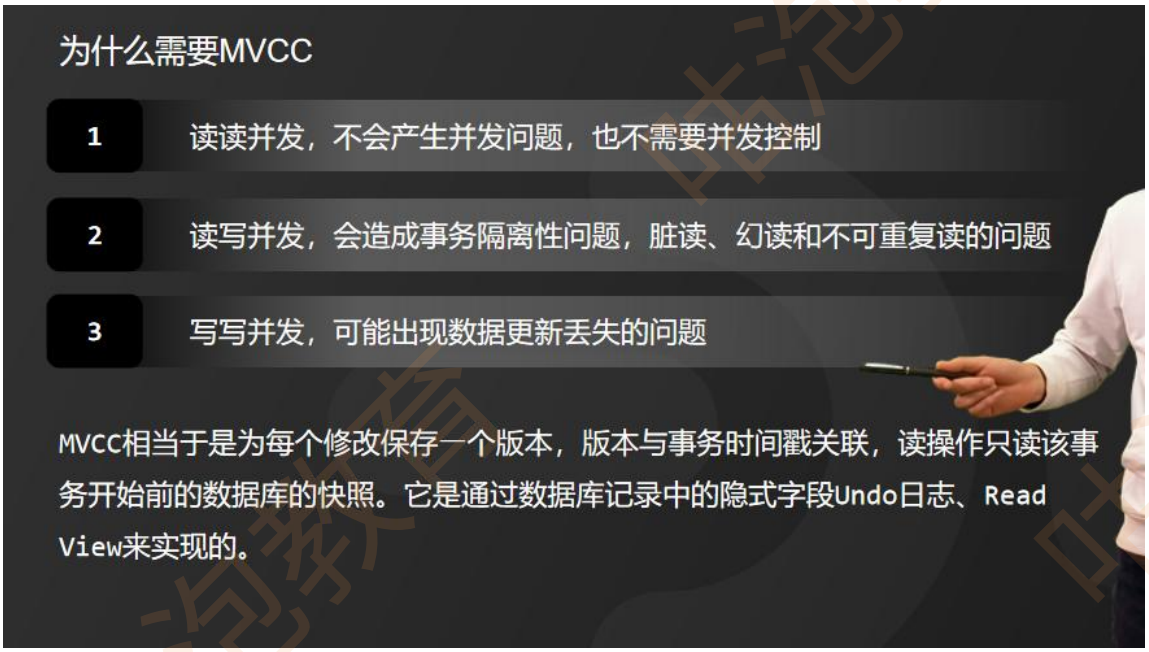
MVCC是为了解决事务操作中多线程并发安全问题的无锁并发控制技术，它的全称是Multi-Version Concurrency Control，中文翻译过来就叫多版本并发控制，简称MVCC。

【导航条：为什么需要MVCC】

（转场，屏幕黑底白字，正中央出现“2 为什么需要MVCC”）

2、为什么需要MVCC

对于MVCC的理解，可以根据数据库的三种并发场景来分析。



第一种是读和读的并发，就是两个线程A和B，同时进行读操作，这种情况下不会产生并发问题，也不需要并发控制。

第二种就是读写并发，就是两个线程A和B在同一时刻分别进行读写操作，这种情况下，可能会造成事务隔离性问题，还可能遇到脏读、幻读和不可重复读的问题。

第三种是写和写的并发，也就是两个线程A和B同时进行写操作，这种情况下可能出现数据更新丢失的问题

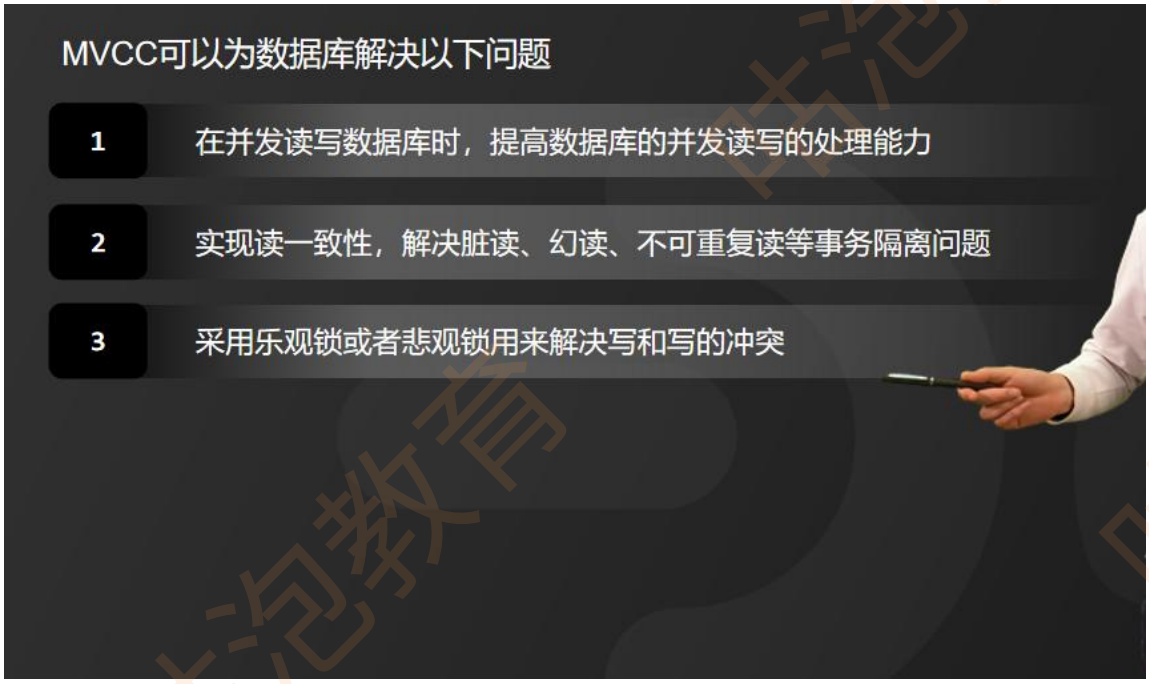
MVCC相当于是为每个修改保存一个版本，版本与事务时间戳关联，读操作只读该事务开始前的数据库的快照。它是通过数据库记录中的隐式字段Undo日志、Read View来实现的。

【导航条：解决什么问题】

（转场，屏幕黑底白字，正中央出现“3 MVCC能解决什么问题”）

3、MVCC能解决什么问题

根据以上的分析，我总结一下MVCC可以为数据库解决以下三个方面的问题



1、在并发读写数据库时，可以做到在读操作时不用阻塞写操作，写操作也不用阻塞读操作，从而提高数据库的并发读写的处理能力。

2、能实现读一致性，从而解决脏读、幻读、不可重复读等不可重复读，但是不能解决数据更新丢失的问题。

3、采用乐观锁或者悲观锁用来解决写和写的冲突，从而最大程度地去提高数据库的并发性能。

【导航条：结尾】

以上就是我对MVCC的理解，如果想了解MVCC实现原理的小伙伴可以在评论区回复666，如果超过100个我就爆肝连更。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

37．为什么Netty线程池默认大小为CPU核数的2倍

【导航条：引言】

有位工作5年的小伙伴问我说，为什么Netty线程池默认大小为CPU核数的2倍，今天，我花2分钟时间给大家专门分享一下我对这个问题的理解。

另外，我花了1个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

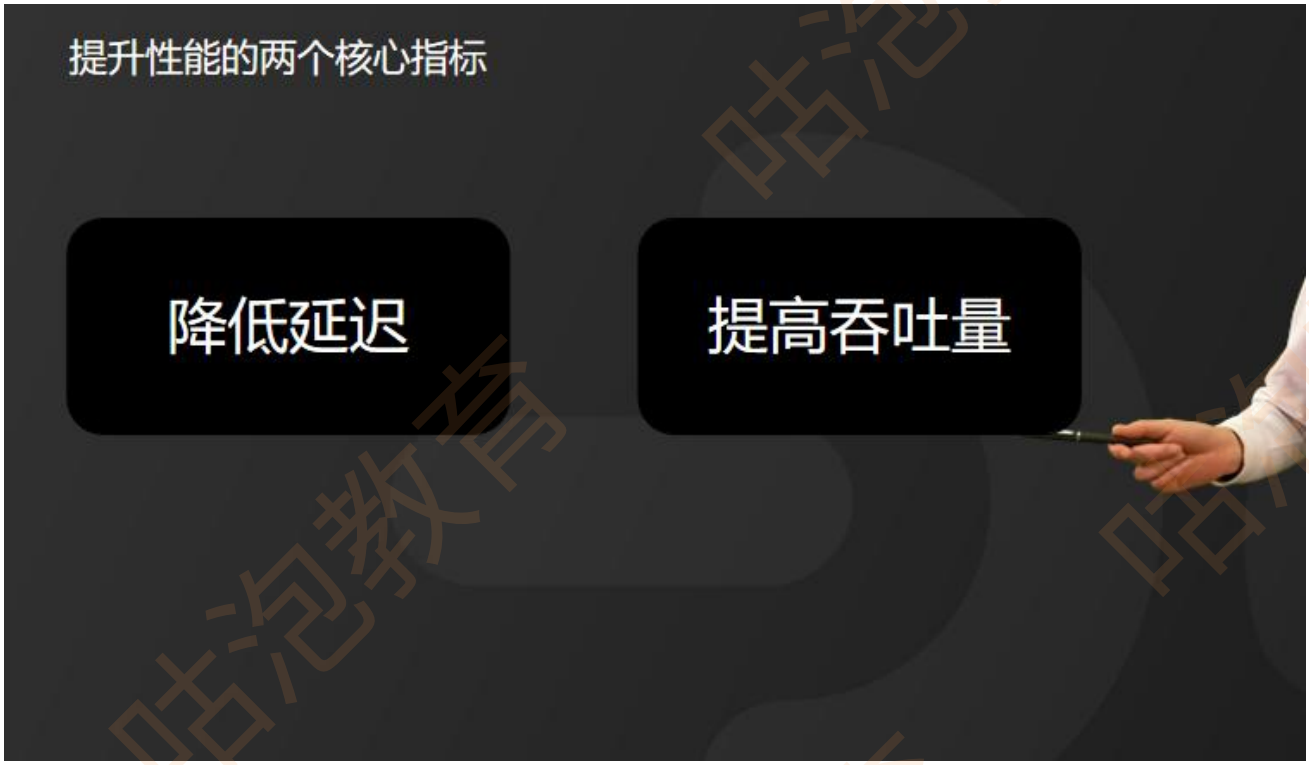
【导航条：分析原因】

（转场，屏幕黑底白字，正中央出现“1 分析原因”）

1、分析原因

我们都知道使用多线程的本质是为了提升程序的性能，总体来说有两个最核心的指标，一个延迟，一个吞吐量。延迟指的是发出请求到收到响应的时间，吞吐量指的是 。这两个指标之间有一定的关联，因为同等条件下延迟越短吞吐量越大，但由于它们是不同的维度，一个是时间，一个是空间，并不能相互转换。

因此，提升性能最主要的目的就是要降低延迟，提高吞吐量。



那我们如何来衡量这些性能指标呢?

【导航条：如何衡量性能】

（转场，屏幕黑底白字，正中央出现“2 如何衡量性能指标”）

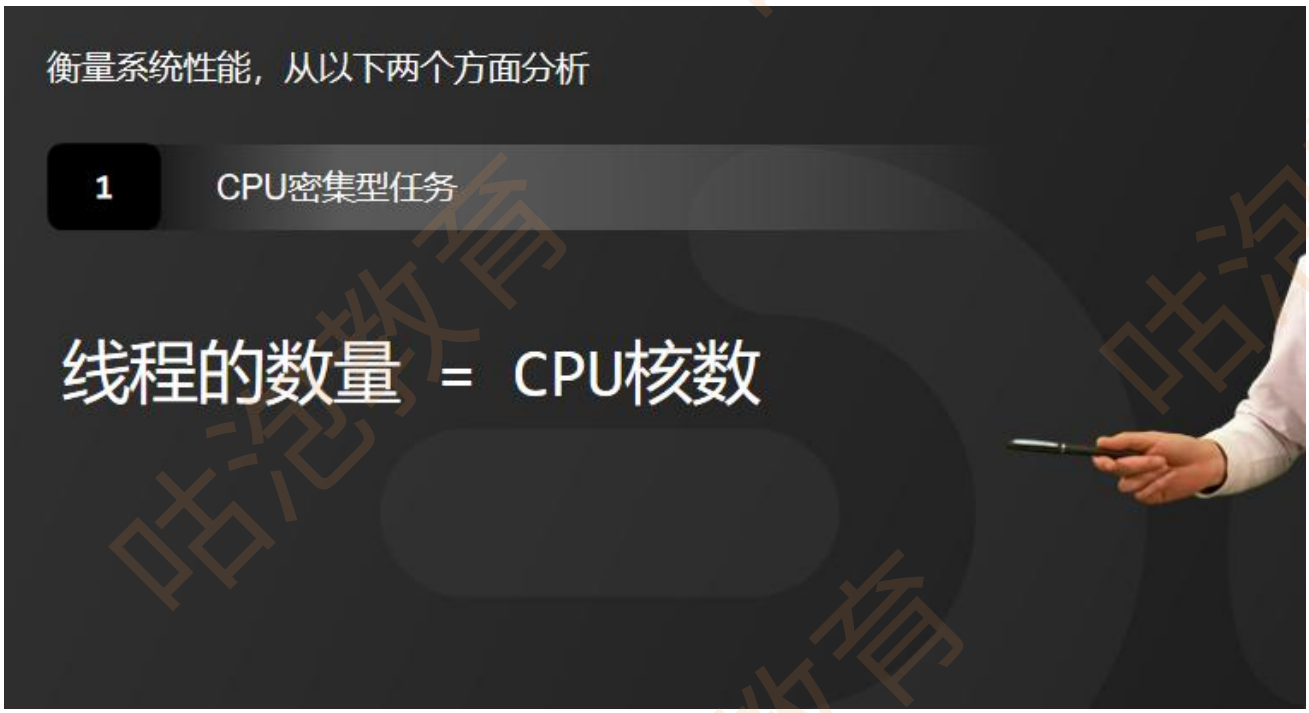
2、如何衡量性能指标

具体来说，要降低延时，就是要提高CPU的处理能力。而提高吞吐量，就是要提高IO读写效率。那么具体如何衡量系统性能，我从以下两个方面来分析：

我们可以将程序分为是I/O密集型任务和CPU密集型任务。

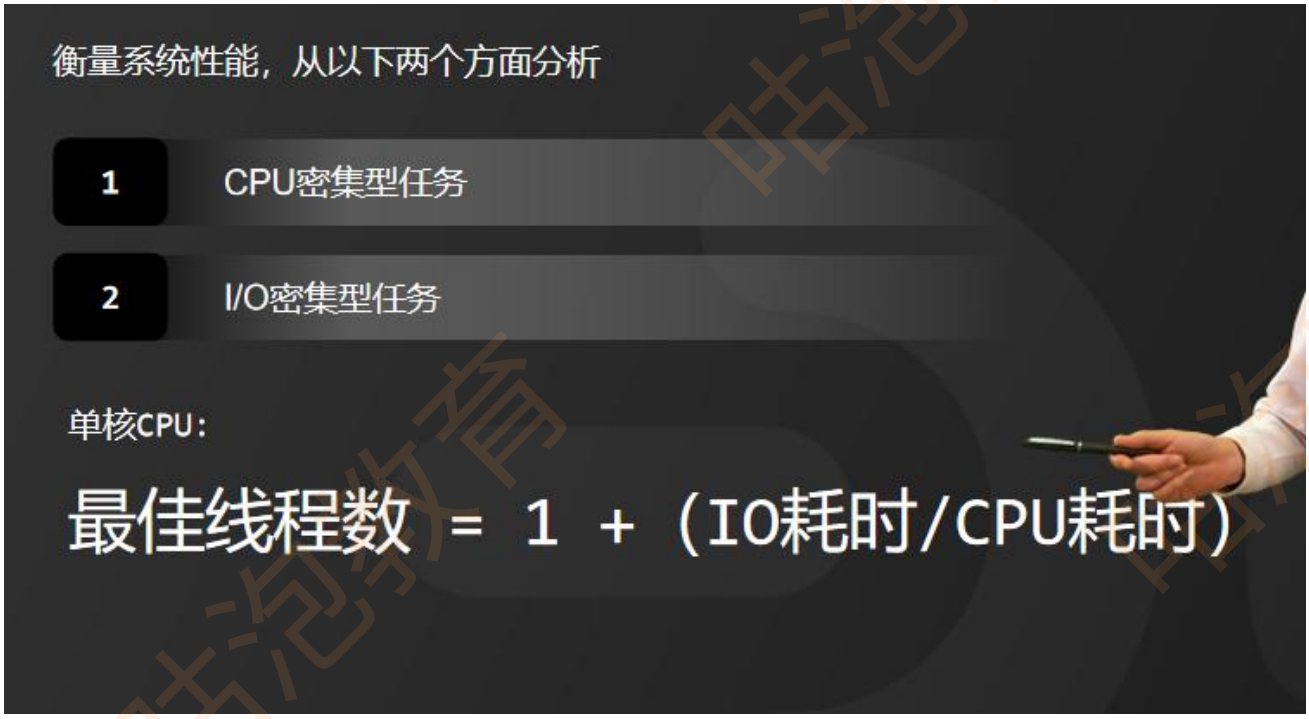
那么第1种情况，对于CPU密集型任务而言，理论上“线程的数量 = CPU核数”就是合适的。但是，在实际应用中的线程数量一般会设置为“CPU核数 + 1”。因为线程有可能因为内存页失效或其他原因导致阻塞，多设

置一个线程可以保证CPU的利用率。



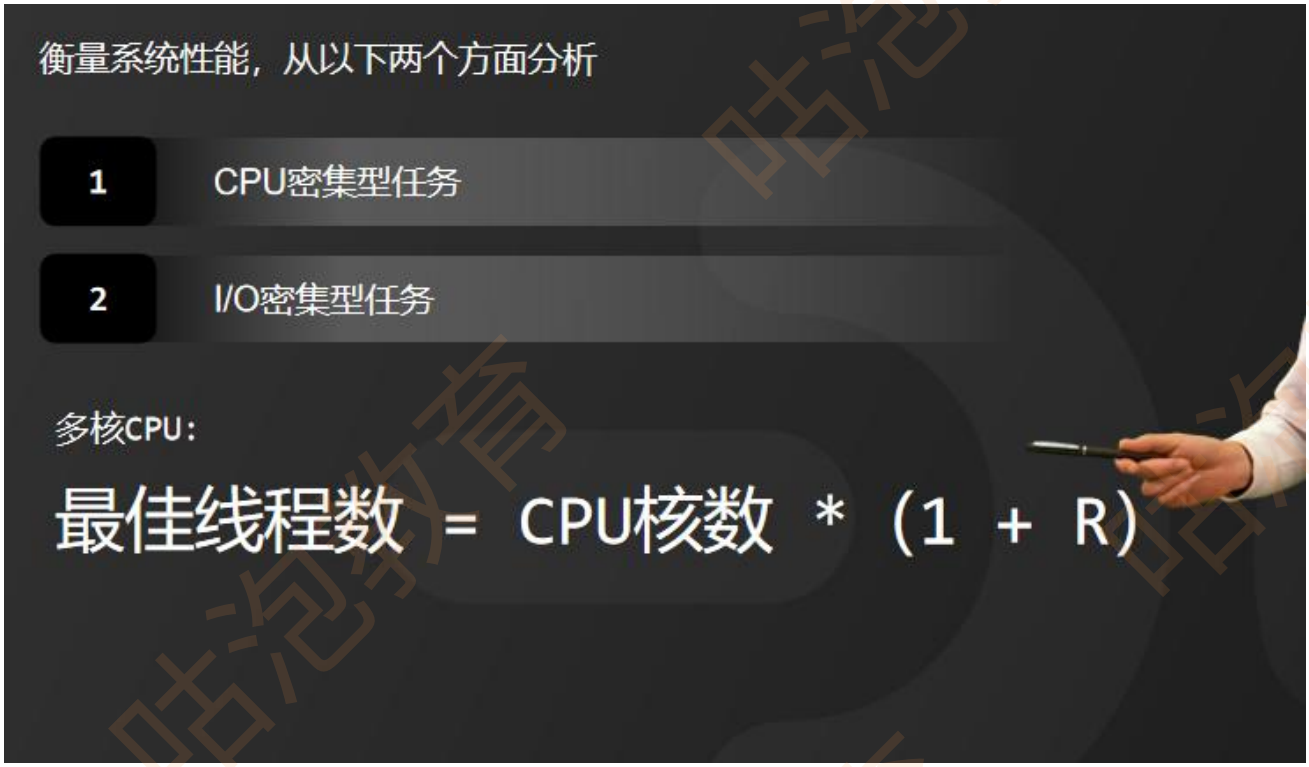
第2种情况，而对于I/O密集型任务而言，我们假设CPU计算和I/O操作的耗时比是 1:1，那么2个线程是最合适的。如果CPU计算和I/O操作的耗时比是 1:2，也就是说3个线程是合适的，这样CPU和I/O设备的利用率都可以达到100%。根据这个推测，我们可以得到这样一个公式：

最佳线程数 = 1 + (IO耗时/CPU耗时)



不过上面这个公式是针对单核CPU，如果是多核CPU只需要等比扩大就可以了，假设IO耗时和CPU耗时比为R，那么计算公式如下：

最佳线程数 = CPU核数 * (1 + R)



而Netty的默认线程池个数，就是假设了I/O耗时和CPU耗时的占比是1:1，实际上Netty有一个参数叫ioRatio，默认为50，它表示在一个轮事件循环中，单个I/O线程执行I/O事件和执行异步任务的耗时占比为 1:1。相当于 R = 1，代入上面的公式，就可以得出Netty默认设置的线程池大小自然就是

默认线程池大小 = CPU核数 * (1 + 1)

也就2倍CPU核数大小。而且Netty的应用场景主要是I/O密集型任务，所以，Netty这样设计是有科学性的。

看到了这里，你是不是豁然开朗了呢?

【导航条：总结与建议】

（转场，屏幕黑底白字，正中央出现“3 总结与使用建议”）

3、总结与使用建议

通过前面的分析，我们已经知道了Netty线程池默认大小未CPU核数2倍的原因，我们在实际开发中，如何来得到一个比较准确的线程池大小呢?

我们可以提前压测，根据压测结果来进行微调。一般情况下，保证生产环境为压测环境的75%即可。如果修改Netty的线程池大小，也一定要考虑ioRatio这个参数是否需要调整，因为2倍CPU核数的大小是假设的I/O耗时和CPU耗时为1:1，调整线程大小之后，性能效果也不一定符合期望值。

在大部分场景下，没有必要太过于关注线程池大小怎么配置，I/O密集型任务使用Netty默认配置就可以了。因为，提高吞吐量也不能只简单的只依赖线程池，还可以通过缓存、微服务拆分，优化业务逻辑、优化算法等方式来协作解决。

【结尾】

好了，以上就是我的分享和理解，对望能对大家有所帮助。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

38．谈谈分布式事务的3种解决方案

【导航条：引言】

前几天，有一位10多年经验的架构师在面试互联网大厂时被问到这样一个问题，说请你谈谈分布式事务的解决方案。那今天，我给大家分享一下我对这个问题的理解。

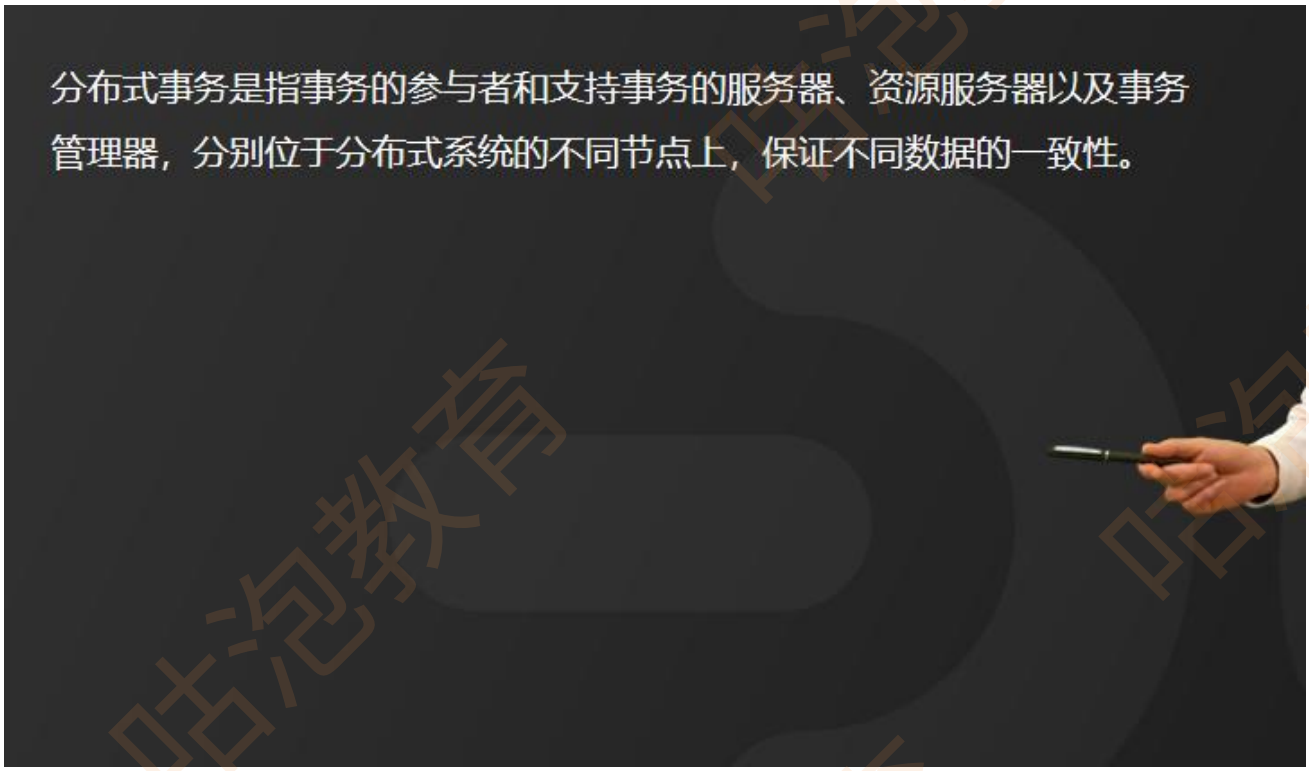
另外，我花了1个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

【导航条：分布式事务定义】

（转场，屏幕黑底白字，正中央出现“1 什么是分布式事务”）

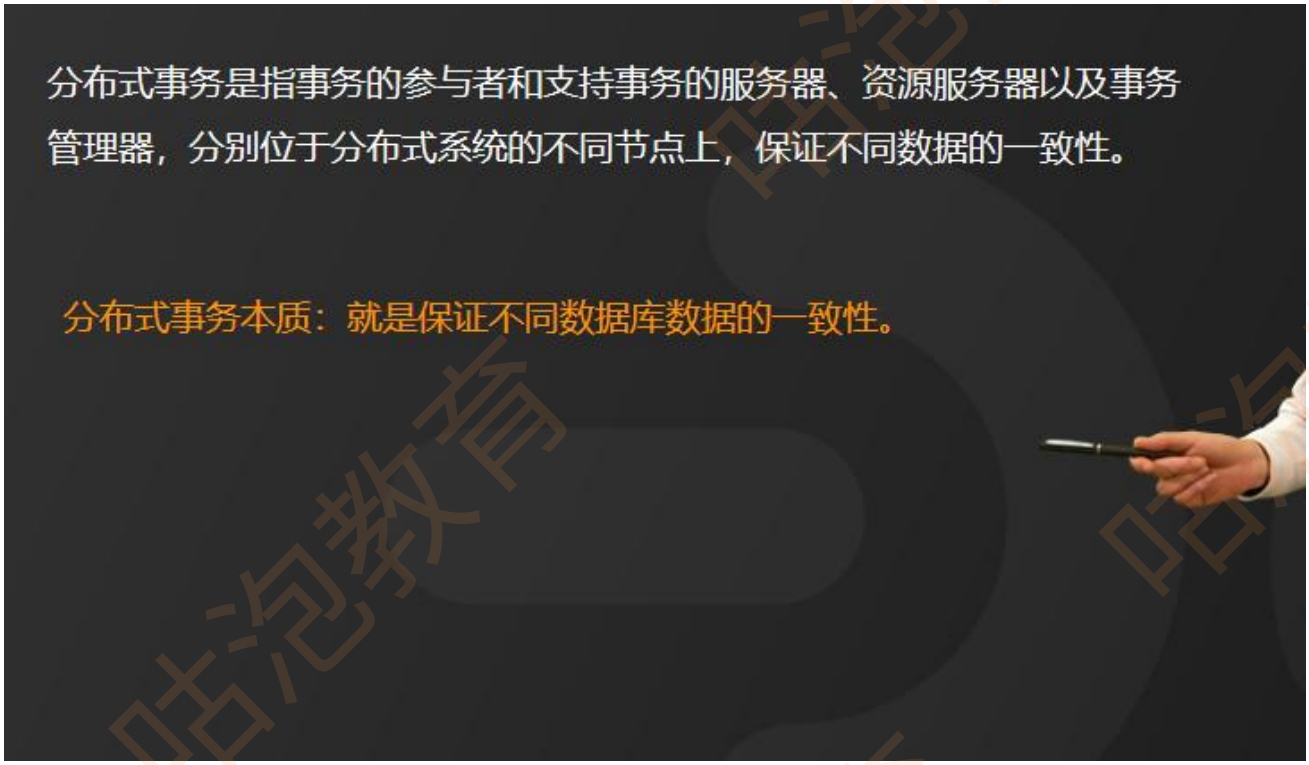
1、什么是分布式事务

分布式事务是指事务的参与者和支持事务的服务器、资源服务器以及事务管理器，分别位于分布式系统的不同节点上，保证不同数据的一致性。



比如大型电商系统中的下单场景，会涉及到扣库存、优惠折扣计算、订单ID生成等这些服务，通常情况下库存、折扣、订单ID生成的服务都位于不同的服务器和数据库中，那么下单是否成功，不仅取决于本地节点的数据库操作，还需要依赖其他服务的执行结果结果，这个时候分布式事务就是保证这些操作要么全部成功，要么全部失败。

因此，本质上来说，分布式事务就是为了保证不同数据库数据的一致性。



【导航条：分布式事务解决方案】

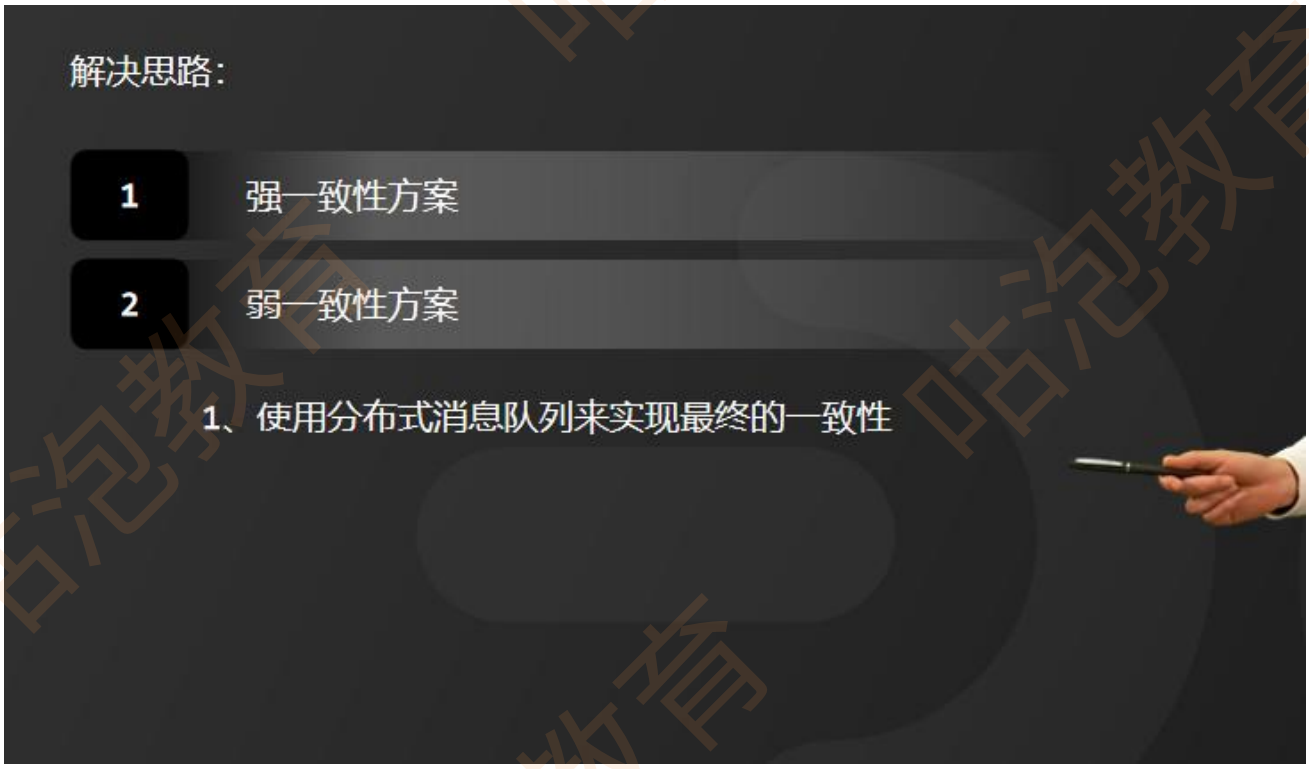
2、分布式事务解决方案

（转场，屏幕黑底白字，正中央出现“2”）

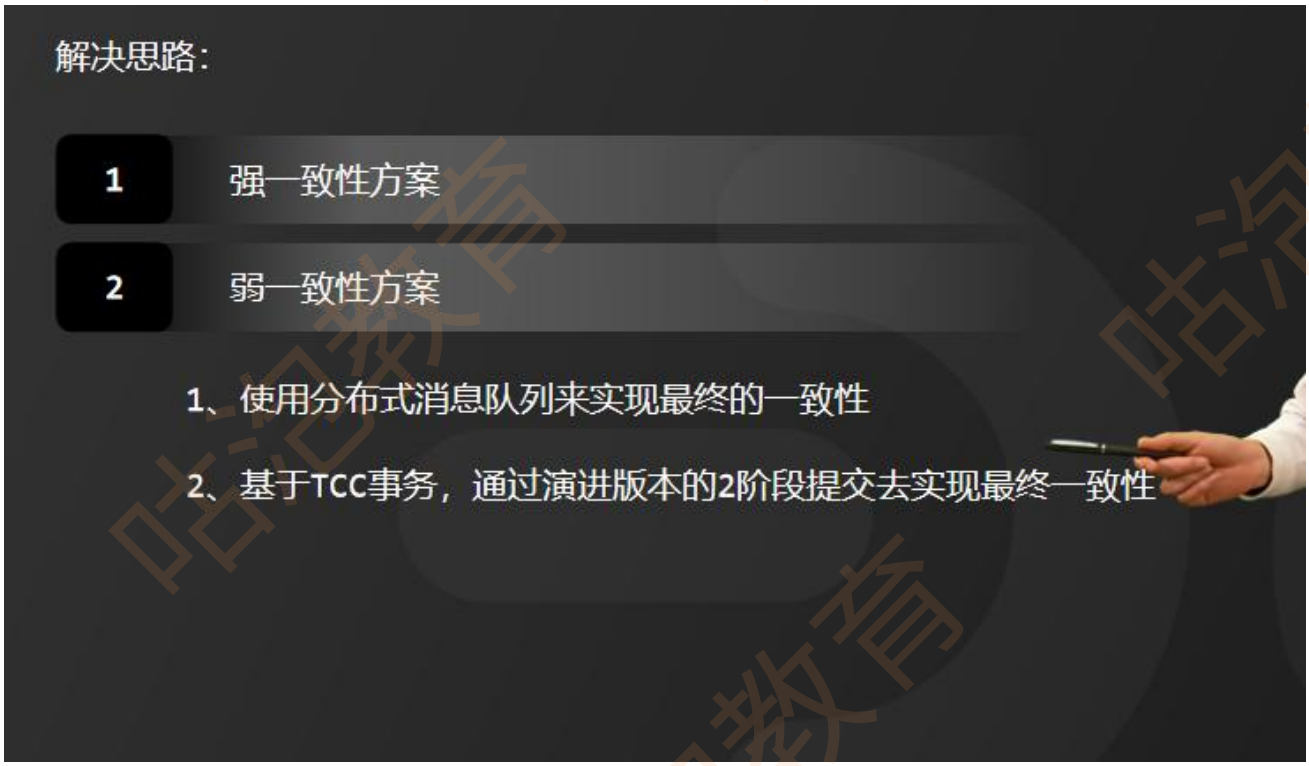
基于CAP定理和Base理论，我们可以知道，对于上述情况产生的分布式问题，我们要么采用强一致性方案，要么采用弱一致性方案。

所谓强一致性方案，是指通过第三方事务管理器来协调多个节点的事务性，保证每一个节点的事务达到同时成功或者同时失败，为了实现这样一个需求。我们可以引入X/Open DTP模型提供的XA协议，基于2阶段提交或者3阶段提交的方式去实现，但是如果全局事务管理器中的多个节点，任意一个节点在进行事务提交确认的时候，由于网络通信延迟导致了阻塞，就会影响到所有节点的事务提交，而这个阻塞过程呢，也会影响到用户的请求线程，这对于用户体验以及整体的性能影响非常大。

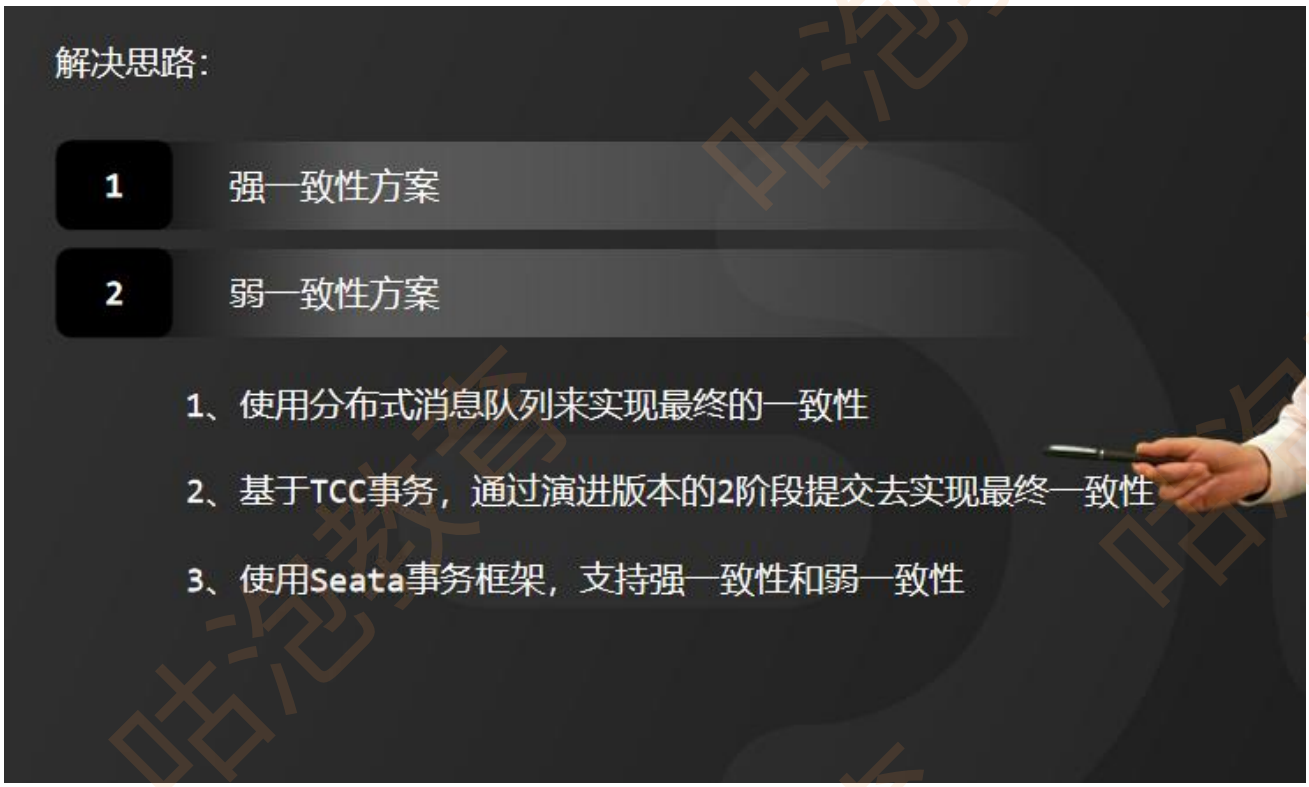
而弱一致性方案就是针对强一致性方案所衍生出来的性能和数据一致性平衡的一个方案。简单来说就是损失掉强一致性，数据在某一个时刻会存在不一致的状态，但是最终这些数据会达成一致，这样的好处是提升了系统的性能。在弱一致性方案中，常见的解决方案有3种：



第1个：使用分布式消息队列来实现最终的一致性。



第2个：基于TCC事务，通过演进版本的2阶段提交去实现最终一致性。



第3个：使用Seata事务框架，它提供了多种事务模型。比如说 AT、XA 、Saga、TCC等，不同的模型提供的是强一致性或者弱一致性的支持。

【结尾】

以上就是我对分布式事务的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

39．synchronized和Lock的区别

【导航条：引言】

最近有多位粉丝被问到synchronized和Lock，据说还是阿里一面的面试题。在分布式开发中，锁是控制线程的重要方式。Java提供了两种锁机制synchronized 和 Lock。接下来，我给大家分享一下我对synchronized和Lock的理解。

另外，我花了1个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

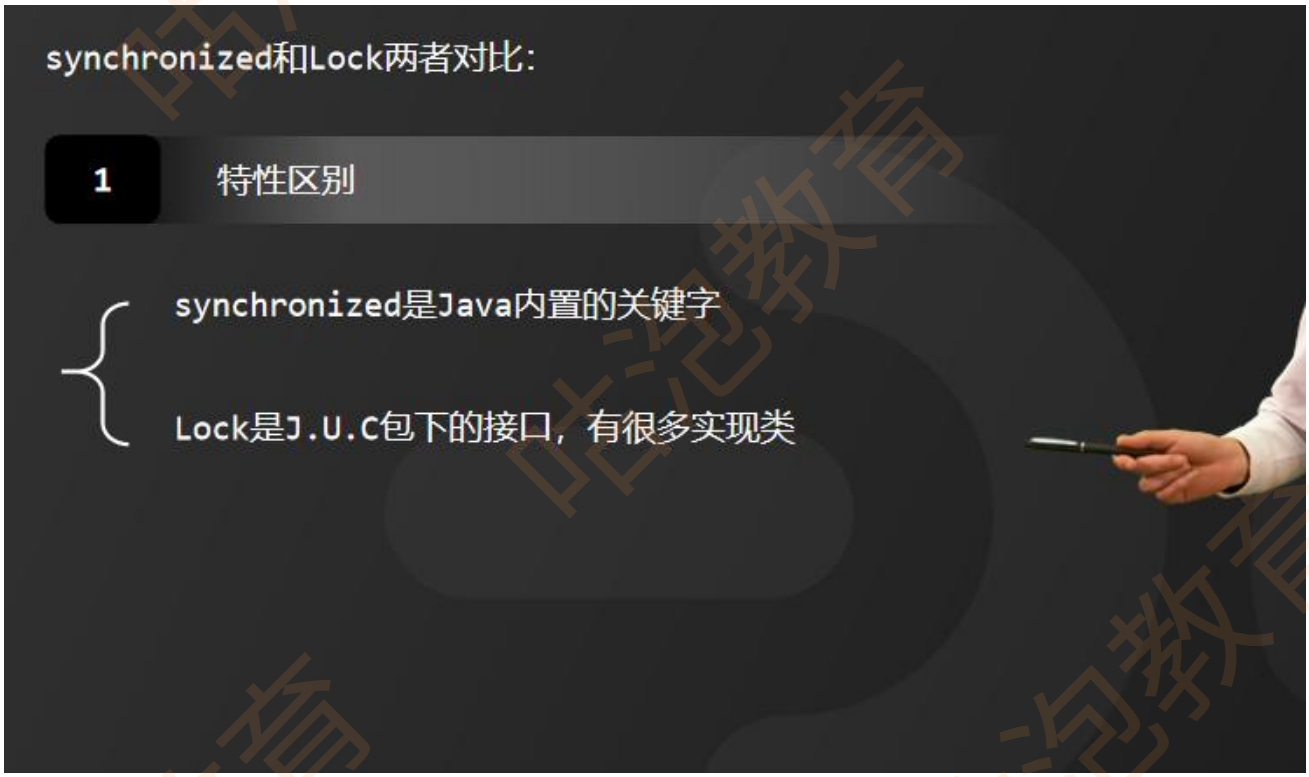
【导航条：两者对比】

（转场，屏幕黑底白字，正中央出现“1 两者对比”）

1、两者对比

synchronized和Lock都是Java中用来解决线程安全问题的一个工具，那么关于synchronized和Lock的区别，我从以下4个方面来给大家来做一个详细的分析：

1）、特性区别



synchronized是Java内置的一个线程同步关键字，

而Lock是J.U.C包下面的一个接口，它有很多实现类，比如ReentrantLock就是它的一个实现类。

2）、用法区别

synchronized可以写在需要同步的对象、方法或者是特定代的码块中。主要有两种写法，比如这样：

一种是把synchronized修饰在方法上

```
//控制方法

public synchronized void sync(){

}
```

一种是把synchronized修饰在代码块上

```
Object lock = new Object();

//控制代码块

public void sync(){

synchronized(lock){



}

}
```

用这种方式来控制锁的生命周期。而Lock控制锁的粒度是通过lock() 和 unlock() 方法来实现的，以ReentrantLock为例，来看这样一段代码：

```
Lock lock = new ReentrantLock();

public void sync(){

lock.lock();  //添加锁

//TODO 线程安全的代码

lock.unlock(); //释放锁
```

```
}
```

这种方式，是可以保证lock()方法和unlock()方法之间的代码是线程安全的。而锁的作用域，取决于Lock实例的生命周期。

Lock比synchronized在使用上相对来说要更加灵活一些。Lock可以自主地去决定什么时候加锁，什么时候释放锁。只需要调用lock()和unlock()这两个方法就可以了。需要注意的是，为了避免死锁，一般我们unlock()方法写在finally块中。

另外，Lock还提供了非阻塞的竞争锁的方法叫trylock()，这个方法可以通过返回true或者fasle来告诉当前线程是否已经有其他线程正在使用锁。

而synchronized是关键字，无法去扩展实现非阻塞竞争锁的方法。另外，synchronized只有代码块执行结束或者代码出现异常的时候才会释放锁，因此，它对锁的释放是被动的。

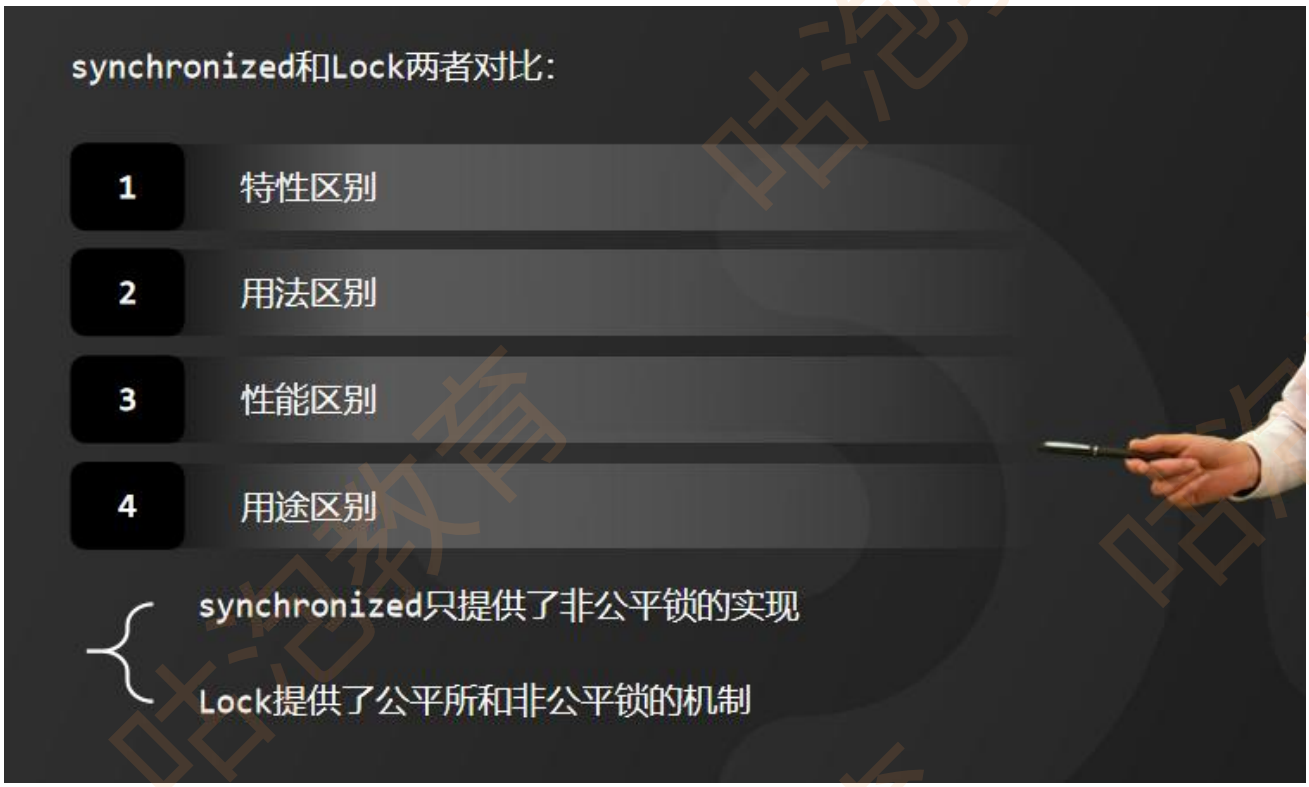
3）、性能区别

synchronized和Lock在性能上差别不大。在实现上有一些区别，

synchronized 采用的是悲观锁机制，synchronized 是托管给 JVM 执行的。在JDK1.6以后采用了偏向锁、轻量级锁、重量级锁及锁升级的方式进行优化。

而 Lock 用的是乐观锁机制。控制锁的代码由用于自定义，也采用CAS自旋锁进行了优化。

4）、用途区别



二者在一般情况下没有什么区别，但是在非常复杂的同步应用中，建议使用Lock。

因为synchronized只提供了非公平锁的实现，而Lock提供了公平所和非公平锁的机制。

公平锁是指线程竞争锁资源的时候，如果已经有其他线程正在排队或者等待锁释放，那么当前竞争锁的线程是无法去插队的。

而非公平锁就是不管是否线程再排队等待锁，它都会去尝试竞争一次锁。

【导航条：总结】

（转场，屏幕黑底白字，正中央出现“2 总结”）

以上，就是我对synchronized和Lock的理解，我还专门整理了一张表格帮助大家更好地理解，有需要这张表的小伙伴可以在我的主页简介中获取。

项	synchronized	Lock
特性	Java的关键字，在JVM层面	J.U.C包中的接口
获取	A获得锁，B等待。A阻塞，B一直等待	可尝试获得锁，线程可以不用一直等待
释放	执行完同步代码或者发生异常，被动释放	在finally中释放锁，避免死锁
状态	无法判断	可以判断
类型	可以重入，不可中断，非公平	可重入，可判断，可公平，可非公平
性能	少量同步	大量同步

【结尾】

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

40．Dubbo和SpringCloud的优缺点对比

【导航条：引言】

其实我个人不太愿意，拿Dubbo和Spring Cloud进行对比，因为它俩最初出现并不是为了解决同一类问题。但是，国内技术是在太卷，加上微服务的盛行，很多互联网大厂也经常会问到这个问题。那么今天，我还是给大家来详细聊一聊。

另外，我花了1个多星期把往期的面试题解析配套文档准备好了，想获取的小伙伴可以在我的煮叶简介中找到。

【导航条：两者对比】

（转场，屏幕黑底白字，正中央出现“1 两者对比”）

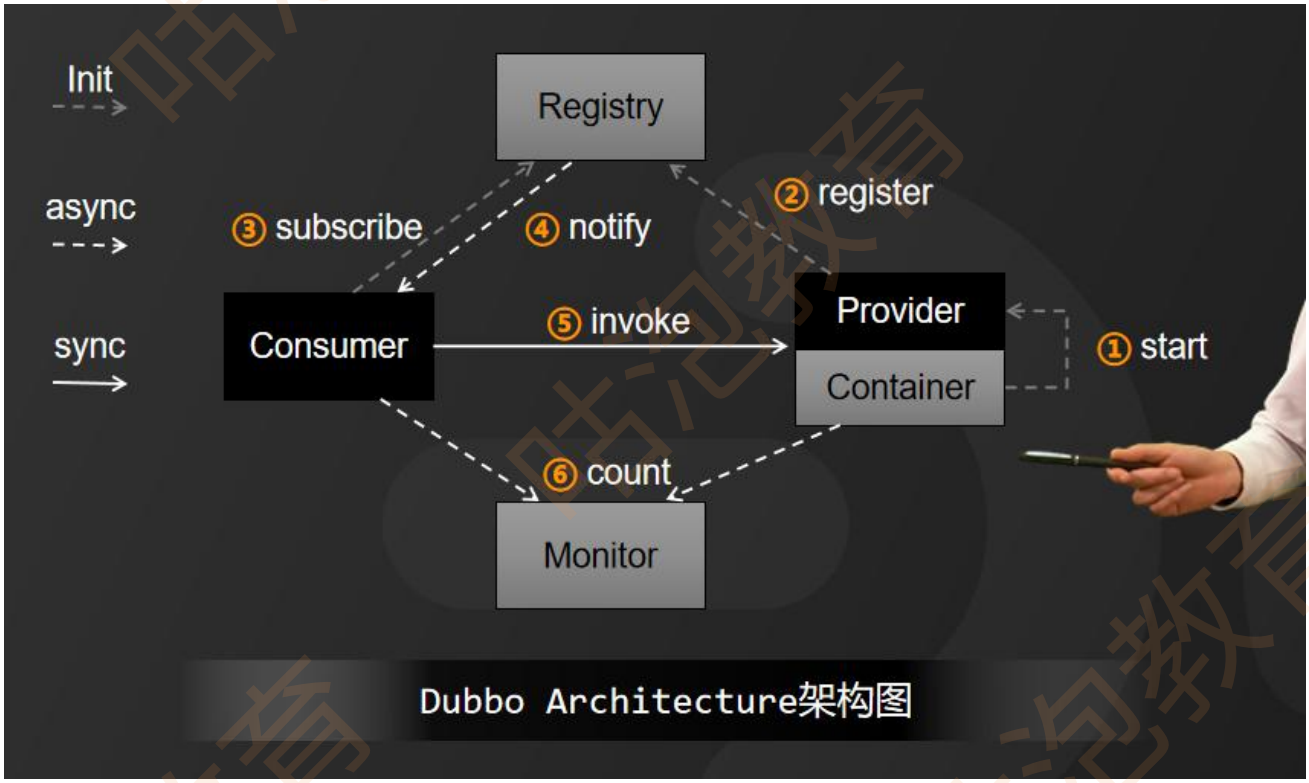
1、两者对比

关于Dubbo和Spring Cloud的优缺点，我以奈菲（Netflix）版本为例，从以下5个方面来分析：

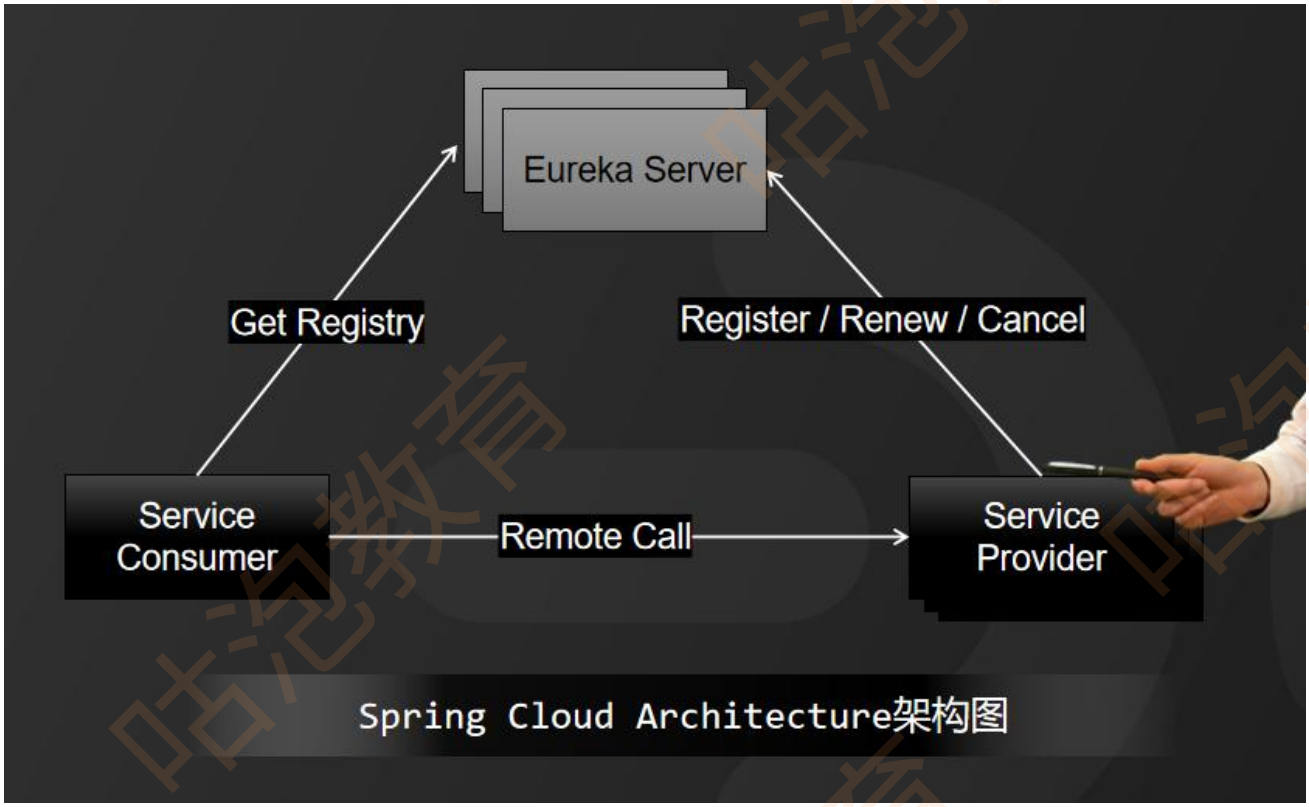
1）、从整体架构上来看

Dubbo和SpringCloud的模式都比较接近，都需要服务提供方，注册中心，服务消费方。差异并不大。

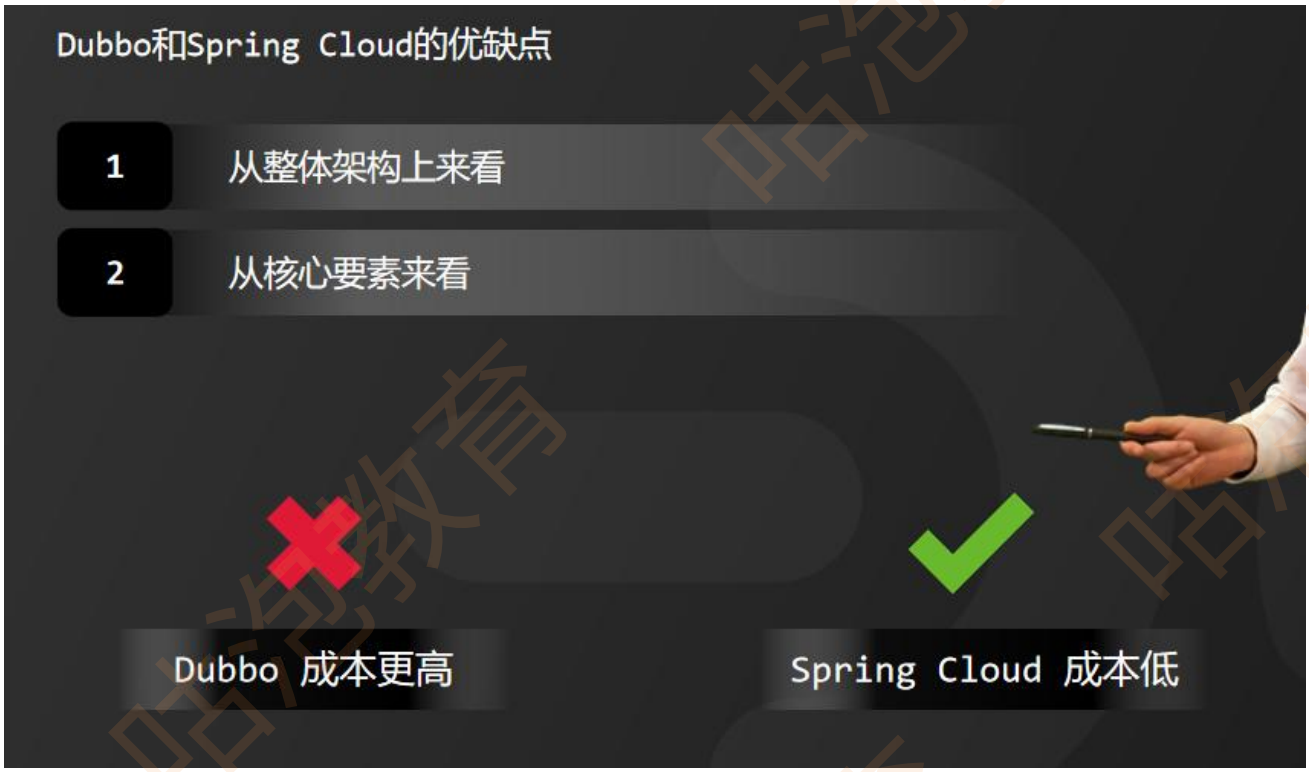
Dubbo的架构图是这样的，



而Spring Cloud的架构图是这样的

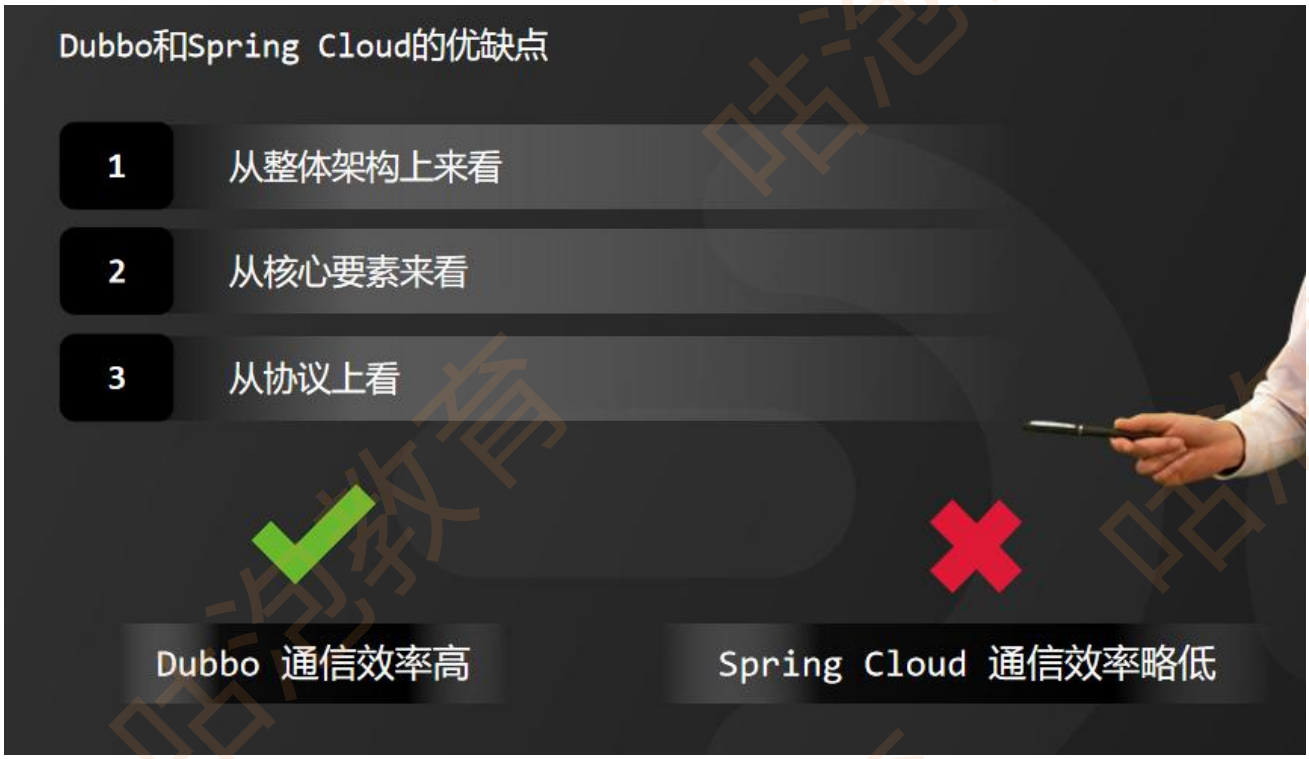

Eureka Server
Get Registry
Register / Renew / Cancel
Service Consumer
Remote Call
Service Provider
Spring Cloud Architecture架构图

2）、从核心要素来看



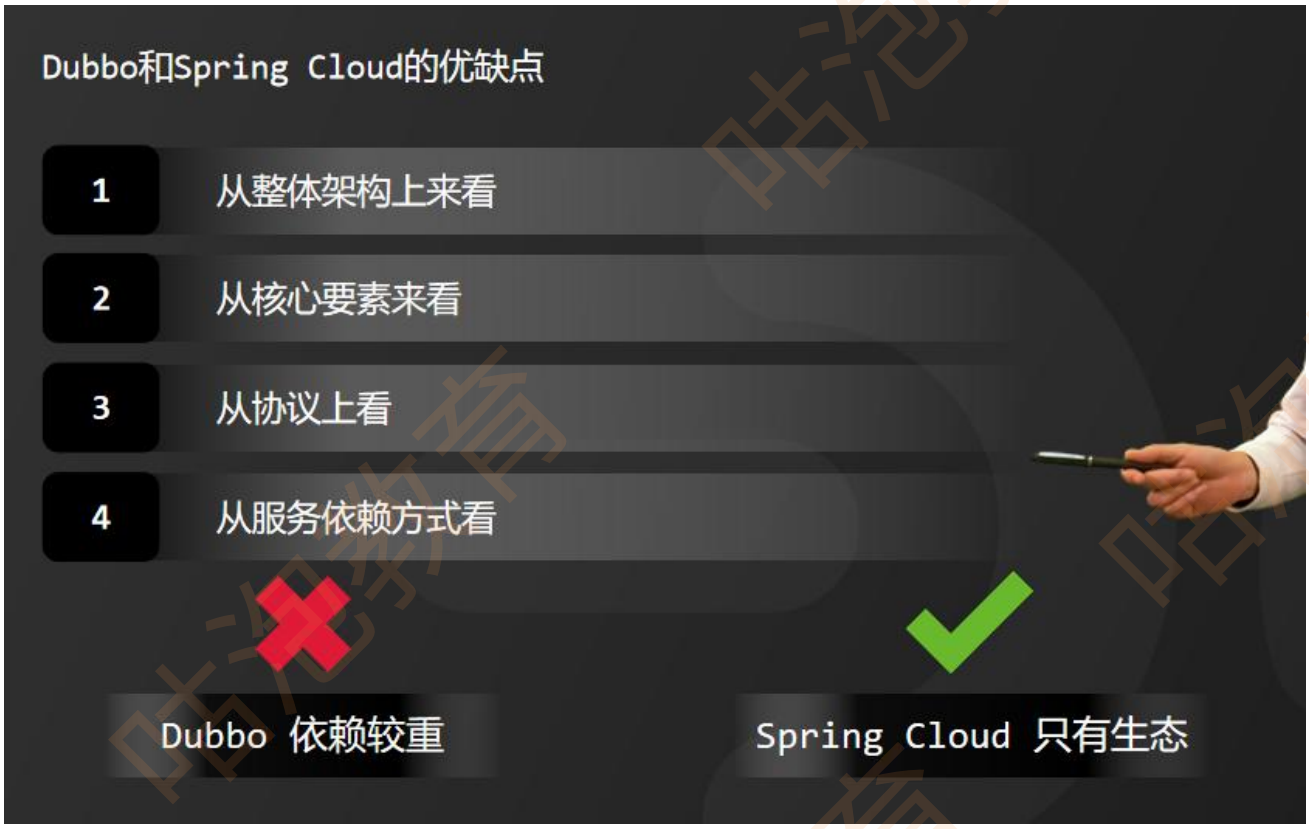
Spring Cloud 更胜一筹，在开发过程中只要整合Spring Cloud的子项目就可以顺利的完成各种组件的融合，而Dubbo需要通过实现各种Filter来做定制，开发成本以及技术难度略高。

3）、从协议上看



Dubbo默认采用的是单一长连接和NIO异步通讯，适合于小数据量大并发的服务调用，以及服务消费者机器数远大于服务提供者机器数的情况。Dubbo还支持其他各种通信协议，而Spring Cloud 使用HTTP协议的REST API。因此，在通信速度上Dubbo略胜。

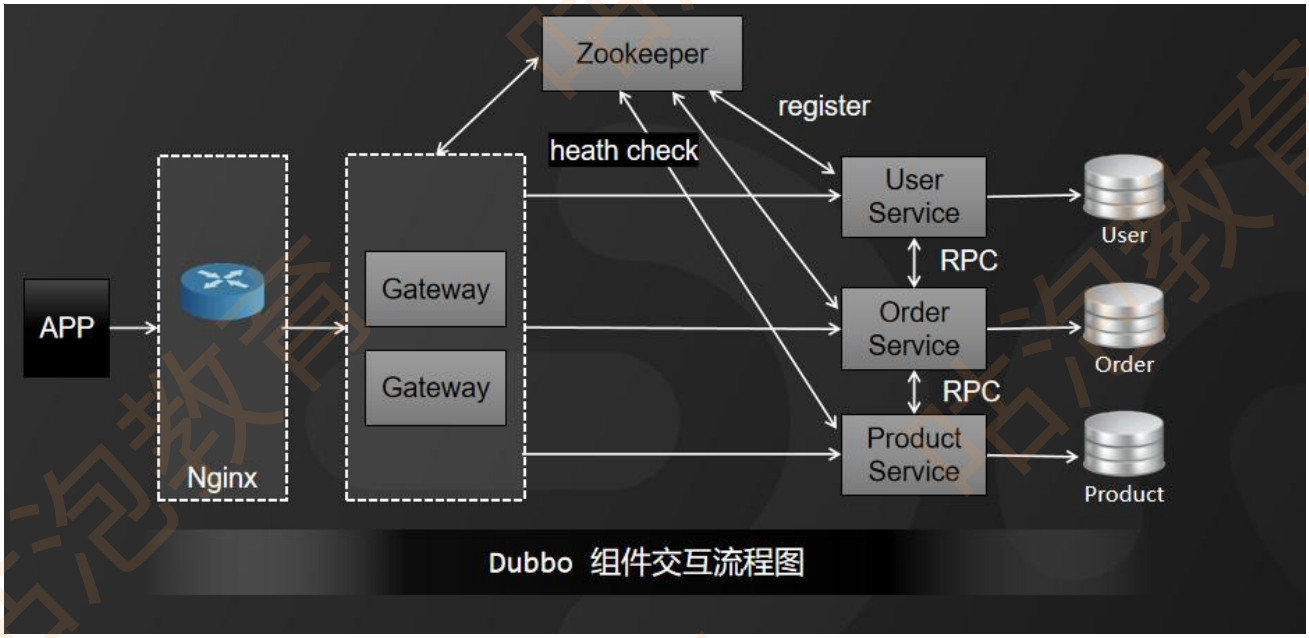
4）、从服务依赖方式看



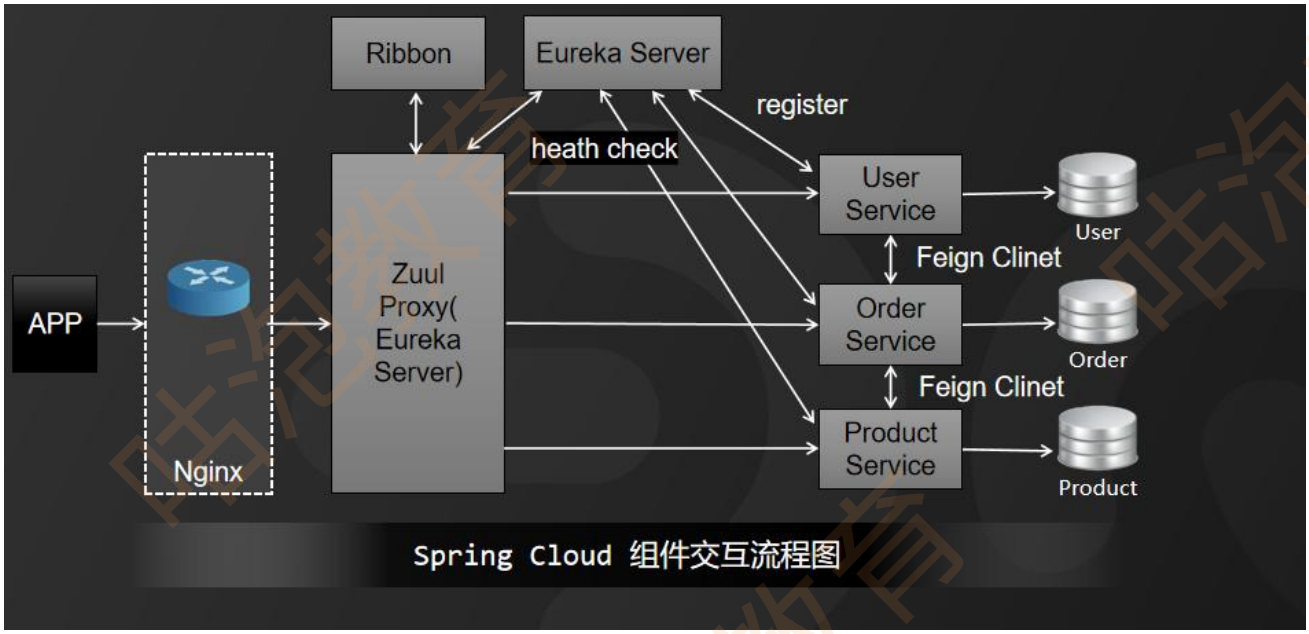
Dubbo服务依赖比较重，需要有完善的版本管理机制，但是程序入侵少。而Spring Cloud是自有生态，省略了版本管理的问题，它使用JSON进行交互，为跨平台调用奠定了基础。

5）、从组件运行流程看

Dubbo的每个组件都是需要部署在单独的服务器上， 用来接收前端请求、聚合服务，并批量调用后台原子服务。每个Service层和单独的DB交互。



而Spring Cloud所有请求都统一通过 API 网关（Zuul）来访问内部服务。网关接收到请求后，从注册中心（Eureka）获取可用服务。由 Ribbon 进行均衡负载后，分发到后端的具体实例。微服务之间通过 Feign 进行通信处理业务。



但是，两者的业务部署方式相同，都需要前置一个网关来隔绝外部直接调用原子服务的风险。Dubbo需要自己开发一套API 网关，而Spring Cloud则可以通过Zuul配置就可以完成网关定制。所以，从使用方式上Spring Cloud更加方便。

【结尾】

以上就是我对Dubbo和Spring Cloud的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

41．为什么MySQL索引结构采用B+树?

【导航条：引言】

一位6年经验的小伙伴去字节面试的时候被问到这样一个问题，为什么MySQL索引结构要采用B+树？这位小伙伴从来就没有思考过这个问题。只因为现在都这么卷，后面还特意查了很多资料，他也希望听听我的见解。

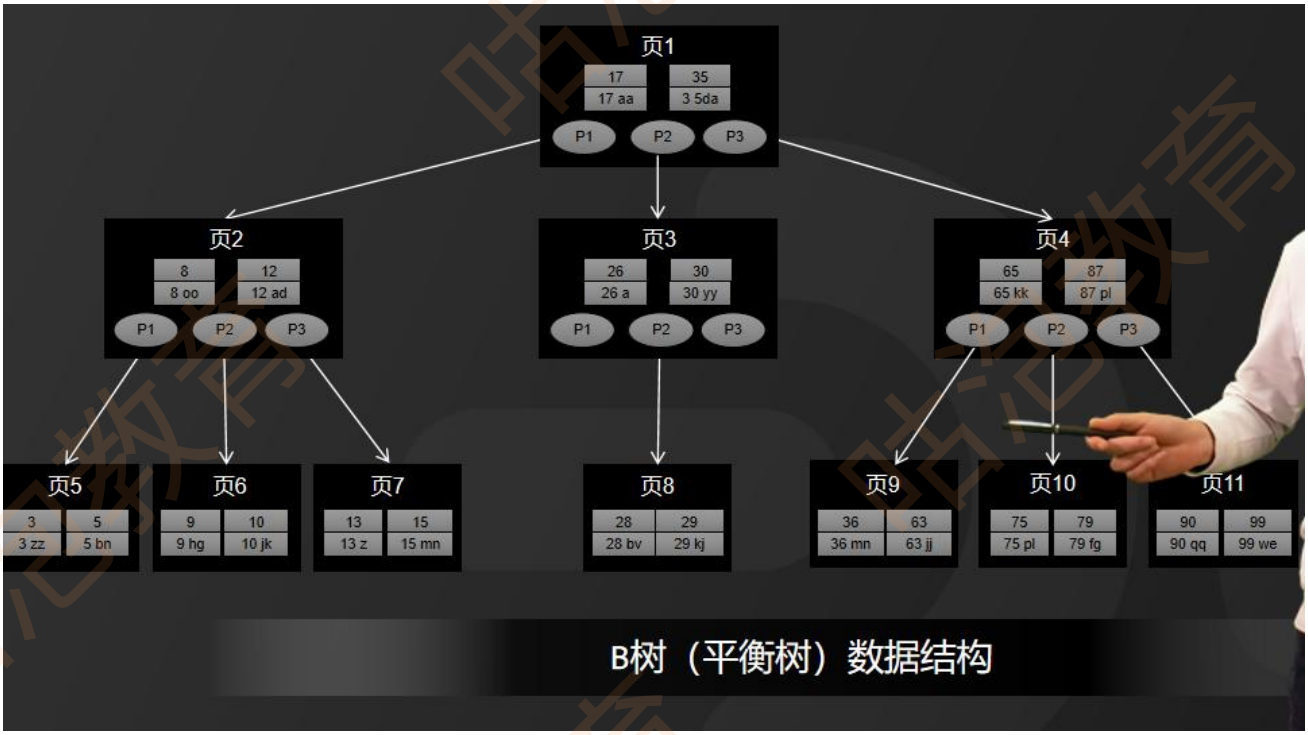
另外，我花了1个多星期把往期的面试题解析配套文档准备好了，一共有10万字，想获取的小伙伴可以在我的煮叶简介中找到。

【导航条：B树和B+树】

（转场，屏幕黑底白字，正中央出现“1 B树和B+树”）

1、B树和B+树

一般来说，数据库的存储引擎都是采用B树或者B+树来实现索引的存储。首先来看B树，如图所示。



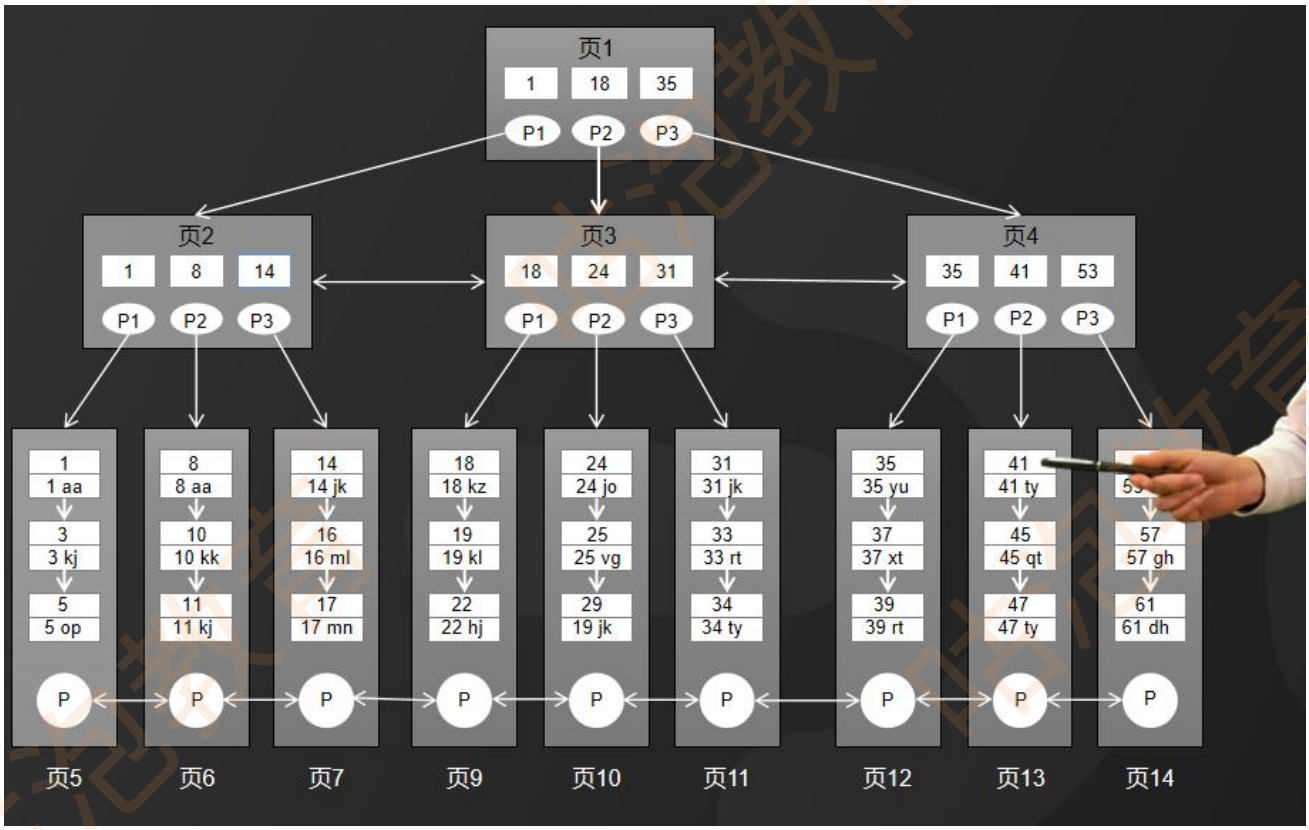
B树是一种多路平衡树，用这种存储结构来存储大量数据，它的整个高度会相比二叉树来说，会矮很多。

而对于数据库而言，所有的数据都将会保存到磁盘上，而磁盘I/O的效率又比较低，特别是在随机磁盘I/O的情况下效率更低。

所以 高度决定了磁盘I/O的次数，磁盘I/O次数越少，对于性能的提升就越大，这也是为什么采用B树作为索引存储结构的原因，如图所示。

而MySQL的InnoDB存储引擎，它用了一种增强的B树结构，也就是B+树来作为索引和数据的存储结构。

相比较于B树结构来说，B+树做了两个方面的优化，如图所示。



1、B+树的所有数据都存储在叶子节点，非叶子节点只存储索引。

2、叶子节点中的数据使用双向链表的方式进行关联。

【导航条：原因分析】

（转场，屏幕黑底白字，正中央出现“2 原因分析”）

2、原因分析

我认为，MySQL索引结构采用B+树，有以下4个原因：

1、从磁盘I/O效率方面来看：B+树的非叶子节点不存储数据，所以树的每一层就能够存储更多的索引数量，也就是说，B+树在层高相同的情况下，比B树的存储数据量更多，间接会减少磁盘I/O的次数。

2、从范围查询效率方面来看：在MySQL中，范围查询是一个比较常用的操作，而B+树的所有存储在叶子节点的数据使用了双向链表来关联，所以B+树在查询的时候只需查两个节点进行遍历就行，而B树需要获取所有节点，因此，B+树在范围查询上效率更高。

3、从全表扫描方面来看：因为，B+树的叶子节点存储所有数据，所以B+树的全局扫描能力更强一些，因为它只需要扫描叶子节点。而B树需要遍历整个树。

4、从自增ID方面来看：基于B+树的这样一种数据结构，如果采用自增的整型数据作为主键，还能更好的避免增加数据的时候，带来叶子节点分裂导致的大量运算的问题。

【导航条：总结】

（转场，屏幕黑底白字，正中央出现“3 总结”）

3、总结

总体来说，我认为技术方案的选型，更多的要根据具体的业务场景来决定，并不一定是说B+树就是最好的选择，就像MongoDB里面采用B树结构，本质上来说，其实是关系型数据库和非关系型数据库的差异。

【结尾】

以上就是我对为什么MySQL索引结构采用B+树 的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

42．谈谈你对MySQL事务隔离级别的理解

【导航条：引言】

一位5年工作经验的粉丝，去阿里面试被问到一个关于数据库事务隔离级别的问题，当时，没有问答上来，希望给他一个参考答案。那么，今天我给大家谈谈我的理解。

另外，我花了1个多星期把往期的面试题解析配套文档准备好了，一共有10W字，想获取的小伙伴可以从我的个人煮叶简介中找到。

【导航条：脏读、幻读、不可重复读】

（转场，屏幕黑底白字，正中央出现“1 脏读、幻读、不可重复读”）

1、脏读、幻读、不可重复读

在SQL操作中，多个事务竞争可能会产生三种不同的现象，分别是脏读、幻读、不可重复读。

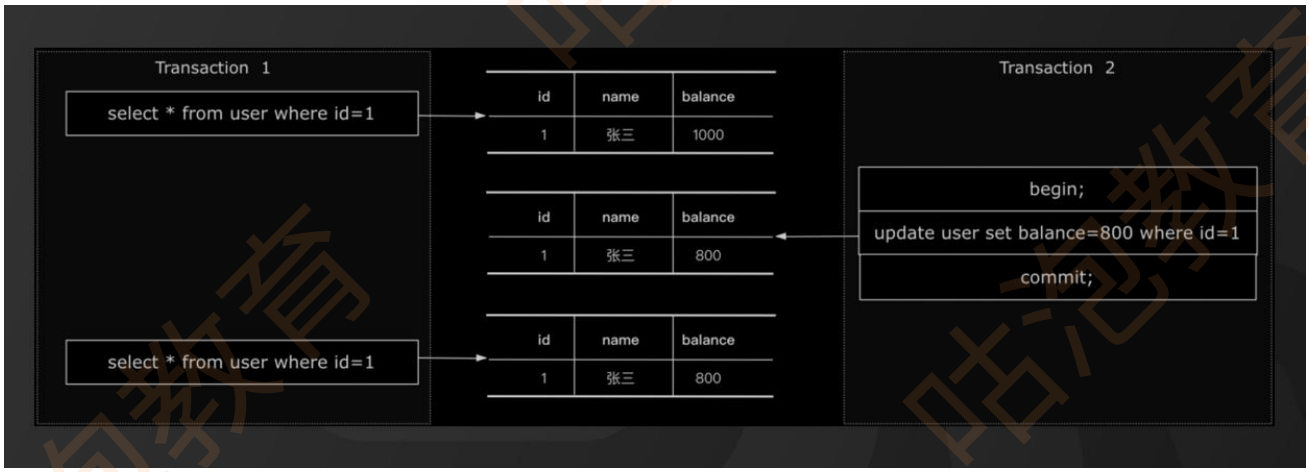
首先来看脏读，如图所示，

假设有两个事务T1/T2同时在执行，T1事务有可能会读取到T2事务未提交的数据，但是未提交的事务T2可能会回滚，也就导致了T1事务读取到最终不一定存在的数据产生脏读的现象。

然后来看幻读，如图所示：

假设有两个事务T1/T2同时执行，事务T1执行范围查询或者范围修改的过程中，事务T2插入了一条属于事务T1范围内的数据并且提交了，这时候在事务T1查询发现多出来了一条数据，或者在T1事务发现这条数据没有被修改，看起来像是产生了幻觉，这种现象称为幻读。

最后来看，不可重复读，如图所示：



假设有两个事务T1/T2同时执行，事务T1在不同的时刻读取同一行数据的时候结果可能不一样，从而导致不可重复读的问题。

【导航条：事务隔离级别】

（转场，屏幕黑底白字，正中央出现“2 事务隔离级别”）

2、事务隔离级别

那么事务隔离级别，就是是为了解决多个并行事务竞争， 。而这脏读、幻读、不可重复读这三种现象在实际应用中，有些业务场景是不能接受这些现象存在的，所以在SQL标准中定义了四种隔离级别，分别是：

读未提交，在这种隔离级别下，可能会产生脏读、不可重复读、幻读。

读已提交（RC），在这种隔离级别下，可能会产生不可重复读和幻读。

可重复读（RR），在这种隔离级别下，可能会产生幻读

串行化，在这种隔离级别下，多个并行事务串行化执行，不会产生安全性问题。

这四种隔离级别里面，只有串行化解决了全部的问题，但这种隔离级别的性能是最低的。

在MySQL里面，InnoDB引擎默认的隔离级别是RR（可重复读），因为它需要保证事务ACID特性中的隔离性特征。

【结尾】

以上就是我对 MySQL事务隔离级别的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

43．什么是双亲委派机制?

【导航条：引言】

今天给大家分享一道国内的一二线互联网公司，高频次出现的面试题。比如，什么是类加载？什么是双亲委派？ 等等。首先，我们来看双亲委派，它全称是Parent Delegation Model，直译过来可能叫做父级委托模型更容易理解。不管它叫什么，如果是你被问到这样的问题，你会不会冷场呢？

那么，今天我给大家分享一下我的理解。

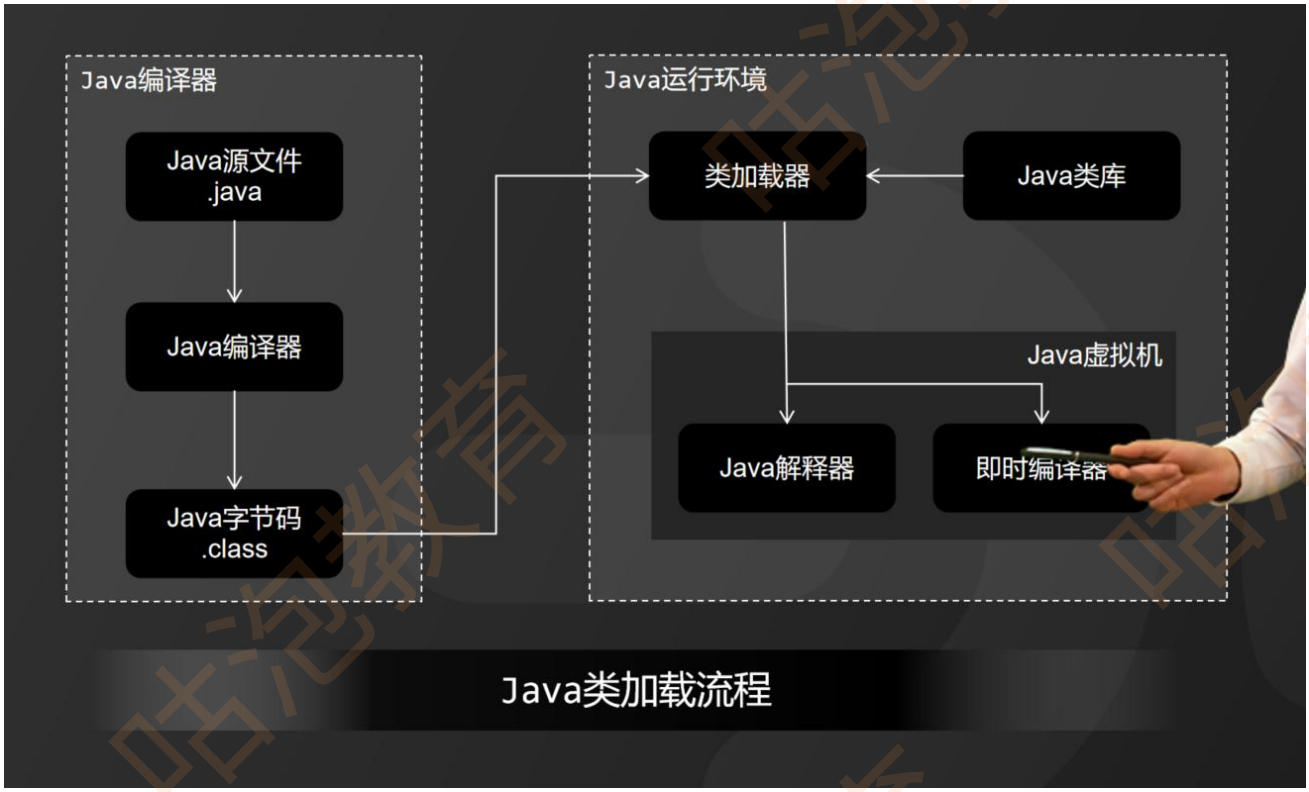
我花了1个多星期把往期的面试题解析配套文档准备好了，一共有10W字，想获取的小伙伴可以从我的个人煮叶简介中找到。

【导航条：类加载机制】

（转场，屏幕黑底白字，正中央出现“1 类加载机制”）

1、类加载机制

要理解双亲委派，首先要理解Java的类加载机制。接下来，我简单介绍一下Java的类加载机制，如图所示：



Java编译器将Java源文件编译成.class文件，再由JVM加载.class文件到内存中，JVM装载完成后得到一个Class字节码对象。拿到字节码对象之后，我们就可以实例化了。

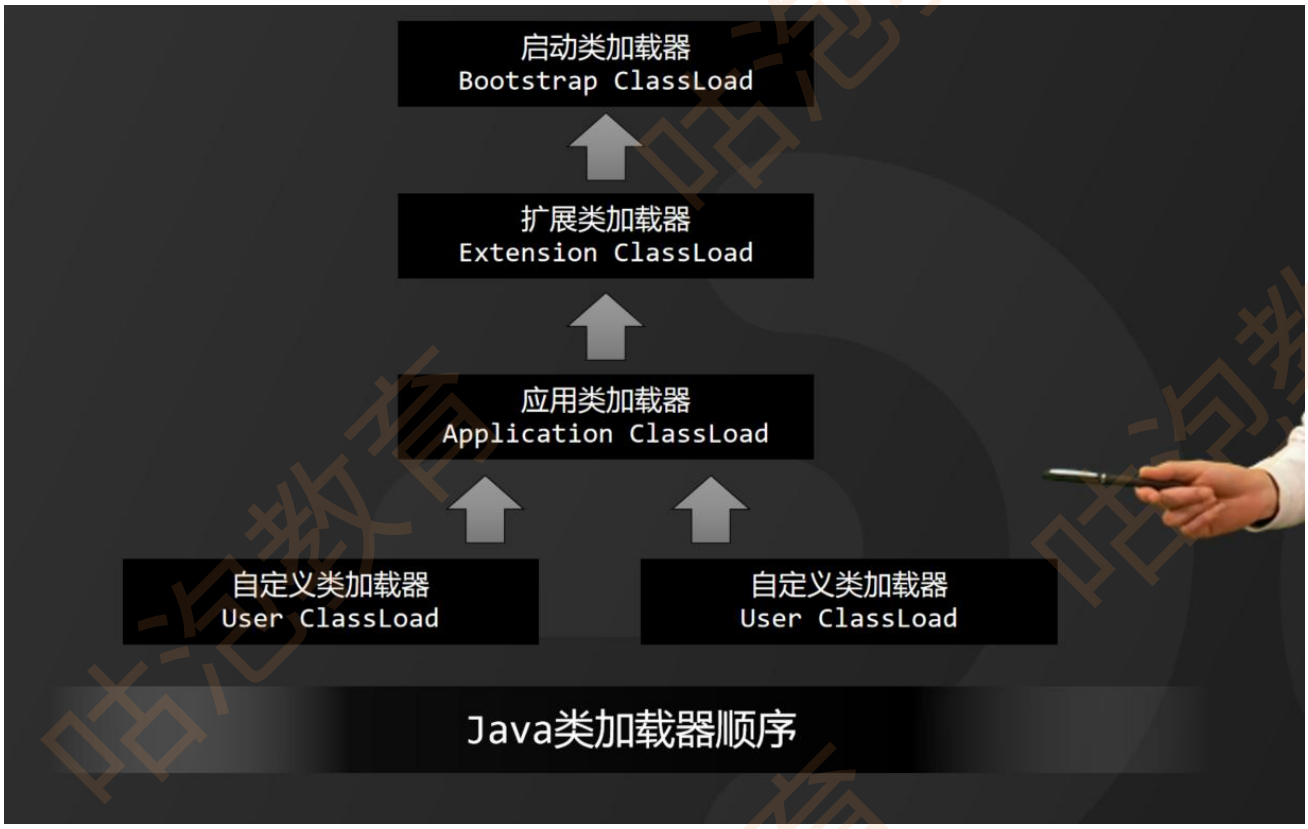
那么，类的加载过程需要使用到加载器。JVM设计了3个类加载器，它们分别是：Bootstrap类加载器、Extension类加载器和 类加载器，这些类加载器分别加载不同作用范围的jar包和.class文件。下面，我给大家详细介绍一下：

1、Bootstrap ClassLoader，主要是负责Java核心类库的加载，也就是 %{JDK_HOME}\lib下的rt.jar、resources.jar等

2、Extension ClassLoader，主要负责%{JDK_HOME}\lib\ext目录下的jar包和class文件

3、Application ClassLoader，主要负责当前应用里面的classpath下的所有jar包和类文件

除了系统自己提供的类加载器以外，还可以通过ClassLoader类实现自定义加载器，去满足一些特殊场景的需求。



【导航条：双亲委派机制】

（转场，屏幕黑底白字，正中央出现“2 双亲委派机制”）

2、双亲委派机制

所谓双亲委派机制，或者叫父级委托模型，就是指按照类加载器的层级关系，逐层进行委派。如图所示：

比如，我们需要加载一个class文件的时候，首先会把这个class的查询和加载委派给父加载器去执行，如果父加载器都无法加载，再尝试自己来加载这个class。

【导航条：总结】

（转场，屏幕黑底白字，正中央出现“3 总结”）

3、总结

我认为，双亲委派机制的设计有2个好处：

保证安全性，因为这种层级关系实际上代表的是一种优先级，也就是所有的类的加载，优先给Bootstrap ClassLoader。那对于核心类库中的类，就没办法去破坏，比如自己写一个java.lang.String，最终还是会交给Bootstrap类加载器加载。再加上每个类加载器的都有不同的作用范围，这就意味着自己写的java.lang.String就没办法去覆盖核心类库中类。

避免重复加载，我认为这种层级关系的设计，可以避免重复加载导致程序混乱的问题。如果父加载器已经加载过了，那么子加载器就没必要去加载了。

【结尾】

好了，以上就是我对双亲委派机制的理解。我是被编程耽误的文艺Tom，如果我的分享对你 有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

44．Netty是什么，为什么要使用Netty?

【导航条：引言】

最近，也不知道什么原因，经常有粉丝问我关于Netty的问题。难道是大厂面试更卷了，开始关注更加底层的框架了？先不深究什么原因了，今天，我给大家分享一下什么是Netty，它能解决什么问题?

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

【导航条：Netty是什么】

（转场，屏幕黑底白字，正中央出现“1 Netty是什么”）

1、Netty是什么

一句话总结，Netty就是一个基于Java NIO封装的高性能的网络通信框架。我从以下三个方面给大家归纳一下：

1、Netty提供了比NIO更简单易用的API，我们可以利用这些封装好的API快速开发自己的网络通信程序。

2、Netty在NIO的基础上还做了很多优化，比如零拷贝机制、、内存池管理等等，因此，总体运行性能会比原

生的NIO更高。

3、Netty内置支持了多种通信协议，如HTTP、WebSocket等，并且针对数据通信的拆包、黏包问题，Netty还内置了解决方案。

【导航条：为什么要用Netty】

（转场，屏幕黑底白字，正中央出现“2 为什么要用Netty”）

2、为什么要用Netty?

Nety相比于直接使Java原生NIO的API来说，选择Netty具备以下优势：

1、Netty提供统一的 API，支持多种通信模型，如阻塞、非阻塞， 以及epoll、poll等模型。

2、Netty可以使用很少的代码实现Reactor多线程模型以及主从线程模型。

3、可以使用自带的编解码器解决 TCP 拆包/粘包问题。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

4、Netty默认提供了多协议的通信支持。

5、Netty处理高吞吐量、低延迟、低资源消耗，比Java原生NIO的API更有优势。

6、经典的开源项目底层也使用到了Netty通信框架， 比如Zookeeper、Dubbo、RocketMQ等等，经历了大型项目的使用和考验更加成熟稳定。

7、Netty对安全性支持也不错，比如支持SSL/TLS等。

【结尾】

好了，以上就是我对Netty的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

45．Netty中提供了哪些线程模型?

【导航条：引言】

最近，我更新了一些Netty相关的内容，于是有很多粉丝开始私信问我一些关于Netty的问题。今天，给大家分享一个大家问得比较多问题，Netty中提供了哪些线程模型?

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

说到线程模型，又不得不说Netty中的Reactor，Reactor直译过来叫做反应堆，它是Netty支持异步多线程的核心组件。常见的Reactor线程模型有三种，分别是：Reactor单线程模型、Reactor多线程模型、主从Reactor多线程模型；

【导航条：单线程】

（转场，屏幕黑底白字，正中央出现“1 单线程单Reactor模型”）

1、单线程单Reactor模型

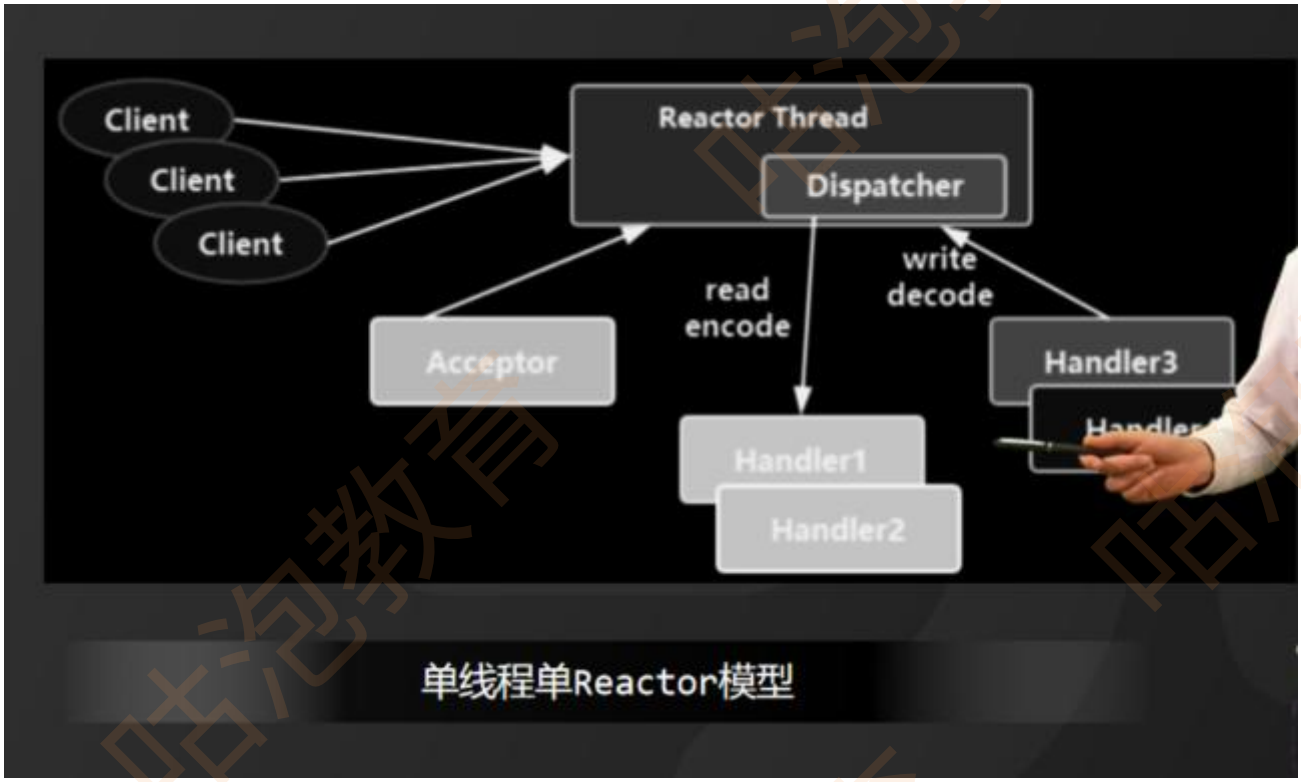
在Reactor模型有三个重要的组件：

1、Reactor： 主要负责将I/O事件发派给对应的Handler

2、Acceptor： 用于处理客户端连接请求

3、Handlers： 执行非阻塞的I/O读写任务

首先来看单线程单Reactor模型，如图所示：



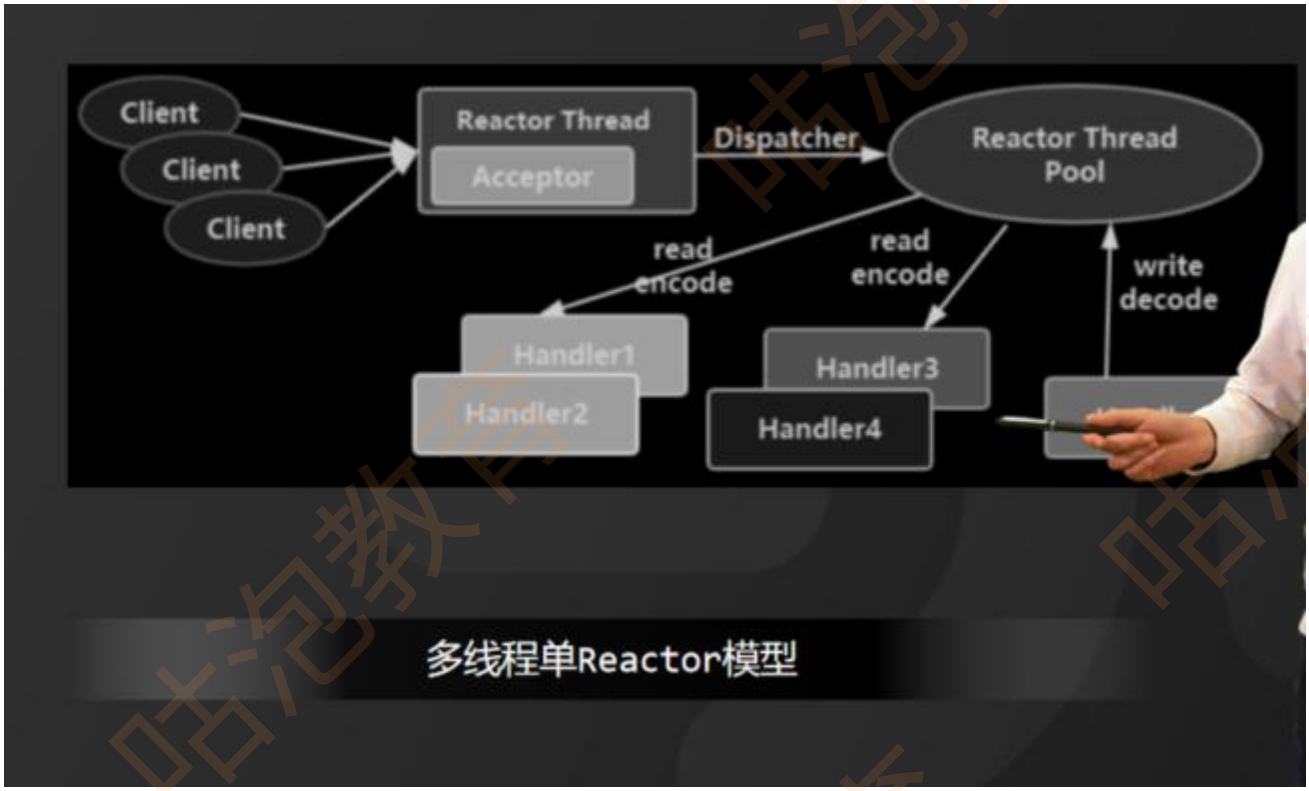
单线程Reactor这种实现方式存在缺点，因为，Handler的执行是串行的，如果其中一个Handler处理线程阻塞，将导致其他的业务处理也会阻塞。而Handler和Reactor在同一个线程中的执行，这也将导致无法接收新的请求。

【导航条：多线程】

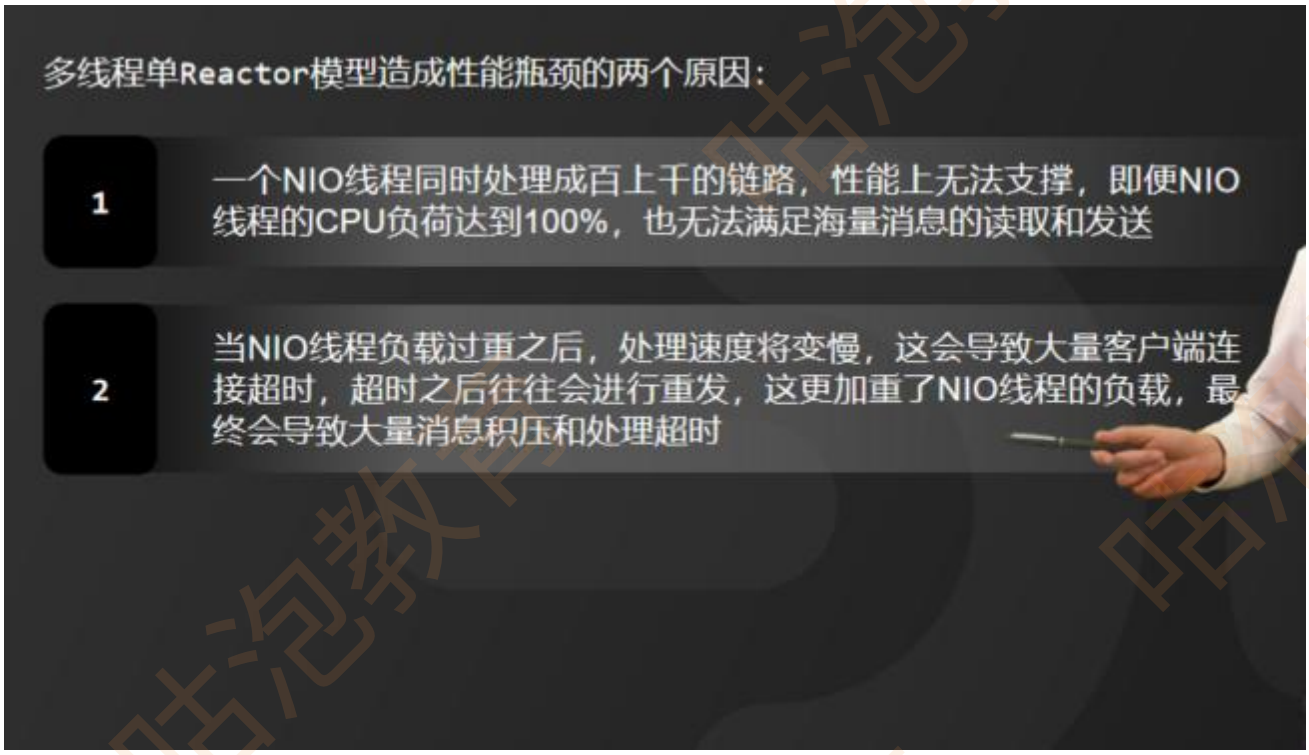
（转场，屏幕黑底白字，正中央出现“2 多线程单Reactor模型”）

2、多线程单Reactor模型

为了解决单线程Reactor的问题，有人提出使用多线程的方式来处理业务逻辑，也就是在业务处理的地方加入线程池，实现异步处理，这样将Reactor和Handler就放在不同的线程中来执行，这就是多线程单Reactor模型。



但是，问题又来了，在多线程单Reactor模型中，所有的I/O操作是由一个Reactor来完成，而 运行在单个线程中，它需要处理包括accept()/read() /write()/connect()等操作，在并发量小的情况下影响不大。一旦并发量上来，出现高负载、高并发或大数据量的应用场景时，容易成为瓶颈，主要有以下2个原因：



1、一个NIO线程同时处理成百上千的链路，性能上无法支撑，即便NIO线程的CPU 达到100%，也无法满足海量消息的读取和发送；

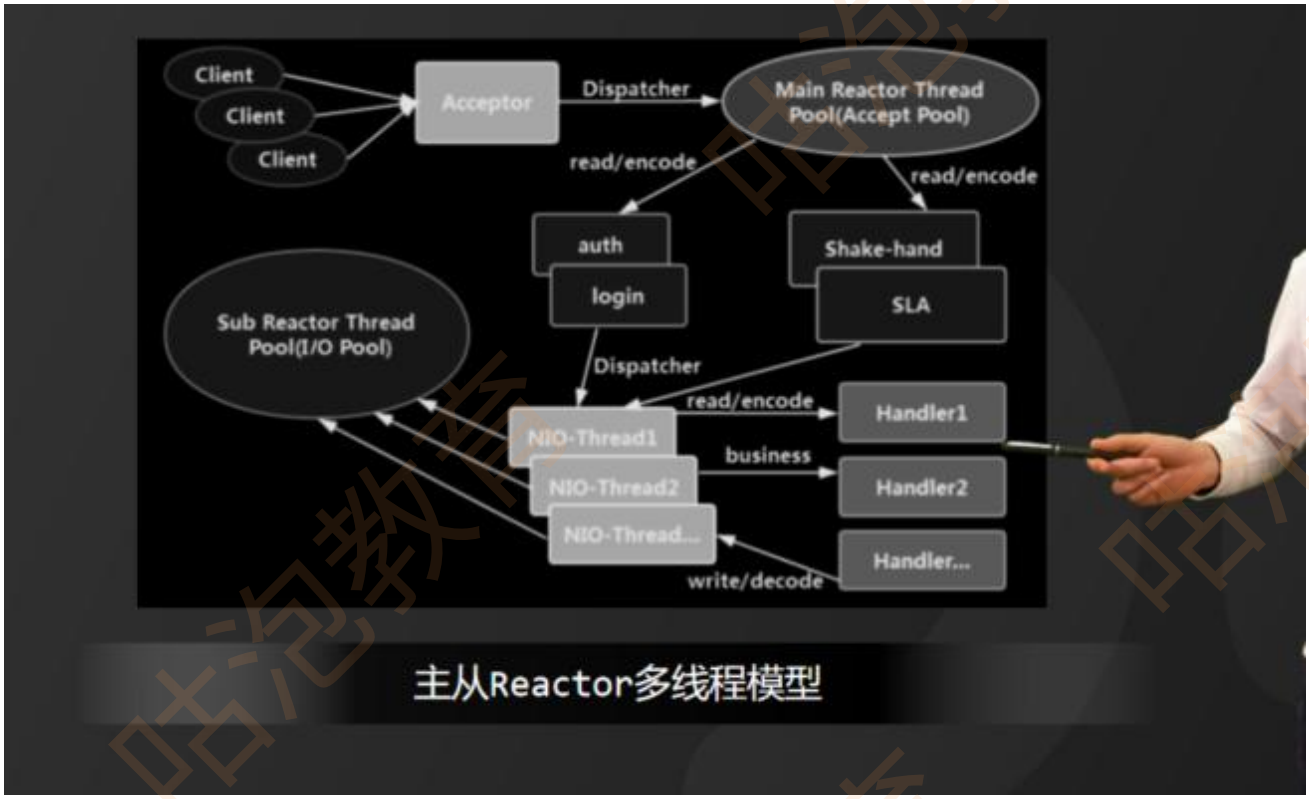
2、当NIO线程负载过重之后，处理速度将变慢，这会导致大量客户端连接超时，超时之后往往会进行重发，这更加重了NIO线程的负载，最终会导致大量消息积压和处理超时，成为系统的性能瓶颈；

所以，我们还可以更进一步优化，引入了主从Reactor多线程模式：

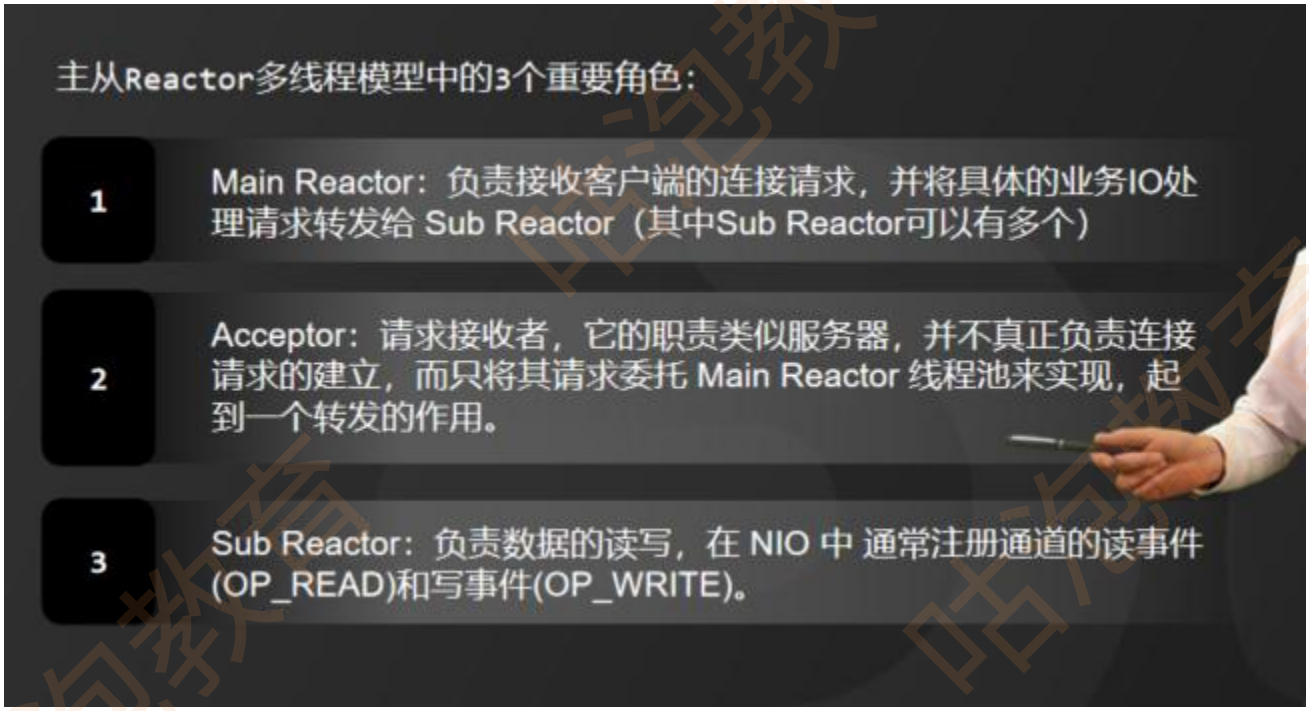
【导航条：主从多线程】

（转场，屏幕黑底白字，正中央出现“3 主从Reactor多线程模型”）

3、主从Reactor多线程模型



如图所示，在主从Reactor多线程模型中，由三个主要的角色：



Main Reactor：负责接收客户端的连接请求，并将具体的业务IO处理请求转发给 Sub Reactor（其中Sub Reactor可以有多个）。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

Acceptor：请求接收者，它的职责类似服务器，并不真正负责连接请求的建立，而只将其请求委托 Main Reactor 线程池来实现，起到一个转发的作用。

Sub Reactor：负责数据的读写，在 NIO 中 通常注册通道的读事件(OP_READ)和写事件(OP_WRITE)。

【结尾】

好了，以上就是我对Netty线程模型的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

46．Netty中有哪些核心组件?

【导航条：引言】

最近又有粉丝问我这样一个问题，说Netty中最核心的组件有哪些？它们都起什么作用？今天，给大家详细聊一聊

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

【导航条：组件分层】

（转场，屏幕黑底白字，正中央出现“1 组件分层”）

1、组件分层

我把Netty的核心组件分为三层，分别是网络通信层、事件调度层和服务编排层。

【导航条：Netty核心组件】

（转场，屏幕黑底白字，正中央出现“2 Netty核心组件”）

2、网络通信层

在网络通信层有三个核心组件：Bootstrap、ServerBootStrap、Channel

Bootstrap：负责客户端启动并用来链接远程Netty Server；

ServerBootStrap：负责服务端监听，用来监听指定端口；

Channel：相当于完成网络通信的载体。

【导航条：事件调度层】

（转场，屏幕黑底白字，正中央出现“1 事件调度层”）

3、事件调度层

事件调度器有两个核心组件：EventLoopGroup与EventLoop

EventLoopGroup：本质上是一个线程池，主要负责接收I/O请求，并分配线程执行处理请求。

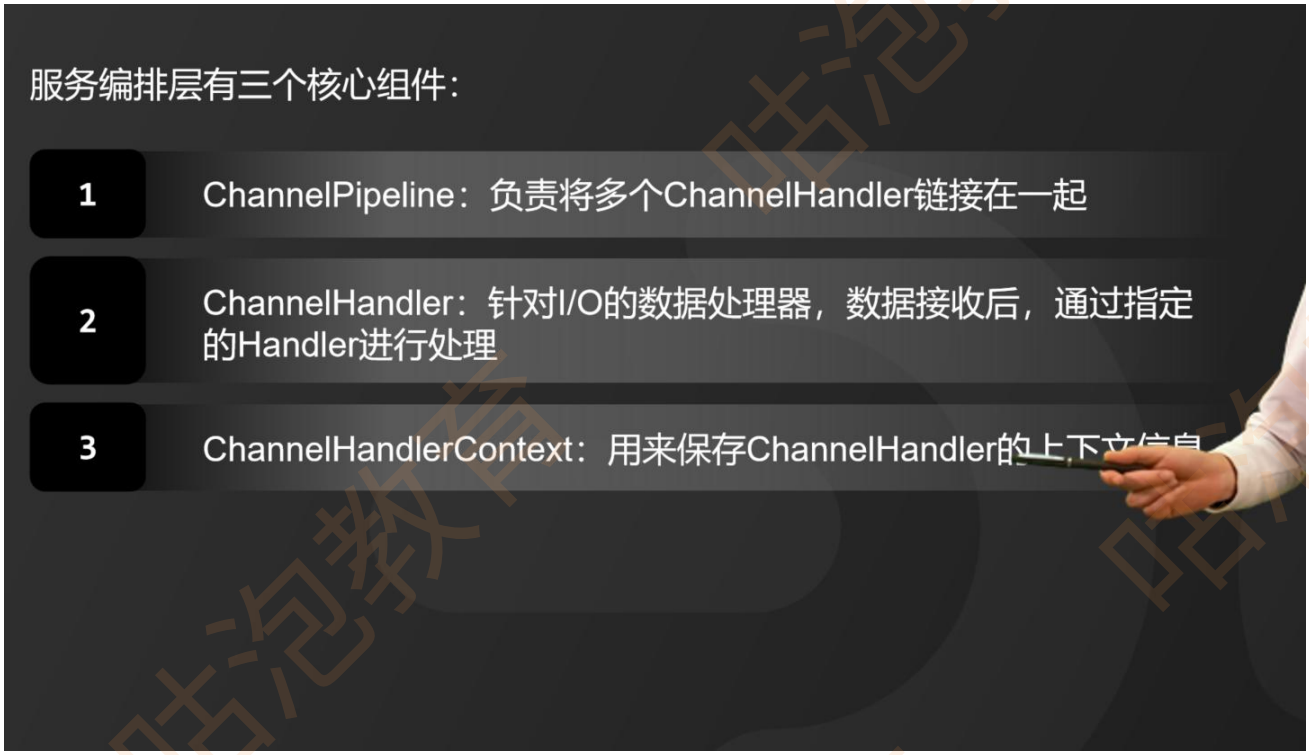
EventLoop：相当于线程池中的线程

【导航条：服务编排层】

（转场，屏幕黑底白字，正中央出现“3 服务编排层*”）*

4、服务编排层

在服务编排层有三个核心组件ChannelPipeline、ChannelHandler、ChannelHandlerContext



ChannelPipeline：负责将多个ChannelHandler链接在一起

ChannelHandler：针对I/O的数据处理器，数据接收后，通过指定的Handler进行处理。

ChannelHandlerContext：用来保存ChannelHandler的上下文信息

【结尾】

以上就是我对Netty核心组件的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

47．MyBatis配置中的#{}和${}有什么区别?

【导航条：引言】

前几天，一位应届生去面试，被问到一个MyBatis中比较基础的问题，说MyBatis中的#号和$符号有什么区别？今天，我给大家来详细介绍一下。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

【导航条：两者区别】

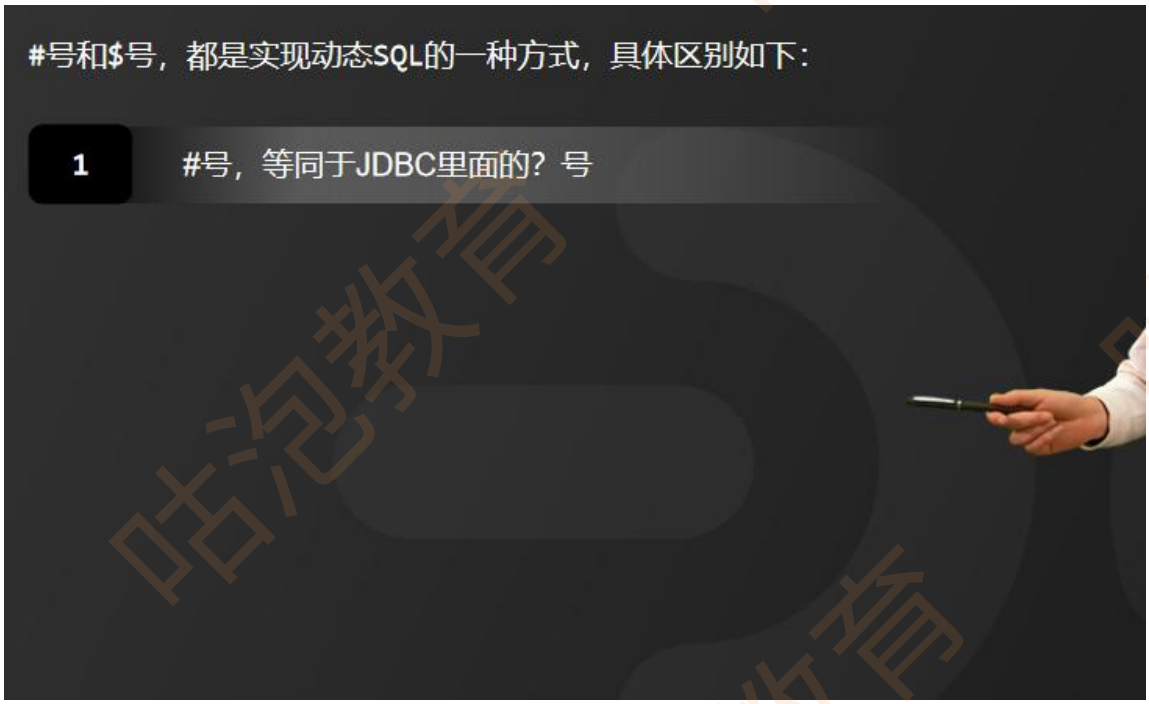
（转场，屏幕黑底白字，正中央出现“1 两者区别”）

1、两者区别

Mybatis提供到的#号和$号，都是实现动态SQL的一种方式，通过这两种方式把参数传递到XML之后，在执行操作之前，Mybatis会对这两种占位符进行动态解析。

下面我给家详细介绍一下#号和$号的区别，

首先，来看#号，等同于JDBC里面的？号（占位符）。



它相当于向PreparedStatement预处理语句中设置参数，而PreparedStatement中的SQL语句是预编译的，如果在设置的参数包含特殊字符，会自动进行转义。所以#号占位符可以防止SQL注入。

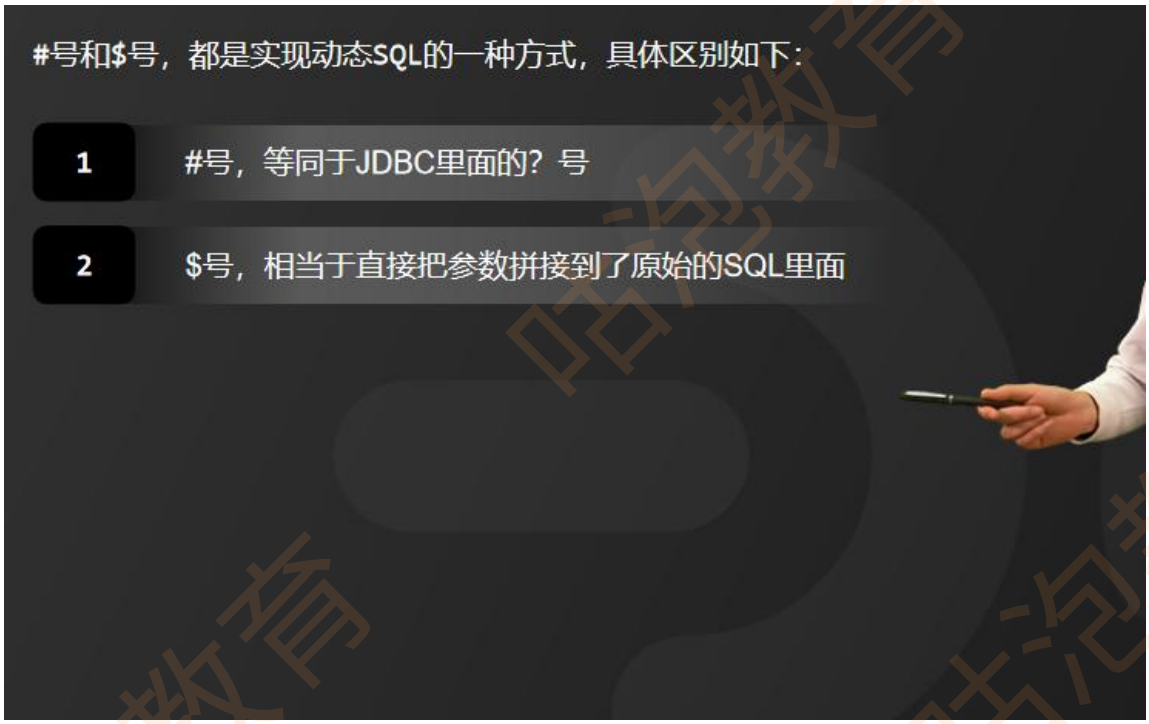
比如：这样一段代码：

解析前：

select * from user order by #{age} desc;

解析后：

select * from user order by ? desc;



而使用$号，相当于直接把参数拼接到了原始的SQL里面，MyBatis不会对它进行特殊处理。

解析前：

select * from user order by ${age} desc;

解析后：

select * from user order by age desc;

所以$和#最大的区别在于，前者是动态参数，后者是占位符， 动态参数无法防止SQL注入的问题，所以在实际应用中，应该尽可能的使用#号占位符。

另外，$符号的动态传参，可以适合应用在一些动态SQL场景中，比如动态传递表名、动态设置排序字段等。

2、总结

一些小的细节如果不注意，就有可能造成巨大的经济损失。在技术如此成熟的互联网时代，还是会有一些网站经常出现SQL注入导致信息泄露的问题。

【结尾】

以上就是我对MyBatis配置#号和$号的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

48．谈谈你对MQ(消息中间件)的理解

【导航条：引言】

昨天，有位56年工作经验的粉丝面试被问到，什么是消息中间件。平时只关注使用，并没有去总结过，竟然被这个问题住了。今天，我给大家来聊一聊。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

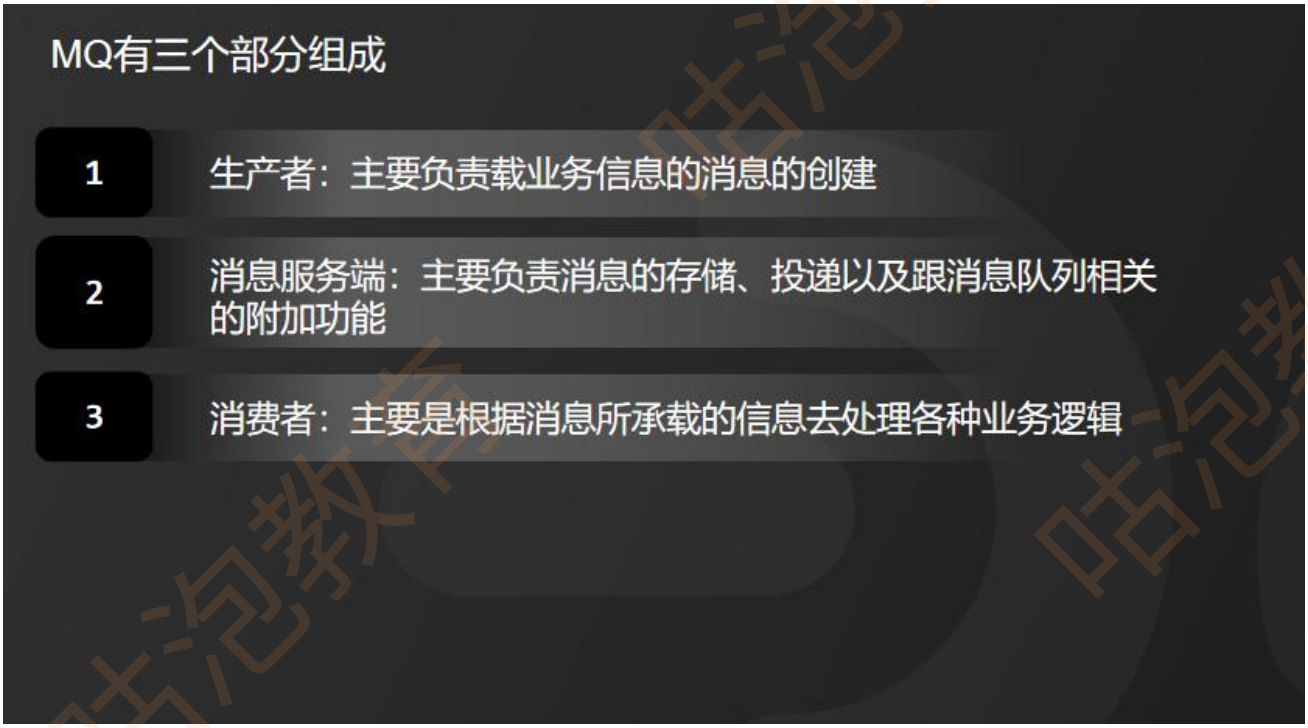
1、什么是MQ

【导航条：什么是MQ】

（转场，屏幕黑底白字，正中央出现“1 什么是MQ”）

MQ全称是Message Queue，直译过来叫做消息队列，主要是作为分布式应用之间实现异步通信的方式。

主要由三个部分组成，分别是生产者、消息服务端和消费者。



生产者（Producer），是生产消息的一端，相当于消息的发起方，主要负责载业务信息的消息的创建。

然后是消息服务端（Server），是处理消息的单元，本质就是用来创建和保存消息队列，它主要负责消息的存储、投递以及跟消息队列相关的附加功能。消息服务端是整个消息队列最核心的组成部分。

第三个是消费者（Consumer），是消费消息的一端，主要是根据消息所承载的信息去处理各种业务逻辑。

2、MQ的应用场景

【导航条：应用场景】

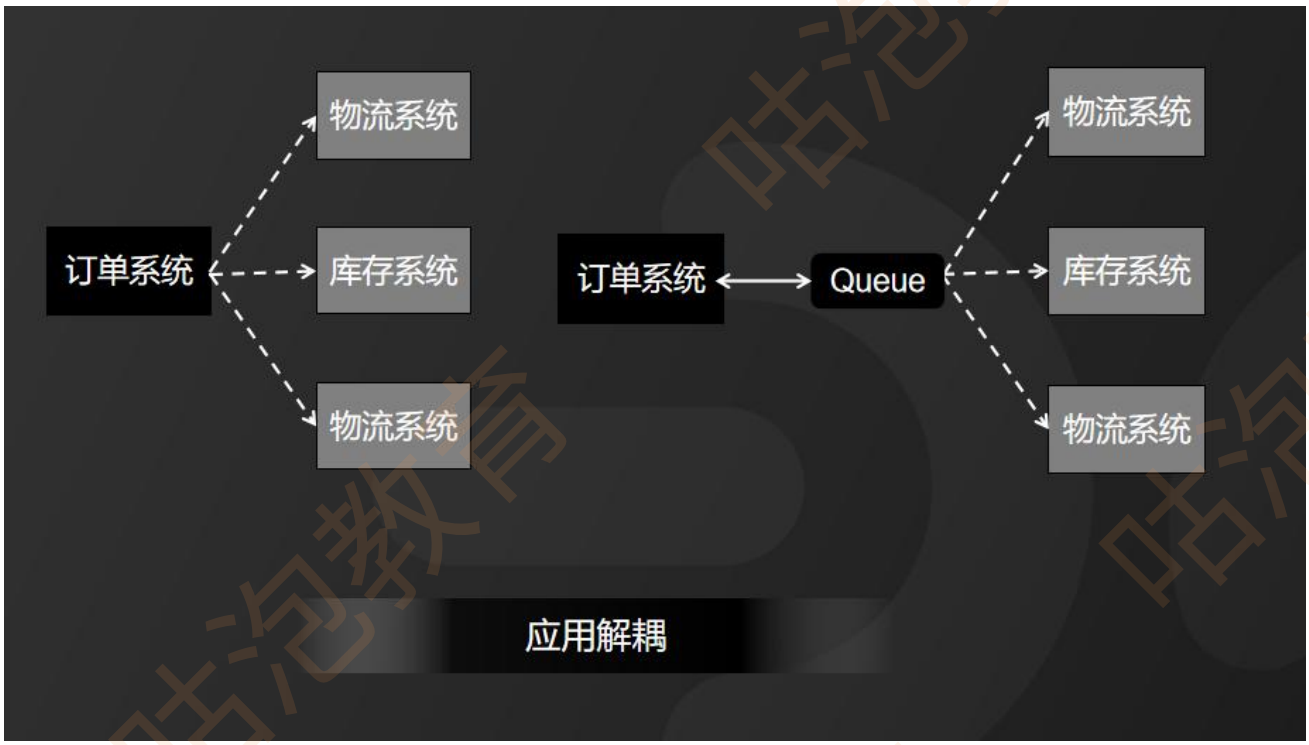
（转场，屏幕黑底白字，正中央出现“2 MQ的应用场景”）

那么，我们为什么要用MQ，它能解决什么问题呢？我从以下三个方面来分析：

第一个是流量消峰，主要是针对大流量入口过大，而业务需求又要短时间响应。但是服务器性能无法满足，导致大量请求积压，从而使得客户端大量等待超时的场景。为了保证高可用，可以把大量的并行任务发送给MQ，MQ再将请求分发给其他服务器，从而平稳地处理后续的业务。起到一个大流量缓冲的作用。

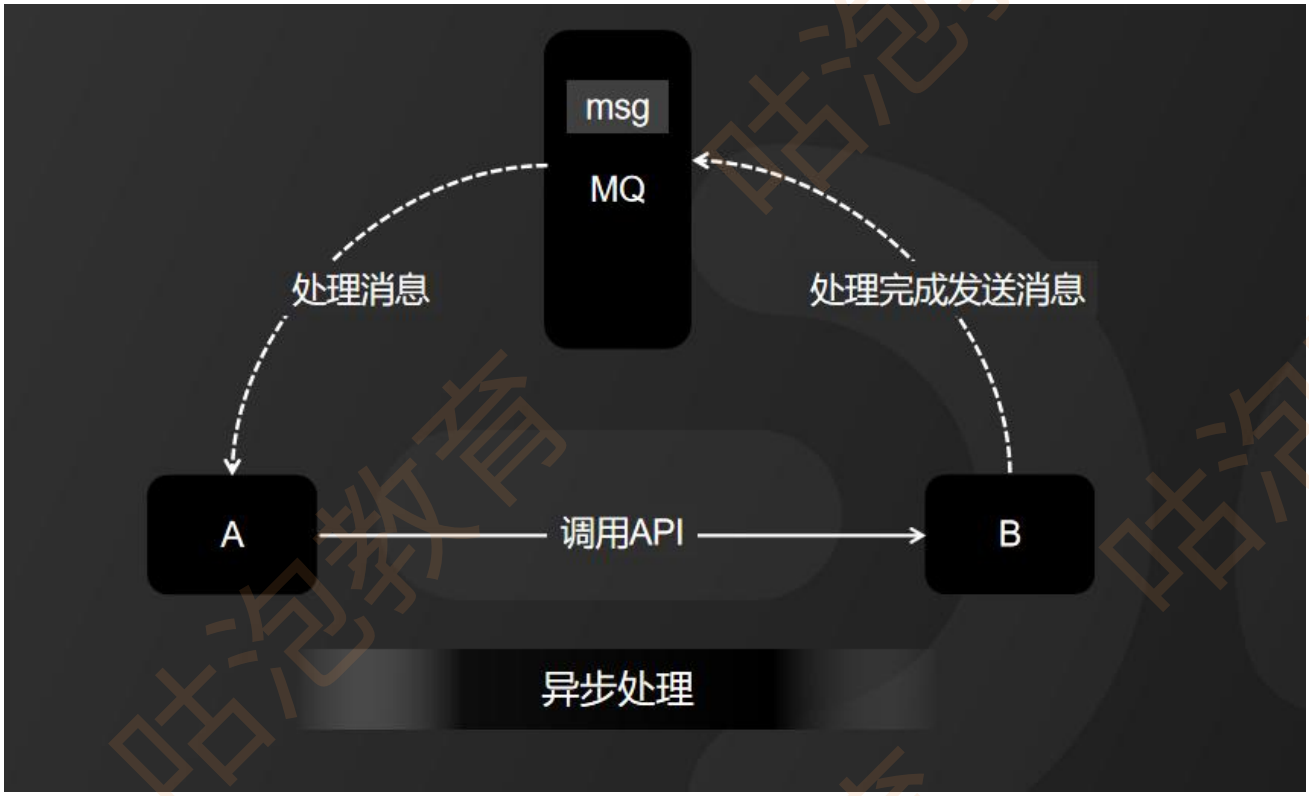
举个例子，假设订单系统最大QPS是1万，这个处理能力应付正常时段在1秒内就能返回结果。但是在流量高峰期，比如促销秒杀，如果QPS达到2万，订单系统就处理不过来了，只能在超过负载后不允许用户下单。如果使用消息队列做缓冲，我们可以取消这个限制，把超出负载的订单分散成一段时间来处理，这时有些用户可能在下单十几秒后才能收到下单成功的操作，但是比不能下单的体验要好。

第二个是应用解耦



可以理解为，把一些相关的、但耦合度不高的系统关联起来，以电商应用为例，比如订单系统、库存系统、物流系统、支付系统有关联，但是有没有那么紧密。每个系统将一些约定好的消息发送到 ，另外的系统直接去消费这些消息就可以了，它可以解决不同系统之间使用不同的框架或者不同编程语言的兼容性问题，从而提高整个系统的灵活性。

第三个是异步处理

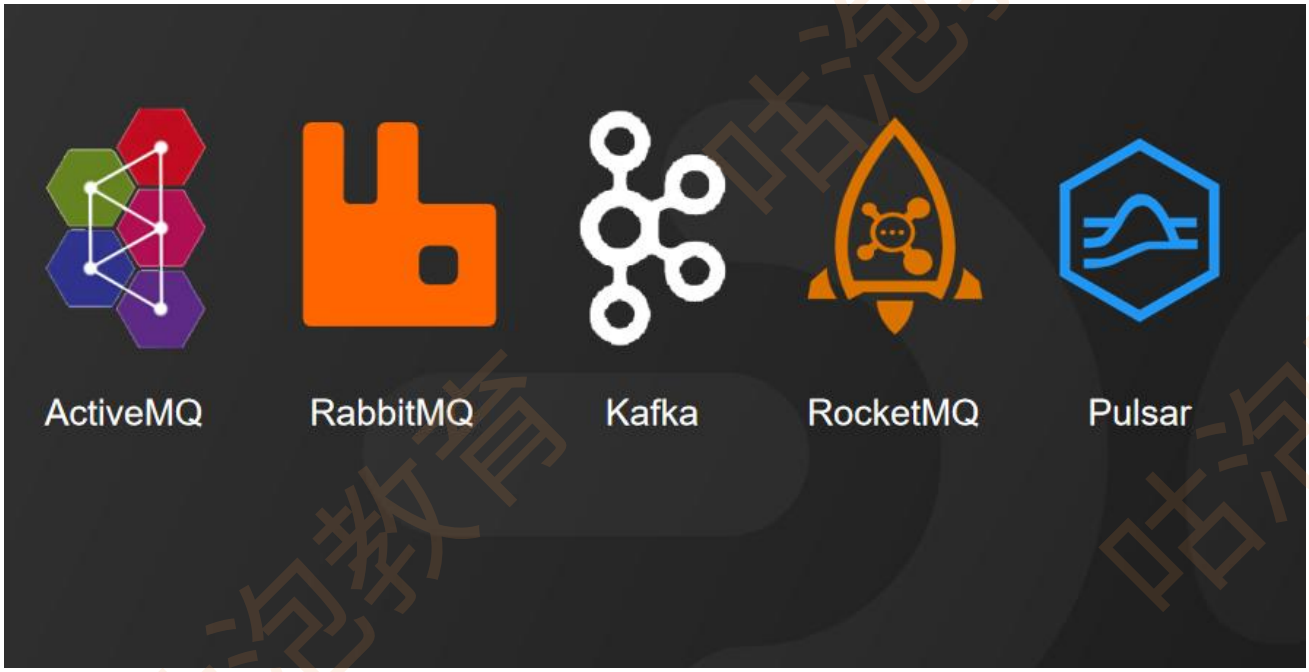


主要应用于实时性要求不严格的场景，比如用户登录发送验证码，支付成功通知等等。服务提供方（生产者）只需要将协商好的消息发送到消息队列，之后的处理就由消费者，也不需要等待消费者的返回结果。

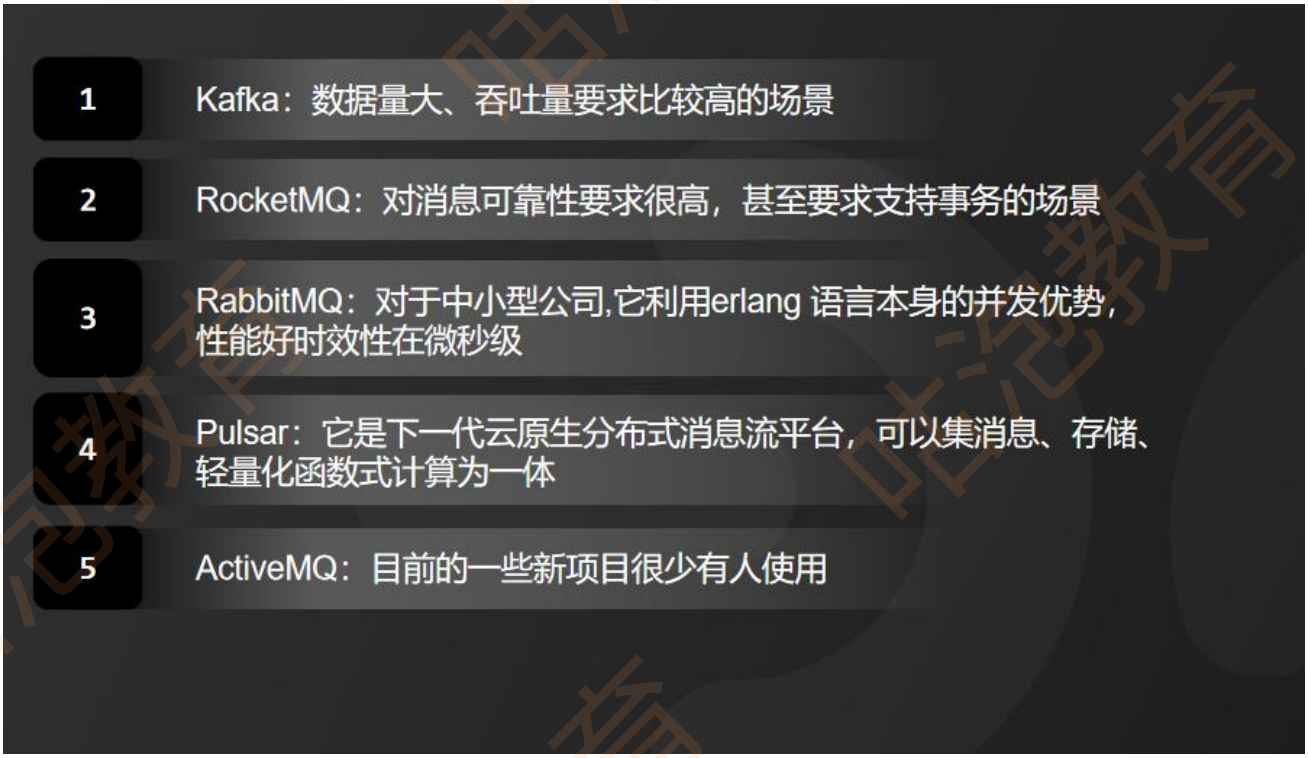
3、如何选择MQ

【导航条：如何选择】

（转场，屏幕黑底白字，正中央出现“3 如何选择”）



目前，主流的消息中间件有ActiveMQ、RabbitMQ、Kafka、RocketMQ、Pulsar。在技术选型的时候，可以根据具体的业务需求更合适的中间件。



数据量大、吞吐量要求比较高的场景一般采用Kafka；

对消息可靠性要求很高，甚至要求支持事务的场景，比如金融互联网，可以选择RocketMQ；

对于中小型公司来说， 可以选择RabbitMQ，它利用erlang 语言本身的并发优势，性能好 在微秒级。

而Pulsar近两年开始流行起来了，它是下一代云原生分布式消息流平台，可以集消息、存储、轻量化函数式计算为一体。

至于ActiveMQ，目前的一些新项目很少有人用了。

【结尾】

好了，以上就是我对MQ的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

49．谈谈你对Redis的理解

【导航条：引言】

有位工作了10年的粉丝跟我说，去面试一家中小型公司，被要求解释什么是Redis？后面，这位粉丝来问我，说这个面试官是不是内虚？至于是不是这个原因我不做评论，但是，养成总结的习惯还是有必要的。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

今天，我来给大家分享一下我对Redis的理解，

【导航条：什么是Redis】

（转场，屏幕黑底白字，正中央出现“1 什么是Redis？）

1、什么是Redis?

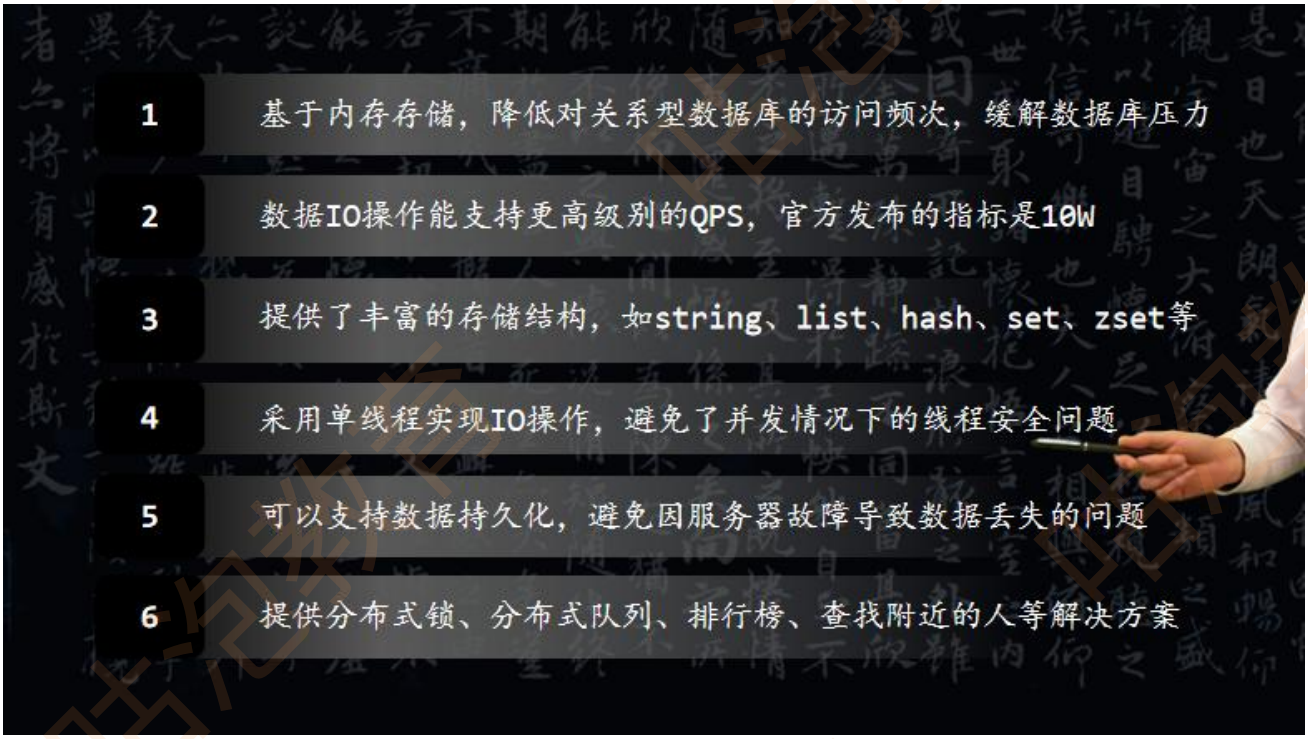
Redis是一个高性能的基于Key-Value结构存储的NoSQL开源数据库。大部分公司采用Redis来实现分布式缓存，用来提高数据查询效率。

【导航条：为什么选Redis】

（转场，屏幕黑底白字，正中央出现“2 为什么选Redis）

2、为什么选Redis

在Web应用发展的初期，系统的访问和并发并不高，交互也比较少。但随着业务的扩大，访问量的提升，使得服务器负载和关系型数据库出现瓶颈，而导致瓶颈的源头，主要体现在磁盘IO上。随着互联网的进一步发展，对系统性能有了更高的要求，Redis的出现，解决了很多问题。至于我们为什么要选择Redis，我总结为以下六个原因：



1）、基于内存存储，可以降低对关系型数据库的访问频次，从而缓解数据库压力

2）、数据IO操作能支持更高级别的QPS，官方发布的指标是10W；

3）、提供了比较多的数据存储结构，比如string、list、hash、set、zset等等。

4）、采用单线程实现IO操作，避免了并发情况下的线程安全问题。

5）、可以支持数据持久化，避免因服务器故障导致数据丢失的问题

6）、Redis还提供了更多高级功能，比如分布式锁、分布式队列、排行榜、查找附近的人等功能，为更复杂的需求提供了成熟的解决方案。

【结尾】

以上就是我对Redis的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

50.放弃FastDFS，拥抱MinIO的7大理由

【导航条：引言】

目前可用于文件存储的网络服务选择也有不少，好比阿里云OSS、七牛云、腾讯云等等，可是收费都有点小贵。为了帮公司节约成本，以前一直是使用FastDFS做为文件服务器，准确的说是图片服务器。直到我发现了MinIO，我决定放弃FastDFS。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

【导航条：什么是MinIO】

(转场，屏幕黑底白字，正中央出现“1 什么是MinIO)

1、什么是MinIO?

先给大家介绍一下什么是MinIO。

MinIO是专门为海量数据存储、人工智能、大数据分析而设计的对象存储系统。据官方介绍，单个对象最大可达5TB。非常适合储海量图片、视频、日志文件、备份数据和容器/虚拟机镜像等。MinIO主要采用Golang语言实现，整个系统都运行在操作系统的用户态空间，客户端与存储服务器之间采用HTTP/HTTPs通信协议。

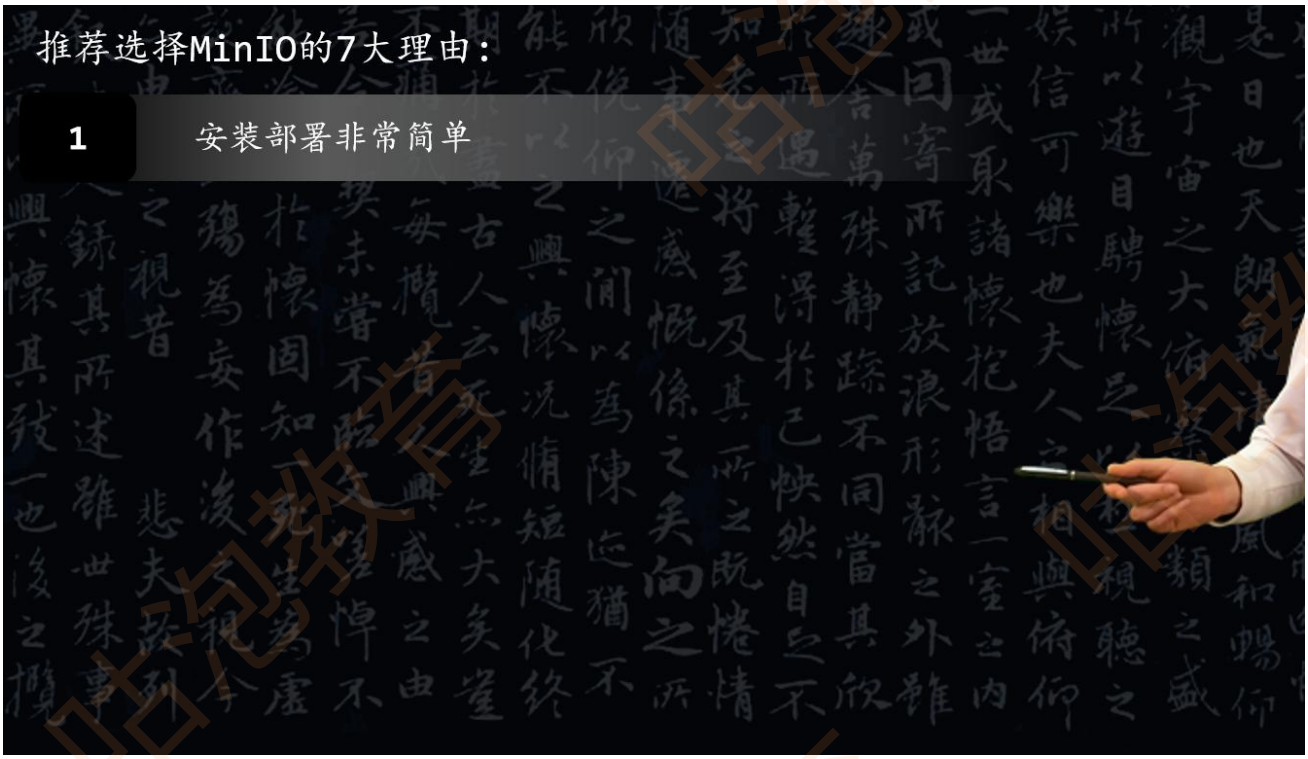
【导航条：为什么选MinIO】

（转场，屏幕黑底白字，正中央出现“2 为什么选MinIO）

2、为什么选MinIO

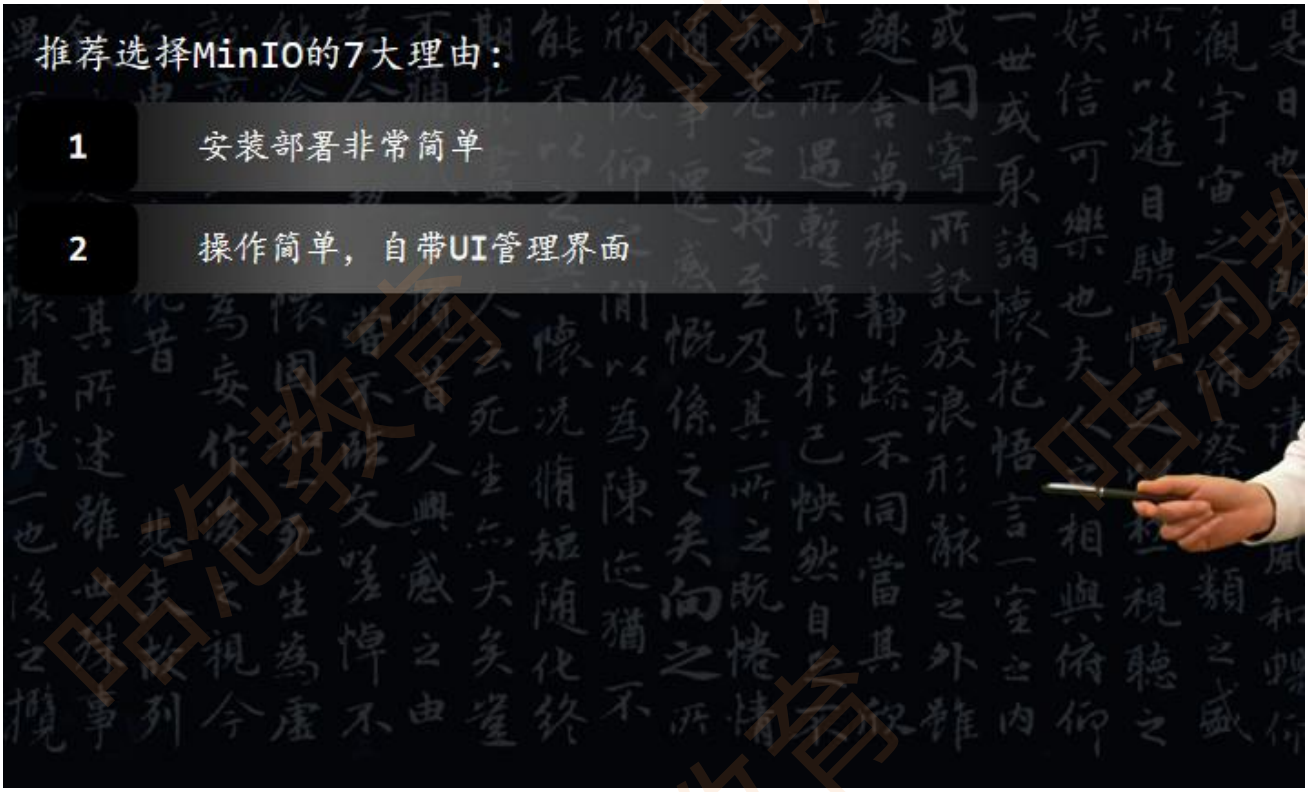
相比于其他方案来说，推荐选择MinIO的7大理由：

1）、安装部署非常简单

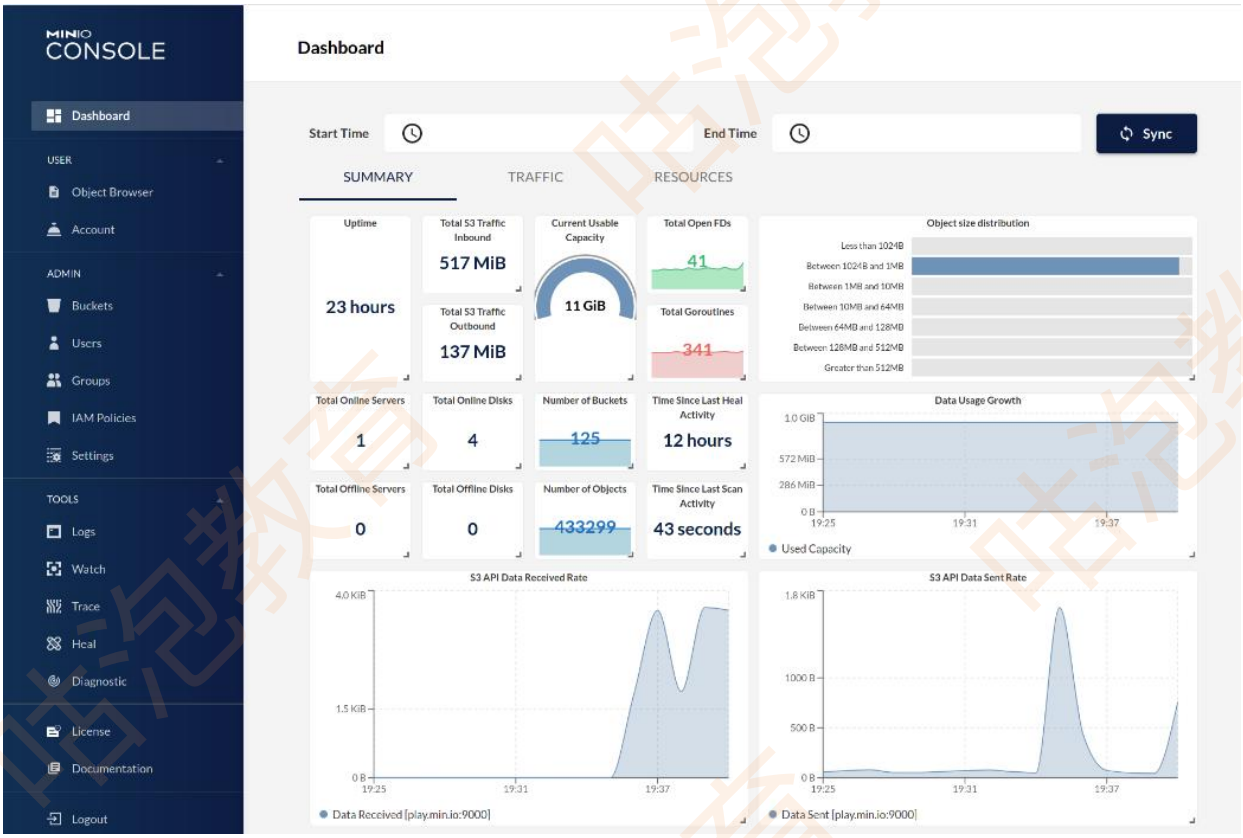


小伙伴们应该都知道，部署FastDFS的时，需要掌握Linux基础、分布式原理、C语言编译、Nginx安装、Ningx防盗链插件这些前置知识。而MinIO只需要执行这几行命令便可轻松搞定，也大大降低了学习和维护成本。

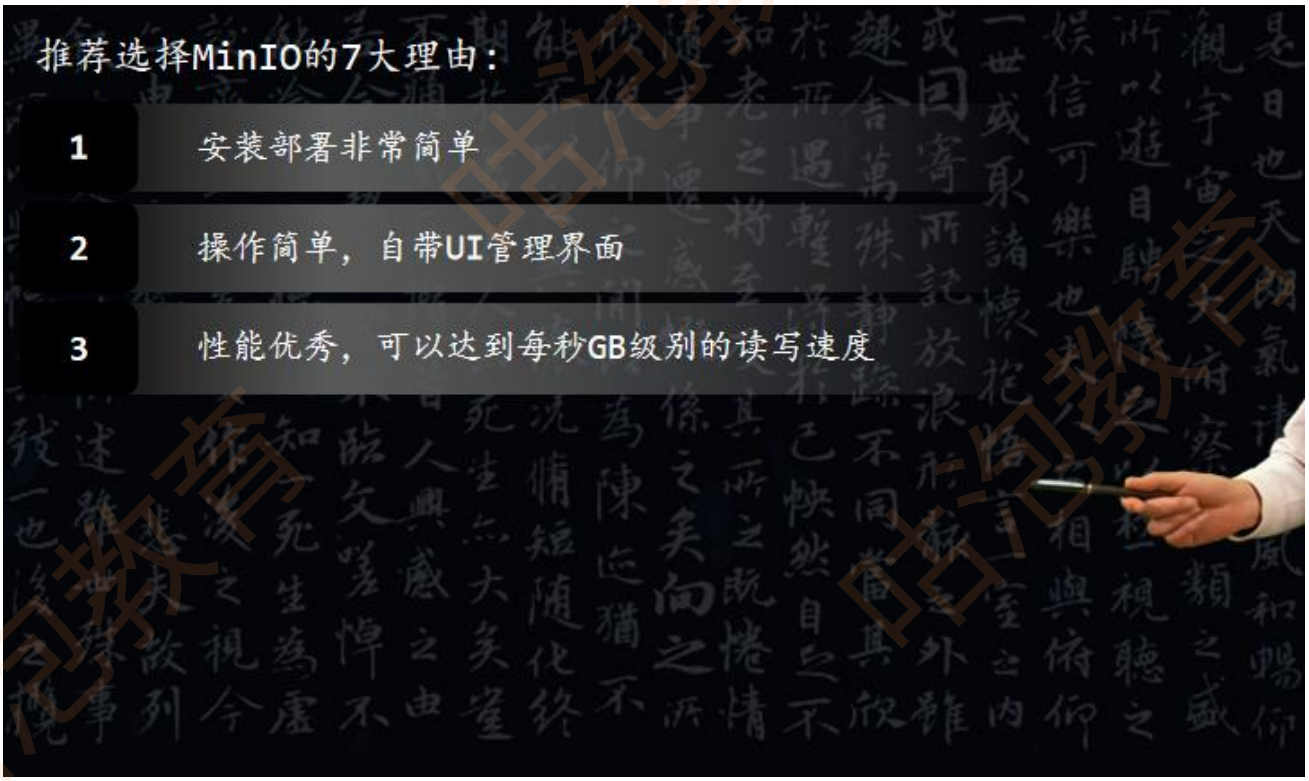
2）、操作简单，自带UI管理界面



MinIO在服务端安装后就自带了UI界面，可谓开箱即用，而FastDFS默认是不带UI界面的。

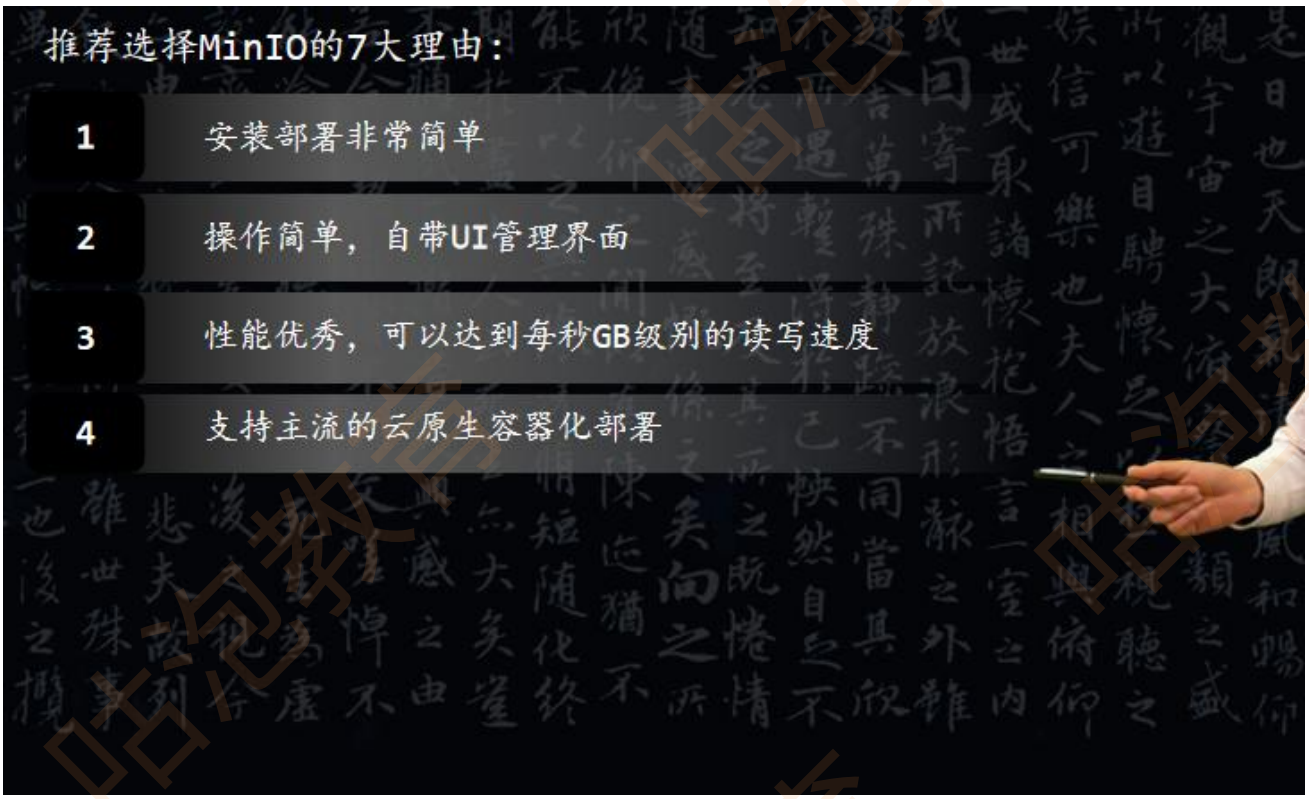


3）、性能优秀，可以达到每秒GB级别的读写速度



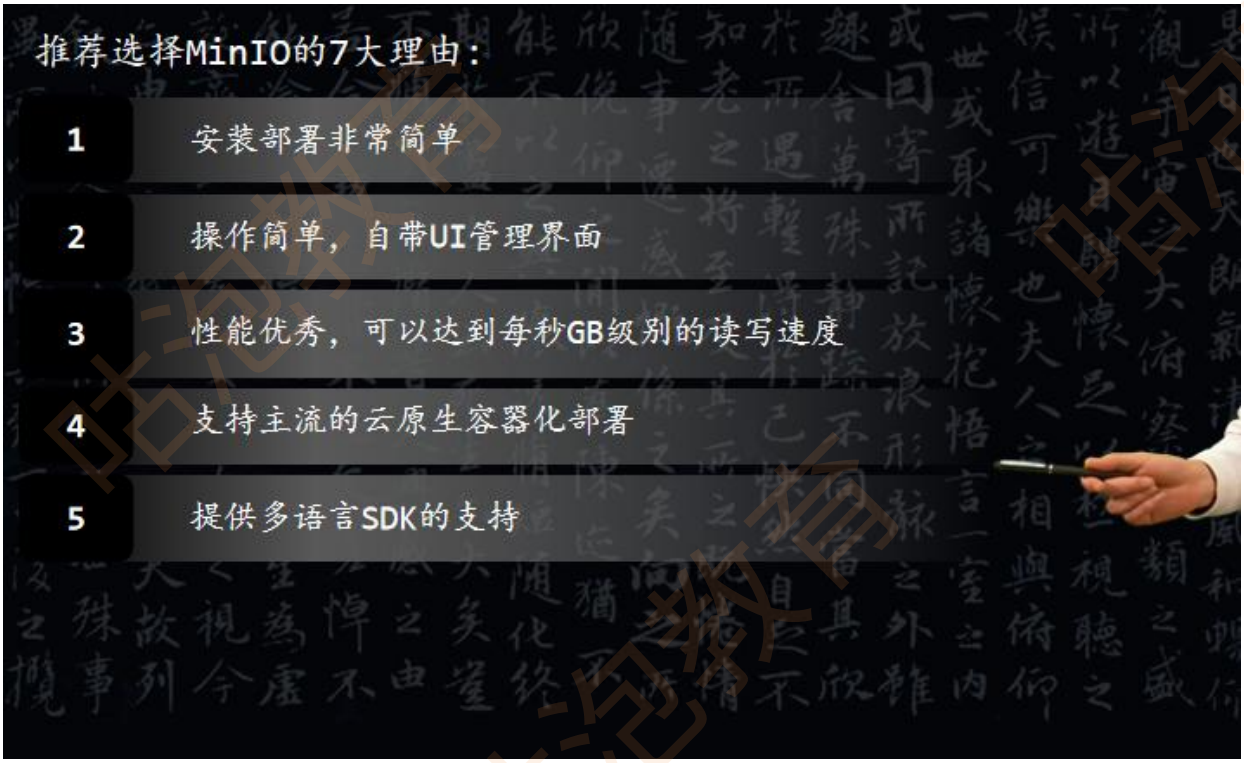
MinIO官方号称是世界上速度最快的对象存储服务器。就算部署在普通的硬件上，读对象的速度能达到183 GB/s，写对象的速度能达到171 GB/s。这是MinIO和其他产品的性能对比图

4）、支持主流的云原生容器化部署



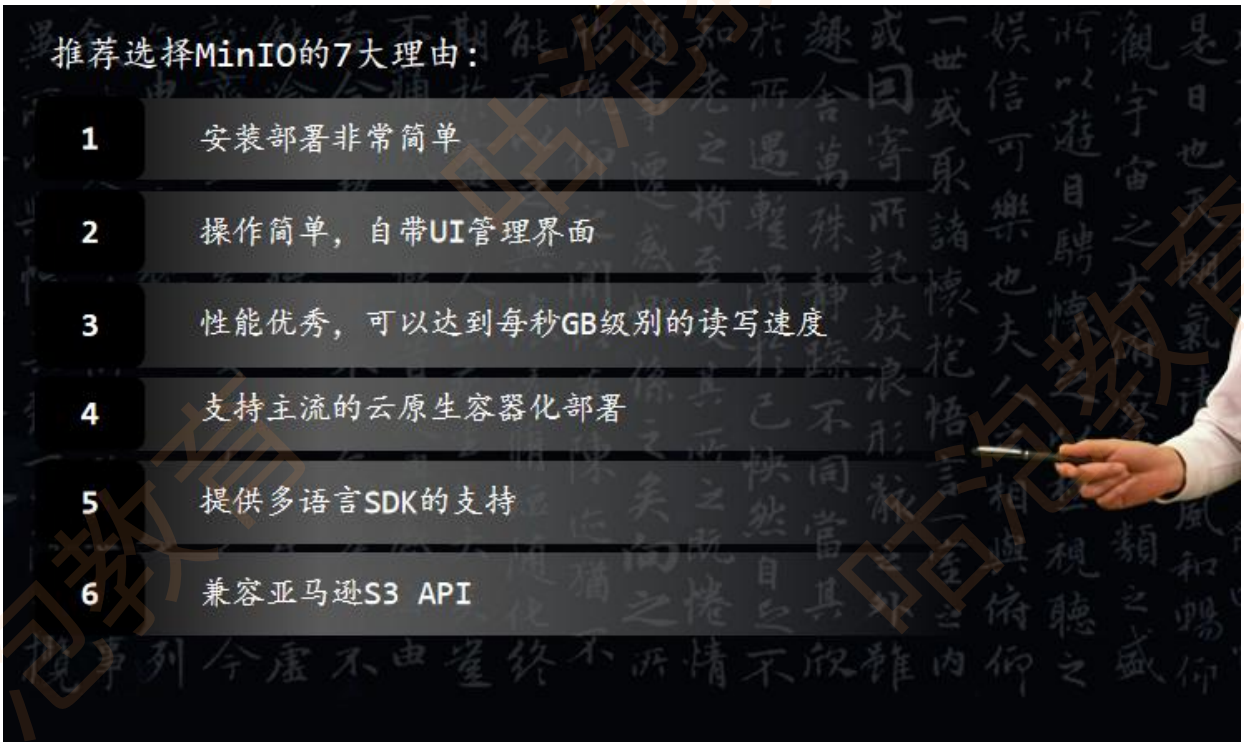
MinIO提供了与K8s、ETCD、Docker等容器化技术深度集成方案，可以说MinIO就是为云环境而生的。

5）、提供多语言SDK的支持



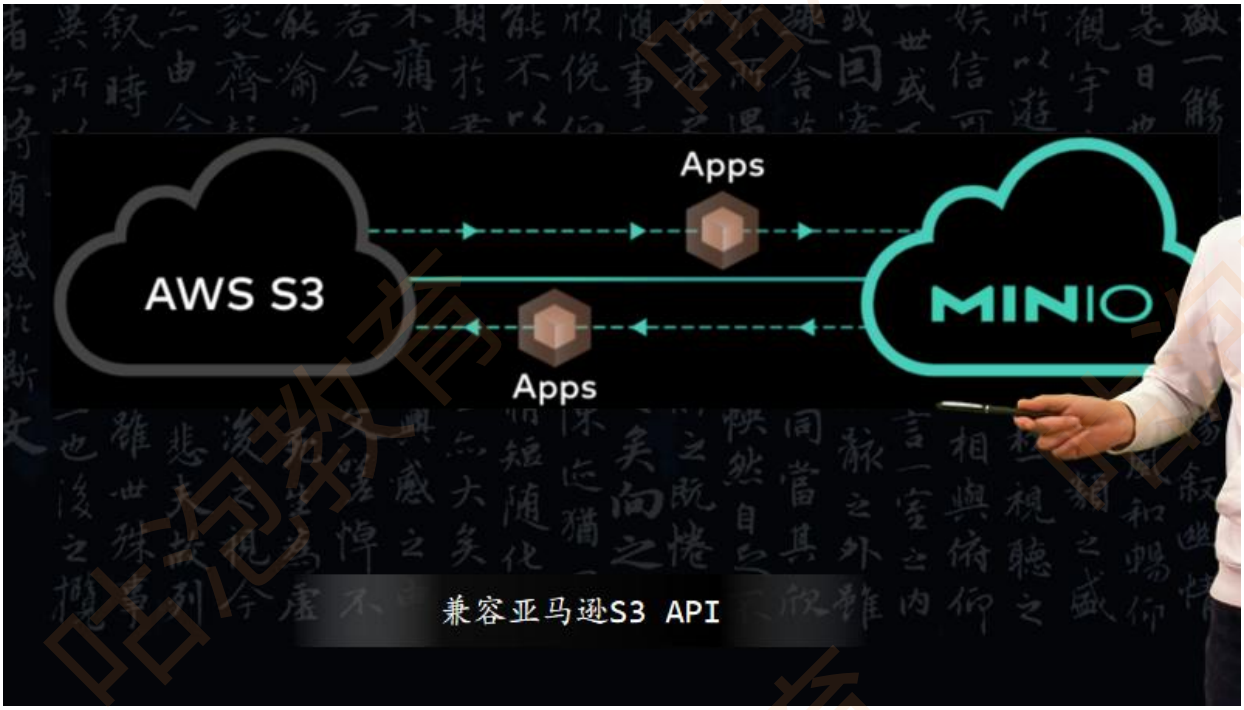
MinIO几乎提供了全部主流开发语言的SDK以及文档，比如Java、Python、Golang、JS、.NET等等。

6）、兼容亚马逊S3 API



亚马逊云的 S3 API（接口协议） 是在全球范围内达到共识的对象存储的协议，是全世界认可的对象

存储标准。而MinIO是第一个采用S3兼容协议的产品之一。



兼容S3 API有什么好处呢？相当于目前为了节约服务器成本，选择用MinIO自主开发对象存储系统，等企业壮大之后，不想再运维基础设施，可以直接将程序平移到云厂商，不需要重新开发。

7）、参考学习文档非常全面

官方提供了非常多的参考文档，而且 是全面开源的，在很多开源社区也非常活跃。

【结尾】

以上就是我对MinIO的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

51.谈谈你对Zookeeper的理解

【导航条：引言】

对一个没有接触过分布式的小伙伴来说，可能还不太清楚Zookeeper在分布式应用中的重要性，今天有位从传统行业转型到互联网的小小伙伴问我，希望跟他讲讲Zookeeper什么，能解决什么问 题？今天，我给大家详细介绍一下。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶

简介中找到。

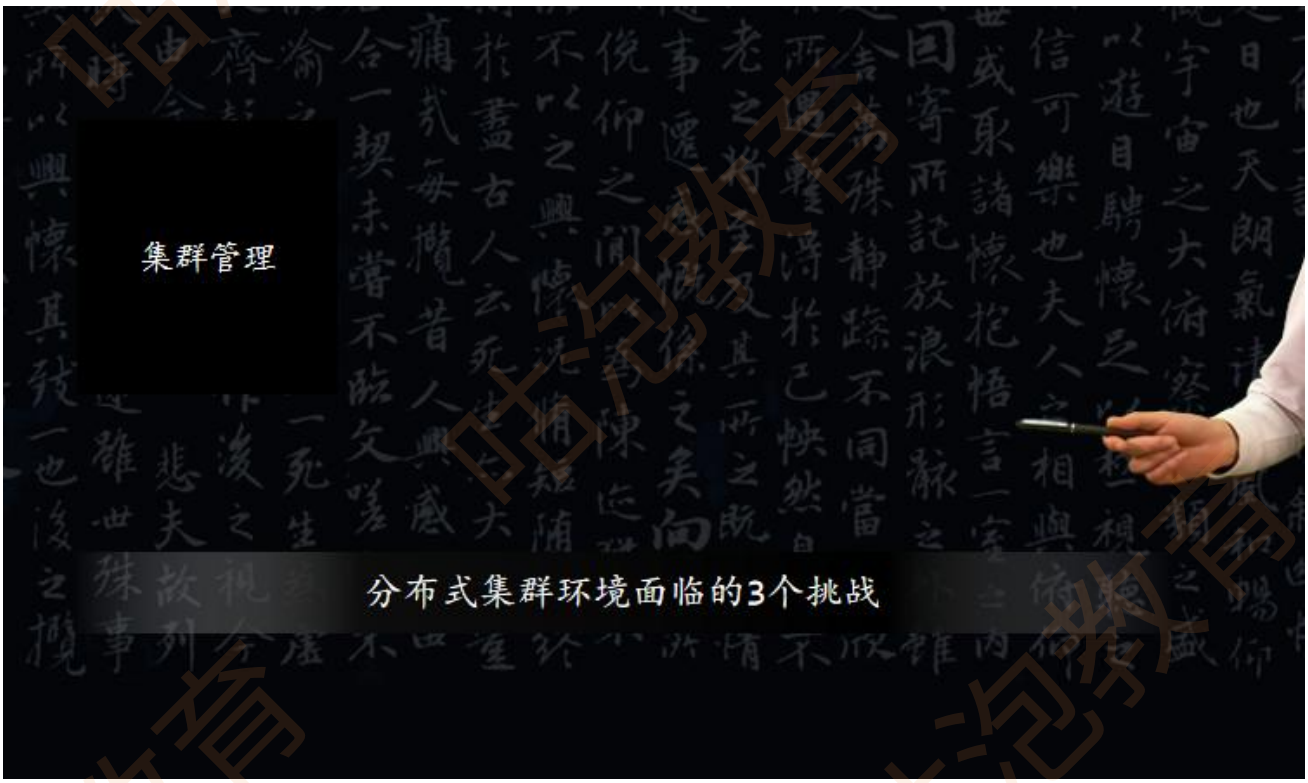
【导航条：什么是Zookeeper】

（转场，屏幕黑底白字，正中央出现“1 什么是Zookeeper）

1、什么是Zookeeper?

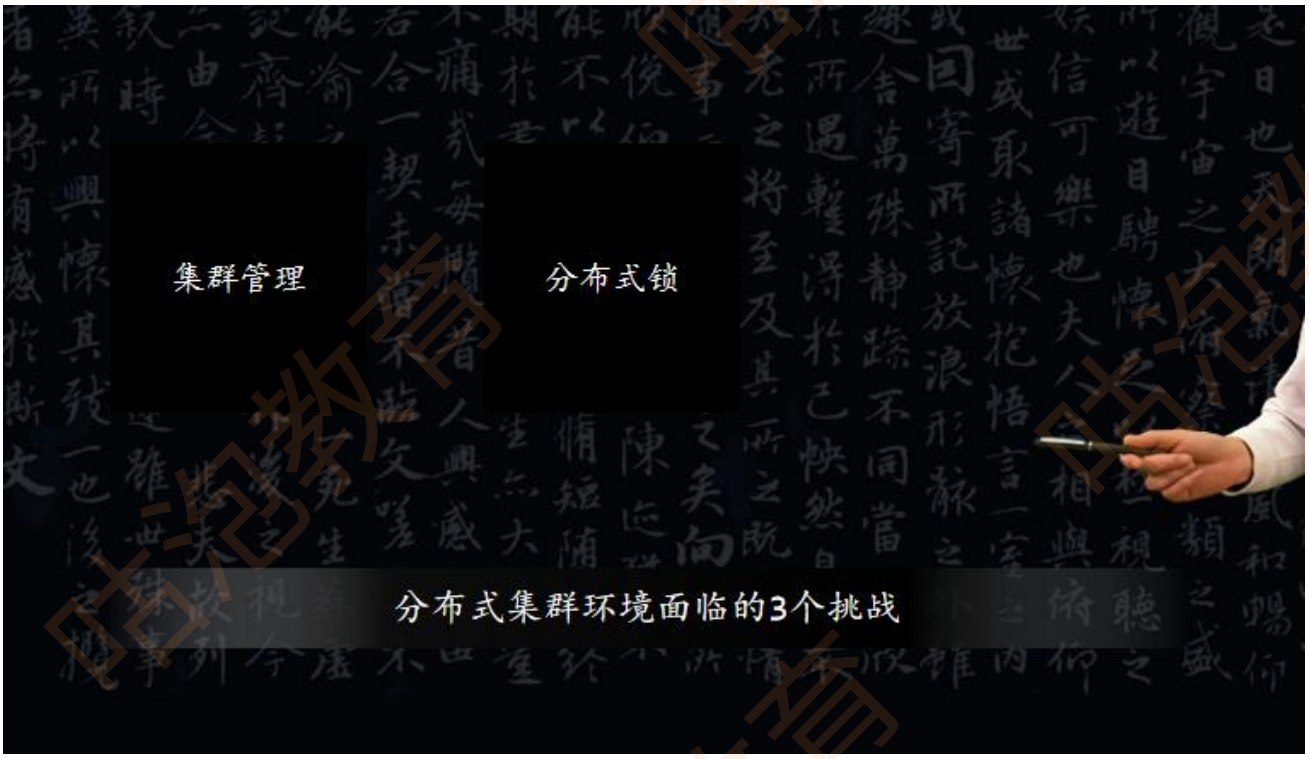
在讲什么Zookeeper之前，有必要先介绍一下分布式多个节点组成的集群环境中，所面临的3个常见的挑战。

第一种，集群管理



为了保证集群的高可用，每个节点都会创建一个数据副本。这种情况下，要保证客户端访问集群中的任意一个节点，都要是最新的数据。

第二种，分布式锁



对于分布式系统来说，如何保证共享资源的并发安全性也是一个比较大的挑战，为了达到这样一个目的，必须要使用跨进程的锁，也就是分布式锁来实现。

第三种，Master选举

在多个节点集群中，为了降低集群数据同步的复杂度，一般会将节点设置为两个角色Master和Slave，Master一般负责数据的读写等事务性操作，而Slave只提供数据的读操作。但是，如何去确定某个节点是Master还是Salve也是一项比较复杂的工作。

Zookeeper的出现，主要是为了满足分布式环境中，以上三种常见的场景需求，作为一个分布式的中间件而存在。它相当于是一个分布式开源的协调组件。简单理解，就相当于是一个裁判员的角色，专门负责协调和解决分布式系统中的各类问题。

【导航条：基本原理】

（转场，屏幕黑底白字，正中央出现“2分布式协调的基本原理）

2、分布式协调的基本原理

1、集群管理

Zookeeper提供了CP模型，来保证集群中的每个节点的数据一致性。当然，ZK本身的集群并不是强

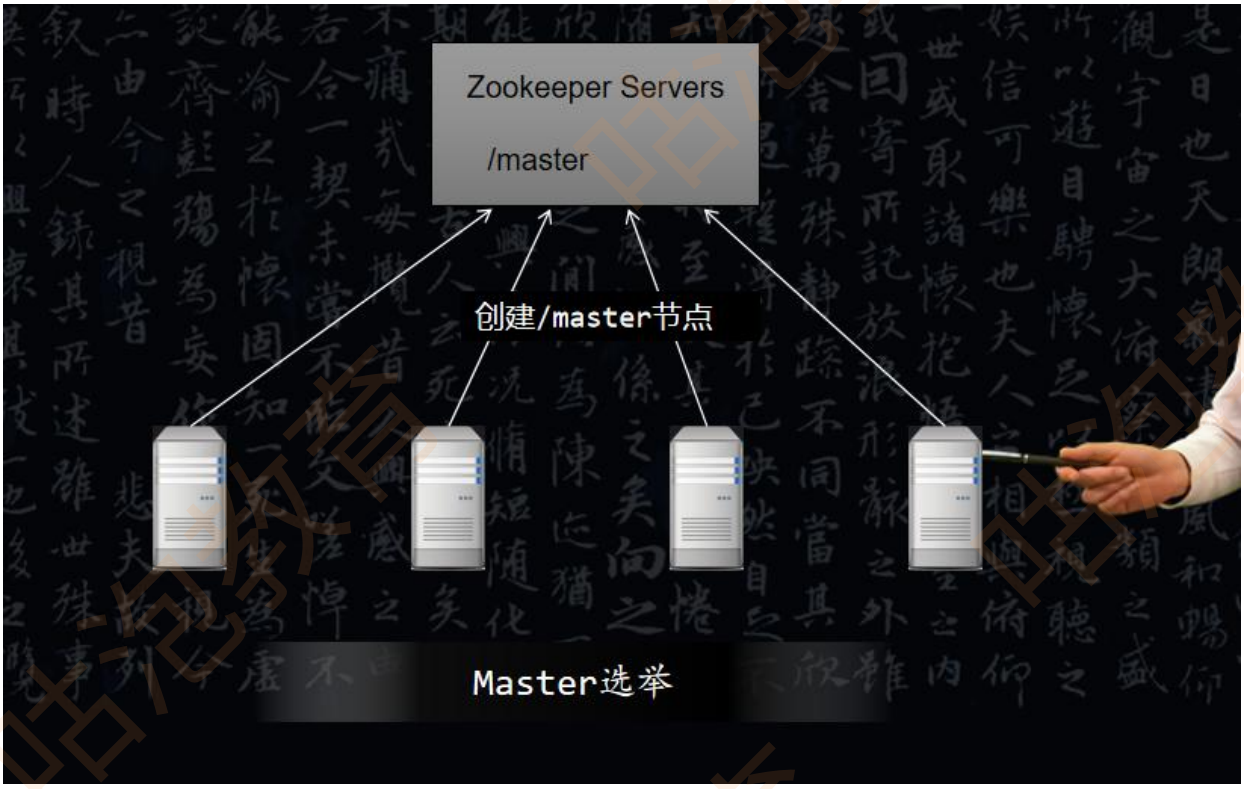
制一致性的，而是一个顺序一致性模型。如果我们需要去保证CP特性的话，我们需要调动“sync”同步方法进行同步。

2、分布式锁

Zookeeper提供了多种不同的节点类型，比如持久化节点、临时节点、有序节点和容器节点等等。对于分布式锁这个场景来说，Zookeeper可以利用有序节点这样一个特性来实现。除此之外呢，还可以利用同一级节点的唯一特性来实现分布式锁。

3、Master选举

Zookeeper可以利用持久化节点来存储和管理其他集群节点的一些信息。从而去进行Master选举的一种机制，或者还可以利用集群中的一些有序节点的特性来实现Master选举。目前，主流的Kafka、HBase、Hadoop都是通过Zookkeeper来实现集群节点的主从选举。



【导航条：总结】

（转场，屏幕黑底白字，正中央出现“3 总结）

3、总结

总的来说，Zookeeper本就是一个经典的分布式数据一致性解决方案。它主要致力于分布式应用中的一些高性能、高可用，并且就严格访问顺序控制的一些能力模型，实现分布式的协调服务。它底层的数据一致性算法是基于Paxos算法演进而来的ZAB协议实现。

【结尾】

以上就是我对Zookeeper的理解。想更深入了解Zookeeper Watch机制、Leader选举底层原理的小伙伴，可以在评论留言。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

52.执行SQL响应比较慢，你有哪些排查思路?

【导航条：引言】

如果面试问你，执行SQL响应慢，你有哪些排查思路和解决方案？这是一位去某里面试的小伙伴跟我分享的面试真题，那今天我给大家来分享一下我的思路。

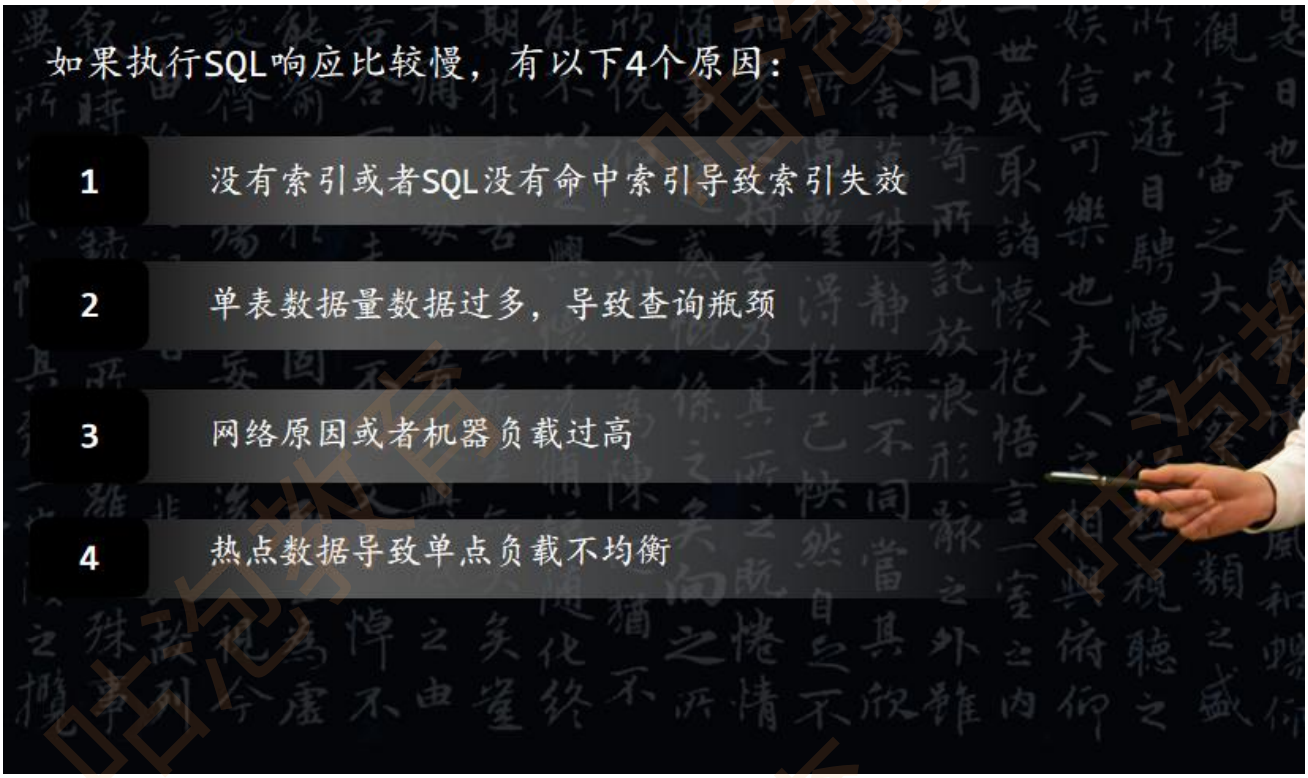
另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：排查思路】

（转场，屏幕黑底白字，正中央出现“1 排查思路）

1、排查思路

如果执行SQL响应比较慢，我觉得可能有以下4个原因：



第1个原因：没有索引或者 导致索引失效。

第2个原因：单表数据量数据过多，导致查询瓶颈

第3个原因：网络原因或者机器负载过高。

第4个原因：热点数据导致单点负载不均衡。

接下来，我针对以上几种情况，分别来聊一聊我的解决方案。

【导航条：解决方案】

（转场，屏幕黑底白字，正中央出现“2 解决方案）

2、解决方案

第1种情况：索引失效或者没有没有索引的情况

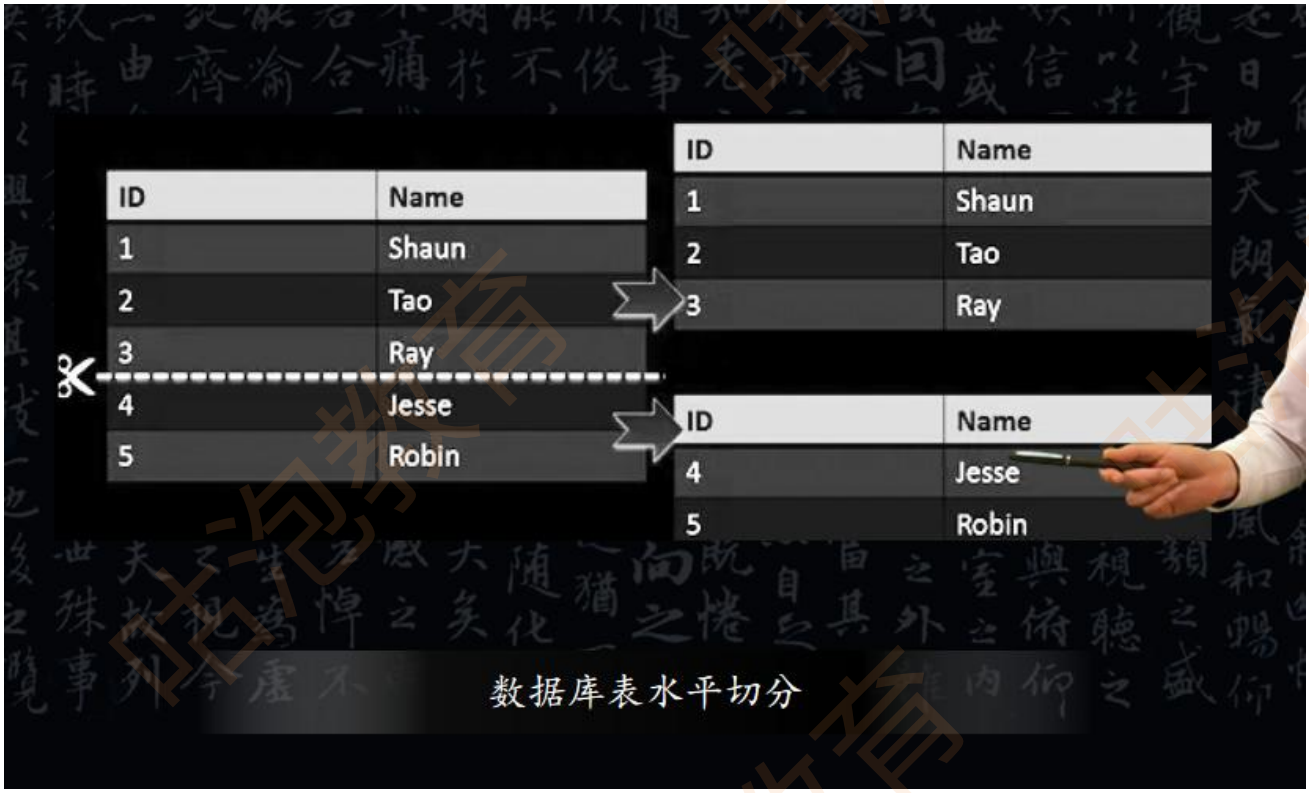
首先，可以打开MySQL的慢查询日志，收集一段时间的慢查询日志内容，然后找出耗时最长的SQL语句，对这些SQL语句进行分析。

比如可以利用执行计划explain去查看SQL是否有命中索引。如果发现慢查询的SQL没有命中索引，可以尝试去优化这些SQL语句，保证SQL走索引执行。如果SQL结构没有办法优化的话，可以考虑在表上再添加对应的索引。我们在优化SQL或者是添加索引的时候，都需要符合最左匹配原则。

第2种情况：单表数据量数据过多，导致查询瓶颈的情况。即使SQL语句走了索引，表现性能也不会特别好。这个时候我们需要考虑对表进行切分。表切分规则一般分为两种，一种是水平切分，一种是垂直切分。

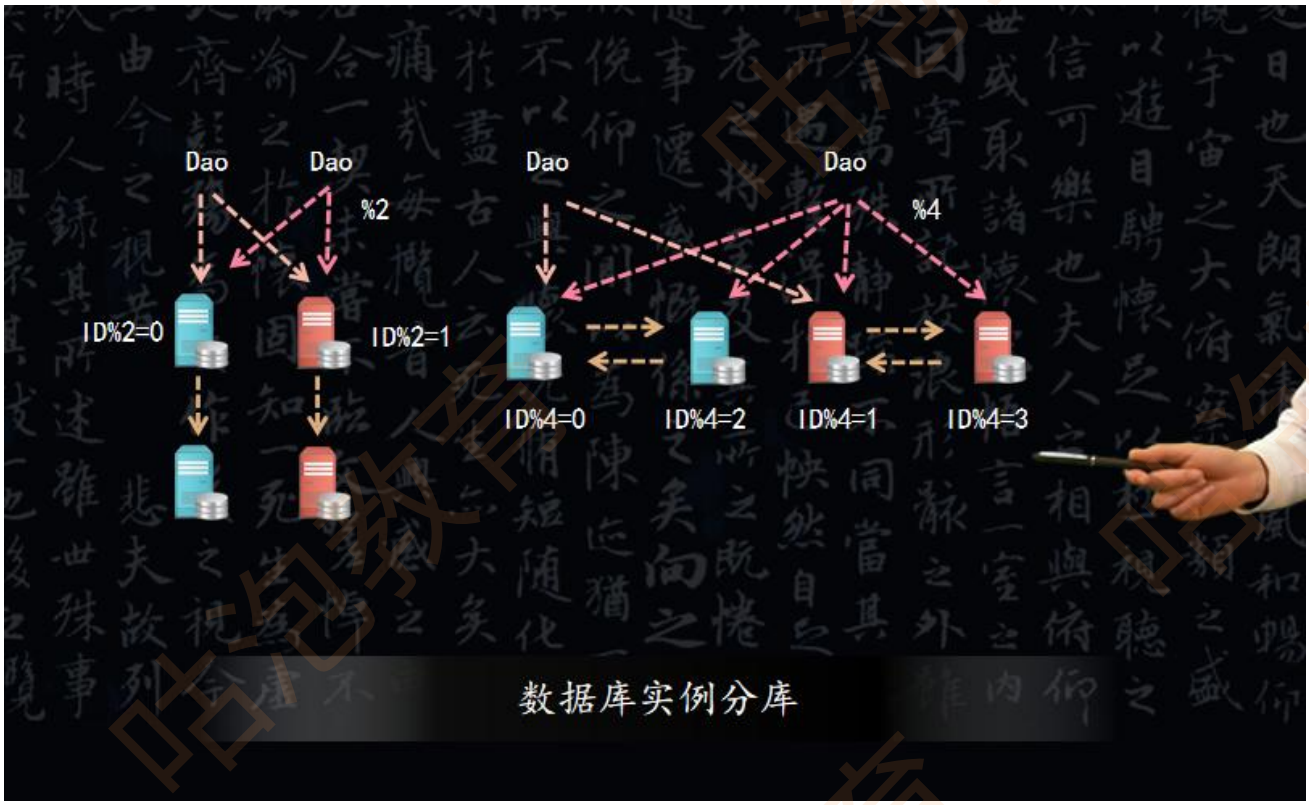
水平切分的意思是把一张数据行数达到千万级别的大表，按照业务主键切分为多张小表，这些小表可能达

到100张甚至1000张。



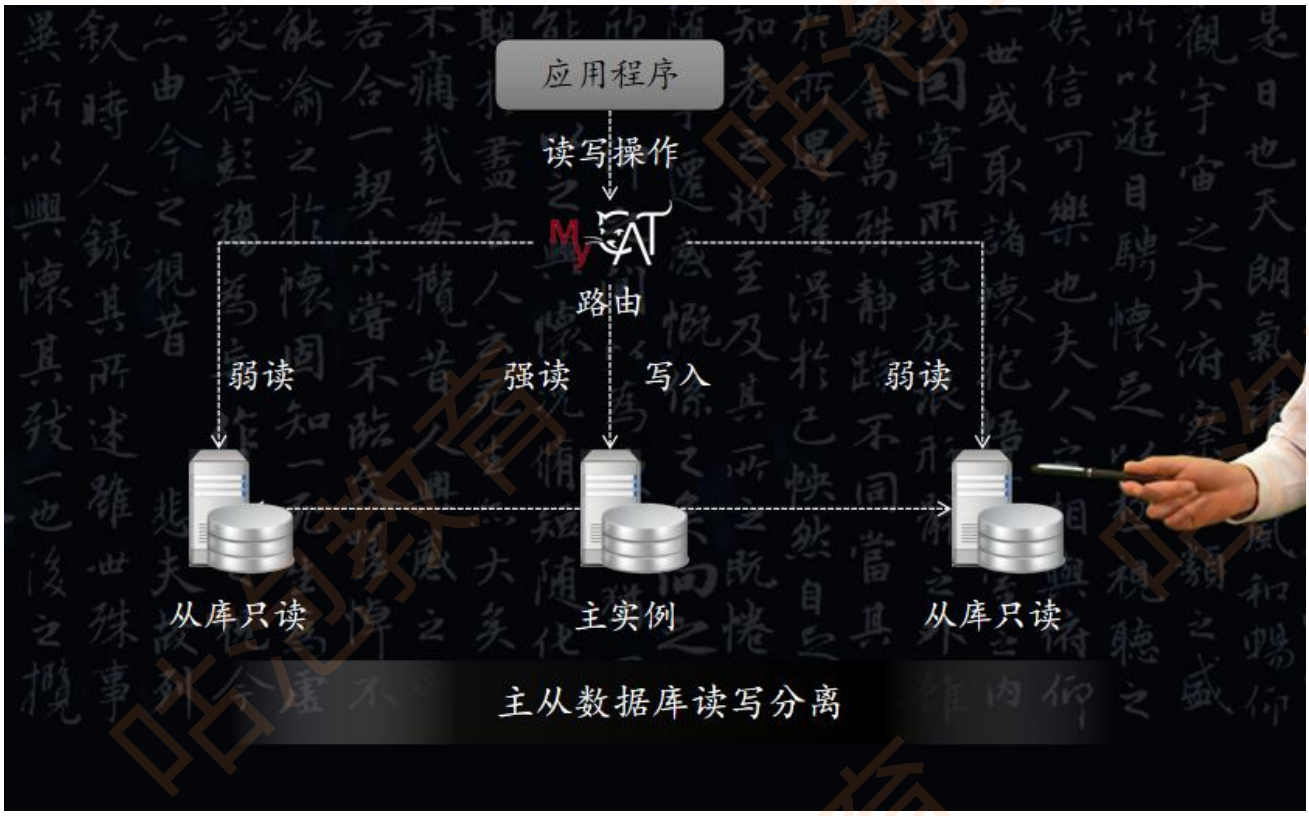
那垂直切分的意思是，将一张单表中的多个列，按照业务逻辑把关联性比较大的列放到同一张表中去。

除了这种分表之外，我们还可以分库，



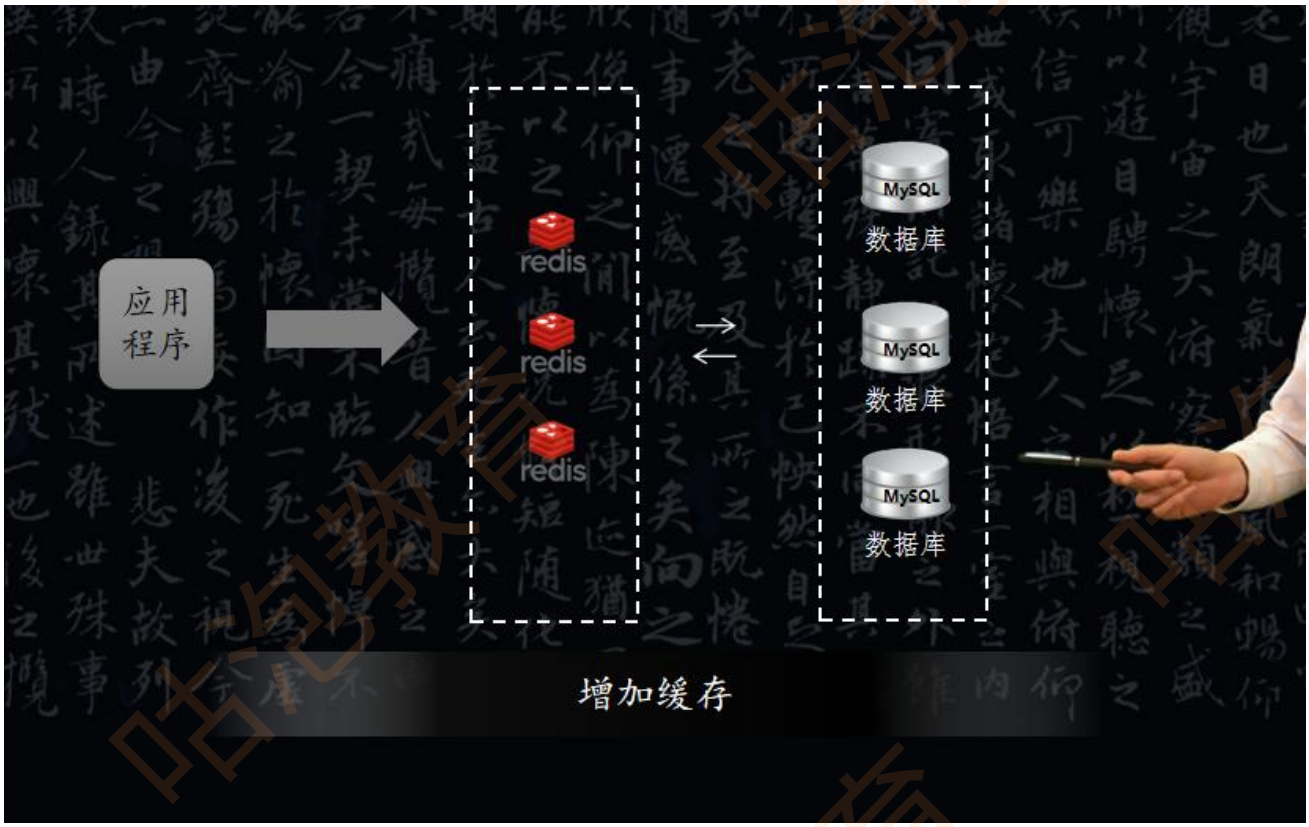
比如我们已经拆分完1000表，然后，把后缀为0-100的表放到同一个数据库实例中，然后，100-200的表放到另一个数据库实例中，依此类推把1000表存放到10个数据库实例中。这样的话，我们就可以根据业务主键把请求路由到不同数据库实例，从而让每一个数据库实例承担的流量比较小，达到提高数据库性能的目的。

第3种情况：网络原因或者机器负载过高的情况，我们可以进行读写分离.



比如MySQL支持一主多从的分布式部署，我们可以将主库只用来处理写数据的操作，而多个从库只用来处理读操作。在流量比较大的场景中，可以增加从库来提高数据库的负载能力，从而提升数据库的总体性能。

第4种情况：热点数据导致单点负载不均衡的情况。



这种情况下，除了对数据库本身的调整以外，还可以增加缓存。将查询比较频繁的热点数据预存到缓存当中，比如Redis、MongoDB、ES等，以此来缓解数据的压力，从而提高数据库的响应速度。

【结尾】

以上就是我对执行SQL响应慢的排查思路和解决方案的理解，希望能够帮助到大家。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

53.B树和B+树的区别什么?

【导航条：引言】

联合索引今天，我给大家介绍一种面试中经常被问到数据结构树。大家可能也经常会听到二叉树、二叉查找树、AVL平衡二叉树、B树、 等等，那今天我给大家一次性讲清楚。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：树的演变】

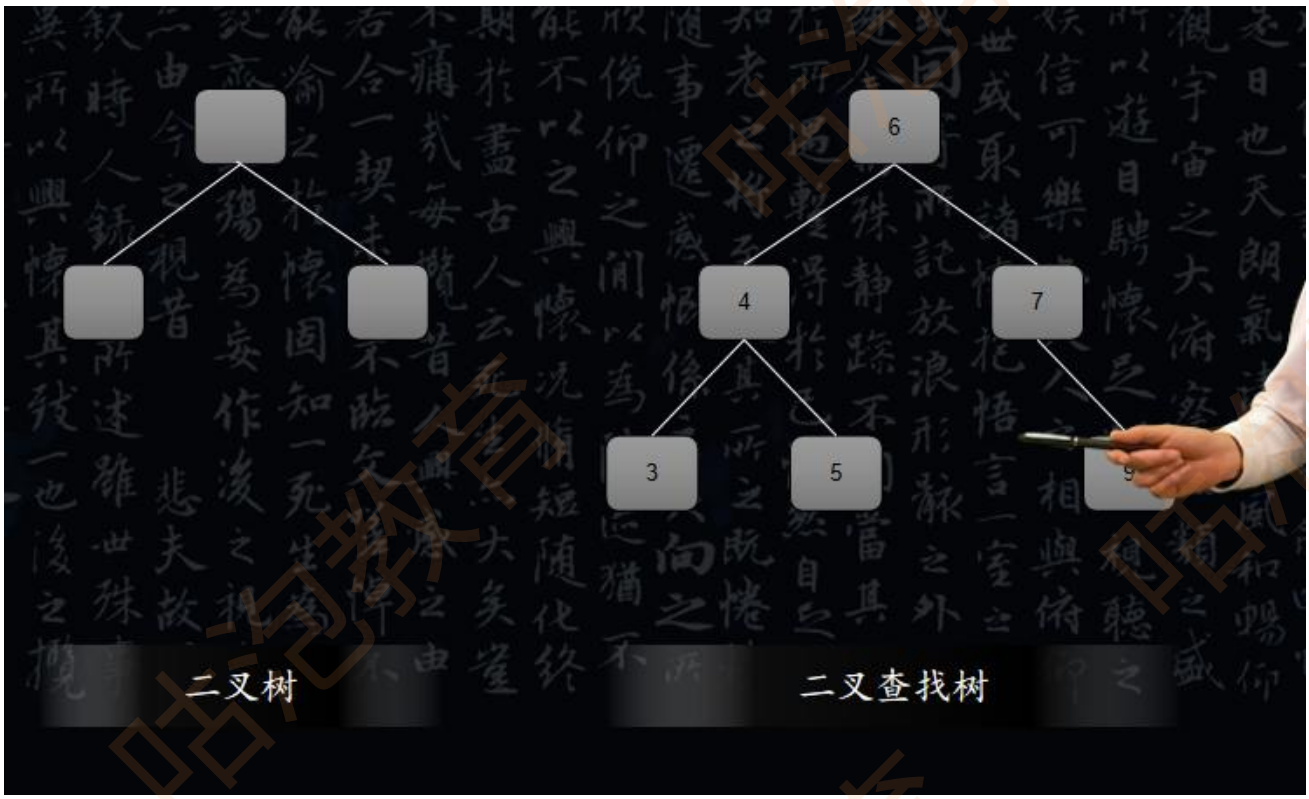
（转场，屏幕黑底白字，正中央出现“1 树形结构的演变历史）

1、树形结构的演变历史

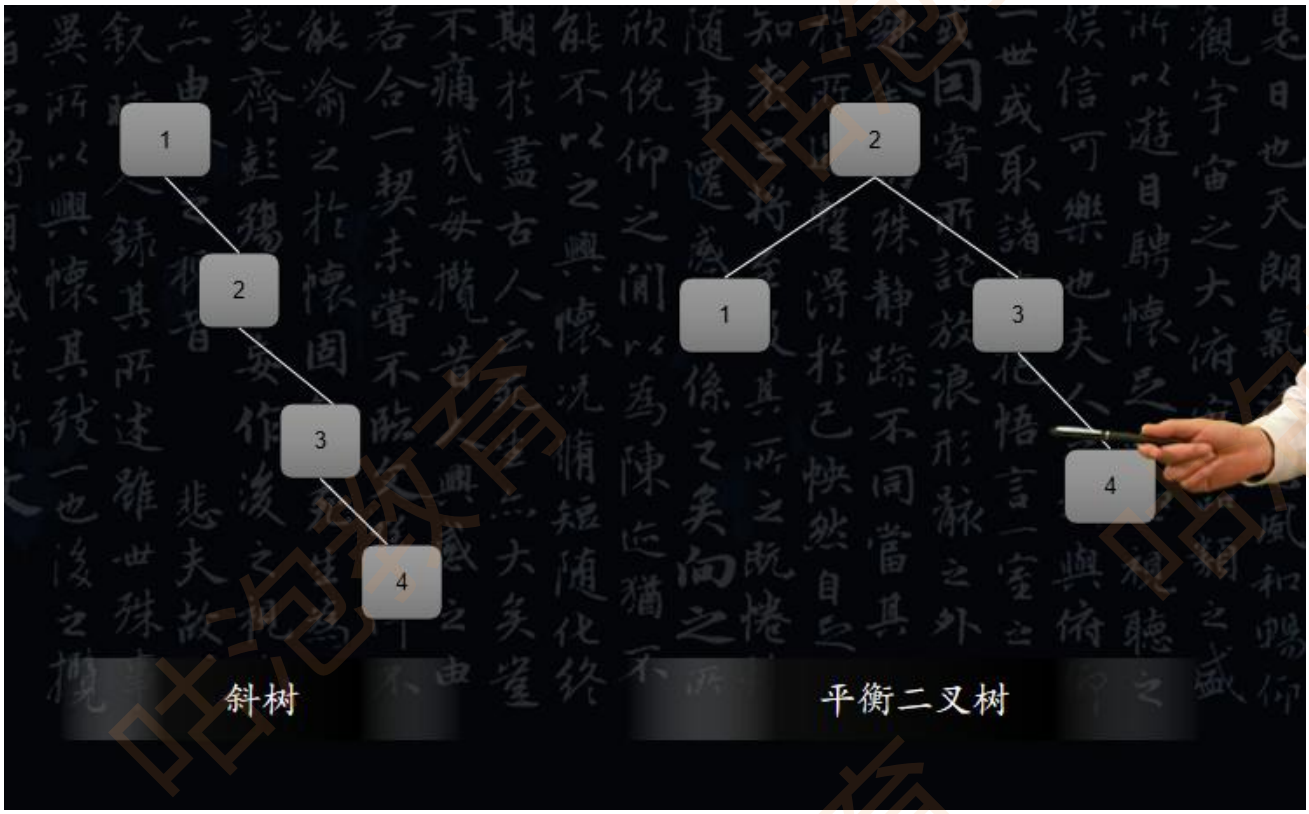
树是一种数据结构，它的结构形状如同一棵树木，但是是倒立的状态。

树的分叉就相当于树形数据结构中的节点，树上的节点可以从树根无限延伸。如图所示：

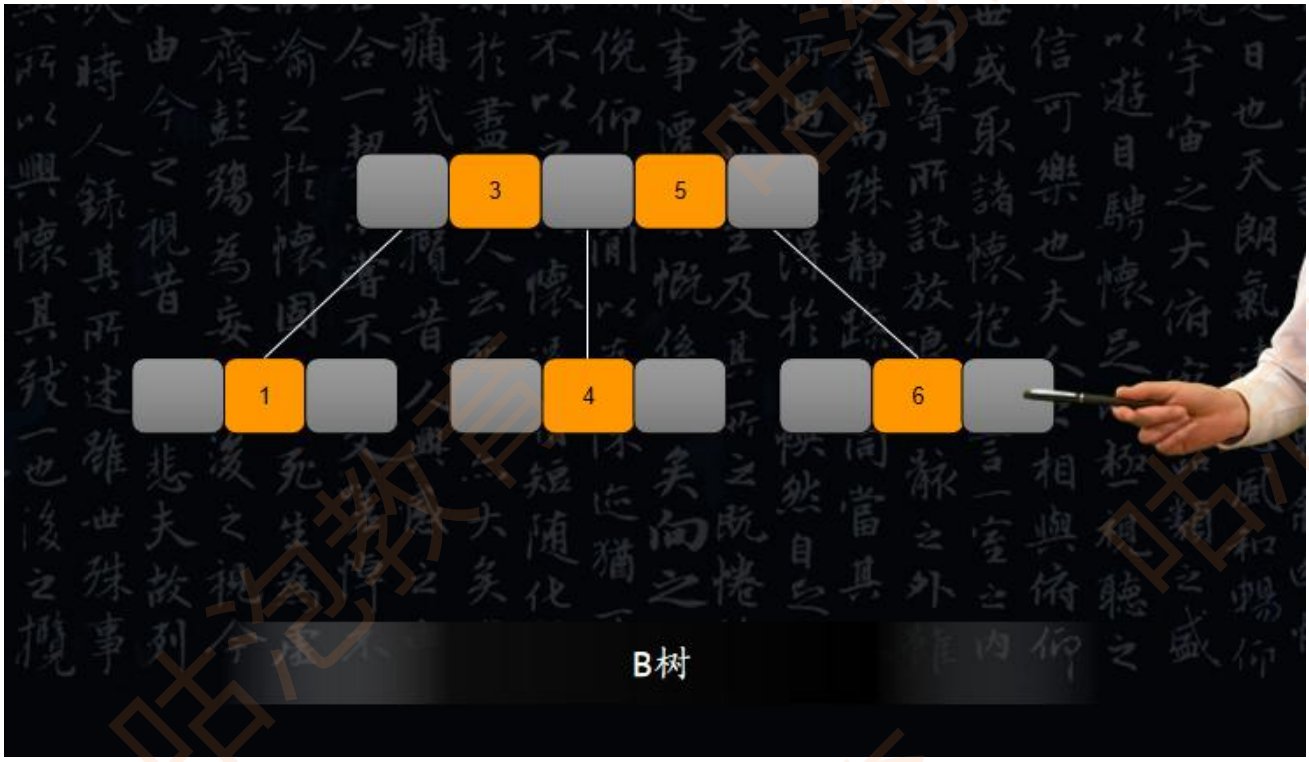
但是，这种无限延伸的树在实际开发中一般用于可无限扩展的树形功能菜单。这样的无限级、无限宽扩展的结构查询性能会比较低。因此，为了提高树形结构的查询性能，于是就出现了二叉树，来看这么一张图：



所谓二叉树是指每个节点最多支持两个分叉，相比于单向链表来说它多了一个分支。而二叉查找树，是在二叉树的基础上增加一个规则。它的规则是左子树的所有子节点都要小于它的根节点，而右侧子节点要大于它的根节点。



我们再来看这个图，二叉查找树有可能会出现斜树问题，从而导致时间复杂度会增加。因此，又引入了一种叫做平衡二叉树的机制。它具有二叉查找树的所有特点，同时还增加了一个规则。它的规则是左右两个子树的高度差的绝对值不能超过1。 为了达到这样一个平衡，所以它会引入一个左旋和右旋的机制，来实现树的平衡。



我们再来看这张图，这是B树。它是一种多路平衡查找树。它既满足平衡二叉树的规则，又可以有多个子路。子路的数量呢取决于它的关键字的数量。如图所示，根节点中有两个关键字3和5，那么它能够拥有的子路数量等于关键字的数量加1。因此，从这个特征来看，在存储同样数量的情况下，平衡二叉树的高度一定要大于B树的高度。

【导航条：B树和B+树的区别】

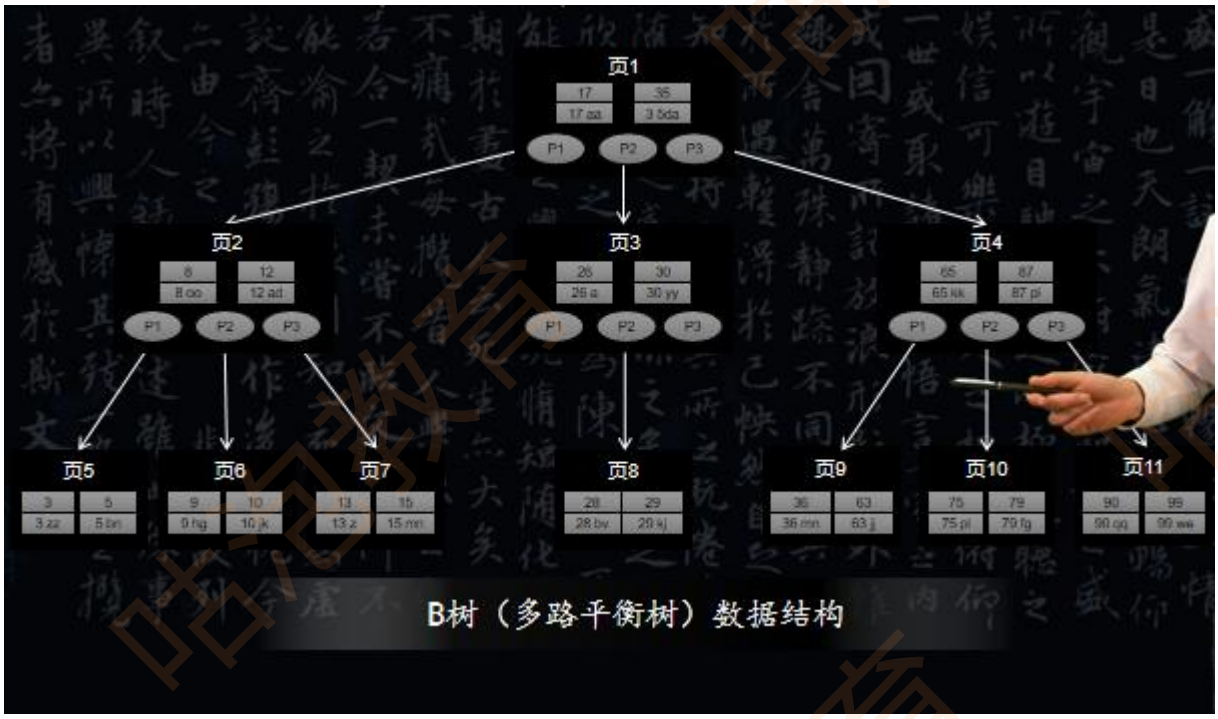
（转场，屏幕黑底白字，正中央出现“1 B树和B+树的区别）

2、B树和B+树的区别

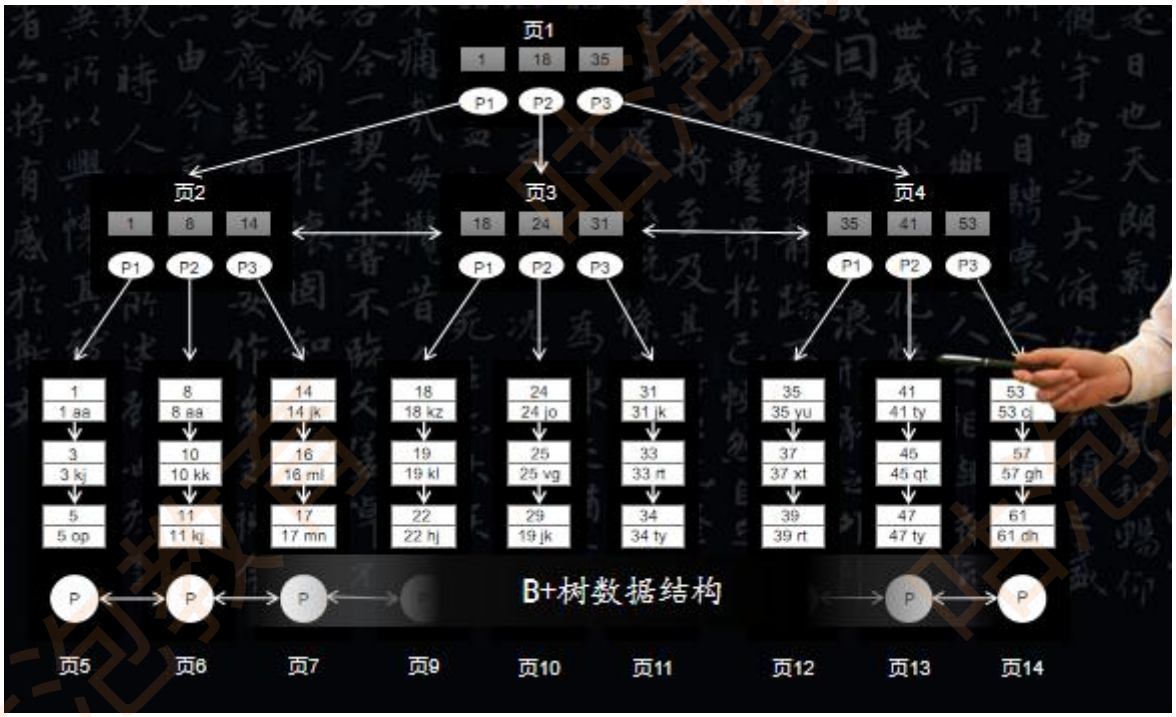
B+树呢，其实是在B树的基础上做了增强，和B树有两个最大的区别：

第1个：B树的数据存储在每个节点上，而B+树中的数据只存储在叶子节点上，并且通过链表的方式将所有叶子节点全部串联起来

第2个：B+树的子树数量等于它的关键字的数量，而B树是关键字数量加1。我们来看这个图：



这个是B树的存储结构。从B树的结构上可以看到，每个节点都会存储数据。我们再来看这个图：



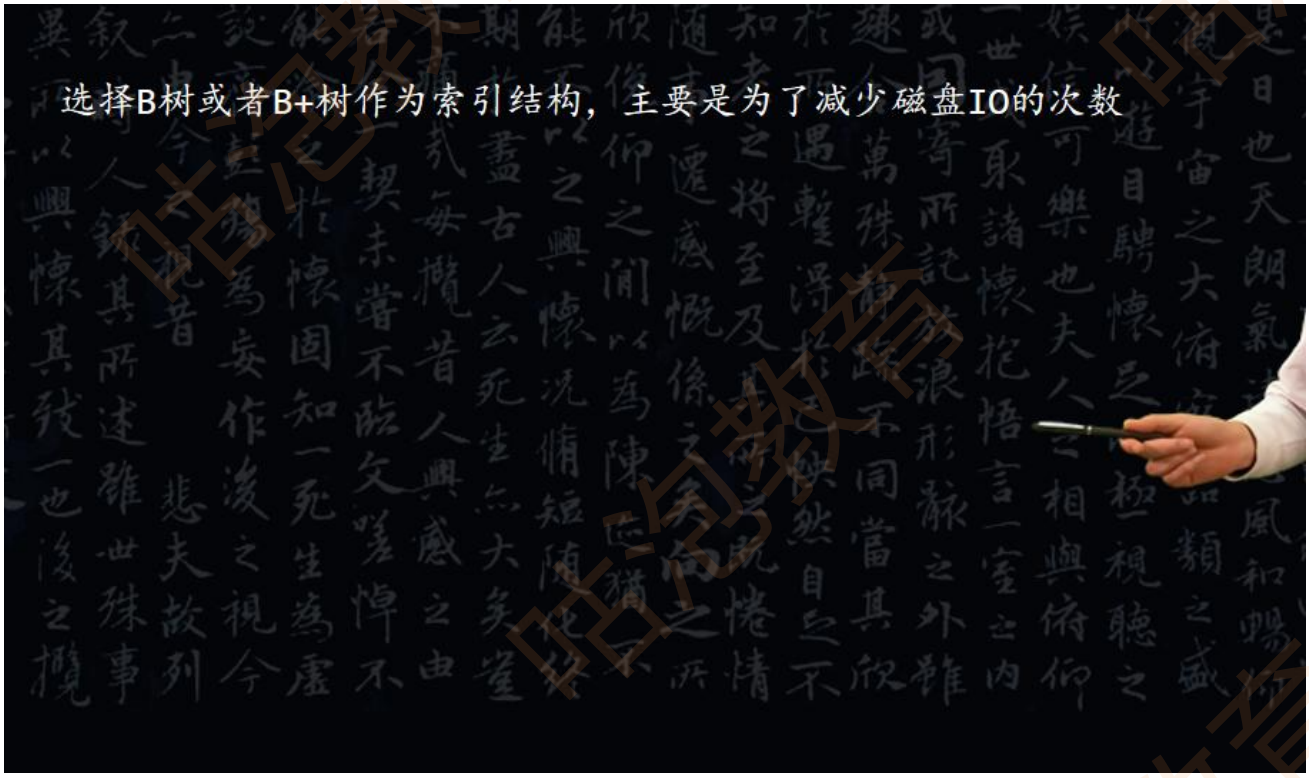
这是一个B+树的结构。B+树的数据是存储在叶子节点上的，并且呢，叶子节点的数据是用双向链表来关联。

【导航条：选择B树和B+树的理由】

（转场，屏幕黑底白字，正中央出现“3 选择B树和B+树的理由）

3、选择B树和B+树的理由

那为什么要用B树或者B+树来做索引结构呢?



因为AVL树的高度要比B树或者B+树的高度更高，而高度就意味着磁盘IO的数量，所以，为了减少磁盘IO的次数，文件系统或者数据库才会使用B树，或者B+树来做索引结构。

在比较经典的程序应用中，MongoDB使用的是B树，MongoDB中所有的节点都有Data域，只要找到指定索引就可以进行访问，无疑单次查询会更快。

而MySQL作为一个关系型数据库，数据的关联性是非常强的，区间访问是常见的一种情况，B+树由于数据全部存储在叶子节点，并且通过指针串在一起，这样就很容易的进行区间遍历甚至全部遍历。

【结尾】

以上就是我对B树和B+树的理解。程序的本质就是数据结构加算法。数据结构在实际开发中非常常见，比如数组、链表、双向链表、红黑树、跳跃表、B树、B+树、队列等。所以，数据结构是编程最重要的基本功之一，很多大厂面试也经常会问到。同时，基本功也是决定大家在技术路上能够达到的高度的重要因素。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

54.谈谈你对NoSQL的理解

【导航条：引言】

有粉丝给我留言说，希望我更新一期关于NoSQ的视频，那今天，咱们就来聊一聊我对NoSQL的理解。如果你也有想要我分享的面试题，请在评论区告诉我，后续安排。

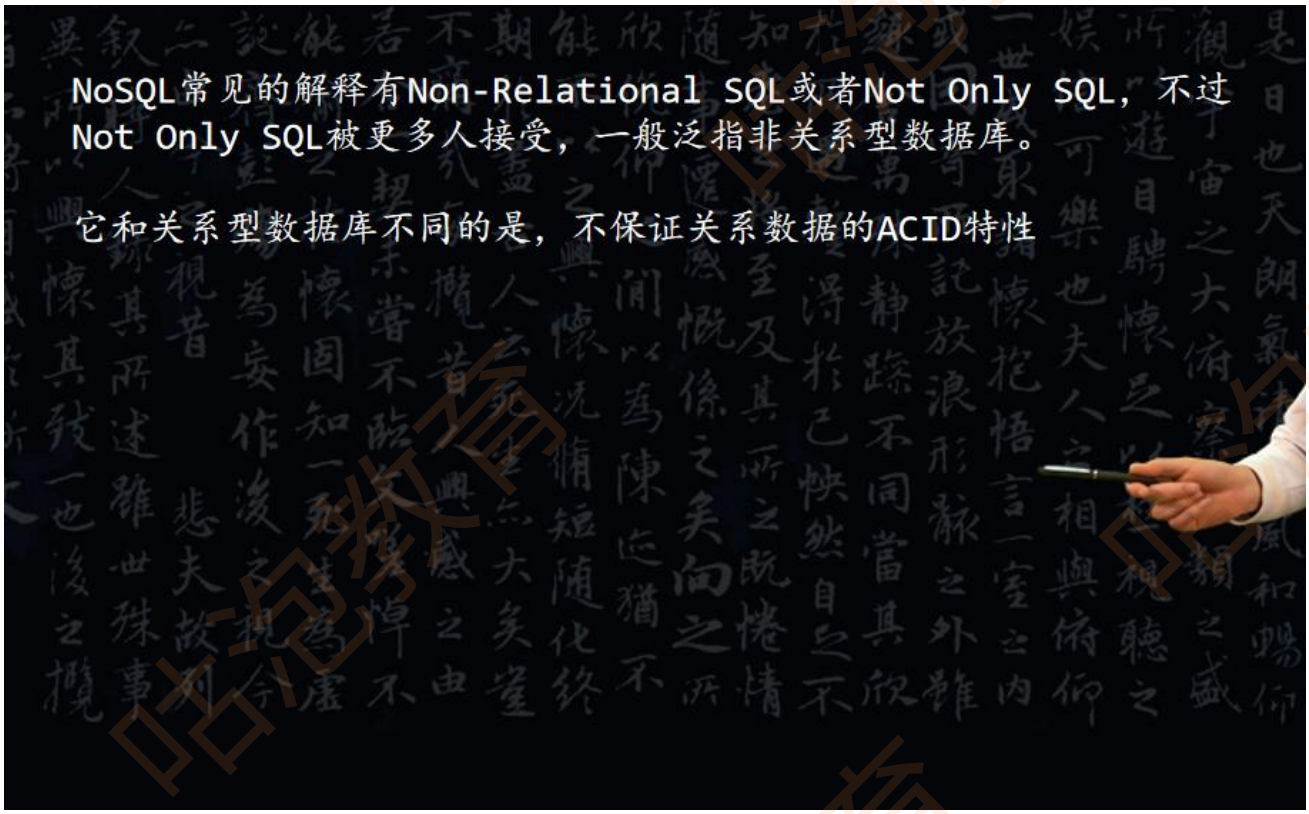
另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：什么是NoSQL】

（转场，屏幕黑底白字，正中央出现“1 什么是NoSQL）

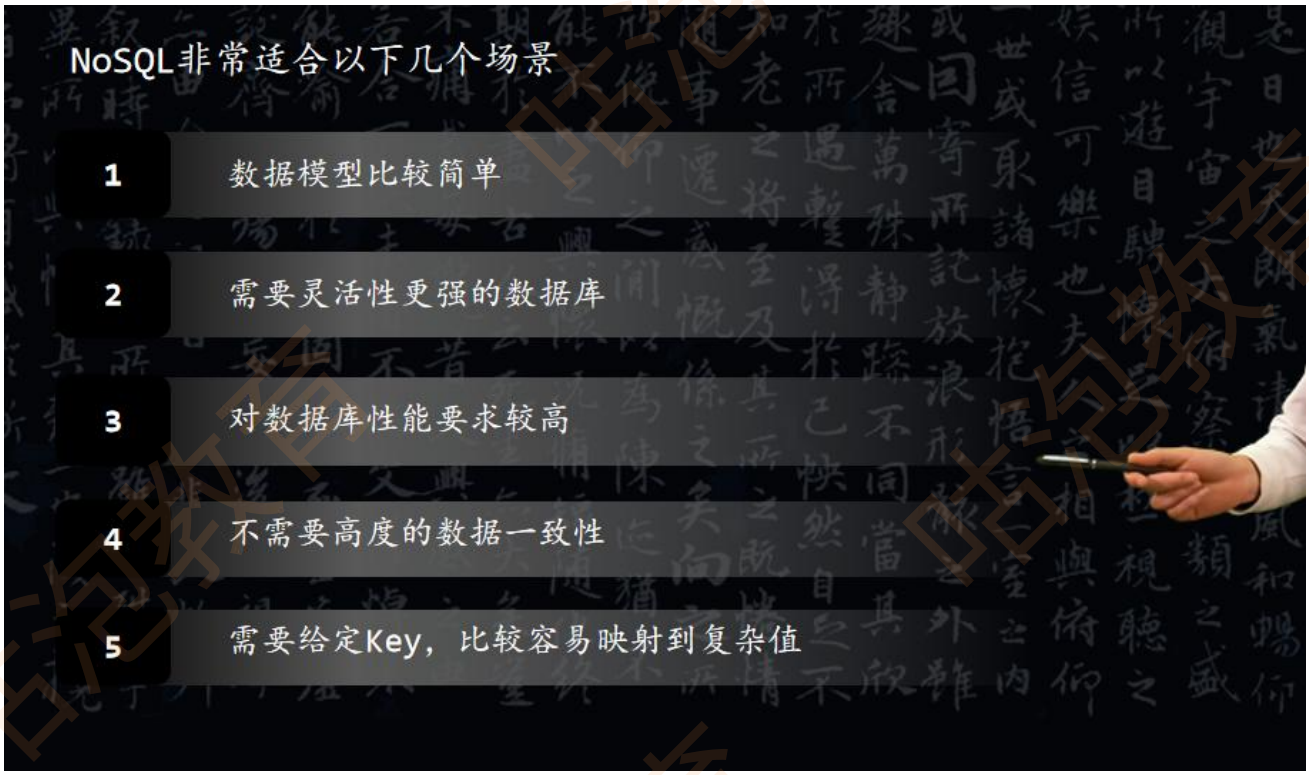
1、什么是NoSQL?

NoSQL呢常见的解释有Non-Relational SQL或者Not Only SQL，不过Not Only SQL被更多人接受，一般泛指非关系型数据库。它和关系型数据库不同的是，不保证关系数据的ACID特性。



随着互联网的发展，NoSQL数据库的产生就是为了解决超大规模和高并发系统中多重数据种类带来的挑战，特别是大数据应用的难题。目前具有代表性的NoSQL数据库有Redis、HBase、ES、MongoDB等等。

NoSQL非常适合以下几个场景：



1、数据模型比较简单；

2、需要灵活性更强的数据库；

3、对数据库性能要求较高；

4、不需要高度的数据一致性；

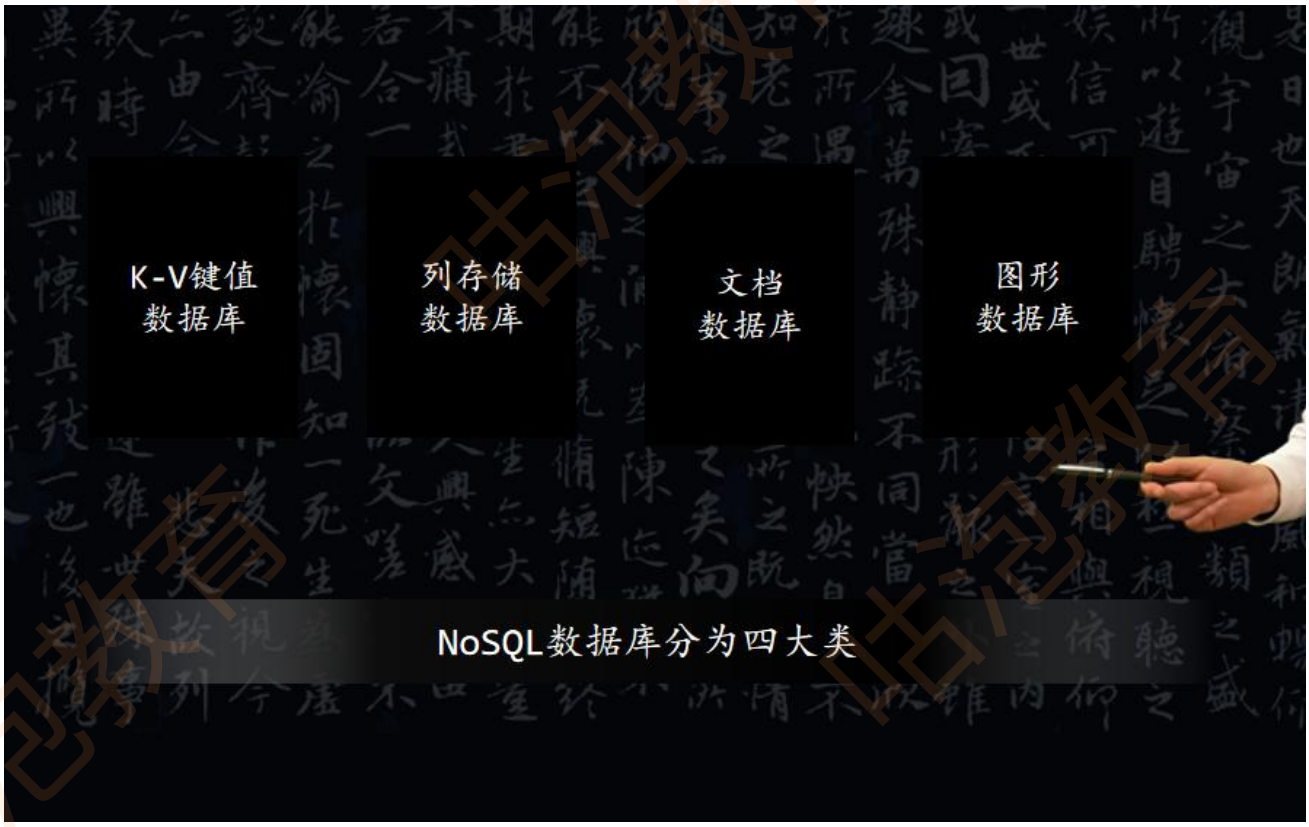
5、对于给定Key，比较容易映射复杂值的环境。

【导航条：NoSQL分类】

（转场，屏幕黑底白字，正中央出现“2 NoSQL分类）

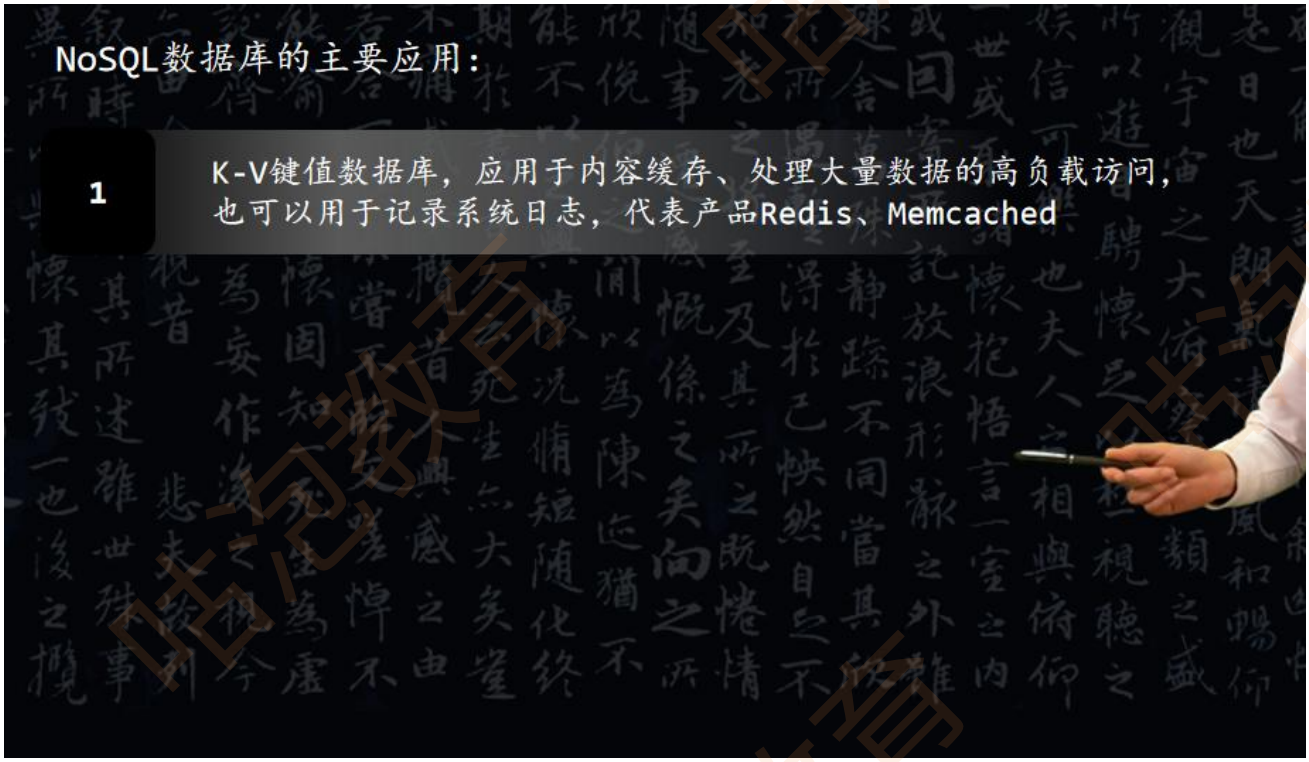
2、NoSQL分类

通常来说，现在主流的NoSQL数据库可以分为四大类：K-V键值数据库、列存储数据库、文档数据库和图形数据库。

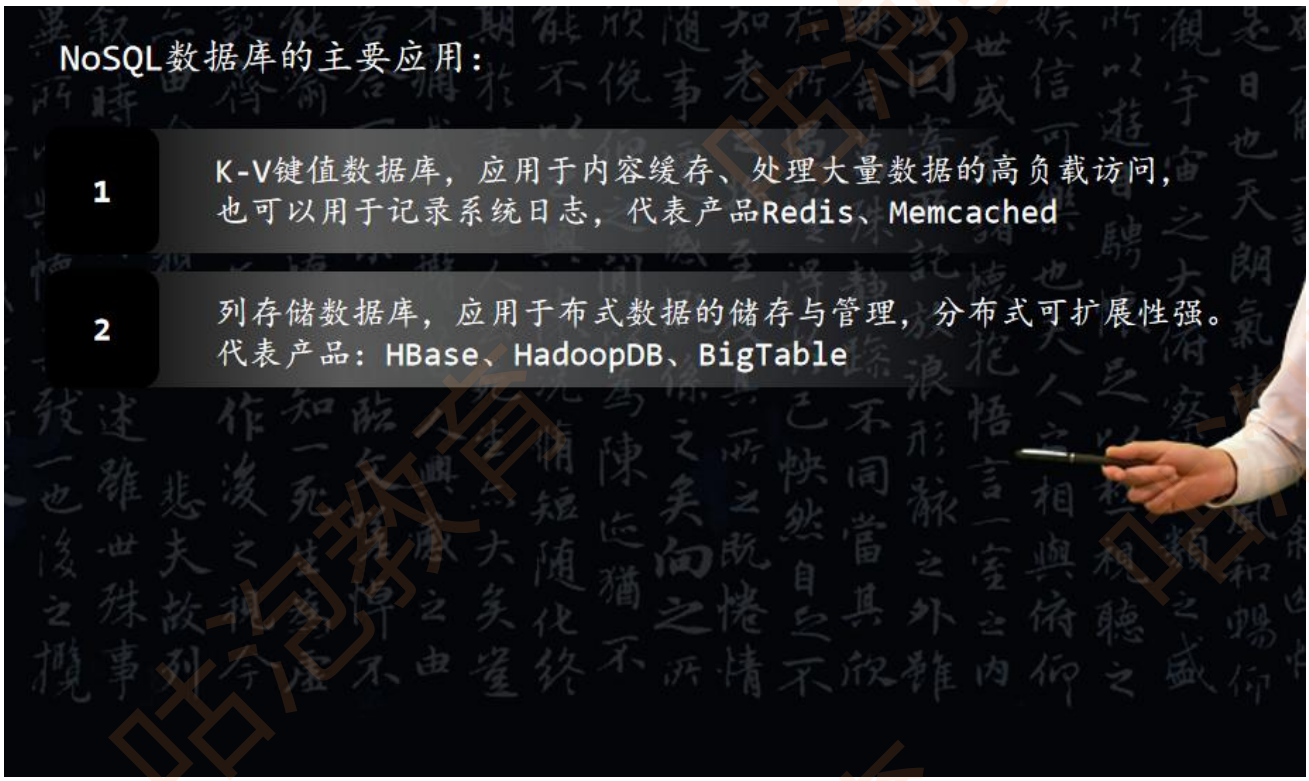


首先来看，K-V键值数据库，主要应用于内容缓存、处理大量数据的高负载访问，也可以用于记录系统日志。比较有

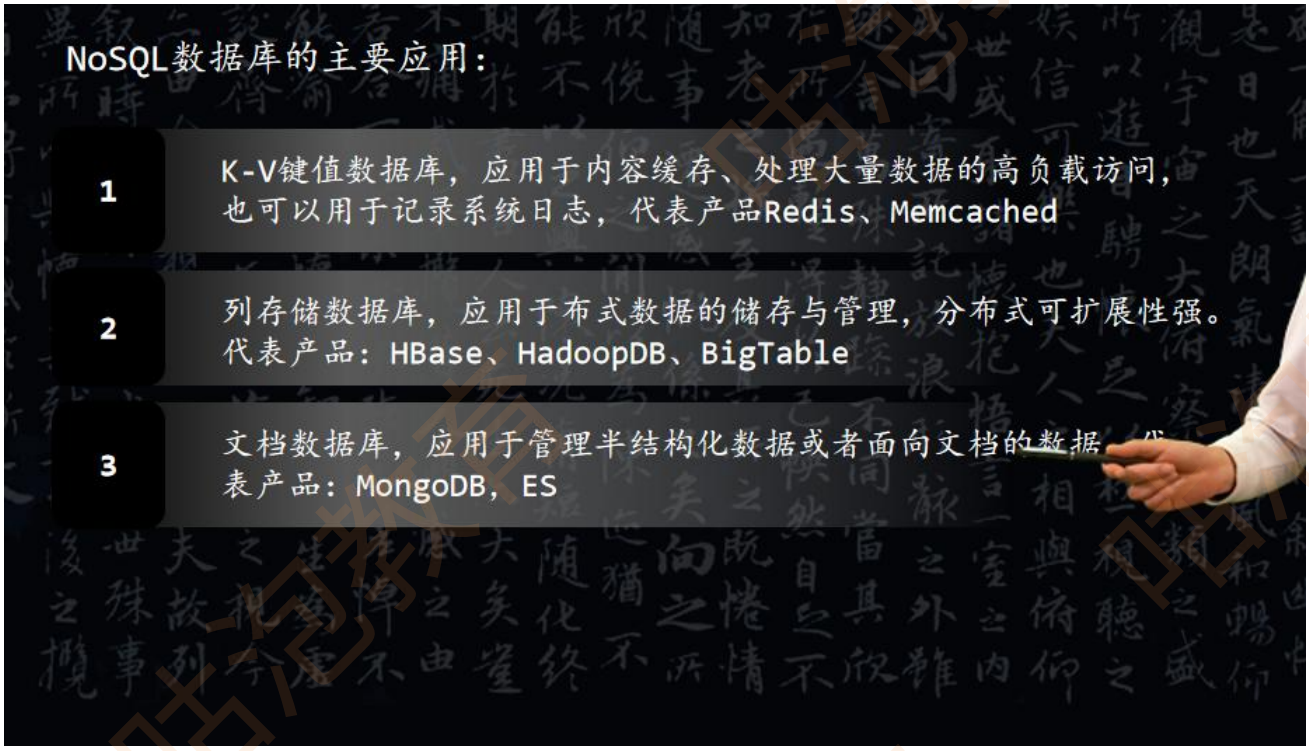
代表性的产品有Redis、Memcached等等。



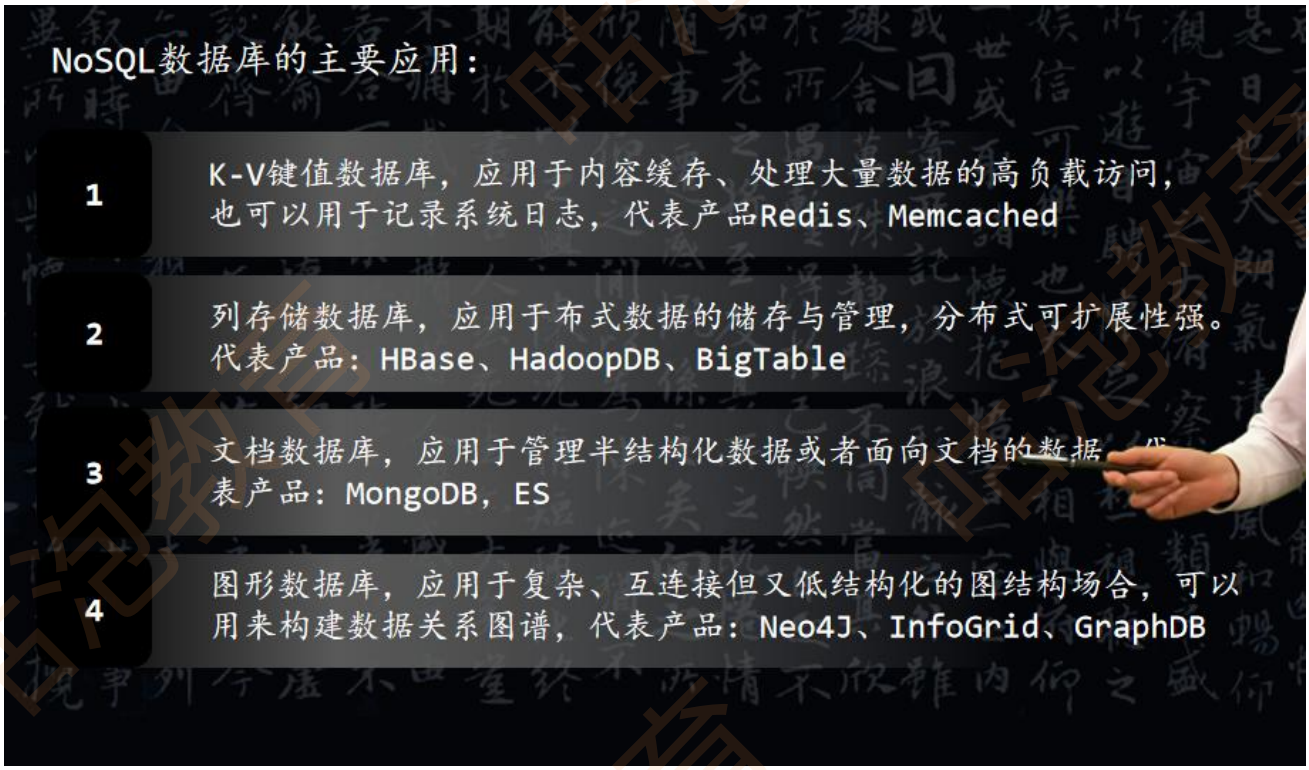
其次是，列存储数据库，主要应用于布式数据的储存与管理，分布式可扩展性强，比较有代表性的产品有HBase、HadoopDB、BigTable等等。



然后就是，文档数据库，主要应用于管理半结构化数据或者面向文档的数据，比较有代表性的产品有MongoDB，ES等等，当然，ES不仅仅只是数据库，它还是一个分布式搜索引擎，是基于Lucene来开发的，现在已经发展成为一个自有生态Elastic Stack。



最后就是，图形数据库，主要应用于复杂、互连接但又低结构化的图结构场合，可以用来构建数据关系图谱，Neo4J、InfoGrid、GraphDB等等。



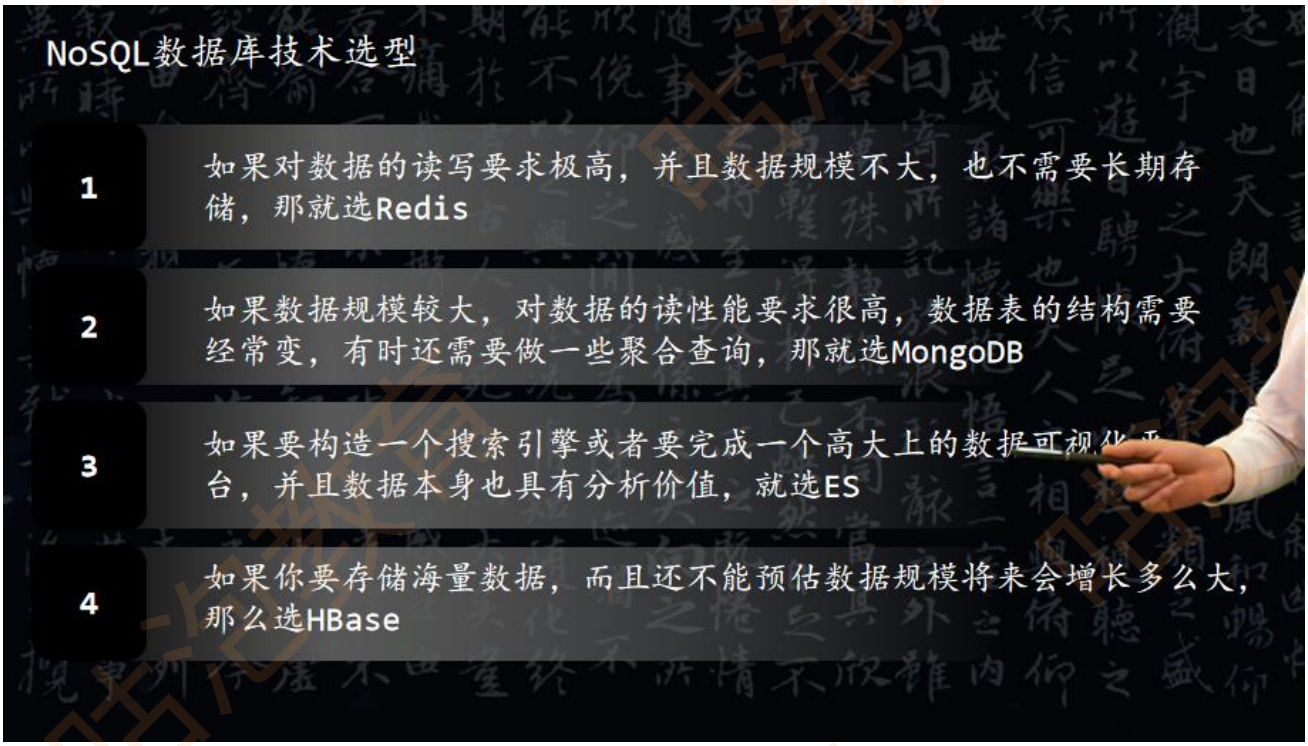
下面我把四种分类的NoSQL数据库整理成了一个表格，并总结了它们的优缺点

【导航条：主流NoSQL对比】

（转场，屏幕黑底白字，正中央出现“3 主流NoSQL对比）

3、主流NoSQL对比

对于NoSQL数据库对互联网开发的重要性就不多说了，在互联网分布式开发场景中，我认为掌握Redis、HBase、Elastic、MongoDB就能满足绝大部分需求。



如果对数据的读写要求极高，并且数据规模不大，也不需要长期存储，那就选Redis；

如果数据规模较大，对数据的读性能要求很高，数据表的结构需要经常变，有时还需要做一些聚合查询，那就选MongoDB；

如果要构造一个搜索引擎或者要完成一个高大上的数据可视化平台，并且数据本身也具有分析价值，就选ES；

如果你要存储海量数据，而且还不能预估数据规模将来会增长多么大，那么选HBase。

这里我也整理了一种表格，大家可以保存一下

NoSQL分类对比

支持情况	差	一般	好	极好
数据规模	redis	elastic	mongoDB	HBASE
查询性能	HBASE	elastic	mongoDB	redis
写入性能	elastic	mongoDB	HBASE	redis
复杂查询检索功能	redis	HBASE	mongoDB	elastic

最后，再给大家来个更加形象的对比：

Redis就相当于开战斗机，MongoDB就相当于坐高铁，HBase就相当于航母，ES那就是帝国铁骑，山地装甲机器人

【结尾】

好了，以上就是我对NoSQL的理解。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

55.为什么SQL语句命中索引比不命中索引要快?

【导航条：引言】

有位粉丝面试高开的时候被问到，为什么SQL语句命中索引比不命中索引要快？虽然自己也知道答案，但被问到的瞬间，就不知道如何组织语言了。今天，我给大家深度分析一下。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：索引的作用】

（转场，屏幕黑底白字，正中央出现“1 索引的作用）

1、索引的作用

想象一下，现在有一本包含几十万字的字典，有几百页厚，同时里面的字是无序排列的。如果在不使用目录的情况下，我们如何从字典中找出需要的字来呢？毫无疑问，我们只能一页一页的翻，显然，这是一项反人类的的工作。

我们必然想的是先看目录，然后，找到相关的字或者偏旁，然后，找到对应的页码再去查找想要找的文字，这样，效率就大大提高了。而事实上，目录就是一种索引，我们说的数据库索引思想和目录的思想一脉相承。

数据库索引最主要的作用就是帮助我们快速检索到想要的数据，从而不至于每次查询都做全局扫描。

假设不使用任何算法的情况下，我们要查询10万条记录中的某一条，在最坏的情况下需要遍历10万次。

但如果使用二分查找算法，则只需要进行$\log_2 20000$次，也就是14.287712次即可。这意味着我们只需对排序后的值进行14次搜索，就可以使用二分查找到想要的唯一值，常见的索引数据结构有B树和B+树。

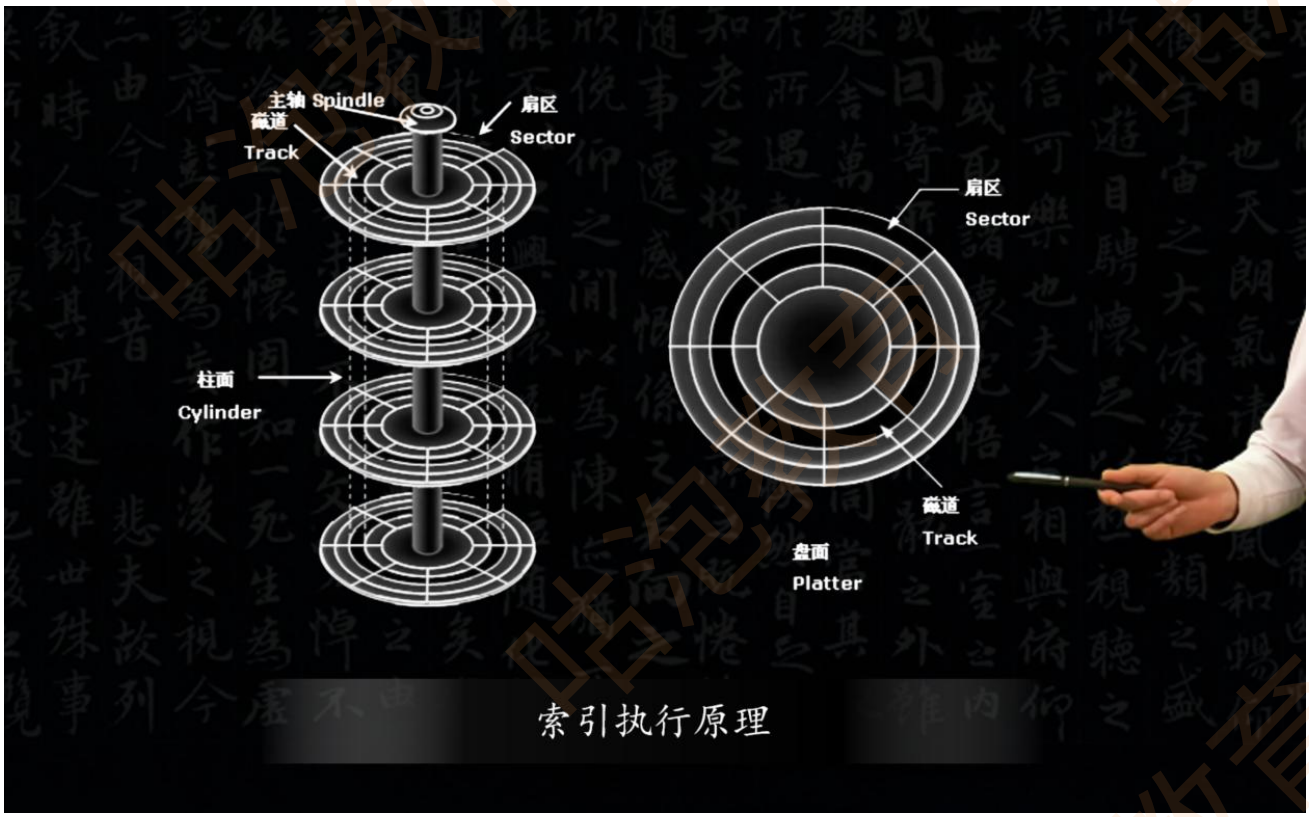
下面我们，以MySQL的InnoDB引擎为例，分析一下索引的工作原理。

【导航条：索引执行原理】

（转场，屏幕黑底白字，正中央出现“2 索引执行原理）

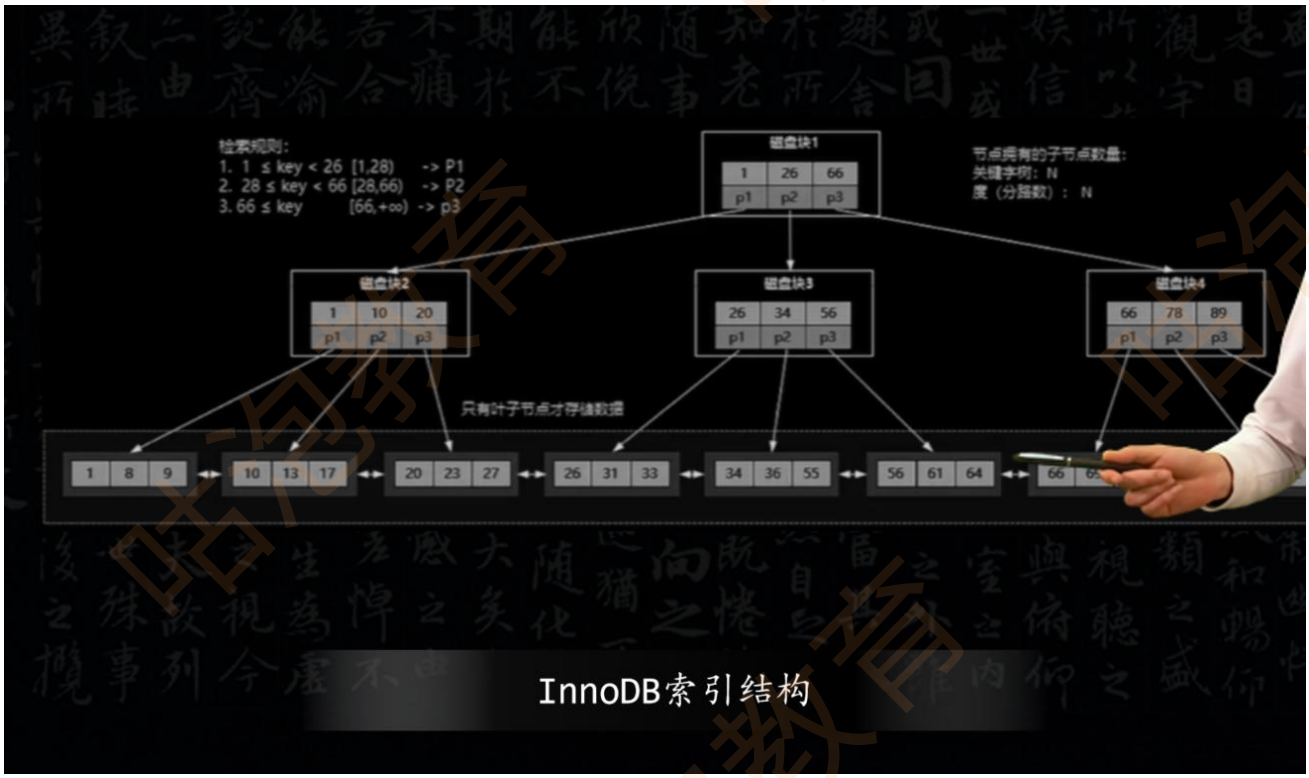
2、索引执行原理

我们知道MySQL的InnoDB引擎采用的是B+树数据结构，当我们去执行SELECT语句查询数据的时候，InnoDB需要从磁盘上去读取数据，而这个过程会涉及到磁盘 以及磁盘的随机IO ， 我们来看这么一个图：



系统会把数据的逻辑地址传给磁盘，磁盘控制线路按照寻址逻辑把逻辑地址翻译成物理地址。也就是确定要读取的数据在哪个磁道、哪个扇区。为了读取这个扇区的数据，需要把磁头放在这个扇区上面，为了实现这样一个点，磁盘会不断地去旋转。把目标扇区旋转到磁头下面，使得磁头能够去找到对应的磁道。这里还会涉及到寻道的时间以及旋转时间的一个损耗。很明显磁盘IO这个过程的性能开销是非常大的，尤其是查询的数据量比较多的情况下。

所以InnotDB里面，干脆对存储在磁盘上的数据建立一个索引，然后把索引数据以及索引列对应的磁盘地址以B+树的方式进行存储。来看这么一个图：



当我们需要查找目标数据的时候，根据索引从B+树中去查找目标数据就行了。由于B+树的子树比较多，所以，只需要较少次数的磁盘IO就能够查找到目标数据。

至于B+树的数据结构，在这里就不分析了。大家可以去我的个人主页看往期视频有讲到。

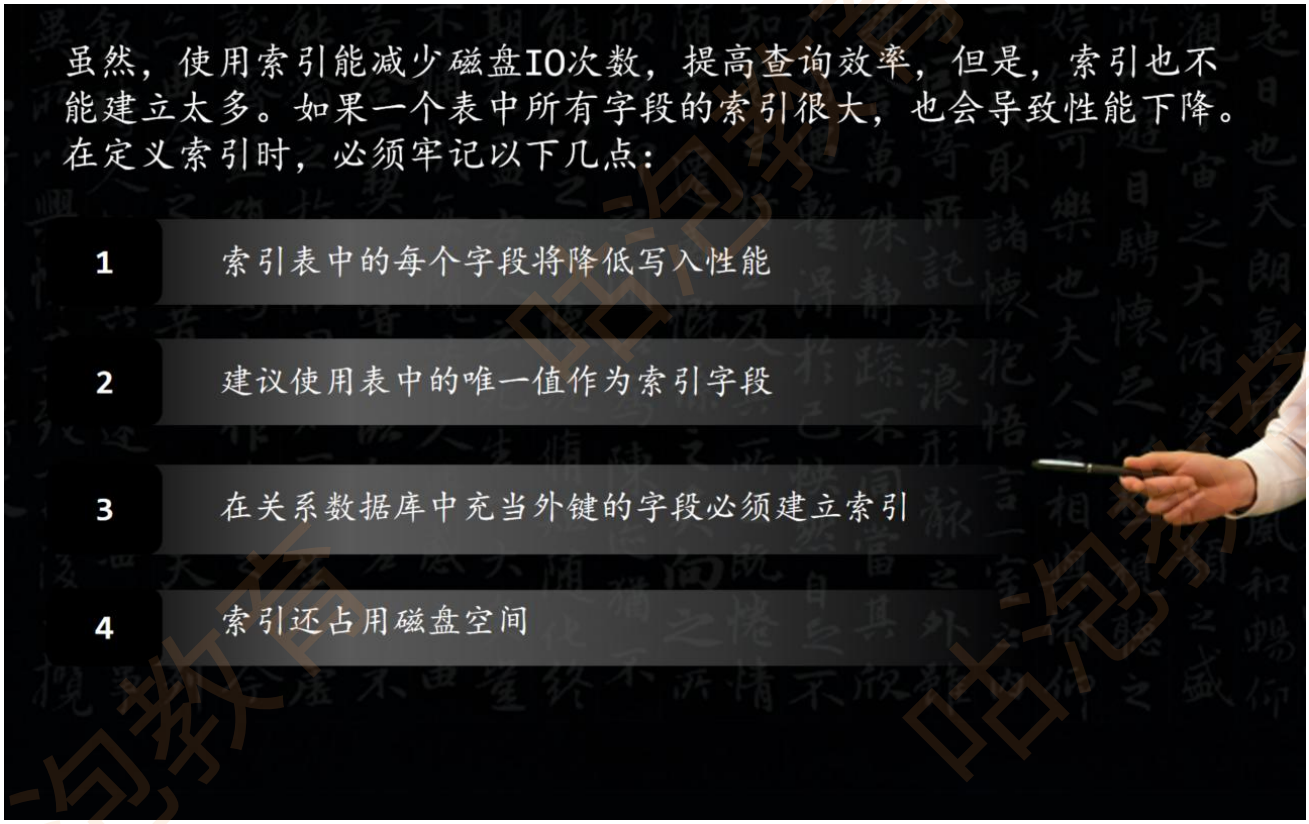
【导航条：索引的弊端】

（转场，屏幕黑底白字，正中央出现“3 索引的弊端）

3、索引的弊端

虽然，使用索引能减少磁盘IO次数，提高查询效率，但是，索引也不能建立太多。如果一个表中所有字段的索引很大，也会导致性能 l下降。想象一下，如果一个索引和一个表一样长，那么它将再次成为一个需要检查的开销。这就好比字典的目录非常详细，但是其长度已经和所有的文字一样长，这个时候目录本身的效率就大大下降了。

那索引有弊端吗？肯定是有的，索引可以提高查询读取性能，而它会将降低写入性能。当有索引时，如果更改一条记录，或者在数据库中插入一条新的记录，它将执行两个写入操作（一个操作是写入记录本身，另一个操作是将更新索引）。



因此，在定义索引时，必须牢记以下几点：

1、索引表中的每个字段将降低写入性能。

2、建议使用表中的唯一值为字段编制索引。

3、在关系数据库中充当外键的字段必须建立索引，因为它们有助于跨多个表进行复杂查询。

4、索引还使用磁盘空间，因此在选择要索引的字段时要小心。

【结尾】

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

56.如何安全地中断一个正在运行的线程?

【导航条：引言】

一个位5年的小伙伴去某东面试被一道并发编程的面试题给Pass了，说”如何中断一个正在运行中的线程？，这个问题很多工作2年的都知道，实在是有些遗憾。

今天，我给大家来分享一下我的回答。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以c在我的个人煮叶简介中找到。

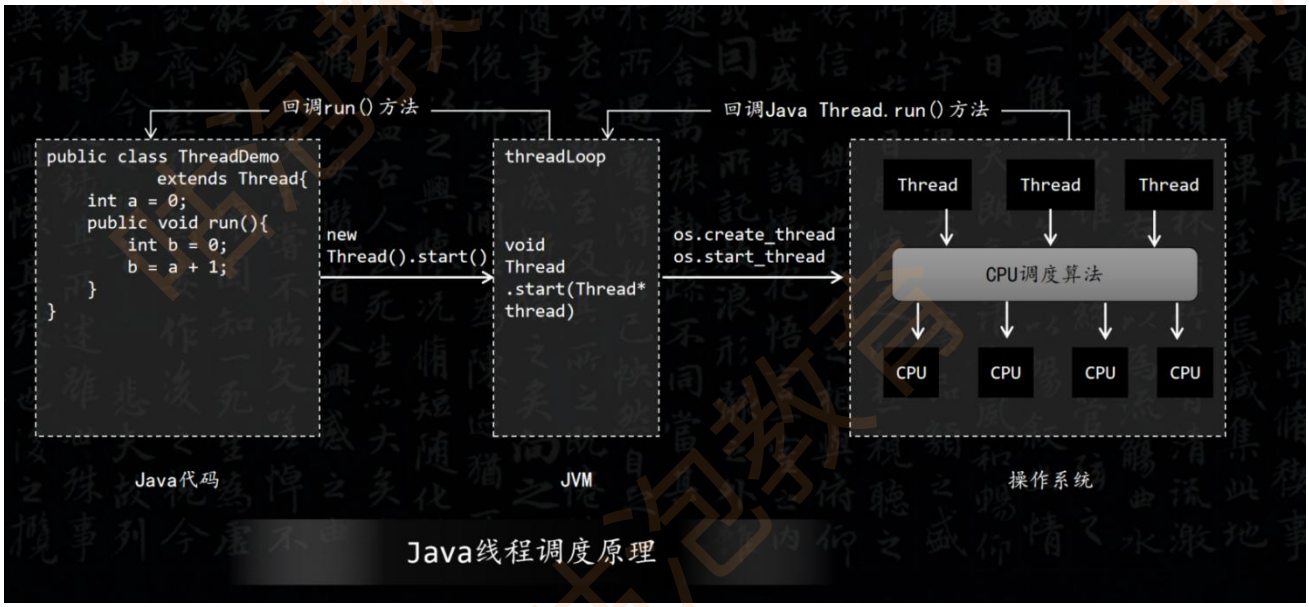
【导航条：什么是线程】

（转场，屏幕黑底白字，正中央出现“1 什么是线程）

1、什么是线程?

回答这个问题之前，先来回顾一下什么线程?

Thread，线程是操作系统进行运算调度的最小单位。所以，线程是系统级别的概念。



而在 Java 里面实现的线程，最终的执行和调度都是由操作系统来决定的，JVM 只是对操作系统层面的线程做了一层包装而已。

所以我们在 Java 里面调用 start() 方法启动一个线程的时候，只是告诉操作系统这个线程可以被执行，但是最终交给 CPU 来执行，是由操作系统的调度算法来决定的。

【导航条：如何安全中断】

（转场，屏幕黑底白字，正中央出现“2 如何安全中断运行中的线程?）

2、如何操作如何安全中断运行中的线程?

从理论上来说，要在 Java 层面去中断一个正在运行的线程，只能像类似于 Linux 里面的 kill 命令结束进程的方式一样，强制终止。

Java Thread 的API里面虽然提供了一个 stop() 方法，可以强行终止线程，但是这种方式是不安全的，因为有可能线程的任务还没有完成，突然中断会导致出现运行结果不正确的问题。

要想安全的中断一个正在运行的线程，只能在线程内部埋下一个钩子，外部程序 通过这个钩子来触发线程的中断命令。

因此，在 Java Thread 里面提供了一个 interrupt() 方法，这个方法要配合isInterrupted()方法来使用，就可以实现安全地中断线程运行。

来看这段代码：

```
Runnable runnable = new Runnable(){
    public void run() {
        while (true) {
            if (Thread.currentThread().isInterrupted()){
                System.out.println("线程被中断了");
                return ;
            } else {
                System.out.println("线程没有被中断");
            }
        }
    }
};
Thread t = new Thread(runnable);
t.start();
Thread.sleep(500);
t.interrupt();
System.out.println("线程中断了，程序到这里了");
```

这种实现方法并不是强制中断，而是告诉正在运行的线程，你可以停止了。何时实际中断，取决于正在运行的线程，所以，它能够保证线程运行结果的安全性。

【结尾】

这个问题，很多工作了 5 年以上的小伙伴都不一定清楚。虽然CRUD能够熟练上手，但是，和其他人拉开差距的往往是工作之外的 8 个小时。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

57.谈谈你对Spring Cloud的理解

【导航条：引言】

最近，有很多应届生也挺焦虑的，说大家都卷，不卷也不行。很多应届生在面试过程中八股文背得不错，但是一聊到项目经验就不知如何说起。

有位小伙伴来问我，说面试被问到对Spring Cloud的理解，自己也回答了，但感觉面试官不满意，说是没有结合项目经验来回答。

今天，我给大家分享一下我的理解，希望能够帮助到各位小伙伴。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：应用背景】

（转场，屏幕黑底白字，正中央出现“1 Spring Cloud的应用背景）

1、Spring Cloud的应用背景

Spring Cloud是Spring官方推出来的一套微服务解决方案。

准确来说，Spring Cloud其实是对微服务架构里面出现各种技术场景，定义了一套标准规范。

然后，在这套标准里面，Spring集成了Netflix公司的OSS开源套件，比如

Zuul 实现应用网关、

Eureka 实现服务注册与发现、

Ribbon实现负载均衡、

Hystrix实现服务熔断

我们可以使用Spring Cloud Netflix这套组件，快速落地微服务架构以及解决微服务治理等一系列问题。

但是随着Netflix OSS相关技术组件的闭源和停止维护，所以Spring官方也自研了一些组件，

比如Gateway实现网关、 LoadBalancer实现负载均衡。

另外，在国内的阿里也在为开源做出积极的贡献，它整合了Spring Cloud的标准，同时，加入了自身的技术，成为了Spring Cloud生态中另一套微服务解决方案Spring Cloud Alibaba。

其中包括Dubbo做RPC通信、Nacos实现服务注册与发现以及动态配置中心、Sentinel实现服务限流和服务降级等等。

【导航条：Spring CloudSpring Cloud的好处】

（转场，屏幕黑底白字，正中央出现“2 使用Spring Cloud的好处）

2、使用Spring Cloud的好处

以上就是我对Spring Cloud的理解。最后，我再补充一下，我认为Spring Cloud生态的出现有两个很重要的意义。

1、在Spring Cloud出现之前，为了解决微服务架构里面的各种技术问题，需要去集成各种开源框架，因为标准和兼容性问题，所以在实践的时候很麻烦。国内还有很些企业基于阿里的Dubbo进行二次开发自研出Dubbo X，而Spring Cloud统一了这样一个标准。

2、使用Spring Cloud降低了微服务架构的开发难度，只需要在Spring Boot的项目基础上通过Starter启动依赖集成相关组件就能轻松解决各种问题。

【结尾】

在现在的互联网分布式应用开发中，Spring Cloud的普及程度非常大了。很多应届生往往知道Spring Cloud怎么用，但是为什么用，以及在什么场景下用，却不一定能明白。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

58.new String("hello")之后，到底创建了几个对象?

【导航条：引言】

最近，有很多优秀的程序员从大厂毕业， 再加上大环境的影响，很多正在找工作的小伙伴也感觉技术面试越来越难，基本上都会问技术底层原理，甚至有些还会问到操作系统层面的知识。 如果技术功底不扎实，确实很难找到合适的岗位。

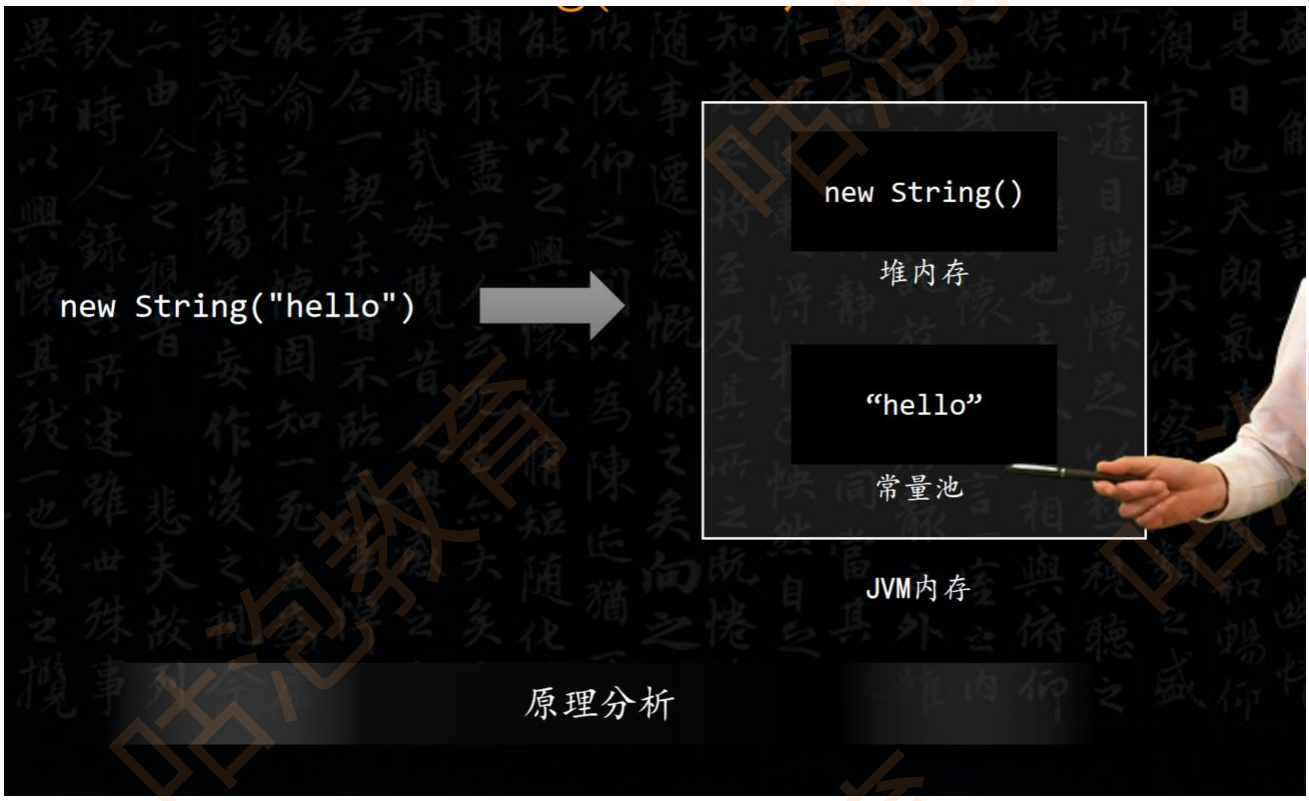
今天，给大家分享这样一个问题，” new String(“hello”)之后，到底创建了几个对象?

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：原理分析】

（转场，屏幕黑底白字，正中央出现“1 原理分析）

1、原理分析



首先，这个代码里面有一个 new 关键字，这个关键字是在程序运行时，根据已经加载的系统类 String，在堆内存里面实例化的一个字符串对象。

然后，在这个 String 的构造方法里面，传递了一个“hello”字符串，因为 String 里面的字符串成员变量是 final 修饰的，所以它是一个字符串常量。

接下来，JVM 会拿字面量“hello”去字符串常量池里面试图去获取它对应的 String 对象引用，如果拿不到，就会在堆内存里面创建一个”hello”的 String 对象，并且把引用保存到字符串常量池里面。

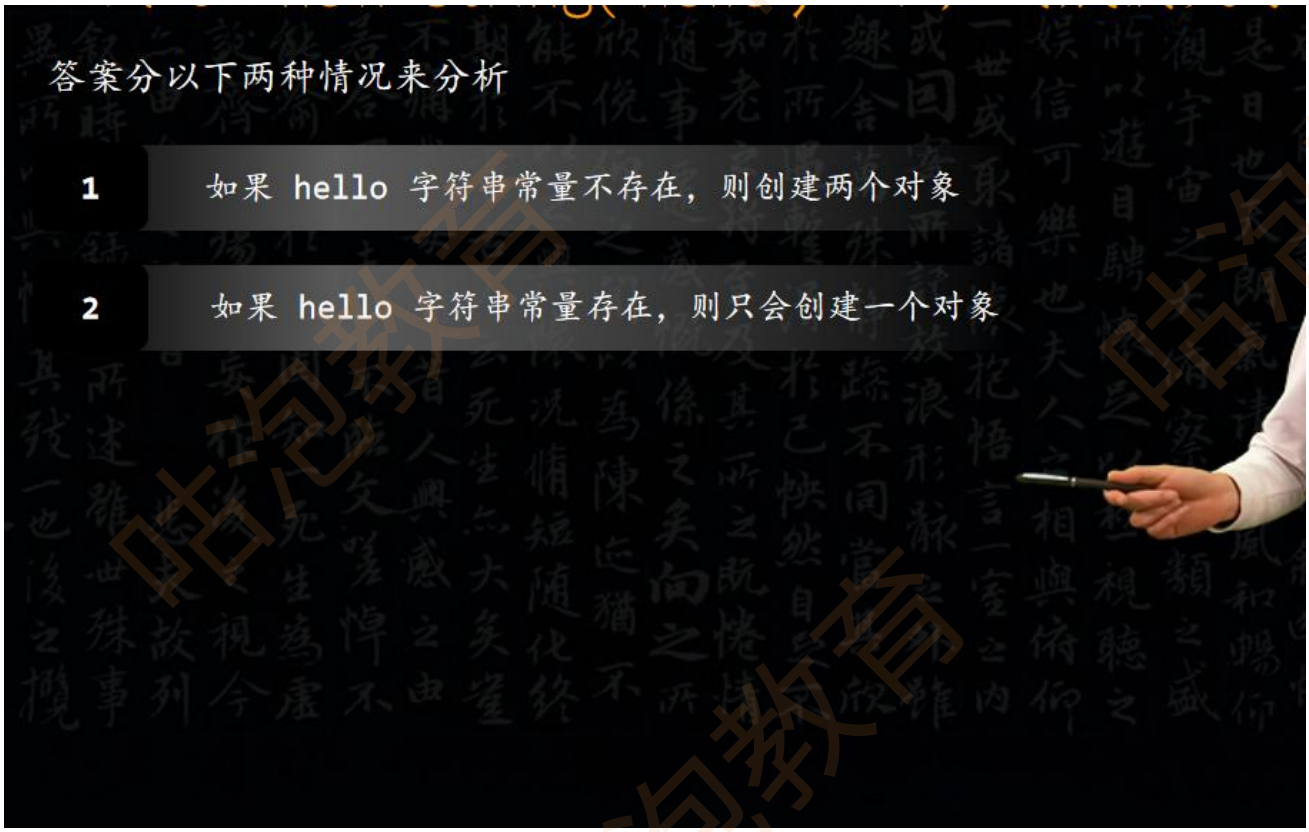
后续如果再有字面量“hello”的定义，因为字符串常量池里面已经存在了字面量“hello”的引用，所以只需要从常量池获取对应的引用就可以了，不需要再创建。

【导航条：答案及扩展】

（转场，屏幕黑底白字，正中央出现“2 答案解析和知识扩展）

2、答案解析和知识扩展

所以，对于这个问题的答案分以下两种情况来分析：



如果 hello 这个字符串常量不存在，则创建两个对象，分别是 hello 这个字符串常量， 以及 new String 这个实例对象。

如果 hello 这字符串常量存在，则只会创建一个对象。

小伙伴们再来看这样一段代码：

```
public static void main(String[] args) {

        String s1 = "hello";

        String s2 = "hello";

        String s3 = "he" + "llo";

        String s4 = "hel" + new String("lo");

        String s5 = new String("hello");

        String s6 = s5.intern();

        String s7 = "h";

        String s8 = "ello";

        String s9 = s7 + s8;

    }
```

它的输出结果如下：

```
System.out.println(s1==s2);//true

System.out.println(s1==s3);//true

System.out.println(s1==s4);//false

System.out.println(s1==s9);//false

System.out.println(s4==s5);//false

System.out.println(s1==s6);//true
```

接下来给大家简单分析一下：

*（希望讲到的位置用IIII IIII

字面量在字符串常量池中的引用；否则，复制一份该字面量到字符串常量池并返回它的引用。因此s1==s6输出true。

【结尾】

其实，要回答好这道题，需要对JVM的运行时内存划分以及JVM常量池有深刻的理解，这也是享元模式在JDK源码中的应用。现在的技术面试已经逐渐趋向于体系化的考查，不再是点状式的提问了。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

59.MyBatis何时使用一级缓存，何时使用二级缓存?

【导航条：引言】

一个5年工作经验的小伙伴，去面某东被问到MyBatis何时使用一级缓存，何时使用二级缓存？去之前还特地复习了MyBatis的相关知识，想着自己用MyBatis用得比较熟练了，竟然在这道题上翻车了。

今天，我给大家来分享一下MyBatis的缓存机制。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：设计思想】

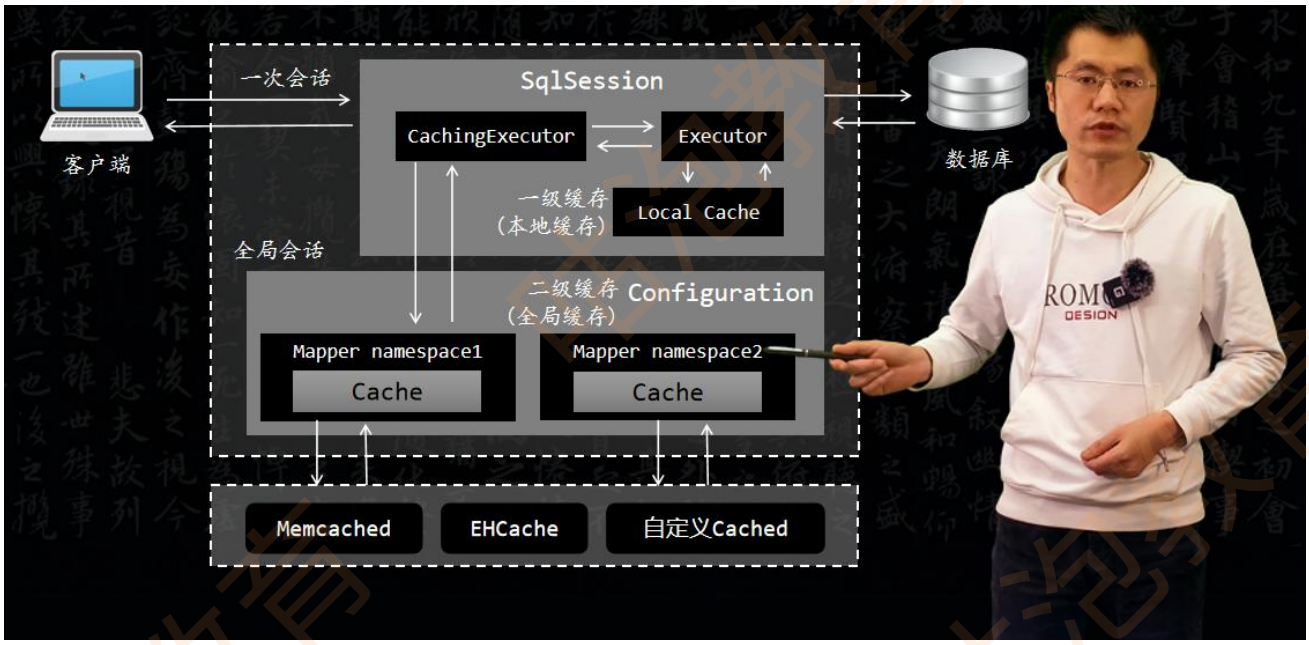
（转场，屏幕黑底白字，正中央出现“1 设计思想）

1、设计思想

Mybatis里面设计了两级缓存来提升数据的检索效率，避免每次数据的访问都需要去查询数据库。

先来看一级缓存，它是SqlSession级别的缓存，也叫本地缓存，因为每个用户在执行查询的时候都需要使用SqlSession来执行，为了避免每次都去查数据库，MyBatis把查询出来的数据保存到SqlSession的本地缓存中，后续的SQL如果命中缓存，就可以直接从本地缓存读取。

那如果想要实现跨SqlSession级别的缓存？一级缓存就无法实现了，因此，MyBatis引入了二级缓存。当多个用户在查询数据的时候，只要有任何一个SqlSession拿到了数据就会放入到二级缓存里面，其他的SqlSession就可以从二级缓存加载数据。



【导航条：原理分析】

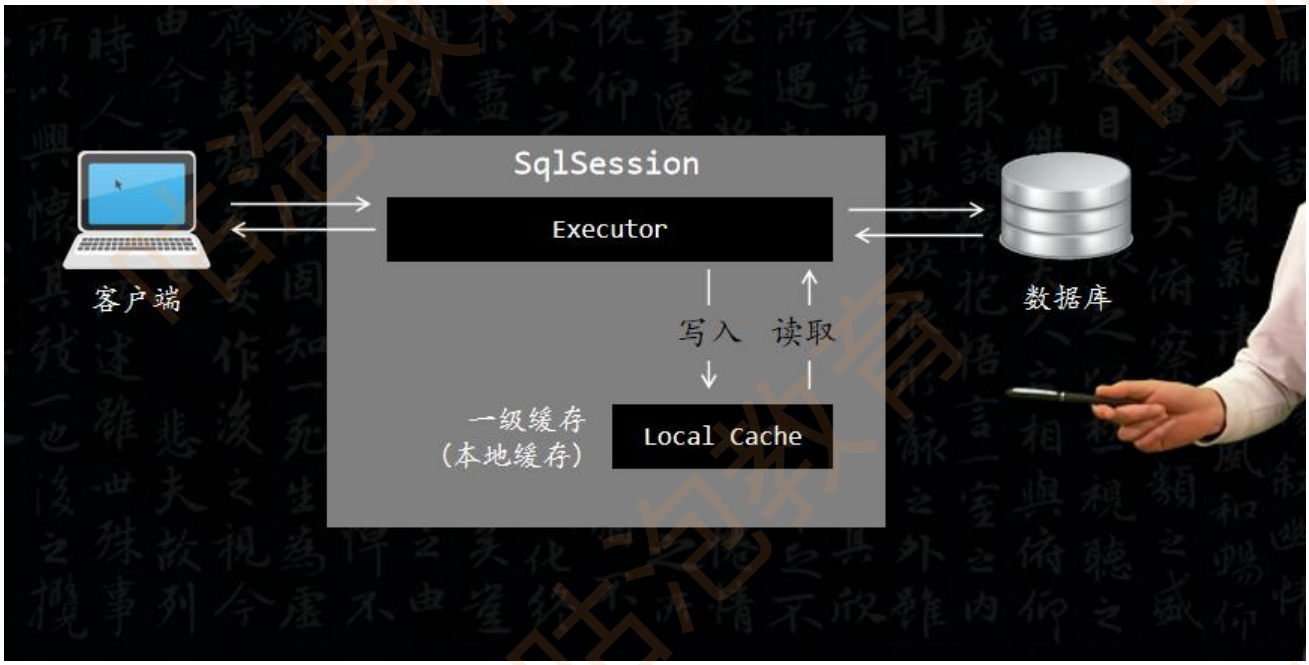
（转场，屏幕黑底白字，正中央出现“2 原理分析）

2、原理分析

接下来，我给大家详细分析一下MyBatis缓存机制的实现原理。先来看一级缓存的实现原理：

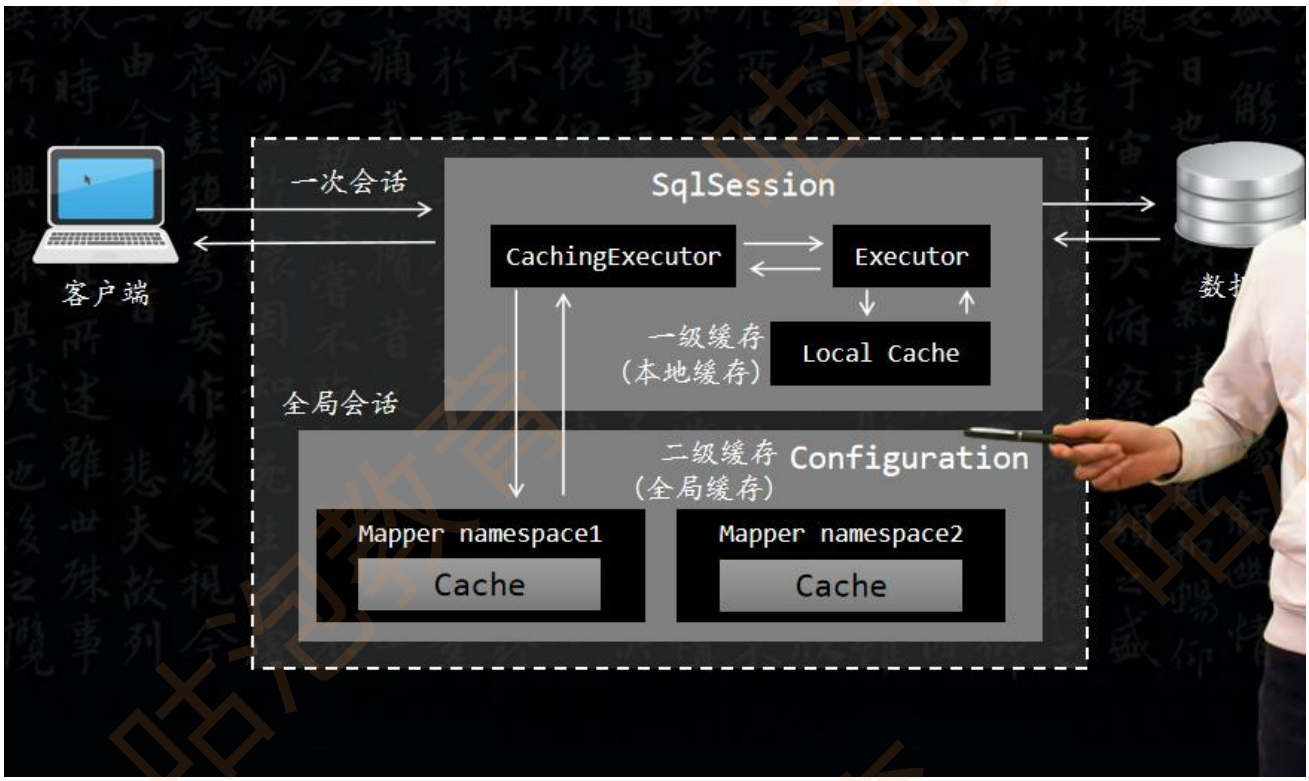
在SqlSession 里面持有一个Executor的对象，每个Executor中有一个LocalCache对象。

当用户发起查询的时候，MyBatis会根据执行语句在Local Cache里面查询，如果没命中，再去查询数据库并写入到LocalCache，否则直接返回。



所以，一级缓存的生命周期只在SqlSession级别，而且在多个SqlSession或者分布式环境下，可能会导致数据库写操作出现脏数据。

那这个时候，就要使用二级缓存。下面来看二级缓存的具体实现原理：



二级缓存使用了一个叫做CachingExecutor的对象，对Executor进行了装饰，在进入一级缓存的查询流程之前，会先通过CachingExecutor进行二级缓存的查询。

开启二级缓存以后，会被多个SqlSession共享，所以它是一个全局缓存。因此它的查询流程是先查二级缓存，再查一级缓存，最后再查数据库。

另外，MyBatis 的二级缓存相对于一级缓存来说，实现了 SqlSession 之间缓存数据的共享，同时缓存粒度也能够控制到 Name Space 级别，并且还可以通过 Cache 接口实现类不同的组合，对 Cache 的可控性也更强。

【结尾】

以上就是我对这个问题的理解。在实际业务场景中，多级缓存的设计思想，非常值得我们学习和借鉴。学习底层技术的目的是为了提高技术思维能力和积累解决方案，才能够胜任更高的职位要求。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

60.什么是Java SPI，它有什么作用?

【导航条：引言】

很多主流框架都用到了SPI机制，比如Dubbo、Spring都有用到，可能很多小伙伴也听说过。今天给大家来聊一聊什么是SPI，它用什么作用?

【导航条：什么是SPI】

(转场，屏幕黑底白字，正中央出现“1什么是SPI)

1、什么是SPI

SPI全称Service Provider Interface，它是Java提供的一套用来被第三方实现或者扩展的API。

如图所示，简单来说，就是我们定义一个标准的接口，然后第三方的库里面可以实现这个接口。

那么，程序在运行的时候，会根据配置信息动态加载第三方实现的类，从而完成功能动态扩展的机制。

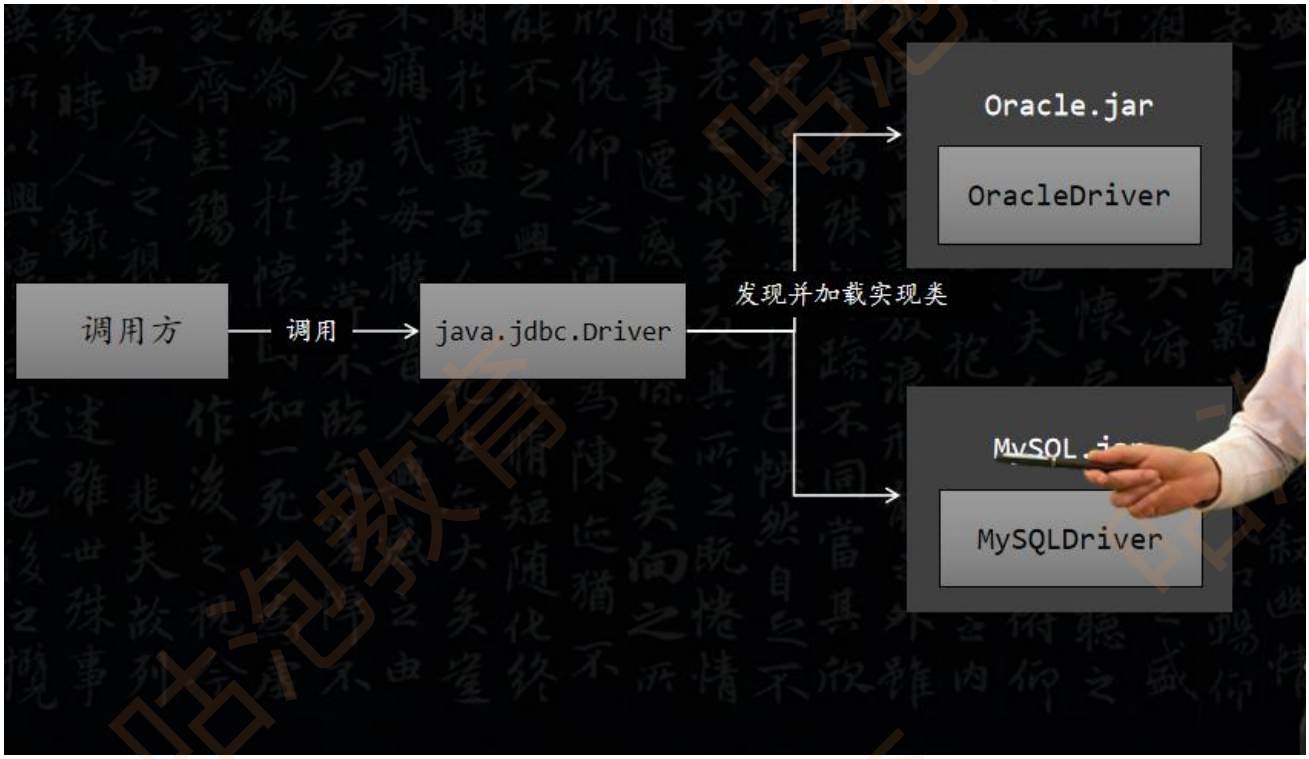
【导航条：SPI应用场景】

（转场，屏幕黑底白字，正中央出现“2 SPI的应用场景）

2、SPI的应用场景

SPI的主要作用是解耦。适用于调用者根据实际使用需要，启用、扩展、或者替换框架的实现策略。

在Java中，SPI机制有一个非常典型的实现案例，就是数据库驱动java.jdbc.Driver



如图所示，JDK里面定义了数据库驱动类Driver，它是一个接口，JDK并没有提供实现，具体的实现是由第三方数据库厂商来完成的。

在程序运行的时候，会根据我们声明的驱动类型，来动态加载对应的扩展实现，从而完成数据库的连接。

除此之外，在很多开源框架里面都借鉴了Java SPI的思想，提供了自己的SPI框架，比如Dubbo定义了ExtensionLoader，实现功能的扩展。

Spring提供了SpringFactoriesLoader，实现外部功能的集成，基于Servlet 3.0规范对ServletContainerInitializer的实现。

【结尾】

Java中SPI的设计主要用到了桥接模式，在实际开发中，借鉴这种思想可以在不修改核心代码的情况下，完成提供功能的增强和扩展，而且不需要显式地依赖于继承或者接口。以上就是我对Java SPI的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

61.如果不使用Spring，我们将如何开发?

【导航条：引言】

曾经有一道面试题掀起了劲爆的浪潮，说如果不使用Spring，我们将如何开发？好多家公司都模仿提问了这么一道面试题，而且好多人也都在各个社区给出了自己的答案。接下来看看网友们是怎么说的。

有说手写IoC的，有说用Servlet完成Web开发的，有说用纯JDBC完成数据库操作的等等，大家回答各有千秋。其实，想想我们上一代的程序员，没有Spring不也照样开发出非常多优秀的系统。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

【导航条：Spring生态】

（转场，屏幕黑底白字，正中央出现“1 Spring生态”）

1、Spring生态

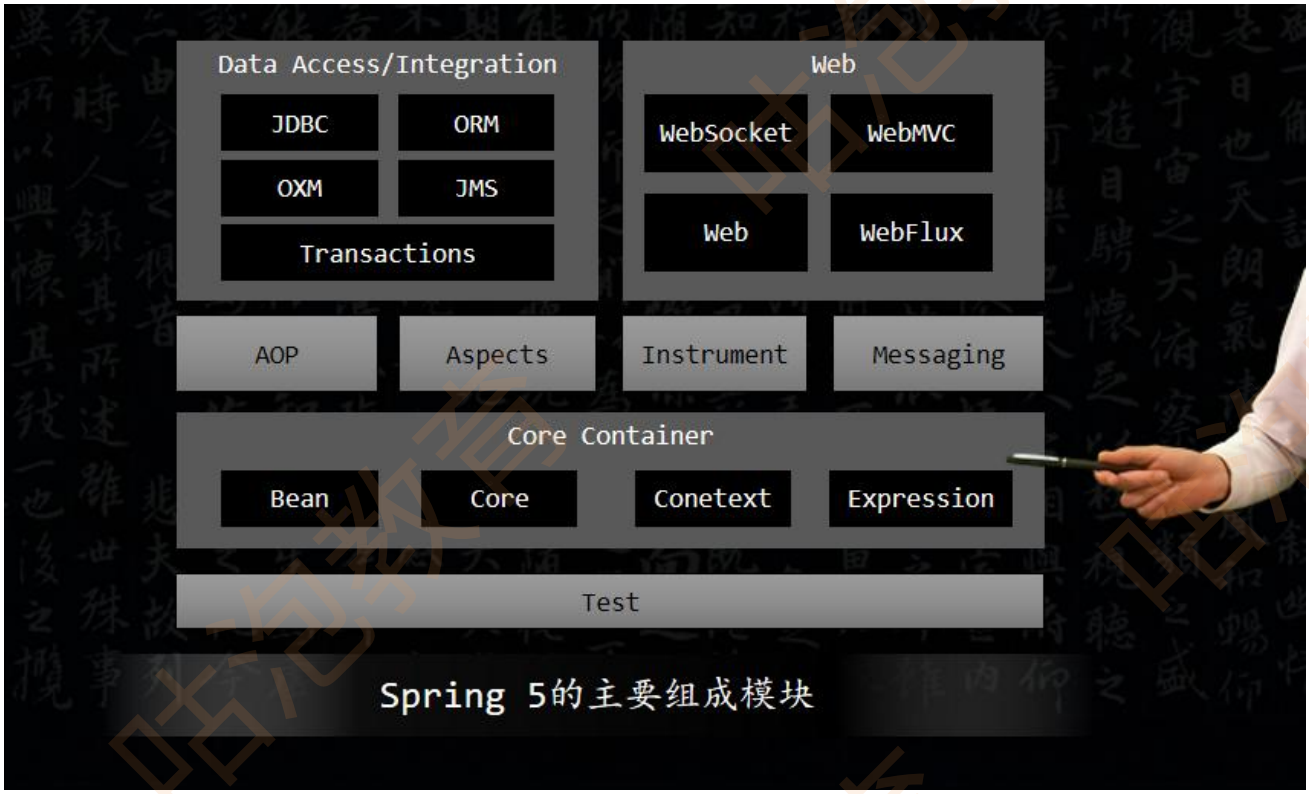
那我们使用Spring到底能给我们带来哪些好处呢?

根据我对Spring 10多年的研究经验，给大家做以下总结。

首先，Spring是一个轻量级的应用框架。当然，众所周知，它提供了 IoC 和 AOP 这两个核心的功能。

它的核心目的是为了简化企业级应用程序的开发，使得开发者只需要关心业务需求，不需要关心 Bean 的管理，以及通过切面增强功能减少代码的侵入性。

当然，Spring发展到现在俨然不只是一个框架，而是一个生态，我们习惯性地称之为Spring全家桶。Spring大约有20个模块，由1300多个不同的文件构成。这些模块可以分为核心容器、AOP和设备支持、数据访问与集成、Web组件、通信报文和集成测试、集成兼容等类。如图所示：



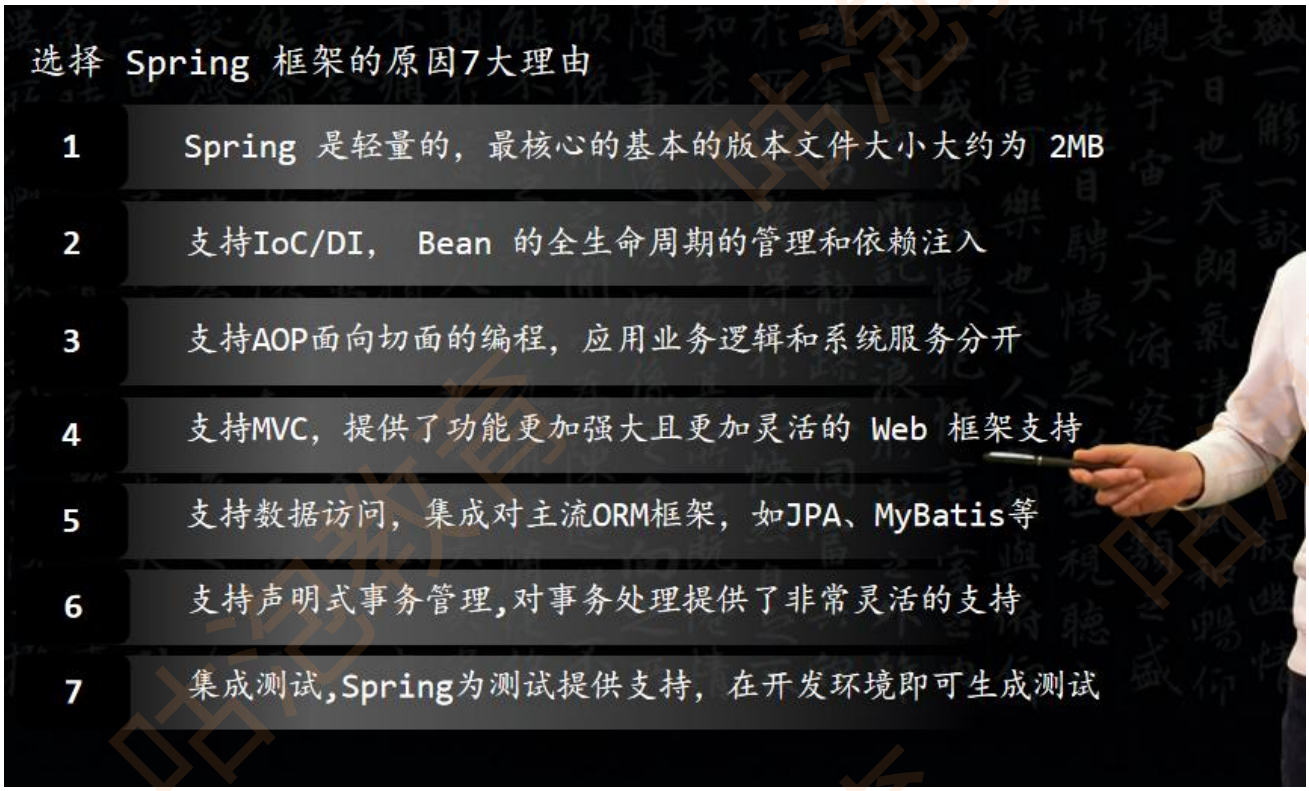
组成Spring框架的每个模块都可以单独存在，也可以将一个或多个模块联合实现。

【导航条：使用Spring的好处】

（转场，屏幕黑底白字，正中央出现“2 使用Spring的好处”）

2、使用Spring的好处

从 Spring 本身的特性来看，我认为有几个关键点是我们选择 Spring 框架的原因。



第1个，轻量：Spring 是轻量的，最核心的基本的版本文件大小大约为 2MB。

第2个，支持IoC/DI：当然，IoC不是Spring独有的。但最重要的是，可以使用Spring 的 IoC 容器实现对 Bean 的全生命周期的管理，还可以通过 DI 实现依赖注入，从而实现了对象依赖的松耦合管理。

第3个，支持AOP面向切面的编程：Spring 支持面向切面的编程，从而把应用业务逻辑和系统服务分开。

第4个，支持MVC：Spring MVC 提供了功能更加强大且更加灵活的 Web 框架支持

第5个，支持数据访问：Spring集成对主流ORM框架的支持，比如JPA、MyBatis等等。

第6个，声明式事务管理：Spring 通过 AOP 实现了事务的统一管理，对应用开发中的事务处理提供了非常灵活的支持

第7个，集成测试：Spring为测试提供支持，在开发环境即可生成测试

最后，Spring生态非常庞大。使用Spring几乎可以满足所有业务场景的开发。更重要的是社区的活跃度和技术的成熟

度都非常高。

【结尾】

以上就是我对Spring的理解。任何一个技术框架的出现，一定为了实现某些业务场景或者是某一类技术问题的解决方案。当大家有了的开发经验之后，对一些技术框架，要有自己的见解和思考。我自己在面试的时候也喜欢问这一类的问题。

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

62.谈谈你对线程安全的理解?

【导航条：引言】

一位4年工作经验的小伙伴，被问到一个非常抽象的问题，说，谈谈你对线程安全性的理解。如果平时只是刷刷面试题的话，遇到这种问题可能不知道如何说起了，往往需要自己组织语言。另外，如果平时积累不够的话，也很难说出一些自己独特的见解来。

今天，我分享一下我对线程安全的理解，希望能给小伙伴们作为参考。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶

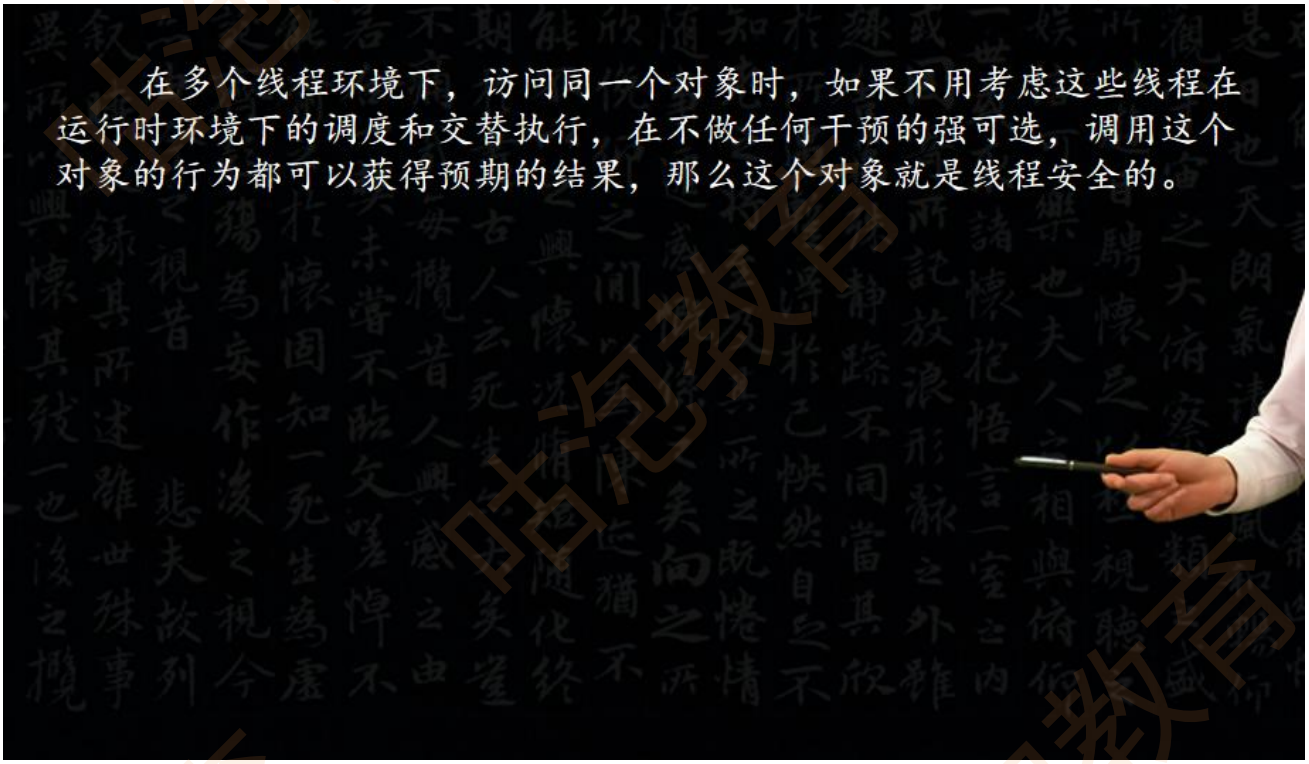
简介中找到。

【导航条：官方定义】

（转场，屏幕黑底白字，正中央出现“1 官方定义”）

1、官方定义

线程安全通常是相对于多线程或者并发的情况下而言的。如果是单线程操作的话，就无所谓线程安全了。



简单来说，就是在多个线程环境下，访问同一个对象时，如果不用考虑这些线程在运行时环境下的调度和交替执行，在不做任何干预的强可选，调用这个对象的行为都可以获得预期的结果，那么这个对象就是线程安全的。

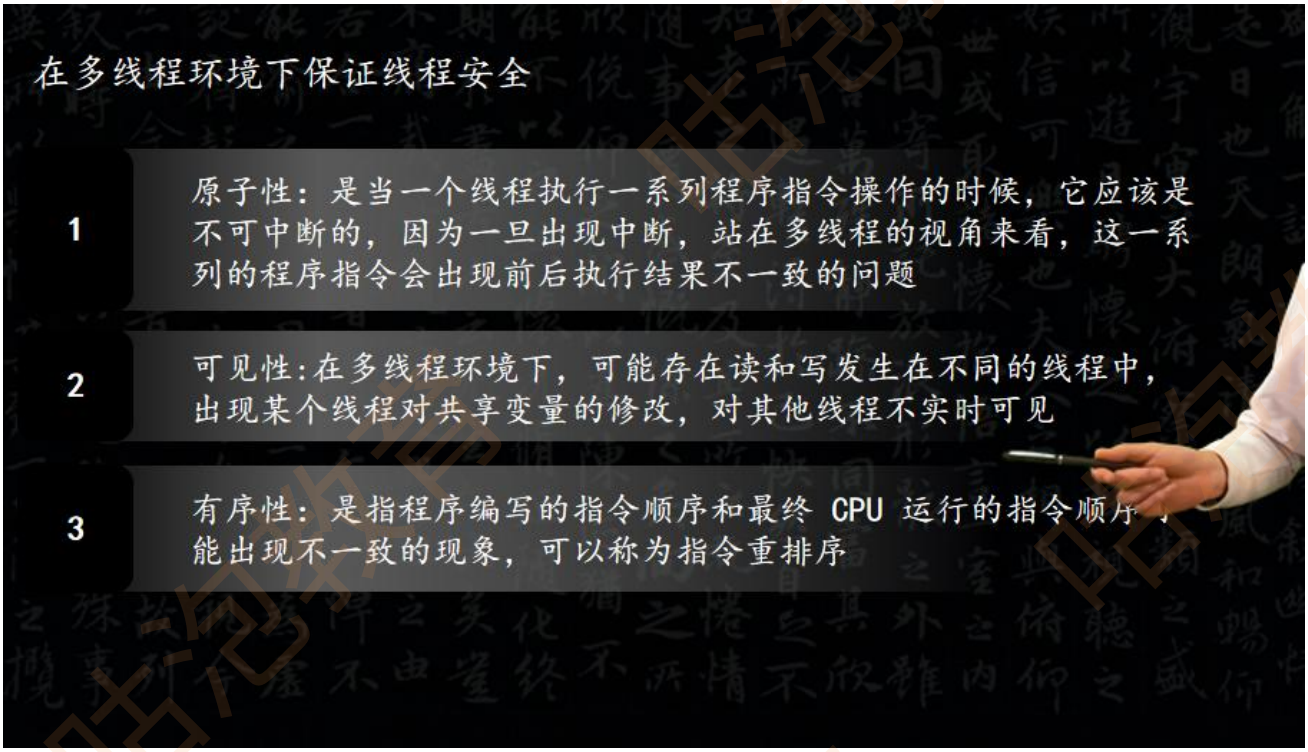
看完这段，是不是还是很难理解？下面，我来分享一下看我是如何理解的。

【导航条：Tom的理解】

（转场，屏幕黑底白字，正中央出现“2 Tom的理解”）

2、Tom的理解

我认为，在多线程环境下保证线程安全，无非就是保证对对象访问的原子性、有序性和多个线程之间的可见性。



原子性呢，说的是当一个线程执行一系列程序指令操作的时候，它应该是不可中断的，因为一旦出现中断，站在多线程的视角来看，这一系列的程序指令会出现前后执行结果不一致的问题。

这个和数据库里面的原子性是一样的，简单来说就是一段程序只能由一个线程完整的执行完成，而不能存在多个线程干扰。CPU的上下文切换， 是导致出现多线程原子性问题的核心原因， 而JDK面也提供了synchronized 关键字来解决原子性问题。

然后，就是可见性，相当于在多线程环境下，可能会存在读和写是发生在不同的线程里面，有可能出现某个线程对共享变量的修改，对其他线程不是实时可见的。导致可见性问题的原因有很多，比如 CPU的高速缓存、CPU的指令重排、编译器的指令重排等因素。

最后，就是有序性，指的是程序编写的指令顺序和最终 CPU 运行的指令顺序可能出现不一致的现象，这种现象

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

也可以称为指令重排序，所以有序性也会导致可见性问题。可见性和有序性可以通过JDK提供的volatile 关键字来解决。

我认为，导致有序性、原子性、可见性问题的本质，是计算机设计的时候，为了最大化提升 CPU 利用率导致的。比如CPU设计了三级缓存、设计了 StoreBuffer、设计了缓存行这种预读机制、在操作系统里面，设计了线程模型、在编译器里面，设计了编译器的深度优化机制等等。

【结尾】

我们都在说，面试造火箭，工作拧螺丝。对于企业来说，如果选择一个对计算机底层原理了解更透彻的程序员，不用担心他滥用线程导致一些不可预测的安全问题。其实也是在降低用人成本。

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

63.什么是可重入锁?它能解决什么问题?

【导航条：引言】

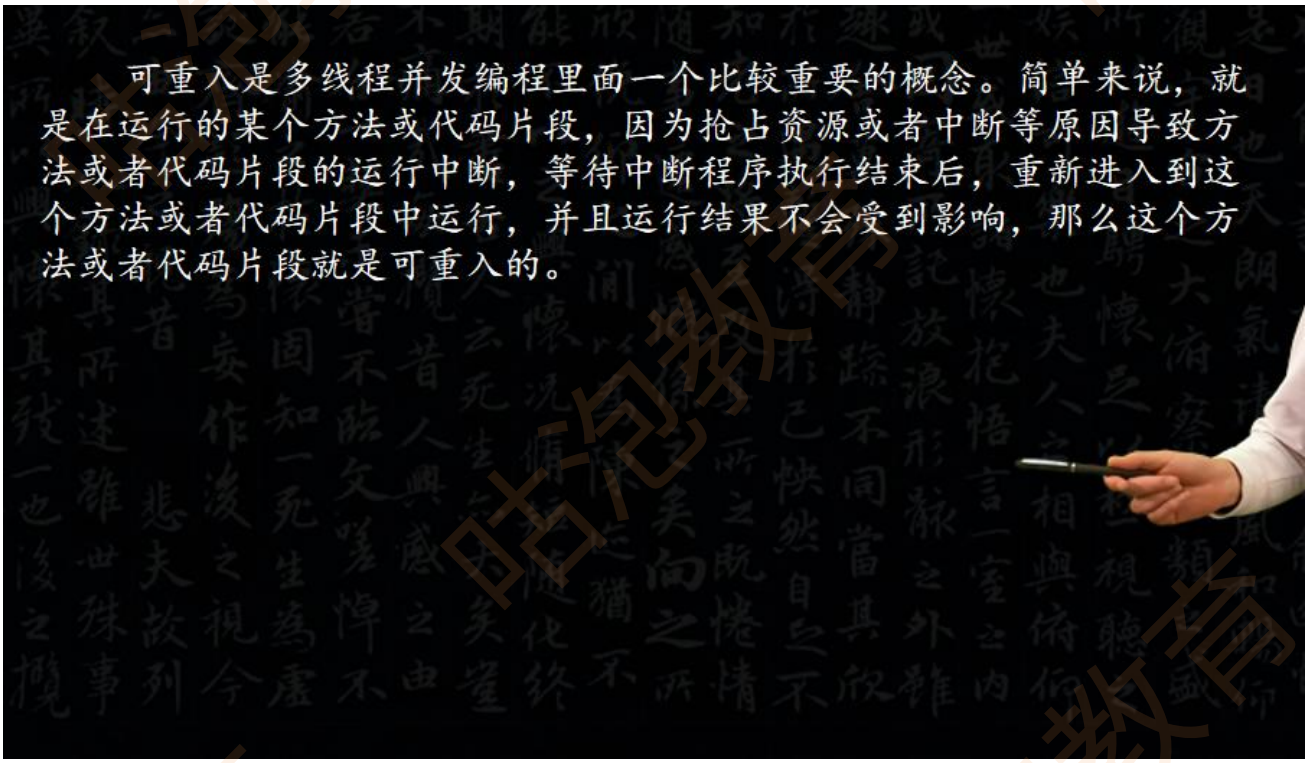
一位3年工作经验的小伙伴，去一家互联网公司面试，结果被面试官怼了。面试官说：”这么简单的问题你都不知道？没法聊了，回去等通知吧“。然后，回来跟我是一阵诉苦。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

【导航条：官方定义】

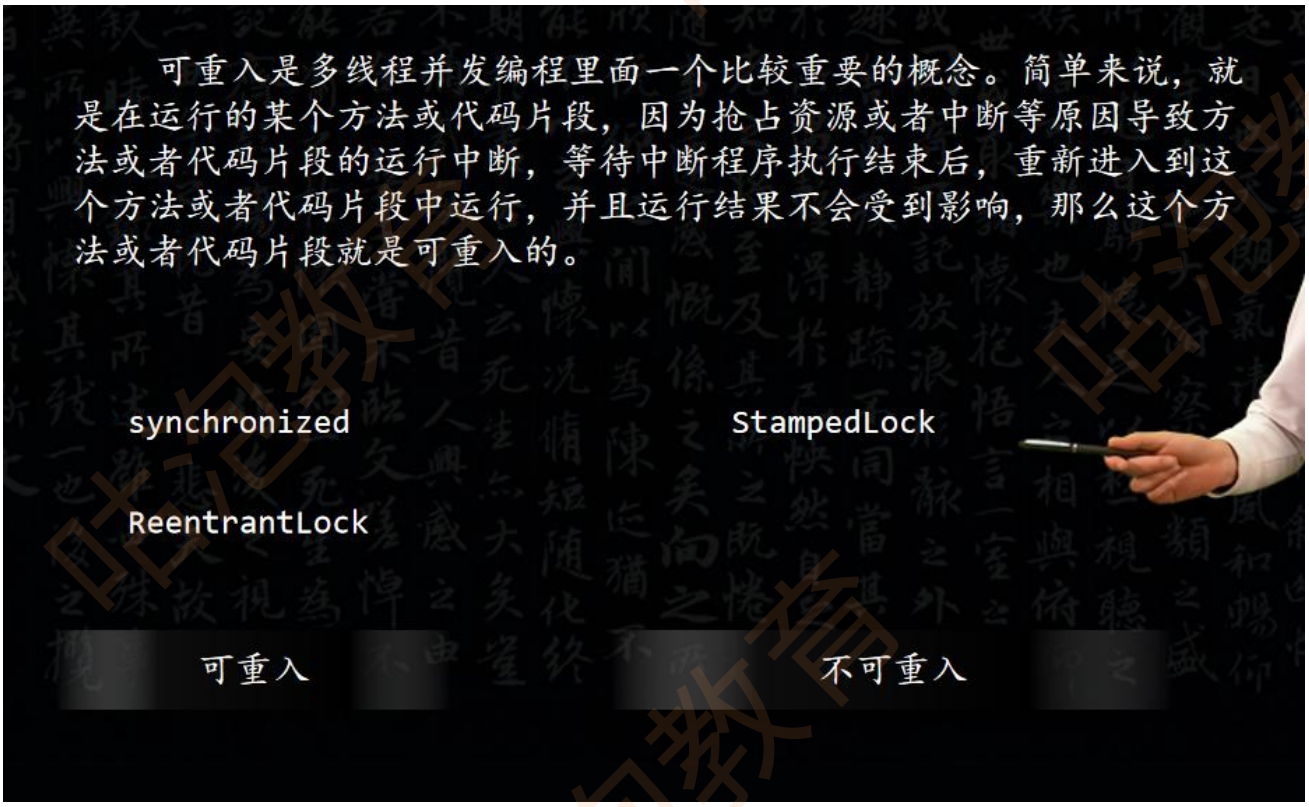
（转场，屏幕黑底白字，正中央出现“2 官方定义”）

1、可重入锁的官方定义



可重入是多线程并发编程里面一个比较重要的概念。简单来说，就是在运行的某个方法或代码片段，因为抢占资源或者中断等原因导致方法或者代码片段的运行中断，等待中断程序执行结束后，重新进入到这个方法或者代码片段中运行，并且运行结果不会受到影响，那么这个方法或者代码片段就是可重入的。

而可重入锁，简单来说就是一个线程如果抢占到了互斥锁资源，在锁释放之前再去竞争同一把锁的时候，不需要等待，只需要记录重入次数。



在多线程并发编程里面，绝大部分锁都是可重入的，比如 synchronized、ReentrantLock 等，但是也有不支持重入的锁，比如 JDK8 里面提供的读写锁StampedLock。

【导航条：应用场景】

（转场，屏幕黑底白字，正中央出现“2 应用场景”）

2、可重入锁的应用场景

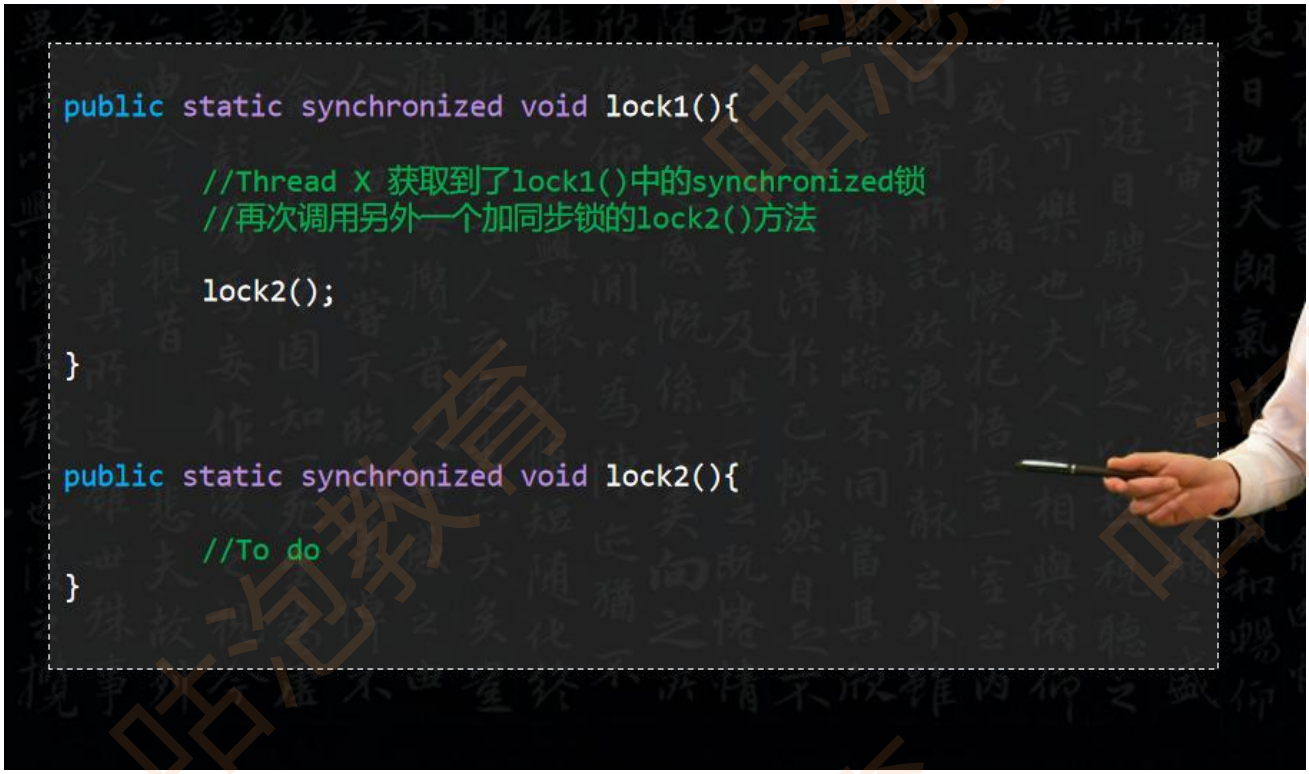
来看这么一段代码：

```
public static synchronized void lock1(){
```

```
    //Thread X 获取到了 lock1()中的 synchronized 锁
    //再次调用另外一个加同步锁的 lock2()方法

    lock2();

}


public static synchronized void lock2(){

    //To do
}
```

咕泡教育

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果



锁的可重入性，主要解决的问题是避免线程死锁的问题。因为一个已经获得同步锁 X 的线程，在释放锁 X 之前再去竞争锁 X 的时候，相当于会出现自己要等待自己释放锁，这很显然是无法成立的。

【结尾】

关于这个问题，其实是考察应聘者的基础知识。互联网大厂对基础的考察会特别深，各位想进互联网的小伙伴还是需要在工作之外去多花一点时间研究。对于 3 年工作经验的小伙伴，也应该要开始关注这一类问题了。

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

64.谈谈你对ReentrantLock 实现原理的理解?

【导航条：引言】

一个3 年工作经验的小伙伴，在面试的时候被这样一个问题。” 谈谈你对ReentrantLock实现原理的理解 “，他当时零零散散的说了一些。但好像没有说关键点。希望我分享一下我的理解。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

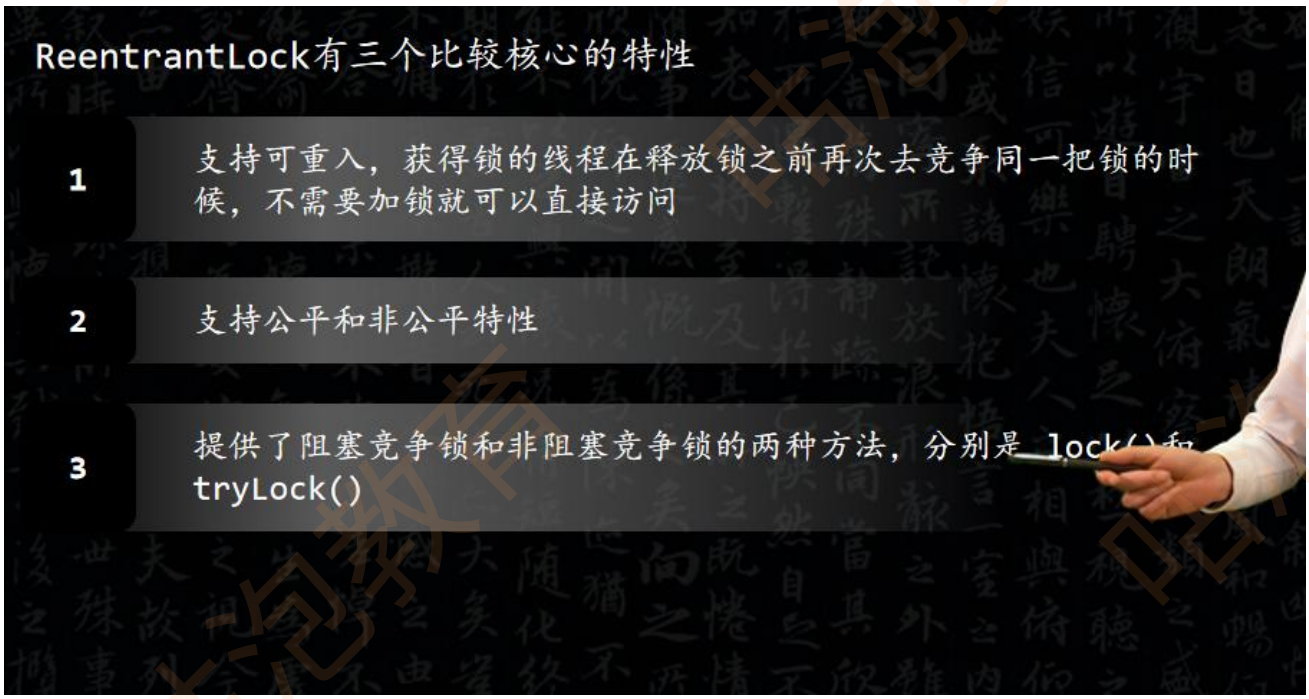
【导航条：官方定义】

（转场，屏幕黑底白字，正中央出现“1 官方定义”）

1、什么是 ReentrantLock

ReentrantLock 是一种可重入的排它锁，主要用来解决多线程对共享资源竞争的问题。

它有四个比较核心的特性：



第1，它支持可重入，也就是获得锁的线程在释放锁之前再次去竞争同一把锁的时候，不需要加锁就可以直接访问。

第2，它支持公平和非公平特性

第3，它提供了阻塞竞争锁和非阻塞竞争锁的两种方法，分别是 lock()和 tryLock()。

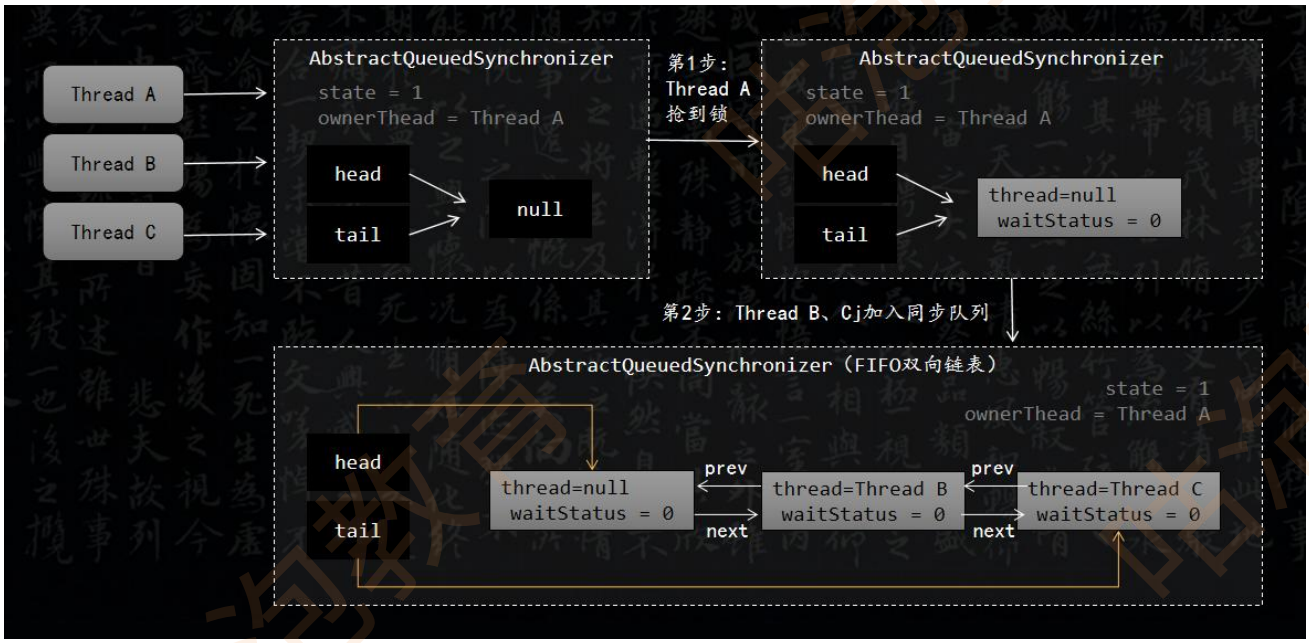
【导航条：实现原理】

（转场，屏幕黑底白字，正中央出现“2 实现原理”）

2、ReentrantLock 的实现原理

ReentrantLock 的底层实现有几个非常关键的技术。

第1个，锁的竞争，ReentrantLock 是通过互斥变量，使用 CAS 机制来实现的。



没有竞争到锁的线程，使用了 AbstractQueuedSynchronizer 这样一个队列同步器来存储，底层是通过双向链表来实现的。当锁被释放之后，会从 AQS 队列里面的头部唤醒下一个等待锁的线程。

第2个，公平和非公平的特性，主要是体现在竞争锁的时候，是否需要判断 AQS 队列存在等待中的线程。

第3个，锁的重入特性，在 AQS 里面有一个成员变量来保存当前获得锁的线程，当同一个线程下次再来竞争锁的时候，就不会去走锁竞争的逻辑，而是直接增加重入次数。

【结尾】

要回答好这样一道面试题，必须要理解 ReentrantLock 的设计思想，虽然很基础，但很重要。地基的深度决定了楼层的高度。

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能

够以此来提高各位粉丝的通过率。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

65.存储MD5值应该用VARCHAR还是用CHAR?

【导航条：引言】

一个5年工作经验的小伙伴，在面试的时候被这样一个问题。说”存储MD5值应该用VARCHAR还是用CHAR“，他一时间不只如何选择，感觉用VARCHAR也可以，用CHAR也行。希望我来帮忙分析一下。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

【导航条：两者区别】

（转场，屏幕黑底白字，正中央出现“1 两者区别”）

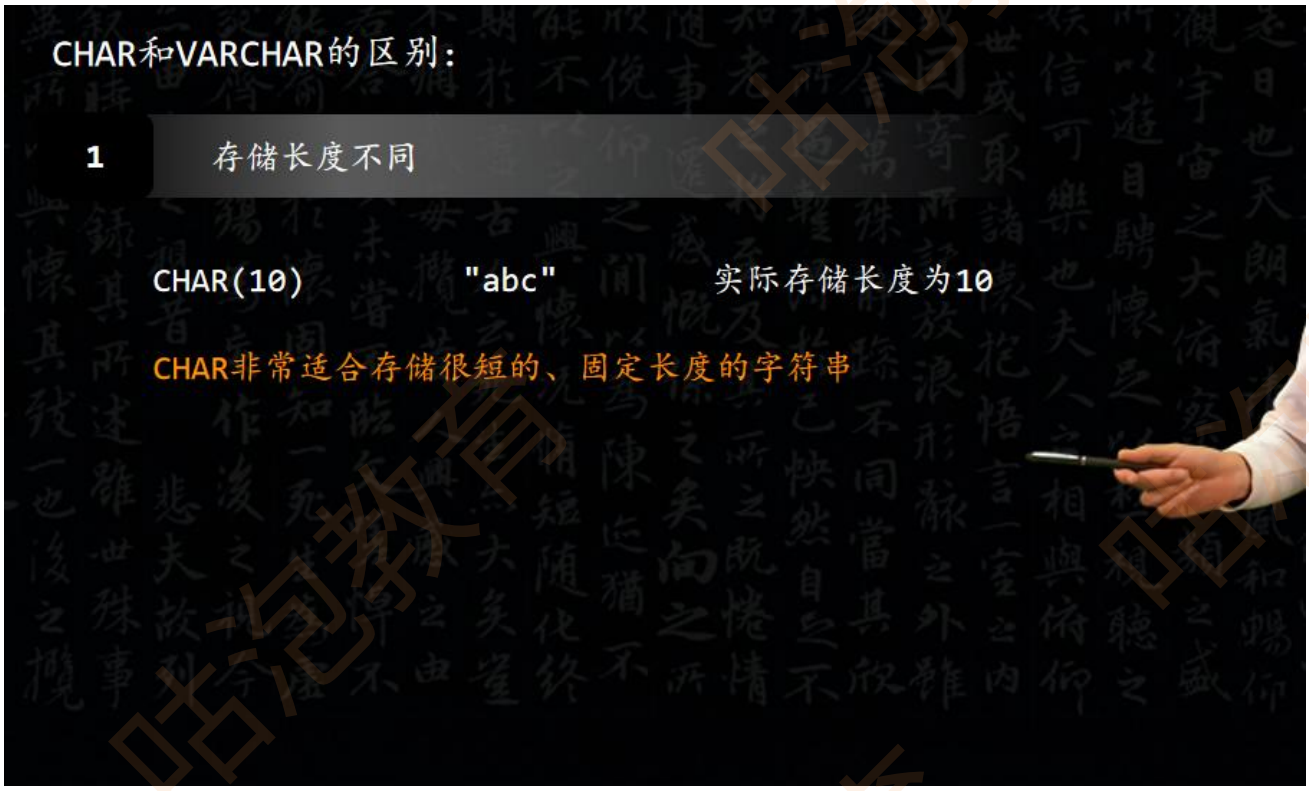
1、两者区别

要回答好这个问题，得了解清楚CHAR和VARCHAR的区别：

第1，存储长度不同

CHAR类型的长度是固定的，VARCHAR的长度是可变的。

假设，当我们当定义CHAR(10)，输入的值是"abc"，但是它占用的空间一样是10个字节，会包含7个空字节。当输入的字符长度超过指定的数时，CHAR会截取超出的字符。而且，当存储为CHAR的时候，MySQL会自动删除输入字符串末尾的空格。

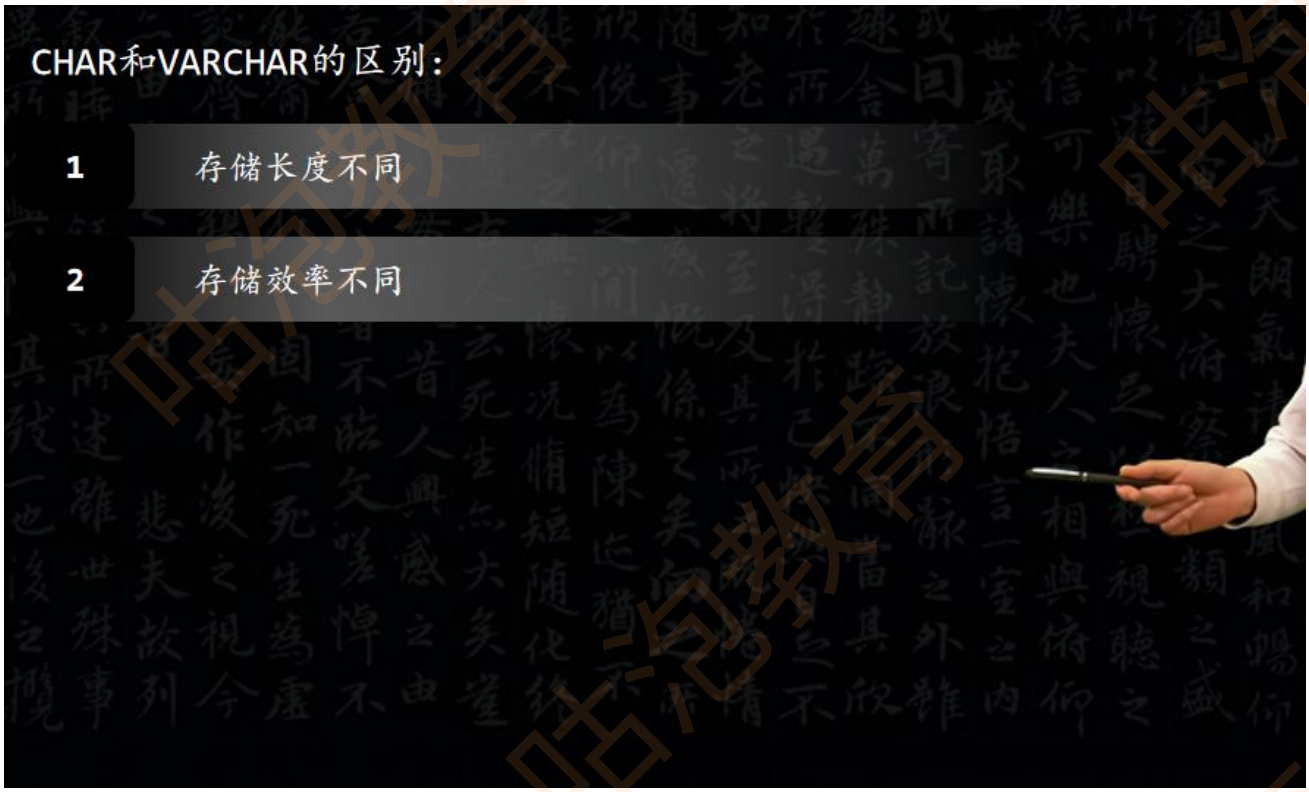


所以，CHAR非常适合存储很短的、固定长度的字符串。

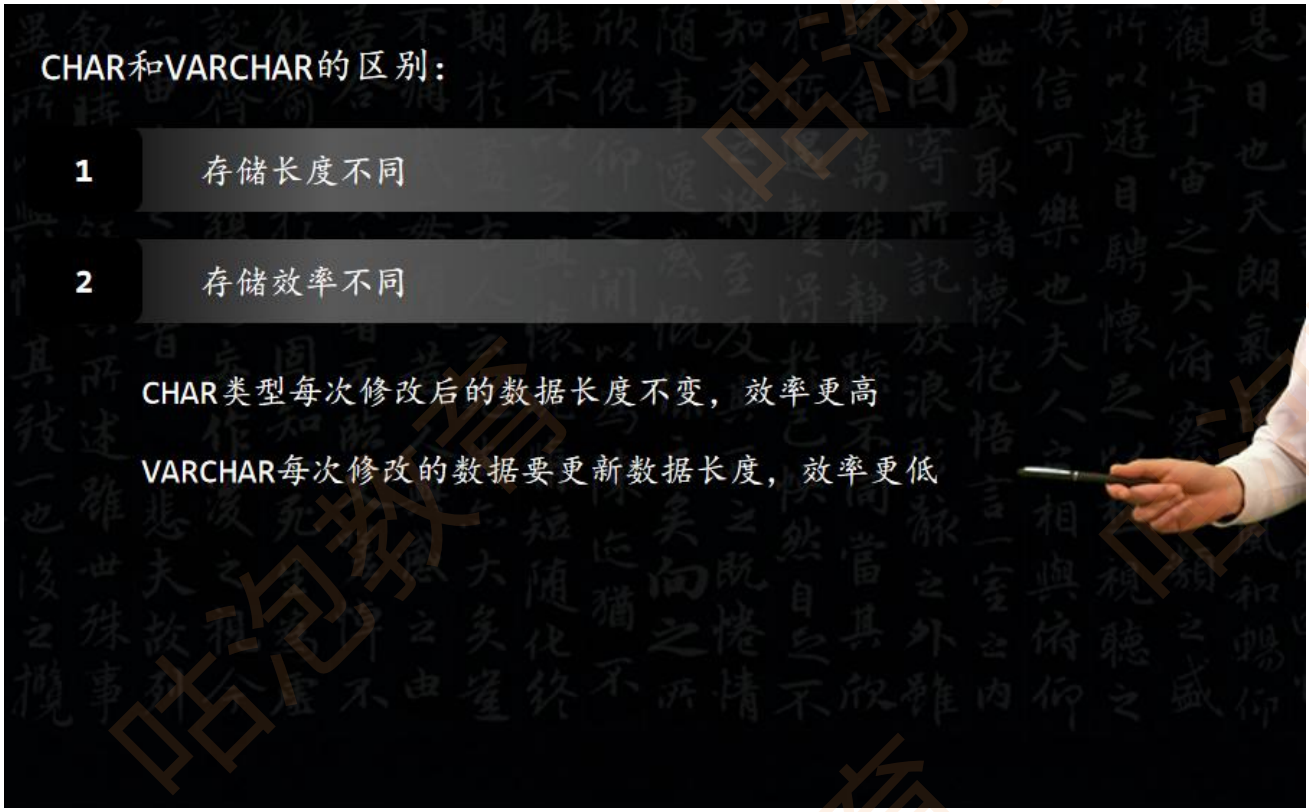
VARCHAR类型用于存储可变长度且非Unicode的字符数据。但可变长度必须在1和8000之间的数值。

比如VARCHAR(10)，然后输入abc三个字符，那么实际存储大小为3个字节。

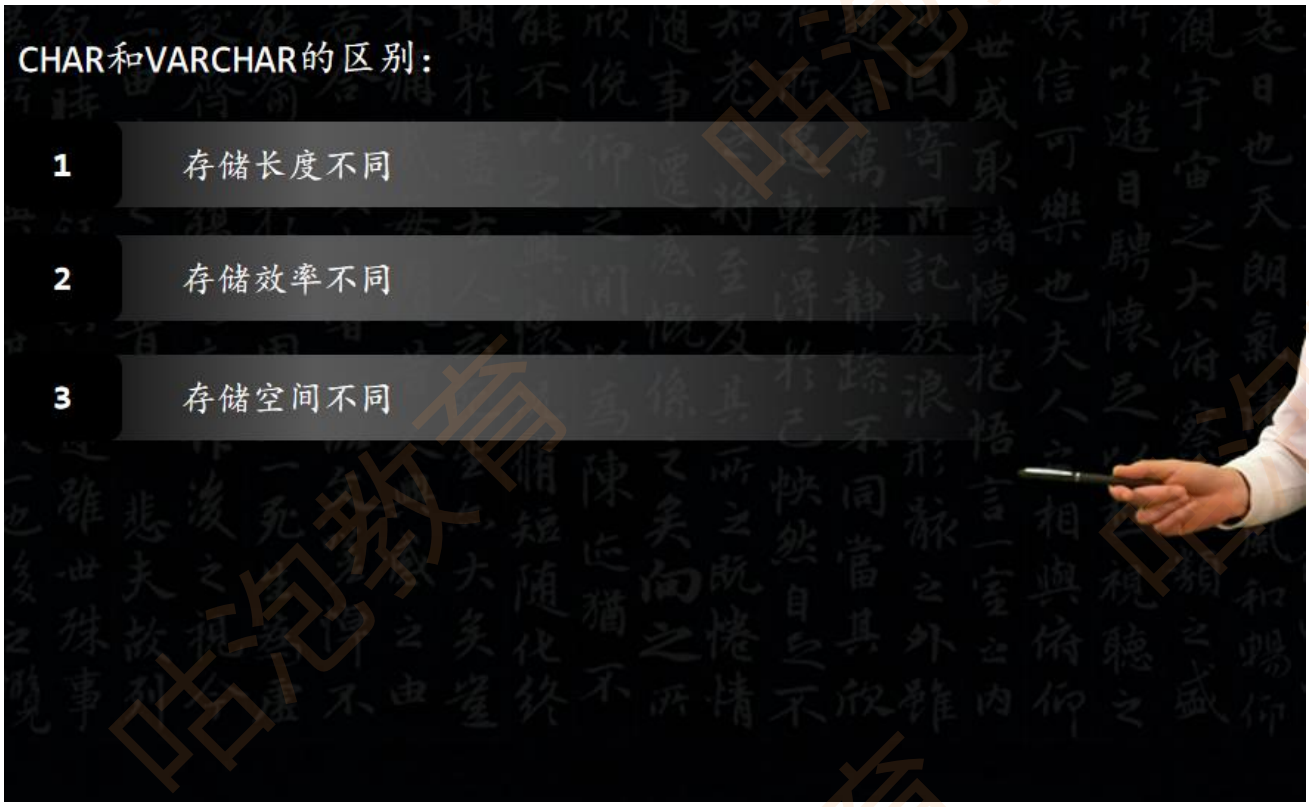
除此之外，VARCHAR还会保留1个或2个额外的字节来记录字符串的实际长度。如果定义的最大长度小于等于255个字节，那么，就会预留1个字节；如果定义的最大长度大于255个字节，那么就会预留2个字节。



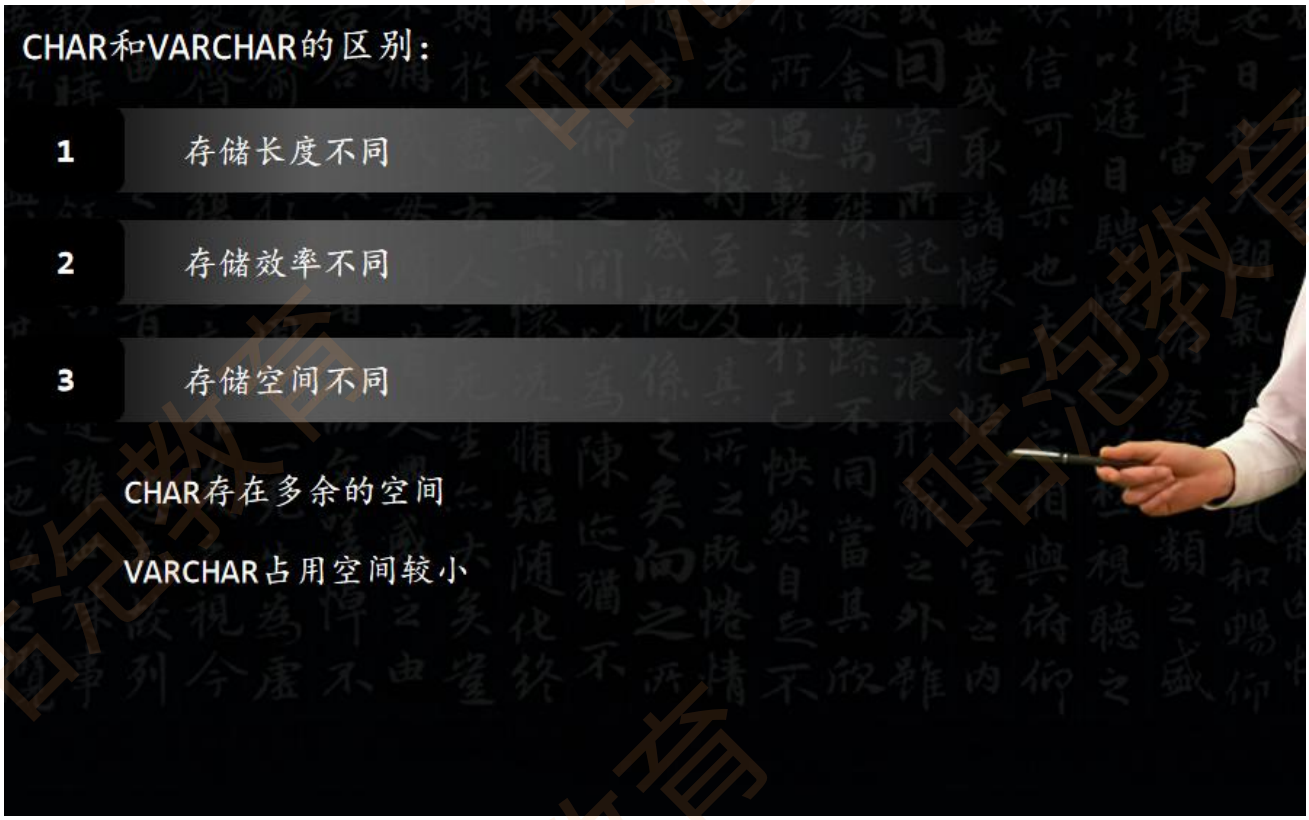
第2，存储效率不同



CHAR类型每次修改后的数据长度不变，效率更高。而VARCHAR每次修改的数据要更新数据长度，效率更低。


CHAR和VARCHAR的区别：
1
存储长度不同
2
存储效率不同
3
存储空间不同

第3，存储空间不同


CHAR和VARCHAR的区别：
1
存储长度不同
2
存储效率不同
3
存储空间不同
CHAR存在多余的空间
VARCHAR占用空间较小

CHAR存储空间是初始的预计长度字符串再加上一个记录字符串长度的字节，可能会存在多余的空间。而VARCHAR存储空间的时候是实际字符串再加上一个记录字符串长度的字节，占用空间较小。

【导航条：答案分析】

（转场，屏幕黑底白字，正中央出现“2 答案分析”）

2、答案分析

根据以上的分析，CHAR非常适合存储MD5值。因为MD5是一个定长的值，对于固定长度的非常短的列，CHAR比VARCHAR效率也更高。

【结尾】

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

66.谈谈你对RabbitMQ工作原理的理解

【导航条：引言】

一个5年工作经验的小伙伴，在面试的时候被这样一个问题。谈谈你对RabbitMQ架构原理的理解。当时，这位小伙伴只解答说，我只会用，原理并没有关注过。那今天我给大家来分享一下我的理解。

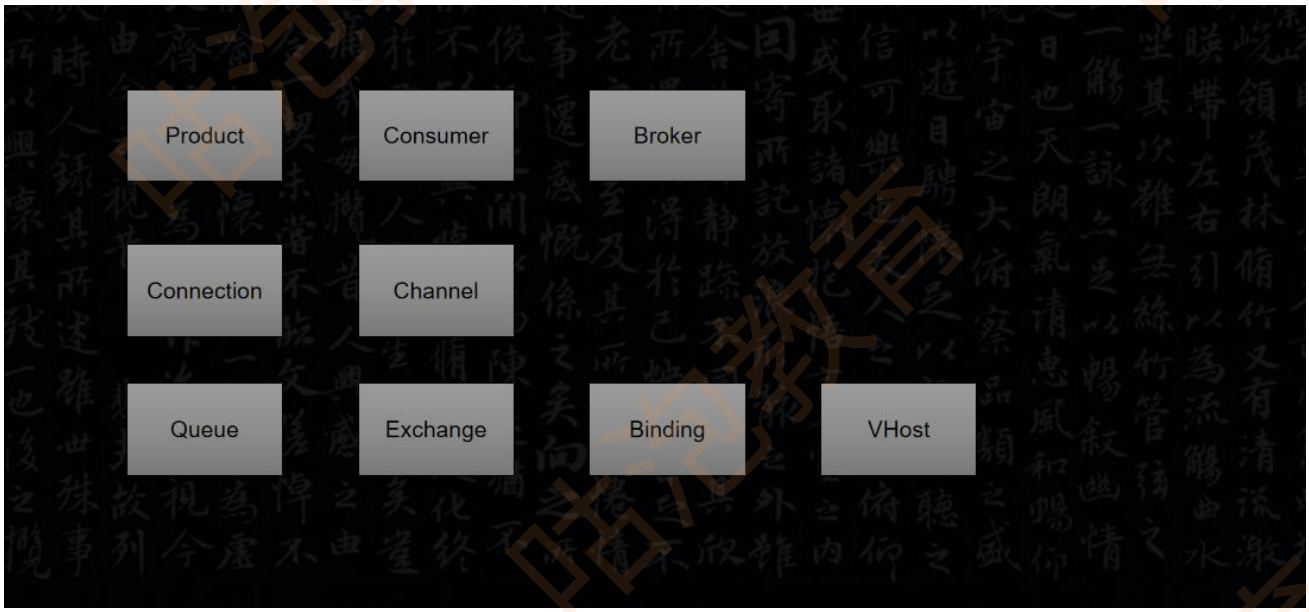
另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

【导航条：核心组件】

（转场，屏幕黑底白字，正中央出现“1 核心组件”）

1、核心组件

要回答这个问题呢，首先要知道RabbitMQ架构中几个核心组件，



它们分别是Product、Cunsumer、Broker、Connection、Channel、Queue、Exchange、Binding、VHost。

【导航条：工作原理】

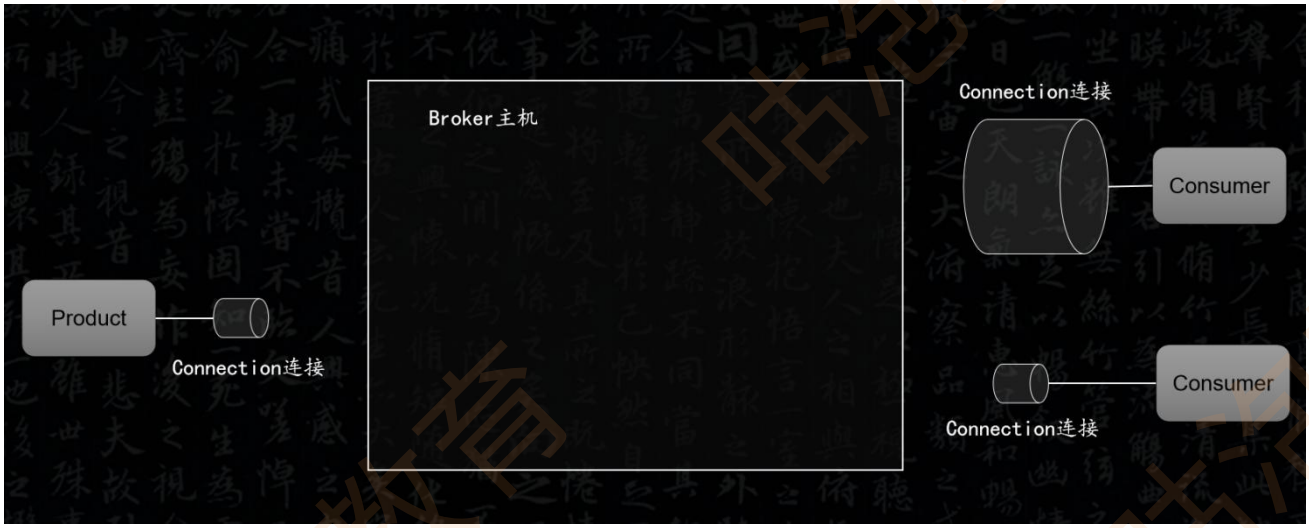
（转场，屏幕黑底白字，正中央出现“1 工作原理”）

2、工作原理

那么，这些组件之间是如何协调工作的呢？下面来看这么一个动画：

首先是，第1个和第2个，Product和Consumer分别是消息的生产者和消息的消费者，这是所有的消息中间件都有两个角色，我就不做过多介绍了。

重点来看，第3个叫Broker，中文翻译过来叫做代理或者中介，我可以把Broker理解为就是安装RabbitMQ的服务器，就像是一栋用来存储和转发消息的房子。



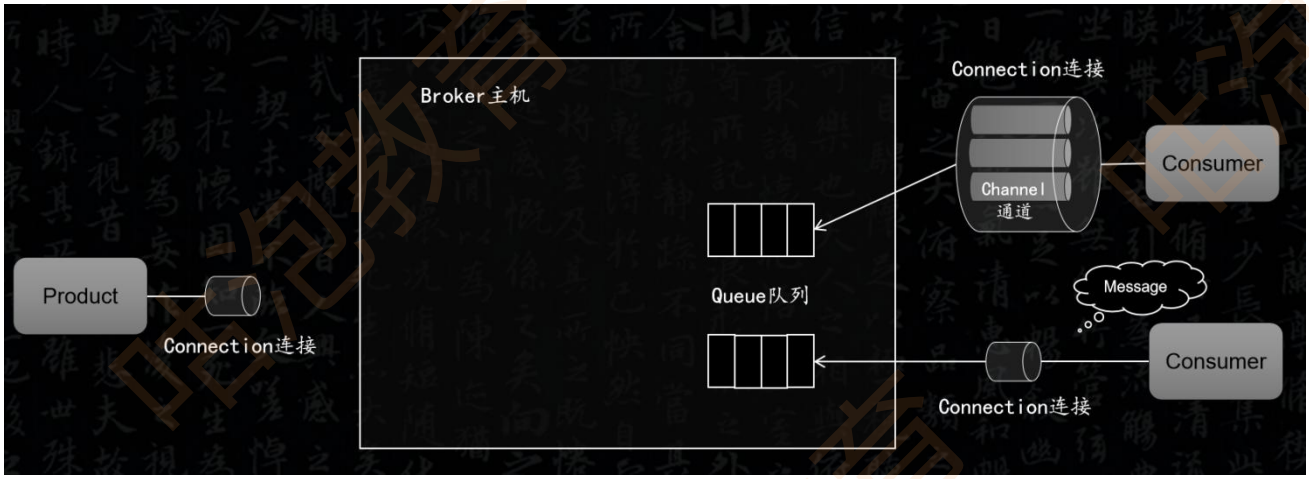
第4个就是，Connection连接。无论是生产者发送消息，还是消费者接收消息，都必须要跟Broker之间建立一个连接，那Connection呢就相当于是一个TCP的长连接。

第5个Channel通道。如果所有的生产者发送消息和消费者接收消息，每次都创建和释放TCP长连接的话，对于Broker来说肯定会造成很大的性能损耗，也会浪费时间。

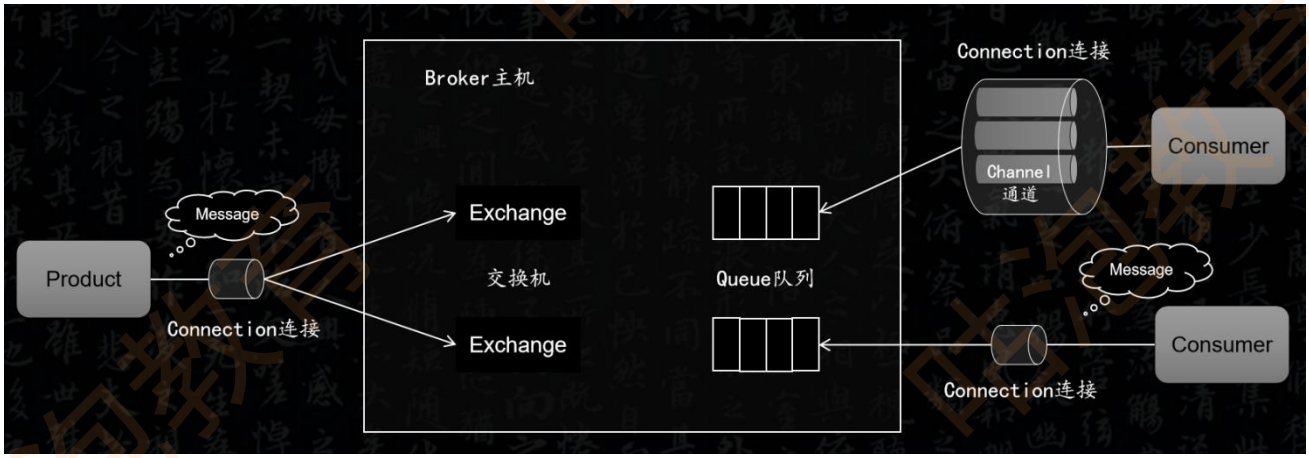
所以在AMQP协议里面引入了Channel的概念，它相当于是一个虚拟的连接。这样我们就可以在已经连接好的TCP长连接里面去创建和释放Channel，大大了减少了资源消耗。

不同的Channel是相互隔离的，每个Channel都有自己的编号。对于每个客户端线程来说，Channel就没必要共享了，各自用自己的Channel。

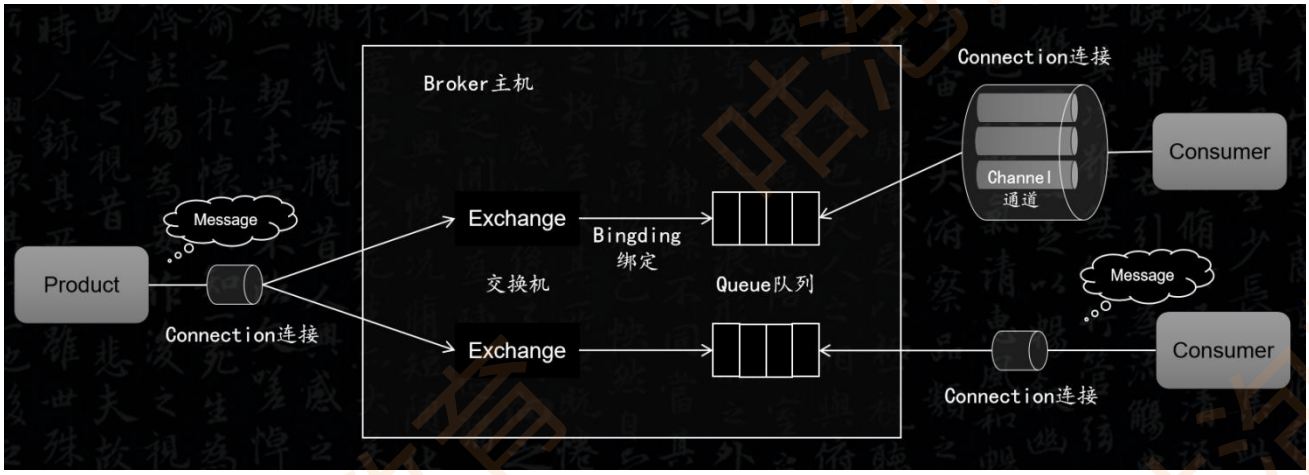
另外一个需要注意的是，Channel是RabbitMQ原生API里面的最重要的编程接口，也就是说我们定义交换机、队列、绑定关系，发送消息，消费消息，调用的都是Channel接口上的方法。



第6个，Queue队列。它专门用来存储消息，Queue也是生产者和消费者的纽带，生产者发送的消息会存储到队列中，而消费者也是从队列中来消费消息。



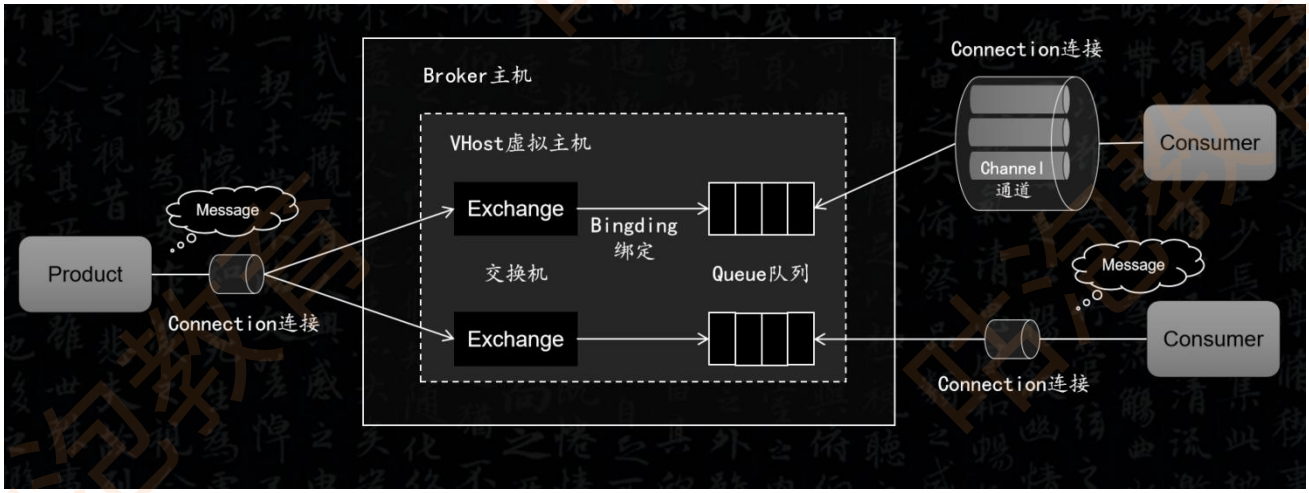
第7个，就是Exchange交换机。相当于消息的路由器。Exchange不会存储消息，它只做一件事情，根据规则分发消息。



第8个，就是Binding绑定。Exchange和存储消息的队列必须建立一个绑定关系，并且为每个队列指定一个特殊的标识。

Exchange和队列是多对多的绑定关系，也就说，一个交换机的消息一个路由给多个队列，一个队列也可以接收来自多个交换机的消息。

绑定关系建立好之后，生产者发送消息到Exchange，也会携带一个特殊的标识。当这个标识跟绑定的标识匹配的时候，消息就会发给一个或者多个符合规则的队列。



最后一个就是Vhost，全称叫Virtual Host虚拟机。为了解决不同业务系统之间的消息隔离，节约硬件成本，

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

我们可以利用RabbitMQ的Vhost来实现资源的隔离和权限的控制。它的功能和其他编程语言中的NameSpace比较类似。

【结尾】

以上就是我对RabiitMQ的架构原理的理解，，小伙伴们，你是不是也理解得更加透彻了呢?

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难!

67.如何保证RabbitMQ的高可用

【导航条：引言】

一个8年工作经验的小伙伴，被问到这样一个问题，说如何保证RabbitMQ的高可用。关于这个问题呢，这位小伙伴倒是有个实操经验，就是不知道如何组织语言。所以，当时面试结果不太理想。今天，我给大家分享一下我的理解。

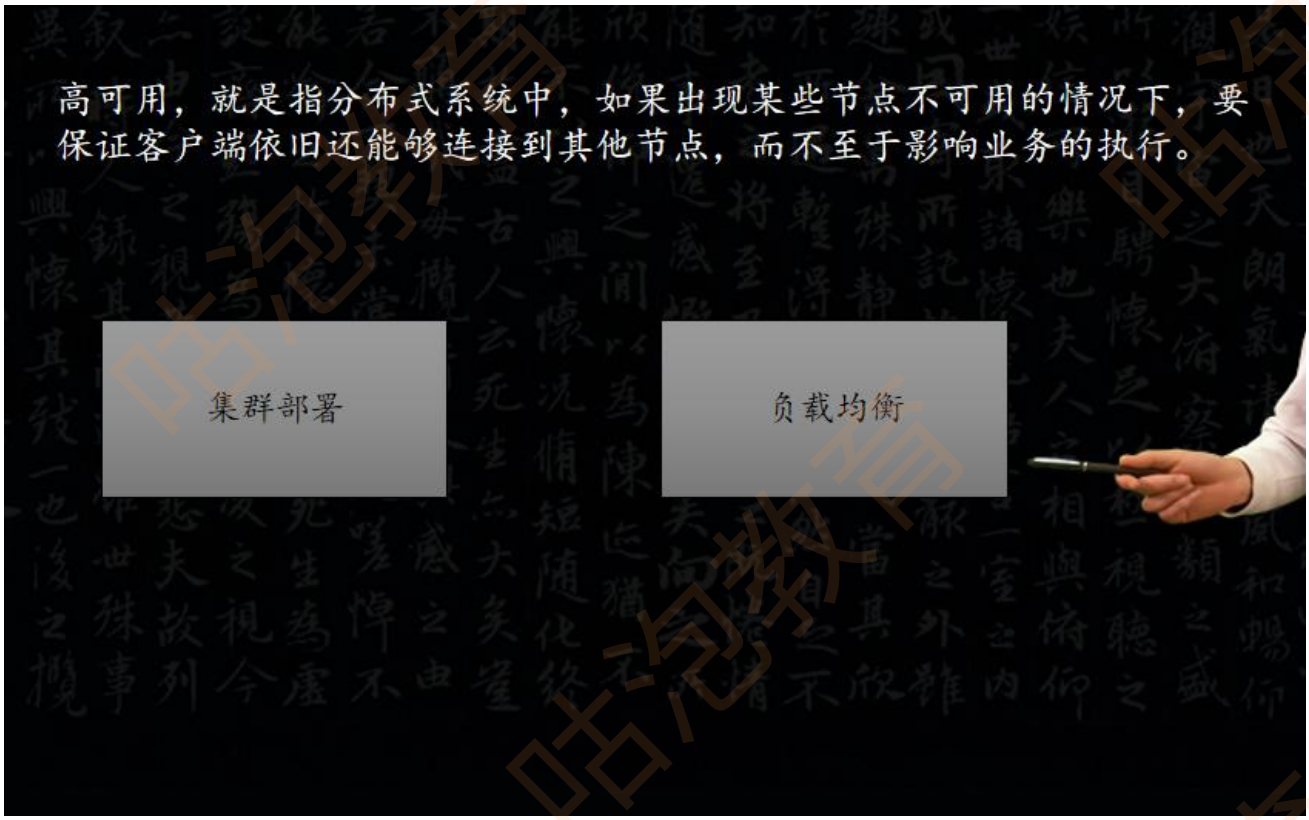
另外，我花了1个多星期，准备了一份10W字的面试题解析配套 l文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

【导航条：实现思路】

(转场，屏幕黑底白字，正中央出现“1 实现思路”)

1、实现思路

所谓高可用呢，就是指分布式系统中，如果出现某些节点不可用的情况下，要保证客户端依旧还能够连接到其他节点，而不至于影响业务的执行。我主要从以下两个方面来实现高可用，一个是集群部署，一个是负载均衡。



RabbitMQ是用Erlang语言编写的，而Erlang天生具备分布式的特性，所以不需要通过引入ZK来实现服务协调。它是使用.erlang.cookie来验证身份，它通过25672端口来实现集群。这个cookie就像暗号一样，只要喊出清楚明白666，就知道是Tom的粉丝。

另外，RabbitMQ的集群节点有两种，一种是磁盘节点（Disc Node），一种是内存节点（RAM Node）。在集群环境中至少需要一个磁盘节点用来持久化元数据，否则，当全部内存节点崩溃时，就无从同步元数据。

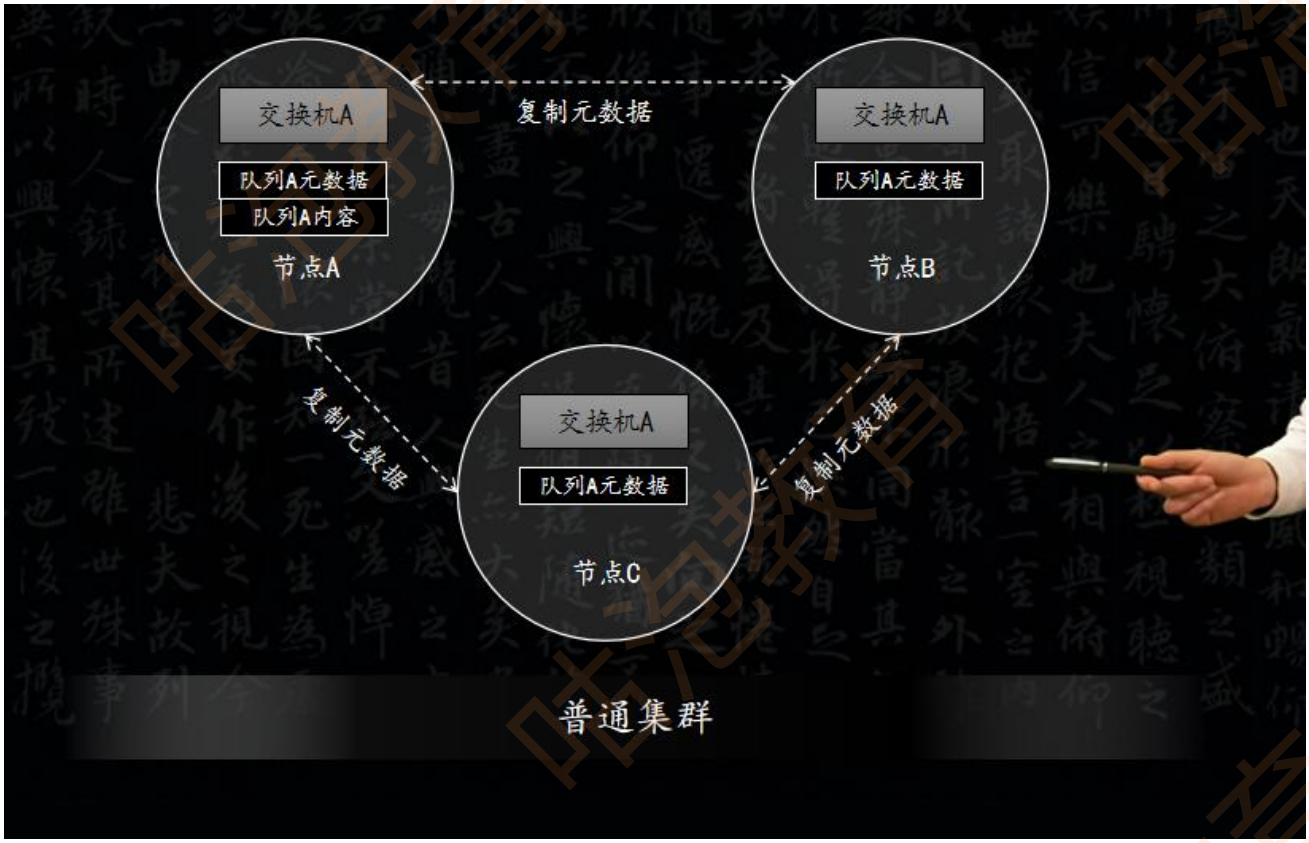
下面，分享一下RabbitMQ的集群模式，分别是普通集群模式和镜像队列模式。先来看普通集群模式。

【导航条：普通集群】

（转场，屏幕黑底白字，正中央出现“2 普通集群”）

2、普通集群

普通集群模式下，不同的节点之间只会相互同步元数据，而不会同步消息内容。



如图所示，队列A的消息只存储在A节点上。B节点和C节点同步了队列A的元数据，但是没有同步队列中的内容。

假如生产者连接的是C节点，要将消息通过交换机A路由到队列A，最终消息还是会转发到A节点上存储，因为队列A存储在A节点上。

同理，如果消费者连接是B节点，同样也要从队列A上拉取消息，消息会从A节点转发到节点B。其它节点只是起到一个路由的作用。

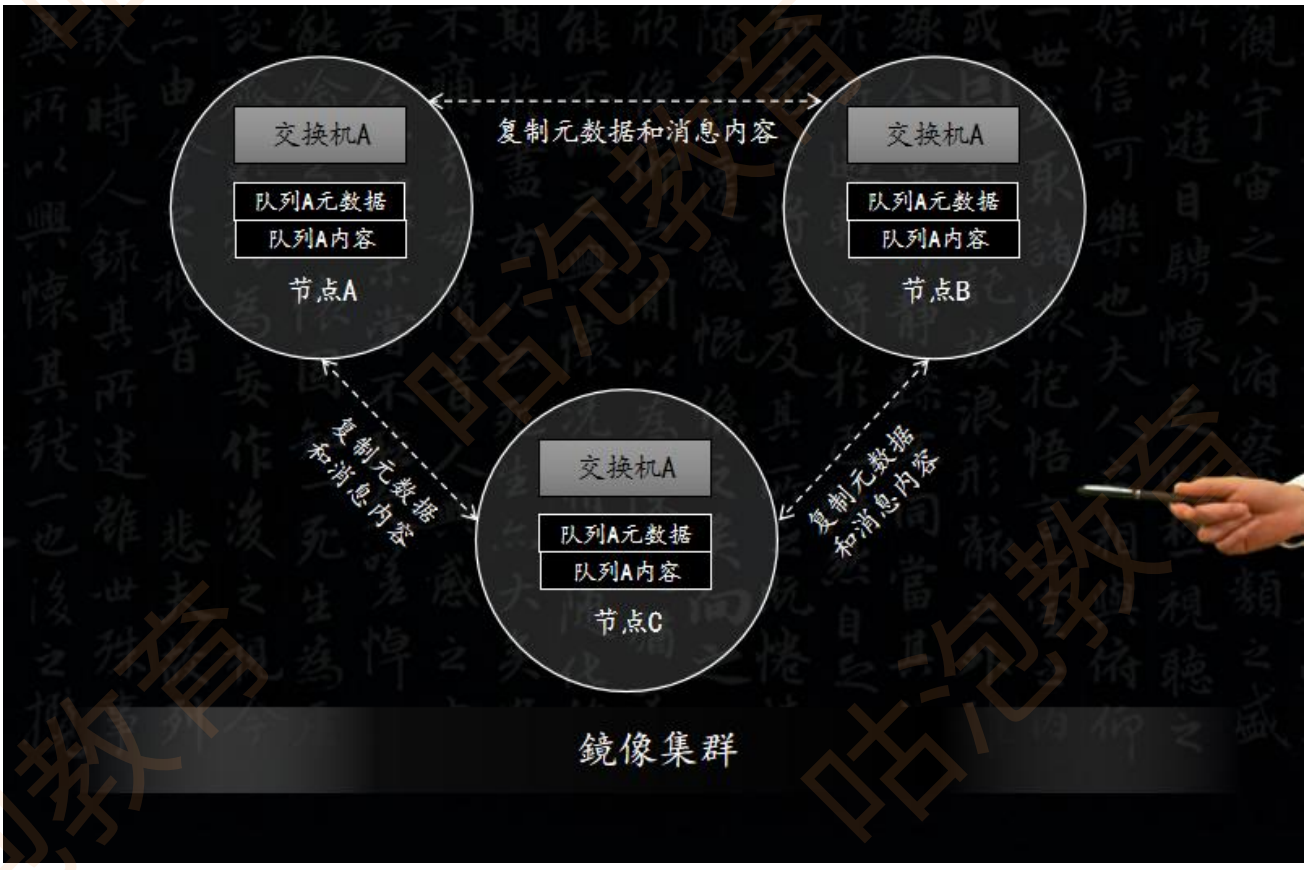
这样做的好处是，可以节约存储和同步数据的网络开销。但如果需要保证队列的高可用性，这时候，就需要用到镜像集群的模式。

【导航条：镜像集群】

（转场，屏幕黑底白字，正中央出现“2 镜像集群”）

3、镜像集群

下面来看镜像集群模式，如图所示，



其实也非常好理解，在镜像队列模式下，消息内容会在所有的镜像节点间同步，可用性更高。不过也有一定的副作用，系统性能会降低，节点过多的情况下同步数据的代价会比较大。

【导航条：负载均衡】

（转场，屏幕黑底白字，正中央出现“4 负载均衡”）

4、负载均衡

集群搭建成功后，要保证高可用，还需要一个负载均衡的组件（例如HAProxy，LVS，Nignx），由负载的组件来做路由。对于，生产者或者消费者而言，只需要连接到负载组件的虚拟IP地址就可以了。

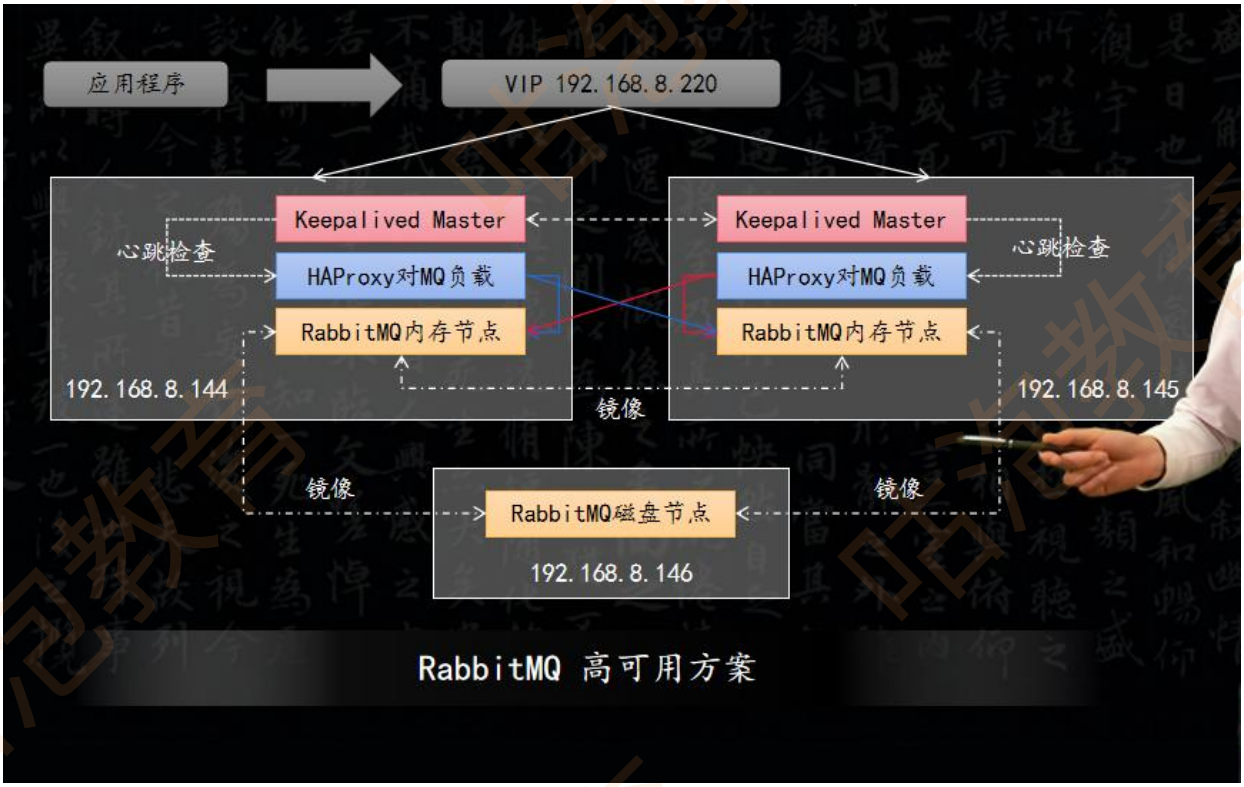
这个负载均衡组件需要满足以下几个条件：

1、它本身有负载功能，可以监控集群中节点的状态，如果某个节点出现异常或者发生故障，就把它剔除掉。

2、为了提高可用性，要能够支持部署多个服务，能够自主完成Master选举

3、Master节点需要对外提供一个虚拟IP，客户端只需要连接到这一个虚拟IP就可以自动完成真实IP的路由。

下面我以HAProxy加Keepalived组合，来实现RabbitMQ的高可用，具体架构如图所示：



当然，实现高可用的方案还有很多，比如我们还可以用LVS，Nignx等等来实现负载均衡。

【结尾】

关于RabbitMQ的高可用方案我就分享这么。我分享的这个方案，不仅仅只适用于RabbitMQ，其实还可以应用于大部分的分布式高可用场景。小伙伴们如果你还有其他更加牛B的方案，可以在评论区告诉我相互学习一下。

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

68.Kafka如何避免消息重复消费?

【导航条：引言】

一个6年工作经验的小伙伴，被问到这样一个问题，说Kafka是如何避免消息重复消费的？面试完之后，这位小伙伴来找到我，希望我能给一个思路。今天，我给大家分享一下我的思路。

另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

要回答好这个问题，首先要分析一下，导致消息重复消费的原因有哪些。

【导航条：原因分析】

（转场，屏幕黑底白字，正中央出现“1 原因分析”）

1、原因分析

我认为，导致Kafka消息重复消费有以下两个原因：

第1个原因是：Kafka消费端重复提交导致消息重复消费。

如图所示，在Broker上存储的消息，都有一个Offset标记，用来记录消费者消费消息的位置。Kafka的消费者是通过offSet标记来维护当 前已经消费的数据，每消费一批数据，Broker就会更新offSet的值，避免重复消费。

而默认情况下，消息消费完以后，会自动提交Offset的值，避免重复消费。

但是Kafka消费端的自动提交，会有一个默认的5秒间隔，也就是说在5秒之后的下一次向Broker拉取消息的时候才提交上一批消费的offset。

所以在消费者消费的过程中，如果遇到应用程序被强制kill掉或者宕机的情况，可能会导致Offset没有及时提交，从而产生重复提交的问题。

第2个原因是：Kafka服务端的Partition再均衡机制导致消息重复消费。

如图所示，在Kafka中有一个Partition Balance机制，就是把多个Partition均衡的分配给多个消费者。消费端会从分配到的Partition里面去消费消息，如果消费者在默认的5分钟内没有处理完这一批消息。就会触发Kafka的Rebalance机制，从而导致offset自动提交失败。而Rebalance之后，消费者还是会从之前没提交的offset位置开始消费，从而导致消息重复消费。

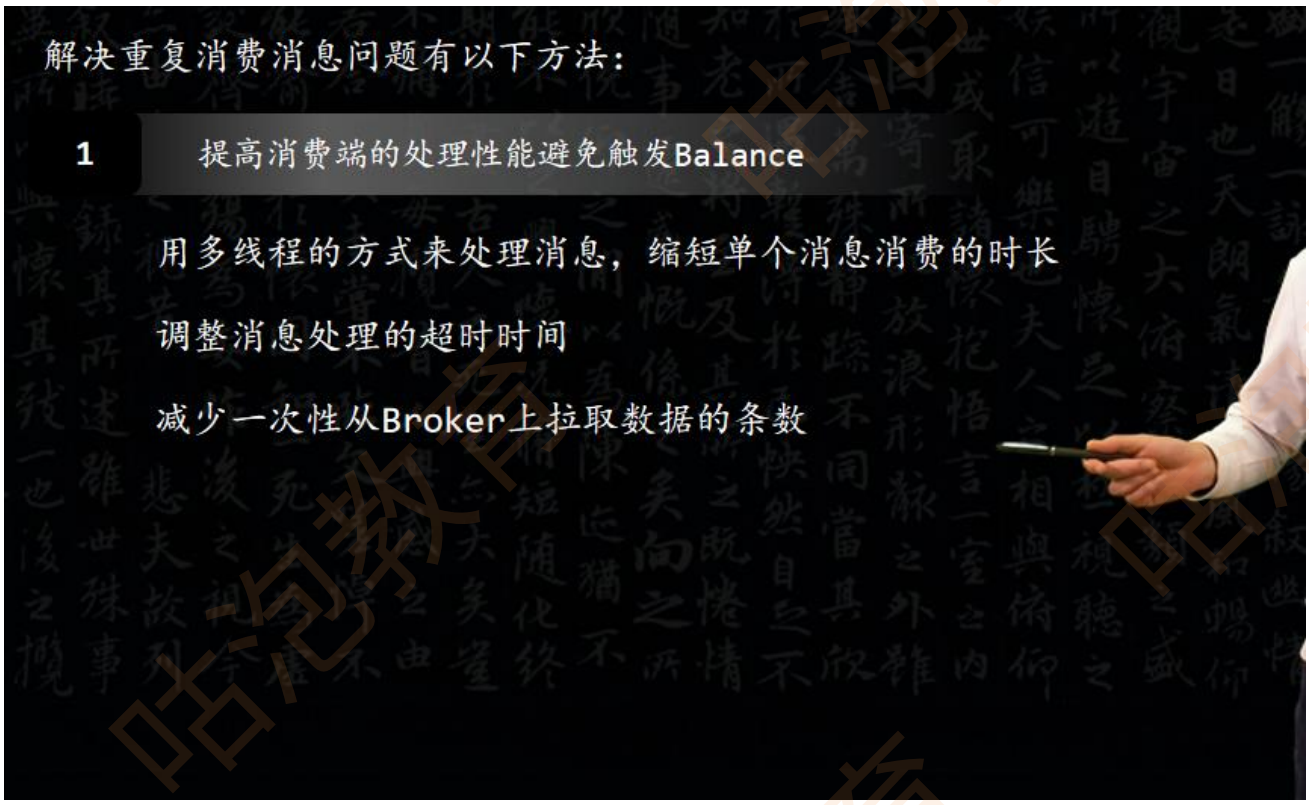
【导航条：解决方案】

（转场，屏幕黑底白字，正中央出现“2 解决方案”）

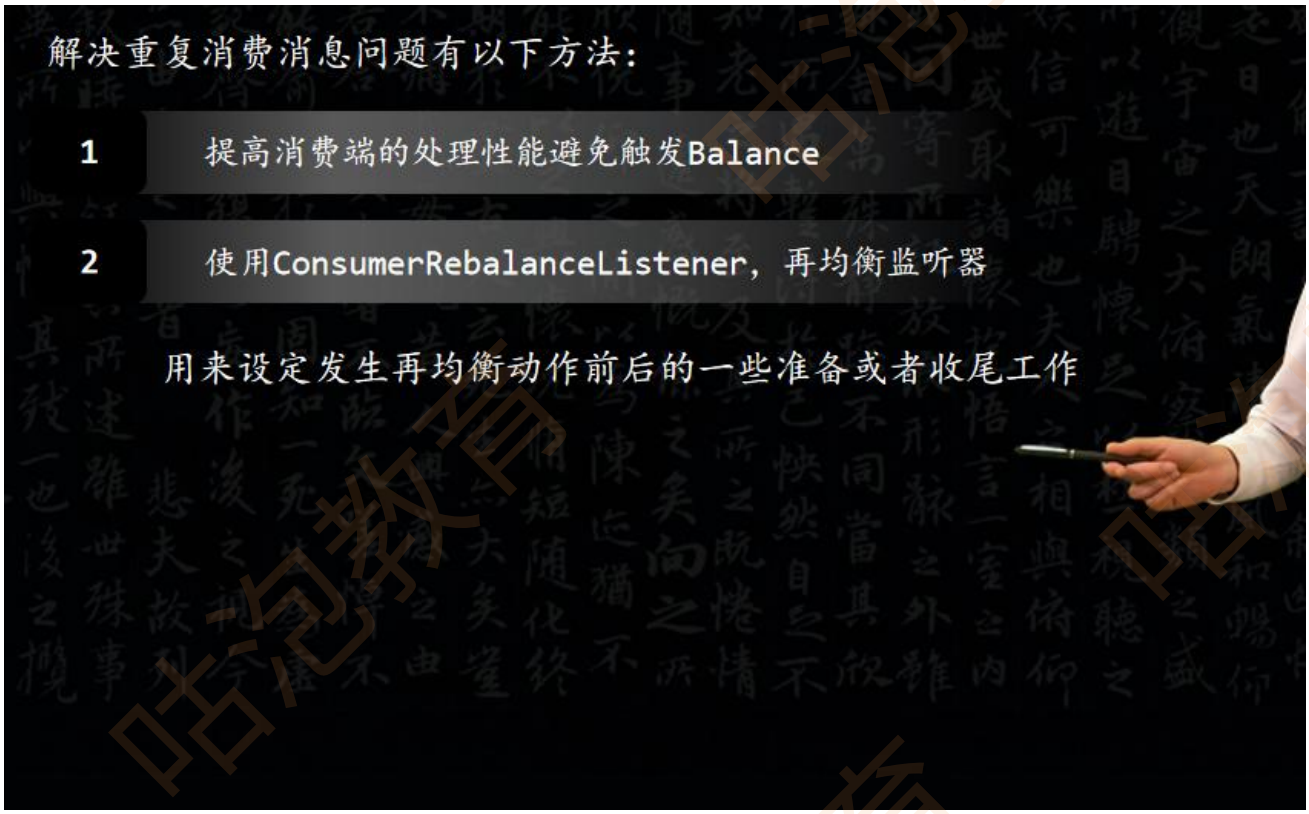
2、解决方案

基于对Kafka消息重复消费的原因分析，我认为可以通过以下两个方法来解决这个问题：

基于这样的背景下，我认为解决重复消费消息问题的方法有几个。



提高消费端的处理性能避免触发Balance，比如可以用多线程的方式来处理消息，缩短单个消息消费的时长。或者还可以调整消息处理的超时时间，也还可以减少一次性从Broker上拉取数据的条数。



使用ConsumerRebalanceListener，再均衡监听器，它可以用来设定发生再均衡动作前后的一些准备或者收尾工作。

开启Kafka的冥等性功能prop.put(ProducerConfig.ENABLE_IDEMPOTENCE_CONFIG,true);

或者将消息生成md5然后保存到MySQL或者Redis中，在处理消息之前先查MySQL或者Redis，进行判断看是否已经消费过。

【结尾】

以上就是我对Kafka避免消息重复消费的解决思路，小伙伴如果更好的思路也可以在评论区讨论一下。

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

69.基于Redis和MySQL的架构，如何保证数据一致性

【导航条：引言】

今天分享一道一线大厂公司高频面试题。“基于Redis和MySQL的架构，如何保证数据一致性”。这个问题难倒了不少工作5年以上的程序员，难的不是问题本身，而是解决这个问题的思路。

【导航条：背景介绍】

（转场，屏幕黑底白字，正中央出现“2 背景介绍”）

1、背景介绍

一般情况下，Redis是用作应用程序和数据库之间读操作的缓存，主要目的是减少数据库IO，还可以提升数

据的IO性能。如图所示，这是Redis加MySQL的整体架构设计。

当应用程序需要去读取某个数据的时候，首先会先尝试去Redis中加载，如果命中就直接返回。如果没有命中，就从数据库查询，查询到数据后再把这个数据缓存到Redis里面。

在这样一个架构中，会出现一个问题，就是一份数据，同时保存在数据库和Redis中，当数据发生变化的时候，需要同时更新Redis和MySQL，由于更新是有先后顺序的，这这种两边写入的环境下，并不能像单纯数据库的操作一样，可以满足ACID特性。因此，就有可能出现一方更新失败，一方更新成功的情况，从而出现数据一致性问题。

【导航条：解决思路】

（转场，屏幕黑底白字，正中央出现“2 解决思路”）

2、解决思路

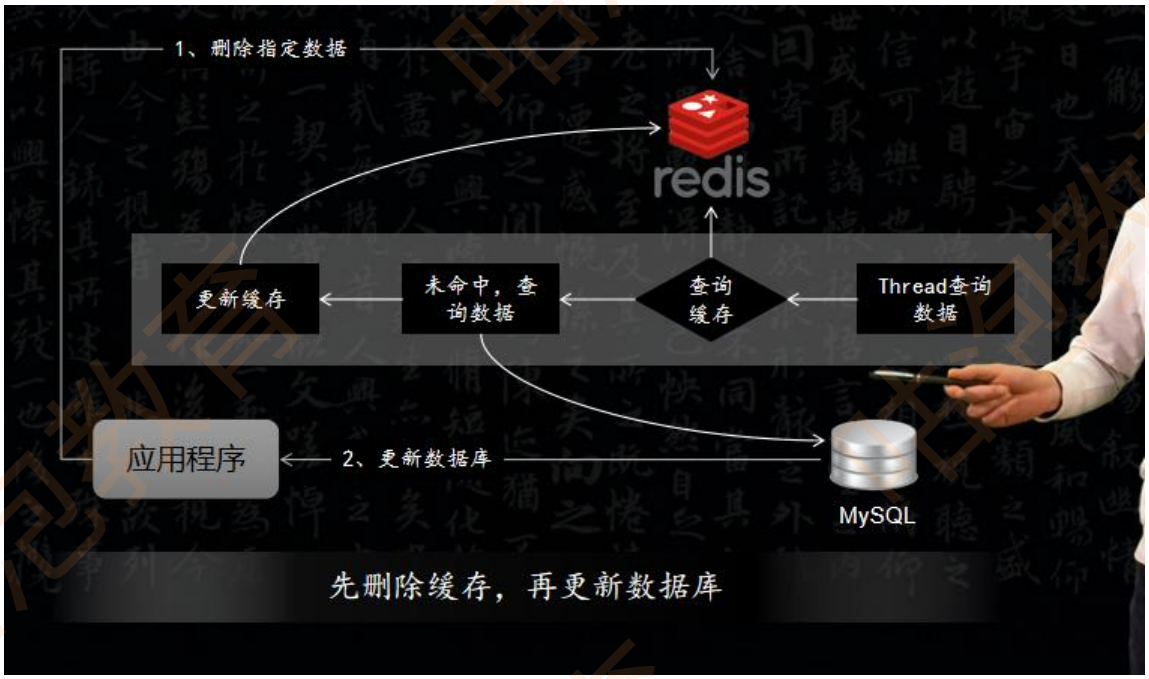
如果出现数据一致性问题，我们该如何解决呢？一般会想到以下两种解决思路。

要么先更新数据库，再更新缓存；

要么先删除缓存，再更新数据库。

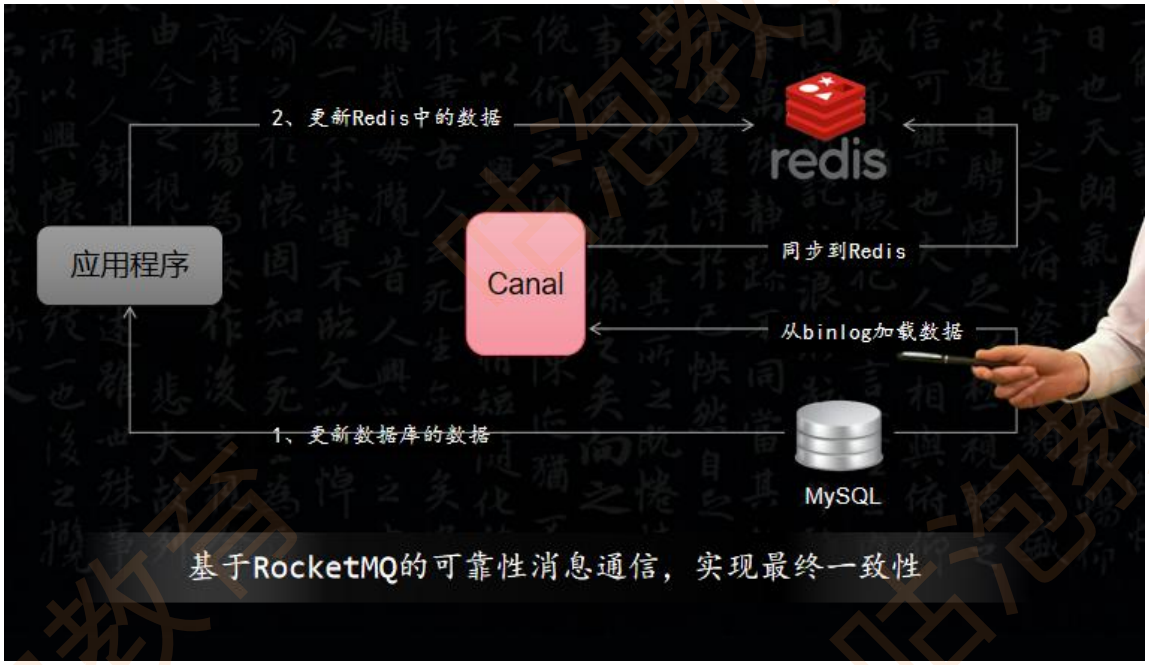
如果是采用先更新数据库，再更新缓存的方案，也会有这样一个问题。假设缓存更新失败，就会导致数据库和Redis中的数据不一致。

那如果是先删除缓存，再更新数据库，理想情况是应用下次访问Redis的时候，发现Redis里面的数据是空的，就从数据库加载保存到Redis里面，那么数据是一致的。但是在极端情况下，并不能保证删除Redis和更新数据库这两个操作的原子性，所以这个过程如果有其他线程来访问，还是会存在数据不一致问题。



所以，如果需要在极端情况下仍然保证Redis和MySQL的数据一致性，就只能采用最终一致性方案。

如图所示，比如基于RocketMQ的可靠性消息通信，来实现最终一致性。



再比如，还可以直接通过Canal组件，来监控MySQL中Binlog的日志，把更新后的数据同步到Redis中。

因为这里是基于最终一致性来实现的，如果业务场景不能接受数据的短期不一致性，那就不能使用这个方

案来做。

以上就是我对这个问题的理解。

【结尾】

我们在面试的时候，面试官还可能会问各种没有场景化的纯粹的技术问题，比如说：“你这个最终一致性方案”还是会存在数据不一致的问题啊？那怎么解决？

先不用慌，技术是为业务服务的，所以不同的业务场景，对于技术的选择和方案的设计都是不同的，所以这个时候，可以反问面试官，具体的业务场景是什么？

大家一定要记住，某个技术方案不可能适用于所有的业务场景，只有最合适的方案，没有最优的方案。

最后，我把之前分享的视频全部整理成了文字，想获取的小伙伴可以从我的个人煮叶简介中找到，希望能够以此来提高各位粉丝的通过率。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

70.谈谈你对volatile 关键字作用和原理的理解

【导航条：引言】

一位6年工作经验的小伙伴，在某里二面的时候被问到“volatile”关键字。然后，就没有然后了...

同样，还有一位4年的小伙伴，去某团面试也被问到“volatile 关键字“。然后，也没有然后了...

volatile关键字是Java并发编程中的一个重要的关键字，这个问题确实问得比较底层了。难道大厂现在都这么卷了吗？那今天呢，我给大家分享一下我的理解，希望能够帮助到大家。

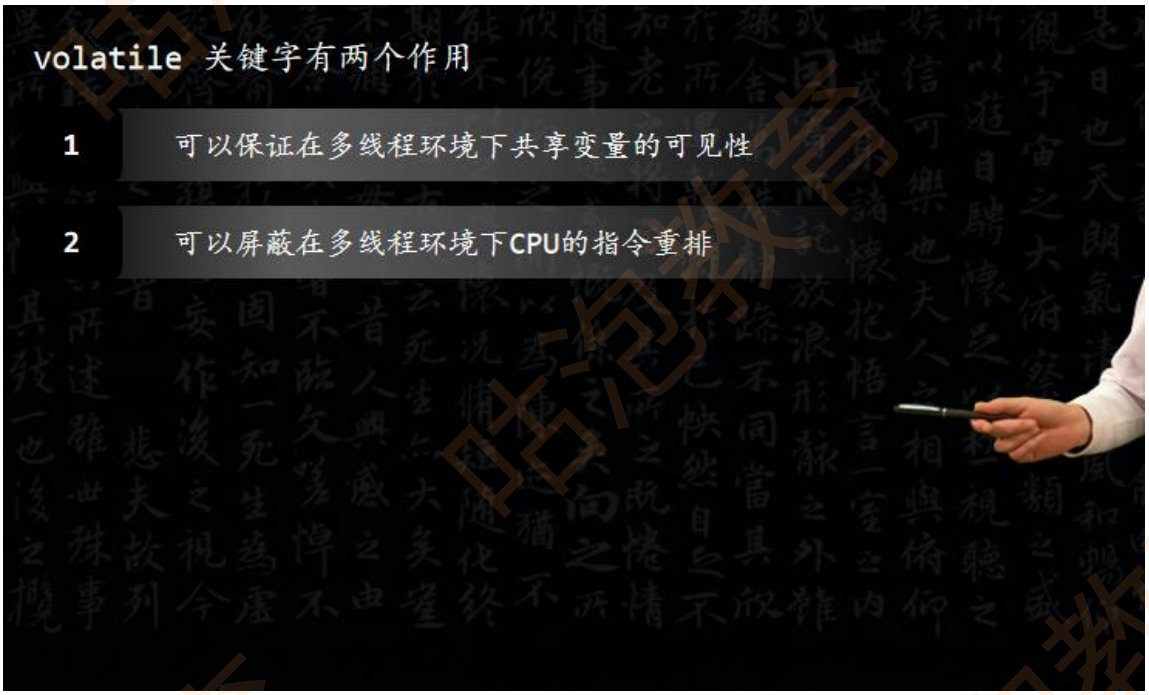
另外，我花了1个多星期，准备了一份10W字的面试题解析配套文档，想获取的小伙伴可以从我的个人煮叶简介中找到。

【导航条：volatile的作用】

（转场，屏幕黑底白字，正中央出现“1 volatile的作用”）

1、volatile的作用

volatile 关键字呢，有两个作用：



第1个，是可以保证在多线程环境下共享变量的可见性。

第2个，是可以屏蔽在多线程环境下CPU的指令重排。

下面，我给大家详细介绍一下可见性和屏蔽指令重排以及volatile的工作原理。

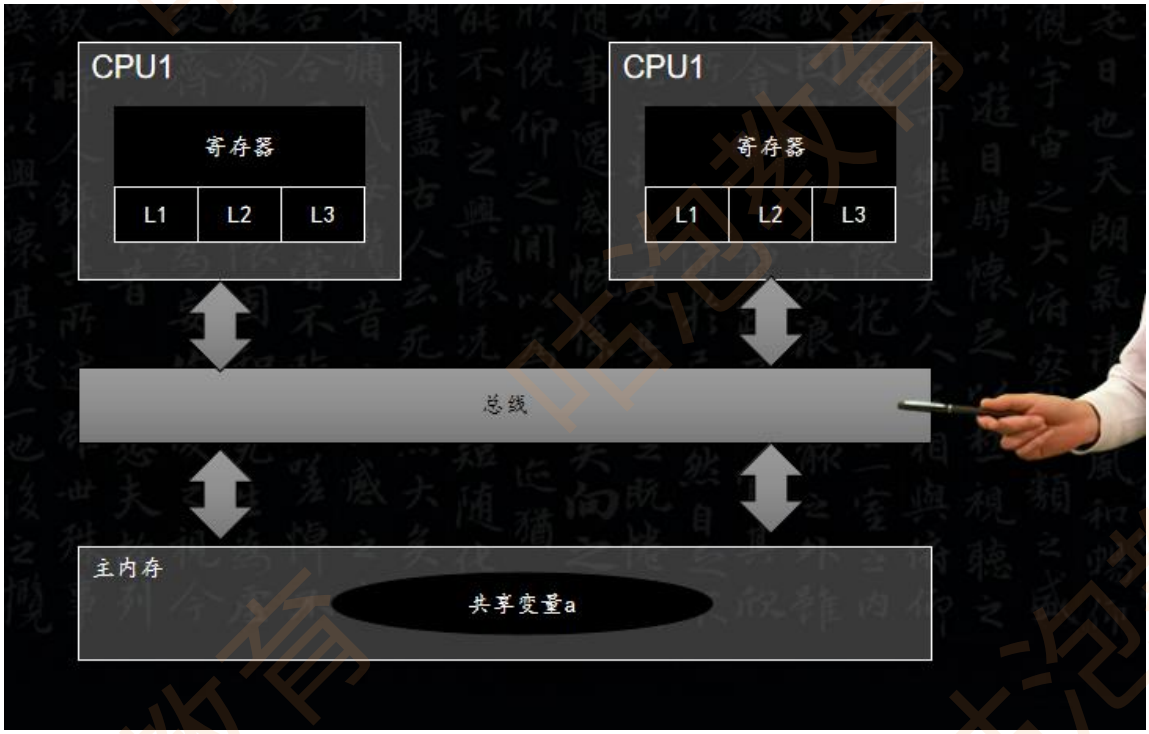
【导航条：原理分析】

（转场，屏幕黑底白字，正中央出现“2 原理分析”）

2、原理分析

先来看变量的可见性，简单来说，就是指当某一个线程对共享变量的修改，其他线程可以立刻看到修改之后的值。其实这个可见性问题，我认为本质上是由以下两个方面造成的。

首先是，CPU的高速缓存。在 CPU 里面设计了三级缓存去解决 CPU 运算效率和内存 IO 效率问题，但是有带来了缓存的一致性问题，而在多线程并行执行的情况下，缓存一致性就会导致可见性问题。



所以，对于增加了 volatile 关键字修饰的共享变量，JVM 虚拟机会自动增加一个#Lock 汇编指令，这个指令会根据 CPU 型号自动添加总线锁或/缓存锁。

我简单介绍一下这两种锁：

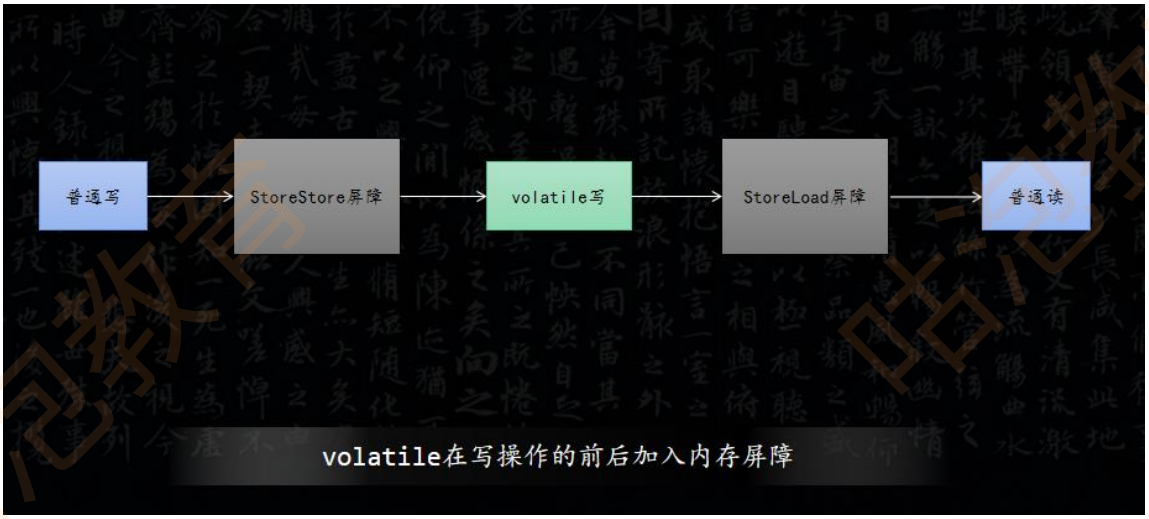
总线锁是锁定了 CPU 的前端总线，从而导致在同一时刻只能有一个线程去和内存通信，这样就避免了多线程并发造成的可见性。

缓存锁是对总线锁的优化，因为总线锁导致了 CPU 的使用效率大幅度下降，所以缓存锁只针对 CPU 三级缓存中的目标数据加锁，缓存锁是使用 MESI 缓存一致性来实现的。

然后，就是屏蔽指令重排，就是指屏蔽CPU指令重排序。意思是在多线程环境下，CPU指令的编写顺序和执行顺序不一致，从而导致可见性问题，为了提升 CPU 的利用率，CPU引入了StoreBuffer 机制，而这一种优化机制会导致 CPU 的乱序执行。当然为了避免这样的问题，CPU 提供了内存屏障指令，上层应用可以在合适的地方插入内存屏障来避免 CPU 指令重排序问题。而volatile就是通过设置内存屏障来禁止指令重排。

volatile内存屏障实现原理主要从以下两个方面来分析：

第1个是：volatile会在变量写操作的前后加入两个内存屏障，来保证前面的写指令和后面的读指令是有序的，如图所示：



学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

第2个是：volatile在变量的读操作后面插入两个指令，禁止后面的读指令和写指令重排序。，如图所示：

volatile其实可以看作是轻量级的synchronized，虽然说volatile不能保证原子性，但是如果在多线程下的操作本身就是原子性操作（例如赋值操作），那么使用volatile会优于synchronized。以上就是我对volatile 关键字的理解。

【结尾】

并发编程是每个程序员必须要掌握好的领域，它里面涵盖的设计思想、和并发问题的解决思路、以及作为一个并发工具，都是非常值得深度研究的。

最后，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

71.实现分布式锁，Zookeeper 和 Redis 哪种更好?

【导航条：引言】

一位6年工作经验的小伙伴，在某厂面试时被问到“实现分布式锁，Zookeeper 和 Redis 哪种更好？ “，这其实是一个开放性的问题。并没有标准答案。那今天呢，我给大家分享一下我的理解，希望能够帮助到大家。

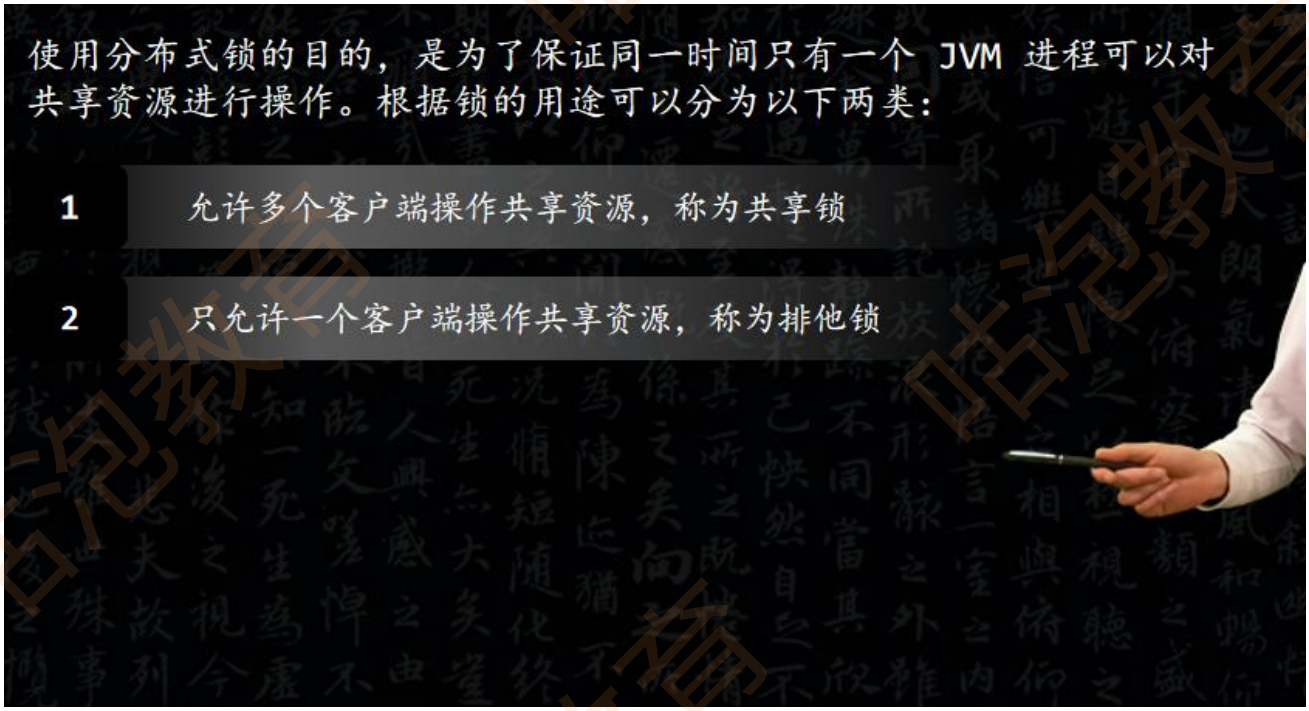
另外，我把往期分享的视频全部整理成了1份20字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：背景介绍】

（转场，屏幕黑底白字，正中央出现“1 为什么使用分布式锁？”）

1、背景介绍

使用分布式锁的目的，是为了保证同一时间只有一个 JVM 进程可以对共享资源进行操作。



根据锁的用途可以分为以下两类：

1、允许多个客户端操作共享资源，我们称为共享锁。

这种锁的一般是对共享资源具有幂等性操作的场景，主要是为了避免重复操作共享资源频繁加锁带来的性能开销。

2、只允许一个客户端操作共享资源，我们称为排他锁。这种锁一般是用在对共享资源操作具有非幂等性操作的场景，也就是需要保证在同一时刻只有一个进程或者线程能够访问这个共享资源。

【导航条：实现方案】

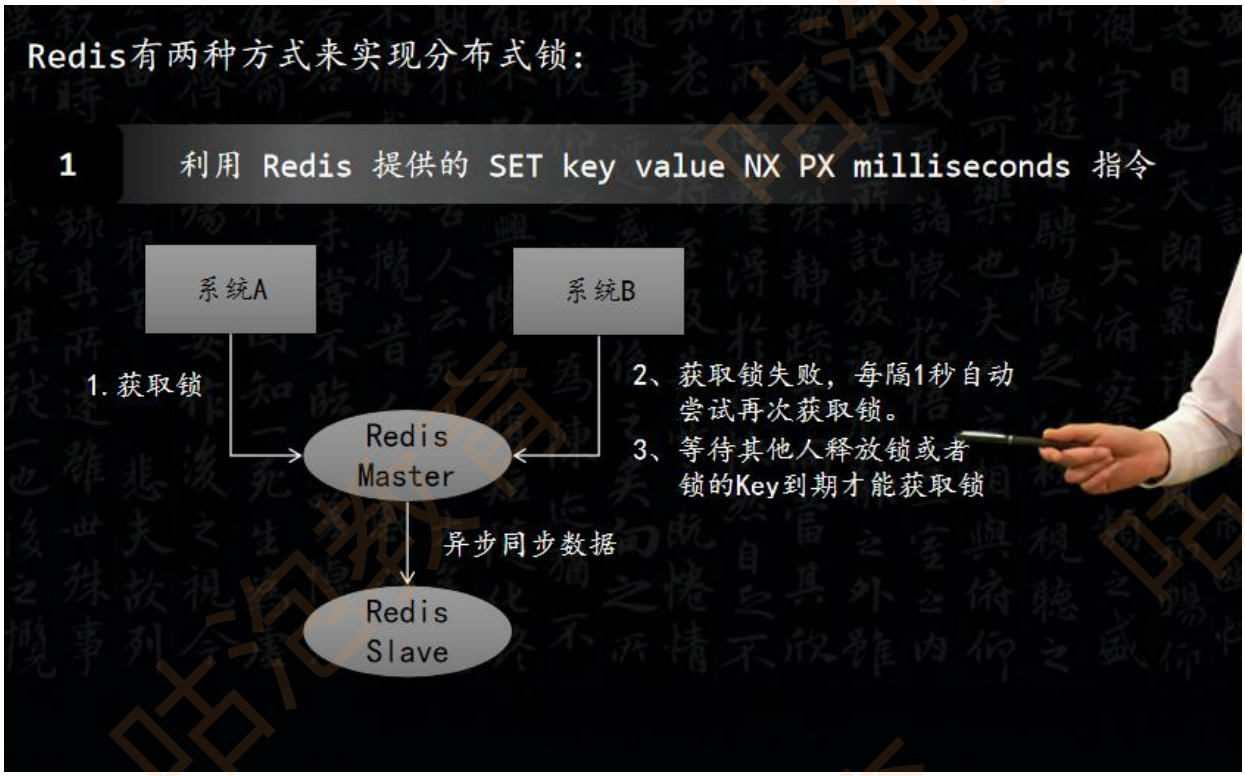
（转场，屏幕黑底白字，正中央出现“2 如何实现分布式锁？”）

2、实现方案

目前实现分布式锁最常用的中间件是 Redis 和 Zookeeper：

首先来看Redis的实现方案，它可以通过两种方式来实现

1. 利用 Redis 提供的 SET key value NX PX milliseconds 指令，



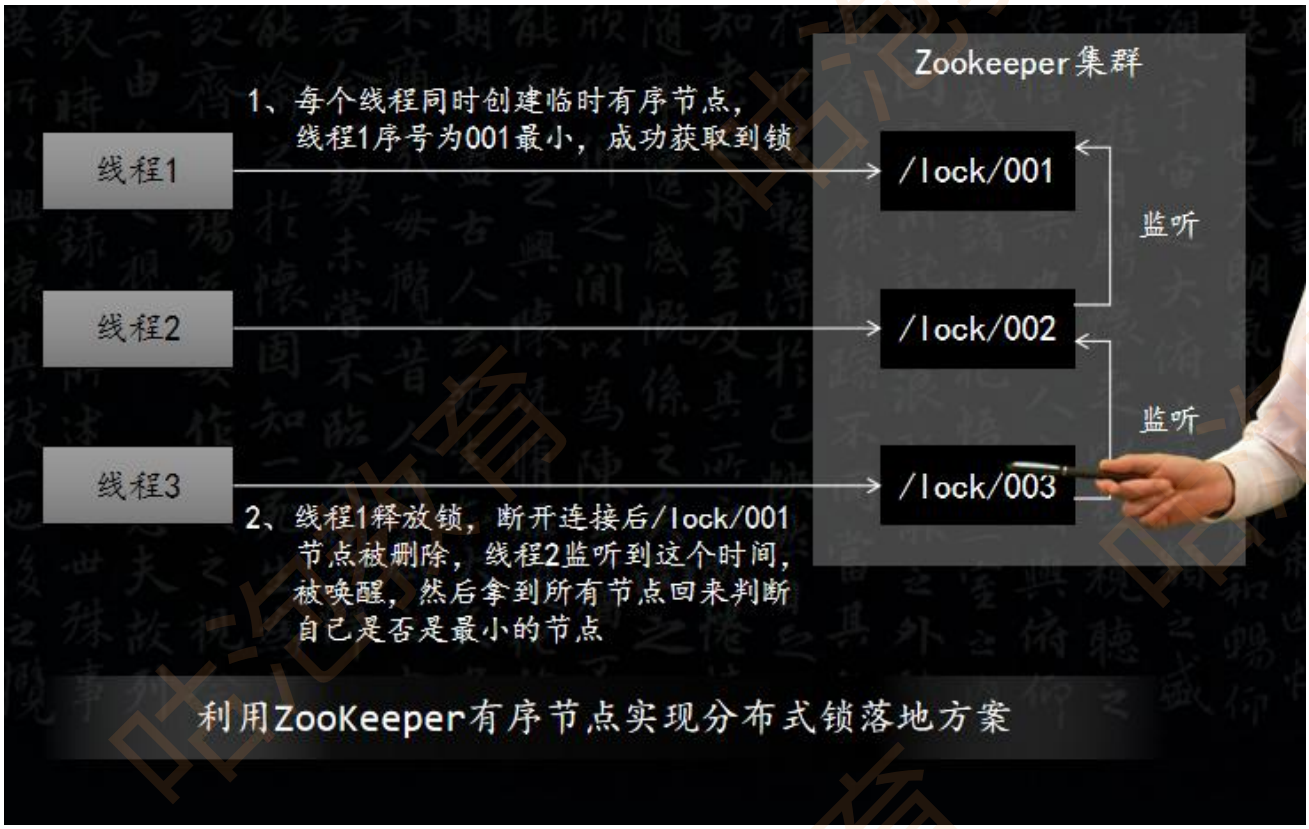
这个指令是设置一个Key-Value，如果 Key 已经存在，则返回 0，否则返回 1，我们基于这个返回值来判断锁的占用情况从而实现分布式锁。

2. 基于 Redission 客户端来实现分布式锁，Redisson 提供了分布式锁的封装方法，我们只需要调用 api 中的 lock（）和 unlock()方法。它帮我们封装锁实现的细节和复杂度。

Redission 所有指令都通过 Lua 脚本执行并支持 Lua 脚本原子性执行， Redission 中有一个 WatchDog 的概念，翻译过来就是看门狗，它会在你获取锁之后，每隔 10 秒帮你把 Key 的超时时间设为 30s，就算一直持有锁也不会出现Key 过期了。“看门狗”的逻辑保证了没有死锁发生。

然后，基于 ZooKeeper 实现分布式锁的落地方案

Zookeeper 实现分布式锁的方法比较多。我推荐使用有序节点来实现，来看这个图，



首先第1步，每个线程或进程在 Zookeeper 上的/lock 目录下创建一个临时有序的节点表示去抢占锁，所有创建的节点会按照先后顺序生成一个带有序编号的节点。

第2步、线程创建节点后，获取/lock 节点下的所有子节点，判断当前线程创建的节点是否是所有的节点的序号最小的。

第3步、如果当前线程创建的节点是所有节点序号最小的节点，则认为获取锁成功。

第4步、如果当前线程创建的节点不是所有节点序号最小的节点，则对节点序号的前一个节点添加一个事件监听，当前一个被监听的节点释放锁之后，触发回调通知，从而再次去尝试抢占锁。

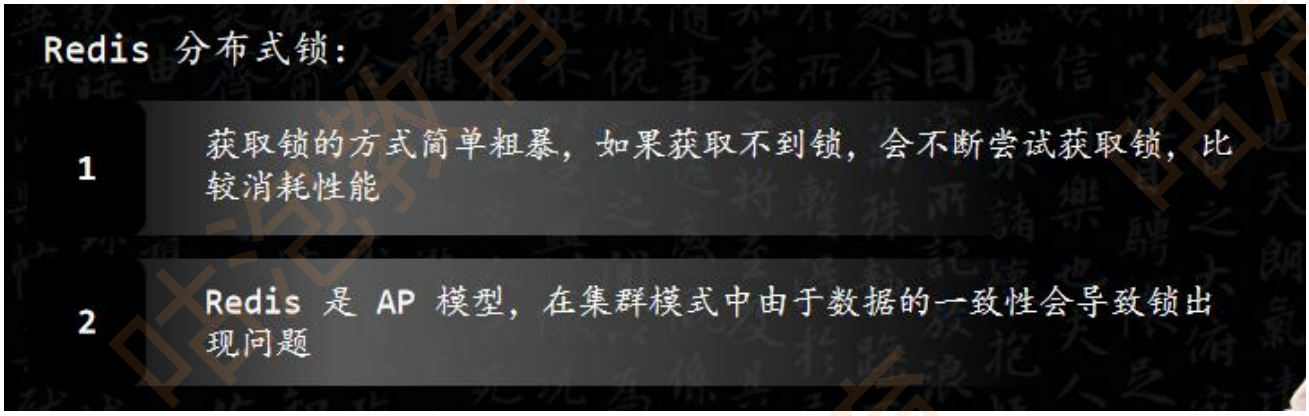
【导航条：两者对比】

（转场，屏幕黑底白字，正中央出现“3 两种方案对比”）

3、两者对比

接下来，咱们来对比一下这两种方案：

对于 Redis 的分布式锁而言，它也有以下缺点：

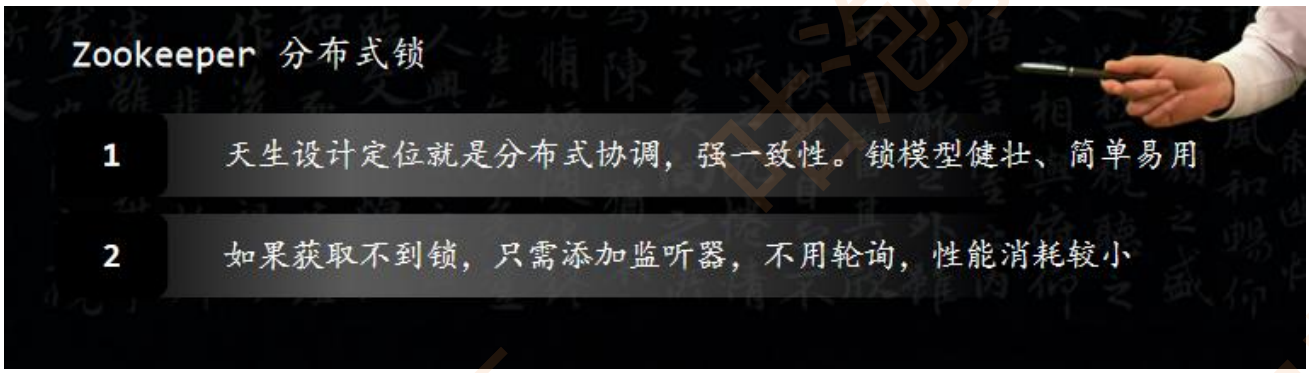


1、它获取锁的方式简单粗暴，如果获取不到锁，会不断尝试获取锁，比较消耗性能。

2、Redis 是 AP 模型，在集群模式中由于数据的一致性会导致锁出现问题，即便使用 Redlock 算法来实现，在某些复杂场景下，也无法保证其实现 100%的可靠性。

不过在实际开发中使用 Redis 实现分布式锁还是比较常见，而且大部分场情况下不会遇到”极端复杂的场景 “，更重要的是 Redis 性能很高，在高并发场景中比较合适。

对于Zookeeper 分布式锁而言：



1、Zookeeper 天生设计定位就是分布式协调，强一致性。锁的模型健壮、简单易用、适合做分布式锁。

2、如果获取不到锁，只需要添加一个监听器就可以了，不用一直轮询，性能消耗较小。

【结尾】

如果要在两者之间做选择，就我个人而言的话，比较推崇 ZK 实现的锁，因为对于分布式锁而言，它应该符合 CP 模型，但是 Redis 是 AP 模型，所以在这个点上，Zookeeper 会更加合适。

最后，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

72.谈谈你对负载均衡的理解

【导航条：引言】

最近有小伙伴来问我，说让我分享一起关于负载均衡的理解，我说，网上有这么多资料了，

怎么还需要我来分享，他说网上的很多资料不系统，难理解。因此就做了这个视频。

另外，我把往期分享的视频全部整理成了1份20字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

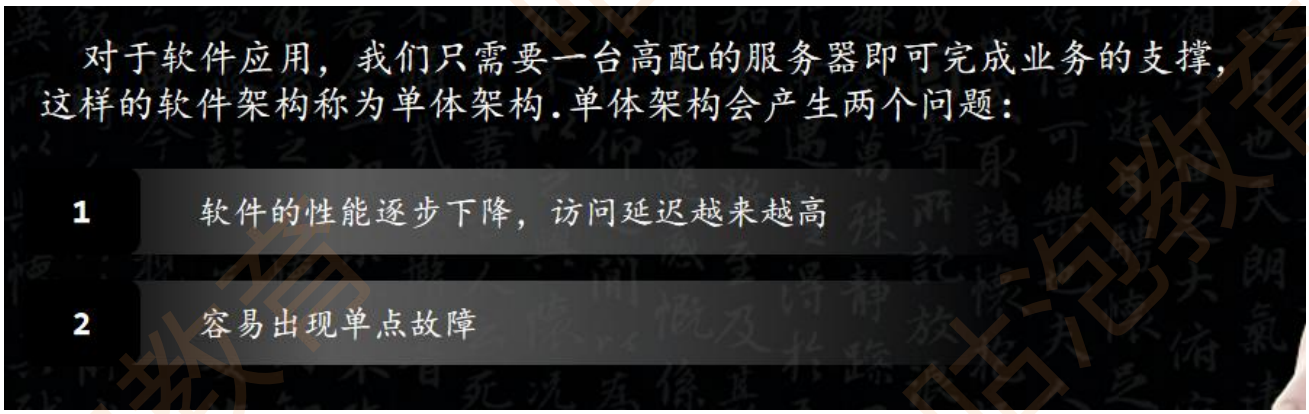
接下来，我从负载均衡产生的背景、负载均衡的实现方案、负载均衡的作用范围、负载均衡的常用策略四个方面来进行分析。

【导航条：产生背景】

（转场，屏幕黑底白字，正中央出现“1 负载均衡产生的背景”）

1、产生背景

在互联网发展早期，由于用户量较少、业务需求也比较简单。对于软件应用，我们只需要一台高配的服务器即可完成业务的支撑，这样的软件架构称为单体架构。



随着用户量的增加，服务器的请流量也随之增加，在这个过程中单体架构会产生两个问题。

1. 软件的性能逐步下降，访问延迟越来越高

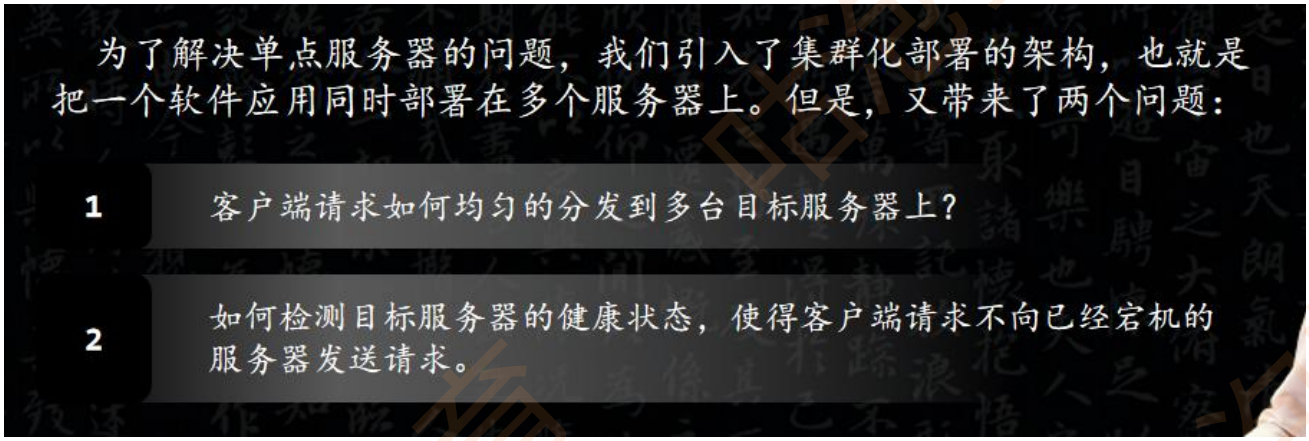
2. 容易出现单点故障

如图所示：

（展示单体架构的图片）

为了解决单点服务器的问题，我们引入了集群化部署的架构，也就是把一个软件应用同时部署在多个服务器上。

但是，这个架构的变化又带来了两个问题：



1. 客户端请求如何均匀的分发到多台目标服务器上?

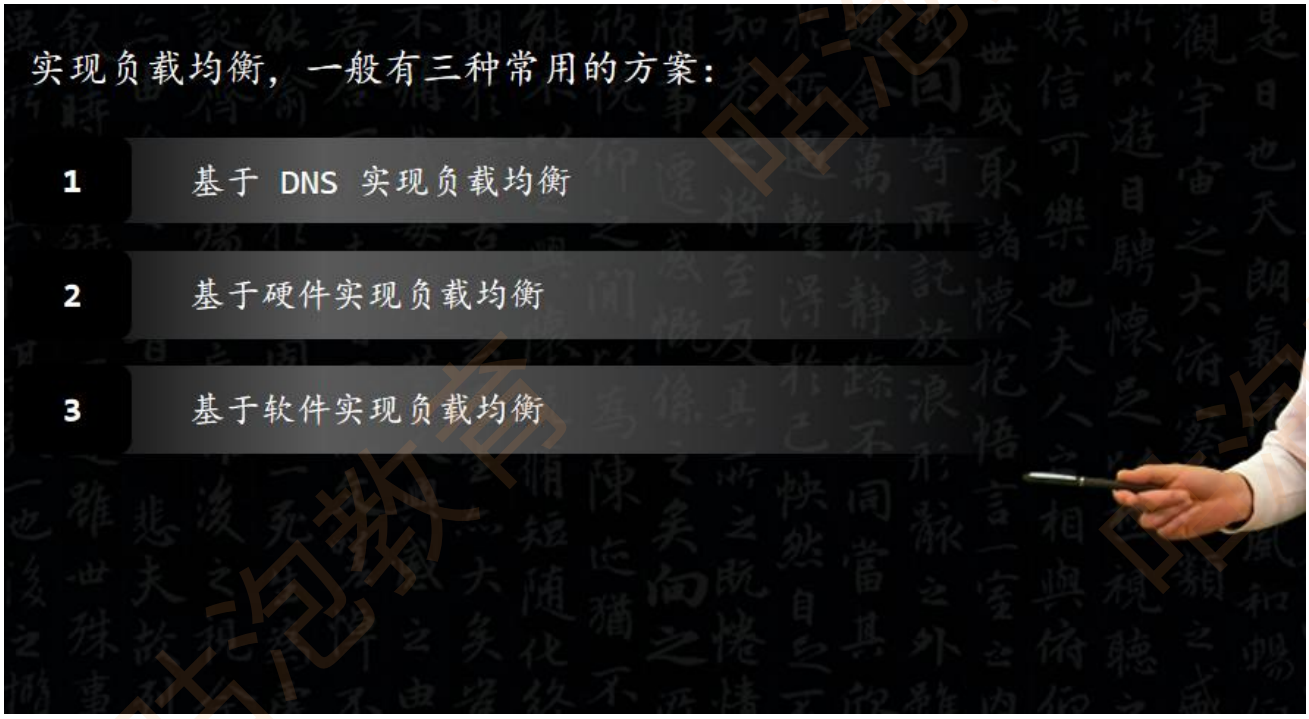
2. 如何检测目标服务器的健康状态，使得客户端请求不向已经宕机的服务器发送请求。

为了解决这两个问题，引入了负载均衡的设计，简单来说，负载均衡机制的核心目的是让客户端的请求合理均匀的分发到多台目标服务器，由于请求被多个节点分发，使得服务端的性能得到有效的提升。

【导航条：实现方案】

（转场，屏幕黑底白字，正中央出现“2 负载均衡的实现方案”）

2、实现方案



实现负载均衡，一般有三种常用的方案：

1、基于 DNS 实现负载均衡

2、基于硬件实现负载均衡

3、基于软件实现负载均衡

先来说一下基于 DNS 实现负载均衡的方式，它的实现方式比较简单，只需要在DNS 服务器上针对某个域名做多个 IP 映射即可。

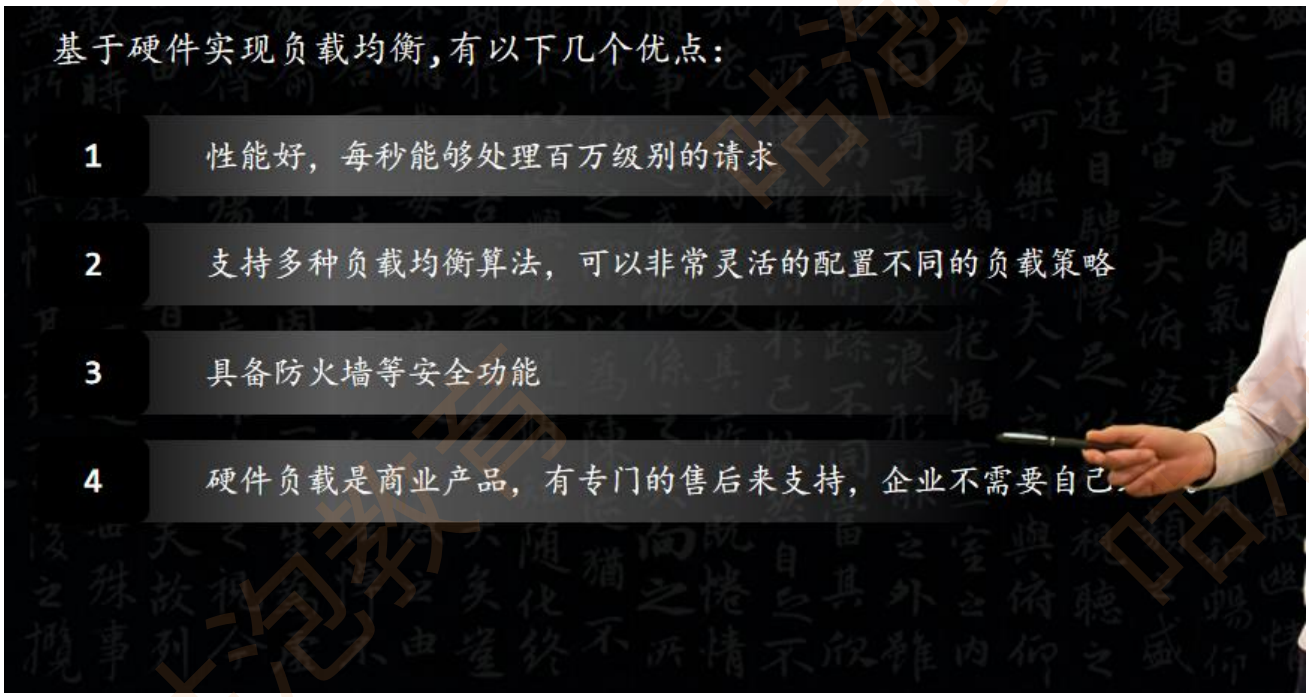
它的工作原理是： 当用户通过域名访问某个网站时，会先通过 DNS 服务器进行域名解析得到一个 IP 地址，DNS 服务器可以随机分配一个 IP 地址进行访问，这样就可以实现目标服务集群的请求分发。

除此之外，DNS 还可以根据不同的地域分配就近机房的 IP，比如长沙的小伙伴，可能会得到在湖南范围内最近的一个机房的 IP，在这个模式下可以实现「就近原则」实现请求处理，缩短了通信距离从而提升网站访问效率。

DNS 实现负载均衡的优点是： 配置简单，实现成本低，无需额外的开发和维护。不过缺点也很明显：

由于 DNS 多级缓存的特性，当我们修改 DNS 配置之后，会因为缓存导致 IP 变更不及时，从而影响负载均衡的效果。

第二种方案，就是基于硬件实现负载均衡



硬件负载设备，我们可以简单把它理解成一个网络设备，它类似于网络交换机，有以下几个优点：

1. 它的性能好，每秒能够处理百万级别的请求，

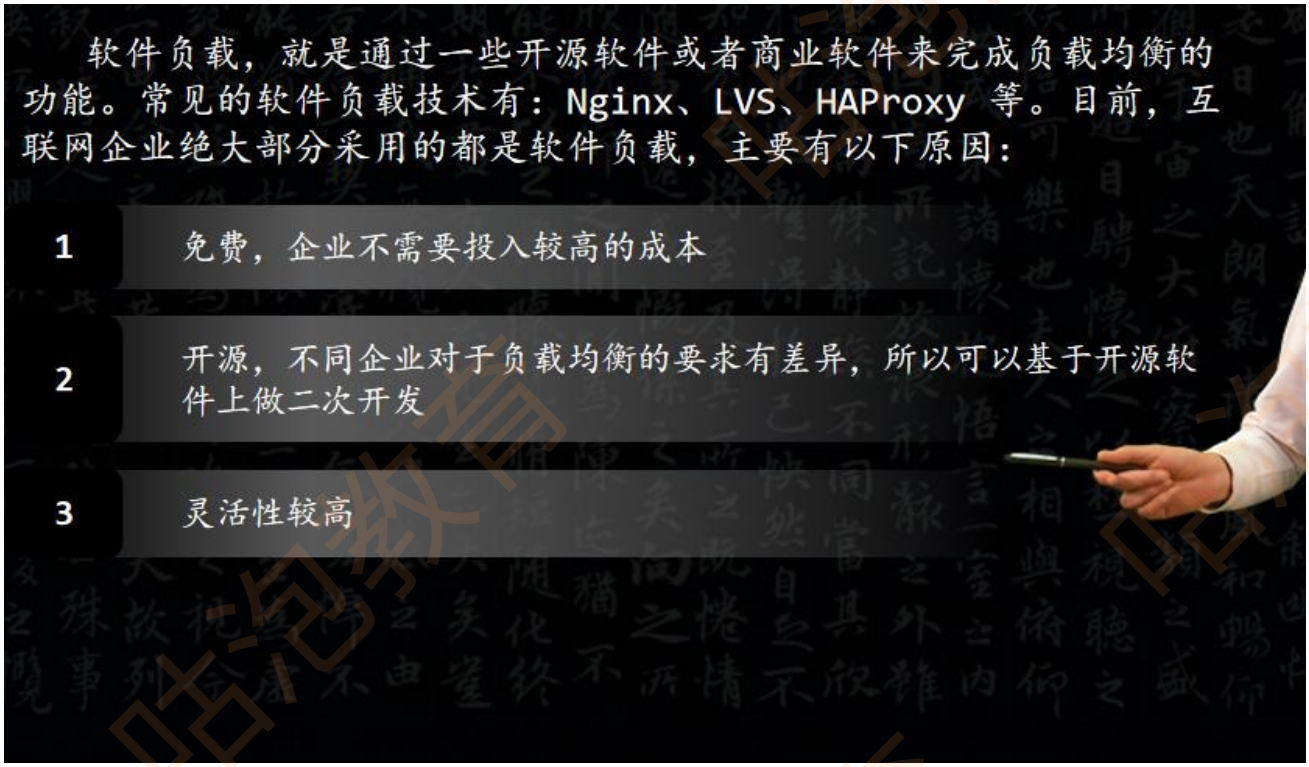
2. 支持多种负载均衡算法，我们可以非常灵活的配置不同的负载策略

3. 它还具备防火墙等安全功能。

4. 硬件负载是商业产品，有专门的售后来支持，所以企业不需要花精力去做维护。

比如：F5 是比较常见的硬件负载设备，由于硬件负载设备价格比较贵，一般应用在大型银行、政府、电信等领域。

第三种方案，就是基于软件实现负载均衡



所谓软件负载，就是通过一些开源软件或者商业软件来完成负载均衡的功能。常见的软件负载技术有：Nginx、LVS、HAProxy 等。

目前互联网企业绝大部分采用的都是软件负载，主要有以下几个原因：

1. 免费，企业不需要投入较高的成本。

2. 开源，不同企业对于负载均衡的要求有差异，所以可以基于开源软件上做二次开发。

3. 灵活性较高

这三种方式，没有好坏之分，只有是否合适，因此大家可以根据实际情况选择。

【导航条：作用范围】

（转场，屏幕黑底白字，正中央出现“2 负载均衡的作用范围”）

3、作用范围

负载均衡是作用在网络通信上，来实现请求的分发。而在网络架构中，基于 OSI 模型，又分为 7 层网络模型，如下表所示：

OSI七层网络模型	TCP/IP四层概念模型	对应网络协议	对应典型设备	区域
应用层(Application)	应用层	TFTP、FTP、NFS、WALS	应用程序，如FTP、SMTP、HTTP	计算机
表示层(Presentation)		Telnet、Rlogin、SNMP、Gopher	编码方式，图像编解码、URL字符传输编码	
会话层(Session)		SMTP、DNS	建立会话，SESSION认证、断点续传	
传输层(Transport)	传输层	TCP、UDP	进程和端口	
网络层(Network)	网际层	IP、ICMP、ARP、RARP、AKP、UUCP	路由器，防火墙、多层交换机	网络
数据链路层(Data Link)	网络接口	FDDI、Ethernet、Arpanet、PDN、SLIP、PPP	网卡、网桥、交换机	
物理层(Physical)		IEER 802.1A、IEEE802.2到IEEE802.11	中继器、集线器、网线、HUB	

这也就是意味着我们可以在网络的某些分层上做请求分发处理，因此根据这样一个特性，对于负载均衡的作用范围又可以分为：

1. 二层负载

2. 三层负载

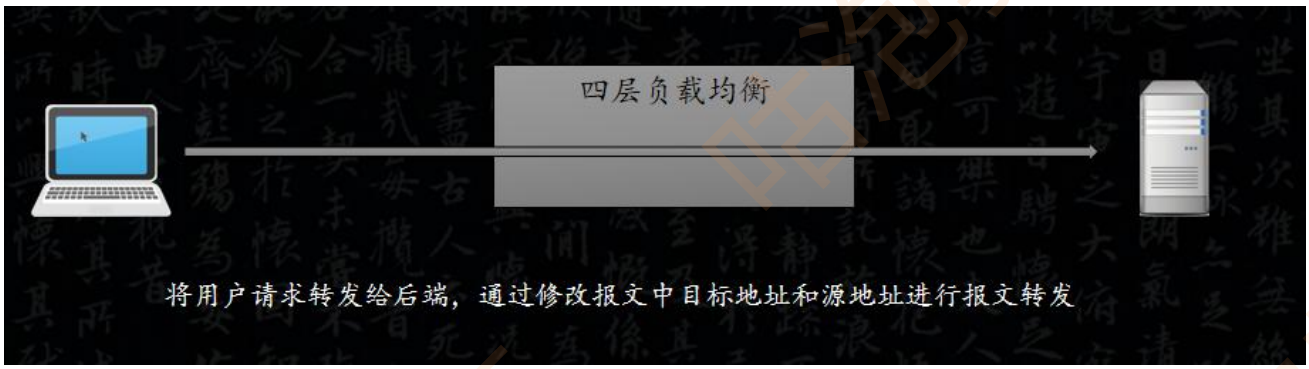
3. 四层负载

4. 七层负载

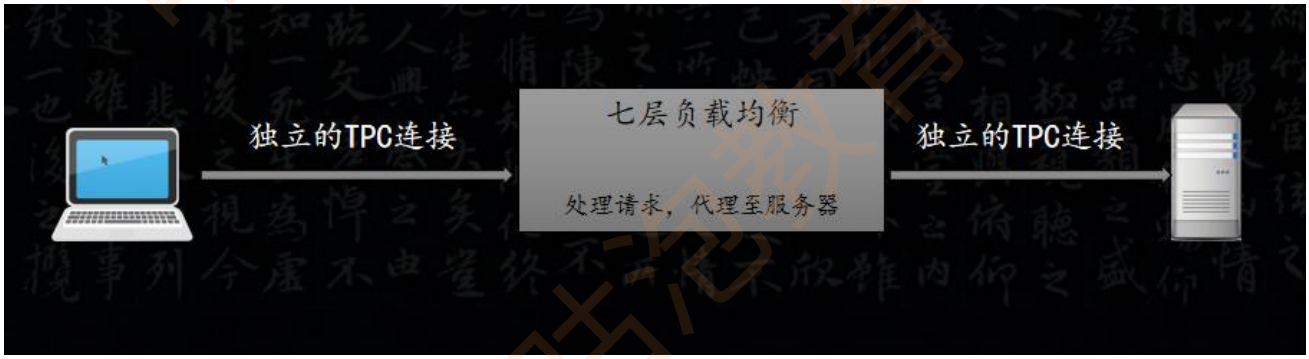
二层负载：基于 Mac 地址来实现请求分发，一般采用虚拟 Mac 的方式实现，服务器收到请求后，通过动态分配后端服务的实际 Mac 地址进行响应从而实现负载均衡

三层负载：基于 IP 层负载，一般通过虚拟 IP 的方式实现，外部请求访问虚拟 IP，服务器收到请求后根据后端实际 IP 地址进行转发。

四层负载： 通过请求报文中的目标地址和端口进行负载，Nginx、F5、LVS 等都可以实现四层负载。



七层负载： 七层负载是基于应用层负载，也就是服务器端可以根据 http 协议中请求的报文信息来决定把请求分发到哪个目标服务器上，比如 Cookie、消息体、RequestHeader 等。



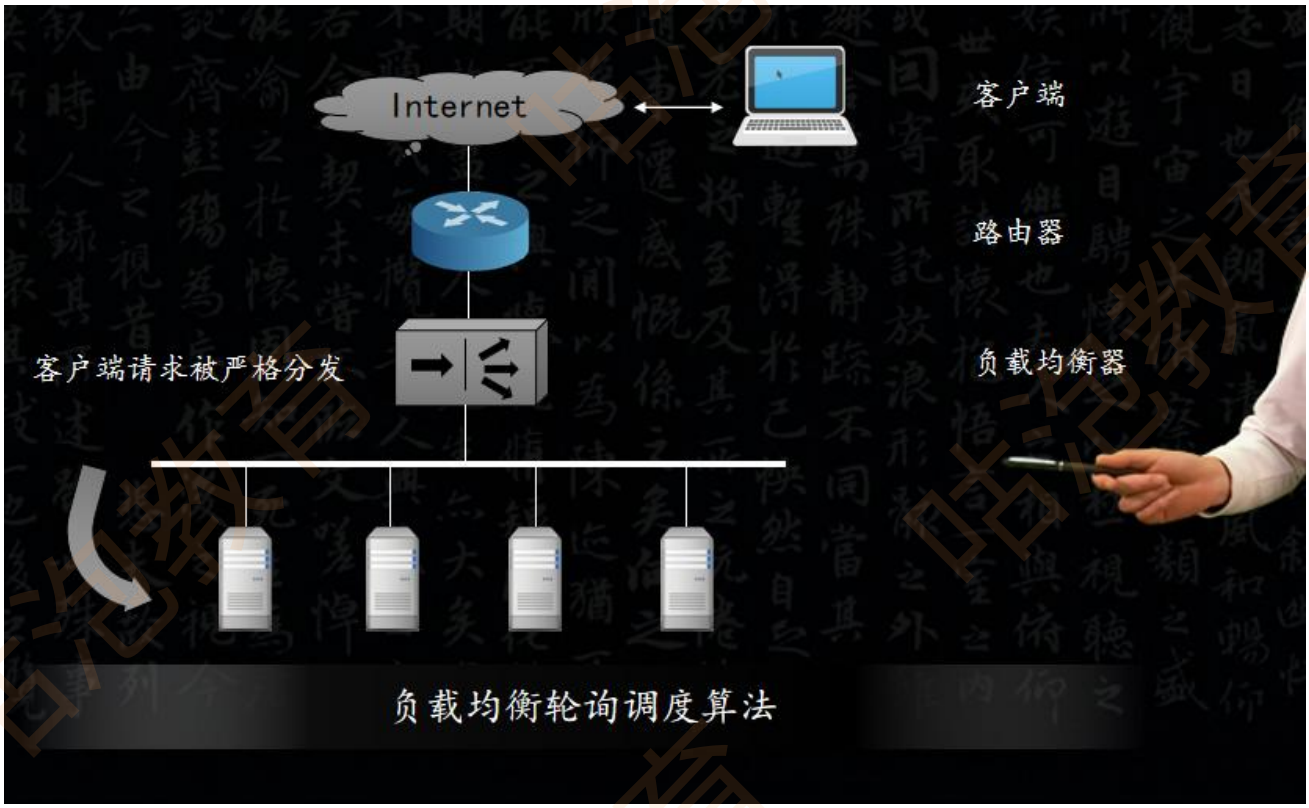
【导航条：常用策略】

（转场，屏幕黑底白字，正中央出现“2 负载均衡的常用策略”）

4、常用策略

所谓负载均衡策略，就是决定当前客户端请求匹配到目标服务器集群中的具体哪个节点。常见的负载均衡策略有，轮询、随机、一致性Hash和最小连接数。

首先来看轮询，也就是多个服务器按照顺序轮询返回，这样每个服务器都能获得相同的请求次数



2. 随机，根据随机算法获得一个目标服务地址，就像古时候皇帝翻牌子一样。

（配图）

由于该算法具备随机性，因此每个服务器获得的请求数量不一定均等。

3. 一致性 Hash，也就是对于具有相同 Hash 码的请求，永远发送到同一个节点上。

4. 最小连接数，根据目标服务器的请求数量来决定请求分发的权重，也就是目标服务集群中，请求更少的节点将会获得更多的请求。这是负载均衡中比较好的策略，真正能够实现目标服务器的请求均衡。

【结尾】

以上就是关于负载均衡相关的内容，当然，负载均衡还有很多值得去挖掘的，比如负载算法如何实现？网络分层模型的原理等。对于网络模型这块，如果有想深度学习的同学，在下方留言”想看“，我会在后续的内容中进行整理

最后，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

73.为什么Spring Boot提倡约定优于配置

【导航条：引言】

在 Spring Boot 中，通过约定优于配置这个思想，可以让我们少写很多的配置，然后就只需要关注业务代码的编写就行。今天呢，我给大家聊聊为什么SpringBoot提倡约定优于配置。

另外，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：原因分析】

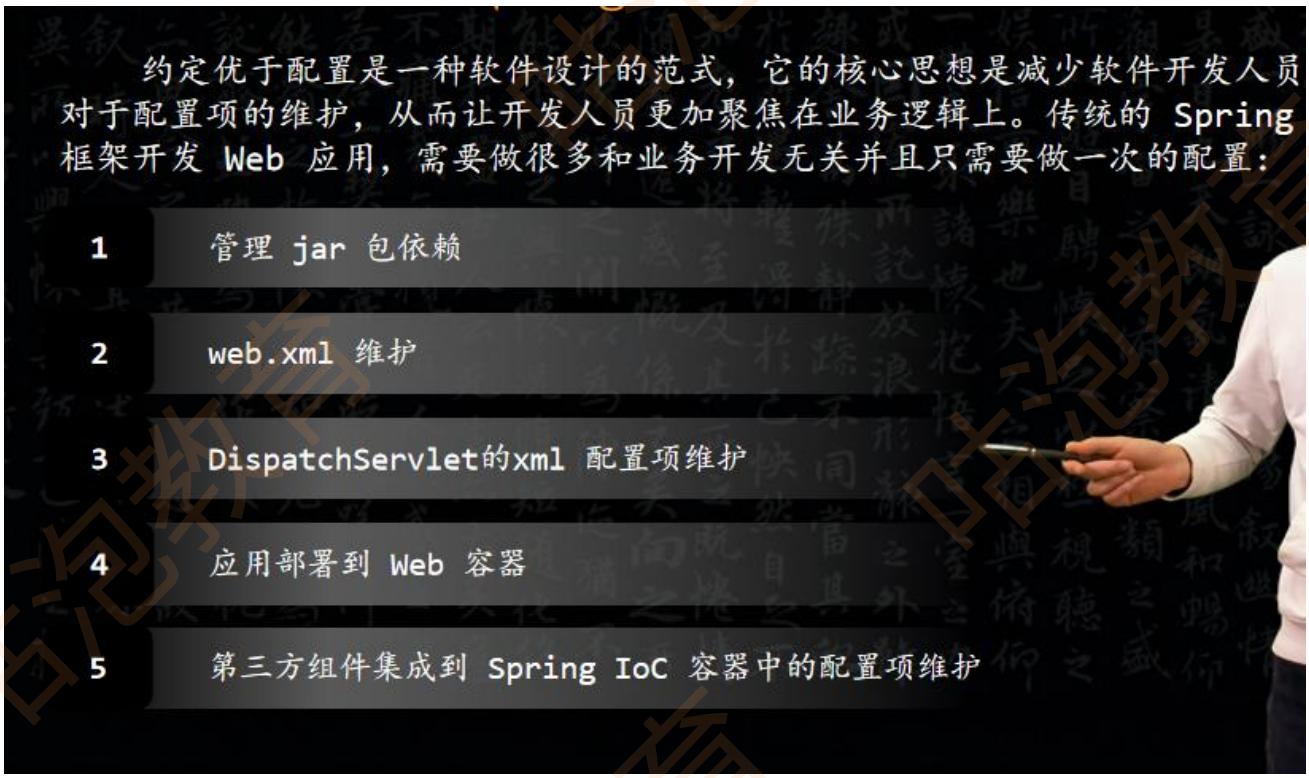
（转场，屏幕黑底白字，正中央出现“1 原因分析”）

1、原因分析

下面，我分享一下我对Spring'Boot提倡约定优于配置的理解。

约定优于配置是一种软件设计的范式，它的核心思想是减少软件开发人员对于配置项的维护，从而让开发人员更加聚焦在业务逻辑上。

Spring Boot 就是约定优于配置这一理念下的产物，它类似于 Spring 框架下的一个脚手架，通过 Spring Boot，我们可以快速开发基于 Spring 生态下的应用程序。



而基于传统的 Spring 框架开发 Web 应用，我们需要做很多和业务开发无关并且只需要做一次的配置，比如：

管理 jar 包依赖

web.xml 维护

DispatchServlet的xml 配置项维护

应用部署到 Web 容器

第三方组件集成到 Spring IoC 容器中的配置项维护

而在 Spring Boot 中，我们不需要再去做这些繁琐的配置，Spring Boot 已经自动帮我们完成了，这就是约定由于配置思想的体现。

【导航条：如何体现】

（转场，屏幕黑底白字，正中央出现“2 如何体现？”）

2、如何体现？

Spring Boot 约定优于配置的体现有很多，比如：

1、Spring Boot Starter 启动依赖，它能帮我们管理所有 jar 包版本；

2、Spring Boot会自动内置Tomcat容器来运行 Web 应用，我们不需要再去单独做应用部署。

3、Spring Boot 通过扫描约定路径下的 Spring.factories文件来识别配置类，实现 Bean 的自动装配。

4、Spring Boot会默认加载的配置文件 application.properties 等等。

总的来说，约定优于配置是一个比较常见的软件设计思想，它的核心本质都是为了更高效以及更的实现软件系统的开发和维护。

【结尾】

以上就是我对这个问题的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指 分享给更多的人。关注我，面试不再难！

74、Dubbo注册中心挂了， 服务还可以继续通信吗?

【导航条：引言】

什么情况？一位工作了 5 年的 Java 程序员，竟然回答不出这个问题？说“Dubbo注册中心挂了， 服务之间还可以继续通信吗”？今天，我话2分钟时间给大家来聊一聊。

另外，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

要回答这个问题，我们得先了解Dubbo的服务感知原理。

【导航条：服务感知原理】

（转场，屏幕黑底白字，正中央出现“1 Dubbo服务感知原理”）

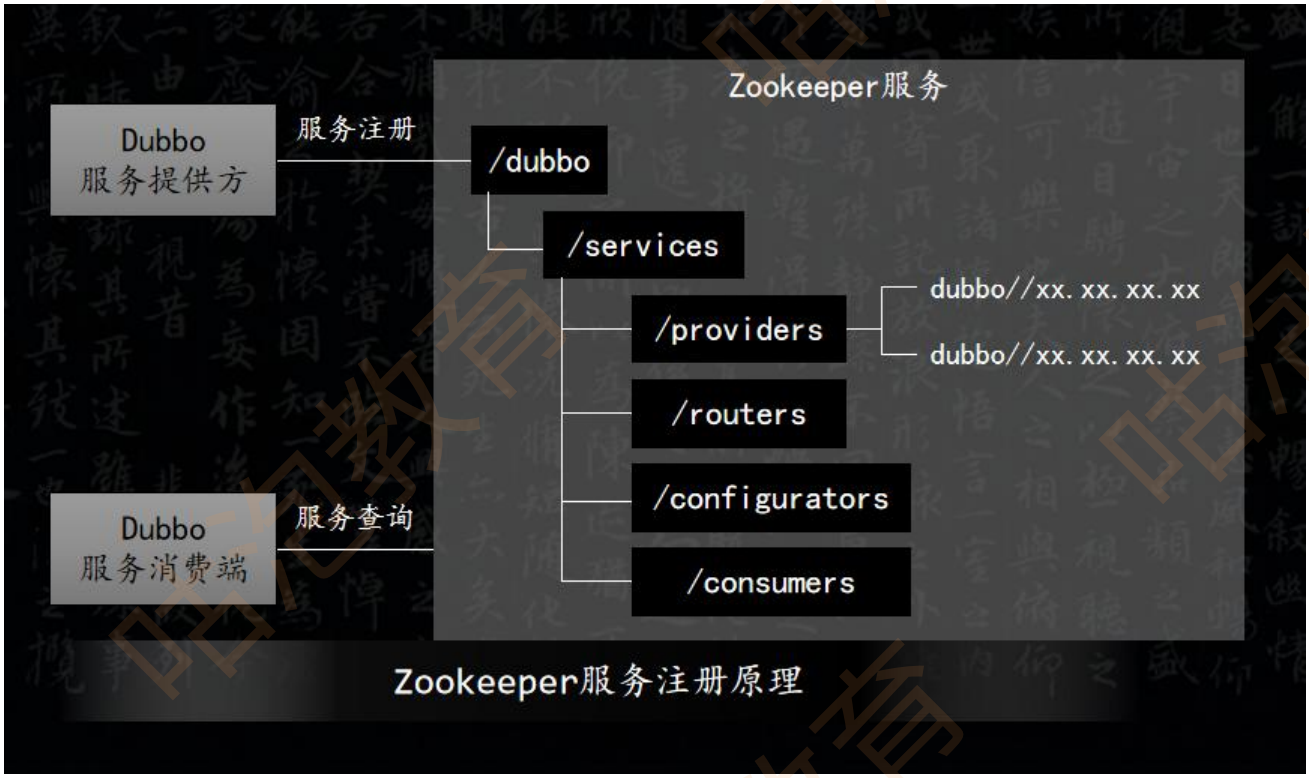
1、服务感知原理

那Dubbo 是如何动态感知服务下线的?

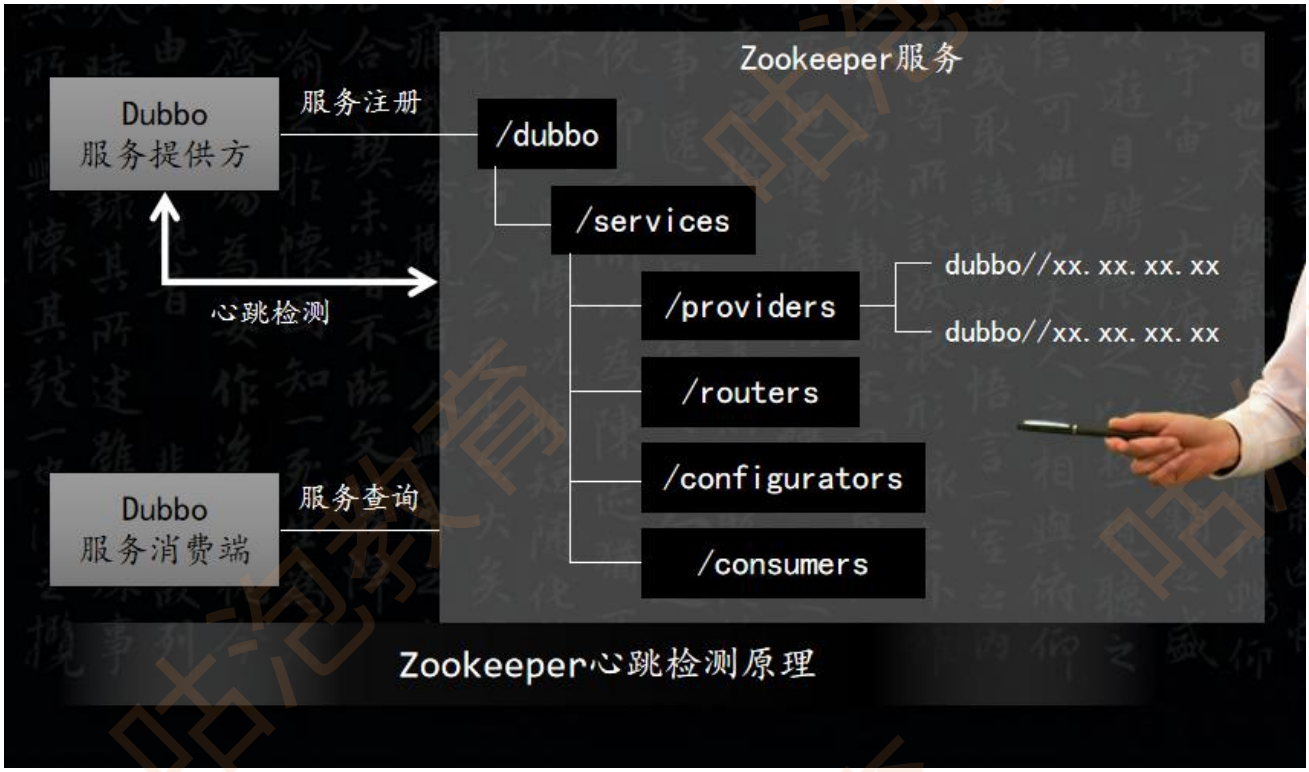
首先，Dubbo 默认采用 Zookeeper 实现服务的注册与服务发现，简单来说，是多个 Dubbo 服务之间的通信地址，是使用 Zookeeper 来维护的。

而在 Zookeeper 上，会采用树形结构的方式来维护 Dubbo 服务提供端的协议地址，Dubbo 服务消费端会从

Zookeeper Server 上去查找目标服务的地址列表，从而完成服务的注册和消费功能。



那Zookeeper呢，就会通过心跳检测机制，来判断 Dubbo 服务提供端的运行状态，来决定是否应该把这个服务从地址列表剔除。



当 Dubbo 服务提供方出现故障导致 Zookeeper 剔除了这个服务的地址，那么 Dubbo 服务消费端需要感知到地址的变化，从而避免后续的请求发送到故障节点，导致请求失败。

也就是说 Dubbo 要提供服务下线的动态感知能力。这个能力是通过 Zookeeper 里面提供的 Watch 机制来实现的。

简单理解呢，就是Dubbo 服务消费端会使用 Zookeeper 里面的 Watch 来针对Zookeeper Server 端的 /providers 节点注册监听，一旦这个节点下的子节点发生变化，Zookeeper Server 就会发送一个事件通知Dubbo Client 端，Dubbo Client 端收到事件以后，就会把本地缓存的这个服务地址删除，这样后续，就不会把请求发送到失败的节点上，完成服务下线感知。

【导航条：答案分析】

（转场，屏幕黑底白字，正中央出现“2 答案分析）

2、答案分析

了解Dubbo服务感知原理以后，我就应该知道，Dubbo的注册中心挂了以后，服务之间还是可以继续通信的。主要是因为Dubbo初始化的时候，消费者会将服务提供者的地址等信息缓存到本地。

以上就是我对这个问题的理解！

【结尾】

以上就是我对这个问题的理解。

Dubbo 是目前非常主流的开源 RPC 框架，在很多的企业都有使用。理解 RPC 底层的工作原理非常有必要，它能帮助开发者提高开发问题的解决效率。

最后，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

75、什么是幂等？如何解决幂等性问题？

【导航条：引言】

一个在传统行业工作了 7 年的粉丝私信我。说他最近去很多互联网公司面试，遇到的很多技术和概念都没听过。其中就有一道题是：”什么是幂等、如何解决幂等性问题“？他说，这个概念听都没听过，怎么可能回答出来。

接下来，我借这位同学的问题，给大家分享一下我对这个问题的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获

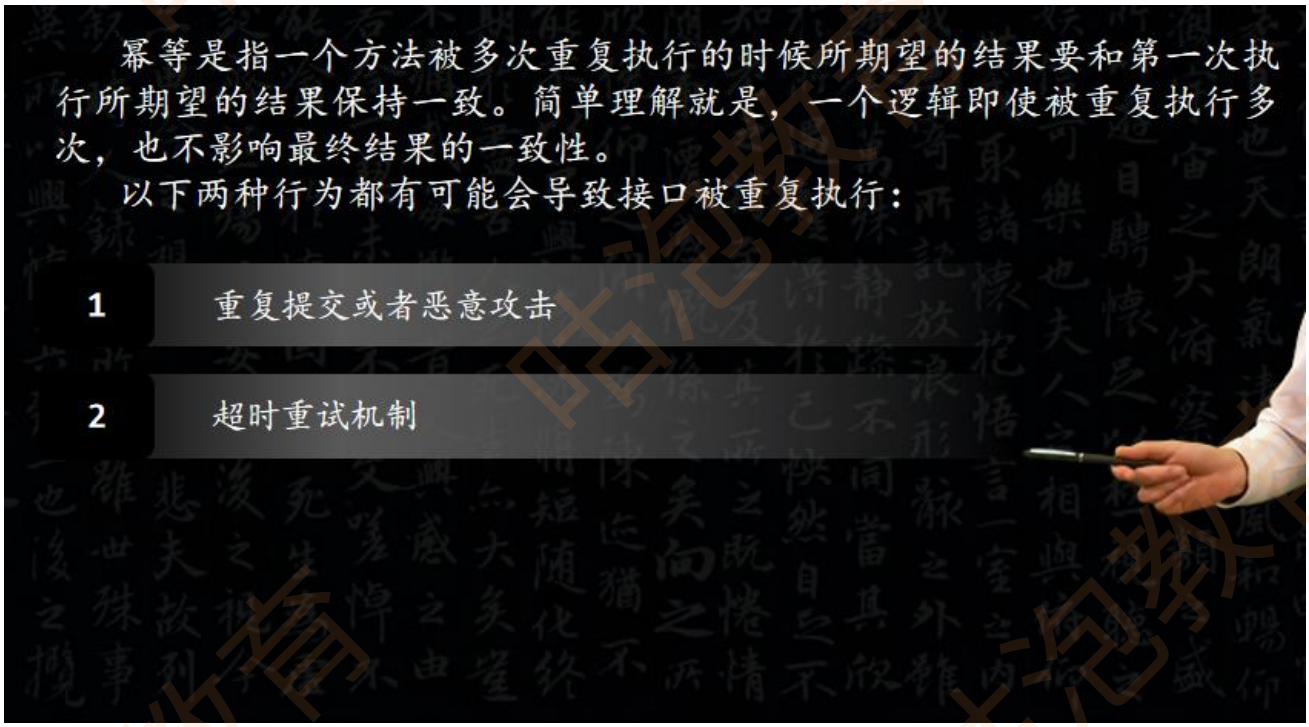
取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：什么是幂等】

（转场，屏幕黑底白字，正中央出现“1 什么是幂等）

1、什么是幂等?

其实幂等，是一个数学上的概念。而在计算机编程领域中，幂等是指一个方法被多次重复执行的时候所期望的结果要和第一次执行所期望的结果保持一致。简单理解就是，一个逻辑即使被重复执行多次，也不影响最终结果的一致性，这叫幂等。



之所以要考虑到幂等性问题，是因为在网络通信中，有这两种行为都有可能会导致接口被重复执行。

第一种是，用户的重复提交或者用户的恶意攻击；

第二种是，分布式系统中，为了避免数据丢失，采用的超时重试机制。

这两种情况都有可能导致服务接口被重复调用。所以在程序设计中，对于数据变更类操作的接口，需要保证接口的幂等性。

而幂等性的核心思想，其实就是保证这个接口的执行结果只影响一次，后续即便再次调用，也不能对数据产生影响，所以基于这样一个需求，我们如何解决幂等性问题呢?

【导航条：如何解决】

（转场，屏幕黑底白字，正中央出现“2 如何解决幂等性问题？）

2、如何解决?

解决幂等性问题的方法有很多，下面我分享一下常用几种方案。

第1种方案，就是使用数据库的唯一约束来实现幂等，比如对于数据插入类的场景，比如创建订单，因为订单号

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

肯定是唯一的，所以如果是多次调用就会触发数据库的唯一约束异常，从而避免一个请求创建多个订单的问题。

第2种方案，就是使用 Redis 提供的 setNX 指令，比如对于 MQ 消费的场景，为了避免 MQ 重复消费导致数据多次被修改的问题，可以在接受到 MQ 的消息时，把这个消息通过 setNX 写入到 Redis 中，一旦这个消息被消费过，就不会再次消费。

第3种方案，使用状态机来实现幂等，所谓的状态机是指一条数据的完整运行状态的转换流程，比如 ，因为它的状态只会向前变更，所以多次修改同一条数据的时候，一旦状态发生变更，那么对这条数据修改造成的影响只会发生一次。

当然，除了以上3种常用的意外，还可以基于 Token 机制或者增加去重表等方法来实现，但是不管是什么方法，无非就是两种思路，

要么就是接口只允许调用一次，比如唯一约束、基于 Redis 的锁机制。

要么就是对数据的影响只会触发一次，比如乐观锁等。

【结尾】

以上就是我对这个问题的理解。

技术这个行业的发展是很快的，如果自己的技术能力和认知跟不上变化。那基本上可以说是被时代淘汰了，所以保持持续学习是非常重要的。

最后，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

76、在秒杀场景中，常用的限流算法有哪些?

【导航条：引言】

一位在传统行业工作了 5 年的程序员。去一个互联网公司面试，被问到一个秒杀的场景题。因为之前完全没接触过分布式相关的项目，单单只是问了限流算法都没有回答不上来，于是向我来求助。

另外，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：什么是限流】

（转场，屏幕黑底白字，正中央出现“1 什么是限流?）

1、什么是限流?

所谓限流，就是指限制流量请求的频次。它主要是在高并发情况下，用于保护系统的一种策略，主要是避免在流量高峰导致系统崩溃，造成系统不可用的问题。

实现限流常见的算法4种，分别是计数器限流算法、滑动窗口限流算法、漏桶限流算法、令牌桶限流算法。下面，我给大家详细介绍每种算法的基本原理。

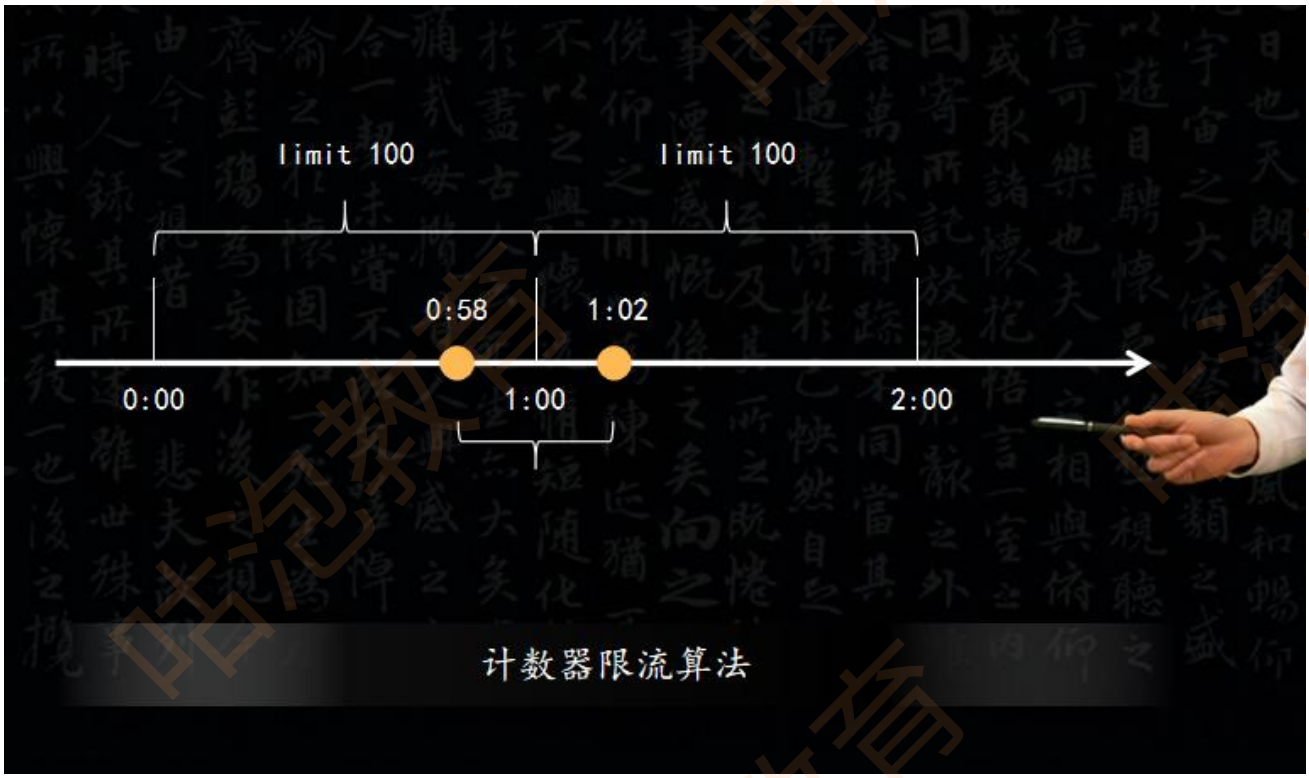
【导航条：限流算法】

（转场，屏幕黑底白字，正中央出现“2 常见的限流算法）

2、限流算法

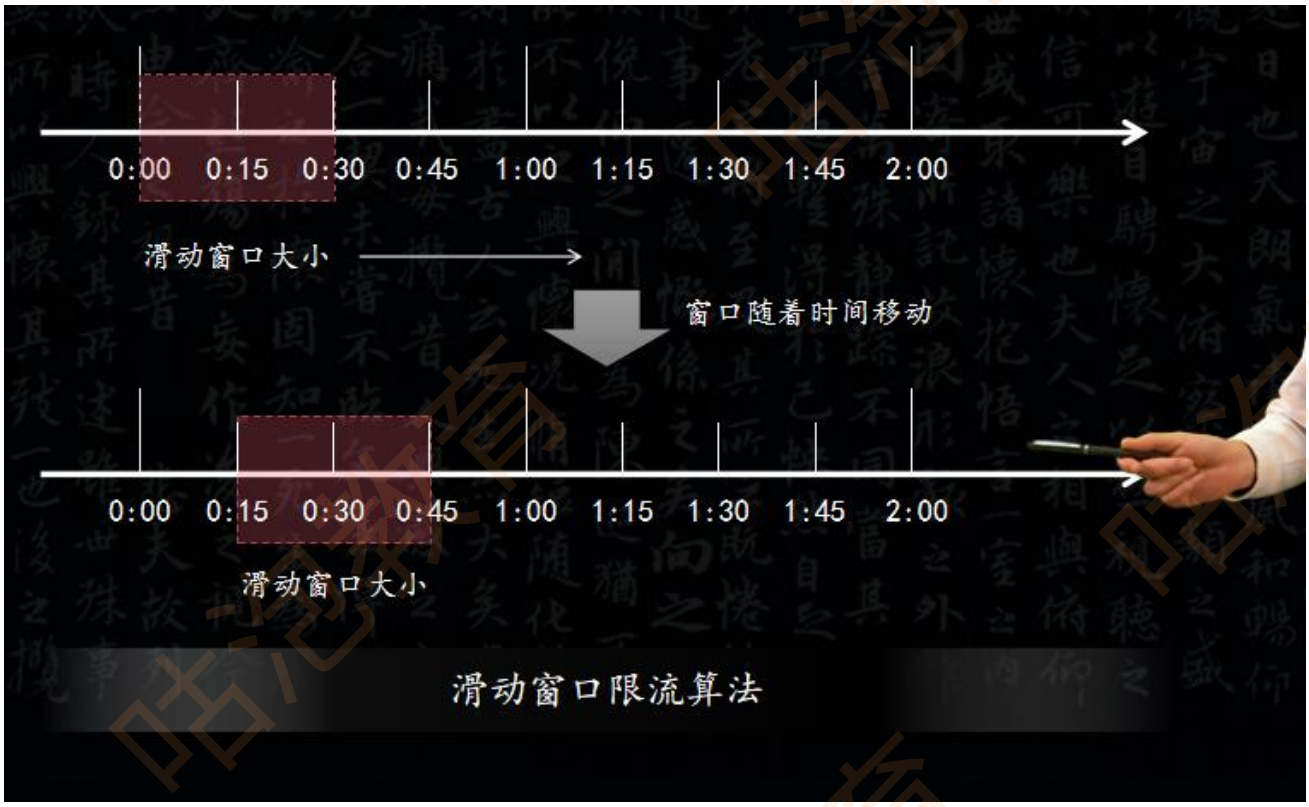
首先来看第一种，

1、计数器限流算法。



一般用在单一维度的访问频率限制上，比如短信验证码每隔 60s只能发送一次，或者接口调用次数等。它的实现方法很简单，就是每调用一次就加 1，处理结束以后减1。

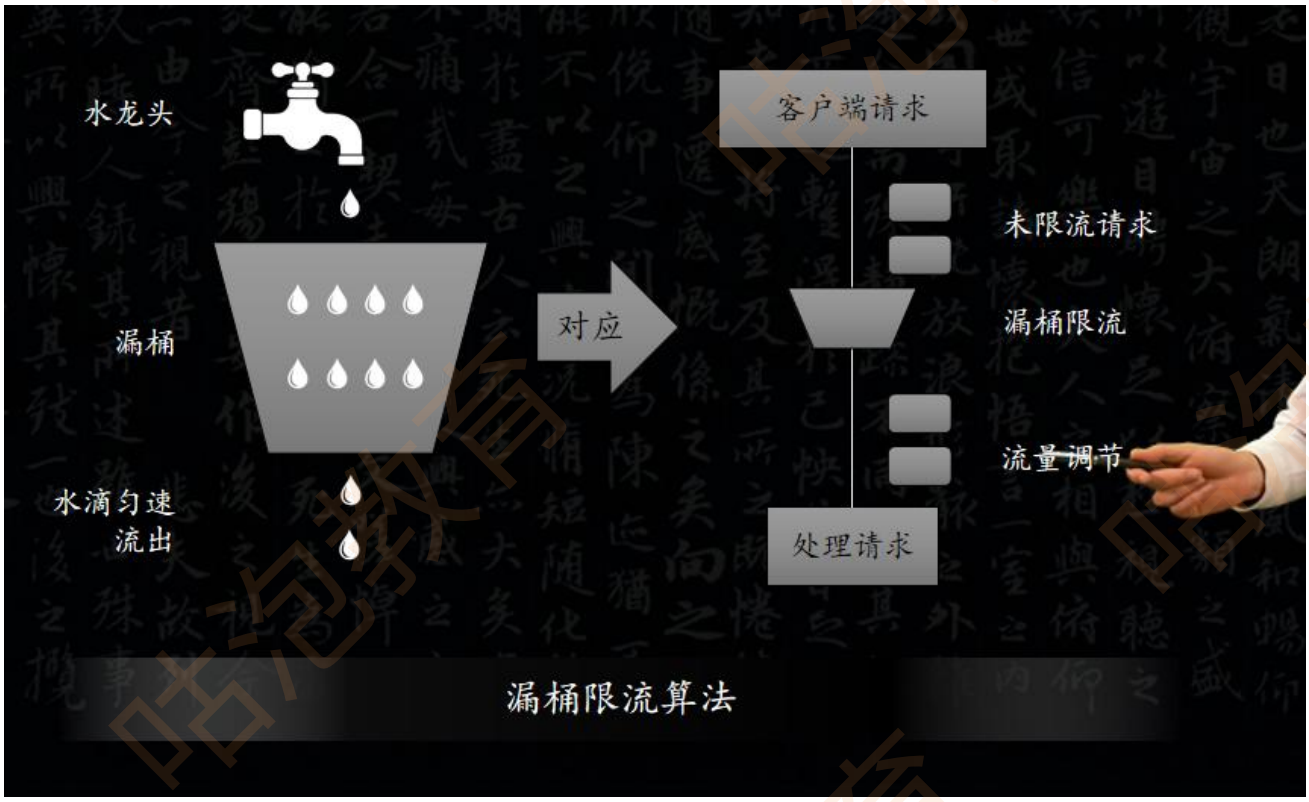
2、滑动窗口限流算法，



本质上也是一种计数器，只是通过以时间为维度的可滑动窗口设计，来减少了临界值带来的并发超过阈值的问题。每次进行数据统计的时候，只需要统计这 个窗口内每个时间刻度的访问量就可以了。

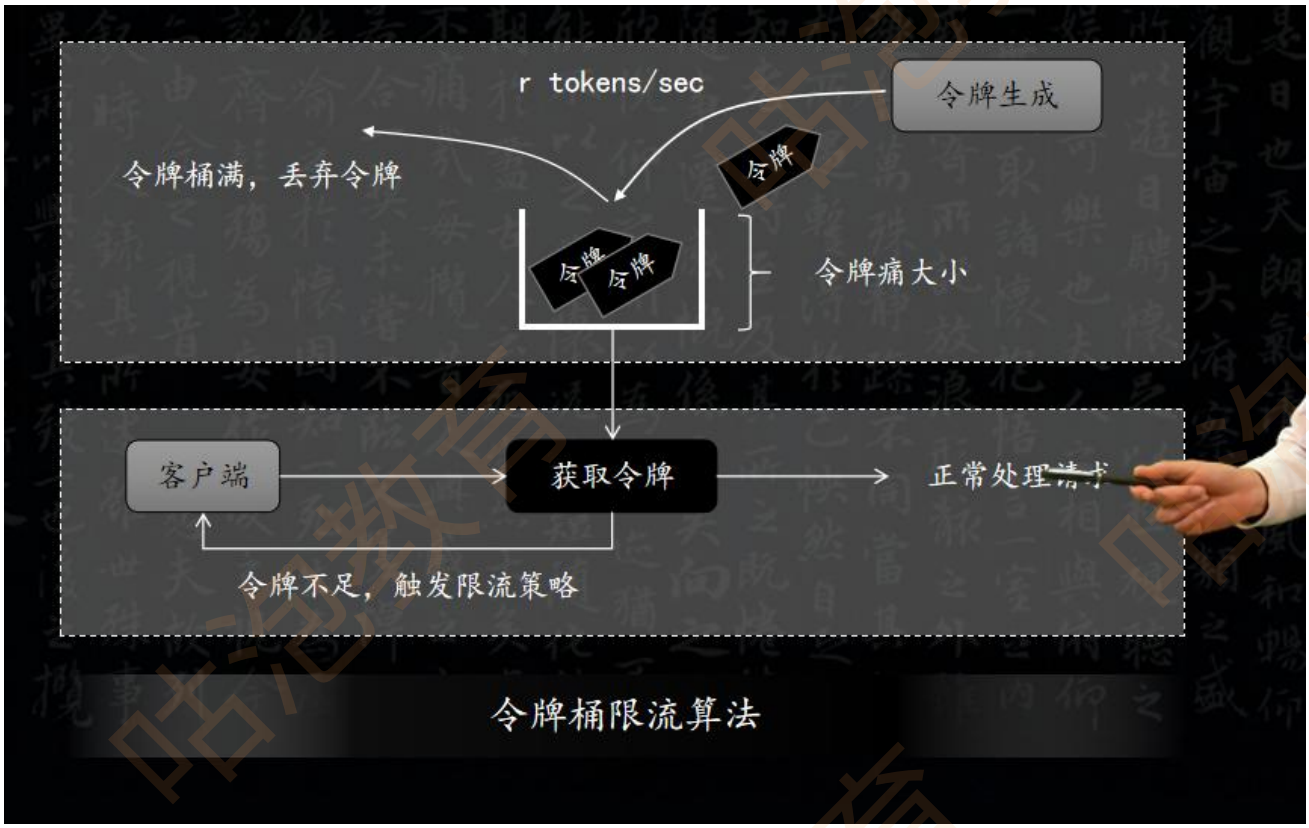
Spring Cloud 中的熔断框架 Hystrix，以及 Spring Cloud Alibaba 中的Sentinel 都采用滑动窗口来做数据统计。

3、漏桶限流算法。



它是一种恒定速率的限流算法，不管请求量是多少，服务端的处理效率是恒定的。基于 MQ 来实现的生产者消费者模型，其实算是一种漏桶限流算法。

4、令牌桶限流算法。



相对漏桶算法来说，它可以处理突发流量的问题。它的核心思想是，令牌桶以恒定速率去生成令牌保存到令牌桶里面，桶的大小是固定的，令牌桶满了以后就不再生成令牌。

每个客户端请求进来的时候，必须要从令牌桶获得一个令牌才能访问，否则排队等待。在流量低峰的时候，令牌桶会出现堆积，因此当出现瞬时高峰的时候，有足够多的令牌可以获取，因此令牌桶能够允许瞬时流量的处理。

网关层面的限流、或者接口调用的限流，都可以使用令牌桶算法，像 Google 的Guava，和 Redisson 的限流，都用到了令牌桶算法。

我认为，限流的本质是实现系统保护，最终选择什么样的算法，一方面取决于统计的精准度，另一方面考虑限流维度和场景的需求。

【结尾】

以上就是我对限流算法的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

77、谈谈你对Zookeeper 选举原理的理解

【导航条：引言】

一位工作了 7 年的程序员，最近在面试时被问到一个关于Zookeeper的问题。因为平时很少研究，所以面试的时候只能一直说：不知道，不知道，不知道。当时，他感觉很尴尬，面试还没结束，就已经知道应该被Pass了。于是又来问我，希望我能分享一期这样的视频。

那今天，我给大家分享一下我的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：什么是Leader选举】

（转场，屏幕黑底白字，正中央出现“1 什么是Leader选举？）

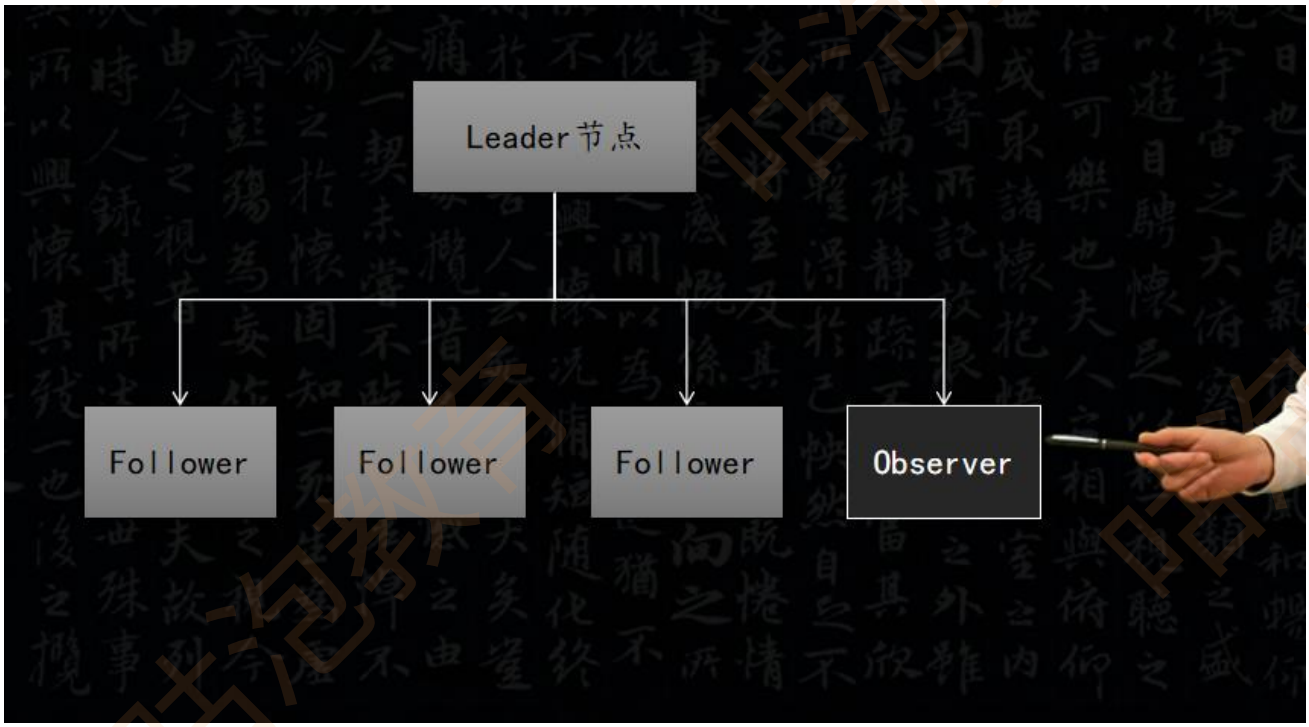
1、什么是Leader选举

首先，Zookeeper 集群节点由三种角色组成，分别是：

Leader，负责所有事务请求的处理，以及过半提交的投票发起和决策。

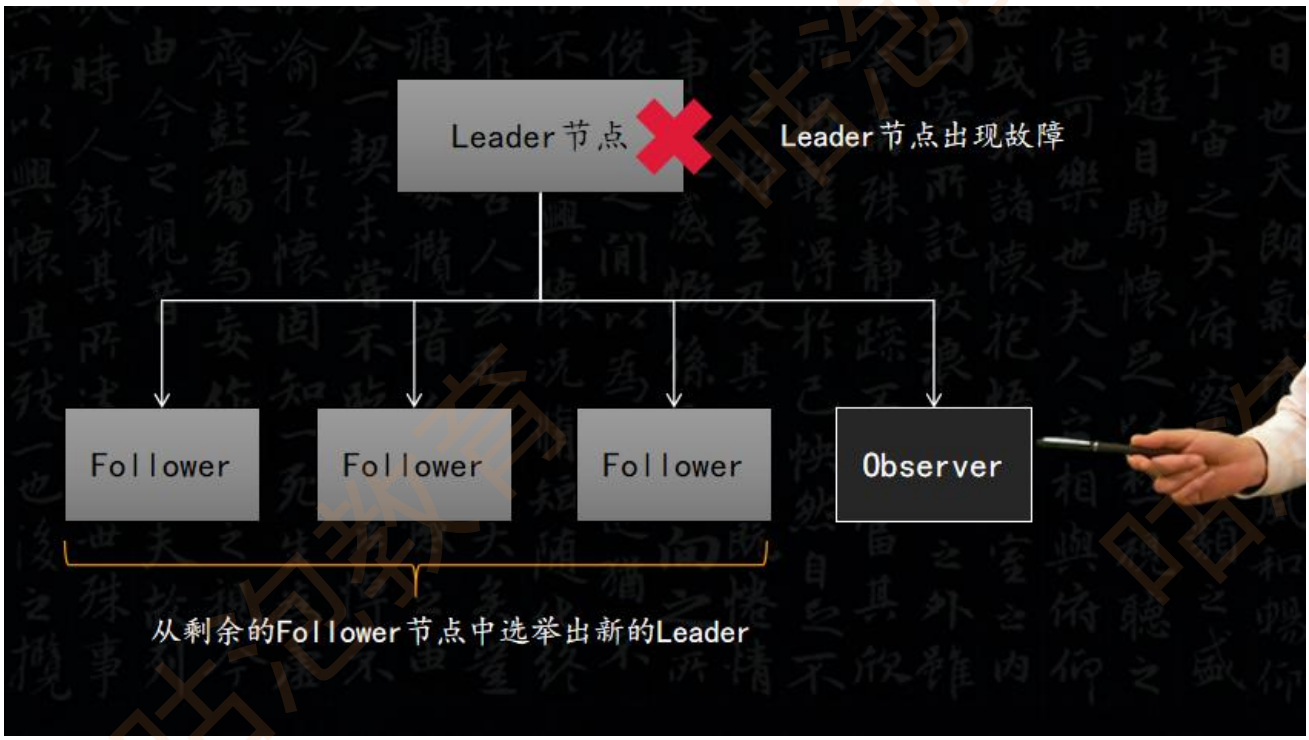
Follower，负责接收客户端的非事务请求，而事务请求会转发给 Leader 节点来处理， 另外，Follower 节点还会参与 Leader 选举的投票。

Observer，负责接收客户端的非事务请求，事务请求会转发给 Leader 节点来处理，Observer 节点不参与任何投票，只是为了扩展 Zookeeper 集群来分担读操作的压力。



其次，Zookeeper 集群是一种典型的中心化架构，也就是会有一个 Leader 作为决策节点，专门负责事务请求的处理和数据的同步。这种架构的好处是可以减少集群架构里面数据同步的复杂度，集群管理会更加简单和稳定。

但是，会带来 Leader 选举的一个问题，也就是说，如果 Leader 节点宕机了，为了保证集群继续提供可靠的服务，Zookeeper 需要从剩下的 Follower 节点里面去选举一个新的节点作为Leader，也就是所谓的 Leader 选举。



【导航条：选举原理】

（转场，屏幕黑底白字，正中央出现“2 选举原理）

2、选举原理

接下来，给大家介绍一下Zookeeper的选举原理。

首先呢，Zookeeper中的每一个节点都会向集群里面的其他节点发送一个票据 Vote，这个票据有三个属性。

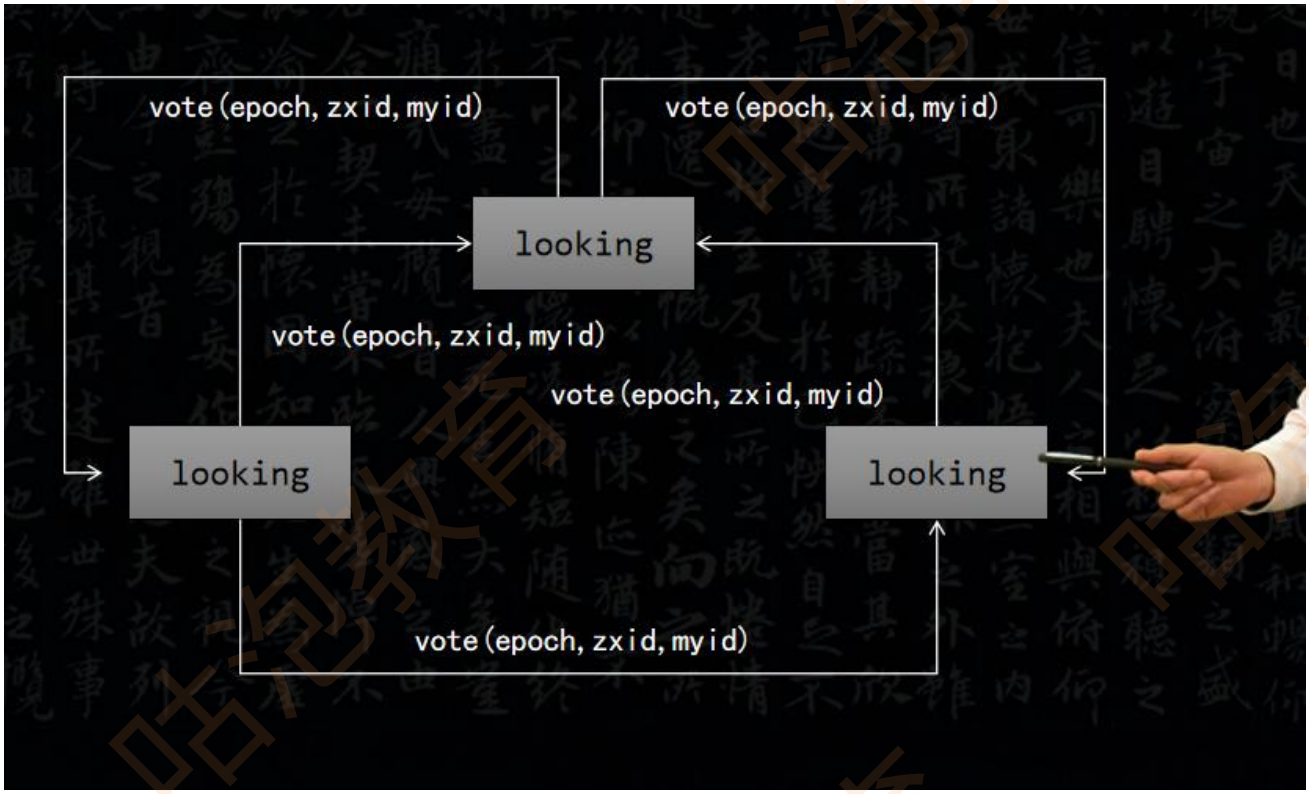
epoch， 逻辑时钟，用来表示当前票据是否过期。

zxid，事务 id，表示当前节点最新存储的数据的事务编号

myid，服务器 id，在 myid 文件里面填写的数字。

每个节点都会选自己当 Leader，所以第一次投票的时候携带的是当前节点的信息。

接下来每个节点用收到的票据和自己节点的票据做比较，根据 epoch、zxid、myid的顺序逐一比较，以值最大的一方获胜。比较结束以后这个节点下次再投票的时候，发送的投票请求就是获胜的 Vote 信息。



然后，通过多轮投票以后，每个节点都会去统计当前达成一致的票据，以少数服从多数的方式，最终获得票据最多的节点成为 Leader。

【结尾】

最后我再补充一下，为什么要选择 epoch/zxid/myid 作为投票评判依据？我是这么理解的。

首先，epoch ，因为网络通信延迟的可能性，有可能在新一轮的投票里面收到上一轮投票的票据，这种数据应该丢弃，否则会影响投票的结果和效率。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

然后，zxid 越大，说明这个节点的数据越接近 leader，所以用 zxid 做判断条件是为了避免数据丢失的问题

最后，myid是服务器 id，这个是避免投票时间过长，直接用 myid 最大值作为快速终结投票的属性。

Leader 选举是一个比较复杂的问题，它涉及到集群节点的数据一致性算法。在很多中间件里面都有涉及到类似的问题，这个思想其实还是很有研究价值的。除此之外，还有 Paxos、raft、等一致性算法。

以上就是我对Zookeeper选举原理的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

78、谈谈你对时间轮的理解

【导航条：引言】

一个=位工作了 7 年的程序员，去字节面试，被问到时间轮的问题。他说这个问题超出了他的知识面，自己也在网上也找了一些文章学习，但还是理解得不是很深刻。他希望让我出一期关于时间轮的面试题解析。今天，就给这位粉丝安排。

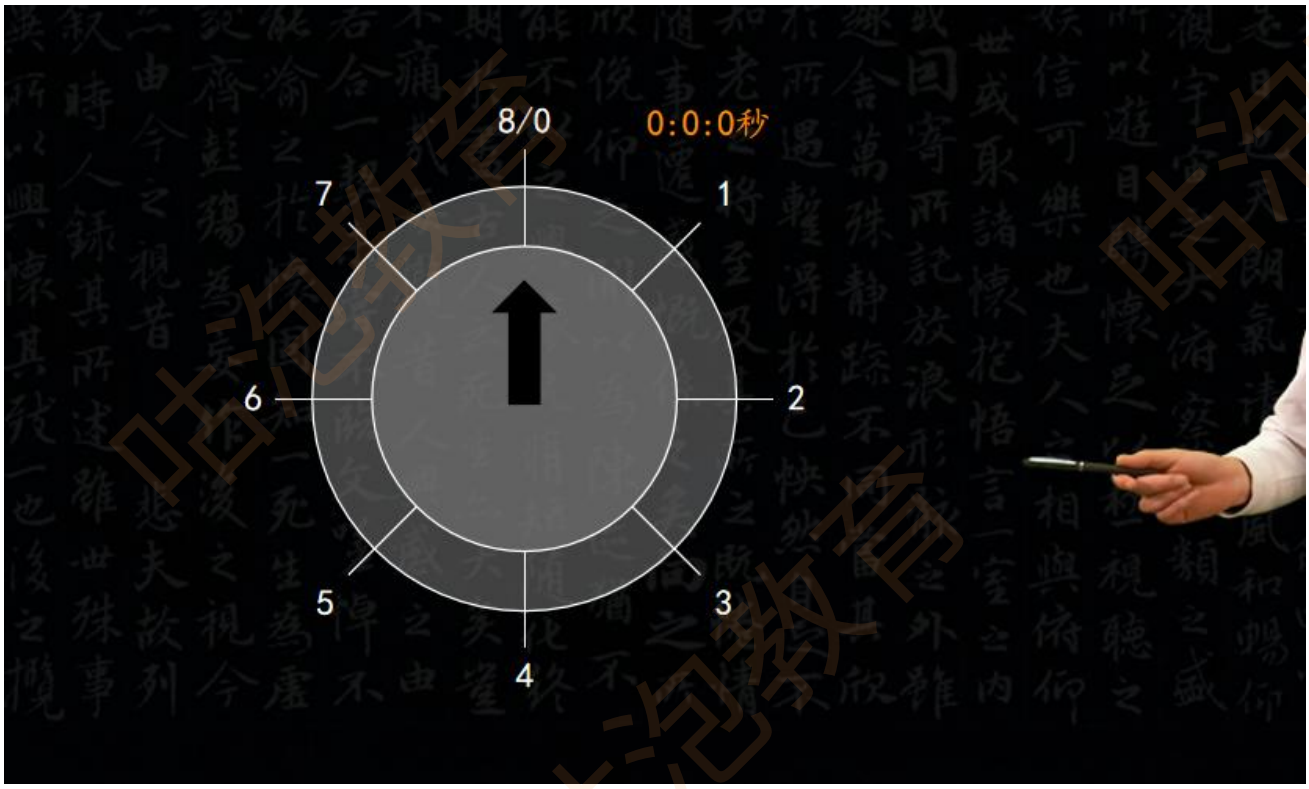
另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：什么时间轮】

(转场，屏幕黑底白字，正中央出现“1什么时间轮?)

1、什么是时间轮

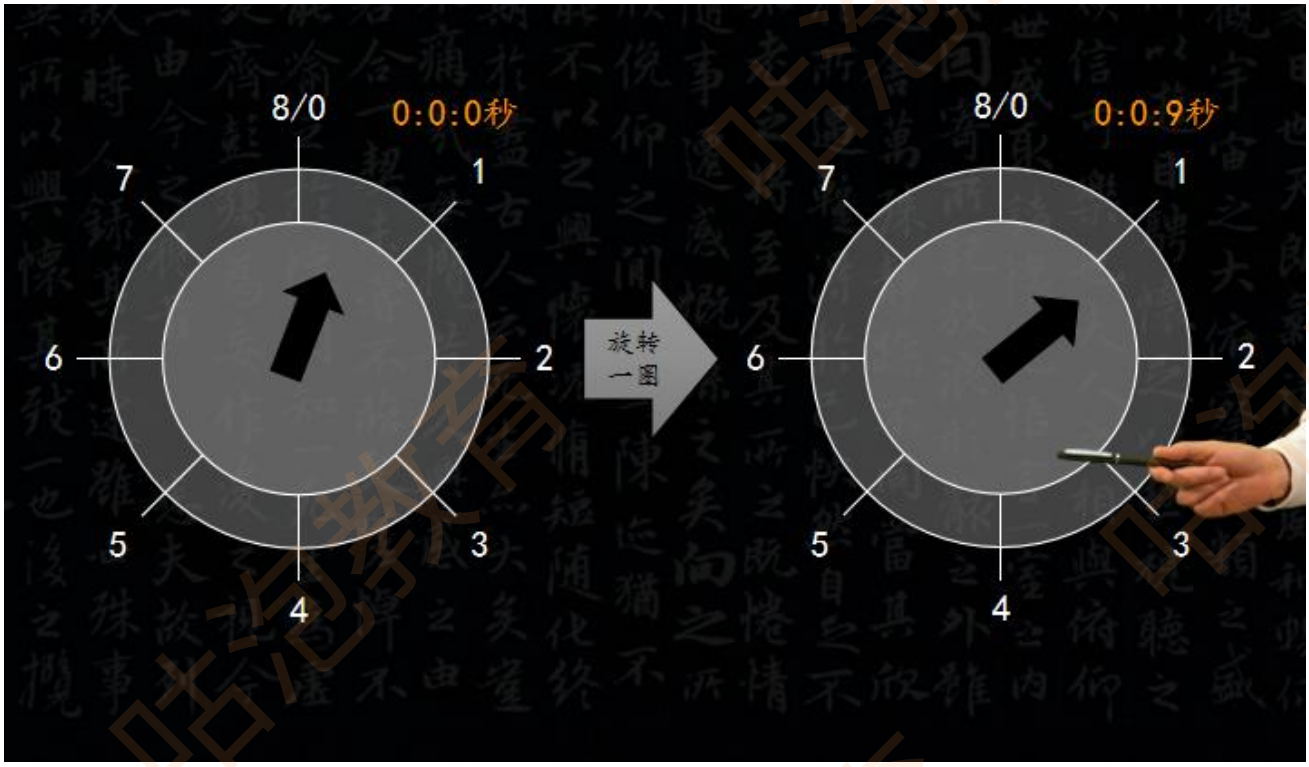
时间轮，简单理解就是一种=个用来存储定时任务的环状数组，它的工作原理和钟表的表盘类似。



它由两个部分组成， 一个是环状数组，另一个是遍历环状数组的指针。

首先，要定义一个固定长度的环状数组，然后数组的每一个元素代表一个时间刻度，假设每个刻度间隔是 1s，那么长度为 8 的数组，就代表 8 秒钟。

然后，就是有一个指针，这个指针按照顺时针无限地循环这个数组，每隔1个最小的时间单位就前进一个数组索引。



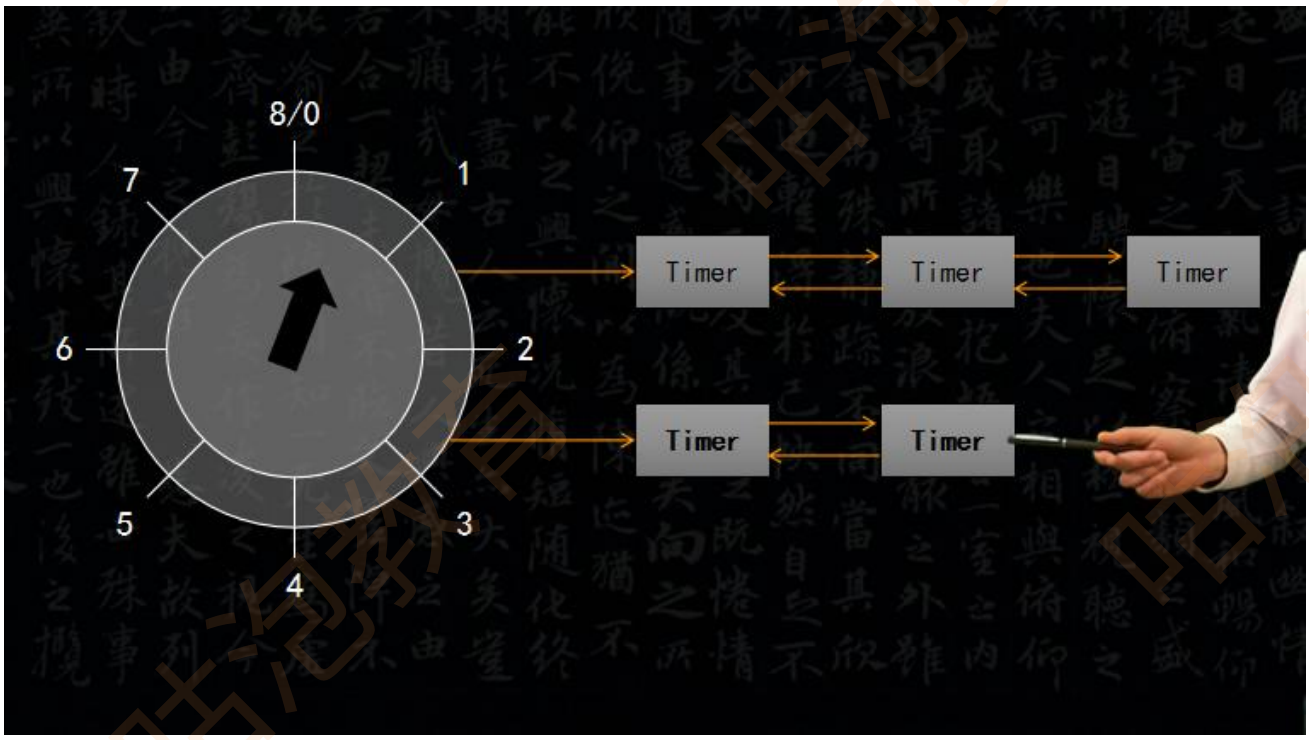
这个指针完整地转1圈，就代表 8 秒钟，转两圈表示 16 秒，假设从 0 点 0 分 0 秒开始，转一圈以后就到了 0 点 0 分 9 秒钟。

【导航条：工作原理】

（转场，屏幕黑底白字，正中央出现“2 时间轮工作原理）

2、工作原理

时间轮，环状数组里面的每个元素，都是用来存储定时任务的容器，当我们向时间轮里面添加一个定时任务的时候，我们会根据定时任务的执行时间计算它所存储的数组下标。当然，有可能在某个时间刻度上会存在多个定时任务，这个时候会采用双向链表的方式来存储。



当指针指向某个数组的时候，就会把这个数组中存储的任务取出来，然后遍历这个链表逐个运行里面的任务。

那如果某个定时任务的执行时间大于环形数组所表示的长度，一般可以使用一个圈数来表示该任务的延迟执行时间。比如，1个第 16 秒要执行的任务，那意味着这个任务应该是在第2圈的数组下标 为0 的位置执行。

【导航条：优、缺点分析】

（转场，屏幕黑底白字，正中央出现“3 时间轮优、缺点分析）

3、优、缺点分析

使用时间轮的方式来管理多个定时任务的好处有很多，我认为有两个比较重要的优点：

1、减少定时任务添加和删除的时间复杂度，提升性能。

2、可以保证每次执行定时器任务都是 O（1）复杂度，在定时器任务密集的情况下，性能优势非常明显。

当然，时间轮也有缺点，对于执行时间非常严格的任务，时间轮不是很适合，因为时间轮算法的精度取决于最小

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

时间单元的粒度。假设以 1s 为一个时间刻度，那小于 1s 的任务就无法被时间轮调度。

时间轮算法在很多框架中都有用到，比如 Dubbo、Netty、Kafka 等。

【结尾】

时间轮算法也是一个比较经典的设计。使用范围比较广，但是在实际应用中，大部分同学接触非常少。我认为这种设计思想或者这种数据结构，在我们实际应用中的某些特定场景也是可以借鉴和使用的。比如定时重试、衰减重试等等。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

79、AQS为什么要用双向链表?

【导航条：引言】

一位工作4年的程序员， 简历上写了精通并发编程， 并且还阅读过AQS（AbstractQueuedSynchronizer）的源码，然后面试官只问了这样一个问题： “AQS 为什么要采用双向链表结构”？，然后就垮了！

其实AQS 大家都不陌生，它是 J.U.C 包里面一个非常重要的线程同步器。今天，我给大家聊聊我的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

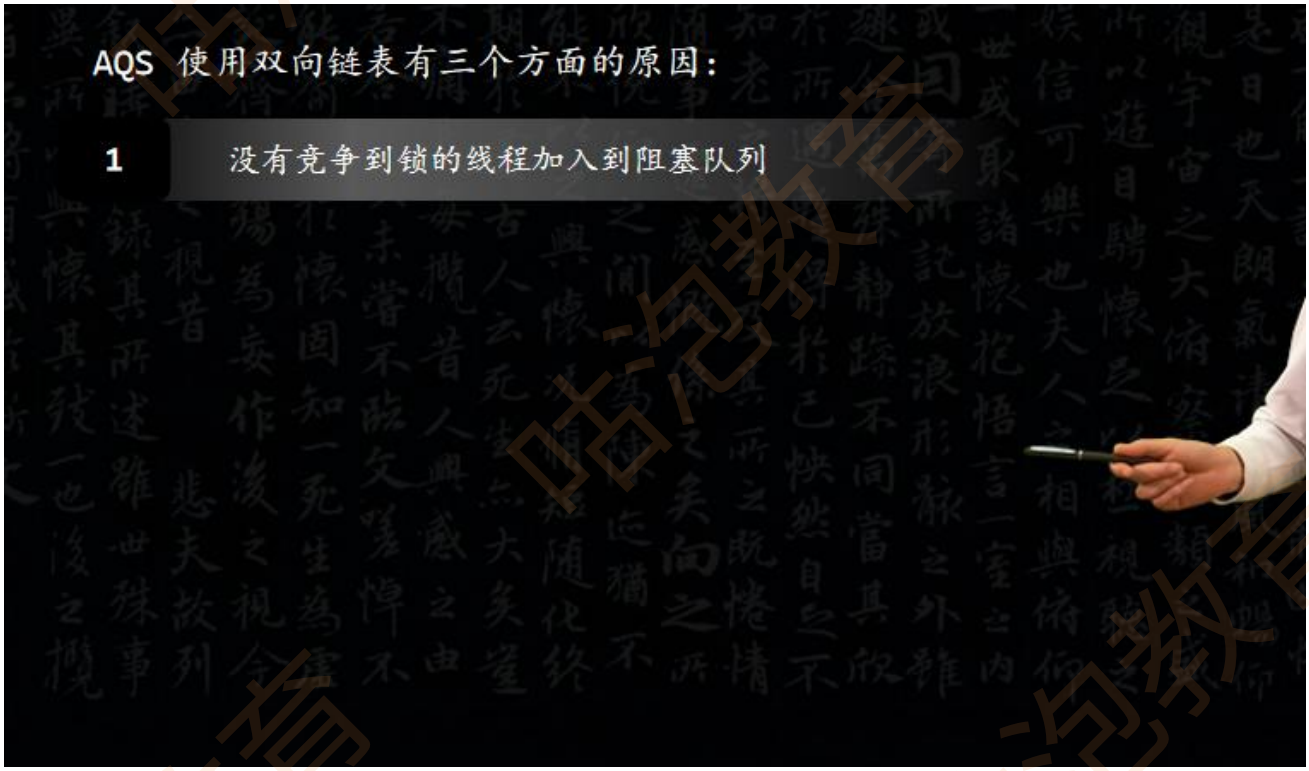
【导航条：原因分析】

（转场，屏幕黑底白字，正中央出现“1 原因分析）

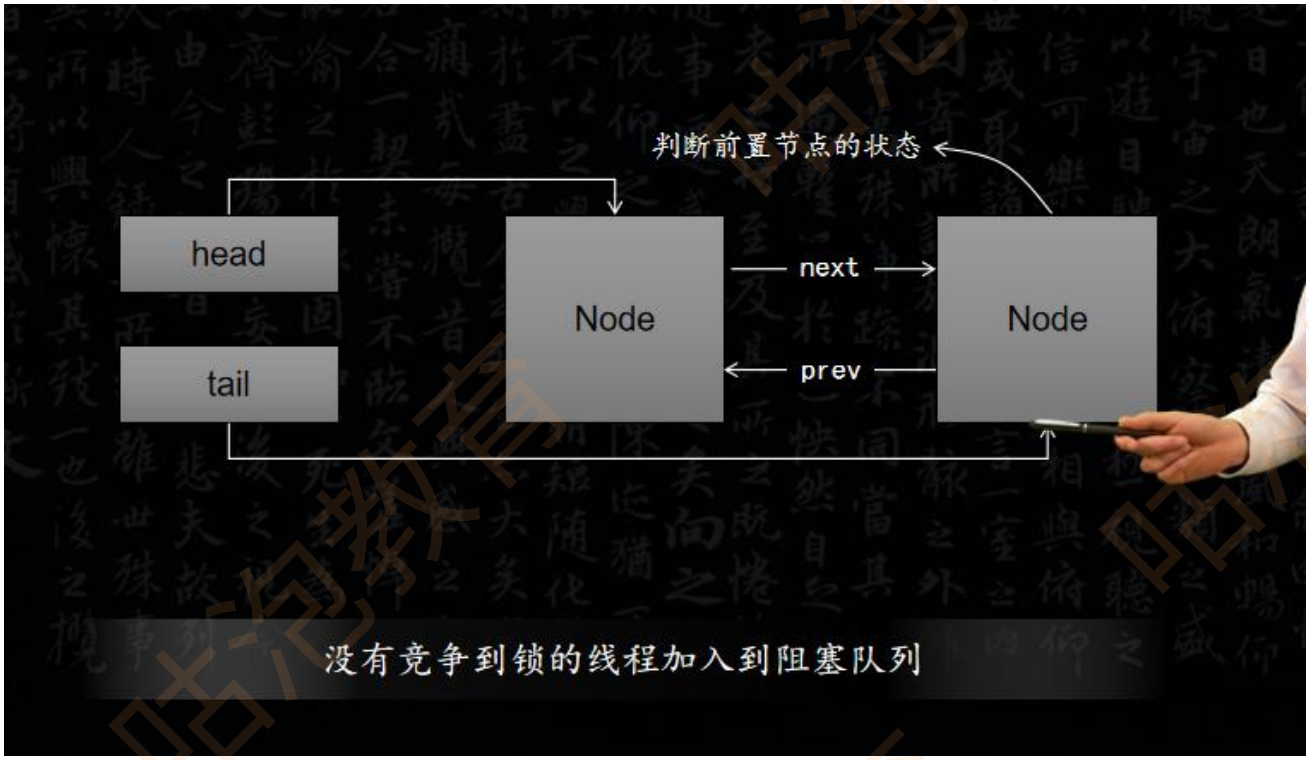
1、原因分析

首先，双向链表有两个指针，一个指针指向前置节点，一个指针指向后继节点。所以，双向链表可以支持常量 O(1) 时间复杂度的情况下找到前驱节点。因此，双向链表在插入和删除操作的时候，要比单向链表简单、高效。

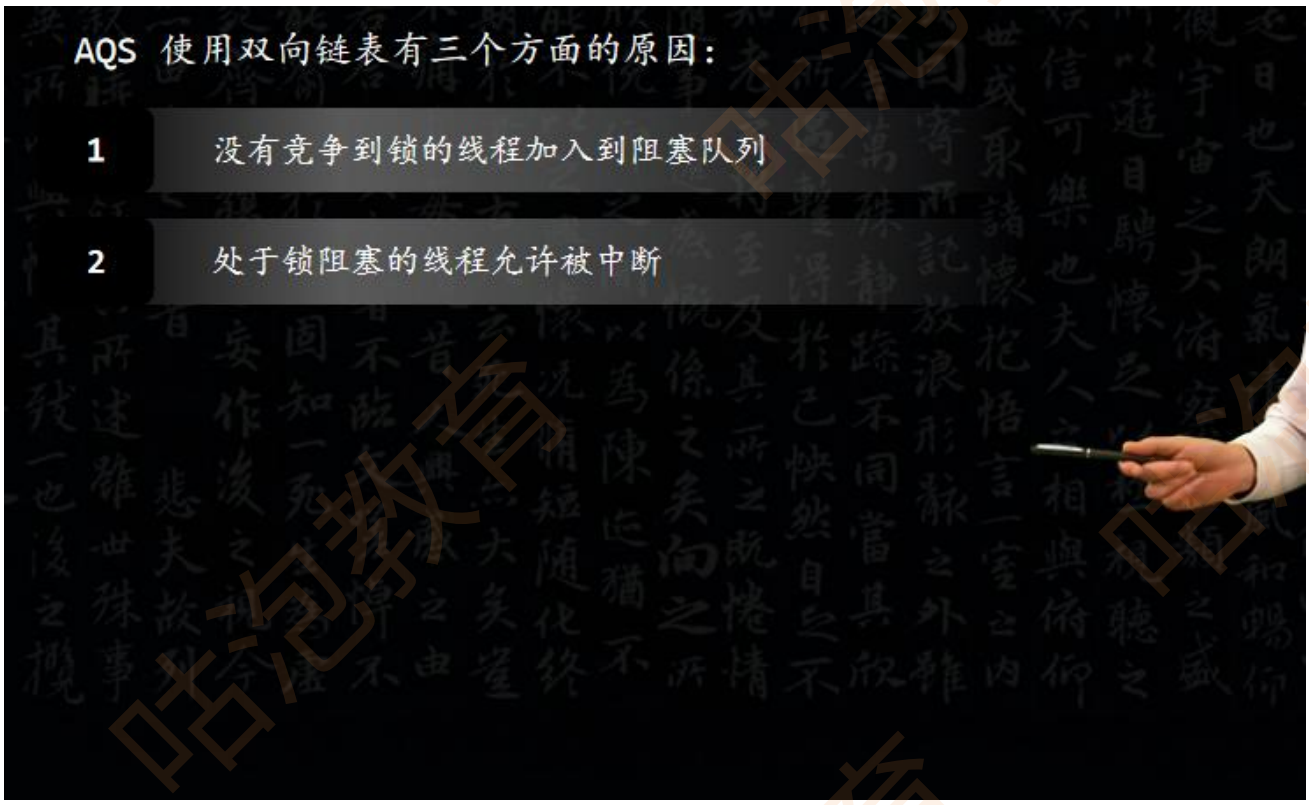
从双向链表的特性来看，我认为 AQS 使用双向链表有三个方面的原因：



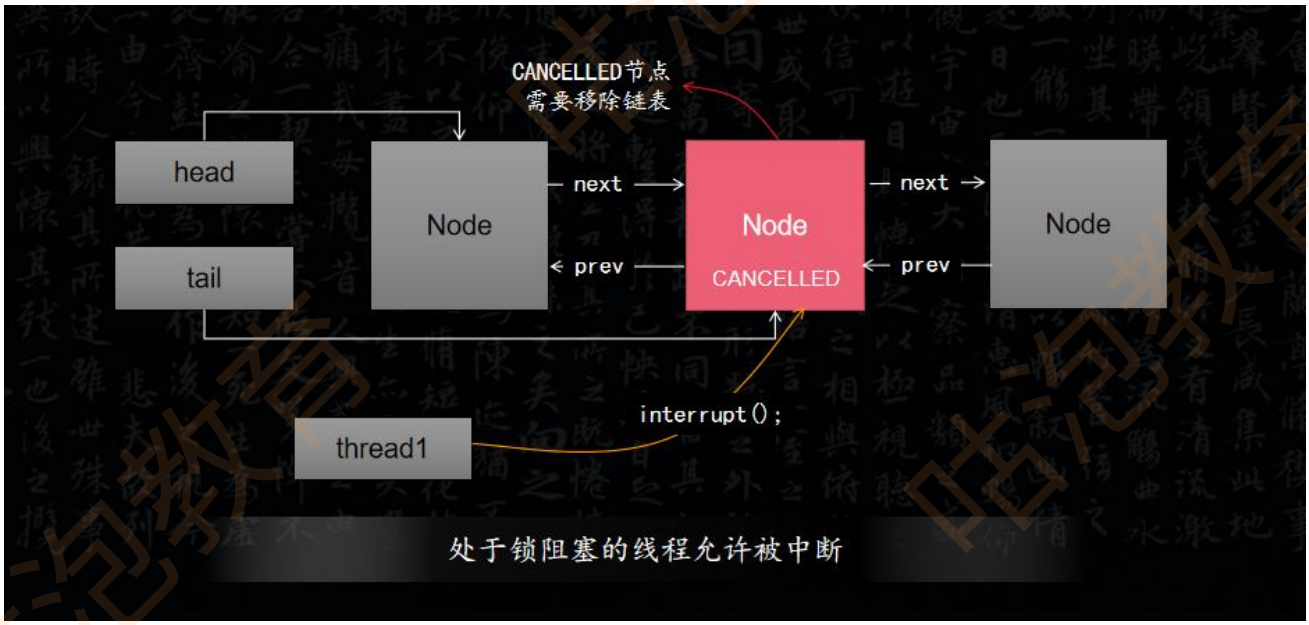
第1个原因，没有竞争到锁的线程加入到阻塞队列，并且阻塞等待的前提是，当前线程所在节点的前置节点是正常状态，这样设计是为了避免链表中存在异常线程导致无法唤醒后续线程的问题。



所以，线程阻塞之前需要判断前置节点的状态，如果没有指针指向前置节点，就需要从 Head 节点开始遍历，性能非常低。

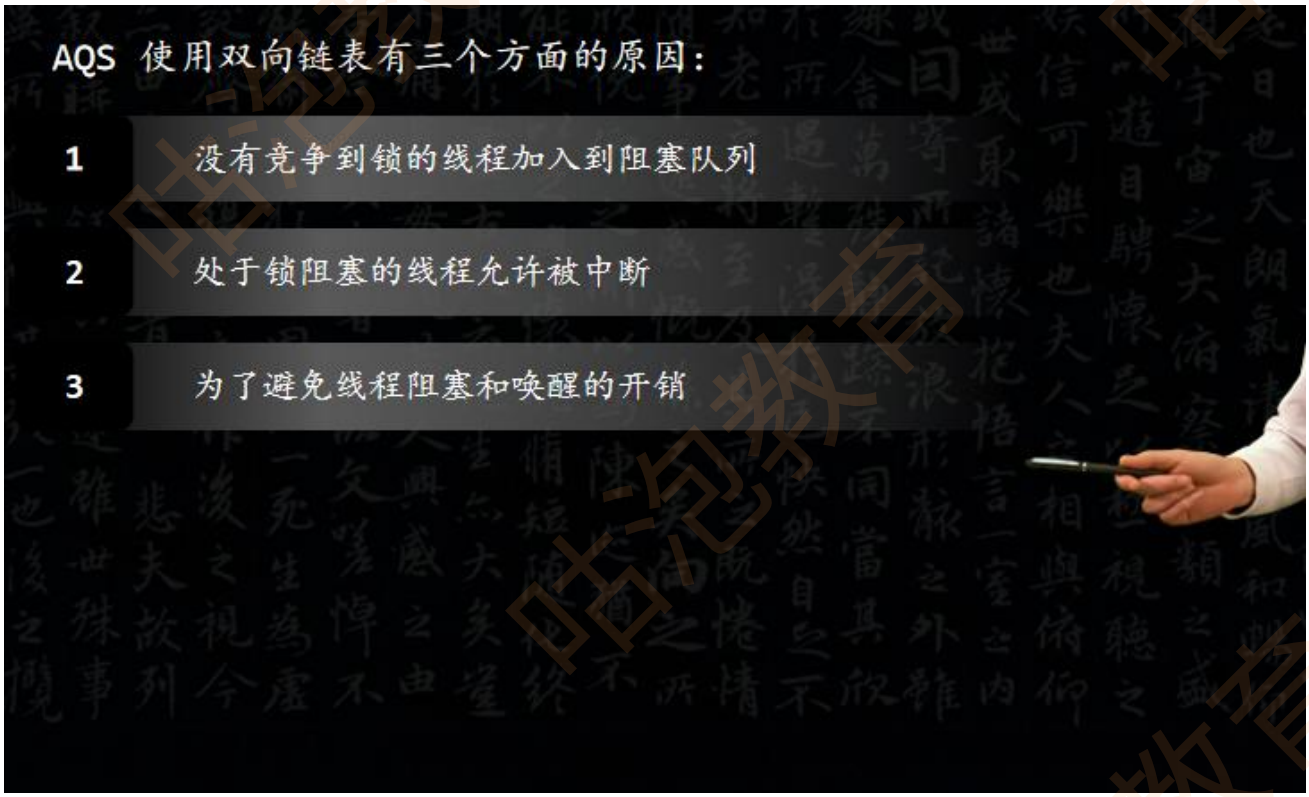


第2个原因，在 Lock 接口里面有一个，lockInterruptibly()方法，这个方法表示处于锁阻塞的线程允许被中断。

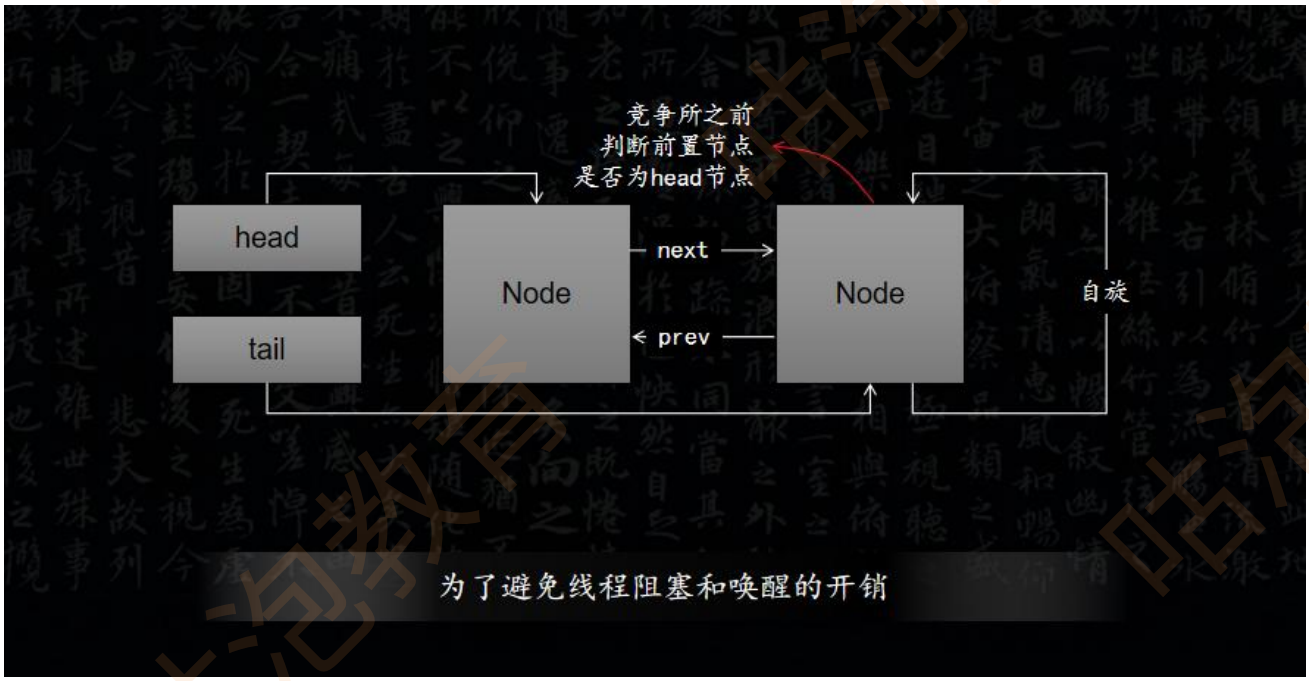


也就是说，没有竞争到锁的线程加入到同步队列等待以后，是允许外部线程通过interrupt()方法触发唤醒并中断的。这个时候，被中断的线程的状态会修改成 CANCELLED。而被标记为 CANCELLED 状态的线程，是不需要去竞争锁的，但是它仍然存在于双向链表里面。

这就意味着在后续的锁竞争中，需要把这个节点从链表里面移除，否则会导致锁阻塞的线程无法被正常唤醒。在这种情况下，如果是单向链表，就需要从 Head 节点开始往下逐个遍历，找到并移除异常状态的节点。同样效率也比较低，还会导致锁唤醒的操作和遍历操作之间的竞争。



第3个原因，是为了避免线程阻塞和唤醒的开销，所以刚加入到链表的线程，首先会通过自旋的方式尝试去竞争锁。



但是实际上按照公平锁的设计，只有头节点的下一个节点才有必要去竞争锁，后续的节点竞争锁的意义不大。否则，就会造成羊群效应，也就是大量的线程在阻塞之前尝试去竞争锁带来比较大的性能开销。

所以，为了避免这个问题，加入到链表中的节点在尝试竞争锁之前，需要判断前置节点是不是头节点，如果不是头节点，就没必要再去触发锁竞争的动作。所以这里会涉及到前置节点的查找，如果是单向链表，那么这个功能的实现会非常复杂。

【结尾】

这个问题，有可能99%的人都回答不上来。对 AQS 理解不深刻的情况下，可能不知道如何回答。理解一个技术为什么这么设计，关键在于它需要解决什么样的问题。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

80、RabbitMQ是如何实现消息路由的?

【导航条：引言】

一个应届生去面试，可能没有什么实战经验，今天被问到一个这样的面试题，说“RabbitMQ是如何实现消息路由的？ “一下子竟然不知道如何组织语言了。今天我给大家分享一下我的理解。

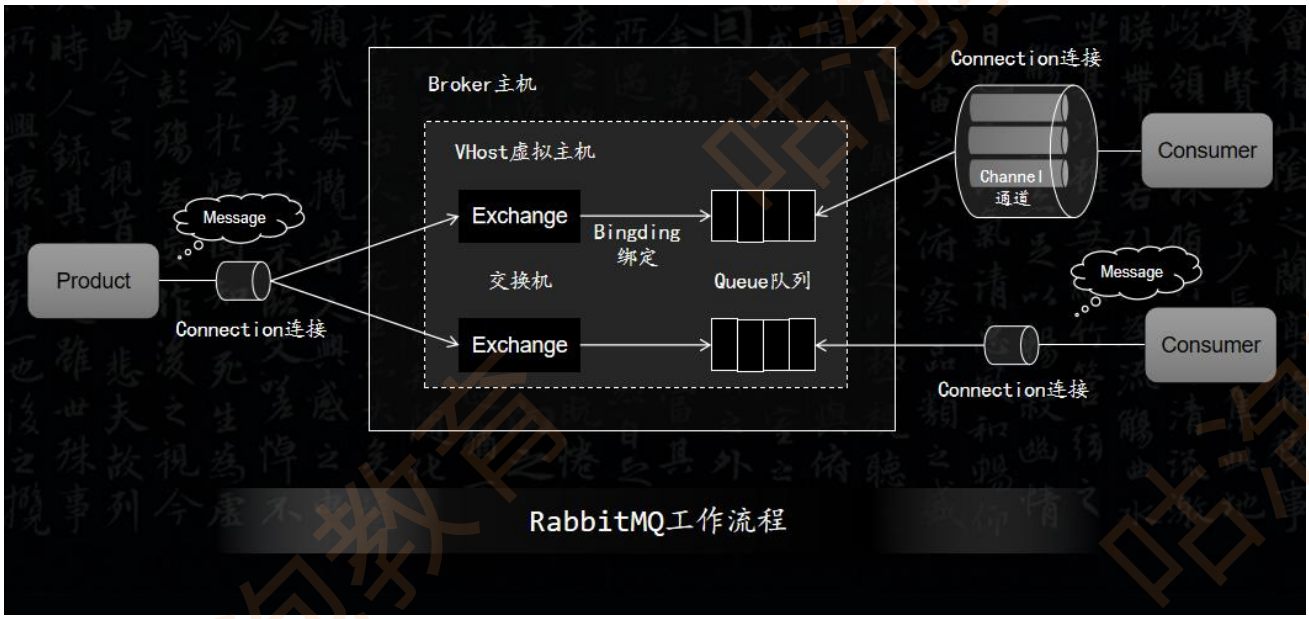
另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：工作流程】

（转场，屏幕黑底白字，正中央出现“1 工作流程）

1、工作流程

RabbitMQ 是一个基于 AMQP 协议实现的分布式消息中间件。AMQP 的具体工作机制是，



生产者把消息发送到 RabbitMQ Broker 上的Exchange 交换机上。Exchange 交换机把收到的消息根据路由规则发给绑定的队列（Queue）。最后再把消息投递给订阅了这个队列的消费者，从而完成消息的异步通讯。

其中，Exchange交换机，可以定义消息的路由规则，将消息路由到指定的队列。然后 Queue队列是消息的载体，每个消息可以根据路由规则路由到一个或者多个队列中。

【导航条：路由策略】

（转场，屏幕黑底白字，正中央出现“1 路由策略）

2、路由策略

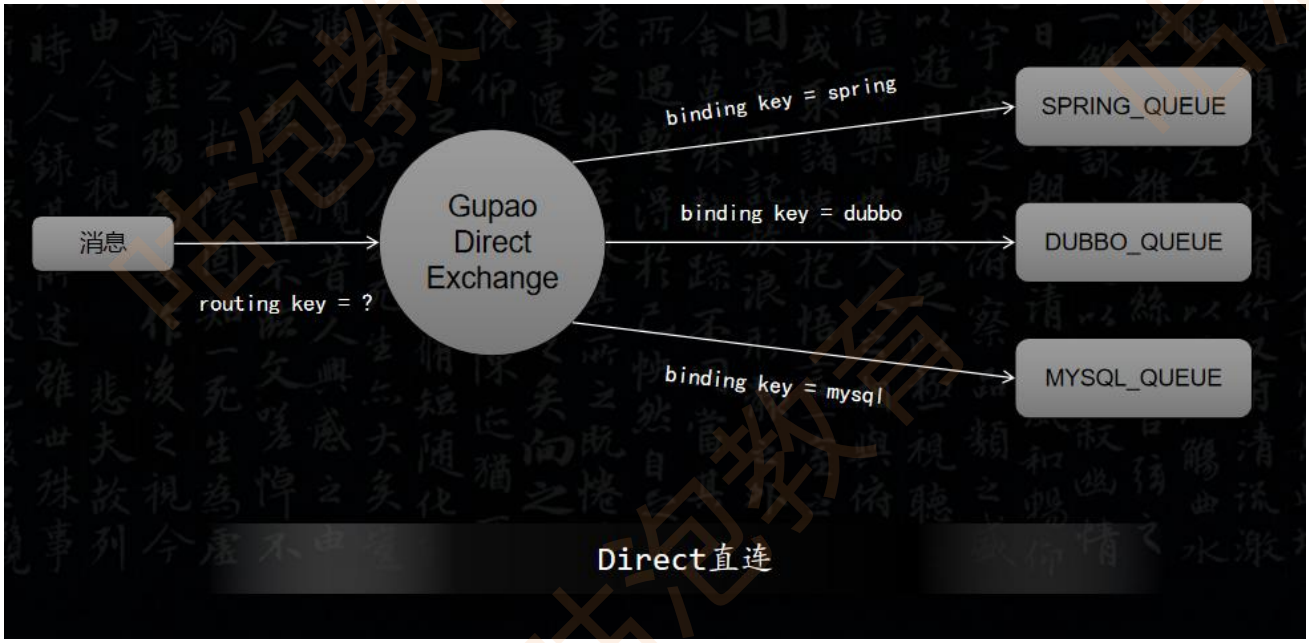
完成RabbitMQ消息路由的核心组件是 Exchange。而消息的路由是由Exchange类型 和 Binding 来决定的。Binding 表示建立 Queue 和 Exchange 之间的绑定关系，每一个绑定关系会存在一个 BindingKey。

通过这种方式相当于在 Exchange 中建立了一个路由关系表。生产者发送消息的时候，需要声明一个

RoutingKey（路由键），Exchange 拿到RoutingKey 之后，根据 RoutingKey 和路由表里面的 BindingKey 进行匹配，而匹配的规则是通过 Exchange类型来决定的。

在 RabbitMQ 中，默认有四种类型的 Exchange：Direct ，Fanout、Topic和Header。

Direct，叫直连，也就是完整匹配方式，需要Routing Key 和 Binding Key 完全一致，相当于点对点的发送。如图所示：



如果发送routing key为”spring”的消息，那么只有第一个队列能收到消息。

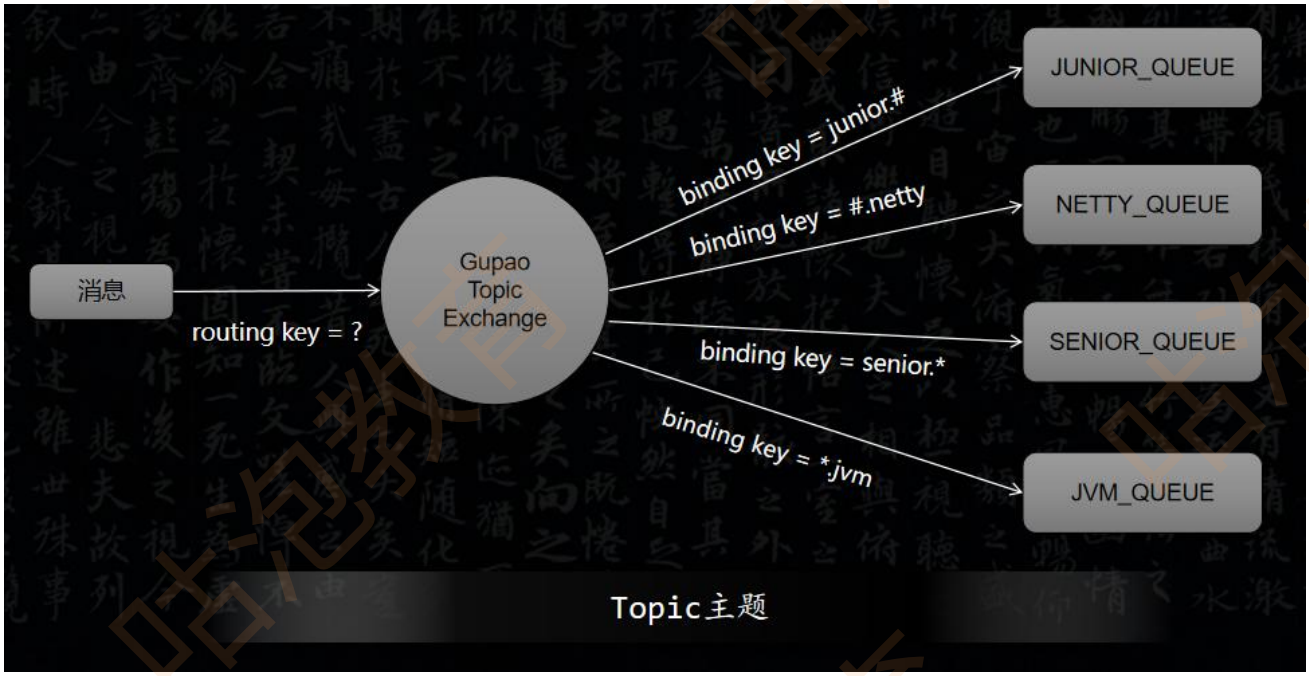
Topic： 叫主题，这种方式是通过设置通配符来动态匹配，相当于正则。就是用Routing Key 去匹配Binging Key。BingingKey支持两个通配符。

代表匹配0个或者多个单词

* 代表匹配不多不少一个单词

另外，BingingKey用点隔开两个单词。

，用*表示使用正则表达式进行匹配，如图所示：

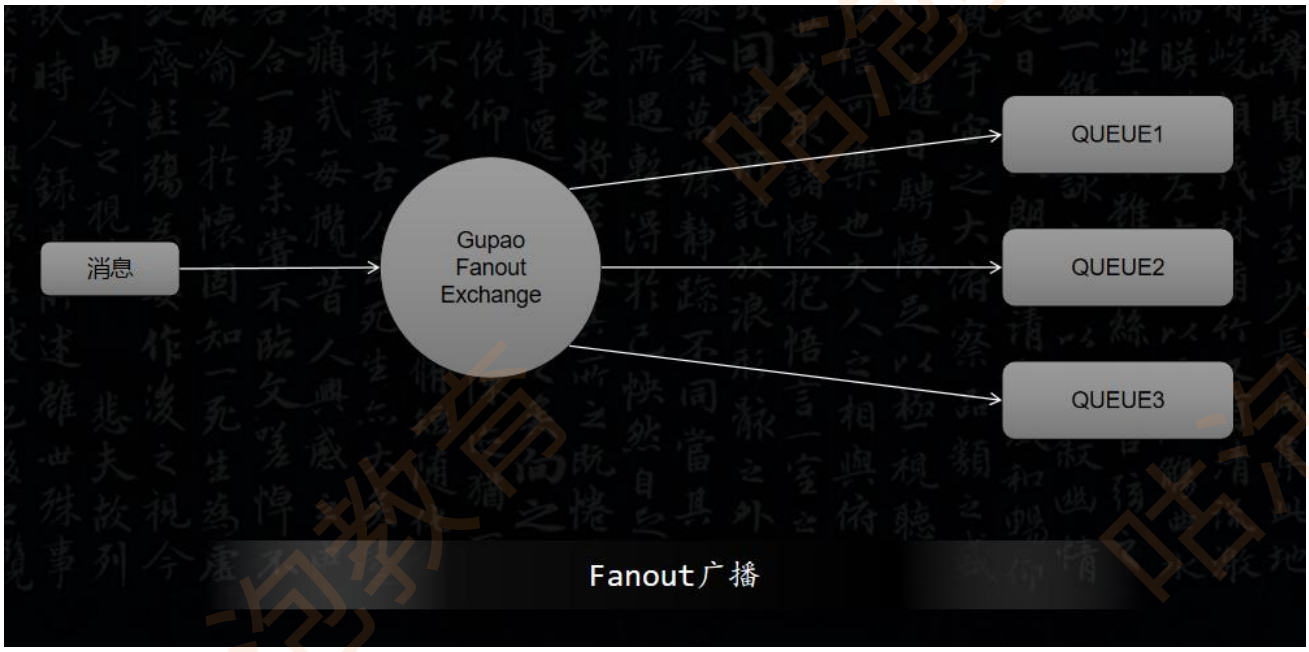


有4个队列绑定到Topic类型的交换机，而且使用不同的绑定键。

如果发送routing key为"junior.abc.jvm" 的消息，那么，只有第一个队列能收到。

如果发送routing key为"senior.netty"的消息，那么，第二个队列和第三个队列能收到。

Fanout：叫广播，这种方式不需要设置Routing Key，而是把消息广播给绑定到当前 Exchange 上的所有队列上。如图所示：



只要发送消息到Fanout Exchange上，那么三个队列都能收到消息

【结尾】

RabbitMQ、Kafka、RocketMQ 是目前最主流的分布式消息中间件。有的同学可能对 Kafka 比较了解，有的同学可能对 RabbitMQ 比较了解。不过，在面试的时候，面试官一般会问你用过的技术组件。通过面试过程中推演出你的学习能力以及对技术的掌握能力，这个方面如果还不错的话，接触一个新的 MQ 组件所消耗的学习成本会比较小。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，

面试不再难!

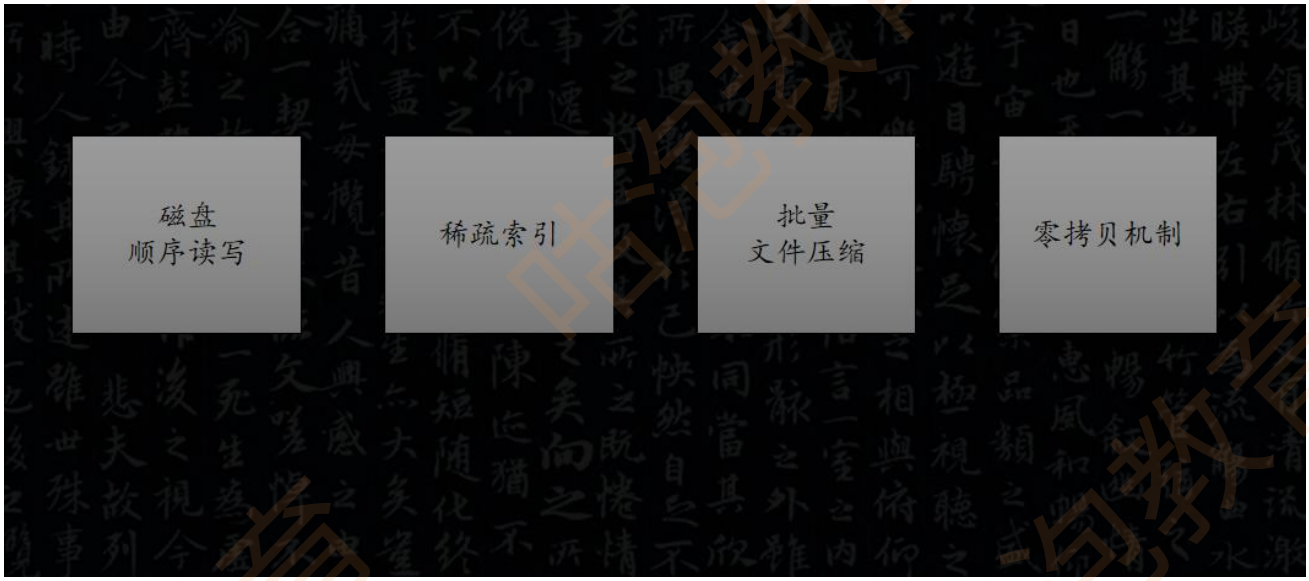
81、Kafka为什么这么快?

【导航条：引言】

最近一位小伙伴去某滴面试，在第二面的时候遇到了这个问题：说”请你简单说一下，Kafka为什么这么快？ “，然后，这位小伙伴努力在大脑里检索了很久，没有回答上来。

那么今天，我给大家分析一下，来看看对这个问题的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。



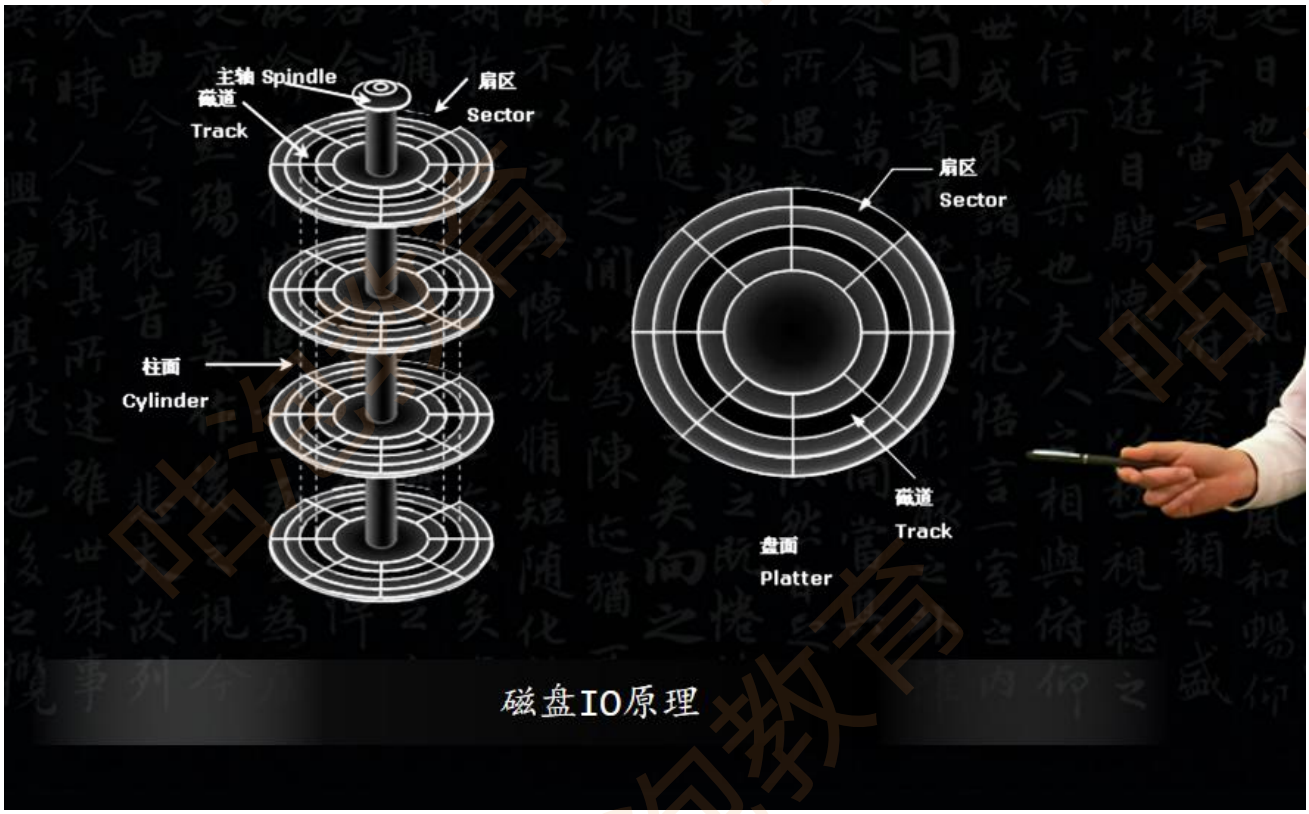
Kafka是一个号称能用普通的PC机也能处理超千万亿的消息吞吐量的实时消息流处理平台。我认为Kafka之所以能支持如此大的吞吐量，而且还能做到性能优秀主要有四个原因：分别是磁盘顺序读写、稀疏索引、批量文件压缩和零拷贝机制。下面分别给大家详细介绍一下：

【导航条：磁盘顺序读写】

（转场，屏幕黑底白字，正中央出现“1 磁盘顺序读写”）

1、磁盘顺序读写

首先要讲一下，磁盘寻址的过程，如图所示：



这个是磁盘的构造。磁盘的盘片不停地旋转，磁头会在磁盘表面画出一个圆形轨迹，这个就叫磁道。从内到位半径不同有很多磁道。然后又用半径线，把磁道分割成了扇区（两根射线之内的扇区组成扇面）。如果要读写数据，必须找到数据对应的扇区，这个过程就叫寻址。

如果读写的多条数据在磁盘上是分散的，寻址会很耗时，这叫随机I/O。

如果读写的数据在磁盘上是集中的，不需要重复寻址的过程，这叫顺序I/O。

而Kafka的Message是不断追加到本地磁盘文件末尾的，而不是随机的写入，这使得Kafka写入吞吐量得到了显著提升。

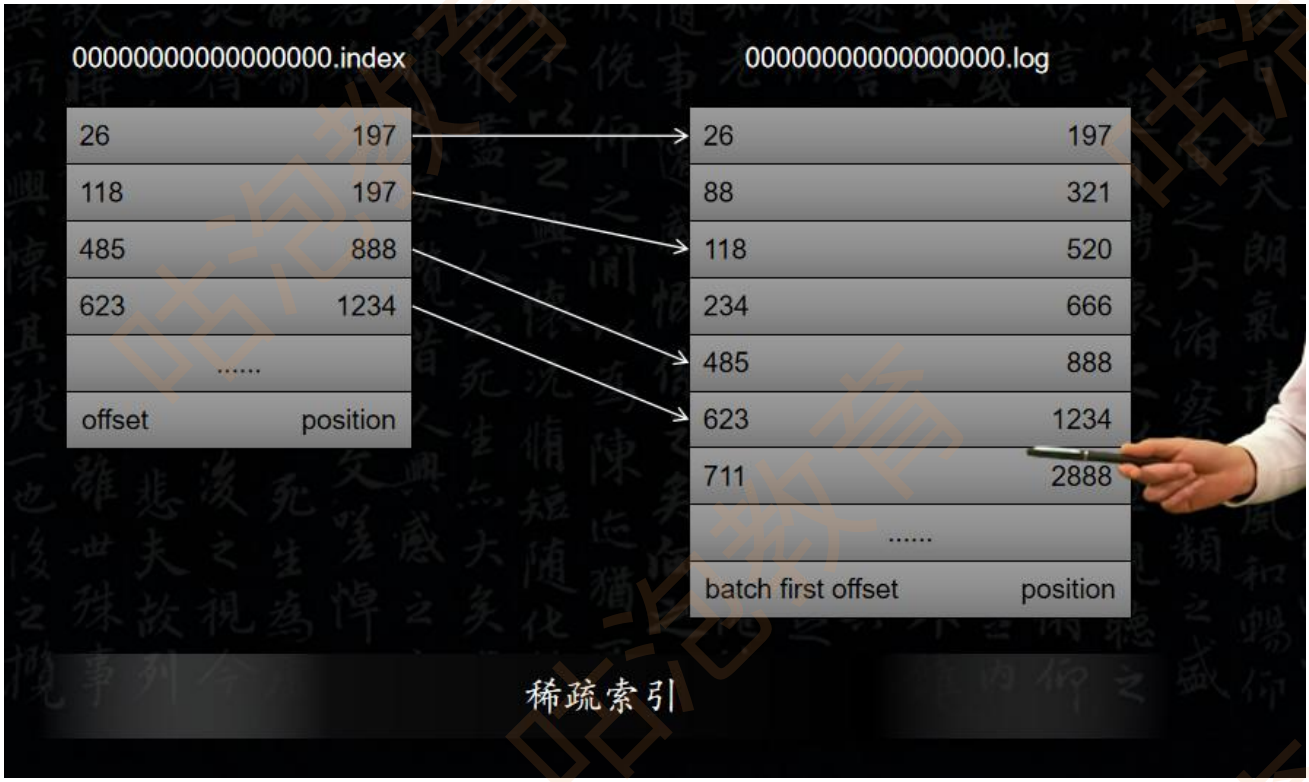
在一定条件下测试，磁盘的顺序读写可以达到53.2M每秒，比内存的随机读写还要快。

【导航条：稀疏索引】

（转场，屏幕黑底白字，正中央出现“2 稀疏索引”）

2、稀疏索引

Kafka的索引并不是每一条消息都会建立索引，而是一种稀疏索引。如图所示，



也就是说，Kafka插入一批消息才会产生一条索引记录。后续利用二分查找，可以大大提高检索效率。

【导航条：批量文件压缩】

（转场，屏幕黑底白字，正中央出现“3 批量文件压缩）

3、批量文件压缩

Kafka默认不会删除数据，它会把所有的消息都变成一个批量的文件。如图所示，它会把相同的Key合并为最后一个Value。

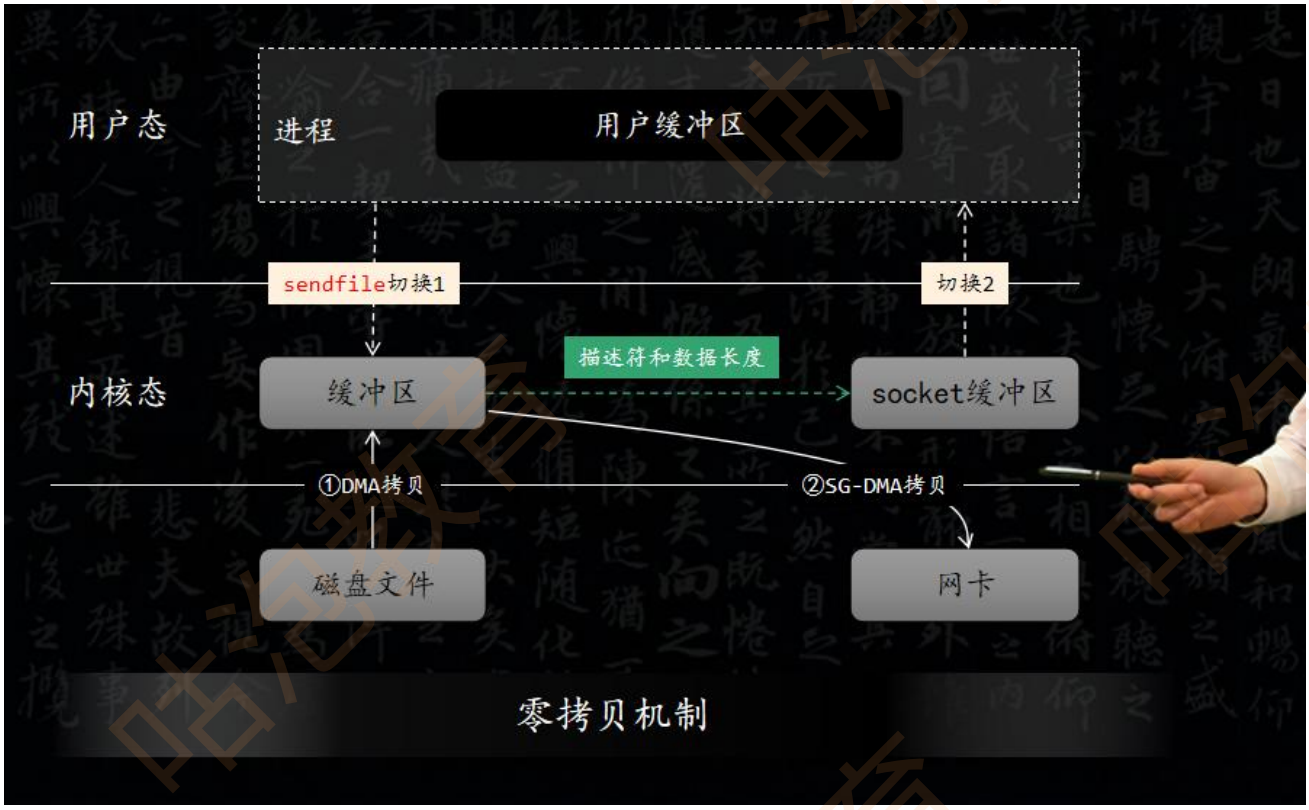
这样对消息进行合理的批量压缩，可以减少网络IO损耗。

【导航条：零拷贝机制】

（转场，屏幕黑底白字，正中央出现“4 零拷贝机制）

4、零拷贝机制

操作系统的虚拟内存分成两部分，一部分是内核空间，一部分是用户空间。这样就可以避免用户进程直接操作内核，保证内核安全。如图所示：



正常情况下，如果用户要从磁盘读取数据，必须先把数据从磁盘拷贝到内核缓冲区，然后在从内核缓冲区到用户缓冲区，最后才能返回给用户。

在Linux操作系统里面提供了一个sendfile函数，可以实现“零拷贝”。意思就是不需要经过用户缓冲区，可以直接把数据拷贝到网卡。

而Kafka中文件传输最终调用的是Java NIO 库里的 transferTo 方法，实际上最后就会使用到Linux sendfile() 系统调用函数。零拷贝技术可以大大地提升文件传输的性能。

【结尾】

这道面试题涉及到一些计算机底层的原理，基本上也是业务程序员的知识盲区。但我想给大家一个建议，做开发其实和建房子一样，要想楼层更高更稳，首先地基要打牢固。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

82.谈谈你对Kafka零拷贝原理的理解

【导航条：引言】

最近一位3年工作经验的小伙伴去某厂面试，被问到这样一个问题，说：”请你简单说一下Kafka的零拷贝原理“。然后，这位小伙伴突然愣住了，什么是零拷贝，零拷贝跟Kafka有关系吗？

那么今天，我给大家来聊一聊我对Kafka零拷贝原理的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：什么是零拷贝】

（转场，屏幕黑底白字，正中央出现“1 什么是零拷贝”）

1、什么是零拷贝？

在实际应用中，如果我们需要把磁盘中的某个文件内容发送到远程服务器上，它必须要经过几个拷贝的过程，如图所示：

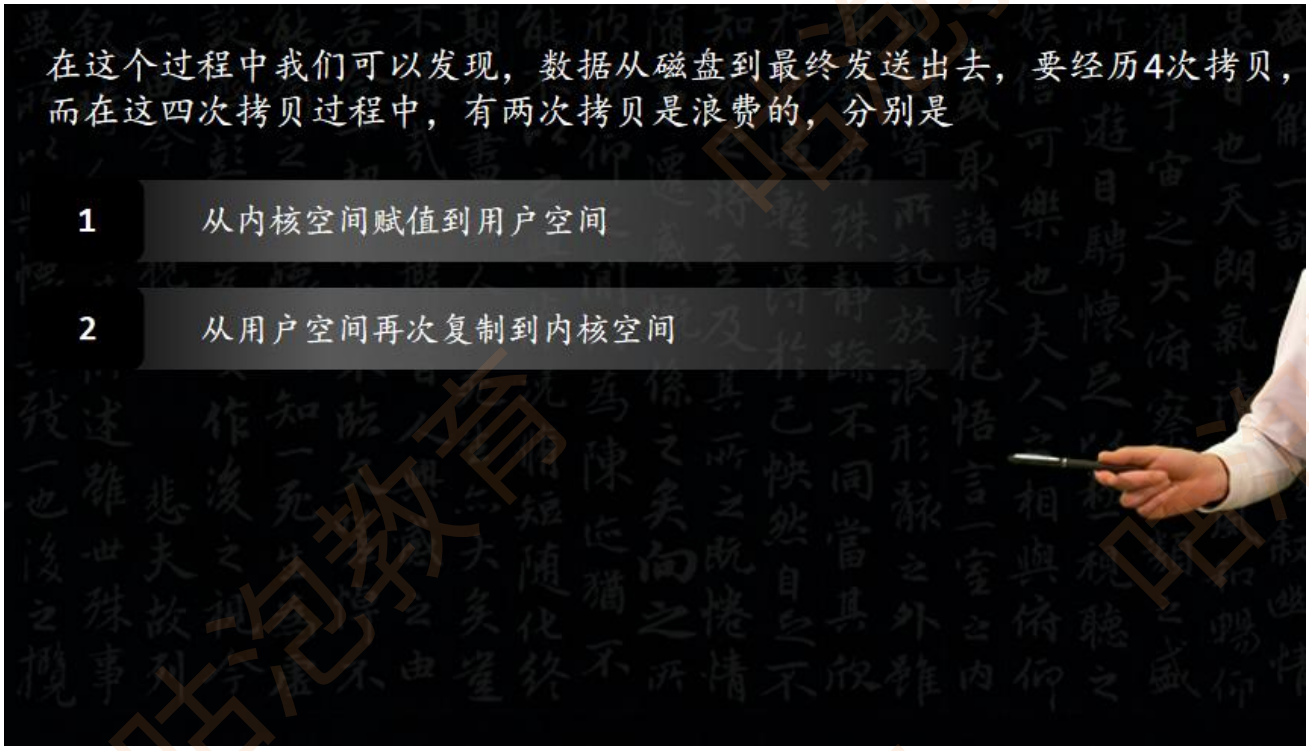
1、从磁盘中读取目标文件内容拷贝到内核缓冲区

2、CPU控制器再把内核缓冲区的数据赋值到用户空间的缓冲区中

3、接着在应用程序中，调用write()方法，把用户空间缓冲区中的数据拷贝到内核下的Socket Buffer中。

4、最后，把在内核模式下的SocketBuffer中的数据赋值到网卡缓冲区（NIC Buffer）

5、网卡缓冲区再把数据传输到目标服务器上。



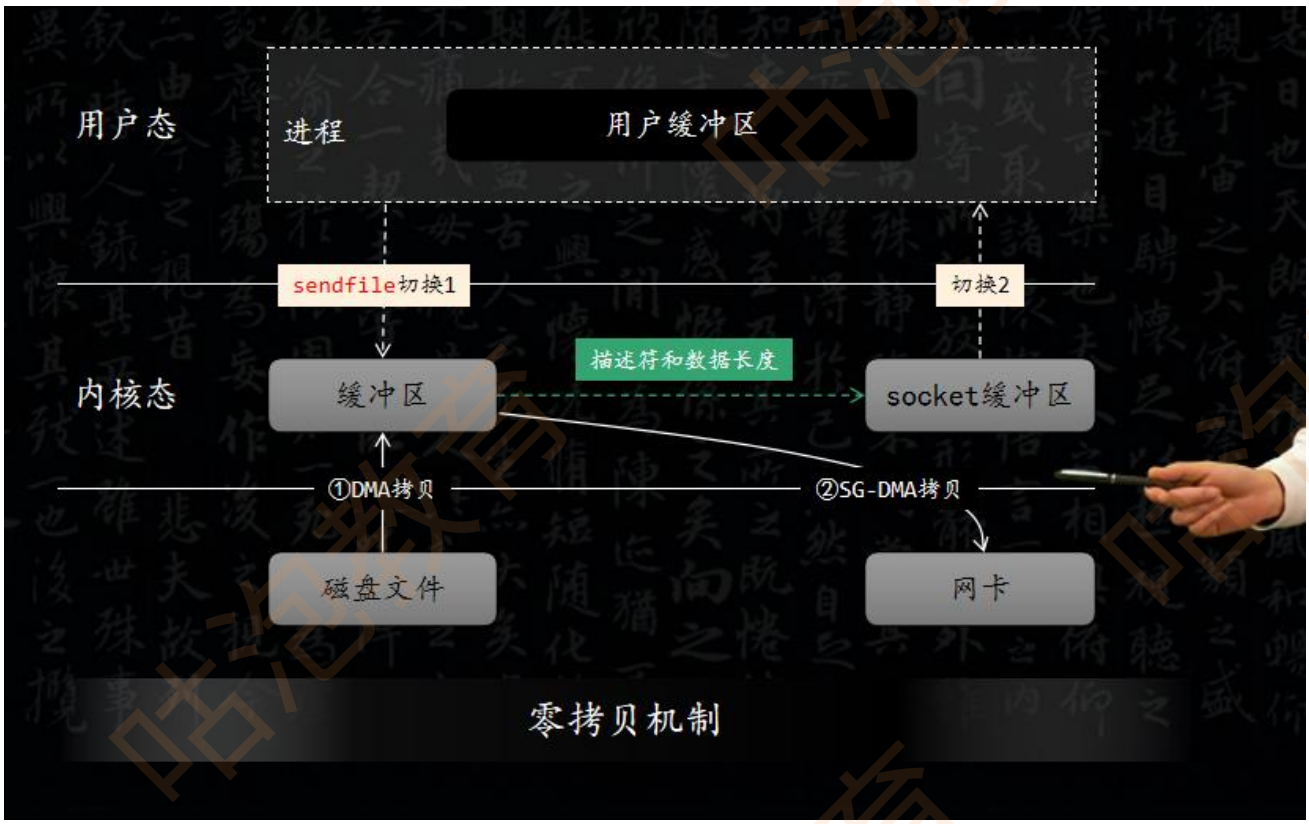
在这个过程中我们可以发现，数据从磁盘到最终发送出去，要经历4次拷贝，而在这四次拷贝过程中，有两次拷贝是浪费的，分别是：

1、从内核空间赋值到用户空间

2、从用户空间再次复制到内核空间

除此之外，由于用户空间和内核空间的切换会带来CPU的上线文切换，对于CPU性能也会造成性能影响。

而零拷贝，就是把这两次多于的拷贝省略掉，应用程序可以直接把磁盘中的数据从内核中直接传输给Socket，而不需要再经过应用程序所在的用户空间，如下图所示。



零拷贝通过DMA（Direct Memory Access）技术把文件内容复制到内核空间中的Read Buffer，

接着把包含数据位置和长度信息的文件描述符加载到Socket Buffer中，DMA引擎直接可以把数据从内核空间中传递给网卡设备。

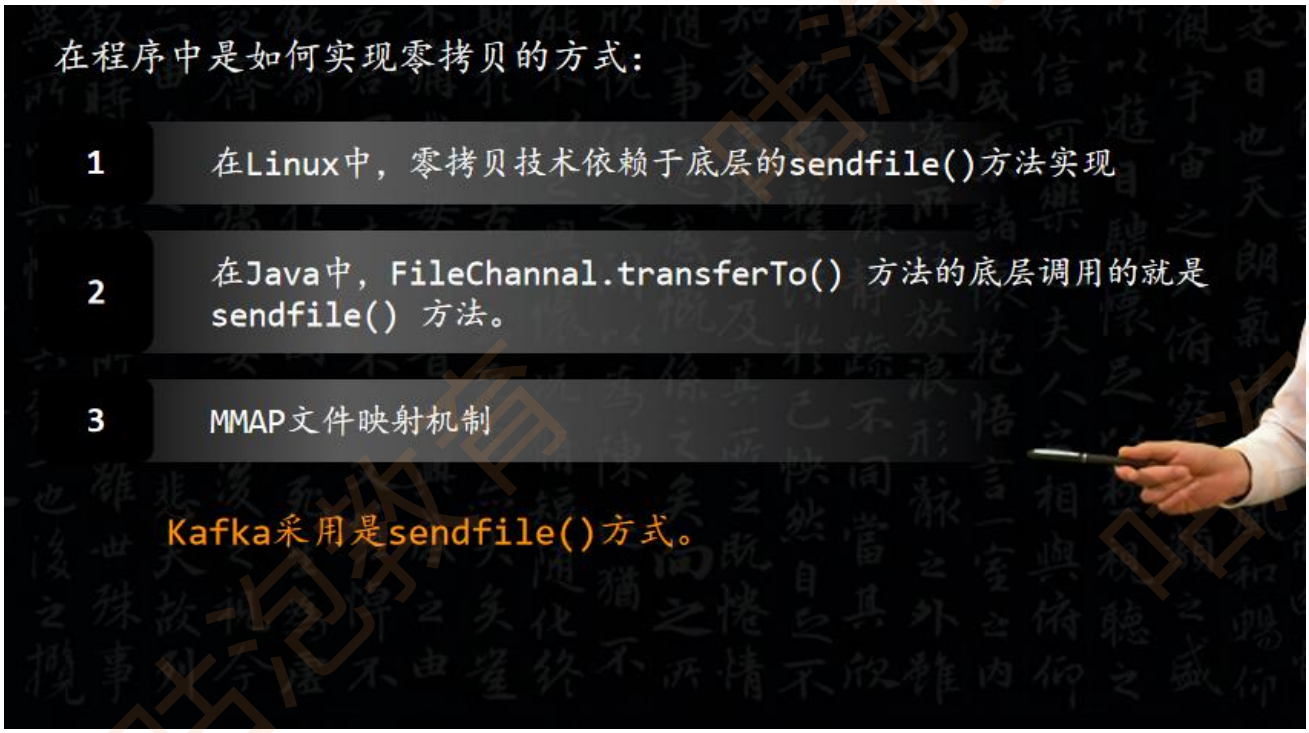
在这个流程中，数据只经历了两次拷贝就发送到了网卡中，并且减少了2次cpu的上下文切换，对于效率有非常大的提高。

【导航条：Kafka零拷贝】

（转场，屏幕黑底白字，正中央出现“2 Kafka零拷贝”）

2、Kafka零拷贝?

所以，所谓零拷贝，并不是完全没有数据赋值，只是相对于用户空间来说，不再需要进行数据拷贝。对于前面说的整个流程来说，零拷贝只是减少了不必要的拷贝次数而已。



在程序中实现零拷贝的方式有三种：

1、在Linux中，零拷贝技术依赖于底层的sendfile()方法实现

2、在Java中，FileChannal.transferTo() 方法的底层调用的就是 sendfile() 方法。

3、MMAP 文件映射机制。它的原理是，将磁盘文件映射到内存，用户通过修改内存就能修改磁盘文件。使用这种方式可以获取很大的I/O提升，省去了用户空间到内核空间复制的开销。

以上就是我对于Kafka中零拷贝原理的理解

【结尾】

本次的面试题涉及到一些计算机底层的原理，大家在平时的业务开发过程中也很少关注。但我想提醒大家，如果要在技术路上走得更远，还是要多关注一些底层的实现原理，把基础打牢固。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

面试不再难!

83.什么是聚集索引和非聚集索引

【导航条：引言】

一位去阿里面试的小伙伴，在第一面就挂了。他跟我说是被数据库里面的几个问题难倒了，他说面试官问了事务隔离级别、MVCC、聚集索引/非聚集索引、B 树、B+树这些，都没回答好。

大厂面试基本上是这样，由点到面去展开，如果你对这个技术理解不够全面，很容易就会被看出来。关于事务隔离级别、MVCC、B树、B+树在往期视频中我都有讲解过。那今天，我给大家来聊一聊聚集索引和非聚集索引。

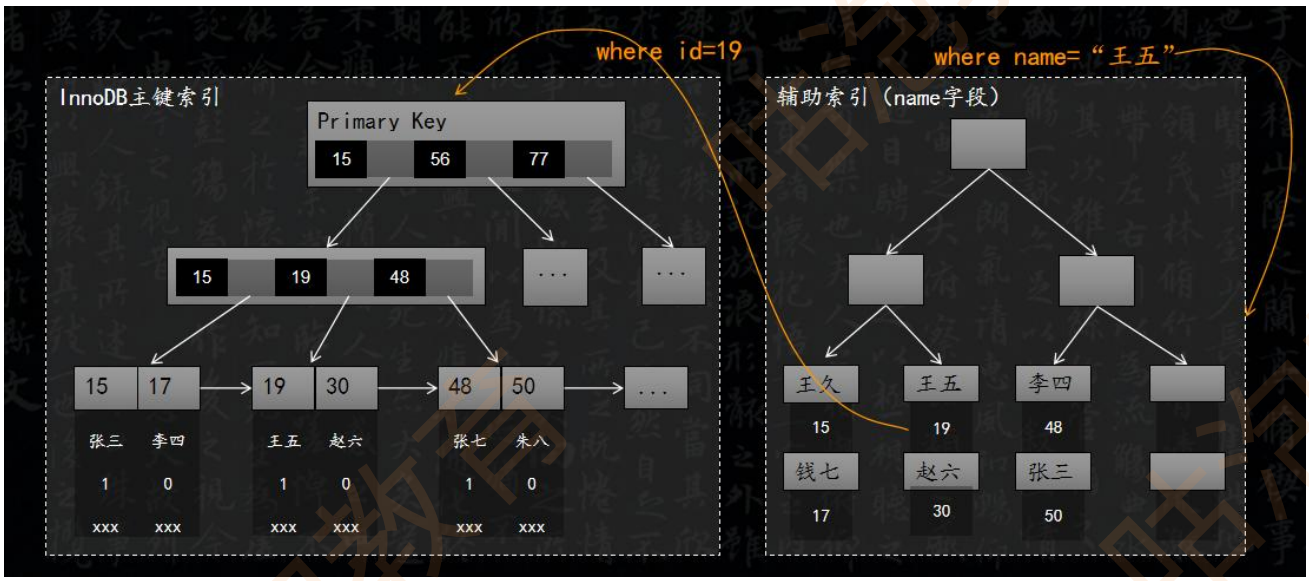
另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：聚集索引和非聚集索引】

（转场，屏幕黑底白字，正中央出现“什么是聚集索引和非聚集索引”）

简单来说，聚集索引就是基于主键创建的索引，除了主键索引以外的其他索引，称为非聚集索引，也叫做二级索引。

由于在 InnoDB 引擎里面，一张表的数据对应的物理文件本身就是按照 B+树来组织的一种索引结构，而聚集索引就是按照每张表的主键来构建一颗 B+树，然后叶子节点里面存储了这个表的每一行数据记录。所以基于 InnoDB 这样的特性，聚集索引并不仅仅是一种索引类型，还代表着一种数据的存储方式。



同时也意味着每个表里面必须要有一个主键，如果没有主键，InnoDB 会默认选择或者添加一个隐藏列作为主键索引来存储这个表的数据行。一般情况是建议使用自增 ID 作为主键，这样的话 ID 本身具有连续性，使得对应的数据也会按照顺序存储在磁盘上，写入性能和检索性能都很高。否则，如果使用 UUID 这种随机 ID，那么在频繁插入数据的时候，就会导致随机磁盘 IO，从而导致性能较低。需要注意的是，InnoDB 里面只能存在一个聚集索引，原因很简单，如果存在多个聚集索引，那么意味着这个表里面的数据存在多个副本，造成磁盘空间的浪费，以及数据维护的困难。

由于在 InnoDB 里面，主键索引表示的是一种数据存储结构，所以如果是基于非聚集索引来查询一条完整的记录，最终还是需要访问主键索引来检索。

【结尾】

这个问题要回答好，还真不容易。涉及到 MySQL里面索引的实现原理。但是如果回答好了，就能够很好的反馈求职者的技术功底，那通过面试就比较容易了。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

84.什么是网络四元组

【导航条：引言】

有一位小伙伴被问到一个刁钻的面试题。“请你说一下网络四元组的理解”，这位小伙伴听到这个问题的时候，完全就懵了。心里在想，"这个是程序员应该懂的吗？这是要让我去做啥？造火箭吗？"

那今天呢，我给大家来聊一聊我对网络四元组的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：产生背景】

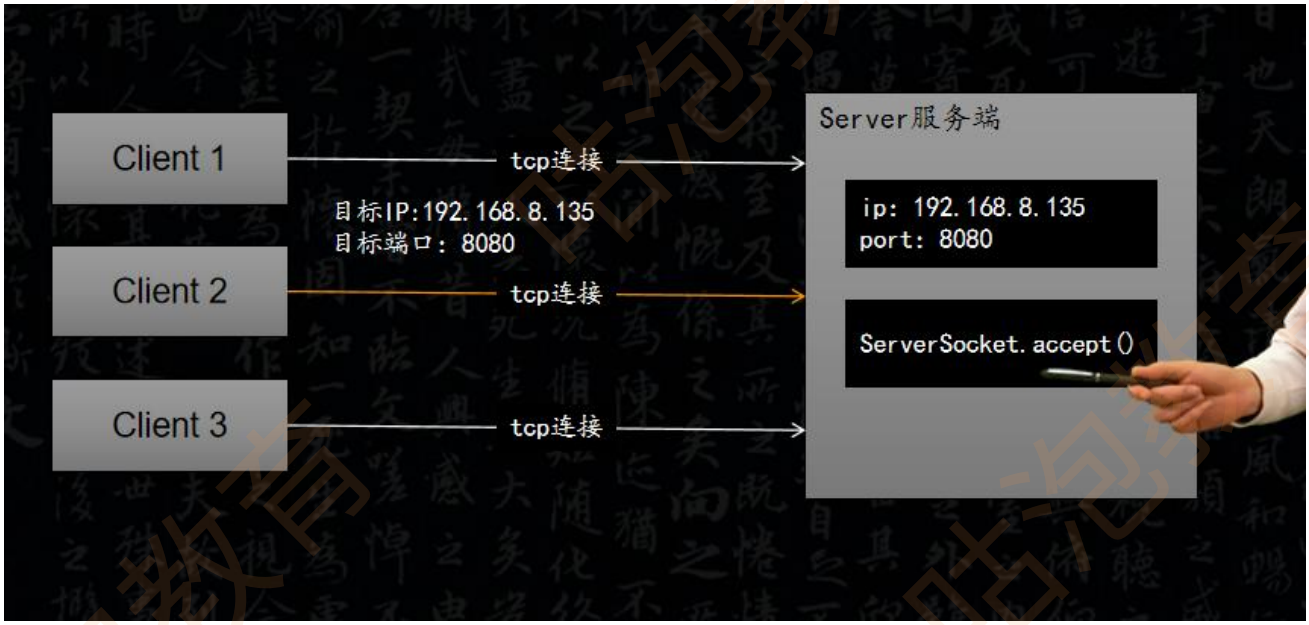
（转场，屏幕黑底白字，正中央出现“1产生背景”）

1、产生背景

四元组，简单理解就是在 TCP 协议中，去确定一个客户端连接的组成要素，它包括源 IP 地址、目标 IP 地址、源端口号、目标端口号。

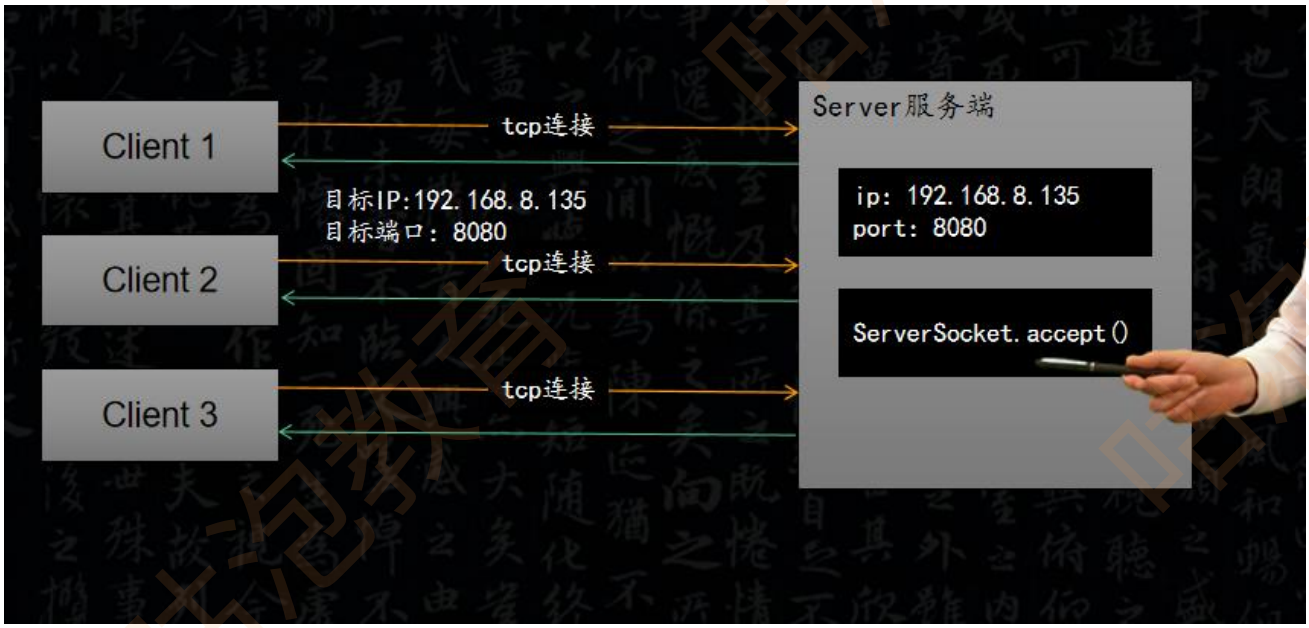
正常情况下，我们对于网络通信的认识可能是这样。服务端通过 ServerSocket 建立一个对指定端口号的监听，比如 8080。客户端通过目标 IP 和端口就可以和服务端建立一个连接，然后进行数据传输。

但是我们知道的是，一个 Server 端可以接收多个客户端的连接，比如像这种情况。



那，当多个客户端连接到服务端的时候，服务端需要去识别每一个连接。并且，TCP 是全双工协议，也就是说

数据允许在连接的两个方向上同时传输，因此这里的客户端，如果是反向通信，它又变成了服务端。



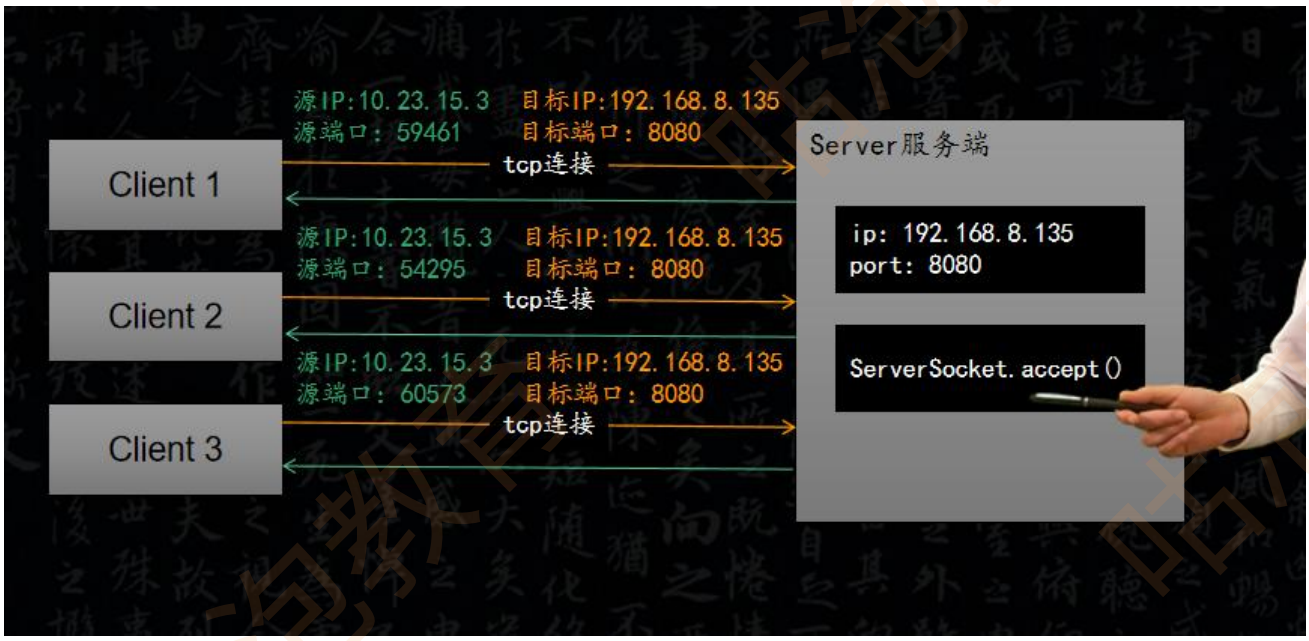
【导航条：四元组作用】

（转场，屏幕黑底白字，正中央出现“2 四元组作用”）

2、四元组作用

所以基于以上这两个原因，就引入了四元组的设计，也就是说，当一个客户端和服务端建立一个 TCP 连接的时候，通过源 IP 地址、目标 IP 地址、源端口号、目标端口号来确定一个唯一的 TCP 连接。因为服务器的 IP 和端口是不变的，只要客户端的 IP 和端口彼此不同就 OK 了。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果



比如像这种情况，同一个客户端主机上有三个连接连到 Server 端，那么这个时候源 IP 相同，源端口号不同。此时建立的四元组就是（10.23.15.3，59461，192.168.8.135，8080）。其中，源端口号是每次建立连接的时候系统自动分配的。

【结尾】

以上就是我对于四元组的理解。网络部分的知识，可能大家作为一个 CURD 工程师，觉得没必要去理解。但是未来呢？至少国内没有条件允许大家做一辈子 CRUD，所以建议大家要“终局思维”来看待自己的职业规划。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

85.什么是服务网格?

【导航条：引言】

有一位粉丝问私信问我的面试题，他说“什么是服务网格”？

服务网格这个概念出来很久了，从 2017 年被提出来，到 2018 年正式爆发，很多云厂商和互联网企业都在纷纷向服务网格靠拢。像蚂蚁集团、美团、百度、网易等一线互联网公司，都有服务网格的落地应用。

今天我给大家来详细聊一聊。另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：服务网格】

（转场，屏幕黑底白字，正中央出现“2 服务网格”）

1、服务网格

我认为，服务网格是微服务架构的更进一步升级，它的核心目的是实现网络通信与业务逻辑的分离，使得开发人员更加专注在业务的实现上。

服务网格，也就是 Service Mesh，它是专门用来处理服务通讯的基础设施层。它的主要功能是处理服务之间的通信，并且负责实现请求的可靠性传递。Service Mesh，我们通常把他称为第三代微服务架构，既然是第三代，那么意味着他是在原来的微服务架构下做的升级。

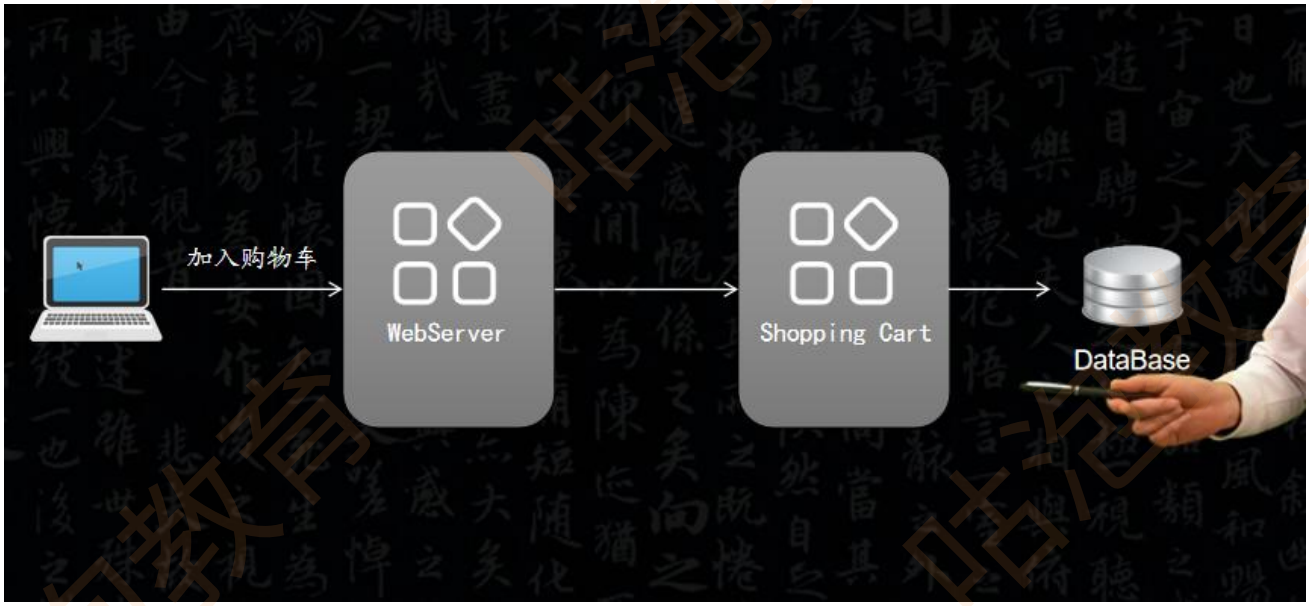
【导航条：演变历史】

（转场，屏幕黑底白字，正中央出现“2 演变历史”）

2、演变历史

为了更好的说明 Service Mesh，那我就不得不说一下微服务架构部分的东西。首先，当我们把一个电商系统以微服务化架构进行拆分后，会的到这样的一个架构，其中包括 WebServer、Payment、inventory 等等。

这些微服务应用，会被部署到 Docker 容器、或者 Kubernetes 集群。由于每个服务的业务逻辑是独立的，比如 payment 会实现支付的业务逻辑、order 实现订单的处理、Webserver 实现客户端请求的响应等。



所以，服务之间必须要相互通信，才能实现功能的完整性。比如用户把一个商品加入购物车，请求会进入到 Webserver，然后转发到 shopping cart 进行处理，并存到数据库。

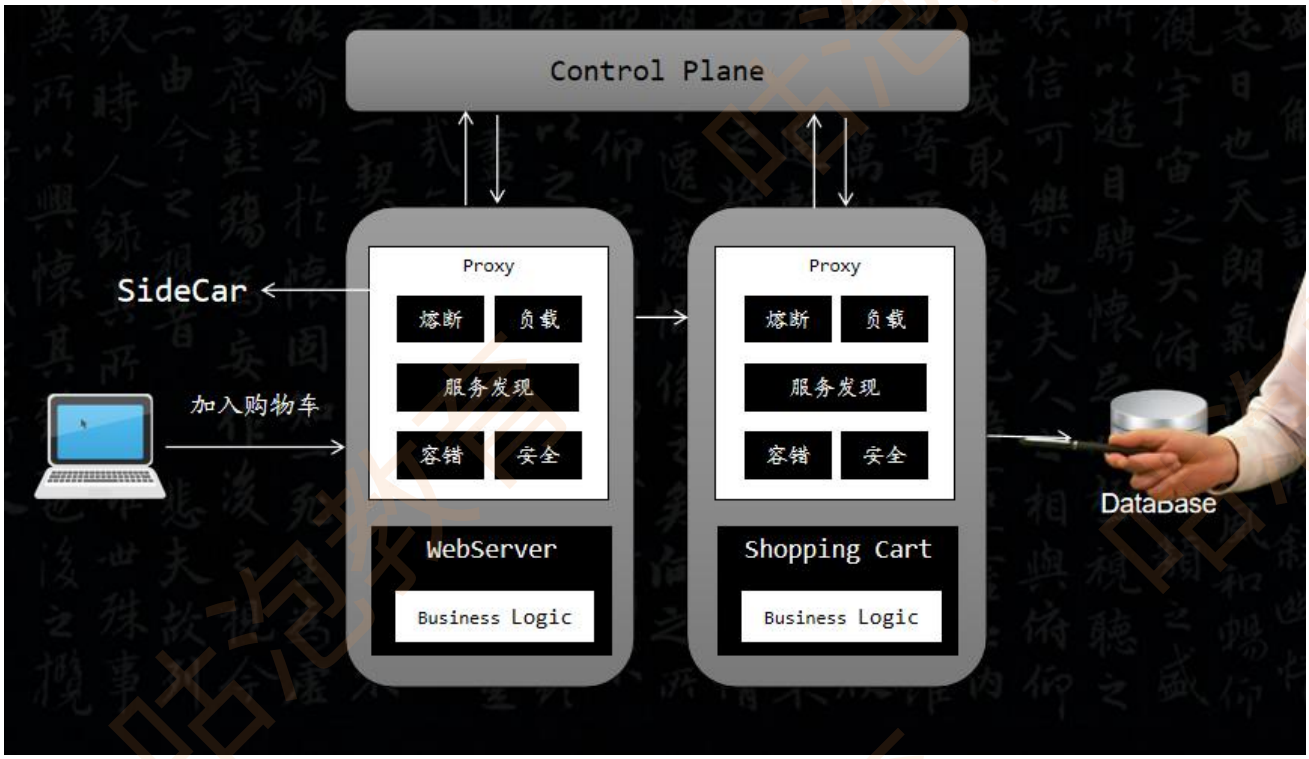
而在这个过程中，每个服务之间必须要知道对方的通信地址，并且当有新的节点加入进来的时候，还需要对这些

通信地址进行动态维护。所以，在第一代微服务架构中，每个微服务除了要实现业务逻辑以外，还需要解决上下游寻址、通讯、以及容错等问题。

于是，在第二代微服务架构下，引入了服务注册中心来实现服务之间的寻址，并且服务之间的容错机制、负载均衡也逐步形成了独立的服务框架，比如主流的Spring Cloud、或者 Spring Cloud Alibaba。

在第二代微服务架构中，负责业务开发的小伙伴不仅仅需要关注业务逻辑，还需要花大量精力去处理微服务中的一些基础性配置工作，虽然 Spring Cloud 已经尽可能去完成了这些事情，但对于开发人员来说，学习 Spring Cloud，以及针对Spring Cloud 的配置和维护，仍然存在较大的挑战。另外呢，也增加了整个微服务的复杂性。

实际上，我认为，“微服务中所有的这些服务注册、容错、重试、安全等工作，都是为了保证服务之间通信的可靠性”。于是，就有了第三代微服务架构，Service Mesh。



原本模块化到微服务框架里的微服务基础能力，被进一步的从一个 SDK 中演进成了一个独立的代理进程-SideCar。

SideCar 的主要职责就是负责各个微服务之间的通信，承载了原本第二代微服务架构中的服务发现、调用容错、服务治理等功能。使得微服务基础能力和业务逻辑迭代彻底解耦。

之所以我们称 Service Mesh 为服务网格，是因为在大规模微服务架构中，每个服务的通信都是由 SideCar 来代理的，各个服务之间的通信拓扑图，看起来就像一个网格形状。

Istio 是目前主流的 Service Mesh 开源框架。

【结尾】

以上就是我对服务网格的理解。Service Mesh 架构其实就是云原生时代的微服务架构，对于大部分企业来说，仍然是处在第二代微服务架构下。

所以，很多小伙伴不一定能够知道。不过，技术是在快速迭代的，有一句话叫“时代抛弃你的时候，连一句再见也不会说”，就像有些人在外包公司干了 10 多年再出来面试，发现很多公司要求的技术栈，他都不会。所以，建议大家要时刻刷新自己的能力，保持竞争优势！

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

86.CPU突然飙高，系统反应慢怎么排查?

【导航条：引言】

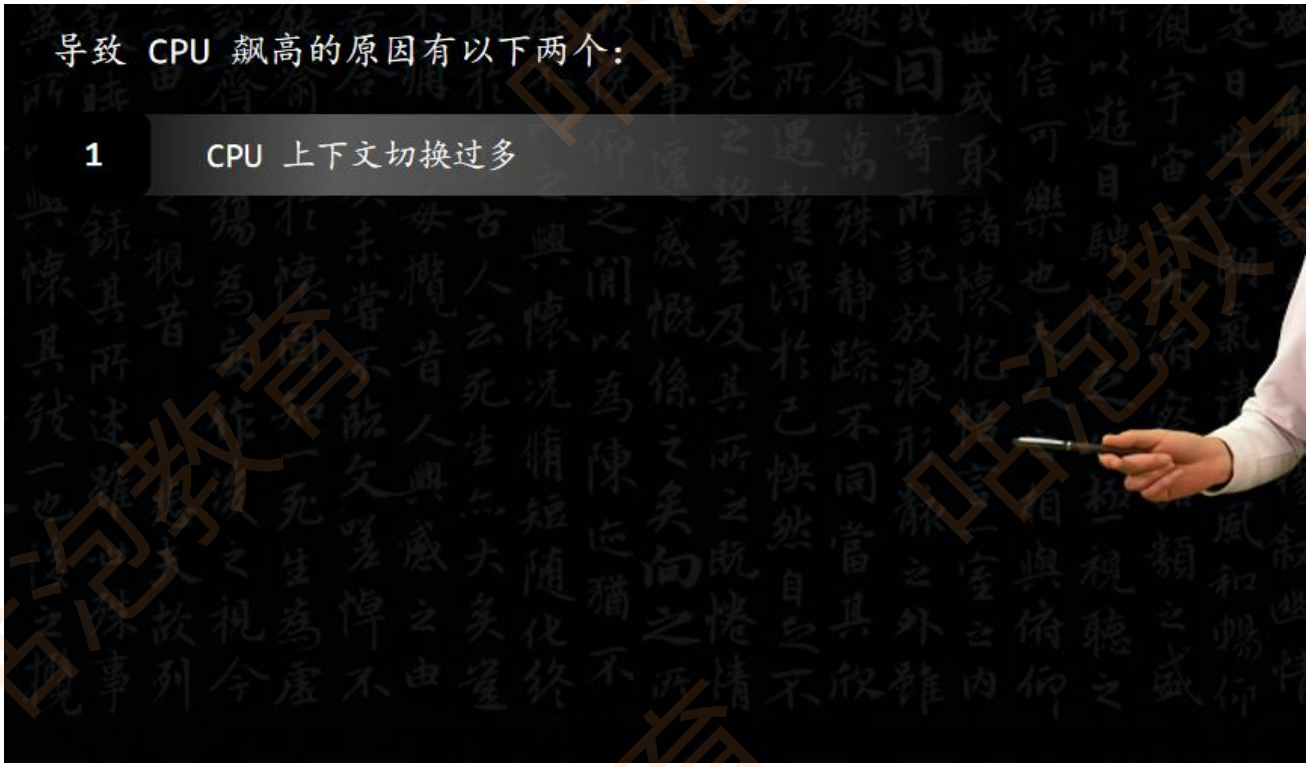
面试过程中，场景类的问题更容易检测出一个开发人员的基本能力。这不，有一位小伙伴去阿里面试，第一面就遇到了关于“CPU 飙高系统反应慢怎么排查”的问题？当时这位小伙伴不知从何下手。

今天，我给大家分享一下我的解决思路。另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：排查思路】

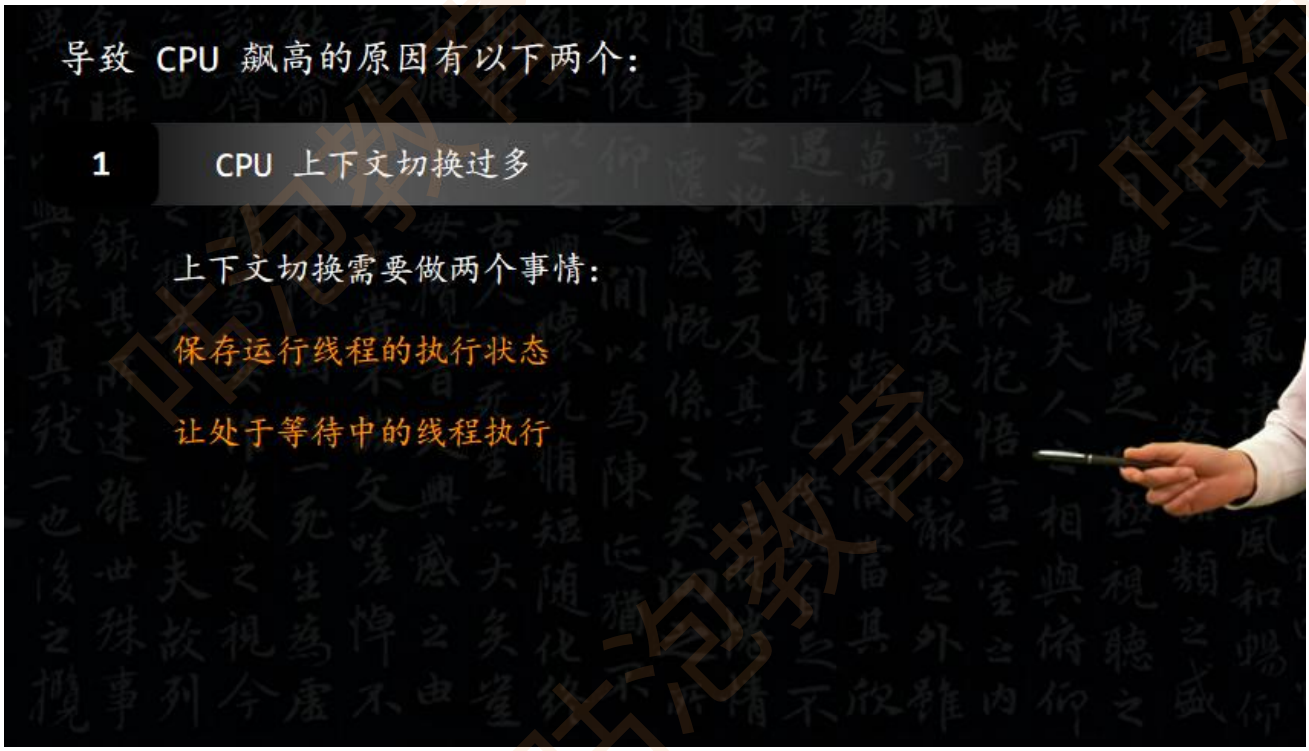
（转场，屏幕黑底白字，正中央出现“排查思路”）

CPU 是整个电脑的核心计算资源，对于一个应用进程来说，CPU 的最小执行单元是线程。导致 CPU 飙高的原因有以下两个：



1、CPU 上下文切换过多。

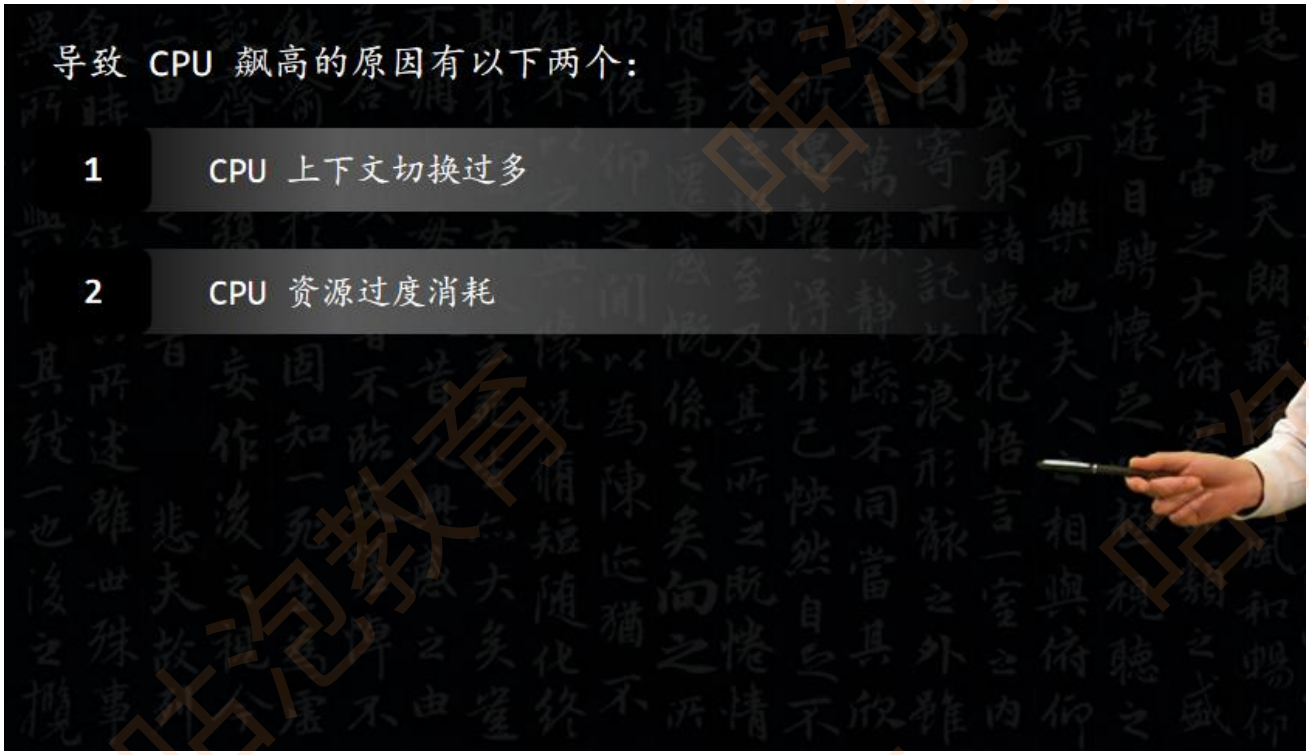
对于 CPU 来说，同一时刻下每个 CPU 核心只能运行一个线程，如果有多个线程要执行，CPU 只能通过上下文切换的方式来执行不同的线程。上下文切换需要做两个事情



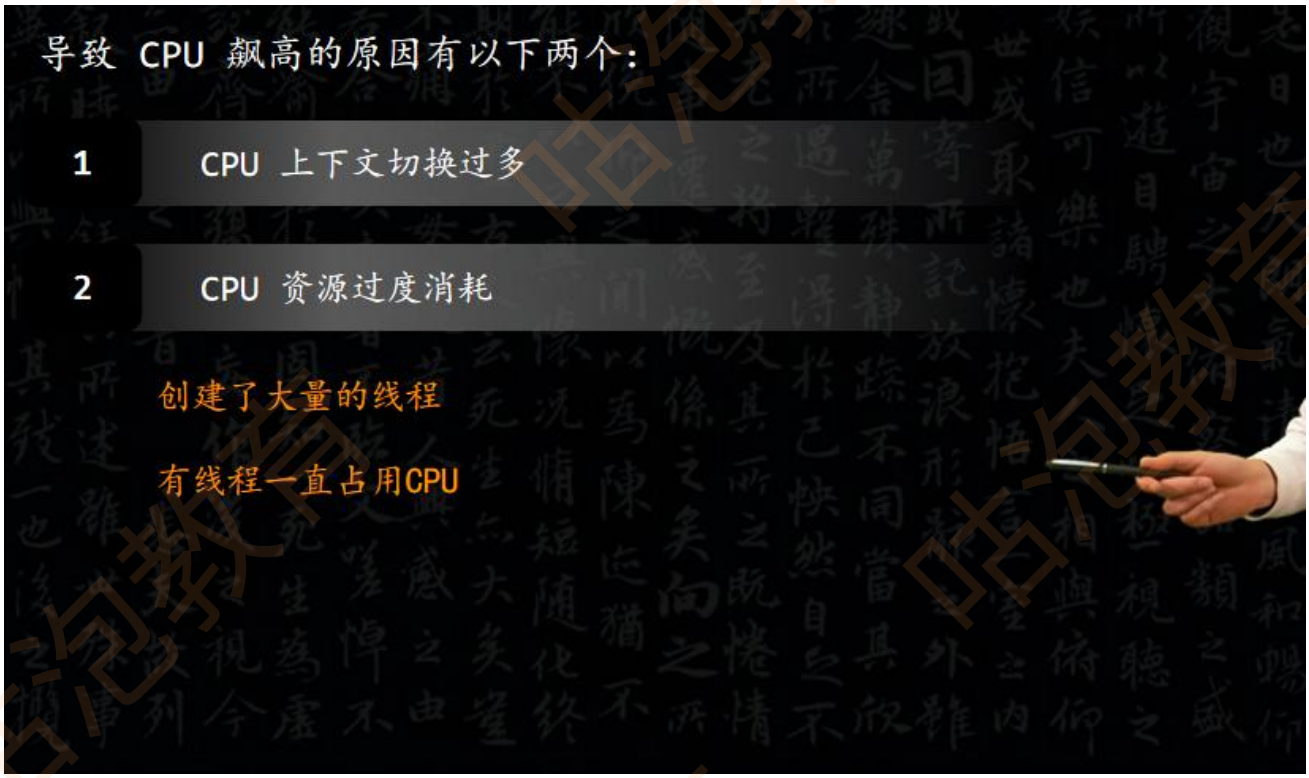
保存运行线程的执行状态

让处于等待中的线程执行

这两个过程需要 CPU 执行内核相关指令实现状态保存，如果较多的上下文切换会占据大量CPU 资源，从而使得 CPU 无法去执行用户进程中的指令，导致响应速度下降。在 Java 中，文件 IO、网络 IO、锁等待、线程阻塞等操作都会造成线程阻塞从而触发上下文切换。


导致 CPU 飙高的原因有以下两个:
1 CPU 上下文切换过多
2 CPU 资源过度消耗

2、CPU 资源过度消耗。


导致 CPU 飙高的原因有以下两个:
1 CPU 上下文切换过多
2 CPU 资源过度消耗
创建了大量的线程
有线程一直占用CPU

也就是在程序中创建了大量的线程，或者有线程一直占用CPU 资源无法被释放，比如死循环！CPU 利用率过高之后，导致应用中的线程无法获得 CPU 的调度，从而影响程序的执行效率！既然是这两个问题导致的 CPU 利用率较高，于是我们可以通过 top 命令，找到CPU 利用率较高的进程，在通过 Shift+H 找到进程中 CPU 消耗过高的线程，这里有两种情况。

CPU 利用率过高的线程一直是同一个，说明程序中存在线程长期占用 CPU 没有释放的情况，这种情况直接通过 jstack 获得线程的 Dump 日志，定位到线程日志后就可以找到问题的代码。

CPU 利用率过高的线程 ID 不断变化，说明线程创建过多，需要挑选几个线程 ID，通过 jstack 去线程 dump 日志中排查。最后有可能定位的结果是程序正常，只是在 CPU 飙高的那一刻，用户访问量较大，导致系统资源不够。

【结尾】

以上就是我对这个问题的理解！从这个问题来看，面试官主要考察实操能力，以及解决问题的思路。如果你没有实操过，但是你知道导致 CPU 飙高这个现象的原因，并说出你的解决思路，通过面试是没问题的。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

87.谈谈你对 Seata 的理解

【导航条：引言】

Seata在大厂也是属于高频的面试题，有一位3年工作经验的小伙伴被问到一道这样的面试题，说“谈谈你对Seata的理解”。那么，今天我给大家来聊一聊。

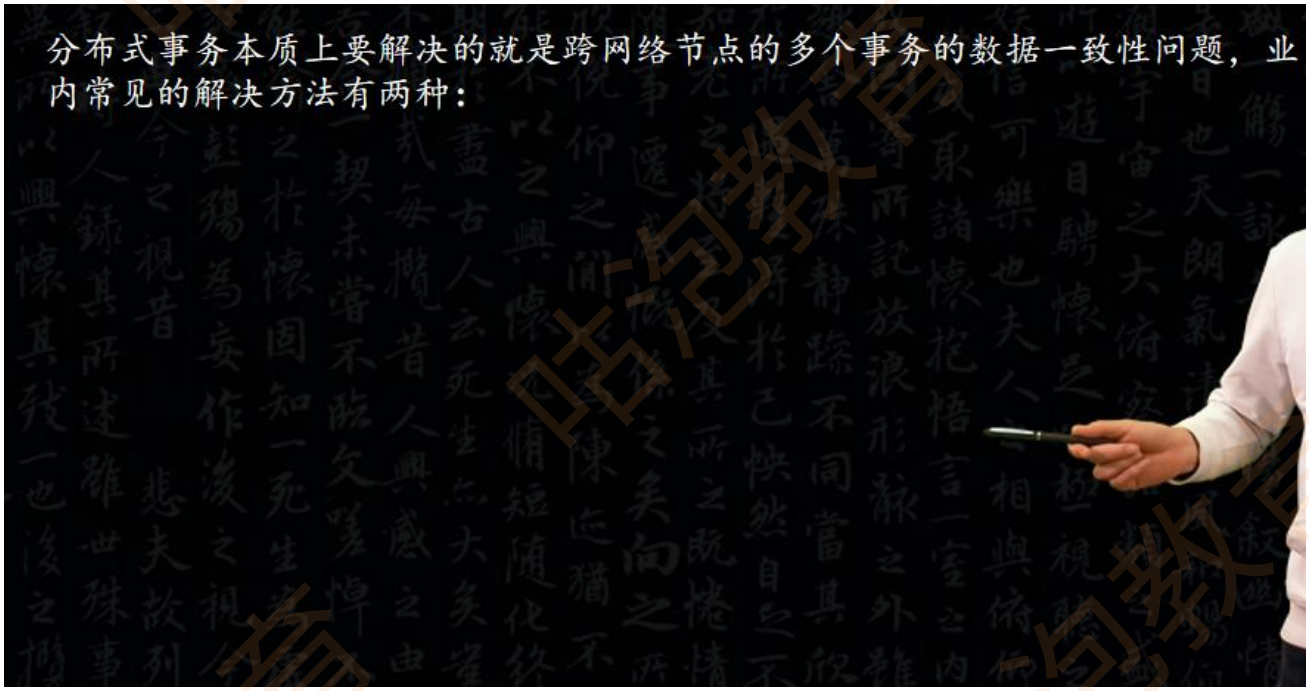
另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：Seata是什么】

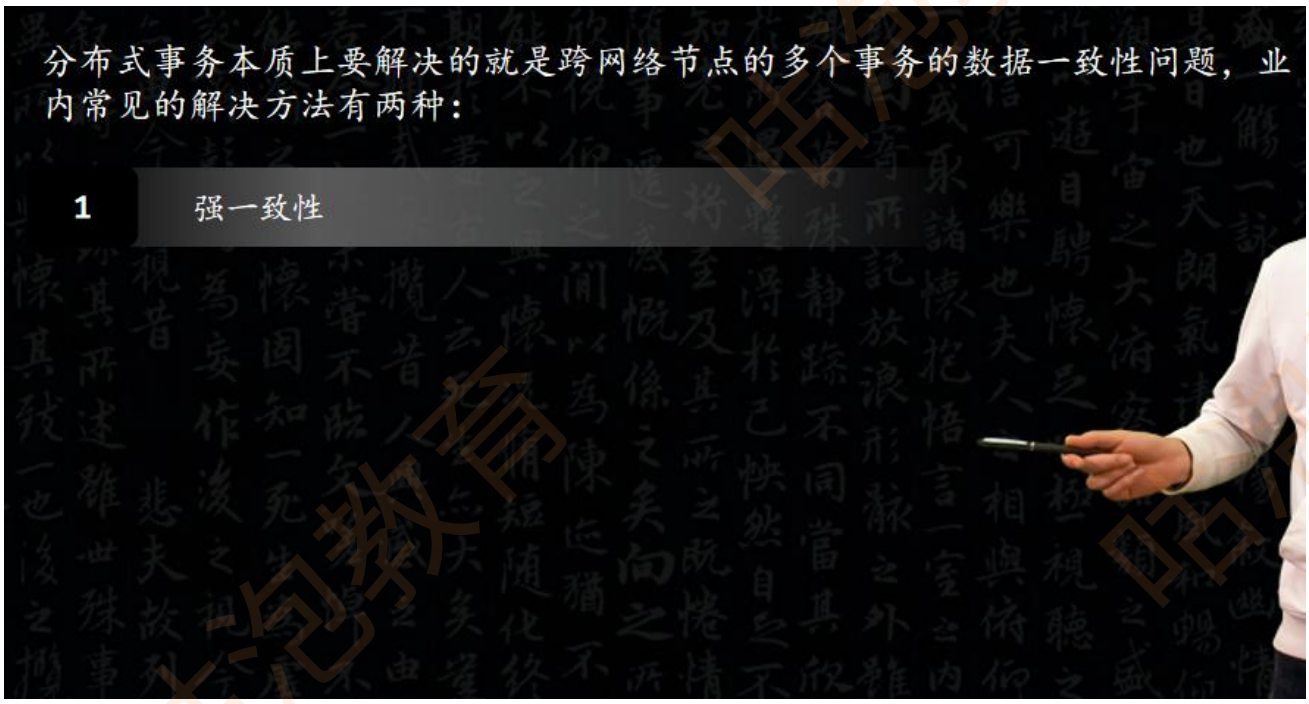
（转场，屏幕黑底白字，正中央出现“1 Seata是什么”）

1、Seata是什么

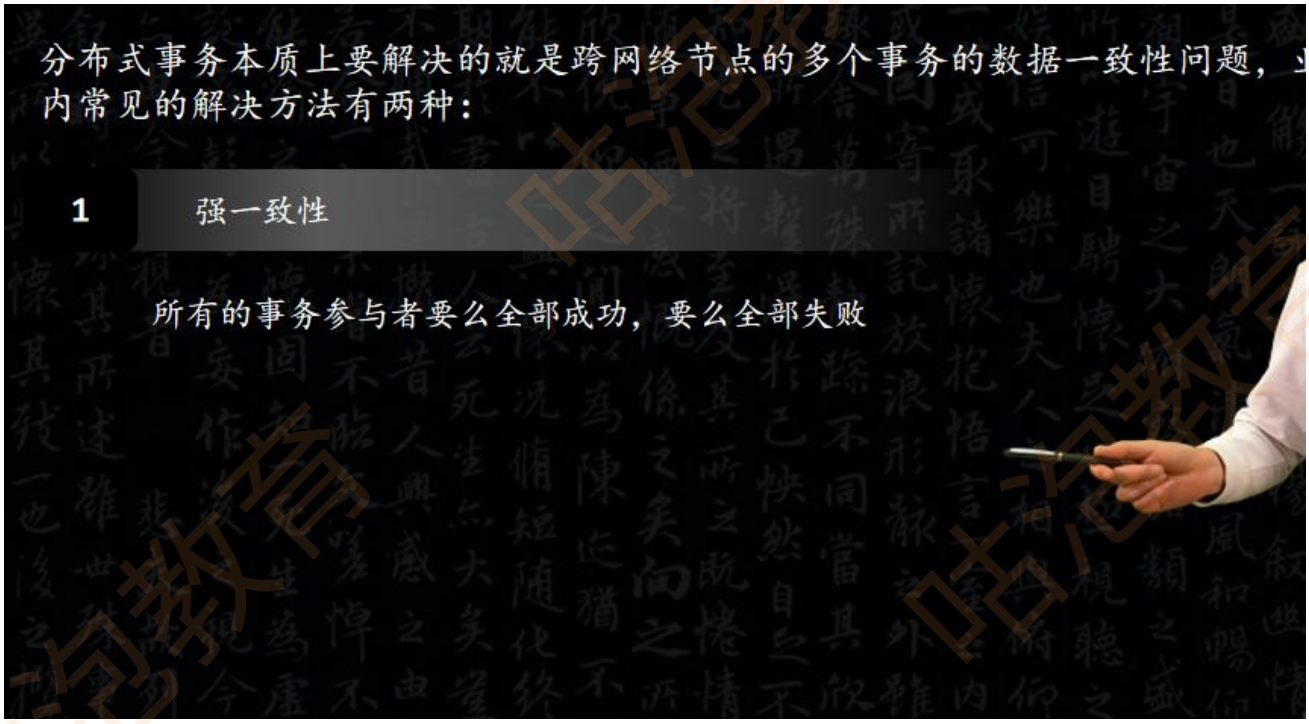
在微服务架构下，由于数据库和应用服务的拆分，导致原本一个事务单元中的多个 DML 操作，变成了跨进程或者跨数据库的多个事务单元的多个 DML 操作，而传统的数据库事务无法解决这类的问题，所以就引出了分布式事务的概念。



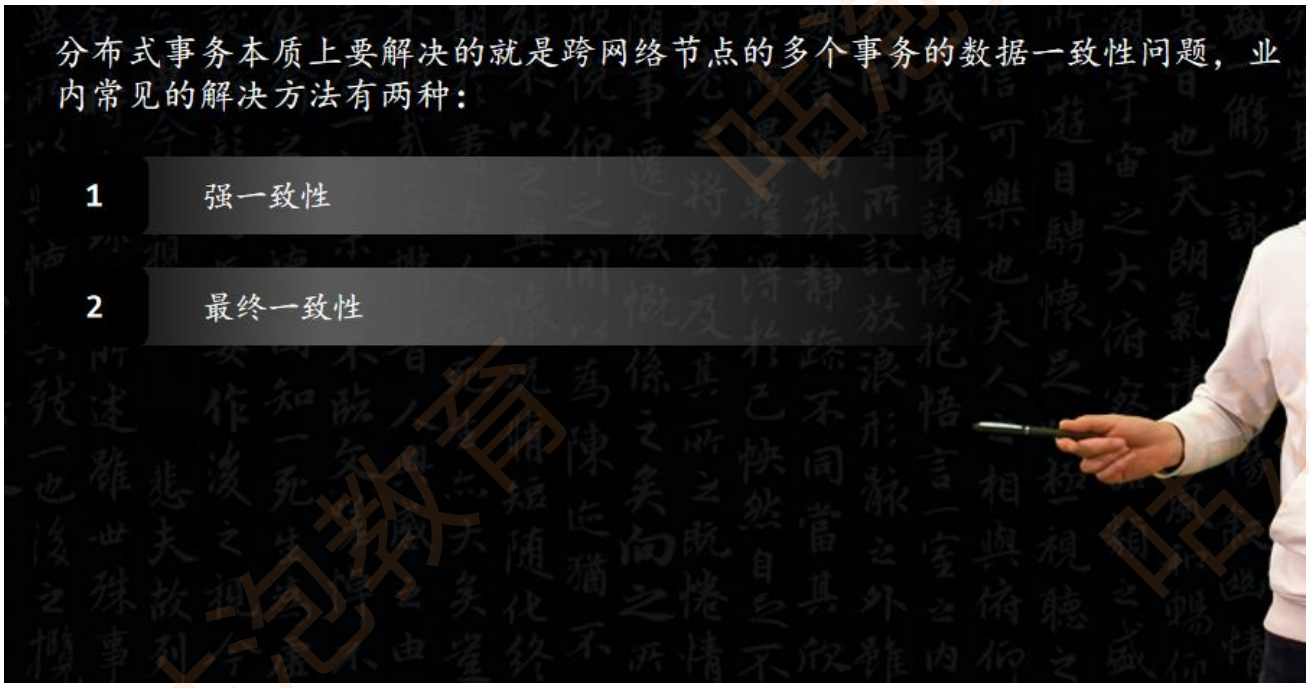
分布式事务本质上要解决的就是跨网络节点的多个事务的数据一致性问题，业内常见的解决方法有两种:



1、强一致性，



就是所有的事务参与者要么全部成功，要么全部失败，全局事务协调者需要知道每个事务参与者的执行状态，再根据状态来决定数据的提交或者回滚！



2、最终一致性，也叫弱一致性，

也就是多个网络节点的数据允许出现不一致的情况，但是在最终的某个时间点会达成数据一致。

基于 CAP 定理我们可以知道，强一致性方案对于应用的性能和可用性会有影响，所以对于数据一致性要求不高的场景，就会采用最终一致性算法。

在分布式事务的实现上，对于强一致性，我们可以通过基于 XA 协议下的二阶段提交来实现，对于弱一致性，可以基于 TCC 事务模型、可靠性消息模型等方案来实现。市面上有很多针对这些理论模型实现的分布式事务框架，我们可以在应用中集成这些框架来实现分布式事务。而 Seata 就是其中一种，它是阿里开源的分布式事务解决方案，提供了高性能且简单易用的分布式事务服务。

【导航条：Seata事务模式】

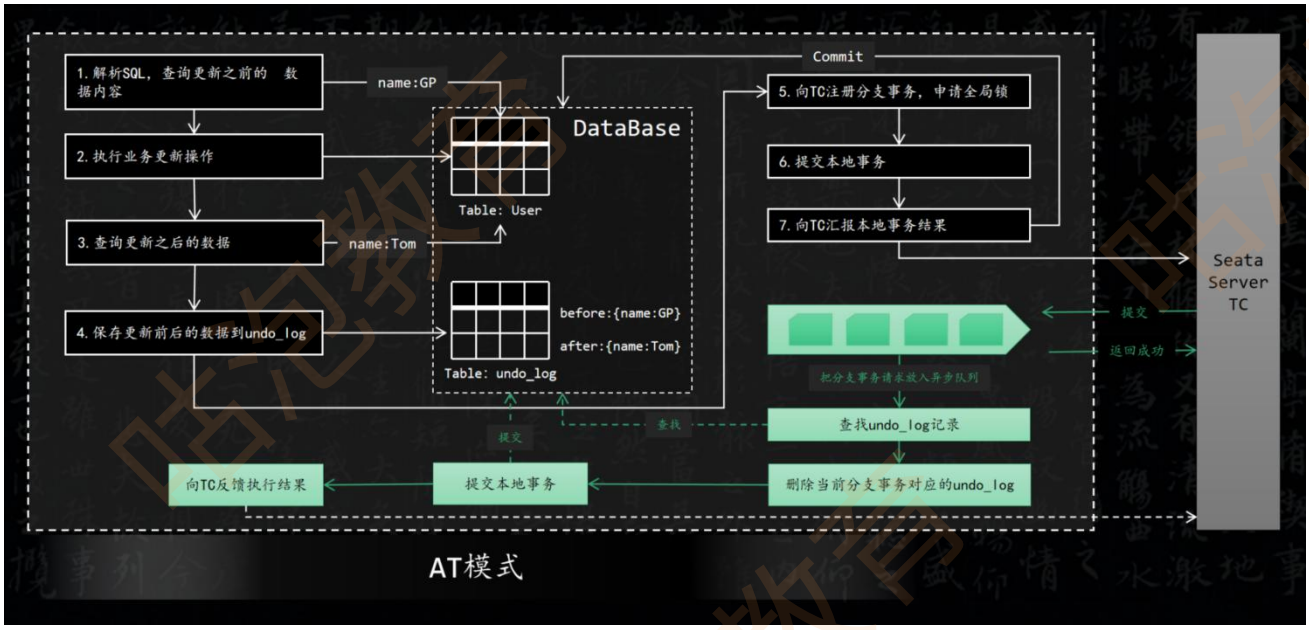
（转场，屏幕黑底白字，正中央出现“1 Seata事务模式”）

2、Seata事务模式

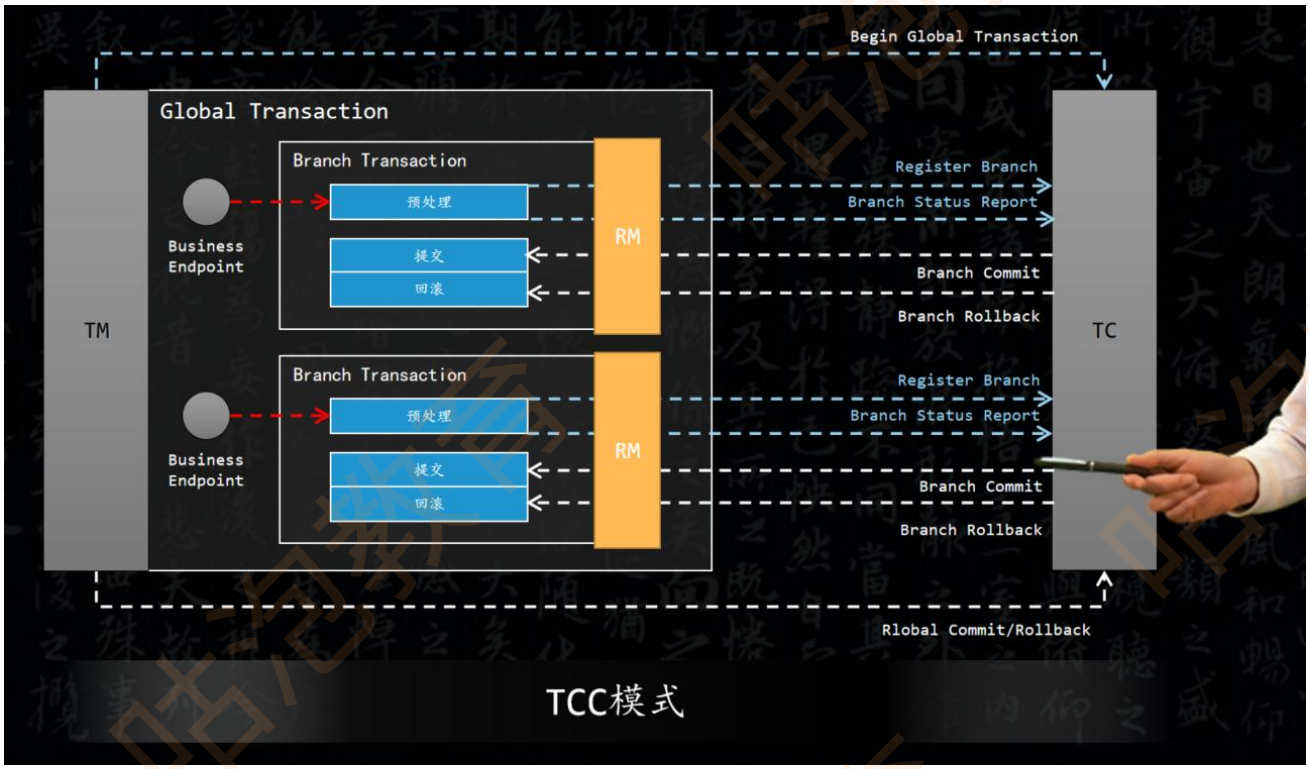
Seata 中封装了四种分布式事务模式，分别是：AT模式、TCC 模式、Saga 模式和XA 模式。下面我给大家详细

介绍一下：

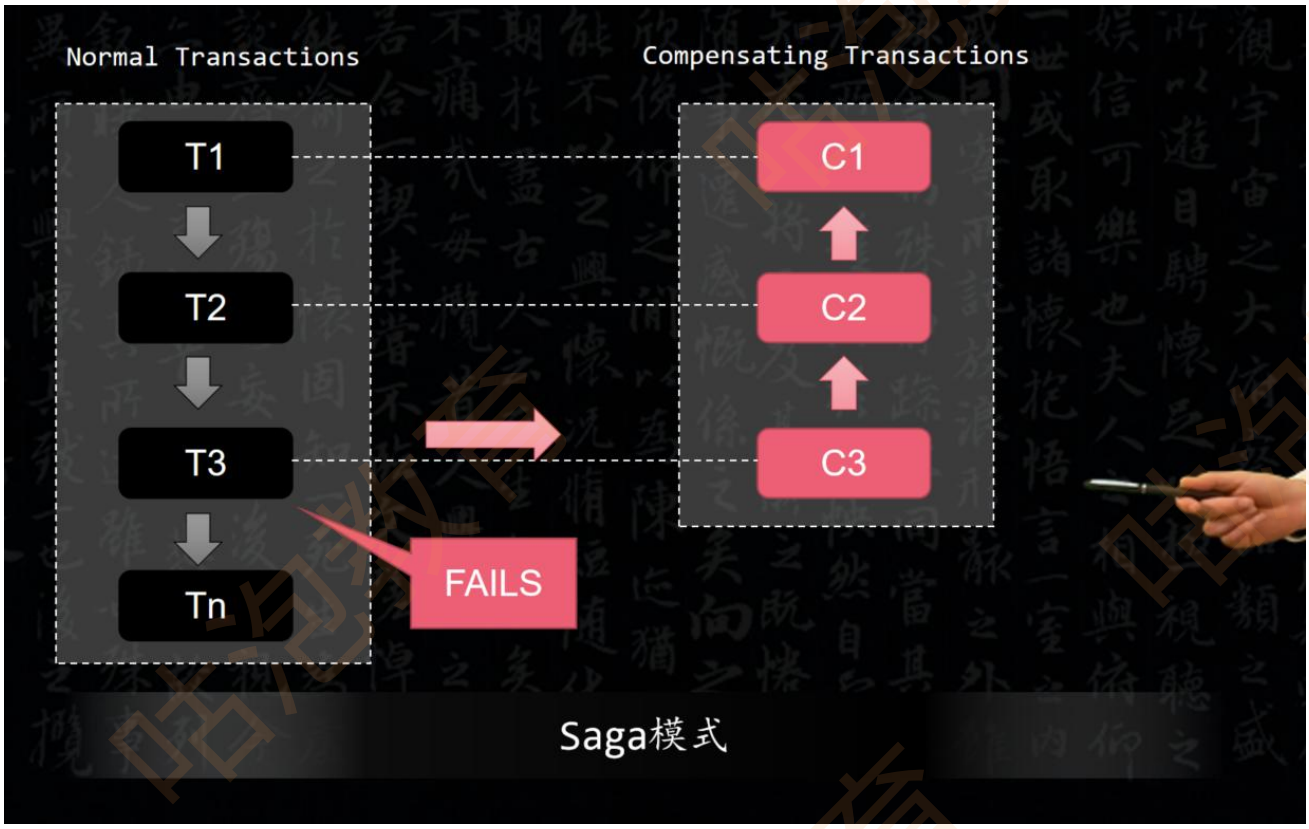
第1种：AT 模式，是一种基于本地事务+二阶段协议来实现的最终数据一致性方案，也是Seata 默认的解决方案



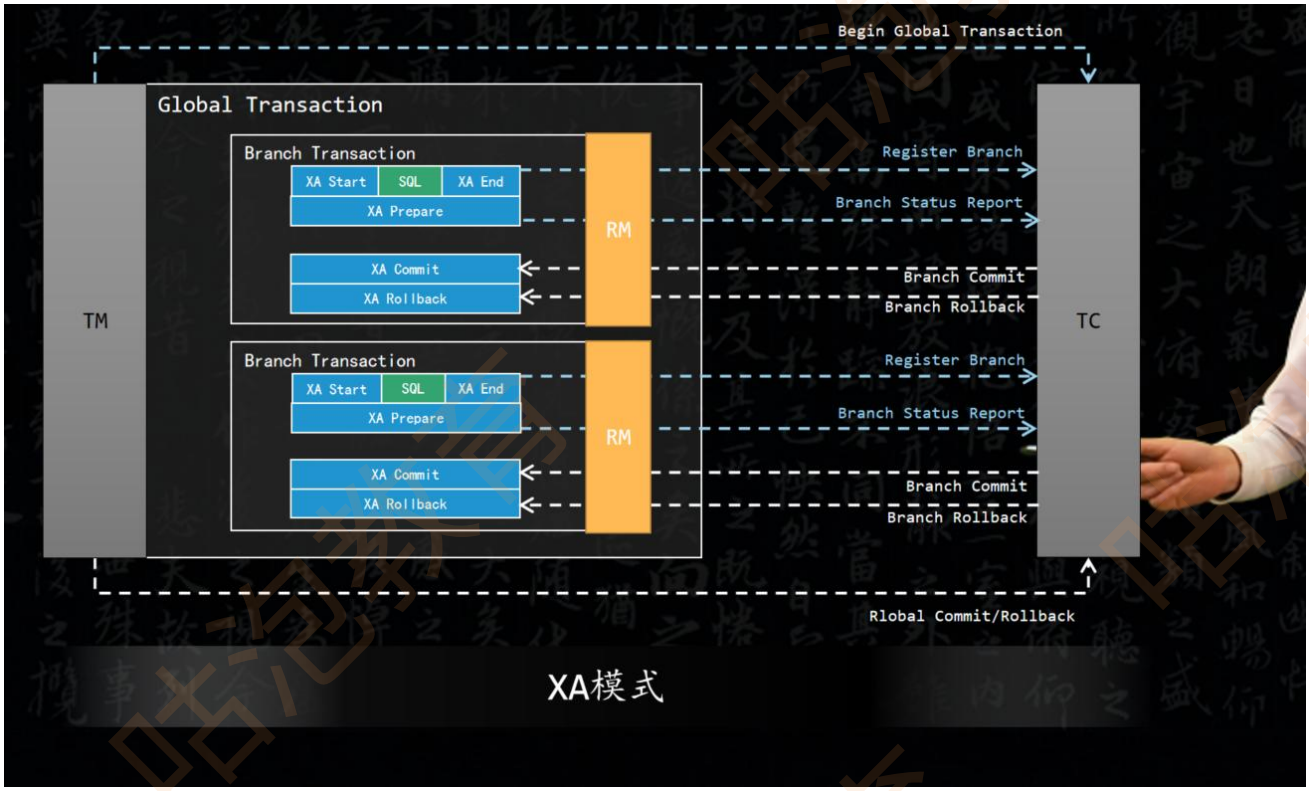
第2种：TCC 模式，TCC 事务是 Try、Confirm、Cancel 三个词语的缩写，简单理解就是把一个完整的业务逻辑拆分成三个阶段，然后通过事务管理器在业务逻辑层面根据每个分支事务的执行情况分别调用该业务的 Confirm 或者 Cacel 方法。



第3种：Saga 模式，Saga 模式是 SEATA 提供的长事务解决方案，在 Saga 模式中，业务流程中每个参与者都提交本地事务，当出现某一个参与者失败则补偿前面已经成功的参与者。



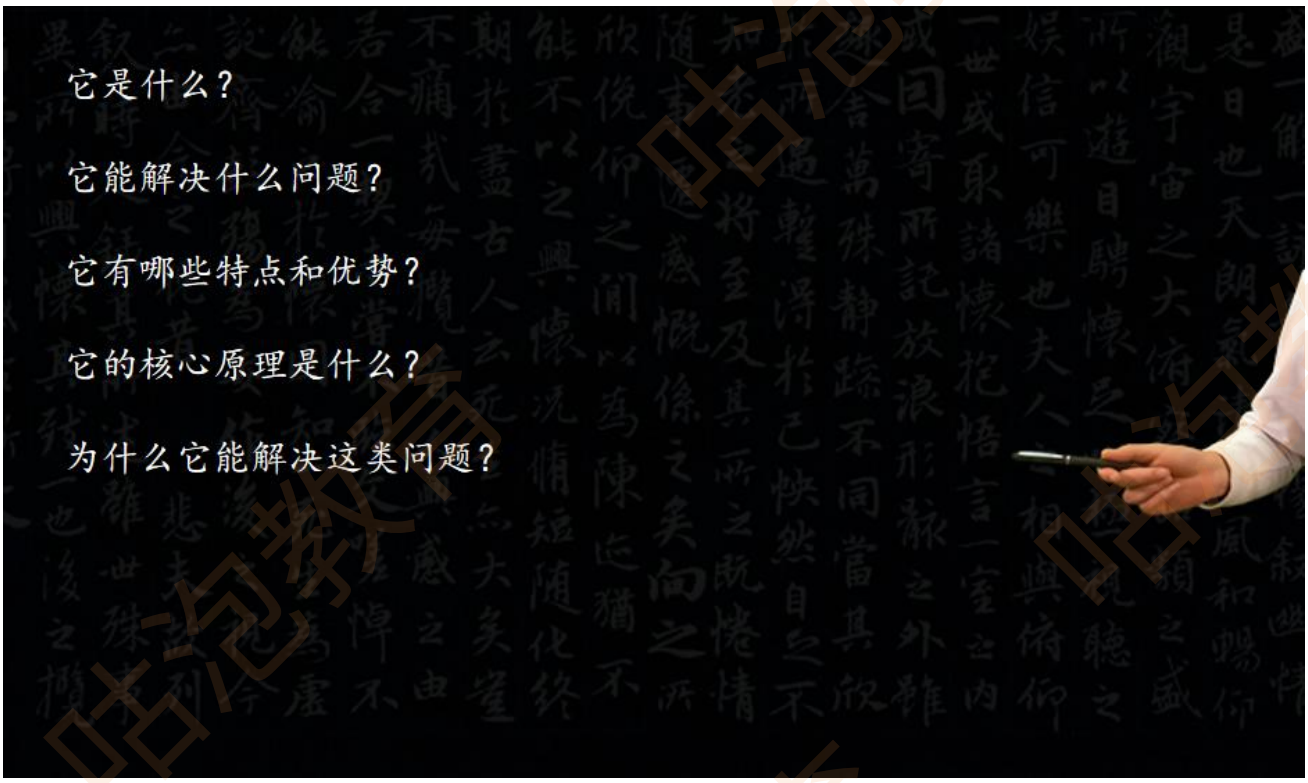
第4种：XA 模式，XA 可以认为是一种强一致性的事务解决方法，它利用事务资源（数据库、消息服务等）对 XA 协议的支持，以 XA 协议的机制来管理分支事务的一种事务模式。



从这四种模型中不难看出，在不同的业务场景中，我们可以使用 Seata 的不同事务模型来解决不同业务场景中的分布式事务问题，因此我们可以认为 Seata是一个一站式的分布式事务解决方案。

【结尾】

以上就是我对Seata的理解！面试的时候经常会遇到这种宽泛的面试题，一般就是问“谈谈你对XX的理解”。遇到这类面试题，先不用慌，我告诉大家一个回答的思路。可以按照技术发展的逻辑，就是先在自己的大脑中的建立一个知识索引，然后基于索引来定位你的知识。



比如，对于这类问题，一般可以建立一个这样的思维逻辑：

它是什么?

它能解决什么问题?

它有哪些特点和优势?

它的核心原理是什么?

为什么它能解决这类问题?

大家对照这个逻辑去回答这类面试题，是不是感觉思路异常更清晰呢?

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，

面试不再难!

88.谈谈你对Kafka数据存储原理的理解?

【导航条：引言】

一位5年工作经验的小伙伴面试的时候被问到这样一个问题，说”谈谈你对Kafka数据存储原理的理解“。然后，这位小伙伴突然愣住了，什么是零拷贝，零拷贝跟Kafka有关系吗?

那么今天，我给大家来聊一聊我对Kafka零拷贝原理的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

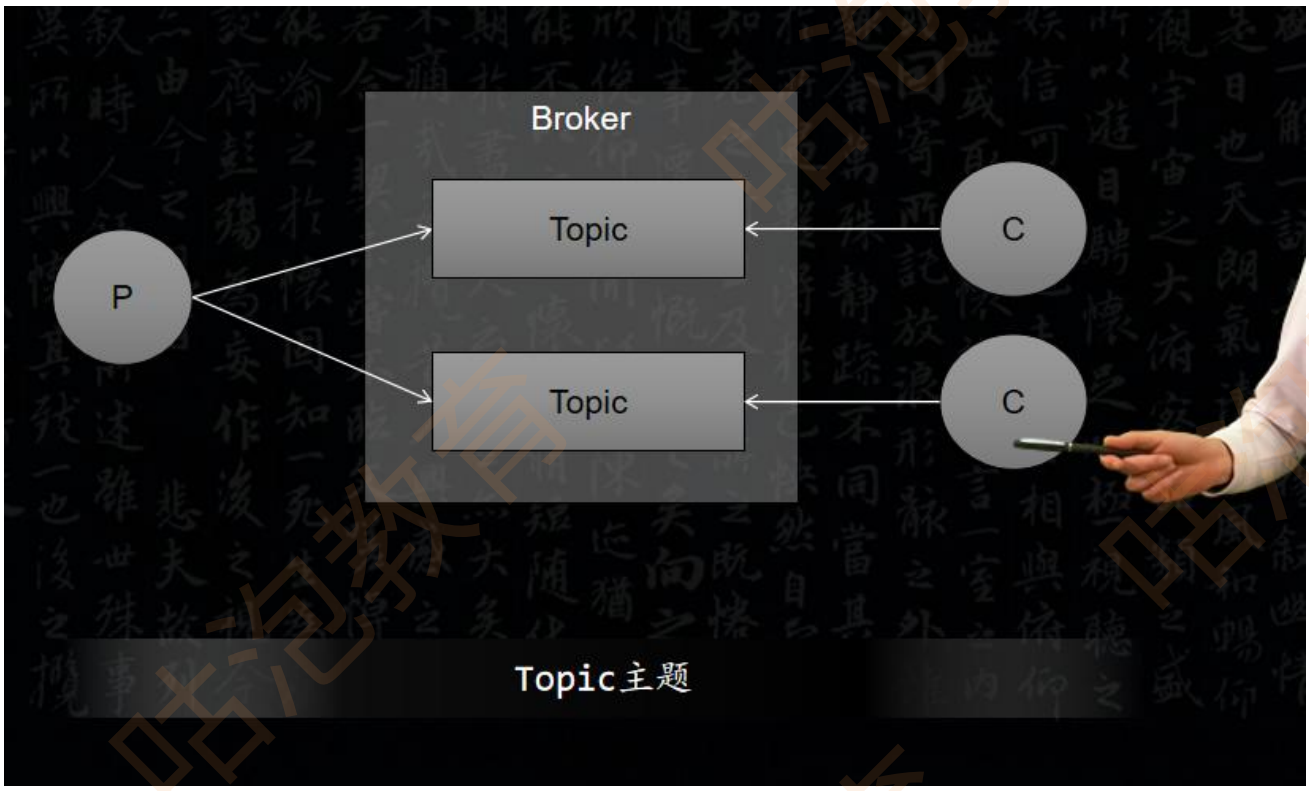
【导航条：主题】

（转场，屏幕黑底白字，正中央出现“1 Topic主题”）

1、Topic主题

在Kafka中，这个用 来存储消息的队列叫做Topic，它是一个逻辑的概念，可以理解为一组消息的集合。

生产者和Topic以及Topic和消费者的关系都是多对多。一个生产者可以发送消息到多个Topic，一个消费者也可以从多个Topic获取消息（但是不建议这么做）。



生产者发送消息时，如果Topic不存在，Kafka默认会自动创建。

【导航条：分区】

（转场，屏幕黑底白字，正中央出现“2 Partition分区”）

2、Partition分区

首先，Kafka为了实现横向扩展，它会把不同的数据存放在不同的Broker上，同时为了降低单台服务器的访问压力，把一个Topic中的数据分隔成多个Partition。在服务器上，每个Partition都有一个物理目录，Topic名字后面的数字标号即代表分区。比如创建一个名为mytopic的主题，数据目录被分布到了3台机器。

如图所示：

mytopic-0存在A节点，mytopic-1存在B节点，mytopic-2存在C节点。

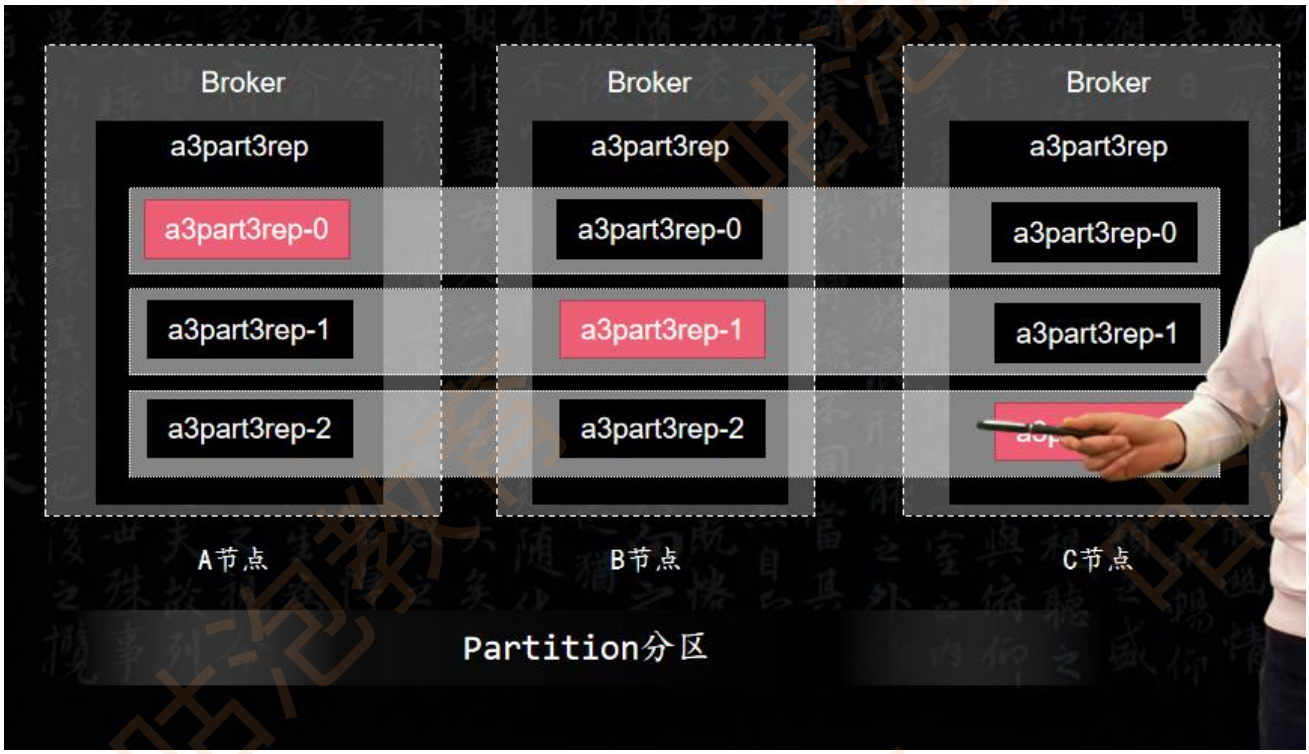
【导航条：副本】

（转场，屏幕黑底白字，正中央出现“3 Replica副本”）

3、Replica副本

另外，Kafa为了提高分区的可靠性，又设计了副本机制。我们创建Topic的时候，通过指定replication-factor副本因子，来确定Topic的副本数。当然，副本因子数必须小于等于节点数，否则会报错。这样就可以保证，绝对不会有一个分区的两个副本分布在同一个节点上，不然副本机制也失去了备份的意义了。

如图所示，创建了一个3个分区3个副本的Topic a3part3rep，被均匀分布到了3个Broker节点上，每个Broker节点互为备份。



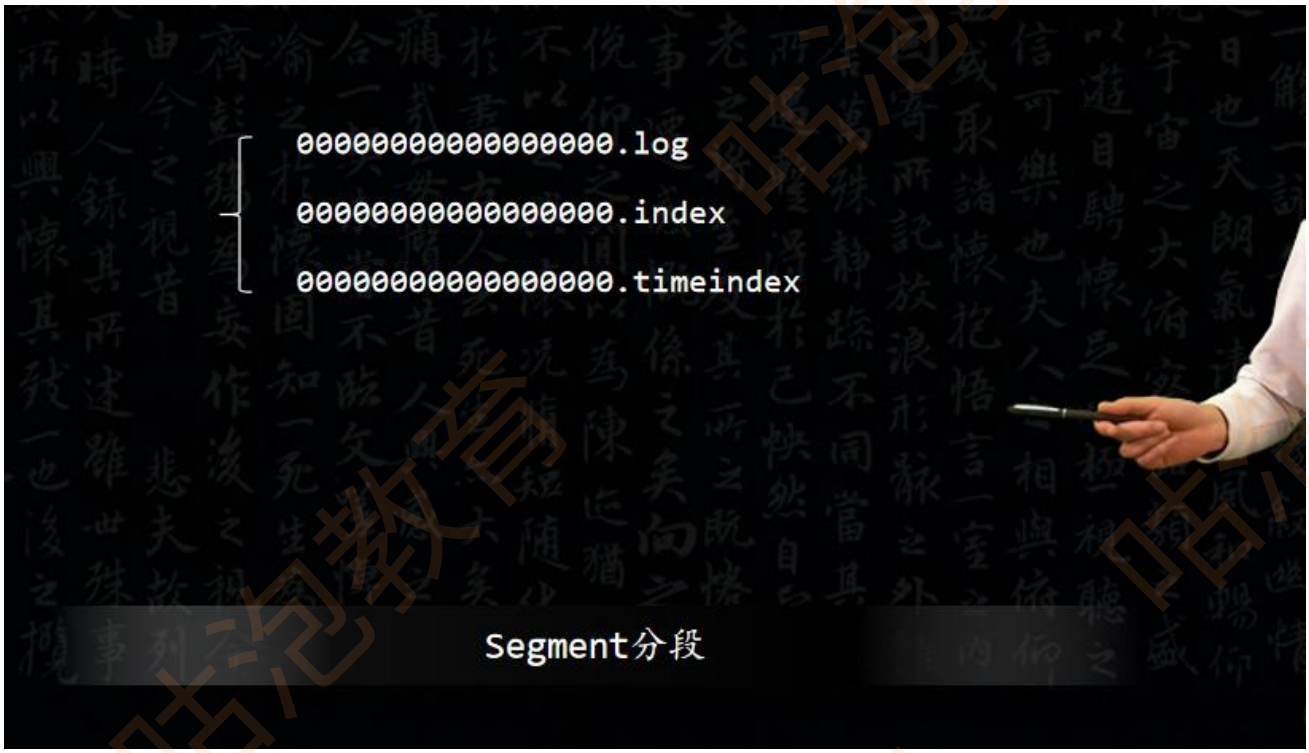
这些所有的副本分为两种角色，Leader对外提供读写服务。Follower唯一的任务就是从Leader异步拉取数据，图中红色的副本为Leade，也被均匀分布在各个节点上，可以保证读写均匀，这样的设计也称为单调读一致性。

【导航条：分段】

（转场，屏幕黑底白字，正中央出现“4 Segment副本”）

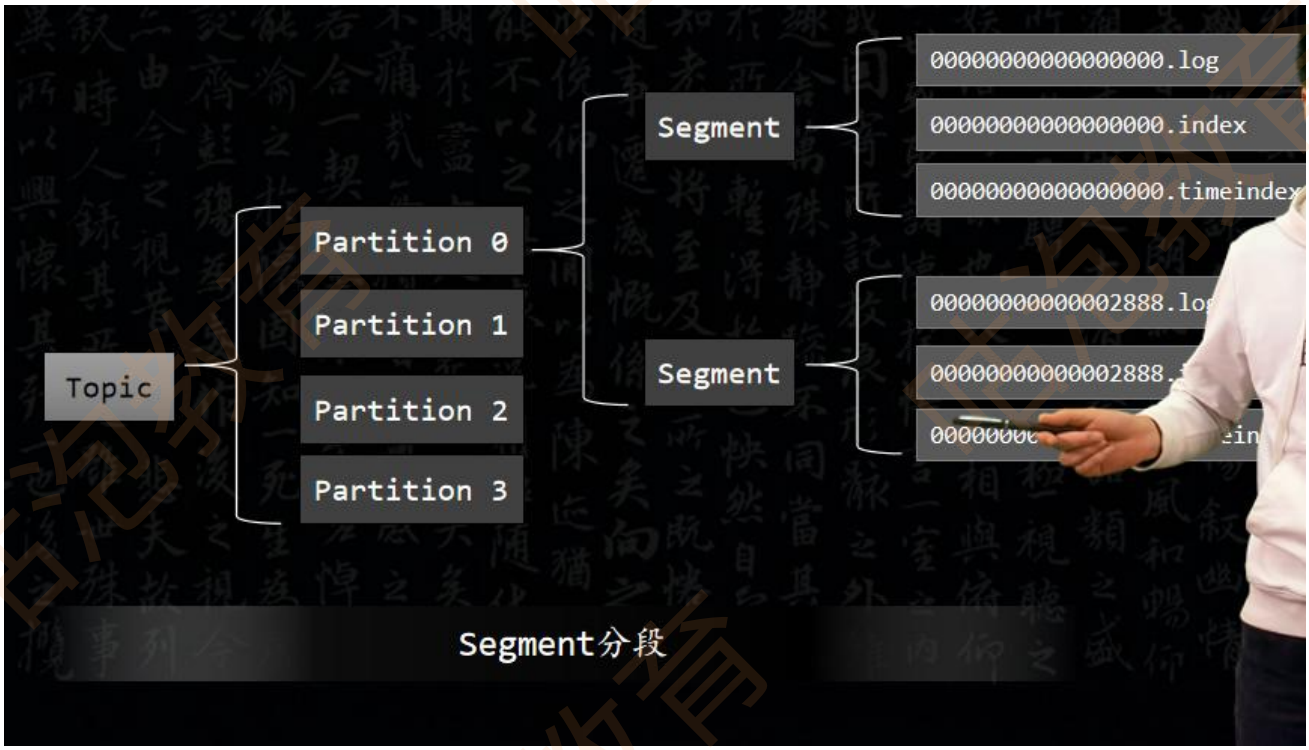
4、Segment分段

Kakfa为了防止Log不断追加导致文件过大，导致检索消息效率变低，一个Partition超出一定大小的时候，就被切割为多个Segment来组织数据。在磁盘上，每个Segment由一个log文件和2个index文件组成。



如图所示，这三个文件是成套出现的。其中.index是用来存储Consumer的Offset偏移量的索引文件，.timeindex是用来存储消息时间戳的索引文件，log文件就是用来存储具体的数据文件。

以切割时记录的Offset值作为文件的名字。它的文件结构是这样的，如图所示：

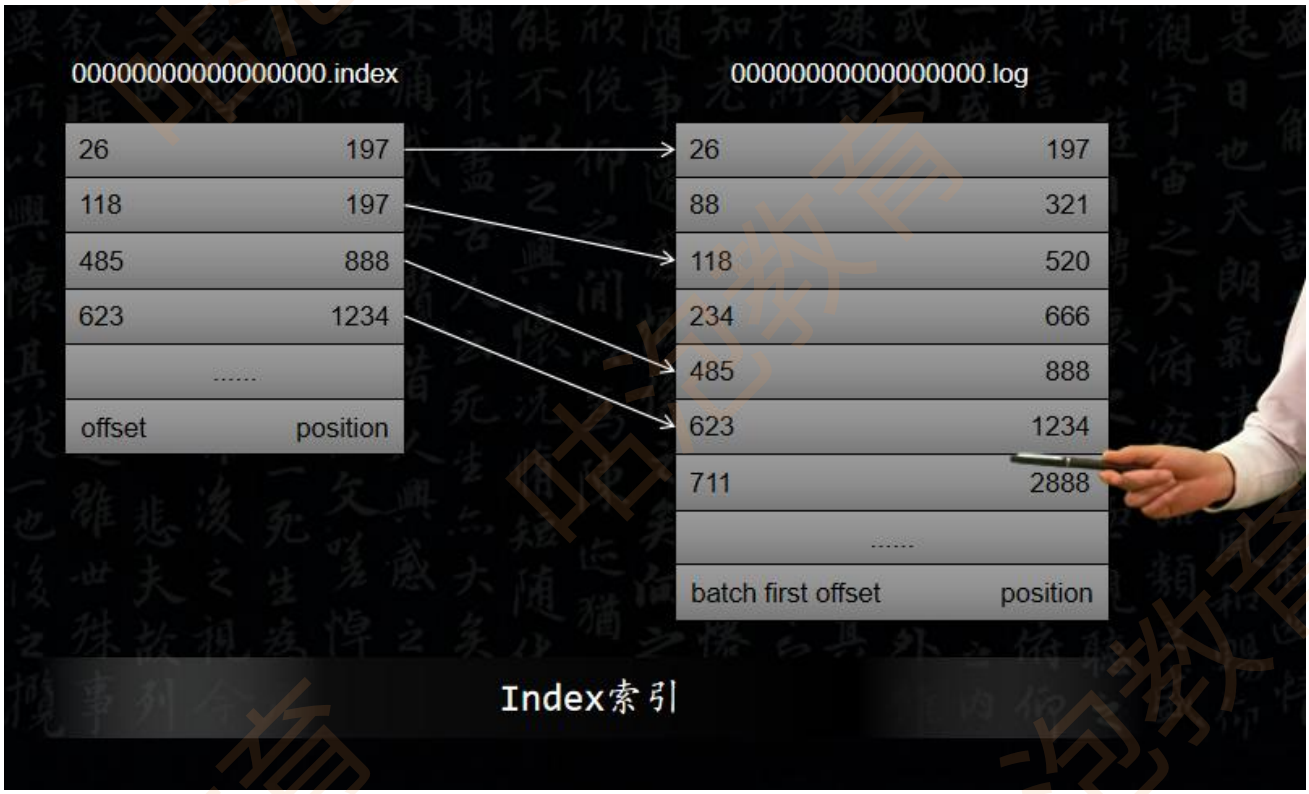


【导航条：索引】

（转场，屏幕黑底白字，正中央出现“4 Index索引”）

5、Index索引

前面我们讲到Kafka设计了两种索引，一种是偏移量索引文件，记录的是Offset和消息在Log文件中的位置映射关系。一种是时间戳索引文件，记录的是时间戳和Offset的关系。为了提高检索效率Kafka并不会为每一条消息都会建立索引，而是采用稀疏索引。也就是说隔一批消息才产生一条索引记录。如图所示：



我们可以通过以参数来设置索引的稀疏程度。

相对来说，越稠密的索引检索数据更快，但是会消耗更多的存储空间；

越的稀疏索引占用存储空间小，但是插入和删除时所需的维护开销也小。

同样，时间戳索引也是采用稀疏索引设计。由于索引文件是以Offset命名的，所以Kafka在检索数据的时候，是采用二分法查找，效率就非常快。

【结尾】

以上就是我对Kafka数据存储原理的理解！

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

89.谈谈你对Kafka副本Leader选举原理的理解?

【导航条：引言】

一位7年工作经验的小伙伴，面试被问到这样一道题，说：”谈谈你对Kafka副本Leader选举原理的理解“。当时，他想，这Kafka用的不就是Zookeeper 的选举吗？难道Kafka又自己搞了一套。没错，这回Kafka自己造了一个轮子。

那么今天，我给大家来聊一聊我对Kafka副本Leader选举原理的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：选举原理】

（转场，屏幕黑底白字，正中央出现“1 选举原理”）

1、选举原理

确实Kafka早期的版本就是直接用Zookeeper来完成选举的。利用了Zookeeper的Watch机制；节点不允许重复写入以及临时节点这些特性。这样实现比较简单，省事。但是也会存在一定的弊端。比如分区和副本数量过多，所有的副本都直接参与选举的话，一旦某个出现节点的增减，就会造成大量的Watch事件被触发，ZooKeeper的就会负载过重，不堪重负。

新版本的Kafka中换了一种实现方式。不是所有的Repalica都参与Leader选举，而是由其中的一个Broker统一来指挥，这个Broker的角色就叫做Controller控制器。

Kafka要先从所有Broker中选出唯一的一个Controller。

所有的Broker会尝试在Zookeeper中创建临时节点/controller，谁先创建成功，谁就是Controller。那如果Controller挂掉或者网络出现问题，ZooKeeper上的临时节点就会消失。其他的Broker通过Watch监听到

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

Controller下线的消息后，继续按照先到先得的原则竞选Controller。这个Controller就相当于选举委员会的主席。

当一个节点成为Controller之后，他就会承担以下职责：

监听Broker变化、监听Topic变化、监听Partition变化、获取和管理Broker、Topic、Partition的信息、管理Partiontion的主从信息。

【导航条：选举规则】

（转场，屏幕黑底白字，正中央出现“1 选举规则）

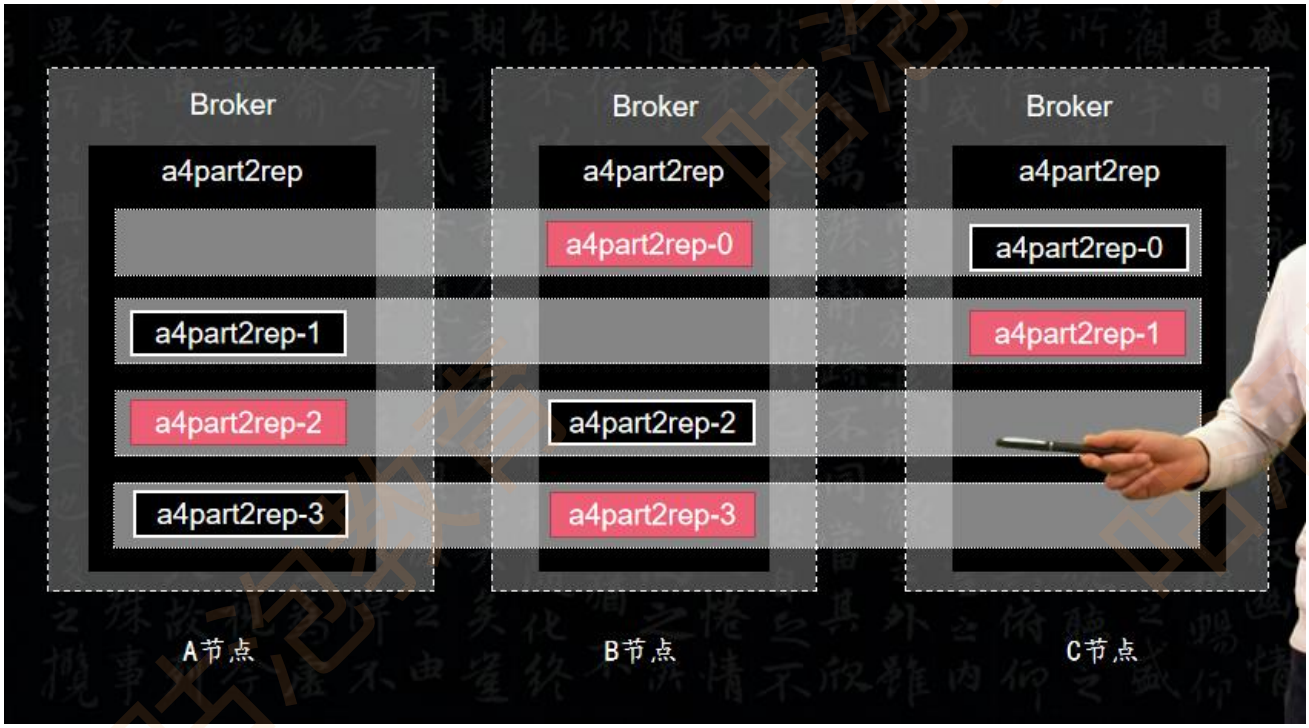
2、选举规则

Controller确定以后，就可以开始做分区选主的事情。接下来就是找候选人。显然，每个Replica都想推荐自己，但不是所有的Replica都有竞选资格。只有在ISR（In-Sync Replicas）保持心跳同步的副本才有资格参与竞选。就好比是皇帝每天着急皇子们开早会，只有每天来打卡的皇子才能加入ISR。那些请假的、迟到的没有资格参与选举。

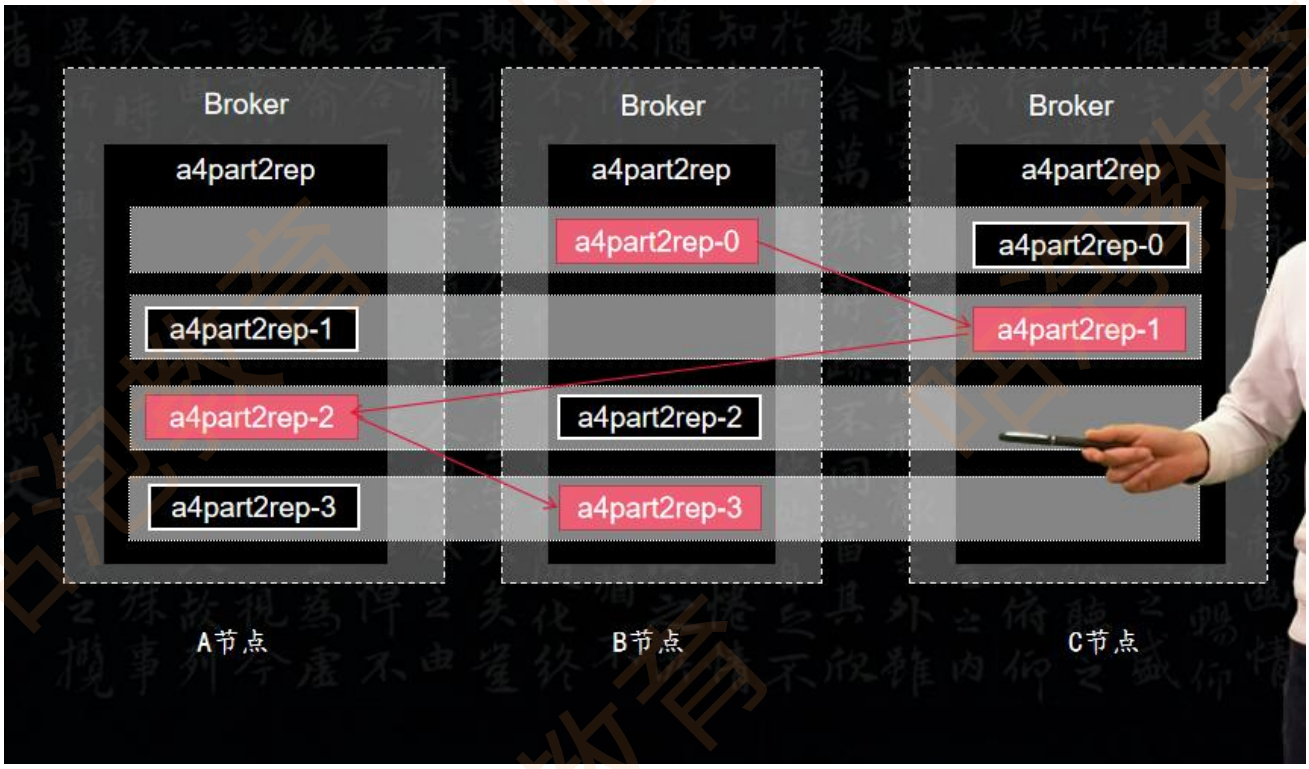
接下来，就是Leader选举，就相当于要在众多皇子中选出太子。在分布式选举中，有非常多的选举协议比如ZAB、Raft等等，他们的思想归纳起来都是：先到先得，少数服从多数。但是Kafka没有用这些方法，而是用了一种自己实现的算法。

提到Kafka官方的解释是，它的选举算法和微软的PacificA算法最相近。大致意思就是，默认是让ISR中第一个Replica变成Leader。比如ISR是1、5、9，优先让1成为Leader。这个跟中国古代皇帝传位是一样的，优先传给皇长子。

假设，我们创建一个4个分区2个副本的Topic，它的Leader分布是这样的，如图所示：



第1个分区的副本Leader，落在B节点上。第2个分区的副本Leader落在C节点上，第3个分区的副本Leader落在A节点上，第4个分区的副本Leader落在B节点上。如果有更多副本，就以此类推。我们发现Leader的选举的规则相当于蛇形走位。



这样设计的好处是可以提高数据副本的容灾能力。将Leader和副本完全错开，从而不至于一挂全挂。

【结尾】

以上就是我对Kafka副本Leader选举原理的理解！

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

90.RocketMQ为什么要放弃Zookeeper?

【导航条：引言】

一位7年工作经验的小伙伴去面架构师岗位，被问到这样一道面试题，说”RocketMQ为什么要放弃Zookeeper“。然后，想了很久好像没关注过，也不敢瞎猜。

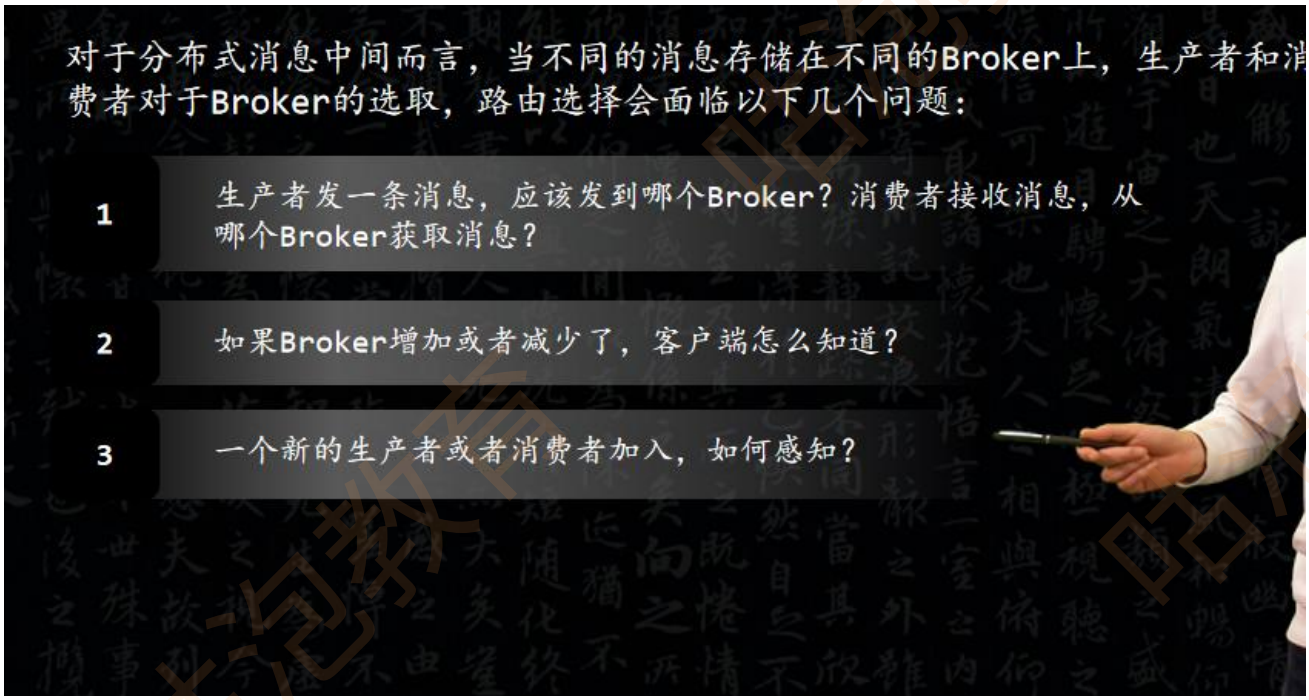
那么今天，我给大家来聊一聊我对这个问题的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：注册中心】

（转场，屏幕黑底白字，正中央出现“1 注册中心）

1、注册中心



对于分布式消息中间件而言，当不同的消息存储在不同的Broker上，生产者和消费者对于Broker的选取，路由选择会面临以下几个问题：

1、生产者发一条消息，应该发到哪个Broker？消费者接收消息，从哪个Broker获取消息？

2、如果Broker增加或者减少了，客户端怎么知道？

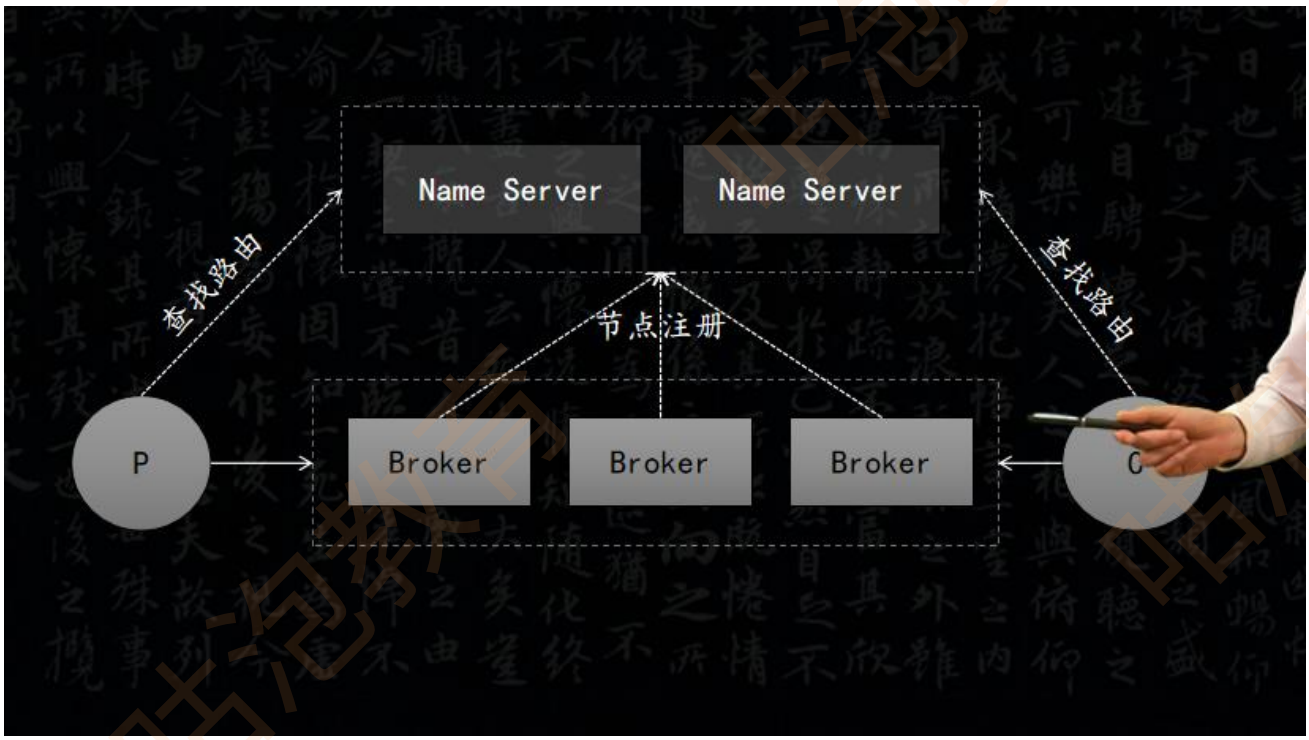
3、一个新的生产者或者消费者加入，如何感知？

所以，只要是跟分布式服务调用的场景，都需要一个注册中心，在RocketMQ当然也中需要有一个这样的角色来管理Broker的信息。

而Kafka就是用了现成的Zookeeper，但是RocketMQ却偏偏没有这么做，而是自己实现了一个服务，这个服务叫做NameServer。

我们可以把NameServer理解为是RocketMQ的路由中心，每一个NameServer节点都保存了全量的路由信息。为了保证高可用，NameServer自身也可以做集群的部署。它的作用有点像Eureka或者Redis的Sentinel。

也就是说，Broker会在NameServer上注册自己，Porducer和Consumer用NameServer来发现Broker。



每个Broker节点在启动时，都会根据配置遍历NameServer列表。

与每个NameServer建立TCP长连接，注册自己的信息，之后每隔30s发送心跳信息。

如果Broker挂掉了，不发送心跳了，NameServer怎么发现呢?

所以，除了主从注册，还有定时探活。每个NameServer每隔10s检查一下各个Broker的最近一次心跳时间，如果发现某个Broker超过120s都没发送心跳，就认为这个Broker已经挂掉了，会将其从路由信息里移除。

【导航条：放弃理由】

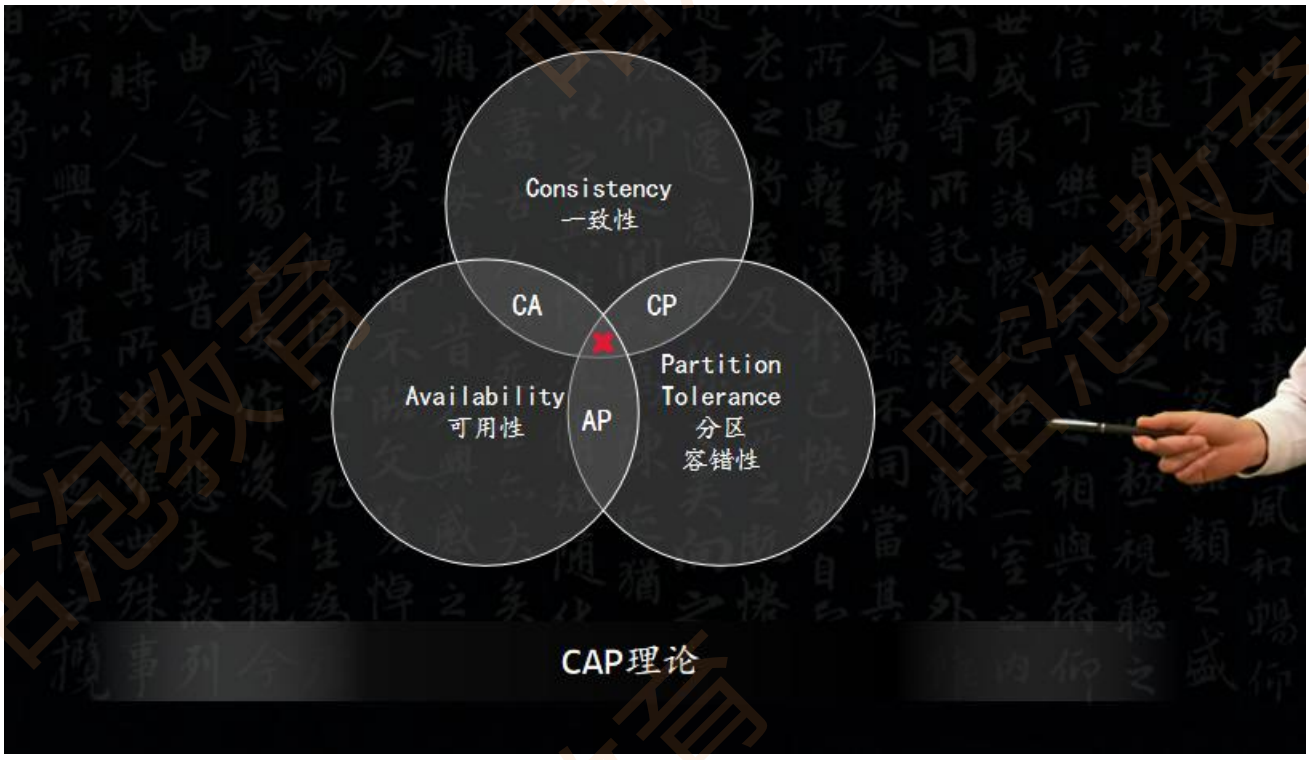
（转场，屏幕黑底白字，正中央出现“2 放弃理由）

2、放弃理由

既然，Nameserver的作用也是用来管理Broker的服务的，也就是服务注册与发现，那为什么不直接用Zookeeper、Consul、etcd、Eureka这样的组件呢?

实际上在RocketMQ的早期版本中，跟Kafka一样也是用Zookeeper实现服务管理的，但到RokcetMQ开源的时候去掉了ZooKeeper依赖，转而采用自己的NameServer。

因为，RocketMQ是一个保持最终一致性的架构设计，它架构决定了它只需要一个轻量级的元数据服务器就足够了，而不需要像Zookeeper这样的强一致性解决方案。不依赖另一个中间件，从而减少整体维护成本。



根据著名的CAP理论，在一致性(Consistency)、可用性(Availability)、分区容错性(Partiton Tolerance)中，Zookeeper实现了CP，而NameServer选择了AP，放弃了实时一致性。

【结尾】

以上就是我对RocketMQ为什么要放弃Zookeeper的理解！

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

91.什么是云原生（终于有人把云原生讲明白了）？

【导航条：引言】

技术的变革，一定是思想先行，云原生是一种构建和运行应用程序的方法，是一套技术体系和方法论。伴随云计算的滚滚浪潮，云原生(CloudNative)的概念应运而生，而且还很火。

今天，我给大家讲一讲，我对云原生的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：什么是云原生】

（转场，屏幕黑底白字，正中央出现“1 什么是云原生）

1、什么是云原生

云原生（Cloud Native）是一个组合词，Cloud+Native。Cloud表示应用程序位于云中，而不是传统的数据中心；Native表示应用程序从设计之初即考虑到云的环境，原生为云而设计，在云上以最佳姿势运行，充分利用和发挥云平台的弹性+分布式优势。

【导航条：云原生四要素】

（转场，屏幕黑底白字，正中央出现“2 云原生四要素）

2、云原生四要素

实现云原生需要具备四个要素：

第1个：必须是微服务；

几乎每个云原生的定义都包含微服务，跟微服务相对的是单体应用，微服务有理论基础，那就是康威定律，指导服务怎么切分。微服务架构能实现服务解耦，内聚更强，变更更易；另一个划分服务的依据就是DDD。

第2个：必须实现容器化；

Docker就是应用最为广泛的容器引擎，在大厂的基础设施中大量使用.容器化为微服务提供实施保障，起到应用隔离作用。而K8s 是容器编排系统，用于容器管理，容器间的负载均衡。

第3个：就是支持DevOps；

DevOps是一个组合词，Dev + Ops，翻译过来就是开发和运维合体，实际上DevOps应该还包括测试。DevOps为云原生提供持续交付能力。

第4个：就是必须满足持续交付；

持续交付又叫做CICD，相当于要实现在线不停机更新，这就要求开发版本和稳定版本并存，需要标准的流程和工具支撑。目前能够提供持续交付的工具也很多，比如Jenkins、Sonar等等。

【结尾】

以上就是我对云原生的理解，云原生作为下一代技术，大家可以持续关注。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

92.谈谈你对Pulsar的理解？

【导航条：引言】

有位粉丝私信我说，说想让我拍一期Pulsar的视频，那今天满足一下这位粉丝要求，谈一谈我对Pulsar的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：Pulser简介】

（转场，屏幕黑底白字，正中央出现“1 Pulser简介）

1 Pulsar特性总结

Pulsar作为下一代云原生分布式消息流平台，它集消息、存储、轻量化函数式计算为一体。

最初在 Yahoo 内部开发和部署，支持过Yahoo 应用服务平台 140 万个主题，日处理超过 1000 亿条消息。 Pulsar 于 2016 年由 Yahoo 开源并捐赠给 Apache进行孵化，2018 年成为 Apache 顶级项目。

【导航条：特性总结】

（转场，屏幕黑底白字，正中央出现“2 Pulsar特性总结）

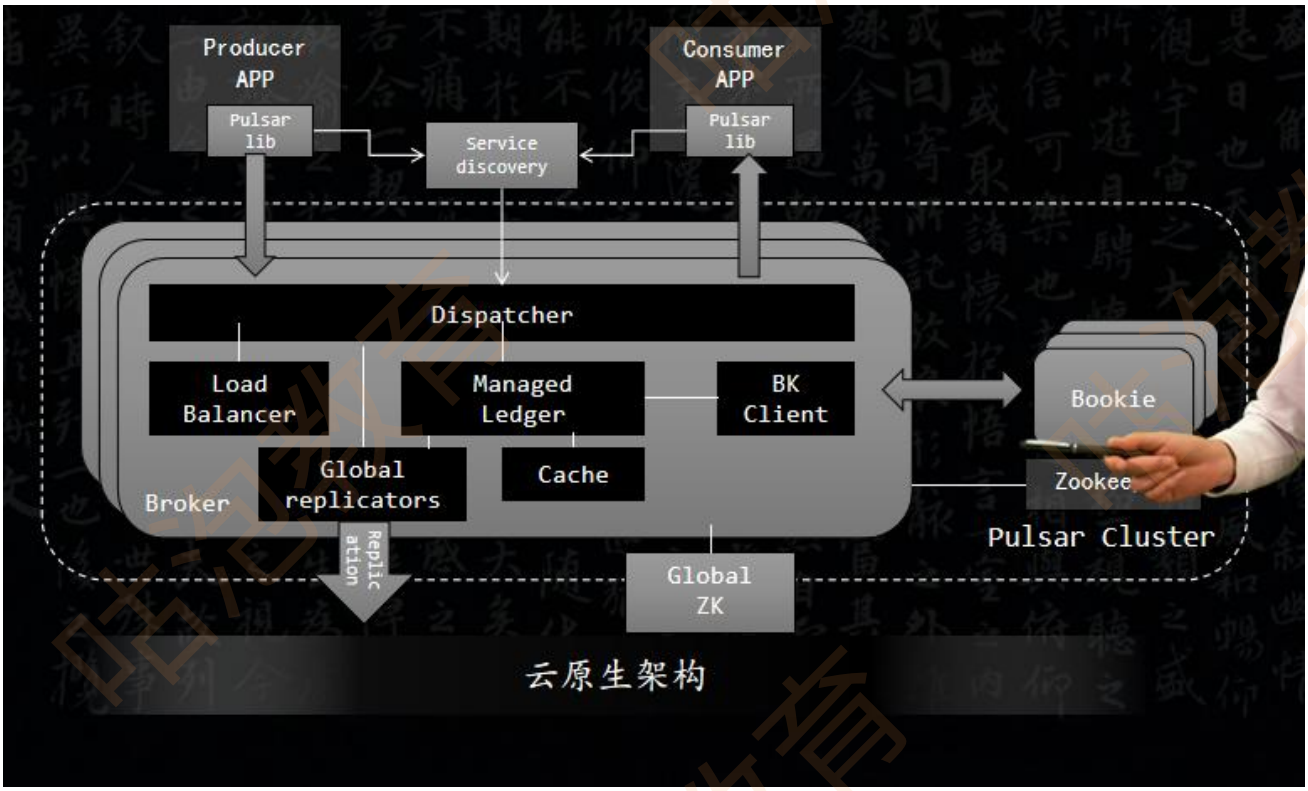
2 Pulsar特性总结

下面，我给大家总结一下Pulsar有以下九大非常重要的特性：

1 云原生架构

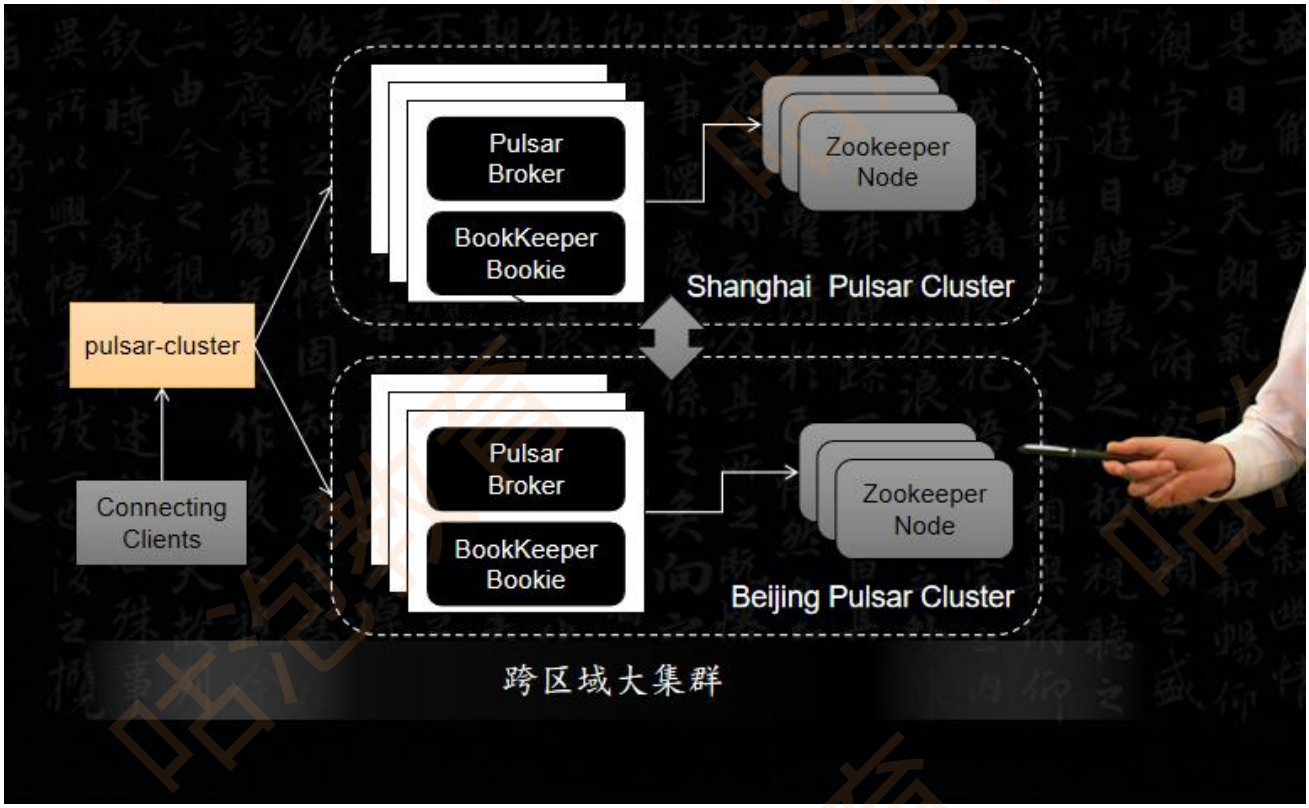
Pulsar采用计算与存储分离的云原生架构，数据从 Broker 搬离，存在共享存储内部。上层是无状态 Broker，复制消息分发和服务；下层是持久化的存储Bookie 集群。Pulsar 存储是分片的，这种构架可以避免扩

容时受限制，实现数据的独立扩展和快速恢复。



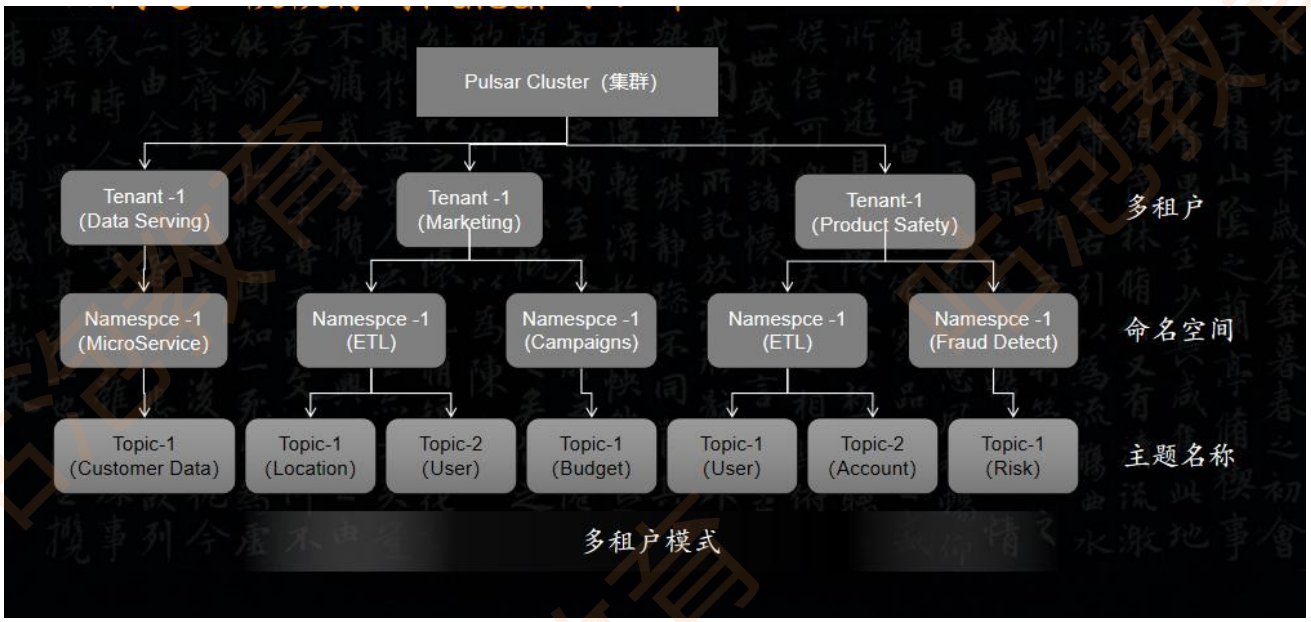
2、跨区域大集群

所谓大集群就是跨机房、跨地域的集群，使得Pulsar的分布式能力不局限于某个机房。



3、多租户模式

Pulsar是一个多租户系统，租户可以跨集群分布，每个租户都可以有单独的认证和授权机制。租户也是存储配额、消息 TTL 和隔离策略的管理单元。

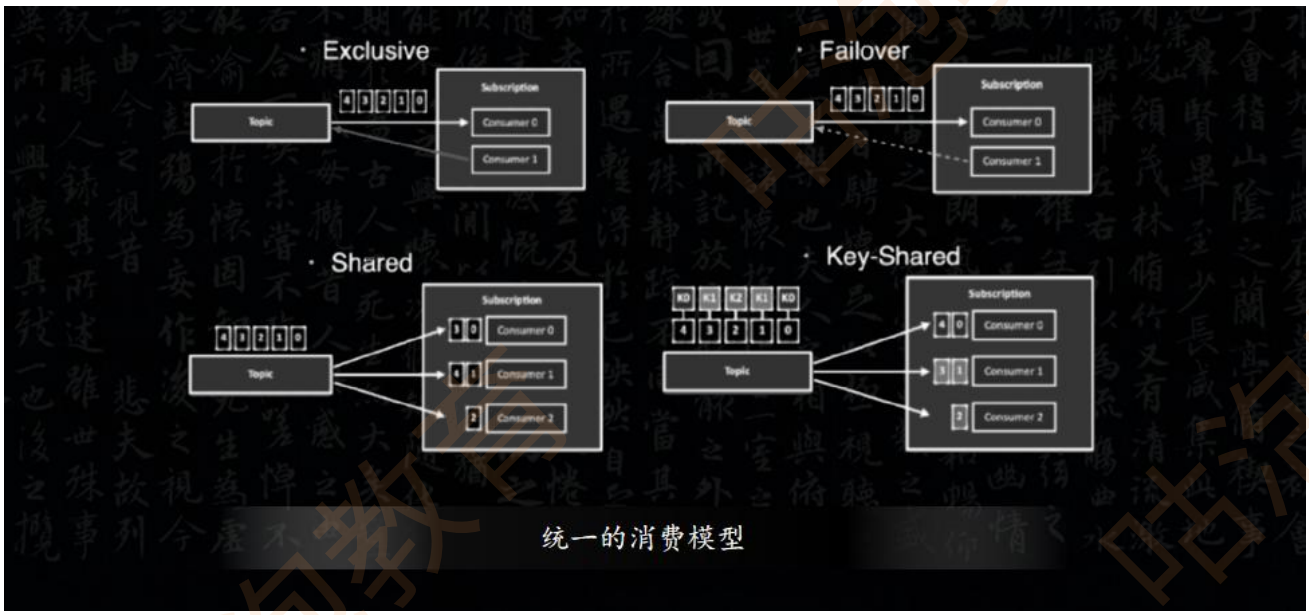


4、统一的存储模型

如图所示，Pulsar提供了统一的消息存储模型，支持对主流的消息中间件的兼容和接入。

5、统一的消费模型

Pulsar提供统一的消费模型，一共支持四种消费模式，分别是独占模式(Exclusive)、灾备模式(Failover)、共享订阅(Shared)、键共享订阅（Key_Shared）



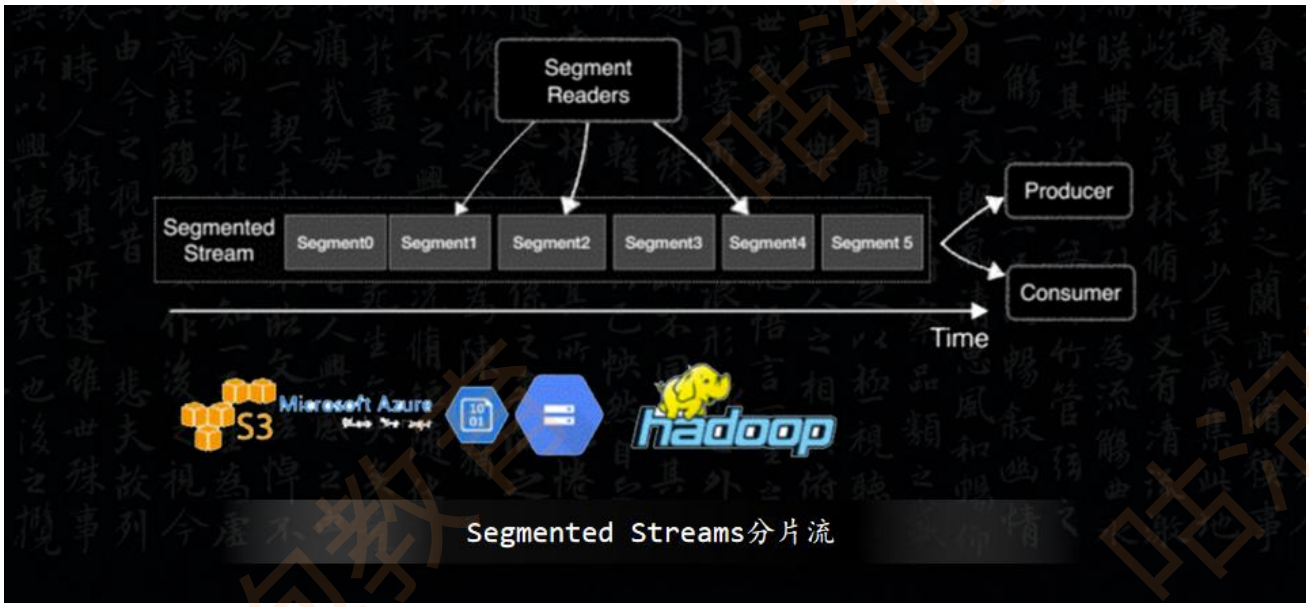
1）独占模式(Exclusive):同一个Topic只能有一个消费者订阅，如果多个消费者订阅，就会出错。 Exclusive模式为默认订阅模式。

2）灾备模式(Failover)：同一个Topic可以有多个消费者订阅，但是只能有一个消费者消费，其他订阅的消费者作为故障转移的消费者，只有当前消费者出了故障才可以进行消费当前的Topic。

3）共享订阅(Shared):同一个Topic可以由多个消费者订阅和消费。消息通过Round Robin轮询机制分发给不同的消费者，并且每个消息仅会被分发给一个消费者。当消费者断开，发送给它的没有被消费 的消息还会被重新分发给其它存活的消费者

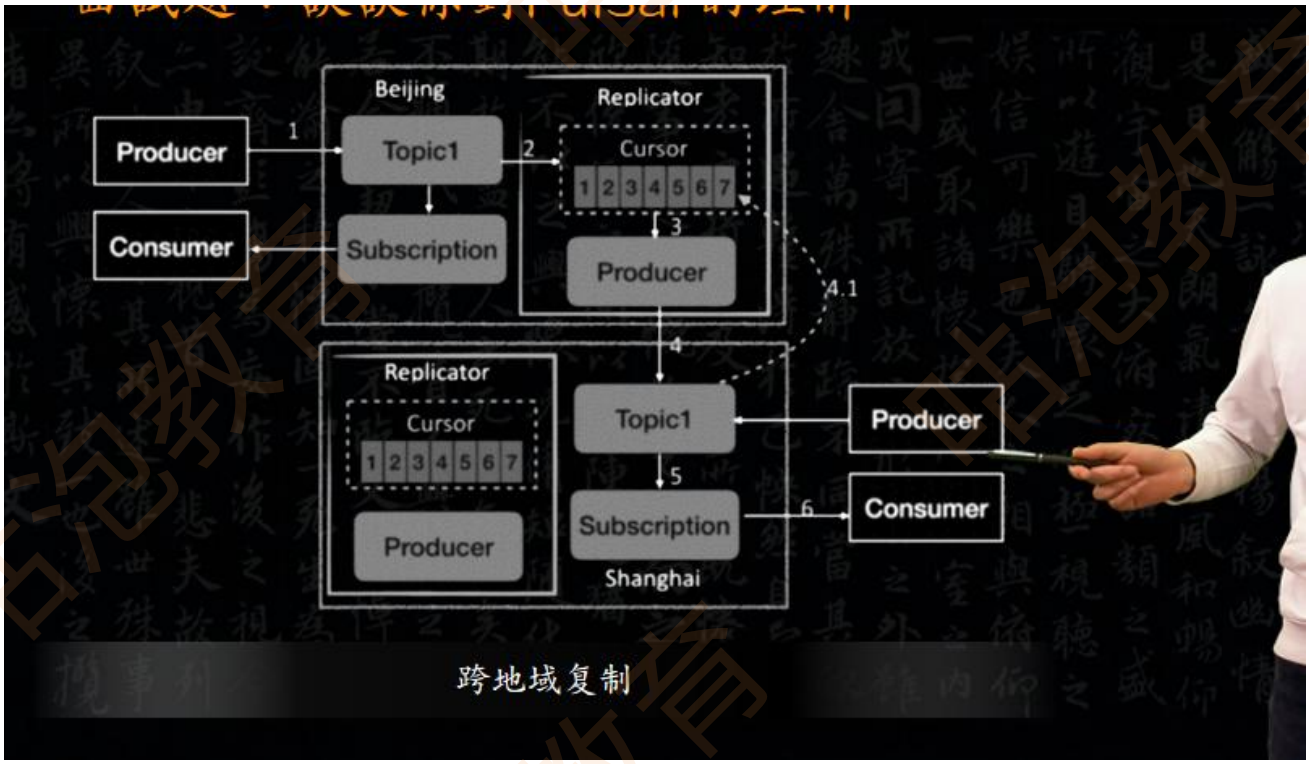
4）键共享订阅（Key_Shared）：消息和消费者都会绑定一个Key，消息只会发送给绑定同一个Key的消费者。如果有新消费者建立连接或者有消费者断开连接，就需要更新一些消息的Key。

6、Segmented Streams分片流



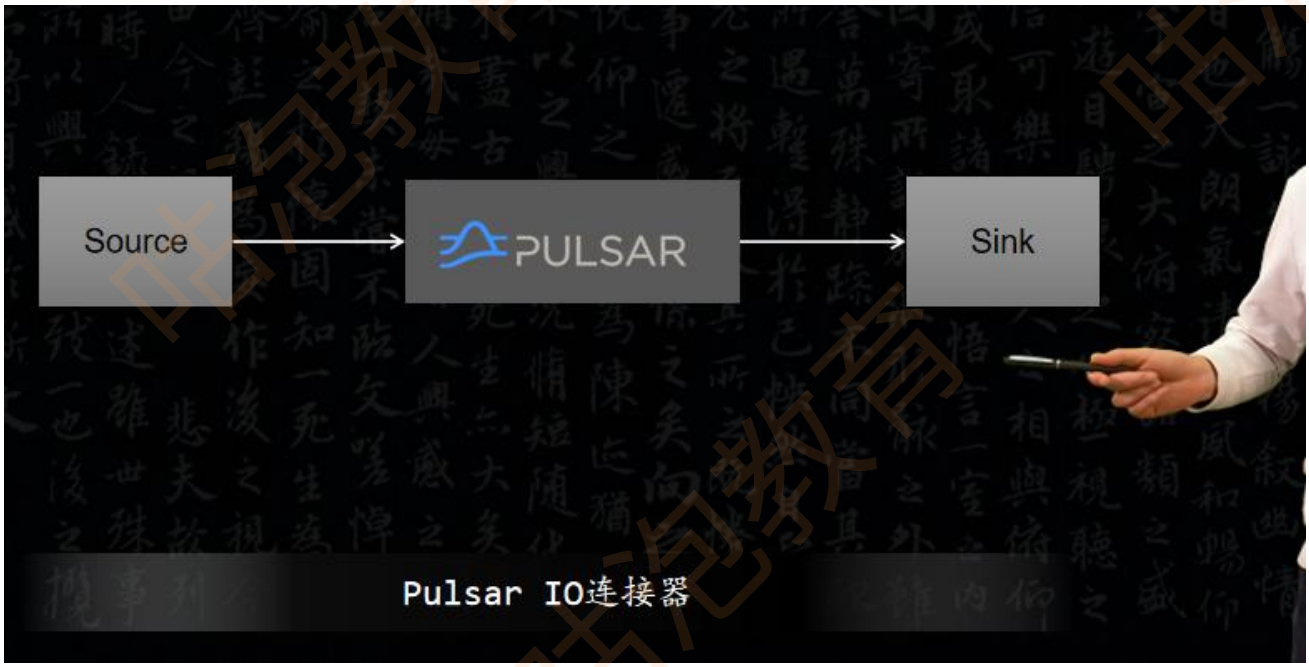
Pulsar 将无界的数据看作是分片的流，分片分散存储在分层存储（Tiered Storage）、BookKeeper 集群和 Broker节点上，而对外提供一个统一的、无界数据的视图

7、跨地域复制



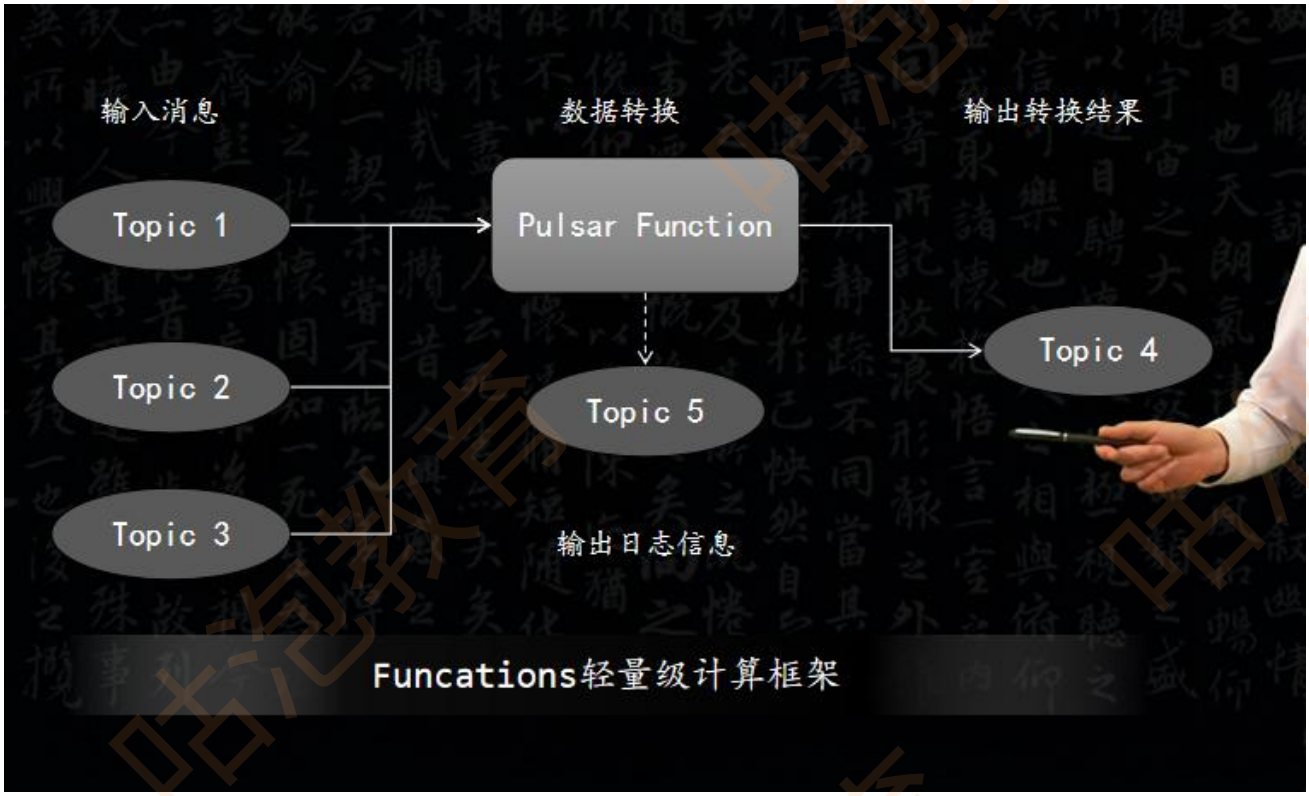
Pulsar 中的跨地域复制是将 Pulsar 中持久化的消息在多个集群间备份。在 Pulsar 2.4.0 中新增了复制订阅模式(Replicated-Subscriptions)，在某个集群失效情况下，这个功能可以在其他集群恢复消费者的消费状态，从而达到热备模式下消息服务的高可用。

8、Pulsar IO连接器



目前 Pulsar IO 支持非常多数据流的连接集成操作: 例如HDFS 、Spark、Flink 、Flume 、ES 、HBase、MySQL、Redis、MongoDB、Kafka、RocketMQ、Rabbit、ActiveMQ、Netty等等等等

9、Funcations轻量级计算框架



Pulsar可以给用户提供一个部署简单、运维简单、API 简单的 FASS（Function as a service）平台。对复杂的大数据处理框架的有力补充。

【结尾】

好了，以上就是我对Pulsar的理解。当然，大部程序员可能还不一定用的上Pulsar，今天的分享就权当给大家做科普了。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

93.Java有几种实现线程池的方式?

【导航条：引言】

一位工作3年工作经验的小伙伴面试时被问到，说，Java原生API中，提供了几种实现线程池的方式？当时他只回答了一种方式。屏幕前的你，是不是也能够完整地回答出来呢?

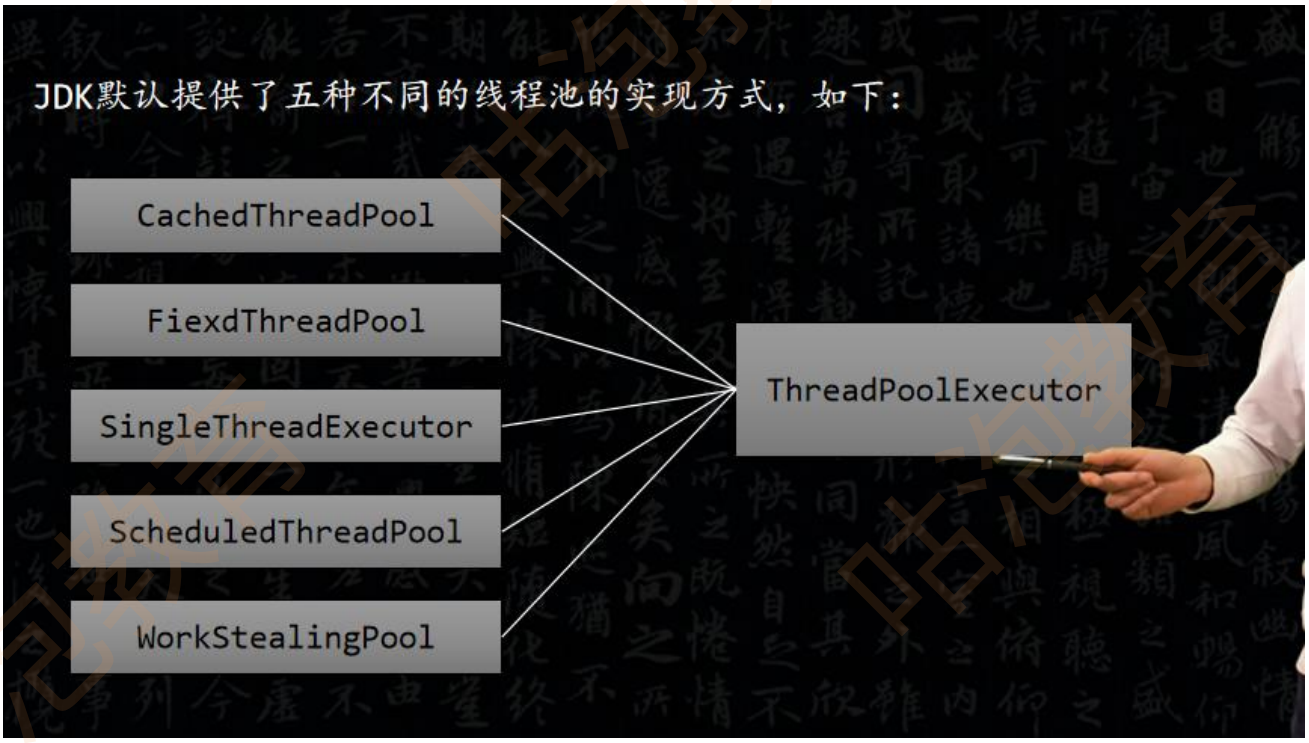
下面我给大家分享一下我的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：实现方式】

(转场，屏幕黑底白字，正中央出现“1 线程池的主要实现方式)

1 线程池的主要实现方式



JDK默认提供了五种不同的线程池的实现方式，分别是CachedThreadPool可以缓存的线程池、

FiexdThreadPool固定线程数量的线程池、SingleThreadExecutor只有一个工作线程的线程池、ScheduledThreadPool可定时调度的线程池、WorkStealingPool可并行处理的线程。

这些线程池，都是通过JDK中的工具类Executors来构建的，线程池内部的最终实现类是一个叫做ThreadPoolExecutor。

【导航条：特征介绍】

（转场，屏幕黑底白字，正中央出现“2 主要特征介绍）

2 主要特征介绍

下面我分别介绍一下每种线程池实现方式的特点：

第1种是：CachedThreadPool，是一种可以缓存的线程池，

它可以用来处理大量短期的突发流量。它有以下三个特点：

1）对最大线程数没有限制，也就是说可以登记线程数可以达到Integer.MAX_VALUE

2) 线程存活时间是60秒

3）阻塞队列用的是SynchronousQueue，这是一种不能存储任何元素的阻塞队列，也就是没提交一个任务，到这个队列里面都需要分配一个工作线程来处理。

因为CachedThreadPool的最大线程数没有限制，所以它可以用来处理大量的任务。另外，每个工作线程有可以存活60秒，是的这些工作线程可以缓存起来，去应对更多的任务处理

第2种是:FiexdThreadPool，是一种固定线程数量的线程池。

它的特点就是：核心线程和最大线程数量都是一个固定的值。如果任务比较多工作线程处理不过来的情况下，就会加入到阻塞队列里面等待执行。

第3种是：SingleThreadExecutor只有一个工作线程的线程池，

并且线程数量无法动态更改，因此，可以保证所有的任务都按照FIFO先进先出的方式顺序执行。

第4种是：ScheduledThreadPool，具有延迟执行功能的线程池，

可以用它来实现定时调度。

第5种是：WorkStealingPool，它是Java 8里面新加入的线程池，

它会在内部构造一个ForkJoinPool，利用工作窃取的算法并行处理请求。

【结尾】

好了，以上就是我对Java线程池的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

94.HashMap什么时候扩容，如何扩容?

【导航条：引言】

一位2年工作经验的小伙伴面试时被问到，说，HashMap什么时候扩容，为什么要扩容？这个问题本身不

是很难，但是这位小伙伴对底层实现原理没有太多关注，所以，被这个问题难住了。

下面我给大家分析一下这个问题的底层逻辑。

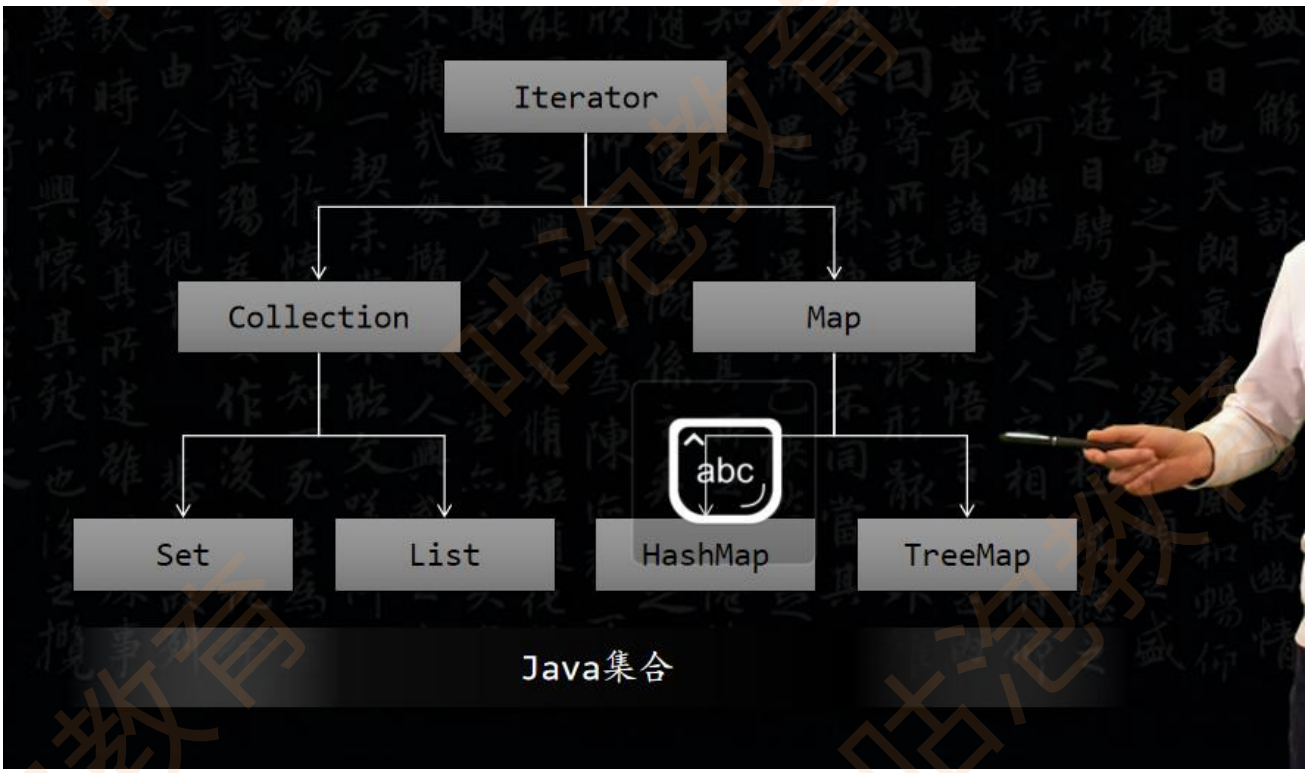
另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：存储容器】

（转场，屏幕黑底白字，正中央出现“1 数据存储容器）

1 数据存储容器

在任何编程语言中，我们经常需要在内存中去临时存放一段数据，我们可以使用官方封装好的一些集合框架。



比如说用List、HashMap、Set等等作为临时数据存储的容器。

当我们创建一个集合对象的时候，实际上就是在内存里面一次性申请了一块内存空间。而这个内存空间的大小是

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

在创建集合对象的时候去指定的。

比如HashMap的默认大小是16。

【导航条：动态扩容】

（转场，屏幕黑底白字，正中央出现“2 动态扩容）

2 动态扩容

在实际开发过程中，我们需要去存储的数据量往往是大于存储容器的默认大小的。所以，出现容量默认大小不能满足需求时，就需要扩容。而这个扩容的动作是由集合自动完成的，每种集合的扩容规则都有差异。但总的扩容原则是，当集合存储容量达到某个阈值的时候，集合就会进行动态扩容，而更好地满足更多数据存储的需求。

而HashMap中，用来存储数据的容器，本质上是一个数组结构。基本的扩容逻辑就是新建一个更长的数据，然后把原来数组里面的数据Copy到新的数组里面就可以了。

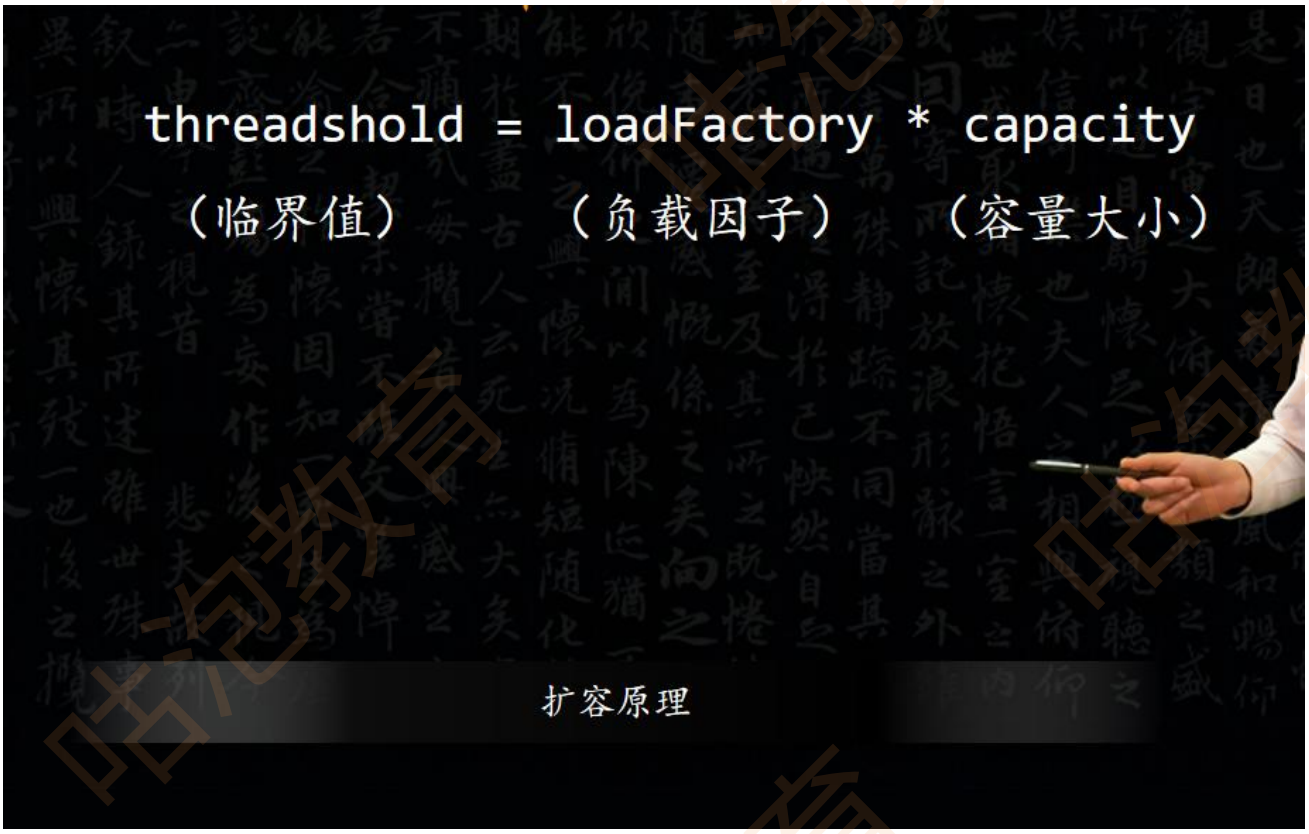
那HashMap是在什么触发扩容呢？它的扩容原理是什么呢？

【导航条：扩容原理】

（转场，屏幕黑底白字，正中央出现“2 扩容原理）

3 扩容原理

当HashMap里面的元素个数超过临界值的时候会自动触发扩容。这个临界值的计算公式如图所示：



它等于负载因子 乘以 容量大小，负载因子的默认值是0.75，而容量大小默认是16,。也就是说，第1次扩容的动作会在元素个数达到12的时候触发，扩容的大小是原来的2倍。HashMap的最大容量是Integer.MAX_VALUE也就是2的31次方减1。

由于动态扩容机制的存在，所以我们在实际应用的时候，最好在集合初始化的时候明确去指定集合的大小，从而避免频繁扩容带来性能上的消耗。

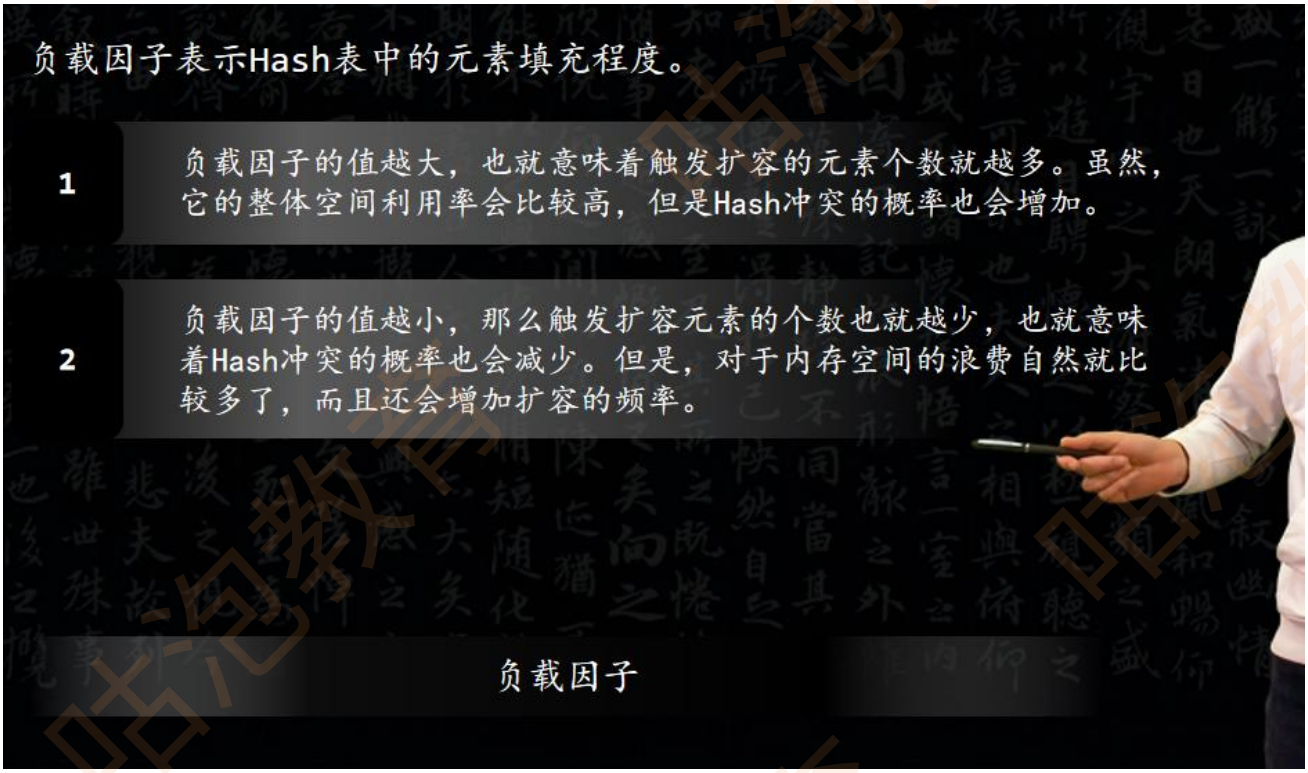
假设，我们向HashMap中插入1024个元素，如果按照默认容量大小是16的情况下，随着元素的不断增加，会造成至少7次扩容。而这7次扩容过程中，需要重新去创建新的Hash表，并且进行数据的迁移，对性能的影响是非常大的。

那为什么负载因子是0.75，而不是其他的值呢?

【导航条：负载因子】

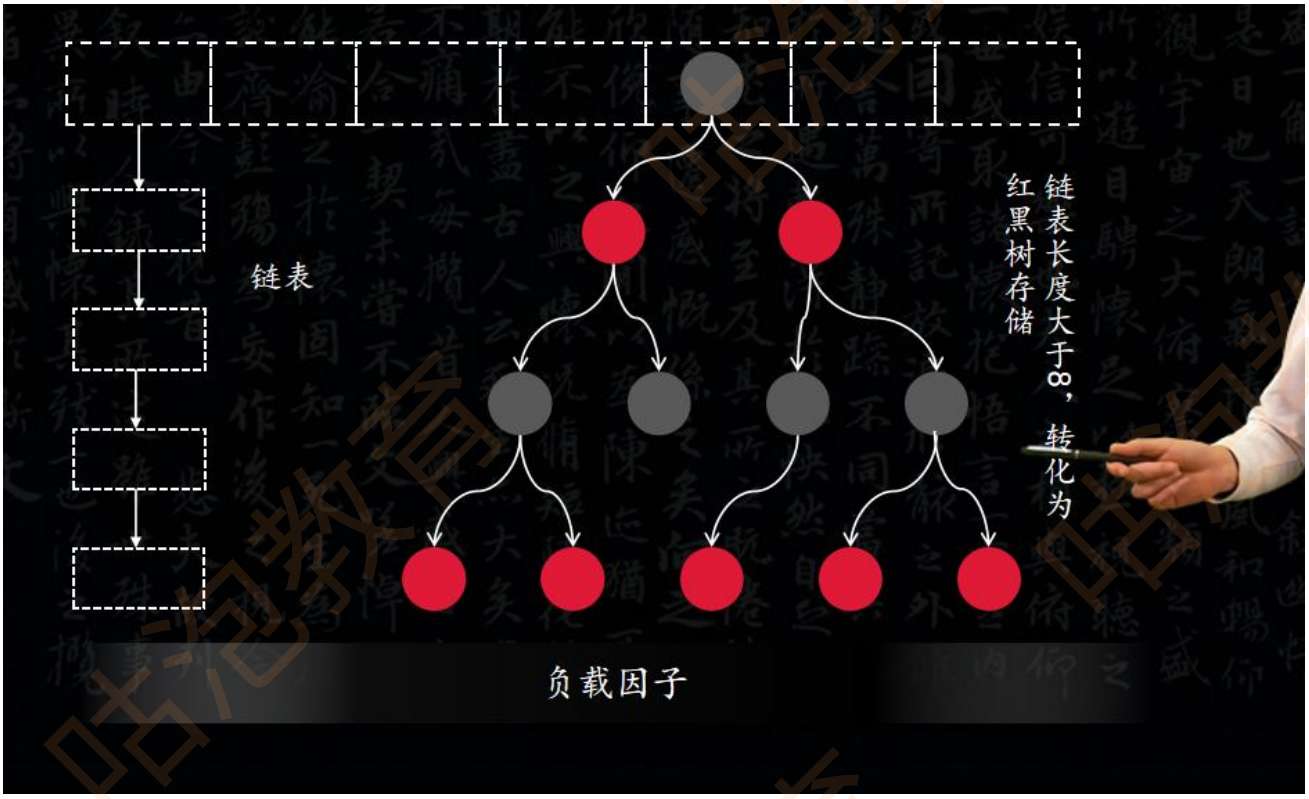
（转场，屏幕黑底白字，正中央出现“3 负载因子）

3 负载因子

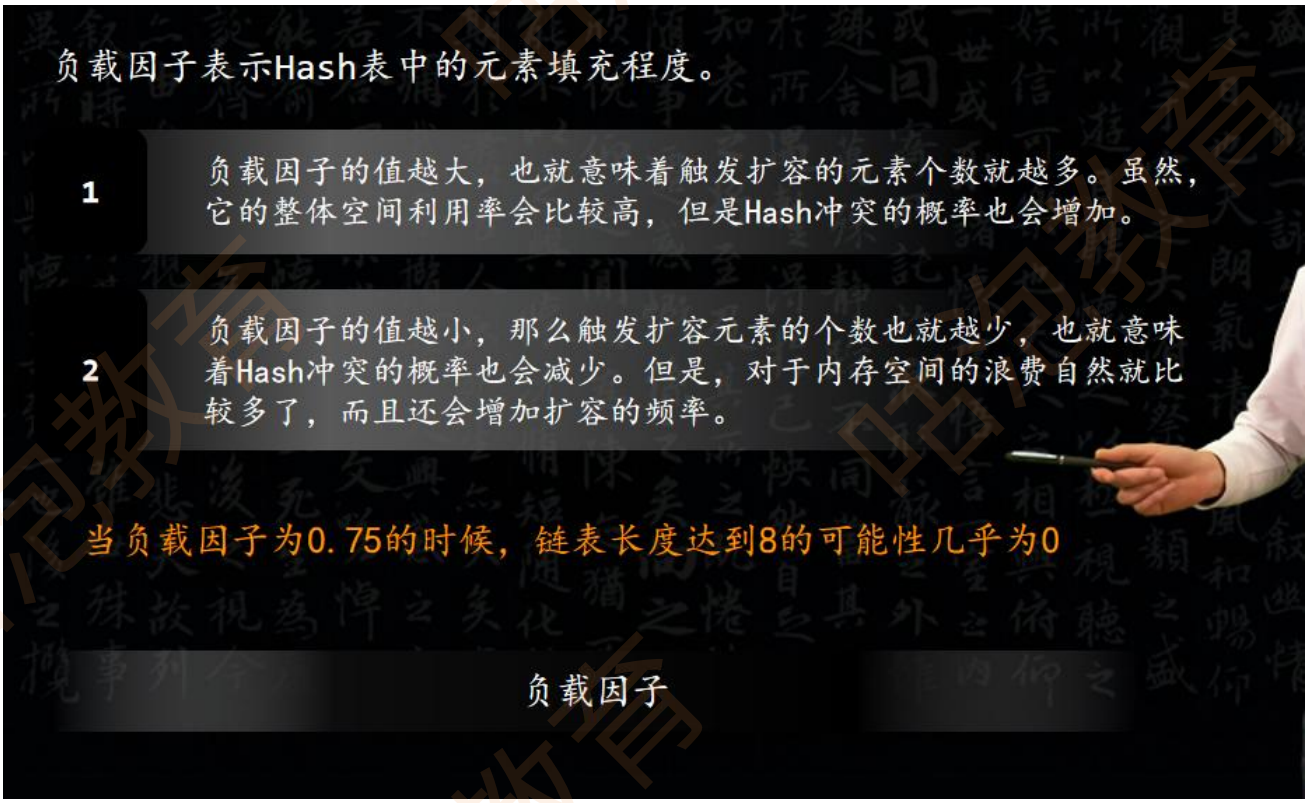


负载因子表示Hash表中的元素填充程度。负载因子的值越大，也就意味着触发扩容的元素个数就越多。虽然，它的整体空间利用率会比较高，但是Hash冲突的概率也会增加。那么，反之，负载因子的值越小，那么触发扩容元素的个数也就越少，也就意味着Hash冲突的概率也会减少。但是，对于内存空间的浪费自然就比较多了，而且还会增加扩容的频率。

因此，扩容因子的值的设置，本质上就是一个冲突的概率以及空间利用率之间的一个平衡。关于0.75这个值的来源，和统计学里面的泊松分布有关系。



我们知道，HashMap采用的是链式寻址的方式来解决Hash冲突的问题。而为了避免链表过长，导致时间复杂度增加的情况，所以，HashMap判断链表长度大于等于8的时候，就会转换为红黑树，从而提升检索的效率。



学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

当负载因子为0.75的时候，链表长度达到8的可能性几乎为0，也就是说，比较好的做到了空间成本和时间成本的平衡。

【结尾】

好了，以上就是我对HashMap扩容的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

95.请你谈一谈ArrayList自动扩容机制的实现原理

【导航条：引言】

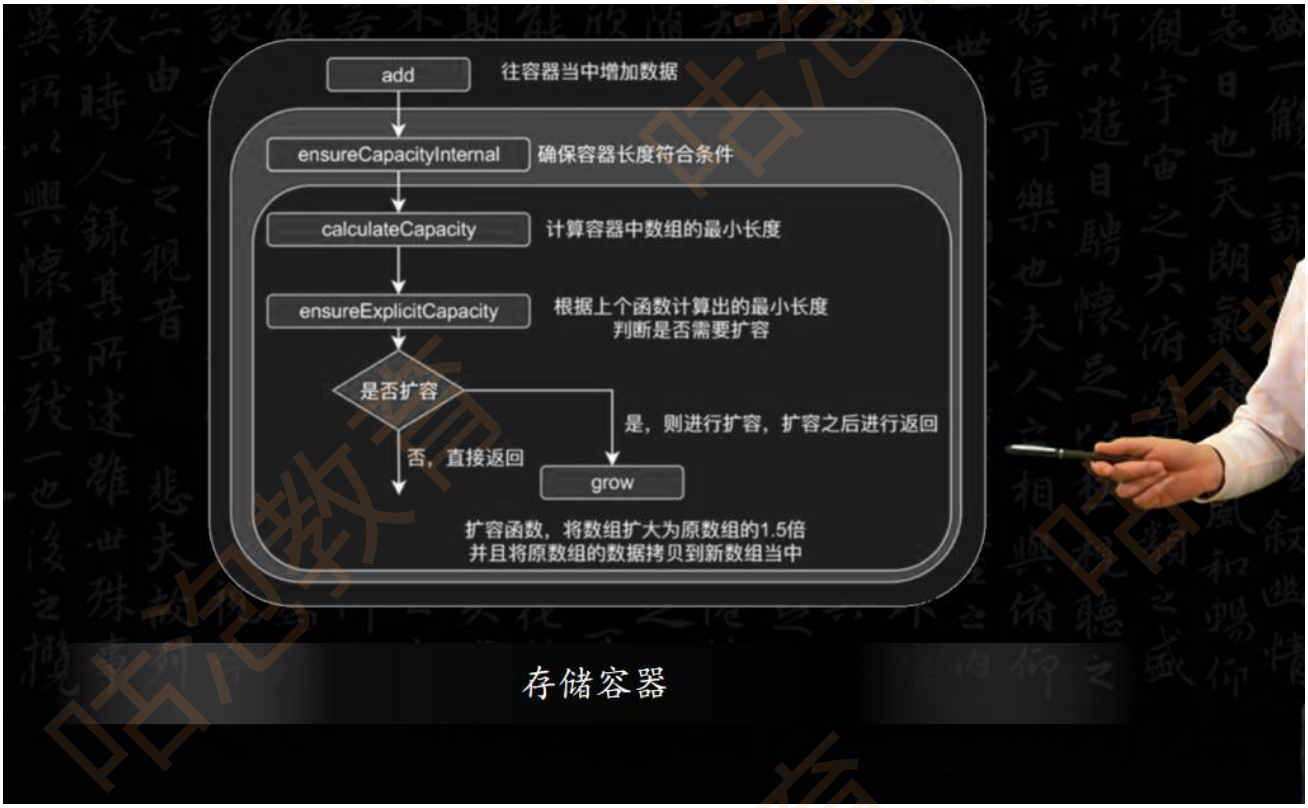
一位3年工作经验的小伙伴面试时被问到，说请你谈一谈ArrayList自动扩容机制的实现原理。这个问题对于稍微看过一点源码的小伙伴来说，其实非常简单。下面我给大家分享一下我对这个问题的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：存储容器】

（转场，屏幕黑底白字，正中央出现“1 存储容器）

1 存储容器

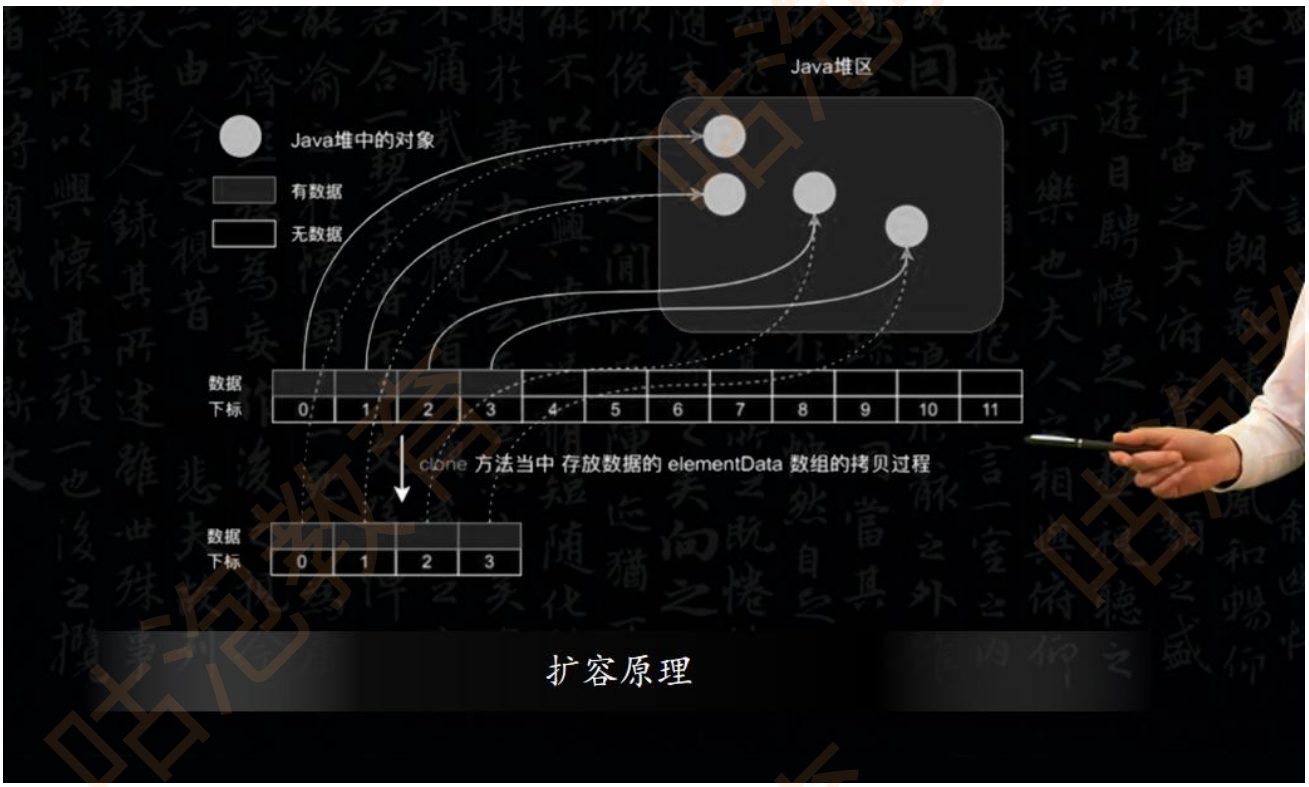


ArrayList是一个数组结构的存储容器，默认情况下，数组的长度是10。当然，我们也可以在构建ArrayList对象的时候，直接指定初始长度。那么，随着程序的运行，不断地往ArrayList中添加数据，当添加的数据达到10个的时候，ArrayList就没有足够的容量来存储后面添加的数据。那么，这个时候，ArrayList会触发自动扩容机制。

【导航条：扩容原理】

（转场，屏幕黑底白字，正中央出现“2 扩容原理）

2 扩容原理



扩容的流程其实也很简单，首先创建一个新的数组，这个数组的长度是原来数组长度的1.5倍。然后，使用Arrays.copyOf()方法把原数组中的数据拷贝到新数组中。扩容完成之后，再把当前要添加的数据加入到新的数组中，从而完成动态扩容的过程。

【结尾】

好了，以上就是我对这个问题的理解。作为一个业务程序员，虽然工作的性质是让大家去写CRUD，不需要过多的去关注技术的底层实现原理。但是，在未来的职业晋升过程中，对技术的理解程度非常重要。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

96.谈谈你对RocketMQ分布式事务原理的理解

【导航条：引言】

有位工作五年的小伙伴在面试的时候被问到RocketMQ的分布式事务实现原理。他说他只知道RocketMQ能够支持事务，但是没有了解过它的事务实现原理。

今天，我给大家分享一下我对这个问题的理解。

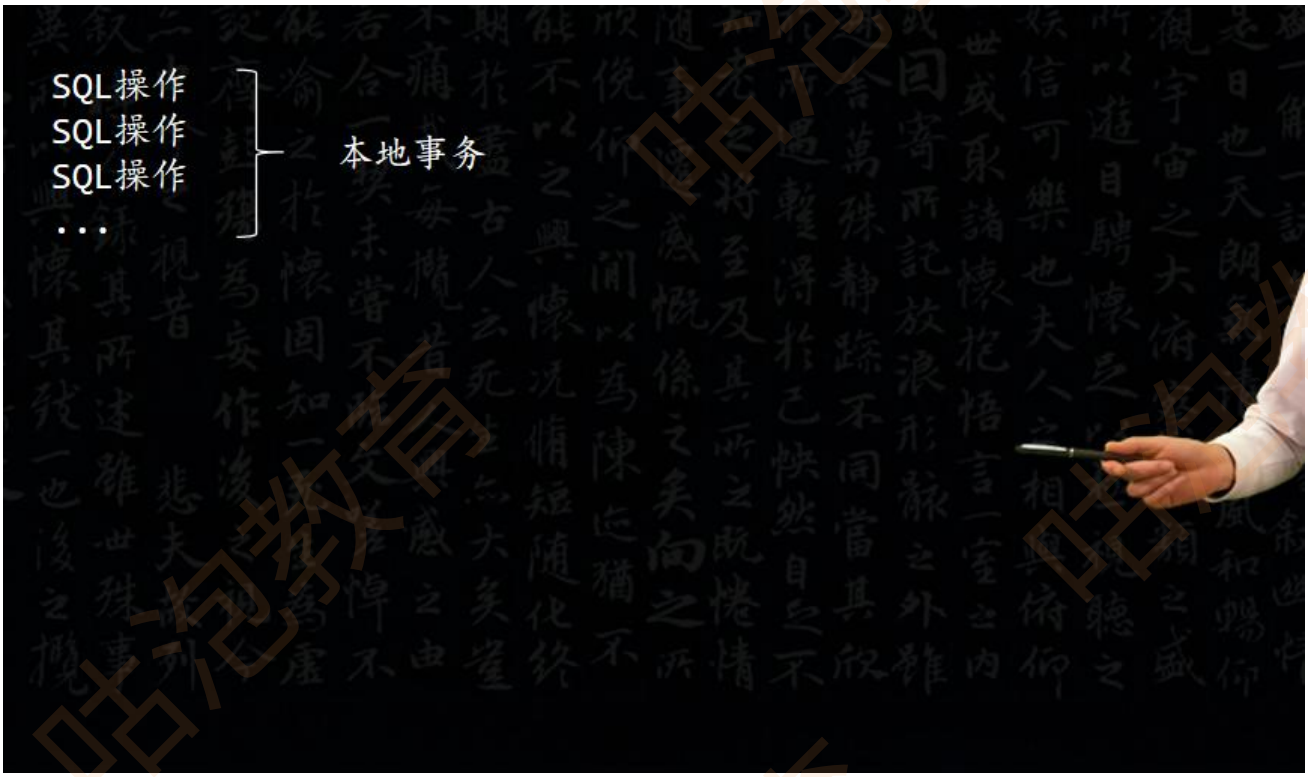
另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：应用场景】

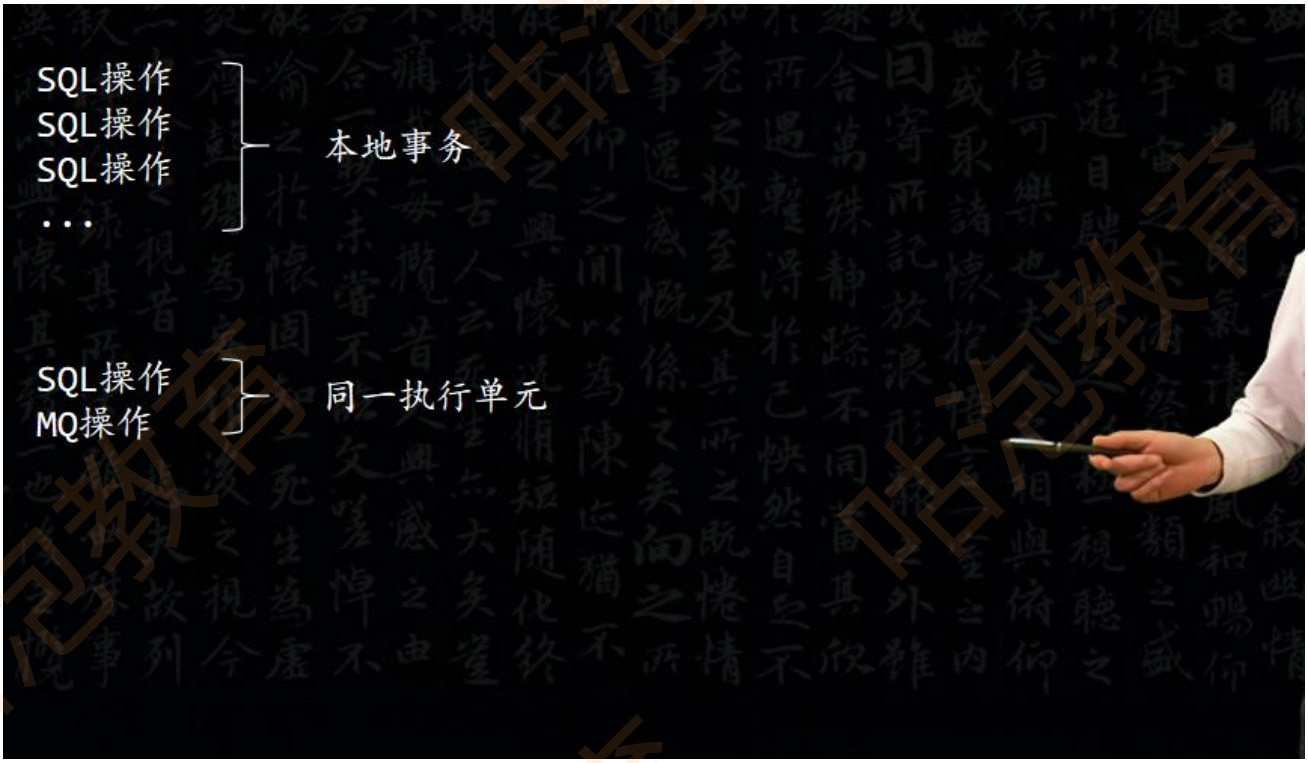
（转场，屏幕黑底白字，正中央出现“1 分布式事务应用场景）

1 分布式事务应用场景

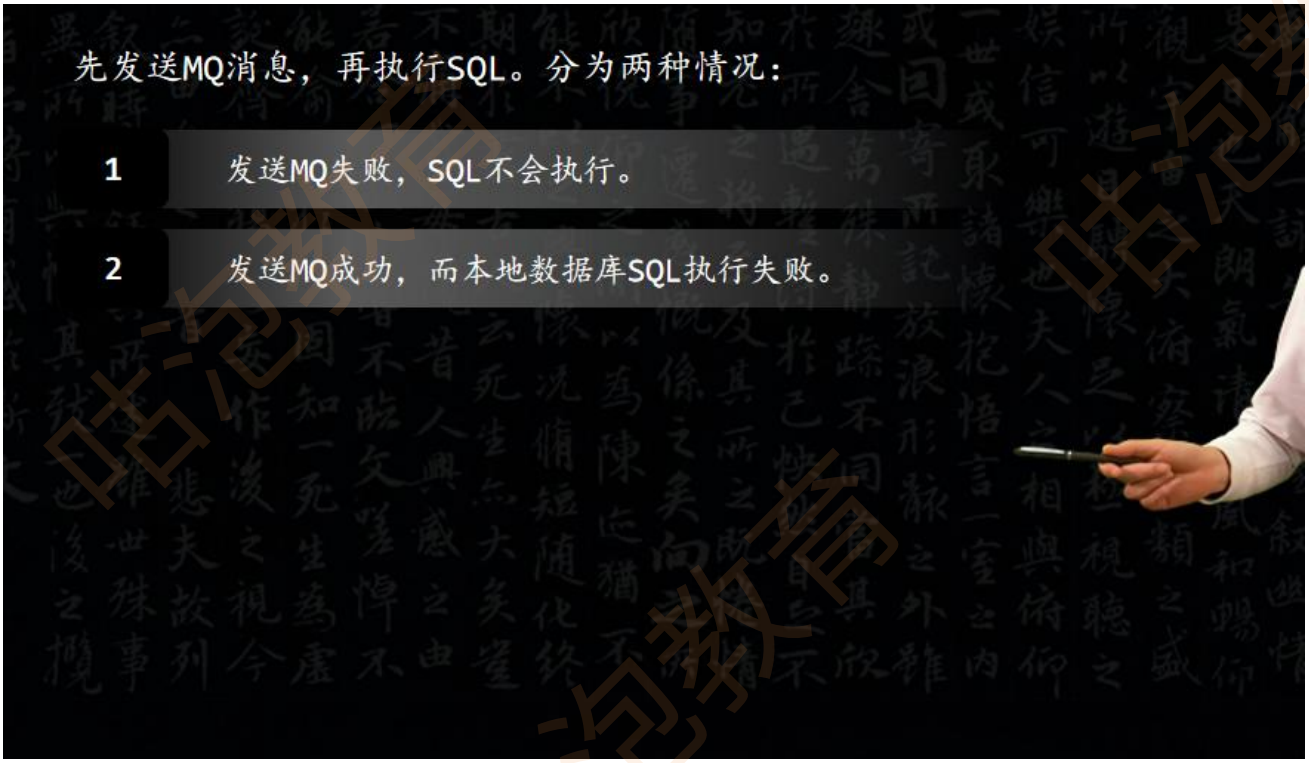
随着应用的拆分，从单体架构变成分布式架构，那么每个服务或者模块也会有自己的数据库。一个业务流程的完成需要经过多次的接口调用或者多条MQ消息的发送。



那么问题来了，如果是执行多条SQL语句，数据库的本地事务可以保证原子性。



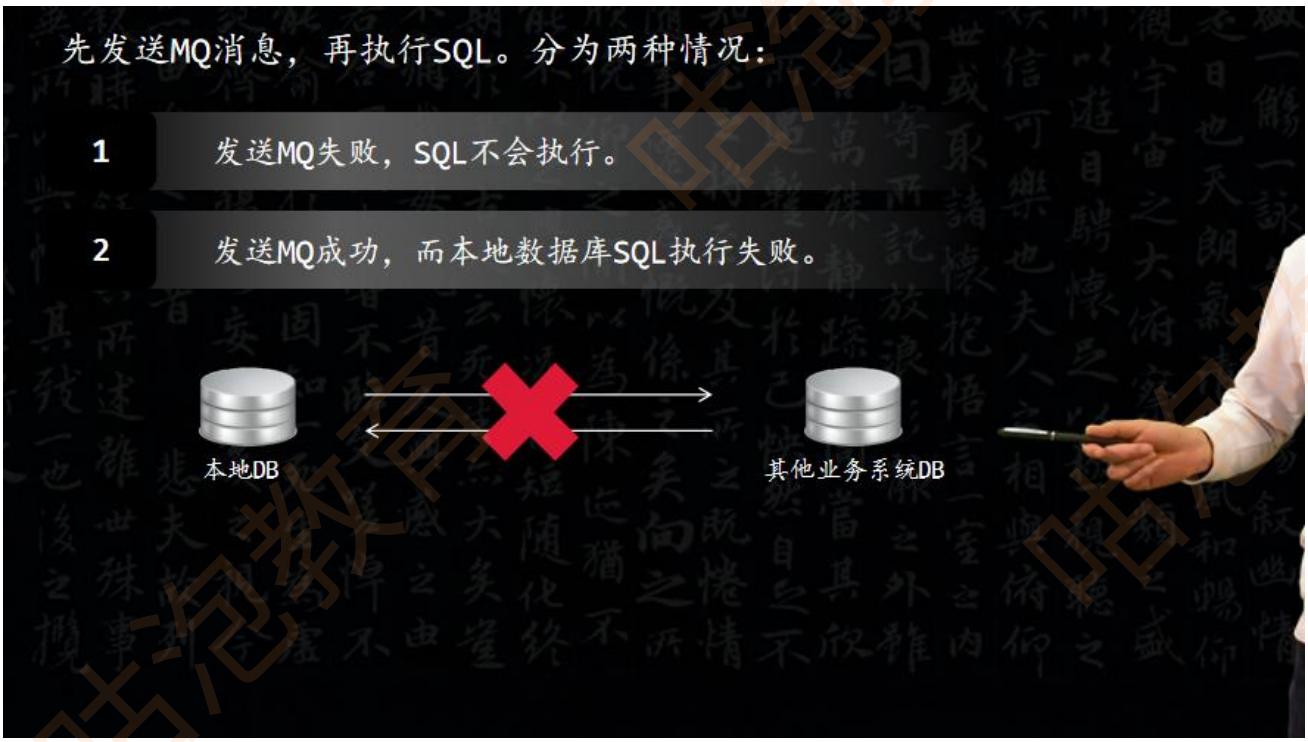
但，如果是一条SQL操作，再加一条MQ的操作，如何才能把它们两个放在同一个逻辑单元里面执行呢？是先执行SQL还是先发送MQ呢?



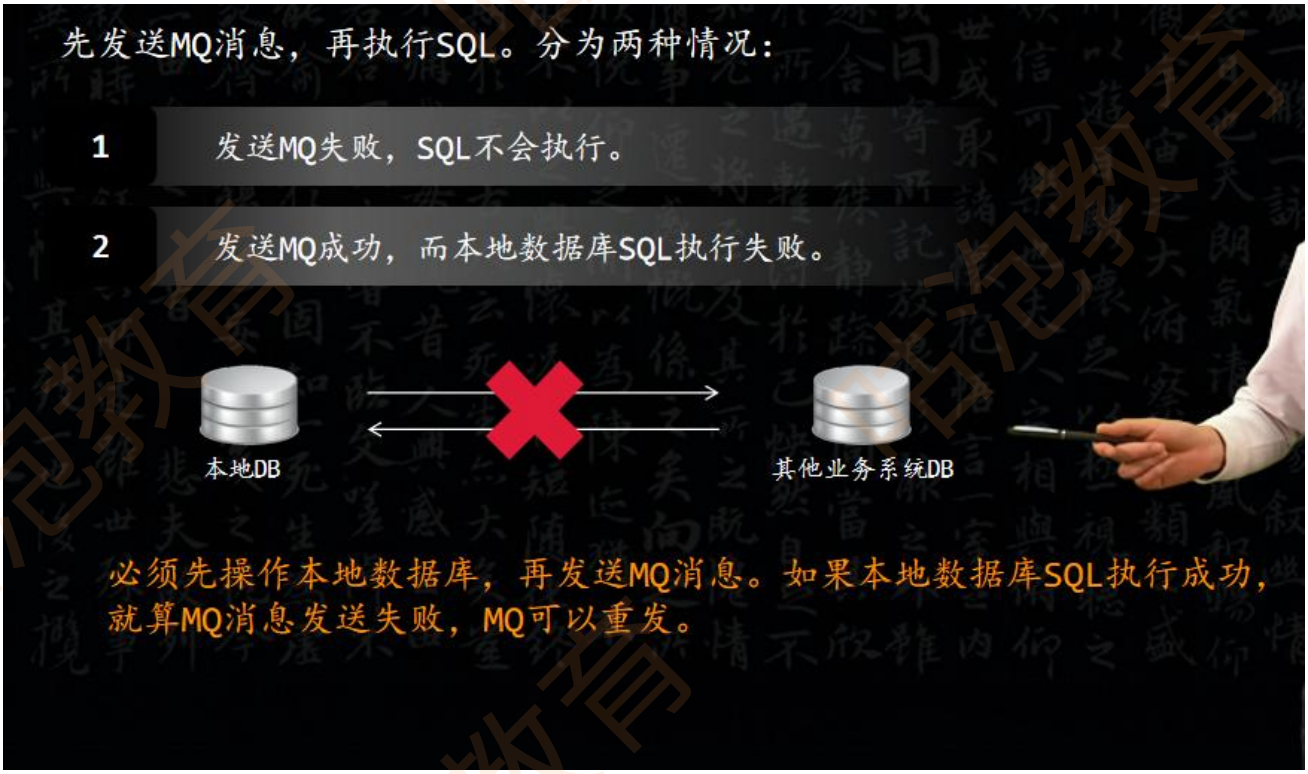
我们来分析一下情况，如果是先发送MQ消息，再执行SQL。这个时候就要分为两种情况：

第1种情况：如果发送MQ失败了，当然SQL也就不会执行了。

第2种情况：如果发送MQ成功了，而本地数据库SQL执行失败。比如出现了网络异常，主键重复或者字段超长等等。



也就是说，下游的业务系统拿到了最新的数据，而自己本地的数据库反而没有。这个时候，本地数据库的数据跟其他系统已经登记的数据就不一样了，而发出去的消息又不可能撤回，有可能已经被消费了，这个叫做覆水难收。



因此，在分布式应用场景中，我们需要调整一下代码执行流程，也就是说必须先操作本地数据库，再发送MQ消息。如果本地数据库SQL执行成功，就算MQ消息发送失败，MQ还可以重发。

【导航条：实现原理】

（转场，屏幕黑底白字，正中央出现“2 分布式事务实现原理）

2 分布式事务实现原理

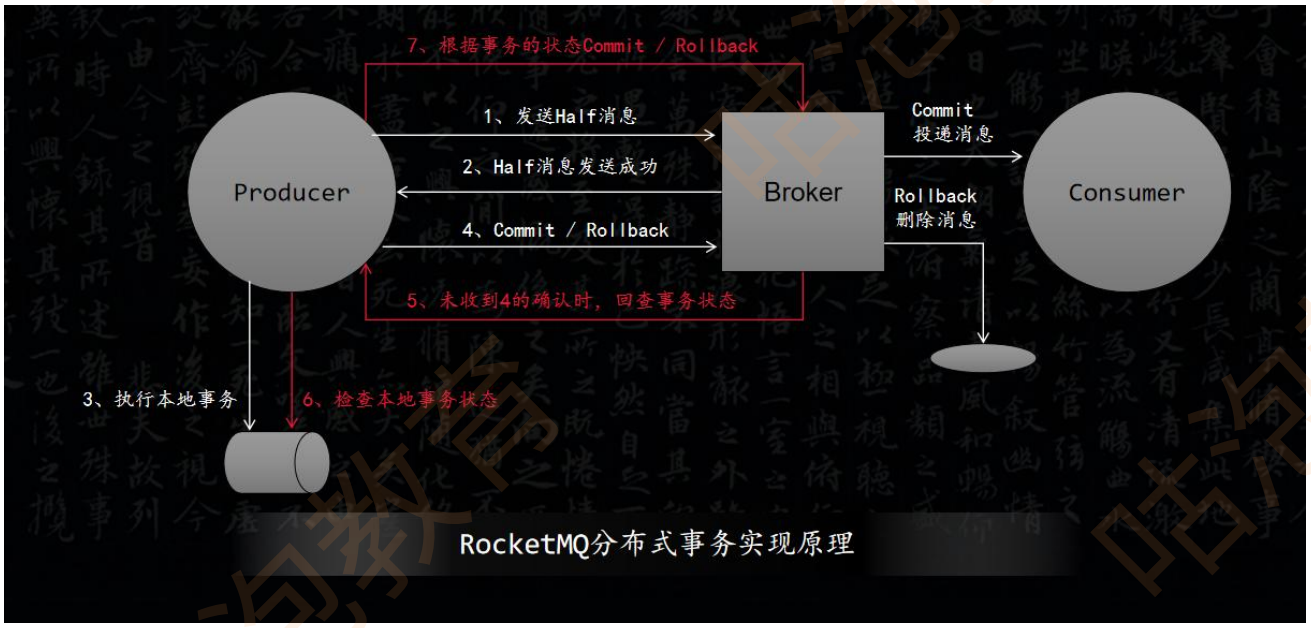
那基于上面的应用场景，应该如何设计发送消息的流程，才能让这两个操作要么都成功，要么都失败呢？

其实，可以参照XA两阶段提交的思想，把发送消息分成两步，然后把操作本地数据库也包括在这个流程中。那么，在介绍原理之前，先科普一下两个新的概念：

1、半消息（Half Message）：也就是暂不能投递消费者的消息。发送方已经将消息成功发送到了 MQ 服务端，但是服务端未收到生产者对这条消息的二次确认，这个时候，这条消息会被标记为“暂不能投递”状态。

2、消息回查（Message Status Check）：由于网络闪断、生产者应用重启等原因，导致某条事务消息的二次确认丢失，MQ 服务端通过扫描发现某条消息长期处于“半消息”时，需要主动向消息生产者询问该消息的最终状态，要么是Commit，要么Rollback。

下面给大家介绍一下RocketMQ的分布式事务实现原理，如图所示，一共分为七个步骤：



第一步：生产者向 MQ 服务端发送消息。

第二步：MQ 服务端将消息持久化成功之后，向发送方 ACK 确认消息已经发送成功，此时消息为半消息。

第三步：发送方开始执行本地数据库事务逻辑。

第四步：发送方根据本地数据库事务执行结果向 MQ Server 提交二次确认，MQ Server 收到 Commit 状态则将半消息标记为可投递，订阅方最终将收到该消息；MQ Server 收到 Rollback 状态则删除半消息，订阅方将不会接受该消息。

第五步：在断网或者是应用重启的特殊情况下，按步骤4提交的二次确认最终未到达 MQ Server，经过固定时间后 MQ Server 将对该消息发起消息回查。

第六步：发送方收到消息回查后，需要检查对应消息的本地事务执行的最终结果。

第七步：发送方根据检查得到的本地事务的最终状态再次提交二次确认，MQ Server 仍按照步骤4对半消息进行操作（Commit/Rollback）。

【结尾】

好了，以上就是我对RocketMQ分布式事务的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

97.谈谈你对JVM中主要GC算法的理解

【导航条：引言】

有位工作五年的小伙伴面试被问到JVM相关的问题，说请你谈谈你对JVM中主要GC算法的理解，我给大家分享一下我的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

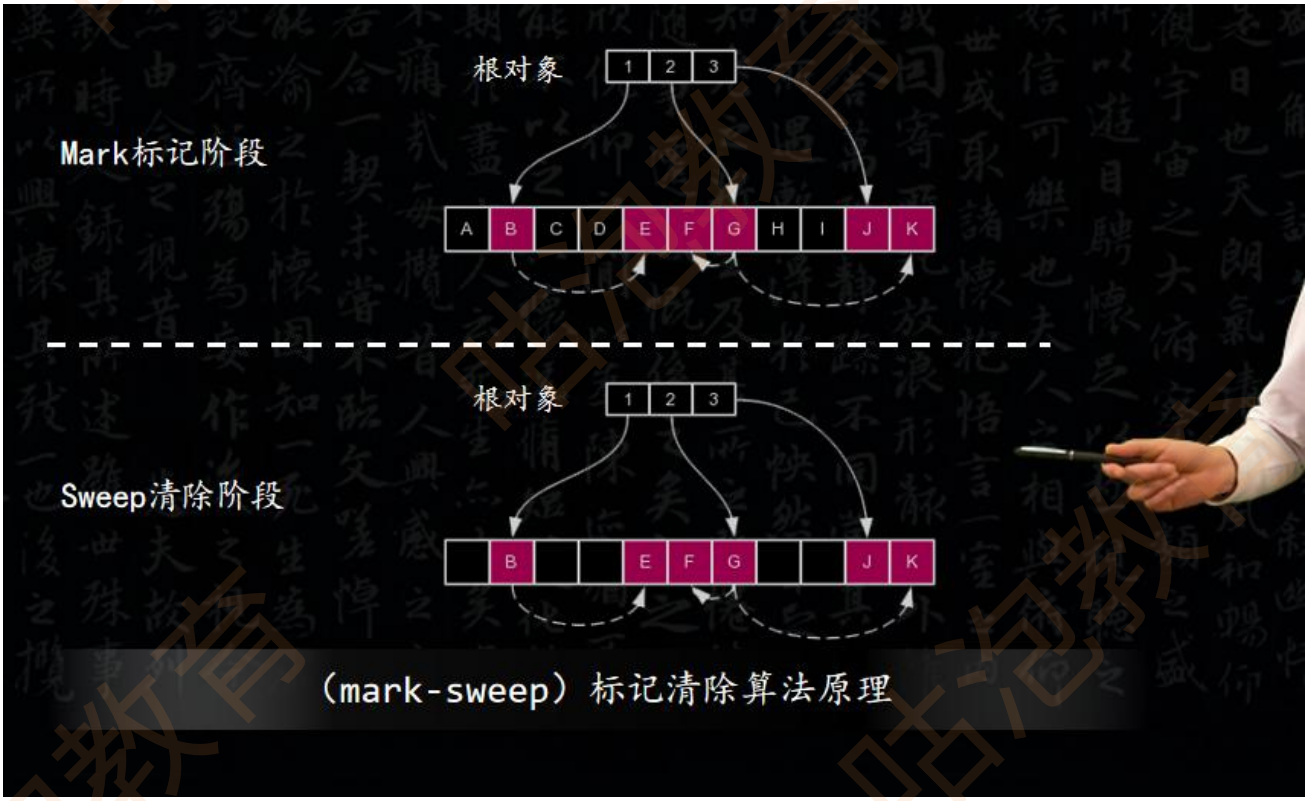
【导航条：垃圾回收算法】

（转场，屏幕黑底白字，正中央出现“1 主要垃圾回收算法）

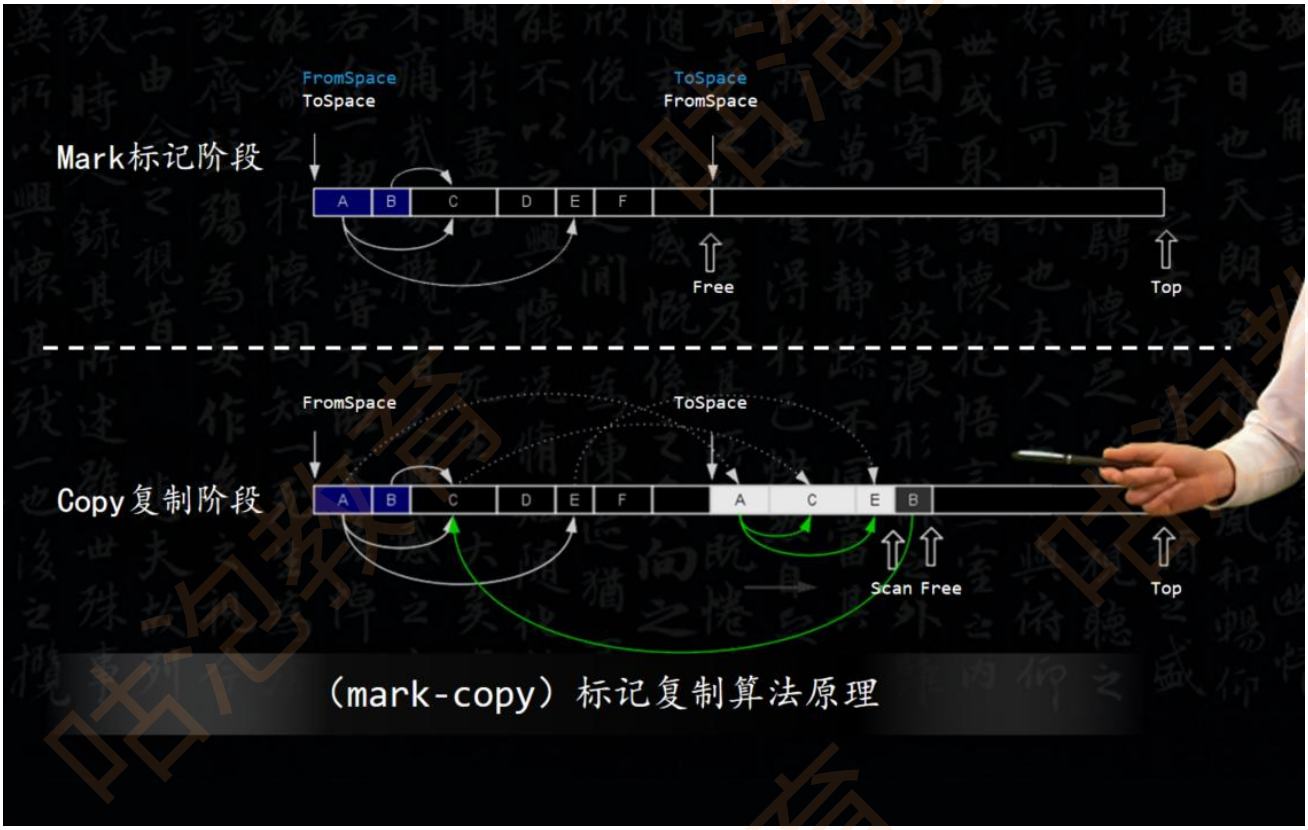
1 主要垃圾回收算法

GC翻译过来叫做垃圾回收，那么JVM中主要的垃圾回收算法有三种，分别是：标记清除算法、标记复制算法和标记整理算法。

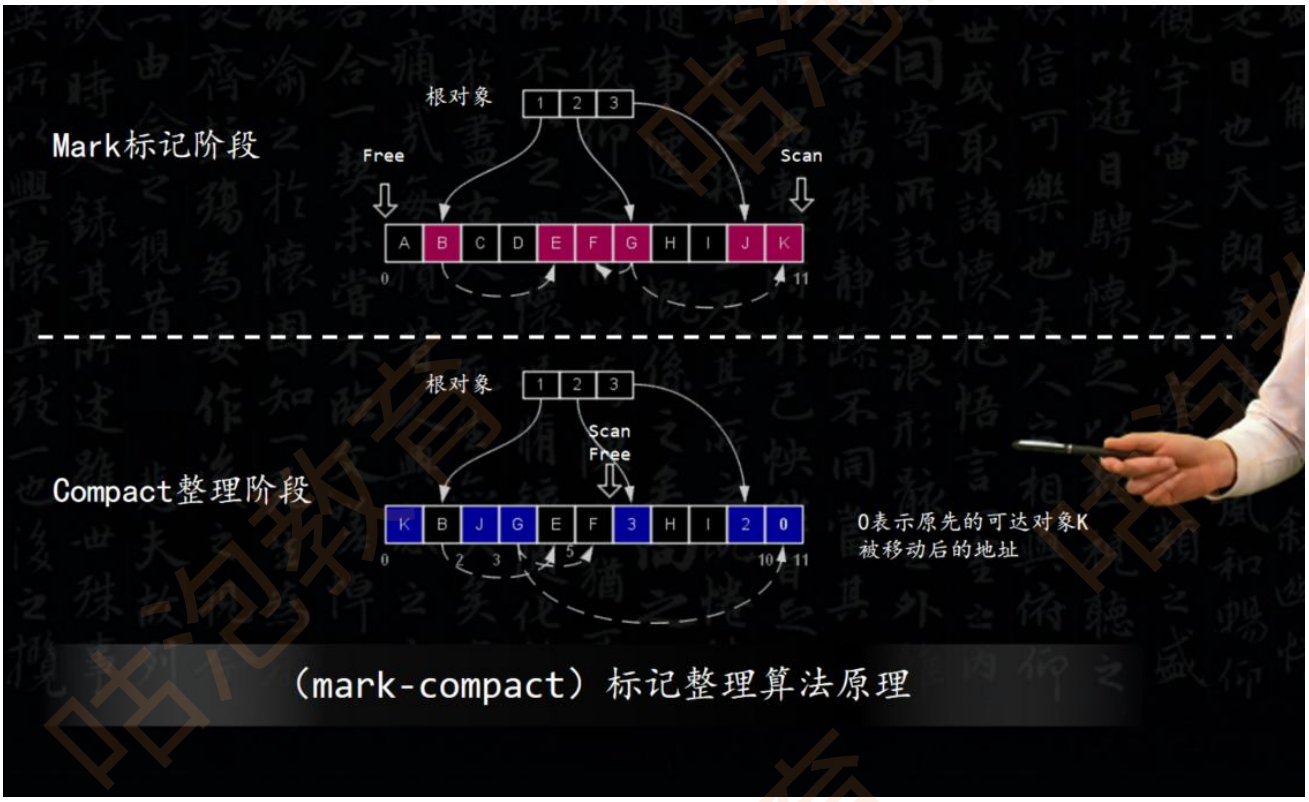
下面，我介绍一下每种算法的核心原理：



首先是，标记清除算法。它是将存活的对象打上标记，那么没有被标记的对象就是需要被回收的垃圾对象，这些垃圾对象会被垃圾回收器直接回收。



然后是，标记复制算法。它是把内存分为两等份，每次只使用其中的一份，等到正在使用的这部分内存满了之后，就会标记出存活的对象，然后把存活的对象拷贝到另一部分闲置的内存中，那留在另一部分内存中的对象，会全部被垃圾回收器回收。那么，原来空闲的内存空间就会变成使用中的状态，而原来使用中的内存空间会被闲置出来继续使用。这就是标记复制算法的一次完整的GC。然后，一直重复这个循环。



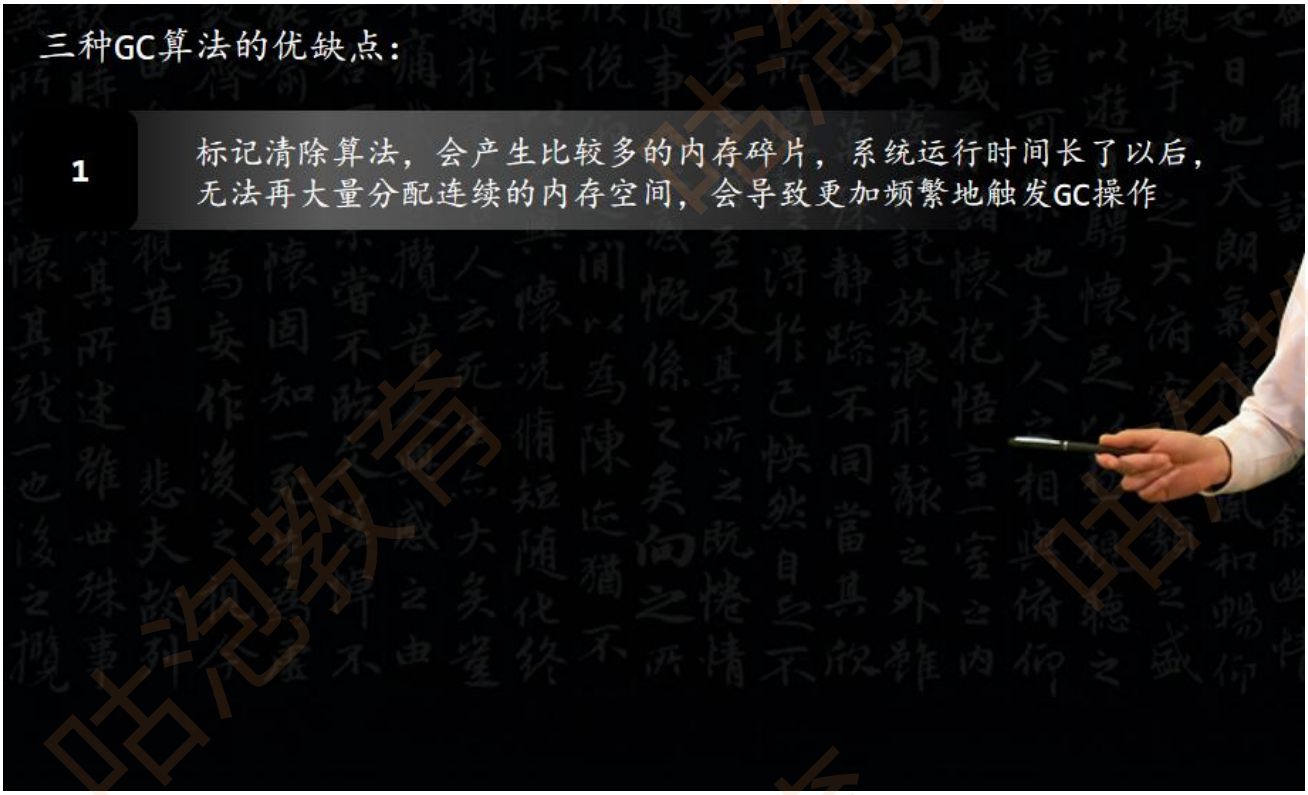
最后是，标记整理算法。它是先标记出存活的对象，然后，把所有存活的对象整理到内存空间的另一端，而没有被标记的对象就是可以被覆盖或者是释放。

【导航条：优缺点分析】

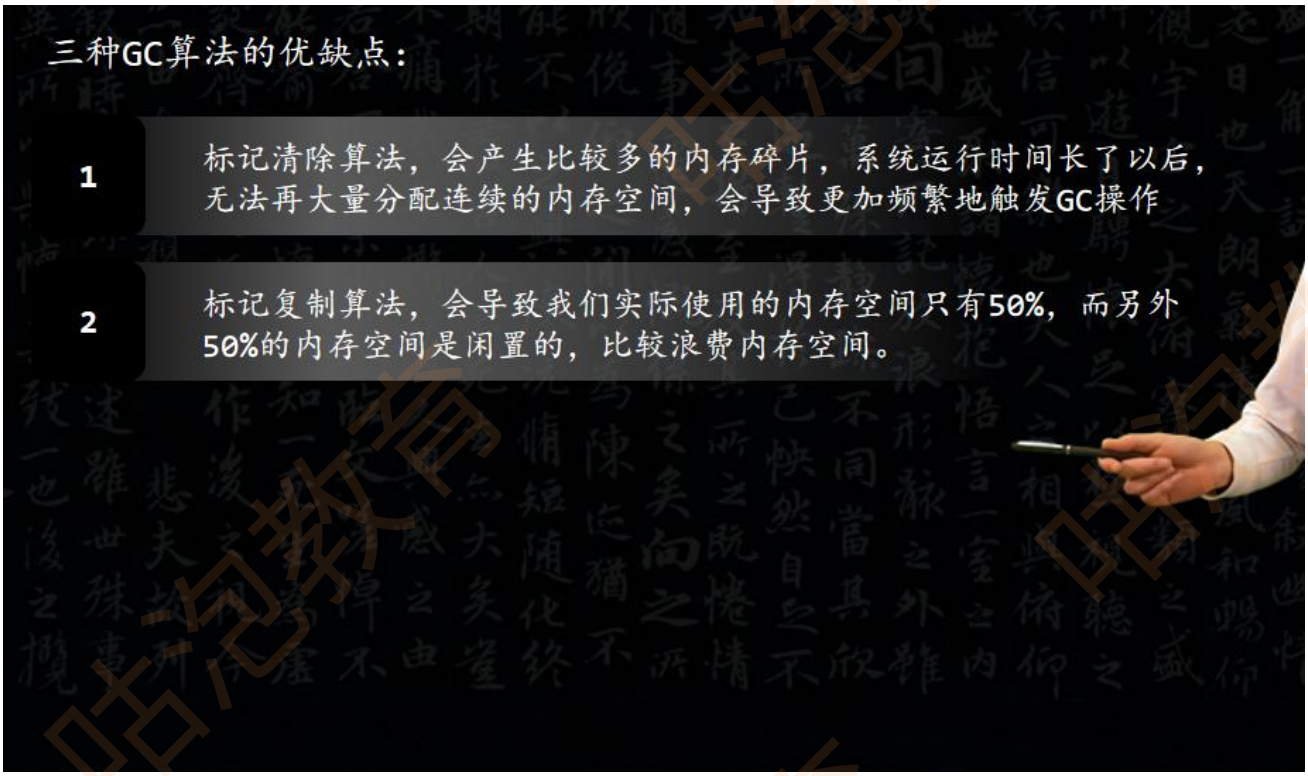
（转场，屏幕黑底白字，正中央出现“2 优缺点分析）

2 优缺点分析

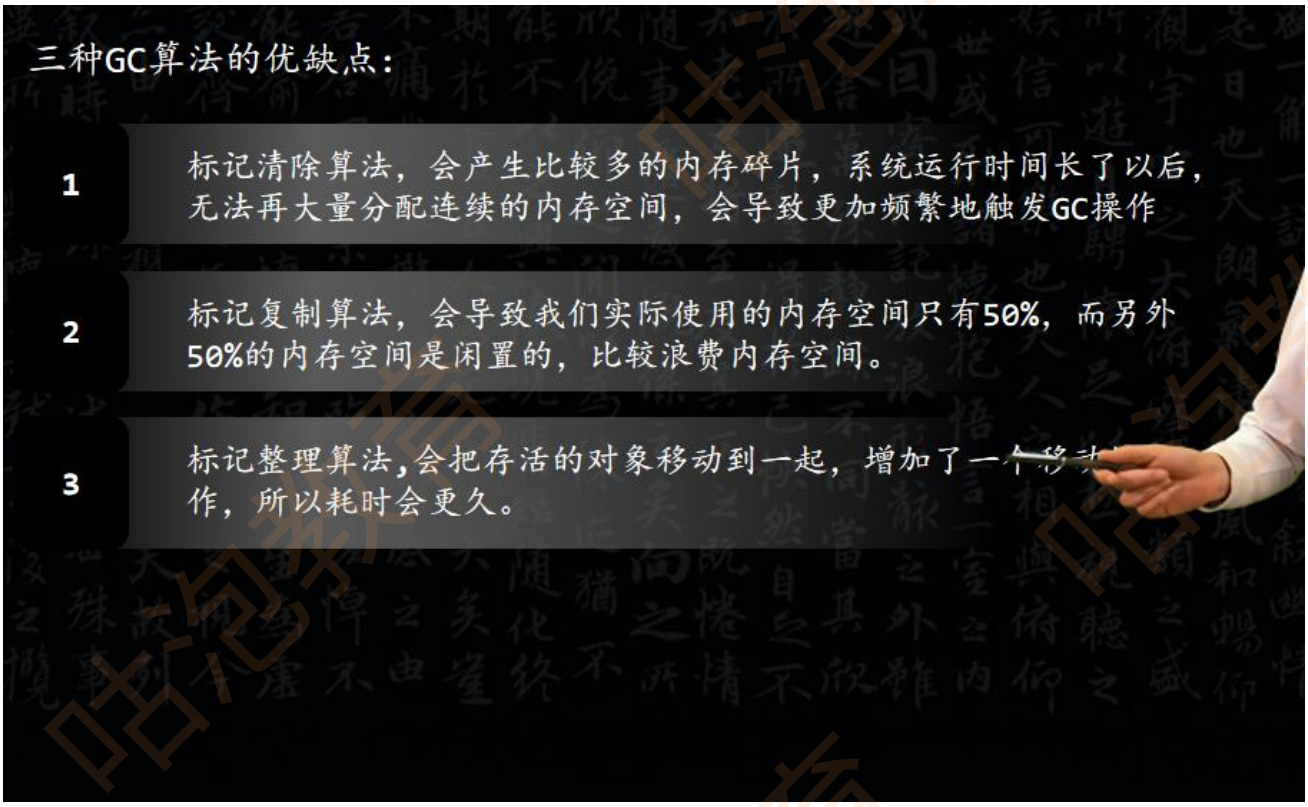
接下来，分析一下三种GC算法的优缺点。



首先来看，标记清除算法。它主要的缺点是会产生比较多的内存碎片，而这些内存碎片，随着系统运行时间的推移，系统运行时间长了以后，无法再大量分配连续的内存空间。这样的话，就会导致更加频繁地触发GC操作。



然后是，标记复制算法，它主要的缺点是，会导致我们实际使用的内存空间只有50%，而另外50%的内存空间是闲置的。所以，比较浪费内存空间。而且，如果大量复制对象的话，垃圾回收的耗时就比较长了，所以，这种算法，更加适合处理存活对象少，垃圾对象比较多的场景。



最后是，标记整理算法。它的本质和标记清除算法是基本差不多，只不过标记整理算法多了一个动作，会把存活的对象移动到一起。从垃圾收集和清理效率来看，增加了一个移动的动作，所以耗时会更久。但是，新分配对象的耗时就会下降。

【导航条：分代回收】

（转场，屏幕黑底白字，正中央出现“3 分代回收）

3 分代回收

因为Java对象基本上都是临时的对象，很快就会被回收。所以，JVM的内存是分代的设计，根据对象在内存中的存活时间，分为年轻代、老年代和永久代。

那么年轻代呢，采用的是标记复制算法，在每次复制的时候，存活下来的对象会很少。

老年代呢，是经历过几次GC的对象，JVM会认为它可能继续存活下去，就不太适合采用标记复制算法，所以，老年代采用的是标记清除算法，比如CMS这种回收器就是采用标记清除的方式。

永久代呢，表示一直会存活的对象，只有在触发Full GC的时候才会被回收。所以，永久代对象创建过多的话，比较容易出现内存溢出。最典型的场景就是，在JSP页面比较多的情况，容易出现永久代内存溢出。

【结尾】

好了，以上就是我对垃圾回收算法的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

98.谈谈你对IO多路复用机制的理解

【导航条：引言】

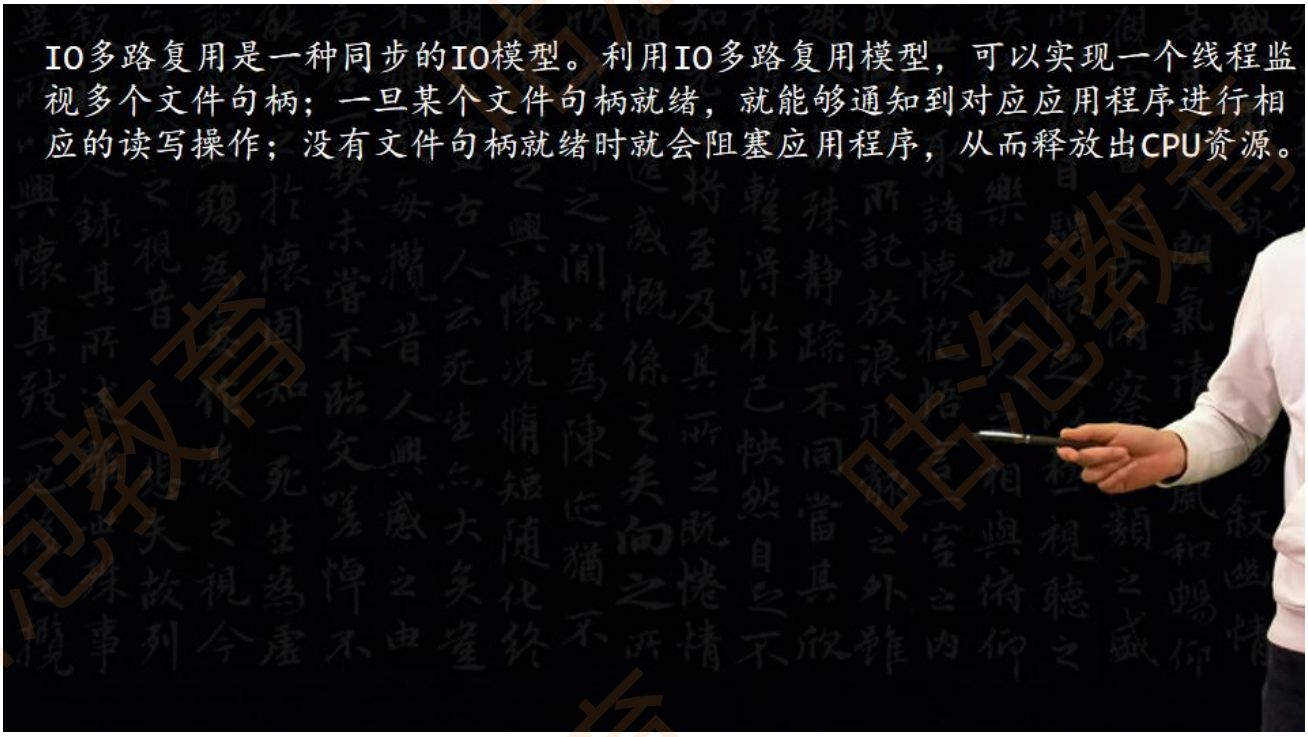
一位工作5年的小伙伴面试时被问到IO相关的问题，说，谈谈你对IO多路复用机制的理解。当时他说只是听过多路复用，具体细节没有了解过。今天，我给大家分享一下我的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

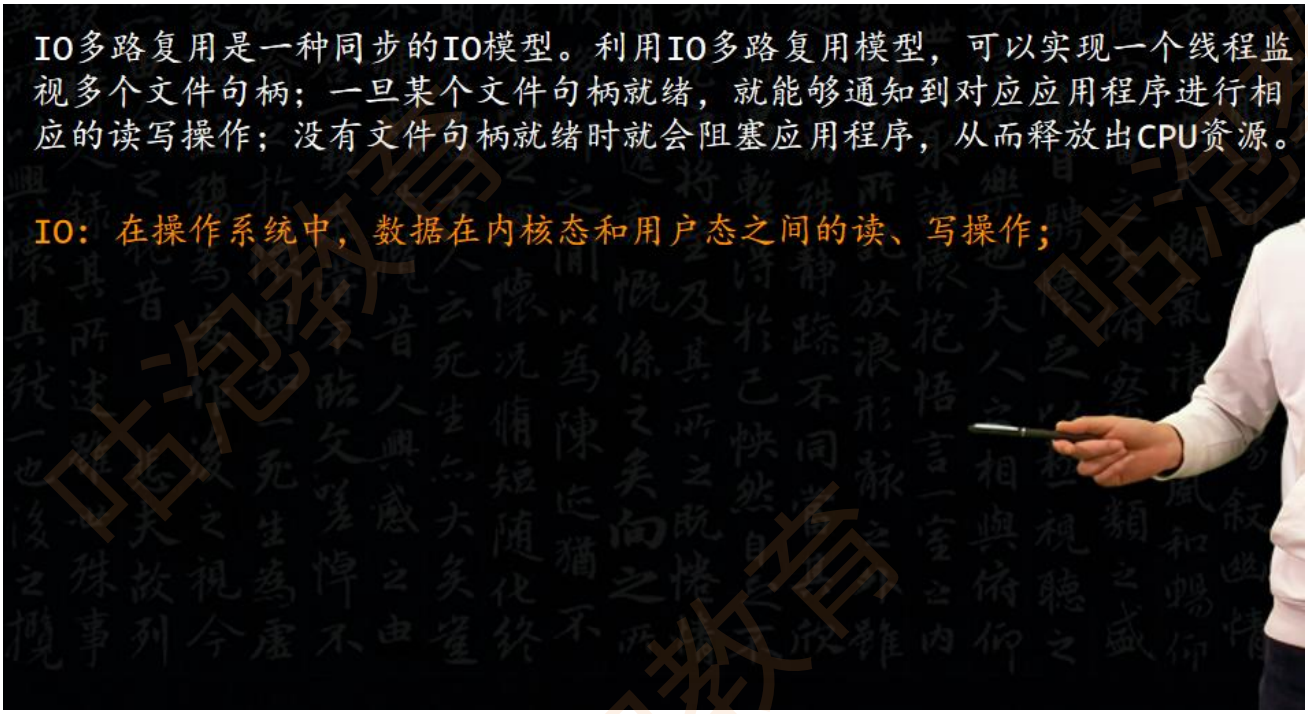
【导航条：IO多路复用】

（转场，屏幕黑底白字，正中央出现“1 IO多路复用）

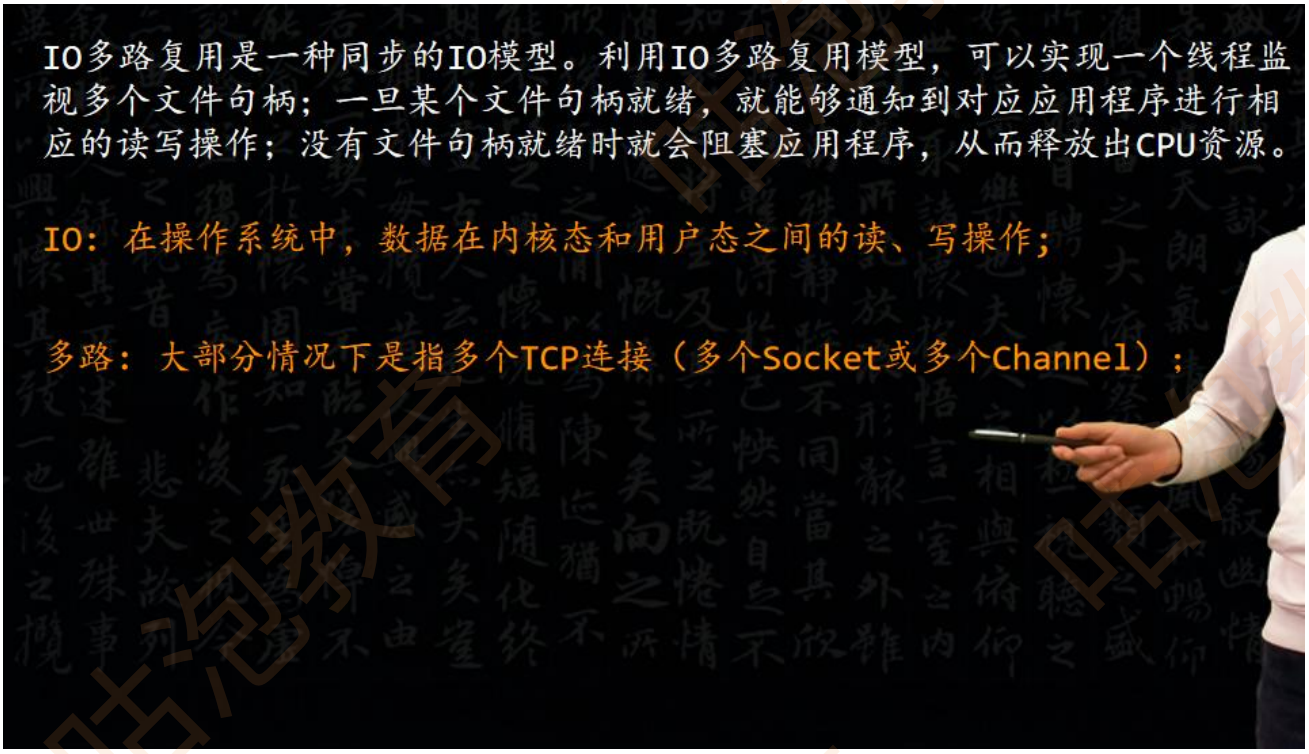
1 IO多路复用



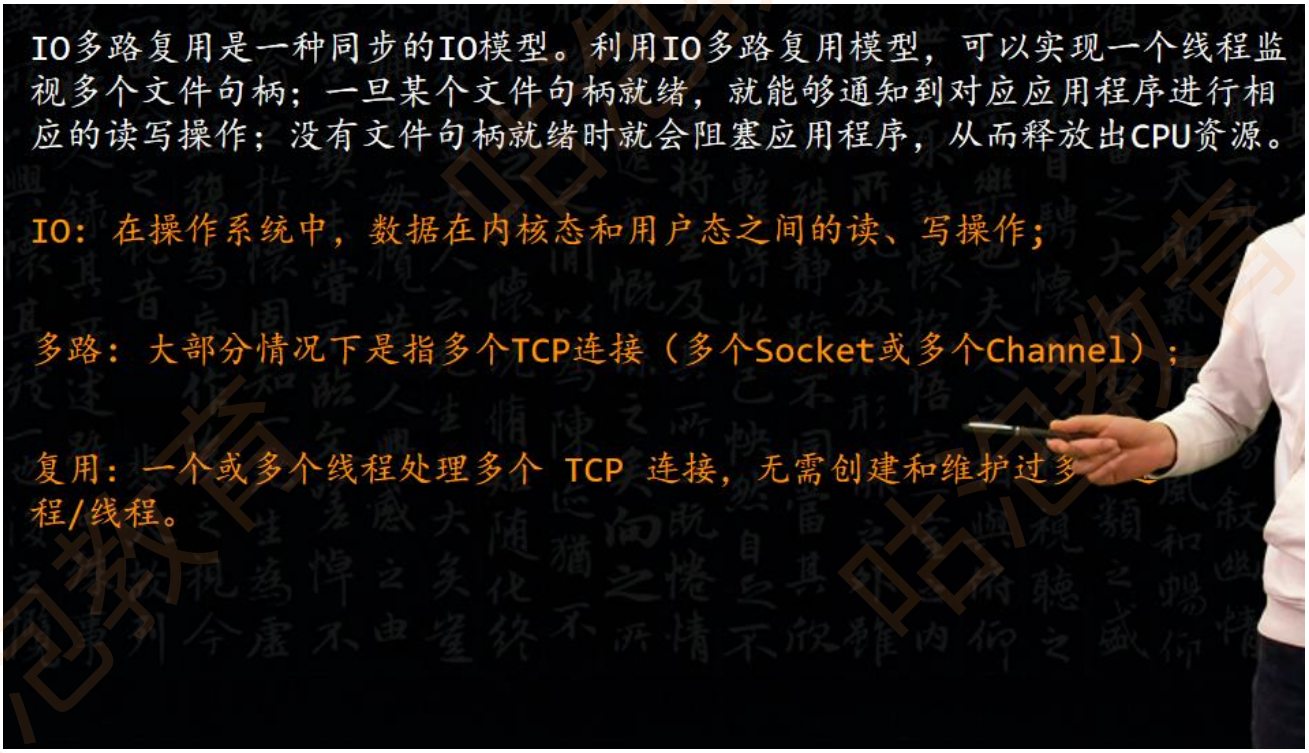
IO多路复用是一种同步的IO模型。利用IO多路复用模型，可以实现一个线程监视多个文件句柄；一旦某个文件句柄就绪，就能够通知到对应应用程序进行相应的读写操作；没有文件句柄就绪时就会阻塞应用程序，从而释放出CPU资源。



IO可以理解为，在操作系统中，数据在内核态和用户态之间的读、写操作，大部分情况下是指网络IO；



多路大部分情况下是指多个TCP连接，也就是多个Socket 或者多个Channel；



复用是指复用一个或多个线程资源。IO多路复用意思就是说，一个或多个线程处理多个 TCP 连接。尽可能地减

少系统开销，无需创建和维护过多的进程/线程。

实现IO多路复用的模型有三种，分别是Select、poll 和 epoll。下面详细介绍一下三种多路复用模型的基本原理和优缺点：

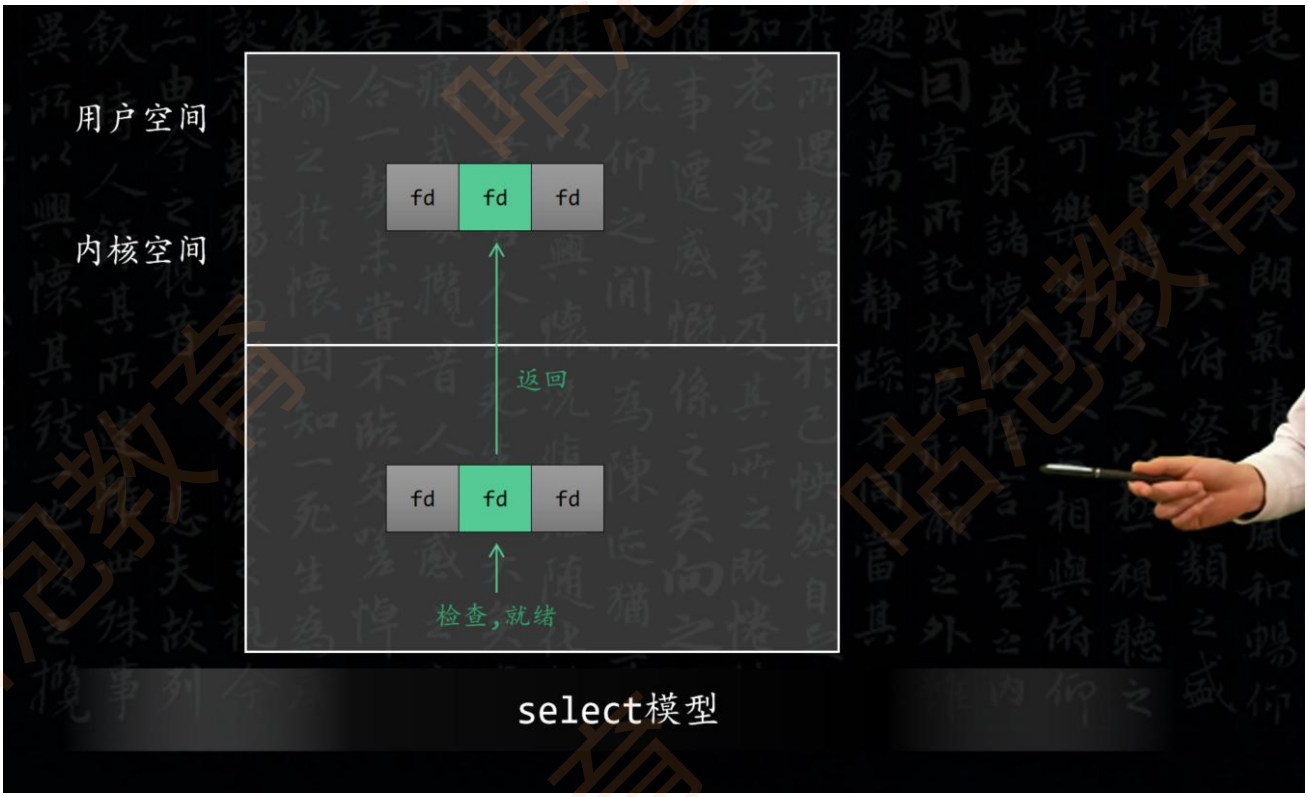
【导航条：Select模型】

（转场，屏幕黑底白字，正中央出现“2 Select模型）

2 select模型

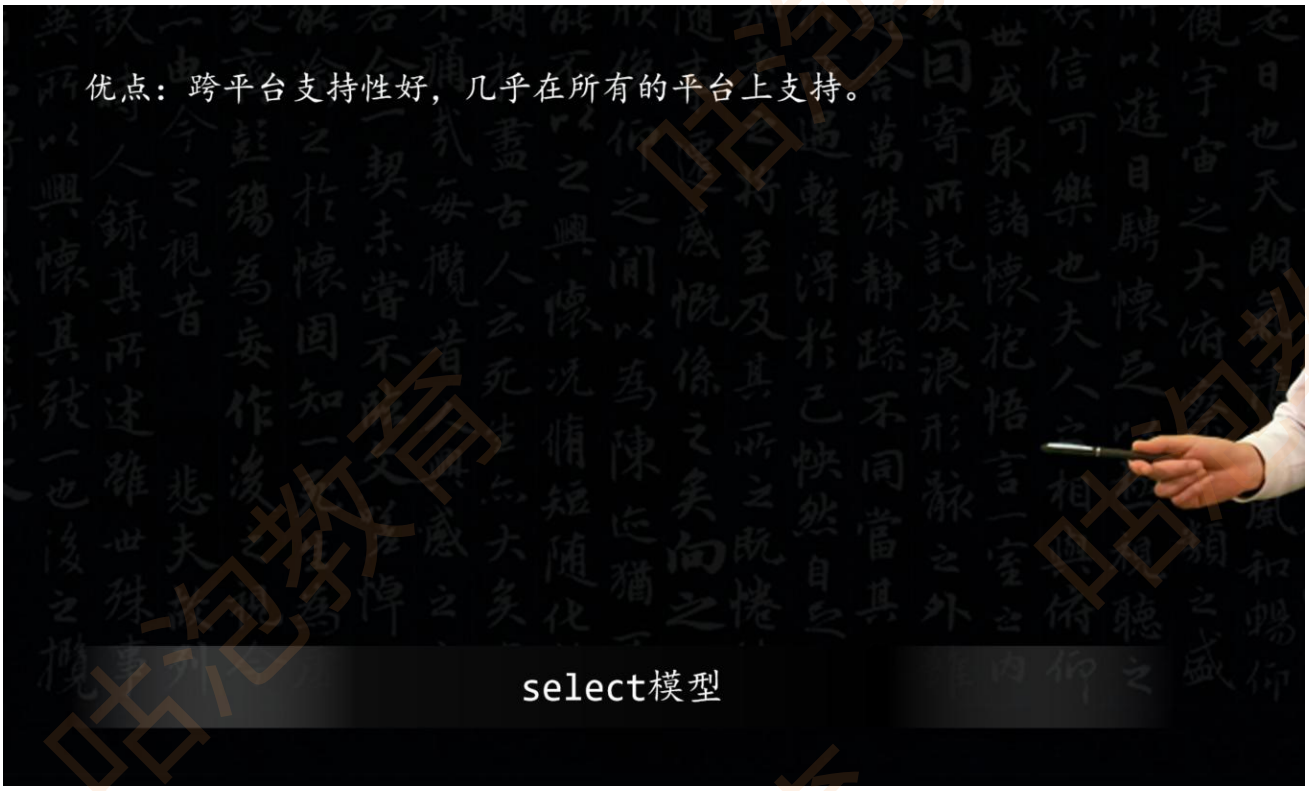
select模型，它的基本原理是，采用轮询和遍历的方式。也就是说，在客户端操作服务器时，会创建三种文件描述符，简称FD。分别是writefds（写描述符）、readfds（读描述符）和 exceptfds（异常描述符）。

而select会阻塞监视这三种文件描述符，等有数据、可读、可写、出异常或超时都会返回；

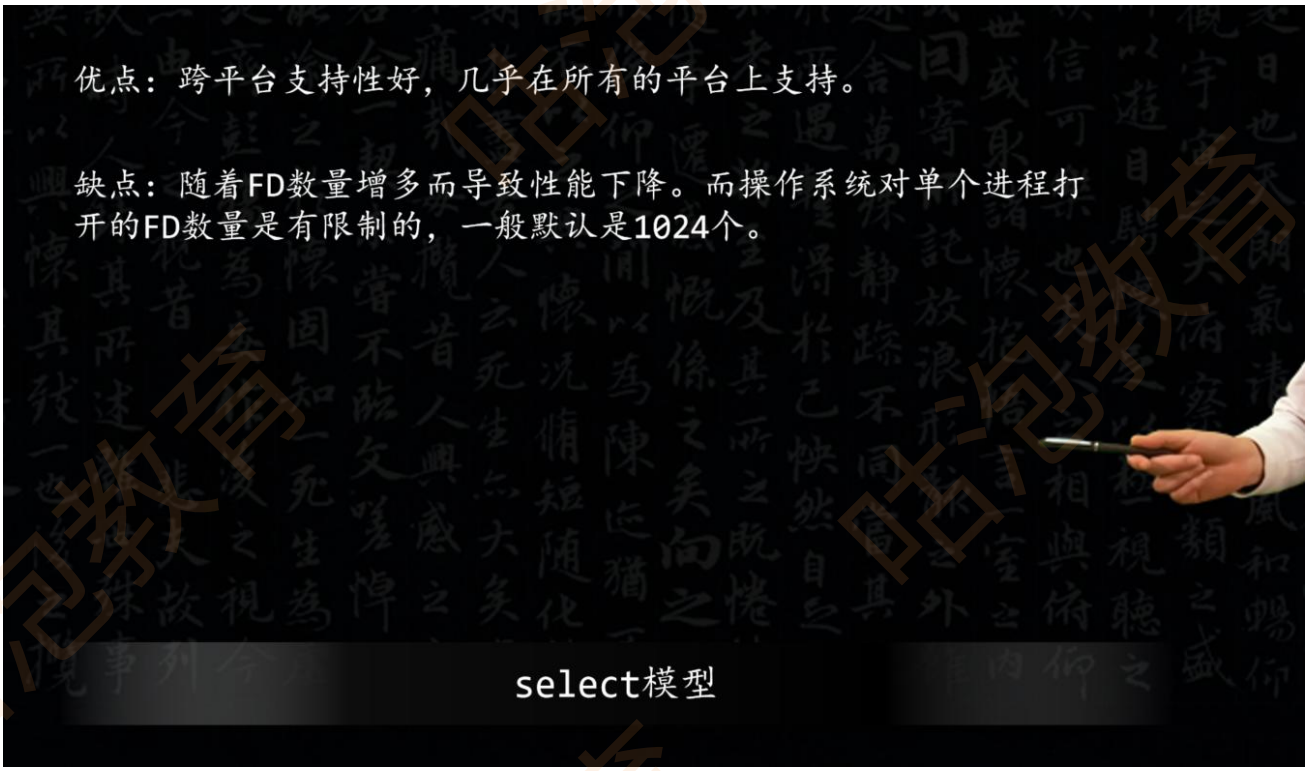


返回后通过遍历fdset，也就是文件描述符的集合，来找到就绪的FD，然后，触发相应的IO操作。

它的优点是跨平台支持性好，几乎在所有的平台上支持。



它的缺点也很明显，由于select是采用轮询的方式进行全盘扫描，因此，随着FD数量增多而导致性能下降。

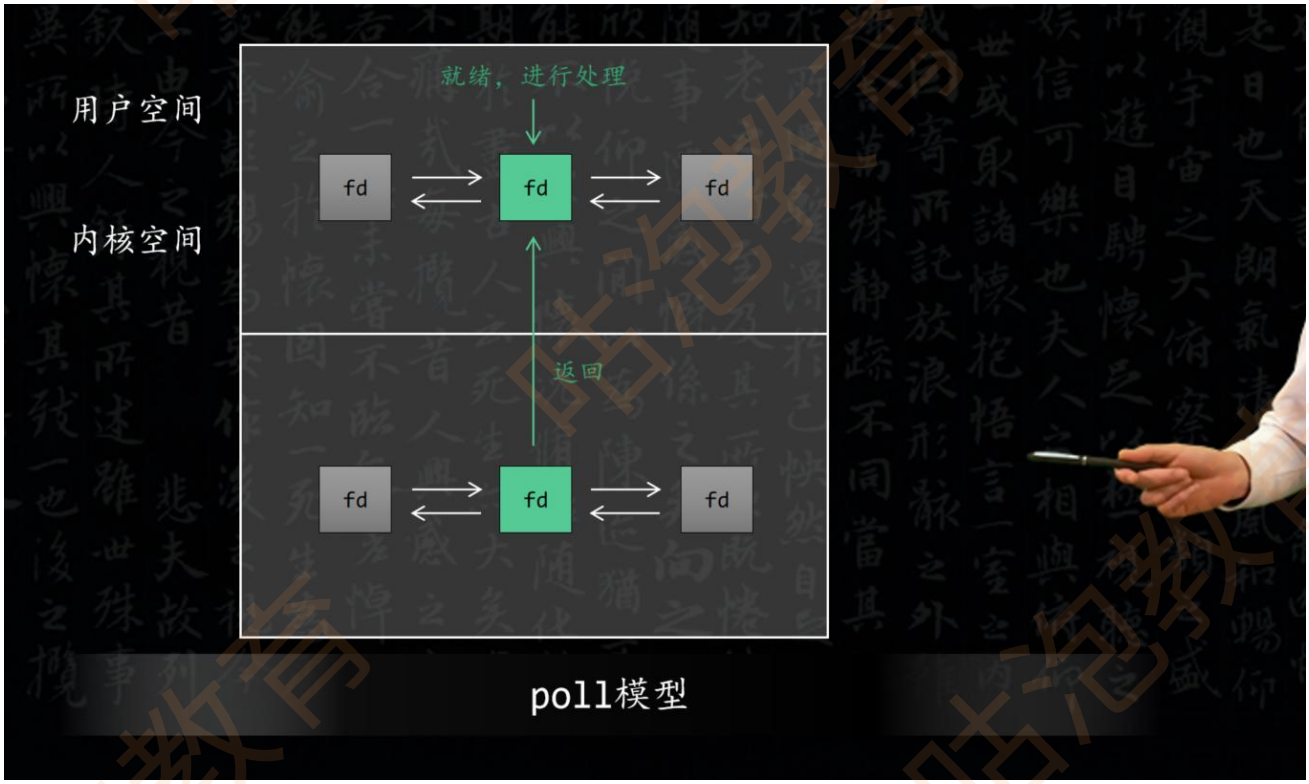


因此，每次调用select()方法，都需要把FD集合从用户态拷贝到内核态，并进行遍历。而操作系统对单个进程打开的FD数量是有限制的，一般默认是1024个。虽然，可以通过操作系统的宏定义FD_SETSIZE修改最大FD数量限制，但是，在IO吞吐量巨大的情况下，效率提升仍然有限。

【导航条：poll模型】

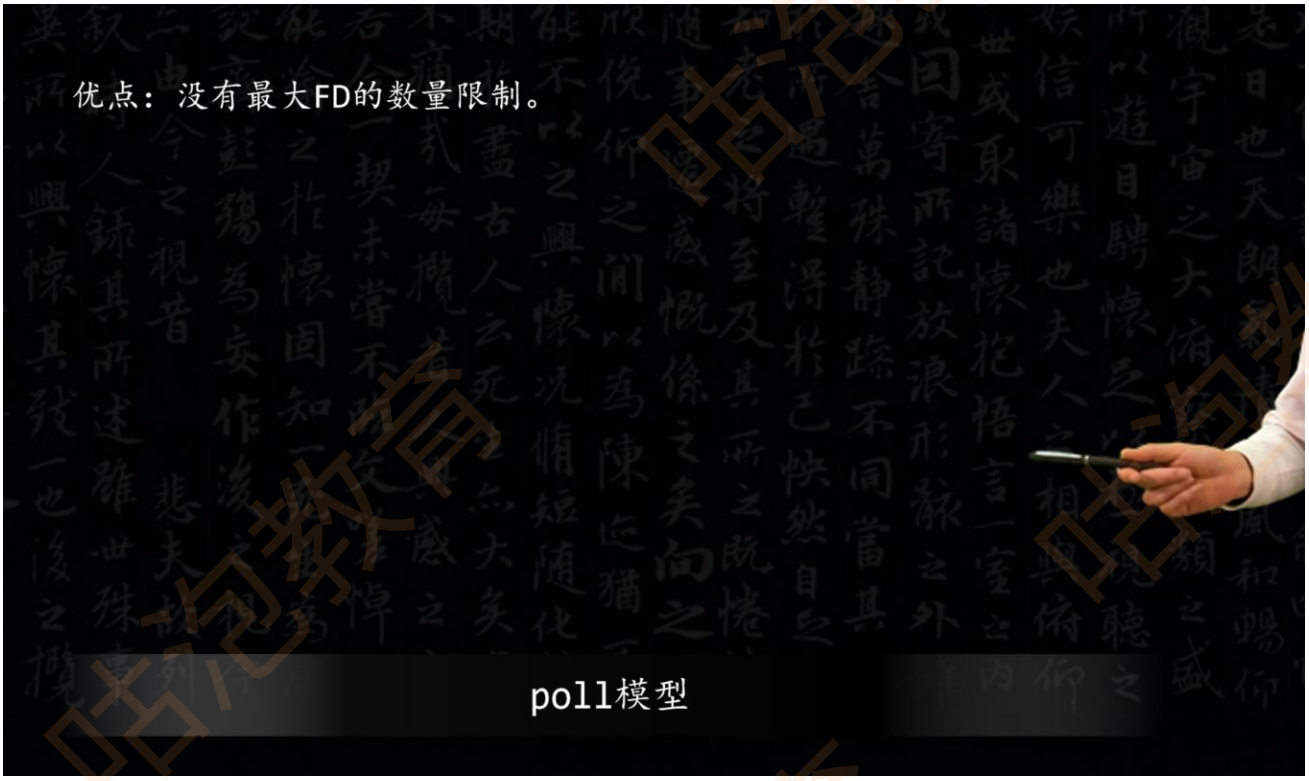
（转场，屏幕黑底白字，正中央出现“2 poll模型）

3 poll模型

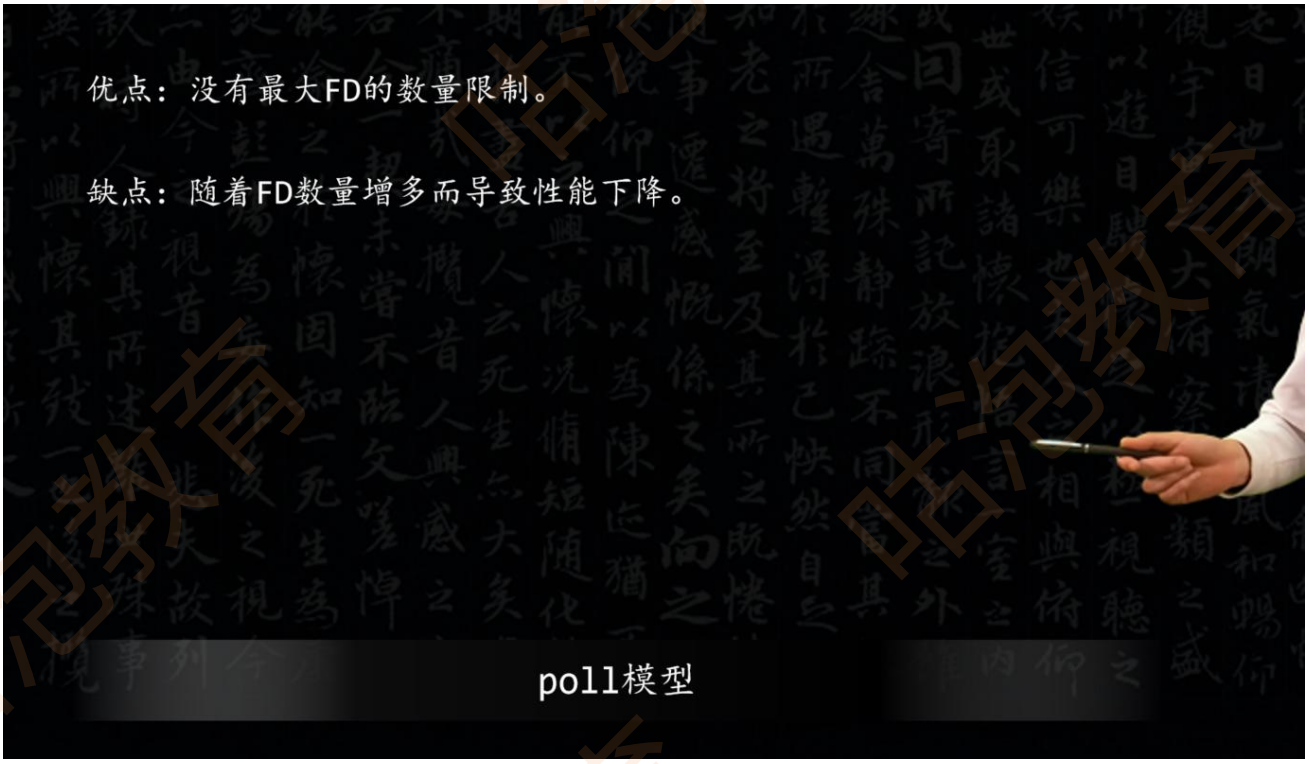


poll 模型的原理与select模型基本一致，也是采用轮询加遍历，唯一的区别就是 poll 采用链表的方式来存储FD。

所以，它的优点点是没有最大FD的数量限制。



它的缺点和select一样，也是采用轮询方式全盘扫描，同样也会随着FD数量增多而导致性能下降。



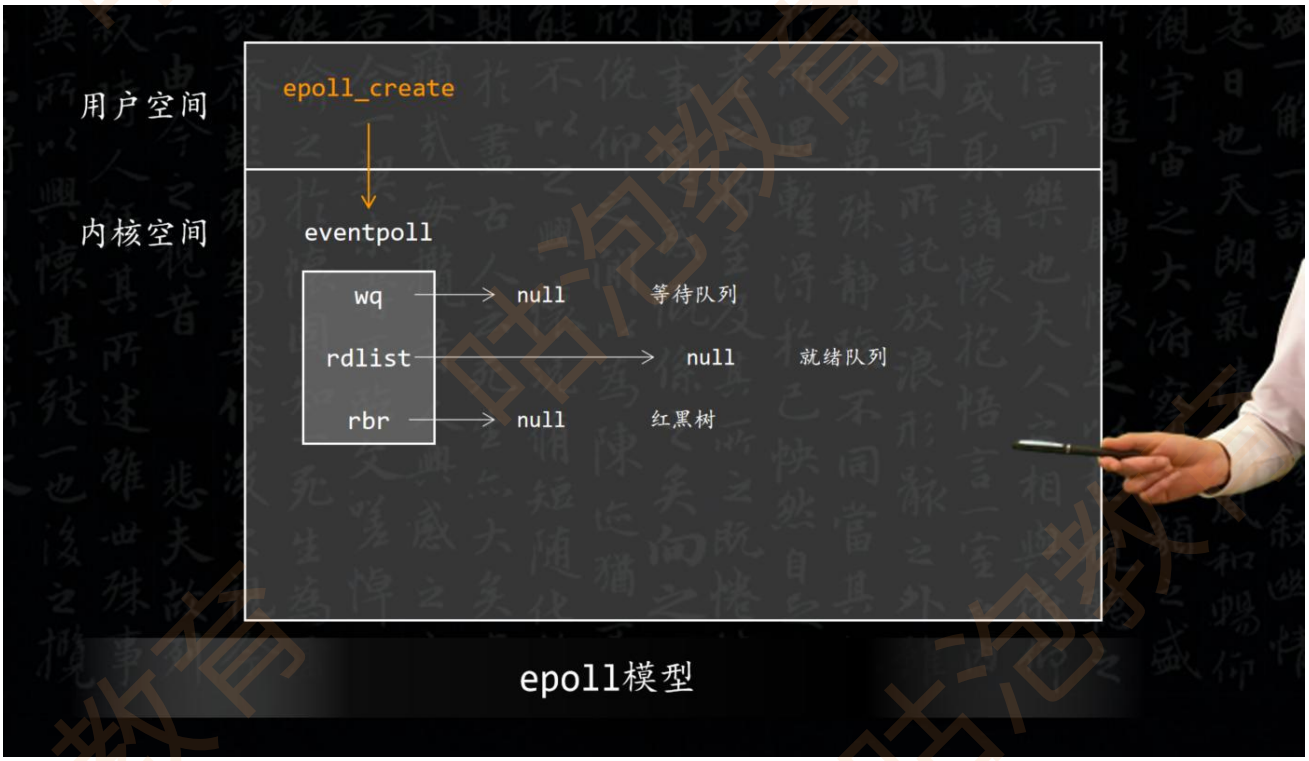
【导航条：epoll模型】

（转场，屏幕黑底白字，正中央出现“4 epoll模型）

4 epoll模型

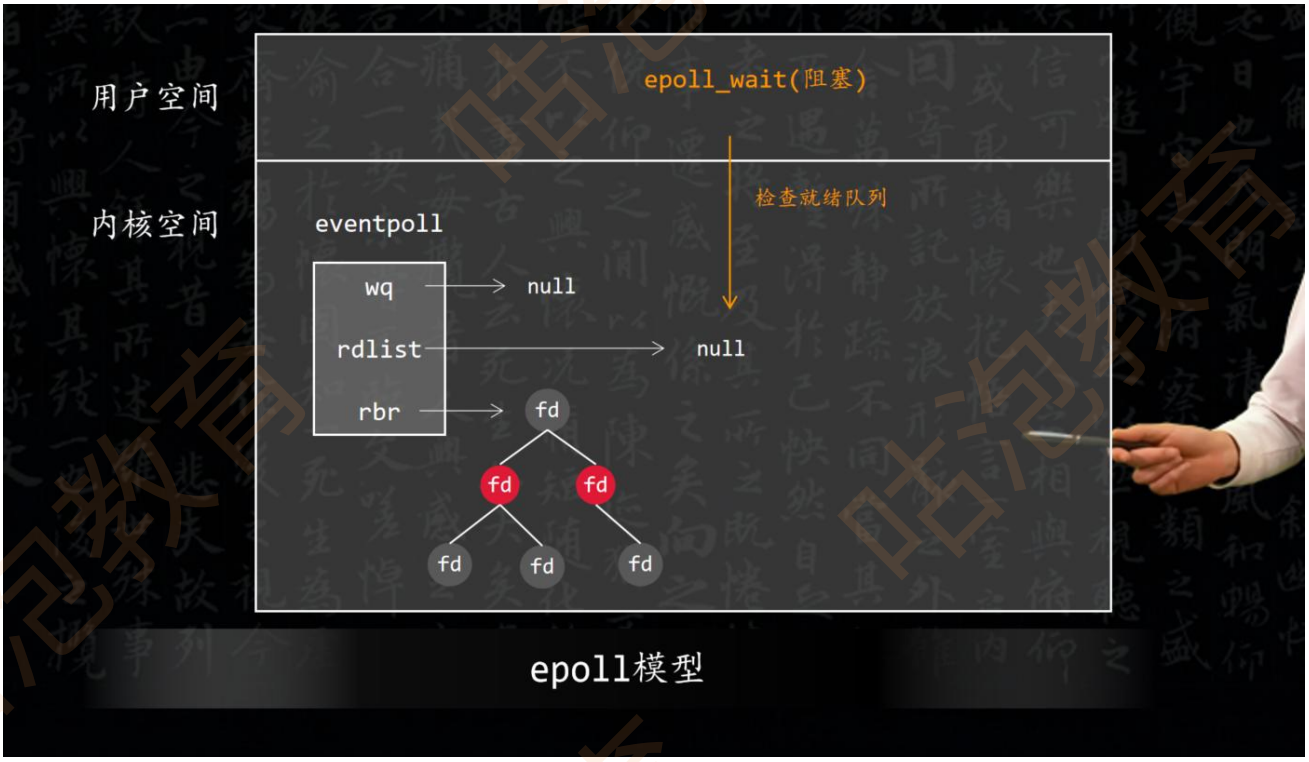
由于select和poll都会因为吞吐量增加而导致性能下降，因此，才出现了epoll模型。

epoll模型是采用时间通知机制来触发相关的IO操作。它没有FD个数限制，而且从用户态拷贝到内核态只需要一次。它主要通过系统底层的函数来注册、激活FD，从而触发相关的 IO 操作，这样大大提高了性能。主要是通过调用以下三个系统函数：

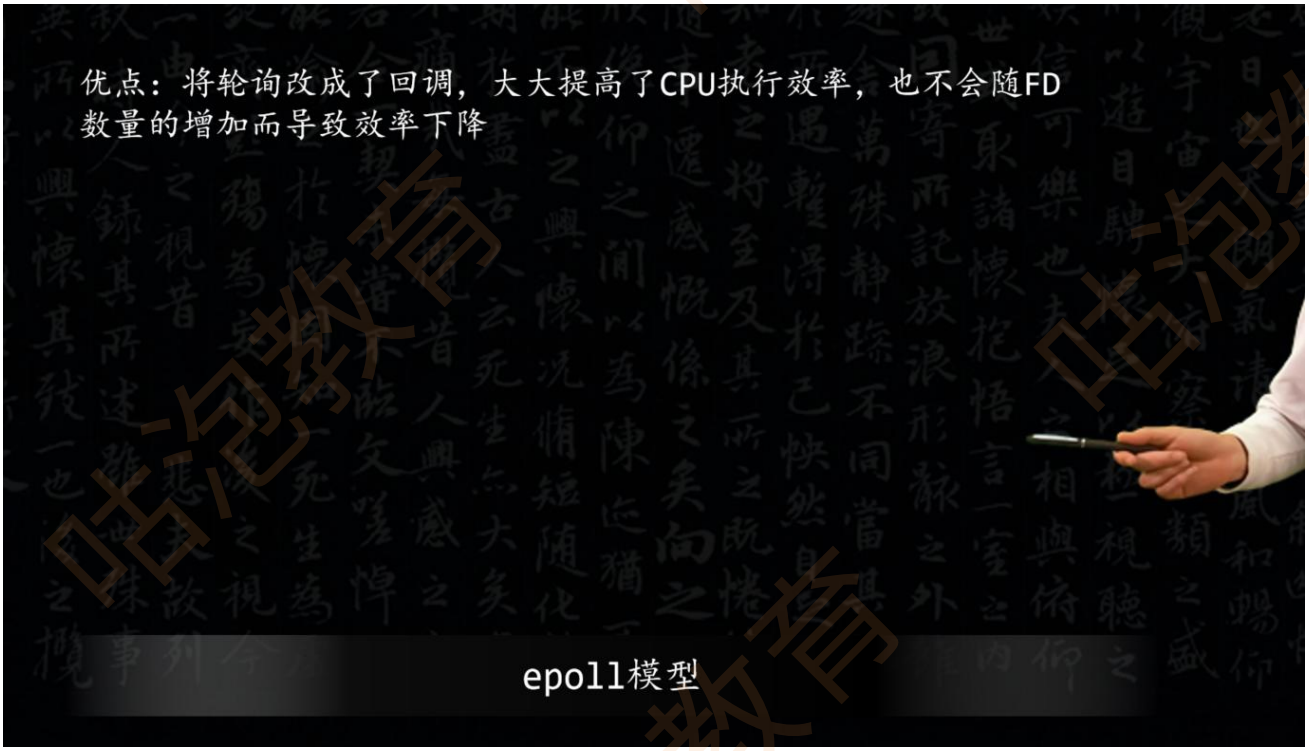


1、epoll_create()函数，在系统启动时，会在Linux内核里面申请一个B+树结构的文件系统，然后，返回epoll对象，也是一个FD。

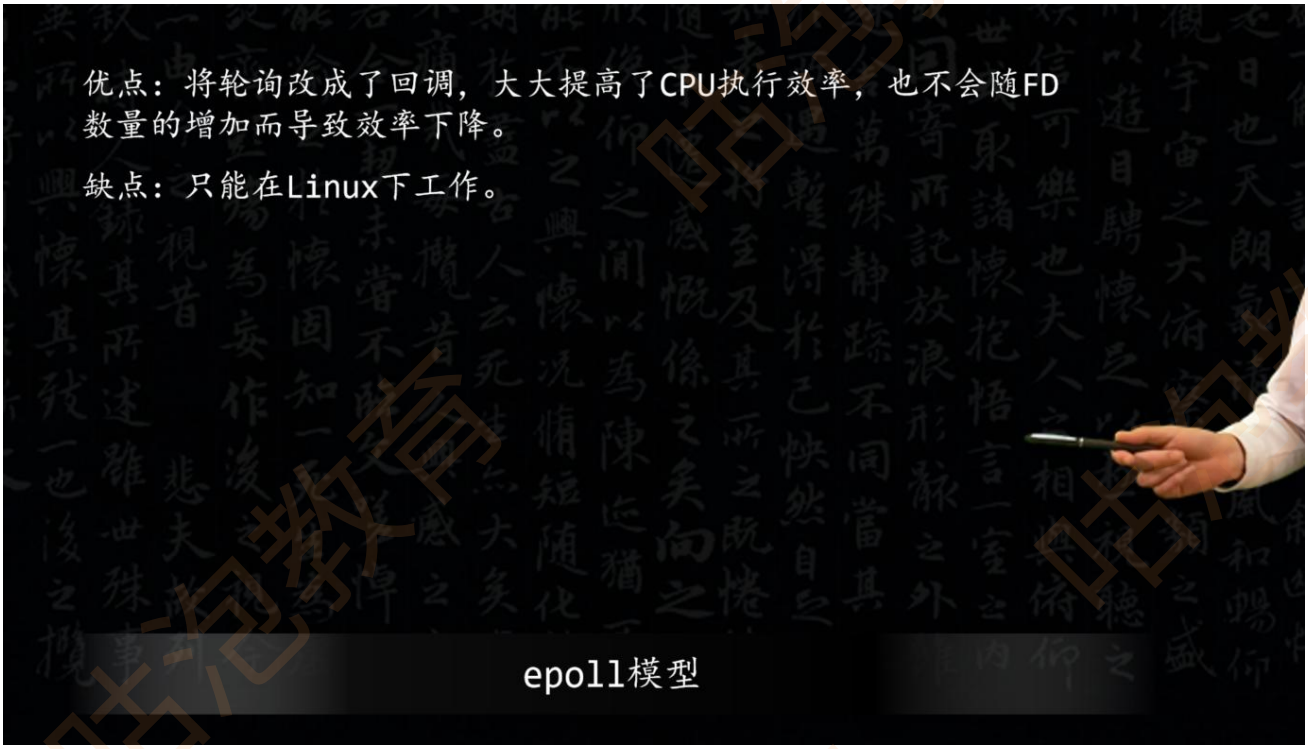
2、epoll_ctl()函数，每新建一个连接的时候，会同步更新epoll对象中的FD，并且绑定一个 callback回调函数。



3、epoll_wait()函数，轮询所有的callback集合，并触发对应的 IO 操作



所以，epoll模型最大的优点是将轮询改成了回调，大大提高了CPU执行效率，也不会随FD数量的增加而导致效率下降。当然，它也没有FD数量限制，也就是说，它能支持的FD上限是操作系统的最大文件句柄数。一般而言，1G内存大概支持 10 万个句柄。分布式系统中常用的组件如Redis、Nginx都是优先采用epoll模型。



它的缺点是只能在Linux下工作。

【导航条：综合对比】

（转场，屏幕黑底白字，正中央出现“5 综合对比”）

5 综合对比

下表是三种多路复用模型的综合对比，有兴趣的小伙伴可以截图保存一下，或者到我的个人煮叶简介中获取。

【结尾】

好了，以上就是我对IO多路复用机制的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

99.谈谈你对Nacos配置动态更新原理的理解

【导航条：引言】

Nacos作为阿里的开源中间件在Spring Cloud生态以后，不管是作为配置中心，还是作为注册中心，因为它简单易用的特性，在互联网公司被广泛运用。随后，大家会发现Nacos相关的面试题也就越来越多了。

这不，又有一位工作8年的小伙伴，被问到这样一道面试题，说，请你详细介绍一下Nacos客户端是如何实现配置动态更新的。今天，我给大家分享一下我对这个问题的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：原理分析】

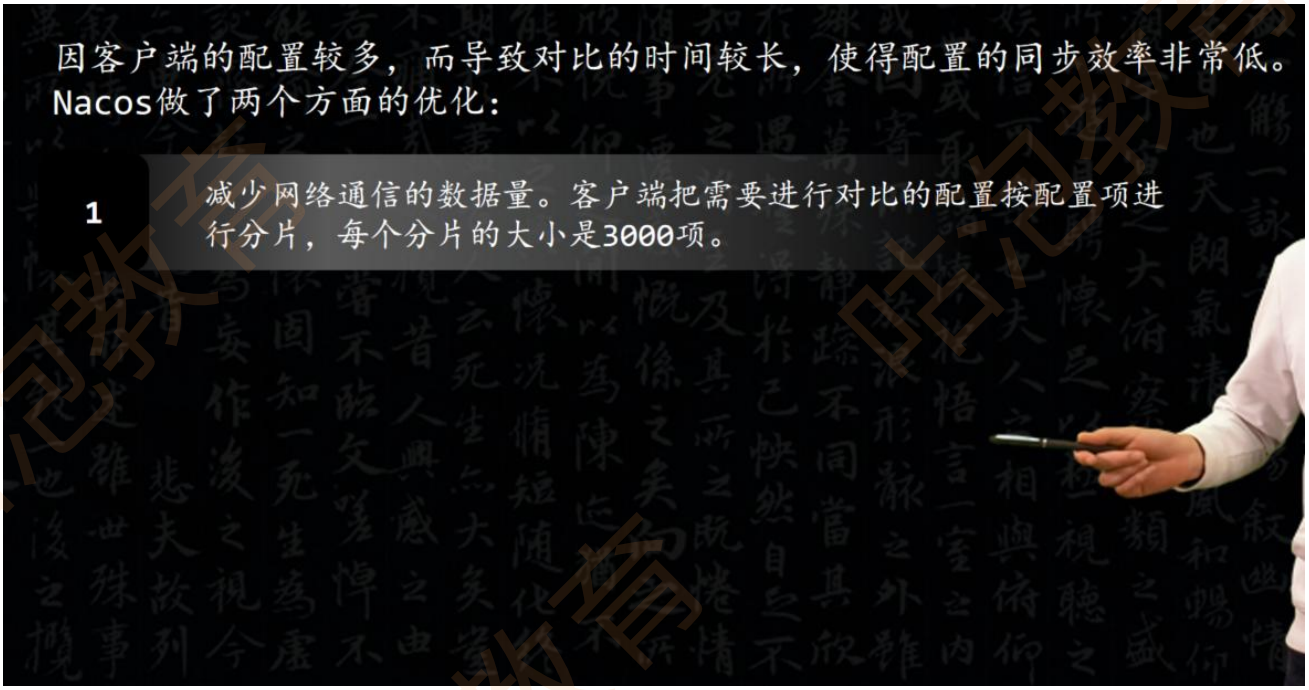
（转场，屏幕黑底白字，正中央出现“ 原理分析）

1 原理分析

下面我给大家分享一下我对Nacos配置动态更新原理的理解。

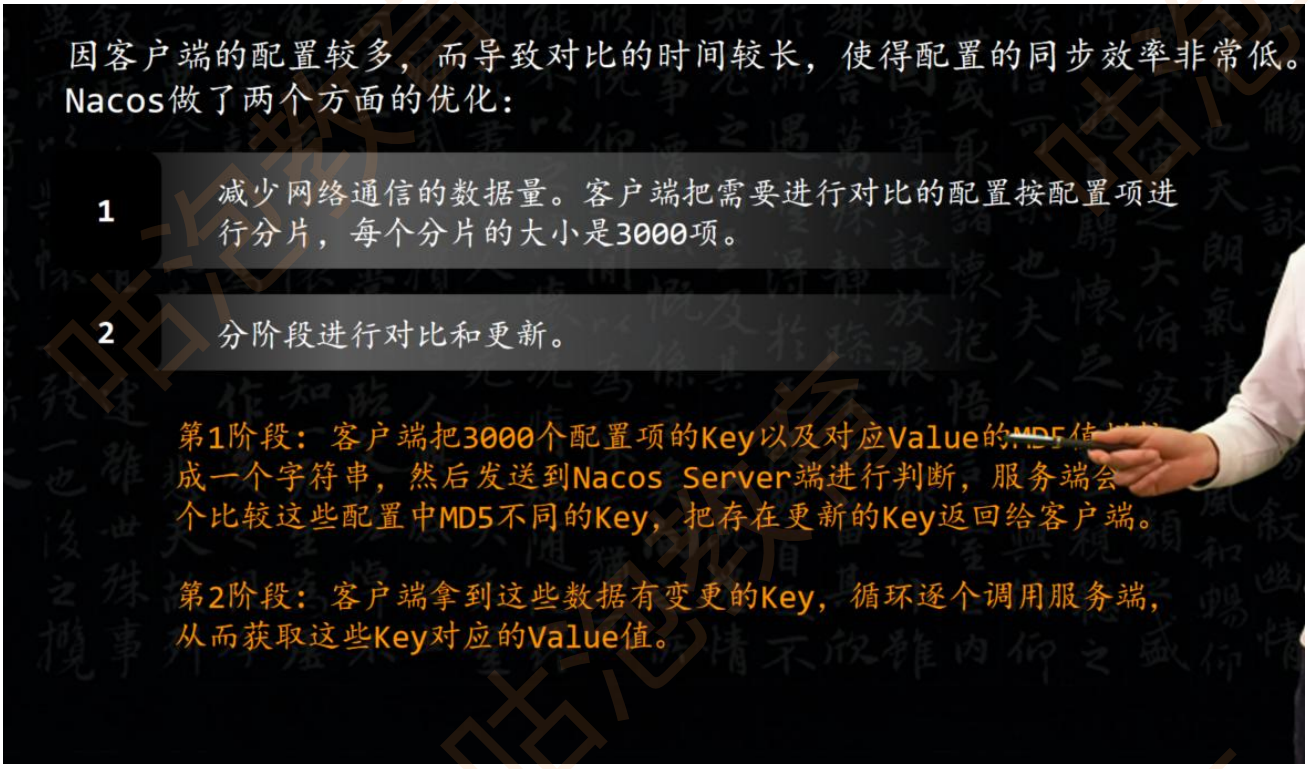
首先，Nacos采用的是长轮询的方式。也就是说，由Nacos Client向Nacos Server端去发起配置更新查询的请求。所谓长轮询，就是客户端发起一次轮询请求到服务器端，当服务器端的配置没有任何变更的时候，这个连接会一直打开，直到服务端有配置变更或者连接超时之后才返回。

Nacos Client端需要去获取服务端变更的配置内容，但前提是需要先进行比较。也就是说将客户端本地缓存的配置信息和服务器端获取的配置信息进行比较。一旦发现本地缓存的配置内容和服务端的配置内容有差异，那么，就表示服务器端的配置有更新。于是，需要把更新的配置拉到本地，在这个过程中，有可能因为客户端的配置比较多，而导致对比的时间较长，使得配置的同步效率非常低。



于是Nacos针对这样一个场景，做了两个方面的优化：

1、减少网络通信的数据量。客户端把需要进行对比的配置按配置项进行分片，每个分片的大小是3000项，也就是说每一次最多拿3000个配置项去Nacos Server端进行对比。



2、分阶段进行对比和更新。第一阶段，客户端把3000个配置项的Key以及对应Value的MD5值拼接成一个字符串，然后发送到Nacos Server端进行判断，服务端会逐个比较这些配置中MD5不同的Key，把存在更新的Key返回给客户端。第二阶段，客户端拿到这些数据有变更的Key，循环逐个调用服务端，从而获取这些Key对应的Value值。

那么，这两个优化的核心目的是去减少网络通信中数据包的大小。把一次大数据包的通信拆分成了多个小数据包的通信。虽然会增加网络通信的次数，但是，提高了整体的数据传输的性能。

最后，再加上长轮询的方式，既减少了Pull的轮询次数，又利用了长轮询的优势，很好地实现了配置动态更新的同步功能。

【结尾】

好了，以上就是我对这个问题的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

100.TCP协议为什么需要三次握手?

【导航条：引言】

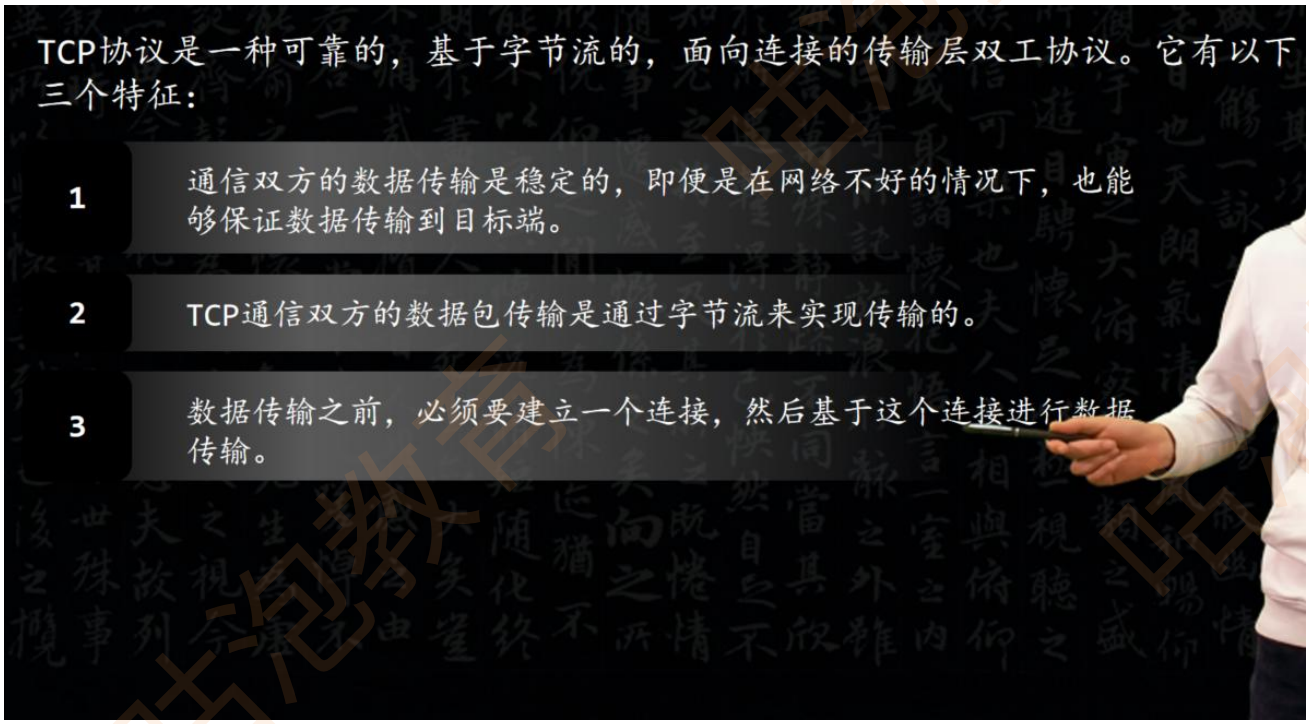
一位5年工作经验的小伙伴面试被问到这样一道面试题，说，TCP协议为什么要设计三次握手。当时这位小伙伴被问得哑口无言。后来，他找到我，说希望做一期视频分享一下。今天，我给大家分享一下我的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：TCP协议】

（转场，屏幕黑底白字，正中央出现“1 TCP协议）

1 TCP协议



TCP协议是一种可靠的，基于字节流的，面向连接的传输层双工协议。它有以下三个特征：

1、通信双方的数据传输是稳定的，即便是在网络不好的情况下，也能够保证数据传输到目标端。

2、TCP通信双方的数据包传输是通过字节流来实现传输的

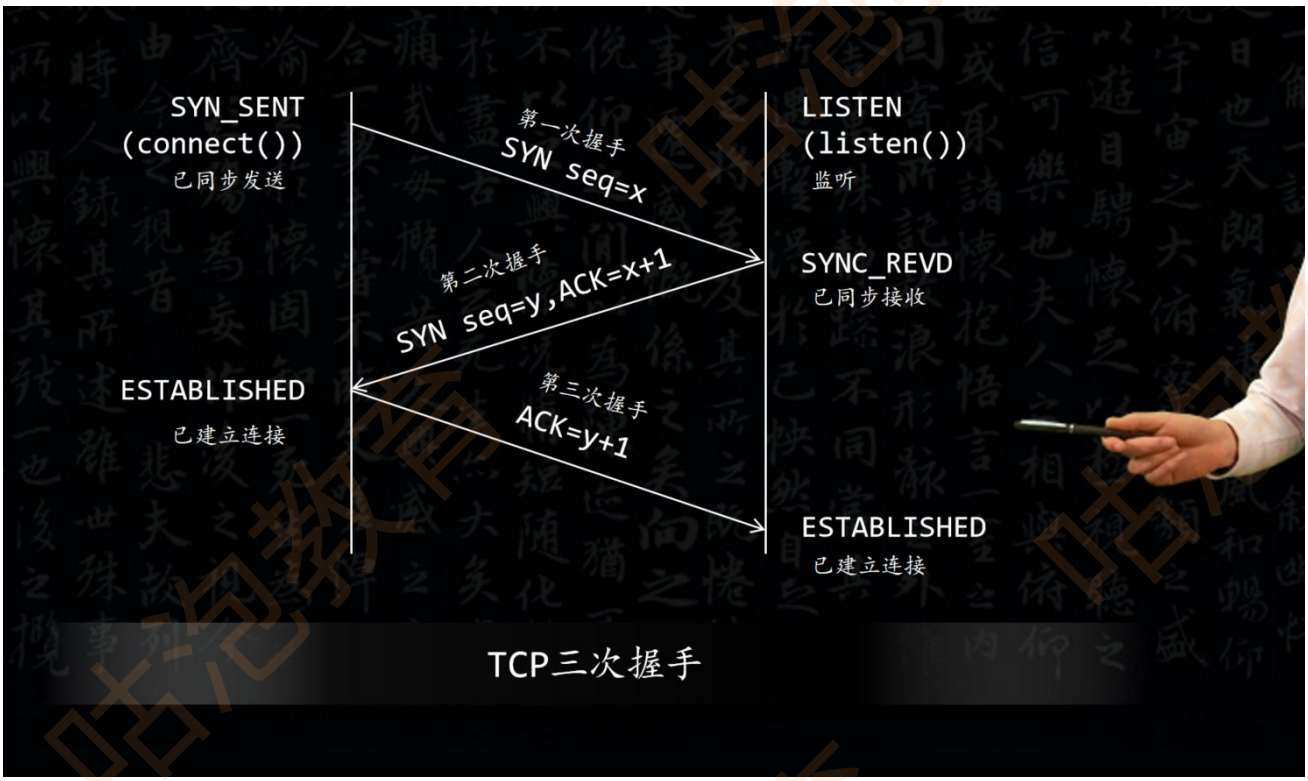
3、数据传输之前，必须要建立一个连接，然后基于这个连接进行数据传输

【导航条：什么是三次握手】

（转场，屏幕黑底白字，正中央出现“2 什么是三次握手）

2 什么是三次握手

因为，TCP是面向连接的协议，在进行数据通信之前需要建立一个可靠的连接。所以，TCP协议采用了三次握手的方式来实现连接的建立。所谓的三次握手，就是通信双方一共需要发送三次请求，才能确保这个连接的建立，具体的执行流程如图所示：



1）客户端向服务端发送连接请求并携带同步序列号SYN。

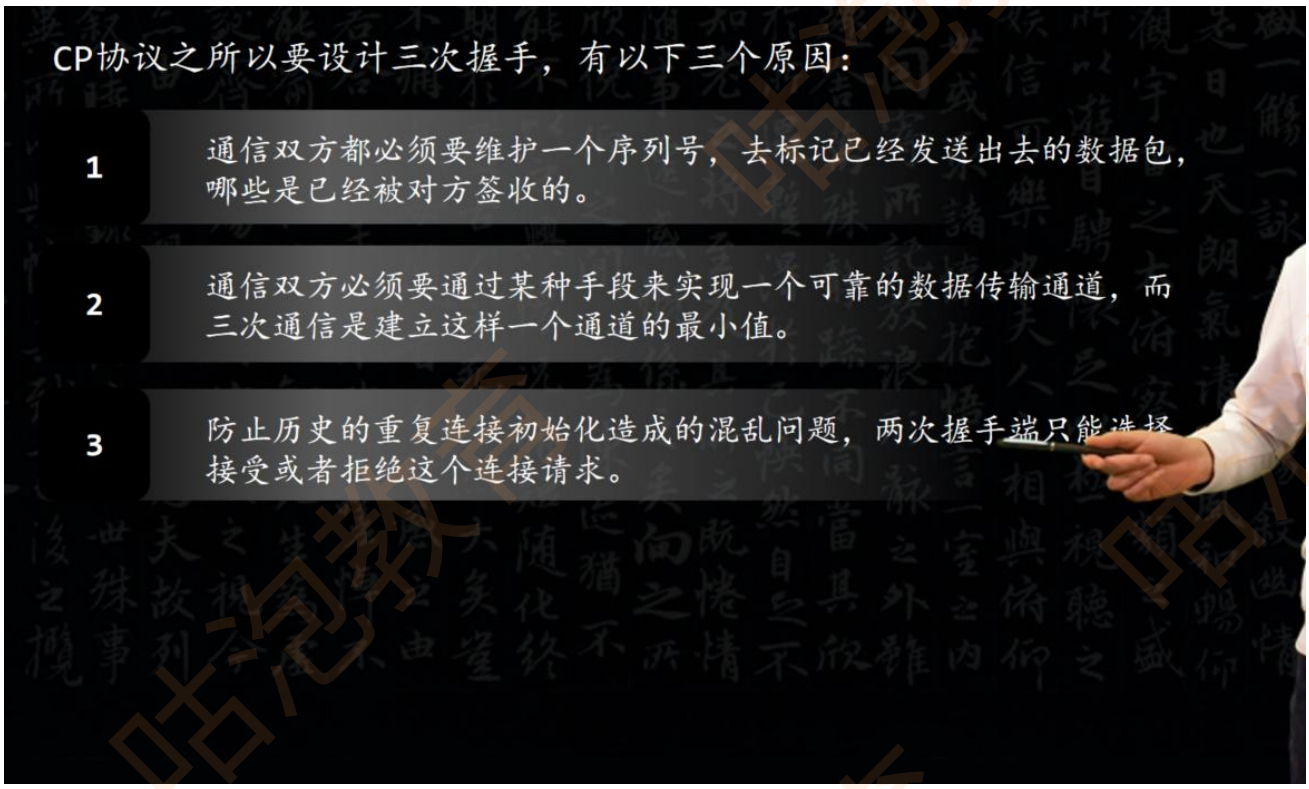
2）服务端收到请求后，发送SYN和ACK， 这里的SYN表示服务端的同步序列号，ACK表示对前面收到请求的一个确认，表示告诉客户端，请求已收到。

3）客户端收到服务端的请求后，再次发送ACK，这个ACK是针对服务端连接的一个确认，表示告诉服务端，请求已收到。

【导航条：为什么要三次握手】

（转场，屏幕黑底白字，正中央出现“2 为什么要三次握手）

2 为什么要三次握手



协议之所以要设计三次握手，我认为有以下三个原因：

1、TCP是可靠性通信协议，所以通信双方都必须要维护一个序列号，去标记已经发送出去的数据包，哪些是已经被对方签收的。而三次握手就是通信双方相互告知序列号的起始值，为了确保这个序列号被收到，所以双方都需要有一个确认的操作。

2、TCP协议需要在一个不可靠的网络环境下实现可靠的数据传输，意味着通信双方必须要通过某种手段来实现一个可靠的数据传输通道，而三次通信是建立这样一个通道的最小值。当然还可以四次、五次，只是没必要浪费这个资源。

3、防止历史的重复连接初始化造成的混乱问题，比如说在网络比较差的情况下，客户端连续多次发送建立连接的请求，假设只有两次握手，那么服务端只能选择接受或者拒绝这个连接请求，但是服务端不知道这次请求是不是之前因为网络堵塞而过期的请求，也就是说服务端不知道当前客户端的连接是有效还是无效。

【结尾】

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

好了，以上就是我对TCP为什么需要三次握手的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

101.Spring中的事务传播行为有哪些?

【导航条：引言】

1位工作2年的小伙伴面试的时候被问到这样一个问题，说，Spring中的事务传播行为有哪些？他说他在面试的时候能想起来一些，但在实际项目开发中又基本不需要配置。所以，在面试的时候回答不全，最后被拒了，觉得有些遗憾。

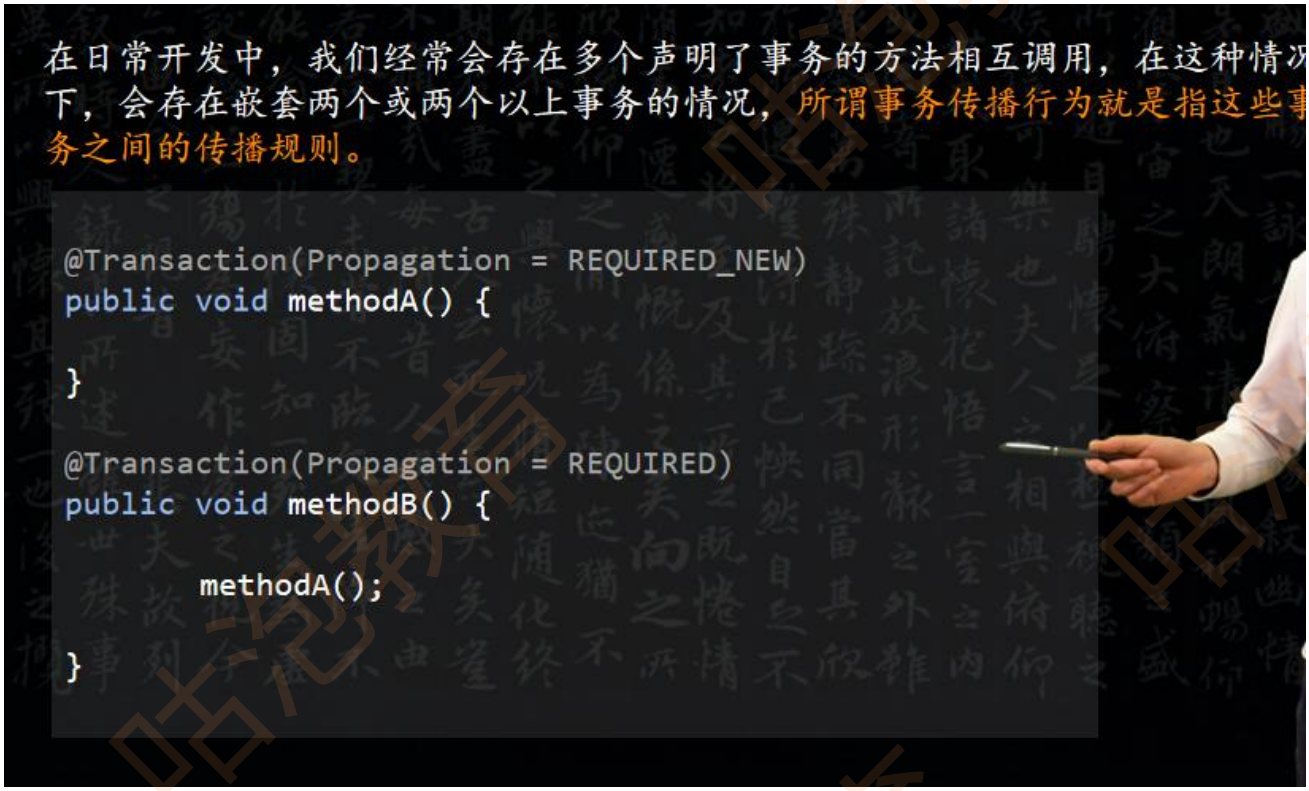
今天，我给大家分享一下，我对Spring传播行为的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的面试通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：事务传播行为】

（转场，屏幕黑底白字，正中央出现“1 事务传播行为）

1、事务传播行为



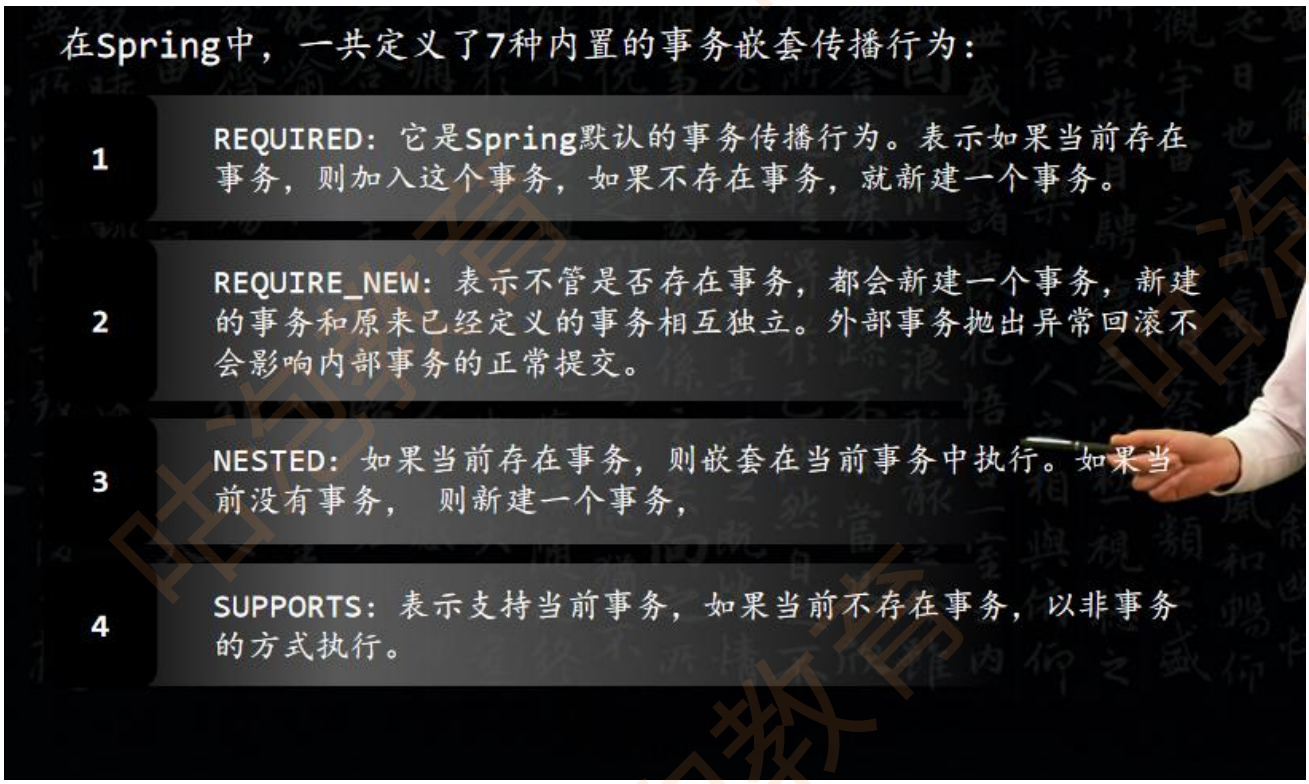
在日常开发中，我们经常会存在多个声明了事务的方法相互调用，在这种情况下，会存在嵌套两个或两个以上事务的情况，所谓事务传播行为就是指这些事务之间的传播规则。来看这样一段代码：

```
@Transaction(Propagation = REQUIRED_NEW)
public void methodA(){
}
@Transaction(Propagation = REQUIRED)
public void methodB(){
    methodA();
}
```

在这段代码中，methodB()调用了methodA()，两个方法都显示的开启了事务。methodA()是开启一个新

事务，还是继续在methodB()这个事务中执行？就取决于事务的传播行为规则的定义。

2、事务嵌套



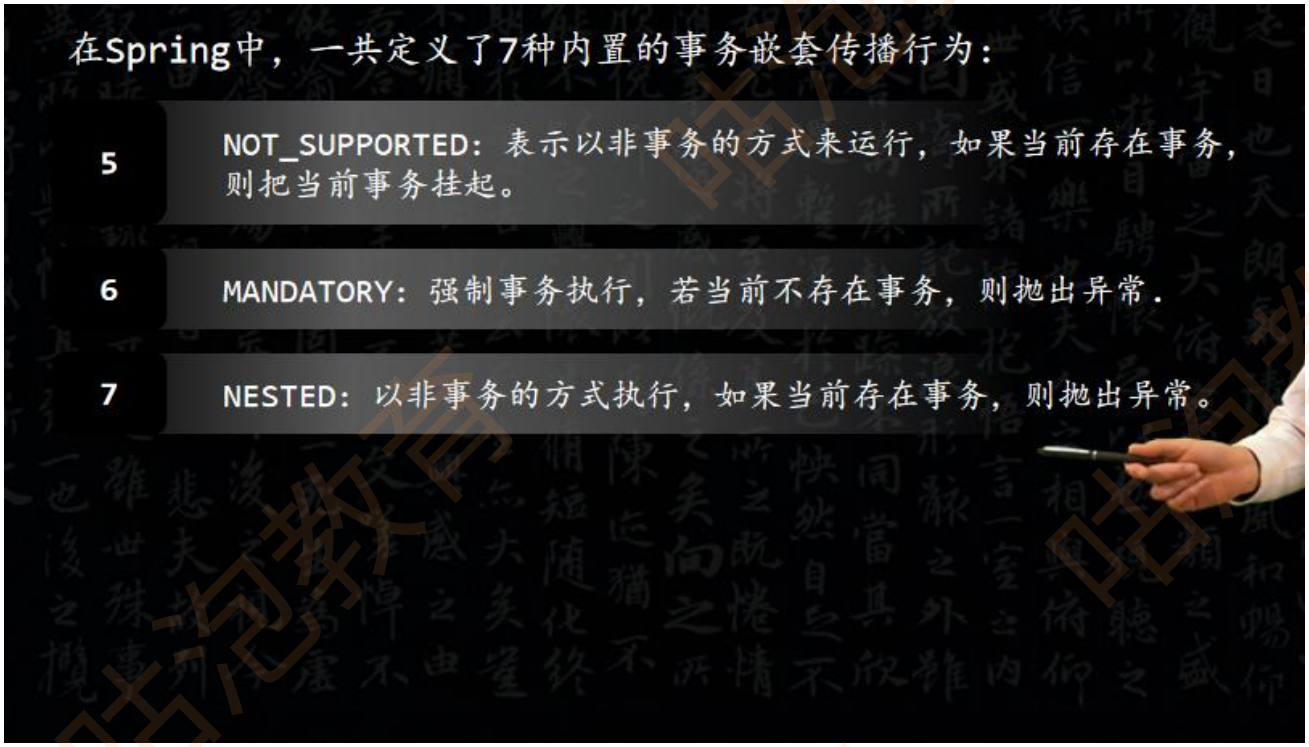
事务嵌套是指两个或两个以上开启的事务的方法嵌套调用，在这种情况下，需要制定这些事务之间的传播行为规则。在Spring中，一共定义了7种内置的事务嵌套传播行为：

第1种：REQUIRED，它是Spring默认的事务传播行为。表示如果当前存在事务，则加入这个事务，如果不存在事务，就新建一个事务。

第2种：REQUIRE_NEW：表示不管是否存在事务，都会新建一个事务，新建的事务和原来已经定义的事务相互独立。外部事务抛出异常回滚不会影响内部事务的正常提交。

第3种：NESTED：如果当前存在事务，则嵌套在当前事务中执行。如果当前没有事务， 则新建一个事务，类似于REQUIRE_NEW。

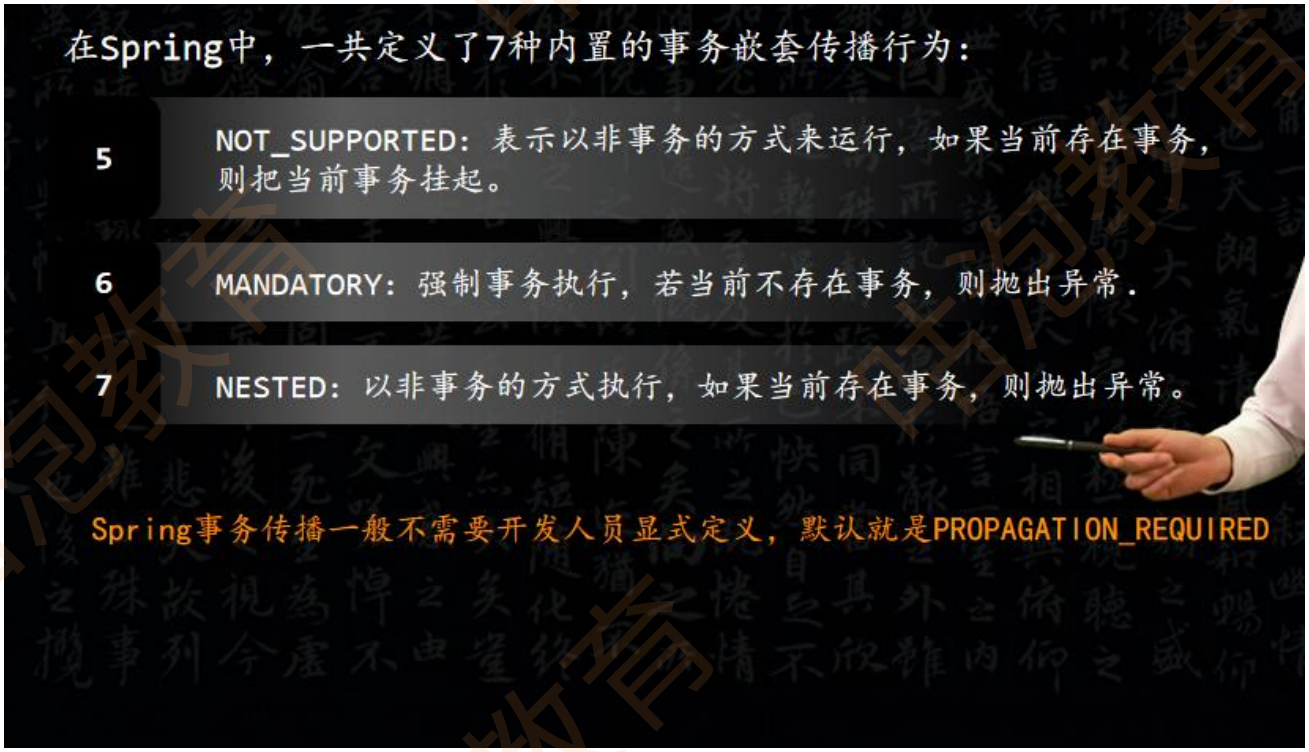
第4种：SUPPORTS：表示支持当前事务，如果当前不存在事务，以非事务的方式执行。



第5种：NOT_SUPPORTED：表示以非事务的方式来运行，如果当前存在事务，则把当前事务挂起。

第6种：MANDATORY：强制事务执行，若当前不存在事务，则抛出异常.

第7种：NEVER：以非事务的方式执行，如果当前存在事务，则抛出异常。



Spring事务传播一般不需要开发人员显式定义，默认就是PROPAGATION_REQUIRED，只要在嵌套事务的情况下需要重点了解。

【结尾】

以上就是我对这个问题的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

102.String、StringBuffer和StringBuilder的区别

【导航条：引言】

可能是因为毕业季，最近有非常多的应届生来咨询我面试相关的问题。这不，今天有一位应届生找到我，说面试被问到这样一道面试题：请谈谈String、StringBuffer和StringBuilder的区别。其实呢，这是一道老生常谈的题，光背八股文还是非常简单的。

既然，大家有这方面的需求，今天我还是给大家分享一下我的理解。

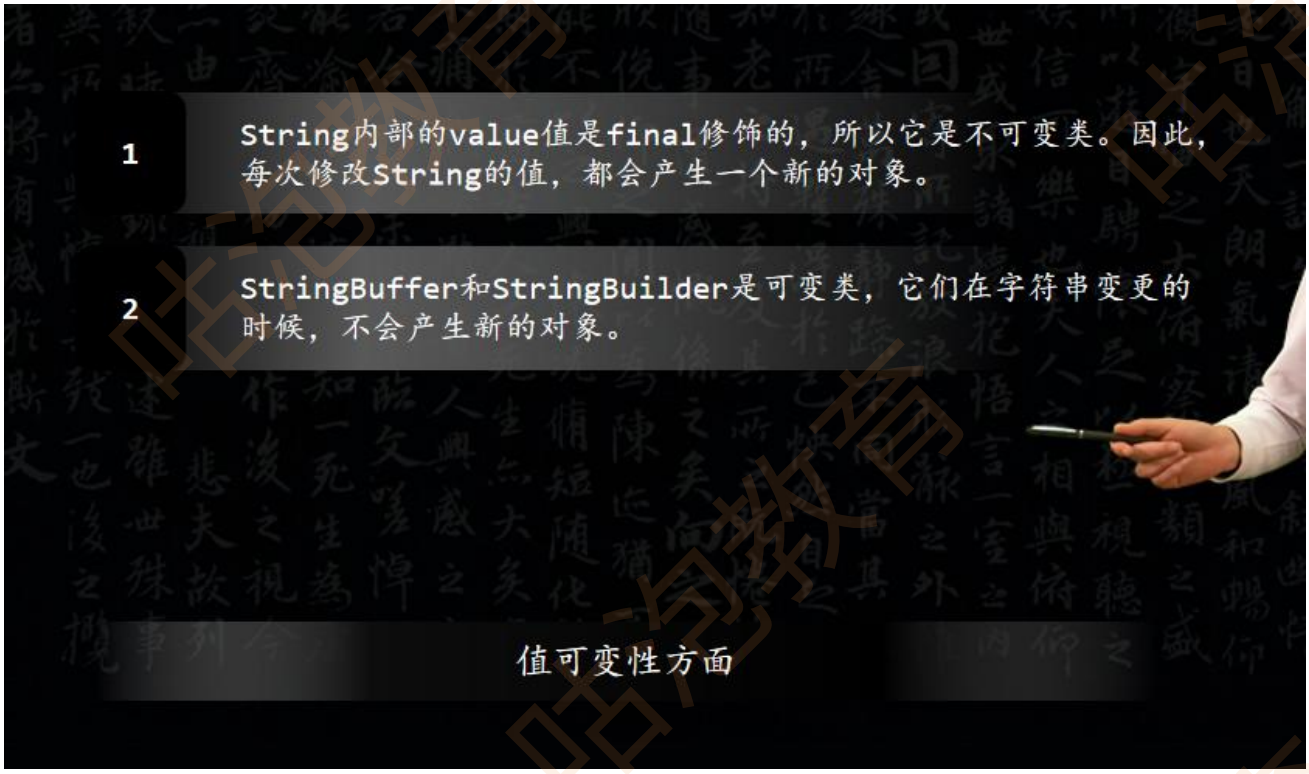
另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的面试通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

关于String、StringBuffer、StringBuilder的区别，我从以下四个方面来理解：

【导航条：值可变性】

（转场，屏幕黑底白字，正中央出现“1 值可变性方面）

1、值可变性方面



String内部的value值是final修饰的，所以它是不可变类。因此，每次修改String的值，都会产生一个新的对象。

而StringBuffer和StringBuilder是可变类，它们在字符串变更的时候，不会产生新的对象。

【导航条：线程安全】

（转场，屏幕黑底白字，正中央出现“2 线程安全方面）

2、线程安全方面

String是不可变类，所以它是线程安全的。

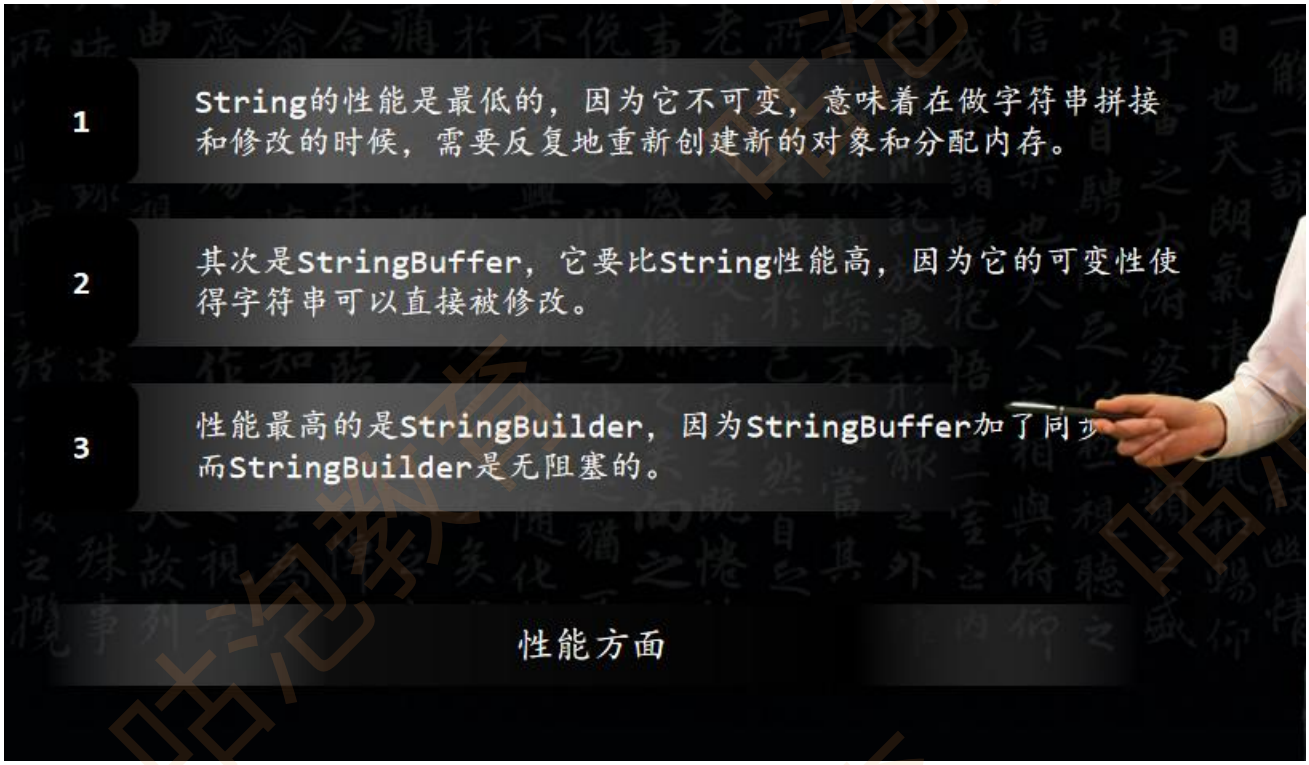
StringBuffer是线程安全的，因为它每个操作方法都加了synchronized同步关键字。

而StringBuilder不是线程安全的。如果在多线程环境下对字符串进行操作，建议使用StringBuffer，单线程情况下，才建议使用StringBuilder。

【导航条：性能】

（转场，屏幕黑底白字，正中央出现“3 性能方面）

3、性能方面



String的性能是最低的，因为它不可变，意味着在做字符串拼接和修改的时候，需要反复地重新创建新的对象和分配内存。

其次是StringBuffer，它要比String性能高，因为它的可变性使得字符串可以直接被修改。

性能最高的是StringBuilder，因为StringBuffer加了同步锁，而StringBuilder是无阻塞的。

【导航条：数据存储】

（转场，屏幕黑底白字，正中央出现“4 数据存储方面）

4、数据存储方面

String存储在字符串常量池中，而StringBuffer和StringBuilder存储在堆内存空间。

最后再补充一下， StringBuilder和StringBuffer都是派生自AbstractStringBuilder这个抽象类。

【结尾】

以上就是我对String、StringBuffer和StringBuilder的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

103.谈谈你对Java线程5种状态流转原理的理解

【导航条：引言】

今天，有位工作5年的小伙伴被问到这样一道面试题，说谈谈你对Java线程5种状态流转原理的理解。当时，平时只关注过线程如何定义和使用，对于线程状态流转脑海一片空白，完全懵了。于是找到我，希望我拍一期视频。

今天，我给大家分享一下我的理解。

另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的面试通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：线程定义】

（转场，屏幕黑底白字，正中央出现“4 线程定义）

1、线程定义

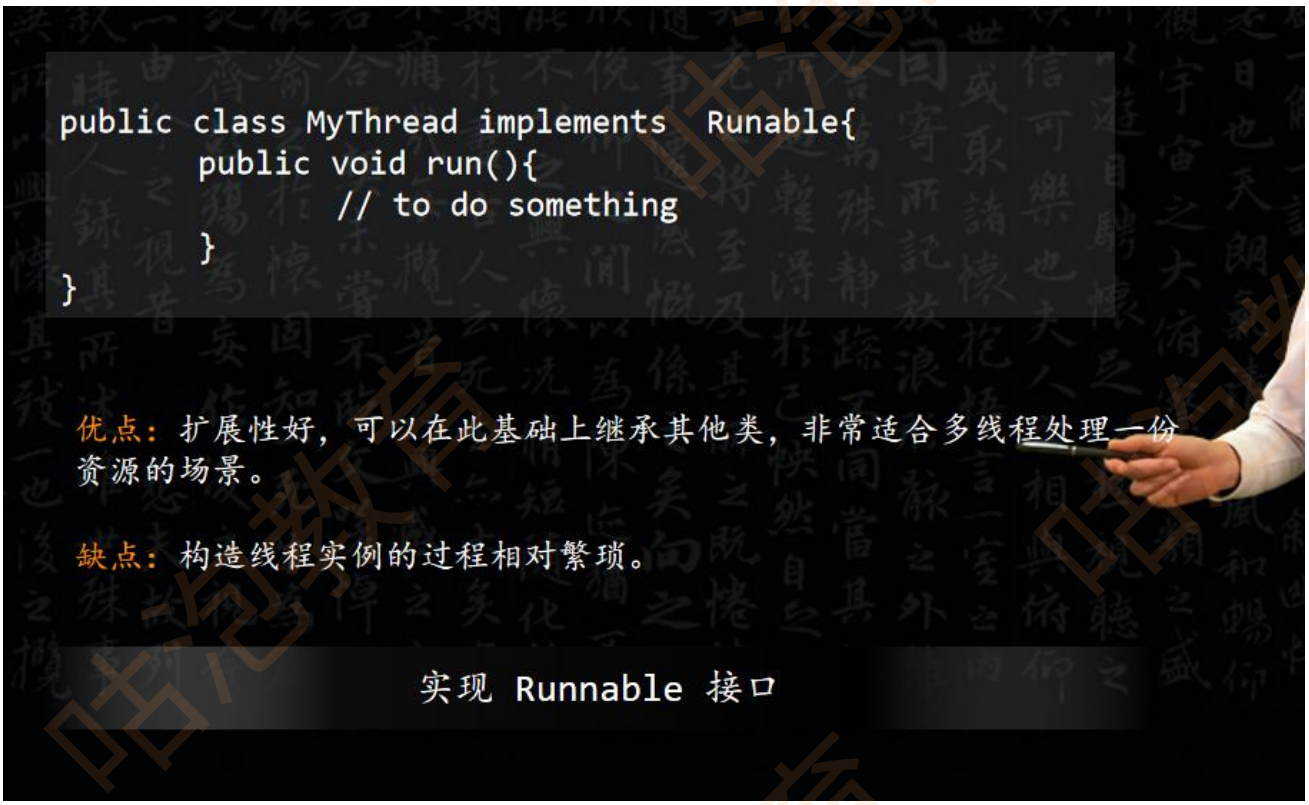
回答线程流转原理这个问题之前，我们先来回忆一下JDK中3种自定义线程的方式以及它们的优缺点。

第1种：是继承Thread类。如代码所示：

```
public class MyThread extends  Thread{
    public void run(){
        // to do something
    }
}
```

这种方式的优点是：实现简单，只需实例化继承类的实例，即可使用线程。

它的缺点是：扩展性不足，Java是单继承的语言，如果一个类已经继承了其他类，就无法通过这种方式实现自定义线程

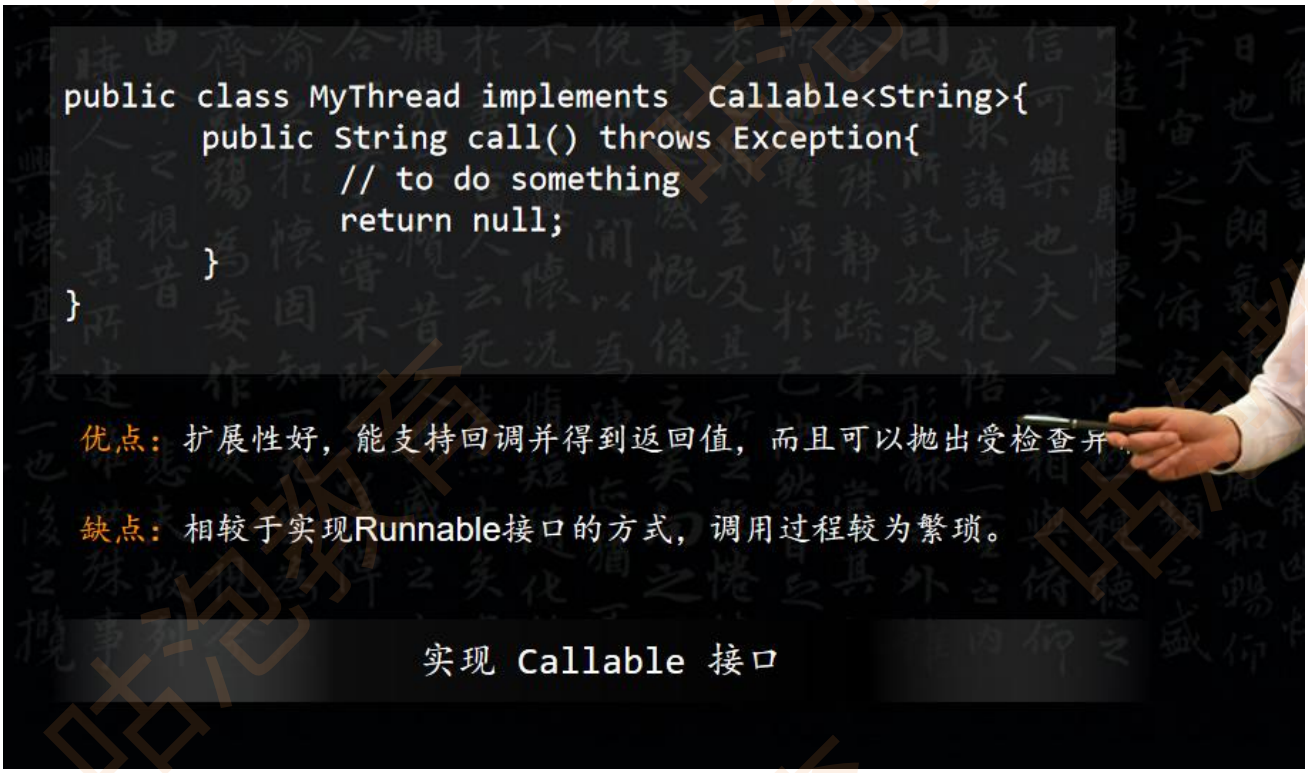


第2种：是实现 Runnable 接口，如代码所示：

```
public class MyThread implements  Runable{
    public void run(){
        // to do something
    }
}
```

它的优点是：扩展性好，可以在此基础上继承其他类，非常适合多线程处理一份资源的场景

它的缺点是：构造线程实例的过程相对繁琐一点



第3种：是实现Callable接口，如代码所示：

```
public class MyThread implements  Callable<String>{
    public String call() throws Exception{
        // to do something
        return null;
    }
}
```

它的优点是：扩展性好，能支持回调并得到返回值，而且可以抛出受检查异常。

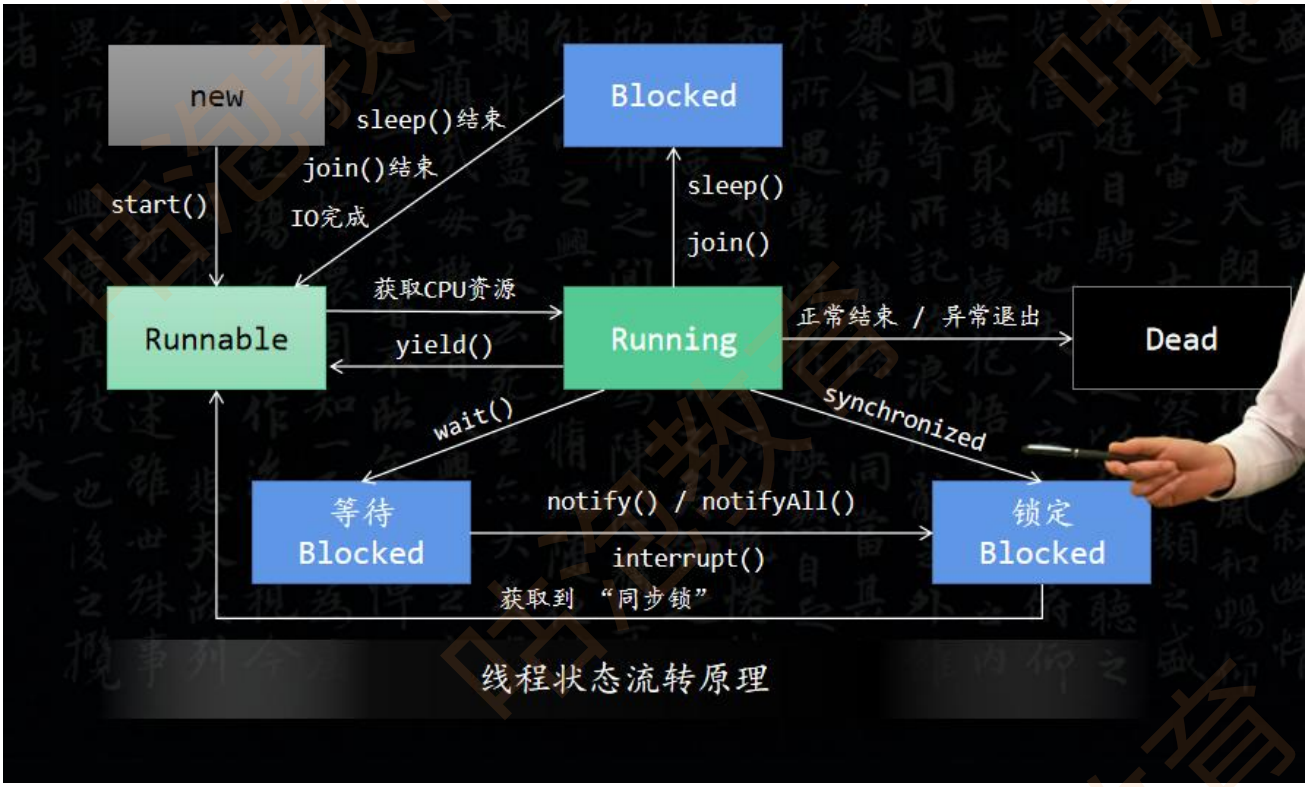
它的缺点是：相较于实现Runnable接口的方式，调用过程较为繁琐。

【导航条：状态流转原理】

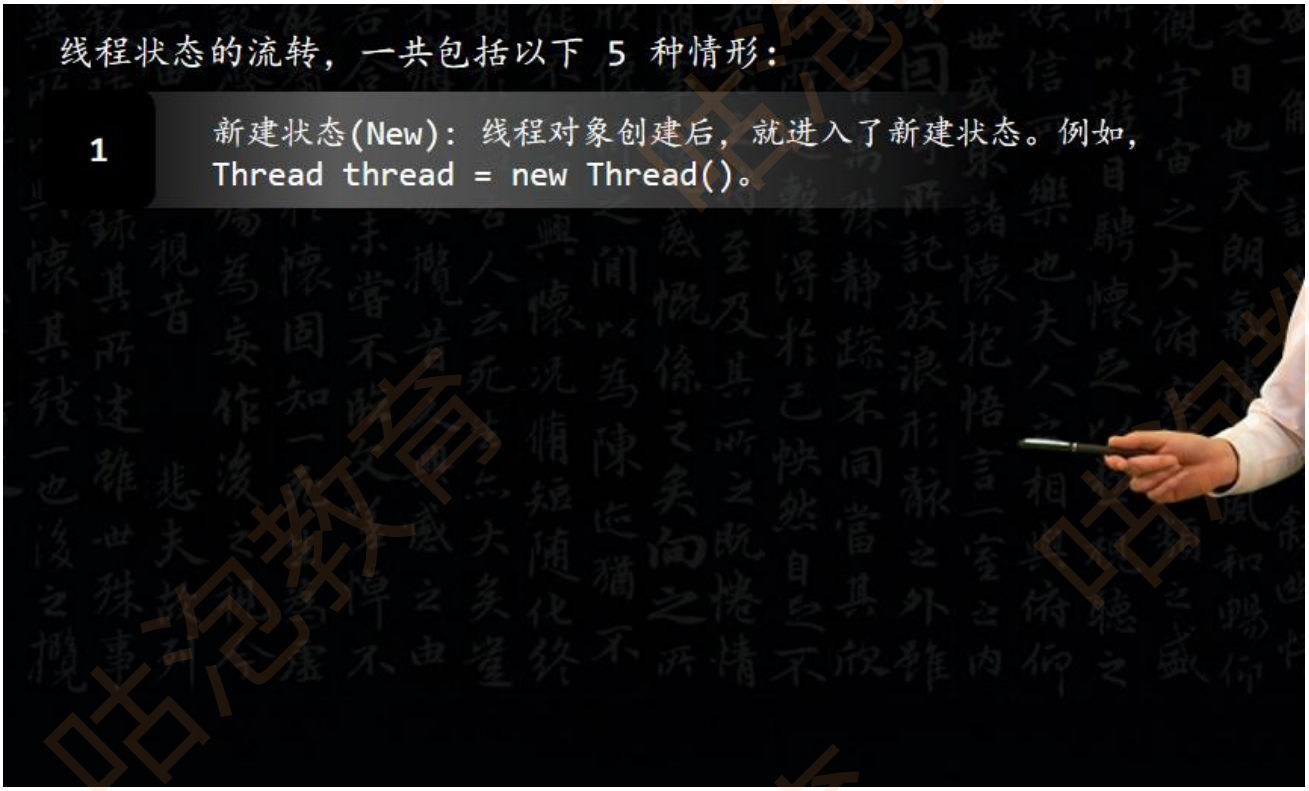
（转场，屏幕黑底白字，正中央出现“2 线程状态流转原理）

2、线程状态流转原理

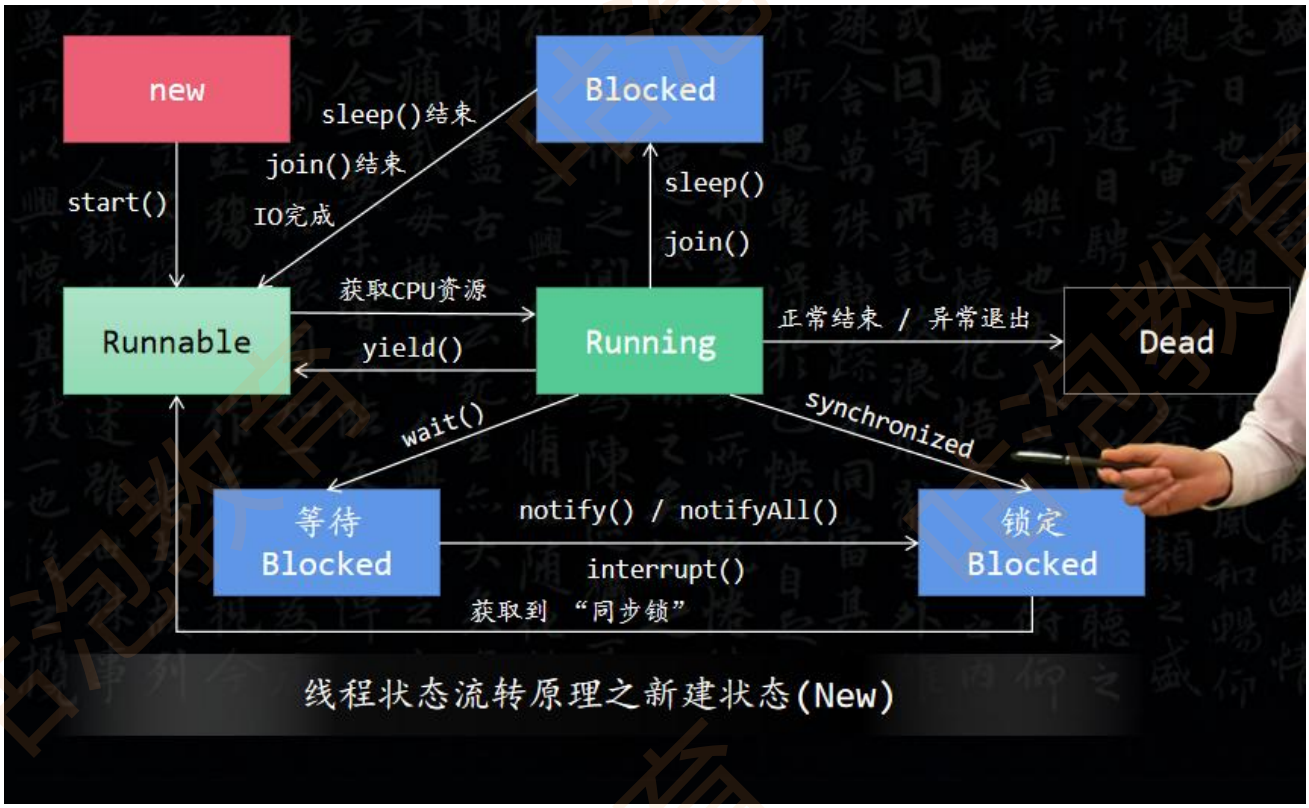
首先来看这样一张图，它涵盖了Java 中多线程各重要知识点。如果掌握了此图，Java 中的多线程也就基本上掌握了。

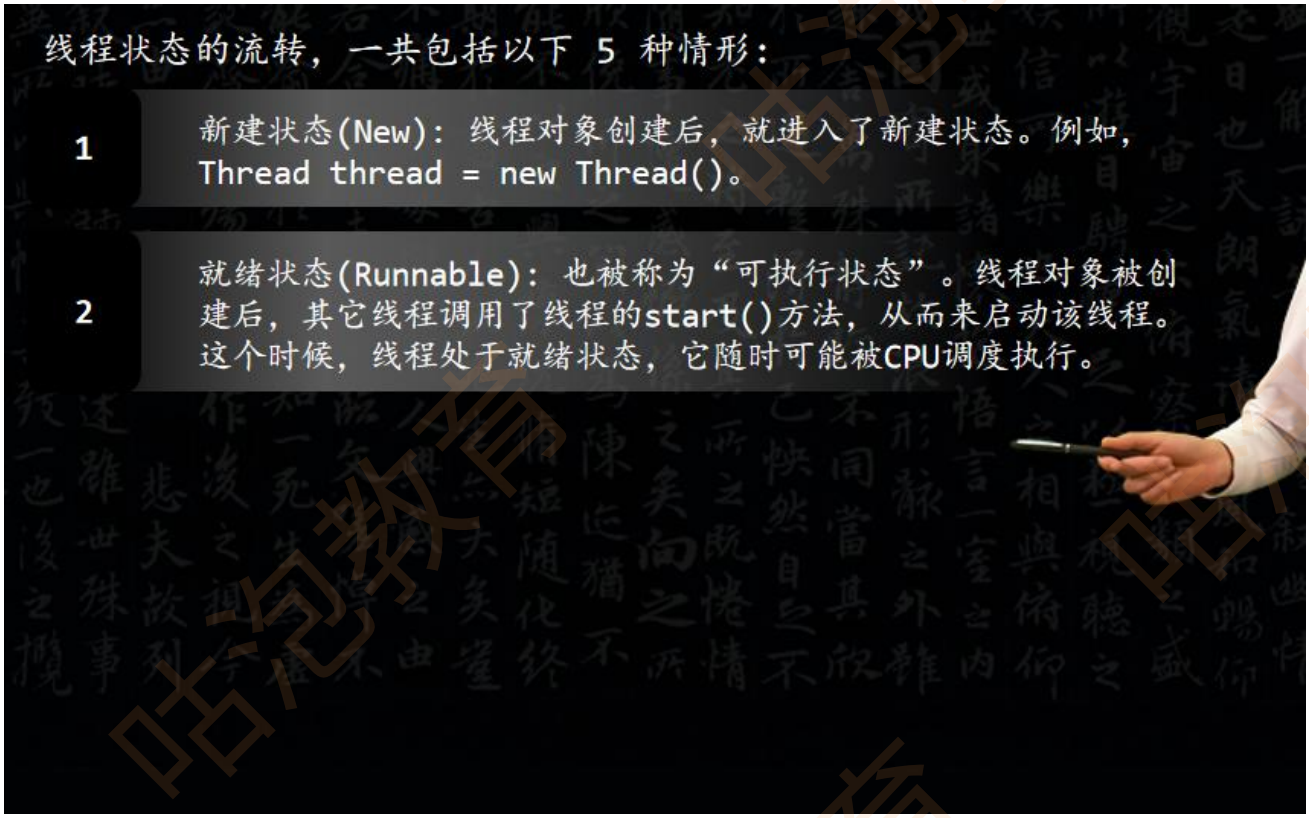


从图中可以看出，线程状态的流转，一共包括以下 5 种情形:

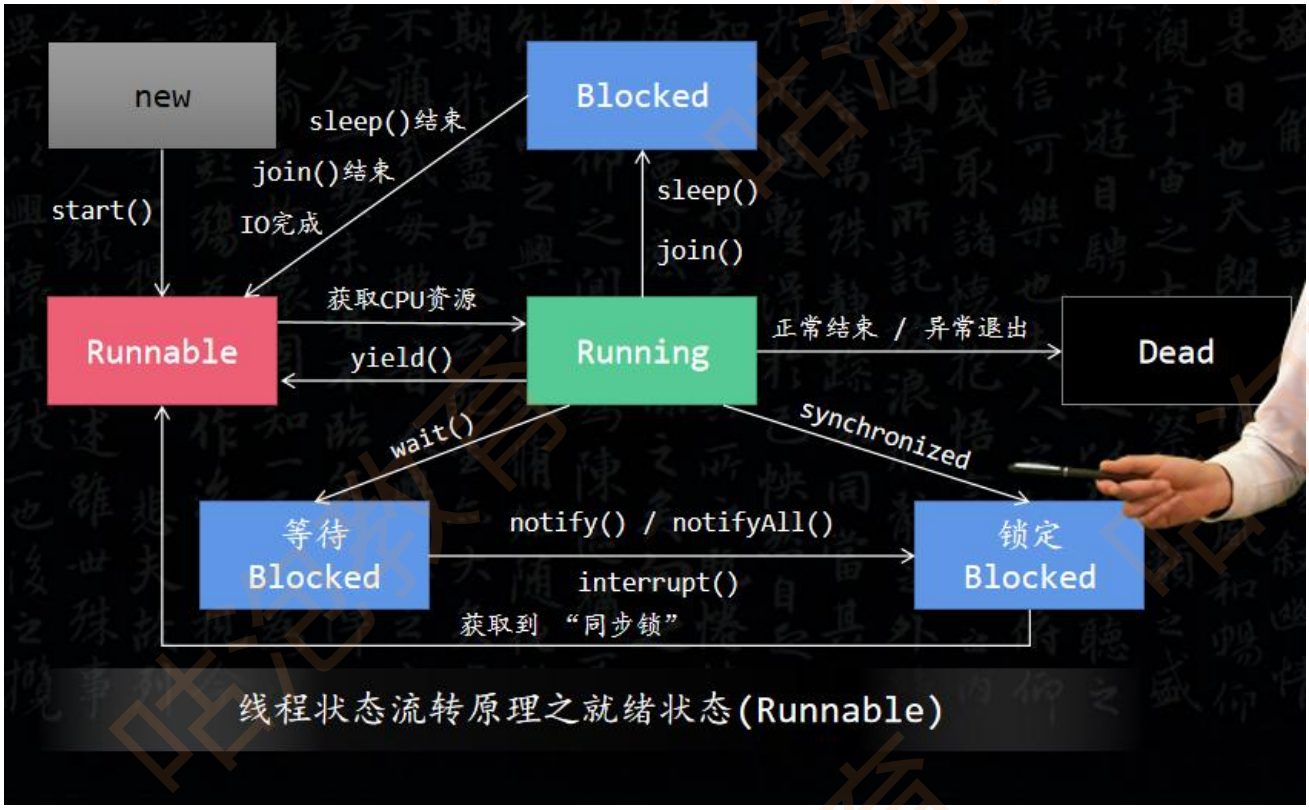


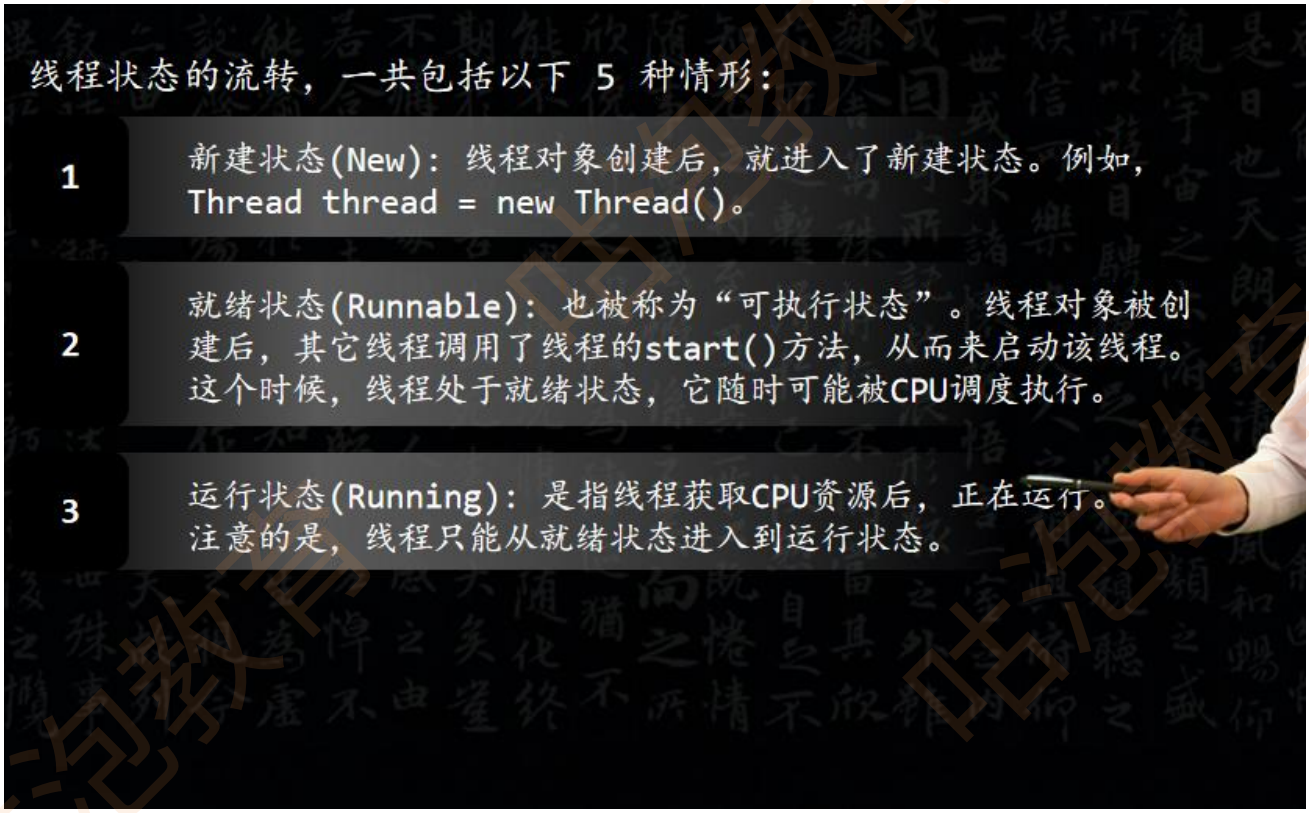
1. 新建状态(New): 线程对象创建后，就进入了新建状态。例如，Thread thread = new Thread()。



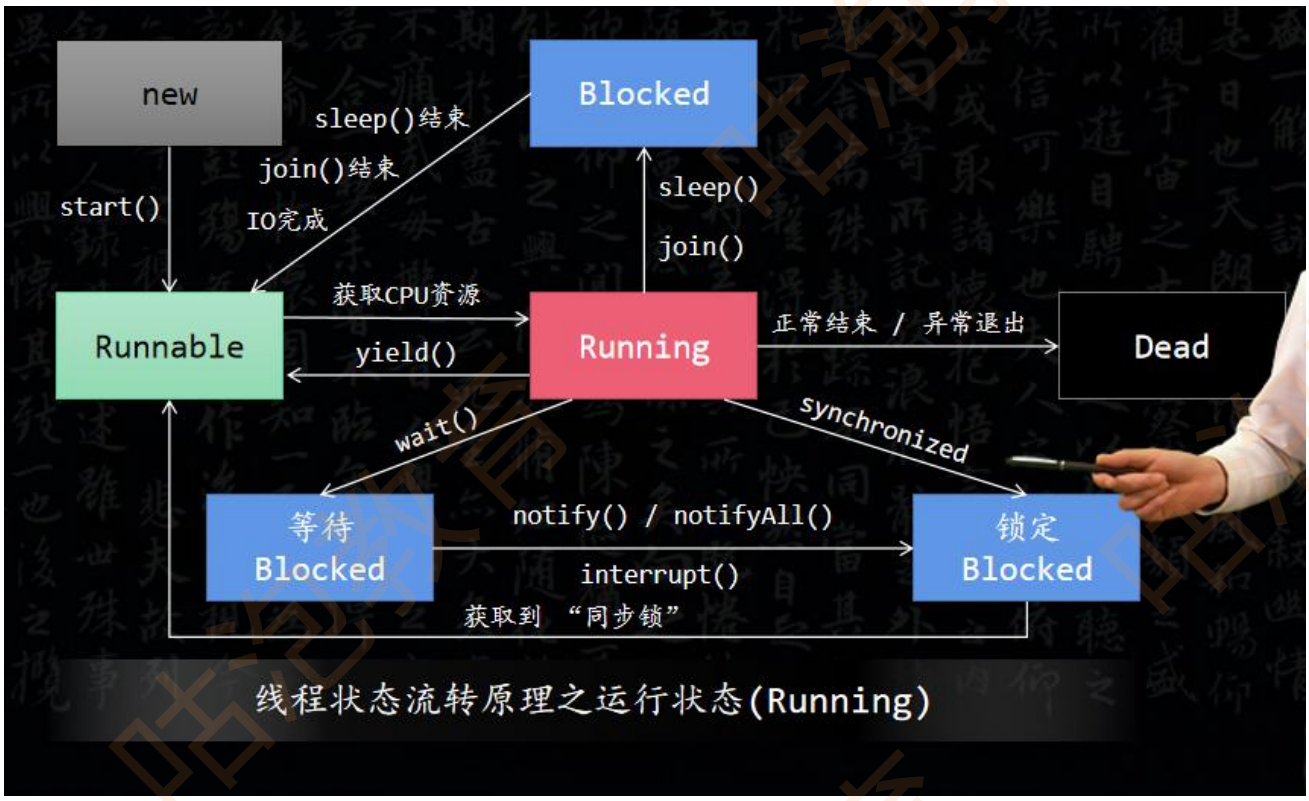


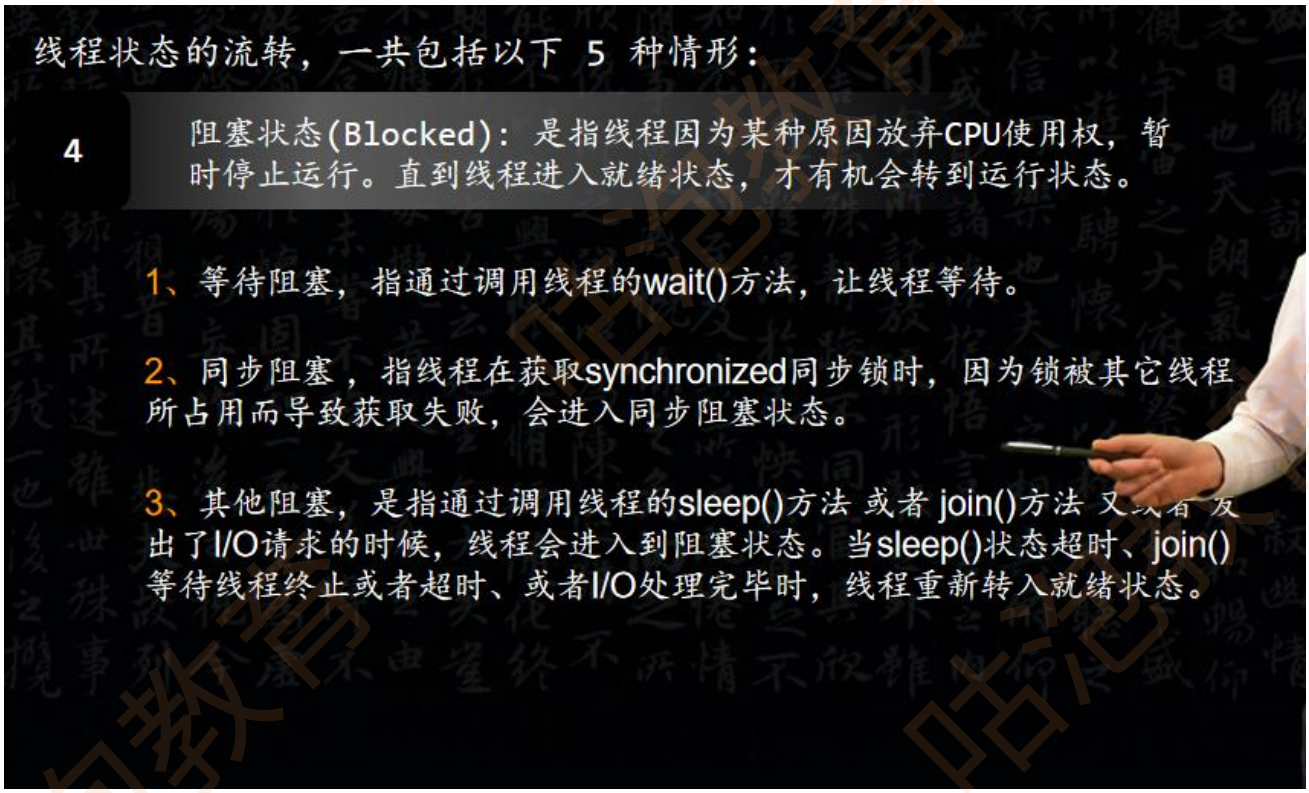
2. 就绪状态(Runnable): 也被称为“可执行状态”。线程对象被创建后，其它线程调用了线程的start()方法，从而来启动该线程。这个时候，线程处于就绪状态，它随时可能被CPU调度执行。





3. 运行状态(Running): 是指线程获取CPU资源后，正在运行。需要注意的是，线程只能从就绪状态进入到运行状态。





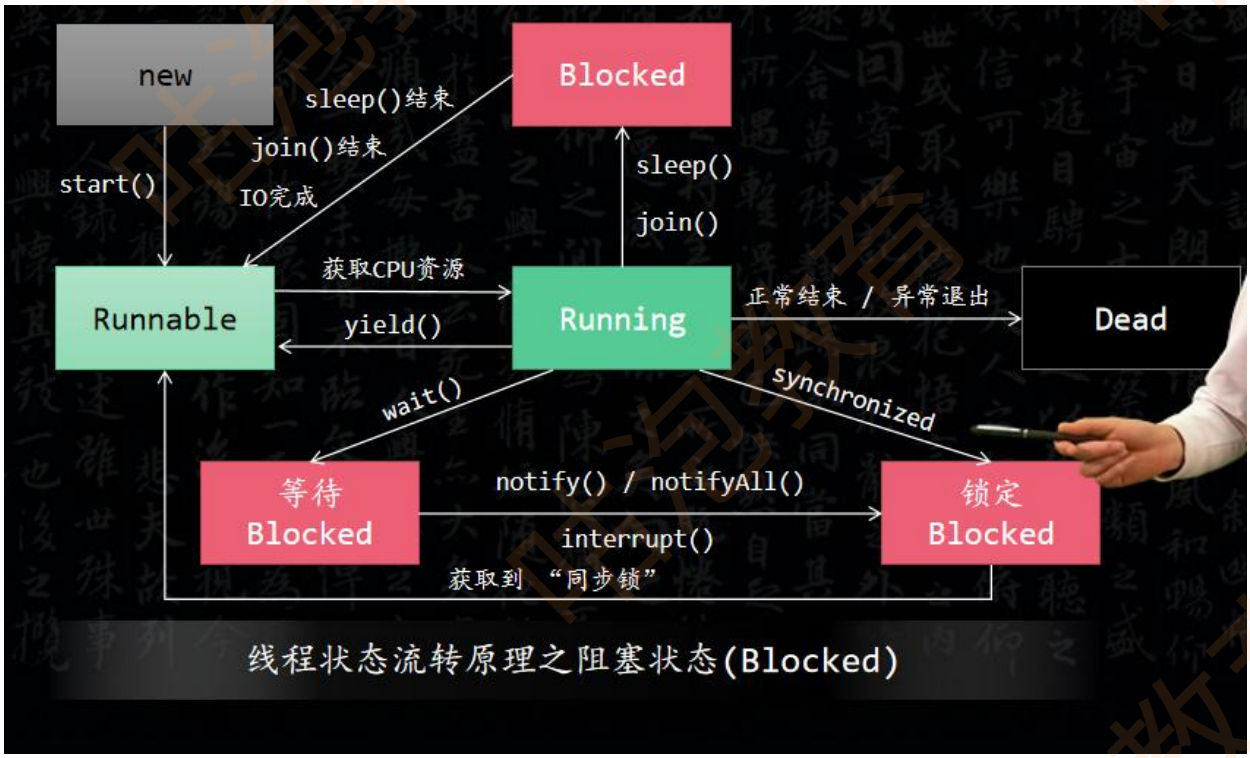
4. 阻塞状态(Blocked): 是指线程因为某种原因放弃CPU使用权，暂时停止运行。直到线程进入就绪状态，才有机会转到运行状态。

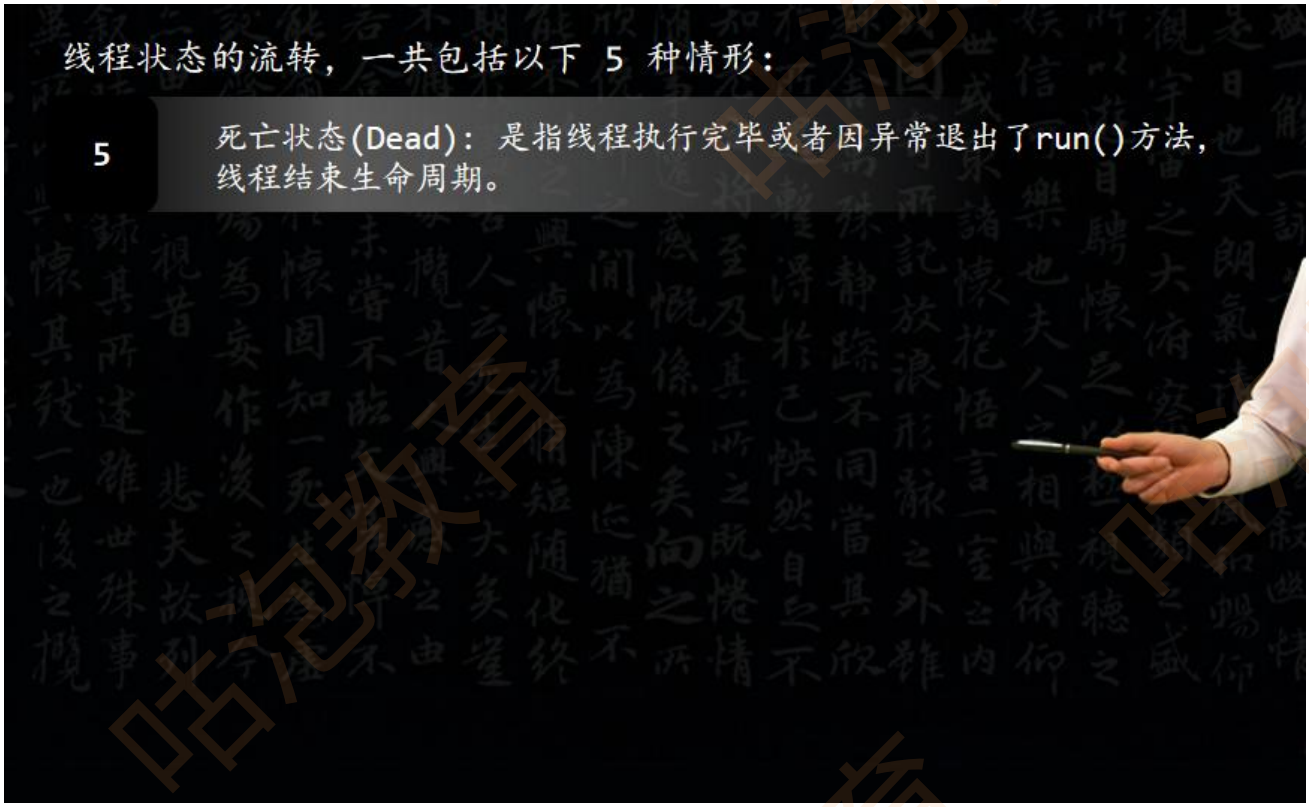
那么，阻塞又分为三种情况：

1) 等待阻塞，指通过调用线程的wait()方法，让线程等待。

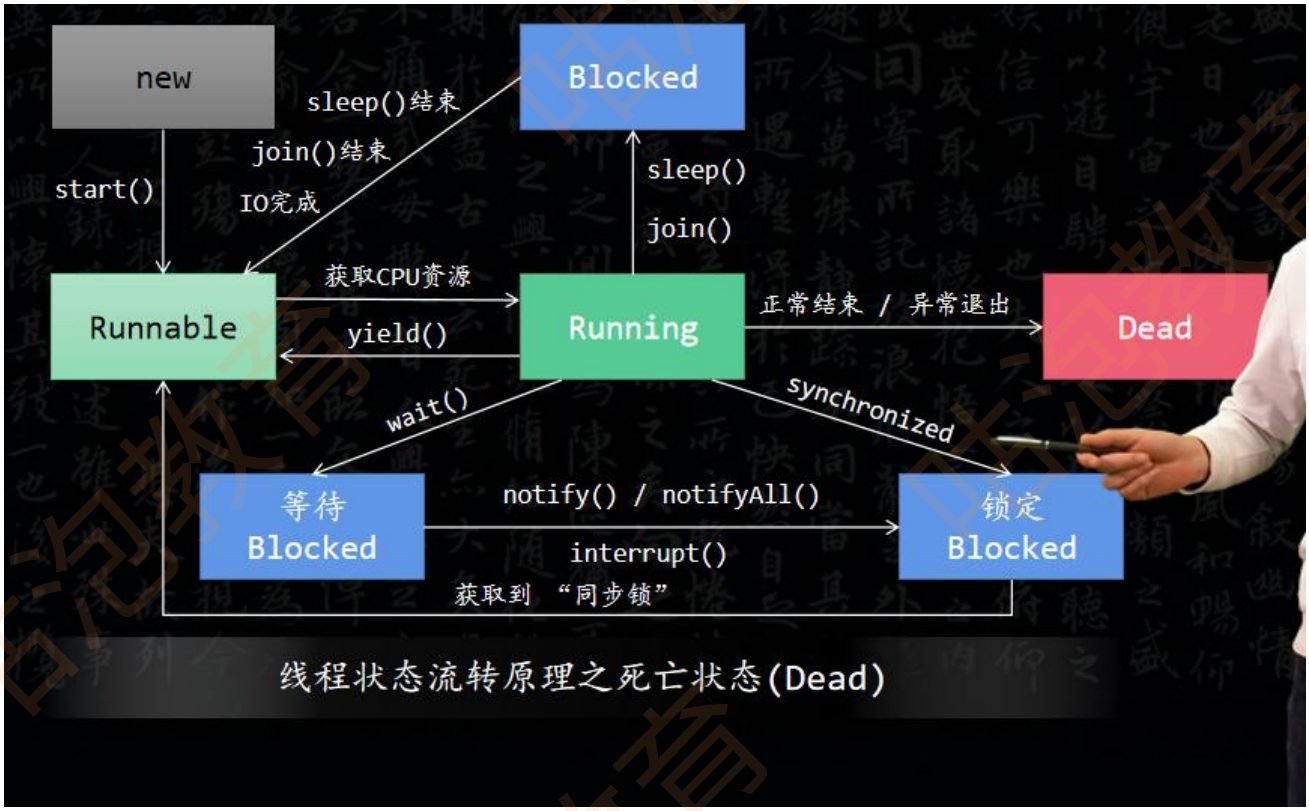
2) 同步阻塞 ，指线程在获取synchronized同步锁时，因为锁被其它线程所占用而导致获取失败，会进入同步阻塞状态。

3) 其他阻塞，是指通过调用线程的sleep()方法 或者 join()方法 又或者 发出了I/O请求的时候，线程会进入到阻塞状态。当sleep()状态超时、join()等待线程终止或者超时、或者I/O处理完毕时，线程重新转入就绪状态。

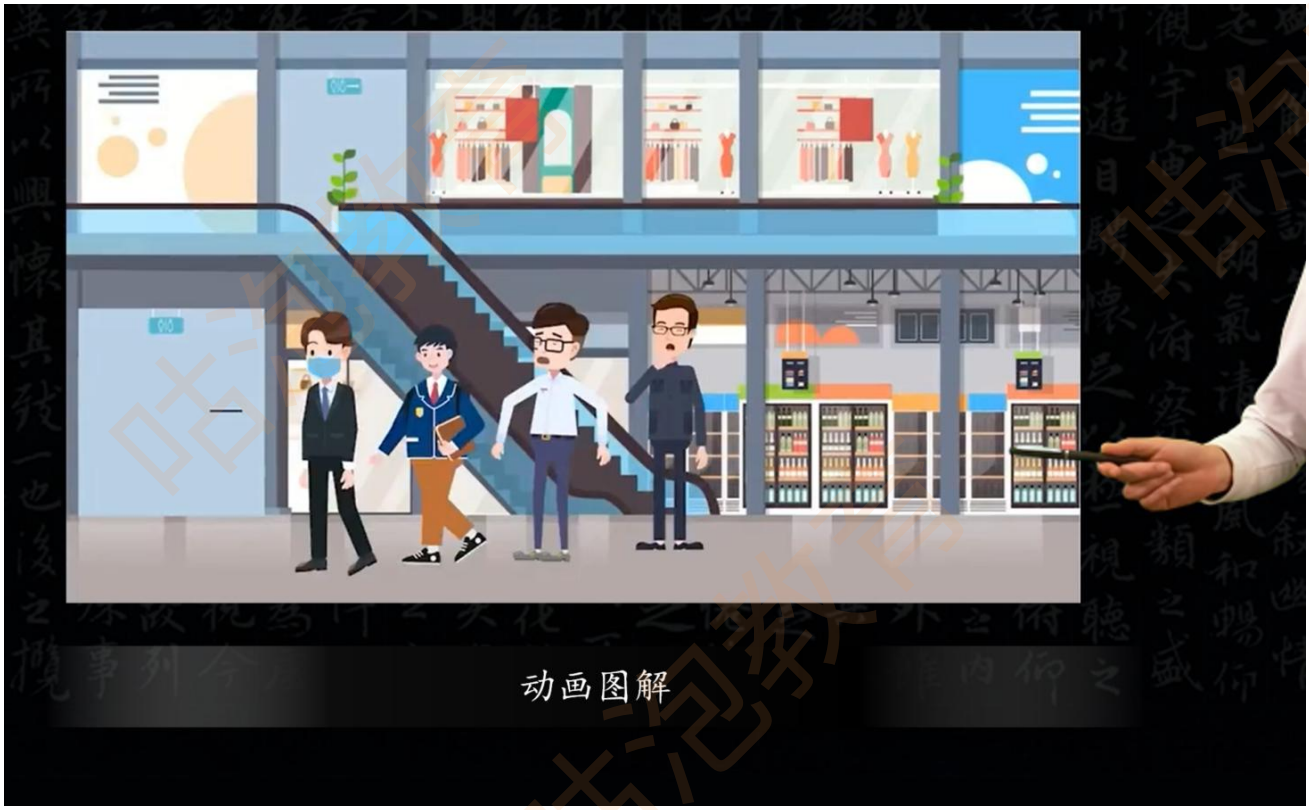




5. 死亡状态(Dead): 是指线程执行完毕或者因异常退出了run()方法，线程结束生命周期。



(此处需要配图)



举个通俗一点的例子来解释上面五种状态，比如我们平时去商场上厕所，准备去上厕所就是新建状态（new），上厕所要排队，排队就是就绪状态（Runnable），有坑位了，轮到你了，蹲坑就是运行状态（Running），蹲完坑发现没有手纸，需要等待其他人送纸过来，这个状态就是阻塞（Blocked），等上完厕所出来，上厕所这件事情结束了线程也就不存在了，就是死亡状态。

需要注意的是：便秘也是阻塞状态，你便秘太久，其他人会等不及，可能会把你赶走，这个就是挂起。还有一种情况就是，如果你便秘占坑位太久，其他人跟你说，先出去酝酿一下，5分钟后再过来蹲坑，这就叫睡眠。

【结尾】

好了，以上就是我对线程状态流转原理的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

104.请你对比一下 Java 和 JavaScript的区别

【导航条：引言】

一位应届生在面试的时候把所有的八股文都答对了。没成想，有位面试官出了个奇招，面试官问了一句说，请你对比一下Java和JavaScript的区别。我这想这样入门级别的问题，应该每一个后端开发人员都能答得上吧。没成想，有位应届毕业生竟然没有答出来，还说JavaScript是对Java的升级。这样的回答，实在是有点不应该。

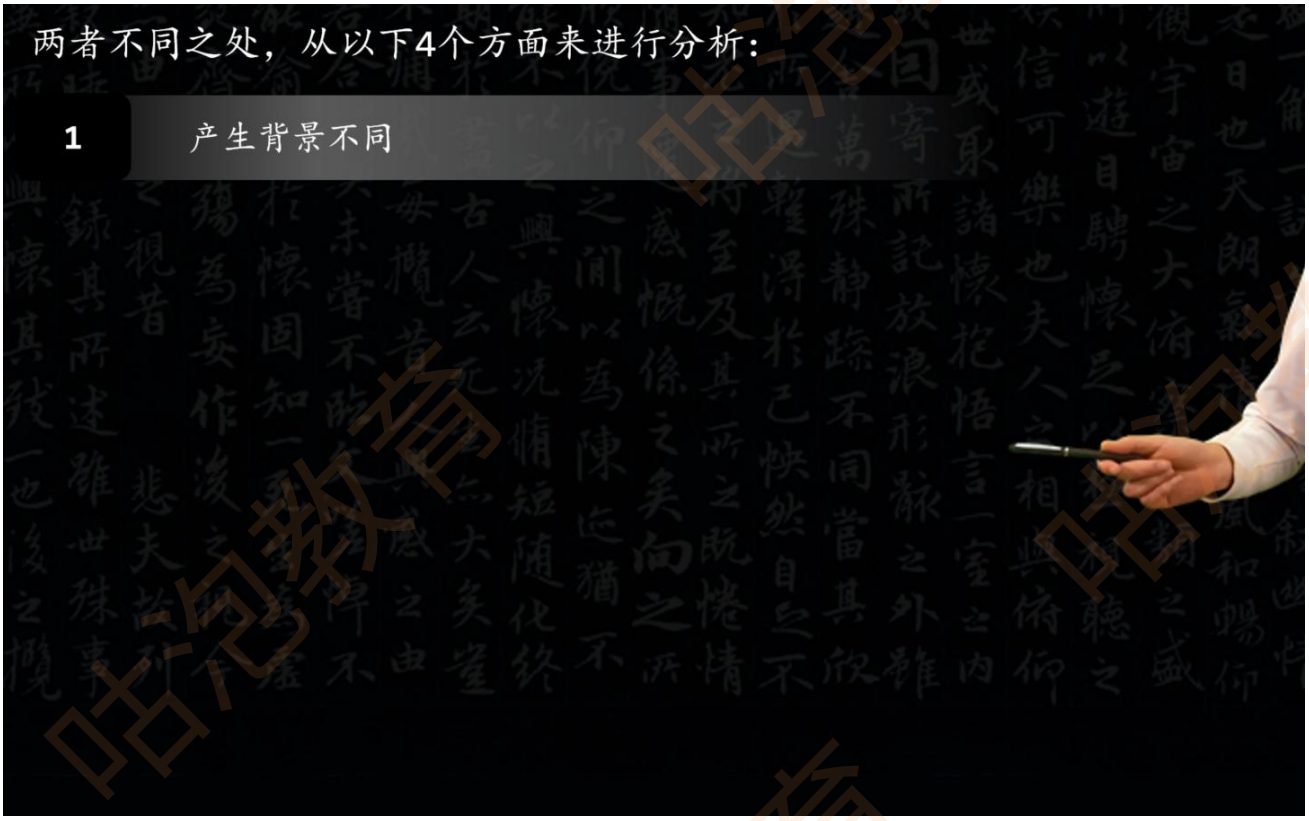
今天呢，我给大家总结一下Java和JavaScript的异同之处，

【导航条：不同之处】

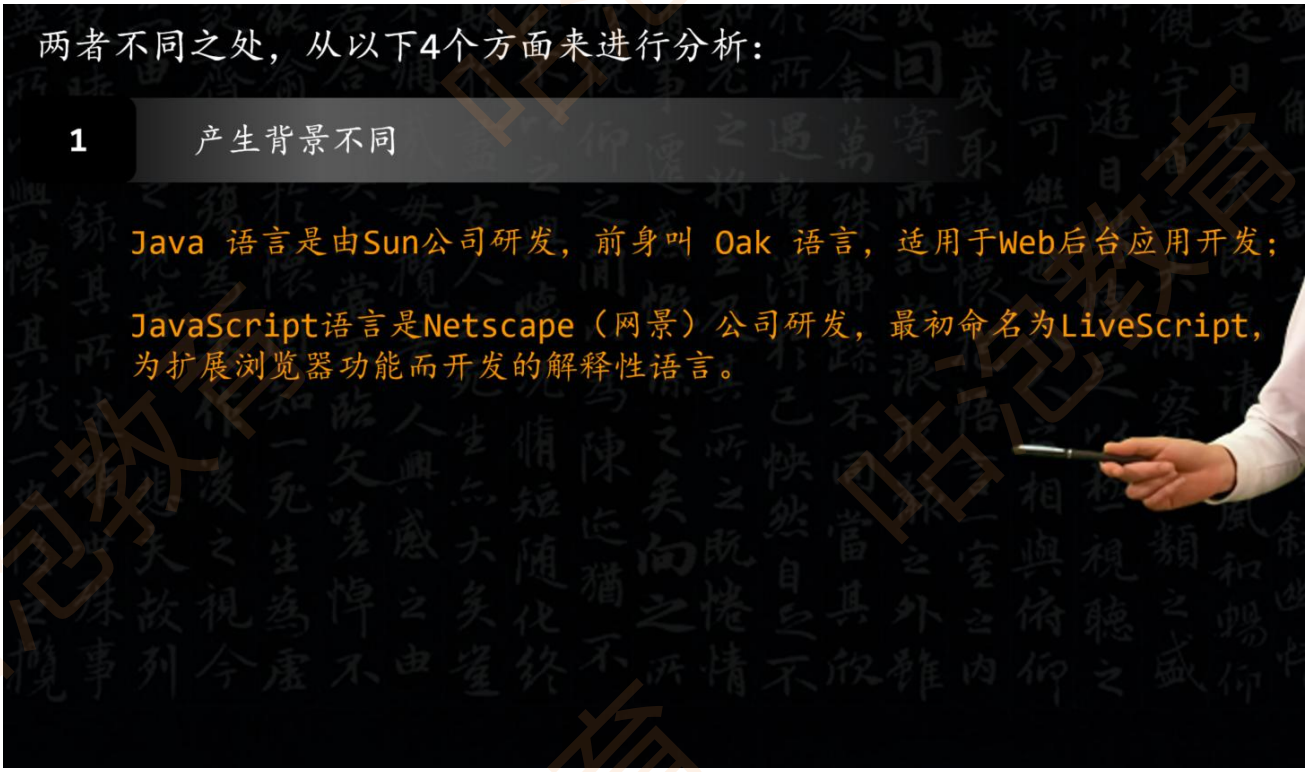
（转场，屏幕黑底白字，正中央出现“1 两者不同之处）

1、两者不同之处

首先总结两者不同之处，我从以下4个方面来进行分析：

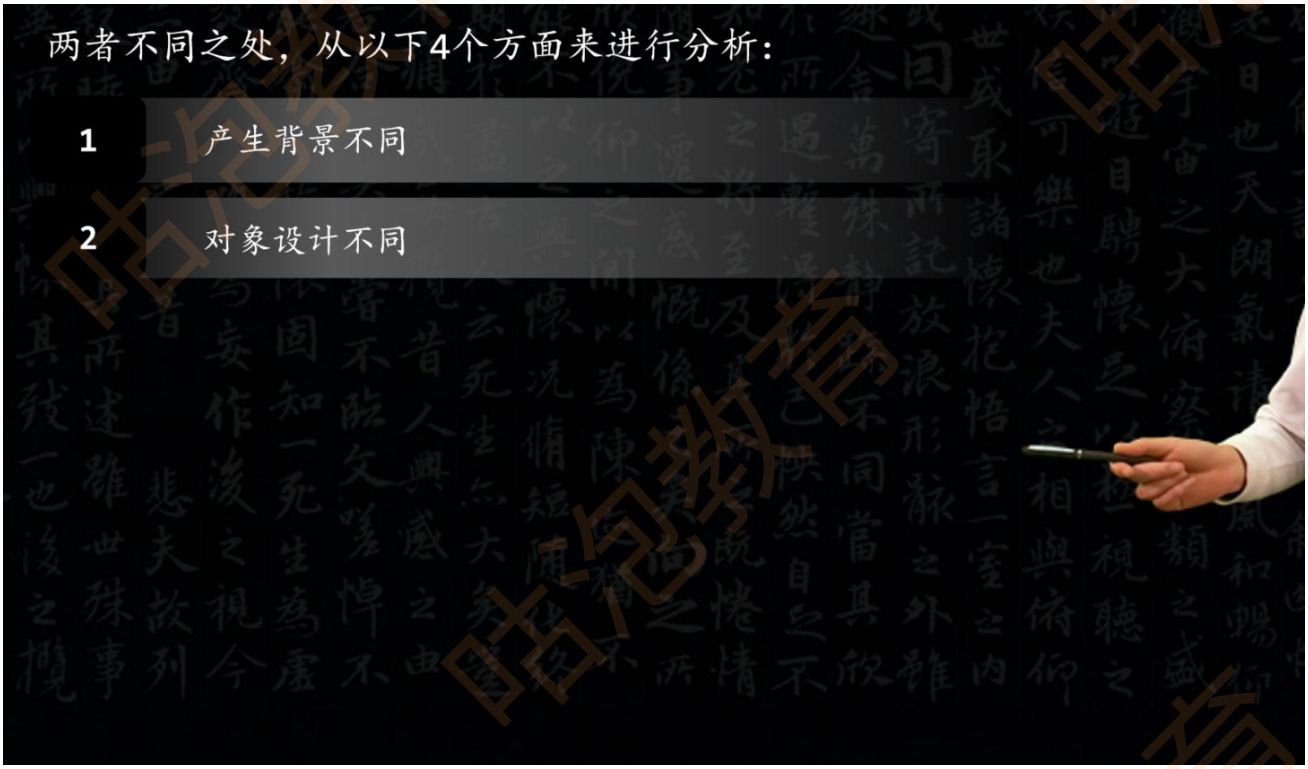

两者不同之处，从以下4个方面来进行分析：
1
产生背景不同

1）产生背景不同

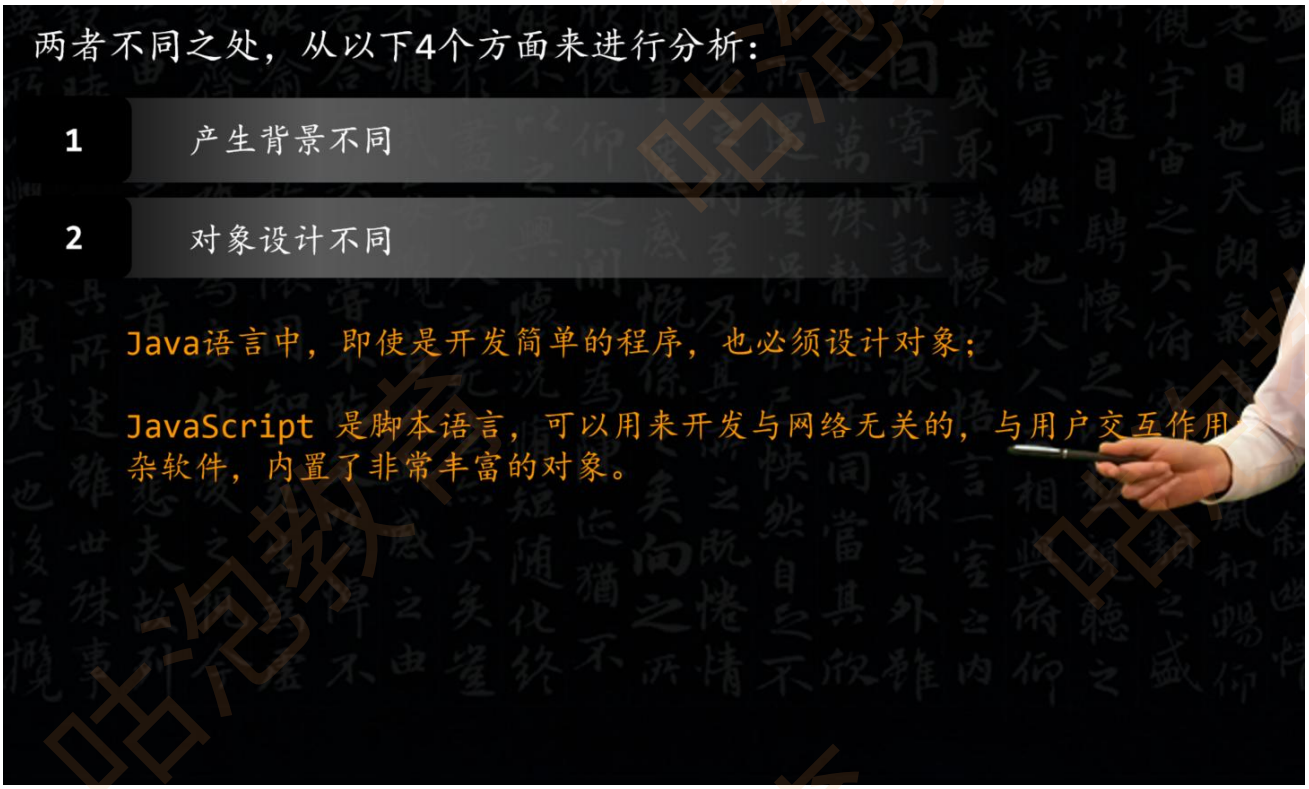

两者不同之处，从以下4个方面来进行分析：
1
产生背景不同
Java 语言是由Sun公司研发，前身叫 Oak 语言，适用于Web后台应用开发；
JavaScript语言是Netscape（网景）公司研发，最初命名为LiveScript，
为扩展浏览器功能而开发的解释性语言。

最初 Java 语言是由Sun公司研发，前身叫 Oak 语言，1995年5月正式对外发布并开源，特别 。

而JavaScript是Netscape（网景）公司的产品，最初被名命名为LiveScript。为了扩展Netscape 浏览器的功能而开发的解释性语言。因为Netscape与Sun合作，Netscape（网景）公司管理层希望它外观看起来像Java，因此取名为JavaScript。

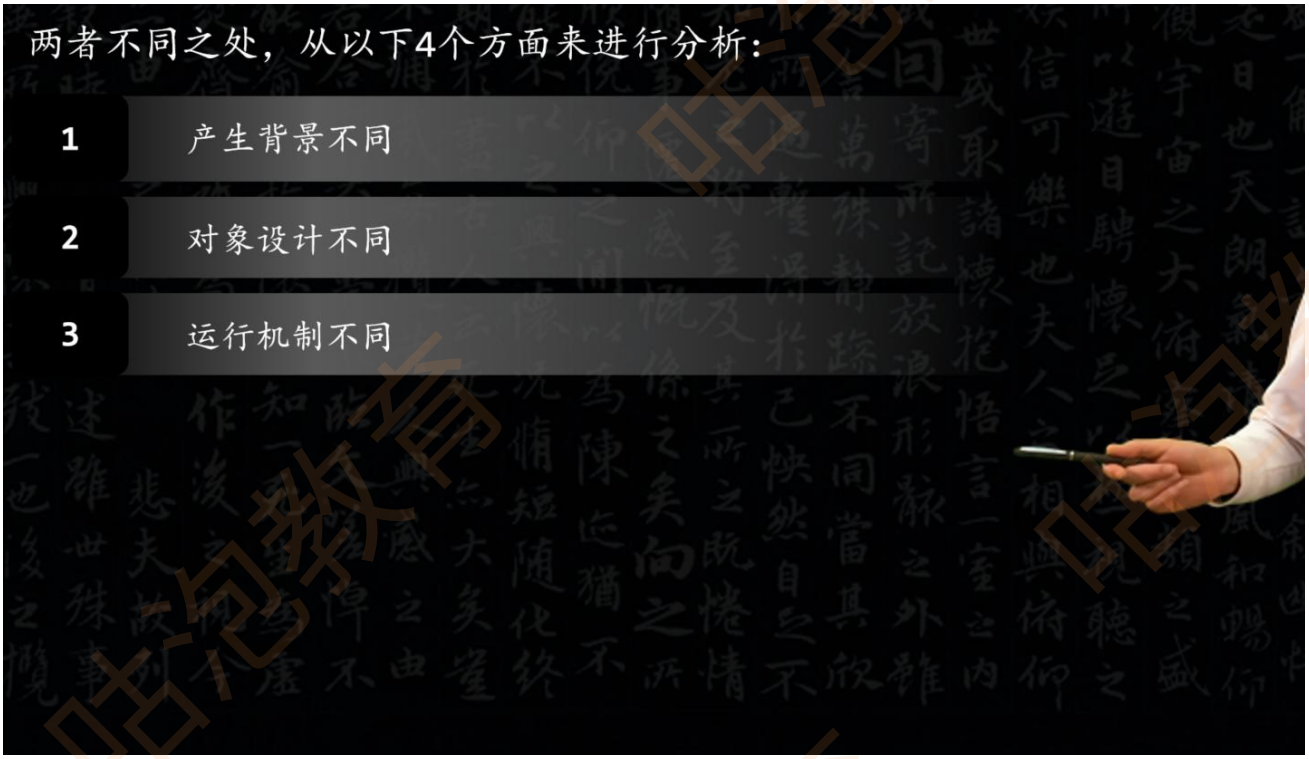


2）对象设计不同

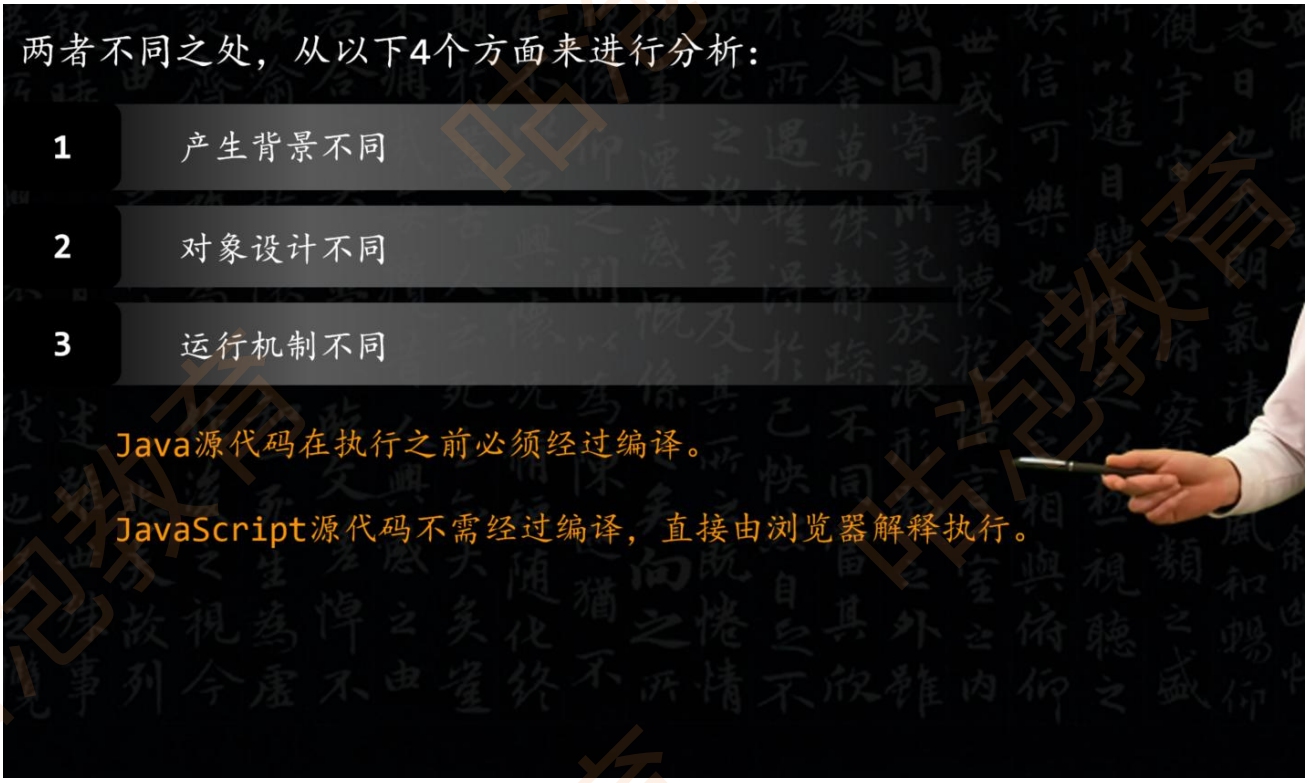


Java 是一种真正的面向对象的语言，即使是开发简单的程序，必须设计对象；

而JavaScript 是脚本语言，可以用来开发与网络无关的，与用户交互作用的复杂软件。它是一种基于对象（Object-Based）和事件驱动（Event-Driven）的编程语言，它内置了非常丰富的对象供开发人员使用。

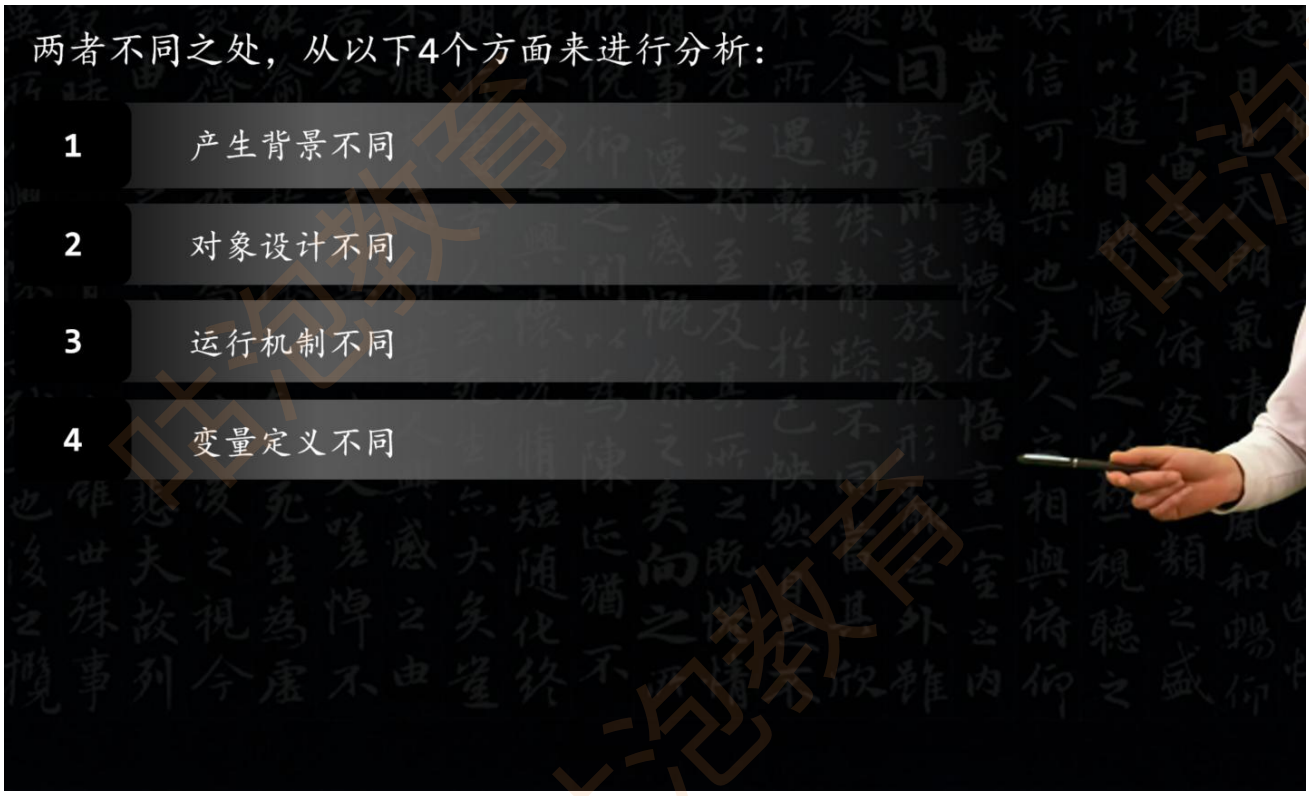

两者不同之处，从以下4个方面来进行分析：
1 产生背景不同
2 对象设计不同
3 运行机制不同

3）运行机制不同

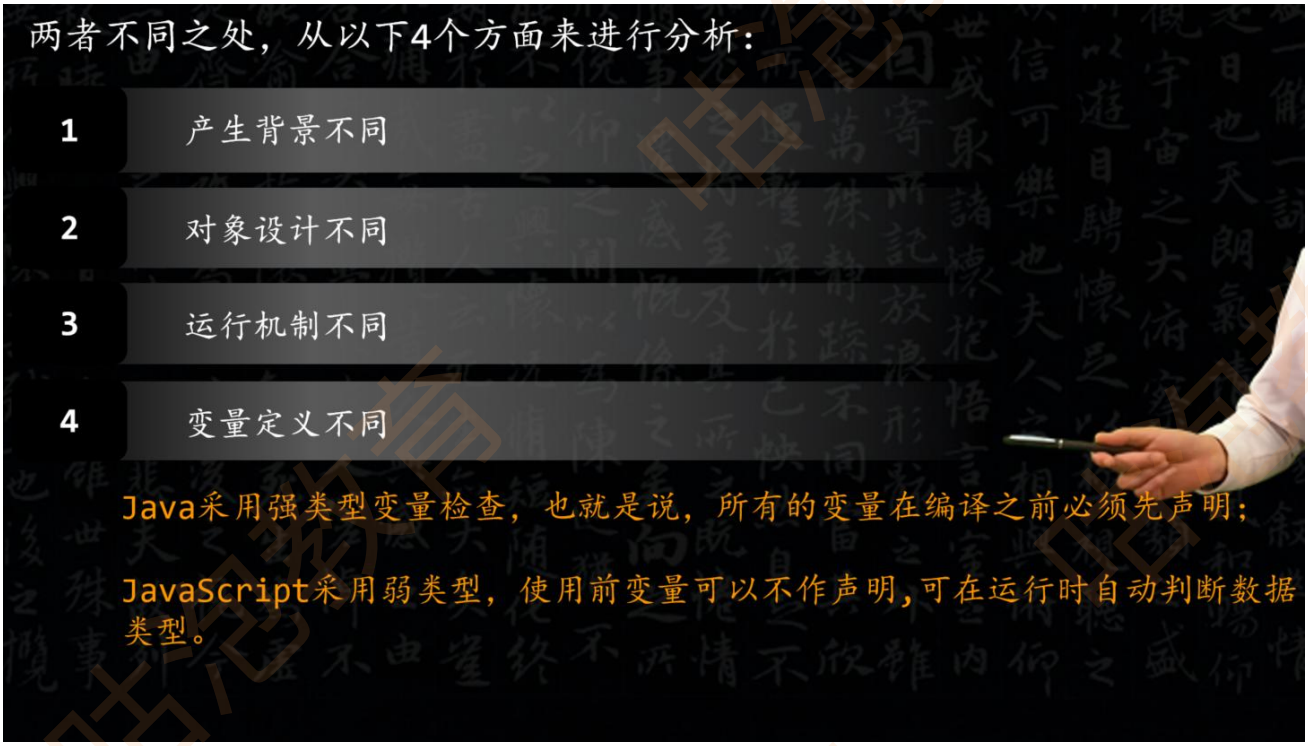

两者不同之处，从以下4个方面来进行分析：
1 产生背景不同
2 对象设计不同
3 运行机制不同
Java源代码在执行之前必须经过编译。
JavaScript源代码不需经过编译，直接由浏览器解释执行。

Java 的源代码在执行之前必须经过编译。

而JavaScript 是一种解释性编程语言，源代码不需经过编译，直接由浏览器解释执行。（目前的浏览器几乎都使用了 JIT（即时编译）技术来提升 JavaScript 的运行效率）



4）变量定义不同



Java 采用强类型变量检查，也就是说，所有的变量在编译之前必须先声明；

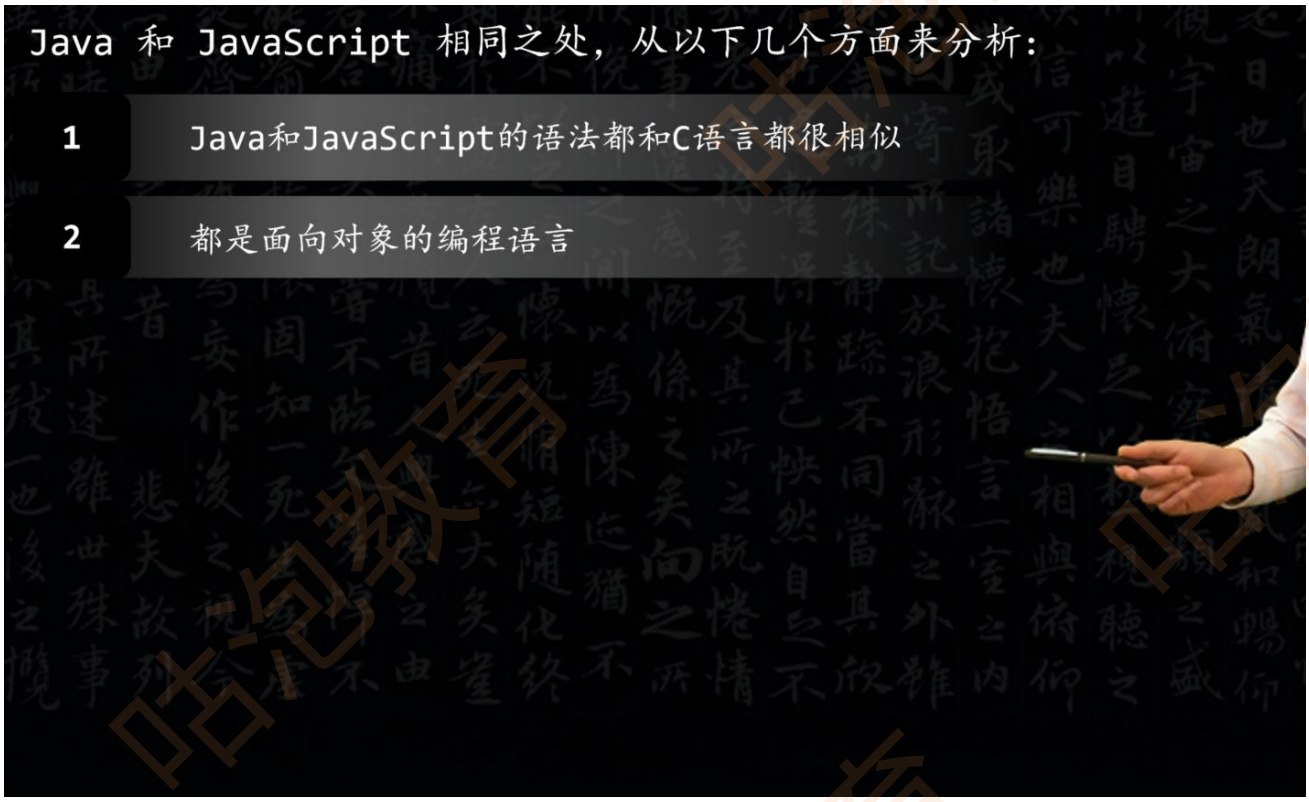
而JavaScript 中变量是弱类型的，甚至在使用前变量可以不作声明。JavaScript 的解释器会在运行时自动判断数据类型。

【导航条：相同之处】

（转场，屏幕黑底白字，正中央出现“2 两者相同之处）

2、两者相同之处

当然，Java 和 JavaScript 也很多相同之处，我从以下几个方面来分析：



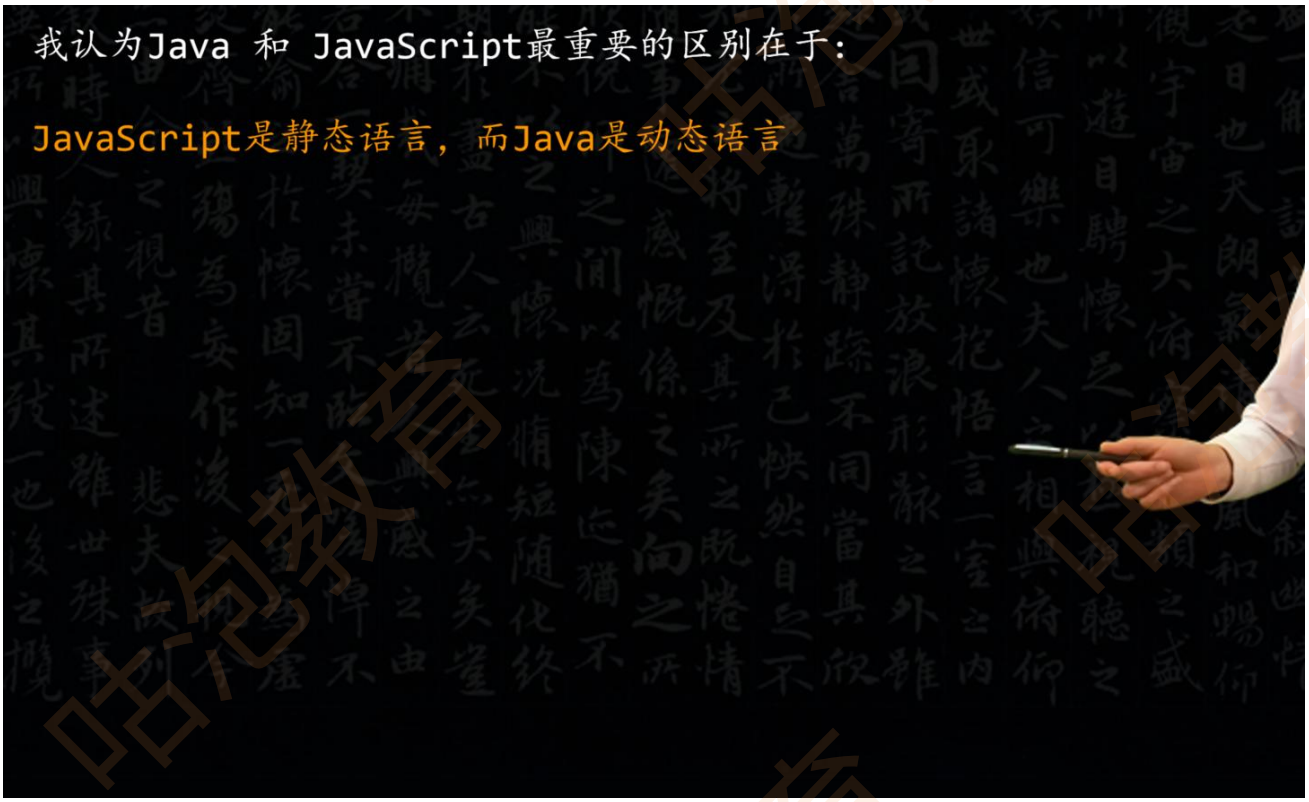
1）Java和JavaScript的语法都和 C 语言都很相似；

2）都是面向对象的。虽然实现的方式略有不同，但是JavaScript 在设计时参照了 Java 的命名规则；

【导航条：总结】

（转场，屏幕黑底白字，正中央出现“3 总结）

3、总结



JavaScript 除了名字长得和 Java 比较像之外，它语言风格相差非常大。JavaScript 在设计时不仅参考了Java，还参考了 Self 和 Scheme 等其他语言。

Java和JavaScript各有所长各有专精，Java的舞台在编程领域，而JavaScript的得心之处是在Web页面中，它几乎无所不能。

我上面列出的相同点和不同点都在网上流传的所谓的标准答案。但是，我认为Java 和 JavaScript最重要的区别在于：JavaScript是静态语言，而Java是动态语言。

目前的编程语言的发展趋势是函数式语言和动态语言。在 Java 中，类（Class）是一等公民，而 JavaScript 中，函数（Function）是一等公民，因此 JavaScript 支持函数式编程，可以使用 Lambda函数和闭包（closure）。

当然，Java 8 也开始支持函数式编程，提供了对 Lambda表达式以及函数式接口的支持。对于这类问题，在面试的时候最好还是用自己的语言回答会更加靠谱，不要背网上所谓的标准答案。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

【结尾】

好了，以上就是我对Java和JavaScript的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

105.谈谈ArrayList、Vector和LinkedList 的存储性能及特性

【导航条：引言】

又有一位工作2年的小伙伴面试的时候，被问到一个集合相关的问题。说请你谈谈ArrayList、Vector和LinkedList 的存储性能及特性。

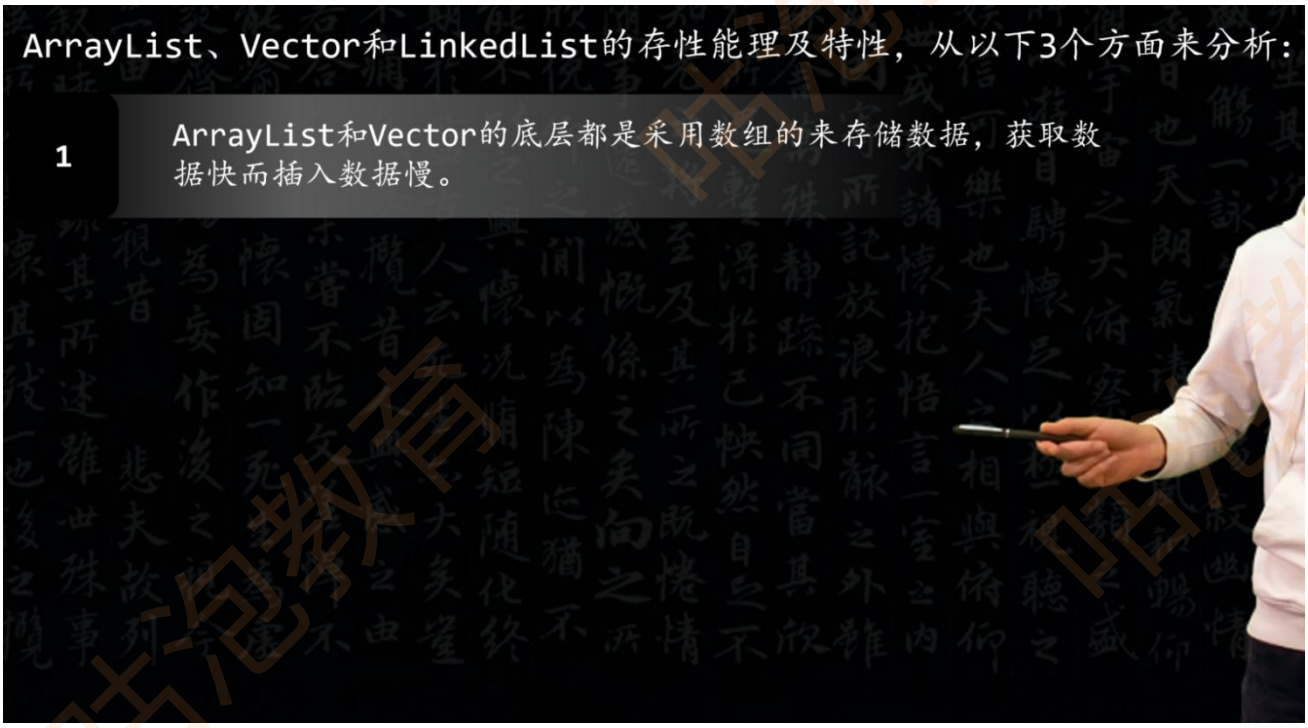
今天呢，我给大家分享一下我对这个问题的理解。

【导航条：存储性能及特性】

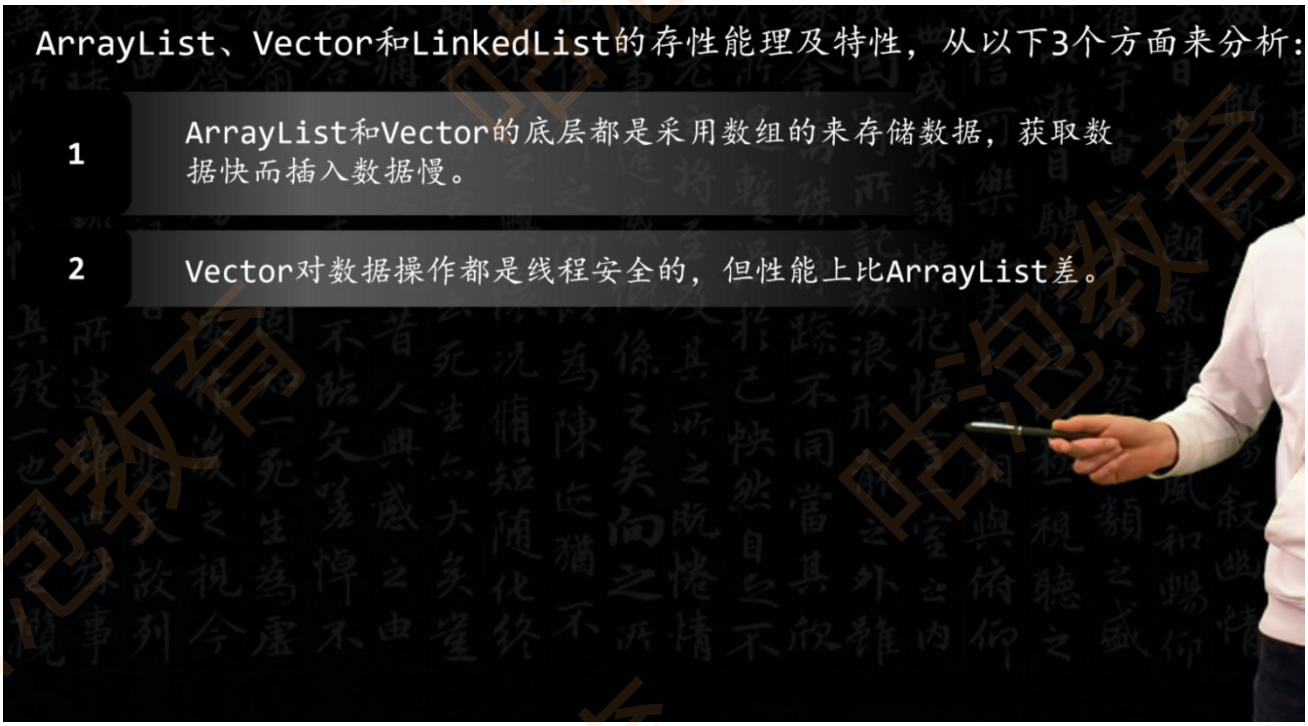
（转场，屏幕黑底白字，正中央出现“1 存储性能及特性）

1、存储性能及特性

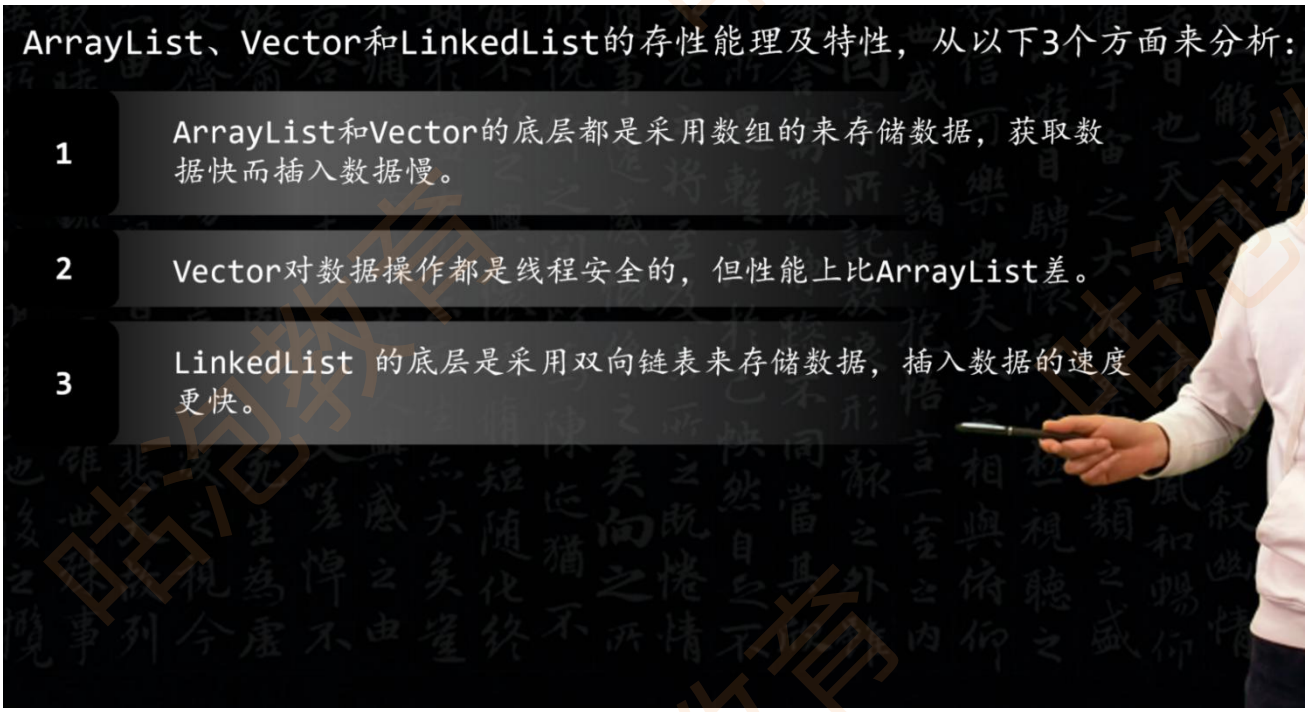
关于ArrayList、Vector和LinkedList 的存性能理及特性，我从以下3个方面来分析：



1）首先，ArrayList 和 Vector 的底层都是采用数组的来存储数据，而且都是根据索引来取数据，这样设计使得获取数据快而插入数据慢。另外，每次扩容都要移动数组中的元素，存储数据量较大的时候会影响读写性能。



2）其次，由于Vector 中的方法都使用了 synchronized 修饰，因此，Vector 中对数据操作都是线程安全的，但性能上比ArrayList 差。



3）然后，LinkedList 的底层是采用双向链表来存储数据的，也就是说将内存中零散的内存单元通过附加的引用关联起来，形成一个可以按序号索引的线性结构，这种链式存储方式与数组的连续存储方式相比，内存的利用率更高。LinkedList获取数据需要根据索引序号，向前或者向后遍历，但是插入数据时只需要记录本项的前后项即可，所以，LinkedList插入数据的速度更快。

4）最后，再补充一点，Vector是Java 早期的版本中提供的容器， 属于遗留容器，官方已经不再推荐使用。但是由于 ArrayList 和 LinkedListed 都是非线程安全的，在多线程环境下，可以使用工具类Collections 的 synchronizedList() 方法，将容器转换成线程安全的容器再使用。这其实也是装饰器模式的一种应用。

【导航条：遗留容器】

（转场，屏幕黑底白字，正中央出现“2 关于遗留容器）

2、关于遗留容器

关于Java中的遗留容器，我最后再补充一下。除Vector之外，还有Hashtable、Dictionary、BitSet、Stack、Properties都是遗留容器，这些容器中，Properties 类存在比较严重的设计缺陷。来看这段源码：

```
/*

Since : JDK1.0
```

```
See Also : native2ascii tool for Solaris, native2ascii tool for Windows

Author : Arthur van Hoff, Michael McCloskey, Xueming Shen

*/

public class Properties extends Hashtable<Object,Object> {



}
```

Properties是一个键和值都是字符串的特殊的键值对映射，在设计上应该是关联一个Hashtable，并将它的两个泛型参数设置为 String 类型，但是 Java API 中的Properties是直接继承了 Hashtable，这很明显是对继承的滥用。主要体现在以下两个方面：

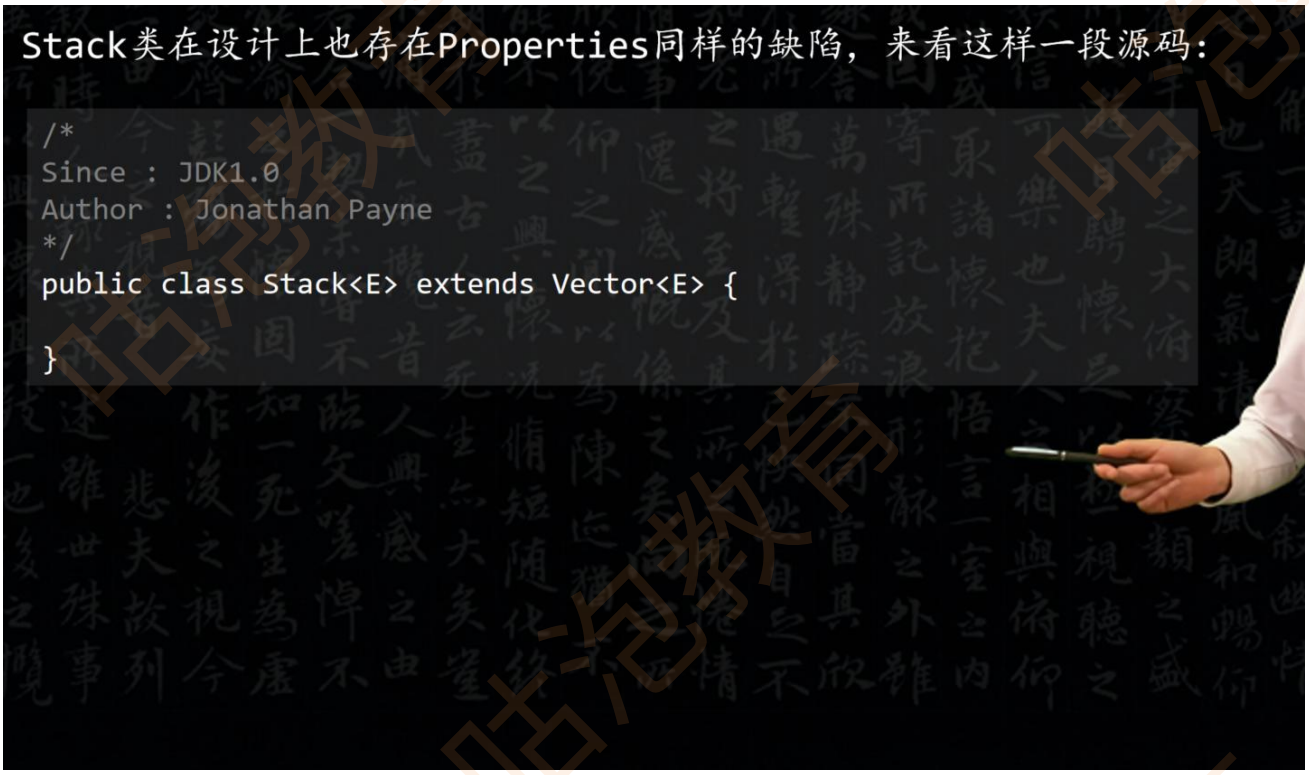
1）首先，根据合成复用原则，这里Properties 和Hashtable的代码复用关系应该是 Has-A 关系，而不是 Is-A

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

关系。

2）另一方面，这两个容器都属于工具类，继承工具类本身就是一个错误的做法，使用工具类最好的方式是Has-A 关系（关联）或Use-A 关系（依赖）。

既然都讲到这里了，最后再扩展一下Stack 类在设计上也存在Properties同样的缺陷。来看这样一段源码：




```
/*

Since : JDK1.0

Author : Jonathan Payne

*/

public class Stack<E> extends Vector<E> {
```

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果


```
}
```

在JDK的util包中，我们发现Stack类也是继承了 Vector，这个设计也是不太合理的。

【结尾】

好了，以上就是我对ArrayList、Vector和LinkedList的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

106.在 Java 中，CountdownLatch 和 CycliBarriar 有什么区别?

【导航条：引言】

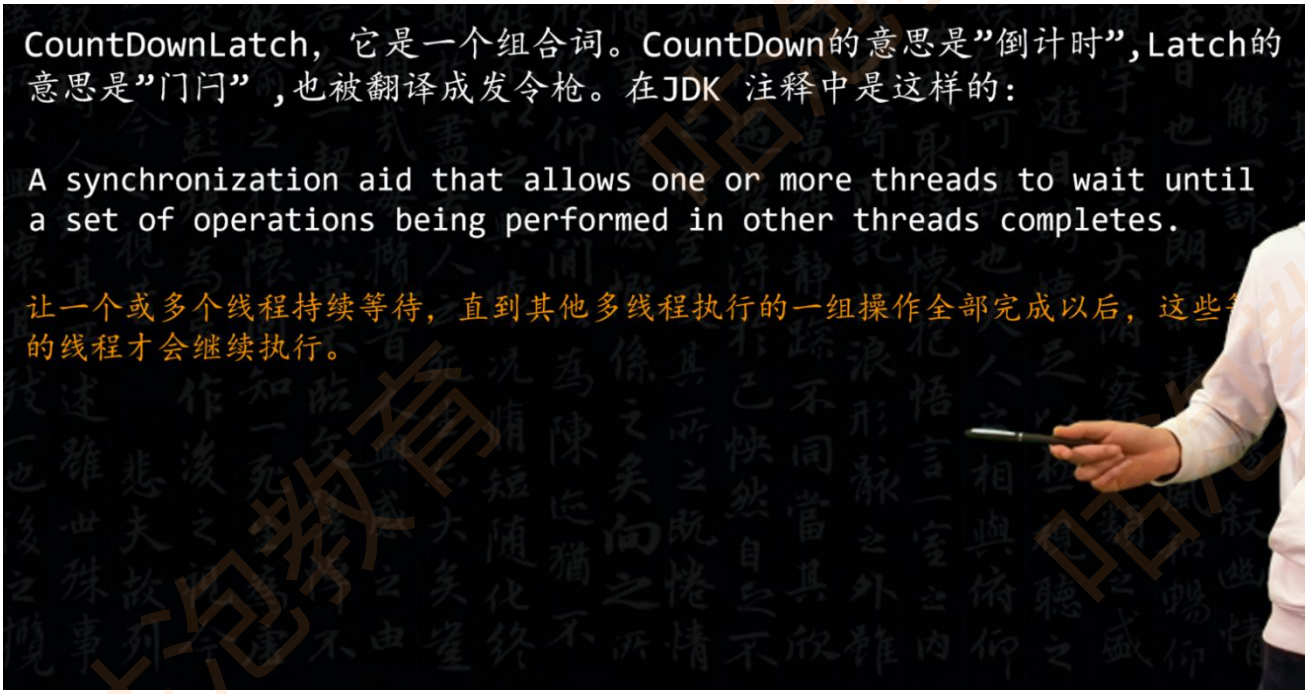
一位工作5年的小伙伴面试的时候，被问到这样一个问题，说，CountdownLatch 和 CycliBarriar 有什么区别？这个对于很多人都会比较陌生，但是接触过并发编程的小伙伴来说还是比较简单的。

今天呢，我给大家分享一下我对这个问题的理解。

【导航条：CountdownLatch】

（转场，屏幕黑底白字，正中央出现“1 CountdownLatch）

1、CountdownLatch



先来看CountDownLatch，它是一个组合词。CountDown 的意思是 倒计时，Latch 的意思是 门闩 ,也被翻译成发令枪。在JDK 注释中是这样的：

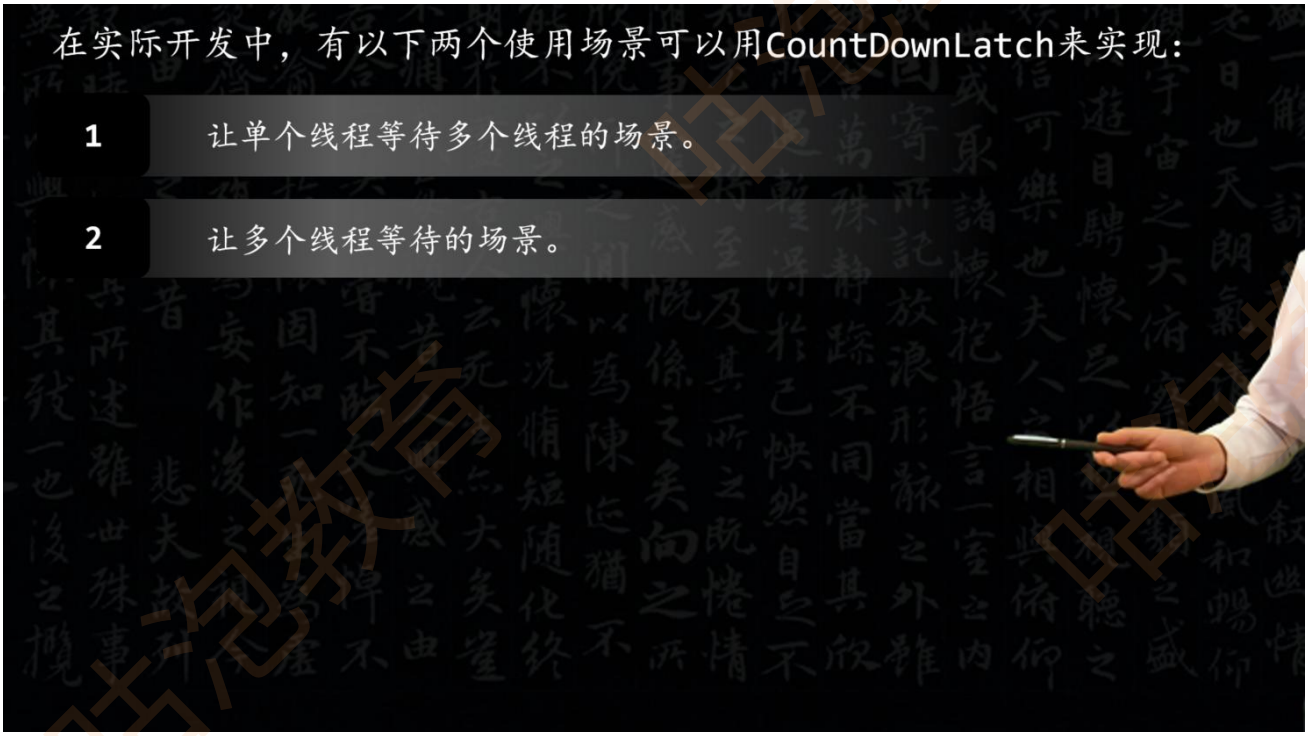
A synchronization aid that allows one or more threads to wait until a set of operations being performed in other threads completes.

翻译过来就是说，让一个或多个线程持续等待，直到其他多线程执行的一组操作全部完成以后，这些等待的线程才会继续执行。

(此处需要配图)

就好比是，有多位选手参加一场百米赛跑，裁判员需要等待全部选手就绪，并且在同一起跑线上。然后，裁判会发出号令：“各就位，预备跑”，随着发令枪响，所有选手才能全部起跑。在这个场景中，各位参赛选手就是线程，而裁判就是CountDownLatch。

我们在实际开发中，有以下两个使用场景可以用CountDownLatch来实现：



1）让单个线程等待多个线程的场景。

比如，一个服务需要从多个远程接口获取数据，我们可以创建多个线程来分别调用远程接口，等待所有远程接口都获得返回数据之后，主服务线程再往下继续执行。像并发计算，结果汇总等等。

2）让多个线程等待的场景。

比如，模拟秒杀场景，让一组线程同时等待，同时恢复执行，实现最大程度的并行性。

需要注意的是，当高并发请求时，Countdownlatch的await方法有可能会引起死锁。

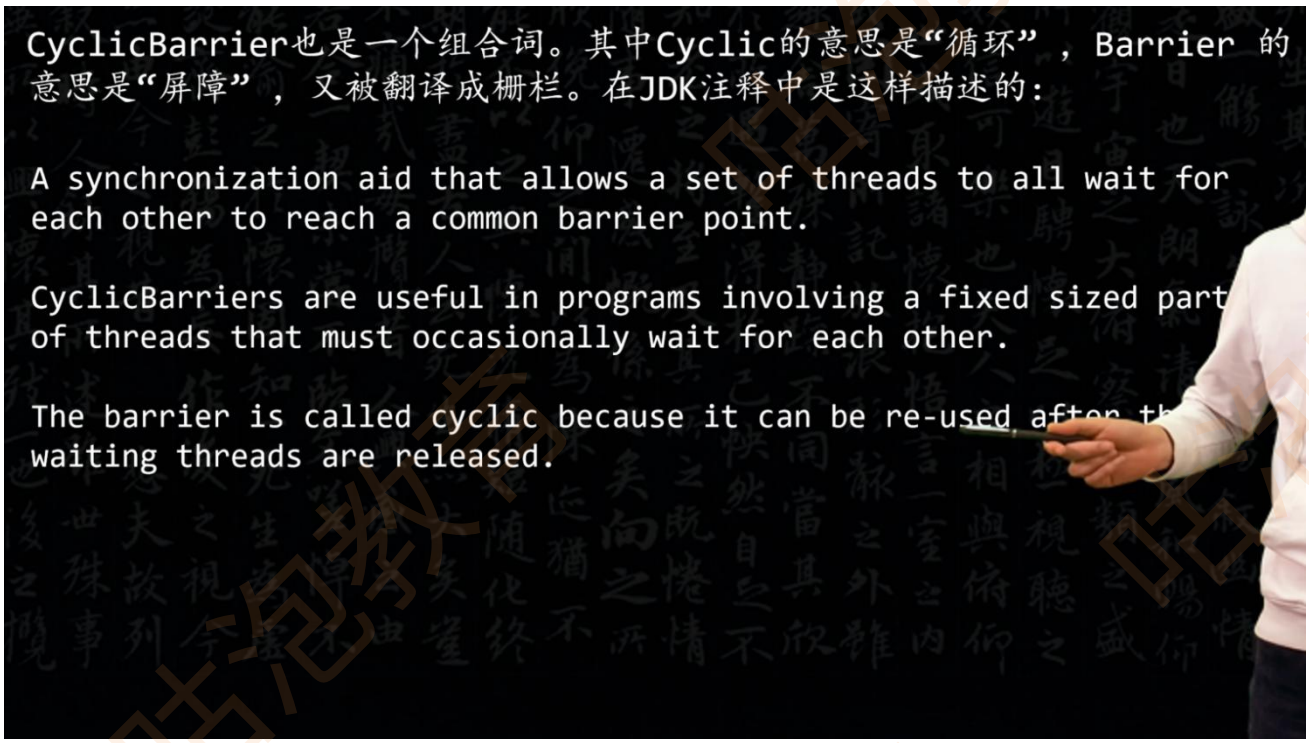
如果线程池中线程的数量较少，在高并发时会出现多个请求占用了全部的线程，但是每个请求又需要await()其他线程，被等待的线程拿不到线程资源无法执行，导致多个请求同时进入线程阻塞，最后形成死锁。

当然，我们可以使用自定义线程池来扩大线程数量，并且建立线程池拒绝机制来解决死锁问题。

【导航条：CyclicBarrier】

（转场，屏幕黑底白字，正中央出现“2 CyclicBarrier）

2、CyclicBarrier

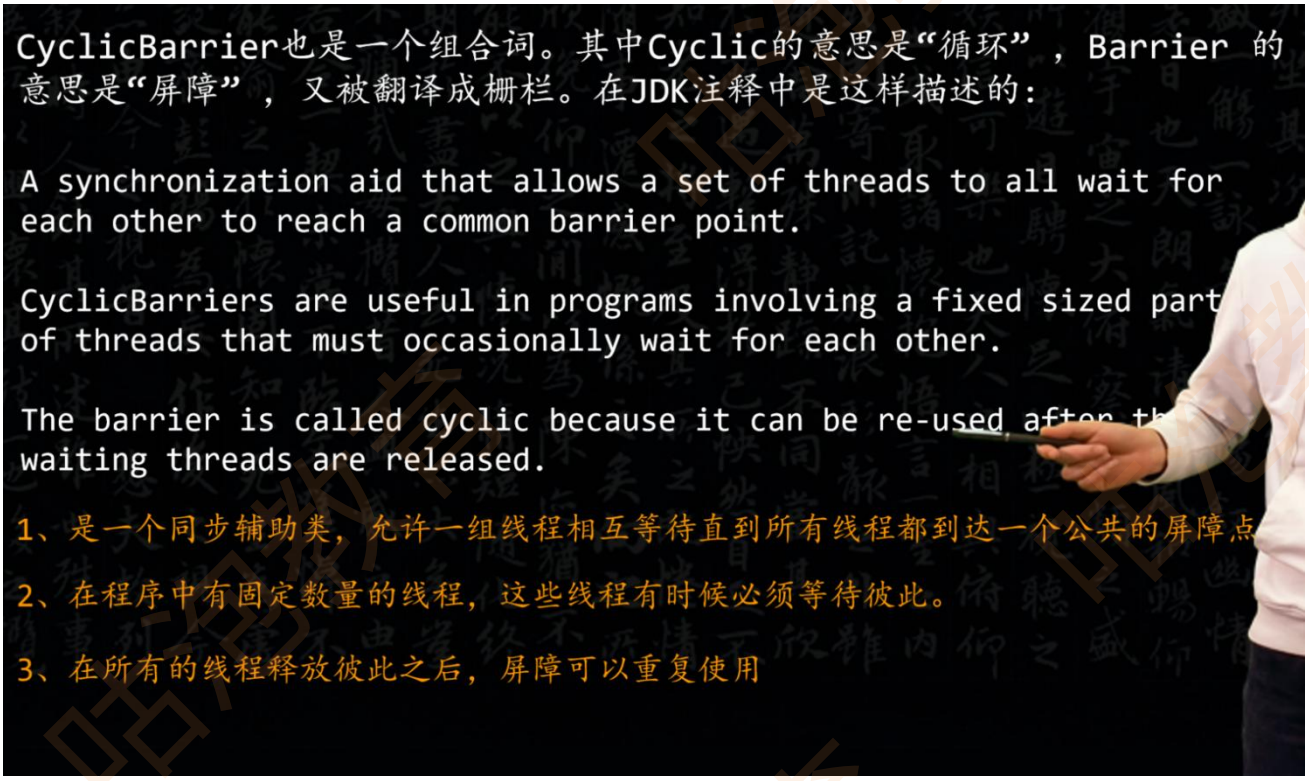


再来看 CyclicBarrier ['saɪklɪk] ['bæriə]，它也是一个组合词。其中Cyclic 的意思是 循环，Barrier 的意思是屏障，又被翻译成栅栏。在JDK 注释中是这样描述的：

A synchronization aid that allows a set of threads to all wait for each other to reach a common barrier point.

CyclicBarriers are useful in programs involving a fixed sized party of threads that must occasionally wait for each other.

The barrier is called cyclic because it can be re-used after the waiting threads are released.

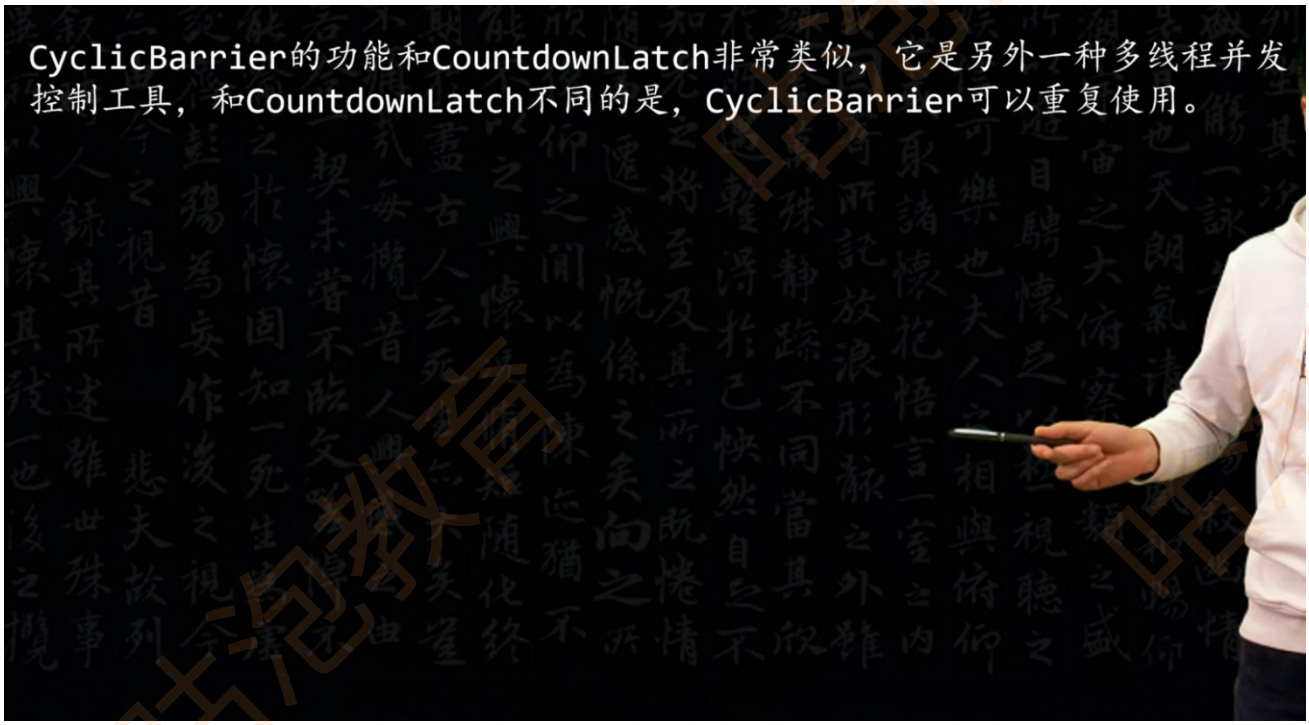


翻译过来总结为以下三点：

1）CyclicBarrier 是一个同步辅助类，它允许一组线程相互等待直到所有线程都到达一个公共的屏障点。

2）在程序中有固定数量的线程，这些线程有时候必须等待彼此，这种情况下，使用 CyclicBarrier 很有帮助。

3）这个屏障之所以用循环修饰，是因为在所有的线程释放彼此之后，这个屏障可以重复使用。



从字面上理解的话，它的功能和CountdownLatch非常类似，也是等待所有参加比赛的选手全部就绪以后，才能开始起跑。它是另外一种多线程并发控制工具，和CountdownLatch不同的是，CyclicBarrier可以重复使用。

我们在实际开发中，CyclicBarrier可以用于多线程计算数据，最后合并计算结果的应用场景。

比如，需要计算N组人一年的平均工资，每组需要多个线程并行计算，计算完一组，再开始下一组，这样就需要多轮并行计算。这个场景下，CyclicBarrier 比 CountDownLatch 更适合。

【导航条：两者区别】

（转场，屏幕黑底白字，正中央出现“3 两者区别）

3、两者区别

最后，总结一下CountDownLatch和CyclicBarrier的区别，从以下四个方面来分析：

1、CountDownLatch的计数器只能使用一次。而CyclicBarrier的计数器可以使用reset() 方法重置。

2、CyclicBarrier能处理更为复杂的业务场景，比如计算发生错误，可以结束阻塞，重置计数器，重新执行程序

3、CyclicBarrier提供getNumberWaiting()方法，可以获得CyclicBarrier阻塞的线程数量，还提供isBroken()方法，可以判断阻塞的线程是否被中断，等等。

4、CountDownLatch会阻塞主线程，CyclicBarrier不会阻塞主线程，只会阻塞子线程。

【结尾】

好了，以上就是我对CountdownLatch 和 CycliBarriar的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

107.谈谈你对OAuth的理解

【导航条：引言】

1位工作5年的小伙伴被问到这样一道面试题，说谈谈你对OAuth的理解。当时，这位小伙伴感觉回答得不是很理想，希望我拍一期视频详细地介绍一下。

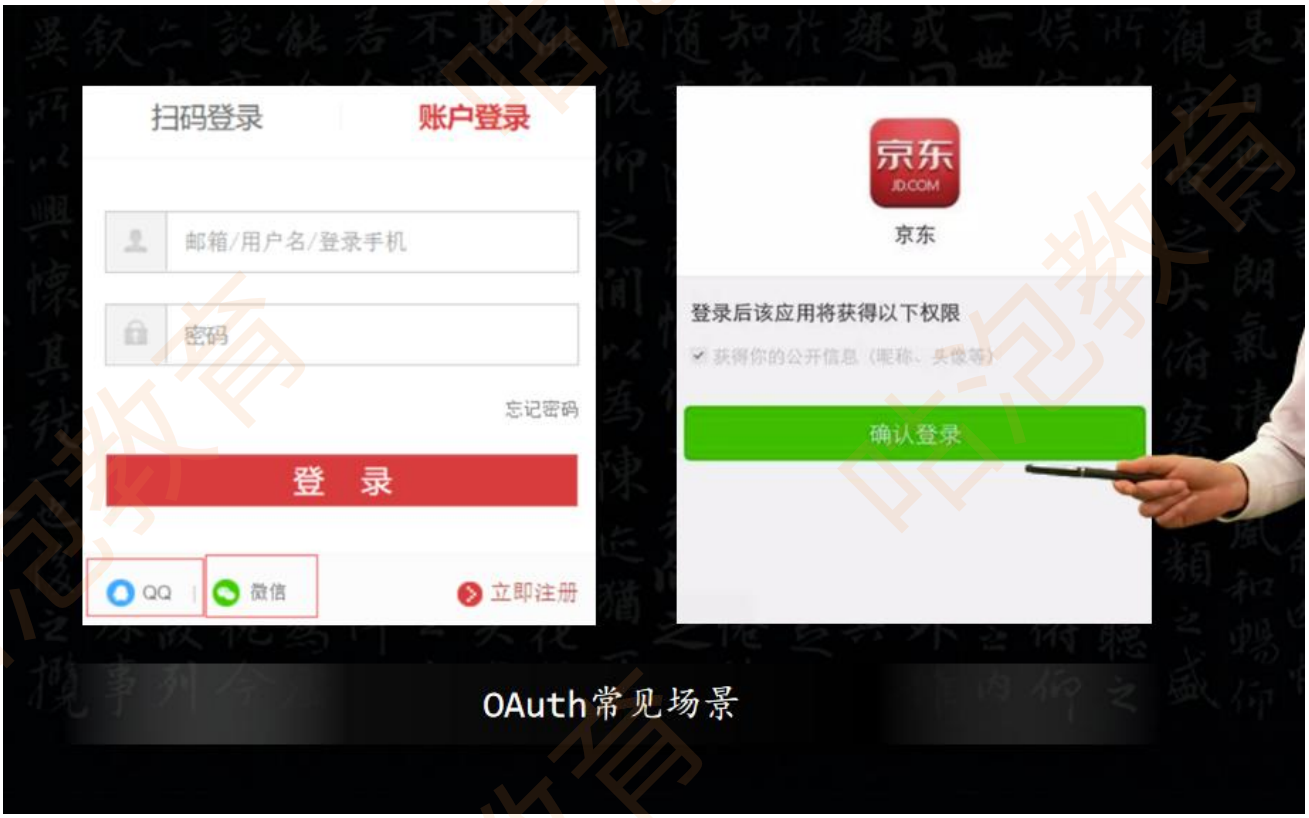
今天，我给大家讲一讲，我对这个问题的理解。

【导航条：什么是 OAuth】

（转场，屏幕黑底白字，正中央出现“1 什么是 OAuth）

1、什么是 OAuth

OAuth是一个关于授权（Authorization）的开放技术标准，在全世界得到广泛应用，它本质上是一种协议，可以在不共享用户用户名和密码的前提下，实现将授权从应用程序另一个应用程序。如图所示：



咱们可以使用QQ或者微信直接登录京东APP，而无需在京东重新注册用户。相当于，用户可以在一个平台上登录，获得授权后，也可在另一平台上执行操作和查看数据。使用最为广泛的场景是SSO（单点登录）。

（此处需要配图）

举个例子，有一位访客需要在房主不在家时进到房间里面，房主又不能把门锁密码告诉访客，而是给预先发给访客一张临时房卡，这张房卡刷一次就失效了。OAuth 的工作方式和这个场景非常类似，一个应用程序向另一个应用发送授权令牌来授予用户访问权限，而不是发送用户的密码。

【导航条：工作原理】

（转场，屏幕黑底白字，正中央出现“2 OAuth的工作原理）

2、OAuth的工作原理

我们知道，任何的身份认证，本质上都是基于对请求方的不信任产生的。而OAuth的出现，主要是解决多个应用之间的授权信任问题。在OAuth的交互过程中有四个参与角色，如图所示：

它们分别是：

1、资源所有者（Resource Owner）：一般是指咱们自己

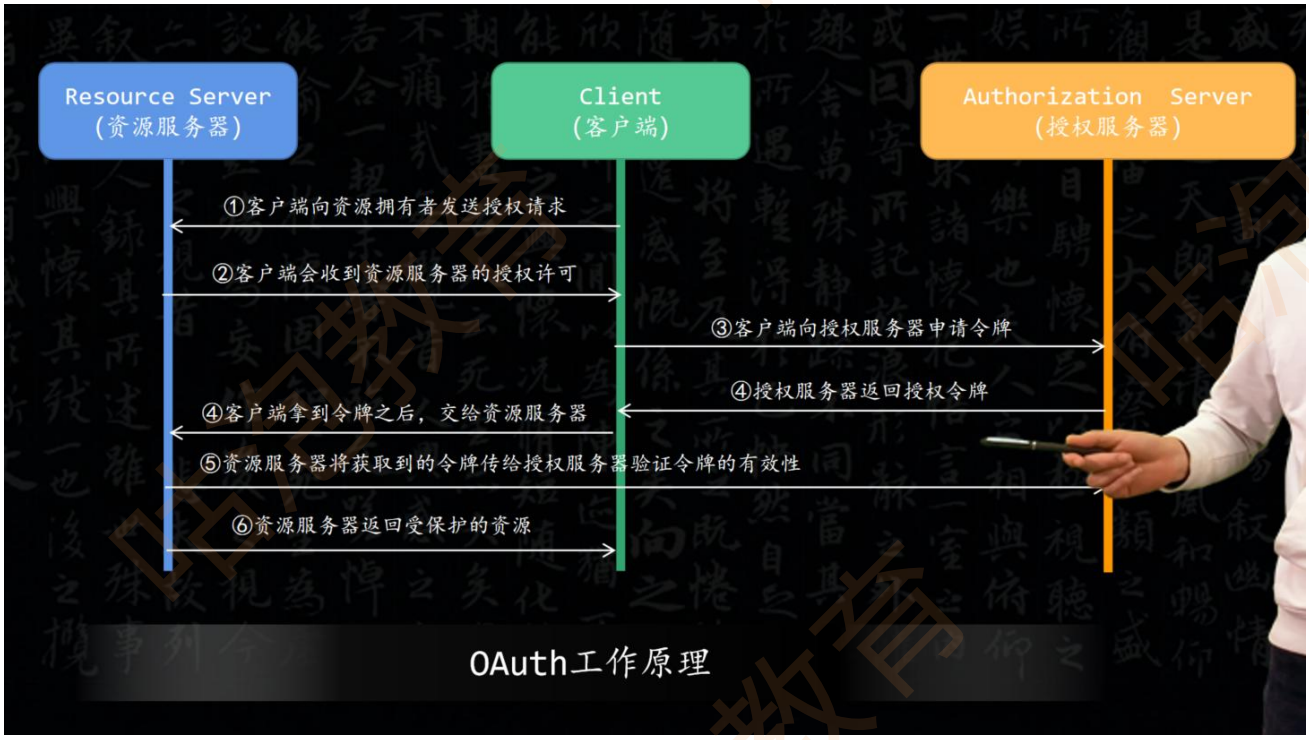
2、客户端（Client）：一般是指需要授权的应用，比如京东APP

3、资源服务器（Resource Server）：一般是指保存信息的服务器，比如QQ密码和微信密码的服务器。

4、授权服务器（Authorization Server）：一般是指提API的服务器，比如QQ的Open API服务，微信的Open API

服务。

一般情况下，资源服务器和认证服务器是同一个服务，这样更方便调用。OAuth的工作原理如图所示：



首先，第1步，客户端向资源拥有者发送授权请求，一般资源拥有者的资源会存放在资源服务器。

第2步，客户端会收到资源服务器的授权许可，

第3步：客户端拿到许可之后，再向授权服务器发送一次验证，给客户端颁发一个Access Token访问令牌。

第4步：客户端拿到令牌之后，交给资源服务器。

第5步：资源服务器会将获取到的令牌传给认证服务器验证令牌的有效性。

第6步：资源服务器验证令牌通过之后，就会返回一个受保护的资源。

在我们看到的这个流程中，最重要的是第2步，也就是在第3步获取授权之前，客户端要先申请许可资源的内容，比如用户头像，用户昵称等等。也就是客户端向资源服务器申请授权的时候，需要填写以下授权所需的信息，分别是：

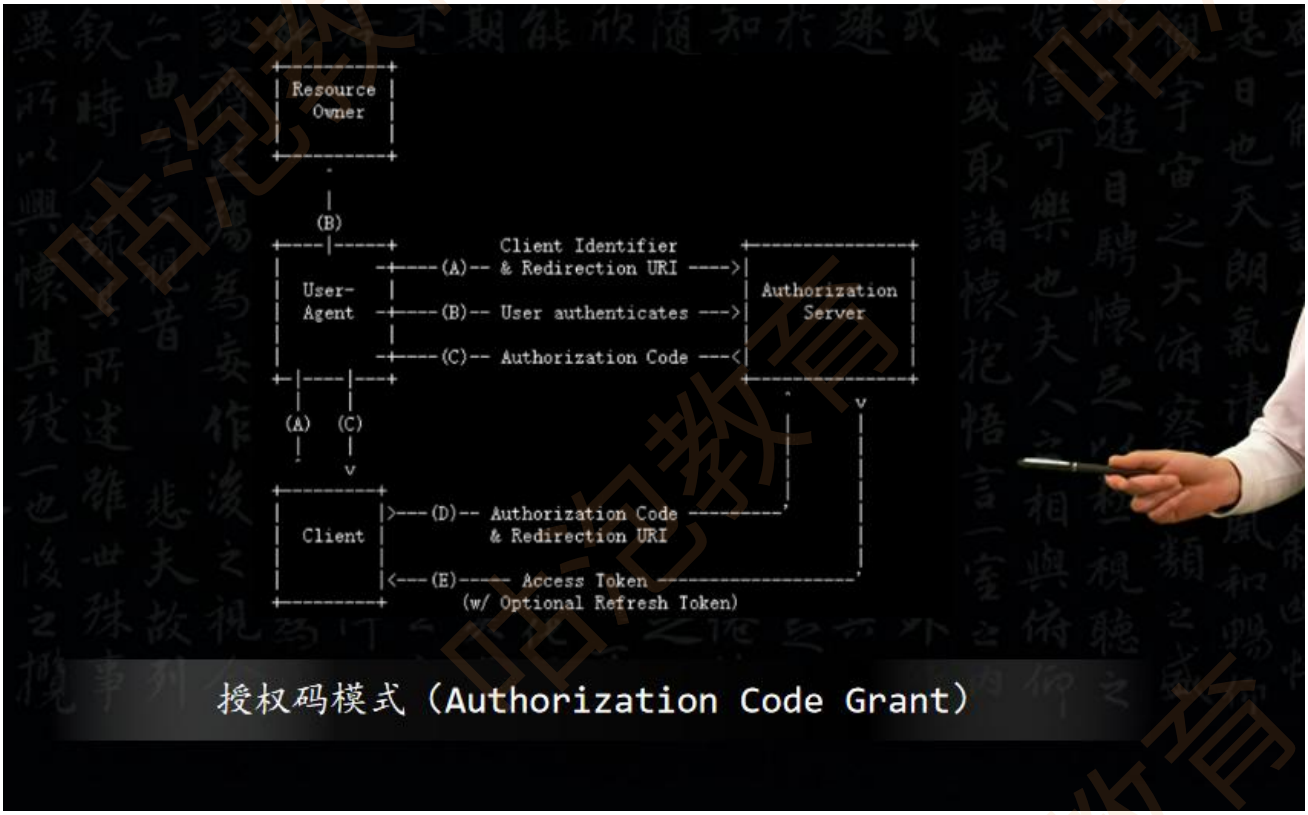
学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

应用名称、应用网站、重定向URI或者回调的URL（redirect_uri）、客户端标识client_id和客户端密钥client_secret。

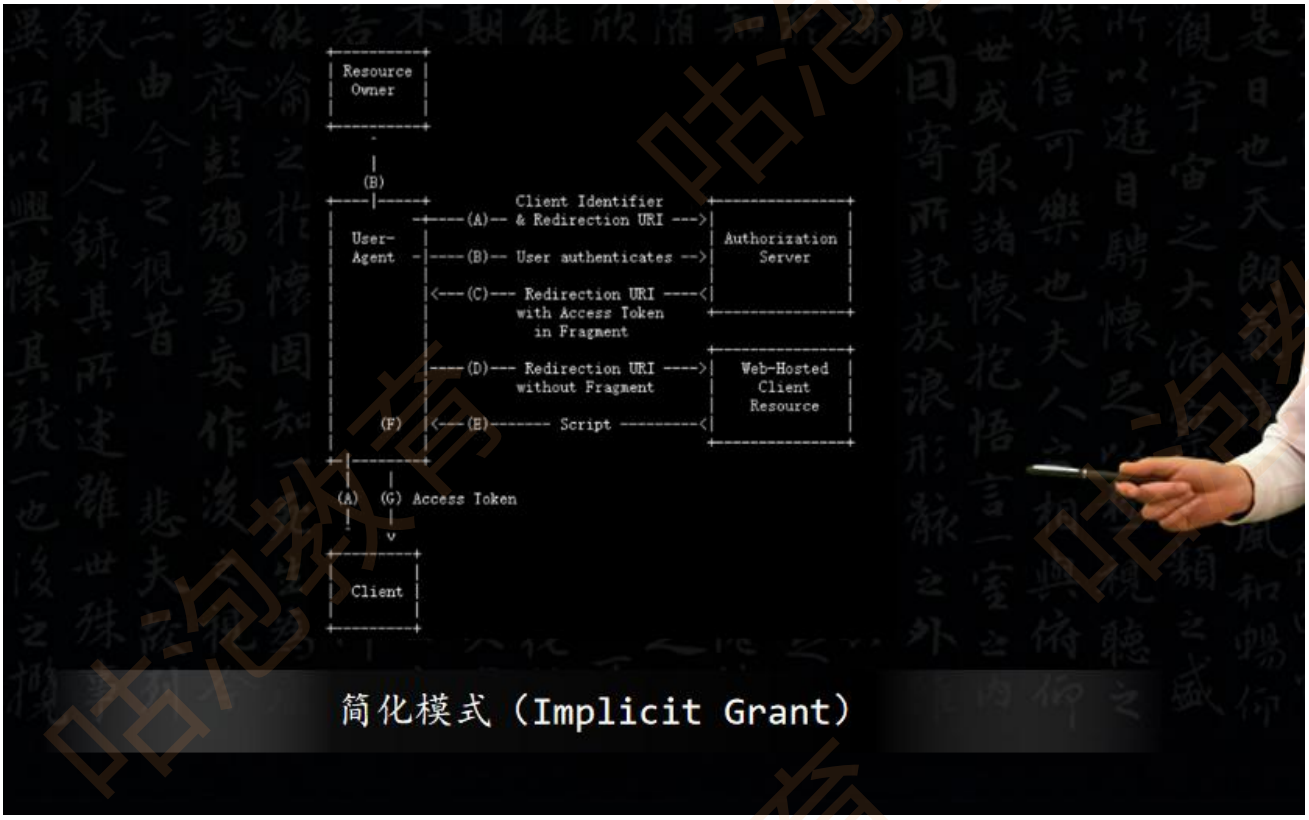
这些信息需要我们在 。

当然，OAuth一共设计了四种授权模式，分别是：

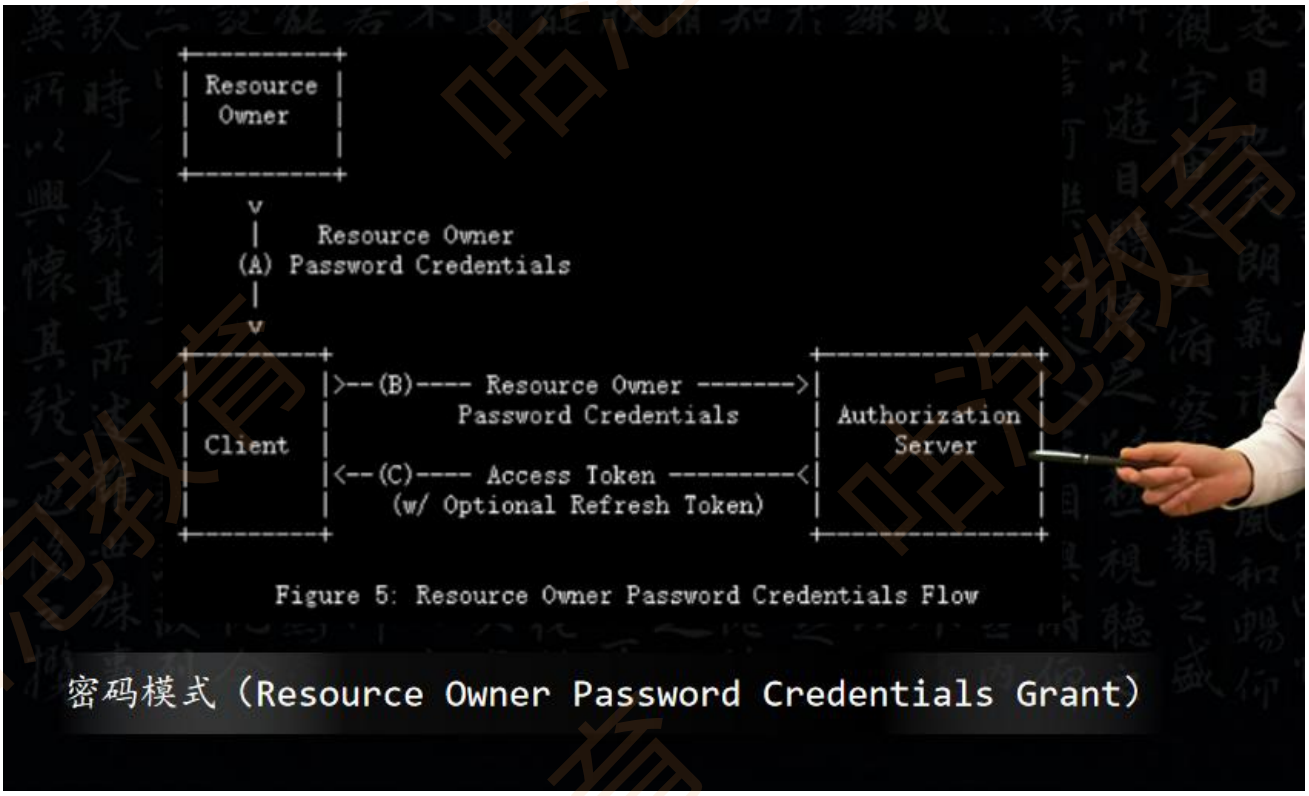
授权码模式（Authorization Code Grant）、



简化模式（Implicit Grant）、

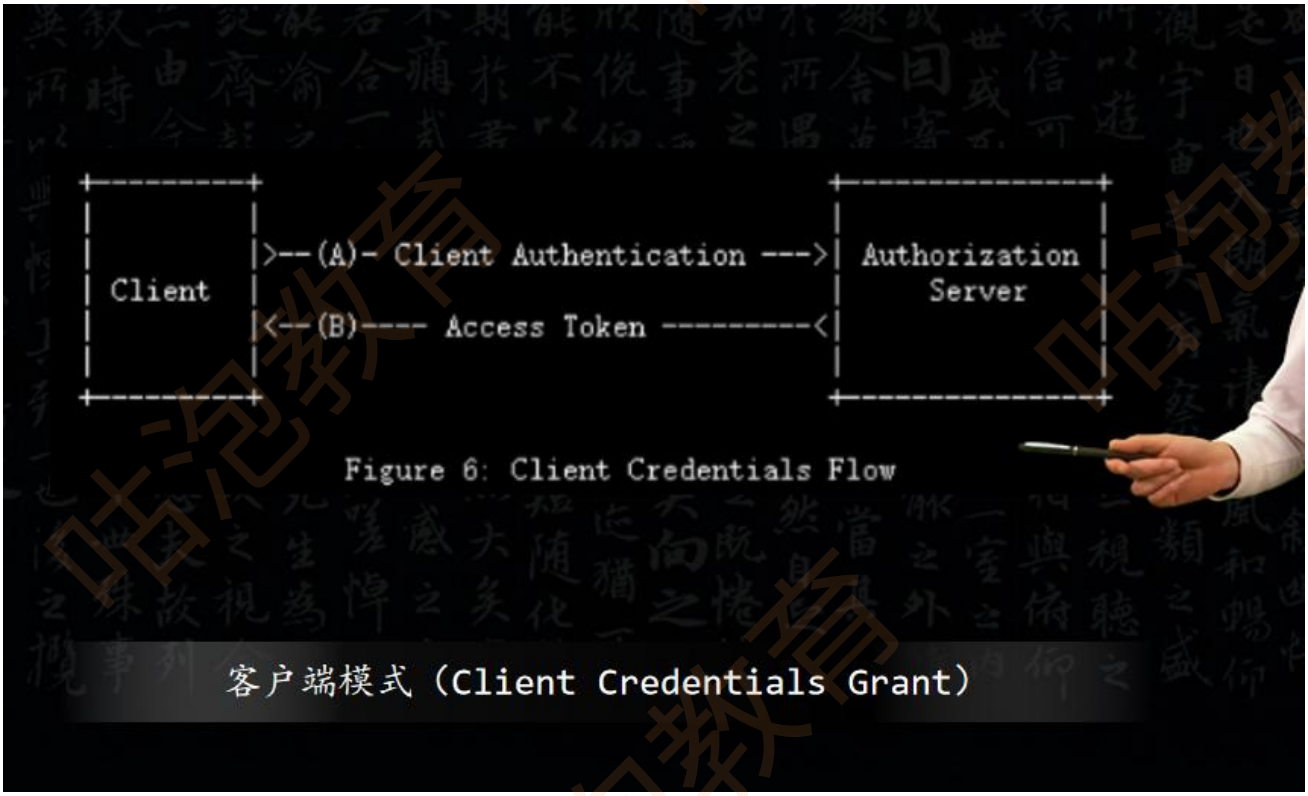


密码模式（Resource Owner Password Credentials Grant）、



学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

客户端模式（Client Credentials Grant），

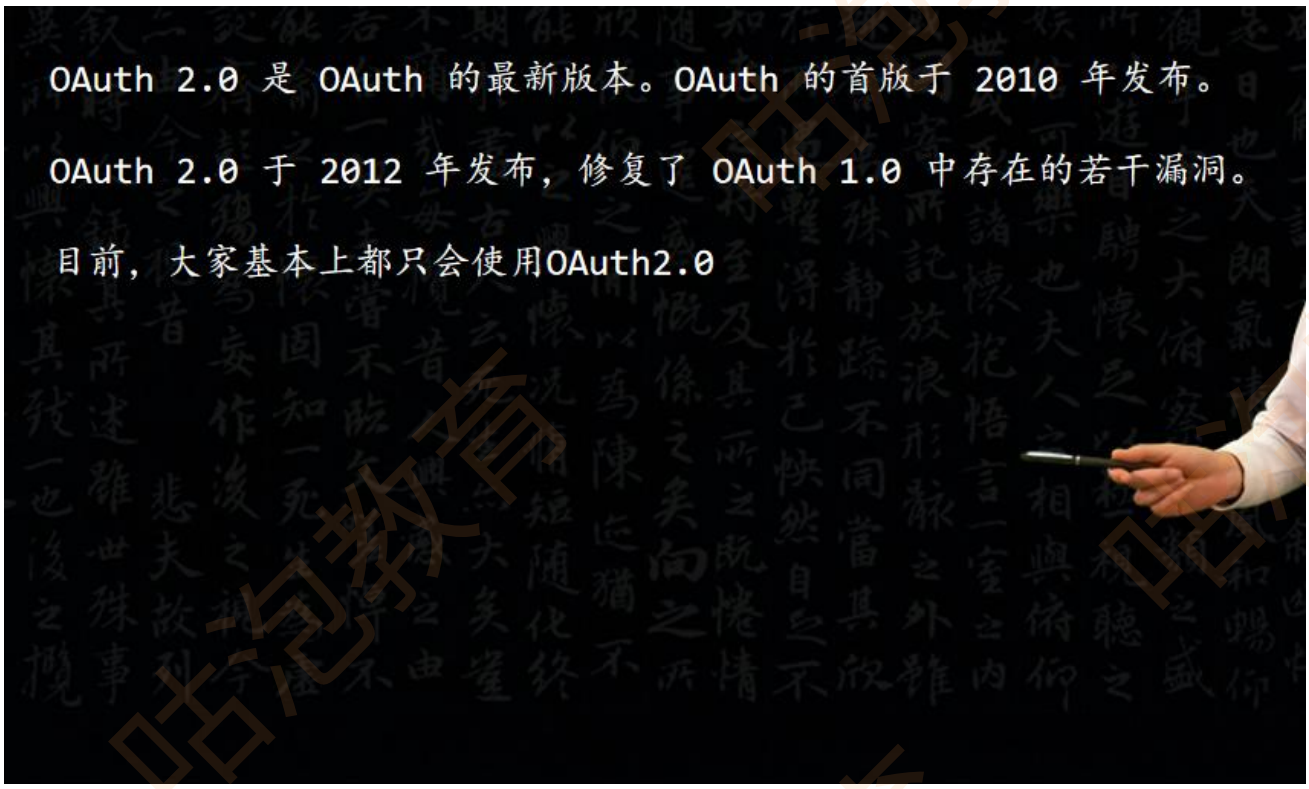


由于视频时长的限制，我这里就不详细介绍了。有兴趣的小伙伴可以在评论区回复666，我可以单独再拍摄一期视频专门讲解。

【导航条：关于OAuth 2.0】

（转场，屏幕黑底白字，正中央出现“3 关于OAuth 2.0）

3、关于OAuth 2.0



OAuth 2.0 是 OAuth 的最新版本。OAuth 的首版于 2010 年发布。OAuth 2.0 于 2012 年发布，修复了 OAuth 1.0 中存在的若干漏洞。目前，大家基本上都只会使用OAuth2.0了。

【结尾】

好了，以上就是我对OAuth的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

108.为什么启动线程不直接调用run()，而要调用start()，如果调用两次start()方法会有什么后果?

【导航条：引言】

1位工作6年的小伙伴去某里P6一面，被问到这样一道面试题，说，为什么启动一个线程不直接调用run()方法，而要调用start()方法来启动，如果调用两次start()会有什么后果？

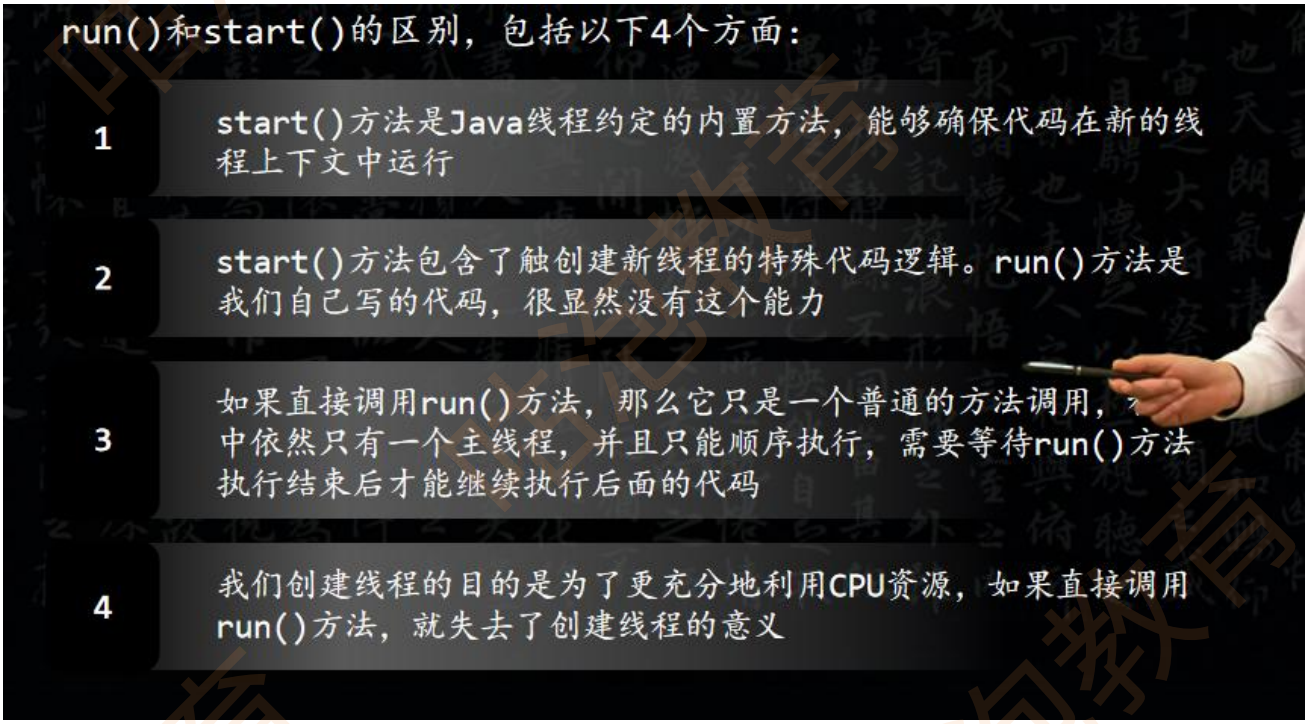
如果，你被问到这样一道面试，你能回答出来吗？今天，我给大家来分享一下，我对这个问题的理解。

【导航条：run()和start()的区别】

（转场，屏幕黑底白字，正中央出现“1 run()和start()的区别）

1、run()和start()的区别

首先回答为什么启动线程不能直接调用run()方法，而要调用start()方法，我从以下4个方面来回答：



第1：start()方法是Java线程约定的内置方法，能够确保代码在新的线程上下文中运行。

第2：start()方法包含了触创建新线程的特殊代码逻辑。run()方法是我们自己写的代码，很显然没有这个能力。

第3：如果直接调用run()方法，那么它只是一个普通的方法调用，程序中依然只有一个主线程，并且只能顺序执行，需要等待run()方法执行结束后才能继续执行后面的代码。

第4：我们创建线程的目的是为了更充分地利用CPU资源，如果直接调用run()方法，就失去了创建线程的意义了。

【导航条：调用两次start()的后果】

（转场，屏幕黑底白字，正中央出现“2 调用两次start()的后果）

2、调用两次start()的后果

了解了run()方法和start()方法的区别，那如果调用两次start()方法会有什么后果呢？在Java中，线程的start()方法只能被调用一次，如果第二次调用会抛出 IllegalThreadStateException，这是一种运行时异常，多次调用 start 被认为是编程错误。

在Java中，线程的运行状态被定义成了5个枚举值，分别是：

1、新建（NEW），线程已经创建好了，但是还没有调用start()方法启动。

2、就绪（RUNNABLE），这个状态下的线程可能正在运行，也可能还在就绪队列里面，等待系统分配

CPU资源。

在操作系统中，会额外区分一种状态叫做RUNNING，但是从 Java API 的角度，并不能表示出来。关于这个问题，我在前面的视频中，有讲过关于《线程状态流转原理》有兴趣的小伙伴可以去我的主页找到。

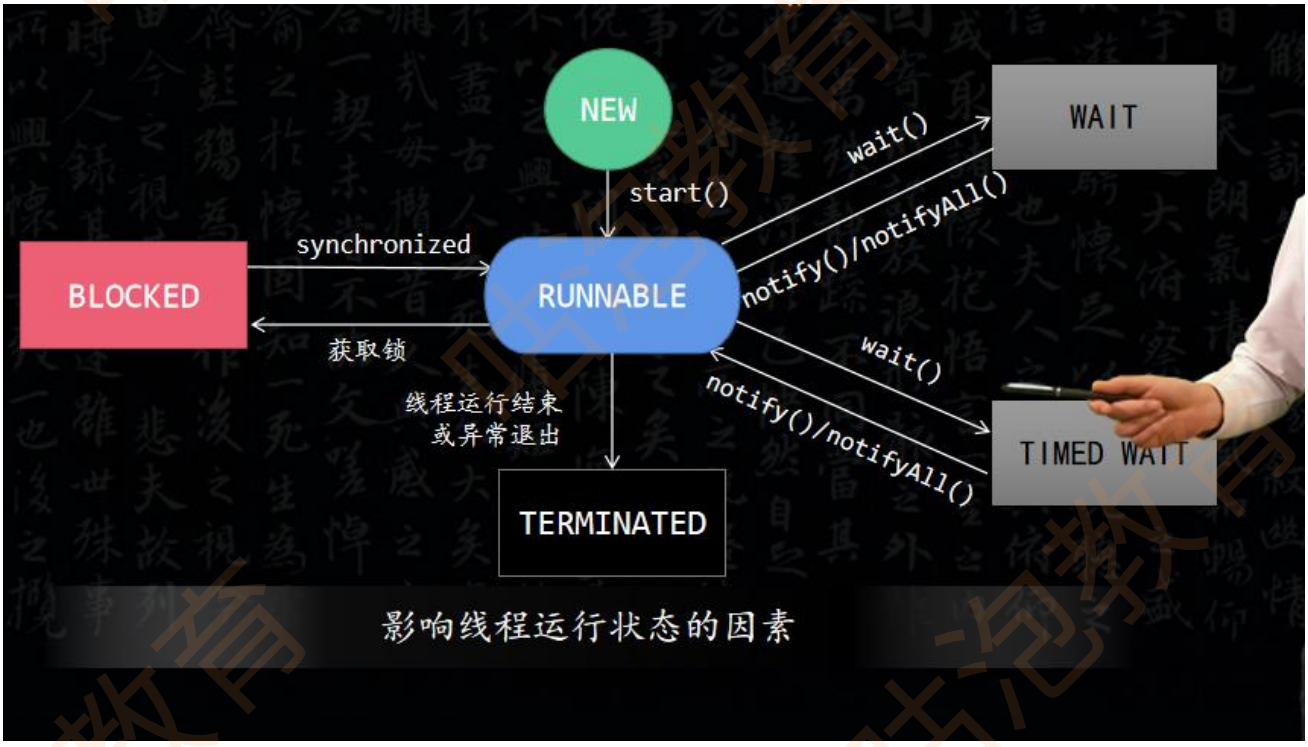
3、阻塞（BLOCKED），表示线程处于等待Monitor Lock的状态。

4、等待（WAITING），表示线程处于条件等待状态，当触发条件后会唤醒。比如wait/notify等。

5、计时等待（TIMED_WAIT），它和WAITING状态是一样的，只是多了一个超时条件触发机制。

6、终止（TERMINATED），表示线程执行结束。

在Java API 中，影响线程运行状态的因素，如图所示：



在我们第一次调用 start() 方法的时候，线程可能处于终止或者其他非 NEW的状态，再次调用start()方法的时候，相当于让这个正在运行的线程重新运行一遍。不管是从线程安全的角度来看，还是从线程本身的执行逻辑来看，它都是不合理的。

因此，为了避免这个问题出现，Java会先去判断当前线程的运行状态。

【结尾】

好了，以上就是我对Java线程的start()方法和run()方法的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

109.谈谈你对Netty中Pipeline工作原理的理解

【导航条：引言】

1位工作8年的小伙伴，去某东面试IM部门，被问到这样一道面试题。说，请你谈一谈你对Netty Pipeline设计原理的理解。当时，他说只是用过Netty的Pipline，原理没有深入了解过，然后就没有然后了。

今天，我给大家讲一讲，我对Netty Pipeline的理解。

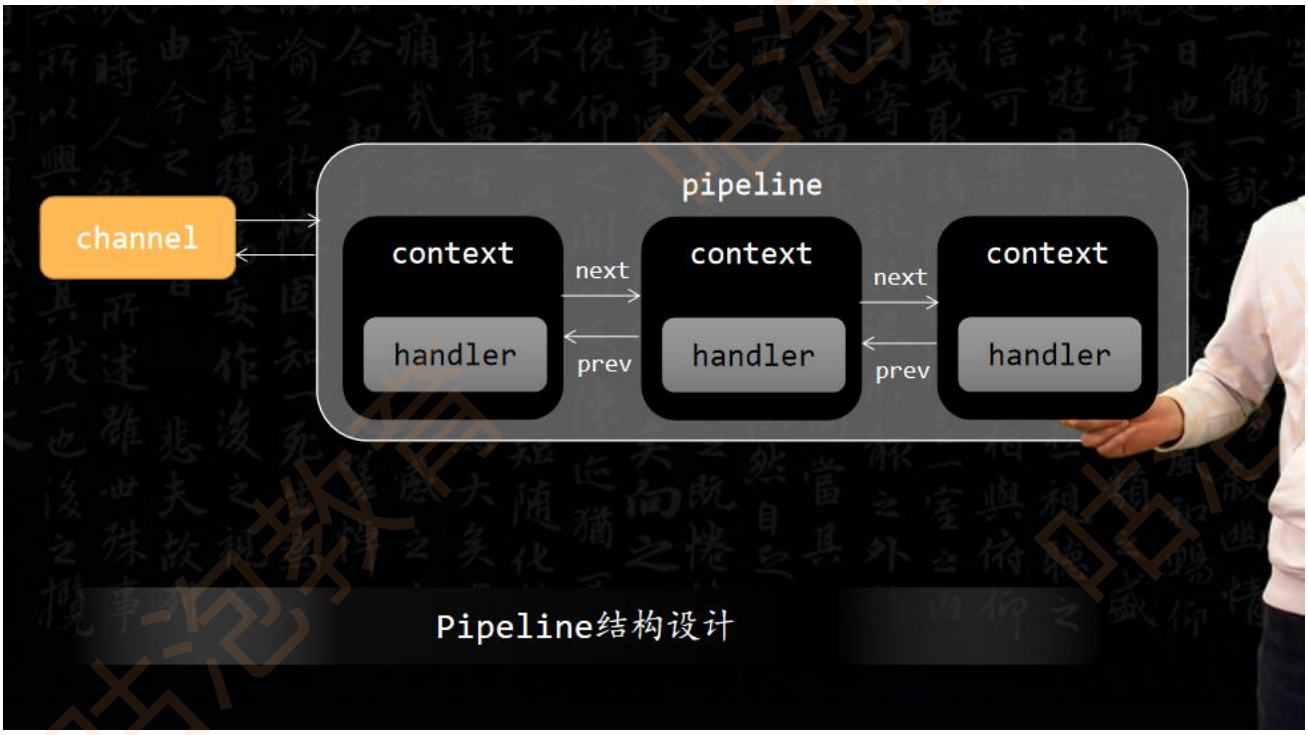
另外，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的面试通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

【导航条：结构设计】

（转场，屏幕黑底白字，正中央出现“1 Pipeline的结构设计）

1、结构设计

Netty中的Pipeline本质上是一个双向链表，它采用了责任链模式。在Netty中每个Channel都有且仅有一个ChannelPipeline与之对应，它们的组成关系如下图所示。



通过上图可以看到，一个Channel包含了一个ChannelPipeline，而ChannelPipeline中又维护了一个由ChannelHandlerContext组成的双向链表。这个链表的头叫HeadContext，链表的尾叫TailContext，并且每个ChannelHandlerContext中又关联着一个ChannelHandler。

【导航条：工作原理】

（转场，屏幕黑底白字，正中央出现“2 Pipeline的工作原理）

2、工作原理

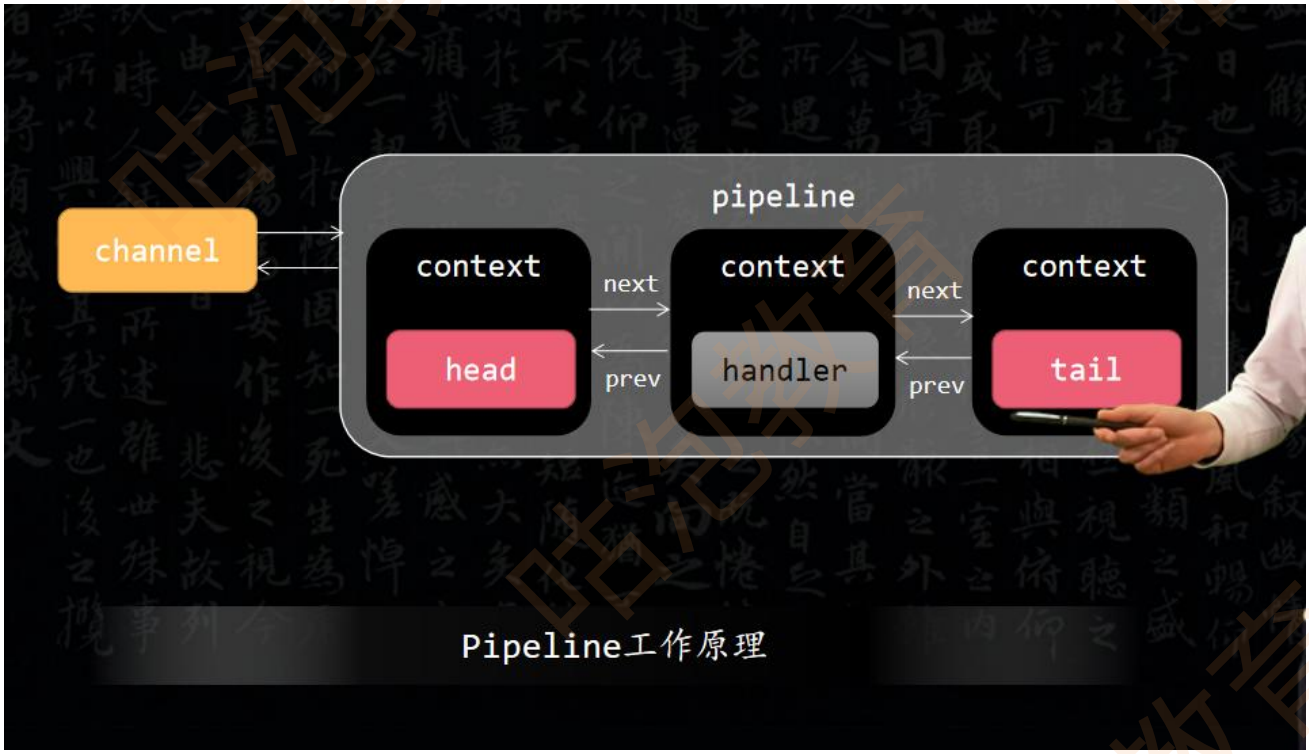
首先来看这样一段代码：

```
Bootstrap client = new Bootstrap();
      client.channel(NioSocketChannel.class)
      .handler(new ChannelInitializer() {
         @Override
         protected void initChannel(Channel ch) throws Exception {
            //接收课客户端请求的处理流程
            ChannelPipeline pipeline = ch.pipeline();
            //对象编码器
            pipeline.addLast("encoder",new ObjectEncoder());
           //对象解码器
```

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果


```
        pipeline.addLast("encoder",new ObjectDecoder());

    }

});
```

对于用过Netty的小伙伴来说，应该非常熟悉。在Netty中，Pipeline的初始化，是通过调用Channel的handler()方法,然后在handler()方法中传入一个叫做ChannelInitializer的对象，通过SocketChannel构建出一个新的Pipeline对象。每次调用addLast()方法，都会在Pipelie的末端插入一个ChannelHandlerContext。如图所示：



每个Context中又会包含一个ChannelHandler，我们通过addLast()方法往Pipeline中添加的对象，并且Handler的添加顺序会影响代码的执行顺序。而这些Handler本质上都是实现编码和解码的功能，不管是编码器还是解码器都必须实现ChannelHandler接口。

图中的Handler就是我们代码程序要执行的逻辑。而Netty默认帮我们实现了非常多内置Handler，我们只需要直接拿过来用就可以了。当然，我们也可以自己实现ChannelHandler接口，来实现自定义的编、码器。比如自定义

通信协议等等。

当所有的Handler全部添加到Pipeline中以后，Netty就会将这些Handler组装成一个双向链表，从而实现串行化调用。从头部往尾部执行的Handler被称为Inbound，用来接收用户请求，从尾部往头部执行的Handler被称为Outbound，用来给用户响应。所以，Inbound可以用来实现解码的功能、而Outbound可以用来实现编码的功能。

【结尾】

好了，以上就是我对Pipeline设计原理的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

110.HashMap和TreeMap的区别

【导航条：引言】

前几天，有一位粉丝在直播间问了我这样一个问题，说HashMap和TreeMap有什么区别。今天，我给大家分享一下我的理解。

【导航条：两者区别】

（转场，屏幕黑底白字，正中央出现“1 两者区别）

1、两者区别

我们知道不管是HashMap还是TreeMap，都是通过对象来对对象进行索引的Map集合。我们把用来索引的对象叫做Key，而索引对应的对象叫做Value。这就是我们平时说的键值对。它们的类关系如图所示：

关于HashMap和TreeMap的区别，我从以下4个方面来分析：

1）数据结构方面

HashMap是基于哈希表+数组来实现的，而TreeMap是基于红黑树实现的。

使用HashMap需要键对象明确定义了hashCode()和equals()这两个方法，而且为了优化HashMap空间的使用，可以调整初始容量大小和扩容。

TreeMap没有大小设置选项，因为，红黑树结构总是处于平衡状态。

2）效率方面

HashMap比TreeMap的性能更高。

HashMap的时间复杂度是O（1），它是通过哈希函数计算的哈希地址。

而TreeMap主要是保证数据平衡，时间复杂度是O（log2 n）。

3）线程安全方面

HashMap和TreeMap都是非线程安全的。

如果在多线程并发情况下建议使用ConcurrentHashMap；

如果既要保证线程安全又要保证顺序，可以使用 Collections.synchronizedMap()方法转化为线程安全的集合。

4）应用场景方面

HashMap是无序的，而TreeMap是有序的。

TreeMap适用于按自然顺序或自定义顺序遍历键的场景。

HashMap适用于在Map中插入、删除和定位元素。

日常开发建议多使用HashMap，只有在需要排序的时候才使用TreeMap。

【导航条：总结】

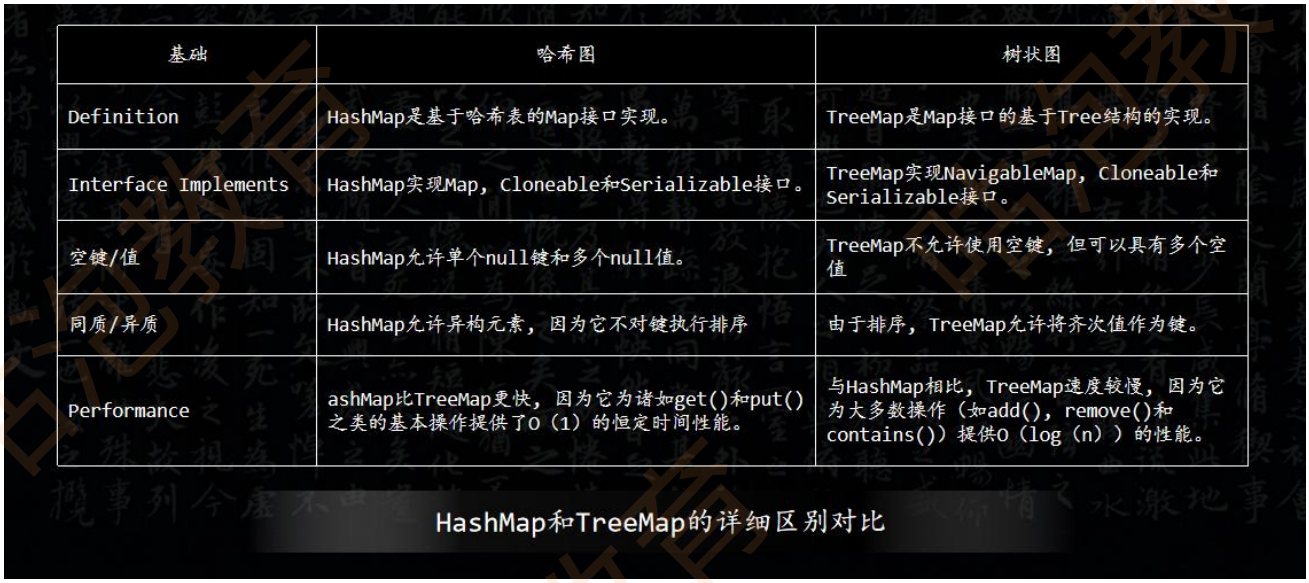
（转场，屏幕黑底白字，正中央出现“2 总结）

2、总结

最后，我把HashMap和TreeMap的更多详细区别，都整理在这张表中了，需要的小伙伴可以在我的个人主页中获取。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

基础	哈希图	树状图
Definition	HashMap是基于哈希表的Map接口实现。	TreeMap是Map接口的基于Tree结构的实现。
Interface Implements	HashMap实现Map, Cloneable和Serializable接口。	TreeMap实现NavigableMap, Cloneable和Serializable接口。
空键/值	HashMap允许单个null键和多个null值。	TreeMap不允许使用空键, 但可以具有多个空值。
同质/异质	HashMap允许异构元素, 因为它不对键执行排序。	由于排序, TreeMap允许将齐次值作为键。
Performance	HashMap比TreeMap更快, 因为它为诸如get()和put()之类的基本操作提供了O（1）的恒定时间性能。	与HashMap相比, TreeMap速度较慢, 因为它为大多数操作（如add(), remove()和contains()）提供O（log（n））的性能。
数据结构	HashMap类使用哈希表。	TreeMap在内部使用Red-Black树, 这是一种自平衡二进制搜索树。
Comparison Method	它使用Object类的equals()方法比较键。Map类的equals()方法将其覆盖。	它使用compareTo()方法比较键。
Functionality	HashMap类仅包含诸如get(), put(), KeySet()等基本功能。	TreeMap类具有丰富的功能, 因为它包含如下功能：tailMap(), firstKey(), lastKey(), pollFirstEntry(), pollLastEntry()。
元素顺序	HashMap不维护任何顺序。	元素以自然顺序（升序）排序。
Uses	当我们不需要按排序顺序的键值对时, 应使用HashMap。	当我们需要按排序（升序）顺序的键值对时, 应使用TreeMap



基础	哈希图	树状图
数据结构	HashMap类使用哈希表。	TreeMap在内部使用Red-Black树，这是一种自平衡二进制搜索树。
Functionality	HashMap类仅包含诸如get()，put()，KeySet()等基本功能。	TreeMap类具有丰富的功能，因为它包含如下功能：tailMap()，firstKey()，lastKey()，pollFirstEntry()，pollLastEntry()。
元素顺序	HashMap不维护任何顺序。	元素以自然顺序（升序）排序。
Uses	当我们不需要按排序顺序的键值对时，应使用HashMap。	当我们需要按排序（升序）顺序的键值对时，应使用TreeMap
Comparison Method	它使用Object类的equals()方法比较键。 Map类的equals()方法将其覆盖。	它使用compareTo()方法比较键。

HashMap和TreeMap的详细区别对比（续表）

【结尾】

好了，以上就是我对HashMap和TreeMap的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

111.在DCL单例写法中，为什么要做两次检查?

【导航条：引言】

有位工作5年的小伙伴，面试的时候被问到这样一道题，说在DCL单例写法中，为什么要做两次检查。要回答好这个问题，需要知道DCL单例的写法以及为什么要这样写?

今天，我给大家详细分析一下。

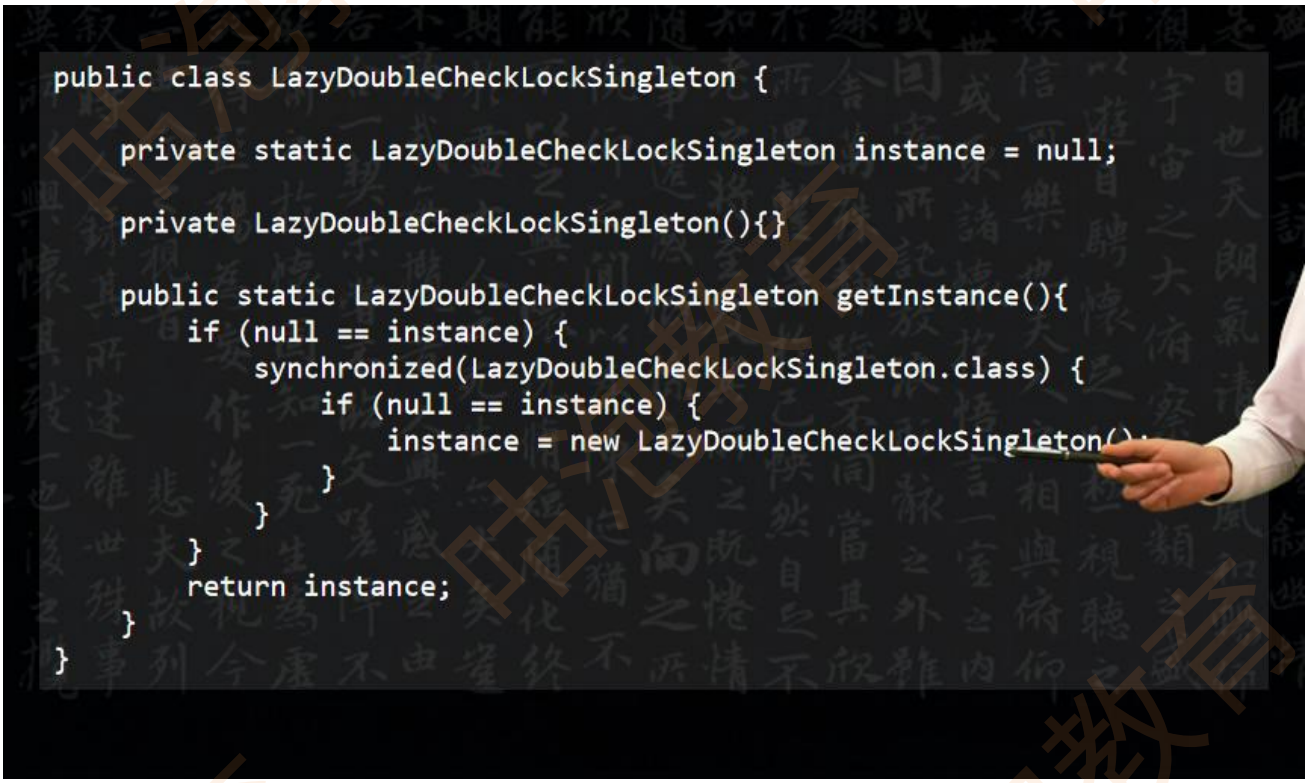
【导航条：什么是DCL】

（转场，屏幕黑底白字，正中央出现“1 什么是DCL）

1、什么是DCL

DCL是一种单例模式写法的简称，全称是Double Check Lock，翻译过来叫双重检查锁。从命名上来理解，就是两次检查加一把锁。那么，两次检查又是检查什么，锁又是锁的什么？

首先，来看这样一段代码，这是比较标准的DCL单例写法。




```
public class LazyDoubleCheckLockSingleton {


    private static LazyDoubleCheckLockSingleton instance = null;
```

```
    private LazyDoubleCheckLockSingleton(){}



    public static LazyDoubleCheckLockSingleton getInstance(){



        if (null == instance) {

            synchronized(LazyDoubleCheckLockSingleton.class) {

                if (null == instance) {

                    instance = new LazyDoubleCheckLockSingleton();

                }

            }



        }



        return instance;

    }



}
```

从代码中，我们发现两次检查的判断条件都是 null == instance，而且两个检查条件是嵌套的。在第1次检查条件的代码块中，加了一段synchronized代码块，synchronized就是锁。

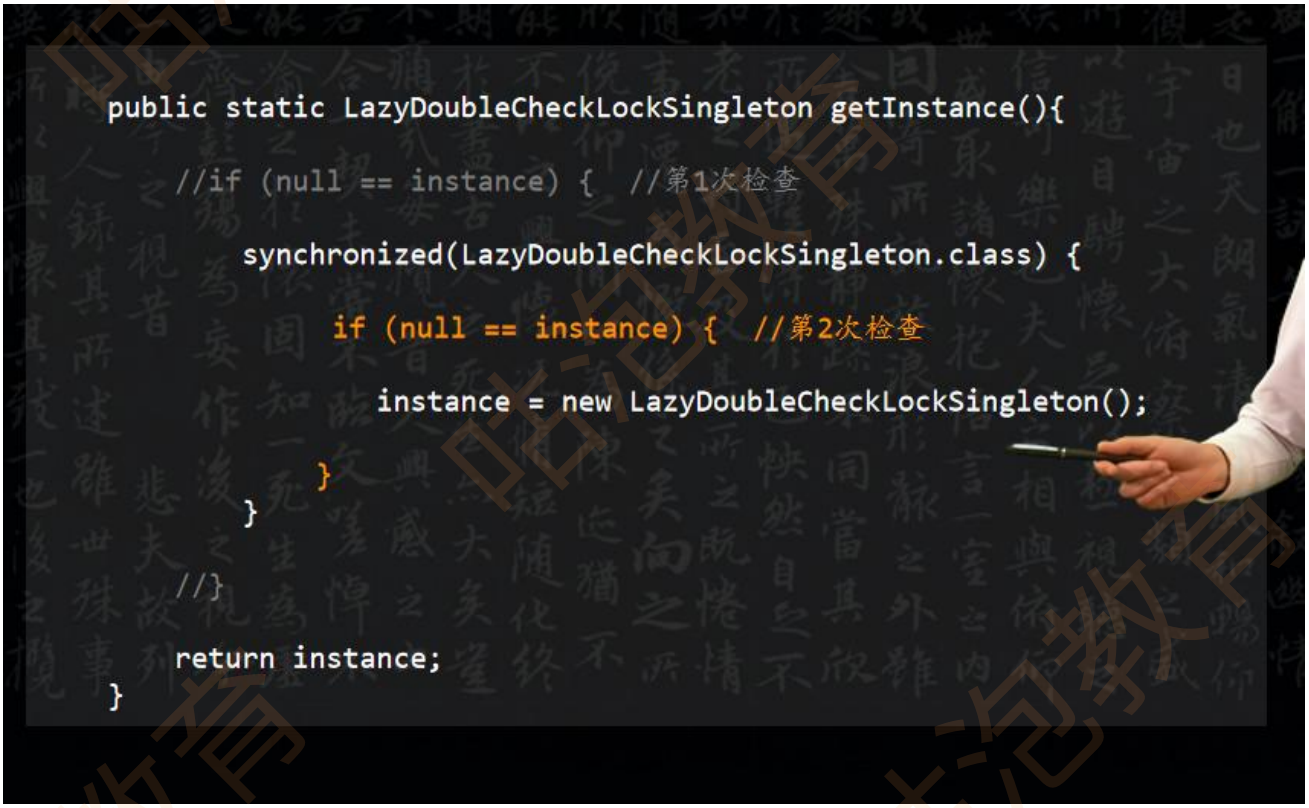
小伙伴们应该都知道，加锁是为了保证线程安全，检查是为了保证内存中只有一个实例。既然，这两个条件是嵌套的，那是不是可以去掉一个条件呢？下面，我们来详细分析一下。

【导航条：两次检查】

（转场，屏幕黑底白字，正中央出现“2 为什么需要两次检查）

2、为什么需要两次检查

我们重点来看这段代码，



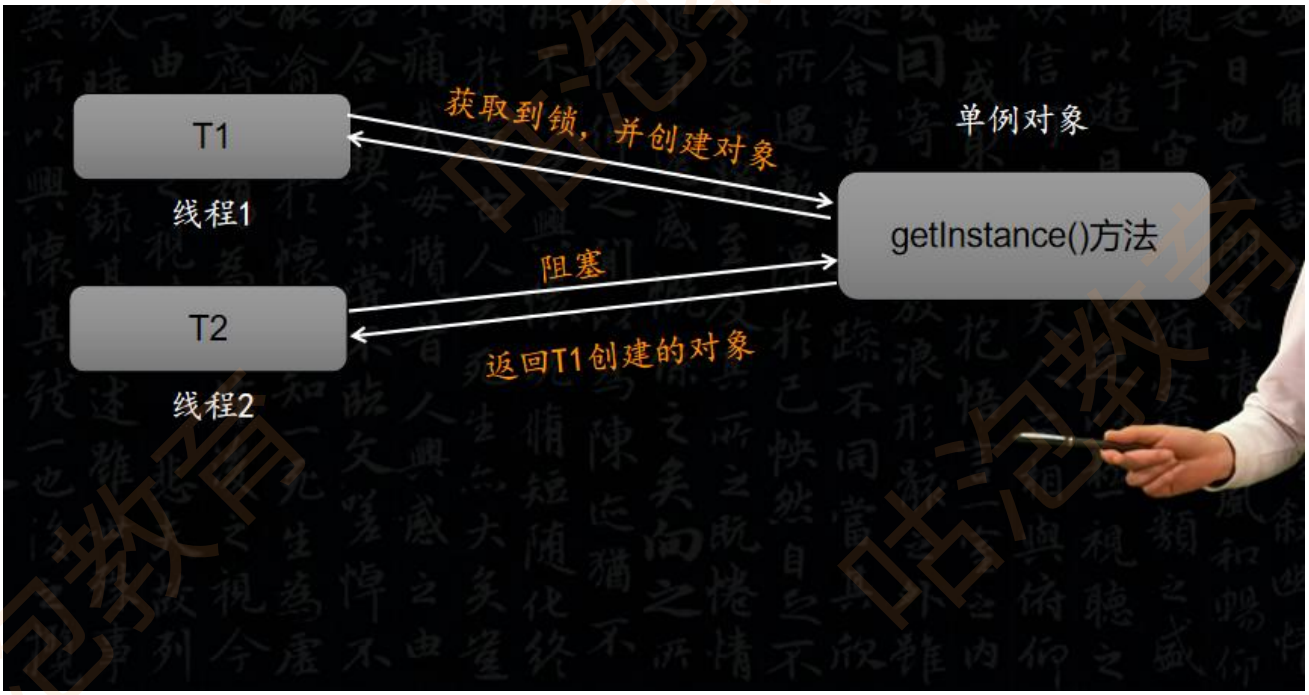

```
public static LazyDoubleCheckLockSingleton getInstance(){

    //if (null == instance) {

        synchronized(LazyDoubleCheckLockSingleton.class) {
```

```
            if (null == instance) {
                instance = new LazyDoubleCheckLockSingleton();
            }
        }

    //}

    return instance;
}
```

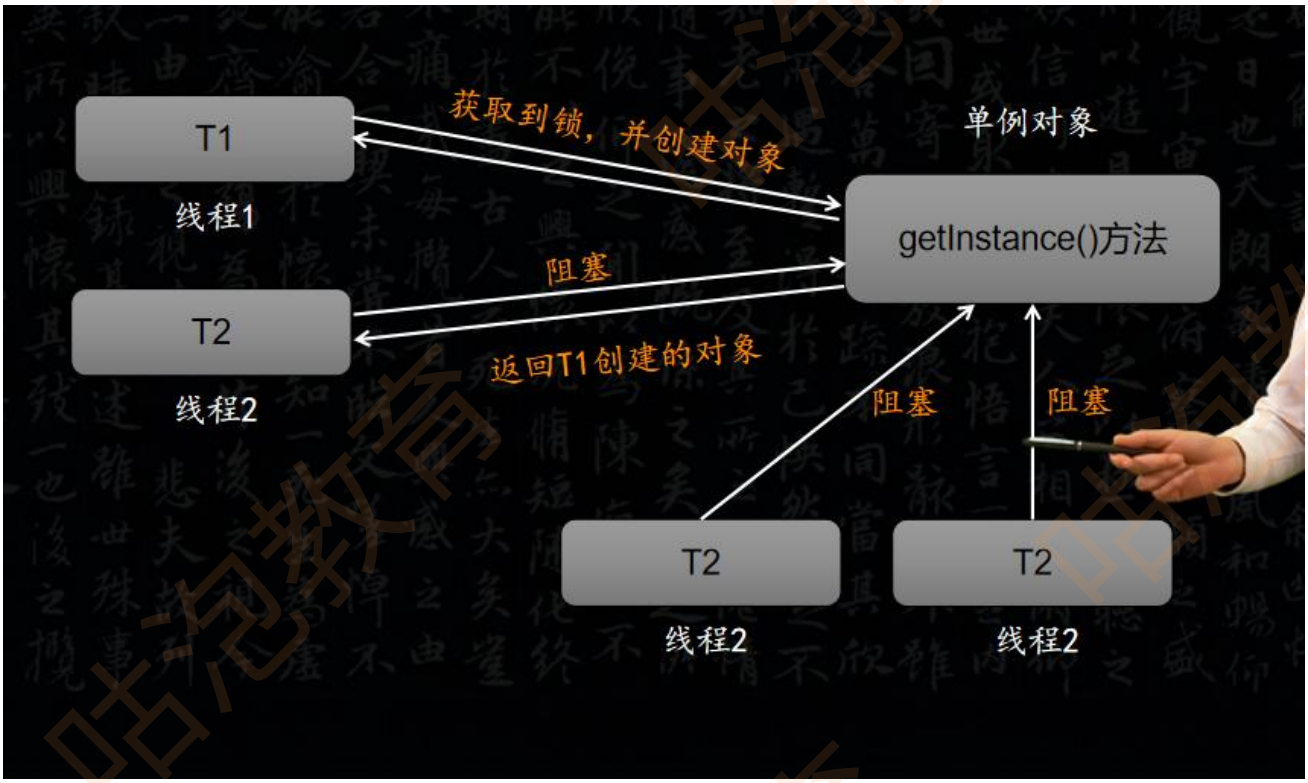
假设，我们去掉第1次检查，只保留第2次检查，如代码所示：

现在，有两个线程T1、T2同时访问getInstance()方法，当T1进入到synchronized代码块的时候，T2就会阻塞。直到T1执行完成释放CPU资源，T2才能获得锁。

T1执行检查条件时，null == instance为ture满足条件，因此，会创建1个新的对象，而T2执行检查条件时，不满足条件，就会直接返回T1创建好的对象，这样就保证了单例。

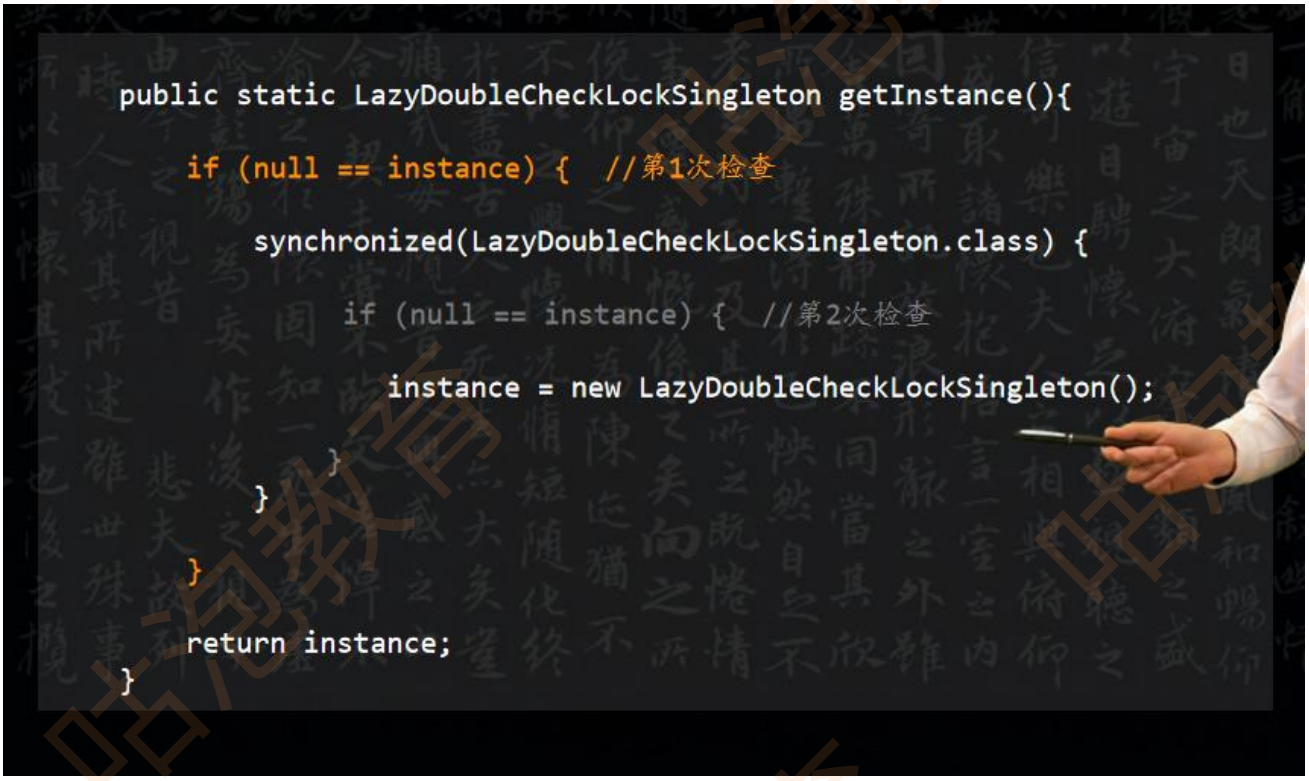


可是，问题来了，后续如果再有其他线程T4、T5、T6出现并发，同时调用getInstance()方法的情况依然会阻塞，



相当于，不管单例对象是否已经创建，每次调用都可能阻塞，会影响程序的执行效率。所以，加上第1次检查的目的是，保证只有第一次出现并发的情况会阻塞，提高性能。

那假设，去掉第2次检查,只保留第1次检查，如代码所示：




```
public static LazyDoubleCheckLockSingleton getInstance(){
 if (null == instance) {
     synchronized(LazyDoubleCheckLockSingleton.class) {
        //if (null == instance) {
           instance = new LazyDoubleCheckLockSingleton();
        //}
     }
  }
```

```
    return instance;

}
```

还是，线程T1、T2同时访问getInstance()方法，此时T1、T2同时满足条件，两个线程会按顺序执行synchronized代码块中的逻辑。假设T1先执行创建对象，那么，T2获得锁的时候，依然会创建对象，而且还会覆盖T1创建的对象，

这就相当于破坏了单例。

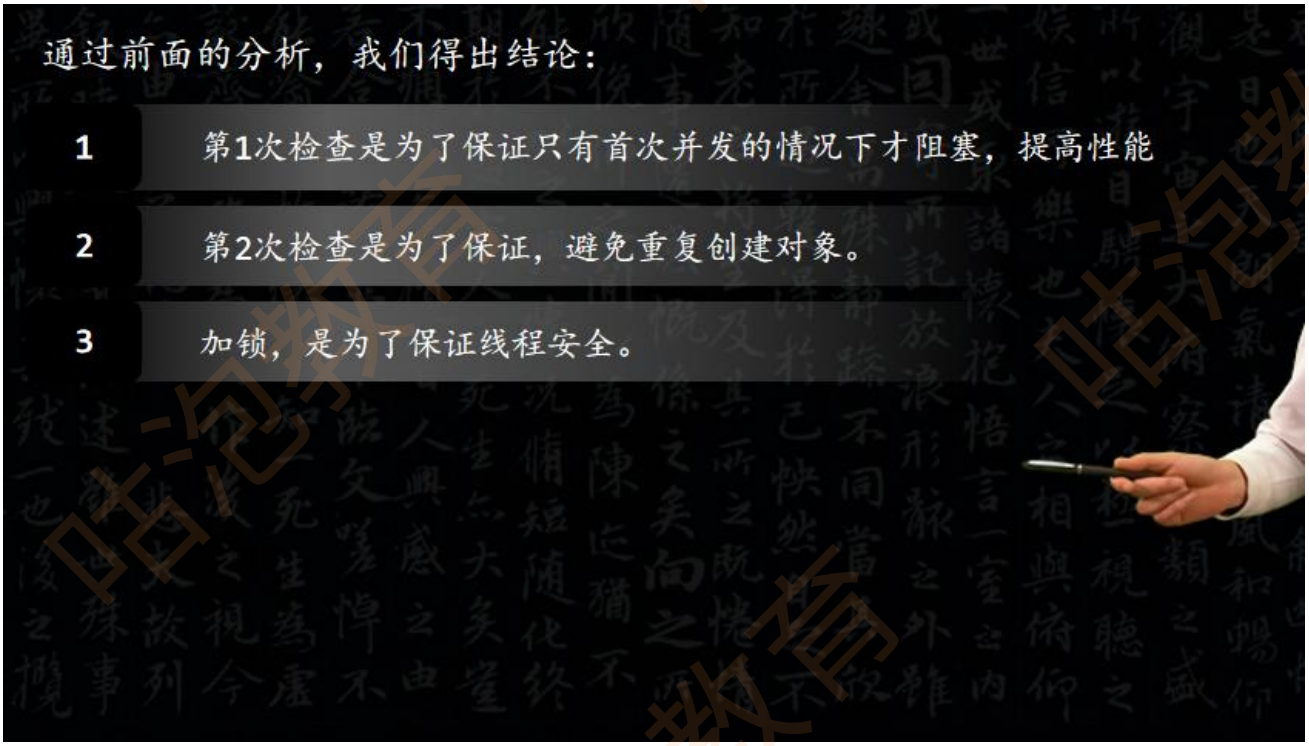
因此，第2次检查的目的是，保证单例，避免重复创建单例对象。

【导航条：总结】

（转场，屏幕黑底白字，正中央出现“3 总结）

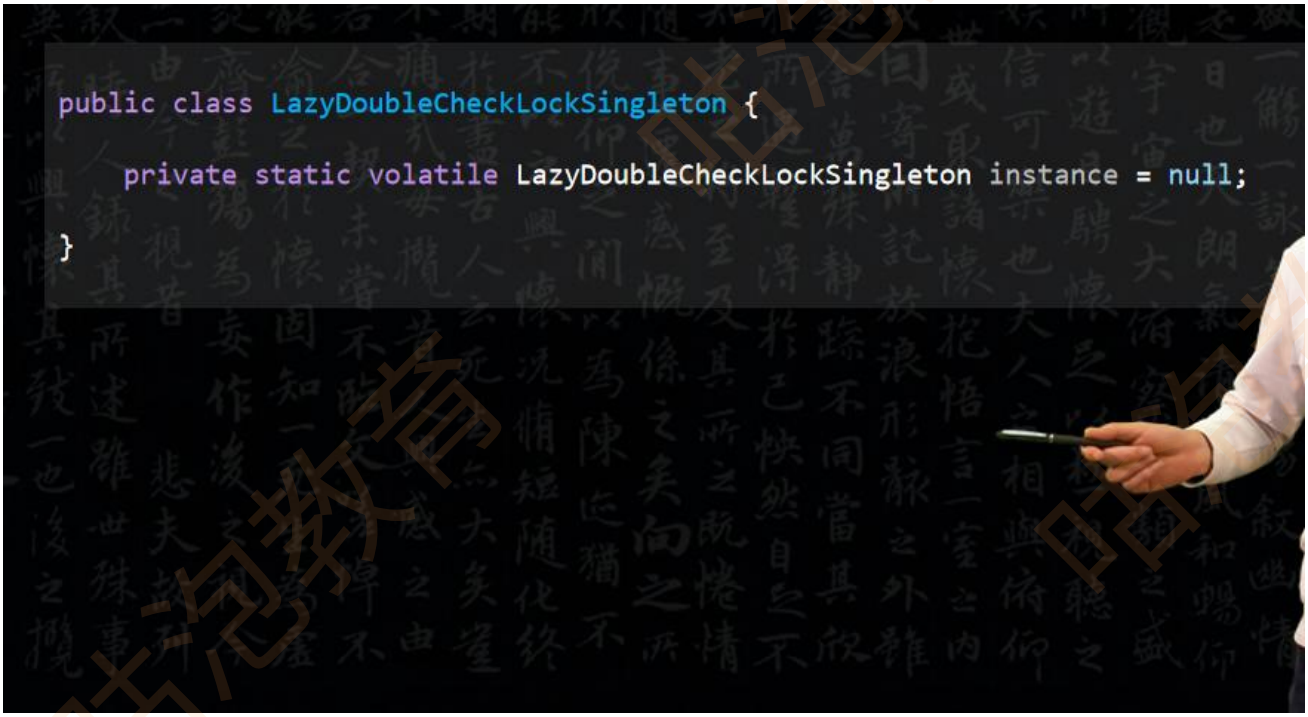
3、总结

通过前面的分析，我们得出结论，DCL单例写法中，



第1次检查是为了保证只有首次并发的情况下才阻塞，提高性能，第2次检查是为了保证，避免重复创建对象。加锁，当然就是为了保证线程安全。

在今天的分享，我还有一个细节没有讲到，就是在并发情况下，new一个对象可能会出现指令重排的现象。这时候，我们需要给声明的单例对象加上volatile关键字，保证可见性。




```
public class LazyDoubleCheckLockSingleton {



    private static volatile LazyDoubleCheckLockSingleton instance = null;

}
```

至于volatile关键字为什么能解决指令重排的问题，小伙伴们可以去我的主页，在往期视频中有专门详细分析这个问题，本期视频就不重复讲解了。

【结尾】

好了，以上就是我对DCL两次检查的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

112.谈谈你对RPC框架的理解

【导航条：引言】

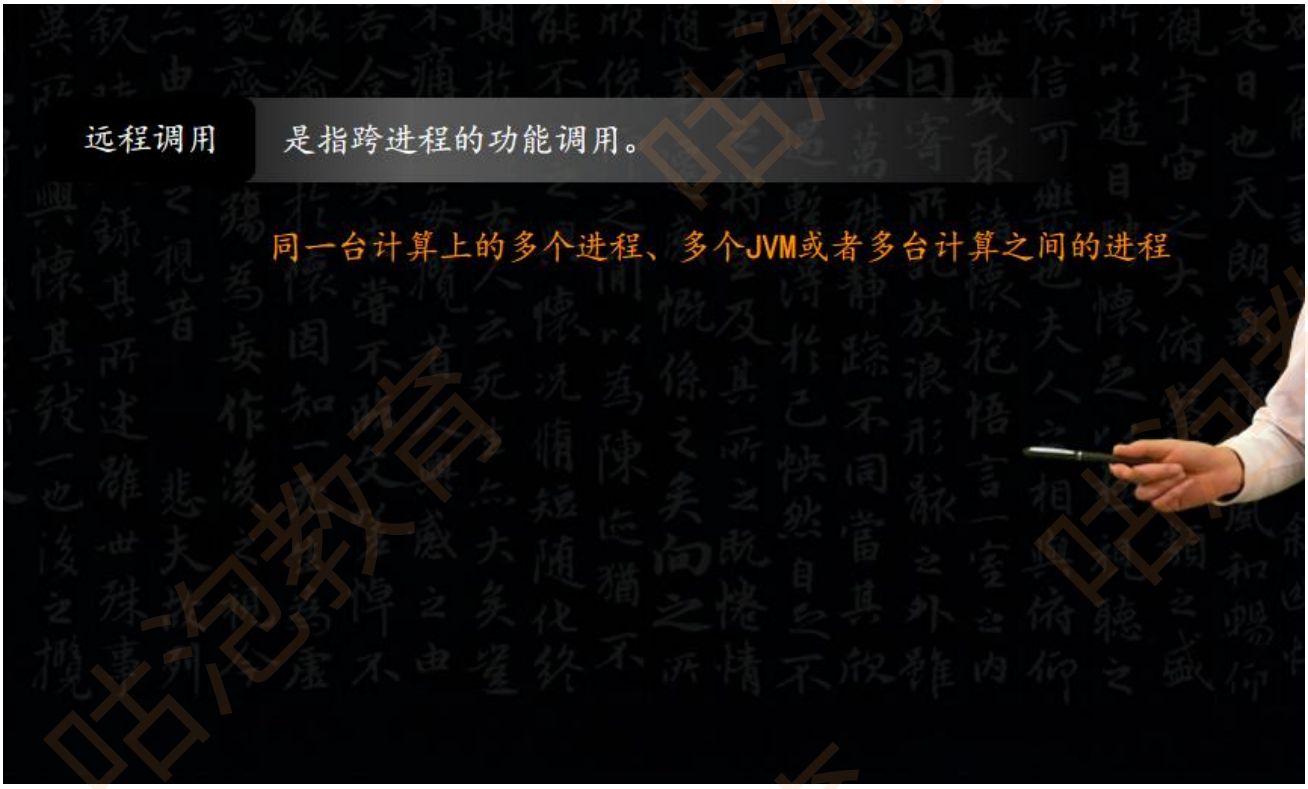
有位工作6年的小伙伴，面试的时候被问到这样一道题，说谈谈你对RPC的理解。在分布式微服务架构中，远程通信是最基本的需求。常见的远程通信方式有基于REST架构的HTTP协议，以及基于RPC协议的RPC框架。今天，我给大家分享一下我的理解。

【导航条：什么是RPC】

（转场，屏幕黑底白字，正中央出现“1 RPC）

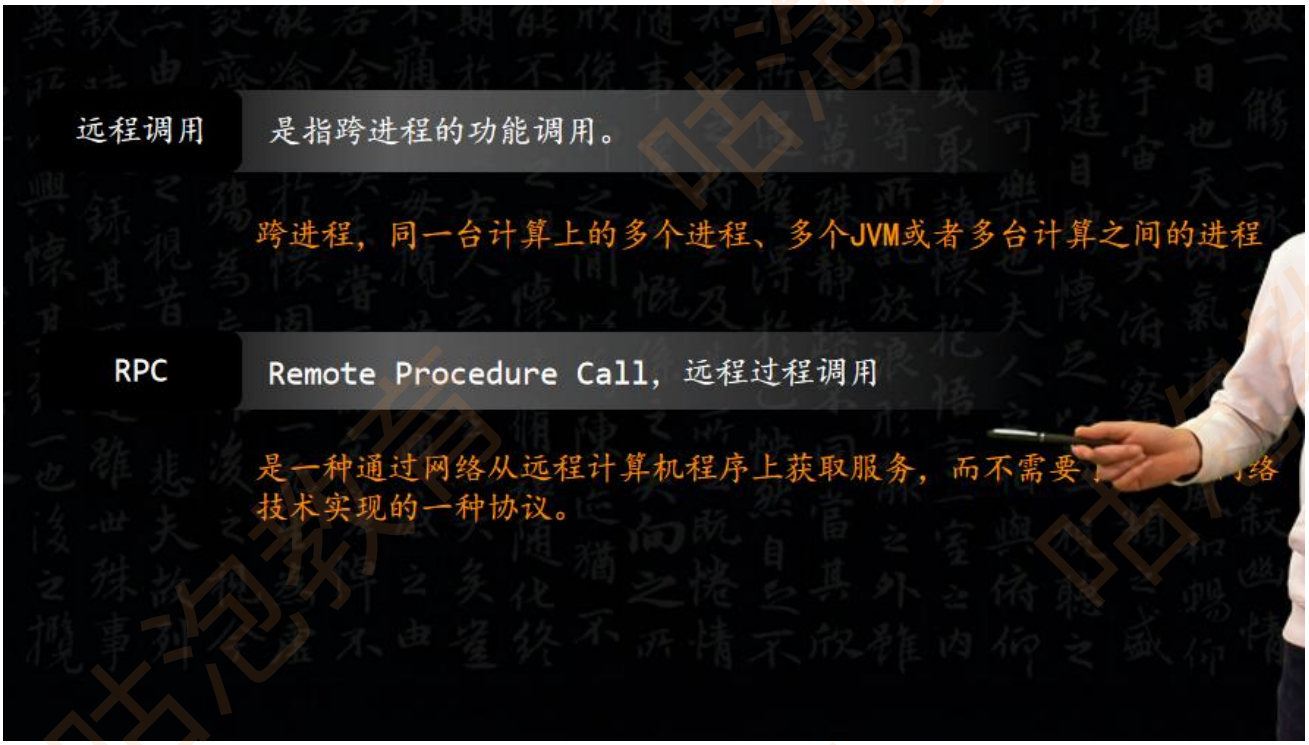
1、什么是RPC

首先，给大家介绍一下什么是远程调用。很多小伙伴会误以为，远程调用是指跨越物理距离上的远程。实际上，准确地说，远程调用是指跨进程的功能调用。

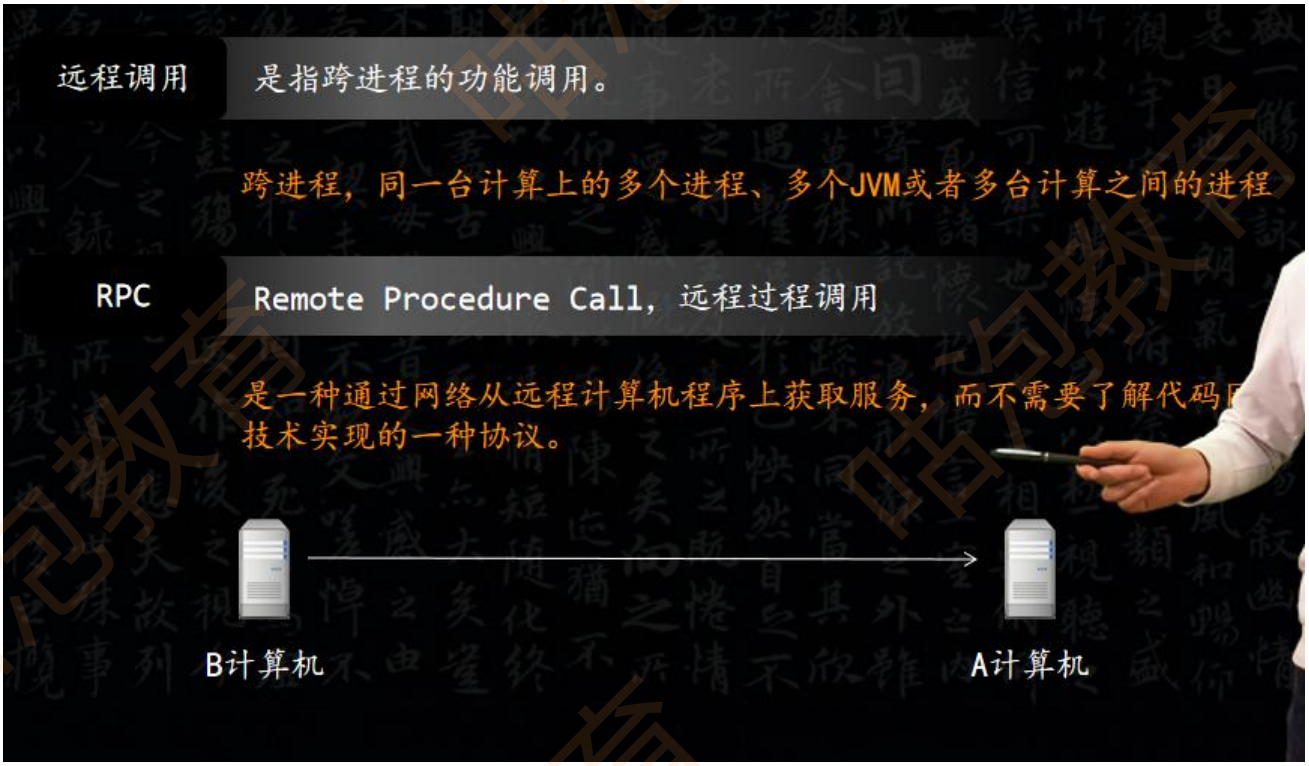


跨进程，可以理解为同一台计算上的多个进程、多个JVM或者多台计算之间的进程。

那什么是RPC呢？RPC的全称是Remote Procedure Call，翻译过来叫远程过程调用。



它是一种通过网络从远程计算机程序上获取服务，而不需要了解代码网络技术实现的一种协议。凡是符合这种协议的框架，都可以称之为RPC框架。



通俗理解就是，A计算机提供一个服务，B计算机可以像调用本地服务一样去调用A计算机提供的服务，这就是RPC的主要作用。

其实，RPC的应用在很早之前就出现了。在40年以前，也就是1981年的时候，由一家叫做Nelson的公司提出，并把它应用到了分布式系统之间的通信协议。

【导航条：RPC协议】

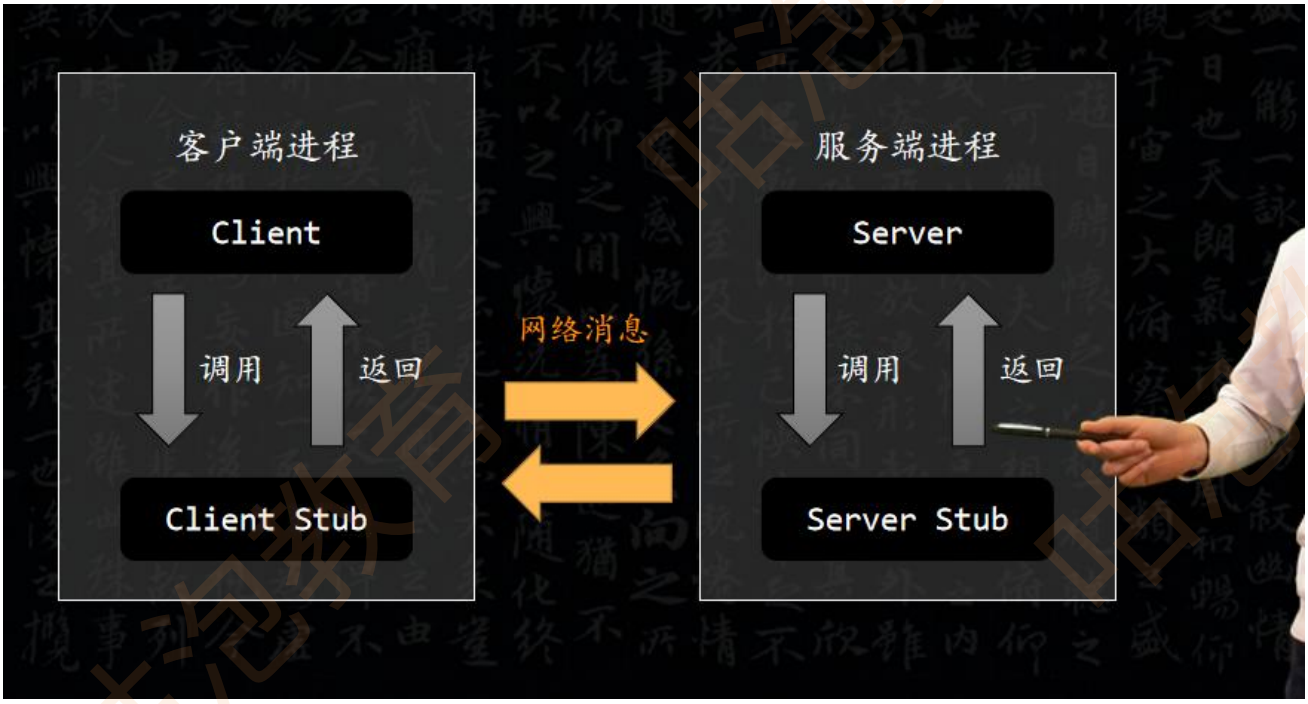
（转场，屏幕黑底白字，正中央出现“2 RPC协议）

2、RPC协议

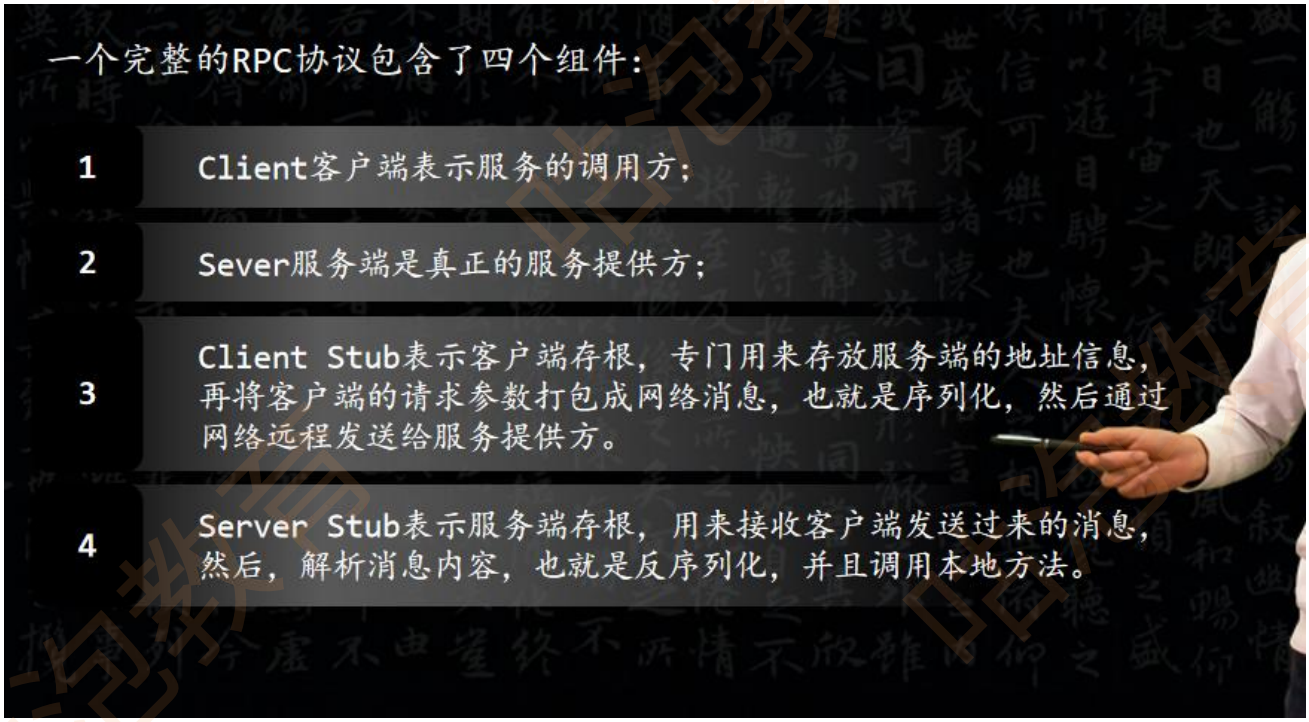
要实现RPC，需要通过网络进行数据传输，并且对调用的过程进行封装。现在比较流行的RPC框架一般都是采用TCP协议作为底层的传输协议。当然，其他协议也是可以的，比如UDP。

RPC协议，强调的是过程调用，调用的过程对于用户来说是完全透明的，用户不需要关心调用细节。可以像调用本地服务一样去调用远程服务。

我们来看这样一张图，



一个完整的RPC协议包含了四个组件，分别是Client、Server、Client Stub和Sever Stub。



Client客户端表示服务的调用方；

Sever服务端是真正的服务提供方；

Client Stub表示客户端存根，专门用来存放服务端的地址信息，再将客户端的请求参数打包成网络消息，也就是序列化，然后通过网络远程发送给服务提供方。

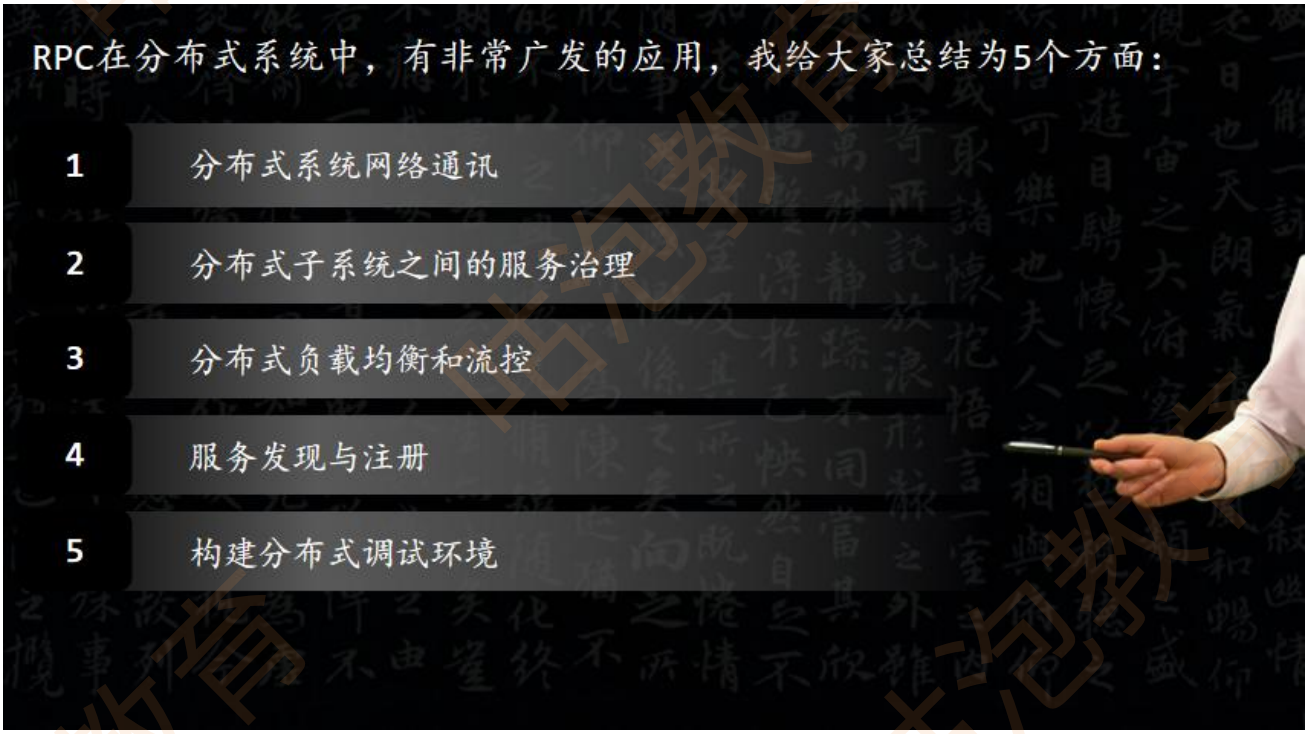
Server Stub表示服务端存根，用来接收客户端发送过来的消息，然后，解析消息内容，也就是反序列化，并且调用本地方法。

【导航条：应用场景】

（转场，屏幕黑底白字，正中央出现“3 RPC）

3、RPC的应用场景

RPC在分布式系统中，有非常广发的应用，我给大家总结为5个方面：



1、分布式系统网络通讯

2、分布式子系统之间的服务治理

3、分布式负载均衡和流控

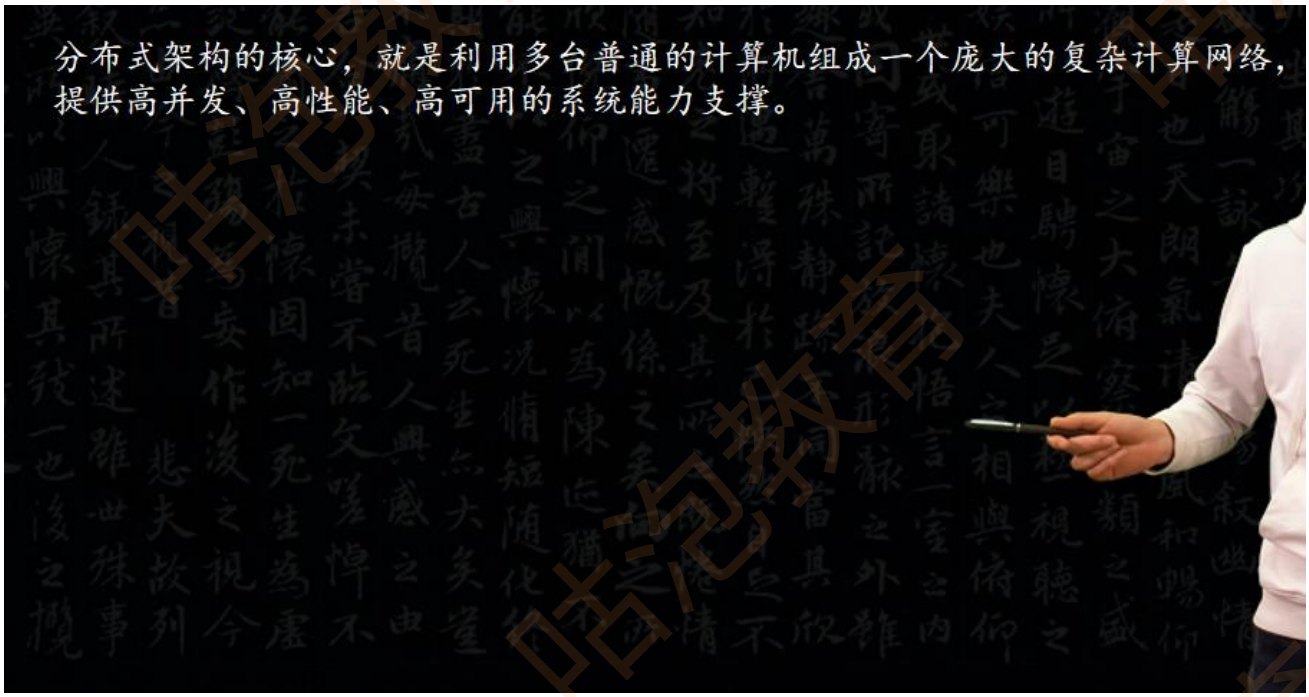
4、服务发现与注册

5、构建分布式调试环境

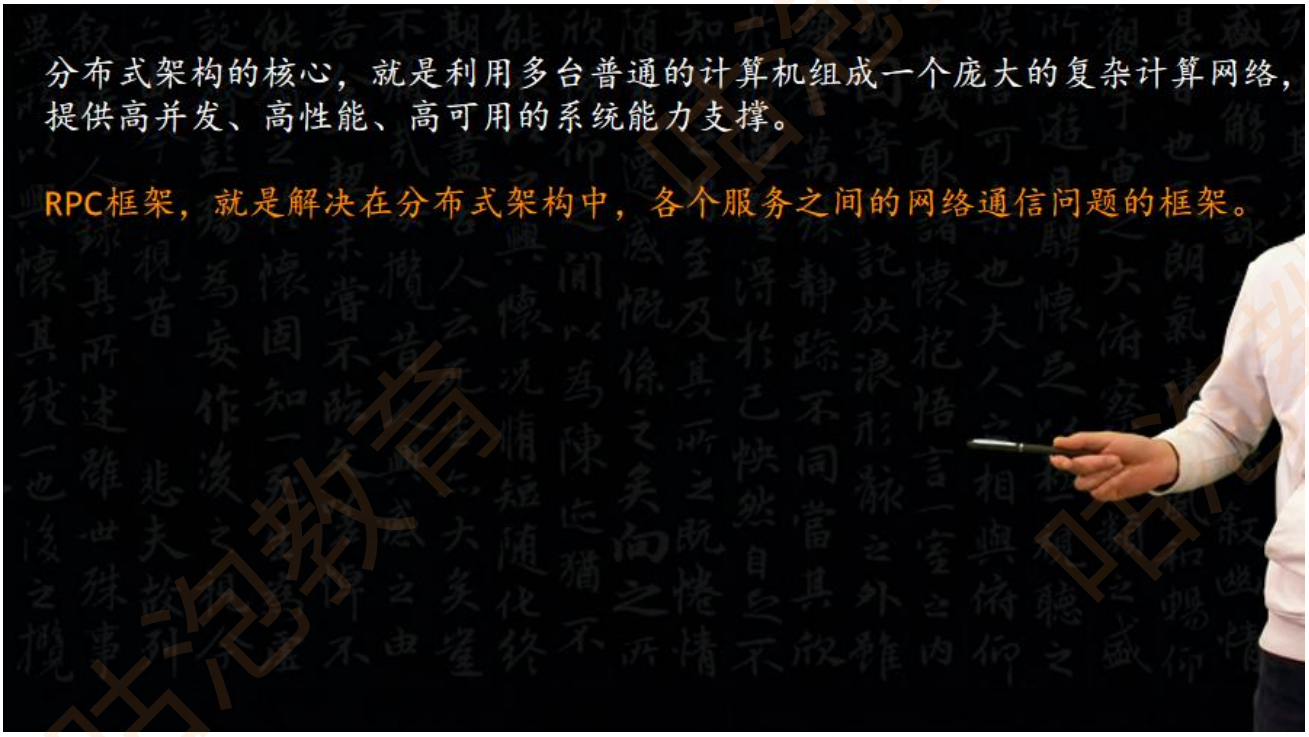
【导航条：RPC框架】

（转场，屏幕黑底白字，正中央出现“4 RPC框架）

4、RPC框架



在互联网应用开发中，随着业务的复杂度增加，一般都会采用分布式架构。分布式架构的核心，就是利用多台普通的计算机组成一个庞大的复杂计算网络，提供高并发、高性能、高可用的系统能力支撑。



在分布式架构中，原本的单体应用服务被拆分成多个独立部署的服务，分布式在计算机网络上，这些服务必然，需要通过网络进行数据通信和交互。而RPC框架，就是解决在分布式架构中，各个服务之间的网络通信问题的框架。

在Java应用中，JDK 1.1版本的时候就提供了对RPC的支持框架，叫做RMI。由于RMI不能实现跨域语言的远程调用。

后面，广发采用WebService来实现远程调用，但是，WebService需要定义非常复杂的Scheme文件，导致无效数据内容过于臃肿。

之后，就被HTTP加JSON的方案所代替了。

RPC发展到现在，实现的方式也非常多样，但是基本都实现了跨语言的远程调用。

一般来说，RPC框架都应用于大型企业，只有在业务复杂度和用户体量都比较大的时候，需要对服务进行解耦，从而达到扩展性强、部署灵活的目的。目前比较流行的开源RPC框架有Goole的gRPC、Facebook的Thrift、Alibaba的Dubbo。这些框架除了提供基础的远程通信功能以外，还会在系统性能、传输效率、服务治理等方面做出一些优化设计。

比如，阿里开源的RPC框架Dubbo就提供了非常丰富的服务治理功能。

【结尾】

好了，看完本期的视频，相信各位小伙伴对于RPC又有了一个更加清晰的认识和了解。面试的时候，如果被问到RPC的问题，是不是知道怎么了呢？以上就是我对RPC的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

113、谈谈你对IaaS、PaaS、SaaS的理解

【导航条：引言】

有位工作6年的小伙伴，去面试架构师岗位，被问到这样一道题，说谈谈你对IaaS、PaaS、SaaS的理解。今天，我给大家分享一下我的理解。

【导航条：云服务和XaaS】

（转场，屏幕黑底白字，正中央出现“1 云服务和XaaS）

1、云服务和XaaS

要理解这三者的关系，我们先了解云服务。所谓云服务，是指利用互联网提供动态易扩展的虚拟化资源整合服务。云服务的主体架构，主要有基础设施服务、平台服务、软件服务。

通过抽象和分层，在云服务领域创造了一个词，叫做“一切即服务”的概念，也就是X as a Service。

我们通常所说的：

IaaS全称为Infrastructure as a Service，也就是基础设施即服务的简称；

PaaS的全称为Platform as a Service，也就是平台即服务的简称；

SaaS的全称为Software as a Service，也就是软件即服务的简称。

除了这些以外，还衍生出了很多类似的概念，比如 DaaS（数据即服务）、OaaS（运维即服务）等等。在云服务架构中，IaaS、PaaS 和 SaaS三种架构是为最核心的架构。

【导航条：三者对比】

（转场，屏幕黑底白字，正中央出现“2 三者对比）

2、三者对比

首先是，IaaS基础设施即服务（Infrastructure as a Service），它是面向企业或者开发者，提供基础资源支持，包括：计算、存储、网络等等。

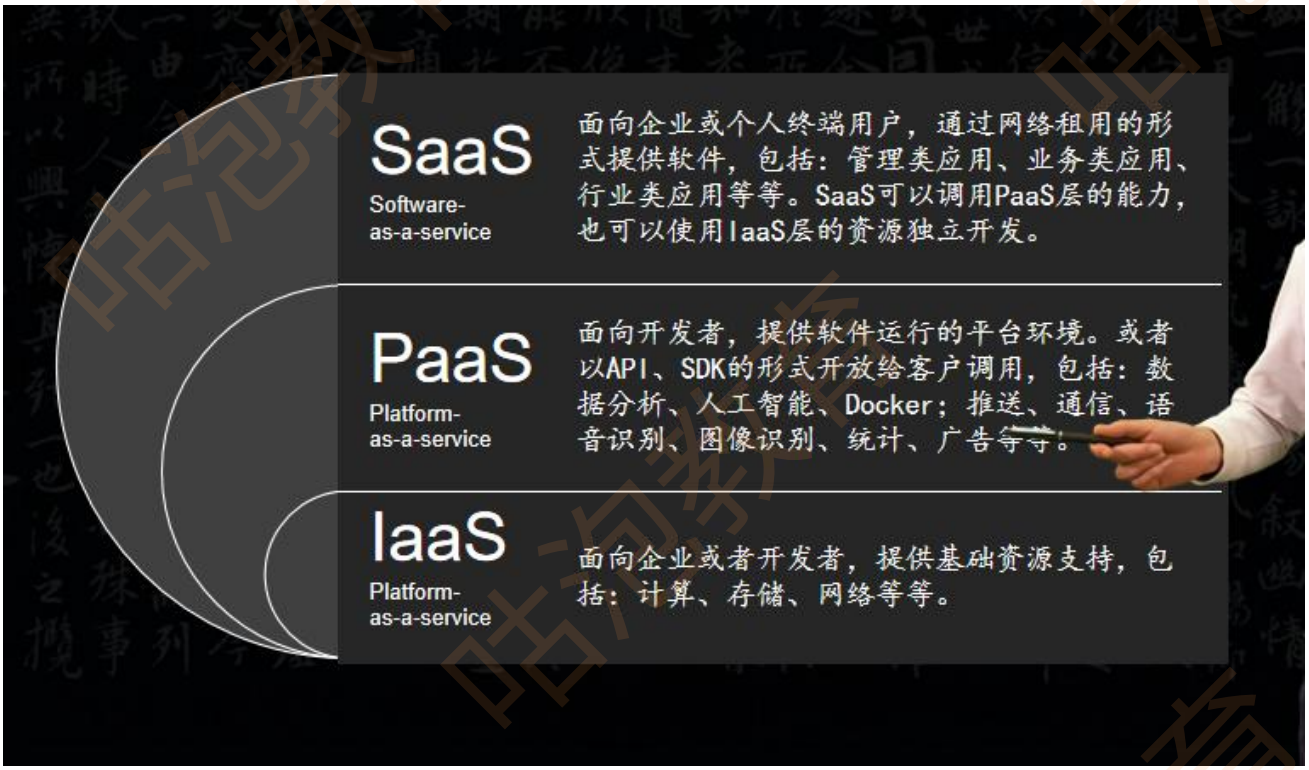
有了IaaS以后，企业在开发APP时，只需在公有云平台上注册一个账号，花点钱，配置各种云服务器，各种大小的存储，各种带宽的网络，都配齐，不用操心诸如机房选址、设备采购、实体服务器、存储、网络等问题，只要一个账号，便解决了。

而作为程序员，只需在服务器上安装各类程序，中间件，数据库等等，包括前端与后台，测试，上线，迭代，更新的操作都要以IaaS为基础，否则，前期的工作就都需要自己来完成。

然后，就是PaaS平台即服务（Platform as a Service）它面向开发者，提供软件运行的平台环境。或者以API、SDK的形式开放给客户调用，包括：数据分析、人工智能、Docker；推送、通信、语音识别、图像识别、统计、广告等等。

PaaS是在IaaS的基础之上，解决了操作系统、数据库、运行时环境、中间件、各种框架的搭建操作问题，有了PaaS，程序员只需要专心的开发自己的APP就行了。

做PaaS需要的技术能力和资源比做SaaS要求更高，因为SaaS服务商的强项在于业务和应用，而不是平台技术和服务能力。



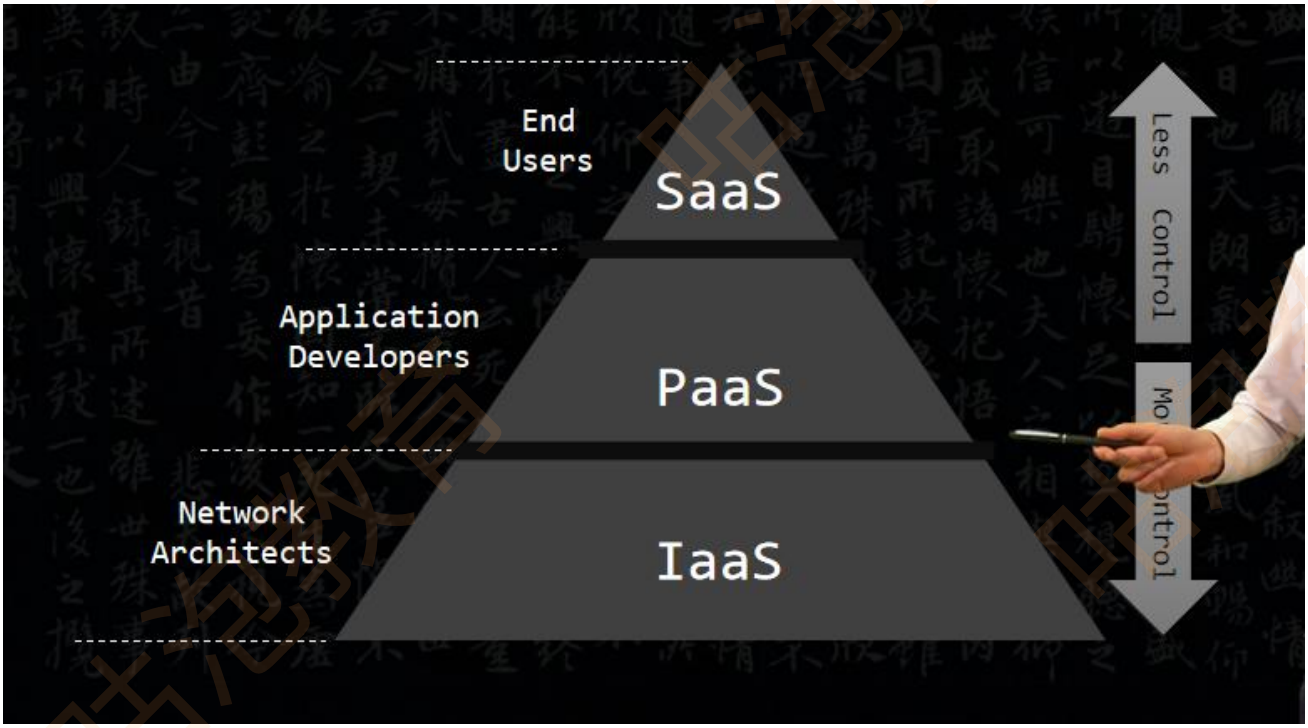
最后，就是SaaS 软件即服务（Software as a Service），是面向企业或个人终端用户，通过网络租用的形式提供软件，包括：管理类应用、业务类应用、行业类应用等等。SaaS可以调用PaaS层的能力，也可以使用IaaS层的资源独立开发。

(这张配图需要更换一

下)

SaaS就相当于是现成的了，根本用不着开发，用户只需要专注在自己的业务上就行，比如写代码、维护等各种定制化的工作，都交给SaaS厂商。用户只需要一个续费账号就能解决所有问题。

不论是 IaaS、PaaS 和 SaaS 三者都有非常庞大的产品群体和用户服务群体，三者之间的架构关系，如图所示。



从Iaas到SaaS的过程，企业需要付出的开发成本越来越小，云服务化的程度也越来越高，如图所示：

【结尾】

好了，以上就是我对SaaS、PaaS、IaaS的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

114、什么条件下会产生死锁，如何避免死锁?

【导航条：引言】

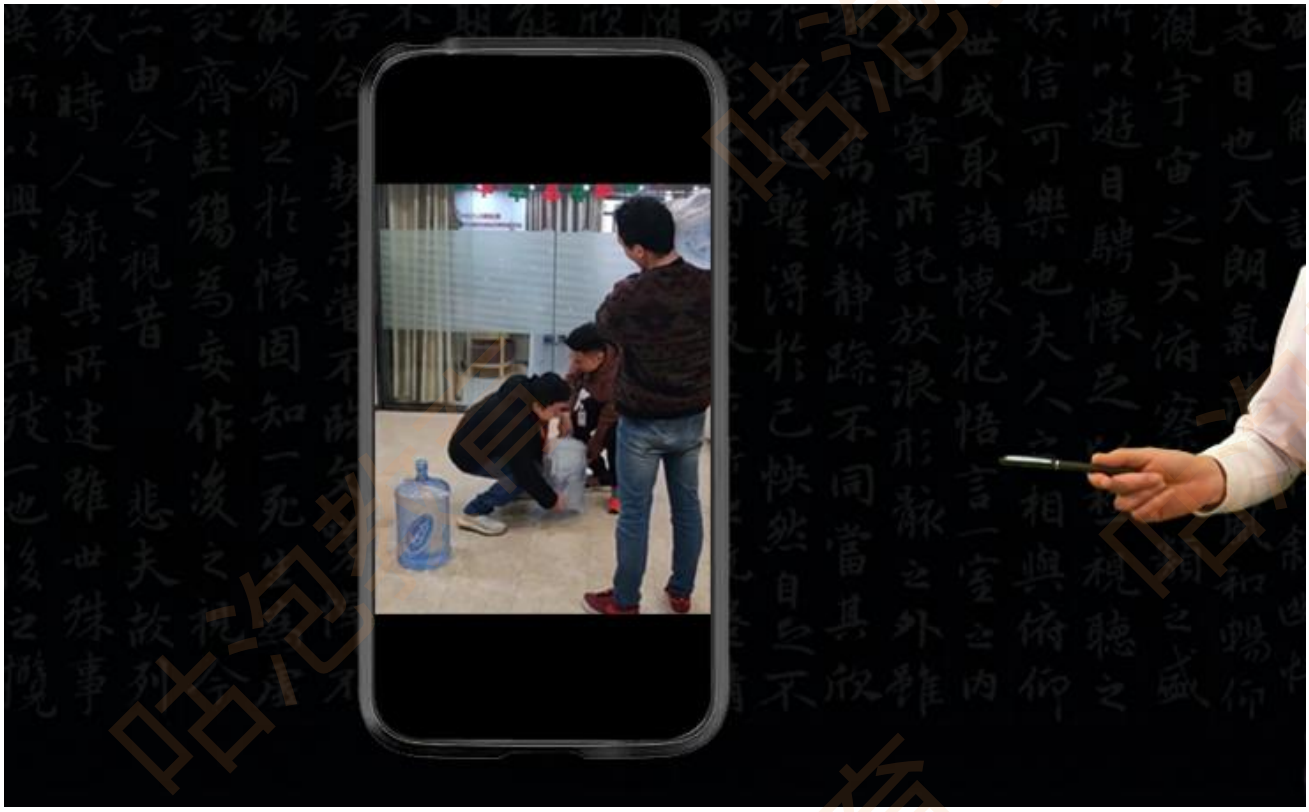
1位工作4年的小伙伴，又被问到一个关于锁的问题，说什么条件下会产生死锁，如何避免死锁？他说，面试之前还特意刷了题，面试的时候就是脑子一片空白不知道怎么回答。今天，我给大家彻底讲明白。

【导航条：什么是死锁】

（转场，屏幕黑底白字，正中央出现“1 什么是死锁”）

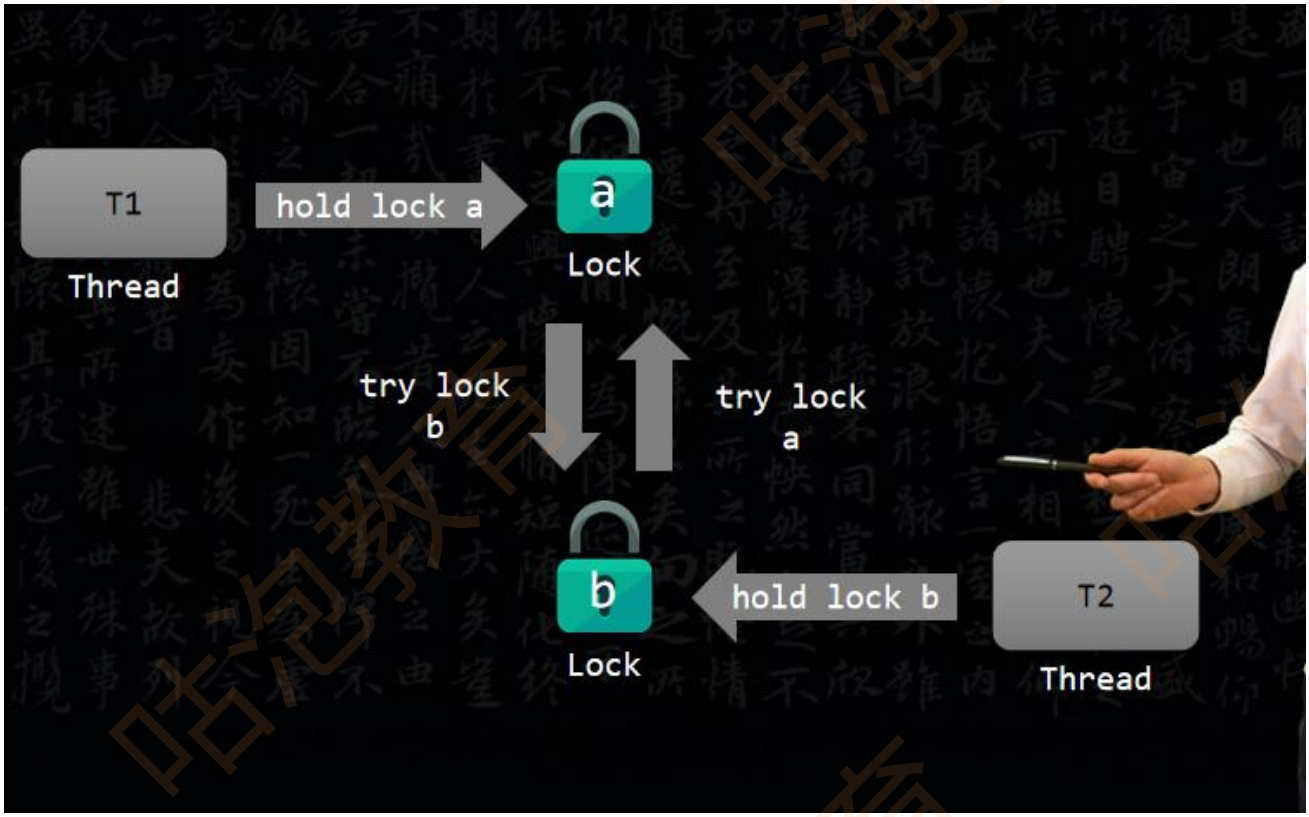
1、什么是死锁?

先来看这样一段视频，*（左侧区域播放视频5秒，人在右侧站立不动）*



看完你也许已经懂了，那到底什么是死锁呢？

（开始出现动画效果）



死锁，简单来说就是两个或者两个以上的线程在执行过程中，去争夺同一个共享资源导致相互等待的现象。如果没有外部干预，线程会一直处于阻塞状态，无法往下执行。这样一直等待处于阻塞状态的线程，被称为死锁线程。

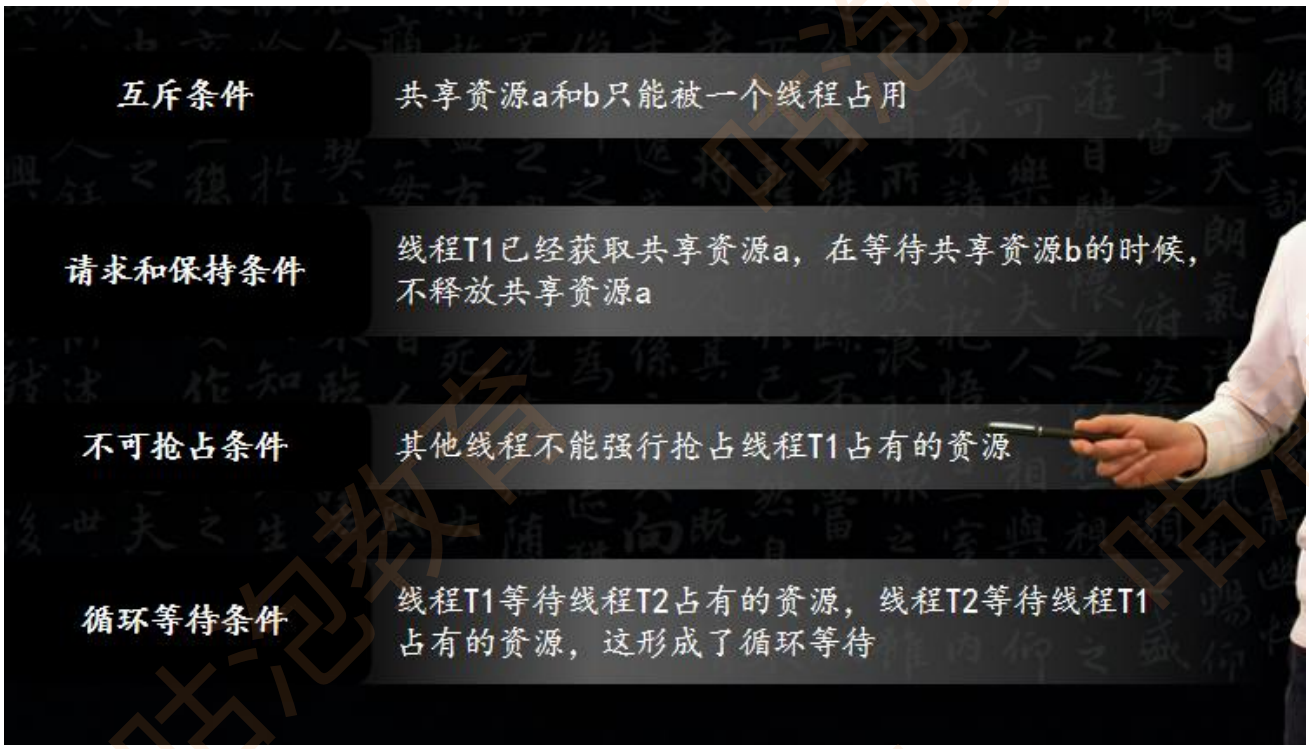
【导航条：产生死锁的原因】

（转场，屏幕黑底白字，正中央出现“2 产生死锁的原因”）

2、产生死锁的原因

产生死锁需要同时满足以下四个条件：

（开始出现动画效果）



第一个：互斥条件，共享资源a和b只能被一个线程占用；

第二个：请求和保持条件，线程T1已经获取共享资源a，在等待共享资源b的时候，不释放共享资源a；

第三个：不可抢占条件，其他线程不能强行抢占线程T1占有的资源；

第四个：循环等待条件，线程T1等待线程T2占有的资源，线程T2等待线程T1占有的资源，这形成了循环等待。

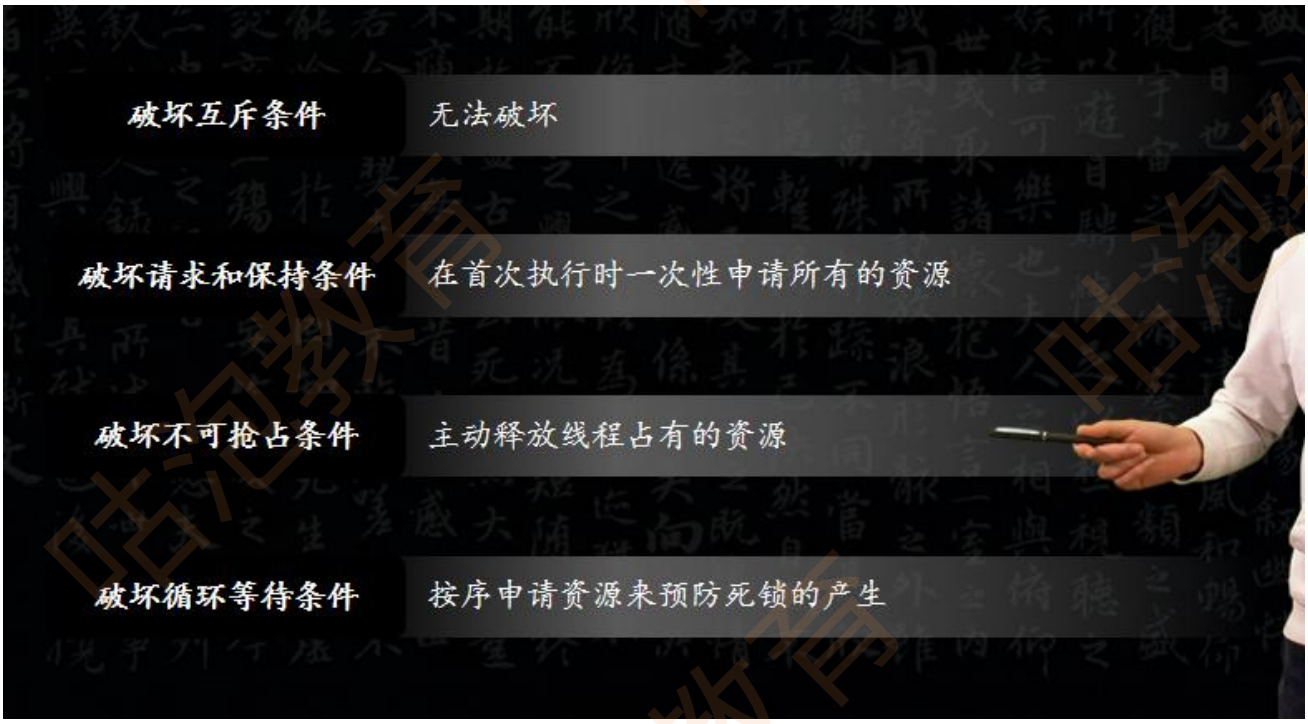
【导航条：如何避免死锁】

（转场，屏幕黑底白字，正中央出现“3 如何避免死锁”）

3、如何避免死锁?

线程产生死锁之后，只能通过外部干预来解决问题，比如重启程序，或者Kill线程。所以，我们只能在写代

码时规避死锁的产生。那么如何避免死锁产生呢？根据产生死锁的四个必要条件，我们只需要破坏其中任何一个条件就可以解决。



第一个互斥条件是没有办法被破坏的，因为它是互斥锁的基本约束。其他三个条件都可以通过人工干预来破坏。比如请求保持条件，我们可以在首次执行一次性申请所有的资源，这样就不存在等待锁的问题了。

第二个，对于不可抢占条件来说，占用部分资源的线程在进一步申请其他资源的时候如果申请不到，我们可以主动释放它占有的资源。这样不可抢占这个条件就被破坏了。

第三个，对于循环等待条件来说，可以通过按序申请资源来预防死锁的产生。所谓按序申请，就是给资源编号，所有线程可以按照线性化的序号顺序去申请共享资源，先申请需要序号小的，再申请序号大的，这样循环等待自然就不存在了。

【结尾】

以上就是我对死锁的理解。

我是被编程耽误的文艺Tom，如果大家还有其他疑问，请在评论区留言。如果本次面试解析对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

115、谈谈你对ES的理解

【导航条：引言】

有粉丝在评论区留言说，让我拍一期关于ES的视频。今天，我给大家分享一下我对ES的理解。

【导航条：*ES是什么*】

（转场，屏幕黑底白字，正中央出现“1 ES是什么）

1、ES是什么

ES全称是Elastic Search，它是一个建立在全文搜索引擎库Lucene基础上的开源搜索和分析引擎。ES它本身具有分布式存储、检索速度快的特性。所以，我们经常会用它来实现全文检索的功能。

Elastic官网对ES的定义已经不再是ElasticSearch这一个组件，而是指Elastic Stack生态。

而Elastic Stack主要包括ElasticSearch、Logstash、Kibana，这三个经典组合也称之为ELK。ElasticSearch主要用来做数据存储、Logstash主要用来做数据采集，Kibana主要用来做数据可视化展示。

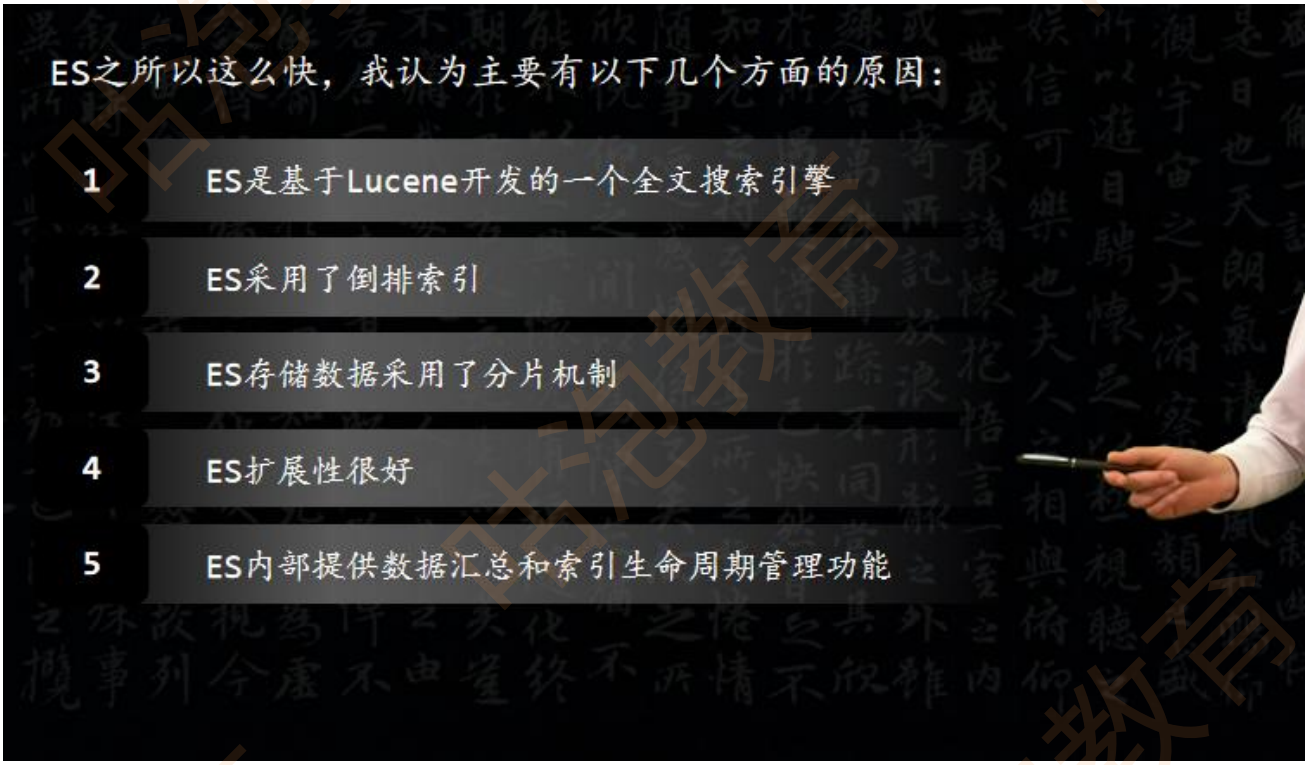
因为ES应用更广泛的场景还是ElasticSearch，所以，下面我们给大家分享的ES还是单指ElasticSearch。比如，网站搜索，日志聚集和检索，这些都可能会涉及到TB级别的数据场景，用ES是一个比较好的选择。

【导航条：ES为什么这么快】

（转场，屏幕黑底白字，正中央出现“2 ES为什么这么快）

2、ES为什么这么快

ES之所以这么快，我认为主要有以下几个方面的原因：



1、ES是基于Lucene开发的一个全文搜索引擎，一方面Lucene是擅长管理大量的索引数据；另外一方面，它会对数据进行分词以后再保存索引。这样，能够去提升数据的检索效率。

2、ES采用了倒排索引。所谓倒排索引就是通过属性值来确定数据记录位置的索引，从而避免全表扫描的问题。

3、ES存储数据采用了分片机制。

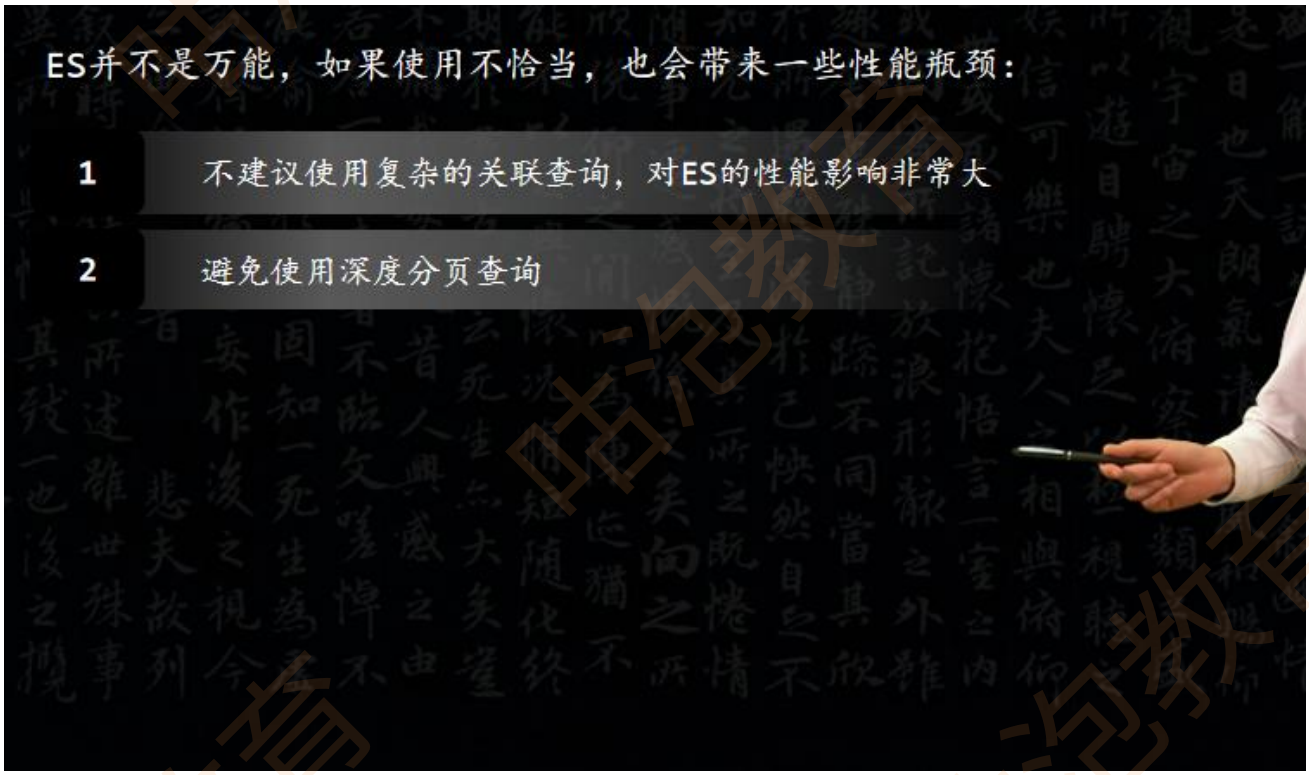
4、ES扩展性很好，支持通过水平扩展的方式来动态增加节点，从而提升ES的处理性能。能够支持上百台服务器节点的扩展，并且支持TB级别的结构化数据和非结构化数据。

5、ES内部提供的数据汇总和索引生命周期管理的功能，更加便于高效地存储和检索数据。

【导航条：ES应用指南】

（转场，屏幕黑底白字，正中央出现“3 ES应用指南）

3、ES应用指南



当然，ES并不是万能，如果使用不恰当，也会带来一些性能瓶颈。不太建议使用复杂的关联查询，这对ES的性能影响非常大。

另外，还要避免深度分页查询。因为，ES的分页是通过from和size参数来实现，也就是说，在查询的时候，每

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

个分片必须要先构造一个长度为from + size的优先队列，然后回传的网关节点。网关节点再对这些优先队列进行排序，再找到正确的size文档。而当from足够大的情况下，容易造成OOM以及网络传输性能下降的问题。

【结尾】

以上就是我对ES的理解。

最后，我把往期分享的视频全部整理成了1份20W字的文档，后续还会连更，希望能够以此来提高各位粉丝的通过率，想获取的小伙伴可以在我的个人煮叶简介中找到。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

116、Java中，4种对象引用之间的区别是什么？

【导航条：引言】

一位工作4年的小伙伴面试被问到这样一道题，说Java中，4种对象引用之间的区别是什么？这道面试题问得比较偏门，更多应用在一些底层类库或者是框架中，涉及到的知识点包括JVM、GC等，一般人确实很少关注。

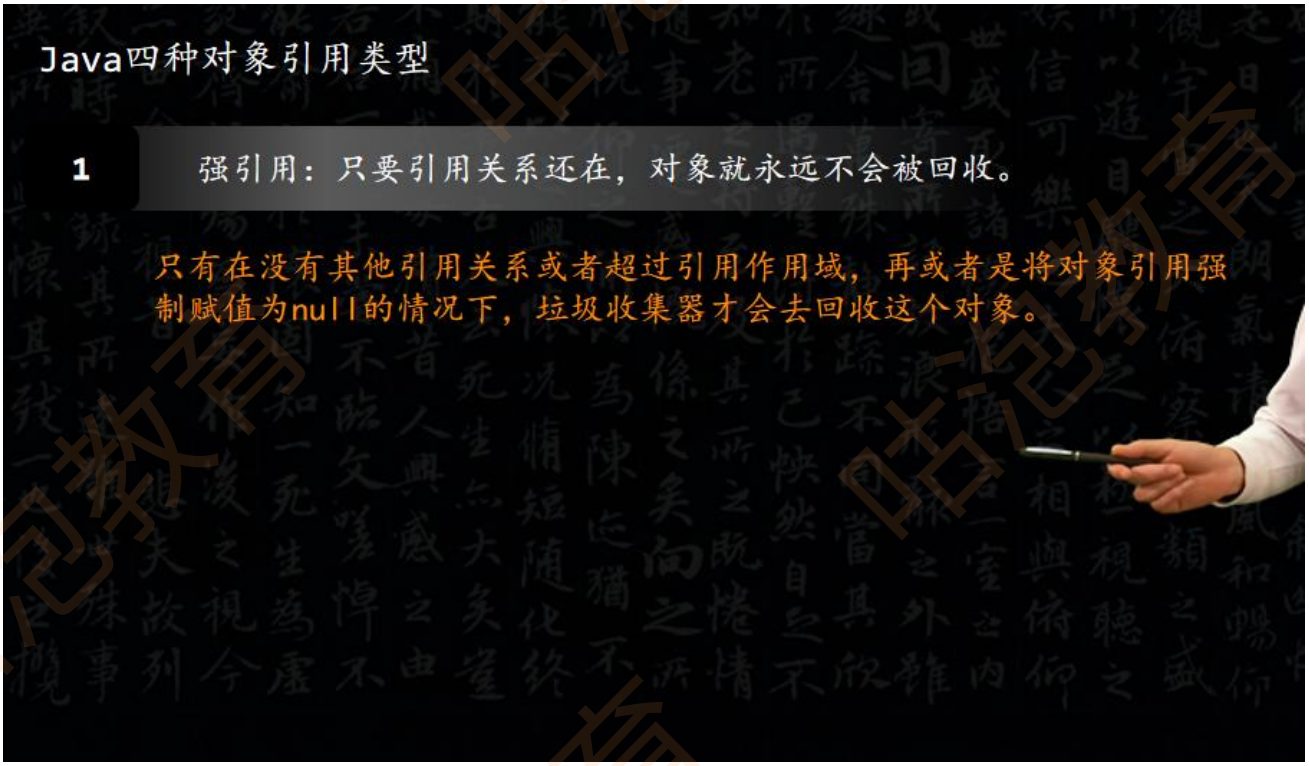
那既然都问到我这里来了，还是要给大家分享一下我的理解。

【导航条：*4种引用类型*】

（转场，屏幕黑底白字，正中央出现“1 4种引用类型）

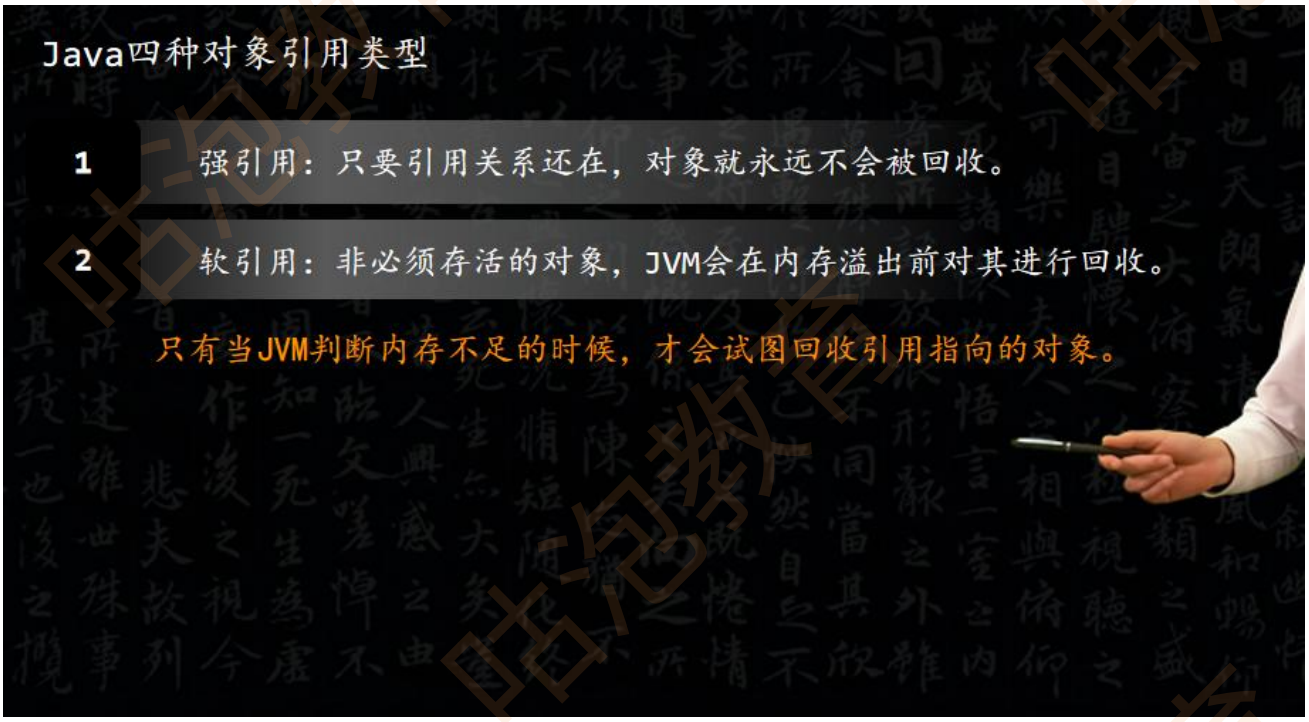
Java中的4种引用类型主要是指强、软、弱、虚，

它们主要是体现对象不同的可达性状态和对GC的影响。下面我给大家详细地分析一下我对这四种引用类型的理解。



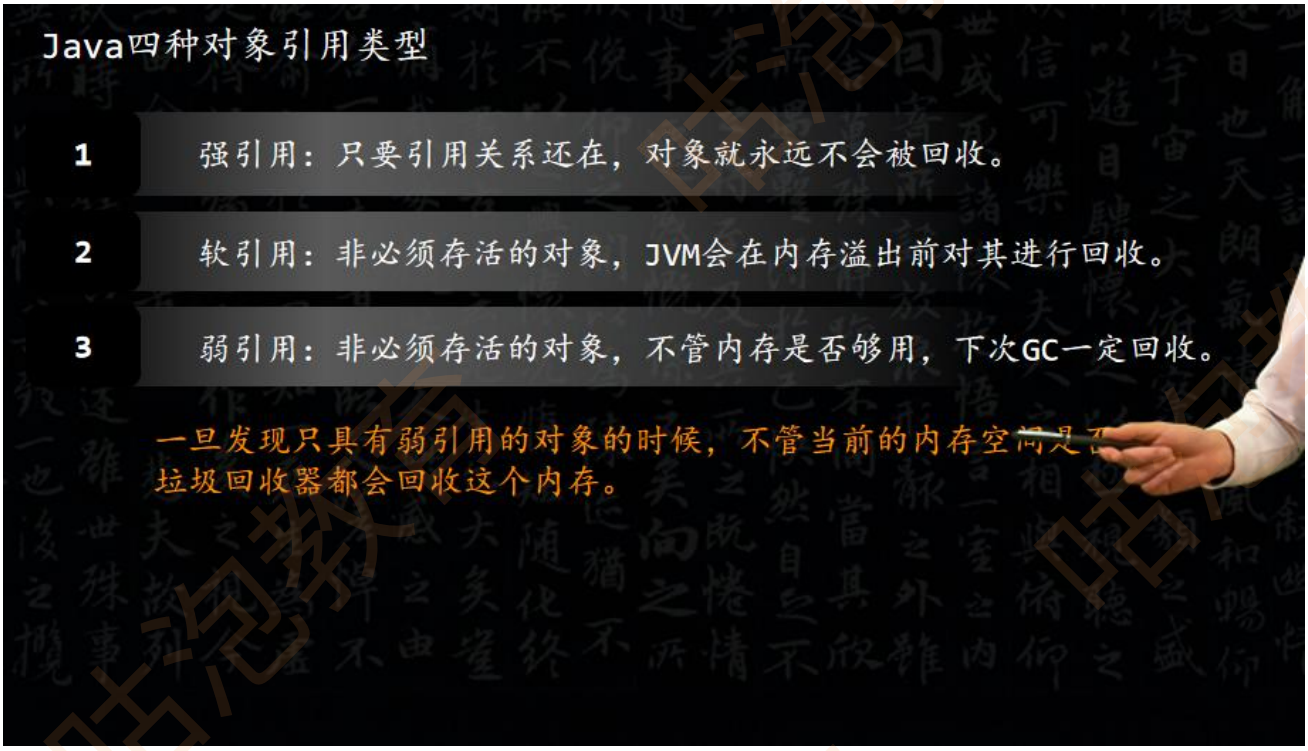
1、强引用：只要引用关系还在，对象就永远不会被回收。

其实就是指普通对象的引用，只要还有引用关系存在，就表示对象还活着，垃圾收集器无法回收这一类对象。只有在没有其他引用关系或者超过引用作用域，再或者是将对象引用强制赋值为null的情况下，垃圾收集器才会去回收这个对象。



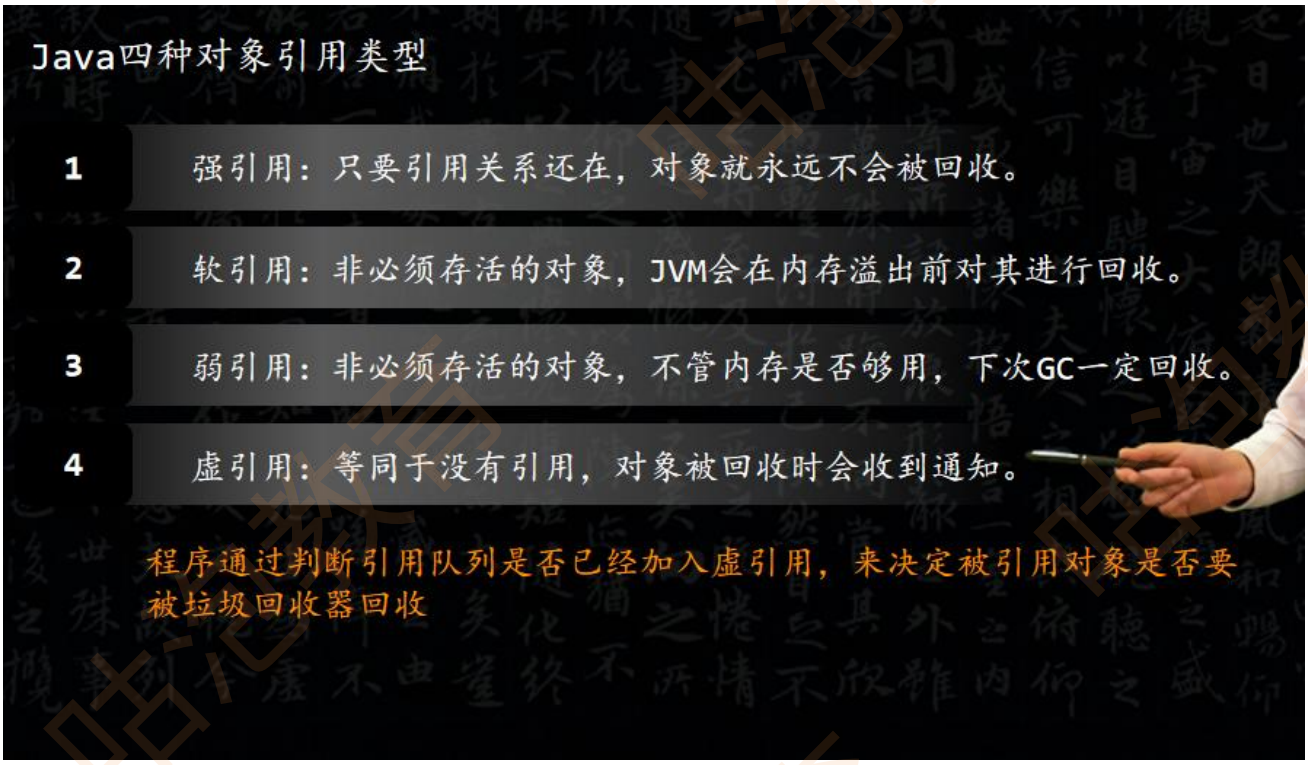
2、软引用：非必须存活的对象，JVM会在内存溢出前对其进行回收。

是一种相对于强引用来说弱一些的引用。可以让对象豁免一些垃圾收集的操作。只有当JVM判断内存不足的时候，才会试图回收引用指向的对象。那软引用通常是用来实现内存敏感的一些缓存，如果还有空闲内存就可以暂时保留缓存，当内存不足的时候会清理掉。这样就可以保证使用缓存的同时，不会耗尽内存。



3、弱引用：非必须存活的对象，不管内存是否够用，下次GC一定回收。

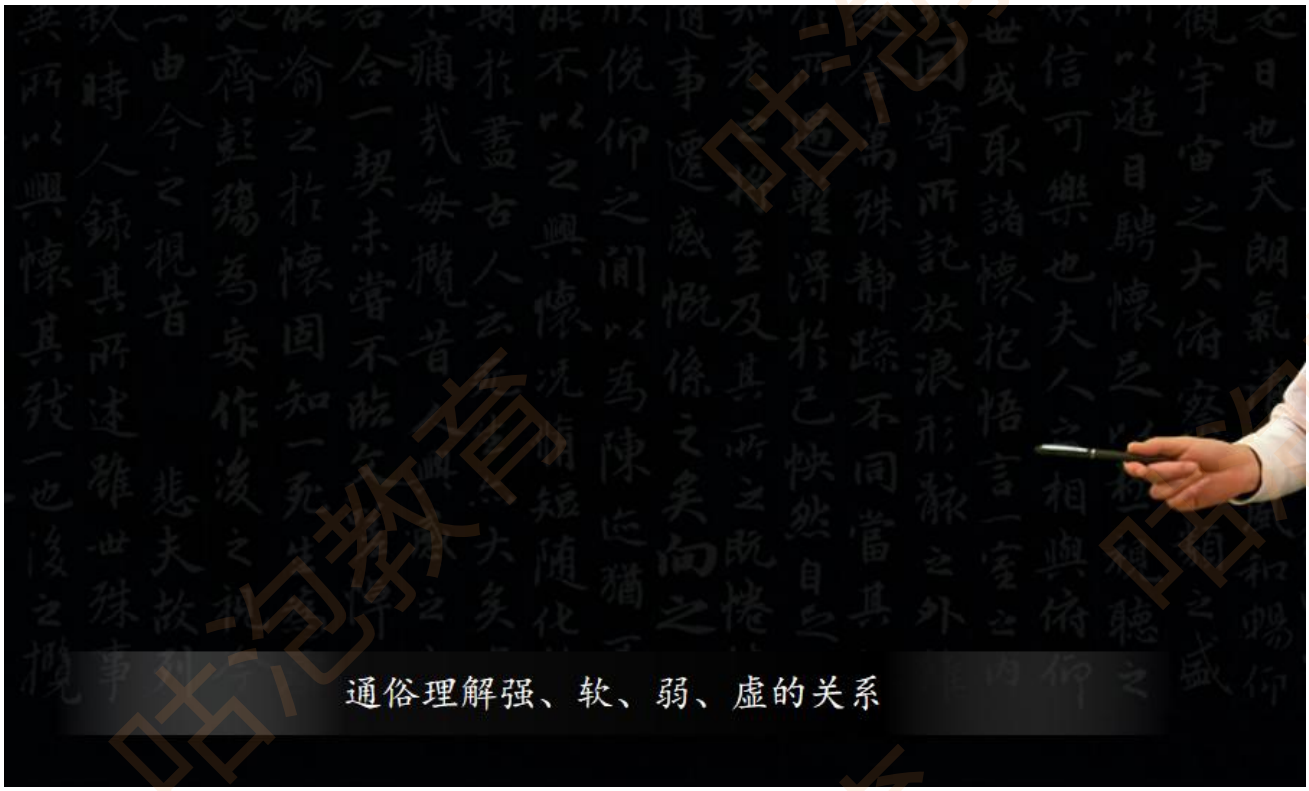
弱引用是相对于强引用而言的，它是允许存在引用关联的情况下，能被垃圾回收的对象。在垃圾回收器线程扫描它所管辖的内存区域的过程中，一旦发现只具有弱引用的对象的时候，不管当前的内存空间是否足够，垃圾回收器都会回收这个内存。



4、虚引用：等同于没有引用，对象被回收时会收到通知。

虚引用不会决定对象的生命周期，它提供一种确保对象被finalize以后，去做某些事情的机制。当垃圾回收期准备去回收一个对象的时候，如果发现他还有虚引用，就会在回收对象的内存之前，把这个虚引用加入到与之关联的引用队列中，程序可以通过判断引用队列是否已经加入虚引用，来决定被引用对象是否要被垃圾回收器回收。然后，我们就可以在引用对象被回收之前执行一些必要的操作。所以，虚引用必须和引用队列一起使用，

(以下需要配图)



以上的分享还是比较官方，举个通俗例子，大家就会更容易理解。

强引用就好比电视剧中的男主角，怎么都死不了。

软引用就像女主角，虽有一段经历，还是没走到最后。

弱引用就是男二号，注定用来牺牲的。

虚引用就是路人甲了。

【结尾】

以上就是我对Java中4种引用对象的理解。有兴趣的小伙伴，可以更进一步去研究一下。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

117、Spring中用到了哪些设计模式?

【导航条：引言】

一位应届毕业生被问到这样一道面试题，说Spring用到了哪些设计模式？其实只要Spring使用得够熟练，回答这道题还是非常轻松的。因为Spring的命名非常规范，基本上从类名就可以看得出来用到了哪些设计模式。

今天，我给大家分享一下我的理解。

【导航条：*主要设计模式*】

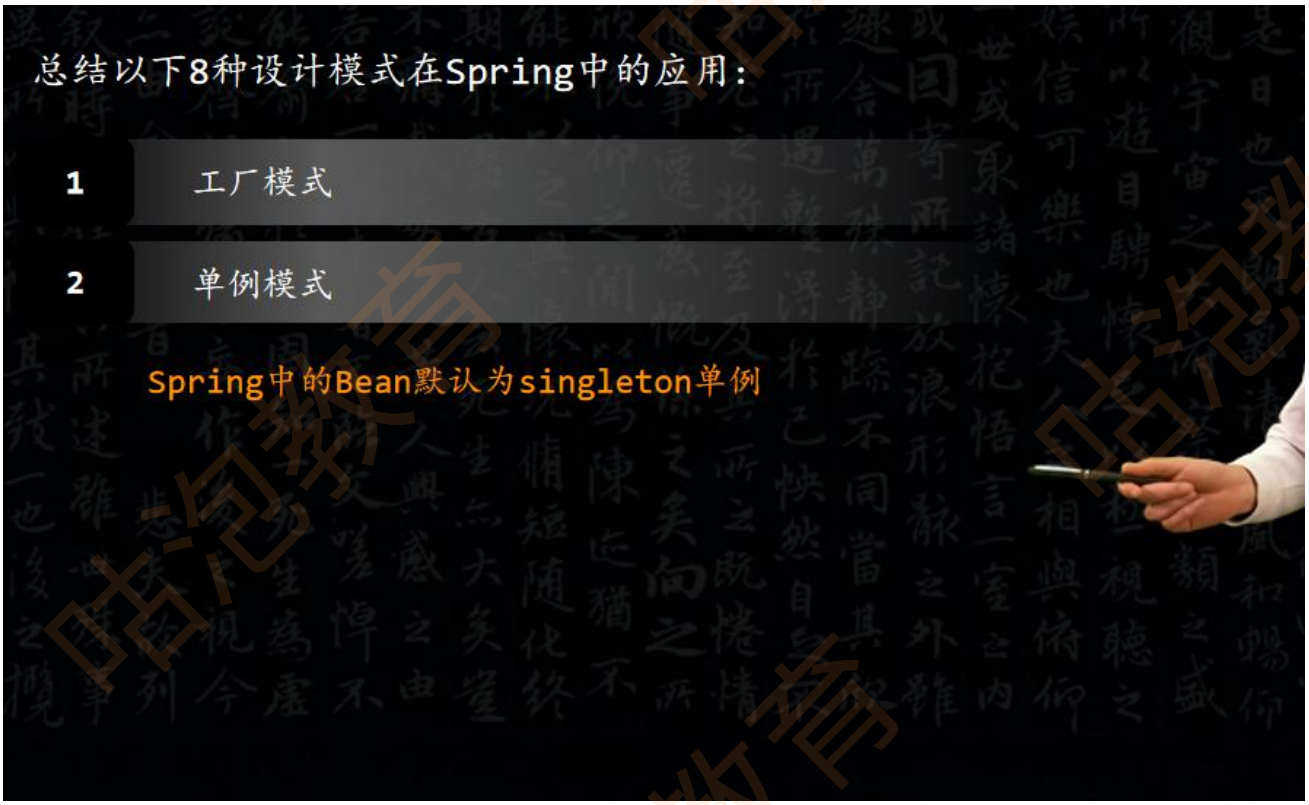
（转场，屏幕黑底白字，正中央出现“1 主要设计模式）

我给大家总结以下8种设计模式在Spring中的应用：

1、工厂模式。

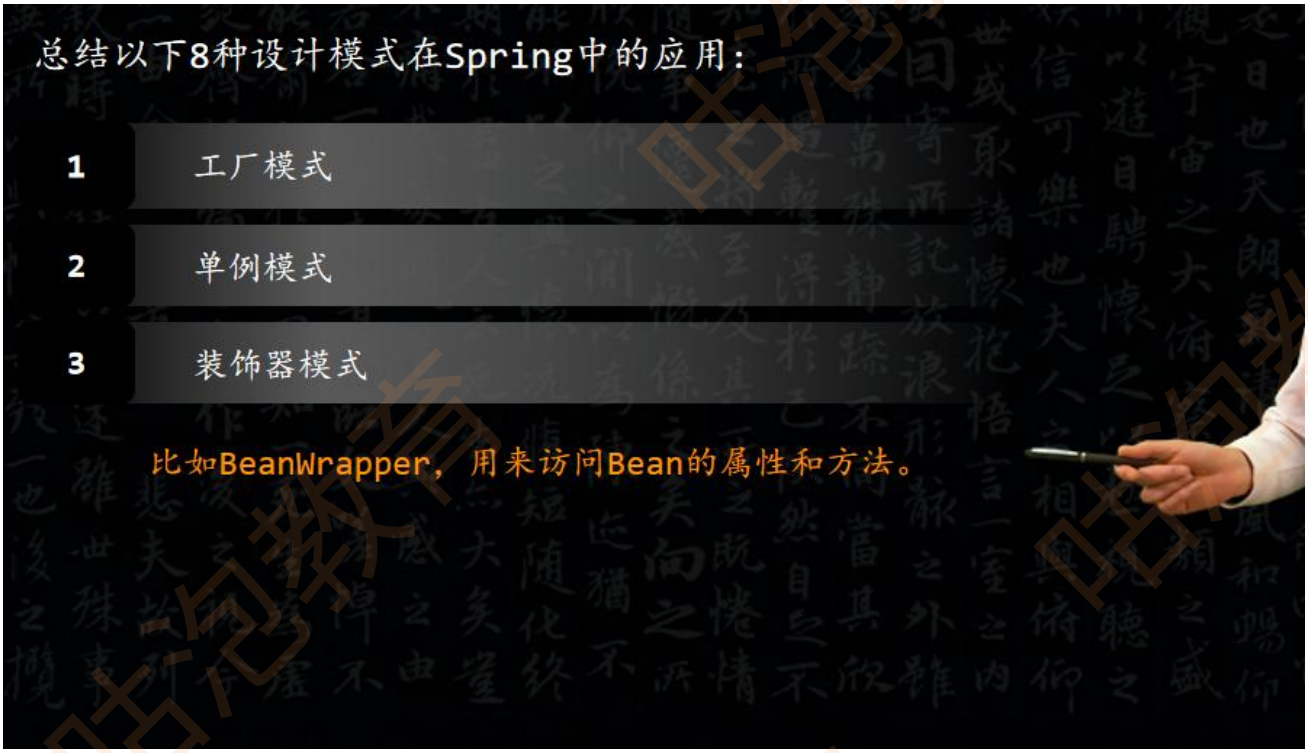
Spring的BeanFactory类，就是使用了简单工厂模式。它主要提供getBean()方法，用来创建对象的实例；我们

见得比较多的ApplicationContext也是继承自BeanFactory。



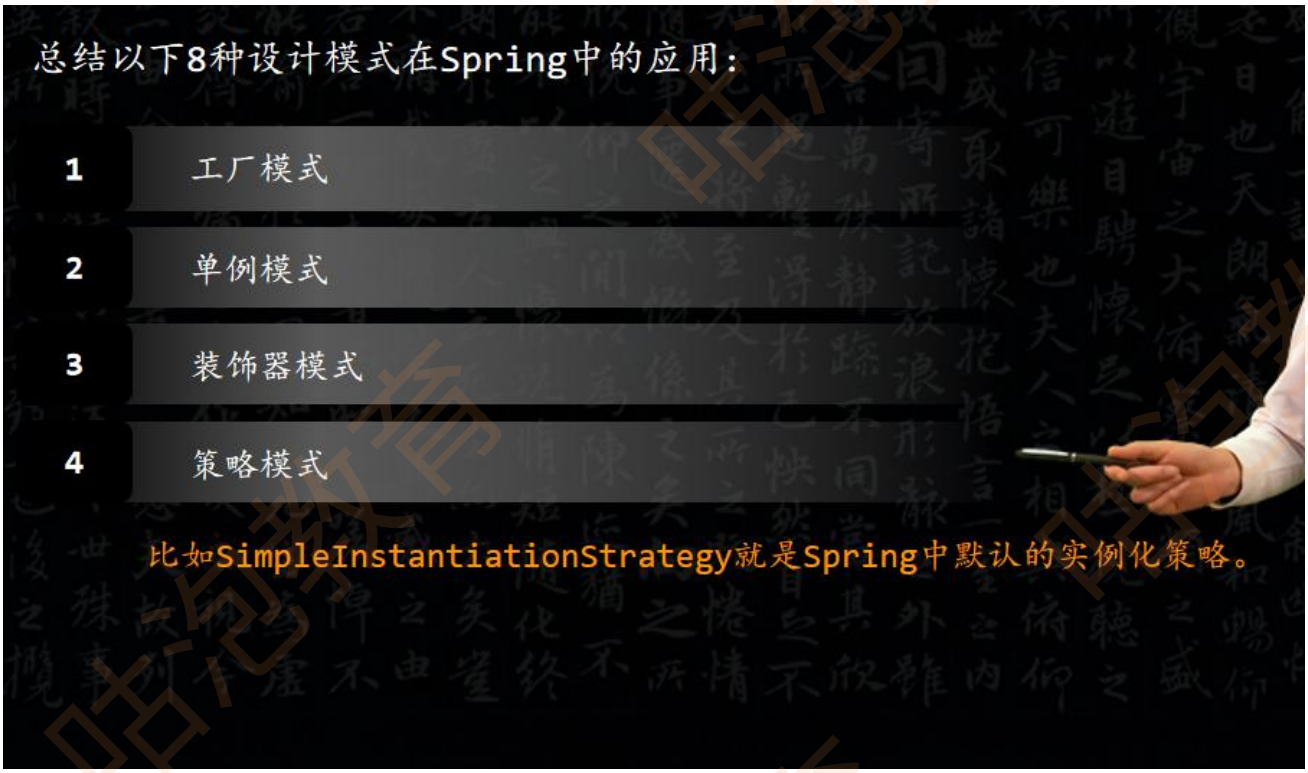
2、单例模式。

Spring中的Bean默认为singleton单例。我们可以通过配置Bean的作用域scope参数来进行修改。Spring Bean一共有5种内置的作用域，分别是singleton、prototype、request、session、globalSession。



3、装饰器模式。

在Spring中，只要见到以Wrapper命名的类基本都是使用装饰器模式。比如BeanWrapper，用来访问Bean的属性和方法。

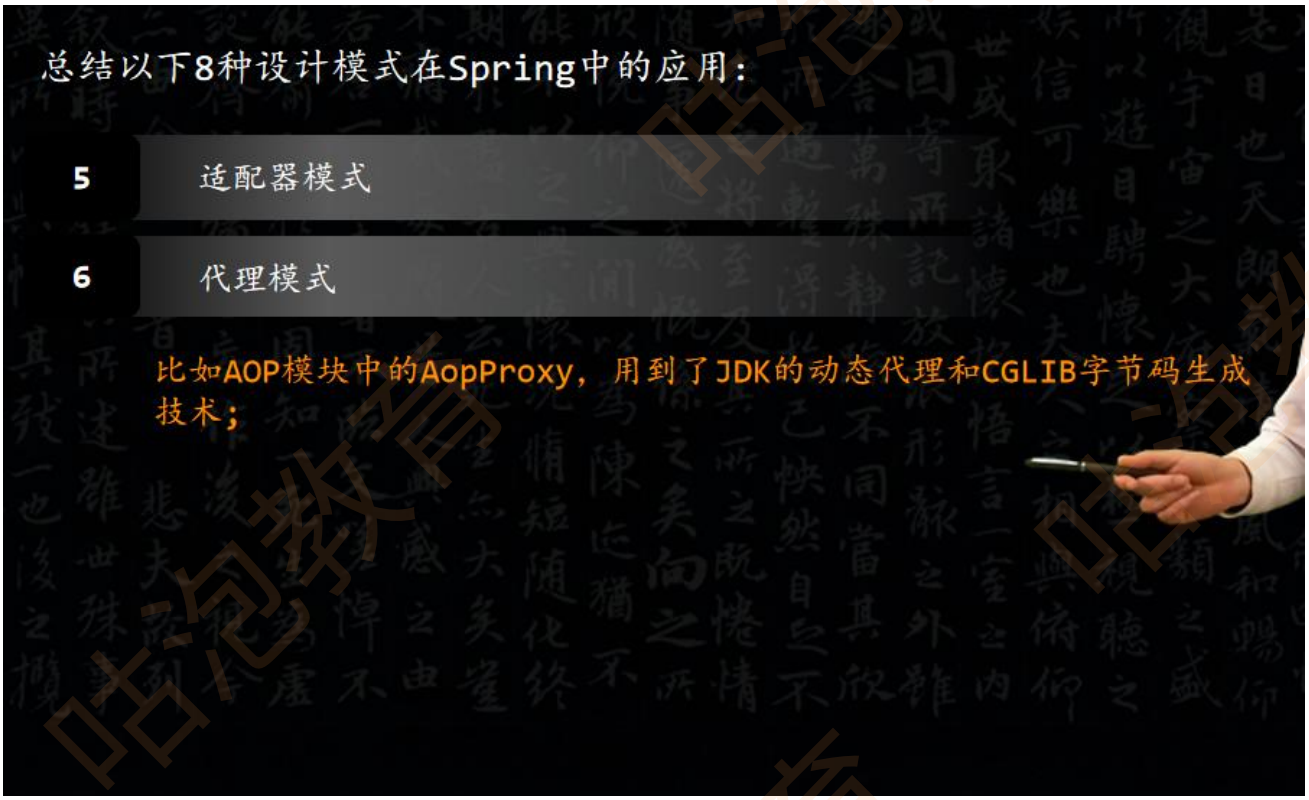


4、策略模式。

Spring中Bean的实例化采用的就是策略模式。因为Bean的实例化包含原生对象的实例化，和代理对象的实例化，不同对象实例化的逻辑也不一样，所以实例化策略也不一样，比如SimpleInstantiationStrategy就是Spring中默认的实例化策略。

5、适配器模式。

在Spring，只要是以Adapter命名的类基本都是适配器模式的应用。比如MVC模块中的HandlerAdapter。



6、代理模式。

比如AOP模块中的AopProxy，用到了JDK的动态代理和CGLIB字节码生成技术;

7、模板方法模式。

主要用来解决代码重复的问题。Spring提供了非常多的模板类来减少重复代码，基本都是以Template结尾，比如RestTemplate,JmsTemplate,JdbcTemplate。

8、观察者模式。

主要用于当一个对象的状态发生改变时，所有依赖于它的对象都会得到通知，在Spring中一般以Listener结尾，比如ApplicationListener等等。

【结尾】

当然，Spring是一个非常经典的框架，它用的的设计模式也非常多。而且很多设计模式经常是混合使用的，都是你中有我，我中有你。所以，我们在阅读代码的时候，还不能简单地断定某个类就一定是只用了某一种设计模式。

以上就是我对Spring中主要设计模式的分享。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

118、导致Spring事务失效的原因有哪些?

【导航条：引言】

一位2年工作经验的小伙伴被问到这样一道面试题，说请你谈谈导致Spring事务失效的原因有哪些?

今天，我给大家分享一下我的理解。

【导航条：*事务失效原因*】

（转场，屏幕黑底白字，正中央出现“1 事务失效原因）

下面给大家总结一下，导致Spring事务失效的原因主要有以下8个：

1、方法没有被public修饰

在开发过程中，如果@Transactional事务注解添加在不是public修饰的方法上，这个时候，Spring的事务

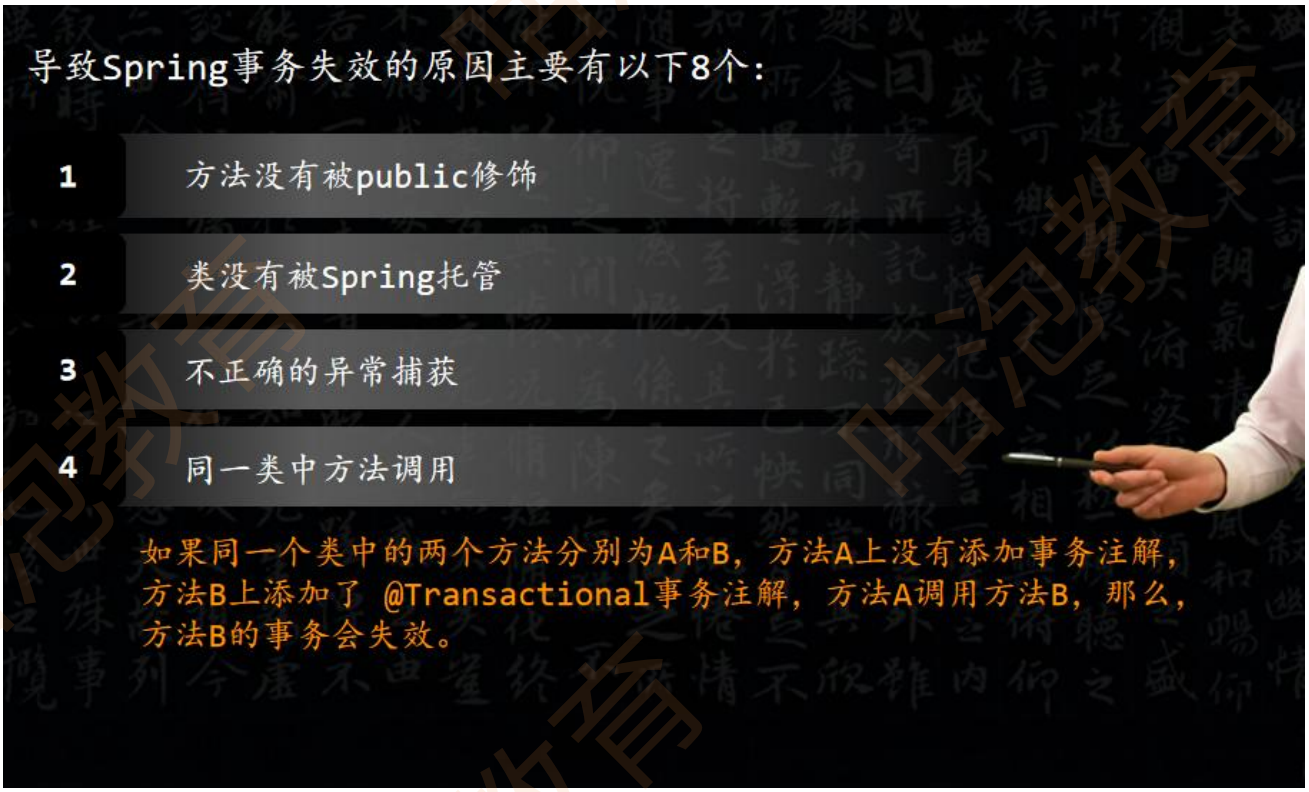
就会失效。

2、类没有被Spring托管

如果事务方法所在的类没有加载到Spring IoC容器中，也就是说，事务方法所在的类没有被Spring管理，从而导致Spring无法实现代理，所以，Spring事务也会失效。

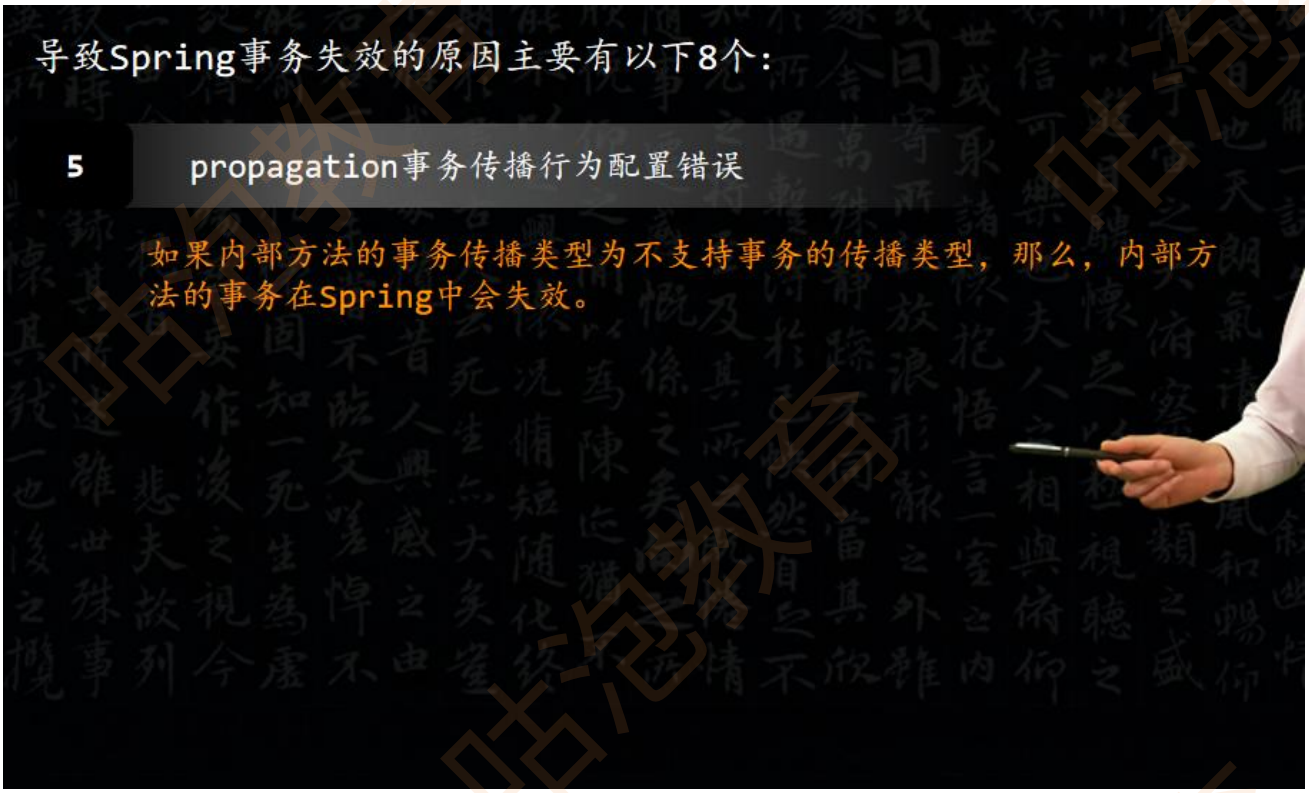
3、不正确的异常捕获

如果事务方法抛出异常被 catch 处理了，导致 @Transactional 无法回滚而导致事务失效。



4、同一类中方法调用

如果同一个类中的两个方法分别为A和B，方法A上没有添加事务注解，方法B上添加了 @Transactional事务注解，方法A调用方法B，那么，方法B的事务会失效。



5、propagation事务传播行为配置错误

如果内部方法的事务传播类型为不支持事务的传播类型，那么，内部方法的事务在Spring中会失效。

6、rollbackFor参数设置错误。

如果在@Transactional注解中rollbackFor参数标注了错误的异常类型，那么，Spring事务的回滚就无法识别，导致事务回滚失效。

7、没有配置事务管理器

即使在代码中使用了Spring的事务管理的功能，但是在项目中，如果没有配置Spring的事务管理器，Spring的事务也不会生效。

8、数据库本身不支持事务

Spring事务生效的前提是所连接的数据库要支持事务，如果底层的数据库都不支持事务，那么，Spring的事务肯定会失效。例如，如果使用的数据库为MySQL，并且选用了MyISAM存储引擎，则Spring的事务就会失效。

【结尾】

以上就是我对Spring事务失效原因的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

119、Spring中实现异步调用的方式有哪些?

【导航条：引言】

一位3年工作经验的小伙伴被问到这样一道面试题，说Spring中实现异步调用的方式有哪些?

今天，我给大家分享一下我的理解。

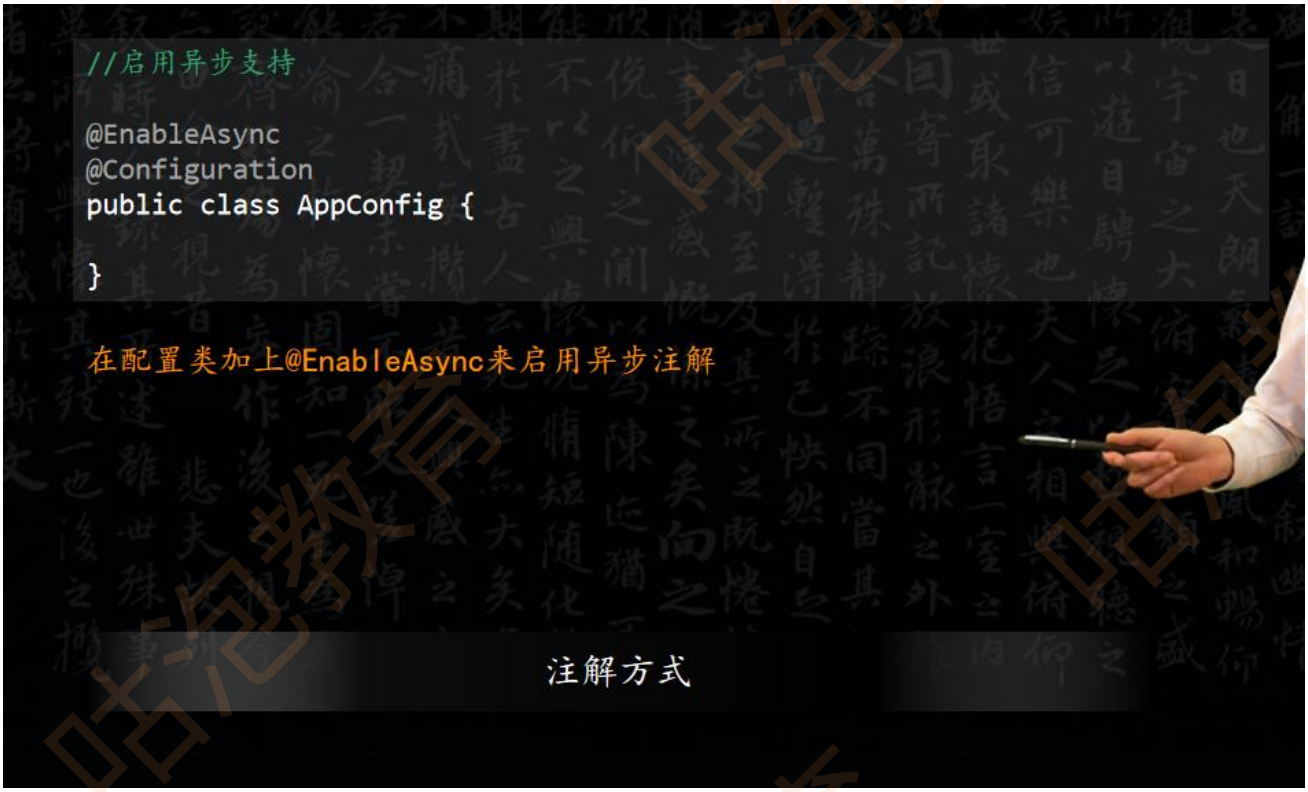
在Spring中，实现异步调用主要有三种方式，分别是注解方式、内置线程池方式和自定义线程池方式。

【导航条：注解方式】

(转场，屏幕黑底白字，正中央出现“1 注解方式)

1、注解方式

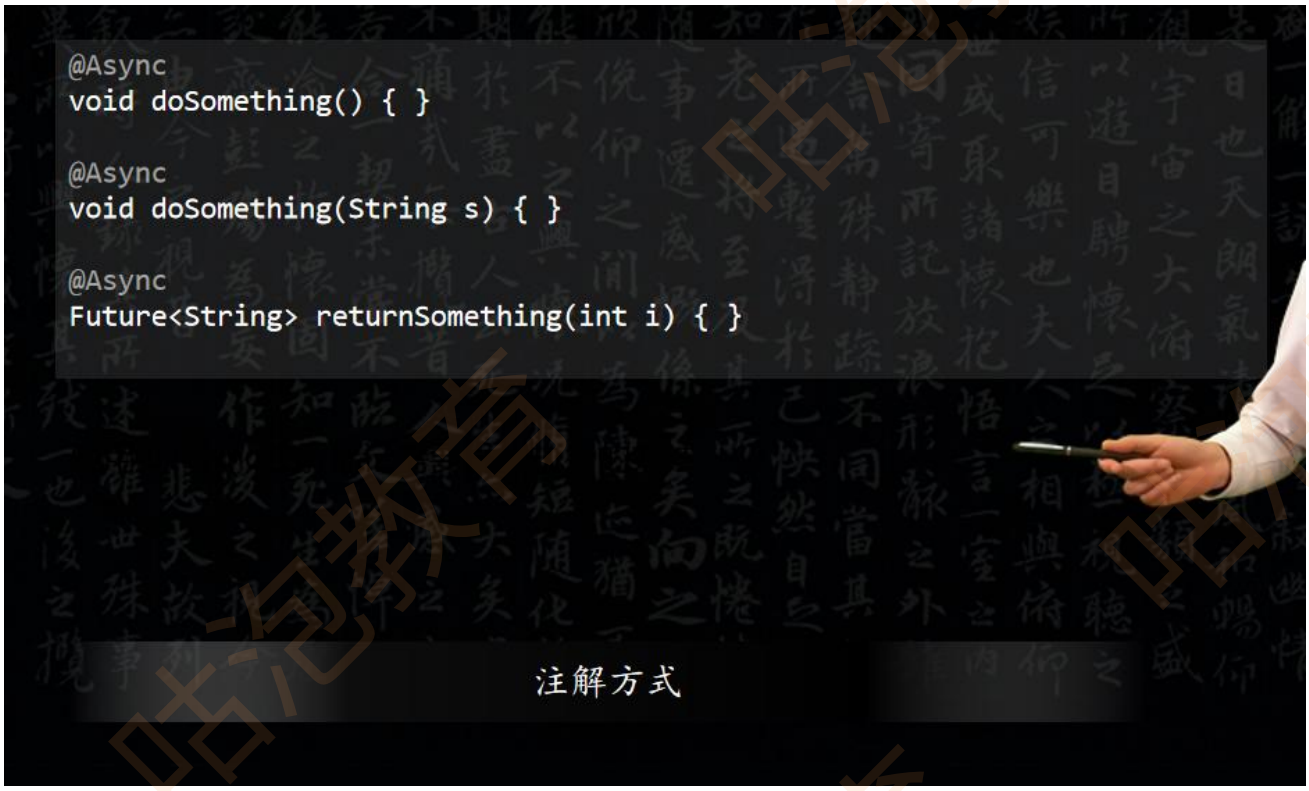
可以在配置类和方法上加特定注解。首先，在配置类加上@EnableAsync来启用异步注解，



如代码所示：

```
@EnableAsync//启用异步支持

@Configuration

public class AppConfig {

}
```

然后，使用@Async注解标记需要异步执行的方法，



如代码所示：

```
@Async

void doSomething() {

    // this will be run asynchronously

}



@Async

void doSomething(String s) {

    // this will be run asynchronously

}
```

```
@Async

Future<String> returnSomething(int i) {

   // this will be run asynchronously

}
```

使用@Async标记的异步方法可以带参数，也可以带有返回值。返回值类型必须是java.util.concurrent.Future或其子类，可以是以下3种类型：

1）由Java原生API提供的Future。

2）由Spring提供的ListenableFuture后者AsyncResult。

3）Java 8提供的CompletableFuture。

需要说明的是，@Async**默认会使用SimpleAsyncTaskExecutor来执行，而这个线程池不会复用线程**。所以，通常要使用异步处理，我们都会自定义线程池。

【导航条：内置线程池】

（转场，屏幕黑底白字，正中央出现“2 内置线程池方式）

2、内置线程池方式

可以使用Spring内置的线程池来实现异步调用，比如ThreadPoolTaskExecutor 和SimpleAsyncTaskExecutor。

Spring提供了许多TaskExecutor的内置实现。下面简单介绍5种内置的线程池。

1）SimpleAsyncTaskExecutor：它不会复用线程，每次调用都是启动一个新线程。

2）ConcurrentTaskExecutor：它是Java API中Executor实例的适配器。

3）ThreadPoolTaskExecutor：**这个线程池是最常用的**。它公开了用于配置的bean属性，并将它包装在TaskExecutor中。

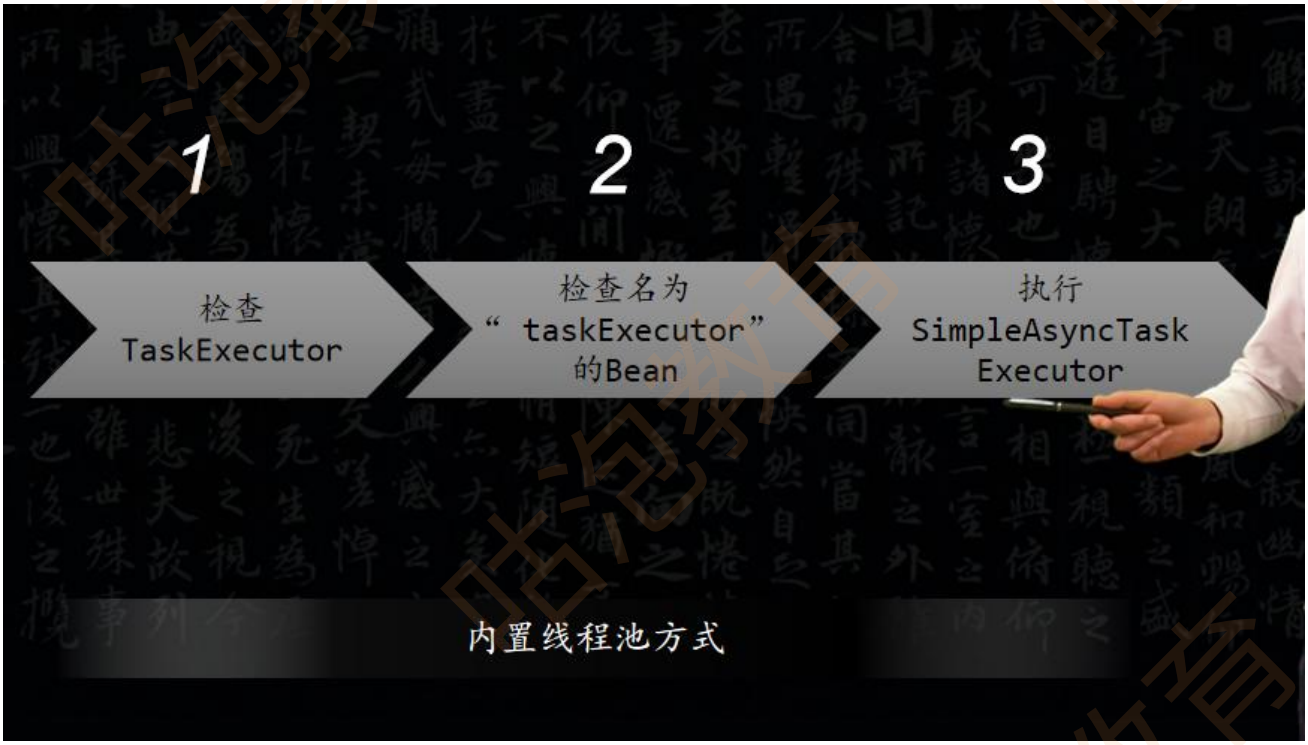
4）WorkManagerTaskExecutor：它基于CommonJ WorkManager来实现的，并且是在Spring上下文中的

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

WebLogic或WebSphere中设置CommonJ线程池的工具类。

5）DefaultManagedTaskExecutor：主要用于支持JSR-236兼容的运行时环境，它是使用JNDI获得ManagedExecutorService，作为CommonJ WorkManager的替代方案。

通常情况下，ThreadPoolTaskExecuto最为常用，只要当ThreadPoolTaskExecutor不能满足需求时，可以使用ConcurrentTaskExecutor。如果在代码中声明了多个线程池，Spring会默认按照以下搜索顺序来调用线程池：



第一步，检查上下文中的**唯一**TaskExecutor Bean。

第二步，检查名为“taskExecutor”的Executor Bean。

第三步，以上都无法无法处理，就会使用SimpleAsyncTaskExecutor来执行。

【导航条：自定义线程池】

（转场，屏幕黑底白字，正中央出现“3 自定义线程池方式）

3、自定义线程池方式

可以通过实现AsyncConfigurer接口或者直接继承AsyncConfigurerSupport类来自定义线程池。但是非完全托管的Bean和完全托管的Bean实现方式有点小差异。

首先，来看非完全托管的Spring Bean，实现方式如代码所示：

```
@Configuration

@EnableAsync

public class AppConfig implements AsyncConfigurer {



   @Override

   public Executor getAsyncExecutor() {

      ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();

      executor.setCorePoolSize(7);

      executor.setMaxPoolSize(42);

      executor.setQueueCapacity(11);
```

```
        executor.setThreadNamePrefix("MyExecutor-");
        executor.initialize();//手动初始化
        return executor;
    }


    @Override
    public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
        return new MyAsyncUncaughtExceptionHandler();
    }
}
```

在这段代码中，ThreadPoolTask Executor不是完全托管的Spring bean。

然后，来看完全托管的Spring Bean，实现方式如代码所示：

```
@Configuration

@EnableAsync

public class AppConfig implements AsyncConfigurer {



    @Bean

    @Override

    public Executor getAsyncExecutor() {

        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();

        executor.setCorePoolSize(7);

        executor.setMaxPoolSize(42);
```

```
        executor.setQueueCapacity(11);
        executor.setThreadNamePrefix("MyExecutor-");
        //executor.initialize();//不用手动调用
        return executor;
    }

    @Override
    public AsyncUncaughtExceptionHandler getAsyncUncaughtExceptionHandler() {
        return new MyAsyncUncaughtExceptionHandler();
    }
}
```

只要在异步方法上添加@Bean注解，不需要手动调用线程池的initialize()方法，在Bean在初始化之后会自动调用。需要注意的是，在同级类中直接调用异步方法无法实现异步。

【结尾】

以上就是我对Spring实现异步调用的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

120、Spring中有几种依赖注入的方式?

【导航条：引言】

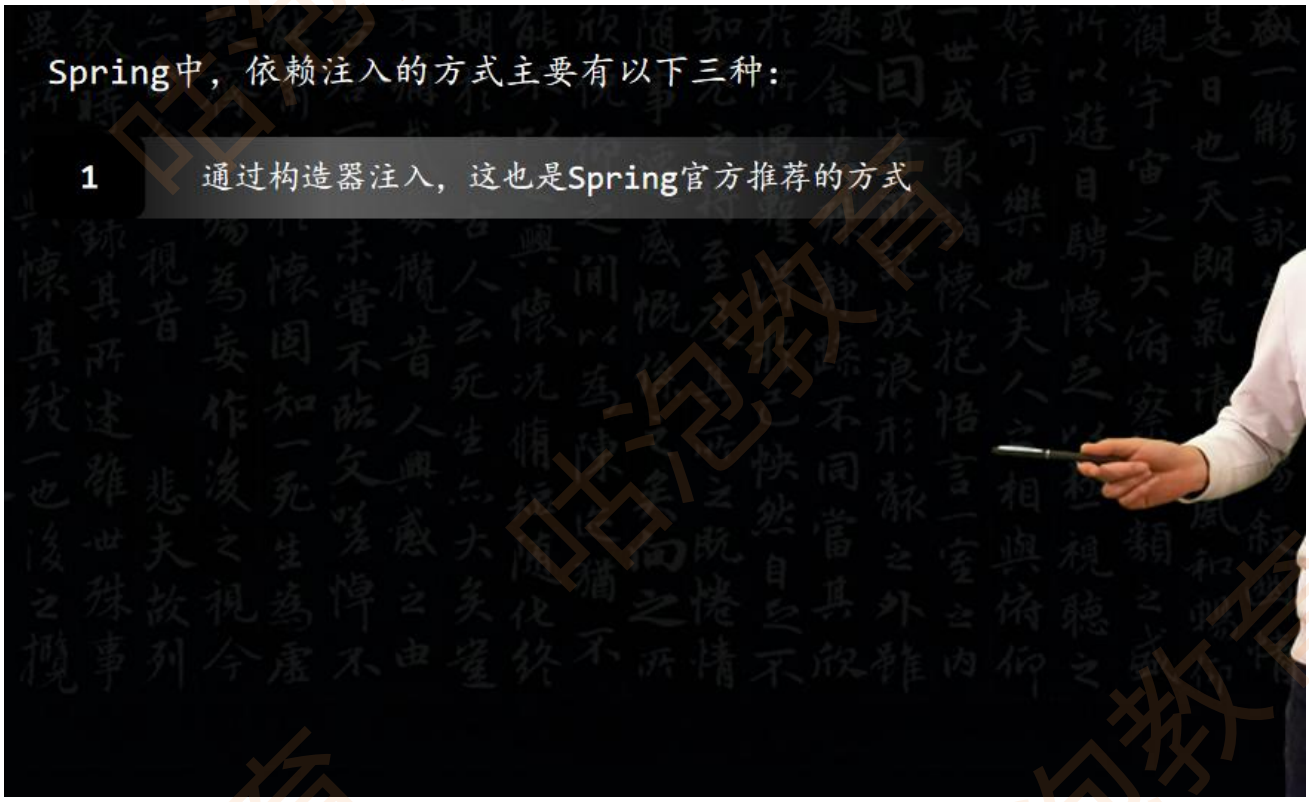
一位应届毕业生被问到这样一道面试题，说Spring中有几种依赖注入的方式?

今天，我给大家分享一下我的理解。

【导航条：Spring依赖注入方式】

（转场，屏幕黑底白字，正中央出现“Spring依赖注入方式）

Spring中，依赖注入的方式主要有以下三种：



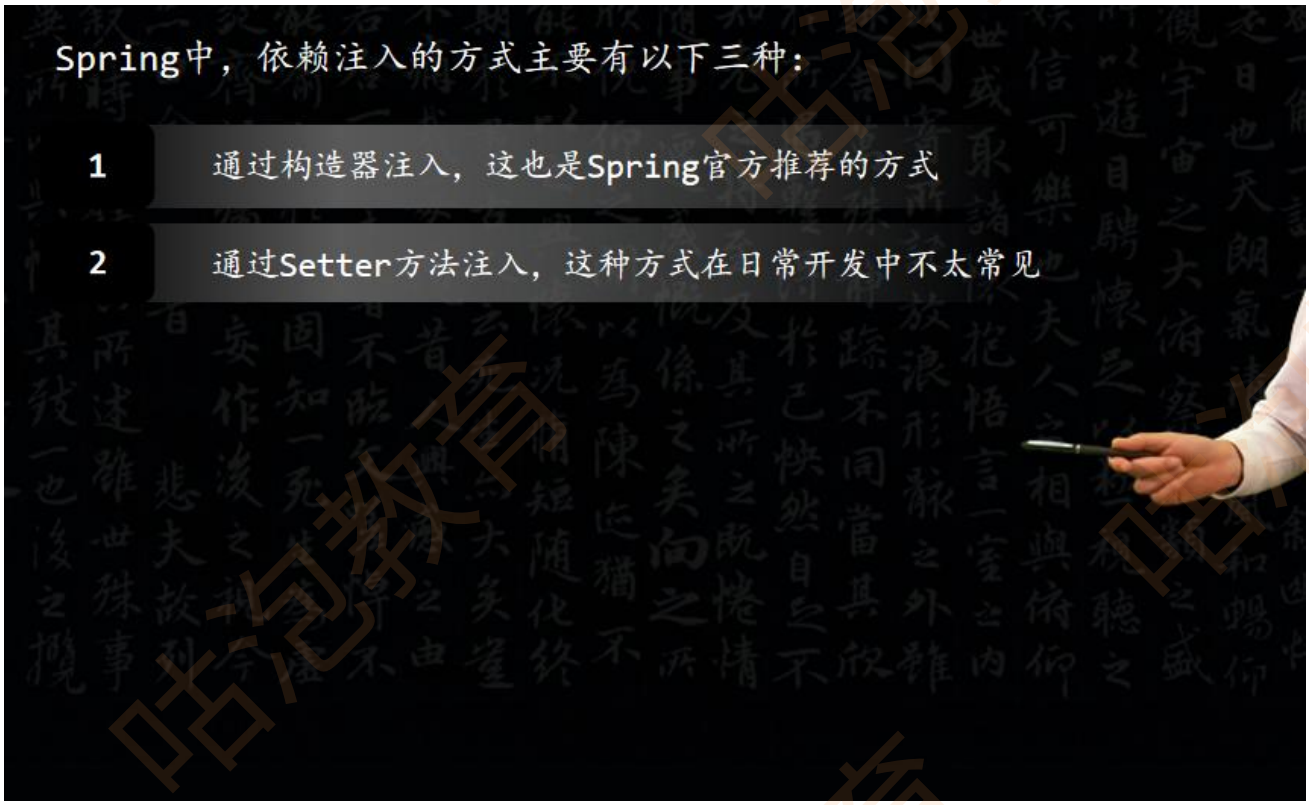
1、通过构造器注入，这也是Spring官方推荐的方式，如代码所示：

```
private DemoService demoService;



@Autowire

public DemoController (DemoService demoService){

   this.demoService = demoService;

}
```

这种方式，在注入对象很多的情况下，构造器参数列表会很长，所以，不够灵活。

当然，好处就是，对象初始化完成后，就可可获得可使用的对象。

另外，还可以检测循环依赖。



2、通过Setter方法注入，这种方式在日常开发中不太常见，如代码所示：

```
private DemoService demoService;

@Autowire

public setDemoService (ExampleService demoService){

    this.demoService = demoService;

}
```

这种方式是利用set方法来注入，可以确保注入前不依赖Spring容器。每个set方法单独注入一个对象，便于灵活控制，并且可以实现选择性注入。当然，也可以检测循环依赖。

3、通过Filed属性注入，比如用@Autowired 、 @Resource 等这些注解，如代码所示：

```
@Autowire

private DemoService demoService;
```

这种方式，控制了对象的外部可见性，也就是说，不是被Spring容器托管的对象，无法自动注入。

但是，Filed注入的方式不能被检测是否出现依赖循环。还有就是被final修饰的属性，无法赋值。

【结尾】

以上就是我对Spring依赖注入方式的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

121、谈谈你对Spring MVC中九大组件的理解

【导航条：引言】

一位应届毕业生被问到这样一道面试题，说谈谈你对Spring MVC中的九大组件的理解。

今天，我给大家分享一下我的理解。

【导航条：九大组件】

（转场，屏幕黑底白字，正中央出现“1 Spring MVC九大组件）

1、Spring MVC九大组件

使用Spring MVC框架时，它的主要入口是DispatcherServlet类，Spring MVC子容器初始化时，会调用DispatcherServlet的onRefresh()方法，而onRefresh()方法只做了一件事，就是调用initStrategies()方法来初

始化Spring MVC的九大组件，如源码所示：

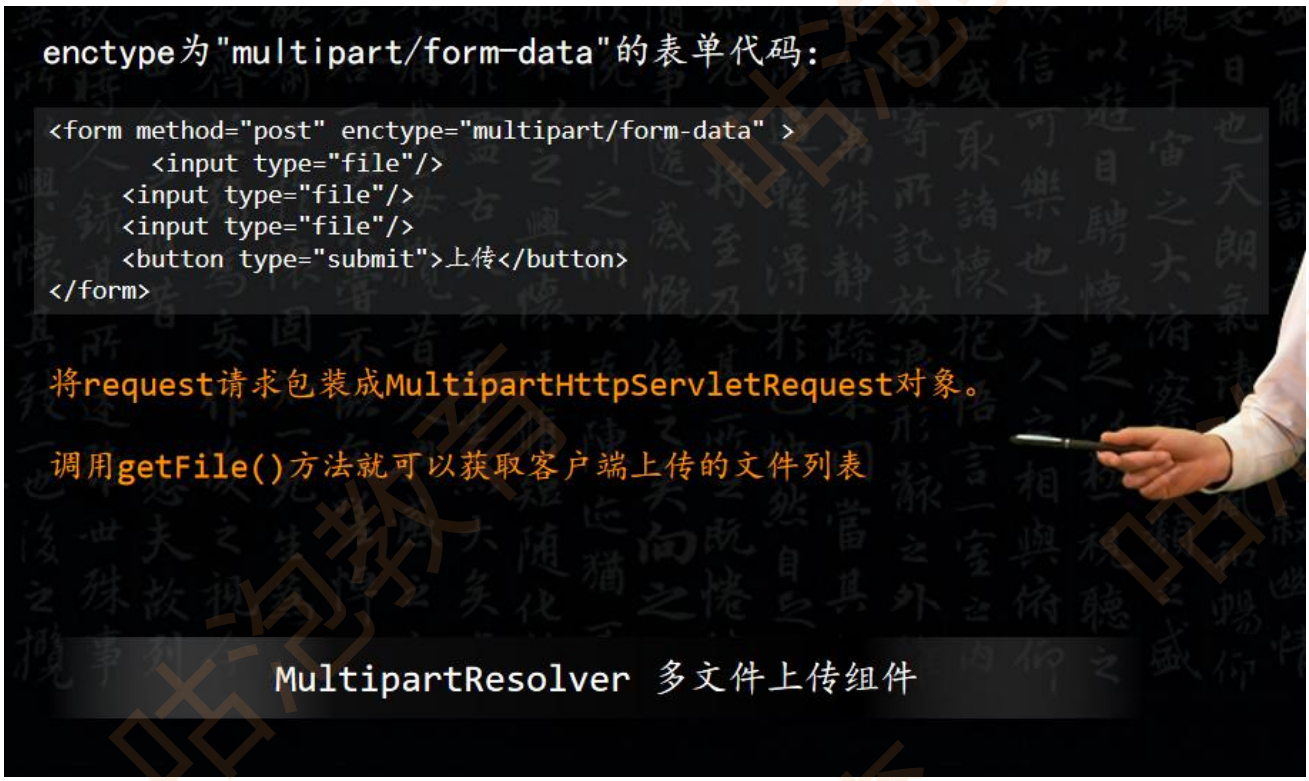
```
protected void onRefresh(ApplicationContext context) {

this.initStrategies(context);

}



protected void initStrategies(ApplicationContext context) {

this.initMultipartResolver(context);

 this.initLocaleResolver(context);

 this.initThemeResolver(context);

 this.initHandlerMappings(context);

 this.initHandlerAdapters(context);

 this.initHandlerExceptionResolvers(context);

 this.initRequestToViewNameTranslator(context);

 this.initViewResolvers(context);

 this.initFlashMapManager(context);

}
```

顾名思义，initStrategies()方法直译过来就是初始化策略，Spring MVC把九大组件设计成九大策略，其实就是为了明确各个组件的职责，达到解耦的目的。

Spring MVC的九大组件按照初始化顺序分别为：MultipartResolver多文件上传组件、LocaleResolver多语言支持组件、ThemeResolver主题模板处理组件、HandlerMappings URL映射组件、HandlerAdapters业务逻辑适配组件、HandlerExceptionResolvers异常处理组件、RequestToViewNameTranslator视图名称提取组件、ViewResolvers视图渲染组件和FlashMapManager闪存管理组件。

下面给大家详细分析一下，每个组件的功能和职责。

1、MultipartResolver 多文件上传组件。



用于支持多文件上传，如代码所示：

```
<form method="post" enctype="multipart/form-data" >
    <input type="file"/>
   <input type="file"/>
   <input type="file"/>
   <button type="submit">上传</button>
</form>
```

主要逻辑就是将enctype为"multipart/form-data"的表单request请求包装成MultipartHttpServletRequest。程序员在开发的时候，只需要调用MultipartHttpServletRequest的 getFile()方法，就可以获取客户端上传的文件列表了。

2、LocaleResolver多语言支持组件。

用于支持国际化多语言切换，LocaleResolver的主要作用就是从 request 中解析出 local 参数的值，如源码所示：

```
public interface LocaleResolver {

Locale resolveLocale(HttpServletRequest request);

 void setLocale(HttpServletRequest request, HttpServletResponse response, Locale local);}

}
```

resolveLocale()方法是从 request 中解析出 local，setLocale()方法是将指定的 local 值设置到 request 中。而 local 大多数情况下都是用来做国际化处理的，配合多语言字典properties来使用，例如中国的Local值

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

为zh_CN。

3、ThemeResolver主题模板处理组件。

主要用于支持Web页面的多主题风格。可以通过ThemeResolver来读取和解析页面主题样式配置。实现原理和LocaleResolver类似，也是配置一套 properties 文件，根据不同参数来切换读取；当然，使用ThemeResolver也是可以实现国际化。如源代码所示：

```
public interface ThemeResolver {

String resolveThemeName(HttpServletRequest request);

void setThemeName(HttpServletRequest request, HttpServletResponse response, String themeName);
```

```
}
```

它的主要方法也和LocaleResolver类似，一个从request提取主题名称的方法，一个设定主题名称的方法。

4、HandlerMappingURL映射组件。

主要是用来保存Url和业务逻辑的对应关系，它本质上就是一个Map，Key为URL值就是对应Controller中配置了@RequestMapping注解的方法。但是在Spring源码中，被封装成了一个HandlerMapping对象。然后，每个HandlerMapping对象都被缓存在一个List中。如源码所示：

```
private List<HandlerMapping> handlerMappings;


private void initHandlerMappings(ApplicationContext context) {
```

```
    this.handlerMappings = null;



    if (this.detectAllHandlerMappings) {
        Map<String, HandlerMapping> matchingBeans =
                BeanFactoryUtils.beansOfTypeIncludingAncestors(context, HandlerMapping.class, true, false);
        if (!matchingBeans.isEmpty()) {
            this.handlerMappings = new ArrayList<>(matchingBeans.values());
            AnnotationAwareOrderComparator.sort(this.handlerMappings);
        }
    }
    ...
}
```

5、HandlerAdapter业务逻辑适配组件。

主要功能是动态解析参数以及动态适配业务逻辑对应的Handler，如源码所示：

```
public interface HandlerAdapter {

boolean supports(Object handler);

ModelAndView handle(HttpServletRequest request, HttpServletResponse response, Object handler) throws

Exception;

long getLastModified(HttpServletRequest request, Object handler);

}
```

在HandlerAdapter中提供了一个叫做handle()这样一个方法，第三个参数 Object handler 第三个参数其实就是业务处理器。在DispatcherServlet中的doDispath()方法中被调用。而handler对象就是根据用户请求的Url从HandlerMapping获取的HandlerMapiping对象。

在HandlerApdater的handle()方法中，首先会动态解析用户传过来的参数，并完成数据类型转化。然后，反射调用HandlerMapiping封装的Controller中的方法，最后，将调用方法的返回结果统一封装为ModelAndView。

6、HandlerExceptionResolver异常处理组件。

主要用于拦截对不同异常的个性化处理，Spring可以给不同的异常配置不同的ModelAndView，HandlerExceptionResolver根据异常类型，的将处理封装为一个ModelAndView从而将异常信息转换为更加友好的Web页面展示，如源码所示：

```
public interface HandlerExceptionResolver {

ModelAndView resolveException(HttpServletRequest request, HttpServletResponse response, Object handler, Exception ex);

}
```

HandlerExceptionResolver组件只有一个方法，就是将异常解析为ModelAndView。

当然，HandlerExceptionResolver自己发生异常或者在异常页面渲染过程中发生异常

HandlerExceptionResolver不会处理。Spring可以配置一个全局的500页面或者404页面来处理这个问题。

7、RequestToViewNameTranslator视图名称提取组件。

这个组件的主要功能是可从request中提取viewName。这个viewName设置在url参数上，也可以设置在request的header上。如源码所示：

```
public interface RequestToViewNameTranslator {

String getViewName(HttpServletRequest request) throws Exception;

}
```

这个其实还是挺有意思的，就是将 request 请求转换为视图名称。它只有一个getViewName()方法。

8、ViewResolvers视图渲染组件。

它的作用相当于模板引擎，就是根据视图名称找到视图对应的模板文件，然后进行解析，如源码所示：

```
public interface ViewResolver {

  View resolveViewName(String viewName, Locale local) throws Exception;

}
```

ViewResolvers组件只有一个resolveViewName()方法，

我们看到resolveViewName()方法有两个参数。

第一个参数viewName，是String类型，它其实就是视图名称，对应的就是模板文件的名称。第二个参数local，前面我们讲过代表的是本地语言环境，可以用来做国际化。

resolveViewName()方法的放回值是一个View对象。而View对象就是用来渲染页面的，也就是说将程序返回的结果填入到具体的模板里面，生成具体的视图文件，比如：jsp，ftl，html 等等。

9、FlashMapManage闪存管理组件。

它相当于一个参数缓存器，用来保证请求跳转过程中参数不丢失，和Struts 2中的ValueStack值栈非常类似。主要是 redirect重定向的时候，参数传递会丢失，FlashMapManage就能大显身手，可以做到Redirect重定向和Forward转发同样的效果，如源码所示：

```
public interface FlashMapManager {

FlashMap retrieveAndUpdate(HttpServletRequest request, HttpServletResponse response);

void saveOutputFlashMap(FlashMap flashMap, HttpServletRequest request, HttpServletResponse

response);

}
```

FlashMapManage主要有两个方法，

retrieveAndUpdate()方法是用来恢复参数的，而且对于恢复过的和超时的参数都会被删除掉。saveOutputFlashMap() 这个方法是用来保存参数的。

FlashMapManager默认会将参数保存在 Session 中，在日常开发中，如果不想将参数暴露在 Url路径中，那就可以在请求转发时，在参数中添加@RedirectAttributes注解将参数缓存，然后在下一个处理器中就可以获取到。

【结尾】

以上就是我对Spring MVC中的九大组件的理解。

需要注意的是ModelAndView和View并不属于MVC的九大组件之中，ModelAndView只是对ViewName和Model的封装，然后作为返回值把信息反馈给用户。并没有包含任何执行逻辑。而View只是对模板文件的封装，它是用作参数来传递。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

122、Java保证线程安全的方式有哪些?

【导航条：引言】

一位工作5年的小伙伴面试时被问到这样一道题，说Java保证线程安全的方式有哪些?

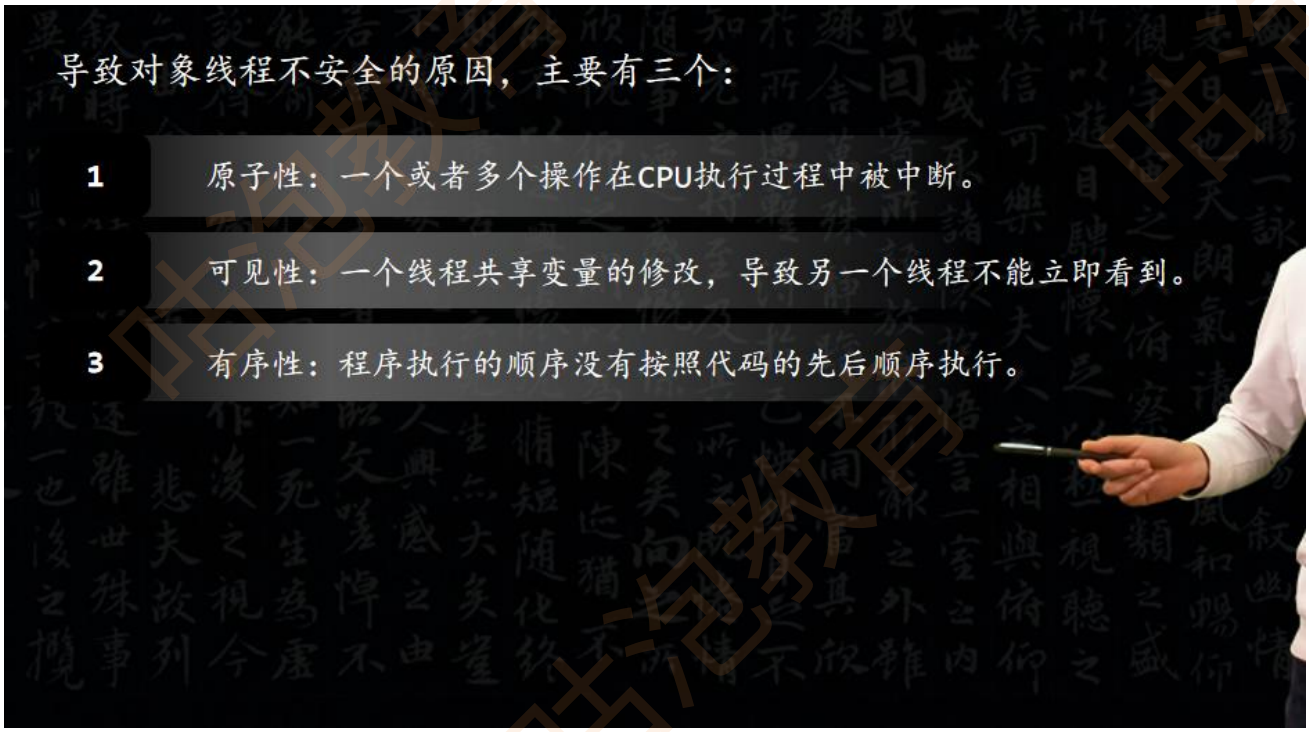
今天，我给大家分享一下我的理解。

【导航条：线程不安全的原因】

（转场，屏幕黑底白字，正中央出现“1 线程不安全的原因）

1、线程不安全的原因

回答这个问题之前，得先了解导致对象线程不安全的原因，主要有三个：

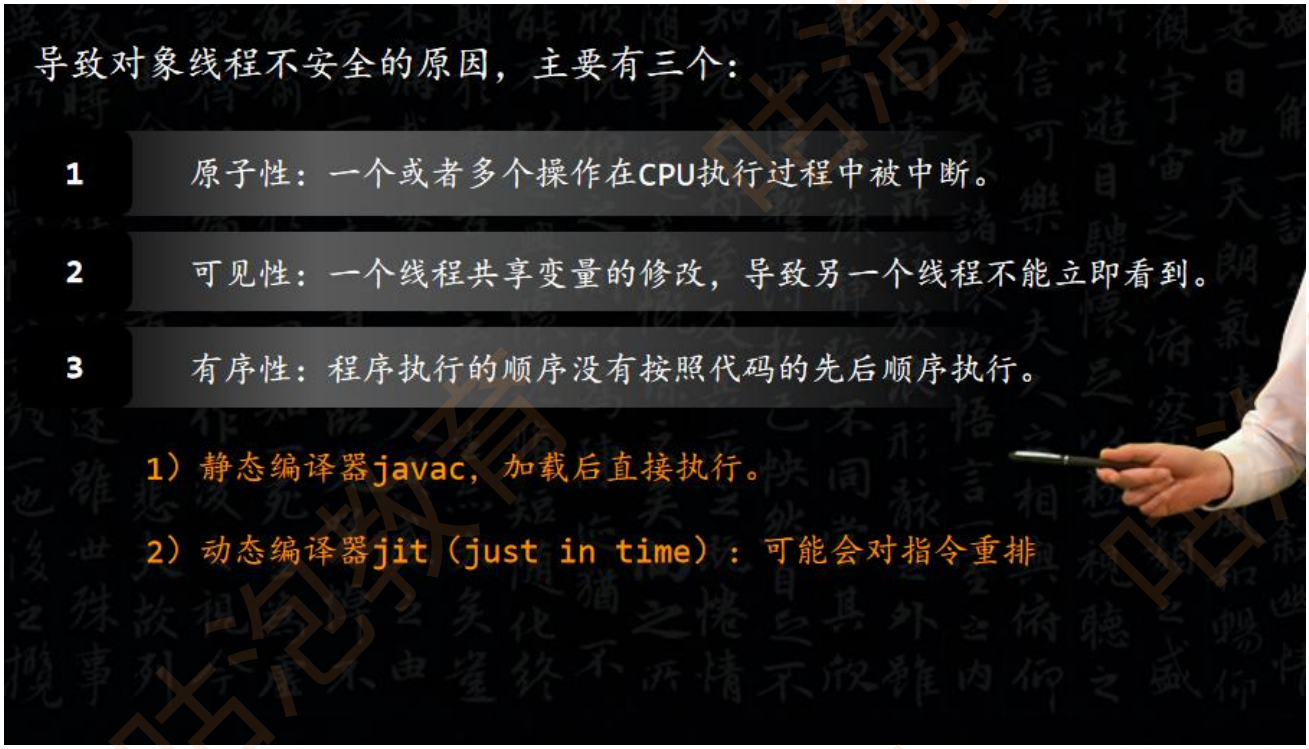


1、原子性：一个或者多个操作在CPU执行过程中被中断。

2、可见性：一个线程对象共享变量的修改，导致另一个线程不能立即看到。

3、有序性：程序执行的顺序没有按照代码的先后顺序执行。

原子性和可见性比较容易理解，重点分析一下有序性。为什么程序执行的顺序会和代码的编写顺序不一致呢？这就得理解Java平台的两种编译器，静态编译器javac和动态编译器jit（just in time）。



静态编译器是将.java文件编译成.class文件，JVM加载后就可以执行了。

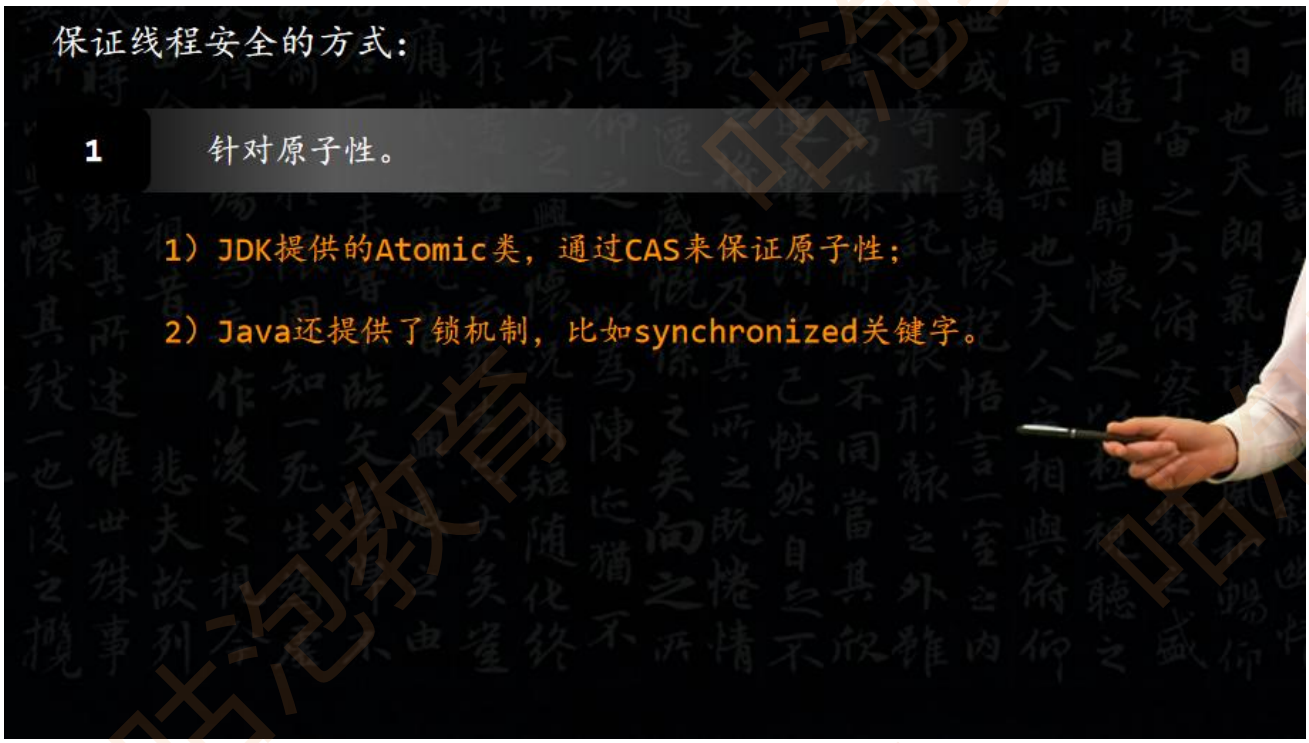
而动态编译器是要将.class文件编译成机器码，再由JVM执行。有时候，动态编译器为了程序的整体性能会对指令进行重排序，但是，这又会导致源代码中指定的内存访问顺序和实际的执行顺序不一致，就会出现线程不安全的问题。

【导航条：如何保证线程安全】

（转场，屏幕黑底白字，正中央出现“2 如何保证线程安全？）

2、如何保证线程安全

那么，针对以上三种情况，如何保证对象的线程安全呢？



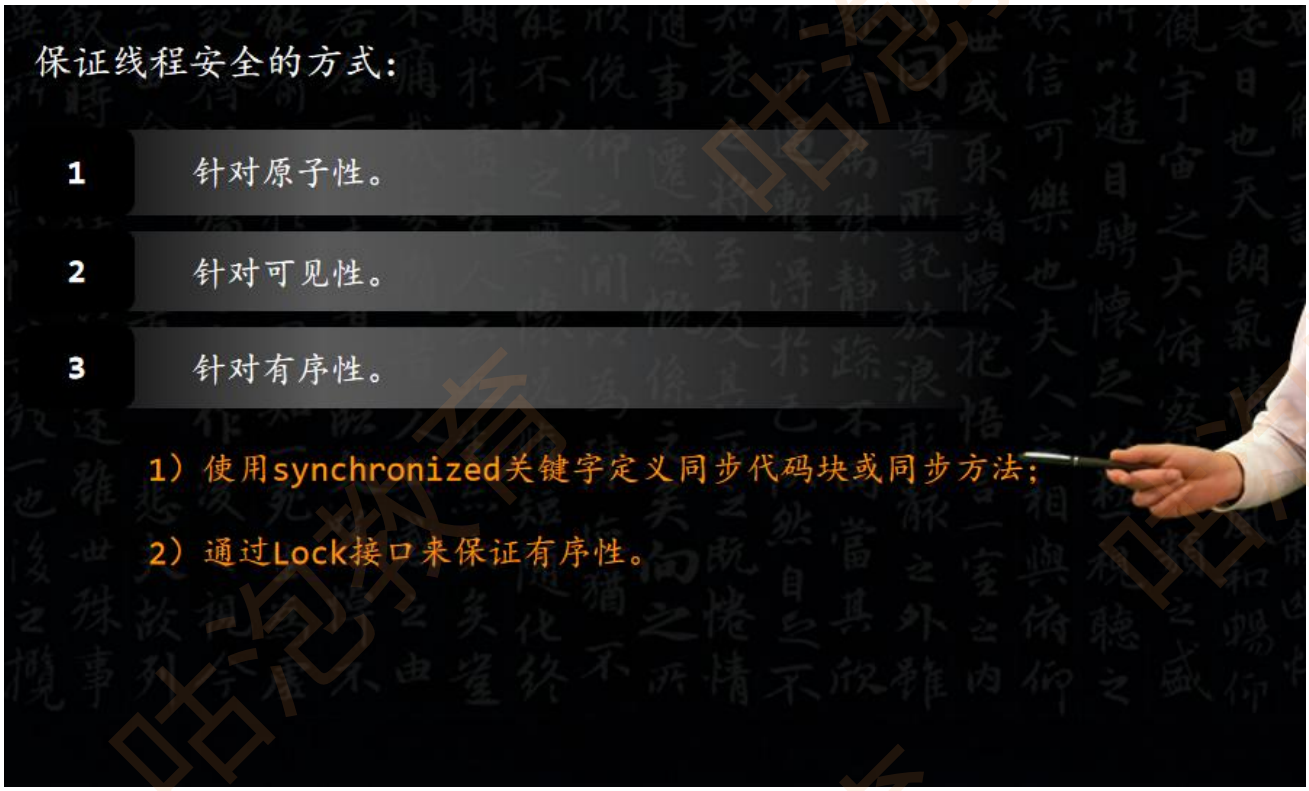
第1个，针对原子性。

1）JDK提供了非常多的Atomic类，比如AtomicInteger、AtomicLong、AtomicBoolean等等。这些类都是通过CAS来保证原子性。

2）另外，Java还提供了各种锁机制，来保证锁内的代码块在同一时刻只能被一个线程执行。比如用synchronized加锁。这样，就可以保证一个线程对资源进行读、写操作时，其他线程不可以对这个资源进行操作，从而保证了线程安全。

第2个，针对可见性。

同样可以使用synchronized关键字加锁来解决。与此同时，Java提供了volatile关键字。它要优于synchronized的性能，同样也可以保证修改后对其他线程可见。volatile一般用于对变量的写操作，不依赖于当前值的场景中，比如状态标记量等等。



第3个，针对有序性。

也可以使用synchronized关键字定义同步代码块，或者同步方法来保证有序性。另外也可以通过Lock接口来保证有序性。

【结尾】

以上就是对Java保证线程安全的思路。当然，保证对象线程安全的方式还有很多，比如还可以使用ThreadLocal实现多个线程之间的数据隔离，使用final关键字等等，我这里就不一一列举了。最后，我留一个思考题，单用volatile关键字，能保证线程安全吗？小伙伴们也可以在评论区帮忙补充一下。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

123、前后端分离后，Java Web开发如何解决跨域问题

【导航条：引言】

做Web开发，经常会遇到跨域问题，小伙伴们在面试中，也经常被问到。这不，又有一位工作3年的小伙伴被问到这样一道题，说前后端分离后，如果解决跨域问题。

今天，我给大家分享一下我的理解。

这个问题也有很多小伙伴单独问过我，很多小伙伴知道如何解决跨域问题，但是却说不清楚跨域到底是怎么产生的。所以，回答跨域解决方案之前，我们先来介绍一下跨域产生的原因。

【导航条：产生原因】

（转场，屏幕黑底白字，正中央出现“1 产生原因）

1、产生原因

因为一般的浏览器都有一个安全机制，叫做同源策略限制。所谓同源策略就是指用户输入的URL中包含的协议、域名、端口都完全相同。也就是说，我们使用浏览器访问网页时，必须符合同源策略的请求才能访问。如果有一项不同，浏览器会觉得有安全风险，就不想让你使用这个接口的数据。

比如，在http://localhost:8080/index 页面中，用Ajax访问https://localhost:8081/index.json接口数据的时候，这两个URL的协议和端口不相同，也就是不同源，这就产生了跨域访问。当然，浏览器还是会将这个请求发送到后台服务器，但是，浏览器不会接收服务器响应结果。

（需要配图）

举个更通俗的例子，就好比你去肯德基点餐，非要点一碗兰州拉面，店员虽然很鄙视你，但他还是会打电话问一下兰州拉面馆问一下，兰州拉面馆说不给肯德基供货。所以，你没有吃到兰州拉面。这其中，有个反常的操作，就是虽然你点的兰州拉面，但是肯德基店员还是帮你打电话问了，是拉面馆不给肯德基供货的，如果拉面馆说给肯德基供货而且把面送过来了，那么你就能吃到兰州拉面了。

这个案例中，店员就相当于是浏览器，肯德基呢就相当于当前看到的网页，兰州拉面馆就是相当于后台服务的接口。肯德基和兰州拉面不是同一个老板，相当于是不同源。拉州拉面就是你想要的接口数据。

但如果使用Postman等开发工具进行交互是不会出现跨域问题的，这是浏览器特有的限制。

其实，跨域问题也并不是前后端分离后才有的，后端开发的程序员一般都遇到过跨域问题。只是前后端分离开发以后，前端开发体现跨域问题更加明显了，经常要找后端开发人员来解决。

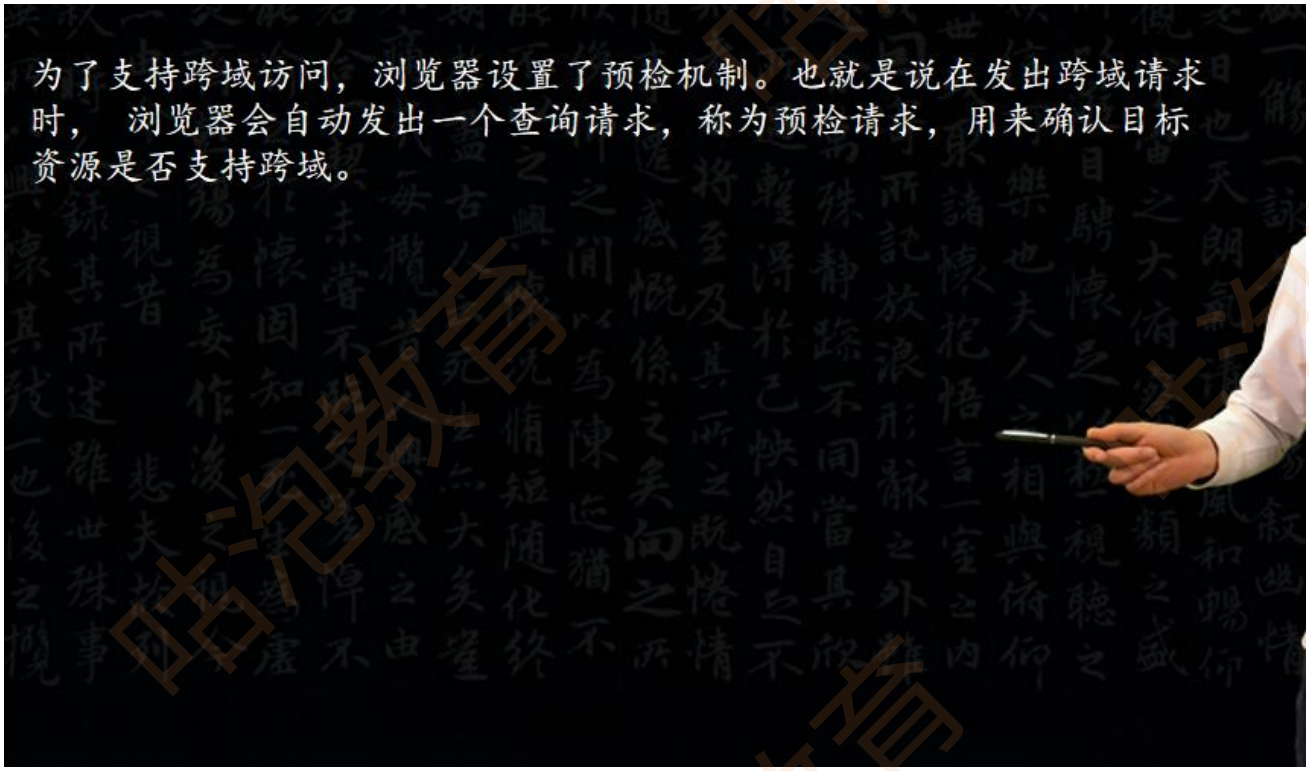
【导航条：预检请求】

（转场，屏幕黑底白字，正中央出现“2 预检请求）

2、预检请求

为了支持跨域访问，浏览器设置了预检机制。也就是说在发出跨域请求时， 浏览器会自动发出一个查询请

求，称为预检请求， 用来确认目标资源是否支持跨域。

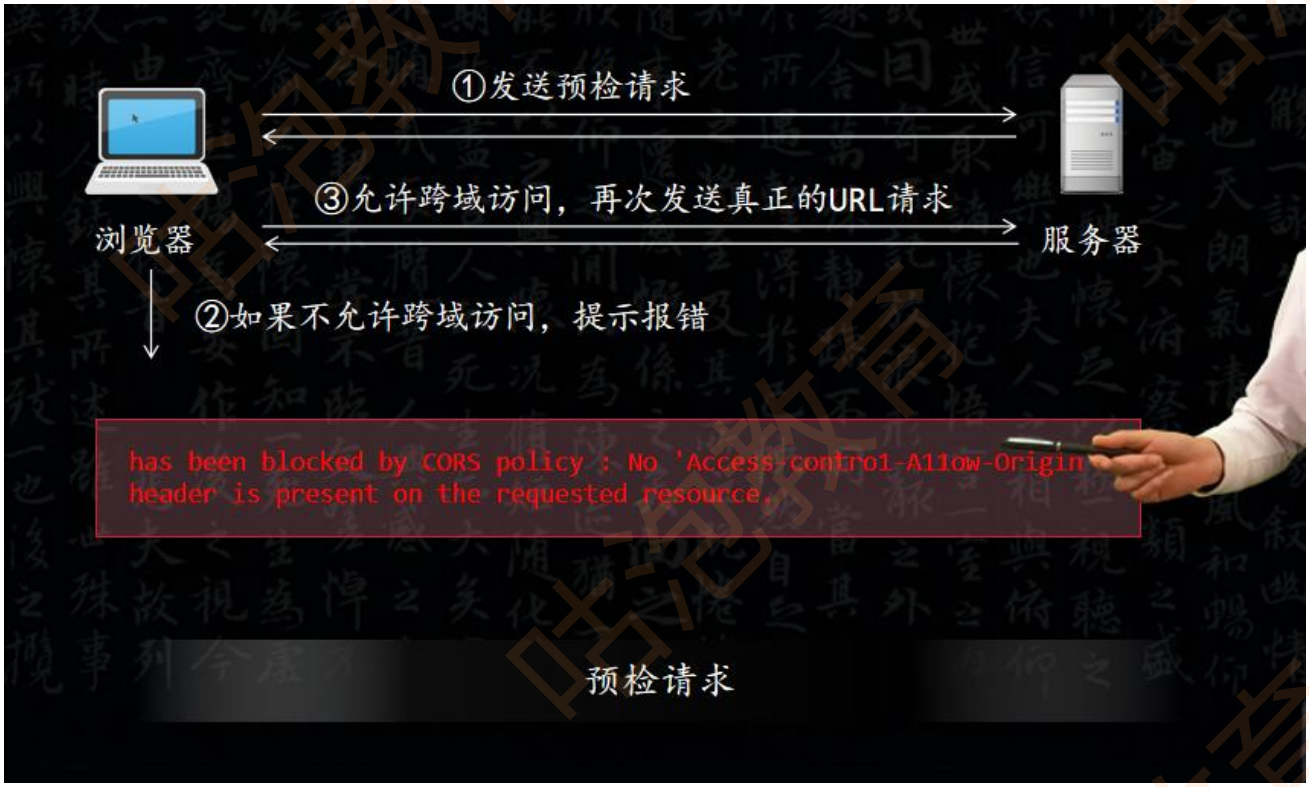


如果请求要满足以下条件，浏览器才不会发送预检请求：

1、请求方法是GET 、PosT .HEAD其中任意一个

2、请求头中包含Accept、Accept-Language、Content-Language、Content-Type、DPR、Downlink、Save-Data、Viewport.Width、Width字段。

3、Content-Type的值是text/plain 、multipart/form-data ,application/x-ww-form-urlencoded 中任意一个。



但是，在实际项目开发中，我们请求的Content-Type一般是是text/html、application/json等格式，或者使用自定义请求头，都会触发预检请求。

浏览器获取到预检请求的响应结果之后，判断服务器如果授权允许访问这个资源，就会再次请求真正的URL，如果不允许就会报这样一个错。

has been blocked by CORS policy : No 'Access-contro1-A11ow-Origin' header is present on the requested resource.

【导航条：如何解决】

（转场，屏幕黑底白字，正中央出现“3 如何解决）

3、如何解决

我们可以利用浏览器的预检机制。

我只需要在后端服务添加CORS策略的配置就可以解决跨域问题。CORS全称是Cross Origin Resource Sharing，翻译过来叫做跨域资源共享。具体解决方案，有以下4种：

1）、如果是普通的Web项目，只需要在服务的根目录下添加一个crossdomain.xml文件即可。文件格式如下：

```
<?xml version="1.0"?>

<!DOCTYPE cross-domain-policy SYSTEM "http://www.adobe.com/xml/dtds/cross-domain-policy.dtd">

<cross-domain-policy>

   <site-control permitted-cross-domain-policies="all"/>

   <allow-access-from domain="*"/>

   <allow-access-from domain="*"/>

   <allow-http-request-headers-from domain="*" headers="*"/>

</cross-domain-policy>
```

当然，使用这种方式不够灵活，在授权过度的情况下，会存在一些安全隐患。

2）、如果是Spring项目的话，可以添加一个处理跨域的过滤器或者拦截器。如代码所示：

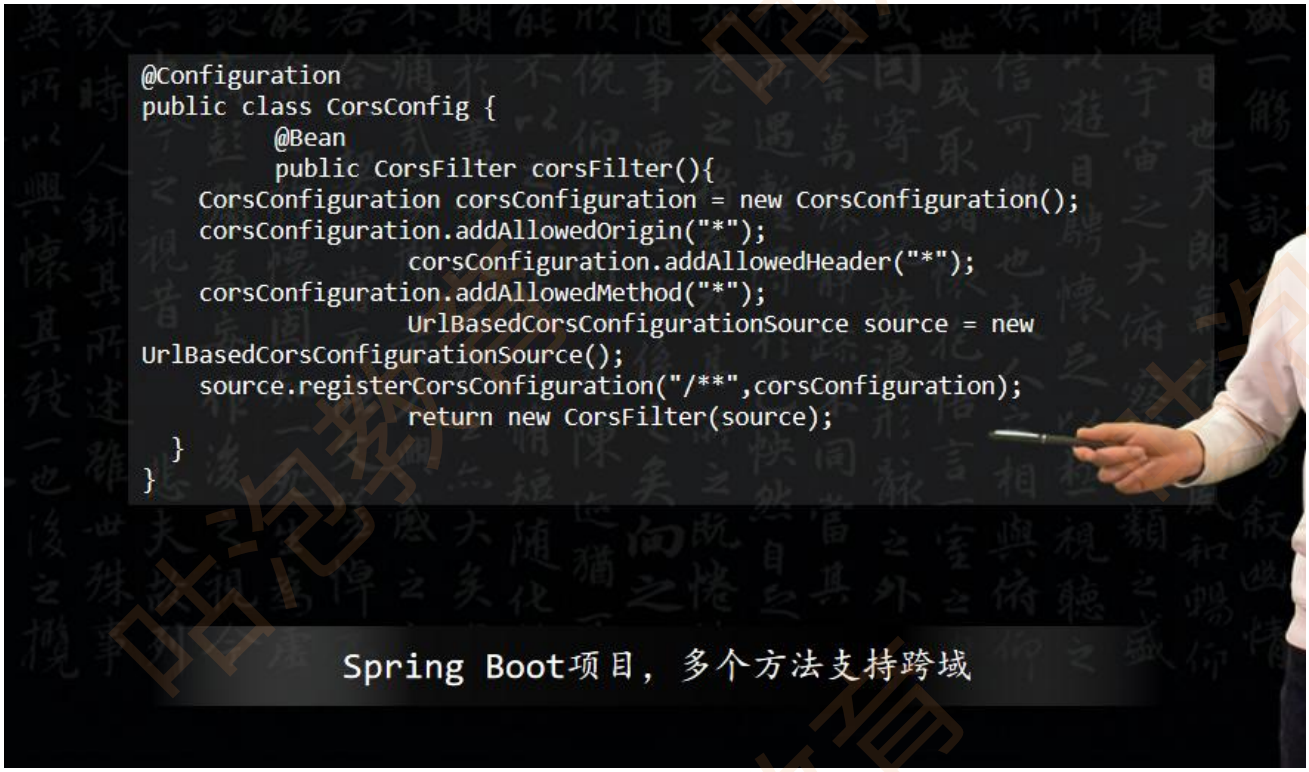
```
public class CorsFilter implements Filter {



   @Override

   public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain) throws IOException,
ServletException {

      HttpServletResponse response = (HttpServletResponse) res;

      response.setHeader("Access-Control-Allow-Origin", "*");

      response.setHeader("Access-Control-Allow-Methods", "POST, GET, OPTIONS, DELETE");

      response.setHeader("Access-Control-Max-Age", "3600");
```

```
        response.setHeader("Access-Control-Allow-Headers", "x-requested-with");

        chain.doFilter(req, res);

    }

}
```

3）、如果是Spring Boot项目的话，只需要在方法上添加@CrossOrigin注解即可，如代码所示：

```
@GetMapping("/list")

@CrossOrigin

public List<String> list(){

...

}
```

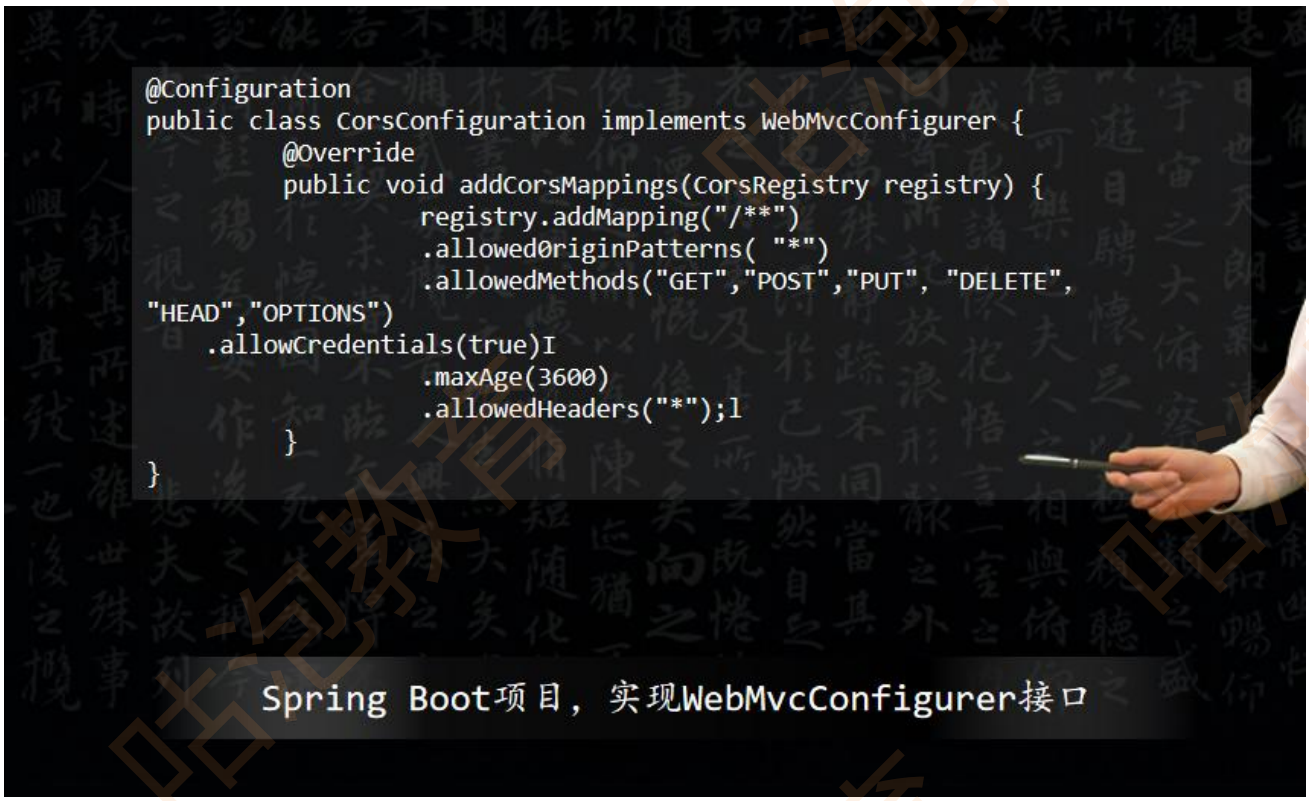
如果需要支持跨域的方法非常多情况下，可以添加过滤器，比Spring更简洁，如代码所示：




```
@Configuration
public class CorsConfig {
@Bean
public CorsFilter corsFilter(){
    CorsConfiguration corsConfiguration = new CorsConfiguration();
    corsConfiguration.addAllowedOrigin("*");
corsConfiguration.addAllowedHeader("*");
    corsConfiguration.addAllowedMethod("*");
UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
```

```
    source.registerCorsConfiguration("/**",corsConfiguration);

return new CorsFilter(source);

  }

}
```

4）、Spring Boot项目还有一种更方便的方式，可以实现WebMvcConfigurer接口来实现跨域支持，如代码所示：




```
@Configuration

public class CorsConfiguration implements WebMvcConfigurer {

@Override

public void addCorsMappings(CorsRegistry registry) {

registry.addMapping("/**")

.allowed0riginPatterns( "*")

.allowedMethods("GET","POST","PUT", "DELETE", "HEAD","OPTIONS")

    .allowCredentials(true)I

.maxAge(3600)

.allowedHeaders("*");l

}
```

```
}
```

只需要重写addCorsMapping() 方法就可以了。

【结尾】

以上就是对Java Web跨域问题的解决方案。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

124、谈谈你对Swagger工作流程的理解

【导航条：引言】

现在的Java开发，一般都会用到API生成工具Open API，今天一位工作2年的小伙伴突然被问到Swagger工作流程，一下子无言以对。于是，来找到我，希望我能科普一下。

今天，我给大家分享一下我的理解。

【导航条：Swagger简介】

（转场，屏幕黑底白字，正中央出现“1 Swagger简介）

1、Swagger简介

记得多年以前，在Swagger还没有出现的时候，我还用自己手写的Maven插件，来实现自动生成API的功能。界面有点丑，给大家秀一下：

2.10、获取订单列表

作者 未描述作者

最后发布日期 未描述最后修改时间

功能描述 获取用户订单列表，包括购买商品订单、预约服务订单、物业缴费订单、快递订单、话费充值等订单都可以调用此接口

是否验证授权码 是

是否验证调用者IP 否

是否验证产品版本 否

HTTP请求方式 GET/POST

请求URL http://i.3dsq.me/mobile/my/getOrders.json

请求参数

参数名称	类型	必填	默认值	描述
process	整数	否	无	无
enc	字符串	是	无	授权码，加密规则：私钥 + 当前登录用户的id

返回结果说明 详情请参看返回结果示例

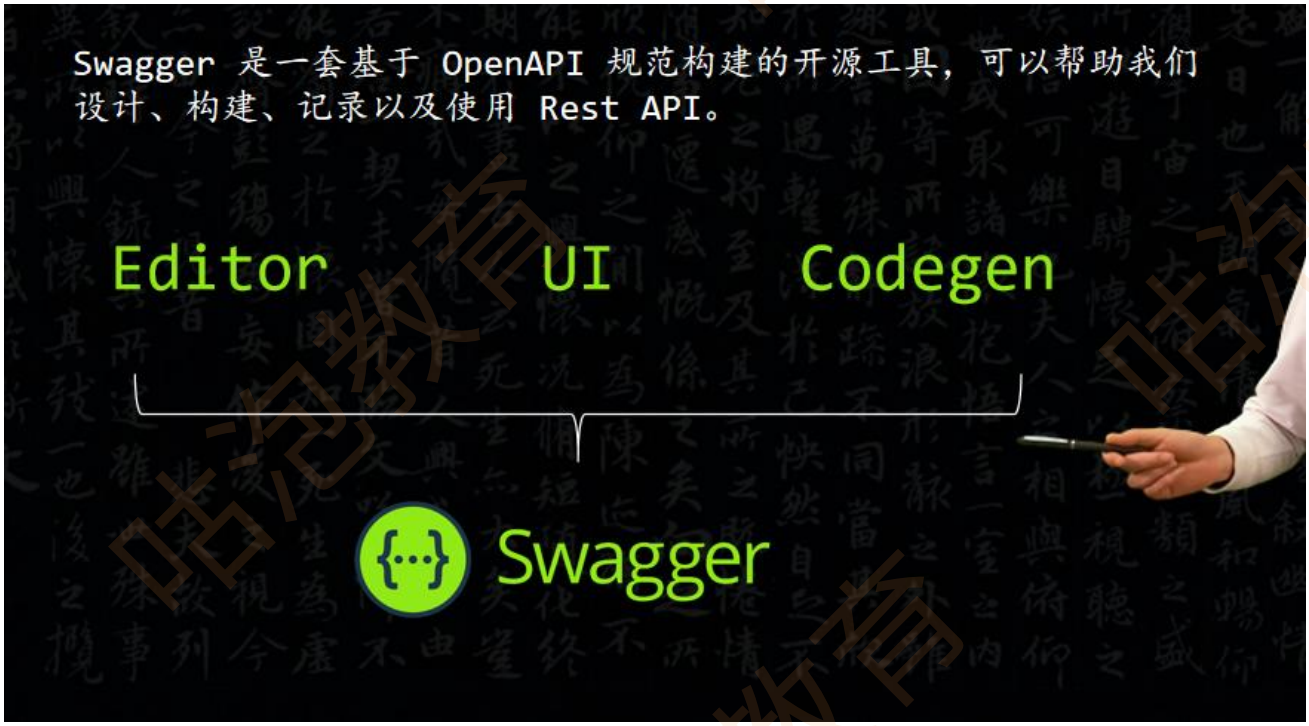
请求示例 http://i.3dsq.me/mobile/my/getOrders.json?enc=ce6078738a0f4c0874879dec38e2aa2b

目录

1、后台管理接口
2、个人中心
3、公共接口
4、资源服务
5、用户认证
6、附近搜索
7、小区搜索
8、购物车
9、分类查询
10、商品相关
11、服务相关
12、店铺相关
13、购物相关
14、社区版块
15、话题点赞
16、回复话题
17、发布和获取话题

当时，还想着开源，后来Swagger问世之后，我就把源码从github上下了。

Swagger 是一套基于 OpenAPI 规范构建的开源工具，可以帮助我们设计、构建、记录以及使用 Rest API。Swagger 主要包含了以下三个部分：



Swagger Editor：基于浏览器的编辑器，我们可以使用它编写我们 OpenAPI 规范。

Swagger UI：它会将我们编写的 OpenAPI 规范呈现为交互式的 API 文档。

Swagger Codegen：它可以通过为 OpenAPI规范定义的任何 API 生成服务器存根和客户端 SDK 来简化构建过程。

【导航条：为什么要使用】

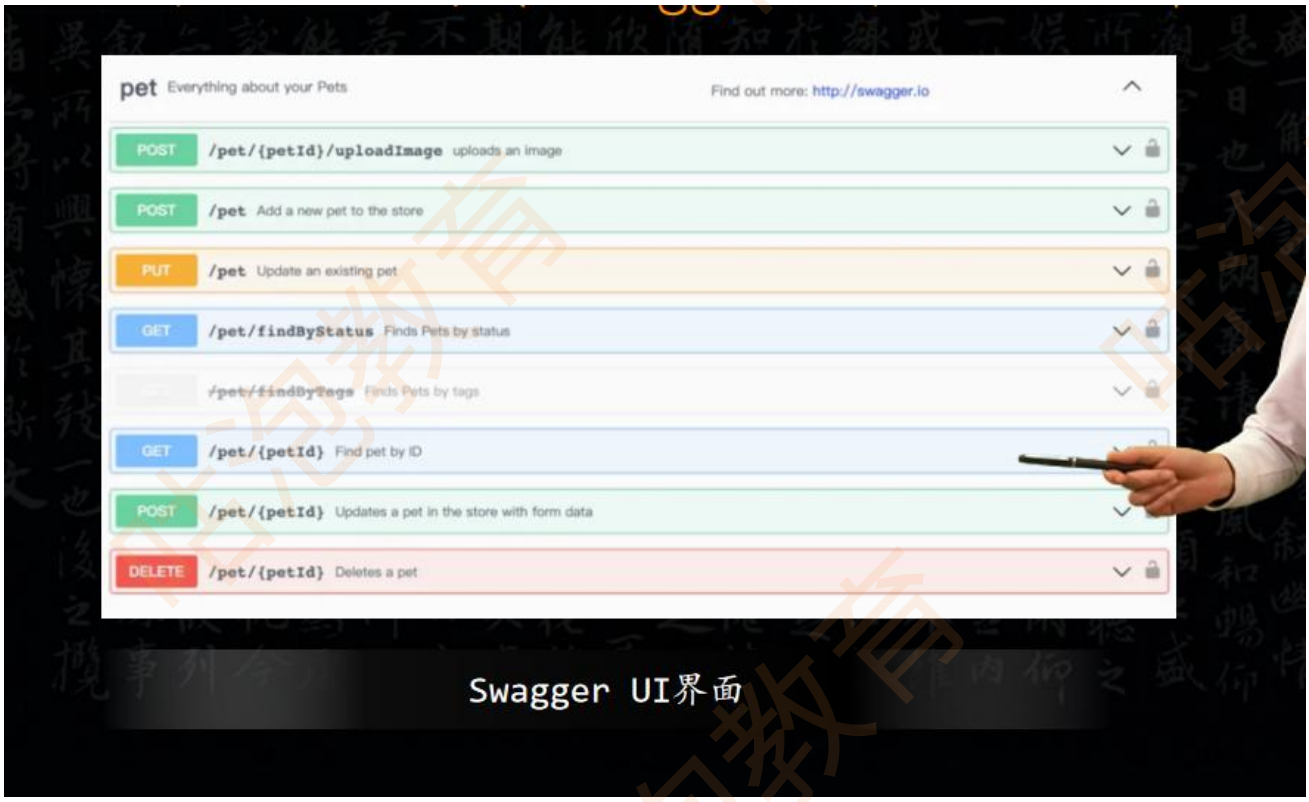
（转场，屏幕黑底白字，正中央出现“2 为什么要使用Swagger）

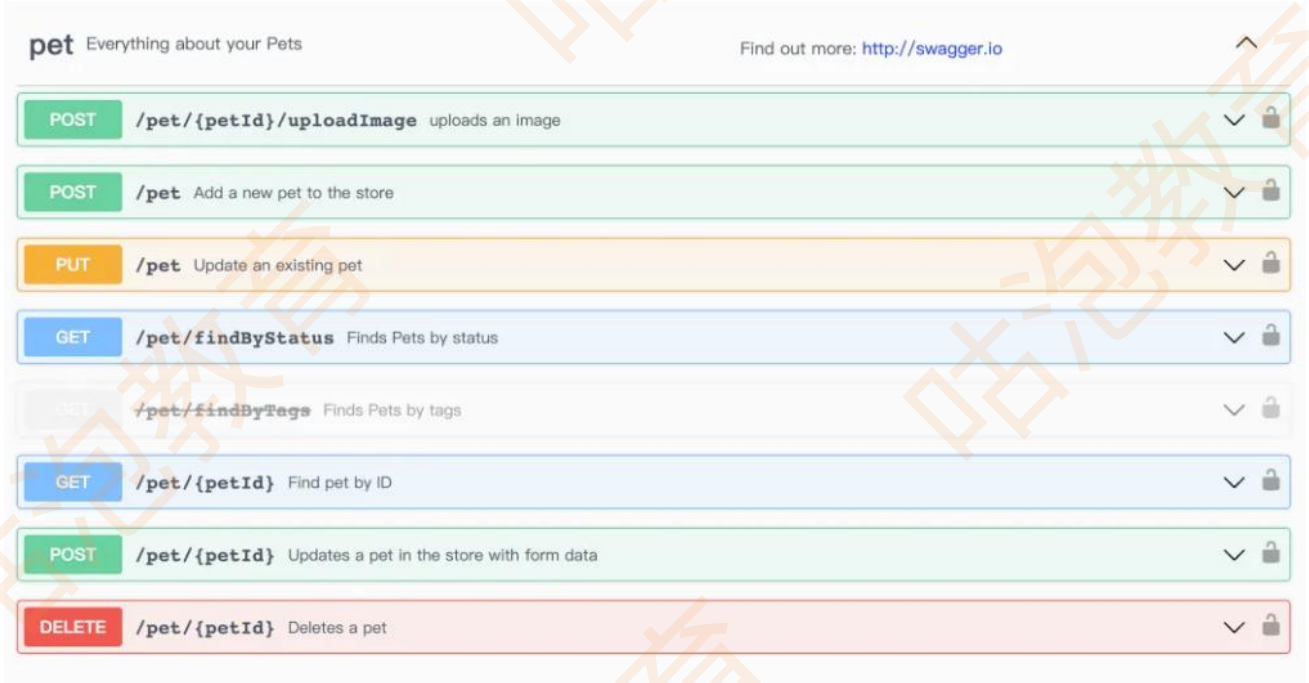
2、为什么要使用 Swagger

在前后端分离开发以后，维持一份及时更新且完整的 Rest API 文档，能够极大的提高的开发效率。传统意

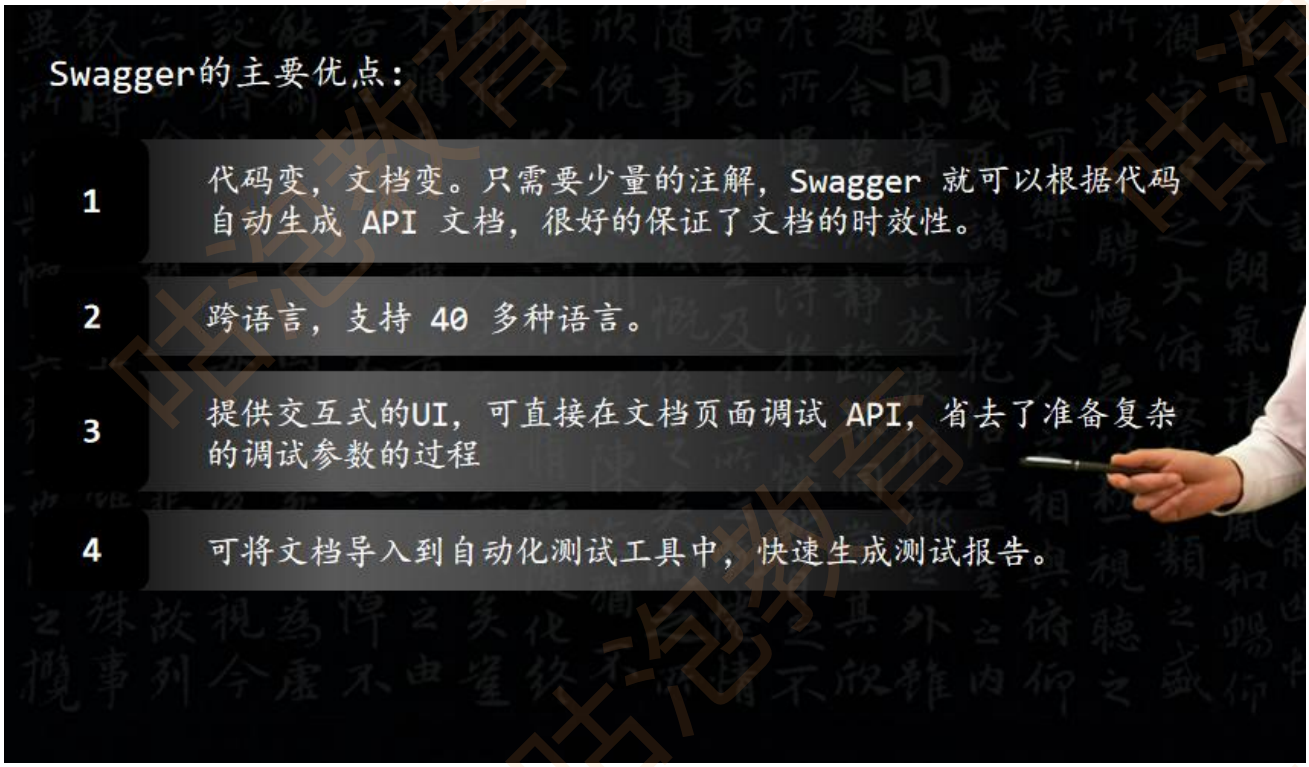
学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

义上的文档都是后端开发人员手动编写的，一般是以Doc或者是md等形式离线传播。而在开发阶段，接口修改非常频繁，就很难保证文档更新的及时性，久而久之就失去了参考意义，反而还会加大团建之间的沟通成本。





而 Swagger 给我们提供了一个全新的维护 API 文档的方式，只要项目发布，就能够自动更新，而且可以同步到线上，使用者只需要记住一个固定的网址，实时刷新就能访问到最新版本的API文档了。下面我总结一下Swagger的主要优点：



1）代码变，文档变。只需要少量的注解，Swagger 就可以根据代码自动生成 API 文档，很好的保证了文档的时效性。

2）跨语言性，支持 40 多种语言。

3）提供交互式的UI，我们可以直接在文档页面调试 API，省去了准备复杂的调试参数的过程。

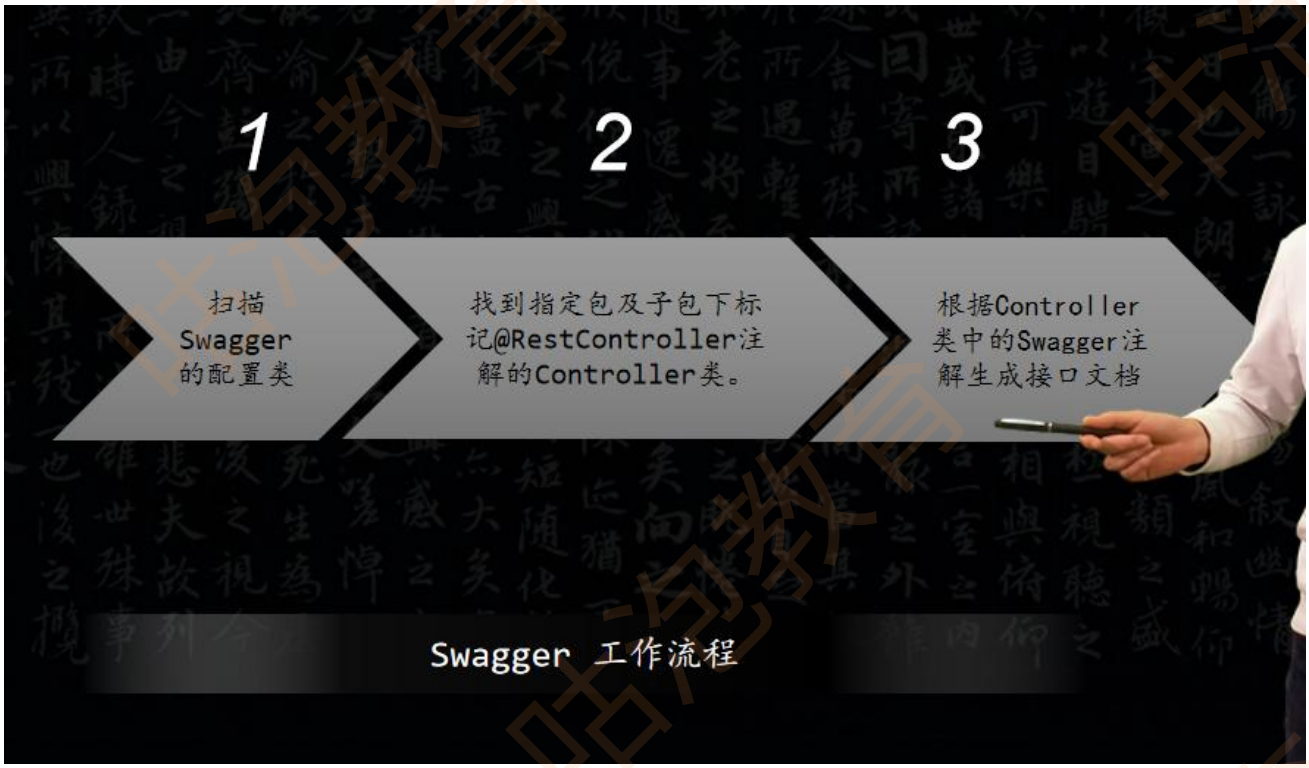
4）还可以将文档导入到自动化测试工具中，快速生成测试报告。

【导航条：工作流程】

（转场，屏幕黑底白字，正中央出现“3 Swagger工作流程）

3、Swagger工作流程

Swagger接口生成工作流程：



1、系统启动时，扫描Swagger的配置类

2、在此类中指定来要扫描的包路径，找到在此包下及子包下标记@RestController注解的Controller类。还可以通过以下这些注解来灵活配置一些参数。

比如：配置发送错误返回的信息 @ApiError ，配置一个或者多个请求参数，@ApiImplicitParam、@ApiImplicitParams等等。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

3、根据Controller类中的Swagger注解生成接口文档，启动项目，访问项目虚拟路径/swagger-ui，查看生成的文档内容。

【结尾】

以上就是对Swagger工作流程的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

125、一个空的Java Object对象到底占多大内存?

【导航条：引言】

一个空的Java Object对象到底占多大内存呢？一个工作5年的小伙伴被这个问题搞晕了。后来，他找到我，希望我能出一期视频。

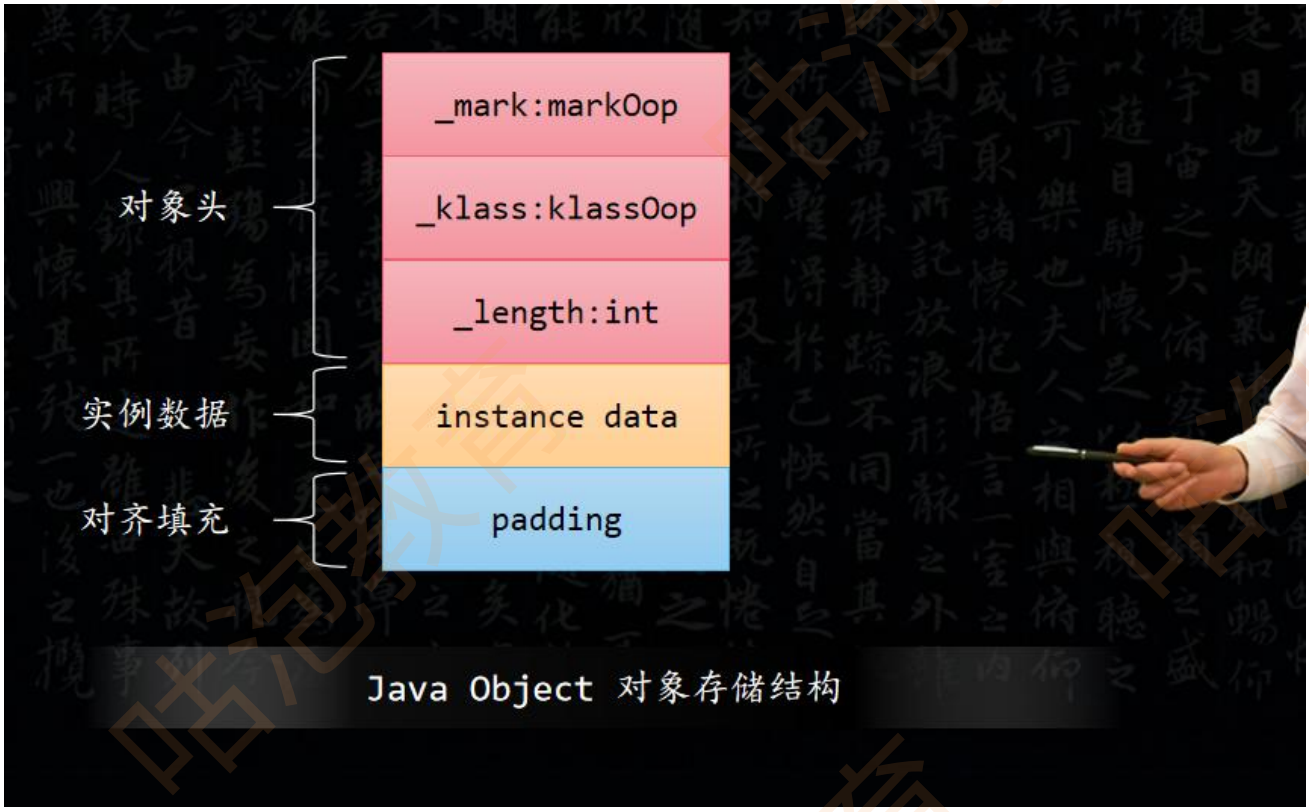
今天，我给大家分享一下我的理解。

【导航条：存储结构分析】

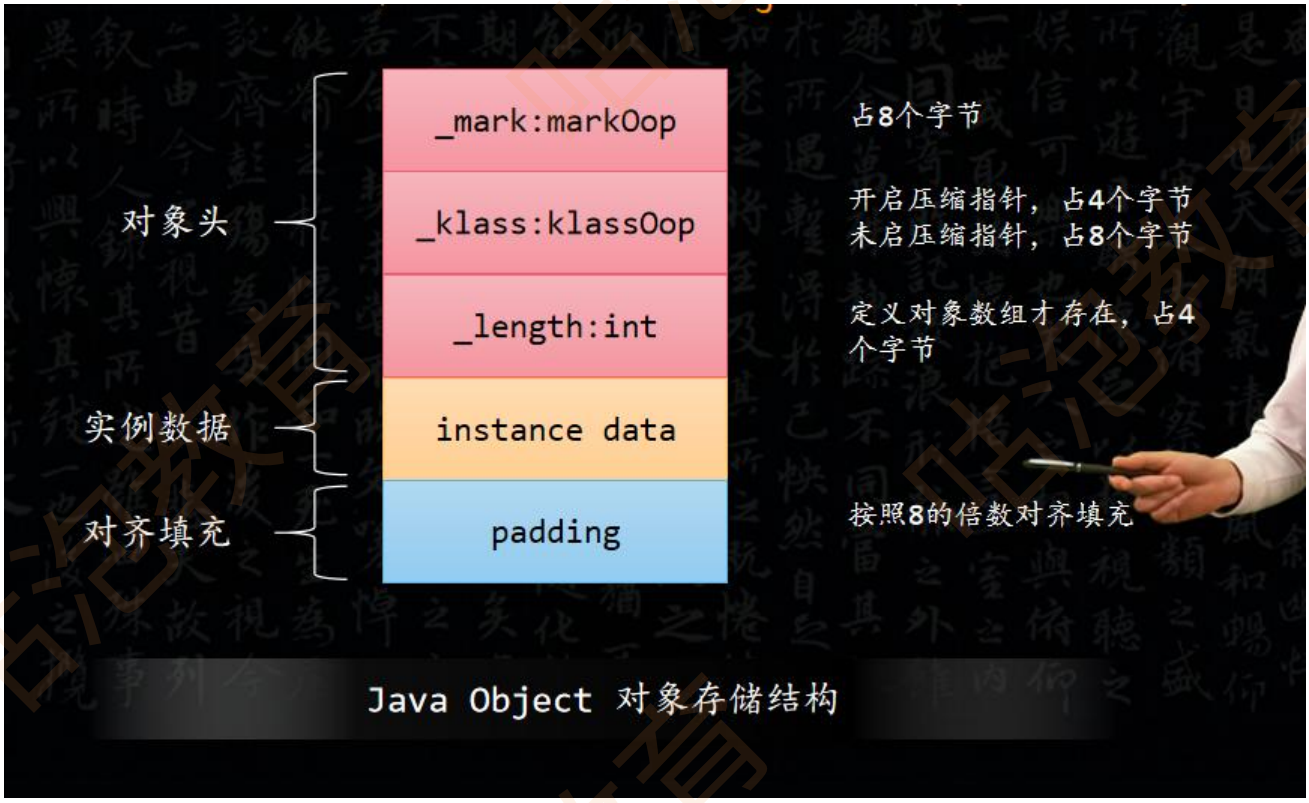
（转场，屏幕黑底白字，正中央出现“1 对象存储结构分析）

1、对象存储结构分析

回答这个问题之前，我们首先得了解Java对象在JVM中的存储结构。以HotSpot虚拟机为例，一个对象在堆内存里面的内存布局是使用OOP结构来表示的，



它主要由三个部分组成，分别是对象头、实例数据和对齐填充。



1）对象头，包括Markword、类元指针、数组长度。其中Markword用来存储对象运行时的相关数据，比如hashCode、GC分代年龄等。在64位操作系统中占8个字节，32位操作系统中占4个字节。类元指针指向当前实例对象所属哪个类，开启指针压缩的情况下占4个字节，未开启则占8个字节。

数组长度只有对象数组才会存在，占4个字节

2）实例数据，主要用来存储对象中的字段信息。

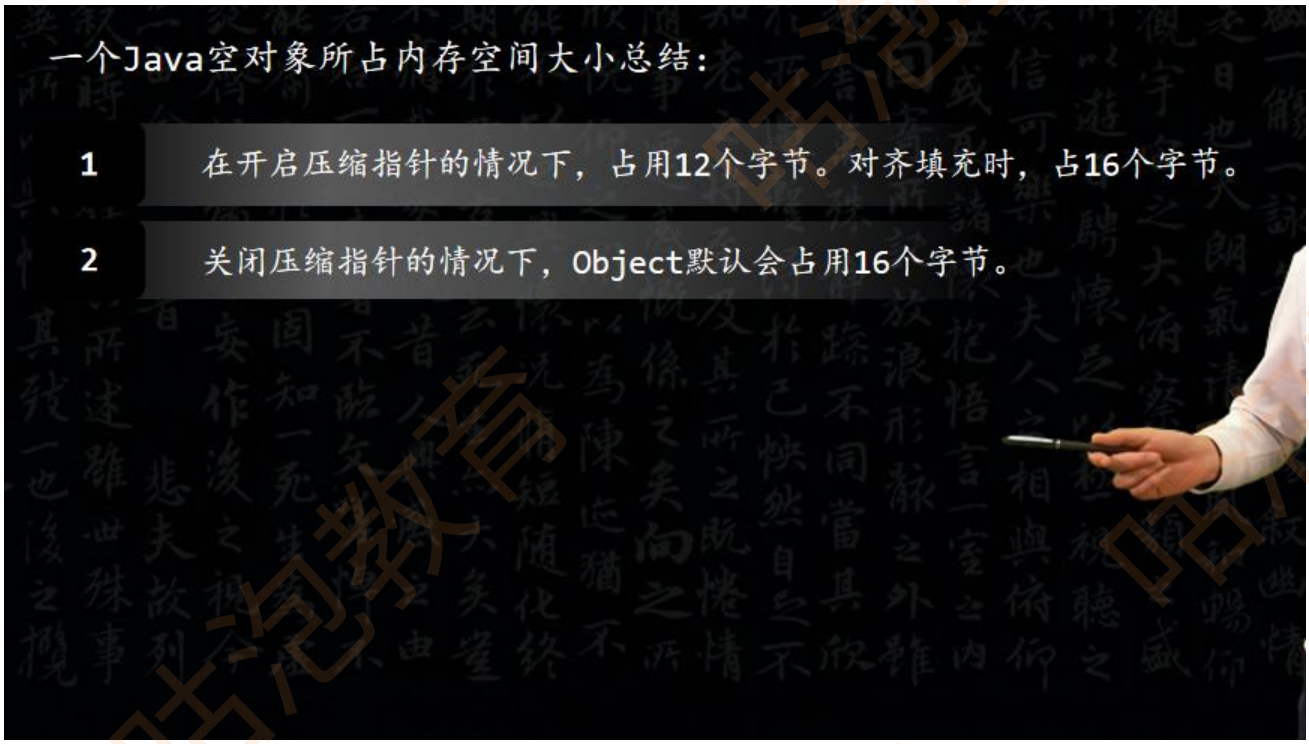
3）对齐填充，用来补充实现Java对象大小的倍数对齐。在JVM中，Java对象的大小需要按照8个字节或者8个字节的倍数来对齐，从而避免伪共享问题。

【导航条：结论总结】

（转场，屏幕黑底白字，正中央出现“2 结论总结）

2、结论总结

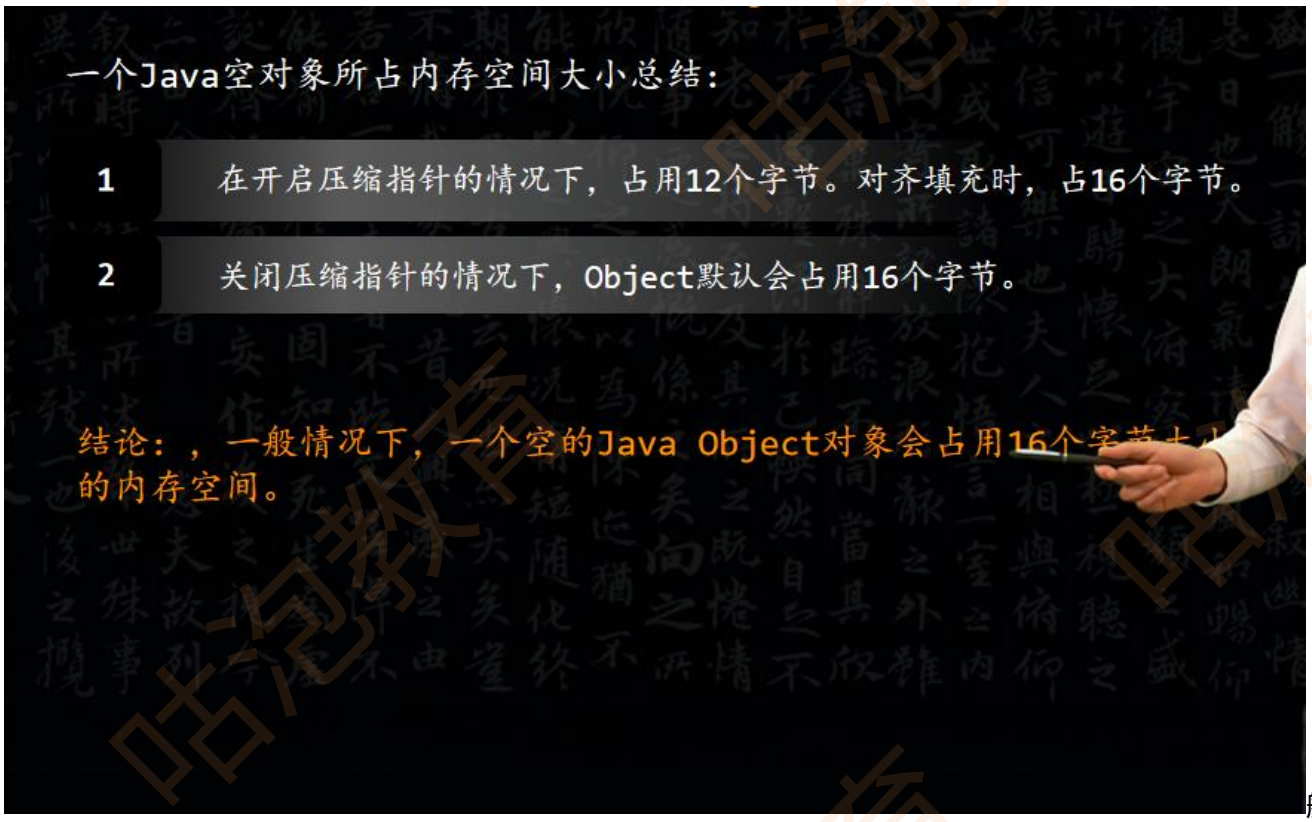
根据以上的分析，我们总结一下。



1）一个Java空对象，在开启压缩指针的情况下，占用12个字节。其中，Markword占8个字节、类元指针占4个字节。但是为了避免伪共享问题，JVM会按照8个字节的倍数进行填充，所以会在对齐填充区填充4个字节，变成16个字节长度。

2）那么在关闭压缩指针的情况下，Object默认会占用16个字节。其中，Markword占8个字节、类元指针占4个字节， 对齐填充占4个字节。16个字节正好是8的整数倍，因此不需要填充。

所以结论是，一



般情况下，一个空的Java Object对象会占用16个字节大小的内存空间。这个问题不仅仅考察JVM基础，还考察求职者对于JVM对于对象内存布局的理解程度。对于内存布局的理解可以帮助我们更好的解决JVM应用上的实际问题。

【结尾】

以上就是对Java空对象的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

126、每天都在用的SimpleDateFormat是线程安全的吗?

【导航条：引言】

一位工作两年的小伙伴被问到这样一道面试，说SimpleDateFormat是线程安全的吗？我们每天都在使用SimpleDateFormat这个工具类，屏幕前的你能回答出来吗?

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

面试官问这个问题的主要目的是希望应聘者能说出解决方案，并不只是回答是或者不是。

今天，我给大家分享一下我对这个问题的理解。

【导航条：原因分析】

（转场，屏幕黑底白字，正中央出现“1 原因分析）

1、原因分析

先说答案，SimpleDateFormat不是线程安全的。

因为，SimpleDateFormat类的内部有一个Calendar对象引用，这个对象主要用来储存和这个SimpleDateFormat相关的日期信息。

当我们把SimpleDateFormat作为多个线程的共享资源来使用的时候，那就意味着多个线程之间会共享这个SimpleDateFormat里面的Calendar引用。如果多个线程同时于操作这个Calendar对象的情况下，就会出现数据脏

读的现象，从而导致一些不可预料的错误。

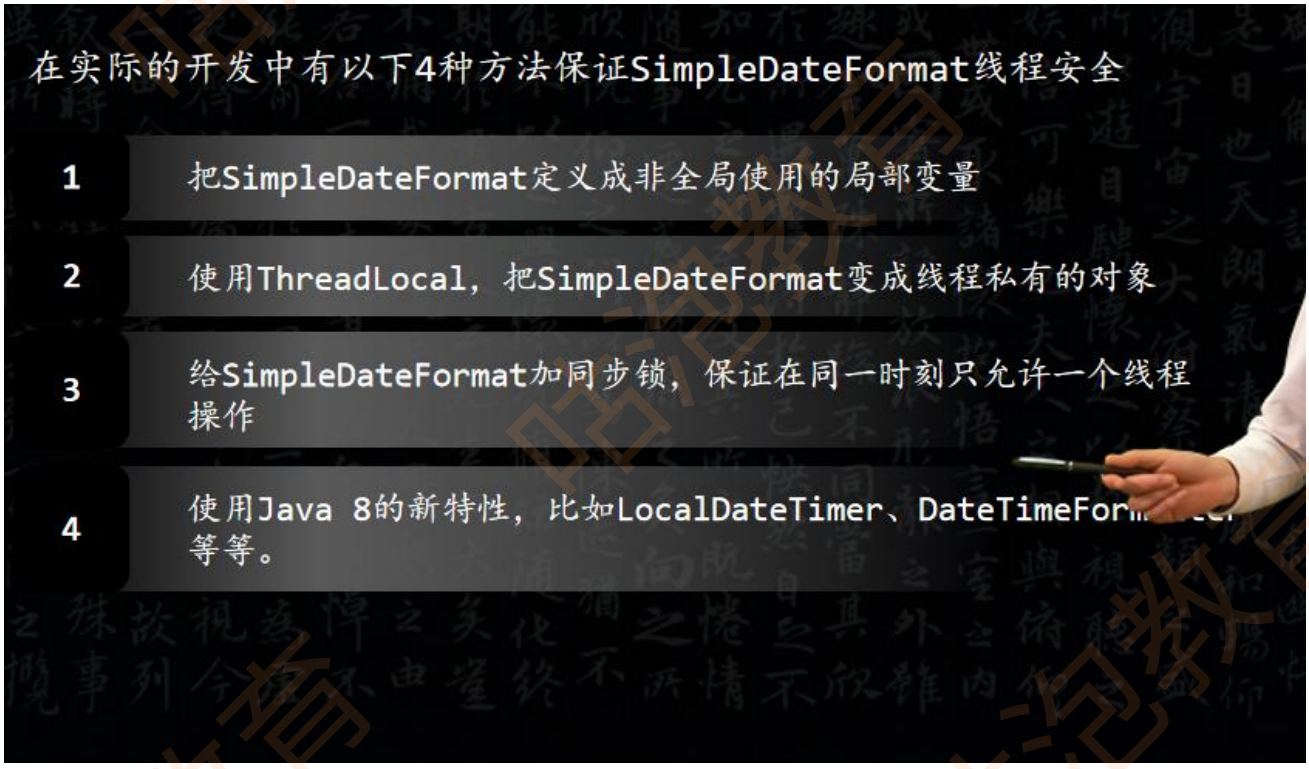
那么，保证SimpleDateFormat线程安全呢?

【导航条：解决方案】

（转场，屏幕黑底白字，正中央出现“2 解决方案）

2、解决方案

在实际的开发中，我认为有以下4种方法可以保证SimpleDateFormat线程安全。



1）、可以把SimpleDateFormat定义成非全局使用的局部变量，这样每个线程调用的时候都创建一个新的实例。

2）、可以使用ThreadLocal，把SimpleDateFormat变成一个线程私有的对象。

3）、定义SimpleDateFormat的时候，加上同步锁，这样就能够保证在同一时刻只允许一个线程操作

4）、使用Java 8的新特性，在Java8中引入了一些线程安全的日期操作API，比如LocalDateTimer、DateTimeFormatter 等等。

【结尾】

这道题本质上考察的是如何保证一个Java对象的线程安全问题，因为一旦遇到线程安全问题非常严重且也很隐蔽。虽然现在很多框架都在刻意屏幕复杂性，但是多线程的问题总是绕不开的。因此多线程是大多数大厂面试的基本内容。

以上就是对SimpleDateFormat线程安全这个问题的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

127、简述雪花算法的实现原理

【导航条：引言】

一位工作4年的小伙伴，去某东面试时被问到这样一道题，说请你简述一下雪花算法的实现原理。屏幕前的小伙伴，如果你遇到这个问题，你会怎么回答？

今天，我给大家分享一下我的理解。

【导航条：什么是雪花算法】

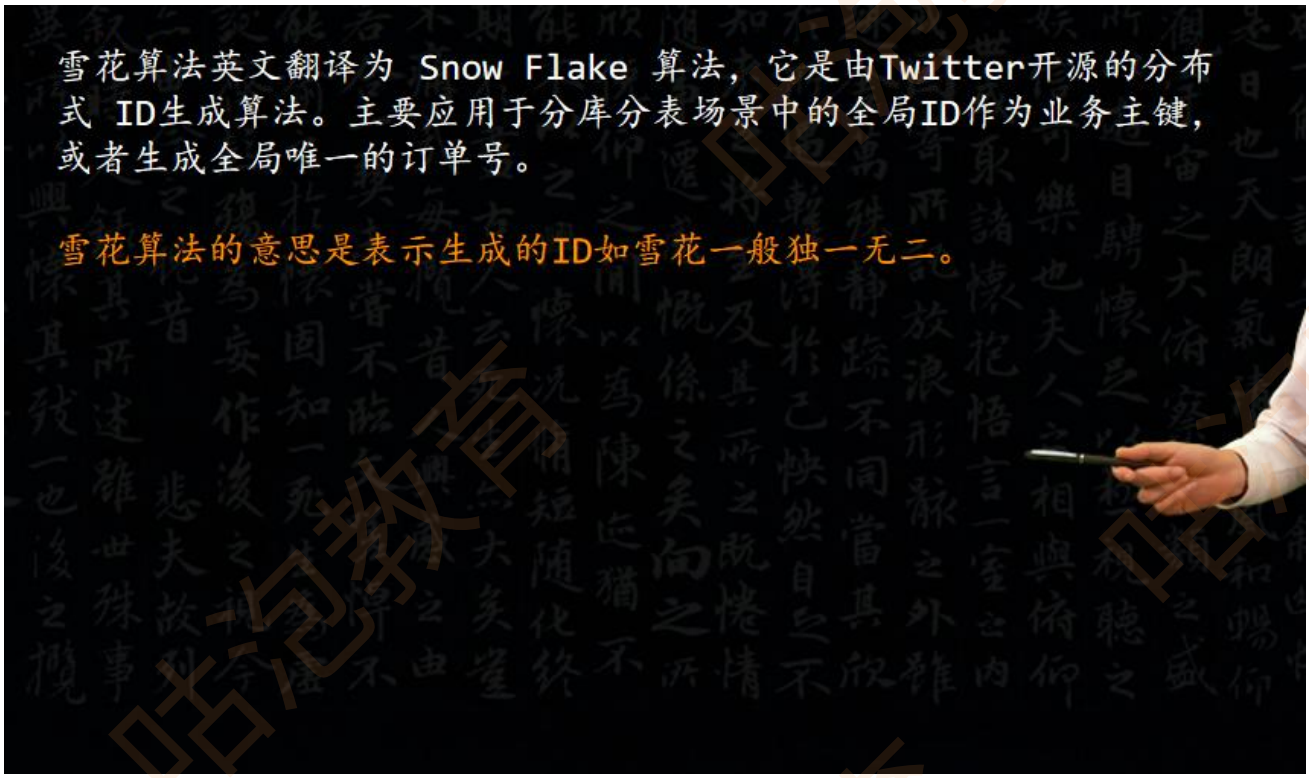
（转场，屏幕黑底白字，正中央出现“1 什么是雪花算法）

1、什么是雪花算法

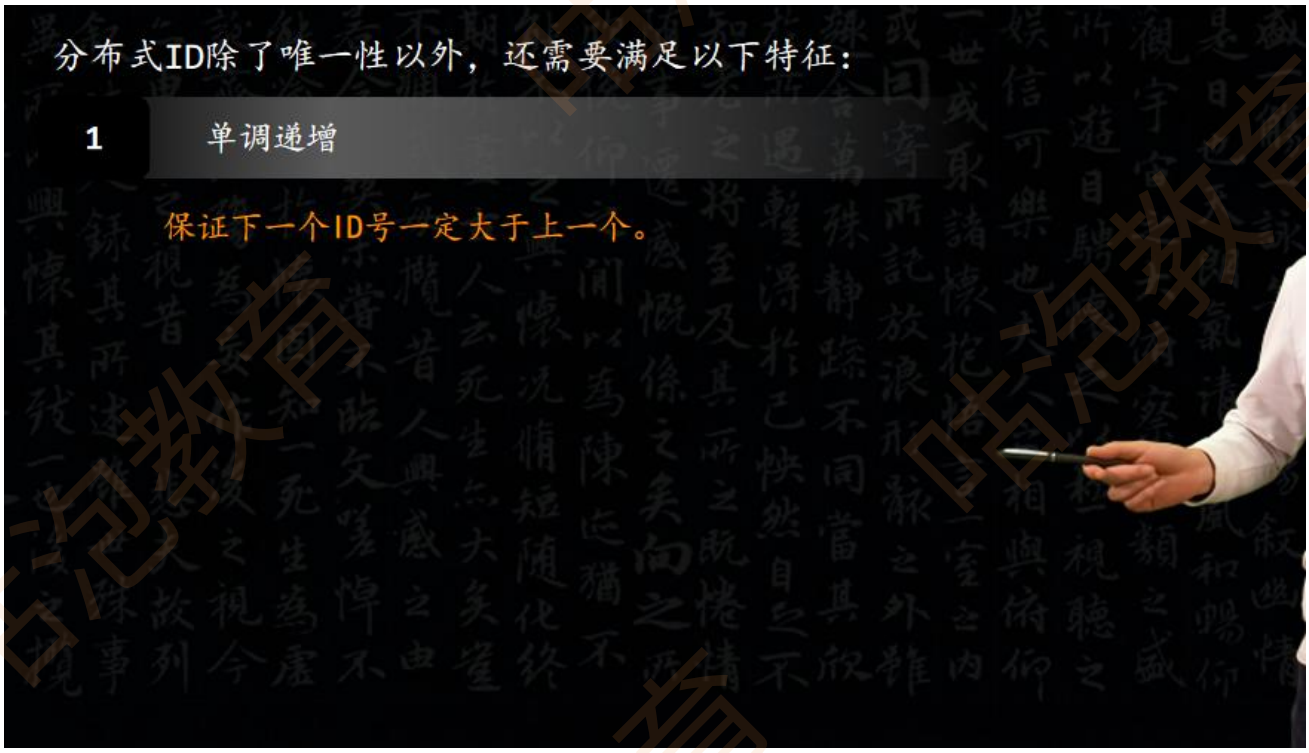
雪花算法英文翻译为 Snow Flake 算法，它是由Twitter开源的分布式 ID生成算法。主要应用于分库分表场

景中的全局ID作为业务主键，或者生成全局唯一的订单号。

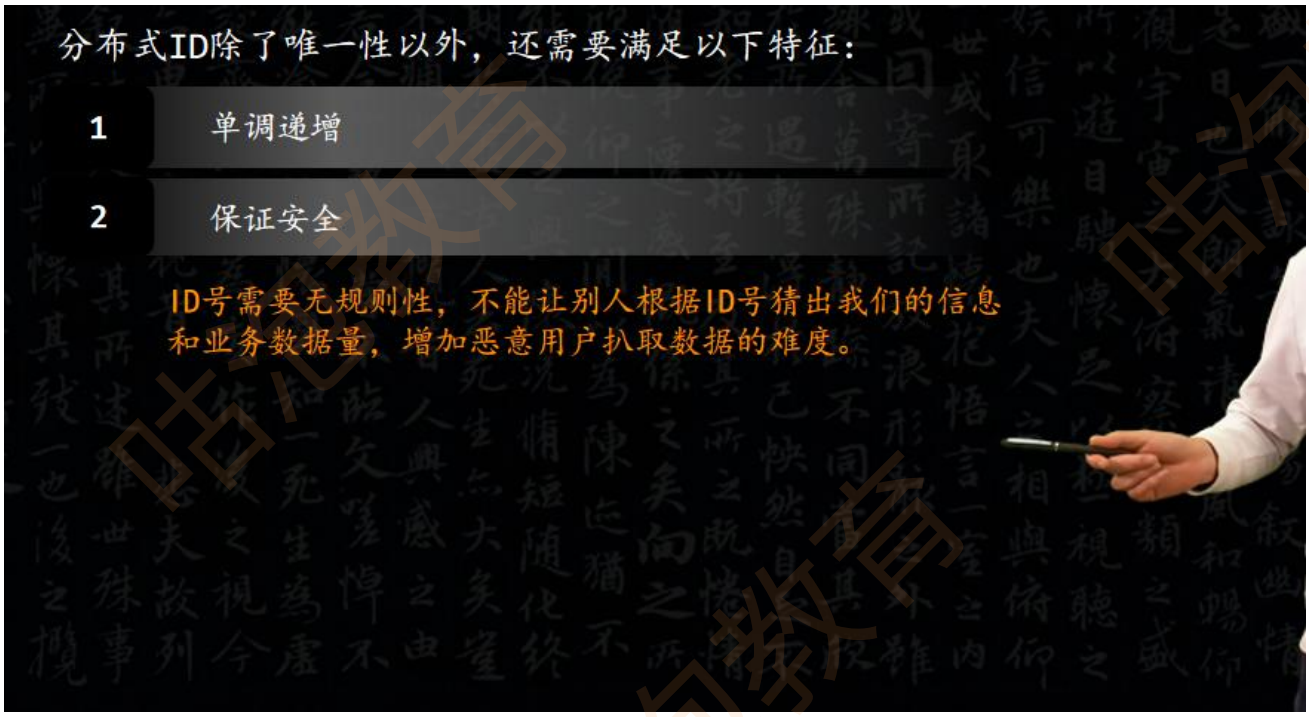
那为什么要叫雪花算法呢？据相关研究表示，一般的雪花大约由10的19次方个水分子组成。在雪花形成过程中，会形成不同的结构分支，所以说大自然中不存在两片完全一样的雪花，每一片雪花都拥有自己独特的形状。雪花算法的意思是表示生成的ID如雪花一般独一无二。



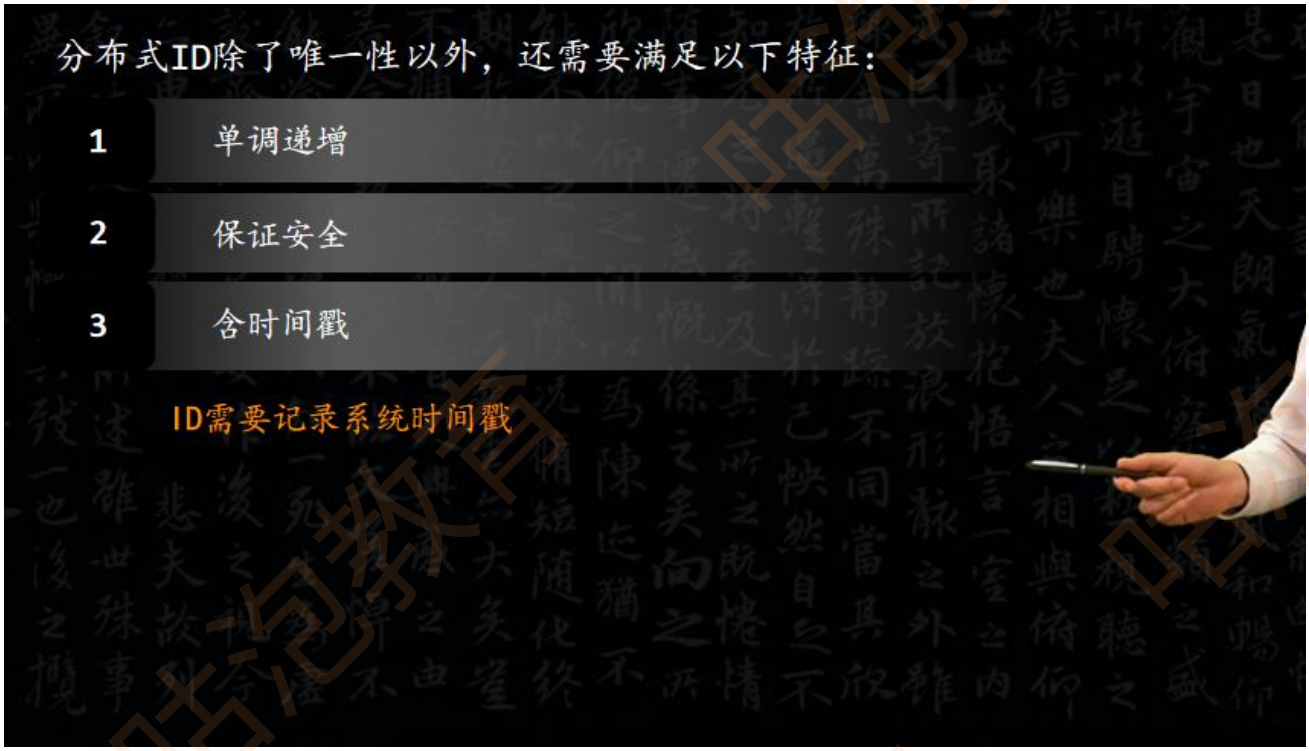
其实，单单解决唯一ID这个问题有很多的解决方法，比如UUID、系统时间戳、Redis原子递增、数据库全局表自增ID等等。但是在实际应用中，我们需要的ID除了唯一性以外，还需要满足以下特征：



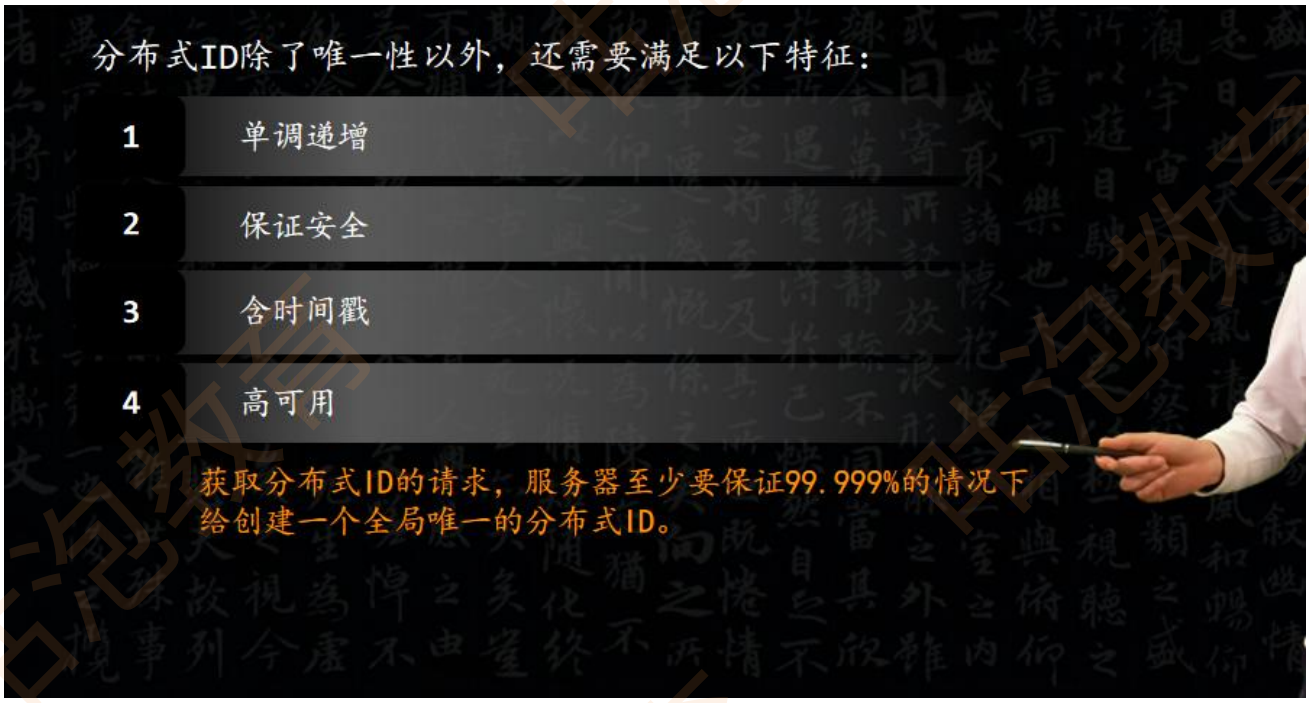
1）、单调递增：保证下一个ID号一定大于上一个。



2）、保证安全：ID号需要无规则性，不能让别人根据ID号猜出我们的信息和业务数据量，增加恶意用户扒取数据的难度。



3）、含时间戳：需要记录系统时间戳



4）、高可用：发布一个获取分布式ID的请求，服务器至少要保证99.999%的情况下给创建一个全局唯一的分布式ID。

5）、低延迟：发布一个获取分布式ID的请求，要快，急速。

6）、高QPS：假如并发一口气10万个创建分布式ID请求同时杀过来，服务器要顶得住并且成功创建10万个分布式ID。

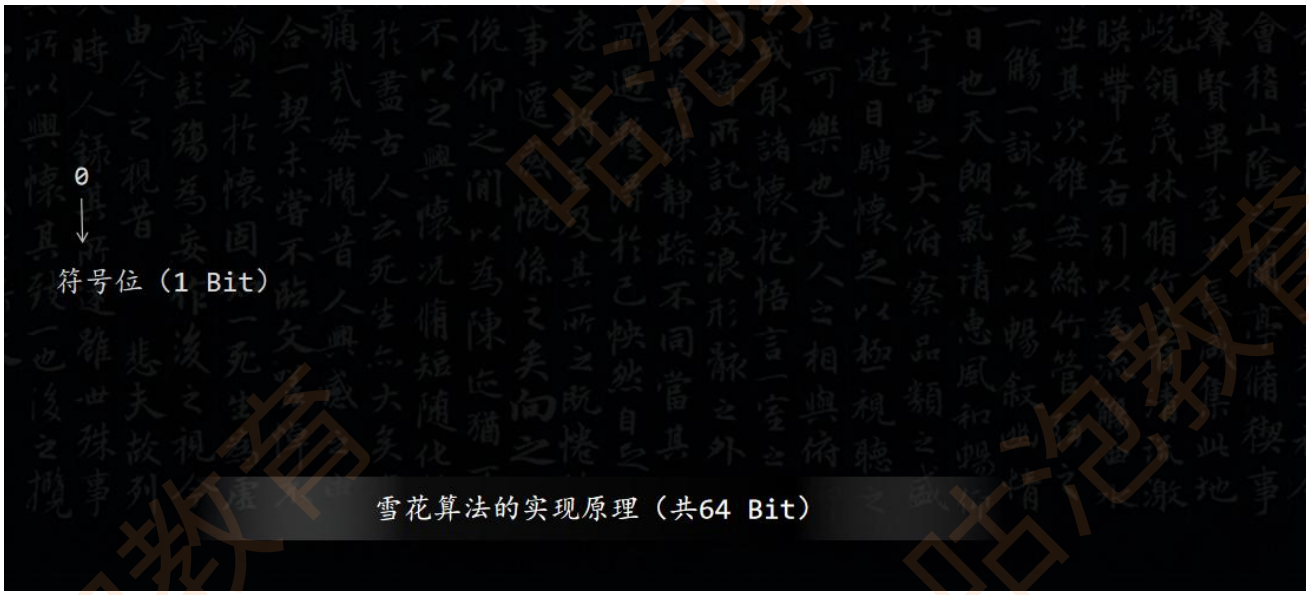
雪花算法就是一个比较符合这类特征的全局唯一算法。在很多大厂的全局ID组件中，都有用到，比如百度的UidGenerator、美团的Leaf算法等等。

【导航条：实现原理】

（转场，屏幕黑底白字，正中央出现“2 实现原理）

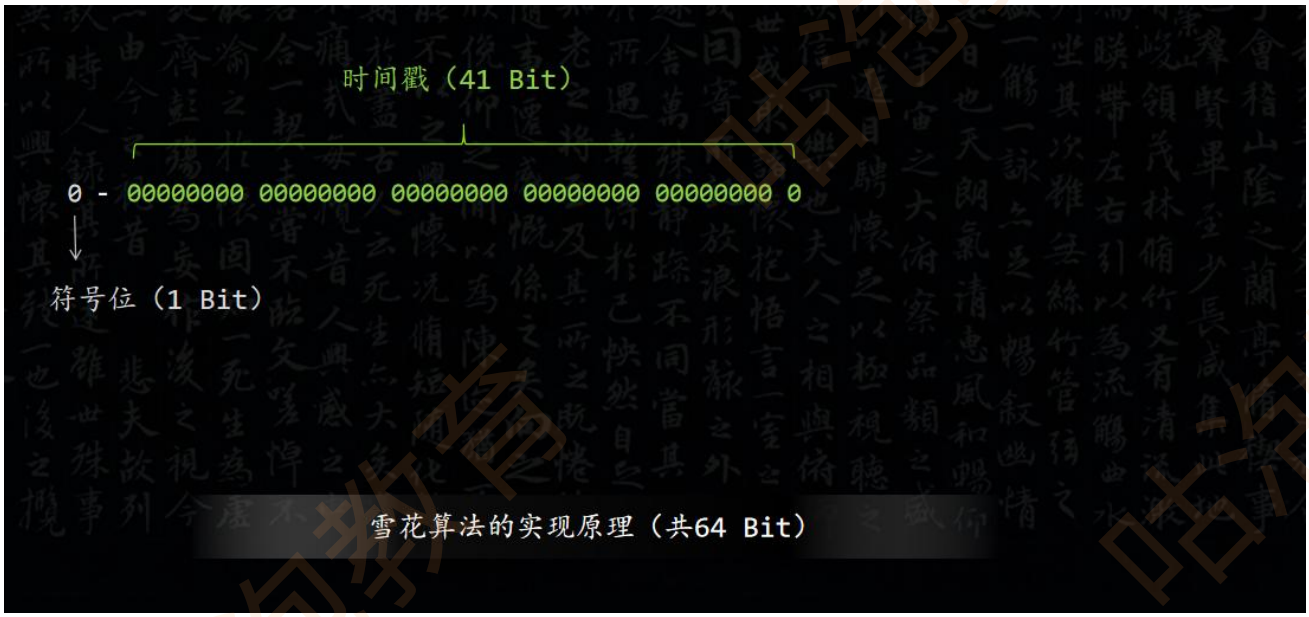
2、实现原理

雪花算法那是一个由64个Bit(比特)位组成的long类型的数字。如图所示，它分为四个部分。

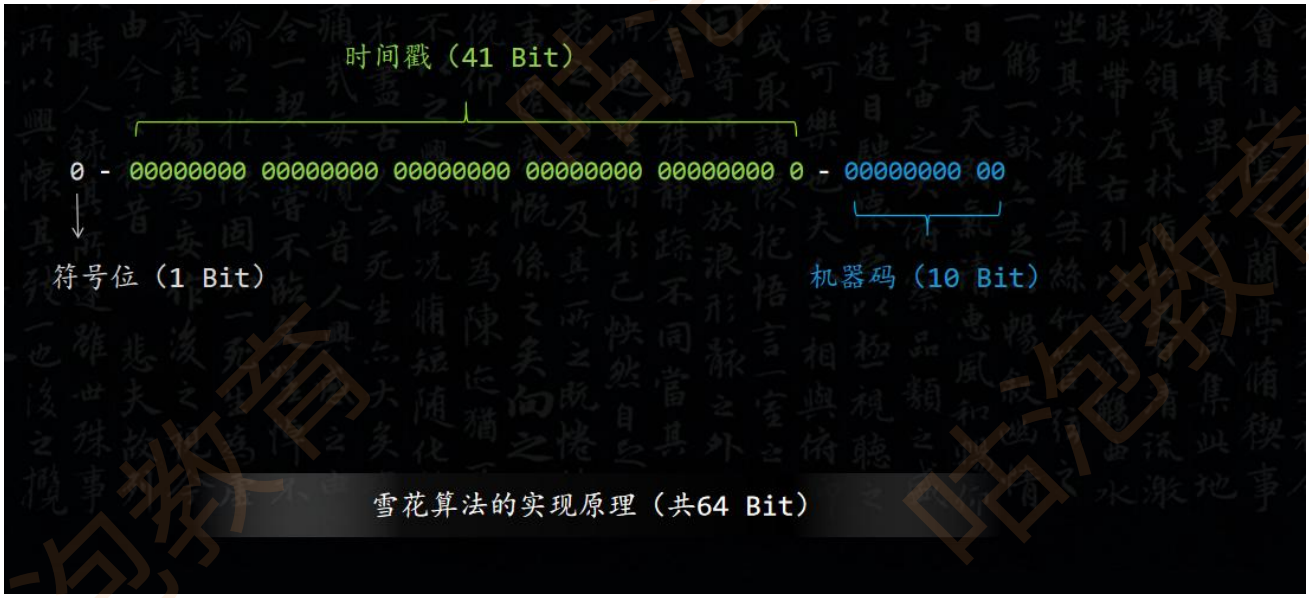


第一部分，用1个bit（比特）位来表示1个符号位，因为ID一般不会是负数，所以一般情况下就是0。

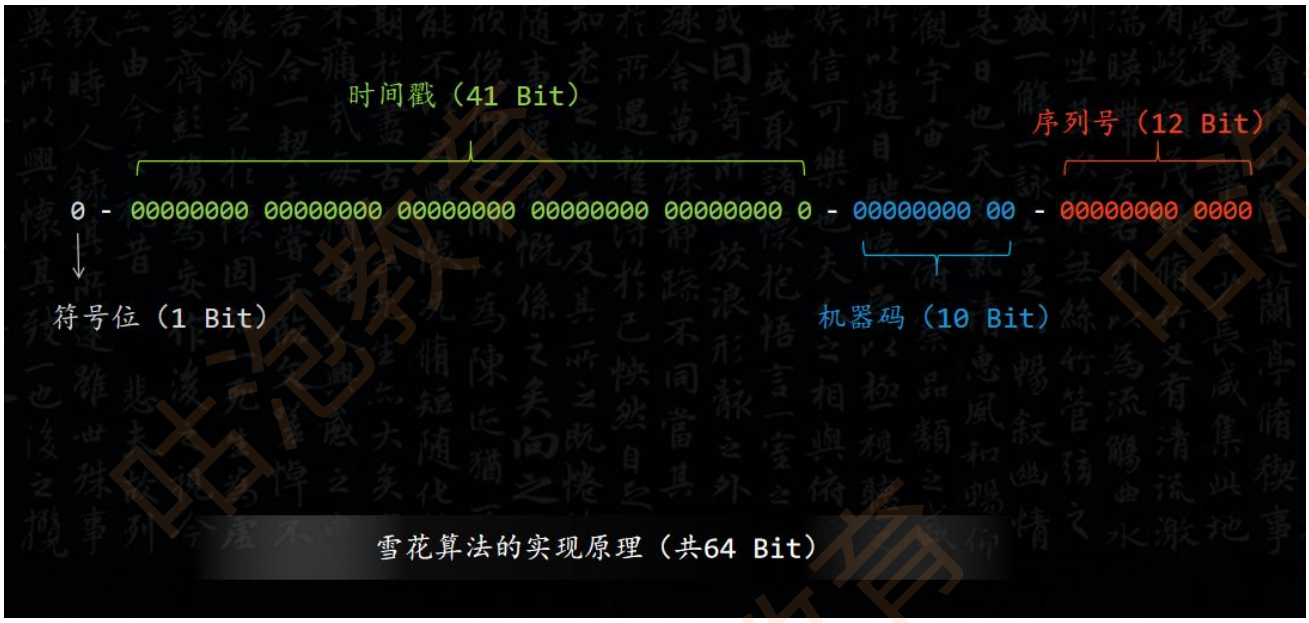
学Java架构·来咕泡教育
50000+ 技术人
都在咕泡教育拿到职场结果



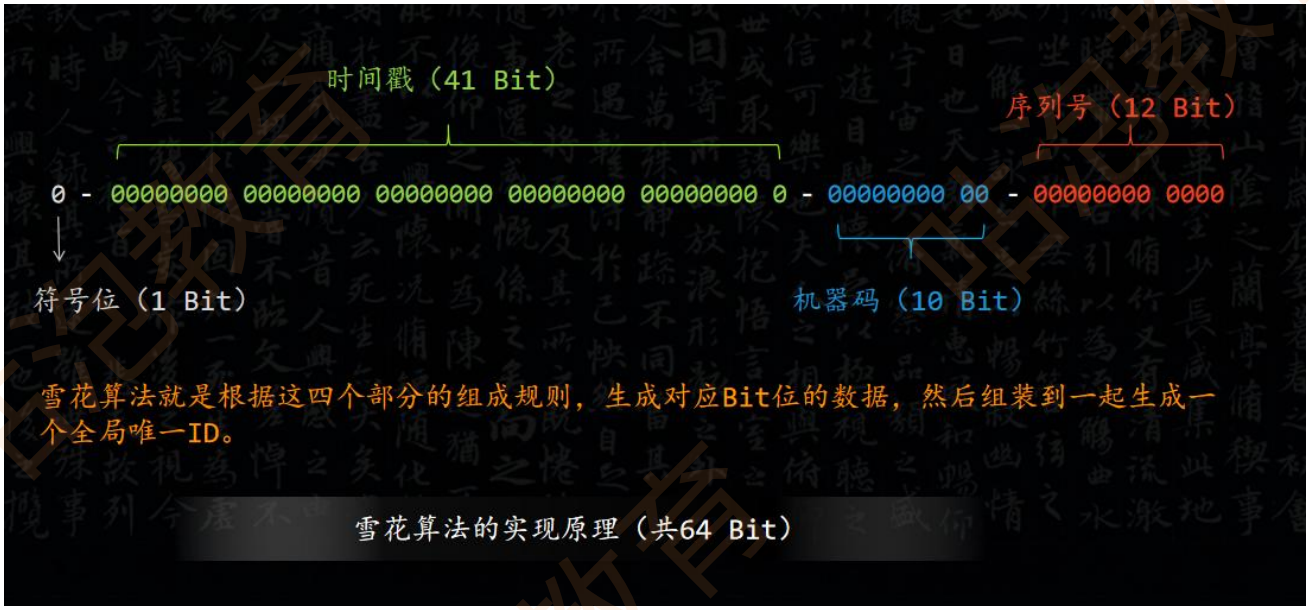
第二部分，用41个bit(比特)位来表示系统时间戳，这个时间戳就是系统时间记录的毫秒数。41个bit可以表示的最大数字为2的41次方减1毫秒，换算成时间为69年。



第三部分，用10个bit（比特）位来技术工作机器的ID，用来保证多个服务器上生成ID的唯一性。如果存在跨机房部署的情况，还可以把这10个比特位拆分为两组，每组5个bit（比特）位。前面的5个bit（比特）表示机房ID，后面5个bit（比特）表示机器ID。10个比特位最大值是2的10次方，也就是最多1024台机器。



第四部分，用12个比特位来表示递增序列，用来记录同一毫秒内产生不同ID的能力。它的最大值为2的12次方减1，也就是4096。



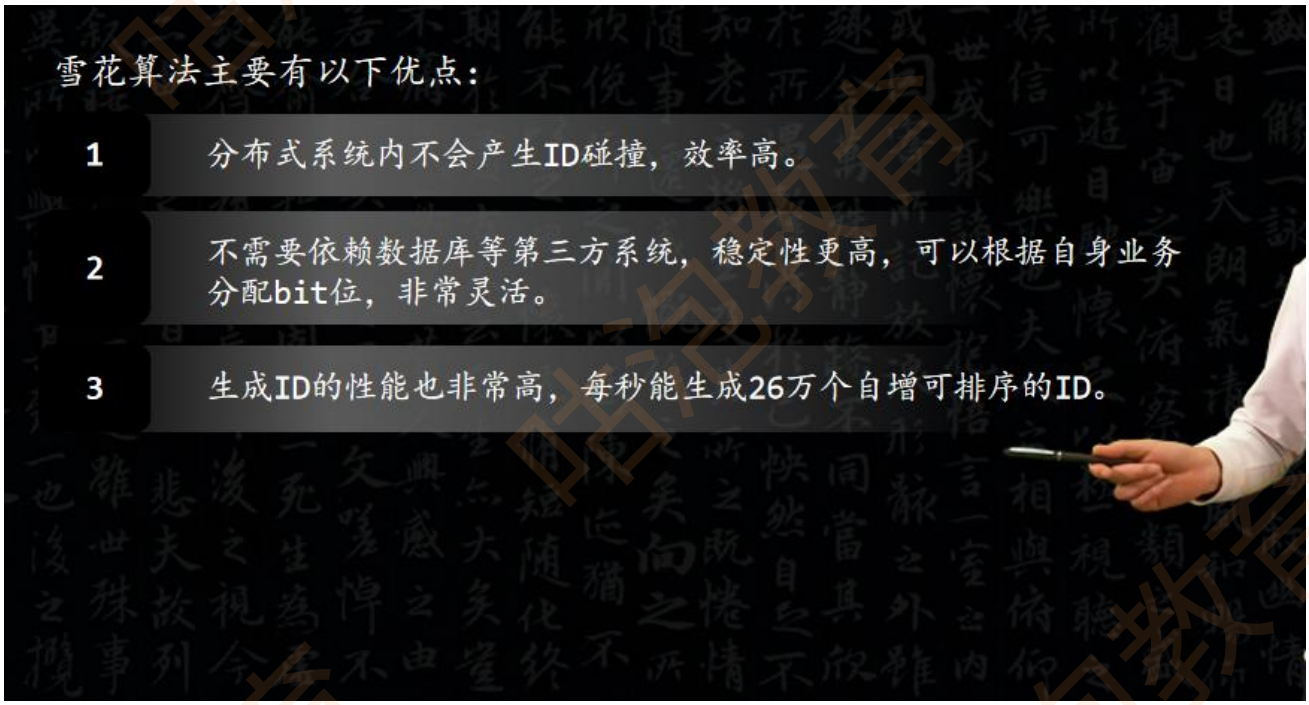
雪花算法就是根据这四个部分的组成规则，生成对应Bit位的数据，然后组装到一起生成一个全局唯一ID。

【导航条：雪花算法的优缺点】

（转场，屏幕黑底白字，正中央出现“3 雪花算法的优缺点）

3、雪花算法的优缺点

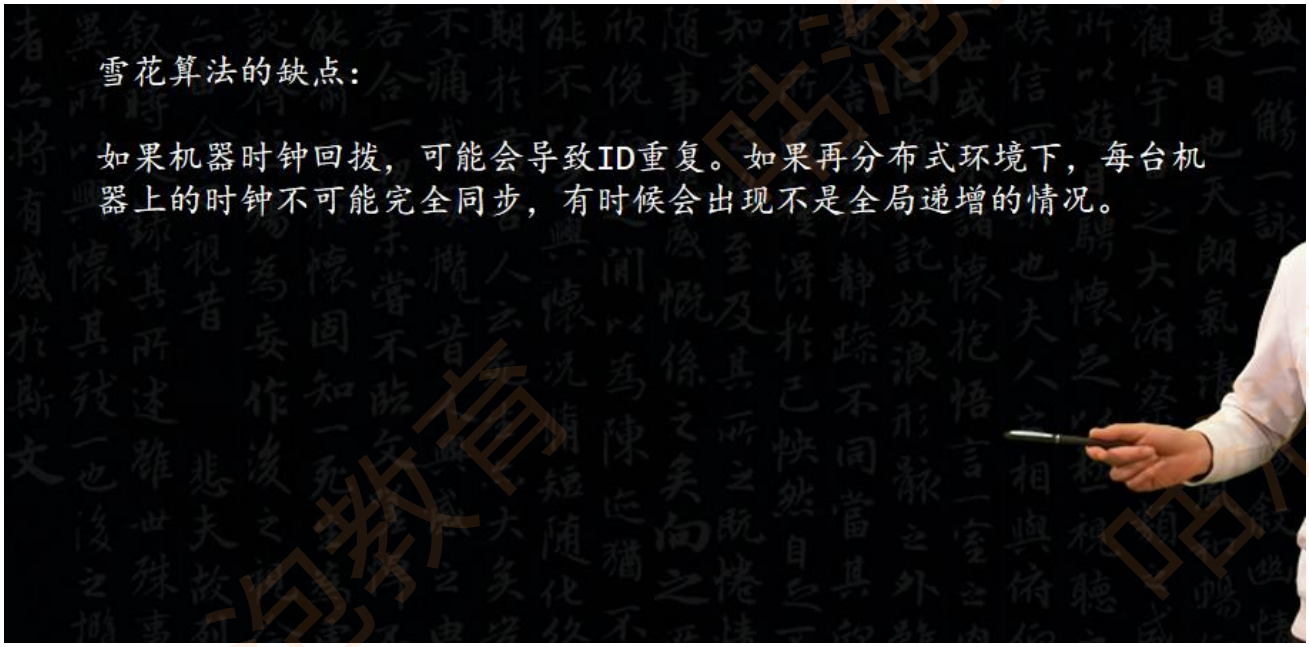
雪花算法主要有以下优点：



1）、分布式系统内不会产生ID碰撞，效率高。

2）、不需要依赖数据库等第三方系统，稳定性更高，可以根据自身业务分配bit位，非常灵活。

3）、生成ID的性能也非常高，每**秒能生成26万个自增可排序的ID**。



当然，雪花算法那也有缺点，因为它依赖机器时钟，如果机器时钟回拨，可能会导致ID重复。如果再分布式环境下，每台机器上的时钟不可能完全同步，有时候会出现不是全局递增的情况。当然，大部分情况下可以忽略这一缺点。

【结尾】

其实雪花算法，本身并不复杂。我们只要把它的设计理念和思想讲明白，就能够获得面试官的认可，以上就是我对雪花算法的理解。最后，我用Java代码实现了一个雪花算法，有兴趣的小伙伴可以在评论区留言获取，加深一下理解。

（以下代码可用文档编辑滑动展示）

请至钉钉文档查看附件《20220928_133408.mp4》

```
public class SnowFlake {

    /**

     * 开始时间截 (2015-01-01)

     */
```

```
private final long twepoch = 1420041600000L;



/**
 * 机器 id 所占的位数
 */
private final long workerIdBits = 5L;



/**
 * 数据标识 id 所占的位数
 */
private final long dataCenterIdBits = 5L;



/**
 * 支持的最大机器 id，结果是 31 (这个移位算法可以很快的计算出几位二进制数所能表示的最大十进制数)
 */
private final long maxWorkerId = ~(-1L << workerIdBits);

/**
 * 支持的最大数据标识 id，结果是 31
```

```
 */
private final long maxDataCenterId = ~(-1L << dataCenterIdBits);



/**
 * 序列在 id 中占的位数
 */
private final long sequenceBits = 12L;



/**
 * 机器 ID 向左移 12 位
 */
private final long workerIdShift = sequenceBits;



/**
 * 数据标识 id 向左移 17 位(12+5)
 */
private final long dataCenterIdShift = sequenceBits + workerIdBits;



/**
```

```
 * 时间截向左移 22 位(5+5+12)

 */

private final long timestampLeftShift = sequenceBits + workerIdBits + dataCenterIdBits;



/**

 *

 * 生成序列的掩码，这里为 4095 (0b111111111111=0xfff=4095)

 */

private final long sequenceMask = ~(-1L << sequenceBits);



/**

 * 工作机器 ID(0~31)

 */

private volatile long workerId;



/**

 * 数据中心 ID(0~31)

 */

private volatile long dataCenterId;
```

```
    /**
     * 毫秒内序列(0~4095)
     */
    private volatile long sequence = 0L;

    /**
     * 上次生成 ID 的时间截
     */
    private volatile long lastTimestamp = -1L;


//==============================Constructors=================================

    /**
     * 构造函数
     *
     * @param workerId     工作 ID (0~31)
```

```
    * @param dataCenterId 数据中心 ID (0~31)
    */


   public SnowFlake(long workerId, long dataCenterId) {
       if (workerId > maxWorkerId || workerId < 0) {
           throw new IllegalArgumentException(String.format("worker Id can't be greater than %d or less than
0", maxWorkerId));
       }
       if (dataCenterId > maxDataCenterId || dataCenterId < 0) {
           throw new IllegalArgumentException(String.format("dataCenter Id can't be greater than %d or less
than 0", maxDataCenterId));
       }
       this.workerId = workerId;
       this.dataCenterId = dataCenterId;
   }


   //
==============================Methods==================================
=========
```

```
/**
 * 获得下一个 ID (该方法是线程安全的)
 *  如果一个线程反复获取 Synchronized 锁，那么 synchronized 锁将变成偏向锁。
 * @return SnowflakeId
 */
public synchronized long nextId() throws RuntimeException {
    long timestamp = timeGen();

    //如果当前时间小于上一次 ID 生成的时间戳，说明系统时钟回退过这个时候应当抛出异常
    if (timestamp < lastTimestamp) {
        throw new RuntimeException((String.format("Clock moved backwards.  Refusing to generate id for %d milliseconds", lastTimestamp - timestamp)));

    }

    //如果是同一时间生成的，则进行毫秒内序列
    if (lastTimestamp == timestamp) {
        sequence = (sequence + 1) & sequenceMask;
        //毫秒内序列溢出
```

```
        if (sequence == 0) {
            //阻塞到下一个毫秒,获得新的时间戳
            timestamp = tilNextMillis(lastTimestamp);
        }
    }
    //时间戳改变，毫秒内序列重置
    else {
        sequence = 0L;
    }

    //上次生成 ID 的时间截
    lastTimestamp = timestamp;

    //移位并通过或运算拼到一起组成 64 位的 ID
    return ((timestamp - twepoch) << timestampLeftShift)
            | (dataCenterId << dataCenterIdShift)
            | (workerId << workerIdShift)
            | sequence;
}
```

```
/**
 * 阻塞到下一个毫秒，直到获得新的时间戳
 *
 * @param lastTimestamp 上次生成 ID 的时间截
 * @return 当前时间戳
 */
private long tilNextMillis(long lastTimestamp) {
    long timestamp = timeGen();
    while (timestamp <= lastTimestamp) {
        timestamp = timeGen();
    }
    return timestamp;
}

/**
 * 返回以毫秒为单位的当前时间
 *
 * @return 当前时间(毫秒)
```

```
    */
    private long timeGen() {
        return System.currentTimeMillis();
    }


}
```

128、如何理解Java中眼花缭乱的各种并发锁?

【引言】

在互联网公司面试中，很多小伙伴都被问到过关于锁的问题。

今天，我给大家一次性把Java并发锁的全家桶彻底讲明白。包括互斥锁、读写锁、重入锁、公平锁、悲观锁、自旋锁、偏向锁等等等等。视频有点长，大家一定要全部看完，保证你会醍醐灌顶。

【导航条：锁的由来】

（转场，屏幕黑底白字，正中央出现“1 锁的由来）

1、锁的由来

在并发编程中，经常会遇到两个以上的线程访问同一个共享变量，当同时对共享变量进行读写操作时，就会产生数据不一致的情况。

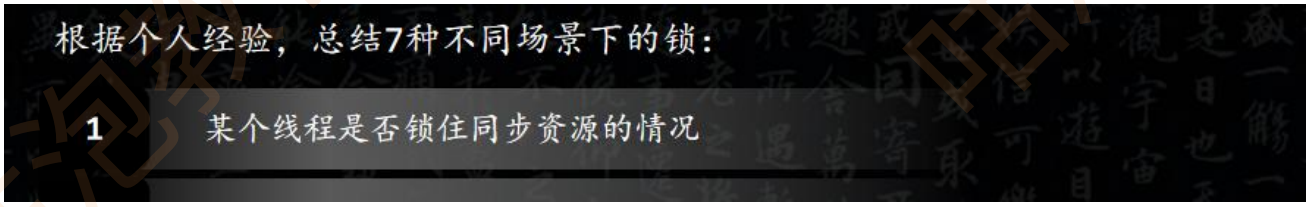
随着线程并发技术的发展，在多线程环境中，对线程访问资源的限制也越来越多。为了保证资源获取的有序性和占用性，都是通过并发锁来控制的。

【导航条：锁的应用场景】

（转场，屏幕黑底白字，正中央出现“2 锁的应用场景）

2、锁的应用场景

下面，我根据个人经验以及并发场景下线程的处理逻辑，总结为以下7个场景，不同场景使用不同的锁。



1）某个线程是否锁住同步资源的情况

如果要锁住同步资源则使用悲观锁，不锁住同步资源使用乐观锁。

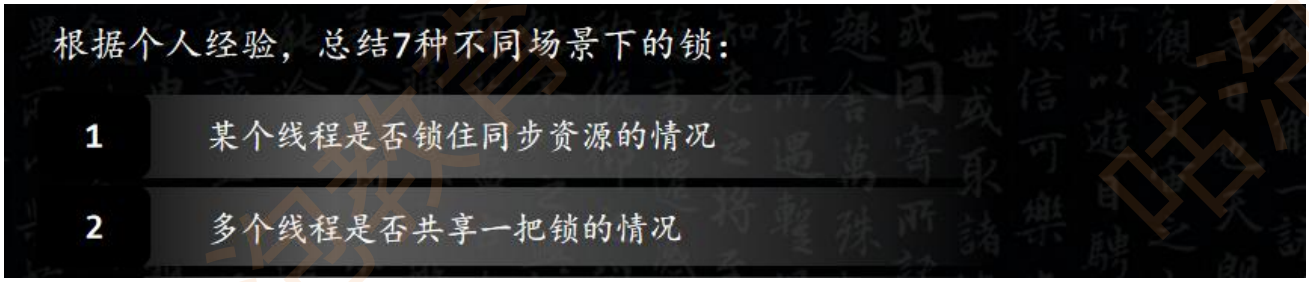
所谓悲观锁，就是每次拿数据的时候都认为会有别人修改，所以在读数据的时候都会上锁，其他线程数据就会阻塞，直到拿到锁。

举个例子，假设厕所只有一个坑位，悲观锁就是上厕所会第一时间把门反锁上，这样其他人上厕所只能在门外等候，这就是阻塞。

而乐观锁就是开着门，当然在这个场景下一般也不会这么做。所以，乐观锁，就是每次拿数据的时候都假设为别人不会修改，所以不会上锁；只是在更新数据的时候去判断之前有没有别的线程更新了这个数据。如果这个数据没有被更新，当前线程将自己修改的数据成功写入。如果数据已经被其他线程更新了，要么报错，要么自动重试。

乐观锁与悲观锁是一种广义上的概念，没有谁优谁劣。乐观锁适用于写少读多的场景，因为不用上锁、释放锁，省去了锁的开销，从而提升了吞吐量。

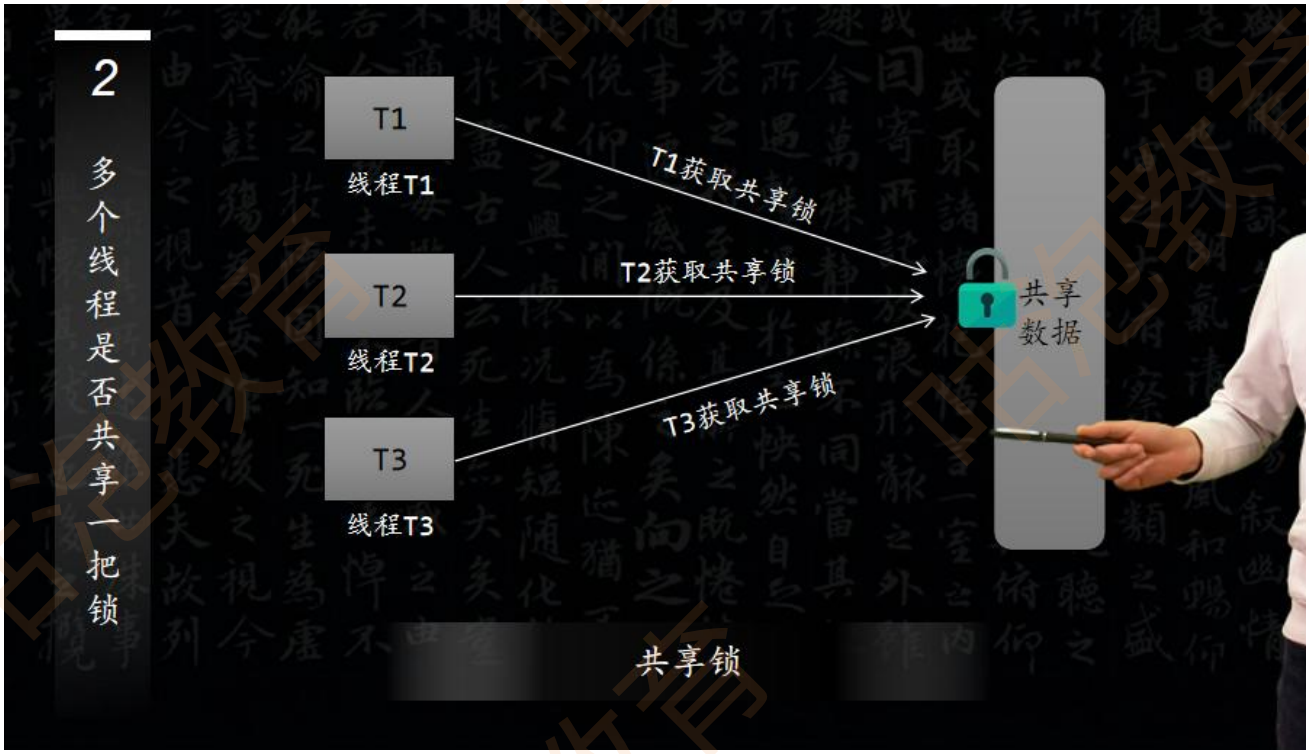
而悲观锁适用于写多读少的场景，因为线程间竞争激励，如果使用乐观锁会导致线程不断进行重试，这样反而还降低了性能。



2）多个线程是否共享一把锁的情况

如果在并发情况下，多个线程共享一把锁就是使用共享锁，如果不能共享一把锁就是排它锁或者叫独占锁、独享锁。

共享锁是指锁可被多个线程所持有。如果一个线程对数据加上共享锁后，那么其他线程只能对数据再加共享锁，不能加独占锁。获得共享锁的线程只能读数据，不能修改数据。

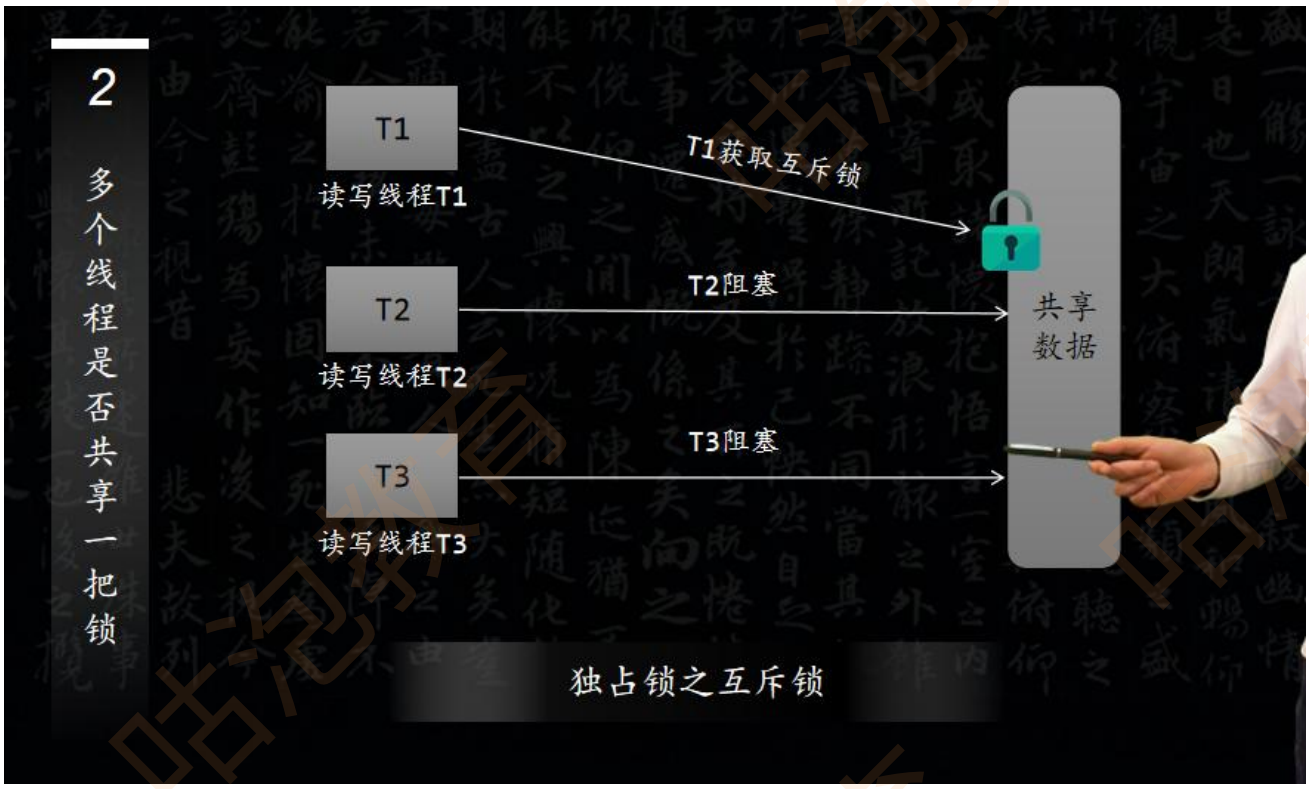


在 JDK 中 ReentrantReadWriteLock 就是一种共享锁。

而独占锁是指锁一次只能被一个线程所持有。如果一个线程对数据加上排他锁后，那么其他线程不能再对该数据加任何类型的锁。获得独占锁的线程即能读数据又能修改数据。

JDK中的synchronized和J.U.C(java.util.concurrent)包中Lock的实现类都是独占锁。

另外，互斥锁是独占锁的一种常规实现，是指某一资源同时只允许一个访问者对其进行访问，具有唯一性和排它性。



互斥锁一次只能一个线程拥有互斥锁，其他线程只有等待。

而读写锁是共享锁的一种具体实现。读写锁管理一组锁，一个是只读的锁，一个是写锁。

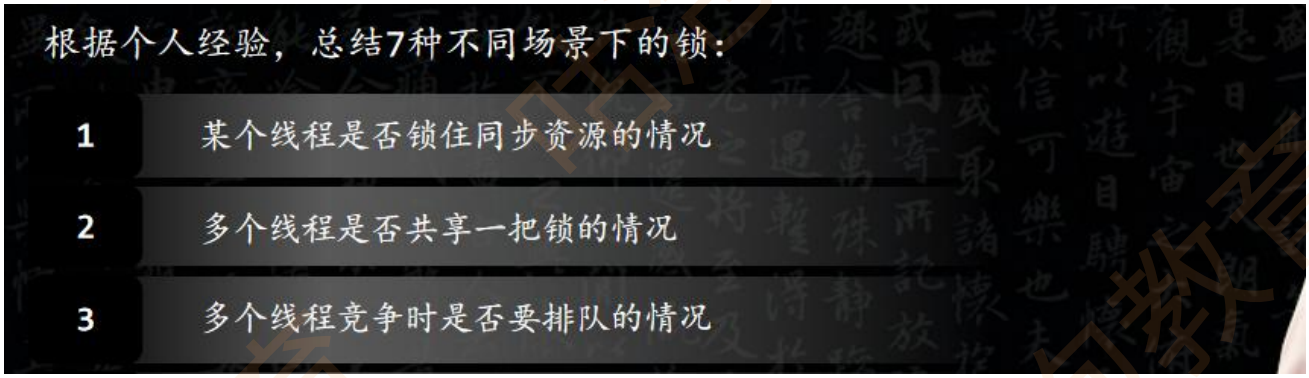
读锁可以在没有写锁的时候被多个线程同时持有，而写锁是独占的。写锁的优先级要高于读锁，一个获得了读锁的线程必须能看到前一个释放的写锁所更新的内容。

读写锁相比于互斥锁并发程度更高，每次只有一个写线程，但是同时可以有多个线程并发读。

在 JDK 中定义了一个读写锁的接口ReadWriteLock，如源码所示：

```
public interface ReadWriteLock {

 /* 获取读锁 */

 Lock readLock();

 /* 获取写锁 */

 Lock writeLock();

}
```

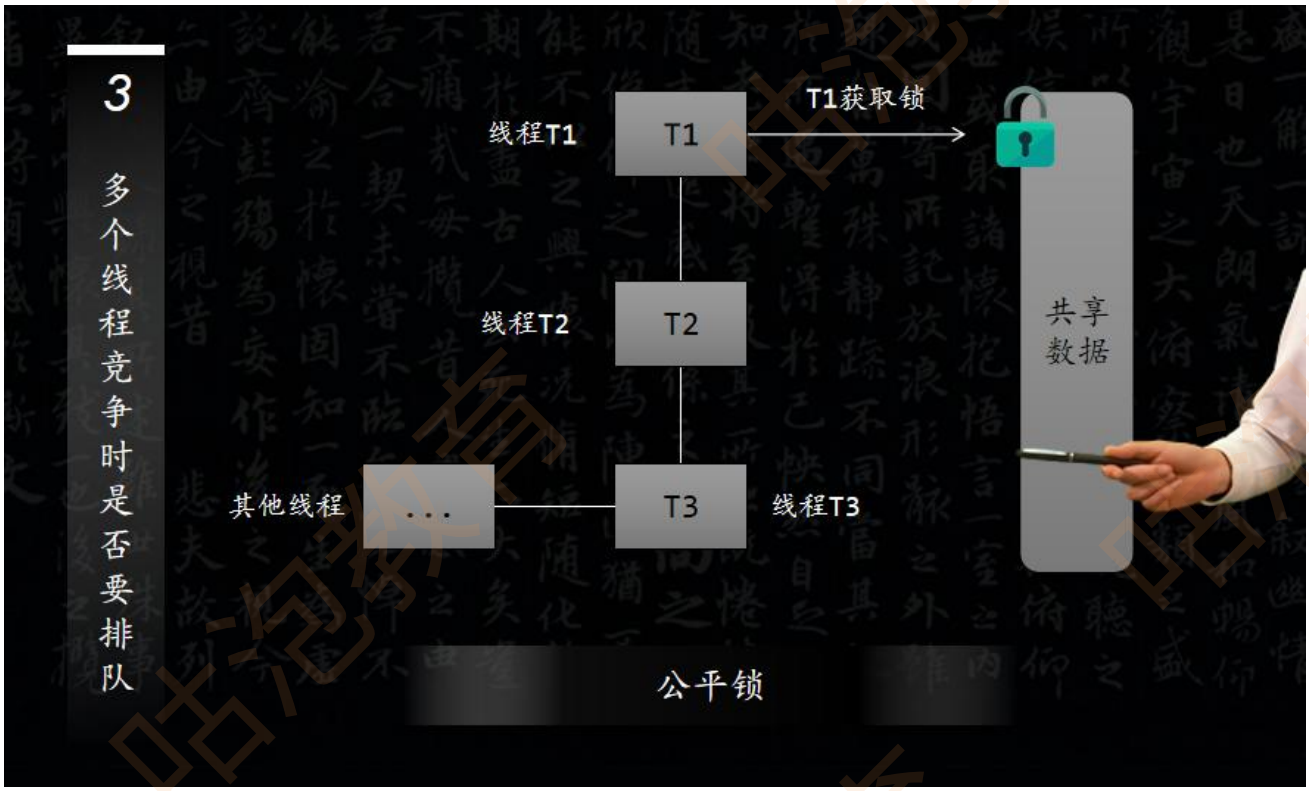
ReentrantReadWriteLock 实现了ReadWriteLock接口，ReentrantReadWriteLock 支持锁降级不支持锁升级，可以由写锁降为读锁。



3）多个线程竞争时是否要排队的情况

多个线程竞争排队获取锁的情况，使用公平锁，如果，使用非公平锁。

所谓公平锁是指多个线程按照申请锁的顺序来获取锁，这里类似排队买票，先来的人先买，后来的人在队尾排着，这是公平的。



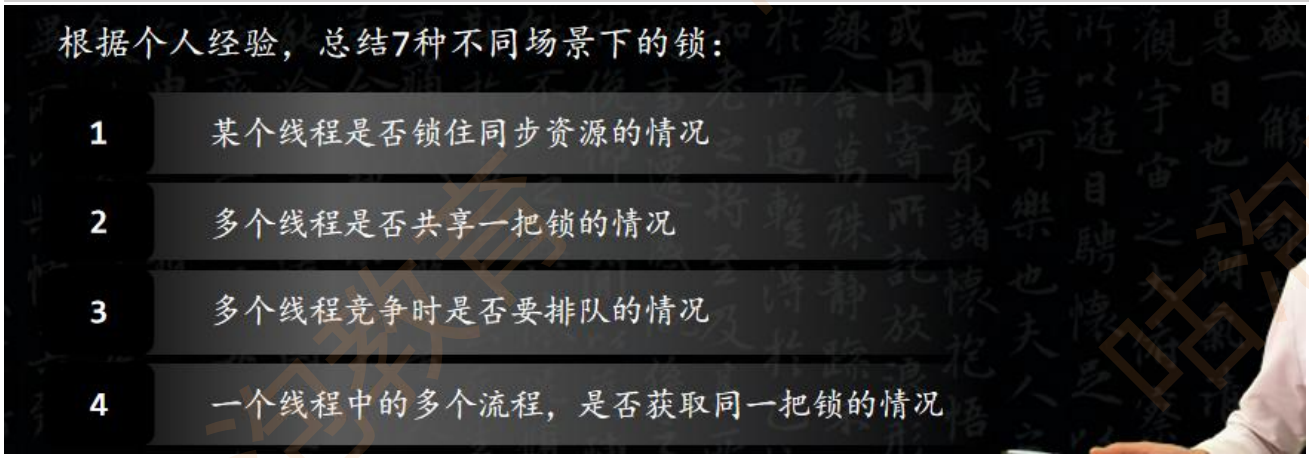
在 Java 中可以通过构造函数初始化公平锁，如代码所示：

```
/**
 * 创建一个可重入锁，
 * true 表示公平锁，
 * false 表示非公平锁。
 * 默认非公平锁
 */
Lock lock = new ReentrantLock(true);
```

非公平锁是指多个线程获取锁的顺序并不是按照申请锁的顺序，有可能后申请的线程比先申请的线程优先获取锁，在高并发环境下，有可能造成优先级翻转，或者某个线程一直得不到锁的饥饿状态。

在 Java 中 synchronized 关键字是非公平锁，ReentrantLock默认也是非公平锁，如代码所示：

```
/*** 创建一个可重入锁，true 表示公平锁，false 表示非公平锁。默认非公平锁*/

Lock lock = new ReentrantLock(false);
```



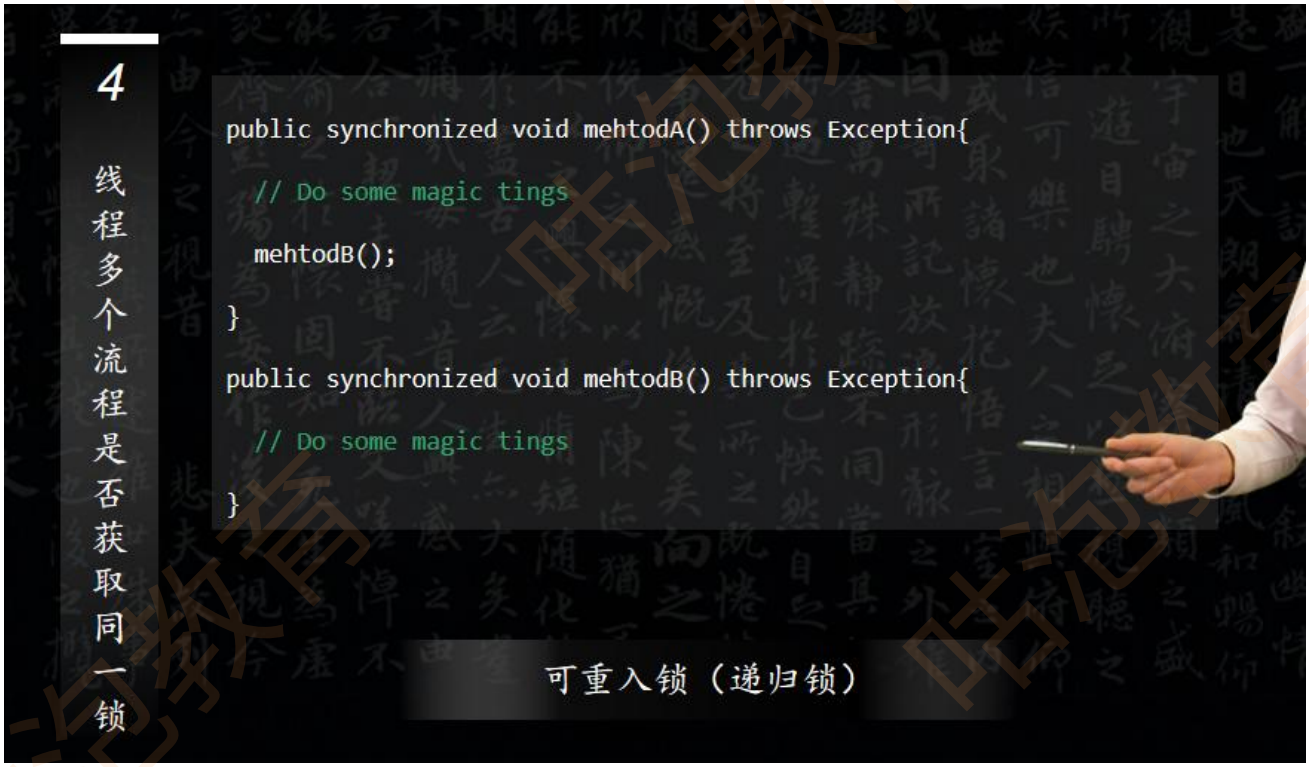
4）一个线程中的多个流程，是否获取同一把锁的情况

如果一个线程中的多个流程能获取同一把锁，就使用可重入锁，如果线程的多个流程不能获取通一把锁，就是用不可重入锁。

可重入锁又称为递归锁，是指同一个线程在外层方法获取了锁，在进入内层方法会自动获取锁。

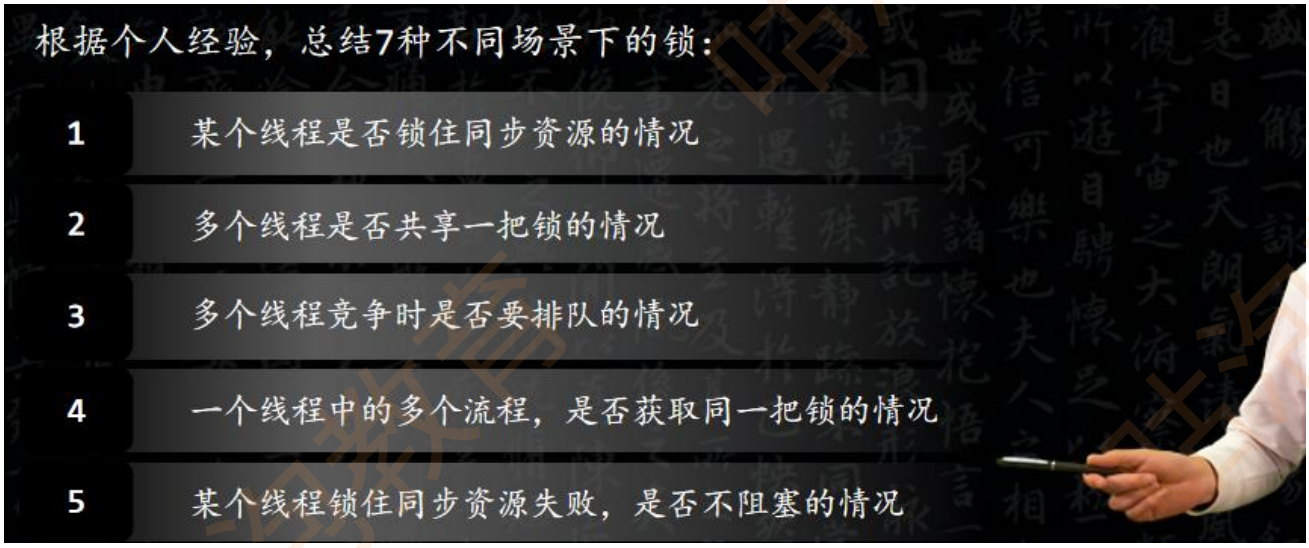
对于Java ReentrantLock而言, 他的名字就可以看出是一个可重入锁。对于synchronized而言，也是一个可重入锁。可重入锁的一个好处是可一定程度避免死锁。以 synchronized 为例，来看这样一段代码：

```
public synchronized void mehtodA() throws Exception{

  // Do some magic tings

  mehtodB();

}

public synchronized void mehtodB() throws Exception{

  // Do some magic tings

}
```



在这段代码中 methodA() 调用 methodB()，如果一个线程调用methodA() 已经获取了锁再去调用methodB() 就不需要再次获取锁了，这就是可重入锁的特性。如果是不可重入锁的话，mehtodB() 可能不会被

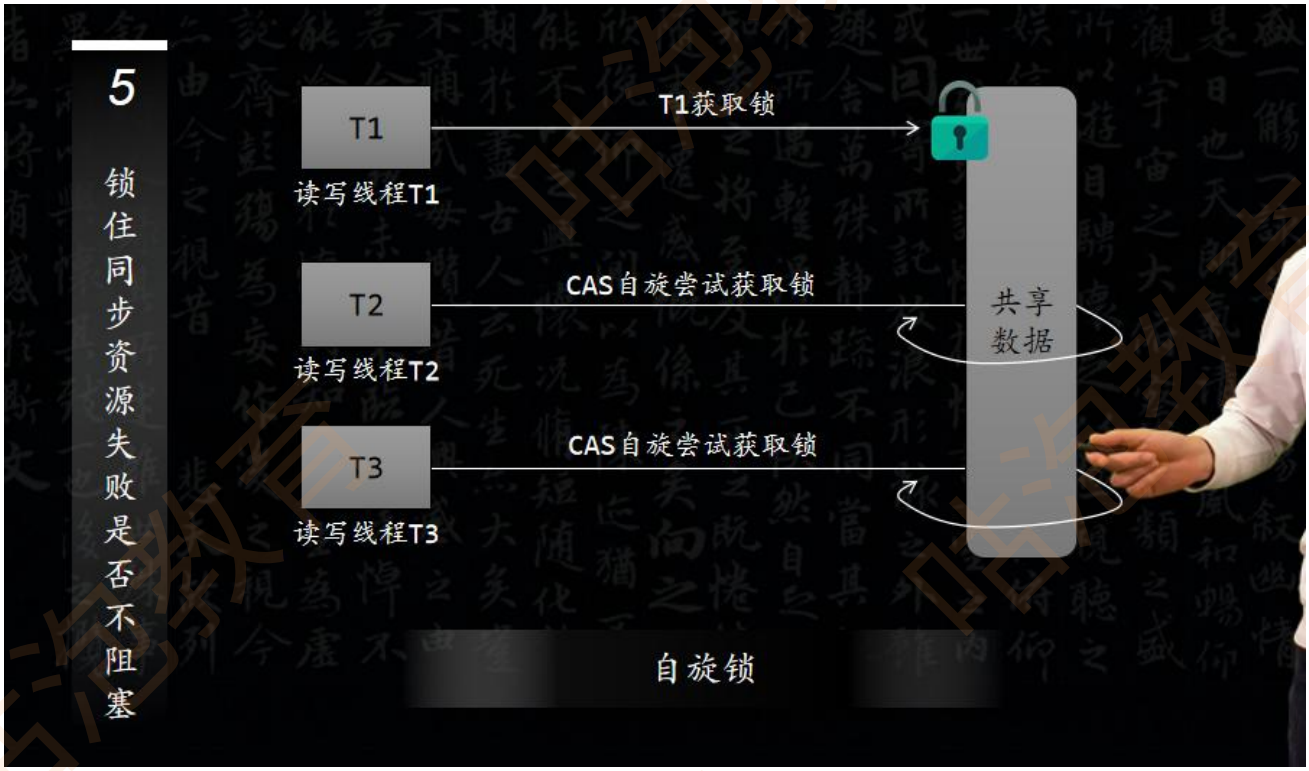
当前线程执行，可能造成死锁。



5）某个线程锁住同步资源失败，是否不阻塞的情况

如果某个线程锁住同步资源失败，但是希望这个线程不阻塞，就可以使用自旋锁或者自适应自旋锁。

自旋锁是指线程在没有获得锁时不是被直接挂起，而是执行一个忙循环，这个忙循环就是所谓的自旋。

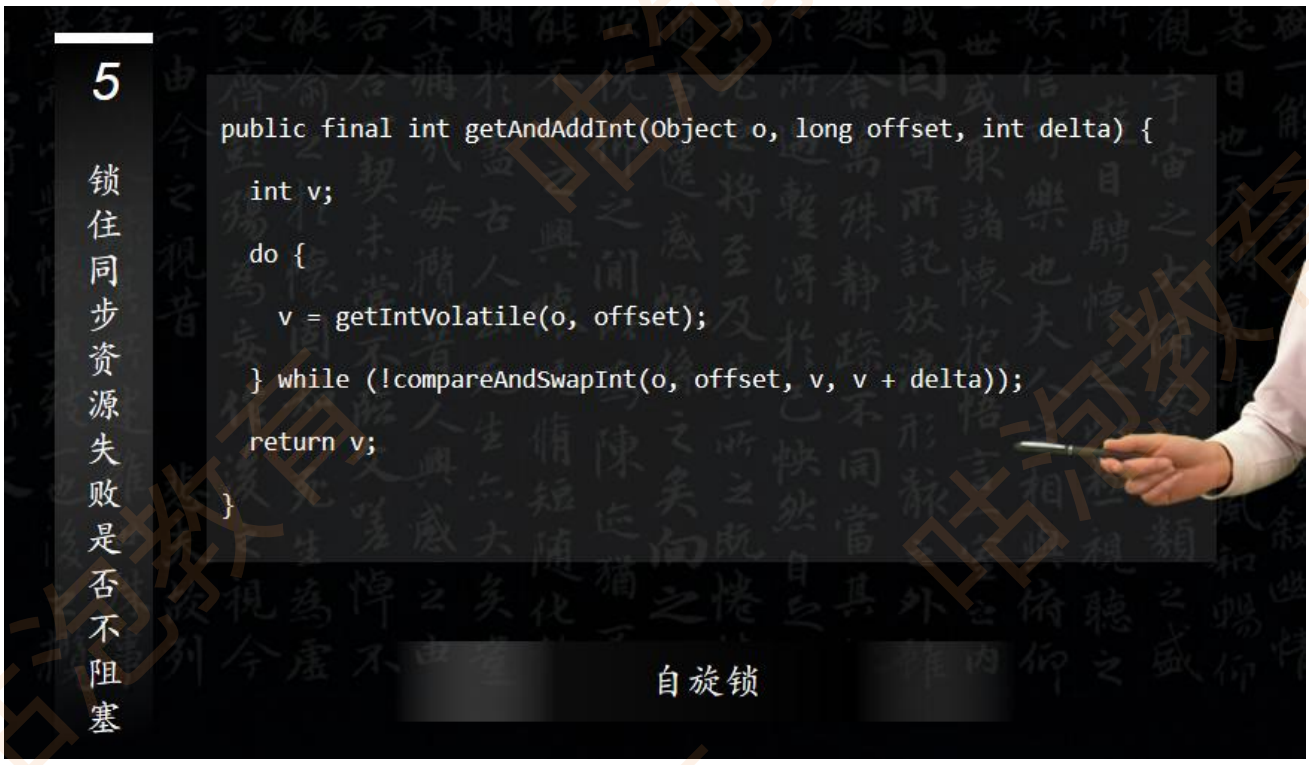


自旋锁的目的是为了减少线程被挂起的几率，因为线程的挂起和唤醒也都是耗资源的操作。

如果锁被另一个线程占用的时间比较长，即使自旋了之后当前线程还是会被挂起，忙循环就会变成浪费系统资源的操作，反而降低了整体性能。因此自旋锁是不适应锁占用时间长的并发情况的。

在 Java 中，AtomicInteger 类就有自旋的操作，来看这样一段代码：

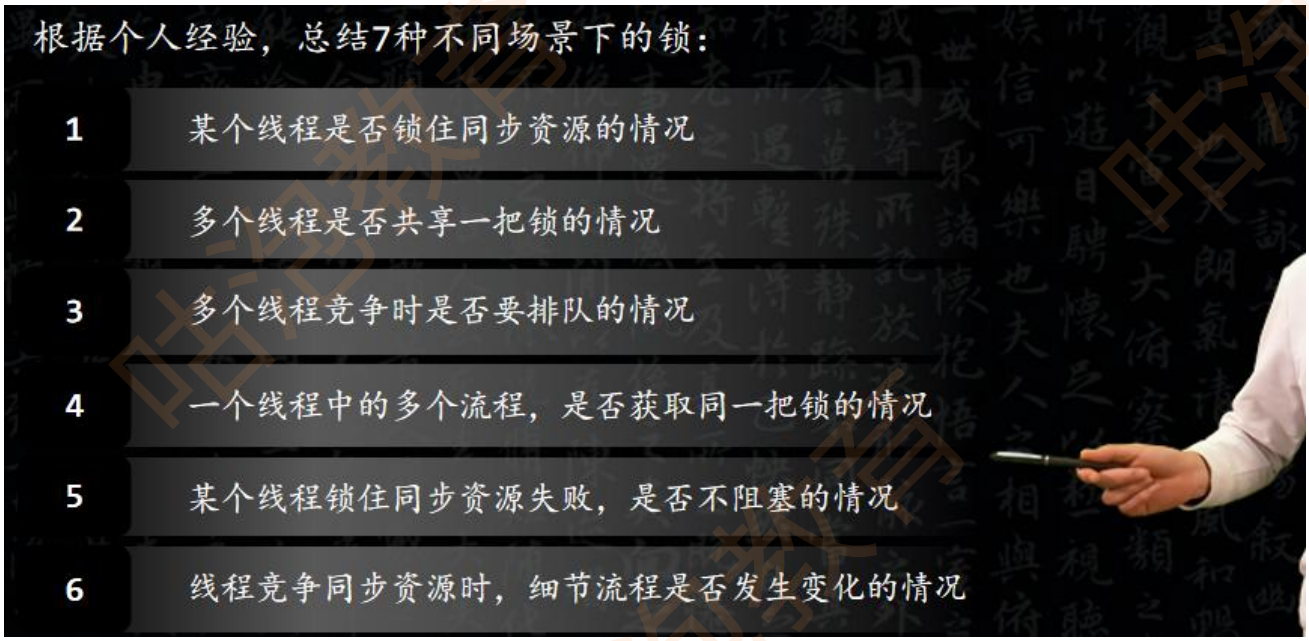
```
public final int getAndAddInt(Object o, long offset, int delta) {
 int v;
 do {
  v = getIntVolatile(o, offset);
 } while (!compareAndSwapInt(o, offset, v, v + delta));
 return v;
}
```



循环条件调用compareAndSwapInt()方法，被称为CAS操作，如果失败就会一直循环获取当前 value 值然

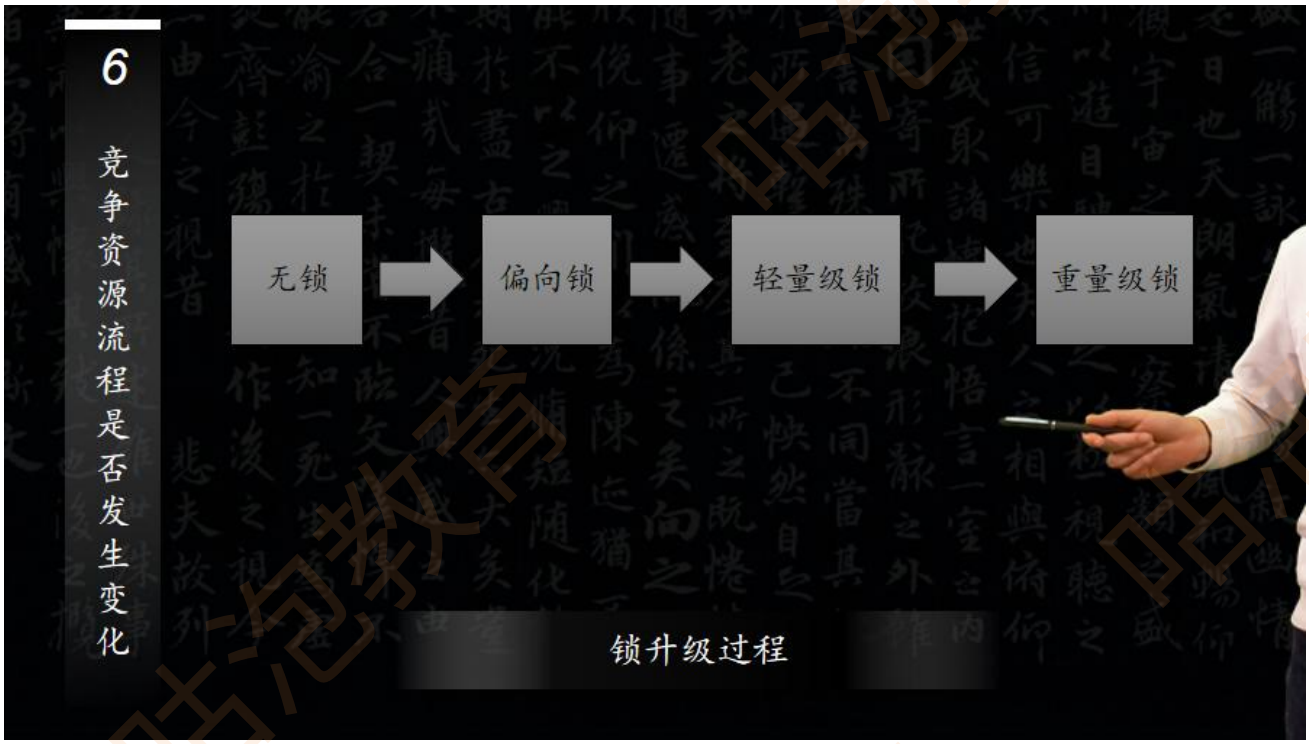
后重试，这个过程叫自旋。

在JDK1.6引入了自适应自旋，这个就比较智能了，自旋时间不再固定，由前一次在同一个锁上的自旋时间以及锁的拥有者的状态来决定。如果虚拟机认为这次自旋也很有可能再次成功那就会次序较多的时间，如果自旋很少成功，那以后可能就直接省略掉自旋过程，避免浪费处理器资源。



6）线程竞争同步资源时，细节流程是否发生变化的情况

JDK1.6 为了提升性能减少获得锁和释放锁所带来的消耗，引入了4种锁的状态：无锁、偏向锁、轻量级锁和重量级锁，它会随着多线程的竞争情况逐渐升级，但不能降级。



如果多个线程中，只有一个线程能修改资源成功，其他资源只是重试，不锁住资源，称为无锁状态，其实就是乐观锁。

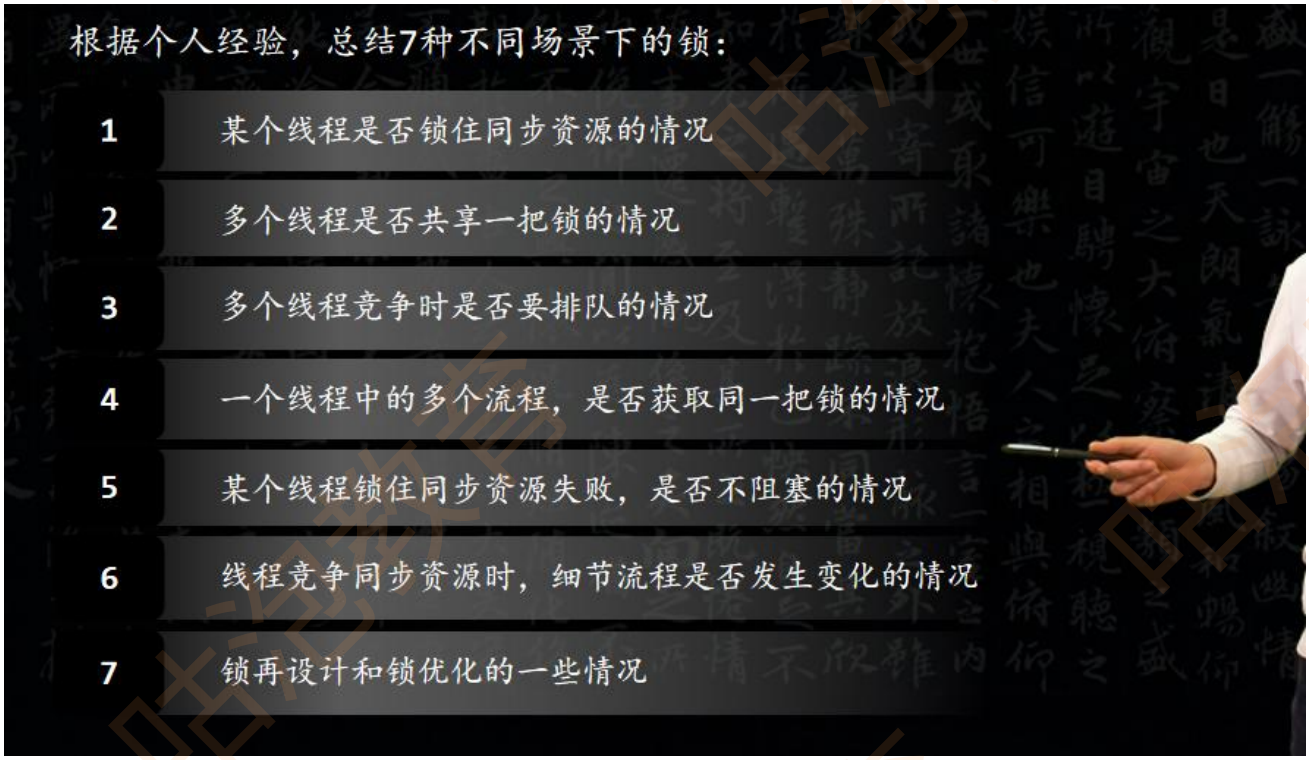
第一个线程访问加锁的资源自动获取锁，不存在多线程竞争的情况，资源偏向于第一个访问锁的线程，每次访问线程不需要重复获取锁，这种状态称为偏向锁。偏向锁的实现是通过控制对象Mark Word的标志位来实现的，如果当前是可偏向状态，需要进一步判断对象头存储的线程 ID 是否与当前线程 ID 一致，如果一致直接进入。

当线程竞争变得比较激烈时，偏向锁就会升级为轻量级锁，轻量级锁认为虽然竞争是存在的，但是理想情况下竞争的程度很低，通过自旋方式等待上一个线程释放锁。

但如果线程并发进一步加剧，线程的自旋超过了一定次数，或者一个线程持有锁，一个线程在自旋，又来了第三个线程访问的时候，轻量级锁就会膨胀为重量级锁，重量级锁会使除了当时拥有锁的线程以外的所有线程都阻塞。

升级到重量级锁其实就是互斥锁了，一个线程拿到锁，其余线程都会处于阻塞等待状态。

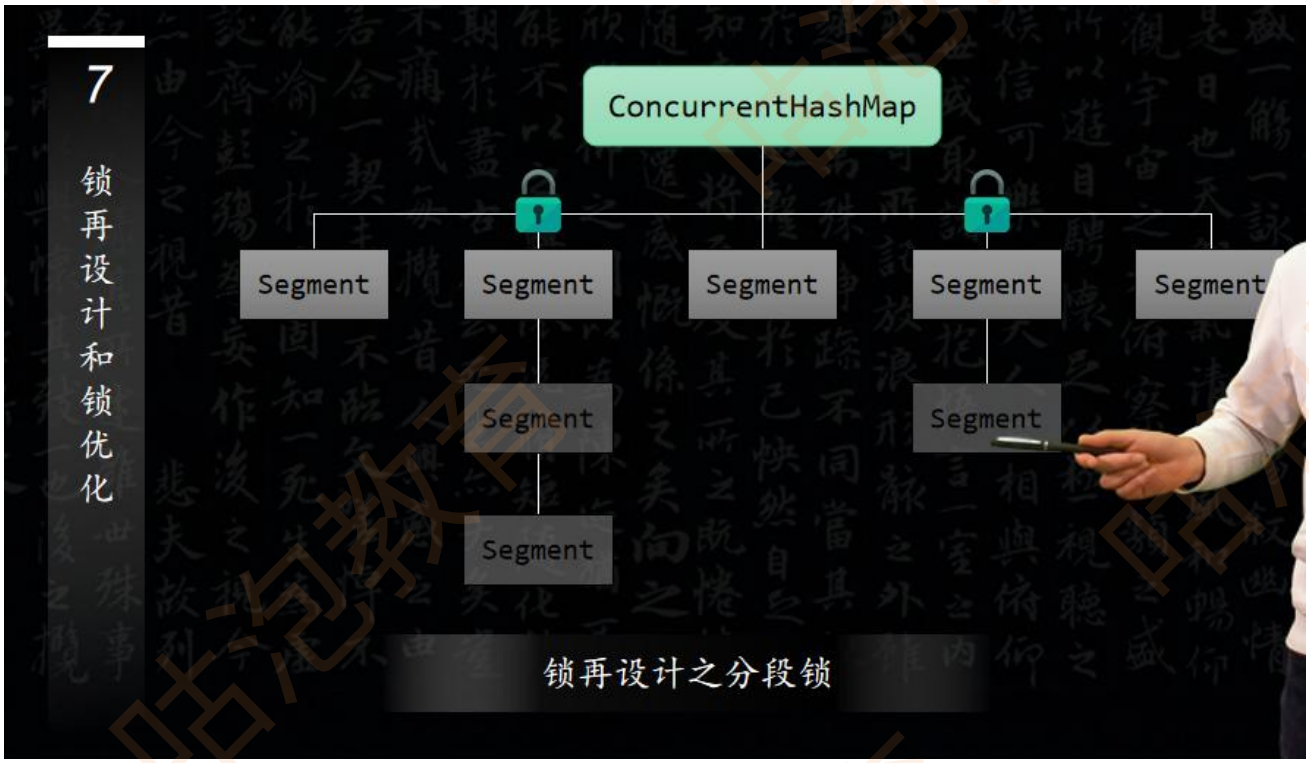
在 Java 中，synchronized 关键字内部实现原理就是这样一个锁升级的过程。



7）最后，就是锁再设计和锁优化的一些情况

先来看分段锁，它是一种锁的再次设计，并不是具体的一种锁。

分段锁设计目的是将锁的粒度进一步细化，当操作不需要更新整个数组的时候，就仅仅针对数组中的一项进行加锁操作。



在 Java 语言中 CurrentHashMap 底层使用分段锁Segment，来支持多线程并发操作。

另外，就是锁优化，包括锁消除、锁粗化。

锁粗化就是将多个同步块的数量减少，并将单个同步块的作用范围扩大，本质上就是将多次上锁、解锁的请求合并为一次同步请求。

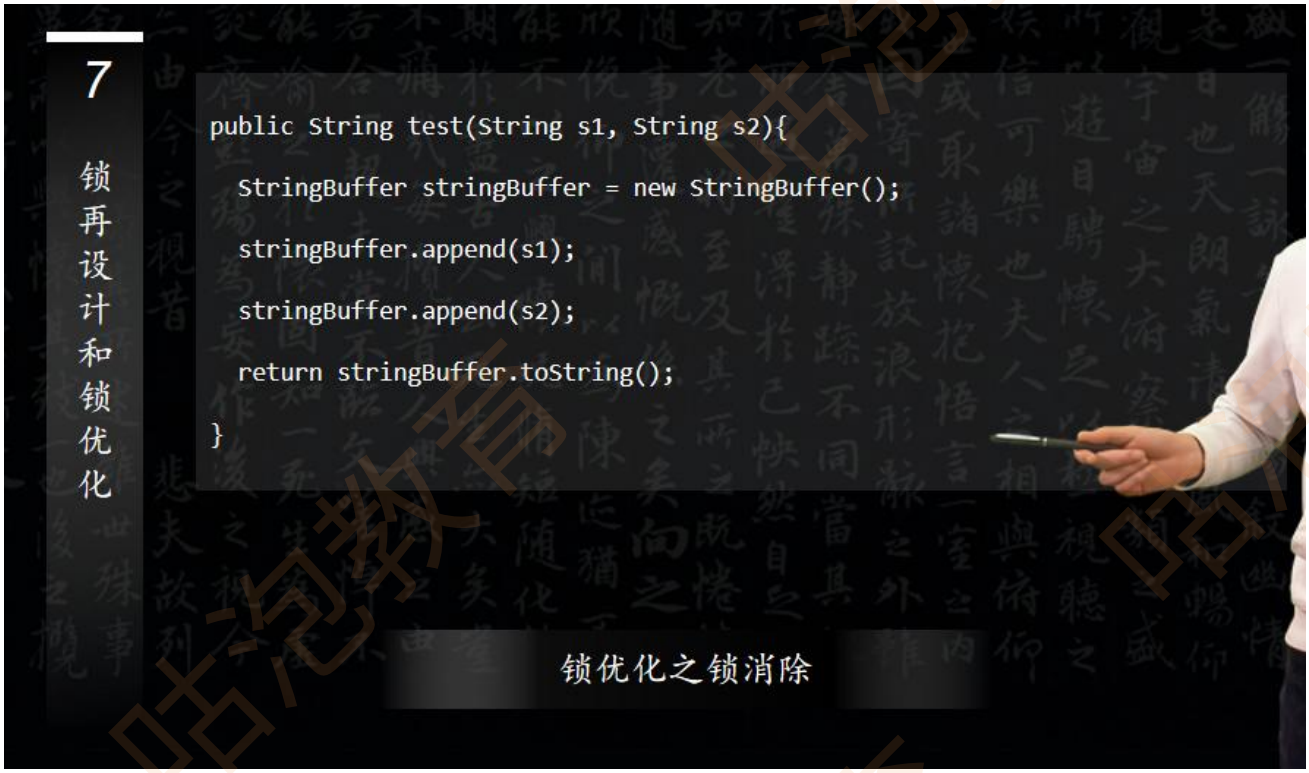
举个例子，一个循环体中有一个代码同步块，每次循环都会执行加锁解锁操作。如代码所示：

```
private static final Object LOCK = new Object();

for(int i = 0;i < 100; i++) {

 synchronized(LOCK){

   // do some magic things

 }

}
```

经过锁粗化后，就变成下面这个样子：

```
synchronized(LOCK){
  for(int i = 0;i < 100; i++) {
    // do some magic things
  }
}
```

锁消除是指虚拟机编译器在运行时检测到了共享数据没有竞争的锁，从而将这些锁进行消除。举个例子让大家更好理解，来看这样一段代码：

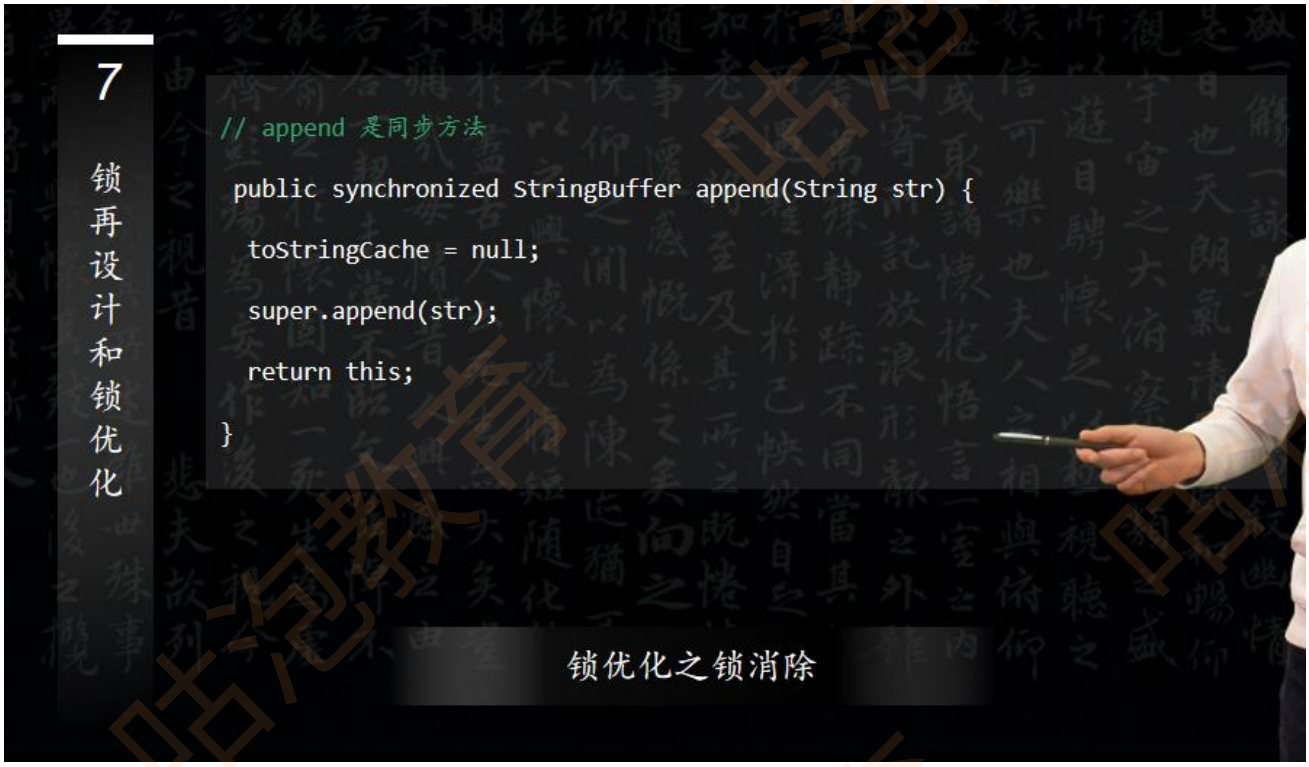



```
public String test(String s1, String s2){
 StringBuffer stringBuffer = new StringBuffer();
 stringBuffer.append(s1);
 stringBuffer.append(s2);
 return stringBuffer.toString();
}
```

上面代码中的test() 方法，主要作用是将字符串 s1 和字符串 s2 串联起来。

test() 方法中的三个变量s1, s2,和StringBuffer都是局部变量，局部变量是存储在栈上的，而栈又是线程私有的，所以就算有多个线程访问 test() 方法也是线程安全的。

我们都知道 StringBuffer 是线程安全的类，因为append()方法是同步方法，如源码所示：

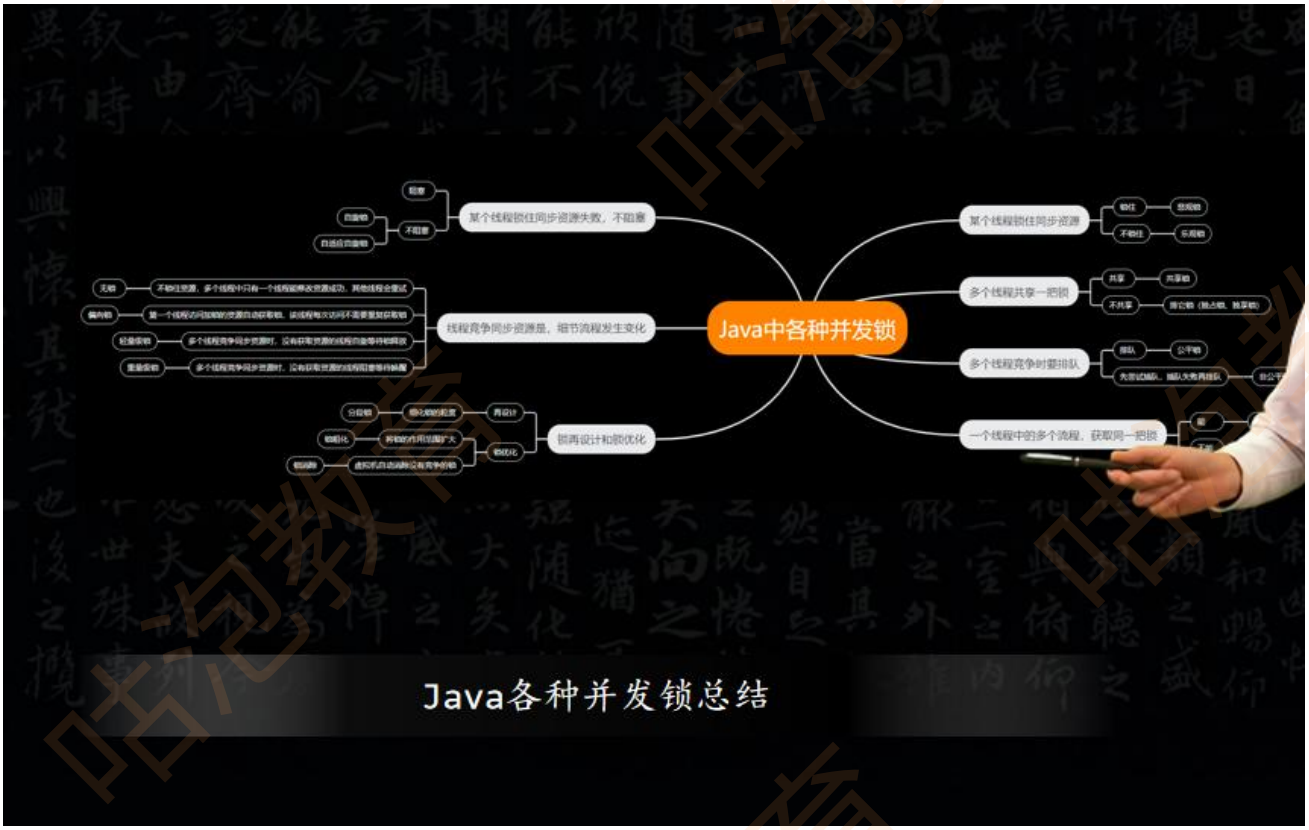



```
// append 是同步方法
public synchronized StringBuffer append(String str) {
 toStringCache = null;
 super.append(str);
 return this;
}
```

但是 test() 方法本来就是线程安全的，为了提升效率，虚拟机自动帮我们消除了这些同步锁，这个过程就被称为锁消除。

【结尾】

好了，前面讲了这么多，相信大家已经理解了各种眼花缭乱的锁。最后，我用一张脑图完整地总结了各种锁的应用场景。大家可以在面试前拿出来看看，只要是被问到锁相关的问题，相信你一定能够吊打面试官了。



脑图分享链接：https://www.processon.com/view/link/633412ea07912955b20d7938

最后，分享几个关于锁的高频面试题，看看大家能不能回答出来

1、ReentrantLock与synchronized 的区别

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

2、synchronized和volatile的区别

3、synchronized和lock的区别

4、什么是死锁以及如何避免死锁问题

如果你回答不出来，可以去我的主页看看，这些面试题在往期的视频中都有分享过。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

129、数据量大了，就一定要分库分表吗？

【引言】

有位小伙伴在阿里的面试中，被问到，说只要数据量大了，就一定要分库分表吗？如果你直接回答，是的话，这大概率就会被Pass了。

今天，我给大家分享一下，我的理解。

【导航条：分库分表】

（转场，屏幕黑底白字，正中央出现“1 分库分表）

1、分库分表

分库和分表不能一概而论。分库和分表各自的应用场景，以及它们要解决的问题也都不一样。下面我从三个方面来分析，分别是只分库不分表、只分表不分库、既分库又分表这三种应用场景。

1)、只分库不分表

当数据库的读写访问量过高，还有可能会出现数据库连接不够用的情况。这个时候我们就需要考虑分库，通过增加数据库实例的方式来获得更多的数据库连接，从而提升系统的并发性能。

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

2)、只分表不分库

当单表存储的数据量非常大的情况下，并且并发量也不高，数据库的连接也还够用。但是数据写入和查询的性能出现了瓶颈，这个时候就需要考虑分表了。将数据拆分到多张表中来减少单表存储的数据量，从而提升读写的效率。

3)、既分库又分表

结合前面的两种情况，如果同时满足前面的两个条件，也就是数据连接也不够用，并且单表的数据量也很大，从而导致数据库读写速度变慢的情况，这个时候就要考虑既分库又分表。

【导航条：结尾】

当然，我上面所述的只是每种情况的使用场景，并不是绝对的。有些小伙伴可能会说，很多系统在开发之出就已经完成分库分表了，那是不是过度设计呢?

要不要提前设计分库分表，还要根据业务的发展来考虑。在《阿里开发手册》中建议，单表行数超过500万行或者单表容量超过2GB，才推荐进行分库分表，如果预计三年后数据量根本达不到这个级别，就不必要在创建表时就分库分表。

以上就是我对分库分表的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难!

130、什么时候才需要做数据库读写分离?

【引言】

很多小伙伴可能会认为数据库的负载问题就应该使用“读写分离”来解决。这其实是一个非常大的误区，

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

我们要使用“读写分离”，首先要搞明白“读写分离”能解决什么问题，而不仅仅只会使用这个技术。

【导航条：什么是读写分离】

（转场，屏幕黑底白字，正中央出现“1 什么是读写分离）

1、什么是读写分离

读写分离就是将数据库分为主数据库和从数据库，主库用来接收数据写入的操作，多个从库用来接收读数据的操作，主库将写入的数据通过一些机制实时同步到从库，比如Binlog等等。

【导航条：什么是读写分离】

（转场，屏幕黑底白字，正中央出现“2 读写分离解决什么问题？”）

2、读写分离解决什么问题?

大多数互联网公司的业务场景，往往都是读多写少。当访问量过大情况下，数据库查询首先会成为瓶颈，这个时候，如果我们希望能够线性的提升数据库的查询性能，消除读写锁冲突，从而提升数据库的写性能，那么就可以考虑采用读、写分离架构。用一句话概括，读写分离是用来解决数据库的读性能瓶颈的。

【导航条：什么时候需要读写分离】

（转场，屏幕黑底白字，正中央出现“3 什么时候需要读写分离？”）

3、什么时候需要读写分离?

并不是任何查询性能瓶颈都需要考虑读写分离的，如果仅仅只是因为并发压力， 是最低级的解决方案。解决查询性能瓶颈我们还可以考虑其他的解决方案，比如数据分片、缓存或者加入搜索引擎等等，这里就不详细

赘述，后面可以单独做一期视频来讲解。小伙伴们可以关注我的主页获取更多干货内容。

【导航条：结尾】

以上就是我对分库分表的理解。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

131、生产环境秒杀接口并发量井喷，你该怎么办？

【引言】

昨天有位小伙伴被面试官问到这样一个问题，说生产环境秒杀接口并发量井喷剧增，负载过高，快扛不住了，你该怎么办？你的第一反应会采取什么方案来解决？理由又是什么？

接下来，那我来分享一下我的方案。

【导航条：解决方案】

（转场，屏幕黑底白字，正中央出现“1 解决方案”）

1、解决方案

限流，先保证自己不挂；

临时动态扩容，加服务器；

分析具体原因，比如是线程池不足还是内存，GC等问题，判断是否可以做优化，比如通过异步队列，加缓存，优化数据库，

无法优化的情况下，有成本考虑重构，比如做DB水平拆分，应用的垂直拆分等等

最后加成本，加机器

遇到这样的问题，我的第一反应就是增加缓存。因为，增加缓存是解决系统性能问题最快速、最高效的方案，它能够快速提升系统的线性吞吐量，效果也最为明显。这就相当于是用空间来换时间。曾经还有人说过，缓存就是解决性能问题的万金油，哪里存在性能瓶颈就往哪里加缓存。

【导航条：选择】

（转场，屏幕黑底白字，正中央出现“2 选择理由”）

2、选择理由

其实，缓存的设计思想在架构设计中是非常常见的。比如我们每天用的OS操作系统，不管是Windows还是Linux包括Mac OS都有系统缓存、用户缓存。磁盘也有磁盘缓存区、CPU有CPU有缓存区。再比如，我们常用的经典框架中，也有经常使用到缓存，Spring有IoC缓存，MyBatis有一级缓存、二级缓存。在架构设计中，可以说缓存是无处不在。

【导航条：总结】

（转场，屏幕黑底白字，正中央出现“3 总结”）

3、总结

最后，总结一下。当我们并发量过高扛不住的时候，可以优先采用缓存来缓解负载压力。比如将读取频繁的数据写到缓存中，动态页面静态化。加上缓存之后，如果负载压力依然过大；再来考虑增加限流策略，比如消息队列；如果增加限流还是压力大，最后，再考虑增加服务器节点。

【导航条：结尾】

好了，以上就是我对这个问题的理解。那么如果你遇到接口并发过高扛不住的时候，你的首选方案是什么呢？小伙伴们可以分享到到评论区供大家学习一下。

我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难！

132、哪些场景不适合使用单例模式？

【导航条：引言】

经常有小伙伴问我，设计模式学了这么久，每次看到概念也都能理解。但是，就是不知道怎么用，在哪里能用？我告诉大家，设计模式，不是为了要用而用的，而是作为前人总结下来的经验，等到哪天需要用的时候，你能想起来为你所用。

今天我跟大家分享一下，什么场景下需要用单例模式，什么场景下不应该用单例模式？回答这个问题之前，我得先问小伙伴们一个问题。你觉得一家公司能有两个CEO吗?如果有两个CEO，请问谁做决策，员工应该听谁的？所以，在这种情况下，很显然公司的CEO只能是单例。

另外，往期面试题解析的配套文档我已经准备好，想获得的可以在我的煮叶简介中找到。好，我们先来看官方对单例的定义。

【导航条：官方定义】

（转场，屏幕黑底白字，正中央出现“1 官方定义”）

1、官方定义

官方原文是这样的：

Ensure a class has only one instance,and provide a global point of access to it.

翻译过来就是：

指确保一个类在任何情况下都绝对只有一个实例，并提供一个全局访问点

【导航条：如何理解】

（转场，屏幕黑底白字，正中央出现“2 如何理解”）

2、如何理解

那么，怎么来理解这句话呢？这句话有两个重点，就是任何情况和全局访问点。

任何情况：可能包括多线程、克隆、反序列化、反射等等各种情况。

全局访问点：意思就是统一获取单例对象的入口。

对照我一开始提出的CEO案例来理解的话，CEO应该被设计为单例，统一行动，统一指挥。如果出现多个CEO发出不同指令，员工就不知道听谁的了。

【导航条：】

（转场，屏幕黑底白字，正中央出现“3 使用建议”）

3、使用建议

单例模式适合在哪些场景下使用？

如果某个共享资源，使用频次非常高，而且不可替代性也很强，就应该被设计为单例。比如，Spring中的IoC容器就是单例、JDK中的Runtime也是单例。

那哪些场景不适合设计为单例模式呢?

比如要经常被赋值传递的对象，Vo、Pojo等等，就不适合设置为单例。

最后，一句总结单例模式就是保证独一无二。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难!

133.哪些场景不适合使用策略模式?

【导航条：引言】

滴，老年卡；滴，学生卡；滴正常卡。我们在坐公交车的时候啊，这个场景每天都在上演。那如果，让你来设计这样一套刷卡的结算逻辑，你最先想到的是用什么设计模式呢?

如果，让我来设计，我最先想到的就是策略模式。另外，我把往期面试题解析的配套文档我已经准备好，想获得的可以在我的煮叶简介中找到。那么什么场景要使用策略模式，什么场景又不应该使用策略模式呢？我们可以先来看官方对策略模式的定义。

1、官方定义

官方原文是:

Define a family of algorithms, encapsulate each one, and make them interchangeable.

翻译过来就是:

将定义的算法家族分别封装起来，让它们之间可以互相替换，从而让算法的变化不会影响到使用算法的用户，

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

2、如何理解

看上去，很抽象比较难理解。这里提到三个关键点，一个是封装，二是可以相互替换，三是就算算法变化也不能影响用户的使用。

那对照，前面的公交车刷卡的场景，每种卡的优惠折扣都不一样，并且也结算方式都不一样，但是每种卡可以相互交换使用。比如，增加一个卡种，残疾人卡，只需要重新增加一种折扣就行了，后续继续扩展其他卡种，也不影响已经有卡的用户使用。

在日常开发中，使用策略模式的场景还非常多，比如支付时对支付方式的选择、出行时，对交通工具的选择，等等等等。

3、使用建议

那策略模式适合在哪些场景下使用呢?

需要经常自由切换执行逻辑和规则的场景，就非常适合用策略模式，可以屏蔽算法规则之间的差异，交给用户选择，也可以减少程序中大篇幅的 if...else...判断。比如Spring源码中的Resource资源类，JDK源码中的Comparator类，都是策略模式的应用。

哪些场景下又不适合使用策略模式呢?

如果两种逻辑之间关联性本来就比较大，而且变化也比较快。这种就情况，就算if...else...再多，也不适合用策略模式。比如，每天都有促销活动，价格每天都在浮动，这种情况就不适合使用策略模式。

最后，一句总结策略模式就是把选择权交给用户。我是被编程耽误的文艺Tom，如果我的分享对你有帮助，请动动手指一键三连分享给更多的人。关注我，面试不再难!

三、一周技术提升清单

技术提升课程安排每周更新

请联系小助理获取最新课程安排

时间	主攻技术点	课程内容
11.28	分布式锁	Redis为什么能做分布式锁 Redission分布式锁怎么实现重入锁 Redission分布式锁看门狗机制原理分析
11.29-12.01	Spring Cloud Alibaba	从架构的演变重新认识Spring Cloud Alibaba 2.Spring Cloud Alibaba核心组件深度剖析
11.30	ConcurrentHashMap8源码解读	一切得先从HashMap1开始聊起 假如需要重构，你要做好哪些准备 真要重构，你应该如何进行技术选型 你的技术栈能否帮你实现架构重构

12.02	为什么MQ在分布式架构中大行其道	为什么要使用MQ RabbitMQ工作模型分析 Exchange路由机制详解 基于SpringBoot的RabbitMQ实战

四、薪资备查表

全国主要城市互联网行业各岗位薪资范围备查表（咕泡科技2022年版）

（最后更新2022.01，单位：K/月）

数据来源BOSS直聘

序号	城市	属性	Java S1（全栈开发）0-1年		Java S2（资深开发）1-3年		Java A1（架构师）3-5年		Java A2（百万架构师）5年以上		大数据S1（全栈开发）1-3年		大数据S2（资深开发）3-5年		ES S2（资深开发）3-5年		AI S1（全栈工程师）1-3年		AI S2（资深工程师）3-5年		测试 S1（初级工程师）1-3年		测试 S2（高级工程师）3-5年		前端 S1（初级开发）1-3年		前端 S2（资深开发）3-5年		前端 A1（架构师）5年以上	
			最低	最高	最低	最高	最低	最高	最低	最高	最低	最高	最低	最高	最低	最高	最低	最高	最低	最高	最低	最高	最低	最高	最低	最高	最低	最高	最低	最高
B	北京	一线	15	25	25	50	35	60	50	100	10	30	30	50	20	40	20	40	30	60	15	20	20	40	15	30	25	50	30	70
S	上海	一线	12	22	22	40	30	55	45	90	10	30	20	30	15	40	15	30	30	50	10	18	15	35	10	15	30	50	30	60
G	广州	一线	10	18	20	35	25	40	40	80	8	15	15	30	25	50	20	30	25	50	7	15	15	25	8	13	20	30	30	50
S	深圳	一线	13	20	25	45	35	50	50	90	11	25	20	50	25	50	20	40	30	60	10	22	20	40	15	25	30	50	40	60
H	杭州	一线	10	20	20	35	30	45	45	100	10	40	20	40	20	30	20	40	25	50	7	12	20	30	15	30	20	50	30	100
C	成都	新一线	10	15	15	30	25	50	30	60	8	20	20	40	20	40	15	30	20	40	8	13	10	15	7	12	15	25	20	40
	长沙	新一线	9	15	15	30	20	40	30	60	10	20	15	30	--	--	8	15	30	50	7	11	12	18	6	10	10	15	15	26
	重庆	二线	9	12	10	15	18	30	25	50	10	15	13	26	--	--	9	14	30	40	5	10	9	14	7	10	12	24	30	50
W	武汉	新一线	15	25	20	40	26	50	30	60	7	20	12	30	--	--	12	24	30	50	6	10	13	25	7	12	10	15	20	40
S	苏州	二线	10	15	20	30	25	40	30	60	8	20	25	35	12	16	13	23	20	30	5	12	10	20	5	10	15	25	20	40
T	天津	准一线	8	16	15	25	20	40	30	50	7	20	20	35	10	15	7	15	15	30	5	13	15	20	4	8	12	20	15	30
H	合肥	准二线	11	20	15	30	20	35	30	50	7	14	30	50	--	--	8	13	20	40	4	8	6	10	4	10	10	18	16	30
N	南京	新一线	11	20	15	25	25	45	30	50	10	24	25	50	--	--	19	35	20	40	4	8	10	15	13	26	20	40	25	45
X	西安	二线	8	15	12	18	16	30	30	50	13	26	20	40	20	40	15	30	17	34	8	18	15	40	4	9	10	20	25	50
	厦门	二线	10	15	14	20	15	30	30	50	10	15	20	30	--	--	15	20	20	40	6	12	12	20	8	13	18	30	30	50
W	无锡	准二线	9	15	13	22	18	35	30	50	8	12	15	25	--	--	10	15	15	25	4	6	7	12	4	8	8	15	25	40
Z	郑州	准二线	8	14	12	18	15	30	25	45	8	13	10	15	--	--	8	13	18	35	4	8	8	13	4	13	20	30	—	—

五、Java开发程序员职业路线图

Java互联网程序员职业成长路线图（2022版）
路线1
路线2
路线3
停靠站
路线4
路线5
路线6
换乘站
3路线
Line 3
（￥100W+/年）
首席技术官
Chief Technical Officer
（￥70-100W/年）
技术专家(P8)
Technical Proficient
培训讲师
Training Techer
（￥70-100W+/年）
4路线
Line 4
资深架构师(P7)
System Architect
（￥60-80W/年）
架构师(P6)
Application Architect
（￥30-60W/年）
技术方向
Technical Direction
管理方向
Senior Programmer
项目经理
Project Mamager
（￥20-50W/年）
普通家庭
Nomal Family
（￥1-2W/年）
入职搬砖
Entry Employment
（￥2-3W/年）
小白程序员
Introduction Programmer
（￥3-5W/年）
初级开发(P3)
Junior Programmer
（￥6-8W/年）
全栈开发(P4)
Intermediate Programmer
（￥8-15W/年）
资深开发(P5)
Senior Programmer
（￥15-30W/年）
1路线
Line 1
开始
Begin
家里有矿
Rich Family
2路线
Line 2
勿做程序员
Non Programmer
首席执行官
Chief Executive Officer
（￥60-100W+/年）
研发总监
R & D Director
（￥50-80W/年）
研发主管
Department Leader
（￥30-60W/年）
5路线
Line 5
培训讲师
Training Techer
（￥50-70W/年）
6路线
Line 6

Java互联网程序员黄金5年岗位能力模型
10年以上
8-10年
5-8年
4-5年
3-4年
1-2年
1年
1年以下
程序员黄金5年
(23-28岁)
大厂年薪15W
平均10W
掌握编程语法
具备抽象思维
大厂年薪30W
平均20W
基础开发能力
独立执行能力
高熟练度
大厂年薪40W
平均25W
需求转化能力
理论实践能力
总结复盘能力
大厂年薪60W
平均30W
技术选型能力
判断分析能力
独立思考能力
大厂年薪100W
平均60W
业务架构能力
技术前瞻能力
风险控制能力
大厂年薪80W
平均70W
项目管理能力
团队协同能力
全局思维能力
大厂年薪100W以上
平均80W
自我认知能力
技术领导力思
维格局
入门开发
全栈开发
资深开发
架构师
资深架构师
开发经理
技术总监

六、完整学习路线图

第1周

01、七大软件设计原则
02、工厂模式与单例模式
03、装饰器模式与代理模式
04、委派模式与策略模式
05、建造者模式与原型模式
06、各设计模式对比与复盘

第2周

01、Spring5注解编程基础组件介绍
02、用30个类手写Spring V2.0版本之IOC与DI
03、用30个类手写Spring V2.0版本之AOP实现
04、用30个类手写Spring V2.0版本之MVC实现
05、Spring Bean生命周期与循环依赖
06、Spring事务传播原理及数据库事务操作原理

第3周

01、Mybatis应用分析与最佳实践
02、Mybatis体系结构与工作原理
03、Mybatis插件原理及与Spring的集成
04、Mybatis总结复习

第4周

01、微服务的现状与未来
02、微服务核心之重新认识SpringBoot
03、SpringBoot核心特性及设计思想
04、手写实现一个starter组件、自动装配
05、SpringBoot总结与复习

第三阶段
分布式微服务与高并发

第5周

01、了解多线程的意义和使用
02、并发编程带来的挑战
03、探索线程安全背后的本质以及volatile
04、线程基础阶段性总结和扩展

第6周

01、理解J.U.C中的ReentrantLock
02、常见并发工具的使用及原理解析
03、深入分析阻塞队列及原子操作等并发工具
04、深入分析ConcurrentHashMap的源码与设计

第7周

01、线程池的设计与原理解析
02、线程池与并发总结与复习
03、JVM应用与基本参数介绍
04、JVM进行篇监测工具

第8周

01、JVM性能调优原则
02、JVM高级特性分析
03、JVM性能调优实战
04、JVM总结与复习

第9周

01、MySQL架构与SQL执行流程
02、MySQL索引深入剖析
03、MySQL事务与锁详解
04、MySQL总结与复习

第10周

01、Redis基础与设计思想
02、Redis底层原理剖析
03、Redis分布式集群部署原理
04、Redis分布式实战

第11周

01、RabbitMQ工作模型
02、RabbitMQ可靠性投递
03、Kafka架构
04、Kafka架构原理

第12周

01、消息队列对比&消息队列可靠性&有序性场景
02、Nacos的基本应用
03、Nacos Server监控配置 & 服务注册源码分析
04、Nacos Raft协议的原理源码分析

第13周

01、Nacos Server注册中心
02、Nacos配置中心的使用&源码分析
03、Dubbo Spring Cloud基本应用
04、Dubbbo的应用以及高级特性
05、Dubbo高级功能分析以及整体架构

第14周

01、Dubbo服务注册源码分析&Dubbo SPI
02、Dubbok扩展点源码分析&服务发布与注册源码分析
03、Dubbo的服务消费流程分析 & 消费者源码分析
04、Dubbo的服务消费流程源码分析
05、Sentinel限流熔断基本应用
06、Sentinel动态限流规则以及集群限流

第15周

01、Sentinel服务熔断应用&源码分析
02、Sentinel服务熔断应用&源码分析
03、分布式事务Seata的原理和使用
04、分布式事务Seata的原理和使用

第四阶段
毕业项目实战指导

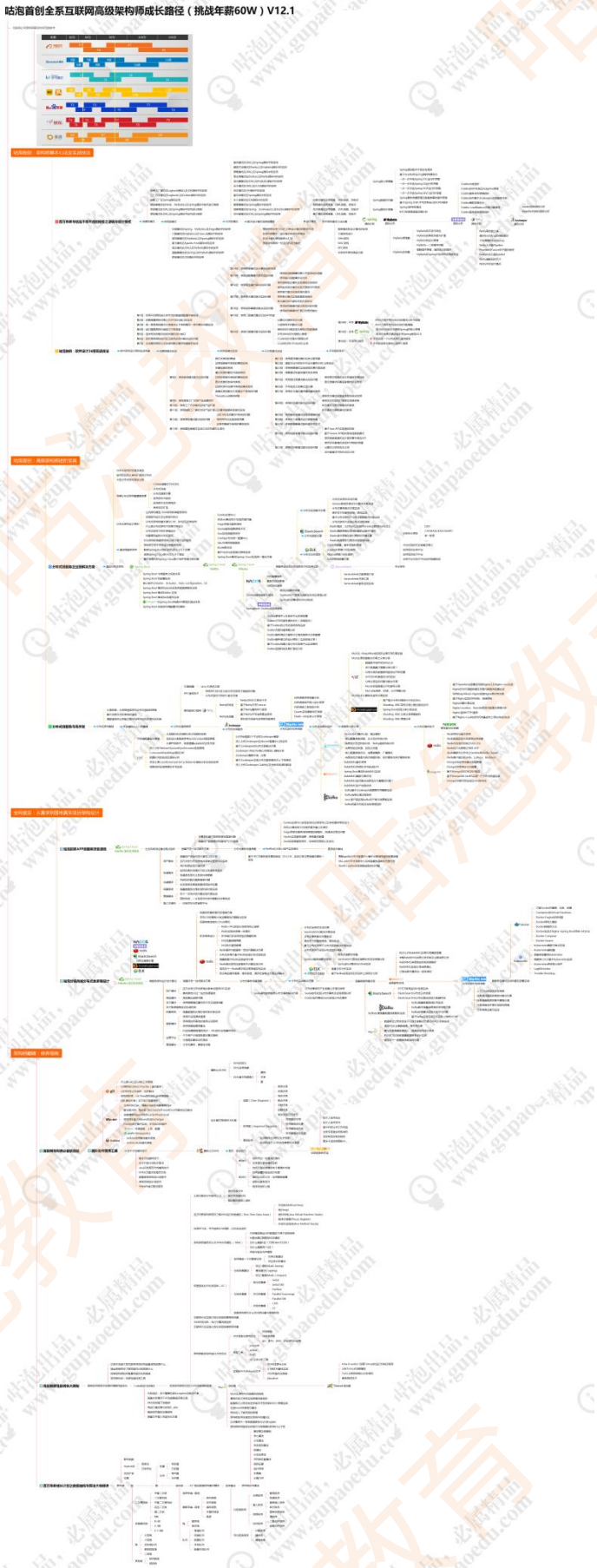
第16周

01、秒杀整体架构图讲解
02、秒杀场景介绍与环境搭建以及秒杀表结构设计
03、商品秒杀热门数据实时同步
04、商品秒杀程序隔离解决方案

第17周

01、网关鉴权
02、服务安全控制
03、百万并发站点构建

附：学习路线对应的详细课程规划



灰灰老师精讲面试题合集

1. 请你说下对InnoDB索引数据结构的理解?

1.面试官：今天问你一个简单的问题 请你说下对InnoDB索引数据结构的理解?

灰灰老师：好的 面试官

首先，在说Mysql InnoDB索引之前，可以先讲一下InnoDB的数据存储。

2.面试官：我在问你InnoDB索引数据结构的理解，为什么你要先讲一下InnoDB的数据存储

灰灰老师：

面试官是这样的，因为Mysql的索引要讲清楚跟数据存储有一定的关系。

在InnoDB中的数据都是以行的形式存储在page页，并且page页跟page页以双向链表进行关联。

我们在去查询数据的时候，如果基于我们的链表去查询，由于链表的查询性能是O(n)，数据越大，查询的性能会越来越慢。

所以，Mysql提出了一个索引的概念，不同的储存引擎的索引方式可能不一样。

3.面试官：恩，那InnoDB的采用了哪些实现方式呢?

InnoDB的实现方式采取了跟书目录一样的思想，为每个page页创建一个目录记录，这些目录记录也都会保存到page页，由于page页大小有限，所以就会有多个目录page。 当有多个目录page的时候，再为目录page创建一个目录，直到根节点。

最后，形成了一个，叶子节点是存有真实数据的page页的双向链表，而非叶子节点是存储目录的page页的树形结构。这样的数据结构我们就称为B+树。

4.面试官：那你简单讲一下他带了哪些优势

灰灰老师：

第一呢 之前是链表遍历查询，现在是树的查询，树的查询性能比链表要高很多，查询次数由树的高度来决定，树越矮跟磁盘的io次数也就越低

第二呢 查询相对稳定，因为数据全部在叶子节点，不管查询什么数据，都需要查询到叶子节点

5.面试官：那为什么不用二叉树或者红黑树?

灰灰老师：

这里也有两点原因

首先二叉树会有斜树的可能，会退化成链表，那么索引树就没有作用

其次像红黑树，树的高度会很高。同样的，B-树的数据结构虽然跟B+很类似，但是B-没有目录节点的说话，每个节点都存储真实的数据，那样由于每个page要存的数据变大，节点就会越多，树层级就越高，查询来讲整体变慢。

所以，InnoDB索引的目的是通过空间换时间，并且尽可能的去减少磁盘的IO次数。

这也是为什么用B+树，而不用跳表等其他数据结构的原因。

2. 请你说下对Redis锁的理解?

1.面试官：看了你的简历，外包工作三年，精通Redis，那我们现在开始面试，请你简单说下你对Redis锁的理解

灰灰老师：好的面试官

首先，我们得讲下什么是锁，所谓锁，就是某些共享的资源，我不希望有并发执行的可能，而是让这些资源串行执行。

比如下单，同一个订单我只能由一个人同时去支付。

所以，锁的核心作用，我有线程在做的时候，其他线程不能做。

2.面试官：那既然说到锁了，那你怎么去实现一把锁：

灰灰老师：实现锁的必要条件 有个标记 这个标记来决定是否有线程在执行 ，并且这个标记获取必须是安全的，2这个标记必须是线程可见的

比如厕所的门，如果2个人看到门开着，只能有1个人进入，还有这个门对盲人无效，盲人必须通过其他方式来确认是否有人。

Redis实现分布式锁也满足了上述条件：

第一呢.因为Redis是k.v结构，我们可以把key做为一个标记，比如支付订单111，我们可以把pay:111作为key

第二呢.并且这个key是线程可见的， 因为Redis里面的指令执行是单线程的。

还有就是要保证获取这个key的安全性就可以，由于是单线程指令执行，我们可以用setnx，如果不存在就能设置成功。

或者采用lua脚本来保证多个指令的原子性。

所以在我看来，Redis能实现分布式锁最关键的点在于Redis的命令执行是单线程。

3.面试官：回答还不错，那再问一个问题 Redisson做了什么事情呢?

灰灰老师：Redisson呢 其实是Redis的一个客户端，它对Redis能做的一些事情都进行了封装，用户使用的时候只要去用已经封装的功能就行。

比如分布式锁，Redisson就进行了很好的封装，主要实现了5件事情.

第一.基于Redis的发布与订阅 以及Semaphore实现了锁的等待，就是没拿到锁我可以等待一段时间。

第二.实现了可重入锁，基于Redis的Hash数据结构

第三.为了死锁，给锁添加了过期时间

第四.为了防止业务没有执行完，锁被释放，利用时间轮添加了续期机制

第五.因为Redis是属于ap模型，在发生分区容错的时候，优先保证高可用，所以会牺牲一定的数据一致性。为了防止锁丢失，也提供了联锁的概念。

---

3. 我看你在项目中有用到线程池，那你能说下线程池的执行原理么?

2.面试官：哼 你也别不服气，随便考你一道题，说下对线程池的理解?

灰灰：表情思考一下

首先多线程，是我们在开发中提升并发能力的一个常用的方案，但是我们一般不会自己去手动创建线程，因为这样线程不易于管理，每个人都去创建线程，可能会出现无限的线程创建，导致响应速度反而越来越慢。

所以，我们一般会用线程池来创建线程，线程池的核心思想其实是线程复用，我希望用固定的线程数，来执行不定的任务。

我们juc包下的Executors其实封装了很多线程池的实现方式，比如newFixedThreadPool newCachedThreadPool等等

但是我们在工作中不建议直接用封装好的方式，因为如果用封装好的，很多人不会去看底层的实现原理，导致乱用。线程

如果没有用好，影响的不是某断代码或者某个服务，而是整台机器，所以要使用，建议把线程池的原理弄清楚。

这是我对线程池的理解。

3.面试官，那你说下线程池要传哪些参数?

在讲到线程池的原理的时候，我们必须要知道线程池的7个参数

1.核心线程数 默认会开启的线程数量

2.最大线程数 任务过多，我会额外开启的最大线程

3.线程等待时间 没有任务的时候，这个线程过多久会回收（如果能回收的话）

4.等待单位

5.阻塞队列 放任务的队列

6.线程的创建工厂

7.拒绝策略 在某些情况下，我会丢失任务 如果有拒绝策略 ，会进入拒绝策略

这些参数都会在流程中能使用到,每个参数都有每个参数的作用。

4.面试官，你刚才说线程池的流程会用到这些参数，那分别在哪里用到?

参数的使用，要先捋下线程池的执行流程。

线程池去执行任务最终是通过exectue方法去执行，传的任务是实现了Runnable接口的对象并实现run方法。实现流程如下

1.先会判断已经开启的线程数是否小于核心线程，如果是，会创建一个worker对象，这个worker是实现了runnable接口的，并且worker对象持有一个Thread对象。通过启动这个Thread去开启线程。这是核心线程与线程工厂的使用场景，核心线程来判断我是否需要创建新的thread，线程工厂决定我怎么去创建线程。

2.如果线程池已创建的线程数大于等于核心线程数，会把任务给到阻塞队列，已经开启的线程会自旋去阻塞队列获取任务。

阻塞队列就是用来存储我来不及执行的任务的。

3.如果我已经开启了核心线程数的线程，并且阻塞队列放不下了，那么我们会去创建额外的线程来帮助执行，但是总线程数不能超过最大线程。

最大线程是用来限制我线程池能开启的最大线程

4.如果阻塞队列满 了，并且也开启了最大线程的线程数，将不在接收任务，会进入拒绝策略。拒绝策略可以自己实现，实现RejectedExecutionHandler接口即可。

现在我们知道了5个参数的作用，还有2个参数在细节里面，就是过期时间与过期单位。

这2个参数 是当阻塞队列没有任务的时候，我要去回收多余的线程或者回收所有的线程的时候，一个超时等待时间，也就是阻塞队列的poll的时间。

5.面试官：我看你提到了核心线程数 那这个核心线程跟最大线程有什么区别?

其实7大参数里面的核心线程数与最大核心线程数，其实就是一个线程的上限判断。核心线程，是来一个任务我就会开启一个线程，直到达到核心线程数为止。

最大线程是我的阻塞队列满了的时候，我需要开启线程来帮助处理任务。但是他们的开启回收逻辑是一模一样的。

默认会保证核心线程数的线程在阻塞等待，不被回收，那么至于哪些回收，哪些等待，就看线程哪个执行到判断现有线程数是否大于核心线程数的逻辑。

所以核心线程数与最大线程数只是用来做线程判断，对于线程本身来讲，没有任何区别。

4. 我们都知道Redis属于一个内存型数据库，那么内存如果满了，怎么处理?

Redis是一个内存型数据库，所有的数据是基于内存操作的，内存都是有大小限制的，在启动实例的时候还能指定内存大小。

首先，我们在设计的时候，尽量去节省内存或者防止内存不足，比如设置key的过期时间，到了时间会自动从内存移除，在存储数据的时候，会进行压缩，会搭建cluster集群来做负载。

如果真的发生了内存满了，Redis会提供不同的策略来处理

1.默认是novication,就是内存满了的时候 会报OOM错误，这个时候，数据能读但是不能写。

2.如果发现内存满了后，为了保证这次的数据能顺利写入，会去想办法淘汰一些之前的数据，那么淘汰哪些数据？Redis会提供一些算法，比如LRU、LFU、随机、根据过期时间来。

还会提供淘汰数据的范围选择：是去所有的key里面淘汰还是去设置了过期时间的key里淘汰。

面试官： LFU根据最少次数来淘汰，会有一个时效性问题，就是有些数据可能不是热点数据，但是由于之前的点击次数还在，那就导致旧数据淘汰不了，新的数据一直被淘汰，怎么解决？

Redis对象内部会用一个24bit的字段来记录对象的访问时间跟次数，前面16bit代表的是最后访问时间，后8bit代表的是次数，会根据前面的时间计算出这个对象多久没访问，如果很久没访问，会根据配置来减少对应的次数。

这样，越久时间不访问，次数减得越多。就能解决时效性问题。

5. reentantLock的源码看过嘛，它是怎么实现的？

reentantLock是基于aqs实现的一个可重入锁。它的实现主要有2种不同方式，一种是公平锁、一种是非公平锁。

他通过aqs中的state字段来维护是否获得锁或者重入了几次，并且这个字段需要用volatile来修饰，保证线程之间的可见性。

并且去抢占这个标记必须是安全的，所以会通过CAS来保证抢占过程的安全性。

如果是不同的线程，通过cas判断是否能拿到锁，如果是同一个线程重复抢占锁，那么state+1，这样就实现了可重入。

如果拿到锁的线程，去执行相关业务代码，如果拿不到锁，会加入一个等待队列，是一个双向链表，这个队列也属于aqs维护，加入队列后park，这个时候线程就等待拿到锁的线程释放锁去唤醒。

当拿锁的线程unlock后，会去唤醒等待队列的第一个等待的线程。自旋去获取抢占锁。

面试官：你刚才讲了2个实现，公平锁与非公平锁，他们的区别在哪？

他们的大致流程都是一样的，他们的区别主要在于去获取锁的时候，公平锁会去判断是不是等待队列的第一个线程或者没有线程在等待，如果是，就能获取锁；如果有线程在等待，必须按照队列的顺序来获取锁。新的线程必须排在等待队列后面。这也是公平的由来，任何线程都必须按照等待队列的顺序来获取锁

而非公平，只要锁释放，没有加入等待队列的线程能提前通过cas去抢占锁，可能拿到锁的时间节点比等待队列的线程更早。

6. Redis主从数据一致性怎么保证

求职者：

年底了，贵司的这个岗位也是急着要人，给我一次面试机会，面完再决定要不要发offer

求职者：

Redis主从数据一致性，主要是讲Redis主从数据是怎么同步的，主要有3种，分别是全量同步、增量同步、指令同步。每种同步方式的时机不一样。

全量同步

1.全量同步，一般发生在第一次建立主从关系、或者跟主断开时间比较久的场景。

它的步骤为：

1.slave向master发起同步指令，master收到以后，会通过bgsave指令生成一个快照RDB文件，同步给slave，slave拿到后，会通过master同步的快照文件进行加载。这个时候，主生成rdb文件时候的所有数据都以及同步给了slave。

2.但是bgsave指令是不会阻塞其他指令执行的，所以master在生成快照文件时，还是能接收新的指令执行。这些指令master会先保存到一个叫replication_buffer的内存区间，等slave加载完快照文件后会同步。

面试官：

那为什么还需要增量同步呢?

求职者：

数据同步，slave是定时会发起的，假如每次同步，都把主的所有数据都进行同步，那么性能会很慢，大部分时候，slave可能只跟master相差一部分数据。那么只需要同步这部分数据。

slave发起同步的时候，还会带有上次同步的偏移量，然后跟master的最新的偏移量比较，如果相差的数据在master的积压缓存（一个专门存储master最新数据并且会覆盖的内存区间）能查询到的话，那么只需要把相差的数据同步给slave。这就叫做增量同步。

指令同步：master输入的指令会异步同步给slave。

7. ThreadLocaL如何防止内存泄漏?

求职者:

是的，今天来公司面试Java中级开发的岗位，还请面试官多多指导

求职者:

首先，ThreadLocal里面的数据都是以entry的数组保存在我们的thread里的，其中entry的key为threadLocal对象，value是你设置的值。

entry的key设置为弱引用

因为thread这个对象的生命周期其实是跟thread绑定的。那么如果key为强引用的话。ThreadLocaL的对象一直会被线程的entry持有，然后ThreadLocaL对象回收不了，但是外面其实是可以把threadLocal对象强引用去掉的。那么这个时候，这个对象以及整个entry对象都是无效的，本来应该是需要回收的。但是因为强引用回收不了

所以thread的key是弱引用，这样我无效的时候我能GC回收对象。一定程度上面防止了内存泄漏问题。

但是不够

因为虽然key的对象能被回收了，。但是entry还是不能回收，但是这些entry的key为null了，也没用了，需要回收。

所以，。threadLocal通过各种机制，在使用的时候，。会去回收key=null的entry对象

比如当我去set的时候，会从当前冲突的位置向前或者向后找到第一个null的为止，并且把找到的所有的被GC回收掉的entry清除！！并且rehash，供其他对象使用，达到一个时间与空间的平衡。

还是不够，因为如果没有hash冲突，自身是不能走清除的。所以我们一般使用的使用，用完要remove。

8. HashMap是怎么解决哈希冲突的?

求职者

首先，我们得知道下HashMap的存储方式，HashMap是k-v的数据结构， 底层是一个entry数组的数据结构， 并且根据key计算出一个hash值，取模entry数组长度-1，得到一个下标位置来存储数据。

所谓hash冲突 ，就是不同数据hash算法计算出来的结果可能是相同的，那么不同的数据就会放在同一个下标位置。

通常解决hash冲突的方法有4种。

1.开放寻址法，包含线性探测、平方探测等等，就是从发生冲突的那个位置开始，按照一定的次序从hash表中找到一个空闲的位置，然后把发生冲突的元素存入到这个空闲位置中。ThreadLocal就用到了线性探测法来解决hash冲突的。

2.链式寻址法，这是一种非常常见的方法，简单理解就是把存在hash冲突的key，以单向链表的方式来存储。

3.再hash法，就是当通过某个hash函数计算的key存在冲突时，再用另外一个hash函数对这个key做hash，一直运算直到不再产生冲突。这种方式会增加计算时间，性能影响较大。

4.建立公共溢出区， 就是把hash表分为基本表和溢出表两个部分，凡事存在冲突的元素，一律放入到溢出表中。

HashMap就是通过链式寻址法来解决的，但是链式地址有个问题：当链表过长的时候，会影响查询性能，所以在HashMap里，当链表长度大于8的时候，会转为红黑树，提升查询性能。但是树在添加数据的时候也会有树的分裂合并，所以在树节点小于6的时候，又会转为链表

这个 是我对HashMap哈希冲突的理解。

面试官

HashMap在并发的时候会有什么问题?

求职者

因为HashMap添加数据的时候，不是线程安全的，所以当并发添加数据的时候，有可能导致数据丢失，比如：当2个数据同时链地址到一个节点的时候，只会有1个是成功的。

还有，因为entry节点会有数据结构的变化，比如当链表长度达到8并且数组长度达到64后，会转为链表。那么在并发的时候，可能会产生数据结构类型转换错误。因为有可能在操作的时候，其他线程把数据结构变更。

所以，如果在有并发要求的场景，可以用currentHashMap或者hashTable。它们是线程安全的。

9. 你说下跳表跟B+树的区别?

求职者

首先，跳表，其实就是一个链表结构。添加数据时都会随机一个层级，然后把每个数据层级相同的节点以链表的方式进行链接。

查询的时候，从最外层的链表开始查询，思想跟二分法的思想很类似。

B+树，叶子节点存有真实数据，非叶子节点是这些真实节点的目录页，查询的时候，从根目录开始查询，树的高度越高，查询的次数也就越多。

跳表跟B+树都是用空间来换取时间，用额外的空间来保存链表或者目录页，来提升查询性能，但是也有一定的区别。

1.层高，B+树三层就能支持千万级别的数据，但是跳表，存储相同的数据量需要更高的层级，这也是为什么我们innoDB索引用B+树而不用跳表的原因，InnoDb需要跟磁盘IO,层级越高，IO次数也就越多，但是Redis的zset是用的跳表，因为Redis是基于内存操作，也就没有磁盘IO的概念，所以跳表反而更简单。

2.操作数据，跳表要比B+树快，因为B+树，在数据操作的时候，要去维护B+树，所以会有树的分裂与合并，但是跳表是随机一个层次，实现相对简单。

10. Redis sentinel集群和Cluster集群有什么区别?

Redis sentinel集群和Cluster集群有什么区别?

求职者:

对于这个问题，我认为可以从3个方面来回答

首先，集群，我们首先想到的是主从集群，但是Redis的主从，只能做数据备份，但是当主发生宕机的情况下，从是不能够自动升级为主的。

所以我们需要一些其他的自动灾备集群方案。最常见的是2种，一种是sentinel哨兵集群，一种的cluster集群。

至于它们的区别主要有以下几个点:

1. Redis 哨兵集群是单独启动哨兵服务对主从节点进行监控，所以原则上还是主从，只不过哨兵实现了主从自动故障转移，它可以实现读写分离，分担Redis读操作的压力。

而Redis Cluster 集群的Slave节点只是实现冷备机制，它只有在Master宕机之后才会工作。

2. 从集群架构上来说，Redis 哨兵集群是一主多从， 而Redis Cluster是多主多从

3. 哨兵集群是一主多从，所有的数据都会存储在我的主，而cluster集群数据会通过虚拟槽的分片机制分配到不同的主服务器，所以扛压这块，哨兵集群受限于单个服务器的资源配置，而cluster是多主联合扛压。

4. RedisCluster集群，采用虚拟槽的分片机制，能够灵活的在线进行扩容、缩容，以及灵活的进行数据隔离。

11. Redis分布式锁跟zk分布式锁的区别在哪?

看你简历上，写的精通Redis

那你详细描述下Redis分布式锁跟zk分布式锁的区别?

求职者

我们先来分析下他们2个的实现逻辑

1.Redis是基于它的命令执行是单线程的这个特性，所以在获取数据的时候不会有并发问题来去做分布式锁的，其中Redisson客户端，对分布式锁进行了很好的包装，redisson分布式锁 基于lua+hash来实现加锁逻辑，同时利用Redis的发布与订阅以及seamphore来实现锁的等待。并且添加了锁续期等功能。

2.zk的分布式锁，主要是基于zk的临时有序节点+watch机制来实现的，每个线程去尝试加锁都会创建一个临时有序节点，然后拿锁的时候，判断是否是最小的那几个节点，如果不是，就线程安全的去监听上一个节点。然后进行wait。

区别主要体现在以下几个方面：

1.因为Zk是基于临时有序节点去获取锁，所以每个线程去抢占锁，都会去创建一个临时节点，当抢占锁的线程比较多的时候，对ZK集群的压力会比较大。而Redis则是通过判断key是否存在来获取锁。

2.Redis属于cap里面的ap，优先去保证高可用，所以在某些场景下，比如主从切换，可能导致数据丢失，导致锁丢失，而zk则是使用zap协议优先保证数据一致性，所以安全性方面，Redis要比ZK低，但是也有像联锁的方式去提升可靠性。

3.性能方面，Redis基于内存操作，所以性能比zk要高。

12. 什么是缓存雪崩、缓存穿透、缓存击穿？你怎么解决？

请具体分析一下什么是缓存雪崩、缓存穿透、缓存击穿？

如果在开发过程中遇到了这些问题，你会怎么解决？

求职者

其实，缓存雪崩、穿透以及击穿，最终导致的问题都是我的请求绕过了我们的缓存中间件，直接打到了DB的集群，导致DB的压力过大或者造成DB崩溃的现象。只不过场景稍微有些不一样。

*缓存雪崩

缓存雪崩主要有大量的key同时打到DB，那么产生这种场景有2种主要原因。

1.缓存中间件服务不可用，缓存失效，导致所有的请求全部打到DB，那么这种解决方案就是尽可能保证缓存中间件的高可用，比如Redis缓存，那么就采用Redis的cluster集群，以及提前做好缓存组件报警机制，杜绝宕机。

2.缓存中大部分key同时过期，比如Redis中key同时过期，对于这样的情况，可以在失效时间上增加一个1到5分钟的随机值。或者对热数据，进行访问的时候，自动续期。

*缓存穿透

缓存穿透是指缓存和数据库中都没有的数据，但是用户一直请求不存在的数据！这时的用户很可能就是攻击者，恶意搞你们公司的，攻击会导致数据库压力过大。

1.我们可以在缓存之外再加一层性能更好的过滤方案，比如布隆过滤器或者布谷鸟过滤器。如果数据不存在，直接在过滤器进行过滤掉，不会打到Redis跟DB。

2.如果发现是恶意攻击，可以对IP进行处理，加入IP黑名单。

*缓存击穿

单个key过期的时候有大量并发，我们可以采用dcl锁等方式，但是我觉得如果要采用锁来保证，那么就没有必要用到缓存。

所以，在我看来，您提出来的这个问题，有点过于放大了它带来的影响。

首先，在一个成熟的系统里面，对于比较重要的热点数据，必然会有一个专门缓存系统来维护，同时它的过期时间的维护必然和其他业务的key会有一定的差别。而且非常重要的场景，我们还会设计多级缓存系统。

其次，即便是触发了缓存雪崩，数据库本身的容灾能力也并没有那么脆弱，数据库的主从、双主、读写分离这些策略都能够很好的缓解并发流量。

最后，数据库本身也有最大连接数的限制，超过限制的请求会被拒绝，再结合熔断机制，也能够很好的保护数据库系统，最多就是造成部分用户体验不好。

13. 缓存跟DB的一致性怎么产生的?

我看你之前的项目开发中用过Redis做缓存，你有遇到过缓存跟db的一致性的问题吗

求职者

Redis缓存这一块，我的实战经验还是非常丰富的

首先，我们缓存是为了减少跟DB的压力。

当我们去访问数据时，我们先去缓存读取，如果缓存有，直接不需要跟DB进行交互，如果缓存没有，那么从DB获取，然后存到缓存中。

当数据发生变更的时候，我们也会做好缓存依赖，会同步到缓存，以Redis为例，我们会删除Redis的数据。

但是，就算做好了缓存依赖，但是因为操作DB跟删除Redis不是一个原子操作，所以，在同步到到Redis之前，可能有其他的线程改动了DB的数据。简单举个例子。

1.线程A请求缓存，没有缓存，从DB拿到1，要去更新到缓存，但是还没更新到缓存

2.线程B将1更新为2，并且删除缓存，DB的值为2

3.线程A更新缓存，redis为1

所以，根本原因其实就是并发带来的原子性问题。

面试官

那你们是如何解决的呢?

求职者

不可取的强一致性方案

延时双删

所谓延时双删，就是在更新DB后，等待一段时间，再进行Redis删除！来等待其他的线程拿到的都是最新数据!

也会产生很多问题。

1.延时多久？不知道其他线程要多久。

2.不够优雅，代码中写入延时代码。

采用锁机制，不让有并发

在更新的时候，采取锁的机制，不让其他线程进行删除操作!

但是会拖慢整个性能，违背了Redis的初衷

所以，我们只能采用最终一致性，不应该去保证强一致性

最终一致性方案

每个缓存设置过期时间

设置过期时间，就算数据不一致，也只是在有效时间内的不一致。

Mysql canal等数据同步工具

捕捉到DB的更改，同步到相关Redis，相对比较复杂，要知道每个数据对应的缓存。

RocketMQ

比如基于RocketMQ的可靠性消息通信，来实现最终一致性。

14. 请说下对动态代理的理解

长沙跟深圳的薪资那是天差地别

建议你这边还是要调整一下心理预期

我们先面试吧

请具体说说你对动态代理的理解

首先，所谓动态代理，它的核心思想在于拦截，去拦截被代理类的方法，当方法被代理类的方法的时候，走到自定义的拦截器

动态代理的实现方式有2种：

1.JDK动态代理

jdk代理里面有个很重要的类，Proxy，会通过这个Proxy类的newProxyInstance方法去反射生成一个代理类，生成的代理类必须实现被代理类实现的接口。

newProxyInstance的3个参数，第一个：classLoader类加载器 去加载我生成的代理类　2.要代理的接口　3.调用接口方法的时候要走到哪个拦截器，拦截器必须实现InvocationHandler接口的invoke方法，后面拦截流程全部走到invoke方法。

因为JDK动态代理是基于实现接口去做的，所以代理只能代理接口 或者实现了接口的类。

2.CGLIB代理

CGLIB代理的关键类是Enhancer，通过Enhancer去创建代理类以及设置被代理类，CGLIB是通过继承被代理类的方式去实现拦截，所以会设置Superclass以及Callback，Callback就是我们的拦截类。

CGLIB是通过ASM框架去生成代理类的，所以生成比较复杂。

使用场景：我们的Spring里面的aop支持jdk以及cglib代理，我们Mybatis的主流程以及插件就是用到了jdk动态代理。

15. 项目中有用过Redis么，用在哪里?

面试官

先问一点专业问题吧

项目中有用过Redis么，用在哪里?

灰灰

我们项目中有用到，主要用了3个场景：

1.缓存相关的场景，我们是做在线教育的，内容模板会有很多课程相关，这些数据在DB单表有5.600W；如果走mysql查询会很比较慢，用户体验感比较差，并发也上不去。所以我们做了些接口缓存、课程内容的对象缓存。提升了性能的同时，还解决了本地缓存不一致问题。

2.同时，我们由于它是分布式的，并且可以设置过期时间，也会用来保存用户token。因为token也是有过期时间的，用Redis来保存刚好满足。

3.我们还用它去基于日期+incr自增指令实现了一个分布式ID。因为我们的课程ID比较大，需要分库分表，数据库自增满足不了我们需求。

面试官

你们缓存的单个key是多大的？会去控制key的大小么

灰灰

我们的key因为有压缩，所以我们的key一般会控制在10K之内，比较大的key我们是不会放入Redis的。

因为如果Key过大，会由于带宽、网络等原因导致指令返回速度过慢，Redis又是单线程的，会阻塞其他指令执行。

面试官

有遇到过缓存跟DB的数据不一致么，怎么产生的？怎么解决？

灰灰

遇到过的，主要是因为操作DB跟redis这2个操作，不是原子性操作，所以在并发的时候，可能会导致数据不一致。其实就是并发导致的原子性问题。在拿到数据准备放入Redis的時候，另外的线程对DB数据进行了改动。

解决方案：

最优最好的方案：给Redis的缓存数据加上过期时间。达到最终一致性，就算不一致，在过期时间到了后，就能一致。

当然网上也有说通过锁、延迟双删；但是我觉得都不太适合我们的应用场景的。因为我们Redis本来就是为了提升性能。如果再加上一些消耗性能的方式来保证强一致性，那么你干脆不要用Redis。

面试官

设置过期时间会从Redis删除，你知道怎么删的吗？

灰灰

删除的话，Redis主要有2种方式：

1.惰性删除　当我去获取这个key的时候，判断有没有过期，过期了就删除

2.定期过期　Redis会有个事件驱动机制，每隔一段时间会去扫描我的key里面是否有过期的数据，如果有，我就去删除。

隔多久可以根据config文件配置，扫描也不会扫描所有的，而是分批去扫描。

面试官

还知道Redis的其他的一些数据类型的使用场景么？

灰灰

我们工作中，刚才我讲的缓存、自增ID、保存token其实都是基于String做的。

但是由于Redis性能比DB高，所以根据不同的缓存场景，我可以用不同的数据类型来支撑，比如hash我就可以用来做对象型的数据缓存，比如我们之前课程会有浏览数、购买数 这些数量可以用hash来存储。

还有一些集合类的缓存，可以用list、set存储。 如果我的数据有分数排行的，可以用zset做排行缓存。

Redis2.8.9后引入了基于HyperLogLog算法的HyperLogLog数据类型。可以做一些统计类的数据，比如UV，这个的好处就是内存会占用很小很小。但是会有一定的误差，也只能做统计。

Redis3.2后，也加入了GEO的数据类型。可以做一些查询附近地址等功能。可以存储经纬度，然后可以拿到彼此的相差距离，比如搜索饭店，哪个离我最近，有多远。

Redis5.0后又引入了Stream，可以做类似mq的发布订阅功能，并且支持ack回执。

16. Redis怎么做缓存设计的?

你刚提到Redis做缓存，那么性能肯定比Mysql要高，

那Redis是怎么设计的?

灰灰

1.Redis是基于内存操作的，节省了跟磁盘的实时IO，但是后台还是会异步进行磁盘备份。所以数据一致性不强，会有数据丢失。

2.由于是基于内存操作，所以单次执行非常快，瓶颈不在cpu。设计的是单线程指令执行。减少了执行指令时候的线程切换

3.Redis本身就是一个k-v保存结构。查询数据的时间接近O(1),只有当数据发生冲突的时候才会遍历链表，跟HashMap很类似。

4.Redis底层的数据结构支持，思想就是空间换时间，比如SDS、跳表

5.网络模型采用的是多路复用。大大减少了网络IO的时间消耗

面试官

跳表用在哪里？跳表的思路是什么

灰灰

跳表主要用在zset的有序数据集合类型。

跳表的思路跟Mysql的b+树差不多，主要是我的数据会保存一个随机的层级，每个层级会跟相同层级的数据建立一个双向链表。

找数据的时候，会从最外层开始，然后数据是有序的，查询过程跟二分法一样。

核心思想就是用空间来换取时间，提升查询性能。

面试官

你说k-v跟HashMap很类似，解决冲突用的是链表，那为什么Redis用头插，而HashMap用的是尾插

灰灰

HashMap用尾插，是因为在并发的场景下，可能会导致链表的死链。

但是Redis是单线程执行，所以不会有并发导致死链，头插法比尾插速度要快很多。所以用头插法即可。

面试官

那Redis有没有像HashMap一样的扩容机制

灰灰

Redis肯定也有扩容机制，因为如果没有扩容的话，会导致链表越来越长，从而降低查询性能。

只不过Redis的扩容机制跟HashMap有点不一样，Redis会有2个hashTable，第二个table只有再扩容的时候使用，当第一个table的容量达到一定量，这个量正常是已有的数据是table大小的时候就会扩容，但是当有在进行持久化的时候，使用量是table容量的5倍的时候扩容

扩容也不会一下子都扩容完成，因为一下子把所有的数据从第一个table移到到第二个table耗时太长。

所以会采用渐进式rehash，分批次的将数据迁移到第二个table。然后把第一个table变量指向新table。第二个table赋空。

17. Redis内存满了，如何处理

我们接着聊技术

你有遇到过内存满的情况么，这种一般怎么处理?

灰灰

有遇到过的，内存满了的话，会报OOM错误，数据能读不能写了，因为没有足够的内存去接收新的数据。

至于解决方案：

1.首先想到的肯定是去加内存，包括加大单机的内存，或者横向扩容

当然，如果你允许一些老的、或者冷门的数据删除的话，你也可以在config文件设置淘汰策略。

默认官方提供8种淘汰策略：有3种算法(LRU、LFU、random)，然后每种算法的淘汰范围都可以设置淘汰范围 allkeys还是volatile 是淘汰所有的数据还是只淘汰设置了过期时间的。再加上novication不淘汰、根据volatile-ttl过期时间淘汰

1. volatile-lru：从已设置过期时间的数据中挑选最近最少使用的数据淘汰
2. volatile-lfu：从已设置过期时间的数据中挑选使用次数最少的数据淘汰
3. volatile-ttl：从已设置过期时间的数据中挑选将要过期的数据淘汰
4. volatile-random：从已设置过期时间的数据中任意选择数据淘汰
5. allkeys-lru：从所有数据中挑选最近最少使用的数据淘汰
6. allkeys-lfu：从所有数据中挑选使用次数最少的数据淘汰
7. allkeys-random：从所有数据中任意选择数据淘汰
8. noeviction（驱逐）：禁止驱逐数据，新写入操作会报错

面试官

Redis中有个淘汰池的概念，它的具体作用是什么?

灰灰

首先，淘汰是根据淘汰算法去找到需要淘汰的数据，比如lru就是找到最久未使用的，LFU是找到最少使用的。

但是如果去遍历所有的key，那么就会导致要精确的查出来要淘汰的数据会很慢，从而阻塞数据的添加。

所以Redis中采用的是取样淘汰，取样数量可以设置，从取样的数据中找到最适合淘汰的。这样就大大的影响了淘汰的精确度。

所以，加了一个淘汰池的概念，默认是16，如果淘汰池没满，那么取样出来适合淘汰的数据，假如淘汰池。

如果满了，跟里面的数据做比较，如果比淘汰池里面的数据更适合淘汰。就顶掉之前的。

然后淘汰的时候，就从淘汰池中末尾淘汰。

所以，在我看来，淘汰池的作用就是去提升我们淘汰的精确度。

面试官

基于内存操作，怎么去保证我宕机的时候，数据不会丢失?

灰灰

虽然是基于内存操作，但是持久化还是会做的，Redis重启的时候，都会从我们的磁盘加载数据，然后再基于内存操作。

数据也会持久化到磁盘，只不过实时性不是特别高，同步方式主要有2种，RDB与AOF。

RDB默认的持久化方式，触发时机可以通过save指令来配置。同时shutdown、flushall、save/bgsave指令也会触发

AOF默认不开启，但是开启了就会默认用aof来加载数据。

aof是指令追加的形式来刷新到磁盘，但是如果每次都要保证刷到磁盘的话，就每次都需要跟磁盘IO，所以Redis会提供一些选择方案:

比如默认每s刷到磁盘，或者每次刷到磁盘，或者交给操作系统，方案不一样，数据一致性跟性能就会有不同的影响。

面试官

AOF指令追加，那么文件越来越大怎么办?

灰灰

aof文件会有重写的过程，重写就是当我的文件过大的时候，我会触发重写，这个过大到底多少才大，是可以配置的。

重写其实就是压缩的过程，会以RDB的文件格式来压缩现有的aof。所以也是AOF+RDB的混合模式。

18. 说下对Redis集群的理解

面试官

你们之前公司用的Redis集群是什么?

灰灰

我们用的Redis cluster集群

面试官

你们用cluster集群是基于什么考虑的?

灰灰

cluster是一个去中心化多主多从的集群架构。

cluster集群除了能实现主从自动切换以外，cluster还可以做到数据分片，我可以把我的数据灵活的分散到不同的主，然后主对应的从会读取主的数据。

面试官

那数据怎么知道我要往哪个主写

灰灰

cluster里面会有个虚拟槽的概念，其实就是0-16383的一个数值。

然后我们的key会通过一个hash算法，然后取模16384得到一个0到16383的值。

这个虚拟槽又会对应到不同的机器，并且虚拟槽跟机器的对应关系是能手动维护的，这样我就知道这个key会放在哪台机器呢。

并且整个集群维护会非常灵活，一些热点的key我都可以单独部署一台主。

面试官

你说key是根据一个hash算法得到虚拟槽的，那我希望一个订单相关的key都到一台机器，比如订单的统计数据、订单的基本信息等等，是不同的key，会分散到不同的机器，查询就会查询多台机器，怎么解决?

灰灰

作者也想到了这个具体的业务问题，所以，当你希望相关的数据放在同一个key的话，你可以在key上加{}.这样hash的算法只会去根据{}里面的值来进行虚拟槽的计算了。

面试官

cluster里面的slave是怎么保证跟master一致的?

灰灰

它的流程是这样的:

slave会每隔一段时间发起同步指令，这个同步指令会包含我之前同步的数据的偏移量以及我对应的masterId;

2.master收到指令后，会去判断传过来的masterid是否跟自己的ID一致，如果不一致，那么说明之前没有同步过，master会通过bgsave生成一个rdb文件给到slave，slave会清空自己的数据然后基于rdb文件去加载数据。

但是由于bgsave是不会阻塞主线程的，所以master会有新的指令，这些新的指令会暂存到master的一个内存空间，这个内存空间 每个slave独立。等slave rdb文件加载完后，把新的指令给到slave执行。

这个就是我们经常说的全量同步。

但是有一种场景，我的slave可能只是断开了一段时间，只有一部分数据没跟master同步。所以会有个增量过程。

master收到指令后，如果masterId跟自己的ID一致，那么会根据数据的偏移量去一个内存缓存中找相差的数据，这个缓存叫做积压缓存，大小可以设置，跟mysql的redolog很类似，是覆盖写。如果能找到相差的数据，那么就同步给slave。

但是因为覆盖写，所以可能断开太久，这个相差的数据找不到了，也会触发全量同步。

总结下：master如果根据slave的参数，能找到相差的数据，就把相差的数据同步给slave.如果找不到，那么就触发全量同步。

同时，master的指令也会异步同步给slave。

19. 看门狗的实现原理是什么

面试官:

你的项目是真实的吗？有没有包装?

灰灰:

这个都是我日常做过的

绝对没有掺假

面试官:

你们项目中，Redis用的是哪个客户端?

灰灰：

Redis对不同的语言都支持不同的客户端，因为我们是基于Springboot开发,所以我们的客户端主要是Springboot自带的letuce

也用过Redisson,因为它对于我们的业务做了很好的封装，用起来也比较简单。比如锁啊这些。

面试官：

那Redisson对分布式锁做了哪些改进?

灰灰：

首先，redis能做分布式锁，主要是因为是单线程执行的，所以如果能在一个指令里面操作完获取以及设置锁的话，就不会有并发问题。

比如setnx指令，以及新版的set支持相关的参数。当然也可以用lua脚本来保证多个指令的原子性。

但是基于简单的一条指令去做锁的话，

1.是一个不可重入锁

2.另外一个锁过期时间不好把控，可能会出现业务没有执行完锁过期。

所以redisson主要解决这2个问题，当然还有一些比如读写锁、联锁等等。

可重入的话基于lua+hash去实现可重入锁。

然后时间把控，如果你不知道锁多久过期，redisson会基于看门狗来续期。

当然还有些基础的，比如我没拿到锁，会通过redis的发布与订阅+Semaphore去实现锁等待。

面试官：

看门狗具体怎么实现的？能讲讲么

灰灰：

看门狗其实是时间轮+递归的方式去实现

时间轮就是hashWheelTimer，是一个netty包下的类，它主要实现的功能是延时执行任务。

它的实现逻辑是会启动一个线程去轮询数组，然后任务根据延时多久添加到对应数组，如果轮询到了的话，通过多线程去执行相关的任务。

基于hashWheelTimer根据设置的看门狗时间/3 去延时执行任务：这个任务：判断key是否存在，如果存在，就续期，并且递归，,递归就是自己、调用自己，10s后再去判断key是否存在。做到只要key还在就一直续期。如果key不存在，就不再递归。

面试官：

Redisson为什么有联锁，目的是什么?

灰灰：

联锁的目的是因为redis是属于ap模式的中间件，会存在数据丢失，那么锁就会失效。

所以联锁主要做的一件事情就是尽可能的去保证数据不丢失，加锁会加在不同的独立集群机器。当满足一半成功就成功。其实主要思想就是把鸡蛋分散到不同的篮子，降低风险。 只要不是超过一半的失败，就是成功的。

20. Redis线上阻塞要如何排查

面试官：

假如Redis线上突然阻塞了，你会怎么排查的?

灰灰：

指令阻塞其他指令

因为Redis执行指令是单线程的，因为单次执行速度会非常快，但是如果你让单次执行变慢了，那么也会阻塞后续的指令执行。

哪些情况会让指令变慢：

1.指令获取的数据很多，比如大数据量下执行keys、hgetall、smembers等指令。我们可以通过查看Redis的慢查找到问题，不要去执行慢查操作

2.大Key，我单次查询的的数据过大，也会导致单次执行变慢。所以我们需要拆分大key。

CPU扛不住了

因为Redis处理命令只用到一个cpu，所以当cpu超过我们负荷的时候，也会导致阻塞。

1.Redis是否有牺牲cpu去换取内存的一些配置，比如数据类型底层数据结构的配置。有些数据类型是去追求极致空间，所以会牺牲CPU。

2.如果也没有，那就是需要加机器了

持久化阻塞

1.虽然说Redis的持久化是异步做的，但是for子进程的时候就慢，那么也会导致问题。

2.AOF刷盘的时候，由于磁盘压力，导致操作时间够久，主线程为了数据一致性会阻塞指令。

其他原因

1.其他的程序跟Redis部署在一台机器，导致cpu之间的相互影响

2.网络问题　包括网络是否稳定，是否有延迟、带宽是否足够

3.客户端连接数是否达到了Redis的最大连接数

21. 项目访问量激增，如何应对

面试官：

你们公司的接口有做过压测么，你的接口线上qps最大的是多少？

灰灰：

有的，我们之前的话由于是做在线教育的，所以我们的高峰期一般是在晚上，因为晚上才是学习的高峰期，所以晚上的话qps能够达到2000左右，而白天平时的话也就4.500的样子。

然后我们压测的话，单机qps必须达到1500到2000，然后线上会有5台机器来负载

面试官：

刚才你讲你们是会有课程售卖，假如双11了，请求量可能会暴增，你们怎么处理**

灰灰：

我们的双11其实是比较弱化的，重要的是双12。

1.预估双12的量，但是这个量也没法去准确的评估，大概只能根据以往的量跟活动力度来预估一个大概的量，活动的话比如有没有秒杀活动，还是说是组团活动，如果有秒杀那么并发量可能会更高。

还有根据往年的双12的流量情况来预估一个大概值，流量大概是在平时流量的5到10倍。因为活动也不只有双12当天，而是一个周期，所以流量也会分散。

2.全链路压测由于是微服务，并且用到了比较多的组件，比如MQ、redis、mysql等等，所以整个链路会比较长，我们压测不能局限在单个接口，需要进行一个全链路压测。

3.找到链路瓶颈、哪个服务、哪个接口、还是说哪个组件服务拖慢了性能,比如是mysql还是redis，找到了链路瓶颈，我们就会去解决，先会去优化相关代码，比如代码是否有低级错误，有不应该的重复请求等等。查询语句是否有走到索引。有哪些数据可以做预热等等。

4.如果代码或者用法没有问题，那么就是到了机器的瓶颈，就会对应的去加机器，对于热点接口以及服务进行扩容。

5.降级对于复杂的但是不是主流的场景，进行静态降级或者暂时不可用

6.像实时监控与报警、与限流这些常见的高可用策略我们平时就会有。

面试官

刚才双12是知道流量暴增的点，那么突发的情况怎么解决?

灰灰：

首先，我们会有限流，会控制请求流量，导致系统被过多请求压垮。但是可能会

导致很多请求是进不来的。

如果要做到请求尽可能的能处理，我们之前没有对应的措施，但是我觉得可以

使用一些监控工具来监控流量的暴增，

可以让这个监控工具自动调用容器编排系统的钩子函数来进行动态的扩容。比如

说我有一个Docker服务集群，里面有100个服务，能够同时承载10万的并发量，

此时，如果流量监控工具监控到有20万的流量打过来，那么，这个流量监控工

具就会自动去调用Docker服务集群的一个接口，这个接口会帮助我们把100个

服务动态扩容到200个服务。

但是具体的，我之前没有接触过，只是一个思路。

22. 不同时机面试系列Mysql事务

面试官：请你说下你对InnoDB事务隔离级别的了解，你在工作中关注隔离级别么?

好的面试官

聊到事务隔离级别，我们得先聊下事务

所谓事务，就是工作的原子单元，可以去进行提交、回滚。在数据库中的工作，就是执行的sql语句，这些语句可以是单条语句也可以是多条语句。所以，其实就是去保证单条语句或者多条sql语句作为一个原子操作，要么全部成功要么全部回滚。

那么这也是我们数据库ACID设计原则里面的A,所谓ACID，是我们去设置数据库的时候，尽可能去保证数据的一致性与可靠性的一些设计原则。就像我们写代码的时候会去考虑设计原则，比如依赖倒置原则。

A Atomicity ætəˈmɪsəti 原子性 就是通过事务的提交回滚来实现的

C Consistency kənˈsɪstənsi 一致性 保证数据进来一致的

I Isolation aɪsəˈleɪʃn 每个更改数据的会话希望是相互不影响的

D Durability dʊrəˈbɪləti 持久性 保证数据不会丢失

那么其中的I 就是事务的隔离级别。

为什么会有这些隔离级别呢?

学Java架构·来咕泡教育 50000+ 技术人 都在咕泡教育拿到职场结果

因为事务是可以并行执行的，那么如果并行执行，就会导致一定的数据一致性问题。比如我们经常讲的脏读、不可重复读、幻读。

那么如果业务不能容忍这些数据一致性问题，作者就得去解决它，但是如果解决了一致性问题，性能就会有很大的影响。所以，作者干脆将这个难题交给了用户自己选择，这样不同的隔离级别就出来了。

分别是

RU

RC

RR

串行化

这4个隔离级别，越往上，性能越差，但是数据一致性越好。

RU读到未提交的数据，所以会有脏读、不可重复读、幻读问题

RC 读已提交的，但是没有解决不可重复读跟幻读问题

RR 在innoDB里面通过MVCC以及LBCC来解决了不可重复读跟幻读问题。也是innodb默认的隔离级别

至于串行化，就是不让事务有并行执行的可能，性能最低，一般不会选择

那么在工作中，也可以基于性能与数据一致性，来灵活的选择隔离级别，用得比较多的RC。

面试官：你刚才讲RR通过MVCC以及LBCC来解决幻读问题，但是RC也有啊，为什么RC没有解决不可重复读跟幻读。

【灰灰老师回答问题】

这块其实是因为不同的隔离级别在MVCC于LBCC的实现逻辑有点不一样。

首先，我们来讲下MVCC的流程，MVCC其实就是查询的时候，会去生成一个readView的class结构，里面会保存一些事务ID数据，比如当前存活的事务ID有哪些，然后跟数据的事务ID进行比对，来决定数据是否展示。

RR 只有在第一次查询的时候会生成一个readView，后面的查询比对的都是第一个查询的时候的结果，所以后面查不到新的值

RC是每次查询都会生成一个新的readView，这样每次查询我就能拿到最新的数据了，也就产生了不可重复读跟幻读问题

而LBCC是通过锁的形式去解决不可重复读跟幻读问题，就是我在改数据同一条或者同一批数据的时候，其他事务不能进行改动。但是RR相比于RC多了一个间隙锁，所谓间隙锁，就是会根据你锁的条件会锁定一个区间，所以没有幻读，但是RC隔离级别下，间隙锁只会用在重复键的检查。

这是我对这个问题的理解。

23. 请你说下线程池的参数，具体什么含义是什么?

面试官：请你说下线程池的参数，具体什么含义是什么?

你好，面试官

线程池其实是线程从池化技术，主要思路是在固定的线程数去执行不定的task任务，并且提供一些线程管理相关的钩子方法。

怎么做到线程复用的，主要就是开启线程后，如果我还有任务，线程就会自旋去阻塞队列获取任务，当队列没有任务的时候，线程通过阻塞队列进行阻塞，这样能保证线程是不会回收的，如果需要回收，那么可以通过阻塞队列的poll方法去获取任务，在阻塞一定时间后回收，如果不需要回收，则通过take方法一直阻塞获取。

这样我就会有固定的线程一直在自旋获取任务，就做到了线程复用。

面试官：那线程池一般开启几个线程?

【灰灰老师回答】这个线程的开启数量，一般要看具体的业务场景。

1.考虑你任务的数量，如果任务并发过多，导致阻塞队列放不下，那么需要较大的线程来提升消费的速度

2.要看你任务消耗的资源是io多还是cpu多，如果io多可以稍微多点线程，cpu偏多的话可以相对来说少点，比如就cpu核数。

3.机器的配置也会影响具体的线程数的配置

这个如果要达到一个理想值，要通过压测才能得到一个适当的值，不能天马行空去猜。过多会导致cpu浪费，一直在空旋，过少就没法达到比较好的性能要求。

24. 线程结束的方式有哪些?

面试官：线程结束的方式有哪些?

1.线程执行完毕后就会自动回收

2.我们也可以采用stop、suspend去强制关闭，但是不建议实现，现在Thread类中这个方法也是过时的，因为强制关闭，你没法跟踪到线程里面执行到哪里 导致业务异常或者数据一致性问题。

3.如果不强制关闭，那么可以用interrupt方法来优雅的关闭，所谓优雅的关闭，其实就是告诉线程一个标记，线程里面可以通过这个标记来判断，外面是否有人要我关闭，如果需要我关闭，那么线程自己来决定要不要执行完run方法。所以是否结束线程还是取决于线程自己。

这也是为什么我们的sleep等方法需要抛出InterruptedException的原因，因为线程在阻塞的时候，如果外面有人告诉你需要中断，你得进行InterruptedException响应。

还有一个 异常结束，当线程中发生了未捕获的异常，线程就会异常结束，可以通过Thread.UncaughtExceptionHandler处理未捕获的异常。

25. Mysql里面的RedoLog和BinLog，他们的作用在哪?

好的 面试官，RedoLog是innoDB里面的事务日志，既然是事务日志，所以它的最终目的是保证我的数据一致性，也是InnoDB满足ACID模型的一块很重要的支持。

binlog不属于存储引擎级别，不管是什么存储引擎都会有的，是Mysql服务自己去实现的。

binlog其实就是一些二进制日志记录文件，主要目的是数据灾备以及主从节点之间的数据同步。

面试官：那innoDb为什么不直接每次数据操作的时候，都保证同步到磁盘，而要加一个RedoLog?

【灰灰老师回答】那为什么要设计一个redoLog，是因为InnoDB存储引擎为了提高数据交互性能，加了一个bufferpool内存缓存，如果我数据每次操作都同步给磁盘，就会很慢。

所以，InnoDB采用的是异步的刷盘机制，但是异步刷盘又会导致数据丢失。因此，InnoDB设计了一个RedoLog的重做日志，每次操作实时同步到RedoLog的file文件，当然，默认也会实时刷新到磁盘，但是解决了数据直接同步到磁盘的2个性能问题：

第一个数据交互的最小单位是page，单次交互的量比较大，第二个是数据属于随机IO，而redolog属于顺序IO。

面试官：你刚才说binlog是用来同步主从数据的，那假如我RedoLog写成功了，但是binlog没有成功，那是不是会出现主节点有数据，但是没有同步到binlog?

【灰灰老师回答】好的，redoLog如果成功了，binLog没有，那么就会出现主有，从没有，如果先写binlog再写redolog，也会出现从有，但是主可能没有。

因而，这2个日志是尽可能的去保证都要写成功的。

所以，在Mysql里面，他们实现的机制是2pc 二阶段提交，redoLog先是准备状态，然后再去写binLog，只有binLog写成功了，redolog才会真正的刷新到磁盘。只有当redoLog 跟binLog都有，才能保证数据一致性。否则，数据回滚。

26. 一条mysql查询语句很慢，怎么去排查优化

面试官：你们公司用的mysql，假如，一条mysql查询语句很慢。你会怎么去排查优化?

面试者：好的。面试官。

1.首先，我会去检查是否是语句查询问题，比如是否是连接数的问题，因为在高并发的场景，这个问题也会经常出现的，可以通过show status来查看相关连接，里面包含现在的连接数等信息

2.如果不是网络跟连接问题，那么就需要去优化sql语句。

优化sql语句，最主要的是要用explain进行分析，explain里面有几个关键字段，一个type 连接类型，会有all、index 、range等等，一般最好能达到index，如果是all那么就是全表扫描。

还有就是key里面用到的字段。还有rows,代表估计检查扫描的行，数量越大，性能越慢，

extra ˈekstrə 附加信息，order by排序是否走到索引等等，可以在附加信息查看。

通过explain能找出一些比较明显的问题，比如是否用到索引，where条件、orderby、group by是否用到了索引，如果没有用到，就要去看下索引是否满足最左匹配，或者用到了运算或者范围查询等等使下一个索引列失效。

就需要去变动相关的索引列的顺序或者更改查询语句。

当然，还有一些小表驱动大表，这个就是存储引擎与mysql server层的一些交互优化，尽量减少server跟存储引擎的一些交互。

当不能快速找到问题的时候，就具体情况去具体分析，比如一些复杂的sql语句，那么可以一个一个的去优化，把每个语句、或者每个查询条件单独拆开去分析，找到哪个语句或者哪个查询条件比较慢，然后就能找到慢的问题了。

27. MySql count性能对比

【面试官】

Mysql在获取表的总数的时候，你知道count(*)、count(1)、count(字段)的区别么?

求职者:

好的面试官

count，其实是一个聚合函数，聚合数据的总数。

不同的存储引擎实现的方式可能又稍许不一样，我们以默认的innoDB存储引擎为例。

其实就是根据where条件去进行扫描，得到扫描结果后，一行一行去判断，如果count括号里的不是null，那么累计值+1，否则不加，最后返回一个累计的总数。

所以，括号里面的值在一定程度是会影响性能的。

count(*) ，也就是整条数据，由于整条数据肯定不会为null，所以，专门做了优化，不会去取全部字段。直接扫描多少数据统计数据即可。

count(1) 扫描到一条数据，直接返回一个数字1，不会取扫描的字段，也不会去判断是否为null。

这2个也是我们用得最多的，性能最好的。

但是如果你count(字段)，有2种情况。

1.首先去看字段是否能走索引，如果没索引，那么扫描数据不会走索引树，比有索引的字段慢。

2.字段是否可为null，如果可为null，那么必须要去判断扫描出来的数据是否为null，为null不统计，所以不可为null的比可为null的要快。

这是我对count聚合函数的理解。

28. 假如我Myql的表很大，进行分页的时候，limit 1000000 加载很慢的话，你是怎么解决的呢?

【面试官】

假如我Myql的表很大，进行分页的时候，limit 1000000 加载很慢的话，你是怎么解决的呢?

求职者：好的，面试官，这个问题在生产环境也是比较常见的。

limit m，n ;其实去扫描m+n条数据，然后过滤掉前面的m条数据，当m越大，那么需要扫描的数据也就越多，性能也会越来越慢。

针对这种情况，有以下几种方案可以进行一定的优化。

1.如果id是趋势递增的，那么每次查询都可以返回这次查询最大的ID，然后下次查询，加上大于 上次最大id的条件，这样会通过主键索引去扫描，并且扫描数量会少很多很多。因为只需要扫描where条件的数据

2.先limit出来主键ID，然后用主表跟查询出来的ID进行inner join 内连接，这样，也能一定上提速，因为减少了回表，查询ID只需要走聚集索引就行。

3.当然，如果mysql级别优化不了了。我们也可以对分页数据进行缓存，比如Redis缓存，数据进行变动的时候，做好缓存依赖即可。

4.因为越往后，一般用户行为触及不到，比如你去看淘宝，不会去翻后面几百页的数据，所以，业务层面也可以做一些让步，比如不做后面几百页的数据。

29. 谈谈你对分布式ID的理解

求职者:

我研究生毕业后，一直在杭州某互联网大厂担任Java开发。来贵司应聘高级开发，我的期

待薪资年包50W

求职者:

没问题，您可以随便考察

求职者:

首先，分布式ID，是因为随着业务的发展，数据或者ID的生成会在不同的地方，比如常见

的数据的分库分表，在分库分表的情况下，假如不同的表生成的ID不能保证唯一性，那么

就会导致一个ID不能保证一条数据的唯一性，就会带来问题。

所以分布式ID的主要目的：不论何时何地，生成的ID都是唯一的。

实现的方式有很多:

1.Mysql的全局ID生成表

2.UUID（有些版本会重复）

3.Redis的incr自增ID

4.zk的有序节点序号

5.mongoDB的objectId、

等等这些方案都能解决唯一性的问题。

但是除了唯一性以外，我们还需要考虑更多的其他因素：

1.比如是否趋势递增，否则在Mysql会导致主键索引树的分裂与合并。

2.安全性，通过ID是否会导致数据泄露

3.高可用以及高性能，满足业务的需求，不能生成一个ID很慢或者动不动不可用。

所以，为了性能与可靠性以及成本，市面上还有一种雪花算法是用得比较多，他是一种基于位运算来实现的，并且实现简单，没有太多外部依赖。

很多大公司也有基于雪花写了自己的ID生成方案，比如美。